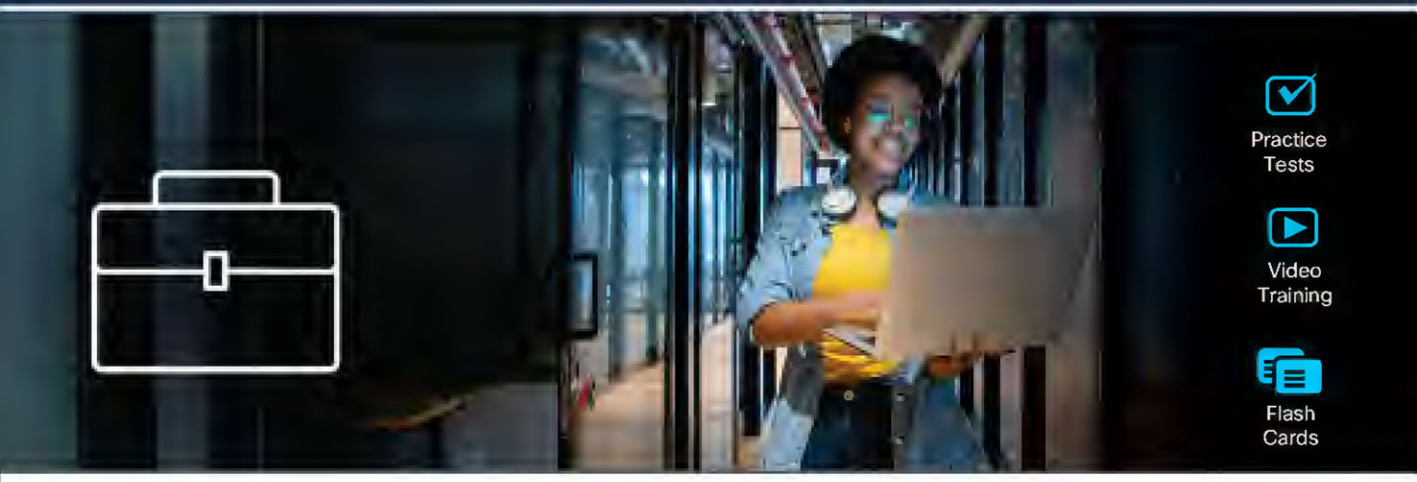
Official Cert Guide Library




Study Planner
Review Exercises
Labs

# CCNA

## 200-301

2nd Edition

**WENDELL ODOM,** CCIE® No. 1624
**JASON GOOLEY,** CCIE® No. 38759
**DAVID HUCABY,** CCIE® No. 4594

ciscopress.com




# Official Cert Guide

Advance your IT career with hands-on learning

Hands-On Labs

Review Exercises

Config Checklists

# CCNA 200-301

## Volume 1

**WENDELL ODOM**,
CCIE® NO. 1624 EMERITUS

ciscopress.com

# *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition

## Companion Website and Pearson Test Prep Access Code

Access interactive study tools on this book's companion website, including practice test software, review exercises, Key Term flash card application, a study planner, and more!

To access the companion website, simply follow these steps:

1. Go to www.ciscopress.com/register.
2. Enter the **print book ISBN:** 9780138229634.
3. Answer the security question to validate your purchase.
4. Go to your account page.
5. Click on the **Registered Products** tab.
6. Under the book listing, click on the **Access Bonus Content** link.

When you register your book, your Pearson Test Prep practice test access code will automatically be populated with the book listing under the Registered Products tab. You will need this code to access the practice test that comes with this book. You can redeem the code at **PearsonTestPrep.com**. Simply choose Pearson IT Certification as your product group and log into the site with the same credentials you used to register your book. Click the **Activate New Product** button and enter the access code. More detailed instructions on how to redeem your access code for both the online and desktop versions can be found on the companion website.

If you have any issues accessing the companion website or obtaining your Pearson Test Prep practice test access code, you can contact our support team by going to **ciscopress.com/support**.

*This page intentionally left blank*

# CCNA 200-301

## **Official** Cert Guide Volume 1

## Second Edition

**WENDELL ODOM,** CCIE No. 1624

**Cisco Press**

# CCNA 200-301 Official Cert Guide, Volume 1, Second Edition

Wendell Odom

Copyright© 2024 Pearson Education, Inc.

Published by:
Cisco Press
Hoboken, New Jersey

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system, without written permission from the publisher, except for the inclusion of brief quotations in a review.

$PrintCode

Library of Congress Control Number: 2024934291

ISBN-13: 978-0-13-822963-4

ISBN-10: 0-13-822963-5

## Warning and Disclaimer

This book is designed to provide information about the Cisco CCNA 200-301 exam. Every effort has been made to make this book as complete and as accurate as possible, but no warranty or fitness is implied.

The information is provided on an "as is" basis. The author, Cisco Press, and Cisco Systems, Inc. shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained in this book or from the use of the discs or programs that may accompany it.

The opinions expressed in this book belong to the author and are not necessarily those of Cisco Systems, Inc.

## Trademark Acknowledgments

All terms mentioned in this book that are known to be trademarks or service marks have been appropriately capitalized. Cisco Press or Cisco Systems, Inc., cannot attest to the accuracy of this information. Use of a term in this book should not be regarded as affecting the validity of any trademark or service mark.

# Figure Credits

Figure 2.7 a: Anton Samsonov/123RF

Figure 2.7 b: indigolotos/123RF

Figure 19.10: Microsoft Corporation

Figures 19.11, 28.12: Apple Inc

Figure 19.12: Linux Foundation

## Special Sales

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

## Feedback Information

At Cisco Press, our goal is to create in-depth technical books of the highest quality and value. Each book is crafted with care and precision, undergoing rigorous development that involves the unique expertise of members from the professional technical community.

Readers' feedback is a natural continuation of this process. If you have any comments regarding how we could improve the quality of this book, or otherwise alter it to better suit your needs, you can contact us through email at feedback@ciscopress.com. Please make sure to include the book title and ISBN in your message.

We greatly appreciate your assistance.

Please contact us with concerns about any potential bias at https://www.pearson.com/report-bias.html.

**GM K12, Early Career and Professional Learning:** Soo Kang

**Alliances Manager, Cisco Press:** Caroline Antonio

**Director, ITP Product Management:** Brett Bartow

**Managing Editor:** Sandra Schroeder

**Development Editor:** Christopher Cleveland

**Senior Project Editor:** Tonya Simpson

**Copy Editor:** Chuck Hutchinson

**Technical Editor:** Denise Donohue

**Editorial Assistant:** Cindy Teeters

**Cover Designer:** Chuti Prasertsith

**Composition:** codeMantra

**Indexer:** Ken Johnson

**Proofreader:** Donna E. Mulder

Americas Headquarters
Cisco Systems, Inc.
San Jose, CA

Asia Pacific Headquarters
Cisco Systems (USA) Pte. Ltd.
Singapore

Europe Headquarters
Cisco Systems International BV
Amsterdam, The Netherlands

Cisco has more than 200 offices worldwide. Addresses, phone numbers, and fax numbers are listed on the Cisco Website at **www.cisco.com/go/offices.**

CCDE, CCENT, Cisco Eos, Cisco HealthPresence, the Cisco logo, Cisco Lumin, Cisco Nexus, Cisco StadiumVision, Cisco TelePresence, Cisco WebEx, DCE, and Welcome to the Human Network are trademarks; Changing the Way We Work, Live, Play, and Learn and Cisco Store are service marks; and Access Registrar, Aironet, AsyncOS, Bringing the Meeting To You, Catalyst, CCDA, CCDP, CCIE, CCIP, CCNA, CCNP, CCSP, CCVP, Cisco, the Cisco Certified Internetwork Expert logo, Cisco IOS, Cisco Press, Cisco Systems, Cisco Systems Capital, the Cisco Systems logo, Cisco Unity, Collaboration Without Limitation, EtherFast, EtherSwitch, Event Center, Fast Step, Follow Me Browsing, FormShare, GigaDrive, HomeLink, Internet Quotient, IOS, iPhone, iQuick Study, IronPort, the IronPort logo, LightStream, Linksys, MediaTone, MeetingPlace, MeetingPlace Chime Sound, MGX, Networkers, Networking Academy, Network Registrar, PCNow, PIX, PowerPanels, ProConnect, ScriptShare, SenderBase, SMARTnet, Spectrum Expert, StackWise, The Fastest Way to Increase Your Internet Quotient, TransPath, WebEx, and the WebEx logo are registered trademarks of Cisco Systems, Inc. and/or its affiliates in the United States and certain other countries.

All other trademarks mentioned in this document or website are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company. (0812R)

## About the Author

**Wendell Odom**, CCIE Enterprise No. 1624, was the first Cisco Press author for Cisco certification guides. He wrote all prior editions of this book, along with books on topics ranging from introductory networking to CCENT, CCNA R&S, CCNA DC, CCNP ROUTE, CCNP QoS, and CCIE R&S. In his four decades as a networker, he has worked as a network engineer, consultant, systems engineer, instructor, and course developer. He now spends his time focused on updating the CCNA books, his blog (www.certskills.com), building his new CCNA YouTube channel (www.youtube.com/@NetworkUpskill), and teaching online (www.certskills.com/courses). You can find him at www.LinkedIn.com/in/WendellOdom, Twitter (@WendellOdom), and at his blog, which provides a variety of free CCNA learning resources.

## About the Technical Reviewer

**Denise Donohue**, CCIE No. 9566 (Routing and Switching), has worked with information systems since the mid-1990s and network architecture since 2004. During that time, she has worked with a wide range of networks, private and public, of all sizes, across most industries. Her focus is on aligning business and technology. Denise has authored several Cisco Press books and frequently shares her knowledge in webinars and seminars, and at conferences.

# Dedications

For Fay York Odom (1938–2022), the best mom ever.

## Acknowledgments

Brett Bartow and I have been a team for a few decades. He has had more to do with the successes of the Cisco Press product line than anyone else. More than ever, his insights and wisdom have been a key to navigating Cisco's big changes to certifications back in 2020. Now with Cisco's 2023 pivot to a lean development model for certifications, with the possibility of new exam content annually, Brett's leadership matters more than ever. (See "Your Study Plan" for more about what that new lean development cycle means.) He's always a great partner in working through big-picture direction as well as features to make the books the best they can be for our readers. It is always appreciated, but not voiced every time—so thanks, Brett, for your consistent leadership and wisdom!

Chris Cleveland did the development editing for the very first Cisco Press exam certification guide way back in 1998, and he still can't seem to get away from us! Seriously, when Brett and I first discuss any new book, my first priority is to ask whether Chris has time to develop the book—and lobby if there are any barriers! It's always a pleasure working with you, Chris.

The technical editors also have a meaningful positive impact on the books. And we got Denise Donohue to do it! Denise and I teamed up to write the *CCIE R&S Official Cert Guide* for two editions, and she has written extensively herself—which is why I wondered if we could get her help. Her deep technical skills, along with her unique insights into the book authoring process, have been a great help to both weed out the mistakes and get good advice on how to improve the chapters.

Cisco's move to an annual exam update cadence (they at least consider updating each exam once per year) has more impact on the production side of our publishing process than it does on the authoring side. Knowing early that both Sandra and Tonya are back at it, finding ways to continue the high quality while being creative with the new publication cycle sets me more at ease. When writing, I could rest knowing that the second half of the process, which happens after I've finished 99 percent of my work, will be done well!

Thanks to all the production team for making the magic happen. I usually do not interact with you directly, beyond Sandra and Tonya, but I see your work, and the books truly improve through the process! From fixing all my grammar and passive-voice sentences to pulling the design and layout together, they do it all; thanks for putting it all together and making it look easy.

A special thank you to you readers who write in with suggestions and possible errors, and especially those of you who post online at the Cisco Learning Network and at my blog (www.certskills.com). More so than any edition I can remember, reader comments have had more to do with changes I made to improve existing content in these editions. The comments I received directly and those I overheard by participating at CLN made this edition a better book. (See the heading "Feedback Information" just a page or so back to see how to get in touch with us!)

My wonderful wife, Kris, and I reached our 25th anniversary while working on this edition. She makes this challenging work lifestyle a breeze, even happily scheduling our 25th-anniversary vacation around the book schedule! Thanks to my daughter Hannah for the perspectives on how 20-somethings think about learning and studying. And thanks to Jesus Christ, Lord of everything in my life.

# Contents at a Glance

Introduction xxxiv

Your Study Plan 2

**Part I Introduction to Networking 15**

Chapter 1 Introduction to TCP/IP Networking 16

Chapter 2 Fundamentals of Ethernet LANs 36

Chapter 3 Fundamentals of WANs and IP Routing 62

Part I Review 84

**Part II Implementing Ethernet LANs 87**

Chapter 4 Using the Command-Line Interface 88

Chapter 5 Analyzing Ethernet LAN Switching 112

Chapter 6 Configuring Basic Switch Management 132

Chapter 7 Configuring and Verifying Switch Interfaces 158

Part II Review 184

**Part III Implementing VLANs and STP 187**

Chapter 8 Implementing Ethernet Virtual LANs 188

Chapter 9 Spanning Tree Protocol Concepts 222

Chapter 10 RSTP and EtherChannel Configuration 256

Part III Review 296

**Part IV IPv4 Addressing 299**

Chapter 11 Perspectives on IPv4 Subnetting 300

Chapter 12 Analyzing Classful IPv4 Networks 324

Chapter 13 Analyzing Subnet Masks 338

Chapter 14 Analyzing Existing Subnets 356

Chapter 15 Subnet Design 378

Part IV Review 402

**Part V IPv4 Routing 405**

Chapter 16 Operating Cisco Routers 406

Chapter 17 Configuring IPv4 Addresses and Static Routes 426

Chapter 18 IP Routing in the LAN 454

Chapter 19 IP Addressing on Hosts 486

Chapter 20 Troubleshooting IPv4 Routing 512

Part V Review 534

**Part VI OSPF 537**

Chapter 21 Understanding OSPF Concepts 538

Chapter 22 Implementing Basic OSPF Features 562

Chapter 23 Implementing Optional OSPF Features 584

Chapter 24 OSPF Neighbors and Route Selection 608

Part VI Review 634

**Part VII IP Version 6 637**

Chapter 25 Fundamentals of IP Version 6 638

Chapter 26 IPv6 Addressing and Subnetting 654

Chapter 27 Implementing IPv6 Addressing on Routers 668

Chapter 28 Implementing IPv6 Addressing on Hosts 696

Chapter 29 Implementing IPv6 Routing 722

Part VII Review 746

**Part VIII Exam Updates 749**

Chapter 30 *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition Exam Updates 750

**Part IX Appendixes 757**

Appendix A Numeric Reference Tables 759

Appendix B Exam Topics Cross-Reference 765

Appendix C Answers to the "Do I Know This Already?" Quizzes 779

Glossary 809

Index 840

### Online Appendixes

Appendix D Practice for Chapter 12: Analyzing Classful IPv4 Networks

Appendix E Practice for Chapter 13: Analyzing Subnet Masks

Appendix F Practice for Chapter 14: Analyzing Existing Subnets

Appendix G Practice for Chapter 15: Subnet Design

Appendix H Practice for Chapter 25: Fundamentals of IP Version 6

Appendix I Practice for Chapter 27: Implementing IPv6 Addressing on Routers

Appendix J Study Planner

Appendix K Topics from Previous Editions

Appendix L LAN Troubleshooting

Appendix M Variable-Length Subnet Masks

## Reader Services

To access additional content for this book, simply register your product. To start the registration process, go to www.ciscopress.com/register and log in or create an account.* Enter the product ISBN 9780138229634 and click **Submit**. After the process is complete, you will find any available bonus content under Registered Products.

*Be sure to check the box that you would like to hear from us to receive exclusive discounts on future editions of this product.

# Contents

**Introduction xxxiv**

**Your Study Plan 2**

A Brief Perspective on Cisco Certification Exams 2

Five Study Plan Steps 3

Step 1: Think in Terms of Parts and Chapters 3

Step 2: Build Your Study Habits Around the Chapter 4

Step 3: Use Book Parts for Major Milestones 5

Step 4: Use Volume 2's Final Review Chapter 6

Step 5: Set Goals and Track Your Progress 6

Things to Do Before Starting the First Chapter 7

Bookmark the Companion Website 7

Bookmark/Install Pearson Test Prep 7

Understand This Book's PTP Databases and Modes 7

*Exams in the Retail (Print) Editions 8*

Exams with Individual Premium Edition eBooks 10

*Exams with Two Individual Premium Edition eBooks 10*

Exams with CCNA Premium Edition Library 11

Practice Viewing Per-Chapter Book (DIKTA) Questions 12

Practice by Using Per-Part Review Questions 12

Join the Cisco Learning Network CCNA Community 12

Getting Started: Now 12

**Part I Introduction to Networking 15**

**Chapter 1 Introduction to TCP/IP Networking 16**

"Do I Know This Already?" Quiz 17

Foundation Topics 18

Perspectives on Networking 18

TCP/IP Networking Model 19

History Leading to TCP/IP 20

Overview of the TCP/IP Networking Model 21

TCP/IP Application Layer 23

*HTTP Overview 23*

*HTTP Protocol Mechanisms 23*

TCP/IP Transport Layer 24

*TCP Error Recovery Basics 25*

*Same-Layer and Adjacent-Layer Interactions 25*

TCP/IP Network Layer 26
*Internet Protocol and the Postal Service 26*
*Internet Protocol Addressing Basics 27*
*IP Routing Basics 28*
TCP/IP Data-Link and Physical Layers 29
Data Encapsulation Terminology 31
Names of TCP/IP Messages 31
OSI Networking Model and Terminology 32
*Comparing OSI and TCP/IP Layer Names and Numbers 33*
Chapter Review 34

**Chapter 2 Fundamentals of Ethernet LANs 36**
"Do I Know This Already?" Quiz 36
Foundation Topics 38
An Overview of LANs 38
Typical SOHO LANs 39
Typical Enterprise LANs 40
The Variety of Ethernet Physical Layer Standards 41
Consistent Behavior over All Links Using the Ethernet Data-Link Layer 42
Building Physical Ethernet LANs with UTP 43
Transmitting Data Using Twisted Pairs 43
Breaking Down a UTP Ethernet Link 44
UTP Cabling Pinouts for 10BASE-T and 100BASE-T 46
*Straight-Through Cable Pinout 46*
*Choosing the Right Cable Pinouts 48*
*Automatic Rewiring with Auto-MDIX 49*
UTP Cabling Pinouts for 1000BASE-T 49
Building Physical Ethernet LANs with Fiber 50
Fiber Cabling Transmission Concepts 50
Using Fiber with Ethernet 52
Sending Data in Ethernet Networks 53
Ethernet Data-Link Protocols 53
*Ethernet Addressing 54*
*Identifying Network Layer Protocols with the Ethernet Type Field 56*
*Error Detection with FCS 56*
Sending Ethernet Frames with Switches and Hubs 57
*Sending in Modern Ethernet LANs Using Full Duplex 57*
*Using Half Duplex with LAN Hubs 58*
Chapter Review 60

**Chapter 3 Fundamentals of WANs and IP Routing 62**

"Do I Know This Already?" Quiz 62

Foundation Topics 64

Wide-Area Networks 64

Leased-Line WANs 64

*Physical Details of Leased Lines 65*

*Data-Link Details of Leased Lines 66*

*How Routers Use a WAN Data Link 67*

Ethernet as a WAN Technology 69

*Ethernet WANs That Create a Layer 2 Service 70*

*How Routers Route IP Packets Using Ethernet WAN Links 70*

IP Routing 71

Network Layer Routing (Forwarding) Logic 72

*Host Forwarding Logic: Send the Packet to the Default Router 73*

*R1 and R2's Logic: Routing Data Across the Network 73*

*R3's Logic: Delivering Data to the End Destination 73*

How Network Layer Routing Uses LANs and WANs 73

How IP Addressing Helps IP Routing 76

*Rules for Groups of IP Addresses (Networks and Subnets) 76*

*The IP Header 77*

How IP Routing Protocols Help IP Routing 77

Other Network Layer Features 79

Using Names and the Domain Name System 79

The Address Resolution Protocol 80

ICMP Echo and the ping Command 81

Chapter Review 82

**Part I Review 84**

**Part II Implementing Ethernet LANs 87**

**Chapter 4 Using the Command-Line Interface 88**

"Do I Know This Already?" Quiz 88

Foundation Topics 90

Accessing the Cisco Catalyst Switch CLI 90

Cisco Catalyst Switches 90

Accessing the Cisco IOS XE CLI 91

*The Operating System in Cisco Catalyst Switches 92*

*Accessing the IOS XE CLI 92*

*Cabling the Console Connection 93*

*Configuring a Terminal Emulator 95*
*Accessing the CLI with Telnet and SSH 96*
*User and Enable (Privileged) Modes 96*
*Password Security for CLI Access from the Console 98*
*Accessing the CLI with the WebUI 99*
CLI Help Features 101
The debug and show Commands 103
Configuring Cisco IOS Software 103
Configuration Submodes and Contexts 104
Storing Switch Configuration Files 106
Copying and Erasing Configuration Files 109
Chapter Review 109

**Chapter 5 Analyzing Ethernet LAN Switching 112**
"Do I Know This Already?" Quiz 112
Foundation Topics 114
LAN Switching Concepts 114
Overview of Switching Logic 115
Forwarding Known Unicast Frames 116
Learning MAC Addresses 118
Flooding Unknown Unicast and Broadcast Frames 119
Avoiding Loops Using Spanning Tree Protocol 120
LAN Switching Summary 121
Verifying and Analyzing Ethernet Switching 121
Demonstrating MAC Learning 122
Switch Interfaces 123
Finding Entries in the MAC Address Table 124
Managing the MAC Address Table (Aging, Clearing) 126
MAC Address Tables with Multiple Switches 127
Chapter Review 128

**Chapter 6 Configuring Basic Switch Management 132**
"Do I Know This Already?" Quiz 132
Foundation Topics 134
Securing the Switch CLI 134
Securing User Mode and Privileged Mode with Simple Passwords 135
Securing User Mode Access with Local Usernames and Passwords 139

Securing User Mode Access with External Authentication Servers 141
Securing Remote Access with Secure Shell 142
Enabling and Securing the WebUI 145
Enabling IPv4 for Remote Access 146
Host and Switch IP Settings 147
Configuring IPv4 on a Switch 149
Configuring a Switch to Learn Its IP Address with DHCP 150
Verifying IPv4 on a Switch 150
Miscellaneous Settings Useful in the Lab 151
History Buffer Commands 151
The logging synchronous, exec-timeout, and no ip domain-lookup Commands 152
Chapter Review 153

**Chapter 7 Configuring and Verifying Switch Interfaces 158**

"Do I Know This Already?" Quiz 158
Foundation Topics 161
Configuring Switch Interface Speed and Duplex 161
IEEE Autonegotiation Concepts 161
*Autonegotiation Under Working Conditions 161*
*Autonegotiation Results When Only One Node Uses Autonegotiation 163*
*Autonegotiation and LAN Hubs 165*
Configuring Autonegotiation, Speed, and Duplex 165
*Using Autonegotiation on Cisco Switches 165*
*Setting Speed and Duplex Manually 169*
Using Auto-MDIX on Cisco Switches 170
Managing Switch Interface Configuration 172
The Description and Interface Range Commands 172
Administratively Controlling Interface State with shutdown 173
Removing Configuration with the no Command 174
Analyzing Switch Interface Status and Statistics 176
Interface Status Codes 176
The Duplex Mismatch Issue 177
Common Layer 1 Problems on Working Interfaces 178
Chapter Review 180

**Part II Review 184**

**Part III Implementing VLANs and STP 187**

**Chapter 8 Implementing Ethernet Virtual LANs 188**

"Do I Know This Already?" Quiz 189

Foundation Topics 191

Virtual LAN Concepts 191

Creating Multiswitch VLANs Using Trunking 192

*VLAN Tagging Concepts 193*

*The 802.1Q and ISL VLAN Trunking Protocols 194*

Forwarding Data Between VLANs 195

*The Need for Routing Between VLANs 195*

*Routing Packets Between VLANs with a Router 196*

VLAN and VLAN Trunking Configuration and Verification 197

Creating VLANs and Assigning Access VLANs to an Interface 197

*VLAN Configuration Example 1: Full VLAN Configuration 198*

*VLAN Configuration Example 2: Shorter VLAN Configuration 201*

VLAN Trunking Protocol 201

VLAN Trunking Configuration 203

Implementing Interfaces Connected to Phones 207

*Data and Voice VLAN Concepts 208*

*Data and Voice VLAN Configuration and Verification 209*

*Summary: IP Telephony Ports on Switches 212*

Troubleshooting VLANs and VLAN Trunks 212

Confirm the Correct Access VLAN Is Assigned 213

Access VLANs Undefined or Disabled 213

Mismatched Trunking Operational States 215

The Supported VLAN List on Trunks 217

Mismatched Native VLAN on a Trunk 218

Chapter Review 218

**Chapter 9 Spanning Tree Protocol Concepts 222**

"Do I Know This Already?" Quiz 223

Foundation Topics 224

STP and RSTP Basics 224

The Need for Spanning Tree 225

What Spanning Tree Does 227

How Spanning Tree Works 228

*The STP Bridge ID and Hello BPDU 229*

*Electing the Root Switch 230*

*Choosing Each Switch's Root Port 232*
*Choosing the Designated Port on Each LAN Segment 234*
Configuring to Influence the STP Topology 235
Details Specific to STP (and Not RSTP) 236
STP Activity When the Network Remains Stable 236
STP Timers That Manage STP Convergence 237
Changing Interface States with STP 238
Rapid STP Concepts 239
Comparing STP and RSTP 240
RSTP and the Alternate (Root) Port Role 241
RSTP States and Processes 243
RSTP and the Backup (Designated) Port Role 244
RSTP Port Types 245
Optional STP Features 246
EtherChannel 246
PortFast 247
BPDU Guard 248
BPDU Filter 248
*BPDU Filter to Prevent Loops on PortFast Ports 249*
*BPDU Filter to Disable STP on a Port 249*
Root Guard 250
Loop Guard 251
Chapter Review 254

**Chapter 10 RSTP and EtherChannel Configuration 256**

"Do I Know This Already?" Quiz 256
Foundation Topics 259
Understanding RSTP Through Configuration 259
The Need for Multiple Spanning Trees 260
STP Modes and Standards 260
The Bridge ID and System ID Extension 261
Identifying Switch Priority and the Root Switch 263
*Switch Priority and Identifying the Root Switch 263*
*Switch Priority Using Root Primary and Secondary 265*
RSTP (One Tree) and RPVST+ (One Tree Per VLAN) 266
Identifying Port Cost, Role, and State 266
Identifying Optional STP Features 269
PortFast and BPDU Guard 269

*PortFast and BPDU Guard on an Access Port with One Endpoint 269*
*PortFast on VLAN Trunks and Voice Pseudo-Trunks 271*
*Global Configuration of PortFast and BPDU Guard 273*
BPDU Filter 274
*Conditional BPDU Filtering with Global Configuration 275*
*Disabling STP with BPDU Filter Interface Configuration 277*
Root Guard 278
Loop Guard 279
Configuring Layer 2 EtherChannel 281
Configuring a Manual Layer 2 EtherChannel 281
Configuring Dynamic EtherChannels 284
Interface Configuration Consistency with EtherChannels 287
EtherChannel Load Distribution 289
Chapter Review 291

**Part III Review 296**

**Part IV IPv4 Addressing 299**

**Chapter 11 Perspectives on IPv4 Subnetting 300**
"Do I Know This Already?" Quiz 300
Foundation Topics 302
Introduction to Subnetting 302
Subnetting Defined Through a Simple Example 302
Operational View Versus Design View of Subnetting 303
Analyze Subnetting and Addressing Needs 304
Rules About Which Hosts Are in Which Subnet 304
Determining the Number of Subnets 305
Determining the Number of Hosts per Subnet 307
One Size Subnet Fits All—Or Not 307
*Defining the Size of a Subnet 308*
*One Size Subnet Fits All 308*
*Multiple Subnet Sizes (Variable-Length Subnet Masks) 309*
*One Mask for All Subnets, or More Than One 310*
Make Design Choices 311
Choose a Classful Network 311
*Public IP Networks 311*
*Growth Exhausts the Public IP Address Space 312*
*Private IP Networks 313*
*Choosing an IP Network During the Design Phase 314*

Choose the Mask 314
*Classful IP Networks Before Subnetting 315*
*Borrowing Host Bits to Create Subnet Bits 315*
*Choosing Enough Subnet and Host Bits 316*
*Example Design: 172.16.0.0, 200 Subnets, 200 Hosts 317*
*Masks and Mask Formats 318*
Build a List of All Subnets 318
Plan the Implementation 320
Assigning Subnets to Different Locations 320
Choose Static and Dynamic Ranges per Subnet 321
Chapter Review 322

**Chapter 12 Analyzing Classful IPv4 Networks 324**
"Do I Know This Already?" Quiz 324
Foundation Topics 325
Classful Network Concepts 325
Setting the Context of Public Networks and CIDR Blocks 326
IPv4 Network Classes and Related Facts 328
*The Number and Size of the Class A, B, and C Networks 329*
*Address Formats 330*
*Default Masks 330*
Number of Hosts per Network 331
Deriving the Network ID and Related Numbers 331
Unusual Network IDs and Network Broadcast Addresses 333
Practice with Classful Networks 334
Practice Deriving Key Facts Based on an IP Address 334
Practice Remembering the Details of Address Classes 335
Chapter Review 335

**Chapter 13 Analyzing Subnet Masks 338**
"Do I Know This Already?" Quiz 338
Foundation Topics 340
Subnet Mask Conversion 340
Three Mask Formats 340
Converting Between Binary and Prefix Masks 341
Converting Between Binary and DDN Masks 342
Converting Between Prefix and DDN Masks 344
Practice Converting Subnet Masks 344

Identifying Subnet Design Choices Using Masks 345
Masks Divide the Subnet's Addresses into Two Parts 346
Masks and Class Divide Addresses into Three Parts 347
Classless and Classful Addressing 348
Calculations Based on the IPv4 Address Format 349
Practice Analyzing Subnet Masks 350
Masks and CIDR Blocks 351
Chapter Review 352

**Chapter 14 Analyzing Existing Subnets 356**
"Do I Know This Already?" Quiz 356
Foundation Topics 358
Defining a Subnet 358
An Example with Network 172.16.0.0 and Four Subnets 358
Subnet ID Concepts 360
Subnet Broadcast Address 361
Range of Usable Addresses 361
Analyzing Existing Subnets: Binary 362
Finding the Subnet ID: Binary 362
Finding the Subnet Broadcast Address: Binary 363
Binary Practice Problems 364
Shortcut for the Binary Process 365
A Brief Note About Boolean Math 367
Finding the Range of Addresses 367
Analyzing Existing Subnets: Decimal 367
Analysis with Easy Masks 367
Predictability in the Interesting Octet 368
Finding the Subnet ID: Difficult Masks 369
*Resident Subnet Example 1 370*
*Resident Subnet Example 2 371*
*Resident Subnet Practice Problems 371*
Finding the Subnet Broadcast Address: Difficult Masks 372
*Subnet Broadcast Example 1 372*
*Subnet Broadcast Example 2 372*
*Subnet Broadcast Address Practice Problems 373*
Practice Analyzing Existing Subnets 373
A Choice: Memorize or Calculate 373
Chapter Review 374

**Chapter 15 Subnet Design 378**

"Do I Know This Already?" Quiz 378

Foundation Topics 380

Choosing the Mask(s) to Meet Requirements 380

Review: Choosing the Minimum Number of Subnet and Host Bits 380

No Masks Meet Requirements 381

One Mask Meets Requirements 382

Multiple Masks Meet Requirements 383

*Finding All the Masks: Concepts 383*

*Finding All the Masks: Math 384*

*Choosing the Best Mask 385*

The Formal Process 385

Practice Choosing Subnet Masks 386

*Practice Problems for Choosing a Subnet Mask 386*

Finding All Subnet IDs 386

First Subnet ID: The Zero Subnet 387

Finding the Pattern Using the Magic Number 388

A Formal Process with Fewer Than 8 Subnet Bits 389

*Example 1: Network 172.16.0.0, Mask 255.255.240.0 390*

*Example 2: Network 192.168.1.0, Mask 255.255.255.224 391*

Finding All Subnets with Exactly 8 Subnet Bits 393

Finding All Subnets with More Than 8 Subnet Bits 393

*Process with 9–16 Subnet Bits 393*

*Process with 17 or More Subnet Bits 395*

Practice Finding All Subnet IDs 396

*Practice Problems for Finding All Subnet IDs 396*

Chapter Review 396

**Part IV Review 402**

**Part V IPv4 Routing 405**

**Chapter 16 Operating Cisco Routers 406**

"Do I Know This Already?" Quiz 406

Foundation Topics 407

Installing Cisco Routers 407

Installing Enterprise Routers 408

*The Cisco Router Operating Systems 409*

*Cisco Integrated Services Routers 410*

*The Cisco Catalyst Edge Platform 411*
*Physical Installation 412*
Installing SOHO Routers 412
Enabling IPv4 Support on Cisco Router Interfaces 413
Accessing the Router CLI 414
Router Interfaces 415
*Interface Status Codes 417*
*Router Interface IP Addresses 418*
*Ethernet Interface Autonegotiation 420*
*Bandwidth and Clock Rate on Serial Interfaces 423*
Router Auxiliary Port 423
Chapter Review 423

**Chapter 17 Configuring IPv4 Addresses and Static Routes 426**
"Do I Know This Already?" Quiz 427
Foundation Topics 428
IP Routing 428
IPv4 Routing Process Reference 429
An Example of IP Routing 431
*Host Forwards the IP Packet to the Default Router (Gateway) 432*
*Routing Step 1: Decide Whether to Process the Incoming Frame 432*
*Routing Step 2: De-encapsulation of the IP Packet 433*
*Routing Step 3: Choosing Where to Forward the Packet 433*
*Routing Step 4: Encapsulating the Packet in a New Frame 434*
*Routing Step 5: Transmitting the Frame 435*
Configuring IP Addresses and Connected Routes 435
Connected Routes and the ip address Command 436
Common Mistakes with the ip address Subcommand 438
The ARP Table on a Cisco Router 439
Configuring Static Routes 440
Static Network Routes 441
*Verifying Static Network Routes 442*
*Ethernet Outgoing Interfaces and Proxy ARP 443*
Static Default Routes 443
Static Host Routes 445
Floating Static Routes 447

Troubleshooting Static Routes 448
*Incorrect Static Routes That Appear in the IP Routing Table 448*
*The Static Route Does Not Appear in the IP Routing Table 449*
*The Correct Static Route Appears but Works Poorly 450*
Chapter Review 450

**Chapter 18 IP Routing in the LAN 454**
"Do I Know This Already?" Quiz 455
Foundation Topics 457
VLAN Routing with Router 802.1Q Trunks 457
Configuring ROAS 458
Verifying ROAS 461
Troubleshooting ROAS 463
VLAN Routing with Layer 3 Switch SVIs 464
Configuring Routing Using Switch SVIs 464
Verifying Routing with SVIs 466
Troubleshooting Routing with SVIs 467
*SVI Interface State with Autostate Enabled 467*
*SVI Interface State with Autostate Disabled 469*
VLAN Routing with Layer 3 Switch Routed Ports 469
Implementing Routed Interfaces on Switches 470
Implementing Layer 3 EtherChannels 473
Troubleshooting Layer 3 EtherChannels 476
VLAN Routing on a Router's LAN Switch Ports 477
Configuring Routing for Embedded Switch Ports 478
Verifying Routing for Embedded Switch Ports 480
Identifying Switched Ports in Routers 481
Chapter Review 482

**Chapter 19 IP Addressing on Hosts 486**
"Do I Know This Already?" Quiz 486
Foundation Topics 488
Dynamic Host Configuration Protocol 488
DHCP Concepts 488
*APIPA IP Addresses (169.254.x.x) 490*
*Supporting DHCP for Remote Subnets with DHCP Relay 490*
*Information Stored at the DHCP Server 492*

Configuring DHCP Features on Routers and Switches 493
*Configuring DHCP Relay 494*
*Configuring a Switch as DHCP Client 495*
*Configuring a Router as DHCP Client 496*
Identifying Host IPv4 Settings 497
Host Settings for IPv4 497
Host IP Settings on Windows 499
Host IP Settings on macOS 502
Host IP Settings on Linux 504
Troubleshooting Host IP Settings 506
*A Working Windows Host with Static IP Configuration 506*
*A Failed Windows DHCP Client Due to IP Connectivity Issues 507*
*A Working Windows DHCP Client with Incorrect Settings 508*
Chapter Review 510

**Chapter 20 Troubleshooting IPv4 Routing 512**
"Do I Know This Already?" Quiz 512
Foundation Topics 513
Problem Isolation Using the ping Command 513
Ping Command Basics 513
Strategies and Results When Testing with the ping Command 514
*Testing Longer Routes from Near the Source of the Problem 514*
*Using Extended Ping to Test the Reverse Route 517*
*Testing LAN Neighbors with Standard Ping 519*
*Testing LAN Neighbors with Extended Ping 520*
*Testing WAN Neighbors with Standard Ping 521*
Using Ping with Names and with IP Addresses 522
Problem Isolation Using the traceroute Command 524
traceroute Basics 524
*How the traceroute Command Works 525*
*Standard and Extended traceroute 526*
Telnet and SSH 527
Common Reasons to Use the IOS Telnet and SSH Client 528
IOS Telnet and SSH Examples 529
Chapter Review 530

**Part V Review 534**

**Part VI OSPF 537**

**Chapter 21 Understanding OSPF Concepts 538**

"Do I Know This Already?" Quiz 538

Foundation Topics 540

Comparing Dynamic Routing Protocol Features 540

Routing Protocol Functions 541

Interior and Exterior Routing Protocols 542

Comparing IGPs 543

*IGP Routing Protocol Algorithms 543*

*Metrics 544*

*Other IGP Comparisons 545*

OSPF Concepts and Operation 546

OSPF Overview 546

*Topology Information and LSAs 546*

*Applying Dijkstra SPF Math to Find the Best Routes 547*

Becoming OSPF Neighbors 548

*The Basics of OSPF Neighbors 548*

*Meeting Neighbors and Learning Their Router ID 548*

Exchanging the LSDB Between Neighbors 550

*Fully Exchanging LSAs with Neighbors 550*

*Maintaining Neighbors and the LSDB 551*

*Using Designated Routers on Ethernet Links 552*

Calculating the Best Routes with SPF 553

OSPF Areas and LSAs 555

OSPF Areas 555

How Areas Reduce SPF Calculation Time 556

(OSPFv2) Link-State Advertisements 557

*Router LSAs Build Most of the Intra-Area Topology 558*

*Network LSAs Complete the Intra-Area Topology 559*

Chapter Review 560

**Chapter 22 Implementing Basic OSPF Features 562**

"Do I Know This Already?" Quiz 562

Foundation Topics 564

Implementing OSPFv2 Using network Commands 564

OSPF Single-Area Configuration 565

Wildcard Matching with the network Command 566

Verifying OSPF Operation 569

Verifying OSPF Configuration 572

Configuring the OSPF Router ID 574
Implementing Multiarea OSPF 575
Implementing OSPFv2 Using Interface Subcommands 576
OSPF Interface Configuration Example 576
Verifying OSPF Interface Configuration 578
Chapter Review 580

**Chapter 23 Implementing Optional OSPF Features 584**
"Do I Know This Already?" Quiz 584
Foundation Topics 586
OSPF Network Types 586
The OSPF Broadcast Network Type 587
*Verifying Operations with Network Type Broadcast 588*
*Using Priority and RID to Influence the DR/BDR Election 590*
The OSPF Point-to-Point Network Type 592
Additional Optional OSPFv2 Features 594
OSPF Passive Interfaces 594
OSPF Default Routes 597
OSPF Metrics (Cost) 599
*Setting the Cost Directly 599*
*Setting the Cost Based on Interface and Reference Bandwidth 600*
OSPF Hello and Dead Intervals 602
Chapter Review 604

**Chapter 24 OSPF Neighbors and Route Selection 608**
"Do I Know This Already?" Quiz 608
Foundation Topics 611
OSPF Neighbor Relationships 611
OSPF Neighbor Requirements 611
Issues That Prevent Neighbor Adjacencies 612
*Finding Area Mismatches 613*
*Finding Duplicate OSPF Router IDs 613*
*Finding OSPF Hello and Dead Timer Mismatches 614*
*Shutting Down the OSPF Process 615*
*Shutting Down OSPF on an Interface 617*
Issues That Allow Neighbors but Prevent IP Routes 618
*Mismatched MTU Settings 618*
*Mismatched OSPF Network Types 618*
*Both Neighbors Using OSPF Priority 0 618*
*Examples That Show OSPF Neighbors but No Routes 619*

Route Selection 621
Equal-Cost Multipath OSPF Routes 621
Multiple Routes Learned from Competing Sources 622
IP Forwarding with the Longest Prefix Match 625
*Using Your Subnetting Math Skills to Predict the Choice of Best Route 626*
*Using show ip route address to Find the Best Route 628*
*Interpreting the IP Routing Table 628*
Chapter Review 630

**Part VI Review 634**

**Part VII IP Version 6 637**

**Chapter 25 Fundamentals of IP Version 6 638**
"Do I Know This Already?" Quiz 638
Foundation Topics 640
Introduction to IPv6 640
The Historical Reasons for IPv6 640
The IPv6 Protocols 642
IPv6 Routing 643
IPv6 Routing Protocols 645
IPv6 Addressing Formats and Conventions 646
Representing Full (Unabbreviated) IPv6 Addresses 646
Abbreviating and Expanding IPv6 Addresses 647
*Abbreviating IPv6 Addresses 647*
*Expanding Abbreviated IPv6 Addresses 648*
Representing the Prefix Length of an Address 649
Calculating the IPv6 Subnet Prefix (Subnet ID) 649
Finding the IPv6 Subnet Prefix 649
Working with More-Difficult IPv6 Prefix Lengths 651
Chapter Review 652

**Chapter 26 IPv6 Addressing and Subnetting 654**
"Do I Know This Already?" Quiz 654
Foundation Topics 655
Global Unicast Addressing Concepts 655
Public and Private IPv6 Addresses 656
The IPv6 Global Routing Prefix 657
Address Ranges for Global Unicast Addresses 659
IPv6 Subnetting Using Global Unicast Addresses 659

*Deciding Where IPv6 Subnets Are Needed 660*
*The Mechanics of Subnetting IPv6 Global Unicast Addresses 660*
*Listing the IPv6 Subnet Prefix (Subnet ID) 662*
*List All IPv6 Subnets 663*
*Assign Subnets to the Internetwork Topology 663*
Assigning Addresses to Hosts in a Subnet 664
Unique Local Unicast Addresses 664
Subnetting with Unique Local IPv6 Addresses 665
The Need for Globally Unique Local Addresses 666
Chapter Review 667

**Chapter 27 Implementing IPv6 Addressing on Routers 668**
"Do I Know This Already?" Quiz 668
Foundation Topics 670
Implementing Unicast IPv6 Addresses on Routers 670
Static Unicast Address Configuration 671
*Configuring the Full 128-Bit Address 671*
*Enabling IPv6 Routing 672*
*Verifying the IPv6 Address Configuration 673*
*Generating a Unique Interface ID Using Modified EUI-64 674*
*IPv6 Address Attributes 678*
Dynamic Unicast Address Configuration 679
Special Addresses Used by Routers 680
Link-Local Addresses 680
*Link-Local Address Concepts 680*
*Creating Link-Local Addresses on Routers 681*
*Routing IPv6 with Only Link-Local Addresses on an Interface 683*
IPv6 Multicast Addresses 684
*Well-Known Multicast Addresses 684*
*Multicast Address Scopes 686*
*Solicited-Node Multicast Addresses 687*
The Unspecified and Loopback Addresses 689
Anycast Addresses 689
IPv6 Addressing Configuration Summary 690
Chapter Review 691

**Chapter 28 Implementing IPv6 Addressing on Hosts 696**
"Do I Know This Already?" Quiz 696
Foundation Topics 698

The Neighbor Discovery Protocol 698
Discovering Neighbor Link Addresses with NDP NS and NA 699
Discovering Routers with NDP RS and RA 702
Discovering Prefixes with NDP RS and RA 703
Discovering Duplicate Addresses Using NDP NS and NA 705
NDP Summary 705
Dynamic Configuration of Host IPv6 Settings 706
Using Stateful DHCP 706
*Differences Between Stateful DHCPv6 and DHCPv4 707*
*DHCPv6 Relay Agents 708*
Using Stateless Address Autoconfiguration 710
*Building an IPv6 Address Using SLAAC 710*
*Combining SLAAC with Stateless DHCP 711*
*Combining SLAAC with RA-Based DNS Server Configuration 712*
Permanent and Temporary SLAAC Addresses 712
Troubleshooting Host IPv6 Addressing 714
Verifying IPv6 Connectivity from Hosts 714
*Host Commands to Find IPv6 Interface Addresses 714*
*Testing IPv6 Connectivity with ping and traceroute 716*
Verifying Host Connectivity from Nearby Routers 718
Chapter Review 719

**Chapter 29 Implementing IPv6 Routing 722**

"Do I Know This Already?" Quiz 722
Foundation Topics 724
Connected and Local IPv6 Routes 724
Rules for Connected and Local Routes 725
Example of Connected IPv6 Routes 725
Examples of Local IPv6 Routes 727
Static IPv6 Network Routes 728
Static Network Routes Using an Outgoing Interface 729
Static Network Routes Using Next-Hop IPv6 Address 730
*Example Static Network Route with a Next-Hop GUA 731*
*Example Static Network Route with a Next-Hop LLA 733*
Static Default, Host, and Floating Static IPv6 Routes 735
Static IPv6 Default Routes 735
Static IPv6 Host Routes 737
Floating Static IPv6 Routes 739

Troubleshooting Static IPv6 Routes 741
Troubleshooting Incorrect Static Routes That Appear in the IPv6 Routing Table 741
The Static Route Does Not Appear in the IPv6 Routing Table 743
Chapter Review 744

**Part VII Review 746**

**Part VIII Exam Updates 749**

**Chapter 30 *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition Exam Updates 750**

The Purpose of This Chapter 750
Additional Technical Content 751
About Possible Exam Updates 751
Impact on You and Your Study Plan 753
News about the Next CCNA Exam Release 754
Updated Technical Content 754

**Part IX Appendixes 757**

**Appendix A Numeric Reference Tables 759**

**Appendix B Exam Topics Cross-Reference 765**

**Appendix C Answers to the "Do I Know This Already?" Quizzes 779**

**Glossary 809**

**Index 840**

## Online Appendixes

**Appendix D Practice for Chapter 12: Analyzing Classful IPv4 Networks**

**Appendix E Practice for Chapter 13: Analyzing Subnet Masks**

**Appendix F Practice for Chapter 14: Analyzing Existing Subnets**

**Appendix G Practice for Chapter 15: Subnet Design**

**Appendix H Practice for Chapter 25: Fundamentals of IP Version 6**

**Appendix I Practice for Chapter 27: Implementing IPv6 Addressing on Routers**

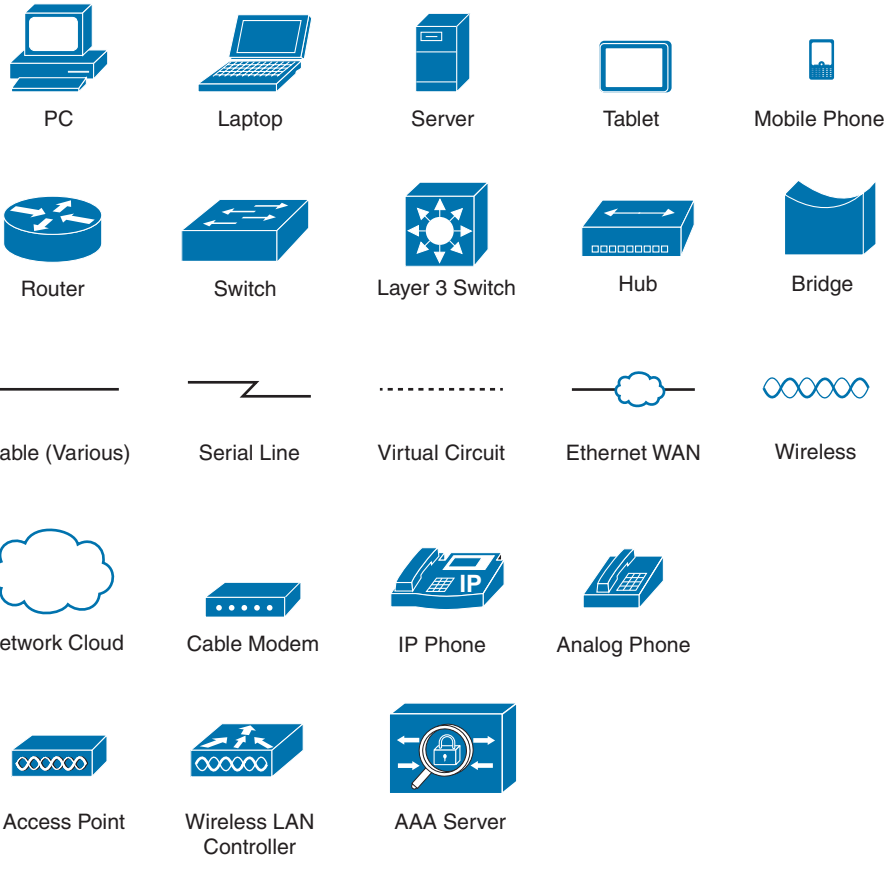
**Appendix J Study Planner**

**Appendix K Topics from Previous Editions**

**Appendix L LAN Troubleshooting**

**Appendix M Variable-Length Subnet Masks**

## Icons Used in This Book




## Command Syntax Conventions

The conventions used to present command syntax in this book are the same conventions used in the IOS Command Reference. The Command Reference describes these conventions as follows:

- **Boldface** indicates commands and keywords that are entered literally as shown. In actual configuration examples and output (not general command syntax), boldface indicates commands that are manually input by the user (such as a **show** command).
- *Italic* indicates arguments for which you supply actual values.
- Vertical bars (|) separate alternative, mutually exclusive elements.
- Square brackets ([ ]) indicate an optional element.
- Braces ({ }) indicate a required choice.
- Braces within brackets ([{ }]) indicate a required choice within an optional element.

# Introduction

Do not skip the intro!

You are setting out on a journey to achieve your CCNA certification. For many, that step happens at the beginning of a new career path. For others, CCNA validates their knowledge and skills already learned on the job.

Regardless of your path, the journey takes some time and effort. I encourage you to spend some time in the Introduction to learn more about CCNA and the books so you can have the best experience preparing for CCNA! To that end, this introduction discusses these main points:

Cisco Certifications and the CCNA

Book Features

Book Elements (Reference)

About Getting Hands-on Skills

About IP Subnetting

# Cisco Certifications and the CCNA

Congratulations! If you're reading far enough to look at this book's Introduction, you've probably already decided to go for your Cisco certification. Cisco has been the dominant vendor in networking for decades. If you want to be taken seriously as a network engineer, building your Cisco skills using Cisco certifications makes perfect sense. Where to start? CCNA.

## Cisco Certifications as of 2024

The changes Cisco made in 2020 consolidated the certification tracks from about ten tracks down to the five tracks shown in Figure I-1. Cisco next made changes to various exams in 2023 and 2024; those changes updated the exams but maintained the same five tracks. The CCNA, CCNP, and CCIE certification levels progress through higher challenge levels, with CCNA as the foundation for all.

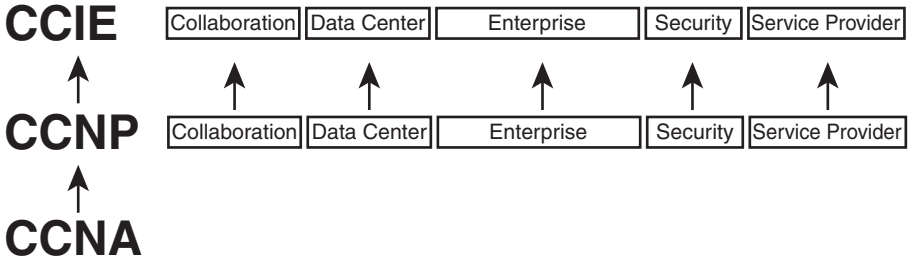



**Figure I-1** *Cisco CCNA, CCNP, and CCIE Certifications*

The following list gives a few details of the history of these certification tracks: They are

**CCNA – Cisco Certified Network Associate:** Cisco began CCNA with a single CCNA certification back in 1998. They later expanded CCNA to include ten different CCNA

certifications about different technology areas. Cisco retired all the varieties of CCNA back in 2020, leaving us again with a single CCNA certification, now referred to as simply "CCNA."

**CCNP – Cisco Certified Network Professional:** Cisco followed the same progression with different CCNP certifications over time, starting with one in 1998. The big changes in 2020 consolidated the lineup to five CCNP certifications, all of which benefit from having knowledge of CCNA before moving on to CCNP.

**CCIE – Cisco Certified Internetwork Expert:** First introduced in 1993, these expert-level certifications require both a written exam plus a one-day practical exam with extensive hands-on lab challenges.

Beyond the CCNA, CCNP, and CCIE certifications, Cisco offers two other certification tracks, one for network automation and another for cybersecurity. The CCNA certification can be helpful as a foundation for those tracks as well. They are

**DevNet Certifications:** The DevNet Associate, DevNet Professional, and DevNet Expert certifications mirror the progression of CCNA/CCNP/CCIE, just without using those specific acronyms. The DevNet certifications focus on software development and APIs that matter to managing networks.

**CyberOps Certifications:** The CyberOps Associate and CyberOps Professional certifications mirror the progression of CCNA/CCNP. These security exams focus on security concepts, security monitoring, host-based analysis, network intrusion analysis, and security policies and procedures.

## How to Get Your CCNA Certification

As you saw in Figure I-1, all career certification paths now begin with CCNA. So how do you get the CCNA certification? Today, you have one and only one option to achieve CCNA certification:

Take and pass one exam: The Cisco 200-301 CCNA exam.

To take the 200-301 exam, or any Cisco exam, you will use the services of Pearson VUE. The process works something like this:

1. Establish a login at https://vue.com/cisco (or use your existing login).
2. Register for, schedule a time and place, and pay for the Cisco 200-301 exam, all from the VUE website.
3. Take the exam at the VUE testing center or from home with a video proctor watching to prevent cheating.
4. You will receive a notice of your score, and whether you passed, before you leave the testing center.

## Content in the CCNA 200-301 Exam

We've all thought it, wondered, for almost every important test we ever took, and maybe even asked the teacher: "What's on the test?" For the CCNA exam, and for all Cisco certification exams, Cisco tells us.

Cisco publishes an exam blueprint for every Cisco exam, with the blueprint listing the exam topics for the exam. To find them, browse www.cisco.com/go/certifications, look for the CCNA page, and navigate until you see the exam topics. And if you haven't already done so, create a bookmark folder for CCNA content in your web browser and bookmark a link to this page.

The exam blueprint organizes the exam topics into groups called domains. The document also tells us the percentage of points on the exam that come from each domain. For instance, every CCNA exam should score 25 percent of your points from the exam topics in the IP Connectivity domain. The exam does not tell you the domain associated with each question, but the percentages give us a better idea of the importance of the domains for the exam. Figure I-2 shows the domains of the CCNA 200-301 Version 1.1 blueprint, the percentages, and the number of primary exam topics in each.

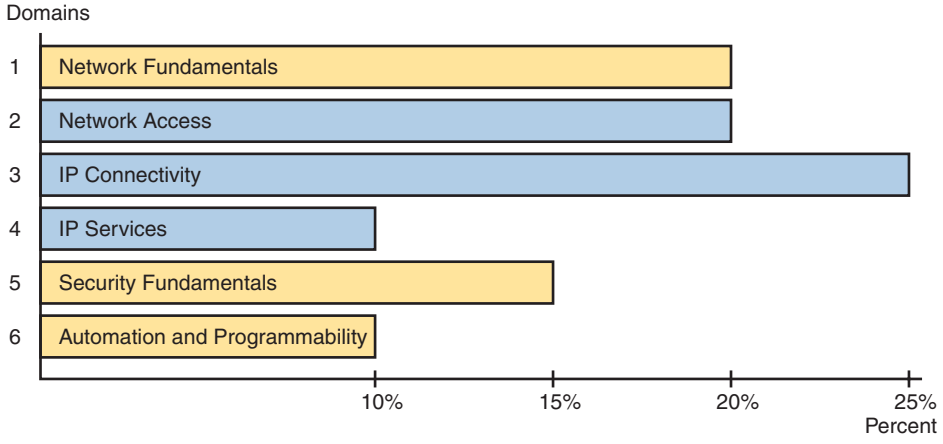



**Figure I-2** *CCNA 200-301 Domains and Percentage of Exam Score*

Within each domain, the exam topic document lists exam topics that follow two different styles of wording. The main exam topics use a verb in the phrase that tells you the level of mastery required; I call those primary exam topics. The exam topics document shows subtopics that I refer to as secondary exam topics. Those do not have a verb, but list more technology details (nouns), and assume the verb from the primary exam topic. For instance, the following excerpt from the exam topics document lists one primary exam topic with the *describe* verb, with more detail added by two secondary exam topics.

1.13 Describe switching concepts

1.13.a MAC learning and aging

1.13.b Frame switching

## Exam Topic Verbs (Depth) and Nouns (Breadth)

Understanding an exam topic requires that you think about each exam topic wording, focusing on the verbs and nouns. The nouns identify the technical topics, such as LAN switching, IP routing, protocols like OSPF, and so on. The verbs in each primary exam topic inform us about the type and depth of knowledge and skill tested per the exam topics.

For example, consider the following primary exam topic:

**Describe** IPsec remote access and site-to-site VPNs

I'm sure you know what the word *describe* means in the normal use of the term. But for people who build exams, the verb has special meaning as to what the exam questions should and should not require of the test taker. For instance, you should be ready to describe whatever "IPsec remote access and site-to-site VPNs" are. But the exam should not ask you to perform higher performance verbs, like *analyze* or *configure*.

Figure I-3 shows a pyramid with verbs found in Cisco exam blueprints. It shows the lower-skill verbs at the bottom and higher skills at the top. An exam topic with a lower verb should not be tested with questions from higher knowledge and skill levels. For instance, with the exam topic "describe…first hop redundancy protocols," you should not expect to need to configure, verify, or troubleshoot the feature.

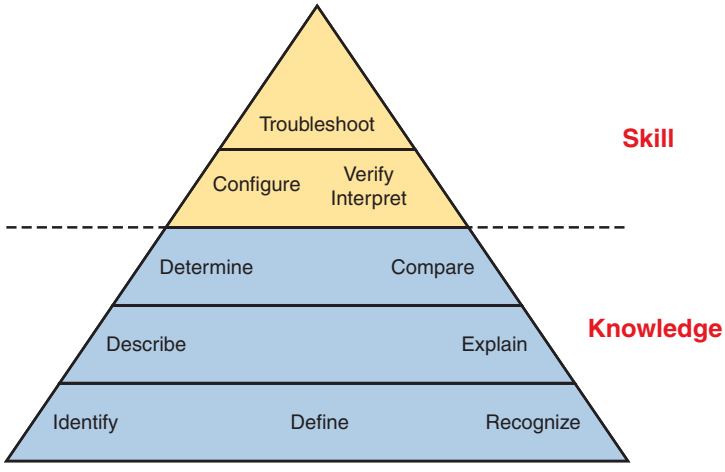



**Figure I-3** *Cisco Exam Topic Verbs*

Knowing that, how should you study? Well, instead of a many-layer pyramid, think of it as two layers: Knowledge and Skill. When learning content whose exam topics use verbs from the lower three rows of the pyramid, study the same way no matter which of those verbs the exam topic uses. Learn the topic well. Be ready to describe it, explain it, and interpret the meaning. For content with exam topics with the verbs *configure* and *verify*, think of those as including the first level of knowledge, plus also requiring configuration and verification skills. Also, think about the common configuration mistakes so you can troubleshoot those mistakes.

## Comparing the Exam and Exam Topics

Cisco tells us that the exam can include more technical topics than those listed as nouns in the exam topics. Cisco also tells us that the exam topics give us general guidance. Once you get into the content, you will understand what they mean: any noun listed in the exam topics has many related protocols, standards, features, concepts, or device commands that Cisco did not list in the exam topics. Let's explore that concept to give you some perspective.

First, to see what Cisco tells us about the exam versus the exam topics, return to cs.co/go/certifications or cisco.com/go/ccna. Find the CCNA exam topics and open the PDF version (the text we need to consider is currently only in the PDF version). Open the PDF and spend 10–15 seconds scanning it.

Did you read the first two paragraphs, the ones before the list of exam topics? Or did you skip those and move straight to the long list of exam topics? Many people skip those paragraphs. One of those tells us much about the exam versus the exam topics, so I've copied it here, with emphasis added:

> The following topics are ***general guidelines*** for the content likely to be included on the exam. However, ***other related topics may also appear on any specific delivery of the exam***. To better reflect the contents of the exam and for clarity purposes, the ***guidelines below may change at any time without notice***.

Together, the first two emphasized phrases tell us that the exam may go beyond the literal words in the exam topics. Let me give you a couple of examples. First, prerequisite knowledge must be inferred from the literal exam topics. For instance, consider this exam topic:

Configure and verify IPv4 addressing and subnetting

The skills to configure IPv4 addresses take only a few minutes to learn. Understanding what the numbers mean takes much longer. In fact, I'd say 95 percent of your work will be to understand the prerequisite knowledge—but the exam topics do not list those prerequisites, like understanding subnetting concepts, applying the subnet mask to an address, calculating the range of addresses in a subnet, and so on.

I develop the scope of the books with the preceding in mind. You will certainly read about all topics that appear in the exam topics. Consider that view a narrow interpretation of the exam topics. But you will also learn about terms, concepts, and product features not specifically mentioned in the exam topics, from my broad and deep interpretation of the exam topics, based on Cisco's approach to their exams. We try to predict what Cisco will include, starting from the exam topics. Figure I-4 shows the idea.

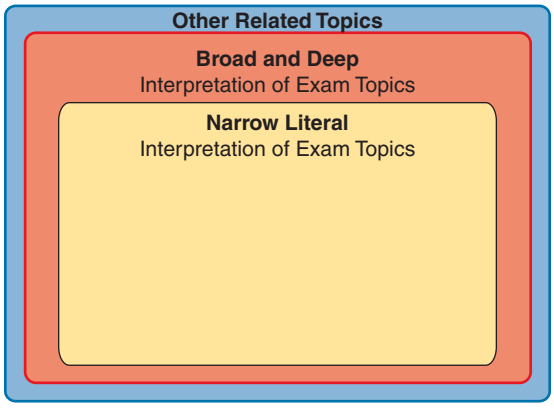



**Figure I-4** *Scope Concept: Exam Versus Exam Topics*

Although we can build a book around the exam topics, we cannot predict every concept and command included in the actual CCNA exam. The general nature of the exam topics, the allowance of "other related topics," plus other factors, make predicting all exam content impossible. But we do promise to discuss 100 percent of the exam topic details and to diligently seek the right balance of a broad interpretation of those topics to make you well prepared for the exam.

### How to Prepare for the Generalized Exam Topics

Given the possibility of topic areas not listed in the exam topics, how should you go about preparing for the CCNA exam? Let me give you a few suggestions.

1. Follow the suggestions in the upcoming section "Your Study Plan" just before Chapter 1.
2. Practice hands-on Command Line Interface (CLI) skills. The later section of the Introduction titled "About Building Hands-On Skills" discusses some ways to practice.
3. Pay close attention to troubleshooting topics in the book.
4. Practice all math-related skills, over time, until you master them.
5. Ensure you know all exam topic content as listed in the exam topics. Read the exam topics, consider your own literal interpretation, and when uncertain or confused, dig in and study further.
6. Trust that the book uses its broad interpretation of the exam topics to help you learn as much as possible that might be on the exam.

## Types of Questions on the CCNA 200-301 Exam

You can expect the following kinds of questions on the exam; just be aware that the style of questions may change over time.

- Multiple-choice, single-answer
- Multiple-choice, multiple-answer
- Drag-and-drop
- Lab

For the multichoice questions, the exam software gives us a few important advantages:

- There is no penalty for guessing.
- Multichoice questions with a single correct answer require you to answer and allow only one answer.
- Multichoice questions with multiple correct answers tell you the number of correct answers and warn you if you have not selected that many answers.

For instance, if a question tells you there are two correct answers, and you select only one and then try to move to the next question, the app reminds you that you should choose another answer before moving on.

As for drag-and-drop, some questions use simple text blocks that you move from one list to another. However, you might see questions where you move items in a network diagram or some other creative use of drag-and-drop.

Finally, Cisco introduced lab questions (formally called performance-based questions) in 2022. Lab questions present you with a lab scenario with a lab pod of virtual routers and switches running in the background; you get console access to a few devices. Your job: find the missing or broken configuration and reconfigure the devices so that the lab scenario works. The best way to practice for these questions is to practice in the lab; more on that in the section titled "About Building Hands-On Skills."

As an aside, prior Cisco exams had Sim questions instead of lab questions. Sim questions required the same from us: read the scenario and fix the configuration. However, Sim questions used simulated Cisco devices with limited command support, which frustrated some test takers. The lab questions use real Cisco operating systems running in a virtual environment, so they provide a much more realistic experience compared to old Sim questions.

## Book Features

This book includes many study features beyond the core explanations and examples in each chapter. This section acts as a reference to the various features in the book.

### The CCNA Books: Volume 1 and Volume 2

The CCNA exam covers a large amount of content, and it does not fit in a single volume. As a result, Cisco Press has long published books for the CCNA exam as a two-book set. Volume 1 covers about half of the content, and Volume 2 covers the rest, as shown in Figure I-5. To best use both books, start in Volume 1 and work through the book in order, and then do the same with Volume 2.

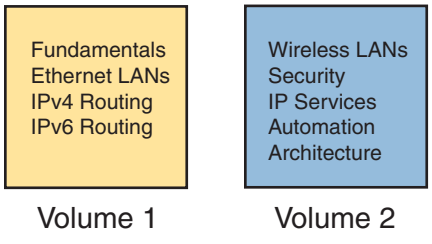



**Figure I-5** *Two Books for CCNA 200-301*

When you start each new chapter, review the list of exam topics that begins the chapter. The book does not follow the same order of exam topics in the blueprint, but instead follows a more effective order for learning the topics. For reference, look to Appendix B, "Exam Topics Cross-Reference," in the back of the book. The appendix includes:

- A list of exam topics and the chapter(s) covering each topic
- A list of chapters and the exam topics covered in each chapter

## Exam Blueprint Versions and Book Editions

Cisco made minor changes to the CCNA exam blueprint in 2024, the first change to the CCNA 200-301 exam since the year 2020. The much more important change (announced in 2023) had to do with the entire Cisco certification program about how Cisco announces and releases new exams and exam blueprints. Before 2023, when Cisco changed any CCNA or CCNP exam, they also changed the exam number, and the announcement was sudden. Those days are gone.

You should read and understand Cisco's long-term strategy for being more forthright about exam plans as detailed at www.cisco.com/go/certroadmap. Summarizing some key points, when Cisco changes an exam in the future, Cisco will keep the same exam number. To identify the changes, they will use a major.minor version numbering plan for every exam blueprint. More importantly, Cisco tells us when they will consider changing CCNA each year, but we know when Cisco will announce changes and when the new exam will be released, within a few months' timing.

The exam blueprint version changes based on two determinations: 1) whether Cisco will change the exam that year at all, and 2) if so, whether Cisco considers the changes to be major or minor. For instance, Cisco considered making a change to CCNA during February–April 2023 but chose not to change it, announcing that fact in the May–July 2023 timeframe. In 2024, Cisco chose to make minor changes to the CCNA blueprint. As a result, the former CCNA blueprint version 1.0 (major version 1, minor version 0) changed to version 1.1, increasing the minor version by 1.

Looking forward, if the next three future CCNA blueprint changes are also minor, they would be blueprint versions 1.2, 1.3, and 1.4. However, if any of them are major, that version would move to the next major version (2.0), with subsequent minor version changes as 2.1, 2.2, and so on.

Cisco also tells us that each year, internally, Cisco considers what to do with CCNA in the February–April timeframe. They will announce their plans to us all between May–July, and they will release the new exam (if changes are being made) sometime in the six months or so following the announcement.

As for the publishing plans to support that new update cycle, you should read and monitor the publisher's web page at www.ciscopress.com/newcerts. Also, opt in for communications on that page so the publisher will email you about future plans and updates.

Summarizing a few key points about the publisher's plans, this book, the second edition, was written for version 1.1 of the CCNA 200-301 blueprint, but it should be the book used for CCNA for subsequent blueprint versions as well. During the life of this second edition book, Cisco may update the CCNA 200-301 exam blueprint a few times, while this book (plus the Volume 2 second edition book) may remain unchanged. New exam content may be made available as electronic downloads. At some point, a new edition will be appropriate. (Figure I-6 shows one example of what might happen over time, with downloadable PDFs between editions.)

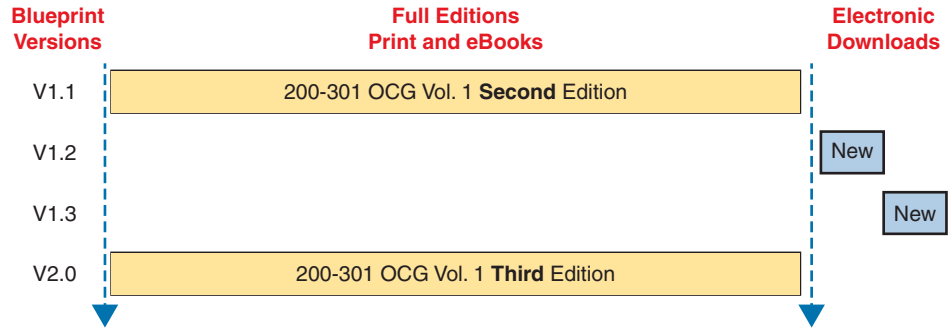



**Figure I-6** *Possible Progression of Book Editions, New Content Release, Versus Exams*

**NOTE** I cannot stress enough: monitor both the Cisco Press and Cisco pages linked in the preceding paragraphs, and opt in for communications at those pages, to stay aware of any exam and publishing plans. Also, consider watching my blog (www.certskills.com), where I expect to post about changes.

When you finish the technology chapters in this book (Chapters 1–29), make sure to also read Chapter 30, "Exam Updates." We post updated versions of that chapter online. We use that chapter to deliver small content updates to you as well as to inform you about future content updates. Make sure to read through that chapter and learn how to download the latest version of the chapter, discovering if new content has been posted after this book was published.

Also, just to reduce confusion about the book titles, note that the prior edition of this book is nearly identical to this book's title. Comparing the titles:

- *CCNA 200-301 Official Cert Guide, Volume 1* (the prior edition, published in 2019, for 200-301 exam blueprint version 1.0)
- *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition (this edition, published in 2024, for 200-301 exam blueprint version 1.1 and beyond)

## Comparing This Edition to the Previous

This book replaces a similar book that applied to the former CCNA 200-301 exam blueprint 1.0. Some of you may buy this book but have already begun studying with the prior edition. The following list of major changes to this book versus the previous one may help you avoid rereading identical or similar content in this book.

**Chapter 4:** Added a short topic comparing Cisco switch operating systems. Also introduced the WebUI interface for Cisco routers and switches per an exam topic change.

**Chapter 6:** Explored the differences in the **transport input** configuration command based on device models and operating systems. Also showed how to configure the WebUI on a switch per an exam topic change.

**Chapter 7:** Revised and expanded sections about Ethernet autonegotiation concepts and auto-MDIX.

**Chapter 8:** Added a large closing section about troubleshooting VLANs and VLAN trunks.

**Chapter 9:** Removed EtherChannel load distribution concept details, and added concept details about BPDU Guard, BPDU filter, loop guard, and root guard.

**Chapter 10:** Expanded/revised the second half of the chapter (Layer 2 EtherChannels), and added a new middle section about verifying BPDU Guard, BPDU filter, loop guard, and root guard.

**Chapter 12:** Added a few pages comparing public IP networks and public CIDR blocks.

**Chapter 15:** Added an entire new chapter about subnetting design issues. The chapter was in the prior edition but as a PDF-only appendix.

**Chapter 16:** (Formerly Chapter 15) Expanded the discussion of router interface autonegotiation and differences between IOS and IOS XE.

**Chapter 17:** (Formerly Chapter 16) Added a small section discussing possible mistakes with the **ip address** interface subcommand.

**Chapter 18:** (Formerly Chapter 17) Added two topics: SVI interface auto-state and the configuration of Ethernet switch ports in a router.

**Chapter 19:** (Formerly Volume 2, Chapter 7) Moved the chapter to Volume 1 from Volume 2 and expanded the discussion of host commands.

**Chapters 21–24:** (Formerly Chapters 19–21) Chapters 21–24 reorganize and expand the OSPF topics. Chapter 24 closes with a revised section about route selection.

**Chapter 28:** This new chapter contains the final topic of former Chapter 25 (IPv6 NDP), unchanged. The second half of the chapter has new content about host IPv6 commands and IPv6 address attributes.

If you find the preceding information useful, consider looking in two other places that allow us to provide ongoing updates and to answer questions. First, I expect to post blog posts about the new CCNA exam changes, as always, at my blog (www.certskills.com). Look there for posts in the News section (click the General menu item and then News), for posts made around the time the new exams release.

Second, look to the companion website for this book for details about future exam revisions and publishing plans. The companion website gives the publisher a place to list details about changes moving forward. See this Introduction's later section titled "The Companion Website for Online Content" for the instructions for finding the site.

## Chapter Features

Beginning to study CCNA can be overwhelming at first due to the volume. The best way to overcome that reaction requires a change in mindset: *treat each chapter as a separate study task*. Breaking your study into manageable tasks helps a lot.

Each chapter of this book is a self-contained short course about one small topic area, organized for reading and study. I create chapters so they average about 20 pages to cover

the technology so that no one chapter takes too long to complete. Each chapter breaks down as follows:

**"Do I Know This Already?" quizzes:** Each chapter begins with a pre-chapter quiz so you can self-assess how much you know coming into the chapter.

**Foundation Topics:** This is the heading for the core content section of the chapter, with average length of 20 pages.

**Chapter Review:** This section includes a list of study tasks useful to help you remember concepts, connect ideas, and practice skills-based content in the chapter.

Do not read the "Foundation Topics" section of chapter after chapter without pausing to review and study. Each "Chapter Review" section uses a variety of other book features to help you study and internalize that chapter's content, including the following:

- **Review Key Topics:** All the content in the books matters, but some matters more. Cisco Press certification guides use a Key Topic icon next to those items in the "Foundation Topics" section. The "Chapter Review" section lists the key topics in a table. You can scan the chapter to review them or review the Key Topics more conveniently using the companion website.
- **Complete Tables from Memory:** We convert some tables in the book to interactive study tables called memory tables. You access memory tables from the companion website. Memory tables repeat the table, but with parts of the table removed. You can then fill in the table to exercise your memory and click to check your work.
- **Key Terms You Should Know:** The "Chapter Review" section lists the key terminology from the chapter. For a manual process with the book, think about each term and use the Glossary to cross-check your own mental definitions. Alternately, review the key terms with the "Key Terms Flashcards" app on the companion website.
- **Labs:** You should practice hands-on skills for any exam topics with the *configure* and *verify* verbs. The upcoming section titled "About Building Hands-On Skills" discusses your lab options. Also, the Chapter and Part Reviews refer you to lab exercises specific to the chapter or part.
- **Command References:** Some book chapters discuss the configure and verify exam topics, so they list various router and switch commands. The "Chapter Review" section of those chapters includes command reference tables, useful both for reference and for study. Just cover one column of the table and see how much you can remember and complete mentally.
- **Review DIKTA Questions:** Even if you used the DIKTA questions to begin the chapter, re-answering those questions can prove a useful way to review facts. By design, I do not mention the DIKTA questions in the "Chapter Review" sections but do suggest using them again for all chapters in a part during Part Review. Use the Pearson Test Prep (PTP) web app to easily use those questions any time you have a few minutes, a device, and Internet access.
- **Subnetting Exercises:** Several chapters ask you to perform some math processes related to either IPv4 or IPv6 addressing. The "Chapter Review" section asks you

to do additional practice problems, where applicable. The problems can be found in Appendices D through I, in PDF form, on the companion website, along with those same exercises as interactive web apps.

## Part Features

Your second mindset change: Use the book parts as major milestones in your study journey. Each part groups a small number of related chapters together. Take the time at the end of each part to review all topics in the part, effectively rewarding yourself with a chance to deepen your knowledge and internalize more of the content before moving to the next part. Figure I-7 shows the concept.

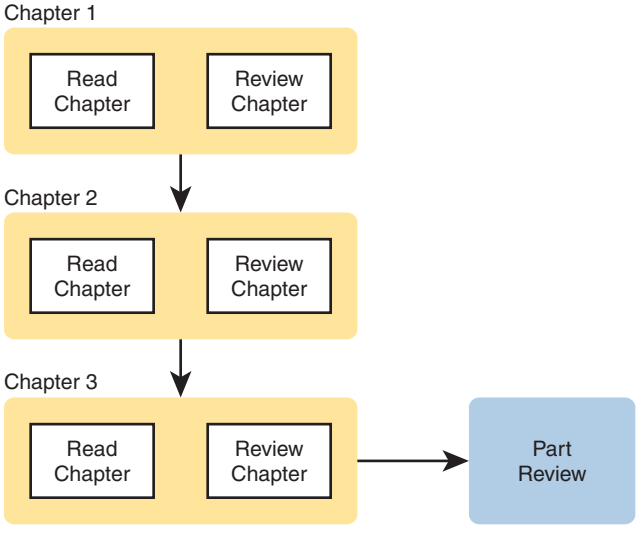



**Figure I-7** *Part Review: The Second Review of Most Content*

The Part Review element at the end of each part suggests review and study activities. Spaced reviews—that is, reviewing content several times over the course of your study—help improve retention. Using the Chapter and Part Review process, the Part Review serves as your second review of the content in each chapter. The Part Review repeats some Chapter Review activities and offers some new ones, including a reminder to use practice questions set aside specifically for Part Review.

## The Companion Website for Online Content

Some Chapter and Part Review tasks can be done from the book. However, several of them work better as an interactive online tool. For instance, you can take a "Do I Know This Already?" quiz by reading the pages of the book, but you can also use the PTP testing software. As another example, when you want to review the key terms from a chapter, you can find all those in electronic flashcards.

This book's companion website hosts all the electronic components of the book. The companion website gives you a big advantage: you can do most of your Chapter and

Part Review work from anywhere using the interactive tools on the site. The advantages include

- **Easier to use:** Instead of having to print out copies of the appendices and do the work on paper, you can use these new apps, which provide you with an easy-to-use, interactive experience that you can easily run over and over.
- **Convenient:** When you have a spare 5–10 minutes, go to the book's website and review content from one of your recently finished chapters.
- **Good break from reading:** Sometimes looking at a static page after reading a chapter lets your mind wander. Breaking up your reading with some review from the keyboard can help keep you focused on the activity.

The interactive Chapter Review elements should improve your chances of passing as well. Our in-depth reader surveys over the years show that those who do the Chapter and Part Reviews learn more. Those who use the interactive review elements tend to do the review tasks more often. So, take advantage of the tools and maybe you will be more successful as well. Table I-1 summarizes these interactive applications and the traditional book features that cover the same content.

**Table I-1** *Book Features with Both Traditional and App Options*

| Feature | Traditional | App |
|---|---|---|
| Key Topic | The "Chapter Review" section lists the key topics. To review, flip pages in the chapter. | Key Topics Table app with links to view each key topic |
| Config Checklist | This list of steps, in text, describes how to configure a feature. | Config Checklist app, where you complete the checklist by adding commands |
| Key Terms | Terms are listed in each "Chapter Review" section; review using the end-of-book Glossary. | Key Terms Flash Cards app |
| Appendices Subnetting Practice | Appendices D–I provide static text practice problems and answers in the PDF appendices. | A variety of apps, one per problem type, in the "Memory Tables and Practice Exercises" section |
| In-chapter Subnetting practice | Look at the problems in the chapter and refer to the answer tables at the end of the chapter. | App that offers a fill-in-the-blank answer space and grades your answer |

The companion website also includes links to download, navigate, or stream for these types of content:

- Pearson Sim Lite Desktop App
- Pearson Test Prep (PTP) Desktop App
- Pearson Test Prep (PTP) Web App
- Videos

## How to Access the Companion Website

To access the companion website, which gives you access to the electronic content with this book, start by establishing a login at www.ciscopress.com and register your book. To do so, simply go to www.ciscopress.com/register and enter the ISBN of the print book: 9780138229634. After you have registered your book, go to your account page and click the **Registered Products** tab. From there, click the **Access Bonus Content** link to get access to the book's companion website.

Note that if you buy the *Premium Edition eBook and Practice Test* version of this book from Cisco Press, your book will automatically be registered on your account page. Simply go to your account page, click the **Registered Products** tab, and select **Access Bonus Content** to access the book's companion website.

## How to Access the Pearson Test Prep (PTP) App

You have two options for installing and using the Pearson Test Prep application: a web app and a desktop app. To use the Pearson Test Prep application, start by finding the registration code that comes with the book. You can find the code in these ways:

- You can get your access code by registering the print ISBN 9780138229634 on ciscopress.com/register. Make sure to use the print book ISBN, regardless of whether you purchased an eBook or the print book. After you register the book, your access code will be populated on your account page under the Registered Products tab. Instructions for how to redeem the code are available on the book's companion website by clicking the Access Bonus Content link.
- If you purchase the Premium Edition eBook and Practice Test directly from the Cisco Press website, the code will be populated on your account page after purchase. Just log in at ciscopress.com, click Account to see details of your account, and click the digital purchases tab.

**NOTE** After you register your book, your code can always be found in your account under the Registered Products tab.

Once you have the access code, to find instructions about both the PTP web app and the desktop app, follow these steps:

**Step 1.** Open this book's companion website as shown earlier in this Introduction under the heading, "How to Access the Companion Website."

**Step 2.** Click the **Practice Exams** button.

**Step 3.** Follow the instructions listed there for both installing the desktop app and using the web app.

Note that if you want to use the web app only at this point, just navigate to pearsontestprep.com, log in using the same credentials used to register your book or purchase the Premium Edition, and register this book's practice tests using the registration code you just found. The process should take only a couple of minutes.

## Feature Reference

The following list provides an easy reference to get the basic idea behind each book feature:

- **Practice exam:** The book gives you the rights to the Pearson Test Prep (PTP) testing software, available as a web app and a desktop app. Use the access code on a piece of cardboard in the sleeve in the back of the book, and use the companion website to download the desktop app or navigate to the web app (or just go to www.pearsontestprep.com).
- **eBook:** Pearson offers an eBook version of this book that includes extra practice tests as compared to the print book. The product includes two versions of the eBook: PDF (for reading on your computer) and EPUB (for reading on your tablet, mobile device, or Kindle, Nook, or other e-reader). It also includes additional practice test questions and enhanced practice test features, including links from each question to the specific heading in the eBook file.
- **Mentoring videos:** The companion website also includes a number of videos about other topics as mentioned in individual chapters. Some of the videos explain common mistakes made with CCNA topics, whereas others provide sample CCNA questions with explanations.
- **Subnetting videos:** The companion website contains a series of videos that show you how to calculate various facts about IP addressing and subnetting (in particular, using the shortcuts described in this book).
- **Subnetting practice appendices/web apps:** The companion website contains appendices with a set of subnetting practice problems and answers. This is a great resource to practice building subnetting skills. You can use these same practice problems with applications from the "Memory Tables and Practice Exercises" section of the companion website.
- **CCNA 200-301 Network Simulator Lite:** This lite version of the best-selling CCNA Network Simulator from Pearson provides you with a means, right now, to experience the Cisco command-line interface (CLI). No need to go buy real gear or buy a full simulator to start learning the CLI. Just install it from the companion website.
- **CCNA Simulator:** If you are looking for more hands-on practice, you might want to consider purchasing the CCNA Network Simulator. You can purchase a copy of this software from Pearson at http://pearsonitcertification.com/networksimulator or other retail outlets. To help you with your studies, Pearson has created a mapping guide that maps each of the labs in the simulator to the specific sections in each volume of the CCNA Cert Guide. You can get this mapping guide free on the Extras tab on the book product page: www.ciscopress.com/title/9780138229634.
- **Author's websites:** The author maintains a blog site that has a large number of free lab exercises about CCNA content, additional sample questions, and other exercises. Additionally, the site indexes all content so you can study based on the book chapters and parts. To find it, navigate to www.certskills.com. Additionally, look for CCNA activities and lectures at his YouTube channel (www.youtube.com/@networkupskill).

## Book Organization, Chapters, and Appendices

This book contains 29 chapters about CCNA topics organized into seven parts. The core chapters cover the following topics:

- **Part I: Introduction to Networking**
  - **Chapter 1, "Introduction to TCP/IP Networking,"** introduces the central ideas and terms used by TCP/IP and explains the TCP/IP networking model.
  - **Chapter 2, "Fundamentals of Ethernet LANs,"** introduces the concepts and terms used when building Ethernet LANs.
  - **Chapter 3, "Fundamentals of WANs and IP Routing,"** covers the basics of the data-link layer for WANs in the context of IP routing but emphasizes the main network layer protocol for TCP/IP. This chapter introduces the basics of IPv4, including IPv4 addressing and routing.
- **Part II: Implementing Ethernet LANs**
  - **Chapter 4, "Using the Command-Line Interface,"** explains how to access the text-based user interface of Cisco Catalyst LAN switches.
  - **Chapter 5, "Analyzing Ethernet LAN Switching,"** shows how to use the Cisco CLI to verify the current status of an Ethernet LAN and how it switches Ethernet frames.
  - **Chapter 6, "Configuring Basic Switch Management,"** explains how to configure Cisco switches for basic management features, such as remote access using Telnet and SSH.
  - **Chapter 7, "Configuring and Verifying Switch Interfaces,"** shows how to configure a variety of switch features that apply to interfaces, including duplex/speed.
- **Part III: Implementing VLANs and STP**
  - **Chapter 8, "Implementing Ethernet Virtual LANs,"** explains the concepts and configuration surrounding virtual LANs, including VLAN trunking.
  - **Chapter 9, "Spanning Tree Protocol Concepts,"** discusses the concepts behind IEEE Spanning Tree Protocol (STP), including Rapid STP (RSTP) and how they make some switch interfaces block frames to prevent frames from looping continuously around a redundant switched LAN.
  - **Chapter 10, "RSTP and EtherChannel Configuration,"** shows how to configure and verify RSTP and Layer 2 EtherChannels on Cisco switches.
- **Part IV: IPv4 Addressing**
  - **Chapter 11, "Perspectives on IPv4 Subnetting,"** walks you through the entire concept of subnetting, from starting with a Class A, B, or C network to a completed subnetting design as implemented in an enterprise IPv4 network.
  - **Chapter 12, "Analyzing Classful IPv4 Networks,"** explains how IPv4 addresses originally fell into several classes, with unicast IP addresses being in Class A, B, and C. This chapter explores all things related to address classes and the IP network concept created by those classes.

- **Chapter 13, "Analyzing Subnet Masks,"** shows how an engineer can analyze the key facts about a subnetting design based on the subnet mask. This chapter shows how to look at the mask and IP network to determine the size of each subnet and the number of subnets.
- **Chapter 14, "Analyzing Existing Subnets,"** describes how most troubleshooting of IP connectivity problems starts with an IP address and mask. This chapter shows how to take those two facts and find key facts about the IP subnet in which that host resides.
- **Chapter 15, "Subnet Design,"** discusses IPv4 subnetting from the perspective of creating a list of useful subnets, based on a subnet mask, from one Class A, B, or C network.

- **Part V: IPv4 Routing**
  - **Chapter 16, "Operating Cisco Routers,"** is like Chapter 8, focusing on basic device management, but it focuses on routers instead of switches.
  - **Chapter 17, "Configuring IPv4 Addresses and Static Routes,"** discusses how to add IPv4 address configuration to router interfaces and how to configure static IPv4 routes.
  - **Chapter 18, "IP Routing in the LAN,"** shows to a configuration and troubleshooting depth different methods to route between VLANs, including Router-on-a-Stick (ROAS), Layer 3 switching with SVIs, Layer 3 switching with routed ports, and using Layer 3 EtherChannels.
  - **Chapter 19, "IP Addressing on Hosts,"** discusses how IP hosts receive their IPv4 settings from either static configuration or using DHCP.
  - **Chapter 20, "Troubleshooting IPv4 Routing,"** focuses on how to use two key troubleshooting tools to find routing problems: the **ping** and **traceroute** commands.
- **Part VI: OSPF**
  - **Chapter 21, "Understanding OSPF Concepts,"** introduces the fundamental operation of the Open Shortest Path First (OSPF) protocol, focusing on link state fundamentals, neighbor relationships, flooding link state data, and calculating routes based on the lowest cost metric.
  - **Chapter 22, "Implementing Basic OSPF Features,"** shows the most basic OSPF configuration using two methods: OSPF router subcommands and interface subcommands.
  - **Chapter 23, "Implementing Optional OSPF Features,"** discusses a wide variety of OSPF configuration options.
  - **Chapter 24, "OSPF Neighbors and Route Selection,"** examines the conditions that must be true before two routers will succeed in becoming OSPF neighbors. It also takes a closer look at the choices a router makes when choosing between competing routes and how the router uses those routes.

- **Part VII: IP Version 6**
  - **Chapter 25, "Fundamentals of IP Version 6,"** discusses the most basic concepts of IP version 6, focusing on the rules for writing and interpreting IPv6 addresses.
  - **Chapter 26, "IPv6 Addressing and Subnetting,"** works through the two branches of unicast IPv6 addresses—global unicast addresses and unique local addresses—that act somewhat like IPv4 public and private addresses, respectively.
  - **Chapter 27, "Implementing IPv6 Addressing on Routers,"** shows how to configure IPv6 routing and addresses on routers, while discussing a variety of special IPv6 addresses.
  - **Chapter 28, "Implementing IPv6 Addressing on Hosts,"** discusses how IPv6 hosts receive their IPv6 settings from either static configuration, DHCP, or SLAAC. It also discusses the NDP protocol suite.
  - **Chapter 29, "Implementing IPv6 Routing,"** shows how to add static routes to an IPv6 router's routing table.
- **Part VIII: Exam Updates**
  - **Chapter 30, "*CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, Exam Updates,"** has two purposes. First, the author will update this appendix with new content mid-edition as needed. The appendix details the download instructions. Additionally, it discusses Cisco's open approach to exam revision and release, called the Cisco Certification Roadmap.
- **Part IX: Appendixes**
  - **Appendix A, "Numeric Reference Tables,"** lists several tables of numeric information, including a binary-to-decimal conversion table and a list of powers of 2.
  - **Appendix B, "Exam Topics Cross-Reference,"** provides some tables to help you find where each exam objective is covered in the book.
  - **Appendix C, "Answers to the 'Do I Know This Already?' Quizzes,"** includes the explanations to all the "Do I Know This Already" quizzes.
  - The **Glossary** contains definitions for all the terms listed in the "Key Terms You Should Know" sections at the conclusion of the chapters.
- **Online Appendixes**
- **Practice Appendices**

  The following appendixes are available in digital format from the companion website. These appendixes provide additional practice for several networking processes that use some math.
  - **Appendix D, "Practice for Chapter 12: Analyzing Classful IPv4 Networks"**
  - **Appendix E, "Practice for Chapter 13: Analyzing Subnet Masks"**
  - **Appendix F, "Practice for Chapter 14: Analyzing Existing Subnets"**
  - **Appendix G, "Practice for Chapter 15: Subnet Design"**

- **Appendix H, "Practice for Chapter 25: Fundamentals of IP Version 6"**
- **Appendix I, "Practice for Chapter 27: Implementing IPv6 Addressing on Routers"**

- **Miscellaneous Appendices**
  - **Appendix J, "Study Planner,"** is a spreadsheet with major study milestones, where you can track your progress through your study.
- **Content from Previous Editions**

  From edition to edition, some readers have asked that we keep some select chapters with the book. Keeping content that Cisco removed from the exam, but that may still be useful, can help the average reader as well as instructors who use the materials to teach courses with this book. The following appendices hold this edition's content from previous editions:

  - **Appendix K, "Topics from Previous Editions,"** is a collection of small topics from prior editions. None of the topics justify a complete appendix by themselves, so we collect the small topics into this single appendix.
  - **Appendix L, "LAN Troubleshooting,"** examines the most common LAN switching issues and how to discover those issues when troubleshooting a network.
  - **Appendix M, "Variable-Length Subnet Masks,"** moves away from the assumption of one subnet mask per network to multiple subnet masks per network, which makes subnetting math and processes much more challenging. This appendix explains those challenges.

## About Building Hands-On Skills

To do well on the CCNA exam, you need skills in using Cisco routers and switches, specifically the Cisco command-line interface (CLI). The Cisco CLI is a text-based command-and-response user interface; you type a command, and the device (a router or switch) displays messages in response.

For the exam, CLI skills help you in a couple of ways. First, lab questions require CLI skills. Each lab question can take 7–8 minutes if you know the topic, so poor CLI skills can cost several minutes per lab question. Additionally, any question type can ask about CLI commands, so the more comfortable you are remembering commands, parameters, and what they do, the more points you will pick up on the exam.

This next section walks through the options of what is included in the book, with a brief description of lab options outside the book.

### Config Lab Exercises

I created some lab exercises called Config Labs and put them on my blog. Each Config Lab details a straightforward lab exercise. It begins with a scenario, a topology, and existing configuration. You choose the configuration to add to each device to meet the goals of the scenario.

To make the labs accessible to all, the blog has no login requirements and no cost. You can do each lab just by viewing the page, reading, and writing your answer on paper or

typing it in an editor. Optionally, you can attempt most labs in the Cisco Packet Tracer Simulator. In either case, the Config Lab page lists the intended answer, so you can check your work.

To find the Config Labs, first go to www.certskills.com. Navigate from the top menus for "Labs." Alternatively, use the advanced search link, from which you can combine search parameters to choose a book chapter or part, and to search for Config Lab posts.

Note that the blog organizes these Config Lab posts by book chapter, so you can easily use them at both Chapter Review and Part Review. See the "Your Study Plan" element that follows the Introduction for more details about those review sections.

## A Quick Start with Pearson Network Simulator Lite

The decision of how to get hands-on skills can be a little scary at first. The good news: You have a free and simple first step to experience the CLI: install a desktop simulator app called Pearson Network Simulator Lite (or NetSim Lite) that comes with this book.

Pearson builds a CCNA Simulator app designed to help you learn most of the CCNA configure and verify exam topics. They also make a free lite version of the Simulator, included with this book. The lite version gives you the means to experience the Cisco CLI just after a 5–10-minute installation process. No need to go buy real gear or buy a full simulator to start learning the CLI. Just install the Sim Lite from the companion website.

This latest version of NetSim Lite includes labs associated with Part II of this book, plus a few more from Part III. (Part II is the first book part that includes any CLI commands.) So, make sure to use the NetSim Lite to learn the basics of the CLI to get a good start.

Of course, one reason that you get access to the NetSim Lite is that the publisher hopes you will buy the full product. However, even if you do not use the full product, you can still learn from the labs that come with NetSim Lite while deciding about what options to pursue.

## The Pearson Network Simulator

The Config Labs and the Pearson Network Simulator Lite both fill specific needs, and they both come with the book. However, you need more than those two tools.

The single best option for lab work to do along with this book is the paid version of the Pearson Network Simulator. This simulator product simulates Cisco routers and switches so that you can learn for CCNA certification. But more importantly, it focuses on learning for the exam by providing a large number of useful lab exercises. Reader surveys tell us that those people who use the Simulator along with the book love the learning process and rave about how the book and Simulator work well together.

Of course, you need to make a decision for yourself and consider all the options. Thankfully, you can get a great idea of how the full Simulator product works by using the Pearson Network Simulator Lite product included with the book. Both have the same base code, same user interface, and same types of labs. Try the Lite version to decide if you want to buy the full product.

On a practical note, when you want to do labs when reading a chapter or doing Part Review, the Simulator organizes the labs to match the book. Just look for the Sort by Chapter tab in the Simulator's user interface.

At the time this book was published, Pearson had no plan to update its CCNA Simulator product to a new version, as the current edition covers the latest exam topics. A software update will be issued that maps the labs to the organization of the new Cert Guide chapter structure by the summer of 2024.

### More Lab Options

Many other lab options exist. For instance, you can use real Cisco routers and switches. You can buy them, new or used, or borrow them at work. For example, you can buy routers and switches that are useful for CCNA learning but are two or three product generations old. You can also find sites from which you can rent time on real devices or virtual devices.

Cisco also makes a free simulator that works very well as a learning tool: Cisco Packet Tracer. Unlike the Pearson Network Simulator, it does not include lab exercises that direct you as to how to go about learning each topic. However, you can usually find lab exercises that rely on Packet Tracer, like the Config Labs at my blog. If interested in more information about Packet Tracer, check out www.certskills.com/ptinstall.

Cisco offers a virtualization product that lets you run router and switch operating system (OS) images in a virtual environment on your PC. This tool, the Cisco Modeling Labs–Personal Edition (CML PE), lets you create a lab topology, start the operating system for each device, and connect to the CLI of real router and switch OS images. There is a fee, and you may need a PC hardware upgrade to use it effectively. Check out www.cisco.com/go/cml for more information, and inquire for more information at the Cisco Learning Network's CML community (learningnetwork.cisco.com).

The next two options work somewhat like CML PE, but with free software but no Cisco Operating Systems supplied. GNS3 (gns3.com) and EVE-NG (eve-ng.net) support creating topologies of virtual routers and switches that run real Cisco operating systems. Both have free options. However, both require that you provide the OS images. Also, as with CML PE, you may need to buy a small server or at least upgrade your personal computer to run more than a few routers and switches in a lab topology.

This book does not tell you what option to use, but you should plan on getting some hands-on practice somehow. For people starting with CCNA, many use some simulator like Pearson Sim Lite and the free Cisco Packet Tracer simulator. If you go far in your Cisco certification journey, you will likely try at least one of the virtualization options and also use real gear. The important thing to know is that most people need to practice using the Cisco CLI to be ready to pass these exams.

## About IP Subnetting

IP addressing and subnetting skills remain as one of the top five most important topics for real networking jobs and for the CCNA exam. This book devotes Part IV to the topic, with several practice problem appendices to match. You can learn all you need for CCNA from those in-book chapters, and you can practice with the appendices. You can even use the interactive versions of the appendices on the companion website.

Because IP subnetting is so important, I created a video course called "IP Subnetting from Beginning to Mastery LiveLessons." You can buy the course outright from Cisco Press or access the course at O'Reilly Learning (learning.oreilly.com). This video course

teaches IPv4 subnetting from start to finish. The course includes instruction on all aspects of IPv4 subnetting, many examples, and close to 100 video practice exercises.

However, to be clear, you do not need to buy the subnetting course; the book has all you need to learn subnetting well. But if you prefer to use the IP Subnetting video course, you do not have to use the course and read the chapters in this book's Part IV. Instead, use one of these plans if you get the video course:

- Learn from the first 19 lessons in the course, which cover the same content as Part IV of this book—and again ignore the chapters in Part IV.
- Study primarily from the book chapters and supplement your reading with the video course. Because the course teaches the topics in a different order, use the information in Table I-2 to decide what video course lessons to use.

**Table I-2** *Using the IP Subnetting Video Course Instead of Volume 1, Part IV*

| Volume 1 Chapter | Video Course Lessons |
|---|---|
| 11 | 1–3, 10 |
| 12 | 11, 12 |
| 13 | 8, 9 |
| 14 | 4–7 |
| 15 | 13–19 |

## For More Information

If you have any comments about the book, submit them via www.ciscopress.com. Just go to the website, select **Contact Us**, and type your message.

Cisco might make changes that affect the CCNA certification from time to time. You should always check www.cisco.com/go/ccna for the latest details.

*CCNA 200-301 Official Cert Guide*, *Volume 1*, Second Edition, helps you attain CCNA certification. This is the CCNA certification book from the only Cisco-authorized publisher. We at Cisco Press believe that this book certainly can help you achieve CCNA certification, but the real work is up to you! I trust that your time will be well spent.

*This page intentionally left blank*



# Your Study Plan

You just got this book. You have probably already read (or quickly skimmed) the Introduction. You are probably now wondering whether to start reading here or skip ahead to Chapter 1, "Introduction to TCP/IP Networking."

Stop to read this section about how to create your own study plan for the CCNA 200-301 exam. Your study will go much better if you take time (maybe 15 minutes) to think about a few key points about studying before starting this journey. That is what this section will help you do.

## A Brief Perspective on Cisco Certification Exams

Cisco sets the bar pretty high for passing the CCNA 200-301 exam. Most anyone can study and pass the exam, but it takes more than just a quick read through the book and the cash to pay for the exam.

The challenge of the exam comes from many angles. First, the exam covers many concepts and commands specific to Cisco devices. Beyond knowledge, all these Cisco exams also require deep skills. You must be able to analyze and predict what really happens in a network, configure Cisco devices to work correctly in those networks, and troubleshoot problems when the network does not work correctly.

The more challenging questions on these exams work a lot like a jigsaw puzzle, but with four out of every five puzzle pieces not even in the room. To solve the puzzle, you have to mentally re-create the missing pieces. To do that, you must know each networking concept and remember how the concepts work together.

For instance, you might encounter a question that asks you why two routers cannot exchange routing information using the OSPF routing protocol. The question would supply some of the information, like some jigsaw puzzle pieces, as represented with the white pieces in Figure 1. You have to apply your knowledge of IPv4 routing, IPv4 addressing, and the OSPF protocol to the scenario in the question to come up with some of the other pieces of the puzzle. For a given question, some pieces of the puzzle might remain a mystery, but with enough of the puzzle filled in, you should be able to answer the question. And some pieces will just remain unknown for a given question.

These skills require that you prepare by doing more than just reading and memorizing. Of course, you need to read many pages in this book to learn many individual facts and how these facts relate to each other. But a big part of this book lists exercises that require more than just simply reading, exercises that help you build the skills to solve these networking puzzles.

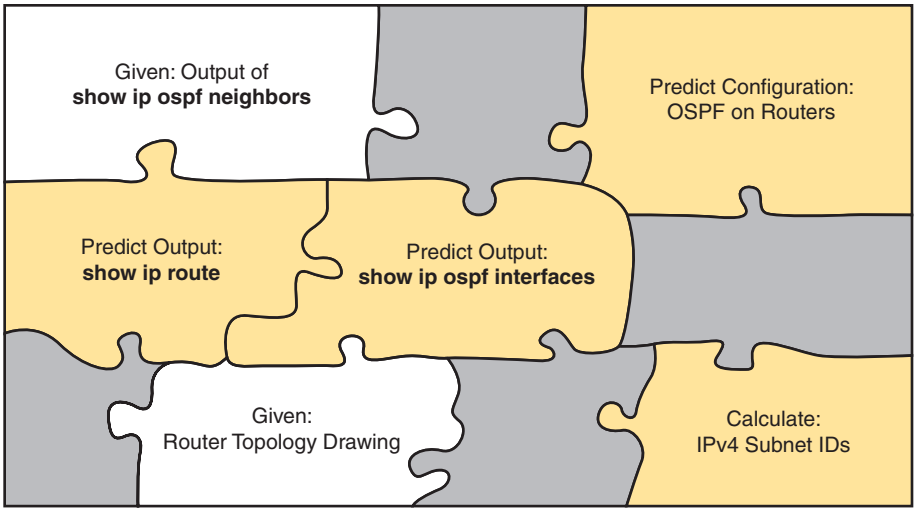



**Figure 1** *Filling In Puzzle Pieces with Your Analysis Skills*

## Five Study Plan Steps

What do you need to do to be ready to pass, beyond reading and remembering all the facts? You need to develop skills. You need to mentally link each idea with other related ideas. Doing that requires additional work. To help you along the way, the next few pages give you five key planning steps to take so that you can more effectively build those skills and make those connections, before you dive into this exciting but challenging world of learning networking on Cisco gear.

### Step 1: Think in Terms of Parts and Chapters

The first step in your study plan is to get the right mindset about the size and nature of the task you have set out to accomplish. This is a large book, and to be ready for the CCNA 200-301 exam, you need to complete it and then the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition. You cannot think about these two books as one huge task, or you might get discouraged. So, break the task down into smaller tasks.

The good news here is that the book is designed with obvious breakpoints and built-in extensive review activities. In short, the book is more of a study system than a book.

The first step in your study plan is to visualize this book not as one large book but as components. First, visualize the book as seven smaller parts. Then, within each part, visualize each part as three-to-five chapters. Your study plan has you working through the chapters and then reviewing the material in that part before moving on, as shown in Figure 2.

Now your plan has the following:

**1 large task:** Read and master all content in the book.

**7 medium tasks/book:** Read and master a part.

**4 small tasks/part:** Read and master a chapter.

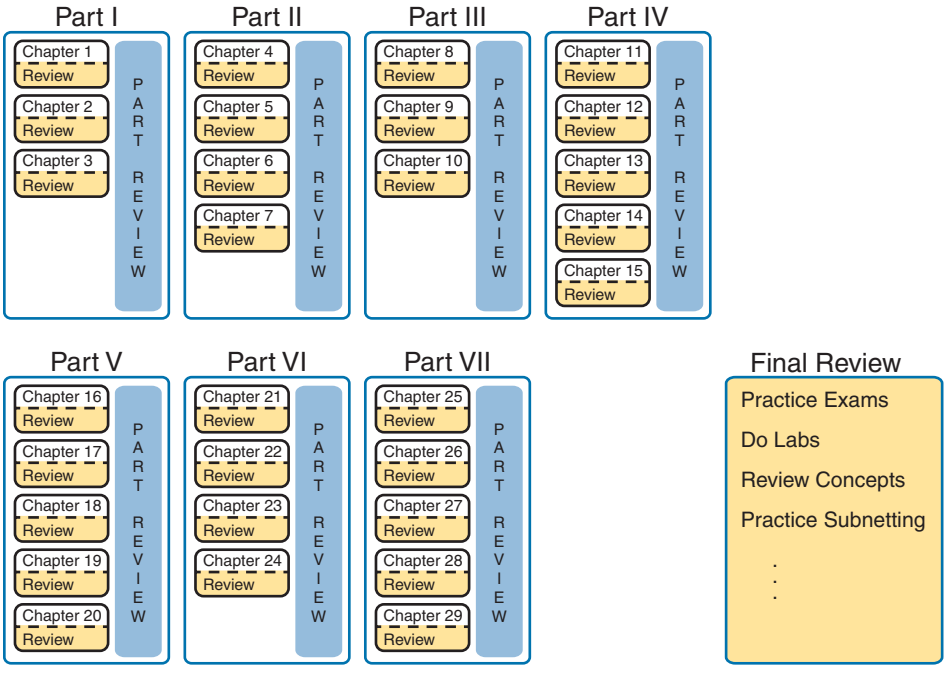



**Figure 2** *Seven Parts, with an Average of Four Chapters Each, with Part Reviews*

## Step 2: Build Your Study Habits Around the Chapter

For your second step, possibly the most important step, approach each chapter with the same process as shown in Figure 3. The chapter pre-quiz (called a DIKTA quiz, or "Do I Know This Already?" quiz) helps you decide how much time to spend reading versus skimming the core of the chapter, called the "Foundation Topics." The "Chapter Review" section then gives instructions on studying and reviewing what you just read.

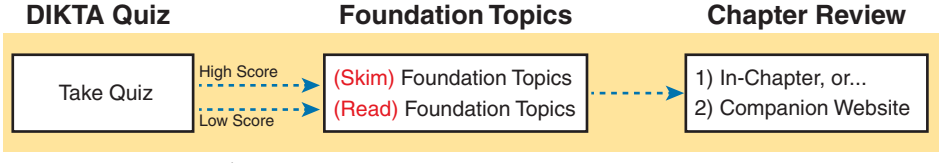



**Figure 3** *Suggested Approach to Each Chapter*

The book has no long chapters, on purpose. They average about 20 pages for the Foundation Topics (which is the part of the chapter with new content). Because we keep the size reasonable, you can complete all of a chapter in one or two short study sessions. For instance, when you begin a new chapter, if you have an hour or an hour and a half, you should be able to complete a first reading of the chapter and at least make a great start on it. And even if you do not have enough time to read the entire chapter, look for the major headings inside the chapter; each chapter has two to three major headings, and those make a great place to stop reading when you need to wait to complete the reading in the next study sessions.

The Chapter Review tasks are very important to your exam-day success. Doing these tasks after you've read the chapter really does help you get ready. Do not ignore Chapter Review! The chapter-ending review tasks help you with the first phase of deepening your knowledge and skills of the key topics, remembering terms, and linking the concepts together in your brain so that you can remember how it all fits together. The following list describes most of the activities you will find in the "Chapter Review" sections:

- Review key topics
- Review key terms
- Answer the DIKTA questions
- Re-create config checklists
- Review command tables
- Review memory tables
- Do lab exercises
- Watch video
- Do subnetting exercises

## Step 3: Use Book Parts for Major Milestones

Studies show that to master a concept and/or skill, you should plan to go through multiple study sessions to review the concept and to practice the skill. The "Chapter Review" section at the end of each chapter is the first such review, while the Part Review acts as that second review at the end of each part.

Plan time to do the Part Review task at the end of each part, using the Part Review elements found at the end of each part. You should expect to spend about as much time on one Part Review as you would on one entire chapter. So, in terms of planning your time, think of the Part Review itself as another chapter.

Figure 4 lists the names of the parts in this book. Note that Parts II and III are related (Ethernet), and Parts IV through VI discuss IP version 4 (IPv4). Part VII then moves on to IP Version 6 (IPv6). Each part ends with a Part Review section of two to four pages, with notes about what tools and activities to use.

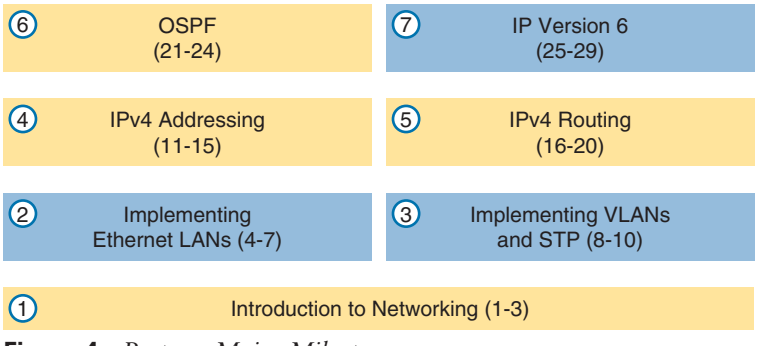



**Figure 4** *Parts as Major Milestones*

Also, consider setting a goal date for finishing each part of the book (and a reward, as well). Plan a break, some family time, some time out exercising, eating some good food, whatever helps you get refreshed and motivated for the next part.

## Step 4: Use Volume 2's Final Review Chapter

Your fourth step has one overall task: perform the details outlined in the "Final Exam Review" chapter at the end of the *CCNA 200-301 Official Cert Guide*, *Volume 2*, Second Edition. Note that you have no exam to take at the end of this Volume 1 book, so keep working with Volume 2 when you complete this book. Once you're finished with both books, Volume 2's "Final Exam Review" will direct you.

## Step 5: Set Goals and Track Your Progress

Your fifth study plan step spans the entire timeline of your study effort. Before you start reading the book and doing the rest of these study tasks, take the time to make a plan, set some goals, and be ready to track your progress.

While making lists of tasks may or may not appeal to you, depending on your personality, goal setting can help everyone studying for these exams. And to do the goal setting, you need to know what tasks you plan to do.

As for the list of tasks to do when studying, you do not have to use a detailed task list. (You could list every single task in every chapter-ending "Chapter Review" section, every task in the Part Reviews, and every task in the "Final Review" chapter.) However, listing the major tasks can be enough.

You should track at least two tasks for each typical chapter: reading the "Foundation Topics" section and doing the Chapter Review at the end of the chapter. And, of course, do not forget to list tasks for Part Reviews and Final Review. Table 1 shows a sample for Part I of this book.

**Table 1** Sample Excerpt from a Planning Table

| Element | Task | Goal Date | First Date Completed | Second Date Completed (Optional) |
|---|---|---|---|---|
| Chapter 1 | Read Foundation Topics | | | |
| Chapter 1 | Do Chapter Review tasks | | | |
| Chapter 2 | Read Foundation Topics | | | |
| Chapter 2 | Do Chapter Review tasks | | | |
| Chapter 3 | Read Foundation Topics | | | |
| Chapter 3 | Do Chapter Review tasks | | | |
| Part I Review | Do Part Review activities | | | |

**NOTE** Appendix J, "Study Planner," on the companion website, contains a complete planning checklist like Table 1 for the tasks in this book. This spreadsheet allows you to update and save the file to note your goal dates and the tasks you have completed.

Use your goal dates to manage your study, and not as a way to get discouraged if you miss a date. Pick reasonable dates that you can meet. When setting your goals, think about how fast you read and the length of each chapter's "Foundation Topics" section, as listed in the table of contents. Then, when you finish a task sooner than planned, move up the next few goal dates.

If you miss a few dates, do *not* start skipping the tasks listed at the ends of the chapters! Instead, think about what is impacting your schedule—real life, commitment, and so on—and either adjust your goals or work a little harder on your study.

## Things to Do Before Starting the First Chapter

Now that you understand the big ideas behind a good study plan for the book, take a few more minutes for a few overhead actions that will help. Before leaving this section, look at some other tasks you should do either now or around the time you read the first few chapters to help make a good start in the book.

### Bookmark the Companion Website

The companion website contains links to all the tools you need for chapter and part review. In fact, it includes a chapter-by-chapter and part-by-part breakdown of all the review activities. Before you finish the first chapter, make sure and follow the instructions in the Introduction's section titled "The Companion Website for Online Content," get access, and bookmark the page.

Also, if you did not yet read about the companion website in the Introduction or explore the site, take a few minutes to look at the resources available on the site.

### Bookmark/Install Pearson Test Prep

Like many other Cisco Press books, this book includes the rights to use the Pearson Test Prep (PTP) software and the rights to use some exam questions related to this book. PTP has many useful study features:

- Both a web and desktop version for your convenience and choice
- History tracking of your simulated exam attempts, synchronized between web and desktop
- Study mode, which lets you see the correct answers with each question and the related explanations
- Practice exam mode, which simulates exam conditions, hiding answers/explanations and timing the exam event
- Filters to let you choose questions based on chapter(s) and/or part(s)

You should take a few minutes to set up your PTP installation. Refer to the Introduction's section titled "How to Access the Pearson Test Prep (PTP) App" for details.

### Understand This Book's PTP Databases and Modes

When you activate a product in PTP, you gain the rights to that product's exams. Understanding those exams helps you choose when to use them and when to delay using different exams to save those questions for later. The publisher includes different exams with different products, so this section works through the various options with you.

### Exams in the Retail (Print) Editions

The retail edition—that is, the printed book—of Volume 1 comes with four exams, as shown on the left side of Figure 5. Volume 2 comes with four similar exams, plus four exams described as CCNA exams. If you buy the retail edition of the CCNA Library, you receive all the exams shown in the figure.

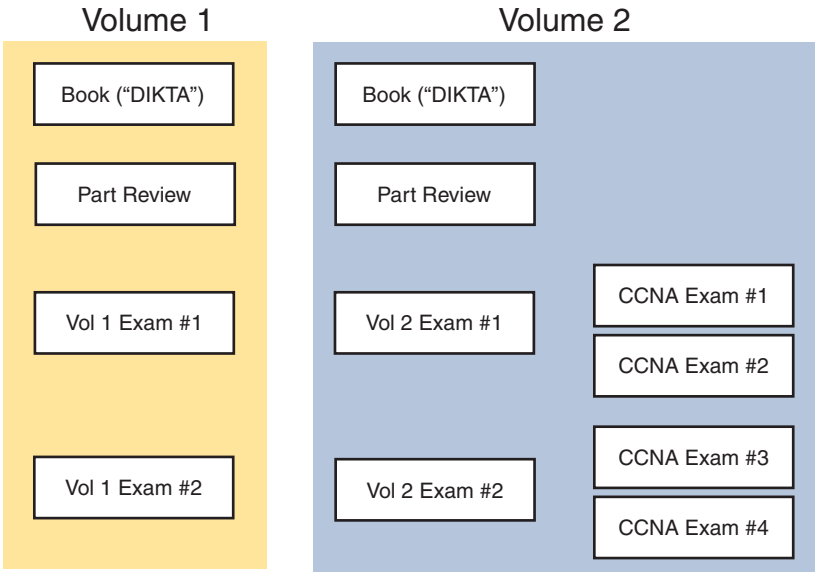



**Figure 5** *PTP Exams/Exam Databases with the Retail (Print) Editions*

First, note that the exams found in both volumes contain only questions about that volume and not the other volume. For instance, the Part Review exam in Volume 1 has questions only about Volume 1, while the Part Review exam in Volume 2 has questions only about Volume 2. The CCNA exams have questions across the breadth of both books.

Use the book exams during chapter review for each chapter. Each chapter begins with a short pre-chapter quiz called a “Do I Know This Already?” or DIKTA quiz. You will likely read the DIKTA quizzes inside each chapter without using the PTP app; however, when reviewing the chapter after reading it, answer that chapter’s DIKTA questions again using the PTP app. (The app refers to the exam with all that volume’s DIKTA questions as the book exam.)

Use the Part Review exam during part review. Unlike the book exams, these questions exist only inside a PTP exam. In fact, we created them specifically as a review exercise at the end of each book part.

Use Volume 1 Exams 1 and 2 whenever you like. Those exams (and the similarly named exams for Volume 2) contain more unique questions, usually a little more than 90 per exam. Each question has an assigned objective, which matches the most relevant chapter. From the PTP app, you can use PTP questions for any combination of chapters, with any combination of these exams, for an impromptu study session. For example, after finishing the first 10 chapters of Volume 1, you could select both these exams, Chapters 1–10, and answer those questions.

Use the exams called CCNA exams for timed practice exams. Volume 2 (but not Volume 1) includes four CCNA exams. Only the CCNA exams contain questions from Volumes 1 and 2, built to provide individual exams useful for a practice exam event. When ready, pick any CCNA exam (a little more than 90 questions), use them as a practice exam, and give yourself 120 minutes. You build your test-taking skills with time pressure, and later, you can return to the results, identify missed questions, and dig in to learn those topics.

Be aware that the exams labeled Volume X Exam Y contain the same questions used to create the CCNA exams. The questions in the CCNA exams come from the Volume 1 and 2 exams, as shown in Figure 6.

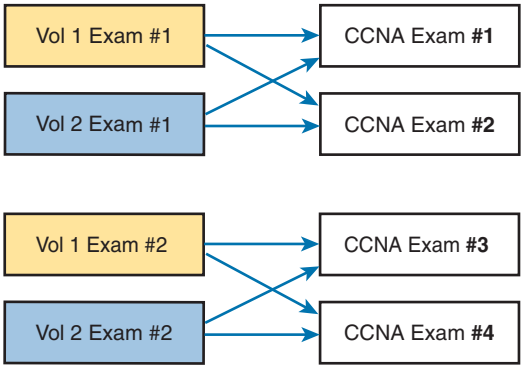



**Figure 6** *Source of Questions for the CCNA Exams*

Some readers do not care about the overlaps shown in Figure 6, but others do. Those who do care want to protect their timed practice exam events so that they do not see those questions before taking a timed practice exam event. If that sounds like you, choose now which exams to use and which to avoid using.

- If you do not plan on using timed practice exam events, use Volume 1 exams 1 and 2 plus Volume 2 exams 1 and 2, at any time, and ignore the CCNA exams.
- If you plan to use all four CCNA exams for timed practice exams, avoid using Volume 1 exams 1 and 2 plus Volume 2 exams 1 and 2, to avoid seeing the questions before the timed events.
- Split the difference. Use Volume 1 exam 1 and Volume 2 Exam 1 at any time. Later, use CCNA exams 3 and 4.

I recommend using study mode in all cases except for the timed practice exam, as follows:

**Study mode:** Study mode works best when you are still working on understanding and learning the content. In study mode, you can see the answers immediately, so you can study the topics more easily. Study mode also shows an "answer" button that reveals an explanation to each question, so you can get immediate insights into questions you miss.

**Practice mode:** This mode lets you practice an exam event somewhat like the actual exam. It gives you a preset number of questions from all chapters with a timed event. Practice exam mode also gives you a score for that timed event.

## Exams with Individual Premium Edition eBooks

The publisher sells these related Premium Edition eBook and Practice Test products:

- *CCNA 200-301 Official Cert Guide, Volume 1 Premium Edition and Practice Test*, Second Edition
- *CCNA 200-301 Official Cert Guide, Volume 2 Premium Edition and Practice Test*, Second Edition
- *CCNA 200-301 Official Cert Guide Library Premium Edition eBook and Practice Test*, Second Edition

With the retail editions, purchasing the two volumes individually gives you the same PTP exams as purchasing the retail CCNA Library, which is a box with both volumes. With the Premium Edition eBooks, you see different exams depending on whether you buy each volume separately or as a two-book library. This section examines what you receive when you buy both individual volumes, with the following section detailing the exams that come with the Premium Edition Library.

### Exams with Two Individual Premium Edition eBooks

If you purchase the Premium Edition eBook products for both Volumes 1 and 2, you get each in various formats (PDF, ePub). Also, owning the eBooks enables PTP to link all questions' explanations to a book page that gives more detail about the topics—a great study feature. So, if you are confused by an answer, just click that link in that question's explanation to open the PDF and read more.

As for the PTP exams, you receive all the exams shown earlier in Figure 5, plus additional exams. The phrase "…and Practice Test" at the end of the long formal name of the products refers to the additional practice questions. Figure 7 shows the additional exams you receive if you buy both the Volume 1 and Volume 2 Premium Edition products, adding more than 350 additional unique questions.

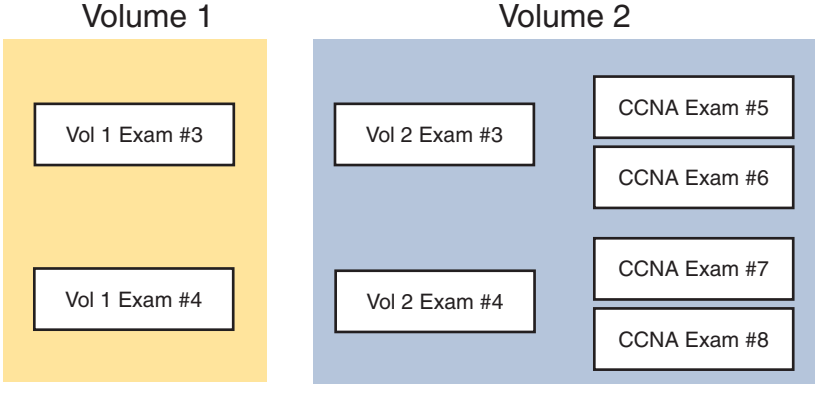



**Figure 7** *Exams Added: Volume 1 and Volume 2 Premium Edition vs. Print Edition*

As with the other CCNA exams listed in Figure 6, the questions in CCNA exams 5–8 come from the Volume X Exam Y exams in Figure 7. Volume 1 and 2 exams 3 feed into CCNA exams 5 and 6, with Volume 1 and 2 exams 4 feeding into CCNA exams 7 and 8. So, you

have the same choices to make about avoiding using some exams—just with many more questions to use.

## Exams with CCNA Premium Edition Library

If you do not yet own any Premium Edition eBooks but want to choose, the CCNA Premium Edition Library version makes more sense for most people. It is cheaper than buying both Volume 1 and Volume 2 Premium Editions. It also includes the same questions as when purchasing the two individual products, but organizes the questions differently.

The CCNA Premium Edition Library has the book and part review exams for both volumes—the same exams as the other options. Plus, it has the same eight CCNA exams as seen in the other options, with content spread across the breadth of CCNA exam topics, which you can use at any time, in study mode or practice exam mode, as shown in Figure 8.

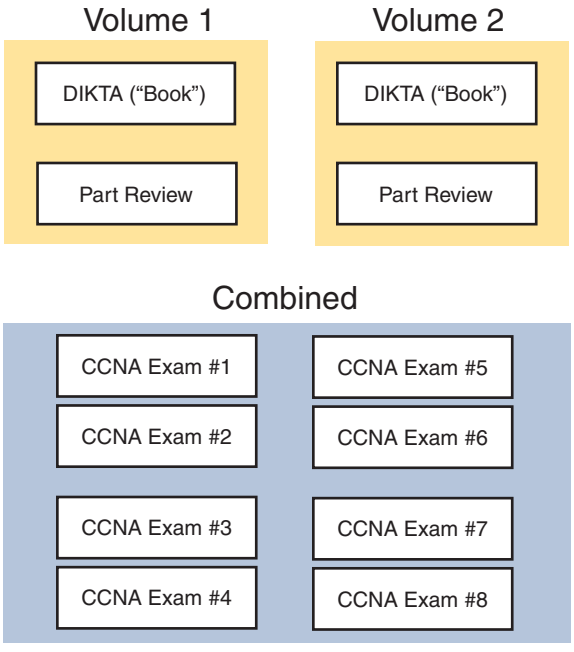



**Figure 8** *PTP Exams/Exam Databases with CCNA Library Premium Edition*

The CCNA Premium Edition Library includes all the same questions you would get if you bought the individual Volume 1 and Volume 2 Premium Edition products—it just doesn't have the exams labeled Vol 1 Exam N and Vol 2 Exam N. You do not lose any function, but be more thoughtful about using the exams. For example, you can use some CCNA exams in study mode for review any time. For example, you could take this approach:

- Use CCNA exams 1–5 in study mode throughout the study experience for extra practice during chapter review, part review, or any time.
- Reserve CCNA exams 6–8 for timed practice exam events at the end of your study.

If you have this product, take the time right now to choose an approach. How many timed practice CCNA exams do you expect to take? Choose which CCNA exams to reserve for

that purpose and use the other CCNA exams in study mode. Then, experiment with the PTP app to get comfortable choosing the exams, the objectives, and the mode (study or practice exam).

If you like practice questions when studying, you should have enough. The retail product exams, per Figure 5, have around 900 unique questions. Adding the CCNA Premium Edition Library bumps that to over 1200 unique questions.

### Practice Viewing Per-Chapter Book (DIKTA) Questions

Take a few minutes to experiment with and understand how to use PTP to answer questions from a single chapter's book quiz, as follows:

**Step 1.** Start the PTP web or desktop app.

**Step 2.** From the main (home) menu, select the item for this product, with a name like *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, and click **Open Exam**.

**Step 3.** The top of the next window that appears should list some exams. Check the **Book Questions** box, and uncheck the other boxes. This selects the "book" questions (that is, the DIKTA questions from the beginning of each chapter).

**Step 4.** On this same window, click at the bottom of the screen to deselect all objectives (chapters). Then select the box beside each chapter in the part of the book you are reviewing.

**Step 5.** Select any other options on the right side of the window.

**Step 6.** Click **Start** to start reviewing the questions.

### Practice by Using Per-Part Review Questions

Your PTP access also includes a Part Review exam created solely for study during the Part Review process. To view these questions, follow the same process as you did with DIKTA/book questions, but select the Part Review database rather than the book database. PTP has a clear name for this database: Part Review Questions.

### Join the Cisco Learning Network CCNA Community

Register (for free) at learningnetwork.cisco.com (the Cisco Learning Network, or CLN) and join the CCNA community. This group allows you to both lurk and participate in discussions about topics related to the CCNA exam. Register (for free), join the group, and set up an email filter to redirect the messages to a separate folder. Even if you do not spend time reading all the posts yet, later, when you have time to read, you can browse through the posts to find interesting topics (or just search the posts).

Also, the CLN CCNA community, along with my blog and YouTube channel, are the best places to find me online.

## Getting Started: Now

Now dive in to your first of many short, manageable tasks: reading the relatively short Chapter 1. Enjoy!

*This page intentionally left blank*



This first part of the book introduces the fundamentals of the most important topics in TCP/IP networking. Chapter 1 provides a broad look at TCP/IP, introducing the common terms, big concepts, and major protocols for TCP/IP. Chapter 2 then examines local-area networks (LANs), which are networks that connect devices that are located near to each other, for instance, in the same building. Chapter 3 then shows how to connect those LANs across long distances with wide-area networks (WANs) with a focus on how routers connect LANs and WANs to forward data between any two devices in the network.

# Part I

## Introduction to Networking

**Chapter 1:** Introduction to TCP/IP Networking

**Chapter 2:** Fundamentals of Ethernet LANs

**Chapter 3:** Fundamentals of WANs and IP Routing

**Part I Review**



CHAPTER 1

# Introduction to TCP/IP Networking

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.3 Compare physical interface and cabling types**
    - **1.3.a Single-mode fiber, multimode fiber, copper**
    - **1.3.b Connections (Ethernet shared media and point-to-point)**

Welcome to the first chapter in your study for CCNA! This chapter begins Part I, which focuses on the basics of networking.

Networks work correctly because the various devices and software follow the rules. Those rules come in the form of standards and protocols, which are agreements of a particular part of how a network should work. However, the sheer number of standards and protocols available can make it difficult for the average network engineer to think about and work with networks—so the world of networking has used several networking models over time. Networking models define a structure and different categories (layers) of standards and protocols. As new standards and protocols emerge over time, networkers can think of those new details in the context of a working model.

You can think of a networking model as you think of a set of architectural plans for building a house. A lot of different people work on building your house, such as framers, electricians, bricklayers, painters, and so on. The blueprint helps ensure that all the different pieces of the house work together as a whole. Similarly, the people who make networking products, and the people who use those products to build their own computer networks, follow a particular networking model. That networking model defines rules about how each part of the network should work, as well as how the parts should work together so that the entire network functions correctly.

Today, TCP/IP rules as the most pervasive networking model in use. You can find support for TCP/IP on practically every computer operating system (OS) in existence today, from mobile phones to mainframe computers. Every network built using Cisco products today supports TCP/IP. And not surprisingly, the CCNA exam focuses heavily on TCP/IP. This chapter uses TCP/IP for one of its main purposes: to present various concepts about networking using the context of the different roles and functions in the TCP/IP model.

Note that most chapters cover topics about some specific CCNA exam topic. However, this chapter does not. Instead, it describes background information about the TCP/IP model and ideas you need to know about so you can better understand the detail included in CCNA.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 1-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Perspectives on Networking | None |
| TCP/IP Networking Model | 1–4 |
| Data Encapsulation Terminology | 5–6 |

1. Which of the following protocols are examples of TCP/IP transport layer protocols? (Choose two answers.)
   - **a.** Ethernet
   - **b.** HTTP
   - **c.** IP
   - **d.** UDP
   - **e.** SMTP
   - **f.** TCP
2. Which of the following protocols are examples of TCP/IP data-link layer protocols? (Choose two answers.)
   - **a.** Ethernet
   - **b.** HTTP
   - **c.** IP
   - **d.** UDP
   - **e.** SMTP
   - **f.** TCP
   - **g.** 802.11
3. The process of HTTP asking TCP to send some data and making sure that it is received correctly is an example of what?
   - **a.** Same-layer interaction
   - **b.** Adjacent-layer interaction
   - **c.** TCP/IP model
   - **d.** All of these answers are correct.
4. The process of TCP on one computer marking a TCP segment as segment 1 and the receiving computer then acknowledging the receipt of TCP segment 1 is an example of what?
   - **a.** Data encapsulation
   - **b.** Same-layer interaction

**c.** Adjacent-layer interaction

**d.** TCP/IP model

**e.** All of these answers are correct.

**5.** The process of a web server adding a TCP header to the contents of a web page, followed by adding an IP header and then adding a data-link header and trailer, is an example of what?

**a.** Data encapsulation

**b.** Same-layer interaction

**c.** TCP/IP model

**d.** All of these answers are correct.

**6.** Which of the following terms is used specifically to identify the entity created when encapsulating data inside data-link layer headers and trailers?

**a.** Data

**b.** Chunk

**c.** Segment

**d.** Frame

**e.** Packet

# Foundation Topics

## Perspectives on Networking

So, you are new to networking. If you're like many people, your perspective about networks might be that of a user of the network, as opposed to the network engineer who builds networks. For some, your view of networking might be based on how you use the Internet, from home, using a high-speed Internet connection like fiber Ethernet or cable TV, as shown in Figure 1-1.

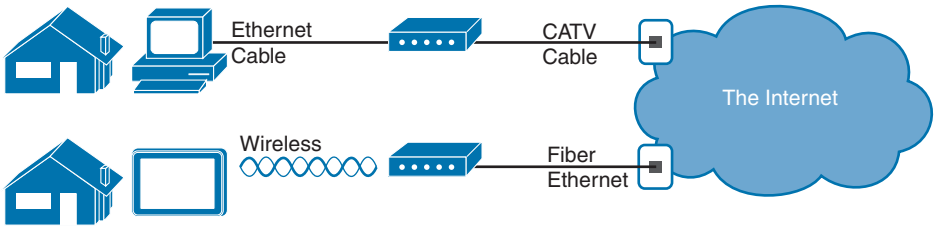



**Figure 1-1** *End-User Perspective on High-Speed Internet Connections*

The top part of the figure shows a typical high-speed cable Internet user. The PC connects to a cable modem using an Ethernet cable. The cable modem then connects to a cable TV (CATV) outlet in the wall using a round coaxial cable—the same kind of cable used to connect your TV to the CATV wall outlet. Because cable Internet services provide service continuously, the user can just sit down at the PC and start sending email, browsing websites, making Internet phone calls, and using other tools and applications.

The lower part of the figure uses Ethernet between the home and service provider. First, the tablet computer uses wireless technology that goes by the name *wireless local-area*

*network (wireless LAN)*. In this example, the router uses a different technology, Ethernet, using a fiber-optic cable, to communicate with the Internet.

Both home-based networks and networks built for use by a company make use of similar networking technologies. The Information Technology (IT) world refers to a network created by one corporation, or enterprise, for the purpose of allowing its employees to communicate, as an *enterprise network*. The smaller networks at home, when used for business purposes, often go by the name small office/home office (SOHO) networks.

Users of enterprise networks have some idea about the enterprise network at their company or school. People realize that they use a network for many tasks. PC users might realize that their PC connects through an Ethernet cable to a matching wall outlet, as shown at the top of Figure 1-2. Those same users might use wireless LANs with their laptop when going to a meeting in the conference room as well. Figure 1-2 shows these two end-user perspectives on an enterprise network.

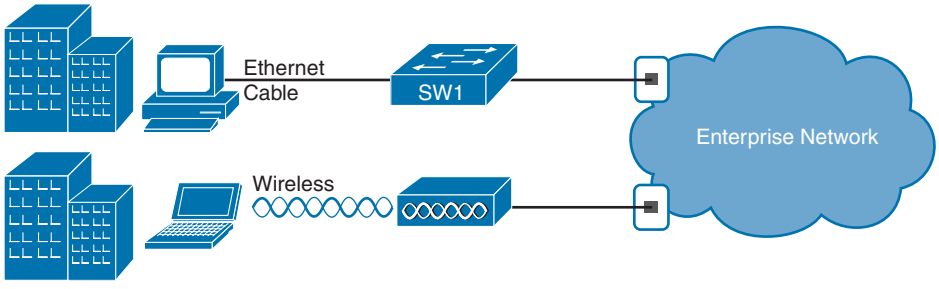



**Figure 1-2** *Example Representation of an Enterprise Network*

**NOTE** In networking diagrams, a cloud represents a part of a network whose details are not important to the purpose of the diagram. In this case, Figure 1-2 ignores the details of how to create an enterprise network.

Some users might not even have a concept of the network at all. Instead, these users just enjoy the functions of the network—the ability to post messages to social media sites, make phone calls, search for information on the Internet, listen to music, and download countless apps to their phones—without caring about how it works or how their favorite device connects to the network.

Regardless of how much you already know about how networks work, this book and the related certification help you learn how networks do their job. That job is simply this: moving data from one device to another. The rest of this chapter—and the rest of this first part of the book—reveals the basics of how to build enterprise networks so that they can deliver data between two devices.

## TCP/IP Networking Model

A **networking model**, sometimes also called either a *networking architecture* or *networking blueprint*, refers to a comprehensive set of documents. Individually, each document describes one small function required for a network; collectively, these documents define

everything that should happen for a computer network to work. Some documents define a *protocol*, which is a set of logical rules that devices must follow to communicate. Other documents define some physical requirements for networking. For example, a document could define the voltage and current levels used on a particular cable when transmitting data.

You can think of a networking model as you think of an architectural blueprint for building a house. Sure, you can build a house without the blueprint. However, the blueprint can ensure that the house has the right foundation and structure so that it will not fall down, and it has the correct hidden spaces to accommodate the plumbing, electrical, gas, and so on. Also, the many different people who build the house using the blueprint—such as framers, electricians, bricklayers, painters, and so on—know that if they follow the blueprint, their part of the work should not cause problems for the other workers.

Similarly, you could build your own network—write your own software, build your own networking cards, and so on—to create a network. However, it is much easier to simply buy and use products that already conform to some well-known networking model or blueprint. Because the networking product vendors build their products with some networking model in mind, their products should work well together.

## History Leading to TCP/IP

Today, the world of computer networking uses one networking model: TCP/IP. However, the world has not always been so simple. Once upon a time, networking protocols didn't exist, including TCP/IP. Vendors created the first networking protocols; these protocols supported only that vendor's computers.

For example, IBM, the computer company with the largest market share in many markets back in the 1970s and 1980s, published its Systems Network Architecture (SNA) networking model in 1974. Other vendors also created their own proprietary networking models. As a result, if your company bought computers from three vendors, network engineers often had to create three different networks based on the networking models created by each company, and then somehow connect those networks, making the combined networks much more complex. The left side of Figure 1-3 shows the general idea of what a company's enterprise network might have looked like back in the 1980s, before TCP/IP became common in enterprise internetworks.

Although vendor-defined proprietary networking models often worked well, having an open, vendor-neutral networking model would aid competition and reduce complexity. The International Organization for Standardization (ISO) took on the task to create such a model, starting as early as the late 1970s, beginning work on what would become known as the Open Systems Interconnection (OSI) networking model. ISO had a noble goal for the OSI model: to standardize data networking protocols to allow communication among all computers across the entire planet. ISO worked toward this ambitious and noble goal, with participants from most of the technologically developed nations on Earth participating in the process.

---

Answers to the "Do I Know This Already?" quiz:

**1** D and F **2** A and G **3** B **4** B **5** A **6** D

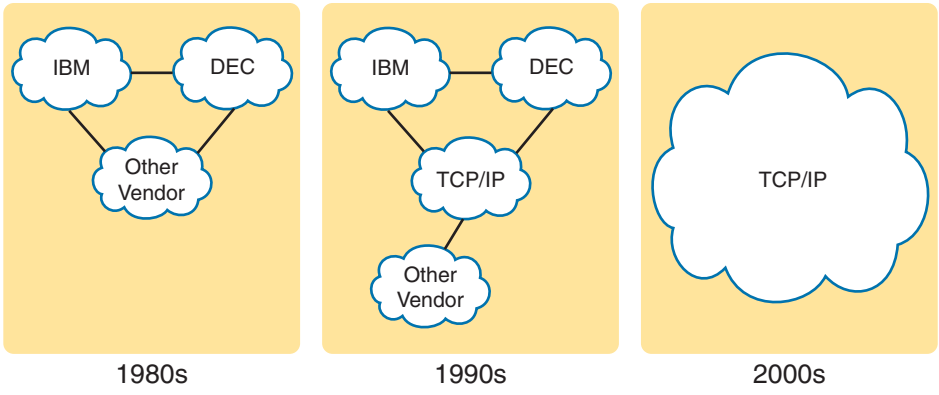



**Figure 1-3** *Historical Progression: Proprietary Models to the Open TCP/IP Model*

A second, less-formal effort to create an open, vendor-neutral, public networking model sprouted forth from a U.S. Department of Defense (DoD) contract. Researchers at various universities volunteered to help further develop the protocols surrounding the original DoD work. These efforts resulted in a competing open networking model called TCP/IP.

During the 1990s, companies began adding OSI, TCP/IP, or both to their enterprise networks. However, by the end of the 1990s, TCP/IP had become the common choice, and OSI fell away. The center part of Figure 1-3 shows the general idea behind enterprise networks in that decade—still with networks built upon multiple networking models but including TCP/IP.

Here in the twenty-first century, TCP/IP dominates. Some proprietary networking models still exist, but they have mostly been discarded in favor of TCP/IP. The OSI model, whose development suffered in part due to a standard-first-code-second approach, never succeeded in the marketplace. And TCP/IP, the networking model originally created almost entirely by a bunch of volunteers, with a code-first-standardize-second approach, has become the most prolific networking model ever, as shown on the right side of Figure 1-3.

In this chapter, you will read about some of the basics of TCP/IP. Although you will learn some interesting facts about TCP/IP, the true goal of this chapter is to help you understand what a networking model or networking architecture really is and how it works.

Also in this chapter, you will learn about some of the jargon used with OSI. Will any of you ever work on a computer that is using the full OSI protocols instead of TCP/IP? Probably not. However, you will often use terms relating to OSI.

## Overview of the TCP/IP Networking Model

The TCP/IP model both defines and references a large collection of protocols that allow computers to communicate. To define a protocol, TCP/IP uses documents called *Requests For Comments (RFC)*. (You can find these RFCs using any online search engine.) Each layer broadly defines a set of functions that helps create a working communication system, and each RFC gives the specifics about an option to implement one or more of the functions at some layer of the model.

The TCP/IP model also avoids repeating work already done by some other standards body or vendor consortium by simply referring to standards or protocols created by those groups.

For example, the Institute of Electrical and Electronics Engineers (IEEE) defines Ethernet LANs; the TCP/IP model does not define Ethernet in RFCs, but refers to IEEE Ethernet as an option.

To help people understand a networking model, each model breaks the functions into a small number of categories called *layers*. Each layer includes protocols and standards that relate to that category of functions, as shown in Figure 1-4.

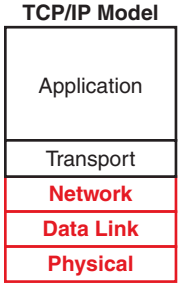



**Figure 1-4** *The TCP/IP Networking Model*

**NOTE** The network layer, shown as the middle layer of the TCP/IP model in Figure 1-4, may also be called the *Internet* layer in reference to its primary protocol, the Internet Protocol (IP).

On a single computer, different components implement different protocols and standards from different layers. Imagine buying a new computer and connecting it to either your home network or network at work using a cable or wireless. You would expect that you can open a web browser and connect to a website without much more effort. To make all that work, the network hardware in your computer—an integrated network interface card (NIC)—implements some physical layer standards to support physical communications. That NIC also supports the related data-link standards. The OS on the computer implements protocols from the network and transport layers. Finally, that web browser implements some application layer protocols (for instance, HTTP or HTTPS.)

More generally, the TCP/IP model in Figure 1-4 shows the more common terms and layers used when people talk about TCP/IP today. The physical layer focuses on how to transmit bits over each link. The data-link layer focuses on the rules that control the use of the physical link, analogous to how we need standards for roads, cars, and traffic signals. The network layer focuses on delivering data over the entire path from the original sending computer to the final destination computer, analogous to how a national postal service arranges for unique postal addresses and a system to deliver mail to all those addresses. The top two layers focus more on the applications that need to send and receive data—for instance, how to identify data, how to ask for the data to be sent, and how to recover the data if lost in transmission.

Many of you will have already heard of several TCP/IP protocols, like the examples listed in Table 1-2. Most of the protocols and standards in this table will be explained in more detail as you work through this book. Following the table, this section takes a closer look at the layers of the TCP/IP model.

**Table 1-2** TCP/IP Architectural Model and Example Protocols

| TCP/IP Architecture Layer | Example Protocols |
|---|---|
| Application | HTTPS, POP3, SMTP |
| Transport | TCP, UDP |
| Network | IP, ICMP |
| Data Link & Physical | Ethernet, 802.11 (Wi-Fi) |

## TCP/IP Application Layer

TCP/IP application layer protocols provide services to the application software running on a computer. The application layer does not define the application itself, but it defines services that applications need. For example, application protocol HTTP defines how web browsers can pull the contents of a web page from a web server. In short, the application layer provides an interface between software running on a computer and the network itself.

Arguably, the most popular TCP/IP application today is the web browser. Many major software vendors either have already changed or are changing their application software to support access from a web browser. And thankfully, using a web browser is easy: You start a web browser on your computer and select a website by typing the name of the website, and the web page appears.

### HTTP Overview

What really happens to allow that web page to appear on your web browser?

Imagine that Bob opens his browser. His browser has been configured to automatically ask for web server Larry's default web page, or *home page*. The general logic looks like Figure 1-5.

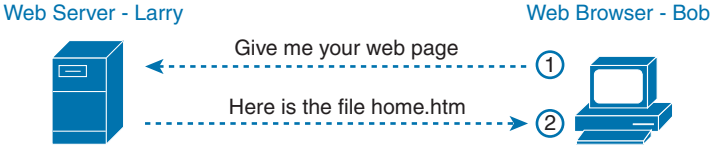



**Figure 1-5** *Basic Application Logic to Get a Web Page*

So, what really happened? Bob's initial request actually asks Larry to send his home page back to Bob. Larry's web server software has been configured to know that the default web page is contained in a file called home.htm. Bob receives the file from Larry and displays the contents of the file in Bob's web browser window.

### HTTP Protocol Mechanisms

Taking a closer look, this example shows how applications on each endpoint computer—specifically, the web browser application and web server application—use a TCP/IP application layer protocol. To make the request for a web page and return the contents of the web page, the applications use the Hypertext Transfer Protocol (HTTP).

HTTP did not exist until Tim Berners-Lee created the first web browser and web server in the early 1990s. Berners-Lee gave HTTP functionality to ask for the contents of web pages, specifically by giving the web browser the ability to request files from the server and giving

the server a way to return the content of those files. The overall logic matches what was shown in Figure 1-5; Figure 1-6 shows the same idea, but with details specific to HTTP.

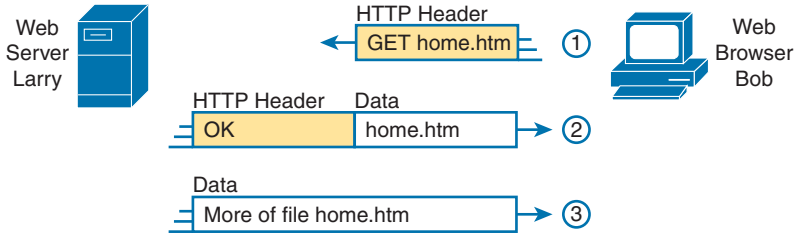



**Figure 1-6** *HTTP GET Request, HTTP Reply, and One Data-Only Message*

**NOTE** The full version of most web addresses—also called Uniform Resource Locators (URL) or Universal Resource Identifiers (URI)—begins with the letters *http*, which means that HTTP is used to transfer the web pages.

To get the web page from Larry, at Step 1, Bob sends a message with an HTTP header. Generally, protocols use headers as a place to put information used by that protocol. This HTTP header includes the request to "get" a file. The request typically contains the name of the file (home.htm, in this case), or if no filename is mentioned, the web server assumes that Bob wants the default web page.

Step 2 in Figure 1-6 shows the response from web server Larry. The message begins with an HTTP header, with a return code (200), which means something as simple as "OK" returned in the header. HTTP also defines other return codes so that the server can tell the browser whether the request worked. (Here is another example: If you ever looked for a web page that was not found, and then received an HTTP 404 "not found" error, you received an HTTP return code of 404.) The second message also includes the first part of the requested file.

Step 3 in Figure 1-6 shows another message from web server Larry to web browser Bob, but this time without an HTTP header. HTTP transfers the data by sending multiple messages, each with a part of the file. Rather than wasting space by sending repeated HTTP headers that list the same information, these additional messages simply omit the header.

Chapter 5 in *CCNA 200-301 Official Cert Guide, Volume 2*, revisits both HTTP going into more depth about HTTP, secure HTTP, and the various versions of HTTP.

## TCP/IP Transport Layer

Although many TCP/IP application layer protocols exist, the TCP/IP transport layer includes a smaller number of protocols. The two most commonly used transport layer protocols are the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP).

Transport layer protocols provide services to the application layer protocols that reside one layer higher in the TCP/IP model. How does a transport layer protocol provide a service to a higher-layer protocol? This section introduces that general concept by focusing on a single service provided by TCP: error recovery. The *CCNA 200-301 Official Cert Guide*, *Volume 2*, Second Edition includes a chapter, "Introduction to TCP/IP Transport and Applications," which examines the transport layer in more detail.

## TCP Error Recovery Basics

To appreciate what the transport layer protocols do, you must think about the layer above the transport layer, the application layer. Why? Well, each layer provides a service to the layer above it, like the error-recovery service provided to application layer protocols by TCP.

For example, in Figure 1-5, Bob and Larry used HTTP to transfer the home page from web server Larry to Bob's web browser. But what would have happened if Bob's HTTP GET request had been lost in transit through the TCP/IP network? Or, what would have happened if Larry's response, which included the contents of the home page, had been lost? Well, as you might expect, in either case, the page would not have shown up in Bob's browser.

TCP/IP needs a mechanism to guarantee delivery of data across a network. Because many application layer protocols probably want a way to guarantee delivery of data across a network, the creators of TCP included an error-recovery feature. To recover from errors, TCP uses the concept of acknowledgments. Figure 1-7 outlines the basic idea behind how TCP notices lost data and asks the sender to try again.

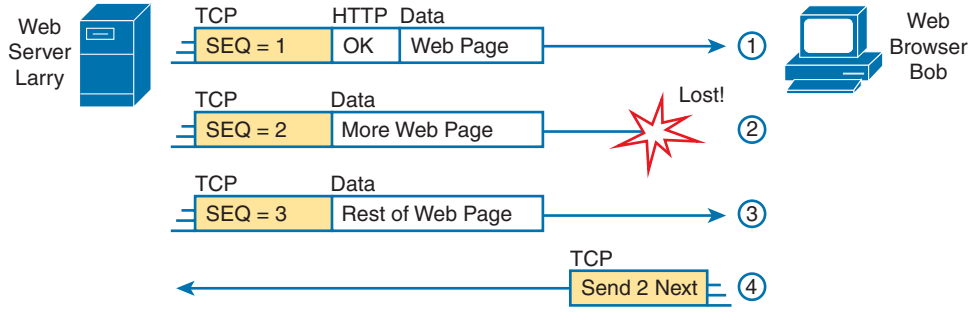



**Figure 1-7** *TCP Error-Recovery Services as Provided to HTTP*

Figure 1-7 shows web server Larry sending a web page to web browser Bob, using three separate messages. Note that this figure shows the same HTTP headers as Figure 1-6, but it also shows a TCP header. The TCP header shows a sequence number (SEQ) with each message. In this example, the network has a problem, and the network fails to deliver the TCP message (called a **segment**) with sequence number 2. When Bob receives messages with sequence numbers 1 and 3, but does not receive a message with sequence number 2, Bob realizes that message 2 was lost. That realization by Bob's TCP logic causes Bob to send a TCP segment back to Larry, asking Larry to send message 2 again.

## Same-Layer and Adjacent-Layer Interactions

Figure 1-7 also demonstrates a function called **adjacent-layer interaction**, which refers to the concepts of how adjacent layers in a networking model, on the same computer, work together. In this example, the higher-layer protocol (HTTP) wants error recovery, so it uses the next lower-layer protocol (TCP) to perform the service of error recovery; the lower layer provides a service to the layer above it.

Figure 1-7 also shows an example of a similar function called **same-layer interaction**. When a particular layer on one computer wants to communicate with the same layer on another computer, the two computers use headers to hold the information that they want to communicate. For example, in Figure 1-7, Larry set the sequence numbers to 1, 2, and 3 so that Bob could notice when some of the data did not arrive. Larry's TCP process created

that TCP header with the sequence number; Bob's TCP process received and reacted to the TCP segments.

Table 1-3 summarizes the key points about how adjacent layers work together on a single computer and how one layer on one computer works with the same networking layer on another computer.

**Table 1-3** Summary: Same-Layer and Adjacent-Layer Interactions

| Concept | Description |
|---|---|
| Same-layer interaction on different computers | The two computers use a protocol to communicate with the same layer on another computer. The protocol defines a header that communicates what each computer wants to do. |
| Adjacent-layer interaction on the same computer | On a single computer, one lower layer provides a service to the layer just above. The software or hardware that implements the higher layer requests that the next lower layer perform the needed function. |

## TCP/IP Network Layer

The application layer includes many protocols. The transport layer includes fewer protocols, most notably, TCP and UDP. The TCP/IP network layer includes a small number of protocols, but only one major protocol: the Internet Protocol (IP). In fact, the name TCP/IP is simply the names of the two most common protocols (TCP and IP) separated by a /.

IP provides several features, most importantly, addressing and routing. This section begins by comparing IP's addressing and routing with another commonly known system that uses addressing and routing: the postal service. Following that, this section introduces IP addressing and routing. (More details follow in Chapter 3, "Fundamentals of WANs and IP Routing.")

### Internet Protocol and the Postal Service

Imagine that you just wrote two letters: one to a friend on the other side of the country and one to a friend on the other side of town. You addressed the envelopes and put on the stamps, so both are ready to give to the postal service. Is there much difference in how you treat each letter? Not really. Typically, you would just put them in the same mailbox and expect the postal service to deliver both letters.

The postal service, however, must think about each letter separately, and then make a decision of where to send each letter so that it is delivered. For the letter sent across town, the people in the local post office probably just need to put the letter on another truck.

For the letter that needs to go across the country, the postal service sends the letter to another post office, then another, and so on, until the letter gets delivered across the country. At each post office, the postal service must process the letter and choose where to send it next.

To make it all work, the postal service has regular routes for small trucks, large trucks, planes, boats, and so on, to move letters between postal service sites. The service must be able to receive and forward the letters, and it must make good decisions about where to send each letter next, as shown in Figure 1-8.

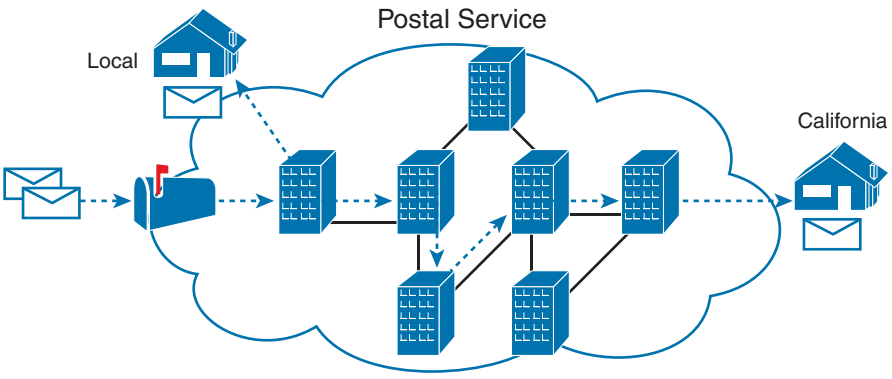



**Figure 1-8** *Postal Service Forwarding (Routing) Letters*

Still thinking about the postal service, consider the difference between the person sending the letter and the work that the postal service does. The person sending the letters expects that the postal service will deliver the letter most of the time. However, the person sending the letter does not need to know the details of exactly what path the letters take. In contrast, the postal service does not create the letter, but it accepts the letter from the customer. Then, the postal service must know the details about addresses and postal codes that group addresses into larger groups, and it must have the ability to deliver the letters.

The TCP/IP application and transport layers act like the person sending letters through the postal service. These upper layers work the same way regardless of whether the endpoint host computers are on the same LAN or are separated by the entire Internet. To send a message, these upper layers ask the layer below them, the network layer, to deliver the message.

The lower layers of the TCP/IP model act more like the postal service to deliver those messages to the correct destinations. To do so, these lower layers must understand the underlying physical network because they must choose how to best deliver the data from one host to another.

So, what does this all matter to networking? Well, the network layer of the TCP/IP networking model, primarily defined by the Internet Protocol (IP), works much like the postal service. IP defines that each host computer should have a different IP address, just as the postal service defines addressing that allows unique addresses for each house, apartment, and business. Similarly, IP defines the process of routing so that devices called routers can work like the post office, forwarding **packets** of data so that they are delivered to the correct destinations. Just as the postal service created the necessary infrastructure to deliver letters—post offices, sorting machines, trucks, planes, and personnel—the network layer defines the details of how a network infrastructure should be created so that the network can deliver data to all computers in the network.

## Internet Protocol Addressing Basics

IP defines addresses for several important reasons. First, each device that uses TCP/IP—each TCP/IP *host*—needs a unique address so that it can be identified in the network. IP also defines how to group addresses together, just like the postal system groups addresses based on postal codes (like ZIP codes in the United States).

To understand the basics, examine Figure 1-9, which shows the familiar web server Larry and web browser Bob; but now, instead of ignoring the network between these two computers, part of the network infrastructure is included.

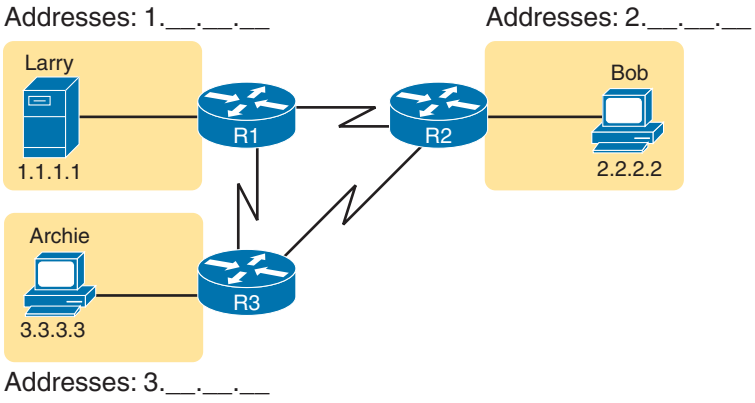



**Figure 1-9** *Simple TCP/IP Network: Three Routers with IP Addresses Grouped*

First, note that Figure 1-9 shows some sample IP addresses. Each IP address has four numbers, separated by periods. In this case, Larry uses IP address 1.1.1.1, and Bob uses 2.2.2.2. This style of number is called a dotted-decimal notation (DDN).

Figure 1-9 also shows three groups of addresses. In this example, all IP addresses that begin with 1 must be on the upper left, as shown in shorthand in the figure as 1.__.__.__. All addresses that begin with 2 must be on the right, as shown in shorthand as 2.__.__.__. Finally, all IP addresses that begin with 3 must be at the bottom of the figure.

In addition, Figure 1-9 introduces icons that represent IP routers. Routers are networking devices that connect the parts of the TCP/IP network together for the purpose of routing (forwarding) IP packets to the correct destination. Routers do the equivalent of the work done by each post office site: They receive IP packets on various physical interfaces, make decisions based on the IP address included with the packet, and then physically forward the packet out some other network interface.

## IP Routing Basics

The TCP/IP network layer, using the IP protocol, provides a service of forwarding IP packets from one device to another. Any device with an IP address can connect to the TCP/IP network and send packets. This section shows a basic IP routing example for perspective.

**NOTE** The term *IP host* refers to any device, regardless of size or power, that has an IP address and connects to any TCP/IP network.

Figure 1-10 repeats the familiar case in which web server Larry wants to send part of a web page to Bob, but now with details related to IP. On the lower left, note that server Larry has the familiar application data, HTTP header, and TCP header ready to send. In addition, the message now contains an IP header. The IP header includes a source IP address of Larry's IP address (1.1.1.1) and a destination IP address of Bob's IP address (2.2.2.2).

Key Top

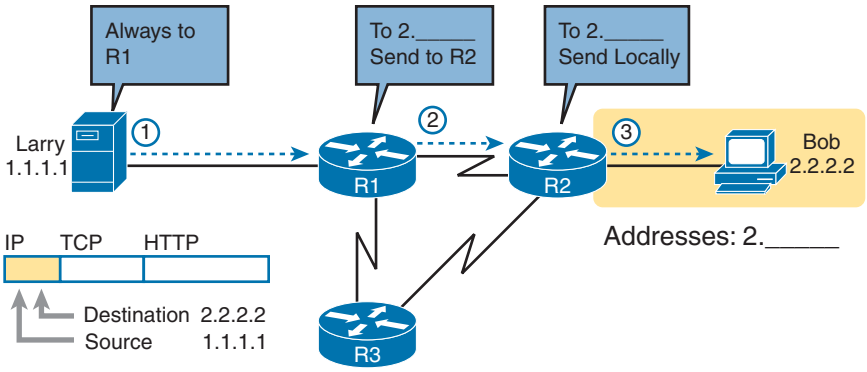



**Figure 1-10** *Basic Routing Example*

Step 1, on the left of Figure 1-10, begins with Larry being ready to send an IP packet. Larry's IP process chooses to send the packet to some router—a nearby router on the same LAN—with the expectation that the router will know how to forward the packet. (This logic is much like you or me sending all our letters by putting them in a nearby mailbox.) Larry doesn't need to know anything more about the topology or the other routers.

At Step 2, Router R1 receives the IP packet, and R1's IP process makes a decision. R1 looks at the destination address (2.2.2.2), compares that address to its known IP routes, and chooses to forward the packet to Router R2. This process of forwarding the IP packet is called *IP routing* (or simply *routing*).

At Step 3, Router R2 repeats the same kind of logic used by Router R1. R2's IP process will compare the packet's destination IP address (2.2.2.2) to R2's known IP routes and make a choice to forward the packet to the right, on to Bob.

You will learn IP to more depth than any other protocol while preparing for CCNA. More than half the chapters in this book discuss some feature that relates to addressing, IP routing, and how routers perform routing.

## TCP/IP Data-Link and Physical Layers

The TCP/IP model's data-link and physical layers define the protocols and hardware required to deliver data across some physical network. The two work together quite closely; in fact, some standards define both the data-link and physical layer functions. The physical layer defines the cabling and energy (for example, electrical signals) that flow over the cables. Some rules and conventions exist when sending data over the cable, however; those rules exist in the data-link layer of the TCP/IP model.

Focusing on the data-link layer for a moment, just like every layer in any networking model, the TCP/IP data-link layer provides services to the layer above it in the model (the network layer). When a host's or router's IP process chooses to send an IP packet to another router or host, that host or router then uses link-layer details to send that packet to the next host/router.

Because each layer provides a service to the layer above it, take a moment to think about the IP logic related to Figure 1-10. In that example, host Larry's IP logic chooses to send the IP packet to a nearby router (R1). However, while Figure 1-10 shows a simple line between Larry

and router R1, that drawing means that some Ethernet LAN sits between the two. Figure 1-11 shows four steps of what occurs at the link layer to allow Larry to send the IP packet to R1.

Key Topic

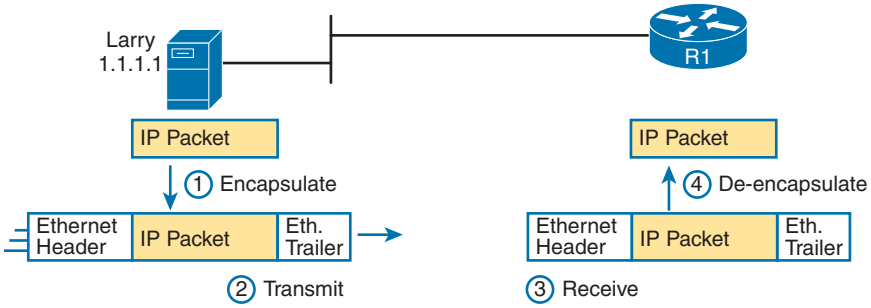



**Figure 1-11** *Larry Using Ethernet to Forward an IP Packet to Router R1*

**NOTE** Figure 1-11 depicts the Ethernet as a series of lines. Networking diagrams often use this convention when drawing Ethernet LANs, in cases where the actual LAN cabling and LAN devices are not important to some discussion, as is the case here. The LAN would have cables and devices, like LAN switches, which are not shown in this figure.

Figure 1-11 shows four steps. The first two occur on Larry, and the last two occur on Router R1, as follows:

**Step 1.** Larry encapsulates the IP packet between an Ethernet header and Ethernet trailer, creating an Ethernet **frame**.

**Step 2.** Larry physically transmits the bits of this Ethernet frame, using electricity flowing over the Ethernet cabling.

**Step 3.** Router R1 physically receives the electrical signal over a cable and re-creates the same bits by interpreting the meaning of the electrical signals.

**Step 4.** Router R1 **de-encapsulates** the IP packet from the Ethernet frame by removing and discarding the Ethernet header and trailer.

By the end of this process, Larry and R1 have worked together to deliver the packet from Larry to Router R1.

**NOTE** Protocols define both headers and trailers for the same general reason, but headers exist at the beginning of the message and trailers exist at the end.

The data-link and physical layers include a large number of protocols and standards. For example, the link layer includes all the variations of Ethernet protocols and wireless LAN protocols discussed throughout this book.

In short, the TCP/IP physical and data-link layers include two distinct functions, respectively: functions related to the physical transmission of the data, plus the protocols and rules that control the use of the physical media.

## Data Encapsulation Terminology

As you can see from the explanations of how HTTP, TCP, IP, and Ethernet do their jobs, when sending data, each layer adds its own header (and for data-link protocols, also a trailer) to the data supplied by the higher layer. The term **encapsulation** refers to the process of putting headers (and sometimes trailers) around some data.

Many of the examples in this chapter show the encapsulation process. For example, web server Larry encapsulated the contents of the home page inside an HTTP header in Figure 1-6. The TCP layer encapsulated the HTTP headers and data inside a TCP header in Figure 1-7. IP encapsulated the TCP headers and the data inside an IP header in Figure 1-10. Finally, the Ethernet link layer encapsulated the IP packets inside both a header and a trailer in Figure 1-11.

The process by which a TCP/IP host sends data can be viewed as a five-step process. The first four steps relate to the encapsulation performed by the four TCP/IP layers, and the last step is the actual physical transmission of the data by the host. In fact, if you use the five-layer TCP/IP model, one step corresponds to the role of each layer. The steps are summarized in the following list:

**Step 1.** **Create and encapsulate the application data with any required application layer headers.** For example, the HTTP OK message can be returned in an HTTP header, followed by part of the contents of a web page.

**Step 2.** **Encapsulate the data supplied by the application layer inside a transport layer header.** For end-user applications, a TCP or UDP header is typically used.

**Step 3.** **Encapsulate the data supplied by the transport layer inside a network layer (IP) header.** IP defines the IP addresses that uniquely identify each computer.

**Step 4.** **Encapsulate the data supplied by the network layer inside a data-link layer header and trailer.** This layer uses both a header and a trailer.

**Step 5.** **Transmit the bits.** The physical layer encodes a signal onto the medium to transmit the frame.

The numbers in Figure 1-12 correspond to the five steps in this list, graphically showing the same concepts. Note that because the application layer often does not need to add a header, the figure does not show a specific application layer header, but the application layer will also at times add a header as well.

### Names of TCP/IP Messages

One reason this chapter takes the time to show the encapsulation steps in detail has to do with terminology. When talking and writing about networking, people use *segment*, *packet*, and *frame* to refer to the messages shown in Figure 1-13 and the related list. Each term has a specific meaning, referring to the headers (and possibly trailers) defined by a particular layer and the data encapsulated following that header. Each term, however, refers to a different layer: segment for the transport layer, packet for the network layer, and frame for the link layer. Figure 1-13 shows each layer along with the associated term.

Key Top

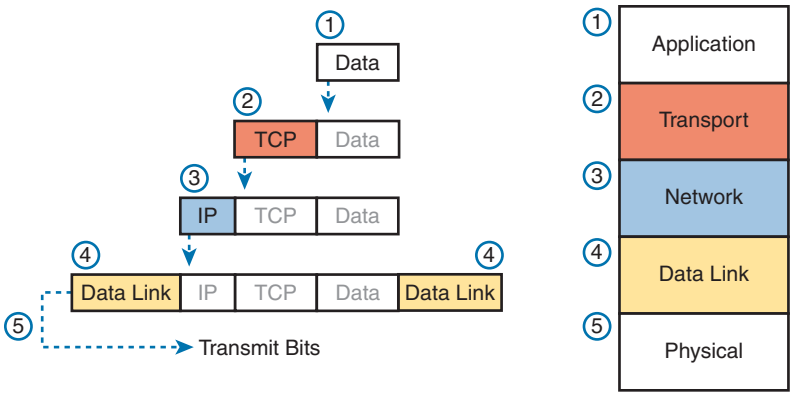



**Figure 1-12** *Five Steps of Data Encapsulation: TCP/IP*

Key Topic

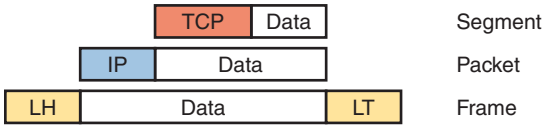



**Figure 1-13** *Perspectives on Encapsulation and "Data"**

* The letters *LH* and *LT* stand for link header and link trailer, respectively, and refer to the data-link layer header and trailer.

Figure 1-13 also shows the encapsulated data as simply "data." When you are focusing on the work done by a particular layer, the encapsulated data typically is unimportant. For example, an IP packet can indeed have a TCP header after the IP header, an HTTP header after the TCP header, and data for a web page after the HTTP header. However, when discussing IP, you probably just care about the IP header, so everything after the IP header is just called data. So, when you are drawing IP packets, everything after the IP header is typically shown simply as data.

**NOTE** You will also see the generic term *protocol data unit*, or *PDU*, used to refer to any message defined by a protocol. A TCP segment, IP packet, and Ethernet frame are all PDUs, for instance.

## OSI Networking Model and Terminology

At one point in the history of the OSI model, many people thought that OSI would win the battle of the networking models discussed earlier. If that had occurred, instead of running TCP/IP on every computer in the world, those computers would be running with OSI.

However, OSI did not win. In fact, OSI no longer exists as a networking model that could be used instead of TCP/IP, although some of the original protocols referenced by the OSI model still exist.

So, why is OSI even in this book? Terminology. During those years in which many people thought the OSI model would become commonplace in the world of networking (mostly in

the late 1980s and early 1990s), many vendors and protocol documents started using terminology from the OSI model. That terminology remains today. So, while you will never need to work with a computer that uses OSI, to understand modern networking terminology, you need to understand something about OSI.

## Comparing OSI and TCP/IP Layer Names and Numbers

The OSI model has many similarities to the TCP/IP model from a basic conceptual perspective. It has layers, and each layer defines a set of typical networking functions. As with TCP/IP, the OSI layers each refer to multiple protocols and standards that implement the functions specified by each layer. Just as for TCP/IP, the OSI committees did not create new protocols or standards in some cases, instead referencing other protocols that were already defined. For example, the IEEE defines Ethernet standards, so the OSI committees did not waste time specifying a new type of Ethernet; it simply referred to the IEEE Ethernet standards.

Today, the OSI model can be used as a standard of comparison to other networking models. Figure 1-14 compares the seven-layer OSI model with the commonly used five-layer TCP/IP model and the old original four-layer TCP/IP model.

Key Topic

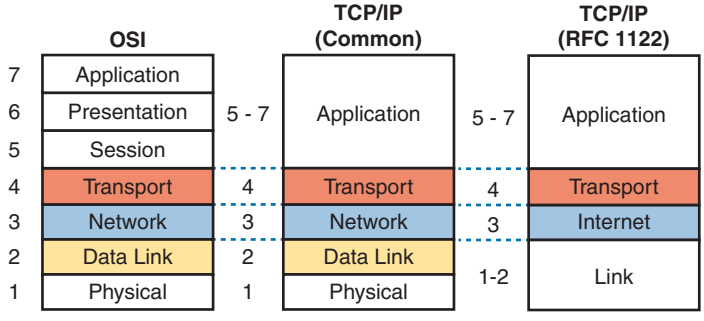



**Figure 1-14** *OSI Model Compared to the Two TCP/IP Models*

**NOTE** The CCNA exam topics no longer mention the OSI or TCP/IP models; however, you should know both and the related terminology for everyday network engineering discussions. While today you will see the five-layer model used throughout the industry, and in this book, the figure includes the original RFC 1122 four-layer model for perspective.

Note that the TCP/IP model in use today, in the middle of the figure, uses the exact same layer names as OSI at the lower layers. The functions generally match as well, so for the purpose of discussing networking, and reading networking documentation, think of the bottom four layers as equivalent, in name, in number, and in meaning.

Even though the world uses TCP/IP today rather than OSI, we tend to use the numbering from the OSI layer. For instance, when referring to an application layer protocol in a TCP/IP network, the world still refers to the protocol as a "Layer 7 protocol." Also, while TCP/IP includes more functions at its application layer, OSI breaks those into session, presentation, and application layers. Most of the time, no one cares much about the distinction, so you will see references like "Layer 5–7 protocol," again using OSI numbering.

For the purposes of this book, know the mapping between the five-layer TCP/IP model and the seven-layer OSI model shown in Figure 1-14, and know that layer number references to Layer 7 really do match the application layer of TCP/IP as well.

## Chapter Review

The "Your Study Plan" element, just before Chapter 1, discusses how you should study and practice the content and skills for each chapter before moving on to the next chapter. That element introduces the tools used here at the end of each chapter. If you haven't already done so, take a few minutes to read that section. Then come back here and do the useful work of reviewing the chapter to help lock into memory what you just read.

Review this chapter's material using either the tools in the book or the interactive tools for the same material found on the book's companion website. Table 1-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 1-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP Online |

## Review All the Key Topics

Key Topic

**Table 1-5** Key Topics for Chapter 1

| Key Topic Elements | Description | Page Number |
|---|---|---|
| Figure 1-10 | Shows the general concept of IP routing | 29 |
| Figure 1-11 | Depicts the data-link services provided to IP for the purpose of delivering IP packets from host to host | 30 |
| Figure 1-12 | Identifies the five steps to encapsulate data on the sending host | 32 |
| Figure 1-13 | Shows the meaning of the terms *segment*, *packet*, and *frame* | 32 |
| Figure 1-14 | Compares the OSI and TCP/IP networking models | 33 |

## Key Terms You Should Know

adjacent-layer interaction, de-encapsulation, encapsulation, frame, networking model, packet, same-layer interaction, segment

*This page intentionally left blank*



CHAPTER 2

# Fundamentals of Ethernet LANs

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.b Layer 2 and Layer 3 switches**

**1.2 Describe characteristics of network topology architectures**

**1.2.e Small office/home office (SOHO)**

**1.3 Compare physical interface and cabling types**

**1.3.a Single-mode fiber, multimode fiber, copper**

**1.3.b Connections (Ethernet shared media and point-to-point)**

Most enterprise computer networks can be separated into two general types of technology: local-area networks (LANs) and wide-area networks (WANs). LANs typically connect nearby devices: devices in the same room, in the same building, or in a campus of buildings. In contrast, WANs connect devices that are typically relatively far apart. Together, LANs and WANs create a complete enterprise computer network, working together to do the job of a computer network: delivering data from one device to another.

Many types of LANs have existed over the years, but today's networks use two general types of LANs: Ethernet LANs and wireless LANs. Ethernet LANs happen to use cables for the links between nodes, and because many types of cables use copper wires, Ethernet LANs are often called **wired LANs**. Ethernet LANs also make use of fiber-optic cabling, which includes a fiberglass core that devices use to send data using light. In comparison to Ethernet, **wireless LANs** do not use wires or cables, instead using radio waves for the links between nodes; Part I of the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, discusses wireless LANs at length.

This chapter introduces Ethernet LANs, with more detailed coverage in Parts II and III of this book.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 2-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| An Overview of LANs | 1–2 |
| Building Physical Ethernet LANs with UTP | 3–4 |
| Building Physical Ethernet LANs with Fiber | 5 |
| Sending Data in Ethernet Networks | 6–9 |

1. Some end-user devices connect to a LAN using a cable while others use wireless. Which answer best characterizes which devices use Ethernet to connect to the LAN?
   a. Only the end-user devices that use cables are using Ethernet.
   b. Only the end-user devices that use wireless are using Ethernet.
   c. Both the end-user devices using cables and those using wireless are using Ethernet.
   d. Neither the end-user devices using cables nor those using wireless are using Ethernet.
2. Which of the following Ethernet standards defines Gigabit Ethernet over UTP cabling?
   a. 10GBASE-T
   b. 100BASE-T
   c. 1000BASE-T
   d. None of the other answers are correct.
3. Which of the following is true about Ethernet crossover cables for Fast Ethernet?
   a. Pins 1 and 2 are reversed on the other end of the cable.
   b. Pins 1 and 2 on one end of the cable connect to pins 3 and 6 on the other end of the cable.
   c. Pins 1 and 2 on one end of the cable connect to pins 3 and 4 on the other end of the cable.
   d. The cable can be up to 1000 meters long to cross over between buildings.
   e. None of the other answers are correct.
4. Each answer lists two types of devices used in a 100BASE-T network. If these devices were connected with UTP Ethernet cables, which pairs of devices would require a straight-through cable? (Choose three answers.)
   a. PC and router
   b. PC and switch
   c. Hub and switch
   d. Router and hub
   e. Wireless access point (Ethernet port) and switch

5. Which of the following are advantages of using multimode fiber for an Ethernet link instead of UTP or single-mode fiber? (Choose two answers.)
   a. To achieve the longest distance possible for that single link.
   b. To extend the link beyond 100 meters while keeping initial costs as low as possible.
   c. To make use of an existing stock of laser-based SFP/SFP+ modules.
   d. To make use of an existing stock of LED-based SFP/SFP+ modules.
6. Which of the following is true about the CSMA/CD algorithm?
   a. The algorithm never allows collisions to occur.
   b. Collisions can happen, but the algorithm defines how the computers should notice a collision and how to recover.
   c. The algorithm works with only two devices on the same Ethernet.
   d. None of the other answers are correct.
7. Which of the following is true about the Ethernet FCS field?
   a. Ethernet uses FCS for error recovery.
   b. It is 2 bytes long.
   c. It resides in the Ethernet trailer, not the Ethernet header.
   d. It is used for encryption.
8. Which of the following are true about the format of Ethernet addresses? (Choose three answers.)
   a. Each manufacturer puts a unique OUI code into the first 2 bytes of the address.
   b. Each manufacturer puts a unique OUI code into the first 3 bytes of the address.
   c. Each manufacturer puts a unique OUI code into the first half of the address.
   d. The part of the address that holds the manufacturer's code is called the MAC.
   e. The part of the address that holds the manufacturer's code is called the OUI.
   f. The part of the address that holds the manufacturer's code has no specific name.
9. Which of the following terms describe Ethernet addresses that can be used to send one frame that is delivered to multiple devices on the LAN? (Choose two answers.)
   a. Burned-in address
   b. Unicast address
   c. Broadcast address
   d. Multicast address

# Foundation Topics

## An Overview of LANs

The term **Ethernet** refers to a family of LAN standards that together define the physical and data-link layers of the world's most popular wired LAN technology. The standards, defined by the Institute of Electrical and Electronics Engineers (**IEEE**), define the cabling, the connectors on the ends of the cables, the protocol rules, and everything else required to create an Ethernet LAN.

## Typical SOHO LANs

To begin, first think about a small office/home office (SOHO) LAN today, specifically a LAN that uses only Ethernet LAN technology. First, the LAN needs a device called an Ethernet *LAN switch*, which provides many physical ports into which cables can be connected. An Ethernet uses *Ethernet cables*, which is a general reference to any cable that conforms to any of several Ethernet standards. The LAN uses Ethernet cables to connect different Ethernet devices or nodes to one of the switch's Ethernet ports.

Figure 2-1 shows a drawing of a SOHO Ethernet LAN. The figure shows a single LAN switch, five cables, and five other Ethernet nodes: three PCs, a printer, and one network device called a *router*. (The router connects the LAN to the WAN, in this case to the Internet.)

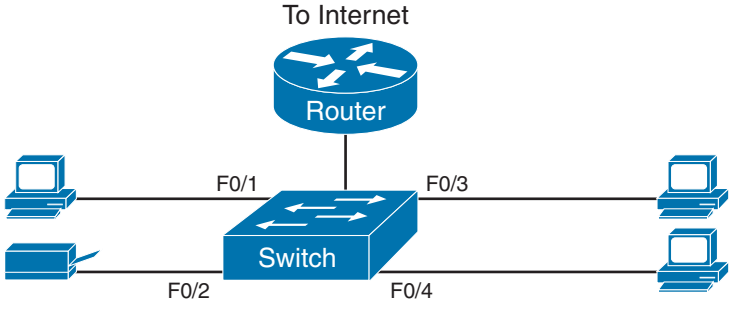



**Figure 2-1** *Typical Small Ethernet-Only SOHO LAN*

Although Figure 2-1 shows the switch and router as separate devices, many SOHO Ethernet LANs today combine the router and switch into a single device. Vendors sell consumer-grade integrated networking devices that work as a router and Ethernet switch, as well as doing other functions. These devices typically have "router" on the packaging, but many models also have four-port or eight-port Ethernet LAN switch ports built into the device.

Typical SOHO LANs today also support wireless LAN connections. You can build a single SOHO LAN that includes both Ethernet LAN technology as well as wireless LAN technology, which is also defined by the IEEE. Wireless LANs, defined by the IEEE using standards that begin with 802.11, use radio waves to send the bits from one node to the next.

Most wireless LANs rely on yet another networking device: a wireless LAN access point (AP). The AP acts somewhat like an Ethernet switch, in that all the wireless LAN nodes communicate with the wireless AP. If the network uses an AP that is a separate physical device, the AP then needs a single Ethernet link to connect the AP to the Ethernet LAN, as shown in Figure 2-2.

Note that Figure 2-2 shows the router, Ethernet switch, and wireless LAN access point as three separate devices so that you can better understand the different roles. However, most SOHO networks today would use a single device, often labeled as a "wireless router," that does all these functions.

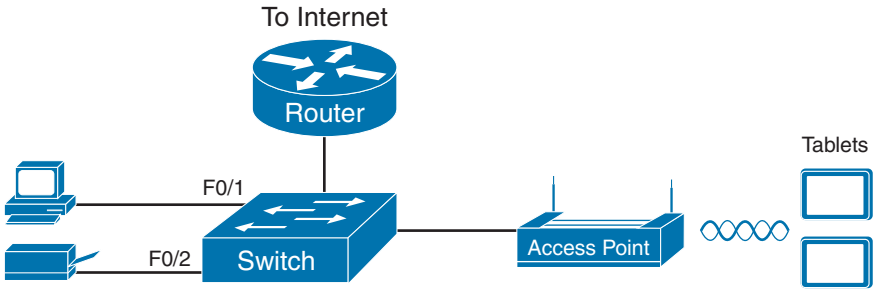



**Figure 2-2** *Typical Small Wired and Wireless SOHO LAN*

## Typical Enterprise LANs

Enterprise networks have similar needs compared to a SOHO network, but on a much larger scale. For example, enterprise Ethernet LANs begin with LAN switches installed in a wiring closet behind a locked door on each floor of a building. The electricians install the Ethernet cabling from that wiring closet to cubicles and conference rooms where devices might need to connect to the LAN. At the same time, most enterprises also support wireless LANs in the same space, to allow people to roam around and still work and to support a growing number of devices that do not have an Ethernet LAN interface.

Figure 2-3 shows a conceptual view of a typical enterprise LAN in a three-story building. Each floor has an Ethernet LAN switch and a wireless LAN AP. To allow communication between floors, each per-floor switch connects to one centralized distribution switch. For example, PC3 can send data to PC2, but it would first flow through switch SW3 to the first floor to the distribution switch (SWD) and then back up through switch SW2 on the second floor.

Key Topic

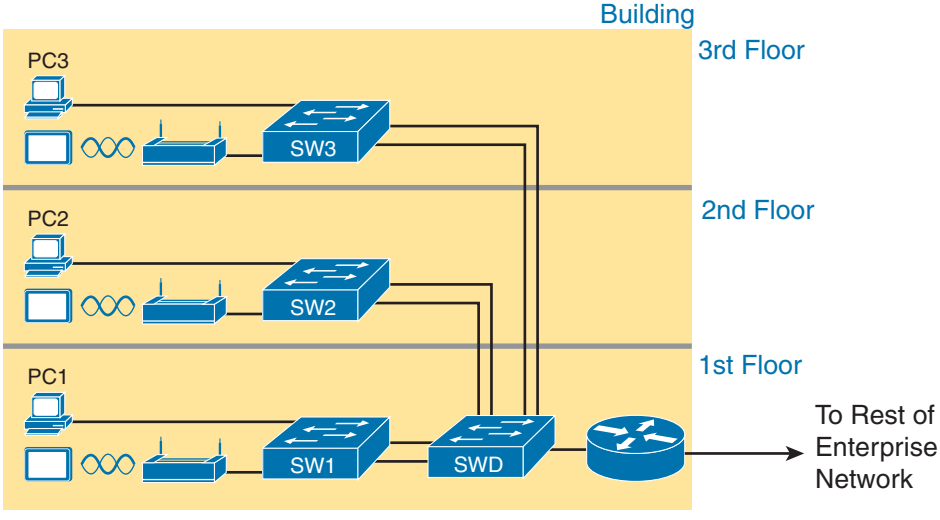



**Figure 2-3** *Single-Building Enterprise Wired and Wireless LAN*

Answers to the "Do I Know This Already?" quiz:

**1** A **2** C **3** B **4** B, D, and E **5** B and D **6** B **7** C **8** B, C, and E **9** C and D

The figure also shows the typical way to connect a LAN to a WAN using a router. LAN switches and wireless access points work to create the LAN itself. Routers connect to both the LAN and the WAN. To connect to the LAN, the router simply uses an Ethernet LAN interface and an Ethernet cable, as shown on the lower right of Figure 2-3.

The rest of this chapter focuses on Ethernet in particular.

## The Variety of Ethernet Physical Layer Standards

The term *Ethernet* refers to an entire family of standards. Some standards define the specifics of how to send data over a particular type of cabling, and at a particular speed. Other standards define protocols, or rules, that the Ethernet nodes must follow to be a part of an Ethernet LAN. All these Ethernet standards come from the IEEE and include the number 802.3 as the beginning part of the standard name.

Ethernet supports a large variety of options for physical Ethernet links given its long history over the last 40 or so years. Today, Ethernet includes many standards for different kinds of optical and copper cabling, and for speeds from 10 megabits per second (Mbps) up to 400 gigabits per second (Gbps). The standards also differ as far as the types and length of the cables.

The most fundamental cabling choice has to do with the materials used inside the cable for the physical transmission of bits: either copper wires or glass fibers. Devices using unshielded twisted-pair (UTP) cabling transmit data over electrical circuits via the copper wires inside the cable. Fiber-optic cabling, the more expensive alternative, allows Ethernet nodes to send light over glass fibers in the center of the cable. Although more expensive, optical cables typically allow longer cabling distances between nodes.

To be ready to choose the products to purchase for a new Ethernet LAN, a network engineer must know the names and features of the different Ethernet standards supported in Ethernet products. The IEEE defines Ethernet physical layer standards using a couple of naming conventions. The formal name begins with 802.3 followed by some suffix letters. The IEEE also uses more meaningful shortcut names that identify the speed, as well as a clue about whether the cabling is UTP (with a suffix that includes *T*) or fiber (with a suffix that includes *X*). Table 2-2 lists a few Ethernet physical layer standards. First, the table lists enough names so that you get a sense of the IEEE naming conventions.

Key Topic

**Table 2-2** Examples of Types of Ethernet

| Speed | Common Name | Informal IEEE Standard Name | Formal IEEE Standard Name | Cable Type, Maximum Length |
|---|---|---|---|---|
| 10 Mbps | Ethernet | 10BASE-T | 802.3 | Copper, 100 m |
| 100 Mbps | Fast Ethernet | 100BASE-T | 802.3u | Copper, 100 m |
| 1000 Mbps | Gigabit Ethernet | 1000BASE-LX | 802.3z | Fiber, 5000 m |
| 1000 Mbps | Gigabit Ethernet | 1000BASE-T | 802.3ab | Copper, 100 m |
| 10 Gbps | 10 Gig Ethernet | 10GBASE-T | 802.3an | Copper, 100 m |

**NOTE** Fiber-optic cabling contains long thin strands of fiberglass. The attached Ethernet nodes send light over the glass fiber in the cable, encoding the bits as changes in the light.

**NOTE** You might expect that a standard that began at the IEEE over 40 years ago would be stable and unchanging, but the opposite is true. The IEEE, along with active industry partners, continue to develop new Ethernet standards with longer distances, different cabling options, and faster speeds. Check out the Ethernet Alliance web page (www.EthernetAlliance.org) and look for the roadmap for some great graphics and tables about the latest happenings with Ethernet.

## Consistent Behavior over All Links Using the Ethernet Data-Link Layer

Although Ethernet includes many physical layer standards, Ethernet acts like a single LAN technology because it uses the same data-link layer standard over all types of Ethernet physical links. That standard defines a common Ethernet header and trailer. (As a reminder, the header and trailer are bytes of overhead data that Ethernet uses to do its job of sending data over a LAN.) No matter whether the data flows over a UTP cable or any kind of fiber cable, and no matter the speed, the data-link header and trailer use the same format.

While the physical layer standards focus on sending bits over a cable, the Ethernet data-link protocols focus on sending an **Ethernet frame** from source to destination Ethernet node. From a data-link perspective, nodes build and forward frames. As first defined in Chapter 1, "Introduction to TCP/IP Networking," the term *frame* specifically refers to the header and trailer of a data-link protocol, plus the data encapsulated inside that header and trailer. The various Ethernet nodes simply forward the frame, over all the required links, to deliver the frame to the correct destination.

Figure 2-4 shows an example of the process. In this case, PC1 sends an Ethernet frame to PC3. The frame travels over a UTP link to Ethernet switch SW1, then over fiber links to Ethernet switches SW2 and SW3, and finally over another UTP link to PC3. Note that the bits actually travel at four different speeds in this example: 10 Mbps, 1 Gbps, 10 Gbps, and 100 Mbps, respectively.

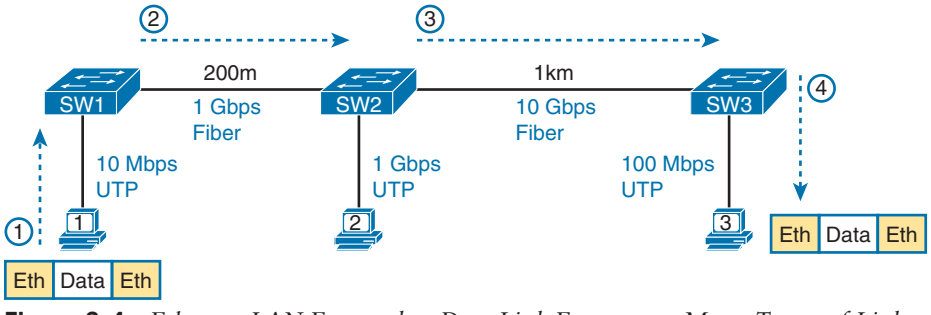



**Figure 2-4** *Ethernet LAN Forwards a Data-Link Frame over Many Types of Links*

So, what is an Ethernet LAN? It is a combination of user devices, LAN switches, and different kinds of cabling. Each link can use different types of cables, at different speeds. However, they all work together to deliver Ethernet frames from the one device on the LAN to some other device.

The rest of this chapter takes these concepts a little deeper. The next section examines how to build a physical Ethernet network using UTP cabling, followed by a similar look at using

fiber cabling to build Ethernet LANs. The chapter ends with some discussion of the rules for forwarding frames through an Ethernet LAN.



## Building Physical Ethernet LANs with UTP

The next section of this chapter focuses on the individual physical links between any two Ethernet nodes, specifically those that use unshielded twisted-pair (UTP) cabling. Before the Ethernet network as a whole can send Ethernet frames between user devices, each node must be ready and able to send data over an individual physical link.

This section focuses on the three most commonly used Ethernet standards: **10BASE-T** (Ethernet), **100BASE-T** (**Fast Ethernet**, or FE), and **1000BASE-T** (**Gigabit Ethernet**, or GE). Specifically, this section looks at the details of sending data in both directions over a UTP cable. It then examines the specific wiring of the UTP cables used for 10-Mbps, 100-Mbps, and 1000-Mbps Ethernet.

### Transmitting Data Using Twisted Pairs

While it is true that Ethernet sends data over UTP cables, the physical means to send the data uses electricity that flows over the wires inside the UTP cable. To better understand how Ethernet sends data using electricity, break the idea down into two parts: how to create an electrical circuit and then how to make that electrical signal communicate 1s and 0s.

First, to create one electrical circuit, Ethernet defines how to use the two wires inside a single twisted pair of wires, as shown in Figure 2-5. The figure does not show a UTP cable between two nodes, but instead shows two individual wires that are inside the UTP cable. An electrical circuit requires a complete loop, so the two nodes, using circuitry on their Ethernet ports, connect the wires in one pair to complete a loop, allowing electricity to flow.

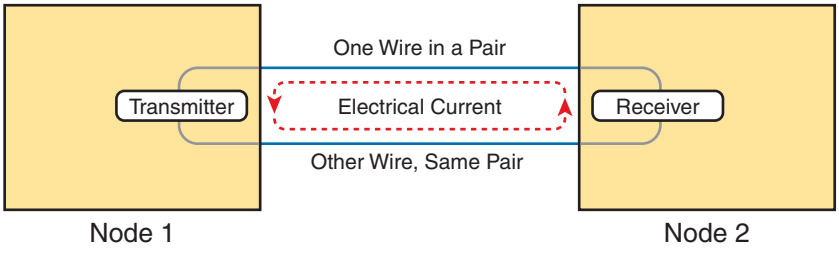



**Figure 2-5** *Creating One Electrical Circuit over One Pair to Send in One Direction*

To send data, the two devices follow some rules called an *encoding scheme*. The idea works a lot like when two people talk using the same language: The speaker says some words in a particular language, and the listener, because she speaks the same language, can understand the spoken words. With an encoding scheme, the transmitting node changes the electrical signal over time, while the other node, the receiver, using the same rules, interprets those changes as either 0s or 1s. (For example, 10BASE-T uses an encoding scheme that encodes a binary 0 as a transition from higher voltage to lower voltage during the middle of a 1/10,000,000th-of-a-second interval.)

Note that in an actual UTP cable, the wires will be twisted together, instead of being parallel, as shown in Figure 2-5. The twisting helps solve some important physical transmission

issues. When electrical current passes over any wire, it creates **electromagnetic interference (EMI)** that interferes with the electrical signals in nearby wires, including the wires in the same cable. (EMI between wire pairs in the same cable is called *crosstalk*.) Twisting the wire pairs together helps cancel out most of the EMI, so most networking physical links that use copper wires use twisted pairs.

## Breaking Down a UTP Ethernet Link

The term **Ethernet link** refers to any physical cable between two Ethernet nodes. To learn about how a UTP Ethernet link works, it helps to break down the physical link into those basic pieces, as shown in Figure 2-6: the cable itself, the connectors on the ends of the cable, and the matching ports on the devices into which the connectors will be inserted.

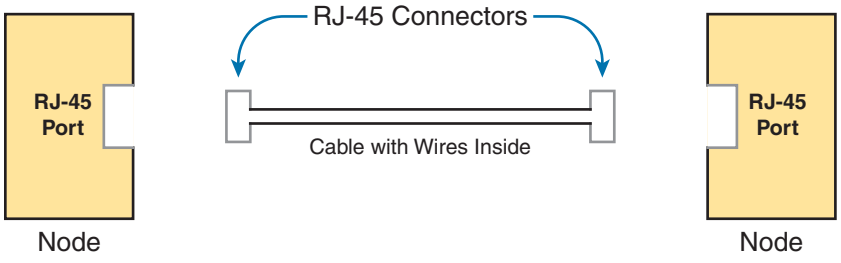



**Figure 2-6** *Basic Components of an Ethernet Link*

First, think about the UTP cable itself. The cable holds some copper wires, grouped as twisted pairs. The 10BASE-T and 100BASE-T standards require two pairs of wires, while the 1000BASE-T standard requires four pairs. Each wire has a color-coded plastic coating, with the wires in a pair having a color scheme. For example, for the blue wire pair, one wire's coating is all blue, while the other wire's coating is blue-and-white striped.

Many Ethernet UTP cables use an RJ-45 connector on both ends. The **RJ-45** connector has eight physical locations into which the eight wires in the cable can be inserted, called *pin positions*, or simply *pins*. These pins create a place where the ends of the copper wires can touch the electronics inside the nodes at the end of the physical link so that electricity can flow.

**NOTE** If available, find a nearby Ethernet UTP cable and examine the connectors closely. Look for the pin positions and the colors of the wires in the connector.

To complete the physical link, the nodes each need an RJ-45 **Ethernet port** that matches the RJ-45 connectors on the cable so that the connectors on the ends of the cable can connect to each node. PCs often include this RJ-45 Ethernet port as part of a **network interface card (NIC)**, which can be an expansion card on the PC or can be built into the system itself. Switches typically have many RJ-45 ports because switches give user devices a place to connect to the Ethernet LAN. Figure 2-7 shows photos of the cables, connectors, and ports.

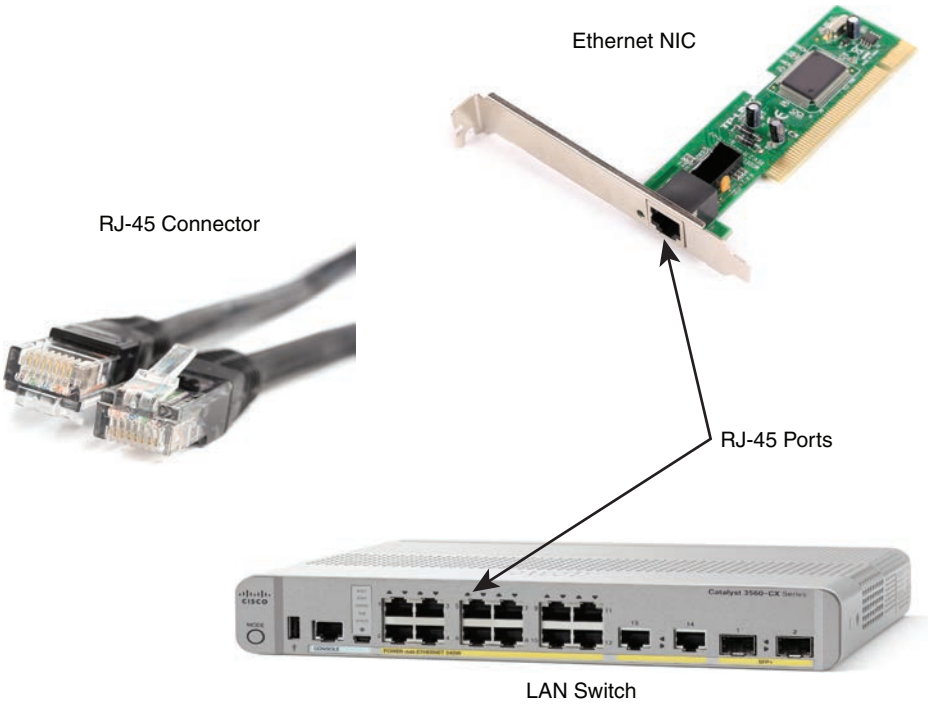



**Figure 2-7** *RJ-45 Connectors and Ports*

The figure shows a connector on the left and ports on the right. The left shows the eight pin positions in the end of the RJ-45 connector. The upper right shows an Ethernet NIC that is not yet installed in a computer. The lower-right part of the figure shows the side of a Cisco switch, with multiple RJ-45 ports, allowing multiple devices to easily connect to the Ethernet network.

Finally, while RJ-45 connectors with UTP cabling can be common, Cisco LAN switches often support other types of connectors as well. When you buy one of the many models of Cisco switches, you need to think about the mix and numbers of each type of physical ports you want on the switch.

To give its customers flexibility as to the type of Ethernet links, even after the customer has bought the switch, Cisco switches include some physical ports whose port hardware (the **transceiver**) can be changed later, after you purchase the switch.

For example, Figure 2-8 shows a photo of a Cisco switch with one of the swappable transceivers. In this case, the figure shows an enhanced small form-factor pluggable plus (SFP+) transceiver, which runs at 10 Gbps, just outside two SFP+ slots on a Cisco 3560CX switch. The SFP+ itself is the silver-colored part below the switch, with a black cable connected to it.

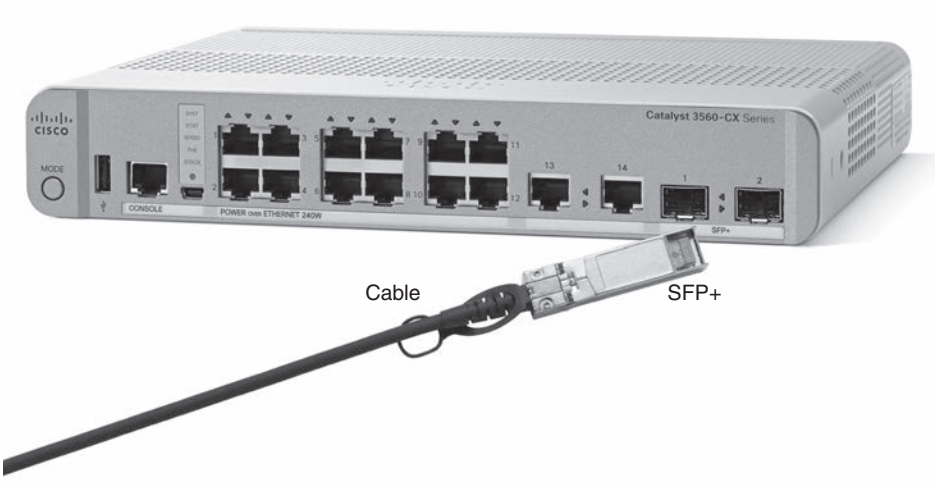



**Figure 2-8** *10-Gbps SFP+ with Cable Sitting Just Outside a Catalyst 3560CX Switch*

**Gigabit Ethernet Interface Converter (GBIC):** The original form factor for a removable transceiver for Gigabit interfaces; larger than SFPs.

**Small Form-Factor Pluggable (SFP):** The replacement for GBICs, used on Gigabit interfaces, with a smaller size, taking less space on the side of the networking card or switch.

**Small Form-Factor Pluggable Plus (SFP+):** Same size as the SFP, but used on 10-Gbps interfaces. (The Plus refers to the increase in speed compared to SFPs.)

## UTP Cabling Pinouts for 10BASE-T and 100BASE-T

So far in this section, you have learned about the equivalent of how to drive a truck on a 1000-acre ranch: You could drive the truck all over the ranch, any place you wanted to go, and the police would not mind. However, as soon as you get on the public roads, the police want you to behave and follow the rules. Similarly, so far this chapter has discussed the general principles of how to send data, but it has not yet detailed some important rules for Ethernet cabling: the rules of the road so that all the devices send data using the right wires inside the cable.

This next topic discusses some of those rules, specifically for the 10-Mbps 10BASE-T and the 100-Mbps 100BASE-T. Both use UTP cabling in similar ways (including the use of only two wire pairs). A short comparison of the wiring for 1000BASE-T (Gigabit Ethernet), which uses four pairs, follows.

### Straight-Through Cable Pinout

10BASE-T and 100BASE-T use two pairs of wires in a UTP cable, one for each direction, as shown in Figure 2-9. The figure shows four wires, all of which sit inside a single UTP cable that connects a PC and a LAN switch. In this example, the PC on the left transmits using the top pair, and the switch on the right transmits using the bottom pair.

Key Topic

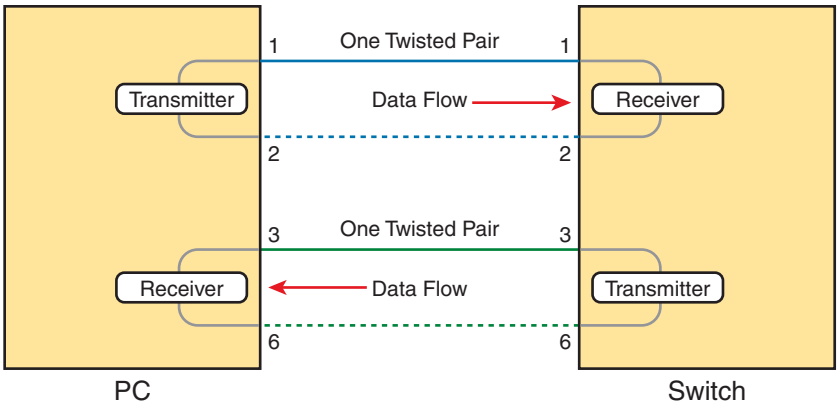



**Figure 2-9** *Using One Pair for Each Transmission Direction with 10- and 100-Mbps Ethernet*

For correct transmission over the link, the wires in the UTP cable must be connected to the correct pin positions in the RJ-45 connectors. For example, in Figure 2-9, the transmitter on the PC on the left must know the pin positions of the two wires it should use to transmit. Those two wires must be connected to the correct pins in the RJ-45 connector on the switch so that the switch's receiver logic can use the correct wires.

To understand the wiring of the cable—which wires need to be in which pin positions on both ends of the cable—you need to first understand how the NICs and switches work. As a rule, Ethernet NIC transmitters use the pair connected to pins 1 and 2; the NIC receivers use a pair of wires at pin positions 3 and 6. LAN switches, knowing those facts about what Ethernet NICs do, do the opposite: Their receivers use the wire pair at pins 1 and 2, and their transmitters use the wire pair at pins 3 and 6. The switch effectively reverses the transmit and receive logic of the endpoint device.

To make the preceding logic work, the UTP cable must use a **straight-through cable** *pinout* convention. The term *pinout* refers to the wiring of which color wire is placed in each of the eight numbered pin positions in the RJ-45 connector. An Ethernet straight-through cable connects the wire at pin 1 on one end of the cable to pin 1 at the other end of the cable; the wire at pin 2 needs to connect to pin 2 on the other end of the cable; pin 3 on one end connects to pin 3 on the other, and so on. Figure 2-10 shows the concept of straight-through pinout with two pairs—one pair at pins 1,2 and another at pins 3,6, as used by 10BASE-T and 100BASE-T.

Ke Top

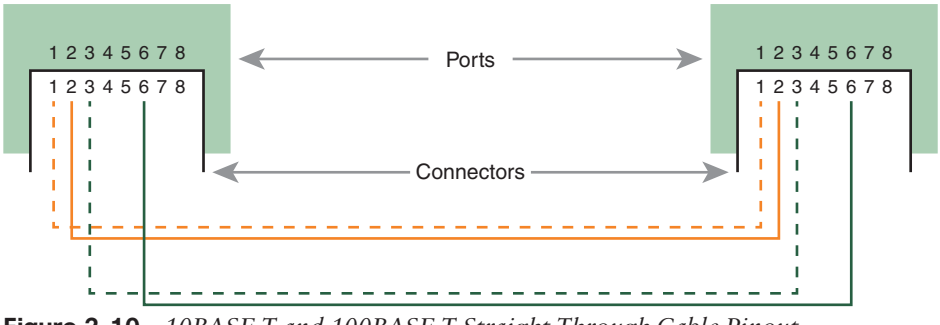



**Figure 2-10** *10BASE-T and 100BASE-T Straight-Through Cable Pinout*

They effectively reverse or cross over the transmit and receive functions, so that for links between a switch and an endpoint device, the cable can use the straight-through pinout shown in Figure 2-9.

A straight-through cable works correctly when the nodes use opposite pairs for transmitting data, as seen in Figure 2-9. However, when connecting two devices that transmit on the same pins, you then need another type of cabling pinout called a **crossover cable** pinout. The crossover cable pinout crosses the pair at the transmit pins on each device to the receive pins on the opposite device.

While that previous sentence is true, this concept is much clearer with a figure such as Figure 2-11. The figure shows what happens on a link between two switches. The two switches both transmit on the pair at pins 3 and 6, and they both receive on the pair at pins 1 and 2. So, the cable must connect a pair at pins 3 and 6 on each side to pins 1 and 2 on the other side, connecting to the other node's receiver logic.

Key Topic

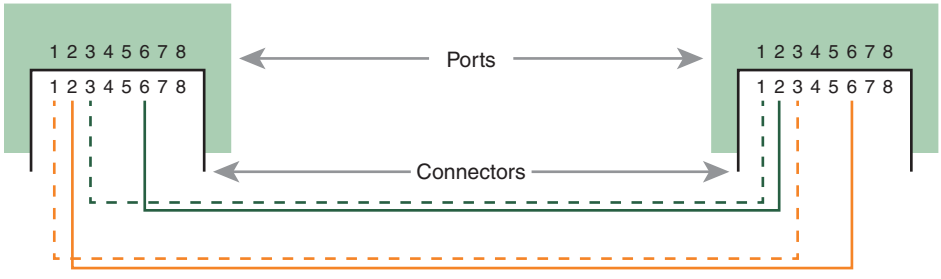



**Figure 2-11** *Crossover Ethernet Cable Pinouts*

## Choosing the Right Cable Pinouts

For the exam, you should be well prepared to choose which type of cable (straight-through or crossover) is needed in each part of the network. The key is to know whether a device acts like a PC NIC, transmitting at pins 1 and 2, or like a switch, transmitting at pins 3 and 6. Then, just apply the following logic:

**Crossover cable:** If the endpoints transmit on the same pin pair

**Straight-through cable:** If the endpoints transmit on different pin pairs

Table 2-3 lists the devices and the pin pairs they use, assuming that they use 10BASE-T and 100BASE-T.

Key Topic

**Table 2-3** 10BASE-T and 100BASE-T Pin Pairs Used

| Transmits on Pins 1,2 | Transmits on Pins 3,6 |
|---|---|
| PC NICs | Hubs |
| Routers | Switches |
| Wireless access point (Ethernet interface) | — |

For example, Figure 2-12 shows a campus LAN in a single building. In this case, several straight-through cables are used to connect PCs to switches. In addition, the cables connecting the switches require *crossover cables*.

Key Topic

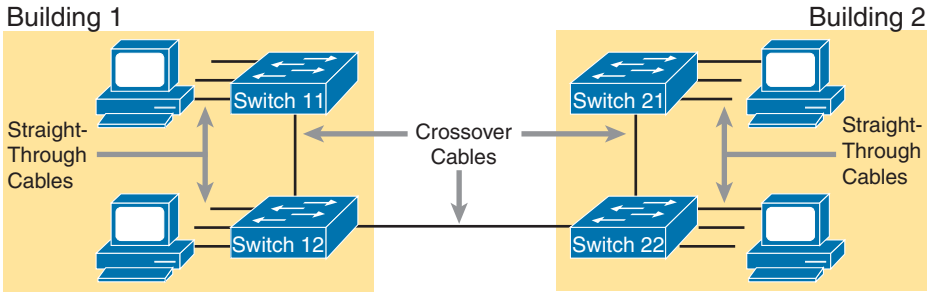



**Figure 2-12** *Typical Uses for Straight-Through and Crossover Ethernet Cables*

### Automatic Rewiring with Auto-MDIX

Using the wrong cabling pinout with a UTP cable happens to be one of the more common implementation mistakes. Back in 1998, with the introduction of Gigabit Ethernet, the IEEE added a new feature to Ethernet that defines how any device can use electrical pulses to sense if the cable has the wrong cable pinout. For instance, if the link needs a crossover cable, but the installer connected a straight-through cable, this feature can sense the incorrect pinout, and then redirect the electrical signals to the correct pairs to compensate so that the link works. The Ethernet standard calls this feature *automatic medium-dependent interface crossover* (**auto-MDIX**).

Auto-MDIX allows sites to use straight-through pinouts on all cables. For instance, the entire cable plant could use a straight-through pinout. On the links that need a crossover pinout, the auto-MDIX on the switch port will sense the use of the straight-through pinout and then internally swap the pairs used by the transceiver to make the link work. (Additionally, most installations use four-pair UTP cables that support Gigabit Ethernet so that all links support 10-, 100-, or 1000-Mbps Ethernet.)

## UTP Cabling Pinouts for 1000BASE-T

*1000BASE-T* (*Gigabit Ethernet*) differs from 10BASE-T and 100BASE-T as far as the cabling and pinouts. First, 1000BASE-T requires four wire pairs. Second, it uses more advanced electronics that allow both ends to transmit and receive simultaneously on each wire pair. However, the wiring pinouts for 1000BASE-T work almost identically to the earlier standards, adding details for the additional two pairs.

The straight-through cable for 1000BASE-T uses the four wire pairs to create four circuits, but the pins need to match. It uses the same pinouts for two pairs as do the 10BASE-T and 100BASE-T standards, and it adds a pair at pins 4 and 5 and another pair at pins 7 and 8, as shown in Figure 2-13.

Just as with 10BASE-T and 100BASE-T, 1000BASE-T (Gigabit Ethernet) uses straight-through cable pinout for some links but crossover cables in other cases. The Gigabit Ethernet crossover cable crosses pairs A and B in the figure (the pairs at pins 1,2 and 3,6) and also pairs C and D (the pair at pins 4,5 with the pair at pins 7,8). You need a crossover cable pinout in the same cases listed earlier in Table 2-3—for instance, between two switches.

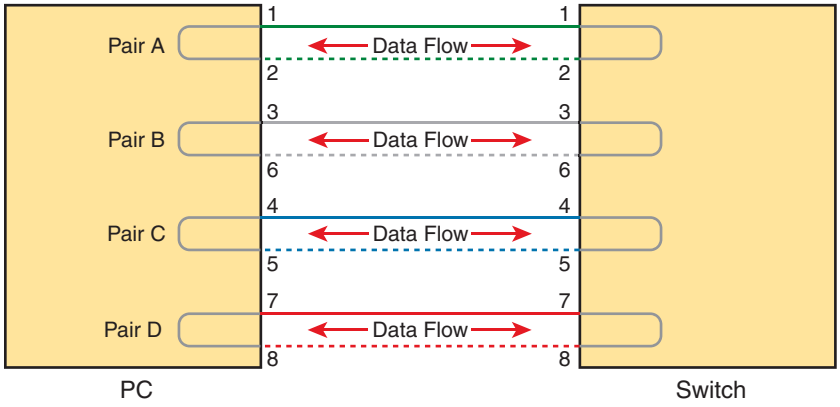



**Figure 2-13** *Four-Pair Straight-Through Cable to 1000BASE-T*

# Building Physical Ethernet LANs with Fiber

The capability of many UTP-based Ethernet standards to use a cable length up to 100 meters means that the majority of Ethernet cabling in an enterprise uses UTP cables. The distance from an Ethernet switch to every endpoint on the floor of a building will likely be less than 100m. In some cases, however, an engineer might prefer to use fiber cabling for some links in an Ethernet LAN, first to reach greater distances, but for other reasons as well. This next section examines a few of the tradeoffs after discussing the basics of how to transmit data over fiber cabling.

## Fiber Cabling Transmission Concepts

*Fiber-optic cabling* uses glass as the medium through which light passes, varying that light over time to encode 0s and 1s. It might seem strange at first to use glass given that most of us think of glass in windows. Window glass is hard, unbending, and if you hit or bend it enough, the glass will probably shatter—all bad characteristics for a cabling material.

Instead, fiber-optic cables use fiberglass, which allows a manufacturer to spin a long thin string (fiber) of flexible glass. A **fiber-optic cable** holds the fiber in the middle of the cable, allowing the light to pass through the glass—which is a very important attribute for the purposes of sending data.

Although sending data through a glass fiber works well, the glass fiber by itself needs some help. The glass could break, so the glass fiber needs some protection and strengthening. Figure 2-14 shows a cutout with the components of a fiber cable for perspective.

The three outer layers of the cable protect the interior of the cable and make the cables easier to install and manage, while the inner **cladding** and **core** work together to create the environment to allow transmission of light over the cable. A light source, called the *optical transmitter*, shines a light into the core. Light can pass through the core; however, light reflects off the cladding back into the core. Figure 2-15 shows an example with a light-emitting diode (LED) transmitter. You can see how the cladding reflects the light back into the core as it travels through the core.

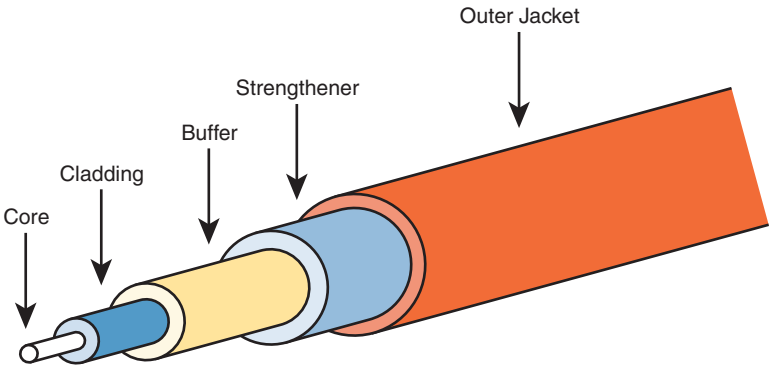



**Figure 2-14** *Components of a Fiber-Optic Cable*

Key Topic

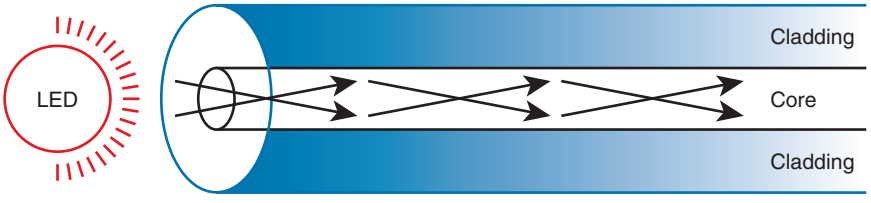



**Figure 2-15** *Transmission on Multimode Fiber with Internal Reflection*

The figure shows the normal operation of a **multimode fiber**, characterized by the fact that the cable allows for multiple angles (modes) of light waves entering the core.

In contrast, **single-mode fiber** uses a smaller-diameter core, around one-fifth the diameter of common multimode cables (see Figure 2-16). To transmit light into a much smaller core, a laser-based transmitter sends light at a single angle (hence the name *single-mode*).




**Figure 2-16** *Transmission on Single-Mode Fiber with Laser Transmitter*

Both multimode and single-mode cabling have important roles in Ethernet and meet different needs. Multimode improves the maximum distances over UTP, and it uses less expensive transmitters as compared with single-mode. Standards do vary; for instance, the standards for 10 Gigabit Ethernet over Fiber allow for distances up to 400m, which would often allow for connection of devices in different buildings in the same office park. Single-mode allows distances into the tens of kilometers, but with slightly more expensive SFP/SFP+ hardware.

To transmit between two devices, you need two cables, one for each direction, as shown in Figure 2-17. The concept works much like having two electrical circuits with the original UTP Ethernet standards. Note that the transmit port on one device connects to a cable that connects to a receive port on the other device, and vice versa with the other cable.

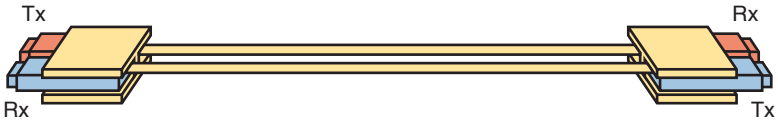



**Figure 2-17** *Two Fiber Cables with Tx Connected to Rx on Each Cable*

## Using Fiber with Ethernet

To use fiber with Ethernet switches, you need to use a switch with either built-in ports that support a particular optical Ethernet standard, or a switch with modular ports that allow you to change the Ethernet standard used on the port. Refer back to Figure 2-8, which shows a photo of a switch with two SFP+ ports, into which you could insert any of the supported SFP+ modules. Those SFP+ ports support a variety of 10-Gbps standards like those listed in Table 2-4.

**Table 2-4** A Sampling of IEEE 802.3 10-Gbps Fiber Standards

| Standard | Cable Type | Max Distance* |
|---|---|---|
| 10GBASE-S | MM | 400m |
| 10GBASE-LX4 | MM | 300m |
| 10GBASE-LR | SM | 10km |
| 10GBASE-E | SM | 30km |

* The maximum distances are based on the IEEE standards with no repeaters.

For instance, to build an Ethernet LAN in an office park, you might need to use some multimode and single-mode fiber links. In fact, many office parks might already have fiber cabling installed for the expected future use by the tenants in the buildings. If each building was within a few hundred meters of at least one other building, you could use multimode fiber between the buildings and connect switches to create your LAN.

**NOTE** Outside of the need to study for CCNA, if you need to look more deeply at fiber Ethernet and SFP/SFP+, check out tmgmatrix.cisco.com as a place to search for and learn about compatible SFP/SFP+ hardware from Cisco.

Although distance might be the first criterion to consider when thinking about whether to use UTP or fiber cabling, a few other tradeoffs exist as well. UTP wins again on cost, because the cost goes up as you move from UTP, to multimode, and then to single-mode, due to the extra cost for the transmitters like the SFP and SFP+ modules. UTP has some negatives, however. First, UTP might work poorly in some electrically noisy environments such as factories, because UTP can be affected by electromagnetic interference (EMI). Also, UTP cables emit a faint signal outside the cable, so highly secure networks may choose to use fiber, which does not create similar emissions, to make the network more secure. Table 2-5 summarizes these tradeoffs.

Key Topic

**Table 2-5** Comparisons Between UTP, MM, and SM Ethernet Cabling

| Criteria | UTP | Multimode | Single-Mode |
|---|---|---|---|
| Relative Cost of Cabling | Low | Medium | Medium |
| Relative Cost of a Switch Port | Low | Medium | High |
| Approximate Max Distance | 100m | 500m | 40km |
| Relative Susceptibility to Interference | Some | None | None |
| Relative Risk of Copying from Cable Emissions | Some | None | None |

# Sending Data in Ethernet Networks

Although physical layer standards vary quite a bit, other parts of the Ethernet standards work the same regardless of the type of physical Ethernet link. Next, this final major section of this chapter looks at several protocols and rules that Ethernet uses regardless of the type of link. In particular, this section examines the details of the Ethernet data-link layer protocol, plus how Ethernet nodes, switches, and hubs forward Ethernet frames through an Ethernet LAN.

## Ethernet Data-Link Protocols

One of the most significant strengths of the Ethernet family of protocols is that these protocols use the same data-link standard. In fact, the core parts of the data-link standard date back to the original Ethernet standards.

The Ethernet data-link protocol defines the Ethernet frame: an Ethernet header at the front, the encapsulated data in the middle, and an Ethernet trailer at the end. Ethernet actually defines a few alternate formats for the header, with the frame format shown in Figure 2-18 being commonly used today.

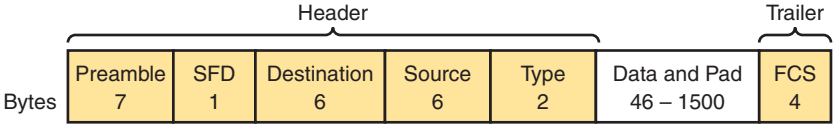



**Figure 2-18** *Commonly Used Ethernet Frame Format*

While all the fields in the frame matter, some matter more to the topics discussed in this book. Table 2-6 lists the fields in the header and trailer and a brief description for reference, with the upcoming pages including more detail about a few of these fields.

**Table 2-6** IEEE 802.3 Ethernet Header and Trailer Fields

| Field | Bytes | Description |
|---|---|---|
| Preamble | 7 | Synchronization. |
| Start Frame Delimiter (SFD) | 1 | Signifies that the next byte begins the Destination MAC Address field. |
| Destination MAC Address | 6 | Identifies the intended recipient of this frame. |
| Source MAC Address | 6 | Identifies the sender of this frame. |

| Field | Bytes | Description |
|---|---|---|
| **Type** | 2 | Defines the type of protocol listed inside the frame; today, most likely identifies IP version 4 (IPv4) or IP version 6 (IPv6). |
| **Data and Pad*** | 46–1500 | Holds data from a higher layer, typically an L3PDU (usually an IPv4 or IPv6 packet). The sender adds padding to meet the minimum length requirement for this field (46 bytes). |
| **Frame Check Sequence (FCS)** | 4 | Provides a method for the receiving NIC to determine whether the frame experienced transmission errors. |

* The IEEE 802.3 specification limits the data portion of the 802.3 frame to a minimum of 46 and a maximum of 1500 bytes. The term *maximum transmission unit* (MTU) defines the maximum Layer 3 packet that can be sent over a medium. Because the Layer 3 packet rests inside the data portion of an Ethernet frame, 1500 bytes is the largest IP MTU allowed over an Ethernet.

## Ethernet Addressing

The source and destination Ethernet address fields play a huge role in how Ethernet LANs work. The general idea for each is relatively simple: The sending node puts its own address in the source address field and the intended Ethernet destination device's address in the destination address field. The sender transmits the frame, expecting that the Ethernet LAN, as a whole, will deliver the frame to that correct destination.

**Ethernet addresses**, also called *Media Access Control (MAC) addresses*, are 6-byte-long (48-bit-long) binary numbers. For convenience, most computers list MAC addresses as 12-digit hexadecimal numbers. Cisco devices typically add some periods to the number for easier readability as well; for example, a Cisco switch might list a MAC address as 0000.0C12.3456.

Most MAC addresses represent a single NIC or other Ethernet port, so these addresses are often called a *unicast* Ethernet address. The term *unicast* is simply a formal way to refer to the fact that the address represents one interface to the Ethernet LAN. (This term also contrasts with two other types of Ethernet addresses, *broadcast* and *multicast*, which will be defined later in this section.)

The entire idea of sending data to a destination unicast MAC address works well, but it works only if all the unicast MAC addresses are unique. If two NICs tried to use the same MAC address, there could be confusion. (The problem would be like the confusion caused to the postal service if you and I both tried to use the same mailing address. Would the postal service deliver mail to your house or mine?) If two PCs on the same Ethernet tried to use the same MAC address, to which PC should frames sent to that MAC address be delivered?

Ethernet solves this problem using an administrative process so that, at the time of manufacture, all Ethernet devices are assigned a universally unique MAC address. Before a manufacturer can build Ethernet products, it must ask the IEEE to assign the manufacturer a universally unique 3-byte code, called the organizationally unique identifier (OUI). The manufacturer agrees to give all NICs (and other Ethernet products) a MAC address that begins with its assigned 3-byte OUI. The manufacturer also assigns a unique value for the last 3 bytes, a number that manufacturer has never used with that OUI. As a result, the MAC address of every device in the universe is unique.

**NOTE** The IEEE also calls these universal MAC addresses *global MAC addresses*.

Figure 2-19 shows the structure of the unicast MAC address, with the OUI.



Key Topic

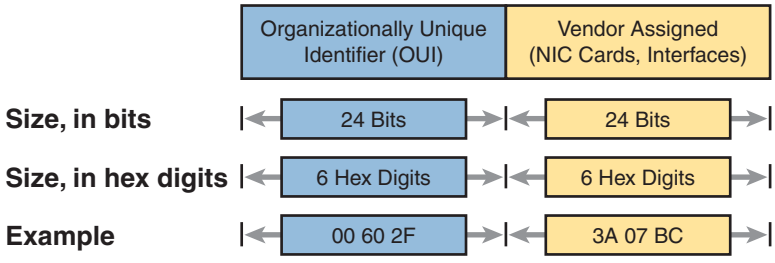



**Figure 2-19** *Structure of Unicast Ethernet Addresses*

Ethernet addresses go by many names: LAN address, Ethernet address, hardware address, burned-in address, physical address, universal address, or MAC address. For example, the term burned-in address (BIA) refers to the idea that a permanent MAC address has been encoded (burned into) the ROM chip on the NIC. As another example, the IEEE uses the term *universal address* to emphasize the fact that the address assigned to a NIC by a manufacturer should be unique among all MAC addresses in the universe.

In addition to unicast addresses, Ethernet also uses group addresses. *Group addresses* identify more than one LAN interface card. A frame sent to a group address might be delivered to a small set of devices on the LAN, or even to all devices on the LAN. In fact, the IEEE defines two general categories of group addresses for Ethernet:

**Broadcast address:** Frames sent to this address should be delivered to all devices on the Ethernet LAN. It has a value of FFFF.FFFF.FFFF.

**Multicast addresses:** Frames sent to a multicast Ethernet address will be copied and forwarded to a subset of the devices on the LAN that volunteers to receive frames sent to a specific multicast address.

Table 2-7 summarizes most of the details about MAC addresses.

**Table 2-7** LAN MAC Address Terminology and Features

| LAN Addressing Term or Feature | Description |
|---|---|
| **MAC address** | Media Access Control. 802.3 (Ethernet) defines the MAC sublayer of IEEE Ethernet. |
| Ethernet address, NIC address, LAN address | Other names often used instead of MAC address. These terms describe the 6-byte address of the LAN interface card. |
| Burned-in address | The 6-byte address assigned by the vendor making the card. |
| **Unicast address** | A term for a MAC address that represents a single LAN interface. |
| **Broadcast address** | An address that means "all devices that reside on this LAN right now." |
| Multicast address | On Ethernet, a multicast address implies some subset of all devices currently on the Ethernet LAN. |

### Identifying Network Layer Protocols with the Ethernet Type Field

While the Ethernet header's address fields play an important and more obvious role in Ethernet LANs, the Ethernet Type field plays a much less obvious role. The Ethernet Type field, or EtherType, sits in the Ethernet data-link layer header, but its purpose is to directly help the network processing on routers and hosts. Basically, the Type field identifies the type of network layer (Layer 3) packet that sits inside the Ethernet frame.

First, think about what sits inside the data part of the Ethernet frame shown earlier in Figure 2-19. Typically, it holds the network layer packet created by the network layer protocol on some device in the network. Over the years, those protocols have included IBM Systems Network Architecture (SNA), Novell NetWare, Digital Equipment Corporation's DECnet, and Apple Computer's AppleTalk. Today, the most common network layer protocols are both from TCP/IP: IP version 4 (IPv4) and IP version 6 (IPv6).

The original host has a place to insert a value (a hexadecimal number) to identify the type of packet encapsulated inside the Ethernet frame. However, what number should the sender put in the header to identify an IPv4 packet as the type? Or an IPv6 packet? As it turns out, the IEEE manages a list of EtherType values, so that every network layer protocol that needs a unique EtherType value can have a number. The sender just has to know the list. (Anyone can view the list; just go to www.ieee.org and search for *EtherType*.)

For example, a host can send one Ethernet frame with an IPv4 packet and the next Ethernet frame with an IPv6 packet. Each frame would have a different Ethernet Type field value, using the values reserved by the IEEE, as shown in Figure 2-20.

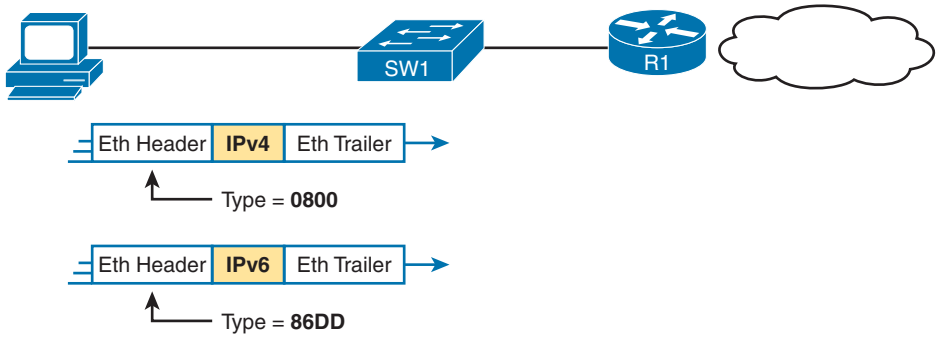



**Figure 2-20** *Use of Ethernet Type Field*

### Error Detection with FCS

Ethernet also defines a way for nodes to find out whether a frame's bits changed while crossing over an Ethernet link. (Usually, the bits could change because of some kind of electrical interference, or a bad NIC.) Ethernet, like most data-link protocols, uses a field in the data-link trailer for the purpose of error detection.

The Ethernet **Frame Check Sequence** *(FCS)* field in the Ethernet trailer—the only field in the Ethernet trailer—gives the receiving node a way to compare results with the sender, to discover whether errors occurred in the frame. The sender applies a complex math formula to the frame before sending it, storing the result of the formula in the FCS field. The receiver applies the same math formula to the received frame. The receiver then compares its own results with the sender's results. If the results are the same, the frame did not change; otherwise, an error occurred and the receiver discards the frame.

Note that *error detection* does not also mean *error recovery*. Ethernet defines that the errored frame should be discarded, but Ethernet does not attempt to recover the lost frame. Other protocols, notably TCP, recover the lost data by noticing that it is lost and sending the data again.

## Sending Ethernet Frames with Switches and Hubs

Ethernet LANs behave slightly differently depending on whether the LAN has mostly modern devices, in particular, LAN switches instead of some older LAN devices called LAN hubs. Basically, the use of more modern switches allows the use of full-duplex logic, which is much faster and simpler than half-duplex logic, which is required when using hubs. The final topic in this chapter looks at these basic differences.

### Sending in Modern Ethernet LANs Using Full Duplex

Modern Ethernet LANs use a variety of Ethernet physical standards, but with standard Ethernet frames that can flow over any of these types of physical links. Each individual link can run at a different speed, but each link allows the attached nodes to send the bits in the frame to the next node. They must work together to deliver the data from the sending Ethernet node to the destination node.

The process is relatively simple, on purpose; the simplicity lets each device send a large number of frames per second. Figure 2-21 shows an example in which PC1 sends an Ethernet frame to PC2.

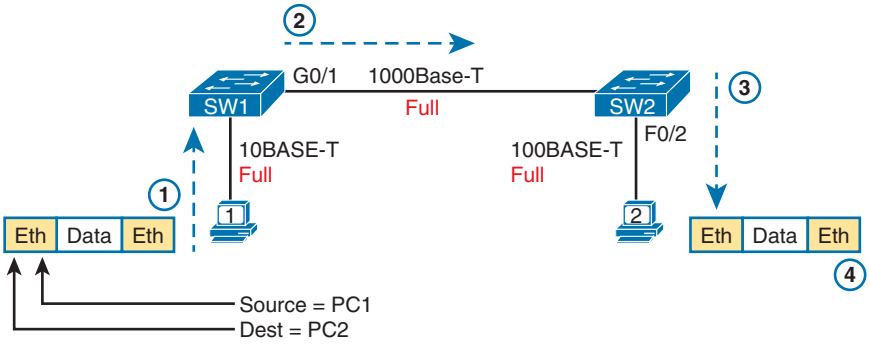



**Figure 2-21** *Example of Sending Data in a Modern Ethernet LAN*

Following the steps in the figure:

1. PC1 builds and sends the original Ethernet frame, using its own MAC address as the source address and PC2's MAC address as the destination address.
2. Switch SW1 receives and forwards the Ethernet frame out its G0/1 interface (short for Gigabit interface 0/1) to SW2.
3. Switch SW2 receives and forwards the Ethernet frame out its F0/2 interface (short for Fast Ethernet interface 0/2) to PC2.
4. PC2 receives the frame, recognizes the destination MAC address as its own, and processes the frame.

The Ethernet network in Figure 2-21 uses full duplex on each link, but the concept might be difficult to see.

Full duplex means that the NIC or switch port has no half-duplex restrictions. So, to understand full duplex, you need to understand half duplex, as follows:

Key Topic

**Half duplex:** The device must wait to send if it is currently receiving a frame; in other words, it cannot send and receive at the same time.

**Full duplex:** The device does not have to wait before sending; it can send and receive at the same time.

So, with all PCs and LAN switches, and no LAN hubs, all the nodes can use full duplex. All nodes can send and receive on their port at the same instant in time. For example, in Figure 2-21, PC1 and PC2 could send frames to each other simultaneously, in both directions, without any half-duplex restrictions.

### Using Half Duplex with LAN Hubs

To understand the need for half-duplex logic in some cases, you have to understand a little about an older type of networking device called a LAN hub. When the IEEE first introduced 10BASE-T in 1990, Ethernet switches did not exist yet; instead, networks used a device called a LAN hub. Like a switch, a LAN hub provided a number of RJ-45 ports as a place to connect links to PCs; however, hubs used different rules for forwarding data.

LAN hubs forward data using physical layer standards rather than data-link standards and are therefore considered to be Layer 1 devices. When an electrical signal comes in one hub port, the hub repeats that electrical signal out all other ports (except the incoming port). By doing so, the data reaches all the rest of the nodes connected to the hub, so the data hopefully reaches the correct destination. The hub has no concept of Ethernet frames, of addresses, making decisions based on those addresses, and so on.

The downside of using LAN hubs is that if two or more devices transmitted a signal at the same instant, the electrical signals collide and become garbled. The hub repeats all received electrical signals, even if it receives multiple signals at the same time. For example, Figure 2-22 shows the idea, with PCs Archie and Bob sending an electrical signal at the same instant of time (at Steps 1A and 1B) and the hub repeating both electrical signals out toward Larry on the left (Step 2).

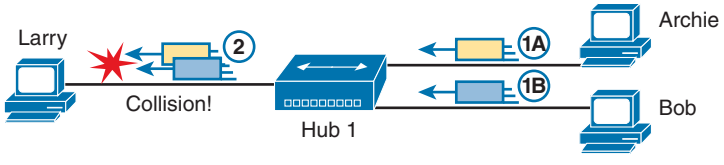



**Figure 2-22** *Collision Occurring Because of LAN Hub Behavior*

**NOTE** For completeness, note that the hub floods each frame out all other ports (except the incoming port). So, Archie's frame goes to both Larry and Bob; Bob's frame goes to Larry and Archie.

If you replace the hub in Figure 2-22 with a LAN switch, the switch prevents the collision on the left. The switch operates as a Layer 2 device, meaning that it looks at the data-link header and trailer. A switch would look at the MAC addresses, and even if the switch needed

to forward both frames to Larry on the left, the switch would send one frame and queue the other frame until the first frame was finished.

Now back to the issue created by the hub's logic: collisions. To prevent these collisions, the Ethernet nodes must use half-duplex logic instead of full-duplex logic. A problem occurs only when two or more devices send at the same time; half-duplex logic tells the nodes that if someone else is sending, wait before sending.

For example, back in Figure 2-22, imagine that Archie began sending his frame early enough so that Bob received the first bits of that frame before Bob tried to send his own frame. Bob, at Step 1B, would notice that he was receiving a frame from someone else, and using half-duplex logic, would simply wait to send the frame listed at Step 1B.

Nodes that use half-duplex logic actually use a relatively well-known algorithm called carrier sense multiple access with collision detection (CSMA/CD). The algorithm takes care of the obvious cases but also the cases caused by unfortunate timing. For example, two nodes could check for an incoming frame at the exact same instant, both realize that no other node is sending, and both send their frames at the exact same instant, causing a collision. CSMA/CD covers these cases as well, as follows:

**Step 1.** A device with a frame to send listens until the Ethernet is not busy.

**Step 2.** When the Ethernet is not busy, the sender begins sending the frame.

**Step 3.** The sender listens while sending to discover whether a collision occurs; collisions might be caused by many reasons, including unfortunate timing. If a collision occurs, all currently sending nodes do the following:

- **a.** They send a jamming signal that tells all nodes that a collision happened.
- **b.** They independently choose a random time to wait before trying again, to avoid unfortunate timing.
- **c.** The next attempt starts again at Step 1.

Although most modern LANs do not often use hubs and therefore do not need to use half duplex, enough old hubs still exist in enterprise networks so that you need to be ready to understand duplex issues. Each NIC and switch port has a duplex setting. For all links between PCs and switches, or between switches, use full duplex. However, for any link connected to a LAN hub, the connected LAN switch and NIC port should use half duplex. Note that the hub itself does not use half-duplex logic, instead just repeating incoming signals out every other port.

Figure 2-23 shows an example, with full-duplex links on the left and a single LAN hub on the right. The hub then requires SW2's F0/2 interface to use half-duplex logic, along with the PCs connected to the hub.

Key Topi

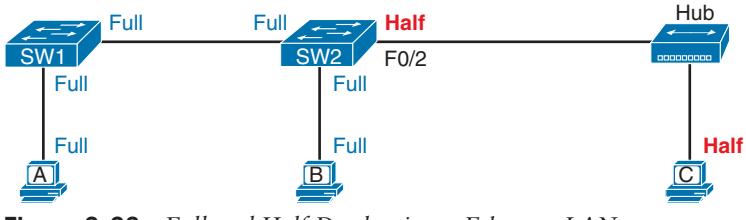



**Figure 2-23** *Full and Half Duplex in an Ethernet LAN*

Before I close the chapter, note that the discussion of full and half duplex connects to two specific terms from CCNA exam topic 1.3.b, but those connections may not be obvious. First, the term *Ethernet shared media* (from the exam topic) refers to designs that use hubs, require CSMA/CD, and therefore share the bandwidth. The idea behind the term comes from the fact that the devices connected to the hub share the network because they must use CSMA/CD, and CSMA/CD enforces rules that allow only one device to successfully send a frame at any point in time.

By contrast, the term *Ethernet point-to-point* in that same exam topic emphasizes the fact that in a network built with switches, each (point-to-point) link works independently of the others. Because of the full-duplex logic discussed in this section, a frame can be sent on every point-to-point link in an Ethernet at the same time.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 2-8 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 2-8** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |

## Review All the Key Topics

Key Topic

**Table 2-9** Key Topics for Chapter 2

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 2-3 | Drawing of a typical wired and wireless enterprise LAN | 40 |
| Table 2-2 | Several types of Ethernet LANs and some details about each | 41 |
| Figure 2-9 | Conceptual drawing of transmitting in one direction each over two different electrical circuits between two Ethernet nodes | 47 |
| Figure 2-10 | 10- and 100-Mbps Ethernet straight-through cable pinouts | 47 |
| Figure 2-11 | 10- and 100-Mbps Ethernet crossover cable pinouts | 48 |
| Table 2-3 | List of devices that transmit on wire pair 1,2 and pair 3,6 | 48 |
| Figure 2-12 | Typical uses for straight-through and crossover Ethernet cables | 49 |
| Figure 2-15 | Physical transmission concepts in a multimode cable | 51 |
| Table 2-5 | Comparison between UTP, MM, and SM Ethernet cabling | 53 |
| Figure 2-19 | Format of Ethernet MAC addresses | 55 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Definitions of half duplex and full duplex | 58 |
| Figure 2-23 | Examples of which interfaces use full duplex and which interfaces use half duplex | 59 |



## Key Terms You Should Know

10BASE-T, 100BASE-T, 1000BASE-T, auto-MDIX, broadcast address, cladding, core, crossover cable, electromagnetic interference (EMI), Ethernet, Ethernet address, Ethernet frame, Ethernet link, Ethernet port, Fast Ethernet, fiber-optic cable, Frame Check Sequence, Gigabit Ethernet, IEEE, MAC address, multimode fiber, network interface card (NIC), RJ-45, single-mode fiber, straight-through cable, transceiver, unicast address, wired LAN, wireless LAN



CHAPTER 3

# Fundamentals of WANs and IP Routing

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.a Routers**

**1.2 Describe characteristics of network topology architectures**

**1.2.d WAN**

This chapter introduces WANs and the various features of the TCP/IP network layer.

First, for WANs, note that the current CCNA blueprint does not examine WANs in detail as an end to themselves. However, to understand IP routing, you need to understand the basics of the two types of WAN links introduced in the first major section of this chapter: serial links and Ethernet WAN links. In their most basic form, these WAN links connect routers that sit at sites that can be miles to hundreds of miles apart, allowing communications between remote sites.

The rest of the chapter then turns to the TCP/IP Network layer, with IP as the center of the discussion. The second section of the chapter discusses the major features of IP: routing, addressing, and routing protocols. The final section of the chapter examines a few protocols other than IP that also help the TCP/IP Network layer create a network that allows end-to-end communication between endpoints.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 3-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Wide-Area Networks | 1, 2 |
| IP Routing | 3–6 |
| Other Network Layer Features | 7 |

1. Which of the following terms is not commonly used to describe a serial link?
   a. Private line
   b. Point-to-point link
   c. Leased circuit
   d. E-line
2. Two routers, R1 and R2, connect using an Ethernet WAN service. The service provides point-to-point service between these two routers only, as a Layer 2 Ethernet service. Which of the following are the most likely to be true about this WAN? (Choose two answers.)
   a. R1 will connect to a physical Ethernet link, with the other end of the cable connected to R2.
   b. R1 will connect to a physical Ethernet link, with the other end of the cable connected to a device at the WAN service provider point of presence.
   c. R1 will forward data-link frames to R2 using an HDLC header/trailer.
   d. R1 will forward data-link frames to R2 using an Ethernet header/trailer.
3. Imagine a network in which PC1 connects to the same Ethernet LAN as Router1, PC2 connects to the same LAN as Router2, and the two routers connect to each other with a PPP serial link. When PC1 sends data to PC2, and Router1 removes the Ethernet header and trailer, which of the following is true?
   a. Router1 does not use the removed Ethernet header/trailer again.
   b. Router1 re-encapsulates the packet in a PPP frame that uses the Ethernet addresses from the removed header.
   c. Router1 also removes the IP header before forwarding the data to Router2.
   d. Router1 re-encapsulates the packet in a new Ethernet frame before forwarding the packet to Router2.
4. Which of the following does a router normally use when making a decision about routing TCP/IP packets?
   a. Destination MAC address
   b. Source MAC address
   c. Destination IP address
   d. Source IP address
   e. Destination MAC and IP addresses
5. Which of the following are true about a LAN-connected TCP/IP host and its IP routing (forwarding) choices?
   a. The host always sends packets to its default gateway.
   b. The host never sends packets to its default gateway.
   c. The host sends packets to its default gateway if the destination IP address is in a different subnet than the host.
   d. The host sends packets to its default gateway if the destination IP address is in the same subnet as the host.

6. Which of the following are functions of a routing protocol? (Choose two answers.)
   a. Advertising known routes to neighboring routers
   b. Learning routes to directly connected subnets
   c. Learning routes as advertised to the router by neighboring routers
   d. Forwarding IP packets based on a packet's destination IP address
7. A company implements a TCP/IP network, with PC1 sitting on an Ethernet LAN. Which of the following protocols and features requires PC1 to learn information from some other server device?
   a. ARP
   b. ping
   c. DNS
   d. None of these answers are correct.

# Foundation Topics

## Wide-Area Networks

Imagine a typical day at the branch office at some enterprise. The user sits at some endpoint device: a PC, tablet, phone, and so on. It connects to a LAN, either via an Ethernet cable or using a wireless LAN. However, the user happens to be checking information on a website, and that web server sits at the home office of the company. To make that work, the data travels over one or more **wide-area network (WAN)** links.

WAN technologies define the physical (Layer 1) standards and data-link (Layer 2) protocols used to communicate over long distances. This first section examines two WAN technologies: leased-line WANs and Ethernet WANs. Leased-line WANs have been an option since the 1960s, but are seldom used today. However, because leased-line WANs have been around many years, and have been a part of CCNA for its entire history, you need to know the basics of leased-line WANs for the current CCNA.

Conversely, companies use *Ethernet WAN* links much more often today as compared to leased lines. Those links use the same Ethernet data-link protocols as discussed in the previous chapter, but they use physical layer Ethernet standards that work over the much longer distances required for WANs. The next few pages examine leased-line WANs first, followed by Ethernet WANs.

### Leased-Line WANs

To connect LANs using a WAN, the internetwork uses a router connected to each LAN, with a WAN link between the routers. For the WAN link, the enterprise's network engineer must do some planning and then order some kind of WAN link from a WAN service provider. That provider installs the WAN link between the two routers, as shown in Figure 3-1. Note that drawings of leased-line WAN links represent the link with a crooked line (a lightning bolt) to imply the WAN link is a leased line.

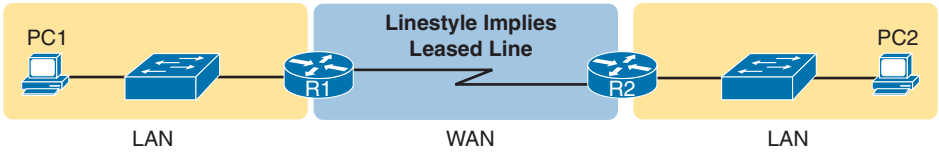



**Figure 3-1** *Small Enterprise Network with One Leased Line*

This section begins by examining the physical details of leased lines, followed by a discussion of the two common data-link protocols for leased lines, HDLC and PPP.



## Physical Details of Leased Lines

The **leased line** service, a physical layer service, delivers bits in both directions, at a predetermined speed, using full-duplex logic. In fact, conceptually it acts as if you had a full-duplex crossover Ethernet link between two routers, as shown in Figure 3-2. The leased line uses two pairs of wires, one pair for each direction of sending data, which allows full-duplex operation.

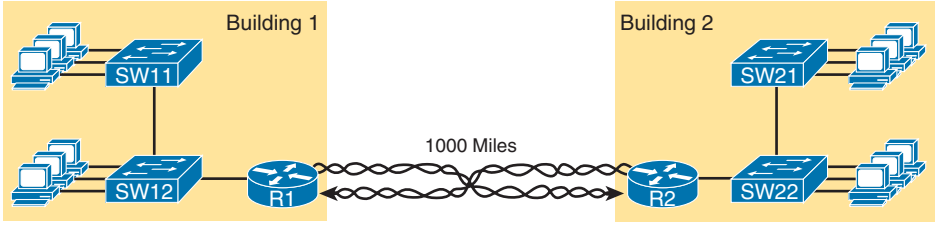



**Figure 3-2** *Conceptual View of the Leased-Line Service*

To create a leased line, some physical path must exist between the two routers on the ends of the link. The physical cabling must leave the customer buildings where each router sits; however, the **telco** does not simply install one cable between the two buildings. Instead, it uses what is typically a large and complex network that creates the appearance of a cable between the two routers.

Figure 3-3 shows a conceptual view of a small part of the telco network. Telcos put their equipment in buildings called central offices (COs). The telco installs cables from the CO to most every other building in the city, expecting to sell services to the people in those buildings one day. The telco would then configure its switches to use some of the capacity on each cable to send data in both directions, creating the equivalent of a crossover cable between the two routers.

Given the long history of leased lines, the industry uses a variety of terms to refer to the same kind of WAN link. For instance, the term *leased line* emphasizes the fact that the telco leases the use of the leased line to a customer, but the customer does not permanently own the line. Table 3-2 lists some of the many names for leased lines, mainly so that in a networking job, you have a chance to translate from the terms each person uses with a basic description as to the meaning of the name.

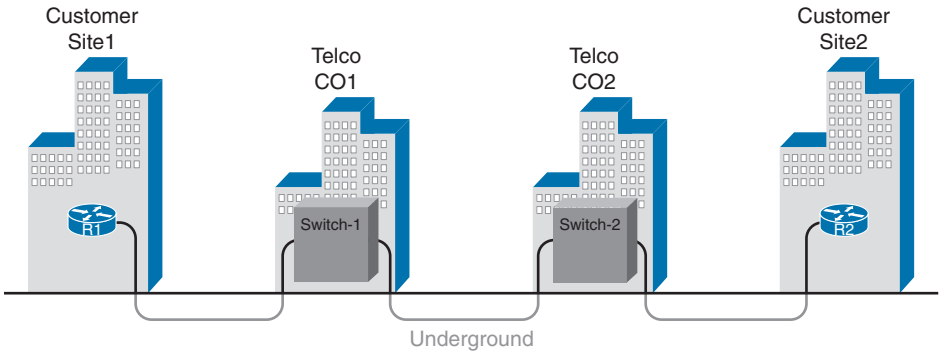



**Figure 3-3** *Possible Cabling Inside a Telco for a Short Leased Line*

**Table 3-2** Different Names for a Leased Line

| Name | Meaning or Reference |
|---|---|
| Leased circuit, Circuit | The words *line* and *circuit* are often used as synonyms in telco terminology; *circuit* makes reference to the electrical circuit between the two endpoints. |
| Serial link, Serial line | The words *link* and *line* are also often used as synonyms. *Serial* in this case refers to the fact that the bits flow serially and that routers use **serial interfaces**. |
| Point-to-point link, Point-to-point line | These terms refer to the fact that the topology stretches between two points, and two points only. (Some older leased lines allowed more than two devices.) |
| T1 | This specific type of leased line transmits data at 1.544 megabits per second (1.544 Mbps). |
| WAN link, Link | Both of these terms are very general, with no reference to any specific technology. |
| Private line | This term refers to the fact that the data sent over the line cannot be copied by other telco customers, so the data is private. |

## Data-Link Details of Leased Lines

A leased line provides a Layer 1 service. In other words, it promises to deliver bits between the devices connected to the leased line. However, the leased line itself does not define a data-link layer protocol to be used on the leased line. So, to make use of the leased line, the routers on the ends of the line use one of two data-link protocols: High-Level Data Link Control (**HDLC**) or Point-to-Point Protocol (PPP).

All data-link protocols perform a similar role: to control the correct delivery of data over a physical link of a particular type. For example, the Ethernet data-link protocol uses a destination address field to identify the correct device that should receive the data and an FCS field that allows the receiving device to determine whether the data arrived correctly. The WAN data-link protocols HDLC and PPP provide similar functions.

---

Answers to the "Do I Know This Already?" quiz:

**1** D **2** B, D **3** A **4** C **5** C **6** A, C **7** C

HDLC and PPP have even less work to do than Ethernet because of the simple point-to-point topology of a leased line. Using HDLC as an example, when one router sends an HDLC frame over a leased line, the frame can go to only one destination device: to the router on the other end of the link. So, while HDLC has an address field, the destination is implied, and the actual address value is unimportant. The idea is sort of like when I have lunch with my friend Gary, and only Gary. I do not need to start every sentence with "Hey, Gary." He knows I am talking to him.

Figure 3-4 shows the HDLC frame format as an example, with Table 3-3 that follows describing the fields. However, note that HDLC and PPP have a similar frame format, although the newer PPP (defined in the 1990s) has more features and functions (plus additional optional headers) than the older HDLC (defined in the 1970s.)



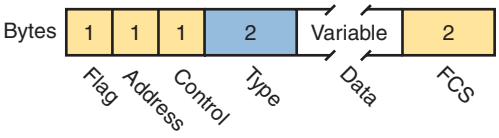



**Figure 3-4** *HDLC Framing*

**Table 3-3** Comparing HDLC and PPP Header Fields to Ethernet

| HDLC & PPP Field | Ethernet Equivalent | Description |
|---|---|---|
| Flag | Preamble, SFD | Lists a recognizable bit pattern so that the receiving nodes realize that a new frame is arriving. |
| Address | Destination Address | Identifies the destination device, but not interesting for leased line point-to-point topologies. |
| Control | N/A | No longer of use today for links between routers. |
| Type | Type | Identifies the type of Layer 3 packet encapsulated inside the data portion of the frame. |
| FCS | FCS | Identifies a field used by the error detection process. (It is the only trailer field in this table.) |

## How Routers Use a WAN Data Link

Leased lines connect to routers, and routers focus on delivering packets to a destination host. However, routers physically connect to both LANs and WANs, with those LANs and WANs requiring that data be sent inside data-link frames. So, now that you know a little about HDLC and PPP, it helps to think about how routers use these data-link protocols when sending data.

First, the TCP/IP network layer focuses on forwarding **IP packets** from the sending host to the destination host. The underlying LANs and WANs just act as a way to move the packets to the next router or end-user device. Figure 3-5 shows that network layer perspective.

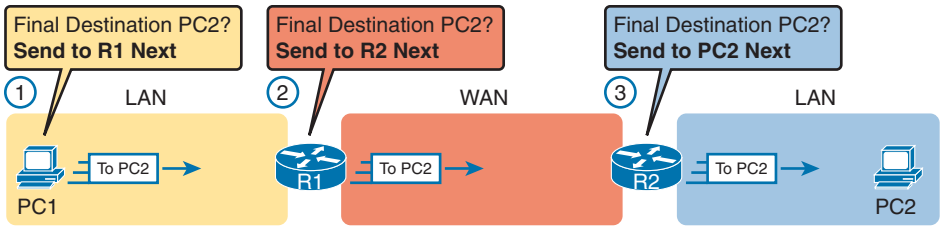



**Figure 3-5** *IP Routing Logic over LANs and WANs*

Following the steps in the figure, for a packet sent by PC1 to PC2's IP address:

1. PC1's network layer (IP) logic tells it to send the packet to a nearby router (R1).
2. Router R1's network layer logic tells it to forward (route) the packet out the leased line to Router R2 next.
3. Router R2's network layer logic tells it to forward (route) the packet out the LAN link to PC2 next.

While Figure 3-5 shows the network layer logic, the PCs and routers must rely on the LANs and WANs in the figure to actually move the bits in the packet. Figure 3-6 shows the same figure, with the same packet, but this time showing some of the data-link layer logic used by the hosts and routers. Basically, three separate data-link layer steps encapsulate the packet, inside a data-link frame, over three hops through the internetwork: from PC1 to R1, from R1 to R2, and from R2 to PC2.

Key Topic

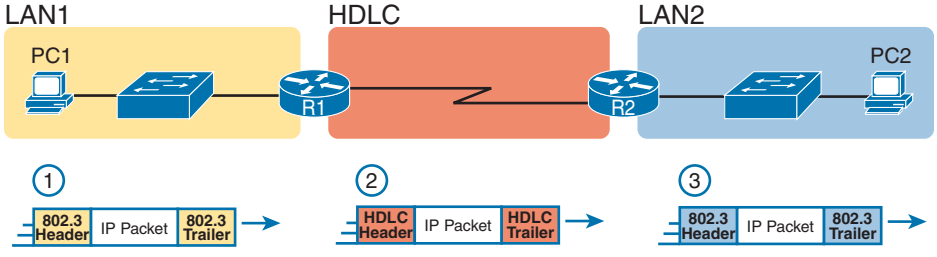



**Figure 3-6** *General Concept of Routers De-encapsulating and Re-encapsulating IP Packets*

Following the steps in the figure, again for a packet sent by PC1 to PC2's IP address:

1. To send the IP packet to Router R1 next, PC1 encapsulates the IP packet in an Ethernet frame that has the destination MAC address of R1.
2. Router R1 de-encapsulates (removes) the IP packet from the Ethernet frame, encapsulates (inserts) the packet into an HDLC frame using an HDLC header and trailer, and forwards the HDLC frame to Router R2 next.
3. Router R2 de-encapsulates (removes) the IP packet from the HDLC frame, encapsulates (inserts) the packet into an Ethernet frame that has the destination MAC address of PC2, and forwards the Ethernet frame to PC2.

In summary, a leased line with HDLC creates a WAN link between two routers so that they can forward packets for the devices on the attached LANs. The leased line itself provides the

physical means to transmit the bits, in both directions. The HDLC frames provide the means to encapsulate the network layer packet correctly so that it crosses the link between routers. Similarly, if the routers use PPP instead of HDLC, then the routers encapsulate the packets in PPP frames rather than HDLC frames.

Leased lines have many benefits that have led to their relatively long life in the WAN marketplace. These lines are simple for the customer, are widely available, are of high quality, and are private. However, they do have some negatives as well compared to newer WAN technologies, including a higher cost and typically longer lead times to get the service installed. Additionally, by today's standards, leased-line WAN links are slow, with the fastest leased line speeds in the tens of megabits per second (Mbps). Newer and faster WAN technology has moved the WAN market away from leased lines, with Ethernet WANs being common here in the 2020s.



## Ethernet as a WAN Technology

For the first several decades of the existence of Ethernet, Ethernet was only appropriate for LANs. The restrictions on cable lengths and devices might allow a LAN that stretched a kilometer or two, to support a campus LAN, but that was the limit.

As time passed, the IEEE improved Ethernet standards in ways that made Ethernet a reasonable WAN technology. For example, the 1000BASE-LX standard uses single-mode fiber cabling, with support for a 5-km cable length; the 1000BASE-ZX standard supports an even longer 70-km cable length. As time went by, and as the IEEE improved cabling distances for fiber Ethernet links, Ethernet became a reasonable WAN technology.

Today, many WAN service providers (SP) offer WAN services that take advantage of Ethernet. SPs offer a wide variety of these Ethernet WAN services, with many different names. But all of them use a similar model, with Ethernet used between the customer site and the SP's network, as shown in Figure 3-7.

Key Topic

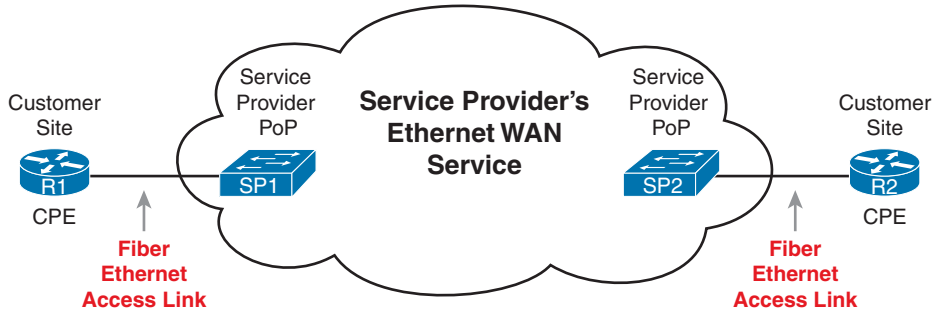



**Figure 3-7** *Fiber Ethernet Link to Connect a CPE Router to a Service Provider's WAN*

The model shown in Figure 3-7 has many of the same ideas of how a telco creates a leased line, as shown earlier in Figure 3-3, but now with Ethernet links and devices. The customer connects to an Ethernet link using a router interface. The (fiber) Ethernet link leaves the customer building and connects to some nearby SP location called a point of presence (PoP). Instead of a telco switch as shown in Figure 3-3, the SP uses an Ethernet switch. Inside the SP's network, the SP uses any technology that it wants to create the specific Ethernet WAN services.

### Ethernet WANs That Create a Layer 2 Service

Ethernet WAN services include a variety of specific services that vary in ways that change how routers use those services. However, for the purposes of CCNA, you just need to understand the most basic Ethernet WAN service, one that works much like an Ethernet crossover cable—just over a WAN. In other words:

- Logically, behaves like a point-to-point connection between two routers
- Physically, behaves as if a physical fiber Ethernet link existed between the two routers

This book refers to this particular Ethernet WAN service with a couple of the common names:

Key Topic

**Ethernet WAN:** A generic name to differentiate it from an Ethernet LAN.

**Ethernet point-to-point link:** A term that emphasizes the topology of a typical Ethernet WAN link that has exactly two endpoints: the routers on the two ends of the link.

**Ethernet Line Service (E-Line):** A term from the Metro Ethernet Forum (MEF) for the kind of point-to-point Ethernet WAN service shown throughout this book.

**Ethernet over MPLS (EoMPLS):** A term that refers to Multiprotocol Label Switching (MPLS), a technology that can be used to create the Ethernet service for the customer.

So, if you can imagine two routers, with a single Ethernet link between the two routers, you understand what this particular Ethernet WAN service does, as shown in Figure 3-8. In this case, the two routers, R1 and R2, connect with an Ethernet WAN service instead of a serial link. The routers use Ethernet interfaces, and they can send data in both directions at the same time. Physically, each router actually connects to some SP PoP, as shown earlier in Figure 3-7, but logically, the two routers can send Ethernet frames to each other over the link.

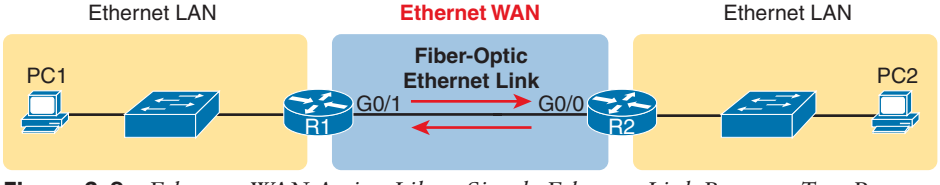



**Figure 3-8** *Ethernet WAN Acting Like a Simple Ethernet Link Between Two Routers*

### How Routers Route IP Packets Using Ethernet WAN Links

WANs, by their very nature, give IP routers a way to forward IP packets from a LAN at one site, over the WAN, and to another LAN at another site. The routing logic works the same whether the WAN link happens to be a serial link or an Ethernet WAN link, with the encapsulation details being slightly different. With an Ethernet WAN link, the link uses Ethernet for both Layer 1 and Layer 2 functions, so the routers encapsulate using the familiar Ethernet header and trailer, as shown in the middle of Figure 3-9. Also, note that the figure shows a small cloud over the Ethernet link as a way to tell us that the link is an Ethernet WAN link, rather than an Ethernet LAN link.

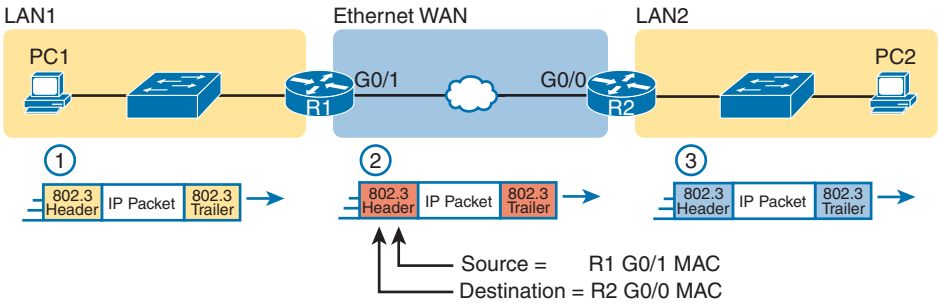



**Figure 3-9** *Routing over an Ethernet WAN Link*



**NOTE** The 802.3 headers/trailers in the figure are different at each stage! Make sure to notice the reasons in the step-by-step explanations that follow.

The figure shows the same three routing steps as shown with the serial link in the earlier Figure 3-6. In this case, all three routing steps use the same Ethernet (802.3) protocol. However, note that each frame's data-link header and trailer are different. Each router discards the old data-link header/trailer and adds a new set, as described in these steps. Focus mainly on Step 2, because compared to the similar example shown in Figure 3-6, Steps 1 and 3 are unchanged:

1. To send the IP packet to Router R1 next, PC1 encapsulates (inserts) the IP packet in an Ethernet frame that has the destination MAC address of R1.
2. Router R1 de-encapsulates (removes) the IP packet from the Ethernet frame and encapsulates (inserts) the packet into a new Ethernet frame, with a new Ethernet header and trailer. The destination MAC address is R2's G0/0 MAC address, and the source MAC address is R1's G0/1 MAC address. R1 forwards this frame over the Ethernet WAN service to R2 next.
3. Router R2 de-encapsulates (removes) the IP packet from the Ethernet frame, encapsulates (inserts) the packet into an Ethernet frame that has the destination MAC address of PC2, and forwards the Ethernet frame to PC2.

Throughout this book, the WAN links (serial and Ethernet) will connect routers as shown here, with the focus being on the LANs and IP routing. The rest of the chapter turns our attention to a closer look at IP routing.

## IP Routing

Many protocol models have existed over the years, but today the TCP/IP model dominates. And at the network layer of TCP/IP, two options exist for the main protocol around which all other network layer functions revolve: IP version 4 (IPv4) and IP version 6 (IPv6). Both IPv4 and IPv6 define the same kinds of network layer functions, but with different details. This chapter introduces these network layer functions for IPv4, leaving the IPv6 details for Part VII of this book.

**NOTE** All references to IP in this chapter refer to the older and more established IPv4, and all use of the term **IP address** refers to an IPv4 address.

Internet Protocol (IP) focuses on the job of routing data, in the form of IP packets, from the source host to the destination host. IP does not concern itself with the physical transmission of data, instead relying on the lower TCP/IP layers for those functions. Instead, IP concerns itself with the logical details, rather than physical details, of delivering data. In particular, the network layer specifies how packets travel end to end over a TCP/IP network, even when the packet crosses many different types of LAN and WAN links.

This next major section of the chapter examines IP routing in more depth. First, IP defines what it means to route an IP packet from sending host to destination host, while using successive data-link protocols. This section then examines how IP addressing rules help to make IP routing much more efficient by grouping addresses into subnets. This section closes by looking at the role of IP **routing protocols**, which give routers a means by which to learn routes to all the IP subnets in an internetwork.

## Network Layer Routing (Forwarding) Logic

Routers and end-user computers (called *hosts* in a TCP/IP network) work together to perform IP routing. The host operating system (OS) has TCP/IP software, including the software that implements the network layer. Hosts use that software to choose where to send IP packets, often to a nearby router. Those routers make choices of where to send the IP packet next. Together, the hosts and routers deliver the IP packet to the correct destination, as shown in the example in Figure 3-10.

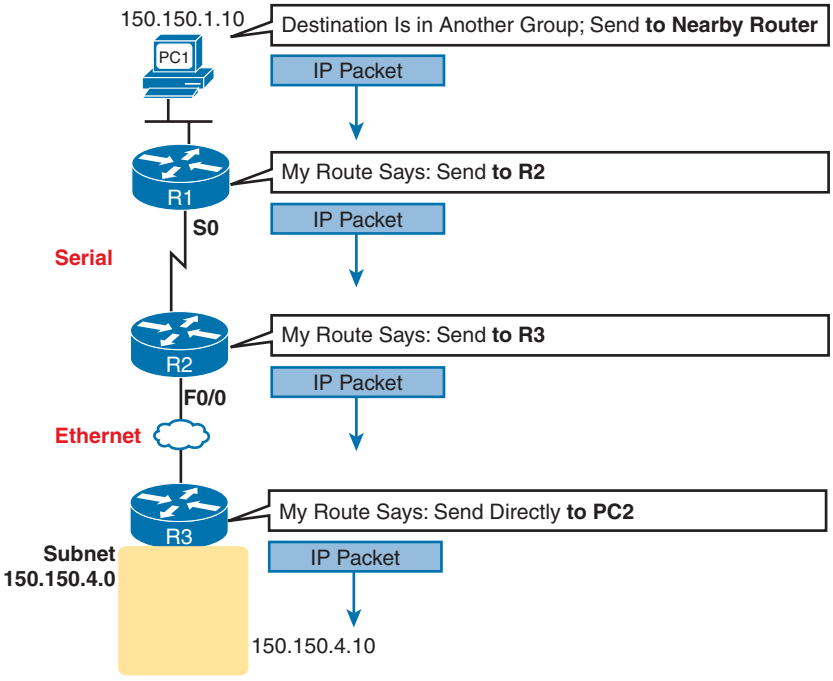



**Figure 3-10** *Routing Logic: PC1 Sending an IP Packet to PC2*

The IP packet, created by PC1, goes from the top of the figure all the way to PC2 at the bottom of the figure. The next few pages discuss the network layer routing logic used by each device along the path.

**NOTE** The term *path selection* is sometimes used to refer to the routing process shown in Figure 3-10. At other times, it refers to routing protocols, specifically how routing protocols select the best route among the competing routes to the same destination.



### Host Forwarding Logic: Send the Packet to the Default Router

In this example, PC1 does some basic analysis and then chooses to send the IP packet to the router so that the router will forward the packet. PC1 analyzes the destination address and realizes that PC2's address (150.150.4.10) is not on the same LAN as PC1. So PC1's logic tells it to send the packet to a device whose job it is to know where to route data: a nearby router, on the same LAN, called PC1's **default router**.

To send the IP packet to the default router, the sender sends a data-link frame across the medium to the nearby router; this frame includes the packet in the data portion of the frame. That frame uses data-link layer (Layer 2) addressing in the data-link header to ensure that the nearby router receives the frame.

**NOTE** The *default router* is also referred to as the *default gateway*.

### R1 and R2's Logic: Routing Data Across the Network

All routers use the same general process to route the packet. Each router keeps an *IP* **routing table**. This table lists IP address *groupings*, called **IP networks** and **IP subnets**. When a router receives a packet, it compares the packet's destination IP address to the entries in the routing table and makes a match. This matching entry also lists directions that tell the router where to forward the packet next.

In Figure 3-10, R1 would have matched the destination address (150.150.4.10) to a routing table entry, which in turn told R1 to send the packet to R2 next. Similarly, R2 would have matched a routing table entry that told R2 to send the packet, over an Ethernet WAN link, to R3 next.

The routing concept works a little like driving down the freeway when approaching a big interchange. You look up and see signs for nearby towns, telling you which exits to take to go to each town. Similarly, the router looks at the IP routing table (the equivalent of the road signs) and directs each packet over the correct next LAN or WAN link (the equivalent of a road).

### R3's Logic: Delivering Data to the End Destination

The final router in the path, R3, uses almost the same logic as R1 and R2, but with one minor difference. R3 needs to forward the packet directly to PC2, not to some other router. On the surface, that difference seems insignificant. In the next section, when you read about how the network layer uses LANs and WANs, the significance of the difference will become obvious.

## How Network Layer Routing Uses LANs and WANs

While the network layer routing logic ignores the physical transmission details, the bits still have to be transmitted. To do that work, the network layer logic in a host or router must

hand off the packet to the data-link layer protocols, which, in turn, ask the physical layer to actually send the data. The data-link layer adds the appropriate header and trailer to the packet, creating a frame, before sending the frames over each physical network.

The routing process forwards the network layer packet from end to end through the network, while each data-link frame only takes a smaller part of the trip. Each successive data-link layer frame moves the packet to the next device that thinks about network layer logic. In short, the network layer thinks about the bigger view of the goal, like "Send this packet to the specified next router or host…," while the data-link layer thinks about the specifics, like "Encapsulate the packet in a data-link frame and transmit it." The following list summarizes the major steps in a router's internal network layer routing for each packet beginning with the frame arriving in a router interface:

Key Topic

**Step 1.** Use the data-link Frame Check Sequence (FCS) field to ensure that the frame had no errors; if errors occurred, discard the frame.

**Step 2.** Assuming that the frame was not discarded at Step 1, discard the old data-link header and trailer, leaving the IP packet.

**Step 3.** Compare the IP packet's destination IP address to the routing table, and find the route that best matches the destination address. This route identifies the outgoing interface of the router and possibly the next-hop router IP address.

**Step 4.** Encapsulate the IP packet inside a new data-link header and trailer, appropriate for the outgoing interface, and forward the frame.

Figure 3-11 works through a repeat example of a packet sent by PC1 to PC2, followed by a detailed analysis of each device's routing logic. Each explanation includes the details about how PC1 and each of the three routers builds the appropriate new data-link headers.

Key Topic

**R1 Routing Table**

| Subnet | Interface | Next Hop |
|---|---|---|
| 150.150.4.0 | Serial0 | 150.150.2.7 |

**R2 Routing Table**

| Subnet | Interface | Next Hop |
|---|---|---|
| 150.150.4.0 | FastEth0/0 | 150.150.3.1 |

**R3 Routing Table**

| Subnet | Interface | Next Hop |
|---|---|---|
| 150.150.4.0 | Gigabit0/0 | N/A |

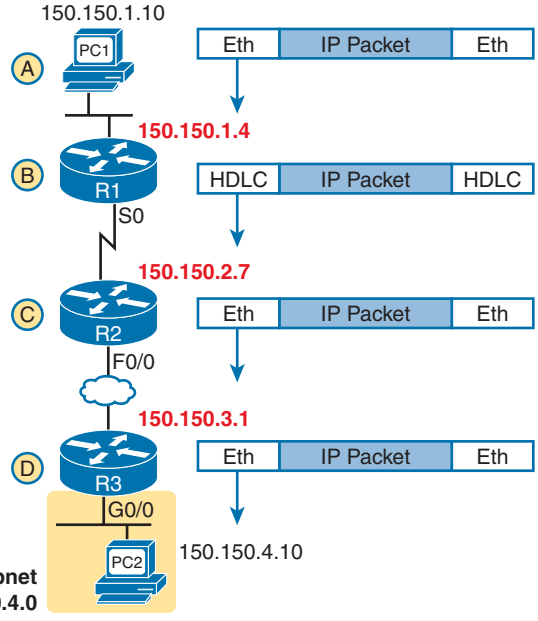



**Figure 3-11** *Network Layer and Data-Link Layer Encapsulation*

The following list explains the forwarding logic at each router, focusing on how the routing integrates with the data link.

**Step A.** **PC1 sends the packet to its default router.** PC1's network layer logic builds the IP packet, with a destination address of PC2's IP address (150.150.4.10). The network layer also performs the analysis to decide that 150.150.4.10 is not in the local IP subnet, so PC1 needs to send the packet to R1 (PC1's default router). PC1 places the IP packet into an Ethernet data-link frame, with a destination Ethernet address of R1's Ethernet address. PC1 sends the frame on to the Ethernet.

**Step B.** **R1 processes the incoming frame and forwards the packet to R2.** Because the incoming Ethernet frame has a destination MAC of R1's Ethernet MAC, R1 decides to process the frame. R1 checks the frame's FCS for errors, and if none, R1 discards the Ethernet header and trailer. Next, R1 compares the packet's destination address (150.150.4.10) to its routing table and finds the entry for subnet 150.150.4.0. Because the destination address of 150.150.4.10 is in that subnet, R1 forwards the packet out the interface listed in that matching route (Serial0) to next-hop Router R2 (150.150.2.7). R1 must first encapsulate the IP packet into an HDLC frame.

**Step C.** **R2 processes the incoming frame and forwards the packet to R3.** R2 repeats the same general process as R1 when R2 receives the HDLC frame. R2 checks the FCS field and finds that no errors occurred and then discards the HDLC header and trailer. Next, R2 compares the packet's destination address (150.150.4.10) to its routing table and finds the entry for subnet 150.150.4.0, a route that directs R2 to send the packet out interface Fast Ethernet 0/0 to next-hop router 150.150.3.1 (R3). But first, R2 must encapsulate the packet in an Ethernet header. That header uses R2's MAC address and R3's MAC address on the Ethernet WAN link as the source and destination MAC address, respectively.

**Step D.** **R3 processes the incoming frame and forwards the packet to PC2.** Like R1 and R2, R3 checks the FCS, discards the old data-link header and trailer, and matches its own route for subnet 150.150.4.0. R3's routing table entry for 150.150.4.0 shows that the outgoing interface is R3's Ethernet interface, but there is no next-hop router because R3 is connected directly to subnet 150.150.4.0. All R3 has to do is encapsulate the packet inside a new Ethernet header and trailer, but with a destination Ethernet address of PC2's MAC address.

Because the routers build new data-link headers and trailers, and because the new headers contain data-link addresses, the PCs and routers must have some way to decide what data-link addresses to use. An example of how the router determines which data-link address to use is the IP Address Resolution Protocol (ARP). *ARP dynamically learns the data-link address of an IP host connected to a LAN*. For example, at the last step, at the bottom of Figure 3-11, Router R3 would use ARP once to learn PC2's MAC address before sending any packets to PC2.

## How IP Addressing Helps IP Routing

IP defines network layer addresses that identify any host or router interface that connects to a TCP/IP network. The idea basically works like a postal address: Any interface that expects to receive IP packets needs an IP address, just like you need a postal address before receiving mail from the postal service. This next short topic introduces the idea of IP networks and subnets, which are the groups of addresses defined by IP.

**NOTE** IP defines the word *network* to mean a very specific concept. To avoid confusion when writing about IP addressing, this book (and others) often avoids using the term *network* for other uses. In particular, this book uses the term *internetwork* to refer more generally to a network made up of routers, switches, cables, and other equipment.

### Rules for Groups of IP Addresses (Networks and Subnets)

TCP/IP groups IP addresses together so that IP addresses used on the same physical network are part of the same group. IP calls these address groups an *IP network* or an *IP subnet*. Using that same postal service analogy, each IP network and IP subnet works like a postal code (or in the United States, a ZIP code). All nearby postal addresses are in the same postal code (ZIP code), while all nearby IP addresses must be in the same IP network or IP subnet.

IP defines specific rules about which IP address should be in the same IP network or IP subnet. Numerically, the addresses in the same group have the same value in the first part of the addresses. For example, Figures 3-10 and 3-11 could have used the following conventions:

- Hosts on the top Ethernet: Addresses start with 150.150.1
- Hosts on the R1–R2 serial link: Addresses start with 150.150.2
- Hosts on the R2–R3 Ethernet WAN link: Addresses start with 150.150.3
- Hosts on the bottom Ethernet: Addresses start with 150.150.4

From the perspective of IP routing, the grouping of IP addresses means that the routing table can be much smaller. A router can list one routing table entry for each IP network or subnet, representing a group of addresses, instead of one entry for every single IP address.

While the list shows just one example of how IP addresses may be grouped, the rules for how to group addresses using subnets will require some work to master the concepts and math. Part IV of this book details IP addressing and subnetting, along with other subnetting videos and practice questions. However, the brief version of two of the foundational rules of **subnetting** can be summarized as follows:

Key Topic

- Two IP addresses, not separated from each other by a router, must be in the same group (subnet).
- Two IP addresses, separated from each other by at least one router, must be in different groups (subnets).

It's similar to the USPS ZIP code system and how it requires local governments to assign addresses to new buildings. It would be ridiculous to have two houses next door to each

other with different postal/ZIP codes. Similarly, it would be silly to have people who live on opposite sides of the country to have addresses with the same postal/ZIP code.

### The IP Header

The routing process also makes use of the IPv4 header, as shown in Figure 3-12. The header lists a 32-bit source IP address, as well as a 32-bit destination IP address. The header, of course, has other fields, a few of which matter for other discussions in this book. The book will refer to this figure as needed, but otherwise, be aware of the 20-byte IP header and the existence of the source and destination IP address fields. Note that in the examples so far in this chapter, while routers remove and add data-link headers each time they route a packet, the IP header remains, with the IP addresses unchanged by the IP routing process.



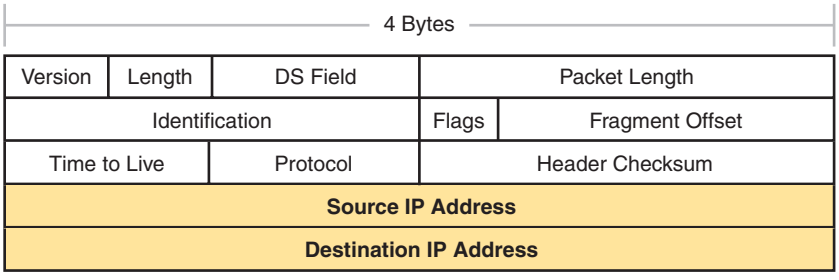



**Figure 3-12** *IPv4 Header, Organized as 4 Bytes Wide for a Total of 20 Bytes*

## How IP Routing Protocols Help IP Routing

For routing logic to work on both hosts and routers, each host and router needs to know something about the TCP/IP internetwork. Hosts need to know the IP address of their default router so that hosts can send packets to remote destinations. Routers, however, need to know routes so they forward packets to each and every reachable IP network and IP subnet.

The best method for routers to know all the useful routes is to configure the routers to use the same IP routing protocol. If you enable the same routing protocol on all the routers in a TCP/IP internetwork, with the correct settings, the routers will send routing protocol messages to each other. As a result, all the routers will learn routes for all the IP networks and subnets in the TCP/IP internetwork.

IP supports a small number of different IP routing protocols. All use some similar ideas and processes to learn IP routes, but different routing protocols do have some internal differences. All the routing protocols use the same general steps for learning routes:

Key Topic

**Step 1.** Each router, independent of the routing protocol, adds a route to its routing table for each subnet directly connected to the router.

**Step 2.** Each router's routing protocol tells its neighbors about the routes in its routing table, including the directly connected routes and routes learned from other routers.

**Step 3.** Each router's routing protocol listens to messages from neighboring routers and learns routes, with the next-hop router of that route typically being the neighbor from which the route was learned.

Also, note that at the final step, routers may have to choose between multiple routes to reach a single subnet. When that happens, routers place the best currently available route to reach a subnet (based on a measurement called a metric) into the routing table.

Figure 3-13 shows an example of how a routing protocol works, using the same diagram as in Figures 3-10 and 3-11. In this case, IP subnet 150.150.4.0, which consists of all addresses that begin with 150.150.4.0, sits on the Ethernet at the bottom of the figure. The figure shows the advertisement of routes for subnet 150.150.4.0 from bottom to top, as described in detail following the figure.

Key Topic

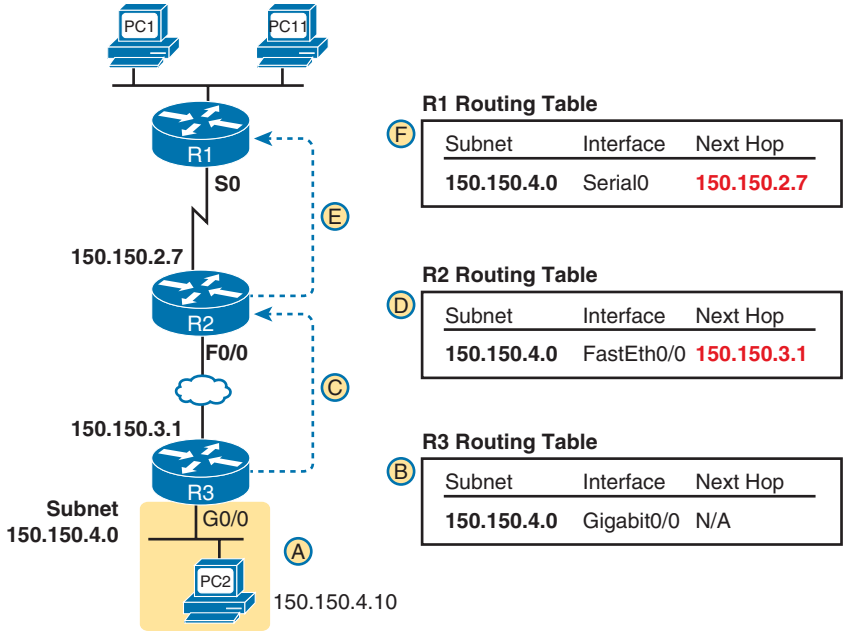



**Figure 3-13** *Example of How Routing Protocols Advertise About Networks and Subnets*

Follow items A through F shown in the figure to see how each router learns its route to 150.150.4.0.

**Step A.** Subnet 150.150.4.0 exists as a subnet at the bottom of the figure, connected to Router R3.

**Step B.** R3 adds a connected route for 150.150.4.0 to its IP routing table; this happens without help from the routing protocol.

**Step C.** R3 sends a routing protocol message, called a *routing update*, to R2, causing R2 to learn about subnet 150.150.4.0.

**Step D.** R2 adds a route for subnet 150.150.4.0 to its routing table.

**Step E.** R2 sends a similar routing update to R1, causing R1 to learn about subnet 150.150.4.0.

**Step F.** R1 adds a route for subnet 150.150.4.0 to its routing table. The route lists R1's own Serial0 as the outgoing interface and R2 as the next-hop router IP address (150.150.2.7).

# Other Network Layer Features

The TCP/IP network layer defines many functions beyond IP. Sure, IP plays a huge role in networking today, defining IP addressing and IP routing. However, other protocols and standards, defined in other Requests For Comments (RFC), play an important role for network layer functions as well. For example, routing protocols like Open Shortest Path First (OSPF) exist as separate protocols, defined in separate RFCs.

This last short section of the chapter introduces three other network layer features that should be helpful to you when reading through the rest of this book. These last three topics just help fill in a few holes, helping to give you some perspective and helping you make sense of later discussions as well. The three topics are



- Domain Name System (DNS)
- Address Resolution Protocol (ARP)
- Ping

## Using Names and the Domain Name System

Can you imagine a world in which every time you used an application, you had to refer to it by IP address? Instead of using easy names like google.com or cisco.com, you would have to remember and type IP addresses, like 64.233.177.100. (At press time, 64.233.177.100 was an address used by Google, and you could reach Google's website by typing that address in a browser.) Certainly, asking users to remember IP addresses would not be user friendly and could drive some people away from using computers at all.

Thankfully, TCP/IP defines a way to use **hostnames** to identify other computers. The user either never thinks about the other computer or refers to the other computer by name. Then, protocols dynamically discover all the necessary information to allow communications based on that name.

For example, when you open a web browser and type in the hostname **www.google.com**, your computer does not send an IP packet with destination IP address www.google.com; it sends an IP packet to an IP address used by the web server for Google. TCP/IP needs a way to let a computer find the IP address used by the listed hostname, and that method uses the Domain Name System (**DNS**).

Enterprises use the DNS process to resolve names into the matching IP address, as shown in the example in Figure 3-14. In this case, PC11, on the left, needs to connect to a server named Server1. At some point, the user either types in the name Server1 or some application on PC11 refers to that server by name. At Step 1, PC11 sends a DNS message—a DNS query—to the DNS server. At Step 2, the DNS server sends back a DNS reply that lists Server1's IP address. At Step 3, PC11 can now send an IP packet to destination address 10.1.2.3, the address used by Server1.

Note that the example in Figure 3-14 shows a cloud for the TCP/IP network because the details of the network, including routers, do not matter to the name resolution process. Routers treat the DNS messages just like any other IP packet, routing them based on the destination IP address. For example, at Step 1 in the figure, the DNS query will list the DNS server's IP address as the destination address, which any routers will use to forward the packet.

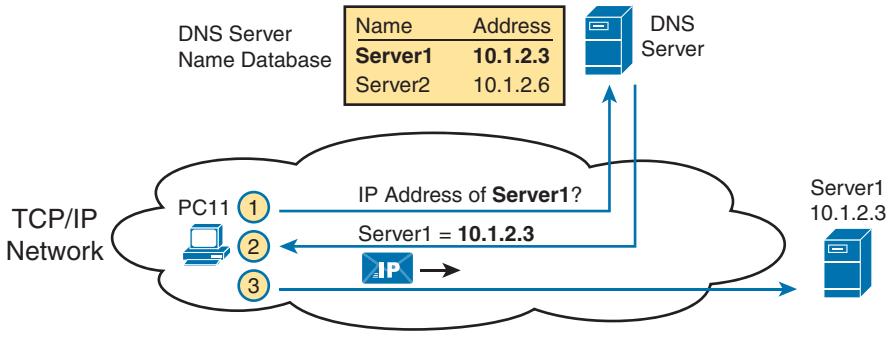



**Figure 3-14** *Basic DNS Name Resolution Request*

Finally, DNS defines much more than just a few messages. DNS defines protocols, as well as standards for the text names used throughout the world, and a worldwide set of distributed DNS servers. The domain names that people use every day when web browsing, which look like www.example.com, follow the DNS naming standards. Also, no single DNS server knows all the names and matching IP addresses, but the information is distributed across many DNS servers. So, the DNS servers of the world work together, forwarding queries to each other, until the server that knows the answer supplies the desired IP address information.

## The Address Resolution Protocol

As discussed in depth throughout this chapter, IP routing logic requires that hosts and routers encapsulate IP packets inside data-link layer frames. For Ethernet interfaces, how does a router know what MAC address to use for the destination? It uses **ARP**.

On Ethernet LANs, whenever a host or router needs to encapsulate an IP packet in a new Ethernet frame, the host or router knows all the important facts to build that header—except for the destination MAC address. The host knows the IP address of the next device, either another host IP address or the default router IP address. A router knows the IP route used for forwarding the IP packet, which lists the next router's IP address. However, the hosts and routers do not know those neighboring devices' MAC addresses beforehand.

TCP/IP defines the Address Resolution Protocol (ARP) as the method by which any host or router on a LAN can dynamically learn the MAC address of another IP host or router on the same LAN. ARP defines a protocol that includes the *ARP Request*, which is a message that makes the simple request "if this is your IP address, please reply with your MAC address." ARP also defines the *ARP Reply* message, which indeed lists both the original IP address and the matching MAC address.

Figure 3-15 shows an example that uses the same router and host from the bottom part of the earlier Figure 3-13. The figure shows the ARP Request sent by router R3, on the left of the figure, as a LAN broadcast. All devices on the LAN will then process the received frame. On the right, at Step 2, host PC2 sends back an ARP Reply, identifying PC2's MAC address. The text beside each message shows the contents inside the ARP message itself, which lets PC2 learn R3's IP address and matching MAC address, and R3 learn PC2's IP address and matching MAC address.

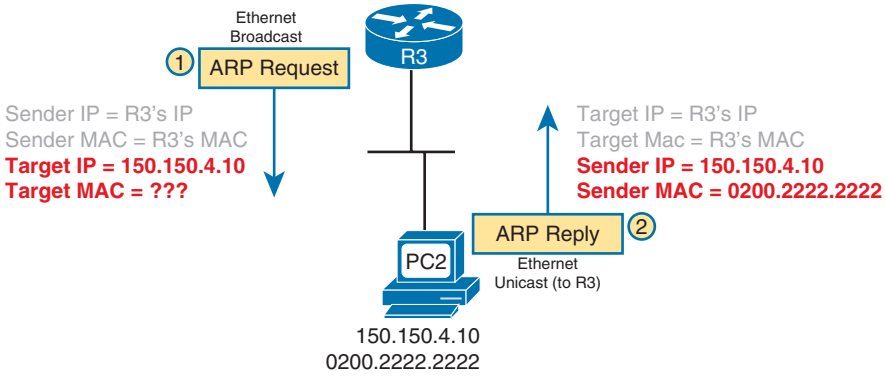



**Figure 3-15** *Sample ARP Process*

Note that hosts and routers remember the ARP results, keeping the information in their *ARP cache* or *ARP table*. A host or router only needs to use ARP occasionally, to build the ARP cache the first time. Each time a host or router needs to send a packet encapsulated in an Ethernet frame, it first checks its ARP cache for the correct IP address and matching MAC address. Hosts and routers will let ARP cache entries time out to clean up the table, so occasional ARP Requests can be seen.

**NOTE** You can see the contents of the ARP cache on most PC operating systems by using the **arp -a** command from a command prompt.

## ICMP Echo and the ping Command

After you have implemented a TCP/IP internetwork, you need a way to test basic IP connectivity without relying on any applications to be working. The primary tool for testing basic network connectivity is the **ping** command.

**Ping** uses the Internet Control Message Protocol (ICMP), sending a message called an *ICMP echo request* to another IP address. The computer with that IP address should reply with an *ICMP echo reply*. If that works, you successfully have tested the IP network. In other words, you know that the network can deliver a packet from one host to the other and back. ICMP does not rely on any application, so it really just tests basic IP connectivity—Layers 1, 2, and 3 of the OSI model. Figure 3-16 outlines the basic process.

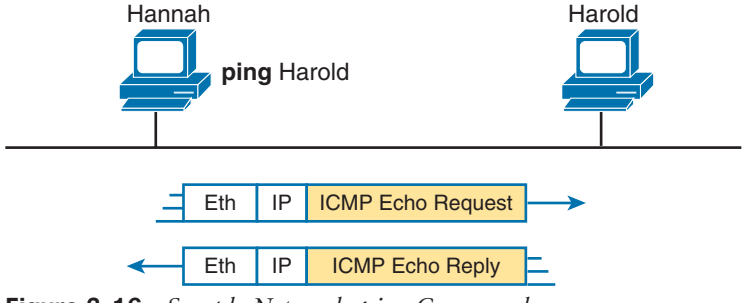



**Figure 3-16** *Sample Network, ping Command*

Note that while the **ping** command uses ICMP, ICMP does much more. ICMP defines many messages that devices can use to help manage and control the IP network.

## Chapter Review

The "Your Study Plan" element, just before Chapter 1, discusses how you should study and practice the content and skills for each chapter before moving on to the next chapter. That element introduces the tools used here at the end of each chapter. If you haven't already done so, take a few minutes to read that section. Then come back here and do the useful work of reviewing the chapter to help lock into memory what you just read.

Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Table 3-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 3-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |

## Review All the Key Topics

Key Topic

**Table 3-5** Key Topics for Chapter 3

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 3-6 | Router de-encapsulation and re-encapsulation | 68 |
| Figure 3-7 | Ethernet WAN—physical connections | 69 |
| List | Common terms to describe an Ethernet WAN link | 70 |
| List | Four-step process of how routers route (forward) packets | 74 |
| Figure 3-11 | IP Routing and Encapsulation | 74 |
| List | Two statements about how IP expects IP addresses to be grouped into networks or subnets | 76 |
| List | Three-step process of how routing protocols learn routes | 77 |
| Figure 3-13 | IP Routing Protocol Basic Process | 78 |
| Figure 3-14 | Example that shows the purpose and process of DNS name resolution | 80 |
| Figure 3-15 | Example of the purpose and process of ARP | 81 |

## Key Terms You Should Know

ARP, default router (default gateway), DNS, Ethernet Line Service (E-Line), Ethernet WAN, HDLC, hostname, IP address, IP network, IP packet, IP subnet, leased line, ping, routing protocol, routing table, serial interface, subnetting, telco, wide-area network (WAN)

*This page intentionally left blank*



# Part I Review

Keep track of your part review progress with the checklist shown in Table P1-1. Details on each task follow the table.

**Table P1-1** Part I Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Chapter Review Interactive Elements | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book, using the PTP software. Refer to the Introduction to this book, the section titled "How to View Only DIKTA Questions by Chapter or Part," for help with how to make the PTP software show you DIKTA questions for this part only.

## Answer Part Review Questions

For this task, answer the Part Review questions for this part of the book, using the PTP software. Refer to the Introduction to this book, the section titled "How to View Part Review Questions," for help with how to make the PTP software show you Part Review questions for this part only. (Note that if you use the questions but then want even more, get the Premium Edition of the book, as detailed in the Introduction, in the section "Other Features," under the item labeled "eBook.")

## Review Key Topics

Browse back through the chapters and look for the Key Topic icons. If you do not remember some details, take the time to reread those topics, or use the Key Topics application(s) found on the companion website.

Take the time to find the companion website for this book and bookmark that page as detailed in the Introduction section titled "Bookmark the Companion Website," in the Your Study Plan section just before Chapter 1.

## Use Per-Chapter Interactive Review Elements

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.

*This page intentionally left blank*



Part I provided a broad look at the fundamentals of all parts of networking, focusing on Ethernet LANs, WANs, and IP routing. Parts II and III now drill into depth about the details of Ethernet, which was introduced in Chapter 2, "Fundamentals of Ethernet LANs."

Part II begins that journey by discussing the basics of building a small Ethernet LAN with Cisco Catalyst switches. The journey begins by showing how to access the user interface of a Cisco switch so that you can see evidence of what the switch is doing and to configure the switch to act in the ways you want it to act. At this point, you should start using whatever lab practice option you chose in the "Your Study Plan" section that preceded Chapter 1, "Introduction to TCP/IP Networking." (And if you have not yet finalized your plan for how to practice your hands-on skills, now is the time.)

After you complete Chapter 4 and see how to get into the command-line interface (CLI) of a switch, the next three chapters step through some important foundations of how to implement LANs—foundations used by every company that builds LANs with Cisco gear. Chapter 5 takes a close look at Ethernet switching—that is, the logic used by a switch—and how to know what a particular switch is doing. Chapter 6 shows the ways to configure a switch for remote access with Telnet and Secure Shell (SSH), along with a variety of other useful commands that will help you when you work with any real lab gear, simulator, or any other practice tools. Chapter 7, the final chapter in Part II, shows how to configure and verify the operation of switch interfaces for several important features, including speed, duplex, and autonegotiation.

# Part II

## Implementing Ethernet LANs

**Chapter 4:** Using the Command-Line Interface

**Chapter 5:** Analyzing Ethernet LAN Switching

**Chapter 6:** Configuring Basic Switch Management

**Chapter 7:** Configuring and Verifying Switch Interfaces

**Part II Review**



CHAPTER 4

# Using the Command-Line Interface

**This chapter covers the following exam topics:**

**2.0 Network Access**

**2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, TACACS+/RADIUS, and cloud managed)**

The CCNA exam focuses on skills like understanding how LANs work, configuring different switch features, verifying that those features work correctly, and finding the root cause of the problem when a feature is not working correctly. Before doing the more important work, you must first learn how to access and use the user interface of the switch, called the **command-line interface (CLI)**.

This chapter begins that process by showing the basics of how to access the switch's CLI. These skills include how to access the CLI and how to issue verification commands to check on the status of the LAN. This chapter also includes the processes of how to configure the switch and how to save that configuration.

Note that this chapter focuses on processes that provide a foundation for the exam topics which include the verbs *identify*, *configure*, or *verify*. Most of the rest of the chapters in Parts II and III of this book then go on to include details of the particular commands you can use to verify and configure different switch features.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 4-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Accessing the Cisco Catalyst Switch CLI | 1–3 |
| Configuring Cisco IOS Software | 4–6 |

1. In what modes can you type the command **show mac address-table** and expect to get a response with MAC table entries? (Choose two answers.)
   a. User mode
   b. Enable mode
   c. Global configuration mode
   d. Interface configuration mode

2. In which of the following modes of the CLI could you type the command **reload** and expect the switch to reboot?
   a. User mode
   b. Enable mode
   c. Global configuration mode
   d. Interface configuration mode
3. Which of the following is a difference between Telnet and SSH as supported by a Cisco switch?
   a. SSH encrypts the passwords used at login, but not other traffic; Telnet encrypts nothing.
   b. SSH encrypts all data exchange, including login passwords; Telnet encrypts nothing.
   c. Telnet is used from Microsoft operating systems, and SSH is used from UNIX and Linux operating systems.
   d. Telnet encrypts only password exchanges; SSH encrypts all data exchanges.
4. What type of switch memory is used to store the configuration used by the switch when it is up and working?
   a. RAM
   b. ROM
   c. Flash
   d. NVRAM
   e. Bubble
5. What command copies the configuration from RAM into NVRAM?
   a. **copy running-config tftp**
   b. **copy tftp running-config**
   c. **copy running-config start-up-config**
   d. **copy start-up-config running-config**
   e. **copy startup-config running-config**
   f. **copy running-config startup-config**
6. A switch user is currently in console line configuration mode. Which of the following would place the user in enable mode? (Choose two answers.)
   a. Using the **exit** command once
   b. Using the **end** command once
   c. Pressing the Ctrl+Z key sequence once
   d. Using the **quit** command

# Foundation Topics

## Accessing the Cisco Catalyst Switch CLI

Cisco uses the concept of a command-line interface (CLI) with its router and LAN switch products. The CLI is a text-based interface in which the user, typically a network engineer, enters a text command and presses Enter. Pressing Enter sends the command to the switch, which tells the device to do something. The switch does what the command says, and in some cases, the switch replies with some messages stating the results of the command.

Cisco Catalyst switches also support other methods to both monitor and configure a switch. For example, a switch can provide a web interface so that an engineer can open a web browser to connect to a web server running in the switch. Switches also can be controlled and operated using network management software.

This book discusses only Cisco Catalyst enterprise-class switches, and in particular, how to use the Cisco CLI to monitor and control these switches. This first major section of the chapter first examines these Catalyst switches in more detail and then explains how a network engineer can get access to the CLI to issue commands.

### Cisco Catalyst Switches

When I was updating this chapter for expected publication in the year 2023, Cisco LAN switches fell into three product families:

- **Cisco Catalyst switches**, which serve as typical enterprise switches for use throughout an enterprise.
- **Cisco Nexus switches**, which Cisco designs for use in data centers. In comparison to Catalyst switches, Nexus switches support faster ports and more high-speed ports on a single switch, plus other features optimized for data centers.
- **Cisco Meraki switches**, built for enterprises, meet a need for LAN switches (and other network devices) that use a simple, cloud-based management portal that provides easy onboarding of new devices.

Throughout its history, when the CCNA exam mentions *switches*, it refers to Cisco Catalyst switches, and not to Nexus or Meraki switches.

Within the Cisco Catalyst brand of LAN switches, Cisco produces a wide variety of switch series or families. Each switch series includes several specific models of switches that have similar features, similar price-versus-performance tradeoffs, and similar internal components.

For example, when I was writing the latest version of this chapter, Cisco offered the Cisco Catalyst 9000 switch family as its primary Enterprise switch family. That family includes series that go by numbers like 9200, 9300, 9400, and so on, with specific models within each series. For instance, Cisco positions the 9200 and 9300 series of switches as full-featured access layer switches for enterprises.

---

Answers to the "Do I Know This Already?" quiz:

**1** A, B **2** B **3** B **4** A **5** F **6** B, C

Figure 4-1 shows two models of 9200 switches. The lower switch in the figure has 48 fixed RJ-45 unshielded twisted-pair (UTP) 10/100/1000 ports, meaning that these ports can auto-negotiate the use of 10BASE-T (10 Mbps), 100BASE-T (100 Mbps), or 1000BASE-T (1 Gbps) Ethernet. The upper switch has 24 such ports.

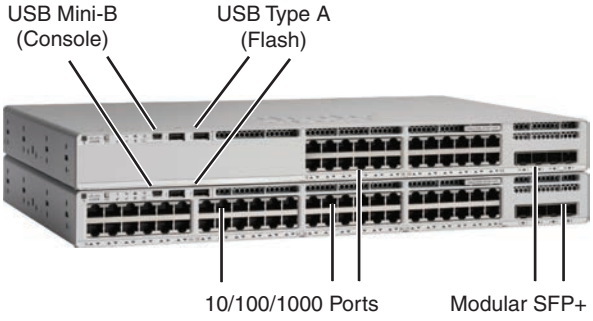



**Figure 4-1** *Cisco Catalyst 9200L Switch Models, Front View*



Cisco refers to a switch's physical connectors as either *interfaces* or *ports*, with an interface type and interface number. Switches use interface types such as Ethernet, FastEthernet, GigabitEthernet, TenGigabit, and so on for faster speeds. For Ethernet interfaces that support running at multiple speeds, the switch uses the interface type of the fastest supported speed. For example, the switch refers to a 10/100/1000 interface (that is, an interface that runs at 10 Mbps, 100 Mbps, or 1000 Mbps) as a GigabitEthernet port, no matter the current speed used on the interface.

LAN switches also use a numeric interface identifier (interface ID) to identify the specific port. Interface IDs can use one, two, or three digits, with the digits separated by a slash, as chosen by Cisco when they designed the switch. For instance, Cisco Catalyst 9000 switches typically use three-digit identifiers like GigabitEthernet 1/0/1 and GigabitEthernet 1/0/2. Table 4-2 lists a few examples of interface identifiers and abbreviations for those identifiers based on the speeds supported by an interface.

**Table 4-2** LAN Switch Interface ID Examples with Abbreviations

| Speeds Supported | Common Name | Example Switch Interface ID | Valid Abbreviations |
|---|---|---|---|
| 10 Mbps | Ethernet | Ethernet0/0 | E0/0, Et0/0, Eth0/0 |
| 10/100 Mbps | 10/100 | FastEthernet 0/1 | F0/1, Fa0/1 |
| 10/100/1000 Mbps | 10/100/1000 | GigabitEthernet 1/0/1 | G1/0/1, Gi1/0/1 |
| 1G/2.5G/5G/10G | Multigig | TenGigabit 1/0/2 | T1/0/2, Te1/0/2 |

## Accessing the Cisco IOS XE CLI

The operating system (OS) in Catalyst switches creates a human-focused interface in which you can type commands and see lines of text in response. The next topic introduces that interface, called the command-line interface (CLI), but first the text gives a few background details about the operating system itself.

### The Operating System in Cisco Catalyst Switches

Initially in its history, Cisco switches used an OS called CatOS, short for Catalyst OS. Cisco did not develop its first switch family, instead purchasing a company (Crescendo Communications) in 1993—a company that had named its switch family "Catalyst," with the OS called CatOS.

When Cisco first got into the LAN switch business, Cisco routers used an OS written by Cisco, called Internetwork Operating System (**IOS**)—not IOS XE, simply IOS. Unsurprisingly, the router IOS CLI in Cisco routers worked differently than the switch CatOS CLI produced by the vendor that Cisco bought. Over time, Cisco created a switch OS based on IOS, so, mostly in the 1990s and 2000s, when Cisco released a new switch product family, it used IOS for Catalyst switches instead of CatOS. By moving its switches to use IOS, eventually all Cisco enterprise-class routers and switches used the same IOS CLI, making them easier to manage for networkers. Figure 4-2 depicts the migration timeline in general terms.

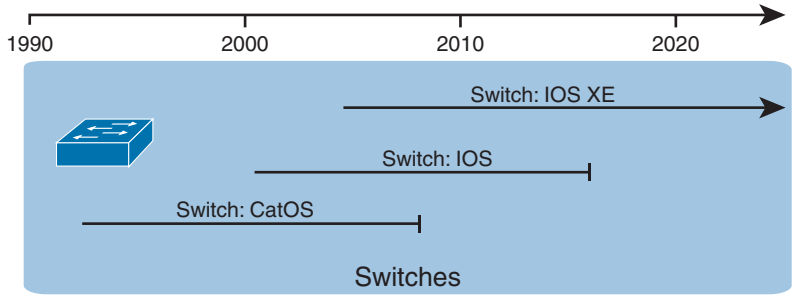



**Figure 4-2** *Cisco Catalyst Operating System Long-Term Migration*

Cisco made yet another migration to a different OS for Catalyst switches, this time to use **IOS XE**. To create IOS XE, Cisco took IOS and modernized the internal software architecture. IOS XE, often referred to simply as XE, has features to improve uptime and the ability to maintain devices without requiring rebooting (reloading) the device. But it keeps the same familiar CLI; in fact, if you learned the CLI using a device running IOS, you might not even notice when later using a device running IOS XE.

For the purposes of CCNA, almost everything you see with commands and the CLI applies to older IOS-based switches as well as newer switches that use IOS XE. However, when a difference exists, the text will point out the difference.

**NOTE** The book refers to IOS and IOS XE with the common term *IOS*, unless the need exists to differentiate between the two.

### Accessing the IOS XE CLI

Cisco IOS Software for Catalyst switches implements and controls logic and functions performed by a Cisco switch. Besides controlling the switch's performance and behavior, Cisco IOS also defines an interface for humans called the CLI. The Cisco IOS CLI allows the user to use a terminal emulation program, which accepts text entered by the user. When the user presses Enter, the terminal emulator sends that text to the switch. The switch processes

the text as if it is a command, does what the command says, and sends text back to the terminal emulator.

The switch CLI can be accessed through three popular methods: the console, **Telnet**, and **Secure Shell (SSH)**. Two of these methods (Telnet and SSH) use the IP network in which the switch resides to reach the switch. The console is a physical port built specifically to allow access to the CLI. Figure 4-3 depicts the options.

Key Topic

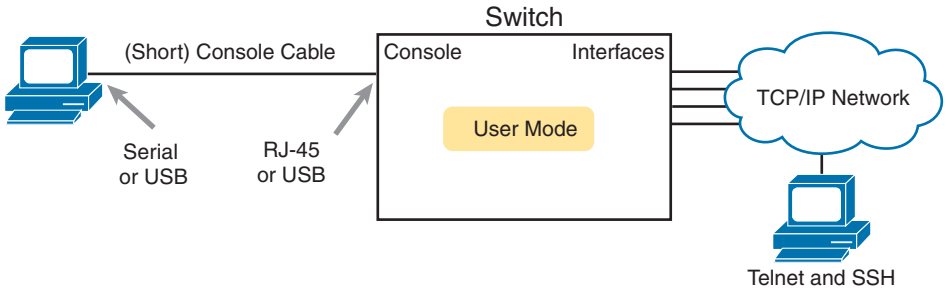



**Figure 4-3** *CLI Access Options*

Console access requires both a physical connection between a PC (or other user device) and the switch's console port, as well as some software on the PC. Telnet and SSH require software on the user's device, but they rely on the existing TCP/IP network to transmit data. The next few pages detail how to connect the console and set up the software for each method to access the CLI.

## Cabling the Console Connection

The physical console connection, both old and new, uses three main components: the physical console port on the switch, a physical serial port on the PC, and a cable that works with the console and serial ports. However, the physical cabling details have changed slowly over time, mainly because of advances and changes with serial interfaces on PC hardware. For this next topic, the text looks at three cases: newer connectors on both the PC and the switch, older connectors on both, and a third case with the newer (USB) connector on the PC but with an older connector on the switch.

Most PCs today use a familiar standard USB connector for the console connection. Cisco has been including USB console ports in new router and switch models for well over a decade. All you have to do is look at the switch to make sure you have the correct style of USB connector to match the USB console port (often a USB mini-B connector.) In the simplest form, you can use any USB port on the PC, with a USB cable, connected to the USB console port on the switch or router, as shown on the far-right side of Figure 4-4.

The case on the far left in the figure shows an older console connection, typical of how you would have connected to a switch over ten years ago. Before PCs used USB ports, they used serial ports for serial communications. The PC serial port had a D-shell connector (roughly rectangular) with nine pins (often called a DB-9). The console port looks like any Ethernet RJ-45 port (but is typically colored in blue and with the word *console* beside it on the switch). The older-style cabling used a standard RJ-45 to DB-9 converter plug and a UTP **rollover cable** with RJ-45 connectors on each end. The rollover pinout uses eight wires, rolling the wire at pin 1 to pin 8, pin 2 to pin 7, pin 3 to pin 6, and so on.

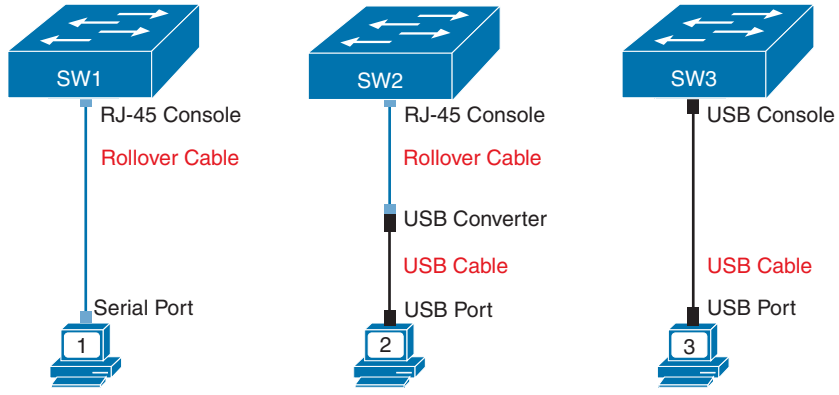



**Figure 4-4** *Console Connection to a Switch*

The center case in the figure shows a variation that you might use on occasion that combines the cabling concepts from the left and right cases in the figure. You use the USB port on your PC but the RJ-45 console port on the switch. In fact, for some very old switch models, the switch has only an RJ-45 console port but no USB console port, requiring this kind of console cabling. In this case, you need a USB converter plug that converts from the older rollover console cable (with RJ-45 connectors) to a USB connector, as shown in the middle of Figure 4-4.

**NOTE** When using the USB options, you typically also need to install a software driver so that your PC's OS knows that the device on the other end of the USB connection is the console of a Cisco device. Also, you can easily find photos of these cables and components online, with searches like "cisco console cable," "cisco usb console cable," or "console cable converter."

Figure 4-5 shows a Cisco 9200L switch, rear view, which shows the RJ-45 console connector. The 9200L locates the USB console port (a mini-B USB port) on the front panel, as seen earlier in Figure 4-1.

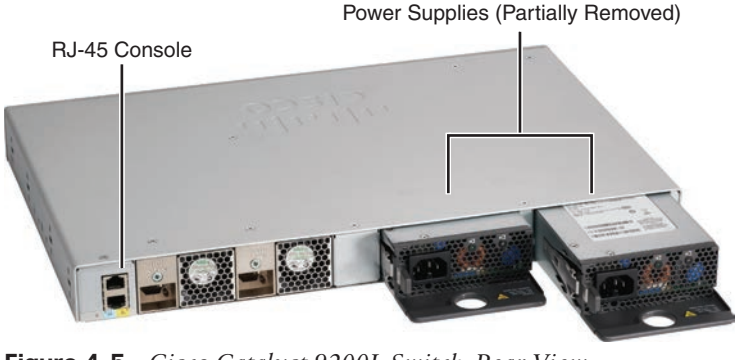



**Figure 4-5** *Cisco Catalyst 9200L Switch, Rear View*

### Configuring a Terminal Emulator

After the PC is physically connected to the console port, a terminal emulator software package must be installed and configured on the PC. The terminal emulator software treats all data as text. It accepts the text typed by the user and sends it over the console connection to the switch. Similarly, any bits coming into the PC over the console connection are displayed as text for the user to read.

The emulator must be configured to use the PC's serial port to match the settings on the switch's console port settings. The default console port settings on a switch are as follows. Note that the last three parameters are referred to collectively as 8N1:

Key Topic

- 9600 bits/second
- No hardware flow control
- 8-bit ASCII
- No parity bits
- 1 stop bit



Figure 4-6 shows one such terminal emulator. The image shows the window created by the emulator software in the background, with some output of a **show** command. The foreground, in the upper right, shows a settings window that lists the default console settings as listed just before this paragraph.

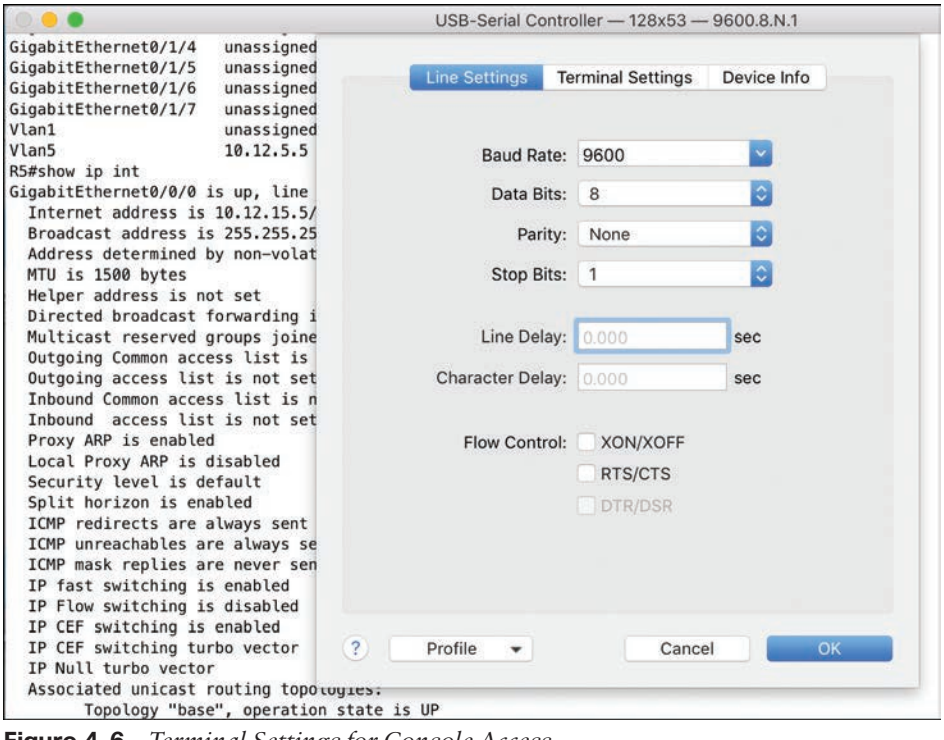



**Figure 4-6** *Terminal Settings for Console Access*

### Accessing the CLI with Telnet and SSH

For many years, terminal emulator applications have supported far more than the ability to communicate over a USB (or serial) port to a local device (like a switch's console). Terminal emulators support a variety of TCP/IP applications as well, including Telnet and SSH. Telnet and SSH both allow the user to connect to another device's CLI, but instead of connecting through a console cable to the console port, the traffic flows over the same IP network that the networking devices are helping to create.

Telnet uses the concept of a Telnet client (the terminal application) and a Telnet server (the switch in this case). A *Telnet client*, the device that sits in front of the user, accepts keyboard input and sends those commands to the *Telnet server*. The Telnet server accepts the text, interprets the text as a command, and replies back.

Cisco Catalyst switches enable a Telnet server by default, but switches need a few more configuration settings before you can successfully use Telnet to connect to a switch. Chapter 6, "Configuring Basic Switch Management," covers switch configuration to support Telnet and SSH in detail.

Using Telnet in a lab today makes sense, but Telnet poses a significant security risk in production networks. Telnet sends all data (including any username and password for login to the switch) as clear-text data. SSH gives us a much better option.

Think of SSH as the much more secure Telnet cousin. Outwardly, you still open a terminal emulator, connect to the switch's IP address, and see the switch CLI, no matter whether you use Telnet or SSH. The differences exist behind the scenes: SSH encrypts the contents of all messages, including the passwords, avoiding the possibility of someone capturing packets in the network and stealing the password to network devices.

### User and Enable (Privileged) Modes

All three CLI access methods covered so far (console, Telnet, and SSH) place the user in an area of the CLI called *user EXEC mode*. User EXEC mode, sometimes also called **user mode**, allows the user to look around but not break anything. The "EXEC mode" part of the name refers to the fact that in this mode, when you enter a command, the switch executes the command and then displays messages that describe the command's results.

**NOTE** If you have not used the CLI before, you might want to experiment with the CLI from the Sim Lite product. You can find this resource on the companion website as mentioned in the Introduction.

Cisco IOS supports a more powerful EXEC mode called *privileged* mode (also known as **enable mode**). The formal name, *privileged mode*, refers to the fact that IOS allows powerful (or privileged) commands from that mode. Informally, engineers refer to the mode as *enable mode* because of the **enable** EXEC command, which moves the user from user mode to enable mode, as shown in Figure 4-7. For example, you can use the **reload** command, which tells the switch to reinitialize or reboot Cisco IOS, only from privileged mode.

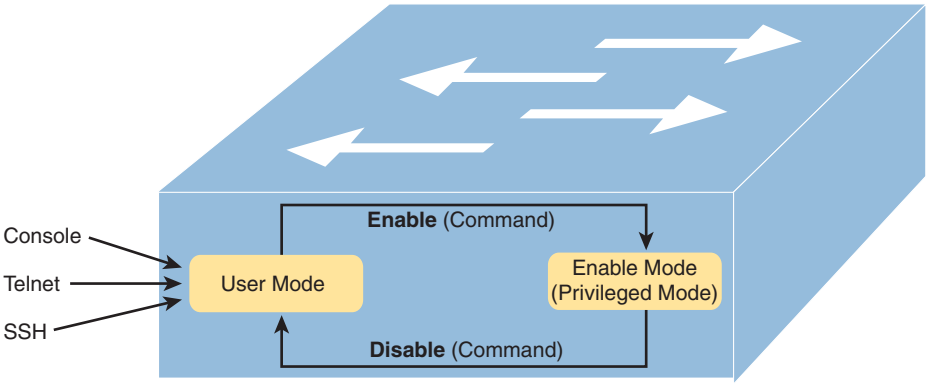



**Figure 4-7** *User and Privileged Modes*



**NOTE** If the command prompt ends with a >, the user is in user mode; if it ends with a #, the user is in enable mode.

Example 4-1 demonstrates the differences between user and enable modes. The example shows the output that you could see in a terminal emulator window, for instance, when connecting from the console. In this case, the user sits at the user mode prompt ("Certskills1>") and tries the **reload** command. The **reload** command tells the switch to reinitialize or reboot Cisco IOS, so IOS allows this powerful command to be used only from enable mode. IOS rejects the **reload** command from user mode. Then the user moves to privileged (enable) mode using the **enable** EXEC command. At that point, IOS accepts the **reload** command now that the user is in enable mode.

**Example 4-1** *Example of Privileged Mode Commands Being Rejected in User Mode*

```
Press RETURN to get started.

User Access Verification

Password:
Certskills1>
Certskills1> reload
Translating "reload"
% Unknown command or computer name, or unable to find computer address
Certskills1> enable
Password:
Certskills1#
Certskills1# reload

Proceed with reload? [confirm] y
00:08:42: %SYS-5-RELOAD: Reload requested by console. Reload Reason: Reload Command.
```

**NOTE** The commands that can be used in either user (EXEC) mode or enable (EXEC) mode are called EXEC commands.

This example is the first instance of this book showing you the output from the CLI, so it is worth noting a few conventions. The bold text represents what the user typed, and the nonbold text is what the switch sent back to the terminal emulator. Also, the typed passwords do not show up on the screen for security purposes.

### Password Security for CLI Access from the Console

A Cisco switch, with default settings, remains relatively secure when locked inside a wiring closet, because by default, a switch allows console access only. By default, the console requires no password at all, and no password to reach enable mode for users that happened to connect from the console. The reason is that if you have access to the physical console port of the switch, you already have pretty much complete control over the switch. You could literally get out your screwdriver, remove the switch from the rack, and walk off with it, or you could unplug the power, or follow well-published procedures to go through password recovery to break into the CLI and then configure anything you want to configure.

However, many people use simple password protection for console users. Simple passwords can be configured at two points in the login process from the console: when the user connects from the console, and when any user moves to enable mode (using the **enable** EXEC command). You may have noticed that back in Example 4-1, the user saw a password prompt at both points.

Example 4-2 shows the additional configuration commands that were configured prior to collecting the output in Example 4-1. The output holds an excerpt from the EXEC command **show running-config**, which lists the current configuration in the switch.

**Example 4-2** *Nondefault Basic Configuration*

```
Certskills1# show running-config
! Output has been formatted to show only the parts relevant to this discussion
hostname Certskills1
!
enable secret love
!
line console 0
 login

password faith
! The rest of the output has been omitted
Certskills1#
```

Working from top to bottom, note that the first configuration command listed by the **show running-config** command sets the switch's hostname to Certskills1. The switch uses the hostname to begin the command prompt.

Next, note that the lines with a **!** in them are comment lines, both in the text of this book and in the real switch CLI.

The **enable secret love** configuration command defines the password that all users must use to reach enable mode. So, no matter whether users connect from the console, Telnet, or SSH,

they would use the password love when prompted for a password after typing the **enable** EXEC command.

Finally, the last three lines configure the console password. The first line (**line console 0**) is the command that identifies the console, basically meaning "these next commands apply to the console only." The **login** command tells IOS to perform simple password checking (at the console). Remember, by default, the switch does not ask for a password for console users. Finally, the **password faith** command defines the password the console user must type when prompted.

This example just scratches the surface of the kinds of security configuration you might choose to configure on a switch, but it does give you enough detail to configure switches in your lab and get started (which is the reason I put these details in this first chapter of Part II). Note that Chapter 6 shows the configuration steps to add support for Telnet and SSH (including password security), and Chapter 9 of the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, "Securing Network Devices," shows additional security configuration as well.



## Accessing the CLI with the WebUI

Engineers use SSH to access the network device CLI as a routine part of their daily work. Cisco also provides a graphical interface to manage individual Cisco switches and routers, referred to as the device's HTTP server or, in later years, as the WebUI (Web User Interface). Once configured, an engineer can use any web browser to connect to the IP address of the switch or router, supply the correct login information, and see web pages that allow management of that single device. That web interface also includes a method to access the CLI.

First, Figures 4-8 and 4-9 show a couple of example pages of the graphical interface. Figure 4-8 shows a screenshot of the web server dashboard's Switch View. This page shows an image representing the side of the switch, with all switch ports shown and colored lights representing the same status colors you would see on the physical switch.

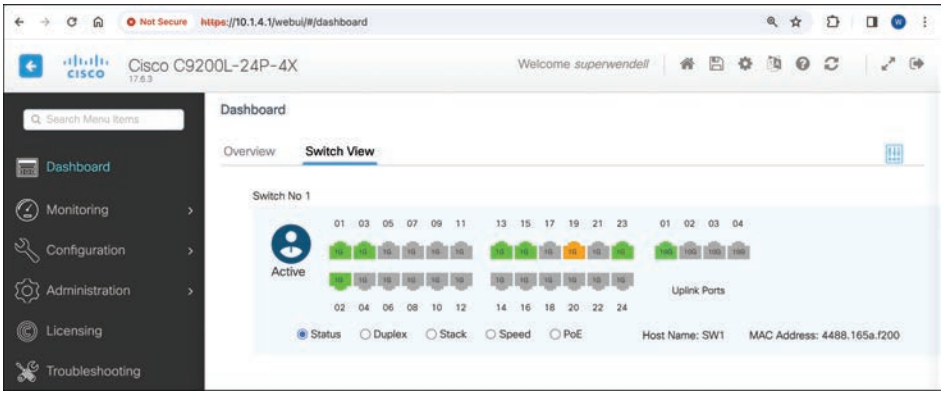



**Figure 4-8** *Cisco Switch Web Interface Dashboard: Switch View*

**NOTE** The examples in this section come from a Cisco Catalyst 9200L switch running IOS XE Version 17.6.3, visible in the upper left of the screenshot in Figure 4-8. However, the specifics of the user interface and available options in the left-side menus vary across device types, models, and IOS versions. So, use the figures in this section only to get a general idea of the functions of this tool.

The WebUI supports direct configuration and verification options that do not require knowledge of the CLI. For example, Figure 4-9 shows a screenshot with the user's mouse hovering over the word "Configuration" in the left menu, causing the page to reveal various configuration options. Earlier, the user had chosen the STP option, so the background in the screenshot shows a partial view of the Spanning Tree Protocol (STP) configuration screen. So, you could point and click through the user interface to configure some of the options covered for CCNA.

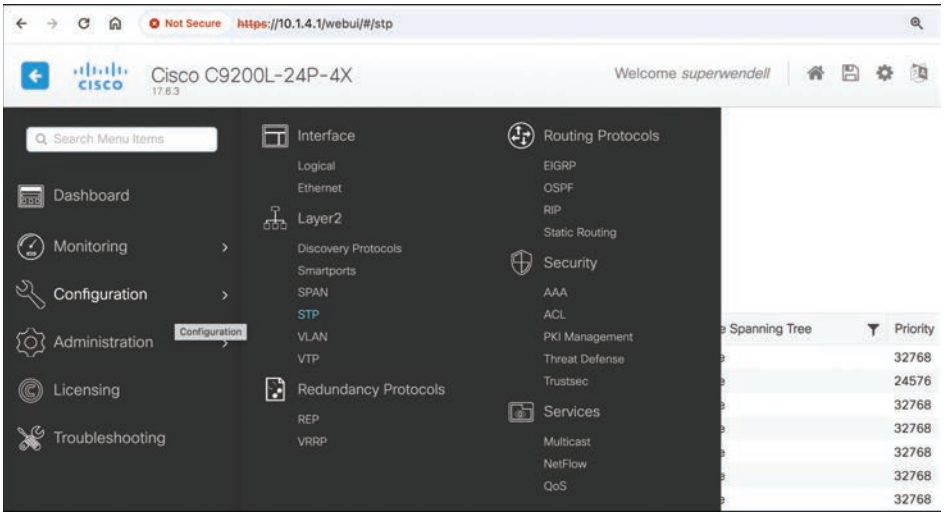



**Figure 4-9** *Configuration Options in the Switch Web Server Configuration Menu*

If you have a Cisco switch or router available, you should take the time to use the WebUI and look around at some of the configuration and verification options. Again, be aware that your device might show different layouts and options versus the screenshots shown here.

The more recent versions of the WebUI provide access to the CLI without your device needing to have an SSH or Telnet client installed, as seen in Figure 4-10. Once you navigate to the correct page (**Administration > Command Line Interface**), you type the CLI command into the upper box, click a button to send it to the device, and the response messages appear in the lower box. The figure shows the output of the **show interfaces status** command.

The WebUI can be pretty useful when you do not have any of your devices with you. If your devices with SSH clients installed are back at your desk and you need to log in to a few network devices, borrow any device with a web browser, and connect to routers and switches. Although the WebUI is useful, most engineers prefer using an SSH client when working in the CLI because the clients are much more usable. For instance, in Figure 4-10, note the poor column alignment of the command output. An SSH client would typically not have those kinds of alignment issues, which can make reading the output more difficult.

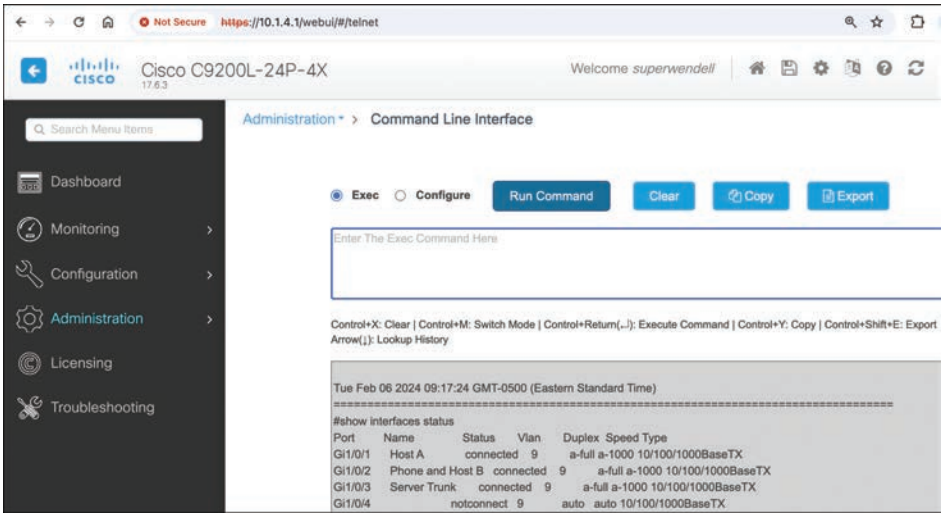



**Figure 4-10** *CLI Access from the Switch Web Server Interface*

**NOTE** On a related note, vendors like Cisco also sell sophisticated stand-alone network management applications. These typically have graphical interfaces and comprehensive management for all available features and support all devices using one user interface. In comparison, the WebUI provides basic features on a per-device basis. You will learn more about one such application, Cisco Catalyst Center, in Chapter 22, "Cisco Software-Defined Access (Cisco SD-Access)," in the book *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition.

## CLI Help Features

If you printed the Cisco IOS Command Reference documents, you would end up with a stack of paper several feet tall. No one should expect to memorize all the commands—and no one does. You can use several easy, convenient tools to help remember commands and save time typing. As you progress through your Cisco certifications, the exams will cover progressively more commands. However, you should know the methods of getting command help.

Table 4-3 summarizes command-recall help options available at the CLI. Note that, in the first column, *command* represents any command. Likewise, *parm* represents a command's parameter. For example, the third row lists *command* **?**, which means that commands such as **show ?** and **copy ?** would list help for the **show** and **copy** commands, respectively.

**Table 4-3** Cisco IOS Software Command Help

| What You Enter | What Help You Get |
|---|---|
| **?** | This option provides help for all commands available in this mode. |
| *command* **?** | With a space between the command and the **?**, the switch lists text to describe all the first parameter options for the command. |
| *com***?** | This option lists commands that start with **com**. |
| *command parm***?** | This option lists all parameters beginning with the **parameter typed so far**. (Notice that there is no space between *parm* and the **?**.) |

| What You Enter | What Help You Get |
|---|---|
| *command parm*<Tab> | Pressing the Tab key causes IOS to spell out the rest of the word, assuming that you have typed enough of the word so there is only one option that begins with that string of characters. |
| *command parm1* ? | If a space is inserted before the question mark, the CLI lists all the next parameters and gives a brief explanation of each. |

When you enter the **?**, the Cisco IOS CLI reacts immediately; that is, you don't need to press the Enter key or any other keys. The device running Cisco IOS also redisplays what you entered before the **?** to save you some keystrokes. If you press Enter immediately after the **?**, Cisco IOS tries to execute the command with only the parameters you have entered so far.

The information supplied by using help depends on the CLI mode. For example, when **?** is entered in user mode, the commands allowed in user mode are displayed, but commands available only in enable mode (not in user mode) are not displayed. Also, help is available in **configuration mode**, which is the mode used to configure the switch. In fact, configuration mode has many different subconfiguration modes, as explained in the section "Configuration Submodes and Contexts," later in this chapter. So, you can get help for the commands available in each configuration submode as well. (Note that this might be a good time to use the free Sim Lite product on the companion website: to do so, open any lab, use the question mark, and try some commands.)

Cisco IOS stores the commands that you enter in a history buffer, storing ten commands by default. The CLI allows you to move backward and forward in the historical list of commands and then edit the command before reissuing it. These key sequences can help you use the CLI more quickly on the exams. Table 4-4 lists the commands used to manipulate previously entered commands.

**Table 4-4** Key Sequences for Command Edit and Recall

| Keyboard Command | What Happens |
|---|---|
| Up arrow or Ctrl+P | This keyboard command displays the most recently used command. If you press it again, the next most recent command appears, until the history buffer is exhausted. (The *P* stands for previous.) |
| Down arrow or Ctrl+N | If you have gone too far back into the history buffer, these keys take you forward to the more recently entered commands. (The *N* stands for next.) |
| Left arrow or Ctrl+B | This keyboard command moves the cursor backward in the currently displayed command without deleting characters. (The *B* stands for back.) |
| Right arrow or Ctrl+F | This keyboard command moves the cursor forward in the currently displayed command without deleting characters. (The *F* stands for forward.) |
| Backspace | This keyboard command moves the cursor backward in the currently displayed command, deleting characters. |

### The debug and show Commands

By far, the single most popular Cisco IOS command is the **show** command. The **show** command has a large variety of options, and with those options, you can find the status of almost every feature of Cisco IOS. Essentially, the **show** command lists the currently known facts about the switch's operational status.

For example, consider the output from the **show mac address-table dynamic** command listed in Example 4-3. This **show** command, issued from user mode, lists the table the switch uses to make forwarding decisions. A switch's MAC address table basically lists the data that a switch uses to do its primary job.

**Example 4-3** *Example MAC Address Table*

```
Certskills1> show mac address-table dynamic
 Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  31    0200.1111.1111    DYNAMIC     Gi0/1
  31    0200.3333.3333    DYNAMIC     Fa0/3
  31    1833.9d7b.0e9a    DYNAMIC     Gi0/1
  10    1833.9d7b.0e9a    DYNAMIC     Gi0/1
  10    30f7.0d29.8561    DYNAMIC     Gi0/1
   1    1833.9d7b.0e9a    DYNAMIC     Gi0/1
  12    1833.9d7b.0e9a    DYNAMIC     Gi0/1
Total Mac Addresses for this criterion: 7
Certskills1>
```

The **debug** command also tells the user details about the operation of the switch. However, while the **show** command lists status information at one instant of time—more like a photograph—the **debug** command acts more like a live video camera feed. Once you issue a **debug** command, IOS remembers, issuing messages over time as events continue to occur. Any switch user can choose to receive those messages, with the switch sending the messages to the console by default. Most of the commands used throughout this book to verify operation of switches and routers are **show** commands.

## Configuring Cisco IOS Software

You will want to configure every switch in an Enterprise network, even though the switches will forward traffic even with default configuration. This section covers the basic configuration processes, including the concept of a configuration file and the locations in which the configuration files can be stored. Although this section focuses on the configuration process, and not on the configuration commands themselves, you should know all the commands covered in this chapter for the exams, in addition to the configuration processes.

Configuration mode accepts *configuration commands*—commands that tell the switch the details of what to do and how to do it. User and privileged modes accept EXEC commands, which return output, or possibly take an action like reloading the switch, but commands in

these modes do not change any configuration settings. Figure 4-11 illustrates the navigation among configuration mode, user EXEC mode, and privileged EXEC mode.

Commands entered in configuration mode update the active configuration file. *These changes to the configuration occur immediately each time you press the Enter key at the end of a command.* Be careful when you enter a configuration command!

Key Topic

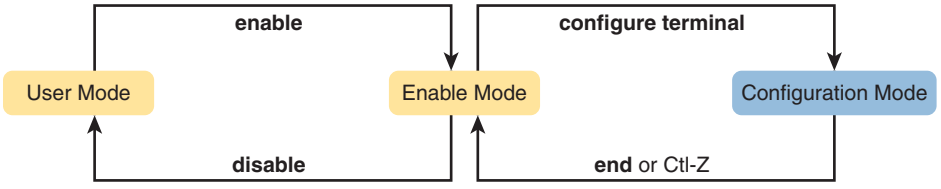



**Figure 4-11** *CLI Configuration Mode Versus EXEC Modes*

## Configuration Submodes and Contexts

Configuration mode supports a multitude of commands. To help organize the configuration, IOS groups some kinds of configuration commands together. To do that, when using configuration mode, you move from the initial mode—global configuration mode—into subcommand modes. *Context-setting commands* move you from one configuration subcommand mode, or context, to another. These context-setting commands tell the switch the topic about which you will enter the next few configuration commands. More importantly, the context tells the switch the topic you care about right now, so when you use the **?** to get help, the switch gives you help about that topic only.

**NOTE** *Context-setting* is not a Cisco term. It is just a description used here to help make sense of configuration mode.

The best way to learn about configuration submodes is to use them, but first, take a look at these upcoming examples. For instance, the **interface** command is one of the most commonly used context-setting configuration commands. For example, the CLI user could enter interface configuration mode by entering the **interface FastEthernet 0/1** configuration command. Asking for help in interface configuration mode displays only commands that are useful when configuring Ethernet interfaces. Commands used in this context are called *subcommands*—or, in this specific case, *interface subcommands*. When you begin practicing with the CLI using real equipment, the navigation between modes can become natural. For now, consider Example 4-4, which shows the following:

- Movement from enable mode to global configuration mode by using the **configure terminal** EXEC command.
- Use of a **hostname Fred** global configuration command to configure the switch's name. Using a global command from global configuration mode leaves you in global configuration mode.
- Movement from global configuration mode to console line configuration mode (using the **line console 0** command). The **line** command is another of the small set of context-setting commands that move you to another submode.

- Setting the console's simple password to **hope** (using the **password hope** line subcommand). Using a subcommand while in that submode leaves the command prompt in that submode.
- Movement from console configuration mode to interface configuration mode (using the **interface** *type number* command). The **interface** command is another of the small set of context-setting commands that move you to another submode.
- Setting the speed to 100 Mbps for interface Fa0/1 (using the **speed 100** interface subcommand).
- Movement from interface configuration mode back to global configuration mode (using the **exit** command).

**Example 4-4** *Navigating Between Different Configuration Modes*

```
Switch# configure terminal
Switch(config)# hostname Fred
Fred(config)# line console 0
Fred(config-line)# password hope
Fred(config-line)# interface FastEthernet 0/1
Fred(config-if)# speed 100
Fred(config-if)# exit
Fred(config)#
```

The text inside parentheses in the command prompt identifies the configuration mode. For example, the first command prompt after you enter configuration mode lists (config), meaning global configuration mode. After the **line console 0** command, the text expands to (config-line), meaning line configuration mode. Each time the command prompt changes within config mode, you have moved to another configuration mode.

Table 4-5 shows the most common command prompts in configuration mode, the names of those modes, and the context-setting commands used to reach those modes.

Key Topic

**Table 4-5** Common Switch Configuration Modes

| Prompt | Name of Mode | Context-Setting Command(s) to Reach This Mode |
|---|---|---|
| hostname(config)# | Global | None—first mode after **configure terminal** |
| hostname(config-line)# | Line | **line console 0**<br>**line vty 0 15** |
| hostname(config-if)# | Interface | **interface** *type number* |
| hostname(config-vlan)# | VLAN | **vlan** *number* |

You should practice until you become comfortable moving between the different configuration modes, back to enable mode, and then back into the configuration modes. However, you can learn these skills just doing labs about the topics in later chapters of the book. For now, Figure 4-12 shows most of the navigation between global configuration mode and the four configuration submodes listed in Table 4-5.

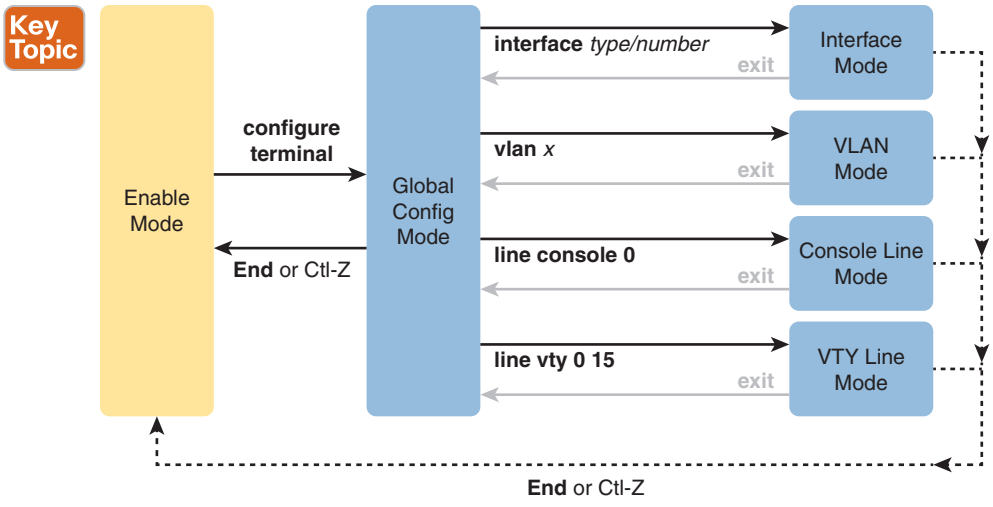



**Figure 4-12** *Navigation In and Out of Switch Configuration Modes*

You really should stop and try navigating around these configuration modes. If you have not yet decided on a lab strategy, install the Pearson Sim Lite software from the companion website. It includes the simulator and a couple of lab exercises. Start any lab, ignore the instructions, and just get into configuration mode and move around between the configuration modes shown in Figure 4-12.

No set rules exist for what commands happen to be global commands or subcommands. Instead, you learn new commands, see whether they are global commands, or if subcommands, you also learn the required mode. But generally, Cisco uses global commands for settings that apply to the entire switch and subcommands that apply to one component or feature. For example:

The global command **hostname** sets the one hostname for the entire switch.

The interface subcommand **speed** configures a setting for a specific interface, so it works only in interface configuration submode.

## Storing Switch Configuration Files

When you configure a switch, it needs to use the configuration. It also needs to be able to retain the configuration in case the switch loses power. Cisco switches contain random-access memory (RAM) to store data while Cisco IOS is using it, but RAM loses its contents when the switch loses power or is reloaded. To store information that must be retained when the switch loses power or is reloaded, Cisco switches use several types of more permanent memory, none of which has any moving parts. By avoiding components with moving parts (such as traditional disk drives), switches can maintain better uptime and availability.

The following list details the four main types of memory found in Cisco switches, as well as the most common use of each type:

- **RAM:** Sometimes called DRAM, for dynamic random-access memory, RAM is used by the switch just as it is used by any other computer: for working storage. The running (active) configuration file is stored here.

- **Flash memory:** Either a chip inside the switch or a removable memory card, flash memory stores fully functional Cisco IOS images and is the default location where the switch gets its Cisco IOS at boot time. Flash memory also can be used to store any other files, including backup copies of configuration files.
- **ROM:** Read-only memory (ROM) stores a bootstrap (or boothelper) program that is loaded when the switch first powers on. This bootstrap program then finds the full Cisco IOS image and manages the process of loading Cisco IOS into RAM, at which point Cisco IOS takes over operation of the switch.
- **NVRAM:** Nonvolatile RAM (NVRAM) stores the initial or startup configuration file that is used when the switch is first powered on and when the switch is reloaded.

Figure 4-13 summarizes this same information in a briefer and more convenient form for memorization and study.



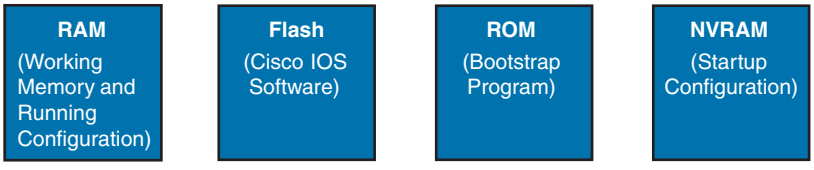



**Figure 4-13** *Cisco Switch Memory Types*

Cisco IOS stores the collection of configuration commands in a *configuration file*. In fact, switches use multiple configuration files—one file for the initial configuration used when powering on, and another configuration file for the active, currently used running configuration as stored in RAM. Table 4-6 lists the names of these two files, their purpose, and their storage location.

Key Topic

**Table 4-6** Names and Purposes of the Two Main Cisco IOS Configuration Files

| Configuration Filename | Purpose | Where It Is Stored |
|---|---|---|
| startup-config | Stores the initial configuration used anytime the switch reloads Cisco IOS. | NVRAM |
| running-config | Stores the currently used configuration commands. This file changes dynamically when someone enters commands in configuration mode. | RAM |

Essentially, when you use configuration mode, you change only the **running-config file**. This means that the configuration example earlier in this chapter (Example 4-4) updates only the running-config file. However, if the switch lost power right after that example, all that configuration would be lost. If you want to keep that configuration, you have to copy the running-config file into NVRAM, overwriting the old **startup-config file**.

Example 4-5 demonstrates that commands used in configuration mode change only the running configuration in RAM. The example shows the following concepts and steps:

**Step 1.** The example begins with both the running and startup-config having the same hostname, per the **hostname hannah** command.

**Step 2.** The hostname is changed in configuration mode using the **hostname harold** command.

**Step 3.** The **show running-config** and **show startup-config** commands show the fact that the hostnames are now different, with the **hostname harold** command found only in the running-config.

**Example 4-5** *How Configuration Mode Commands Change the Running-Config File, Not the Startup-Config File*

```
! Step 1 next (two commands)
!
hannah# show running-config
! (lines omitted)
hostname hannah
! (rest of lines omitted)

hannah# show startup-config
! (lines omitted)
hostname hannah
! (rest of lines omitted)
! Step 2 next. Notice that the command prompt changes immediately after
! the hostname command.

hannah# configure terminal
hannah(config)# hostname harold
harold(config)# exit
! Step 3 next (two commands)
!
harold# show running-config
! (lines omitted) - just showing the part with the hostname command
hostname harold
!
harold# show startup-config
! (lines omitted) - just showing the part with the hostname command
hostname hannah
```

### Copying and Erasing Configuration Files

The configuration process updates the running-config file, which is lost if the router loses power or is reloaded. Clearly, IOS needs to provide us a way to copy the running configuration so that it will not be lost, so it will be used the next time the switch reloads or powers on. For instance, Example 4-5 ended with a different running configuration (with the **hostname harold** command) versus the startup configuration.

In short, the EXEC command **copy running-config startup-config** backs up the running-config to the startup-config file. This command overwrites the current startup-config file with what is currently in the running-config file.

In addition, in the lab, you may want to just get rid of all existing configuration and start over with a clean configuration. To do that, you can erase the startup-config file using three different commands:

```
write erase
erase startup-config
erase nvram:
```

Once the startup-config file is erased, you can reload or power off/on the switch, and it will boot with the now-empty startup configuration.

Note that Cisco IOS does not have a command that erases the contents of the running-config file. To clear out the running-config file, simply erase the startup-config file, and then **reload** the switch, and the running-config will be empty at the end of the process.

**NOTE** Cisco uses the term *reload* to refer to what most PC operating systems call rebooting or restarting. In each case, it is a re-initialization of the software. The **reload** EXEC command causes a switch to reload.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or on the book's companion website. Refer to the "Your Study Plan" element section titled "Step 2: Build Your Study Habits Around the Chapter" for more details. Table 4-7 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 4-7** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |
| Review command tables | | Book |

## Review All the Key Topics

Key Topic

**Table 4-8** Key Topics for Chapter 4

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 4-3 | Three methods to access a switch CLI | 93 |
| Figure 4-4 | Cabling options for a console connection | 94 |
| List | A Cisco switch's default console port settings | 95 |
| Figure 4-11 | Navigation between user, enable, and global config modes | 104 |
| Table 4-5 | A list of configuration mode prompts, the name of the configuration mode, and the command used to reach each mode | 105 |
| Figure 4-12 | Configuration mode context-setting commands | 106 |
| Table 4-6 | The names and purposes of the two configuration files in a switch or router | 107 |

## Key Terms You Should Know

command-line interface (CLI), configuration mode, enable mode, IOS, IOS XE, rollover cable, running-config file, Secure Shell (SSH), startup-config file, Telnet, user mode

## Command References

Tables 4-9 and 4-10 list configuration and verification commands used in this chapter, respectively. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 4-9** Chapter 4 Configuration Commands

| Command | Mode and Purpose |
|---|---|
| **line console 0** | Global command that changes the context to console configuration mode. |
| **login** | Line (console and vty) configuration mode. Tells IOS to prompt for a password (no username). |
| **password** *pass-value* | Line (console and vty) configuration mode. Sets the password required on that line for login if the **login** command (with no other parameters) is also configured. |
| **interface** *type port-number* | Global command that changes the context to interface mode—for example, **interface FastEthernet 0/1**. |
| **speed** *value* | Interface subcommand that sets the Ethernet interface speed on interfaces that support multiple speeds. |
| **hostname** *name* | Global command that sets this switch's hostname, which is also used as the first part of the switch's command prompt. |
| **exit** | Command that moves back to the next higher mode in configuration mode. |

| Command | Mode and Purpose |
|---|---|
| **end** | Command that exits configuration mode and goes back to enable mode from any of the configuration submodes. |
| Ctrl+Z | This is not a command, but rather a two-key combination (pressing the Ctrl key and the letter Z) that together do the same thing as the **end** command. |

**Table 4-10** Chapter 4 EXEC Command Reference

| Command | Purpose |
|---|---|
| **no debug all**<br>**undebug all** | Privileged mode EXEC command to disable all currently enabled debugs. |
| **reload** | Privileged mode EXEC command that reboots the switch or router. |
| **copy running-config startup-config** | Privileged mode EXEC command that saves the active config, replacing the startup-config file used when the switch initializes. |
| **copy startup-config running-config** | Privileged mode EXEC command that merges the startup-config file with the currently active config file in RAM. |
| **show running-config** | Privileged mode EXEC command that lists the contents of the running-config file. |
| **write erase**<br>**erase startup-config**<br>**erase nvram:** | Privileged mode EXEC command that erases the startup-config file. |
| **quit** | EXEC command that disconnects the user from the CLI session. |
| **show startup-config** | Privileged mode EXEC command that lists the contents of the startup-config (initial config) file. |
| **enable** | User mode EXEC command that moves the user from user mode to enable (privileged) mode and prompts for a password if one is configured. |
| **disable** | Privileged mode EXEC command that moves the user from privileged mode to user mode. |
| **configure terminal** | Privileged mode EXEC command that moves the user into configuration mode. |
| **show mac address-table** | EXEC command that lists the contents of a switch forwarding (MAC) table. |



CHAPTER 5

# Analyzing Ethernet LAN Switching

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.1 Explain the role and function of network components**
    - **1.1.b Layer 2 and Layer 3 switches**
  - **1.13 Describe switching concepts**
    - **1.13.a MAC learning and aging**
    - **1.13.b Frame switching**
    - **1.13.c Frame flooding**
    - **1.13.d MAC address table**
- **2.0 Network Access**
  - **2.5 Interpret basic operations of Spanning Tree Protocols**

When you buy a Cisco Catalyst Ethernet switch, the switch is ready to work. All you have to do is take it out of the box, power on the switch by connecting the power cable to the switch and a power outlet, and connect hosts to the switch using the correct unshielded twisted-pair (UTP) cables. You do not have to configure anything else, or connect to the console and login, or do anything: the switch just starts forwarding Ethernet frames.

In Part II of this book, you will learn how to build, configure, and verify the operation of Ethernet LANs. In Chapter 4, "Using the Command-Line Interface," you learned how to move around in the CLI, issue commands, and configure the switch. This chapter takes a short but important step in that journey by explaining the logic a switch uses when forwarding Ethernet frames.

This chapter breaks the content into two major sections. The first reviews and then further develops the concepts behind LAN switching, which were first introduced back in Chapter 2, "Fundamentals of Ethernet LANs." The second section then uses IOS **show** commands to verify that Cisco switches actually learned the MAC addresses, built its MAC address table, and forwarded frames.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 5-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| LAN Switching Concepts | 1–4 |
| Verifying and Analyzing Ethernet Switching | 5–6 |

1. Which of the following statements describes part of the process of how a switch decides to forward a frame destined for a known unicast MAC address?
   a. It compares the unicast destination address to the bridging, or MAC address, table.
   b. It compares the unicast source address to the bridging, or MAC address, table.
   c. It forwards the frame out all interfaces in the same VLAN except for the incoming interface.
   d. It compares the destination IP address to the destination MAC address.
   e. It compares the frame's incoming interface to the source MAC entry in the MAC address table.
2. Which of the following statements best describes the forwarding logic that a LAN switch, with all interfaces assigned to VLAN 1 as per default settings, uses for an incoming frame with a destination MAC address of FFFF.FFFF.FFFF?
   a. It forwards the frame out all switch ports.
   b. It forwards the frame out all switch ports except the arrival port.
   c. It forwards the frame out all ports that had earlier registered to ask to receive broadcasts.
   d. It discards the frame.
3. Which of the following statements best describes what a switch does with a frame destined for an unknown unicast address?
   a. It forwards out all interfaces in the same VLAN except for the incoming interface.
   b. It forwards the frame out the one interface identified by the matching entry in the MAC address table.
   c. It compares the destination IP address to the destination MAC address.
   d. It compares the frame's incoming interface to the source MAC entry in the MAC address table.
4. Which of the following comparisons does a switch make when deciding whether a new MAC address should be added to its MAC address table?
   a. It compares the unicast destination address to the bridging, or MAC address, table.
   b. It compares the unicast source address to the bridging, or MAC address, table.
   c. It compares the VLAN ID to the bridging, or MAC address, table.
   d. It compares the destination IP address's ARP cache entry to the bridging, or MAC address, table.

5. A Cisco Catalyst switch has 24 10/100 ports, numbered 0/1 through 0/24. Ten PCs connect to the ten lowest numbered ports, with those PCs working and sending data over the network. The other ports are not connected to any device. Which of the following answers lists facts displayed by the **show interfaces status** command?
   a. Port Ethernet 0/1 is in a connected state.
   b. Port Fast Ethernet 0/11 is in a connected state.
   c. Port Fast Ethernet 0/5 is in a connected state.
   d. Port Ethernet 0/15 is in a notconnected state.
6. Consider the following output from a Cisco Catalyst switch:

```
SW1# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address        Type         Ports
----    -----------        --------     -----
   1    02AA.AAAA.AAAA     DYNAMIC      Gi0/1
   1    02BB.BBBB.BBBB     DYNAMIC      Gi0/2
   1    02CC.CCCC.CCCC     DYNAMIC      Gi0/3
Total Mac Addresses for this criterion: 3
```

   Which of the following answers is true about this switch?
   a. The output proves that port Gi0/2 connects directly to a device that uses address 02BB.BBBB.BBBB.
   b. The switch has learned three MAC addresses since the switch powered on.
   c. The three listed MAC addresses were learned based on the destination MAC address of frames forwarded by the switch.
   d. Address 02CC.CCCC.CCCC was learned from the source MAC address of a frame that entered port Gi0/3.

# Foundation Topics

## LAN Switching Concepts

A modern Ethernet LAN connects user devices as well as servers into some switches, with the switches then connecting to each other, sometimes in a design like Figure 5-1. Part of the LAN, called a campus LAN, supports the end-user population as shown on the left of the figure. End-user devices connect to LAN switches, which in turn connect to other switches so that a path exists to the rest of the network. The campus LAN switches sit in wiring closets close to the end users. On the right, the servers used to provide information to the users also connect to the LAN. Those servers and switches often sit in a closed room called a *data center*, with connections to the campus LAN to support traffic to/from the users.

To forward traffic from a user device to a server and back, each switch performs the same kind of logic, independently from each other. The first half of this chapter examines the logic—how a switch chooses to forward an Ethernet frame, when the switch chooses to not forward the frame, and so on.

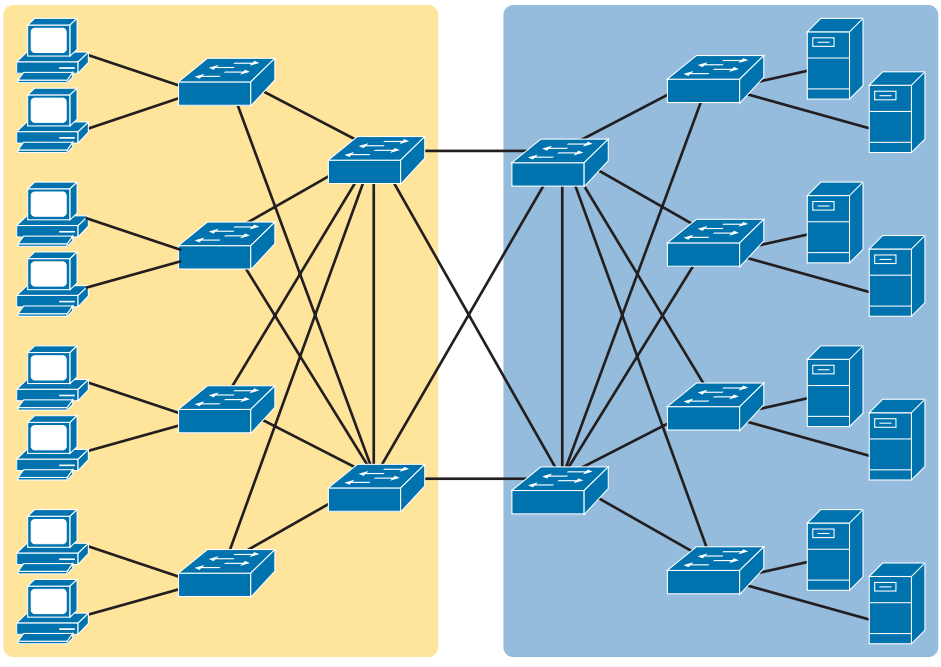



**Figure 5-1** *Campus LAN and Data Center LAN, Conceptual Drawing*

## Overview of Switching Logic

Ultimately, the role of a LAN switch is to forward Ethernet frames. LANs exist as a set of user devices, servers, and other devices that connect to switches, with the switches connected to each other. The LAN switch has one primary job: to forward frames to the correct destination (MAC) address. And to achieve that goal, switches use logic—logic based on the source and destination MAC address in each frame's Ethernet header.

LAN switches receive Ethernet frames and then make a switching decision: either forward the frame out some other ports or ignore the frame. To accomplish this primary mission, switches perform three actions:

Key Topic

1. Deciding when to forward a frame or when to filter (not forward) a frame, based on the destination MAC address
2. Preparing to forward future frames by learning the source MAC address of each frame received by the switch
3. Cooperating with all switches to prevent the endless looping of frames by using Spanning Tree Protocol (STP)

The first action is the switch's primary job, whereas the other two items are overhead functions.

**NOTE** Throughout this book's discussion of LAN switches, the terms *switch port* and *switch interface* are synonymous.

Most of the upcoming discussions and figures about Ethernet switching focus on the use of the ever-present destination and source MAC address fields in the header. Both are 6 bytes long (represented as 12 hex digits in the book) and are a key part of the switching logic discussed in this section. Refer back to Chapter 2's section titled "Ethernet Data-Link Protocols" for a discussion of the header in detail for more info on the rest of the Ethernet frame. Figure 5-2 repeats the frame format here for reference.

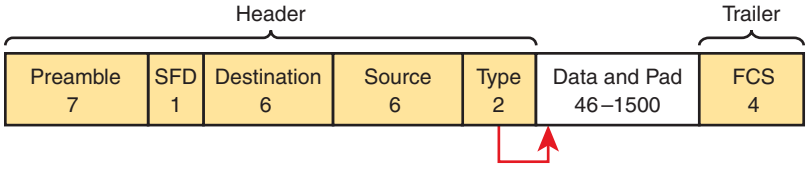



**Figure 5-2** *IEEE 802.3 Ethernet Frame (One Variation)*

Now on to the details of how Ethernet switching works!

## Forwarding Known Unicast Frames

To decide whether to forward a frame, a switch uses a dynamically built table that lists MAC addresses and outgoing interfaces. Switches compare the frame's destination MAC address to this table to decide whether the switch should forward a frame or simply ignore it. For example, consider the simple network shown in Figure 5-3, with Fred sending a frame to Barney.

Key Topic

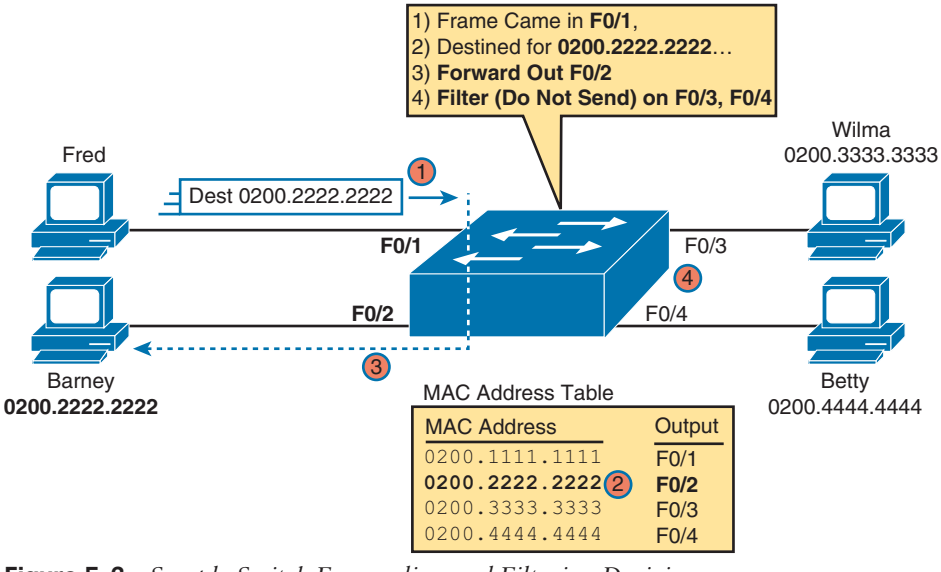



**Figure 5-3** *Sample Switch Forwarding and Filtering Decision*

Answers to the "Do I Know This Already?" quiz:

**1** A **2** B **3** A **4** B **5** C **6** D

In this figure, Fred sends a frame with destination address 0200.2222.2222 (Barney's MAC address). The switch compares the destination MAC address (0200.2222.2222) to the MAC address table, matching the bold table entry. That matched table entry tells the switch to forward the frame out port F0/2, and only port F0/2.

**NOTE** A switch's **MAC address table** is also called the *switching table*, or *bridging table*, or even the *content-addressable memory (CAM) table*, in reference to the type of physical memory used to store the table.

A switch's MAC address table lists the location of each MAC relative to that one switch. In LANs with multiple switches, each switch makes an independent forwarding decision based on its own MAC address table. Together, they forward the frame so that it eventually arrives at the destination.

For example, Figure 5-4 shows the first switching decision in a case in which Fred sends a frame to Wilma, with destination MAC 0200.3333.3333. The topology has changed versus the previous figure, this time with two switches, and Fred and Wilma connected to two different switches. Figure 5-4 shows the first switch's logic, in reaction to Fred sending the original frame. Basically, the switch receives the frame in port F0/1, finds the destination MAC (0200.3333.3333) in the MAC address table, sees the outgoing port of G0/1, so SW1 forwards the frame out its G0/1 port.

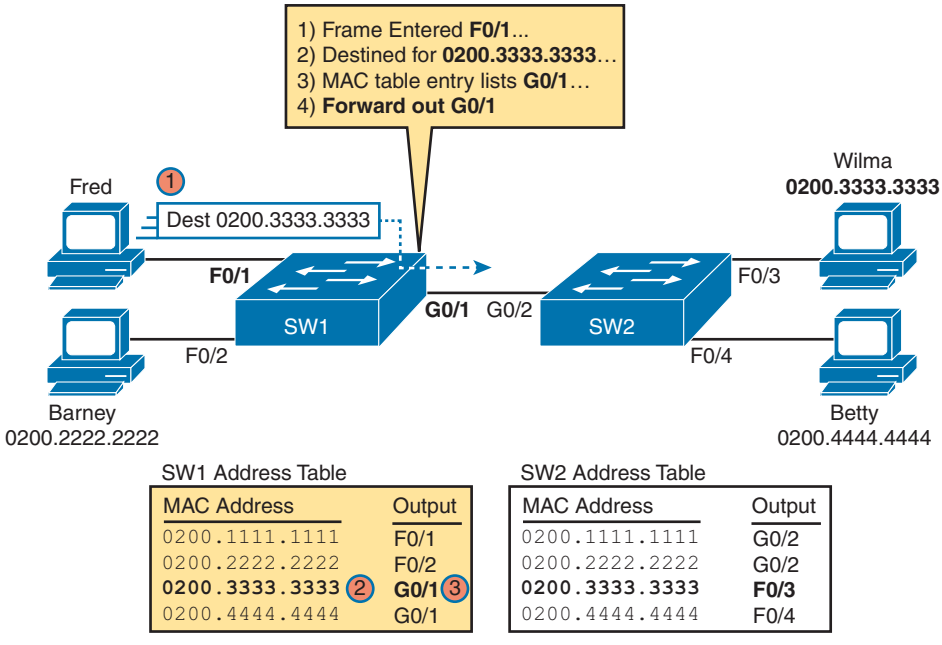



**Figure 5-4** *Forwarding Decision with Two Switches: First Switch*

That same frame, after being forwarded by switch SW1, arrives at switch SW2, entering SW2's G0/2 interface. As shown in Figure 5-5, SW2 uses the same logic steps, but using SW2's table. The MAC table lists the forwarding instructions for that switch only. In this case, switch SW2 forwards the frame out its F0/3 port, based on SW2's MAC address table.

Key Topic

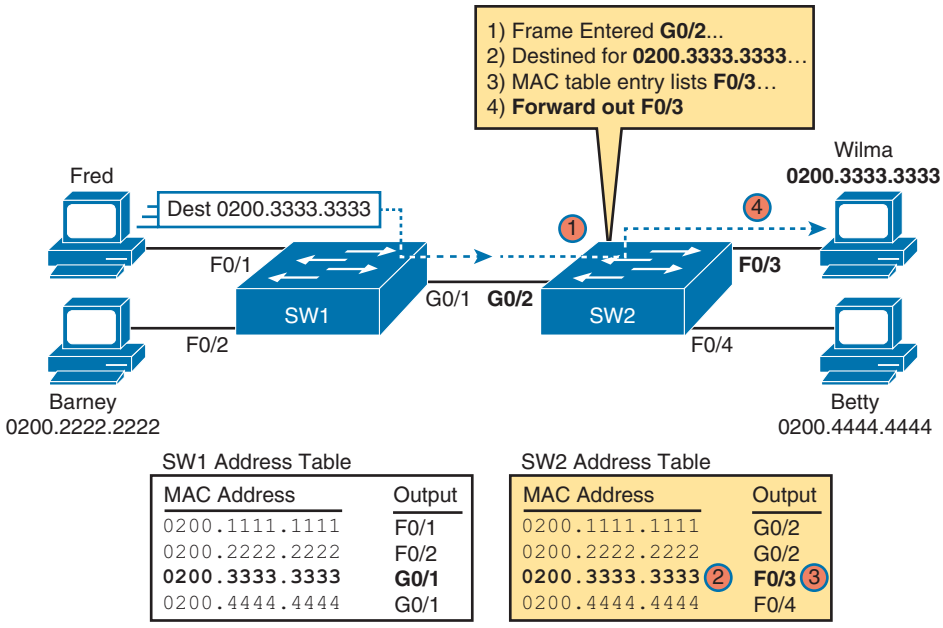



**Figure 5-5** *Forwarding Decision with Two Switches: Second Switch*

**NOTE** The term *forward-versus-filter decision* emphasizes the switch's choice to forward the frame out some ports but not forward (filter) the frame from being sent out other ports.

The examples so far use switches that happen to have a MAC table with all the MAC addresses listed. As a result, the destination MAC address in the frame is known to the switch. The frames are called **known unicast frames**, or simply known unicasts, because the destination address is a unicast address, and the destination is known. As shown in these examples, switches forward known unicast frames out one port: the port as listed in the MAC table entry for that MAC address.

## Learning MAC Addresses

Thankfully, the networking staff does not have to type in all those MAC table entries. Instead, the switches do their second main function: to learn the MAC addresses and interfaces to put into its address table. With a complete MAC address table, the switch can make accurate forwarding and filtering decisions as just discussed.

Switches build the address table by listening to incoming frames and examining the *source MAC address* in the frame. If a frame enters the switch and the source MAC address is not in the MAC address table, the switch creates an entry in the table. That table entry lists the interface from which the frame arrived. Switch learning logic is that simple.

Figure 5-6 depicts the same single-switch topology network as Figure 5-3, but before the switch has built any address table entries. The figure shows the first two frames sent in this network: first a frame from Fred addressed to Barney, and then Barney's response, addressed to Fred.

Key Topic

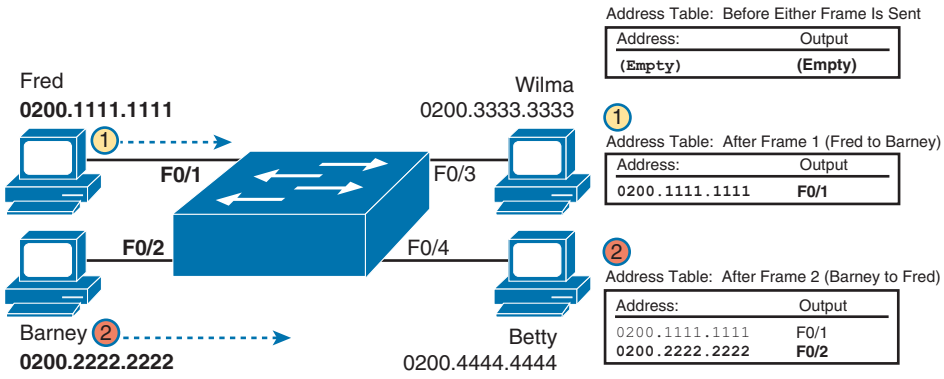



**Figure 5-6** *Switch Learning: Empty Table and Adding Two Entries*

The figure does not show lines for where the frames flow, focusing instead on the arrival of the frames into the switch.

Focus on the learning process and how the MAC table grows at each step as shown on the right side of the figure. The switch begins with an empty MAC table, as shown in the upper-right part of the figure. Then Fred sends his first frame (labeled "1") to Barney, so the switch adds an entry for 0200.1111.1111, Fred's MAC address, associated with interface F0/1. Why F0/1? The frame sent by Fred entered the switch's F0/1 port. SW1's logic runs something like this: "The source is MAC 0200.1111.1111, the frame entered F0/1, so from my perspective, 0200.1111.1111 must be reachable out my port F0/1."

Continuing the example, when Barney replies in Step 2, the switch adds a second entry, this one for 0200.2222.2222, Barney's MAC address, along with interface F0/2. Why F0/2? The frame Barney sent entered the switch's F0/2 interface. Learning always occurs by looking at the source MAC address in the frame and adds the incoming interface as the associated port.

## Flooding Unknown Unicast and Broadcast Frames

Now again turn your attention to the forwarding process, using the topology in Figure 5-5. What do you suppose the switch does with Fred's first frame, the one that occurred when there were no entries in the MAC address table? As it turns out, when there is no matching entry in the table, switches **forward** the frame out all interfaces (except the incoming interface) using a process called *flooding*. And the frame whose destination address is unknown to the switch is called an **unknown unicast frame**, or simply an *unknown unicast*.

Switches **flood** unknown unicast frames. Flooding means that the switch forwards copies of the frame out all ports, except the port on which the frame was received. The idea is simple: if you do not know where to send it, send it everywhere, to deliver the frame. And, by the way, that device will likely then send a reply—and then the switch can learn that device's MAC address and forward future frames out one port as a known unicast frame.

Switches also flood LAN **broadcast frames** (frames destined to the Ethernet broadcast address of FFFF.FFFF.FFFF) because this process helps deliver a copy of the frame to all devices in the LAN.

Figure 5-7 shows the same scenario as in Figure 5-6, with the first frame sent by Fred, when the switch's MAC table is empty—but focusing on switch forwarding. At Step 1, Fred sends the frame. At Step 2, the switch floods a copy of the frame out all three of the other interfaces.

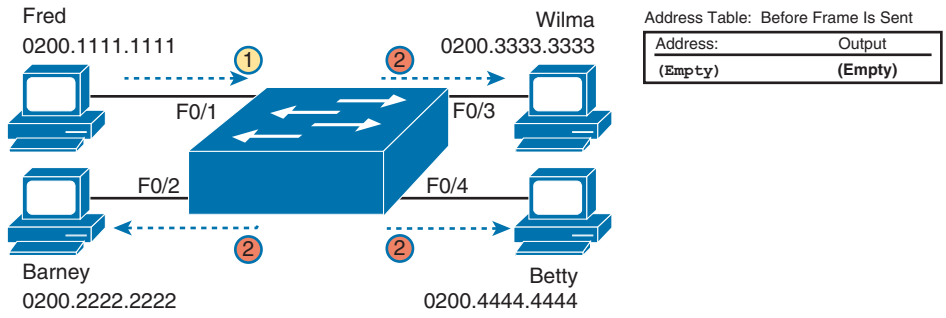



**Figure 5-7** *Switch Flooding: Unknown Unicast Arrives, Floods Out Other Ports*

## Avoiding Loops Using Spanning Tree Protocol

The third primary feature of LAN switches is loop prevention, as implemented by the **Spanning Tree Protocol (STP)**. Without STP, any flooded frames would loop for an indefinite period of time in Ethernet networks with physically redundant links. To prevent looping frames, STP blocks some ports from forwarding frames so that only one active path exists between any pair of LAN segments.

A simple example makes the need for STP more obvious. Remember, switches flood unknown unicast frames and broadcast frames. Figure 5-8 shows an unknown unicast frame, sent by Larry to Bob, which loops forever because the network has redundancy but no STP. Note that the figure shows one direction of the looping frame only, just to reduce clutter, but a copy of the frame would also loop the other direction.

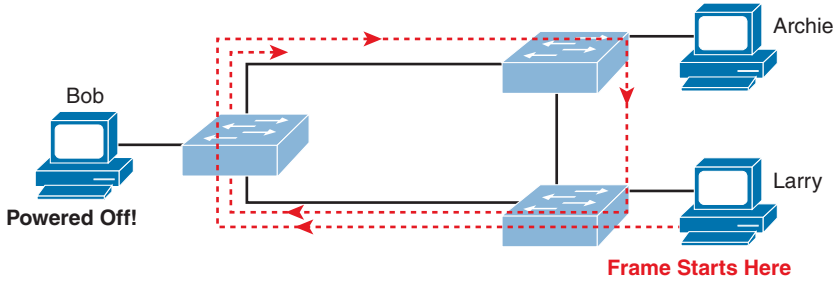



**Figure 5-8** *Network with Redundant Links but Without STP: The Frame Loops Forever*

The flooding of this frame would cause the frame to rotate around the three switches; because none of the switches list Bob's MAC address in their address tables, each switch floods the frame. And while the flooding process is a good mechanism for forwarding unknown unicasts and broadcasts, the continual flooding of traffic frames as in the figure can completely congest the LAN to the point of making it unusable.

A topology like Figure 5-8, with redundant links, is good, but we need to prevent the bad effect of those looping frames. To avoid Layer 2 loops, all switches need to use STP. STP causes each interface on a switch to settle into either a blocking state or a forwarding state. *Blocking* means that the interface cannot forward or receive data frames, while *forwarding*

means that the interface can send and receive data frames. If a correct subset of the interfaces is blocked, only a single currently active logical path exists between each pair of LANs.

Chapter 9 of this book, "Spanning Tree Protocol Concepts," examines STP in depth, including how STP prevents loops.

### LAN Switching Summary

Switches use Layer 2 logic, examining the Ethernet data-link header to choose how to process frames. In particular, switches make decisions to forward and filter frames, learn MAC addresses, and use STP to avoid loops, as follows:

Key Topic

**Step 1.** Switches forward frames based on the destination MAC address:

- **a.** If the destination MAC address is a broadcast, multicast, or unknown destination unicast (a unicast not listed in the MAC table), the switch floods the frame.
- **b.** If the destination MAC address is a known unicast address (a unicast address found in the MAC table):
  - **i.** If the outgoing interface listed in the MAC address table is different from the interface in which the frame was received, the switch forwards the frame out the outgoing interface.
  - **ii.** If the outgoing interface is the same as the interface in which the frame was received, the switch filters the frame, meaning that the switch simply ignores the frame and does not forward it.

**Step 2.** Switches learn MAC address table entries based on the source MAC address:

- **a.** For each received frame, note the source MAC address and incoming interface ID.
- **b.** If not yet in the MAC address table, add an entry listing the MAC address and incoming interface.

**Step 3.** Switches use STP to prevent loops by causing some interfaces to block, meaning that they do not send or receive frames.

## Verifying and Analyzing Ethernet Switching

A Cisco Catalyst switch comes from the factory ready to switch frames. All you have to do is connect the power cable, plug in the Ethernet cables, and the switch starts switching incoming frames. Connect multiple switches together, and they are ready to forward frames between the switches as well. And the big reason behind this default behavior has to do with the default settings on the switches.

Cisco Catalyst switches come ready to get busy switching frames because of settings like these:

- The interfaces are enabled by default, ready to start working once a cable is connected.
- All interfaces are assigned to VLAN 1.
- 10/100 and 10/100/1000 interfaces use autonegotiation by default.

- The MAC learning, forwarding, flooding logic all works by default.
- STP is enabled by default.

This second section of the chapter examines how switches will work with these default settings, showing how to verify the Ethernet learning and forwarding process.

## Demonstrating MAC Learning

To see a switch's MAC address table, use the **show mac address-table** command. With no additional parameters, this command lists all known MAC addresses in the MAC table, including some overhead static MAC addresses that you can ignore. To see all the dynamically learned MAC addresses only, instead use the **show mac address-table dynamic** command, as seen in Example 5-1.

Key Topic

**Example 5-1** show mac address-table dynamic *for Figure 5-9*

```
SW1# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
   1    0200.1111.1111    DYNAMIC     Fa0/1
   1    0200.2222.2222    DYNAMIC     Fa0/2
   1    0200.3333.3333    DYNAMIC     Fa0/3
   1    0200.4444.4444    DYNAMIC     Fa0/4
Total Mac Addresses for this criterion: 4
SW1#
```

First, focus on two columns of the table: the MAC Address and Ports columns of the table. The values should look familiar: they match the earlier single-switch example, as repeated here as Figure 5-9. Note the four MAC addresses listed, along with their matching ports, as shown in the figure.

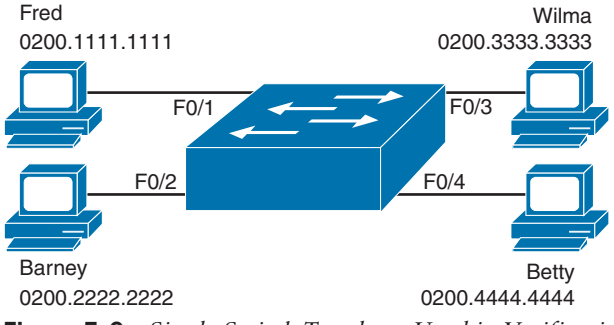



**Figure 5-9** *Single Switch Topology Used in Verification Section*

Next, look at the Type field in the heading of the output table. The column tells us how the switch learned the MAC address as described earlier in this chapter; in this case, the switch learned all MAC addresses dynamically. You can also statically predefine MAC table entries

using a couple of different features, including port security, and those would appear as Static in the Type column.

Finally, the VLAN column of the output gives us a chance to briefly discuss how virtual LANs (VLANs) impact switching logic. LAN switches forward Ethernet frames inside a VLAN. What that means is if a frame enters via a port in VLAN 1, then the switch will forward or flood that frame out other ports in VLAN 1 only, and not out any ports that happen to be assigned to another VLAN. Chapter 8, "Implementing Ethernet Virtual LANs," looks at all the details of how switches forward frames when using VLANs.

## Switch Interfaces

The first example assumes that you installed the switch and cabling correctly, and that the switch interfaces work. You can easily check the status of those interfaces with the **show interfaces status** command, as shown in Example 5-2.

**Example 5-2** **show interfaces status** *on Switch SW1*

```
SW1# show interfaces status

Port      Name               Status       Vlan       Duplex  Speed  Type
Fa0/1                        connected    1          a-full  a-100 10/100BaseTX
Fa0/2                        connected    1          a-full  a-100 10/100BaseTX
Fa0/3                        connected    1          a-full  a-100 10/100BaseTX
Fa0/4                        connected    1          a-full  a-100 10/100BaseTX
Fa0/5                        notconnect   1          auto     auto 10/100BaseTX
Fa0/6                        notconnect   1          auto     auto 10/100BaseTX
Fa0/7                        notconnect   1          auto     auto 10/100BaseTX
Fa0/8                        notconnect   1          auto     auto 10/100BaseTX
Fa0/9                        notconnect   1          auto     auto 10/100BaseTX
Fa0/10                       notconnect   1          auto     auto 10/100BaseTX
Fa0/11                       notconnect   1          auto     auto 10/100BaseTX
Fa0/12                       notconnect   1          auto     auto 10/100BaseTX
Fa0/13                       notconnect   1          auto     auto 10/100BaseTX
Fa0/14                       notconnect   1          auto     auto 10/100BaseTX
Fa0/15                       notconnect   1          auto     auto 10/100BaseTX
Fa0/16                       notconnect   1          auto     auto 10/100BaseTX
Fa0/17                       notconnect   1          auto     auto 10/100BaseTX
Fa0/18                       notconnect   1          auto     auto 10/100BaseTX
Fa0/19                       notconnect   1          auto     auto 10/100BaseTX
Fa0/20                       notconnect   1          auto     auto 10/100BaseTX
Fa0/21                       notconnect   1          auto     auto 10/100BaseTX
Fa0/22                       notconnect   1          auto     auto 10/100BaseTX
Fa0/23                       notconnect   1          auto     auto 10/100BaseTX
Fa0/24                       notconnect   1          auto     auto 10/100BaseTX
Gi0/1                        notconnect   1          auto     auto 10/100/1000BaseTX
Gi0/2                        notconnect   1          auto     auto 10/100/1000BaseTX
```

Focus on the Port column for a moment. As a reminder, Cisco Catalyst switches name their ports based on the fastest specification supported, so in this case, the switch has 24 interfaces named FastEthernet and two named GigabitEthernet. Many commands abbreviate those terms, this time as Fa for FastEthernet and Gi for GigabitEthernet. (The example happens to come from a Cisco Catalyst switch that has 24 10/100 ports and two 10/100/1000 ports.)

The Status column, of course, tells us the status or state of the port. In this case, the lab switch had cables and devices connected to ports F0/1–F0/4 only, with no other cables connected. As a result, those first four ports have a state of connected, meaning that the ports have a cable and are functional. The notconnect state means that the port is not yet functioning. It may mean that there is no cable installed, but other problems may exist as well. (The section "Analyzing Switch Interface Status and Statistics," in Chapter 7, "Configuring and Verifying Switch Interfaces," works through the details of what causes a switch interface to fail.)

**NOTE** You can see the status for a single interface in a couple of ways. For instance, for F0/1, the command **show interfaces f0/1 status** lists the status in a single line of output as in Example 5-2. The **show interfaces f0/1** command (without the **status** keyword) displays a detailed set of messages about the interface.

The **show interfaces** command has a large number of options. One particular option, the **counters** option, lists statistics about incoming and outgoing frames on the interfaces. In particular, it lists the number of unicast, multicast, and broadcast frames (both the in and out directions), and a total byte count for those frames. Example 5-3 shows an example, again for interface F0/1.

**Example 5-3** **show interfaces f0/1 counters** *on Switch SW1*

```
SW1# show interfaces f0/1 counters

Port            InOctets    InUcastPkts      InMcastPkts      InBcastPkts
Fa0/1            1223303          10264              107               18

Port           OutOctets    OutUcastPkts    OutMcastPkts     OutBcastPkts
Fa0/1            3235055          13886            22940              437
```

## Finding Entries in the MAC Address Table

With a single switch and only four hosts connected to them, you can just read the details of the MAC address table and find the information you want to see. However, in real networks, with lots of interconnected hosts and switches, just reading the output to find one MAC address can be hard to do. You might have hundreds of entries—page after page of output—with each MAC address looking like a random string of hex characters. (The book uses easy-to-recognize MAC addresses to make it easier to learn.)

Thankfully, Cisco IOS supplies several more options on the **show mac address-table** command to make it easier to find individual entries. First, if you know the MAC address, you

can search for it—just type in the MAC address at the end of the command, as shown in Example 5-4. All you have to do is include the **address** keyword, followed by the actual MAC address. If the address exists, the output lists the address. Note that the output lists the exact same information in the exact same format, but it lists only the line for the matching MAC address.

**Example 5-4** **show mac address-table dynamic** *with the* **address** *Keyword*

```
SW1# show mac address-table dynamic address 0200.1111.1111
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
   1    0200.1111.1111    DYNAMIC     Fa0/1
Total Mac Addresses for this criterion: 1
```

While this information is useful, often the engineer troubleshooting a problem does not know the MAC addresses of the devices connected to the network. Instead, you might be troubleshooting while looking at a network topology diagram and want to look at all the MAC addresses learned off a particular port. IOS supplies that option with the **show mac address-table dynamic interface** command. Example 5-5 shows one example, for switch SW1's F0/1 interface.

**Example 5-5** **show mac address-table dynamic** *with the* **interface** *Keyword*

```
SW1# show mac address-table dynamic interface fastEthernet 0/1
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
   1    0200.1111.1111    DYNAMIC     Fa0/1
Total   Mac Addresses for this criterion: 1
```

Finally, you may also want to find the MAC address table entries for one VLAN. You guessed it—you can add the **vlan** parameter, followed by the VLAN number. Example 5-6 shows two such examples from the same switch SW1 from Figure 5-9—one for VLAN 1, where all four devices reside, and one for a nonexistent VLAN 2.

**Example 5-6** *The* **show mac address-table vlan** *command*

```
SW1# show mac address-table dynamic vlan 1
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
   1    0200.1111.1111    DYNAMIC     Fa0/1
```

```
   1    0200.2222.2222    DYNAMIC     Fa0/2
   1    0200.3333.3333    DYNAMIC     Fa0/3
   1    0200.4444.4444    DYNAMIC     Fa0/4
Total Mac Addresses for this criterion: 4
SW1#
SW1# show mac address-table dynamic vlan 2
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
SW1#
```

## Managing the MAC Address Table (Aging, Clearing)

This chapter closes with a few comments about how switches manage their MAC address tables. MAC addresses do not remain in the table indefinitely. The switch will remove the entries due to age, due to the table filling, and you can remove entries using a command.

First, for aging out MAC table entries, switches remove entries that have not been used for a defined number of seconds (default of 300 seconds). To do that, switches keep a timer for each MAC table entry that increases over time. However, the switch resets the timer to 0 when it receives another frame with that same source MAC address. Timers that have not been reset continue to grow, and once a timer reaches the aging setting, the switch removes the MAC table entry.

Example 5-7 shows the aging timer setting for the entire switch. The aging time can be configured to a different time, globally and per-VLAN using the **mac address-table aging-time** *time-in-seconds* **[vlan** *vlan-number***]** global configuration command. The example shows a case with all defaults, with the global setting of 300 seconds, and no per-VLAN overrides.

**Example 5-7** *The MAC Address Default Aging Timer Displayed*

```
SW1# show mac address-table aging-time
Global Aging Time:  300
Vlan    Aging Time
----    ----------
SW1#

SW1# show mac address-table count

Mac Entries for Vlan 1:
---------------------------
Dynamic Address Count   : 4
Static  Address Count   : 0
Total Mac Addresses     : 4

Total Mac Address Space Available: 7299
```

Each switch also removes the oldest table entries, even if they are younger than the aging time setting, if the table fills. The MAC address table uses content-addressable memory (CAM), a physical memory that has great table lookup capabilities. However, the size of the table depends on the size of the CAM in a particular model of switch and based on some configurable settings in the switch. When a switch tries to add a new MAC table entry and finds the table full, the switch removes the oldest table entry to make space. For perspective, the end of Example 5-7 lists the size of a Cisco Catalyst switch's MAC table at about 8000 entries—the same four existing entries from the earlier examples, with space for 7299 more.

Finally, you can remove the dynamic entries from the MAC address table with the **clear mac address-table dynamic** command. Note that the **show** commands in this chapter can be executed from user and enable mode, but the **clear** command happens to be an enable mode command. The command also allows parameters to limit the types of entries cleared, as follows:

- **By VLAN: clear mac address-table dynamic vlan** *vlan-number*
- **By Interface: clear mac address-table dynamic interface** *interface-id*
- **By MAC address: clear mac address-table dynamic address** *mac-address*



## MAC Address Tables with Multiple Switches

Finally, to complete the discussion, it helps to think about an example with multiple switches, just to emphasize how MAC learning, forwarding, and flooding happen independently on each LAN switch.

Consider the topology in Figure 5-10, and pay close attention to the port numbers. The ports were purposefully chosen so that neither switch used any of the same ports for this example. That is, switch SW2 does have a port F0/1 and F0/2, but I did not plug any devices into those ports when making this example. Also note that all ports are in VLAN 1, and as with the other examples in this chapter, all default configuration is used other than the hostname on the switches.

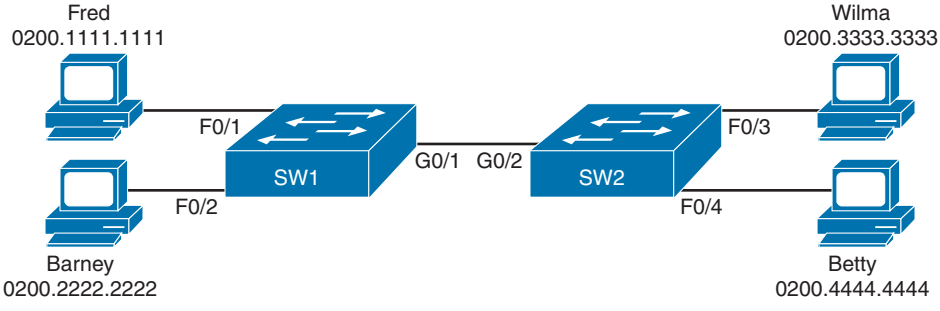



**Figure 5-10** *Two-Switch Topology Example*

Think about a case in which both switches learn all four MAC addresses. For instance, that would happen if the hosts on the left communicate with the hosts on the right. SW1's MAC address table would list SW1's own port numbers (F0/1, F0/2, and G0/1) because SW1 uses that information to decide where SW1 should forward frames. Similarly, SW2's MAC table

lists SW2's port numbers (F0/3, F0/4, G0/2 in this example). Example 5-8 shows the MAC address tables on both switches for that scenario.

**Example 5-8** *The MAC Address Table on Two Switches*

```
SW1# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
   1    0200.1111.1111    DYNAMIC     Fa0/1
   1    0200.2222.2222    DYNAMIC     Fa0/2
   1    0200.3333.3333    DYNAMIC     Gi0/1
   1    0200.4444.4444    DYNAMIC     Gi0/1
Total Mac Addresses for this criterion: 4
```

```
! The next output is from switch SW2
SW2# show mac address-table dynamic
   1    0200.1111.1111    DYNAMIC     Gi0/2
   1    0200.2222.2222    DYNAMIC     Gi0/2
   1    0200.3333.3333    DYNAMIC     Fa0/3
   1    0200.4444.4444    DYNAMIC     Fa0/4
Total Mac Addresses for this criterion: 4
```

## Chapter Review

Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Table 5-2 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 5-2** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Do labs | | Book, Sim Lite, blog |
| Review command tables | | Book |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 5-3** Key Topics for Chapter 5

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Three main functions of a LAN switch | 115 |
| Figure 5-3 | Process to forward a known unicast frame | 116 |
| Figure 5-5 | Process to forward a known unicast, second switch | 118 |
| Figure 5-6 | Process to learn MAC addresses | 119 |
| List | Summary of switch forwarding logic | 121 |
| Example 5-1 | The **show mac address-table dynamic** command | 122 |

## Do Labs

The Sim Lite software is a version of Pearson's full simulator learning product with a subset of the labs, included free with this book. The subset of labs mostly relates to this part of the book, so take the time to try some of the labs.

As always, also check the author's blog site pages for configuration exercises (Config Labs) at http://www.certskills.com.

5

## Key Terms You Should Know

broadcast frame, flood, forward, known unicast frame, MAC address table, Spanning Tree Protocol (STP), unknown unicast frame

## Command References

Table 5-4 lists the verification commands used in this chapter. As an easy review exercise, cover the left column, read the right, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 5-4** Chapter 5 EXEC Command Reference

| Command | Mode/Purpose/Description |
|---|---|
| **show mac address-table** | Shows all MAC table entries of all types |
| **show mac address-table dynamic** | Shows all dynamically learned MAC table entries |
| **show mac address-table dynamic vlan** *vlan-id* | Shows all dynamically learned MAC table entries in that VLAN |
| **show mac address-table dynamic address** *mac-address* | Shows the dynamically learned MAC table entries with that MAC address |
| **show mac address-table dynamic interface** *interface-id* | Shows all dynamically learned MAC table entries associated with that interface |
| **show mac address-table count** | Shows the number of entries in the MAC table and the total number of remaining empty slots in the MAC table |

| Command | Mode/Purpose/Description |
|---|---|
| **show mac address-table aging-time** | Shows the global and per-VLAN aging timeout for inactive MAC table entries |
| **show interfaces** *id* **counters** | Lists packet counters for the listed interface ID |
| **show interfaces status** | Lists one line per interface on the switch, with basic status and operating/ information for each |
| **clear mac address-table dynamic [vlan** *vlan-number*] **[interface** *interface-id*] **[address** *mac-address*] | Clears (removes) dynamic MAC table entries: either all (with no parameters), or a subset based on VLAN ID, interface ID, or a specific MAC address |

Note that this chapter also includes reference to one configuration command, so it does not call for the use of a separate table. For review, the command is

**mac address-table aging-time** *time-in-seconds* [**vlan** *vlan-number*]

*This page intentionally left blank*



CHAPTER 6

# Configuring Basic Switch Management

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.6 Configure and verify IPv4 addressing and subnetting**
- **2.0 Network Access**
  - **2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, RADIUS/TACACS+)**
- **4.0 IP Services**
  - **4.6 Configure and verify DHCP client and relay**
  - **4.8 Configure network devices for remote access using SSH**
- **5.0 Security Fundamentals**
  - **5.3 Configure device access control using local passwords**

The tasks of a switch fall into a small set of categories called planes. The data plane includes the process of forwarding frames received by the switch. The control plane refers to the processes that control and change the switch's data plane. The control plane includes configuration to enable or disable an interface, to control the speed used by each interface, and the dynamic processes of Spanning Tree to block some ports to prevent loops, and so on. The third plane, the management plane, refers to device management features. Those include Telnet and SSH, used to connect to the CLI, and other management features.

This chapter discusses the most basic management plane features in a Cisco switch. The first section of the chapter discusses configuring different kinds of login security for console, Telnet, and SSH users. The second section shows how to enable remote switch management by configuring switch IPv4 settings. The last section then explains a few practical matters that can make your life in the lab a little easier.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 6-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Securing the Switch CLI | 1–3 |
| Enabling IPv4 for Remote Access | 4–5 |
| Miscellaneous Settings Useful in the Lab | 6 |

1. Imagine that you have configured the **enable secret** command, followed by the **enable password** command, from the console. You log out of the switch and log back in at the console. Which command defines the password that you had to enter to access privileged mode?
   a. **enable password**
   b. **enable secret**
   c. Neither
   d. The **password** command, if it is configured
2. An engineer wants to set up simple password protection with no usernames for some switches in a lab, for the purpose of keeping curious coworkers from logging in to the lab switches from their desktop PCs. Which of the following commands would be a useful part of that configuration?
   a. A **login** vty mode subcommand
   b. A **password** *password* console subcommand
   c. A **login local** vty subcommand
   d. A **transport input ssh** vty subcommand
3. An engineer had formerly configured a Cisco 2960 switch to allow Telnet access so that the switch expected a password of **mypassword** from the Telnet user. The engineer then changed the configuration to support Secure Shell. Which of the following commands could have been part of the new configuration? (Choose two answers.)
   a. A **username** *name* **secret** *password* vty mode subcommand
   b. A **username** *name* **secret** *password* global configuration command
   c. A **login local** vty mode subcommand
   d. A **transport input ssh** global configuration command
4. An engineer's desktop PC connects to a switch at the main site. A router at the main site connects to each branch office through a serial link, with one small router and switch at each branch. Which of the following commands must be configured on the branch office switches, in the listed configuration mode, to allow the engineer to telnet to the branch office switches and supply only a password to log in? (Choose three answers.)
   a. The **ip address** command in interface configuration mode
   b. The **ip address** command in global configuration mode
   c. The **ip default-gateway** command in VLAN configuration mode
   d. The **ip default-gateway** command in global configuration mode
   e. The **password** command in console line configuration mode
   f. The **password** command in vty line configuration mode

5. A Layer 2 switch configuration places all its physical ports into VLAN 2. An attached router uses address/mask 172.16.2.254/24. The IP address plan calls for the switch to use address/mask 172.16.2.250/24 and to use the router as its default gateway. The switch needs to support SSH connections into the switch from any subnet in the network. Which of the following commands are part of the required configuration in this case? (Choose two answers.)
   a. The **ip address 172.16.2.250 255.255.255.0** command in interface vlan 1 configuration mode.
   b. The **ip address 172.16.2.250 255.255.255.0** command in interface vlan 2 configuration mode.
   c. The **ip default-gateway 172.16.2.254** command in global configuration mode.
   d. The switch cannot support SSH because all its ports connect to VLAN 2, and the IP address must be configured on interface VLAN 1.
6. Which of the following line subcommands tells a switch to wait until a show command's output has completed before displaying log messages on the screen?
   a. **logging synchronous**
   b. **no ip domain-lookup**
   c. **exec-timeout 0 0**
   d. **history size 15**

# Foundation Topics

## Securing the Switch CLI

By default, a user can connect to the console and reach **enable mode** with no security checks and no passwords required. In contrast, the default settings disallow all **Telnet** and **Secure Shell (SSH)** users from even seeing a login prompt. Those defaults make sense, given that if you can get to the console port of the switch, you already have control over the switch physically. But clearly, protecting the console makes sense, as does opening up SSH and Telnet access to appropriate users.

This first topic in the chapter examines how to configure login security for a Cisco Catalyst switch. Securing the CLI includes protecting access to enable mode, because from enable mode, an attacker could reload the switch or change the configuration. Protecting user mode is also important, because attackers can see the status of the switch, learn about the network, and find new ways to attack the network.

In particular, this section covers the following login security topics:

- Securing user mode and privileged mode with simple passwords
- Securing user mode access with local usernames
- Securing user mode access with external authentication servers
- Securing remote access with Secure Shell (SSH)

Note that all remote management protocols, like Telnet and SSH, require IP configuration on the switch, which is not discussed until the second major section of this chapter, "Enabling IPv4 for Remote Access."

## Securing User Mode and Privileged Mode with Simple Passwords

The defaults work great for a brand new switch, but in production, you will want to secure access through the console as well as enable remote login via Telnet and/or SSH so you can sit at your desk and log in to all the switches in the LAN. Keep in mind, however, that you should not open the switch for just anyone to log in and change the configuration, so some type of secure login should be used.

Most people use a simple shared password for access to lab gear. This method uses a password only—with no username—with one password for console users and a different password for Telnet users. Console users must supply the *console password*, as configured in console line configuration mode. Telnet users must supply the *Telnet password*, also called the vty password, so called because the configuration sits in vty line configuration mode. Figure 6-1 summarizes these options for using shared passwords from the perspective of the user logging in to the switch.

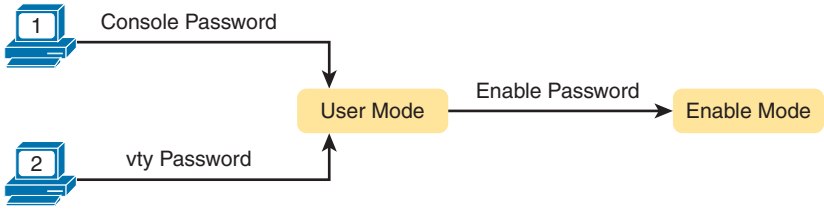



**Figure 6-1** *Simple Password Security Concepts*

**NOTE** This section refers to several passwords as *shared* passwords. Users share these passwords in that all users must know and use that same password. In other words, each user does not have a unique username/password to use, but rather, all the appropriate staff knows and uses the same password.

In addition, Cisco switches protect enable mode (also called privileged mode) with yet another shared password called the *enable password*. From the perspective of the network engineer connecting to the CLI of the switch, once in user mode, the user types the **enable** EXEC command. This command prompts the user for this enable password; if the user types the correct password, IOS moves the user to enable mode.

Example 6-1 shows an example of the user experience of logging in to a switch from the console when the shared console password and the shared enable password have both been set. Note that before this example began, the user started the terminal emulator, physically connected a laptop to the console cable, and then pressed the Enter key to make the switch respond as shown at the top of the example.

**Example 6-1** *Console Login and Movement to Enable Mode*

```
(User now presses enter to start the process. This line of text does not appear.)

User Access Verification

Password: faith
Switch> enable
Password: love
Switch#
```

Note that the example shows the password text as if typed (faith and love), along with the **enable** command that moves the user from user mode to enable mode. In reality, the switch hides the passwords when typed, to prevent someone from reading over your shoulder to see the passwords.

To configure the shared passwords for the console, Telnet, and for enable mode, you need to configure several commands. However, the parameters of the commands can be pretty intuitive. Figure 6-2 shows the configuration of all three of these passwords.

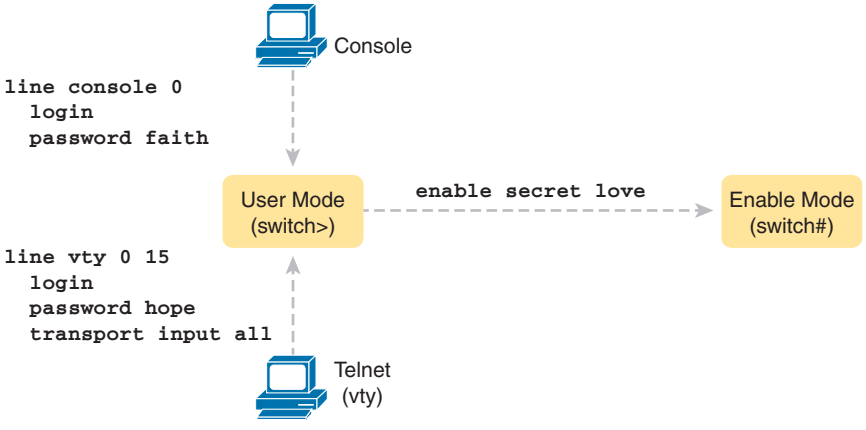



**Figure 6-2** *Simple Password Security Configuration*

The configuration for these three passwords does not require a lot of work. First, the console and vty password configuration sets the password based on the context: console mode for the console (**line con 0**), and vty line configuration mode for the Telnet password (**line vty 0 15**). Then inside console mode and vty mode, respectively, the two commands in each mode are as follows:

**password** *password-value*: Defines the actual password used on the console or vty

**login:** Tells IOS to enable the use of a simple shared password (with no username) on this line (console or vty), so that the switch asks the user for a password

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** A **3** B, C **4** A, D, F **5** B, C **6** A

The configured enable password, shown on the right side of the figure, applies to all users, no matter whether they connect to user mode via the console, Telnet, or otherwise. The command to configure the enable password is a global configuration command: **enable secret** *password-value*.

**NOTE** Older IOS versions used the command **enable password** *password-value* to set the enable password, and that command still exists in IOS. However, the **enable secret** command is much more secure. In real networks, use **enable secret**. Chapter 10, "Securing Network Devices," in the *CCNA 200-301 Official Cert Guide*, *Volume 2*, Second Edition, explains more about the security levels of various password mechanisms, including a comparison of the **enable secret** and **enable password** commands.

To help you follow the process, and for easier study later, use the configuration checklist before the example. The configuration checklist collects the required and optional steps to configure a feature as described in this book. The configuration checklist for shared passwords for the console, Telnet, and enable passwords is

Config Checklist

**Step 1.** Configure the enable password with the **enable secret** *password-value* command.

**Step 2.** Configure the console password:

- **a.** Use the **line con 0** command to enter console configuration mode.
- **b.** Use the **password** *password-value* subcommand to set the value of the console password.
- **c.** Use the **login** subcommand to enable console password security using a simple password.

**Step 3.** Configure the Telnet (vty) password:

- **a.** Use the **line vty 0 15** command to enter vty configuration mode for all 16 vty lines (numbered 0 through 15).
- **b.** Use the **password** *password-value* subcommand to set the value of the vty password.
- **c.** Use the **login** subcommand to enable console password security using a simple password.
- **d.** Use the **transport input all** subcommand (or similar) to enable Telnet as an input protocol for the vty lines.

**NOTE** The section "Securing Remote Access with Secure Shell," later in this chapter, provides more detail about the **transport input** subcommand.

Example 6-2 shows the configuration process as noted in the configuration checklist, along with setting the enable secret password. Note that the lines which begin with a ! are comment lines; they are there to guide you through the configuration.

Example 6-3 shows the resulting configuration in the switch per the **show running-config** command. The gray lines highlight the new configuration. Note that many unrelated lines of output have been deleted from the output to keep focused on the password configuration.

Key Topic

**Example 6-2** *Configuring Basic Passwords*

```
! Enter global configuration mode and set the enable password.
!
Switch# configure terminal
Switch(config)# enable secret love
!
! At Step 2 in the checklist, enter console configuration mode, set the
! password value to "faith" and enable simple passwords for the console.
! The exit command moves the user back to global config mode.
!
Switch#(config)# line console 0
Switch#(config-line)# password faith
Switch#(config-line)# login
Switch#(config-line)# exit
!
! The next few lines do basically the same configuration, except it is
! for the vty lines. Telnet users will use "hope" to login.
!
Switch#(config)# line vty 0 15
Switch#(config-line)# password hope
Switch#(config-line)# login
Switch#(config-line)# transport input all
Switch#(config-line)# end
Switch#
```

**Example 6-3** *Resulting Running-Config File (Subset) per Example 6-2 Configuration*

```
Switch# show running-config
!
Building configuration...

Current configuration: 1333 bytes
!
version 12.2
!
enable secret 5 $1$OwtI$A58c2XgqWyDNeDnv51mNR.
!
interface FastEthernet0/1
!
interface FastEthernet0/2
!
! Several lines have been omitted here - in particular, lines for
! FastEthernet interfaces 0/3 through 0/23.
!
interface FastEthernet0/24
```

```
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/2
!
line con 0
 password faith
login
!
line vty 0 4
 password hope
 login
 transport input all
!
line vty 5 15
 password hope
 login
 transport input all
```

**NOTE** For historical reasons, the output of the **show running-config** command, in the last six lines of Example 6-3, separates the first five vty lines (0 through 4) from the rest (5 through 15).



## Securing User Mode Access with Local Usernames and Passwords

Cisco switches support two other login security methods that both use per-user username/password pairs instead of a shared password with no username. One method, referred to as **local usernames** and passwords, configures the username/password pairs locally—that is, in the switch's configuration. Switches support this local username/password option for the console, for Telnet, and even for SSH, but do not replace the enable password used to reach enable mode.

The configuration to migrate from using the simple shared passwords to instead using local usernames/passwords requires only some small configuration changes, as shown in Figure 6-3.

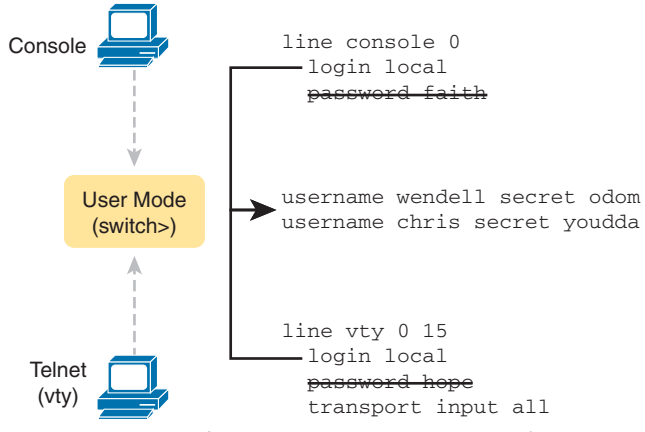



**Figure 6-3** *Configuring Switches to Use Local Username Login Authentication*

Working through the configuration in the figure, first, the switch of course needs to know the list of username/password pairs. To create these, repeatedly use the **username** *name* **secret** *password* global configuration command. Then, to enable this different type of console or Telnet security, simply enable this login security method with the **login local** line. Basically, this command means "use the local list of usernames for login." You can also use the **no password** command (without even typing in the password) to clean up any remaining password subcommands from console or vty mode because these commands are not needed when using local usernames and passwords.

The following checklist details the commands to configure local username login, mainly as a method for easier study and review:

Config Checklist

**Step 1.** Use the **username** *name* **secret** *password* global configuration command to add one or more username/password pairs on the local switch.

**Step 2.** Configure the console to use locally configured username/password pairs:

- **a.** Use the **line con 0** command to enter console configuration mode.
- **b.** Use the **login local** subcommand to enable the console to prompt for both username and password, checked versus the list of local usernames/passwords.
- **c.** (Optional) Use the **no password** subcommand to remove any existing simple shared passwords, just for good housekeeping of the configuration file.

**Step 3.** Configure Telnet (vty) to use locally configured username/password pairs.

- **a.** Use the **line vty 0 15** command to enter vty configuration mode for all 16 vty lines (numbered 0 through 15).
- **b.** Use the **login local** subcommand to enable the switch to prompt for both username and password for all inbound Telnet users, checked versus the list of local usernames/passwords.
- **c.** (Optional) Use the **no password** subcommand to remove any existing simple shared passwords, just for good housekeeping of the configuration file.
- **d.** Use the **transport input all** subcommand (or similar) to enable Telnet as an input protocol for the vty lines.

When a Telnet user connects to the switch configured as shown in Figure 6-3, the user will be prompted first for a username and then for a password, as shown in Example 6-4. The username/password pair must be from the list of local usernames; otherwise, the login is rejected.

**Example 6-4** *Telnet Login Process After Applying Configuration in Figure 6-3*

```
SW2# telnet 10.9.9.19
Trying 10.9.9.19 ... Open


User Access Verification
```

```
Username: wendell
Password:
SW1> enable
Password:
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)#^Z
SW1#
*Mar 1 02:00:56.229: %SYS-5-CONFIG_I: Configured from console by wendell on vty0
(10.9.9.19)
```

**NOTE** Example 6-4 does not show the password value as having been typed because Cisco switches do not display the typed password for security reasons.

## Securing User Mode Access with External Authentication Servers

The end of Example 6-4 points out one of the many security improvements when requiring each user to log in with their own username. The end of the example shows the user entering configuration mode (**configure terminal**) and then immediately leaving (**end**). Note that when a user exits configuration mode, the switch generates a **log message**. If the user logged in with a username, the log message identifies that username; note the "wendell" in the log message.

However, using a username/password configured directly on the switch causes some administrative headaches. For instance, every switch and router needs the configuration for all users who might need to log in to the devices. Then, when any changes need to happen, like an occasional change to the passwords for good security practices, the configuration of all devices must be changed.

A better option would be to use tools like those used for many other IT login functions. Those tools allow for a central place to securely store all username/password pairs, with tools to make users change their passwords regularly, tools to revoke users when they leave their current jobs, and so on.

Cisco switches allow exactly that option using an external server called an authentication, authorization, and accounting (**AAA**) server. These servers hold the usernames/passwords. Typically, these servers allow users to do self-service and forced maintenance to their passwords. Many production networks use AAA servers for their switches and routers today.

The underlying login process requires some additional work on the part of the switch for each user login, but once set up, the username/password administration is much less. When an **AAA server** is used for authentication, the switch (or router) simply sends a message to the AAA server asking whether the username and password are allowed, and the AAA server replies. Figure 6-4 shows an example, with the user first supplying a username/password, the switch asking the AAA server, and the server replying to the switch stating that the username/password is valid.

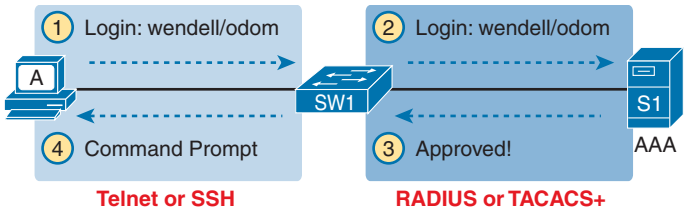



**Figure 6-4** *Basic Authentication Process with an External AAA Server*

While the figure shows the general idea, note that the information flows with a couple of different protocols. On the left, the connection between the user and the switch or router uses Telnet or SSH. On the right, the switch and AAA server typically use either the RADIUS or TACACS+ protocol, both of which encrypt the passwords as they traverse the network.

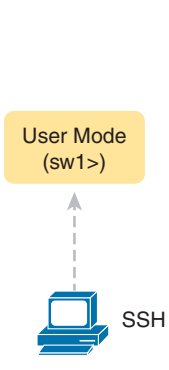
## Securing Remote Access with Secure Shell

So far, this chapter has focused on the console and on Telnet, mostly ignoring SSH. Telnet has one serious disadvantage: all data in the Telnet session flows as clear text, including the password exchanges. So, anyone who can capture the messages between the user and the switch (in what is called a man-in-the-middle attack) can see the passwords. SSH encrypts all data transmitted between the SSH client and server, protecting the data and passwords.

SSH can use the same local login authentication method as Telnet, with the locally configured username and password. (SSH cannot rely on authentication methods that do not include a username, like shared passwords.) So, the configuration to support local usernames for Telnet, as shown previously in Figure 6-3, also enables local username authentication for incoming SSH connections.

Figure 6-5 shows one example configuration of what is required to support SSH. The figure repeats the local username configuration as shown earlier in Figure 6-3, as used for Telnet. Figure 6-5 shows three additional commands required to complete the configuration of SSH on the switch.

Key Topic




SSH-Specific Configuration

```
hostname sw1
ip domain-name example.com
! Next Command Uses FQDN "sw1.example.com"
crypto key generate rsa
```

Local Username Configuration (Like Telnet)

```
username wendell secret odom
username chris secret youdda
!
line vty 0 15
  login local
  transport input all
```

**Figure 6-5** *Adding SSH Configuration to Local Username Configuration*

IOS uses the three SSH-specific configuration commands in the figure to create the SSH encryption keys. The SSH server uses the fully qualified domain name (FQDN) of the switch as input to create that key. The switch creates the FQDN from the hostname and domain name of the switch. Figure 6-5 begins by setting both values (just in case they are not already configured). Then the third command, the **crypto key generate rsa** command, generates the SSH encryption keys.

Seeing the configuration happen in configuration mode, step by step, can be particularly helpful with SSH configuration. Note in particular that in this example, the **crypto key** command prompts the user for the key modulus; you could also add the parameters **modulus** *modulus-value* to the end of the **crypto key** command to add this setting on the command. Example 6-5 shows the commands in Figure 6-5 being configured, with the encryption key as the final step.

**Example 6-5** *SSH Configuration Process to Match Figure 6-5*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
!
! Step 1 next. The hostname is already set, but it is repeated just
! to be obvious about the steps.
!
SW1(config)# hostname SW1
SW1(config)# ip domain name example.com
SW1(config)# crypto key generate rsa
The name for the keys will be: SW1.example.com

Choose the size of the key modulus in the range of 512 to 2048 for your
  General Purpose Keys. Choosing a key modulus greater than 512 may take
  a few minutes.

How many bits in the modulus [1024]: 1024
% Generating 1024 bit RSA keys, keys will be non-exportable...
[OK] (elapsed time was 4 seconds)
SW1(config)#
!
! Optionally, set the SSH version to version 2 (only) - preferred
!
SW1(config)# ip ssh version 2
!
! Next, configure the vty lines for local username support, just like
! with Telnet
!
SW1(config)# line vty 0 15
SW1(config-line)# login local
SW1(config-line)# transport input all
SW1(config-line)# exit
!
```

```
! Define the local usernames, just like with Telnet
!
SW1(config)# username wendell secret odom
SW1(config)# username chris secret youdaman
SW1(config)# ^Z
SW1#
```

**NOTE** Older IOS versions used the syntax **ip domain-name** *domain-name* rather than the newer **ip domain name** *domain-name* (with a space instead of a dash.)

Both the Telnet and SSH examples throughout this chapter so far list the **transport input all** subcommand in vty configuration mode. The **transport input** command identifies the protocols allowed in the vty ports, with the **all** keyword including SSH and Telnet. Using **transport input all** lets you support everything when getting started learning, but for production devices you might want to instead choose some different options, like not supporting Telnet at all due to its poor security. Some common options include

**transport input all** or **transport input telnet ssh:** Support both Telnet and SSH

**transport input none:** Support neither

**transport input telnet:** Support only Telnet

**transport input ssh:** Support only SSH

Over the years, the default settings for this command have varied a bit based on device type, OS, and OS version. As a result, for production devices, it makes sense to pick the setting you want and configure it, rather than relying on your memory of a default setting for a particular device and software version. For instance, many companies prefer to disable any possibility of Telnet access, but allow SSH, using **transport input ssh**.

For the exams, be ready to look for the **transport input** setting to ensure it supports SSH, or Telnet, or both, depending on the scenario. As a strategy for the exam, look for the command to confirm its settings when looking at any Telnet or SSH configuration. Also, be aware of the traditional defaults: Many older switches defaulted to **transport input all**, while older routers defaulted to **transport input none**, with more recent Cisco switches and routers now defaulting to the more-secure **transport input ssh**.

To complete this section about SSH, the following configuration checklist details the steps for one method to configure a Cisco switch to support SSH using local usernames. (SSH support in IOS can be configured in several ways; this checklist shows one simple way to configure it.) The process shown here ends with a comment to configure local username support on vty lines, as was discussed earlier in the section titled "Securing User Mode Access with Local Usernames and Passwords."

Config Checklist

**Step 1.** Configure the switch to generate a matched public and private key pair to use for encryption:

**a.** If not already configured, use the **hostname** *name* in global configuration mode to configure a hostname for this switch.

b. If not already configured, use the **ip domain name** *name* in global configuration mode to configure a domain name for the switch, completing the switch's FQDN.

c. Use the **crypto key generate rsa** command in global configuration mode (or the **crypto key generate rsa modulus** *modulus-value* command to avoid being prompted for the key modulus) to generate the keys. (Use at least a 768-bit key to support SSH version 2.)

**Step 2.** (Optional) Use the **ip ssh version 2** command in global configuration mode to override the default of supporting both versions 1 and 2, so that only SSHv2 connections are allowed.

**Step 3.** (Optional) If not already configured with the setting you want, configure the vty lines to accept SSH and whether to also allow Telnet:

a. Use the **transport input ssh** command in vty line configuration mode to allow SSH only.

b. Use the **transport input all** command or **transport input telnet ssh** command in vty line configuration mode to allow both SSH and Telnet.

**Step 4.** Use various commands in vty line configuration mode to configure local username login authentication as discussed earlier in this chapter.

Two key commands give some information about the status of SSH on the switch. First, the **show ip ssh** command lists status information about the SSH server itself. The **show ssh** command then lists information about each SSH client currently connected to the switch. Example 6-6 shows samples of each, with user wendell currently connected to the switch.

**Example 6-6** *Displaying SSH Status*

```
SW1# show ip ssh
SSH Enabled - version 2.0
Authentication timeout: 120 secs; Authentication retries: 3

SW1# show ssh
Connection Version Mode  Encryption    Hmac          State              Username
0          2.0     IN    aes126-cbc    hmac-sha1     Session started    wendell
0          2.0     OUT   aes126-cbc    hmac-sha1     Session started    wendell
%No SSHv1 server connections running.
```

## Enabling and Securing the WebUI

The section "Accessing the CLI with the WebUI" in Chapter 4 shows examples of the WebUI user interface but does not discuss any configurations. Next, you will discover how to configure the most common settings to support this feature and to secure it using a **username** command—but with a new twist.

The HTTP server, the WebUI, has a long history as an integrated IOS feature in switches and routers. Over time, Cisco changed the user interface and some configuration commands. So,

rather than focus on the trivial matters, focus on these steps that would be a common-sense configuration in a switch (or router) today:

- Use the **no ip http server** global command to disable the HTTP server (port 80) (traditionally enabled by default).
- Use the **ip http secure-server** global command to enable the HTTPS server (port 443, uses TLS) (traditionally enabled by default).
- Use the **ip http authentication local** global command to define the authentication method to use locally defined usernames (traditionally defaults to use the enable password).
- Use the **username** *name* **priority 15 password** *pass-value* global command to define one or more usernames with privilege level 15.

Talking through the items in the list, IOS has long used defaults that enable both the HTTP and HTTPS servers. With both enabled, users can connect from browsers by typing URLs that begin http:// (therefore not encrypting the traffic) or https:// (therefore encrypting the traffic). For better security, Cisco recommends disabling the HTTP server.

The HTTP server has long allowed three options to log in to a device from a web browser:

- Using the enable password
- Using a local username/password
- Using the AAA settings on the device

More recent IOS versions move away from the **enable** option. Also, using both a username and a password makes more sense from a security perspective.

To access all the features of the HTTP server (the WebUI), when using local usernames for authentication, you must configure a privilege level of 15 for the user. IOS internally defines user privilege levels, by default creating two levels, 0 and 15. IOS assigns user mode to level 0 and privileged mode (enable mode) to level 15. When CLI users move from user mode to enable mode, they improve their priority level from 0 to 15.

For the WebUI, if you log in using a username with the **privilege 15** option, you receive access to all WebUI features, including the ability to use the CLI configuration mode, install new software, erase the configuration, and reload the router. So, the **username** *name* **priority 15 password** *pass-value* global command creates a way to enter privileged mode immediately. (If you omit the **priority 15** option and log in to the WebUI with that username you can log in, but you cannot do advanced features, including using the CLI to configure or verify a feature.)

## Enabling IPv4 for Remote Access

To allow Telnet, SSH, or WebUI access to the switch, and to allow other IP-based management protocols (for example, Simple Network Management Protocol, or SNMP) to function as intended, the switch needs an IP address, as well as a few other related settings. The IP address has nothing to do with how switches forward Ethernet frames; it simply exists to support overhead management traffic (control plane traffic).

This next topic begins by explaining the IPv4 settings needed on a switch, followed by the configuration. Note that although switches can be configured with IPv6 addresses with commands similar to those shown in this chapter, this chapter focuses solely on IPv4. All references to IP in this chapter imply IPv4.

## Host and Switch IP Settings

A switch needs the same kind of IP settings as a PC with a single Ethernet interface. For perspective, a PC has a CPU, with the operating system running on the CPU. It has an Ethernet network interface card (NIC). The OS configuration includes an IP address associated with the NIC, either configured or learned dynamically with Dynamic Host Configuration Protocol (DHCP).

A switch uses the same ideas, except that the switch needs to use a virtual NIC inside the switch. Like a PC, a switch has a real CPU, running an OS (called IOS). The switch obviously has lots of Ethernet ports, but instead of assigning its management IP address to any of those ports, the switch then uses a NIC-like concept called a switch virtual interface (SVI), or more commonly, a **VLAN interface**, that acts like the switch's own NIC. Then the settings on the switch look something like a host, with the switch configuration assigning IP settings, like an IP address, to this VLAN interface, as shown in Figure 6-6.

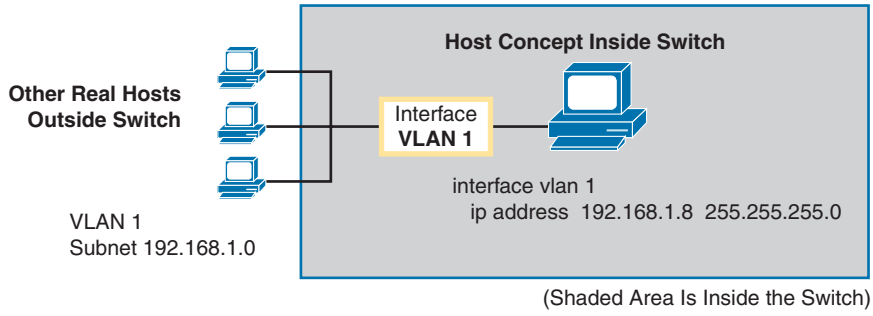



**Figure 6-6** *Switch Virtual Interface (SVI) Concept Inside a Switch*

By using interface VLAN 1 for the IP configuration, the switch can then send and receive frames on any of the ports in VLAN 1. In a Cisco switch, by default, all ports are assigned to VLAN 1.

In most networks, switches configure many VLANs, so the network engineer has a choice of where to configure the IP address. That is, the management IP address does not have to be configured on the VLAN 1 interface (as configured with the **interface vlan 1** command seen in Figure 6-6).

A Layer 2 Cisco LAN switch needs only one IP address for management purposes. However, you can choose to use any VLAN to which the switch connects. The configuration then includes a VLAN interface for that VLAN number, with an appropriate IP address.

For example, Figure 6-7 shows a Layer 2 switch with some physical ports in two different VLANs (VLANs 1 and 2). The figure also shows the subnets used on those VLANs. The network engineer could choose to use either

- Interface VLAN 1, with an IP address in subnet 192.168.1.0
- Interface VLAN 2, with an IP address in subnet 192.168.2.0

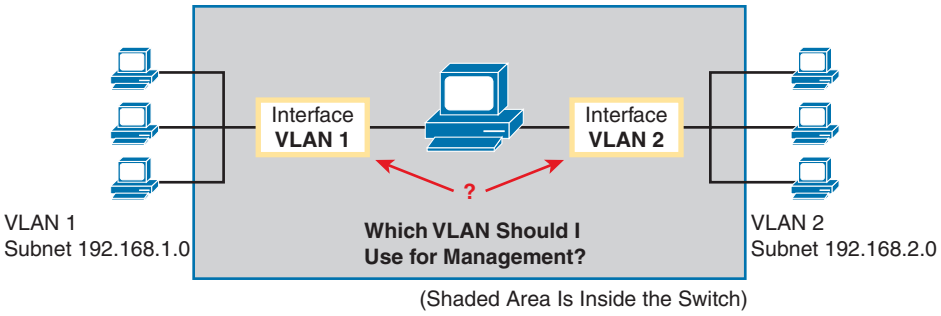



**Figure 6-7** *Choosing One VLAN on Which to Configure a Switch IP Address*

Note that you should not try to use a VLAN interface for which there are no physical ports assigned to the same VLAN. If you do, the VLAN interface will not reach an up/up state, and the switch will not have the physical ability to communicate outside the switch.

**NOTE** Some Cisco switches can be configured to act as either a Layer 2 switch or a Layer 3 switch. When acting as a Layer 2 switch, a switch forwards Ethernet frames as discussed in depth in Chapter 5, "Analyzing Ethernet LAN Switching." Alternatively, a switch can also act as a *multilayer switch* or *Layer 3 switch*, which means the switch can do both Layer 2 switching and Layer 3 IP routing of IP packets, using the Layer 3 logic normally used by routers. This chapter assumes all switches are Layer 2 switches. Chapter 18, "IP Routing in the LAN," discusses Layer 3 switching in depth along with using multiple VLAN interfaces at the same time.

Configuring the IP address (and mask) on one VLAN interface allows the switch to send and receive IP packets with other hosts in a subnet that exists on that VLAN; however, the switch cannot communicate outside the local subnet without another configuration setting called the **default gateway**. The reason a switch needs a default gateway setting is the same reason that hosts need the same setting—because of how hosts think when sending IP packets. Specifically:

- To send IP packets to hosts in the same subnet, send them directly
- To send IP packets to hosts in a different subnet, send them to the local router; that is, the default gateway

Figure 6-8 shows the ideas. In this case, the switch (on the right) will use IP address 192.168.1.200 as configured on interface VLAN 1. However, to communicate with host A, on the far left of the figure, the switch must use Router R1 (the default gateway) to forward IP packets to host A. To make that work, the switch needs to configure a default gateway setting, pointing to Router R1's IP address (192.168.1.1 in this case). Note that the switch and router both use the same mask, 255.255.255.0, which puts the addresses in the same subnet.

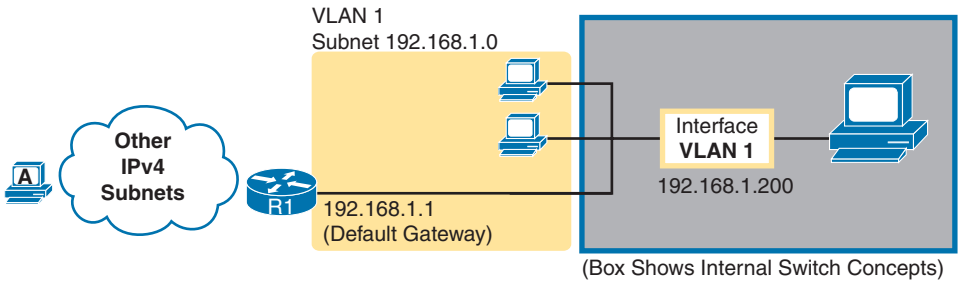



**Figure 6-8** *The Need for a Default Gateway*

## Configuring IPv4 on a Switch

A switch configures its IPv4 address and mask on this special NIC-like *VLAN interface*. The following steps list the commands used to configure IPv4 on a switch, assuming that the IP address is configured to be in VLAN 1, with Example 6-7 that follows showing an example configuration.

Config Checklist

**Step 1.** Use the **interface vlan 1** command in global configuration mode to enter interface VLAN 1 configuration mode.

**Step 2.** Use the **ip address** *ip-address mask* command in interface configuration mode to assign an IP address and mask.

**Step 3.** Use the **no shutdown** command in interface configuration mode to enable the VLAN 1 interface if it is not already enabled.

**Step 4.** Add the **ip default-gateway** *ip-address* command in global configuration mode to configure the default gateway.

**Step 5.** (Optional) Add the **ip name-server** *ip-address1 ip-address2 …* command in global configuration mode to configure the switch to use the Domain Name System (**DNS**) to resolve names into their matching IP address.



**Example 6-7** *Switch Static IP Address Configuration*

```
Emma# configure terminal
Emma(config)# interface vlan 1
Emma(config-if)# ip address 192.168.1.200 255.255.255.0
Emma(config-if)# no shutdown
00:25:07: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
00:25:08: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed
 state to up
Emma(config-if)# exit
Emma(config)# ip default-gateway 192.168.1.1
```

On a side note, this example shows a particularly important and common command: the [**no**] **shutdown** command. To administratively enable an interface on a switch, use the **no shutdown** interface subcommand; to disable an interface, use the **shutdown** interface subcommand.

This command can be used on the physical Ethernet interfaces that the switch uses to switch Ethernet messages in addition to the VLAN interface shown here in this example.

Also, pause long enough to look at the messages that appear just below the **no shutdown** command in Example 6-7. Those messages are syslog messages generated by the switch stating that the switch did indeed enable the interface. Switches (and routers) generate syslog messages in response to a variety of events, and by default, those messages appear at the console. Chapter 13, "Device Management Protocols," in the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, discusses syslog messages in more detail.

## Configuring a Switch to Learn Its IP Address with DHCP

The switch can also use Dynamic Host Configuration Protocol (DHCP) to dynamically learn its IPv4 settings. (Typically, engineers do not do so, instead statically configuring switch IP addresses, but this section covers the concept to be complete as compared to the exam topics.) Basically, all you have to do is tell the switch to use DHCP on the interface and enable the interface. Assuming that DHCP works in this network, the switch will learn all its settings. The following list details the steps, again assuming the use of interface VLAN 1, with Example 6-8 that follows showing an example:

Config Checklist

**Step 1.** Enter VLAN 1 configuration mode using the **interface vlan 1** global configuration command, and enable the interface using the **no shutdown** command as necessary.

**Step 2.** Assign an IP address and mask using the **ip address dhcp** interface subcommand.

**Example 6-8** *Switch Dynamic IP Address Configuration with DHCP*

```
Emma# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Emma(config)# interface vlan 1
Emma(config-if)# ip address dhcp
Emma(config-if)# no shutdown
Emma(config-if)# ^Z
Emma#
00:38:20: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
00:38:21: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up
```

## Verifying IPv4 on a Switch

The switch IPv4 configuration can be checked in several places. First, you can always look at the current configuration using the **show running-config** command. Second, you can look at the IP address and mask information using the **show interfaces vlan** *x* command, which shows detailed status information about the VLAN interface in VLAN *x*. Finally, if using DHCP, use the **show dhcp lease** command to see the (temporarily) leased IP address and other parameters. (Note that the switch does not store the DHCP-learned IP configuration in the running-config file.) Example 6-9 shows sample output from these commands to match the configuration in Example 6-8.

**Example 6-9** *Verifying DHCP-Learned Information on a Switch*

```
Emma# show dhcp lease
Temp IP addr: 192.168.1.101   for peer on Interface: Vlan1
Temp sub net mask: 255.255.255.0
   DHCP Lease server: 192.168.1.1, state: 3 Bound
   DHCP transaction id: 1966
   Lease: 86400 secs,  Renewal: 43200 secs,  Rebind: 75600 secs
Temp default-gateway addr: 192.168.1.1
   Next timer fires after: 11:59:45
   Retry count: 0   Client-ID: cisco-0019.e86a.6fc0-Vl1
   Hostname: Emma
Emma# show interfaces vlan 1
Vlan1 is up, line protocol is up
  Hardware is EtherSVI, address is 0019.e86a.6fc0 (bia 0019.e86a.6fc0)
  Internet address is 192.168.1.101/24
  MTU 1500 bytes, BW 1000000 Kbit, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
! lines omitted for brevity
Emma# show ip default-gateway
192.168.1.1
```

The output of the **show interfaces vlan 1** command lists two important details related to switch IP addressing. First, this **show** command lists the interface status of the VLAN 1 interface—in this case, "up and up." If the VLAN 1 interface is not up, the switch cannot use its IP address to send and receive management traffic. Notably, if you forget to issue the **no shutdown** command, the VLAN 1 interface remains in its default shutdown state and is listed as "administratively down" in the **show** command output.

Second, note that the output lists the interface's IP address on the third line. If you statically configure the IP address, as in Example 6-7, the IP address will always be listed; however, if you use DHCP and DHCP fails, the **show interfaces vlan** *x* command will not list an IP address here. When DHCP works, you can see the IP address with the **show interfaces vlan 1** command, but that output does not remind you whether the address is either statically configured or DHCP leased. So it does take a little extra effort to make sure you know whether the address is statically configured or DHCP-learned on the VLAN interface.

## Miscellaneous Settings Useful in the Lab

This last short section of the chapter touches on a couple of commands that can help you be a little more productive when practicing in a lab.

### History Buffer Commands

When you enter commands from the CLI, the switch saves the last several commands in the **history buffer**. Then, as mentioned in Chapter 4, "Using the Command-Line Interface," you can use the up-arrow key or press Ctrl+P to move back in the history buffer to retrieve a command you entered a few commands ago. This feature makes it easy and fast to use a set of commands repeatedly. Table 6-2 lists some of the key commands related to the history buffer.

**Table 6-2** Commands Related to the History Buffer

| Command | Description |
|---|---|
| **show history** | An EXEC command that lists the commands currently held in the history buffer. |
| **terminal history size** *x* | From EXEC mode, this command allows a single user to set, just for this one login session, the size of his or her history buffer. |
| **history size** *x* | A configuration command that, from console or vty line configuration mode, sets the default number of commands saved in the history buffer for the users of the console or vty lines, respectively. |

## The logging synchronous, exec-timeout, and no ip domain-lookup Commands

These next three configuration commands have little in common, other than the fact that they can be useful settings to reduce your frustration when using the console of a switch or router.

The console automatically receives copies of all unsolicited syslog messages on a switch. The idea is that if the switch needs to tell the network administrator some important and possibly urgent information, the administrator might be at the console and might notice the message.

Unfortunately, IOS (by default) displays these syslog messages on the console's screen at any time—including right in the middle of a command you are entering, or in the middle of the output of a **show** command. Having a bunch of text show up unexpectedly can be a bit annoying.

You could simply disable the feature that sends these messages to the console and then re-enable the feature later using the **no logging console** and **logging console** global configuration commands. For example, when working from the console, if you want to temporarily not be bothered by log messages, you can disable the display of these messages with the **no logging console** global configuration command, and then when finished, enable them again.

However, IOS supplies a reasonable compromise, telling the switch to display syslog messages only at more convenient times, such as at the end of output from a **show** command. To do so, just configure the **logging synchronous** console line subcommand, which basically tells IOS to synchronize the syslog message display with the messages requested using **show** commands.

Another way to improve the user experience at the console is to control timeouts of the login session from the console or when using Telnet or SSH. By default, the switch automatically disconnects console and vty (Telnet and SSH) users after 5 minutes of inactivity. The **exec-timeout** *minutes seconds* line subcommand enables you to set the length of that inactivity timer. In the lab (but not in production), you might want to use the special value of 0 minutes and 0 seconds, meaning "never time out."

Finally, IOS has an interesting combination of features that can make you wait for a minute or so when you mistype a command. First, IOS tries to use DNS **name resolution** on IP

hostnames—a generally useful feature. If you mistype a command, however, IOS thinks you want to telnet to a host by that name. With all default settings in the switch, the switch tries to resolve the hostname, cannot find a DNS server, and takes about a minute to time out and give you control of the CLI again.

To avoid this problem, configure the **no ip domain-lookup** global configuration command, which disables IOS's attempt to resolve the hostname into an IP address.

Example 6-10 collects all these commands into a single example, as a template for some good settings to add in a lab switch to make you more productive.

**Example 6-10** *Commands Often Used in the Lab to Increase Productivity*

```
no ip domain-lookup
!
line console 0
 exec-timeout 0 0
 logging synchronous
 history size 20
!
line vty 0 15
 exec-timeout 0 0
 logging synchronous
 history size 20
```



## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element section titled "Step 2: Build Your Study Habits Around the Chapter" for more details. Table 6-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 6-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review config checklists | | Book, website |
| Do labs | | Sim Lite, blog |
| Review command tables | | Book |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 6-4** Key Topics for Chapter 6

| Key Topic Element | Description | Page Number |
|---|---|---|
| Example 6-2 | Example of configuring password login security (no usernames) | 138 |
| Figure 6-5 | SSH configuration commands with related username login security | 142 |

## Key Terms You Should Know

AAA, AAA server, default gateway, DNS, enable mode, history buffer, local username, log message, name resolution, Secure Shell (SSH), Telnet, VLAN interface

## Do Labs

The Sim Lite software is a version of Pearson's full simulator learning product with a subset of the labs, included with this book for free. The subset of labs mostly relates to this chapter. Take the time to try some of the labs. As always, also check the author's blog site pages for configuration exercises (Config Labs) at https://www.certskills.com.

## Command References

Tables 6-5, 6-6, 6-7, and 6-8 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 6-5** Login Security Commands

| Command | Mode/Purpose/Description |
|---|---|
| **line console 0** | Changes the context to console configuration mode. |
| **line vty** *1st-vty last-vty* | Changes the context to vty configuration mode for the range of vty lines listed in the command. |
| **login** | Console and vty configuration mode. Tells IOS to prompt for a password. |
| **password** *pass-value* | Console and vty configuration mode. Lists the password required if the **login** command (with no other parameters) is configured. |
| **login local** | Console and vty configuration mode. Tells IOS to prompt for a username and password, to be checked against locally configured **username** global configuration commands on this switch or router. |
| **username** *name* **secret** *pass-value* | Global command. Defines one of possibly multiple usernames and associated passwords, used for user authentication. Used when the **login local** line configuration command has been used. |

| Command | Mode/Purpose/Description |
|---|---|
| **crypto key generate rsa** [**modulus** *512..2048*] | Global command. Creates and stores (in a hidden location in flash memory) the keys required by SSH. |
| **transport input** {**telnet** \| **ssh** \| **all** \| **none**} | vty line configuration mode. Defines whether Telnet/SSH access is allowed into this switch. Both values can be configured on one command to allow both Telnet and SSH access (the default). |
| **ip domain name** *fqdn* | Global command. Defines the fully-qualified domain name (*fqdn*) for the DNS domain in which the switch or router resides. |
| **hostname** *name* | Global command. Sets the *name* that the device uses for itself, which is also used at the initial text in the command prompt. |
| **ip ssh version 2** | Global command. Sets the SSH server to use only version 2, rather than the default of supporting both versions 1 and 2. |

**Table 6-6** Switch IPv4 Configuration

| Command | Mode/Purpose/Description |
|---|---|
| **interface vlan** *number* | Changes the context to VLAN interface mode. For VLAN 1, allows the configuration of the switch's IP address. |
| **ip address** *ip-address subnet-mask* | VLAN interface mode. Statically configures the switch's IP address and mask. |
| **ip address dhcp** | VLAN interface mode. Configures the switch as a DHCP client to discover its IPv4 address, mask, and default gateway. |
| **ip default-gateway** *address* | Global command. Configures the switch's default gateway IPv4 address. Not required if the switch uses DHCP. |
| **ip name-server** *server-ip-1 server-ip-2 …* | Global command. Configures the IPv4 addresses of DNS servers, so any commands when logged in to the switch will use the DNS for name resolution. |

**Table 6-7** Other Switch Configuration

| Command | Mode/Purpose/Description |
|---|---|
| **hostname** *name* | Global command. Sets this switch's hostname, which is also used as the first part of the switch's command prompt. |
| **enable secret** *pass-value* | Global command. Sets this switch's password that is required for any user to reach enable mode. |
| **history size** *length* | Line config mode. Defines the number of commands held in the history buffer, for later recall, for users of those lines. |
| **logging synchronous** | Console or vty mode. Tells IOS to send log messages to the user at natural break points between commands rather than in the middle of a line of output. |
| **[no] logging console** | Global command. Disables or enables the display of log messages to the console. |

| Command | Mode/Purpose/Description |
|---|---|
| **exec-timeout** *minutes* [*seconds*] | Console or vty mode. Sets the inactivity timeout so that after the defined period of no action, IOS closes the current user login session. |
| **no ip domain-lookup** | Global command. Disables the use of the DNS client on the switch. |

**Table 6-8** Chapter 6 EXEC Command Reference

| Command | Purpose |
|---|---|
| **show running-config** | Lists the currently used configuration. |
| **show running-config \| begin line vty** | Pipes (sends) the command output to the **begin** command, which only lists output beginning with the first line that contains the text "line vty." |
| **show dhcp lease** | Lists any information the switch acquires as a DHCP client. This includes IP address, subnet mask, and default gateway information. |
| **show crypto key mypubkey rsa** | Lists the public and shared key created for use with SSH using the **crypto key generate rsa** global configuration command. |
| **show ip ssh** | Lists status information for the SSH server, including the SSH version. |
| **show ssh** | Lists status information for current SSH connections into and out of the local switch. |
| **show interfaces vlan** *number* | Lists the interface status, the switch's IPv4 address and mask, and much more. |
| **show ip default-gateway** | Lists the switch's setting for its IPv4 default gateway. |
| **terminal history size** *x* | Changes the length of the history buffer for the current user only, only for the current login to the switch. |
| **show history** | Lists the commands in the current history buffer. |

*This page intentionally left blank*



CHAPTER 7

# Configuring and Verifying Switch Interfaces

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.b Layer 2 and Layer 3 switches**

**1.3 Compare physical interface and cabling types**

**1.3.b Connections (Ethernet shared media and point-to-point)**

**1.4 Identify interface and cable issues (collisions, errors, mismatch duplex, and/or speed)**

The chapters in Part II of this book move back and forth between switch administration and core switch functions. In Chapter 4, "Using the Command-Line Interface," you read about the fundamentals of the command-line interface (CLI) and how to use commands that configure and verify switch features. In Chapter 5, "Analyzing Ethernet LAN Switching," you learned about the primary purpose of a switch—forwarding Ethernet frames—and how to see that process in action by looking at the switch MAC address table. Chapter 6, "Configuring Basic Switch Management," then moved back to more administrative tasks, where you learned a few management plane features, like how to configure the switch to support Telnet and Secure Shell (SSH) by configuring IP address and login security.

This chapter focuses on more core switch features, specifically how to configure switch interfaces so that they work. The first section shows how to configure switch interfaces to use the correct speed and duplex, primarily by using IEEE autonegotiation. The second section examines some administrative settings on switches, including how to disable and re-enable an interface. The final section then focuses on how to use **show** commands on a switch to verify switch interface status and interpret the output to find some of the more common issues with switch interfaces.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book and on the companion website, includes answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 7-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Configuring Switch Interface Speed and Duplex | 1–3 |
| Managing Switch Interface Configuration | 4–5 |
| Analyzing Switch Interface Status and Statistics | 6–8 |

1. Switch SW1 connects its G1/0/1 port to PC1. Both devices use IEEE autonegotiation and have 10/100/1000 ports. Which answer describes how the switch chooses its G1/0/1 speed and duplex settings? (Choose two answers.)
   a. Speed by comparing capabilities per received autonegotiation messages
   b. Speed by analyzing the electrical signal of incoming Ethernet frames from PC1
   c. Duplex by comparing capabilities per received autonegotiation messages
   d. Duplex by analyzing the electrical signal of incoming Ethernet frames from PC1
   e. Duplex by choosing a default based on the chosen speed
2. In which of the following modes of the CLI could you configure the duplex setting for interface Fast Ethernet 0/5?
   a. User mode
   b. Enable mode
   c. Global configuration mode
   d. VLAN mode
   e. Interface configuration mode
3. Switch SW1 connects its G1/0/1 port to PC1. Both devices are 10/100/1000 ports. While the switch port uses IEEE autonegotiation, PC1 has disabled it. Which answer describes how the switch chooses its G1/0/1 speed and duplex settings?
   a. Speed by comparing capabilities per received autonegotiation messages
   b. Speed by analyzing the electrical signal of incoming Ethernet frames from PC1
   c. Duplex by comparing capabilities per received autonegotiation messages
   d. Duplex by analyzing the electrical signal of incoming Ethernet frames from PC1
   e. Duplex by choosing a default based on the chosen speed
4. Switch interface G1/0/5 has been cabled correctly in anticipation of some weekend work. However, the engineer needs to prevent the interface from being used until she enables it remotely during a change window this weekend. Which action helps achieve that goal?
   a. Unplug the cable.
   b. Issue the **shutdown g1/0/5** global configuration command.
   c. Issue the **shutdown** interface subcommand under interface **g1/0/5**.
   d. Issue the **disable g1/0/5** global configuration command.
   e. Issue the **enable** interface subcommand under interface **g1/0/5**.

5. An engineer configures Cisco switch SW1 with the commands **interface range G1/0/10-20** and then **description connected to endpoint device**. The engineer exits configuration mode and issues a **show running-config** command. Which answers best describe the related output? (Choose two answers.)
   a. The output includes the **interface range** command.
   b. The output does not include the **interface range** command.
   c. The output lists one **description connected to endpoint device** interface subcommand.
   d. The output lists 11 **description connected to endpoint device** interface subcommands.
6. The output from the switch command **show interfaces status** shows interface Fa0/1 in a "disabled" state. Which of the following is true about interface Fa0/1? (Choose three answers.)
   a. The interface is configured with the **shutdown** command.
   b. The **show interfaces fa0/1** command will list the interface with two status codes of administratively down and line protocol down.
   c. The **show interfaces fa0/1** command will list the interface with two status codes up and down.
   d. The interface cannot currently be used to forward frames.
   e. The interface can currently be used to forward frames.
7. Switch SW1 Gigabit 1/0/1 connects to switch SW2's Gigabit 1/0/2 interface, both 10/100/1000 ports. The switch SW2 configuration includes the **speed** and **duplex** commands, the combination of which happens to disable autonegotiation on that port. Which combination of settings in SW2's **speed** and **duplex** commands results in a duplex mismatch between SW1 and SW2? (Choose two answers.)
   a. **speed 100** and **duplex full**
   b. **speed 100** and **duplex half**
   c. **speed 10** and **duplex full**
   d. **speed 10** and **duplex half**
8. Switch SW1 connects via a cable to switch SW2's G1/0/1 port. Which of the following conditions is the most likely to cause SW1's late collision counter to continue to increment?
   a. SW2's G1/0/1 has been configured with a **shutdown** interface subcommand.
   b. The two switches have been configured with different values on the **speed** interface subcommand.
   c. A duplex mismatch exists with SW1 set to full duplex.
   d. A duplex mismatch exists with SW1 set to half duplex.

# Foundation Topics

## Configuring Switch Interface Speed and Duplex

When physically creating an Ethernet LAN, you must consider the cabling and connectors that match the dozens of physical layer Ethernet standards that help you meet the physical requirements for the LAN. Once chosen and installed, any interfaces connected to fiber cabling require no additional configuration; however, UTP cabling can have different pinouts, and UTP supports different speeds and duplex settings. So once installed, interfaces that use UTP cabling may need additional configuration.

This first major section of the chapter examines interface speed and duplex settings, along with the IEEE autonegotiation process. It also discusses auto-MDIX, a feature that deals with pinout issues. Finally, this entire chapter continues the goal of helping you learn more about CLI navigation and conventions.

### IEEE Autonegotiation Concepts

Ethernet NICs and switch ports often support multiple standards and therefore support multiple speeds. For instance, you will see designations like these:

**10/100:** A port that supports 10- and 100-Mbps Ethernet

**10/100/1000:** A port that supports 10-, 100-, and 1000-Mbps Ethernet

Using hardware that supports multiple standards and speeds allows for much easier growth and migration over time—mainly because the devices can automatically sense the fastest speed using the IEEE autonegotiation feature. For instance, many switches today have many 10/100/1000 ports. Many newer end-user devices also have a 10/100/1000 Ethernet NIC, so the PC and switch can autonegotiate to 1000 Mbps (1 Gbps). Older devices, and some specialized devices that might not need faster speeds, might support only 10 Mbps or 100 Mbps. **Autonegotiation** gives the devices on each link the means to agree to use the best speed without manually configuring the speed on each switch port.

IEEE autonegotiation defines a process by which both devices on each link tell the neighboring device their capabilities. Once both endpoints learn about the other, they use the standard with the fastest speed. Autonegotiation also defines **full duplex** as the preferred option over **half duplex**, assuming both devices support that option.

#### Autonegotiation Under Working Conditions

With IEEE autonegotiation, a device declares its capabilities by sending a series of Fast Link Pulses (FLPs). The data in the FLP messages include bits that identify the Ethernet standards supported by the device and the duplex ability supported, in effect declaring the speeds and duplex settings supported by the device.

The FLPs work even before the endpoints choose a physical layer standard. FLPs use out-of-band electrical signaling, independent of the various physical layer standards for Ethernet frame transmission. Any device that supports autonegotiation supports using these out-of-band FLP messages. The FLPs solve the problem of how the devices can send information to each other even before the link is up and working for normal data transmission.

Figure 7-1 shows three examples of autonegotiation working as intended. First, the company installs cabling that supports 10BASE-T, 100BASE-T, and 1000BASE-T, that is, cabling with four wire pairs of appropriate quality. In the three examples, the PC and switch both use autonegotiation. The cables work with correct straight-through pinouts. The switch, in this case, has all 10/100/1000 ports, while the PC NICs support different options (10 only, 10/100, and 10/100/1000), as per the figure.

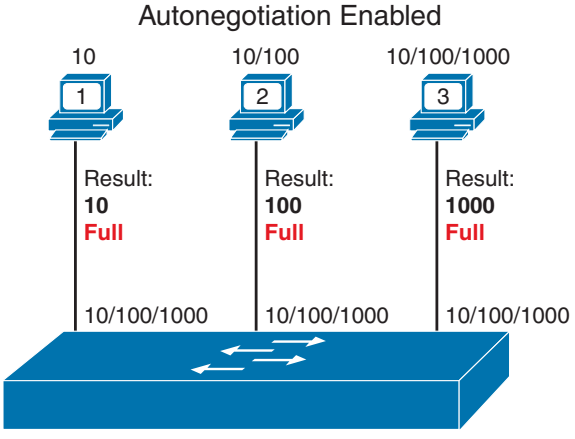



**Figure 7-1** *IEEE Autonegotiation Results with Both Nodes Working Correctly*

The following list breaks down the logic, one PC at a time:

- **PC1:** PC1 sends autonegotiation FLPs that declare support for 10 Mbps only (not 100 or 1000 Mbps) and support for both full and half duplex. The switch announces support for 10, 100, and 1000 Mbps and both duplex settings. As a result, both the PC and the switch choose the fastest speed (10 Mbps) and the best duplex (full) that both support.
- **PC2:** PC2 declares support for 10 and 100 Mbps and both full and half duplex. The switch again claims support for 10, 100, and 1000 Mbps and both full and half duplex. Both devices use the best common speed and duplex (100 Mbps and full duplex).
- **PC3:** PC3 uses a 10/100/1000 NIC, supporting all three speeds, so both the NIC and switch port choose 1000 Mbps and full duplex.

Summarizing, the following list details autonegotiation rules when both endpoints use it:

Key Topic

- Both endpoints send messages, out-of-band compared to any specific data transmission standard, using Fast Link Pulses (FLPs).

---

Answers to the "Do I Know This Already?" quiz:

**1** A, C **2** E **3** B, E **4** C **5** B, D **6** A, B, D **7** A, C **8** D

- The messages declare all supported speed and duplex combinations.
- After hearing from the link partner, each device chooses the fastest speed supported by both devices and the best duplex (full being better than half duplex).

### Autonegotiation Results When Only One Node Uses Autonegotiation

With both devices using autonegotiation, the result is obvious. Both devices use the fastest speed and best duplex supported by both devices.

Cisco repeatedly recommends using autonegotiation on both ends of all Ethernet links that support it—but if you must disable it, make sure to configure both speed and duplex on both ends of the link. For instance, some installations prefer to predefine the speed and duplex on links between two switches. To do so, use commands such as **speed 1000** (meaning 1000 Mbps, or 1 Gbps) and **duplex full**. If configured with these same values on both ends of the link, the link will work due to matching settings of 1000BASE-T with full duplex.

However, many devices have the capability to disable autonegotiation, which means an engineer can make the poor choice to disable autonegotiation on one end of the link but not the other. In real networks, do not do that, because it can cause problems like a *duplex mismatch* and *late collisions*.

To understand what happens, first consider the device with autonegotiation disabled. It must use some physical layer standard (typically due to a static configuration setting), so it begins sending Ethernet frames that conform to that standard—but it does not send FLPs. The other device (the one that uses autonegotiation) sends FLPs, but receives none. Once the device that is attempting autonegotiation realizes it is not receiving FLPs, it can examine the incoming electrical signal of those Ethernet frames and notice the differences between the signals used for 10BASE-T, 100BASE-T, and so on in the incoming signals. Knowing that, it uses the same standard, solving the question of what speed to use.

The choice of duplex requires using a default. If the speed is 10 or 100 Mbps, the device attempting autonegotiation uses half duplex. Otherwise, it chooses full duplex.

The IEEE refers to the logic used by autonegotiation when the other device has disabled autonegotiation as **parallel detection**, summarized as follows:

Key Topic

- **Speed:** Detect the neighboring device's physical layer standard by analyzing the neighbor's incoming frames. Use that speed.
- **Duplex:** Make a default choice based on speed—half duplex if the speed is 10 or 100 Mbps, and full duplex if faster.

**NOTE** Ethernet interfaces using speeds more than 1 Gbps always use full duplex.

Figure 7-2 shows three examples of autonegotiation parallel detection logic. In each case, the PC configuration has disabled autonegotiation while the switch (with all 10/100/1000 ports) continues to use autonegotiation. The top of the figure shows the configured settings on each PC NIC, with the choices made by the switch listed next to each switch port.

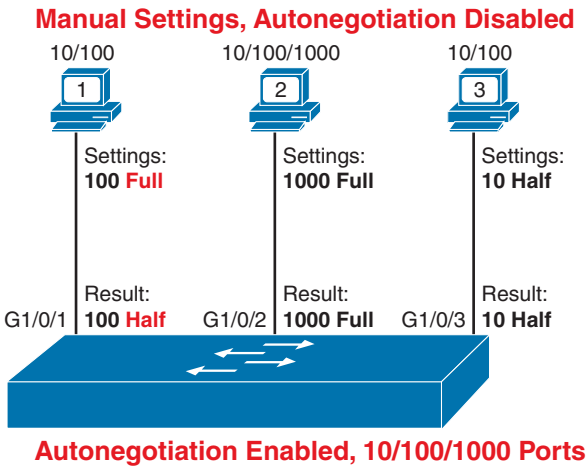



**Figure 7-2** *IEEE Autonegotiation Results Using Parallel Detection Logic*

Reviewing each link, left to right:

- **PC1:** The PC uses 100 Mbps and full duplex settings. The switch receives no autonegotiation FLP messages on port G1/0/1, instead sensing that PC1 is sending frames at 100 Mbps. Then the switch chooses to use half duplex per the defaults (half duplex if the speed is 10 or 100 Mbps).
- **PC2:** The switch uses the same steps and logic as for the link to PC1. Switch port G1/0/2 senses the use of 1000BASE-T signaling, with a speed of 1 Gbps, and chooses a full duplex per the speed-based defaults.
- **PC3:** The PC uses the worst settings possible, with the slower speed (10 Mbps) and the worse duplex setting (half). The switch port receives no FLP messages, so it senses the use of 10BASE-T per the incoming frames, uses 10 Mbps, and chooses half duplex per the speed-based defaults.

Take a closer look at the PC1 example: It shows a poor result called a **duplex mismatch**. The two nodes (PC1 and SW1's port G1/0/1) both use the same 100 Mbps so that they can send data. However, PC1, using full duplex, does not attempt to use carrier sense multiple access with collision detection (CSMA/CD) logic and sends frames at any time. Switch port G1/0/1, using half duplex, does use CSMA/CD. As a result, switch port G1/0/1 will believe collisions occur on the link if, when sending a frame, PC1 also sends a frame. When that happens, the switch port will stop transmitting, back off, resend frames, and so on. As a result, the link is up, but it performs poorly. The upcoming section titled "The Duplex Mismatch Issue" will explore this problem with a focus on how to recognize the symptoms of a duplex mismatch.

**NOTE** To emphasize, in real networks, use autonegotiation. If you have specific reasons not to use it, ensure you configure the devices on both ends of the link and use the same settings.

### Autonegotiation and LAN Hubs

LAN hubs also impact how autonegotiation works. Hubs do not participate in autonegotiation, they do not generate FLP messages, and they do not forward the autonegotiation FLP messages sent by connected devices. As a result, devices connected to a hub receive no FLP messages and use only IEEE autonegotiation parallel detection rules. That can work, but it often results in the devices using 10 Mbps and half duplex.

Figure 7-3 shows an example of a small Ethernet LAN that uses an old 10BASE-T hub. The devices on the right (PC3 and PC4) sense the speed as 10 Mbps per the incoming signal, and then they choose to use the default duplex when using 10 Mbps of half duplex. Using 10 Mbps and half duplex works well in this case: The PCs on the right need to use half duplex because the hub requires any attached devices to use half duplex to avoid collisions.

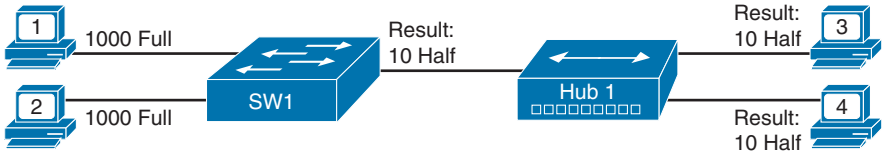



**Figure 7-3** *IEEE Autonegotiation with a LAN Hub*

## Configuring Autonegotiation, Speed, and Duplex

For an Ethernet link to work correctly, the link needs a working cable, and the endpoints need to use the same physical layer standard and duplex setting. With Cisco switches, the default switch setting to use autonegotiation should make the switch use the right speed and duplex so that the link works. The following pages show how to verify the autonegotiation process to see what a switch has chosen to use on an interface, along with how to manually set the speed and duplex with some other related commands.

### Using Autonegotiation on Cisco Switches

Figure 7-4 shows a small network used in the next few examples. The figure shows a working link with an installed cable, a powered-on device (PC1), and both devices using IEEE autonegotiation. The other two links do not work because, in one case, no cable has been connected, and in the other, the cable is installed, but the device is powered off.

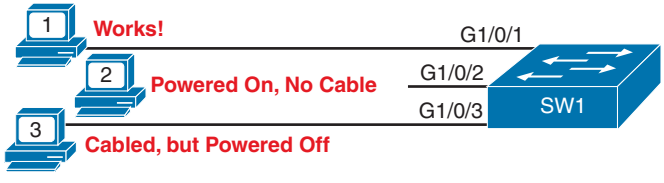



**Figure 7-4** *Network Topology to Match Examples 7-1 Through 7-4*

The switch (SW1) uses the default autonegotiation settings in the first few examples. To configure those settings overtly, you would configure the interface subcommands **speed auto** and **duplex auto**. However, because the **show running-config** and **show startup-config** commands generally do not show default configuration commands, the absence of the **speed** and **duplex** commands in Example 7-1 confirms the interfaces use autonegotiation.

Key Topic

**Example 7-1** *Confirming All Default Settings on the Switch Interfaces in Figure 7-1*

```
SW1# show running-config
! Lines omitted for brevity
interface GigabitEthernet1/0/1
!
interface GigabitEthernet1/0/2
!
interface GigabitEthernet1/0/3
!
! Lines omitted for brevity
SW1#
```

Example 7-2 shows how you can overcome your doubts about whether the absence of the **speed auto** and **duplex auto** interface subcommands confirms those settings. The example shows the commands configured on an interface. Even after configuring the commands, the **show running-config** command does not display them, confirming them as the default settings. (You can use that process for any configuration command, by the way.)

**Example 7-2** *Confirming Autonegotiation Is the Default Setting*

```
SW1# configure terminal
SW1(config)# interface gigabitEthernet 1/0/1
SW1(config-if)# speed auto
SW1(config-if)# duplex auto
SW1(config-if)# ^Z
SW1# show running-config interface gigabitEthernet 1/0/1
Building configuration...

Current configuration : 38 bytes
!
interface GigabitEthernet1/0/1
end
SW1#
```

Also, as a quick way to check interface configuration, note the **show running-config interface GigabitEthernet1/0/1** command at the end of the example. It is a supported command that shows only the configuration for the listed interface.

The better way to confirm the operation of autonegotiation on a switch interface relies on the **show interfaces status** command. Example 7-3 shows the output based on the state from Figure 7-4. In particular, note the Duplex and Speed columns of the output, with the following list identifying the meaning:

**a-full:** Full duplex, with the a- meaning the switch learned the value using autonegotiation.

**a-1000:** 1000 Mbps (1 Gbps), with the a- meaning the switch learned the setting using autonegotiation.

**auto:** The interface will use autonegotiation when the link physically works.

Key Topic

**Example 7-3** *Interpreting Autonegotiation Clues Before/After Completion*

```
SW1# show interfaces status
Port       Name               Status       Vlan       Duplex  Speed Type
Gi1/0/1                       connected    1          a-full a-1000 10/100/1000BaseTX
Gi1/0/2                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/3                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/4                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/5                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/6                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/7                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/8                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/9                       notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/10                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/11                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/12                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/13                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/14                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/15                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/16                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/17                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/18                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/19                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/20                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/21                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/22                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/23                      notconnect   1          auto     auto 10/100/1000BaseTX
Gi1/0/24                      notconnect   1          auto     auto 10/100/1000BaseTX
Te1/1/1                       connected    1          full      10G SFP-10GBase-SR
Te1/1/2                       notconnect   1          auto     auto unknown
Te1/1/3                       notconnect   1          auto     auto unknown
Te1/1/4                       notconnect   1          auto     auto unknown
```

The first few output lines in Example 7-3 confirm that port G1/0/1 works with autonegotiation. Given the highlighted values in the duplex and speed columns, you can see that port G1/0/1 uses 1000BASE-T and full duplex, as learned by autonegotiation (per the a- prefix).

The output lines for ports G1/0/2 and G1/0/3 show the normal state for a nonworking port. The notconnect state per the Status column means that the link is not functional—in PC2's case because no cable exists and in PC3's case because PC3 is powered off. Autonegotiation can work only if the physical link works.

Beyond the **show interfaces status**, the **show interfaces** command also gives some autonegotiation data. Example 7-4 shows the output for working interface G1/0/1 from Figure 7-4. Note the highlighted text about seven lines into the example, which shows the speed and duplex used on the link; however, note that this command on working interface G1/0/1 lists no information on whether it used autonegotiation.

Key Topic

**Example 7-4** **show interfaces** *Command Autonegotiation Clues*

```
SW1# show interfaces gigabitEthernet 1/0/1
GigabitEthernet1/0/1 is up, line protocol is up (connected)
  Hardware is Gigabit Ethernet, address is 4488.165a.f201 (bia 4488.165a.f201)
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full-duplex, 1000Mb/s, media type is 10/100/1000BaseTX
  input flow-control is on, output flow-control is unsupported
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 00:00:01, output 00:00:00, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/2000/0/0 (size/max/drops/flushes); Total output drops: 0
  Queueing strategy: fifo
  Output queue: 0/40 (size/max)
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
     14228 packets input, 1870879 bytes, 0 no buffer
     Received 14223 broadcasts (14222 multicasts)
     0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
     0 watchdog, 14222 multicast, 0 pause input
     0 input packets with dribble condition detected
     56865 packets output, 7901974 bytes, 0 underruns
     Output 17109 broadcasts (0 multicasts)
     0 output errors, 0 collisions, 2 interface resets
     0 unknown protocol drops
     0 babbles, 0 late collision, 0 deferred
     0 lost carrier, 0 no carrier, 0 pause output
     0 output buffer failures, 0 output buffers swapped out

SW1# show interfaces g1/0/2
GigabitEthernet1/0/2 is down, line protocol is down (notconnect)
  Hardware is Gigabit Ethernet, address is 4488.165a.f202 (bia 4488.165a.f202)
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Auto-duplex, Auto-speed, media type is 10/100/1000BaseTX
! Lines omitted for brevity
```

However, the **show interfaces** command does give some insight into autonegotiation for a nonworking interface such as G1/0/2, as seen at the end of Example 7-4. On an interface with default settings **speed auto** and **duplex auto**, the command output implies those settings

with the auto-duplex and auto-speed text. However, the output shows those specific settings if configured to a specific speed or duplex.

### Setting Speed and Duplex Manually

The switch **speed** and **duplex** interface subcommands can set an interface's specific speed and duplex. Risking being repetitive: In production networks, use and trust autonegotiation. However, for exam preparation, to cover all the combinations, if you want to configure the settings instead, Cisco recommends that you configure both devices on the ends of the link (to the same values, of course).

Figure 7-5 and Example 7-5 show an example of manually configuring the speed and duplex on a link between two switches. As per the recommendation, the engineer configures both devices with the same settings, and the link works.

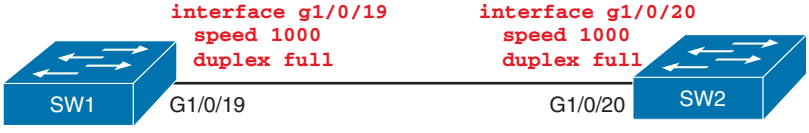



**Figure 7-5** *Configuring Speed and Duplex on a Switch-to-Switch Link*

Key Topic

**Example 7-5** *Configuring Speed and Duplex on Both Ends of a Link*

```
SW1# show running-config interface g1/0/19
Building configuration...

Current configuration : 63 bytes
!
interface GigabitEthernet1/0/19
 speed 1000
 duplex full
end
```

```
! Now, on switch SW2
SW2# show running-config interface g1/0/20
Building configuration...

Current configuration : 64 bytes
!
interface GigabitEthernet1/0/20
 speed 1000
 duplex full
end
SW1#
```

In the scenario shown in Example 7-5, the two switches set both the speed and duplex, so they do not need to use autonegotiation. In this case, they just begin using the 1000BASE-T standard with full duplex, and the link works.

Example 7-6 shows a hidden gem in the output of the **show interfaces status** command that tells you the switch did not use autonegotiation in this case. First, note that switch SW1's

G1/0/19 interface reaches a connected state, so the link works. The Duplex and Speed columns list full and 1000 without the "a-" prefix. The absence of the "a-" means that the interface did not use autonegotiation to choose the setting, instead using the configuration. (See Example 7-3 for the earlier example showing values of a-full and a-1000.)

Key Topic

**Example 7-6** show interfaces status *Without Using Autonegotiation*

```
SW1# show interfaces g1/0/20 status

Port         Name              Status       Vlan       Duplex  Speed Type
Gi1/0/19                       connected    1            full   1000
10/100/1000BaseTX

SW1# show interfaces g1/0/19
GigabitEthernet1/0/19 is up, line protocol is up (connected)
  Hardware is Gigabit Ethernet, address is 4488.165a.f213 (bia 4488.165a.f213)
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full-duplex, 1000Mb/s, media type is 10/100/1000BaseTX
! Lines omitted for brevity
```

**NOTE** On some Cisco Catalyst switch ports, configuring both speed and duplex disables autonegotiation on that port. On others, it does not. In real networks, should you ever need to configure both the speed and duplex to specific values, take extra care to confirm that the speed and duplex match on both ends of the link.

## Using Auto-MDIX on Cisco Switches

For switch interfaces to work, meaning they reach a connected state, the engineer should install a correct cable between the switch port and some other connected device. For UTP cabling, the cables often terminate with RJ-45 connectors. The cables must also conform to a straight-through or crossover cable pinout, as discussed in the section, "Building Physical Ethernet LANs with UTP," in Chapter 2, "Fundamentals of Ethernet LANs."

Chapter 2 also introduced a related concept called **auto-MDIX**, or automatic medium-dependent interface crossover. Auto-MDIX, when enabled, gives an Ethernet interface the ability to sense when the attached cable uses the wrong cable pinout and to overcome the problem. For instance, a link between two switches should use a crossover cable pinout. If the cable has a straight-through pinout, the auto-MDIX feature can sense the issue and swap pairs in the interface electronics, achieving the same effect as a crossover cable.

Figure 7-6 and Examples 7-7 and 7-8 demonstrate auto-MDIX. The figure shows a case with two switches connected with a straight-through cable. Cisco Catalyst switches use auto-MDIX by default, with a default interface subcommand of **mdix auto**. As with the **speed** and **duplex** commands earlier in this chapter, the default interface subcommand **mdix auto** command does not appear in the configuration. Example 7-7 confirms the absence of the **mdix auto** subcommand but confirms that the link reaches a connected state.

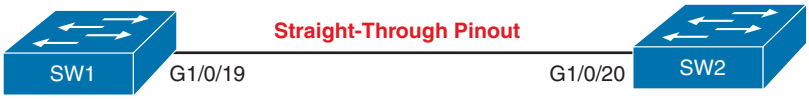



**Figure 7-6** *Switch-Switch Link That Needs Crossover but Uses Straight-Through Cable*

**Example 7-7** *Switch-Switch Link Works with All Default Settings*

```
SW1# running-config interface GigabitEthernet 1/0/19
Building configuration...

Current configuration : 39 bytes
!
interface GigabitEthernet1/0/19
end

SW1# show interfaces 1/0/19 status

Port        Name               Status       Vlan       Duplex  Speed Type
Gi1/0/19                       connected    1          a-full a-1000 10/100/1000BaseTX
```

Auto-MDIX works if either one or both endpoints on the link enable auto-MDIX. In Example 7-7, both switches default to auto-MDIX, but only one needs to swap the wire pairs. If you disable auto-MDIX on just one side, the other side swaps the pairs. To prevent auto-MDIX from doing its valuable work, disable it on both ends of the link using the **no mdix auto** interface subcommand.

Example 7-8 shows just that example. Before the example, the engineer configured switch SW2 with the **no mdix auto** interface subcommand. The example shows the process to disable it on switch SW1. As a result, neither switch uses auto-MDIX, and the interface fails to a notconnect state.

**Example 7-8** *Switch-Switch Link Fails with Auto-MDIX Disabled on Both Ends*

```
SW1# running-config interface GigabitEthernet 1/0/19
SW1# config t
Enter configuration commands, one per line.  End with CNTL/Z.
SW1(config)# int g1/0/19
SW1(config-if)# no mdix auto
SW1(config-if)#
*Oct  5 12:50:22.177: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet1/0/19, changed state to down
*Oct  5 12:50:23.175: %LINK-3-UPDOWN: Interface GigabitEthernet1/0/19, changed state
to down
SW1(config-if)# ^Z
SW1#
SW1# show interfaces g1/0/19 status

Port        Name               Status       Vlan       Duplex  Speed Type
Gi1/0/19                       notconnect   1            auto   auto 10/100/1000BaseTX
```

On a side note, IOS supports many valid abbreviations for the **interface GigabitEthernet 1/0/19** global configuration command, including the **int g1/0/19** command shown in the example.

## Managing Switch Interface Configuration

This next section of the chapter examines a small number of additional interface subcommands, specifically the **description** and **shutdown** commands. The first gives you the ability to document facts about each interface, while the second gives you the means to disable and enable the interface administratively.

This section also explains more about the mechanisms of the IOS CLI for interfaces, with some discussion and examples of removing configuration from an interface using the **no** command.

### The Description and Interface Range Commands

The **description** *text* interface subcommand lets you add a text description to the interface. For the text, you can use keyboard characters, including spaces, with Cisco switches typically supporting around 200 characters of description text. For instance, if you want to store some information in the switch about the interface and the device connected to it, you could document the details with the **description** interface subcommand.

Example 7-9 demonstrates the mechanics of the process with a simple description added to interface G1/0/1 from Figure 7-4. The example shows the configuration plus the output from some **show** commands that repeat the detail. Note that the **show interfaces status** command does not leave enough space for the entire **description** command's text, but the **show interface** command does.

Key Topic

**Example 7-9** *Configuring and Verifying the* **description** *Command on Switch SW1*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface GigabitEthernet 1/0/1
SW1(config-if)# description Link to PC1, using autonegotiation
SW1(config-if)# ^Z
SW1# show interfaces g1/0/1 status

Port         Name               Status       Vlan     Duplex  Speed Type
Gi1/0/1      Link to PC1, using connected    1        a-full a-1000
10/100/1000BaseTX
SW1# show interfaces g1/0/1
GigabitEthernet1/0/1 is up, line protocol is up (connected)
  Hardware is Gigabit Ethernet, address is 4488.165a.f201 (bia 4488.165a.f201)
  Description: Link to PC1, using autonegotiation
! Lines omitted for brevity
SW1#
```

Example 7-10 demonstrates how to configure the same interface subcommand on multiple interfaces simultaneously, saving effort. The example uses the **description** command again but after the **interface range GigabitEthernet 1/0/2 - 10** command. The **interface range**

command tells IOS to apply subsequent subcommands to multiple interfaces, in this case, Gi1/0/2 through Gi1/0/10.

**Example 7-10** *Configuring Multiple Interfaces Using the* **interface range** *Command*

```
SW1# configure terminal
SW1(config)# interface range g1/0/2 - 10
SW1(config-if-range)# description Interface not in use
SW1(config-if-range)# ^Z
SW1#
```

IOS does not put the **interface range** command into the configuration. Instead, it acts as if you had typed the subcommands under every single interface in the specified range. Example 7-11 shows an excerpt from the **show running-config** command, listing the configuration of interfaces G1/0/2 –3 from the configuration in Example 7-10. The example shows the same **description** command on both interfaces; to save space, the example does not bother to show all interfaces G1/0/2 through G1/0/10.

**Example 7-11** *How IOS Expands the Subcommands Typed After* **interface range**

```
SW1# show running-config
! Lines omitted for brevity
interface GigabitEthernet1/0/2
 description Interface not in use
!
interface GigabitEthernet1/0/3
 description Interface not in use
! Lines omitted for brevity
SW1# show interfaces description
Interface                  Status         Protocol Description
Vl1                        up             up
Gi0/0                      admin down     down
Gi1/0/1                    up             up       Link to PC1, using
autonegotiation
Gi1/0/2                    down           down     Interface not in use
Gi1/0/3                    down           down     Interface not in use
! Lines omitted for brevity
```

## Administratively Controlling Interface State with shutdown

As you might imagine, network engineers need a way to enable and disable an interface using a command. In an odd turn of phrase, Cisco switches use the **shutdown** command to disable an interface and the **no shutdown** command to enable an interface. While the **no shutdown** command might seem like an odd command to enable an interface at first, you will use this command a lot in the lab, and it will become second nature. (Most people use the abbreviations **shut** and **no shut**.)

Example 7-12 shows an example of disabling an interface using the **shutdown** subcommand. In this case, switch SW1 has a working interface G1/0/1. The user connects to the switch console and disables the interface. IOS generates a log message each time an interface fails or recovers, and log messages appear at the console, as shown in the example.

Key Topic

**Example 7-12** *Administratively Disabling an Interface with* **shutdown**

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface GigabitEthernet 1/0/1
SW1(config-if)# shutdown
*Oct  6 16:33:14.911: %LINK-5-CHANGED: Interface GigabitEthernet1/0/1, changed state
to administratively down
*Oct  6 16:33:15.911: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet1/0/1, changed state to down
SW1(config-if)#
```

To bring the interface back up again, you must follow the same process but use the **no shutdown** command instead.

Before leaving the simple but oddly named **shutdown/no shutdown** commands, examine the new status codes in the output in Example 7-13. The **show interfaces status** command lists one line of output per interface and, when shut down, lists the interface status as "disabled." That makes logical sense to most people. The **show interfaces** command (without the **status** keyword) lists many lines of output per interface, giving a much more detailed picture of interface status and statistics. With that command, the interface status comes in two parts, with one part using the phrase "administratively down," matching the highlighted log message in Example 7-12.

Key Topic

**Example 7-13** *Interface Status When Configured with the* **shutdown** *Command*

```
SW1# show interfaces g1/0/1 status

Port          Name               Status       Vlan     Duplex  Speed Type
Gi1/0/1       Link to PC1, using disabled     1          auto   auto
10/100/1000BaseTX

SW1# show interfaces g1/0/1
GigabitEthernet1/0/1 is administratively down, line protocol is down (disabled)
  Hardware is Gigabit Ethernet, address is 4488.165a.f201 (bia 4488.165a.f201)
  Description: Link to PC1, using autonegotiation
! Lines omitted for brevity
```

## Removing Configuration with the no Command

The chapters in Part II of this book have two broad goals: to help you learn some specific topics about LAN switches and to also learn about how to use the switch CLI. Some examples have more to do with learning about the CLI, which is the case for the examples that follow.

For any IOS configuration command that you might configure, you also need to consider this question: How can you remove that configuration? With some IOS configuration commands (but not all), you can revert to the default setting by issuing a **no** version of the command. What does that mean? Let me give you a few examples:

- If you configured **speed 1000** on an interface, the **no speed** command on that same interface reverts to the default speed setting (which happens to be **speed auto**).
- Similarly, if you configured an earlier **duplex half** or **duplex full** command, the **no duplex** command in interface mode for the same interface reverts the configuration to the default **duplex auto**.

- If you configured a **description** command with some text, to go back to the default state of having no **description** command for that interface, you can use the **no description** command when in interface configuration mode for that same interface.

Example 7-14 shows a sample interface configuration with four interface subcommands configured. Example 7-15 demonstrates the configuration's removal using various **no** commands.

**Example 7-14** *Existing Configuration on Switch SW1 Interface G1/0/21*

```
SW1# show running-config interface g1/0/21
Building configuration...

Current configuration : 96 bytes
!
interface GigabitEthernet1/0/21
 description link to switch SW2
 speed 1000
 duplex full
 shutdown
end
```

**Example 7-15** *Removing Various Configuration Settings Using the* **no** *Command*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface gigabitethernet 1/0/21
SW1(config-if)# no speed
SW1(config-if)# no duplex
SW1(config-if)# no description
SW1(config-if)# no shutdown
SW1(config-if)# ^Z
SW1#
SW1# show running-config interface g1/0/21
Building configuration...

Current configuration : 39 bytes
!
interface GigabitEthernet1/0/21
end
SW1#
```

In particular, interface g1/0/21 has no interface subcommands at the bottom of Example 7-15.

**NOTE** The **show running-config** and **show startup-config** commands typically do not display default configuration settings. Hence, the absence of interface subcommands under an interface means that all possible subcommands use default values. You can see the configured and default settings using the **show running-config all** command. (Be warned, the **show running-config all** command lists many commands, about ten times the output of the **show running-config** command.)

Alternatively, if the goal is to revert all interface subcommands to their default settings, later IOS versions provide a single command: the **default interface** *interface-id* global configuration command. For instance, if switch SW1 had the configuration shown in Example 7-14 again, the global configuration command **default interface g1/0/21** would accomplish the same result as the list of **no** commands seen in Example 7-15, resulting in all default configuration on the interface.

# Analyzing Switch Interface Status and Statistics

This final major section of the chapter examines how to verify the interfaces work correctly. This section also looks at those more unusual cases in which the interface is working but not working well, as revealed by different interface status codes and statistics.

## Interface Status Codes

Cisco switches use two different sets of interface status codes. The switch **show interfaces** and **show interfaces description** commands list a two-code status named the *line status* and *protocol status*. Together, these two status values identify the state of the interface. Generally, the line status refers to whether the Layer 1 standard works, while the protocol status refers to whether Layer 2 works.

**NOTE** This book refers to these two status codes in shorthand by just listing the two codes with a slash between them, such as *up/up* or *down/down*.

Other commands, like the **show interfaces status** command, use a single-code interface status. The single-word status words correlate to different combinations of the two-code interface status codes, as seen in Table 7-2. For example, the **show interfaces status** command lists the *connected* state for working interfaces, while the **show interfaces** and **show interfaces description** commands list an *up/up* state.

Key Topic

**Table 7-2** LAN Switch Interface Status Codes

| Line Status | Protocol Status | Interface Status | Typical Root Cause |
|---|---|---|---|
| administratively down | down | disabled | The **shutdown** command is configured on the interface. |
| down | down | notconnect | No cable; bad cable; wrong cable pinouts with MDIX disabled; speed mismatch; the neighboring device is (a) powered off, (b) **shutdown**, or (c) error disabled. |
| up | down | notconnect | Not expected on LAN switch physical interfaces. |
| down | down (err-disabled) | err-disabled | Port security (or other feature) has disabled the local interface. |
| up | up | connected | The interface is working. |

Examining the notconnect state for a moment, note that this state has many causes. For example, this state includes the more obvious cases, like a missing or broken cable. Some

examples of the root causes of cabling problems that result in a notconnect state include the following:

- The installation of any equipment that uses electricity, even non-IT equipment, can interfere with the transmission on the cabling and link fail.
- The cable could be damaged, for example, if it lies under the carpet. The electrical signal can eventually degrade if the user's chair squashes the cable.
- Although optical cables do not suffer from electromagnetic interference (EMI), someone can try to be helpful and move a fiber-optic cable out of the way—by bending it too much. A bend into too tight a shape can prevent the cable from transmitting bits (called *macrobending*).

## The Duplex Mismatch Issue

You might think the up/up (connected) state means everything works. Indeed, you want your switch interfaces in that state; however, a variety of problems can occur on a working interface in that state, one being a *duplex mismatch*, as discussed next.

If the devices on the ends of a link use the same physical layer standard with a working cable, the interfaces reach the up/up (connected) state. However, the endpoints can also have opposite duplex settings (a duplex mismatch), with full on one side and half on the other. The interfaces remain up/up and data flows; however, the device that uses half duplex experiences unnecessary errors and retransmissions.

You should remember the recommendation by now: Avoid problems like duplex mismatches by using autonegotiation on both devices on each link. However, because the CCNA exam topics have mentioned the duplex mismatch problem for most of its history, take a closer look at how a duplex mismatch can occur.

Figure 7-7 shows a classic case that results in a duplex mismatch with some Cisco switch ports. Some Cisco switch ports disable autonegotiation on interfaces configured with both a specific speed and duplex setting. The device that uses autonegotiation uses parallel detection rules to discover the speed and then uses a default duplex based on that speed, but that default duplex setting may not match the device on the other end.

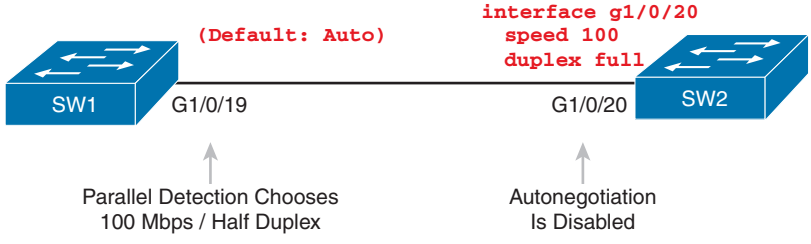



**Figure 7-7** *Classic Case for Duplex Mismatch Between Switches*

Figure 7-7 shows that scenario, with switch SW2's G1/0/20 interface configured with **speed 100** and **duplex full**, disabling autonegotiation. The logic runs like this:

1. Switch SW2 sets speed 100 and duplex full.
2. SW2 disables autonegotiation FLP messages.

3. Because it receives no FLP messages, switch SW1 moves on to use autonegotiation parallel detection logic:
   a. SW1 senses that SW2 uses 100 Mbps speed.
   b. SW1 chooses to use half duplex based on the default table (half duplex if the speed is 10 or 100 Mbps; otherwise, full duplex).

Finding a duplex mismatch can be much more difficult than finding a speed mismatch because speed mismatches result in a failed link, but a duplex mismatch does not. In the duplex mismatch case as shown in Figure 7-7, *if the duplex settings do not match on the ends of an Ethernet segment, the switch interface will still be in a connected state.*

To identify duplex mismatch problems, you have to check the duplex setting on each end of the link to see if the values mismatch. You can also watch for incrementing collision and late collision counters, as explained in the next section.

**NOTE** Some Cisco switch ports do not disable autonegotiation when configured with both **speed** and **duplex** as shown in Figure 7-7. It appears that switch ports that support Power over Ethernet (PoE) do not disable autonegotiation, while ports that do not support PoE do disable autonegotiation, as shown in the figure. However, I found these facts only from experimentation, and not from any Cisco documentation, so be cautious and test if you decide to configure both speed and duplex. Also, the fact that some ports disable autonegotiation when configured with both speed and duplex, but some do not, provides yet another reason to simply use autonegotiation on both ends of the link.

On switch ports that continue using autonegotiation, even after you configure the speed and duplex commands, a duplex mismatch should not occur, because autonegotiation works. Working through the Figure 7-7 example again, but assuming a port that does not disable autonegotiation, consider this sequence:

1. Switch SW2 sets speed 100 and duplex full.
2. SW2 continues to send autonegotiation FLP messages, declaring it can support only 100 Mbps and full duplex.
3. SW1, using autonegotiation, receives SW2's FLPs, and agrees to use 100 Mbps and full duplex—avoiding the duplex mismatch.

## Common Layer 1 Problems on Working Interfaces

When the interface reaches the connected (up/up) state, the switch considers the interface to be working. The switch, of course, tries to use the interface, and at the same time, the switch keeps various interface counters. These interface counters can help identify problems that can occur even though the interface reaches a connected state, like issues related to the just-completed duplex mismatch problem. This section explains some of the related concepts and a few of the most common problems.

The receiving device might receive a frame whose bits have changed values whenever the physical transmission has problems. These frames do not pass the error detection logic as implemented in the FCS field in the Ethernet trailer, as covered in Chapter 2. The receiving device discards the frame and counts it as an *input error*. Cisco switches list this error as a CRC error, as highlighted in Example 7-16. (Cyclic redundancy check [CRC] is a term related to how the frame check sequence [FCS] math detects an error.)

**Example 7-16** *Interface Counters for Layer 1 Problems*

```
SW1# show interfaces gi1/0/1
! lines omitted for brevity
     Received 3943 broadcasts (3941 multicasts)
     0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
     0 watchdog, 3941 multicast, 0 pause input
     0 input packets with dribble condition detected
     18843 packets output, 1726956 bytes, 0 underruns
     Output 10 broadcasts (16378 multicasts)
     0 output errors, 0 collisions, 3 interface resets
     0 unknown protocol drops
     0 babbles, 0 late collision, 0 deferred
     0 lost carrier, 0 no carrier, 0 pause output
     0 output buffer failures, 0 output buffers swapped out
```

The numbers of input and CRC errors are just a few of the counters in the output of the **show interfaces** command. The challenge is to decide which counters you need to think about, which ones show that a problem is happening, and which ones are normal and of no concern.

The example highlights several counters as examples so that you can start to understand which ones point to problems and which ones are just counting everyday events that are not problems. The following list shows a short description of each highlighted counter in the order shown in the example:

Key Topic

**Runts:** Frames that did not meet the minimum frame size requirement (64 bytes, including the 18-byte destination MAC, source MAC, type, and FCS). Collisions can cause it.

**Giants:** Frames that exceed the maximum frame size requirement (default 1518 bytes, including the 18-byte destination MAC, source MAC, type, and FCS).

**Input Errors:** A total of many counters, including runts, giants, no buffer, CRC, frame, overrun, and ignored counts.

**CRC:** Received frames that did not pass the FCS math; can be caused by collisions.

**Frame:** Received frames that have an illegal format, for example, ending with a partial byte, can be caused by collisions.

**Packets Output:** Total number of packets (frames) forwarded out the interface.

**Output Errors:** Total number of packets (frames) that the switch port tried to transmit but for which some problem occurred.

**Collisions:** Counter of all collisions that occur when the interface is transmitting a frame.

**Late Collisions:** The subset of all collisions that happen after the 64th byte of the frame has been transmitted. (In a properly working Ethernet LAN, collisions should occur within the first 64 bytes; **late collisions** today often point to a duplex mismatch.)

Note that many of these counters increment on a properly working interface that uses the CSMA/CD process to implement half duplex. So, a switch interface with an increasing collision counter might not have a problem. However, one problem, called late collisions, points to the classic duplex mismatch problem.

If a LAN design follows cabling guidelines, all collisions should occur by the end of the 64th byte of any frame. When a half-duplex switch interface has already sent 64 bytes of a frame and receives a frame on that same interface, the switch senses a collision. In this case, the collision is a late collision, and the switch increments the late collision counter in addition to the usual CSMA/CD actions to send a jam signal, wait a random time, and try again.

With a duplex mismatch, like the mismatch between SW1 and SW2 in Figure 7-7, the half-duplex interface will likely see the late collisions counter increment. Why? The half-duplex interface sends a frame (SW1), but the full-duplex neighbor (SW2) sends at any time, even after the 64th byte of the frame sent by the half-duplex switch. So, just keep repeating the **show interfaces** command, and if you see the late collisions counter incrementing on a half-duplex interface, you might have a duplex mismatch problem.

A working interface (in an up/up state) can also suffer from issues related to the physical cabling. The cabling problems might not be bad enough to cause a complete failure, but the transmission failures result in some frames failing to pass successfully over the cable. For example, excessive interference on the cable can cause the various input error counters to keep growing, especially the CRC counter. In particular, if the CRC errors grow, but the collision counters do not, the problem might simply be interference on the cable.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element section titled "Step 2: Build Your Study Habits Around the Chapter" for more details. Table 7-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 7-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Review memory tables | | Book, website |
| Do labs | | Sim Lite, blog |

## Review All the Key Topics

Key Topic

**Table 7-4** Key Topics for Chapter 7

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | IEEE autonegotiation rules when both link partners participate | 162 |
| List | IEEE autonegotiation rules when only one link partner participates (parallel detection) | 163 |
| Example 7-1 | Confirming All Default Settings on the Switch Interfaces | 166 |
| Example 7-3 | Interpreting Autonegotiation Clues Before/After Completion | 167 |
| Example 7-4 | **show interfaces** Command Autonegotiation Clues | 168 |
| Example 7-5 | Configuring Speed and Duplex on Both Ends of a Link | 169 |
| Example 7-6 | **show interfaces status** Without Using Autonegotiation | 170 |
| Example 7-9 | Configuring and Verifying the **description** Command on Switch SW1 | 172 |
| Example 7-12 | Administratively Disabling an Interface with **shutdown** | 174 |
| Example 7-13 | Interface Status When Configured with the **shutdown** Command | 174 |
| Table 7-2 | Two types of interface state terms and their meanings | 176 |
| List | Explanations of different error statistics on switch interfaces | 179 |

## Key Terms You Should Know

10/100, 10/100/1000, auto-MDIX, autonegotiation, duplex mismatch, full duplex, half duplex, late collisions, parallel detection

## Do Labs

The Sim Lite software is a version of Pearson's full simulator learning product with a subset of the labs, included free with this book. The subset of labs mostly relates to this part. Take the time to try some of the labs. As always, also check the author's blog site pages for configuration exercises (Config Labs) at https://www.certskills.com.

## Command References

Tables 7-5 and 7-6 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 7-5** Switch Interface Configuration

| Command | Mode/Purpose/Description |
|---|---|
| **interface** *type port-number* | Changes context to interface mode. The type is typically Fast Ethernet or Gigabit Ethernet. The possible port numbers vary depending on the model of switch—for example, Fa0/1, Fa0/2, and so on. |
| **interface range** *type port-number - end-port-number* | Changes the context to interface mode for a range of consecutively numbered interfaces. The subcommands that follow then apply to all interfaces in the range. |
| **shutdown \| no shutdown** | Interface mode. Disables or enables the interface, respectively. |
| **speed** {**10 \| 100 \| 1000 \| auto**} | Interface mode. Manually sets the speed to the listed speed or, with the auto setting, automatically negotiates the speed. |
| **duplex** {**auto \| full \| half**} | Interface mode. Manually sets the duplex to half or full, or to autonegotiate the duplex setting. |
| **description** *text* | Interface mode. Lists any information text the engineer wants to track for the interface, such as the expected device on the other end of the cable. |
| **no duplex**<br>**no speed**<br>**no description** | Reverts to the default setting for each interface subcommand of **speed auto**, **duplex auto**, and the absence of a **description** command. |
| **default interface** *interface-id* | Reverts to the default setting for all interface subcommands on an interface. |
| **[no] mdix auto** | Interface subcommand. Enables (**auto mdix**) or disables (**no auto mdix**) the auto-MDIX feature. |

**Table 7-6** Chapter 7 EXEC Command Reference

| Command | Purpose |
|---|---|
| **show running-config** | Lists the currently used configuration while omitting most default settings |
| **show running-config interface** *type number* | Displays the running-configuration excerpt of the listed interface and its subcommands only |
| **show running-config all** | Displays the running-configuration while including all default settings |
| **show interfaces** [*type number*] **status** | Lists one output line per interface (or for only the listed interface if included), noting the description, operating state, and settings for duplex and speed on each interface |
| **show interfaces** [*type number*] | Lists detailed status and statistical information about all interfaces (or the listed interface only) |
| **show interfaces description** | Displays one line of information per interface, with a two-item status (similar to the **show interfaces** command status), and includes any description configured on the interfaces |

*This page intentionally left blank*



# Part II Review

Keep track of your part review progress with the checklist shown in Table P2-1. Details on each task follow the table.

**Table P2-1** Part II Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Review Appendix L on the Companion Website | | |
| Watch video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book, using the PTP software.

## Answer Part Review Questions

For this task, answer the Part Review questions for this part of the book, using the PTP software.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson ICND1 or CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Blog: Config Labs:** The author's blog includes a series of configuration-focused labs that you can do on paper or with Cisco Packet Tracer in about 15 minutes. To find them, open https://www.certskills.com and look under the Labs menu item.

**Other:** If using other lab tools, as a few suggestions: make sure to experiment heavily with VLAN configuration and VLAN trunking configuration. Also, spend some time changing interface settings like **speed** and **duplex** on a link between two switches, to make sure that you understand which cases would result in a duplex mismatch.

## Review Appendix L on the Companion Website

Appendix L on the companion website contains a chapter from the previous edition of the book that focused on troubleshooting. That appendix, named "LAN Troubleshooting," can be useful as a tool to review the topics in this part of the book. (Note that if you use this extra appendix, you can ignore the mentions of Port Security until you have reached that topic in the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition.)

## Watch Videos

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.



Part II of this book introduces the basics of Ethernet LANs, both in concept and in how to implement the features. However, the two primary features discussed in Part III of this book—Virtual LANs (VLANs) and Spanning Tree Protocol (STP)—impact almost everything you have learned about Ethernet so far.

VLANs allow a network engineer to create separate Ethernet LANs through simple configuration choices. The ability to separate some switch ports into one VLAN and other switch ports into another VLAN gives network designers a powerful tool for creating networks. Once created, VLANs also have a huge impact on how a switch works, which then impacts how you verify and troubleshoot the operation of a campus LAN.

The two VLAN-related exam topics (2.1 and 2.2) use the verbs *configure* and *verify*. To support that depth, Chapter 8 opens this part with details about the concepts related to VLANs, VLAN trunks, and EtherChannels. It then shows a variety of configuration options, sprinkled with some troubleshooting topics.

STP acts to prevent frames from looping repeatedly around a LAN that has redundant links. Without STP, switches would forward broadcasts and some other frames around and around the LAN, eventually clogging the LAN so much as to make it unusable.

The CCNA 200-301 version 1.1 exam blueprint refers to STP using only topic 2.5. That topic uses the plural phrase *Spanning Tree Protocols*, in reference to the original Spanning Tree Protocol (STP) and its replacement Rapid STP (RSTP). That exam topic uses the verb *identify*, which places more importance on interpreting STP operations using **show** commands rather than every configuration option.

As for this part of the book, Chapter 9 focuses on STP concepts, with Chapter 10 focusing on configuration and verification—but with emphasis on interpreting STP behavior. It also discusses Layer 2 EtherChannel configuration.

# Part III

## Implementing VLANs and STP

**Chapter 8:** Implementing Ethernet Virtual LANs

**Chapter 9:** Spanning Tree Protocol Concepts

**Chapter 10:** RSTP and EtherChannel Configuration

**Part III Review**



CHAPTER 8

# Implementing Ethernet Virtual LANs

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.13 Describe switching concepts**
    - **1.13.a MAC learning and aging**
    - **1.13.b Frame switching**
    - **1.13.c Frame flooding**
    - **1.13.d MAC address table**
- **2.0 Network Access**
  - **2.1 Configure and verify VLANs (normal range) spanning multiple switches**
    - **2.1.a Access ports (data and voice)**
    - **2.1.b Default VLAN**
    - **2.1.c InterVLAN connectivity**
  - **2.2 Configure and verify interswitch connectivity**
    - **2.2.a Trunk ports**
    - **2.2.b 802.1Q**
    - **2.2.c Native VLAN**

So far in this book, you have learned that Ethernet switches receive Ethernet frames, make decisions, and then forward (switch) those Ethernet frames. That core logic revolves around MAC addresses, the interface in which the frame arrives, and the interfaces out which the switch forwards the frame.

While true, that logic omits any consideration of virtual LANs (VLANs). VLANs impact the switching logic for each frame because each VLAN acts as a subset of the switch ports in an Ethernet LAN. Switches believe each Ethernet frame to be received in an identifiable VLAN, forwarded based on MAC table entries for that VLAN, and forwarded out ports in that VLAN. This chapter explores those concepts and others related to VLANs.

As for the organization of the chapter, the first major section of the chapter explains the core concepts. These concepts include how VLANs work on a single switch, how to use VLAN trunking to create VLANs that span across multiple switches, and how to forward traffic between VLANs using a router. The second major section shows how to configure VLANs and VLAN trunks: how to statically assign interfaces to a VLAN. The final major section discusses some issues that can arise when using VLANs and trunks and how to avoid those issues.

Also, this chapter goes beyond the normal chapter page length. If you want to break up your study into two reading sections, stop the first session when you reach the heading "Implementing Interfaces Connected to Phones."

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 8-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Virtual LAN Concepts | 1–3 |
| VLAN and VLAN Trunking Configuration and Verification | 4–6 |
| Troubleshooting VLANs and VLAN Trunks | 7–8 |

1. In a LAN, which of the following terms best equates to the term *VLAN*?
   - **a.** Collision domain
   - **b.** Broadcast domain
   - **c.** Subnet
   - **d.** Single switch
   - **e.** Trunk
2. Imagine a small lab network with one LAN switch, with the ports configured to be in three different VLANs. How many IP subnets do you need for the hosts connected to the switch ports, assuming that all hosts in all VLANs want to use TCP/IP?
   - **a.** 0
   - **b.** 1
   - **c.** 2
   - **d.** 3
   - **e.** You cannot tell from the information provided.
3. Switch SW1 sends a frame to switch SW2 using 802.1Q trunking. Which of the answers describes how SW1 changes or adds to the Ethernet frame before forwarding the frame to SW2?
   - **a.** Inserts a 4-byte header and does change the MAC addresses.
   - **b.** Inserts a 4-byte header and does not change the MAC addresses.
   - **c.** Encapsulates the original frame behind an entirely new Ethernet header.
   - **d.** None of the other answers are correct.

4. You are told that switch 1 is configured with the **switchport mode dynamic auto** command on its Fa0/5 interface, which is connected to switch 2. Which of the following settings on the **switchport mode** command on switch 2 would allow trunking to work? (Choose two answers.)
   a. **trunk**
   b. **dynamic auto**
   c. **dynamic desirable**
   d. **access**
   e. None of the other answers are correct.
5. A switch has just arrived from Cisco. The switch has never been configured with any VLANs, but VTP has been disabled. An engineer configures the **vlan 22** and **name Hannahs-VLAN** commands and then exits configuration mode. Which of the following are true? (Choose two answers.)
   a. VLAN 22 is listed in the output of the **show vlan brief** command.
   b. VLAN 22 is listed in the output of the **show running-config** command.
   c. VLAN 22 is not created by this process.
   d. VLAN 22 does not exist in that switch until at least one interface is assigned to that VLAN.
6. Which of the following commands identify switch interfaces as being trunking interfaces—interfaces that currently operate as VLAN trunks? (Choose two answers.)
   a. **show interfaces**
   b. **show interfaces switchport**
   c. **show interfaces trunk**
   d. **show trunks**
7. In a switch that disables VTP, an engineer configures the commands **vlan 30** and **shutdown vlan 30**. Which answers should be true about this switch? (Choose two answers.)
   a. The **show vlan brief** command should list VLAN 30.
   b. The **show running-config** command should list VLAN 30.
   c. The switch should forward frames that arrive in access ports in VLAN 30.
   d. The switch should forward frames that arrive in trunk ports tagged with VLAN 30.
8. The **show interfaces g0/1 trunk** command provides three lists of VLAN IDs. Which items would limit the VLANs that appear in the first of the three lists of VLANs?
   a. A **shutdown vlan 30** global command
   b. A **switchport trunk allowed vlan** interface subcommand
   c. An STP choice to block on G0/1
   d. A **no vlan 30** global command

# Foundation Topics

## Virtual LAN Concepts

Before understanding **VLANs**, you must first have a specific understanding of the definition of a LAN. For example, from one perspective, a LAN includes all the user devices, servers, switches, routers, cables, and wireless access points in one location. However, an alternative narrower definition of a LAN can help in understanding the concept of a virtual LAN:

A LAN includes all devices in the same broadcast domain.

A broadcast domain includes the set of all LAN-connected devices, so that when any of the devices sends a broadcast frame, all the other devices get a copy of the frame. So, from one perspective, you can think of a LAN and a broadcast domain as being basically the same thing.

Using only default settings, a switch considers all its interfaces to be in the same broadcast domain. That is, for one switch, when a broadcast frame entered one switch port, the switch forwarded that broadcast frame out all other ports. With that logic, to create two different LAN broadcast domains, you had to buy two different Ethernet LAN switches, as shown in Figure 8-1.

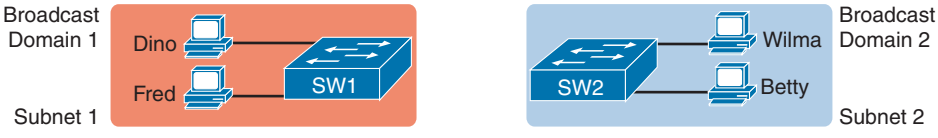



**Figure 8-1** *Creating Two Broadcast Domains with Two Physical Switches and No VLANs*

By using two VLANs, a single switch can accomplish the same goals of the design in Figure 8-1—to create two broadcast domains—with a single switch. With VLANs, a switch can configure some interfaces into one broadcast domain and some into another, creating multiple broadcast domains. These individual broadcast domains created by the switch are called *virtual LANs (VLANs)*.

For example, in Figure 8-2, the single switch creates two VLANs, treating the ports in each VLAN as being completely separate. The switch would never forward a frame sent by Dino (in VLAN 1) over to either Wilma or Betty (in VLAN 2).

Key Topic

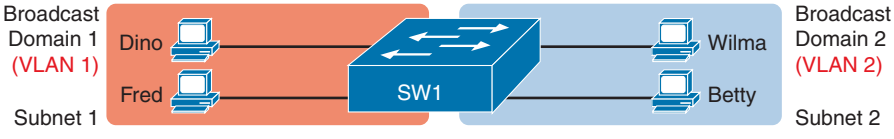



**Figure 8-2** *Creating Two Broadcast Domains Using One Switch and VLANs*

Designing campus LANs to use more VLANs, each with a smaller number of devices, often helps improve the LAN in many ways. For example, a broadcast sent by one host in a VLAN will be received and processed by all the other hosts in the VLAN—but not by hosts in a different VLAN. Limiting the number of hosts that receive a single broadcast frame reduces the number of hosts that waste effort processing unneeded broadcasts. It also reduces

security risks because fewer hosts see frames sent by any one host. These are just a few reasons for separating hosts into different VLANs. The following list summarizes the most common reasons for choosing to create smaller broadcast domains (VLANs):




- To reduce CPU overhead on each device, improving host performance, by reducing the number of devices that receive each broadcast frame
- To reduce security risks by reducing the number of hosts that receive copies of frames that the switches flood (broadcasts, multicasts, and unknown unicasts)
- To improve security for hosts through the application of different security policies per VLAN
- To create more flexible designs that group users by department, or by groups that work together, instead of by physical location
- To solve problems more quickly, because the failure domain for many problems is the same set of devices as those in the same broadcast domain
- To reduce the workload for the Spanning Tree Protocol (STP) by limiting a VLAN to a single access switch

The rest of this chapter looks closely at the mechanics of how VLANs work across multiple Cisco switches, including the required configuration. To that end, the next section examines VLAN trunking, a feature required when installing a VLAN that exists on more than one LAN switch.

## Creating Multiswitch VLANs Using Trunking

Configuring VLANs on a single switch requires only a little effort: you simply configure each port to tell it the VLAN number to which the port belongs. With multiple switches, you have to consider additional concepts about how to forward traffic between the switches.

When you are using VLANs in networks that have multiple interconnected switches, the switches need to use *VLAN trunking* on the links between the switches. VLAN trunking causes the switches to use a process called *VLAN tagging*, by which the sending switch adds another header to the frame before sending it over the **trunk**. This extra trunking header includes a *VLAN identifier* (VLAN ID) field so that the sending switch can associate the frame with a particular VLAN ID, and the receiving switch can then know in what VLAN each frame belongs.

Figure 8-3 shows an example that demonstrates VLANs that exist on multiple switches, but it does not use trunking. First, the design uses two VLANs: VLAN 10 and VLAN 20. Each switch has two ports assigned to each VLAN, so each VLAN exists in both switches. To forward traffic in VLAN 10 between the two switches, the design includes a link between switches, with that link fully inside VLAN 10. Likewise, to support VLAN 20 traffic between switches, the design uses a second link between switches, with that link inside VLAN 20.

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** D **3** B **4** A, C **5** A, B **6** B, C **7** A, B **8** B

The design in Figure 8-3 functions perfectly. For example, PC11 (in VLAN 10) can send a frame to PC14. The frame flows into SW1, over the top link (the one that is in VLAN 10) and over to SW2.

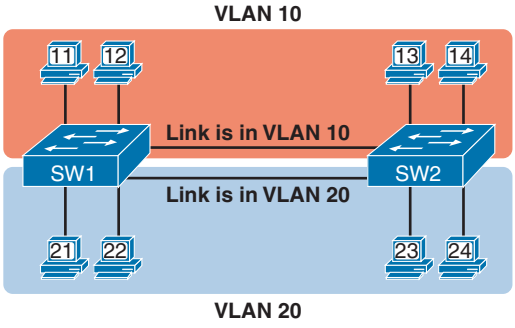



**Figure 8-3** *Multiswitch VLAN Without VLAN Trunking*

The design shown in Figure 8-3 works, but it simply does not scale very well. It requires one physical link between switches to support every VLAN. If a design needed 10 or 20 VLANs, you would need 10 or 20 links between switches, and you would use 10 or 20 switch ports (on each switch) for those links.

## VLAN Tagging Concepts

VLAN trunking creates one link between switches that supports as many VLANs as you need. As a VLAN trunk, the switches treat the link as if it were a part of all the VLANs. At the same time, the trunk keeps the VLAN traffic separate, so frames in VLAN 10 would not go to devices in VLAN 20, and vice versa, because each frame is identified by VLAN number as it crosses the trunk. Figure 8-4 shows the idea, with a single physical link between the two switches.

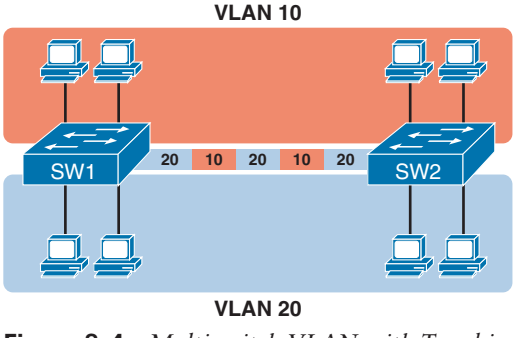



**Figure 8-4** *Multiswitch VLAN with Trunking*

The use of trunking allows switches to forward frames from multiple VLANs over a single physical connection by adding a small header to the Ethernet frame. For example, Figure 8-5 shows PC11 sending a broadcast frame on interface Fa0/1 at Step 1. To flood the frame, switch SW1 needs to forward the broadcast frame to switch SW2. However, SW1 needs to let SW2 know that the frame is part of VLAN 10, so that after the frame is received, SW2 will flood the frame only into VLAN 10, and not into VLAN 20. So, as shown at Step 2,

before sending the frame, SW1 adds a VLAN header to the original Ethernet frame, with the VLAN header listing a VLAN ID of 10 in this case.

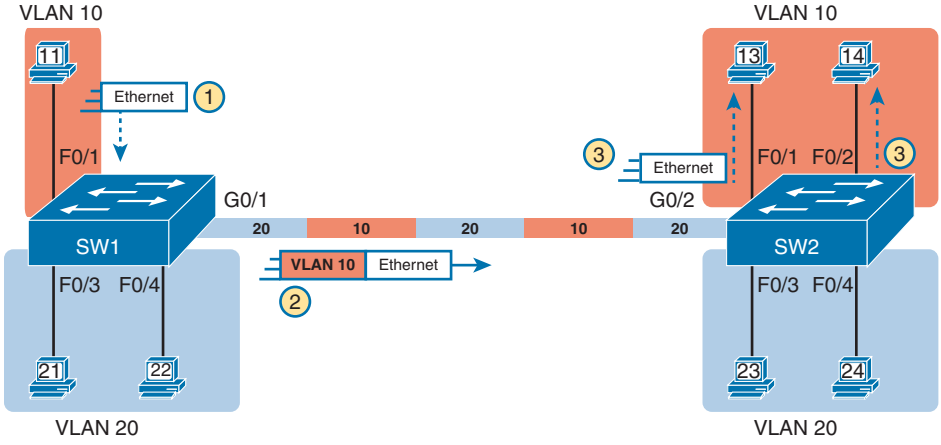



**Figure 8-5** *VLAN Trunking Between Two Switches*

When SW2 receives the frame, it understands that the frame is in VLAN 10. SW2 then removes the VLAN header, forwarding the original frame out its interfaces in VLAN 10 (Step 3).

For another example, consider the case when PC21 (in VLAN 20) sends a broadcast. SW1 sends the broadcast out port Fa0/4 (because that port is in VLAN 20) and out Gi0/1 (because it is a trunk, meaning that it supports multiple different VLANs). SW1 adds a trunking header to the frame, listing a VLAN ID of 20. SW2 strips off the trunking header after determining that the frame is part of VLAN 20, so SW2 knows to forward the frame out only ports Fa0/3 and Fa0/4, because they are in VLAN 20, and not out ports Fa0/1 and Fa0/2, because they are in VLAN 10.

## The 802.1Q and ISL VLAN Trunking Protocols

Cisco has supported two different trunking protocols over the years: Inter-Switch Link (ISL) and IEEE **802.1Q**. Cisco created the ISL years before 802.1Q, in part because the IEEE had not yet defined a VLAN trunking standard. Today, 802.1Q has become the more popular trunking protocol, with Cisco not even bothering to support ISL in many of its switch models today.

While both ISL and 802.1Q tag each frame with the VLAN ID, the details differ. 802.1Q inserts an extra 4-byte 802.1Q VLAN header into the original frame's Ethernet header, as shown at the top of Figure 8-6. As for the fields in the 802.1Q header, only the 12-bit VLAN ID field inside the 802.1Q header matters for topics discussed in this book. This 12-bit field supports a theoretical maximum of 212 (4096) VLANs, but in practice it supports a maximum of 4094. (Both 802.1Q and ISL use 12 bits to tag the VLAN ID, with two reserved values [0 and 4095].)

Cisco switches break the range of VLAN IDs (1–4094) into two ranges: the normal range and the extended range. All switches can use normal-range VLANs with values from 1 to 1005. Only some switches can use extended-range VLANs with VLAN IDs from 1006 to 4094. The rules for which switches can use extended-range VLANs depend on the configuration of the VLAN Trunking Protocol (**VTP**), which is discussed briefly in the section "VLAN Trunking Configuration," later in this chapter.

Key Topic

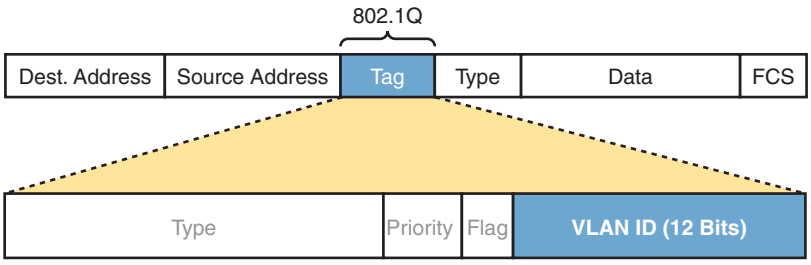



**Figure 8-6** *802.1Q Trunking*

802.1Q also defines one special VLAN ID on each trunk as the **native VLAN** (defaulting to use VLAN 1). In a normally working 802.1Q trunk, both endpoints use trunking, and both use the same native VLAN. Neither end, when sending a frame assigned to this native VLAN, adds the 802.1Q header. Both switches, knowing that untagged frames mean that the frame is part of the native VLAN, treat untagged frames as being part of the native VLAN.

The IEEE included the native VLAN concept for cases in which one device operates as a trunk while the other side does not. The nontrunking endpoint may do so temporarily or permanently (typically because of misconfiguration or a lack of 802.1Q support). In such cases, the nontrunking device will be confused and discard any received frames that contain a trunking header. However, when the trunking side sends frames as part of the native VLAN—untagged—the nontrunking side will understand the frame and consider it to be part of the access VLAN assigned to the interface. So, the native VLAN gives engineers a tool to allow for cases of making one VLAN work over the link, even when some trunking issue might exist.

## Forwarding Data Between VLANs

If you create a campus LAN that contains many VLANs, you typically still need all devices to be able to send data to all other devices. This next topic discusses some concepts about how to route data between those VLANs.

### The Need for Routing Between VLANs

LAN switches that forward data based on Layer 2 logic, as discussed so far in this book, often go by the name *Layer 2 switch*. For example, Chapter 5, "Analyzing Ethernet LAN Switching," discussed how LAN switches receive Ethernet frames (a Layer 2 concept), look at the destination Ethernet MAC address (a Layer 2 address), and forward the Ethernet frame out some other interface. All those concepts are defined by Layer 2 protocols, hence the name Layer 2 switch.

Layer 2 switches perform their logic per VLAN. For example, in Figure 8-7, the two PCs on the left sit in VLAN 10, in subnet 10. The two PCs on the right sit in a different VLAN (20), with a different subnet (20). Note that the figure repeats earlier Figure 8-2, but with the switch broken into halves, to emphasize the point that Layer 2 switches will not forward data between two VLANs.

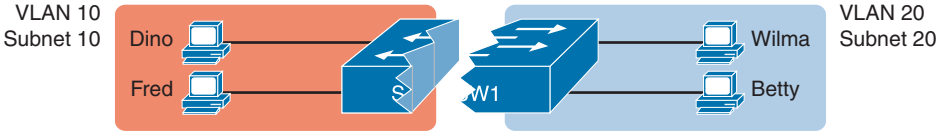



**Figure 8-7** *Layer 2 Switch Does Not Route Between the VLANs*

8

As shown in the figure, when configured with some ports in VLAN 10 and others in VLAN 20, the switch acts like two separate switches in which it will forward traffic. In fact, one goal of VLANs is to separate traffic in one VLAN from another, preventing frames in one VLAN from leaking over to other VLANs. For example, when Dino (in VLAN 10) sends any Ethernet frame, if SW1 is a Layer 2 switch, that switch will not forward the frame to the PCs on the right in VLAN 20.

### Routing Packets Between VLANs with a Router

When VLANs are included in a campus LAN design, the devices in a VLAN need to be in the same subnet. Following the same design logic, devices in different VLANs need to be in different subnets.

To forward packets between VLANs, the network must use a device that acts as a router. You can use an actual router or use a switch that can perform some functions like a router. These switches that also perform Layer 3 routing functions go by the name *multilayer switch* or **Layer 3 switch**. This section first discusses how to forward data between VLANs when using Layer 2 switches and ends with a brief discussion of how to use Layer 3 switches.

For example, Figure 8-8 shows a router that can route packets between subnets 10 and 20. The figure shows the same Layer 2 switch as shown in Figure 8-7, with the same perspective of the switch being split into parts with two different VLANs, and with the same PCs in the same VLANs and subnets. Now Router R1 has one LAN physical interface connected to the switch and assigned to VLAN 10, and a second physical interface connected to the switch and assigned to VLAN 20. With an interface connected to each subnet, the Layer 2 switch can keep doing its job—forwarding frames inside a VLAN, while the router can do its job—routing IP packets between the subnets.

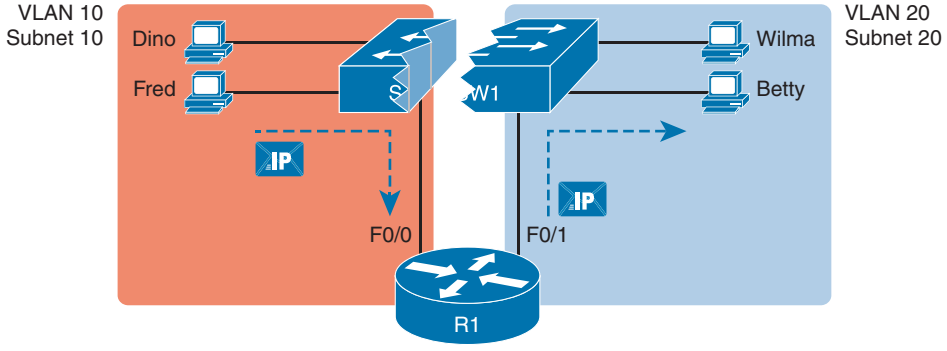



**Figure 8-8** *Routing Between Two VLANs on Two Physical Interfaces*

The figure shows an IP packet being routed from Fred, which sits in one VLAN/subnet, to Betty, which sits in the other. The Layer 2 switch forwards two different Layer 2 Ethernet frames: one in VLAN 10, from Fred to R1's F0/0 interface, and the other in VLAN 20, from R1's F0/1 interface to Betty. From a Layer 3 perspective, Fred sends the IP packet to its default router (R1), and R1 routes the packet out another interface (F0/1) into another subnet where Betty resides.

The design in Figure 8-8 works, but there are several different solutions for routing packets between VLANs. This chapter shows the option of using a separate physical router, with a separate link per VLAN, because it can be the easiest of the options to understand and

visualize. Chapter 18, "IP Routing in the LAN," works through those other features for routing packets between VLANs.

# VLAN and VLAN Trunking Configuration and Verification

Cisco switches do not require any configuration to work. You can purchase Cisco switches, install devices with the correct cabling, turn on the switches, and they work. You would never need to configure the switch, and it would work fine, even if you interconnected switches, until you needed more than one VLAN. But if you want to use VLANs—and most enterprise networks do—you need to add some configuration.

This chapter separates the VLAN configuration details into two major sections. The first section looks at how to configure *static access interfaces*—switch interfaces configured to be in one VLAN only, therefore not using VLAN trunking. The second part shows how to configure interfaces that do use VLAN trunking.

## Creating VLANs and Assigning Access VLANs to an Interface

This section shows how to create a VLAN, give the VLAN a name, and assign interfaces to a VLAN. To focus on these basic details, this section shows examples using a single switch, so VLAN trunking is not needed.

For a Cisco switch to forward frames in a particular VLAN, the switch must be configured to believe that the VLAN exists. In addition, the switch must have nontrunking interfaces (called **access interfaces** or **static access interfaces**) assigned to the VLAN and/or trunks that support the VLAN. The configuration steps for access interfaces are as follows:

Config Checklist

**Step 1.** To configure a new VLAN, follow these steps:

- **a.** From configuration mode, use the **vlan** *vlan-id* command in global configuration mode to create the VLAN and to move the user into VLAN configuration mode.
- **b.** (Optional) Use the **name** *name* command in VLAN configuration mode to list a name for the VLAN. If not configured, the VLAN name is VLAN*ZZZZ*, where *ZZZZ* is the four-digit decimal VLAN ID.

**Step 2.** For each access interface, follow these steps:

- **a.** Use the **interface** *type number* command in global configuration mode to move into interface configuration mode for each desired interface.
- **b.** Use the **switchport access vlan** *id-number* command in interface configuration mode to specify the VLAN number associated with that interface.
- **c.** (Optional) Use the **switchport mode access** command in interface configuration mode to make this port always operate in access mode (that is, to not trunk).

While the list might look a little daunting, the process on a single switch is actually pretty simple. For example, if you want to put the switch's ports in three VLANs—11, 12, and 13—you first add three **vlan** commands: **vlan 11**, **vlan 12**, and **vlan 13**. Then, for each interface, add a **switchport access vlan 11** (or **12** or **13**) command to assign that interface to the proper VLAN.

**NOTE** The term **default VLAN** (as shown in the exam topics) refers to the default setting on the **switchport access vlan** *vlan-id* command, and that default is VLAN ID 1. In other words, by default, each port is assigned to access VLAN 1.

### VLAN Configuration Example 1: Full VLAN Configuration

Examples 8-1, 8-2, and 8-3 work through one scenario with VLAN configuration and verification. To begin, Example 8-1 begins by showing the VLANs in switch SW1 in Figure 8-9, with all default settings related to VLANs.

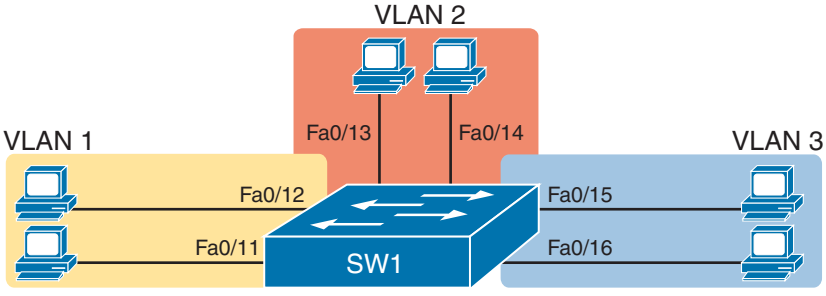



**Figure 8-9** *Network with One Switch and Three VLANs*

**Example 8-1** *Configuring VLANs and Assigning VLANs to Interfaces*

```
SW1# show vlan brief
VLAN Name                              Status     Ports
---- --------------------------------- --------- -------------------------------
1    default                           active    Fa0/1, Fa0/2, Fa0/3, Fa0/4
                                                 Fa0/5, Fa0/6, Fa0/7, Fa0/8
                                                 Fa0/9, Fa0/10, Fa0/11, Fa0/12
                                                 Fa0/13, Fa0/14, Fa0/15, Fa0/16
                                                 Fa0/17, Fa0/18, Fa0/19, Fa0/20
                                                 Fa0/21, Fa0/22, Fa0/23, Fa0/24
                                                 Gi0/1, Gi0/2
1002 fddi-default                      act/unsup
1003 token-ring-default                act/unsup
1004 fddinet-default                   act/unsup
1005 trnet-default                     act/unsup
```

The example begins with the **show vlan brief** command, confirming the default settings of five nondeletable VLANs, with all interfaces assigned to VLAN 1. VLAN 1 cannot be deleted but can be used. VLANs 1002–1005 cannot be deleted and cannot be used as access VLANs today. In particular, note that this switch has 24 Fast Ethernet ports (Fa0/1–Fa0/24) and two Gigabit Ethernet ports (Gi0/1 and Gi0/2), all of which are listed as being in VLAN 1

per that first command's output, confirming that by default, Cisco switches assign all ports to VLAN 1.

Next, Example 8-2 shows steps that mirror the VLAN configuration checklist, namely the configuration of VLAN 2, plus the assignment of VLAN 2 as the access VLAN on two ports: Fa0/13 and Fa0/14.

**Example 8-2** *Configuring VLANs and Assigning VLANs to Interfaces*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# vlan 2
SW1(config-vlan)# name Freds-vlan
SW1(config-vlan)# exit
SW1(config)# interface range fastethernet 0/13 - 14
SW1(config-if)# switchport access vlan 2
SW1(config-if)# switchport mode access
SW1(config-if)# end

SW1# show vlan brief

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/1, Fa0/2, Fa0/3, Fa0/4
                                                Fa0/5, Fa0/6, Fa0/7, Fa0/8
                                                Fa0/9, Fa0/10, Fa0/11, Fa0/12
                                                Fa0/15, Fa0/16, Fa0/17, Fa0/18
                                                Fa0/19, Fa0/20, Fa0/21, Fa0/22
                                                Fa0/23, Fa0/24, Gi0/1, Gi0/2
2    Freds-vlan                       active    Fa0/13, Fa0/14
1002 fddi-default                     act/unsup
1003 token-ring-default               act/unsup
1004 fddinet-default                  act/unsup
1005 trnet-default                    act/unsup
```

Take a moment to compare the output of the **show vlan brief** commands in Example 8-2 (after adding the configuration) versus Example 8-1. Example 8-2 shows new information about VLAN 2, with ports Fa0/13 and Fa0/14 no longer being listed with VLAN 1, but now listed as assigned to VLAN 2.

To complete this scenario, Example 8-3 shows a little more detail about the VLAN itself. First, the **show running-config** command lists both the **vlan 2** and **switchport access vlan 2** commands as configured in Example 8-2. Also, note that earlier Example 8-2 uses the **interface range** command, with one instance of the **switchport access vlan 2** interface subcommand. However, Example 8-3 shows how the switch actually applied that command to both Fa0/13 and Fa0/14. Example 8-3 ends with the **show vlan id 2** command, which confirms the operational status that ports Fa0/13 and Fa0/14 are assigned to VLAN 2.

**Example 8-3** *Configuring VLANs and Assigning VLANs to Interfaces*

```
SW1# show running-config
! Many lines omitted for brevity
! Early in the output:
vlan 2
 name Freds-vlan
!
! more lines omitted for brevity
interface FastEthernet0/13
 switchport access vlan 2
 switchport mode access
!
interface FastEthernet0/14
 switchport access vlan 2
 switchport mode access
!

SW1# show vlan id 2
VLAN Name                             Status     Ports
---- -------------------------------- --------- -------------------------------
2    Freds-vlan                       active     Fa0/13, Fa0/14

VLAN Type  SAID       MTU   Parent RingNo BridgeNo Stp  BrdgMode Trans1 Trans2
---- ----- ---------- ----- ------ ------ -------- ---- -------- ------ ------
2    enet  100010     1500  -      -      -        -    -        0      0

Remote SPAN VLAN
----------------
Disabled

Primary Secondary Type              Ports
------- --------- ----------------- ------------------------------------------
```

The example surrounding Figure 8-9 uses six switch ports, all of which need to operate as access ports. That is, each port should not use trunking but instead should be assigned to a single VLAN, as assigned by the **switchport access vlan** *vlan-id* command. For ports that should always act as access ports, add the optional interface subcommand **switchport mode access**. This command tells the switch to always be an access interface and disables the protocol that negotiates trunking (Dynamic Trunking Protocol [DTP]) with the device on the other end of the link. (The upcoming section "VLAN Trunking Configuration" discusses more details about the commands that allow a port to negotiate whether it should use trunking.)

**NOTE** The companion website for this book includes a video that works through a different VLAN configuration example.

### VLAN Configuration Example 2: Shorter VLAN Configuration

Example 8-2 shows how to configure a VLAN and add two ports to the VLAN as access ports. Example 8-4 does the same, this time with VLAN 3, and this time with a much briefer alternative configuration. The configuration completes the configuration of the design shown in Figure 8-9, by adding two ports to VLAN 3.

**Example 8-4** *Shorter VLAN Configuration Example (VLAN 3)*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface range Fastethernet 0/15 - 16
SW1(config-if-range)# switchport access vlan 3
% Access VLAN does not exist. Creating vlan 3
SW1(config-if-range)# ^Z

SW1# show vlan brief

VLAN Name                              Status     Ports
---- --------------------------------- --------- -------------------------------
1 default                              active      Fa0/1, Fa0/2, Fa0/3, Fa0/4
                                                   Fa0/5, Fa0/6, Fa0/7, Fa0/8
                                                   Fa0/9, Fa0/10, Fa0/11, Fa0/12
                                                   Fa0/17, Fa0/18, Fa0/19, Fa0/20
                                                   Fa0/21, Fa0/22, Fa0/23, Fa0/24
                                                   Gi0/1, Gi0/2
2 Freds-vlan                           active      Fa0/13, Fa0/14
3 VLAN0003                             active      Fa0/15, Fa0/16
1002 fddi-default                      act/unsup
1003 token-ring-default                act/unsup
1004 fddinet-default                   act/unsup
1005 trnet-default                     act/unsup
```

Example 8-4 shows how a switch can dynamically create a VLAN—the equivalent of the **vlan** *vlan-id* global config command—when the **switchport access vlan** interface subcommand refers to a currently unconfigured VLAN. This example begins with SW1 not knowing about VLAN 3. With the addition of the **switchport access vlan 3** interface subcommand, the switch realized that VLAN 3 did not exist, and as noted in the shaded message in the example, the switch created VLAN 3, using a default name (VLAN0003). The engineer did not need to type the **vlan 3** global command to create VLAN 3; the switch did that automatically. No other steps are required to create the VLAN. At the end of the process, VLAN 3 exists in the switch, and interfaces Fa0/15 and Fa0/16 are in VLAN 3, as noted in the shaded part of the **show vlan brief** command output.

## VLAN Trunking Protocol

Before showing more configuration examples, you also need to know something about a Cisco protocol and tool called the VLAN Trunking Protocol (VTP). VTP is a Cisco proprietary tool on Cisco switches that advertises each VLAN configured in one switch (with the **vlan** *number* command) so that all the other switches in the campus learn about that VLAN.

This book does not discuss VTP as an end to itself for a few different reasons. First, the current CCNA 200-301 exam blueprint ignores VTP, as do the CCNP Enterprise Core and CCNP Enterprise Advanced Routing blueprints. Additionally, many enterprises choose to disable VTP.

Also, you can easily disable VTP so that it has no impact on your switches in the lab, which is exactly what I did when building all the examples in this book.

However, VTP has some small impact on how every Cisco Catalyst switch works, even if you avoid using VTP. This brief section introduces enough details of VTP so that you can understand the impact of VTP in a Cisco Catalyst switch.

First, all examples in this book (and in Volume 2) use switches that disable VTP in some way. Interestingly, for much of VTP's decades of existence, most switches did support an option to completely disable VTP. Instead, to effectively disable VTP, the engineer would set the switch to use **VTP transparent mode** (with the **vtp mode transparent** global command). Many newer switches now have an option to disable VTP completely with the **vtp mode off** global command. For the purposes of this book, configuring a switch with either transparent mode or off mode disables VTP.

Note that both transparent and off modes prevent VTP from learning and advertising about VLAN configuration. Those modes allow a switch to configure all VLANs, including standard- and extended-range VLANs. Additionally, switches using transparent or off modes list the **vlan** configuration commands in the running-config file.

In contrast, switches in VTP server or client mode behave differently. Just in case you do lab exercises with real switches or with simulators, and you see unusual results with VLANs, check the VTP status with the **show vtp status** command. If your switch uses VTP server or client mode, you will find

- The server switches can configure VLANs in the standard range only (1–1005).
- The client switches cannot configure VLANs.
- Both servers and clients may be learning new VLANs from other switches and seeing their VLANs deleted by other switches because of VTP.
- The **show running-config** command does not list any **vlan** commands; you must use other **show** commands to find out about the configured VLANs.

If possible in the lab, disable VTP for your switch configuration practice until you decide to learn more about VTP for other purposes.

**NOTE** Do not change VTP settings on any switch that also connects to the production network until you know how VTP works and you talk with experienced colleagues. Doing so can cause real harm to your LAN. For example, if the switch you configure connects to other switches, which in turn connect to switches used in the production LAN, you could accidentally change the VLAN configuration in other switches with serious impact to the operation of the network. You could delete VLANs and cause outages. Be careful and never experiment with VTP settings on a switch unless it and the other switches connected to it have absolutely no physical links connected to the production LAN.

## VLAN Trunking Configuration

Trunking configuration between two Cisco switches can be simple if you just statically configure trunking. You could literally add one interface subcommand for the switch interface on each side of the link (**switchport mode trunk**), and you would create a VLAN trunk that supported all the VLANs known to each switch.

However, trunking configuration on Cisco switches includes many more options, including several options for dynamically negotiating various trunking settings. The configuration can either predefine different settings or tell the switch to negotiate the settings, as follows:

- **The type of trunking:** IEEE 802.1Q, ISL, or negotiate which one to use, on switches that support both types of trunking.
- **The administrative mode:** Whether to always trunk, always not trunk, or negotiate whether to trunk or not.

First, consider the type of trunking. For many years, Cisco has not bothered to include ISL support in new switch product families, preferring IEEE 802.1Q. On older switches that support both, use the **switchport trunk encapsulation** {**dot1q** | **isl** | **negotiate**} interface subcommand to either configure the type to use or to allow Dynamic Trunking Protocol (DTP) to negotiate the type (which prefers ISL if both support it).

DTP can also negotiate whether the two devices on the link agree to trunk at all, as guided by the local switch port's administrative mode. The **administrative mode** refers to the configuration setting for whether trunking should be used. Cisco switches use the **switchport mode** interface subcommand to define the administrative trunking mode, as listed in Table 8-2. The switch interface's **operational mode** shows whether the interface operates as a trunk or an access port, which depends on the configuration on both switches plus other factors.

Key Topic

**Table 8-2** Trunking Administrative Mode Options with the **switchport mode** Command

| Command Option | Description |
|---|---|
| **access** | Always act as an access (nontrunk) port |
| **trunk** | Always act as a trunk port |
| **dynamic desirable** | Initiates negotiation messages and responds to negotiation messages to dynamically choose whether to start using trunking |
| **dynamic auto** | Passively waits to receive trunk negotiation messages, at which point the switch will respond and negotiate whether to use trunking |

For example, consider the two switches shown in Figure 8-10. This figure expands the design shown earlier in Figure 8-9, with a trunk to a new switch (SW2) and with parts of VLANs 1 and 3 on ports attached to SW2. The two switches use a Gigabit Ethernet link for the trunk. In this case, the trunk does not dynamically form by default because both switches default to an administrative mode of *dynamic auto*, meaning that neither switch initiates the trunk negotiation process. When one switch is changed to use *dynamic desirable* mode, which does initiate the negotiation, the switches negotiate to use trunking, specifically 802.1Q because the switches support only 802.1Q.

Example 8-5 begins with SW1 configured as shown in Examples 8-2 and 8-4; that is, SW1 has two ports each assigned to VLANs 1, 2, and 3. However, both SW1 and SW2 currently

have all default settings on the interfaces that connect the two switches. With the default setting of **switchport mode dynamic auto**, the two switches do not trunk.

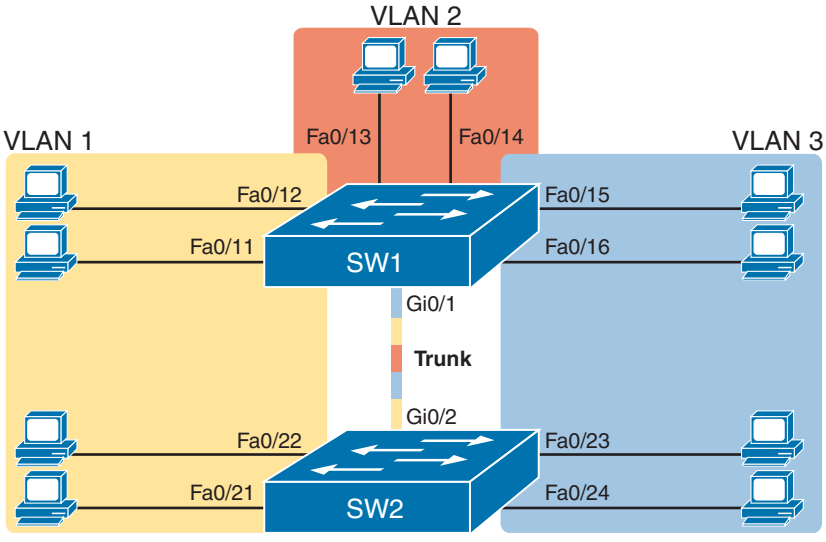



**Figure 8-10** *Network with Two Switches and Three VLANs*

**Example 8-5** *Initial (Default) State: Not Trunking Between SW1 and SW2*

```
SW1# show interfaces gigabit 0/1 switchport
Name: Gi0/1
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk private VLANs: none
Operational private-vlan: none
```

```
Trunking VLANs Enabled: ALL
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Protected: false
Unknown unicast blocked: disabled
Unknown multicast blocked: disabled
Appliance trust: none

! Note that the next command results in a single empty line of output.
SW1# show interfaces trunk
SW1#
```

First, focus on the highlighted items from the output of the **show interfaces switchport** command at the beginning of Example 8-5. The output lists the default administrative mode setting of dynamic auto. Because SW2 also defaults to dynamic auto, the command lists SW1's operational status as "access," meaning that it is not trunking. ("Dynamic auto" tells both switches to sit there and wait on the other switch to start the negotiations.) The third shaded line points out the only supported type of trunking (802.1Q). (On a switch that supports both ISL and 802.1Q, this value would by default list "negotiate," to mean that the type of encapsulation is negotiated.) Finally, the operational trunking type is listed as "native," which is a reference to the 802.1Q native VLAN.

The end of the example shows the output of the **show interfaces trunk** command, but with no output. This command lists information about all interfaces that currently operationally trunk; that is, it lists interfaces that currently use VLAN trunking. With no interfaces listed, this command also confirms that the link between switches is not trunking.

Next, consider Example 8-6, which shows the new configuration that enables trunking. In this case, SW1 is configured with the **switchport mode dynamic desirable** command, which asks the switch to both negotiate as well as to begin the negotiation process, rather than waiting on the other device. The example shows that as soon as the command is issued, log messages appear showing that the interface goes down and then back up again, which happens when the interface transitions from access mode to trunk mode.



**Example 8-6** *SW1 Changes from Dynamic Auto to Dynamic Desirable*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface gigabit 0/1
SW1(config-if)# switchport mode dynamic desirable
SW1(config-if)# ^Z
SW1#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to
down
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to
up
SW1# show interfaces gigabit 0/1 switchport
```

```
Name: Gi0/1
Switchport: Enabled
Administrative Mode: dynamic desirable
Operational Mode: trunk
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: dot1q
Negotiation of Trunking: On
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)
! lines omitted for brevity
```

Example 8-6 repeats the **show interfaces gi0/1 switchport** command seen in Example 8-5, but after configuring VLAN trunking, so this time the output shows that SW1's G0/1 interface now operates as a trunk. Note that the command still lists the administrative settings, which denote the configured values along with the operational settings, which list what the switch is currently doing. SW1 now claims to be in an operational mode of *trunk*, with an operational trunking encapsulation of dot1Q.

Example 8-7 now repeats the same **show interfaces trunk** command that showed no output at all back in Example 8-5. Now that SW1 trunks on its G0/1 port, the output in Example 8-7 lists G0/1, confirming that G0/1 is now operationally trunking. The next section discusses the meaning of the output of this command. Also, note that the end of the example repeats the **show vlan id 2** command; of note, it includes the trunk port G0/1 in the output because the trunk port can forward traffic in VLAN 2.

**Example 8-7** *A Closer Look at SW1's G0/1 Trunk Port*

```
SW1# show interfaces trunk

Port        Mode             Encapsulation  Status        Native vlan
Gi0/1       desirable        802.1q         trunking      1

Port        Vlans allowed on trunk
Gi0/1       1-4094

Port        Vlans allowed and active in management domain
Gi0/1       1-3

Port        Vlans in spanning tree forwarding state and not pruned
Gi0/1       1-3

SW1# show vlan id 2
VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
2    Freds-vlan                       active    Fa0/13, Fa0/14, G0/1
```

```
VLAN Type  SAID       MTU   Parent RingNo BridgeNo Stp  BrdgMode Trans1 Trans2
---- ----- ---------- ----- ------ ------ -------- ---- -------- ------ ------
2    enet  100010     1500  -      -      -        -    -        0      0

Remote SPAN VLAN
----------------

Disabled
Primary Secondary Type              Ports
------- --------- ----------------- ------------------------------------------
```

For the exams, you should be ready to interpret the output of the **show interfaces switchport** command, realize the administrative mode implied by the output, and know whether the link should operationally trunk based on those settings. Table 8-3 lists the combinations of the trunking administrative modes and the expected operational mode (trunk or access) resulting from the configured settings. The table lists the administrative mode used on one end of the link on the left and the administrative mode on the switch on the other end of the link across the top of the table.

Key Topic

**Table 8-3** Expected Trunking Operational Mode Based on the Configured Administrative Modes

| Administrative Mode | Access | Dynamic Auto | Trunk | Dynamic Desirable |
|---|---|---|---|---|
| **access** | Access | Access | Do Not Use* | Access |
| **dynamic auto** | Access | Access | Trunk | Trunk |
| **trunk** | Do Not Use* | Trunk | Trunk | Trunk |
| **dynamic desirable** | Access | Trunk | Trunk | Trunk |

* When two switches configure a mode of "access" on one end and "trunk" on the other, problems occur. Avoid this combination.

Finally, before we leave the discussion of configuring trunks, Cisco recommends disabling trunk negotiation on most ports for better security. The majority of switch ports on most switches will be used to connect to users and configured with the command **switchport mode access**, which also disables DTP. For ports without the **switchport mode access** command—for instance, ports statically configured to trunk with the **switchport mode trunk** command—DTP still operates, but you can disable DTP negotiations altogether using the **switchport nonegotiate** interface subcommand.

## Implementing Interfaces Connected to Phones

This next topic is strange, at least in the context of access links and trunk links. In the world of IP telephony, telephones use Ethernet ports to connect to an Ethernet network so they can use IP to send and receive voice traffic sent via IP packets. To make that work, the switch's Ethernet port acts like an access port, but at the same time, the port acts like a trunk in some ways. This last topic of the chapter works through those main concepts.

## Data and Voice VLAN Concepts

Before IP telephony, a PC could sit on the same desk as a phone. The phone happened to use UTP cabling, with that phone connected to some voice device (often called a *voice switch* or a *private branch exchange [PBX]*). The PC, of course, connected using a unshielded twisted-pair (UTP) cable to the usual LAN switch that sat in the wiring closet—sometimes in the same wiring closet as the voice switch. Figure 8-11 shows the idea.

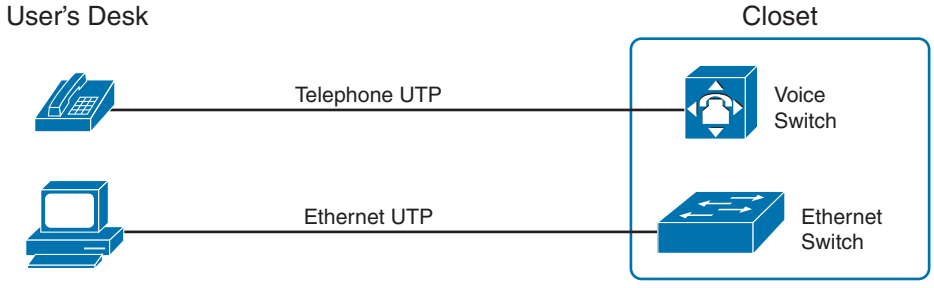



**Figure 8-11** *Before IP Telephony: PC and Phone, One Cable Each, Connect to Two Different Devices*

The term *IP telephony* refers to the branch of networking in which the telephones use IP packets to send and receive voice as represented by the bits in the data portion of the IP packet. The phones connect to the network like most other end-user devices, using either Ethernet or Wi-Fi. These new IP phones did not connect via cable directly to a voice switch, instead connecting to the IP network using an Ethernet cable and an Ethernet port built in to the phone. The phones then communicated over the IP network with software that replaced the call setup and other functions of the PBX. (The current products from Cisco that perform this IP telephony control function are called *Cisco Unified Communication Manager*.)

The migration from using the already-installed telephone cabling to these new IP phones that needed UTP cables that supported Ethernet caused some problems in some offices. In particular:

- The older non-IP phones used a category of UTP cabling that often did not support 100-Mbps or 1000-Mbps Ethernet.
- Most offices had a single UTP cable running from the wiring closet to each desk, but now two devices (the PC and the new IP phone) both needed a cable from the desktop to the wiring closet.
- Installing a new cable to every desk would be expensive; plus you would need more switch ports.

To solve this problem, Cisco embedded small three-port switches into each phone.

IP telephones have included a small LAN switch, on the underside of the phone, since the earliest IP telephone products. Figure 8-12 shows the basic cabling, with the wiring closet cable connecting to one physical port on the embedded switch, the PC connecting with a short patch cable to the other physical port, and the phone's internal CPU connecting to an internal switch port.

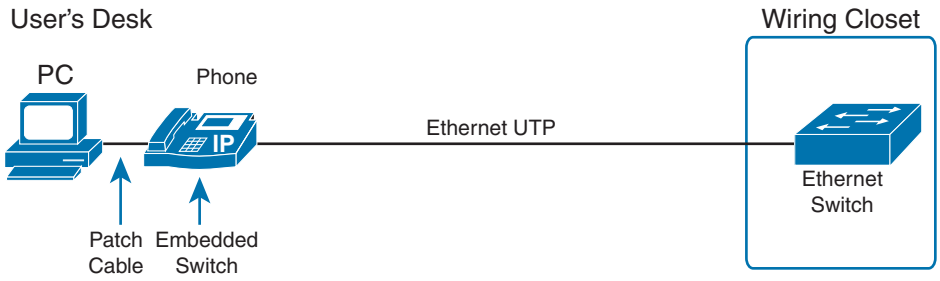



**Figure 8-12** *Cabling with an IP Phone, a Single Cable, and an Integrated Switch*

Sites that use IP telephony, which includes most every company today, now have two devices off each access port. In addition, Cisco best practices for IP telephony design tell us to put the phones in one VLAN and the PCs in a different VLAN. To make that happen, the switch port acts a little like an access link (for the PC's traffic) and a little like a trunk (for the phone's traffic). The configuration defines two VLANs on that port, as follows:

Key Topic

**Data VLAN:** Same idea and configuration as the access VLAN on an access port but defined as the VLAN on that link for forwarding the traffic for the device connected to the phone on the desk (typically the user's PC).

**Voice VLAN:** The VLAN defined on the link for forwarding the phone's traffic. Traffic in this VLAN is typically tagged with an 802.1Q header.

Figure 8-13 illustrates this design with two VLANs on access ports that support IP telephones.

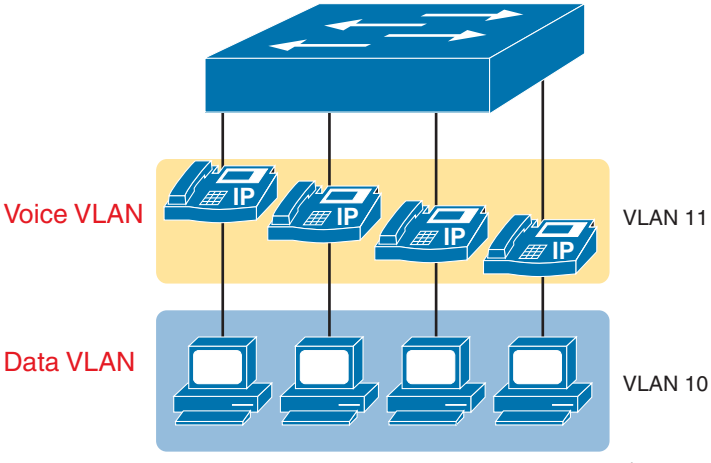



**Figure 8-13** *A LAN Design, with Data in VLAN 10 and Phones in VLAN 11*

## Data and Voice VLAN Configuration and Verification

Configuring a switch port to support IP phones, once you know the planned voice and data VLAN IDs, requires just a few easy commands. Making sense of the **show** commands once it is configured, however, can be a challenge. The port acts like an access port in many ways. However, with most configuration options, the voice frames flow with an 802.1Q header so

that the link supports frames in both VLANs on the link. But that makes for some different **show** command output.

Example 8-8 shows an example configuration. In this case, all four switch ports F0/1–F0/4 begin with a default configuration. The configuration adds the new data and **voice VLANs**. The example then configures all four ports as access ports and defines the access VLAN, which is also called the **data VLAN** when discussing IP telephony. Finally, the configuration includes the **switchport voice vlan 11** command, which defines the voice VLAN used on the port. The example matches Figure 8-13, using ports F0/1–F0/4.

**Example 8-8** *Configuring the Voice and Data VLAN on Ports Connected to Phones*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# vlan 10
SW1(config-vlan)# vlan 11
SW1(config-vlan)# interface range FastEthernet0/1 - 4
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 10
SW1(config-if)# switchport voice vlan 11
SW1(config-if)#^Z
SW1#
```

**NOTE** Cisco phones exchange configuration data with the switch using the Cisco Discovery Protocol (CDP) or Link-Layer Discovery Protocol (LLDP). Switches enable CDP by default, so the configuration shows no specific configuration commands for it. *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, Chapter 13, "Device Management Protocols," discusses CDP concepts and configuration.

The following list details the configuration steps for easier review and study:

Config Checklist

**Step 1.** Use the **vlan** *vlan-id* command in global configuration mode to create the data and voice VLANs if they do not already exist on the switch.

**Step 2.** Configure the data VLAN like an access VLAN, as usual:

- **a.** Use the **interface** *type number* command in global configuration mode to move into interface configuration mode.
- **b.** Use the **switchport access vlan** *id-number* command in interface configuration mode to define the data VLAN.
- **c.** Use the **switchport mode access** command in interface configuration mode to make this port always operate in access mode (that is, to not trunk).

**Step 3.** Use the **switchport voice vlan** *id-number* command in interface configuration mode to set the voice VLAN ID.

Verifying the status of a switch port configured like Example 8-8 shows some different output compared to the pure access port and pure trunk port configurations seen earlier in

this chapter. For example, the **show interfaces switchport** command shows details about the operation of an interface, including many details about access ports. Example 8-9 shows those details for port F0/4 after the configuration in Example 8-8 was added.

**Example 8-9** *Verifying the Data VLAN (Access VLAN) and Voice VLAN*

```
SW1# show interfaces FastEthernet 0/4 switchport
Name: Fa0/4
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
Negotiation of Trunking: Off
Access Mode VLAN: 10 (VLAN0010)
Trunking Native Mode VLAN: 1 (default)
Administrative Native VLAN tagging: enabled
Voice VLAN: 11 (VLAN0011)
! The rest of the output is omitted for brevity
```

Working through the first three highlighted lines in the output, all those details should look familiar for any access port. The **switchport mode access** configuration command statically configures the administrative mode to be an access port, so the port of course operates as an access port. Also, as shown in the third highlighted line, the **switchport access vlan 10** configuration command defined the access mode VLAN as highlighted here.

The fourth highlighted line shows the one small new piece of information: the voice VLAN ID, as set with the **switchport voice vlan 11** command in this case. This small line of output is the only piece of information in the output that differs from the earlier access port examples in this chapter.

These ports act more like access ports than trunk ports. In fact, the **show interfaces** *type number* **switchport** command boldly proclaims, "Operational Mode: static access." However, one other **show** command reveals just a little more about the underlying operation with 802.1Q tagging for the voice frames.

As mentioned earlier, the **show interfaces trunk** command—that is, the command that does not include a specific interface in the middle of the command—lists the operational trunks on a switch. With IP telephony ports, the ports do not show up in the list of trunks either—providing evidence that these links are *not* treated as trunks. Example 8-10 shows just such an example.

However, the **show interfaces trunk** command with the interface listed in the middle of the command, as is also shown in Example 8-10, does list some additional information. Note that in this case, the **show interfaces F0/4 trunk** command lists the status as not-trunking, but with VLANs 10 and 11 allowed on the trunk. (Normally, on an access port, only the access VLAN is listed in the "VLANs allowed on the trunk" list in the output of this command.)

**Example 8-10** *Allowed VLAN List and the List of Active VLANs*

```
SW1# show interfaces trunk
SW1# show interfaces F0/4 trunk

Port        Mode             Encapsulation  Status        Native vlan
Fa0/4       off              802.1q         not-trunking  1

Port        Vlans allowed on trunk
Fa0/4       10-11

Port        Vlans allowed and active in management domain
Fa0/4       10-11

Port        Vlans in spanning tree forwarding state and not pruned
Fa0/4       10-11
```

### Summary: IP Telephony Ports on Switches

It might seem as though this short topic about IP telephony and switch configuration includes a lot of small twists and turns and trivia, and it does. The most important items to remember are as follows:




- Configure these ports like a normal access port to begin: Configure it as a static access port and assign it an access VLAN.
- Add one more command to define the voice VLAN (**switchport voice vlan** *vlan-id*).
- Look for the mention of the voice VLAN ID, but no other new facts, in the output of the **show interfaces** *type number* **switchport** command.
- Look for both the voice and data (access) VLAN IDs in the output of the **show interfaces** *type number* **trunk** command.
- Do not expect to see the port listed in the list of operational trunks as listed by the **show interfaces trunk** command.

## Troubleshooting VLANs and VLAN Trunks

A switch's data plane forwarding processes depend in part on VLANs and VLAN trunking. This final section of the chapter focuses on issues related to VLANs and VLAN trunks that could prevent LAN switching from working properly, focusing on a few items not yet discussed in the chapter. In particular, this section examines these steps an engineer can take to avoid issues:

**Step 1.** Confirm that the correct access VLANs have been assigned.

**Step 2.** Confirm that all VLANs are both defined and active.

**Step 3.** Check the allowed VLAN lists on both ends of each trunk to ensure that all VLANs intended to be used are included.

**Step 4.** Check for incorrect trunk configuration settings that result in one switch operating as a trunk, with the neighboring switch not operating as a trunk.

**Step 5.** Check the native VLAN settings on both ends of the trunk to ensure the settings match.

## Confirm the Correct Access VLAN Is Assigned

To ensure that each access interface has been assigned to the correct VLAN, engineers need to confirm an interface operates as an access interface (as opposed to a **trunk interface**), determine the access VLAN assigned to the access interface, and compare the information to the documentation. The **show** commands listed in Table 8-4 can be particularly helpful in this process.

Key Topic

**Table 8-4** Commands That Can Find Access Ports and VLANs

| EXEC Command | Description |
|---|---|
| **show vlan brief**<br>**show vlan** | Lists each VLAN and all interfaces assigned to that VLAN (but does not include operational trunks) |
| **show vlan id** *num* | Lists both access and trunk ports in the VLAN |
| **show interfaces status** | On ports operating as access ports, it lists the access VLAN, and on ports operating as trunk ports, it lists the word *trunk*. |
| **show interfaces** *type number* **switchport** | Identifies the interface's access VLAN and voice VLAN, plus the configured and operational mode (access or trunk) |
| **show mac address-table** | Lists MAC table entries, including the associated VLAN |

If possible, start this step with the **show vlan** and **show vlan brief** commands. Both commands list all known VLANs and all access interfaces assigned to each VLAN, whether the interface is in a working or nonworking state. Be aware, however, that these two commands do not list operational trunks, so if you want to also see the trunk ports that support a VLAN, use the **show vlan id** *number* command.

After you determine the access interfaces and associated VLANs, if the interface is assigned to the wrong VLAN, use the **switchport access vlan** *vlan-id* interface subcommand to assign the correct VLAN ID.

8

## Access VLANs Undefined or Disabled

Switches do not forward frames for VLANs that are (a) not known because the VLAN is not configured or has not been learned with VTP or (b) the VLAN is known but is disabled (shut down). This next topic summarizes the best ways to confirm that a switch knows that a particular VLAN exists, and if it exists, determines the shutdown state of the VLAN.

First, on the issue of whether a VLAN exists on a switch, a VLAN can be defined to a switch in two ways: using the **vlan** *number* global configuration command, or it can be learned from another switch using VTP. As mentioned earlier in this chapter, the examples in this book assume that you are not using VTP. If you discover that a VLAN does not exist on a switch, simply configure the VLAN as discussed earlier in the section, "Creating VLANs and Assigning Access VLANs to an Interface."

In addition to checking the configuration, you can check for the status of the VLAN (as well as whether it is known to the switch) using the **show vlan** command. No matter the VTP

mode, this command will list all VLANs known to the switch, plus one of two VLAN state values, depending on the current state: either *active* or *act/lshut*. The second of these states means that the VLAN is shut down. Shutting down a VLAN disables the VLAN on that switch only, so *the switch will not forward frames in that VLAN*.

Switch IOS gives you two similar configuration methods with which to disable (**shutdown**) and enable (**no shutdown**) a VLAN. Example 8-11 shows how, first by using the global command **[no] shutdown vlan** *number* and then using the VLAN mode subcommand **[no] shutdown**. The example shows the global commands enabling and disabling VLANs 10 and 20, respectively, and using VLAN subcommands to enable and disable VLANs 30 and 40, respectively. The **show vlan brief** command at the end of the example confirms the shutdown (act/lshut) state of VLANs 10 and 30.

**Example 8-11** *Enabling and Disabling VLANs on a Switch*

```
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# no shutdown vlan 10
SW1(config)# shutdown vlan 20
SW1(config)# vlan 30
SW1(config-vlan)# no shutdown
SW1(config-vlan)# vlan 40
SW1(config-vlan)# shutdown
SW2(config-vlan)# end

SW1# show vlan brief

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Gi1/0/1, Gi1/0/2, Gi1/0/3
                                                Gi1/0/4, Gi1/0/5, Gi1/0/6
                                                Gi1/0/7, Gi1/0/8, Gi1/0/9
                                                Gi1/0/10, Gi1/0/11, Gi1/0/12
                                                Gi1/0/13, Gi1/0/14, Gi1/0/15
                                                Gi1/0/16, Gi1/0/17, Gi1/0/18
                                                Gi1/0/19, Gi1/0/20, Gi1/0/21
                                                Gi1/0/22, Gi1/0/23, Gi1/0/24
                                                Te1/1/1, Te1/1/2, Te1/1/3
                                                Te1/1/4
10   VLAN0010                         act/lshut
20   VLAN0020                         active
30   VLAN0030                         act/lshut
40   VLAN0040                         active
1002 fddi-default                     act/unsup
1003 token-ring-default               act/unsup
1004 fddinet-default                  act/unsup
1005 trnet-default                    act/unsup
```

**NOTE** The output of the **show vlan brief** command also lists a state of "act/unsup" for the reserved VLAN IDs 1002–1005, with "unsup" meaning "unsupported."

Example 8-12 shows another way to find the access VLANs on different ports by using the **show interfaces status** command. The switch has been configured before gathering the output in Example 8-12. First, the switch configures the first three ports as access ports in VLAN 10 and the next three as access ports in VLAN 20. The switch also configures two TenGigabitEthernet interfaces as trunks; note the word *trunk* under the VLAN heading for those interfaces.

**Example 8-12** *Displaying Access Port VLANs with* **show interfaces status**

```
SW1# show interfaces status

Port      Name               Status       Vlan       Duplex  Speed Type
Gi1/0/1                      connected    10         a-full a-1000 10/100/1000BaseTX
Gi1/0/2                      connected    10         a-full a-1000 10/100/1000BaseTX
Gi1/0/3                      connected    10         a-full a-1000 10/100/1000BaseTX
Gi1/0/4                      connected    20         a-full a-1000 10/100/1000BaseTX
Gi1/0/5                      connected    20         a-full a-1000 10/100/1000BaseTX
Gi1/0/6                      connected    20         a-full a-1000 10/100/1000BaseTX
! Lines For G1/0/7 - G1/0/24 omitted for brevity
Te1/1/1                      connected    trunk        full    10G SFP-10Gbase-SR
Te1/1/2                      connected    trunk        full    10G SFP-10Gbase-SR
Te1/1/3                      notconnect   1            auto   auto unknown
Te1/1/4                      notconnect   1            auto   auto unknown
```

## Mismatched Trunking Operational States

Trunking can be configured correctly so that both switches use trunking. However, trunks can also be misconfigured, with a couple of different results: either both switches do not trunk, or one switch trunks and the other does not. Both results cause problems.

The most common incorrect configuration—which results in both switches not trunking—is a configuration that uses the **switchport mode dynamic auto** command on both switches on the link. The word *auto* just makes us all want to think that the link would trunk automatically, but this command is both automatic and passive. As a result, both switches passively wait on the other device on the link to begin negotiations. Example 8-13 highlights those parts of the output from the **show interfaces switchport** command that confirm both the configured and operational states. Note that the output lists the operational mode as "static access" rather than "trunking."

**Example 8-13** *Operational Trunking State*

```
SW1# show interfaces TenGigabitEthernet1/1/1 switchport
Name: Te1/1/1
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
! lines omitted for brevity
```

A different incorrect trunking configuration has an even worse result: one switch trunks, sending tagged frames, but the neighboring switch does not trunk, so the neighboring switch discards any frames it receives that have a VLAN tag in the header. When this combination of events happens, the interface works in that the status on each end will be up/up or connected. Traffic in the native VLAN will actually cross the link successfully because those frames have no VLAN tags (headers). However, traffic in all the rest of the VLANs will not cross the link.

Figure 8-14 shows the incorrect configuration along with which side trunks and which does not. The side that trunks (SW1 in this case) enables trunking using the command **switchport mode trunk** but also disables Dynamic Trunking Protocol (DTP) negotiations using the **switchport nonegotiate** command. SW2's configuration also helps create the problem, by using one of the two trunking options that rely on DTP. Because SW1 has disabled DTP, SW2's DTP negotiations fail, and SW2 chooses to not trunk.

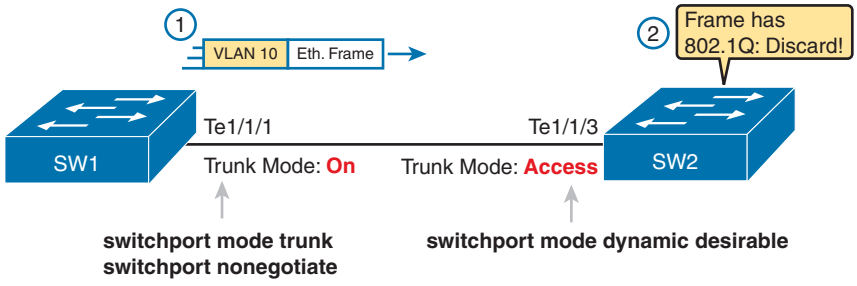



**Figure 8-14** *Mismatched Trunking Operational States*

The figure shows what happens when using this incorrect configuration. At Step 1, SW1 could (for example) forward a frame in VLAN 10. However, SW2 would view any frame that arrives with an 802.1Q header as illegal because the frame has an 802.1Q header, and SW2 treats its G0/2 port as an access port. So, SW2 discards any 802.1Q frames received on that port.

The trunking issues shown here can be easily avoided by checking the configuration and by checking the trunk's operational state (mode) on both sides of the trunk. The best commands to check trunking-related facts are **show interfaces trunk** and **show interfaces switchport**. Just be aware that the switches do not prevent you from making these configuration mistakes.

## The Supported VLAN List on Trunks

A Cisco switch can forward traffic for all defined and active VLANs. However, a particular VLAN trunk may not forward traffic for a defined and active VLAN for a variety of other reasons. You should learn how to identify which VLANs a particular trunk port currently supports and the reasons why the switch might not be forwarding frames for a VLAN on that trunk port.

The first category in this step can be easily done using the **show interfaces** *interface-id* **trunk** command, which lists information only about currently operational trunks. The best place to begin with this command is the last section of output, which lists the VLANs whose traffic will be forwarded over the trunk. Any VLANs that make it to this final list of VLANs in the command output meet the following criteria:

- The VLAN has not been removed from the *allowed VLAN list* on the trunk (as configured with the **switchport trunk allowed vlan** interface subcommand).
- The VLAN exists and is active on the local switch (as seen in the **show vlan** command).
- The VLAN has not been VTP-pruned from the trunk because the switch has disabled VTP. The trunk is in an STP forwarding state in that VLAN (as also seen in the **show spanning-tree vlan** *vlan-id* command).

The **switchport trunk allowed vlan** interface subcommand gives the network engineer a method to administratively limit the VLANs whose traffic uses a trunk. If the engineer wants all defined VLANs to be supported on a trunk, the engineer simply does not configure this command. If the engineer would like to limit the trunk to support a subset of the VLANs known to the switch, however, the engineer can add one or more **switchport trunk allowed vlan** interface subcommands.

For instance, in a switch that has configured VLANs 1 through 100, but no others, by default the switch would allow traffic in all 100 VLANs. However, the trunk interface command **switchport trunk allowed vlan 1-60** would limit the trunk to forward traffic for VLANs 1 through 60, but not the rest of the VLANs. Example 8-14 shows a sample of the command output from the **show interfaces trunk** command, which confirms the first list of VLAN IDs now lists VLANs 1–60. Without the **switchport trunk allowed vlan** command, the first list would have included VLANs 1–4094.

**Example 8-14** *Allowed VLAN List and List of Active VLANs*

```
SW1# show interfaces trunk

Port        Mode         Encapsulation   Status       Native vlan
Te1/1/1     desirable    802.1q          trunking     1

Port        Vlans allowed on trunk
Te1/1/1     1-60

Port        Vlans allowed and active in management domain
Te1/1/1     1-59

Port        Vlans in spanning tree forwarding state and not pruned
Te1/1/1     1-58
```

The output of the **show interfaces trunk** command creates three separate lists of VLANs, each under a separate heading. These three lists show a progression of reasons why a VLAN is not forwarded over a trunk. Table 8-5 summarizes the headings that precede each list and the reasons why a switch chooses to include or not include a VLAN in each list. For instance, in Example 8-14, VLAN 60 has been shut down, and VLAN 59 happens to be in an STP blocking state. (Chapter 9, "Spanning Tree Protocol Concepts," has more information about STP.)

Key Topic

**Table 8-5** VLAN Lists in the **show interfaces trunk** Command

| List Position | Heading | Reasons |
|---|---|---|
| First | VLANs allowed | VLANs 1–4094, minus those removed by the **switchport trunk allowed** command |
| Second | VLANs allowed and active… | The first list, minus VLANs not defined to the local switch (that is, there is not a **vlan** global configuration command or the switch has not learned of the VLAN with VTP), and also minus those VLANs in shutdown mode |
| Third | VLANs in spanning tree… | The second list, minus VLANs in an STP blocking state for that interface, and minus VLANs VTP pruned from that trunk |

### Mismatched Native VLAN on a Trunk

Unfortunately, it *is* possible to set the native VLAN ID to different VLANs on either end of the trunk, using the **switchport trunk native vlan** *vlan-id* command. If the native VLANs differ according to the two neighboring switches, the switches will cause frames sent in the native VLAN to jump from one VLAN to the other.

For example, if switch SW1 sends a frame using native VLAN 1 on an 802.1Q trunk, SW1 does not add a VLAN header, as is normal for the native VLAN. When switch SW2 receives the frame, noticing that no 802.1Q header exists, SW2 assumes that the frame is part of SW2's configured native VLAN. If SW2 has been configured to think VLAN 2 is the native VLAN on that trunk, SW2 will try to forward the received frame into VLAN 2. (This effect of a frame being sent in one VLAN but then being believed to be in a different VLAN is called *VLAN hopping*.)

## Chapter Review

Review this chapter's material using either the tools in the book or the interactive tools for the same material found on the book's companion website. Table 8-6 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 8-6** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review config checklists | | Book, website |
| Review command tables | | Book |
| Review memory tables | | Website |
| Do labs | | Sim Lite, blog |
| Watch Video | | Website |

## Review All the Key Topics

Key Topic

**Table 8-7** Key Topics for Chapter 8

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 8-2 | Basic VLAN concept | 191 |
| List | Reasons for using VLANs | 192 |
| Figure 8-5 | Diagram of VLAN trunking | 194 |
| Figure 8-6 | 802.1Q header | 195 |
| Table 8-2 | Options of the **switchport mode** command | 203 |
| Table 8-3 | Expected trunking results based on the configuration of the **switchport mode** command | 207 |
| List | Definitions of data VLAN and voice VLAN | 209 |
| List | Summary of data and voice VLAN concepts, configuration, and verification | 212 |
| Table 8-4 | Commands to find access ports and assigned VLANs | 213 |
| Table 8-5 | Analysis of the three VLAN lists in the output from the **show interfaces** *interface-id* **trunk** command | 218 |

## Key Terms You Should Know

802.1Q, access interface, data VLAN, default VLAN, Layer 3 switch, native VLAN, static access interface, trunk, trunk interface, trunking administrative mode, trunking operational mode, VLAN, voice VLAN, VTP, VTP transparent mode

## Do Labs

The Sim Lite software is a version of Pearson's full simulator learning product with a subset of the labs, included free with this book. The Sim Lite with this book includes a couple of labs about VLANs. Also, check the author's blog site pages for configuration exercises (Config Labs) at https://www.certskills.com.

## Command References

Tables 8-8 and 8-9 list configuration and verification commands used in this chapter, respectively. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 8-8** Chapter 8 Configuration Command Reference

| Command | Description |
|---|---|
| **vlan** *vlan-id* | Global config command that both creates the VLAN and puts the CLI into VLAN configuration mode. |
| **name** *vlan-name* | VLAN subcommand that names the VLAN. |
| [**no**] **shutdown** | VLAN mode subcommand that enables (**no shutdown**) or disables (**shutdown**) the VLAN. |
| [**no**] **shutdown vlan** *vlan-id* | Global config command that has the same effect as the [**no**] **shutdown** VLAN mode subcommands. |
| **vtp mode** {**server** \| **client** \| **transparent** \| **off**} | Global config command that defines the VTP mode. |
| **switchport mode** {**access** \| **dynamic** {**auto** \| **desirable**} \| **trunk**} | Interface subcommand that configures the trunking administrative mode on the interface. |
| **switchport access vlan** *vlan-id* | Interface subcommand that statically configures the interface into that one VLAN. |
| **switchport trunk encapsulation** {**dot1q** \| **isl** \| **negotiate**} | Interface subcommand that defines which type of trunking to use, assuming that trunking is configured or negotiated. |
| **switchport trunk native vlan** *vlan-id* | Interface subcommand that defines the native VLAN for a trunk port. |
| **switchport nonegotiate** | Interface subcommand that disables the negotiation of VLAN trunking. |
| **switchport voice vlan** *vlan-id* | Interface subcommand that defines the voice VLAN on a port, meaning that the switch uses 802.1Q tagging for frames in this VLAN. |
| **switchport trunk allowed vlan** *vlan-list* | Interface subcommand that defines the list of allowed VLANs. Ignores the existing list of allowed VLANs. |
| **switchport trunk allowed vlan** {**add** \| **remove**} *vlan-list* | Interface subcommand that adds to or removes from the current set of allowed VLANs on a trunk, adjusting from the existing list of allowed VLANs. |
| **switchport trunk allowed vlan** {**all** \| **none** \| **except** *vlan-list*} | Interface subcommand that defines the allowed VLAN list as either all VLANs, no VLANs, or all except those in the configured list. Ignores the existing list of allowed VLANs. |

**Table 8-9** Chapter 8 EXEC Command Reference

| Command | Description |
|---|---|
| **show interfaces status** | On ports operating as access ports, it lists the access VLAN, and on ports operating as trunk ports, it lists the word *trunk*. |
| **show interfaces** *interface-id* **switchport** | Lists information about any interface regarding administrative settings and operational state. |

| Command | Description |
|---|---|
| **show interfaces trunk** | Lists information about all operational trunks (but no other interfaces), including the list of VLANs that can be forwarded over the trunk. |
| **show interfaces** *interface-id* **trunk** | Lists trunking status about the listed interface, regardless of whether the interface currently operates as a trunk. |
| **show vlan** [**brief**] | Lists each VLAN and all interfaces assigned to that VLAN (but does not include operational trunks). |
| **show vlan** {**id** *vlan-id* \| **name** *vlan-name*} | Lists information about a specific VLAN by ID or name, and interfaces, including trunks. |
| **show vtp status** | Lists VTP configuration and status information. |



CHAPTER 9

# Spanning Tree Protocol Concepts

**This chapter covers the following exam topics:**

- **2.0 Network Access**
  - **2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP)**
  - **2.5 Interpret basic operations of Rapid PVST+ Spanning Tree Protocol**
    - **2.5.a Root port, root bridge (primary/secondary), and other port names**
    - **2.5.b Port states and roles**
    - **2.5.c PortFast**
    - **2.5.d Root Guard, loop guard, BPDU filter, and BPDU guard**

Spanning Tree Protocol (STP) allows Ethernet LANs to have redundant links in a LAN while overcoming the known problems that occur when adding those extra links. Using redundant links in a LAN design allows the LAN to keep working even when some links fail or even when some entire switches fail. Proper LAN design should add enough redundancy so that no single point of failure crashes the LAN; STP allows the design to use redundancy without causing other problems.

Historically, the IEEE first standardized STP as part of the IEEE 802.1D standard back in 1990, with pre-standard versions working even before that time. Over time, the industry and IEEE improved STP, with the eventual replacement of STP with an improved protocol: Rapid Spanning Tree Protocol (RSTP). The IEEE first released RSTP as amendment 802.1w and, in 2004, integrated RSTP into the 802.1D standard.

Today, most networks use RSTP rather than STP; however, STP and RSTP share many of the same mechanisms, and RSTP's improvements can be best understood in comparison to STP. For that reason, this chapter presents some details that apply only to STP, as a learning tool to help you understand RSTP.

This chapter organizes the material into four sections. The first section presents some core concepts about how both STP and RSTP discover a tree made of nodes (switches) and links so that no loops exist in a network. The second section then takes a brief look at the area in which STP differs the most from RSTP: in how STP reacts to changes in the network. The third section then shows how RSTP works much better than STP when reacting to changes. The final section touches on a variety of small optional STP and RSTP features.

Finally, be warned that this chapter, as well as Chapter 10, are a little longer than expected. If you like to think of each chapter as one study session, you might need to think about splitting this chapter into two study sessions. For the first study session, you should stop at the section, "Rapid STP Concepts." The second study session would consist of the "Rapid STP Concepts" and "Optional STP Features" sections.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 9-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| STP and RSTP Basics | 1–2 |
| Details Specific to STP (and Not RSTP) | 3–4 |
| Rapid STP Concepts | 5–6 |
| Optional STP Features | 7–8 |

1. Which of the following port states are stable states used when STP has completed convergence? (Choose two answers.)
   a. Blocking
   b. Forwarding
   c. Listening
   d. Learning
   e. Discarding
2. Which of the following bridge IDs wins election as root, assuming that the switches with these bridge IDs are in the same network?
   a. 32769:0200.1111.1111
   b. 32769:0200.2222.2222
   c. 4097:0200.1111.1111
   d. 4097:0200.2222.2222
   e. 40961:0200.1111.1111
3. Which of the following are transitory port states used only during the process of STP convergence? (Choose two answers.)
   a. Blocking
   b. Forwarding
   c. Listening
   d. Learning
   e. Discarding
4. Which of the following facts determines how often a nonroot bridge or switch sends an STP Hello BPDU message?
   a. The Hello timer as configured on that switch.
   b. The Hello timer as configured on the root switch.

c. It is always every 2 seconds.
d. The switch reacts to BPDUs received from the root switch by sending another BPDU 2 seconds after receiving the root BPDU.

5. Which of the following RSTP port states have the same name and purpose as a port state in traditional STP? (Choose two answers.)
   a. Blocking
   b. Forwarding
   c. Listening
   d. Learning
   e. Discarding
6. RSTP adds features beyond STP that enable ports to be used for a role if another port on the same switch fails. Which of the following statements correctly describe a port role that is waiting to take over for another port role? (Choose two answers.)
   a. An alternate port waits to become a root port.
   b. A backup port waits to become a root port.
   c. An alternate port waits to become a designated port.
   d. A backup port waits to become a designated port.
7. What STP/RSTP feature causes an interface to be placed in the forwarding state as soon as the interface is physically active?
   a. STP
   b. EtherChannel
   c. Root Guard
   d. PortFast
8. Which optional STP feature reacts to a subset of incoming STP BPDUs to disable the port, but allows and processes some other STP BPDUs?
   a. Loop Guard
   b. BPDU Guard
   c. Root Guard
   d. PortFast

# Foundation Topics

## STP and RSTP Basics

Without some mechanism like **Spanning Tree Protocol (STP)** or **Rapid STP (RSTP)**, a LAN with redundant links would cause Ethernet frames to loop for an indefinite period of time. With STP or RSTP enabled, some switches block ports so that these ports do not forward frames. STP and RSTP intelligently choose which ports block, with two goals in mind:

- All devices in a VLAN can send frames to all other devices. In other words, STP or RSTP does not block too many ports, cutting off some parts of the LAN from other parts.
- Frames have a short life and do not loop around the network indefinitely.

STP and RSTP strike a balance, allowing frames to be delivered to each device, without causing the problems that occur when frames loop through the network over and over again.

**NOTE** This first major section of the chapter explains details of both STP and RSTP, so this section uses the term *STP/RSTP* to refer to these protocols together. Note that this term is just a convenient shorthand. Later in the chapter, the text will point out differences between STP and RSTP and begin using the terms *STP* and *RSTP* separately, referring to only the specific protocol.

STP/RSTP prevents looping frames by adding an additional check on each interface before a switch uses it to send or receive user traffic. That check: If the port is in STP/RSTP **forwarding state** in that VLAN, use it as normal; if it is in STP/RSTP **blocking state**, however, block all user traffic and do not send or receive user traffic on that interface in that VLAN.

Note that these STP/RSTP states do not change the other information you already know about switch interfaces. The interface's state of connected/notconnect does not change. The interface's operational state as either an access or trunk port does not change. STP/RSTP adds this additional state, with the blocking state basically disabling the interface.

In many ways, the preceding two paragraphs sum up what STP/RSTP does. However, the details of how STP/RSTP does its work can take a fair amount of study and practice. This first major section of the chapter begins by explaining the need for STP/RSTP and the basic ideas of what STP/RSTP does to solve the problem of looping frames. The majority of this section then looks at how STP/RSTP goes about choosing which switch ports to block to accomplish its goals.

## The Need for Spanning Tree

STP/RSTP prevents three common problems in Ethernet LANs. All three problems occur as a side effect of one fact: without STP/RSTP, some Ethernet frames would loop around the network for a long time (hours, days, literally forever if the LAN devices and links never failed).

Just one looping frame causes what is called a *broadcast storm*. Broadcast storms happen when any kind of Ethernet frames—broadcast frames, multicast frames, or unknown-destination unicast frames—loop around a LAN indefinitely. Broadcast storms can saturate all the links with copies of that one single frame, crowding out good frames, as well as significantly impacting end-user device performance by making the PCs process too many broadcast frames.

To help you understand how this occurs, Figure 9-1 shows a sample network in which Bob sends a broadcast frame. The dashed lines show how the switches forward the frame when STP/RSTP does not exist.

Remember that LAN switch? That logic tells switches to flood broadcasts out all interfaces in the same VLAN except the interface in which the frame arrived. In Figure 9-1, that means SW3 forwards Bob's frame to SW2, SW2 forwards the frame to SW1, SW1 forwards the frame back to SW3, and SW3 forwards it back to SW2 again.

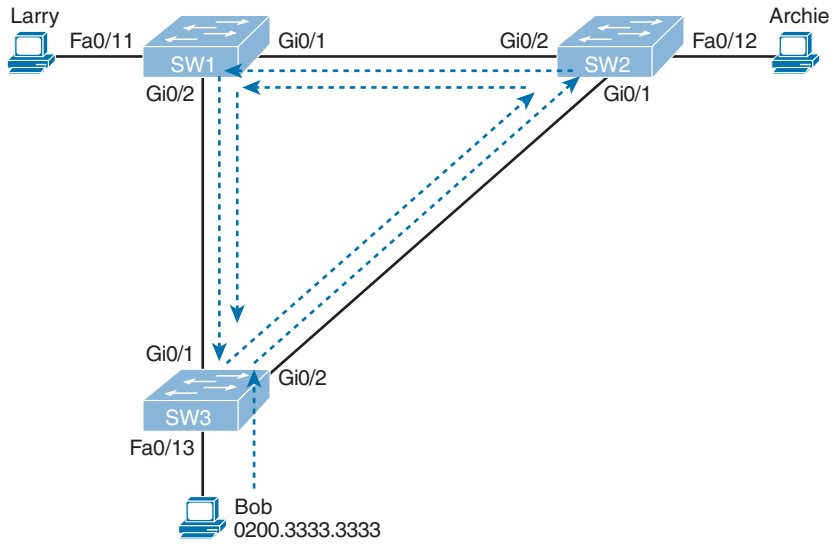



**Figure 9-1** *Broadcast Storm*

When broadcast storms happen, frames like the one in Figure 9-1 keep looping until something changes—someone shuts down an interface, reloads a switch, or does something else to break the loop. Also note that the same event happens in the opposite direction. When Bob sends the original frame, SW3 also forwards a copy to SW1, SW1 forwards it to SW2, and so on.

The storm also causes a much more subtle problem called *MAC table instability*. MAC table instability means that the switches' MAC address tables keep changing because frames with the same source MAC arrive on different ports. To see why, follow this example, in which SW3 begins Figure 9-1 with a MAC table entry for Bob, at the bottom of the figure, associated with port Fa0/13:

0200.3333.3333 Fa0/13 VLAN 1

However, now think about the switch-learning process that occurs when the looping frame goes to SW2, then SW1, and then back into SW3's Gi0/1 interface. SW3 thinks, "Hmm…the source MAC address is 0200.3333.3333, and it came in my Gi0/1 interface. Update my MAC table!" This results in the following entry on SW3, with interface Gi0/1 instead of Fa0/13:

0200.3333.3333 Gi0/1 VLAN 1

At this point, SW3 itself cannot correctly deliver frames to Bob's MAC address. At that instant, if a frame arrives at SW3 destined for Bob—a different frame than the looping frame that causes the problems—SW3 incorrectly forwards the frame out Gi0/1 to SW1, creating even more congestion.

The looping frames in a broadcast storm also cause a third problem: multiple copies of the frame arrive at the destination. Consider a case in which Bob sends a frame to Larry but

---

Answers to the "Do I Know This Already?" quiz:

**1** A, B **2** C **3** C, D **4** B **5** B, D **6** A, D **7** D, **8** C

none of the switches know Larry's MAC address. Switches flood frames sent to unknown destination unicast MAC addresses. When Bob sends the frame destined for Larry's MAC address, SW3 sends a copy to both SW1 and SW2. SW1 and SW2 also flood the frame, causing copies of the frame to loop. SW1 also sends a copy of each frame out Fa0/11 to Larry. As a result, Larry gets multiple copies of the frame, which may result in an application failure, if not more pervasive networking problems.

Table 9-2 summarizes the main three classes of problems that occur when STP/RSTP is not used in a LAN that has redundancy.

Key Topic

**Table 9-2** Three Classes of Problems Caused by Not Using STP in Redundant LANs

| Problem | Description |
|---|---|
| Broadcast storms | The forwarding of a frame repeatedly on the same links, consuming significant parts of the links' capacities |
| MAC table instability | The continual updating of a switch's MAC address table with incorrect entries, in reaction to looping frames, resulting in frames being sent to the wrong locations |
| Multiple frame transmission | A side effect of looping frames in which multiple copies of one frame are delivered to the intended host, confusing the host |

## What Spanning Tree Does

STP/RSTP prevents loops by placing each switch port in either a forwarding state or a blocking state. Interfaces in the forwarding state act as normal, forwarding and receiving frames. However, interfaces in a blocking state do not process any frames except STP/RSTP messages (and some other overhead messages). Interfaces that block do not forward user frames, do not learn MAC addresses of received frames, and do not process received user frames.

Figure 9-2 shows a simple STP/RSTP tree that solves the problem shown in Figure 9-1 by placing one port on SW3 in the blocking state.

Key Topic

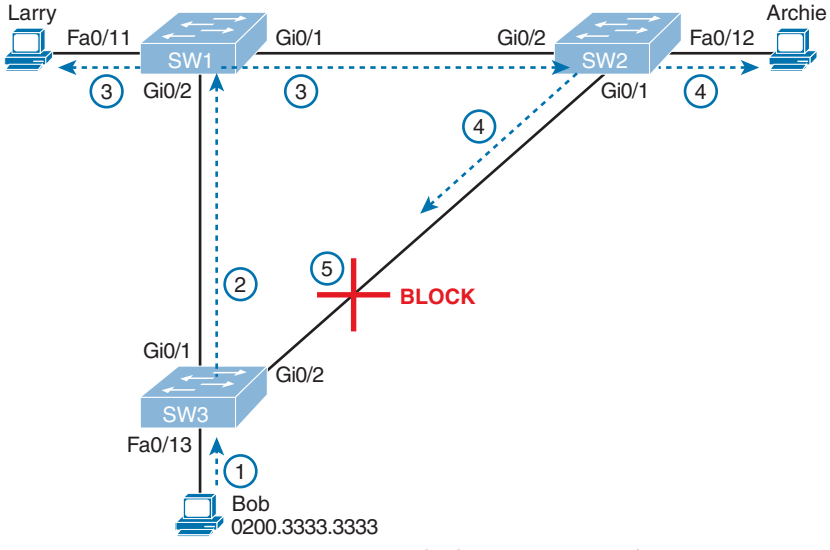



**Figure 9-2** *What STP/RSTP Does: Blocks a Port to Break the Loop*

Now when Bob sends a broadcast frame, the frame does not loop. As shown in the steps in the figure:

**Step 1.** Bob sends the frame to SW3.

**Step 2.** SW3 forwards the frame only to SW1, but not out Gi0/2 to SW2, because SW3's Gi0/2 interface is in a blocking state.

**Step 3.** SW1 floods the frame out both Fa0/11 and Gi0/1.

**Step 4.** SW2 floods the frame out Fa0/12 and Gi0/1.

**Step 5.** SW3 physically receives the frame, but it ignores the frame received from SW2 because SW3's Gi0/2 interface is in a blocking state.

With the STP/RSTP topology in Figure 9-2, the switches simply do not use the link between SW2 and SW3 for traffic in this VLAN, which is the minor negative side effect of STP. However, if either of the other two links fails, STP/RSTP converges so that SW3 forwards instead of blocks on its Gi0/2 interface.

**NOTE** The term *STP convergence* refers to the process by which the switches collectively realize that something has changed in the LAN topology and determine whether they need to change which ports block and which ports forward.

That completes the description of what STP/RSTP does, placing each port into either a forwarding or blocking state. The more interesting question, and the one that takes a lot more work to understand, is how and why STP/RSTP makes its choices. How does STP/RSTP manage to make switches block or forward on each interface? And how does it converge to change state from blocking to forwarding to take advantage of redundant links in response to network outages? The following pages answer these questions.

## How Spanning Tree Works

The STP/RSTP algorithm creates a spanning tree of interfaces that forward frames. The tree structure of forwarding interfaces creates a single path to and from each Ethernet link, just like you can trace a single path in a living, growing tree from the base of the tree to each leaf.

**NOTE** STP existed before LAN switches, with the STP processes performed by devices called bridges. Some terms and command output still use the older term *bridge*, so when working with STP/RSTP, consider the terms *bridge* and *switch* synonymous.

The process used by STP, sometimes called the *spanning-tree algorithm* (STA), chooses the interfaces that should be placed into a forwarding state. For any interfaces not chosen to be in a forwarding state, STP/RSTP places the interfaces in a blocking state. In other words, STP/RSTP simply picks which interfaces should forward, and any interfaces left over go to a blocking state.

STP/RSTP uses three criteria to choose whether to put an interface in a forwarding state:

- STP/RSTP elects a **root switch**. STP puts all working interfaces on the root switch in a forwarding state.
- Each nonroot switch calculates the least-cost path between itself and the root switch based on STP/RSTP interface costs. The switch port that begins that least-cost path is its *root port* (RP), and the cost is the switch's **root cost**.
- In a modern Ethernet that uses switches, each physical link connects two devices only. With two switches on a link, the switch with the lowest root cost becomes the *designated switch,* and its port connected to that link is the **designated port** (DP) on the link.

**NOTE** A root switch places all interfaces into a forwarding state because each port on the root switch always wins its designated port (DP) election. However, it is easier to just remember that all the root switches' working interfaces will forward frames.

All other interfaces are placed in a blocking state. Table 9-3 summarizes the reasons STP/RSTP places a port in a forwarding or blocking state.

Key Topic

**Table 9-3** STP/RSTP: Reasons for Forwarding or Blocking

| Characterization of Port | STP State | Description |
|---|---|---|
| All the root switch's ports | Forwarding | The root switch is always the designated switch on all connected segments. |
| Each nonroot switch's root port | Forwarding | The port through which the switch has the least cost to reach the root switch (lowest root cost). |
| Each LAN's designated port | Forwarding | The switch forwarding the Hello on to the segment, with the lowest root cost, is the designated switch for that segment. |
| All other working ports | Blocking | The port is not used for forwarding user frames, nor are any frames received on these interfaces considered for forwarding. |

**NOTE** STP/RSTP should not use nonworking interfaces in the working STP/RSTP topology. To remove all nonworking interfaces from consideration—that is, all that are not in a connected (up/up) interface state—STP/RSTP assigns nonworking ports with the *disabled port role*. STP/RSTP does not consider these ports as potential RP or DP ports. STP also places these ports into a **disabled state** (much like blocking).

### The STP Bridge ID and Hello BPDU

The STA begins with an election of one switch to be the root switch. To better understand this election process, you need to understand the STP/RSTP messages sent between switches as well as the concept and format of the identifier used to uniquely identify each switch.

The STP/RSTP **bridge ID** (BID) is an 8-byte value unique to each switch. The bridge ID consists of a 2-byte priority field and a 6-byte system ID, with the system ID being based on a universal (burned-in) MAC address in each switch. Using a burned-in MAC address ensures that each switch's bridge ID will be unique.

STP/RSTP defines messages called **bridge protocol data units (BPDU)**, also called configuration BPDUs, which switches use to exchange information with each other. The most common BPDU, called a **Hello BPDU**, lists many details, including the sending switch's BID. By listing its own unique BID, switches can tell which switch sent which Hello BPDU. Table 9-4 lists some of the key information in the Hello BPDU.

Key Topic

**Table 9-4** Fields in the STP Hello BPDU

| Field | Description |
|---|---|
| Root bridge ID | The bridge ID of the switch that the sender of this Hello currently believes to be the root switch |
| Sender's bridge ID | The bridge ID of the switch sending this Hello BPDU |
| Sender's root cost | The STP/RSTP cost between this switch and the current root |
| Timer values on the root switch | Includes the Hello timer, MaxAge timer, and **forward delay** timer |

For the time being, just keep the first three items from Table 9-4 in mind as the following sections work through the three steps in how STP/RSTP chooses the interfaces to place into a forwarding state. Next, the text examines the three main steps in the STP/RSTP process.

### Electing the Root Switch

Switches elect a root switch based on the BIDs in the BPDUs. The root switch is the switch with the lowest numeric value for the BID. Because the two-part BID starts with the priority value, essentially the switch with the lowest priority becomes the root. For example, if one switch has priority 4096, and another switch has priority 8192, the switch with priority 4096 wins, regardless of what MAC address was used to create the BID for each switch.

If a tie occurs based on the priority portion of the BID, the switch with the lowest MAC address portion of the BID is the root. No other tiebreaker should be needed because switches use one of their own universal (burned-in) MAC addresses as the second part of their BIDs. So, if the priorities tie, and one switch uses a MAC address of 0200.0000.0000 as part of the BID and the other uses 0811.1111.1111, the first switch (MAC 0200.0000.0000) becomes the root switch.

STP/RSTP elects a root switch in a manner not unlike a political election. The process begins with all switches claiming to be the root by sending Hello BPDUs listing their own BID as the root BID. If a switch hears a Hello that lists a better (lower) BID, that switch stops advertising itself as root and starts forwarding the superior Hello. The Hello sent by the better switch lists the better switch's BID as the root. It works like a political race in which a less-popular candidate gives up and leaves the race, throwing his support behind the more popular candidate. Eventually, everyone agrees which switch has the best (lowest) BID, and everyone supports the elected switch—which is where the political race analogy falls apart.

**NOTE** A better Hello, meaning that the listed root's BID is better (numerically lower), is called a *superior Hello*; a worse Hello, meaning that the listed root's BID is not as good (numerically higher), is called an *inferior Hello*.

Figure 9-3 shows the beginning of the root election process. In this case, SW1 has advertised itself as root, as have SW2 and SW3. However, SW2 now believes that SW1 is a better root, so SW2 is now forwarding the Hello originating at SW1. So, at this point, the figure shows SW1 is saying Hello, claiming to be root; SW2 agrees and is forwarding SW1's Hello that lists SW1 as root; but SW3 is still claiming to be best, sending its own Hello BPDUs, listing SW3's BID as the root.

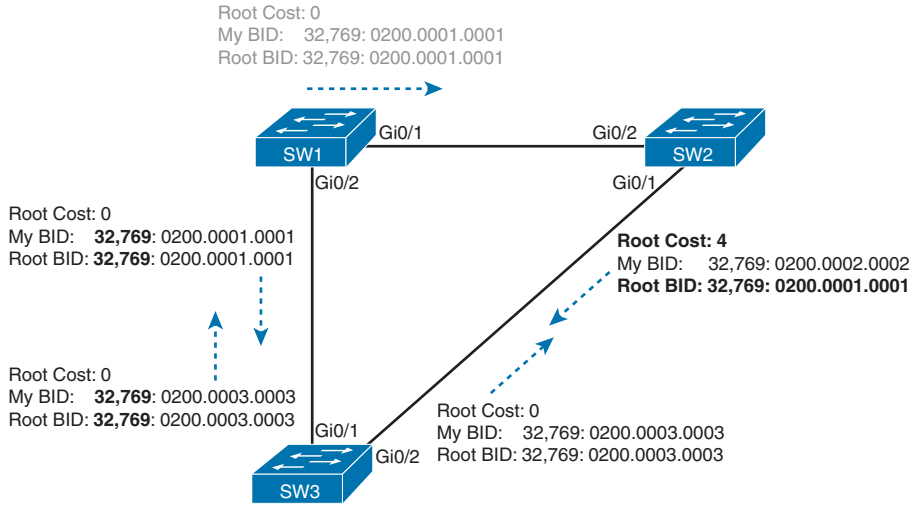



**Figure 9-3** *Beginnings of the Root Election Process*

Two candidates still exist in Figure 9-3: SW1 and SW3. So, who wins? Well, from the BID, the lower-priority switch wins; if a tie occurs, the lower MAC address wins. As shown in the figure on the left, the switch priority values (32,769) tie, but SW1 has a lower BID (32,769:0200.0001.0001) than SW3 (32,769:0200.0003.0003) due to its lower MAC address. SW1 wins, and SW3 now also believes that SW1 is the better switch. Figure 9-4 shows the resulting Hello messages sent by the switches.

Summarizing, the root election happens through each switch claiming to be root, with the best switch being elected based on the numerically lowest BID. Breaking down the BID into its components, the comparisons can be made as:

Key Topic

- The lowest priority
- If that ties, the lowest switch MAC address

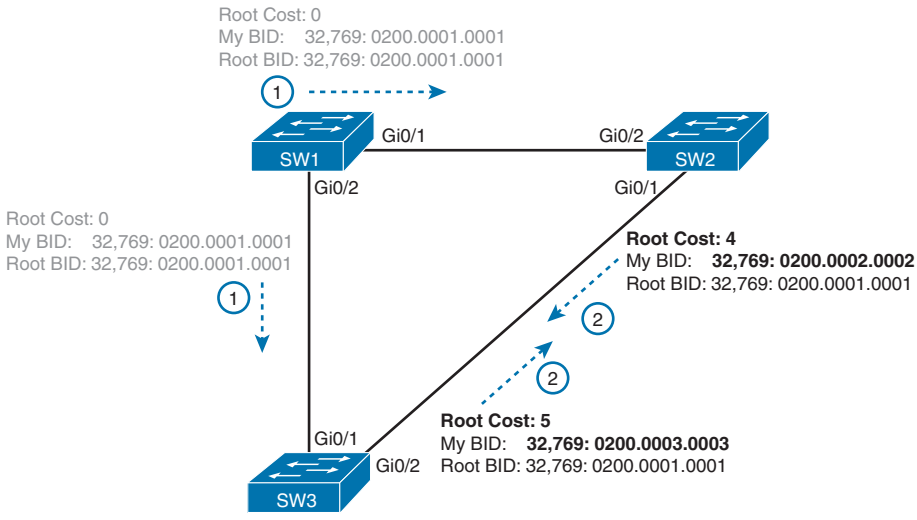



**Figure 9-4** *SW1 Wins the Election*

## Choosing Each Switch's Root Port

The second part of the STP/RSTP process occurs when each nonroot switch chooses its one and only *root port* (RP). A switch's RP is its interface through which it has the least STP/RSTP cost to reach the root switch (least root cost).

You can easily see the idea behind a switch's cost to reach the root. Just look at a network diagram that shows the root switch, lists the STP/RSTP cost associated with each switch port, and identifies the nonroot switch in question. Switches use a different process than you can see by looking at a network diagram, of course, but using a diagram can make it easier to learn the idea.

Figure 9-5 shows just such a figure, with the same three switches shown in the last several figures. SW1 has already won the election as root, and the figure considers the cost from SW3's perspective. (Note that the figure uses some nondefault cost settings.)

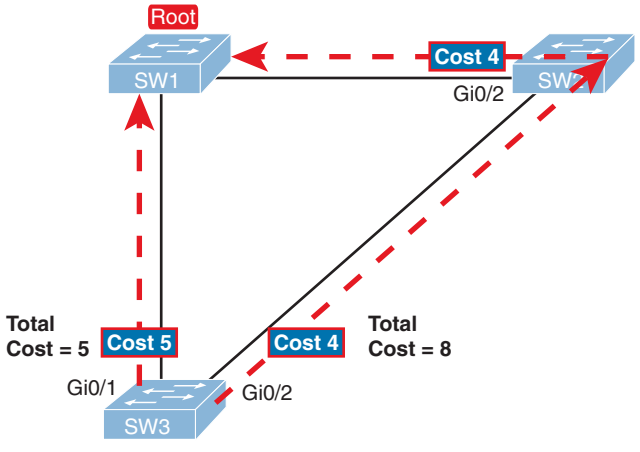



**Figure 9-5** *How a Human Might Calculate STP/RSTP Cost from SW3 to the Root (SW1)*

SW3 has two possible physical paths to send frames to the root switch: the direct path to the left and the indirect path to the right through switch SW2. The cost is the sum of the costs of all the *switch ports the frame would exit* if it flowed over that path. (The calculation ignores the inbound ports.) As you can see, the cost over the direct path out SW3's G0/1 port has a total cost of 5, and the other path has a total cost of 8. SW3 picks its G0/1 port as root port because it is the port that is part of the least-cost path to send frames to the root switch.

Switches come to the same conclusion but using a different process. Instead, they add their local interface STP/RSTP cost to the root cost listed in each received Hello BPDU. The STP/RSTP port cost is simply an integer value assigned to each interface, per VLAN, for the purpose of providing an objective measurement that allows STP/RSTP to choose which interfaces to add to the STP/RSTP topology. The switches also look at their neighbor's root cost, as announced in Hello BPDUs received from each neighbor.

Figure 9-6 shows an example of how switches calculate their best root cost and then choose their root port, using the same topology and STP/RSTP costs as shown in Figure 9-5. STP/RSTP on SW3 calculates its cost to reach the root over the two possible paths by adding the advertised cost (in Hello messages) to the interface costs listed in the figure.

Key Topic

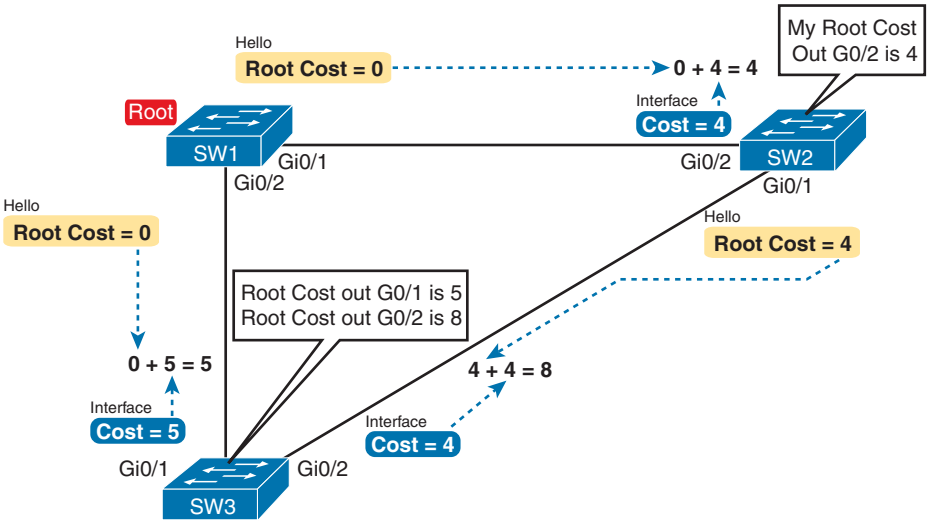



**Figure 9-6** *How STP/RSTP Actually Calculates the Cost from SW3 to the Root*

Focus on the process for a moment. The root switch sends Hellos, with a listed root cost of 0. The idea is that the root's cost to reach itself is 0.

Next, look on the left of the figure. SW3 takes the received cost (0) from the Hello sent by SW1 and adds the interface cost (5) of the interface on which that Hello was received. SW3 calculates that the cost to reach the root switch, out that port (G0/1), is 5.

On the right side, SW2 has realized its best cost to reach the root is cost 4. So, when SW2 forwards the Hello toward SW3, SW2 lists a root cost 4. SW3's STP/RSTP port cost on port G0/2 is 4, so SW3 determines a total cost to reach root out its G0/2 port of 8.

As a result of the process depicted in Figure 9-6, SW3 chooses Gi0/1 as its RP because the cost to reach the root switch through that port (5) is lower than the other alternative (Gi0/2, cost 8). Similarly, SW2 chooses Gi0/2 as its RP, with a cost of 4 (SW1's advertised cost of 0 plus SW2's Gi0/2 interface cost of 4). Each switch places its root port into a forwarding state.

Switches need a tiebreaker to use in case the best root cost ties for two or more paths. If a tie occurs, the switch applies these three tiebreakers to the paths that tie, in order, as follows:

1. Choose based on the lowest neighbor bridge ID.
2. Choose based on the lowest neighbor port priority.
3. Choose based on the lowest neighbor internal port number.

### Choosing the Designated Port on Each LAN Segment

The final STP/RSTP step determines the designated port on each LAN segment. The switch with the lowest root cost wins the competition. With two switches on the ends of a link, the switch with the lower cost to reach the root becomes the designated switch, with its port on that link becoming the DP on that segment. On a link with a switch and a second device that does not use STP/RSTP (for instance, a router or PC), the switch becomes the DP.

The process to choose the DP begins with the switch(es) forwarding Hellos onto the link. The Hello messages include a field to list the root cost of the switch that forwarded the Hello. Determining which switch has the lowest root cost requires a simple comparison. Once a switch port becomes the DP, it continues to forward Hellos onto the link.

For example, earlier Figure 9-4 shows in bold text the parts of the Hello messages from both SW2 and SW3 that determine the choice of DP on that segment. Note that both SW2 and SW3 list their respective root cost (cost 4 on SW2 and cost 5 on SW3). SW2 lists the lower cost, so SW2's Gi0/1 port is the designated port on that LAN segment.

All DPs are placed into a forwarding state; in this case, SW2's Gi0/1 interface will be in a forwarding state.

If the advertised costs tie, the switches break the tie by choosing the switch with the lower BID. In this case, SW2 would also have won, with a BID of 32769:0200.0002.0002 versus SW3's 32769:0200.0003.0003.

**NOTE** Two additional tiebreakers are needed in some cases, although these would be unlikely today. A single switch can connect two or more interfaces to the same collision domain by connecting to a hub. In that case, the one switch hears its own BPDUs. So, if a switch ties with itself, two additional tiebreakers are used: the lowest interface STP/RSTP priority and, if that ties, the lowest internal interface number.

The only interface that does not have a reason to be in a forwarding state on the three switches in the examples shown in Figures 9-3 through 9-6 is SW3's Gi0/2 port. So, the STP/RSTP process is now complete. Table 9-5 outlines the state of each port and shows why it is in that state.

**Table 9-5** State of Each Interface

| Switch Interface | State | Reason Why the Interface Is in Forwarding State |
|---|---|---|
| SW1, Gi0/1 | Forwarding | The interface is on the root switch, so it becomes the DP on that link. |
| SW1, Gi0/2 | Forwarding | The interface is on the root switch, so it becomes the DP on that link. |
| SW2, Gi0/2 | Forwarding | The root port of SW2. |
| SW2, Gi0/1 | Forwarding | The designated port on the LAN segment to SW3. |
| SW3, Gi0/1 | Forwarding | The root port of SW3. |
| SW3, Gi0/2 | Blocking | Not the root port and not the designated port. |

Note that the examples in this section focus on the links between the switches, but switch ports connected to endpoint devices should become DPs and settle into a forwarding state. Working through the logic, each switch will forward BPDUs on each port as part of the process to determine the DP on that LAN. Endpoints should ignore those messages because they do not run STP/RSTP, so the switch will win and become the DP on every access port.

## Configuring to Influence the STP Topology

STP/RSTP works by default on Cisco switches, so all the settings needed by a switch have a useful default. Switches have a default BID, based on a default priority value and adding a universal MAC address that comes with the switch hardware. Additionally, switch interfaces have default STP/RSTP costs based on the current operating speed of the switch interfaces.

Network engineers often want to change the STP/RSTP settings to then change the choices STP/RSTP makes in a given LAN. Two main tools available to the engineer are to configure the bridge ID and to change STP/RSTP port costs.

First, to change the BID, the engineer can set the priority used by the switch, while continuing to use the universal MAC address as the final 48 bits of the BID. For instance, giving a switch the lowest priority value among all switches will cause that switch to win the root election.

Port costs also have default values, per port, per VLAN. You can configure these port costs, which will in turn impact many switch's calculations of the root cost. For instance, to favor one link, give the ports on that link a lower cost, or to avoid a link, give the ports a higher cost.

Table 9-6 lists the default port costs suggested by IEEE. IOS on Cisco switches has long used the default settings as defined as far back as the 1998 version of the STP standard. The latest IEEE standards suggest values that are more useful when using links faster than 10 Gbps. You can configure a switch to use the old (short) or new (long) defaults using the global command (**spanning-tree pathcost method** {**short** | **long**})—but it makes sense to use the same setting on all switches in the same campus.

Key Topic

**Table 9-6** Default Port Costs According to IEEE

| Ethernet Speed | IEEE Cost: 1998 (and Before) | IEEE Cost: 2004 (and After) |
|---|---|---|
| 10 Mbps | 100 | 2,000,000 |
| 100 Mbps | 19 | 200,000 |
| 1 Gbps | 4 | 20,000 |
| 10 Gbps | 2 | 2000 |
| 100 Gbps | N/A | 200 |
| 1 Tbps | N/A | 20 |

Also, be aware that Cisco switches choose the default cost based on the operating speed of the link, not the maximum speed. That is, if a 10/100/1000 port runs at 10 Mbps for some reason, its default STP cost on a Cisco switch is 100.

## Details Specific to STP (and Not RSTP)

As promised in the introduction to this chapter, the first section showed features that apply to both STP and RSTP. This next heading acts as the turning point, with the next several pages being about STP only. The upcoming section titled "Rapid STP Concepts" then shows details specific to RSTP, in contrast to STP.

Once the engineer has finished all STP configuration, the STP topology should settle into a stable state and not change, at least until the network topology changes. This section examines the ongoing operation of STP while the network is stable, and then it covers how STP converges to a new topology when something changes.

Note that almost all the differences between STP and RSTP revolve around the activities of waiting for and reacting to changes in the topology. STP performed well for the era and circumstances in which it was created. The "rapid" in RSTP refers to the improvements to how fast RSTP could react when changes occur—so understanding how STP reacts will be useful to understand why RSTP reacts faster. These next few pages show the specifics of STP (and not RSTP) and how STP reacts to and manages convergence when changes happen in an Ethernet LAN.

### STP Activity When the Network Remains Stable

When the network remains stable, the root switch generates a new Hello BPDU, with the nonroot switches forwarding copies of the Hello so that the Hellos traverse the spanning tree. The root generates a new Hello every Hello time (default 2 seconds), with each nonroot switch forwarding the Hello out any interfaces in a forwarding state after updating the Hello. The following steps summarize the steady-state operation when nothing is currently changing in the STP topology:

Key Topic

**Step 1.** The root creates and sends a Hello BPDU, with a root cost of 0, out all its working interfaces (those in a forwarding state).

**Step 2.** The nonroot switches receive the Hello on their root ports. After changing the Hello to list their own BID as the sender's BID and listing that switch's root cost, the switch forwards the Hello out all designated ports.

**Step 3.** Steps 1 and 2 repeat every Hello time until something changes.

When a switch fails to receive a Hello in its root port, it knows a problem might be occurring in the network. Each switch relies on these periodically received Hellos from the root as a way to know that its path to the root is still working. When a switch ceases to receive the Hellos, or receives a Hello that lists different details, something has failed, so the switch reacts and starts the process of changing the spanning-tree topology.

## STP Timers That Manage STP Convergence

For various reasons, the STP convergence process requires the use of three timers, listed in Table 9-7. Note that all switches use the timers as dictated by the root switch, which the root lists in its periodic Hello BPDU messages.

Key Topic

**Table 9-7** STP Timers

| Timer | Default Value | Description |
|---|---|---|
| Hello | 2 seconds | The time period between Hellos created by the root. |
| MaxAge | 10 times Hello | How long any switch should wait, after ceasing to hear Hellos, before trying to change the STP topology. |
| Forward delay | 15 seconds | Delay that affects the process that occurs when an interface changes from a blocking state to a forwarding state. A port stays in an interim **listening state**, and then an interim **learning state**, for the number of seconds defined by the forward delay timer. |

If a switch does not get one expected Hello BPDU within the Hello time, the switch continues as normal. However, if the Hellos do not show up again within MaxAge time, the switch reacts by taking steps to change the STP topology. With default settings, MaxAge is 20 seconds (ten times the default Hello timer of 2 seconds). So, a switch would go 20 seconds without hearing a Hello before reacting.

After MaxAge expires, the switch essentially makes all its STP choices again, based on any Hellos it receives from other switches. It reevaluates which switch should be the root switch. If the local switch is not the root, it chooses its RP. And it determines whether it is the DP on each of its other links.

The best way to describe STP convergence is to show an example using the same familiar topology. Figure 9-7 shows the same familiar figure, with SW3's Gi0/2 in a blocking state, but SW1's Gi0/2 interface has just failed.

In the scenario shown in the figure, SW3 reacts to the change because SW3 fails to receive its expected Hellos on its Gi0/1 interface. However, SW2 does not need to react because SW2 continues to receive its periodic Hellos in its Gi0/2 interface. In this case, SW3 reacts either when **MaxAge** time passes without hearing the Hellos, or as soon as SW3 notices that interface Gi0/1 has failed. (If the interface fails, the switch can assume that the Hellos will not be arriving in that interface anymore.)

Now that SW3 can act, it begins by reevaluating the choice of root switch. SW3 still receives the Hellos from SW2, as forwarded from the root (SW1). SW1 still has a lower BID than SW3; otherwise, SW1 would not have already been the root. So, SW3 decides that SW1 wins the root election and that SW3 is not the root.

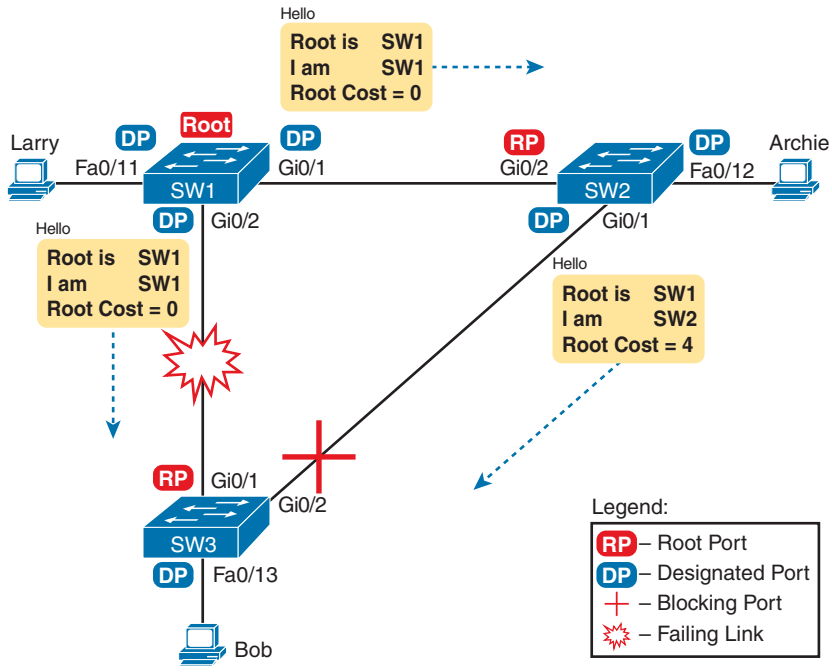



**Figure 9-7** *Initial STP State Before SW1-SW3 Link Fails*

Next, SW3 reevaluates its choice of RP. At this point, SW3 is receiving Hellos on only one interface: Gi0/2. Whatever the calculated root cost, Gi0/2 becomes SW3's new RP. (The cost would be 8, assuming the same STP costs in earlier Figures 9-5 and 9-6.)

SW3 then reevaluates its role as DP on any other interfaces. In this example, no real work needs to be done. SW3 was already the DP on interface Fa0/13, and it continues to be the DP because no other switches connect to that port.

## Changing Interface States with STP

STP uses the idea of roles and states. *Roles*, like root port role and designated port role, relate to how STP analyzes the LAN topology. *States*, like forwarding and blocking, tell a switch whether to send or receive frames. When STP converges, a switch chooses new port roles, and the port roles determine the state (forwarding or blocking).

Switches using STP can simply move immediately from a forwarding to a blocking state, but they must take extra time to transition from a blocking state to a forwarding state. For instance, when switch SW3 in Figure 9-7 formerly used port G0/1 as its RP (a role), that port was in a forwarding state. After convergence, G0/1 might be neither an RP nor a DP; the switch can immediately move that port to a blocking state.

However, when a port that formerly blocked needs to transition to forwarding, the switch first puts the port through two intermediate interface states. These temporary STP states help prevent temporary loops:

Key Topic

- **Listening:** Like the blocking state, the interface does not forward frames. The switch removes old stale (unused) MAC table entries for which no frames are received from each MAC address during this period. These stale MAC table entries could be the cause of the temporary loops.
- **Learning:** Interfaces in this state still do not forward frames, but the switch begins to learn the MAC addresses of frames received on the interface.

STP moves an interface from blocking to listening, then to learning, and then to a forwarding state. STP leaves the interface in each interim state for a time equal to the forward delay timer, which defaults to 15 seconds. As a result, a convergence event that causes an interface to change from blocking to forwarding requires 30 seconds to transition from blocking to forwarding. In addition, a switch might have to wait MaxAge seconds (default 20 seconds) before even choosing to move an interface from a blocking to a forwarding state.

For example, follow what happens with an initial STP topology as shown in Figures 9-3 through 9-6, with the SW1-to-SW3 link failing as shown in Figure 9-7. If SW1 simply quit sending Hello messages to SW3, but the link between the two did not fail, SW3 would wait MaxAge seconds before reacting (20 seconds is the default). SW3 would actually quickly choose its ports' STP roles, but then wait 15 seconds each in listening and learning states on interface Gi0/2, resulting in a 50-second convergence delay.

Table 9-8 summarizes spanning tree's various interface states for easier review.

Key Topic

**Table 9-8** IEEE STP (not RSTP) States

| State | Forwards Data Frames? | Learns MACs Based on Received Frames? | Transitory or Stable State? |
|---|---|---|---|
| Blocking | No | No | Stable |
| Listening | No | No | Transitory |
| Learning | No | Yes | Transitory |
| Forwarding | Yes | Yes | Stable |
| Disabled | No | No | Stable |

## Rapid STP Concepts

The original STP worked well given the assumptions about networks and networking devices in that era. However, as with any computing or networking standard, as time passes, hardware and software capabilities improve, so new protocols emerge to take advantage of those new capabilities. For STP, one of the most significant improvements over time has been the introduction of Rapid Spanning Tree Protocol (RSTP), introduced as standard IEEE 802.1w.

**NOTE** Just to make sure you are clear about the terminology: Throughout the rest of the chapter, *STP* refers to the original STP standard only, and use of the term *RSTP* does not include STP.

Before getting into the details of RSTP, it helps to make sense of the standards numbers a bit: 802.1w was actually an amendment to the 802.1D standard. The IEEE first published 802.1D in 1990, and anew in 1998. After the 1998 version of 802.1D, the IEEE published the 802.1w amendment to 802.1D in 2001, which first standardized RSTP.

Over the years, other meaningful changes happened in the standards as well, although those changes probably do not impact most networkers' thinking when it comes to working with STP or RSTP. But to be complete, the IEEE replaced STP with RSTP in the revised 802.1D standard in 2004. In another move, in 2011 the IEEE moved all the RSTP details into a revised 802.1Q standard. As it stands today, RSTP actually sits in the 802.1Q standards document.

As a result, when reading about RSTP, you will see documents, books, videos, and the like that refer to RSTP and include various references to 802.1w, 802.1D, and 802.1Q—and they might all be correct based on timing and context. At the same time, many people refer to RSTP as 802.1w because that was the first IEEE document to define it. However, for the purposes of this book, focus instead on the RSTP acronym rather than the IEEE standards numbers used with RSTP over its history.

**NOTE** The IEEE sells its standards, but through the "Get IEEE 802" program, you can get free PDFs of the current 802 standards. To read about RSTP today, you will need to download the 802.1Q standard and then look for the sections about RSTP.

Now on to the details about RSTP in this chapter. As discussed throughout this chapter, RSTP and STP have many similarities, so this section next compares and contrasts the two. Following that, the rest of this section discusses the concepts unique to RSTP that are not found in STP—alternate root ports, different port states, backup ports, and the port roles used by RSTP.

## Comparing STP and RSTP

RSTP works just like STP in several ways, as discussed in the first major section of the chapter. To review:

Key Topic

- RSTP and STP elect the root switch using the same rules and tiebreakers.
- RSTP and STP switches select their root ports with the same rules.
- RSTP and STP elect designated ports on each LAN segment with the same rules and tiebreakers.
- RSTP and STP place each port in either forwarding or blocking state, although RSTP calls the blocking state the **discarding state**.

In fact, RSTP works so much like STP that they can both be used in the same network. RSTP and STP switches can be deployed in the same network, with RSTP features working in switches that support it and traditional STP features working in the switches that support only STP.

With all these similarities, you might be wondering why the IEEE bothered to create RSTP in the first place. The overriding reason is convergence. STP takes a relatively long time to converge (50 seconds with the default settings when all the wait times must be followed). RSTP improves network convergence when topology changes occur, usually converging within a few seconds (or in slow conditions, in about 10 seconds).

RSTP changes and adds to STP in ways that avoid waiting on STP timers, resulting in quick transitions from forwarding to discarding (blocking) state and vice versa. Specifically, RSTP,

compared to STP, defines more cases in which the switch can avoid waiting for a timer to expire, such as the following:

Key Topic

- RSTP adds a mechanism by which a switch can replace its root port, without any waiting to reach a forwarding state (in some conditions).
- RSTP adds a new mechanism to replace a designated port, without any waiting to reach a forwarding state (in some conditions).
- RSTP lowers waiting times for cases in which RSTP must wait for a timer.

For instance, imagine a failure case in which a link remains up, but for some reason, a nonroot switch stops hearing the Hello BPDUs it had been hearing in the past. STP requires a switch to wait for MaxAge seconds, which STP defines based on ten times the Hello timer, or 20 seconds, by default. RSTP shortens this timer, defining MaxAge as three times the Hello timer. Additionally, RSTP can send messages to the neighboring switch to inquire whether a problem has occurred rather than wait for timers.

The best way to get a sense for these mechanisms is to see how the RSTP alternate port role and the backup port role both work. RSTP uses the term *alternate port* to refer to a switch's other ports that could be used as the root port if the root port ever fails. The *backup port* concept provides a backup port on the local switch for a designated port. (Note that backup ports apply only to designs that use hubs, so they are unlikely to be useful today.) However, both are instructive about how RSTP works. Table 9-9 lists these RSTP port roles.

Key Topic

**Table 9-9** Port Roles in RSTP

| Function | Port Role |
|---|---|
| Nonroot switch's best path to the root | **Root port** |
| Port that will be used to replace the root port when the root port fails | **Alternate port** |
| Switch port designated to forward onto a collision domain | **Designated port** |
| Port that will be used to replace a designated port when a designated port fails | **Backup port** |
| Port in a nonworking interface state—that is, anything other than connected (up/up) | **Disabled port** |

RSTP differs from STP in a few other ways as well. For instance, with STP, the root switch creates a Hello, with all other switches updating and forwarding the Hello. With RSTP, each switch independently generates its own Hellos. Additionally, RSTP allows for queries between neighbors, rather than waiting on timers to expire, as a means to avoid waiting to learn information. These types of protocol changes help RSTP-based switches isolate what has changed in a network and react quickly to choose a net RSTP topology.

The next few pages work through some of those overt RSTP features that differ from STP.

## RSTP and the Alternate (Root) Port Role

With STP, each nonroot switch places one port in the STP root port (RP) role. RSTP follows that same convention, with the same exact rules for choosing the RP. RSTP then takes another step beyond STP, naming other possible RPs, identifying them as *alternate ports*.

To be an alternate port, both the RP and the alternate port must receive Hellos that identify the same root switch. For instance, in Figure 9-8, SW1 is the root. SW3 will receive Hello BPDUs on two ports: G0/1 and G0/2. Both Hellos list SW1's bridge ID (BID) as the root switch, so whichever port is not the root port meets the criteria to be an alternate port. SW3 picks G0/1 as its root port in this case and then makes G0/2 an alternate port.

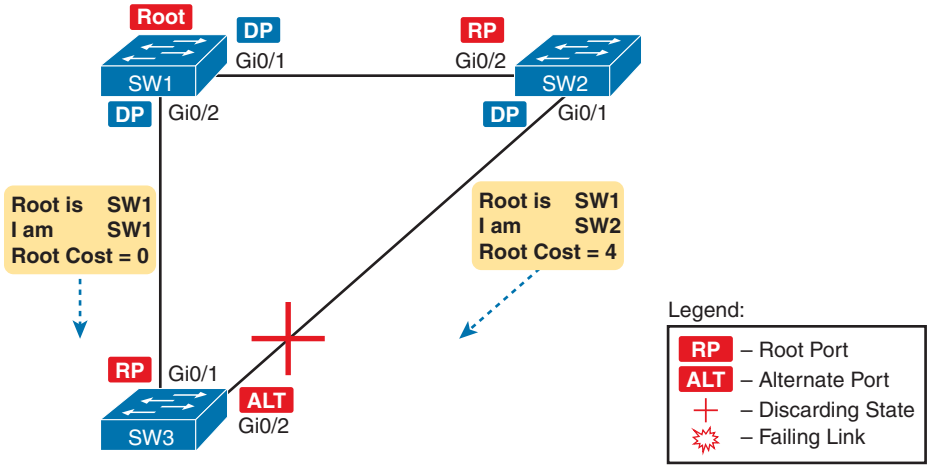



**Figure 9-8** *Example of SW3 Making G0/2 Become an Alternate Port*

An alternate port basically works like the second-best option for the root port. The alternate port can take over for the former root port, often very rapidly, without requiring a wait in other interim RSTP states. For instance, when the root port fails, the switch changes the former root port's role and state: (a) the role from root port to a *disabled port* (because the interface failed), and (b) the state from forwarding to discarding (RSTP uses the discarding state for ports in the disabled port role). Then, without waiting on any timers, the switch changes the alternate port to be the root port and immediately changes its port state from discarding to forwarding.

Notably, the new root port also does not need to spend time in other states, such as the learning state, instead moving immediately to the forwarding state.

Figure 9-9 shows an example of RSTP convergence. SW3's root port before the failure shown in this figure is SW3's G0/1, the link connected directly to SW1 (the root switch). Then SW3's link to SW1 fails as shown in Step 1 of the figure.

Following the steps in Figure 9-9:

**Step 1.** The link between SW1 and SW3 fails, so SW3's current root port (Gi0/1) fails.

**Step 2.** SW3 and SW2 exchange RSTP messages to confirm that SW3 will now transition its former alternate port (Gi0/2) to be the root port. This action causes SW2 to flush the required MAC table entries.

**Step 3.** SW3 transitions G0/1 to the disabled port role and G0/2 to the root port role.

**Step 4.** SW3 transitions G0/2 to a forwarding state immediately, without using the learning state, because this is one case in which RSTP knows the transition will not create a loop.

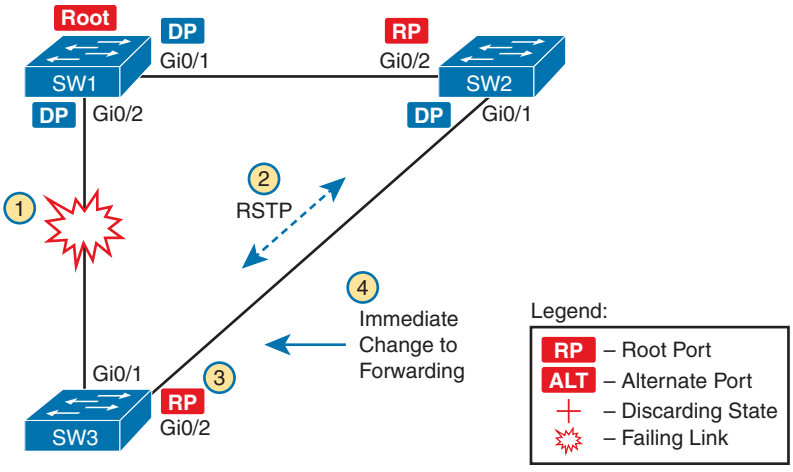



**Figure 9-9** *Convergence Events with SW3 G0/1 Failure*

As soon as SW3 realizes its G0/1 interface has failed, the process shown in the figure takes very little time. None of the processes rely on timers, so as soon as the work can be done, the convergence completes. (This particular convergence example takes about 1 second in a lab.)

## RSTP States and Processes

The depth of the previous example does not point out all the details of RSTP, of course; however, the example does show enough details to discuss RSTP states and internal processes.

Both STP and RSTP use *port states*; however, RSTP differs from STP in a few of the details. Both use the forwarding state for the same purpose. For the two interim states used by STP during convergence (listening and learning), RSTP does not use the listening state, considering it unnecessary, but it uses the learning state for the same purpose as STP. Finally, RSTP uses the *discarding* state instead of the STP *blocking* state.

To be complete, note that both RSTP and STP have port roles and states related to ports that are not currently working. Those rules are

- Both STP and RSTP define and use a disabled port role for any port in a nonworking interface state. (For instance, interfaces with no cable installed or interfaces configured with the **shutdown** command configured.)
- STP uses the disabled state for such ports. RSTP does not have a separate disabled state but instead uses the discarding state.

Honestly, the intricacies of the disabled role and states probably do not matter much; it seems obvious that nonworking interfaces cannot be a useful part of a working spanning tree. Regardless, Table 9-10 shows the list of STP and RSTP states for comparison and easier study.

Key Topic

**Table 9-10** Port States Compared: STP and RSTP

| Function | STP State | RSTP State |
|---|---|---|
| Port is not in a working (connected) state, either due to failure or due to shutdown | Disabled | Discarding |
| Stable state that ignores incoming data frames and is not used to forward data frames | Blocking | Discarding |
| Interim state without MAC learning and without forwarding | Listening | Not used |
| Interim state with MAC learning and without forwarding | Learning | Learning |
| Stable state that allows MAC learning and forwarding of data frames | Forwarding | Forwarding |

RSTP uses different internal processes compared to STP in an effort to speed convergence. As an example, consider what happens when STP changes a port's role, resulting in the need to move from a blocking to a forwarding state:

1. The switch changes the timer it uses to timeout MAC table entries to be equal to the STP forward delay timer (default 15 seconds).
2. The switch transitions the interface to the listening state, remaining there for the forward delay time (default 15 seconds).
3. After the forward delay time passes:
   a. All MAC table entries affected by this topology change will have been timed out of the table, removing the possibility of loops.
   b. The switch can now transition the port to the learning state, remaining there for the forward delay time.
4. After spending the forward delay time in the learning state, the switch transitions the port to the forwarding state.

To converge more quickly, RSTP avoids relying on timers. RSTP switches tell each other (using messages) that the topology has changed. Those messages also direct neighboring switches to flush the contents of their MAC tables in a way that removes all the potentially loop-causing entries, without a wait. As a result, RSTP creates more scenarios in which a formerly discarding port can immediately transition to a forwarding state, without waiting, and without using the learning state, as shown in the example in Figure 9-9.

## RSTP and the Backup (Designated) Port Role

The RSTP backup port role also improves convergence. It provides fast failover for the designated port role, but only in designs like Figure 9-10, when one switch connects with multiple ports to the same hub. Because moderns LANs do not use hubs, you will be unlikely to see this RSTP feature in practice, but to be ready for all RSTP features, consider this example.

Figure 9-10 shows a design in which switch SW3 connects with one port and SW4 connects with two ports to the same hub. SW4's port F0/1 happens to win the election as designated port (DP). In this topology, switch SW4's port F0/2 can act as a backup port.

With a backup port, if the current designated port fails, SW4 can start using the backup port as the new designated port with rapid convergence. For instance, if SW4's F0/1 interface were to fail, SW4 could transition F0/2 to the designated port role, without any delay in moving from the discarding state to a forwarding state.

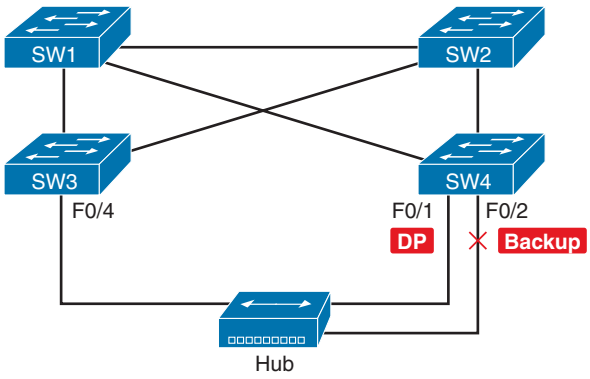



**Figure 9-10** *RSTP Backup Port Example*

## RSTP Port Types

The final RSTP concept included here relates to some terms RSTP uses to refer to different types of ports and the links that connect to those ports.

To begin, consider the basic image in Figure 9-11. It shows several links between two switches. RSTP considers these links to be point-to-point links and the ports connected to them to be point-to-point ports because the link connects exactly two devices (points).

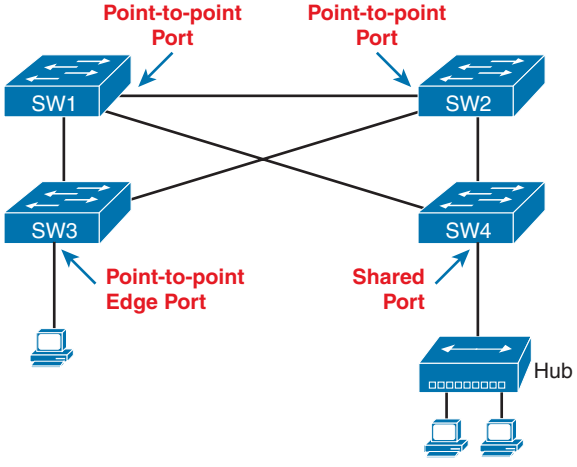



**Figure 9-11** *RSTP Link Types*

RSTP further classifies point-to-point ports into two categories. Point-to-point ports that connect two switches are not at the edge of the network and are simply called *point-to-point ports*. Ports that instead connect to a single endpoint device at the edge of the network, like a PC or server, are called *point-to-point edge ports*, or simply *edge ports*. In Figure 9-11, SW3's switch port connected to a PC is an edge port.

Finally, RSTP defines the term *shared* to describe ports connected to a hub. The term *shared* comes from the fact that hubs create a shared Ethernet; hubs also force the attached switch port to use half-duplex logic. RSTP assumes that all half-duplex ports may be

connected to hubs, treating ports that use half duplex as shared ports. RSTP converges more slowly on shared ports as compared to all point-to-point ports.

## Optional STP Features

To close out the chapter, the last major section describes optional features that make STP work even better or be more secure: EtherChannel, PortFast, BPDU Guard, Root Guard, and Loop Guard.

### EtherChannel

One of the best ways to lower STP's convergence time is to avoid convergence altogether. **EtherChannel** provides a way to prevent STP convergence from being needed when only a single port or cable failure occurs.

EtherChannel combines multiple parallel segments of equal speed (up to eight) between the same pair of switches, bundled into an EtherChannel. The switches treat the EtherChannel as a single interface with regard to STP. As a result, if one of the links fails, but at least one of the links is up, STP convergence does not have to occur. For example, Figure 9-12 shows the familiar three-switch network, but now with two Gigabit Ethernet connections between each pair of switches.

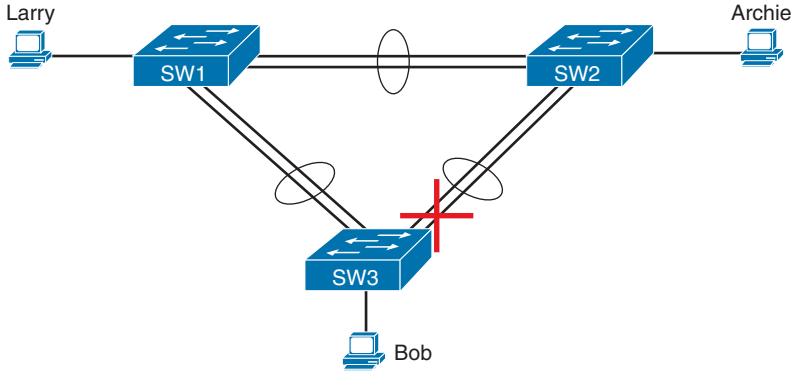



**Figure 9-12** *Two-Segment EtherChannels Between Switches*

With each pair of Ethernet links configured as an EtherChannel, STP treats each EtherChannel as a single link. In other words, both links to the same switch must fail for a switch to need to cause STP convergence. Without EtherChannel, if you have multiple parallel links between two switches, STP blocks all the links except one. With EtherChannel, all the parallel links can be up and working simultaneously while reducing the number of times STP must converge, making the network more available.

The current CCNA exam blueprint includes a topic for configuring both Layer 2 EtherChannels (as described here) and Layer 3 EtherChannels. Chapter 10, "RSTP and EtherChannel Configuration," shows how to configure Layer 2 EtherChannels, while Chapter 18, "IP Routing in the LAN," shows how to configure Layer 3 EtherChannels. Note that Layer 2 EtherChannels combine links that switches use as switch ports, with the switches using Layer 2 switching logic to forward and receive Ethernet frames over the EtherChannels. Layer 3 EtherChannels also combine links, but the switches use Layer 3 routing logic to forward packets over the EtherChannels.

## PortFast

Switch ports that connect directly to endpoints rather than to other switches eventually use the designated port (DP) role and a forwarding state. However, with default port settings, they go through some interim steps:

1. After the interface reaches the connected (up/up) state, the switch STP logic initially places the port in the STP discarding (blocking) state.
2. The switch begins sending BPDUs out of the port and listens for incoming BPDUs, to decide whether the switch port should take on a root port (RP) or a DP role.
3. Because the connected device is not a switch, the local switch receives no incoming BPDUs. As a result, the switch port wins the DP election and becomes a DP.
4. The switch takes interim steps to transition to a forwarding state:
   a. STP moves the port first to the listening state, then to the learning state, and then to the forwarding state. The interim states require 15 seconds each by default (per the default Forward Delay timer).
   b. RSTP follows the same process, except it has no listening state. It moves from discarding to learning to forwarding.

**PortFast** bypasses the process. After the interface reaches a connected state, STP PortFast immediately moves the port to the DP role and the forwarding state without delay. As long as the interface remains up, the STP role remains as DP with a forwarding state.

PortFast exists to support access links connected to endpoints, as shown in Figure 9-13. You know the switch always wins the DP election on those links because the endpoints do not use STP. Those endpoints can include single computers, phones with connected computers, or a trunk connected to a server.

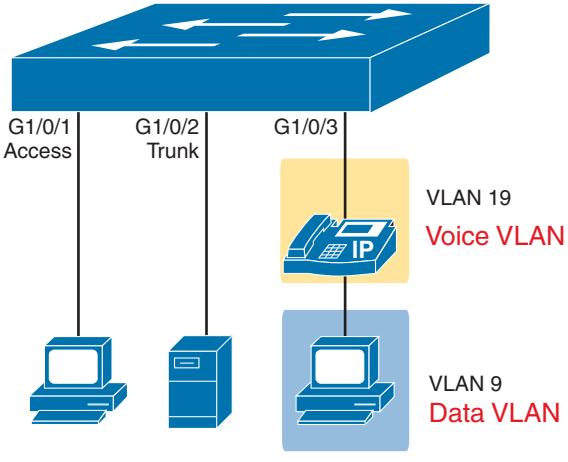



**Figure 9-13** *Access Ports Appropriate for the PortFast Feature*

As you might guess from the fact that PortFast speeds convergence, STP did not include the PortFast feature, but RSTP included it as one of its rapid convergence optimizations. You might recall the mention of RSTP port types, particularly point-to-point edge port types, around Figure 9-11. RSTP, by design of the protocol, converges quickly on these ports of

type point-to-point edge by bypassing the learning state. Note that Cisco introduced this idea with its PortFast years before the IEEE finalized RSTP, which is why we still call the feature PortFast many decades later. In practice, Cisco switches enable RSTP point-to-point edge ports by enabling PortFast on the port.

While very useful and popular, PortFast also introduces the risk of creating loops. PortFast ports should never connect to bridges or switches. With PortFast enabled, the port ignores incoming BPDUs, always acts as a designated port, and always forwards. As a result, an unexpected or rogue switch connected to a PortFast-enabled port can create a forwarding loop.

## BPDU Guard

PortFast creates an exposure: the possibility of a forwarding loop if an unexpected switch connects to a PortFast port. The Cisco **BPDU Guard** feature helps defeat these kinds of problems by disabling a port if it receives any BPDUs in the port. So, this feature is handy on ports that should be used only as an access port and never connected to another switch.

Figure 9-14 shows the basic idea. The device on the right should be an endpoint device such as a PC. Instead, a new switch named Rogue-1 connects to the legitimate switch SW3—a switch that uses BPDU Guard to protect against this scenario. At Step 2, as soon as switch Rogue-1 sends a BPDU into SW3, SW3's BPDU Guard logic reacts, disabling the port.

Key Topic

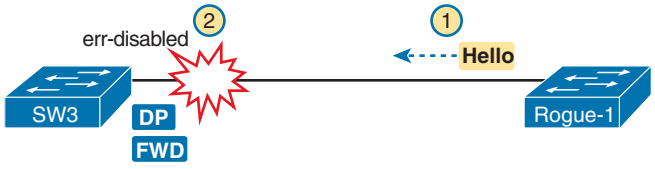



**Figure 9-14** *Basic Logic of BPDU Guard*

Most switch access port configurations combine PortFast and BPDU Guard, although both can be used independently. Using PortFast without protection against the scenario in Figure 9-14 risks a loop. Using BPDU Guard on these same ports makes sense because if another switch connects to such a port, the local switch can disable the port to prevent a loop.

## BPDU Filter

The **BPDU Filter** feature defines two different functions under one named feature. Both fall under the name BPDU filter because, behind the scenes, they filter (discard) STP BPDUs.

First, the BPDU Filter feature can be used as an alternative to BPDU Guard when used along with PortFast. Both BPDU Guard and BPDU Filter prevent loops when unexpected BPDUs arrive in a PortFast-enabled port. However, BPDU Filter reacts to those incoming BPDUs by disabling PortFast logic (which creates the possibility of a forwarding loop) and restoring normal STP logic on the port (which prevents the possibility of a forwarding loop).

The other BPDU Filter feature disables STP on an interface. When enabled, BPDU Filter discards all sent and received BPDUs on an interface.

The following pages take a closer look at each logic branch.

### BPDU Filter to Prevent Loops on PortFast Ports

Most campus LANs make widespread use of PortFast, both on access ports and on server ports that use trunking. However, with PortFast alone but with no other features, you risk someone unplugging those devices and replacing them with switches to create a forwarding loop. Figure 9-15 shows such a case, where switch SW1's two ports use PortFast but the endpoints were replaced with two rogue switches.

Key Topic

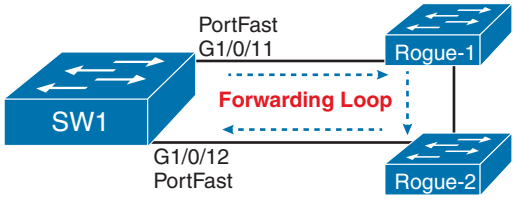



**Figure 9-15** *Possible Loop on PortFast Ports with Rogue Switches*

To protect against such a case, you can use either BPDU Guard or BPDU Filter. Ports that use PortFast should not receive BPDUs under normal circumstances, but nothing prevents people from replacing devices as shown in Figure 9-15. If that happens, BPDU Filter reacts by disabling PortFast on the port. The port then reverts to normal STP rules. STP will then determine a new port role and status as needed, use STP rules, and prevent any forwarding loop. The port may block or forward, depending on the results of that normal STP logic.

The previous paragraph gets at the reason why an engineer might choose this BPDU Filter feature; however, the following list details the specific steps, which includes some BPDU filtering.

Key Topic

1. To enable this branch of BPDU Filter logic, use the related global command **spanning-tree portfast bpdufilter default**.
2. IOS finds all ports that currently use PortFast and enables conditional BPDU Filter on those ports.
3. When any such port comes up:
   a. BPDU Filter allows STP to send BPDUs as normal for 10 × Hello Time (2 × 10 seconds, or 20 seconds total by default).
   b. If the port receives zero BPDUs in that time, BPDU Filter begins filtering (discarding) sent BPDUs.
4. BPDU Filter monitors to notice any incoming BPDUs. If they begin to arrive:
   a. It disables PortFast on the port.
   b. It reverts the port to use STP with normal STP rules.

### BPDU Filter to Disable STP on a Port

The BPDU Filter feature can be configured to enable a completely different style of logic. As you will see in Chapter 10's section titled "BPDU Filter," the configuration does not make it obvious which style the configuration enables. However, when enabled with an interface subcommand, BPDU Filter does just what the name says: It filters (discards) all BPDUs on the ports, both outgoing and incoming.

Using BPDU Filter in this way effectively disables STP on the port. Using this feature is dangerous: Do not use it in production until you fully test and understand it.

As an example of why you might want to use BPDU Filter to disable STP on a port, consider the scenario in Figure 9-16. Two IT groups build networks inside the same company. They want to connect their LANs together, but the two groups (Left and Right) do not want the other group's STP to influence theirs—for example, which switch would be allowed to win the root election, a left or a right switch? The two groups meet and decide to keep their STP topologies separate. To do so, they filter all BPDUs sent between the two using BPDU Filter, disabling STP on the one link that connects the two LANs.

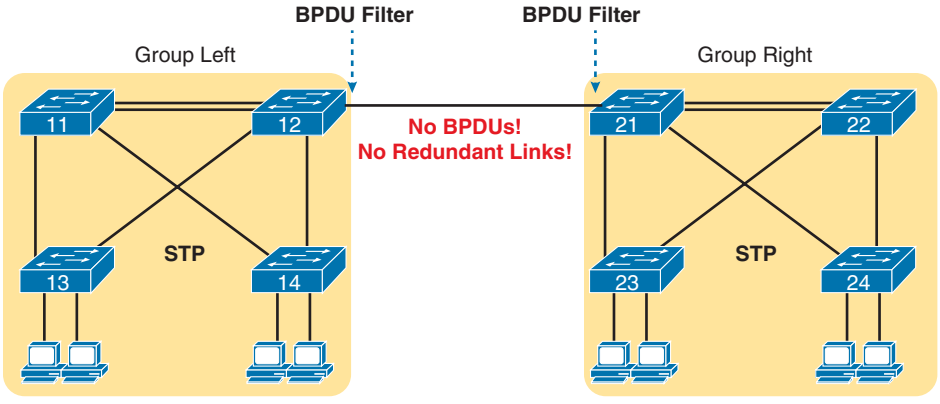



**Figure 9-16** *A Scenario to Consider Disabling STP on a Link (BPDU Filtering)*

## Root Guard

The **Root Guard** feature monitors incoming BPDUs on a port and reacts if the received BPDU would change the root switch in the VLAN. To understand why that might be useful, consider the topology with six switches in Figure 9-17. The notation by each switch defines how good the priority is on each switch. If you configure the switches with different priorities, you know and control which switches will become the root switch, with switches SW5 and SW6 being the least likely.

Key Topic

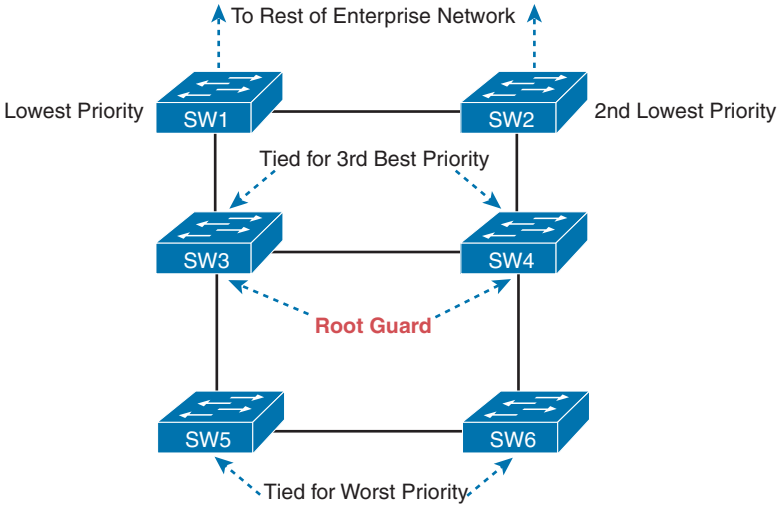



**Figure 9-17** *Larger LAN Showing Most and Least Likely Root Switches*

Now imagine that your organization planned the design in the figure, but you directly control switches SW1 through SW4 while another organization controls switches SW5 and SW6. Both groups agree that SW1 should be the root switch, with SW2 taking over if SW2 fails; however, that agreement does not prevent a configuration mistake on switch SW5 or SW6.

For example, someone in the other group could configure switch SW5 with a priority lower than SW1's priority, and SW5 would become the root switch, creating a suboptimal STP topology. Switch SW5 would send a superior BPDU into the network if such an unplanned configuration occurred. With this topology, SW3 would receive the superior BPDU. Similarly, SW4 might receive a superior BPDU from SW6 if someone lowered switch SW6's priority.

Root Guard, enabled per port, enables the port to operate normally with STP except in this one special case: when it receives a **superior BPDU** in the port. So, you enable Root Guard on ports connected to other switches, usually trunks, and STP works normally; however, Root Guard on a port also prevents the election of a different root switch that is reachable through that port. Root Guard achieves that by listening for incoming Hellos on the port, and if it is a superior Hello, the switch disables the port by using a special STP port state (the **broken state**). Root Guard recovers the port automatically when the superior Hellos no longer occur.

The choice to use Root Guard begins with a close analysis of the topology. You must agree which switches should never become root and which ports should never receive superior BPDUs.

## Loop Guard

The final optional tool mentioned here, Loop Guard, protects against a specific case in an STP topology. This feature helps protect the worst priority switch (highest BID) in a typical STP design, such as switch SW3 in Figure 9-18. The mechanisms apply to regular operation in a typical STP design, forcing some review of topics from earlier in the chapter.

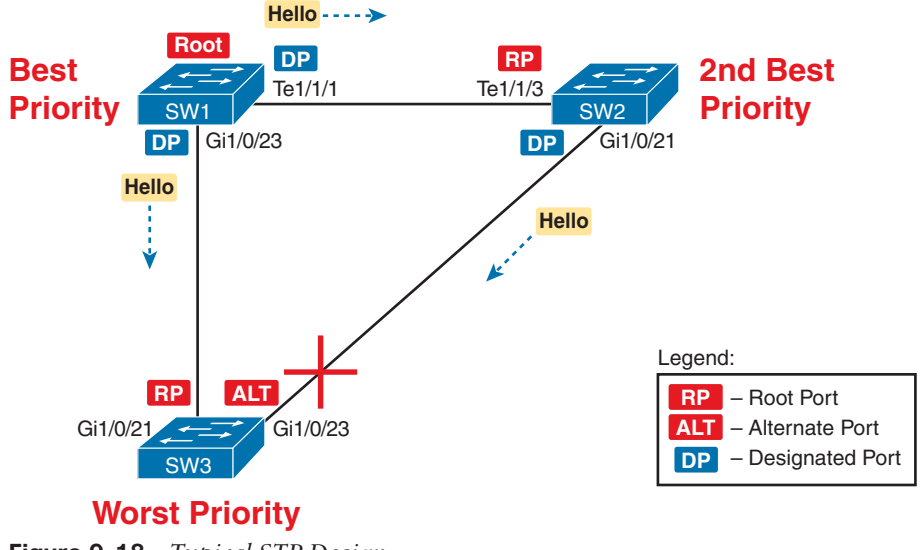



**Figure 9-18** *Typical STP Design*

A typical STP design uses one switch as the known root under normal circumstances. To achieve that, you assign that switch the lowest priority. Another switch has the role of being the best backup, so you assign it the second-best priority value. Switches SW1 and SW2 play those roles in Figure 9-18. All the other switches have a lower priority than SW1 and SW2, such as switch SW3.

Next, consider an observation about this design, about the worst switch (SW3). SW3's ports on its switch-to-switch links seldom become designated ports. Why? STP determines the DP based on lowest root cost of the switches on the link, and breaks ties based on the better (lower) BID. SW3 loses the DP election tiebreaker in this scenario.

However, SW3's ports have other roles, just not the DP role, as follows:

- One link will be its root port (G1/0/21 in Figure 9-18) and settle into a forwarding state.
- The other will be an alternate port (G1/0/23 in Figure 9-18) and settle into a discarding state.

The second meaningful observation is that the worst switch (SW3) receives repeated Hellos in both switch-to-switch links. The figure shows the Hellos. The rule is that switches forward Hellos out designated ports every Hello time (the default is 2 seconds).

The third fact that helps you understand Loop Guard involves unidirectional links. Figure 9-19 shows the idea. The interfaces on the ends of a fiber link will be up and in a connected state, but one of the two fibers has a problem. The problem could prevent frames from arriving at the other end of the fiber. Figure 9-19 shows such a case, with only the right-to-left direction working.

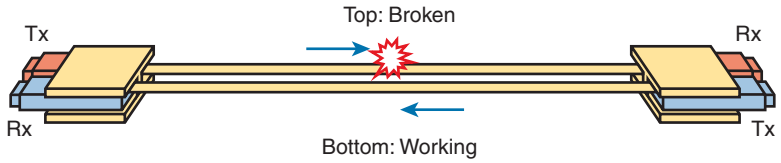



**Figure 9-19** *Unidirectional Fiber Link*

To summarize, Loop Guard relies on these underlying facts:

- A switch with the worst (highest) priority in a design has switch-to-switch ports in the root port (RP) and alternate port (ALT) roles but seldom the designated port (DP) role.
- Those same ports normally receive Hellos every Hello time because each link's neighboring switch is the DP.
- A **unidirectional link** can occur, in which the interface state remains up (connected) on both ends but frames cannot flow in one direction.

The **Loop Guard** feature takes advantage of the above observations to protect against a class of failures that, without Loop Guard, results in one of switch SW3's switch-switch links

becoming a DP. Figure 9-20 shows an adjustment to Figure 9-18 for one scenario, with the SW2-SW3 link becoming unidirectional:

1. The SW2-SW3 link's fiber for transmitting from SW2 to SW3 fails.
2. Because SW2's G1/0/21 and SW3's G1/0/23 ports remain in a connected state, there is no interface failure to influence STP.
3. SW3 ceases to receive incoming Hello messages on its G1/0/23 port.
4. After the appropriate timeouts, SW3 believes it is the only switch on the link connected to its G1/0/23 interface, so it changes the port to a DP and moves toward a forwarding state.

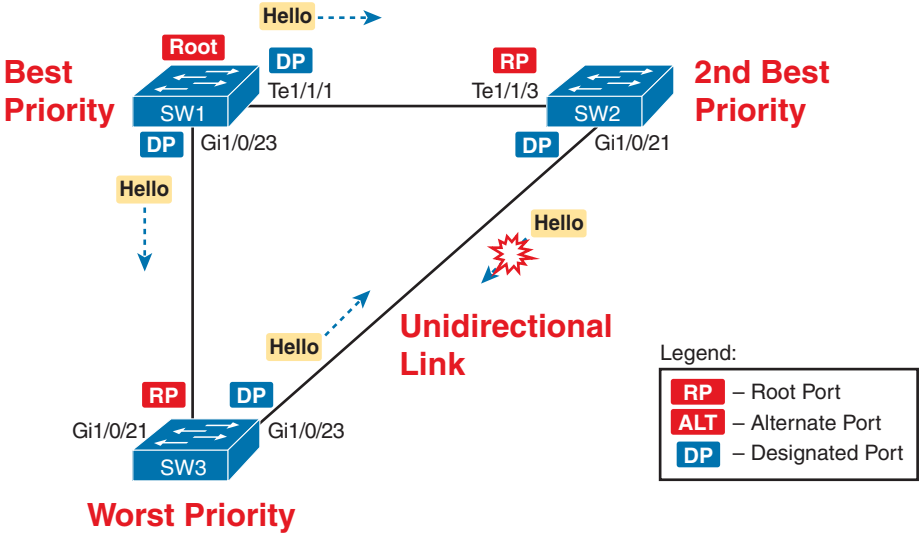



**Figure 9-20** *SW3's ALT Port Becomes DP, Forwards; All Links Forward, Create Loop*

Take a moment and look at the ends of the switch-to-switch links. All ports are either an RP or DP, both of which use a forwarding state. As a result, a forwarding loop exists. Note that with the unidirectional link between switches SW2 and SW3, broken in the direction from SW2 to SW3, the forwarding loop exists only in the opposite (counter-clockwise) direction.

Finally, you can now appreciate the Loop Guard feature. You enable it on an interface and it takes the following actions:

Key Topic

> If the port is a root or alternate port, prevent it from becoming a designated port by moving it to the special broken STP state.

As with Root Guard, you must work through the underlying concepts to decide the ports on which to enable Loop Guard. Generally, you need it only on fiber-optic links connected to other switches. You should also only choose ports on switches with a poor (high) STP priority, so they normally have switch-to-switch ports in the RP or ALT roles but not the DP role.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 9-11 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 9-11** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Key Topic

**Table 9-12** Key Topics for Chapter 9

| Key Topic Element | Description | Page Number |
|---|---|---|
| Table 9-2 | Lists the three main problems that occur when not using STP in a LAN with redundant links | 227 |
| Figure 9-2 | How STP blocks to break a loop | 227 |
| Table 9-3 | Lists the reasons why a switch chooses to place an interface into forwarding or blocking state | 229 |
| Table 9-4 | Lists the most important fields in Hello BPDU messages | 230 |
| List | Logic for the root switch election | 231 |
| Figure 9-6 | Shows how switches calculate their root cost | 233 |
| Table 9-6 | Lists the original and current default STP port costs for various interface speeds | 236 |
| Step list | A summary description of steady-state STP operations | 236 |
| Table 9-7 | STP timers | 237 |
| List | Definitions of what occurs in the listening and learning states | 239 |
| Table 9-8 | Summary of STP and RSTP states | 239 |
| List | Key similarities between 802.1D STP and 802.1w RSTP | 240 |
| List | RSTP mechanisms for faster convergence compared to STP | 241 |
| Table 9-9 | List of 802.1w port roles | 241 |
| Table 9-10 | Comparisons of port states with 802.1D and 802.1w | 244 |
| Figure 9-14 | Basic logic for BPDU Guard | 248 |
| Figure 9-15 | An example forwarding loop risk with PortFast | 249 |
| List | Conditional BPDU Filter logic applied to PortFast ports | 249 |
| Figure 9-17 | Locations to apply Root Guard | 250 |
| Paragraph | Loop Guard rules | 253 |

## Key Terms You Should Know

alternate port (role), backup port (role), blocking state, BPDU Filter, BPDU Guard, bridge ID, bridge protocol data unit (BPDU), broken state, designated port, designated port (role), disabled port (role), disabled state, discarding state, EtherChannel, forward delay, forwarding state, Hello BPDU, learning state, listening state, Loop Guard, MaxAge, PortFast, Rapid STP (RSTP), root cost, Root Guard, root port (role), root switch, Spanning Tree Protocol (STP), superior BPDU, unidirectional link



CHAPTER 10

# RSTP and EtherChannel Configuration

**This chapter covers the following exam topics:**

- **2.0 Network Access**
  - **2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP)**
  - **2.5 Interpret basic operations of Rapid PVST+ Spanning Tree Protocol**
    - **2.5.a Root port, root bridge (primary/secondary), and other port names**
    - **2.5.b Port states and roles**
    - **2.5.c PortFast**
    - **2.5.d Root guard, loop guard, BPDU filter, and BPDU guard**

This chapter shows how to configure Rapid Spanning Tree Protocol (RSTP) and Layer 2 EtherChannels.

The first two sections take a little different approach than many other CLI-focused parts of the book, based on exam topic 2.5. That exam topic begins with the phrase "interpret basic operations," which emphasizes the concepts behind what happens in the CLI. The first major section examines STP concepts using some configuration, with much focus on interpreting **show** command output. The second major section, about exam topics 2.5.c and 2.5.d, takes the same approach.

The EtherChannel content, in the final major section of the chapter, follows a typical flow for most configuration/verification topics in a certification guide: it reviews concepts, shows configurations, followed by **show** commands that point out the configuration settings and operational state. The details include how to manually configure a channel, how to cause a switch to dynamically create a channel, and how EtherChannel load distribution works.

I encourage you to practice and experiment with STP from the CLI. If you're looking for lab exercises, check out the Config Labs at my blog site related to this chapter, as detailed in the introduction to this book. (Or, just go to certskills.com/config-labs for more info.)

Finally, like Chapter 9, this chapter is longer than I prefer. If you like to think of each chapter as one study session, you might need to think about splitting this chapter into two study sessions. Stop the first study session just as you reach the third major section, titled "Configuring Layer 2 EtherChannel." The second study session would consist of the "Configuring Layer 2 EtherChannel" section.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom

of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 10-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Understanding RSTP Through Configuration | 1–3 |
| Identifying Optional STP Features | 4–5 |
| Configuring Layer 2 EtherChannel | 6–8 |

1. Which type value on the **spanning-tree mode** *type* global command enables the use of RSTP?
   a. **rapid-pvst**
   b. **pvst**
   c. **rstp**
   d. **rpvst**
2. Examine the following output from the **show spanning-tree vlan 5** command, which describes a root switch in a LAN. Which answers accurately describe facts related to the root's bridge ID? (Choose two answers.)

```
SW1# show spanning-tree vlan 5

VLAN0005
   Spanning tree enabled protocol rstp
   Root ID Priority 32773
            Address     1833.9d7b.0e80
            Cost        15
            Port        25 (GigabitEthernet0/1)
            Hello Time 2 sec Max Age 20 sec Forward Delay 15 sec
```

   a. The system ID extension value, in decimal, is 5.
   b. The root's configured priority value is 32773.
   c. The root's configured priority value is 32768.
   d. The system ID extension value, in hexadecimal, is 1833.9d7b.0e80.
3. With Cisco's RPVST+, which of the following actions does a switch take to identify which VLAN is described by a BPDU? (Choose three answers.)
   a. Adds a VLAN tag when forwarding a BPDU on trunks
   b. Adds the VLAN ID in an extra TLV in the BPDU
   c. Lists the VLAN ID as the middle 12 bits of the System ID field of the BPDU
   d. Lists the VLAN ID in the System ID Extension field of the BPDU

4. A switch port in access mode connects to a single laptop. An attacker replaces the laptop with a LAN switch that uses STP. Which combination of features, enabled on the access port, results in the switch changing the port's interface state to err-disabled? (Choose two answers.)
   a. PortFast enabled but BPDU Guard not enabled
   b. PortFast and BPDU Guard enabled
   c. PortFast disabled but BPDU Guard enabled
   d. PortFast and BPDU Guard disabled
5. Root Guard has acted to disable port G1/0/1. An engineer uses the **show interfaces status** and **show spanning-tree vlan 1** commands to investigate the current status. Which answers list facts expected due to Root Guard's actions? (Choose two answers.)
   a. The STP port state per **show spanning-tree vlan 1** shows BKN (broken).
   b. The interface state per **show interfaces status** shows err-disabled.
   c. The STP port state per **show spanning-tree vlan 1** shows FWD (forwarding).
   d. The STP port state per **show spanning-tree vlan 1** shows BLK (blocking).
   e. The interface state per **show interfaces status** shows connected.
6. An engineer configures a switch to put interfaces G0/1 and G0/2 into the same Layer 2 EtherChannel. Which of the following terms is used in the configuration commands?
   a. **EtherChannel**
   b. **PortChannel**
   c. **Ethernet-Channel**
   d. **Channel-group**
7. Which combinations of keywords on the **channel-group** interface subcommand on two neighboring switches will cause the switches to use LACP and attempt to add the link to the EtherChannel? (Choose two answers.)
   a. **desirable** and **active**
   b. **passive** and **active**
   c. **active** and **auto**
   d. **active** and **active**
8. A Cisco Catalyst switch needs to send frames over a Layer 2 EtherChannel. Which answer best describes how the switch balances the traffic over the four active links in the channel?
   a. Breaks each frame into fragments of approximately one-fourth of the original frame, sending one fragment over each link
   b. Sends the entire frame over one link, alternating links in sequence for each successive frame
   c. Sends the entire frame over one link, choosing the link by applying some math to fields in each frame's headers
   d. Sends the entire frame over one link, using the link with the lowest percent utilization as the next link to use

# Foundation Topics

## Understanding RSTP Through Configuration

Cisco IOS switches today typically default to using RSTP rather than STP, with default settings so that RSTP works with no configuration. You can buy some Cisco switches and connect them with Ethernet cables in a redundant topology, and RSTP will ensure that frames do not loop. And even if some switches use RSTP and some use STP, the switches can interoperate and still build a working spanning tree—and you never even have to think about changing any settings!

Although RSTP works without any configuration, most medium-size to large-size campus LANs benefit from some RSTP configuration. For instance, Figure 10-1 shows a typical LAN design model, with two distribution layer switches (D1 and D2). The design may have dozens of access layer switches that connect to end users; the figure shows just three access switches (A1, A2, and A3). For a variety of reasons, most network engineers make the distribution layer switches be the root.

Key Topic

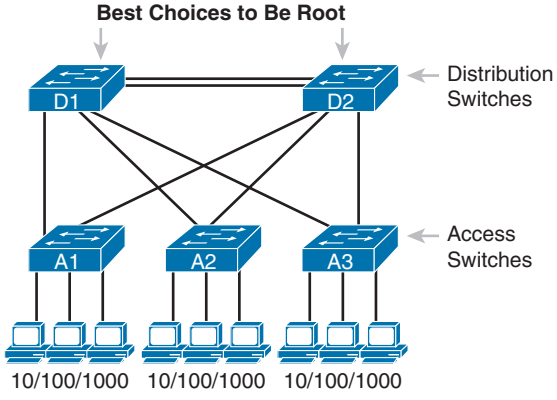



**Figure 10-1** *Typical Configuration Choice: Making Distribution Switch Be Root*

**NOTE** Cisco uses the term *access switch* to refer to switches used to connect to endpoint devices. The term *distribution switch* refers to switches that do not connect to endpoints but rather connect to each access switch, providing a means to distribute frames throughout the LAN. The term *uplink* refers to the switch-to-switch links, usually trunks, between access and distribution switches. If you want to read more about LAN design concepts and terms, refer to the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, Chapter 17, "LAN Architecture."

As discussed in the introduction to this chapter, this first section of the chapter examines a variety of STP/RSTP configuration topics, but with a goal of revealing a few more details about how STP/RSTP operate. Following this opening section about STP/RSTP configuration, the next section examines how to configure Layer 2 EtherChannels and how that impacts STP/RSTP.

## The Need for Multiple Spanning Trees

The IEEE first standardized STP as the IEEE 802.1D standard, first published back in 1990. To put some perspective on that date, Cisco did not have a LAN switch product line at the time, and virtual LANs did not exist yet. Instead of multiple VLANs in a physical Ethernet LAN, the physical Ethernet LAN existed as one single broadcast domain, with one instance of STP.

By the mid-1990s, VLANs had appeared on the scene, along with LAN switches. The emergence of VLANs posed a challenge for STP—the only type of STP available at the time—because STP defined a single common spanning tree (CST) topology for the entire LAN. The IEEE needed an option to create multiple spanning trees so that traffic could be balanced across the available links, as shown in Figure 10-2. With two different STP instances, SW3 could block on a different interface in each VLAN, as shown in the figure.

Key Topi

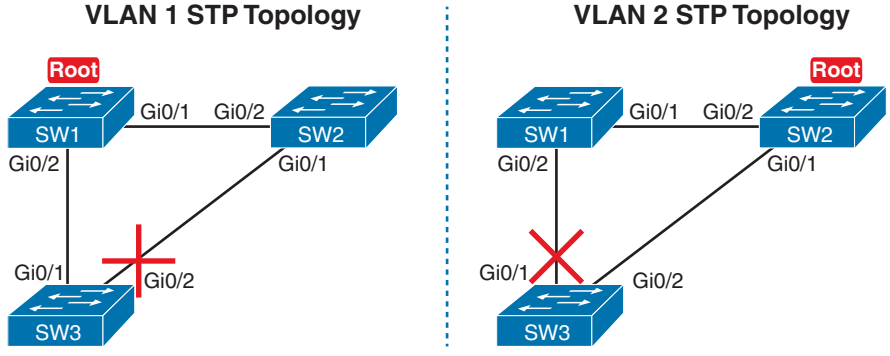



**Figure 10-2** *Load Balancing with One Tree for VLAN 1 and Another for VLAN 2*

## STP Modes and Standards

Because of the sequence of events over the history of the various STP family of protocols, vendors like Cisco needed to create their own proprietary features to create the per-VLAN spanning tree concept shown in Figure 10-2. That sequence resulted in the following:

- When STP was the only STP standard back in the 1990s with 802.1D, Cisco created the STP-based Per VLAN Spanning Tree Plus (**PVST+**) protocol, which creates one spanning tree instance per VLAN.
- When the IEEE introduced RSTP (in 802.1D amendment 802.1w, in 2001), they again defined it as a means to create a single spanning tree.
- When Cisco added support for RSTP to its switches, it created the **Rapid PVST+** (RPVST+) protocol. RPVST+ provided more features than standardized RSTP, including one tree per VLAN.
- To create multiple spanning trees, the IEEE did not adopt Cisco's PVST+ or RPVST+. Instead, the IEEE created a different method: Multiple Spanning Tree Protocol (MSTP), originally defined in 802.1Q amendment 802.1s.

---

Answers to the "Do I Know This Already?" quiz:

**1** A **2** A, C **3** A, B, D **4** B, C **5** A, E **6** D **7** B, D **8** C

Figure 10-3 shows the features as a timeline for perspective.

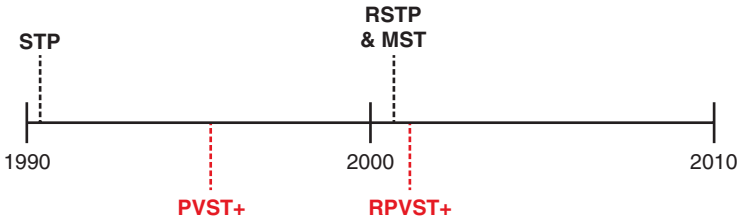



**Figure 10-3** *Timeline of Per-VLAN and Multiple STP Features*

Today, Cisco Catalyst switches give us three options to configure on the **spanning-tree mode** command, which tells the switch which type of STP to use. Note that the switches do not support STP or RSTP with the single tree (CST). They can use either the Cisco-proprietary and STP-based PVST+, Cisco-proprietary and RSTP-based RPVST+, or the IEEE standard MSTP. Table 10-2 summarizes some of the facts about these standards and options, along with the keywords used on the **spanning-tree mode** global configuration command. Example 10-1, which follows, shows the command options in global configuration mode.

Key Topic

**Table 10-2** STP Standards and Configuration Options

| Name | Based on STP or RSTP? | # Trees | Original IEEE Standard | Config Parameter |
|---|---|---|---|---|
| STP | STP | 1 (CST) | 802.1D | N/A |
| PVST+ | STP | 1/VLAN | 802.1D | **pvst** |
| RSTP | RSTP | 1 (CST) | 802.1w | N/A |
| Rapid PVST+ | RSTP | 1/VLAN | 802.1w | **rapid-pvst** |
| MSTP | RSTP | 1 or more* | 802.1s | **mst** |

* MSTP allows the definition of as many instances (multiple spanning tree instances, or MSTIs) as chosen by the network designer but does not require one per VLAN.

**Example 10-1** *Cisco Switch Spanning Tree modes*

```
SW1(config)# spanning-tree mode ?
  mst         Multiple spanning tree mode
  pvst        Per-Vlan spanning tree mode
  rapid-pvst  Per-Vlan rapid spanning tree mode
SW1(config)#
```

## The Bridge ID and System ID Extension

To support the idea of multiple spanning trees, whether one per VLAN or simply multiple as created with MSTP, the protocols must consider the VLANs and VLAN trunking. (That's one reason why RSTP and MSTP now exist as part of the 802.1Q standard, which defines VLANs and VLAN trunking.) To help make that work, the IEEE redefined the format of the original BID value to help make per-VLAN instances of STP/RSTP become a reality.

Originally, a switch's BID was formed by combining the switch's 2-byte priority and its 6-byte MAC address. The IEEE later revised the 2-byte priority field as shown in

Figure 10-4 as a 4-bit priority field and a 12-bit subfield called the **system ID extension** (which represents the VLAN ID).

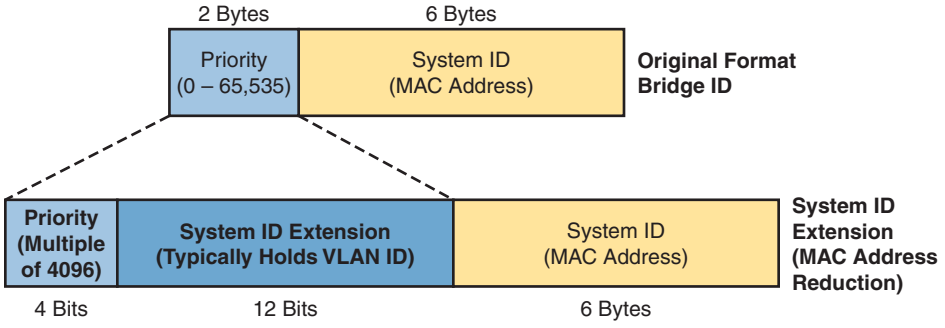



**Figure 10-4** *STP System ID Extension*

Cisco switches enable you to configure only the priority part of the BID. The switch fills in its universal (burned-in) MAC address as the system ID. It also plugs in the VLAN ID of a VLAN in the 12-bit system ID extension field; you cannot change that behavior either.

However, configuring the priority field may be one of the strangest things to configure on a Cisco router or switch. Focusing on the top of Figure 10-4, the priority field was originally a 16-bit number, which represented a decimal number from 0 to 65,535. Because of that history, the configuration command (**spanning-tree vlan** *vlan-id* **priority** *x*) requires a decimal number between 0 and 65,535. However, because the modern use of this field reserves the final 12 bits for the VLAN ID, IOS restricts the command to multiples of 4096. Table 10-3 shows the reason: The allowed decimal values, when viewed as 16-bit binary values, have all zeros in the final 12 bits.

**Table 10-3** STP/RSTP Configurable Priority Values

| Decimal Value | 16-bit Binary Equivalent | Decimal Value | 16-bit Binary Equivalent |
|---|---|---|---|
| 0 | **0000** 0000 0000 0000 | 32768 | **1000** 0000 0000 0000 |
| 4096 | **0001** 0000 0000 0000 | 36864 | **1001** 0000 0000 0000 |
| 8192 | **0010** 0000 0000 0000 | 40960 | **1010** 0000 0000 0000 |
| 12288 | **0011** 0000 0000 0000 | 45056 | **1011** 0000 0000 0000 |
| 16384 | **0100** 0000 0000 0000 | 49152 | **1100** 0000 0000 0000 |
| 20480 | **0101** 0000 0000 0000 | 53248 | **1101** 0000 0000 0000 |
| 24576 | **0110** 0000 0000 0000 | 57344 | **1110** 0000 0000 0000 |
| 28672 | **0111** 0000 0000 0000 | 61440 | **1111** 0000 0000 0000 |

Example 10-2 shows how to configure the priority setting for each VLAN. Note that switches default to a base priority of 32,768.

**Example 10-2** *Help Shows Requirements for Using Increments of 4096 for Priority*

```
SW1(config)# spanning-tree vlan 1 priority ?
  <0-61440>  bridge priority in increments of 4096
SW1(config)#
```

## Identifying Switch Priority and the Root Switch

Cisco Catalyst switches configure the priority value using a number that represents a 16-bit value; however, the system ID extension exists as the low-order 12 bits of that same number. This next topic works through connecting those ideas.

When the switch builds its BID to use for RSTP in a VLAN, it must combine the configured priority with the VLAN ID of that VLAN. Interestingly, the configured priority results in a 16-bit priority that always ends with 12 binary 0s. That fact makes the process of combining values to create the BID a little simpler for the switch and possibly a little simpler for network engineers once you understand it all.

First, consider the process shown in Figure 10-5. The top shows the configured priority value (decimal 32768), in 16-bit binary form, with a System ID Extension of 12 zeros. Moving down the figure, you see the binary version of a VLAN ID (decimal 9). At the last step, the switch replaces those last 12 bits of the System ID Extension with the value that matches the VLAN ID and uses that value as the first 16 bits of the BID.

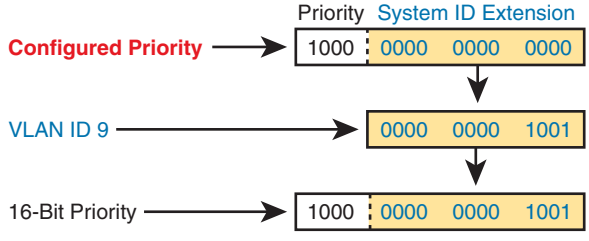



**Figure 10-5** *Configured Priority (16-Bit) and System ID Extension (12-Bit) Added*

### Switch Priority and Identifying the Root Switch

In most STP designs, you favor two switches to become the root switch. Often, the design calls for one switch to be the root switch, with a second switch ready to take over as root if the first switch fails. Figure 10-6 shows the idea, with switch SW1 as the primary and switch SW2 as the secondary. Additionally, note that the figure uses the interface type abbreviation of Te for TenGigabitEthernet. Also, the switch shows only links between switches to simplify the discussion.

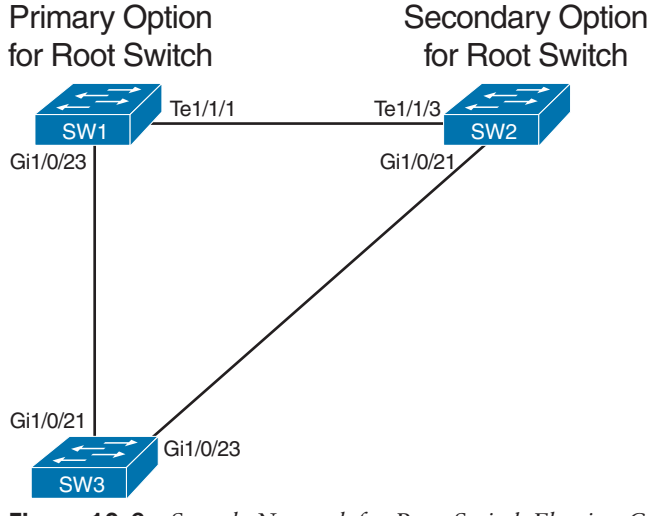



**Figure 10-6** *Sample Network for Root Switch Election Configuration*

Cisco switches default to use a default base priority of 32,768. To achieve the STP goals in the figure, the engineer needs to lower switch SW2's priority to lower than the default value and the value on switch SW1 to even lower. For example, to do just that for the STP instance for VLAN 9, the engineer could do the following:

- **On SW1:** Configure the **spanning-tree vlan 9 priority 24576** global command.
- **On SW2:** Configure the **spanning-tree vlan 9 priority 28672** global command.
- **On SW3:** Rely on the default base priority of 32,768.

The **show spanning-tree vlan 9** command, featured in Example 10-3, shows many facts about STP operation. Of particular importance, it lists about five lines of output about the root switch, starting with the line that begins with "Root ID." Following that, it lists several lines about the local switch: the switch on which this command was run. In this case, the output comes from nonroot switch SW3, so the section about the root switch refers to another switch (SW1).

**Example 10-3** *Examining the 16-bit Priority from Nonroot Switch SW3*

```
SW3# show spanning-tree vlan 9
VLAN0009
  Spanning tree enabled protocol rstp
  Root ID     Priority    24585
              Address     4488.165a.f200
              Cost        4
              Port        21 (GigabitEthernet1/0/21)
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID   Priority    32777  (priority 32768 sys-id-ext 9)
              Address     5cfc.6608.2880
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
              Aging Time  300 sec

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/1             Desg FWD 4         128.1    P2p
Gi1/0/2             Desg FWD 4         128.2    P2p
Gi1/0/3             Desg FWD 4         128.3    P2p
Gi1/0/21            Root FWD 4         128.21   P2p
Gi1/0/23            Altn BLK 4         128.23   P2p
```

Look closely at those early message groups that begin with Root ID (about the root switch) and bridge ID (about the local switch). Both sections identify a bridge ID in two parts: the priority and MAC address. The output for the local switch breaks the priority down into the base priority and the VLAN ID. You can also confirm that the local switch is not the root switch based on two different facts:

- The bridge ID values (priority plus MAC address) in the two message sections differ, with the one in the root ID section identifying the root switch.

- The "Root ID" section lists a line with a port number (G1/0/21). That line identifies the local switch's root port. Only nonroot switches have a root port, confirming the local switch is not the root switch.

The output also notes the priority of the root switch and the local switch. The highlighted line for the local switch shows a priority of 32,777, broken down in the same line as a base priority of the (default) value of 32,768 and the system ID extension, or VLAN, of 9 in this case. Earlier, the section about the root switch lists a priority of 24,585. Knowing the output is about VLAN 9, subtract 9 to get the base priority of 24,576.

Consider the output in Example 10-4 from root switch SW1 for comparison. Of note:

- The sections about the root switch and the local switch both list the same priority and MAC address, confirming the local switch is the root switch.
- The section about the root switch does not list a root port; root switches do not have root ports.
- The section about the root switch states, "This bridge is the root."

**Example 10-4** *Examining the 16-bit Priority from Root Switch SW1*

```
SW1# show spanning-tree vlan 9
VLAN0009
  Spanning tree enabled protocol rstp
  Root ID     Priority    24585
              Address     4488.165a.f200
              This bridge is the root
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID   Priority    24585  (priority 24576 sys-id-ext 9)
              Address     4488.165a.f200
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
              Aging Time  300 sec

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/23            Desg FWD 4         128.23   P2p
Te1/1/1             Desg FWD 2         128.25   P2p
```

Finally, for one more fact of many in the detailed output from this command, note that the first highlighted line in Examples 10-3 and 10-4 both show the phrase "protocol rstp." That phrase occurs when using PVST+, per the **spanning-tree mode rapid-pvst** global command.



## Switch Priority Using Root Primary and Secondary

Examples 10-3 and 10-4 relied on the direct configuration of the best priority in switch SW1 (24,576) and second best in switch SW2 (28,672). However, knowing that most STP designs identify the best and second-best switches to use as the root switch, Cisco provides two

related commands that mirror that idea. To configure two switches to be the two most likely switches to be the root switch, simply configure

**spanning-tree vlan** *x* **root primary** (on the switch that should be primary)

**spanning-tree vlan** *x* **root secondary** (on the switch that should be secondary)

Both of these commands use some different IOS logic compared to most configuration commands. Both commands cause IOS to choose a priority value when the command is added to the configuration. Then, IOS does not store the above commands; instead, it stores the priority setting in the **spanning-tree vlan** *x* **priority** *value* command. The command with **root primary** or **root secondary** does not appear in the configuration.

When configuring **root primary**, the switch looks at the priority of the current root switch and chooses either (a) 24,576 or (b) 4096 less than the current root's priority (if the current root's priority is 24,576 or less). The **root secondary** option always results in that switch using a priority of 28,672; the value will be less (better) than other switches that use the default of 32,768 and higher (worse) than any switch configured as **root primary**.

## RSTP (One Tree) and RPVST+ (One Tree Per VLAN)

To complete some of the conceptual discussion about the bridge ID, focus on the standard RSTP and its Cisco-proprietary cousin RPVST+. Both use the RSTP mechanisms as discussed in Chapter 9, "Spanning Tree Protocol Concepts," but RPVST+ uses the mechanisms for every VLAN, while standard RSTP does not. So how do their methods differ?

Key Topic

- RSTP creates one tree—the common spanning tree (CST)—while RPVST+ creates one tree for each and every VLAN.
- RSTP sends one set of RSTP messages (BPDUs) in the network, while RPVST+ sends one set of messages per VLAN.
- RSTP sends messages to multicast address 0180.C200.0000 (per the IEEE standard), while RPVST+ uses multicast address 0100.0CCC.CCCD (an address chosen by Cisco).
- On VLAN trunks, RSTP sends all BPDUs in the native VLAN without a VLAN header/tag. RPVST+ sends BPDUs for each VLAN, respectively. For instance, BPDUs about VLAN 9 have an 802.1Q header that lists VLAN 9.
- RSTP sets the BID's VLAN field (extended system ID) value to 0000.0000.0000, meaning "no VLAN," while RPVST+ uses the VLAN ID.

In other words, standard RSTP behaves as if VLANs do not exist, while Cisco's RPVST+ integrates VLAN information into the entire process.

**NOTE** Some documents refer to the feature of sending BPDUs over trunks with VLAN tags matching the same VLAN as *BPDU tunneling*.

## Identifying Port Cost, Role, and State

Only the bridge ID values impact the root switch choice, and that choice has a direct impact on the STP topology. However, changes to STP interface costs change each nonroot switch's

calculation of its root cost. The root cost then impacts the choice of root port and designated ports, which can impact the STP topology.

To explore those concepts, configure Figure 10-7. It repeats the topology of Figure 10-6, assuming that switch SW1 is the root switch and all port costs in VLAN 9 use default values. The figure focuses the discussion on switch SW3, which calculates possible root costs over two paths. The lower root cost, which uses SW3's port G1/0/21, becomes SW3's root port. SW3's G1/0/23 loses the designated port election due to SW3's root cost of 4 versus SW2's root cost of 2.

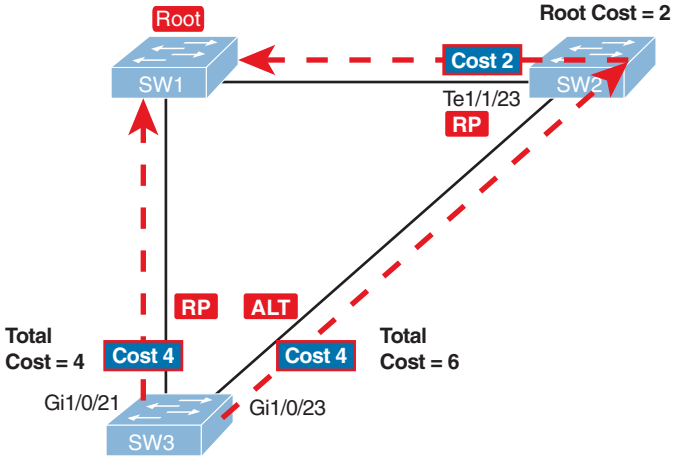



**Figure 10-7** *Sample Network for Root Switch Election Configuration*

The **show spanning-tree vlan 9** command identifies those same facts. Example 10-5 shows an excerpt, with only the two interfaces that connect to other switches listed. The Role column lists the STP roles of Root and Altn (Alternate), while the abbreviated Sts heading refers to the State or Status. (Note that even though the switch uses RSTP, the output still uses the term BLK for blocking rather than the correct RSTP term of discarding.) Finally, the output lists the STP interface cost.

**Example 10-5** *Switch SW3 Port Costs, Roles, and States with Default Costs*

```
SW1# show spanning-tree vlan 9 | begin Interface
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/21            Root FWD 4         128.21   P2p
Gi1/0/23            Altn BLK 4         128.23   P2p
```

As for the root cost, look back to Example 10-3, which also shows output from switch SW3. Find the initial section about the root switch and the line that shows "Cost 4"—that line identifies switch SW3's root cost rather than an interface's cost.

The STP topology shown in Figure 10-7 works well and you would likely not want to change it. But just to show how to do so, consider Example 10-6. It changes SW3's post cost on its root port (G1/0/21) from 4 to 10. As a result, the path out SW3's G1/0/23 port toward switch SW2 has a lower root cost. The example begins with a **debug** command, so you can see interesting facts about the actions IOS takes with this straightforward configuration.

**Example 10-6** *Changing SW3 Port Cost Setting Triggers New Root and Alternate Port Choices*

```
SW3# debug spanning-tree events
Spanning Tree event debugging is on

SW3# conf t
Enter configuration commands, one per line.  End with CNTL/Z.
SW3(config)# int g1/0/21
SW3(config-if)# spanning-tree vlan 9 cost 10
SW3(config-if)#
Oct 19 11:34:38.983: RSTP(9): updt roles, received superior bpdu on Gi1/0/23
Oct 19 11:34:38.983: RSTP(9): Gi1/0/23 is now root port
Oct 19 11:34:38.987: RSTP(9): Gi1/0/21 blocked by re-root
Oct 19 11:34:38.987: RSTP(9): Gi1/0/21 is now alternate
Oct 19 11:34:38.987: STP[9]: Generating TC trap for port GigabitEthernet1/0/23
SW3(config-if)# end
SW3# show spanning-tree vlan 9
VLAN0009
  Spanning tree enabled protocol rstp
  Root ID     Priority    24585
              Address     4488.165a.f200
              Cost        6
              Port        23 (GigabitEthernet1/0/23)
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID   Priority    32777  (priority 32768 sys-id-ext 9)
              Address     5cfc.6608.2880
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
              Aging Time  300 sec

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/21            Altn BLK 10        128.21   P2p
Gi1/0/23            Root FWD 4         128.23   P2p
```

Comparing the output from Example 10-6 to Example 10-5, you can see that switch SW3's ports swapped roles. G1/0/21 now blocks, with G1/0/23 now in a FWD or forwarding state. Also, in the highlighted lines toward the top of the **show spanning-tree vlan 9** output, you see the updated root cost of 6 and another reference to the root port, G1/0/23.

**Author's Note** Alternately, you can configure the STP port cost on an interface, for all VLANs, with the **spanning-tree cost** *value* interface subcommand. Doing so sets that value as the cost on that interface for any VLANs for which the interface does not also have a VLAN-specific version of the command (as shown in Example 10-6).

# Identifying Optional STP Features

Chapter 9's section titled "Optional STP Features" introduced the concepts behind four STP features: PortFast, BPDU Guard, Root Guard, and Loop Guard. This next major section, with a similar title, focuses on identifying the presence of each feature and the results of using it. Along the way, you will also learn the basics of how to configure each.

## PortFast and BPDU Guard

As discussed back in Chapter 9, PortFast provides both a useful feature but also a notable risk. Once the interface reaches a connected state, PortFast logic moves a port immediately to the STP designated port (DP) role and to a forwarding state. However, that behavior can cause a forwarding loop if the port becomes connected to a switch (with other connections to the rest of the network) instead of to an endpoint device. By combining PortFast with BPDU Guard, you prevent loops by disabling the port if any BPDUs arrive in the port. So, it makes sense to examine both features together.

### PortFast and BPDU Guard on an Access Port with One Endpoint

First, consider the classic case to use both features: a switch port (G1/0/1) connected to a PC. Example 10-7 shows the configuration for both features on that port. Take time to read the long warning messages IOS generates in response to the **spanning-tree portfast** interface subcommand, basically suggesting the use of BPDU Guard.

**Example 10-7** *Enabling PortFast and BPDU Guard on Access Port G1/0/1*

```
SW1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
SW1(config)#
SW1(config)# interface g1/0/1
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 9
SW1(config-if)# spanning-tree portfast
%Warning: portfast should only be enabled on ports connected to a single
 host. Connecting hubs, concentrators, switches, bridges, etc... to this
 interface  when portfast is enabled, can cause temporary bridging loops.
 Use with CAUTION

%Portfast has been configured on GigabitEthernet1/0/1 but will only
have effect when the interface is in a non-trunking mode.
SW1(config-if)# spanning-tree bpduguard ?
  disable  Disable BPDU guard for this interface
  enable   Enable BPDU guard for this interface

SW1(config-if)# spanning-tree bpduguard enable
SW1(config-if)#
```

First, the **spanning-tree portfast** subcommand (with no additional keywords) tells IOS to enable PortFast logic only on an access port. If the port operates as a trunk, IOS does not apply PortFast logic. In fact, the highlighted portion of the long message that IOS generates when you configure the command reminds you of that fact.

As for BPDU Guard, the **spanning-tree bpduguard enable** interface subcommand applies BPDU Guard logic to the port regardless of whether operating as an access or trunk port and regardless of whether PortFast is used. Once enabled, BPDU Guard uses the following trigger, action, and recovery steps:




- **Trigger:** Any BPDU arrives in a port that has BPDU Guard enabled.
- **Actions:**
  - IOS places the interface into an error disabled (err-disabled) interface state.
  - STP removes the interface from the STP instance because the interface fails—that it, it is no longer up (connected).
- **Recovery:**
  - By default, the interface must be configured first with a **shutdown** command and then with a **no shutdown** command.
  - Alternately, and not discussed here, you can configure error disable recovery parameters to automatically recover the port after some time.

Example 10-8 shows a before-and-after example. It begins with switch SW1 port G1/0/1 as configured in Example 10-7, with both PortFast and BPDU Guard enabled. Example 10-8 begins with port G1/0/1 in an STP port role of Desg (designated) and a port state of FWD (forwarding), as consistent with a port in PortFast mode. It also reveals an interface state of connected. Also, in that first command's output, the highlighted port type of "P2p Edge" has great importance: the word "Edge" appears only if PortFast is both configured and enabled, so it confirms that the port uses PortFast.

**Example 10-8** *Example of BPDU Guard Disabling a Port*

```
SW1# show spanning-tree interface g1/0/1
Vlan                Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
VLAN0009            Desg FWD 4         128.1    P2p Edge

SW1# show interfaces g1/0/1 status

Port         Name              Status       Vlan       Duplex  Speed Type
Gi1/0/1      Host A            connected    9          a-full a-1000
10/100/1000BaseTX
SW1#
SW1# ! The cable was removed from the PC and connected to a LAN switch.
SW1#
*Jan 30 17:08:19.024: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEther-
net1/0/1, changed state to down
*Jan 30 17:08:20.024: %LINK-3-UPDOWN: Interface GigabitEthernet1/0/1, changed state
to down
*Jan 30 17:08:30.364: %SPANTREE-2-BLOCK_BPDUGUARD: Received BPDU on port Gi1/0/1
with BPDU Guard enabled. Disabling port.
```

```
*Jan 30 17:08:30.364: %PM-4-ERR_DISABLE: bpduguard error detected on Gi1/0/1, put-
ting Gi1/0/1 in err-disable state
SW1#
SW1# show spanning-tree interface g1/0/1
no spanning tree info available for GigabitEthernet1/0/1

SW1# show interfaces g1/0/1 status

Port          Name               Status       Vlan       Duplex  Speed Type
Gi1/0/1       Host A             err-disabled 9            auto    auto
10/100/1000BaseTX
```

In the middle of the example we replaced the attached PC with a switch. The log messages reveal SW1's port G1/0/1 failing and recovering. As soon as the port came up again, BPDU Guard noticed the incoming BPDUs from the neighboring switch—and disabled the port, as seen in the highlighted log messages.

The bottom of the example repeats the same two **show** commands as the top of the example, revealing the actions taken. The interface state of err-disabled (error disabled) confirms BPDU Guard disabled the interface due to errors. The response from the **show spanning tree interface g1/0/1** command no longer lists information about this port, implying that the interface is no longer part of that spanning tree.

## PortFast on VLAN Trunks and Voice Pseudo-Trunks

Cisco IOS also supports PortFast on trunk ports. You should not use PortFast on trunk ports connected to other switches, but you can use it on trunk ports connected to endpoints, as seen in the center of Figure 10-8. You can also use it on the pseudo-trunk created for voice ports connected to IP phones, as seen in port G1/0/3 in the figure. (For a review of voice VLAN configuration, refer to Chapter 8's section titled "Data and Voice VLAN Configuration and Verification.")

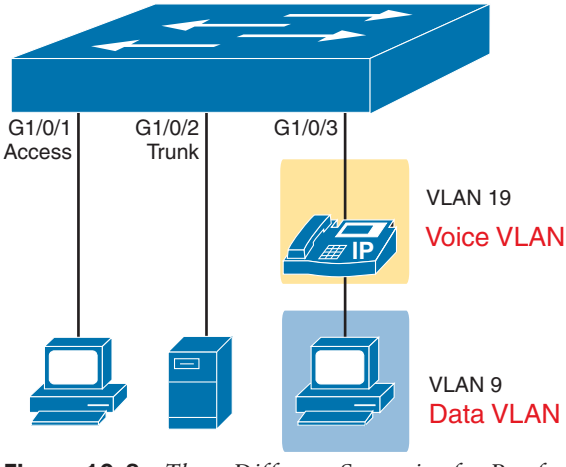



**Figure 10-8** *Three Different Scenarios for Portfast and BPDU Guard*

The PortFast configuration for the voice port looks identical to the access ports. The only small difference happens with IOS automatically adding the **spanning-tree portfast** interface subcommand when you first configure the voice VLAN using the **switchport voice vlan** *vlan-id* interface subcommand.

Note that the **spanning-tree portfast** interface subcommand requires IOS to decide whether to apply PortFast logic or not, based on whether the command does or does not include the **trunk** keyword. The logic is:

**spanning-tree portfast:** Use PortFast if the port operates as an access port.

**spanning-tree portfast trunk:** Use PortFast if the port operates as a trunk.

Example 10-9 shows the configuration for switch SW1 port G1/0/2, attached via a trunk to a server. Note the different warning message in Example 10-9 versus Example 10-7's sample configuration.

**Example 10-9** *Configuring Portfast and BPDU Guard on a Trunk*

```
SW1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
SW1(config)# interface g1/0/2
SW1(config-if)# switchport mode trunk
SW1(config-if)# spanning-tree portfast trunk
%Warning: portfast should only be enabled on ports connected to a single
 host. Connecting hubs, concentrators, switches, bridges, etc... to this
 interface  when portfast is enabled, can cause temporary bridging loops.
 Use with CAUTION
SW1(config-if)# spanning-tree bpduguard enable
SW1(config-if)#
```

You must be ready to discover whether IOS decided to apply PortFast logic to a port. To do so, use the **show spanning-tree** command (without the **interface** keyword) as seen in Example 10-10. The lower part of the command output lists one line per interface for all interfaces active in that spanning tree, including both access and trunk links. The value under the Type heading in the **show spanning-tree** command output reveals whether PortFast is being used:

- **P2p Edge:** Port operates with PortFast logic
- **P2p (without "Edge"):** Port does not use PortFast logic

**Example 10-10** *Confirming Portfast with Port Type Edge*

```
SW1# show spanning-tree vlan 9 | begin Interface
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/1             Desg FWD 4         128.1    P2p Edge
Gi1/0/2             Desg FWD 4         128.2    P2p Edge
Gi1/0/3             Desg FWD 4         128.3    P2p Edge
Gi1/0/23            Desg FWD 4         128.23   P2p
Te1/1/1             Desg FWD 2         128.25   P2p
```

### Global Configuration of PortFast and BPDU Guard

The configuration examples so far use interface subcommands that override any global settings. IOS defaults to global settings whose values disable both features per port; however, you can enable each feature globally. For interfaces with no related subcommands, IOS enables the feature on the interface. Then, you should identify the interfaces for which the feature should be disabled and use another interface subcommand to disable the feature per interface, as needed.

For example, consider an access layer switch with 48 access ports connected to endpoints, plus two trunk ports connected to other switches. You probably want to enable both PortFast and BPDU Guard on all 48 access ports. Rather than requiring the interface subcommands on all 48 of those ports, enable both the features globally, and then disable them on the uplink ports.

First, consider PortFast. Configuring the **spanning-tree portfast default** global command tells IOS to apply PortFast to some ports, based on the following conditions:

- Enable PortFast on ports operating as access ports only.
- Ignore ports configured to disable PortFast with the **spanning-tree portfast disable** interface subcommand.

To see that logic in action, work through the examples in Figure 10-9. The switch uses the global command **spanning-tree portfast default**. Port G1/0/11 has no **spanning-tree portfast** subcommands and is configured to be an access port, so IOS applies PortFast to that port. However, port G1/0/12 has a **spanning-tree portfast disable** subcommand, so IOS does not apply PortFast. Port G1/0/13 fails to meet the required conditions because it uses trunking.

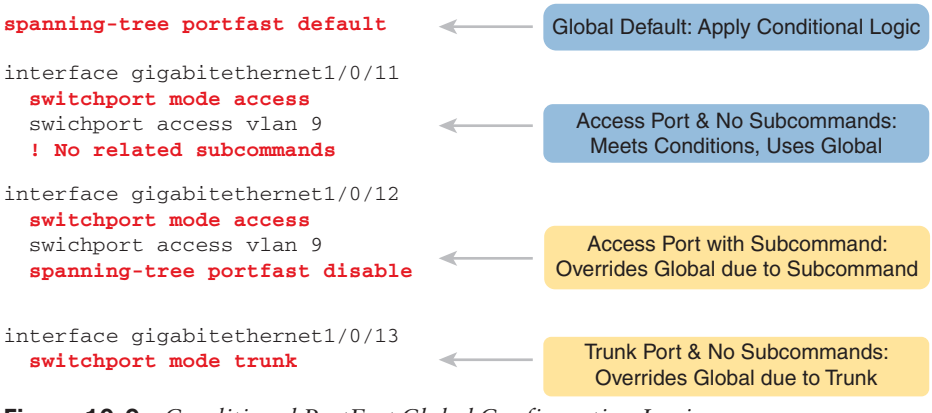



**Figure 10-9** *Conditional PortFast Global Configuration Logic*

IOS also supports a similar configuration process for BPDU Guard, but with this configuration, BPDU Guard is tied to PortFast. You configure the **spanning-tree portfast bpduguard default** global command. Doing so asks IOS to enable BPDU Guard under these conditions:

- Enable BPDU Guard on ports that currently use PortFast.
- Ignore ports configured to disable BPDU Guard with the **spanning-tree bpduguard disable** interface subcommand.

For the exam, interpreting **show** command output might be more important than the intricacies of the configuration. For example, exam topic 2.5, the only one that mentions STP features, uses the verb *interpret* rather than *configure* or *verify*. So, be ready to interpret **show** command output and even predict the related configuration.

As an example of determining the configuration from the **show** commands, consider interfaces G1/0/7 and G1/0/8 on switch SW1. The configuration (not shown) uses this convention:

- **G1/0/7:** Uses interface subcommands **spanning-tree portfast** and **spanning-tree bpduguard enable**, not relying on the global configuration commands.
- **G1/0/8:** Uses no relevant interface subcommands, instead relying on the global configuration commands **spanning-tree portfast default** and **spanning-tree portfast bpduguard default**.

Example 10-11 shows truncated output from the **show spanning-tree interface** command for both interfaces. Compare the highlighted lines about PortFast and BPDU Guard to see the different information. On port G1/0/8, the phrase "by default" confirms that G1/0/8 uses the default setting per the global command. The absence of "by default" in the output for port G1/0/7 means those settings come from interface subcommands.

**Example 10-11** *Interpreting the Source of PortFast and BPDU Guard Configuration*

```
SW1# show spanning-tree interface g1/0/7 detail | begin portfast
   The port is in the portfast mode
   Link type is point-to-point by default
   Bpdu guard is enabled
   BPDU: sent 387, received 0

SW1# show spanning-tree interface g1/0/8 detail | begin portfast
   The port is in the portfast mode by default
   Link type is point-to-point by default
   Bpdu guard is enabled by default
   BPDU: sent 774, received 0
```

## BPDU Filter

In Chapter 9's section about this same topic, also titled "BPDU Filter," you learned about the two different logic branches of this feature. To review:

1. Using a global configuration command, enable BPDU Filter, after which IOS applies BPDU Filter on PortFast ports only. While applied, it monitors for incoming BPDUs. When incoming BPDUs occur, BPDU Filter disables PortFast logic, so that the port then uses normal STP logic on the port.
2. Using an interface subcommand, enable BPDU filter on the port. BPDU Filter discards all outgoing and incoming BPDUs on the port, effectively disabling STP on the port.

This section examines both logic branches in order.

### Conditional BPDU Filtering with Global Configuration

To enable the conditional logic of BPDU Filter, you must toggle from the default global setting of **no spanning-tree portfast bpdufilter enable** to the same command without the **no** option: the **spanning-tree portfast bpdufilter enable** global command.

Similar to the effect of the **spanning-tree portfast bpduguard enable** global command, the **spanning-tree portfast bpdufilter enable** global command asks IOS to enable BPDU Filter under these conditions:

- Enable BPDU Filter on ports that currently use PortFast.
- Ignore ports configured to disable BPDU Filter with the **spanning-tree bpdufilter disable** interface subcommand.

Example 10-12 shows a straightforward scenario on switch SW1 port G1/0/23. It shows a classic access port, in VLAN 9, with PortFast enabled with an interface subcommand—along with conditional BPDU Filter enabled globally.

**Example 10-12** *BPDU Filter as Global Default, Applied to Port G1/0/23*

```
spanning-tree portfast bpdufilter enable
!
interface GigabitEthernet1/0/23
  switchport mode access
  switchport access vlan 9
  spanning-tree portfast
  ! No BPDU Filter subcommands present
```

You should be ready to think about the configuration and understand the rules IOS applies—and to also see the evidence of the choices in IOS command output. First, to review how IOS interprets and applies the configuration:

- The combination of port G1/0/23 as an access port (from the **switchport mode access** subcommand), with the command to enable PortFast on access ports (the **spanning-tree portfast** subcommand without the **trunk** keyword), enables PortFast.
- The one global command tells IOS to find current PortFast ports (no matter whether access or trunk port) and enable BPDU Filter conditional logic.

Example 10-13 shows evidence of these listed results, while an endpoint connects to the port rather than a rogue switch. Look for the following:

1. The first command, **show spanning-tree**, lists interfaces in the tree for that VLAN. It lists the port type for G1/0/23 as P2p Edge—the word Edge confirms that the port currently uses Portfast.
2. The final command, in the final line, lists a counter of 11 sent BPDUs and 0 received. That confirms the switch sent 11 Hellos before BPDU Filter stopped sending them after 20 seconds. If nothing changes, the received BPDU counter remains at 0 because the attached endpoint device does not send BPDUs to the switch.

3. The phrase at the end of the example "Bpdu filter is enabled by default," at the end of the **show spanning-tree interface** command, reveals that the BPDU Filter configuration uses a global command. This command's output includes the phrase "by default" when the global configuration setting is the reason the feature is enabled.
4. Conversely, that same **show spanning-tree interface** command output reveals that the PortFast configuration uses an interface subcommand. The phrase "by default" does not occur at the end of the line about PortFast, implying the configuration comes from an interface subcommand.

**Example 10-13** *Key Status Values When an Endpoint Connects to Port G1/0/23*

```
SW1# show spanning-tree vlan 9 | begin Interface
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/1             Desg FWD 4         128.1    P2p Edge
Gi1/0/2             Desg FWD 4         128.2    P2p Edge
Gi1/0/3             Desg FWD 4         128.3    P2p Edge
Gi1/0/23            Desg FWD 4         128.23   P2p Edge
Te1/1/1             Desg FWD 2         128.25   P2p

SW1# show spanning-tree interface g1/0/23

Vlan                Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
VLAN0009            Desg FWD 4         128.23   P2p Edge

SW1# show spanning-tree interface g1/0/23 detail | begin portfast
   The port is in the portfast mode
   Link type is point-to-point by default
   Bpdu filter is enabled by default
   BPDU: sent 11, received 0
```

As configured using the global command, IOS applies conditional BPDU Filter logic. To see that in action, Example 10-14 begins with the replacement of the attached PC with a rogue switch. The example does not show the related log messages, but know that the interface fails and recovers. The example repeats the same **show** commands as in the previous example, but with these differences:

1. The first command, **show spanning-tree**, lists the port type for G1/0/23 as P2p but without the word Edge. That output change is what confirms IOS no longer applies PortFast logic to this port.
2. The final command, in the final line, lists BPDU counters. In this case, the neighboring switch becomes root and continues to send BPDUs into port G1/0/23. The received BPDU counter will continue to increment over time.
3. The **show spanning-tree interface** command no longer includes the line that mentions the BPDU Filter feature.

**Example 10-14** *SW1 Port G1/0/23 Connects to a Rogue Switch*

```
! Someone disconnects the endpoint off SW1's G1/0/23 and attaches a switch:
! G1/0/23 fails and recovers…

SW1# show spanning-tree vlan 9 | begin Interface
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/23            Root FWD 4         128.23   P2p

SW1# show spanning-tree interface g1/0/23 detail | begin portfast
   The port is in the portfast mode
   Link type is point-to-point by default
   BPDU: sent 6, received 138
! Line "bpdu filter is enabled by default" does not appear above.
```

### Disabling STP with BPDU Filter Interface Configuration

The other type of BPDU Filter logic always filters all outgoing and incoming BPDUs on a port, in effect disabling STP on the interface. To configure it, simply configure the **spanning-tree bpdufilter enable** interface subcommand.

**NOTE** That previous statement ought to scare you—one simple command can disable STP on one link. With redundant links in the LAN, that one command, on one port, can create a forwarding loop that makes the entire LAN unusable. That is not an exaggeration. Be very careful before using this command!

Example 10-15 shows a typical configuration. This feature makes sense on links between switches, so it would likely be a trunk port.

**Example 10-15** *Disabling STP on a Port Between STP Domains*

```
interface TenGigabitEthernet 1/1/1
  switchport mode trunk
  spanning-tree bpdufilter enable
```

Example 10-16 shows just a few lines of output that are the keys to confirming the feature works. The end of the **show spanning-tree interface** command has a line that confirms the BPDU Filter feature is enabled; however, that line does not end with "by default" as seen near the end of Example 10-14. That tiny difference in the text signals a huge difference in logic! The absence of "by default" means IOS enables BPDU Filter in this case due to an interface subcommand, which means IOS applies the absolute filtering logic of BPDU Filter—disabling STP on the interface.

**Example 10-16** *Typical Evidence of BPDU Filter to Disable STP*

```
SW1# show spanning-tree vlan 9 interface te1/1/1 detail | begin Bpdu
   Bpdu filter is enabled
   BPDU: sent 0, received 0
```

To support the claim that IOS applies absolute filtering of all outgoing and incoming BPDUs on the port, note that the counters in the final output line show 0s. Over time, it will continue to show 0s, because BPDU Filter discards all BPDUs on the port.

## Root Guard

The most challenging part of working with Root Guard involves analyzing the STP design to decide which ports might benefit from using it. Chapter 9's section with this same title, "Root Guard," discusses the logic, but to summarize, you consider only ports connected to other switches. Then you look for special cases of switch ports that should never receive a superior Hello from a new root switch based on the intended STP design. Root Guard then monitors for incoming superior Hellos, disabling the port when that occurs. Root Guard may not apply to any ports for some networks, but the choice of ports begins by thinking about the STP design, as discussed in Chapter 9.

If ports need to use Root Guard, the implementation and verification take a few short steps. The configuration uses a single option: the **spanning-tree guard root** interface subcommand. There is no global command to change the default, with the default being not to use Root Guard on an interface.

As for the actions taken by Root Guard, Chapter 9 described the big concepts, but the following list provides more detail that you will see in the upcoming CLI examples.



- **Trigger:** Root Guard acts after receiving a superior BPDU in the port.
- **Actions:** Once triggered, Root Guard takes these actions:
  - The actions occur per VLAN, based on the VLAN of the superior Hello.
  - STP places the port in a broken (BRK) state for that VLAN, which discards all traffic (like the discarding and blocking states).
  - STP describes the port as being in a root inconsistent state in the port type information in show commands.
- **Recovery:** When the incoming superior BPDUs cease for a time, STP reverts to its prior STP state without any operator intervention.

Examples 10-17 and 10-18 combine to show an example of Root Guard in action. First, Example 10-17 shows the simple configuration, with the log message that confirms IOS enabled the feature.

**Example 10-17** *Configuring Root Guard on Switch SW3 Port G1/0/11*

```
SW3# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
SW3(config)# interface g1/0/11
SW3(config-if)# spanning-tree guard root
SW3(config-if)#
Oct 21 11:02:31.145: %SPANTREE-2-ROOTGUARD_CONFIG_CHANGE: Root guard enabled on port
GigabitEthernet1/0/11.
SW3#
```

To see Root Guard in action, look for log messages, the BRK (broken) port state, and the root inconsistent state. Example 10-18 does just that, as follows:

1. The first two commands confirm port G1/0/11's interface state (connected), STP role (Desg, or designated), STP forwarding state (FWD), and port type (P2P).
2. A log message reveals that Root Guard blocked the port. (Just before that, but not shown, I lowered the priority of the neighboring switch.)
3. The last two commands at the bottom of the example reveal:
   a. No change in in port G1/0/11's interface state (connected) or STP role (designated).
   b. A change to the STP forwarding state BKN, meaning broken, and port type P2p ROOT Inc, meaning Root Inconsistent.

**Example 10-18** *Identifying Root Guard Root Inconsistent State*

```
SW3# show interfaces g1/0/11 status

Port      Name               Status       Vlan       Duplex  Speed Type
Gi1/0/11                     connected    trunk      a-full a-1000 10/100/1000BaseTX

SW3# show spanning-tree vlan 9 int g1/0/11

Vlan                 Role Sts Cost      Prio.Nbr Type
-------------------- ---- --- ---------- -------- --------------------------------
VLAN0009             Desg FWD 4         128.11   P2p

SW3#! Neighboring switch priority was lowered so it sends a superior BPDU.

Oct 21 11:03:14.472: %SPANTREE-2-ROOTGUARD_BLOCK: Root guard blocking port
GigabitEthernet1/0/11 on VLAN0009.

SW3# show interfaces g1/0/11 status
Port      Name               Status       Vlan       Duplex  Speed Type
Gi1/0/11                     connected    trunk      a-full a-1000 10/100/1000BaseTX

SW3# show spanning-tree vlan 9 int g1/0/11

Vlan                 Role Sts Cost      Prio.Nbr Type
-------------------- ---- --- ---------- -------- --------------------------------
VLAN0009             Desg BKN*4         128.11   P2p *ROOT_Inc
```

## Loop Guard

Chapter 9's section with this same title, "Loop Guard," discusses the compound factors that lead to the specific scenario where you can apply Loop Guard. Understanding those conditions and choosing ports that can effectively use Loop Guard takes some effort. However, the configuration takes only a little effort once chosen, as seen in the following list. You can either enable it directly on the interface or change the global default to enable it on all switch interfaces—but then disable it on selected interfaces as needed. Use these steps.




**Step 1.** Use the **spanning-tree guard loop** interface subcommand to enable Loop Guard on the interfaces selected to use the feature.

**Step 2.** To use the global default:

**a.** Use the **spanning-tree loopguard default** global command to change the Loop Guard default from disabled to enabled on all point-to-point switch interfaces.

**b.** Use the **no spanning-tree guard loop** interface subcommand to disable Loop Guard on the interfaces selected not to use the feature.

Example 10-19 shows the simple configuration on a single port. The **show spanning-tree interface** command then confirms that Loop Guard is enabled, with the absence of the "by default" phrase implying it was configured using an interface subcommand.

**Example 10-19** *Configuring Loop Guard on a Port*

```
SW3# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
SW3(config)# interface g1/0/21
SW3(config-if)# spanning-tree guard loop
SW3(config-if)#

SW3# show spanning-tree vlan 9 int g1/0/21 detail
 Port 21 (GigabitEthernet1/0/21) of VLAN0009 is root forwarding
   Port path cost 4, Port priority 128, Port Identifier 128.21.
   Designated root has priority 24585, address 4488.165a.f200
   Designated bridge has priority 24585, address 4488.165a.f200
   Designated port id is 128.23, designated path cost 0
   Timers: message age 16, forward delay 0, hold 0
   Number of transitions to forwarding state: 7
   Link type is point-to-point by default
   Loop guard is enabled on the port
   BPDU: sent 139, received 165425

! Link becomes unidirectional; switch SW3 G1/0/21 ceases to receive BPDUs
*Feb 23 17:11:19.087: %SPANTREE-2-LOOPGUARD_BLOCK: Loop guard blocking port
GigabitEthernet1/0/23 on VLAN0009.
SW3# show spanning-tree vlan 9 int g1/0/21 detail | include 1/0/21
 Port 21 (GigabitEthernet1/0/21) of VLAN0009 is broken  (Loop Inconsistent)
```

The example ends by showing what happens when a unidirectional link occurs on this port. The end of the example shows a comment to show when the failure occurs and the log message noting the Loop Guard acted for VLAN 9. The final output shows how Loop Guard moved the port to a broken state (Loop Inconsistent) in that VLAN, similar to the Root Inconsistent state used by Root Guard. Loop Guard also recovers automatically when Hellos begin to arrive in the port again.

# Configuring Layer 2 EtherChannel

As introduced in Chapter 9, "Spanning Tree Protocol Concepts," two neighboring switches can treat multiple parallel links between each other as a single logical link called an *EtherChannel*. Without EtherChannel, a switch treats each physical port as an independent port, applying MAC learning, forwarding, and STP logic per physical port. With EtherChannel, the switch applies all those same processes to a group of physical ports as one entity: the EtherChannel. Without EtherChannel, with parallel links between two switches, STP/RSTP would block all links except one, but with EtherChannel, the switch can use all the links, load balancing the traffic over the links.

**NOTE** All references to EtherChannel in this chapter refer to Layer 2 EtherChannels, not to Layer 3 EtherChannels (as discussed in Chapter 18, "IP Routing in the LAN"). CCNA 200-301 exam topics include both Layer 2 and Layer 3 EtherChannels.

EtherChannel might be one of the most challenging switch features to make work. First, the configuration has several options, so you have to remember the details of which options work together. Second, the switches also require a variety of other interface settings to match among all the links in the channel, so you have to know those settings as well.

This section shows how to configure a Layer 2 EtherChannel, first through manual (static) configuration and then by allowing dynamic protocols to create the channel. Cisco recommends using the dynamic method, but the static method can be a little easier to learn initially, so we begin with the static or manual option. This section closes with some information about some common configuration issues that occur with Layer 2 EtherChannels.

## Configuring a Manual Layer 2 EtherChannel

To configure a Layer 2 EtherChannel so that all the ports always attempt to be part of the channel, simply add the correct **channel-group** configuration command to each physical interface, on each switch, all with the **on** keyword, and all with the same number. The **on** keyword tells the switches to place a physical interface into an EtherChannel, and the number identifies the PortChannel interface number that the interface should be a part of.

Before getting into the configuration and verification, however, you need to start using three terms as synonyms: **EtherChannel**, **PortChannel**, and **Channel-group**. Oddly, IOS uses the **channel-group** configuration command, but then to display its status, IOS uses the **show etherchannel** command. Then the output of this **show** command refers to neither an "EtherChannel" nor a "Channel-group," instead using "PortChannel." So, pay close attention to these three terms in the example.

To configure an EtherChannel manually, follow these steps:

Key Topic

Config Checklist

**Step 1.** Add the **channel-group** *number* **mode on** command in interface configuration mode under each physical interface that should be in the channel to add it to the channel.

**Step 2.** Use the same number for all commands on the same switch, but the channel-group number on the neighboring switch can differ.

Example 10-20 shows a simple example, with two links between switches SW1 and SW2, as shown in Figure 10-10. The example begins with all default interface configuration on the interfaces used for the EtherChannel. The configuration first shows the ports being configured as VLAN trunks and then manually configured to be in channel-group 1 (SW1) and channel-group 2 (SW2).

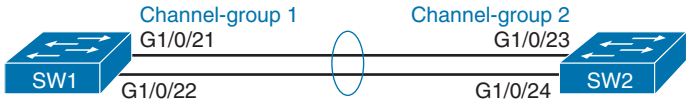



**Figure 10-10** *Sample LAN Used in EtherChannel Example*

**Example 10-20** *Configuring EtherChannel—Both SW1 and SW2*

```
! First, on switch SW1
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface range g1/0/21-22
SW1(config-if-range)# switchport mode trunk
SW1(config-if-range)# channel-group 1 mode on
```

```
! Next, on switch SW2
SW2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW2(config)# interface range g1/0/23-24
SW2(config-if-range)# switchport mode trunk
SW2(config-if-range)# channel-group 2 mode on
```

Interestingly, IOS reacts to the **channel-group** interface subcommands to create a matching port-channel interface on the switch. Example 10-21 shows an excerpt from the **show running-config** command on switch SW1, listing the new port-channel 1 interface in the configuration, along with the two physical interfaces.

**Example 10-21** *Configuration Results from Example 10-20*

```
SW1# show running-config
! Lines omitted for brevity
!
interface Port-channel1
 switchport mode trunk
!
! Lines omitted for brevity
interface GigabitEthernet1/0/21
 switchport mode trunk
 channel-group 1 mode on
!
```

```
interface GigabitEthernet1/0/22
 switchport mode trunk
 channel-group 1 mode on

SW1# show interfaces portchannel 1
Port-channel1 is up, line protocol is up (connected)
  Hardware is EtherChannel, address is 4488.165a.f215 (bia 4488.165a.f215)
  MTU 1500 bytes, BW 2000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full-duplex, 1000Mb/s, link type is auto, media type is N/A
  input flow-control is on, output flow-control is unsupported
  Members in this channel: Gi1/0/21 Gi1/0/22
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 02:12:51, output 00:00:00, output hang never
  Last clearing of "show interface" counters never
! Interface statistics output removed for brevity
```

The end of Example 10-21 gives more insight into the portchannel interface in the output of the **show interfaces portchannel 1** command. Like all output from the **show interfaces** command, the output lists both an interface and protocol state ("up" and "up" in this case), and the interface bandwidth, noted with the text "BW." However, the output shows the bandwidth as 2,000,000 Kbps, or 2 Gbps, because the portchannel has two active 1-Gbps links. Also, in the final highlighted line, the output lists the currently active interfaces in the portchannel.

The **show etherchannel** command, shown at the top of Example 10-22, lists basic configuration information about the channel per earlier Example 10-20. In this case, it identifies the configured portchannel number (1), with two ports configured to be in the channel. It lists the protocol as a dash (–), meaning that it does not use LACP or PAgP, implying the use of the **channel-group mode on** command. Note that this command does not list status information, only configuration information.

**Example 10-22** *Exploring SW1 PortChannel Configuration and Status*

```
SW1# show etherchannel
                Channel-group listing:
                ----------------------

Group: 1
----------
Group state = L2
Ports: 2   Maxports = 8
Port-channels: 1 Max Port-channels = 1
Protocol:    -
Minimum Links: 0
```

```
SW1# show etherchannel summary
Flags:  D - down        P - bundled in port-channel
        I - stand-alone s - suspended
        H - Hot-standby (LACP only)
        R - Layer3      S - Layer2
        U - in use      f - failed to allocate aggregator

        M - not in use, minimum links not met
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port

        A - formed by Auto LAG

Number of channel-groups in use: 1
Number of aggregators:           1

Group  Port-channel  Protocol    Ports
------+-------------+-----------+-----------------------------------------------
1      Po1(SU)           -        Gi1/0/21(P)    Gi1/0/22(P)
```

The **show etherchannel summary** command at the end of Example 10-22 provides status information. The output begins with an extensive status code legend. The lines at the bottom list each PortChannel along with the ports and their status. In the lowest line of output with highlights, note the code P, which means that both of those ports are bundled (working) in the channel. A status of (SU), per the legend, means the channel is in use and acts as a Layer 2 EtherChannel (rather than a Layer 3 EtherChannel).

## Configuring Dynamic EtherChannels

Cisco switches also support two different configuration options that use a dynamic protocol to negotiate whether a particular link becomes part of an EtherChannel or not. Basically, the configuration enables a protocol for a particular channel-group number. At that point, the switch can use the protocol to send messages to/from the neighboring switch and discover whether their configuration settings pass all checks. If a given physical link passes, the link is added to the EtherChannel and used; if not, it is placed in a down state, and not used, until the configuration inconsistency can be resolved.

Most Cisco Catalyst switches support the Cisco-proprietary Port Aggregation Protocol (**PAgP**) and the IEEE standard Link Aggregation Control Protocol (**LACP**). Although differences exist between the two, to the depth discussed here, they both accomplish the same task: negotiate so that only links that pass the configuration checks are actually used in an EtherChannel. (Note that the IEEE originally defined LACP in amendment 802.3ad but now defines it in IEEE standard 802.1AX.)

One difference of note is that LACP does support more links in a channel—16—as compared to PaGP's maximum of 8. With LACP, only 8 can be active at one time, with the others waiting to be used should any of the other links fail.

To configure either protocol, a switch uses the **channel-group** configuration commands on each switch, but with a keyword that either means "use this protocol and begin negotiations" or "use this protocol and wait for the other switch to begin negotiations." As shown in Figure 10-11, the **desirable** and **auto** keywords enable PAgP, and the **active** and **passive** keywords enable LACP. With these options, at least one side has to begin the negotiations. In other words, with PAgP, at least one of the two sides must use **desirable**, and with LACP, at least one of the two sides must use **active**.

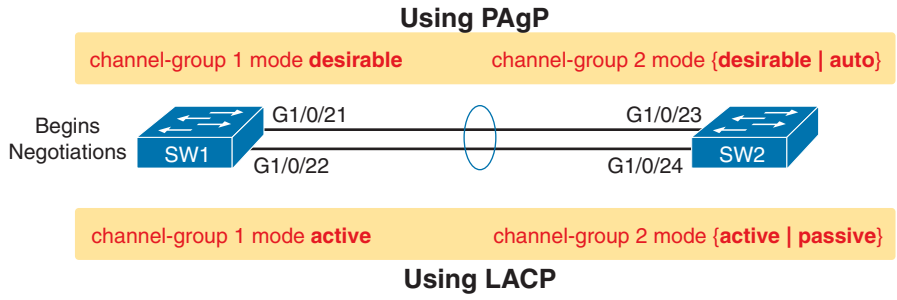



**Figure 10-11** *Correct EtherChannel Configuration Combinations*

**NOTE** Do not use the **on** parameter on one end, and either **auto** or **desirable** (or for LACP, **active** or **passive**) on the neighboring switch. The **on** option uses neither PAgP nor LACP, so a configuration that uses **on**, with PAgP or LACP options on the other end, would prevent the EtherChannel from working.

As an example, consider the topology in Figure 10-11, which uses the same switches and ports as Figure 10-10 and the last several examples. However, the next example starts fresh, with the manual EtherChannel no longer configured and no interface commands on any of the interfaces in use. Example 10-23 shows a dynamic configuration with LACP on both switches, with the **channel-group 1 mode active** interface subcommand on SW1 and **channel-group 2 mode passive** on SW2.

**Example 10-23** *Configuring an LACP Dynamic EtherChannel—Both SW1 and SW2*

```
! First, on switch SW1
SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW1(config)# interface range g1/0/21-22
SW1(config-if-range)# switchport mode trunk
SW1(config-if-range)# channel-group 1 mode active
```

```
! Next, on switch SW2
SW2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
SW2(config)# interface range g1/0/23-24
SW2(config-if-range)# switchport mode trunk
SW2(config-if-range)# channel-group 2 mode passive
```

As with the manual configuration and verification in Examples 10-20 and 10-21, the switch creates a portchannel interface in reaction to the configuration shown in Example 10-23. Example 10-24 confirms the related settings, with group or portchannel 1, two ports in the channel, specifically SW1's G1/0/21 and G1/0/22. Also, note that the output lists the protocol as LACP, because the configuration commands in Example 10-23 use keywords **active** and **passive**, both of which enable LACP.

**Example 10-24** *EtherChannel Verification: SW1 with LACP Active Mode*

```
SW1# show etherchannel port-channel
                Channel-group listing:
                ----------------------

Group: 1
----------
                Port-channels in the group:
                ---------------------------

Port-channel: Po1    (Primary Aggregator)

------------

Age of the Port-channel   = 0d:00h:11m:35s
Logical slot/port   = 31/1          Number of ports = 2
HotStandBy port = null
Port state          = Port-channel Ag-Inuse
Protocol            =   LACP
Port security       = Disabled
Fast-switchover     = disabled
Fast-switchover Dampening = disabled

Ports in the Port-channel:

Index   Load   Port        EC state        No of bits
------+------+------+------------------+-----------
  0     00     Gi1/0/21       Active             0
  0     00     Gi1/0/22       Active             0

Time since last port bundled:    0d:00h:02m:17s     Gi1/0/22
Time since last port Un-bundled: 0d:00h:02m:25s     Gi1/0/22
```

Before leaving the core EtherChannel configuration and verification topics, think about EtherChannels and Spanning Tree together. STP/RSTP prefers the better links based on STP/RSTP link costs. An EtherChannel may have more than one working link, and the number of active links changes as links fail and recover. So, by default, IOS calculates the default STP cost for EtherChannel based on the number of active links.

For example, the STP/RSTP default costs prefer 10 Gbps over EtherChannels with 1 Gbps links, and EtherChannels of multiple 1 Gbps links over a single 1 Gbps link, with the following default port costs:

- **Default Cost 4:** EtherChannels with one active 1 Gbps link and any single 1 Gbps link (without an EtherChannel)
- **Default Cost 3:** EtherChannels with 2 to 8 active 1 Gbps links
- **Default Cost 2:** A single 10 Gbps link

Example 10-25 shows the default STP cost on a couple of 1 Gbps interfaces along with interface PortChannel 1, whose configuration resides earlier in Example 10-23.

**Example 10-25** *EtherChannel Verification: SW1 with LACP Active Mode*

```
SW1# show spanning-tree vlan 1 | begin Interface
Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi1/0/23            Desg FWD 4         128.23   P2p
Gi1/0/24            Desg FWD 4         128.24   P2p
Po1                 Desg FWD 3         128.456  P2p
```

## Interface Configuration Consistency with EtherChannels

Even when the **channel-group** commands have all been configured correctly, other configuration settings can prevent a switch from using a physical port in an EtherChannel—even physical ports manually configured to be part of the channel. The next topic examines those reasons.

First, before using a physical port in a dynamic EtherChannel, the switch compares the new physical port's configuration to the existing ports in the channel. That new physical interface's settings must be the same as the existing ports' settings; otherwise, the switch does not add the new link to the list of approved and working interfaces in the channel. That is, the physical interface remains configured as part of the PortChannel, but it is not used as part of the channel, often being placed into some nonworking state.

The list of items the switch checks includes the following:




- Speed
- Duplex
- Operational access or trunking state (all must be access, or all must be trunks)
- If an access port, the access VLAN
- If a trunk port, the allowed VLAN list (per the **switchport trunk allowed** command)
- If a trunk port, the native VLAN

Example 10-26 shows a failure of one link in the EtherChannel due to a purposeful misconfiguration of the native VLAN on SW1's G1/0/21 port. Example 10-26 begins with the configuration of VLAN 21 as the native VLAN on port G1/0/21; port G1/0/22 defaults to

native VLAN 1. The log messages following that configuration show the interface failing, with the **show interfaces g1/0/21** command listing a "down (suspended)" protocol state for that interface. The usual **show etherchannel port-channel** command lists only one port as bundled in the channel.

**Example 10-26** *Native VLAN Mismatch Removes SW1 G1/0/21 from EtherChannel*

```
SW1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
SW1(config)# interface g1/0/21
SW1(config-if)# switchport trunk native vlan 21
SW1(config-if)# ^Z
SW1#
Jun 17 16:11:34.217: %EC-5-CANNOT_BUNDLE2: Gi1/0/21 is not compatible with Gi1/0/22
and will be suspended (native vlan of Gi1/0/21 is 21, Gi1/0/22 id 1)
Jun 17 16:11:35.220: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet1/0/21, changed state to down

SW1# show interfaces gigabitethernet 1/0/21
GigabitEthernet1/0/21 is up, line protocol is down (suspended)
  Hardware is Gigabit Ethernet, address is 4488.165a.f215 (bia 4488.165a.f215)
! lines omitted for brevity

SW1# show etherchannel port-channel
                Channel-group listing:
                ----------------------

Group: 1
----------
                Port-channels in the group:
                ---------------------------

Port-channel: Po1    (Primary Aggregator)

------------

Age of the Port-channel   = 0d:05h:26m:48s
Logical slot/port   = 31/1          Number of ports = 1
HotStandBy port = null
Port state          = Port-channel Ag-Inuse
Protocol            =   LACP
Port security       = Disabled
Fast-switchover     = disabled
Fast-switchover Dampening = disabled

Ports in the Port-channel:
```

```
Index   Load   Port        EC state        No of bits
------+------+------+------------------+-----------
  0     00     Gi1/0/22       Active            0

Time since last port bundled:    0d:00h:05m:44s     Gi1/0/22
Time since last port Un-bundled: 0d:00h:01m:09s     Gi1/0/21
```

The output in Example 10-26 closes with two useful messages at the end of the **show etherchannel port-channel** command output: the interface ID and the timing of the most recent ports bundled and unbundled with the EtherChannel. In this case, it shows the most recent addition of the still-working G1/0/22 interface and the just-suspended G1/0/21 interface.

## EtherChannel Load Distribution

When using Layer 2 EtherChannels, a switch's MAC learning process associates MAC addresses with the PortChannel interfaces and not the underlying physical ports. Later, when a switch makes a forwarding decision to send a frame out a PortChannel interface, the switch must do more work: to decide which specific physical port to use to forward the frame. IOS documentation refers to those rules as **EtherChannel load distribution** or *load balancing*. Figure 10-12 shows the main idea.

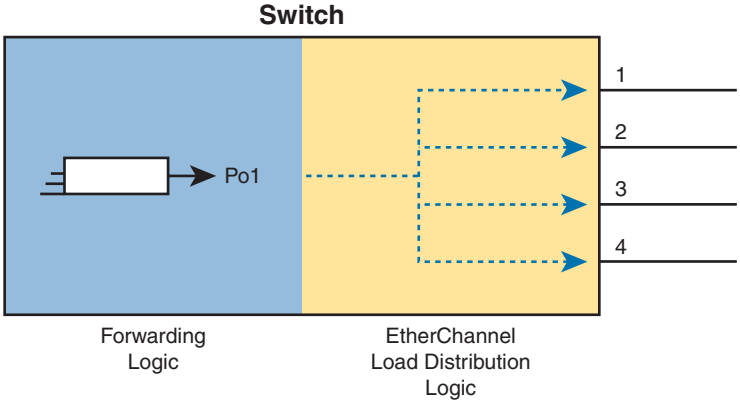



**Figure 10-12** *Forwarding Concepts from Outgoing PortChannel to Physical Interfaces*

EtherChannel load distribution makes the choice for each frame based on various numeric values found in the Layer 2, 3, and 4 headers. The process uses one configurable setting as input: the load distribution method as defined with the **port-channel load-balance** *method* global command. The process then performs some match against the fields identified by the configured method.

Table 10-4 lists the most common methods. However, note that some switches may support only MAC-based methods, or only MAC- and IP-based methods, depending on the model and software version.

**Table 10-4** EtherChannel Load Distribution Methods

| Configuration Keyword | Math Uses... | Layer |
|---|---|---|
| **src-mac** | Source MAC address | 2 |
| **dst-mac** | Destination MAC address | 2 |
| **src-dst-mac** | Both source and destination MAC | 2 |
| **src-ip** | Source IP address | 3 |
| **dst-ip** | Destination IP address | 3 |
| **src-dst-ip** | Both source and destination IP | 3 |
| **src-port** | Source TCP or UDP port | 4 |
| **dst-port** | Destination TCP or UDP port | 4 |
| **src-dst-port** | Both source and destination TCP or UDP port | 4 |

To appreciate why you might want to use different methods, you need to consider the results of how switches make their choice. (The discussion here focuses on the result, and not the logic, because the logic remains internal to the switch, and Cisco does not document how each switch model or IOS version works internally.) However, the various load distribution algorithms do share some common goals:

- To cause all messages in a single application flow to use the same link in the channel, rather than being sent over different links. Doing so means that the switch will not inadvertently reorder the messages sent in that application flow by sending one message over a busy link that has a queue of waiting messages, while immediately sending the next message out an unused link.
- To integrate the load distribution algorithm work into the hardware forwarding ASIC so that load distribution works just as quickly as the work to forward any other frame.
- To use all the active links in the EtherChannel, adjusting to the addition and removal of active links over time.
- Within the constraints of the other goals, balance the traffic across those active links.

In short, the algorithms first intend to avoid message reordering, make use of the switch forwarding ASICs, and use all the active links. However, the algorithm does not attempt to send the exact same number of bits over each link over time. The algorithm does try to balance the traffic, but always within the constraints of the other goals.

Whatever load distribution method you choose, the method identifies fields in the message headers. Any messages in the same application flow will then have the same values in the fields used by the load distribution algorithm and will always be forwarded over the same link. For example, when a user connects to a website, that web server may return thousands of packets to the client. Those thousands of packets should flow over the same link in the EtherChannel.

For instance, with the load distribution method of **src-mac** (meaning source MAC address), all frames with the same MAC address flow over one link. Figure 10-13 shows the idea with pseudo (generic) MAC addresses, with the load distribution sending frames with source MAC 1 over link 1, source MAC 2 over link 2, and source MAC 3 over link 3.

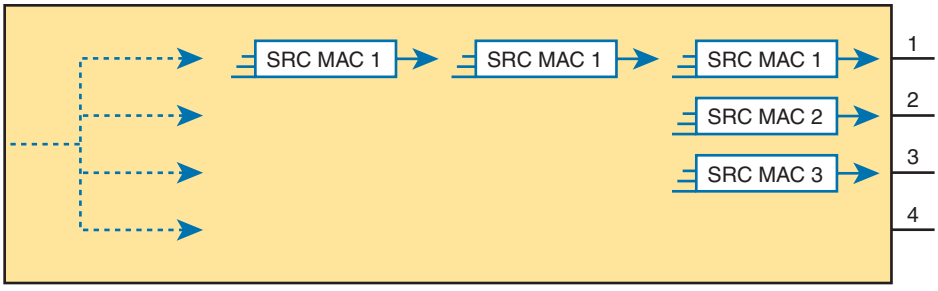



**Figure 10-13** *Distributing All Frames with the Same MAC Out the Same Interface*

Cisco provides a variety of load distribution options so that the engineer can examine the flows in the network with the idea of finding which fields have the most variety in their values: source and destination MAC, or IP address, or transport layer port numbers. The more variety in the values in the fields, the better the balancing effects, and the lower the chance of sending disproportionate amounts of traffic over one link.

**NOTE** The algorithm focuses on the low-order bits in the fields in the headers because the low-order bits typically differ the most in real networks, while the high-order bits do not differ much. By focusing on the lower-order bits, the algorithm achieves better balancing of traffic over the links.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 10-5 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 10-5** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review config checklists | | Book, website |
| Review command tables | | Book |
| Review memory tables | | Website |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 10-6** Key Topics for Chapter 10

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 10-1 | Typical design choice for which switches should be made to be root | 259 |
| Figure 10-2 | Conceptual view of load-balancing benefits of PVST+ | 260 |
| Table 10-2 | STP Standards and Configuration Options | 261 |
| Figure 10-4 | Shows the format of the system ID extension of the STP priority field | 262 |
| List | Facts about RPVST+'s methods versus RSTP | 266 |
| List | Trigger, actions, and recovery for BPDU Guard | 270 |
| List | Trigger, actions, and recovery for Root Guard | 278 |
| List | Configuration options with Loop Guard | 280 |
| List | Steps to manually configure an EtherChannel | 281 |
| List | Items a switch compares in a new physical port's configuration to the existing ports in the channel | 287 |

## Key Terms You Should Know

Channel-group, EtherChannel, EtherChannel load distribution, LACP, PAgP, PortChannel, PVST+, Rapid PVST+, system ID extension

## Command References

Tables 10-7 and 10-8 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 10-7** Chapter 10 Configuration Command Reference

| Command | Description |
|---|---|
| **spanning-tree mode** {**pvst** \| **rapid-pvst** \| **mst**} | Global configuration command to set the STP mode. |
| **spanning-tree** [**vlan** *vlan-number*] **root primary** | Global configuration command that changes this switch to the root switch. The switch's priority is changed to the lower of either 24,576 or 4096 less than the priority of the current root bridge when the command was issued. |
| **spanning-tree** [**vlan** *vlan-number*] **root secondary** | Global configuration command that sets this switch's STP base priority to 28,672. |
| **spanning-tree vlan** *vlan-id* **priority** *priority* | Global configuration command that changes the bridge priority of this switch for the specified VLAN. |
| **spanning-tree** [**vlan** *vlan-number*] **cost** *cost* | Interface subcommand that changes the STP cost to the configured value. |

| Command | Description |
|---|---|
| **spanning-tree** [**vlan** *vlan-number*] **port-priority** *priority* | Interface subcommand that changes the STP port priority in that VLAN (0 to 240, in increments of 16). |
| **spanning-tree portfast** | Interface subcommand that enables PortFast if the port is also an access port. |
| **spanning-tree portfast trunk** | Interface subcommand that enables PortFast if the port is also a trunk port. |
| **spanning-tree bpduguard enable** | Interface subcommand that enables BPDU Guard on the interface under all conditions. |
| **spanning-tree portfast disable** | Interface subcommand that reverses the **spanning-tree portfast** command. |
| **spanning-tree bpduguard disable** | Interface subcommand that reverses the **spanning-tree bpduguard enable** command. |
| **spanning-tree portfast enable** | Global command that changes the default interface setting to the same logic as if the **spanning-tree portfast** interface subcommand were configured. |
| **spanning-tree portfast bpduguard default** | Global command that changes the default interface setting to enable BPDU Guard if the port is also actively using PortFast. |
| **spanning-tree bpdufilter enable** | Interface subcommand that enables BPDU Filter on the interface under all conditions, disabling STP on the interface. |
| **spanning-tree portfast bpdufilter default** | Global command that directs IOS to enable BPDU Filter conditional logic, which toggles away from using PortFast as needed if the port is also actively using PortFast. |
| **[no] spanning-tree guard root** | Interface subcommand to enable or disable (with the **no** option) Root Guard. |
| **[no] spanning-tree guard loop** | Interface subcommand to enable or disable (with the **no** option) Loop Guard. |
| **spanning-tree loopguard default** | Global command to change the default setting on interfaces to enable Loop Guard. |
| **channel-group** *channel-group-number* **mode** {**auto** \| **desirable** \| **active** \| **passive** \| **on**} | Interface subcommand that enables EtherChannel on the interface. |

**Table 10-8** Chapter 10 EXEC Command Reference

| Command | Description |
|---|---|
| **show spanning-tree** | Lists details about the state of STP on the switch, including the state of each port. |
| **show spanning-tree vlan** *vlan-id* | Lists STP information for the specified VLAN. |
| **show spanning-tree vlan** *vlan-id* **interface** *interface-id* **[detail]** | Lists STP information for the specified VLAN about the specific interface. |

| Command | Description |
|---|---|
| **show etherchannel** [*channel-group-number*] {**brief** \| **detail** \| **port** \| **port-channel** \| **summary**} | Lists information about the state of EtherChannels on this switch. |
| **show interfaces portchannel** *number* | Lists information typical of the **show interfaces** command and also lists the interfaces included in the EtherChannel. |
| **show etherchannel** | Displays configuration settings for each EtherChannel. |
| **show etherchannel** [*number*] {**summary** \| **portchannel** \| **detail**} | Displays status information about all EtherChannels, or the one specific EtherChannel. The final parameter suggests the briefest option (**summary**) to the most detailed (**detail**). |

*This page intentionally left blank*



# Part III Review

Keep track of your part review progress with the checklist shown in Table P3-1. Details on each task follow the table.

**Table P3-1** Part III Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Review Appendices | | |
| Watch Video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book, using the PTP software.

## Answer Part Review Questions

For this task, answer the Part Review questions for this part of the book, using the PTP software.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Do Labs

Depending on your chosen lab tool, here are some suggestions for what to do in the labs:

**Pearson Network Simulator:** If you use the full Pearson CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.) Note that the Sim Lite that comes with this book also has a couple of labs about VLANs.

**Blog: Config Labs:** The author's blog includes a series of configuration-focused labs that you can do on paper or with Cisco Packet Tracer in about 15 minutes. To find them, open https://www.certskills.com and look under the Labs menu item.

**Other:** If using other lab tools, as a few suggestions: make sure to experiment heavily with VLAN configuration and VLAN trunking configuration.

## Dig Deeper with Appendices on the Companion Website

The chapters in Part III of the book recommended the following appendices for extra reading. If you care to read further, consider

- **Appendix L, "LAN Troubleshooting":** An appendix from the previous edition of the book. It includes topics about VLANs, trunks, and STP and how to troubleshoot each.

## Watch Video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.



The book makes a big transition at this point. Part I gave you a broad introduction to networking, and Parts II and III went into some detail about the dominant LAN technology today: Ethernet. Part IV transitions from Ethernet to the network layer details that sit above Ethernet and WAN technology, specifically IP version 4 (IPv4).

Thinking about the network layer requires engineers to shift how they think about addressing. Ethernet allows the luxury of using universal MAC addresses, assigned by the manufacturers, with no need to plan or configure addresses. Although the network engineer needs to understand MAC addresses, MAC already exists on each Ethernet NIC, and switches learn the Ethernet MAC addresses dynamically without even needing to be configured to do so. As a result, most people operating the network can ignore the specific MAC address values for most tasks.

Conversely, IP addressing gives you flexibility and allows choice; however, those features require planning, along with a much deeper understanding of the internal structure of the addresses. People operating the network must be more aware of the network layer addresses when doing many tasks. To better prepare you for these Layer 3 addressing details, this part breaks down the addressing details into five chapters, with an opportunity to learn more in preparation for the CCNP Enterprise certification.

Part IV examines most of the basic details of IPv4 addressing and subnetting, mostly from the perspective of operating an IP network. Chapter 11 takes a grand tour of IPv4 addressing as implemented inside a typical enterprise network. Chapters 12 through 15 look at some of the specific questions people must ask themselves when operating an IPv4 network.

This section includes all the details you need to learn for the CCNA 200-301 V1.1 blueprint's IPv4 addressing and subnetting exam topics. Many people have learned subnetting from these chapters over the years; however, some people have asked for video and more practice with subnetting—and understandably so. If that's you, consider these additional products as well, both of which can be found at ciscopress.com:

- IP Subnetting: From Beginning to Mastery (Video Course)
- IP Subnetting Practice Question Kit (Practice Questions Product)

We mention these here before you begin Part IV because, if you decide to use them, you might want to use them alongside the chapters in this part.

# Part IV

## IPv4 Addressing

**Chapter 11:** Perspectives on IPv4 Subnetting

**Chapter 12:** Analyzing Classful IPv4 Networks

**Chapter 13:** Analyzing Subnet Masks

**Chapter 14:** Analyzing Existing Subnets

**Chapter 15:** Subnet Design

**Part IV Review**



CHAPTER 11

# Perspectives on IPv4 Subnetting

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.6 Configure and verify IPv4 addressing and subnetting**

**1.7 Describe private IPv4 addressing**

Most entry-level networking jobs require you to operate and troubleshoot a network using a preexisting IP addressing and subnetting plan. The CCNA exam assesses your readiness to use preexisting IP addressing and subnetting information to perform typical operations tasks, such as monitoring the network, reacting to possible problems, configuring addresses for new parts of the network, and troubleshooting those problems.

However, you also need to understand how networks are designed and why. Anyone monitoring a network must ask the question, "Is the network working *as designed*?" If a problem exists, you must consider questions such as "What happens when the network works normally, and what is different right now?" Both questions require you to understand the intended design of the network, including details of the IP addressing and subnetting design.

This chapter provides some perspectives and answers for the bigger issues in IPv4 addressing. What addresses can be used so that they work properly? What addresses should be used? When told to use certain numbers, what does that tell you about the choices made by some other network engineer? How do these choices impact the practical job of configuring switches, routers, hosts, and operating the network on a daily basis? This chapter helps to answer these questions while revealing details of how IPv4 addresses work.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 11-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Analyze Subnetting and Addressing Needs | 1–3 |
| Make Design Choices | 4–7 |

1. Host A is a PC, connected to switch SW1 and assigned to VLAN 1. Which of the following are typically assigned an IP address in the same subnet as host A? (Choose two answers.)
   a. The local router's WAN interface
   b. The local router's LAN interface
   c. All other hosts attached to the same switch
   d. Other hosts attached to the same switch and also in VLAN 1
2. Why does the formula for the number of hosts per subnet ($2^H - 2$) require the subtraction of two hosts?
   a. To reserve two addresses for redundant default gateways (routers)
   b. To reserve the two addresses required for DHCP operation
   c. To reserve addresses for the subnet ID and default gateway (router)
   d. To reserve addresses for the subnet broadcast address and subnet ID
3. A Class B network needs to be subnetted such that it supports 100 subnets and 100 hosts/subnet. Which of the following answers list a workable combination for the number of network, subnet, and host bits? (Choose two answers.)
   a. Network = 16, subnet = 7, host = 7
   b. Network = 16, subnet = 8, host = 8
   c. Network = 16, subnet = 9, host = 7
   d. Network = 8, subnet = 7, host = 17
4. Which of the following are private IP networks? (Choose two answers.)
   a. 172.31.0.0
   b. 172.32.0.0
   c. 192.168.255.0
   d. 192.1.168.0
   e. 11.0.0.0
5. Which of the following are public IP networks? (Choose three answers.)
   a. 9.0.0.0
   b. 172.30.0.0
   c. 192.168.255.0
   d. 192.1.168.0
   e. 1.0.0.0
6. Before Class B network 172.16.0.0 is subnetted by a network engineer, what parts of the structure of the IP addresses in this network already exist, with a specific size? (Choose two answers.)
   a. Network
   b. Subnet
   c. Host
   d. Broadcast

7. Consider the size of the network, subnet, and host parts of the address structure for Class B network 172.16.0.0 without any subnetting. Compare that to an updated plan that subnets the network using one mask for all subnets. Which answer describes what changed in the network, subnet, or host fields for the new plan that uses subnetting in comparison to the earlier plan that does not?
   a. The subnet part got smaller.
   b. The host part got smaller.
   c. The network part got smaller.
   d. The host part was removed.
   e. The network part was removed.

# Foundation Topics

## Introduction to Subnetting

Say you just happened to be at the sandwich shop when it was selling the world's longest sandwich. You're pretty hungry, so you go for it. Now you have one sandwich, but because it's over 2 kilometers long, you realize it's a bit more than you need for lunch all by yourself. To make the sandwich more useful (and more portable), you chop the sandwich into meal-size pieces and give the pieces to other folks around you who are also ready for lunch.

Huh? Well, subnetting, at least the main concept, is similar to this sandwich story. You start with one network, but it is just one large network. As a single large entity, it might not be useful, and it is probably far too large. To make it useful, you chop it into smaller pieces, called **subnets**, and assign those subnets to be used in different parts of the enterprise internetwork.

This short first section of the chapter introduces IP subnetting. First, it shows the general ideas behind a completed subnet design that indeed chops (or subnets) one **network** into subnets. The rest of this section describes the many design steps that you would take to create just such a subnet design. By the end of this section, you should have the right context to then read through the subnetting design steps introduced throughout the rest of this chapter.

**NOTE** All the chapters from this chapter up until Chapter 25, "Fundamentals of IP Version 6," focus on IPv4 rather than IPv6. All references to *IP* refer to IPv4 unless otherwise stated.

### Subnetting Defined Through a Simple Example

An IP network—in other words, a Class A, B, or C network—is simply a set of consecutively numbered IP addresses that follow some preset rules. These Class A, B, and C rules define that for a given network, all the addresses in the network have the same value in some of the octets of the addresses. For example, Class B network 172.16.0.0 consists of all IP addresses that begin with 172.16: 172.16.0.0, 172.16.0.1, 172.16.0.2, and so on, through 172.16.255.255. Another example: Class A network 10.0.0.0 includes all addresses that begin with 10.

An IP subnet is simply a subset of a Class A, B, or C network. If fact, the word *subnet* is a shortened version of the phrase *subdivided network*. For example, one subnet of Class B network 172.16.0.0 could be the set of all IP addresses that begin with 172.16.1, and would include 172.16.1.0, 172.16.1.1, 172.16.1.2, and so on, up through 172.16.1.255. Another subnet of that same Class B network could be all addresses that begin with 172.16.2.

To give you a general idea, Figure 11-1 shows some basic documentation from a completed subnet design that could be used when an engineer subnets Class B network 172.16.0.0.

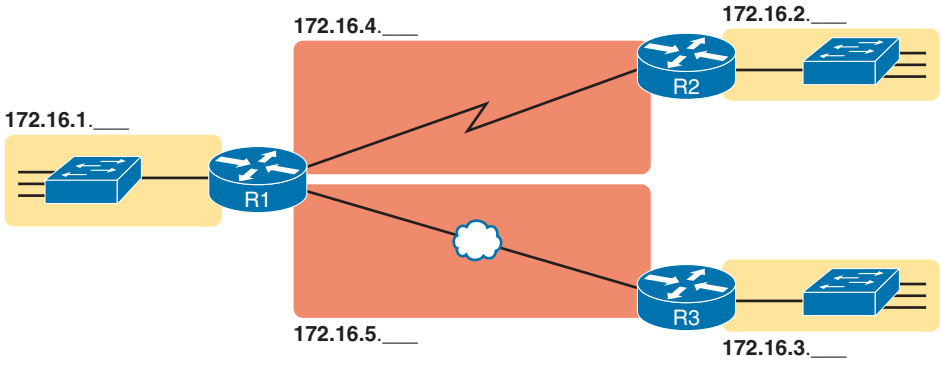



**Figure 11-1** *Subnet Plan Document*

The design shows five subnets—one for each of the three LANs and one each for the two WAN links. The small text note shows the rationale used by the engineer for the subnets: each subnet includes addresses that have the same value in the first three octets. For example, for the LAN on the left, the number shows 172.16.1.__, meaning "all addresses that begin with 172.16.1." Also, note that the design, as shown, does not use all the addresses in Class B network 172.16.0.0, so the engineer has left plenty of room for growth.

## Operational View Versus Design View of Subnetting

Most IT jobs require you to work with subnetting from an operational view. That is, someone else, before you got the job, designed how IP addressing and subnetting would work for that particular enterprise network. You need to interpret what someone else has already chosen.

To fully understand IP addressing and subnetting, you need to think about subnetting from both a design and operational perspective. For example, Figure 11-1 simply states that in all these subnets, the first three octets must be equal. Why was that convention chosen? What alternatives exist? Would those alternatives be better for your internetwork today? All these questions relate more to subnetting design rather than to operation.

To help you see both perspectives, this chapter focuses more on design issues by moving through the entire design process for the purpose of introducing the bigger picture of IP subnetting. The next three chapters each examine one topic from this chapter from an operational perspective, with the final chapter in this part returning to subnet design for a closer look.

The remaining three main sections of this chapter examine each of the steps listed in Figure 11-2, in sequence.

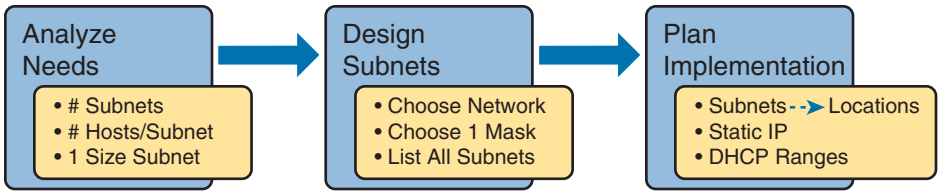



**Figure 11-2** *Subnet Planning, Design, and Implementation Tasks*

# Analyze Subnetting and Addressing Needs

This section discusses the meaning of four basic questions that can be used to analyze the addressing and subnetting needs for any new or changing enterprise network:

1. Which hosts should be grouped together into a subnet?
2. How many subnets does this internetwork require?
3. How many host IP addresses does each subnet require?
4. Will we use a single subnet size for simplicity, or not?

## Rules About Which Hosts Are in Which Subnet

Every device that connects to an IP internetwork needs to have an IP address. These devices include computers used by end users, servers, mobile phones, laptops, IP phones, tablets, and networking devices like routers, switches, and firewalls. In short, any device that uses IP to send and receive packets needs an IP address.

**NOTE** In a discussion of IP addressing, the term *network* has specific meaning: a Class A, B, or C IP network. To avoid confusion with that use of the term *network*, this book uses the terms *internetwork* and *enterprise network* when referring to a collection of hosts, routers, switches, and so on.

The IP addresses must be assigned according to some basic rules—and for good reasons. To make routing work efficiently, IP addressing rules group addresses into groups called subnets. The rules are as follows:

Key Topic

- Addresses in the same subnet are not separated by a router.
- Addresses in different subnets are separated by at least one router.

Figure 11-3 shows the general concept, with hosts A and B in one subnet and host C in another. In particular, note that hosts A and B are not separated from each other by any routers. However, host C, separated from A and B by at least one router, must be in a different subnet.

The idea that hosts on the same link must be in the same subnet is much like the postal code concept. All mailing addresses in the same town use the same postal code (ZIP codes in the United States). Addresses in another town, whether relatively nearby or on the other side of the country, have a different postal code. The postal code gives the postal service a better ability to automatically sort the mail to deliver it to the right location. For the same general reasons, hosts on the same LAN are in the same subnet, and hosts in different LANs are in different subnets.

---

Answers to the "Do I Know This Already?" quiz:

**1** B, D **2** D **3** B, C **4** A, C **5** A, D, E **6** A, C **7** B

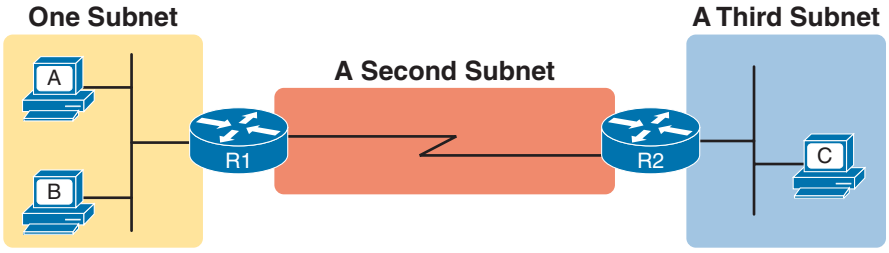



**Figure 11-3** *PC A and B in One Subnet and PC C in a Different Subnet*

Note that the point-to-point WAN link in the figure also needs a subnet. Figure 11-3 shows Router R1 connected to the LAN subnet on the left and to a WAN subnet on the right. Router R2 connects to that same WAN subnet. To do so, both R1 and R2 will have IP addresses on their WAN interfaces, and the addresses will be in the same subnet. (An Ethernet WAN link has the same IP addressing needs, with each of the two routers having an IP address in the same subnet.)

The Ethernet LANs in Figure 11-3 also show a slightly different style of drawing, using simple lines with no Ethernet switch. Drawings of Ethernet LANs when the details of the LAN switches do not matter simply show each device connected to the same line, as shown in Figure 11-3. (This kind of drawing mimics the original Ethernet cabling before switches and hubs existed.)

Finally, because the routers' main job is to forward packets from one subnet to another, routers typically connect to multiple subnets. For example, in this case, Router R1 connects to one LAN subnet on the left and one WAN subnet on the right. To do so, R1 will be configured with two different IP addresses, one per interface. These addresses will be in different subnets because the interfaces connect the router to different subnets.

## Determining the Number of Subnets

To determine the number of subnets required, the engineer must think about the internetwork as documented and count the locations that need a subnet. To do so, the engineer requires access to network diagrams, VLAN configuration details, and details about WAN links. For the types of links discussed in this book, you should plan for one subnet for every one of the following:

Key Topic

- VLAN
- Point-to-point serial link
- Ethernet WAN (Ethernet Line Service)

**NOTE** Other WAN technologies outside the scope of the CCNA exam topics allow subnetting options other than one subnet per pair of routers on the WAN (as shown here). However, this book only uses point-to-point WAN technologies—serial links and Ethernet WAN links—that have one subnet for each point-to-point WAN connection between two routers.

For example, imagine that the network planner has only Figure 11-4 on which to base the subnet design.

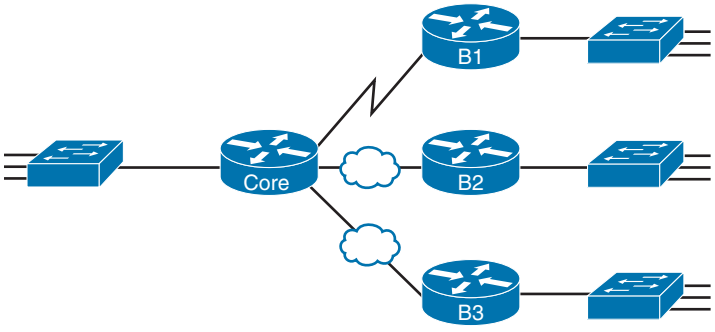



**Figure 11-4** *Four-Site Internetwork with Small Central Site*

The number of subnets required cannot be fully predicted with only this figure. Certainly, three subnets will be needed for the WAN links, one per link. However, each LAN switch can be configured with a single VLAN or with multiple VLANs. You can be certain that you need at least one subnet for the LAN at each site, but you might need more.

Next, consider the more detailed version of the same figure as shown in Figure 11-5. In this case, the figure shows VLAN counts in addition to the same Layer 3 topology (the routers and the links connected to the routers). It also shows that the central site has many more switches, but the key fact on the left, regardless of how many switches exist, is that the central site has a total of 12 VLANs. Similarly, the figure lists each branch as having two VLANs. Along with the same three WAN subnets, this internetwork requires 21 subnets.

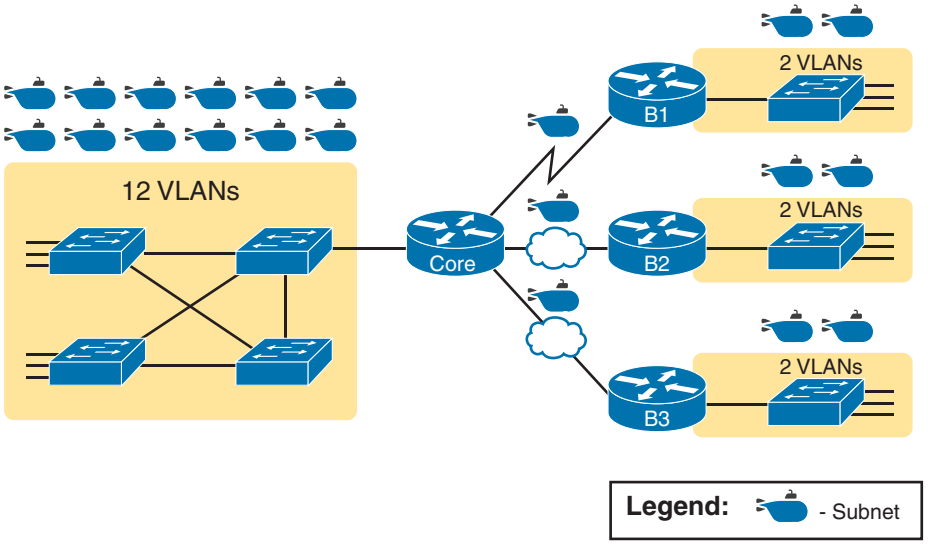



**Figure 11-5** *Four-Site Internetwork with Larger Central Site*

Finally, in a real job, you would consider the needs today as well as how much growth you expect in the internetwork over time. Any subnetting plan should include a reasonable estimate of the number of subnets you need to meet future needs.

## Determining the Number of Hosts per Subnet

Determining the number of hosts per subnet requires knowing a few simple concepts and then doing a lot of research and questioning. Every device that connects to a subnet needs an IP address. For a totally new network, you can look at business plans—numbers of people at the site, devices on order, and so on—to get some idea of the possible devices. When expanding an existing network to add new sites, you can use existing sites as a point of comparison and then find out which sites will get bigger or smaller. And don't forget to count the router interface IP address in each subnet and the switch IP address used to remotely manage the switch.

Instead of gathering data for each and every site, planners often just use a few typical sites for planning purposes. For example, maybe you have some large sales offices and some small sales offices. You might dig in and learn a lot about only one large sales office and only one small sales office. Add that analysis to the fact that point-to-point links need a subnet with just two addresses, plus any analysis of more one-of-a-kind subnets, and you have enough information to plan the addressing and subnetting design.

For example, in Figure 11-6, the engineer has built a diagram that shows the number of hosts per LAN subnet in the largest branch, B1. For the two other branches, the engineer did not bother to dig to find out the number of required hosts. As long as the number of required IP addresses at sites B2 and B3 stays below the estimate of 50, based on larger site B1, the engineer can plan for 50 hosts in each branch LAN subnet and have plenty of addresses per subnet.

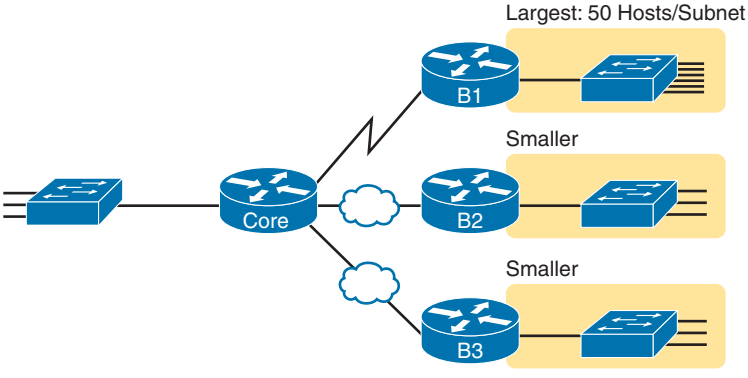



**Figure 11-6** *Large Branch B1 with 50 Hosts/Subnet*

## One Size Subnet Fits All—Or Not

The final choice in the initial planning step is to decide whether you will use a simpler design by using a one-size-subnet-fits-all philosophy. A subnet's size, or length, is simply the number of usable IP addresses in the subnet. A subnetting design can either use one size subnet or varied sizes of subnets, with pros and cons for each choice.

### Defining the Size of a Subnet

Before you finish this book, you will learn all the details of how to determine the size of the subnet. For now, you just need to know a few specific facts about the size of subnets. Chapter 12, "Analyzing Classful IPv4 Networks," and Chapter 13, "Analyzing Subnet Masks," give you a progressively deeper knowledge of the details.

The engineer assigns each subnet a **subnet mask**, and that mask, among other things, defines the size of that subnet. The mask sets aside a number of *host bits* whose purpose is to number different host IP addresses in that subnet. Because you can number $2^x$ things with x bits, if the mask defines H host bits, the subnet contains $2^H$ unique numeric values.

However, the subnet's size is not $2^H$. It's $2^H - 2$ because two numbers in each subnet are reserved for other purposes. Each subnet reserves the numerically lowest value for the *subnet number* and the numerically highest value as the *subnet broadcast address*. As a result, the number of usable IP addresses per subnet is $2^H - 2$.

**NOTE** The terms *subnet number*, *subnet ID*, and *subnet address* all refer to the number that represents or identifies a subnet.

Figure 11-7 shows the general concept behind the three-part structure of an IP address (as defined by the subnet mask), focusing on the host part and the resulting subnet size.

Key Topic

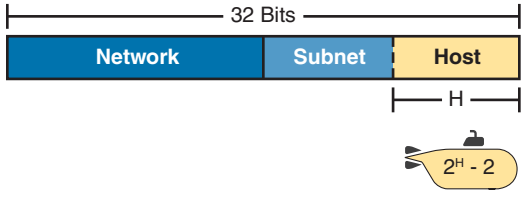



**Figure 11-7** *Subnet Size Concepts*

### One Size Subnet Fits All

To choose to use a single-size subnet in an enterprise network, you must use the same mask for all subnets because the mask defines the size of the subnet. But which mask?

One requirement to consider when choosing that one mask is this: that one mask must provide enough host IP addresses to support the largest subnet. To do so, the number of host bits (H) defined by the mask must be large enough so that $2^H - 2$ is larger than (or equal to) the number of host IP addresses required in the largest subnet.

For example, consider Figure 11-8. It shows the required number of hosts per LAN subnet. (The figure ignores the subnets on the WAN links, which require only two IP addresses each.) The branch LAN subnets require only 50 host addresses, but the main site LAN subnet requires 200 host addresses. To accommodate the largest subnet, you need at least 8 host bits. Seven host bits would not be enough because $2^7 - 2 = 126$. Eight host bits would be enough because $2^8 - 2 = 254$, which is more than enough to support 200 hosts in a subnet.

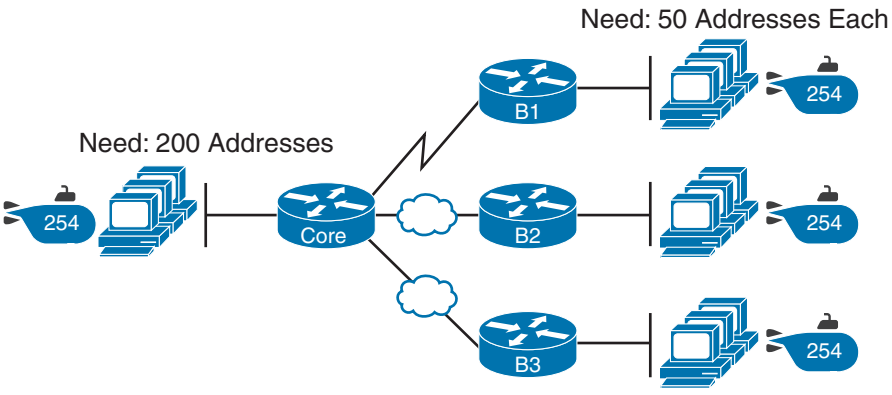



**Figure 11-8** *Network Using One Subnet Size*

What's the big advantage when using a single-size subnet? Operational simplicity. In other words, keeping it simple. Everyone on the IT staff who has to work with networking can get used to working with one mask—and one mask only. Staff members will be able to answer all subnetting questions more easily because everyone gets used to doing subnetting math with that one mask.

The big disadvantage for using a single-size subnet is that it wastes IP addresses. For example, in Figure 11-8, all the branch LAN subnets support 254 addresses, while the largest branch subnet needs only 50 addresses. The WAN subnets need only two IP addresses, but each supports 254 addresses, again wasting more IP addresses.

The wasted IP addresses do not actually cause a problem in most cases, however. Most organizations use private IP networks in their enterprise internetworks, and a single Class A or Class B private network can supply plenty of IP addresses, even with the waste.

### Multiple Subnet Sizes (Variable-Length Subnet Masks)

To create multiple sizes of subnets in one Class A, B, or C network, the engineer must create some subnets using one mask, some with another, and so on. Different masks mean different numbers of host bits, and a different number of hosts in some subnets based on the $2^H - 2$ formula.

For example, consider the requirements listed earlier in Figure 11-8. It showed one LAN subnet on the left that needs 200 host addresses, three branch subnets that need 50 addresses, and three WAN links that need two addresses. To meet those needs, but waste fewer IP addresses, three subnet masks could be used, creating subnets of three different sizes, as shown in Figure 11-9.

The smaller subnets now waste fewer IP addresses compared to the design shown earlier in Figure 11-8. The subnets on the right that need 50 IP addresses have subnets with 6 host bits, for $2^6 - 2 = 62$ available addresses per subnet. The WAN links use masks with 2 host bits, for $2^2 - 2 = 2$ available addresses per subnet.

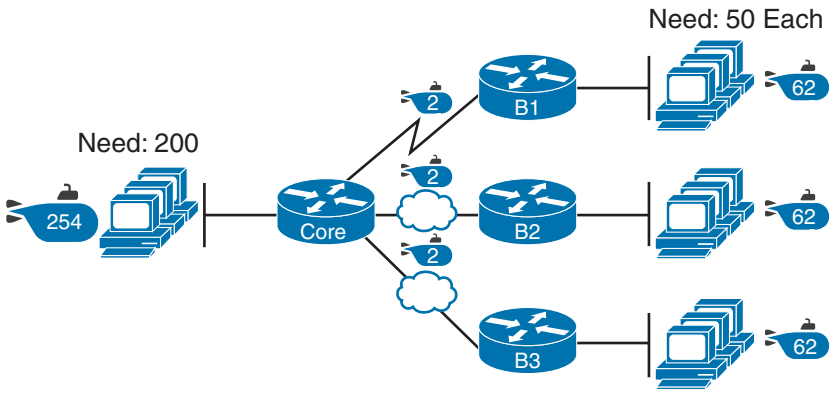



**Figure 11-9** *Three Masks, Three Subnet Sizes*

However, some are still wasted because you cannot set the size of the subnet as some arbitrary size. All subnets will be a size based on the $2^H - 2$ formula, with H being the number of host bits defined by the mask for each subnet.

## One Mask for All Subnets, or More Than One

For the most part, this book explains subnetting using designs that use a single mask, creating a single subnet size for all subnets. Why? First, it makes the process of learning subnetting easier. Second, some types of analysis that you can do about a network—specifically, calculating the number of subnets in the classful network—make sense only when a single mask is used.

However, you still need to be ready to work with designs that use more than one mask in different subnets of the same Class A, B, or C network. In fact, a design that does just that is said to be using **variable-length subnet masks (VLSM)**. For example, the internetwork in Figure 11-10 shows 11 subnets: two with a mask of /30 and nine with a mask of /24. By using more than one mask among all the subnets of one Class A network (10.0.0.0), the design uses VLSM.

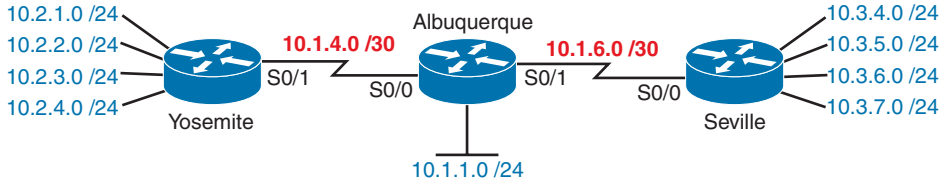



**Figure 11-10** *Internetwork with VLSM: Network 10.0.0.0, >1 Mask*

Although VLSM causes problems when using some older IP routing protocols, the CCNA 200-301 V1.1 blueprint includes only the OSPF routing protocol, and VLSM causes no problems with OSPF. Just be aware of the term and what it means and that it should not impact the features included in the current CCNA exam.

**NOTE** VLSM has been featured in the CCNA exam topics in the past. If you want to read a little more about VLSM, check out Appendix M, "Variable-Length Subnet Masks," on the companion website for this book.

## Make Design Choices

Now that you know how to analyze the IP addressing and subnetting needs, the next major step examines how to apply the rules of IP addressing and subnetting to those needs and make some choices. In other words, now that you know how many subnets you need and how many host addresses you need in the largest subnet, how do you create a useful subnetting design that meets those requirements? The short answer is that you need to do the three tasks shown on the right side of Figure 11-11.

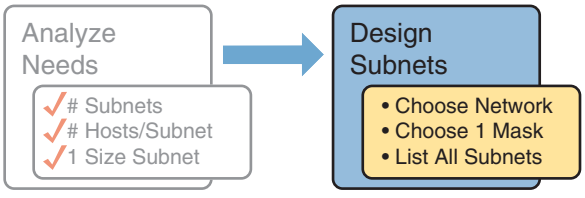



**Figure 11-11** *Input to the Design Phase, and Design Questions to Answer*

### Choose a Classful Network

In the original design for what we know of today as the Internet, companies used registered *public* **classful IP networks** when implementing TCP/IP inside the company. By the mid-1990s, an alternative became more popular: **private IP networks**. This section discusses the background behind these two choices because it impacts the choice of what IP network a company will then subnet and implement in its enterprise internetwork.

#### Public IP Networks

The original design of the Internet required that any company that connected to the Internet had to use a *registered* **public IP network**. To do so, the company would complete some paperwork, describing the enterprise's internetwork and the number of hosts existing, plus plans for growth. After submitting the paperwork, the company would receive an assignment of either a Class A, B, or C network.

Public IP networks—and the administrative processes surrounding them—ensure that all the companies that connect to the Internet use unique IP addresses. In particular, after a public IP network is assigned to a company, only that company should use the addresses in that network. That guarantee of uniqueness means that Internet routing can work well because there are no duplicate public IP addresses.

For example, consider the example shown in Figure 11-12. Company 1 has been assigned public Class A network 1.0.0.0, and company 2 has been assigned public Class A network 2.0.0.0. Per the original intent for public addressing in the Internet, after these public network assignments have been made, no other companies can use addresses in Class A networks 1.0.0.0 or 2.0.0.0.

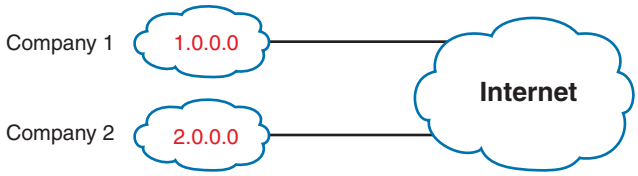



**Figure 11-12** *Two Companies with Unique Public IP Networks*

This original address assignment process ensured unique IP addresses across the entire planet. The idea is much like the fact that your telephone number should be unique in the universe, your postal mailing address should also be unique, and your email address should also be unique. If someone calls you, your phone rings, but no one else's phone rings. Similarly, if company 1 is assigned Class A network 1.0.0.0, and the engineers at Company 1 assign address 1.1.1.1 to a particular PC, that address should be unique in the universe. A packet sent through the Internet to destination 1.1.1.1 should arrive only at this one PC inside company 1, instead of being delivered to some other host.

### Growth Exhausts the Public IP Address Space

By the early 1990s, the world was running out of public IP networks that could be assigned. During most of the 1990s, the number of hosts newly connected to the Internet was growing at a double-digit pace *per month*. Companies kept following the rules, asking for public IP networks, and it was clear that the current address-assignment scheme could not continue without some changes. Simply put, the number of Class A, B, and C networks supported by the 32-bit address in IP version 4 (IPv4) was not enough to support one public classful network per organization, while also providing enough IP addresses in each company.

**NOTE** The universe has run out of public IPv4 addresses in a couple of significant ways. IANA, which assigns public IPv4 address blocks to the five Regional Internet Registries (RIR) around the globe, assigned the last of the IPv4 address spaces in early 2011. By 2015, ARIN, the RIR for North America, exhausted its supply of IPv4 addresses, so companies must return unused public IPv4 addresses to ARIN before they have more to assign to new companies. Try an online search for "ARIN depletion" to see pages about the current status of available IPv4 address space for just one RIR example.

The Internet community worked hard during the 1990s to solve this problem, coming up with several solutions, including the following:

Key Topic

- A new version of IP (IPv6), with much larger addresses (128 bit)
- Assigning a subset of a public IP network to each company, instead of an entire public IP network, to reduce waste, using a feature called classless interdomain routing (CIDR)
- Network Address Translation (NAT), which allows the use of private IP networks

These three solutions matter to real networks today. However, to stay focused on the topic of subnet design, this chapter focuses on the third option, and in particular, the private IP networks that can be used by an enterprise when also using NAT. (Be aware that Chapter 14, "Network Address Translation," in the *CCNA 200-301 Official Cert Guide, Volume 2*,

Second Edition, gives more detail about the last two bullets in the list, while Part VII of this book discusses the first bullet item (IPv6) in more depth.

Focusing on the third item in the bullet list, NAT allows multiple companies to use the exact same *private IP network*, using the same IP addresses as other companies while still connecting to the Internet. For example, Figure 11-13 shows the same two companies connecting to the Internet as in Figure 11-12, but now with both using the same private Class A network 10.0.0.0.

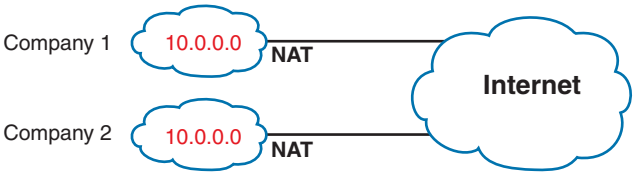



**Figure 11-13** *Reusing the Same Private Network 10.0.0.0 with NAT*

Both companies use the same classful IP network (10.0.0.0). Both companies can implement their subnet design internal to their respective enterprise internetworks, without discussing their plans. The two companies can even use the exact same IP addresses inside network 10.0.0.0. And amazingly, at the same time, both companies can even communicate with each other through the Internet.

The technology called Network Address Translation makes it possible for companies to reuse the same IP networks, as shown in Figure 11-13. NAT does this by translating the IP addresses inside the packets as they go from the enterprise to the Internet, using a small number of public IP addresses to support tens of thousands of private IP addresses. That one bit of information is not enough to understand how NAT works; however, to keep the focus on subnetting, the book defers the discussion of how NAT works until *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition. For now, accept that most companies use NAT, and therefore, they can use private IP networks for their internetworks.

### Private IP Networks

When using NAT—and almost every organization that connects to the Internet uses NAT—the company can simply pick one or more of the private IP networks from the list of reserved private IP network numbers. RFC 1918 defines the list of available private IP networks, which is summarized in Table 11-2.

**Table 11-2** RFC 1918 Private Address Space

| Class of Networks | Private IP Networks | Number of Networks |
|---|---|---|
| A | 10.0.0.0 | 1 |
| B | 172.16.0.0 through 172.31.0.0 | 16 |
| C | 192.168.0.0 through 192.168.255.0 | 256 |

**NOTE** In each class I teach online, I survey students about who uses network 10.0.0.0 in their company, with an average response of 60–70 percent using private class A network 10.0.0.0.

From the perspective of making IPv4 work for the entire world, private IP networks have helped preserve and extend IPv4 and its use in every enterprise and throughout the Internet. In particular, private networks have improved IPv4's implementation worldwide by



- **Avoiding using another organization's public address range for private networks:** Some organizations have a part of their networks that need zero Internet access. The hosts in that part of their network need IP addresses. RFC 1918 suggests that truly private networks—that is, networks with no need for Internet connectivity—use addresses from the RFC 1918 list of private networks.
- **Delaying IPv4 address exhaustion:** To delay the day in which all public IPv4 addresses were assigned to organizations as public addresses, RFC 1918 calls for the use of NAT along with private networks for the addresses internal to an organization.
- **Reducing Internet routers' routing table size:** Using private networks also helps reduce the size of the IP routing tables in Internet routers. For instance, routers in the Internet do not need routes for the private IP networks used inside organizations (in fact, ISPs filter those routes).

### Choosing an IP Network During the Design Phase

Today, most organizations use private IP networks along with NAT, which requires a small CIDR block of public addresses. Some companies use a public address block, like a public Class A, B, or C network, per the original plan that would allow each company to have its own unique public network. Some smaller number of companies might use a public CIDR block for all addresses—in effect a subset of a Class A, B, or C network—and subnet that block.

For the purposes of this book, most examples use private IP network numbers. For the design step to choose a network number, just choose a private Class A, B, or C network from the list of RFC 1918 private networks. Regardless, the subnetting math works the same whether you begin with a public or private IP network.

After the choice to use a private IP network has been made, just pick one that has enough IP addresses. You can have a small internetwork and still choose to use private Class A network 10.0.0.0. It might seem wasteful to choose a Class A network that has over 16 million IP addresses, especially if you need only a few hundred. However, there's no penalty or problem with using a private network that is too large for your current or future needs.

## Choose the Mask

If design engineers followed the topics in this chapter so far, in order, they would know the following:

- The number of subnets required
- The number of hosts/subnet required
- That a choice was made to use only one mask for all subnets so that all subnets are the same size (same number of hosts/subnet)
- The classful IP network number that will be subnetted

This section completes the design process, at least the parts described in this chapter, by discussing how to choose that one mask to use for all subnets. First, this section examines default masks, used when a network is not subnetted, as a point of comparison. Next, the concept of borrowing host bits to create subnet bits is explored. Finally, this section ends with an example of how to create a subnet mask based on the analysis of the requirements.

## Classful IP Networks Before Subnetting

Before an engineer subnets a classful network, the network is a single group of addresses. In other words, the engineer has not yet subdivided the network into many smaller subsets called *subnets*.

When thinking about an unsubnetted classful network, the addresses in a network have only two parts: the **network part** and **host part**. Comparing any two addresses in the classful network:

- The addresses have the same value in the network part.
- The addresses have different values in the host part.

The actual sizes of the network and host parts of the addresses in a network can be easily predicted, as shown in Figure 11-14.

Key Topic

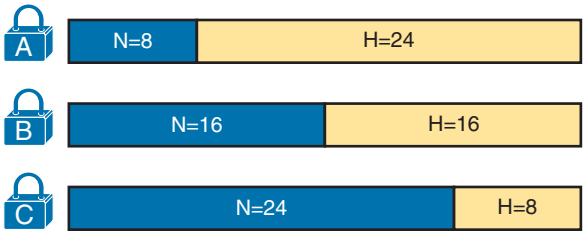



**Figure 11-14** *Format of Unsubnetted Class A, B, and C Networks*

In Figure 11-14, N and H represent the number of network and host bits, respectively. Class rules define the number of network octets (1, 2, or 3) for Classes A, B, and C, respectively; the figure shows these values as a number of bits. The number of host octets is 3, 2, or 1, respectively.

Continuing the analysis of a classful network before subnetting, the number of addresses in one classful IP network can be calculated with the same $2^H - 2$ formula previously discussed. In particular, the size of an unsubnetted Class A, B, or C network is as follows:

- **Class A:** $2^{24} - 2 = 16{,}777{,}214$
- **Class B:** $2^{16} - 2 = 65{,}534$
- **Class C:** $2^8 - 2 = 254$

## Borrowing Host Bits to Create Subnet Bits

To subnet a network, the designer thinks about the network and host parts, as shown in Figure 11-15, and then the engineer adds a third part in the middle: the **subnet part**. However, the designer cannot change the size of the network part or the size of the entire

address (32 bits). To create a subnet part of the address structure, the engineer borrows bits from the host part. Figure 11-15 shows the general idea.

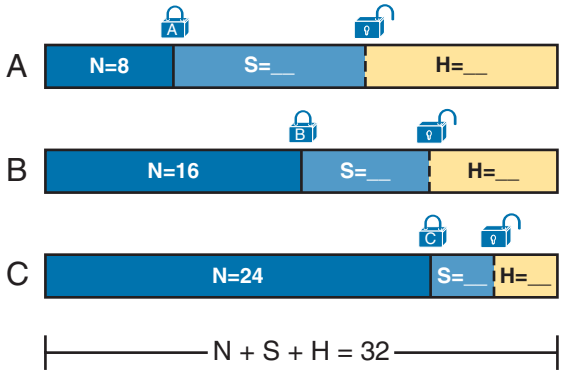



**Figure 11-15** *Concept of Borrowing Host Bits*

Figure 11-15 shows a rectangle that represents the subnet mask. N, representing the number of network bits, remains locked at 8, 16, or 24, depending on the class. Conceptually, the designer moves a (dashed) dividing line into the host field, with subnet bits (S) between the network and host parts, and the remaining host bits (H) on the right. The three parts must add up to 32 because IPv4 addresses consist of 32 bits.

### Choosing Enough Subnet and Host Bits

The design process requires a choice of where to place the dashed line shown in Figure 11-15. But what is the right choice? How many subnet and host bits should the designer choose? The answers hinge on the requirements gathered in the early stages of the planning process:

- Number of subnets required
- Number of hosts/subnet

The bits in the subnet part create a way to uniquely number the different subnets that the design engineer wants to create. With 1 subnet bit, you can number $2^1$ or 2 subnets. With 2 bits, $2^2$ or 4 subnets; with 3 bits, $2^3$ or 8 subnets; and so on. The number of subnet bits must be large enough to uniquely number all the subnets, as determined during the planning process.

At the same time, the remaining number of host bits must also be large enough to number the host IP addresses in the largest subnet. Remember, in this chapter, we assume the use of a single mask for all subnets. This single mask must support both the required number of subnets and the required number of hosts in the largest subnet. Figure 11-16 shows the concept.

Figure 11-16 shows the idea of the designer choosing a number of subnet (S) and host (H) bits and then checking the math. $2^S$ must be more than the number of required subnets, or the mask will not supply enough subnets in this IP network. Also, $2^H - 2$ must be more than the required number of hosts/subnet.

Key Topic

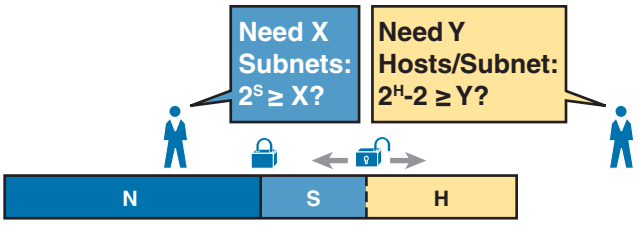



**Figure 11-16** *Borrowing Enough Subnet and Host Bits*

**NOTE** The idea of calculating the number of subnets as $2^S$ applies only in cases where a single mask is used for all subnets of a single classful network, as is being assumed in this chapter.

To effectively design masks, or to interpret masks that were chosen by someone else, you need a good working memory of the powers of 2. Appendix A, "Numeric Reference Tables," lists a table with powers of 2 up through $2^{32}$ for your reference.

### Example Design: 172.16.0.0, 200 Subnets, 200 Hosts

To help make sense of the theoretical discussion so far, consider an example that focuses on the design choice for the subnet mask. In this case, the planning and design choices so far tell us the following:

- Use a single mask for all subnets.
- Plan for 200 subnets.
- Plan for 200 host IP addresses per subnet.
- Use private Class B network 172.16.0.0.

To choose the mask, the designer asks this question:

How many subnet (S) bits do I need to number 200 subnets?

You can see that S = 7 is not large enough ($2^7 = 128$), but S = 8 is enough ($2^8 = 256$). So, you need *at least* 8 subnet bits.

Next, the designer asks a similar question, based on the number of hosts per subnet:

How many host (H) bits do I need to number 200 hosts per subnet?

The math is basically the same, but the formula subtracts 2 when counting the number of hosts/subnet. You can see that H = 7 is not large enough ($2^7 - 2 = 126$), but H = 8 is enough ($2^8 - 2 = 254$).

Only one possible mask meets all the requirements in this case. First, the number of network bits (N) must be 16 because the design uses a Class B network. The requirements tell us that the mask needs at least 8 subnet bits and at least 8 host bits. The mask has only 32 bits in it; Figure 11-17 shows the resulting mask.

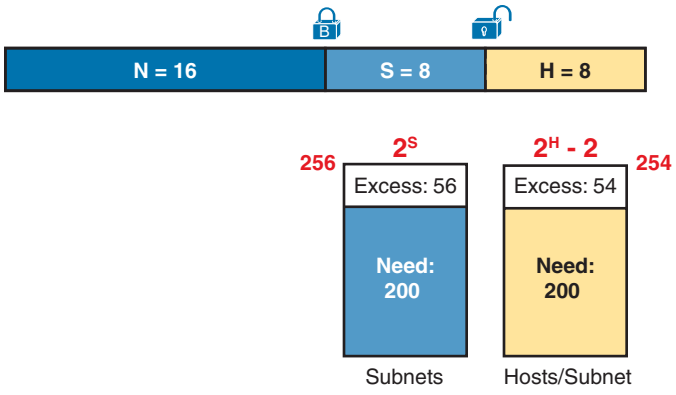



**Figure 11-17** *Example Mask Choice, N = 16, S = 8, H = 8*

### Masks and Mask Formats

Although engineers think about IP addresses in three parts when making design choices (network, subnet, and host), the subnet mask gives the engineer a way to communicate those design choices to all the devices in the subnet.

The subnet mask is a 32-bit binary number with a number of binary 1s on the left and with binary 0s on the right. By definition, the number of binary 0s equals the number of host bits; in fact, that is exactly how the mask communicates the idea of the size of the host part of the addresses in a subnet. The beginning bits in the mask equal binary 1, with those bit positions representing the combined network and subnet parts of the addresses in the subnet.

Because the network part always comes first, then the subnet part, and then the host part, the subnet mask, in binary form, cannot have interleaved 1s and 0s. Each subnet mask has one unbroken string of binary 1s on the left, with the rest of the bits as binary 0s.

After the engineer chooses the classful network and the number of subnet and host bits in a subnet, creating the binary subnet mask is easy. Just write down N 1s, S 1s, and then H 0s (assuming that N, S, and H represent the number of network, subnet, and host bits). Figure 11-18 shows the mask based on the previous example, which subnets a Class B network by creating 8 subnet bits, leaving 8 host bits.

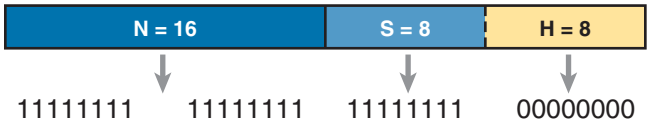



**Figure 11-18** *Creating the Subnet Mask—Binary—Class B Network*

In addition to the binary mask shown in Figure 11-18, masks can also be written in two other formats: the familiar **dotted-decimal notation** (DDN) seen in IP addresses and an even briefer *prefix* notation. Chapter 13 discusses these formats and how to convert between the different formats.

## Build a List of All Subnets

Building a list of all subnets, the final task of the subnet design step, determines the actual subnets that can be used, based on all the earlier choices. The earlier design work

determined the Class A, B, or C network to use, and the (one) subnet mask to use that supplies enough subnets and enough host IP addresses per subnet. But what are those subnets? How do you identify or describe a subnet? This section answers these questions.

A subnet consists of a group of consecutive numbers. Most of these numbers can be used as IP addresses by hosts. However, each subnet reserves the first and last numbers in the group, and these two numbers cannot be used as IP addresses. In particular, each subnet contains the following:

Key Topic

- **Subnet number:** Also called the *subnet ID* or *subnet address*, this number identifies the subnet. It is the numerically smallest number in the subnet. It cannot be used as an IP address by a host.
- **Subnet broadcast:** Also called the *subnet broadcast address* or *directed broadcast address*, this is the last (numerically highest) number in the subnet. It also cannot be used as an IP address by a host.
- **IP addresses:** All the numbers between the subnet ID and the subnet broadcast address can be used as a host IP address.

For example, consider the earlier case in which the design results were as follows:

| | |
|---|---|
| Network | 172.16.0.0 (Class B) |
| Mask | 255.255.255.0 (for all subnets) |

With some math, the facts about each subnet that exists in this Class B network can be calculated. In this case, Table 11-3 shows the first ten such subnets. It then skips many subnets and shows the last two (numerically largest) subnets.

**Table 11-3** First Ten Subnets, Plus the Last Few, from 172.16.0.0, 255.255.255.0

| Subnet Number | IP Addresses | Broadcast Address |
|---|---|---|
| 172.16.0.0 | 172.16.0.1 – 172.16.0.254 | 172.16.0.255 |
| 172.16.1.0 | 172.16.1.1 – 172.16.1.254 | 172.16.1.255 |
| 172.16.2.0 | 172.16.2.1 – 172.16.2.254 | 172.16.2.255 |
| 172.16.3.0 | 172.16.3.1 – 172.16.3.254 | 172.16.3.255 |
| 172.16.4.0 | 172.16.4.1 – 172.16.4.254 | 172.16.4.255 |
| 172.16.5.0 | 172.16.5.1 – 172.16.5.254 | 172.16.5.255 |
| 172.16.6.0 | 172.16.6.1 – 172.16.6.254 | 172.16.6.255 |
| 172.16.7.0 | 172.16.7.1 – 172.16.7.254 | 172.16.7.255 |
| 172.16.8.0 | 172.16.8.1 – 172.16.8.254 | 172.16.8.255 |
| 172.16.9.0 | 172.16.9.1 – 172.16.9.254 | 172.16.9.255 |
| Skipping many… | | |
| 172.16.254.0 | 172.16.254.1 – 172.16.254.254 | 172.16.254.255 |
| 172.16.255.0 | 172.16.255.1 – 172.16.255.254 | 172.16.255.255 |

After you have the network number and the mask, calculating the subnet IDs and other details for all subnets requires some math. In real life, most people use subnet calculators or

subnet-planning tools. For the CCNA exam, you need to be ready to find this kind of information, as discussed in Chapter 15, "Subnet Design."

## Plan the Implementation

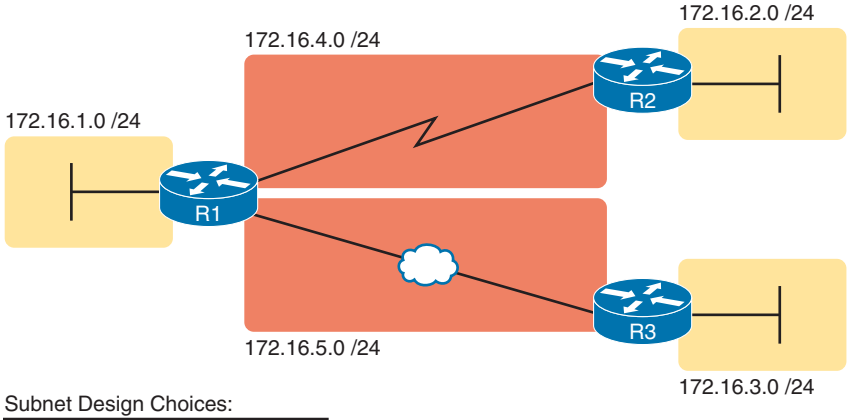
The next step, planning the implementation, is the last step before actually configuring the devices to create a subnet. The engineer first needs to choose where to use each subnet. For example, at a branch office in a particular city, which subnet from the subnet planning chart (Table 11-3) should be used for each VLAN at that site? Also, for any interfaces that require static IP addresses, which addresses should be used in each case? Finally, what range of IP addresses from inside each subnet should be configured in the DHCP server, to be dynamically leased to hosts for use as their IP address? Figure 11-19 summarizes the list of implementation planning tasks.




**Figure 11-19** *Facts Supplied to the Plan Implementation Step*

### Assigning Subnets to Different Locations

The job is simple: Look at your network diagram, identify each location that needs a subnet, and pick one from the table you made of all the possible subnets. Then, track it so that you know which ones you use where, using a spreadsheet or some other purpose-built subnet-planning tool. That's it! Figure 11-20 shows a sample of a completed design using Table 11-3, which happens to match the initial design sample shown way back in Figure 11-1.




Subnet Design Choices:

**Class B 172.16.0.0 /24 (255.255.255.0)**

**Figure 11-20** *Example of Subnets Assigned to Different Locations*

Although this design could have used any five subnets from Table 11-3, in real networks, engineers usually give more thought to some strategy for assigning subnets. For example, you might assign all LAN subnets lower numbers and WAN subnets higher numbers. Or you might slice off large ranges of subnets for different divisions of the company. Or you might follow that same strategy but ignore organizational divisions in the company, paying more attention to geographies.

For example, for a U.S.-based company with a smaller presence in both Europe and Asia, you might plan to reserve ranges of subnets based on continent. This kind of choice is particularly useful when later trying to use a feature called route summarization. Figure 11-21 shows the general benefit of placing addressing in the network for easier route summarization, using the same subnets from Table 11-3 again.

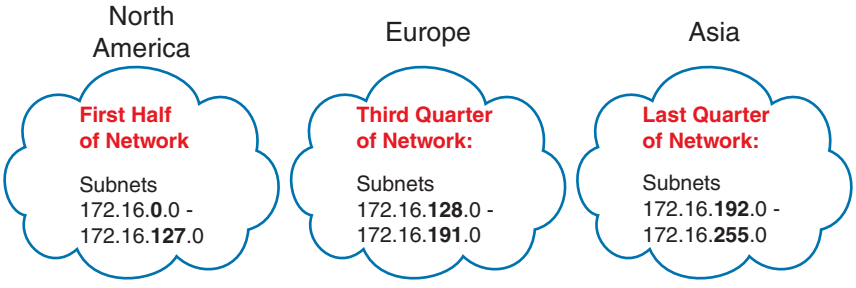



**Figure 11-21** *Reserving 50 Percent of Subnets for North America and 25 Percent Each for Europe and Asia*

## Choose Static and Dynamic Ranges per Subnet

Devices receive their IP address and mask assignment in one of two ways: dynamically by using Dynamic Host Configuration Protocol (DHCP) or statically through configuration. For DHCP to work, the network engineer must tell the DHCP server the subnets for which it must assign IP addresses. In addition, that configuration limits the DHCP server to only a subset of the addresses in the subnet. For static addresses, you simply configure the device to tell it what IP address and mask to use.

To keep things as simple as possible, most shops use a strategy to separate the static IP addresses on one end of each subnet, and the DHCP-assigned dynamic addresses on the other. It does not really matter whether the static addresses sit on the low end of the range of addresses or the high end.

For example, imagine that the engineer decides that, for the LAN subnets in Figure 11-20, the DHCP pool comes from the high end of the range, namely, addresses that end in .101 through .254. (The address that ends in .255 is, of course, reserved.) The engineer also assigns static addresses from the lower end, with addresses ending in .1 through .100. Figure 11-22 shows the idea.

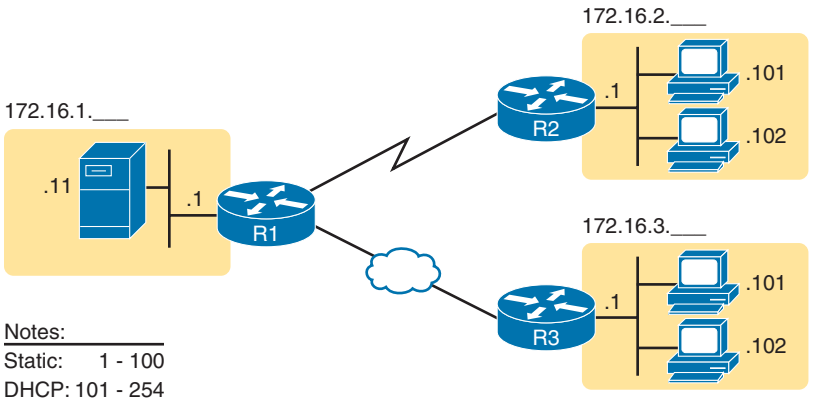



**Figure 11-22** *Static from the Low End and DHCP from the High End*

Figure 11-22 shows all three routers with statically assigned IP addresses that end in .1. The only other static IP address in the figure is assigned to the server on the left, with address 172.16.1.11 (abbreviated simply as .11 in the figure).

On the right, each LAN has two PCs that use DHCP to dynamically lease their IP addresses. DHCP servers often begin by leasing the addresses at the bottom of the range of addresses, so in each LAN, the hosts have leased addresses that end in .101 and .102, which are at the low end of the range chosen by design.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 11-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 11-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Key Topic

**Table 11-5** Key Topics for Chapter 11

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Key facts about subnets | 304 |
| List | Rules about what places in a network topology need a subnet | 305 |
| Figure 11-7 | Locations of the network, subnet, and host parts of an IPv4 address | 308 |
| List | Features that extended the life of IPv4 | 312 |
| List | Motivations for using private IP networks | 314 |
| Figure 11-14 | Formats of Class A, B, and C addresses when not subnetted | 315 |
| Figure 11-15 | Formats of Class A, B, and C addresses when subnetted | 316 |
| Figure 11-16 | General logic when choosing the size of the subnet and host parts of addresses in a subnet | 317 |
| List | Items that together define a subnet | 319 |

## Key Terms You Should Know

classful IP network, dotted-decimal notation, host part, network, network part, private IP network, public IP network, subnet, subnet mask, subnet part, variable-length subnet masks (VLSM)



CHAPTER 12

# Analyzing Classful IPv4 Networks

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.6 Configure and verify IPv4 addressing and subnetting**

**1.7 Describe private IPv4 addressing**

When operating a network, you often start investigating a problem based on an IP address and mask. Based on the IP address alone, you should be able to determine several facts about the Class A, B, or C network in which the IP address resides.

This chapter lists the key facts about classful IP networks and explains how to discover these facts. Following that, this chapter lists some practice problems. Before moving to the next chapter, you should practice until you can consistently determine all these facts, quickly and confidently, based on an IP address.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 12-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Classful Network Concepts | 1–5 |

1. Which of the following are not valid Class A network IDs? (Choose two answers.)
   a. 1.0.0.0
   b. 130.0.0.0
   c. 127.0.0.0
   d. 9.0.0.0
2. Which of the following are not valid Class B network IDs?
   a. 130.0.0.0
   b. 191.255.0.0
   c. 128.0.0.0
   d. 150.255.0.0
   e. All are valid Class B network IDs.

3. Which of the following are true about IP address 172.16.99.45's IP network? (Choose two answers.)
   a. The network ID is 172.0.0.0.
   b. The network is a Class B network.
   c. The default mask for the network is 255.255.255.0.
   d. The number of host bits in the unsubnetted network is 16.
4. Which of the following are true about IP address 192.168.6.7's IP network? (Choose two answers.)
   a. The network ID is 192.168.6.0.
   b. The network is a Class B network.
   c. The default mask for the network is 255.255.255.0.
   d. The number of host bits in the unsubnetted network is 16.
5. Which of the following is a network broadcast address?
   a. 10.1.255.255
   b. 192.168.255.1
   c. 224.1.1.255
   d. 172.30.255.255

# Foundation Topics

## Classful Network Concepts

Imagine that you have a job interview for your first IT job. As part of the interview, you're given an IPv4 address and mask: 10.4.5.99, 255.255.255.0. What can you tell the interviewer about the classful **network** (in this case, the Class A network) in which the IP address resides?

This section, the first of two major sections in this chapter, reviews the concepts of **classful IP networks** (in other words, Class A, B, and C networks). In particular, this chapter examines how to begin with a single IP address and then determine the following facts:

- Class (A, B, or C)
- Default mask
- Number of network octets/bits
- Number of host octets/bits
- Number of host addresses in the network
- Network ID
- Network broadcast address
- First and last usable address in the network

Before getting into this chapter's processes, the text first discusses the context. Analyzing Class A, B, and C networks has a valuable purpose, and it remains an important skill—but it might not be as useful in some networks. This section begins by setting that context

regarding public networks, large public CIDR blocks, and small public CIDR blocks. Then the rest of the section dives into finding all the facts about the classful IP network an address would reside in if the assignment process still assigned the entire Class A, B, or C network.

## Setting the Context of Public Networks and CIDR Blocks

As discussed in Chapter 11, "Perspectives on IPv4 Subnetting," the original plan called for every organization to register for and use a public IP network of one of three classes: A, B, or C. That approach existed in the 1980s up through the 2010s when the Regional Internet Registries (RIRs) assigned their final IP **network numbers**. Even with the RIRs exhausting their supply of IPv4 addresses, companies that received assignments for a public Class A, B, or C network continue to use them. Figure 12-1 shows a conceptual diagram with some Class A networks that could still be used today.

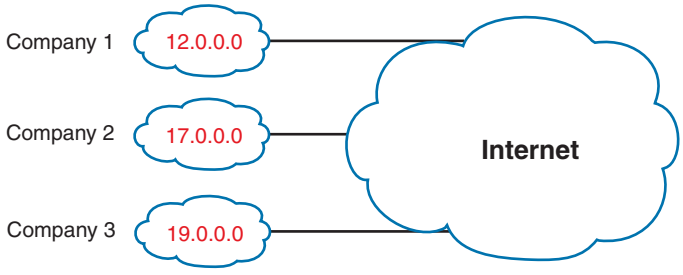



**Figure 12-1** *Concept: Companies Using Separate Public Class A Networks*

One of the address conservation options that emerged in the 1990s called for using more than the three set sizes of Class A, B, and C networks, instead allowing for any block size that is a power of 2. Starting in that decade, the Internet Assigned Numbers Authority (IANA), the global owner of the address space, along with the five RIRs, began assigning public *classless interdomain routing (CIDR) blocks*. Doing so allowed companies to receive assignments that did not waste as many addresses.

For instance, instead of assigning public network 1.0.0.0 to one company, an RIR could subdivide that network into smaller blocks that followed some simple rules. For instance, each block has to include a power of 2 number of addresses. Figure 12-2 shows the concept, with three other companies receiving assignments of public CIDR blocks that look like subsets of former Class A network 1.0.0.0.

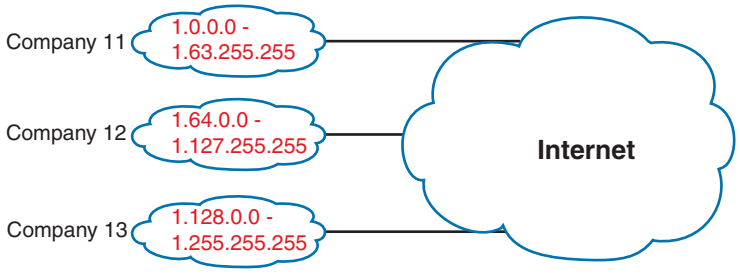



**Figure 12-2** *Concept: Companies Using Public CIDR Blocks*

---

Answers to the "Do I Know This Already?" quiz:

**1** B, C **2** E **3** B, D **4** A, C **5** D

A company using a large public **CIDR block** implements it like a public Class A, B, or C network. Whether starting with a public Class A, B, or C network, or a public CIDR block, the company has one large block of public addresses. The company can then create a subnetting plan to subdivide the large block further, picking which subnets to use in each location. All the hosts in the network use an address from the large public address block.

Over time, a third option emerged that used even fewer public addresses—and it became the most popular option. A company uses a private IP network to address all the hosts within the company. The company also needs a small public CIDR block, while using NAT to translate addresses for packets that flow to and from the Internet. Often, a block of 4, 8, or 16 addresses gives even a large company plenty of public IP address capacity to use with NAT. All companies can use any private IP network for hosts inside the company, as shown in Figure 12-3.

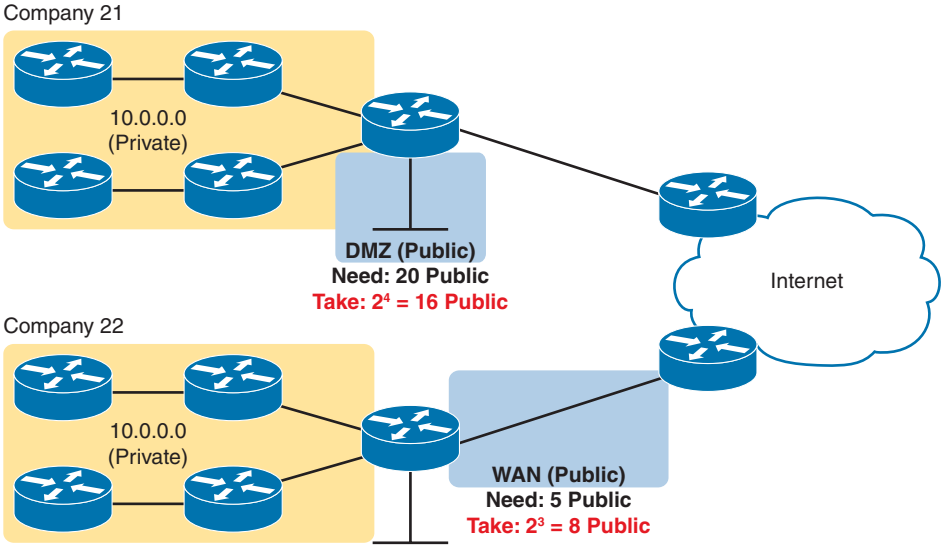



**Figure 12-3** *Concept: Small CIDR Block with Private IP Network*

Now back to the main point of this chapter, which focuses on processes to find key facts about Class A, B, and C networks, when starting with a **unicast IP address**. Regardless of whether the address happens to be from a public classful network, a public CIDR block, or a private classful network, learning the processes in this chapter will help you in your journey to understand networks and subnetting math. Most of you will use these foundational skills at your company or your clients' companies, and it helps prepare you for the exam.

To complete the discussion of the context, be aware that if you start with one public IP address, you cannot tell from that address whether it comes from a public classful network or a public CIDR block. However, this chapter focuses on determining the facts about a Class A, B, or C network, assuming it comes from a public IP network rather than a public CIDR block. Now on to the math and processes!

**NOTE** You can discover if a public unicast address is part of a public classful network assignment, or a public CIDR block, with a search of WHOIS records. As for the process, go to ARIN's WHOIS search page (https://search.arin.net/rdap/), with a search for any public IP address. For example, on that page, a search for 17.1.1.1 reveals an address block of 17.0.0.0/8. As you will learn in this chapter and the next, the 17.0.0.0/8 notation represents Class A network 17.0.0.0, meaning that the 17.1.1.1 address comes from a public Class A assignment.

## IPv4 Network Classes and Related Facts

IP version 4 (IPv4) defines five address classes. Three of the classes, Classes A, B, and C, consist of unicast IP addresses. Unicast addresses identify a single host or interface so that the address uniquely identifies the device. Class D addresses serve as multicast addresses, so that one packet sent to a Class D multicast IPv4 address can actually be delivered to multiple hosts. Finally, Class E addresses were originally intended for experimentation but were changed to simply be reserved for future use. The class can be identified based on the value of the first octet of the address, as shown in Table 12-2.

Key Topic

**Table 12-2** IPv4 Address Classes Based on First Octet Values

| Class | First Octet Values | Purpose |
|---|---|---|
| A | 1–126 | Unicast (large networks) |
| B | 128–191 | Unicast (medium-sized networks) |
| C | 192–223 | Unicast (small networks) |
| D | 224–239 | Multicast |
| E | 240–255 | Reserved (formerly experimental) |

After you identify the class of a unicast address as either A, B, or C, many other related facts can be derived just through memorization. Table 12-3 lists that information for reference and later study; each of these concepts is described in this chapter.

Key Topic

**Table 12-3** Key Facts for Classes A, B, and C

| | Class A | Class B | Class C |
|---|---|---|---|
| First octet range | 1–126 | 128–191 | 192–223 |
| Valid network numbers | 1.0.0.0–126.0.0.0 | 128.0.0.0–191.255.0.0 | 192.0.0.0–223.255.255.0 |
| Total networks | $2^7 - 2 = 126$ | $2^{14} = 16{,}384$ | $2^{21} = 2{,}097{,}152$ |
| Hosts per network | $2^{24} - 2$ | $2^{16} - 2$ | $2^8 - 2$ |
| Octets (bits) in network part | 1 (8) | 2 (16) | 3 (24) |
| Octets (bits) in host part | 3 (24) | 2 (16) | 1 (8) |
| Default mask | 255.0.0.0 | 255.255.0.0 | 255.255.255.0 |

Note that the address ranges of all addresses that begin with 0 and all addresses that begin with 127 are reserved. Had they not been reserved since the creation of Class A networks, as listed in RFC 791 (published in 1981), then they might have been known as Class A networks

0.0.0.0 and 127.0.0.0. Because they are reserved, however, the address space has 126 Class A networks, and not 128. Also, note that there are no similar reserved ranges to begin/end the Class B and C ranges.

In addition to the reservation of what would be Class A networks 0.0.0.0 and 127.0.0.0 for other purposes, other newer RFCs have also reserved small pieces of the Class A, B, and C address space. So, tables like Table 12-3, with the count of the numbers of Class A, B, and C networks, are a good place to get a sense of the size of the number; however, the number of reserved networks does change slightly over time (albeit slowly) based on these other reserved address ranges.

**NOTE** If you are interested in seeing all the reserved IPv4 address ranges, just do an Internet search on "IANA IPv4 special-purpose address registry."

### The Number and Size of the Class A, B, and C Networks

Table 12-3 lists the range of Class A, B, and C network numbers; however, some key points can be lost just referencing a table of information. This section examines the Class A, B, and C network numbers, focusing on the more important points and the exceptions and unusual cases.

First, the number of networks from each class significantly differs. Only 126 Class A networks exist: network 1.0.0.0, 2.0.0.0, 3.0.0.0, and so on, up through network 126.0.0.0. However, 16,384 Class B networks exist, with more than 2 million Class C networks.

Next, note that the size of networks from each class also significantly differs. Each Class A network is relatively large—over 16 million host IP addresses per network—so they were originally intended to be used by the largest companies and organizations. Class B networks are smaller, with over 65,000 hosts per network. Finally, Class C networks, intended for small organizations, have 254 hosts in each network. Figure 12-4 summarizes those facts.

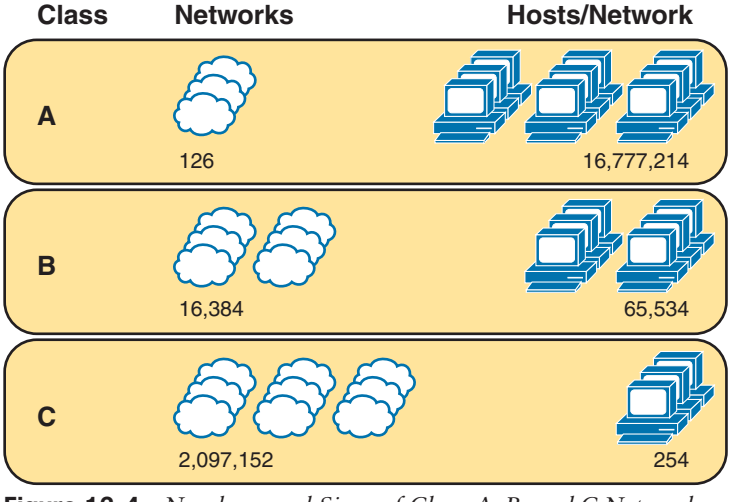



**Figure 12-4** *Numbers and Sizes of Class A, B, and C Networks*

### Address Formats

In some cases, an engineer might need to think about a Class A, B, or C network as if the network has not been subdivided through the subnetting process. In such a case, the addresses in the classful network have a structure with two parts: the **network part** (sometimes called the *prefix*) and the **host part**. Then, comparing any two IP addresses in one network, the following observations can be made:

Key Topic

The addresses in the same network have the same values in the network part.

The addresses in the same network have different values in the host part.

For example, in Class A network 10.0.0.0, by definition, the network part consists of the first octet. As a result, all addresses have an equal value in the network part, namely a 10 in the first octet. If you then compare any two addresses in the network, the addresses have a different value in the last three octets (the host octets). For example, IP addresses 10.1.1.1 and 10.1.1.2 have the same value (10) in the network part, but different values in the host part.

Figure 12-5 shows the format and sizes (in number of bits) of the network and host parts of IP addresses in Class A, B, and C networks, before any subnetting has been applied.

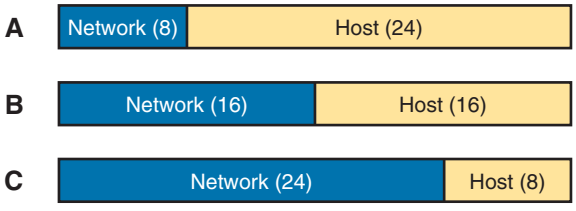



**Figure 12-5** *Sizes (Bits) of the Network and Host Parts of Unsubnetted Classful Networks*

### Default Masks

Although we humans can easily understand the concepts behind Figure 12-5, computers prefer numbers. To communicate those same ideas to computers, each network class has an associated **default mask** that defines the size of the network and host parts of an unsubnetted Class A, B, and C network. To do so, the mask lists binary 1s for the bits considered to be in the network part and binary 0s for the bits considered to be in the host part.

For example, Class A network 10.0.0.0 has a network part of the first single octet (8 bits) and a host part of the last three octets (24 bits). As a result, the Class A default mask is 255.0.0.0, which in binary is

11111111 00000000 00000000 00000000

Figure 12-6 shows default masks for each network class, both in binary and dotted-decimal format.

**NOTE** Decimal 255 converts to the binary value 11111111. Decimal 0, converted to 8-bit binary, is 00000000. See Appendix A, "Numeric Reference Tables," for a conversion table.

Key Topic

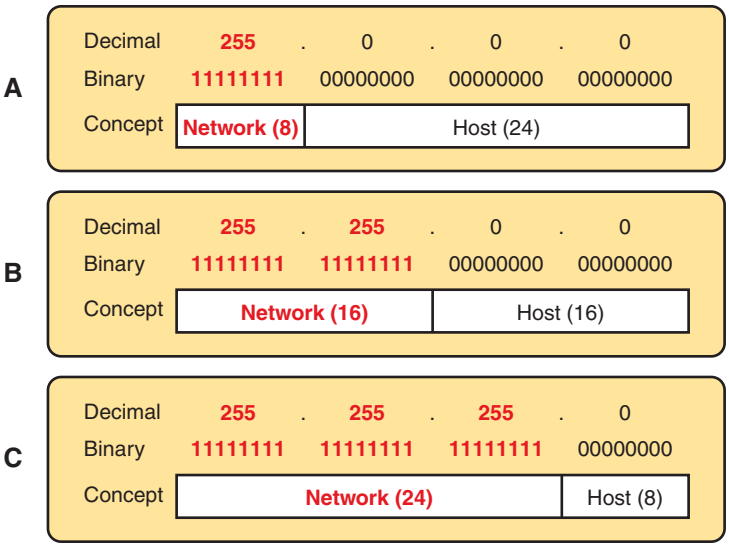



**Figure 12-6** *Default Masks for Classes A, B, and C*

## Number of Hosts per Network

Calculating the number of hosts per network requires some basic binary math. First, consider a case where you have a single binary digit. How many unique values are there? There are, of course, two values: 0 and 1. With 2 bits, you can make four combinations: 00, 01, 10, and 11. As it turns out, the total combination of unique values you can make with N bits is $2^N$.

Host addresses—the IP addresses assigned to hosts—must be unique. The host bits exist for the purpose of giving each host a unique IP address by virtue of having a different value in the host part of the addresses. So, with H host bits, $2^H$ unique combinations exist.

However, the number of hosts in a network is not $2^H$; instead, it is $2^H - 2$. Each network reserves two numbers that would have otherwise been useful as host addresses but have instead been reserved for special use: one for the **network ID** and one for the **network broadcast address**. As a result, the formula to calculate the number of host addresses per Class A, B, or C network is

Key Topic

$$2^H - 2$$

where $H$ is the number of host bits.

## Deriving the Network ID and Related Numbers

Each classful network has four key numbers that describe the network. You can derive these four numbers if you start with just one IP address in the network. The numbers are as follows:

- Network number
- First (numerically lowest) usable address
- Last (numerically highest) usable address
- Network broadcast address

First, consider both the network number and first usable IP address. The *network number*, also called the *network ID* or **network address**, identifies the network. By definition, the network number is the numerically lowest number in the network. However, to prevent any ambiguity, the people who made up IP addressing added the restriction that the network number cannot be assigned as an IP address. So, the lowest number in the network is the network ID. Then, the first (numerically lowest) host IP address is *one larger than* the network number.

Next, consider the network broadcast address along with the last (numerically highest) usable IP address. The TCP/IP RFCs define a network broadcast address as a special address in each network. This broadcast address could be used as the destination address in a packet, and the routers would forward a copy of that one packet to all hosts in that classful network. Numerically, a network broadcast address is always the highest (last) number in the network. As a result, the highest (last) number usable as an IP address is the address that is *one less than* the network broadcast address.

Simply put, if you can find the network number and network broadcast address, finding the first and last usable IP addresses in the network is easy. For the exam, you should be able to find all four values with ease; the process is as follows:

Key Topic

**Step 1.** Determine the class (A, B, or C) based on the first octet.

**Step 2.** Mentally divide the network and host octets based on the class.

**Step 3.** To find the network number, change the IP address's host octets to 0.

**Step 4.** To find the first address, add 1 to the fourth octet of the network ID.

**Step 5.** To find the broadcast address, change the network ID's host octets to 255.

**Step 6.** To find the last address, subtract 1 from the fourth octet of the network broadcast address.

The written process actually looks harder than it is. Figure 12-7 shows an example of the process, using Class A IP address 10.17.18.21, with the circled numbers matching the process.

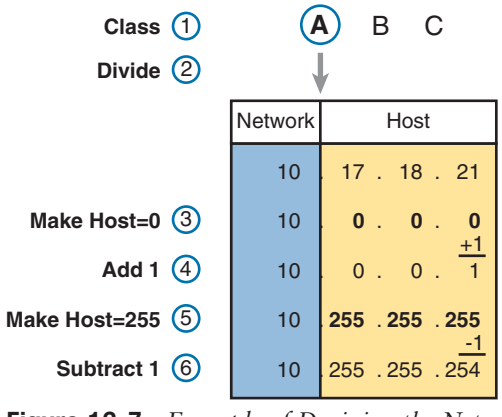



**Figure 12-7** *Example of Deriving the Network ID and Other Values from 10.17.18.21*

Figure 12-7 shows the identification of the class as Class A (Step 1) and the number of network/host octets as 1 and 3, respectively. So, to find the network ID at Step 3, the figure copies only the first octet, setting the last three (host) octets to 0. At Step 4, just copy the network ID and add 1 to the fourth octet. Similarly, to find the broadcast address at Step 5, copy the network octets but set the host octets to 255. Then, at Step 6, subtract 1 from the fourth octet to find the last (numerically highest) usable IP address.

Just to show an alternative example, consider IP address 172.16.8.9. Figure 12-8 shows the process applied to this IP address.

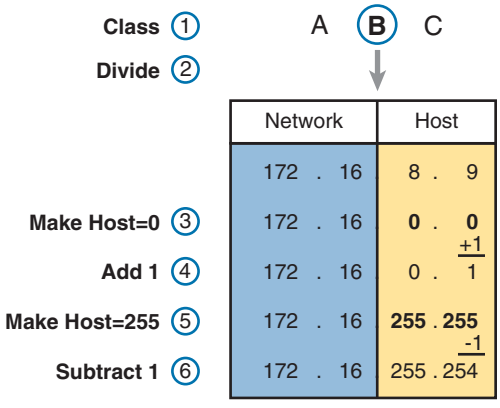



**Figure 12-8** *Example Deriving the Network ID and Other Values from 172.16.8.9*

Figure 12-8 shows the identification of the class as Class B (Step 1) and the number of network/host octets as 2 and 2, respectively. So, to find the network ID at Step 3, the figure copies only the first two octets, setting the last two (host) octets to 0. Similarly, Step 5 shows the same action, but with the last two (host) octets being set to 255.

## Unusual Network IDs and Network Broadcast Addresses

Some of the more unusual numbers in and around the range of Class A, B, and C network numbers can cause some confusion. This section lists some examples of numbers that make many people make the wrong assumptions about the meaning of the number.

For Class A, the first odd fact is that the range of values in the first octet omits the numbers 0 and 127. As it turns out, what would be Class A network 0.0.0.0 was originally reserved for some broadcasting requirements, so all addresses that begin with 0 in the first octet are reserved. What would be Class A network 127.0.0.0 is still reserved because of a special address used in software testing, called the loopback address (127.0.0.1).

For Class B (and C), some of the network numbers can look odd, particularly if you fall into a habit of thinking that 0s at the end mean the number is a network ID, and 255s at the end mean it's a network broadcast address. First, Class B network numbers range from 128.0.0.0 to 191.255.0.0, for a total of $2^{14}$ networks. However, even the first (lowest number) Class B network number (128.0.0.0) looks a little like a Class A network number because it ends with three 0s. However, the first octet is 128, making it a Class B network with a two-octet network part (128.0).

For another Class B example, the high end of the Class B range also might look strange at first glance (191.255.0.0), but this is indeed the numerically highest of the valid Class B network numbers. This network's broadcast address, 191.255.255.255, might look a little like a Class A broadcast address because of the three 255s at the end, but it is indeed the broadcast address of a Class B network.

Similarly to Class B networks, some of the valid Class C network numbers do look strange. For example, Class C network 192.0.0.0 looks a little like a Class A network because of the last three octets being 0, but because it is a Class C network, it consists of all addresses that begin with three octets equal to 192.0.0. Similarly, 223.255.255.0, another valid Class C network, consists of all addresses that begin with 223.255.255.

## Practice with Classful Networks

As with all areas of IP addressing and subnetting, you need to practice to be ready for the CCNA exam. You should practice some while reading this chapter to make sure that you understand the processes. At that point, you can use your notes and this book as a reference, with a goal of understanding the process. After that, keep practicing this and all the other subnetting processes. Before you take the exam, you should be able to always get the right answer, and with speed. Table 12-4 summarizes the key concepts and suggestions for this two-phase approach.

**Table 12-4** Keep-Reading and Take-Exam Goals for This Chapter's Topics

| Time Frame | After Reading This Chapter | Before Taking the Exam |
|---|---|---|
| Focus on… | Learning how | Being correct and fast |
| Tools Allowed | All | Your brain and a notepad |
| Goal: Accuracy | 90% correct | 100% correct |
| Goal: Speed | Any speed | 10 seconds |

### Practice Deriving Key Facts Based on an IP Address

Practice finding the various facts that can be derived from an IP address, as discussed throughout this chapter. To do so, complete Table 12-5.

**Table 12-5** Practice Problems: Find the Network ID and Network Broadcast

| | IP Address | Class | Network Octets | Host Octets | Network ID | Network Broadcast Address |
|---|---|---|---|---|---|---|
| 1 | 1.1.1.1 | | | | | |
| 2 | 128.1.6.5 | | | | | |
| 3 | 200.1.2.3 | | | | | |
| 4 | 192.192.1.1 | | | | | |
| 5 | 126.5.4.3 | | | | | |
| 6 | 200.1.9.8 | | | | | |
| 7 | 192.0.0.1 | | | | | |
| 8 | 191.255.1.47 | | | | | |
| 9 | 223.223.0.1 | | | | | |

The answers are listed in the section "Answers to Earlier Practice Problems," later in this chapter.

### Practice Remembering the Details of Address Classes

Tables 12-2 and 12-3, shown earlier in this chapter, summarized some key information about IPv4 address classes. Tables 12-6 and 12-7 show sparse versions of these same tables. To practice recalling those key facts, particularly the range of values in the first octet that identifies the address class, complete these tables. Then, refer to Tables 12-2 and 12-3 to check your answers. Repeat this process until you can recall all the information in the tables.

**Table 12-6** Sparse Study Table Version of Table 12-2

| Class | First Octet Values | Purpose |
|---|---|---|
| A | | |
| B | | |
| C | | |
| D | | |
| E | | |

**Table 12-7** Sparse Study Table Version of Table 12-3

| | Class A | Class B | Class C |
|---|---|---|---|
| First octet range | | | |
| Valid network numbers | | | |
| Total networks | | | |
| Hosts per network | | | |
| Octets (bits) in network part | | | |
| Octets (bits) in host part | | | |
| Default mask | | | |

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 12-8 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 12-8** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Practice analyzing classful IPv4 networks | | Website, Appendix D |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 12-9** Key Topics for Chapter 12

| Key Topic Elements | Description | Page Number |
|---|---|---|
| Table 12-2 | Address classes | 328 |
| Table 12-3 | Key facts about Class A, B, and C networks | 328 |
| List | Comparisons of network and host parts of addresses in the same classful network | 330 |
| Figure 12-6 | Default masks | 331 |
| Paragraph | Function to calculate the number of hosts per network | 331 |
| List | Steps to find information about a classful network | 332 |

## Key Terms You Should Know

CIDR block, classful IP network, default mask, host part, network, network address, network broadcast address, network ID, network number, network part, unicast IP address

## Additional Practice for This Chapter's Processes

For additional practice with analyzing classful networks, you may do a set of practice problems using your choice of tools:

**Application:** From the companion website, in the section titled "Memory Tables and Practice Exercises," use the "Practice Exercise: Analyzing Classful IPv4 Networks."

**PDF:** Practice the same problems using companion website Appendix D, "Practice for Chapter 12: Analyzing Classful IPv4 Networks."

## Answers to Earlier Practice Problems

Table 12-5, shown earlier, listed several practice problems. Table 12-10 lists the answers.

**Table 12-10** Practice Problems: Find the Network ID and Network Broadcast

| | IP Address | Class | Network Octets | Host Octets | Network ID | Network Broadcast |
|---|---|---|---|---|---|---|
| 1 | 1.1.1.1 | A | 1 | 3 | 1.0.0.0 | 1.255.255.255 |
| 2 | 128.1.6.5 | B | 2 | 2 | 128.1.0.0 | 128.1.255.255 |
| 3 | 200.1.2.3 | C | 3 | 1 | 200.1.2.0 | 200.1.2.255 |
| 4 | 192.192.1.1 | C | 3 | 1 | 192.192.1.0 | 192.192.1.255 |
| 5 | 126.5.4.3 | A | 1 | 3 | 126.0.0.0 | 126.255.255.255 |

| | IP Address | Class | Network Octets | Host Octets | Network ID | Network Broadcast |
|---|---|---|---|---|---|---|
| 6 | 200.1.9.8 | C | 3 | 1 | 200.1.9.0 | 200.1.9.255 |
| 7 | 192.0.0.1 | C | 3 | 1 | 192.0.0.0 | 192.0.0.255 |
| 8 | 191.255.1.47 | B | 2 | 2 | 191.255.0.0 | 191.255.255.255 |
| 9 | 223.223.0.1 | C | 3 | 1 | 223.223.0.0 | 223.223.0.255 |

The class, number of network octets, and number of host octets all require you to look at the first octet of the IP address to determine the class. If a value is between 1 and 126, inclusive, the address is a Class A address, with one network and three host octets. If a value is between 128 and 191 inclusive, the address is a Class B address, with two network and two host octets. If a value is between 192 and 223, inclusive, it is a Class C address, with three network octets and one host octet.

The last two columns can be found based on Table 12-3, specifically the number of network and host octets along with the IP address. To find the network ID, copy the IP address, but change the host octets to 0. Similarly, to find the network broadcast address, copy the IP address, but change the host octets to 255.

The last three problems can be confusing and were included on purpose so that you could see an example of these unusual cases, as follows.

## Answers to Practice Problem 7 (from Table 12-5)

Consider IP address 192.0.0.1. First, 192 is on the lower edge of the first octet range for Class C; as such, this address has three network octets and one host octet. To find the network ID, copy the address, but change the single host octet (the fourth octet) to 0, for a network ID of 192.0.0.0. It looks strange, but it is indeed the network ID.

The network broadcast address choice for problem 7 can also look strange. To find the broadcast address, copy the IP address (192.0.0.1), but change the last octet (the only host octet) to 255, for a broadcast address of 192.0.0.255. In particular, if you decide that the broadcast should be 192.255.255.255, you might have fallen into the trap of logic, like "Change all 0s in the network ID to 255s," which is not the correct logic. Instead, change all host octets in the IP address (or network ID) to 255s.

## Answers to Practice Problem 8 (from Table 12-5)

The first octet of problem 8 (191.255.1.47) sits on the upper edge of the Class B range for the first octet (128–191). As such, to find the network ID, change the last two octets (host octets) to 0, for a network ID of 191.255.0.0. This value sometimes gives people problems because they are used to thinking that 255 somehow means the number is a broadcast address.

The broadcast address, found by changing the two host octets to 255, means that the broadcast address is 191.255.255.255. It looks more like a broadcast address for a Class A network, but it is actually the broadcast address for Class B network 191.255.0.0.

## Answers to Practice Problem 9 (from Table 12-5)

Problem 9, with IP address 223.223.0.1, is near the high end of the Class C range. As a result, only the last (host) octet is changed to 0 to form the network ID 223.223.0.0. It looks a little like a Class B network number at first glance because it ends in two octets of 0. However, it is indeed a Class C network ID (based on the value in the first octet).



CHAPTER 13

# Analyzing Subnet Masks

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.6 Configure and verify IPv4 addressing and subnetting**

The subnet mask used in one or many subnets in an IP internetwork says a lot about the intent of the subnet design. First, the mask divides addresses into two parts: *prefix* and *host*, with the host part defining the size of the subnet (that is, the number of hosts in the subnet). Then, the class (A, B, or C) further divides the structure of addresses in a subnet, breaking the prefix part into the *network* and *subnet* parts. The subnet part defines the number of subnets that could exist inside one classful IP network, assuming that one mask is used throughout the classful network.

The subnet mask holds the key to understanding several important subnetting design points. However, to analyze a subnet mask, you first need some basic math skills with masks. The math converts masks between the three different formats used to represent a mask:

- Binary
- Dotted-decimal notation (DDN)
- Prefix (also called classless interdomain routing [CIDR])

This chapter has two major sections. The first focuses on the mask formats and the math used to convert between the three formats. The second section explains how to take an IP address and its subnet mask and analyze those values. In particular, it shows how to determine the three-part format of the IPv4 address and describes the facts about the subnetting design that are implied by the mask.

**NOTE** The majority of the chapter assumes subnetting of a Class A, B, or C network. The end of the chapter provides some discussion of the similar case of subnetting a public CIDR block.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 13-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Subnet Mask Conversion | 1–3 |
| Identifying Subnet Design Choices Using Masks | 4–7 |

1. Which of the following answers lists the prefix (CIDR) format equivalent of 255.255.254.0?
   - **a.** /19
   - **b.** /20
   - **c.** /23
   - **d.** /24
   - **e.** /25
2. Which of the following answers lists the prefix (CIDR) format equivalent of 255.255.255.240?
   - **a.** /26
   - **b.** /28
   - **c.** /27
   - **d.** /30
   - **e.** /29
3. Which of the following answers lists the dotted-decimal notation (DDN) equivalent of /30?
   - **a.** 255.255.255.192
   - **b.** 255.255.255.252
   - **c.** 255.255.255.240
   - **d.** 255.255.254.0
   - **e.** 255.255.255.0
4. Working at the help desk, you receive a call and learn a user's PC IP address and mask (10.55.66.77, mask 255.255.255.0). When thinking about this using classful logic, you determine the number of network (N), subnet (S), and host (H) bits. Which of the following is true in this case?
   - **a.** N=12
   - **b.** S=12
   - **c.** H=8
   - **d.** S=8
   - **e.** N=24
5. Working at the help desk, you receive a call and learn a user's PC IP address and mask (192.168.9.1/27). When thinking about this using classful logic, you determine the number of network (N), subnet (S), and host (H) bits. Which of the following is true in this case?
   - **a.** N=24
   - **b.** S=24

**c.** H=8

**d.** H=7

**6.** Which of the following statements is true about classless IP addressing concepts?

**a.** Uses a 128-bit IP address

**b.** Applies only for Class A and B networks

**c.** Separates IP addresses into network, subnet, and host parts

**d.** Ignores Class A, B, and C network rules

**7.** Which of the following masks, when used as the only mask within a Class B network, would supply enough subnet bits to support 100 subnets? (Choose two answers.)

**a.** /24

**b.** 255.255.255.252

**c.** /20

**d.** 255.255.252.0

# Foundation Topics

## Subnet Mask Conversion

This section describes how to convert between different formats for the subnet mask. You can then use these processes when you practice. If you already know how to convert from one format to the other, go ahead and move to the section "Practice Converting Subnet Masks," later in this chapter.

### Three Mask Formats

Subnet masks can be written as 32-bit binary numbers, but not just any binary number. In particular, the binary subnet mask must follow these rules:




- The value must not interleave 1s and 0s.
- If 1s exist, they are on the left.
- If 0s exist, they are on the right.

For example, the following values would be illegal. The first is illegal because the value interleaves 0s and 1s, and the second is illegal because it lists 0s on the left and 1s on the right:

```
10101010 01010101 11110000 00001111
00000000 00000000 00000000 11111111
```

The following two binary values meet the requirements, in that they have all 1s on the left, followed by all 0s, with no interleaving of 1s and 0s:

```
11111111 00000000 00000000 00000000
11111111 11111111 11111111 00000000
```

Two alternative subnet mask formats exist so that we humans do not have to work with 32-bit binary numbers. One format, **dotted-decimal notation (DDN)**, converts each set of 8 bits into the decimal equivalent. For example, the two previous binary masks would convert to the following DDN subnet masks because binary 11111111 converts to decimal 255, and binary 00000000 converts to decimal 0:

255.0.0.0

255.255.255.0

Although the DDN format has been around since the beginning of IPv4 addressing, the third mask format was added later, in the early 1990s: the *prefix* format. This format takes advantage of the rule that the subnet mask starts with some number of 1s, and then the rest of the digits are 0s. Prefix format lists a slash (/) followed by the number of binary 1s in the **binary mask**. Using the same two examples as earlier in this section, the prefix format equivalent masks are as follows:

/8

/24

Note that although the terms *prefix* or **prefix mask** can be used, the terms **CIDR mask** or *slash mask* can also be used. This newer prefix style mask was created around the same time as the classless interdomain routing (CIDR) specification back in the early 1990s, and the acronym CIDR grew to be used for anything related to CIDR, including prefix-style masks. In addition, the term *slash mask* is sometimes used because the value includes a slash mark (/).

You need to get comfortable working with masks in different formats. The rest of this section examines how to convert between the three formats.

## Converting Between Binary and Prefix Masks

Converting between binary and prefix masks should be relatively intuitive after you know that the prefix value is simply the number of binary 1s in the binary mask. For the sake of completeness, the processes to convert in each direction are

Key Topic

**Binary to prefix:** Count the number of binary 1s in the binary mask, and write the total, in decimal, after a /.

**Prefix to binary:** Write P binary 1s, where P is the prefix value, followed by as many binary 0s as required to create a 32-bit number.

Tables 13-2 and 13-3 show some examples.

**Table 13-2** Example Conversions: Binary to Prefix

| Binary Mask | Logic | Prefix Mask |
|---|---|---|
| 11111111 11111111 11000000 00000000 | Count 8 + 8 + 2 = 18 binary 1s | /18 |
| 11111111 11111111 11111111 11110000 | Count 8 + 8 + 8 + 4 = 28 binary 1s | /28 |
| 11111111 11111000 00000000 00000000 | Count 8 + 5 = 13 binary 1s | /13 |

**Table 13-3** Example Conversions: Prefix to Binary

| Prefix Mask | Logic | Binary Mask |
|---|---|---|
| /18 | Write 18 1s, then 14 0s, total 32 | 11111111 11111111 11000000 00000000 |
| /28 | Write 28 1s, then 4 0s, total 32 | 11111111 11111111 11111111 11110000 |
| /13 | Write 13 1s, then 19 0s, total 32 | 11111111 11111000 00000000 00000000 |

## Converting Between Binary and DDN Masks

By definition, a dotted-decimal number (DDN) used with IPv4 addressing contains four decimal numbers, separated by dots. Each decimal number represents 8 bits. So, a single DDN shows four decimal numbers that together represent some 32-bit binary number.

Conversion from a DDN mask to the binary equivalent is relatively simple to describe but can be laborious to perform. First, to do the conversion, the process is as follows:

For each octet, perform a decimal-to-binary conversion.

However, depending on your comfort level with doing decimal-to-binary conversions, that process can be difficult or time-consuming. If you want to think about masks in binary for the exam, consider picking one of the following methods to do the conversion and practicing until you can do it quickly and accurately:

- Do the decimal-binary conversions, but practice your decimal-binary conversions to get fast. If you choose this path, consider the Cisco Binary Game, which you can find by searching its name at the Cisco Learning Network (CLN), or try this link: https://learningnetwork.cisco.com/s/binary-game.
- Use the decimal-binary conversion chart in Appendix A, "Numeric Reference Tables." This lets you find the answer more quickly now, but you cannot use the chart on exam day.
- Memorize the nine possible decimal values that can be in a **decimal mask**, and practice using a reference table with those values.

The third method, which is the method recommended in this book, takes advantage of the fact that any and every DDN mask octet must be one of only nine values. Why? Well, remember how a binary mask cannot interleave 1s and 0s, and the 0s must be on the right? It turns out that only nine different 8-bit binary numbers conform to these rules. Table 13-4 lists the values, along with other relevant information.

Key Topic

**Table 13-4** Nine Possible Values in One Octet of a Subnet Mask

| Binary Mask Octet | Decimal Equivalent | Number of Binary 1s |
|---|---|---|
| 00000000 | 0 | 0 |
| 10000000 | 128 | 1 |
| 11000000 | 192 | 2 |
| 11100000 | 224 | 3 |

Answers to the "Do I Know This Already?" quiz:

**1** C **2** B **3** B **4** C **5** A **6** D **7** A, B

| Binary Mask Octet | Decimal Equivalent | Number of Binary 1s |
|---|---|---|
| 11110000 | 240 | 4 |
| 11111000 | 248 | 5 |
| 11111100 | 252 | 6 |
| 11111110 | 254 | 7 |
| 11111111 | 255 | 8 |

Many subnetting processes can be done with or without binary math. Some of those processes—mask conversion included—use the information in Table 13-4. You should plan to memorize the information in the table. I recommend making a copy of the table to keep handy while you practice. (You will likely memorize the contents of this table simply by practicing the conversion process enough to get both good and fast at the conversion.)

Using the table, the conversion processes in each direction with binary and decimal masks are as follows:

Key Topic

**Binary to decimal:** Organize the bits into four sets of eight. For each octet, find the binary value in the table and write down the corresponding decimal value.

**Decimal to binary:** For each octet, find the decimal value in the table and write down the corresponding 8-bit binary value.

Tables 13-5 and 13-6 show some examples.

**Table 13-5** Conversion Example: Binary to Decimal

| Binary Mask | Logic | Decimal Mask |
|---|---|---|
| 11111111 11111111 11000000 00000000 | 11111111 maps to 255<br>11000000 maps to 192<br>00000000 maps to 0 | 255.255.192.0 |
| 11111111 11111111 11111111 11110000 | 11111111 maps to 255<br>11110000 maps to 240 | 255.255.255.240 |
| 11111111 11111000 00000000 00000000 | 11111111 maps to 255<br>11111000 maps to 248<br>00000000 maps to 0 | 255.248.0.0 |

**Table 13-6** Conversion Examples: Decimal to Binary

| Decimal Mask | Logic | Binary Mask |
|---|---|---|
| 255.255.192.0 | 255 maps to 11111111<br>192 maps to 11000000<br>0 maps to 00000000 | 11111111 11111111 11000000 00000000 |
| 255.255.255.240 | 255 maps to 11111111<br>240 maps to 11110000 | 11111111 11111111 11111111 11110000 |

| Decimal Mask | Logic | Binary Mask |
|---|---|---|
| 255.248.0.0 | 255 maps to 11111111<br>248 maps to 11111000<br>0 maps to 00000000 | 11111111 11111000 00000000 00000000 |

## Converting Between Prefix and DDN Masks

When you are learning, the best way to convert between the prefix and decimal formats is to first convert to binary. For example, to move from decimal to prefix, first convert decimal to binary and then from binary to prefix.

For the exams, set a goal to master these conversions doing the math in your head. While learning, you will likely want to use paper. To train yourself to do all this without writing it down, instead of writing each octet of binary, just write the number of binary 1s in that octet.

Figure 13-1 shows an example with a prefix-to-decimal conversion. The left side shows the conversion to binary as an interim step. For comparison, the right side shows the binary interim step in shorthand that just lists the number of binary 1s in each octet of the binary mask.

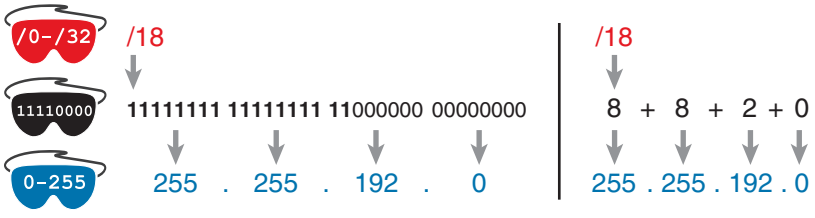



**Figure 13-1** *Conversion from Prefix to Decimal: Full Binary Versus Shorthand*

Similarly, when converting from decimal to prefix, mentally convert to binary along the way, and as you improve, just think of the binary as the number of 1s in each octet. Figure 13-2 shows an example of such a conversion.

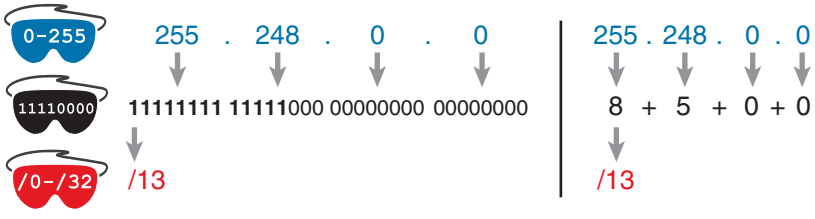



**Figure 13-2** *Conversion from Decimal to Prefix: Full Binary Versus Shorthand*

Note that Appendix A has a table that lists all 33 legal subnet masks, with all three formats shown.

## Practice Converting Subnet Masks

Before moving to the second half of this chapter, and thinking about what these subnet masks mean, first do some practice. Practice the processes discussed in this chapter until you get the right answer most of the time. Later, before taking the exam, practice more until

you master the topics in this chapter and can move pretty fast, as outlined in the right column of Table 13-7.

**Table 13-7** Keep-Reading and Take-Exam Goals for This Chapter's Topics

| Time Frame | Before Moving to the Next Section | Before Taking the Exam |
|---|---|---|
| Focus On… | Learning how | Being correct and fast |
| Tools Allowed | All | Your brain and a notepad |
| Goal: Accuracy | 90% correct | 100% correct |
| Goal: Speed | Any speed | 10 seconds |

Table 13-8 lists eight practice problems. The table has three columns, one for each mask format. Each row lists one mask, in one format. Your job is to find the mask's value in the other two formats for each row. Table 13-12, located in the section "Answers to Earlier Practice Problems," later in this chapter, lists the answers.

**Table 13-8** Practice Problems: Find the Mask Values in the Other Two Formats

| Prefix | Binary Mask | Decimal |
|---|---|---|
| | 11111111 11111111 11000000 00000000 | |
| | | 255.255.255.252 |
| /25 | | |
| /16 | | |
| | | 255.0.0.0 |
| | 11111111 11111111 111111**00** 00000000 | |
| | | 255.254.0.0 |
| /27 | | |

## Identifying Subnet Design Choices Using Masks

Subnet masks have many purposes. In fact, if ten experienced network engineers were independently asked, "What is the purpose of a subnet mask?" the engineers would likely give a variety of true answers. The subnet mask plays several roles.

This chapter focuses on one particular use of a subnet mask: defining the prefix part of the IP addresses in a subnet. The prefix part must be the same value for all addresses in a subnet. In fact, a single subnet can be defined as all IPv4 addresses that have the same value in the prefix part of their IPv4 addresses.

While the previous paragraph might sound a bit formal, the idea is relatively basic, as shown in Figure 13-3. The right side of the figure shows a subnet of all addresses that begin with 172.16.2 and another subnet made of all addresses that begin with 172.16.3. In this example, the prefix—the part that has the same value in all the addresses in the subnet—is the first three octets.

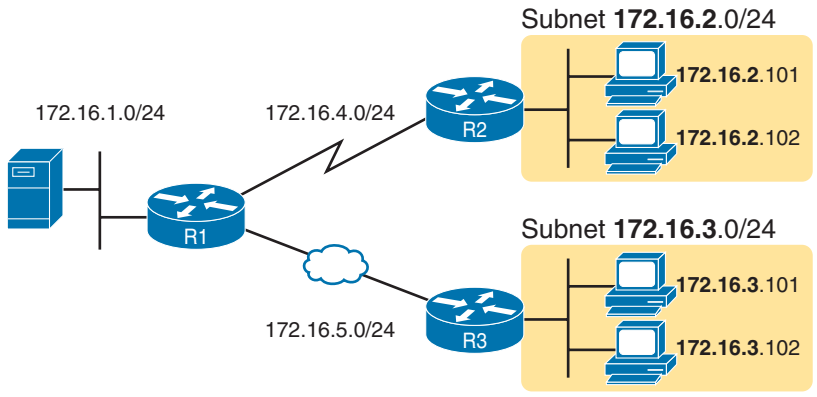



**Figure 13-3** *Simple Subnet Design, with Mask /24*

While people can sit around a conference table and talk about how a prefix is three octets long, computers communicate that same concept using a subnet mask. In this case, the subnets use a subnet mask of /24, which means that the prefix part of the addresses is 24 bits (3 octets) long.

This section explains more about how to use a subnet mask to understand this concept of a prefix part of an IPv4 address, along with these other uses for a subnet mask. Note that this section discusses the first five items in the list.

Key Topic

- Defines the size of the prefix (combined network and subnet) part of the addresses in a subnet
- Defines the size of the host part of the addresses in the subnet
- Can be used to calculate the number of hosts in the subnet
- Provides a means for the network designer to communicate the design details—the number of subnet and host bits—to the devices in the network
- Under certain assumptions, can be used to calculate the number of subnets in the entire classful network
- Can be used in binary calculations of both the subnet ID and the subnet broadcast address

## Masks Divide the Subnet's Addresses into Two Parts

The subnet mask subdivides the IP addresses in a subnet into two parts: the *prefix*, or *subnet part*, and the *host part*.

The prefix part identifies the addresses that reside in the same subnet because all IP addresses in the same subnet have the same value in the prefix part of their addresses. The idea is much like the postal code (ZIP codes in the United States) in mailing addresses. All mailing addresses in the same town have the same postal code. Likewise, all IP addresses in the same subnet have identical values in the prefix part of their addresses.

The host part of an address identifies the host uniquely inside the subnet. If you compare any two IP addresses in the same subnet, their host parts will differ, even though the prefix parts of their addresses have the same value. To summarize these key comparisons:

Key Topic

**Prefix (subnet) part:** Equal in all addresses in the same subnet

**Host part:** Different in all addresses in the same subnet

For example, imagine a subnet that, in concept, includes all addresses whose first three octets are 10.1.1. So, the following list shows several addresses in this subnet:

10.1.1.**1**

10.1.1.**2**

10.1.1.**3**

In this list, the prefix or subnet parts (the first three octets of 10.1.1) are equal. The host part (the last octet [in bold]) is different. So, the prefix or subnet part of the address identifies the group, and the host part identifies the specific member of the group.

The subnet mask defines the dividing line between the prefix and the host part. To do so, the mask creates a conceptual line between the binary 1s in the binary mask and the binary 0s in the mask. In short, if a mask has P binary 1s, the prefix part is P bits long and the rest of the bits are host bits. Figure 13-4 shows the concept.

Key Topic

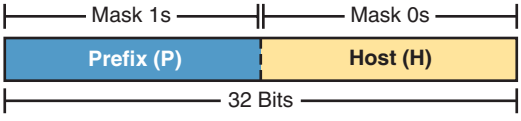



**Figure 13-4** *Prefix (Subnet) and Host Parts Defined by Mask 1s and 0s*

Figure 13-5, shows a specific example using mask 255.255.255.0. Mask 255.255.255.0 (/24) has 24 binary 1s, for a prefix length of 24 bits.

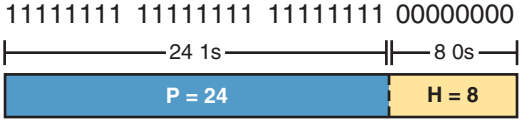



**Figure 13-5** *Mask 255.255.255.0: P=24, H=8*

## Masks and Class Divide Addresses into Three Parts

In addition to the two-part view of IPv4 addresses, you can also think about IPv4 addresses as having three parts. When subnetting a class A, B, or C network, just apply Class A, B, and C rules to the address format to define the network part at the beginning of the address. This added logic divides the prefix into two parts: the *network* part and the *subnet* part. The class defines the length of the network part, with the subnet part simply being the rest of the prefix. Figure 13-6 shows the idea.

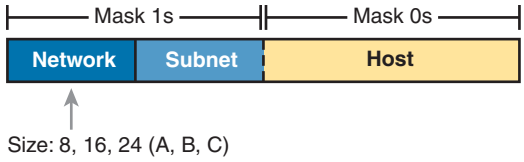



**Figure 13-6** *Class Concepts Applied to Create Three Parts*

The combined network and subnet parts act like the prefix because all addresses in the same subnet must have identical values in the network and subnet parts. The size of the host part remains unchanged, whether viewing the addresses as having two parts or three parts.

To be complete, Figure 13-7 shows the same example as in the previous section, with the subnet of "all addresses that begin with 10.1.1." In that example, the subnet uses mask 255.255.255.0, and the addresses are all in Class A network 10.0.0.0. The class defines 8 network bits, and the mask defines 24 prefix bits, meaning that 24 – 8 = 16 subnet bits exist. The host part remains as 8 bits per the mask.

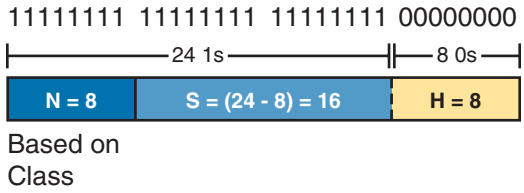



**Figure 13-7** *Subnet 10.1.1.0, Mask 255.255.255.0: N=8, S=16, H=8*

## Classless and Classful Addressing

The terms **classless addressing** and **classful addressing** refer to the two different ways to think about IPv4 addresses as described so far in this chapter. Classful addressing means that you think about Class A, B, and C rules, so the prefix is separated into the network and subnet parts, as shown in Figures 13-6 and 13-7. Classless addressing means that you ignore the Class A, B, and C rules and treat the prefix part as one part, as shown in Figures 13-4 and 13-5. The following more formal definitions are listed for reference and study:

**Classless addressing:** The concept that an IPv4 address has two parts—the prefix part plus the host part—as defined by the mask, with *no consideration of the class* (A, B, or C).

**Classful addressing:** The concept that an IPv4 address has three parts—network, subnet, and host—as defined by the mask *and Class A, B, and C rules*.

**NOTE** Unfortunately, the networking world uses the terms *classless* and *classful* in a couple of different ways. In addition to the classless and classful addressing described here, each routing protocol can be categorized as either a *classless routing protocol* or a *classful routing protocol*. In another use, the terms *classless routing* and *classful routing* refer to some details of how Cisco routers forward (route) packets using the default route in some cases. As a result, these terms can be easily confused and misused. So, when you see the words *classless* and *classful*, be careful to note the context: addressing, routing, or routing protocols.

## Calculations Based on the IPv4 Address Format

After you know how to break an address down using both classless and classful addressing rules, you can easily calculate a couple of important facts using some basic math formulas.

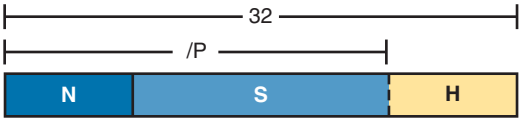
First, for any subnet, after you know the number of host bits, you can calculate the number of host IP addresses in the subnet. Next, if you know the number of subnet bits (using classful addressing concepts) and you know that only one subnet mask is used throughout the network, you can also calculate the number of subnets in the network. The formulas just require that you know the powers of 2:

**Hosts in the subnet:** $2^H - 2$, where H is the number of host bits.

**Subnets in the network:** $2^S$, where S is the number of subnet bits. Only use this formula if only one mask is used throughout the network.

**NOTE** The section "Choose the Mask" in Chapter 11, "Perspectives on IPv4 Subnetting," details many concepts related to masks, including comments about this assumption of one mask throughout a single Class A, B, or C network.

The sizes of the parts of IPv4 addresses can also be calculated. The math is basic, but the concepts are important. Keeping in mind that IPv4 addresses are 32 bits long, the two parts with classless addressing must add up to 32 (P + H = 32), and with classful addressing, the three parts must add up to 32 (N + S + H = 32). Figure 13-8 shows the relationships.




Class:
A: N = 8
B: N = 16
C: N = 24

**Figure 13-8** *Relationship Between /P, N, S, and H*

You often begin with an IP address and mask, both when answering questions on the CCNA exam and when examining problems that occur in real networks. Based on the information in this chapter and earlier chapters, you should be able to find all the information in Figure 13-8 and then calculate the number of hosts/subnet and the number of subnets in the network. For reference, the following process spells out the steps:

Key Topic

**Step 1.** Convert the mask to prefix format (/P) as needed. (See the earlier section "Practice Converting Subnet Masks" for review.)

**Step 2.** Determine N based on the class. (See Chapter 12, "Analyzing Classful IPv4 Networks," for review.)

**Step 3.** Calculate S = P – N.

**Step 4.** Calculate H = 32 – P.

**Step 5.** Calculate hosts/subnet: $2^H - 2$.

**Step 6.** Calculate number of subnets: $2^S$.

For example, consider the case of IP address 8.1.4.5 with mask 255.255.0.0 by following this process:

**Step 1.** 255.255.0.0 = /16, so P=16.

**Step 2.** 8.1.4.5 is in the range 1–126 in the first octet, so it is Class A; so N=8.

**Step 3.** S = P – N = 16 – 8 = 8.

**Step 4.** H = 32 – P = 32 – 16 = 16.

**Step 5.** $2^{16} - 2$ = 65,534 hosts/subnet.

**Step 6.** $2^{8}$ = 256 subnets.

Figure 13-9 shows a visual analysis of the same problem.

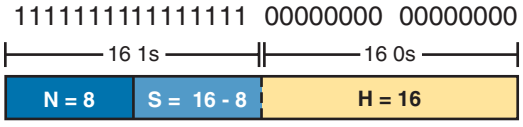



**Figure 13-9** *Visual Representation of Problem: 8.1.4.5, 255.255.0.0*

For another example, consider address 200.1.1.1, mask 255.255.255.252 by following this process:

**Step 1.** 255.255.255.252 = /30, so P=30.

**Step 2.** 200.1.1.1 is in the range 192–223 in the first octet, so it is Class C; so N=24.

**Step 3.** S = P – N = 30 – 24 = 6.

**Step 4.** H = 32 – P = 32 – 30 = 2.

**Step 5.** $2^{2} - 2$ = 2 hosts/subnet.

**Step 6.** $2^{6}$ = 64 subnets.

This example uses a popular mask for serial links because serial links require only two host addresses, and the mask supports only two host addresses.

## Practice Analyzing Subnet Masks

As with the other subnetting math in this book, using a two-phase approach may help. Take time now to practice until you feel as though you understand the process. Then, before the exam, make sure you master the math. Table 13-9 summarizes the key concepts and suggestions for this two-phase approach.

**Table 13-9** Keep-Reading and Take-Exam Goals for This Chapter's Topics

| Time Frame | Before Moving to the Next Chapter | Before Taking the Exam |
|---|---|---|
| Focus On… | Learning how | Being correct and fast |
| Tools Allowed | All | Your brain and a notepad |
| Goal: Accuracy | 90% correct | 100% correct |
| Goal: Speed | Any speed | 15 seconds |

On a piece of scratch paper, answer the following questions. In each case:

- Determine the structure of the addresses in each subnet based on the class and mask, using classful IP addressing concepts. In other words, find the size of the network, subnet, and host parts of the addresses.
- Calculate the number of hosts in the subnet.
- Calculate the number of subnets in the network, assuming that the same mask is used throughout.

1. 8.1.4.5, 255.255.254.0
2. 130.4.102.1, 255.255.255.0
3. 199.1.1.100, 255.255.255.0
4. 130.4.102.1, 255.255.252.0
5. 199.1.1.100, 255.255.255.224

The answers are listed in the section "Answers to Earlier Practice Problems," later in this chapter.

## Masks and CIDR Blocks

As described in the Introduction, most of this chapter, particularly the second major section, expects the more typical case of subnetting a Class A, B, or C network. However, in some cases, a company will use a large public *CIDR block* instead of a classful public IP network or a classful private IP network. While more common today for enterprises to use private Class A, B, and C IP networks for the addresses for most of their hosts, it still helps to understand how you would subnet if using a large public CIDR block instead.

First, the classless addressing view of the mask need not change. However, the classful view of the mask, which interprets the address structure with a network field length based on Class A, B, and C rules, needs to be adjusted. Thankfully, the concepts are straightforward and do not require new processes or math. In short:

> Formerly, you defined a network field based on Class A, B, and C rules. With a CIDR block, instead, use the prefix length specified by the CIDR block.

To explain a little more, note that a CIDR block assignment lists a prefix ID and length. The prefix ID represents the group, like a network ID or subnet ID, and is the numerically lowest number in the CIDR block. The CIDR block's prefix length acts like the block's default mask. For example, the numbers 1.1.0.0/16 imply "all addresses that begin with 1.1." The /16 mask means that all addresses in the CIDR block have the same value in their first 16 bits.

To summarize using Figure 13-10, instead of using Class A, B, or C rules to create an 8-, 16-, or 24-bit network field, begin with a CIDR field based on the prefix length of the CIDR field (16 bits in the previous example).

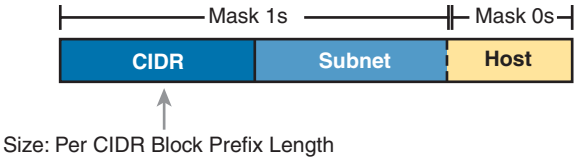



**Figure 13-10** *CIDR and Subnet Masks Divide Addresses into Three Parts*

Just as with the classful view of the address structure, when subnetting the CIDR block, the width of the CIDR field remains the same. When you choose a mask, the more binary 1s, the more subnet bits and subnets in the design. The more host bits in the mask, the more hosts/subnet.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 13-10 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 13-10** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |
| Practice analyzing subnet masks | | Website, Appendix E |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 13-11** Key Topics for Chapter 13

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Rules for binary subnet mask values | 340 |
| List | Rules to convert between binary and prefix masks | 341 |
| Table 13-4 | Nine possible values in a decimal subnet mask | 342 |
| List | Rules to convert between binary and DDN masks | 343 |
| List | Some functions of a subnet mask | 346 |
| List | Comparisons of IP addresses in the same subnet | 347 |
| Figure 13-4 | Two-part classless view of an IP address | 347 |
| Figure 13-6 | Three-part classful view of an IP address | 348 |
| List | Definitions of classful addressing and classless addressing | 348 |
| List | Formal steps to analyze masks and calculate values | 349 |

## Key Terms You Should Know

binary mask, CIDR mask, classful addressing, classless addressing, decimal mask, dotted-decimal notation (DDN), prefix mask

## Additional Practice for This Chapter's Processes



You can do more practice with the processes in this chapter with a pair of practice sets. One focuses on interpreting existing masks, while the other gives you practice with converting between mask formats. You may do each practice set using the following tools:

**Application:** From the companion website, in the section titled "Memory Tables and Practice Exercises," use the "Analyzing Subnet Masks" and "Converting Masks" applications, listed under the Chapter Review for this chapter.

**PDF:** Practice the same problems found in both these apps using companion website Appendix E, "Practice for Chapter 13: Analyzing Subnet Masks."

## Answers to Earlier Practice Problems

Table 13-8, shown earlier, listed several practice problems for converting subnet masks; Table 13-12 lists the answers.

**Table 13-12** Answers to Problems in Table 13-8

| Prefix | Binary Mask | Decimal |
|---|---|---|
| /18 | 11111111 11111111 11000000 00000000 | 255.255.192.0 |
| /30 | 11111111 11111111 11111111 11111100 | 255.255.255.252 |
| /25 | 11111111 11111111 11111111 10000000 | 255.255.255.128 |
| /16 | 11111111 11111111 00000000 00000000 | 255.255.0.0 |
| /8 | 11111111 00000000 00000000 00000000 | 255.0.0.0 |
| /22 | 11111111 11111111 11111100 00000000 | 255.255.252.0 |
| /15 | 11111111 11111110 00000000 00000000 | 255.254.0.0 |
| /27 | 11111111 11111111 11111111 11100000 | 255.255.255.224 |

Table 13-13 lists the answers to the practice problems from the earlier section "Practice Analyzing Subnet Masks."

**Table 13-13** Answers to Problems from Earlier in the Chapter

| | Problem | /P | Class | N | S | H | $2^S$ | $2^H - 2$ |
|---|---|---|---|---|---|---|---|---|
| 1 | 8.1.4.5 255.255.254.0 | 23 | A | 8 | 15 | 9 | 32,768 | 510 |
| 2 | 130.4.102.1 255.255.255.0 | 24 | B | 16 | 8 | 8 | 256 | 254 |
| 3 | 199.1.1.100 255.255.255.0 | 24 | C | 24 | 0 | 8 | N/A | 254 |
| 4 | 130.4.102.1 255.255.252.0 | 22 | B | 16 | 6 | 10 | 64 | 1022 |
| 5 | 199.1.1.100 255.255.255.224 | 27 | C | 24 | 3 | 5 | 8 | 30 |

The following list reviews the problems:

1. For 8.1.4.5, the first octet (8) is in the 1–126 range, so it is a Class A address, with 8 network bits. Mask 255.255.254.0 converts to /23, so P – N = 15, for 15 subnet bits. H can be found by subtracting /P (23) from 32, for 9 host bits.
2. The address 130.4.102.1 is in the 128–191 range in the first octet, making it a Class B address, with N = 16 bits. The 255.255.255.0 converts to /24, so the number of subnet bits is 24 – 16 = 8. With 24 prefix bits, the number of host bits is 32 – 24 = 8.
3. The third problem purposely shows a case where the mask does not create a subnet part of the address. The address, 199.1.1.100, has a first octet between 192 and 223, making it a Class C address with 24 network bits. The prefix version of the mask is /24, so the number of subnet bits is 24 – 24 = 0. The number of host bits is 32 minus the prefix length (24), for a total of 8 host bits. So in this case, the mask shows that the network engineer is using the default mask, which creates no subnet bits and no subnets.
4. With the same address as the second problem, 130.4.102.1 is a Class B address with N = 16 bits. This problem uses a different mask, 255.255.252.0, which converts to /22. This makes the number of subnet bits 22 – 16 = 6. With 22 prefix bits, the number of host bits is 32 – 22 = 10.
5. With the same address as the third problem, 199.1.1.100 is a Class C address with N = 24 bits. This problem uses a different mask, 255.255.255.224, which converts to /27. This makes the number of subnet bits 27 – 24 = 3. With 27 prefix bits, the number of host bits is 32 – 27 = 5.

*This page intentionally left blank*



CHAPTER 14

# Analyzing Existing Subnets

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.6 Configure and verify IPv4 addressing and subnetting**

Often, a networking task begins with the discovery of the IP address and mask used by some host. Then, to understand how the internetwork routes packets to that host, you must find key pieces of information about the subnet, specifically the following:

- Subnet ID
- Subnet broadcast address
- Subnet's range of usable unicast IP addresses

This chapter discusses the concepts and math to take a known IP address and mask, and then fully describe a subnet by finding the values in this list. These specific tasks might well be the most important IP skills in the entire IP addressing and subnetting topics in this book because they might be the most commonly used tasks when operating and troubleshooting real networks. Also note that as with the other chapters, this chapter shows examples that subnet a Class A, B, or C network, rather than a CIDR block.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 14-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Defining a Subnet | 1 |
| Analyzing Existing Subnets: Binary | 2 |
| Analyzing Existing Subnets: Decimal | 3–6 |

1. When you think about an IP address using classful addressing rules, an address can have three parts: network, subnet, and host. If you examined all the addresses in one subnet, in binary, which of the following answers correctly states which of the three parts of the addresses will be equal among all addresses? (Choose the best answer.)
   a. Network part only
   b. Subnet part only
   c. Host part only
   d. Network and subnet parts
   e. Subnet and host parts
2. Which of the following statements are true regarding the binary subnet ID, subnet broadcast address, and host IP address values in any single subnet? (Choose two answers.)
   a. The host part of the broadcast address is all binary 0s.
   b. The host part of the subnet ID is all binary 0s.
   c. The host part of a usable IP address can have all binary 1s.
   d. The host part of any usable IP address must not be all binary 0s.
3. Which of the following is the resident subnet ID for IP address 10.7.99.133/24?
   a. 10.0.0.0
   b. 10.7.0.0
   c. 10.7.99.0
   d. 10.7.99.128
4. Which of the following is the resident subnet for IP address 192.168.44.97/30?
   a. 192.168.44.0
   b. 192.168.44.64
   c. 192.168.44.96
   d. 192.168.44.128
5. Which of the following is the subnet broadcast address for the subnet in which IP address 172.31.77.201/27 resides?
   a. 172.31.201.255
   b. 172.31.255.255
   c. 172.31.77.223
   d. 172.31.77.207
6. A fellow engineer tells you to configure the DHCP server to lease the last 100 usable IP addresses in subnet 10.1.4.0/23. Which of the following IP addresses could be leased as a result of your new configuration?
   a. 10.1.4.156
   b. 10.1.4.254
   c. 10.1.5.200
   d. 10.1.7.200
   e. 10.1.255.200

# Foundation Topics

## Defining a Subnet

An IP subnet is a subset of a classful network, created by choice of some network engineer. However, that engineer cannot pick just any arbitrary subset of addresses; instead, the engineer must follow certain rules, such as the following:

Key Topic

- The subnet contains a set of consecutive numbers.
- The subnet holds $2^H$ numbers, where H is the number of host bits defined by the subnet mask.
- Two special numbers in the range cannot be used as IP addresses:
  - The first (lowest) number acts as an identifier for the subnet (subnet ID).
  - The last (highest) number acts as a *subnet broadcast address*.
- The remaining addresses, whose values sit between the subnet ID and subnet broadcast address, are used as *unicast IP addresses*.

This section reviews and expands the basic concepts of the subnet ID, subnet broadcast address, and range of addresses in a subnet.

### An Example with Network 172.16.0.0 and Four Subnets

Imagine that you work at the customer support center, where you receive all initial calls from users who have problems with their computer. You coach the user through finding her IP address and mask: 172.16.150.41, mask 255.255.192.0. One of the first and most common tasks you will do based on that information is to find the subnet ID of the subnet in which that address resides. (In fact, this subnet ID is sometimes called the **resident subnet** because the IP address exists in or resides in that subnet.)

Before getting into the math, examine the mask (255.255.192.0) and classful network (172.16.0.0) for a moment. From the mask, based on what you learned in Chapter 13, "Analyzing Subnet Masks," you can find the structure of the addresses in the subnet, including the number of host and subnet bits. That analysis tells you that two subnet bits exist, meaning that there should be four ($2^2$) subnets. Figure 14-1 shows the idea.

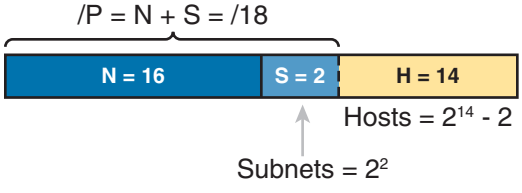



**Figure 14-1** *Address Structure: Class B Network, /18 Mask*

---

Answers to the "Do I Know This Already?" quiz:

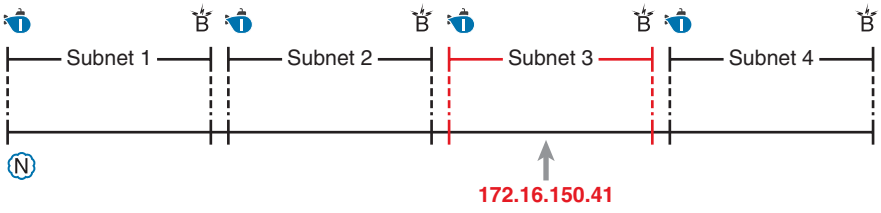
**1** D **2** B, D **3** C **4** C **5** C **6** C

**NOTE** This chapter, like the others in this part of the book, assumes that one mask is used throughout an entire classful network.

Because each subnet uses a single mask, all subnets of this single IP network must be the same size, because all subnets have the same structure. In this example, all four subnets will have the structure shown in the figure, so all four subnets will have $2^{14} - 2$ host addresses.

Next, consider the big picture of what happens with this example subnet design: the one Class B network now has four subnets of equal size. Conceptually, if you represent the entire Class B network as a number line, each subnet consumes one-fourth of the number line, as shown in Figure 14-2. Each subnet has a subnet ID—the numerically lowest number in the subnet—so it sits on the left of the subnet. And each subnet has a **subnet broadcast address**—the numerically highest number in the subnet—so it sits on the right side of the subnet.





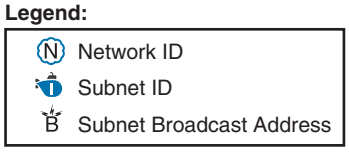
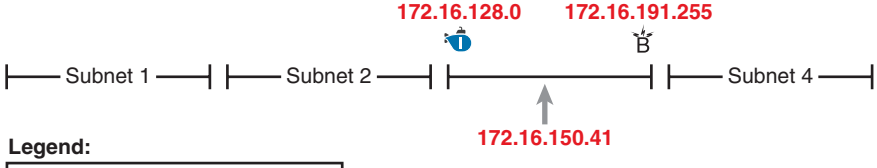




**Figure 14-2** *Network 172.16.0.0, Divided into Four Equal Subnets*

The rest of this chapter focuses on how to take one IP address and mask and discover the details about that one subnet in which the address resides. In other words, you see how to find the resident subnet of an IP address. Again, using IP address 172.16.150.41 and mask 255.255.192.0 as an example, Figure 14-3 shows the resident subnet, along with the subnet ID and subnet broadcast address that bracket the subnet.



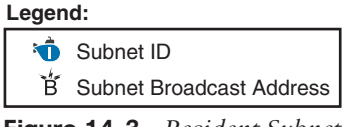




**Figure 14-3** *Resident Subnet for 172.16.150.41, 255.255.192.0*

## Subnet ID Concepts

A subnet ID is simply a number used to succinctly represent a subnet. When listed along with its matching subnet mask, the subnet ID identifies the subnet and can be used to derive the subnet broadcast address and range of addresses in the subnet. Rather than having to write down all these details about a subnet, you simply need to write down the subnet ID and mask, and you have enough information to fully describe the subnet.

The subnet ID appears in many places, but it is seen most often in IP routing tables. For example, when an engineer configures a router with its IP address and mask, the router calculates the subnet ID and puts a route into its routing table for that subnet. The router typically then advertises the subnet ID/mask combination to neighboring routers with some IP routing protocol. Eventually, all the routers in an enterprise learn about the subnet—again using the subnet ID and subnet mask combination—and display it in their routing tables. (You can display the contents of a router's IP routing table using the **show ip route** command.)

Unfortunately, the terminology related to subnets can sometimes cause problems. First, the terms **subnet ID**, **subnet number**, and **subnet address** are synonyms. In addition, people sometimes simply say *subnet* when referring to both the idea of a subnet and the number that is used as the subnet ID. When talking about routing, people sometimes use the term *prefix* instead of *subnet*. The term *prefix* refers to the same idea as *subnet*; it just uses terminology from the classless addressing way to describe IP addresses, as discussed in Chapter 13's section "Classless and Classful Addressing."

The biggest terminology confusion arises between the terms *network* and *subnet*. In the real world, people often use these terms synonymously, and that is perfectly reasonable in some cases. In other cases, the specific meaning of these terms, and their differences, matter to what is being discussed.

For example, people often might say, "What is the network ID?" when they really want to know the subnet ID. In another case, they might want to know the Class A, B, or C network ID. So, when one engineer asks something like, "What's the net ID for 172.16.150.41 slash 18?" use the context to figure out whether he wants the literal classful network ID (172.16.0.0, in this case) or the literal subnet ID (172.16.128.0, in this case).

For the exams, be ready to notice when the terms *subnet* and *network* are used, and then use the context to figure out the specific meaning of the term in that case.

Table 14-2 summarizes the key facts about the subnet ID, along with the possible synonyms, for easier review and study.

Key Topic

**Table 14-2** Summary of Subnet ID Key Facts

| | |
|---|---|
| **Definition** | Number that represents the subnet |
| **Numeric Value** | First (smallest) number in the subnet |
| **Literal Synonyms** | Subnet number, subnet address, prefix, resident subnet |
| **Common-Use Synonyms** | Network, network ID, network number, network address |
| **Typically Seen In…** | Routing tables, documentation |

## Subnet Broadcast Address

The subnet broadcast address has two main roles: to be used as a destination IP address for the purpose of sending packets to all hosts in the subnet, and as a means to find the high end of the range of addresses in a subnet.

The original purpose for the subnet broadcast address was to give hosts a way to send one packet to all hosts in a subnet and to do so efficiently. For example, a host in subnet A could send a packet with a destination address of subnet B's subnet broadcast address. The routers would forward this one packet just like a unicast IP packet sent to a host in subnet B. After the packet arrives at the router connected to subnet B, that last router would then forward the packet to all hosts in subnet B, typically by encapsulating the packet in a data-link layer broadcast frame. As a result, all hosts in host B's subnet would receive a copy of the packet.



The subnet broadcast address also helps you find the range of addresses in a subnet because the subnet broadcast address is the last (highest) number in a subnet's range of addresses. To find the low end of the range, calculate the subnet ID; to find the high end of the range, calculate the subnet broadcast address.

Table 14-3 summarizes the key facts about the subnet broadcast address, along with the possible synonyms, for easier review and study.

Key Topic

**Table 14-3** Summary of Subnet Broadcast Address Key Facts

| | |
|---|---|
| **Definition** | A reserved number in each subnet that, when used as the destination address of a packet, causes the device to forward the packet to all hosts in that subnet |
| **Numeric Value** | Last (highest) number in the subnet |
| **Literal Synonyms** | Directed broadcast address |
| **Broader-Use Synonyms** | Network broadcast |
| **Typically Seen In…** | Calculations of the range of addresses in a subnet |

## Range of Usable Addresses

The engineers implementing an IP internetwork need to know the range of unicast IP addresses in each subnet. Before you can plan which addresses to use as statically assigned IP addresses, which to configure to be leased by the DHCP server, and which to reserve for later use, you need to know the range of usable addresses.

To find the range of usable IP addresses in a subnet, first find the subnet ID and the subnet broadcast address. Then, just add 1 to the fourth octet of the subnet ID to get the first (lowest) usable address, and subtract 1 from the fourth octet of the subnet broadcast address to get the last (highest) usable address in the subnet.

For example, Figure 14-3 showed subnet ID 172.16.128.0, mask /18. The first usable address is simply one more than the subnet ID (in this case, 172.16.128.1). That same figure showed a subnet broadcast address of 172.16.191.255, so the last usable address is one less, or 172.16.191.254.

Now that this section has described the concepts behind the numbers that collectively define a subnet, the rest of this chapter focuses on the math used to find these values.

# Analyzing Existing Subnets: Binary

What does it mean to "analyze a subnet"? For this book, it means that you should be able to start with an IP address and mask and then define key facts about the subnet in which that address resides. Specifically, that means discovering the subnet ID, subnet broadcast address, and range of addresses. The analysis can also include the calculation of the number of addresses in the subnet as discussed in Chapter 13, but this chapter does not review those concepts.

Many methods exist to calculate the details about a subnet based on the address/mask. This section begins by discussing some calculations that use binary math, with the next section showing alternatives that use only decimal math. Although many people prefer the decimal method for going fast on the exams, the binary calculations ultimately give you a better understanding of IPv4 addressing. In particular, if you plan to move on to attain Cisco certifications beyond CCNA, you should take the time to understand the binary methods discussed in this section, even if you use the decimal methods for the exams.

## Finding the Subnet ID: Binary

The two following statements summarize the logic behind the binary value of any subnet ID:

> All numbers in the subnet (subnet ID, subnet broadcast address, and all usable IP addresses) have the same value in the prefix part of the numbers.
>
> The subnet ID is the lowest numeric value in the subnet, so its host part, in binary, is all 0s.

To find the subnet ID in binary, you take the IP address in binary and change all host bits to binary 0. To do so, you need to convert the IP address to binary. You also need to identify the prefix and host bits, which can be easily done by converting the mask (as needed) to prefix format. Figure 14-4 shows the idea, using the same address/mask as in the earlier examples in this chapter: 172.16.150.41, mask /18.

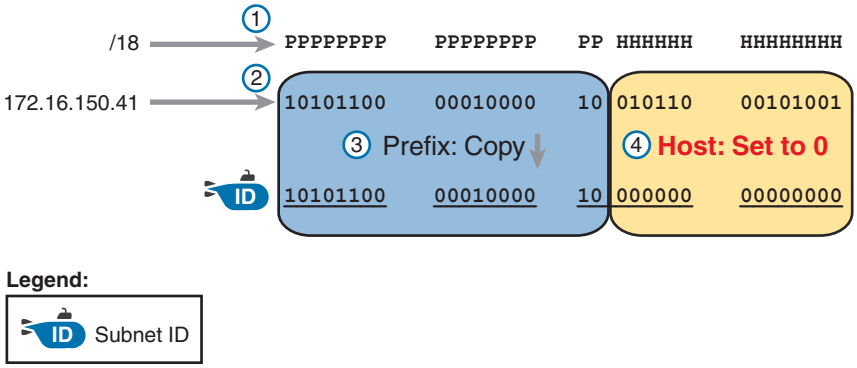



**Figure 14-4** *Binary Concept: Convert the IP Address to the Subnet ID*

Starting at the top of Figure 14-4, the format of the IP address is represented with 18 prefix (P) and 14 host (H) bits in the mask (Step 1). The second row (Step 2) shows the binary version of the IP address, converted from the dotted-decimal notation (DDN) value 172.16.150.41.

The next two steps show the action to copy the IP address's prefix bits (Step 3) and give the host bits a value of binary 0 (Step 4). This resulting number is the subnet ID (in binary).

The last step, not shown in Figure 14-4, is to convert the subnet ID from binary to decimal. This book shows that conversion as a separate step, in Figure 14-5, mainly because many people make a mistake at this step in the process. When converting a 32-bit number (like an IP address or IP subnet ID) back to an IPv4 DDN, you must follow this rule:

> Convert 8 bits at a time from binary to decimal, regardless of the line between the prefix and host parts of the number.



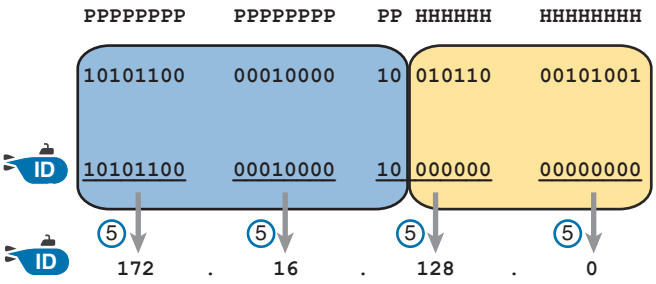



**Figure 14-5** *Converting the Subnet ID from Binary to DDN*

Figure 14-5 shows this final step. Note that the third octet (the third set of 8 bits) has 2 bits in the prefix and 6 bits in the host part of the number, but the conversion occurs for all 8 bits.

**NOTE** You can do the numeric conversions in Figures 14-4 and 14-5 by relying on the conversion table in Appendix A, "Numeric Reference Tables." To convert from DDN to binary, for each octet, find the decimal value in the table and then write down the 8-bit binary equivalent. To convert from binary back to DDN, for each octet of 8 bits, find the matching binary entry in the table and write down the corresponding decimal value. For example, 172 converts to binary 10101100, and 00010000 converts to decimal 16.

## Finding the Subnet Broadcast Address: Binary

Finding the subnet broadcast address uses a similar process. To find the subnet broadcast address, use the same binary process used to find the subnet ID, but instead of setting all the host bits to the lowest value (all binary 0s), set the host part to the highest value (all binary 1s). Figure 14-6 shows the concept.

The process in Figure 14-6 demonstrates the same first three steps shown in Figure 14-4. Specifically, it shows the identification of the prefix and host bits (Step 1), the results of converting the IP address 172.16.150.41 to binary (Step 2), and the copying of the prefix bits (first 18 bits, in this case) at Step 3. The difference occurs in the host bits on the right at Step 4, changing all host bits (the last 14, in this case) to the largest possible value (all binary 1s). Step 5 then converts the 32-bit subnet broadcast address to DDN format. Also, remember that with any conversion from DDN to binary or vice versa, the process always converts using 8 bits at a time. In particular, in this case, the entire third octet of binary 10111111 is converted to decimal 191.

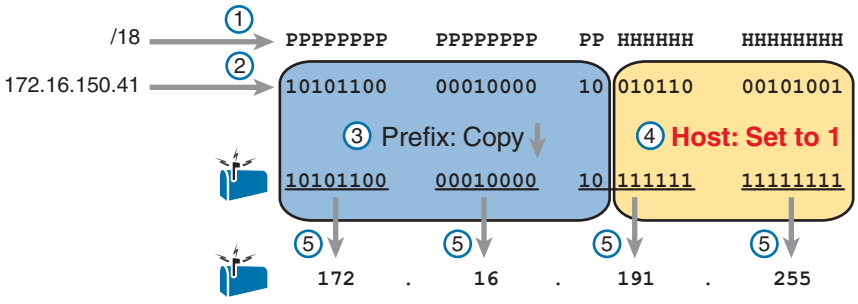



**Figure 14-6** *Finding a Subnet Broadcast Address: Binary*

## Binary Practice Problems

Figures 14-4 and 14-5 demonstrate a process to find the subnet ID using binary math. The following process summarizes those steps in written form for easier reference and practice:

Key Topic

**Step 1.** Convert the mask to prefix format to find the length of the prefix (/P) and the length of the host part (32 – P).

**Step 2.** Convert the IP address to its 32-bit binary equivalent.

**Step 3.** Copy the prefix bits of the IP address.

**Step 4.** Write down 0s for the host bits.

**Step 5.** Convert the resulting 32-bit number, 8 bits at a time, back to decimal.

The process to find the subnet broadcast address is exactly the same, except in Step 4, you set the bits to 1s, as shown in Figure 14-6.

Take a few moments and run through the following five practice problems on scratch paper. In each case, find both the subnet ID and subnet broadcast address. Also, record the prefix style mask:

1. 8.1.4.5, 255.255.0.0
2. 130.4.102.1, 255.255.255.0
3. 199.1.1.100, 255.255.255.0
4. 130.4.102.1, 255.255.252.0
5. 199.1.1.100, 255.255.255.224

Tables 14-4 through 14-8 show the results for the five different examples. The tables show the host bits in bold, and they include the binary version of the address and mask and the binary version of the subnet ID and subnet broadcast address.

**Table 14-4** Subnet Analysis for Subnet with Address 8.1.4.5, Mask 255.255.0.0

| Prefix Length | /16 | 11111111 11111111 **00000000 00000000** |
|---|---|---|
| Address | 8.1.4.5 | 00001000 00000001 **00000100 00000101** |
| Subnet ID | 8.1.0.0 | 00001000 00000001 **00000000 00000000** |
| Broadcast Address | 8.1.255.255 | 00001000 00000001 **11111111 11111111** |

**Table 14-5** Subnet Analysis for Subnet with Address 130.4.102.1, Mask 255.255.255.0

| Prefix Length | /24 | 11111111 11111111 11111111 **00000000** |
|---|---|---|
| Address | 130.4.102.1 | 10000010 00000100 01100110 **00000001** |
| Subnet ID | 130.4.102.0 | 10000010 00000100 01100110 **00000000** |
| Broadcast Address | 130.4.102.255 | 10000010 00000100 01100110 **11111111** |

**Table 14-6** Subnet Analysis for Subnet with Address 199.1.1.100, Mask 255.255.255.0

| Prefix Length | /24 | 11111111 11111111 11111111 **00000000** |
|---|---|---|
| Address | 199.1.1.100 | 11000111 00000001 00000001 **01100100** |
| Subnet ID | 199.1.1.0 | 11000111 00000001 00000001 **00000000** |
| Broadcast Address | 199.1.1.255 | 11000111 00000001 00000001 **11111111** |

**Table 14-7** Subnet Analysis for Subnet with Address 130.4.102.1, Mask 255.255.252.0

| Prefix Length | /22 | 11111111 11111111 111111**00 00000000** |
|---|---|---|
| Address | 130.4.102.1 | 10000010 00000100 011001**10 00000001** |
| Subnet ID | 130.4.100.0 | 10000010 00000100 011001**00 00000000** |
| Broadcast Address | 130.4.103.255 | 10000010 00000100 011001**11 11111111** |

**Table 14-8** Subnet Analysis for Subnet with Address 199.1.1.100, Mask 255.255.255.224

| Prefix Length | /27 | 11111111 11111111 11111111 111**00000** |
|---|---|---|
| Address | 199.1.1.100 | 11000111 00000001 00000001 011**00100** |
| Subnet ID | 199.1.1.96 | 11000111 00000001 00000001 011**00000** |
| Broadcast Address | 199.1.1.127 | 11000111 00000001 00000001 011**11111** |

## Shortcut for the Binary Process

The binary process described in this section so far requires that all four octets be converted to binary and then back to decimal. However, you can easily predict the results in at least three of the four octets, based on the DDN mask. You can then avoid the binary math in all but one octet and reduce the number of binary conversions you need to do.

First, consider an octet, and that octet only, whose DDN mask value is 255. The mask value of 255 converts to binary 11111111, which means that all 8 bits are prefix bits. Thinking through the steps in the process, at Step 2, you convert the address to some number. At Step 3, you copy the number. At Step 4, you convert the same 8-bit number back to decimal. All you did in those three steps, in this one octet, is convert from decimal to binary and convert the same number back to the same decimal value!

In short, for any mask octet of value 255, the subnet ID and subnet broadcast address octet equal the IP address's value in that same octet.

For example, the resident subnet ID for 172.16.150.41, mask 255.255.192.0 is 172.16.128.0. The first two mask octets are 255. Rather than think about the binary math, you could just start by copying the address's value in those two octets: 172.16.

Another shortcut exists for octets whose DDN mask value is decimal 0, or binary 00000000. With a decimal mask value of 0, the math always results in a decimal 0 for the subnet ID, no matter the beginning value in the IP address. Specifically, just look at Steps 4 and 5 in this case: At Step 4, you would write down 8 binary 0s, and at Step 5, you would convert 00000000 back to decimal 0.

The following revised process steps take these two shortcuts into account. However, when the mask is neither 0 nor 255, the process requires the same conversions. At most, you have to do only one octet of the conversions. To find the subnet ID, apply the logic in these steps for each of the four octets:

Key Topic

**Step 1.** If the mask = 255, copy the decimal IP address for that octet.

**Step 2.** If the mask = 0, write down a decimal 0 for that octet.

**Step 3.** If the mask is neither 0 nor 255 in this octet, use the same binary logic as shown in the section "Finding the Subnet ID: Binary," earlier in this chapter.

Figure 14-7 shows an example of this process, again using 172.16.150.41, 255.255.192.0.

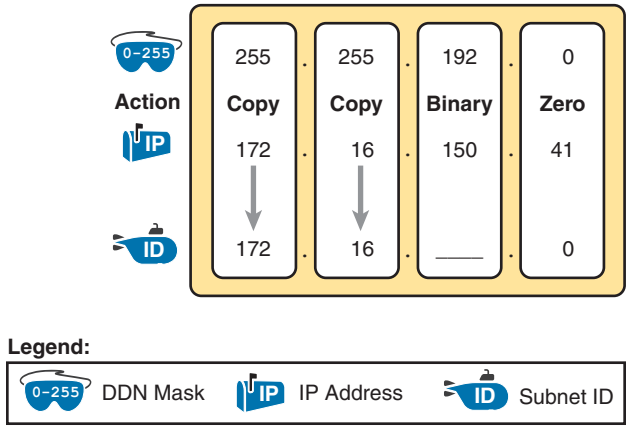



**Figure 14-7** *Binary Shortcut Example*

To find the subnet broadcast address, you can use a decimal shortcut similar to the one used to find the subnet ID: for DDN mask octets equal to decimal 0, set the decimal subnet broadcast address value to 255 instead of 0, as noted in the following list:

Key Topic

**Step 1.** If the mask = 255, copy the decimal IP address for that octet.

**Step 2.** If the mask = 0, write down a decimal 255 for that octet.

**Step 3.** If the mask is neither 0 nor 255 in this octet, use the same binary logic as shown in the section "Finding the Subnet Broadcast Address: Binary," earlier in this chapter.

### A Brief Note About Boolean Math

So far, this chapter has described how humans can use binary math to find the subnet ID and subnet broadcast address. However, computers typically use an entirely different binary process to find the same values, using a branch of mathematics called *Boolean algebra*. Computers already store the IP address and mask in binary form, so they do not have to do any conversions to and from decimal. Then, certain Boolean operations allow the computers to calculate the subnet ID and subnet broadcast address with just a few CPU instructions.

You do not need to know Boolean math to have a good understanding of IP subnetting. However, in case you are interested, computers use the following Boolean logic to find the subnet ID and subnet broadcast address, respectively:

Perform a *Boolean AND* of the IP address and mask. This process converts all host bits to binary 0s.

Invert the mask and then perform a *Boolean OR* of the IP address and inverted subnet mask. This process converts all host bits to binary 1s.



### Finding the Range of Addresses

Finding the range of usable addresses in a subnet, after you know the subnet ID and subnet broadcast address, requires only simple addition and subtraction. To find the first (lowest) usable IP address in the subnet, simply add 1 to the fourth octet of the subnet ID. To find the last (highest) usable IP address, simply subtract 1 from the fourth octet of the subnet broadcast address.

## Analyzing Existing Subnets: Decimal

Analyzing existing subnets using the binary process works well. However, some of the math takes time for most people, particularly the decimal-binary conversions. And you need to do the math quickly for the Cisco CCNA exam. For the exam, you really should be able to take an IP address and mask, and calculate the subnet ID and range of usable addresses within about 15 seconds. When using binary methods, most people require a lot of practice to be able to find these answers, even when using the abbreviated binary process.

This section discusses how to find the subnet ID and subnet broadcast address using only decimal math. Most people can find the answers more quickly using this process, at least after a little practice, as compared with the binary process. However, the decimal process does not tell you anything about the meaning behind the math. So, if you have not read the earlier section "Analyzing Existing Subnets: Binary," it is worthwhile to read it for the sake of understanding subnetting. This section focuses on getting the right answer using a method that, after you have practiced, should be faster.

### Analysis with Easy Masks

With three easy subnet masks in particular, finding the subnet ID and subnet broadcast address requires only easy logic and literally no math. Three easy masks exist:

255.0.0.0

255.255.0.0

255.255.255.0

These easy masks have only 255 and 0 in decimal. In comparison, difficult masks have one octet that has neither a 255 nor a 0 in the mask, which makes the logic more challenging.

**NOTE** The terms *easy mask* and *difficult mask* are created for use in this book to describe the masks and the level of difficulty when working with each.

When the problem uses an easy mask, you can quickly find the subnet ID based on the IP address and mask in DDN format. Just use the following process for each of the four octets to find the subnet ID:

**Step 1.** If the mask octet = 255, copy the decimal IP address.

**Step 2.** If the mask octet = 0, write a decimal 0.

A similar simple process exists to find the subnet broadcast address, as follows:

**Step 1.** If the mask octet = 255, copy the decimal IP address.

**Step 2.** If the mask octet = 0, write a decimal 255.

Before moving to the next section, take some time to fill in the blanks in Table 14-9. Check your answers against Table 14-15 in the section "Answers to Earlier Practice Problems," later in this chapter. Complete the table by listing the subnet ID and subnet broadcast address.

**Table 14-9** Practice Problems: Find Subnet ID and Broadcast, Easy Masks

| | IP Address | Mask | Subnet ID | Broadcast Address |
|---|---|---|---|---|
| 1 | 10.77.55.3 | 255.255.255.0 | | |
| 2 | 172.30.99.4 | 255.255.255.0 | | |
| 3 | 192.168.6.54 | 255.255.255.0 | | |
| 4 | 10.77.3.14 | 255.255.0.0 | | |
| 5 | 172.22.55.77 | 255.255.0.0 | | |
| 6 | 1.99.53.76 | 255.0.0.0 | | |

## Predictability in the Interesting Octet

Although three masks are easier to work with (255.0.0.0, 255.255.0.0, and 255.255.255.0), all other subnet masks make the decimal math a little more difficult, so we call these masks *difficult masks*. With difficult masks, one octet has a value of neither a 0 nor a 255. To bring attention to the one octet with the most difficult value, this book refers to that octet as the *interesting octet*.

If you take some time to think about different problems and focus on the interesting octet, you will begin to see a pattern. This section takes you through that examination so that you can learn how to predict the pattern, in decimal, and find the subnet ID.

First, the subnet ID value has a predictable decimal value because of the assumption that a single subnet mask is used for all subnets of a single classful network. The chapters in this part of the book assume that, for a given classful network, the design engineer chooses to use a single subnet mask for all subnets. (See the section "One Size Subnet Fits All—Or Not" in Chapter 11, "Perspectives on IPv4 Subnetting," for more details.)

To see that predictability, consider some planning information written down by a network engineer, as shown in Figure 14-8. The figure shows four different masks the engineer is considering using in an IPv4 network, along with Class B network 172.16.0.0. The figure shows the third-octet values for the subnet IDs that would be created when using masks 255.255.128.0, 255.255.192.0, 255.255.224.0, and 255.255.240.0, from top to bottom in the figure.

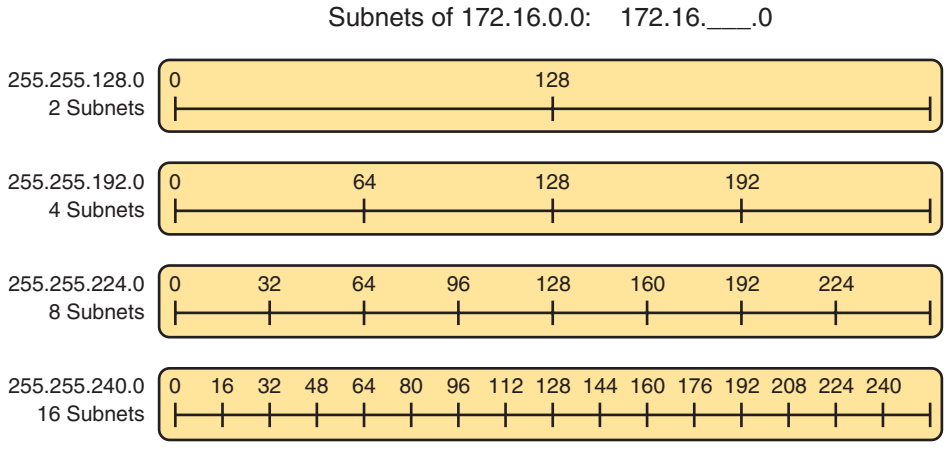



**Figure 14-8** *Numeric Patterns in the Interesting Octet*

First, to explain the figure further, look at the top row of the figure. If the engineer uses 255.255.128.0 as the mask, the mask creates two subnets, with subnet IDs 172.16.0.0 and 172.16.128.0. If the engineer uses mask 255.255.192.0, the mask creates four subnets, with subnet IDs 172.16.0.0, 172.16.64.0, 172.16.128.0, and 172.16.192.0.

If you take the time to look at the figure, the patterns become obvious. In this case:

Mask: 255.255.128.0 Pattern: Multiples of 128
Mask: 255.255.192.0 Pattern: Multiples of 64
Mask: 255.255.224.0 Pattern: Multiples of 32
Mask: 255.255.240.0 Pattern: Multiples of 16

To find the subnet ID, you just need a way to figure out what the pattern is. If you start with an IP address and mask, just find the subnet ID closest to the IP address, without going over, as discussed in the next section.

## Finding the Subnet ID: Difficult Masks

The following written process lists all the steps to find the subnet ID, using only decimal math. This process adds to the earlier process used with easy masks. For each octet:

Key Topic

**Step 1.** If the mask octet = 255, copy the decimal IP address.

**Step 2.** If the mask octet = 0, write a decimal 0.

**Step 3.** If the mask is neither, refer to this octet as the *interesting octet*:

**a.** Calculate the *magic number* as 256 – mask.

**b.** Set the subnet ID's value to the multiple of the magic number that is closest to the IP address without going over.

The process uses two new terms created for this book: *magic number* and *interesting octet*. The term *interesting octet* refers to the octet identified at Step 3 in the process; in other words, it is the octet with the mask that is neither 255 nor 0. Step 3A then uses the term *magic number*, which is derived from the DDN mask. Conceptually, the magic number is the number you add to one subnet ID to get the next subnet ID in order, as shown in Figure 14-8. Numerically, it can be found by subtracting the DDN mask's value, in the interesting octet, from 256, as mentioned in Step 3A.

The best way to learn this process is to see it happen. In fact, if you can, stop reading now, use the companion website for this book, and watch the videos about finding the subnet ID with a difficult mask. These videos demonstrate this process. You can also use the examples on the next few pages that show the process being used on paper. Then follow the practice opportunities outlined in the section "Practice Analyzing Existing Subnets," later in this chapter.

### Resident Subnet Example 1

For example, consider the requirement to find the resident subnet for IP address 130.4.102.1, mask 255.255.240.0. The process does not require you to think about prefix bits versus host bits, convert the mask, think about the mask in binary, or convert the IP address to and from binary. Instead, for each of the four octets, choose an action based on the value in the mask. Figure 14-9 shows the results; the circled numbers in the figure refer to the step numbers in the written process to find the subnet ID, as listed in the previous few pages.

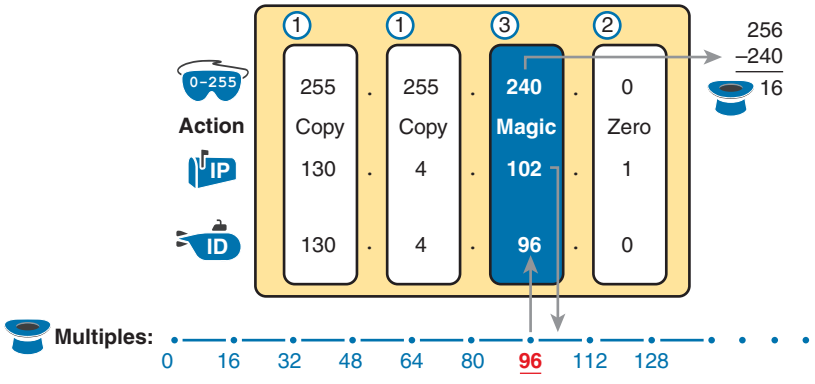



**Figure 14-9** *Find the Subnet ID: 130.4.102.1, 255.255.240.0*

First, examine the three uninteresting octets (1, 2, and 4, in this example). The process keys on the mask, and the first two octets have a mask value of 255, so simply copy the IP address to the place where you intend to write down the subnet ID. The fourth octet has a mask value of 0, so write down a 0 for the fourth octet of the subnet ID.

The most challenging logic occurs in the interesting octet, which is the third octet in this example, because of the mask value 240 in that octet. For this octet, Step 3A asks you to calculate the magic number as 256 – mask. That means you take the mask's value in the interesting octet (240, in this case) and subtract it from 256: 256 – 240 = 16. The subnet ID's value in this octet must be a multiple of decimal 16, in this case.

Step 3B then asks you to find the multiples of the magic number (16, in this case) and choose the one closest to the IP address without going over. Specifically, that means that you should

mentally calculate the multiples of the magic number, starting at 0. (Do not forget to start at 0!) Count, starting at 0: 0, 16, 32, 48, 64, 80, 96, 112, and so on. Then, find the multiple closest to the IP address value in this octet (102, in this case), without going over 102. So, as shown in Figure 14-9, you make the third octet's value 96 to complete the subnet ID of 130.4.96.0.

### Resident Subnet Example 2

Consider another example: 192.168.5.77, mask 255.255.255.224. Figure 14-10 shows the results.

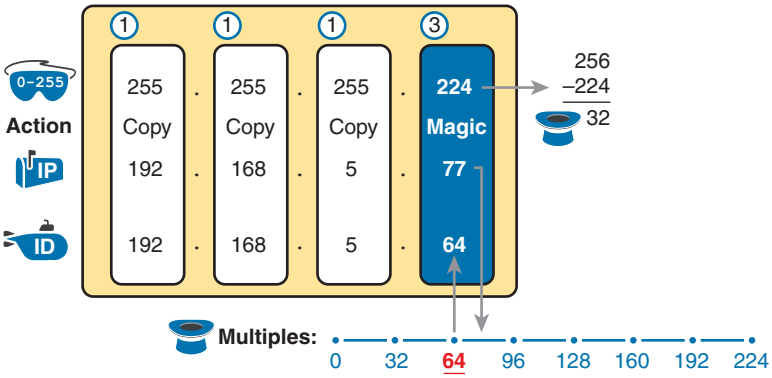



**Figure 14-10** *Resident Subnet for 192.168.5.77, 255.255.255.224*

The three uninteresting octets (1, 2, and 3, in this case) require only a little thought. For each octet, each with a mask value of 255, just copy the IP address.

For the interesting octet, at Step 3A, the magic number is 256 – 224 = 32. The multiples of the magic number are 0, 32, 64, 96, and so on. Because the IP address value in the fourth octet is 77, in this case, the multiple must be the number closest to 77 without going over; therefore, the subnet ID ends with 64, for a value of 192.168.5.64.

### Resident Subnet Practice Problems

Before moving to the next section, take some time to fill in the blanks in Table 14-10. Check your answers against Table 14-16 in the section "Answers to Earlier Practice Problems," later in this chapter. Complete the table by listing the subnet ID in each case. The text following Table 14-16 also lists explanations for each problem.

**Table 14-10** Practice Problems: Find Subnet ID, Difficult Masks

| Problem | IP Address | Mask | Subnet ID |
|---|---|---|---|
| 1 | 10.77.55.3 | 255.248.0.0 | |
| 2 | 172.30.99.4 | 255.255.192.0 | |
| 3 | 192.168.6.54 | 255.255.255.252 | |
| 4 | 10.77.3.14 | 255.255.128.0 | |
| 5 | 172.22.55.77 | 255.255.254.0 | |
| 6 | 1.99.53.76 | 255.255.255.248 | |

## Finding the Subnet Broadcast Address: Difficult Masks

To find a subnet's broadcast address, you can use a similar process. For simplicity, this process begins with the subnet ID, rather than the IP address. If you happen to start with an IP address instead, use the processes in this chapter to first find the subnet ID, and then use the following process to find the subnet broadcast address for that same subnet. For each octet:

Key Topic

**Step 1.** If the mask octet = 255, copy the subnet ID.

**Step 2.** If the mask octet = 0, write 255.

**Step 3.** If the mask is neither, identify this octet as the *interesting octet*:

**a.** Calculate the *magic number* as 256 – mask.

**b.** Take the subnet ID's value, add the magic number, and subtract 1 (ID + magic – 1).

### Subnet Broadcast Example 1

The first example continues the first example from the section "Finding the Subnet ID: Difficult Masks," earlier in this chapter, as demonstrated in Figure 14-9. That example started with the IP address/mask of 130.4.102.1, 255.255.240.0, and showed how to find subnet ID 130.4.96.0. Figure 14-11 now begins with that subnet ID and the same mask.

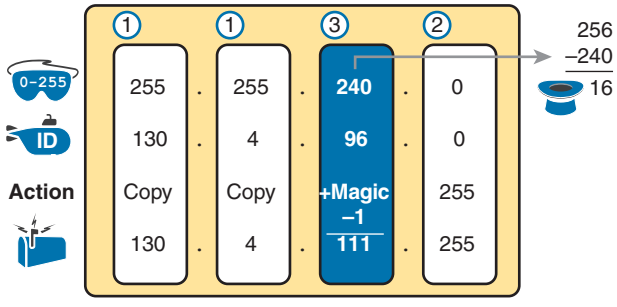



**Figure 14-11** *Find the Subnet Broadcast: 130.4.96.0, 255.255.240.0*

First, examine the three uninteresting octets (1, 2, and 4). The process keys on the mask, and the first two octets have a mask value of 255, so simply copy the subnet ID to the place where you intend to write down the subnet broadcast address. The fourth octet has a mask value of 0, so write down a 255 for the fourth octet.

The logic related to the interesting octet occurs in the third octet in this example because of the mask value 240. First, Step 3A asks you to calculate the magic number, as 256 – mask. (If you had already calculated the subnet ID using the decimal process in this book, you should already know the magic number.) At Step 3B, you take the subnet ID's value (96), add the magic number (16), and subtract 1, for a total of 111. That makes the subnet broadcast address 130.4.111.255.

### Subnet Broadcast Example 2

Again, this example continues an earlier example, from the section "Resident Subnet Example 2," as demonstrated in Figure 14-10. That example started with the IP address/mask of 192.168.5.77, mask 255.255.255.224 and showed how to find subnet ID 192.168.5.64. Figure 14-12 now begins with that subnet ID and the same mask.

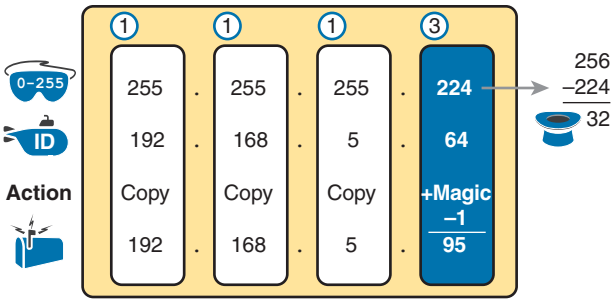



**Figure 14-12** *Find the Subnet Broadcast: 192.168.5.64, 255.255.255.224*



First, examine the three uninteresting octets (1, 2, and 3). The process keys on the mask, and the first three octets have a mask value of 255, so simply copy the subnet ID to the place where you intend to write down the subnet broadcast address.

The interesting logic occurs in the interesting octet, the fourth octet in this example, because of the mask value 224. First, Step 3A asks you to calculate the magic number, as 256 – mask. (If you had already calculated the subnet ID, it is the same magic number because the same mask is used.) At Step 3B, you take the subnet ID's value (64), add magic (32), and subtract 1, for a total of 95. That makes the subnet broadcast address 192.168.5.95.

### Subnet Broadcast Address Practice Problems

Before moving to the next section, take some time to do several practice problems on a scratch piece of paper. Go back to Table 14-10, which lists IP addresses and masks, and practice by finding the subnet broadcast address for all the problems in that table. Then check your answers against Table 14-17 in the section "Answers to Earlier Practice Problems," later in this chapter.

## Practice Analyzing Existing Subnets

As with the other subnetting math in this book, using a two-phase approach may help. Take time now to practice until you feel like you understand the process. Then, before the exam, make sure you master the math. Table 14-11 summarizes the key concepts and suggestions for this two-phase approach.

**Table 14-11** Keep-Reading and Take-Exam Goals for This Chapter's Topics

| Time Frame | Before Moving to the Next Chapter | Before Taking the Exam |
|---|---|---|
| Focus On… | Learning how | Being correct and fast |
| Tools Allowed | All | Your brain and a notepad |
| Goal: Accuracy | 90% correct | 100% correct |
| Goal: Speed | Any speed | 20–30 seconds |

### A Choice: Memorize or Calculate

As described in this chapter, the decimal processes to find the subnet ID and subnet broadcast address do require some calculation, including the calculation of the magic number (256 – mask). The processes also use a DDN mask, so if an exam question gives you a prefix-style mask, you need to convert to DDN format before using the process in this book.

Over the years, some people have told me they prefer to memorize a table to find the magic number. These tables could list the magic number for different DDN masks and prefix masks, so you avoid converting from the prefix mask to DDN. Table 14-12 shows an example of such a table. Feel free to ignore this table, use it, or make your own.

**Table 14-12** Reference Table: DDN Mask Values, Binary Equivalent, Magic Numbers, and Prefixes

| Prefix, interesting octet 2 | /9 | /10 | /11 | /12 | /13 | /14 | /15 | /16 |
|---|---|---|---|---|---|---|---|---|
| Prefix, interesting octet 3 | /17 | /18 | /19 | /20 | /21 | /22 | /23 | /24 |
| Prefix, interesting octet 4 | /25 | /26 | /27 | /28 | /29 | /30 | | |
| Magic number | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| DDN mask in the interesting octet | 128 | 192 | 224 | 240 | 248 | 252 | 254 | 255 |

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 14-13 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 14-13** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |
| Practice mask analysis | | Website, Appendix F |
| Practice analyzing existing subnets | | Website, Appendix F |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 14-14** Key Topics for Chapter 14

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Definition of a subnet's key numbers | 358 |
| Table 14-2 | Key facts about the subnet ID | 360 |
| Table 14-3 | Key facts about the subnet broadcast address | 361 |
| List | Steps to use binary math to find the subnet ID | 364 |
| List | General steps to use binary and decimal math to find the subnet ID | 366 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Steps to use decimal and binary math to find the subnet broadcast address | 366 |
| List | Steps to use only decimal math to find the subnet ID | 369 |
| List | Steps to use only decimal math to find the subnet broadcast address | 372 |



## Key Terms You Should Know

resident subnet, subnet address, subnet broadcast address, subnet ID, subnet number

## Additional Practice for This Chapter's Processes

You can do more practice with the processes in this chapter with a pair of practice sets. Both give you practice at analyzing existing subnets. You may do each practice set using the following tools:

**Application:** From the companion website, in the section titled "Memory Tables and Practice Exercises," use the "Analyzing Existing Subnets" exercises 1 and 2, listed under the Chapter Review for this chapter.

**PDF:** Practice the same problems found in these apps using companion website Appendix F, "Practice for Chapter 14: Analyzing Existing Subnets."

## Answers to Earlier Practice Problems

This chapter includes practice problems spread around different locations in the chapter. The answers are located in Tables 14-15, 14-16, and 14-17.

**Table 14-15** Answers to Problems in Table 14-9

| | IP Address | Mask | Subnet ID | Broadcast Address |
|---|---|---|---|---|
| 1 | 10.77.55.3 | 255.255.255.0 | 10.77.55.0 | 10.77.55.255 |
| 2 | 172.30.99.4 | 255.255.255.0 | 172.30.99.0 | 172.30.99.255 |
| 3 | 192.168.6.54 | 255.255.255.0 | 192.168.6.0 | 192.168.6.255 |
| 4 | 10.77.3.14 | 255.255.0.0 | 10.77.0.0 | 10.77.255.255 |
| 5 | 172.22.55.77 | 255.255.0.0 | 172.22.0.0 | 172.22.255.255 |
| 6 | 1.99.53.76 | 255.0.0.0 | 1.0.0.0 | 1.255.255.255 |

**Table 14-16** Answers to Problems in Table 14-10

| | IP Address | Mask | Subnet ID |
|---|---|---|---|
| 1 | 10.77.55.3 | 255.248.0.0 | 10.72.0.0 |
| 2 | 172.30.99.4 | 255.255.192.0 | 172.30.64.0 |
| 3 | 192.168.6.54 | 255.255.255.252 | 192.168.6.52 |
| 4 | 10.77.3.14 | 255.255.128.0 | 10.77.0.0 |
| 5 | 172.22.55.77 | 255.255.254.0 | 172.22.54.0 |
| 6 | 1.99.53.76 | 255.255.255.248 | 1.99.53.72 |

The following list explains the answers for Table 14-16:

1. The second octet is the interesting octet, with magic number 256 – 248 = 8. The multiples of 8 include 0, 8, 16, 24, …, 64, 72, and 80. Here, 72 is closest to the IP address value in that same octet (77) without going over, making the subnet ID 10.72.0.0.
2. The third octet is the interesting octet, with magic number 256 – 192 = 64. The multiples of 64 include 0, 64, 128, and 192. Here, 64 is closest to the IP address value in that same octet (99) without going over, making the subnet ID 172.30.64.0.
3. The fourth octet is the interesting octet, with magic number 256 – 252 = 4. The multiples of 4 include 0, 4, 8, 12, 16, …, 48, 52, and 56. Here, 52 is the closest to the IP address value in that same octet (54) without going over, making the subnet ID 192.168.6.52.
4. The third octet is the interesting octet, with magic number 256 – 128 = 128. Only two multiples exist that matter: 0 and 128. Here, 0 is the closest to the IP address value in that same octet (3) without going over, making the subnet ID 10.77.0.0.
5. The third octet is the interesting octet, with magic number 256 – 254 = 2. The multiples of 2 include 0, 2, 4, 6, 8, and so on—essentially all even numbers. Here, 54 is closest to the IP address value in that same octet (55) without going over, making the subnet ID 172.22.54.0.
6. The fourth octet is the interesting octet, with magic number 256 – 248 = 8. The multiples of 8 include 0, 8, 16, 24, …, 64, 72, and 80. Here, 72 is closest to the IP address value in that same octet (76) without going over, making the subnet ID 1.99.53.72.

**Table 14-17** Answers to Problems in the Section "Subnet Broadcast Address Practice Problems"

| | Subnet ID | Mask | Broadcast Address |
|---|---|---|---|
| 1 | 10.72.0.0 | 255.248.0.0 | 10.79.255.255 |
| 2 | 172.30.64.0 | 255.255.192.0 | 172.30.127.255 |
| 3 | 192.168.6.52 | 255.255.255.252 | 192.168.6.55 |
| 4 | 10.77.0.0 | 255.255.128.0 | 10.77.127.255 |
| 5 | 172.22.54.0 | 255.255.254.0 | 172.22.55.255 |
| 6 | 1.99.53.72 | 255.255.255.248 | 1.99.53.79 |

The following list explains the answers for Table 14-17:

1. The second octet is the interesting octet. Completing the three easy octets means that the broadcast address in the interesting octet will be 10.___.255.255. With magic number 256 – 248 = 8, the second octet will be 72 (from the subnet ID), plus 8, minus 1, or 79.
2. The third octet is the interesting octet. Completing the three easy octets means that the broadcast address in the interesting octet will be 172.30.___.255. With magic number 256 – 192 = 64, the interesting octet will be 64 (from the subnet ID), plus 64 (the magic number), minus 1, for 127.

3. The fourth octet is the interesting octet. Completing the three easy octets means that the broadcast address in the interesting octet will be 192.168.6.___. With magic number 256 – 252 = 4, the interesting octet will be 52 (the subnet ID value), plus 4 (the magic number), minus 1, or 55.
4. The third octet is the interesting octet. Completing the three easy octets means that the broadcast address will be 10.77.___.255. With magic number 256 – 128 = 128, the interesting octet will be 0 (the subnet ID value), plus 128 (the magic number), minus 1, or 127.
5. The third octet is the interesting octet. Completing the three easy octets means that the broadcast address will be 172.22.___.255. With magic number 256 – 254 = 2, the broadcast address in the interesting octet will be 54 (the subnet ID value), plus 2 (the magic number), minus 1, or 55.
6. The fourth octet is the interesting octet. Completing the three easy octets means that the broadcast address will be 1.99.53.___. With magic number 256 – 248 = 8, the broadcast address in the interesting octet will be 72 (the subnet ID value), plus 8 (the magic number), minus 1, or 79.



CHAPTER 15

# Subnet Design

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.6 Configure and verify IPv4 addressing and subnetting**

So far in this book, most of the discussion about IPv4 used examples with the addresses and masks already given. This book has shown many examples already, but the examples so far do not ask you to pick the IP address or pick the mask. Instead, as discussed back in Chapter 11, "Perspectives on IPv4 Subnetting," this book so far has assumed that someone else designed the IP addressing and subnetting plan, and this book shows how to implement it.

This chapter turns that model around. It goes back to the progression of building and implementing IPv4, as discussed in Chapter 11, as shown in Figure 15-1. This chapter picks up the story right after some network engineer has chosen a Class A, B, or C network to use for the enterprise's IPv4 network. And then this chapter discusses the design choices related to picking one subnet mask to use for all subnets (the first major section) and what subnet IDs that choice creates (the second major section).

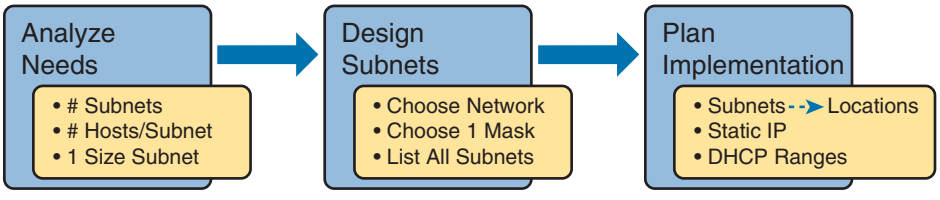



**Figure 15-1** *Subnet Design and Implementation Process from Chapter 11*

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 15-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Choosing the Mask(s) to Meet Requirements | 1–3 |
| Finding All Subnet IDs | 4–6 |

1. An IP subnetting design effort is under way at a company. So far, the senior engineer has decided to use private Class B network 172.23.0.0. The design calls for 100 subnets, with the largest subnet needing 500 hosts. Management requires that the design accommodate 50 percent growth in the number of subnets and the size of the largest subnet. The requirements also state that a single mask must be used throughout the Class B network. How many masks meet the requirements?
   a. 0
   b. 1
   c. 2
   d. 3+
2. An IP subnetting design requires 200 subnets and 120 hosts/subnet for the largest subnets, and requires that a single mask be used throughout the one private IP network that will be used. The design also requires planning for 20 percent growth in the number of subnets and number of hosts/subnet in the largest subnet. Which of the following answers lists a private IP network and mask that, if chosen, would meet the requirements?
   a. 10.0.0.0/25
   b. 10.0.0.0/22
   c. 172.16.0.0/23
   d. 192.168.7.0/24
3. An engineer has planned to use private Class B network 172.19.0.0 and a single subnet mask throughout the network. The answers list the masks considered by the engineer. Choose the mask that, among the answers, supplies the largest number of hosts per subnet, while also supplying enough subnet bits to support 1000 subnets.
   a. 255.255.255.0
   b. /26
   c. 255.255.252.0
   d. /28
4. An engineer has calculated the list of subnet IDs, in consecutive order, for network 172.30.0.0, assuming that the /22 mask is used throughout the network. Which of the following are true? (Choose two answers.)
   a. Any two consecutive subnet IDs differ by a value of 22 in the third octet.
   b. Any two consecutive subnet IDs differ by a value of 16 in the fourth octet.
   c. The list contains 64 subnet IDs.
   d. The last subnet ID is 172.30.252.0.
5. Which of the following are valid subnet IDs for private network 192.168.9.0, using mask /29, assuming that mask /29 is used throughout the network?
   a. 192.168.9.144
   b. 192.168.9.58
   c. 192.168.9.242
   d. 192.168.9.9

6. Which of the following is not a valid subnet ID for private network 172.19.0.0, using mask /24, assuming that mask /24 is used throughout the network?
   a. 172.19.0.0
   b. 172.19.1.0
   c. 172.19.255.0
   d. 172.19.0.16

# Foundation Topics

## Choosing the Mask(s) to Meet Requirements

This first major section examines how to find all the masks that meet the stated requirements for the number of subnets and the number of hosts per subnet. To that end, the text assumes that the designer has already determined these requirements and has chosen the network private number to be subnetted. The designer has also made the choice to use a single subnet mask value throughout the classful network.

Armed with the information in this chapter, you can answer questions such as the following, a question that matters both for real engineering jobs and the Cisco exams:

> You are using Class B network 172.16.0.0. You need 200 subnets and 200 hosts/subnet. Which of the following subnet mask(s) meet the requirements? (This question is then followed by several answers that list different subnet masks.)

To begin, this section reviews the concepts in Chapter 11's section "Choose the Mask." That section introduced the main concepts about how an engineer, when designing subnet conventions, must choose the mask based on the requirements.

After reviewing the related concepts from Chapter 11, this section examines this topic in more depth. In particular, this chapter looks at three general cases:

- No masks meet the requirements.
- One and only one mask meets the requirements.
- Multiple masks meet the requirements.

For this last case, the text discusses how to determine all masks that meet the requirements and the tradeoffs related to choosing which one mask to use.

### Review: Choosing the Minimum Number of Subnet and Host Bits

The network designer must examine the requirements for the number of subnets and number of hosts/subnet, and then choose a mask. As discussed in detail in Chapter 13, "Analyzing Subnet Masks," a classful view of IP addresses defines the three-part structure of an IP address: network, subnet, and host. The network designer must choose the mask so that the number of subnet and host bits (S and H, respectively, in Figure 15-2) meet the requirements.

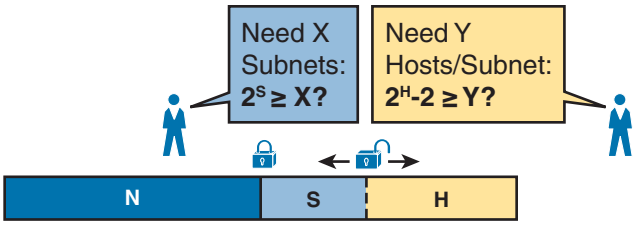



**Figure 15-2** *Choosing the Number of Subnet and Host Bits*

Basically, the designer must choose S subnet bits so that the number of subnets that can be uniquely numbered with S bits ($2^S$) is at least as large as the required number of subnets. The designer applies similar logic to the number of host bits H, while noting that the formula is $2^H - 2$, because of the two reserved numbers in each subnet. So, keeping the powers of 2 handy, as shown in Table 15-2, will be useful when working through these problems.



**Table 15-2** Powers of 2 Reference for Designing Masks

| Number of Bits | $2^X$ | Number of Bits | $2^X$ | Number of Bits | $2^X$ | Number of Bits | $2^X$ |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 5 | 32 | 9 | 512 | 13 | 8192 |
| 2 | 4 | 6 | 64 | 10 | 1024 | 14 | 16,384 |
| 3 | 8 | 7 | 128 | 11 | 2048 | 15 | 32,768 |
| 4 | 16 | 8 | 256 | 12 | 4096 | 16 | 65,536 |

More formally, the process must determine the minimum values for both S and H that meet the requirements. The following list summarizes the initial steps to choose the mask:

**Step 1.** Determine the number of network bits (N) based on the class.

**Step 2.** Determine the smallest value of S, so that $2^S$ => X, where X represents the required number of subnets.

**Step 3.** Determine the smallest value of H, so that $2^H - 2$ => Y, where Y represents the required number of hosts/subnet.

The next three sections examine how to use these initial steps to choose a subnet mask.

## No Masks Meet Requirements

After you determine the required number of subnet and host bits, those bits might not fit into a 32-bit IPv4 subnet mask. Remember, the mask always has a total of 32 bits, with binary 1s in the network and subnet parts and binary 0s in the host part. For the exam, a question might provide a set of requirements that simply cannot be met with 32 total bits.

For example, consider the following sample exam question:

> A network engineer is planning a subnet design. The engineer plans to use Class B network 172.16.0.0. The network has a need for 300 subnets and 280 hosts per subnet. Which of the following masks could the engineer choose?

The three-step process shown in the previous section shows that these requirements mean that a total of 34 bits will be needed, so no mask meets the requirements. First, as a Class B

network, 16 network bits exist, with 16 host bits from which to create the subnet part and to leave enough host bits to number the hosts in each subnet. For the number of subnet bits, S=8 does not work, because $2^8 = 256 < 300$. However, S=9 works, because $2^9 = 512 => 300$. Similarly, because $2^8 - 2 = 254$, which is less than 300, 8 host bits are not enough but 9 host bits ($2^9 - 2 = 510$) are just enough.

These requirements do not leave enough space to number all the hosts and subnet, because the network, subnet, and host parts add up to more than 32:

N=16, because as a Class B network, 16 network bits exist.

The minimum S=9, because S=8 provides too few subnets ($2^8 = 256 < 300$) but S=9 provides $2^9 = 512$ subnets.

The minimum H=9, because H=8 provides too few hosts ($2^8 - 2 = 254 < 280$) but H=9 provides $2^9 - 2 = 510$ hosts/subnet.

Figure 15-3 shows the resulting format for the IP addresses in this subnet, after the engineer has allocated 9 subnet bits on paper. Only 7 host bits remain, but the engineer needs 9 host bits.

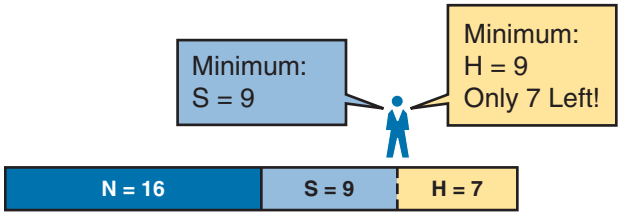



**Figure 15-3** *Too Few Bits for the Host Part, Given the Requirements*

## One Mask Meets Requirements

The process discussed in this chapter in part focuses on finding the smallest number of subnet bits and the smallest number of host bits to meet the requirements. If the engineer tries to use these minimum values, and the combined network, subnet, and host parts add up to exactly 32 bits, exactly one mask meets the requirements.

For example, consider a revised version of the example in the previous section, with smaller numbers of subnets and hosts, as follows:

A network engineer is planning a subnet design. The engineer plans to use Class B network 172.16.0.0. The network has a need for 200 subnets and 180 hosts per subnet. Which of the following masks could the engineer choose?

The three-step process to determine the numbers of network, minimum subnet, and minimum host bits results in a need for 16, 8, and 8 bits, respectively. As before, with a Class B network, 16 network bits exist. With a need for only 200 subnets, S=8 does work, because $2^8 = 256 => 200$; 7 subnet bits would not supply enough subnets ($2^7 = 128$). Similarly, because $2^8 - 2 = 254 => 180$, 8 host bits meet the requirements; 7 host bits (for 126 total hosts/subnet) would not be enough.

Figure 15-4 shows the resulting format for the IP addresses in this subnet.

---

Answers to the "Do I Know This Already?" quiz:

**1** A **2** B **3** B **4** C, D **5** A **6** D

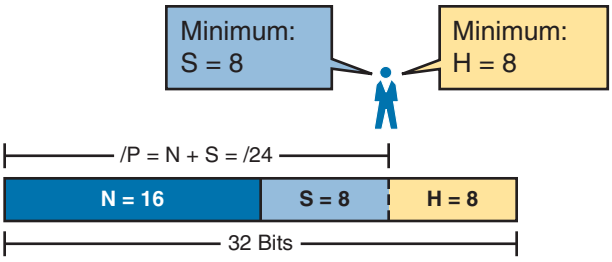



**Figure 15-4** *One Mask That Meets the Requirements*

Figure 15-4 shows the mask conceptually. To find the actual mask value, simply record the mask in prefix format (/P), where P = N + S or, in this case, /24.

## Multiple Masks Meet Requirements



Depending on the requirements and choice of network, several masks might meet the requirements for the numbers of subnets and hosts/subnet. In these cases, you need to find all the masks that could be used. Then, you have a choice, but what should you consider when choosing one mask among all those that meet your requirements? This section shows how to find all the masks, as well as the facts to consider when choosing one mask from the list.

### Finding All the Masks: Concepts

To help you better understand how to find all the subnet masks in binary, this section uses two major steps. In the first major step, you build the 32-bit binary subnet mask on paper. You write down binary 1s for the network bits, binary 1s for the subnet bits, and binary 0s for the host bits, just as always. However, you will use the minimum values for S and H. And when you write down these bits, you will not have 32 bits yet!

For example, consider the following problem, similar to the earlier examples in this chapter but with some changes in the requirements:

> A network engineer is planning a subnet design. The engineer plans to use Class B network 172.16.0.0. The network has a need for 50 subnets and 180 hosts per subnet. Which of the following masks could the engineer choose?

This example is similar to an earlier example, except that only 50 subnets are needed in this case. Again, the engineer is using private IP network 172.16.0.0, meaning 16 network bits. The design requires only 6 subnet bits in this case, because $2^6 = 64 => 50$, and with only 5 subnet bits, $2^5 = 32 < 50$. The design then requires a minimum of 8 host bits.

One way to discuss the concepts and find all the masks that meet these requirements is to write down the bits in the subnet mask: binary 1s for the network and subnet parts and binary 0s for the host part. However, think of the 32-bit mask as 32-bit positions, and when writing the binary 0s, *write them on the far right*. Figure 15-5 shows the general idea.

Figure 15-5 shows 30 bits of the mask, but the mask must have 32 bits. The 2 remaining bits might become subnet bits, being set to binary 1. Alternatively, these 2 bits could be made host bits, being set to binary 0. The engineer simply needs to choose based on whether he would like more subnet bits, to number more subnets, or more host bits, to number more hosts/subnet.

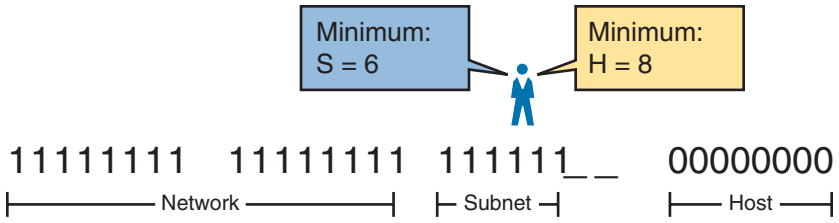



**Figure 15-5** *Incomplete Mask with N=16, S=6, and H=8*

Regardless of the requirements, when choosing any IPv4 subnet mask, you must always follow this rule:

Key Topic

A subnet mask begins with all binary 1s, followed by all binary 0s, with no interleaving of 1s and 0s.

With the example shown in Figure 15-5, with 2 open bits, one value (binary 01) breaks this rule. However, the other three combinations of 2 bits (00, 10, and 11) do not break the rule. As a result, three masks meet the requirements in this example, as shown in Figure 15-6.

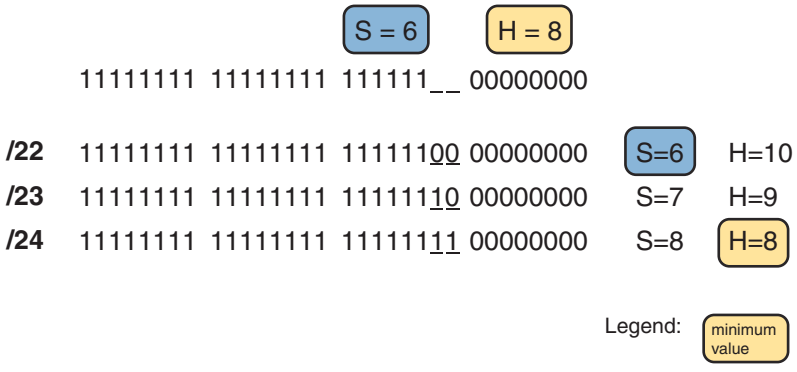



**Figure 15-6** *Three Masks That Meet the Requirements*

In the three masks, the first has the least number of subnet bits among the three masks, but therefore has the most number of host bits. So, the first mask maximizes the number of hosts/subnet. The last mask uses the minimum value for the number of host bits, therefore using the most number of subnet bits allowed while still meeting the requirements. As a result, the last mask maximizes the number of subnets allowed.

### Finding All the Masks: Math

Although the concepts related to the example shown in Figures 15-5 and 15-6 are important, you can find the range of masks that meets the requirements more easily just using some simple math. The process to find the masks requires only a few steps, after you know N and the minimum values of S and H. The process finds the value of /P when using the least number of subnet bits, and when using the least number of host bits, as follows:

Key Topic

**Step 1.** Calculate the shortest prefix mask (/P) based on the *minimum value of S*, where P = N + S.

**Step 2.** Calculate the longest prefix mask (/P) based on the *minimum value of H*, where P = 32 – H.

**Step 3.** The range of valid masks includes all /P values between the two values calculated in the previous steps.

For example, in the example shown in Figure 15-6, N=16, the minimum S=6, and the minimum H=8. The first step identifies the shortest prefix mask (the /P with the smallest value of P) of /22 by adding N and S (16 + 6). The second step identifies the longest prefix mask that meets the requirements by subtracting the smallest possible value for H (8, in this case) from 32, for a mask of /24. The third step reminds us that the range is from /22 to /24, meaning that /23 is also an option.

### Choosing the Best Mask

When multiple possible masks meet the stated requirements, the engineer has a choice of masks. That, of course, begs some questions: Which mask should you choose? Why would one mask be better than the other? The reasons can be summarized into three main options:



Key Topic

**To maximize the number of hosts/subnet:** To make this choice, use the shortest prefix mask (that is, the mask with the smallest /P value), because this mask has the largest host part.

**To maximize the number of subnets:** To make this choice, use the longest prefix mask (that is, the mask with the largest /P value), because this mask has the largest subnet part.

**To increase both the numbers of supported subnets and hosts:** To make this choice, choose a mask in the middle of the range, which gives you both more subnet bits and more host bits.

For example, in Figure 15-6, the range of masks that meet the requirements is /22 – /24. The shortest mask, /22, has the least subnet bits but the largest number of host bits (10) of the three answers, maximizing the number of hosts/subnet. The longest mask, /24, maximizes the number of subnet bits (8), maximizing the number of subnets, at least among the options that meet the original requirements. The mask in the middle, /23, provides some growth in both subnets and hosts/subnet.

## The Formal Process

Although this chapter has explained various steps in finding a subnet mask to meet the design requirements, it has not yet collected these concepts into a list for the entire process. The following list collects all these steps into one place for reference. Note that this list does not introduce any new concepts compared to the rest of this chapter; it just puts all the ideas in one place.

Key Topic

**Step 1.** Find the number of network bits (N) per class rules.

**Step 2.** Calculate the minimum number of subnet bits (S) so that $2^S$ => the number of required subnets.

**Step 3.** Calculate the minimum number of host bits (H) so that $2^H - 2$ => the number of required hosts/subnet.

**Step 4.** If N + S + H > 32, no mask meets the need.

**Step 5.** If N + S + H = 32, one mask meets the need. Calculate the mask as /P, where P = N + S.

**Step 6.** If N + S + H < 32, multiple masks meet the need:

- **a.** Calculate mask /P based on the minimum value of S, where P = N + S. This mask maximizes the number of hosts/subnet.
- **b.** Calculate mask /P based on the minimum value of H, where P = 32 – H. This mask maximizes the number of possible subnets.
- **c.** Note that the complete range of masks includes all prefix lengths between the two values calculated in Steps 6A and 6B.

### Practice Choosing Subnet Masks

Take the usual two-phase approach to learning new subnetting math and processes. Take the time now to practice to make sure you understand the fundamentals, using the book and notes as needed. Then, sometime before taking the exam, practice until you can reach the goals in the right column of Table 15-3.

**Table 15-3** Keep-Reading and Take-Exam Goals for Choosing a Subnet Mask

| Time Frame | Before Moving to the Next Chapter | Before Taking the Exam |
|---|---|---|
| Focus On… | Learning how | Being correct and fast |
| Tools Allowed | All | Your brain and a notepad |
| Goal: Accuracy | 90% correct | 100% correct |
| Goal: Speed | Any speed | 15 seconds |

#### Practice Problems for Choosing a Subnet Mask

The following list shows three separate problems, each with a classful network number and a required number of subnets and hosts/subnet. For each problem, determine the minimum number of subnet and host bits that meet the requirements. If more than one mask exists, note which mask maximizes the number of hosts/subnet and which maximizes the number of subnets. If only one mask meets the requirements, simply list that mask. List the masks in prefix format:

1. Network 10.0.0.0, need 1500 subnets, need 300 hosts/subnet
2. Network 172.25.0.0, need 130 subnets, need 127 hosts/subnet
3. Network 192.168.83.0, need 8 subnets, need 8 hosts/subnet

Table 15-8, found in the later section “Answers to Earlier Practice Problems,” lists the answers.

## Finding All Subnet IDs

After the person designing the IP subnetting plan has chosen the one mask to use throughout the Class A, B, or C network, that person will soon need to start assigning specific subnet IDs for use in specific VLANs, WAN links, and other places in the internetwork that need a subnet. But what are those subnet IDs? As it turns out, after the network ID and one subnet mask for all subnets have been chosen, finding all the subnet IDs just requires doing a little math. This second major section of this chapter focuses on that math, which focuses on a single question:

> Given a single Class A, B, or C network, and the single subnet mask to use for all subnets, what are all the subnet IDs?

When learning how to answer this question, you can think about the problem in either binary or decimal. This chapter approaches the problem using decimal. Although the process itself requires only simple math, the process requires practice before most people can confidently answer this question.

The decimal process begins by identifying the first, or numerically lowest, subnet ID. After that, the process identifies a pattern in all subnet IDs for a given subnet mask so that you can find each successive subnet ID through simple addition. This section examines the key ideas behind this process first; then you are given a formal definition of the process.

**NOTE** Some videos included on the companion website describe the same fundamental processes to find all subnet IDs. You can view those videos before or after reading this section, or even instead of reading this section, as long as you learn how to find all subnet IDs. The process step numbering in the videos might not match the steps shown in this edition of the book.



## First Subnet ID: The Zero Subnet

The first step in finding all subnet IDs of one network is incredibly simple: Copy the network ID. That is, take the Class A, B, or C network ID—in other words, the classful network ID—and write it down as the first subnet ID. No matter what Class A, B, or C network you use, and no matter what subnet mask you use, the first (numerically lowest) subnet ID is equal to the network ID.

For example, if you begin with classful network 172.20.0.0, no matter what the mask is, the first subnet ID is 172.20.0.0.

This first subnet ID in each network goes by two special names: either **subnet zero** or **zero subnet**. The origin of these names is related to the fact that a network's zero subnet, when viewed in binary, has a subnet part of all binary 0s. In decimal, the zero subnet can be easily identified, because the zero subnet always has the exact same numeric value as the network ID itself.

In the past, engineers avoided using zero subnets because of the ambiguity with one number that could represent the entire classful network, or it could represent one subnet inside the classful network. To help control that, IOS has a global command that can be set one of two ways:

**ip subnet-zero**, which allows the configuration of addresses in the zero subnet.

**no ip subnet-zero**, which prevents the configuration of addresses in the zero subnet.

Although most sites use the default setting to allow zero subnets, you can use the **no ip subnet-zero** command to prevent configuring addresses that are part of a zero subnet. Example 15-1 shows how a router rejects an **ip address** command after changing to use **no ip subnet-zero**. Note that the error message does not mention the zero subnet, instead simply stating "bad mask."

**Example 15-1** *Effects of* **[no] ip subnet-zero** *on a Local Router*

```
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# no ip subnet-zero
R1(config)# interface g0/1
R1(config-if)# ip address 10.0.0.1 255.255.255.0
Bad mask /24 for address 10.0.0.1
```

Note that the **no ip subnet-zero** command affects the local router's **ip address** commands, as well as the local router's **ip route** commands (which define static routes). However, it does not affect the local router's routes as learned with a routing protocol.

## Finding the Pattern Using the Magic Number

Subnet IDs follow a predictable pattern, at least when using our assumption of a single subnet mask for all subnets of a network. The pattern uses the *magic number*, as discussed in Chapter 14, "Analyzing Existing Subnets." To review, the magic number is 256, minus the mask's decimal value, in a particular octet that this book refers to as the *interesting octet*.

Figure 15-7 shows four examples of these patterns with four different masks. For example, just look at the top of the figure to start. It lists mask 255.255.128.0 on the left. The third octet is the interesting octet, with a mask value other than 0 or 255 in that octet. The left side shows a magic number calculated as 256 – 128 = 128. So, the pattern of subnet IDs is shown in the highlighted number line; that is, the subnet IDs when using this mask will have either a 0 or 128 in the third octet. For example, if using network 172.16.0.0, the subnet IDs would be 172.16.0.0 and 172.16.128.0.

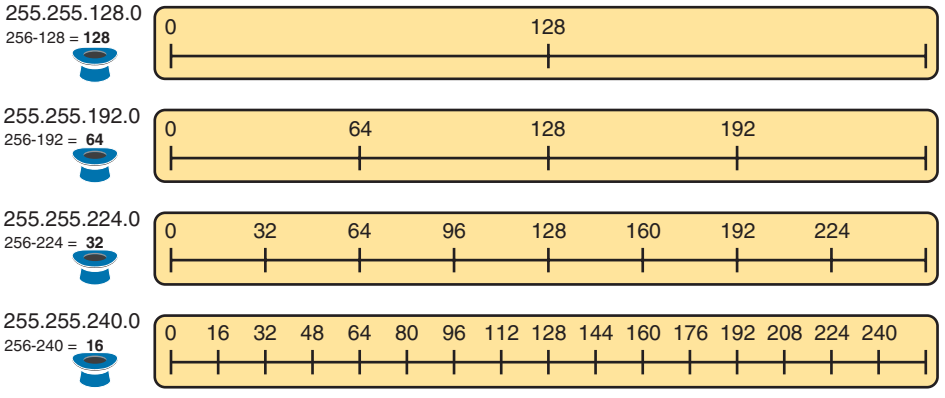



**Figure 15-7** *Patterns with Magic Numbers for Masks /17 – /20*

Now focus on the second row, with another example, with mask 255.255.192.0. This row shows a magic number of 64 (256 – 192 = 64), so the subnet IDs will use a value of 0, 64, 128, or 192 (multiples of 64) in the third octet. For example, if used with network 172.16.0.0, the subnet IDs would be 172.16.0.0, 172.16.64.0, 172.16.128.0, and 172.16.192.0.

Looking at the third row/example, the mask is 255.255.224.0, with a magic number of 256 – 224 = 32. So, as shown in the center of the figure, the subnet ID values will be multiples of 32. For example, if used with network 172.16.0.0 again, this mask would tell us that the subnet IDs are 172.16.0.0, 172.16.32.0, 172.16.64.0, 172.16.96.0, and so on.

Finally, for the bottom example, mask 255.255.240.0 makes the magic number, in the third octet, be 16. So, all the subnet IDs will be a multiple of 16 in the third octet, with those values shown in the middle of the figure.

## A Formal Process with Fewer Than 8 Subnet Bits

Although it can be easy to see the patterns in Figure 15-7, it might not be as obvious exactly how to apply those concepts to find all the subnet IDs in every case. This section outlines a specific process to find all the subnet IDs.

To simplify the explanations, this section assumes that fewer than 8 subnet bits exist. Later, the section "Finding All Subnets with More Than 8 Subnet Bits," describes the full process that can be used in all cases.



First, to organize your thoughts, you might want to organize the data into a chart like Table 15-4. This book refers to this chart as the list-all-subnets chart.

**Table 15-4** Generic List-All-Subnets Chart

| Octet | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Mask | | | | |
| Magic Number | | | | |
| Network Number/Zero Subnet | | | | |
| Next Subnet | | | | |
| Next Subnet | | | | |
| Next Subnet | | | | |
| Broadcast Subnet | | | | |
| Out of Range—Used by Process | | | | |

A formal process to find all subnet IDs, given a network and a single subnet mask, is as follows:

Key Topic

**Step 1.** Write down the subnet mask, in decimal, in the first empty row of the table.

**Step 2.** Identify the interesting octet, which is the one octet of the mask with a value other than 255 or 0. Draw a rectangle around the column of the interesting octet.

**Step 3.** Calculate and write down the magic number by subtracting the *subnet mask's interesting octet* from 256.

**Step 4.** Write down the classful network number, which is the same number as the zero subnet, in the next empty row of the list-all-subnets chart.

**Step 5.** To find each successive subnet number:

- **a.** For the three uninteresting octets, copy the previous subnet number's values.
- **b.** For the interesting octet, add the magic number to the previous subnet number's interesting octet.

**Step 6.** When the sum calculated in Step 5B reaches 256, stop the process. The number with the 256 in it is out of range, and the previous subnet number is the broadcast subnet.

Although the written process is long, with practice, most people can find the answers much more quickly with this decimal-based process than by using binary math. As usual, most people learn this process best by seeing it in action, exercising it, and then practicing it. To that end, review the two following examples, and watch the related videos on the companion website for additional examples.

### Example 1: Network 172.16.0.0, Mask 255.255.240.0

To begin this example, focus on the first four of the six steps, when subnetting network 172.16.0.0 using mask 255.255.240.0. Figure 15-8 shows the results of these first four steps:

**Step 1.** Record mask 255.255.240.0, which was given as part of the problem statement. (Figure 15-8 also shows the network ID, 172.16.0.0, for easy reference.)

**Step 2.** The mask's third octet is neither 0 nor 255, which makes the third octet interesting.

**Step 3.** Because the mask's value in the third octet is 240, the magic number = 256 – 240 = 16.

**Step 4.** Because the network ID is 172.16.0.0, the first subnet ID, the zero subnet, is also 172.16.0.0.

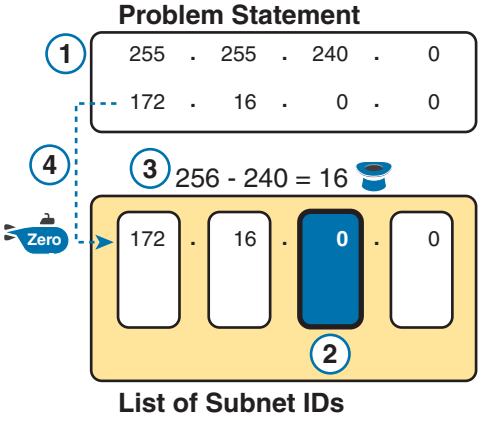



**Figure 15-8** *Results of First Four Steps: 172.16.0.0, 255.255.240.0*

These first four steps discover the first subnet (the zero subnet) and get you ready to do the remaining steps by identifying the interesting octet and the magic number. Step 5 in the process tells you to copy the three boring octets and add the magic number (16, in this case) in the interesting octet (octet 3, in this case). Keep repeating this step until the interesting octet value equals 256 (per Step 6). When the total is 256, you have listed all the subnet IDs, and the line with 256 on it is not a correct subnet ID. Figure 15-9 shows the results of the Step 5 actions.

Key Topic

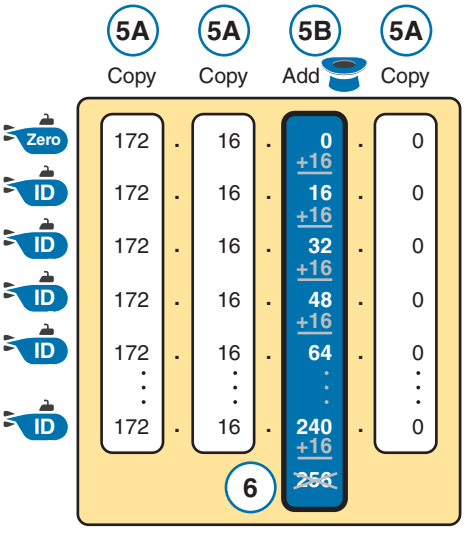



**Figure 15-9** *List of Subnet IDs: 172.16.0.0, 255.255.240.0*

**NOTE** In any list of all the subnet IDs of a network, the numerically highest subnet ID is called the **broadcast subnet**. Decades ago, engineers avoided using the broadcast subnet. However, using the broadcast subnet causes no problems. The term *broadcast subnet* has its origins in the fact that if you determine the subnet broadcast address inside the broadcast subnet, it has the same numeric value as the network-wide broadcast address.

**NOTE** People sometimes confuse the terms *broadcast subnet* and *subnet broadcast address*. The *broadcast subnet* is one subnet, namely the numerically highest subnet; only one such subnet exists per network. The term *subnet broadcast address* refers to the one number in each and every subnet that is the numerically highest number in that subnet.

### Example 2: Network 192.168.1.0, Mask 255.255.255.224

With a Class C network and a mask of 255.255.255.224, this example makes the fourth octet the interesting octet. However, the process works the same, with the same logic, just with the interesting logic applied in a different octet. As with the previous example, the following list outlines the first four steps, with Figure 15-10 showing the results of the first four steps:

**Step 1.** Record mask 255.255.255.224, which was given as part of the problem statement, and optionally record the network number (192.168.1.0).

**Step 2.** The mask's fourth octet is neither 0 nor 255, which makes the fourth octet interesting.

**Step 3.** Because the mask's value in the fourth octet is 224, the magic number = 256 – 224 = 32.

**Step 4.** Because the network ID is 192.168.1.0, the first subnet ID, the zero subnet, is also 192.168.1.0.

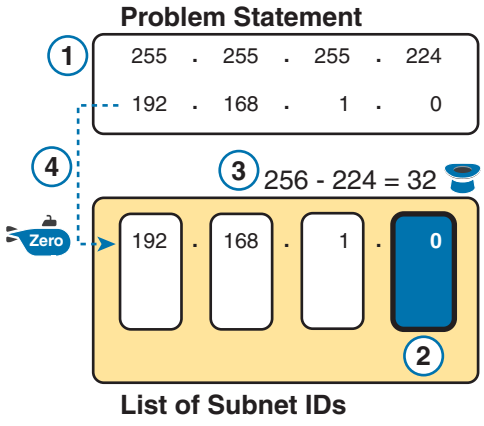



**Figure 15-10** *Results of First Four Steps: 192.168.1.0, 255.255.255.224*

From this point, Step 5 in the process tells you to copy the values in the first three octets and then add the magic number (32, in this case) in the interesting octet (octet 4, in this case). Keep doing so until the interesting octet value equals 256 (per Step 6). When the total is 256, you have listed all the subnet IDs, and the line with 256 on it is not a correct subnet ID. Figure 15-11 shows the results of these steps.

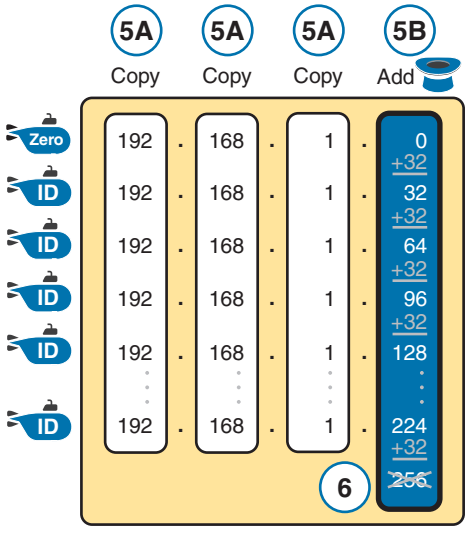



**Figure 15-11** *List of Subnet IDs: 192.168.1.0, 255.255.255.224*

## Finding All Subnets with Exactly 8 Subnet Bits

The formal process in the earlier section "A Formal Process with Fewer Than 8 Subnet Bits" identified the interesting octet as the octet whose mask value is neither a 255 nor a 0. If the mask defines exactly 8 subnet bits, you must use a different logic to identify the interesting octet; otherwise, the same process can be used. In fact, the actual subnet IDs can be a little more intuitive.

Only two cases exist with exactly 8 subnet bits:

A Class A network with mask 255.255.0.0; the entire second octet contains subnet bits.

A Class B network with mask 255.255.255.0; the entire third octet contains subnet bits.

In each case, use the same process as with less than 8 subnet bits, but identify the interesting octet as the one octet that contains subnet bits. Also, because the mask's value is 255, the magic number will be 256 – 255 = 1, so the subnet IDs are each 1 larger than the previous subnet ID.



For example, for 172.16.0.0, mask 255.255.255.0, the third octet is the interesting octet and the magic number is 256 – 255 = 1. You start with the zero subnet, equal in value to network number 172.16.0.0, and then add 1 in the third octet. For example, the first four subnets are as follows:

172.16.0.0 (zero subnet)

172.16.1.0

172.16.2.0

172.16.3.0

## Finding All Subnets with More Than 8 Subnet Bits

Earlier, the section "A Formal Process with Fewer Than 8 Subnet Bits" assumed fewer than 8 subnet bits for the purpose of simplifying the discussions while you learn. In real life, you need to be able to find all subnet IDs with any valid mask, so you cannot assume fewer than 8 subnet bits.

The examples that have at least 9 subnet bits have a minimum of 512 subnet IDs, so writing down such a list would take a lot of time. To conserve space, the examples will use shorthand rather than list hundreds or thousands of subnet IDs.

The process with fewer than 8 subnet bits told you to count in increments of the magic number in one octet. With more than 8 subnet bits, the new expanded process must tell you how to count in multiple octets. So, this section breaks down two general cases: (a) when 9–16 subnet bits exist, which means that the subnet field exists in only two octets, and (b) cases with 17 or more subnet bits, which means that the subnet field exists in three octets.

### Process with 9–16 Subnet Bits

To understand the process, you need to know a few terms that the process will use. Figure 15-12 shows the details, with an example that uses Class B network 130.4.0.0 and mask 255.255.255.192. The lower part of the figure details the structure of the addresses per the mask: a network part of two octets because it is a Class B address, a 10-bit subnet part per the mask (/26), and 6 host bits.

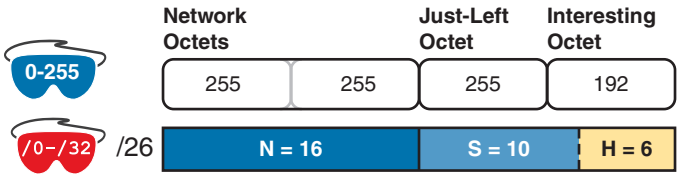



**Figure 15-12** *Fundamental Concepts and Terms for the >8 Subnet Bit Process*

In this case, subnet bits exist in two octets: octets 3 and 4. For the purposes of the process, the rightmost of these octets is the interesting octet, and the octet just to the left is the cleverly named *just-left* octet.

The updated process, which makes adjustments for cases in which the subnet field is longer than 1 octet, tells you to count in increments of the magic number in the interesting octet, but count by 1s in the just-left octet. Formally:

Key Topic

**Step 1.** Calculate subnet IDs using the 8-subnet-bits-or-less process. However, when the total adds up to 256, move to the next step; consider the subnet IDs listed so far as a *subnet block*.

**Step 2.** Copy the previous subnet block, but add 1 to the just-left octet in all subnet IDs in the new block.

**Step 3.** Repeat Step 2 until you create the block with a just-left octet of 255, but go no further.

To be honest, the formal concept can cause you problems until you work through some examples, so even if the process remains a bit unclear in your mind, you should work through the following examples instead of rereading the formal process.

First, consider an example based on Figure 15-12, with network 130.4.0.0 and mask 255.255.255.192. Figure 15-12 already showed the structure, and Figure 15-13 shows the subnet ID block created at Step 1.

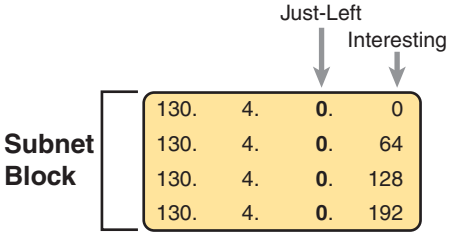



**Figure 15-13** *Step 1: Listing the First Subnet ID Block*

The logic at Step 1, to create this subnet ID block of four subnet IDs, follows the same magic number process seen before. The first subnet ID, 130.4.0.0, is the zero subnet. The next three subnet IDs are each 64 bigger, because the magic number, in this case, is 256 – 192 = 64.

Steps 2 and 3 from the formal process tell you how to create 256 subnet blocks, and by doing so, you will list all 1024 subnet IDs. To do so, create 256 total subnet blocks: one with a 0 in the just-left octet, one with a 1 in the just-left octet, and another with a 2 in the just-left octet, up through 255. The process continues through the step at which you create the subnet block with 255 in the just-left octet (third octet, in this case). Figure 15-14 shows the idea, with the addition of the first few subnet blocks.

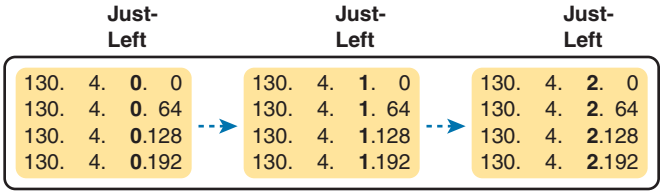



**Figure 15-14** *Step 2: Replicating the Subnet Block with +1 in the Just-Left Octet*

This example, with 10 total subnet bits, creates 256 blocks of four subnets each, for a total of 1024 subnets. This math matches the usual method of counting subnets, because $2^{10} = 1024$.

## Process with 17 or More Subnet Bits

To create a subnet design that allows 17 or more subnet bits to exist, the design must use a Class A network. In addition, the subnet part will consist of the entire second and third octets, plus part of the fourth octet. That means a lot of subnet IDs: at least $2^{17}$ (or 131,072) subnets. Figure 15-15 shows an example of just such a structure, with a Class A network and a /26 mask.

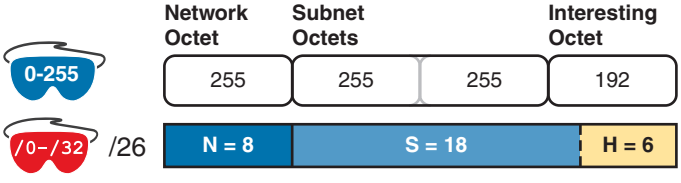



**Figure 15-15** *Address Structure with 18 Subnet Bits*

To find all the subnet IDs in this example, you use the same general process as with 9–16 subnet bits, but with many more subnet blocks to create. In effect, you have to create a subnet block for all combinations of values (0–255, inclusive) in both the second and third octets. Figure 15-16 shows the general idea. Note that with only 2 subnet bits in the fourth octet in this example, the subnet blocks will have four subnets each.

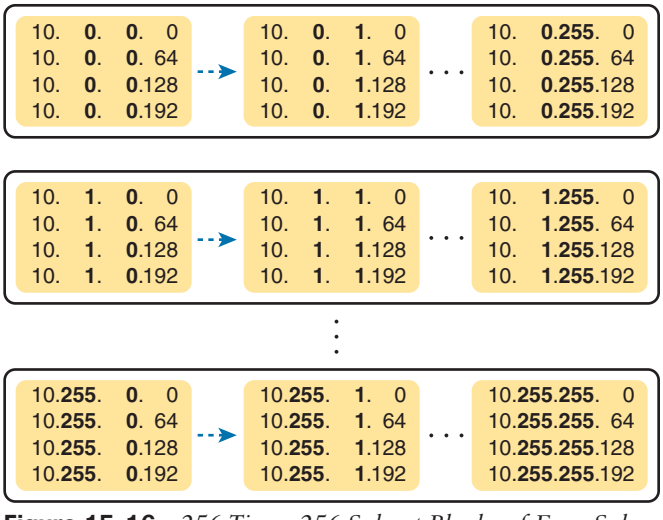



**Figure 15-16** *256 Times 256 Subnet Blocks of Four Subnets*

### Practice Finding All Subnet IDs

*Before moving to the next chapter*, practice until you get the right answer most of the time—but use any tools you want and take all the time you need. Then, you can move on with your reading. *Before taking the exam*, practice until you reach the goals in the right column of Table 15-5, which summarizes the key concepts and suggestions for this two-phase approach.

**Table 15-5** Keep-Reading and Take-Exam Goals for This Chapter's Topics

| Time Frame | Before Moving to the Next Chapter | Before Taking the Exam |
|---|---|---|
| **Focus On…** | Learning how | Being correct and fast |
| **Tools Allowed** | All | Your brain and a notepad |
| **Goal: Accuracy** | 90% correct | 100% correct |
| **Goal: Speed** | Any speed | 45 seconds |

#### Practice Problems for Finding All Subnet IDs

The following list shows three separate problems, each with a classful network number and prefix-style mask. Find all subnet IDs for each problem:

1. 192.168.9.0/27
2. 172.30.0.0/20
3. 10.0.0.0/17

The section "Answers to Earlier Practice Problems," later in this chapter, lists the answers.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 15-6 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 15-6** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Practice subnet design | | Website, Appendix G |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 15-7** Key Topics for Chapter 15

| Key Topic Element | Description | Page Number |
|---|---|---|
| Definition | Facts about binary values in subnet masks | 384 |
| List | The shorter three-step process to find all prefix masks that meet certain requirements | 384 |
| List | Reasons to choose one subnet mask versus another | 385 |
| Step list | The complete process for finding and choosing masks to meet certain requirements | 385 |
| Step list | Formal steps to find all subnet IDs when fewer than 8 subnet bits exist | 389 |
| Figure 15-9 | An example of adding the magic number in the interesting octet to find all subnet IDs | 391 |
| Step list | Formal steps to find all subnet IDs when more than 8 subnet bits exist | 394 |



## Key Terms You Should Know

broadcast subnet, subnet zero, zero subnet

## Additional Practice for This Chapter's Processes

For additional practice with subnet mask design and finding all subnet IDs, you may do the same set of practice problems using your choice of tools:

**PDF:** Practice using online Appendix G, "Practice for Chapter 15: Subnet Design."

## Answers to Earlier Practice Problems

### Answers to Practice Choosing Subnet Masks

The earlier section "Practice Choosing Subnet Masks" listed three practice problems. The answers are listed here so that the answers are nearby but not visible from the list of problems. Table 15-8 lists the answers, with notes related to each problem following the table.

**Table 15-8** Practice Problems: Find the Masks That Meet Requirements

| Problem | Class | Minimum Subnet Bits | Minimum Host Bits | Prefix Range | Prefix to Maximize Subnets | Prefix to Maximize Hosts |
|---|---|---|---|---|---|---|
| 1 | A | 11 | 9 | /19 – /23 | /23 | /19 |
| 2 | B | 8 | 8 | /24 | — | — |
| 3 | C | 3 | 4 | /27 – /28 | /28 | /27 |

1. N=8, because the problem lists Class A network 10.0.0.0. With a need for 1500 subnets, 10 subnet bits supply only 1024 subnets (per Table 15-2), but 11 subnet bits (S) would provide 2048 subnets—more than the required 1500. Similarly, the smallest number of host bits would be 9, because $2^8 - 2 = 254$, and the design requires 300 hosts/subnet. The shortest prefix mask would then be /19, found by adding N (8) and the smallest usable number of subnet bits S (11). Similarly, with a minimum H value of 9, the longest prefix mask, maximizing the number of subnets, is 32 – H = /23.
2. N=16, because the problem lists Class B network 172.25.0.0. With a need for 130 subnets, 7 subnet bits supply only 128 subnets (per Table 15-2), but 8 subnet bits (S) would provide 256 subnets—more than the required 130. Similarly, the smallest number of host bits would be 8, because $2^7 - 2 = 126$—close to the required 127, but not quite enough, making H=8 the smallest number of host bits that meets requirements. Note that the network, minimum subnet bits, and minimum host bits add up to 32, so only one mask meets the requirements, namely /24, found by adding the number of network bits (16) to the minimum number of subnet bits (8).
3. N=24, because the problem lists Class C network 192.168.83.0. With a need for 8 subnets, 3 subnet bits supply enough, but just barely. The smallest number of host bits would be 4, because $2^3 - 2 = 6$, and the design requires 8 hosts/subnet. The shortest prefix mask would then be /27, found by adding N (24) and the smallest usable number of subnet bits S (3). Similarly, with a minimum H value of 4, the longest prefix mask, maximizing the number of subnets, is 32 – H = /28.

## Answers to Practice Finding All Subnet IDs

The earlier section "Practice Finding All Subnet IDs" listed three practice problems. The answers are listed here so that they are not visible from the same page as the list of problems.

### Answer, Practice Problem 1

Problem 1 lists network 192.168.9.0, mask /27. The mask converts to DDN mask 255.255.255.224. When used with a Class C network, which has 24 network bits, only 3 subnet bits exist, and they all sit in the fourth octet. So, this problem is a case of fewer than 8 subnet bits, with the fourth octet as the interesting octet.

To get started listing subnets, first write down the zero subnet and then start adding the magic number in the interesting octet. The zero subnet equals the network ID (192.168.9.0, in this case). The magic number, calculated as 256 – 224 = 32, should be added to the previous subnet ID's interesting octet. Table 15-9 lists the results.

**Table 15-9** List-All-Subnets Chart: 192.168.9.0/27

| Octet | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Mask | 255 | 255 | 255 | 224 |
| Magic Number | — | — | — | 32 |
| Classful Network/Subnet Zero | 192 | 168 | 9 | 0 |
| First Nonzero Subnet | 192 | 168 | 9 | 32 |
| Next Subnet | 192 | 168 | 9 | 64 |

| Octet | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Next Subnet | 192 | 168 | 9 | 96 |
| Next Subnet | 192 | 168 | 9 | 128 |
| Next Subnet | 192 | 168 | 9 | 160 |
| Next Subnet | 192 | 168 | 9 | 192 |
| Broadcast Subnet | 192 | 168 | 9 | 224 |
| Invalid—Used by Process | 192 | 168 | 9 | 256 |

### Answer, Practice Problem 2

Problem 2 lists network 172.30.0.0, mask /20. The mask converts to DDN mask 255.255.240.0. When used with a Class B network, which has 16 network bits, only 4 subnet bits exist, and they all sit in the third octet. So, this problem is a case of fewer than 8 subnet bits, with the third octet as the interesting octet.

To get started listing subnets, first write down the zero subnet and then start adding the magic number in the interesting octet. The zero subnet equals the network ID (or 172.30.0.0, in this case). The magic number, calculated as 256 – 240 = 16, should be added to the previous subnet ID's interesting octet. Table 15-10 lists the results.

**Table 15-10** List-All-Subnets Chart: 172.30.0.0/20

| Octet | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Mask | 255 | 255 | 240 | 0 |
| Magic Number | — | — | 16 | — |
| Classful Network/Subnet Zero | 172 | 30 | 0 | 0 |
| First Nonzero Subnet | 172 | 30 | 16 | 0 |
| Next Subnet | 172 | 30 | 32 | 0 |
| Next Subnet | 172 | 30 | Skipping… | 0 |
| Next Subnet | 172 | 30 | 224 | 0 |
| Broadcast Subnet | 172 | 30 | 240 | 0 |
| Invalid—Used by Process | 172 | 30 | 256 | 0 |

### Answer, Practice Problem 3

Problem 3 lists network 10.0.0.0, mask /17. The mask converts to DDN mask 255.255.128.0. When used with a Class A network, which has 8 network bits, 9 subnet bits exist. Using the terms unique to this chapter, octet 3 is the interesting octet, with only 1 subnet bit in that octet, and octet 2 is the just-left octet, with 8 subnet bits.

In this case, begin by finding the first subnet block. The magic number is 256 – 128 = 128. The first subnet (zero subnet) equals the network ID. So, the first subnet ID block includes the following:

10.0.0.0

10.0.128.0

Then, you create a subnet block for all 256 possible values in the just-left octet, or octet 2 in this case. The following list shows the first three subnet ID blocks, plus the last subnet ID block, rather than listing page upon page of subnet IDs:

10.0.0.0 (zero subnet)

10.0.128.0

10.1.0.0

10.1.128.0

10.2.0.0

10.2.128.0

…

10.255.0.0

10.255.128.0 (broadcast subnet)

*This page intentionally left blank*



# Part IV Review

Keep track of your part review progress with the checklist in Table P4-1. Details on each task follow the table.

**Table P4-1** Part IV Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Subnetting Exercises in PDF Appendices on Companion Website | | |
| Interactive Subnetting Exercises on Companion Website | | |
| Watch Video | | |

## Repeat All DIKTA Questions

For this task, use the PTP software to answer the "Do I Know This Already?" questions again for the chapters in this part of the book.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Watch Video

The companion website includes a variety of subnetting videos. Some have the familiar common mistake and Q&A format as the other chapters, whereas others simply work through various subnetting processes. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

## Subnetting Exercises

Chapters 12 through 15 list some subnetting exercises, along with time and accuracy goals. Now is a good time to work on those goals. Some options include the following:

**Practice from this book's appendices or web applications:** This book includes four appendices of subnetting practice exercises, all available from the companion website. From that website, to use the static PDFs, look to the section titled "Study Resources." To find the interactive versions of the exercises in those same appendices, look to the section titled "Memory Tables and Practice Exercises." Look for these topics/appendices:

Appendix D, "Practice for Chapter 12: Analyzing Classful IPv4 Networks"

Appendix E, "Practice for Chapter 13: Analyzing Subnet Masks"

Appendix F, "Practice for Chapter 14: Analyzing Existing Subnets"

Appendix G, "Practice for Chapter 15: Subnet Design"

**IP Subnetting Practice Question Kit:** As mentioned in the Introduction to the book, if you want large amounts of subnetting practice, with explanations that match the processes in the book, consider purchasing the IP Subnetting Practice Question Kit, available at ciscopress.com.

**Author's blog:** I've written a few dozen subnetting exercises on the blog over the years. Just look at the Questions menu item at the top of the page, and you will see a variety of IPv4 addressing and subnetting question types. Start at https://www.certskills.com.



Parts V and VI work together to reveal the details of how to implement IPv4 routing in Cisco routers. To that end, Part V focuses on the most common features for Cisco routers, including IP address configuration, connected routes, and static routes. Part VI then goes into some detail about the one IP routing protocol discussed in this book: OSPF Version 2 (OSPFv2).

Part V follows a progression of topics. First, Chapter 16 examines the fundamentals of routers—the physical components, how to access the router command-line interface (CLI), and the configuration process. Chapter 16 makes a close comparison of the switch CLI and its basic administrative commands so that you need to learn only new commands that apply to routers but not to switches.

Chapter 17 then moves on to discuss how to configure routers to route IPv4 packets in the most basic designs. Those designs require a simple IP address/mask configuration on each interface, with the addition of a static route command—a command that directly configures a route into the IP routing table—for each destination subnet.

Chapter 18 continues the progression into more challenging but more realistic configurations related to routing between subnets in a LAN environment. Most LANs use many VLANs, with one subnet per VLAN. Cisco routers and switches can be configured to route packets between those subnets, with more than a few twists in the configuration.

Chapter 19 moves the focus from routers to hosts. Hosts rely on a working internetwork of routers, but the hosts need IP settings as well, with an IP address, mask, default gateway, and DNS server list. This chapter examines how hosts can dynamically learn these IP settings using Dynamic Host Configuration Protocol (DHCP), and the role the routers play in that process. The chapter also shows how to display and understand IP settings on hosts.

Finally, Part V closes with a chapter about troubleshooting IPv4 routing. Chapter 20 features the **ping** and **traceroute** commands, two commands that can help you discover not only whether a routing problem exists but also where the problem exists. The other chapters show how to confirm whether a route has been added to one router's routing table, while the commands discussed in Chapter 20 teach you how to test the end-to-end routes from sending host to receiving host.

# Part V

## IPv4 Routing

**Chapter 16:** Operating Cisco Routers

**Chapter 17:** Configuring IPv4 Addresses and Static Routes

**Chapter 18:** IP Routing in the LAN

**Chapter 19:** IP Addressing on Hosts

**Chapter 20:** Troubleshooting IPv4 Routing

**Part V Review**



CHAPTER 16

# Operating Cisco Routers

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.a Routers**

**1.2 Describe characteristics of network topology architectures**

**1.2.e Small office/home office (SOHO)**

**1.6 Configure and verify IPv4 addressing and subnetting**

This chapter begins a series of chapters that focus on specific Cisco router features. It begins by discussing Cisco routers: hardware, operating system, interfaces, and other components that comprise a router. This first section helps give you concrete examples of interfaces and devices before getting into the many concept and topology drawings to come.

The second section of the chapter then discusses the command-line interface (CLI) on a Cisco router, which has the same look and feel as the Cisco switch CLI. However, unlike switches, routers require some minimal configuration before they will do their primary job: to forward IP packets. The second section of this chapter discusses the concepts and commands to configure a router so it begins forwarding IP packets on its interfaces.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 16-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Installing Cisco Routers | 1 |
| Enabling IPv4 Support on Cisco Router Interfaces | 2–6 |

1. Which operating systems run on Cisco enterprise routers and use a CLI that works much like the CLI on Cisco LAN switches? (Choose two answers.)
   - **a.** CatOS
   - **b.** IOS
   - **c.** Windows
   - **d.** IOS XE

2. Which action would you expect to be true of a router CLI interaction that is not true when configuring a LAN switch that performs only Layer 2 switching functions?
   a. Moving from global to physical interface configuration mode
   b. Configuring an IP address in physical interface configuration mode
   c. Configuring a 10/100/1000 port's settings related to speed and autonegotiation
   d. Configuring a console password
3. Which answers list a task that could be helpful in making a router interface G0/0 ready to route packets? (Choose two answers.)
   a. Configuring the **ip address** *address mask* command in G0/0 configuration mode
   b. Configuring the **ip address** *address* and **ip mask** *mask* commands in G0/0 configuration mode
   c. Configuring the **no shutdown** command in G0/0 configuration mode
   d. Setting the interface **description** in G0/0 configuration mode
4. The output of the **show ip interface brief** command on R1 lists interface status codes of "down" and "down" for interface GigabitEthernet 0/0. The interface connects to a LAN switch with a UTP straight-through cable. Which of the following could be true?
   a. The **shutdown** command is currently configured for router interface G0/0.
   b. The **shutdown** command is currently configured for the switch interface on the other end of the cable.
   c. The router was never configured with an **ip address** command on the interface.
   d. The router was configured with the **no ip address** command.
5. Which of the following commands list the IP address but not the subnet mask of an interface?
   a. **show running-config**
   b. **show protocols** *type number*
   c. **show ip interface brief**
   d. **show interfaces**
6. Which of the following is different on the Cisco switch CLI for a Layer 2 switch as compared with the Cisco router CLI?
   a. The commands used to configure simple password checking for the console
   b. The number of IP addresses configured
   c. The configuration of the device's hostname
   d. The configuration of an interface description

# Foundation Topics

## Installing Cisco Routers

Routers collectively provide the main feature of the network layer—the capability to forward packets end to end through a network. As introduced in Chapter 3, "Fundamentals of WANs and IP Routing," routers forward packets by connecting to various physical network links, like Ethernet LAN, Ethernet WAN, and serial WAN links, then using Layer 3 routing

logic to choose where to forward each packet. As a reminder, Chapter 2, "Fundamentals of Ethernet LANs," covered the details of making those physical connections to Ethernet networks, while Chapter 3 covered the basics of cabling with WAN links.

This section examines some of the details of router installation and cabling, first from the enterprise perspective and then from the perspective of connecting a typical small office/home office (SOHO) to an ISP using high-speed Internet.

## Installing Enterprise Routers

A typical enterprise network has a few centralized sites as well as lots of smaller remote sites. To support devices at each site (the computers, IP phones, printers, and other devices), the network includes at least one LAN switch at each site. In addition, each site has a router, which connects to the LAN switch and to some WAN link. The WAN link provides connectivity from each remote site, back to the central site, and to other sites through the connection to the central site.

Figures 16-1 and 16-2 show a couple of different kinds of network diagrams that might be used to represent an enterprise network. The style of Figure 16-1 supports discussions about Layer 3 topics, showing the subnet IDs, masks, and interface IP addresses in shorthand. The figure also keeps the physical and data-link details to a minimum with these conventions:

**Ethernet LAN:** Simple straight lines with one or more LAN switches implied but not shown.

**Ethernet WAN:** Shown as a straight line, often with a cloud over it, with some kind of Ethernet interface identifier shown by the router (in this case, G0/1/0 and G0/0/0, which refers to GigabitEthernet interfaces).

**Serial WAN:** A line with a crooked part in the middle (a "lightning bolt") represents a typical point-to-point serial link as introduced in Chapter 3.

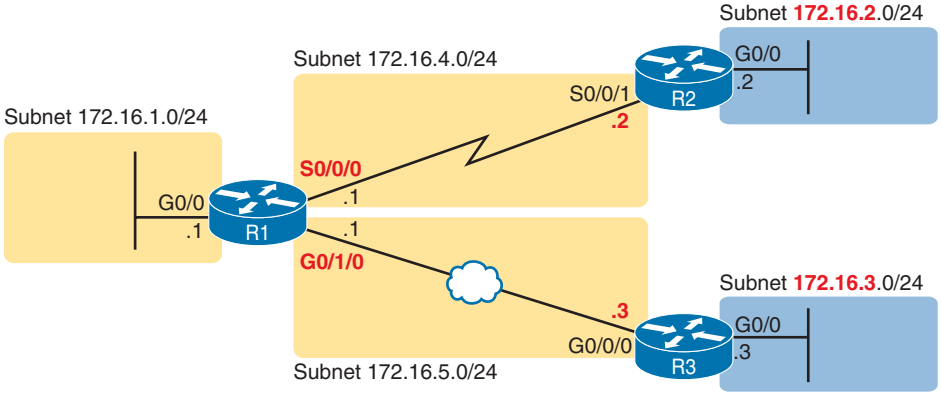



**Figure 16-1** *Generic Enterprise Network Diagram*

Answers to the "Do I Know This Already?" quiz:

**1** B, D **2** B **3** A, C **4** B **5** C **6** B

In comparison, Figure 16-2 shows the same network, with more detail about the physical cabling but with IP details removed. Focusing on the LANs, all the lines connected to the LAN switches could be the standard UTP cabling with RJ-45 connectors.

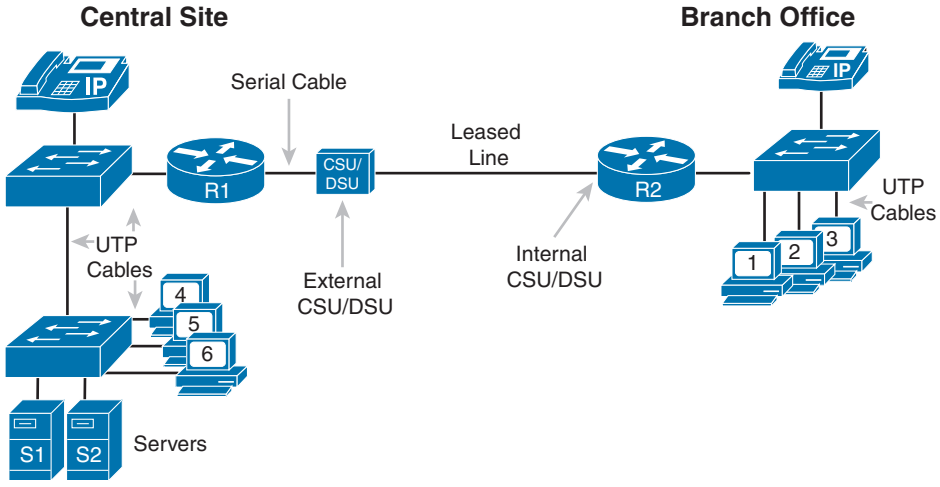



**Figure 16-2** *More Detailed Cabling Diagram for the Same Enterprise Network*



Next, consider the hardware on the ends of the serial link between the two routers. In a real serial link that runs through a service provider, the link terminates at a channel service unit/data service unit (CSU/DSU). The CSU/DSU can either sit outside the router as a separate device (as shown on the left at Router R1) or integrated into the router's serial interface hardware (as shown on the right).

As for cabling, the service provider will run the cable into the enterprise's wiring closet and often put an RJ-48 connector (same size as an RJ-45 connector) on the end of the cable. That cable should connect to the CSU/DSU. With an internal CSU/DSU (as with Router R2 in Figure 16-2), the router serial port has an RJ-48 port to which the serial cable should connect. With an external CSU/DSU, the CSU/DSU must be connected to the router's serial card via a short serial cable.

## The Cisco Router Operating Systems

All routers have the usual components found in a computer: a CPU, RAM, permanent memory (usually flash memory), and other electronics. They also run an operating system (OS), which goes by the name **IOS**. The original Cisco routers used IOS; even today, some current router products use IOS. However, Cisco has created other enterprise-class router product families that use a different variation of IOS named **IOS XE**.

Cisco created IOS XE in the 2000s to improve the IOS software architecture. Those improvements may not be evident to the casual observer, but to name a few, IOS XE reduces unplanned and planned downtime, better protects against cyberattacks, and aids network automation. For instance, IOS XE devices can support upgrading the OS while continuing to forward frames and packets, while IOS cannot.

Thankfully, IOS XE uses the same familiar CLI as IOS. Both use the same commands, for the most part, the same command syntax, navigation and modes, and so on. If you learned the CLI using an IOS router, you might not even notice when using a router that runs IOS XE later.

Because the differences between IOS and IOS XE do not matter in most cases in this book, the book uses the term *IOS* almost exclusively to refer to the router OS. When differences in IOS versus IOS XE matter, the text will note the differences.

## Cisco Integrated Services Routers

Product vendors, including Cisco, typically provide several different types of router hardware. Today, routers often do much more work than simply routing packets; in fact, they serve as a device or platform from which to provide many network services.

As an example, consider the networking functions needed at a typical branch office. A typical enterprise branch office needs a router for WAN/LAN connectivity, and a LAN switch to provide a high-performance local network and connectivity into the router and WAN. Many branches also need voice-over-IP (VoIP) services to support IP phones, and several security services as well. Plus, it is hard to imagine a site with users that does not have Wi-Fi access today. So, rather than require multiple separate devices at one site, as shown in Figure 16-2, Cisco offers single devices that act as both router and switch and provide other functions as well.

For the sake of learning and understanding the different functions, this book focuses on using a separate switch and separate router, which provides a much cleaner path for learning the basics. However, most Cisco products and product families support a variety of functions in one device.

The Cisco **Integrated Services Router (ISR)** product families include routers that perform much more than routing. Cisco introduced the first ISRs in the mid-2000s, with some ISR families still in existence in the early 2020s when this chapter was most recently updated. The name itself emphasizes the role of being a router, while also integrating other functions (services).

Figure 16-3 shows a Cisco 4321 ISR, with some of the more important features highlighted. The top part of the figure shows a full view of the back of the router. This model comes with two built-in Gigabit Ethernet interfaces and two modular slots that allow you to add small cards called Network Interface Modules (NIMs). The bottom of the figure shows one sample NIM (a NIM that provides two serial interfaces). The router has other items as well, including both an RJ-45 and USB console port.

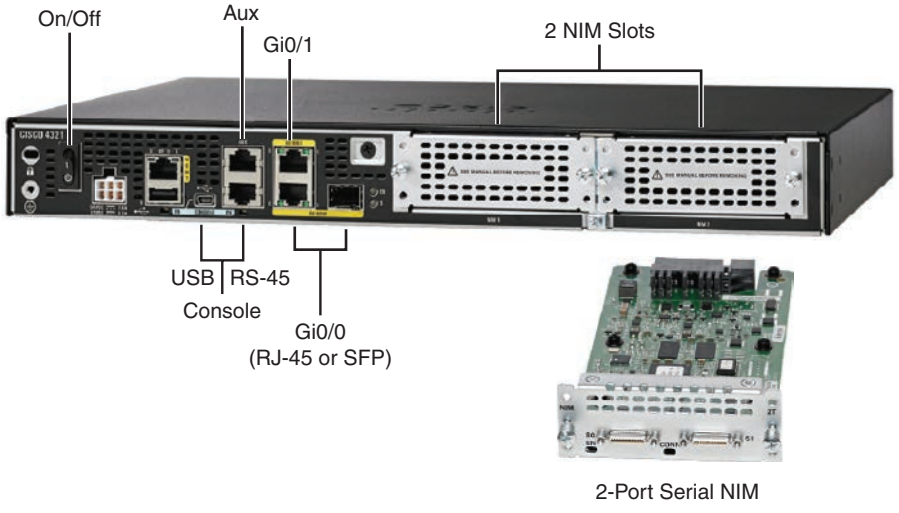



**Figure 16-3** *Photos of a Model 4321 Cisco Integrated Services Router (ISR)*

The figure shows an important feature for using routers to connect to both Ethernet LANs and Ethernet WAN services. Look closely at Figure 16-3's Gigabit interfaces. G0/1 refers to interface GigabitEthernet0/1 and is an RJ-45 port that supports UTP cabling only. However, interface G0/0 (short for GigabitEthernet0/0) has some interesting features:

- The router has two ports for one interface (G0/0).
- You can use one or the other at any point in time, but not both.
- One physical port is an RJ-45 port that supports copper cabling (implying that it is used to connect to a LAN).
- The other G0/0 physical port is a small form pluggable (SFP) port that would support various fiber Ethernet standards, allowing the port to be used for Ethernet WAN purposes.

Cisco commonly makes one or more of the Ethernet ports on its Enterprise class routers support SFPs so that the engineer can choose an SFP that supports the type of Ethernet cabling provided by the Ethernet WAN service provider.



**NOTE** When building a lab network to study for CCNA or CCNP, because your devices will be in the same place, you can create Ethernet WAN links by using the RJ-45 ports and a UTP cable without the need to purchase an SFP for each router.

### The Cisco Catalyst Edge Platform

In 2020, Cisco announced a new router family with the **Cisco Catalyst Edge Platform** branding, a departure from the previous ISR branding. The new family includes models in the 8000 series, such as the 8200, 8300, and 8500. These new models replaced some existing Cisco ISR families in the Cisco product lineup, but not all.

If the ISR brand emphasized the multifunction nature of modern routers, the *Catalyst Edge Platform* emphasizes that point even more. First, the name does not even use the word *router*, even though the devices use IOS XE and act primarily as routers. The word *platform* emphasizes that the device serves as a platform to run various network services.

Also, note that the new branding uses the term *Catalyst*, a term historically used only for switches. Before the Catalyst 8000 series, the term *Catalyst* always referred to switches, but now Cisco uses the term more broadly. Just be aware in your travels outside of your CCNA studies that when you come across the Catalyst 8000 series devices, think of them as routers.

As for exam preparation, this chapter provides information about ISRs and Catalyst 8000s to provide some context. However, the exam topics do not mention specific Cisco product families. But knowing something about router models and families can help you connect the more generalized topics in CCNA to the devices in your network at work.

### Physical Installation

Armed with the cabling details in images like Figure 16-2 and the router hardware details in photos like Figure 16-3, you can physically install a router. To install a router, follow these steps:

Key Topic

**Step 1.** For any Ethernet LAN interface, connect the RJ-45 connector of an appropriate copper Ethernet cable between the RJ-45 Ethernet port on the router and one of the LAN switch ports.

**Step 2.** For any serial WAN ports:

- **Step A.** If using an external CSU/DSU, connect the router's serial interface to the CSU/DSU and the CSU/DSU to the line from the telco.
- **Step B.** If using an internal CSU/DSU, connect the router's serial interface to the line from the telco.

**Step 3.** For any Ethernet WAN ports:

- **Step A.** When ordering the Ethernet WAN service, confirm the required Ethernet standard and SFP type required to connect to the link, and order the SFPs.
- **Step B.** Install the SFPs into the routers, and connect the Ethernet cable for the Ethernet WAN link to the SFP on each end of the link.

**Step 4.** Connect the router's console port to a PC (as discussed in Chapter 4, "Using the Command-Line Interface"), as needed, to configure the router.

**Step 5.** Connect a power cable from a power outlet to the power port on the router.

**Step 6.** Power on the router using the on/off switch.

Note that Cisco enterprise routers typically have an on/off switch, while switches do not.

## Installing SOHO Routers

The terms **enterprise router** and *small office/home office (SOHO) router* act as a pair of contrasting categories for routers, both in terms of how vendors like Cisco provide to the market, and how enterprises use and configure those devices. The term *enterprise router* typically refers to a router that a company would use in a permanent business location, while a **SOHO router** would reside at an employee's home or at a small permanent site with just a few people. However, as you might guess, the line between a router acting as an enterprise router and a SOHO router is blurry, so use these terms as general categories.

Even with that general comparison, SOHO routers typically have two features that an enterprise router would be less likely to have:

- SOHO routers almost always use the Internet and virtual private network (VPN) technology for its WAN connection to send data back and forth to the rest of the enterprise.
- SOHO routers almost always use a multifunction device that does routing, LAN switching, VPN, wireless, and maybe other features.

For instance, at an enterprise business location, the building may contain enterprise routers, separate Ethernet switches, and separate wireless access points (APs), all connected together.

At a permanent business site with four employees and 10 total devices in the network, one SOHO router could provide all those same features in one device.

For instance, Figure 16-4 shows a typical SOHO site. The three icons that represent a router, switch, and access point all exist inside one box; the figure shows them separately to emphasize the fact that the one SOHO router provides several functions. On the left, the SOHO router provides wired and wireless LAN servers, and on the right, it provides WAN access through a cable Internet connection.

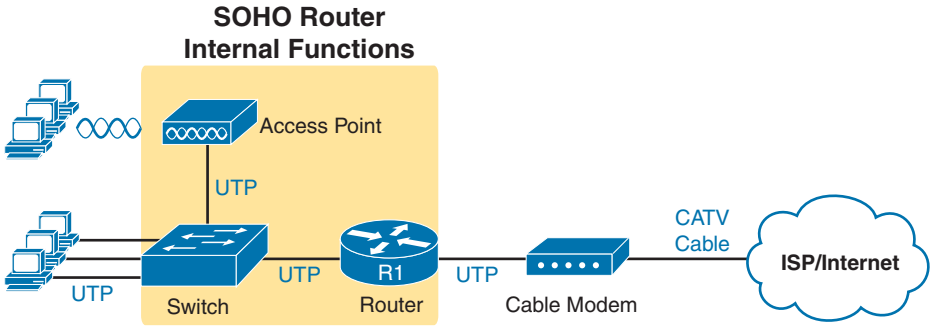



**Figure 16-4** *Devices in a SOHO Network with High-Speed CATV Internet*

Figure 16-4 does not reflect the physical reality of a SOHO router, so Figure 16-5 shows one cabling example. The figure shows user devices on the left, connecting to the router via wireless or via Ethernet UTP cabling. On the right in this case, the router uses an external cable modem to connect to the coaxial cable provided by the ISP. Then the router must use a normal UTP Ethernet port to connect a short Ethernet cable between the SOHO router and the cable modem.

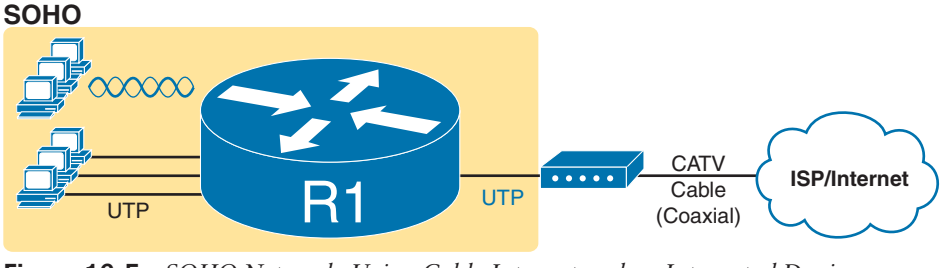



**Figure 16-5** *SOHO Network, Using Cable Internet and an Integrated Device*

## Enabling IPv4 Support on Cisco Router Interfaces

Routers support a relatively large number of features, with a large number of configuration and EXEC commands to support those features. You will learn about many of these features throughout the rest of this book.

**NOTE** For perspective, the Cisco router documentation includes a command reference, with an index to every single router command. A quick informal count listed over 5000 CLI commands.

This second section of the chapter focuses on commands related to router interfaces. To make routers work—that is, to route IPv4 packets—the interfaces must be configured. This section introduces the most common commands that configure interfaces, make them work, and give the interfaces IP addresses and masks.

## Accessing the Router CLI

The router command-line interface (CLI) works much like a switch. In fact, rather than repeat the detail in Chapter 4, just assume the CLI details from that chapter also apply to routers. If the details from Chapter 4 are not fresh in your memory, it might be worthwhile to spend a few minutes briefly reviewing that chapter as well as Chapter 7, "Configuring and Verifying Switch Interfaces," before reading further. The following list reviews some of the basic CLI features in common between switches and routers:

Key Topic

- User and Privileged (enable) mode
- Entering and exiting configuration mode, using the **configure terminal**, **end**, and **exit** commands and the Ctrl+Z key sequence
- Configuration of console, Telnet (vty), and enable secret passwords
- Configuration of Secure Shell (SSH) encryption keys and username/password login credentials
- Configuration of the hostname and interface description
- Configuration of an interface to be administratively disabled (**shutdown**) and administratively enabled (**no shutdown**)
- Navigation through different configuration mode contexts using commands like **line console 0** and **interface** *type number*
- CLI help, command editing, and command recall features
- The meaning and use of the startup-config (in NVRAM), running-config (in RAM), and external servers (like TFTP), along with how to use the **copy** command to copy the configuration files and IOS images

The most significant differences in the CLI between Cisco routers and switches come from the primary functions of each device: Layer 2 LAN switching by switches and Layer 3 IP routing by routers. LAN switches consider physical interfaces *Layer 2 interfaces*, or *switched interfaces*, meaning the switch should receive Ethernet frames on the interface and pass those into the LAN switching logic. Routers consider their physical interfaces to be *Layer 3 interfaces* or *routed interfaces*. Layer 3 interfaces expect to receive frames, then de-encapsulate the packet found inside the frame, and pass the packet off to the internal routing processes of the router.

The best way to become comfortable with the differences between switches and routers is to learn more and more commands on each. The chapters from here to the end of this book, and several in *CCNA 200-301 Official Cert Guide, Volume 2,* Second Edition, reveal more detail about router features and the related commands. Here are a few of the router CLI topics you will learn about for routers, with one sample command each:

Key Topic

- Configuring interface IP addresses (**ip address** *address mask*)
- Configuring IP routing protocols (e.g., OSPF) (**router ospf** *process-id*)

- Verifying the IP routing table (**show ip route**)
- Configuring static IP routes (**ip route** *subnet mask next-hop-address*)

**NOTE** Modern routers and switches often support both routing and switching features. Network engineers can then decide whether they need a switch in a particular location in a network design (usually when stronger switching features are required) or a router (when stronger routing features are needed). When studying for CCNA, be aware that routers and switches often support commands for both routing and switching, commands that you might otherwise expect to see only in one or the other.

Cisco routers require the same commands to secure Telnet and SSH, so take some time to review the "Securing the Switch CLI" section in Chapter 6, "Configuring Basic Switch Management." Additionally, take extra care with one command related to Telnet and SSH configuration: the **transport input** line subcommand. The command has different defaults on switches and routers, and the default has migrated to a new setting as well. For instance, older enterprise-class routers using IOS (not IOS XE) often default to use **transport input none**, meaning the router supported neither Telnet nor SSH. Enterprise-class routers running IOS XE often default to **transport input ssh**, supporting SSH but not Telnet.



## Router Interfaces

One minor difference between Cisco switches and routers is that routers support a much wider variety of interfaces. Today, LAN switches support only Ethernet interfaces of various speeds. Routers support a variety of other types of interfaces, including serial interfaces, cable TV, 4G/5G wireless, and others not mentioned in this book.

Most Cisco routers have at least one Ethernet interface of some type. Many of those Ethernet interfaces support multiple speeds and use autonegotiation, so for consistency, the router IOS refers to these interfaces based on the fastest speed. For example, a 10-Mbps-only Ethernet interface would be configured with the **interface ethernet** *number* configuration command, a 10/100 interface with the **interface fastethernet** *number* command, and a 10/100/1000 interface with the **interface gigabitethernet** *number* command. However, when discussing these interfaces all together, engineers would simply call them *ethernet interfaces*, regardless of the maximum speed.

Some Cisco routers have serial interfaces. As you might recall from Chapter 3, Cisco routers use serial interfaces to connect to a serial link. Each point-to-point serial link can then use High-Level Data Link Control (HDLC, the default) or Point-to-Point Protocol (PPP).

Router commands refer to interfaces first by the type of interface (Ethernet, Fast Ethernet, Gigabit Ethernet, Serial, and so on) and then with a unique interface identifier (a number) on that router. Depending on the router model, the interface identifiers might be a single number, two numbers separated by a slash, or three numbers separated by slashes. For example, all of the following configuration commands are correct on at least one model of Cisco router:

```
interface ethernet 0
interface fastethernet 0/1
interface gigabitethernet 0/0
interface gigabitethernet 0/1/0
interface serial 1/0/1
interface cellular 0/0/1
```

Two of the most common commands to display the interfaces, and their status, are the **show ip interface brief** and **show interfaces** commands. The first of these commands displays a list with one line per interface, with some basic information, including the interface IP address and interface status. The second command lists the interfaces, but with a large amount of information per interface. Example 16-1 shows a sample of each command. The output comes from an ISR router that has both a GigabitEthernet0/0 interface and a GigabitEthernet0/1/0 interface, showing a case with both two-digit and three-digit interface identifiers.

**Example 16-1** *Listing the Interfaces in a Router*

```
R1# show ip interface brief
Interface                  IP-Address      OK? Method Status                Protocol
Embedded-Service-Engine0/0 unassigned      YES NVRAM  administratively down down
GigabitEthernet0/0         172.16.1.1      YES NVRAM  down                  down
GigabitEthernet0/1         unassigned      YES NVRAM  administratively down down
Serial0/0/0                172.16.4.1      YES manual up                    up
Serial0/0/1                unassigned      YES unset  administratively down down
GigabitEthernet0/1/0       172.16.5.1      YES NVRAM  up                    up

R1# show interfaces gigabitEthernet 0/1/0
GigabitEthernet0/1/0 is up, line protocol is up
  Hardware is EHWIC-1GE-SFP-CU, address is 0201.a010.0001 (bia 30f7.0d29.8570)
  Description: Link in lab to R3's G0/0/0
  Internet address is 172.16.5.1/24
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive set (10 sec)
  Full Duplex, 1Gbps, media type is RJ45
  output flow-control is XON, input flow-control is XON
  ARP type: ARPA, ARP Timeout 04:00:00
  Last input 00:00:29, output 00:00:08, output hang never
  Last clearing of "show interface" counters never
  Input queue: 0/75/0/0 (size/max/drops/flushes); Total output drops: 0
  Queueing strategy: fifo
  Output queue: 0/40 (size/max)
  5 minute input rate 0 bits/sec, 0 packets/sec
  5 minute output rate 0 bits/sec, 0 packets/sec
     12 packets input, 4251 bytes, 0 no buffer
     Received 12 broadcasts (0 IP multicasts)
     0 runts, 0 giants, 0 throttles
     0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
     0 watchdog, 0 multicast, 0 pause input
     55 packets output, 8098 bytes, 0 underruns
     0 output errors, 0 collisions, 0 interface resets
```

```
     0 unknown protocol drops
     0 babbles, 0 late collision, 0 deferred
     0 lost carrier, 0 no carrier, 0 pause output
     0 output buffer failures, 0 output buffers swapped out
```

**NOTE** Commands that refer to router interfaces can be significantly shortened by truncating the words. For example, **sh int gi0/0** or **sh int g0/0** can be used instead of **show interfaces gigabitethernet0/0**. In fact, many network engineers, when looking over someone's shoulder, would say something like "just do a show int G-I-oh-oh command" in this case, rather than speaking the long version of the command.

Also, note that the **show interfaces** command lists a text interface description on about the third line, if configured. In this case, interface G0/1/0 had been previously configured with the **description Link in lab to R3's G0/0/0** command in interface configuration mode for interface G0/1/0. The **description** interface subcommand provides an easy way to keep small notes about what router interfaces connect to which neighboring devices, with the **show interfaces** command listing that information.



### Interface Status Codes

Each interface has two *interface status codes*. To be usable, the two interface status codes must be in an "up" state. The first status code refers to whether Layer 1 is working, and the second status code generally (but not always) refers to whether the data-link layer protocol is working. Table 16-2 summarizes these two status codes.

Key Topic

**Table 16-2** Interface Status Codes and Their Meanings

| Name | Location | General Meaning |
|---|---|---|
| Line status | First status code | Refers to the Layer 1 status. (For example, is the cable installed, is it the right/wrong cable, is the device on the other end powered on?) |
| Protocol status | Second status code | Refers generally to the Layer 2 status. It is always down if the line status is down. If the line status is up, a protocol status of down is usually caused by a mismatched data-link layer configuration. |

Several combinations of interface status codes exist, as summarized in Table 16-3. The table lists the status codes in order, from being disabled on purpose by the configuration to a fully working state.

Key Topic

**Table 16-3** Typical Combinations of Interface Status Codes

| Line Status | Protocol Status | Typical Reasons |
|---|---|---|
| Administratively down | Down | The interface has a **shutdown** command configured on it. |
| Down | Down | The interface is not **shutdown**, but the physical layer has a problem. For example, no cable has been attached to the interface, or with Ethernet, the switch interface on the other end of the cable is shut down, or the switch is powered off, or the devices on the ends of the cable use a different transmission speed. |

| Line Status | Protocol Status | Typical Reasons |
|---|---|---|
| Up | Down | Almost always refers to data-link layer problems, most often configuration problems. For example, serial links have this combination when one router was configured to use PPP and the other defaults to use HDLC. |
| Up | Up | Both Layer 1 and Layer 2 of this interface are functioning. |

For some examples, look back at Example 16-1's **show ip interface brief** command, to the three interfaces in the following list. The interfaces in this list each have a different combination of interface status codes; the list details the specific reasons for this status code in the lab used to create this example for the book.

**G0/0:** The interface is down/down, in this case because no cable was connected to the interface.

**G0/1:** The interface is administratively down/down, because the configuration includes the **shutdown** command under the G0/1 interface.

**S0/0/0:** The interface is up/up because a serial cable is installed, is connected to another router in a lab, and is working.

## Router Interface IP Addresses

Cisco enterprise routers require at least some configuration beyond the default configuration before they will do their primary job: routing IP packets. First, default router behavior may place an interface in a disabled (shutdown) state. Second, routers use a default interface command of **no ip address**, meaning the interface has no IP address or mask configured. For a router interface to function for routing packets, the interface must be up and configured with an IP address and mask.

To configure the address and mask, simply use the **ip address** *address mask* interface subcommand and ensure that the interface is enabled by using the **no shutdown** command. Figure 16-6 shows a simple IPv4 network with IPv4 addresses on Router R1, with Example 16-2 showing the matching configuration.

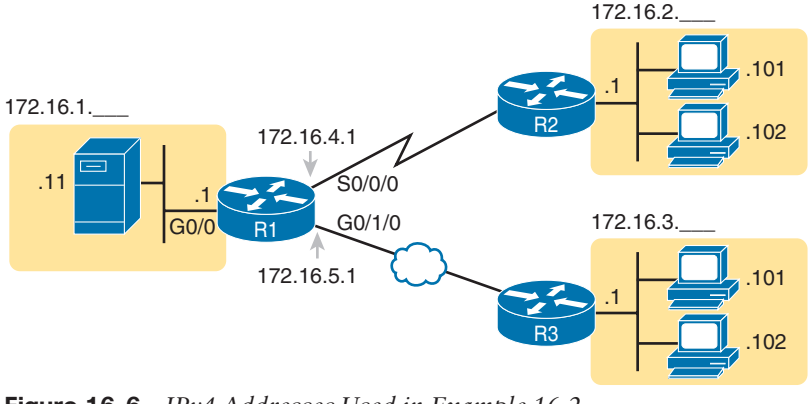



**Figure 16-6** *IPv4 Addresses Used in Example 16-2*

**Example 16-2** *Configuring IP Addresses on Cisco Routers*

```
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1config)# interface G0/0
R1(config-if)# ip address 172.16.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface S0/0/0
R1(config-if)# ip address 172.16.4.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface G0/1/0
R1(config-if)# ip address 172.16.5.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# ^Z
R1#
```

The **show protocols** command in Example 16-3 confirms the state of each of the three R1 interfaces in Figure 16-6 and the IP address and mask configured on those same interfaces.



**Example 16-3** *Verifying IP Addresses on Cisco Routers*

```
R1# show protocols
Global values:
   Internet Protocol routing is enabled
Embedded-Service-Engine0/0 is administratively down, line protocol is down
GigabitEthernet0/0 is up, line protocol is up
   Internet address is 172.16.1.1/24
GigabitEthernet0/1 is administratively down, line protocol is down
Serial0/0/0 is up, line protocol is up
   Internet address is 172.16.4.1/24
Serial0/0/1 is administratively down, line protocol is down
GigabitEthernet0/1/0 is up, line protocol is up
   Internet address is 172.16.5.1/24
```

One of the first actions to take when verifying whether a router is working is to find the interfaces, check the interface status, and check to see whether the correct IP addresses and masks are used. Examples 16-1 and 16-3 showed samples of the key **show** commands, while Table 16-4 summarizes those commands and the types of information they display.

Key Topic

**Table 16-4** Key Commands to List Router Interface Status

| Command | Lines of Output per Interface | IP Configuration Listed | Interface Status Listed? |
|---|---|---|---|
| **show ip interface brief** | 1 | Address | Yes |
| **show protocols** [*type number*] | 1 or 2 | Address/mask | Yes |
| **show interfaces** [*type number*] | Many | Address/mask | Yes |

### Ethernet Interface Autonegotiation

The first step to make a router Ethernet interface reach an up/up state requires the installation of the correct cable. For instance, for Ethernet WAN links, the physical standard may need to support distances of several kilometers, so the interface hardware typically requires fiber-optic cabling. Router LAN interfaces often use UTP cabling that connects to nearby LAN switches, with the expected straight-through cable pinout.

For Ethernet interfaces that allow multiple standards and speeds, you also need to ensure the interfaces use autonegotiation or use correct static settings. IOS routers use the same concepts and commands as Cisco switches, as discussed in the section, "Configuring Autonegotiation, Speed, and Duplex," in Chapter 7. IOS XE routers use the same autonegotiation processes but with different commands, so read the next few pages to work through those differences.

First, for the familiar, the top part of Figure 16-7 shows an example with an IOS Router R1 connected to a Cisco switch. The figure shows all the default settings on both the router and the switch, which cause both devices to perform IEEE autonegotiation.

Key Topic

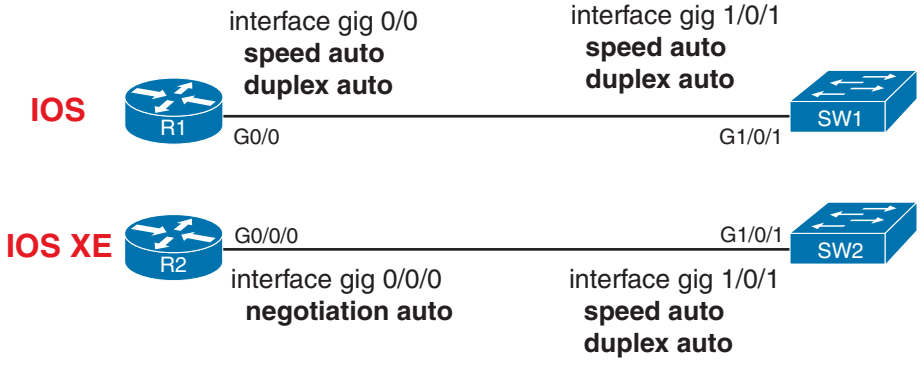



**Figure 16-7** *Default Ethernet Autonegotiation Configuration: IOS and IOS XE*

The bottom half of the figure focuses on the different configuration in IOS XE Router R2. The IOS XE default interface subcommand, **negotiation auto**, does just what it appears to do—it enables IEEE autonegotiation. As a result, both devices use autonegotiation to choose the link speed and duplex.

The **show interfaces** EXEC command reveals the chosen duplex and speed. For an IOS (not IOS XE) example, refer to Example 16-1, about nine lines into the **show interfaces** command output, to see the speed (1Gbps) and duplex (full) listed. However, none of the IOS **show** commands specifically note that the router used autonegotiation.

For the IOS XE case, several commands reveal the use of IEEE autonegotiation, as seen in Example 16-4's output from IOS XE Router R2 from Figure 16-7. The **show interfaces** command (in the highlighted line) confirms the speed and duplex. The highlighted line also lists "link type is auto," meaning it autonegotiated the values.

**Example 16-4** *Verifying IOS XE Autonegotiation Results (Default)*

```
! Output below is from IOS XE router R2
R2# show interfaces g0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  Hardware is C1111-2x1GE, address is 0200.1111.1111 (bia 2436.dadf.5680)
  Internet address is 10.1.1.1/24
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive not supported
  Full Duplex, 1000Mbps, link type is auto, media type is RJ45
! Lines omitted for brevity
R2# show interfaces g0/0/0 controller | include Autoneg
Admin State Up MTU 1500 Speed 1000mbps Duplex full Autoneg On Stats Interval 5
```

The IOS XE **show interfaces g0/0/0 controller** command at the end of the example filters the output to show only the subset of lines that include the text "Autoneg." That line also confirms the use of autonegotiation.



Whether you are using IOS or IOS XE, the recommendations for using autonegotiation on router Ethernet interfaces mirror the recommendations for switch interfaces, summarized here as follows:

- Use autonegotiation on both ends of an Ethernet link.
- If you must set the speed or duplex, ensure you configure the devices on both ends of the link to use the same speed and duplex.
- Avoid configuring speed and duplex on one end while relying on autonegotiation on the other.

If you must manually configure the settings, IOS and IOS XE differ slightly in how to configure speed and duplex manually. IOS routers use the same conventions as switches: just configure both the **speed** and **duplex** interface subcommands to specific settings, as shown in the left of Figure 16-8. With IOS routers, configuring both speed and duplex also disables autonegotiation.

Key Topic

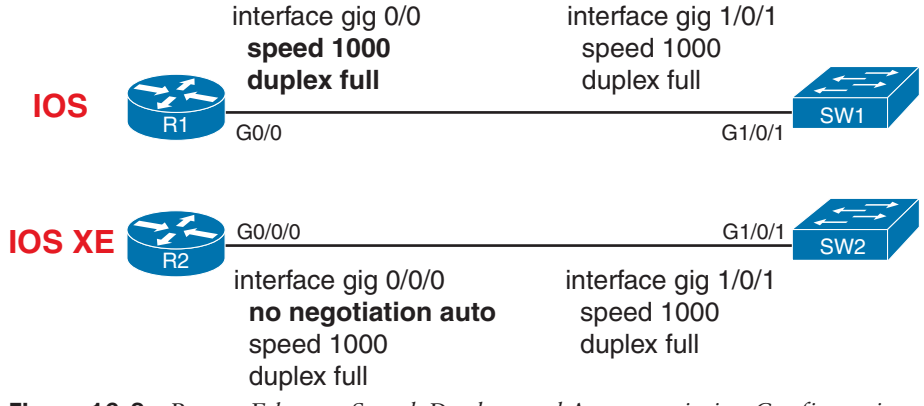



**Figure 16-8** *Router Ethernet Speed, Duplex, and Autonegotiation Configuration*

IOS XE requires the commands shown by Router R2 in the figure, with Example 16-5 demonstrating those same configuration steps. The example begins by showing how IOS XE does not allow the configuration of the **speed** or **duplex** commands until you disable autonegotiation with the **no negotiation auto** interface subcommand. After disabling autonegotiation, IOS XE allows setting the specific **speed** and **duplex** values. The example then shows log messages informing you that the interface failed and then recovered, which is common when using new speed and duplex settings, with approximately ten seconds to work through the process.

**Example 16-5** *IOS XE Router: Setting Speed and Duplex*

```
R2# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)# interface g0/0/0
R2(config-if)# speed 1000
Auto-negotiation is enabled. Speed cannot be set

R2(config-if)# no negotiation auto
R2(config-if) # speed 1000
R2(config-if) # duplex full
*Oct 14 12:24:16.014: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet0/0/0, changed state to down
*Oct 14 12:24:24.207: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet0/0/0, changed state to up
R2(config-if)# ^Z
*Oct 14 12:19:10.210: %LINK-3-UPDOWN: Interface GigabitEthernet0/0/0, changed state
to up
R2#
R2# show interfaces g0/0/0 controller
GigabitEthernet0/0/0 is up, line protocol is up
  Hardware is C1111-2x1GE, address is 0200.1111.1111 (bia 2436.dadf.5680)
  Internet address is 10.1.1.1/24
  MTU 1500 bytes, BW 1000000 Kbit/sec, DLY 10 usec,
     reliability 255/255, txload 1/255, rxload 1/255
  Encapsulation ARPA, loopback not set
  Keepalive not supported
  Full Duplex, 1000Mbps, link type is force-up, media type is RJ45
  output flow-control is on, input flow-control is on
! Lines omitted for brevity
Driver Configuration Block:
Admin State Up MTU 1500 Speed 1000mbps Duplex full Autoneg Off Stats Interval 5
! More lines omitted for brevity
```

The highlighted text near the end of Example 16-5 confirms the resulting speed and duplex. It also identifies that autonegotiation was not used, with the phrase "link type is force-up," in contrast to the "link type is auto" text in the autonegotiation in Example 16-4. Later in the output, one line explicitly lists the fact that autonegotiation is off.

**NOTE** On IOS XE routers, the [**no**] **negotiation auto** command controls autonegotiation. In IOS routers, configuring **speed** and **duplex** to specific values disables autonegotiation.

### Bandwidth and Clock Rate on Serial Interfaces

Cisco included serial WAN topics in the CCNA exam topic from its inception in 1998 until the CCNA 200-301 Version 1.0 blueprint in early 2020. Even though Cisco no longer mentions serial links in the exam topics, the exam might show them with the expectation that you at least understand the basics, such as the fact that two routers can send data over a serial link if the router interfaces on both ends are up/up and the routers have IP addresses in the same subnet.

However, some of you will want to make serial links work in a lab because you have some serial interface cards in your lab. If so, take the time to look at a few pages in the section titled "Bandwidth and Clock Rate on Serial Interfaces," in Appendix K, "Topics from Previous Editions," which shows how to cable and configure a WAN serial link in the lab.

## Router Auxiliary Port

Both routers and switches have a console port to allow administrative access, but most Cisco routers have an extra physical port called an auxiliary (Aux) port. The Aux port typically serves as a means to make a phone call to connect into the router to issue commands from the CLI.

The Aux port works like the console port, except that the Aux port is typically connected through a cable to an external analog modem, which in turn connects to a phone line. Then, the engineer uses a PC, terminal emulator, and modem to call the remote router. After being connected, the engineer can use the terminal emulator to access the router CLI, starting in user mode as usual.

Aux ports can be configured beginning with the **line aux 0** command to reach aux line configuration mode. From there, all the commands for the console line, covered mostly in Chapter 6, can be used. For example, the **login** and **password** *password* subcommands on the aux line could be used to set up simple password checking when a user dials in.

# Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 16-5 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 16-5** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review command tables | | Book |
| Review memory tables | | Website |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 16-6** Key Topics for Chapter 16

| Key Topic | Description | Page Number |
|---|---|---|
| List | Steps required to install a router | 412 |
| List | Similarities between a router CLI and a switch CLI | 414 |
| List | IOS features typical of routers (and all devices that perform routing) | 414 |
| Table 16-2 | Router interface status codes and their meanings | 417 |
| Table 16-3 | Combinations of the two interface status codes and the likely reasons for each combination | 417 |
| Table 16-4 | Commands useful to display interface IPv4 addresses, masks, and interface status | 419 |
| Figure 16-7 | Comparison of default autonegotiation configuration, routers with IOS and IOS XE | 420 |
| Figure 16-8 | Comparison of configuration to disable autonegotiation and use a set speed/duplex, routers with IOS and IOS XE | 421 |

## Key Terms You Should Know

Cisco Catalyst Edge Platform, enterprise router, Integrated Services Router (ISR), IOS, IOS XE, SOHO router

## Command References

Tables 16-7 and 16-8 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 16-7** Chapter 16 Configuration Command Reference

| Command | Description |
|---|---|
| **interface** *type number* | Global command that moves the user into configuration mode of the named interface. |
| **ip address** *address mask* | Interface subcommand that sets the router's IPv4 address and mask. |
| **[no] shutdown** | Interface subcommand that enables (**no shutdown**) or disables (**shutdown**) the interface. |

| Command | Description |
|---|---|
| **duplex** {**full** \| **half** \| **auto**} | IOS (not XE) interface command that sets the duplex, or sets the use of IEEE autonegotiation, for router LAN interfaces that support multiple speeds. |
| **speed** {**10** \| **100** \| **1000** \| **auto**} | IOS (not XE) interface command for router Gigabit (10/100/1000) interfaces that sets the speed at which the router interface sends and receives data, or sets the use of IEEE autonegotiation. |
| **description** *text* | An interface subcommand with which you can type a string of text to document information about that particular interface. |
| **[no] negotiation auto** | IOS XE (not IOS) interface command that enables or disables (**y**) the use of IEEE autonegotiation on the interface. There is no equivalent for routers that use IOS. |
| **duplex** {**full** \| **half** } | IOS XE (not IOS) interface command that sets the duplex, allowed only if the interface is already configured with the **no negotiation auto** subcommand. |
| **speed** {**10** \| **100** \| **1000**} | IOS XE (not IOS) interface command that sets the speed, allowed only if the interface is already configured with the **no negotiation auto** subcommand. |



**Table 16-8** Chapter 16 EXEC Command Reference

| Command | Purpose |
|---|---|
| **show interfaces** [*type number*] | Lists a large set of informational messages about each interface, or about the one specifically listed interface. |
| **show ip interface brief** | Lists a single line of information about each interface, including the IP address, line and protocol status, and the method with which the address was configured (manual or Dynamic Host Configuration Protocol [DHCP]). |
| **show protocols** [*type number*] | Lists information about the listed interface (or all interfaces if the interface is omitted), including the IP address, mask, and line/protocol status. |
| **show interfaces** [*type number*] **controller** | IOS XE (not IOS) command that lists detail about interface hardware and driver behavior, including detail about Ethernet autonegotiation processes and results. |



CHAPTER 17

# Configuring IPv4 Addresses and Static Routes

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.6 Configure and verify IPv4 addressing and subnetting**
- **3.0 IP Connectivity**
  - **3.1 Interpret the components of routing table**
    - **3.1.a Routing protocol code**
    - **3.1.b Prefix**
    - **3.1.c Network mask**
    - **3.1.d Next hop**
    - **3.1.e Administrative distance**
    - **3.1.f Metric**
    - **3.1.g Gateway of last resort**
  - **3.2 Determine how a router makes a forwarding decision by default**
    - **3.2.a Longest prefix match**
    - **3.2.b Administrative distance**
  - **3.3 Configure and verify IPv4 and IPv6 static routing**
    - **3.3.a Default route**
    - **3.3.b Network route**
    - **3.3.c Host route**
    - **3.3.d Floating static**

Routers route IPv4 packets. That simple statement actually carries a lot of hidden meaning. For routers to route packets, routers follow a routing process, and that routing process relies on information called IP routes. Each IP route lists a destination—an IP network, IP subnet, or some other group of IP addresses. Each route also lists instructions that tell the router where to forward packets sent to addresses in that IP network or subnet. For routers to do a good job of routing packets, routers need to have a detailed, accurate list of IP routes.

Routers use three methods to add IPv4 routes to their IPv4 routing tables. Routers first learn **connected routes**, which are routes for subnets attached to a router interface. Routers can also use **static routes**, which are routes created through a configuration command (**ip route**) that tells the router what route to put in the IPv4 **routing table**. And routers can

use a routing protocol, in which routers tell each other about all their known routes, so that all routers can learn and build dynamic routes to all networks and subnets.

This chapter examines IP routing in depth with the most straightforward routes that can be added to a router's routing table. The router starts with a detailed look at the IP packet routing (forwarding process)—a process that relies on each router having useful IP routes in their routing tables. The second section then examines connected routes, which are routes to subnets that exist on the interfaces connected to the local router. The third section then examines static routes, which are routes the network engineer configures directly.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 17-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| IP Routing | 1, 2 |
| Configuring IP Addresses and Connected Routes | 3 |
| Configuring Static Routes | 4–6 |

1. Router R1 lists a route in its routing table. Which of the following answers list a fact from a route that the router uses when matching the packet's destination address? (Choose two answers.)
   - **a.** Mask
   - **b.** Next-hop router
   - **c.** Subnet ID
   - **d.** Outgoing interface
2. PC1 sends an IP packet to PC2. To do so, PC1 sends the packet to Router R1, which routes it to Router R2, which routes it to Router R3, which routes it to the final destination (PC2). All links use Ethernet. How many routers de-encapsulate the IP packet from an Ethernet frame during its journey from PC1 to PC2?
   - **a.** 0
   - **b.** 1
   - **c.** 2
   - **d.** 3
3. After configuring a working router interface with IP address/mask 10.1.1.100/26, which of the following routes would you expect to see in the output of the **show ip route** command? (Choose two answers.)
   - **a.** A connected route for subnet 10.1.1.64 255.255.255.192
   - **b.** A connected route for subnet 10.1.1.0 255.255.255.0

c. A local route for host 10.1.1.100 255.255.255.192
d. A local route for host 10.1.1.100 255.255.255.255
e. A local route for host 10.1.1.64 255.255.255.255

4. Which of the following pieces of information could be listed in a correct **ip route** command on a local router? (Choose two answers.)
   a. The local router's IP address on the link between the two routers
   b. The next-hop router's IP address on the link between the two routers
   c. The next-hop router's interface ID on the link between the two routers
   d. The local router's interface ID on the link between the two routers
5. Which of the following commands correctly configures a static route?
   a. **ip route 10.1.3.0 255.255.255.0 10.1.130.253**
   b. **ip route 10.1.3.0 serial 0**
   c. **ip route 10.1.3.0 /24 10.1.130.253**
   d. **ip route 10.1.3.0 /24 serial 0**
6. A network engineer configures the **ip route 10.1.1.0 255.255.255.0 s0/0/0** command on a router and then issues a **show ip route** command from enable mode. No routes for subnet 10.1.1.0/24 appear in the output. Which of the following could be true?
   a. The **ip route** command has incorrect syntax and was rejected in config mode.
   b. Interface s0/0/0 is down.
   c. The router has no up/up interfaces in Class A network 10.0.0.0.
   d. The **ip route** command is missing a next-hop router IP address.

# Foundation Topics

## IP Routing

IP routing—the process of forwarding IP packets—delivers packets across entire TCP/IP networks, from the device that originally builds the IP packet to the device that is supposed to receive the packet. In other words, IP routing delivers IP packets from the sending host to the destination host.

The complete end-to-end routing process relies on network layer logic on hosts and on routers. The sending host uses Layer 3 concepts to create an IP packet, forwarding the IP packet to the host's default gateway (default router). The process requires Layer 3 logic on the routers as well, by which the routers compare the destination address in the packet to their routing tables, to decide where to forward the IP packet next.

The routing process also relies on data-link and physical details at each link. IP routing relies on serial WAN links, Ethernet WAN links, Ethernet LANs, wireless LANs, and many other networks that implement data-link and physical layer standards. These lower-layer devices and protocols move the IP packets around the TCP/IP network by encapsulating and transmitting the packets inside data-link layer frames.

The previous two paragraphs summarize the key concepts about IP routing as introduced back in Chapter 3, "Fundamentals of WANs and IP Routing." Next, this section reviews IP routing, while taking the discussion another step or two deeper, taking advantage of the additional depth of knowledge discussed in all the earlier chapters in this book.

## IPv4 Routing Process Reference

Because you already saw the basics back in Chapter 3, this section collects the routing process into steps for reference. The steps use many specific Ethernet LAN terms discussed in Parts II and III of this book and some IP addressing terms discussed in Part IV. The upcoming descriptions and example then discuss these summaries of routing logic to make sure that each step is clear.

The routing process starts with the host that creates the IP packet. First, the host asks the question: Is the destination IP address of this new packet in my local subnet? The host uses its own IP address/mask to determine the range of addresses in the local subnet. Based on its own opinion of the range of addresses in the local subnet, a LAN-based host acts as follows:

Key Topic

**Step 1.** If the destination is local, send directly:

- **a.** Find the destination host's MAC address. Use the already-known Address Resolution Protocol (ARP) table entry, or use ARP messages to learn the information.
- **b.** Encapsulate the IP packet in a data-link frame, with the destination data-link address of the destination host.

**Step 2.** If the destination is not local, send to the default gateway:

- **a.** Find the default gateway's MAC address. Use the already-known Address Resolution Protocol (ARP) table entry, or use ARP messages to learn the information.
- **b.** Encapsulate the IP packet in a data-link frame, with the destination data-link address of the default gateway.

17

Figure 17-1 summarizes these same concepts. In the figure, host A sends a local packet directly to host D. However, for packets to host B, on the other side of a router and therefore in a different subnet, host A sends the packet to its default router (R1). (As a reminder, the terms *default gateway* and *default router* are synonyms.)

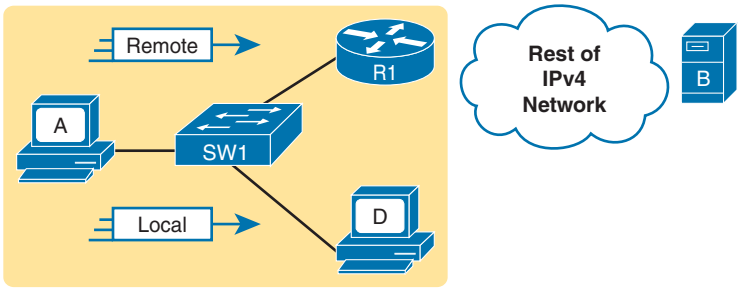



**Figure 17-1** *Host Routing Logic Summary*

Routers have a little more routing work to do as compared with hosts. While the host logic began with an IP packet sitting in memory, a router has some work to do before getting to that point. With the following five-step summary of a router's routing logic, the router takes the first two steps just to receive the frame and extract the IP packet, before thinking about the packet's destination address at Step 3. The steps are as follows:

Key Topic

1. For each received data-link frame, choose whether or not to process the frame. Process it if
   a. The frame has no errors (per the data-link trailer Frame Check Sequence [FCS] field).
   b. The frame's destination data-link address is the router's address (or an appropriate multicast or broadcast address).
2. If choosing to process the frame at Step 1, de-encapsulate the packet from inside the data-link frame.
3. Make a routing decision. To do so, compare the packet's destination IP address to the routing table and find the route that matches the destination address. This route identifies the **outgoing interface** of the router and possibly the **next-hop router**.
4. Encapsulate the packet into a data-link frame appropriate for the outgoing interface. When forwarding out LAN interfaces, use ARP as needed to find the next device's MAC address.
5. Transmit the frame out the outgoing interface, as listed in the matched IP route.

This routing process summary lists many details, but sometimes you can think about the routing process in simpler terms. For example, leaving out some details, this paraphrase of the step list details the same big concepts:

> The router receives a frame, removes the packet from inside the frame, decides where to forward the packet, puts the packet into another frame, and sends the frame.

To give you a little more perspective on these steps, Figure 17-2 breaks down the same five-step routing process as a diagram. The figure shows a packet arriving from the left, entering a router Ethernet interface, with an IP destination of host C. The figure shows the packet arriving, encapsulated inside an Ethernet frame (both header and trailer).

Key Topic

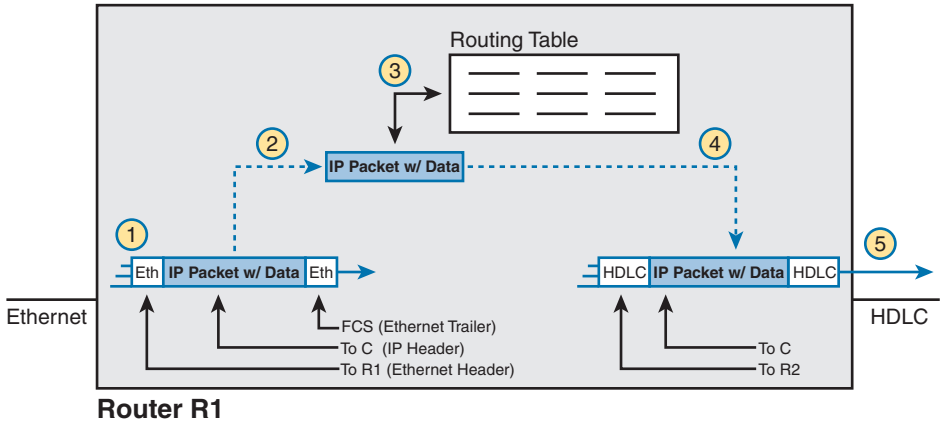



**Figure 17-2** *Router Routing Logic Summary*

---

Answers to the "Do I Know This Already?" quiz:

**1** A, C **2** D **3** A, D **4** B, C **5** A **6** B

Router R1 processes the frame and packet as shown with the numbers in the figure, matching the same five-step process described just before the figure, as follows:

1. Router R1 notes that the received Ethernet frame passes the FCS check and that the destination Ethernet MAC address is R1's MAC address, so R1 processes the frame.
2. R1 de-encapsulates the IP packet from inside the Ethernet frame's header and trailer.
3. R1 compares the IP packet's destination IP address to R1's IP routing table.
4. R1 encapsulates the IP packet inside a new data-link frame, in this case, inside a High-Level Data Link Control (HDLC) header and trailer.
5. R1 transmits the IP packet, inside the new HDLC frame, out the serial link on the right.

**NOTE** This chapter uses several figures that show an IP packet encapsulated inside a data-link layer frame. These figures often show both the data-link header as well as the data-link trailer, with the IP packet in the middle. The IP packets all include the IP header, plus any encapsulated data.

## An Example of IP Routing

The next several pages walk you through an example that discusses each routing step, in order, through multiple devices. The example uses a case in which host A (172.16.1.9) sends a packet to host B (172.16.2.9), with host routing logic and the five steps showing how R1 forwards the packet.

Figure 17-3 shows a typical IP addressing diagram for an IPv4 network with typical address abbreviations. The diagram can get a little too messy if it lists the full IP address for every router interface. When possible, these diagrams usually list the subnet and then the last octet or two of the individual IP addresses—just enough so that you know the IP address but with less clutter. For example, host A uses IP address 172.16.1.9, taking from subnet 172.16.1.0/24 (in which all addresses begin 172.16.1) and the .9 beside the host A icon. As another example, R1 uses address 172.16.1.1 on its LAN interface, 172.16.4.1 on one serial interface, and 172.16.5.1 on an Ethernet WAN interface.

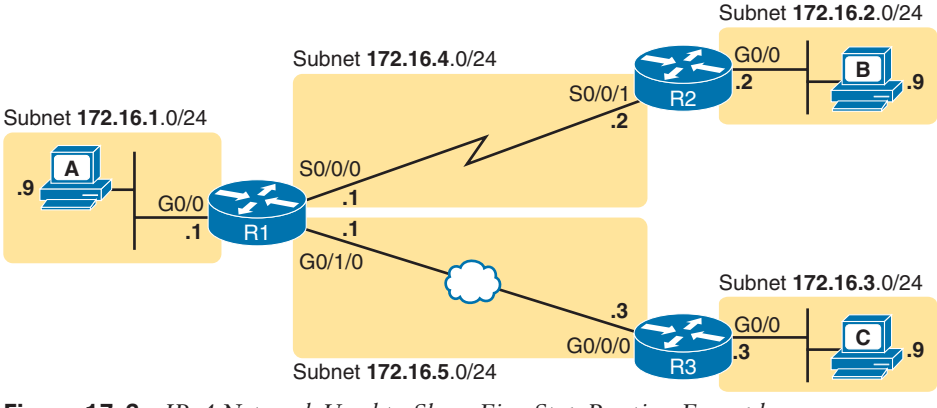



**Figure 17-3** *IPv4 Network Used to Show Five-Step Routing Example*

Now on to the example, with host A (172.16.1.9) sending a packet to host B (172.16.2.9).

### Host Forwards the IP Packet to the Default Router (Gateway)

In this example, host A uses some application that sends data to host B (172.16.2.9). After host A has the IP packet sitting in memory, host A's logic reduces to the following:

- My IP address/mask is 172.16.1.9/24, so my local subnet contains numbers 172.16.1.0–172.16.1.255 (including the subnet ID and subnet broadcast address).
- The destination address is 172.16.2.9, which is clearly not in my local subnet.
- Send the packet to my default gateway, which is set to 172.16.1.1.
- To send the packet, encapsulate it in an Ethernet frame. Make the destination MAC address be R1's G0/0 MAC address (host A's default gateway).

Figure 17-4 pulls these concepts together, showing the destination IP address and destination MAC address in the frame and packet sent by host A in this case. Note that the figure uses a common drawing convention in networking, showing an Ethernet as a few lines, hiding all the detail of the Layer 2 switches.

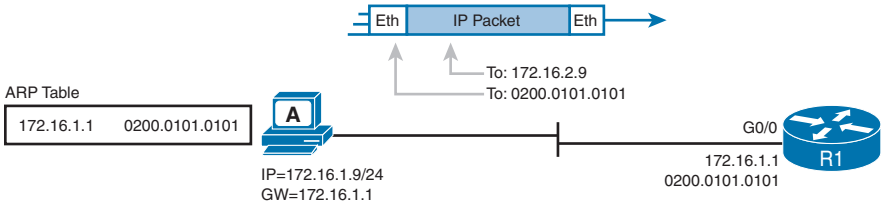



**Figure 17-4** *Host A Sends Packet Through Router R1 to Host B*

### Routing Step 1: Decide Whether to Process the Incoming Frame

Routers receive many frames in an interface, particularly LAN interfaces. However, a router can and should ignore some of those frames. So, the first step in the routing process begins with a decision of whether a router should process the frame or silently discard (ignore) the frame.

First, the router does a simple but important check (Step 1A in the process summary) so that the router ignores all frames that had bit errors during transmission. The router uses the data-link trailer's FCS field to check the frame, and if errors occurred in transmission, the router discards the frame. (The router makes no attempt at error recovery; that is, the router does not ask the sender to retransmit the data.)

The router also checks the destination data-link address (Step 1B in the summary) to decide whether the frame is intended for the router. For example, frames sent to the router's unicast MAC address for that interface are clearly sent to that router. However, a router can actually receive a frame sent to some other unicast MAC address, and routers should ignore these frames.

For example, think back to how LAN switches forward unknown unicast frames—frames for which the switch does not list the destination MAC address in the MAC address table. The LAN switch floods those frames. The result? Routers sometimes receive frames destined for

some other device, with some other device's MAC address listed as the destination MAC address. Routers should ignore those frames.

Figure 17-5 shows a working example in which host A sends a frame destined for R1's MAC address. After receiving the frame, R1 confirms with the FCS that no errors occurred, and R1 confirms that the frame is destined for R1's MAC address (0200.0101.0101 in this case). All checks have been passed, so R1 will process the frame. (Note that the large rectangle in the figure represents the internals of Router R1.)

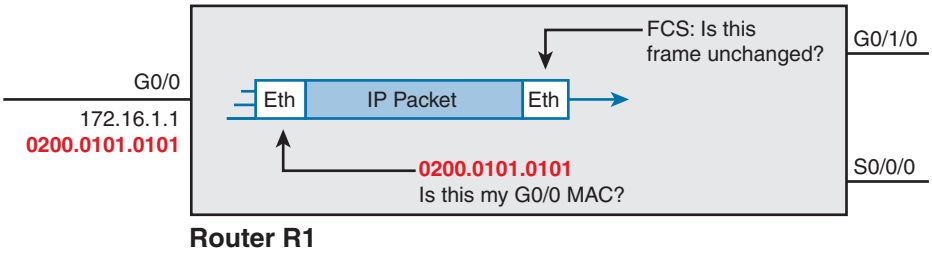



**Figure 17-5** *Routing Step 1, on Router R1: Checking FCS and Destination MAC*

## Routing Step 2: De-encapsulation of the IP Packet

After the router knows that it ought to process the received frame (per Step 1), the next step is relatively simple: de-encapsulating the packet. In router memory, the router no longer needs the original frame's data-link header and trailer, so the router removes and discards them, leaving the IP packet, as shown in Figure 17-6. Note that the destination IP address remains unchanged (172.16.2.9).

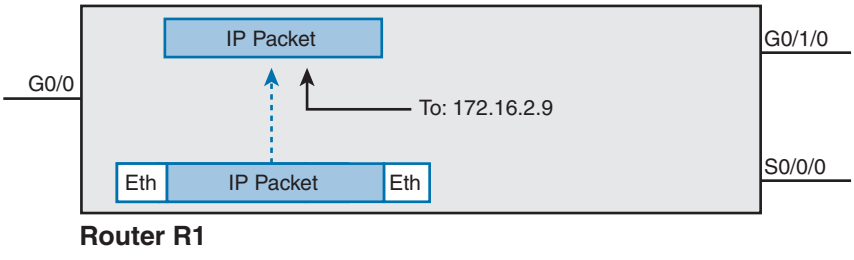



**Figure 17-6** *Routing Step 2 on Router R1: De-encapsulating the Packet*

## Routing Step 3: Choosing Where to Forward the Packet

While routing Step 2 required little thought, Step 3 requires the most thought of all the steps. At this point, the router needs to make a choice about where to forward the packet next. That process uses the router's IP routing table, with some matching logic to compare the packet's destination address with the table.

First, an IP routing table lists multiple routes. Each individual route contains several facts, which in turn can be grouped as shown in Figure 17-7. Part of each route is used to match the destination address of the packet, while the rest of the route lists forwarding instructions: where to send the packet next.

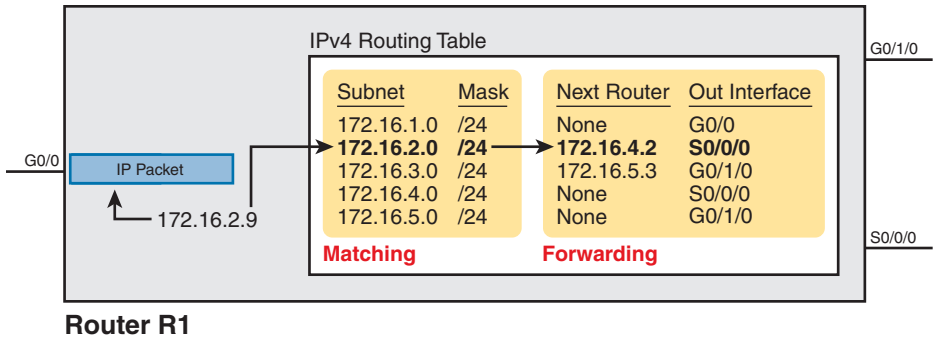



**Figure 17-7** *Routing Step 3 on Router R1: Matching the Routing Table*

Focus on the entire routing table for a moment, and notice the fact that it lists five routes. Earlier, Figure 17-3 showed the entire example network, with five subnets, so R1 has a route for each of the five subnets.

Next, look at the part of the five routes that Router R1 will use to match packets. To fully define each subnet, each route lists both the subnet ID and the subnet mask. When matching the IP packet's destination with the routing table, the router looks at the packet's destination IP address (172.16.2.9) and compares it to the range of addresses defined by each subnet. Specifically, the router looks at the subnet and mask information; with a little math, the router can figure out in which of these subnets 172.16.2.9 resides (the route for subnet 172.16.2.0/24).

Finally, look to the right side of the figure, to the forwarding instructions for these five routes. After the router matches a specific route, the router uses the forwarding information in the route to tell the router where to send the packet next. In this case, the router matched the route for subnet 172.16.2.0/24, so R1 will forward the packet out its own interface S0/0/0, to Router R2 next, listed with its next-hop router IP address of 172.16.4.2.

**NOTE** Routes for remote subnets typically list both an outgoing interface and next-hop router IP address. Routes for subnets that connect directly to the router list only the outgoing interface because packets to these destinations do not need to be sent to another router.

### Routing Step 4: Encapsulating the Packet in a New Frame

At this point, the router knows how it will forward the packet. However, routers cannot forward a packet without first wrapping a data-link header and trailer around it (encapsulation).

Encapsulating packets for serial links does not require a lot of thought, but the current CCNA exam does not require a lot from us. Point-to-point serial WAN links use either HDLC (the default) or PPP as the data-link protocol. However, we can ignore any data-link logic, even ignoring data-link addressing, because serial links have only two devices on the link: the sender and the then-obvious receiver; the data-link addressing does not matter. In this example, R1 forwards the packet out S0/0/0, after encapsulating the packet inside an HDLC frame, as shown in Figure 17-8.

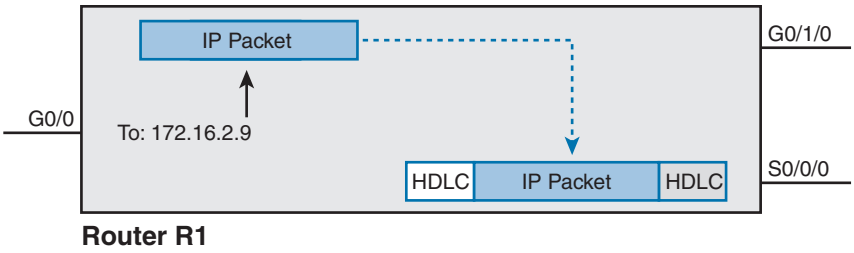



**Figure 17-8** *Routing Step 4 on Router R1: Encapsulating the Packet*

Note that with some other types of data links, the router has a little more work to do at this routing step. For example, sometimes a router forwards packets out an Ethernet interface. To encapsulate the IP packet, the router would need to build an Ethernet header, and that Ethernet header's destination MAC address would need to list the correct value.

For example, consider a packet sent by that same PC A (172.16.1.9) in Figure 17-3 but with a destination of PC C (172.16.3.9). When R1 processes the packet, R1 matches a route that tells R1 to forward the packet out R1's G0/1/0 Ethernet interface to 172.16.5.3 (R3) next. R1 needs to put R3's MAC address in the header, and to do that, R1 uses its IP ARP table information, as shown in Figure 17-9. If R1 did not have an ARP table entry for 172.16.5.3, R1 would first have to use ARP to learn the matching MAC address.

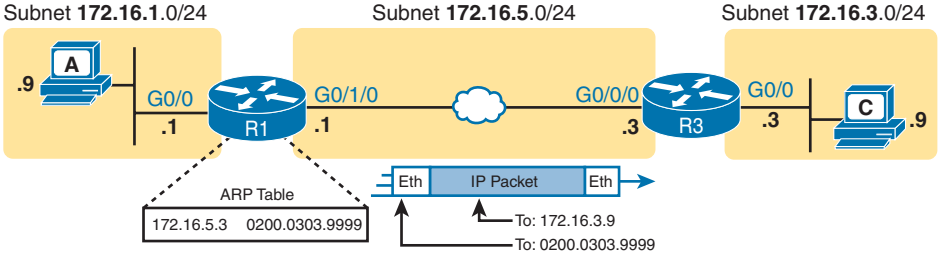



**Figure 17-9** *Routing Step 4 on Router R1 with a LAN Outgoing Interface*

### Routing Step 5: Transmitting the Frame

After the frame has been prepared, the router simply needs to transmit the frame. The router might have to wait, particularly if other frames are already waiting their turn to exit the interface.

## Configuring IP Addresses and Connected Routes

Cisco routers enable IPv4 routing globally, by default. Then, to make the router be ready to route packets on a particular interface, the interface must be configured with an IP address and the interface must be configured such that it comes up, reaching a "line status up, line protocol up" state. Only at that point can routers route IP packets in and out a particular interface.

After a router can route IP packets out one or more interfaces, the router needs some routes. Routers can add routes to their routing tables through three methods:

Key Topic

**Connected routes:** Added because of the configuration of the **ip address** interface subcommand on the local router

**Static routes:** Added because of the configuration of the **ip route** global command on the local router

**Routing protocols:** Added as a function by configuration on all routers, resulting in a process by which routers dynamically tell each other about the network so that they all learn routes

This second of three sections discusses several variations on how to configure connected routes, while the next major section discusses static routes.

## Connected Routes and the ip address Command

A Cisco router automatically adds a route to its routing table for the subnet connected to each interface, assuming that the following two facts are true:

Key Topic

- The interface is in a working state. In other words, the interface status in the **show interfaces** command lists a line status of up and a protocol status of up.
- The interface has an IP address assigned through the **ip address** interface subcommand.

The concept of connected routes is relatively basic. The router, of course, needs to know the subnet number connected to each of its interfaces, so the router can route packets to that subnet. The router does the math, taking the interface IP address and mask and calculating the subnet ID. However, the router only needs that route when the interface is up and working, so the router includes a connected route in the routing table only when the interface is working.

Example 17-1 shows the connected routes on Router R1 in Figure 17-10. The first part of the example shows the configuration of IP addresses on all three of R1's interfaces. The end of the example lists the output from the **show ip route** command, which lists these routes with a C as the route code, meaning *connected*.

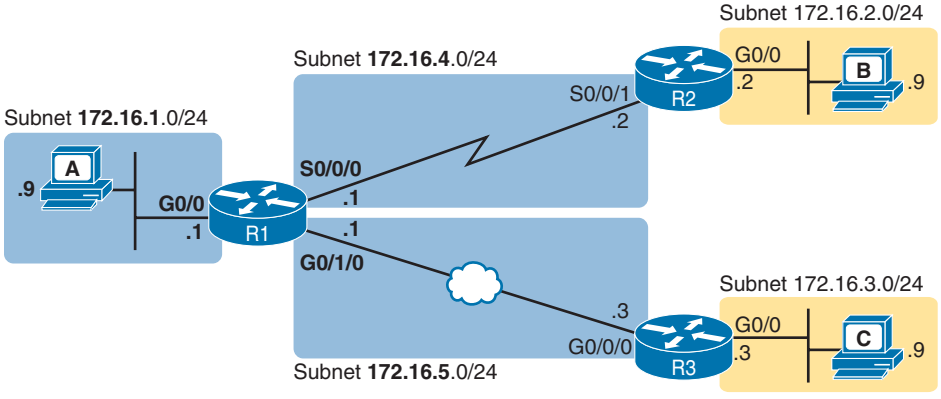



**Figure 17-10** *Sample Network to Show Connected Routes*

**Example 17-1** *Connected and Local Routes on Router R1*

```
! Excerpt from show running-config follows...
!
interface GigabitEthernet0/0
 ip address 172.16.1.1 255.255.255.0
!
interface Serial0/0/0
 ip address 172.16.4.1 255.255.255.0
!
interface GigabitEthernet0/1/0
 ip address 172.16.5.1 255.255.255.0

R1# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR

Gateway of last resort is not set

      172.16.0.0/16 is variably subnetted, 6 subnets, 2 masks
C        172.16.1.0/24 is directly connected, GigabitEthernet0/0
L        172.16.1.1/32 is directly connected, GigabitEthernet0/0
C        172.16.4.0/24 is directly connected, Serial0/0/0
L        172.16.4.1/32 is directly connected, Serial0/0/0
C        172.16.5.0/24 is directly connected, GigabitEthernet0/1/0
L        172.16.5.1/32 is directly connected, GigabitEthernet0/1/0
```

Each time you do a lab or see an example with output from the **show ip route** command, look closely at the heading line for each classful network and the indented lines that follow. The indented lines list routes to specific subnets. In Example 17-1, the heading line shows Class B network 172.16.0.0 as "172.16.0.0/16", with /16 representing the default mask for a Class B network. Why? IOS groups the output by Class A, B, or C network. In this case, the output shows a heading line that tells us that the following indented lines have to do with network 172.16.0.0/16.

Take a moment to look closely at each of the three highlighted routes below the heading line that references Class B network 172.16.0.0. Each lists a C in the first column, and each has text that says "directly connected"; both identify the route as connected to the router. The early part of each route lists the matching parameters (subnet ID and mask), as shown in the earlier example in Figure 17-7. The end of each of these routes lists the outgoing interface.

Note that the router also automatically produces a different kind of route, called a *local route*. The local routes define a route for the one specific IP address configured on the router interface. Each local route has a /32 prefix length, defining a *host route*, which defines a route just for that one IP address. For example, the last local route, for 172.16.5.1/32, defines a route that matches only the IP address of 172.16.5.1. Routers use these local routes that list their own local IP addresses to more efficiently forward packets sent to the router itself.

The **show ip route** command in the example reveals a few of the specific subitems within exam topic 3.1 (per CCNA 200-301 V1.1), with later examples in this chapter revealing even more details. This section shows details related to the following terms from the exam topics:

- **Routing Protocol Code:** The legend at the top of the **show ip route** output (about nine lines) lists all the routing protocol codes (exam topic 3.1.a). This book references the codes for connected routes (C), local (L), static (S), and OSPF (O).
- **Prefix:** The word *prefix* (exam topic 3.1.b) is just another name for subnet ID.
- **Mask:** Each route lists a prefix (subnet ID) and network mask (exam topic 3.1.c) in prefix format, for example, /24.

## Common Mistakes with the ip address Subcommand

If you follow a correct IP address plan, the **ip address** commands on your routers should be accepted. They should also work correctly to support endpoint hosts in LANs in their roles as the default gateway. However, several mistakes are possible with this command, mistakes that might not be obvious at first glance.

This next topic examines a few of those mistakes, using Figure 17-11 as a backdrop. The figure shows a more detailed view of the LAN connected to Router R1 in Figure 17-10, now with two PCs in it.

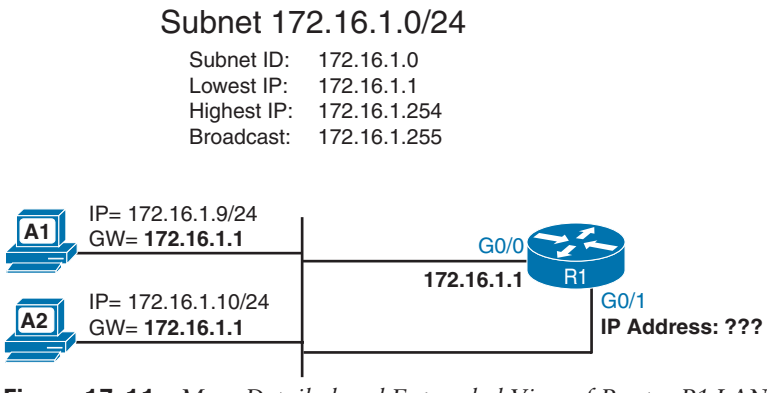



**Figure 17-11** *More Detailed and Expanded View of Router R1 LAN*

First, IOS rejects the **ip address** command in configuration mode if it uses a reserved number in the subnet, such as the subnet ID or the subnet broadcast address. Example 17-2 shows three rejected **ip address** commands. The first attempts to configure the subnet ID as the address, while the second attempts to configure the subnet broadcast address. The final

example shows the command with an invalid subnet mask (which is also rejected). Note that the error message does not reveal the specific reason, instead giving a cryptic reference to the idea that the values have a problem when using that mask.

**Example 17-2** *IOS Rejects* **ip address** *Commands with Reserved Addresses*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip address 172.16.1.0 255.255.255.0
Bad mask /24 for address 172.16.1.0
R1(config-if)# ip address 172.16.1.255 255.255.255.0
Bad mask /24 for address 172.16.1.255
R1(config-if)# ip address 172.16.1.1 255.0.255.0
Bad mask 0xFF00FF00 for address 172.16.1.1
R1(config-if)#
```

You can also configure the **ip address** command with values that IOS accepts but that are incorrect per your address plan—resulting in problems in the network. For example, looking at Figure 17-11, the default gateway setting on the two PCs implies that router R1's address should be 172.16.1.1. Router R1 would accept the **ip address 172.16.1.2 255.255.255.0** command on its G0/0 interface, setting the wrong address value—but the router has no mechanism to know that its address does not match the PC's default gateway settings. Instead, the PCs cannot communicate outside the subnet. The solution: Configure the correct address on Router R1.

As a third issue, one router may connect only one interface to a subnet. In Figure 17-11, Router R1 has two interfaces (G0/0 and G0/1) connected to the same LAN. If you attempted to assign both interfaces an IP address in that same 172.16.1.0/24 subnet, and both interfaces were in an up/up state, IOS would reject the second **ip address** command. Example 17-3 shows that sequence, with R1's G0/0 configured first and R1's G0/1 configured second (with the rejection).

**Example 17-3** *IOS Rejects* **ip address** *Command for Second Interface in the Same Subnet*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip address 172.16.1.1 255.255.255.0
R1(config)# interface gigabitEthernet 0/1
R1(config-if)# ip address 172.16.1.2 255.255.255.0
% 172.16.1.0 overlaps with GigabitEthernet0/0
R1(config-if)#
```

## The ARP Table on a Cisco Router

After a router has added these connected routes, the router can route IPv4 packets between those subnets. To do so, the router makes use of its IP ARP table.

The IPv4 **ARP table** lists the IPv4 address and matching MAC address of hosts connected to the same subnet as the router. When forwarding a packet to a host on the same subnet, the router encapsulates the packet, with a destination MAC address as found in the ARP table. If the router wants to forward a packet to an IP address on the same subnet as the router but does not find an ARP table entry for that IP address, the router will use ARP messages to learn that device's MAC address.

Example 17-4 shows R1's ARP table based on the previous example. The output lists R1's own IP address of 172.16.1.1, with an age of -, meaning that this entry does not time out. Dynamically learned ARP table entries have an upward counter, like the 35-minute value for the ARP table entry for IP address 172.16.1.9. By default, IOS will time out (remove) an ARP table entry after 240 minutes in which the entry is not used. (IOS resets the timer to 0 when an ARP table entry is used.) Note that to experiment in the lab, you might want to empty all dynamic entries (or a single entry for one IP address) using the **clear ip arp** [*ip-address*] EXEC command.

**Example 17-4** *Displaying a Router's IP ARP Table*

```
R1# show ip arp
Protocol  Address          Age (min)  Hardware Addr   Type   Interface
Internet  172.16.1.1             -   0200.0101.0101  ARPA   GigabitEthernet0/0
Internet  172.16.1.9            35   0200.aaaa.aaaa  ARPA   GigabitEthernet0/0
```

Thinking about how Router R1 forwards a packet to host A (172.16.1.9), over that final subnet, R1 does the following:

1. R1 looks in the ARP table for an entry for 172.16.1.9.
2. R1 encapsulates the IP packet in an Ethernet frame, adding destination 0200.aaaa.aaaa to the Ethernet header (as taken from the ARP table).
3. R1 transmits the frame out interface G0/0.

## Configuring Static Routes

In real networks, you will find connected routes on every router, which the routers create for subnets connected to their interfaces. You will use dynamic routing protocols like OSPF and EIGRP to learn routes for the rest of the subnets in the enterprise, routes for subnets remote from the local router.

Enterprises use static routes—that is, routes added to a routing table through direct configuration of the **ip route** command—much less often than connected routes and routes learned with dynamic routing protocols. However, static routes can be useful at times. Studying static routes also happens to be a wonderful learning tool as well. And, of course, the exam topics list several variations of static routes, so you need to learn the topic for the exam.

**NOTE** The CCNA 200-301 exam V1.1, exam topic 3.3, subdivides IPv4 (and IPv6) static routes into four subtopics: network routes, default routes, host routes, and floating static routes. This section explains all four types for IPv4 as noted in the upcoming headings.

## Static Network Routes

IOS allows the definition of individual static routes using the **ip route** global configuration command. Every **ip route** command defines a destination subnet with a subnet ID and mask. The command also lists the forwarding instructions, listing the outgoing interface or the next-hop router's IP address (or both). IOS then takes that information and adds that route to the IP routing table.

A static **network route** defines either a subnet or an entire Class A, B, or C network in the **ip route** command. In contrast, a **default route** defines the set of all destination IP addresses, while a **host route** defines a single IP address (that is, an address of one host).

Figure 17-12 shows a new network diagram to be used in the upcoming examples. Note that the design uses the same subnets as some previous figures in this chapter, but it uses two GigabitEthernet WAN links and no serial links.

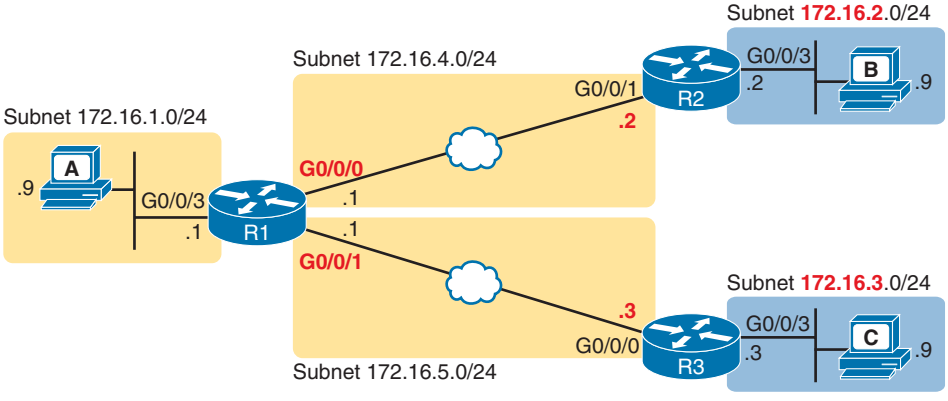



**Figure 17-12** *Sample Network Used in Static Route Configuration Examples*

Figure 17-13 shows the concepts behind the potential network routes on Router R1 for the subnet to the right of Router R2 (subnet 172.16.2.0/24). To create that static network route on R1, R1 will configure the subnet ID and mask. For the forwarding instructions, R1 will configure either R1's outgoing interface (G0/0/0) or R2's IP address as the next-hop router IP address (172.16.4.2).

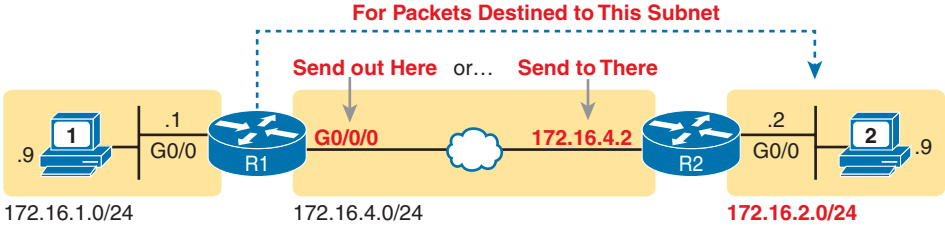



**Figure 17-13** *Static Route Configuration Concept*

Example 17-5 shows the configuration of a couple of sample static routes, using the three-router topology in Figure 17-12. The example shows static routes on R1, one route for each of the two subnets on the right side of the figure. The first command defines a route for subnet 172.16.2.0 255.255.255.0, the subnet off Router R2. The forwarding instruction parameters list only R1's outgoing interface, G0/0/0. This route basically states: To send packets to the subnet off Router R2, send them out my local G0/0/0 interface.

**Example 17-5** *Static Routes Added to R1*

```
ip route 172.16.2.0 255.255.255.0 G0/0/0
ip route 172.16.3.0 255.255.255.0 172.16.5.3
```

The second route has the same kind of logic, except listing the neighboring router's IP address on the WAN link as the next-hop router. This route basically says this: To send packets to the subnet off Router R3 (172.16.3.0 255.255.255.0), send the packets to R3's WAN IP address next (172.16.5.3).

### Verifying Static Network Routes

Static routes, when seen in the IP routing table, list the same parameters included in the configuration command. To see how, closely compare the output in Example 17-6's **show ip route static** command. This command lists only static routes, in this case listing the two static routes configured in Example 17-5. First, the two lines listing the static routes begin with legend code S, meaning static. Both list the subnet and mask defined in the respective routes, although the output uses prefix-style masks. But note the key difference: for the route configured with only the outgoing interface, the route lists only the outgoing interface, and for the route configured with only the next-hop IP address, the output lists only the next-hop address.

**Example 17-6** *Static Routes Added to R1*

```
R1# show ip route static
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
! Legend lines omitted for brevity

      172.16.0.0/16 is variably subnetted, 8 subnets, 2 masks
S        172.16.2.0/24 is directly connected, GigabitEthernet0/0/0S
S        172.16.3.0/24 [1/0] via 172.16.5.3
```

Although statically configured, IOS adds and removes static routes from the routing table over time based on whether the outgoing interface is working or not. For example, in this case, if R1's G0/0/0 interface fails, R1 removes the static route to 172.16.2.0/24 from the IPv4 routing table. Later, when the interface comes up again, IOS adds the route back to the routing table.

Examples 17-3 and 17-4 show how to configure static network routes to support left-to-right packet flow in Figure 17-12, but routers need routes for all subnets to support packet flow in all directions. Even in a small lab used for CCNA learning, if you use only static routes, you need static routes for all subnets for both directions of packet flow. For instance, in Figure 17-13, to support packet flow from PC A to PC B, you need:

A route on Router R1 for PC B's subnet 172.16.2.0/24

A route on Router R2 for PC A's subnet 172.16.1.0/24

### Ethernet Outgoing Interfaces and Proxy ARP

While both styles of static routes in Example 17-5 work—with either next-hop IP address or outgoing interface—using the next-hop address is better. Both work, but using a next-hop address causes much less confusion when the staff begin to wonder about how those static routes work behind the scenes. In short, using the next-hop address does not force the use of proxy ARP on the neighboring router, but using an outgoing interface on an Ethernet link does require proxy ARP.

This section attempts to explain how proxy ARP works and how static routes make use of it. To begin, consider this general definition of proxy ARP:

1. A router receives in interface X an ARP request, whose target is not in the subnet connected to interface X.
2. The router has a route to forward packets to that target address, and that route should not forward the packet back out onto the same interface causing a loop. In other words, the router has a useful route to forward packets to the target.
3. The router is therefore willing and useful to act as a proxy for the target host. To do so, the router supplies its own MAC address in the ARP Reply.

While true, it helps to work through an example. Consider Example 17-5 again with the **ip route 172.16.2.0 255.255.255.0 G0/0/0** command. Imagine PC A sends a packet to PC B at address 172.16.2.9. The packet arrives at Router R1, R1 matches the packet to the static route for subnet 172.16.2.0/24, and now Router R1 must decide from the forwarding instructions how to forward the packet. Router R1 uses logic as follows:

1. The route's forwarding instructions show the destination subnet as connected to interface G0/0/0. That causes R1 to send an ARP Request looking for 172.16.2.9 as the target address. In effect, R1's route makes R1 behave as if the destination subnet 172.16.2.0/24 is connected to port G0/0/0. Of course, PC B will not receive the ARP Request, as it resides on the other side of a router.
2. R2 receives R1's ARP for target 172.16.2.9. The ARP meets the requirements of proxy ARP (an address not in the local subnet, but R2 has a useful route matching address 172.16.2.9). R2 sends an ARP reply listing R2's interface G0/0/1 MAC—a proxy ARP Reply.
3. R1 receives the ARP Reply, so R1 can now forward packets meant for 172.16.2.9. R1 forwards these packets encapsulated in an Ethernet frame, with R2's G0/0/1 MAC as the destination MAC address.
4. R2 receives the frame and routes the encapsulated packet to PC B, as with normal routing processes.

As you can understand from the previous page or so, the underlying logic behind using an outgoing Ethernet interface on a route requires quite a lot of analysis. For real-world applications, use static routes with a next-hop address instead.

## Static Default Routes

When a router tries to route a packet, the router might not match the packet's destination IP address with any route. When that happens, the router normally just discards the packet.

Routers can use a *default route* to match all packets, so that if a packet does not match any other more specific route in the routing table, the router will forward the packet based on

the default route. Routers can learn default routes with a routing protocol, or you can configure a default static route.

Figure 17-14 shows one classic example in which a company has many branch office routers, like R2 and R3 in the figure, each of which has a single WAN connection. No matter how many routes a router might learn for specific subnets in those isolated branch offices, all the forwarding details in those routes would be the same for each route. For instance, all routes on Router R2, for remote subnets, would use outgoing interface G0/0/1 and next-hop address 172.16.4.1. Router R2 could instead use a default route, which matches all destinations, with the same forwarding instructions.

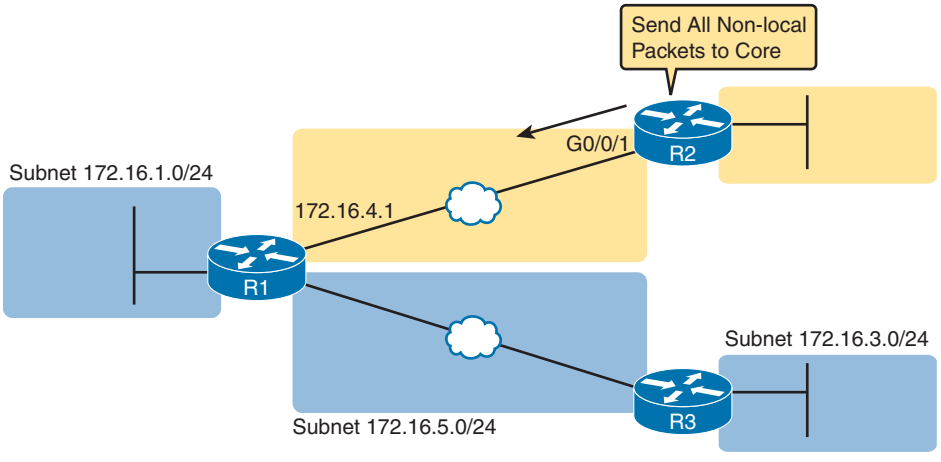



**Figure 17-14** *Example Use of Static Default Routes*

IOS allows the configuration of a static default route by using special values for the subnet and mask fields in the **ip route** command: 0.0.0.0 and 0.0.0.0. For example, the command **ip route 0.0.0.0 0.0.0.0 172.16.4.1** creates the static route depicted in Figure 17-14 on Router R2—a route that matches all IP packets—and sends those packets to R1 (172.16.4.1). Example 17-7 shows that static default route, but take care to notice the unexpected location of the route for 0.0.0.0/0.

**Example 17-7** *Adding a Static Default Route on R2 (Figure 17-14)*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# ip route 0.0.0.0 0.0.0.0 172.16.4.1
R2(config)# ^Z
R2# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
```

```
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is 172.16.4.1 to network 0.0.0.0S*

S*    0.0.0.0/0 [1/0] via 172.16.4.1
      172.16.0.0/16 is variably subnetted, 4 subnets, 2 masks
C        172.16.2.0/24 is directly connected, GigabitEthernet0/0/0
L        172.16.2.2/32 is directly connected, GigabitEthernet0/0/0
C        172.16.4.0/24 is directly connected, GigabitEthernet0/0/1
L        172.16.4.2/32 is directly connected, GigabitEthernet0/0/1
```

The output of the **show ip route** command lists a few new and interesting facts. First, it lists the route as 0.0.0.0/0, another notation for a subnet and mask of 0.0.0.0 and 0.0.0.0. The output lists a code of S, meaning static, but also with a *, meaning it is a *candidate default route*. (A router can learn about more than one default route, and the router then has to choose which one to use; the * means that it is at least a candidate to become the default route.) Just above the list of routes, the "Gateway of Last Resort" line refers to the chosen default route; in this case, R2 has only one candidate, so the output shows the just-configured static default route with next-hop address 172.16.4.1.

**NOTE** If you attempt to re-create some of the examples from the book as part of your lab practice, note that if you configure the network as shown in Figures 17-13 and 17-14, with the configuration in Examples 17-5 and 17-7, PCs A and B should be able to ping each other. As an exercise, you can think about what network or default static routes to add to R3 so that PC C can ping the other two PCs as well.



## Static Host Routes

The term *network route* refers to a route that matches addresses in an entire IP network or subnet, while a default route refers to a route that matches all destinations. In contrast, the term *host route* defines a route to a single host address. To configure such a static route, the **ip route** command uses an IP address plus a mask of 255.255.255.255 so that the matching logic matches just that one address.

Why use a static host route? Honestly, it is the least likely style of static route to use because network engineers seldom need to solve some routing problem for which a static host route will help. However, just to get the gist of what it can do, consider the topology in Figure 17-15, which expands the topology shown in the previous few figures.

First, imagine you run a dynamic routing protocol like OSPF, and Router R1 learns a route for subnet 172.16.10.0/26. Depending on the conditions of the network, R1's route for 172.16.10.0/26 may send packets to R2 next or R3 next. Most enterprises would be content in either case. The figure shows that route running through the lower router, R3.

Now imagine there was some unusual business reason such that all packets meant for host D must flow through Router R2 if it is working. The figure also shows that route, for host D only (172.16.10.4).

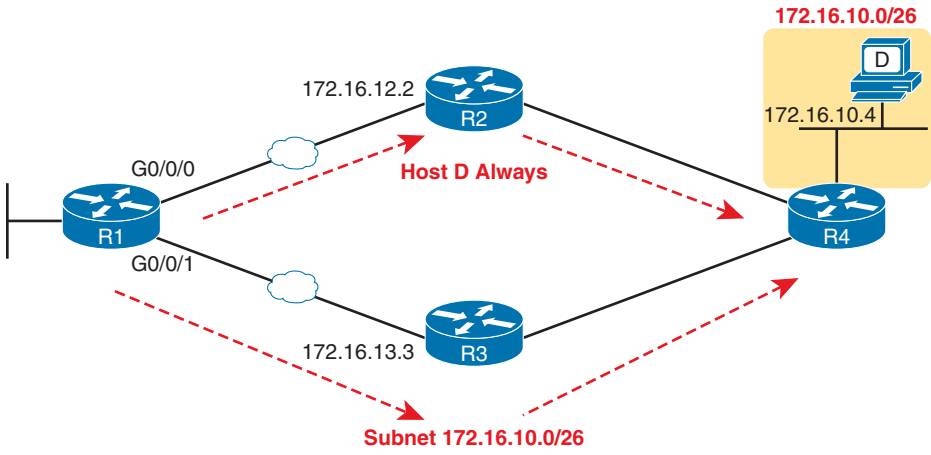



**Figure 17-15** *An Example Possible Use for a Static Host Route*

The preceding scenario creates a need for a static host route. For instance, all the routers use OSPF to dynamically learn network routes for all subnets in the design. However, the engineer also configures the one host route on R1 as discussed in the last few paragraphs, as seen at the top of Example 17-8.

**Example 17-8** *Host Route Configuration Versus an Overlapping Static Network Route*

```
R1# configure terminal
! The static host route for one IP address within that same subnet
R1(config)# ip route 172.16.10.4 255.255.255.255 172.16.4.2
R1(config)# ^Z
R1# show ip route
! Irrelevant portions omitted for brevity

      172.16.0.0/16 is variably subnetted, 12 subnets, 4 masks
O IA     172.16.10.0/26 [110/3] via 172.16.5.3, 01:52:58, GigabitEthernet0/0/1
S        172.16.10.4/32 [1/0] via 172.16.4.2
```

Note that it is normal for a host route to overlap with some other network route as seen here. A router always attempts to place the best route for each prefix (subnet) into the IP routing table. However, the host route uses a mask of 255.255.255.255 (/32), and the dynamic route for the subnet that address resides in uses a different mask. Router R1 does not view those routes as routes for the same subnet, but as different routes, and places both into its routing table.

Once in the routing table, when a packet arrives for that one special host address (172.16.10.4), R1 uses the host route. Why? Routers use the most specific route (that is, the route with the longest prefix length) when the packet's destination address matches multiple routes in the routing table. So, a packet sent to 172.16.10.4 would match the host route because a /32 mask is longer and more specific than the route with a /26 mask. Router R1 will forward that packet to next-hop router 172.16.4.2 per the host route. Packets sent to other destinations in subnet 172.16.10.0/26 would be sent to next-hop router 172.16.5.3, because those match only one route.

Note that the section "IP Forwarding with the Longest Prefix Match" near the end of Chapter 25, "Fundamentals of IP Version 6," gets into more detail about how a router chooses a route when multiple routes match a packet destination.

## Floating Static Routes

**Floating static routes** exist in the configuration of a router, but the router floats the route into the routing table and out of the routing table based on certain conditions. Why? As part of a broader configuration that lets the router create a temporary WAN link when the primary WAN link fails.

To see how that works, consider the example illustrated in Figure 17-16, which shows routers R1 and R2 from the previous few figures. The Ethernet WAN link serves as the only working WAN link most of the time, but now both routers have a cellular interface. The routers can connect to each other using a 4G/LTE/5G network when the primary link goes down.

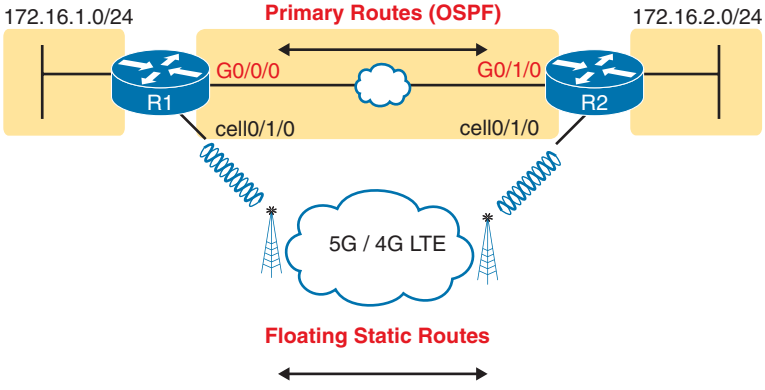



**Figure 17-16** *Using a Floating Static Route to Key Subnet 172.16.2.0/24*

Interestingly, the configuration to use the cellular network to back up the Ethernet WAN link hinges on the routes. Routers use a routing protocol like OSPF on the permanent links, so in this case, R1 learns an OSPF route to subnet 172.16.2.0/24 that uses the Ethernet WAN link. R1 also defines a floating static route for that same subnet that directs packets over the cellular interface to Router R2. Although configured, Router R1 then chooses to add that floating static route to the routing table *only when the OSPF route is not available*. If the Ethernet link is up, R1 routes packets over that link. If not, it routes packets over the cellular network.

To create a floating static route, you configure a route but assign it a higher (worse) administrative distance than the competing dynamically learned route. When a router learns of more than one route to the same subnet/mask combination, the router must first decide which routing source has the better **administrative distance**, with lower being better, and then use the route learned from the better source. Default settings make routers treat static routes as better than any routes learned by a dynamic routing protocol. A floating static route reverses that logic.

To implement a floating static route, you need to use a parameter on the **ip route** command that sets the administrative distance for just that route, making the value larger than the

default administrative distance of the routing protocol. For example, the **ip route 172.16.2.0 255.255.255.0 cell0/1/0 130** command on R1 (in Figure 17-15) would do exactly that—setting the static route's administrative distance to 130. As long as the primary link stays up, and OSPF on R1 learns a route for 172.16.2.0/24, with a default OSPF administrative distance of 110, R1 ignores the floating static route. When the Ethernet WAN link fails, R1 loses its OSPF-learned route for that subnet, so R1 places the floating static route into the routing table. (The rest of the configuration, not shown here, tells the router how to make the call to create the backup link over the cellular network.)

Finally, note that while the **show ip route** command lists the administrative distance of most routes, as the first of two numbers inside two brackets, the **show ip route** *subnet* command plainly lists the administrative distance. Example 17-9 shows a sample, matching this most recent example.

**Example 17-9** *Displaying the Administrative Distance of the Static Route*

```
R1# show ip route static
! Legend omitted for brevity
       172.16.0.0/16 is variably subnetted, 6 subnets, 2 masks
S         172.16.2.0/24 is directly connected, Cellular0/1/0

R1# show ip route 172.16.2.0
Routing entry for 172.16.2.0/24
  Known via "static", distance 130, metric 0 (connected)
  Routing Descriptor Blocks:
  * directly connected, via Cellular0/1/0
      Route metric is 0, traffic share count is 1
```

## Troubleshooting Static Routes

These final few pages about IPv4 static routes examine some issues that can occur with static routes, both reviewing some reasons mentioned over the last few pages, while adding more detail. This topic breaks static route troubleshooting into three perspectives:

- The route is in the routing table but is incorrect.
- The route is not in the routing table.
- The route is in the routing table and is correct, but the packets do not arrive at the destination host.

### Incorrect Static Routes That Appear in the IP Routing Table

This first troubleshooting item can be obvious, but it is worth pausing to think about. A static route is only as good as the input typed into the **ip route** command. IOS checks the syntax, and as mentioned earlier, makes a few other checks that this section reviews in the next heading. But once those checks are passed, IOS puts the route into the IP routing table, even if the route had poorly chosen parameters.

For instance, if you wanted to configure a static route for subnet 172.16.2.0/24, but configured the route for subnet 172.16.222.0/24, the router will accept the command—but it is for the wrong subnet. Or, when choosing the next-hop router address, you looked at a network diagram, and used the next-hop address of a router to the left—but the destination subnet

was on the right side of the figure. Or you could look at documentation and choose the next-hop address for the command, and it is in the correct destination subnet—but it is not the address of the next-hop router. In all these cases, the router would accept the command and even add the route to the routing table, but the route would not be useful.

As another example, IOS will reject the **ip route** command if the first two parameters reveal the configuration of an IP address within the subnet rather than the subnet ID. For instance, back in Example 17-5, the commands used mask 255.255.255.0, with subnet IDs 172.16.2.0 and 172.16.3.0. Example 17-10 shows a repeat of those same commands, but now with addresses from within those subnets: 172.16.2.1 and 172.16.3.1. In both cases, IOS shows an error message and does not add the command to the configuration.

**Example 17-10** *IOS Rejects* **ip route** *Commands That Use an Address in the Subnet*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1-8200(config)# ip route 172.16.2.1 255.255.255.0 G0/0/0
%Inconsistent address and mask
R1-8200(config)# ip route 172.16.3.1 255.255.255.0 172.16.5.3
%Inconsistent address and mask
R1-8200(config)#
```

When you see an exam question that has static routes, and you see them in the output of **show ip route**, remember to check on these items:



Key Topic

- Are the subnet ID and mask correct?
- Is the next-hop IP address correct and referencing an IP address on a neighboring router?
- Does the next-hop IP address identify the correct router?
- Is the outgoing interface correct and referencing an interface on the local router (that is, the same router where the static route is configured)?

### The Static Route Does Not Appear in the IP Routing Table

After configuring an **ip route** command, IOS might or might not add the route to the IP routing table. IOS also considers the following before adding the route to its routing table:

Key Topic

- For **ip route** commands that list an outgoing interface, that interface must be in an up/up state.
- For **ip route** commands that list a next-hop IP address, the local router must have a route to reach that next-hop address.

For example, earlier in Example 17-5, R1's command **ip route 172.16.3.0 255.255.255.0 172.16.5.3** defines a static route. Before adding the route to the IP routing table, R1 looks for an existing IP route to reach 172.16.5.3. In that case, R1 will find a connected route for subnet 172.16.5.0/24 as long as its Ethernet WAN link is up. As a result, R1 adds the static route to subnet 172.16.3.0/24. Later, if R1's G0/0/1 were to fail, R1 would remove its connected route to 172.16.5.0/24 from the IP routing table—an action that would also then cause R1 to remove its static route to 172.16.3.0/24.

You can configure a static route so that IOS ignores these basic checks, always putting the IP route in the routing table. To do so, just use the **permanent** keyword on the **ip route** command. For example, if you add the **permanent** keyword to the end of the two commands as demonstrated in Example 17-11, R1 would now add these routes, regardless of whether the two WAN links were up.

**Example 17-11** *Permanently Adding Static Routes to the IP Routing Table (Router R1)*

```
ip route 172.16.2.0 255.255.255.0 G0/0/0 permanent
ip route 172.16.3.0 255.255.255.0 172.16.5.3 permanent
```

Note that although the **permanent** keyword lets the router keep the route in the routing table without checking the outgoing interface or route to the next-hop address, it does not magically fix a broken route. For example, if the outgoing interface fails, the route will remain in the routing table, but the router cannot forward packets because the outgoing interface is down.

### The Correct Static Route Appears but Works Poorly

This last section is a place to make two points—one mainstream and one point to review a bit of trivia.

First, on the mainstream point, the static route can be perfect, but the packets might not arrive because of other problems. An incorrect static route is just one of many items to check when you're troubleshooting problems like "host A cannot connect to server B." The root cause may be the static route, or it may be something else. Chapter 20, "Troubleshooting IPv4 Routing," goes into some depth about troubleshooting these types of problems.

On the more specific point, be wary of any **ip route** command with the **permanent** keyword. IOS puts these routes in the routing table with no checks for accuracy. You should check whether the outgoing interface is down and/or whether the router has a route to reach the next-hop address.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 17-2 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 17-2** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 17-3** Key Topics for Chapter 17

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Steps taken by a host when forwarding IP packets | 429 |
| List | Steps taken by a router when forwarding IP packets | 430 |
| Figure 17-2 | Diagram of five routing steps taken by a router | 430 |
| Figure 17-7 | Breakdown of IP routing table with matching and forwarding details | 434 |
| List | Three common sources from which routers build IP routes | 435 |
| List | Rules regarding when a router creates a connected route | 436 |
| Figure 17-13 | Static route configuration concept | 441 |
| List | Troubleshooting checklist for routes that do appear in the IP routing table | 449 |
| List | Troubleshooting checklist for static routes that do not appear in the IP routing table | 449 |

## Key Terms You Should Know

administrative distance, ARP table, connected route, default route, floating static route, host route, network route, next-hop router, outgoing interface, routing table, static route



## Command References

Tables 17-4 and 17-5 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 17-4** Chapter 17 Configuration Command Reference

| Command | Description |
|---|---|
| **ip address** *ip-address mask* | Interface subcommand that assigns the interface's IP address |
| **interface** *type number.subint* | Global command to create a subinterface and to enter configuration mode for that subinterface |
| [**no**] **ip routing** | Global command that enables (**ip routing**) or disables (**no ip routing**) the routing of IPv4 packets on a router or Layer 3 switch |
| **ip route** *prefix mask* {*ip-address* \| *interface-type interface-number*} [*distance*] [**permanent**] | Global configuration command that creates a static route |

**Table 17-5** Chapter 17 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip route** | Lists the router's entire routing table |
| **show ip route [connected \| static \| ospf]** | Lists a subset of the IP routing table |
| **show ip route** *ip-address* | Lists detailed information about the route that a router matches for the listed IP address |
| **show arp, show ip arp** | Lists the router's IPv4 ARP table |
| **clear ip arp [ip-address]** | Removes all dynamically learned ARP table entries, or if the command lists an IP address, removes the entry for that IP address only |

*This page intentionally left blank*



CHAPTER 18

# IP Routing in the LAN

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.1 Explain the role and function of network components**
    - **1.1.a Routers**
    - **1.1.b Layer 2 and Layer 3 switches**
  - **1.6 Configure and verify IPv4 addressing and subnetting**
- **2.0 Network Access**
  - **2.1 Configure and verify VLANs (normal range) spanning multiple switches**
    - **2.1.c InterVLAN connectivity**
  - **2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP)**

The preceding two chapters showed how to configure an IP address and mask on a router interface, making the router ready to route packets to/from the subnet implied by that address/mask combination. While true and useful, all the examples so far ignored the LAN switches and the possibility of VLANs. In fact, the examples so far show the simplest possible cases: the attached switches as Layer 2 switches, using only one VLAN, with the router configured with one **ip address** command on its physical interface. This chapter takes a detailed look at how to configure routers so that they route packets to/from the subnets that exist on each and every VLAN.

Because Layer 2 switches do not forward Layer 2 frames between VLANs, a network must use a device that performs IP routing to route IP packets between subnets. That device can be a router, or it can be a switch that also includes routing features.

To review, Ethernet defines the concept of a VLAN, while IP defines the concept of an IP subnet, so a VLAN is not equivalent to a subnet. However, the devices in one VLAN typically use IP addresses in one subnet. By the same reasoning, devices in two different VLANs are normally in two different subnets. For two devices in different VLANs to communicate with each other, routers must connect to the subnets that exist on each VLAN, and then the routers forward IP packets between the devices in those subnets.

This chapter discusses multiple methods of routing between VLANs, all of which can be useful in different scenarios:

- **VLAN Routing with Router 802.1Q Trunks:** The first section discusses configuring a router to use VLAN trunking on a link connected to a Layer 2 switch. The router does the routing, with the switch creating the VLANs. The link between the router and switch uses VLAN trunking so that the router has an interface connected to each

VLAN/subnet. This feature is known as routing over a VLAN trunk and also known as **router-on-a-stick (ROAS)**.

- **VLAN Routing with Layer 3 Switch SVIs:** The second section discusses using a LAN switch that supports both Layer 2 switching and Layer 3 routing (called a **Layer 3 switch** or **multilayer switch**). To route, the Layer 3 switch configuration uses interfaces called switched virtual interfaces (SVIs), which are also called VLAN interfaces.
- **VLAN Routing with Layer 3 Switch Routed Ports:** The third major section of the chapter discusses an alternative to SVIs called **routed ports**, in which the physical switch ports are made to act like interfaces on a router. This third section also introduces the concept of an EtherChannel as used as a routed port in a feature called **Layer 3 EtherChannel (L3 EtherChannel)**.
- **VLAN Routing on a Router's LAN Switch Ports:** The final major section discusses Cisco routers that include integrated LAN switch ports. For instance, rather than installing a small router plus a separate small switch at a branch office, you could install a router that has a small set of integrated switch ports. This section shows how to configure the router's switch ports and the internal logic, so the router routes packets for the subnets on those switch ports.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 18-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| VLAN Routing with Router 802.1Q Trunks | 1, 2 |
| VLAN Routing with Layer 3 Switch SVIs | 3, 4 |
| VLAN Routing with Layer 3 Switch Routed Ports | 5, 6 |
| VLAN Routing on a Router's LAN Switch Ports | 7 |

1. Router 1 has a Fast Ethernet interface 0/0 with IP address 10.1.1.1. The interface is connected to a switch. This connection is then migrated to use 802.1Q trunking. Which of the following commands could be part of a valid configuration for Router 1's Fa0/0 interface, assuming the trunk uses VLAN 4 as one of the supported VLANs? (Choose two answers.)
   a. interface fastethernet 0/0.4
   b. dot1q enable
   c. dot1q enable 4
   d. trunking enable
   e. trunking enable 4
   f. encapsulation dot1q 4

2. Router R1 has a router-on-a-stick (ROAS) configuration with two subinterfaces of interface G0/1: G0/1.1 and G0/1.2. Physical interface G0/1 is currently in a down/down state. The network engineer then configures a **shutdown** command when in interface configuration mode for G0/1.1 and a **no shutdown** command when in interface configuration mode for G0/1.2. Which answers are correct about the interface state for the subinterfaces? (Choose two answers.)
   a. G0/1.1 will be in a down/down state.
   b. G0/1.2 will be in a down/down state.
   c. G0/1.1 will be in an administratively down state.
   d. G0/1.2 will be in an up/up state.
3. A Layer 3 switch has been configured to route IP packets between VLANs 1, 2, and 3 using SVIs, which connect to subnets 172.20.1.0/25, 172.20.2.0/25, and 172.20.3.0/25, respectively. The engineer issues a **show ip route connected** command on the Layer 3 switch, listing the connected routes. Which of the following answers lists a piece of information that should be in at least one of the routes?
   a. Interface Gigabit Ethernet 0/0.3
   b. Next-hop router 172.20.2.1
   c. Interface VLAN 2
   d. Mask 255.255.255.0
4. An engineer has successfully configured a Layer 3 switch with SVIs for VLANs 2 and 3 with autostate enabled by default. Hosts in the subnets using VLANs 2 and 3 can ping each other with the Layer 3 switch routing the packets. The next week, the network engineer receives a call that those same users can no longer ping each other. If the problem is with the Layer 3 switching function, which of the following could have caused the problem? (Choose two answers.)
   a. Six (or more) out of ten working VLAN 2 access ports failing due to physical problems
   b. A **shutdown** command issued from interface VLAN 4 configuration mode
   c. A **no vlan 2** command issued from global configuration mode
   d. A **shutdown** command issued from VLAN 2 configuration mode
5. A LAN design uses a Layer 3 EtherChannel between two switches SW1 and SW2, with port-channel interface 1 used on both switches. SW1 uses ports G0/1, G0/2, G0/3, and G0/4 in the channel. Which of the following is true about SW1's configuration to enable the channel to route IPv4 packets correctly? (Choose two answers.)
   a. The **ip address** command must be on the port-channel 1 interface.
   b. The **ip address** command must be on interface G0/1 (lowest numbered port).
   c. The port-channel 1 interface must be configured with the **no switchport** command.
   d. Interface G0/1 must be configured with the **routedport** command.

6. A LAN design uses a Layer 3 EtherChannel between two switches: SW1 and SW2. An engineer adds SW1 port G0/1 to the channel successfully but later fails when adding SW1 port G0/2 to the channel. Which answers list a configuration setting on port G0/2 that would cause this issue? (Choose two answers.)
   a. A different STP cost (**spanning-tree cost** *value*)
   b. A different speed (**speed** *value*)
   c. A default setting for switchport (**switchport**)
   d. A different access VLAN (**switchport access vlan** *vlan-id*)
7. A router has some routed and some switched physical ports. Which interface subcommands would you expect to be supported only on the switched ports?
   a. The **switchport access vlan** *vlan-id* subcommand
   b. The **ip address** *address mask* subcommand
   c. The **description** *text* subcommand
   d. The **hostname** *name* subcommand

## Foundation Topics

# VLAN Routing with Router 802.1Q Trunks

Almost all enterprise networks use VLANs. To route IP packets in and out of those VLANs, some devices (either routers or Layer 3 switches) need to have an IP address in each subnet and have a connected route to each of those subnets. Then the IP addresses on those routers or Layer 3 switches can serve as the default gateways in those subnets.

This chapter breaks down the LAN routing options into five categories:

- Use a router, with one router LAN interface and cable connected to the switch for each and every VLAN (typically not used)
- Use a router, with a VLAN trunk connecting to a LAN switch (known as router-on-a-stick, or ROAS)
- Use a Layer 3 switch with switched virtual interfaces (SVIs)
- Use a Layer 3 switch with routed interfaces (which may or may not be Layer 3 EtherChannels)
- Use a router with integrated switch ports, configuring it much like a Layer 3 switch with SVIs

Of the items in the list, the first option works, but to be practical, it requires far too many interfaces. It is mentioned here only to make the list complete.

As for the other options, this chapter discusses each in turn. Real networks use these features, with the choice to use one or the other driven by the design and needs for a particular part of the network. Figure 18-1 shows cases in which these options could be used.

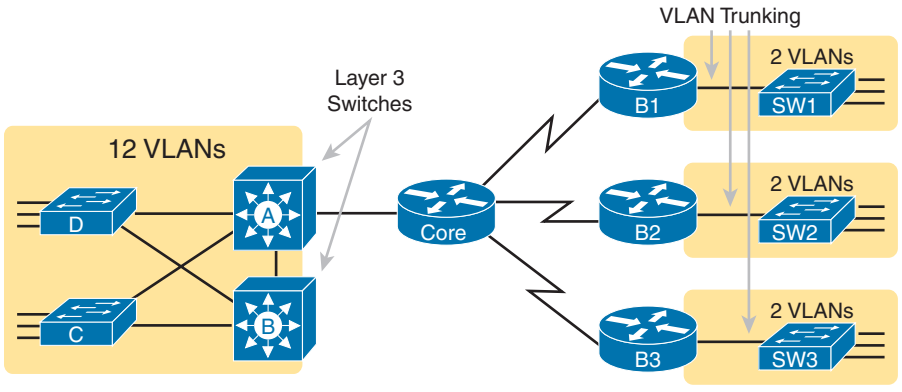



**Figure 18-1** *Layer 3 Switching at the Central Site, ROAS at Branch Offices*

Figure 18-1 shows two switches, labeled A and B, which could act as Layer 3 switches—both with SVIs and routed interfaces. The figure shows a central site campus LAN on the left, with 12 VLANs. At the central site, two of the switches act as Layer 3 switches, combining the functions of a router and a switch, routing between all 12 subnets/VLANs. Those Layer 3 switches could use SVIs, routed interfaces, or both.

Figure 18-1 also shows a classic case for using a router with a VLAN trunk. Sites like the remote sites on the right side of the figure may have a WAN-connected router and a LAN switch. These sites might use ROAS to take advantage of the router's ability to route over an 802.1Q trunk.

Note that Figure 18-1 just shows an example. The engineer could use Layer 3 switching at each site or routers with VLAN trunking at each site.

## Configuring ROAS

This next topic discusses how routers route packets to subnets associated with VLANs connected to a router 802.1Q trunk. That long description can be a bit of a chore to repeat each time someone wants to discuss this feature, so over time, the networking world has instead settled on a shorter and more interesting name for this feature: router-on-a-stick (ROAS).

ROAS uses router VLAN trunking configuration to give the router a logical interface connected to each VLAN. Because the router then has an interface connected to each VLAN, the router can also be configured with an IP address in the subnet that exists on each VLAN.

The router needs to have an IP address/mask associated with each VLAN on the trunk. However, the router has only one physical interface for the link connected to the trunk. Cisco solves this problem by creating multiple virtual router interfaces, one for each supported VLAN on that trunk. Cisco calls these virtual interfaces **subinterfaces**, and the router configuration includes an **ip address** command for each subinterface.

Figure 18-2 shows the concept with Router B1, one of the branch routers from Figure 18-1. Because this router needs to route between only two VLANs, the figure also shows two

---

Answers to the "Do I Know This Already?" quiz:

**1** A, F **2** B, C **3** C **4** C, D **5** A, C **6** B, C **7** A

subinterfaces, named G0/0/0.10 and G0/0/0.20, which create a new place in the configuration where the per-VLAN configuration settings can be made. The router treats frames tagged with VLAN 10 as if they came in or out of G0/0/0.10 and frames tagged with VLAN 20 as if they came in or out G0/0/0.20.

Key Topic

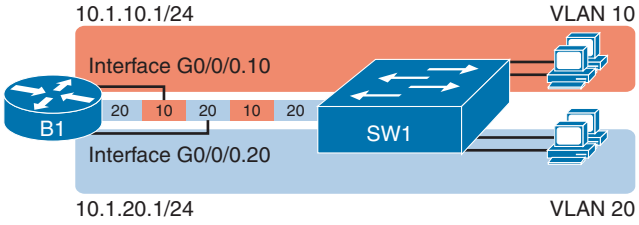



**Figure 18-2** *Subinterfaces on Router B1*

In addition, note that most Cisco routers do not attempt to negotiate trunking, so both the router and switch need to manually configure trunking. This chapter discusses the router side of that trunking configuration; the matching switch interface would need to be configured with the **switchport mode trunk** command.

Example 18-1 shows a full example of the 802.1Q trunking configuration required on Router B1 in Figure 18-2. More generally, these steps detail how to configure 802.1Q trunking on a router:

Config Checklist

**Step 1.** Use the **interface** *type number.subint* command in global configuration mode to create a unique subinterface for each VLAN that needs to be routed.

**Step 2.** Use the **encapsulation dot1q** *vlan_id* command in subinterface configuration mode to enable 802.1Q and associate one specific VLAN with the subinterface.

**Step 3.** Use the **ip address** *address mask* command in subinterface configuration mode to configure IP settings (address and mask).

18

**Example 18-1** *Router 802.1Q Configuration per Figure 18-2*

```
B1# show running-config
! Only pertinent lines shown
interface gigabitethernet 0/0/0
! No IP address or encapsulation up here!
!
interface gigabitethernet 0/0/0.10
 encapsulation dot1q 10
 ip address 10.1.10.1 255.255.255.0
!
interface gigabitethernet 0/0/0.20
 encapsulation dot1q 20
 ip address 10.1.20.1 255.255.255.0
```

First, look at the subinterface numbers. The subinterface number begins with the period, like .10 and .20 in this case. These numbers can be any number from 1 up through a very

large number (over 4 billion). The number just needs to be unique among all subinterfaces associated with this one physical interface. In fact, the subinterface number does not even have to match the associated VLAN ID. (The **encapsulation** command, and not the subinterface number, defines the VLAN ID associated with the subinterface.)

**NOTE** Although not required, most sites do choose to make the subinterface number match the VLAN ID, as shown in Example 18-1, just to avoid confusion.

Each subinterface configuration lists two subcommands. One command (**encapsulation**) enables trunking and defines the VLAN the router associates with the frames entering and exiting the subinterface. The **ip address** command works the same way it does on any other router interface.

Now that the router has a working interface, with IPv4 addresses configured, the router can route IPv4 packets on these subinterfaces. That is, the router treats these subinterfaces like any physical interface in terms of adding connected routes, matching those routes, and forwarding packets to/from those connected subnets.

Example 18-1 uses two VLANs, 10 and 20, neither of which is the native VLAN on the trunk. ROAS can make use of the native VLAN, with two variations in the configuration, so it requires a little extra thought. The native VLAN can be configured on a subinterface, or on the physical interface, or ignored as in Example 18-1. Each 802.1Q trunk has one native VLAN, and if the router needs to route packets for a subnet that exists in the native VLAN, then the router needs some configuration to support that subnet. The two options to define a router interface for the native VLAN are



- Configure the **ip address** command on the physical interface, but without an **encapsulation** command; the router considers this physical interface to be using the native VLAN.
- Configure the **ip address** command on a subinterface and use the **encapsulation dot1q** *vlan-id* **native** subcommand to tell the router both the VLAN ID and the fact that it is the native VLAN.

Example 18-2 shows both native VLAN configuration options with a small change to the same configuration in Example 18-1. In this case, VLAN 10 becomes the native VLAN. The top part of the example shows the option to configure the router physical interface to use native VLAN 10. The second half of the example shows how to configure that same native VLAN on a subinterface. In both cases, the switch configuration also needs to be changed to make VLAN 10 the native VLAN.

**Example 18-2** *Two Configuration Options for Native VLAN 10 on Router B1*

```
! First option: put the native VLAN IP address on the physical interface
interface gigabitethernet 0/0/0
 ip address 10.1.10.1 255.255.255.0
!
interface gigabitethernet 0/0/0.20
 encapsulation dot1q 20
 ip address 10.1.20.1 255.255.255.0
```

```
! Second option: like Example 18-1, but add the native keyword
interface gigabitethernet 0/0/0.10
 encapsulation dot1q 10 native
 ip address 10.1.10.1 255.255.255.0
!
interface gigabitethernet 0/0/0.20
 encapsulation dot1q 20
 ip address 10.1.20.1 255.255.255.0
```

## Verifying ROAS

Beyond using the **show running-config** command, ROAS configuration on a router can be best verified with two commands: **show ip route** [**connected**] and **show vlans**. As with any router interface, as long as the interface is in an up/up state and has an IPv4 address configured, IOS will put a connected (and local) route in the IPv4 routing table. So, a first and obvious check would be to see if all the expected connected routes exist. Example 18-3 lists the connected routes per the configuration shown in Example 18-1.

**Example 18-3** *Connected Routes Based on Example 18-1 Configuration*

```
B1# show ip route connected
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
! Legend omitted for brevity

      10.0.0.0/8 is variably subnetted, 4 subnets, 2 masks
C        10.1.10.0/24 is directly connected, GigabitEthernet0/0/0.10
L        10.1.10.1/32 is directly connected, GigabitEthernet0/0/0.10
C        10.1.20.0/24 is directly connected, GigabitEthernet0/0/0.20
L        10.1.20.1/32 is directly connected, GigabitEthernet0/0/0.20
```

As for interface and subinterface state, note that the ROAS subinterface state does depend to some degree on the physical interface state. In particular, the subinterface state cannot be better than the state of the matching physical interface. For instance, so far, the physical interface and related subinterfaces on Router B1 rested in an up/up state. If you unplugged the cable from that physical port, the physical and subinterfaces all fail to a down/down state. Or, if you shut down the physical interface, the physical and subinterfaces move to an administratively down state, as seen in Example 18-4.

**Example 18-4** *Subinterface State Tied to Physical Interface State*

```
B1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
B1(config)# interface g0/0/0
B1(config-if)# shutdown
B1(config-if)# ^Z
B1# show ip interface brief | include 0/0/0
GigabitEthernet0/0/0         unassigned      YES manual administratively down down
GigabitEthernet0/0/0.10      10.1.10.1       YES manual administratively down down
GigabitEthernet0/0/0.20      10.1.20.1       YES manual administratively down down
```

Additionally, the subinterface state can also be enabled and disabled independently from the physical interface, using the **no shutdown** and **shutdown** commands in subinterface configuration mode. For instance, the physical interface and subinterface .10 can remain up/up, while subinterface .20 can be independently shut down.

Another useful ROAS verification command, **show vlans**, spells out which router trunk interfaces use which VLANs, which VLAN is the native VLAN, plus some packet statistics. The fact that the packet counters are increasing can be useful when verifying whether traffic is happening or not. Example 18-5 shows a sample, based on the Router B1 configuration in Example 18-2 (bottom half), in which native VLAN 10 is configured on subinterface G0/0/0.10. Note that the output identifies VLAN 1 associated with the physical interface, VLAN 10 as the native VLAN associated with G0/0/0.10, and VLAN 20 associated with G0/0/0.20. It also lists the IP addresses assigned to each interface/subinterface.

**Example 18-5** *Router* **show vlans** *Command Reveals Router ROAS Configuration*

```
B1# show vlans
VLAN ID: 1 (IEEE 802.1Q Encapsulation)

   Protocols Configured:          Received:        Transmitted:

VLAN trunk interfaces for VLAN ID 1:
GigabitEthernet0/0/0

GigabitEthernet0/0/0 (1)
      Total 5 packets, 330 bytes input
      Total 20 packets, 3134 bytes output

VLAN ID: 10 (IEEE 802.1Q Encapsulation)

 This is configured as native Vlan for the following interface(s) :
GigabitEthernet0/0/0    Native-vlan Tx-type: Untagged

   Protocols Configured:          Received:        Transmitted:
                     IP                  0                   0

VLAN trunk interfaces for VLAN ID 10:
GigabitEthernet0/0/0.10

GigabitEthernet0/0/0.10 (10)
                     IP: 10.1.10.1
      Total 38 packets, 5696 bytes input
      Total 2 packets, 128 bytes output

VLAN ID: 20 (IEEE 802.1Q Encapsulation)
```

```
   Protocols Configured:          Received:        Transmitted:
                        IP                   0                   0

VLAN trunk interfaces for VLAN ID 20:
GigabitEthernet0/0/0.20

GigabitEthernet0/0/0.20 (20)
                        IP: 10.1.20.1
      Total 0 packets, 0 bytes input
      Total 2 packets, 128 bytes output
```

## Troubleshooting ROAS

The biggest challenge when troubleshooting ROAS has to do with the fact that if you misconfigure only the router or misconfigure only the switch, the other device on the trunk has no way to know that the other side is misconfigured. That is, the router configuration might be correct, but routing might still fail because of problems on the attached switch. So, troubleshooting ROAS often begins with checking the configuration on both the router and switch because there is no status output on either device that tells you where the problem might be.

First, to check ROAS on the router, you need to start with the intended configuration and ask questions about the configuration:

Key Topic

1. Is each non-native VLAN configured on the router with an **encapsulation dot1q** *vlan-id* command on a subinterface?
2. Do those same VLANs exist on the trunk on the neighboring switch (**show interfaces trunk**), and are they in the allowed list, not VTP pruned, and not STP blocked?
3. Does each router ROAS subinterface have an IP address/mask configured per the planned configuration?
4. If using the native VLAN, is it configured correctly on the router either on a subinterface (with an **encapsulation dot1q** *vlan-id* **native** command) or implied on the physical interface?
5. Is the same native VLAN configured on the neighboring switch's trunk in comparison to the native VLAN configured on the router?
6. Are the router physical or ROAS subinterfaces configured with a **shutdown** command?

For some of these steps, you need to be ready to investigate possible VLAN trunking issues on the LAN switch. Many Cisco router interfaces do not negotiate trunking. As a result, ROAS relies on static trunk configuration on both the router and switch. If the switch has any problems with VLANs or the VLAN trunking configuration on its side of the trunk, the router has no way to realize that the problem exists.

For example, imagine you configured ROAS on a router just like in Example 18-1 or Example 18-2. However, the switch on the other end of the link had no matching configuration. For instance, maybe the switch did not even define VLANs 10 and 20. Maybe the switch did not configure trunking on the port connected to the router. Even with blatant

misconfiguration or missing configuration on the switch, the router still shows up/up ROAS interfaces and subinterfaces, IP routes in the output of **show ip route**, and meaningful configuration information in the output of the **show vlans** command. The router will forward packets (encapsulated inside frames) to the switch, but the switch's configuration does not give it enough information to forward the frame correctly.

# VLAN Routing with Layer 3 Switch SVIs

Using a router with ROAS to route packets makes sense in some cases, particularly at small remote sites. In sites with a larger LAN, network designers choose to use Layer 3 switches for most inter-VLAN routing.

A Layer 3 switch (also called a multilayer switch) is one device, but it executes logic at two layers: Layer 2 LAN switching and Layer 3 IP routing. The Layer 2 switch function forwards frames inside each VLAN, but it will not forward frames between VLANs. The Layer 3 forwarding (routing) logic forwards IP packets between VLANs by applying IP routing logic to IP packets sent by the devices in those VLANs.

Layer 3 switches typically support two configuration options to enable IPv4 routing inside the switch, specifically to enable IPv4 on switch interfaces. This section explains one option, an option that uses switched virtual interfaces (SVI). The following major section of the chapter deals with the other option for configuring IPv4 addresses on Layer 3 switches: routed interfaces.

## Configuring Routing Using Switch SVIs

The configuration of a Layer 3 switch mostly looks like the Layer 2 switching configuration shown back in Parts II and III of this book, with a small bit of configuration added for the Layer 3 functions. The Layer 3 switching function needs a virtual interface connected to each VLAN internal to the switch. These **VLAN interfaces** act like router interfaces, with an IP address and mask. The Layer 3 switch has an IP routing table, with connected routes off each of these VLAN interfaces. Cisco refers to these virtual interfaces as **switched virtual interfaces (SVIs)**.

To show the concept of Layer 3 switching with SVIs, the following example uses the same branch office with two VLANs shown in the earlier examples, but now the design will use Layer 3 switching in the LAN switch. Figure 18-3 details the physical connections, subnets, and VLAN IDs. Figure 18-4 then shows the internal routing logic and SVIs.

Key Topic

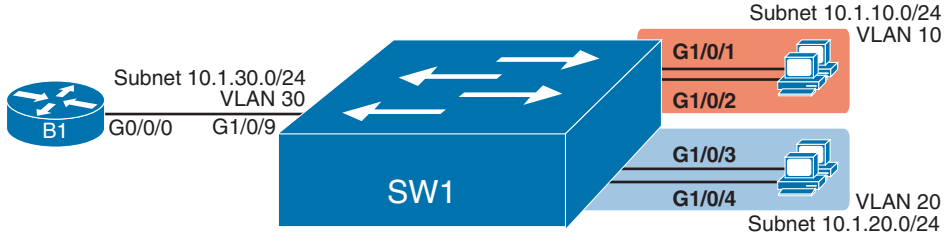



**Figure 18-3** *Physical Interfaces and Subnets for Layer 3 Switching Example*

Key Topic

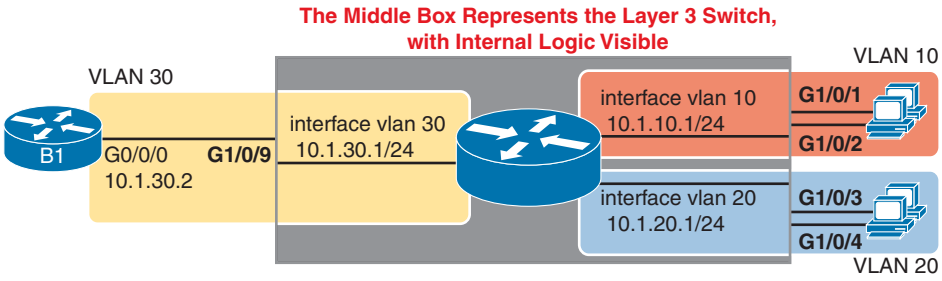



**Figure 18-4** *Internal VLAN Interfaces for Layer 3 Switching Example*

Note that Figure 18-4 represents the internals of the Layer 3 switch within the box in the middle of the figure. The branch still has two user VLANs (10 and 20), so the Layer 3 switch needs one VLAN interface (SVI) for each VLAN. The figure shows a router icon inside the gray box to represent the Layer 3 switching (routing) function, with two VLAN interfaces on the right side of that icon. In addition, the traffic still needs to get to Router B1 (a physical router) to access the WAN, so the switch uses a third VLAN (VLAN 30 in this case) for the link to Router B1. The switch treats the link between the Layer 3 switch and Router B1 as an access link; the router would be unaware of any VLANs and would not need VLAN trunking configuration.

The following steps show how to configure Layer 3 switching using SVIs.

Config Checklist

**Step 1.** Enable IP routing on the switch, as needed:

- **a.** (As needed) Use a model-specific command to change the switch forwarding ASIC settings to make space for IPv4 routes and reload the switch to make those settings take effect.
- **c.** Use the **ip routing** command in global configuration mode to enable the IPv4 routing function in IOS software and to enable key commands like **show ip route**.

**Step 2.** Configure each SVI interface, one per VLAN for which routing should be done by this Layer 3 switch:

- **a.** Use the **interface vlan** *vlan_id* command in global configuration mode to create a VLAN interface and to give the switch's routing logic a Layer 3 interface connected into the VLAN of the same number.
- **b.** Use the **ip address** *address mask* command in VLAN interface configuration mode to configure an IP address and mask on the VLAN interface, enabling IPv4 routing on that VLAN interface.
- **c.** (As needed) Use the **no shutdown** command in interface configuration mode to enable the VLAN interface (if it is currently in a shutdown state).

**NOTE** Regarding Step 1A, some older switch models do not support IP routing until you reprogram the switch's forwarding ASIC. For instance, the 2960 and 2960-XR switches popularly sold by Cisco in the 2010s required the **sdm prefer** global command followed by a reload of the switch. Newer Cisco switch models default to support IP routing and do not require similar steps.

Example 18-6 shows the configuration to match Figure 18-4. In this case, the switch defaulted to support IP routing in the forwarding ASIC without any special commands. The example shows the related configuration on all three VLAN interfaces.

**Example 18-6** *VLAN Interface Configuration for Layer 3 Switching*

```
ip routing
!
interface vlan 10
 ip address 10.1.10.1 255.255.255.0
!
interface vlan 20
 ip address 10.1.20.1 255.255.255.0
!
interface vlan 30
 ip address 10.1.30.1 255.255.255.0
```

## Verifying Routing with SVIs

With the VLAN configuration shown in the previous section, the switch is ready to route packets between the VLANs as shown in Figure 18-4. To support the routing of packets, the switch adds connected IP routes as shown in Example 18-7; note that each route is listed as being connected to a different VLAN interface.

**Example 18-7** *Connected Routes on a Layer 3 Switch*

```
SW1# show ip route connected
! legend omitted for brevity

       10.0.0.0/8 is variably subnetted, 6 subnets, 2 masks
C         10.1.10.0/24 is directly connected, Vlan10
L         10.1.10.1/32 is directly connected, Vlan10
C         10.1.20.0/24 is directly connected, Vlan20
L         10.1.20.1/32 is directly connected, Vlan20
C         10.1.30.0/24 is directly connected, Vlan30
L         10.1.30.1/32 is directly connected, Vlan30
```

The switch would also need additional routes to the rest of the network (not shown in the figures in this chapter). The Layer 3 switch could use static routes or a routing protocol, depending on the capabilities of the switch. For instance, if you then enabled OSPF on the Layer 3 switch, the configuration and verification would work the same as it does on a router, as discussed in Part VI, "OSPF." The routes that IOS adds to the Layer 3 switch's IP routing table list the VLAN interfaces as outgoing interfaces.

**NOTE** Some models of Cisco enterprise switches, based on the specific model, IOS version, and IOS feature set, support different capabilities for IP routing and routing protocols. For use in real networks, check the capabilities of the switch model by using the Cisco Feature Navigator (CFN) tool at www.cisco.com/go/cfn.

## Troubleshooting Routing with SVIs

On a physical router, when using physical interfaces, the router can assign a state to the interface based on physical factors, like whether the cable is installed or if the router receives any electricity or light over the cable. However, SVIs act as a virtual interface for the routing function in a Layer 3 switch. The switch must use some other logic to decide whether to place the SVI into a working (up/up) state.

Layer 3 switches use one of two interface configuration settings to dictate the logic to determine interface state: either **autostate** or **no autostate**. The default setting on a VLAN interface, **autostate**, checks several factors about the underlying VLAN to determine the state of the VLAN interface. By configuring the **no autostate** VLAN interface subcommand, the switch instead uses much simpler logic with fewer checks. The following few pages explain both.

### SVI Interface State with Autostate Enabled

The VLAN interface acts as the interface between the switch's routing function and the VLAN. For that VLAN interface to work properly, the VLAN must work properly. In particular, for a VLAN interface to be in an up/up state when using the autostate setting:

Key Topic

**Step 1.** The VLAN must be defined on the local switch (either explicitly or learned with VTP).

**Step 2.** The switch must have at least one up/up interface using the VLAN, including:

- **a.** An up/up access interface assigned to that VLAN
- **b.** A trunk interface for which the VLAN is in the allowed list, is STP forwarding, and is not VTP pruned

**Step 3.** The VLAN must be administratively enabled (that is, not **shutdown**).

**Step 4.** The VLAN interface must be administratively enabled (that is, not **shutdown**).



Do not miss this point: VLAN and the VLAN interface are related but separate ideas, each configured separately in the CLI. A VLAN interface, configured with the **interface vlan** *vlan-id* global command, creates a switch's Layer 3 interface connected to the VLAN. A VLAN, created with the **vlan** *vlan-id* global command, creates the VLAN. If you want to route packets for the subnets on VLANs 11, 12, and 13, using SVIs, you must configure the VLAN interfaces with those same VLAN IDs 11, 12, and 13. The VLANs with those same VLAN IDs must also exist.

IOS supports the function to disable and enable both a VLAN and a VLAN interface with the **shutdown** and **no shutdown** commands (as mentioned in Steps 3 and 4 in the preceding list). As part of the configuration checklist tasks, check the status to ensure that all the configuration enables all the related VLANs and VLAN interfaces.

Example 18-8 shows three scenarios, each of which leads to one of the VLAN interfaces in the previous configuration example (Figure 18-4, Example 18-6) to fail. At the beginning of the example, all three VLAN interfaces are up/up. To begin the example, VLANs 10, 20, and 30

each have at least one access interface up and working. The example works through three scenarios:

- **Scenario 1:** The last access interface in VLAN 10 is shut down (G1/0/1), so IOS shuts down the VLAN 10 interface.
- **Scenario 2:** VLAN 20 (not VLAN interface 20, but VLAN 20) is deleted, which results in IOS then bringing down (not shutting down) the VLAN 20 interface.
- **Scenario 3:** VLAN 30 (not VLAN interface 30, but VLAN 30) is shut down, which results in IOS then bringing down (not shutting down) the VLAN 30 interface.

**Example 18-8** *Three Examples That Cause VLAN Interfaces to Fail*

```
SW1# show interfaces status
! Only ports related to the example are shown
Port      Name               Status       Vlan       Duplex  Speed Type
Gi1/0/1                      connected    10         a-full  a-100 10/100/1000BaseTX
Gi1/0/2                      notconnect   10           auto   auto 10/100/1000BaseTX
Gi1/0/3                      connected    20         a-full  a-100 10/100/1000BaseTX
Gi1/0/4                      connected    20         a-full  a-100 10/100/1000BaseTX
Gi1/0/9                      connected    30         a-full a-1000 10/100/1000BaseTX

SW1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.

! Case 1: Interface G1/0/1, the last up/up access interface in VLAN 10, is shutdown
SW1(config)# interface GigabitEthernet 1/0/1
SW1(config-if)# shutdown
SW1(config-if)#
*Apr 2 19:54:08.784: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan10, changed
state to down
*Apr 2 19:54:10.772: %LINK-5-CHANGED: Interface GigabitEthernet1/0/1, changed state
to administratively down
*Apr 2 19:54:11.779: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEther-
net1/0/1, changed state to down

! Case 2: VLAN 20 is deleted
SW1(config)# no vlan 20
SW1(config)#
*Apr 2 19:54:39.688: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan20, changed
state to down

! Case 3: VLAN 30, the VLAN from the switch to the router, is shutdown
SW1(config)# vlan 30
SW1(config-vlan)# shutdown
SW1(config-vlan)# exit
SW1(config)#
*Apr 2 19:55:25.204: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan30, changed
state to down
```

```
! Final status of all three VLAN interfaces is below
SW1# show ip interface brief | include Vlan
Vlan1                  unassigned      YES manual administratively down down
Vlan10                 10.1.10.1       YES manual up                    down
Vlan20                 10.1.20.1       YES manual up                    down
Vlan30                 10.1.30.1       YES manual up                    down
```

Note that the example ends with the three VLAN interfaces in an up/down state per the **show ip interface brief** command.

### SVI Interface State with Autostate Disabled

With autostate disabled, the switch checks only whether the VLAN is defined on the switch, either explicitly or learned by VTP. It ignores all the other checks performed when using autostate. If no matching VLAN exists, the switch places the VLAN interface in an up/down state.

Example 18-9 shows how to determine whether autostate is enabled from the **show interfaces vlan** command output. In the example, the engineer already configured the **no autostate** command under interface VLAN 10, with interface VLAN 20 using the default setting of **autostate**.

**Example 18-9** *Recognizing the Autostate Setting on VLAN Interfaces*

```
SW1# show interfaces vlan 10
Vlan10 is up, line protocol is up , Autostate Disabled
! Lines omitted for brevity
SW1# show interfaces vlan 20
Vlan10 is up, line protocol is up , Autostate Enabled
! Lines omitted for brevity
```



## VLAN Routing with Layer 3 Switch Routed Ports

When Layer 3 switches use SVIs, the physical interfaces on the switches act like they always have: as Layer 2 interfaces. That is, the physical interfaces receive Ethernet frames, the switch learns the source MAC address of the frame, and the switch forwards the frame based on the destination MAC address. That logic occurs independently from any configured routing logic.

When using a Layer 3 switch, the switch acts as the default router for endpoint hosts. As usual, to send a packet to a default router, a host uses ARP to learn the default router's MAC address and then encapsulates the packet in a frame destined to the default router's MAC address. As a result, when using a Layer 3 switch with SVIs, hosts send their frames to the SVI's MAC address. Those frames arrive in a physical switch port, which forwards the frame based on the destination MAC address, but to the internal destination of the VLAN interface. That process triggers internal routing actions like stripping data-link headers, making a routing decision, and so on.

Alternately, the Layer 3 switch configuration can make a physical port act like a router interface instead of a switch interface. To do so, the switch configuration makes that port a

routed port. On a *routed port*, the switch does not perform Layer 2 switching logic on that frame. Instead, frames arriving in a routed port trigger the Layer 3 routing logic, including

1. Stripping off the incoming frame's Ethernet data-link header/trailer
2. Making a Layer 3 forwarding decision by comparing the destination IP address to the IP routing table
3. Adding a new Ethernet data-link header/trailer to the packet
4. Forwarding the packet, encapsulated in a new frame

This third major section of the chapter examines routed interfaces as configured on Cisco Layer 3 switches, but with a particular goal in mind: to also discuss Layer 3 EtherChannels. L3 EtherChannels use routed ports, so before learning about L3 EtherChannels you must first understand routed ports.

## Implementing Routed Interfaces on Switches

When a Layer 3 switch needs a Layer 3 interface connected to a subnet, and only one physical interface connects to that subnet, the design can use a routed port instead of an SVI. Stated differently, when routing over a point-to-point link connected to one other device only, using a routed port makes sense. Conversely, when the Layer 3 switch needs a Layer 3 interface connected to a subnet, and many physical interfaces on the switch connect to that subnet, the design must use an SVI.

To see why, consider the design in Figure 18-5, which repeats the same design from Figure 18-4, which was used in the SVI examples. In that design, the gray rectangle on the right represents the switch and its internals. On the right of the switch, at least two access ports sit in both VLAN 10 and VLAN 20. The Layer 3 switch must use SVIs as an interface into those VLANs because two or more ports connect to the VLAN.

Key Topic

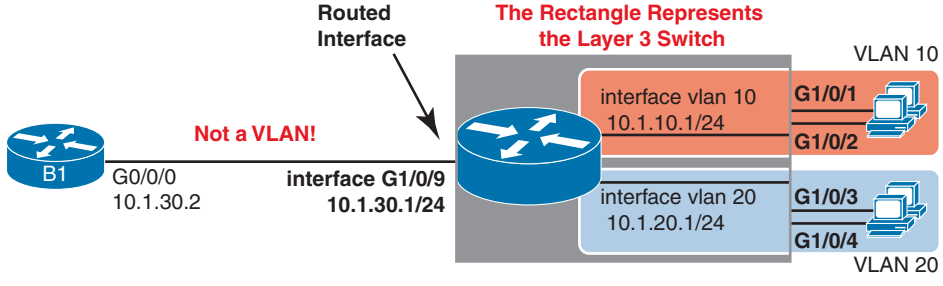



**Figure 18-5** *Routing on a Routed Interface on a Switch*

The link on the left of the figure connects from the switch to Router B1. The design needs routing between Router B1 and the switch. While earlier Example 18-6 and Example 18-7 show how to accomplish that routing with an access port and an SVI on the switch, using a routed port works as well, given that the design creates a point-to-point topology between the two devices.

Enabling a switch interface to be a routed interface instead of a switched interface is simple: just use the **no switchport** subcommand on the physical interface. Cisco switches capable

of being a Layer 3 switch use a default of the **switchport** command to each switch physical interface. Think about the word *switchport* for a moment. With that term, Cisco tells the switch to treat the port like it is a port on a switch—that is, a Layer 2 port on a switch. To make the port stop acting like a switch port and instead act like a router port, use the **no switchport** command on the interface.

Once the port is acting as a routed port, think of it like a router interface. That is, configure the IP address on the physical port, as implied in Figure 18-5. Example 18-10 shows a completed configuration for the interfaces configured on the switch in Figure 18-5. Note that the design uses the exact same IP subnets as the example that showed SVI configuration in Example 18-6, but now, the port connected to subnet 10.1.30.0 has been converted to a routed port. All you have to do is add the **no switchport** command to the physical interface and configure the IP address on the physical interface.

**Example 18-10** *Configuring Interface G0/1 on Switch SW1 as a Routed Port*

```
ip routing
!
interface vlan 10
 ip address 10.1.10.1 255.255.255.0
!
interface vlan 20
 ip address 10.1.20.1 255.255.255.0
!
interface gigabitethernet 1/0/9
 no switchport
 ip address 10.1.30.1 255.255.255.0
```

Once configured, the routed interface will show up differently in command output in the switch. In particular, for an interface configured as a routed port with an IP address, like interface GigabitEthernet1/0/9 in the previous example:

Key Topic

**show interfaces:** Similar to the same command on a router, the output will display the IP address of the interface. (Conversely, for switch ports, this command does not list an IP address.)

**show interfaces status:** Under the "VLAN" heading, instead of listing the access VLAN or the word *trunk*, the output lists the word *routed*, meaning that it is a routed port.

**show ip route:** Lists the routed port as an outgoing interface in routes.

**show interfaces** *type number* **switchport:** If a routed port, the output is short and confirms that the port is not a switch port. (If the port is a Layer 2 port, this command lists many configuration and status details.)

Example 18-11 shows samples of all four of these commands as taken from the switch as configured in Example 18-10.

**Example 18-11** *Verification Commands for Routed Ports on Switches*

```
SW1# show interfaces g1/0/9
GigabitEthernet1/0/9 is up, line protocol is up (connected)
  Hardware is Gigabit Ethernet, address is 4488.165a.f277 (bia 4488.165a.f277)
  Internet address is 10.1.30.1/24
! lines omitted for brevity

SW1# show interfaces status
! Only ports related to the example are shown; the command lists physical only
Port      Name               Status       Vlan       Duplex  Speed Type
Gi1/0/1                      connected    10         a-full a-1000 10/100/1000BaseTX
Gi1/0/2                      connected    10         a-full a-1000 10/100/1000BaseTX
Gi1/0/3                      connected    20         a-full a-1000 10/100/1000BaseTX
Gi1/0/4                      connected    20         a-full a-1000 10/100/1000BaseTX
Gi1/0/9                      connected    routed     a-full a-1000 10/100/1000BaseTX

SW1# show ip route
! legend omitted for brevity

       10.0.0.0/8 is variably subnetted, 6 subnets, 2 masks
C         10.1.10.0/24 is directly connected, Vlan10
L         10.1.10.1/32 is directly connected, Vlan10
C         10.1.20.0/24 is directly connected, Vlan20
L         10.1.20.1/32 is directly connected, Vlan20
C         10.1.30.0/24 is directly connected, GigabitEthernet1/0/9
L         10.1.30.1/32 is directly connected, GigabitEthernet1/0/9

SW1# show interfaces g0/1 switchport
Name: Gi1/0/9
Switchport: Disabled
```

So, with two options—SVI and routed ports—where should you use each?

> For any topologies with a point-to-point link between two devices that do routing, a routed interface works better. For any other topology, you must use SVIs.

Figure 18-6 shows an example of where to use SVIs and where to use routed ports in a typical core/distribution/access design. In this design, the core (Core1, Core2) and distribution (D11, D12, D21, D22) switches perform Layer 3 switching. The access switches (labeled A11, A12, and so on) perform only Layer 2 switching. All the ports that are links directly between the Layer 3 switches can be routed interfaces. For VLANs for which many interfaces (access and trunk) connect to the VLAN, SVIs make sense because the SVIs can send and receive traffic out multiple ports in the same VLAN on the same switch. In this design, all the ports on Core1 and Core2 will be routed ports, while the four distribution switches will use some routed ports and some SVIs.

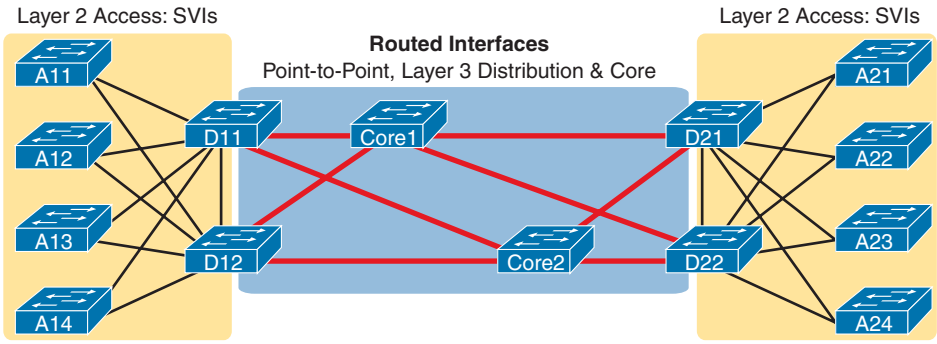



**Figure 18-6** *Using Routed Interfaces for Core and Distribution Layer 3 Links*

## Implementing Layer 3 EtherChannels

So far, this section has stated that routed interfaces can be used with a single point-to-point link between pairs of Layer 3 switches, or between a Layer 3 switch and a router. However, in most designs, the network engineers use at least two links between each pair of distribution and core switches, as shown in Figure 18-7.

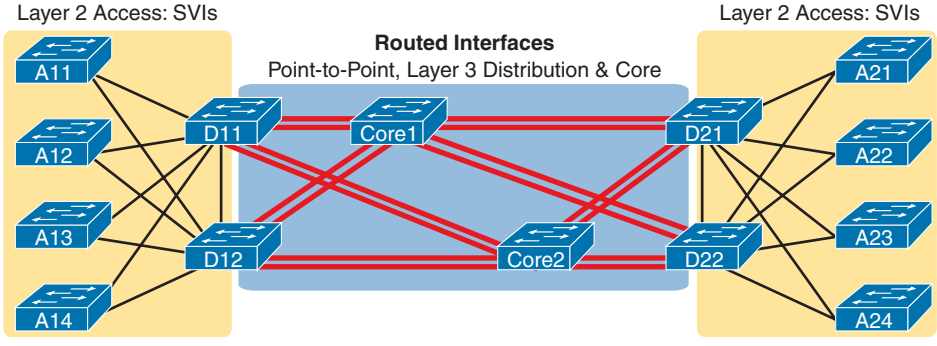



**Figure 18-7** *Two Links Between Each Distribution and Core Switch*

While each individual port in the distribution and core could be treated as a separate routed port, it is better to combine each pair of parallel links into a Layer 3 EtherChannel. Without using EtherChannel, you can still make each port on each switch in the center of the figure be a routed port. It works. However, once you enable a routing protocol but don't use EtherChannels, each Layer 3 switch will now learn two IP routes with the same neighboring switch as the next hop—one route over one link, another route over the other link.

Using a Layer 3 EtherChannel makes more sense with multiple parallel links between two switches. By doing so, each pair of links acts as one Layer 3 link. So, each pair of switches has one routing protocol neighbor relationship with the neighbor, and not two. Each switch learns one route per destination per pair of links, and not two. IOS then balances the traffic, often with better balancing than the balancing that occurs with the use of multiple IP routes to the same subnet. Overall, the Layer 3 EtherChannel approach works much better than leaving each link as a separate routed port and using Layer 3 balancing.

Compared to what you have already learned, configuring a Layer 3 EtherChannel takes only a little more work. Chapter 10, "RSTP and EtherChannel Configuration," already showed you how to configure an EtherChannel. This chapter has already shown how to make a port a Layer 3 routed port. Next, you have to combine the two ideas by combining both the EtherChannel and routed port configuration. The following checklist shows the steps, assuming a static definition for the EtherChannel.

Config Checklist

**Step 1.** Configure the physical interfaces as follows, in interface configuration mode:

- **a.** Add the **no switchport** command to make each physical port a routed port.
- **b.** Add the **channel-group** *number* **mode on** command to add it to the channel. Use the same number for all physical interfaces on the same switch, but the number used (the channel-group number) can differ on the two neighboring switches.

**Step 2.** Configure the PortChannel interface:

- **a.** Use the **interface port-channel** *number* command to move to port-channel configuration mode for the same channel number configured on the physical interfaces.
- **b.** Add the **no switchport** command to make sure that the port-channel interface acts as a routed port. (IOS may have already added this command.)
- **c.** Use the **ip address** *address mask* command to configure the address and mask.

**NOTE** Cisco uses the term *EtherChannel* in concepts discussed in this section and then uses the term *PortChannel*, with command keyword **port-channel**, when verifying and configuring EtherChannels. For the purposes of understanding the technology, you may treat these terms as synonyms. However, it helps to pay close attention to the use of the terms *PortChannel* and *EtherChannel* as you work through the examples in this section because IOS uses both.

Example 18-12 shows an example of the configuration for a Layer 3 EtherChannel for switch SW1 in Figure 18-8. The EtherChannel defines port-channel interface 12 and uses subnet 10.1.12.0/24. The design makes use of TenGigabit interfaces between the two switches.

Key Topic

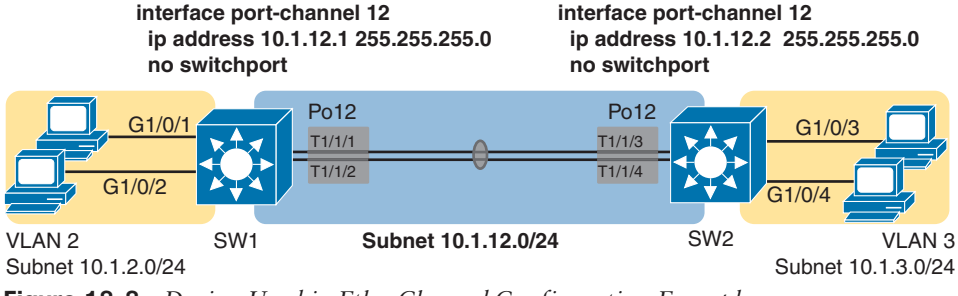



**Figure 18-8** *Design Used in EtherChannel Configuration Examples*

**Example 18-12** *Layer 3 EtherChannel Configuration on Switch SW1*

```
interface TenGigabit1/1/1
 no switchport
 no ip address
 channel-group 12 mode on
!
interface TenGigabit1/1/2
 no switchport
 no ip address
 channel-group 12 mode on
!
interface Port-channel12
 no switchport
 ip address 10.1.12.1 255.255.255.0
```

Of particular importance, note that although the physical interfaces and PortChannel interface are all routed ports, the IP address should be placed on the PortChannel interface only. In fact, when the **no switchport** command is configured on an interface, IOS adds the **no ip address** command to the interface. Then configure the IP address on the PortChannel interface only.

Once configured, the PortChannel interface appears in several commands, as shown in Example 18-13. The commands that list IP addresses and routes refer to the PortChannel interface. Also, note that the **show interfaces status** command lists the fact that the physical ports and the port-channel 12 interface are all routed ports.

**Example 18-13** *Verification Commands Listing Interface Port-Channel12 from Switch SW1*

```
SW1# show interfaces port-channel 12
Port-channel12 is up, line protocol is up (connected)
  Hardware is EtherChannel, address is 4488.165a.f26c (bia 4488.165a.f26c)
  Internet address is 10.1.12.1/24
! lines omitted for brevity

SW1# show interfaces status
! Only ports related to the example are shown.
Port        Name             Status       Vlan       Duplex  Speed Type
Te1/1/1                      connected    routed       full    10G SFP-10GBase-SR
Te1/1/2                      connected    routed       full    10G SFP-10GBase-SR
Po12                         connected    routed     a-full    a-10G N/A

SW1# show ip route
! legend omitted for brevity
       10.0.0.0/8 is variably subnetted, 4 subnets, 2 masks
C         10.1.2.0/24 is directly connected, Vlan2
L         10.1.2.1/32 is directly connected, Vlan2
C         10.1.12.0/24 is directly connected, Port-channel12
L         10.1.12.1/32 is directly connected, Port-channel12
```

For a final bit of verification, you can examine the EtherChannel directly with the **show etherchannel summary** command as listed in Example 18-14. Note in particular that it lists a flag legend for characters that identify key operational states, such as whether a port is bundled (included) in the PortChannel (P) and whether it is acting as a routed (R) or switched (S) port.

**Example 18-14** *Verifying the EtherChannel*

```
SW1# show etherchannel 12 summary
Flags:  D - down        P - bundled in port-channel
        I - stand-alone s - suspended
        H - Hot-standby (LACP only)
        R - Layer3      S - Layer2
        U - in use      f - failed to allocate aggregator

        M - not in use, minimum links not met
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port

        A - formed by Auto LAG


Number of channel-groups in use: 1
Number of aggregators:           1

Group  Port-channel  Protocol    Ports
------+-------------+-----------+-----------------------------------------------
12     Po12(RU)         -          Te1/1/1(P)      Te1/1/2(P)
```

## Troubleshooting Layer 3 EtherChannels

When you are troubleshooting a Layer 3 EtherChannel, there are two main areas to consider. First, you need to look at the configuration of the **channel-group** command, which enables an interface for an EtherChannel. Second, you should check a list of settings that must match on the interfaces for a Layer 3 EtherChannel to work correctly.

As for the **channel-group** interface subcommand, this command can enable EtherChannel statically or dynamically. If dynamic, this command's keywords imply either Port Aggregation Protocol (PAgP) or Link Aggregation Control Protocol (LACP) as the protocol to negotiate between the neighboring switches whether they put the link into the EtherChannel.

If all this sounds vaguely familiar, it is the exact same configuration covered way back in the Chapter 10 section "Configuring Dynamic EtherChannels." The configuration of the **channel-group** subcommand is exactly the same, with the same requirements, whether configuring Layer 2 or Layer 3 EtherChannels. So, it might be a good time to review those

EtherChannel configuration details from Chapter 10. Regardless of when you review and master those commands, note that the configuration of the EtherChannel (with the **channel-group** subcommand) is the same, whether Layer 2 or Layer 3.

Additionally, you must do more than just configure the **channel-group** command correctly for all the physical ports to be bundled into the EtherChannel. Layer 2 EtherChannels have a longer list of requirements, but Layer 3 EtherChannels also require a few consistency checks between the ports before they can be added to the EtherChannel. The following is the list of requirements for Layer 3 EtherChannels:

Key Topic

**no switchport:** The PortChannel interface must be configured with the **no switchport** command, and so must the physical interfaces. If a physical interface is not also configured with the **no switchport** command, it will not become operational in the EtherChannel.

**Speed:** The physical ports in the channel must use the same speed.

**duplex:** The physical ports in the channel must use the same duplex.

## VLAN Routing on a Router's LAN Switch Ports

The earlier major sections of this chapter all show designs with hosts connected to a switch. The switch might do the routing, or a router connected to the switch might do the routing, but the hosts are connected via a cable to the switch.

This last major section discusses a design with only a router, specifically one whose hardware includes a set of LAN switch ports. In effect, the router hardware embeds a LAN switch into the hardware. As a result, the router needs to implement both routing and switching.

Designs that include small branch offices can make good use of a router with embedded switch ports. Those sites need a router that can route over the WAN and may need only a few LAN ports. Many vendors (Cisco included) offer router products that include a small set of LAN switch ports for such cases. Many consumer-grade routers, like the one you have at home for Internet access, also embed a small number of switch ports.



As an example of the hardware, Figure 18-9 shows photos of an ISR1K router and a Catalyst 8200L edge router. Some ISR1Ks, positioned by Cisco as enterprise-class routers for smaller sites, include a few routed ports plus a set of **switched ports**. The routed ports act like ports on routers, as discussed so far in this book, receiving and forwarding IP packets while requiring the configuration of an IP address and mask. The switch ports work like ports on a LAN switch, forwarding and receiving Ethernet frames, with the router's switching logic learning entries for a MAC address table.

The bottom of the figure shows a photo of three Catalyst 8200 edge platforms (routers). The models in the figure each support four fixed routed Ethernet ports (two RJ-45 and two modular SFP slots). The Network Interface Module (NIM) slot allows for expansion with modular NIMs, including NIMs with four and eight LAN switch ports.

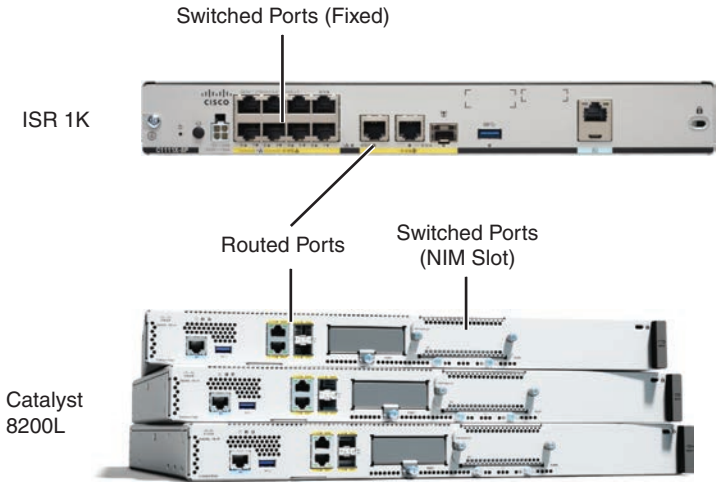



**Figure 18-9** *Cisco ISR 1000 and Catalyst 8200L Routers*

## Configuring Routing for Embedded Switch Ports

Routers that have switch ports implement both routing and switching. To make that work, configure LAN switching commands on the switch ports and routing commands for the routed interfaces. To stitch the logic together, use a concept familiar from multilayer switch configuration: SVI interfaces.

Figures 18-10 and Figure 18-11 give the context for an example. Figure 18-10 shows Router B1, at a branch office, with four switch ports connected to some PCs. The design calls for two VLANs with the same VLAN IDs and subnets, as in several earlier examples in this chapter. The router (the same model of ISR1K shown in Figure 18-9) uses routed port G0/0/1 to connect to the WAN.

Key Topic

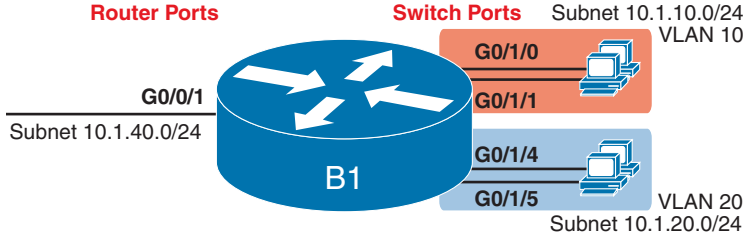



**Figure 18-10** *Physical Representation of Router with Routed and Switched Ports*

Figure 18-11 shows the internal logic using SVIs that give the routing process inside Router B1 interfaces that connect directly to the subnets that reside on VLANs 10 and 20. The concepts should look familiar; the internal concepts work like a Layer 3 switch.

Key Topic

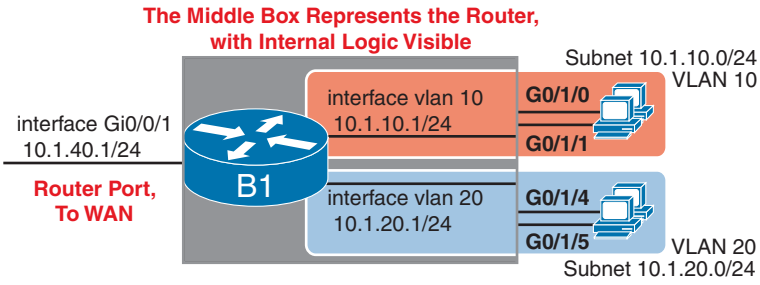



**Figure 18-11** *Configuration View of a Router with Routed and Switched Ports*

The configuration steps include familiar tasks for the LAN switch, other familiar tasks for IP routing and the routed ports, and the additional task of configuring the SVIs. To review, the following configuration checklist lists a minimal set of tasks, beginning with a default configuration, to enable IP on the three interfaces so that the router has connected routes to all three subnets and can route packets between those connected subnets.

Config Checklist

**Step 1.** Configure VLANs and assign access VLANs to the switch ports.

- **a.** Use the **vlan** *vlan-number* command in global configuration mode to create each VLAN.
- **b.** Use the **switchport access vlan** *vlan-id* command in interface configuration mode on the switched interfaces to assign the correct access VLAN.
- **c.** (As needed) Use the **no shutdown** command in interface configuration mode to enable the interface.

**Step 2.** Configure an SVI interface for each VLAN used by the switch ports.

- **a.** Use the **interface vlan** *vlan_id* command in global configuration mode to create a VLAN interface. Use the same number as the VLAN ID.
- **b.** Use the **ip address** *address mask* command in VLAN interface configuration mode to assign an IP address and mask to the interface, enabling IPv4 processing on the interface.
- **c.** (As needed) Use the **no shutdown** command in interface configuration mode to enable the VLAN interface.

**Step 3.** Configure any routed interfaces with IP addresses.

- **a.** Use the **ip address** *address mask* command in interface configuration mode to assign an IP address and mask to the interface, enabling IPv4 processing.
- **b.** Configure any routing protocols as needed.

Example 18-15 shows a sample configuration to match Figure 18-11. The top of the configuration adds VLANs 10 and 20, assigning them as access VLANs to the ports per the figure. The **interface vlan** commands create the SVIs, with IP address configuration added to each. The final interface, G0/0/1, a routed port, shows traditional router interface configuration with an **ip address** command but without a **switchport access** subcommand.

**Example 18-15** *VLAN Interface Configuration for Routing to/from Switch Port VLANs*

```
vlan 10
vlan 20
!
interface GigabitEthernet0/1/0
 switchport access vlan 10
!
interface GigabitEthernet0/1/1
 switchport access vlan 10
!
interface GigabitEthernet0/1/4
 switchport access vlan 20
!
interface GigabitEthernet0/1/5
 switchport access vlan 20
!
interface vlan 10
 ip address 10.1.10.1 255.255.255.0
!
interface vlan 20
 ip address 10.1.20.1 255.255.255.0
!
interface gigabitEthernet0/0/1
 description physical, routed interface

ip address 10.1.40.1 255.255.255.0
```

## Verifying Routing for Embedded Switch Ports

With the VLAN configuration shown in the previous section, the router is ready to route packets between the subnets as shown in Figure 18-11. The output in Example 18-16 lists two routes as connected to different VLAN interfaces, with one route connected to the one routed interface (G0/0/1). Beyond those routes, the router also needs routes to remote subnets. The router could use static routes or a routing protocol.

**Example 18-16** *Connected Routes on Router B1 from Example 18-15*

```
B1# show ip route connected
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
! Remaining legend omitted for brevity

      10.0.0.0/8 is variably subnetted, 13 subnets, 2 masks
C        10.1.10.0/24 is directly connected, Vlan10
L        10.1.10.1/32 is directly connected, Vlan10
C        10.1.20.0/24 is directly connected, Vlan20
L        10.1.20.1/32 is directly connected, Vlan20
C        10.1.30.0/24 is directly connected, GigabitEthernet0/0/1
L        10.1.30.1/32 is directly connected, GigabitEthernet0/0/1
```

The router adds the connected routes when the related interface reaches an up/up state. For SVIs, the state varies based on rules related to the state of the underlying VLAN and the ports in that VLAN. Those rules work just as described in this chapter's earlier section titled "Troubleshooting Routing with SVIs."

The router performs LAN switching with the same conventions you learned about LAN switches in Parts II and III of this book. For instance, the router supports the exact commands you expect to find on LAN switches to support the LAN switch ports. Example 18-17 shows the MAC addresses learned on the four switch ports shown in the most recent figures. It confirms that addresses learned on ports G0/1/0 and G0/1/1 exist in VLAN 10 while those learned on ports G0/1/4 and G0/1/5 exist in VLAN 20.

**Example 18-17** *MAC Address Table on the Router*

```
B1# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  10    0200.aaaa.aaaa    DYNAMIC     Gi0/1/0
  10    0200.bbbb.bbbb    DYNAMIC     Gi0/1/1
  20    0200.cccc.cccc    DYNAMIC     Gi0/1/4
  20    0200.dddd.dddd    DYNAMIC     Gi0/1/5
Total Mac Addresses for this criterion: 4
```

**NOTE** Switches use the **show vlan** [**brief**] command to display VLAN configuration. Routers with embedded switch ports have a command that mimics the switch command: **show vlan-switch** [**brief**] (IOS) and **show vlan** [**brief**] (IOS XE). As a reminder, the similar **show vlans** command on routers does not list VLANs but rather ROAS configuration.



## Identifying Switched Ports in Routers

When you look at a router or use the CLI of a router, how do you know which Ethernet ports happen to be routed ports and which are switch ports? Finding the answer can be difficult. This last short section tells you how.

First, you do not configure a router port's role as a routed or switch port. Instead, Cisco creates router interfaces that act as one or the other. You can research the router model and family at www.cisco.com and learn about the fixed ports that come with each model. That documentation also identifies modular options for cards like NIMs.

You can also discover the router's switched ports from the CLI, but the conventions are not obvious. Most **show** commands do not help, but a few do. Of note:

Key Topic

**show running-config, show startup-config:** These commands do not identify routed versus switched ports.

**show interfaces, show interfaces description, show ip interface brief, show protocols:** All these commands list all interfaces, routed and switched, with no notation of which is which.

**show interfaces status:** This command lists only switched ports.

Using **show interfaces status** to identify the switch ports makes the most sense.

Additionally, you can confirm which ports act as routed versus switched ports by trying to configure commands supported only on one or the other. For instance, routed ports accept the **ip address** *address mask* subcommand but reject the **switchport access vlan** *vlan-id* subcommand. Switched ports accept the opposite.

Example 18-18 shows a few cases of identifying routed and switched ports using Router B1 from the previous examples. It first shows only the switched ports G0/1/0–G0/1/7 in the **show interfaces status** command output. Note the output follows the format from LAN switches, identifying the access VLAN assigned to each interface. Following that, the output shows a switched port rejecting the **ip address** interface subcommand.

**Example 18-18** *Displaying Switched Interfaces but Omitting Routed Interfaces*

```
B1# show interfaces status

Port         Name               Status       Vlan       Duplex  Speed Type
Gi0/1/0                         connected    10         a-full a-1000 10/100/1000BaseTX
Gi0/1/1                         connected    10         a-full a-1000 10/100/1000BaseTX
Gi0/1/2                         notconnect   1            auto   auto 10/100/1000BaseTX
Gi0/1/3                         notconnect   1            auto   auto 10/100/1000BaseTX
Gi0/1/4                         connected    20         a-full a-1000 10/100/1000BaseTX
Gi0/1/5                         connected    20         a-full a-1000 10/100/1000BaseTX
Gi0/1/6                         notconnect   1            auto   auto 10/100/1000BaseTX
Gi0/1/7                         notconnect   1            auto   auto 10/100/1000BaseTX
B1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
B1(config)# interface g0/1/7
B1(config-if)# ip address 10.1.90.1 255.255.255.0
               ^
% Invalid input detected at '^' marker.
B1(config-if)#
```

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 18-2 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 18-2** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review config checklists | | Book, website |
| Review command tables | | Book |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 18-3** Key Topics for Chapter 18

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 18-2 | Concept of VLAN subinterfaces on a router | 459 |
| List | Two alternative methods to configure the native VLAN in a ROAS configuration | 460 |
| List | Troubleshooting suggestions for ROAS configuration | 463 |
| Figure 18-3 | Layer 3 switching with SVIs concept and configuration | 464 |
| Figure 18-4 | Layer 3 switching with routed ports concept and configuration | 465 |
| List | Troubleshooting suggestions for correct operation of a Layer 3 switch that uses SVIs | 467 |
| Figure 18-5 | A figure showing the concepts behind IP routing on a routed interface on a switch | 470 |
| List | **show** commands that list Layer 3 routed ports in their output | 471 |
| Figure 18-8 | Layer 3 EtherChannel concept and configuration | 474 |
| List | List of configuration settings that must be consistent before IOS will bundle a link with an existing Layer 3 EtherChannel | 477 |
| Figure 18-10 | A concept diagram showing a router with routed and switched ports | 478 |
| Figure 18-11 | Conceptual view of the configuration for a router with integrated LAN switch ports | 479 |
| List | Router commands and whether they list routed ports, switched ports, or both | 481 |

## Key Terms You Should Know

Layer 3 EtherChannel (L3 EtherChannel), Layer 3 switch, multilayer switch, routed port, router-on-a-stick (ROAS), subinterfaces, switched port, switched virtual interface (SVI), VLAN interface

## Command References

Tables 18-4 and 18-5 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 18-4** Chapter 18 Configuration Command Reference

| Command | Description |
|---|---|
| **interface** *type number.subint* | Router global command to create a subinterface and to enter configuration mode for that subinterface |
| **encapsulation dot1q** *vlan-id* [**native**] | Router subinterface subcommand that tells the router to use 802.1Q trunking, for a particular VLAN, and with the **native** keyword, to not encapsulate in a trunking header |
| [**no**] **ip routing** | Global command that enables (**ip routing**) or disables (**no ip routing**) the routing of IPv4 packets on a router or Layer 3 switch |
| **interface vlan** *vlan-id* | Global command to create a VLAN interface and to enter VLAN interface configuration mode; valid on Layer 3 switches and on routers that have embedded LAN switch ports |
| [**no**] **switchport** | Layer 3 switch subcommand that makes the port act as a Layer 2 port (**switchport**) or Layer 3 routed port (**no switchport**) |
| **interface port-channel** *channel-number* | A switch command to enter PortChannel configuration mode and also to create the PortChannel if not already created |
| **channel-group** *channel-number* **mode** {**auto** \| **desirable** \| **active** \| **passive** \| **on**} | Interface subcommand that enables EtherChannel on the interface |
| [**no**] **autostate** | Interface subcommand on VLAN interfaces that enables (**autostate**) or disables (**no autostate**) the autostate feature. |
| **vlan** *vlan-id* | Global config command that both creates the VLAN and puts the CLI into VLAN configuration mode |
| **name** *vlan-name* | VLAN subcommand that names the VLAN |
| [**no**] **shutdown** | VLAN mode subcommand that enables (**no shutdown**) or disables (**shutdown**) the VLAN |

**Table 18-5** Chapter 18 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip route** | Lists the router's entire routing table |
| **show ip route** [**connected**] | Lists a subset of the IP routing table |
| **show vlans** | Lists VLAN configuration and statistics for VLAN trunks configured on routers |

| Command | Description |
|---|---|
| **show interfaces** [**interface** *type number*] | Lists detailed status and statistical information, including IP address and mask, about all interfaces (or the listed interface only) |
| **show interfaces** [**interface** *type number*] **status** | Among other facts, for switch ports, lists the access VLAN or the fact that the interface is a trunk; or, for routed ports, lists "routed" |
| **show interfaces** *interface-id* **switchport** | For switch ports, lists information about any interface regarding administrative settings and operational state; for routed ports, the output simply confirms the port is a routed (not switched) port |
| **show interfaces vlan** *number* | Lists the interface status, the switch's IPv4 address and mask, and much more |
| **show etherchannel** [*channel-group-number*] **summary** | Lists information about the state of EtherChannels on this switch, including whether the channel is a Layer 2 or Layer 3 EtherChannel |
| **show mac address-table dynamic** | Shows all dynamically learned MAC table entries on LAN switches and on routers that have embedded switch ports |
| **show vlan-switch** [**brief**] | (IOS only) On a router with embedded LAN switch ports, lists the VLAN configuration |
| **show vlan** [**brief**] | (IOS XE only) On a router with embedded LAN switch ports, lists the VLAN configuration |



CHAPTER 19

# IP Addressing on Hosts

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.10 Verify IP parameters for Client OS (Windows, Mac OS, Linux)**

**4.0 IP Services**

**4.3 Explain the role of DHCP and DNS within the network**

**4.6 Configure and verify DHCP client and relay**

In the world of TCP/IP, the word *host* refers to any device with an IP address: your phone, your tablet, a PC, a server, a router, a switch—any device that uses IP to provide a service or just needs an IP address to be managed. The term *host* includes some less-obvious devices as well: the electronic advertising video screen at the mall, your electrical power meter that uses the same technology as mobile phones to submit your electrical usage information for billing, your new car.

No matter the type of host, any host that uses IPv4 needs four IPv4 settings to work properly:

- IP address
- Subnet mask
- Default routers
- DNS server IP addresses

This chapter discusses these basic IP settings on hosts. The chapter begins by discussing how a host can dynamically learn these four settings using the Dynamic Host Configuration Protocol (DHCP). The second half of this chapter then shows how to find the settings on hosts and the key facts to look for when displaying the settings.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 19-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Dynamic Host Configuration Protocol | 1–4 |
| Identifying Host IPv4 Settings | 5, 6 |

1. A PC connects to a LAN and uses DHCP to lease an IP address for the first time. Of the usual four DHCP messages that flow between the PC and the DHCP server, which ones does the client send? (Choose two answers.)
   a. Acknowledgment
   b. Discover
   c. Offer
   d. Request
2. Which of the following kinds of information are part of a DHCP server configuration? (Choose two answers.)
   a. Ranges of IP addresses in subnets that the server should lease
   b. Ranges of IP addresses to not lease per subnet
   c. DNS server hostnames
   d. The default router IP and MAC address in each subnet
3. Which answers list a criterion for choosing which router interfaces need to be configured as a DHCP relay agent? (Choose two answers.)
   a. If the subnet off the interface does not include a DHCP server
   b. If the subnet off the interface does include a DHCP server
   c. If the subnet off the interface contains DHCP clients
   d. If the router interface already has an **ip address dhcp** command
4. A router connects to an Internet service provider (ISP) using its G0/0/0 interface, with the **ip address dhcp** command configured. What does the router do with the DHCP-learned default gateway information?
   a. The router ignores the default gateway value learned from the DHCP server.
   b. The router uses the default gateway just like a host, ignoring its routing table.
   c. The router forwards received packets based on its routing table but uses its default gateway setting to forward packets it generates itself.
   d. The router adds a default route based on the default gateway to its IP routing table.
5. In the following excerpt from a command on a Mac, which of the following parts of the output represent information learned from a DHCP server? (Choose two answers.)

```
Macprompt$ ifconfig en0
En1: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
        options=10b<RXCSUM,TXCSUM,VLAN_HWTAGGING,AV>
        ether 00:6d:e7:b1:9a:11
        inet 172.16.4.2 netmask 0xffffff00 broadcast 172.16.4.255
```

**a.** 00:6d:e7:b1:9a:11
**b.** 172.16.4.2
**c.** 0xffffff00
**d.** 172.16.4.255

6. Which of the following commands on a Windows OS should list both the host's IP address and DNS servers' IP addresses as learned with DHCP?
   **a.** ifconfig
   **b.** ipconfig
   **c.** ifconfig /all
   **d.** ipconfig /all

## Foundation Topics

# Dynamic Host Configuration Protocol

Dynamic Host Configuration Protocol (DHCP) provides one of the most commonly used services in a TCP/IP network. The vast majority of hosts in a TCP/IP network are user devices, and the vast majority of user devices learn their IPv4 settings using DHCP.

Using DHCP has several advantages over the other option of manually configuring IPv4 settings. The configuration of host IP settings sits in a DHCP server, with each client learning these settings using DHCP messages. As a result, the host IP configuration is controlled by the IT staff, rather than on local configuration on each host, resulting in fewer user errors. DHCP allows the permanent assignment of host addresses, but more commonly, a temporary lease of IP addresses. With these leases, the **DHCP server** can reclaim IP addresses when a device is removed from the network, making better use of the available addresses.

DHCP also enables mobility. For example, every time a user moves to a new location with a tablet computer—to a coffee shop, a client location, or back at the office—the user's device can connect to another wireless LAN, use DHCP to lease a new IP address in that LAN, and begin working on the new network. Without DHCP, the user would have to ask for information about the local network and configure settings manually, with more than a few users making mistakes.

Although DHCP works automatically for user hosts, it does require some preparation from the network, with some configuration on routers. In some enterprise networks, that router configuration can be a single command on many of the router's LAN interfaces (**ip helper-address** *server-ip*), which identifies the DHCP server by its IP address. In other cases, the router acts as the DHCP server. Regardless, the routers have some role to play.

This first major section of the chapter takes a tour of DHCP, including concepts and the router configuration to enable the routers to work well with a separate DHCP server.

## DHCP Concepts

Sit back for a moment and think about the role of DHCP for a host computer. The host acts as a **DHCP client**. As a DHCP client, the host begins with no IPv4 settings—no IPv4 address, no mask, no default router, and no **DNS server** IP addresses. But a DHCP client does have

knowledge of the DHCP protocol, so the client can use that protocol to (a) discover a DHCP server and (b) request to lease an IPv4 address.

DHCP uses the following four messages between the client and server. (Also, to help remember the messages, note that the first letters spell DORA):

**Discover:** Sent by the DHCP client to find a willing DHCP server

**Offer:** Sent by a DHCP server to offer to lease to that client a specific IP address (and inform the client of its other parameters)

**Request:** Sent by the DHCP client to ask the server to lease the IPv4 address listed in the Offer message

**Acknowledgment:** Sent by the DHCP server to assign the address and to list the mask, default router, and DNS server IP addresses

DHCP clients, however, have a somewhat unique problem: they do not have an IP address yet, but they need to send these DHCP messages inside IP packets. To make that work, DHCP messages make use of two special IPv4 addresses that allow a host that has no IP address to still be able to send and receive messages on the local subnet:

Key Topic

**0.0.0.0:** An address reserved for use as a source IPv4 address for hosts that do not yet have an IP address.

**255.255.255.255:** The local broadcast IP address. Packets sent to this destination address are broadcast on the local data link, but routers do not forward them.

To see how these addresses work, Figure 19-1 shows an example of the IP addresses used between a host (A) and a DHCP server on the same LAN. Host A, a client, sends a Discover message, with source IP address of 0.0.0.0 because Host A does not have an IP address to use yet. Host A sends the packet to destination 255.255.255.255, which is sent in a LAN broadcast frame, reaching all hosts in the subnet. The client hopes that there is a DHCP server on the local subnet. Why? Packets sent to 255.255.255.255 only go to hosts in the local subnet; Router R1 will not forward this packet.



**NOTE** Figure 19-1 shows one example of the addresses that can be used in a DHCP request. This example shows details assuming the DHCP client chooses to use a DHCP option called the *broadcast flag*; all examples in this book assume the broadcast flag is used.

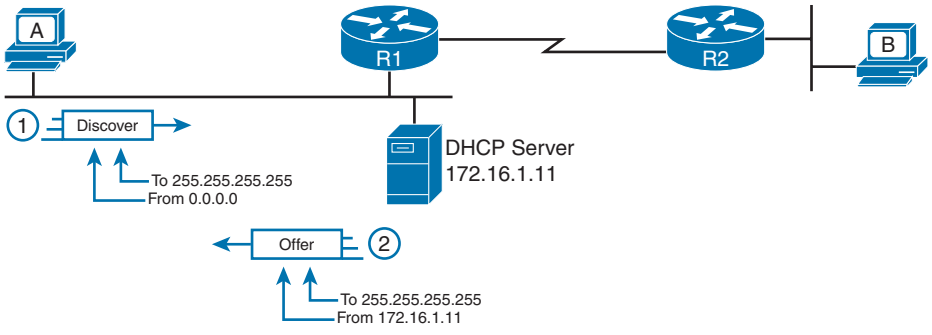



**Figure 19-1** *DHCP Discover and Offer*

Now look at the Offer message sent back by the DHCP server. The server sets the destination IP address to 255.255.255.255 again. Why? Host A still does not have an IP address, so the server cannot send a packet directly to host A. So, the server sends the packet to "all local hosts in the subnet" address (255.255.255.255). (The packet is also encapsulated in an Ethernet broadcast frame.)

Note that all hosts in the subnet receive the Offer message. However, the original Discover message lists a number called the client ID, which includes the host's MAC address, that identifies the original host (Host A in this case). As a result, Host A knows that the Offer message is meant for host A. The rest of the hosts will receive the Offer message, but notice that the message lists another device's DHCP client ID, so the rest of the hosts ignore the Offer message.

### APIPA IP Addresses (169.254.x.x)

If the DHCP process fails for a DHCP client, hosts have a default means to self-assign an IP address using a feature called Automatic Private IP Addressing (**APIPA**). When the DHCP process fails, the DHCP client self-assigns an APIPA IP address from within a subset of the 169.254.0.0 Class B network, along with default mask 255.255.0.0.

Using an APIPA address does not help devices work like a normal host. For instance, hosts do not know of a default router or learn a list of DNS servers. The host can send packets only to other APIPA hosts on the same LAN.

The APIPA process works like this:

1. The client chooses any IP address from network 169.254.0.0 (actual address range: 169.54.1.0 – 169.254.254.255).
2. The DHCP client discovers if any other host on the same link already uses the APIPA address it chose by using ARP to perform Duplicate Address Detection (DAD). If another host already uses the same address, the client stops using the address and chooses another.
3. The client can send/receive packets on the local network only.

### Supporting DHCP for Remote Subnets with DHCP Relay

Network engineers have a major design choice to make with DHCP: Do they put a DHCP server in every LAN subnet or locate a DHCP server in a central site? The question is legitimate. Cisco routers can act as the DHCP server, so a distributed design could use the router at each site as the DHCP server. With a DHCP server in every subnet, as shown in Figure 19-1, the protocol flows stay local to each LAN.

However, a centralized DHCP server approach has advantages as well. In fact, some Cisco design documents suggest a centralized design as a best practice, in part because it allows for centralized control and configuration of all the IPv4 addresses assigned throughout the enterprise.

---

Answers to the "Do I Know This Already?" quiz:

**1** B, D **2** A, B **3** A, C **4** D **5** B, C **6** D

With a centralized DHCP server, those DHCP messages that flowed only on the local subnet in Figure 19-1 somehow need to flow over the IP network to the centralized DHCP server and back. To make that work, the routers connected to the remote LAN subnets need an interface subcommand: the **ip helper-address** *server-ip* command.

The **ip helper-address** *server-ip* subcommand tells the router to do the following for the messages coming in an interface, from a DHCP client:

Key Topic

1. Watch for incoming DHCP messages, with destination IP address 255.255.255.255.
2. Change that packet's source IP address to the router's incoming interface IP address.
3. Change that packet's destination IP address to the address of the DHCP server (as configured in the **ip helper-address** command).
4. Route the packet to the DHCP server.

This command gets around the "do not route local subnet broadcast packets sent to address 255.255.255.255" rule by changing the destination IP address. Once the destination has been set to match the DHCP server's IP address, the network can route the packet to the server.

**NOTE** This feature, by which a router relays DHCP messages by changing the IP addresses in the packet header, is called *DHCP relay*.

Figure 19-2 shows an example of the process. Host A sits on the left, as a DHCP client. The DHCP server (172.16.2.11) sits on the right. R1 has an **ip helper-address 172.16.2.11** command configured, under its G0/0 interface. At Step 1, Router R1 notices the incoming DHCP packet destined for 255.255.255.255. Step 2 shows the results of changing both the source and destination IP address, with R1 routing the packet.

Key Topic

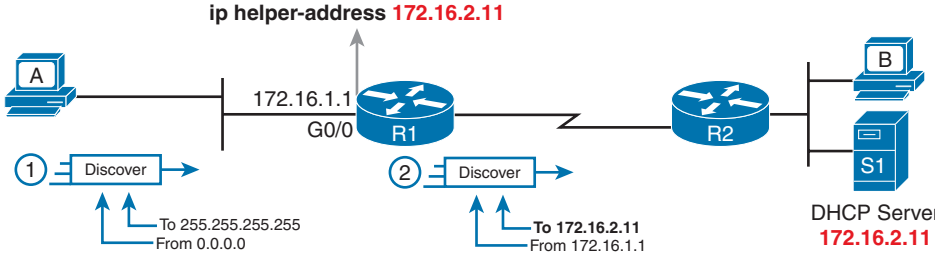



**Figure 19-2** *IP Helper Address Effect*

The router uses a similar process for the return DHCP messages from the server. First, for the return packet from the DHCP server, the server simply reverses the source and destination IP address of the packet received from the router (relay agent). For example, in Figure 19-2, the Discover message lists source IP address 172.16.1.1, so the server sends the Offer message back to destination IP address 172.16.1.1.

When a router receives a DHCP message addressed to one of the router's own IP addresses, the router realizes the packet might be part of the DHCP relay feature. When that happens, the **DHCP relay agent** (Router R1) needs to change the destination IP address so that the

real DHCP client (host A), which does not have an IP address yet, can receive and process the packet.

Figure 19-3 shows one example of how these addresses work, when R1 receives the DHCP Offer message sent to R1's own 172.16.1.1 address. R1 changes the packet's destination to 255.255.255.255 and forwards it out G0/0, because the packet was destined to G0/0's 172.16.1.1 IP address. As a result, all hosts in that LAN (including the DHCP client A) will receive the message.

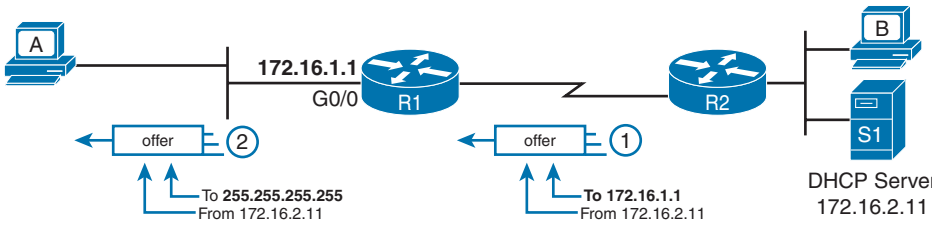



**Figure 19-3** *IP Helper Address for the Offer Message Returned from the DHCP Server*

Many enterprise networks use a centralized DHCP server, so the normal router configuration includes an **ip helper-address** command on every LAN interface/subinterface. With that standard configuration, user hosts off any router LAN interface can always reach the DHCP server and lease an IP address.

### Information Stored at the DHCP Server

A DHCP server might sound like some large piece of hardware, sitting in a big locked room with lots of air conditioning to keep the hardware cool. However, like most servers, the server is actually software, running on some server OS. The DHCP server could be a piece of software downloaded for free and installed on an old PC. However, because the server needs to be available all the time, to support new DHCP clients, most companies install the software on a very stable and highly available data center, with high availability features. The DHCP service is still created by software, however.

To be ready to answer DHCP clients and to supply them with an IPv4 address and other information, the DHCP server (software) needs configuration. DHCP servers organize these IPv4 settings per subnet, listing an address pool and a default router setting—settings that differ from subnet to subnet. The following list shows the types of settings the DHCP server needs to know to support DHCP clients:

**Subnet ID and mask:** The DHCP server can use this information to know all addresses in the subnet. (The DHCP server knows to not lease the subnet ID or subnet broadcast address.)

**Reserved (excluded) addresses:** The server needs to know which addresses in the subnet to *not* lease. This list allows the engineer to reserve addresses to be used as static IP addresses. For example, most router and switch IP addresses, server addresses, and addresses of most anything other than user devices use a statically assigned IP address. Most of the time, engineers use the same convention for all subnets, either reserving the lowest IP addresses in all subnets or reserving the highest IP addresses in all subnets.

**Default router(s):** This is the IP address of the router on that subnet.

**DNS IP address(es):** This is a list of DNS server IP addresses.

Figure 19-4 shows the concept behind the preconfiguration on a DHCP server for two LAN-based subnets: 172.16.1.0/24 and 172.16.2.0/24. The DHCP server sits on the right. For each subnet, the server defines all the items in the list. In this case, the configuration reserves the lowest IP addresses in the subnet to be used as static addresses.

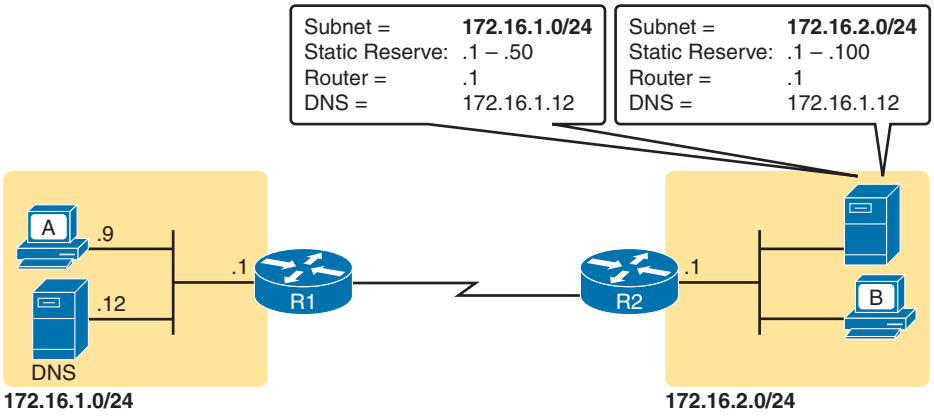



**Figure 19-4** *Preconfiguration on a DHCP Server*

The configuration can list other parameters as well. For example, it can set the time limit for leasing an IP address. The server leases an address for a time (usually a number of days), and then the client can ask to renew the lease. If the client does not renew, the server can reclaim the IP address and put it back in the pool of available IP addresses. The server configuration sets the maximum time for the lease.

DHCP uses three allocation modes, based on small differences in the configuration at the DHCP server. *Dynamic allocation* refers to the DHCP mechanisms and configuration described throughout this chapter. Another method, *automatic allocation*, sets the DHCP lease time to infinite. As a result, once the server chooses an address from the pool and assigns the IP address to a client, the IP address remains with that same client indefinitely. A third mode, *static allocation*, preconfigures the specific IP address for a client based on the client's MAC address. That specific client is the only client that then uses the IP address. (Note that this chapter shows examples and configuration for dynamic allocation only.)

Additionally, the DHCP server can be configured to supply some other useful configuration settings. For instance, a server can supply the IP address of a Trivial File Transfer Protocol (TFTP) server. TFTP servers provide a basic means of storing files that can then be transferred to a client host. For instance, Cisco IP phones rely on TFTP to retrieve several configuration files when the phone initializes. DHCP plays a key role by supplying the IP address of the TFTP server that the phones should use.

## Configuring DHCP Features on Routers and Switches

Cisco routers and switches support a variety of features. Routers can be configured to act as a DHCP server with just a few straightforward commands—a feature useful in the lab and

in some limited cases. More commonly, the enterprise uses a centralized DHCP server (that does not run on a router) but with the DHCP relay feature on most every router interface. Finally, Cisco routers and switches can also act as DHCP clients, learning their IP addresses from a DHCP server.

This section discusses the DHCP configuration topics mentioned for the CCNA 200-301 V1.1 exam blueprint. Those include the router DHCP relay feature and the configuration to enable DHCP client services on both switches and routers.

**NOTE** The CCNA 200-301 V1.1 exam blueprint does not mention the DHCP server function, but many people like to use the IOS DHCP server in the lab for testing with DHCP. If interested in how to configure a DHCP server on a router, refer to Appendix K, "Topics from Previous Editions."

### Configuring DHCP Relay

Configuring DHCP relay requires a simple decision and a single straightforward configuration command. First, you must identify the interfaces that need the feature. The DHCP relay feature must be configured for any router interface that connects to a subnet where



- DHCP clients exist in the subnet
- DHCP servers do not exist in the subnet

Once such interfaces have been identified, the configuration requires the **ip helper-address** interface subcommand on each of those interfaces. For instance, in the topology of the previous three figures, R1's G0/0 interface needs to be configured with the **ip helper-address 172.16.2.11** interface subcommand. Once enabled on an interface, the IOS DHCP relay agent makes changes in the incoming DHCP messages' addresses as described earlier in the chapter. Without the DHCP relay agent, the DHCP request never arrives at the server.

To verify the relay agent, you can use the **show running-config** command and look for the single configuration command or use the **show ip interface g0/0** command as shown in Example 19-1. The highlighted line confirms the configured setting. Note that if there were no **ip helper-address** commands configured on the interface, the highlighted line would instead read "Helper address is not set."

**Example 19-1** *Listing the Current Helper Address Setting with* **show ip interface**

```
R1# show ip interface g0/0
GigabitEthernet0/0 is up, line protocol is up
 Internet address is 172.16.1.1/24
 Broadcast address is 255.255.255.255
 Address determined by non-volatile memory
 MTU is 1500 bytes
 Helper address is 172.16.2.11
! Lines omitted for brevity
```

### Configuring a Switch as DHCP Client

A switch can act as a DHCP client to lease its management IP address. In most cases, you will want to instead use a static IP address so that the staff can more easily identify the switch's address for remote management. However, as an example of how a DHCP client can work, this next topic shows how to configure and verify DHCP client operations on a switch.

**NOTE** Chapter 6, "Configuring Basic Switch Management," also shows this same example of how to configure a switch to be a DHCP client. This chapter repeats the example here so you can see all the related DHCP configuration details in a single place in this volume.

To configure a switch to use DHCP to lease an address, configure a switch's IP address as normal, but with the **ip address dhcp** interface subcommand. Example 19-2 shows a sample.

**Example 19-2** *Switch Dynamic IP Address Configuration with DHCP*

```
Emma# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Emma(config)# interface vlan 1
Emma(config-if)# ip address dhcp
Emma(config-if)# no shutdown
Emma(config-if)# ^Z
Emma#
00:38:20: %LINK-3-UPDOWN: Interface Vlan1, changed state to up
00:38:21: %LINEPROTO-5-UPDOWN: Line protocol on Interface Vlan1, changed state to up
```

To verify that DHCP worked, start with the traditional way to check IP addresses on switch VLAN interfaces: the **show interfaces vlan** *x* command as demonstrated in Example 19-3. First, check the interface state, because the switch does not attempt DHCP until the VLAN interface reaches an up/up state. Notably, if you forget to issue the **no shutdown** command, the VLAN 1 interface will remain in a shutdown state and be listed as "administratively down" in the **show** command output. Then, if DHCP has not yet worked, you will see the highlighted line shown in the upper part of the example. Once the switch leases an IP address, you will see the different text shown in the bottom half of the example.

**Example 19-3** *Verifying DHCP-Learned IP Address on a Switch*

```
! First, output when the switch has not yet leased an IP address
Emma# show interfaces vlan 1
Vlan1 is up, line protocol is up, Autostate Enabled
 Hardware is Ethernet SVI, address is 4488.165a.f247 (bia 4488.165a.f247)
 Internet address will be negotiated using DHCP
! lines omitted for brevity
! Next, the output after DHCP works
Emma# show interfaces vlan 1
Vlan1 is up, line protocol is up, Autostate Enabled
 Hardware is Ethernet SVI, address is 4488.165a.f247 (bia 4488.165a.f247)
 Internet address is 192.168.1.101/24
! lines omitted for brevity
```

Interestingly, output line three, which lists the address, appears the same whether you configure the address explicitly or whether the switch leases the address. For instance, the latter half of Example 19-3 lists 192.168.1.101 as the address, but with no information to identify whether the IP address is a static or DHCP-learned IP address.

To see more details specific to DHCP, instead use the **show dhcp lease** command to see the (temporarily) leased IP address and other parameters. (Note that the switch does not store the DHCP-learned IP configuration in the running-config file.) Example 19-4 shows sample output. Note also that the switch learns its default-gateway setting using DHCP as well.

Key Topic

**Example 19-4** *Verifying DHCP-Learned Information on a Switch*

```
Emma# show dhcp lease
Temp IP addr: 192.168.1.101 for peer on Interface: Vlan1
Temp sub net mask: 255.255.255.0
 DHCP Lease server: 192.168.1.1, state: 3 Bound
 DHCP transaction id: 1966
 Lease: 86400 secs, Renewal: 43200 secs, Rebind: 75600 secs
Temp default-gateway addr: 192.168.1.1
 Next timer fires after: 11:59:45
 Retry count: 0 Client-ID: cisco-0019.e86a.6fc0-Vl1
 Hostname: Emma

Emma# show ip default-gateway
192.168.1.1
```

## Configuring a Router as DHCP Client

Just as with switches, you can configure router interfaces to lease an IP address using DHCP rather than using a static IP address, although those cases will be rare. In most every case it makes more sense to statically configure router interface IP addresses with the address listed in the **ip address** *address mask* interface subcommand. However, configuring a router to lease an address using DHCP makes sense in some cases with a router connected to the Internet; in fact, most every home-based router does just that.

A router with a link to the Internet can learn its IP address and mask with DHCP and also learn the neighboring ISP router's address as the **default gateway**. Figure 19-5 shows an example, with three routers on the left at one enterprise site. Router R1 uses DHCP to learn its IP address (192.0.2.2) from the ISP router over a connection to the Internet.

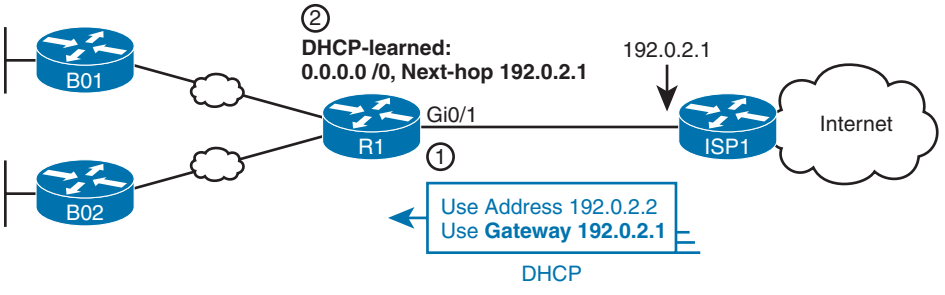



**Figure 19-5** *Enterprise Router Building and Advertising Default Routes with DHCP Client*

The DHCP process supplies a default gateway IP address to Router R1, but routers do not normally use a default gateway setting; only hosts use a default gateway setting. However, the router takes advantage of that information by turning that default gateway IP address into the basis for a default route. For instance, in Figure 19-5, Router R1 dynamically adds a default route to its routing table with the default gateway IP address from the DHCP message—which is the ISP router's IP address—as the next-hop address. At that point, R1 has a good route to use to forward packets into the Internet.

Additionally, Router R1 can distribute that default route to the rest of the routers using an interior routing protocol like OSPF. See the section "OSPF Default Routes" in Chapter 23, "Implementing Optional OSPF Features," for more information.

Example 19-5 shows the configuration on Router R1 to match Figure 19-5. Note that it begins with R1 configuring its G0/1 interface to use DHCP to learn the IP address to use on the interface, using the **ip address dhcp** command.

**Example 19-5** *Learning an Address and Default Static Route with DHCP*

```
R1# configure terminal
R1(config)# interface gigabitethernet0/1
R1(config-if)# ip address dhcp
R1(config-if)# end
R1#
R1# show ip route static
! Legend omitted
Gateway of last resort is 192.0.2.1 to network 0.0.0.0

S* 0.0.0.0/0 [254/0] via 192.0.2.1
```

The end of the example shows the resulting default route. Oddly, IOS displays this route as a static route (destination 0.0.0.0/0), although the route is learned dynamically based on the DHCP-learned default gateway. To recognize this route as a DHCP-learned default route, look to the administrative distance value of 254. IOS uses a default administrative distance of 1 for static routes configured with the **ip route** configuration command but a default of 254 for default routes added because of DHCP.



# Identifying Host IPv4 Settings

Whether learned using DHCP or not, every host that uses IP version 4 needs to have some settings to work correctly. This second major section of the chapter examines those settings and shows examples of those settings on Windows, Linux, and macOS.

## Host Settings for IPv4

To work correctly, an IPv4 host needs to know these values:

Key Topic

- DNS server IP addresses
- Default gateway (router) IP address
- Device's own IP address
- Device's own subnet mask

To review the basics, the host must know the IP address of one or more DNS servers to send the servers' name resolution requests. For enterprises, the servers may reside in the enterprise, as shown in Figure 19-6. The host on the left (sometimes called an endpoint) typically knows the addresses of at least two DNS servers for redundancy. If the first DNS fails to respond, the endpoint can then attempt name resolution with the next DNS server.

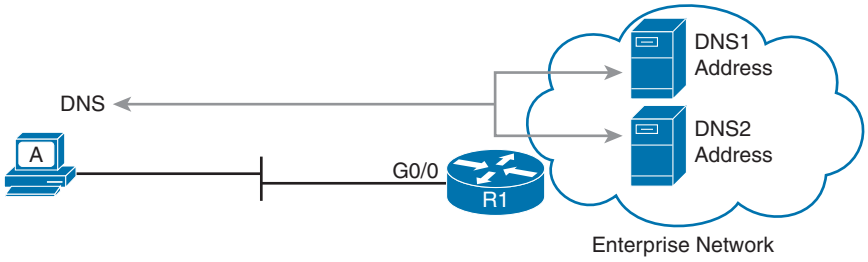



**Figure 19-6** *Host A Needs to Know the IP Address of the DNS Servers*

Each endpoint needs to know the IP address of a router that resides in the same subnet. The endpoint uses that router as its default router or default gateway, as shown in Figure 19-7. From a host logic perspective, the host can then forward packets destined for addresses outside the subnet to the default router, with that router then forwarding the packet based on its routing table.

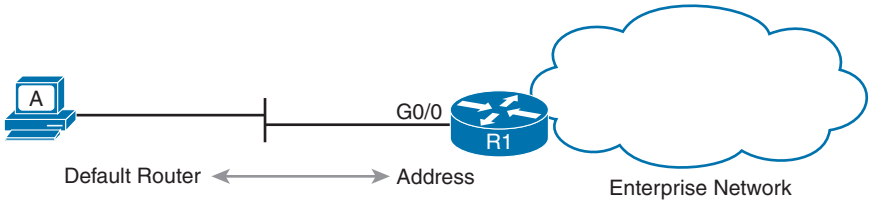



**Figure 19-7** *Host Default Router Setting Should Equal Router Interface Address*

Of course, each device needs its own IP address and subnet mask. Equally as important, note that the host and the default router must agree on the addresses inside the subnet. The host will use the address and mask to do the math to determine which addresses are in the same subnet and which are in other subnets. For routing to work correctly, the default router's interface address and mask should result in the same definition of the subnet with the same addresses, as shown in Figure 19-8.

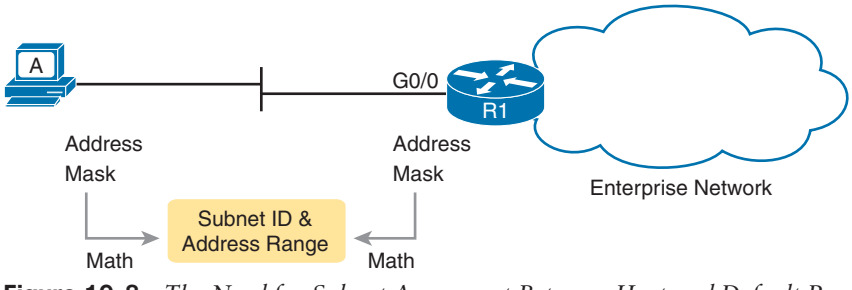



**Figure 19-8** *The Need for Subnet Agreement Between Host and Default Router*

The rest of this section shows examples of these settings in the graphical user interface (GUI) and command-line interface (CLI) of three different host operating systems.

The examples for the remainder of the chapter use hosts in subnet 10.1.1.0/24, as shown on the left side of Figure 19-9.

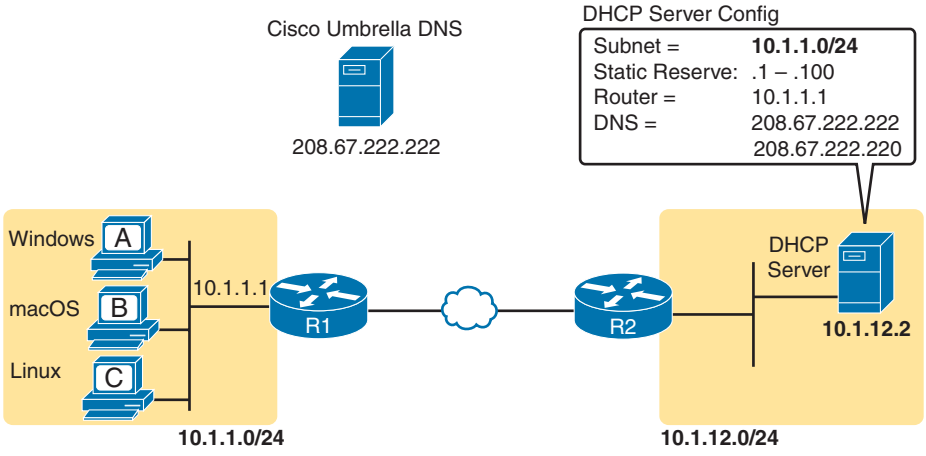



**Figure 19-9** *Subnet 10.1.1.0/24 Used for the Upcoming Host Examples*

## Host IP Settings on Windows

Almost every OS in the world—certainly the more common OSs people work with every day—have a fairly easy-to-reach settings window that lists most if not all the IPv4 settings in one place.

For example, Figure 19-10 shows the Network configuration screen from a Windows host from the network area of the Windows Control Panel. This image shows the four significant settings: address, mask, router, and DNS. Note that this host, Windows Host A in Figure 19-9, uses DHCP to learn its settings from the DHCP server in that figure.

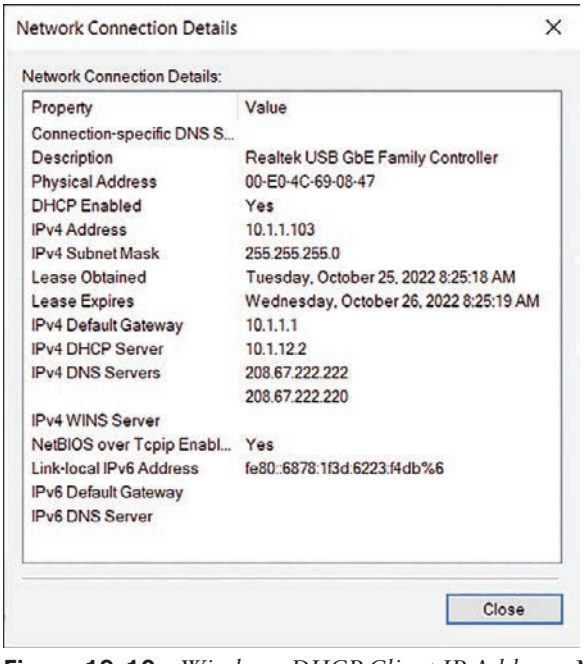



**Figure 19-10** *Windows DHCP Client IP Address, Mask, and Default Router Settings*

However, beyond the GUI, most OSs have a variety of networking commands available from a command line. For many decades and Windows versions, the **ipconfig** and **ipconfig /all** commands supply the most direct help, as shown in Example 19-6. As you can see, both list the address, mask, and default gateway. The **ipconfig /all** command lists much more detail, including the **DNS server list**, confirmation that the host used DHCP, along with the IP address of the DHCP server.

Key Topic

**Example 19-6** **ipconfig** *and* **ipconfig /all** *(Windows)*

```
Windows_A> ipconfig
! Lines omitted for brevity
Ethernet adapter Ethernet 5:

   Connection-specific DNS Suffix . :
   Link-local IPv6 Address . . . . . : fe80::6878:1f3d:6223:f4db%6
   IPv4 Address. . . . . . . . . . . : 10.1.1.103
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.1.1.1

Windows_A> ipconfig /all
! Lines omitted for brevity
Ethernet adapter Ethernet 5:

   Connection-specific DNS Suffix .  :
   Description . . . . . . . . . . . : Realtek USB GbE Family Controller
   Physical Address. . . . . . . . . : 00-E0-4C-69-08-47
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::6878:1f3d:6223:f4db%6(Preferred)
   IPv4 Address. . . . . . . . . . . : 10.1.1.103(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Tuesday, October 25, 2022 8:25:19 AM
   Lease Expires . . . . . . . . . . : Wednesday, October 26, 2022 8:25:19 AM
   Default Gateway . . . . . . . . . : 10.1.1.1
   DHCP Server . . . . . . . . . . . : 10.1.12.2
   DHCPv6 IAID . . . . . . . . . . . : 100720716
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-19-A0-DC-F0-24-FD-52-CB-50-C9
   DNS Servers . . . . . . . . . . . : 208.67.222.222
                                       208.67.222.220
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

While Example 19-6 shows the traditional **ipconfig** Windows command, Example 19-7 shows a similar command as part of the Windows network shell. The network shell commands replace some of the older Windows command-line commands like **ipconfig**.

**Example 19-7** *Windows Network Shell* **show config** *Command*

```
Windows_A> netsh interface ip show config

Configuration for interface "Ethernet 5"
   DHCP enabled:                         Yes
   IP Address:                           10.1.1.103
   Subnet Prefix:                        10.1.1.0/24 (mask 255.255.255.0)
   Default Gateway:                      10.1.1.1
   Gateway Metric:                       0
   InterfaceMetric:                      25
   DNS servers configured through DHCP:  208.67.222.222
                                         208.67.222.220
   Register with which suffix:           Primary only
   WINS servers configured through DHCP: None

! Next, use the network shell to allow repeated commands.
Windows_A> netsh
netsh> interface ip
netsh interface ip> show config

Configuration for interface "Ethernet 5"
   DHCP enabled:                         Yes
   IP Address:                           10.1.1.103
   Subnet Prefix:                        10.1.1.0/24 (mask 255.255.255.0)
! Remaining lines omitted; same output as at the top of the example.

netsh interface ip> show dnsservers

Configuration for interface "Ethernet 5"
   DNS servers configured through DHCP:  208.67.222.222
                                         208.67.222.220
   Register with which suffix:           Primary only
```

Interestingly, you can issue network shell commands as a single command (as seen at the top of Example 19-7) or by navigating within the network shell interface. The example shows the use of the **netsh** command and then **interface ip** so that you reach a mode where you can use short versions of many IP-related commands, like the **show config** and **show dnsservers** commands. (Note that all output has been edited to show only the entries related to the Ethernet interface used in the example.)

You should also look at the host's IP routing table, focusing on two routes that direct most routing in the host. When using DHCP, the host creates these routes based on information learned from the DHCP server:

- A default route, with the default gateway as the next-hop address, with the destination subnet and mask listed as 0.0.0.0 and 0.0.0.0.
- A route to the subnet connected to the working interface, subnet 10.1.1.0/24 in this case, calculated from the interface IP address and mask.

Example 19-8 displays Windows host A's routing table with two commands: the older **netstat -rn** command and the newer **netsh interface ip show route** command. The example has been edited to list only routes related to subnet 10.1.1.0/24 and the Ethernet LAN interface used in the example. Note that a gateway of "on-link" means that the PC thinks the destination is on the local subnet (link).

Key Topic

**Example 19-8** *Windows Host IPv4 Routes*

```
Windows_A> netstat -rn
IPv4 Route Table
===========================================================================
Active Routes:
Network Destination         Netmask    Gateway    Interface  Metric
          0.0.0.0           0.0.0.0    10.1.1.1   10.1.1.103     25
         10.1.1.0     255.255.255.0    On-link    10.1.1.103    281
       10.1.1.103   255.255.255.255    On-link    10.1.1.103    281
       10.1.1.255   255.255.255.255    On-link    10.1.1.103    281
! Lines omitted for brevity
Windows_A> netsh interface ip show route
Publish Type      Met Prefix                    Idx Gateway/Interface Name
------- --------- --- ------------------------ --- ------------------------
No      Manual    0   0.0.0.0/0                6 10.1.1.1
No      System    256 10.1.1.0/24              6 Ethernet 5
No      System    256 10.1.1.103/32            6 Ethernet 5
No      System    256 10.1.1.255/32            6 Ethernet 5
!       Lines     omitted for brevity
```

## Host IP Settings on macOS

Although the details vary as compared to Windows, macOS has a graphical interface to see network settings and a variety of network commands. This section shows examples of each, beginning with Figure 19-11. It shows the network settings in macOS for an Ethernet interface, with the address, mask, default router, and DNS server addresses. Also note the setting states that the interface is using DHCP.

Both macOS and Linux support the **ifconfig** command to list information similar to the Windows **ipconfig /all** command. However, the **ifconfig** command lacks an option like **/all**, listing no information about the default gateway or DNS servers. Example 19-9 gives an example of the **ifconfig** command from Mac Host B in Figure 19-11, along with another command that lists the default router.

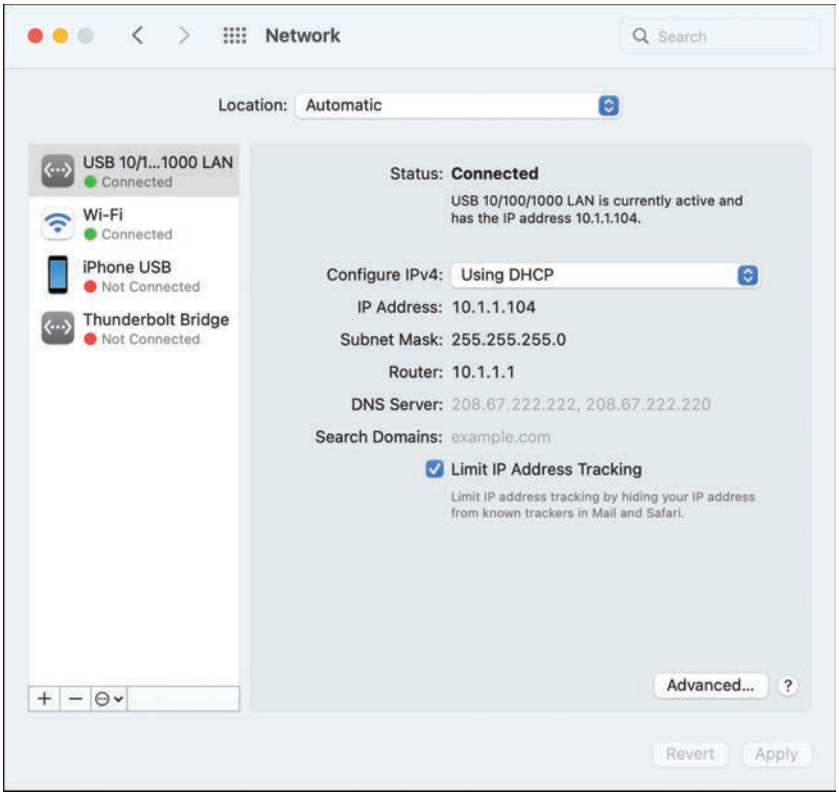



**Figure 19-11** *macOS DHCP Client IP Address, Mask, and Default Router Settings*

Key Topic

**Example 19-9** ifconfig *and* networksetup -getinfo *(macOS)*

```
Mac_B$ ifconfig en8
 en8: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
       options=6467<RXCSUM,TXCSUM,VLAN_MTU,TSO4,TSO6,CHANNEL_IO,PARTIAL_CSUM,
       ZEROINVERT_CSUM>
          ether 00:e0:4c:68:1f:26
          inet6 fe80::184c:56f9:fd3b:d6e7%en8 prefixlen 64 secured scopeid 0x14
          inet 10.1.1.104 netmask 0xffffff00 broadcast 10.1.1.255
          nd6 options=201<PERFORMNUD,DAD>
          media: autoselect (1000baseT <full-duplex>)
          status: active

Mac_B$ networksetup -getinfo "USB 10/100/1000 LAN"
DHCP Configuration
IP address: 10.1.1.104
Subnet mask: 255.255.255.0
Router: 10.1.1.1
Client ID:
```

```
IPv6: Automatic
IPv6 IP address: none
IPv6 Router: none
Ethernet Address: 00:e0:4c:68:1f:26
```

Focusing on the **ifconfig** command, it lists the subnet mask in hex, listing 0xffffff00 in the example. To convert the mask to dotted decimal, convert each pair of hex digits to binary and then from binary to the decimal equivalent, giving you four octets for the dotted-decimal mask. In this case, the 0xffffff00 begins with three pairs of ff, which convert to binary 11111111. Each binary 11111111 converts to decimal 255: the last hex pair, 00, converts to binary 00000000, which converts to decimal 0. The final decimal mask is the familiar value 255.255.255.0.

The **networksetup** command in Example 19-9 lists more detail than the older **ifconfig** command. It lists the usual IP settings, other than the DNS server list. Note that the **networksetup** macOS command has a large variety of options.

Like Windows, macOS adds a default route to its host routing table based on the default gateway and a route to the local subnet calculated based on the IP address and mask. And like Windows, macOS uses the **netstat -rn** command to list those routes—but with several differences in the output. Of note, in the macOS sample shown in Example 19-10, the output represents the default route using the word *default* rather than the destination subnet and mask of 0.0.0.0 and 0.0.0.0. The example highlights the default route and the route to the connected subnet. As usual, the example aids readability by limiting the output to those routes related to network 10.0.0.0.

**Example 19-10** **netstat -rn** *Command (macOS)*

```
Mac_B> netstat -rn
Routing tables

Internet:
Destination      Gateway            Flags        Netif Expire
default          10.1.1.1           UGScg        en8
10.1.1/24        link#20            UCS          en8         !
10.1.1.1/32      link#20            UCS          en8         !
10.1.1.1         2:0:11:11:11:11    UHLWIir      en8       385
10.1.1.104/32    link#20            UCS          en8         !
! lines omitted for brevity
```

## Host IP Settings on Linux

As with the other desktop OSs, Linux shows networking settings from the GUI as well as with commands. However, be aware that the Linux world includes many different Linux distributions. Additionally, the Linux architecture separates the OS from the desktop (the graphical interface). So, other Linux users may use different GUI administration tools and commands to see network settings.

Figure 19-12 shows an example from the Ubuntu Linux distribution using the Mate desktop (www.ubuntu-mate.org). The host, Host C in Figure 19-9, uses DHCP and learns an address

in subnet 10.1.1.0/24. Figure 19-12 shows IP details about an Ethernet adapter and includes the IPv4 address, mask, default router, and DNS IP addresses.

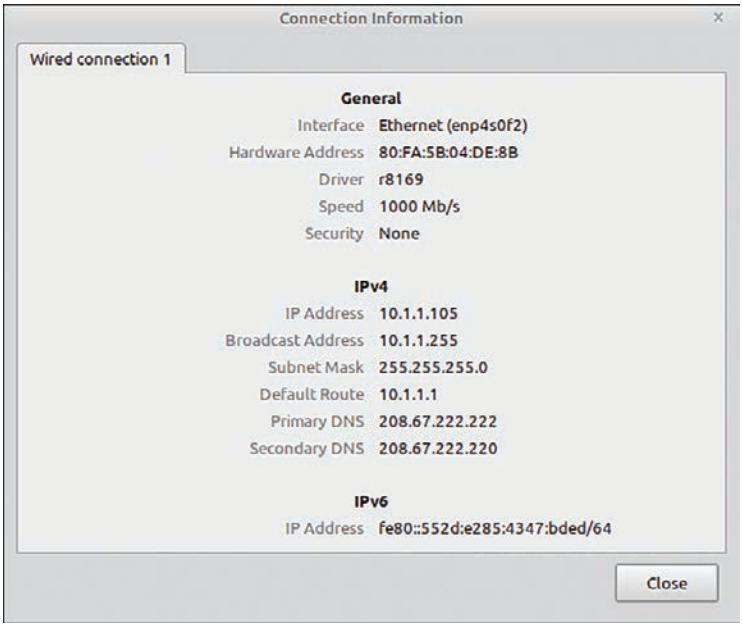



**Figure 19-12** *Linux DHCP Client IP Address, Mask, and Default Router Settings*

Linux hosts often support a large set of commands from the command line. However, an older set of commands, referenced together as *net-tools*, has been deprecated in Linux to the point that some Linux distributions do not include net-tools. (You can easily add net-tools to most Linux distributions.) The net-tools library has popular commands like **ifconfig** and **netstat -rn**. To replace those tools, Linux uses the *iproute* library, which includes a set of replacement commands and functions, many performed with the **ip** command and some parameters.

Example 19-11 shows a sample of the Linux **ifconfig** command and the replacement command **ip address** for the same interface detailed in Figure 19-12. Both commands list the Ethernet MAC and IPv4 addresses, along with the subnet mask.

**Example 19-11** **ifconfig** *and* **ip address** *Commands (Linux)*

```
Linux_C$ ifconfig
enp4s0f2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
        inet 10.1.1.105 netmask 255.255.255.0 broadcast 10.1.1.255
        inet6 fe80::552d:e285:4347:bded prefixlen 64 scopeid 0x20<link>
        ether 80:fa:5b:04:de:8b txqueuelen 1000 (Ethernet)
        RX packets 4958 bytes 384498 (384.4 KB)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 16270 bytes 1320108 (1.3 MB)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
Linux_C$ ip address
2: enp4s0f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 80:fa:5b:04:de:8b brd ff:ff:ff:ff:ff:ff
    inet 10.1.1.105/24 brd 10.1.1.255 scope global dynamic noprefixroute enp4s0f2
      valid_lft 81056sec preferred_lft 81056sec
    inet6 fe80::552d:e285:4347:bded/64 scope link noprefixroute
      valid_lft forever preferred_lft forever
```

Example 19-12 shows another pair of older and newer Linux commands: the **netstat -rn** and **ip route** commands. The **netstat -rn** command lists the default route with a style that shows the destination and mask both as 0.0.0.0. As usual, the default route uses the learned default gateway IP address (10.1.1.1) as the next-hop IP address. The **netstat -rn** output also lists a route to the local subnet (10.1.1.0/24). The **ip route** command lists those same routes but with much different formatting and phrasing.

**Example 19-12** **netstat -rn** *and* **ip route** *Commands (Linux)*

```
Linux_C$ netstat -rn
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
0.0.0.0         10.1.1.1        0.0.0.0         UG        0 0          0 enp4s0f2
10.1.1.0        0.0.0.0         255.255.255.0   U         0 0          0 enp4s0f2
! Lines omitted for brevity

Linux_C$ ip route
default via 10.1.1.1 dev enp4s0f2 proto dhcp metric 20100
10.1.1.0/24 dev enp4s0f2 proto kernel  scope link  src 10.1.1.105 metric 100
! Lines omitted for brevity
```

## Troubleshooting Host IP Settings

The examples for Windows, macOS, and Linux shown over the last few pages show the successful results for DHCP clients A, B, and C per Figure 19-9. To round out your perspective, the final few examples in this section show a few different scenarios with the resulting output from the popular Windows **ipconfig /all** command. The scenarios, all from Windows Host A in the figure, are as follows:

1. Host A with static IP configuration, using correct settings.
2. Host A as a DHCP client, but with a network issue that prevents the DHCP client and server from communicating.
3. Host A as a DHCP client with a working network so that DHCP works, but with the DHCP server supplying some incorrect information (wrong default gateway address).

### A Working Windows Host with Static IP Configuration

Example 19-13 shows the first scenario. Someone has disabled DHCP on Windows Host A and configured all IP settings manually. They chose well, using correct values per

Figure 19-9. In particular, the figure shows addresses .1 – .100 reserved for static addresses, so the person who manually configured Host A assigned 10.1.1.99.

Key Topic

**Example 19-13** *Working Windows Host with Static IP Setting (No DHCP)*

```
Windows_A> ipconfig /all
! Showing only the Ethernet adapter connected to subnet 10.1.1.0/24
Ethernet adapter Ethernet 5:

   Connection-specific DNS Suffix .  :
   Description . . . . . . . . . . . : Realtek USB GbE Family Controller
   Physical Address. . . . . . . . . : 00-E0-4C-69-08-47
   DHCP Enabled. . . . . . . . . . . : No
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::6878:1f3d:6223:f4db%6(Preferred)
   IPv4 Address. . . . . . . . . . . : 10.1.1.99(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.1.1.1
   DHCPv6 IAID . . . . . . . . . . . : 100720716
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-19-A0-DC-F0-24-FD-52-CB-50-C9
   DNS Servers . . . . . . . . . . . : 208.67.222.222
                                       208.67.222.220
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

The **ipconfig /all** command output in Example 19-13 implies that the host uses static settings by noting that DHCP is disabled (per the first highlighted line). Take a moment to compare that line to the same line back in Example 19-7, in which Host A acts as a DHCP client.

The output here in Example 19-13 also signals that the host uses a static setting rather than DHCP, but it tells us that by the absence of several lines of output usually shown for DHCP clients, omitting the lines for

- DHCP Enabled
- Lease Obtained
- Lease Expires

Knowing those facts, you should be able to distinguish between cases in which the Windows host does and does not use DHCP.

### A Failed Windows DHCP Client Due to IP Connectivity Issues

The following example, again taken from Windows Host A, shows a case in which the host similarly acts as a DHCP client to follow the design from Figure 19-9. However, any problem that prevents the DHCP DORA messages from flowing between the client and DHCP server results in a failure of the DHCP process. The client does not lease an address or learn the other IP details from the DHCP server, but it self-assigns an APIPA address and uses the APIPA default mask of 255.255.0.0. (For a review of APIPA, see the section titled "APIPA IP Addresses (169.254.x.x)" earlier in this chapter.)

Example 19-14 shows the resulting **ipconfig /all** command. The output confirms the client enables DHCP. However, the following facts identify that the process failed:

- It omits the output line that would list the DHCP server's IP address.
- It omits the two output lines that list DHCP lease details.
- It shows an APIPA IP address that begins 169.254.
- It lists no IPv4 default gateway address.
- It lists no IPv4 DNS servers.

**Example 19-14** *Windows DHCP Client Fails Due to IP Connectivity Failure*

```
Windows_A> ipconfig /all
! Showing only the Ethernet adapter connected to subnet 10.1.1.0/24
Ethernet adapter Ethernet 5:

   Connection-specific DNS Suffix .  :
   Description . . . . . . . . . . . : Realtek USB GbE Family Controller
   Physical Address. . . . . . . . . : 00-E0-4C-69-08-47
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::6878:1f3d:6223:f4db%6(Preferred)
   Autoconfiguration IPv4 Address. . : 169.254.244.219(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.0.0
   Default Gateway . . . . . . . . . :
   DHCPv6 IAID . . . . . . . . . . . : 100720716
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-19-A0-DC-F0-24-FD-52-CB-50-C9
   DNS Servers . . . . . . . . . . . : fec0:0:0:ffff::1%1
                                       fec0:0:0:ffff::2%1
                                       fec0:0:0:ffff::3%1
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

### A Working Windows DHCP Client with Incorrect Settings

For one final scenario, imagine a case in which the DHCP process works but the server configuration is incorrect. As a result, the DHCP client can learn incorrect information and not be able to communicate in some cases. For instance, the DHCP process could complete with the client learning these inaccurate or incomplete settings:

- Wrong subnet mask
- Wrong or missing default gateway
- Wrong or missing DNS server list

Example 19-15 shows a case with the incorrect default gateway (10.1.1.254) configured in the DHCP server's settings for subnet 10.1.1.0/24. The correct default gateway per the design is Router R1 (10.1.1.1). The **ipconfig /all** output in Example 19-15 shows the usual indications

that DHCP worked, showing it as enabled, with a lease start and end time. It also shows the host's IP address, mask, default gateway, and DNS servers. But compared to the intended design, the output shows that the host learned a default gateway setting for a nonexistent router.

**Example 19-15** *Windows DHCP Client Learns Incorrect Default Gateway*

```
Windows_A> ipconfig /all

Ethernet adapter Ethernet 5:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Realtek USB GbE Family Controller
   Physical Address. . . . . . . . . : 00-E0-4C-69-08-47
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::6878:1f3d:6223:f4db%6(Preferred)
   IPv4 Address. . . . . . . . . . . : 10.1.1.103(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Tuesday, October 25, 2022 7:54:17 AM
   Lease Expires . . . . . . . . . . : Wednesday, October 26, 2022 7:54:16 AM
   Default Gateway . . . . . . . . . : 10.1.1.254
   DHCP Server . . . . . . . . . . . : 10.1.12.2
   DHCPv6 IAID . . . . . . . . . . . : 100720716
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-19-A0-DC-F0-24-FD-52-CB-50-C9
   DNS Servers . . . . . . . . . . . : 208.67.222.222
                                       208.67.222.220
   NetBIOS over Tcpip. . . . . . . . : Enabled

Windows_A> netstat -rn

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination          Netmask      Gateway     Interface  Metric
          0.0.0.0          0.0.0.0   10.1.1.254     10.1.1.103      25
         10.1.1.0    255.255.255.0      On-link     10.1.1.103     281
! Lines omitted for brevity
```

The underlying solution to this problem is to fix the configuration mistake in the DHCP server. For the exam, the more likely scenario would be for you to think about the design and notice a difference between what the DHCP client learned and the correct settings. Also, note that the next chapter, "Troubleshooting IPv4 Routing," gives some insights into how to explore different symptoms for problems in an IP network.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 19-2 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 19-2** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 19-3** Key Topics for Chapter 19

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Definitions of special IPv4 addresses 0.0.0.0 and 255.255.255.255 | 489 |
| List | Four logic steps created by the **ip helper-address** command | 491 |
| Figure 19-2 | What the **ip helper-address** command changes in a DHCP Discover message | 491 |
| List | The two facts that must be true about a subnet for a router to need to be a DHCP relay agent for that subnet | 494 |
| Example 19-4 | Switch commands that confirm the details of DHCP client operations based on the **ip address dhcp** interface subcommand | 496 |
| List | The IPv4 settings expected on an end-user host | 497 |
| Example 19-6 | Output from a Windows **ipconfig /all** command when the host successfully uses DHCP | 500 |
| Example 19-8 | IP routes on a Windows host | 502 |
| Example 19-9 | Output from a macOS **ifconfig** command plus two **networksetup** commands | 503 |
| Example 19-13 | Output from a Windows **ipconfig /all** command when the host uses static IP configuration | 507 |

## Key Terms You Should Know

APIPA, default gateway, DHCP client, DHCP relay agent, DHCP server, DNS server, DNS server list

## Command References

Tables 19-4, 19-5, and 19-6 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 19-4** Chapter 19 Configuration Command Reference

| Command | Description |
|---|---|
| **ip helper-address** *IP-address* | An interface subcommand that tells the router to notice local subnet broadcasts (to 255.255.255.255) that use UDP, and change the source and destination IP address, enabling DHCP servers to sit on a remote subnet |
| **ip address dhcp** | An interface subcommand that tells the router or switch to use DHCP to attempt to lease a DHCP address from a DHCP server |

**Table 19-5** Chapter 19 EXEC Command Reference

| Command | Description |
|---|---|
| **show arp**, **show ip arp** | Command that lists the router's IPv4 ARP table |
| **show dhcp lease** | Switch command that lists information about addresses leased because of the configuration of the **ip address dhcp** command |
| **show ip default-gateway** | Switch command that lists the switch's default gateway setting, no matter whether learned by DHCP or statically configured |

**Table 19-6** Chapter 19 Generic Host Networking Command Reference

| Command | Description |
|---|---|
| **ipconfig /all** | (Windows) Lists IP address, mask, gateway, and DNS servers |
| **ifconfig** | (Mac, Linux) Lists IP address and mask for an interface |
| **networksetup -getinfo** *interface* | (Mac) Lists IP settings including default router |
| **netstat -rn** | (Windows, Mac, Linux) Lists the host's routing table, including a default route that uses the DHCP-learned default gateway |
| **arp -a** | (Windows, Mac, Linux) Lists the host's ARP table |
| **ip address** | (Linux) Lists IP address and mask information for interfaces; the Linux replacement for **ifconfig** |
| **ip route** | (Linux) Lists routes, including the default route and a route to the local subnet; the Linux replacement for **netstat -rn** |
| **netsh interface ip show addresses** | (Windows) Windows network shell command to list interface IP address configuration settings; a replacement for the **ipconfig /all** command |
| **netsh interface ip show route** | (Windows) Windows network shell command to list IPv4 routes; a replacement for the **netstat -rn** command |



CHAPTER 20

# Troubleshooting IPv4 Routing

**This chapter covers the following exam topics:**

- **2.0 Network Access**
  - **2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, TACACS+/RADIUS, and cloud managed)**
- **3.0 IP Connectivity**
  - **3.3 Configure and verify IPv4 and IPv6 static routing**
    - **3.3.a Default route**
    - **3.3.b Network route**
    - **3.3.c Host route**
    - **3.3.d Floating static**

This chapter turns our attention to routing from end-to-end across an entire enterprise network. How do you troubleshoot an IPv4 network? How do you verify correct operation, identify root causes, and fix those for various IP routing features? How do you do that in the presence of an IP addressing and subnetting plan, requiring you to apply all that subnetting math from Part IV of this book and the basic address/mask and static route configuration from the other chapters here in Part V? This chapter answers some of those questions.

In particular, this chapter focuses on two tools and how to use them: *ping* and *traceroute*. Both tools test the IPv4 data plane; that is, the ability of each networking device to route or forward IPv4 packets. This chapter devotes a major section each to **ping** and **traceroute**. The chapter then ends with a short discussion of two other router tools that can also be useful for troubleshooting: Telnet and Secure Shell (SSH).

## "Do I Know This Already?" Quiz

I put DIKTA quizzes in most of the chapters as a tool to help you decide how to approach reading a chapter. However, this chapter does not have a DIKTA quiz because I think you should read it regardless of your prior knowledge. As with all chapters in this book, this chapter introduces new concepts, but it also acts as a tool to review and deepen your understanding of IP routing. Hope you enjoy the perspectives on using **ping** and **traceroute** in this chapter.

# Foundation Topics

## Problem Isolation Using the ping Command

Someone sends you an email or text, or a phone message, asking you to look into a user's network problem. You Secure Shell (SSH) to a router and issue a **ping** command that works. What does that result rule out as a possible reason for the problem? What does it rule in as still being a possible root cause?

Then you issue another **ping** to another address, and this time the ping fails. Again, what does the failure of that **ping** command tell you? What parts of IPv4 routing may still be a problem, and what parts do you now know are not a problem?

The **ping** command gives us one of the most common network troubleshooting tools. When the **ping** command succeeds, it confirms many individual parts of how IP routing works, ruling out some possible causes of the current problem. When a **ping** command fails, it often helps narrow down where in the internetwork the root cause of the problem may be happening, further isolating the problem.

This section begins with a brief explanation of how ping works. It then moves on to some suggestions and analysis of how to use the **ping** command to isolate problems by removing some items from consideration.

### Ping Command Basics

The **ping** command tests connectivity by sending packets to an IP address, expecting the device at that address to send packets back. The command sends packets that mean "if you receive this packet, and it is addressed to you, send a reply back." Each time the **ping** command sends one of these packets and receives the message sent back by the other host, the **ping** command knows a packet made it from the source host to the destination and back.

More formally, the **ping** command uses the Internet Control Message Protocol (ICMP), specifically the **ICMP echo request** and **ICMP echo reply** messages. ICMP defines many other messages as well, but these two messages were made specifically for connectivity testing by commands like **ping**. As a protocol, ICMP does not rely on TCP or UDP, and it does not use any application layer protocol. It functions as part of Layer 3, as a control protocol to assist IP by helping manage the IP network functions.

Figure 20-1 shows the ICMP messages, with IP headers, in an example. In this case, the user at host A opens a command prompt and issues the **ping 172.16.2.101** command, testing connectivity to host B. The command sends one echo request and waits (Step 1); host B receives the messages and sends back an echo reply (Step 2).

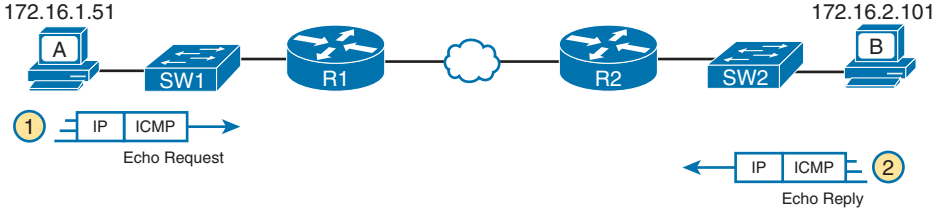



**Figure 20-1** *Concept Behind* **ping 172.16.2.101** *on Host A*

The **ping** command is supported on many different devices and many common operating systems. The command has many options: the name or IP address of the destination, how many times the command should send an echo request, how long the command should wait (timeout) for an echo reply, how big to make the packets, and many other options. Example 20-1 shows a sample from host A, with the same command that matches the concept in Figure 20-1: a **ping 172.16.2.101** command on host A.

**Example 20-1** *Sample Output from Host A's* **ping 172.16.2.101** *Command*

```
Mac_A$ ping 172.16.2.101
PING 172.16.2.101 (172.16.2.101): 56 data bytes
64 bytes from 172.16.2.101: icmp_seq=0 ttl=64 time=1.112 ms
64 bytes from 172.16.2.101: icmp_seq=1 ttl=64 time=0.673 ms
64 bytes from 172.16.2.101: icmp_seq=2 ttl=64 time=0.631 ms
64 bytes from 172.16.2.101: icmp_seq=3 ttl=64 time=0.674 ms
64 bytes from 172.16.2.101: icmp_seq=4 ttl=64 time=0.642 ms
64 bytes from 172.16.2.101: icmp_seq=5 ttl=64 time=0.656 ms
^C
--- 172.16.2.101 ping statistics ---
6 packets transmitted, 6 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.631/0.731/1.112/0.171 ms
```

## Strategies and Results When Testing with the ping Command

Often, the person handling initial calls from users about problems (often called a customer support rep, or CSR) cannot issue **ping** commands from the user's device. In some cases, talking users through typing the right commands and making the right clicks on their machines can be a problem. Or the user just might not be available. As an alternative, using different **ping** commands from different routers can help isolate the problem.

The problem with using **ping** commands from routers, instead of from the host that has the problem, is that no single router **ping** command can exactly replicate a **ping** command done from the user's device. However, each different **ping** command can help isolate a problem further. The rest of this section of **ping** commands discusses troubleshooting IPv4 routing by using various **ping** commands from the command-line interface (CLI) of a router.

### Testing Longer Routes from Near the Source of the Problem

Most problems begin with some idea like "host X cannot communicate with host Y." A great first troubleshooting step is to issue a **ping** command from X for host Y's IP address. However, assuming the engineer does not have access to host X, the engineer can instead issue the **ping** from the router nearest X, typically the router acting as host X's default gateway.

For instance, in Figure 20-1, imagine that the user of host A had called IT support with a problem related to sending packets to host B. A **ping 172.16.2.101** command on host A would be a great first troubleshooting step, but the CSR cannot access host A or get in touch with the user of host A. So, the CSR telnets to Router R1 and pings host B from there, as shown in Example 20-2.

**Example 20-2** *Router R2 Pings Host B (Two Commands)*

```
R1# ping 172.16.2.101
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.2.101, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 1/2/4 ms
R1# ping 172.16.2.101
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.2.101, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/2/4 ms
```

First, take a moment to review the output of the first IOS **ping** command. By default, the Cisco IOS **ping** command sends five echo messages, with a timeout of 2 seconds. If the command does not receive an echo reply within 2 seconds, the command considers that message to be a failure, and the command lists a period. If a successful reply is received within 2 seconds, the command displays an exclamation point. So, in this first command, the first echo reply timed out, whereas the other four received a matching echo reply within 2 seconds.

As a quick aside, the example shows a common and normal behavior with **ping** commands: the first **ping** command shows one failure to start, but then the rest of the messages work. This usually happens because some device in the end-to-end route is missing an ARP table entry.

Now think about troubleshooting and what a working **ping** command tells us about the current behavior of this internetwork. First, focus on the big picture for a moment:

- R1 can send ICMP echo request messages to host B (172.16.2.101).
- R1 sends these messages from its outgoing interface's IP address (by default), 172.16.4.1 in this case.
- Host B can send ICMP echo reply messages to R1's 172.16.4.1 IP address (hosts send echo reply messages to the IP address from which the echo request was received).

Figure 20-2 shows the packet flow.

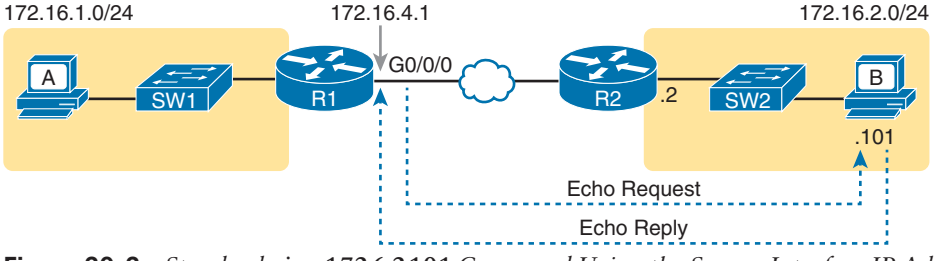



**Figure 20-2** *Standard* **ping 172.6.2.101** *Command Using the Source Interface IP Address*

Next, think about IPv4 routing. In the forward direction, R1 must have a route that matches host B's address (172.16.2.101); this route will be either a static route or one learned with a

routing protocol. R2 also needs a route for host B's address, in this case a connected route to B's subnet (172.16.2.0/24), as shown in the top arrow lines in Figure 20-3.

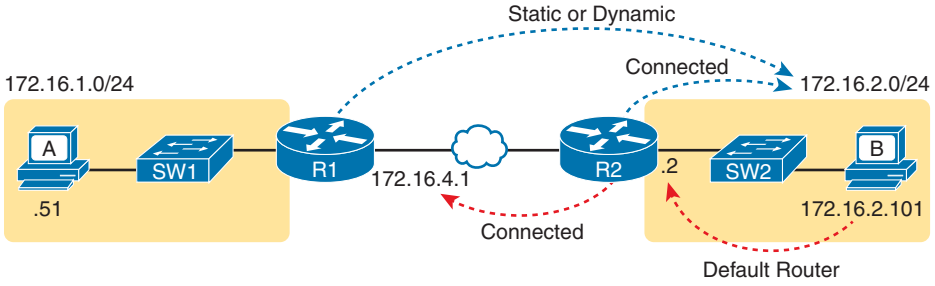



**Figure 20-3** *Layer 3 Routes Needed for R1's* **ping 172.16.2.101** *to Work*

The arrow lines on the bottom of Figure 20-3 show the routes needed to forward the ICMP echo reply message back to Router R1's 172.16.4.1 interface. First, host B must have a valid default router setting because 172.16.4.1 sits in a different subnet than host B. R2 must also have a route that matches destination 172.16.4.1 (in this case, likely to be a connected route).

The working **ping** commands in Example 20-2 also require the data-link and physical layer details to be working. The WAN link must be working: The router interfaces must be up/up, which typically indicates that the link can pass data. On the LAN, R2's LAN interface must be in an up/up state. In addition, everything discussed about Ethernet LANs must be working because the **ping** confirmed that the packets went all the way from R1 to host B and back. In particular

- The switch interfaces in use are in a connected (up/up) state.
- Port security (discussed in the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition) does not filter frames sent by R2 or host B.
- STP has placed the right ports into a forwarding state.

The **ping 172.16.2.101** command in Example 20-2 also confirms that IP access control lists (ACLs) did not filter the ICMP messages. One ACL contains a set of matching rules and actions: some matched packets are filtered (discarded), while others can continue on their path as normal. ACLs can examine packets as they enter or exit a router interface, so Figure 20-4 shows the various locations on routers R1 and R2 where an ACL could have filtered (discarded) the ICMP messages. (Note that an outbound ACL on Router R1 would not filter packets created on R1, so there is no rightward-facing arrow over R1.)

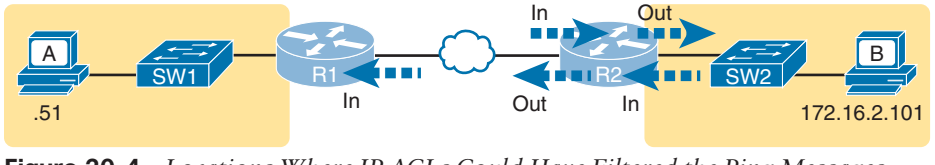



**Figure 20-4** *Locations Where IP ACLs Could Have Filtered the Ping Messages*

Finally, the working **ping 172.16.2.101** command on R1 can also be used to reasonably predict that ARP worked and that switch SW2 learned MAC addresses for its MAC address

table. R2 and host B need to know each other's MAC addresses so that they can encapsulate the IP packet inside an Ethernet frame, which means both must have a matching ARP table entry. The switch learns the MAC address used by R2 and by host B when it sends the ARP messages or when it sends the frames that hold the IP packets. Figure 20-5 shows the type of information expected in those tables.

Key Topic

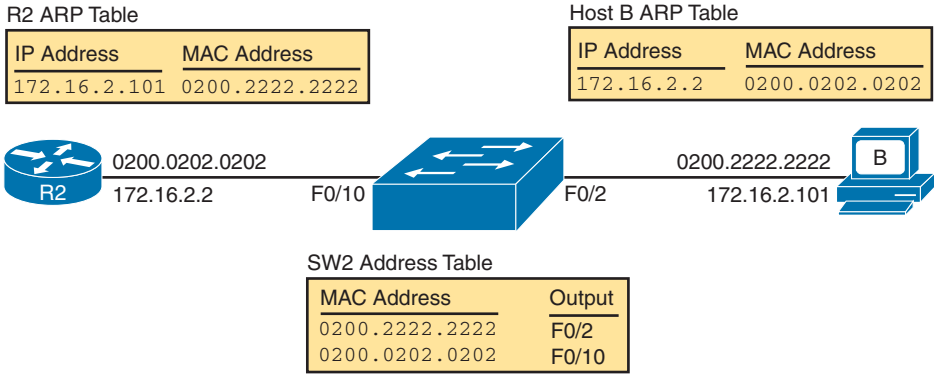



**Figure 20-5** *Router and Host ARP Tables, with the Switch MAC Address Table*

As you can see from the last few pages, a strategy of using a **ping** command from near the source of the problem can rule out a lot of possible root causes of any problems between two hosts—assuming the **ping** command succeeds. However, this **ping** command does not act exactly like the same **ping** command on the actual host. To overcome some of what is missing in the **ping** command from a nearby router, the next several examples show some strategies for testing other parts of the path between the two hosts that might have a current problem.

## Using Extended Ping to Test the Reverse Route

Pinging from the default router, as discussed in the past few pages, misses an opportunity to test IP routes more fully. Such tests check the **forward route**, that is, the route toward the destination. However, a ping from the default route does not test the **reverse route** back toward the original host.

For instance, referring to the internetwork in Figure 20-2 again, note that the reverse routes do not point to an address in host A's subnet. When R1 processes the **ping 172.16.2.101** command, R1 has to pick a source IP address to use for the echo request, and routers choose the *IP address of the outgoing interface*. The echo request from R1 to host B flows with source IP address 172.16.4.1 (R1's G0/0/0 IP address). The echo reply flows back to that same address (172.16.4.1).

A standard ping often does not test the reverse route that you need to test. In this case, the standard **ping 172.16.2.101** command on R1 does not test whether the routers can route back to subnet 172.16.1.0/24, instead testing their routes for subnet 172.16.4.0. A better ping test would test the route back to host A's subnet; an *extended ping* from R1 can cause that test to happen. An extended ping allows R1's **ping** command to use R1's LAN IP address from within subnet 172.16.1.0/24. Then, the echo reply messages would flow to host A's subnet, as shown in Figure 20-6.

Key Topic

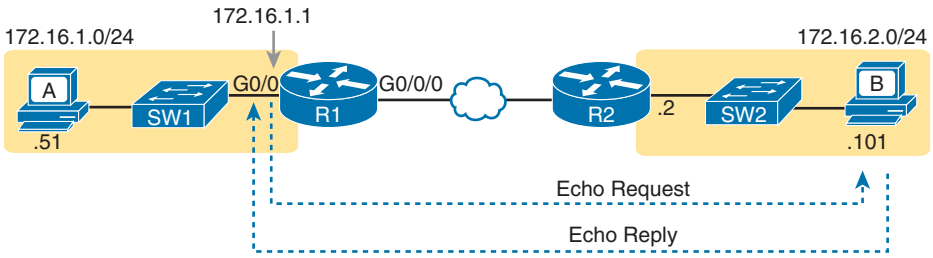



**Figure 20-6** *Extended Ping Command Tests the Route to 172.16.1.51 (Host A)*

The extended **ping** command does allow the user to type all the parameters on a potentially long command, but it also allows users to simply issue the **ping** command, press Enter, with IOS then asking the user to answer questions to complete the command, as shown in Example 20-3. The example shows the **ping** command on R1 that matches the logic in Figure 20-6. This same command could have been issued from the command line as **ping 172.16.2.101 source 172.16.1.1**.

**Example 20-3** *Testing the Reverse Route Using the Extended Ping*

```
R1# ping
Protocol [ip]:
Target IP address: 172.16.2.101
Repeat count [5]:
Datagram size [100]:
Timeout in seconds [2]:
Extended commands [n]: y
Source address or interface: 172.16.1.1
Type of service [0]:
Set DF bit in IP header? [no]:
Validate reply data? [no]:
Data pattern [0xABCD]:
Loose, Strict, Record, Timestamp, Verbose[none]:
Sweep range of sizes [n]:
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.2.101, timeout is 2 seconds:
Packet sent with a source address of 172.16.1.1
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/2/4 ms
```

This particular extended **ping** command tests the same routes for the echo request going to the right, but it forces a better test of routes pointing back to the left for the ICMP echo reply. For that direction, R2 needs a route that matches address 172.16.1.1, which is likely to be a route for subnet 172.16.1.0/24—the same subnet in which host A resides.

From a troubleshooting perspective, using both standard and extended **ping** commands can be useful. However, neither can exactly mimic a **ping** command created on the host itself because the routers cannot send packets with the host's IP address. For instance, the

extended **ping** in Example 20-3 uses source IP address 172.16.1.1, which is not host A's IP address. As a result, neither the standard nor extended **ping** commands in these two examples so far in this chapter can test for some kinds of problems, such as the following:

- IP ACLs that discard packets based on host A's IP address but allow packets that match the router's IP address
- LAN switch port security that filters A's frames (based on A's MAC address)
- IP routes on routers that happen to match host A's 172.16.1.51 address, with different routes that match R1's 172.16.1.1 address
- Problems with host A's default router setting

**NOTE** IP ACLs and LAN switch port security are covered in the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition. For now, know that IP ACLs can filter packets on routers, focusing on the Layer 3 and 4 headers. Port security can be enabled on Layer 2 switches to filter based on source MAC addresses.

### Testing LAN Neighbors with Standard Ping

Testing using a **ping** of another device on the LAN can quickly confirm whether the LAN can pass packets and frames. Specifically, a working **ping** rules out many possible root causes of a problem. For instance, Figure 20-7 shows the ICMP messages that occur if R1 issues the command **ping 172.16.1.51**, pinging host A, which sits on the same VLAN as R1.

Key Topic

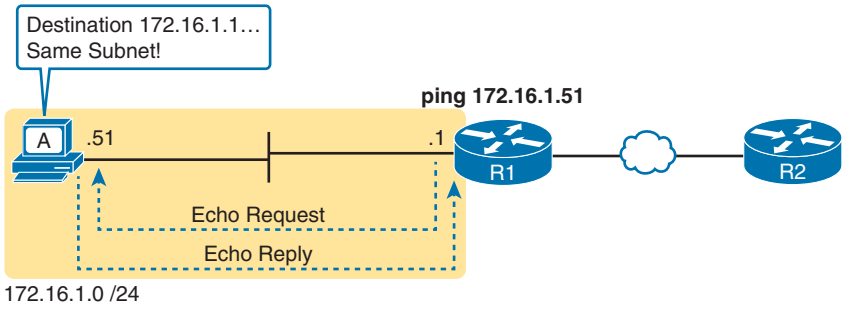



**Figure 20-7** *Standard* **ping** *Command Confirms That the LAN Works*

If the ping works, it confirms the following, which rules out some potential issues:

- The host with address 172.16.1.51 replied.
- The LAN can pass unicast frames from R1 to host 172.16.1.51 and vice versa.
- You can reasonably assume that the switches learned the MAC addresses of the router and the host, adding those to the MAC address tables.
- Host A and Router R1 completed the ARP process and list each other in their respective Address Resolution Protocol (ARP) tables.

The failure of a ping, even with two devices on the same subnet, can point to a variety of problems, like those mentioned in this list. For instance, if the **ping 172.16.1.51** on R1 fails (refer to Figure 20-7), that result points to this list of potential root causes:



- **IP addressing problem:** Host A or the router could be configured with the wrong IP address.
- **IP mask problem:** Using an incorrect subnet mask on either the host or the router would change their calculation view of the range of addresses in the attached subnet, which would affect their forwarding logic. For example, the host, with address 172.16.1.51 but incorrect mask 255.255.255.240, would think that the router's address of 172.16.1.1 is in a different subnet.
- **DHCP problems:** If you are using Dynamic Host Configuration Protocol (DHCP), many problems could exist. Chapter 19, "IP Addressing on Hosts," discusses those possibilities in some depth.
- **VLAN trunking problems:** The router could be configured for 802.1Q trunking, when the switch is not (or vice versa).
- **LAN problems:** A wide variety of issues could exist with the Layer 2 switches, preventing any frames from flowing between host A and the router.

So, whether the ping works or fails, simply pinging a LAN host from a router can help further isolate the problem.

### Testing LAN Neighbors with Extended Ping

A standard ping of a LAN host from a router does not test that host's default router setting. However, an extended ping can test the host's default router setting. Both tests can be useful, especially for problem isolation, because



- If a standard ping of a local LAN host works…
- But an extended ping of the same LAN host fails…
- The problem likely relates somehow to the host's default router setting.

First, to understand why the standard and extended ping results have different effects, consider first the standard **ping 172.16.1.51** command on R1, as shown previously in Figure 20-7. As a standard **ping** command, R1 used its LAN interface IP address (172.16.1.1) as the source of the ICMP Echo. So, when the host (A) sent back its ICMP echo reply, host A considered the destination of 172.16.1.1 as being on the same subnet. Host A's ICMP echo reply message, sent back to 172.16.1.1, would work even if host A did not have a default router setting at all!

In comparison, Figure 20-8 shows the difference when using an extended ping on Router R1. An extended ping from local Router R1, using R1's WAN IP address of 172.16.4.1 as the source of the ICMP echo request, means that host A's ICMP echo reply will flow to an address in another subnet, which makes host A use its default router setting.

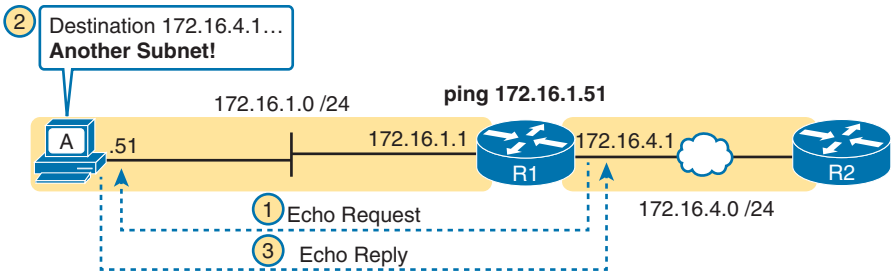



**Figure 20-8** *Extended* **ping** *Command Does Test Host A's Default Router Setting*

The comparison between the previous two figures shows one of the most classic mistakes when troubleshooting networks. Sometimes, the temptation is to connect to a router and ping the host on the attached LAN, and it works. So, the engineer moves on, thinking that the network layer issues between the router and host work fine, when the problem still exists with the host's default router setting.

## Testing WAN Neighbors with Standard Ping

As with a standard ping test across a LAN, a standard ping test between routers over a serial or Ethernet WAN link tests whether the link can pass IPv4 packets. With a properly designed IPv4 addressing plan, two routers on the same serial or Ethernet WAN link should have IP addresses in the same subnet. A ping from one router to the IP address of the other router confirms that an IP packet can be sent over the link and back, as shown in the **ping 172.16.4.2** command on R1 in Figure 20-9.

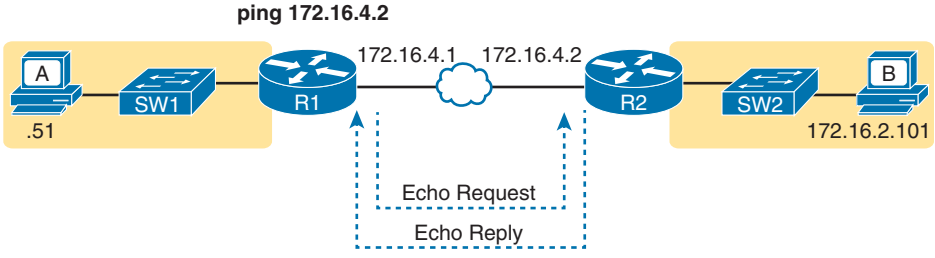



**Figure 20-9** *Pinging Across a WAN Link*

A successful ping of the IP address on the other end of an Ethernet WAN link that sits between two routers confirms several specific facts, such as the following:

- Both routers' WAN interfaces are in an up/up state.
- The Layer 1 and 2 features of the link work.
- The routers believe that the neighboring router's IP address is in the same subnet.
- Inbound ACLs on both routers do not filter the incoming packets, respectively.
- The remote router is configured with the expected IP address (172.16.4.2 in this case).

Pinging the other neighboring router does not test many other features. However, although the test is limited in scope, it does let you rule out WAN links as having a Layer 1 or 2 problem, and it rules out some basic Layer 3 addressing problems.

## Using Ping with Names and with IP Addresses

All the ping examples so far in this chapter show a ping of an IP address. However, the **ping** command can use **hostnames**, and pinging a hostname allows the network engineer to further test whether the Domain Name System (**DNS**) process works.

First, most every TCP/IP application uses hostnames rather than IP addresses to identify the other device. No one opens a web browser and types in 72.163.4.185. Instead, they type in a web address, like https://www.cisco.com, which includes the hostname www.cisco.com. Then, before a host can send data to a specific IP address, the host must first ask a DNS server to resolve that hostname into the matching IP address.

For example, in the small internetwork used for several examples in this chapter, a **ping B** command on host A tests A's DNS settings, as shown in Figure 20-10. When host A sees the use of a hostname (B), it first looks in its local DNS name cache to find out whether it has already resolved the name B. If not, host A first asks the DNS to supply (resolve) the name into its matching IP address (Step 1 in the figure). Only then does host A send a packet to 172.16.2.101, host B's IP address (Step 2).

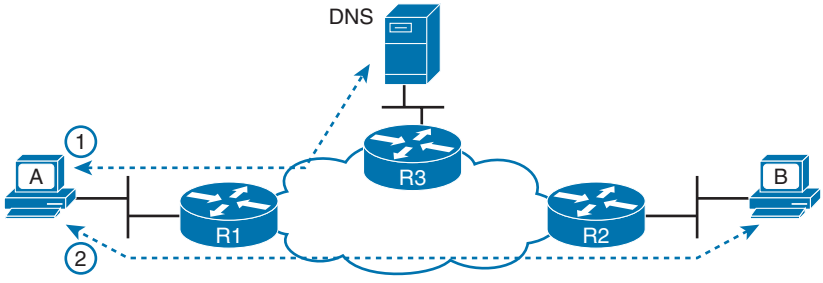



**Figure 20-10** *DNS Name Resolution by Host A*

When troubleshooting, testing from the host by pinging using a hostname can be very helpful. The command, of course, tests the host's own DNS client settings. For instance, a classic comparison is to first ping the destination host using the hostname, which requires a DNS request. Then, repeat the same test, but use the destination host's IP address instead of its name, which does not require the DNS request. If the ping of the hostname fails but the ping of the IP address works, the problem usually has something to do with DNS.

Routers and switches can also use name resolution for commands that refer to hosts, such as the **ping** and **traceroute** commands. The networking device can use DNS, locally defined hostnames, or both. Example 20-4 shows an example DNS configuration on Router R1 from the most recent examples. In particular:

Key Topic

- The **ip domain lookup** command tells the router to attempt to use a DNS server.
- The **ip name-server** {*address* [*address address*]} command defines the list of DNS server IP addresses.
- The **ip domain name** *domain-name* command defines the domain used by the device.

In the example, note that once configured to use DNS, the **ping hostB** command works. The command output shows the IP address the DNS resolution process found for name hostB, 172.16.2.101.

**Example 20-4** *Configuring to Use DNS (Router R1), DNS Has hostB Name*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# ip domain lookup
R1(config)# ip domain name example.com
R1(config)# ip name-server 8.8.8.8 8.8.8.4
R1(config)# ^Z
R1#
R1# ping hostB
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.16.2.101, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/4 ms
```

**NOTE** Older IOS versions used a syntax of **ip domain-name** *domain-name* rather than the newer **ip domain name** *domain-name* (with a space instead of a dash).

**NOTE** When practicing, you might want to disable DNS resolution, particularly in lab devices, using the **no ip domain lookup** command. Cisco routers and switches enable DNS resolution by default with a setting of **ip domain lookup**, but with no name servers identified with the **ip name-server** command. The result of these two default settings causes the router or switch to perform name resolution on a name by broadcasting for a DNS server on each connected subnet. Additionally, if you mistype the first word of a command, IOS thinks you mean that word to be a hostname, and it attempts to perform name resolution on the mistyped command. The result: for any typo of the first word in a command, the default name resolution settings cause a few minutes wait until you get control of the CLI again.

In a lab environment, when not expecting to use DNS, disable DNS resolution with the **no ip domain lookup** command.

You can also configure the router or switch to use locally configured hostnames (or to use both locally configured names and DNS). Use a configuration like that in Example 20-5, adding the **ip host** *name address* global configuration command for each hostname. The router or switch will look for local hostnames whether you use DNS or have the **ip domain lookup** command configured.

**Example 20-5** *Configuring to Use Local Hostnames, R1 Config Has hostB Name*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# ip host hostB 172.16.2.101
R1(config)# ^Z
R1#
```

```
R1# show hosts
Default domain is example.com
Name servers are 8.8.8.8, 8.8.8.4
NAME   TTL   CLASS    TYPE      DATA/ADDRESS
-----------------------------------------
 101.2.16.172.in-addr.arpa        10       IN       PTR      hostB
 hostB                            10       IN       A        172.16.2.101
```

## Problem Isolation Using the traceroute Command

Like **ping**, the **traceroute** command helps network engineers isolate problems. Here is a comparison of the two:

Key Topic

- Both send messages in the network to test connectivity.
- Both rely on other devices to send back a reply.
- Both have wide support on many different operating systems.
- Both can use a hostname or an IP address to identify the destination.
- On routers, both have a standard and extended version, allowing better testing of the reverse route.

The biggest differences relate to the more detailed results in the output of the **traceroute** command and the extra time and effort it takes **traceroute** to build that output. This next major section examines how traceroute works; plus it provides some suggestions on how to use this more detailed information to more quickly isolate IP routing problems.

### traceroute Basics

Imagine some network engineer or CSR starts to troubleshoot some problem. The engineer pings from the user's host, pings from a nearby router, and after a few commands, convinces herself that the host can indeed send and receive IP packets. The problem might not be solved yet, but the problem does not appear to be a network problem.

Now imagine the next problem comes along, and this time the **ping** command fails. It appears that some problem does exist in the IP network. Where is the problem? Where should the engineer look more closely? Although the **ping** command can prove helpful in isolating the source of the problem, the **traceroute** command may be a better option. The **traceroute** command systematically helps pinpoint routing problems by showing how far a packet goes through an IP network before being discarded.

The **traceroute** command identifies the routers in the forward route from source host to destination host. Specifically, it lists the next-hop IP address of each router that would be in each of the individual routes. For instance, a **traceroute 172.16.2.101** command on host A in Figure 20-11 would identify an IP address on Router R1, another on Router R2, and then host B, as shown in the figure. Example 20-6, which follows, lists the output of the command, taken from host A.

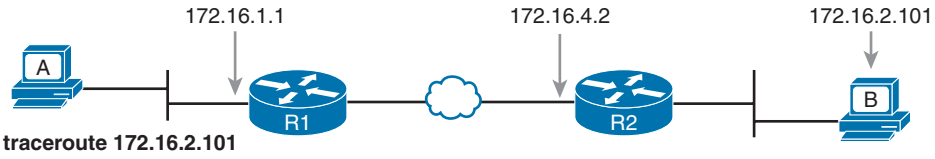



**Figure 20-11** *IP Addresses Identified by a Successful* **traceroute 172.16.2.101** *Command*

**Example 20-6** *Output from* **traceroute 172.16.2.101** *on Host A*

```
Mac_A$ traceroute 172.16.2.101
traceroute to 172.16.2.101, 64 hops max, 52 byte packets
  1 172.16.1.1 (172.16.1.1) 0.870 ms 0.520 ms 0.496 ms
  2 172.16.4.2 (172.16.4.2) 8.263 ms 7.518 ms 9.319 ms
  3 172.16.2.101 (172.16.2.101) 16.770 ms 9.819 ms 9.830 ms
```

## How the traceroute Command Works

The **traceroute** command gathers information by generating packets that trigger error messages from routers; these messages identify the routers, letting the **traceroute** command list the routers' IP addresses in the output of the command. That error message is the ICMP Time-to-Live Exceeded (TTL Exceeded) message, originally meant to notify hosts when a packet had been looping around a network.

Ignoring traceroute for a moment and instead focusing on IP routing, IPv4 routers defeat routing loops in part by discarding looping IP packets. To do so, the IPv4 header holds a field called Time To Live (TTL). The original host that creates the packet sets an initial TTL value. Then each router that forwards the packet decrements the TTL value by 1. When a router decrements the TTL to 0, the router perceives the packet is looping, and the router discards the packet. The router also notifies the host that sent the discarded packet by sending an ICMP TTL Exceeded message.

Now back to traceroute. Traceroute sends messages with low TTL values to make the routers send back a TTL Exceeded message. Specifically, a **traceroute** command begins by sending several packets (usually three), each with the header TTL field equal to 1. When that packet arrives at the next router—host A's default Router R1 in the example of Figure 20-12—the router decrements TTL to 0 and discards the packet. The router then sends host A the TTL Exceeded message, which identifies the router's IP address to the **traceroute** command.

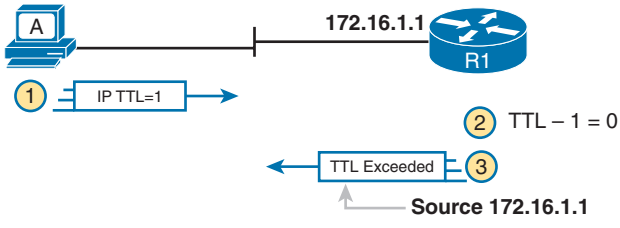



**Figure 20-12** *How* **traceroute** *Identifies the First Router in the Route*

The **traceroute** command sends several TTL=1 packets, checking them to see whether the TTL Exceeded messages flow from the same router, based on the source IP address of the TTL Exceeded message. Assuming the messages come from the same router, the **traceroute** command lists that IP address as the next line of output on the command.

To find all the routers in the path, and finally confirm that packets flow all the way to the destination host, the **traceroute** command sends a small set of packets with TTL=1, then a small set with TTL=2, then 3, 4, and so on, until the destination host replies. Figure 20-13 shows the packet from the second set with TTL=2. In this case, one router (R1) actually forwards the packet, while another router (R2) happens to decrement the TTL to 0, causing a TTL Exceeded message to be sent back to host A.

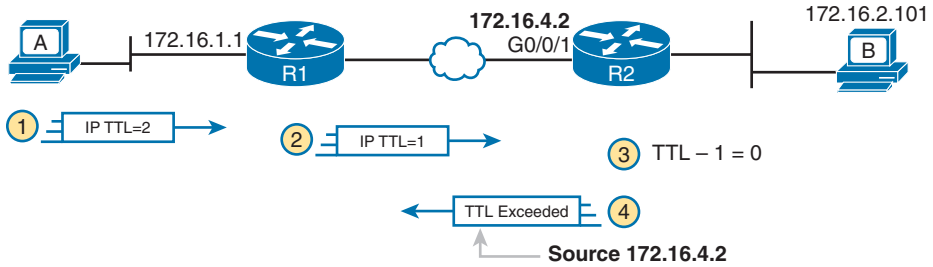



**Figure 20-13** *TTL=2 Message Sent by* **traceroute**

The figure shows these four steps:

1. The **traceroute** command sends a packet from the second set with TTL=2.
2. Router R1 processes the packet and decrements TTL to 1. R1 forwards the packet.
3. Router R2 processes the packet and decrements TTL to 0. R2 discards the packet.
4. R2 notifies the sending host of the discarded packet by sending a TTL Exceeded ICMP message. The source IP address of that message is 172.16.4.2.

Finally, the choice of source IP address to use on the time-exceeded message returned by routers has a big impact on the output of the **traceroute** command. Most routers use simpler logic that also makes command output like **traceroute** more consistent and meaningful. That logic: choose the TTL Exceeded message's source IP address based on the interface in which the discarded original message arrived. In the example in Figure 20-13, the original message at Step 2 arrived on R2's G0/0/1 interface, so at Step 3, R2 uses G0/0/1's IP address as the source IP address of the TTL Exceeded message, and as the interface out which to send the message.

### Standard and Extended traceroute

The standard and extended options for the **traceroute** command give you many of the same options as the **ping** command. For instance, Example 20-7 lists the output of a standard **traceroute** command on Router R1. Like the standard **ping** command, a standard **traceroute** command chooses an IP address based on the outgoing interface for the packet sent by the command. So, in this example, the packets sent by R1 come from source IP address 172.16.4.1, R1's G0/0/0 IP address.

**Example 20-7** *Standard* **traceroute** *Command on R1*

```
R1# traceroute 172.16.2.101
Type escape sequence to abort.
Tracing the route to 172.16.2.101
VRF info: (vrf in name/id, vrf out name/id)
   1 172.16.4.2 0 msec 0 msec 0 msec
   2 172.16.2.101 0 msec 0 msec *
```

The extended **traceroute** command, as shown in Example 20-8, follows the same basic command structure as the extended **ping** command. The user can type all the parameters on one command line, but it is much easier to just type **traceroute**, press Enter, and let IOS prompt for all the parameters, including the source IP address of the packets (172.16.1.1 in this example).

**Example 20-8** *Extended* **traceroute** *Command on R1*

```
R1# traceroute
Protocol [ip]:
Target IP address: 172.16.2.101
Source address: 172.16.1.1
Numeric display [n]:
Timeout in seconds [3]:
Probe count [3]:
Minimum Time to Live [1]:
Maximum Time to Live [30]:
Port Number [33434]:
Loose, Strict, Record, Timestamp, Verbose[none]:
Type escape sequence to abort.
Tracing the route to 172.16.2.101
VRF info: (vrf in name/id, vrf out name/id)
   1 172.16.4.2 0 msec 0 msec 0 msec
   2 172.16.2.101 0 msec 0 msec *
```

Both the **ping** and **traceroute** commands exist on most operating systems, including Cisco IOS. However, some operating systems use a slightly different syntax for **traceroute**. For example, most Windows operating systems support **tracert** and **pathping**, and not **traceroute**. Linux and OS X support the **traceroute** command.



**NOTE** Host OS **traceroute** commands usually create ICMP echo requests. The Cisco IOS **traceroute** command instead creates IP packets with a UDP header. This bit of information may seem trivial at this point. However, note that an ACL may actually filter the traffic from a host's **traceroute** messages but not the router **traceroute** command, or vice versa.

## Telnet and SSH

The **ping** and **traceroute** commands do give networkers two great tools to begin isolating the cause of an IP routing problem. However, these two commands tell us nothing about the

operation state inside the various network devices. Once you begin to get an idea of the kinds of problems and the possible locations of the problems using **ping** and **traceroute**, the next step is to look at the status of various router and switch features. One way to do that is to use Telnet or Secure Shell (SSH) to log in to the devices.

## Common Reasons to Use the IOS Telnet and SSH Client

Normally, a network engineer would log in to the remote device using a Telnet or SSH client on a PC, tablet, or any other user device. In fact, often, the same software package does both Telnet and SSH. However, in some cases, you might want to take advantage of the Telnet and SSH client built in to IOS on the routers and switches to Telnet/SSH from one Cisco device to the next.

To understand why, consider the example shown in Figure 20-14. The figure shows arrowed lines to three separate IP addresses on three separate Cisco routers. PC1 has attempted to Telnet to each address from a different tab in PC1's Telnet/SSH client. However, R2 happens to have an error in its routing protocol configuration, so R1, R2, and R3 fail to learn any routes from each other. As a result, PC1's Telnet attempt to both 10.1.2.2 (R2) and 10.1.3.3 (R3) fails.

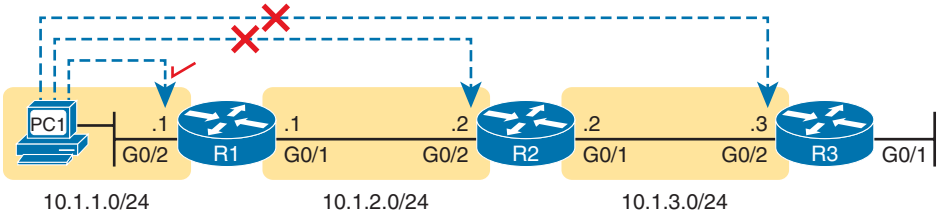



**Figure 20-14** *Telnet Works from PC1 to R1 but Not to R2 or R3*

In some cases, like this one, a Telnet or SSH login from the network engineer's device can fail, while you could still find a way to log in using the **telnet** and **ssh** commands to use the Telnet and SSH clients on the routers or switches. With this particular scenario, all the individual data links work; the problem is with the routing protocol exchanging routes. PC1 can ping R1's 10.1.1.1 IP address, R1 can ping R2's 10.1.2.2 address, and R2 can ping R3's 10.1.3.3 address. Because each link works, and each router can send and receive packets with its neighbor on the shared data link, you could Telnet/SSH to each successive device.

Figure 20-15 shows the idea. PC1 begins with a Telnet/SSH connection into Router R1, as shown on the left. Then the user issues the **telnet 10.1.2.2** command from R1 to Telnet to R2. Once logged in to R2, the user can issue commands on R2. Then from R2, the user could issue the **telnet 10.1.3.3** command to Telnet to R3, from which the user could issue commands on R3.

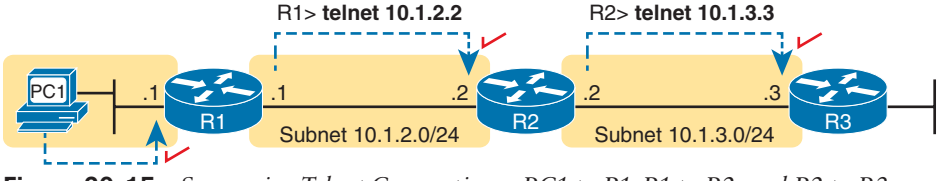



**Figure 20-15** *Successive Telnet Connections: PC1 to R1, R1 to R2, and R2 to R3*

The Telnet connections shown in Figure 20-15 work because each Telnet in this case uses source and destination addresses in the same subnet. For example, R1's **telnet 10.1.2.2** command uses 10.1.2.2 as the destination, of course. R1 uses the outgoing interface IP address used to send packets to 10.1.2.2, 10.1.2.1 in this case. Because each of these **telnet** commands connects to an IP address in a connected subnet, the routing protocol could be completely misconfigured, and you could still Telnet/SSH to each successive device to troubleshoot and fix the problem.

Network engineers also use the IOS Telnet and SSH client just for preference. For instance, if you need to log in to several Cisco devices, you could open several windows and tabs on your PC, and log in from your PC (assuming the network was not having problems). Or you could log in from your PC to some nearby Cisco router or switch, and from there Telnet or SSH to other Cisco devices.

## IOS Telnet and SSH Examples

Using the IOS Telnet client via the **telnet** *host* command is pretty simple. Just use the IP address or hostname to identify the host to which you want to connect, and press Enter. Example 20-9 shows an example based on Figure 20-15, with R1 using Telnet to connect to 10.1.2.2 (R2).

**Example 20-9** *Telnet from R1 to R2 to View Interface Status on R2*

```
R1# telnet 10.1.2.2
Trying 10.1.2.2 ... Open


User Access Verification


Username: wendell
Password:
R2>
R2> show ip interface brief
Interface                  IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0         unassigned      YES unset  administratively down down
GigabitEthernet0/1         10.1.3.2        YES manual up                    up
GigabitEthernet0/2         10.1.2.2        YES manual up                    up
GigabitEthernet0/3         unassigned      YES unset  administratively down down
```

Take the time to pay close attention to the command prompts. The example begins with the user logged in to Router R1, with the R1# command prompt. After the user issues the **telnet 10.1.2.2** command, R2 asks the user for both a username and password because Router R2 uses local username authentication, which requires those credentials. The **show ip interfaces brief** command at the end of the output shows Router R2's interfaces and IP addresses again per Example 20-9 and Figure 20-15.

The **ssh -l** *username host* command in Example 20-10 follows the same basic ideas as the **telnet** *host* command, but with an SSH client. The **-l** flag means that the next parameter is the login username. In this case, the user begins logged in to Router R1 and then uses the **ssh -l wendell 10.1.2.2** command to SSH to Router R2. R2 expects a username/password of wendell/odom, with wendell supplied in the command and odom supplied when R2 prompts the user.

**Example 20-10** *SSH Client from R1 to R2 to View Interface Status on R2*

```
R1# ssh -l wendell 10.1.2.2

Password:

R2>
Interface                  IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0         unassigned      YES unset  administratively down down
GigabitEthernet0/1         10.1.3.2        YES manual up                    up
GigabitEthernet0/2         10.1.2.2        YES manual up                    up
GigabitEthernet0/3         unassigned      YES unset  administratively down down
```

When you have finished using the other router, you can log out from your Telnet or SSH connection using the **exit** or **quit** command.

Finally, note that IOS supports a mechanism to use hotkeys to move between multiple Telnet or SSH sessions from the CLI. Basically, starting at one router, you could telnet or SSH to a router, do some commands, and instead of using the **exit** command to end your connection, you could keep the connection open while still moving back to the command prompt of the original router. For instance, if starting at Router R1, you could telnet to R2, R3, and R4, suspending but not exiting those Telnet connections. Then you could easily move between the sessions to issue new commands with a few keystrokes.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 20-1 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 20-1** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |

## Review All the Key Topics

Key Topic

**Table 20-2** Key Topics for Chapter 20

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 20-5 | ARP tables on Layer 3 hosts, with MAC address tables on Layer 2 switch | 517 |
| Figure 20-6 | How extended ping in IOS performs a better test of the reverse route | 518 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 20-7 | Why a standard ping over a LAN does not exercise a host's default router logic | 519 |
| List | Network layer problems that could cause a ping to fail between a router and host on the same LAN subnet | 520 |
| List | Testing a host's default router setting using extended ping | 520 |
| List | DNS configuration commands | 522 |
| List | Comparisons between the **ping** and **traceroute** commands | 524 |

## Key Terms You Should Know

DNS, extended ping, forward route, hostname, ICMP echo reply, ICMP echo request, ping, reverse route, traceroute

## Command References

Tables 20-3 and 20-4 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 20-3** Chapter 20 Configuration Command Reference

| Command | Description |
|---|---|
| **[no] ip domain lookup** | Router/switch global command to enable or disable (**no**) the device from using DNS resolution when commands use a hostname |
| **ip domain name** *name* | Router/switch global command to define the DNS domain used by this device |
| **ip name-server** *address* [*address*...] | Global command to define one or more DNS server IP addresses |
| **ip host** *name address* | Global command that defines a hostname available on the local device |

**Table 20-4** Chapter 20 EXEC Command Reference

| Command | Description |
|---|---|
| **ping** {*hostname* \| *address*} | EXEC command that sends ICMP Echo Request packets to the address, expecting the distant host to send ICMP Echo Reply messages in return |
| **traceroute** {*hostname* \| *address*} | EXEC command that messages to a distance host, expecting to force intermediate routes to send ICMP destination unreachable messages, to identify the routers in the path to the distant host |
| **show host** | Command to list the device's known hostnames and corresponding IP addresses |

| Command | Description |
|---|---|
| **show ip interface brief** | A router command that lists its interfaces and IP addresses, plus a few more facts, with one line per interface |
| **telnet** {*hostname* \| *address*} | Command to initiate a Telnet connection to a remote host |
| **ssh -l** *username* {*hostname* \| *address*} | Command to initiate an SSH connection, for the listed username, to a remote host |

*This page intentionally left blank*



# Part V Review

Keep track of your part review progress with the checklist in Table P5-1. Details on each task follow the table.

**Table P5-1** Part V Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Watch Video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book, using the PTP software.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Do Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Blog Config Labs:** The author's blog includes a series of configuration-focused labs that you can do on paper or with Cisco Packet Tracer in about 15 minutes. To find them, open https://www.certskills.com and look under the Labs menu item.

**Other:** If using other lab tools, here are a few suggestions: Make sure to experiment heavily with IPv4 addressing, static routing, and Layer 3 switching. In each case, test all your routes using **ping** and **traceroute**.

## Watch Video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.



Part IV began the story in this book about IP Version 4 (IPv4) addressing. Part V continued that story with how to implement addressing in Cisco routers, along with a variety of methods to route packets between local interfaces. But those topics delayed the discussion of one of the most important topics in TCP/IP, namely IP routing protocols, as discussed in Part VI.

Routers use IP routing protocols to learn about the subnets in an internetwork, choose the current-best routes to reach each subnet, and add those routes to each router's IP routing table. Cisco chose to include one and only one IP routing protocol in the CCNA 200-301 Version 1.1 blueprint: the Open Shortest Path First (OSPF) routing protocol. This entire part focuses on OSPF as an end to itself and to show the principles of routing protocols.

# Part VI

## OSPF

**Chapter 21:** Understanding OSPF Concepts

**Chapter 22:** Implementing Basic OSPF Features

**Chapter 23:** Implementing Optional OSPF Features

**Chapter 24:** OSPF Neighbors and Route Selection

**Part VI Review**



CHAPTER 21

# Understanding OSPF Concepts

**This chapter covers the following exam topics:**

- **3.0 IP Connectivity**
  - **3.2 Determine how a router makes a forwarding decision by default**
    - **3.2.b Administrative distance**
    - **3.2.c Routing protocol metric**
  - **3.4 Configure and verify single area OSPFv2**
    - **3.4.a Neighbor adjacencies**
    - **3.4.b Point-to-point**
    - **3.4.c Broadcast (DR/BR selection)**
    - **3.4.d Router ID**

Every enterprise uses some dynamic routing protocol inside their network so that the routers cooperatively learn routes to all subnets. But in the decades that led to TCP/IP becoming the common networking model used on all computers, several routing protocols emerged as candidates to be used by those enterprises. As a result, even today, enterprises choose from a small set of alternative routing protocols. Of those, Cisco includes Open Shortest Path First (OSPF) in the CCNA 200-301 V1.1 blueprint.

To establish come context, this chapter begins by examining the different routing protocols, their similar goals, and their differences in implementation. With that context in mind, the rest of the chapter then examines the basic concepts of how OSPF operates. The second major section of the chapter gets into the foundations of OSPF: running OSPF on each router, becoming neighbors, exchanging data about routes, and calculating IP routes to be used by the IP routing table. The final major section then looks more closely at how OSPF internally represents network topologies as a database of network links and their states—the OSPF link-state database (LSDB).

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 21-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Comparing Dynamic Routing Protocol Features | 1–3 |
| OSPF Concepts and Operation | 4, 5 |
| OSPF Areas and LSAs | 6 |

1. Which of the following routing protocols uses link-state logic?
   a. RIPv1
   b. RIPv2
   c. EIGRP
   d. OSPF
2. Which of the following routing protocols use a metric that is, by default, at least partially affected by link bandwidth? (Choose two answers.)
   a. RIPv1
   b. RIPv2
   c. EIGRP
   d. OSPF
3. Which of the following interior routing protocols support VLSM? (Choose three answers.)
   a. RIPv1
   b. RIPv2
   c. EIGRP
   d. OSPF
4. Two routers using OSPFv2 have become neighbors and exchanged all LSAs. As a result, Router R1 now lists some OSPF-learned routes in its routing table. Which of the following best describes how R1 uses those recently learned LSAs to choose which IP routes to add to its IP routing table?
   a. R1 copies a route from every LSA into its routing table.
   b. R1 copies a route from certain types of LSAs into its routing table.
   c. R1 runs SPF against the LSAs to calculate the routes.
   d. R1 does not use the LSAs when choosing what routes to add.
5. Which of the following OSPF neighbor states is expected when the exchange of topology information is complete between two OSPF neighbors?
   a. 2-way
   b. Full
   c. Up/up
   d. Final

6. A company has a small/medium-sized network with 15 routers and 40 subnets and uses OSPFv2. Which of the following is considered an advantage of using a single-area design as opposed to a multiarea design?
   a. It reduces the CPU processing overhead on most routers.
   b. It reduces the frequency of running the SPF algorithm due to interface status changes.
   c. It allows for simpler planning and operations.
   d. It reduces memory consumption.

# Foundation Topics

## Comparing Dynamic Routing Protocol Features

Routers add IP routes to their routing tables using three methods: connected routes, static routes, and routes learned by using dynamic routing protocols. Before we get too far into the discussion, however, it is important to define a few related terms and clear up any misconceptions about the terms *routing protocol*, *routed protocol*, and *routable protocol*. These terms are generally defined as follows:

- **Routing protocol:** A set of messages, rules, and algorithms used by routers for the overall purpose of learning routes. This process includes the exchange and analysis of routing information. Each router chooses the best route to each subnet (path selection) and finally places those best routes in its IP routing table. Examples include RIP, EIGRP, OSPF, and BGP.
- **Routed protocol and routable protocol:** Synonyms, both terms refer to a protocol that defines packets that can be routed (forwarded) by a router. Routers forward packets defined by routed protocols. Examples include IP Version 4 (IPv4) and IP Version 6 (IPv6).

**NOTE** The term *path selection* sometimes refers to part of the job of a routing protocol, in which the routing protocol chooses the best route.

Even though routing protocols (such as OSPF) are different from routed protocols (such as IP), they do work together very closely. The routing process forwards IP packets, but if a router does not have any routes in its IP routing table that match a packet's destination address, the router discards the packet. Routers need routing protocols so that the routers can learn all the possible routes and add them to the routing table so that the routing process can forward (route) routable protocols such as IP.

## Routing Protocol Functions

Cisco IOS software supports several IP routing protocols, performing the same general functions:

Key Topic

1. Learn routing information about IP subnets from neighboring routers.
2. Advertise routing information about IP subnets to neighboring routers.
3. If more than one possible route exists to reach one subnet, pick the best route based on a **metric**.
4. If the network topology changes—for example, a link fails—react by advertising that some routes have failed and pick a new currently best route. (This process is called **convergence**.)

**NOTE** A neighboring router connects to the same link as another router, such as the same WAN link or the same Ethernet LAN.

Figure 21-1 shows an example of three of the four functions in the list. Router R1, in the lower left of the figure, must choose the best route to reach the subnet connected off Router R2, on the bottom right of the figure. Following the steps in the figure:

**Step 1.** R2 advertises a route to the lower right subnet—172.16.3.0/24—to both Router R1 and R3.

**Step 2.** After R3 learns about the route to 172.16.3.0/24 from R2, R3 advertises that route to R1.

**Step 3.** R1 must choose between the two routes it learned about for reaching subnet 172.16.3.0/24—one with metric 1 from R2 and one with metric 2 from R3. R1 chooses the lower metric route through R2 (function 3).

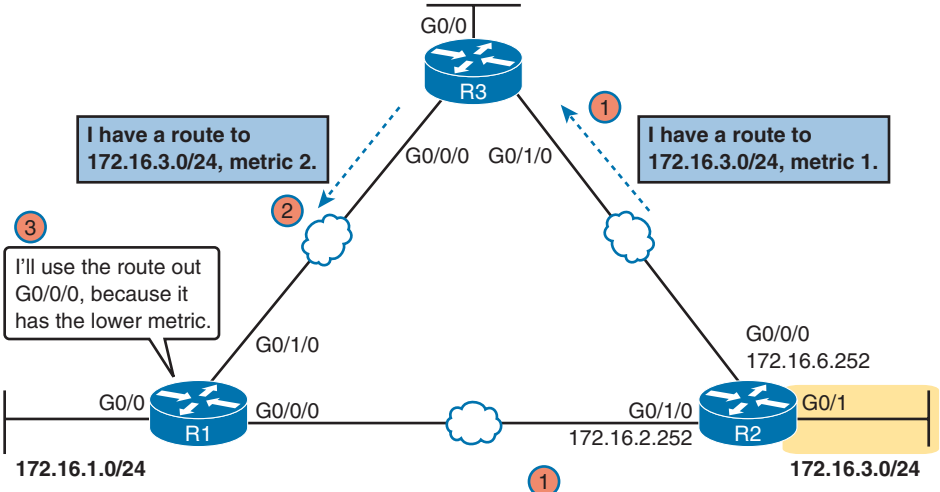



**Figure 21-1** *Three of the Four Basic Functions of Routing Protocols*

The other routing protocol function, *convergence*, occurs when the topology changes—that is, when either a router or link fails or comes back up again. When something changes, the best routes available in the network can change. Convergence simply refers to the process by which all the routers collectively realize something has changed, advertise the information about the changes to all the other routers, and all the routers then choose the currently best routes for each subnet. The ability to converge quickly, without causing loops, is one of the most important considerations when choosing which IP routing protocol to use.

In Figure 21-1, convergence might occur if the link between R1 and R2 failed. In that case, R1 should stop using its old route for subnet 172.16.3.0/24 (directly through R2) and begin sending packets to R3.

## Interior and Exterior Routing Protocols

IP routing protocols fall into one of two major categories: **interior gateway protocols (IGP)** or *exterior gateway protocols* (EGP). The definitions of each are as follows:

Key Topic

- **IGP:** A routing protocol that was designed and intended for use inside a single autonomous system (AS)
- **EGP:** A routing protocol that was designed and intended for use between different autonomous systems

**NOTE** The terms *IGP* and *EGP* include the word *gateway* because routers used to be called gateways.

These definitions use another new term: *autonomous system* (AS). An AS is a network under the administrative control of a single organization. For example, a network created and paid for by a single company is probably a single AS, and a network created by a single school system is probably a single AS. Other examples include large divisions of a state or national government, where different government agencies might be able to build their own networks. Each ISP is also typically a single different AS.

Some routing protocols work best inside a single AS by design, so these routing protocols are called IGPs. Conversely, routing protocols designed to exchange routes between routers in different autonomous systems are called EGPs. Today, Border Gateway Protocol (BGP) is the only EGP.

Each AS can be assigned a number called (unsurprisingly) an *AS number* (ASN). Like public IP addresses, the Internet Assigned Numbers Authority (IANA, www.iana.org) controls the worldwide rights to assigning ASNs. It delegates that authority to other organizations around the world, typically to the same organizations that assign public IP addresses. For example, in North America, the American Registry for Internet Numbers (ARIN, www.arin.net) assigns public IP address ranges and ASNs.

Figure 21-2 shows a small view of the worldwide Internet. The figure shows two enterprises and three ISPs using IGPs (OSPF and EIGRP) inside their own networks and with BGP being used between the ASNs.

---

Answers to the "Do I Know This Already?" quiz:

**1** D **2** C, D **3** B, C, D **4** C **5** B **6** C

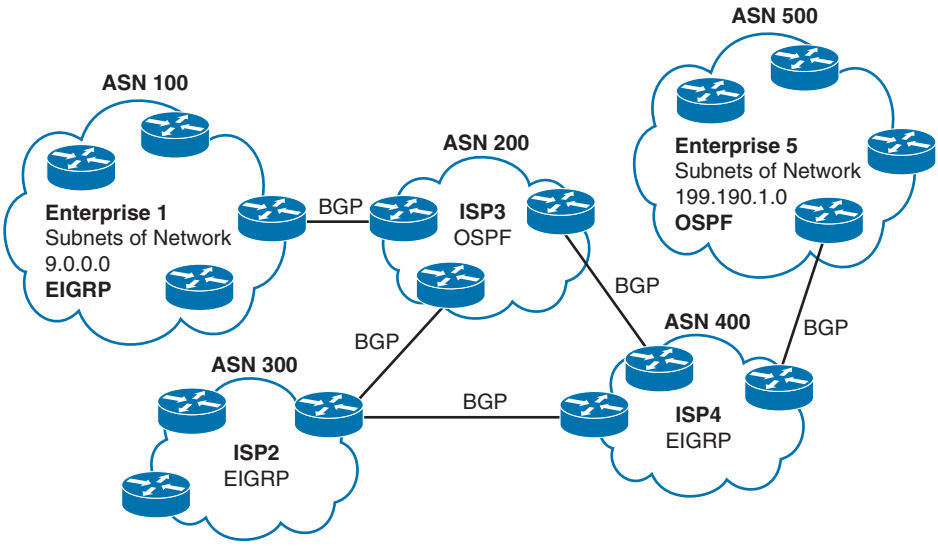



**Figure 21-2** *Comparing Locations for Using IGPs and EGPs*

## Comparing IGPs

Before getting into the details of OSPF, it helps to consider some comparisons between OSPF and the other IGP options. This section takes a brief look at all three routing protocols.

### IGP Routing Protocol Algorithms

A routing protocol's underlying algorithm determines how the routing protocol does its job. The term *routing protocol algorithm* simply refers to the logic and processes used by different routing protocols to solve the problem of learning all routes, choosing the best route to each subnet, and converging in reaction to changes in the internetwork. Three main branches of routing protocol algorithms exist for IGP routing protocols:

Key Topic

- **Distance vector** (sometimes called Bellman-Ford after its creators)
- Advanced distance vector (sometimes called balanced hybrid)
- **Link-state**

Historically speaking, distance vector protocols were invented first, mainly in the early 1980s. Routing Information Protocol (RIP) was the first popularly used IP distance vector protocol, with the Cisco-proprietary Interior Gateway Routing Protocol (IGRP) being introduced a little later.

By the early 1990s, distance vector protocols' somewhat slow convergence and potential for routing loops drove the development of new alternative routing protocols that used new algorithms. Link-state protocols—in particular, Open Shortest Path First (OSPF) and Integrated Intermediate System to Intermediate System (IS-IS)—solved the main issues. They also came with a price: they required extra CPU and memory on routers, with more planning required from the network engineers.

**NOTE** All references to OSPF in this chapter refer to OSPF Version 2 (OSPFv2) unless otherwise stated.

Around the same time as the introduction of OSPF, Cisco created a proprietary routing protocol called Enhanced Interior Gateway Routing Protocol (EIGRP), which used some features of the earlier IGRP protocol. EIGRP solved the same problems as did link-state routing protocols, but EIGRP required less planning and less CPU/RAM overhead. As time went on, EIGRP was classified as a unique type of routing protocol. However, it used more distance vector features than link-state, so the industry refers to its algorithm as either an advanced distance vector protocol or as a balanced hybrid protocol.

### Metrics

Routing protocols choose the best route to reach a subnet by choosing the route with the lowest metric. For example, RIP uses a counter of the number of routers (hops) between a router and the destination subnet, as shown in the example of Figure 21-1. OSPF totals the cost associated with each interface in the end-to-end route, with the cost based on link bandwidth. Table 21-2 lists the most common IP routing protocols and some details about the metric in each case.

Key Topic

**Table 21-2** IP IGP Metrics

| IGP | Metric | Description |
|---|---|---|
| RIPv2 | Hop count | The number of routers (hops) between a router and the destination subnet |
| OSPF | Cost | The sum of all interface cost settings for all links in a route, with the cost default based on interface bandwidth |
| EIGRP | Calculation based on bandwidth and delay | Calculated based on the route's slowest link and the cumulative delay associated with each interface in the route |

A brief comparison of the metric used by the older RIP versus the metric used by OSPF shows some insight into why OSPF and EIGRP surpassed RIP. Figure 21-3 shows an example in which Router B has two possible routes to subnet 10.1.1.0 on the left side of the network: a shorter route over a very slow serial link at 1544 Kbps, or a longer route over two Gigabit Ethernet WAN links.

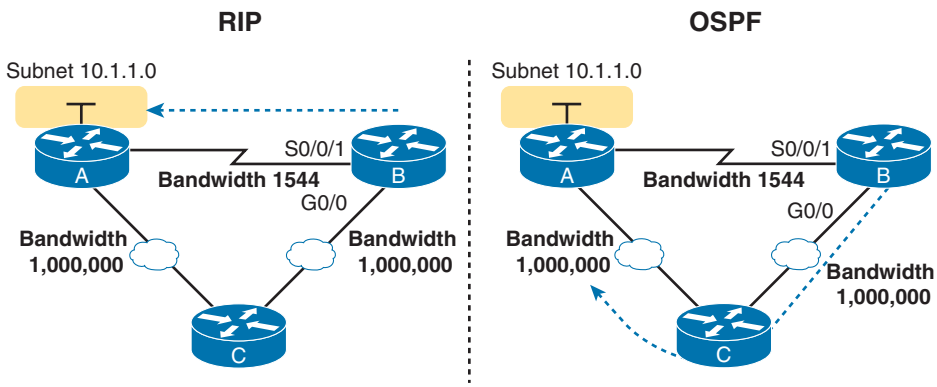



**Figure 21-3** *RIP and OSPF Metrics Compared*

The left side of the figure shows the results of RIP in this network. Using hop count, Router B learns of a one-hop route directly to Router A through B's S0/0/1 interface. B also learns of a two-hop route through Router C, through B's G0/0 interface. Router B chooses the lower hop count route, which happens to go over the slow-speed serial link.

The right side of the figure shows the better choice made by OSPF based on its better metric. To cause OSPF to make the right choice, the engineer could use default settings based on the correct interface bandwidth to match the actual link speeds, thereby allowing OSPF to choose the faster route. (The **bandwidth** interface subcommand does not change the actual physical speed of the interface. It just tells IOS what speed to assume the interface is using.)

### Other IGP Comparisons

Routing protocols can be compared based on many features, some of which matter to the current CCNA exam, whereas some do not. Table 21-3 introduces a few more points and lists the comparison points mentioned in this book for easier study, with a few supporting comments following the table.

**Table 21-3** Interior IP Routing Protocols Compared

| Feature | RIPv2 | EIGRP | OSPF |
|---|---|---|---|
| Classless/sends mask in updates/supports VLSM | Yes | Yes | Yes |
| Algorithm (DV, advanced DV, LS) | DV | Advanced DV | LS |
| Supports manual summarization | Yes | Yes | Yes |
| Cisco-proprietary | No | Yes* | No |
| Routing updates are sent to a multicast IP address | Yes | Yes | Yes |
| Convergence | Slow | Fast | Fast |
| Multicast addresses used | 224.0.0.9 | 224.0.0.10 | 224.0.0.5, 224.0.0.6 |

* Although Cisco created EIGRP and has kept it as a proprietary protocol for many years, Cisco chose to publish EIGRP as an informational RFC in 2013. This allows other vendors to implement EIGRP, while Cisco retains the rights to the protocol.

Regarding the top row of the table, classless routing protocols support variable-length subnet masks (VLSM) as well as manual route summarization by sending routing protocol messages that include the subnet masks in the message. The older RIPv1 and IGRP routing protocols—both classful routing protocols—do not.

Also, note that the older routing protocols (RIPv1, IGRP) sent routing protocol messages to IP broadcast addresses, while the newer routing protocols in the table all use IP multicast destination addresses. The use of multicasts makes the protocol more efficient and causes less overhead and fewer issues with the devices in the subnet that are not running the routing protocol.

# OSPF Concepts and Operation

Routing protocols basically exchange information so routers can learn routes. The routers learn information about subnets, routes to those subnets, and metric information about how good each route is compared to others. The routing protocol can then choose the currently best route to each subnet, building the IP routing table.

Link-state protocols like OSPF take a little different approach to the particulars of what information they exchange and what the routers do with that information once learned. This next (second) major section narrows the focus to only link-state protocols, specifically OSPFv2.

This section begins with an overview of what OSPF does by exchanging data about the network in data structures called **link-state advertisements (LSAs)**. Then the discussion backs up a bit to provide more details about each of three fundamental parts of how OSPF operates: how OSPF routers use *neighbor* relationships, how routers exchange LSAs with neighbors, and then how routers calculate the best routes once they learn all the LSAs.

## OSPF Overview

Link-state protocols build IP routes with a couple of major steps. First, the routers together build a lot of information about the network: routers, links, IP addresses, status information, and so on. Then the routers flood the information, so all routers know the same information. At that point, each router can calculate routes to all subnets, but from each router's own perspective.

### Topology Information and LSAs

Routers using link-state routing protocols need to collectively advertise practically every detail about the internetwork to all the other routers. At the end of the process of *flooding* the information to all routers, every router in the internetwork has the exact same information about the internetwork. Flooding a lot of detailed information to every router sounds like a lot of work, and relative to distance vector routing protocols, it is.

Open Shortest Path First (OSPF), the most popular link-state IP routing protocol, organizes topology information using LSAs and the **link-state database (LSDB)**. Figure 21-4 represents the ideas. Each LSA is a data structure with some specific information about the network topology; the LSDB is simply the collection of all the LSAs known to a router.

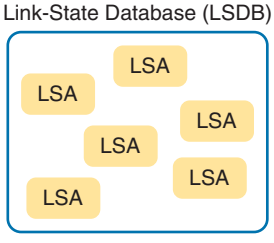



**Figure 21-4** *LSA and LSDB Relationship*

Figure 21-5 shows the general idea of the flooding process, with R8 creating and flooding its *router LSA*. The router LSA for Router R8 describes the router itself, including the existence of subnet 172.16.3.0/24, as seen on the right side of the figure. (Note that Figure 21-5 shows only a subset of the information in R8's router LSA.)

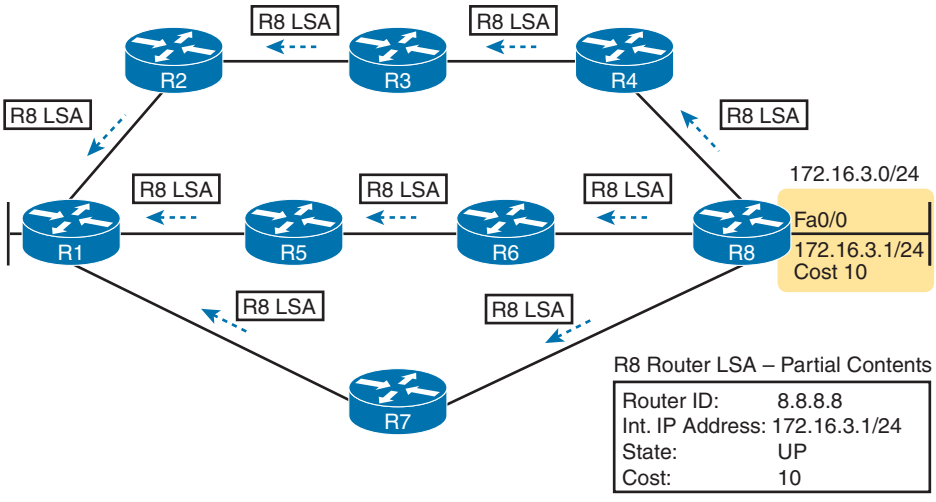



**Figure 21-5** *Flooding LSAs Using a Link-State Routing Protocol*

Figure 21-5 shows the rather basic flooding process, with R8 sending the original LSA for itself, and the other routers flooding the LSA by forwarding it until every router has a copy. The flooding process causes every router to learn the contents of the LSA while preventing the LSA from being flooded around in circles. Basically, before sending an LSA to yet another neighbor, routers communicate, asking "Do you already have this LSA?" and then sending the LSA to the next neighbor only if the neighbor has not yet learned about the LSA.

Once flooded, routers do occasionally reflood each LSA. Routers reflood an LSA when some information changes (for example, when a link goes up or comes down). They also reflood each LSA based on each LSA's separate aging timer (default 30 minutes).

## Applying Dijkstra SPF Math to Find the Best Routes

The link-state flooding process results in every router having an identical copy of the LSDB in memory, but the flooding process alone does not cause a router to learn what routes to add to the IP routing table. Although incredibly detailed and useful, the information in the LSDB does not explicitly state each router's best route to reach a destination.

To build routes, link-state routers have to do some math. Thankfully, you and I do not have to know the math! However, all link-state protocols use a type of math algorithm, called the Dijkstra **Shortest Path First (SPF) algorithm**, to process the LSDB. That algorithm analyzes (with math) the LSDB and builds the routes that the local router should add to the IP routing table—routes that list a subnet number and mask, an outgoing interface, and a next-hop router IP address.

Now that you have the big ideas down, the next several topics walk through the three main phases of how OSPF routers accomplish the work of exchanging LSAs and calculating routes. Those three phases are

**Becoming neighbors:** A relationship between two routers that connect to the same data link, created so that the neighboring routers have a means to exchange their LSDBs.

**Exchanging databases:** The process of sending LSAs to neighbors so that all routers learn the same LSAs.

**Adding the best routes:** The process of each router independently running SPF, on their local copy of the LSDB, calculating the best routes, and adding those to the IPv4 routing table.

## Becoming OSPF Neighbors

Of everything you learn about OSPF in this chapter, OSPF neighbor concepts have the most to do with how you will configure and troubleshoot OSPF in Cisco routers. You configure OSPF to cause routers to run OSPF and become neighbors with other routers. Once that happens, OSPF does the rest of the work to exchange LSAs and calculate routers in the background, with no additional configuration required. This section discusses the fundamental concepts of OSPF neighbors.

### The Basics of OSPF Neighbors

Two routers must meet some compatibility requirements to become neighbors. First, they must both use OSPF and both connect to the same data link. Two routers can become OSPF neighbors if connected to the same VLAN, or same serial link, or same Ethernet WAN link.

Additionally, the two routers must send OSPF messages that declare some OSPF settings, and those settings must be compatible. To do so, the routers send OSPF Hello messages, introducing themselves to the potential neighbor. Assuming the two potential neighbors have compatible OSPF parameters, the two form an OSPF neighbor relationship, and would be displayed in the output of the **show ip ospf neighbor** command.

The OSPF neighbor relationship also lets OSPF know when a neighbor might not be a good option for routing packets right now. Imagine R1 and R2 form a neighbor relationship, learn LSAs, and calculate routes that send packets through the other router. Months later, R1 notices that the neighbor relationship with R2 fails. That failed neighbor connection to R2 makes R1 react: R1 refloods LSAs impacted by the failed link, and R1 runs SPF to recalculate its own routes.

Finally, the OSPF neighbor model allows new routers to be dynamically discovered. That means new routers can be added to a network without requiring every router to be reconfigured. Instead, OSPF routers listen for OSPF Hello messages from new routers and react to those messages, attempting to become neighbors and exchange LSDBs.

### Meeting Neighbors and Learning Their Router ID

The OSPF neighbor relationship begins by exchanging OSPF *Hello* messages, which list each router's **router ID (RID)**. OSPF RIDs are 32-bit numbers, so most command output lists these as dotted-decimal numbers (DDN). By default, IOS chooses one of the router interface's IPv4 addresses to use as its OSPF RID. However, the OSPF RID can be directly configured, as covered in the section "Configuring the OSPF Router ID" in Chapter 22, "Implementing Basic OSPF Features."

As soon as a router has chosen its OSPF RID and some interfaces come up, the router is ready to meet its OSPF neighbors. OSPF routers can become neighbors if they are connected to the same subnet. To discover other OSPF-speaking routers, a router sends multicast OSPF Hello packets to each interface and hopes to receive OSPF Hello packets from other routers connected to those interfaces. Figure 21-6 outlines the basic concept.

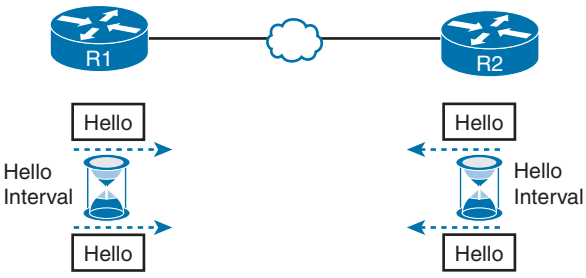



**Figure 21-6** *OSPF Hello Packets*

Routers R1 and R2 both send Hello messages onto the link. They continue to send Hellos at a regular interval based on their Hello timer settings. The Hello messages themselves have the following features:

- The Hello message follows the IP packet header, with IP protocol type 89.
- Hello packets are sent to multicast IP address 224.0.0.5, a multicast IP address intended for all OSPF-speaking routers.
- OSPF routers listen for packets sent to IP multicast address 224.0.0.5, in part hoping to receive Hello packets and learn about new neighbors.

Taking a closer look, Figure 21-7 shows several of the neighbor states used by the early formation of an OSPF neighbor relationship. The figure shows the Hello messages in the center and the resulting neighbor states on the left and right edges of the figure. Each router keeps an OSPF state variable for how it views the neighbor.

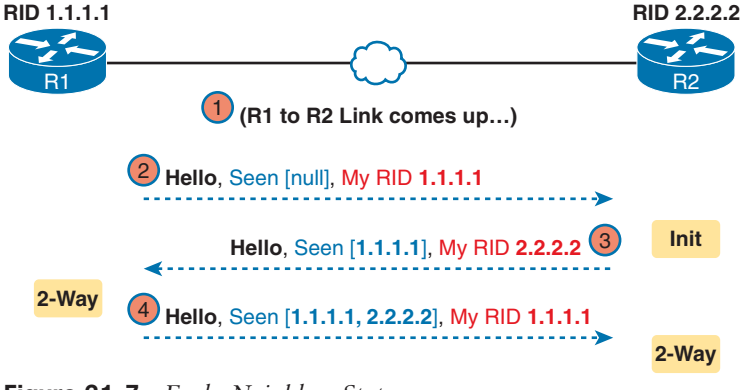



**Figure 21-7** *Early Neighbor States*

Following the steps in the figure, the scenario begins with the link down, so the routers have no knowledge of each other as OSPF neighbors. As a result, they have no state (status) information about each other as neighbors, and they would not list each other in the output of the **show ip ospf neighbor** command. At Step 2, R1 sends the first Hello, so R2 learns of the existence of R1 as an OSPF router. At that point, R2 lists R1 as a neighbor, with an interim beginning state of init.

The process continues at Step 3, with R2 sending back a Hello. This message tells R1 that R2 exists, and it allows R1 to move through the init state and quickly to a 2-way state. At Step 4, R2 receives the next Hello from R1, and R2 can also move to a 2-way state.

The **2-way state** is a particularly important OSPF state. At that point, the following major facts are true:

Key Topic

- The router received a Hello from the neighbor, with that router's own RID listed as being seen by the neighbor.
- The router has performed all checks of settings in the Hello and considers the potential neighbor to have passed all checks so they can become neighbors.
- If both routers reach a 2-way state with each other, they are neighbors and ready to exchange their LSDB with each other.

## Exchanging the LSDB Between Neighbors

One purpose of forming OSPF neighbor relationships is to allow the two neighbors to exchange their databases. This next topic works through some of the details of OSPF database exchange.

### Fully Exchanging LSAs with Neighbors

Once two routers on a link reach the 2-way state, they can immediately move on to the process of database exchange. The database exchange process can be quite involved, with several OSPF messages and several interim neighbor states. This chapter is more concerned with a few of the messages and the final state when database exchange has completed: the **full state**.

After two routers decide to exchange databases, they do not simply send the contents of the entire database. First, they tell each other a list of LSAs in their respective databases—not all the details of the LSAs, just a list. (Think of these lists as checklists.) Next, each router can check which LSAs it already has and then ask the other router for only the LSAs that are not known yet.

For instance, R1 might send R2 a checklist that lists ten LSAs (using an OSPF Database Description, or DD, packet). R2 then checks its LSDB and finds six of those ten LSAs. So, R2 asks R1 (using a Link-State Request packet) to send the four additional LSAs.

Thankfully, most OSPFv2 work does not require detailed knowledge of these specific protocol steps. However, a few of the terms are used quite a bit and should be remembered. In particular, the OSPF messages that actually send the LSAs between neighbors are called **link-state update** (LSU) packets. That is, the LSU packet holds data structures called *link-state advertisements (LSAs)*. The LSAs are not packets, but rather data structures that sit inside the LSDB and describe the topology.

Figure 21-8 pulls some of these terms and processes together, with a general example. The story picks up the example shown in Figure 21-7, with Figure 21-8 showing an example of the database exchange process between Routers R1 and R2. The center shows the protocol messages, and the outer items show the neighbor states at different points in the process. Focus on two items in particular:

- The routers exchange the LSAs inside LSU packets.
- When finished, the routers reach a full state, meaning they have fully exchanged the contents of their LSDBs.

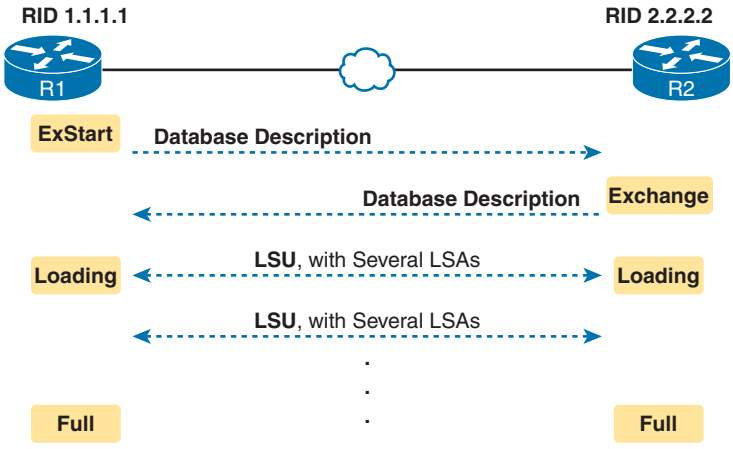



**Figure 21-8** *Database Exchange Example, Ending in a Full State*

## Maintaining Neighbors and the LSDB

Once two neighbors reach a full state, they have done all the initial work to exchange OSPF information between them. However, neighbors still have to do some small ongoing tasks to maintain the neighbor relationship.

First, routers monitor each neighbor relationship using Hello messages and two related timers: the **Hello interval** and the **Dead interval**. Routers send Hellos every Hello interval to each neighbor. Each router expects to receive a Hello from each neighbor based on the Hello interval, so if a neighbor is silent for the length of the Dead interval (by default, four times as long as the Hello interval), the loss of Hellos means that the neighbor has failed.

Next, routers must react when the topology changes as well, and neighbors play a key role in that process. When something changes, one or more routers change one or more LSAs. Then the routers must flood the changed LSAs to each neighbor so that the neighbor can change its LSDB.

For example, imagine a LAN switch loses power, so a router's G0/0 interface fails from up/up to down/down. That router updates an LSA that shows the router's G0/0 as being down. That router then sends the LSA to its neighbors, and that neighbor in turn sends it to its neighbors, until all routers again have an identical copy of the LSDB. Each router's LSDB now reflects the fact that the original router's G0/0 interface failed, so each router will then use SPF to recalculate any routes affected by the failed interface.

A third maintenance task done by neighbors is to reflood each LSA occasionally, even when the network is completely stable. By default, each router that creates an LSA also has the responsibility to reflood the LSA every 30 minutes (the default), even if no changes occur. (Note that each LSA has a separate timer, based on when the LSA was created, so there is no single big event where the network is overloaded with flooding LSAs.)

The following list summarizes these three maintenance tasks for easier review:

- Maintain neighbor state by sending Hello messages based on the Hello interval and listening for Hellos before the Dead interval expires

- Flood any changed LSAs to each neighbor
- Reflood unchanged LSAs as their lifetime expires (default 30 minutes)

### Using Designated Routers on Ethernet Links

OSPF behaves differently on some types of interfaces based on a per-interface setting called the OSPF *network type*. On Ethernet links, OSPF defaults to use a network type of *broadcast*, which causes OSPF to elect one of the routers on the same subnet to act as the **designated router (DR)**. The DR plays a key role in how the database exchange process works, with different rules than with point-to-point links.

To see how, consider the example that begins with Figure 21-9. The figure shows five OSPFv2 routers on the same Ethernet VLAN. These five OSPF routers elect one router to act as the DR and one router to be a **backup designated router (BDR)**. The figure shows A and B as DR and BDR, for no other reason than the subnet with OSPF network type broadcast will have one of each.

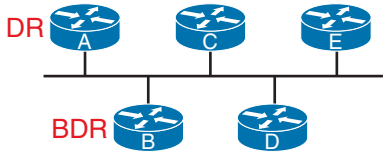



**Figure 21-9** *Routers A and B Elected as DR and BDR*

The database exchange process on an Ethernet link does not happen between every pair of routers on the same VLAN/subnet. Instead, it happens between the DR and each of the other routers, with the DR making sure that all the other routers get a copy of each LSA. In other words, the database exchange happens over the flows shown in Figure 21-10.

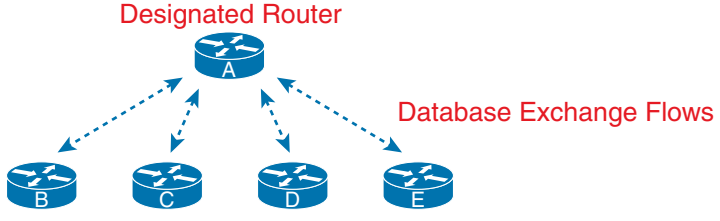



**Figure 21-10** *Database Exchange to and from the DR on an Ethernet*

OSPF defines the backup designated router (BDR) role so that some router can take over for the DR should it fail. When the DR fails, the BDR takes over as DR, and some other router is elected as the new BDR.

The use of a DR/BDR, along with the use of multicast IP addresses, makes the exchange of OSPF LSDBs more efficient on networks that allow more than two routers on the same link. The DR can send a packet to all OSPF routers in the subnet by using multicast IP address 224.0.0.5. IANA reserves this address as the "All SPF Routers" multicast address just for this purpose. For instance, in Figure 21-10, the DR can send one set of messages to all the OSPF routers rather than sending one message to each router.

Similarly, any OSPF router needing to send a message to the DR and also to the BDR (so it remains ready to take over for the DR) can send those messages to the "All SPF DRs" multicast address 224.0.0.6. So, instead of having to send one set of messages to the DR and another set to the BDR, an OSPF router can send one set of messages, making the exchange more efficient.

You will see quite a bit of the DR and BDR theory in **show** commands on a router. Because the DR and BDR both do full database exchange with all the other OSPF routers in the LAN, they reach a full state with all neighbors. However, routers that are neither a DR nor a BDR—called *DROthers* by OSPF—never reach a full state because they do not exchange LSDBs directly with each other. As a result, the **show ip ospf neighbor** command on these DROther routers lists some neighbors in a 2-way state, remaining in that state under normal operation.

For instance, with OSPF working normally on the Ethernet LAN in Figure 21-10, a **show ip ospf neighbor** command on Router C (which is a DROther router) would show the following:

- Two neighbors (A and B, the DR and BDR, respectively) with a full state (called *fully adjacent neighbors*)
- Two neighbors (D and E, which are DROthers) with a 2-way state (called **neighbors**)

OSPF requires some terms to describe all neighbors versus the subset of all neighbors that reach the full state. First, all OSPF routers on the same link that reach the 2-way state—that is, they send Hello messages and the parameters match—are called *neighbors*. The subset of neighbors for which the neighbor relationship reaches the full state are called *adjacent neighbors*. Additionally, OSPFv2 RFC 2328 emphasizes this point by defining two synonyms to the term *adjacent neighbor*: **fully adjacent** and *fully adjacent neighbor*. Finally, while the terms so far refer to the neighbor, two other terms refer to the relationship: *neighbor relationship* refers to any OSPF neighbor relationship, while the term *adjacency* refers to neighbor relationships that reach a full state. Table 21-4 details the terms.

Key Topic

**Table 21-4** Stable OSPF Neighbor States and Their Meanings

| Neighbor State | Term for Neighbor | Term for Relationship |
|---|---|---|
| 2-way | Neighbor | Neighbor Relationship |
| Full | Adjacent Neighbor<br>Fully Adjacent Neighbor | Adjacency |

## Calculating the Best Routes with SPF

OSPF LSAs contain useful information, but they do not contain the specific information that a router needs to add to its IPv4 routing table. In other words, a router cannot just copy information from the LSDB into a route in the IPv4 routing table. The LSAs individually are more like pieces of a jigsaw puzzle, with the picture shown by the completed puzzle showing a topology map of the entire network. So, to know what routes to add to the routing table, each router must do some SPF math to choose the best routes from that router's perspective. The router then adds each route to its routing table: a route with a subnet number and mask, an outgoing interface, and a next-hop router IP address.

Although engineers do not need to know the details of how SPF does the math, they do need to know how to predict which routes SPF will choose as the best route. The SPF algorithm calculates all the routes for a subnet—that is, all possible routes from the router to the

destination subnet. If more than one route exists, the router compares the metrics, picking the best (lowest) metric route to add to the routing table. Although the SPF math can be complex, engineers with a network diagram, router status information, and simple addition can calculate the metric for each route, predicting what SPF will choose.

Once SPF has identified a route, OSPF calculates the metric for a route as follows:

Key Topic

The sum of the OSPF interface costs for all outgoing interfaces in the route.

Figure 21-11 shows an example with three possible routes from R1 to Subnet X (172.16.3.0/24) at the bottom of the figure.

Key Topic

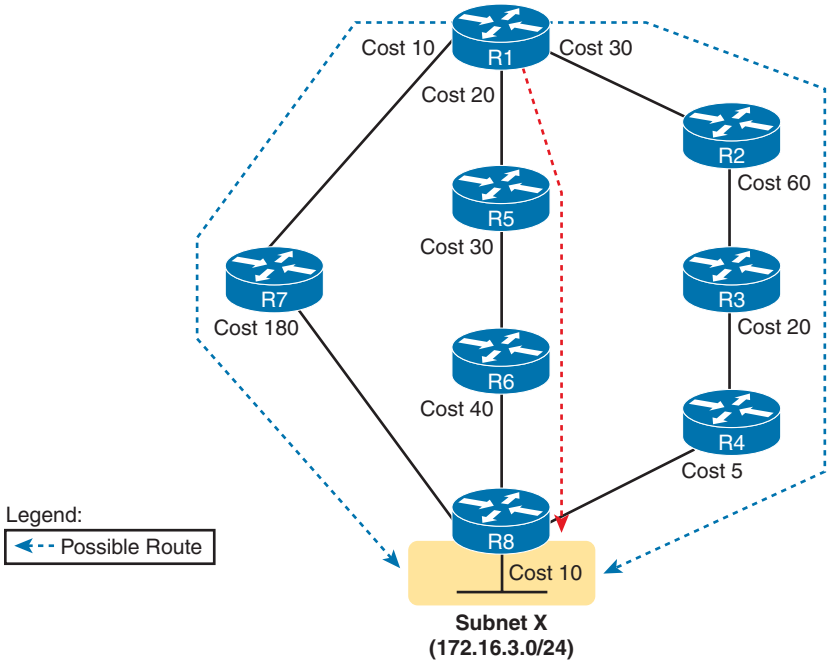



**Figure 21-11** *SPF Tree to Find R1's Route to 172.16.3.0/24*

**NOTE** OSPF considers the costs of the outgoing interfaces (only) in each route. It does not add the cost for incoming interfaces in the route.

Table 21-5 lists the three routes shown in Figure 21-11, with their cumulative costs, showing that R1's best route to 172.16.3.0/24 starts by going through R5.

**Table 21-5** Comparing R1's Three Alternatives for the Route to 172.16.3.0/24

| Route | Location in Figure 21-11 | Cumulative Cost |
|---|---|---|
| R1–R7–R8 | Left | 10 + 180 + 10 = 200 |
| R1–R5–R6–R8 | Middle | 20 + 30 + 40 + 10 = 100 |
| R1–R2–R3–R4–R8 | Right | 30 + 60 + 20 + 5 + 10 = 125 |

As a result of the SPF algorithm's analysis of the LSDB, R1 adds a route to subnet 172.16.3.0/24 to its routing table, with the next-hop router of R5.

In real OSPF networks, an engineer can do the same process by knowing the OSPF cost for each interface. Armed with a network diagram, the engineer can examine all routes, add the costs, and predict the metric for each route.

## OSPF Areas and LSAs

OSPF can be used in some networks with very little thought about design issues. You just turn on OSPF in all the routers, put all interfaces into the same area (usually area 0), and it works! Figure 21-12 shows one such network example, with 11 routers and all interfaces in area 0.

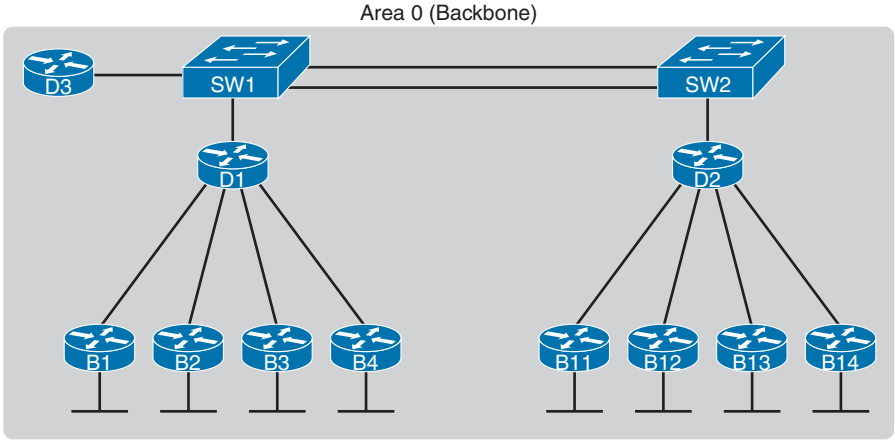



**Figure 21-12** *Single-Area OSPF*

Using a single-area design works well in small- to medium-sized networks. In fact, the CCNA 200-301 V1.1 blueprint specifically mentions **single-area OSPF**, omitting **multiarea OSPF**. However, it helps to think through the concept of areas even for CCNA preparation. The next few pages look at how OSPF area design works, with more reasons as to why areas help make larger OSPF networks work better.

### OSPF Areas

OSPF area design follows a couple of basic rules. To apply the rules, start with a clean drawing of the internetwork, with routers, and all interfaces. Then choose the area for each router interface, as follows:

Key Topic

- Put all interfaces connected to the same subnet inside the same area.
- An area should be contiguous.
- Some routers may be internal to an area, with all interfaces assigned to that single area.
- Some routers may be Area Border Routers (ABRs) because some interfaces connect to the backbone area, and some connect to nonbackbone areas.
- All nonbackbone areas must have a path to reach the backbone area (area 0) by having at least one ABR connected to both the backbone area and the nonbackbone area.

Figure 21-13 shows one example. An engineer started with a network diagram that showed all 11 routers and their links. On the left, the engineer put four WAN links and the LANs connected to branch routers B1 through B4 into area 1. Similarly, he placed the links to branches B11 through B14 and their LANs in area 2. Both areas need a connection to the backbone area, area 0, so he put the LAN interfaces of D1 and D2 into area 0, along with D3, creating the backbone area.

Key Topic

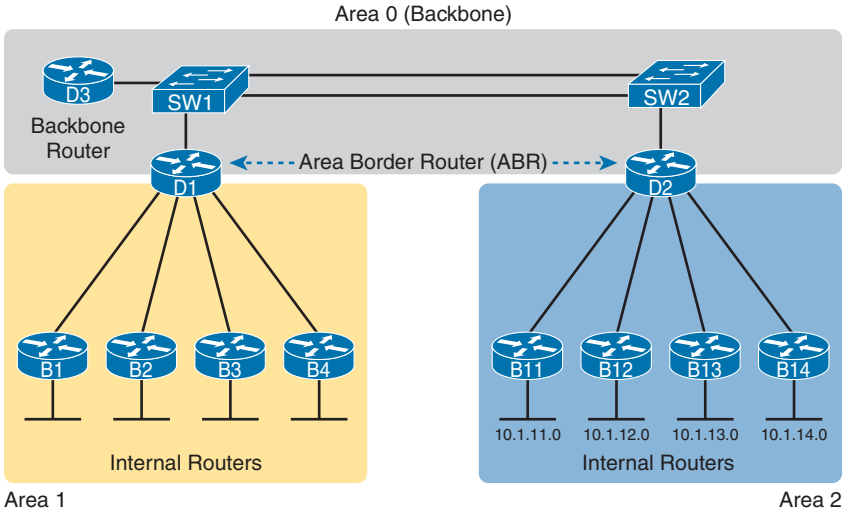



**Figure 21-13** *Three-Area OSPF with D1 and D2 as ABRs*

The figure also shows a few important OSPF area design terms. Table 21-6 summarizes the meaning of these terms, plus some other related terms, but pay closest attention to the terms from the figure.

Key Topic

**Table 21-6** OSPF Design Terminology

| Term | Description |
|---|---|
| **Area Border Router (ABR)** | An OSPF router with interfaces connected to the backbone area and to at least one other area |
| *Backbone router* | A router connected to the backbone area (includes ABRs) |
| **Internal router** | A router in one area (not the backbone area) |
| Area | A set of routers and links that shares the same detailed LSDB information, but not with routers in other areas, for better efficiency |
| **Backbone area** | A special OSPF area to which all other areas must connect—area 0 |
| Intra-area route | A route to a subnet inside the same area as the router |
| Interarea route | A route to a subnet in an area of which the router is not a part |

## How Areas Reduce SPF Calculation Time

Figure 21-13 shows a sample area design and some terminology related to areas, but it does not show the power and benefit of the areas. To understand how areas reduce the work SPF has to do, you need to understand what changes about the LSDB inside an area, as a result of the area design.

SPF spends most of its processing time working through all the topology details, namely routers and the links that connect routers. Areas reduce SPF's workload because, for a given area, the LSDB lists only routers and links inside that area, as shown on the left side of Figure 21-14.

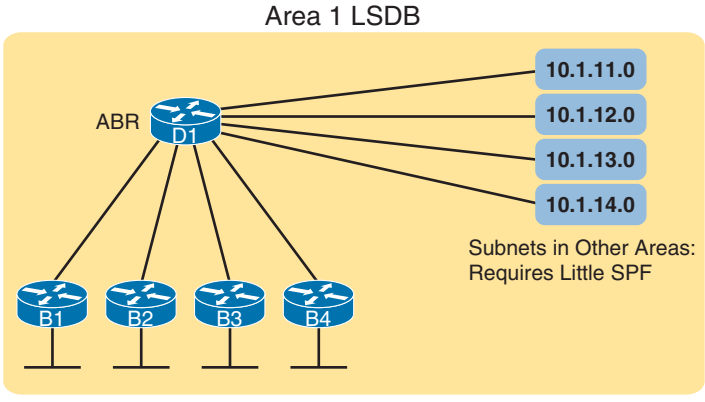



**Figure 21-14** *Smaller Area 1 LSDB Concept*

While the LSDB has less topology information, it still needs information about all subnets in all areas so that each router can create IPv4 routes for all subnets. So, with an area design, OSPFv2 uses brief summary information about the subnets in other areas. These summary LSAs do not include topology information about the other areas; however, each summary LSA *does* list a subnet ID and mask of a subnet in some other area. Summary LSAs do not require SPF processing at all. Instead, these subnets all appear like subnets connected to the ABR (in Figure 21-14, ABR D1).

Using multiple areas improves OSPF operations in many ways for larger networks. The following list summarizes some of the key points arguing for the use of multiple areas in larger OSPF networks:

- Routers require fewer CPU cycles to process the smaller per-area LSDB with the SPF algorithm, reducing CPU overhead and improving convergence time.
- The smaller per-area LSDB requires less memory.
- Changes in the network (for example, links failing and recovering) require SPF calculations only on routers in the area where the link changed state, reducing the number of routers that must rerun SPF.
- Less information must be advertised between areas, reducing the bandwidth required to send LSAs.



## (OSPFv2) Link-State Advertisements

Many people tend to get a little intimidated by OSPF LSAs when first learning about them. Commands like **show ip ospf database** in its many variations list a lot of information about the LSDB. Those details appear to be in some kind of code, using lots of numbers. It can seem like a bit of a mess.

However, if you examine LSAs while thinking about OSPF areas and area design, some of the most common LSA types will make a lot more sense. For instance, think about the LSDB in one area. The topology in one area includes routers and the links between the routers. As it turns out, OSPF defines the first two types of LSAs to define those exact details, as follows:

- One *router LSA* for each router in the area
- One *network LSA* for each network that has a DR plus one neighbor of the DR

Next, think about the subnets in the other areas. The ABR creates summary information about each subnet in one area to advertise into other areas—basically just the subnet IDs and masks—as a third type of LSA:

- One *summary LSA* for each subnet ID that exists in a different area

The next few pages discuss these three LSA types in a little more detail; Table 21-7 lists some information about all three for easier reference and study.

**Table 21-7** The Three OSPFv2 LSA Types Seen with a Multiarea OSPF Design

| LSA Name | LSA Type | Primary Purpose | Contents of LSA |
|---|---|---|---|
| Router | 1 | Describe a router | RID, interfaces, IP address/mask, current interface state (status) |
| Network | 2 | Describe a network that has a DR and BDR | DR and BDR IP addresses, subnet ID, mask |
| Summary | 3 | Describe a subnet in another area | Subnet ID, mask, RID of ABR that advertises the LSA |

### Router LSAs Build Most of the Intra-Area Topology

OSPF needs very detailed topology information inside each area. The routers inside area X need to know all the details about the topology inside area X. And the mechanism to give routers all these details is for the routers to create and flood router (Type 1) and network (Type 2) LSAs about the routers and links in the area.

Router LSAs, also known as Type 1 LSAs, describe the router in detail. Each lists a router's RID, its interfaces, its IPv4 addresses and masks, its interface state, and notes about what neighbors the router knows about via each of its interfaces.

To see a specific instance, first review Figure 21-15. It lists internetwork topology, with subnets listed. Because it's a small internetwork, the engineer chose a single-area design, with all interfaces in backbone area 0.

With the single-area design planned for this small internetwork, the LSDB will contain four router LSAs. Each router creates a router LSA for itself, with its own RID as the LSA identifier. The LSA lists that router's own interfaces, IP address/mask, with pointers to neighbors.

Once all four routers have copies of all four router LSAs, SPF can mathematically analyze the LSAs to create a model. The model looks a lot like the concept drawing in Figure 21-16. Note that the drawing shows each router with an obvious RID value. Each router has pointers that represent each of its interfaces, and because the LSAs identify neighbors, SPF can figure out which interfaces connect to which other routers.

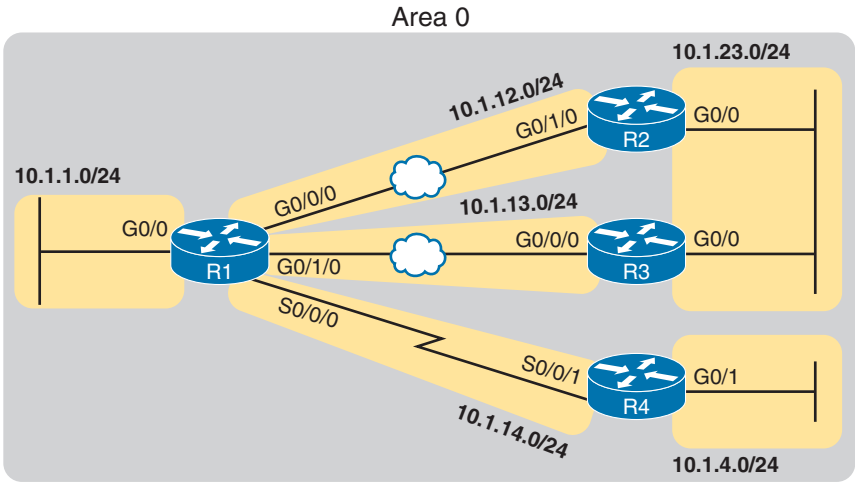



**Figure 21-15** *Enterprise Network with Six IPv4 Subnets*

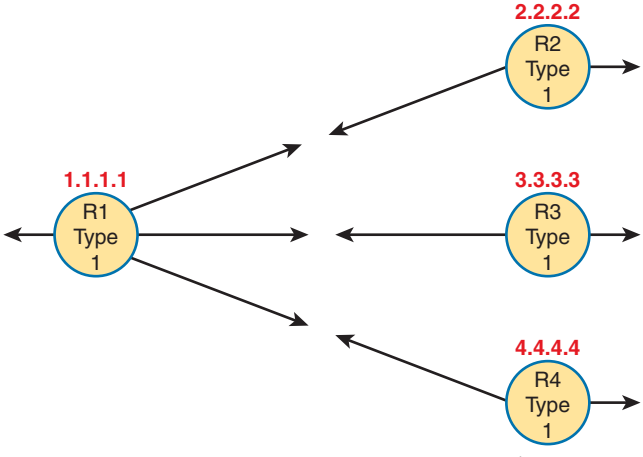



**Figure 21-16** *Type 1 LSAs, Assuming a Single-Area Design*

## Network LSAs Complete the Intra-Area Topology

Whereas router LSAs define most of the intra-area topology, network LSAs define the rest. As it turns out, when OSPF elects a DR on some subnet *and* that DR has at least one neighbor, OSPF treats that subnet as another node in its mathematical model of the network. To represent that network, the DR creates and floods a network (Type 2) LSA for that network (subnet).

For instance, back in Figure 21-15, one Ethernet LAN and two Ethernet WANs exist. The Ethernet LAN between R2 and R3 will elect a DR, and the two routers will become neighbors; so, whichever router is the DR will create a network LSA. Similarly, R1 and R2 connect with an Ethernet WAN, so the DR on that link will create a network LSA. Likewise, the DR on the Ethernet WAN link between R1 and R3 will also create a network LSA.

Figure 21-17 shows the completed version of the intra-area LSAs in area 0 with this design. Note that the router LSAs reference the network LSAs when they exist, which lets the SPF processes connect the pieces together.

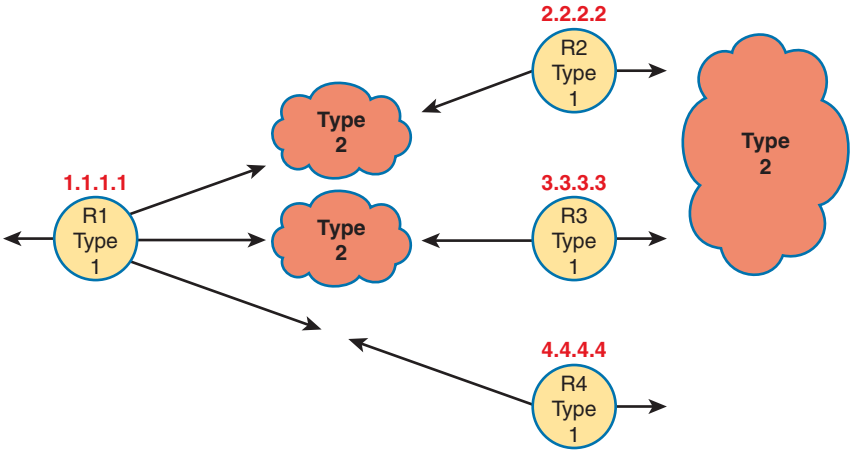



**Figure 21-17** *Type 1 and Type 2 LSAs in Area 0, Assuming a Single-Area Design*

Finally, note that in this single-area design example, no summary (Type 3) LSAs exist at all. These LSAs represent subnets in other areas, and there are no other areas. Given that the CCNA 200-301 V1.1 exam blueprint refers specifically to single-area OSPF designs, this section stops at showing the details of the intra-area LSAs (Types 1 and 2).

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 21-8 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 21-8** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used: |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Key Topic

**Table 21-9** Key Topics for Chapter 21

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Functions of IP routing protocols | 541 |
| List | Definitions of IGP and EGP | 542 |
| List | Types of IGP routing protocols | 543 |
| Table 21-2 | IGP metrics | 544 |
| List | Key facts about the OSPF 2-way state | 550 |
| Table 21-4 | Key OSPF neighbor states | 553 |
| Item | Definition of how OSPF calculates the cost for a route | 554 |
| Figure 21-11 | Example of calculating the cost for multiple competing routes | 554 |
| List | OSPF area design rules | 555 |
| Figure 21-13 | Sample OSPF multiarea design with terminology | 556 |
| Table 21-6 | OSPF design terms and definitions | 556 |

## Key Terms You Should Know

2-way state, Area Border Router (ABR), backbone area, backup designated router (BDR), convergence, Dead interval, designated router (DR), distance vector, full state, fully adjacent, Hello interval, interior gateway protocol (IGP), internal router, link-state, link-state advertisement (LSA), link-state database (LSDB), link-state update, metric, multiarea OSPF, neighbor, router ID (RID), Shortest Path First (SPF) algorithm, single-area OSPF



CHAPTER 22

# Implementing Basic OSPF Features

**This chapter covers the following exam topics:**

- **3.0 IP Connectivity**
  - **3.2 Determine how a router makes a forwarding decision by default**
    - **3.2.b Administrative distance**
    - **3.2.c Routing protocol metric**
  - **3.4 Configure and verify single area OSPFv2**
    - **3.4.a Neighbor adjacencies**
    - **3.4.b Point-to-point**
    - **3.4.c Broadcast (DR/BR selection)**
    - **3.4.d Router ID**

OSPFv2 requires only a few configuration commands if you rely on default settings. To use OSPF, all you need to do is enable OSPF on each interface you intend to use in the network, and OSPF uses messages to discover neighbors and learn routes through those neighbors. OSPF performs many background tasks, and you can discover details about that work using a large number of OSPF **show** commands. However, configuring OSPF, using mostly default settings for all the optional features, requires only a few commands. This chapter sets about to help you learn those minimal settings.

The first major section of this chapter focuses on traditional OSPFv2 configuration using the **network** command, along with the large variety of associated **show** commands. This section teaches you how to make OSPFv2 operate with default settings and convince yourself that it really is working through use of those **show** commands.

The second major section shows an alternative configuration option called OSPF interface mode, in contrast with the traditional OSPF configuration shown in the first section of the chapter. This mode uses the **ip ospf** *process-id* **area** *area-number* configuration command instead of the **network** command.

Along the way, the first major section includes the detail of how to set the OSPF router ID (RID). While optional, configuring a predictable and stable OSPF RID allows easier operation and troubleshooting of OSPF and may be the most important of the optional OSPF settings.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom

of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 22-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Implementing OSPFv2 Using network Commands | 1–4 |
| Implementing OSPFv2 Using Interface Subcommands | 5, 6 |

1. Which of the following **network** commands, following the command **router ospf 1**, enables OSPF on interfaces whose IP addresses are 10.1.1.1, 10.1.100.1, and 10.1.120.1?
   a. **network 10.0.0.0 0.0.0.0 area 0**
   b. **network 10.0.0.0 0.255.255.255 area 0**
   c. **network 10.0.0.0 0.0.0.255 area 0**
   d. **network 10.0.0.0 0.0.255.255 area 0**
2. Which of the following **network** commands, following the command **router ospf 1**, tells this router to start using OSPF on interfaces whose IP addresses are 10.1.1.1, 10.1.100.1, and 10.1.120.1?
   a. **network 10.1.0.0 0.0.255.255 area 0**
   b. **network 10.0.0.0 0.255.255.0 area 0**
   c. **network 10.1.1.0 0.x.1x.0 area 0**
   d. **network 10.1.1.0 255.0.0.0 area 0**
3. Which of the following commands list the OSPF neighbors off interface serial 0/0? (Choose two answers.)
   a. **show ip ospf neighbor**
   b. **show ip ospf interface brief**
   c. **show ip neighbor**
   d. **show ip interface**
   e. **show ip ospf neighbor serial 0/0**
4. When reloading and choosing a new OSPF router ID (RID), a router had working interfaces loopback 1 with IP address 10.8.8.8, loopback 2 with address 10.7.7.7, and GigabitEthernet0/0/0 with 10.9.9.9. The router did not have a **router-id** command in the OSPF process configuration. What RID did the router choose?
   a. 10.7.7.7
   b. 10.8.8.8
   c. 10.9.9.9
   d. The router would fail to choose an RID.

5. An engineer migrates from a more traditional OSPFv2 configuration that uses **network** commands in OSPF configuration mode to instead use OSPFv2 interface configuration. Which of the following commands configures the area number assigned to an interface in this new configuration?
   a. The **area** command in interface configuration mode
   b. The **ip ospf** command in interface configuration mode
   c. The **router ospf** command in interface configuration mode
   d. The **network** command in interface configuration mode
6. An enterprise avoids using the OSPF **network** command, instead preferring to enable OSPF per-interface with the **ip ospf** *process-id* **area** *area-id* interface subcommand. Which **show** command identifies whether an interface has been configured with the **ip ospf** *process-id* **area** *area-id* interface subcommand? (Choose two answers.)
   a. The **show ip ospf interface** command
   b. The **show ip ospf interface brief** command
   c. The **show ip ospf neighbor** command
   d. The **show ip protocols** command

# Foundation Topics

## Implementing OSPFv2 Using network Commands

After an OSPF design has been chosen—a task that can be complex in larger IP internetworks—the configuration can be as simple as enabling OSPF on each router interface and placing that interface in the correct OSPF area. This first major section of the chapter focuses on the required configuration using the traditional OSPFv2 **network** command along with one optional configuration setting: how to set the **OSPF router-id**. Additionally, this section works through how to show the various lists and tables that confirm how OSPF is working.

For reference and study, the following list outlines the configuration steps covered in this first major section of the chapter:

Config Checklist

**Step 1.** Use the **router ospf** *process-id* global command to enter OSPF configuration mode for a particular OSPF process.

**Step 2.** (Optional) Configure the OSPF router ID by doing the following:
   a. Use the **router-id** *id-value* router subcommand to define the router ID, or
   b. Use the **interface loopback** *number* global command, along with an **ip address** *address mask* command, to configure an IP address on a loopback interface (chooses the highest IP address of all working loopbacks), or
   c. Rely on an interface IP address (chooses the highest IP address of all working nonloopbacks).

**Step 3.** Use one or more **network** *ip-address wildcard-mask* **area** *area-id* router subcommands to enable OSPFv2 on any interfaces matched by the configured address and mask, enabling OSPF on the interface for the listed area.

Figure 22-1 shows the relationship between the OSPF configuration commands, with the idea that the configuration creates a routing process in one part of the configuration, and then indirectly enables OSPF on each interface. The configuration does not name the interfaces on which OSPF is enabled, instead requiring IOS to apply some logic by comparing the OSPF **network** command to the interface **ip address** commands. The upcoming example discusses more about this logic.

Key Topic

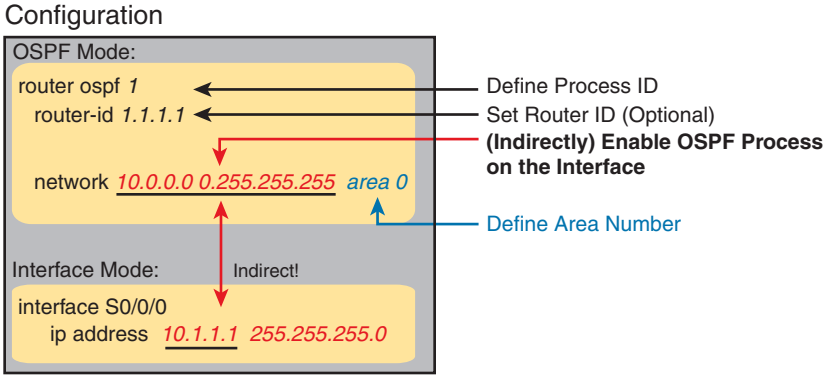



**Figure 22-1** *Organization of OSPFv2 Configuration with the* **network** *Command*

## OSPF Single-Area Configuration

Figure 22-2 shows a sample network that will be used for most examples throughout this chapter. All links reside in area 0, making the area design a single-area design, with four routers. You can think of Router R1 as a router at a central site, with WAN links to each remote site. Routers R2 and R3 might be at one large remote site that needs two WAN links and two routers for WAN redundancy, with both routers connected to the LAN at that remote site. Router R4 might be a typical smaller remote site with a single router needed for that site.

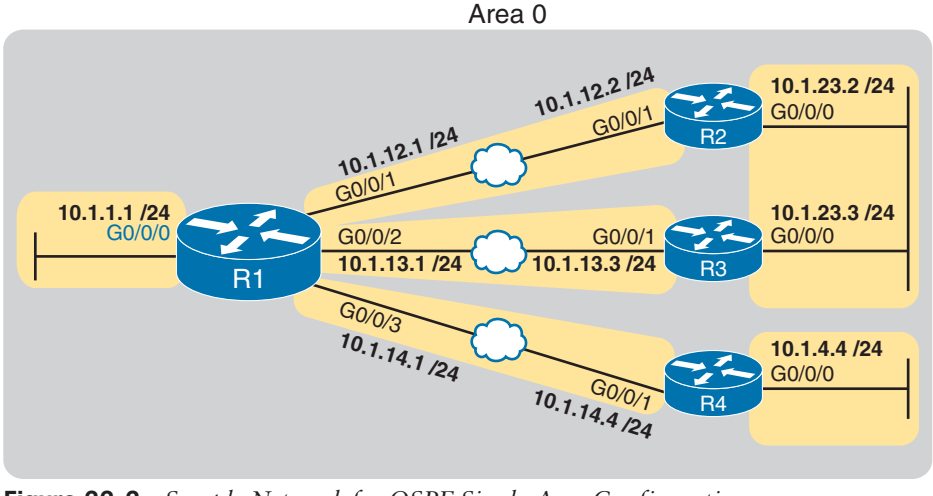



**Figure 22-2** *Sample Network for OSPF Single-Area Configuration*

Example 22-1 shows the IPv4 addressing configuration on Router R1, before getting into the OSPF detail.

**Example 22-1** *IPv4 Address Configuration on R1*

```
interface GigabitEthernet0/0/0
 ip address 10.1.1.1 255.255.255.0
!
interface GigabitEthernet0/0/1
 ip address 10.1.12.1 255.255.255.0
!
interface GigabitEthernet0/0/2
 ip address 10.1.13.1 255.255.255.0
!
interface GigabitEthernet0/0/3
 ip address 10.1.14.1 255.255.255.0
```

The OSPF configuration begins with the **router ospf** *process-id* global command, which puts the user in OSPF configuration mode, and sets the OSPF *process-id* value. The *process-id* number just needs to be unique on the local router, matching between various commands in a router. The *process-id* does not need to match between neighboring routers or other routers in the same area. The value can be any integer between 1 and 65,535.

Second, the configuration needs one or more **network** commands in OSPF mode. These commands tell the router to find its local interfaces that match the first two parameters on the **network** command. Then, for each matched interface, the router enables OSPF on those interfaces, discovers neighbors, creates neighbor relationships, and assigns the interface to the area listed in the **network** command. (Note that the area can be configured as either an integer or a dotted-decimal number, but this book makes a habit of configuring the area number as an integer. The integer area numbers range from 0 through 4,294,967,295.)

Example 22-2 shows an example configuration on Router R1 from Figure 22-2. The **router ospf 1** command enables OSPF process 1, and the single **network** command enables OSPF on all interfaces shown in the figure.

**Example 22-2** *OSPF Single-Area Configuration on R1 Using One* **network** *Command*

```
router ospf 1
 network 10.0.0.0 0.255.255.255 area 0
```

For the specific **network** command in Example 22-2, any matched interfaces are assigned to area 0. However, the first two parameters—the *ip_address* and *wildcard_mask* parameter values of 10.0.0.0 and 0.255.255.255—need some explaining. In this case, the command matches all interfaces shown for Router R1; the next topic explains why.

## Wildcard Matching with the network Command

The key to understanding the traditional OSPFv2 configuration shown in this first example is to understand the OSPF **network** command. The OSPF **network** command compares the

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** A **3** A, E **4** B **5** B **6** A, D

first parameter in the command to each interface IP address on the local router, trying to find a match. However, rather than comparing the entire number in the **network** command to the entire IPv4 address on the interface, the router can compare a subset of the octets, based on the wildcard mask, as follows:

Key Topic

**Wildcard 0.0.0.0:** Compare all four octets. In other words, the numbers must exactly match.

**Wildcard 0.0.0.255:** Compare the first three octets only. Ignore the last octet when comparing the numbers.

**Wildcard 0.0.255.255:** Compare the first two octets only. Ignore the last two octets when comparing the numbers.

**Wildcard 0.255.255.255:** Compare the first octet only. Ignore the last three octets when comparing the numbers.

**Wildcard 255.255.255.255:** Compare nothing; this wildcard mask means that all addresses will match the **network** command.

Basically, a wildcard mask value of decimal 0 in an octet tells IOS to compare to see if the numbers match, and a value of 255 tells IOS to ignore that octet when comparing the numbers.

The **network** command provides many flexible options because of the wildcard mask. For example, in Router R1, many **network** commands could be used, with some matching all interfaces, and some matching a subset of interfaces. Table 22-2 shows a sampling of options, with notes.

**Table 22-2** Example OSPF **network** Commands on R1, with Expected Results

| Command | Logic in Command | Matched Interfaces |
|---|---|---|
| **network 10.1.0.0 0.0.255.255** | Match addresses that begin with 10.1 | G0/0/0<br>G0/0/1<br>G0/0/1<br>G0/0/2 |
| **network 10.0.0.0 0.255.255.255** | Match addresses that begin with 10 | G0/0/0<br>G0/0/1<br>G0/0/1<br>G0/0/2 |
| **network 0.0.0.0 255.255.255.255** | Match all addresses | G0/0/0<br>G0/0/1<br>G0/0/1<br>G0/0/2 |
| **network 10.1.13.0 0.0.0.255** | Match addresses that begin with 10.1.13 | G0/0/2 |
| **network 10.1.13.1 0.0.0.0** | Match one address: 10.1.13.1 | G0/0/2 |

The wildcard mask gives the local router its rules for matching its own interfaces. To show examples of the different options, Example 22-3 shows the configuration on routers R2, R3, and R4, each using different wildcard masks. Note that all three routers (R2, R3, and R4) enable OSPF on all the interfaces shown in Figure 22-2.

**Example 22-3** *OSPF Configuration on Routers R2, R3, and R4*

```
! R2 configuration next - one network command enables OSPF on both interfaces
interface GigabitEthernet0/0/0
 ip address 10.1.23.2 255.255.255.0
!
interface GigabitEthernet0/0/1
 ip address 10.1.12.2 255.255.255.0
!
router ospf 1
 network 10.0.0.0 0.255.255.255 area 0
```

```
! R3 configuration next - One network command per interface
interface GigabitEthernet0/0/0
 ip address 10.1.23.3 255.255.255.0
!
interface GigabitEthernet0/0/1
 ip address 10.1.13.3 255.255.255.0
!
router ospf 1
 network 10.1.13.3 0.0.0.0 area 0
 network 10.1.23.3 0.0.0.0 area 0
```

```
! R4 configuration next - One network command per interface with wildcard 0.0.0.255
interface GigabitEthernet0/0/0
 ip address 10.1.4.4 255.255.255.0
!
interface GigabitEthernet0/0/1
 ip address 10.1.14.4 255.255.255.0
!
router ospf 1
 network 10.1.14.0 0.0.0.255 area 0
 network 10.1.4.0 0.0.0.255 area 0
```

Finally, note that OSPF uses the same wildcard mask logic as defined by Cisco IOS access control lists. The section titled "Finding the Right Wildcard Mask to Match a Subnet" section in Chapter 6 of the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, provides more detail about wildcard masks.

**NOTE** If the wildcard mask octet in a **network** command is 255, the matching address octet should be configured as a 0. Interestingly, IOS will accept a **network** command that breaks this rule, but if you configure a wildcard mask octet as 255, then IOS changes the corresponding address octet to a 0 before putting it into the running configuration file. For example, IOS will change a typed command that begins with **network 1.2.3.4 0.0.255.255** to **network 1.2.0.0 0.0.255.255**.

## Verifying OSPF Operation

As mentioned in Chapter 21, "Understanding OSPF Concepts," OSPF routers use a three-step process to eventually add OSPF-learned routes to the IP routing table. First, they create neighbor relationships. Then they build and flood LSAs between those neighbors so each router in the same area has a copy of the same LSDB. Finally, each router independently computes its own IP routes using the SPF algorithm and adds them to its routing table. This next topic works through how to display the results of each of those steps, which lets you confirm whether OSPF has worked correctly or not.

The **show ip ospf neighbor**, **show ip ospf database**, and **show ip route** commands display information to match each of these three steps, respectively. Figure 22-3 summarizes the commands you can use (and others) when verifying OSPF.

Key Topic

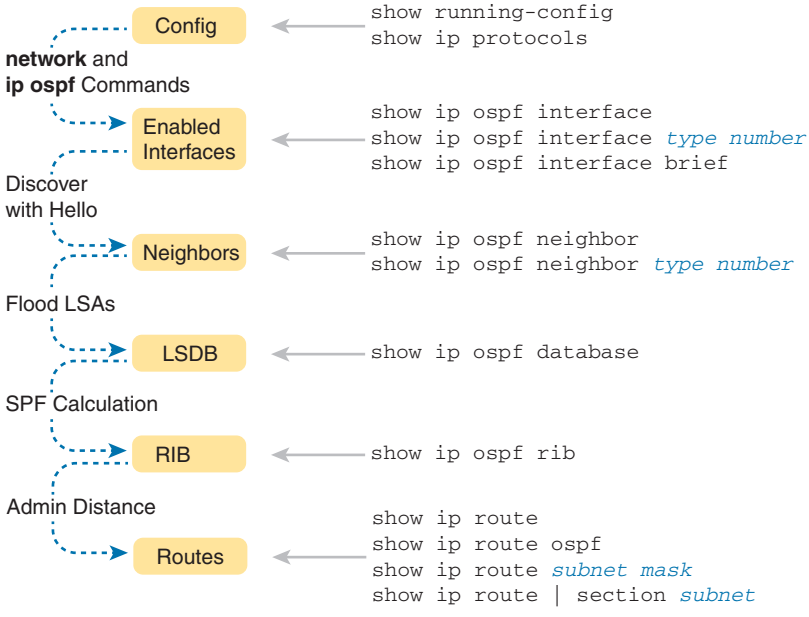



**Figure 22-3** *OSPF Verification Commands*

Many engineers begin OSPF verification by looking at the output of the **show ip ospf neighbor** command. For instance, Example 22-4 shows a sample from Router R1, which should have one neighbor relationship each with routers R2, R3, and R4. Example 22-4 shows all three.

Key Topic

**Example 22-4** *OSPF Neighbors on Router R1 from Figure 22-2*

```
R1# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   FULL/DR         00:00:37    10.1.12.2       GigabitEthernet0/0/1
3.3.3.3           1   FULL/DR         00:00:37    10.1.13.3       GigabitEthernet0/0/2
4.4.4.4           1   FULL/BDR        00:00:34    10.1.14.4       GigabitEthernet0/0/3
```

The detail in the output mentions several important facts, and for most people, working right to left works best in this case. For example, look at the headings:

**Interface:** This is the local router's interface connected to the neighbor. For example, the first neighbor in the list is reachable through R1's G0/0/1 interface.

**Address:** This is the neighbor's IP address on that link. Again, this first neighbor, the neighbor, which is R2, uses IP address 10.1.12.2.

**State:** While many possible states exist, for the details discussed in this chapter, FULL is the correct and fully working state in this case.

**Neighbor ID:** This is the router ID of the neighbor.

**NOTE** Examples 22-4 through 22-8 use configuration not shown here that sets the RID values to easily identify the routers, using 2.2.2.2 for Router R2, 3.3.3.3 for Router R3, and so on. The upcoming section "Configuring the OSPF Router ID" shows how to set the RID.

Once OSPF convergence has completed, a router should list each neighbor. On links that use a designated router (DR), the state will also list the role of the neighboring router after the / (DR, BDR, or DROther). As a result, the normal working states will be:

Key Topic

**FULL/ -:** The neighbor state is full, with the "-" instead of letters meaning that the link does not use a DR/BDR.

**FULL/DR:** The neighbor state is full, and the neighbor is the DR.

**FULL/BDR:** The neighbor state is full, and the neighbor is the backup DR (BDR).

**FULL/DROTHER:** The neighbor state is full, and the neighbor is neither the DR nor BDR. (It also implies that the local router is a DR or BDR because the state is FULL.)

**2WAY/DROTHER:** The neighbor state is 2-way, and the neighbor is neither the DR nor BDR—that is, a DROther router. (It also implies that the local router is also a DROther router because otherwise the state would reach a full state.)

Once a router's OSPF process forms a working neighbor relationship, the routers exchange the contents of their LSDBs, either directly or through the DR on the subnet. Example 22-5 shows the contents of the LSDB on Router R1. Interestingly, with a single-area design, all the routers will have the same LSDB contents once all neighbors are up and all LSAs have been exchanged. So, the **show ip ospf database** command in Example 22-5 should list the same exact information, no matter on which of the four routers it is issued.

**Example 22-5** *OSPF Database on Router R1 from Figure 22-2*

```
R1# show ip ospf database

            OSPF Router with ID (1.1.1.1) (Process ID 1)


                Router Link States (Area 0)
```

```
Link ID         ADV Router      Age         Seq#       Checksum Link count
1.1.1.1         1.1.1.1         431         0x8000008F 0x00DCCA 5
2.2.2.2         2.2.2.2         1167        0x8000007F 0x009DA1 2
3.3.3.3         3.3.3.3         441         0x80000005 0x002FB1 1
4.4.4.4         4.4.4.4         530         0x80000004 0x007F39 2


                Net Link States (Area 0)

Link ID         ADV Router      Age         Seq#       Checksum
10.1.12.2       2.2.2.2         1167        0x8000007C 0x00BBD5
10.1.13.3       3.3.3.3         453         0x80000001 0x00A161
10.1.14.1       4.4.4.4         745         0x8000007B 0x004449
10.1.23.3       3.3.3.3         8           0x80000001 0x00658F
```

For the purposes of this book, do not be concerned about the specifics in the output of this command. However, for perspective, note that the LSDB should list one "Router Link State" (Type 1 Router LSA) for each of the routers in the same area, so with the design based on Figure 22-2, the output lists four Type 1 LSAs. Also, with all default settings in this design, the routers will create a total of four Type 2 Network LSAs as shown, one each for the subnets that have a DR and contain at least two routers in that subnet (the three WAN links plus the LAN to which both R2 and R3 connect).

Next, Example 22-6 shows R4's IPv4 routing table with the **show ip route** command. As configured, with all links working, R4 has connected routes to two of those subnets and should learn OSPF routes to the other subnets.

**Example 22-6** *IPv4 Routes Added by OSPF on Router R4 from Figure 22-2*

```
R4# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
! Additional legend lines omitted for brevity

Gateway of last resort is not set

      10.0.0.0/8 is variably subnetted, 9 subnets, 2 masks
O        10.1.1.0/24 [110/2] via 10.1.14.1, 00:27:24, GigabitEthernet0/0/1
C        10.1.4.0/24 is directly connected, GigabitEthernet0/0/0
L        10.1.4.4/32 is directly connected, GigabitEthernet0/0/0
O        10.1.12.0/24 [110/2] via 10.1.14.1, 00:27:24, GigabitEthernet0/0/1
O        10.1.13.0/24 [110/2] via 10.1.14.1, 00:25:15, GigabitEthernet0/0/1
C        10.1.14.0/24 is directly connected, GigabitEthernet0/0/1
L        10.1.14.4/32 is directly connected, GigabitEthernet0/0/1
O        10.1.23.0/24 [110/3] via 10.1.14.1, 00:27:24, GigabitEthernet0/0/1
```

Any time you want to check OSPF on a router in a small design like the ones in the book, you can count all the subnets, then count the subnets connected to the local router, and know that OSPF should learn routes to the rest of the subnets. Then just use the **show ip route** command and add up how many connected and OSPF routes exist as a quick check of whether all the routes have been learned or not.

In this case, Router R4 has two connected subnets, but six subnets exist per the figure, so Router R4 should learn four OSPF routes. Next look for the code of "O" on the left, which identifies a route as being learned by OSPF. The output lists four such IP routes: one for the LAN subnet off Router R1, one for the LAN subnet connected to both R2 and R3, and one each for the WAN subnets from R1 to R2 and R1 to R3.

Next, examine the first route (to subnet 10.1.1.0/24). It lists the subnet ID and mask, identifying the subnet. It also lists two numbers in brackets. The first, 110, is the administrative distance of the route. All the OSPF routes in this example use the default of 110 (see Table 24-4 in Chapter 24, "OSPF Neighbors and Route Selection," for the list of administrative distance values). The second number, 2, is the OSPF metric for this route. The route also lists the forwarding instructions: the next-hop IP address (10.1.14.1) and R4's outgoing interface (G0/0/1).

**NOTE** The section "Floating Static Routes" in Chapter 17, "Configuring IPv4 Addresses and Static Routes," introduced the concept of administrative distance; however, the section "Multiple Routes Learned from Competing Sources," in Chapter 24 discusses the topic in more depth.

## Verifying OSPF Configuration

Once you can configure OSPF with confidence, you will likely verify OSPF focusing on **OSPF neighbors** and the IP routing table as just discussed. However, if OSPF does not work immediately, you may need to circle back and check the configuration. To do so, you can use these steps:

- If you have enable mode access, use the **show running-config** command to examine the configuration.
- If you have only user mode access, use the **show ip protocols** command to re-create the OSPF configuration.
- Use the **show ip ospf interface [brief]** command to determine whether the router enabled OSPF on the correct interfaces or not based on the configuration.

The best way to verify the configuration begins with the **show running-config** command, of course. However, the **show ip protocols** command repeats the details of the OSPFv2 configuration and does not require enable mode access. Example 22-7 does just that for Router R3.

**Example 22-7** *Router R3 Configuration and the* **show ip protocols** *Command*

```
R3# show running-config | section router ospf 1
router ospf 1
  network 10.1.13.3 0.0.0.0 area 0
  network 10.1.23.3 0.0.0.0 area 0
  router-id 3.3.3.3

R3# show ip protocols
*** IP Routing is NSF aware ***

Routing Protocol is "ospf 1"
  Outgoing update filter list for all interfaces is not set
  Incoming update filter list for all interfaces is not set
  Router ID 3.3.3.3
  Number of areas in this router is 1. 1 normal 0 stub 0 nssa
  Maximum path: 4
  Routing for Networks:
    10.1.13.3 0.0.0.0 area 0
    10.1.23.3 0.0.0.0 area 0
  Routing Information Sources:
    Gateway         Distance      Last Update
    1.1.1.1              110      02:05:26
    4.4.4.4              110      02:05:26
    2.2.2.2              110      01:51:16
  Distance: (default is 110)
```

The highlighted output emphasizes some of the configuration. The first highlighted line repeats the parameters on the **router ospf 1** global configuration command. (The second highlighted item points out the router's router ID, which will be discussed in the next section.) The third set of highlighted lines begins with a heading of "Routing for Networks:" followed by two lines that closely resemble the parameters on the configured **network** commands. In fact, closely compare those last two highlighted lines with the **network** configuration commands at the top of the example, and you will see that they mirror each other, but the **show** command just leaves out the word *network*. For instance:

Configuration: **network 10.1.13.3 0.0.0.0 area 0**

**show** Command: **10.1.13.3 0.0.0.0 area 0**

IOS interprets the **network** commands to choose interfaces on which to run OSPF, so it could be that IOS chooses a different set of interfaces than you predicted. To check the list of interfaces chosen by IOS, use the **show ip ospf interface brief** command, which lists all interfaces that have been enabled for OSPF processing. Verifying the interfaces can be a useful step if you have issues with OSPF neighbors because OSPF must first be enabled on an interface before a router will attempt to discover neighbors on that interface. Example 22-8 shows a sample from Router R1.

**Example 22-8** *Router R1* **show ip ospf interface brief** *Command*

```
R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/0/0      1     0               10.1.1.1/24        1     DR    0/0
Gi0/0/1      1     0               10.1.12.1/24       1     BDR   1/1
Gi0/0/2      1     0               10.1.13.1/24       1     BDR   1/1
Gi0/0/3      1     0               10.1.14.1/24       1     DR    1/1
```

The **show ip ospf interface brief** command lists one line per interface, showing all the interfaces on which OSPF has been enabled. Each line identifies the OSPF process ID (per the **router ospf** *process-id* command), the area, the interface IP address, and the number of neighbors found via each interface.

You may use the command in Example 22-8 quite often, but the **show ip ospf interface** command (without the **brief** keyword) gives much more detail about OSPF per-interface settings. Example 23-4 in Chapter 23, "Implementing Optional OSPF Features," shows an example of the entire output of that command.

## Configuring the OSPF Router ID

While OSPF has many other optional features, most enterprise networks that use OSPF choose to configure each router's OSPF router ID. OSPF-speaking routers must have a router ID (RID) for proper operation. By default, routers will choose an interface IP address to use as the RID. However, many network engineers prefer to choose each router's router ID, so command output from commands like **show ip ospf neighbor** lists more recognizable router IDs.

To choose its RID, a Cisco router uses the following process when the router reloads and brings up the OSPF process. Note that the router stops looking for a router ID to use once one of the steps identifies a value to use.



1. If the **router-id** *rid* OSPF subcommand is configured, this value is used as the RID.
2. If any loopback interfaces have an IP address configured, and the interface has an interface status of up, the router picks the highest numeric IP address among these loopback interfaces.
3. The router picks the highest numeric IP address from all other interfaces whose interface status code (first status code) is up. (In other words, an interface in up/down state will be included by OSPF when choosing its router ID.)

The first and third criteria should make some sense right away: the RID is either configured or is taken from a working interface's IP address. However, this book has not yet explained the concept of a *loopback interface*, as mentioned in Step 2.

A loopback interface is a virtual interface that can be configured with the **interface loopback** *interface-number* command, where *interface-number* is an integer. Loopback interfaces are always in an "up and up" state unless administratively placed in a shutdown state. For example, a simple configuration of the command **interface loopback 0**, followed by **ip address 2.2.2.2 255.255.255.0**, would create a loopback interface and assign it an IP address. Because loopback interfaces do not rely on any hardware, these interfaces can be up/up whenever IOS is running, making them good interfaces on which to base an OSPF RID.

Example 22-9 shows the configuration that existed in Routers R1 and R2 before the creation of the **show** command output earlier in this chapter. R1 set its router ID using the direct method, while R2 used a loopback IP address. Example 22-10 that follows shows the output of the **show ip ospf** command on R1, which identifies the OSPF RID used by R1.

**Example 22-9** *OSPF Router ID Configuration Examples*

```
! R1 Configuration first
router ospf 1
 router-id 1.1.1.1
 network 10.1.0.0 0.0.255.255 area 0
```

```
! R2 Configuration next
!
interface Loopback2
 ip address 2.2.2.2 255.255.255.255
```

**Example 22-10** *Confirming the Current OSPF Router ID*

```
R1# show ip ospf
 Routing Process "ospf 1" with ID 1.1.1.1
! lines omitted for brevity
```

Routers need a stable OSPF RID because any change to the OSPF RID causes a router to close existing neighbor relationships and remove all routes learned through those neighbors. To keep the RID stable, a router chooses its RID when the router first initializes (at power-on or per the **reload** command). So the RID might change at the next reload when the router re-evaluates the RID choice rules based on the current conditions.

However, routers do support one scenario to update their RID without a **reload**, which can be useful for testing in lab. To do so, configure the OSPF **router-id** OSPF subcommand followed by the **clear ip ospf process** EXEC command.

## Implementing Multiarea OSPF

Even though the current CCNA 200-301 V1.1 exam blueprint mentions single area but not multiarea OSPF, you only need to learn one more idea to know how to configure multiarea OSPF. So, this chapter takes a brief page to show how.

For example, consider a multiarea OSPF design as shown in Figure 22-4. It uses the same routers and IP addresses as shown earlier in Figure 22-2, on which all the examples in this chapter have been based so far. However, the design shows three areas instead of the single-area design shown in Figure 22-2.

Configuring the routers in a multiarea design is almost just like configuring OSPFv2 for a single area. To configure multiarea OSPF, all you need is a valid OSPF area design (for instance, like Figure 22-4) and a configuration that places each router interface into the correct area per that design. For example, both of R4's interfaces connect to links in area 4, making R4 an internal router, so any **network** commands on Router R4 will list area 4.

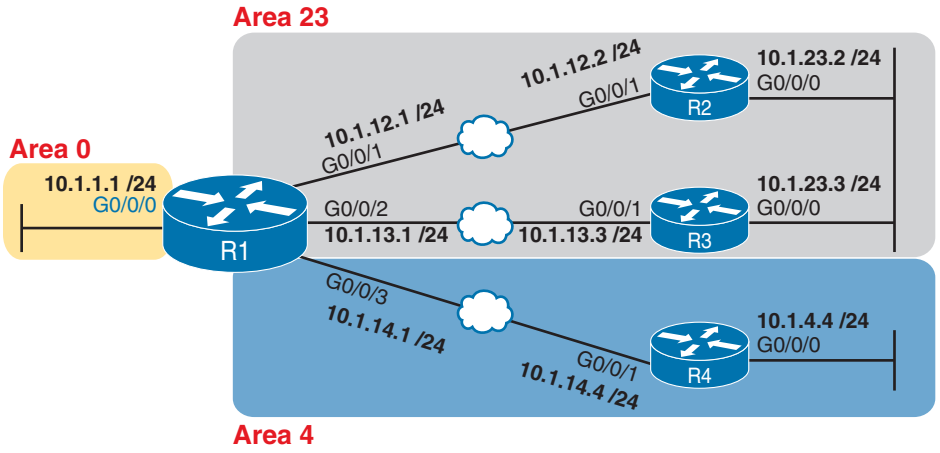



**Figure 22-4** *Area Design for an Example Multiarea OSPF Configuration*

Example 22-11 shows a sample configuration for Router R1. To make the configuration clear, it uses **network** commands with a wildcard mask of 0.0.0.0, meaning each **network** command matches a single interface. Each interface will be placed into either area 0, 23, or 4 to match the figure.

**Example 22-11** *OSPF Configuration on R1, Placing Interfaces into Different Areas*

```
router ospf 1
 network 10.1.1.1 0.0.0.0 area 0
 network 10.1.12.1 0.0.0.0 area 23
 network 10.1.13.1 0.0.0.0 area 23
 network 10.1.14.1 0.0.0.0 area 4
```

# Implementing OSPFv2 Using Interface Subcommands

From the earliest days of OSPFv2 support in Cisco routers, the configuration used the OSPF **network** command as discussed in this chapter. However, that configuration style can be confusing, and it does require some interpretation. As a result, Cisco added another option for OSPFv2 configuration called OSPF interface configuration.

The newer interface-style OSPF configuration still enables OSPF on interfaces, but it does so directly with the **ip ospf** interface subcommand. Instead of matching interfaces with indirect logic using **network** commands, you directly enable OSPFv2 on interfaces by configuring an interface subcommand on each interface.

## OSPF Interface Configuration Example

To show how OSPF interface configuration works, this example basically repeats the example shown earlier in the chapter using the traditional OSPFv2 configuration with **network** commands. So, before looking at the OSPFv2 interface configuration, take a moment to look back to review traditional OSPFv2 configuration with Figure 22-2 and Examples 22-2 and 22-3.

After reviewing the traditional configuration, consider this checklist, which details how to convert from the old-style configuration in Example 22-2 and Example 22-3 to use interface configuration:

Config Checklist

**Step 1.** Use the **no network** *network-id* **area** *area-id* subcommands in OSPF configuration mode to remove the **network** commands.

**Step 2.** Add one **ip ospf** *process-id* **area** *area-id* command in interface configuration mode under each interface on which OSPF should operate, with the correct OSPF process (*process-id*) and the correct OSPF area number.

Figure 22-5 repeats the design for both the original examples in this chapter and for this upcoming interface configuration example.

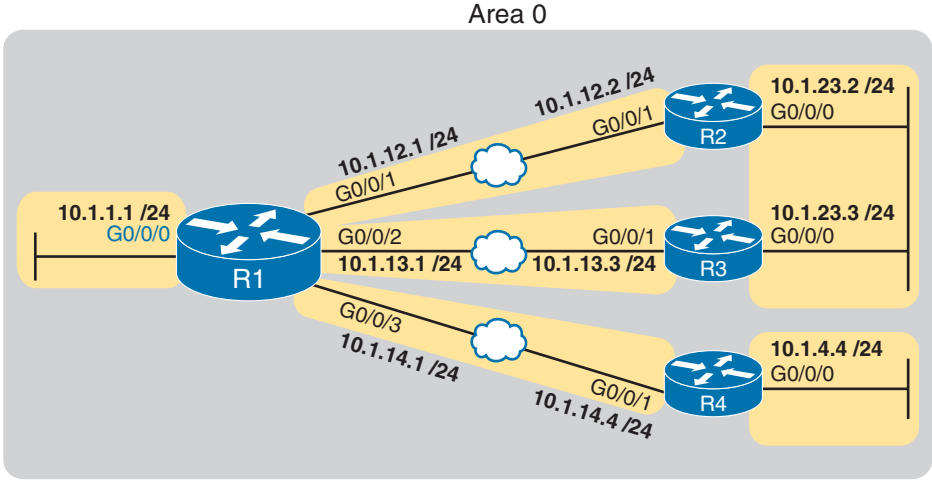



**Figure 22-5** *Area Design Used in the Upcoming OSPF Interface Config Example*

Example 22-2 shows a single **network** command: **network 10.0.0.0 0.255.255.255 area 0**. Example 22-12 follows the steps in the migration checklist, beginning with the removal of the previous configuration using the **no network 10.0.0.0 0.255.255.255 area 0** command. The example then shows the addition of the **ip ospf 1 area 0** command on each of the interfaces on Router R1, enabling OSPF process 1 on the interface and placing each interface into area 0.

**Example 22-12** *Migrating to Use OSPF Interface Subcommand Configuration*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# router ospf 1
R1(config-router)# no network 10.0.0.0 0.255.255.255 area 0
R1(config-router)#
*Apr  8 19:35:24.994: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/0/1
from FULL to DOWN, Neighbor Down: Interface down or detached
*Apr  8 19:35:24.994: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/0/2
from FULL to DOWN, Neighbor Down: Interface down or detached
```

```
*Apr  8 19:35:24.994: %OSPF-5-ADJCHG: Process 1, Nbr 4.4.4.4 on GigabitEthernet0/0/3
from FULL to DOWN, Neighbor Down: Interface down or detached
R1(config-router)# interface g0/0/0
R1(config-if)# ip ospf 1 area 0
R1(config-if)# interface g0/0/1
R1(config-if)# ip ospf 1 area 0
R1(config-if)#
*Apr  8 19:35:52.970: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/0/1
from LOADING to FULL, Loading Done
R1(config-if)# interface g0/0/2
R1(config-if)# ip ospf 1 area 0
R1(config-if)#
*Apr  8 19:36:13.362: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/0/2
from LOADING to FULL, Loading Done
R1(config-if)# interface g0/0/3
R1(config-if)# ip ospf 1 area 0
R1(config-if)#
*Apr  8 19:37:05.398: %OSPF-5-ADJCHG: Process 1, Nbr 4.4.4.4 on GigabitEthernet0/0/3
from LOADING to FULL, Loading Done
R1(config-if)#
```

When reading the example, read from top to bottom, and also consider the details about the failed and recovered neighbor relationships shown in the log messages. Removing the network command disabled OSPF on all interfaces on Router R1, causing all three neighbor relationships to fail. The example then shows the addition of the **ip ospf 1 area 0** command on the LAN interface, which enables OSPF but does not cause a neighbor relationship to form, because no other OSPF routers exist in that subnet. Then the example shows the same command added to each of the WAN links in succession, and in each case, the OSPF neighbor available over that WAN link comes up (as noted in the log messages).

**NOTE** A router's configuration can include both a **network** router subcommand and an **ip ospf** interface subcommand that enable OSPF on the same interface. If those commands refer to different area numbers, IOS uses the area number from the **ip ospf** interface subcommand. Additionally, multiple **network** commands can match the same interface. In that case, IOS uses the order in which the commands appear in OSPF configuration mode.

## Verifying OSPF Interface Configuration

OSPF operates the same way whether you use the new style or old style of configuration. The OSPF area design works the same, neighbor relationships form the same way, routers negotiate to become the DR and BDR the same way, and so on. However, you can see a few small differences in **show** command output when using the newer OSPFv2 configuration if you look closely.

The **show ip protocols** command relists most of the routing protocol configuration, so it does list some different details if you use interface configuration versus the **network** command. With the **ip ospf** interface subcommands, the output lists the phrase "Interfaces Configured Explicitly," as highlighted in Example 22-13. The example first shows the relevant parts of the

**show ip protocols** command when using interface configuration on Router R1, and then lists the same portions of the command from when R1 used **network** commands.

**Example 22-13** *Differences in* **show ip protocols** *Output: Old- and New-Style OSPFv2 Configuration*

```
! First, with the new interface configuration
R1# show ip protocols
! … beginning lines omitted for brevity
  Routing for Networks:
  Routing on Interfaces Configured Explicitly (Area 0):
    GigabitEthernet0/0/0
    GigabitEthernet0/0/1
    GigabitEthernet0/0/2
    GigabitEthernet0/0/3
  Routing Information Sources:
    Gateway         Distance      Last Update
    4.4.4.4              110      00:09:30
    2.2.2.2              110      00:10:49
    3.3.3.3              110      05:20:07
  Distance: (default is 110)
```

```
! For comparison, the old results with the use of the OSPF network command
R1# show ip protocols
! … beginning lines omitted for brevity
  Routing for Networks:
    10.1.0.0 0.0.255.255 area 0
! … ending line omitted for brevity
```

Another small piece of different output exists in the **show ip ospf interface** [*interface*] command. The command lists details about OSPF settings for the interface(s) on which OSPF is enabled. The output also makes a subtle reference to whether that interface was enabled for OSPF with the old or new configuration style. Example 22-14 also begins with output based on interface configuration on Router R1, followed by the output that would exist if R1 still used the old-style **network** command.

Key Topic

**Example 22-14** *Differences in* **show ip ospf interface** *Output with OSPFv2 Interface Configuration*

```
! First, with the new interface configuration
R1# show ip ospf interface g0/0/1
GigabitEthernet0/0/0 is up, line protocol is up
  Internet Address 10.1.12.1/24, Area 0, Attached via Interface Enable
! Lines omitted for brevity
```

```
! For comparison, the old results with the use of the OSPF network command
R1# show ip ospf interface g0/0/1
GigabitEthernet0/0/0 is up, line protocol is up
  Internet Address 10.1.12.1/24, Area 0, Attached via Network Statement
! … ending line omitted for brevity
```

Other than these small differences in a few **show** commands, the rest of the commands show nothing different depending on the style of configuration. For instance, the **show ip ospf interface brief** command does not change depending on the configuration style, nor do the **show ip ospf database**, **show ip ospf neighbor**, or **show ip route** commands.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 22-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 22-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review Config Checklists | | Book, website |
| Review command tables | | Book |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 22-4** Key Topics for Chapter 22

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 22-1 | Organization of OSPFv2 configuration with the **network** command | 565 |
| List | Example OSPF wildcard masks and their meaning | 567 |
| Figure 22-3 | OSPF verification commands | 569 |
| Example 22-4 | Example of the **show ip ospf neighbor** command | 569 |
| List | Neighbor states and their meanings | 570 |
| List | Rules for setting the router ID | 574 |
| Example 22-14 | Differences in **show ip ospf interface** output with OSPF interface configuration | 579 |

## Key Terms You Should Know

OSPF neighbor, OSPF router-id

## Command References

Tables 22-5 and 22-6 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 22-5** Chapter 22 Configuration Command Reference

| Command | Description |
|---|---|
| **router ospf** *process-id* | Global command that enters OSPF configuration mode for the listed process |
| **network** *ip-address wildcard-mask* **area** *area-id* | Router subcommand that enables OSPF on interfaces matching the address/wildcard combination and sets the OSPF area |
| **ip ospf** *process-id* **area** *area-number* | Interface subcommand to enable OSPF on the interface and to assign the interface to a specific OSPF area |
| **ip ospf cost** *interface-cost* | Interface subcommand that sets the OSPF cost associated with the interface |
| **bandwidth** *bandwidth* | Interface subcommand that directly sets the interface bandwidth (Kbps) |
| **auto-cost reference-bandwidth** *number* | Router subcommand that tells OSPF the numerator in the Reference_bandwidth/Interface_bandwidth formula used to calculate the OSPF cost based on the interface bandwidth |
| **router-id** *id* | OSPF command that statically sets the router ID |
| **interface loopback** *number* | Global command to create a loopback interface and to navigate to interface configuration mode for that interface |

**Table 22-6** Chapter 22 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip ospf** | Lists information about the OSPF process running on the router, including the OSPF router ID, areas to which the router connects, and the number of interfaces in each area. |
| **show ip ospf interface brief** | Lists the interfaces on which the OSPF protocol is enabled (based on the **network** commands), including passive interfaces. |
| **show ip ospf interface** [*type number*] | Lists a long section of settings, status, and counters for OSPF operation on all interfaces, or on the listed interface, including the Hello and Dead Timers. |
| **show ip protocols** | Shows routing protocol parameters and current timer values. |
| **show ip ospf neighbor** [*type number*] | Lists brief output about neighbors, identified by neighbor router ID, including current state, with one line per neighbor; optionally, limits the output to neighbors on the listed interface. |
| **show ip ospf neighbor** *neighbor-ID* | Lists the same output as the **show ip ospf neighbor** detail command, but only for the listed neighbor (by neighbor RID). |

| Command | Description |
|---|---|
| **show ip ospf database** | Lists a summary of the LSAs in the database, with one line of output per LSA. It is organized by LSA type (first type 1, then type 2, and so on). |
| **show ip route** | Lists all IPv4 routes. |
| **show ip route ospf** | Lists routes in the routing table learned by OSPF. |
| **clear ip ospf process** | Resets the OSPF process, resetting all neighbor relationships and also causing the process to make a choice of OSPF RID. |

*This page intentionally left blank*



CHAPTER 23

# Implementing Optional OSPF Features

**This chapter covers the following exam topics:**

- **3.0 IP Connectivity**
  - **3.2 Determine how a router makes a forwarding decision by default**
    - **3.2.b Administrative distance**
    - **3.2.c Routing protocol metric**
  - **3.4 Configure and verify single area OSPFv2**
    - **3.4.a Neighbor adjacencies**
    - **3.4.b Point-to-point**
    - **3.4.c Broadcast (DR/BR selection)**
    - **3.4.d Router ID**

The previous chapter showed how to configure the core OSPF settings to make OSPF work. This chapter examines a variety of optional OSPF settings, chosen for two reasons. First, the CCNA exam topics mention or imply coverage of many of the optional features mentioned in this chapter. Second, the optional features listed here happen to be relatively popular in production networks.

The chapter begins with a section about OSPF network types. As a setting on each interface, the OSPF network type dictates whether the router attempts to dynamically discover neighbors, and once discovered, whether routers on the link use a designated router (DR) or not. The section also discusses how to influence which router wins the DR election using OSPF priority and router IDs (RIDs).

The final section then moves on to discuss a variety of smaller optional OSPF configuration topics. The features include topics such as how to use passive interfaces, how to change OSPF costs (which influences the routes OSPF chooses), and how to create a default route advertised by OSPF.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 23-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| OSPF Network Types | 1–4 |
| Additional Optional OSPFv2 Features | 5–6 |

1. Routers R1 and R2, with router IDs 1.1.1.1 and 2.2.2.2, connect over an Ethernet WAN link. If using all default OSPF settings, if the WAN link initializes for both routers at the same time, which of the following answers are true? (Choose two answers.)
   a. Router R1 will become the DR.
   b. Router R1 will dynamically discover the existence of Router R2.
   c. Router R2 will be neither the DR nor the BDR.
   d. Router R1's **show ip ospf neighbor** command will list R2 with a state of "FULL/DR."
2. Routers R1 and R2, with router IDs 1.1.1.1 and 2.2.2.2, connect over an Ethernet WAN link. The configuration uses all defaults, except giving R1 an interface priority of 11 and changing both routers to use OSPF network type point-to-point. If the WAN link initializes for both routers at the same time, which of the following answers are true? (Choose two answers.)
   a. Router R1 will become the DR.
   b. Router R1 will dynamically discover the existence of Router R2.
   c. Router R2 will be neither the DR nor the BDR.
   d. Router R2's **show ip ospf neighbor** command will list R1 with a state of "FULL/DR."
3. Per the command output, with how many routers is Router R9 fully adjacent over its Gi0/0 interface?

```
R9# show ip ospf interface brief
Interface     PID   Area        IP Address/Mask    Cost  State Nbrs F/C
Gi0/0         1     0           10.1.1.1/24        1     DROTH 2/5
```

   a. 7
   b. 0
   c. 5
   d. 2
4. Routers R1 and R2, which use default priority settings, become neighbors, with R1 as the DR and R2 as the BDR. The engineer then configures R2's interface to use OSPF priority 100. Which answers correctly predict any changes in the OSPF neighbor relationship?
   a. Router R2 will immediately become the DR.
   b. Router R2 will become the DR after the neighbor relationship fails.
   c. Router R2 will immediately stop filling the BDR role.
   d. Router R2 will become the DR after four OSPF Hello intervals.

5. Which of the following configuration settings on a router does not influence which IPv4 route a router chooses to add to its IPv4 routing table when using OSPFv2?
   a. **auto-cost reference-bandwidth**
   b. **delay**
   c. **bandwidth**
   d. **ip ospf cost**
6. A network engineer configures the **ip ospf hello-interval 15** subcommand under the interfaces that connect OSPF neighbors R1 and R2 but with no use of the **ip ospf dead-interval** subcommand. Eventually, Router R1's OSPF process fails, but the link between R1 and R2 remains working. How long after Router R1's last Hello does R2 consider its neighbor relationship with R1 to fail?
   a. 10 seconds
   b. 15 seconds
   c. 40 seconds
   d. 60 seconds

# Foundation Topics

## OSPF Network Types

Two CCNA 200-301 V1.1 exam topics might be completely misunderstood without taking a closer look at some default OSPF settings. In particular, the following exam topics refer to a specific per-interface OSPF setting called the *network type*—even listing the keywords used to configure the setting in the exam topics:

3.4.b: **point-to-point**

3.4.c: **broadcast** (DR/BDR selection)

OSPF includes a small number of network types as a setting on each OSPF-enabled interface. The setting tells the router whether or not to dynamically discover OSPF neighbors (versus requiring the static configuration of the neighboring router's IP address) and whether or not the router should attempt to use a designated router (DR) and backup designated router (BDR) in the subnet. Of the two OSPF network types included in the CCNA exam topics, both cause routers to dynamically discover neighbors, but one calls for the use of a DR, whereas the other does not. Table 23-2 summarizes the features of the two OSPF network types mentioned in the exam topics.

Key Topic

**Table 23-2** Two OSPF Network Types and Key Behaviors

| Network Type Keyword | Dynamically Discovers Neighbors | Uses a DR/BDR |
|---|---|---|
| broadcast | Yes | Yes |
| point-to-point | Yes | No |

The rest of this first major section of the chapter explores each type.

## The OSPF Broadcast Network Type

OSPF defaults to use a **broadcast network type** on all types of Ethernet interfaces. Note that all the Ethernet interfaces in the examples in Chapter 22, "Implementing Basic OSPF Features," relied on that default setting.

To see all the details of how the OSPF broadcast network type works, this chapter begins with a different design than the examples in Chapter 22, instead using a single-area design that connects four routers to the same subnet, as shown in Figure 23-1. All links reside in area 0, making the design a single-area design.

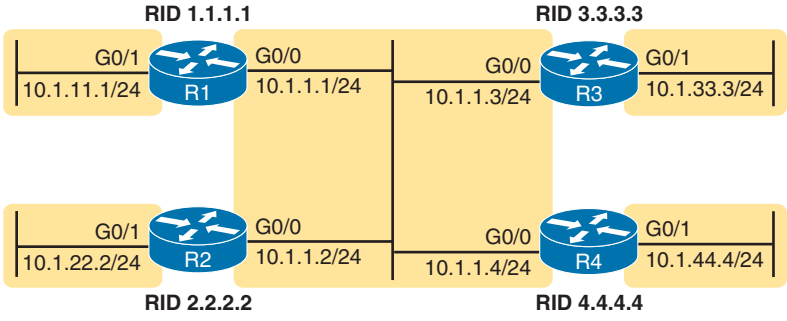



**Figure 23-1** *The Single-Area Design Used in This Chapter*

To get a sense for how OSPF operates with the broadcast network type, imagine that all four routers use a straightforward OSPF interface configuration like the Router R1 configuration shown in Example 23-1. Both GigabitEthernet interfaces on all four routers default to use network type broadcast. Note that the configuration on routers R2, R3, and R4 mirrors R1's configuration except that they use router IDs 2.2.2.2, 3.3.3.3, and 4.4.4.4, respectively, and they use the IP addresses shown in the figure.

**Example 23-1** *R1 OSPF Configuration to Match Figure 23-1*

```
router ospf 1
 router-id 1.1.1.1
!
interface gigabitEthernet0/0
 ip ospf 1 area 0
!
interface gigabitEthernet0/1
 ip ospf 1 area 0
```

This simple design gives us a great backdrop from which to observe the results of the broadcast network type on each router. Both interfaces (G0/0 and G0/1) on each router use the broadcast network type and perform the following actions:

- Attempt to discover neighbors by sending OSPF Hellos to the 224.0.0.5 multicast address (an address reserved for sending packets to all OSPF routers in the subnet)
- Attempt to elect a DR and BDR on each subnet
- On the interface with no other routers on the subnet (G0/1), become the DR

- On the interface with three other routers on the subnet (G0/0), be either DR, BDR, or a DROther router
- When sending OSPF messages to the DR or BDR, send the messages to the all-OSPF-DRs multicast address 224.0.0.6

Example 23-2 shows some of the results using the **show ip ospf neighbor** command. Note that R1 lists R2, R3, and R4 as neighbors (based on their 2.2.2.2, 3.3.3.3, and 4.4.4.4 router IDs), confirming that R1 dynamically discovered the other routers. Also, note that the output lists 4.4.4.4 as the DR and 3.3.3.3 as the BDR.

**Example 23-2** *R1's List of Neighbors*

```
R1# show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   2WAY/DROTHER    00:00:35    10.1.1.2        GigabitEthernet0/0
3.3.3.3           1   FULL/BDR        00:00:33    10.1.1.3        GigabitEthernet0/0
4.4.4.4           1   FULL/DR         00:00:35    10.1.1.4        GigabitEthernet0/0
```

## Verifying Operations with Network Type Broadcast

As discussed in the section "Using Designated Routers on Ethernet Links" in Chapter 21, "Understanding OSPF Concepts," all discovered routers on the link should become neighbors and at least reach the *2-way* state. For all neighbor relationships that include the DR and/or BDR, the neighbor relationship should further reach the *full* state. That section defined the term *fully adjacent* as a special term that refers to neighbors that reach this full state.

The design in Figure 23-1, with four routers on the same LAN, provides just enough routers so that one neighbor relationship will remain in a 2-way state and not reach the full state, as a perfectly normal way for OSPF to operate. Figure 23-2 shows the current conditions when the **show** commands in this chapter were gathered, with R4 as the DR, R3 as the BDR, and with R1 and R2 as DROther routers.

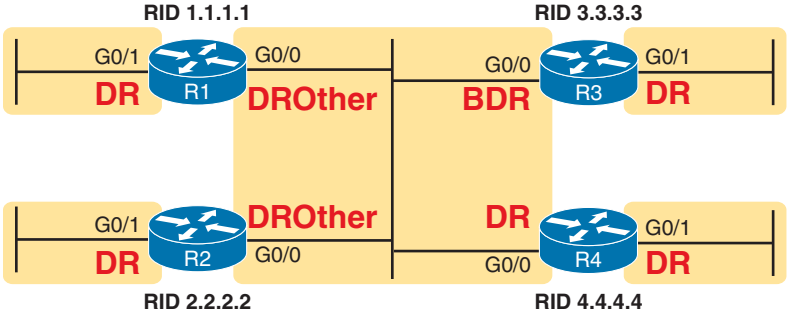



**Figure 23-2** *OSPF DR, BDR, and DROther Roles in the Network*

---

Answers to the "Do I Know This Already?" quiz:

**1** B, D **2** B, C **3** D **4** B **5** B **6** D

Now consider Router R1's neighbors as listed in Example 23-2. R1 has three neighbors, all reachable out its G0/0 interface. However, R1's **show ip ospf neighbor** command refers to the state of R1's relationship with the neighbor: 2-way with router 2.2.2.2. Because both R1 and R2 currently serve as DROther routers—that is, they wait ready to become the BDR if either the DR or BDR fails—their neighbor relationship remains in a 2-way state.

Examining Example 23-2 one last time, R1, as a DROther router itself, has two neighbor relationships that reach a full state: R1's neighbor adjacency with DR R4 and R1's neighbor adjacency with BDR R3. But R1 has a total of three neighbors, all reachable off R1's G0/0 interface.

The next example emphasizes that R1 has three neighbors off its G0/0 interface, with only two as fully adjacent. The far right of the **show ip ospf interface brief** command output in Example 23-3 shows "2/3." meaning two fully adjacent neighbors and three total neighbors on that interface. Also, note that this command's "State" column differs from the **show ip ospf neighbor** commands, because it lists the local router's role on the interface, with R1's G0/1 acting as DR and R1's G0/0 acting as a DROther router.

Key Topic

**Example 23-3** *Router R1 OSPF Interfaces: Local Role and Neighbor Counts*

```
R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/1        1     0               10.1.11.1/24       1     DR    0/0
Gi0/0        1     0               10.1.1.1/24        1     DROTH 2/3
```

So far, this topic has described the effect of the OSPF broadcast network type by taking advantage of the default setting on Ethernet interfaces. To see the setting, use the **show ip ospf interface** command, as shown in Example 23-4. The first highlighted item identifies the network type. However, this command's output restates many of the facts seen in both the **show ip ospf neighbor** and **show ip ospf interface brief** commands in Examples 23-2 and 23-3, so take the time to browse through all of Example 23-4 and focus on the additional highlights to see those familiar items.

**Example 23-4** *Displaying OSPF Network Type Broadcast*

```
R1# show ip ospf interface g0/0
GigabitEthernet0/0 is up, line protocol is up
  Internet Address 10.1.1.1/24, Area 0, Attached via Interface Enable
  Process ID 1, Router ID 1.1.1.1, Network Type BROADCAST, Cost: 1
  Topology-MTID    Cost    Disabled    Shutdown      Topology Name
        0           1         no          no            Base
  Enabled by interface config, including secondary ip addresses
  Transmit Delay is 1 sec, State DROTHER, Priority 1
  Designated Router (ID) 4.4.4.4, Interface address 10.1.1.4
  Backup Designated router (ID) 3.3.3.3, Interface address 10.1.1.3
  Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
    oob-resync timeout 40
    Hello due in 00:00:00
  Supports Link-local Signaling (LLS)
```

```
Cisco NSF helper support enabled
IETF NSF helper support enabled
Index 1/1/1, flood queue length 0
Next 0x0(0)/0x0(0)/0x0(0)
Last flood scan length is 0, maximum is 1
Last flood scan time is 0 msec, maximum is 0 msec
Neighbor Count is 3, Adjacent neighbor count is 2
  Adjacent with neighbor 3.3.3.3  (Backup Designated Router)
  Adjacent with neighbor 4.4.4.4  (Designated Router)
Suppress hello for 0 neighbor(s)
```

Although you would not need to configure an Ethernet interface to use the broadcast network type, for reference, IOS defaults to that setting on Ethernet interfaces per default command **ip ospf network broadcast**.

### Using Priority and RID to Influence the DR/BDR Election

In some cases, you might want to influence the OSPF DR election. However, before deciding that makes sense in every case, note that OSPF DR/BDR election rules will not result in a specific router always being the DR, and another always being the BDR, assuming that each is up and working. In short, here are the rules once a DR and BDR have been elected:

- If the DR fails, the BDR becomes the DR, and a new BDR is elected.
- When a better router enters the subnet, no preemption of the existing DR or BDR occurs.

As a result of these rules, while you can configure a router to be the best (highest priority) router to become the DR in an election, doing so only increases that router's statistical chances of being the DR at a given point in time. If the router with the highest priority fails, other routers will become DR and BDR, and the best router will not be DR again until the current DR and BDR fail, causing new elections.

However, in some cases, you may want to influence the DR/BDR election. To do so, use these settings, listed here in order of precedence:




- **The highest OSPF interface priority:** The highest value wins during an election, with values ranging from 0 to 255. (A value of 0 prevents the router from ever becoming the DR.)
- **The highest OSPF Router ID:** If the priority ties, the election chooses the router with the highest OSPF RID.

For example, imagine all four routers in the design shown in Figure 23-1 trying to elect the DR and BDR at the same time—for instance, after a power hit in which all four routers power off and back on again. No prior DR or BDR exists at this point. They all participate in the election. They all tie with default priority values of 1 (see Example 23-4 for R1's priority

in the **show ip ospf interface** command output). In this case, R4 becomes the DR based on the numerically highest RID of 4.4.4.4, and R3 becomes the BDR based on the next highest RID of 3.3.3.3.

To influence the election, you could set the various RIDs with your preferred router with the highest RID value. However, many networks choose OSPF router IDs to help identify the router easily rather than choosing to make one value higher than its neighbor. Instead, using the **OSPF priority** setting makes better sense. For instance, if an engineer preferred that R1 be the DR, the engineer could add the configuration in Example 23-5 to set R1's interface priority to 99.

**Example 23-5** *Influencing DR/BDR Election Using OSPF Priority*

```
R1# configure terminal
Configuring from terminal, memory, or network [terminal]?
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# interface g0/0
R1(config-if)# ip ospf priority 99
R1(config-if)# ^Z
R1#
R1# show ip ospf interface g0/0 | include Priority
  Transmit Delay is 1 sec, State DROTHER, Priority 99

R1# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   2WAY/DROTHER    00:00:36    10.1.1.2        GigabitEthernet0/0
3.3.3.3           1   FULL/BDR        00:00:30    10.1.1.3        GigabitEthernet0/0
4.4.4.4           1   FULL/DR         00:00:37    10.1.1.4        GigabitEthernet0/0

R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/1        1     0               10.1.11.1/24       1     DR    0/0
Gi0/0        1     0               10.1.1.1/24        1     DROTH 2/3
```

The top of the example shows R1's interface priority value now as 99, and the **show ip ospf interface G0/0** command that follows confirms the setting. However, the last two commands confirm that OSPF does not preempt the existing DR or BDR. Note that the **show ip ospf neighbor** command still lists R4's state as DR, meaning R4 still acts as the DR, so R1, with a higher priority, did not take over. The final command, **show ip ospf interface brief**, lists R1's State (role) as DROTH.

Just to complete the process and show R1 winning a DR election, Example 23-6 shows the results after forcing a free election by failing the LAN switch that sits between the four routers. As expected, R1 wins and becomes DR due to its higher priority, with the other three routers tying based on priority. R4 wins between R2, R3, and R4 due to its higher RID to become the BDR.

**Example 23-6** *Results of a Completely New DR/BDR Election*

```
! Not shown: LAN fails, and then recovers, causing a new OSPF Election
R1# show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   FULL/DROTHER    00:00:37    10.1.1.2        GigabitEthernet0/0
3.3.3.3           1   FULL/DROTHER    00:00:38    10.1.1.3        GigabitEthernet0/0
4.4.4.4           1   FULL/BDR        00:00:38    10.1.1.4        GigabitEthernet0/0

R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/1        1     0               10.1.11.1/24       1     DR    0/0
Gi0/0        1     0               10.1.1.1/24        1     DR    3/3
```

**NOTE** If you have begun to mentally compare OSPF DR elections to STP elections, keep some key differences in mind. First, STP uses lowest-is-best logic, whereas OSPF uses highest-is-best. STP elections allow preemption—for instance, if a new switch appears with a superior (lower) bridge ID (BID) than the current root, the new switch becomes the root switch. OSPF does not preempt, so a new router on a link, with the highest priority, does not take over as DR or BDR. Instead, it wins future elections, eventually becoming the DR.

## The OSPF Point-to-Point Network Type

The other OSPF network type mentioned in the CCNA 200-301 V1.1 blueprint, point-to-point, works well for data links that by their nature have just two routers on the link. For example, consider the topology in Figure 23-3, which shows Router R1 with three WAN links—two Ethernet WAN links and one serial link.

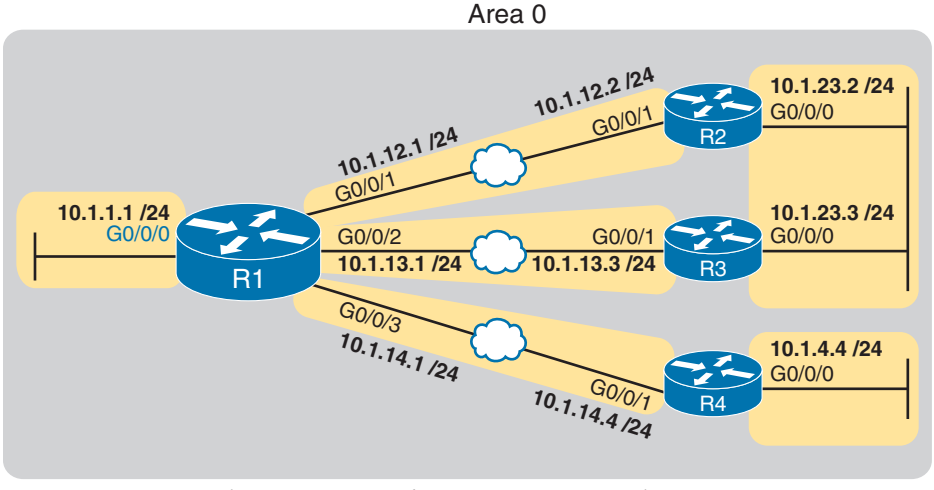



**Figure 23-3** *Sample OSPF Design for Upcoming Examples*

First, consider the older-style serial links (not used in the figure). Serial links between routers do not support the capability to add a third router to the link. With only two devices on the link, using a DR/BDR provides no advantage; instead, it adds a little extra convergence time. Using a network type of point-to-point tells the router to not use a DR/BDR on the link, which makes sense on serial links. In fact, IOS defaults to a setting of **ip ospf network point-to-point** on serial interfaces.

While you might not see many serial links in networks today, some other point-to-point WAN topologies exist, including some Ethernet WAN links. All the Ethernet WAN links used in this book happen to use a point-to-point Ethernet WAN service called an Ethernet Private Wire Service or simply an Ethernet Line (E-Line). For that service, the service provider will send Ethernet frames between two devices (routers) connected to the service, but only those two devices. In other words, an E-Line is a point-to-point service in concept. So, like serial links, Ethernet WAN links with only two routers connected gain no advantage by using a DR/BDR. As a result, many engineers prefer to instead use an OSPF **point-to-point network type** on Ethernet WAN links that in effect act as a point-to-point link.

Example 23-7 shows the configuration of Router R1's G0/0/1 interface in Figure 23-3 to use OSPF network type point-to-point. R2, on the other end of the WAN link, would need the same configuration command on its matching interface.

**Example 23-7** *OSPF Network Type Point-to-Point on an Ethernet WAN Interface on R1*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# interface g0/0/1
R1(config-if)# ip ospf network point-to-point
R1(config-if)#

R1# show ip ospf interface g0/0/1
GigabitEthernet0/0/1 is up, line protocol is up
  Internet Address 10.1.12.1/24, Area 0, Attached via Interface Enable
  Process ID 1, Router ID 1.1.1.1, Network Type POINT_TO_POINT, Cost: 1
  Topology-MTID    Cost    Disabled    Shutdown      Topology Name
        0           4         no          no            Base
  Enabled by interface config, including secondary ip addresses
  Transmit Delay is 1 sec, State POINT_TO_POINT
  Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
    oob-resync timeout 40
    Hello due in 00:00:01
  Supports Link-local Signaling (LLS)
  Cisco NSF helper support enabled
  IETF NSF helper support enabled
  Index 1/3/3, flood queue length 0
  Next 0x0(0)/0x0(0)/0x0(0)
  Last flood scan length is 1, maximum is 3
  Last flood scan time is 0 msec, maximum is 0 msec
  Neighbor Count is 1, Adjacent neighbor count is 1
    Adjacent with neighbor 2.2.2.2
  Suppress hello for 0 neighbor(s)
```

Note the highlighted portions of the **show** command in Example 23-7. The first two highlights note the network type. The final highlight with two lines notes that R1 has one neighbor on the interface, a neighbor with which it has become fully adjacent per the output.

Example 23-8 closes this section with a confirmation of some of those facts with two more commands. Note that the **show ip ospf neighbor** command on R1 lists Router R2 (RID 2.2.2.2) with a full state, but with no DR or BDR designation, instead listing a -. The - acts as a reminder that the link does not use a DR/BDR. The second command, **show ip ospf interface brief**, shows the state (the local router's role) as P2P, which is short for point-to-point, with a counter of 1 for the number of fully adjacent neighbors and total number of neighbors.

Key Topic

**Example 23-8** *OSPF Network Type Point-to-Point on an Ethernet WAN Interface on R1*

```
R1# show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           0   FULL/  -        00:00:39    10.1.12.2       GigabitEthernet0/0/1
! lines omitted for brevity

R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/0/1      1     0               10.1.12.1/24       4     P2P   1/1
! lines omitted for brevity
```

When using Ethernet WAN links that behave as a point-to-point link, consider using OSPF network type point-to-point rather than using the default broadcast type.

# Additional Optional OSPFv2 Features

This final major section of the chapter discusses some popular but optional OSPFv2 configuration features, as listed here in their order of appearance:

- Passive interfaces
- Default routes
- Metrics
- Hello and Dead intervals

## OSPF Passive Interfaces

Once OSPF has been enabled on an interface, the router tries to discover neighboring OSPF routers and form a neighbor relationship. To do so, the router sends OSPF Hello messages on a regular time interval (called the Hello interval). The router also listens for incoming Hello messages from potential neighbors.

Sometimes, a router does not need to form neighbor relationships with neighbors on an interface. Often, no other routers exist on a particular link, so the router has no need to keep

sending those repetitive OSPF Hello messages. In such cases, an engineer can make the interface passive, which means

Key Topic

- OSPF continues to advertise about the subnet that is connected to the interface.
- OSPF no longer sends OSPF Hellos on the interface.
- OSPF no longer processes any received Hellos on the interface.

The result of enabling OSPF on an interface but then making it passive is that OSPF still advertises about the connected subnet, but OSPF also does not form neighbor relationships over the interface.

To configure an interface as passive, two options exist. First, you can add the following command to the configuration of the OSPF process, in router configuration mode:

**passive-interface** *type number*

Alternately, the configuration can change the default setting so that all interfaces are passive by default and then add a **no passive-interface** command for all interfaces that need to not be passive:

**passive-interface default**

**no passive-interface** *type number*

For example, in the sample internetwork in Figure 23-4, Router R1 connects to a LAN with its G0/0/0 interface. With no other OSPF routers on that LAN, R1 will never discover an OSPF neighbor on that interface. Example 23-9 shows two alternative configurations to make R1's G0/0/0 passive to OSPF.

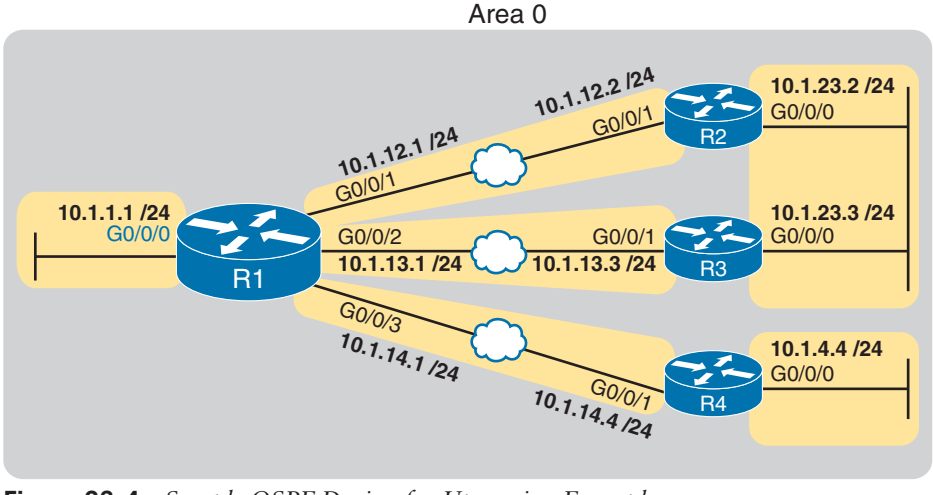



**Figure 23-4** *Sample OSPF Design for Upcoming Examples*

**Example 23-9** *Configuring Passive Interfaces on R1 from Figure 23-4*

```
! First, make each interface passive directly
router ospf 1
 passive-interface GigabitEthernet0/0/0

! Or, change the default to passive, and make the other interfaces not be passive
router ospf 1
 passive-interface default
 no passive-interface GigabitEthernet0/0/1
 no passive-interface GigabitEthernet0/0/2
 no passive-interface GigabitEthernet0/0/3
```

In real internetworks, use the configuration style that requires the least number of commands. For example, a router with 20 interfaces, 18 of which are passive to OSPF, has far fewer configuration commands when using the **passive-interface default** command to change the default to passive. If only two of those 20 interfaces need to be passive, use the default setting, in which all interfaces are not passive, to keep the configuration shorter.

Interestingly, OSPF makes it a bit of a challenge to use **show** commands to find out whether an interface is passive. The **show running-config** command lists the configuration directly, but if you cannot get into enable mode to use this command, note these two facts:

The **show ip ospf interface brief** command lists all interfaces on which OSPF is enabled, including **passive interfaces**.

The **show ip ospf interface** command lists several output lines per interface with a single line that mentions that the interface is passive.

Example 23-10 shows these commands on Router R1, based on the configuration shown in the top of Example 23-9. Note that passive interface G0/0/0 appears in the output of **show ip ospf interface brief**.

**Example 23-10** *Displaying Passive Interfaces*

```
R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/0/0      1     0               10.1.1.1/24        1     DR    0/0
Gi0/0/1      1     0               10.1.12.1/24       1     BDR   1/1
Gi0/0/2      1     0               10.1.13.1/24       1     BDR   1/1
Gi0/0/3      1     0               10.1.14.1/24       1     DR    1/1

R1# show ip ospf interface g0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  Internet Address 10.1.1.1/24, Area 0, Attached via Network Statement
  Process ID 1, Router ID 1.1.1.1, Network Type BROADCAST, Cost: 1
  Topology-MTID    Cost    Disabled    Shutdown      Topology Name
        0           1         no          no            Base
```

```
  Transmit Delay is 1 sec, State DR, Priority 1
  Designated Router (ID) 1.1.1.1, Interface address 10.1.1.1
  No backup designated router on this network
  Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
    oob-resync timeout 40
    No Hellos (Passive interface)
! Lines omitted for brevity
```

## OSPF Default Routes

Chapter 17, "Configuring IPv4 Addresses and Static Routes," showed some of the uses and benefits of default routes, with examples of static default routes. For those exact same reasons, networks can use OSPF to advertise default routes.

The most classic case for using a routing protocol to advertise a default route has to do with an enterprise's connection to the Internet. As a strategy, the enterprise engineer uses these design goals:

- All routers learn specific (nondefault) routes for subnets inside the company; a default route is not needed when forwarding packets to these destinations.
- One router connects to the Internet, and it has a default route that points toward the Internet.
- All routers should dynamically learn a default route, used for all traffic going to the Internet, so that all packets destined to locations in the Internet go to the one router connected to the Internet.

Figure 23-5 shows the idea of how OSPF advertises the default route, with the specific OSPF configuration. In this case, a company connects to an ISP with its Router R1. That router has a static default route (destination 0.0.0.0, mask 0.0.0.0) with a next-hop address of the ISP router. Then the use of the OSPF **default-information originate** command (Step 2) makes the router advertise a default route using OSPF to the remote routers (B1 and B2).

Key Topic

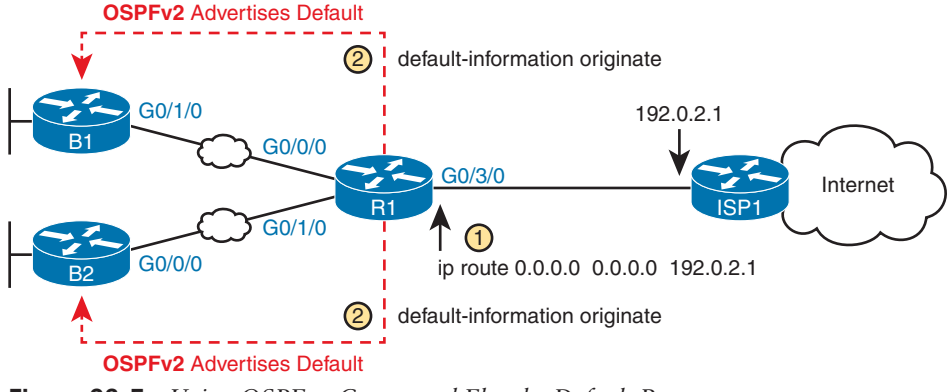



**Figure 23-5** *Using OSPF to Create and Flood a Default Route*

Figure 23-6 shows the default routes that result from OSPF's advertisements in Figure 23-5. On the far left, the branch routers all have OSPF-learned default routes, pointing to R1. R1 itself also needs a default route, pointing to the ISP router, so that R1 can forward all Internet-bound traffic to the ISP.

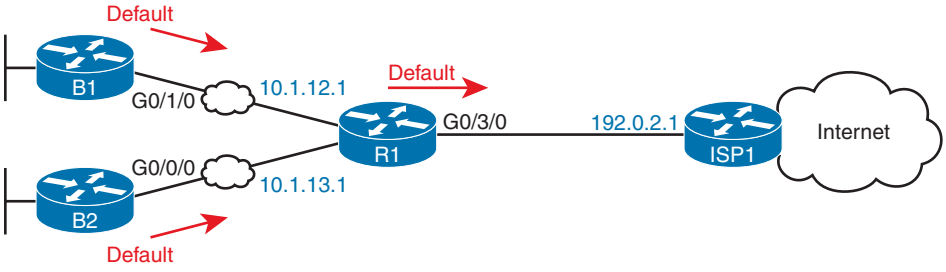



**Figure 23-6** *Default Routes Resulting from the* **default-information originate** *Command*

Example 23-11 first highlights the two commands relevant to the default route on Router R1 in Figure 23-6. The router has a default static route referring to the ISP router as the next-hop router (192.0.2.1). It also has a **default-information originate** command configured under the OSPF process, telling the router to advertise a default route to other OSPF routers—but only if the router currently has a default route.

**Example 23-11** *Relevant OSPF Configuration on Internet Edge Router R1*

```
! Excerpt from a show running-config command on router R1
ip route 0.0.0.0 0.0.0.0 192.0.2.1
!
router ospf 1
  network 10.0.0.0 0.255.255.255 area 0
  router-id 1.1.1.1
  default-information originate
```

Example 23-12 shows the status based on the configuration shown in Example 23-11. First, the top of the example confirms that R1 has a default route—that is, a route to 0.0.0.0/0. The "Gateway of last resort," which refers to the default route currently used by the router, points to next-hop IP address 192.0.2.1, which is the ISP router's IP address.

**Example 23-12** *Default Routes on Routers R1 and B1*

```
! The next command is from Router R1. Note the static code for the default route
R1# show ip route static
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
! Rest of the legend omitted for brevity

Gateway of last resort is 192.0.2.1 to network 0.0.0.0

S*      0.0.0.0/0 [254/0] via 192.0.2.1
```

```
! The next command is from router B1; notice the External route code for the default
```

```
B1# show ip route ospf
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
! Rest of the legend omitted for brevity

Gateway of last resort is 10.1.12.1 to network 0.0.0.0

O*E2    0.0.0.0/0 [110/1] via 10.1.12.1, 00:20:51, GigabitEthernet0/1/0
       10.0.0.0/8 is variably subnetted, 6 subnets, 2 masks
O         10.1.3.0/24 [110/3] via 10.1.12.1, 00:20:51, GigabitEthernet0/1/0
O         10.1.13.0/24 [110/2] via 10.1.12.1, 00:20:51, GigabitEthernet0/1/0
```

The bottom half of the example shows router B1's OSPF-learned default route. B1 lists a route for 0.0.0.0/0 as well. The next-hop router in this case is 10.1.12.1, which is Router R1's IP address on the WAN link. The code on the far left is O*E2, meaning an OSPF-learned route, which is a default route, and is specifically an external OSPF route. B1's gateway of last resort setting uses that one OSPF-learned default route, with next-hop router 10.1.12.1.

Finally, the OSPF subcommand **default-information originate** makes the logic conditional: only advertise a default route into OSPF if you already have some other default route. Adding the **always** keyword to the command (**default-information originate always**) tells OSPF to always advertise a default route into OSPF regardless of whether a default route currently exists.

## OSPF Metrics (Cost)

The section "Calculating the Best Routes with SPF" in Chapter 21 discussed how SPF calculates the metric for each route, choosing the route with the best metric for each destination subnet. OSPF routers can influence that choice by changing the OSPF interface cost using three different configuration options:

- Directly, using the interface subcommand **ip ospf cost** *x*.
- Using the default calculation per interface, and changing the **interface bandwidth** setting, which changes the calculated value.
- Using the default calculation per interface, and changing the OSPF **reference bandwidth** setting, which changes the calculated value.

### Setting the Cost Directly

Setting the cost directly requires a simple configuration command, as shown in Example 23-13. The example sets the cost of two interfaces on Router R1. This example uses the Figure 23-4 topology, with a single area, with OSPF enabled on all interfaces shown in the figure. The **show ip ospf interface brief** command that follows details the cost of each interface. Note that the **show** command confirms the cost settings.

**Example 23-13** *Confirming OSPF Interface Costs*

```
R1# conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# interface g0/0/1
R1(config-if)# ip ospf cost 4
R1(config-if)# interface g0/0/2
R1(config-if)# ip ospf cost 5
R1(config-if)# ^Z
R1#
R1# show ip ospf interface brief
Interface    PID   Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/0/0      1     0               10.1.1.1/24        1     DR    0/0
Gi0/0/1      1     0               10.1.12.1/24       4     DR    1/1
Gi0/0/2      1     0               10.1.13.1/24       5     BDR   1/1
Gi0/0/3      1     0               10.1.14.1/24       1     DR    1/1
```

The output also shows a cost value of 1 for the other Gigabit interfaces, which is the default OSPF cost for any interface faster than 100 Mbps. The next topic discusses how IOS determines the default cost values.

### Setting the Cost Based on Interface and Reference Bandwidth

Routers use a per-interface *bandwidth* setting to describe the speed of the interface. Note that the interface bandwidth setting does not influence the actual transmission speed. Instead, the interface bandwidth acts as a configurable setting to represent the speed of the interface, with the option to configure the bandwidth to match the actual transmission speed…or not. To support this logic, IOS sets a default interface bandwidth value that matches the physical transmission speed when possible, but also allows the configuration of the interface bandwidth using the **bandwidth** *speed* interface subcommand.

OSPF (as well as other IOS features) uses the interface bandwidth to make decisions, with OSPF using the interface bandwidth in its calculation of the default OSPF cost for each interface. IOS uses the following formula to choose an interface's OSPF cost for interfaces that do not have an **ip ospf cost** command configured. IOS puts the interface's bandwidth in the denominator and an OSPF setting called the *reference bandwidth* in the numerator:

Reference_bandwidth / Interface_bandwidth

Note that while you can change both the interface bandwidth and reference bandwidth via configuration, because several IOS features make use of the interface bandwidth setting, you should avoid changing the interface bandwidth to influence the default OSPF cost.

Today, most companies override the default IOS reference bandwidth setting. Cisco chose the default setting (100 Mbps) decades ago in an era with much slower links. As a result, when using that default, any interface bandwidth of 100 Mbps or faster results in a calculated OSPF cost of 1. So, when you re relying on the OSPF cost calculation, it helps to configure the reference bandwidth to a speed faster than the fastest speed link in the network.

To see the issue, consider Table 23-3, which lists several types of interfaces, the default interface bandwidth on those interfaces, and the OSPF cost calculated with the default OSPF reference bandwidth of 100 Mbps (that is, 100,000 Kbps). (OSPF rounds up for these calculations, resulting in a lowest possible OSPF interface cost of 1.)

**Table 23-3** Faster Interfaces with Equal OSPF Costs

| Interface | Interface Default Bandwidth (Kbps) | Formula (Kbps) | OSPF Cost |
|---|---|---|---|
| Serial | 1544 Kbps | 100,000 / 1544 | 64 |
| Ethernet | 10,000 Kbps | 100,000 / 10,000 | 10 |
| Fast Ethernet | 100,000 Kbps | 100,000/100,000 | 1 |
| Gigabit Ethernet | 1,000,000 Kbps | 100,000/1,000,000 | 1 |
| 10 Gigabit Ethernet | 10,000,000 Kbps | 100,000/10,000,000 | 1 |
| 100 Gigabit Ethernet | 100,000,000 Kbps | 100,000/100,000,000 | 1 |

As you can see from the table, with a default reference bandwidth, all interfaces from Fast Ethernet's 100 Mbps and faster tie with their default OSPF cost. As a result, OSPF would treat a 100-Mbps link as having the same cost as a 10- or 100-Gbps link, which is probably not the right basis for choosing routes.

You can still use OSPF's default cost calculation (and many do) just by changing the reference bandwidth with the **auto-cost reference-bandwidth** *speed* OSPF mode subcommand. This command sets a value in a unit of megabits per second (Mbps). Set the reference bandwidth value to a value at least as much as the fastest link speed in the network, but preferably higher, in anticipation of adding even faster links in the future.

For instance, in an enterprise whose fastest links are 10 Gbps (10,000 Mbps), you could set all routers to use **auto-cost reference-bandwidth 10000**, meaning 10,000 Mbps or 10 Gbps. In that case, by default, a 10-Gbps link would have an OSPF cost of 1, while a 1-Gbps link would have a cost of 10, and a 100-MBps link a cost of 100.

Better still, in that same enterprise, use a reference bandwidth of a faster speed than the fastest interface in the network, to allow room for higher speeds. For instance, in that same enterprise, whose fastest link is 10 Gbps, set the reference bandwidth to 40 Gbps or even 100 Gbps to be ready for future upgrades to use 40-Gbps links, or even 100-Gbps links. (For example, use the **auto-cost reference-bandwidth 100000** command, meaning 100,000 Mbps or 100 Gbps.) That causes 100-Gbps links to have an OSPF cost of 1, 40-Gbps links to have a cost of 2, 10-Gbps links to have a cost of 10, and 1-Gbps links to have a cost of 100.

**NOTE** Cisco recommends making the OSPF reference bandwidth setting the same on all OSPF routers in an enterprise network.

For convenient study, the following list summarizes the rules for how a router sets its OSPF interface costs:

Key Topic

1. Set the cost explicitly, using the **ip ospf cost** *x* interface subcommand, to a value between 1 and 65,535, inclusive.
2. Although it should be avoided, change the interface bandwidth with the **bandwidth** *speed* command, with *speed* being a number in kilobits per second (Kbps).
3. Change the reference bandwidth, using router OSPF subcommand **auto-cost reference-bandwidth** *ref-bw*, with a unit of megabits per second (Mbps).

## OSPF Hello and Dead Intervals

OSPF does a lot of interesting work with the OSPF Hello message when initializing neighbors. As explained in the section, "Meeting Neighbors and Learning Their Router ID," in Chapter 21, an OSPF router uses Hello messages to announce the router's presence on the link, with those messages sent to the 224.0.0.5 all-OSPF-routers multicast address. The router also listens for incoming Hello messages from potential neighbors. Upon hearing Hellos from each other, two routers check the settings revealed in the Hello, and if compatible, the two routers can proceed to become neighbors.

This section looks at the other end of the story: what happens when the neighbor relationship fails.

First, while the neighbor relationship continues to work, routers send Hello messages regularly per a per-interface **Hello interval** (also called the *Hello timer*). Cisco IOS defaults to a 10-second Hello interval on all Ethernet interfaces. Why? When a router no longer receives the incoming Hello messages from a neighbor, the router believes the neighbor has failed, and takes the neighbor relationship down.

The process to decide when a neighbor fails uses the assumption on continual incoming Hello messages from the neighbor, combined with a second timer: the **Dead interval** (also called the *Dead timer*). The Dead timer tells the interface how long to wait. That is, how long since receiving a Hello message should a router wait before deciding that the neighbor failed.

Figure 23-7 shows an example of the interaction, focusing on what happens if Router R1 fails, and how Router R2 views the messages and timers. In this case, Router R1 loses power, but R2's Ethernet WAN link remains up because the link between R2 and the Ethernet WAN services does not have any problem. The sequence of events follows the circled step numbers as follows, using the default settings of a 10-second Hello timer and 40-second Dead timer:

1. Router R2 has a working neighbor relationship with R1 (full state).
2. R1 sends a Hello message, as usual; R2 resets its Dead timer to 40.
3. R1 sends another Hello message, 10 seconds later; R2 resets its Dead timer to 40.
4. R1 loses power; R2 receives no further Hellos from R1.
5. The Dead timer counts down to 0.
6. Router R2 changes the neighbor state for neighbor R1 to down and reconverges.

Under normal working conditions, with default settings, the Dead timer gets reset to 40, counts down to 30, and is reset to 40, over and over. However, the process also means that you might want to configure to use a lower Dead interval to improve the speed with which routers react to failures.

Example 23-14 shows the straightforward interface subcommands to set the Hello and Dead intervals. The example is based on the network in Figure 23-4, showing the configuration in Router R1, with the three WAN interfaces that connect to routers R2, R3, and R4. Although not shown, the neighboring routers must also be configured with the same Hello and Dead timer settings.

Key Topic

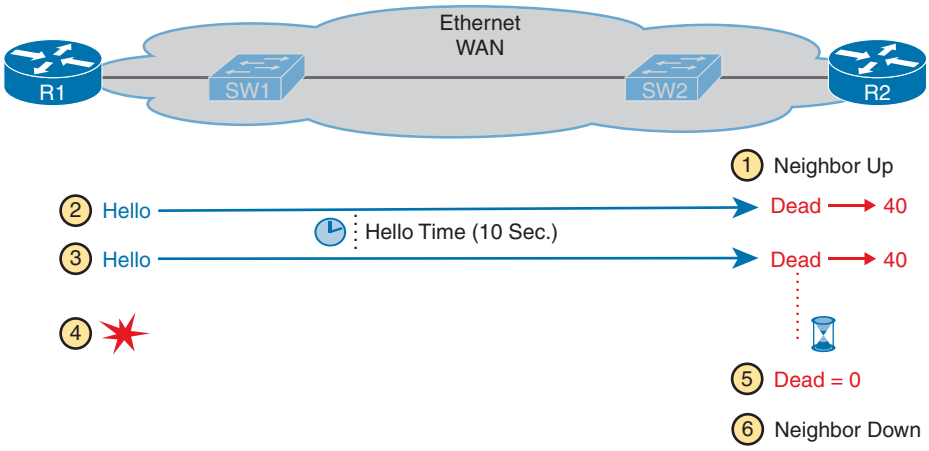



**Figure 23-7** *An Example: R2's Use of the Hello and Dead Intervals*

**Example 23-14** *Configuring OSPF Hello and Dead Intervals*

```
R1# conf t
Enter configuration commands, one per line.  End with CNTL/Z.
! Link connected to router R2
R1(config)# interface g0/0/1
R1(config-if)# ip ospf hello-interval 5
! Link connected to router R3
R1(config)# interface g0/0/2
R1(config-if)# ip ospf dead-interval 20
! Link connected to router R4
R1(config)# interface g0/0/3
R1(config-if)# ip ospf hello-interval 5
R1(config-if)# ip ospf dead-interval 15
R1(config-if)# Ctl-z
R1#
```

Example 23-15 confirms the settings configured in Example 23-14, but with a twist on interface G0/0/1. IOS uses a Hello interval default of 10 seconds on Ethernet interfaces, but a default of four times the Hello interval for the Dead interval. For interface G0/0/1, because only the Hello interval was configured, IOS uses a default dead interval based on 4 × Hello, or 20. However, the other two interfaces follow more predictable logic, summarized by this list, and then confirmed in the highlights in Example 23-15:

- G0/0/1: Hello = 5 (configured) and Dead = 20 (calculated by IOS as 4X Hello)
- G0/0/2: Hello = 10 (IOS default) and Dead = 20 (configured)
- G0/0/3: Hello = 5 and Dead = 15 (both configured)

**Example 23-15** *Confirming New OSPF Hello and Dead Intervals, Per Interface*

```
R1# show ip ospf interface g0/0/1 | include Hello
  Timer intervals configured, Hello 5, Dead 20, Wait 20, Retransmit 5
    Hello due in 00:00:02

R1# show ip ospf interface g0/0/2 | include Hello
  Timer intervals configured, Hello 10, Dead 20, Wait 20, Retransmit 5
    Hello due in 00:00:04

R1# show ip ospf interface g0/0/3 | include Hello
  Timer intervals configured, Hello 5, Dead 15, Wait 15, Retransmit 5
    Hello due in 00:00:03
```

IOS enables us to make poor configuration choices with these two settings. As a fair warning, consider these points:

- The default settings create a 4:1 ratio between the dead:hello timers. That ratio requires four consecutive lost Hellos before the Dead timer would expire. No matter the numbers, consider using the same 4:1 ratio, or at least a 3:1 ratio for your chosen numbers.
- Two neighboring routers must use the same Dead and Hello timer settings. However, IOS allows you to configure different settings on potential neighbors, preventing them from becoming neighbors, so take care to ensure the settings match.
- IOS enables you to make a poor choice to configure a Dead interval smaller than the Hello interval. In that case, the Dead interval expires before a router receives the next Hello. The neighbor relationship fails and recovers repeatedly once per Hello interval. Instead, always set the Dead interval higher than the Hello interval.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 23-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 23-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review Config Checklists | | Book, website |
| Review command tables | | Book |

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Do labs | | Blog |
| Watch videos | | Website |

## Review All the Key Topics

Key Topic

**Table 23-5** Key Topics for Chapter 23

| Key Topic Element | Description | Page Number |
|---|---|---|
| Table 23-2 | Two OSPF network types and key behaviors | 586 |
| Example 23-3 | OSPF interfaces, local roles, and neighbor counts | 589 |
| List | Rules for electing OSPF DR/BDR | 590 |
| Example 23-8 | Evidence of OSPF network type point-to-point | 594 |
| List | Actions taken by OSPF when the interface is passive | 595 |
| Figure 23-5 | Using OSPF to create and flood a default route | 597 |
| List | Rules for how OSPF sets its interface cost settings | 601 |
| Figure 23-7 | Conceptual view of OSPF Hello and Dead timers | 603 |

## Key Terms You Should Know

broadcast network type, Dead interval, Hello interval, interface bandwidth, OSPF priority, passive interface, point-to-point network type, reference bandwidth

## Command References

Tables 23-6 and 23-7 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 23-6** Chapter 23 Configuration Command Reference

| Command | Description |
|---|---|
| **ip ospf network** {**broadcast** \| **point-to-point**} | Interface subcommand used to set the OSPF network type on the interface |
| **ip ospf priority** *value* | Interface subcommand that sets the OSPF priority, used when electing a new DR or BDR |
| **passive-interface** *type number* | Router subcommand that makes the interface passive to OSPF, meaning that the OSPF process will not form neighbor relationships with neighbors reachable on that interface |
| **passive-interface** *default* | OSPF subcommand that changes the OSPF default for interfaces to be passive instead of active (not passive) |
| **no passive-interface** *type number* | OSPF subcommand that tells OSPF to be active (not passive) on that interface or subinterface |

| Command | Description |
|---|---|
| **default-information originate** [**always**] | OSPF subcommand to tell OSPF to create and advertise an OSPF default route, as long as the router has some default route (or to always advertise a default, if the **always** option is configured) |
| **ip ospf cost** *interface-cost* | Interface subcommand that sets the OSPF cost associated with the interface |
| **bandwidth** *bandwidth* | Interface subcommand that directly sets the interface bandwidth (Kbps) |
| **auto-cost reference-bandwidth** *number* | Router subcommand that tells OSPF the numerator in the Reference_bandwidth / Interface_bandwidth formula used to calculate the OSPF cost based on the interface bandwidth |
| **ip ospf hello-interval** *time* | Interface subcommand to set the OSPF Hello interval |
| **ip ospf dead-interval** *time* | Interface subcommand to set the OSPF Dead interval |

**Table 23-7** Chapter 23 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip ospf interface brief** | Lists the interfaces on which the OSPF protocol is enabled (based on the **network** commands), including passive interfaces |
| **show ip ospf interface** [*type number*] | Lists a long section of settings, status, and counters for OSPF operation on all interfaces, or on the listed interface, including the Hello and Dead timers |
| **show ip ospf neighbor** [*type number*] | Lists brief output about neighbors, identified by neighbor router ID, including current state, with one line per neighbor; optionally, limits the output to neighbors on the listed interface |
| **show ip ospf neighbor** *neighbor-ID* | Lists the same output as the **show ip ospf neighbor** detail command, but only for the listed neighbor (by neighbor RID) |
| **show ip route** | Lists all IPv4 routes |
| **show ip route ospf** | Lists routes in the routing table learned by OSPF |

*This page intentionally left blank*



CHAPTER 24

# OSPF Neighbors and Route Selection

**This chapter covers the following exam topics:**

- **3.0 IP Connectivity**
  - **3.1 Interpret the components of routing table**
    - **3.1.e Administrative distance**
  - **3.2 Determine how a router makes a forwarding decision by default**
    - **3.2.a Longest prefix match**
    - **3.2.b Administrative distance**
    - **3.2.c Routing protocol metric**
  - **3.4 Configure and verify single area OSPFv2**
    - **3.4.a Neighbor adjacencies**
    - **3.4.b Point-to-point**
    - **3.4.c Broadcast (DR/BDR selection)**
    - **3.4.d Router ID**

Chapter 21, "Understanding OSPF Concepts," and Chapter 22, "Implementing Basic OSPF Features," discuss the required and most common optional OSPF configuration settings, along with the many verification commands to show how OSPF works with those settings. This chapter continues with more OSPF implementation topics, both to round out the discussion of OSPF and to focus even more on the specific CCNA 200-301 exam topics.

The first major section focuses on neighbors and neighbor adjacencies, as mentioned in yet another of the OSPF exam topics. OSPF routers cannot exchange LSAs with another router unless they first become neighbors. This section discusses the various OSPF features that can prevent OSPF routers from becoming neighbors and how you can go about discovering if those bad conditions exist—even if you do not have access to the running configuration.

The chapter closes with a section called "Route Selection," which discusses OSPF logic plus IP routing logic. This section tackles the question of what route the router should choose, focusing on the cases in which multiple routes exist. The text discusses how OSPF chooses between competing routes and how routers match packet destination addresses to the routes that already exist in the IP routing table.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on

the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 24-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| OSPF Neighbor Relationships | 1–3 |
| Route Selection | 4–7 |

1. An engineer connects Routers R11 and R12 to the same Ethernet LAN and configures them to use OSPFv2. Which answers describe a combination of settings that would prevent the two routers from becoming OSPF neighbors? (Choose two answers.)
   a. R11's interface uses area 11 while R12's interface uses area 12.
   b. R11's OSPF process uses process ID 11 while R12's uses process ID 12.
   c. R11's interface uses OSPF priority 11 while R12's uses OSPF priority 12.
   d. R11's interface uses an OSPF Hello timer value of 11 while R12's uses 12.
2. An engineer connects Routers R13 and R14 to the same Ethernet LAN and configures them to use OSPFv2. Which answers describe a combination of settings that would prevent the two routers from becoming OSPF neighbors?
   a. Both routers' interface IP addresses reside in the same subnet.
   b. Both routers' OSPF process uses process ID 13.
   c. Both routers' OSPF process uses router ID 13.13.13.13.
   d. Both routers' interfaces use an OSPF Dead interval of 40.
3. Router R15 has been a working part of a network that uses OSPFv2. An engineer then issues the **shutdown** command in OSPF configuration mode on R15. Which of the following occurs?
   a. R15 empties its IP routing table of all OSPF routes but keeps its LSDB intact.
   b. R15 empties its OSPF routes and LSDB but keeps OSPF neighbor relationships active.
   c. R15 keeps OSPF neighbors open but does not accept new OSPF neighbors.
   d. R15 keeps all OSPF configuration but ceases all OSPF activities (routes, LSDB, neighbors).
4. Using OSPF, Router R1 learns three routes to subnet 10.1.1.0/24. It first calculates a route through Router R2 with metric 15000, then a route through Router R3 with metric 15001, and then a route through Router R4 with metric 15000. Which routes does the router place into its routing table, assuming all default configuration settings for any features that would impact the answer?
   a. Only the route with R2 as the next-hop router
   b. Both routes with metric 15000
   c. All three routes to subnet 10.1.1.0/24
   d. Only the router with R3 as the next-hop router

5. Router R2 runs both EIGRP and OSPF. It learns two OSPF routes to subnet 172.16.1.0/24, one with metric 1000 and one with metric 2000. It learns two EIGRP routes with metrics 1,000,000 and 2,000,000. If using default settings for any settings that might impact the answer, which route(s) will the router place into the IP routing table?
   a. The metric 1000 OSPF route and the metric 1,0000,000 EIGRP route
   b. The metric 2000 OSPF route and the metric 2,0000,000 EIGRP route
   c. The metric 1000 OSPF route only
   d. The metric 1,0000,000 EIGRP route only
6. Router R3 receives a packet with destination IP address 172.20.89.100. How many of the address ranges defined in the routes per the **show ip route** command match the packet's destination address?

```
R3# show ip route
Gateway of last resort is 172.20.15.5 to network 0.0.0.0

O*E2  0.0.0.0/0 [110/1] via 172.20.15.5, 00:04:56, GigabitEthernet0/1
      172.20.0.0/16 is variably subnetted, 12 subnets, 4 masks
S        172.20.90.9/32 [1/0] via 172.20.11.1
O IA     172.20.88.0/23 [110/3] via 172.20.12.2, 00:03:44, GigabitEthernet0/0/2
O IA     172.20.80.0/20 [110/3] via 172.20.13.3, 00:04:55, GigabitEthernet0/0/3
O IA     172.20.0.0/16 [110/6] via 172.20.14.4, 00:02:14, GigabitEthernet0/0/4
```

   a. 1
   b. 2
   c. 3
   d. 4
   e. 5
7. Router R3 receives a packet with destination IP address 172.20.90.1. Which next-hop IP address does Router R3 use when forwarding the packet?

```
R3# show ip route
Gateway of last resort is 172.20.15.5 to network 0.0.0.0

O*E2  0.0.0.0/0 [110/1] via 172.20.15.5, 00:04:56, GigabitEthernet0/1
      172.20.0.0/16 is variably subnetted, 12 subnets, 4 masks
S        172.20.90.9/32 [1/0] via 172.20.11.1
O IA     172.20.88.0/23 [110/3] via 172.20.12.2, 00:03:44, GigabitEthernet0/0/2
O IA     172.20.80.0/20 [110/3] via 172.20.13.3, 00:04:55, GigabitEthernet0/0/3
O IA     172.20.0.0/16 [110/6] via 172.20.14.4, 00:02:14, GigabitEthernet0/0/4
```

   a. 172.20.11.1
   b. 172.20.12.2
   c. 172.20.13.3
   d. 172.20.14.4
   e. 172.20.15.5

# Foundation Topics

## OSPF Neighbor Relationships

A router's OSPF configuration enables OSPF on a set of interfaces. IOS then attempts to discover other neighbors on those interfaces by sending and listening for OSPF Hello messages. However, once discovered, two routers may not become neighbors. They must have compatible values for several settings as listed in the Hellos exchanged between the two routers. This second major section of the chapter examines those reasons.

### OSPF Neighbor Requirements

After an OSPF router hears a Hello from a new neighbor, the routing protocol examines the information in the Hello and compares that information with the local router's own settings. If the settings match, great. If not, the routers do not become neighbors. Because there is no formal term for all these items that a routing protocol considers, this book just calls them *neighbor requirements*. Table 24-2 lists the neighbor requirements for OSPF, with some comments about the various issues following the table.

Key Topic

**Table 24-2** Neighbor Requirements for OSPF

| Requirement | Required for OSPF | Neighbor Missing If Incorrect |
|---|---|---|
| Interfaces must be in an up/up state. | Yes | Yes |
| Access control lists (ACL) must not filter routing protocol messages. | Yes | Yes |
| Interfaces must be in the same subnet. | Yes | Yes |
| Neighbors must pass routing protocol neighbor authentication (if configured). | Yes | Yes |
| Hello and dead timers must match. | Yes | Yes |
| Router IDs (RID) must be unique. | Yes | Yes |
| Neighbors must be in the same area. | Yes | Yes |
| OSPF process must not be shut down. | Yes | Yes |
| OSPF must not be shut down on the interface. | Yes | Yes |
| Neighboring interfaces must use same MTU setting. | Yes | No |
| Neighboring interfaces must use same OSPF network type. | Yes | No |
| Neighboring interfaces cannot both use priority 0. | Yes | No |

First, consider the meaning of the two rightmost columns. The column labeled "Required for OSPF" means that the item must be working correctly for the neighbor relationship to work correctly. The last column heading notes whether the neighbor will be missing ("yes") in the list of OSPF neighbors in commands like the **show ip ospf neighbor** command.

The table breaks into three sections. The first section lists non-OSPF configuration while the second lists OSPF configuration—all of which prevents OSPF neighbor relationships from forming. The third section lists settings that allow OSPF neighbor relationships, but with other related problems that prevent the addition of correct OSPF routes to the IP routing table.

For reference, Table 24-3 relists some of the requirements from Table 24-2, along with the most useful commands to find the related settings.

Key Topic

**Table 24-3** OSPF Neighbor Requirements and the Best **show/debug** Commands

| Requirement | Best show Command |
|---|---|
| Hello and dead timers must match. | **show ip ospf interface** |
| Neighbors must be in the same area. | **show ip ospf interface brief** |
| RIDs must be unique. | **show ip ospf** |
| Neighbors must pass any neighbor authentication. | **show ip ospf interface** |
| OSPF process must not be shut down. | **show ip ospf, show ip ospf interface** |

The rest of this section looks at some of the items from Table 24-3 in a little more detail.

**NOTE** One configuration choice that people sometimes think is an issue, but is not, is the process ID as defined by the **router ospf** *process-id* command. Neighboring routers can use the same process ID values, or different process ID values, with no impact on whether two routers become OSPF neighbors.

## Issues That Prevent Neighbor Adjacencies

The next few pages look at three neighbor issues from Table 24-2, using Figure 24-1's topology in the examples. R1 begins with all correct configuration as listed in Example 24-1. However, later examples introduce configuration mistakes on Routers R2, R3, and R4 as follows:

- R2 has been reconfigured to place both LAN interfaces in area 1, whereas the other three routers' G0/0 interfaces remain in area 0.
- R3 has been reconfigured to use the same RID (1.1.1.1) as R1.
- R4 has been reconfigured with Hello/Dead timers of 5/20 on its G0/0 interface, instead of the default settings of 10/40 used by R1, R2, and R3.

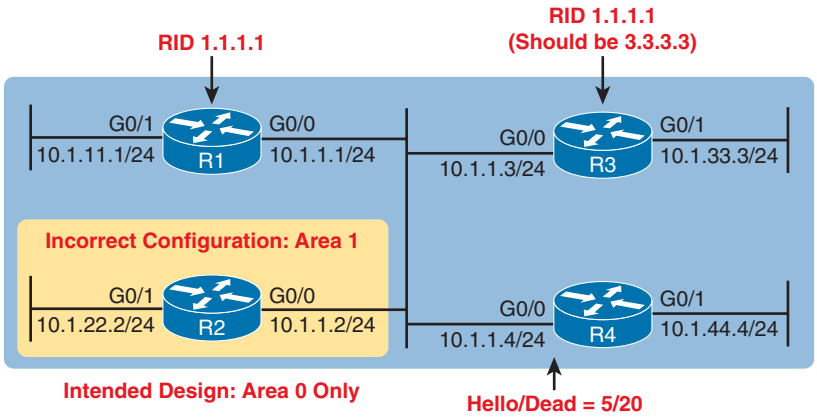



**Figure 24-1** *Summary of Problems That Prevent OSPF Neighbors on the Central LAN*

Answers to the "Do I Know This Already?" quiz:

**1** A, D **2** C **3** D **4** B **5** D **6** D **7** C

**Example 24-1** *Router R1 Configuration with No Configuration Issues*

```
router ospf 1
 router-id 1.1.1.1
!
 interface gigabitEthernet0/0
 ip address 10.1.1.1 255.255.255.0
 ip ospf 1 area 0
!
interface gigabitEthernet0/1
 ip address 10.1.11.1 255.255.255.0
 ip ospf 1 area 0
```

### Finding Area Mismatches

To create an area mismatch, the configuration on some router must place the interface into the wrong area per the design. Figure 24-1 shows the intent to make that mistake on Router R2, placing both its interfaces into area 1 instead of area 0. Example 24-2 shows the configuration, which uses the correct syntax (and is therefore accepted by the router) but sets the wrong area number.

**Example 24-2** *Setting Area 1 on R2's Interfaces, When They Should Be in Area 0*

```
router ospf 1
 router-id 2.2.2.2
!
interface gigabitEthernet0/0
 ip address 10.1.1.2 255.255.255.0
 ip ospf 1 area 1
!
interface gigabitEthernet0/1
 ip address 10.1.22.2 255.255.255.0
 ip ospf 1 area 1
```

With an area mismatch error, the **show ip ospf neighbor** command will not list the neighbor. Because you see nothing in the OSPF neighbor table, to troubleshoot this problem, you need to find the area configuration on each interface on potentially neighboring routers. To do so:

- Check the output of **show running-config** to look for:
  - **ip ospf** *process-id* **area** *area-number* interface subcommands
  - **network** commands in OSPF configuration mode
- Use the **show ip ospf interface** [**brief**] command to list the area number

### Finding Duplicate OSPF Router IDs

Next, Example 24-3 shows R1 and R3 both trying to use RID 1.1.1.1. Due to the duplicate RIDs, neither router will list the other in the output of the **show ip ospf neighbor** command. Interestingly, both routers automatically generate a log message for the duplicate OSPF RID problem between R1 and R3; the end of Example 24-3 shows one such message. For the

exams, just use the **show ip ospf** commands on both R3 and R1 to easily list the RID on each router, noting that they both use the same value.

**Example 24-3** *Comparing OSPF Router IDs on R1 and R3*

```
! Next, on R3: R3 lists the RID of 1.1.1.1
!
R3# show ip ospf
Routing Process "ospf 1" with ID 1.1.1.1
Start time: 00:00:37.136, Time elapsed: 02:20:37.200
! lines omitted for brevity
```

```
! Back to R1: R1 also uses RID 1.1.1.1
R1# show ip ospf
Routing Process "ospf 1" with ID 1.1.1.1
Start time: 00:01:51.864, Time elapsed: 12:13:50.904
! lines omitted for brevity

*May 29 00:01:25.679: %OSPF-4-DUP_RTRID_NBR: OSPF detected duplicate router-id
1.1.1.1 from 10.1.1.3 on interface GigabitEthernet0/0
```

First, focus on the problem: the duplicate RIDs. The first line of the **show ip ospf** command on the two routers quickly shows the duplicate use of 1.1.1.1. To solve the problem, assuming R1 should use 1.1.1.1 and R3 should use another RID (maybe 3.3.3.3), change the RID on R3 and restart the OSPF process. To do so, use the **router-id 3.3.3.3** OSPF subcommand and the EXEC mode command **clear ip ospf process**. (OSPF will not begin using a new RID value until the process restarts, either via command or reload.) At that point, the routers should become neighbors again and be displayed in the output of the **show ip ospf neighbor** command.

**NOTE** There are cases in which routers in different areas can use the same RID and cause no problems in OSPF. However, to be safe, use unique OSPF RIDs throughout the entire OSPF domain (that is, among all routers in your enterprise that use OSPF).

### Finding OSPF Hello and Dead Timer Mismatches

First, as a reminder from chapters past:

- **Hello interval/timer:** The per-interface timer that tells a router how often to send OSPF Hello messages on an interface.
- **Dead interval/timer:** The per-interface timer that tells the router how long to wait without having received a Hello from a neighbor before believing that neighbor has failed. (Defaults to four times the Hello timer.)

Next, consider the problem created on R4, with the configuration of a different Hello timer and dead timer (5 and 20, respectively) as compared with the default settings on R1, R2, and R3 (10 and 40, respectively). A Hello or Dead interval mismatch prevents R4 from becoming

neighbors with any of the other three OSPF routers. Routers list their Hello and Dead interval settings in their Hello messages and choose not to become neighbors if the values do not match. As a result, none of the routers become neighbors with Router R4 in this case.

Example 24-4 shows the easiest way to find the mismatch using the **show ip ospf interface** command on both R1 and R4. This command lists the Hello and Dead timers for each interface, as highlighted in the example. Note that R1 uses 10 and 40 (Hello and Dead), whereas R4 uses 5 and 20.

**Example 24-4** *Finding Mismatched Hello/Dead Timers*

```
R1# show ip ospf interface G0/0
GigabitEthernet0/0 is up, line protocol is up
  Internet Address 10.1.1.1/24, Area 0, Attached via Network Statement
  Process ID 1, Router ID 1.1.1.1, Network Type BROADCAST, Cost: 1
  Topology-MTID  Cost  Disabled  Shutdown   Topology Name
        0         1       no        no         Base
  Transmit Delay is 1 sec, State DR, Priority 1
  Designated Router (ID) 1.1.1.1, Interface address 10.1.1.1
  No backup designated router on this network
  Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
! lines omitted for brevity
```

```
! Moving on to R4 next
!
R4# show ip ospf interface Gi0/0
GigabitEthernet0/0 is up, line protocol is up
   Internet Address 10.1.1.4/24, Area 0, Attached via Network Statement
   Process ID 4, Router ID 10.1.44.4, Network Type BROADCAST, Cost: 1
  Topology-MTID  Cost  Disabled  Shutdown   Topology Name
        0         1       no        no         Base
  Transmit Delay is 1 sec, State DR, Priority 1
  Transmit Delay is 1 sec, State DR, Priority 1
  Designated Router (ID) 10.1.44.4, Interface address 10.1.1.4
  No backup designated router on this network
  Timer intervals configured, Hello 5, Dead 20, Wait 20, Retransmit 5
! lines omitted for brevity
```

## Shutting Down the OSPF Process

Like administratively disabling and enabling an interface, IOS also allows the OSPFv2 routing protocol process to be disabled and enabled with the **shutdown** and **no shutdown** router mode subcommands, respectively. When a routing protocol process is shut down, IOS does the following:

- Brings down all neighbor relationships and clears the OSPF neighbor table
- Clears the LSDB
- Clears the IP routing table of any OSPF-learned routes

At the same time, shutting down OSPF does retain some important details about OSPF, in particular:

- IOS retains all OSPF configuration.
- IOS still lists all OSPF-enabled interfaces in the OSPF interface list (**show ip ospf interface**) but in a DOWN state.

Shutting down the OSPF routing protocol process allows the network engineer to stop using the routing protocol on that router without having to remove all the configuration. Once the process is shut down, the **show ip ospf interface** [**brief**] command should still list some output, as will the **show ip ospf** command, but the rest of the commands will list nothing.

Example 24-5 shows an example on Router R5, as shown in Figure 24-2. R5 is a different router than the one used in earlier examples, but it begins the example with two OSPF neighbors, R2 and R3, with Router IDs 2.2.2.2 and 3.3.3.3. The example shows the OSPF process being shut down, the neighbors failing, and those two key OSPF **show** commands: **show ip ospf neighbor** and **show ip ospf interface brief**.

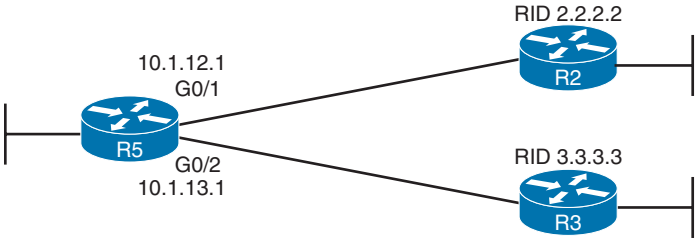



**Figure 24-2** *Example Network to Demonstrate OSPF Process Shutdown*

**Example 24-5** *Shutting Down an OSPF Process, and the Resulting Neighbor States*

```
R5# show ip ospf neighbor
Neighbor ID     Pri   State           Dead Time   Address         Interface
2.2.2.2           1   FULL/DR         00:00:35    10.1.12.2       GigabitEthernet0/1
3.3.3.3           1   FULL/DR         00:00:33    10.1.13.3       GigabitEthernet0/2
R5# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R5(config)# router ospf 1
R5(config-router)# shutdown
R5(config-router)# ^Z
*Mar 23 12:43:30.634: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/1
from FULL to DOWN, Neighbor Down: Interface down or detached
*Mar 23 12:43:30.635: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/2
from FULL to DOWN, Neighbor Down: Interface down or detached
R5# show ip ospf interface brief
Interface    PID     Area            IP Address/Mask    Cost  State Nbrs F/C
Gi0/1        1       0               10.1.12.1/24       1     DOWN  0/0
Gi0/2        1       0               10.1.13.1/24       1     DOWN  0/0
```

```
R5# show ip ospf
 Routing Process "ospf 1" with ID 5.5.5.5
 Start time: 5d23h, Time elapsed: 1d04h
 Routing Process is shutdown
! lines omitted for brevity

R5# show ip ospf neighbor
R5#
R5# show ip ospf database
            OSPF Router with ID (3.3.3.3) (Process ID 1)
R5#
```

First, before the **shutdown**, the **show ip ospf neighbor** command lists two neighbors. After the **shutdown**, the same command lists no neighbors at all. Second, the **show ip ospf interface brief** command lists the interfaces on which OSPF is enabled, on the local router's interfaces. However, it lists a state of DOWN, which is a reference to the local router's state. Also, note that the **show ip ospf** command positively states that the OSPF process is in a shutdown state, while the **show ip ospf database** command output lists only a heading line, with no LSAs.

### Shutting Down OSPF on an Interface

IOS also supports a feature to disable OSPF on an interface without having to remove any OSPF configuration to do so. The feature has the same motivations as the **shutdown** command in router configuration mode: to allow OSPF configuration to remain while stopping OSPF. However, shutting down OSPF on an interface ceases all OSPF operations on that interface rather than all OSPF operations on the router.

To shut down OSPF on an interface, use the **ip ospf shutdown** interface subcommand, and to enable it again, use the **no ip ospf shutdown** command. After you use the **ip ospf shutdown** command, the router changes as follows:

- The router stops sending Hellos on the interface, allowing existing OSPF neighbor relationships to time out.
- The neighbor failure(s) triggers OSPF reconvergence, resulting in the removal of any routes that use the interface as an outgoing interface.
- The router also stops advertising about the subnet on the link that is shut down.

If those ideas sound familiar, the feature works much like OSPF passive interfaces, except that when shut down, OSPF also stops advertising about the connected subnet.

Interestingly, using the **ip ospf shutdown** interface subcommand also does not change a few commands about interfaces. For instance, after you configure interface G0/0/0 with the **ip ospf shutdown** command, the **show ip ospf interface** and **show ip ospf interface brief** commands still show G0/0/0 in the list of interfaces.

## Issues That Allow Neighbors but Prevent IP Routes

Some configuration mismatches prevent learning OSPF routes, but they do allow routers to become neighbors, as noted in the final section of Table 24-2. The issues are



- A mismatched MTU setting
- A mismatched OSPF network type
- Both neighbors using OSPF Priority 0

The next few pages explain the issues, with an example that shows all three misconfigurations along with the resulting **show** commands.

### Mismatched MTU Settings

The maximum transmission unit (MTU) size defines a per-interface setting used by the router for its Layer 3 forwarding logic, defining the largest network layer packet that the router will forward out each interface. The IPv4 MTU size of an interface defines the maximum size IPv4 packet that the router can forward out an interface, and similarly, the IPv6 MTU size defines the largest IPv6 packet.

Routers often use a default IP MTU size of 1500 bytes, with the ability to set the value as well. The **ip mtu** *size* interface subcommand defines the IPv4 MTU setting, and the **ipv6 mtu** *size* command sets the equivalent for IPv6 packets.

Alternatively, you can set the MTU size for IPv4 and IPv6 using the **mtu** *size* interface subcommand. That command sets the MTU for IPv4 on the interface if the **ip mtu** *size* command does not appear and for IPv6 if the **ipv6 mtu** *size* command does not appear.

With different IPv4 MTU settings, two OSPFv2 routers become OSPF neighbors; however, they fail to complete regular OSPF database exchange, reaching other interim OSPF neighbor states and then failing to a down state. Over time, they repeat the process to become neighbors, try to exchange their LSDBs, fail, and fall back to a down state.

### Mismatched OSPF Network Types

In the section, "OSPF Network Types," in Chapter 23, "Implementing Optional OSPF Features," you read about the OSPF broadcast network type, which uses a DR/BDR, and the OSPF point-to-point network type, which does not. Interestingly, if you misconfigure network type settings such that one router uses broadcast, and the other uses point-to-point, the two routers become neighbors and reach a full state. They remain stable in a full state, which means they exchanged their LSDBs; however, neither router can use routes based on LSAs learned from the neighbor.

The reason for not adding the routes has to do with the details of LSAs and how the use of a DR (or not) changes those LSAs. Basically, the two routers expect different details in the LSAs, and the SPF algorithm notices those differences and cannot resolve those differences when calculating routes.

### Both Neighbors Using OSPF Priority 0

OSPF interface priority allows us to influence which router wins a DR or BDR election when using the broadcast network type. The highest priority router wins the election, with 1 as the default setting and allowed values ranging from 0 to 255 decimal.

Priority 0 acts as a special case meaning that the router will not serve as DR or BDR. For instance, in a topology with many routers sharing the same subnet, you could choose to make some routers use priority 0, effectively refusing to become DR or BDR so that you have more predictability when operating the network.

A problem occurs when you make the poor configuration choice to make all routers on a subnet use priority 0. If making that mistake, you have a subnet that must have a DR but for which all routers refuse the role. The routers proceed as normal to use Hellos to discover each other. They list each other as neighbors and reach a 2WAY state. However, because no router serves as the DR, the database exchange process stops at that point. The typical symptom is a stable neighbor in a 2WAY/DROTHER state.

## Examples That Show OSPF Neighbors but No Routes

Figure 24-3 shows a router topology used in an upcoming example. In this case, R1 uses poor configuration choices that cause the problems discussed in the previous few pages. In particular:

**R1 G0/0/1:** Uses network type point-to-point; R2 uses default setting broadcast.

**R1 G0/0/2:** Uses an IPv4 MTU of 1600; R3 uses the default setting 1500.

**R1 G0/0/3:** Both R1 and R4 use an OSPF priority of 0.

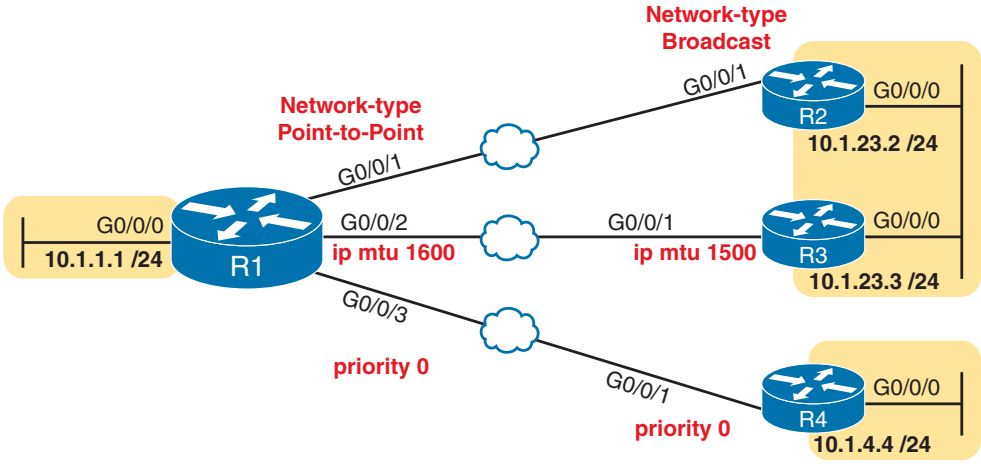



**Figure 24-3** *OSPF Mistakes: Neighbors Formed, but Routes Not Learned*

To begin, Example 24-6 shows the Router R1 configuration to cause the problems shown in the figure. Routers R2, R3, and R4 use all default configuration related to these parameters other than R4, which also requires the **ip ospf priority 0** interface subcommand. The example shows only the relevant configuration.

Example 24-7 shows the results. First, note that Routers R2, R3, and R4 use OSPF RIDs 2.2.2.2, 3.3.3.3, and 4.4.4.4, respectively. The example begins with a **show ip ospf neighbor** command at a time when the neighbor relationships reach their best states.

**Example 24-6** *OSPF Configuration Settings Matching Figure 24-3*

```
R1# show running-config interface g0/0/1
! Neighboring router R2 defaults to network type broadcast
interface GigabitEthernet0/0/1
 ip address 10.1.12.1 255.255.255.0
 ip ospf network point-to-point
 ip ospf 1 area 0

R1# show running-config interface g0/0/2
! Neighboring router R3 defaults to IP MTU 1500
interface GigabitEthernet0/0/2
 ip address 10.1.13.1 255.255.255.0
 ip mtu 1600
 ip ospf 1 area 0

R1# show running-config interface g0/0/3
! Neighboring router R4 is also configured for OSPF priority 0
interface GigabitEthernet0/0/3
 ip address 10.1.14.1 255.255.255.0
 ip ospf priority 0
 ip ospf 1 area 0
```

**Example 24-7** *Resulting OSPF Neighbor Relationships on Router R1*

```
R1# show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address       Interface
4.4.4.4           0   2WAY/DROTHER    00:00:38    10.1.14.4     GigabitEthernet0/0/3
3.3.3.3           0   EXCHANGE/DROTHER00:00:38    10.1.13.3     GigabitEthernet0/0/2
2.2.2.2           0   FULL/  -        00:00:39    10.1.12.2     GigabitEthernet0/0/1
R1#

*Nov  2 21:38:34.046: %OSPF-5-ADJCHG: Process 1, Nbr 3.3.3.3 on GigabitEthernet0/0/2
from EXCHANGE to DOWN, Neighbor Down: Too many retransmissions

R1# show ip ospf neighbor

Neighbor ID     Pri   State           Dead Time   Address       Interface
4.4.4.4           0   2WAY/DROTHER    00:00:31    10.1.14.4     GigabitEthernet0/0/3
3.3.3.3           0   DOWN/DROTHER       -        10.1.13.3     GigabitEthernet0/0/2
2.2.2.2           0   FULL/  -        00:00:34    10.1.12.2     GigabitEthernet0/0/1
```

In this case, R1's neighbor relationship with Router R3 (3.3.3.3) cycles through different states. The middle of the example shows messages about how this neighbor relationship fails. Then the end of the example shows neighbor 3.3.3.3 now in a DOWN state. Over time, the neighbor relationship cycles through repeated attempts.

Note that none of the three neighbor relationships result in any IP routes. In this case, the **show ip route ospf** command on R1 lists zero routes.



## Route Selection

When OSPF calculates routes to a specific subnet, the competition between routes might be simple and obvious. A router might calculate a single route for a subnet and use that route. A router might calculate multiple routes to reach a subnet, so it finds the route with the lowest metric (cost), placing that route into the IP routing table.

However, other scenarios occur both inside and outside of OSPF, requiring more understanding of the logic used by a router. This final major section of the chapter discusses three topics that make us think about the following:

1. How one router chooses between multiple equal-cost OSPF routes for one subnet
2. How one router chooses between multiple routes for one subnet that were learned by different routing protocols
3. When forwarding packets, how routers match routes in the routing table, particularly when a packet's destination address matches more than one route

### Equal-Cost Multipath OSPF Routes

Consider the routes that one OSPF router calculates for one destination subnet. When more than one route to that one subnet exists, one route may have the lowest metric, making OSPF's logic simple: Add that route to the IP routing table. However, when the metrics tie for multiple routes to the same subnet, the router applies different logic: It places multiple **equal-cost routes** in the routing table.

IOS limits the concurrent equal-cost OSPF routes for each destination subnet based on the **maximum-paths** *number* subcommand under **router ospf**. The default varies depending on the router platform, with a common default setting of four concurrent routes.

For example, consider the most recent Figure 24-3, but with all configuration errors fixed, all routers learn routes to all subnets. Router R1 has only one possible route to reach subnet 10.1.4.0/24, located off Router R4. However, R1 has two possible routes to reach subnet 10.1.23.0/24: one route through neighbor R2 and one through neighbor R3. If using OSPF costs 10 for all GigabitEthernet interfaces, both R1's routes to subnet 10.1.23.0/24 have a cost of 20. Example 24-8 shows that result with a list of OSPF-learned routes on Router R1.

Take an extra moment to look closely at the last two lines of output. Note that the output does not repeat the subnet ID and mask, listing it once, but leaving spaces in that position in the last line. That format should help you identify cases like this of multiple routes for the same subnet. Also, note the second number within brackets lists the metric, 20 for both routes, confirming they have an equal metric (cost).

**Example 24-8** *OSPF Routes on Router R1 from Figure 24-3*

```
R1# show ip route ospf
! Legend omitted for brevity

Gateway of last resort is not set

      10.0.0.0/8 is variably subnetted, 14 subnets, 2 masks
O        10.1.4.0/24 [110/20] via 10.1.14.4, 00:00:17, GigabitEthernet0/0/3
O        10.1.23.0/24 [110/20] via 10.1.13.3, 00:00:20, GigabitEthernet0/0/2
                      [110/20] via 10.1.12.2, 00:00:23, GigabitEthernet0/0/1
```

Now, think about packet forwarding instead of thinking about the logic of placing routes into the routing table. Which of the two routes to subnet 10.1.23.0/24 should R1 use? How should it take advantage of the two equal-cost routes? A router could load balance the packets on a per-packet basis, but that is a poor choice for a few reasons. Instead, by default, routers balance based on the individual destination address. The router sends packets for one destination IP address using one route, another destination address using the other route, and so on. Using destination-based load balancing allows for much less router overhead and avoids some of the problems that occur with per-packet load balancing.

**NOTE** The logic in this section often goes by the name **equal-cost multipath (ECMP)**.

## Multiple Routes Learned from Competing Sources

A typical enterprise router first learns connected routes, based on interface IP addresses. Those routes happen to be the best routes to reach those subnets because those subnets connect directly to the router. Additionally, each enterprise router uses one routing protocol (for example, OSPF) to dynamically learn all other routes.

However, in several legitimate cases, one router learns routes using more than connected routes plus one routing protocol. For instance, some routers also have static routes, as discussed in Chapter 17, "Configuring IPv4 Addresses and Static Routes." A router configuration could define a static route to a subnet, while the IGP also learns a route to that same subnet. Which route is better? In other cases, one router might run both OSPF and BGP. Again, if both learn a route to the same subnet, which is better? Or a router might use multiple IGPs, like both OSPF and EIGRP. Again, both might learn a route to the same subnet, begging the same question.

**NOTE** To consider routes to be to the same subnet, they must refer to the same subnet ID and subnet mask.

Routing protocol metrics do not help a router choose between competing routes in these cases. For instance, EIGRP commonly assigns metrics with values in the millions and billions, with OSPF using hundreds or thousands. Additionally, connected and static routes have no

metrics. So metrics cannot help the router choose the best route in these cases, so IOS needs another method to choose between routes from different sources.

When IOS must choose between routes learned using different routing protocols, IOS uses a concept called **administrative distance**. Administrative distance (AD) is a number that denotes how believable an entire routing protocol is on a single router. The lower the number, the better, or more believable, the routing protocol. For example, RIP has a default administrative distance of 120, OSPF uses a default of 110, and EIGRP defaults to 90. When using OSPF and EIGRP, the router will believe the EIGRP route instead of the OSPF route (at least by default). The administrative distance values are configured on a single router and are not exchanged with other routers. Table 24-4 lists the various sources of routing information, along with the default administrative distances.

Key Topic

**Table 24-4** Default Administrative Distances

| Route Type | Administrative Distance |
|---|---|
| Connected | 0 |
| Static | 1 |
| BGP (external routes [eBGP]) | 20 |
| EIGRP (internal routes) | 90 |
| OSPF | 110 |
| IS-IS | 115 |
| RIP | 120 |
| EIGRP (external routes) | 170 |
| BGP (internal routes [iBGP]) | 200 |
| DHCP default route | 254 |
| Unusable | 255 |

**NOTE** The **show ip route** command lists each route's administrative distance as the first of the two numbers inside the brackets. The second number in brackets is the metric.

Figure 24-4 shows what might happen inside each routing process and the choice to prefer the EIGRP route instead of the OSPF route. The left side shows how OSPF learns three routes to subnet 10.1.1.0/24, while EIGRP learns two routes to the same subnet. Each routing protocol chooses the lowest metric route to offer as the best route. However, an additional logic step then considers the administrative distance (AD) of the route. With default settings, the router chooses the EIGRP (AD 90) route instead of the OSPF (AD 110) route for subnet 10.1.1.0/24.

You might wonder at the choice to use more than one routing protocol. One classic case occurs when two companies merge into one company, as shown in Figure 24-5. Company A on the left uses EIGRP with addresses from private network 10.0.0.0. Company B on the right uses OSPF with addresses from private network 172.16.0.0. As a first step to connect the networks, the network engineers install new WAN links between the A1 and B1 router plus the A2 and B2 router as shown in the figure.

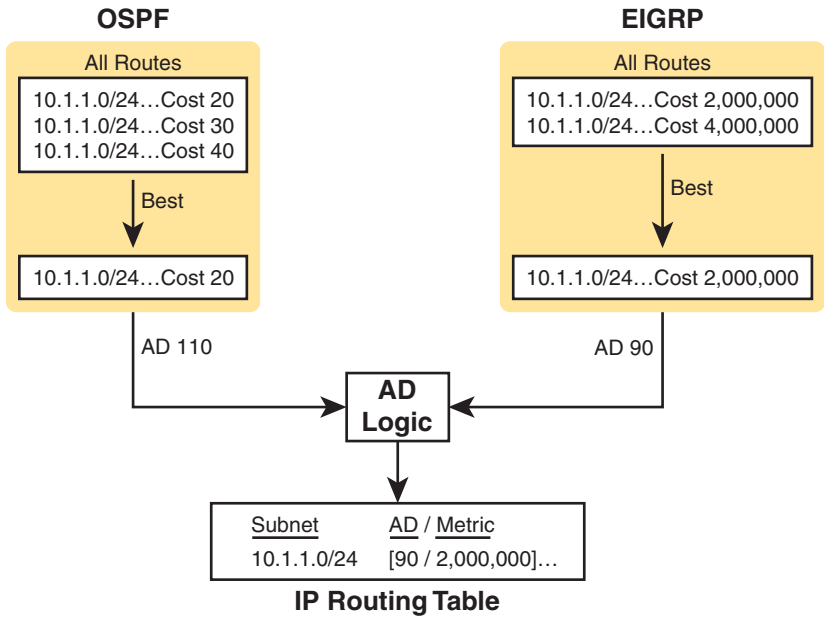



**Figure 24-4** *Logic: Choosing the Lowest AD Route Between OSPF and EIGRP*

Next, the routers need to learn routes from both companies. The EIGRP routers on the left need to learn the company B routes known to OSPF on the right, and vice versa. To do that, the networking staff uses a feature called *route redistribution*, in which a *small set of routers run both routing protocols*. Internal to those routers, the router redistributes (takes routes from one protocol and advertises into the other), taking OSPF routes and advertising those subnets using EIGRP, and vice versa.

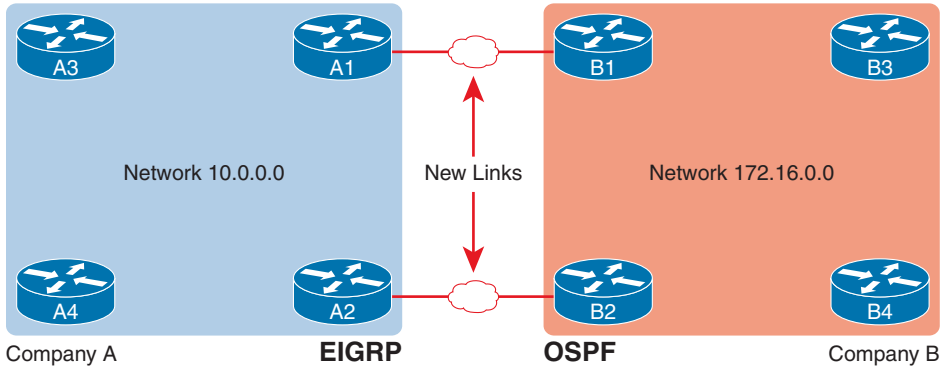



**Figure 24-5** *First Step: Adding WAN Links Between Existing Company A and B Routers*

Figure 24-6 shows the updated scenario, with Routers A1 and A2 performing route redistribution. Most routers continue to run only OSPF or only EIGRP; in this case, Routers B1, B2, B3, and B4 continue to use only OSPF, whereas A3 and A4 use only EIGRP. However, the redistribution process on Routers A1 and A2 advertises routes so that all learn routes to all subnets in network 10.0.0.0 and network 172.16.0.0.

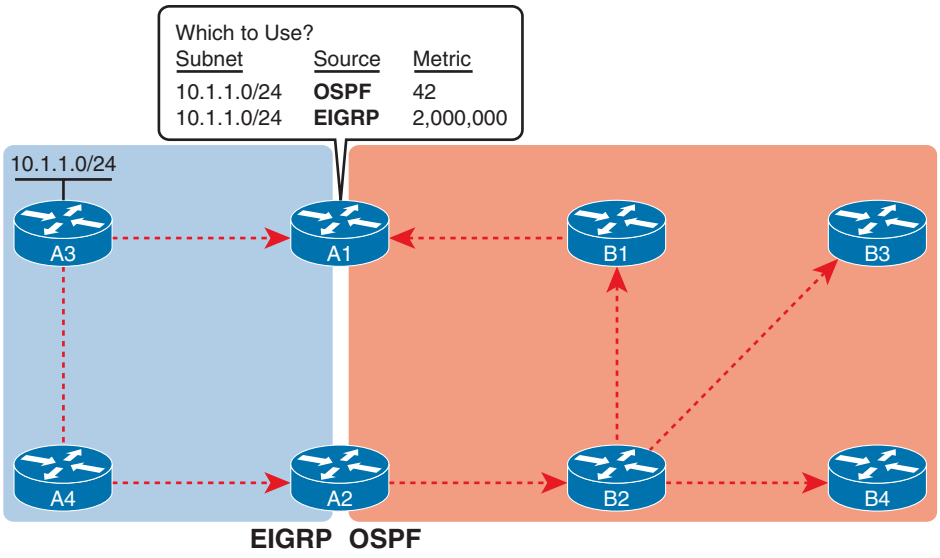



**Figure 24-6** *Example: Router A1 Learns an EIGRP and OSPF Route to Subnet 10.1.1.0/24*

The figure uses dashed lines to show the propagation of routing information about subnet 10.1.1.0/24. First, focusing on the left, all four EIGRP routers in former Company A learn a route for subnet 10.1.1.0/24 using EIGRP. Next, follow the dashed lines along the bottom of the drawing, with Router A2 redistributing a route for 10.1.1.0/24 into OSPF. The OSPF routers all learn routes for 10.1.1.0/24 using OSPF. Now Router A1 has a route for subnet 10.1.1.0/24 learned with EIGRP and another learned with OSPF, creating a real-life scenario in which it must use administrative distance.

**NOTE** The CCNP Enterprise certification discusses route redistribution, both concept and configuration. However, the CCNA V1.1 blueprint does not. It is mentioned here only to show a realistic case when a router needs to use administrative distance.

**NOTE** The section "Floating Static Routes" in Chapter 17 discusses another example of how routers use the AD to choose routes. Take a moment to review that section if you do not recall the details.

Now that you understand the context, try memorizing the administrative distance values in Table 24-4.

## IP Forwarding with the Longest Prefix Match

For the final few pages of this chapter, focus on how a router matches a packet's destination IP address to one of the routes already placed in the routing table. That process goes by a few terms, like *IP routing*, *forwarding*, or the *data plane*. Regardless, focus on the logic.

A router's IP routing process requires that the router compare the destination IP address of each packet with the existing contents of that router's IP routing table. Often, only one route matches a particular destination address. When only one route matches the packet's destination, the action is obvious: forward the packet based on the details listed in that route.

In some cases, multiple routes exist for the exact same subnet, that is, for the exact same subnet and mask. The earlier section, "Equal-Cost Multipath OSPF Routes," discussed how OSPF would choose to add these multiple routes to the routing table, and how a router will load balance packets that match those routes.

This section discusses a different case in which a set of routes lists different subnets whose address ranges overlap. In that case, some packets' destination addresses match multiple routes in the IP routing table. For instance, one route might list subnet 10.1.0.0/16, another 10.1.1.0/25, and another 10.1.1.1/32. The range of addresses in each of those subnets includes 10.1.1.1, so a packet sent to IP address 10.1.1.1 would match all those routes.

Many legitimate router features can cause these multiple overlapping routes to appear in a router's routing table, including

- Static routes (including host routes)
- Route autosummarization
- Manual route summarization
- Default routes

In this case, a router chooses the best route as follows:

Key Topic

When a particular destination IP address matches more than one route in a router's IPv4 routing table, and those routes list different rather than identical subnets (different subnet IDs and masks), the router uses **longest prefix match** logic to match the most specific route—the route with the longest prefix length mask.

### Using Your Subnetting Math Skills to Predict the Choice of Best Route

One way to predict which route a router uses requires you to use your subnetting skills plus the output from the **show ip route** command. To see how it works, an upcoming example uses several overlapping routes learned by Router R1 in Figure 24-7. Example 24-9 focuses on routes that match PC D's IP address (172.16.10.4), matching four routes on Router R1.

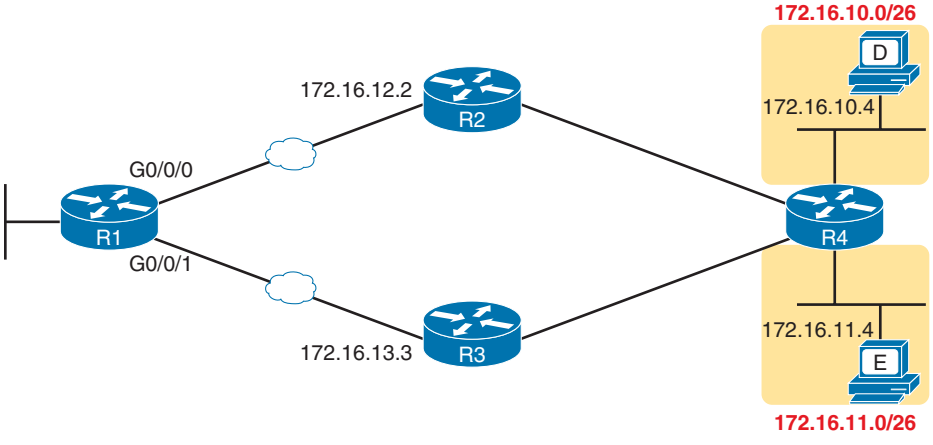



**Figure 24-7** *Topology, Interfaces, and Next-Hop Addresses Used with Example 24-9*

Key Topic

**Example 24-9** show ip route *Command with Overlapping Routes*

```
R1# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route, H - NHRP, l - LISP
       + - replicated route, % - next hop override

Gateway of last resort is 172.16.13.3 to network 0.0.0.0

O*E2  0.0.0.0/0 [110/1] via 172.16.13.3, 00:04:56, GigabitEthernet0/1
      172.16.0.0/16 is variably subnetted, 12 subnets, 4 masks
S        172.16.10.4/32 [1/0] via 172.16.12.2
O IA     172.16.10.0/26 [110/3] via 172.16.13.3, 00:04:56, GigabitEthernet0/0/1
O IA     172.16.10.0/23 [110/6] via 172.16.12.2, 00:04:56, GigabitEthernet0/0/0
O IA     172.16.11.0/26 [110/3] via 172.16.13.3, 00:04:56, GigabitEthernet0/0/1
! Non-overlapping routes omitted for brevity
```

On the exam, or in real life, you would look at each route's subnet ID and mask and do the math to list the range of addresses in each subnet. Table 24-5 shows the result to ensure the math does not get in the way of your understanding the concepts. All the subnets listed in the table match the destination address 172.16.10.4 (PC D).

**Table 24-5** Analysis of Address Ranges for the Subnets in Example 24-9

| Subnet/Prefix of a Route | Address Range | Next-Hop |
|---|---|---|
| 172.16.10.4/32 | 172.16.10.4 (just this one address) | 172.16.12.2 (R2) |
| 172.16.10.0/26 | 172.16.10.0–172.16.10.63 | 172.16.13.3 (R3) |
| 172.16.10.0/23 | 172.16.10.0–172.16.11.255 | 172.16.12.2 (R2) |
| 0.0.0.0/0 | 0.0.0.0–255.255.255.255 (all addresses) | 172.16.13.3 (R3) |

**NOTE** The route listed as 0.0.0.0/0 is the default route.

Working through the logic, note that packet destination 172.16.10.4 matches all routes highlighted in Example 24-9 (the routes also listed in Table 24-5). The various prefix lengths (masks) range from /0 to /32. The longest prefix (largest /P value, meaning the best and most specific route) is /32. So, a packet sent to 172.16.10.4 uses the route to 172.16.10.4/32, not the other routes.

It helps to think through a few more examples. The list identifies other destination addresses and explains why the router matches a specific route in Example 24-9.

**172.16.10.4:** PC D. Matches all four highlighted routes; the longest prefix is /32, the route to router 172.16.10.4/32.

**172.16.10.1:** A different host in PC D's subnet, this destination matches the default route (0 prefix length) and the routes with the /23 and /26 prefix lengths. R1 uses the /26 route for subnet 172.16.10.0/26.

**172.16.10.100:** Per Table 24-5, this address resides in the range of addresses for the /23 route, and it matches the default route as always. R1 uses the route with /23 prefix length for subnet 172.16.10.0/23.

**172.16.12.1:** Matches only the default route with /0 prefix length.

### Using show ip route *address* to Find the Best Route

A second method to identify the route used by a router does not require you to use any subnetting math skills. Instead, use the **show ip route** *address* command, with the packet's destination IP address as the final parameter in the command. The router replies by listing its route when forwarding packets to that address.

For instance, Example 24-10 lists the output of the **show ip route 172.16.10.1** command on the same router used in Example 24-9. The first line of (highlighted) output lists the matched route: the route to 172.16.10.0/26. The rest of the output lists the details of that particular route, including the outgoing interface of GigabitEthernet0/0/1 and the next-hop router of 172.16.13.3.

**Example 24-10** **show ip route** *Command with Overlapping Routes*

```
R1# show ip route 172.16.10.1
Routing entry for 172.16.10.0/26
  Known via "ospf 1", distance 110, metric 3, type inter area
  Last update from 172.16.13.3 on GigabitEthernet0/0/1, 00:44:09 ago
  Routing Descriptor Blocks:
  * 172.16.13.3, from 3.3.3.3, 00:44:09 ago, via GigabitEthernet0/0/1
      Route metric is 3, traffic share count is 1
```

Certainly, if answering a lab question on the exam, use this command because it tells you what the router will choose without you doing the subnetting math.

### Interpreting the IP Routing Table

Here at the end of three consecutive book parts about IP and IP routing, before moving on to Part VII, "IP Version 6," this final topic reviews the most critical router command at the center of the discussion: the **show ip route** command. You have learned the various components of the command output through many examples. This final topic of the chapter pulls the concepts together in one place for easier reference and study.

Figure 24-8 shows the output of a sample **show ip route** command. The figure numbers various parts of the command output for easier reference, with Table 24-6 describing the output noted by each number.

Key Topic

```
                1                                  2        3
     10.0.0.0/8 is variably subnetted, 13 subnets, 5 masks
C       10.1.3.0/26 is directly connected, GigabitEthernet0/1
L       10.1.3.3/32 is directly connected, GigabitEthernet0/1
O       10.1.4.64/26 [110/65] via 10.2.2.10, 14:31:52, Serial0/1/0
        10.2.2.0/30 [110/128] via 10.2.2.5, 14:31:52, Serial0/0/1
4          5     6    7    8              9        10          11
```

**Figure 24-8** **show ip route** *Command Output Reference*

Key Topic

**Table 24-6** Descriptions of the **show ip route** Command Output (refer to Figure 24-8)

| Item | Idea | Value in the Figure | Description |
|---|---|---|---|
| 1 | Classful network | 10.0.0.0/8 | The routing table is organized by classful network. This line is the heading line for classful network 10.0.0.0; it lists the default mask for Class A networks (/8). |
| 2 | Number of subnets | 13 subnets | The number of routes for subnets of the classful network known to this router, from all sources, including local routes—the /32 routes that match each router interface IP address. |
| 3 | Number of masks | 5 masks | The number of different masks used in all routes known to this router inside this classful network. |
| 4 | Legend code | C, L, O | A short code that identifies the source of the routing information. *O* is for OSPF, *D* for EIGRP, *C* for Connected, *S* for static, and *L* for local. (See Example 24-11 for a sample of the legend.) |
| 5 | Prefix (Subnet ID) | 10.2.2.0 | The subnet number of this particular route. |
| 6 | Prefix length (Mask) | /30 | The prefix mask used with this subnet. |
| 7 | Administrative distance | 110 | If a router learns routes for the listed subnet from more than one source of routing information, the router uses the source with the lowest administrative distance (AD). |
| 8 | Metric | 128 | The metric for this route. |
| 9 | Next-hop router | 10.2.2.5 | For packets matching this route, the IP address of the next router to which the packet should be forwarded. |
| 10 | Timer | 14:31:52 | For OSPF and EIGRP routes, this is the time since the route was first learned. |
| 11 | Outgoing interface | Serial0/0/1 | For packets matching this route, the interface out which the packet should be forwarded. |

You should also have a good mastery of the most common codes found in the legend at the beginning of the output from the **show ip route** command. Example 24-11 closes the chapter with one final example, showing the legend but without any routes. Make an effort to commit the highlighted codes to memory.

**Example 24-11** show ip route—*Most Common Legend Codes*

```
R1# show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, m - OMP
       n - NAT, Ni - NAT inside, No - NAT outside, Nd - NAT DIA
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       H - NHRP, G - NHRP registered, g - NHRP registration summary
       o - ODR, P - periodic downloaded static route, l - LISP
       a - application route
       + - replicated route, % - next hop override, p - overrides from PfR
       & - replicated local route overrides by connected
! Lines omitted for brevity
```

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 24-7 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 24-7** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review command tables | | Book |
| Review memory tables | | Website |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 24-8** Key Topics for Chapter 24

| Key Topic Element | Description | Page Number |
|---|---|---|
| Table 24-2 | OSPF neighbor requirements | 611 |
| Table 24-3 | **show** commands to display facts for OSPF neighbor requirements | 612 |
| List | Reasons why routers can become OSPF neighbors but fail to exchange routing information | 618 |
| Table 24-4 | Default Cisco router administrative distance settings | 623 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 24-4 | Concept of how a router uses administrative distance to choose an EIGRP route over an OSPF route | 624 |
| Paragraph | Router logic when a packet's destination address matches multiple IP routes | 626 |
| Example 24-9 | Example of an IP routing table with overlapping IP routes | 627 |
| Figure 24-8 | The Cisco IP routing table field reference | 629 |
| Table 24-6 | Explanations for Figure 24-8's IP routing table | 629 |

## Key Terms You Should Know

administrative distance, equal-cost multipath (ECMP), equal-cost route, longest prefix match

## Command References

Tables 24-9 and 24-10 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 24-9** Chapter 24 Configuration Command Reference

| Command | Description |
|---|---|
| **router ospf** *process-id* | Global command that enters OSPF configuration mode for the listed process |
| **ip ospf** *process-id* **area** *area-number* | Interface subcommand to enable OSPF on the interface and to assign the interface to a specific OSPF area |
| **router-id** *id* | OSPF command that statically sets the router ID |
| **ip ospf hello-interval** *seconds* | Interface subcommand that sets the interval for periodic Hellos |
| **ip ospf dead-interval** *number* | Interface subcommand that sets the OSPF dead timer |
| [**no**] **shutdown** | An OSPF configuration mode command to disable (**shutdown**) or enable (**no shutdown**) the OSPF process |
| [**no**] **ip ospf shutdown** | An interface subcommand to disable or enable OSPF functions on the selected interface |
| **mtu** *size* | An interface subcommand to set the largest packet size (MTU) for all Layer 3 protocols enabled on the interface |
| **ip mtu** *size* | An interface subcommand to set the largest packet size (MTU) for IPv4 packets on the interface, overriding the setting of the **mtu** *size* subcommand |
| **ip ospf priority** *value* | Interface subcommand that sets the OSPF priority, used when electing a new DR or BDR |
| **ip ospf network** {**broadcast** \| **point-to-point**} | Interface subcommand used to set the OSPF network type on the interface |
| **maximum-paths** *number* | OSPF router subcommand that defines the maximum number of equal-cost multipath (ECMP) routes, learned by OSPF, to be added to the routing table at one time |

**Table 24-10** Chapter 24 **show** Command Reference

| Command | Description |
|---|---|
| **show ip protocols** | Shows routing protocol parameters and current timer values, including an effective copy of the routing protocols' **network** commands and a list of passive interfaces |
| **show ip ospf interface brief** | Lists the interfaces on which the OSPF protocol is enabled (based on the **network** commands), including passive interfaces |
| **show ip ospf interface** [*type number*] | Lists detailed OSPF settings for all interfaces, or the listed interface, including Hello and Dead timers and OSPF area |
| **show ip ospf neighbor** | Lists neighbors and current status with neighbors, per interface |
| **show ip ospf** | Lists a group of messages about the OSPF process itself, listing the OSPF Router ID in the first line |
| **show interfaces** | Lists a long set of messages, per interface, that lists configuration, state, and counter information |
| **show ip ospf database** | Displays the contents of the router's OSPF LSDB |
| **show ip route** | Lists all IPv4 routes |
| **show ip route ospf** | Lists the OSPF-learned IPv4 routes in the routing table |
| **show ip route** *address* | Lists details about the one route this router would match for packets destined to the listed address |
| **clear ip ospf process** | Resets the OSPF process, resetting all neighbor relationships and also causing the process to make a choice of OSPF RID |

*This page intentionally left blank*



# Part VI Review

Keep track of your part review progress with the checklist in Table P6-1. Details about each task follow the table.

**Table P6-1** Part VI Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Watch Video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book using the PTP software. See the section "How to View Only DIKTA Questions by Chapter or Part" in the Introduction to this book to learn how to make the PTP software show you DIKTA questions for this part only.

## Answer Part Review Questions

For this task, answer the Part Review questions for this part of the book using the PTP software. See the section "How to View Part Review Questions" in the Introduction to this book to learn how to make the PTP software show you DIKTA questions for this part only.

## Review Key Topics

Review all Key Topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Do Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Blog: Config Labs:** The author's blog includes a series of configuration-focused labs that you can do on paper or with Cisco Packet Tracer in about 15 minutes. To find them, open https://www.certskills.com and look under the Labs menu item.

**Other:** If using other lab tools, here are a few suggestions: make sure to experiment heavily with OSPF configuration and all the optional settings that impact OSPF neighbor compatibility.

## Watch Video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.



So far, this book has mostly ignored IP version 6 (IPv6). This part reverses the trend, collecting all the specific IPv6 topics into five chapters.

The chapters in Part VII walk you through the same topics discussed throughout this book for IPv4, often using IPv4 as a point of comparison. Certainly, many details differ when comparing IPv4 and IPv6. However, many core concepts about IP addressing, subnetting, routing, and routing protocols remain the same. The chapters in this part build on those foundational concepts, adding the specific details about how IPv6 forwards IPv6 packets from one host to another.

# Part VII

## IP Version 6

**Chapter 25:** Fundamentals of IP Version 6

**Chapter 26:** IPv6 Addressing and Subnetting

**Chapter 27:** Implementing IPv6 Addressing on Routers

**Chapter 28:** Implementing IPv6 Addressing on Hosts

**Chapter 29:** Implementing IPv6 Routing

**Part VII Review**



CHAPTER 25

# Fundamentals of IP Version 6

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.8 Configure and verify IPv6 addressing and prefix**

**1.9 Describe IPv6 address types**

IPv4 has been a solid and highly useful part of the growth of TCP/IP and the Internet. For most of the long history of the Internet, and for most corporate networks that use TCP/IP, IPv4 is the core protocol that defines addressing and routing. However, although IPv4 has many great qualities, it has some shortcomings, creating the need for a replacement protocol: IP version 6 (IPv6).

IPv6 defines the same general functions as IPv4, but with different methods of implementing those functions. For example, both IPv4 and IPv6 define addressing, the concepts of subnetting larger groups of addresses into smaller groups, headers used to create an IPv4 or IPv6 packet, and the rules for routing those packets. At the same time, IPv6 handles the details differently; for example, using a 128-bit IPv6 address rather than the 32-bit IPv4 address.

This chapter focuses on the core network layer functions of addressing and routing. The first section of this chapter looks at the big concepts, while the second section looks at the specifics of how to write and type IPv6 addresses.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 25-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Introduction to IPv6 | 1–2 |
| IPv6 Addressing Formats and Conventions | 3–6 |

1. Which of the following was a short-term solution to the IPv4 address exhaustion problem?
   a. IP version 6
   b. IP version 5

**c.** NAT/PAT

**d.** ARP

2. A router receives an Ethernet frame that holds an IPv6 packet. The router then decides to route the packet out an Ethernet WAN link. Which statement is true about how a router forwards an IPv6 packet?

   **a.** The router discards the received frame's Ethernet data-link header and trailer.

   **b.** The router makes the forwarding decision based on the packet's source IPv6 address.

   **c.** The router keeps the incoming frame's Ethernet header, encapsulating the entire frame inside a new IPv6 packet before sending it over the outgoing Ethernet WAN link.

   **d.** The router uses the IPv4 routing table when choosing where to forward the packet.

3. Which of the following is the shortest valid abbreviation for FE80:0000:0000:0000:0100:0000:0000:0123?

   **a.** FE80::100::123

   **b.** FE8::1::123

   **c.** FE80::100:0:0:0:123:4567

   **d.** FE80::100:0:0:123

4. Which of the following is the shortest valid abbreviation for 2000:0300:0040:0005:6000:0700:0080:0009?

   **a.** 2:3:4:5:6:7:8:9

   **b.** 2000:300:40:5:6000:700:80:9

   **c.** 2000:300:4:5:6000:700:8:9

   **d.** 2000:3:4:5:6:7:8:9

5. Which of the following is the unabbreviated version of IPv6 address 2001:DB8::200:28?

   **a.** 2001:0DB8:0000:0000:0000:0000:0200:0028

   **b.** 2001:0DB8::0200:0028

   **c.** 2001:0DB8:0:0:0:0:0200:0028

   **d.** 2001:0DB8:0000:0000:0000:0000:200:0028

6. Which of the following is the correct abbreviated subnet prefix for address 2000:0000:0000:0005:6000:0700:0080:0009, assuming a mask of /64?

   **a.** 2000::5::/64

   **b.** 2000::5:0:0:0:0/64

   **c.** 2000:0:0:5::/64

   **d.** 2000:0:0:5:0:0:0:0/64

# Foundation Topics

## Introduction to IPv6

**IP version 6 (IPv6)** serves as the replacement protocol for IP version 4 (IPv4). To do so, the network can perform a slow migration that uses both, with IPv6 overcoming some of the issues that drove the need for a protocol to replace IPv4.

Unfortunately, that introductory statement creates more questions than it answers. Why does IPv4 need to be replaced? If IPv4 needs to be replaced, when will that happen—and will it happen quickly? What exactly happens when a company or the Internet replaces IPv4 with IPv6? And the list goes on.

While this introductory chapter cannot get into every detail of why IPv4 needs to eventually be replaced by IPv6, the clearest and most obvious reason for migrating TCP/IP networks to use IPv6 is growth. IPv4 uses a 32-bit address, which totals to a few billion addresses. Interestingly, that seemingly large number of addresses is too small. IPv6 increases the address to 128 bits in length. For perspective, IPv6 supplies more than 10,000,000,000,000,000,000,000,000,000,000 times as many addresses as IPv4. IPv4 supplies just under $10^{10}$ addresses, while IPv6 supplies just under $10^{38}$ addresses.

The fact that IPv6 uses a different size address field, with some different addressing rules, means that many other protocols and functions change as well. For example, IPv4 routing—in other words, the packet-forwarding process—relies on an understanding of IPv4 addresses. To support IPv6 routing, routers must understand IPv6 addresses and routing. To dynamically learn routes for IPv6 subnets, routing protocols must support these different IPv6 addressing rules, including rules about how IPv6 creates subnets. As a result, the migration from IPv4 to IPv6 is much more than changing one protocol (IP), but it impacts many protocols.

This first section of the chapter discusses some of the reasons for the change from IPv4 to IPv6, along with the protocols that must change as a result.

### The Historical Reasons for IPv6

In the last 50+ years, the Internet has gone from its infancy to being a huge influence in the world. It first grew through research at universities, from the ARPANET beginnings of the Internet in the late 1960s into the 1970s. The Internet kept growing fast in the 1980s, with the Internet's fast growth still primarily driven by research and the universities that joined in that research.

By the early 1990s, the Internet allowed commercial use, driving Internet growth even higher. Eventually, fixed Internet access from home became common, followed by the pervasive use of the Internet from mobile devices like smartphones. Now the Internet of Things (IoT) fuels Internet growth, adding all kinds of devices throughout industry that can communicate through an IP network. Figure 25-1 shows some of these major milestones with general dates.

---

Answers to the "Do I Know This Already?" quiz:

**1** C **2** A **3** D **4** B **5** A **6** C

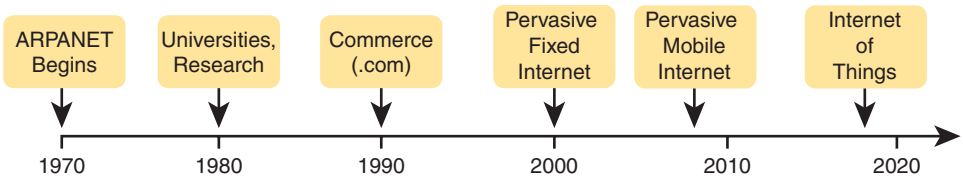



**Figure 25-1** *Some Major Events in the Growth of the Internet*



The incredible growth of the Internet over a fairly long time created a big problem for public IPv4 addresses: the world was running out of addresses. As one example milestone, in 2011, the Internet Assigned Numbers Authority (IANA) allocated the final five /8 address blocks (the same size as a Class A network) to each of the five Regional Internet Registries (RIR). At that point, IANA had no more public IPv4 addresses to allocate to RIRs. The RIRs could no longer receive new allocations of public addresses from IANA but could continue to assign blocks from their remaining public address space.

In the 2010s, the five RIRs eventually assigned all of their public address space. For example, in late 2015, ARIN (North America) announced that it had exhausted its supply. All the RIRs have plans for how to deal with IPv4 address exhaustion, with all either being out of IPv4 address space or using a maintenance plan to reclaim unused IPv4 addresses for reassignment.

These events are significant in that the day has finally come in which new companies can attempt to connect to the Internet, but they can no longer simply use IPv4, ignoring IPv6. Their only option will be IPv6 because IPv4 has no public addresses left.

**NOTE** You can track ARIN's progress through this interesting transition in the history of the Internet at its IPv4 address depletion site: https://www.arin.net/resources/guide/ipv4/. You can also see a summary report at http://ipv4.potaroo.net.

Even though the media has rightfully made a big deal about running out of IPv4 addresses, those who care about the Internet knew about this potential problem since the late 1980s. The problem, generally called the **IPv4 address exhaustion** problem, could literally have caused the huge growth of the Internet in the 1990s to have come to a screeching halt! Something had to be done.

The IETF came up with several short-term solutions to make IPv4 addresses last longer, and one long-term solution: IPv6. However, several other tools like Network Address Translation (NAT) and classless interdomain routing (CIDR) helped extend IPv4's life another couple of decades. IPv6 creates a more permanent and long-lasting solution, replacing IPv4, with a new IPv6 header and new IPv6 addresses. The address size supports a huge number of addresses, solving the address shortage problem for generations (we hope). Figure 25-2 shows some of the major IPv4 address exhaustion timing events.

The rest of this first section examines IPv6, comparing it to IPv4, focusing on the common features of the two protocols. In particular, this section compares the protocols (including addresses), routing, routing protocols, and miscellaneous other related topics.

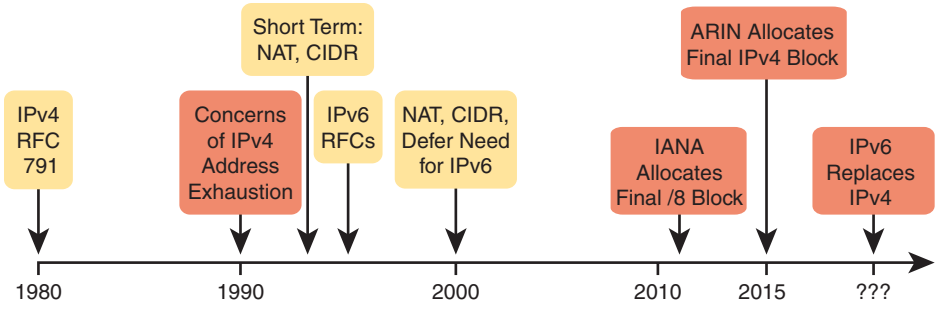



**Figure 25-2** *Timeline for IPv4 Address Exhaustion and Short-/Long-Term Solutions*

**NOTE** You might wonder why the next version of IP is not called IP version 5. There was an earlier effort to create a new version of IP, and it was numbered version 5. IPv5 did not progress to the standards stage. However, to prevent any issues, because version 5 had been used in some documents, the next effort to update IP was numbered as version 6.

## The IPv6 Protocols

The primary purpose of the core IPv6 protocol mirrors the same purpose of the IPv4 protocol. That core IPv6 protocol, as defined in RFC 8200, defines a packet concept, addresses for those packets, and the role of hosts and routers. These rules allow the devices to forward packets sourced by hosts, through multiple routers, so that they arrive at the correct destination host. (IPv4 defines those same concepts for IPv4 back in RFC 791.)

However, because IPv6 impacts so many other functions in a TCP/IP network, many more RFCs must define details of IPv6. Some other RFCs define how to migrate from IPv4 to IPv6. Others define new versions of familiar protocols or replace old protocols with new ones. For example:

**Older OSPF Version 2 Upgraded to OSPF Version 3:** The older Open Shortest Path First (OSPF) version 2 works for IPv4, but not for IPv6, so a newer version, **OSPF version 3 (OSPFv3)**, was created to support IPv6. (Note: OSPFv3 was later upgraded to support advertising both IPv4 and IPv6 routes.)

**ICMP Upgraded to ICMP Version 6:** Internet Control Message Protocol (ICMP) worked well with IPv4 but needed to be changed to support IPv6. The new name is ICMPv6.

**ARP Replaced by Neighbor Discovery Protocol:** For IPv4, Address Resolution Protocol (ARP) discovers the MAC address used by neighbors. IPv6 replaces ARP with a more general Neighbor Discovery Protocol (NDP).

**NOTE** A huge number of Internet RFCs define IPv6. A recent search for "IPv6" at https://www.rfc-editor.org showed over 550 such RFCs.

Although the term *IPv6*, when used broadly, includes many protocols, the one specific protocol called IPv6 defines the new 128-bit IPv6 address. Of course, writing these addresses in binary would be a problem—they probably would not even fit on the width of a piece of

paper! IPv6 defines a shorter hexadecimal format, requiring at most 32 hexadecimal digits (one hex digit per 4 bits), with methods to abbreviate the hexadecimal addresses as well.

For example, all of the following are IPv6 addresses, each with 32 or fewer hex digits.

```
2345:1111:2222:3333:4444:5555:6666:AAAA
2000:1:2:3:4:5:6:A
FE80::1
```

The upcoming section "IPv6 Addressing Formats and Conventions" discusses the specifics of how to represent IPv6 addresses, including how to legally abbreviate the hex address values.



Like IPv4, IPv6 defines a header, with places to hold both the source and destination address fields. Compared to IPv4, the IPv6 header does make some other changes besides simply making the address fields larger. However, even though the IPv6 header is larger than an IPv4 header, the IPv6 header is actually simpler (on purpose). Figure 25-3 shows the required 40-byte part of the IPv6 header.

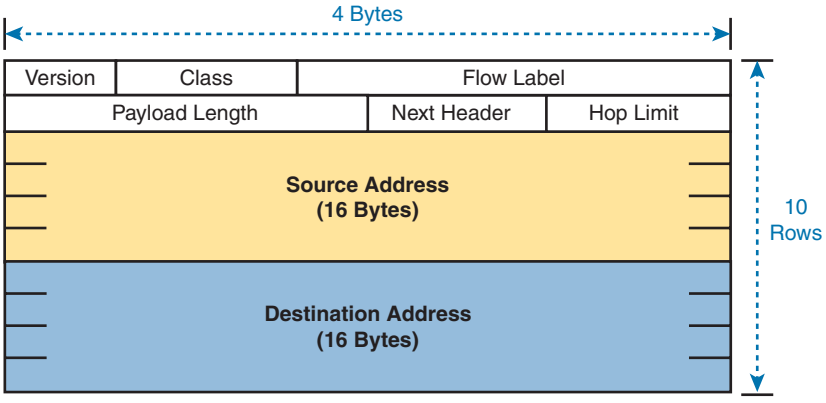



**Figure 25-3** *IPv6 Header*

## IPv6 Routing

As with many functions of IPv6, IPv6 routing looks just like IPv4 routing from a general perspective, with the differences being clear only once you look at the specifics. Keeping the discussion general for now, IPv6 uses these ideas the same way as IPv4:

Key Topic

- To be able to build and send IPv6 packets out an interface, end-user devices need an IPv6 address on that interface.
- End-user hosts need to know the IPv6 address of a default router, to which the host sends IPv6 packets if the destination host is in a different subnet.
- IPv6 routers de-encapsulate and re-encapsulate each IPv6 packet when routing the packet.
- IPv6 routers make routing decisions by comparing the IPv6 packet's destination address to the router's IPv6 routing table; the matched route lists directions of where to send the IPv6 packet next.

**NOTE** You could take the preceding list and replace every instance of IPv6 with IPv4, and all the statements would be true of IPv4 as well.

The next few figures show the concepts with an example. First, Figure 25-4 shows a few settings on a host. The host (PC1) has an address of 2345::1. PC1 also knows its default gateway of 2345::2. (Both values are valid abbreviations for real IPv6 addresses.) To send an IPv6 packet to host PC2, on another IPv6 subnet, PC1 creates an IPv6 packet and sends it to R1, PC1's default gateway.

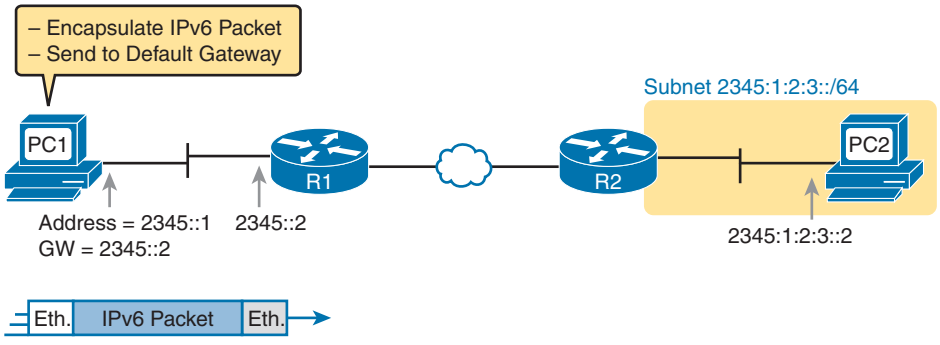



**Figure 25-4** *IPv6 Host Building and Sending an IPv6 Packet*

The router (R1) has many small tasks to do when forwarding this IPv6 packet, but for now, focus on the work R1 does related to encapsulation. As seen in Step 1 of Figure 25-5, R1 receives the incoming data-link frame and extracts (de-encapsulates) the IPv6 packet from inside the frame, discarding the original data-link header and trailer. At Step 2, once R1 knows to forward the IPv6 packet to R2, R1 adds a correct outgoing data-link header and trailer to the IPv6 packet, encapsulating the IPv6 packet.

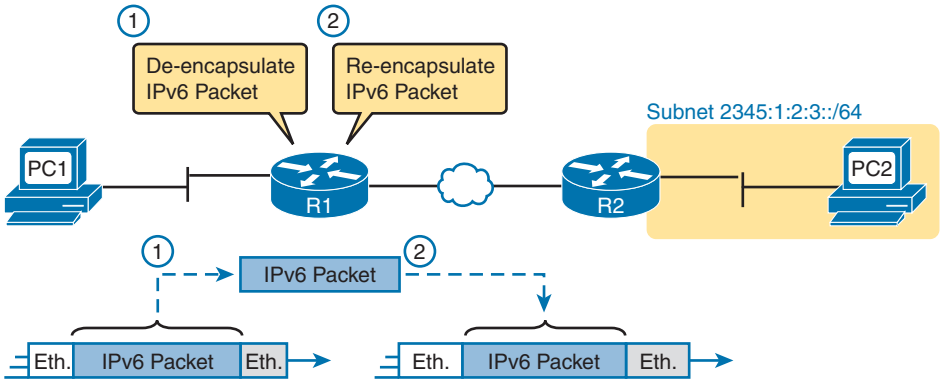



**Figure 25-5** *IPv6 Router Performing Routine Encapsulation Tasks When Routing IPv6*

When a router like R1 de-encapsulates the packet from the data-link frame, it must also decide what type of packet sits inside the frame. To do so, the router must look at a protocol type field in the data-link header, which identifies the type of packet inside the data-link frame. Today, most data-link frames carry either an IPv4 packet or an IPv6 packet.

To route an IPv6 packet, a router must use its IPv6 routing table instead of the IPv4 routing table. The router must look at the packet's destination IPv6 address and compare that address to the router's current IPv6 routing table. The router uses the forwarding instructions in the matched IPv6 route to forward the IPv6 packet. Figure 25-6 shows the overall process.

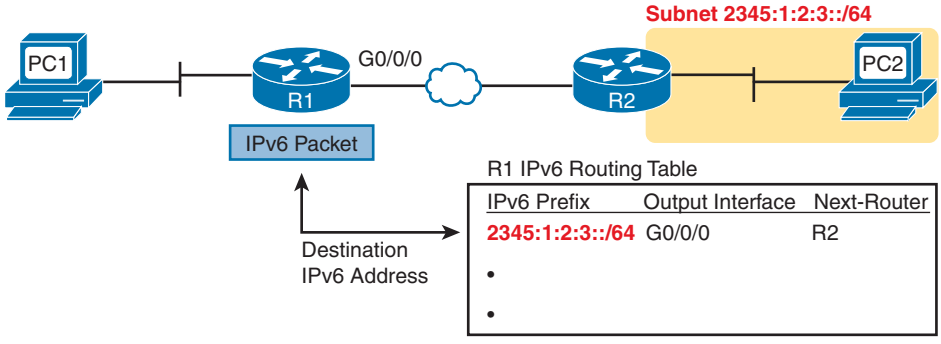



**Figure 25-6** *Comparing an IPv6 Packet to R1's IPv6 Routing Table*

Note that again, the process works like IPv4, except that the IPv6 packet lists IPv6 addresses, and the IPv6 routing table lists routing information for IPv6 subnets (called subnet *prefixes*).

Finally, in most enterprise networks, the routers will route both IPv4 and IPv6 packets at the same time. That is, your company will not decide to adopt IPv6, and then late one weekend night turn off all IPv4 and enable IPv6 on every device. Instead, IPv6 allows for a slow migration, during which some or all routers forward both IPv4 and IPv6 packets. (The migration strategy of running both IPv4 and IPv6 is called *dual stack*.) All you have to do is configure the router to route IPv6 packets, in addition to the existing configuration for routing IPv4 packets.

## IPv6 Routing Protocols

IPv6 routers need to learn routes for all the possible IPv6 subnet prefixes. Just like with IPv4, IPv6 routers use routing protocols, with familiar names, and generally speaking, with familiar functions.

None of the IPv4 routing protocols could be used to advertise IPv6 routes originally. They all required some kind of update to add messages, protocols, and rules to support IPv6. Over time, Routing Information Protocol (RIP), Open Shortest Path First (OSPF), Enhanced Interior Gateway Routing Protocol (EIGRP), and Border Gateway Protocol (BGP) were all updated to support IPv6. Table 25-2 lists the names of these routing protocols, with a few comments.

**Table 25-2** IPv6 Routing Protocols

| Routing Protocol | Defined By | Notes |
|---|---|---|
| RIPng (RIP next generation) | RFC | The "next generation" is a reference to a TV series, *Star Trek: The Next Generation*. |
| *OSPFv3* (OSPF version 3) | RFC | The OSPF you have worked with for IPv4 is actually OSPF version 2, so the new version for IPv6 is OSPFv3. |

| Routing Protocol | Defined By | Notes |
|---|---|---|
| **EIGRPv6 (EIGRP for IPv6)** | Cisco | Cisco owns the rights to the EIGRP protocol, but Cisco also now publishes EIGRP as an informational RFC. |
| MP BGP-4 (Multiprotocol BGP version 4) | RFC | BGP version 4 was created to be highly extendable; IPv6 support was added to BGP version 4 through one such enhancement, MP BGP-4. |

In addition, these routing protocols also follow the same interior gateway protocol (IGP) and exterior gateway protocol (EGP) conventions as their IPv4 cousins. RIPng, EIGRPv6, and OSPFv3 act as interior gateway protocols, advertising IPv6 routes inside an enterprise.

As you can see from this introduction, IPv6 uses many of the same big ideas as IPv4. Both define headers with a source and destination address. Both define the routing of packets, with the routing process discarding old data-link headers and trailers when forwarding the packets. And routers use the same general process to make a routing decision, comparing the packet's destination IP address to the routing table.

The big differences between IPv4 and IPv6 revolve around the bigger IPv6 addresses. The next topic begins looking at the specifics of these IPv6 addresses.

# IPv6 Addressing Formats and Conventions

The CCNA exam requires some fundamental skills in working with IPv4 addresses. For example, you need to be able to interpret IPv4 addresses, like 172.21.73.14. You need to be able to work with prefix-style masks, like /25, and interpret what that means when used with a particular IPv4 address. And you need to be able to take an address and mask, like 172.21.73.14/25, and find the subnet ID.

This second major section of this chapter discusses these same ideas for IPv6 addresses. In particular, this section looks at

- How to write and interpret unabbreviated 32-digit IPv6 addresses
- How to abbreviate IPv6 addresses and how to interpret abbreviated addresses
- How to interpret the IPv6 prefix length (subnet mask)
- How to find the IPv6 subnet prefix ID based on an address and prefix length mask

The biggest challenge with these tasks lies in the sheer size of the numbers. Thankfully, the math to find the subnet ID—often a challenge for IPv4—is easier for IPv6, at least to the depth discussed in this book.

## Representing Full (Unabbreviated) IPv6 Addresses

IPv6 uses a convenient hexadecimal (hex) format for addresses. To make it more readable, IPv6 uses a format with eight sets of four hex digits, with each set of four digits separated by a colon. For example:

```
2340:1111:AAAA:0001:1234:5678:9ABC:1234
```

**NOTE** For convenience, this book uses the term **quartet** for one set of four hex digits, with eight quartets in each IPv6 address. Note that the IPv6 RFCs do not define an equivalent term.

IPv6 addresses also have a binary format, but thankfully, most of the time you do not need to look at the binary version of the addresses. However, in those cases, converting from hex to binary is relatively easy. Just change each hex digit to the equivalent 4-bit value listed in Table 25-3.



**Table 25-3** Hexadecimal/Binary Conversion Chart

| Hex | Binary | Hex | Binary |
|---|---|---|---|
| 0 | 0000 | 8 | 1000 |
| 1 | 0001 | 9 | 1001 |
| 2 | 0010 | A | 1010 |
| 3 | 0011 | B | 1011 |
| 4 | 0100 | C | 1100 |
| 5 | 0101 | D | 1101 |
| 6 | 0110 | E | 1110 |
| 7 | 0111 | F | 1111 |

## Abbreviating and Expanding IPv6 Addresses

IPv6 also defines ways to abbreviate or shorten how you write or type an IPv6 address. Why? Although using a 32-digit hex number works much better than working with a 128-bit binary number, 32 hex digits are still a lot of digits to remember, recognize in command output, and type on a command line. The IPv6 address abbreviation rules let you shorten these numbers.

Computers and routers use the shortest abbreviation, even if you type all 32 hex digits of the address. So even if you would prefer to use the longer unabbreviated version of the IPv6 address, you need to be ready to interpret the meaning of an abbreviated IPv6 address as listed by a router or host. This section first looks at abbreviating addresses and then at expanding addresses.

### Abbreviating IPv6 Addresses

Two basic rules let you, or any computer, shorten or abbreviate an IPv6 address:

Key Topic

1. Inside each quartet of four hex digits, remove up to three leading 0s in the three positions on the left. (Note: At this step, a quartet of 0000 will leave a single 0.)
2. Replace the longest set of two or more consecutive 0000 quartets with a double colon. The abbreviation :: means "two or more quartets of all 0s." However, you can use this replacement only once inside a single address—for the longest set of consecutive 0000 quartets (or the first such set if a tie).

For example, consider the following IPv6 address. The bold digits represent digits in which the address could be abbreviated.

2100**:0000:0000:**0001**:0000:0000:0000:00**56

Applying the first rule, you would look at all eight quartets independently. In each, remove all the leading 0s. Note that five of the quartets have four 0s, so for these, remove only three leading binary 0s, leaving the following value:

```
2100:0:0:1:0:0:0:56
```

While this abbreviation is valid, the address can be abbreviated further, using the second rule. In this case, two instances exist where more than one quartet in a row has only a 0. Pick the longest such sequence, and replace it with ::, giving you the shortest legal abbreviation:

```
2100:0:0:1::56
```

While 2100:0:0:1::56 is indeed the shortest abbreviation, this example happens to make it easier to see the two most common mistakes when abbreviating IPv6 addresses. First, never remove trailing 0s in a quartet (0s on the right side of the quartet). In this case, the first quartet of 2100 cannot be shortened at all because the two 0s trail. So, the following address, which begins now with only FE in the first quartet, is not a correct abbreviation of the original IPv6 address:

```
21:0:0:1::56
```

The second common mistake is to replace both sets of 0000 quartets with a double colon. For example, the following abbreviation would be incorrect for the original IPv6 address listed in this topic:

```
2100::1::56
```

The reason this abbreviation is incorrect is that now you do not know how many quartets of all 0s to substitute into each :: to find the original unabbreviated address.

### Expanding Abbreviated IPv6 Addresses

To expand an IPv6 address back into its full unabbreviated 32-digit number, use two similar rules. The rules basically reverse the logic of the previous two rules:

Key Topic

1. In each quartet, add leading 0s as needed until the quartet has four hex digits.
2. If a double colon (::) exists, count the quartets currently shown; the total should be less than 8. Replace the :: with multiple quartets of 0000 so that eight total quartets exist.

The best way to get comfortable with these addresses and abbreviations is to do some yourself. Table 25-4 lists some practice problems, with the full 32-digit IPv6 address on the left and the best abbreviation on the right. The table gives you either the expanded or abbreviated address, and you need to supply the opposite value. The answers sit at the end of the chapter, in the section "Answers to Earlier Practice Problems."

**Table 25-4** IPv6 Address Abbreviation and Expansion Practice

| Full | Abbreviation |
|---|---|
| 2340:0000:0010:0100:1000:ABCD:0101:1010 | |
| | 30A0:ABCD:EF12:3456:ABC:B0B0:9999:9009 |
| 2222:3333:4444:5555:0000:0000:6060:0707 | |
| | 3210:: |
| 210F:0000:0000:0000:CCCC:0000:0000:000D | |

| Full | Abbreviation |
|---|---|
| | 34BA:B:B::20 |
| FE80:0000:0000:0000:DEAD:BEFF:FEEF:CAFE | |
| | FE80::FACE:BAFF:FEBE:CAFE |



## Representing the Prefix Length of an Address

IPv6 uses a mask concept, called the **prefix length**, similar to IPv4 subnet masks. Similar to the IPv4 prefix-style mask, the IPv6 prefix length is written as a /, followed by a decimal number (there is no dotted-decimal equivalent mask for IPv6). The prefix length defines how many bits on the left side of an IPv6 address are the same value for all addresses within that subnet prefix. If you think of the ideas generically as prefix/length, all addresses in the subnet prefix begin with the same value in number of initial bits as defined by the length.

When writing an IPv6 address and prefix length in documentation, you can choose to leave a space before the /, or not, for readability. However, commands on Cisco devices typically do not allow spaces before or after the /.

```
2222:1111:0:1:A:B:C:D/64
2222:1111:0:1:A:B:C:D /64
```

Finally, note that the prefix length is a number of bits, so with IPv6, the legal value range is from 0 through 128, inclusive.

## Calculating the IPv6 Subnet Prefix (Subnet ID)

With IPv4, you can take an IP address and the associated subnet mask, and calculate the subnet ID. With IPv6 subnetting, you can take an IPv6 address and the associated prefix length, and calculate the IPv6 equivalent of the subnet ID: an *IPv6* **subnet prefix**.

Like with different IPv4 subnet masks, some IPv6 prefix lengths make for an easy math problem to find the IPv6 subnet prefix, while some prefix lengths make the math more difficult. This section looks at the easier cases, mainly because the size of the IPv6 address space lets us all choose to use IPv6 prefix lengths that make the math much easier.

## Finding the IPv6 Subnet Prefix

In IPv6, a subnet prefix represents a group of IPv6 addresses. For now, this section focuses on the math, and only the math, for finding the number that represents that subnet prefix. Chapter 26, "IPv6 Addressing and Subnetting," then starts putting more meaning behind the actual numbers.

Each IPv6 subnet prefix, or subnet if you prefer, has a number that represents the group. Many people just call it a subnet number or subnet ID, using the same terms as IPv4. As with IPv4, you can start with an IPv6 address and prefix length, and find the subnet prefix, with the same general rules that you use in IPv4. If the prefix length is /P, use these rules:

Key Topic

1. Copy the first P bits.
2. Change the rest of the bits to 0.

When using a prefix length that happens to be a multiple of 4, you do not have to think in terms of bits, but in terms of hex digits. A prefix length that is a multiple of 4 means that

each hex digit is either copied or changed to hex 0. For example, a /44 prefix length implies 11 hex digits, a /48 prefix length implies 12 hex digits, and so on. Just for completeness, if the prefix length is indeed a multiple of 4, the process becomes

1. Identify the number of hex digits in the subnet prefix by dividing the prefix length (which is in bits) by 4.
2. Copy the hex digits determined to be in the subnet prefix per the first step.
3. Change the rest of the hex digits to 0.

Figure 25-7 shows an example, with a prefix length of 64. In this case, Step 1 looks at the /64 prefix length and calculates that the subnet prefix has 16 hex digits. Step 2 copies the first 16 digits of the IPv6 address, while Step 3 records hex 0s for the rest of the digits.

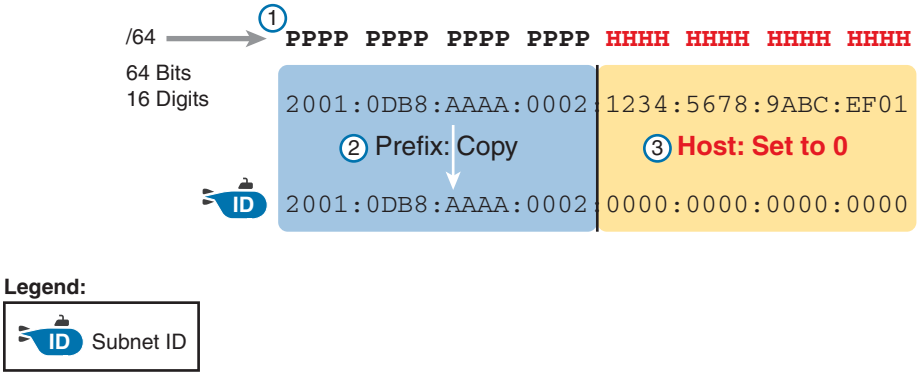



**Figure 25-7** *Creating the IPv6 Subnet Prefix from an Address/Length*

After you find the IPv6 subnet prefix, you should also be ready to abbreviate the IPv6 subnet prefix using the same rules you use to abbreviate IPv6 addresses. However, you should pay extra attention to the end of the subnet prefix because it often has several octets of all 0 values. As a result, the abbreviation typically ends with two colons (::).

For example, consider the following IPv6 address that is assigned to a host on a LAN:

```
2000:1234:5678:9ABC:1234:5678:9ABC:1111/64
```

This example shows an IPv6 address that itself cannot be abbreviated. After you calculate the subnet prefix by zeroing out the last 64 bits (16 digits) of the address, you find the following subnet prefix value:

```
2000:1234:5678:9ABC:0000:0000:0000:0000/64
```

This value can be abbreviated, with four quartets of all 0s at the end, as follows:

```
2000:1234:5678:9ABC::/64
```

To get better at the math, take some time to work through finding the subnet prefix for several practice problems, as listed in Table 25-5. The answers sit at the end of the chapter, in the section "Answers to Earlier Practice Problems."

**Table 25-5** Finding the IPv6 Subnet Prefix from an Address/Length Value

| Address/Length | Subnet Prefix |
|---|---|
| 2340:0:10:100:1000:ABCD:101:1010/64 | |
| 30A0:ABCD:EF12:3456:ABC:B0B0:9999:9009/64 | |
| 2222:3333:4444:5555::6060:707/64 | |
| 3210::ABCD:101:1010/64 | |
| 210F::CCCC:B0B0:9999:9009/64 | |
| 34BA:B:B:0:5555:0:6060:707/64 | |
| 3124::DEAD:CAFE:FF:FE00:1/64 | |
| 2BCD::FACE:BEFF:FEBE:CAFE/64 | |



## Working with More-Difficult IPv6 Prefix Lengths

In Chapter 26, you will read more about IPv6 subnetting—a much simpler topic to learn as compared with IPv4 subnetting. One reason why IPv6 subnetting happens to be simple is that several RFCs, including RFC 4291, "IPv6 Addressing Architecture," recommends that all deployed subnets use a /64 prefix length.

You will also on occasion need to work with prefix lengths shorter than /64 when working through your enterprise's subnetting plan. To get ready for that, this last topic of the chapter provides some practice with other shorter prefix lengths, using the easier cases of prefix lengths that are multiples of 4.

For example, consider the following IPv6 address and prefix length:

```
2000:1234:5678:9ABC:1234:5678:9ABC:1111/56
```

Because this example uses a /56 prefix length, the subnet prefix includes the first 56 bits, or first 14 complete hex digits, of the address. The rest of the hex digits will be 0, resulting in the following subnet prefix:

```
2000:1234:5678:9A00:0000:0000:0000:0000/56
```

This value can be abbreviated, with four quartets of all 0s at the end, as follows:

```
2000:1234:5678:9A00::/56
```

This example shows an easy place to make a mistake. Sometimes, people look at the /56 and think of that as the first 14 hex digits, which is correct. However, they then copy the first 14 hex digits and add a double colon, showing the following:

```
2000:1234:5678:9A::/56
```

This abbreviation is not correct because it removes the trailing "00" at the end of the fourth quartet. If you later expanded this incorrect abbreviated value, it would begin with 2000:1234:5678:009A, not 2000:1234:5678:9A00. So, be careful when abbreviating when the boundary is not at the edge of a quartet.

Once again, some extra practice can help. Table 25-6 uses examples that have a prefix length that is a multiple of 4, but is not on a quartet boundary, just to get some extra practice. The answers sit at the end of the chapter, in the section "Answers to Earlier Practice Problems."

**Table 25-6** Finding the IPv6 Subnet Prefix from an Address/Length Value

| Address/Length | Subnet Prefix |
|---|---|
| 34BA:B:B:0:5555:0:6060:707/48 | |
| 3124:1:20:DEAD:CAFE:FF:FE00:1/48 | |
| 2BCD::FACE:BEFF:FEBE:CAFE/48 | |
| 3FED:F:E0:D00:FACE:BAFF:FE00:0/48 | |
| 210F:A:B:C:CCCC:B0B0:9999:9009/40 | |
| 34BA:B:B:0:5555:0:6060:707/36 | |
| 3124::DEAD:CAFE:FF:FE00:1/60 | |
| 2BCD::FACE:1:BEFF:FEBE:CAFE/56 | |

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 25-7 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 25-7** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Review memory table | | Book, website |

## Review All the Key Topics

Key Topic

**Table 25-8** Key Topics for Chapter 25

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Similarities between IPv4 and IPv6 | 643 |
| List | Rules for abbreviating IPv6 addresses | 647 |
| List | Rules for expanding an abbreviated IPv6 address | 648 |
| List | Process steps to find an IPv6 prefix, based on the IPv6 address and prefix length | 649 |

## Key Terms You Should Know

EIGRP version 6 (EIGRPv6), IP version 6 (IPv6), IPv4 address exhaustion, OSPF version 3 (OSPFv3), prefix length, quartet, subnet prefix

## Additional Practice for This Chapter's Processes

For additional practice with IPv6 abbreviations, you may do the same set of practice problems based on Appendix H, "Practice for Chapter 25: Fundamentals of IP Version 6." You have two options to use:

**PDF:** Navigate to the companion website and open the PDF for Appendix H.

**Application:** Navigate to the companion website and use these applications:

"Practice Exercise: Abbreviating and Expanding Addresses"

"Practice Exercise: Calculating the IPv6 Subnet Prefix"

"Practice Exercise: Calculating the IPv6 Subnet Prefix Round 2"



## Answers to Earlier Practice Problems

This chapter includes practice problems spread around different locations in the chapter. The answers are located in Tables 25-9, 25-10, and 25-11.

**Table 25-9** Answers to Questions in the Earlier Table 25-4

| Full | Abbreviation |
|---|---|
| 2340:0000:0010:0100:1000:ABCD:0101:1010 | 2340:0:10:100:1000:ABCD:101:1010 |
| 30A0:ABCD:EF12:3456:0ABC:B0B0:9999:9009 | 30A0:ABCD:EF12:3456:ABC:B0B0:9999:9009 |
| 2222:3333:4444:5555:0000:0000:6060:0707 | 2222:3333:4444:5555::6060:707 |
| 3210:0000:0000:0000:0000:0000:0000:0000 | 3210:: |
| 210F:0000:0000:0000:CCCC:0000:0000:000D | 210F::CCCC:0:0:D |
| 34BA:000B:000B:0000:0000:0000:0000:0020 | 34BA:B:B::20 |
| FE80:0000:0000:0000:DEAD:BEFF:FEEF:CAFE | FE80::DEAD:BEFF:FEEF:CAFE |
| FE80:0000:0000:0000:FACE:BAFF:FEBE:CAFE | FE80::FACE:BAFF:FEBE:CAFE |

**Table 25-10** Answers to Questions in the Earlier Table 25-5

| Address/Length | Subnet Prefix |
|---|---|
| 2340:0:10:100:1000:ABCD:101:1010/64 | 2340:0:10:100::/64 |
| 30A0:ABCD:EF12:3456:ABC:B0B0:9999:9009/64 | 30A0:ABCD:EF12:3456::/64 |
| 2222:3333:4444:5555::6060:707/64 | 2222:3333:4444:5555::/64 |
| 3210::ABCD:101:1010/64 | 3210::/64 |
| 210F::CCCC:B0B0:9999:9009/64 | 210F::/64 |
| 34BA:B:B:0:5555:0:6060:707/64 | 34BA:B:B::/64 |
| 3124::DEAD:CAFE:FF:FE00:1/64 | 3124:0:0:DEAD::/64 |
| 2BCD::FACE:BEFF:FEBE:CAFE/64 | 2BCD::/64 |

**Table 25-11** Answers to Questions in the Earlier Table 25-6

| Address/Length | Subnet Prefix |
|---|---|
| 34BA:B:B:0:5555:0:6060:707/48 | 34BA:B:B::/48 |
| 3124:1:20:DEAD:CAFE:FF:FE00:1/48 | 3124:1:20::/48 |
| 2BCD::FACE:BEFF:FEBE:CAFE/48 | 2BCD::/48 |
| 3FED:F:E0:D00:FACE:BAFF:FE00:0/48 | 3FED:F:E0::/48 |
| 210F:A:B:C:CCCC:B0B0:9999:9009/40 | 210F:A::/40 |
| 34BA:B:B:0:5555:0:6060:707/36 | 34BA:B::/36 |
| 3124::DEAD:CAFE:FF:FE00:1/60 | 3124:0:0:DEA0::/60 |
| 2BCD::FACE:1:BEFF:FEBE:CAFE/56 | 2BCD:0:0:FA00::/56 |



CHAPTER 26

# IPv6 Addressing and Subnetting

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.8 Configure and verify IPv6 addressing and prefix**

**1.9 Describe IPv6 address types**

**1.9.a Unicast (global, unique local, and link local)**

The Internet Assigned Numbers Authority (IANA) assigns public IPv6 addresses using a process much like IPv4 with CIDR blocks. IANA first defines some IPv6 address space as unicast and other parts as multicast. The majority of the unicast address space serves as public addresses, with small parts of the address space reserved for particular purposes. IANA with the RIRs assign public address blocks as defined with a prefix length of any valid size, commonly in sizes that use /32 to /48 prefix lengths.

This chapter has two major sections. The first examines **global unicast addresses**, which serve as public IPv6 addresses. This section also discusses subnetting in IPv6, which happens to be much simpler than with IPv4. The second major section looks at **unique local addresses**, which serve as private IPv6 addresses.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 26-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Global Unicast Addressing Concepts | 1–4 |
| Unique Local Unicast Addresses | 5 |

1. Which of the following IPv6 addresses appears to be a unique local unicast address based on its first few hex digits?
   a. 3123:1:3:5::1
   b. FE80::1234:56FF:FE78:9ABC
   c. FDAD::1
   d. FF00::5

2. Which of the following IPv6 addresses appears to be a global unicast address, based on its first few hex digits?
   a. 3123:1:3:5::1
   b. FE80::1234:56FF:FE78:9ABC
   c. FDAD::1
   d. FF00::5
3. When subnetting an IPv6 address block, an engineer shows a drawing that breaks the address structure into three pieces. Comparing this concept to a three-part IPv4 address structure, which part of the IPv6 address structure is most like the IPv4 network part of the address?
   a. Subnet ID
   b. Interface ID
   c. Network ID
   d. Global routing prefix
   e. Subnet router anycast
4. When subnetting an IPv6 address block, an engineer shows a drawing that breaks the address structure into three pieces. Assuming that all subnets use the same /64 prefix length, which of the following answers lists the field's name on the far right side of the address?
   a. Subnet ID
   b. Interface ID
   c. Network ID
   d. Global routing prefix
   e. Subnet router anycast
5. For the IPv6 address FD00:1234:5678:9ABC:DEF1:2345:6789:ABCD, which part of the address is considered the global ID of the unique local address?
   a. None; this address has no global ID.
   b. 00:1234:5678:9ABC
   c. DEF1:2345:6789:ABCD
   d. 00:1234:5678
   e. FD00

# Foundation Topics

## Global Unicast Addressing Concepts

This first major section of the chapter focuses on one type of unicast IPv6 addresses: global unicast addresses. As it turns out, many of the general concepts and processes behind these global unicast IPv6 addresses follow the original intent for public IPv4 addresses. So, this section begins with a review of some IPv4 concepts, followed by the details of how a company can use global unicast addresses.

This first section also discusses IPv6 subnetting and the entire process of taking a block of global unicast addresses and creating subnets for one company. This process takes a globally unique global routing prefix, creates IPv6 subnets, and assigns IPv6 addresses from within each subnet, much like with IPv4.

## Public and Private IPv6 Addresses

The original plan for worldwide IPv4 addresses called for each organization connected to the Internet to be assigned a unique public IPv4 network. Each organization could then subnet that network and assign addresses from within that network so that every host in every organization used an IPv4 address unique in the universe. Unfortunately, because the IPv4 address space had too few addresses for that plan to work once every company, organization, and home wanted to connect to the Internet, those responsible for the IPv4 addressing updated their plan.

As part of that revised plan, in the 1990s, companies started using addresses from the private IPv4 address range, as defined in RFC 1918, along with Network Address Translation (NAT). Using NAT and private IPv4 addresses allowed one organization to share a few public globally unique IPv4 addresses for all host connections into the Internet.

IPv6 allows two similar options of public and private unicast addressing, beginning with *global unicast* addresses as the public IPv6 address space. Similar to public IPv4 addresses, IPv6 global unicast addresses rely on an administrative process that assigns each company a unique IPv6 address block. Each company then subnets this IPv6 address block and only uses addresses from within that block. The result is that each company uses addresses that are unique across the globe as well.

The second IPv6 option uses *unique local* IPv6 addresses, which work like the IPv4 private addresses. Companies that do not plan to connect to the Internet and companies that plan to use IPv6 NAT can use unique local addresses. With IPv4, an organization simply picked numbers from the private networks in RFC 1918. With IPv6, you choose by referencing RFC 4193, which suggests a process to randomly choose a unique local prefix. And just as when using private networks with IPv4, when using IPv6 unique local addresses (ULAs), to connect to the Internet, the organization would need a small set of public IPv6 addresses and would need to use NAT.

The following lists summarizes the comparisons between global unicast addresses and unique local addresses:




**Global Unicast Addresses (GUAs):** These addresses work like public IPv4 addresses. The organization that needs IPv6 addresses asks for a registered IPv6 address block, which is assigned as a global routing prefix. After that, only that organization uses the addresses inside that block of addresses—that is, the addresses that begin with the assigned prefix.

**Unique Local Addresses (ULAs):** These addresses work somewhat like private IPv4 addresses, with the possibility that multiple organizations use the exact same addresses, and with no requirement for registering with any numbering authority.

---

Answers to the "Do I Know This Already?" quiz:

**1** C **2** A **3** D **4** B **5** D

The rest of this first major section of the chapter examines global unicast addresses in more detail, while the second major section discusses unique local addresses.

## The IPv6 Global Routing Prefix

IPv6 global unicast addresses (GUAs) allow IPv6 to work more like the original design of the IPv4 Internet. Each organization asks for a block of IPv6 addresses, which no one else can use. That organization further subdivides the address block into smaller chunks called *subnets*. Finally, the engineer chooses an address from the right subnet to assign for use by a host.

That reserved block of IPv6 addresses—a set of addresses only one company can use—is called a **global routing prefix**. Each organization that wants to connect to the Internet and use IPv6 GUAs should ask for and receive a global routing prefix. In comparison, you can think of the global routing prefix like an IPv4 Class A, B, or C network number from the range of public IPv4 addresses, or think of it like a public CIDR block.



The term *global routing prefix* might not make you think of a block of IPv6 addresses at first. The term refers to the idea that Internet routers can have one route that refers to all the addresses inside the address block, without a need to have routes for smaller parts of that block. For example, Figure 26-1 shows three companies with three different IPv6 global routing prefixes; the router on the right (R4) inside the Internet has one IPv6 route for each global routing prefix.

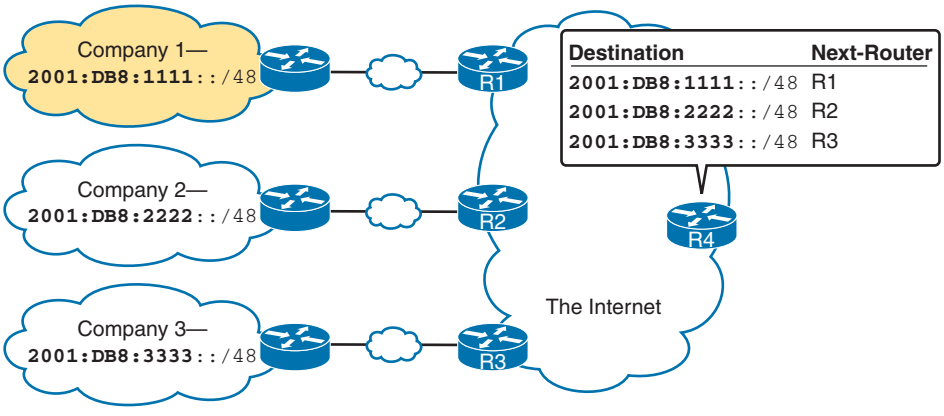



**Figure 26-1** *Three Global Routing Prefixes, with One Route per Prefix*

The figure shows three global routing prefixes, each defining a range of GUAs reserved for use by the three companies in the figure. Consider prefix 2001:DB8:1111::/48. The prefix length, 48, happens to equal 12*4. So, the first phrase that follows gives a more literal and binary view of the meaning of "2001:DB8:1111::/48," while the second line shows the much easier hex view:

Addresses whose first 48 bits equal the first 48 bits of 2001:DB8:1111::

Addresses whose first 12 hex digits equal the first 12 hex digits of 2001:DB8:1111::

The address assignment process sets those IPv6 addresses apart for use by that one company, just like a public IPv4 network or a CIDR address block does in IPv4. All IPv6

addresses inside that company should begin with those first bits in the global routing prefix. No other companies should use IPv6 addresses with that same prefix. And thankfully, IPv6 has plenty of space to allow all companies to have a global routing prefix with plenty of addresses.

Both the IPv6 and IPv4 address assignment processes rely on the same organizations: IANA (along with ICANN), the Regional Internet Registries (RIR), and ISPs. For example, an imaginary company, Company1, received the assignment of a global routing prefix. The prefix means "All addresses whose first 12 hex digits are 2001:0DB8:1111," as represented by prefix 2001:0DB8:1111::/48. To receive that assignment, the process shown in Figure 26-2 happened.

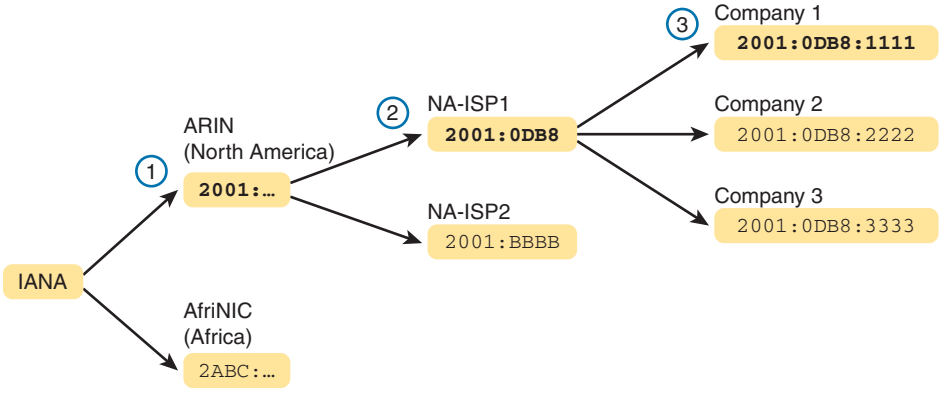



**Figure 26-2** *Prefix Assignment with IANA, RIRs, and ISPs*

The event timeline in the figure uses a left-to-right flow; in other words, the event on the far left must happen first. Following the flow from left to right in the figure:

1. **IANA allocates ARIN prefix 2001::/16:** ARIN (the RIR for North America) asks IANA to allocate a large block of addresses. In this imaginary example, IANA gives ARIN a prefix of "all addresses that begin 2001," or 2001::/16.
2. **ARIN allocates NA-ISP1 prefix 2001:0DB8::/32:** NA-ISP1, an imaginary ISP based in North America, asks ARIN for a new IPv6 prefix. ARIN takes a subset of its 2001::/16 prefix, specifically all addresses that begin with the 32 bits (8 hex digits) 2001:0DB8, and allocates it to the ISP.
3. **NA-ISP1 assigns Company 1 2001:0DB8:1111::/48:** Company 1 decides to start supporting IPv6, so it goes to its ISP, NA-ISP1, to ask for a block of GUAs. NA-ISP1 assigns Company 1 a "small" piece of NA-ISP1's address block, in this case, the addresses that begin with the 48 bits (12 hex digits) of 2001:0DB8:1111 (2001:0DB8:1111::/48).

**NOTE** If you do not plan to connect to the Internet using IPv6 for a while and just want to experiment, you do not need to ask for an IPv6 global routing prefix to be assigned. Just make up IPv6 addresses and configure your devices, or use unique local addresses, as discussed toward the end of this chapter.

## Address Ranges for Global Unicast Addresses

Global unicast addresses make up the majority of the IPv6 address space. However, unlike IPv4, the rules for which IPv6 addresses fall into which category are purposefully more flexible than they were with IPv4 and the rules for IPv4 Classes A, B, C, D, and E.

IANA allocates all IPv6 addresses that begin with hex 2 or 3 as global unicast addresses. The prefix 2000::/3 formally defines that range of numbers, meaning all addresses whose first three bits (per the /3 prefix length) match the first three bits of 2000::. To further explain:

- The first hex digit (2) is binary 0010.
- /3 means the number represents all addresses with the same first three bits as the listed prefix, or 001 binary in this case.
- The hex values whose first three bits are 001 are hex 2 and 3 but no others.
- Therefore, prefix 2000::/3 means all addresses that begin with hex 2 or 3.



IANA can expand the GUA address range beyond 2000::/3 over time if needed. RFC 4291, "IPv6 Addressing Architecture," which lays the foundation of IPv6 addressing, reserves all addresses not otherwise reserved for the GUA address space. However, by current policy, IANA allocates global unicasts only from the 2000::/3 range.

**NOTE** For perspective, the 2000::/3 GUA address space, if allocated to organizations as only /48 prefixes, would provide more than 30 trillion /48 global routing prefixes. Each global routing prefix would allow for 65,536 subnets. IANA may never exhaust the 2000::/3 GUA address space.

Table 26-2 lists the address prefixes discussed in this book and their purpose.

Key Topic

**Table 26-2** Some Types of IPv6 Addresses and Their First Hex Digit(s)

| Address Type | First Hex Digits |
|---|---|
| Global unicast | 2 or 3 |
| Unique local | FD |
| Multicast | FF |
| Link local | FE80 |

## IPv6 Subnetting Using Global Unicast Addresses

After an enterprise has a block of reserved GUAs—in other words, a global routing prefix—the company needs to subdivide that large address block into subnets.

Subnetting IPv6 addresses generally works like IPv4, but with mostly simpler math (hoorah!). Many IPv6 RFCs dictate that all subnets deployed in a network (that is, by configuration on endpoints and routers) use a /64 prefix length. Using /64 as the prefix length for all subnets makes the IPv6 subnetting math just as easy as using a /24 mask for all IPv4 subnets. In addition, the dynamic IPv6 address assignment process works better with a /64 prefix length as well; so in practice, and in this book, expect IPv6 designs to use a /64 prefix length for subnets.

This section progresses through the different parts of IPv6 subnetting while using examples that use a /64 prefix length. The discussion defines the rules about which addresses should be in the same subnet and which addresses need to be in different subnets. Plus, this section looks at how to analyze the global routing prefix and associated prefix length to find all the IPv6 subnet prefixes and the addresses in each subnet.

**NOTE** If the IPv4 subnetting concepts are a little vague, you might want to reread Chapter 11, "Perspectives on IPv4 Subnetting," which discusses the subnetting concepts for IPv4.

## Deciding Where IPv6 Subnets Are Needed

First, IPv6 and IPv4 both use the same concepts about where a subnet is needed: one for each VLAN and one for each point-to-point WAN connection (serial and Ethernet). Figure 26-3 shows an example of the idea, using the small enterprise internetwork of Company 1. Company 1 has two LANs, with a point-to-point Ethernet WAN link connecting the sites. It also has an Ethernet WAN link connected to an ISP. Using the same logic you would use for IPv4, Company 1 needs four IPv6 subnets.

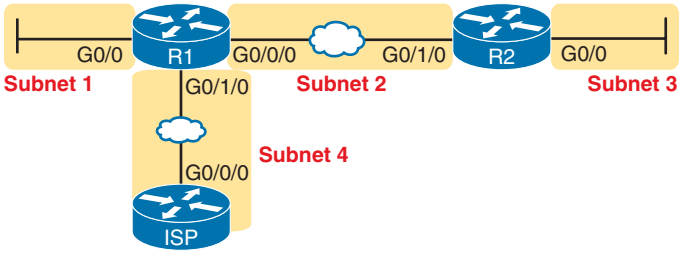



**Figure 26-3** *Locations for IPv6 Subnets*

## The Mechanics of Subnetting IPv6 Global Unicast Addresses

To understand how to subnet your one large block of IPv6 addresses, you need to understand some of the theories and mechanisms IPv6 uses. To learn those details, it can help to compare IPv6 with some similar concepts from IPv4.

With IPv4, without subnetting, an address has two parts: a network part and a host part. Class A, B, and C rules define the length of the network part, with the host part making up the rest of the 32-bit IPv4 address, as shown in Figure 26-4.

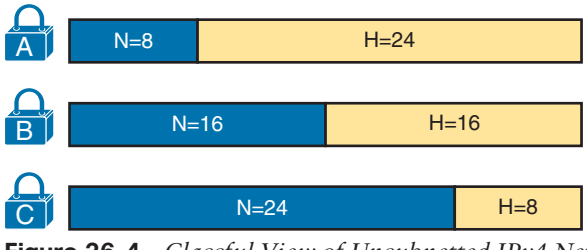



**Figure 26-4** *Classful View of Unsubnetted IPv4 Networks*

To subnet an IPv4 Class A, B, or C network, the network engineer for the enterprise makes some choices. Conceptually, the engineer creates a three-part view of the addresses, adding a subnet field in the center while shortening the host field. (Many people call this "borrowing host bits.") The size of the network part stays locked per the Class A, B, and C rules, with the line between the subnet and host part being flexible, based on the choice of subnet mask. Figure 26-5 shows the field names and concepts idea for a subnetted Class B network.

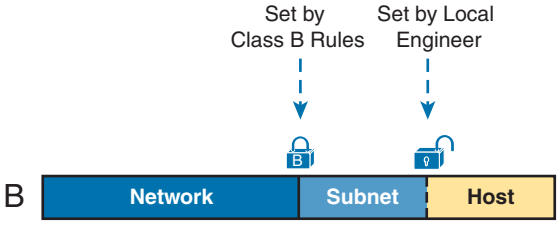



**Figure 26-5** *Classful View of Subnetted IPv4 Networks with Field Names*



First, just think about the general idea with IPv6, comparing Figure 26-6 to Figure 26-5. The IPv6 global routing prefix (the prefix/length assigned by the RIR or ISP) acts like the IPv4 network part of the address structure. The IPv6 subnet ID acts like the IPv4 subnet field. And the right side of the IPv6 address, formally called the **interface ID** (short for interface identifier), or simply *IID*, acts like the IPv4 host field.

Key Topic

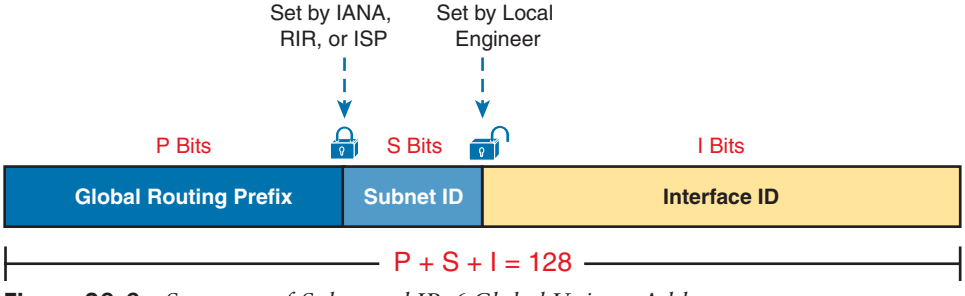



**Figure 26-6** *Structure of Subnetted IPv6 Global Unicast Addresses*

Now focus on the IPv6 global routing prefix and its prefix length. Unlike IPv4, IPv6 has no concept of address classes, so no preset rules determine the prefix length of the global routing prefix. When a company applies to an ISP, RIR, or any other organization that can assign a global routing prefix, that assignment includes both the prefix and the prefix length. After a company receives a global routing prefix and that prefix length, the length of the prefix typically does not change over time and is locked. (Note that the prefix length of the global routing prefix is often between /32 and /48, or possibly as long as /56.)

Next, look to the right side of Figure 26-6 to the interface ID (IID) field. Several RFCs (including RFC 4291, "IPv6 Addressing Architecture") state that IIDs should be 64 bits long. A 64-bit IID works well with the IPv6 methods for dynamic IPv6 address assignment.

Finally, look to the subnet ID field in Figure 26-6. This field creates a place with which to number IPv6 subnets. The subnet ID field length depends on two facts: the global routing prefix's length and the interface ID's length. Assuming the typical 64-bit IID, the subnet ID field is typically 64–P bits, with P being the length of the global routing prefix.

As an example, consider the structure of a specific global unicast IPv6 address, 2001:0DB8:1111:0001:0000:0000:0000:0001, as seen in Figure 26-7. In this case:




- The company was assigned global routing prefix 2001:0DB8:1111/48.
- The company uses the usual 64-bit interface ID and /64 prefix length for the subnet.
- The company has a subnet field of 16 bits, allowing for $2^{16}$ IPv6 subnets.

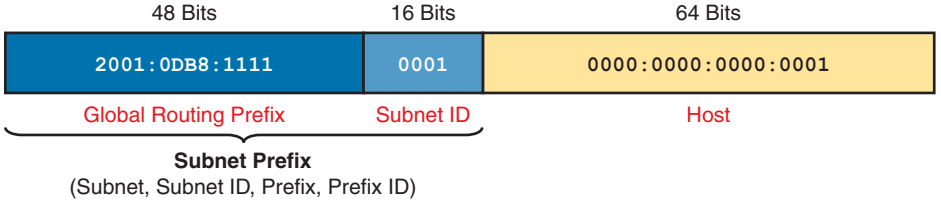



**Figure 26-7** *Address Structure for Company 1 Example*

The example in Figure 26-7, along with a bit of math, shows one reason why so many companies embrace the recommendation of a 64-bit IID. With this structure, Company 1 can support $2^{16}$ possible subnets (65,536). Few companies need that many subnets. However, RIRs and ISPs can be generous in approving much larger blocks of public IPv6 addresses, so even if 65,536 subnets are enough, a company might apply for and receive a global routing prefix assignment with a shorter length, like /32, or /40. That would give them many more subnets ($2^{32}$ and $2^{24}$, respectively), opening up many possibilities for their subnet design and subnet numbering plans, beyond just having enough subnets.

Each subnet has far more than enough hosts per subnet with 64 bits in the interface ID. The motivation for the 64-bit IID comes from supporting various RFCs, particularly those related to dynamic IPv6 address assignment, including DHCP and SLAAC.

### Listing the IPv6 Subnet Prefix (Subnet ID)

When working with IPv6 in Cisco routers, you often work with the numbers representing the subnet. For IPv6, you will use these terms:

**Subnet prefix:** The formal term for the number representing the subnet, including both the global routing prefix and subnet ID shown in Figure 26-7.

**Subnet ID** or **Prefix ID:** Informal terms used as synonyms for subnet prefix.

While the term *subnet prefix* comes from the RFCs, people tend to use the term *subnet ID*, given the history of that term in IPv4. However, note the potential miscommunication between the formal meaning of subnet ID (the middle portion of the address structure per Figure 26-7) or as a synonym for subnet prefix.

Chapter 25, "Fundamentals of IP Version 6," already discussed finding the subnet prefix, given an IPv6 address and prefix length. The math works the same whether working with global unicast addresses or with the unique local addresses discussed later in the chapter. Just to review an example, the subnet prefix for the address in Figure 26-7 would be

```
2001:DB8:1111:1::/64
```

### List All IPv6 Subnets

With IPv4, if you choose to use a single subnet mask for all subnets, you can find all the subnets of a Class A, B, or C network using that one subnet mask. With IPv6, the same ideas apply, and the /64 prefix length means you can find all subnets with no binary math.

To find all the subnet IDs, you simply need to find all the unique values that will fit inside the subnet ID part of the three-part IPv6 address, as shown in Figure 26-6 and Figure 26-7, basically following these rules:

- All subnet IDs begin with the global routing prefix.
- Use a different value in the subnet field to identify each subnet.
- All subnet IDs have all 0s in the interface ID.

As an example, take the IPv6 design shown in Figure 26-7, and think about all the subnet IDs. First, all subnets will use the commonly used /64 prefix length. This company uses a global routing prefix of 2001:0DB8:1111::/48, which defines the first 12 hex digits of all the subnet IDs. The list of all possible IPv6 subnet prefixes provides all binary combinations in the subnet ID field, in this case, the fourth quartet. Also, represent the last four 0000 quartets with a :: symbol. Figure 26-8 shows the beginning of just such a list.

```
  2001:0DB8:1111:0000::    2001:0DB8:1111:0008::
✓ 2001:0DB8:1111:0001::    2001:0DB8:1111:0009::
✓ 2001:0DB8:1111:0002::    2001:0DB8:1111:000A::
✓ 2001:0DB8:1111:0003::    2001:0DB8:1111:000B::
✓ 2001:0DB8:1111:0004::    2001:0DB8:1111:000C::
  2001:0DB8:1111:0005::    2001:0DB8:1111:000D::
  2001:0DB8:1111:0006::    2001:0DB8:1111:000E::
  2001:0DB8:1111:0007::    2001:0DB8:1111:000F::
  Global Routing Prefix Subnet  Global Routing Prefix Subnet
```

**Figure 26-8** *First 16 Possible Subnets with a 16-bit Subnet Field in This Example*

The example allows for 65,536 subnets, so clearly, the example will not list all the possible subnets. However, in that fourth quartet, all combinations of hex values would be allowed.

**NOTE** Each IPv6 subnet ID, more formally called the **subnet router anycast address**, is reserved and should not be used as an IPv6 address for any host.

### Assign Subnets to the Internetwork Topology

After an engineer lists all the possible subnet IDs (based on the subnet design), the next step is to choose which subnet ID to use for each link that needs an IPv6 subnet. As with IPv4, each VLAN, each serial link, each Ethernet WAN link, and many other data-link instances need an IPv6 subnet.

Figure 26-9 shows an example using Company 1 again. The figure uses the four subnets from Figure 26-8 that have check marks beside them. The check marks are just a reminder not to use those four subnets in other locations.

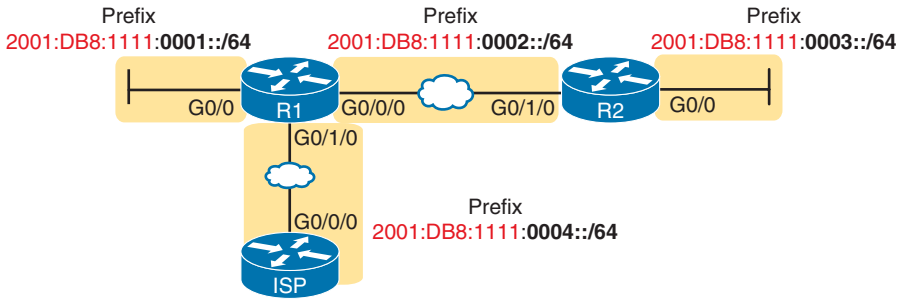



**Figure 26-9** *Subnets in Company 1, with Global Routing Prefix of 2001:0DB8:1111::/48*

### Assigning Addresses to Hosts in a Subnet

Now that the engineer has planned which IPv6 subnet to use in each location, the individual IPv6 addressing can be planned and implemented. Each address must be unique in that no other host interface uses the same IPv6 address. Also, the hosts cannot use the subnet ID itself.

The process of assigning IPv6 addresses to interfaces works similarly to IPv4. Addresses can be configured statically, along with the prefix length, default router, and Domain Name System (DNS) IPv6 addresses. Alternatively, hosts can learn these same settings dynamically, using either Dynamic Host Configuration Protocol (DHCP) or a built-in IPv6 mechanism called Stateless Address Autoconfiguration (SLAAC). Chapter 28, "Implementing IPv6 Addressing on Hosts," discusses IPv6 address assignment in more depth.

For example, Figure 26-10 shows some static IP addresses chosen for the router interfaces based on the subnet choices shown in Figure 26-9. In each case, the router interfaces use an interface ID that is a relatively low number and easily remembered.

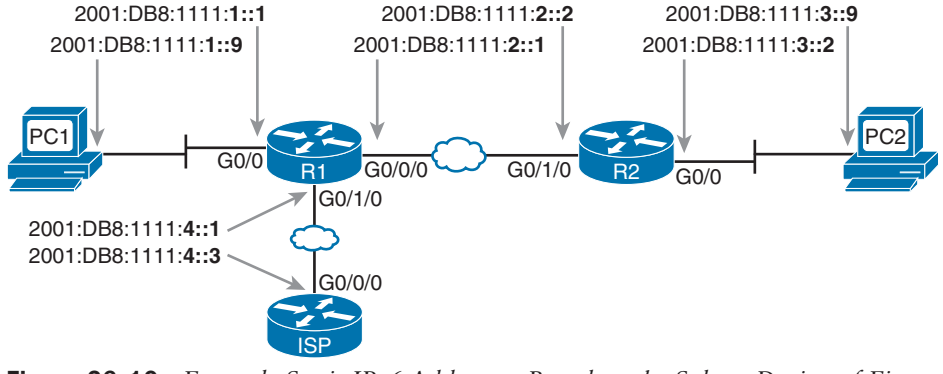



**Figure 26-10** *Example Static IPv6 Addresses Based on the Subnet Design of Figure 26-9*

This chapter puts off the details of how to configure the IPv6 addresses until Chapter 27, "Implementing IPv6 Addressing on Routers."

## Unique Local Unicast Addresses

Unique local addresses (ULAs) act as private IPv6 unicast addresses. These addresses have many similarities with global unicast addresses, particularly in how to subnet. The biggest

difference lies in the literal number (ULAs begin with hex FD) and with the administrative process: the ULA prefixes are not registered with any numbering authority and can be used by multiple organizations.

Although the network engineer creates unique local addresses without any registration or assignment process, the addresses still need to follow some rules, as follows:

Key Topic

- Use FD as the first two hex digits.
- Choose a unique 40-bit global ID.
- Append the global ID to FD to create a 48-bit prefix, used as the prefix for all your addresses.
- Use the next 16 bits as a subnet field.
- Note that the structure leaves a convenient 64-bit interface ID field.



Figure 26-11 shows the format of these unique local unicast addresses.

Key Topic

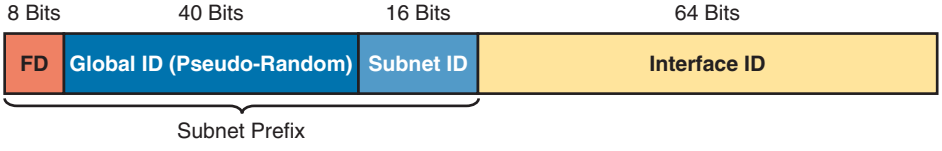



**Figure 26-11** *IPv6 Unique Local Unicast Address Format*

**NOTE** Just to be entirely exact, IANA reserves prefix FC00::/7, and not FD00::/8, as the address range for ULAs. FC00::/7 includes all addresses that begin with hex FC and FD. However, the ULA RFC (4193) also requires setting the eighth bit to 1, which means that all ULAs begin with their first two hex digits as FD.

## Subnetting with Unique Local IPv6 Addresses

Subnetting ULAs works like subnetting GUAs, assuming you begin with a 48-bit global routing prefix. The differences lie in that you must be assigned the GUA global routing prefix and prefix length. With ULAs, you create a prefix locally, with a prefix of /48.

To choose your 48-bit (12 hex digit) ULA prefix, you begin with hex FD and choose ten hex digits via a random number process. Per the ULA RFC, you should use a defined pseudo-random algorithm to determine your global ID. When practicing in lab, you can just make up a number. For example, imagine you chose a 10-hex-digit value of hex 00 0001 0001 and prepend a hex FD, making the entire prefix FD00:0001:0001::/48, or FD00:1:1::/48 when abbreviated.

To create subnets, just as you did in the earlier examples with a 48-bit global routing prefix, treat the entire fourth quartet as the subnet ID field, as shown in Figure 26-11.

Figure 26-12 shows an example subnetting plan using unique local addresses. The example repeats the same topology shown earlier in Figure 26-9; that figure showed subnetting with a global unicast prefix. This example uses the exact same numbers for the fourth quartet's

subnet field, simply replacing the 48-bit global unicast prefix with this new local unique prefix of FD00:1:1.

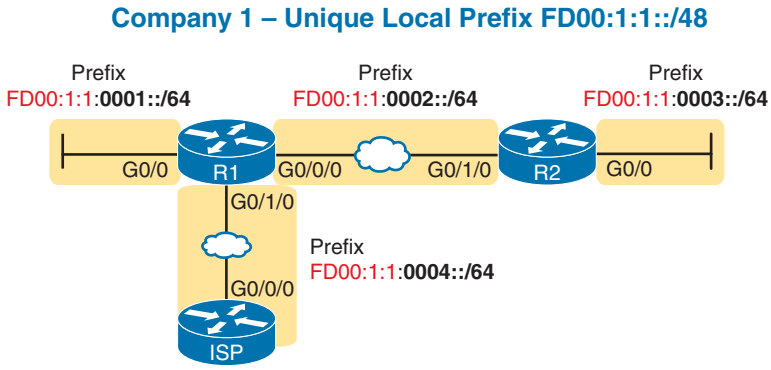



**Figure 26-12** *Subnetting Using Unique Local Addresses*

## The Need for Globally Unique Local Addresses

The example in Figure 26-12 shows an easy-to-remember prefix of FD00:1:1::/48. I made up the easy-to-remember global ID in this example. What global ID would you choose for your company? Would you pick a number that you could not abbreviate and make it shorter? If you had to pick the IPv6 prefix for your ULAs from the options in the following list, which would you pick for your company?

- `FDE9:81BE:A059::/48`
- `FDF0:E1D2:C3B4::/48`
- `FD00:1:1::/48`

Given the freedom to choose, most people would pick an easy-to-remember, short-to-type prefix, like FD00:1:1::/48. And in a lab or other small network used for testing, making up an easy-to-use number is reasonable. However, for use in real corporate networks, you should not just make up any global ID you like. Instead, follow the ULA rules to make your ULAs unique in the universe—even without registering a prefix with an ISP or RIR.

RFC 4193 stresses the importance of choosing your global ID to make it statistically unlikely to be used by other companies. What is the result of unique global IDs at every company? It makes all these **ULA global IDs** unique across the globe. So, if you plan on using ULAs in a real network, use the random number generator logic listed in RFC 4193 to create your prefix.

One of the big reasons to attempt to use a unique prefix, rather than everyone using the same easy-to-remember prefixes, is to be ready for the day your company merges with or buys another company. Today, with IPv4, a high percentage of companies use private IPv4 network 10.0.0.0. When they merge their networks, the fact that both use network 10.0.0.0 makes the network merger more painful than if the companies had used different private IPv4 networks. With IPv6 ULAs, if both companies did the right thing and randomly chose a prefix, they will most likely be using completely different prefixes, making the merger much simpler. However, companies that take the seemingly easy way out and choose an easy-to-remember prefix like FD00:1:1 significantly increase their risk of requiring extra effort when merging with another company that also chose to use that same prefix.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 26-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 26-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory table | | Website |
| Watch video | | Website |



## Review All the Key Topics

Key Topic

**Table 26-4** Key Topics for Chapter 26

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Two types of IPv6 unicast addresses | 656 |
| Table 26-2 | Values of the initial hex digits of IPv6 addresses, and the address type implied by each | 659 |
| Figure 26-6 | Subnetting concepts for IPv6 global unicast addresses | 661 |
| List | Rules for how to find all IPv6 subnet IDs, given the global routing prefix, and prefix length used for all subnets | 662 |
| List | Rules for building unique local unicast addresses | 665 |
| Figure 26-11 | Subnetting concepts for IPv6 unique local addresses | 665 |

## Key Terms You Should Know

global routing prefix, global unicast address, interface ID (IID), subnet ID (prefix ID), subnet prefix, subnet router anycast address, ULA global ID, unique local address



CHAPTER 27

# Implementing IPv6 Addressing on Routers

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.8 Configure and verify IPv6 addressing and prefix**

**1.9 Describe IPv6 address types**

**1.9.a Unicast (global, unique local, and link local)**

**1.9.b Anycast**

**1.9.c Multicast**

**1.9.d Modified EUI 64**

With IPv4 addressing, some devices, like servers and routers, typically use static predefined IPv4 addresses. End-user devices do not mind if their address changes from time to time, and they typically learn an IPv4 address dynamically using DHCP. IPv6 uses the same approach, with servers, routers, and other devices in the control of the IT group often using predefined IPv6 addresses, and with end-user devices using dynamically learned IPv6 addresses.

This chapter focuses on IPv6 address configuration on routers, with the next chapter focusing on end-user devices. This chapter begins with the more obvious IPv6 addressing configuration, with features that mirror IPv4 features, showing how to configure interfaces with IPv6 addresses and view that configuration with **show** commands. The second half of the chapter introduces new IPv6 addressing concepts in comparison to IPv4, showing some other IPv6 addresses used by routers when doing different tasks.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 27-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Implementing Unicast IPv6 Addresses on Routers | 1–4 |
| Special Addresses Used by Routers | 5–8 |

1. Router R1 has an interface named Gigabit Ethernet 0/1, whose MAC address has been set to 0200.0001.000A. Which of the following commands, added in R1's Gigabit Ethernet 0/1 configuration mode, gives this interface a unicast IPv6 address of 2001:1:1:1:1:200:1:A with a /64 prefix length?
   a. **ipv6 address 2001:1:1:1:1:200:1:A/64**
   b. **ipv6 address 2001:1:1:1:1:200:1:A/64 eui-64**
   c. **ipv6 address 2001:1:1:1:1:200:1:A /64 eui-64**
   d. **ipv6 address 2001:1:1:1:1:200:1:A /64**
   e. None of the other answers are correct.
2. Router R1 has an interface named Gigabit Ethernet 0/1, whose MAC address has been set to 5055.4444.3333. This interface has been configured with the **ipv6 address 2000:1:1:1::/64 eui-64** subcommand. What unicast address will this interface use?
   a. 2000:1:1:1:52FF:FE55:4444:3333
   b. 2000:1:1:1:5255:44FF:FE44:3333
   c. 2000:1:1:1:5255:4444:33FF:FE33
   d. 2000:1:1:1:200:FF:FE00:0
3. Router R1 currently supports IPv4, routing packets in and out all its interfaces. R1's configuration must be migrated to support dual-stack operation, routing both IPv4 and IPv6. Which of the following tasks must be performed before the router can also support routing IPv6 packets? (Choose two answers.)
   a. Enable IPv6 on each interface using an **ipv6 address** interface subcommand.
   b. Enable support for both versions with the **ip versions 4 6** global command.
   c. Enable IPv6 routing using the **ipv6 unicast-routing** global command.
   d. Migrate to dual-stack routing using the **ip routing dual-stack** global command.
4. On a router configured with an **ipv6 address** interface subcommand on its G0/0/0 interface, which of the following commands reveals the IPv6 prefix that the router computed based on the address/prefix-length? (Choose two answers.)
   a. **show ipv6 address**
   b. **show ipv6 route connected**
   c. **show ipv6 interface brief**
   d. **show ipv6 interface g0/0/0**
5. Router R1 has an interface named Gigabit Ethernet 0/1, whose MAC address has been set to 0200.0001.000A. The interface is then configured with the **ipv6 address 2001:1:1:1:200:FF:FE01:B/64** interface subcommand; no other **ipv6 address** commands are configured on the interface. Which of the following answers lists the link-local address used on the interface?
   a. FE80::FF:FE01:A
   b. FE80::FF:FE01:B
   c. FE80::200:FF:FE01:A
   d. FE80::200:FF:FE01:B

6. Which of the following multicast addresses is defined as the address for sending packets to only the IPv6 routers on the local link?
   a. FF02::1
   b. FF02::2
   c. FF02::5
   d. FF02::A
7. Host C must forward a packet to its default gateway, 2001:db8:1:1::1234:5678. Host C does not yet have a neighbor table entry for that address. To which solicited-node multicast address will host C send its NDP neighbor solicitation (NS) request?
   a. FF02::1:ff34:5678
   b. FF02::1:ff12:3456
   c. FF02::1ff:1234:5678
   d. FF02::1ff:34:5678
8. A router uses the **ipv6 enable** subcommand on interface G0/0/1, the **ipv6 address autoconfig** command on interface G0/0/2, and the **ipv6 address 2002:db8:1:1::1/64** command on interface G0/0/3. The commands succeed, dynamically or statically creating the expected addresses as typical for each command. Which answers are accurate about the link-local addresses (LLAs) on the various interfaces? (Choose two answers.)
   a. G0/0/1 has no LLA.
   b. G0/0/1 has an LLA.
   c. G0/0/2 has an LLA and global unicast address with identical interface IDs.
   d. G0/0/3 has a global unicast address but no LLA.

## Foundation Topics

# Implementing Unicast IPv6 Addresses on Routers

Every company bases its enterprise network on one or more protocol models or protocol stacks. In the earlier days of networking, enterprise networks used one or more protocol stacks from different vendors, as shown on the left of Figure 27-1. Over time, companies added TCP/IP (based on IPv4) to the mix. Eventually, companies migrated fully to TCP/IP as the only protocol stack in use.

IPv6 might fully replace IPv4 one day; however, for a long migration period, most enterprises will run IPv4 and IPv6 on hosts and routers simultaneously, effectively like in the old days, with multiple protocol stacks. Over time, hosts will use IPv6 more and more until one day, it might be possible to disable IPv4 and run with only IPv6, as shown in Figure 27-2.

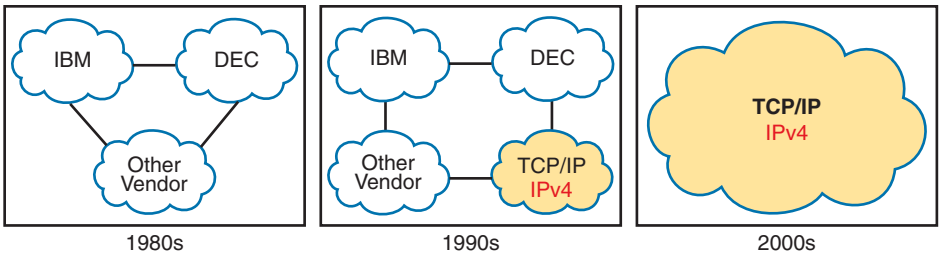



**Figure 27-1** *Migration of Enterprise Networks to Use TCP/IP Stack Only, IPv4*

Key Topic

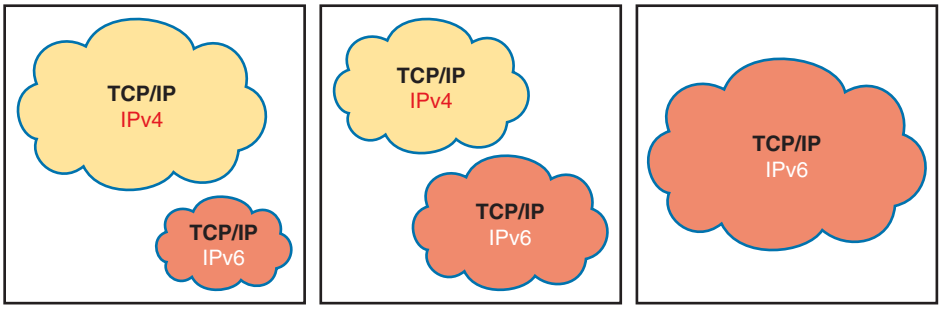



**Figure 27-2** *Possible Path Through a Dual Stack (IPv4 and IPv6) over a Long Period*

Using **dual stacks**—that is, running both IPv4 and IPv6—makes great sense for today's enterprise networks. To do so, configure the routers to route IPv6 packets with IPv6 addresses on their interfaces, and advertise IPv6 routes with a routing protocol, while continuing to do the same with IPv4 addresses and routing protocols. On hosts and servers, implement IPv4 and IPv6 as well. The hosts have new methods to choose when to use IPv6 and when to use IPv4, giving preference to IPv6.

While using a dual-stack strategy works well, the configuration model for IPv6 works much as it does for IPv4. To that end, the first major section of this chapter shows how to configure and verify unicast IPv6 addresses on routers.

## Static Unicast Address Configuration

Cisco routers give us two options for static configuration of IPv6 addresses. You configure the full 128-bit address and the standard /64 prefix length in one case. A second option allows you to configure a 64-bit prefix and let the router derive the second half of the address (the interface ID [IID]). The next few pages show both options.

### Configuring the Full 128-Bit Address

To statically configure the full 128-bit unicast address—either global unicast address (GUA) or unique local address (ULA) —use the **ipv6 address** *address/prefix-length* interface subcommand. The command accepts abbreviated or expanded addresses, a slash, and then prefix length, with no spaces. Examples 27-1 and 27-2 show the configuration of the IPv6 GUAs on routers R1 and R2 from Figure 27-3, respectively.

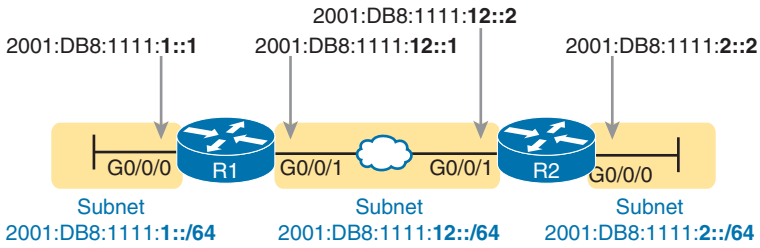



**Figure 27-3** *Sample 128-bit IPv6 Addresses to Be Configured on Cisco Router Interfaces*

**Example 27-1** *Configuring Static IPv6 Addresses on R1*

```
ipv6 unicast-routing
!
interface GigabitEthernet0/0/0
 ipv6 address 2001:DB8:1111:1::1/64
!
! Below, note the expanded address. IOS will abbreviate the address for you.
interface GigabitEthernet0/0/1
 ipv6 address 2001:0db8:1111:0012:0000:0000:0000:0001/64
```

**Example 27-2** *Configuring Static IPv6 Addresses on R2*

```
ipv6 unicast-routing
!
interface GigabitEthernet0/0/0
 ipv6 address 2001:DB8:1111:2::2/64
!
interface GigabitEthernet0/0/1
 ipv6 address 2001:db8:1111:12::2/64
```

**NOTE** The configuration on R1 in Example 27-1 shows the commands as typed. One **ipv6 address** command uses an abbreviated address with uppercase hex digits, while the other uses an expanded address with lowercase hex digits. IOS accepts both commands but then changes them to their abbreviated form with uppercase hex digits.

## Enabling IPv6 Routing

Interestingly, Cisco routers enable IPv4 routing by default but not IPv6 routing. The global command **ip routing** (the default setting) enables the routing of IPv4 packets. To route IPv6 packets, you must also configure the **ipv6 unicast-routing** global command.

If you forget to configure the **ipv6 unicast-routing** global command, the router will still accept your **ipv6 address** interface subcommands. In that case, the router acts as an IPv6

---

Answers to the "Do I Know This Already?" quiz:

**1** A **2** B **3** A, C **4** B, D **5** A **6** B **7** A **8** B, C

host, so it can send IPv6 packets it generates. However, the router will not route IPv6 packets (that is, forward IPv6 packets received in an interface) until you configure the **ipv6 unicast-routing** global command.

### Verifying the IPv6 Address Configuration

IPv6 uses many **show** commands that mimic the syntax of IPv4 **show** commands. For example:

- The **show ipv6 interface brief** command gives you interface IPv6 address info but not prefix length info, similar to the IPv4 **show ip interface brief** command.
- The **show ipv6 interface** command gives the details of IPv6 interface settings, much like the **show ip interface** command does for IPv4.

The one notable difference in the most common commands is that the **show interfaces** command still lists the IPv4 address and mask but tells us nothing about IPv6. So, to see IPv6 interface addresses, use commands that begin with **show ipv6**. Example 27-3 lists a few samples from Router R1, with the explanations following.

**Example 27-3** *Verifying Static IPv6 Addresses on Router R1*

```
! The first interface is in subnet 1
R1# show ipv6 interface GigabitEthernet 0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
  No Virtual link-local address(es):
  Global unicast address(es):
    2001:DB8:1111:1::1, subnet is 2001:DB8:1111:1::/64
  Joined group address(es):
    FF02::1
    FF02::2
    FF02::1:FF00:1
    FF02::1:FF11:1111
  MTU is 1500 bytes
  ICMP error messages limited to one every 100 milliseconds
  ICMP redirects are enabled
  ICMP unreachables are sent
  ND DAD is enabled, number of DAD attempts: 1
  ND reachable time is 30000 milliseconds (using 30000)
  ND advertised reachable time is 0 (unspecified)
  ND advertised retransmit interval is 0 (unspecified)
  ND router advertisements are sent every 200 seconds
  ND router advertisements live for 1800 seconds
  ND advertised default router preference is Medium
  Hosts use stateless autoconfig for addresses.

R1# show ipv6 interface brief
GigabitEthernet0/0/0      [up/up]
    FE80::11FF:FE11:1111
```

```
    2001:DB8:1111:1::1
GigabitEthernet0/0/1      [up/up]
    FE80::32F7:DFF:FE29:8568
    2001:DB8:1111:12::1
GigabitEthernet0/0/2      [administratively down/down]
    unassigned
GigabitEthernet0/0/3      [administratively down/down]
    unassigned
```

First, focus on the output of the **show ipv6 interface** command at the top of the example, which lists interface G0/0/0, showing output about that interface only. The output lists the configured IPv6 address and prefix length and the IPv6 subnet (2001:DB8:1111:1::/64), which the router calculated based on the IPv6 address.

To close the example, the **show ipv6 interface brief** command lists IPv6 addresses, not the prefix length or prefixes. Note that this command also lists all interfaces, whether configured with IPv6 addresses or not.

As with IPv4, the router adds IPv6 connected routes to the IPv6 routing table based on the IPv6 address configuration. Just as with IPv4, the router keeps these connected routes in the IPv6 routing table when the interface is working (up/up) and removes them when the interface is down. Example 27-4 shows the connected IPv6 on Router R1 from Figure 27-3.

**Example 27-4** *Displaying Connected IPv6 Routes on Router R1*

```
R1# show ipv6 route connected
IPv6 Routing Table - default - 5 entries
Codes: C - Connected, L - Local, S - Static, U - Per-user Static route
       B - BGP, R - RIP, H - NHRP, I1 - ISIS L1
       I2 - ISIS L2, IA - ISIS interarea, IS - ISIS summary, D - EIGRP
       EX - EIGRP external, ND - ND Default, NDp - ND Prefix, DCE - Destination
       NDr - Redirect, O - OSPF Intra, OI - OSPF Inter, OE1 - OSPF ext 1
       OE2 - OSPF ext 2, ON1 - OSPF NSSA ext 1, ON2 - OSPF NSSA ext 2
       a - Application, m - OMP
C   2001:DB8:1111:1::/64 [0/0]
     via GigabitEthernet0/0/0, directly connected
C   2001:DB8:1111:12::/64 [0/0]
     via GigabitEthernet0/0/1, directly connected
```

## Generating a Unique Interface ID Using Modified EUI-64

IOS supports two methods to configure a permanent, stable router IPv6 interface address. Examples 27-1 and 27-2 show the first method, in which you configure the entire 128-bit address. The second method uses the same **ipv6 address** command, but you configure only the 64-bit IPv6 prefix for the interface, letting the router automatically generate a unique IID.

To generate the second half of the address, this second method uses rules called *modified EUI-64* (extended unique identifier) or simply **EUI-64**. To use this method, add the **eui-64**

keyword to the end of the command. The router then uses EUI-64 rules to create the IID part of the address, as follows:

Key Topic

1. Split the 6-byte (12-hex-digit) MAC address into two halves (6 hex digits each).
2. Insert FFFE between the two, making the interface ID 16 hex digits (64 bits).
3. Invert the seventh bit of the IID.

Figure 27-4 shows a visual representation of the modified EUI-64 process.

Key Topic

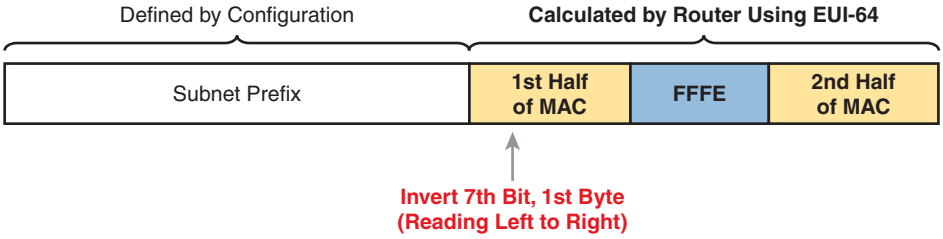



**Figure 27-4** *IPv6 Address Format with Interface ID and EUI-64*

**NOTE** You can find a video about the EUI-64 process on the companion website in the Chapter Review section for this chapter.

Although this process might seem a bit convoluted, it works. Also, with a little practice, you can look at an IPv6 address and quickly notice the FFFE in the middle of the IID and then easily find the two halves of the corresponding interface's MAC address. But you need to be ready to do the same math, in this case to predict the modified EUI-64 formatted IPv6 address on an interface.

Consider the two different examples in Figure 27-5, one on the left and another on the right. Both show all the work except the step to invert the 7th bit. Both start with the MAC address, breaking it into two halves (Step 2). The third step inserts FFFE in the middle, and the fourth step inserts a colon every four hex digits, keeping with IPv6 conventions.

Key Topic

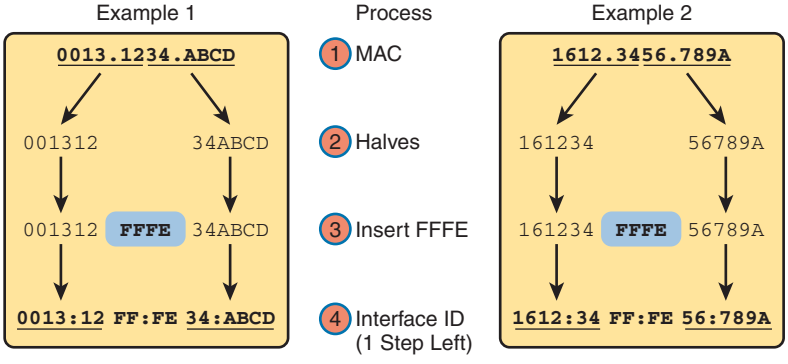



**Figure 27-5** *Two Partial Examples of the EUI-64 Interface ID Process*

To complete the modified EUI-64 process, invert the 7th bit. Following the logic in Figure 27-6, convert the first byte (first two hex digits) from hex to binary. Then invert the seventh of the 8 bits: If it is a 0, make it a 1, or if it is a 1, make it a 0. Then convert the 8 bits back to two hex digits.

Key Topic

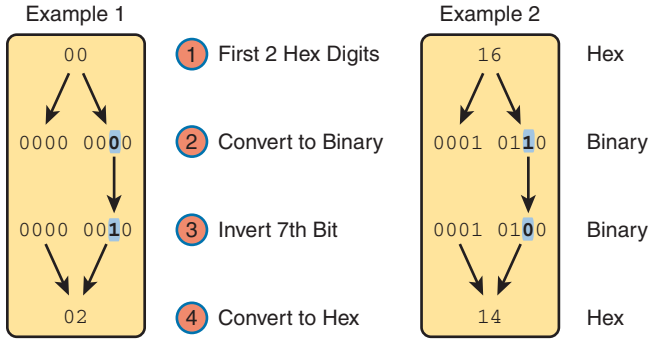



**Figure 27-6** *Inverting the Seventh Bit of an EUI-64 Interface ID Field*

**NOTE** If you do not remember how to do hex-to-binary conversions, take a few moments to review the process. The conversion can be easy if you memorize the 16 hex values for digits 0 through F, with the corresponding binary values. If you do not have those handy in your memory, take a few moments to look at Table A-2 in Appendix A, "Numeric Reference Tables."

For those who prefer decimal shortcuts, you can do the bit-flip math without doing any hex-binary conversions with a little memorization. First, note that the process to invert the seventh bit, when working with a hexadecimal IPv6 address, flips the third of four bits one hex digit. With only 16 possible hex digits, you could memorize what each hex digit becomes if its third bit is inverted, and you can quickly memorize those values with Figure 27-7. To internalize the concepts, redraw the figure several times with these instructions. Just use any piece of scrap paper and use these steps:

**Step 1.** Write hex digits 0 through F, but arrange the digits as shown on the left of the figure, with spacing as shown. (You do not have to draw the arrow lines or the text "A Little Space"—those are instructions for you.)

**Step 2.** Draw lines to connect the nearby pairs of hex digits.

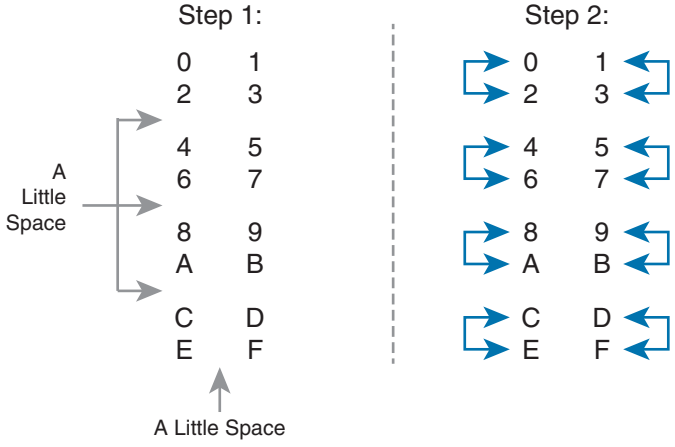



**Figure 27-7** *A Mnemonic Device to Help Memorize Bit Inversion Shortcut*

When inverting the 7th bit, first remember that the 7th bit is in the 2nd hex digit. Then, refer to the right side of Figure 27-7 (or your version), and change the 2nd hex digit based on those pairs of digits. That is, 0 converts to 2; 2 converts to 0; 1 converts to 3; 3 converts to 1; 4 converts to 6; 6 converts to 4; and so on. So, on the exam, if you can arrange the hex digits in the pattern of Figure 27-7 on your notepad, you could avoid doing binary/hexadecimal conversion. Use whichever approach makes you more comfortable.

As usual, the best way to get comfortable forming these modified EUI-64 IIDs is to calculate some yourself. Table 27-2 lists some practice problems, with an IPv6 64-bit prefix in the first column and the MAC address in the second column. Your job is to calculate the full (unabbreviated) IPv6 address using modified EUI-64 rules. The answers are at the end of the chapter, in the section "Answers to Earlier Practice Problems."

**Table 27-2** IPv6 EUI-64 Address Creation Practice

| Prefix | MAC Address | Unabbreviated IPv6 Address |
|---|---|---|
| 2001:DB8:1:1::/64 | 0013.ABAB.1001 | |
| 2001:DB8:1:1::/64 | AA13.ABAB.1001 | |
| 2001:DB8:1:1::/64 | 000C.BEEF.CAFE | |
| 2001:DB8:1:1::/64 | B80C.BEEF.CAFE | |
| 2001:DB8:FE:FE::/64 | 0C0C.ABAC.CABA | |
| 2001:DB8:FE:FE::/64 | 0A0C.ABAC.CABA | |



The **ipv6 address** *prefix/prefix-length* **eui-64** interface subcommand has two key differences with the earlier examples of the **ipv6 address** command. First, the **eui-64** keyword tells the router to build the IID using modified EUI-64 rules. Second, the command should not list an address but instead a prefix, because the router will calculate the IID.

Example 27-5 shows an example that converts Router R1 from earlier Example 27-1 to use EUI-64 for the interface IIDs. It uses the same subnet prefixes as per the earlier example (based on Figure 27-3).

**Example 27-5** *Configuring R1's IPv6 Interfaces Using EUI-64*

```
ipv6 unicast-routing
!
! The ipv6 address command below lists a prefix, not the full address
interface GigabitEthernet0/0/0
 mac-address 0200.1111.1111
 ipv6 address 2001:DB8:1111:1::/64 eui-64
!
interface GigabitEthernet0/0/1
 ipv6 address 2001:DB8:1111:12::/64 eui-64

R1# show ipv6 interface brief
GigabitEthernet0/0/0   [up/up]
    FE80::11FF:FE11:1111
```

```
    2001:DB8:1111:1:0:11FF:FE11:1111
GigabitEthernet0/0/1    [up/up]
    FE80::32F7:DFF:FE29:8568
    2001:DB8:1111:12:32F7:DFF:FE29:8568
GigabitEthernet0/0/2    [administratively down/down]
    unassigned
GigabitEthernet0/0/3    [administratively down/down]
    unassigned
```

For this example, interface G0/0/1 uses its universal MAC address (its burned-in address), but interface G0/0/0 uses a configured MAC address. Following that math:

G0/0/0 – MAC 0200.1111.1111 – IID 0000.11FF.FE11.1111

G0/0/1 – MAC 30F7.0D29.8568 – IID 32F7.0DFF.FE29.8568

Also, be aware that for interfaces that do not have a MAC address, like serial interfaces, the router uses the MAC of the lowest-numbered router interface that does have a MAC.

**NOTE** When you use modified EUI-64, the **ipv6 address** command should list a prefix rather than the full 128-bit IPv6 address. However, suppose you mistakenly type the complete address and still use the **eui-64** keyword. In that case, IOS accepts the command; however, it converts the address to the matching prefix before putting it into the running config file. For example, IOS converts **ipv6 address 2000:1:1:1::1/64 eui-64** to **ipv6 address 2000:1:1:1::/64 eui-64**.

### IPv6 Address Attributes

IOS defines a set of attributes for some IPv6 addresses, displaying those with short all-caps codes in the output from commands like **show ipv6 interface** (but not listed in output from the **show ipv6 interface brief** command). The most common attributes include

**EUI:** The router generated the address using modified EUI-64.

**TEN:** Tentative. A temporary attribute that occurs while the router performs duplicate address detection (DAD).

**CAL:** Calendared. An address with a prescribed lifetime, including a valid and preferred lifetime. They are commonly listed when using DHCP and SLAAC.

**PRE:** Preferred. The preferred address to use on the interface that uses time-based (calendared) addresses. They are commonly listed when using DHCP and SLAAC.

Example 27-6 shows an example using R1's interface G0/0/0 in the previous example. The example shows two consecutive **show ipv6 interface g0/0/0** commands, the first issued immediately after the interface configuration in Example 27-5. At that time, the router had not completed DAD (which takes only a few seconds to complete), so the output lists the TEN attribute along with EUI. The subsequent repeat of the command no longer lists TEN, as DAD completed, but still lists EUI, confirming the router derived the address using EUI-64. (The section "Discovering Duplicate Addresses Using NDP NS and NA" in Chapter 28, "Implementing IPv6 Addressing on Hosts," explains DAD.)

**Example 27-6** *An Example of IPv6 Address Attributes in* **show interfaces**

```
R1# show interfaces g0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
  No Virtual link-local address(es):
  Global unicast address(es):
    2001:DB8:1111:1:0:11FF:FE11:1111, subnet is 2001:DB8:1111:1::/64 [EUI/TEN]
! Lines omitted for brevity
! Next command gathered seconds later...
R1# show interfaces g0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
  No Virtual link-local address(es):
  Global unicast address(es):
    2001:DB8:1111:1:0:11FF:FE11:1111, subnet is 2001:DB8:1111:1::/64 [EUI]
! Lines omitted for brevity
```



## Dynamic Unicast Address Configuration

In most cases, network engineers will statically configure the IPv6 addresses of router interfaces so that the addresses do not change until the engineer changes the router configuration. However, routers can dynamically learn IPv6 addresses. These can be useful for routers connecting to the Internet through some types of Internet access technologies, like cable modems or fiber Internet.

Cisco routers support two ways for the router interface to dynamically learn an IPv6 address to use

- Stateful DHCP
- Stateless Address Autoconfiguration (SLAAC)

Both methods use the familiar **ipv6 address** command. Of course, neither option configures the actual IPv6 address; instead, the commands configure a keyword that tells the router which dynamic method to use to learn its IPv6 address. Example 27-7 shows the configuration, with one interface using stateful DHCP and one using SLAAC.

**Example 27-7** *Router Configuration to Learn IPv6 Addresses with DHCP and SLAAC*

```
! This interface uses DHCP to learn its IPv6 address
interface GigabitEthernet0/0/0
 ipv6 address dhcp
!
! This interface uses SLAAC to learn its IPv6 address
interface GigabitEthernet0/0/1
 ipv6 address autoconfig
```

Endpoints often use one of the dynamic methods to learn their IPv6 address and related settings. Chapter 28, "Implementing IPv6 Addresses on Hosts," discusses the specifics of DHCP and SLAAC.

## Special Addresses Used by Routers

IPv6 configuration on a router begins with the simple steps discussed in the first part of this chapter. After you configure the **ipv6 unicast-routing** global configuration command, to enable the function of IPv6 routing, the addition of a unicast IPv6 address on an interface causes the router to do the following:




- Gives the interface a unicast IPv6 address
- Enables the routing of IPv6 packets in/out that interface
- Defines the IPv6 prefix (subnet) that exists off that interface
- Tells the router to add a connected IPv6 route for that prefix, to the IPv6 routing table, when that interface is up/up

**NOTE** In fact, if you pause and look at the list again, the same ideas happen for IPv4 when you configure an IPv4 address on a router interface.

While all the IPv6 features in this list work much like similar features in IPv4, IPv6 also has a number of additional functions not seen in IPv4. Often, these additional functions use other IPv6 addresses, many of which are multicast addresses. This second major section of the chapter examines the additional IPv6 addresses seen on routers, with a brief description of how they are used.

### Link-Local Addresses

IPv6 uses **link-local addresses (LLA)** as a special kind of unicast IPv6 address. Every interface that supports IPv6 must have an LLA. A small number of important IPv6 protocols then use LLAs to send and receive packets to help with overhead IPv6 functions, like neighbor and address discovery—sometimes even before the device has a GUA or ULA to use. This next topic first looks at how IPv6 uses LLAs and then how routers create them.

#### Link-Local Address Concepts

LLAs provide every device that supports IPv6 the capability to send and receive unicast packets on the local link, before dynamically learning or being configured with their routable IPv6 GUA or ULA. While hosts could send and receive application data using LLAs over a single subnet, LLAs do not exist to support normal application data. Instead, LLAs exist to support some overhead IPv6 protocols—protocols that need some working address to use.

The IPv6 designers wanted better options than IPv4 to support protocols that need to send packets before hosts have their normal unicast address assignment. For instance, before an IPv4 host leases an address with DHCP, it needs to send DHCP messages inside IP packets. To do that, IPv4 defines special addresses 0.0.0.0 and 255.255.255.255. The IPv6 designers looked for a better way, and LLAs provide a solution in some cases.

Hosts self-generate an LLA for themselves and can then use their LLA to send and receive IPv6 packets on the local link. All interfaces that use IPv6 must have an LLA, with the option for the device to dynamically create it without outside help. Once created, the host ensures no other hosts use the same LLA by using duplicate address detection (DAD). If unique, the host keeps using the LLA, but if DAD finds another host using the same LLA, the new host picks a different LLA. Some IPv6 protocols rely on using a host's LLA as the source of IPv6 protocol messages, including Neighbor Discovery Protocol (NDP, which replaces IPv4 ARP) and Dynamic Host Configuration Protocol (DHCP).

LLAs also appear in a surprising place: as the next-hop IPv6 addresses in IPv6 routes, as shown in Figure 27-8. For routes learned by a routing protocol, the **show ipv6 route** command lists the LLA of the neighboring router, rather than the GUA or ULA. For the default router setting on a host, the host refers to the router's LLA.

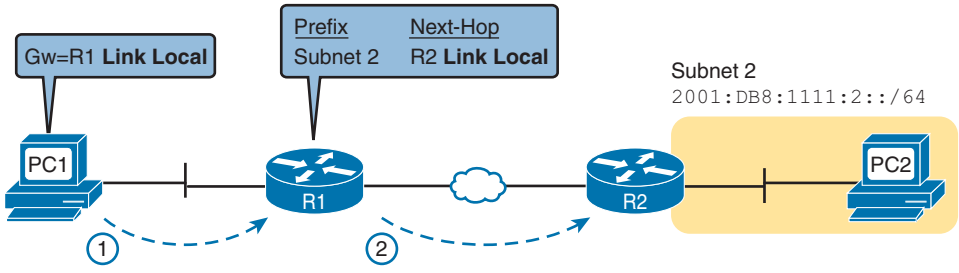



**Figure 27-8** *IPv6 Using Link-Local Addresses as the Next-Hop Address*

The following list summarizes some key facts about link-local addresses:

Key Topic

**Unicast (not multicast):** LLAs represent a single host, and packets sent to a link-local address should be processed by only that one IPv6 host.

**Forwarding scope is the local link only:** Packets sent to a link-local address do not leave the local data link because routers do not forward packets with link-local destination addresses.

**Automatically self-generated:** Every IPv6 host interface (and router interface) can automatically create its own link-local address, solving some initialization problems for hosts before they learn a dynamically learned global unicast address.

**Common uses:** Link-local addresses are used for some overhead protocols that stay local to one subnet and as the next-hop address for IPv6 routes.

### Creating Link-Local Addresses on Routers

By default, routers self-assign an LLA to each IPv6-enabled interface. To do so, the router sets the first half of the address to FE80:0000:0000:0000, as shown on the left side of Figure 27-9. Then the router creates the IID using modified EUI-64 rules (see the earlier section "Generating a Unique Interface ID Using Modified EUI-64" for a review of the process). As a result, a router's complete LLA should be unique because the MAC address that feeds into the EUI-64 process should be unique.

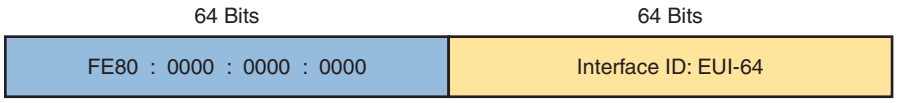



**Figure 27-9** *Link-Local Address Format*

In practice, LLAs begin with FE80, but formally, RFC 4291 reserves prefix FE80::/10, which includes addresses that begin FE8, FE9, FEA, or FEB. However, that same RFC also dictates that the first 64 bits should be FE80:0000:0000:0000. So in practice, LLAs begin with FE80:0000:0000:0000.

When a router automatically generates an interface's LLA, it does not create a matching configuration command. To see the LLA, instead of the **show running-config** command, just use the usual commands that also list the unicast IPv6 address: **show ipv6 interface** and **show ipv6 interface brief**. Example 27-8 shows an example from Router R1 just after it was configured as shown in Example 27-5 (with the **eui-64** keyword on the **ipv6 address** commands).

**Example 27-8** *Comparing Link-Local Addresses with EUI-Generated Unicast Addresses*

```
R1# show ipv6 interface brief
GigabitEthernet0/0/0    [up/up]
    FE80::11FF:FE11:1111
    2001:DB8:1111:1:0:11FF:FE11:1111
GigabitEthernet0/0/1    [up/up]
    FE80::32F7:DFF:FE29:8568
    2001:DB8:1111:12:32F7:DFF:FE29:8568
GigabitEthernet0/0/2    [administratively down/down]
    unassigned
GigabitEthernet0/0/3    [administratively down/down]
    unassigned
```

First, examine the two pairs of highlighted entries in the example. For each of the two interfaces that have a global unicast address (G0/0/0 and G0/0/1), the output lists the GUAs, which happen to begin with 2001 in this case. At the same time, the output also lists the LLAs for each interface, beginning with FE80.

Next, focus on the two addresses listed under interface G0/0/0. If you look closely, you will see that both addresses have the same IID value. Because the GUA configuration used the **ipv6 address 2001:DB8:1111:1::/64 eui-64** command, the router used modified EUI-64 logic to form the IID portion of both the GUA and LLA. For both addresses, the router used the interface MAC address of 0200.1111.1111 and calculated the IID portion of both addresses as 0000:11FF:FE11:1111 (unabbreviated). After abbreviation, Router R1's LLA on interface G0/0/0 becomes FE80::11FF:FE11:1111.

**NOTE** While Cisco routers use the modified EUI-64 method to create the IID for LLAs, most end-user device OSs do not. Instead, to reduce security exposures, they randomly assign IPv6 address IIDs. Chapter 28, which focuses on host IPv6 address assignment, provides additional details and examples.

Cisco IOS also allows the direct configuration of the interface's LLA, with that configured LLA taking precedence, as follows:

- The **ipv6 address** *address* **link-local** interface subcommand configures the LLA; in this case, the router does not create an LLA using EUI-64 rules.
- If not configured, the IOS calculates the LLA using EUI-64 rules.

### Routing IPv6 with Only Link-Local Addresses on an Interface

This chapter has shown four variations on the **ipv6 address** command to configure routable addresses (GUAs and ULAs) so far. To review:

**ipv6 address** *address/prefix-length*: Static configuration of the entire routable address and prefix length

**ipv6 address** *prefix/prefix-length* **eui-64**: Static configuration of the prefix and prefix length, with the router calculating the IID using EUI-64 rules

**ipv6 address dhcp**: Dynamic learning of the address and prefix length using DHCP

**ipv6 address autoconfig**: Dynamic learning of the prefix and prefix length, with the router calculating the IID using EUI-64 rules (SLAAC)



To complete the list, now consider the **ipv6 enable** interface subcommand. It enables IPv6 processing on the interface and causes the router to self-assign an LLA, but it leaves the interface with no routable (GUA or ULA) address.

The purpose of the **ipv6 enable** command will not make sense until you realize that some links, particularly WAN links, do not need a GUA or ULA. Using the backdrop of Figure 27-10, think about the destination of packets sent by hosts like PC1 and PC2. When PC1 sends PC2 an IPv6 packet, the packet holds PC1's and PC2's IPv6 addresses and never contains the WAN link's IPv6 addresses. The hosts do not need to know the routers' WAN interface addresses. (That same logic applies with IPv4 as well.)

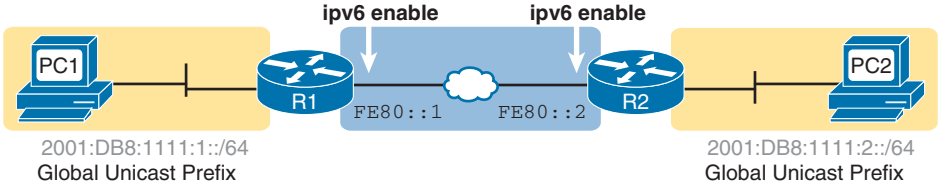



**Figure 27-10** *Typical Use of the* **ipv6 enable** *Command*

Additionally, the routers do not need GUAs or ULAs on the WAN links for routing to work; they only need LLAs. For instance, IPv6 routing protocols use LLAs as the next-hop address when dynamically building IPv6 routes. Additionally, as discussed in Chapter 29, "Implementing IPv6 Routing," static routes can use LLAs for the next-hop address.

In short, IPv6 routing works on WAN links that use only LLAs, with no GUAs or ULAs. As a result, you would not even need to assign an IPv6 subnet prefix to each WAN link. Then to configure the WAN interfaces, use the **ipv6 enable** command, enabling IPv6 and giving each interface a generated link-local IPv6 address.

To use the command, just configure the **ipv6 enable** command on the interfaces on both ends of the WAN link.

## IPv6 Multicast Addresses

IPv6 uses multicast IPv6 addresses for several purposes. Like IPv4, IPv6 includes a range of multicast addresses that can be used by multicast applications, with many of the same fundamental concepts as IPv4 multicasts. For instance, IANA defines the range FF30::/12 (all IPv6 addresses that begin with FF3) as the range of addresses used for multicast applications.

Additionally, different IPv6 RFCs reserve multicast addresses for specific purposes. For instance, OSPFv3 uses FF02::5 and FF02::6 as the all-OSPF-routers and all-DR-Routers multicast addresses, respectively, similar to how OSPFv2 uses IPv4 addresses 224.0.0.5 and 224.0.0.6. IANA and various RFCs use the term *well-known multicast address* to refer to these reserved predefined multicast addresses. IANA reserves address range FF00::/12 (all IPv6 addresses that begin FF0) for these addresses.

This next section focuses on the well-known permanent IPv6 multicast addresses. The first, **link-local multicast addresses**, have a scope that limits them to flow over a single link. The other type is a special overhead multicast address calculated for each host, called the **solicited-node multicast address**.

### Well-Known Multicast Addresses

Stop for a moment and think about some of the control plane protocols discussed throughout this book so far. Some IPv4 control plane protocols use IPv4 broadcasts, meaning that the packet destination address was either 255.255.255.255 (the address for all hosts in the local subnet) or the subnet broadcast address (the address for all hosts in that specific subnet). The sender encapsulates the packet in an Ethernet broadcast frame, destined to the Ethernet broadcast address of FFFF.FFFF.FFFF.

While useful, the IPv4 approach of IPv4 broadcast and LAN broadcast requires every host in the VLAN to process the broadcast frame, even if only one other device needs to think about the message. LAN switches forward LAN broadcasts out every port in the same VLAN, so copies of each LAN broadcast arrive at each host in the subnet. As a result, each host has to process the frame, then the packet, read the type of message, and so on, before choosing to ignore the packet. For example, an IPv4 ARP Request—an IPv4 and LAN broadcast—requires a host to process the Ethernet and ARP details of the message before deciding whether to reply or not.

IPv6, instead of using Layer 3 and Layer 2 broadcasts, uses Layer 3 multicast addresses, which in turn causes Ethernet frames to use Ethernet multicast addresses. LAN switches can use a long-established multicast optimization feature that lets them forward multicast frames to only the hosts that care to receive a copy while not forwarding frames to disinterested hosts. As a result, the IPv6 protocols work more efficiently with fewer unnecessary interruptions to hosts.

For instance, OSPFv3 uses IPv6 multicast addresses FF02::5 and FF02::6. In a subnet, the OSPFv3 routers, when they initialize OSPF on an interface, send messages to register their interest in receiving packets sent to FF02::5 and FF02::6. However, hosts do not register an

interest in receiving packets sent to those addresses. The LAN switches watch for those multicast registration messages and recall which ports connect to hosts interested in packets sent to each multicast address. When receiving frames sent to those multicast addresses, the switches forward them only to the OSPFv3 routers that preregistered to receive them, but no other hosts/routers.

All the IPv6 multicast addresses can benefit from the multicast optimizations. For reference, Table 27-3 lists the most common well-known IPv6 multicast addresses.

**Table 27-3** Key IPv6 Local-Scope Multicast Addresses

| Short Name | Multicast Address | Meaning | IPv4 Equivalent |
|---|---|---|---|
| **All-nodes** | FF02::1 | All-nodes (all interfaces that use IPv6 that are on the link) | 224.0.0.1 |
| **All-routers** | FF02::2 | All-routers (all IPv6 router interfaces on the link) | 224.0.0.2 |
| All-OSPF, All-OSPF-DR | FF02::5, FF02::6 | All OSPF routers and all OSPF-designated routers, respectively | 224.0.0.5, 224.0.0.6 |
| RIPng Routers | FF02::9 | All RIPng routers | 224.0.0.9 |
| EIGRPv6 Routers | FF02::A | All routers using EIGRP for IPv6 (EIGRPv6) | 224.0.0.10 |
| DHCP Relay Agent | FF02::1:2 | All routers acting as a DHCPv6 relay agent | None |



**NOTE** An Internet search of "IPv6 Multicast Address Space Registry" will show the IANA page that lists all the reserved values and the RFC that defines the use of each address.

Example 27-9 repeats the output of the **show ipv6 interface** command to show the multicast addresses used by Router R1 on its G0/0/0 interface. In this case, the highlighted lines show the all-nodes address (FF02::1), all-routers (FF02::2), and two for OSPFv3 (FF02::5 and FF02::6). The highlighted section beginning with the heading "Joined group address(es)" lists the multicast addresses joined by the router on this interface.

**Example 27-9** *Verifying Static IPv6 Addresses on Router R1*

```
R1# show ipv6 interface GigabitEthernet 0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
  No Virtual link-local address(es):
  Global unicast address(es):
    2001:DB8:1111:1::1, subnet is 2001:DB8:1111:1::/64
  Joined group address(es):
    FF02::1
    FF02::2
```

```
    FF02::5
    FF02::6
    FF02::1:FF00:1
    FF02::1:FF11:1111
! Lines omitted for brevity
```

### Multicast Address Scopes

IPv6 RFC 4291 defines IPv6 unicast and multicast addressing details, including the ideas of **IPv6 address scope**. Each scope defines a different set of rules about whether routers should or should not forward a packet, and how far routers should forward packets, based on those scopes.

For instance, an LLA—a unicast IPv6 address—has a **link-local scope**. The scope definition called "link-local" dictates that packets sent to a link-local unicast address should remain on the link and not be forwarded by any router.

Most of the scope discussion in RFC 4291 applies to multicast addresses, using the term *multicast scope*. Per that RFC, the fourth digit of the multicast address identifies the scope, as noted in Table 27-4.

Key Topic

**Table 27-4** IPv6 Multicast Scope Terms

| Scope Name | Fourth Hex Digit | Scope Defined by... | Meaning |
|---|---|---|---|
| **Interface-Local** | 1 | Derived by Device | The packet remains within the device. Useful for internally sending packets to services running on that same host. |
| Link-Local | 2 | Derived by Device | The host that creates the packet can send it onto the link, but no routers forward it. |
| **Site-Local** | 5 | Configuration on Routers | Intended to be more than Link-Local, so routers forward, but must be less than Organization-Local; generally meant to limit packets so they do not cross WAN links. |
| **Organization-Local** | 8 | Configuration on Routers | Intended to be broad, probably for an entire company or organization. Must be broader than Site-Local. |
| Global | E | No Boundaries | No boundaries. |

Breaking down the concepts a little further, packets sent to a multicast address with a link-local scope should stay on the local link, that is, the local subnet. Hosts know they can process a link-local packet if received, as do routers. However, routers know to not route the packet to other subnets because of the scope. Packets with an organization-local scope should be routed inside the organization but not out to the Internet or over a link to another company. (Note that routers can predict the boundaries of some scopes, like

link-local, but they need configuration to know the boundaries of other scopes, for instance, organization-local.)

Comparing a few of the scopes in terms of where the packets can flow, the higher the value in the fourth hex digit, the further away from the sending host the scope allows the packet to be forwarded. Table 27-4 shows that progression top to bottom, while Figure 27-11 shows an example with three scopes: link-local, site-local, and organization-local. In the figure, site-local messages do not cross the WAN, and organization-local messages do not leave the organization over the link to the Internet.

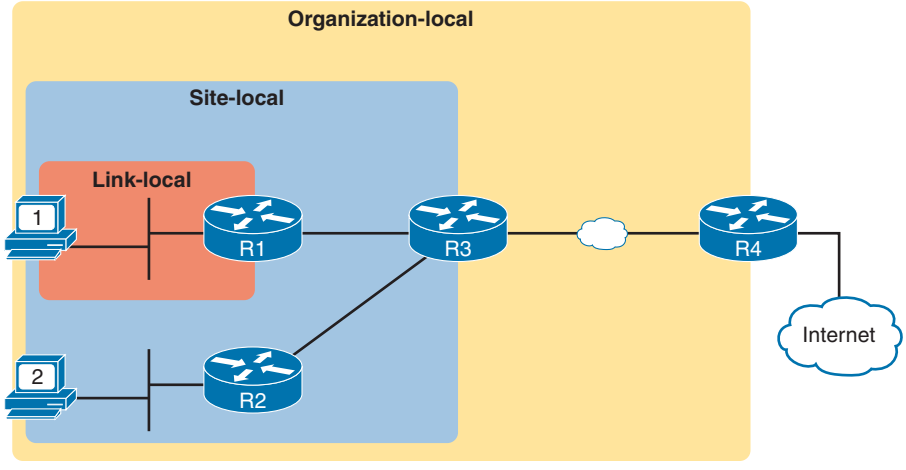



**Figure 27-11** *IPv6 Multicast Scopes*

Finally, the term *link-local* has a couple of common uses in IPv6 and can be confusing. The following descriptions should clarify the different uses of the term:

Key Topic

**Link-local address:** A unicast IPv6 address that begins FE80 with a link-local scope. Devices create their own LLAs. A more complete term for comparison would be *link-local unicast address*.

**Link-local multicast address:** An IPv6 address that begins with FF02. These addresses serve as reserved multicast addresses to which devices apply a link-local scope.

**Link-local scope:** A reference to the scope itself, rather than an address. This scope defines that routers should not forward packets sent to an address in this scope.

## Solicited-Node Multicast Addresses

IPv6 Neighbor Discovery Protocol (NDP) replaces IPv4 ARP, as discussed in Chapter 28. NDP improves the MAC-discovery process compared to IPv4 ARP by sending IPv6 multicast packets to communicate with a second host before the first host knows the second host's MAC address. The process uses the solicited-node multicast address associated with the unicast IPv6 address.

Figure 27-12 shows how to determine the solicited-node multicast address associated with a unicast address. Start with the predefined /104 prefix (26 hex digits) shown in

Figure 27-12. In other words, all the solicited-node multicast addresses begin with the abbreviated FF02::1:FF. In the last 24 bits (6 hex digits), copy the last 6 hex digits of the unicast address into the solicited-node address.

Key Topic

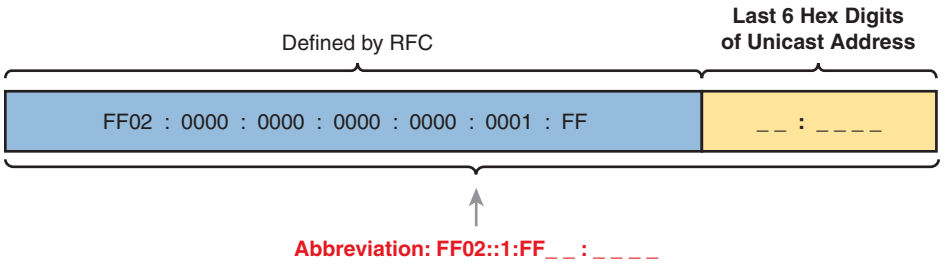



**Figure 27-12** *Solicited-Node Multicast Address Format*

Note that a host or router calculates and uses a matching solicited-node multicast address for every unicast address (GUA, ULA, and LLA) on an interface. The device then registers to receive incoming multicast messages sent to that address. Example 27-10 shows an example in which the router interface has a GUA of 2001:DB8:1111:1::1/64 and an LLA of FE80::11FF:FE11:1111. (You saw these addresses earlier in Examples 27-1 and 27-3.) As a result, the interface has two solicited-node multicast addresses, shown at the end of the output.

**Example 27-10** *Verifying Static IPv6 Addresses on Router R1*

```
R1# show ipv6 interface GigabitEthernet 0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
  No Virtual link-local address(es):
  Global unicast address(es):
    2001:DB8:1111:1::1, subnet is 2001:DB8:1111:1::/64
  Joined group address(es):
    FF02::1
    FF02::2
    FF02::5
    FF02::1:FF00:1
    FF02::1:FF11:1111
 ! Lines omitted for brevity
```

Note that in this case, R1's expanded GUA ends with its last six hex digits as 00:0001, resulting in an unabbreviated solicited-node multicast address of FF02:0000:0000:0000:0000:0001:FF00:0001. This value begins with the 26-hex-digit prefix shown in Figure 27-12, followed by 00:0001. The solicited-node multicast address corresponding to link-local address FE80::11FF:FE11:1111 ends in 11:1111, as shown in the last line of the example.

## The Unspecified and Loopback Addresses

All IPv6 hosts can use two additional special addresses:

Key Topic

- The unspecified IPv6 address, ::, that is, all 0s
- The loopback IPv6 address, ::1, that is, 127 binary 0s with a single 1

Seldom used, a host can use the unspecified address when it has not yet learned a unicast address to use. The address can be used only as a source address, and only used before the host has an LLA or other unicast address to use.

The IPv6 loopback address gives each IPv6 host a way to test its protocol stack. Just like the IPv4 127.0.0.1 loopback address, packets sent to ::1 do not leave the host but are simply delivered down the stack to IPv6 and back up the stack to the application on the local host.

## Anycast Addresses

Imagine that routers collectively need to implement some service. Rather than have one router supply that service, that service works best when implemented on several routers. But the hosts that use the service need to contact only the nearest such service, and the network wants to hide all these details from the hosts. Hosts can send just one packet to an IPv6 address, and the routers will forward the packet to the nearest router that supports that service by virtue of supporting that destination IPv6 address.



IPv6 **anycast addresses** provide that exact function. The *any* part of the name refers to the fact that any of the instances of the service can be used. Figure 27-13 shows this big concept, with two major steps:

**Step 1.** Two routers configure the exact same IPv6 address, designated as an anycast address, to support some service.

**Step 2.** In the future, when any router receives a packet for that anycast address, the other routers simply route the packet to the nearest of the routers that support the address.

Key Topic

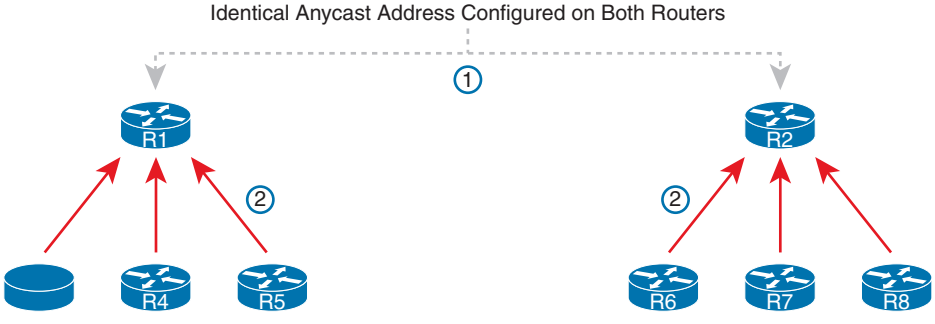



**Figure 27-13** *IPv6 Anycast Addresses*

To make this anycast process work, the routers implementing the anycast address must be configured and then advertise a route for the anycast address. The addresses do not come from a special reserved range of addresses; instead, they are from the unicast address range.

Often, the address is configured with a /128 prefix so that the routers advertise a host route for that one anycast address. At that point, the routing protocol advertises the route just like any other IPv6 route; the other routers cannot tell the difference.

Example 27-11 shows a sample configuration on a router. Note that the actual address (2001:1:1:2::99) looks like any other unicast address. However, note the different **anycast** keyword on the **ipv6 address** command, telling the local router that the address has a special purpose as an anycast address. Finally, note that the **show ipv6 interface** command does identify the address as an anycast address, but the **show ipv6 interface brief** command does not.

**Example 27-11** *Configuring and Verifying IPv6 Anycast Addresses*

```
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# interface gigabitEthernet 0/0/0
R1(config-if)# ipv6 address 2001:1:1:1::1/64
R1(config-if)# ipv6 address 2001:1:1:2::99/128 anycast
R1(config-if)# ^Z
R1#

R1# show ipv6 interface g0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
    IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
    No Virtual link-local address(es):
    Global unicast address(es):
       2001:1:1:1::1, subnet is 2001:1:1:1::/64
       2001:1:1:2::99, subnet is 2001:1:1:2::99/128 [ANY]
 ! Lines omitted for brevity
R1# show ipv6 interface brief g0/0/0
GigabitEthernet0/0/0 [up/up]
     FE80::11FF:FE11:1111
     2001:1:1:1::1
     2001:1:1:2::99
```

**NOTE** The term **subnet-router anycast address** represents a different concept than the anycast address just discussed. Instead, the *subnet-router anycast address* is one special anycast address in each subnet. It is reserved for routers to send a packet to any router on the subnet. The address's value in each subnet is the same number as the subnet ID; the address has the same prefix value as the other addresses and all binary 0s in the interface ID.

## IPv6 Addressing Configuration Summary

This chapter completes the discussion of various IPv6 address types while showing how to enable IPv6 on interfaces. Many implementations will use the **ipv6 address** command on each router LAN interface, and either that same command or the **ipv6 enable** command on

the WAN interfaces. For exam prep, Table 27-5 summarizes the various commands and the automatically generated IPv6 addresses in one place for review and study.

Key Topic

**Table 27-5** Summary of IPv6 Address Types and the Commands That Create Them

| Type | Prefix/Address Notes | Enabled with What Interface Subcommand |
|---|---|---|
| Global unicast | Many prefixes | **ipv6 address** *address/prefix-length*<br>**ipv6 address** *prefix/prefix-length* **eui-64** |
| Unique local | FD00::/8 | **ipv6 address** *address/prefix-length*<br>**ipv6 address** *prefix/prefix-length* **eui-64** |
| Link local | FE80::/10 | **ipv6 address** *address* **link-local**<br>Autogenerated by all **ipv6 address** commands<br>Autogenerated by the **ipv6 enable** command |
| All hosts multicast | FF02::1 | Autogenerated by all **ipv6 address** commands |
| All routers multicast | FF02::2 | Autogenerated by all **ipv6 address** commands |
| Routing protocol multicasts | Various | Added to the interface when the corresponding routing protocol is enabled on the interface |
| Solicited-node multicast | FF02::1:FF /104 | Autogenerated by all **ipv6 address** commands |



## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 27-6 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 27-6** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Review memory tables | | Website |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 27-7** Key Topics for Chapter 27

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 27-2 | Conceptual drawing about the need for dual stacks for the foreseeable future | 671 |
| List | Rules for creating an IPv6 address using EUI-64 rules | 675 |
| Figure 27-4 | IPv6 EUI-64 Address Format and Rules | 675 |
| Figure 27-5 | Conceptual drawing of how to create an IPv6 address using EUI-64 rules | 675 |
| Figure 27-6 | Example of performing the bit inversion when using EUI-64 | 676 |
| List | Functions IOS enables when an IPv6 address is configured on a working interface | 680 |
| List | Key facts about IPv6 link-local addresses | 681 |
| Table 27-4 | Link-local scope terms and meanings | 686 |
| List | Comparisons of the use of the term *link-local* | 687 |
| Figure 27-12 | Conceptual drawing of how to make a solicited-node multicast address | 688 |
| List | Other special IPv6 addresses | 689 |
| Figure 27-13 | Concept of IPv6 anycast addresses | 689 |
| Table 27-5 | IPv6 address summary with the commands that enable each address type | 691 |

## Key Terms You Should Know

all-nodes multicast address, all-routers multicast address, anycast address, dual stack, EUI-64, interface-local scope, IPv6 address scope, link-local address (LLA), link-local multicast address, link-local scope, organization-local scope, site-local scope, solicited-node multicast address, subnet-router anycast address

## Additional Practice for This Chapter's Processes

For additional practice with IPv6 abbreviations, you may do the same set of practice problems using your choice of tools:

For additional practice with calculating IPv6 addresses using EUI-64 rules and finding the solicited-node multicast address based on a unicast address, use the exercises in Appendix I, "Practice for Chapter 27: Implementing IPv6 Addressing on Routers." You have two options to use:

**PDF:** Navigate to the companion website and open the PDF for Appendix I.

**Application:** Navigate to the companion website and open the application "Practice Exercise: EUI-64 and Solicited-Node Multicast Problems."

Additionally, you can create your own problems using any real router or simulator: Get into the router CLI, into configuration mode, and configure the **mac-address** *address* and

**ipv6 address** *prefix*/**64 eui-64** command. Then predict the IPv6 unicast address, link-local address, and solicited-node multicast address; finally, check your predictions against the **show ipv6 interface** command.

## Command References

Tables 27-8 and 27-9 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 27-8** Chapter 27 Configuration Command Reference

| Command | Description |
|---|---|
| **ipv6 unicast-routing** | Global command that enables IPv6 routing on the router |
| **ipv6 address** *ipv6-address*/*prefix-length* | Interface subcommand that manually configures the entire interface IP address and prefix length |
| **ipv6 address** *ipv6-prefix*/*prefix-length* **eui-64** | Interface subcommand that manually configures an IPv6 prefix and prefix length, with the router building the EUI-64 format interface ID automatically |
| **ipv6 address** *ipv6-address*/*prefix-length* [**anycast**] | Interface subcommand that manually configures an address to be used as an anycast address |
| **ipv6 enable** | Command that enables IPv6 on an interface and generates a link-local address |
| **ipv6 address dhcp** | Interface subcommand that enables IPv6 on an interface, causes the router to use DHCP client processes to try to lease an IPv6 address, and creates a link-local address for the interface |
| **ipv6 address autoconfig** | Interface subcommand that tells the router to use SLAAC to find/build its interface IPv6 address |

**Table 27-9** Chapter 27 EXEC Command Reference

| Command | Description |
|---|---|
| **show ipv6 route [connected] [local]** | Lists all IPv6 routes, or the connected routes only, or the local routes only |
| **show ipv6 interface** [*type number*] | Lists IPv6 settings, including link-local and other unicast IP addresses, on all interfaces (or for the listed interface) |
| **show ipv6 interface brief** [ *type number* ] | Lists IPv6 interface status and unicast IPv6 addresses for all interfaces (or for only the listed interface if included) |

## Answers to Earlier Practice Problems

Table 27-2, earlier in this chapter, listed several practice problems in which you needed to calculate the IPv6 address based on EUI-64 rules. Table 27-10 lists the answers to those problems.

**Table 27-10** Answers to IPv6 EUI-64 Address Creation Practice

| Prefix | MAC Address | Unabbreviated IPv6 Address |
|---|---|---|
| 2001:DB8:1:1::/64 | 0013.ABAB.1001 | 2001:0DB8:0001:0001:0213:ABFF:FEAB:1001 |
| 2001:DB8:1:1::/64 | AA13.ABAB.1001 | 2001:0DB8:0001:0001:A813:ABFF:FEAB:1001 |
| 2001:DB8:1:1::/64 | 000C.BEEF.CAFE | 2001:0DB8:0001:0001:020C:BEFF:FEEF:CAFE |
| 2001:DB8:1:1::/64 | B80C.BEEF.CAFE | 2001:0DB8:0001:0001:BA0C:BEFF:FEEF:CAFE |
| 2001:DB8:FE:FE::/64 | 0C0C.ABAC.CABA | 2001:0DB8:00FE:00FE:0E0C:ABFF:FEAC:CABA |
| 2001:DB8:FE:FE::/64 | 0A0C.ABAC.CABA | 2001:0DB8:00FE:00FE:080C:ABFF:FEAC:CABA |

*This page intentionally left blank*



CHAPTER 28

# Implementing IPv6 Addressing on Hosts

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.8 Configure and verify IPv6 addressing and prefix**

**1.9 Describe IPv6 address types**

**1.9.a Unicast (global, unique local, and link local)**

**1.9.d Modified EUI 64**

IPv6 hosts act like IPv4 hosts in many ways, using similar ideas, similar protocols, and even similar or identical commands for the same purpose. At the same time, IPv6 sometimes takes a different approach than IPv4, using a different solution with a new protocol or command. For example:

- Like IPv4, IPv6 hosts use a unicast address, prefix length (mask), default router, and DNS server.
- Like IPv4, IPv6 uses a protocol to dynamically learn the MAC addresses of other hosts in the same LAN-based subnet.
- Unlike IPv4, IPv6 hosts use the Neighbor Discovery Protocol (NDP) for many functions, including those done by IPv4's ARP.
- Like IPv4, IPv6 hosts can use DHCP to learn their four primary IPv6 settings.
- Unlike IPv4, IPv6 supports a dynamic address assignment process other than DHCP, called Stateless Address Autoconfiguration (SLAAC).

This chapter focuses on the four primary IPv6 settings on hosts: the address, prefix length, default router address, and DNS server address. However, to understand how hosts dynamically learn those addresses, this chapter begins its first major section devoted to NDP, which plays a crucial role in several IPv6 processes. The middle section of the chapter then focuses on how hosts dynamically learn their IPv6 settings with DHCP and SLAAC. The final major section of this chapter looks at the tools to verify a host's IPv6 settings, many of which use the same commands used for IPv4.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on

the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 28-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| The Neighbor Discovery Protocol | 1–4 |
| Dynamic Configuration of Host IPv6 Settings | 5–7 |
| Troubleshooting Host IPv6 Addressing | 8 |

1. PC1, PC2, and Router R1 all connect to the same VLAN and IPv6 subnet. PC1 wants to send its first IPv6 packet to PC2. What protocol message will PC1 use to begin the process of discovering PC2's MAC address?
   a. ARP Request
   b. NDP NS
   c. NDP RS
   d. SLAAC NS
2. Which of the following pieces of information does a router supply in an NDP Router Advertisement (RA) message? (Choose two answers.)
   a. Router IPv6 address
   b. Router hostname
   c. IPv6 prefix(es) on the link
   d. IPv6 address of DHCP server
3. Three routers (R1, R2, and R3) connect to the same VLAN and IPv6 subnet. All three routers have responded to various NDP RS messages with NDP RA messages. Which of the answers best describes the kind of NDP information held in the output of the **show ipv6 neighbors** command on R1?
   a. IPv6 neighbors (both routers and hosts) plus their MAC addresses, without noting which are routers
   b. IPv6 neighbors (both routers and hosts) plus their MAC addresses, also noting which are routers
   c. IPv6 routers, with no information about nonrouters, with no MAC address info
   d. IPv6 routers, with no information about nonrouters, but with MAC address info
4. PC1 and Router R1 connect to the same VLAN and IPv6 subnet. The user of PC1 pings the IPv6 address of a host that sits at a remote site so that the packets flow through R1, PC1's default router. PC1 learned all its IPv6 settings dynamically. Which of the following answers lists a protocol or message that PC1 could have used to learn what IPv6 address to use as its default router?
   a. EUI-64
   b. NDP NS
   c. DAD
   d. NDP RS

5. Host PC1 dynamically learns its IPv6 settings using Stateless Address Autoconfiguration (SLAAC). Which one of PC1's settings is most likely to be learned from the stateless DHCPv6 server?
   a. Host address
   b. Prefix length
   c. Default router address
   d. DNS server address(es)
6. Host PC1 dynamically learns its IPv6 settings using Stateless Address Autoconfiguration (SLAAC). Think about the host's unicast address as two parts: the subnet prefix and the interface ID. Which answers list a way that SLAAC learns or builds the value of the interface ID portion of the host's address? (Choose two answers.)
   a. Learned from a DHCPv6 server
   b. Built by the host using EUI-64 rules
   c. Learned from a router using NDP RS/RA messages
   d. Built by the host using a random value
7. An IPv6 host is configured to use DHCP to lease its IPv6 address. The DHCPv6 server is not on the same link but is located at another site. Which answer describes a mechanism the client and routers use to make the DHCPv6 messages flow between the client and server?
   a. The client sends the DHCPv6 Solicit message to multicast address FF02:1:2.
   b. The client sends the DHCPv6 Solicit message to broadcast address FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF.
   c. The client must learn the DHCPv6 server's unicast address from a local router using NDP messages.
   d. The routers use IPv6 multicast routing to forward the Solicit message, unchanged, to the DHCPv6 server.
8. All routers in the network have global unicast addresses (GUAs) configured on their interfaces. The user of PC1, on a LAN, issues a **traceroute** command for a distant host's address. The command succeeds, listing four lines with IPv6 addresses. Which answer best describes the addresses in the **traceroute** output and its use of link-local addresses (LLAs) and GUAs?
   a. All lines list LLAs with no GUAs.
   b. The first line lists an address that matches PC1's default route.
   c. All lines list GUAs with no LLAs.
   d. The last line lists the GUA of the final router in the route.

# Foundation Topics

## The Neighbor Discovery Protocol

IPv4 and IPv6 define a wide range of control and management functions as part of the Internet Control Message Protocol (ICMP). To support similar features in IPv6, the Internet community created ICMPv6, which defines protocols appropriate for IPv6. (For easier comparison, ICMP for IPv4 is often called ICMPv4.)

The **Neighbor Discovery Protocol (NDP)**, a part of ICMPv6 defined in RFC 4861, provides several vital functions in every IPv6 network. Notably, NDP defines the IPv6 equivalent of the IPv4 ARP function. Some of its functions are

Key Topic

**Neighbor MAC Discovery:** An IPv6 LAN-based host will need to learn the MAC address of other hosts in the same subnet. NDP replaces IPv4's ARP, providing messages that replace the ARP Request and Reply messages.

**Router Discovery:** Hosts learn the IPv6 addresses of the available IPv6 routers in the same subnet.

**Prefix Discovery:** Hosts learn the IPv6 subnet prefix and prefix length that the router(s) expect to exist on the link.

**Duplicate Address Detection (DAD):** Before using an IPv6 address, hosts use NDP to perform a Duplicate Address Detection process to ensure no other host uses the same IPv6 address before attempting to use it.

The next few pages explain the listed features.

## Discovering Neighbor Link Addresses with NDP NS and NA

NDP replaces IPv4 ARP using the *Neighbor Solicitation (NS)* and *Neighbor Advertisement (NA)* messages. The NS acts like an IPv4 ARP Request, asking the host with a particular unicast IPv6 address to send back a reply. The NA message acts like an IPv4 ARP Reply, listing that host's MAC address. The following list summarizes the functions:



Key Topic

**Neighbor Solicitation (NS):** This message asks the host with a particular IPv6 address (the target address) to reply with an NA message that lists its MAC address.

**Neighbor Advertisement (NA):** This message lists the sender's IPv6 and MAC addresses. It can be sent in reply to an NS message; if so, the packet is sent to the IPv6 unicast address of the host that sent the original NS message. A host can also send an unsolicited NA, announcing its IPv6 and MAC addresses, in which case the message is sent to the all-IPv6-hosts local-scope multicast address FF02::1.

**NOTE** With NDP, the word *neighbor* refers to hosts on the same data link—for example, the same VLAN.

Figure 28-1 shows an example of how a host (PC1) uses an NS message to learn the MAC address used by another host. The NS message lists a target IPv6 unicast address with the implied question: "What is your link address?" The NA message, in this example, sent back to the original host that asked the question, lists that link address.

At Step 1 of this particular example, PC1 sends the solicitation to find PC2's MAC address. PC1 first looks in its NDP neighbor table, the equivalent of the IPv4 ARP cache, and does not find the MAC address for IPv6 address 2001:DB8:1111:1::22. So, at Step 1, PC1 sends the NDP NS message to the target.

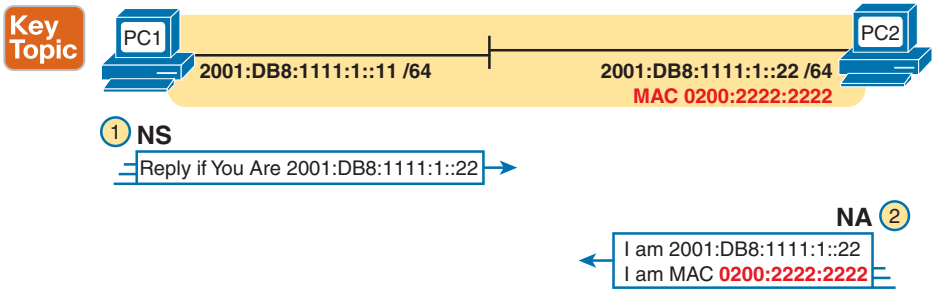



**Figure 28-1** *Example NDP NS/NA Process to Find the Neighbor's Link Addresses*

As a brief aside, be aware that NDP NS messages use a destination IPv6 address of the target's solicited-node multicast address, in this case PC2's solicited-node multicast address FF02::1:FF00:22. PC1 would then encapsulate the IPv6 packet in a frame destined to a multicast Ethernet address. If the network engineers at this company also enabled the multicast optimization feature MLD Snooping, the switches would forward the multicast NS message only to hosts that had earlier registered to receive packets sent to that specific multicast address. The other hosts on the link will never see the NS message. If the LAN switches do not implement MLD, then the switches still flood the frame so that it reaches the intended destination.

At Step 2, PC2 reacts to the received NS message. PC2 sends back an NA message, listing PC2's MAC address. PC1 records PC2's MAC address in PC1's NDP neighbor table.

Example 28-1 shows an example of the **IPv6 neighbor table** on Router R1 based on Figure 28-2. In this case, R1 has learned the MAC addresses for Routers R2 and R3, associated with their respective LLAs. R1 has also learned PC A's LLA and GUA and the matching MAC address. (To connect the output to the figure, pay close attention to the interface column on the far right of the output.)

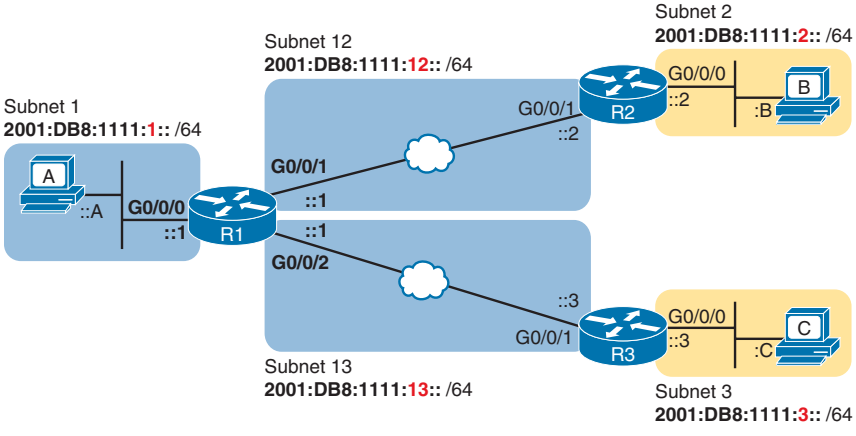



**Figure 28-2** *Sample Network with IPv6 Addresses*

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** A, C **3** A **4** D **5** D **6** B, D **7** A **8** C

**Example 28-1** *IPv6 Neighbor Table on Router R1*

```
R1# show ipv6 neighbors
IPv6 Address                              Age Link-layer Addr State Interface
2001:DB8:1111:1::a                          0 0200.aaaa.aaaa  REACH Gi0/0/0
2001:DB8:1111:1:9EA:93AC:F7CE:D39F          0 3c8c.f8eb.710d  REACH Gi0/0/0
2001:DB8:1111:1:706A:5FDF:AA40:E576        16 3c8c.f8eb.710d  STALE Gi0/0/0
2001:DB8:1111:1:70D0:AE1F:4786:907D         0 80fa.5b04.de8b  STALE Gi0/0/0
2001:DB8:1111:1:7C6B:7B02:DB5C:873F        16 3c8c.f8eb.710d  STALE Gi0/0/0
2001:DB8:1111:1:90A1:C742:1B11:6F10         0 00e0.4c68.1f26  STALE Gi0/0/0
2001:DB8:1111:1:BD2C:9AA4:83E2:6D8F        16 3c8c.f8eb.710d  STALE Gi0/0/0
FE80::AAAA:AAAA                             0 0200.aaaa.aaaa  REACH Gi0/0/0
FE80::184C:56F9:FD3B:D6E7                   0 00e0.4c68.1f26  REACH Gi0/0/0
FE80::552D:E285:4347:BDED                   0 80fa.5b04.de8b  DELAY Gi0/0/0
FE80::706A:5FDF:AA40:E576                   0 3c8c.f8eb.710d  REACH Gi0/0/0
FE80::FF:FE01:2                             0 2436.dadf.9281  REACH Gi0/0/1
FE80::72EA:1AFF:FE9A:D301                   0 70ea.1a9a.d301  REACH Gi0/0/2
```

**NOTE** To view a host's NDP neighbor table, use these commands: (Windows) **netsh interface ipv6 show neighbors**; (Linux) **ip -6 neighbor show**; (macOS) **ndp -an**.



Example 28-2 shows an excerpt from a Windows host neighbor table of a host in the same IPv6 subnet as PC A and Router R1 in Figure 28-2. The beginning output shows network shell command **netsh interface ipv6 show neighbors**, as issued in layers (which makes it much easier to issue additional **netsh** commands later). The highlighted entry lists R1's G0/0/0 link-local address (LLA) and MAC address. Also, the highlighted text at the far right of the line identifies this entry as representing a router. The output also lists several solicited-node multicast addresses.

**Example 28-2** *Example Windows Neighbor Table with* **netsh interface ipv6 show neighbors** *Command*

```
C:\Users\Wendell> netsh
netsh> interface ipv6
netsh interface ipv6> show neighbors
Interface 7: En0

Internet Address                              Physical Address   Type
--------------------------------------------  -----------------  -----------
fe80::11ff:fe11:1111                          02-00-11-11-11-11  Reachable (Router)
ff02::1                                       33-33-00-00-00-01  Permanent
ff02::2                                       33-33-00-00-00-02  Permanent
ff02::1:ff11:1111                             33-33-ff-11-11-11  Permanent
! Lines omitted for brevity
```

```
! Next line shows a Powershell command

PS C:\Users\Wendell> get-NetNeighbor -AddressFamily IPv6
ifIndex IPAddress                      LinkLayerAddress      State       PolicyStore
------- ---------                      ----------------      -----       -----------
49      ff02::1:ff11:1111              33-33-FF-11-11-11     Permanent   ActiveStore
49      fe80::11ff:fe11:1111           02-00-11-11-11-11     Reachable   ActiveStore
49      ff02::2                        33-33-00-00-00-02     Permanent   ActiveStore
49      ff02::1                        33-33-00-00-00-01     Permanent   ActiveStore
! Lines omitted for brevity
```

The example ends with the PowerShell command equivalent to the **netsh interface ipv6 show neighbor** command: **get-NetNeighbor -AddressFamily IPv6**. The latter command lists the same info. Be aware that Microsoft favors using PowerShell commands over older commands like those from netshell.

## Discovering Routers with NDP RS and RA

IPv4 hosts use the concept of an IPv4 default gateway or default router. When the host needs to send a packet to some IPv4 subnet other than the local subnet, the host sends the IPv4 packet to the default router, expecting the router to be able to route the packet to the destination. Note that IPv4 hosts either statically set the IP address of their default gateway or learn it from a server called a Dynamic Host Configuration Protocol (DHCP) server.

IPv6 uses the same concept of a default gateway, but it improves the process using NDP. With IPv6, IPv6 hosts use NDP to dynamically discover all IPv6 routers on the link, learning what routers it can use as a default router. NDP defines two messages that allow any host to discover all routers in the subnet:




**Router Solicitation (RS):** Hosts send this message to the "all-IPv6-routers" local-scope multicast address of FF02::2 to ask all on-link routers to identify themselves.

**Router Advertisement (RA):** Routers send this message in response to an RS message, listing many facts, including the link-local IPv6 address of the router. The message flows to the unicast address of the host that sent the RS. Routers also send unsolicited RA messages, not in response to an RS, but periodically, announcing the same details to all hosts on the link. Routers send unsolicited RA messages to the all-IPv6-hosts local-scope multicast address of FF02::1.

For example, Figure 28-3 shows how host PC A can learn R1's LLA. The process is simple, with PC A first asking and R1 replying.

IPv6 does not use broadcasts, but it does use multicasts to improve efficiency compared to IPv4. In this case, the RS message flows to the all-routers multicast address (FF02::2) so that all routers will receive the message. It has the same good effect as a broadcast with IPv4, without the negatives of a broadcast. In this case, only IPv6 routers will spend CPU cycles processing the RS message, and IPv6 hosts will ignore the message.

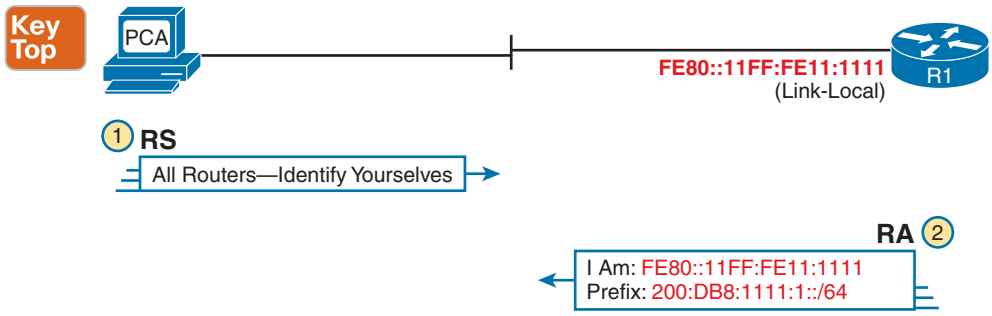



**Figure 28-3** *Example NDP RS/RA Process to Find the Default Routers*

Routers list any neighboring routers they learn about in NDP RA messages with the **show ipv6 routers** command. As an example, consider the topology in earlier Figure 28-2. No routers exist on the LAN connected to R1's G0/0/0 (on the left of the figure), but R1 should learn of both R2 and R3 on its WAN links. Example 28-3 shows the output, highlighting the LLA of the router that sent the NDP RA message, the local R1 interface on which it was received, and the **on-link prefix** advertised by the neighboring router.

**Example 28-3** *Listing All Routers with the* **show ipv6 routers** *Command*

```
R1# show ipv6 routers
Router FE80::FF:FE01:2 on GigabitEthernet0/0/1, last update 2 min
  Hops 64, Lifetime 1800 sec, AddrFlag=0, OtherFlag=0, MTU=1500
  HomeAgentFlag=0, Preference=Medium
  Reachable time 0 (unspecified), Retransmit time 0 (unspecified)
  Prefix 2001:DB8:1111:12::/64 onlink autoconfig
    Valid lifetime 2592000, preferred lifetime 604800
Router FE80::72EA:1AFF:FE9A:D301 on GigabitEthernet0/0/2, last update 0 min
  Hops 64, Lifetime 1800 sec, AddrFlag=0, OtherFlag=0, MTU=1500
  HomeAgentFlag=0, Preference=Medium
  Reachable time 0 (unspecified), Retransmit time 0 (unspecified)
  Prefix 2001:DB8:1111:13::/64 onlink autoconfig
    Valid lifetime 2592000, preferred lifetime 604800
```

**NOTE** To view the routers learned by a host, use these commands: (Windows) **netsh interface ipv6 show neighbors**; (Linux) **ip -6 neighbor**; (macOS) **ndp -rn**.

## Discovering Prefixes with NDP RS and RA

Beyond identifying routers on a link, the NDP RA message also lists the IPv6 prefix and prefix length used on the link. As a result, hosts dynamically learn the subnet(s) that should exist on-link. For example, Figure 28-3 shows an RS/RA exchange example. The RA in the lower right of the figure not only shows the router identifying itself using its link-local address (LLA), but the message also lists one on-link prefix of 2001:db8:1111:1::/64.

IPv6 hosts use a routing table much like IPv4 hosts but build their routes differently. IPv4 hosts build a route for each on-link subnet, with that subnet calculated from the host's IPv4 address and mask. IPv6 hosts build a route for each on-link subnet, but hosts do no calculations. Instead, they rely on the prefixes advertised in the RA messages sent by routers. Routers can advertise multiple on-link prefixes, listing the subnet prefix and prefix length, with the hosts then considering destinations in those subnets as on-link. Also, hosts build a default route, using the information listed in the NDP RA message. To summarize, IPv6 hosts create two important routes based on the RA message, as follows:



- Create a route for each on-link prefix/length learned from a router in an NDP RA message. These on-link routes allow the host to forward packets directly to on-link destinations without using a router.
- Create a default route with a next-hop router address of the router LLA identified in the NDP RA message. The default route allows the host to forward packets destined off-link to the router that resides on-link.

Example 28-4 shows excerpts from two hosts on the same subnet as PC A in Figure 28-3—with the initial output from macOS and the latter output from a Windows PC. Both commands display the host's routing table entries for the default route and one on-link prefix. The example highlights each command's default route, the next-hop address ("gateway"), and on-link prefix 2001:db8:1111:1::/64.

**Example 28-4** *Example macOS Host Routing Table with* **netstat -rn** *Command*

```
Mac% netstat -rnf inet6
default                        fe80::11ff:fe11:1111%en8        UGcg            en8
2001:db8:1111:1::/64           link#20                         UC              en8
fe80::11ff:fe11:1111%en8       2:0:11:11:11:11                 UHLWIir         en8
! Lines omitted for brevity
```

```
Windows PC> netsh
netsh> interface ipv6
netsh interface ipv6> show route

Publish  Type      Met    Prefix                    Idx  Gateway/Interface Name
-------  --------  ---    ------------------------  ---  ------------------------
No       Manual    256    ::/0                      49   fe80::11ff:fe11:1111
No       Manual    256    2001:db8:1111:1::/64      49   Ethernet 6
! Lines omitted for brevity
```

You see some differences when looking closely at the output from macOS versus Windows. However, both hosts use information from RA messages from R1. Notably, macOS lists the default route with the word *default*, while Windows uses the numeric equivalent of ::/0, a number meant to represent all IPv6 addresses. Both list the router's LLA rather than its GUA as the next-hop address.

## Discovering Duplicate Addresses Using NDP NS and NA

IPv6 hosts use the Duplicate Address Detection (DAD) process before they begin using a unicast address to ensure that no other node on that link is already using it. Hosts perform DAD when first using the address, and any time a host interface initializes. Hosts also use DAD, whether using static address configuration or any of the dynamic address configuration options. When performing DAD, if another host already uses that address, the first host simply does not use the address until the problem is resolved.

*DAD* refers to the function, but the function uses NDP NS and NA messages. A host sends an NS message for its own IPv6 address. No other host should be using that address, so no other host should send an NDP NA in reply. However, if another host already uses that address, that host will reply with an NA, identifying a duplicate use of the address.

Figure 28-4 shows an example of DAD. PC1 initializes and does a DAD check, but PC2 already uses the same address. The figure shows the following steps:

1. PC1, before using address 2001:DB8:1111:1::11, must use DAD.
2. PC1 sends an NS message, listing the address PC1 now wants to use (2001:DB8:1111:1::11) as the target.
3. PC2 receives the NS for the address PC2 currently uses, so PC2 sends back an NA.
4. PC1, on receiving the NA message for its IPv6 address, realizes a duplicate address exists.



Key Topic

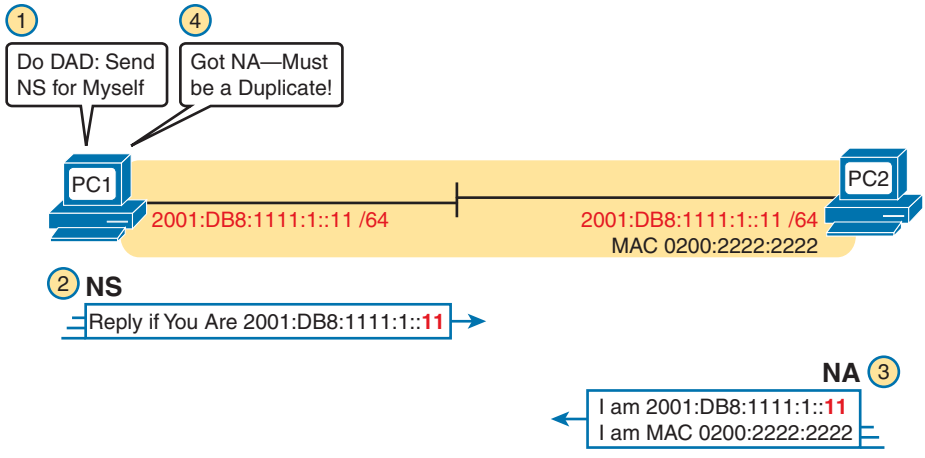



**Figure 28-4** *Example Duplicate Address Detection (DAD) with NDP NS/NA*

## NDP Summary

This chapter explains some of the many important functions performed by NDP. Use Table 28-2 as a study reference for the four NDP features discussed here.

Key Topic

**Table 28-2** NDP Function Summary

| Function | Protocol Messages | Who Discovers Info | Who Supplies Info | Info Supplied |
|---|---|---|---|---|
| Router discovery | RS and RA | Any IPv6 host | Any IPv6 router | Link-local IPv6 address of router |
| Prefix/length discovery | RS and RA | Any IPv6 host | Any IPv6 router | Prefix(es) and associated prefix lengths used on local link |
| Neighbor discovery | NS and NA | Any IPv6 host | Any IPv6 host | Link-layer address (for example, MAC address) used by a neighbor |
| Duplicate Address Detection | NS and NA | Any IPv6 host | Any IPv6 host | A simple confirmation of whether a unicast address is already in use |

## Dynamic Configuration of Host IPv6 Settings

Dynamic Host Configuration Protocol (DHCP) worked well for the dynamic assignment of IPv4 addresses. When the creators of IPv6 protocols looked for a solution for dynamic host address assignment, creating new DHCP protocols for IPv6 made perfect sense. Today, the DHCP Version 6 (DHCPv6) RFC 8415 defines one dynamic IPv6 address assignment method.

However, the creators of IPv6 also wanted another method to assign IPv6 addresses. DHCPv4 uses a server, requiring preconfiguration of the address pools used for each subnet. That model works well in some cases, but using the DHCPv4 model also requires the server to know all the address leases, keeping that information (called state information) about each host (client) and its address.

The creators of IPv6 made two methods for dynamic address assignment:

- **DHCPv6 (Stateful DHCPv6):** This method uses the same stateful model as DHCPv4 using a DHCP server.
- **Stateless Address Autoconfiguration (SLAAC):** The client self-assigns its IPv6 address. This method requires no preconfiguration of address pools and no need for servers to keep state information about the client.

This next major section of the chapter first looks at stateful DHCPv6, followed by SLAAC.

### Using Stateful DHCP

DHCPv6 gives an IPv6 host a way to learn host IPv6 configuration settings using the same general concepts as DHCP for IPv4. The host exchanges messages with a DHCP server. The server supplies the host with configuration information, including a lease of an IPv6 address and DNS server address information.

More specifically, stateful DHCPv6 works like the more familiar DHCP for IPv4 in many other general ways, as follows:

Key Topic

- DHCP clients on a LAN send messages that flow only on the local LAN, hoping to find a DHCP server.
- If the DHCP server sits on the same LAN as the client, the client and server can exchange DHCP messages directly, without needing help from a router.
- If the DHCP server sits on another link as compared to the client, the client and server rely on a router to forward the DHCP messages.
- The router that forwards messages from one link to a server in a remote subnet must be configured as a DHCP relay agent, with knowledge of the DHCP server's IPv6 address.
- Servers have configuration that lists pools of addresses for each subnet from which the server allocates addresses.
- Servers offer a lease of an IP address to a client, from the pool of addresses for the client's subnet; the lease lasts a set time (usually days or weeks).
- The server tracks state information, specifically a client identifier (often based on the MAC address), along with the address currently leased to that client.

DHCPv6 defines two branches: **stateful DHCPv6** and **stateless DHCPv6**. Stateful DHCPv6 works more like the DHCPv4 model, especially related to that last item in the list. A stateful DHCPv6 server tracks information about which client has a lease for what IPv6 address; the fact that the server knows information about a specific client is called state information, making the DHCP server a stateful DHCP server.

Stateless DHCP servers do not lease an address to the client, so a stateless DHCP server does not track any per-client information. The upcoming section, "Using Stateless Address Autoconfiguration," discusses how stateless DHCPv6 servers have an important role when a company decides to use SLAAC.

28

### Differences Between Stateful DHCPv6 and DHCPv4

While stateful DHCPv6 has many similarities to DHCPv4, many particulars differ as well. Figure 28-5 shows the differences: Stateful DHCPv6 supplies the address and the DNS server list. However, the host already relies on NDP RA messages to learn the default router address and the prefix length to use, with enough information to build default and on-link routes. So, with stateful DHCPv6, the server does not supply a default router address or prefix length.

DHCPv6 also updates the protocol messages to use IPv6 packets instead of IPv4 packets, with new messages and fields. For example, Figure 28-6 shows the names of the DHCPv6 messages, which replace the DHCPv4 Discover, Offer, Request, and Acknowledgment (DORA) messages. Instead, DHCPv6 uses the Solicit, Advertise, Request, and Reply (SARR) messages.

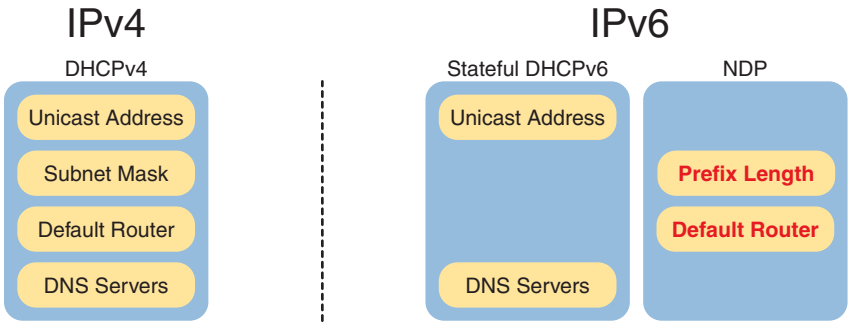



**Figure 28-5** *Sources of Specific IPv6 Settings When Using Stateful DHCP*

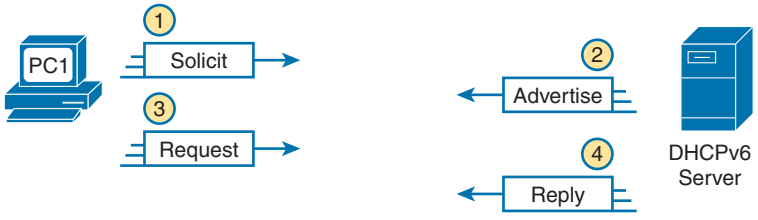



**Figure 28-6** *Four Stateful DHCPv6 Messages Between Client and Server*

The four DHCPv6 messages work in two matched pairs with the same general flow as the similar DHCPv4 messages. The Solicit and Advertise messages complete the process of the client searching for the IPv6 address of a DHCPv6 server (the Solicit message) and the server advertising an address (and other configuration settings) for the client to possibly use (the Advertise message). The Request and Reply messages let the client ask to lease the address, with the server confirming the lease in the Reply message. (Note that stateful DHCPv6 supports a rapid commit option that completes the lease using only the Solicit and Reply messages.)

### DHCPv6 Relay Agents

For enterprises that choose to use stateful DHCPv6, often the DHCP server sits at a central site, far away from many of the clients that use the DHCPv6 server. In those cases, the local router at each site must act as a DHCP relay agent.

The concepts of DHCPv6 relay work like DHCPv4 relay, as discussed in the section "Configuring DHCP Relay" in Chapter 19, "IP Addressing on Hosts." The client sends a message that normally has a link-local scope so that routers do not forward the packet. By enabling DHCPv6 relay, the router then changes the source and destination IP address, forwarding the packet to the DHCP server. When the server sends a reply, it flows to an address on the router (the relay agent). The router again changes the addresses in the packet for correct delivery to the client.

The differences for IPv6 become more apparent when you look at some of the IPv6 addresses used in DHCPv6 messages, like the Solicit message used to lead off a DHCPv6 flow. As shown in Figure 28-7, the client uses the following addresses in the solicit message:

**Source of link-local:** The client uses its link-local address as the packet's source address.

**Destination address of "all-DHCP-agents" FF02::1:2:** The client sends the Solicit message to the link-local scope multicast address FF02::1:2. Only DHCP servers and routers acting as DHCP relay agents listen for these packets.

With a link-local scope multicast destination address, the Solicit message sent by a host would flow only on the local LAN. Figure 28-7 shows how R1, acting as a DHCPv6 relay agent, assists DHCPv6 clients like host A to deliver DHCPv6 packets to the DHCPv6 server.

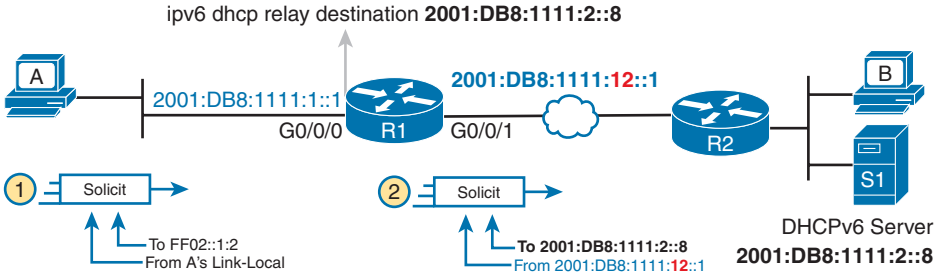



**Figure 28-7** *DHCPv6 Relay Agent and DHCP IPv6 Addresses*

In the figure, Step 1 shows the DHCPv6 Solicit message, which would otherwise stay on the link due to the link-local scope of destination multicast address FF02::1:2. Step 2 then shows the action of the DHCP relay agent on Router R1. Router R1 changes the destination IPv6 address to the configured DHCP server's address (2001:DB8:1111:2::8). The DHCP relay agent also sets the source IPv6 address to the address of its outgoing interface (G0/0/1) as the source IPv6 address, which is slightly different from the DHCPv4 relay agent. R1 then forwards the Solicit message to the server.

Continuing the story beyond the figure, the server sends a reply, specifically a DHCPv6 Advertise message. That message reverses the IPv6 addresses used compared to the Solicit message, so the Advertise message uses a destination address of 2001:DB8:1111:12::1. The relay agent in Router R1 reacts by converting the destination address to host A's LLA and forwarding the packet out the interface toward the client.

Example 28-5 shows the DHCPv6 relay agent configuration for R1 in Figure 28-6. The top of the example shows the **ipv6 dhcp relay** interface subcommand, with reference to the IPv6 address of the DHCPv6 server. The bottom of the figure shows the output of the **show ipv6 interface** command, which confirms that R1 is now listening for multicasts sent to the all-DHCP-agents multicast address FF02::1:2.

**Example 28-5** *Configuring Router R1 to Support Remote DHCPv6 Server*

```
interface GigabitEthernet0/0/0
 ipv6 dhcp relay destination 2001:DB8:1111:2::8

R1# show ipv6 interface g0/0/0
GigabitEthernet0/0/0 is up, line protocol is up
  IPv6 is enabled, link-local address is FE80::11FF:FE11:1111
```

```
  No Virtual link-local address(es):
  Global unicast address(es):
    2001:DB8:1111:1::1, subnet is 2001:DB8:1111:1::/64
  Joined group address(es):
    FF02::1
    FF02::2
    FF02::5
    FF02::6
    FF02::1:2
    FF02::1:FF00:1
    FF02::1:FF11:1111
! Lines omitted for brevity
```

As an aside, note that of the multicast addresses listed under the heading "Joined group address(es)," the first five are well-known multicast addresses (FF02::1, FF02::2, FF02::5, FF02::6, and FF02::1:2), with two solicited-node multicast addresses that begin with FF02::1:FF.

## Using Stateless Address Autoconfiguration

Most companies extensively use DHCPv4, and stateful DHCPv6 makes sense for those same reasons; however, using a stateful DHCP process does have some negatives. Someone has to configure, administer, and manage the DHCP server(s). The configuration includes ranges of IP addresses for every subnet. Then, when a host (client) leases the address, the server notes which client is using which address. All these functions work well, and knowing the information in the DHCP server can be pretty valuable; however, the reliance on a stateful DHCP server requires some thought and attention from the IT staff.

IPv6's **Stateless Address Autoconfiguration (SLAAC)** provides an alternative method for dynamic IPv6 address assignment—without needing a stateful server. In other words, SLAAC does not require a server to assign or lease the IPv6 address, does not require the IT staff to preconfigure a pool of addresses per subnet, and does not require the server to track state information about which device uses which IPv6 address.

The term *SLAAC* refers to both a specific part of how a host learns one IPv6 setting—its IPv6 address—plus the overall process of learning all four key host IPv6 settings (address, prefix length, default router, and the list of DNS server addresses). This next topic begins by looking at the tasks done by SLAAC related to the IPv6 address. Then the text looks at the overall SLAAC process to find all four host settings—a process that also uses NDP and stateless DHCP.

### Building an IPv6 Address Using SLAAC

When using SLAAC, a host does not lease its IPv6 address. Instead, the host learns part of the address from the nearby router—the prefix—and then makes up the rest of its IPv6 address. Specifically, a host using SLAAC to choose its IPv6 address uses the following steps:

1. Learn the IPv6 prefix used on the link from any router, using NDP RS/RA messages.

2. Choose its IPv6 address by making up the interface ID (IID) value to follow the just-learned IPv6 prefix.
3. Before using the address, use DAD to ensure that no other host is already using the same address.

Figure 28-8 depicts the first two steps while noting the two most common ways a host completes the address. Hosts can use modified EUI-64 rules, as discussed in the section, "Generating a Unique Interface ID Using Modified EUI-64," in Chapter 27, "Implementing IPv6 Addressing on Routers," or a random number.

Key Topic

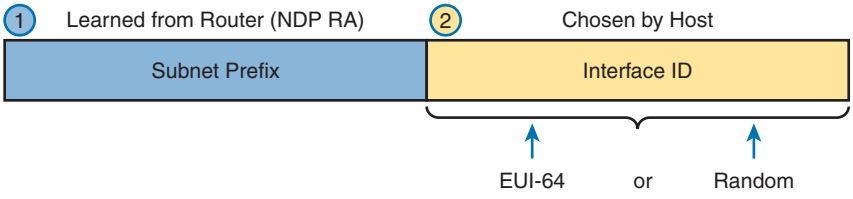



**Figure 28-8** *Host IPv6 Address Formation Using SLAAC*

## Combining SLAAC with Stateless DHCP

When using SLAAC, a host uses three tools to find its four IPv6 settings, as noted in Figure 28-9. SLAAC itself focuses on the IPv6 address only. The host then uses NDP messages to learn the prefix length and the IPv6 addresses of the default routers on the link. Finally, the host uses stateless DHCP to learn the IPv6 addresses of any DNS servers.



Key Topic

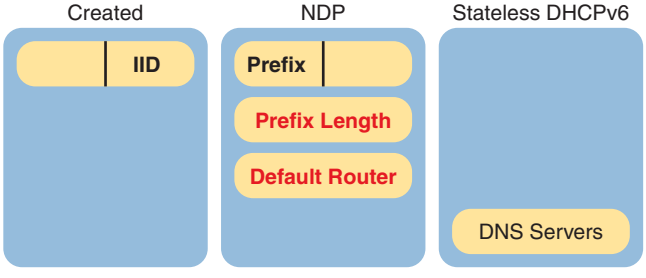



**Figure 28-9** *Sources of Specific IPv6 Settings When Using SLAAC*

When SLAAC uses DHCP to supply the list of DNS servers, the server implements stateless DHCP. With stateless DHCPv6, the DHCPv6 server

- Needs simple configuration only, specifically the short list of DNS server addresses
- Needs no per-subnet configuration: no subnet list, no per-subnet address pools, no list of excluded addresses per subnet, and no per-subnet prefix lengths
- Has no need to track state information about DHCP leases—that is, which devices lease which IPv6 address—because the server does not lease addresses to any clients

Table 28-3 summarizes the key comparison points between stateful and stateless DHCP.

**Table 28-3** Comparison of Stateless and Stateful DHCPv6 Services

| Feature | Stateful DHCP | Stateless DHCP |
|---|---|---|
| Remembers IPv6 address (state information) of clients | Yes | No |
| Leases IPv6 address to client | Yes | No |
| Supplies list of DNS server addresses | Yes | Yes |
| Commonly used with SLAAC | No | Yes |

### Combining SLAAC with RA-Based DNS Server Configuration

SLAAC originally relied on a stateless DHCP server to supply the DNS server list. IPv6 now supports another option to deliver the DNS server list, called RA-based DNS configuration, which removes the need for a stateless DHCP server.

With RA-based DNS Server (RDNSS) configuration, you configure each router interface with the list of DNS servers. Then, when sending each NDP RA message, the router supplies the DNS list in its NDP RA (Router Advertisement) messages. As a result, RDNSS configuration provides a means for automatic assignment of all client IPv6 settings using router configuration only, with no DHCP server at all. However, note that it also requires configuration of the DNS server list on all routers that support IPv6 so that a centralized stateless DHCPv6 server may be more practical to manage.

## Permanent and Temporary SLAAC Addresses

In practice, hosts use SLAAC to generate multiple unicast addresses, specifically a **permanent IPv6 address** plus a series of **temporary IPv6 addresses** over time. The permanent address remains unchanged over time. If that host runs as a server, the host will use the permanent address for all incoming connections. You would also typically add a DNS entry for that address in the DNS server so clients could easily connect.

Hosts use temporary addresses for client applications. For instance, when you open a web browser tab and connect to a website, your host connects using the temporary address rather than the permanent address.

Using temporary addresses makes hosts more secure, as detailed in a series of related RFCs that revolve around RFC 8981. Attackers may gather information and packets sent by a host through various attacks. If the captured packets over time have the same address, the attacker can more easily correlate the information to find a way to gain access to the host or deny services to it. By frequently changing the address, you can prevent the attacker's data analysis from giving them an edge in attacking your host.

When created, a host assigns all SLAAC-created addresses a **preferred lifetime** and **valid lifetime** setting. Each temporary address moves through stages called preferred, valid, and deprecated. During the preferred phase, the host uses the address for new connections. When the preferred lifetime expires, the address becomes valid. The host can continue using a valid address for existing connections but not for new ones. After the valid lifetime expires, the host considers the address deprecated and does not use the address again. Figure 28-10 shows the concepts.




**Figure 28-10** *SLAAC Preferred and Valid Lifetimes for One Temporary Address*

With temporary addresses, the preferred and valid lifetimes have short settings. A typical setting might be 24 hours for the preferred lifetime and one week for the valid lifetime. In that case, every 24 hours, the host would need a new temporary address because the last temporary address moves from a preferred to a valid state. Basically, at any one time, the host needs one temporary address in its preferred lifetime state. Figure 28-11 shows the idea, with the timelines for three consecutive temporary addresses appearing 24 hours after the previous one. At any point, the host has one temporary address with some preferred lifetime remaining, so the host can use the new preferred address for new application connections.

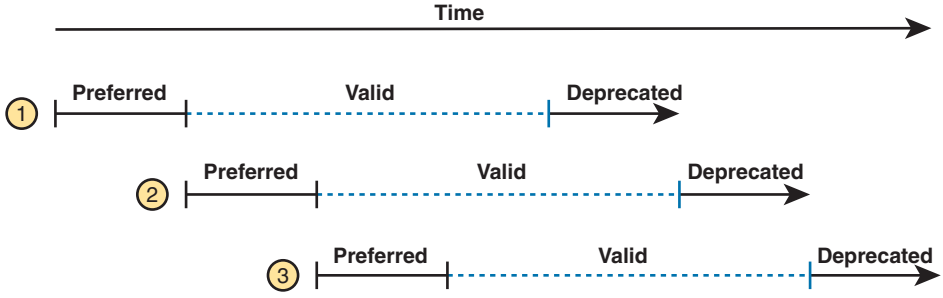



**Figure 28-11** *Creating New Temporary Addresses to Ensure One Has Preferred Life Remaining*

You can see the permanent and temporary addresses in host commands like these:

macOS: **ifconfig en0 inet 6** or **ifconfig -aL inet6**

Linux: **ip -6 address**

Windows: **ipconfig /all** or **netsh interface ipv6 show address** (Windows netshell)

Windows: **Get-NetIPConfiguration -Detailed** (Windows PowerShell)

Example 28-6 shows output from the **netsh interface ipv6 show address** command on Windows. The example shows an excerpt of the output for the one working wireless LAN interface. The output shows its permanent (public) address used for incoming connections, with an infinite valid and preferred lifetime. It also shows the current temporary address, with a preferred lifetime of just under 24 hours and a valid lifetime of just under one week. Also, examine the IIDs of the addresses for ff:fe—again, their absence signals that this host did not use EUI-64 but instead generated a random IID.

**Example 28-6** *Windows SLAAC Addresses with the* **netsh interface ipv6 show address** *Command*

```
C:\Users\Wendell> netsh
netsh> interface ipv6
netsh interface ipv6> show address

Interface 7: Wi-Fi

Addr Type  DAD State    Valid Life Pref. Life Address
---------  -----------  ---------- ---------- ------------------------
Temporary  Preferred    6d23h57m58s   23h49m3s 2001:db8:1111:1:c1cd:7a44:45a5:58f1
Public     Preferred        infinite   infinite 2001:db8:1111:1:f1f5:5cbb:f395:6c51
Other      Preferred        infinite   infinite fe80::f1f5:5cbb:f395:6c51%7
```

Note that upcoming Example 28-7 shows a sample of similar commands for macOS.

# Troubleshooting Host IPv6 Addressing

This chapter's third and final major section examines a few commands to verify and troubleshoot IPv6 addressing configuration on hosts. Specifically, this section examines the host's IPv6 settings and then looks at the usual commands to test whether a host can send packets: **ping** and **traceroute**.

Note that this section lists some commands on different host OSs; however, be aware that this and other chapters do not attempt to show each variation of every networking command on every OS. Instead, the host command examples reinforce the concepts seen earlier in the chapter.

## Verifying IPv6 Connectivity from Hosts

Most end-user OSs support a convenient way to look at IPv6 settings from the graphical user interface. In some cases, all four of the key IPv6 host settings can be on the same window, whereas in other cases, seeing all the settings might require navigation to multiple windows or tabs in the same window. The following few pages first focus on how to find the IPv6 addresses used by hosts, followed by how to test connectivity using the **ping** and **traceroute** commands.

### Host Commands to Find IPv6 Interface Addresses

For example, Figure 28-12 shows a macOS window that lists three IPv6 host settings. The one missing setting, the DNS server setting, is in another tab (as seen near the top of the image).

Take a moment to look at the details in Figure 28-12's image. The image shows the IPv4 settings at the top. The lower half of the window shows the IPv6 settings as having been learned "Automatically," which means that the host will use either stateful DHCP or SLAAC. In this case, the host used SLAAC to give itself two IPv6 addresses inside the same 2001:DB8:1111:1::/64 subnet. However, the graphical interface does not identify the permanent and temporary addresses or lifetimes.

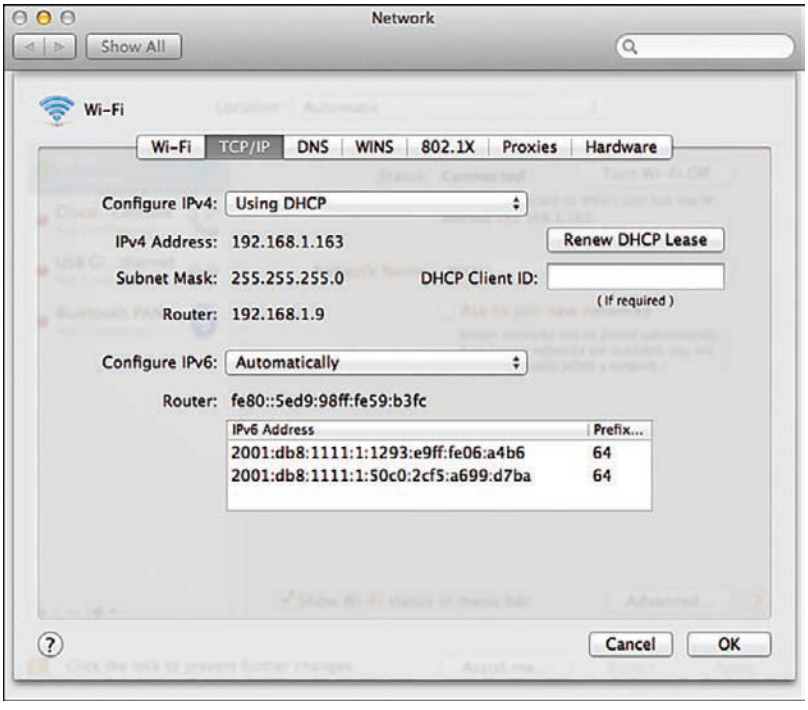



**Figure 28-12** *Three IPv6 Settings for Dynamic Address Assignment on macOS*

28

Hosts support a range of commands to find more detail about the addresses, including lifetimes. Example 28-6 earlier showed the **netsh interface ipv6 show address** command for Windows, for example, with Linux using the **show -6 address** command. Example 28-7 shows these exact details on macOS with two commands. The first command, **ifconfig en8**, lists details about Ethernet, IPv4, and IPv6, on one specific interface (internally numbered as Ethernet number 8, or en8). The command also identifies the preferred address (using the keyword *secured*) and temporary address, as highlighted in the upper part of the example.

**Example 28-7** *Sample* **ifconfig** *Commands from a Mac*

```
Mac% ifconfig en8
en8: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
       options=6467<RXCSUM,TXCSUM,VLAN_MTU,TSO4,TSO6,CHANNEL_IO,PARTIAL_
CSUM,ZEROINVERT_CSUM>
        ether 00:e0:4c:68:1f:26
        inet6 fe80::184c:56f9:fd3b:d6e7%en8 prefixlen 64 secured scopeid 0x14
        inet6 2001:db8:1111:1:106a:dd3e:8e22:a6fb prefixlen 64 autoconf secured
        inet6 2001:db8:1111:1:ec69:15f9:b4fc:fe2c prefixlen 64 autoconf temporary
        inet 192.168.1.120 netmask 0xffffffff broadcast 192.168.1.120
        nd6 options=201<PERFORMNUD,DAD>
        media: autoselect (1000baseT <full-duplex>)
        status: active
```

```
Mac% ifconfig -aL inet6
! Only interface en8 shown for brevity
en8: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
     options=6467<RXCSUM,TXCSUM,VLAN_MTU,TSO4,TSO6,CHANNEL_IO,PARTIAL_
CSUM,ZEROINVERT_CSUM>
     inet6 fe80::8ad:f140:a952:9a46%en8 prefixlen 64 secured scopeid 0xc
     inet6 2001:db8:1111:1:106a:dd3e:8e22:a6fb prefixlen 64 autoconf secured
pltime 604654 vltime 2591854
     inet6 2001:db8:1111:1:a968:a6d9:7fbf:38a6 prefixlen 64 autoconf temporary
pltime 85782 vltime 604654
     nd6 options=201<PERFORMNUD,DAD>
```

The lower part of the output shows how to find the preferred (pltime) and valid (vltime) lifetimes for each address using the **ifconfig -aL inet6** command. The output also lists the word *autoconf*, which implies the host used SLAAC. The output also identifies one permanent address (secured) and the other as the temporary address. Finally, note that the SLAAC-derived addresses appear to use random interface IDs, because the string ff:fe does not exist in the middle of the interface ID.

## Testing IPv6 Connectivity with ping and traceroute

The **ping** and **traceroute** commands make for great connectivity testing tools for IPv4 as well as for IPv6. Some OSs (notably Microsoft Windows variants and Cisco routers and switches) let you use the same **ping** and **traceroute** commands used with IPv4. Some other OSs require a different command, like the **ping6** and **traceroute6** commands used with macOS and Linux. (The upcoming examples show both variations.)

As for the output of the **ping** and **traceroute** commands, most people who understand the commands as used with IPv4 need no coaching to use the commands with IPv6. The output is mostly unchanged compared to the IPv4 equivalents, other than the obvious differences with listing IPv6 addresses. For comparison, the upcoming examples use the internetwork displayed in Figure 28-13.

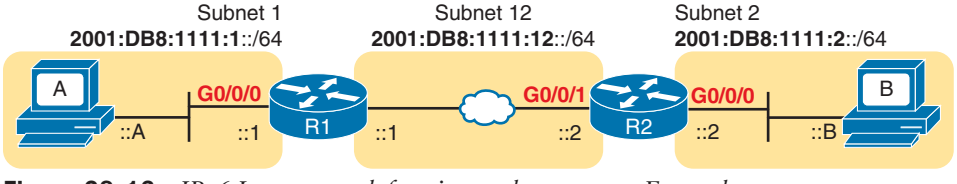



**Figure 28-13** *IPv6 Internetwork for* **ping** *and* **traceroute** *Examples*

Example 28-8 shows three **ping6** commands, taken from PC1, a Linux host. (Linux uses **ping6** and **traceroute6** commands for IPv6.) The first two commands show IPv6 pings, the first to R1's LAN IPv6 address, followed by PC1 pinging PC B's IPv6 address. The final command shows an IPv4 ping for comparison.

Key Topic

**Example 28-8** *The* **ping6** *Command from PC1, for R1 and PC2*

```
! An IPv6 ping, PC A pings R1's address in the same subnet
Linux_A:$ ping6 2001:db8:1111:1::1
PING 2001:db8:1111:1::1 (2001:db8:1111:1::1) 56 data bytes
64 bytes from 2001:db8:1111:1::1: icmp_seq=1 ttl=64 time=1.26 ms
64 bytes from 2001:db8:1111:1::1: icmp_seq=2 ttl=64 time=1.15 ms
^C
--- 2001:db8:1111:1::1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001 ms
rtt min/avg/max/mdev = 1.156/1.210/1.263/0.062 ms

! An IPv6 ping next, ping of PC B from PC A
Linux_A:$ ping6 2001:db8:1111:2::b
PING 2001:db8:1111:2::b (2001:db8:1111:2::b) 56 data bytes
64 bytes from 2001:db8:1111:2::b: icmp_seq=1 ttl=64 time=2.33 ms
64 bytes from 2001:db8:1111:2::b: icmp_seq=2 ttl=64 time=2.59 ms
64 bytes from 2001:db8:1111:2::b: icmp_seq=3 ttl=64 time=2.03 ms
^C
--- 2001:db8:1111:2::b ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003 ms
rtt min/avg/max/mdev = 2.039/2.321/2.591/0.225 ms

! An IPv4 ping next, for comparison - ping of PC B from PC A
Linux_A:$ ping 10.1.2.22
PING 10.1.3.22 (10.1.2.22) 56 data bytes
64 bytes from 10.1.2.22: icmp_seq=1 ttl=64 time=2.45 ms
64 bytes from 10.1.2.22: icmp_seq=2 ttl=64 time=2.55 ms
64 bytes from 10.1.2.22: icmp_seq=3 ttl=64 time=2.14 ms
^C
--- 10.1.3.22 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014 ms
rtt min/avg/max/mdev = 2.04/2.318/2.604/0.224 ms
```



Example 28-9 shows a **traceroute6** command on PC A, finding the route to PC B. The output mirrors the style of output for most IPv4 **traceroute** commands, other than the obvious difference of listing IPv6 addresses. Note that the output lists R1's G0/0/0 IPv6 address, R2's G0/0/1 IPv6 address, and then PC B's address to end the output.

**Example 28-9** *The* **traceroute6** *Command from PC1, for PC2*

```
Linux_A:$ traceroute6 2001:db8:1111:2::b
traceroute to 2001:db8:1111:2::b (2001:db8:1111:2::b) from 2001:db8:1111:1::a,
   30 hops max, 24 byte packets
1  2001:db8:1111:1::1 (2001:db8:1111:1::1)  0.794 ms  0.648 ms  0.604 ms
2  2001:db8:1111:12::2 (2001:db8:1111:12::2)  1.606 ms  1.49 ms  1.497 ms
3  2001:db8:1111:2::b (2001:db8:1111:2::b)  2.038 ms  1.911 ms  1.899 ms
```

**NOTE** The **traceroute/traceroute6** commands learn the addresses using the ICMPv6 time exceeded message. That mechanism results in the command output listing the GUA of the various routers and destination host.

## Verifying Host Connectivity from Nearby Routers

For router verification commands for IPv6, some IPv6 features use the same command as with IPv4, but some substitute "ipv6" for "ip." And in some cases, particularly with functions that do not exist in IPv4 or have changed quite a bit, routers support brand-new commands. This section looks at a couple of router commands useful to verify IPv6 host connectivity, some old and some new for IPv6.

First, for connectivity testing, Cisco routers and switches support the **ping** and **traceroute** commands with the same basic features for IPv6 as with IPv4. The commands accept either an IPv4 or an IPv6 address as input for the standard version of the commands. For the extended versions of these commands, the first prompt question asks for the protocol. Just type **ipv6** instead of using the default of **ip**, and answer the rest of the questions.

Of course, an example helps, particularly for the extended commands. Example 28-10 begins with an extended IPv6 **ping** from R1 to PC B from Figure 28-13, using R1's G0/0/0 interface as the source of the packets. The second command shows a standard IPv6 **traceroute** from R1 to PC B.

Key Topic

**Example 28-10** *Extended* **ping** *and Standard* **traceroute** *for IPv6 from Router R1*

```
R1# ping
Protocol [ip]: ipv6
Target IPv6 address: 2001:db8:1111:2::b
Repeat count [5]:
Datagram size [100]:
Timeout in seconds [2]:
Extended commands? [no]: yes
Source address or interface: GigabitEthernet0/0/0
UDP protocol? [no]:
Verbose? [no]:
Precedence [0]:
DSCP [0]:
Include hop by hop option? [no]:
Include destination option? [no]:
Sweep range of sizes? [no]:
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 2001:DB8:1111:2::b, timeout is 2 seconds:
Packet sent with a source address of 2001:DB8:1111:1::1
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/1/4 ms
```

```
R1# traceroute 2001:db8:1111:2::b
Type escape sequence to abort.
Tracing the route to 2001:DB8:1111:2::b

  1 2001:DB8:1111:12::2 4 msec 0 msec 0 msec
  2 2001:DB8:1111:2::b 0 msec 4 msec 0 msec
```

Another way to verify host settings from a router is to look at the router's NDP neighbor table. All IPv6 hosts, routers included, keep an IPv6 neighbor table: a list of all neighboring IPv6 addresses and matching MAC addresses.

One way to verify whether a neighboring host is responsive is to determine whether it will send back an NDP NA when the router sends it an NDP NS (to discover the host's MAC address). To do so, the router could clear its neighbor table (**clear ipv6 neighbor**) and ping a host on some connected interface. The router will first need to send an NDP NS, and the host must send an NDP NA back. If the router shows that host's MAC address in the neighbor table, the host must have just replied with an NDP NA. Example 28-11 shows a sample of an IPv6 neighbor table from Router R2 in Figure 28-13, using the **show ipv6 neighbors** command.

**Example 28-11** *The* **show ipv6 neighbors** *Command on Router R2*

```
R2# show ipv6 neighbors
IPv6 Address                              Age Link-layer Addr State Interface
2001:DB8:1111:2::B                          0 0200.bbbb.bbbb  STALE Gi0/0/0
FE80::BBFF:FEBB:BBBB                        0 0200.bbbb.bbbb  STALE Gi0/0/0
FE80::FF:FE01:1                             0 2436.dadf.5681  REACH Gi0/0/1
```

Finally, routers can also list information about the available routers on a LAN subnet, which impacts the connectivity available to hosts. As a reminder, routers send NDP RA messages to announce their willingness to act as an IPv6 router on a particular LAN subnet. Cisco routers watch for RA messages from other routers, typically receiving unsolicited RA messages that routers send to the FF02::1 all IPv6 hosts multicast address. The **show ipv6 routers** command lists any other routers but not the local router. Refer to earlier Example 28-3 for a sample of the **show ipv6 routers** command output.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 28-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 28-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory table | | Book, website |
| Review command tables | | Book |

## Review All the Key Topics

Key Topic

**Table 28-5** Key Topics for Chapter 28

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Four functions for which NDP plays a major role | 699 |
| List | Descriptions of the NDP NS and NA messages | 699 |
| Figure 28-1 | Example use of NDP RS and RA | 700 |
| List | Descriptions of the NDP RS and RA messages | 702 |
| Figure 28-3 | Example use of NDP NS and NA | 703 |
| List | Two important routes created by IPv6 hosts based on the RA message | 704 |
| Figure 28-4 | Example use of NDP for Duplicate Address Detection (DAD) | 705 |
| Table 28-2 | Summary of NDP functions discussed in this chapter | 706 |
| List | Similarities between DHCP for IPv4 and stateful DHCP for IPv6 | 707 |
| Figure 28-5 | Sources of host IPv6 configuration when using stateful DHCPv6 | 708 |
| Figure 28-8 | SLAAC address creation concepts | 711 |
| Figure 28-9 | Sources of host IPv6 configuration when using SLAAC | 711 |
| Example 28-8 | Examples of the **ping6** command | 717 |
| Example 28-10 | Using extended **ping** and standard **traceroute** for IPv6 | 718 |

## Key Terms You Should Know

Duplicate Address Detection (DAD), IPv6 neighbor table, Neighbor Advertisement (NA), Neighbor Discovery Protocol (NDP), Neighbor Solicitation (NS), on-link prefix, permanent IPv6 address, preferred lifetime, prefix discovery, Router Advertisement (RA), Router Solicitation (RS), stateful DHCPv6, Stateless Address Autoconfiguration (SLAAC), stateless DHCPv6, temporary IPv6 address, valid lifetime

## Command References

Tables 28-6, 28-7, and 28-8 list configuration and verification commands used in this chapter, respectively. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 28-6** Chapter 28 Configuration Command Reference

| Command | Description |
|---|---|
| **ipv6 dhcp relay destination** *server-address* | Interface subcommand that enables the IPv6 DHCP relay agent |

**Table 28-7** Chapter 28 EXEC Command Reference

| Command | Description |
|---|---|
| **ping** {*host-name* \| *ipv6-address*} | Tests IPv6 routes by sending an ICMP packet to the destination host |
| **traceroute** {*host-name* \| *ipv6-address*} | Tests IPv6 routes by discovering the IP addresses of the routes between a router and the listed destination |
| **show ipv6 neighbors** | Lists the router's IPv6 neighbor table |
| **show ipv6 routers** | Lists any neighboring routers that advertised themselves through an NDP RA message |

**Table 28-8** Chapter 28 Host Command Reference

| Command (Windows/macOS/Linux) | Description |
|---|---|
| **ipconfig / ifconfig / ifconfig** | (Windows/macOS/Linux) Lists interface settings, including IPv4 and IPv6 addresses |
| **ping / ping6 / ping6** | (Windows/macOS/Linux) Tests IP routes by sending an ICMPv6 packet to the destination host |
| **tracert / traceroute6 / traceroute6** | (Windows/macOS/Linux) Tests IP routes by discovering the IPv6 addresses of the routes between a router and the listed destination |
| **netsh interface ipv6 show neighbors / get-Neighbor -AddressFamily IPv6** | (Windows only) Lists IPv6 neighbors with network shell and PowerShell |
| **ndp -an / ip -6 neighbor show** | (macOS/Linux) Lists IPv6 neighbors |
| **netsh interface ipv6 show route / netstat -rnf inet6 / ip -6 route** | (Windows/macOS/Linux) Lists a host's IPv6 routing table |
| **netsh interface ipv6 show address / ifconfig -aL inet6 / ip -6 address** | (Windows/macOS/Linux) Lists a host's interface IPv6 addresses |



CHAPTER 29

# Implementing IPv6 Routing

**This chapter covers the following exam topics:**

- **3.0 IP Connectivity**
  - **3.1 Interpret the components of routing table**
    - **3.1.a Routing protocol code**
    - **3.1.b Prefix**
    - **3.1.c Network mask**
    - **3.1.d Next hop**
    - **3.1.e Administrative distance**
  - **3.3 Configure and verify IPv4 and IPv6 static routing**
    - **3.3.a Default route**
    - **3.3.b Network route**
    - **3.3.c Host route**
    - **3.3.d Floating static**

This last chapter in Part VII of the book completes the materials about IPv6 by examining four major topics. The first section examines IPv6 connected and **local routes**—the routes a router adds to its routing table in reaction to IPv6 address configuration. The second major section discusses static network routes, that is, routes configured by the **ipv6 route** command with a destination of an IPv6 prefix (subnet). The third major section examines other IPv6 static route topics, including default routes, host routes, and floating static routes, with some notes about troubleshooting all static IPv6 routes. The final short section discusses a few tools and tips to troubleshoot IPv6 static routes.

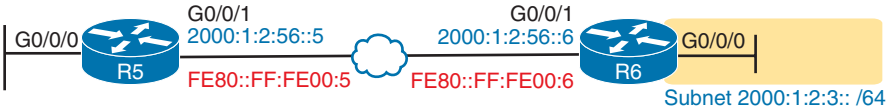
## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 29-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Connected and Local IPv6 Routes | 1, 2 |
| Static IPv6 Network Routes | 3, 4 |
| Static Default, Host, and Floating Static IPv6 Routes | 5, 6 |
| Troubleshooting Static IPv6 Routes | 7 |

Refer to the following figure for questions 1, 3, and 4.




1. Router R6 in the figure has been configured with the **ipv6 address 2000:1:2:3::1/64** command on its G0/0/0 interface. The router also creates a link-local address of FE80::FF:FE00:1. All router interfaces have an up/up status. Which of the following routes will R6 add to its IPv6 routing table? (Choose two answers.)
   - **a.** A route for 2000:1:2:3::/64
   - **b.** A route for FE80::FF:FE00:1/64
   - **c.** A route for 2000:1:2:3::1/128
   - **d.** A route for FE80::FF:FE00:1/128
2. A router has been configured with the **ipv6 address 3111:1:1:1::1/64** command on its G0/0/0 interface and **ipv6 address 3222:2:2:2::1/64** on its G0/0/1 interface. Both interfaces have an up/up state. Which of the following routes would you expect to see in the output of the **show ipv6 route connected** command? (Choose two answers.)
   - **a.** A route for 3111:1:1:1::/64
   - **b.** A route for 3111:1:1:1::1/64
   - **c.** A route for 3222:2:2:2::/64
   - **d.** A route for 3222:2:2:2::2/128
3. An engineer needs to add a static IPv6 route for prefix 2000:1:2:3::/64 to Router R5 in the figure. Which of the following **ipv6 route** commands would result in a working IPv6 route for this subnet prefix? (Choose two answers.)
   - **a.** **ipv6 route 2000:1:2:3::/64 G0/0/1 2000:1:2:56::6**
   - **b.** **ipv6 route 2000:1:2:3::/64 2000:1:2:56::6**
   - **c.** **ipv6 route 2000:1:2:3::/64 G0/0/1** 2001:1:2:56::5
   - **d.** **ipv6 route 2000:1:2:3::/64 G0/0/1**
4. An engineer needs to add a static IPv6 route for prefix 2000:1:2:3::/64 to Router R5 in the figure. Which of the following answers shows a valid static IPv6 route for that subnet on Router R5?
   - **a.** **ipv6 route 2000:1:2:3::/64 2000:1:2:56::5**
   - **b.** **ipv6 route 2000:1:2:3::/64 2000:1:2:56::6**
   - **c.** **ipv6 route 2000:1:2:3::/64 FE80::FF:FE00:5**
   - **d.** **ipv6 route 2000:1:2:3::/64 FE80::FF:FE00:6**
5. When displaying an IPv6 static default route, how does the **show ipv6 route** command represent the concept of the default destination?
   - **a.** With the phrase “Gateway of Last Resort” just above the list of routes
   - **b.** With a prefix value ::/0 in the line for the route
   - **c.** With a prefix value ::/128 in the line for the route

**d.** With a prefix value 2000::/3 in the line for the route

**e.** With the keyword "default" in the line for the route

6. Router R1 has a useful IPv6 configuration so that Router R1 learns a route for subnet 2001:db8:1:2::/64 with OSPF. The route lists an outgoing interface that is an Ethernet WAN link. The engineer then installs cellular interfaces on both routers for WAN backup. As part of the configuration, the engineer configures the **ipv6 route 2001:db8:1:2::/64 cellular0/2/0 200** command. What does the router do in response?

   **a.** It rejects the **ipv6 route** command because of the existing OSPF route for the same prefix.

   **b.** It accepts the **ipv6 route** command into the configuration but does not add a route to the routing table.

   **c.** It accepts the **ipv6 route** command into the configuration and adds the route to the routing table, but it leaves the OSPF-learned route in the routing table.

   **d.** It accepts the **ipv6 route** command into the configuration, adds the route to the routing table, and removes the OSPF-learned route in the routing table.

7. An engineer types the command **ipv6 route 2001:DB8:8:8::/64 2001:DB8:9:9::9 129** in configuration mode of Router R1 and presses Enter. Later, a **show ipv6 route** command lists no routes for subnet 2001:DB8:8:8::/64. Which of the following could have caused the route to not be in the IPv6 routing table?

   **a.** The command uses a next-hop global unicast address, which is not allowed, preventing the router from adding a route.

   **b.** The command must include an outgoing interface parameter, so IOS rejected the **ipv6 route** command.

   **c.** The router has no routes that match the next-hop address 2001:DB8:9:9::9.

   **d.** A route for 2001:DB8:8:8::/64 with administrative distance 110 already exists.

# Foundation Topics

## Connected and Local IPv6 Routes

A Cisco router adds IPv6 routes to its IPv6 routing table for several reasons. Many of you could predict those reasons at this point in your reading, in part because the logic mirrors the logic routers use for IPv4. Specifically, a router adds IPv6 routes based on the following:

Key Topic

- The configuration of IPv6 addresses on working interfaces (connected and local routes)
- The direct configuration of a static route (static routes)
- The configuration of a routing protocol, like OSPFv3, on routers that share the same data link (dynamic routes)

The first two sections of this chapter examine the first of these two topics, with discussions of IPv6 routing protocols residing in the CCNP Enterprise exams.

Also, as an early reminder of a few essential acronyms from earlier chapters, remember that GUA refers to global unicast addresses, the most common routable unicast address

configured on Enterprise router interfaces. Unique local addresses (ULA) serve as private addresses. Finally, routes frequently use a neighboring router's link-local address (LLA), a unicast address that exists on every router and host interface, useful for sending packets over the local link.

## Rules for Connected and Local Routes

Routers add and remove IPv6 connected and **IPv6 local routes** based on the interface configuration and state. First, the router looks for configured unicast addresses on any interfaces by looking for the **ipv6 address** command. Then, if the interface is working—if the interface has a "line status is up, protocol status is up" notice in the output of the **show interfaces** command—the router adds both a connected and local route.

> **NOTE** Routers do not create connected or local IPv6 routes for link-local addresses.

The connected and local routes follow the same general logic as with IPv4. The connected route represents the subnet connected to the interface, whereas the local route is a host route for only the specific IPv6 address configured on the interface.

For example, consider a router configured with a working interface with the **ipv6 address 2000:1:1:1::1/64** command. The router will calculate the subnet ID based on this address and prefix length and place a connected route for that subnet (2000:1:1:1::/64) into the routing table. The router also takes the listed IPv6 address and creates a local route for that address, with a /128 prefix length. (With IPv4, local routes have a /32 prefix length, while IPv6 uses a /128 prefix length, meaning "exactly this one address.")

The following list summarizes the rules about how routers create routes based on the configuration of an interface IPv6 unicast address, for easier review and study:

Key Topic

1. Routers create IPv6 routes based on each unicast IPv6 address on an interface, as configured with the **ipv6 address** command, as follows:
   a. The router creates a route for the subnet (a connected route).
   b. The router creates a local route (/128 prefix length) for the router IPv6 address (a local route).
2. Routers do not create routes based on the link-local addresses associated with the interface.
3. Routers remove the connected and local routes for an interface if the interface fails, and they re-add these routes when the interface is again in a working (up/up) state.

## Example of Connected IPv6 Routes

While the concept of connected and local IPv6 routes works much like IPv4 routes, seeing a few examples can certainly help. To show some sample routes, Figure 29-1 details one sample internetwork used in this chapter. The figure shows the IPv6 subnet IDs. The upcoming examples focus on the connected and local routes on Router R1.

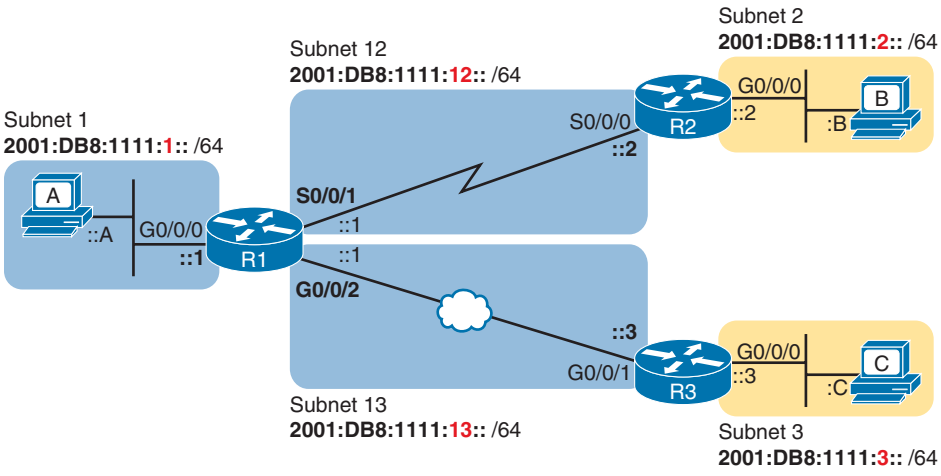



**Figure 29-1** *Sample Network Used to Show Connected and Local Routes*

To clarify the notes in Figure 29-1, note that the figure shows IPv6 prefixes (subnets), with a shorthand notation for the interface IPv6 addresses. The figure shows only the abbreviated interface ID portion of each interface address near each interface. For example, R1's G0/0/0 interface address would begin with subnet ID value 2001:DB8:1111:1, added to ::1, for 2001:DB8:1111:1::1.

Now on to the example of connected routes. To begin, consider the configuration of Router R1 from Figure 29-1, as shown in Example 29-1. The excerpt from the **show running-config** command on R1 shows three working interfaces. Also note that no static route or routing protocol configuration exists.

**Example 29-1** *IPv6 Addressing Configuration on Router R1*

```
ipv6 unicast-routing
!
! Unused interfaces omitted
!
interface GigabitEthernet0/0/0
 ipv6 address 2001:DB8:1111:1::1/64
!
interface Serial0/0/1
 ipv6 address 2001:db8:1111:12::1/64
!
interface GigabitEthernet0/0/2
 ipv6 address 2001:db8:1111:13::1/64
```

Based on Figure 29-1 and Example 29-1, R1 should have three connected IPv6 routes, as highlighted in Example 29-2.

---

Answers to the "Do I Know This Already?" quiz:

**1** A, C **2** A, C **3** A, B **4** B **5** B **6** B **7** C

**Example 29-2** *Routes on Router R1 Before Adding Static Routes or Routing Protocols*

```
R1# show ipv6 route
IPv6 Routing Table - default - 7 entries
Codes: C - Connected, L - Local, S - Static, U - Per-user Static route
       B - BGP, HA - Home Agent, MR - Mobile Router, R - RIP
       H - NHRP, I1 - ISIS L1, I2 - ISIS L2, IA - ISIS interarea
       IS - ISIS summary, D - EIGRP, EX - EIGRP external, NM - NEMO
       ND - ND Default, NDp - ND Prefix, DCE - Destination, NDr - Redirect
       RL - RPL, O - OSPF Intra, OI - OSPF Inter, OE1 - OSPF ext 1
       OE2 - OSPF ext 2, ON1 - OSPF NSSA ext 1, ON2 - OSPF NSSA ext 2
       la - LISP alt, lr - LISP site-registrations, ld - LISP dyn-eid
       lA - LISP away, a - Application
C   2001:DB8:1111:1::/64 [0/0]
     via GigabitEthernet0/0/0, directly connected
L   2001:DB8:1111:1::1/128 [0/0]
     via GigabitEthernet0/0/0, receive
C   2001:DB8:1111:12::/64 [0/0]
     via Serial0/0/1, directly connected
L   2001:DB8:1111:12::1/128 [0/0]
     via Serial0/0/1, receive
C   2001:DB8:1111:13::/64 [0/0]
     via GigabitEthernet0/0/2, directly connected
L   2001:DB8:1111:13::1/128 [0/0]
     via GigabitEthernet0/0/2, receive
L   FF00::/8 [0/0]
     via Null0, receive
```

All three highlighted routes show the same basic kinds of information, so for discussion, focus on the first pair of highlighted lines, which details the connected route for subnet 2001:DB8:1111:1::/64. The first pair of highlighted lines states: The route is a "directly connected" route; the interface ID is GigabitEthernet0/0/0; and the prefix/length is 2001:DB8:1111:1::/64. At the far left, the code letter "C" identifies the route as a connected route (per the legend above). Also note that the numbers in brackets mirror the same ideas as IPv4's **show ip route** command: The first number represents the administrative distance, and the second is the metric.

## Examples of Local IPv6 Routes

Continuing this same example, three local routes should exist on R1 for the same three interfaces as the connected routes. Indeed, that is the case, with one extra local route for other purposes. Example 29-3 shows only the local routes, as listed by the **show ipv6 route local** command, with highlights of one particular local route for discussion.

**Example 29-3** *Local IPv6 Routes on Router R1*

```
R1# show ipv6 route local
! Legend omitted for brevity

L   2001:DB8:1111:1::1/128 [0/0]
     via GigabitEthernet0/0/0, receive
L   2001:DB8:1111:4::1/128 [0/0]
     via Serial0/0/1, receive
L   2001:DB8:1111:5::1/128 [0/0]
     via GigabitEthernet0/0/2, receive
L   FF00::/8 [0/0]
     via Null0, receive
```

For the highlighted local route, look for a couple of quick facts. First, look back to R1's configuration in Example 29-1, and note R1's IPv6 address on its G0/0/0 interface. This local route lists the exact same address. Also note the /128 prefix length, meaning this route matches packets sent to that address (2001:DB8:1111:1::1), and only that address.

**NOTE** While the **show ipv6 route local** command shows all local IPv6 routes, the **show ipv6 route connected** command shows all connected routes.

## Static IPv6 Network Routes

IPv6 static routes require direct configuration with the **ipv6 route** command. Simply put, someone configures the command, and the router places the details from the command into a route in the IPv6 routing table.

The IPv6 network route is the most common type of static route among the types mentioned in the CCNA exam topics (network, host, default, and floating static). A network route, by definition, defines a route to the addresses in a subnet based on the listed prefix and prefix length. The **ipv6 route** global command also lists the forwarding instructions of how this router should forward packets toward that destination prefix by listing the outgoing interface or the address of the next-hop router.

Figure 29-2 shows the general concepts behind a single **ipv6 route** command for a network route. The figure shows the ideas behind a static route on Router R1 for the subnet on the right (subnet 2, or 2001:DB8:1111:2::/64). The command begins with **ipv6 route 2001:DB8:1111:2::/64**, defining the destination subnet. The rest of the command defines the forwarding instructions—either the outgoing interface (S0/0/1), the next-hop IPv6 address, or both.

Now that you understand the big ideas with IPv6 static network routes, the next few pages walk you through a series of examples. In particular, the examples look at configuring static routes with an outgoing interface, then with a next-hop GUA, and then with a next-hop LLA.

Key Topic

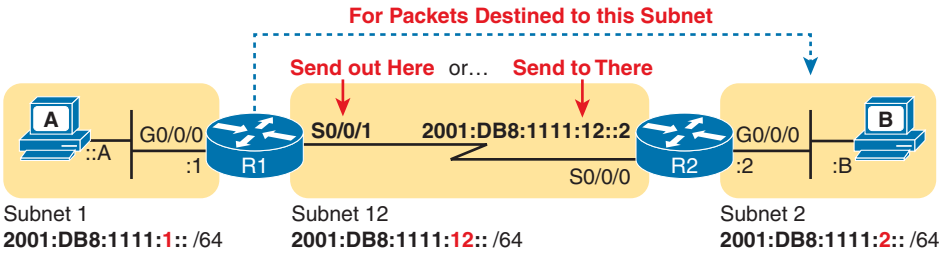



**Figure 29-2** *Logic Behind IPv6 Static Route Commands (IPv6 Route)*

## Static Network Routes Using an Outgoing Interface

This first IPv6 static route example uses the outgoing interface option. As a reminder, for both IPv4 and IPv6 static routes, when the command references an interface, the interface is a local interface; that is, it is an interface on the router where the command is added. In this case, as shown in Figure 29-2, R1's **ipv6 route** command would use interface S0/0/1, as shown in Example 29-4.

**Example 29-4** *Static IPv6 Routes on Router R1*

```
! Static route on router R1
R1(config)# ipv6 route 2001:db8:1111:2::/64 S0/0/1
```

If you were attempting to support packet flow between all hosts, like PCs A and B in the figure, you need static routes on each router for each remote subnet. For example, to support traffic between hosts A and B, R1 is now prepared. Host A will forward all its IPv6 packets to its default router (R1), and R1 can now route those packets destined for host B's subnet out R1's S0/0/1 interface to R2 next. However, Router R2 does not yet have a route that matches addresses in host A's subnet (2001:DB8:1111:1::/64), so a complete static routing solution requires more routes.

Example 29-5 solves this problem by giving Router R2 a static route for subnet 1 (2001:DB8:1111:1::/64). After this route is added, hosts A and B should be able to ping each other.

**Example 29-5** *Static IPv6 Routes on Router R2*

```
! Static route on router R2
R2(config)# ipv6 route 2001:db8:1111:1::/64 s0/0/0
```

Many options exist for verifying the existence of the static route and testing whether hosts can use the route. **ping** and **traceroute** can test connectivity. From the router command line, the **show ipv6 route** command will list all the IPv6 routes. The shorter output of the **show ipv6 route static** command, which lists only static routes, could also be used; Example 29-6 shows that output, with the legend omitted.

**Example 29-6** *Verification of Static Routes Only on R1*

```
R1# show ipv6 route static
! Legend omitted for brevity
S   2001:DB8:1111:2::/64 [1/0]
     via Serial0/0/1, directly connected
```

This command lists many facts about the one static route on R1. First, the code "S" in the left column does identify the route as a static route. (However, the later phrase "directly connected" might mislead you to think this is a connected route; trust the "S" code.) Note that the prefix (2001:DB8:1111:2::/64) matches the configuration (in Example 29-4), as does the outgoing interface (S0/0/1).

When working with IPv6 routes, you will often wonder which route the router will match for a given destination address. You should be ready to think through that question while examining a full IPv6 routing table, but if answering a lab question on the exam, you can use a command that tells you the specific route, as seen in Example 29-7. For example, if host A sent an IPv6 packet to host B (2001:DB8:1111:2::B), would R1 use this static route? As it turns out, R1 would use that route, as confirmed by the **show ipv6 route 2001:DB8:1111:2::B** command. This command asks the router to list the route that the router would use when forwarding packets to that particular address.

**Example 29-7** *Displaying the Route R1 Uses to Forward to Host B*

```
R1# show ipv6 route 2001:db8:1111:2::b
Routing entry for 2001:DB8:1111:2::/64
  Known via "static", distance 1, metric 0
  Route count is 1/1, share count 0
  Routing paths:
    directly connected via Serial0/0/1
      Last updated 00:01:29 ago
```

**NOTE** Only serial interfaces, and not Ethernet interfaces, support using an **ipv6 route** command with only an outgoing interface as the forwarding instructions. With Ethernet interfaces, the router will accept the command, but the router cannot forward packets using the route. See the later section, "Troubleshooting Static IPv6 Routes," for more details on this case and other challenges with static IPv6 routes.

## Static Network Routes Using Next-Hop IPv6 Address

IPv6 static route commands can use only the next-hop router address or a combination of the next-hop router address plus the outgoing interface. Note that these options work when the route forwards packets out any kind of interface, including Ethernet and serial links, but the upcoming examples show an Ethernet WAN link.

Interestingly, IPv6 supports using the neighbor's GUA, ULA, or LLA as the next-hop IPv6 address. The following list provides the working combinations, with Figure 29-3 depicting the logic from R1's perspective for a route with R3 as the next-hop router:

- Next-hop GUA or ULA only
- Next-hop GUA or ULA and outgoing interface
- Next-hop LLA and outgoing interface

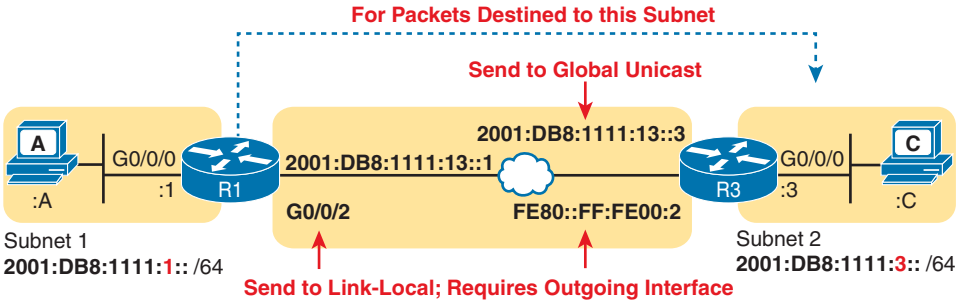



**Figure 29-3** *Using GUA or LLA as the Next-Hop Address for Static IPv6 Routes*

The next few pages walk you through examples, first with a GUA and then with an LLA as a next-hop router.

## Example Static Network Route with a Next-Hop GUA

In Example 29-8, R1 adds a static route that refers to neighbor R3's GUA per Figure 29-3. The command lists subnet 3 (2001:DB8:1111:3::/64), which exists on the far side of Router R3. The route then lists R3's GUA (ending in 13::3), the GUA on the shared subnet between R1 and R3, as the next-hop address.

**Example 29-8** *Static IPv6 Routes Using Global Unicast Addresses*

```
!
R1(config)# ipv6 route 2001:db8:1111:3::/64 2001:DB8:1111:13::3
```

The first two commands in Example 29-9 list detail about the static route configured in Example 29-8. The first command lists all static routes on Router R1, currently just the one static route per Example 29-8. The second command, **show ipv6 route 2001:DB8:1111:3::33**, asks R1 to identify the route it would use to forward packets to the address listed in the command (Host C in the figure uses address **2001:DB8:1111:3::33**). The output details the just-configured static route, proving that R1 uses this new static route when forwarding packets to that host.

**Example 29-9** *Verification of Static Routes to a Next-Hop Global Unicast Address*

```
R1# show ipv6 route static
! Legend omitted for brevity
S   2001:DB8:1111:3::/64 [1/0]
     via 2001:DB8:1111:13::3
```

```
R1# show ipv6 route 2001:db8:1111:3::33
Routing entry for 2001:DB8:1111:3::/64
  Known via "static", distance 1, metric 0
  Route count is 1/1, share count 0
  Routing paths:
    2001:DB8:1111:13::3
      Route metric is 0, traffic share count is 1
      Last updated 00:07:43 ago
```

Interestingly, the **ipv6 route 2001:db8:1111:3::/64 2001:db8:1111:13::3** command in Example 29-8 works, but you might wonder how the router knows what outgoing interface to use because the command does not list it. The router does need to identify the outgoing interface, so it uses an iterative process by finding the route to reach the next-hop GUA. The first iteration happens when the router matches the packet's destination address to the static route. The second iteration occurs when the router finds the route to use when forwarding packets to the next-hop address in the first route. That second iteration matches the route for the subnet to which both the local router and the next-hop router connect—in this case, subnet prefix 2001.db8:1111:13::/64 connected to R1's G0/0/2.

Example 29-10 shows some notes about the iterative lookup to find the outgoing interface. The first command lists the connected route that Router R1 uses that matches next-hop GUA 2002:db8:1111:13::3, namely the route for prefix 2001:db8:1111:13::/64. But the **show ipv6 static detail** command shows the results of the router's iterative route lookup, listing interface G0/0/2 as the outgoing interface. It also lists a code of *, informing us that the router added the route to the Routing Information Base (RIB), referring to the IPv6 routing table.

**Example 29-10** *Understanding the Iterative Lookup to Find the Outgoing Interface*

```
R1# show ipv6 route connected
! Lines omitted to reveal route to the subnet between R1 and R3
C   2001:DB8:1111:13::/64 [0/0]
     via GigabitEthernet0/0/2, directly connected

R1# show ipv6 static detail
IPv6 Static routes Table - default
Codes: * - installed in RIB, u/m - Unicast/Multicast only
Codes for []: P - permanent I - Inactive permanent
       U - Per-user Static route
       N - ND Static route
       M - MIP Static route
       P - DHCP-PD Static route
       R - RHI Static route
       V - VxLan Static route
*   2001:DB8:1111:3::/64 via 2001:DB8:1111:13::3, distance 1
     Resolves to 1 paths (max depth 1)
     via GigabitEthernet0/0/2
```

You can also configure a static IPv6 route with the next-hop GUA plus the outgoing interface (sometimes called a fully specified route). Doing so means that the router does not need to perform the iterative route lookup. Example 29-11 shows just such a case, replacing the static route in Example 29-8 with one that also lists both the next-hop GUA plus G0/0/2 as the outgoing interface. The end of the example confirms that the route listed in the IPv6 routing table lists the next-hop GUA and outgoing interface.

**Example 29-11** *Alternate IPv6 Static Route with Outgoing Interface and GUA*

```
R1# show running-config | include ipv6 route
ipv6 route 2001:DB8:1111:3::/64 GigabitEthernet0/0/2 2001:DB8:1111:13::3

R1# show ipv6 route | section 2001:DB8:1111:3::/64
S   2001:DB8:1111:3::/64 [1/0]
     via 2001:DB8:1111:13::3, GigabitEthernet0/0/2

R1# show ipv6 static detail
! Legend omitted
*   2001:DB8:1111:3::/64 via 2001:DB8:1111:13::3, GigabitEthernet0/0/2, distance 1
```

### Example Static Network Route with a Next-Hop LLA

Static routes that refer to a neighbor's LLA work a little like the preceding two styles of static routes. First, the **ipv6 route** command refers to a next-hop address, namely an LLA; however, the command must also refer to the router's local outgoing interface. Why both? The **ipv6 route** command cannot simply refer to an LLA next-hop address by itself because the iterative route lookup used with GUAs will not work with LLAs. There are no routes for LLAs, so there is no way for a router to use only the LLA as a static route's forwarding instructions. In short, if using the LLA, you also need to configure the outgoing interface.

Figure 29-4 depicts the logic for using both the LLA and outgoing interface to replace the example static route configured in Example 29-8 and explained with Examples 29-9 and 29-10. In this case, the R1 configuration refers to R1's outgoing interface and to Router R2's LLA. The figure also shows the global configuration command required on Router R1.

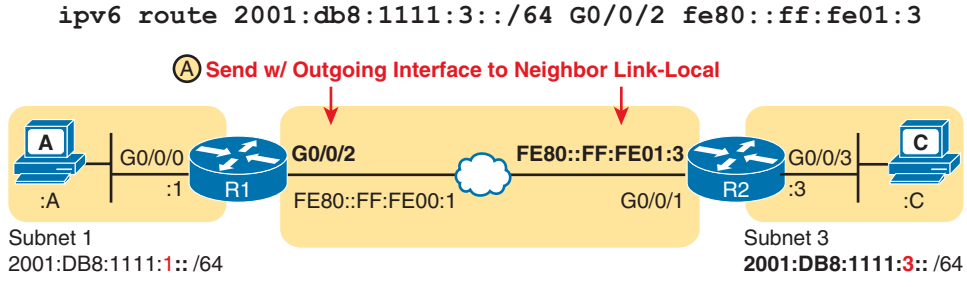



**Figure 29-4** *Example Using LLAs and Outgoing Interface for IPv6 Static Routes*

Example 29-12 shows the configuration of the LLA-based static route on Router R1, with some purposeful mistakes along the way. The first attempt with the **ipv6 route** command omits the outgoing interface, so IOS rejects the command. The second attempt shows the

command after using the up-arrow to retrieve the command and then adding the outgoing interface (G0/0/2) to the end of the command, but that command uses incorrect syntax. The interface ID should *precede* the address. The final **ipv6 route** command shows the correct syntax with IOS accepting the command.

**Example 29-12** *Static IPv6 Network Route Using LLA*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# ipv6 route 2001:db8:1111:3::/64 fe80::ff:fe01:3
% Interface has to be specified for a link-local nexthop
R1(config)# ipv6 route 2001:db8:1111:3::/64 fe80::ff:fe00:2 g0/0/2
                                                           ^
% Invalid input detected at '^' marker.

R1(config)# ipv6 route 2001:db8:1111:3::/64 g0/0/2 fe80::ff:fe01:3
R1(config)#^Z
```

Example 29-13 verifies the configuration in Example 29-12 by repeating the **show ipv6 route static** and **show ipv6 route 2001:DB8:1111:3::33** commands used in Example 29-9. Note that the output from both commands differs slightly versus the earlier example. Because the new configuration commands list both the next-hop address and outgoing interface, the **show** commands in Example 29-13 also list both the next-hop (LLA) and the outgoing interface. If you refer back to Example 29-9, you will see only a next-hop address listed.

**Example 29-13** *Verification of Static Routes to a Next-Hop Link-Local Address*

```
R1# show ipv6 route static
! Legend omitted for brevity

S   2001:DB8:1111:3::/64 [1/0]
      via FE80::FF:FE00:2, GigabitEthernet0/0/2

R1# show ipv6 route 2001:db8:1111:3::33
Routing entry for 2001:DB8:1111:3::/64
  Known via "static", distance 1, metric 0
  Route count is 1/1, share count 0
  Routing paths:
    FE80::FF:FE00:2, GigabitEthernet0/0/2
      Route metric is 0, traffic share count is 1
      Last updated 00:01:01 ago
```

In summary, IPv6 static network routes list a prefix/prefix-length along with forwarding instructions. The forwarding instructions allow for unicast address types—GUA, ULA, or even LLAs. However, there are some restrictions about which options you can use with forwarding instructions. Table 29-2 summarizes the options.

Key Topic

**Table 29-2** Summary Table for IPv6 Network Route Configuration

| Forwarding Instructions | Serial | Ethernet |
|---|---|---|
| Interface only | Yes | No* |
| GUA/ULA only | Yes | Yes |
| Interface plus GUA/ULA | Yes | Yes |
| Interface plus LLA | Yes | Yes |
| LLA only | No | No |

* The **ipv6 route** command can be configured with an outgoing Ethernet interface but no next-hop address, but the route does not forward packets.

# Static Default, Host, and Floating Static IPv6 Routes

If you have been reading this book sequentially, the following topics should be relatively easy. Engineers use IPv6 default, host, and floating static routes for the same purposes as their IPv4 cousins. The only difference comes in the combinations of how to configure the next-hop addresses and outgoing interfaces for IPv6, a topic you just finished learning about in this chapter. Hopefully, most of the following few pages will be both a review of IPv4 default, host, and floating static concepts, along with helpful examples for IPv6 routes.

## Static IPv6 Default Routes

IPv6 supports a default route concept, similar to IPv4. The default route tells the router what to do with an IPv6 packet when the packet matches no other IPv6 route. The straightforward router forwarding logic when no routes match a packet's destination is

- With no default route, the router discards the IPv6 packet.
- With a default route, the router forwards the IPv6 packet based on the default route.

Default routes can be beneficial in a couple of network design cases. For example, with an enterprise network design that uses a single router at each branch office, with one WAN link to each branch, the branch routers have only one possible path over which to forward packets. When using a routing protocol in a large network, the branch router could learn thousands of routes, all of which forward packets over that one WAN link.

Branch routers could use default routes instead of a routing protocol. The branch router would forward all traffic to the core of the network. Figure 29-5 shows just such an example, with two sample branch routers on the right and a core site router on the left, with the default route shown as the dashed line.

To configure a static default route, use a specific value to note the route as a default route: ::/0. An entire prefix of a double colon (::) with no other digits represents an all 0s prefix, and the /0 prefix length signifies that zero of the initial bits must match to be part of the same group. So, ::/0 serves as the abbreviation of "all IPv6 addresses." This idea mirrors the IPv4 convention to refer to the default route as 0.0.0.0/0 in **show** command output or 0.0.0.0 0.0.0.0 in the **ip route** command.

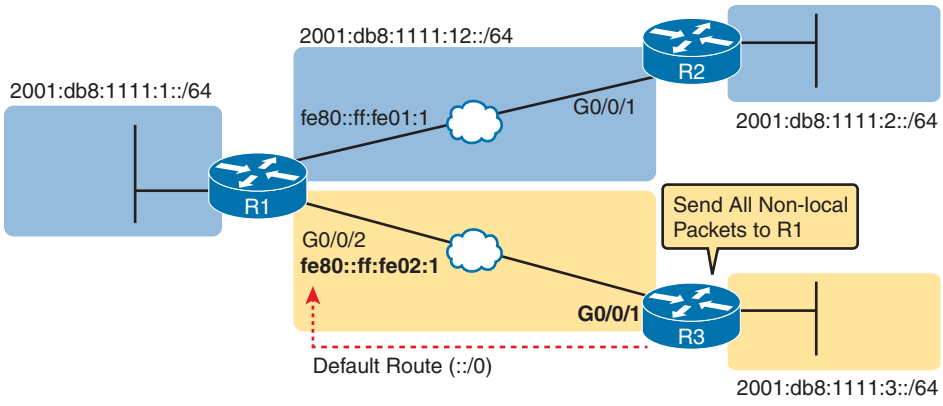



**Figure 29-5** *Using Static Default Routes at Branch Routers*

To configure a static default route, use ::/0 as the destination, and otherwise, just configure the **ipv6 route** command as normal, with the same conventions about forwarding instructions as discussed earlier in this chapter. Example 29-14 shows one such sample static default route on Router R3 from Figure 29-5. The configuration uses next-hop LLA and, therefore, also requires the outgoing interface.

**Example 29-14** *Static Default Route for Branch Router R3*

```
! Forward out R3's G0/0/1 local interface to R1's G0/0/2 LLA FE80::FF:FE02:1.
R3(config)# ipv6 route ::/0 g0/0/1 fe80::ff:fe02:1
```

Cisco IOS provides a simpler representation of IPv6 default routes compared to IPv4 default routes. For instance, the **show ipv6 route** command output does not list a "Gateway of Last Resort" as with IPv4. Instead, it lists the default route destination as ::/0, just as in the **ipv6 route** configuration command. Example 29-15 shows Router R3's routing table, with only the default route, plus the expected connected and local routes for its two working interfaces. The **show ipv6 route ::/0** command output, at the bottom of the example, shows more detail about the default route, including confirmation of the configured prefix of ::/0 and the next-hop LLA and outgoing interface.

**Example 29-15** *Router R3's Static Default Route (Using Outgoing Interface)*

```
R3# show ipv6 route
IPv6 Routing Table - default - 6 entries
Codes: C - Connected, L - Local, S - Static, U - Per-user Static route
       B - BGP, R - RIP, H - NHRP, I1 - ISIS L1
       I2 - ISIS L2, IA - ISIS interarea, IS - ISIS summary, D - EIGRP
       EX - EIGRP external, ND - ND Default, NDp - ND Prefix, DCE - Destination
       NDr - Redirect, O - OSPF Intra, OI - OSPF Inter, OE1 - OSPF ext 1
       OE2 - OSPF ext 2, ON1 - OSPF NSSA ext 1, ON2 - OSPF NSSA ext 2
       a - Application, m - OMP
S   ::/0 [1/0]
     via FE80::FF:FE02:1, GigabitEthernet0/0/1
```

```
C   2001:DB8:1111:3::/64 [0/0]
     via GigabitEthernet0/0/0, directly connected
L   2001:DB8:1111:3::3/128 [0/0]
     via GigabitEthernet0/0/0, receive
C   2001:DB8:1111:13::/64 [0/0]
     via GigabitEthernet0/0/1, directly connected
L   2001:DB8:1111:13::3/128 [0/0]
     via GigabitEthernet0/0/1, receive
L   FF00::/8 [0/0]
     via Null0, receive

R3# show ipv6 route ::/0
Routing entry for ::/0
  Known via "static", distance 1, metric 0
  Route count is 1/1, share count 0
  Routing paths:
    FE80::FF:FE02:1, GigabitEthernet0/0/1
      Route metric is 0, traffic share count is 1
      Last updated 00:04:04 ago
```

## Static IPv6 Host Routes

Both IPv4 and IPv6 allow the definition of static host routes—that is, a route to a single host IP address. With IPv4, those routes use a /32 mask, which identifies a single IPv4 address in the **ip route** command; with IPv6, a /128 mask identifies that single host in the **ipv6 route** command.

As for the syntax, an **IPv6 host route** follows the same rules as an IPv6 network route regarding the forwarding instructions. The only difference comes in the configuration of a full 128-bit address along with a prefix length of /128 so that the route matches a single IPv6 address.

You might never need to use a host route in a real network, but the motivation usually comes from some business reason to forward packets to one host over a different route that the routing protocol would choose. Figure 29-6 shows just such an example. The route for all hosts in the subnet flows over the lower part of the topology; however, you might have business reasons to forward traffic sent to host D over the upper path through Router R2, requiring a host route.

Example 29-16 shows the configuration based on Figure 29-6, with the host route for host D using R2 as the next-hop router, and a static network route for host D's subnet using Router R3 as the next-hop router.

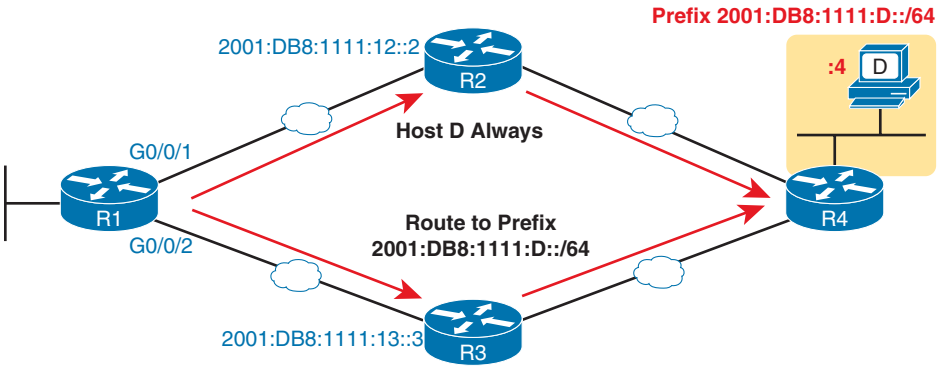



**Figure 29-6** *An Example Topology for an IPv6 Host Route*

**Example 29-16** *Static Host IPv6 Routes on R1, for Host D*

```
! The first command shows the host route, and the second shows the network route.
! Both use next-hop GUA and outgoing interface for clarity.
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# ipv6 route 2001:db8:1111:d::4/128 g0/0/1 2001:db8:1111:12::2
R1(config)# ipv6 route 2001:db8:1111:d::/64 g0/0/2 2001:db8:1111:13::3
R1(config)#^Z
R1#
```

Adding IPv6 host routes typically causes overlapping routes, as seen in this case. When choosing to place routes into the IPv6 routing table, a router always attempts to place the best route for each prefix (subnet) into the IP routing table. That rule applies to each subnet prefix, defined by each combination of prefix and prefix length. In this case, the two static routes have different prefix lengths, so it places both routes into the routing table, as seen in the output from the first command in Example 29-17.

**Example 29-17** *IPv6 Host Route for Host D with Overlapping Network Route*

```
R1# show ipv6 route static
! Legend omitted
S   2001:DB8:1111:D::/64 [1/0]
     via 2001:DB8:1111:13::3, GigabitEthernet0/0/2
S   2001:DB8:1111:D::4/128 [1/0]
     via 2001:DB8:1111:12::2, GigabitEthernet0/0/1

R1# show ipv6 route 2001:db8:1111:d::4
Routing entry for 2001:DB8:1111:D::4/128
  Known via "static", distance 1, metric 0
  Route count is 1/1, share count 0
  Routing paths:
    2001:DB8:1111:12::2, GigabitEthernet0/0/1
      Route metric is 0, traffic share count is 1
```

```
      Last updated 00:02:28 ago

R1# show ipv6 route 2001:db8:1111:d::3
Routing entry for 2001:DB8:1111:D::/64
  Known via "static", distance 1, metric 0
  Route count is 1/1, share count 0
  Routing paths:
    2001:DB8:1111:13::3, GigabitEthernet0/0/2
      Route metric is 0, traffic share count is 1
      Last updated 00:00:43 ago
```

As with IPv4 routes, IPv6 matches a packet to the IPv6 routing table using longest-match logic. Packets sent to host D's address match both routes configured in Example 29-16, but the router chooses the longer prefix (more binary 1s in the prefix length) /128 host route. Stated differently, when matching more than one route, the router uses the route that defines the most specific set of addresses.

The final two **show** commands in Example 29-17 demonstrate Router R1's matching logic. The **show ipv6 route 2001:db8:1111:d::4** command asks the router what route it would use to match packets sent to host D's address, and the output describes the /128 host route, with R2's GUA as the next-hop address. Conversely, the final command lists a different IPv6 address in the same subnet as host D, so R1 will not match the host route. Instead, it matches the network route, listing outgoing interface G0/0/2 with R3's GUA as next-hop address.

## Floating Static IPv6 Routes

Floating static routes are the last of the four specific static route types mentioned in the CCNA exam topics. The general concept works the same for both IPv4 and IPv6 floating static routes, so this section will provide a brief introduction. For more detail on the concepts, please refer to the section "Floating Static Routes" in Chapter 17, "Configuring IPv4 Addresses and Static Routes."

Floating static routes let a router direct traffic over a switched WAN connection, for instance, over a cellular interface—but only when the normally working WAN link fails. For instance, in Figure 29-7, the Ethernet WAN link typically works for months. OSPF learns IPv6 routes using that link. During those same times, the cellular interfaces remain unused, with no cellular call/link between the routers, and no IPv6 routes that use the cellular interfaces.

Floating static routes provide a method to direct packets over the cellular link, which triggers a phone call to create a link between the two routers—but only when the routes that use the primary link fail. To do so, you configure a static route that uses the cellular interface, and the router stores that route in its configuration. However, the router does not add the route to the routing table because of the existing OSPF-learned route for the exact same prefix/length. Once the primary routes fail, the routers add the floating static routes to their routing tables, directing traffic over the cellular link. In this scenario, the cellular link and the floating static route get used only during outages to the primary link.

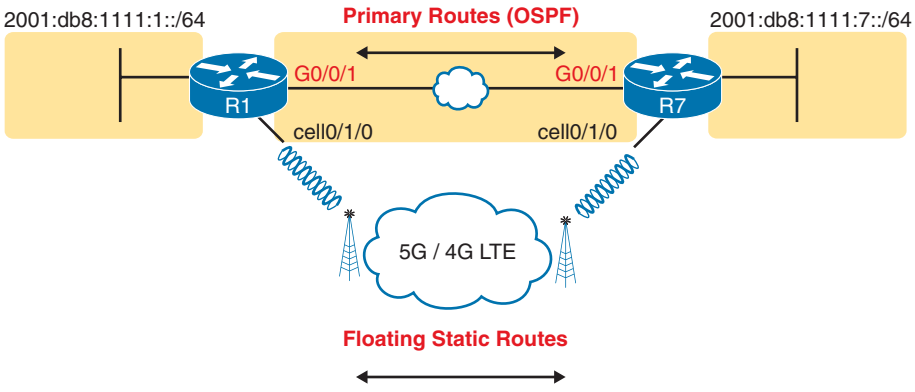



**Figure 29-7** *Using a Floating Static Route to Key Subnet 2001:DB8:1111:7::/64*

Routers rely on each route's **IPv6 administrative distance** (AD) parameter to judge between different sources of routing information (for example, OSPF, EIGRP, or static routes) for routes for the same prefix/length. For instance, in Figure 29-7, an OSPF-learned route on R1, for prefix 2001:db8:1111:7::/64, would have a default AD of 110. Static routes default to an AD of 1, so Router R1 would normally prefer the static route configured with the **ipv6 route 2001:db8:1111:7::/64 cell0/1/0** command versus the OSPF-learned AD 110 route to that same subnet, and replace the OSPF-learned route with the static route.

Floating static routes remain out of the routing table by using a higher (worse) AD setting than the normally used routes. In the recent example, by setting the static route's AD to something higher than OSPF's 110—for instance, with the **ipv6 route 2001:db8:1111:7::/64 cell0/1/0 130** command—Router R1 will instead prefer the AD 110 OSPF-learned route instead of the AD 130 static route.

You can find the AD of a route in the routing table with the usual **show** commands. For instance, the **show ipv6 route** command lists each route's administrative distance as the first of the two numbers inside the brackets. The second number in brackets is the metric. Also, commands like **show ipv6 route** *prefix/length* give detail about one specific route; this command lists the AD simply as "distance."

Table 29-3 lists the various default IPv6 AD values. These defaults mirror the same default AD settings for IPv4 as seen in Table 24-4 in this book. Compared to Table 24-4, this table adds the entry for NDP (which does not apply to IPv4) and removes the mention of the IPv4 DHCP-based default route (AD 254) because IPv6 does not advertise default routes with DHCP.

Key Topic

**Table 29-3** Default IPv6 Administrative Distances

| Route Type | Administrative Distance |
|---|---|
| Connected | 0 |
| Static | 1 |
| NDP default route | 2 |
| BGP (external routes [eBGP]) | 20 |
| EIGRP (internal routes) | 90 |
| IGRP | 100 |

| Route Type | Administrative Distance |
|---|---|
| OSPF | 110 |
| IS-IS | 115 |
| RIP | 120 |
| EIGRP (external routes) | 170 |
| BGP (internal routes [iBGP]) | 200 |
| Unusable | 255 |

## Troubleshooting Static IPv6 Routes

IPv6 static routes have the same potential issues and mistakes as do static IPv4 routes, as discussed in Chapter 17. However, IPv6 static routes do have a few small differences. This last part of the static route content in the chapter looks at troubleshooting IPv6 static routes, reviewing many of the same troubleshooting rules applied to IPv4 static routes, while focusing on the details specific to IPv6.

This topic breaks static route troubleshooting into two perspectives: cases in which the router adds the route to the routing table but the route does not work, and cases in which the router does not add the route to the routing table.

### Troubleshooting Incorrect Static Routes That Appear in the IPv6 Routing Table

A static route is only as good as the input typed into the **ipv6 route** command. IOS checks the syntax of the command, of course. However, IOS cannot tell if you choose the incorrect outgoing interface, next-hop address, or prefix/prefix-length in a static route. If the parameters pass the syntax checks, IOS places the **ipv6 route** command into the running-config file. Then, if no other problem exists (as discussed at the next heading), IOS puts the route into the IP routing table—even though the route may not work because of the poorly chosen parameters.

For instance, a static route that references a next-hop address should list the address of some other router on a subnet that connects the two routers. Imagine Router R1 uses an address 2001:1:1:1::1 on that subnet, with Router R2 using 2001:1:1:1::2. R1's **ipv6 route** command ought to refer to Router R2' 2001:1:1:1::2 address, but nothing prevents the mistake of configuring Router R1's 2001:1:1:1::1 address or any other address in that subnet other than 2001:1:1:1::/64. Router R1 allows the command and adds the route to the IPv6 routing table, but the route cannot possibly forward packets correctly.

When you see an exam question that has static routes, and you see them in the output of **show ipv6 route**, remember that the routes may have incorrect parameters. Mentally review this list of questions that confirm correct configuration and reveal common mistakes, ensuring an answer of yes to each relevant question.

Key Topic

**Step 1.** **Prefix/Length:** Does the **ipv6 route** command reference the correct subnet ID (prefix) and mask (prefix length)?

**Step 2.** **Next-hop LLA:** If configuring a next-hop LLA:

**a.** Does the command also include an outgoing interface?

b. Is the outgoing interface the correct interface (on the router on which the command is configured) based on the topology?

c. Is the configured LLA the LLA of the neighboring router on the same link as the listed outgoing interface?

**Step 3.** **Next-hop GUA/ULA only:** If configuring a next-hop GUA or ULA but no outgoing interface:

a. Does the configured GUA/ULA refer to the neighbor's address, specifically on the neighbor's interface that connects to the local router?

b. Does the local router have a working route that matches the next-hop GUA/ULA? (Usually a connected route.)

**Step 4.** **Outgoing Interface only:** If configuring an outgoing interface but no next-hop address:

a. Is the outgoing interface a serial interface—the only interface type that works in the **ipv6 route** command without also configuring a next-hop address?

b. Is the outgoing interface the correct interface (on the router on which the command is configured) based on the topology?

The preceding troubleshooting checklist works through the various cases in which IOS would accept the configuration of the static IPv6 route, but the route would not work because of the incorrect parameters in context. It helps to see a few examples. Figure 29-8 shows a sample network to use for the examples; all the examples focus on routes added to Router R1, for the subnet on the far right (note that this network uses different IPv6 addresses than the earlier figures).

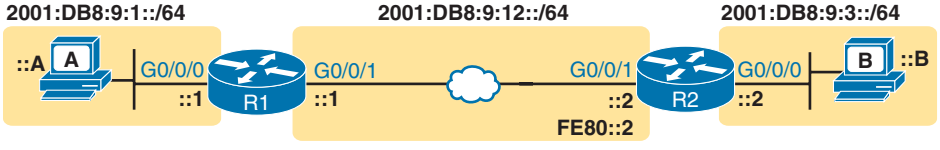



**Figure 29-8** *Sample Topology for Incorrect IPv6 Route Examples*

Example 29-18 shows several **ipv6 route** commands. Of the commands shown, IOS rejects the second command in the example (step 2A), but it accepts and adds routes to the routing table for the others. However, the routes in the accepted commands all have some issue that prevents the route from working as designed. Look for the short comment at the end of each configuration command to see why each is incorrect, and the list following the example for more detail.

**Example 29-18** **ipv6 route** *Commands with Correct Syntax but Incorrect Ideas*

```
ipv6 route 2001:DB8:9:33::/64 2001:DB8:9:12::2 ! Step 1: Wrong prefix
ipv6 route 2001:DB8:9:3::/64 FE80::2           ! Step 2A: Missing outgoing interface
ipv6 route 2001:DB8:9:3::/64 G0/0/0 FE80::2    ! Step 2B: Wrong interface on R1
ipv6 route 2001:DB8:9:3::/64 G0/0/1 FE80::1    ! Step 2C: Wrong neighbor link local
ipv6 route 2001:DB8:9:3::/64 2001:DB8:9:12::3  ! Step 3A: Wrong neighbor address
ipv6 route 2001:DB8:9:3::/64 G0/0/1            ! Step 4A: Also needs next-hop
```

**Step 1.** The prefix (2001:DB8:9:33::) has a typo in the fourth quartet (33 instead of 3). IOS accepts the command and now has a route for a nonexistent subnet.

**Step 2A.** The command uses the correct LLA as the next-hop address but fails to include any outgoing interface. IOS rejects the command.

**Step 2B.** The command uses the correct R2 LLA on the R1-R2 link (FE80::2 per the figure) but uses the incorrect R1 local interface of G0/0/0. (See the next example for more detail.)

**Step 2C.** The figure shows R2's G0/0/1 as using link-local address FE80::2, but the command uses FE80::1, R1's LLA.

**Step 3A.** The figure shows the subnet in the center as 2001:DB8:9:12::/64, with R1 using the ::1 address and R2 using ::2. This command uses a next-hop address ending in ::3 in that subnet that neither router uses.

**Step 4A.** The command lists only an outgoing interface of G0/0/1 (the correct interface), but no next-hop address. Only serial interfaces can be used with this syntax and have the route function correctly. Because it uses an Ethernet outgoing interface, the router needs the next-hop address in the command.

The key takeaway for this section is to know that a route in the IPv6 routing table might be incorrect due to poor choices for the parameters. The fact that a route is in the IPv6 routing table, particularly a static route, does not mean that it is correct.

## The Static Route Does Not Appear in the IPv6 Routing Table

The preceding few pages focused on IPv6 static routes that show up in the IPv6 routing table but unfortunately have incorrect parameters. The next page looks at IPv6 routes that have correct parameters, but IOS does not place them into the IPv6 routing table.



When you add an **ipv6 route** command to the configuration, and the syntax is correct, IOS has a second step in which it considers whether to add that route to the IPv6 routing table. IOS makes the following checks before adding the route; note that IOS uses this same kind of logic for IPv4 static routes:

Key Topic

- For **ipv6 route** commands that list an outgoing interface, that interface must be in an up/up state.
- For **ipv6 route** commands that list a GUA or ULA (that is, not an LLA), the local router must have a route to reach that next-hop address.
- If another IPv6 route exists for that exact same prefix/prefix-length, the static route must have a better (lower) administrative distance than the competing route.

Note that the first two bullet items can change even without any configuration. For instance, as interfaces fail and recover, the interface state will change and the connected IPv6 routes will be removed and reappear in the IPv6 routing table. Depending on the current status, the static routes that rely on those interfaces and routes will be added and removed from the IPv6 routing table.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 29-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 29-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Review memory tables | | Book, website |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 29-5** Key Topics for Chapter 29

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Methods by which a router can build IPv6 routes | 724 |
| List | Rules for IPv6 connected and local routes | 725 |
| Figure 29-2 | IPv6 static route concepts | 729 |
| Table 29-2 | Summary of IPv6 network route configuration | 735 |
| Table 29-3 | IPv6 default administrative distance values | 740 |
| Checklist | List of reasons for IPv6 static route problems that result in a route in the routing table but the route does not work correctly | 741 |
| Checklist | List of reasons for IPv6 static route problems that result in the route not appearing in the routing table | 743 |

## Key Terms You Should Know

IPv6 administrative distance, IPv6 host route, IPv6 local route, local route

## Command References

Tables 29-6 and 29-7 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 29-6** Chapter 29 Configuration Command Reference

| Command | Description |
|---|---|
| **ipv6 route** *prefix/length outgoing-interface* | Global command to define an IPv6 static route, with packets forwarded out the local router interface listed in the command |
| **ipv6 route** *prefix/length outgoing-interface next-hop-lla* | Global command to define an IPv6 static route, with a next-hop link-local address and the required local router outgoing interface |
| **ipv6 route** *prefix/length [outgoing-interface] next-hop-gua* | Global command to define an IPv6 static route, with the next-hop global unicast address and the optional local router outgoing interface |
| **ipv6 route ::/0** *{[next-hop-address] [outgoing-interface]}* | Global command to define a default IPv6 static route |

**Table 29-7** Chapter 29 EXEC Command Reference

| Command | Description |
|---|---|
| **show ipv6 route** [**connected** \| **local** \| **static**] | Lists routes in the routing table. |
| **show ipv6 route** *address* | Displays detailed information about the router's route to forward packets to the IPv6 address listed in the command. If no route matches, it displays "No route found." |
| **show ipv6 route** *prefix/length* | If a route that matches the listed prefix/length exists, the router displays detailed information about the route. |
| **show ipv6 route \| section** *prefix/length* | Displays an excerpt of the output from the **show ipv6 route** command of only the lines related to routes for the listed prefix/length. |
| **show ipv6 static detail** | Displays a static route legend with multiple output lines about each static route, noting whether it is installed in the routing table (RIB) and the administrative distance. |



# Part VII Review

Keep track of your part review progress with the checklist in Table P7-1. Details on each task follow the table.

**Table P7-1** Part VII Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Watch Videos | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, use the PTP software to answer the "Do I Know This Already?" questions again for the chapters in this part of the book.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or using the Key Topics application on the companion website.

## Do Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Blog: Config Labs:** The author's blog includes a series of configuration-focused labs that you can do on paper or with Cisco Packet Tracer in about 15 minutes. To find them, open https://www.certskills.com and look under the Labs menu item.

**Other:** If using other lab tools, here are a few suggestions: configure IPv6 addresses on interfaces, and before using any **show** commands, predict the connected and local routes that should be added to the IPv6 routing table, and predict the link-local (unicast) address and various multicast addresses you expect to see in the output of the **show ipv6 interfaces** command.

## Watch Video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

*This page intentionally left blank*

# Part VIII

## Exam Updates

**Chapter 30:** *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition Exam Updates



CHAPTER 30

# *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition Exam Updates

## The Purpose of This Chapter

For all the other chapters, the content should remain unchanged throughout this edition of the book. Instead, this chapter will change over time, with an updated online PDF posted so you can see the latest version of the chapter even after you purchase this book.

Why do we need a chapter that updates over time? For two reasons:

1. To add more technical content to the book before it is time to replace the current book edition with the next edition. This chapter will include additional technology content and possibly additional PDFs containing more content.
2. To communicate detail about the next version of the CCNA exam, to tell you about our publishing plans for that edition, and what that means to you.

To find the latest version of this chapter, follow the process below. Bookmark the link so that any time you refer to this chapter, begin by downloading a new copy. Use these steps:

**Step 1.** Browse to www.ciscopress.com/register.

**Step 2.** Enter the print book ISBN (even if you are using an eBook): 9780138229634.

**Step 3.** After registering the book, go to your account page and select the Registered Products tab.

**Step 4.** Click on the Access Bonus Content link to access the companion website. Select the **Exam Updates Chapter** link or scroll down to that section to check for updates.

Table 30-1 summarizes the information that this version of the chapter includes. Use the table as a quick reference for the detail to expect in the rest of the chapter.

**Table 30-1** Status for Available New Technical Content for This Edition

| Chapter Version | 1 |
|---|---|
| Most recent CCNA 200-301 blueprint version when this chapter was most recently released | 1.1 |
| Is there technology content in the latter part of this chapter? | No |
| Is there technology content in other downloadable files? | No |
| Links to other file downloads: | N/A |
| Is there information about the specifics of the new exam? | No |

Any additional file downloads, or further instructions for file downloads, will be posted at this book's companion website. See the heading "How to Access the Companion Website" in the Introduction to this book for details on finding this book's companion website.

The following two sections give more detail about the primary purposes of the chapter. Any technical content or exam update detail follows later in the chapter.

## Additional Technical Content

On rare occasions, the need exists for additional book content outside the normal process of creating a new book edition. Several factors might make an author want to add content. However, once an author decides to add content, the question becomes how to deliver the content and reach as many existing readers as possible. This chapter and its electronic update process give us the means to get the content to you.

Looking back at prior *CCNA Official Cert Guide* editions, the need to add content has happened only a few times over more than ten years of using chapters like this one. However, when needed, it has worked well. Using other means, like book registration, social media, and so on, reaches fewer of you, the readers.

You do not need to check frequently for a new PDF. You may want to check more regularly about exam updates, as discussed in the next section. But to check for new technical content, I suggest downloading the PDF again on this cadence:

- After your first reading of this book
- After your first reading of the Volume 2 book

**NOTE** An equivalent chapter exists in the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition. I would also suggest downloading that chapter when you download this chapter.

Of course, if this chapter adds new technical content, treat it like any of the other chapters in the book, and study the content!

## About Possible Exam Updates

Cisco introduced CCNA and CCNP in 1998. For the first 25 years of those certification tracks, Cisco updated the exams on average every 3–4 years. However, Cisco did not pre-announce the exam changes, so exam changes felt very sudden. Usually, a new exam would be announced, with new exam topics, giving you 3–6 months before your only option was to take the new exam. As a result, you could be studying with no idea about Cisco's plans, and the next day, you had a 3–6-month timeline to either pass the old exam or pivot to prepare for the new exam.

Thankfully, Cisco changed their exam release approach in 2023. Called the Cisco Certification Roadmap (https://cisco.com/go/certroadmap), the new plan includes these features:

1. Cisco considers changes to all exam tracks (CCNA, CCNP Enterprise, CCNP Security, and so on) annually.

2. Cisco uses a predefined annual schedule for each track, so you know the timing of possible changes to the exam you are studying for, even before any announcements.
3. The schedule moves in a quarterly sequence:
   a. Privately review the exam to consider what to change.
   b. Publicly announce if an exam is changing, and if so, announce details like exam topics and release date.
   c. Release the new exam.
4. Exam changes might not occur each year. If changes occur, Cisco characterizes them as minor (less than 20 percent change) or major (more than 20 percent change).

The specific dates for a given certification track can be confusing because Cisco organizes the work by fiscal year quarters. As an example, Figure 30-1 shows the 2024 fiscal year. Their fiscal year begins in August, so, for example, the first quarter (Q1) of fiscal year (FY) 2024 began in August 2023.

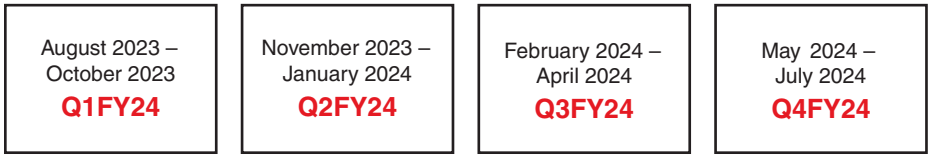



**Figure 30-1** *Cisco Fiscal Year and Months Example (FY2024)*

Focus more on the sequence of the quarters to understand the plan. Figure 30-2 shows an example sequence in which Cisco updates the CCNA 200-301 exam, assuming a minor release (less than 20 percent change).

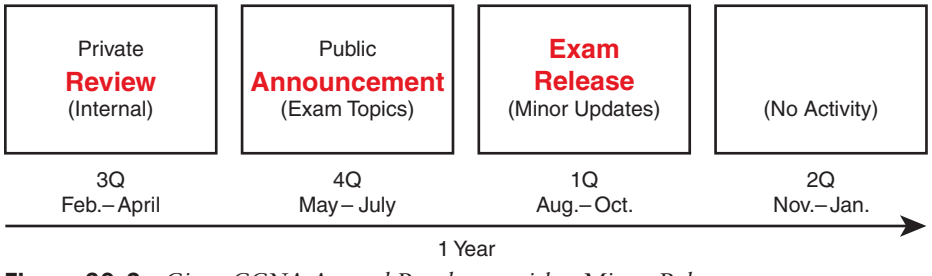



**Figure 30-2** *Cisco CCNA Annual Roadmap with a Minor Release*

Over time, Cisco might make no changes in some years and minor changes in others. For example, Cisco announced CCNA 200-301 version 1.1 in the FY24-FY25 cycle (February 2024–January 2025). Figure 30-3 shows what could happen in the next four years. It first shows a year with no changes, then a year with minor changes, another year with no changes, and a year with major changes.

Figure 30-3 shows an example and does not reveal any secret knowledge about Cisco's plans; however, it shows a volume of change that matches the rate of change to the exam over its long history.

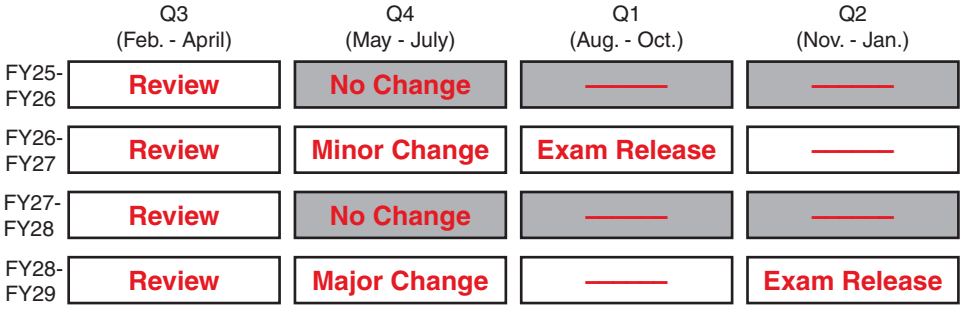



**Figure 30-3** *An Example of What Could Happen with CCNA over Four Years*

## Impact on You and Your Study Plan

Cisco's new policy helps you plan, but it also means that the CCNA exam might change before you pass the current exam. That impacts you, affecting how we deliver this book to you. This chapter gives us a way to communicate in detail about those changes as they occur. But you should watch other spaces as well.

Your study plan ongoing should follow this general process:

**Step 1.** Continue to use the *CCNA 200-301 Official Cert Guides, Volumes 1 and 2*, Second Edition, as is.

**Step 2.** Monitor for updates to this Exam Updates chapter, plus the equivalent chapter in Volume 2, to find additional content for topics Cisco may add to the exam.

**Step 3.** To be aware of changes to the CCNA exam, monitor the information sources listed below.

For those other information sources to watch, bookmark and check these sites for news. In particular:

**Cisco:** Check their Certification Roadmap page: https://cisco.com/go/certroadmap. Make sure to sign up for automatic notifications from Cisco on that page.

**Publisher:** Check this page about new certification products, offers, discounts, and free downloads related to the more frequent exam updates: www.ciscopress.com/newcert.

**Cisco Learning Network:** Subscribe to the CCNA Community at learningnetwork.cisco.com, where I expect ongoing discussions about exam changes over time. If you have questions, search for "roadmap" in the CCNA community, and if you do not find an answer, ask a new one!

**Author:** Look for blog posts labeled as News at www.certskills.com. For every new edition, I post about the new exam, new topics, and how to manage the transition. I will continue to do so now with more detail about the new annual cycle.

As changes arise, I will update this chapter with more detail about exam and book content. Given Cisco's certification roadmap, that means somewhere in each year's fiscal fourth

quarter (May–July). At that point, I will publish an updated version of this chapter, listing our content plans. That detail will likely include the following:

- Content removed, so if you plan to take the new exam version, you can ignore those when studying
- New content planned per new exam topics, so you know what's coming

While I do not think Cisco will change CCNA every year—I predict Cisco will change it less than half the years—everyone needs to be aware of the possibility and timing of exam changes and new exam releases. I will use this chapter to communicate the exam and book content details. Look to the other sites for brief news about plans, but look here for the detail.

The remainder of the chapter shows the new content that may change over time.

## News about the Next CCNA Exam Release

This statement was last updated in March 2024, before the publication of the *CCNA 200-301 Official Certification Guide, Volume 1*, Second Edition.

This version of this chapter has no news to share about the next CCNA exam release.

At the most recent version this chapter, the CCNA 200-301 exam version number was Version 1.1.

## Updated Technical Content

The current version of this chapter has no additional technical content.

*This page intentionally left blank*

*This page intentionally left blank*

# Part IX

## Appendixes

**Appendix A:** Numeric Reference Tables

**Appendix B:** Exam Topics Cross-Reference

**Appendix C:** Answers to the “Do I Know This Already?” Quizzes

**Glossary**

*This page intentionally left blank*

# APPENDIX A

# Numeric Reference Tables

This appendix provides several useful reference tables that list numbers used throughout this book. Specifically:

Table A-1: A decimal-binary cross reference, useful when converting from decimal to binary and vice versa.

**Table A-1** Decimal-Binary Cross Reference, Decimal Values 0–255

| Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value |
|---|---|---|---|---|---|---|---|
| 0 | 00000000 | 32 | 00100000 | 64 | 01000000 | 96 | 01100000 |
| 1 | 00000001 | 33 | 00100001 | 65 | 01000001 | 97 | 01100001 |
| 2 | 00000010 | 34 | 00100010 | 66 | 01000010 | 98 | 01100010 |
| 3 | 00000011 | 35 | 00100011 | 67 | 01000011 | 99 | 01100011 |
| 4 | 00000100 | 36 | 00100100 | 68 | 01000100 | 100 | 01100100 |
| 5 | 00000101 | 37 | 00100101 | 69 | 01000101 | 101 | 01100101 |
| 6 | 00000110 | 38 | 00100110 | 70 | 01000110 | 102 | 01100110 |
| 7 | 00000111 | 39 | 00100111 | 71 | 01000111 | 103 | 01100111 |
| 8 | 00001000 | 40 | 00101000 | 72 | 01001000 | 104 | 01101000 |
| 9 | 00001001 | 41 | 00101001 | 73 | 01001001 | 105 | 01101001 |
| 10 | 00001010 | 42 | 00101010 | 74 | 01001010 | 106 | 01101010 |
| 11 | 00001011 | 43 | 00101011 | 75 | 01001011 | 107 | 01101011 |
| 12 | 00001100 | 44 | 00101100 | 76 | 01001100 | 108 | 01101100 |
| 13 | 00001101 | 45 | 00101101 | 77 | 01001101 | 109 | 01101101 |
| 14 | 00001110 | 46 | 00101110 | 78 | 01001110 | 110 | 01101110 |
| 15 | 00001111 | 47 | 00101111 | 79 | 01001111 | 111 | 01101111 |
| 16 | 00010000 | 48 | 00110000 | 80 | 01010000 | 112 | 01110000 |
| 17 | 00010001 | 49 | 00110001 | 81 | 01010001 | 113 | 01110001 |
| 18 | 00010010 | 50 | 00110010 | 82 | 01010010 | 114 | 01110010 |
| 19 | 00010011 | 51 | 00110011 | 83 | 01010011 | 115 | 01110011 |
| 20 | 00010100 | 52 | 00110100 | 84 | 01010100 | 116 | 01110100 |
| 21 | 00010101 | 53 | 00110101 | 85 | 01010101 | 117 | 01110101 |
| 22 | 00010110 | 54 | 00110110 | 86 | 01010110 | 118 | 01110110 |
| 23 | 00010111 | 55 | 00110111 | 87 | 01010111 | 119 | 01110111 |
| 24 | 00011000 | 56 | 00111000 | 88 | 01011000 | 120 | 01111000 |
| 25 | 00011001 | 57 | 00111001 | 89 | 01011001 | 121 | 01111001 |
| 26 | 00011010 | 58 | 00111010 | 90 | 01011010 | 122 | 01111010 |
| 27 | 00011011 | 59 | 00111011 | 91 | 01011011 | 123 | 01111011 |
| 28 | 00011100 | 60 | 00111100 | 92 | 01011100 | 124 | 01111100 |
| 29 | 00011101 | 61 | 00111101 | 93 | 01011101 | 125 | 01111101 |
| 30 | 00011110 | 62 | 00111110 | 94 | 01011110 | 126 | 01111110 |
| 31 | 00011111 | 63 | 00111111 | 95 | 01011111 | 127 | 01111111 |

| Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value |
|---|---|---|---|---|---|---|---|
| 128 | 10000000 | 160 | 10100000 | 192 | 11000000 | 224 | 11100000 |
| 129 | 10000001 | 161 | 10100001 | 193 | 11000001 | 225 | 11100001 |
| 130 | 10000010 | 162 | 10100010 | 194 | 11000010 | 226 | 11100010 |
| 131 | 10000011 | 163 | 10100011 | 195 | 11000011 | 227 | 11100011 |
| 132 | 10000100 | 164 | 10100100 | 196 | 11000100 | 228 | 11100100 |
| 133 | 10000101 | 165 | 10100101 | 197 | 11000101 | 229 | 11100101 |
| 134 | 10000110 | 166 | 10100110 | 198 | 11000110 | 230 | 11100110 |
| 135 | 10000111 | 167 | 10100111 | 199 | 11000111 | 231 | 11100111 |
| 136 | 10001000 | 168 | 10101000 | 200 | 11001000 | 232 | 11101000 |
| 137 | 10001001 | 169 | 10101001 | 201 | 11001001 | 233 | 11101001 |
| 138 | 10001010 | 170 | 10101010 | 202 | 11001010 | 234 | 11101010 |
| 139 | 10001011 | 171 | 10101011 | 203 | 11001011 | 235 | 11101011 |
| 140 | 10001100 | 172 | 10101100 | 204 | 11001100 | 236 | 11101100 |
| 141 | 10001101 | 173 | 10101101 | 205 | 11001101 | 237 | 11101101 |
| 142 | 10001110 | 174 | 10101110 | 206 | 11001110 | 238 | 11101110 |
| 143 | 10001111 | 175 | 10101111 | 207 | 11001111 | 239 | 11101111 |
| 144 | 10010000 | 176 | 10110000 | 208 | 11010000 | 240 | 11110000 |
| 145 | 10010001 | 177 | 10110001 | 209 | 11010001 | 241 | 11110001 |
| 146 | 10010010 | 178 | 10110010 | 210 | 11010010 | 242 | 11110010 |
| 147 | 10010011 | 179 | 10110011 | 211 | 11010011 | 243 | 11110011 |
| 148 | 10010100 | 180 | 10110100 | 212 | 11010100 | 244 | 11110100 |
| 149 | 10010101 | 181 | 10110101 | 213 | 11010101 | 245 | 11110101 |
| 150 | 10010110 | 182 | 10110110 | 214 | 11010110 | 246 | 11110110 |
| 151 | 10010111 | 183 | 10110111 | 215 | 11010111 | 247 | 11110111 |
| 152 | 10011000 | 184 | 10111000 | 216 | 11011000 | 248 | 11111000 |
| 153 | 10011001 | 185 | 10111001 | 217 | 11011001 | 249 | 11111001 |
| 154 | 10011010 | 186 | 10111010 | 218 | 11011010 | 250 | 11111010 |
| 155 | 10011011 | 187 | 10111011 | 219 | 11011011 | 251 | 11111011 |
| 156 | 10011100 | 188 | 10111100 | 220 | 11011100 | 252 | 11111100 |
| 157 | 10011101 | 189 | 10111101 | 221 | 11011101 | 253 | 11111101 |
| 158 | 10011110 | 190 | 10111110 | 222 | 11011110 | 254 | 11111110 |
| 159 | 10011111 | 191 | 10111111 | 223 | 11011111 | 255 | 11111111 |

Table A-2: A hexadecimal-binary cross reference, useful when converting from hex to binary and vice versa.

**Table A-2** Hex-Binary Cross Reference

| Hex | 4-Bit Binary |
|---|---|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0010 |
| 3 | 0011 |
| 4 | 0100 |
| 5 | 0101 |
| 6 | 0110 |
| 7 | 0111 |
| 8 | 1000 |
| 9 | 1001 |
| A | 1010 |
| B | 1011 |
| C | 1100 |
| D | 1101 |
| E | 1110 |
| F | 1111 |

Table A-3: Powers of 2, from $2^1$ through $2^{32}$.

**Table A-3** Powers of 2

| X | $2^X$ | X | $2^X$ |
|---|---|---|---|
| 1 | 2 | 17 | 131,072 |
| 2 | 4 | 18 | 262,144 |
| 3 | 8 | 19 | 524,288 |
| 4 | 16 | 20 | 1,048,576 |
| 5 | 32 | 21 | 2,097,152 |
| 6 | 64 | 22 | 4,194,304 |
| 7 | 128 | 23 | 8,388,608 |
| 8 | 256 | 24 | 16,777,216 |
| 9 | 512 | 25 | 33,554,432 |
| 10 | 1024 | 26 | 67,108,864 |
| 11 | 2048 | 27 | 134,217,728 |
| 12 | 4096 | 28 | 268,435,456 |
| 13 | 8192 | 29 | 536,870,912 |
| 14 | 16,384 | 30 | 1,073,741,824 |
| 15 | 32,768 | 31 | 2,147,483,648 |
| 16 | 65,536 | 32 | 4,294,967,296 |

Table A-4: Table of all 33 possible subnet masks, in all three formats.

**Table A-4** All Subnet Masks

| Decimal | Prefix | Binary |
|---|---|---|
| 0.0.0.0 | /0 | 00000000 00000000 00000000 00000000 |
| 128.0.0.0 | /1 | 10000000 00000000 00000000 00000000 |
| 192.0.0.0 | /2 | 11000000 00000000 00000000 00000000 |
| 224.0.0.0 | /3 | 11100000 00000000 00000000 00000000 |
| 240.0.0.0 | /4 | 11110000 00000000 00000000 00000000 |
| 248.0.0.0 | /5 | 11111000 00000000 00000000 00000000 |
| 252.0.0.0 | /6 | 11111100 00000000 00000000 00000000 |
| 254.0.0.0 | /7 | 11111110 00000000 00000000 00000000 |
| 255.0.0.0 | /8 | 11111111 00000000 00000000 00000000 |
| 255.128.0.0 | /9 | 11111111 10000000 00000000 00000000 |
| 255.192.0.0 | /10 | 11111111 11000000 00000000 00000000 |
| 255.224.0.0 | /11 | 11111111 11100000 00000000 00000000 |
| 255.240.0.0 | /12 | 11111111 11110000 00000000 00000000 |
| 255.248.0.0 | /13 | 11111111 11111000 00000000 00000000 |
| 255.252.0.0 | /14 | 11111111 11111100 00000000 00000000 |
| 255.254.0.0 | /15 | 11111111 11111110 00000000 00000000 |
| 255.255.0.0 | /16 | 11111111 11111111 00000000 00000000 |
| 255.255.128.0 | /17 | 11111111 11111111 10000000 00000000 |
| 255.255.192.0 | /18 | 11111111 11111111 11000000 00000000 |
| 255.255.224.0 | /19 | 11111111 11111111 11100000 00000000 |
| 255.255.240.0 | /20 | 11111111 11111111 11110000 00000000 |
| 255.255.248.0 | /21 | 11111111 11111111 11111000 00000000 |
| 255.255.252.0 | /22 | 11111111 11111111 11111100 00000000 |
| 255.255.254.0 | /23 | 11111111 11111111 11111110 00000000 |
| 255.255.255.0 | /24 | 11111111 11111111 11111111 00000000 |
| 255.255.255.128 | /25 | 11111111 11111111 11111111 10000000 |
| 255.255.255.192 | /26 | 11111111 11111111 11111111 11000000 |
| 255.255.255.224 | /27 | 11111111 11111111 11111111 11100000 |
| 255.255.255.240 | /28 | 11111111 11111111 11111111 11110000 |
| 255.255.255.248 | /29 | 11111111 11111111 11111111 11111000 |
| 255.255.255.252 | /30 | 11111111 11111111 11111111 11111100 |
| 255.255.255.254 | /31 | 11111111 11111111 11111111 11111110 |
| 255.255.255.255 | /32 | 11111111 11111111 11111111 11111111 |

# APPENDIX B

# Exam Topics Cross-Reference

This appendix lists the exam topics associated with the CCNA 200-301 exam blueprint Version 1.1. Cisco lists the exam topics on its website. Even though changes to the exam topics are rare, you should always review those exam topics for any updates; check www.cisco.com/go/certifications and navigate to the correct exam.

Cisco organizes each list of exam topics by domains, which are major topic areas. Cisco states the percentage of the exam that should come from each domain, so you get some idea of the areas of importance. Traditionally, the score report you receive after taking the exam shows your percentage score in each domain.

This appendix includes two separate types of indices for exam topics:

- **CCNA 200-301 Exam Topic Order:** This section lists the CCNA 200-301 V1.1 exam topics in the same order Cisco lists them on its website, with a list of associated book chapters. This first list shows a cross-reference from each exam topic to the chapters that include at least some material about each topic.
- **Book Chapter Order Versus CCNA 200-301 Exam Topics:** This lists the same CCNA 200-301 V1.1 exam topics but indexed by chapter instead of exam topic. This section lists the chapters in this book, along with the exam topics that the chapter includes. This section basically relists the kind of information found on the first page of each chapter, just in condensed form in one place.

## CCNA 200-301 Exam Topic Order

The CCNA 200-301 exam includes six major topic areas (domains), each with a percentage listed. Table B-1 lists the domains and their percentages.

**Table B-1** CCNA 200-301 V1.1 Exam Topic Domains

| Domain | Percentage |
|---|---|
| Domain 1: Network Fundamentals | 20% |
| Domain 2: Network Access | 20% |
| Domain 3: IP Connectivity | 25% |
| Domain 4: IP Services | 10% |
| Domain 5: Security Fundamentals | 15% |
| Domain 6: Automation and Programmability | 10% |

Tables B-2 through B-7 list the exam topics within each of the six domains. Note that the *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, covers some of the exam topics, while this book covers the rest.

**Table B-2** CCNA 200-301 V1.1 Domain 1 Exam Topics (Network Fundamentals)

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| **1.1 Explain the role and function of network components** | 2, 3, 5, 7 | 1, 10, 18, 21, 22 |
| *1.1.a Routers* | 3, 16, 18 | |
| *1.1.b Layer 2 and Layer 3 Switches* | 2, 5, 7, 18 | |
| *1.1.c Next-generation firewalls and IPS* | | 10 |
| *1.1.d Access points* | | 1 |
| *1.1.e Controllers* | | 4, 22 |
| *1.1.f Endpoints* | | 21 |
| *1.1.g Servers* | | 21 |
| *1.1.h PoE* | | 18 |
| **1.2 Describe characteristics of network topology architectures** | 2, 3 | 18–21 |
| *1.2.a Two-tier* | | 18 |
| *1.2.b Three-tier* | | 18 |
| *1.2.c Spine-leaf* | | 21 |
| *1.2.d WAN* | 3 | 19 |
| *1.2.e Small office/home office (SOHO)* | 2, 16 | 18 |
| *1.2.f On-premises and cloud* | | 20 |
| **1.3 Compare physical interface and cabling types** | 1, 2, 7 | 18 |
| *1.3.a Single-mode fiber, multimode fiber, copper* | 1, 2 | 18 |
| *1.3.b Connections (Ethernet shared media and point-to-point)* | 1, 2, 7 | 18 |
| **1.4 Identify interface and cable issues (collisions, errors, mismatch duplex, and/or speed)** | 7 | |
| **1.5 Compare TCP to UDP** | | 5 |
| **1.6 Configure and verify IPv4 addressing and subnetting** | 6, 11–16, 18 | |
| **1.7 Describe private IPv4 addressing** | 11, 12, 17 | 14 |
| **1.8 Configure and verify IPv6 addressing and prefix** | 25–28 | |
| **1.9 Describe IPv6 address types** | 25–28 | |
| *1.9.a Unicast (global, unique local, and link local)* | 26–28 | |
| *1.9.b Anycast* | 26, 27 | |
| *1.9.c Multicast* | 27 | |
| *1.9.d Modified EUI 64* | 27, 28 | |
| **1.10 Verify IP parameters for Client OS (Windows, Mac OS, Linux)** | 19 | |
| **1.11 Describe wireless principles** | | 1, 3 |
| *1.11.a Nonoverlapping Wi-Fi channels* | | 1 |
| *1.11.b SSID* | | 1 |

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| *1.11.c RF* | | 1 |
| *1.11.d Encryption* | | 3 |
| 1.12 Explain virtualization fundamentals (server virtualization, containers, and VRFs) | | 20 |
| 1.13 Describe switching concepts | 5, 8 | |
| *1.13.a MAC learning and aging* | 5, 8 | |
| *1.13.b Frame switching* | 5, 8 | |
| *1.13.c Frame flooding* | 5, 8 | |
| *1.13.d MAC address table* | 5, 8 | |

**Table B-3** CCNA 200-301 V1.1 Domain 2 Exam Topics (Network Access)

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 2.1 Configure and verify VLANs (normal range) spanning multiple switches | 8, 18 | |
| *2.1.a Access ports (data and voice)* | 8 | |
| *2.1.b Default VLAN* | 8 | |
| *2.1.c InterVLAN Connectivity* | 8, 18 | |
| 2.2 Configure and verify interswitch connectivity | 8 | |
| *2.2.a Trunk ports* | 8 | |
| *2.2.b 802.1Q* | 8 | |
| *2.2.c Native VLAN* | 8 | |
| 2.3 Configure and verify Layer 2 discovery protocols (Cisco Discovery Protocol and LLDP) | | 13 |
| 2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP) | 8–10, 18 | |
| 2.5 Interpret basic operations of Rapid PVST+ Spanning Tree Protocol | 5, 9, 10 | |
| *2.5.a Root port, root bridge (primary/secondary), and other port names* | 9, 10 | |
| *2.5.b Port states and roles* | 9, 10 | |
| *2.5.c PortFast* | 9, 10 | |
| *2.5.d Root Guard, loop guard, BPDU filter, BPDU guard* | 9, 10 | |
| 2.6 Compare Cisco Wireless Architectures and AP modes | | 2 |
| 2.7 Describe physical infrastructure connections of WLAN components (AP, WLC, access/trunk ports, and LAG) | | 4 |

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, and TACACS+/RADIUS, and cloud managed) | 4, 6, 20 | 4 |
| 2.9 Interpret the wireless LAN GUI configuration for client connectivity, such as WLAN creation, security settings, QoS profiles, and advanced settings | | 4 |

**Table B-4** CCNA 200-301 Domain 3 Exam Topics (IP Connectivity)

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 3.1 Interpret the components of routing table | 17, 29 | |
| *3.1.a Routing protocol code* | 17, 29 | |
| *3.1.b Prefix* | 17, 29 | |
| *3.1.c Network mask* | 17, 29 | |
| *3.1.d Next hop* | 17, 29 | |
| *3.1.e Administrative distance* | 17, 24, 29 | |
| *3.1.f Metric* | 17 | |
| *3.1.g Gateway of last resort* | 17 | |
| 3.2 Determine how a router makes a forwarding decision by default | 17, 21–24 | |
| *3.2.a Longest prefix match* | 17, 24 | |
| *3.2.b Administrative distance* | 17, 21–24 | |
| *3.2.c Routing protocol metric* | 21, 21–24 | |
| 3.3 Configure and verify IPv4 and IPv6 static routing | 17, 20, 29 | |
| *3.3.a Default route* | 17, 20, 29 | |
| *3.3.b Network route* | 17, 20, 29 | |
| *3.3.c Host route* | 17, 20, 29 | |
| *3.3.d Floating static* | 17, 20, 29 | |
| 3.4 Configure and verify single area OSPFv2 | 21–24 | |
| *3.4.a Neighbor adjacencies* | 21–24 | |
| *3.4.b Point-to-point* | 21–24 | |
| *3.4.c Broadcast (DR/BDR selection)* | 21–24 | |
| *3.4.d Router ID* | 21–24 | |
| 3.5 Describe the purpose, functions, and concepts of first hop redundancy protocols | | 16 |

**Table B-5** CCNA 200-301 V1.1 Domain 4 Exam Topics (IP Services)

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 4.1 Configure and verify inside source NAT using static and pools | | 14 |
| 4.2 Configure and verify NTP operating in a client and server mode | | 13 |
| 4.3 Explain the role of DHCP and DNS within the network | 19 | 5 |
| 4.4 Explain the function of SNMP in network operations | | 17 |
| 4.5 Describe the use of syslog features including facilities and severity levels | | 13 |
| 4.6 Configure and verify DHCP client and relay | 6, 19 | |
| 4.7 Explain the forwarding per-hop behavior (PHB) for QoS such as classification, marking, queuing, congestion, policing, and shaping | | 15 |
| 4.8 Configure network devices for remote access using SSH | 6 | 10 |
| 4.9 Describe the capabilities and functions of TFTP/FTP in the network | | 17 |

**Table B-6** CCNA 200-301 Domain 5 Exam Topics (Security Fundamentals)

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 5.1 Define key security concepts (threats, vulnerabilities, exploits, and mitigation techniques) | | 9 |
| 5.2 Describe security program elements (user awareness, training, and physical access control) | | 9 |
| 5.3 Configure device access control using local passwords | 6 | 10 |
| 5.4 Describe security password policies elements, such as management, complexity, and password alternatives (multifactor authentication, certificates, and biometrics) | | 9 |
| 5.5 Describe IPsec remote access and site-to-site VPNs | | 19 |
| 5.6 Configure and verify access control lists | | 6, 7, 8 |
| 5.7 Configure Layer 2 security features (DHCP snooping, dynamic ARP inspection, and port security) | | 11, 12 |
| 5.8 Compare authentication, authorization, and accounting concepts | | 9 |

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 5.9 Describe wireless security protocols (WPA, WPA2, and WPA3) | | 3 |
| 5.10 Configure and verify WLAN within the GUI using WPA2 PSK | | 4 |

**Table B-7** CCNA 200-301 V1.1 Domain 6 Exam Topics (Programmability and Automation)

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 6.1 Explain how automation impacts network management | | 21 |
| 6.2 Compare traditional networks with controller-based networking | | |
| 6.3 Describe controller-based, software-defined architecture (overlay, underlay, and fabric) | | 21 |
| *6.3.a Separation of control plane and data plane* | | 21, 22 |
| *6.3.b Northbound and Southbound APIs* | | 21, 22 |
| 6.4 Explain AI (generative and predictive) and machine learning in network operations | | 22 |
| 6.5 Describe characteristics of REST-based APIs (authentication types, CRUD, HTTP verbs, and data encoding) | | 23 |
| 6.6 Recognize the capabilities of configuration management mechanisms such as Ansible and Terraform | | 24 |
| 6.7 Recognize components of JSON-encoded data | | 23 |

## Book Chapter Order Versus CCNA 200-301 Exam Topics

Cisco organizes its exam topics based on the outcome of your learning experience, which is typically not a reasonable order for building the content of a book or course. This section lists the book chapters in sequence, with the exam topics covered in each chapter.

**Table B-8** CCNA 200-301 V1.1: Chapter-to-Exam Topic Mapping

| Book Chapter | Exam Topics Covered |
|---|---|
| **Part I: Introduction to Networking** | |
| Chapter 1: Introduction to TCP/IP Networking | **1.0 Network Fundamentals**<br>1.3 Compare physical interface and cabling types<br>*1.3.a Single-mode fiber, multimode fiber, copper*<br>*1.3.b Connections (Ethernet shared media and point-to-point)* |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 2: Fundamentals of Ethernet LANs | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.b Layer 2 and Layer 3 switches*<br>1.2 Describe characteristics of network topology architectures<br>*1.2.e Small office/home office (SOHO)*<br>1.3 Compare physical interface and cabling types<br>*1.3.a Single-mode fiber, multimode fiber, copper*<br>*1.3.b Connections (Ethernet shared media and point-to-point)* |
| Chapter 3: Fundamentals of WANs and IP Routing | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.a Routers*<br>1.2 Describe characteristics of network topology architectures<br>*1.2.d WAN* |
| **Part II: Implementing Ethernet LANs** | |
| Chapter 4: Using the Command-Line Interface | **2.0 Network Access**<br>2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPs, console, TACACS+/RADIUS, and cloud managed) |
| Chapter 5: Analyzing Ethernet LAN Switching | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.b Layer 2 and Layer 3 switches*<br>1.13 Describe switching concepts<br>*1.13.a MAC learning and aging*<br>*1.13.b Frame switching*<br>*1.13.c Frame flooding*<br>*1.13.d MAC address table*<br>**2.0 Network Access**<br>2.5 Interpret basic operations of Rapid PVST+ Spanning Tree Protocol |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 6: Configuring Basic Switch Management | **1.0 Network Fundamentals**<br>1.6 Configure and calculate IPv4 addressing and subnetting<br>**2.0 Network Access**<br>2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPs, console, TACACS+/RADIUS, and cloud managed)<br>**4.0 IP Services**<br>4.6 Configure and verify DHCP client and relay<br>4.8 Configure network devices for remote access using SSH<br>**5.0 Security Fundamentals**<br>5.3 Configure device access control using local passwords |
| Chapter 7: Configuring and Verifying Switch Interfaces | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.b Layer 2 and Layer 3 Switches*<br>1.3 Compare physical interface and cabling types<br>*1.3.b Connections (Ethernet shared media and point-to-point)*<br>1.4 Identify interface and cable issues (collisions, errors, mismatch duplex, and/or speed) |
| **Part III: Implementing VLANs and STP** | |
| Chapter 8: Implementing Ethernet Virtual LANs | **1.0 Network Fundamentals**<br>1.13 Describe switching concepts<br>*1.13.a MAC learning and aging*<br>*1.13.b Frame switching*<br>*1.13.c Frame flooding*<br>*1.13.d MAC address table*<br>**2.0 Network Access**<br>2.1 Configure and verify VLANs (normal range) spanning multiple switches<br>*2.1.a Access ports (data and voice)*<br>*2.1.b Default VLAN*<br>*2.1.c InterVLAN Connectivity* |

| Book Chapter | Exam Topics Covered |
|---|---|
| | 2.2 Configure and verify interswitch connectivity<br>*2.2.a Trunk ports*<br>*2.2.b 802.1Q*<br>*2.2.c Native VLAN* |
| Chapter 9: Spanning Tree Protocol Concepts | **2.0 Network Access**<br>2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP)<br>2.5 Interpret basic operations of Rapid PVST+ Spanning Tree Protocol<br>*2.5.a Root port, root bridge (primary/secondary), and other port names*<br>*2.5.b Port states and roles*<br>*2.5.c PortFast*<br>*2.5.d Root Guard, loop guard, BPDU filter, BPDU guard* |
| Chapter 10: RSTP and EtherChannel Configuration | **2.0 Network Access**<br>2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP)<br>2.5 Interpret basic operations of Rapid PVST+ Spanning Tree Protocol<br>*2.5.a Root port, root bridge (primary/secondary), and other port names*<br>*2.5.b Port states and roles*<br>*2.5.c PortFast*<br>*2.5.d Root Guard, loop guard, BPDU filter, BPDU guard* |
| **Part IV: IPv4 Addressing** | |
| Chapter 11: Perspectives on IPv4 Subnetting | **1.0 Network Fundamentals**<br>1.6 Configure and verify IPv4 addressing and subnetting<br>1.7 Describe the need for private IPv4 addressing |
| Chapter 12: Analyzing Classful IPv4 Networks | **1.0 Network Fundamentals**<br>1.6 Configure and verify IPv4 addressing and subnetting<br>1.7 Describe the need for private IPv4 addressing |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 13: Analyzing Subnet Masks | **1.0 Network Fundamentals**<br>1.6 Configure and verify IPv4 addressing and subnetting |
| Chapter 14: Analyzing Existing Subnets | **1.0 Network Fundamentals**<br>1.6 Configure and verify IPv4 addressing and subnetting |
| Chapter 15: Subnet Design | **1.0 Network Fundamentals**<br>1.6 Configure and verify IPv4 addressing and subnetting |
| **Part V: IPv4 Routing** | |
| Chapter 16: Operating Cisco Routers | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.a Routers*<br>1.2 Describe characteristics of network topology architectures<br>*1.2.e Small office/home office (SOHO)*<br>1.6 Configure and verify IPv4 addressing and subnetting |
| Chapter 17: Configuring IPv4 Addresses and Static Routes | **1.0 Network Fundamentals**<br>1.6 Configure and verify IPv4 addressing and subnetting<br>**3.0 IP Connectivity**<br>3.1 Interpret the components of routing table<br>*3.1.a Routing protocol code*<br>*3.1.b Prefix*<br>*3.1.c Network mask*<br>*3.1.d Next hop*<br>*3.1.e Administrative distance*<br>*3.1.f Metric*<br>*3.1.g Gateway of last resort*<br>3.2 Determine how a router makes a forwarding decision by default<br>*3.2.a Longest prefix match*<br>*3.2.b Administrative distance*<br>3.3 Configure and verify IPv4 and IPv6 static routing<br>*3.3.a Default route*<br>*3.3.b Network route*<br>*3.3.c Host route*<br>*3.3.d Floating static* |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 18: IP Routing in the LAN | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.a Routers*<br>*1.1.b Layer 2 and Layer 3 Switches*<br>1.6 Configure and verify IPv4 addressing and subnetting<br>**2.0 Network Access**<br>2.1 Configure and verify VLANs (normal range) spanning multiple switches<br>*2.1.c InterVLAN connectivity*<br>2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP) |
| Chapter 19: IP Addressing on Hosts | **1.0 Network Fundamentals**<br>1.10 Verify IP parameters for Client OS (Windows, Mac OS, Linux)<br>**4.0 IP Services**<br>4.3 Explain the role of DHCP and DNS within the network<br>4.6 Configure and verify DHCP client and relay |
| Chapter 20: Troubleshooting IPv4 Routing | **2.0 Network Access**<br>2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, TACACS+/RADIUS, and cloud managed)<br>**3.0 IP Connectivity**<br>3.3 Configure and verify IPv4 and IPv6 static routing<br>*3.3.a Default route*<br>*3.3.b Network route*<br>*3.3.c Host route*<br>*3.3.d Floating static* |

| Book Chapter | Exam Topics Covered |
|---|---|
| **Part VI: OSPF** | |
| Chapter 21: Understanding OSPF Concepts | **3.0 IP Connectivity**<br>3.2 Determine how a router makes a forwarding decision by default<br>*3.2.b Administrative distance*<br>*3.2.c Routing protocol metric*<br>3.4 Configure and verify single area OSPFv2<br>*3.4.a Neighbor adjacencies*<br>*3.4.b Point-to-point*<br>*3.4.c Broadcast (DR/BR selection)*<br>*3.4.d Router ID* |
| Chapter 22: Implementing Basic OSPF Features | **3.0 IP Connectivity**<br>3.2 Determine how a router makes a forwarding decision by default<br>*3.2.b Administrative distance*<br>*3.2.c Routing protocol metric*<br>3.4 Configure and verify single area OSPFv2<br>*3.4.a Neighbor adjacencies*<br>*3.4.c Broadcast (DR/BR selection)*<br>*3.4.d (Router ID)* |
| Chapter 23: Implementing Optional OSPF Features | **3.0 IP Connectivity**<br>3.2 Determine how a router makes a forwarding decision by default<br>*3.2.b Administrative distance*<br>*3.2.c Routing protocol metric*<br>3.4 Configure and verify single area OSPFv2<br>*3.4.a Neighbor adjacencies*<br>*3.4.b Point-to-point*<br>*3.4.c Broadcast (DR/BR selection)*<br>*3.4.d Router ID* |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 24: OSPF Neighbors and Route Selection | **3.0 IP Connectivity**<br>3.1 Interpret components of the routing table<br>*3.1.e Administrative distance*<br>3.2 Determine how a router makes a forwarding decision by default<br>*3.2.a Longest prefix match*<br>*3.2.b Administrative distance*<br>*3.2.c Routing protocol metric*<br>3.4 Configure and verify single area OSPFv2<br>*3.4.a Neighbor adjacencies*<br>*3.4.b Point-to-point*<br>*3.4.c Broadcast (DR/BR selection)*<br>*3.4.d Router ID* |
| **Part VII: IP Version 6** | |
| Chapter 25: Fundamentals of IP Version 6 | **1.0 Network Fundamentals**<br>1.8 Configure and verify IPv6 addressing and prefix |
| Chapter 26: IPv6 Addressing and Subnetting | **1.0 Network Fundamentals**<br>1.8 Configure and verify IPv6 addressing and prefix<br>1.9 Describe IPv6 address types |
| Chapter 27: Implementing IPv6 Addressing on Routers | **1.0 Network Fundamentals**<br>1.8 Configure and verify IPv6 addressing and prefix<br>1.9 Compare and contrast IPv6 address types<br>*1.9.a Unicast (global, unique local, and link local)*<br>*1.9.b Anycast*<br>*1.9.c Multicast*<br>*1.9.d Modified EUI 64* |
| Chapter 28: Implementing IPv6 Addressing on Hosts | **1.0 Network Fundamentals**<br>1.8 Configure and verify IPv6 addressing and prefix<br>1.9 Describe IPv6 address types<br>*1.9.a Unicast (global, unique local, and link local)*<br>*1.9.d Modified EUI 64* |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 29: Implementing IPv6 Routing | **3.0 IP Connectivity**<br>3.1 Interpret components of the routing table<br>*3.1.a Routing protocol code*<br>*3.1.b Prefix*<br>*3.1.c Network mask*<br>*3.1.d Next hop*<br>*3.1.e Administrative distance*<br>3.3 Configure and verify IPv4 and IPv6 static routing<br>*3.3.a Default route*<br>*3.3.b Network route*<br>*3.3.c Host route*<br>*3.3.d Floating static* |

# APPENDIX C

# Answers to the "Do I Know This Already?" Quizzes

## Chapter 1

1. D and F. Of the remaining answers, Ethernet defines both physical and data-link protocols, PPP is a data-link protocol, IP is a network layer protocol, and SMTP and HTTP are application layer protocols.
2. A and G. Of the remaining answers, IP is a network layer protocol, TCP and UDP are transport layer protocols, and SMTP and HTTP are application layer protocols.
3. B. Adjacent-layer interaction occurs on one computer, with two adjacent layers in the model. The higher layer requests services from the next lower layer, and the lower layer provides the services to the next higher layer.
4. B. Same-layer interaction occurs on multiple computers. The functions defined by that layer typically need to be accomplished by multiple computers—for example, the sender setting a sequence number for a segment and the receiver acknowledging receipt of that segment. A single layer defines that process, but the implementation of that layer on multiple devices is required to accomplish the function.
5. A. Encapsulation is defined as the process of adding a header in front of data supplied by a higher layer (and possibly adding a trailer as well).
6. D. By convention, the term *frame* refers to the part of a network message that includes the data-link header and trailer, with encapsulated data. The term *packet* omits the data-link header and trailer, leaving the network layer header with its encapsulated data. The term *segment* omits the network layer header, leaving the transport layer header and its encapsulated data.

## Chapter 2

1. A. The IEEE defines Ethernet LAN standards, with standard names that begin with 802.3, all of which happen to use cabling. The IEEE also defines wireless LAN standards, with standard names that begin with 802.11, which are separate standards from Ethernet.
2. C. The number before the word *BASE* defines the speed, in megabits per second (Mbps): 1000 Mbps equals 1 gigabit per second (1 Gbps). The *T* in the suffix implies twisted-pair or UTP cabling, so 1000BASE-T is the UTP-based Gigabit Ethernet standard name.
3. B. Crossover cables cross the wire at one node's transmit pin pair to the different pins used as the receive pins on the other device. For 10- and 100-Mbps Ethernet, the specific crossover cable wiring connects the pair at pins 1 and 2 on each end of the cable to pins 3 and 6 on the other end of the cable, respectively.

4. B, D, and E. Routers, wireless access point Ethernet ports, and PC NICs all send using pins 1 and 2, whereas hubs and LAN switches transmit on pins 3 and 6. Straight-through cables connect devices that use opposite pin pairs for sending, because the cable does not need to cross the pairs.
5. B and D. Multimode fiber works with LED-based transmitters rather than laser-based transmitters. Two answers mention the type of transmitters, making one of those answers correct and one incorrect.

   Two answers mention distance. The answer that mentions the longest distance possible is incorrect because single-mode cables, not multimode cables, provide the longest distances. The other (correct) answer mentions the tradeoff of multimode being used for distances just longer than UTP's 100-meter limit, while happening to use less expensive hardware than single mode.
6. B. NICs (and switch ports) use the carrier sense multiple access with collision detection (CSMA/CD) algorithm to implement half-duplex logic. CSMA/CD attempts to avoid collisions, but it also notices when collisions do occur, with rules about how the Ethernet nodes should stop sending, wait, and try again later.
7. C. The 4-byte Ethernet FCS field, found in the Ethernet trailer, allows the receiving node to see what the sending node computed with a math formula that is a key part of the error-detection process. Note that Ethernet defines the process of detecting errors (error detection), but not error recovery.
8. B, C, and E. The pre-assigned universal MAC address, given to each Ethernet port when manufactured, breaks the address into two 3-byte halves. The first half is called the organizationally unique identifier (OUI), which the IEEE assigns to the company that builds the product as a unique hex number to be used only by that company.
9. C and D. Ethernet supports unicast addresses, which identify a single Ethernet node, and group addresses, which can be used to send one frame to multiple Ethernet nodes. The two types of group addresses are the *broadcast address* and *multicast address*.

## Chapter 3

1. D. The correct answer lists one term for an Ethernet WAN link between two sites: E-line, short for Ethernet line. The other answers list common synonyms for a serial link.
2. B and D. The physical installation uses a model in which each router uses a physical Ethernet link to connect to some SP device in an SP facility called a point of presence (PoP). The Ethernet link does not span from each customer device to the other. From a data-link perspective, both routers use the same Ethernet standard header and trailer used on LANs; HDLC does not matter on these Ethernet WAN links.
3. A. PC1 will send an Ethernet frame to Router1, with PC1's MAC address as the source address and Router1's MAC address as the destination address. Router1 will remove the encapsulated IP packet from that Ethernet frame, discarding the frame header and trailer, not using it again. Router1 will forward the IP packet by first encapsulating it inside a PPP frame. Router1 will not encapsulate the original

Ethernet frame in the PPP frame but rather the IP packet. The PPP header uses different addresses than Ethernet, so the original frame's Ethernet addresses are not used.

4. C. Routers compare the packet's destination IP address to the router's IP routing table, making a match and using the forwarding instructions in the matched route to forward the IP packet.
5. C. IPv4 hosts generally use basic two-branch logic. To send an IP packet to another host on the same IP network or subnet that is on the same LAN, the sender sends the IP packet directly to that host. Otherwise, the sender sends the packet to its default router (also called the default gateway).
6. A and C. Routers perform all the actions listed in the answers. However, the routing protocol does the functions in the two correct answers. Independent of the routing protocol, a router learns routes for IP subnets and IP networks directly connected to its interfaces. Routers also forward (route) IP packets, but that process is called IP routing, or IP forwarding, and is an independent process compared to the work of a routing protocol.
7. C. Address Resolution Protocol (ARP) does allow PC1 to learn information, but the information is not stored on a server. The **ping** command does let the user at PC1 learn whether packets can flow in the network, but it again does not use a server. With the Domain Name System (DNS), PC1 acts as a DNS client, relying on a DNS server to respond with information about the IP addresses that match a given hostname.



## Chapter 4

1. A and B. The command in the question is an EXEC command that happens to require only user mode access. As such, you can use this command in both user mode and enable mode. Because it is an EXEC command, you cannot use the command (as shown in the question) in configuration mode. Note that you can put the word **do** in front of the EXEC command while in configuration mode (for example, **do show mac address-table**) to issue the command from inside any configuration mode.
2. B. The command referenced in the question, the **reload** command, is an EXEC command that happens to require privileged mode, also known as enable mode. This command is not available in user mode. Note that you can put the word **do** in front of the EXEC command while in configuration mode (for example, **do reload**) to issue the command from inside any configuration mode.
3. B. SSH provides a secure remote login option, encrypting all data flows, including password exchanges. Telnet sends all data (including passwords) as clear text.
4. A. Switches (and routers) keep the currently used configuration in RAM, using NVRAM to store the configuration file that is loaded when the switch (or router) next loads the IOS.
5. F. The startup-config file is in NVRAM, and the running-config file is in RAM.
6. B and C. The **exit** command moves the user one config mode backward, toward global configuration mode, or if already in global configuration mode, it moves the user back to enable mode. From console mode, it moves the user back to global configuration mode. The **end** command and the Ctrl+Z key sequence both move the user back to enable mode regardless of the current configuration submode.

# Chapter 5

1. A. A switch compares the destination MAC address to the MAC address table. If a matching entry is found, the switch forwards the frame out the appropriate interface. If no matching entry is found, the switch floods the frame.
2. B. A switch floods broadcast frames, multicast frames (if no multicast optimizations are enabled), and unknown unicast destination frames (frames whose destination MAC address is not in the MAC address table). FFFF.FFFF.FFFF is the Ethernet broadcast address, so the switch floods the frame, which means that the switch forwards copies of the frame out all other ports except the arrival port.
3. A. A switch floods broadcast frames, multicast frames (if no multicast optimizations are enabled), and unknown unicast destination frames (frames whose destination MAC address is not in the MAC address table). Of the available answers, the correct answer defines how a switch floods a frame.

   Of the incorrect answers, one incorrect answer describes how a switch forwards known unicast frames by finding the matching entry in the MAC address table. Another describes MAC learning, in which the switch learns the source MAC address of incoming frames. Yet another incorrect answer mentions comparing the destination IP address to the destination MAC address, which the switch does not do.
4. B. Switches need to learn the location of each MAC address used in the LAN relative to that local switch. When a switch receives a frame, the source MAC identifies the sender. The interface in which the frame arrives identifies the local switch interface closest to that node in the LAN topology.
5. C. The **show interfaces status** command lists one line of output per interface. Cisco Catalyst switches name the type of interface based on the fastest speed of the interface, so 10/100 interfaces would be Fast Ethernet. With a working connection, ports from FastEthernet 0/1 through 0/10 would be listed in a connected state, while the rest would be listed in a notconnected state.
6. D. For the correct answer, each entry lists the learned MAC address. By definition, dynamically learned MAC addresses are learned by looking at the source MAC address of received frames. (That fact rules out one of the incorrect answers as well.)

   The **show mac address-table dynamic** command lists the current list of MAC table entries, with three known entries at the point at which the command output was gathered. The counter in the last line of output lists the number of current entries, not the total number of learned MAC addresses since the last reboot. For instance, the switch could have learned other MAC addresses whose entries timed out from the MAC address table.

   Finally, the answer that claims that port Gi0/2 connects directly to a device with a particular MAC address may or may not be true. That port could connect to another switch, and another, and so on, with one of those switches connecting to the device that uses the listed MAC address.

## Chapter 6

1. B. If both commands are configured, IOS accepts only the password as configured in the **enable secret** command.

2. A. To answer this question, it might be best to first think of the complete configuration and then find any answers that match the configuration. The commands, in vty line configuration mode, would be **password** *password* and **login**. Only one answer lists a vty subcommand that is one of these two commands.

   Of note in the incorrect answers:

   One answer mentions console subcommands. The console does not define what happens when remote users log in; those details sit in the vty line configuration.

   One answer mentions the **login local** command; this command means that the switch should use the local list of configured usernames/passwords. The question stated that the engineer wanted to use passwords only, with no usernames.

   One answer mentions the **transport input ssh** command, which, by omitting the **telnet** keyword, disables Telnet. While that command can be useful, SSH does not work when using passwords only; SSH requires both a username and a password. So, by disabling Telnet (and allowing SSH only), the configuration would allow no one to remotely log in to the switch.

3. B and C. SSH requires the use of usernames in addition to a password. Using the **username** global command would be one way to define usernames (and matching passwords) to support SSH. The vty lines would also need to be configured to require the use of usernames, with the **login local** vty subcommand being one such option.

   The **transport input ssh** command could be part of a meaningful configuration, but it is not a global configuration command (as claimed in one wrong answer). Likewise, one answer refers to the **username** command as a command in vty config mode, which is also the wrong mode.

4. A, D, and F. To allow access through Telnet, the switch must have password security enabled, at a minimum using the **password** vty line configuration subcommand. In addition, the switch needs an IP address (configured under one VLAN interface) and a default gateway when the switch needs to communicate with hosts in a different subnet.

5. B and C. To allow SSH or Telnet access, a switch must have a correct IP configuration. That includes the configuration of a correct IP address and mask on a VLAN interface. That VLAN interface then must have a path out of the switch via ports assigned to that VLAN. In this case, with all ports assigned to VLAN 2, the switch must use interface VLAN 2 (using the **interface vlan 2** configuration command).

   To meet the requirement to support login from hosts outside the local subnet, the switch must configure a correct default gateway setting with the **ip default-gateway 172.16.2.254** global command in this case.

6. A. The **logging synchronous** line subcommand synchronizes the log message display with other command output so the log message does not interrupt a **show**

command's output. The **no ip domain-lookup** command is not a line subcommand. The other two incorrect answers are line subcommands but do not configure the function listed in the question.

## Chapter 7

1. A and C. Because both devices use IEEE autonegotiation, they declare their speed and duplex capabilities to the other via messages sent out-of-band using Fast Link Pulses (FLPs). Then both devices choose the fastest speed that both support. They also use the best duplex setting that both support, with full duplex being better than half.
2. E. Cisco switches support per-interface settings for speed (with the **speed** command) and duplex (with the **duplex** command) in interface configuration mode.
3. B and E. Because the PC disables autonegotiation, it does not send autonegotiation FLP messages. However, it does start sending Ethernet frames based on the physical layer standard configured on the PC. After not receiving any autonegotiation FLP messages, the switch port analyzes the incoming signal to determine the standard used by the PC to send Ethernet frames. That analysis identifies the speed used by the PC, so the switch chooses to also use that speed. The switch then chooses the duplex based on a table of defaults: full duplex for speeds of 1 Gbps and faster or half duplex for speeds slower than 1 Gbps.
4. C. The **shutdown** interface subcommand administratively disables the interface, while the engineer can log in remotely on the weekend and configure the **no shutdown** interface subcommand to re-enable the interface. Note that unplugging the cable would prevent the interface from being used but would not allow the engineer to enable the interface remotely during the weekend change window. The **disable** and **enable** commands shown in a few answers do not exist.
5. B and D. The **interface range** global configuration command identifies a set of interfaces and moves the user into interface configuration mode. At that point, the switch applies any interface subcommands to all the interfaces in that range. However, the switch does not keep the **interface range** command in the configuration. For instance, in this case, the switch would list the **description** command under all the interfaces in the range: interfaces GigabitEthernet1/0/10 through GigabitEthernet 1/0/20 (11 interfaces in total).
6. A, B, and D. The disabled state in the **show interfaces status** command is the same as an "administratively down and down" state shown in the **show interfaces** command. The interface must be in a connected state (per the **show interfaces status** command) before the switch can send frames out of the interface.
7. A and C. First, note that some Cisco switch ports disable autonegotiation when configured with both **speed** and **duplex** on a port, as described in this question. A problem occurs when the combination of configured speed and duplex leads the autonegotiating device on the other end of the link to choose a different duplex setting—an effect called a duplex mismatch.

   To summarize the scenario in this case, SW1 uses autonegotiation but receives no autonegotiation messages from SW2. As a result, SW1 then begins using alternate

logic (called parallel detection) that does not rely on autonegotiation messages. SW2 does not use autonegotiation but has a speed set, so SW2 begins sending Ethernet frames per that speed on the link. SW1 senses the speed of those signals and uses the same speed, so a speed mismatch will not exist. However, SW1 then has to choose the duplex based on a default table, choosing half duplex if the speed is 10 or 100 Mbps, and full duplex if the speed is 1 Gbps or faster.

In the case of SW2 using **speed 100** and **duplex full** settings, SW1 senses the 100-Mbps speed and defaults to use half duplex, resulting in a duplex mismatch. Similar logic applies to the **speed 10** and **duplex full** case.



8. D. For the two answers about a duplex mismatch, that condition does cause collisions, particularly late collisions. However, only the side using CSMA/CD logic (the half-duplex side) has any concept of collisions. So, if switch SW1 were using half duplex and SW2 using full duplex, SW1 would likely see late collisions and show the counter incrementing over time.

   If switch SW2 had shut down its interface, switch SW1's interface would be in a down/down state, and none of the counters would increment. Also, if both switch ports use different speed settings, the ports would be in a down/down state, and none of the interface counters would increment.

## Chapter 8

1. B. A VLAN is a set of devices in the same Layer 2 broadcast domain. A subnet often includes the exact same set of devices, but it is a Layer 3 concept. A collision domain refers to a set of Ethernet devices, but with different rules than VLAN rules for determining which devices are in the same collision domain.
2. D. Although a subnet and a VLAN are not equivalent concepts, the devices in one VLAN are typically in the same IP subnet and vice versa.
3. B. The 802.1Q trunking defines a 4-byte header, inserted after the original frame's destination and source MAC address fields. The insertion of this header does not change the original frame's source or destination address. The header itself holds a 12-bit VLAN ID field, which identifies the VLAN associated with the frame.
4. A and C. The **dynamic auto** setting means that the switch can negotiate trunking, but it can only respond to negotiation messages, and it cannot initiate the negotiation process. So, the other switch must be configured to trunk (**switchport mode trunk**) or to initiate the dynamic negotiation process (**switchport mode dynamic desirable**).
5. A and B. The configured VTP setting of VTP transparent mode means that the switch can configure VLANs, so the VLAN is configured. In addition, the VLAN configuration details, including the VLAN name, show up as part of the running-config file.
6. B and C. The **show interfaces switchport** command lists both the administrative and operational status of each port. When a switch considers a port to be trunking, this command lists an operational trunking state of "trunk." The **show interfaces trunk** command lists a set of interfaces—the interfaces that are currently operating as trunks. So, both of these commands identify interfaces that are operational trunks.

7. A and B. On switches that do not use VTP (by using VTP modes off or transparent), the switch lists all VLAN configuration in the configuration file (making one answer correct). Also, the **show vlan brief** command lists all defined VLANs, regardless of VTP mode and regardless of shutdown state. As a result, the two answers that mention commands are correct.

   The other two answers are incorrect because VLAN 30 has been shut down, which means the switch will not forward frames in that VLAN, regardless of whether they arrive on access or trunk ports.

8. B. The first list of VLAN IDs includes all VLANs (1–4094) except those overtly removed per the details in any **switchport trunk allowed vlan** interface subcommands on the trunk interface. If no such commands are configured, the first list in the output will include 1–4094. The two incorrect answers that mention VLAN 30 list conditions that change the second of two lists of VLANs in the command output, while STP's choice to block an interface would impact the third list.

## Chapter 9

1. A and B. Listening and learning are transitory port states, used only when moving from the blocking to the forwarding state. Discarding is not an STP port state.
2. C. The smallest numeric bridge ID wins the election. The bridge IDs in the answers break into a decimal priority on the left, a colon, and then the 12-digit hexadecimal MAC address of the switch. Of all the answers, two tie with the lowest priority (4097). Of those, the correct answer lists a lower MAC address.
3. C and D. Listening and learning are transitory port states used only when moving from the blocking to the forwarding state. Discarding is not an STP port state. Forwarding and blocking are stable states.
4. B. Nonroot switches forward Hellos received from the root; the root sends these Hellos based on the root's configured Hello timer.
5. B and D. RSTP uses port states forwarding, learning, and discarding. Forwarding and learning perform the same functions as the port states used by traditional STP.
6. A and D. With RSTP, an alternate port is an alternate to the root port when a switch's root port fails. A backup port takes over for a designated port if the designated port fails.
7. D. The PortFast feature allows STP/RSTP to move a port from blocking to forwarding without going through the interim listening and learning states. STP allows this exception when the link is known to have no switch on the other end of the link, removing the risk of a switching loop. Cisco created PortFast before the IEEE released RSTP, but RSTP included the equivalent feature as well, so it is a feature of both STP and RSTP.

   BPDU Guard is a common feature to use at the same time as PortFast because it watches for incoming bridge protocol data units (BPDUs), which should not happen on an access port, and prevents the loops from a rogue switch by disabling the port.
8. C. Root Guard on a switch interface monitors incoming STP BPDUs, processing them as normal with STP, with one exception: superior BPDUs. Superior BPDUs identify a

new root switch with a better (lower) Bridge ID. Root Guard reacts to receiving such a BPDU and disables the port.

For the other answers, BPDU Guard monitors for incoming BPDUs but disables the port for any received BPDU on the port. Neither PortFast nor Loop Guard monitor incoming BPDUs or react to them.

## Chapter 10

1. A. Of the four answers, only **pvst** and **rapid-pvst** are valid options on the command. Of those, the **rapid-pvst** option enables Rapid Per VLAN Spanning Tree (RPVST+), which uses RSTP. The **pvst** option enables Per VLAN Spanning Tree (PVST), which uses STP, not RSTP. The other two options, if attempted, would cause the command to be rejected because these option do not exist.
2. A and C. The system ID extension (or extended system ID) part of a bridge ID contains 12 bits and sits after the 4-bit priority field and before the 48-bit system ID. Switches use this field to store the VLAN ID when using STP or RSTP to build spanning trees per VLAN. So, of the two answers that mention the system ID extension, the one that lists the VLAN ID, in this case 5, is correct.

   The output also lists a priority of 32773. However, that output lists the decimal equivalent of the 16-bit priority value. In reality, this decimal value is the sum of the configured decimal priority plus the VLAN ID: 32768 + 5 = 32773. So in this case, the root's configured priority is 32,768.
3. A, B, and D. Cisco's Rapid Per VLAN Spanning Tree (RPVST+) creates one spanning tree instance per VLAN. To do so, it sends BPDUs per VLAN. Each switch identifies itself with a unique Bridge ID (BID) per VLAN, made unique per VLAN by adding the VLAN ID to the system ID extension 12-bit field of the BID. RVPST also adds a new Type-Length Value (TLV) to the BPDU itself, which includes a place to list the VLAN ID. Finally, when transmitting the BPDUs over VLAN trunks, the switch uses a trunking header that lists the VLAN ID (a practice sometimes called tunneling in 802.1Q). The receiving switch can check all three locations that list the VLAN ID to ensure that they all agree about what VLAN the BPDU is describing. Of the four answers, the three correct answers describe the three actual locations in which RPVST+ lists the VLAN ID.
4. B and C. BPDU Guard disables a port by placing it into an error disabled (err-disabled) interface state. BPDU Guard does so when it is enabled on the interface, regardless of whether PortFast is also enabled. The two correct answers both state that BPDU Guard is enabled, while the two incorrect answers list it as disabled.
5. A and E. Root Guard reacts to the receipt of a superior BPDU by disabling the port. To do so, it leaves the interface state as is in a connected state. Instead, it manipulates the STP port state, changing it to a special state called broken. The commands display the broken state with letters BRK. So, the **show interfaces status** command lists a connected interface state, while the **show spanning-tree** command lists a port state of BRK, or broken, which stops all traffic on the interface.

6. D. IOS uses the **channel-group** configuration command to create an EtherChannel. Then the term *etherchannel* is used in the **show etherchannel** command, which displays the status of the channel. The output of this **show** command then names the channel a *PortChannel*. The only answer that is not used somewhere in IOS to describe this multilink channel is *Ethernet-Channel*.

7. B and D. The **channel-group** command will direct the switch to use LACP to dynamically negotiate to add a link to an EtherChannel when the command uses the **active** and **passive** keywords, respectively. The **desirable** and **passive** keywords direct the switch to use PAgP instead of LACP. Of the four answers, the two correct answers use two LACP values, while the two incorrect answers use at least one value that would cause the switch to use PAgP, making the answers incorrect.

   Of the two correct answers, both combinations result in the switches attempting to add the link to an EtherChannel using LACP as the negotiation protocol. If both switches used the **passive** keyword, they would both sit and wait for the other switch to begin sending LACP messages and therefore never attempt to add the link to the channel.

8. C. EtherChannel load distribution, or load balancing, on Cisco Catalyst switches uses an algorithm. The algorithm examines some fields in the various headers, so messages that have the same values in those fields always flow over the same link in a particular EtherChannel. Note that it does not break the frames into smaller fragments or use a round-robin approach that ignores the header values, and it does not examine link utilization when making the choice.

## Chapter 11

1. B and D. The general rule to determine whether two devices' interfaces should be in the same subnet is whether the two interfaces are separated from each other by a router. To provide a way for hosts in one VLAN to send data to hosts outside that VLAN, a local router must connect its LAN interface to the same VLAN as the hosts and have an address in the same subnet as the hosts. All the hosts in that same VLAN on the same switch would not be separated from each other by a router, so these hosts would also be in the same subnet. However, another PC, connected to the same switch but in a different VLAN, will require its packets to flow through a router to reach Host A, so Host A's IP address would need to be in a different subnet compared to this new host.

2. D. By definition, two address values in every IPv4 subnet cannot be used as host IPv4 addresses: the first (lowest) numeric value in the subnet for the subnet ID and the last (highest) numeric value in the subnet for the subnet broadcast address.

3. B and C. At least 7 subnet bits are needed because $2^6 = 64$, so 6 subnet bits could not number 100 different subnets. Seven subnet bits could because $2^7 = 128 => 100$. Similarly, 6 host bits is not enough because $2^6 - 2 = 62$, but 7 host bits is enough because $2^7 - 2 = 126 => 100$.

   The number of network, subnet, and host bits must total 32 bits, making one of the answers incorrect. The answer with 8 network bits cannot be correct because the question states that a Class B network is used, so the number of network bits must

always be 16. The two correct answers have 16 network bits (required because the question states the use of a Class B network) and at least 7 subnet and host bits each.

4. A and C. The private IPv4 networks, defined by RFC 1918, are Class A network 10.0.0.0, the 16 Class B networks from 172.16.0.0 to 172.31.0.0, and the 256 Class C networks that begin with 192.168.

5. A, D, and E. The private IPv4 networks, defined by RFC 1918, are Class A network 10.0.0.0, the 16 Class B networks from 172.16.0.0 to 172.31.0.0, and the 256 Class C networks that begin with 192.168. The three correct answers are from the public IP network range, and none are reserved values.

6. A and C. An unsubnetted Class A, B, or C network has two parts: the network and host parts.

7. B. An unsubnetted Class A, B, or C network has two parts: the network and host parts. The subnet part does not exist in that case. To perform subnetting, the engineer creates a new subnet part by choosing to use a subnet mask, defining a smaller number of host bits, which makes space for some bits to be used to number different subnets. So, the host part of the address structure gets smaller in the after-subnetting case. The subnet part of the address structure moves from size 0 (nonexistent) to some number of subnet bits after the engineer chooses a subnet (nondefault) mask. The network part remains a constant size throughout, whether subnetting or not.



## Chapter 12

1. B and C. Class A networks have a first octet in the range of 1–126, inclusive, and their network IDs have a 0 in the last three octets. A network ID of 130.0.0.0 is actually a Class B network (first octet range 128–191, inclusive). All addresses that begin with 127 are reserved, so 127.0.0.0 is not a Class A network.

2. E. All Class B networks begin with values between 128 and 191, inclusive, in their first octets. The network ID has any value in the 128–191 range in the first octet, and any value from 0 to 255 inclusive in the second octet, with decimal 0s in the final two octets. Two of the answers show a 255 in the second octet, which is acceptable. Two of the answers show a 0 in the second octet, which is also acceptable.

3. B and D. The first octet (172) is in the range of values for Class B addresses (128–191). As a result, the network ID can be formed by copying the first two octets (172.16) and writing 0s for the last two octets (172.16.0.0). The default mask for all Class B networks is 255.255.0.0, and the number of host bits in all unsubnetted Class B networks is 16.

4. A and C. The first octet (192) is in the range of values for Class C addresses (192–223). As a result, the network ID can be formed by copying the first three octets (192.168.6) and writing 0 for the last octet (192.168.6.0). The default mask for all Class C networks is 255.255.255.0, and the number of host bits in all unsubnetted Class C networks is 8.

5. D. To find the network broadcast address, first determine the class, and then determine the number of host octets. At that point, convert the host octets to 255 to create the network broadcast address. In this case, 10.1.255.255 is in a Class A network, with the last three octets as host octets, for a network broadcast address of

10.255.255.255. For 192.168.255.1, it is a Class C address, with the last octet as the host part, for a network broadcast address of 192.168.255.255. Address 224.1.1.255 is a Class D address, so it is not in any unicast IP network and the question does not apply. For 172.30.255.255, it is a Class B address, with the last two octets as host octets, so the network broadcast address is 172.30.255.255.

## Chapter 13

1. C. If you think about the conversion one octet at a time, the first two octets each convert to 8 binary 1s. The 254 converts to 8-bit binary 11111110, and the decimal 0 converts to 8-bit binary 00000000. So, the total number of binary 1s (which defines the prefix length) is 8 + 8 + 7 + 0 = /23.
2. B. If you think about the conversion one octet at a time, the first three octets each convert to 8 binary 1s. The 240 converts to 8-bit binary 11110000, so the total number of binary 1s (which defines the prefix length) is 8 + 8 + 8 + 4 = /28.
3. B. Remember that /30 is the equivalent of the mask that in binary has 30 binary 1s. To convert that to DDN format, write down all the binary 1s (30 in this case), followed by binary 0s for the remainder of the 32-bit mask. Then take 8 bits at a time and convert from binary to decimal (or memorize the nine possible DDN mask octet values and their binary equivalents). Using the /30 mask in this question, the binary mask is 11111111 11111111 11111111 11111100. Each of the first three octets is all binary 1s, so each converts to 255. The last octet, 11111100, converts to 252, for a DDN mask of 255.255.255.252. See Appendix A, "Numeric Reference Tables," for a decimal/binary conversion table.
4. C. The size of the network part is always either 8, 16, or 24 bits, based on whether it is Class A, B, or C, respectively. As a Class A address, N=8. The mask 255.255.255.0, converted to prefix format, is /24. The number of subnet bits is the difference between the prefix length (24) and N, so S=16 in this case. The size of the host part is a number that, when added to the prefix length (24), gives you 32, so H=8 in this case.
5. A. The size of the network part is always either 8, 16, or 24 bits, based on whether it is Class A, B, or C, respectively. As a Class C address, N=24. The number of subnet bits is the difference between the prefix length (27) and N, so S=3 in this case. The size of the host part is a number that, when added to the prefix length (27), gives you 32, so H=5 in this case.
6. D. Classless addressing rules define a two-part IP address structure: the prefix and the host part. This logic ignores Class A, B, and C rules and can be applied to the 32-bit IPv4 addresses from any address class. By ignoring Class A, B, and C rules, classless addressing ignores any distinction as to the network part of an IPv4 address.
7. A and B. The masks in binary define a number of binary 1s, and the number of binary 1s defines the length of the prefix (network + subnet) part. With a Class B network, the network part is 16 bits. To support 100 subnets, the subnet part must be at least 7 bits long. Six subnet bits would supply only $2^6$ = 64 subnets, while 7 subnet bits supply $2^7$ = 128 subnets. The /24 answer supplies 8 subnet bits, and the 255.255.255.252 answer supplies 14 subnet bits.

## Chapter 14

1. D. When using classful IP addressing concepts as described in Chapter 13, "Analyzing Subnet Masks," addresses have three parts: network, subnet, and host. For addresses in a single classful network, the network parts must be identical for the numbers to be in the same network. For addresses in the same subnet, both the network and subnet parts must have identical values. The host part differs when comparing different addresses in the same subnet.
2. B and D. In any subnet, the subnet ID is the smallest number in the range, the subnet broadcast address is the largest number, and the usable IP addresses sit between them. All numbers in a subnet have identical binary values in the prefix part (classless view) and network + subnet part (classful view). To be the lowest number, the subnet ID must have the lowest possible binary value (all 0s) in the host part. To be the largest number, the broadcast address must have the highest possible binary value (all binary 1s) in the host part. The usable addresses do not include the subnet ID and subnet broadcast address, so the addresses in the range of usable IP addresses never have a value of all 0s or 1s in their host parts.
3. C. The mask converts to 255.255.255.0. To find the subnet ID, for each octet of the mask that is 255, you can copy the IP address's corresponding values. For mask octets of decimal 0, you can record a 0 in that octet of the subnet ID. As such, copy the 10.7.99 and write a 0 for the fourth octet, for a subnet ID of 10.7.99.0.
4. C. First, the resident subnet (the subnet ID of the subnet in which the address resides) must be numerically smaller than the IP address, which rules out one of the answers. The mask converts to 255.255.255.252. As such, you can copy the first three octets of the IP address because of their value of 255. For the fourth octet, the subnet ID value must be a multiple of 4, because 256 – 252 (mask) = 4. Those multiples include 96 and 100, and the right choice is the multiple closest to the IP address value in that octet (97) without going over. So, the correct subnet ID is 192.168.44.96.
5. C. The resident subnet ID in this case is 172.31.77.192. You can find the subnet broadcast address based on the subnet ID and mask using several methods. Following the decimal process in the book, the mask converts to 255.255.255.224, making the interesting octet be octet 4, with magic number 256 – 224 = 32. For the three octets where the mask = 255, copy the subnet ID (172.31.77). For the interesting octet, take the subnet ID value (192), add magic (32), and subtract 1, for 223. That makes the subnet broadcast address 172.31.77.223.
6. C. To answer this question, you need to find the range of addresses in the subnet, which typically then means you need to calculate the subnet ID and subnet broadcast address. With a subnet ID/mask of 10.1.4.0/23, the mask converts to 255.255.254.0. To find the subnet broadcast address, following the decimal process described in this chapter, you can copy the subnet ID's first two octets because the mask's value is 255 in each octet. You write a 255 in the fourth octet because the mask has a 0 on the fourth octet. In octet 3, the interesting octet, add the magic number (2) to the subnet ID's value (4), minus 1, for a value of 2 + 4 – 1 = 5. (The magic number in this case is calculated as 256 – 254 = 2.) That makes the broadcast address 10.1.5.255. The last usable address is 1 less: 10.1.5.254. The range that includes the last 100 addresses is 10.1.5.155 – 10.1.5.254.

# Chapter 15

1. A. With 50 percent growth, the mask needs to define enough subnet bits to create 150 subnets. As a result, the mask needs at least 8 subnet bits (7 subnet bits supply $2^7$, or 128, subnets, and 8 subnet bits supply $2^8$, or 256, subnets). Similarly, the need for 50 percent growth in the size for the largest subnet means that the host part needs enough bits to number 750 hosts/subnet. Nine host bits are not enough ($2^9 - 2 = 510$), but 10 host bits supply 1022 hosts/subnet ($2^{10} - 2 = 1022$). With 16 network bits existing because of the choice to use a Class B network, the design needs a total of 34 bits (at least) in the mask (16 network, 8 subnet, 10 host), but only 32 bits exist—so no single mask meets the requirements.

2. B. With a growth of 20 percent, the design needs to support 240 subnets. Seven subnet bits do not meet the need ($2^7 = 128$), but 8 subnet bits do meet the need ($2^8 = 256$). To support 120 hosts/subnet, with 20% growth, the mask should support 144 hosts/subnet. That number requires 8 host bits ($2^8 - 2 = 254$). As a result, you need a minimum 8 subnet bits and 8 host bits.

   The right answer, 10.0.0.0/22, has 8 network bits because the network class is Class A, 14 subnet bits (/22 – 8 = 14), and 10 host bits (32 – 22 = 10). This mask supplies at least 8 subnet bits and at least 8 host bits.

   The answer with the /25 mask supplies only 7 host bits, making it incorrect. The answer showing 172.16.0.0/23 supplies 9 host bits, which is enough; however, it uses 16 network bits with the Class B network, leaving too few subnet bits (7). The answer that shows Class C network 192.168.7.0 with mask /24 supplies 8 host bits but 0 subnet bits.

3. B. To support 1000 subnets, 10 subnet bits ($2^{10} = 1024$) are needed. The design uses a Class B network, which means that 16 network bits exist as well. So, the shortest mask that meets the requirements is 255.255.255.192, or /26, composed of 16 network plus 10 subnet bits. The /28 answer also supplies enough subnets to meet the need, but compared to /26, /28 supplies fewer host bits and so fewer hosts/subnet.

4. C and D. The mask converts to 255.255.252.0, so the difference from subnet ID to subnet ID (called the magic number in this chapter) is 256 – 252 = 4. So, the subnet IDs start with 172.30.0.0, then 172.30.4.0, then 172.30.8.0, and so on, adding 4 to the third octet. The mask, used with a Class B network, implies 6 subnet bits, for 64 total subnet IDs. The last of these, 172.30.252.0, can be recognized in part because the third octet, where the subnet bits sit, has the same value as the mask in that third octet.

5. A. The first (numerically lowest) subnet ID is the same number as the classful network number, or 192.168.9.0. The remaining subnet IDs are each 8 larger than the previous subnet ID, in sequence, or 192.168.9.8, 192.168.9.16, 192.168.9.24, 192.168.9.32, and so on, through 192.168.9.248.

6. D. Using mask /24 (255.255.255.0), the subnet IDs increment by 1 in the third octet. The reasoning is that with a Class B network, 16 network bits exist, and with mask /24, the next 8 bits are subnet bits, so the entire third octet contains subnet bits. All the subnet IDs will have a 0 as the last octet, because the entire fourth octet consists of host bits. Note that 172.19.0.0 (the zero subnet) and 172.19.255.0 (the broadcast subnet) might look odd but are valid subnet IDs.

## Chapter 16

1. B and D. Cisco routers originally used IOS, with some models today still using IOS. Most of the enterprise router product line uses the newer IOS XE operating system. CatOS, short for Catalyst OS, refers to the original Cisco switch operating system.
2. B. The switch and router CLI follows the same basic flow with many commands in common. The three incorrect answers are incorrect because they describe actions that can occur on both routers and switches. However, the user must configure router interfaces with IP addresses. Switches, when used as Layer 2 switches only, do not need any IP addresses on their Layer 2 physical interfaces.
3. A and C. To route packets on an interface, the router interface configuration must include an IP address and mask. One correct command shows the correct single command used to configure both values, while one incorrect command shows those settings as two separate (nonexistent) commands. Also, to route packets, the interface much reach an "up/up" state; that is, the **show interfaces** and other commands list two status values, and both must be "up." The **no shutdown** command enables the interface so that it can reach an up/up state, assuming the interface has correct cabling and is connected to an appropriate device. One incorrect answer mentions the **description** command, which is useful but has nothing to do with making the interface work properly.
4. B. If the first of the two status codes is "down," it typically means that a Layer 1 problem exists. In this case, the question states that the router connects to a switch with a UTP straight-through cable, which is the correct cable pinout. Of the two answers that mention the **shutdown** command, if the router interface were shut down, the first router status code would be "administratively down," so that answer is incorrect. However, if the neighboring device interface sits in a shutdown state, the router will sense no electrical signals over the cable, seeing that as a physical problem, and place the interface into a "down/down" state, making that answer correct.

   Second, the two answers that mention interface IP addresses have no impact on the status codes of the **show interfaces brief** command. Both answers imply that the interface does not have an IP address configured; however, the IP address configuration has no effect on the interface status codes, making both answers incorrect.
5. C. The **show ip interface brief** command lists all the interface IPv4 addresses but none of the masks. The other three commands list both the address and mask.
6. B. A router has one IPv4 address for each interface in use, whereas a LAN switch has a single IPv4 address that is just used for accessing the switch. The rest of the answers list configuration settings that use the same conventions on both routers and switches.

## Chapter 17

1. A and C. The route defines the group of addresses represented by the route using the subnet ID and mask. The router can use those numbers to find the range of addresses that should be matched by this route. The other two answers list facts useful when forwarding packets that happen to match the route.

2. D. Each time a router routes an IP packet, it de-encapsulates (removes) the IP packet from the incoming data-link frame. Once it decides where to forward the packet next, it re-encapsulates the packet in a new data-link frame. That occurs even if the incoming and outgoing data links happen to be the same type, as is the case in this scenario. So, all three routers de-encapsulate the IP packet. Since all links are Ethernet links, all three de-encapsulation actions removed the packet from an Ethernet frame.
3. A and D. First, for the subnetting math, address 10.1.1.100, with mask /26, implies a subnet ID of 10.1.1.64. Also, mask /26 converts to a DDN mask of 255.255.255.192. For any working router interface, after adding the **ip address** command to configure an address and mask, the router adds a connected route for the subnet. In this case, that means the router adds a connected route for subnet 10.1.1.64 255.255.255.192. The router also adds a route called a local route, which is a route for the interface IP address with a 255.255.255.255 mask. In this case, that means the router adds a local route for address 10.1.1.100 with mask 255.255.255.255.
4. B and C. The **ip route** command can refer to the IP address of the next-hop router on the link between the two routers, or to the local router's outgoing interface ID. The incorrect answers reverse those items, mentioning the local router's IP address and the next-hop router's interface ID.
5. A. The correct syntax lists a subnet number, then a subnet mask in dotted-decimal form, and then either an outgoing interface or a next-hop IP address. Of the incorrect answers, one omits the subnet mask, while two use a prefix-style mask instead of a DDN mask.
6. B. The network engineer issued the command, but the router did not add an IP route. So, either the command had a syntax error, or the router accepted the command but has some reason to believe that it should not add a route to the table.

   Two (incorrect) answers suggest the command has a syntax error: one answer with a general claim of a syntax error, and another explicitly stating that the next-hop IP address is missing. However, the **ip route 10.1.1.0 255.255.255.0 s0/0/0** command is syntactically correct. Note that with outgoing interface S0/0/0 listed, the command does not need a next-hop IP address.

   As for reasons why IOS would not add a route once it accepts the command into the configuration, IOS performs several checks of the contents of a valid **ip route** command before adding the route to the routing table. It checks whether the outgoing interface is up/up (as noted in this question's correct answer) and whether it has a route to reach the next-hop address. Also, if the router already has a route to the same subnet learned from another source, the router checks whether the other route has a better administrative distance.

## Chapter 18

1. A and F. Of all the commands listed, only the two correct answers are syntactically correct router configuration commands. The command to enable 802.1Q trunking is **encapsulation dot1q** *vlan_id*.
2. B and C. Subinterface G0/1.1 must be in an administratively down state due to the **shutdown** command being issued on that subinterface. For subinterface G0/1.2, its

status cannot be administratively down because of the **no shutdown** command. G0/1.2's state will then track to the state of the underlying physical interface. With a physical interface state of down/down, subinterface G0/1.2 will be in a down/down state in this case.

**3.** C. The configuration of the Layer 3 switch's routing feature uses VLAN interfaces. The VLAN interface numbers must match the associated VLAN ID, so with VLANs 1, 2, and 3 in use, the switch will configure **interface vlan 1**, **interface vlan 2** (which is the correct answer), and **interface vlan 3**. The matching connected routes, like all connected IP routes, will list the VLAN interfaces.

As for the incorrect answers, a list of connected routes will not list any next-hop IP addresses. Each route will list an outgoing interface; the outgoing interface will not be a physical interface, but rather a VLAN interface, because the question states that the configuration uses SVIs. Finally, all the listed subnets have a /25 mask, which is 255.255.255.128, so none of the routes will list a 255.255.255.0 mask.

**4.** C and D. First, for the correct answers, a Layer 3 switch will not route packets on a VLAN interface unless it is in an up/up state. When using autostate, a VLAN interface will only be up/up if the matching VLAN (with the same VLAN number) exists on the switch, is not shut down, and at least one port is up and active in that VLAN. For one correct answer, if the **no vlan 2** command were issued, deleting VLAN 2, the switch would move interface VLAN 2 to a up/down state so it could no longer route packets. For the other correct answer, disabling VLAN 2 with the **shutdown** command in VLAN configuration mode has the same result.

As for the incorrect answers, when using autostate, a Layer 3 switch needs only one access port or trunk port forwarding for a VLAN to enable routing for that VLAN, so nine of the ten access ports in VLAN 2 could fail, leaving one working port, and the switch would keep routing for VLAN 2.

A **shutdown** of VLAN 4 does not affect routing for VLAN interfaces 2 and 3. Had that answer listed VLAN 2 or 3, it would be a reason to make routing fail for that VLAN interface.

**5.** A and C. With a Layer 3 EtherChannel, the physical ports and the port-channel interface must disable the behavior of acting like a switch port and therefore act like a routed port, through the configuration of the **no switchport** interface subcommand. (The **routedport** command is not an IOS command.) Once created, the physical interfaces should not have an IP address configured. The port-channel interface (the interface representing the EtherChannel) should be configured with the IP address.

**6.** B and C. With a Layer 3 EtherChannel, two configuration settings must be the same on all the physical ports, specifically the speed and duplex as set with the **speed** and **duplex** commands. Additionally, the physical ports and port-channel port must all have the **no switchport** command configured to make each act as a routed port. So, having a different speed setting, or being configured with **switchport** rather than **no switchport**, would prevent IOS from adding interface G0/2 to the Layer 3 EtherChannel.

As for the wrong answers, both would cause an issue adding the port to a Layer 2 EtherChannel but do not cause a problem with a Layer 3 EtherChannel. Once Layer 2 operations have been disabled because of the **no switchport** command, those

settings do not then cause problems for the Layer 3 EtherChannel. So, Layer 2 settings about access VLANs, trunking allowed lists, and STP settings, which must match before an interface can be added to a Layer 2 EtherChannel, do not matter for a Layer 3 EtherChannel.

7. A. On a router that has some routed ports, plus some switched ports, IOS supports LAN switching subcommands on the switched ports only. So, when in interface configuration mode for one of a router's switched interfaces, IOS accepts the **switchport access** command but not the **ip address** command. The router supports the **description** subcommand on both switched and routed ports, making that answer incorrect. Finally, one answer lists a global command (**hostname**), making that answer incorrect because the question asks for interface subcommands.

## Chapter 19

1. B and D. The client sends a Discover message, with the server returning an Offer message. The client then sends a Request, with the server sending back the IP address in the Acknowledgment message.
2. A and B. The two correct answers list the two primary facts that impact which IP addresses the server will lease to clients. For the incorrect answer about DNS servers, the DHCP server does supply the IP address of the DNS servers, but not the hostnames of the DNS servers. Also, the DHCP server supplies the IP address (but not the MAC address) of the default gateway in each subnet.
3. A and C. A router needs to act as a DHCP relay agent if DHCP clients exist on the connected subnet and there is no DHCP server in that subnet. If a DHCP server exists in the subnet, the router does not need to forward DHCP messages to a remote DHCP server (which is the function of a DHCP relay agent). The answer that mentions the **ip address dhcp** command makes the router interface act as a DHCP client and has nothing to do with DHCP relay agent.
4. D. The **ip address dhcp** command tells the router to obtain its address using DHCP. The router learns all the same information that a normal DHCP client would learn. The router uses the address listed as the default gateway to build a default route, using the default gateway IP address as the next-hop address. The router continues to work like a router always does, forwarding packets based on its IP routing table.
5. B and C. The output shows the MAC address, IP address, subnet mask (in hex format), and the subnet broadcast address. Of those, the DHCP server supplies the information in the two correct answers. The two incorrect answers mention the MAC address (not supplied by DHCP, but known to the device's NIC) and the subnet broadcast address (calculated by the host).
6. D. Windows supports both **ipconfig** and **ipconfig /all** commands, but the **ipconfig** command does not mention the DNS servers. Note that the **ifconfig** command works on Linux and macOS but not Windows, and the **ifconfig /all** command is an invalid command on all three.

## Chapter 20

There are no questions for this chapter.

## Chapter 21

1. D. Both versions of RIP use distance vector logic, and EIGRP uses a different kind of logic, characterized either as advanced distance vector or a balanced hybrid.
2. C and D. Both versions of RIP use the same hop-count metric, neither of which is affected by link bandwidth. EIGRP's metric, by default, is calculated based on bandwidth and delay. OSPF's metric is a sum of outgoing interfaces costs, with those costs (by default) based on interface bandwidth.
3. B, C, and D. Of the listed routing protocols, only the old RIP Version 1 (RIP-1) protocol does not support variable-length subnet masks (VLSM).
4. C. LSAs contain topology information that is useful in calculating routes, but the LSAs do not directly list the route that a router should add to its routing table. In this case, R1 would run a calculation called the Shortest Path First (SPF) algorithm, against the LSAs, to determine what IP routes to add to the IP routing table.
5. B. Neighboring OSPF routers that complete the database exchange are considered fully adjacent and rest in a full neighbor state. The up/up and final states are not OSPF states at all. The 2-way state is either an interim state or a stable state between some routers on the same VLAN.
6. C. The correct answer is the one advantage of using a single-area design. The three wrong answers are advantages of using a multiarea design, with all reasons being much more important with a larger internetwork.

## Chapter 22

1. B. The **network 10.0.0.0 0.255.255.255 area 0** command matches all three interface IP addresses because it compares the first octet only (10) and matches in each case.

   The three incorrect answers do not match all three interface IP addresses because they each compare at least one octet that does not match the address in the **network** command:

   **network 10.0.0.0 0.0.0.0** requires an exact match of all four octets (10.0.0.0), which matches no interfaces.

   **network 10.0.0.0 0.0.0.255** requires an exact match of the first three octets (10.0.0), which matches none of the interface IP addresses.

   **network 10.0.0.0 0.0.255.255** requires an exact match of the first two octets (10.0), which matches none of the interface IP addresses.
2. A. The **network 10.1.0.0 0.0.255.255 area 0** command matches all IP addresses that begin with 10.1, enabling OSPF in area 0 on all interfaces. The three incorrect answers do not match all three interface IP addresses because they each compare at least one octet that does not match the address in the **network** command:

   **network 10.0.0.0 0.255.255.0** ignores the middle two octets but compares the first (10) and last (0) octets to the interface addresses. The first octet matches, but the fourth octet matches none of the addresses.

**network 10.1.1.0 0.x.1x.0** does not meet syntax requirements because of the letters (x) in the wildcard mask. It would be rejected when attempted in configuration mode.

**network 10.1.1.0 255.0.0.0** ignores the first octet but compares the last three octets (1.1.0) to the addresses. None of the addresses end in 1.1.0, so no addresses match this command.

3. A and E. Of the three wrong answers, two are real commands that simply do not list the OSPF neighbors. **show ip ospf interface brief** lists interfaces on which OSPF is enabled but does not list neighbors. **show ip interface** lists IPv4 details about interfaces, but none related to OSPF. One incorrect answer, **show ip neighbor**, is not a valid IOS command.
4. B. The rule for choosing the OSPF RID begins with the **router-id** command in the OSPF process configuration, but the router had no such command. The next rule considers all working (up/up) loopback interfaces, and among those, OSPF chooses the numerically highest IP address. In this case, two such loopback interfaces exist, with loopback 1, with address 10.8.8.8, having the numerically highest IP address.
5. B. With OSPFv2 interface configuration mode, the configuration looks just like the traditional configuration, with a couple of exceptions. The **network** router subcommand is no longer required. Instead, each interface on which OSPF should be enabled is configured with an **ip ospf** *process-id* **area** *area-id* interface subcommand. This command refers to the OSPF routing process that should be enabled on the interface and specifies the OSPFv2 area.
6. A and D. Many of the **show** commands for OSPF do not happen to note whether OSPF happens to be enabled due to an interface subcommand the (**ip ospf** interface subcommand) or a router subcommand (the **network** command). The **show ip protocols** command lists all interfaces on which OSPF has been enabled using the **ip ospf interface** subcommand under the heading "Routing on Interfaces Configured Explicitly." Additionally, the **show ip ospf interface** command, which lists many lines of output per interface, lists the phrase "Attached via Interface Enable." Also, although not in the answers, you can also look at the configuration with the **show running-config** or **show startup-config** command.

## Chapter 23

1. B and D. By default, IOS assigns Ethernet interfaces an OSPF network type of broadcast, with an OSPF interface priority of 1. As a result, both routers attempt to discover the other routers on the link (which identifies one correct answer).

   The broadcast network type means that the routers also attempt to elect a DR and BDR. With a priority tied, the routers choose the DR based on the highest router ID (RID) values, meaning that R2 will become the DR and R1 will become the BDR. These facts show why the two incorrect answers are incorrect. The other correct answer is correct because the **show ip ospf neighbor** command lists the local router's neighbor relationship state (FULL) and the role filled by that neighbor (DR), which would be the output shown on R1 when R2 is acting as DR.

2. B and C. First, the OSPF point-to-point network type causes the two routers to dynamically discover neighbors, making one answer correct.

   Next, IOS assigns a default OSPF interface priority of 1, so R1's configured priority of 11 would be better in a DR/BDR election. However, the point-to-point network type causes the router to not use a DR/BDR on the interface. As a result, the answer about R1 becoming the DR is incorrect (because no DR exists at all), and the answer listing a state of "FULL/DR" is incorrect for the same reason. However, the answer that claims that R2 will be neither DR nor BDR is true because no DR or BDR is elected.

3. D. The **show ip ospf interface brief** command lists a pair of counters under the heading "Nbrs F/C" on the far right of the output. The first of the two numbers represents the number of fully adjacent neighbors (2 in this case), and the second number represents the total number of neighbors.

4. B. The default OSPF priority setting is 1. Once configured with 100, R2 has a higher priority. However, the routers only use the priority values when electing a new DR, so as long as the neighbor relationship is stable, no new DR election will occur. So, any change to make R2 (with higher priority) the DR occurs only after a failure that breaks the current neighbor relationship.Two of the answers refer to other timing as to when R2 becomes the DR. Another distractor states that R2 will cease to serve as BDR, which is not the case.

5. B. SPF calculates the cost of a route as the sum of the OSPF interface costs for all outgoing interfaces in the route. The interface cost can be set directly (**ip ospf cost**), or IOS uses a default based on the reference bandwidth and the interface bandwidth. Of the listed answers, **delay** is the only setting that does not influence OSPFv2 metric calculations.

6. D. The configuration of the interface subcommand **ip ospf hello-interval 15** sets the Hello interval to 15. Also, without any explicit configuration of a Dead interval, IOS also sets the Dead interval to 4X the Hello interval or 60 in this case. The question stem describes the timing and purpose of the Dead interval: how long to wait after not receiving any more Hellos before believing the neighbor has failed.

## Chapter 24

1. A and D. As worded, the correct answers list a scenario that would prevent the neighbor relationship. One correct answer mentions the use of two different OSPF areas on the potential OSPF neighbors; to become neighbors, the two routers must use the same area number. The other correct answer mentions the use of two different Hello timers, a mismatch that causes two routers to reject each other and to not become neighbors.

   The two incorrect answers list scenarios that do not cause issues, making them incorrect answers. One mentions mismatched OSPF process IDs; OSPF process IDs do not need to match for two routers to become neighbors. The other incorrect answer (that is, a scenario that does not cause a problem) mentions the use of two different priority values. The priority values give OSPF a means to prefer one router over the other when electing a DR/BDR, so the setting is intended to be set to different values on different routers and does not cause a problem.

2. C. As worded, the correct answers should be a scenario that would prevent the neighbor relationship. The answers all list values that are identical or similar on the two routers. Of those, the use of an identical OSPF Router ID (RID) on the two routers prevents them from becoming neighbors, making that one answer correct.

   Of the incorrect answers, both routers must have the same Dead interval, so both using a Dead interval of 40 causes no issues. The two routers can use any OSPF process ID (the same or different value, it does not matter), making that answer incorrect. Finally, the two routers' IP addresses must be in the same subnet, so again that scenario does not prevent R13 and R14 from becoming neighbors.

3. D. The OSPF **shutdown** command tells the OSPF process to stop operating. That process includes removing any OSPF-learned routes from the IP routing table, clearing the router's LSDB, and closing existing OSPF neighbor relationships. In effect, it causes OSPF to stop working on the router, but it does retain the configuration so that a **no shutdown** command in OSPF configuration mode will cause the router to start using OSPF again with no changes to the configuration.

4. B. OSPF uses an equal-cost multipath feature, in which when it calculates multiple routes for the same subnet that tie with the lowest metric, the router places multiple routes into the routing table. IOS limits the number of such routes for one destination subnet per the **maximum-paths** setting on the router, which typically defaults to 6. The router would not use the route with metric 15001, as it is worse than the other two routes' metric of 15000.

5. D. Within a routing protocol, the routing protocol will choose the best route based on the metric. As a result, OSPF picks the metric 1000 route while EIGRP chooses its metric 1,000,000 route. Then the router must choose between the two routing protocol sources using the administrative distance. With default settings, EIGRP has a better administrative distance of 90 versus OSPF's 110. As a result, the router places the best EIGRP route into its routing table, the route learned by EIGRP with metric 1,000,000.

6. D. Each route defines a range of IP addresses as follows:

   - 172.20.90.9/32: 172.20.90.9 only
   - 172.20.88.0/23: 172.20.88.0–172.20.89.255
   - 172.20.80.0/20: 172.20.80.0–172.20.95.255
   - 172.20.0.0/16: 172.20.0.0–172.20.255.255
   - 0.0.0.0/0: 0.0.0.0–255.255.255.255

   Given those ranges, a packet destined for address 172.20.89.100 matches all but the first route in the list.

7. C. Each route defines a range of IP addresses, as follows:

   - 172.20.90.9/32: 172.20.90.9 only
   - 172.20.88.0/23: 172.20.88.0–172.20.89.255
   - 172.20.80.0/20: 172.20.80.0–172.20.95.255

- 172.20.0.0/16: 172.20.0.0–172.20.255.255
- 0.0.0.0/0: 0.0.0.0–255.255.255.255

Given those ranges, a packet destined for address 172.20.90.1 matches the last three routes in the list. Among those, the router will use the most specific route, the route with the largest number of prefix bits. As a result, the router uses the route with prefix length /20, which has a next-hop address of 172.20.13.3.



## Chapter 25

1. C. NAT, specifically the PAT feature that allows many hosts to use private IPv4 addresses while being supported by a single public IPv4 address, was one short-term solution to the IPv4 address exhaustion problem. IP version 5 existed briefly as an experimental protocol and had nothing to do with IPv4 address exhaustion. IPv6 directly addresses the IPv4 address exhaustion problem, but it is a long-term solution. ARP has no impact on the number of IPv4 addresses used.
2. A. Routers use the same process steps when routing IPv6 packets as they do when routing IPv4 packets. Routers route IPv6 packets based on the IPv6 addresses listed inside the IPv6 header by comparing the destination IPv6 address to the router's IPv6 routing table. As a result, the router discards the incoming frame's data-link header and trailer, leaving an IPv6 packet. The router compares the destination (not source) IPv6 address in the header to the router's IPv6 (not IPv4) routing table and then forwards the packet based on the matched route.
3. D. If you are following the steps in the book, the first step removes up to three leading 0s in each quartet, leaving FE80:0:0:0:100:0:0:123. This value leaves two strings of consecutive all-0 quartets; when you change the longest string of all 0s to ::, the address is FE80::100:0:0:123.
4. B. This question has many quartets that make it easy to make a common mistake: removing trailing 0s in a quartet of hex digits. Only leading 0s in a quartet and not trailing 0s should be removed. Many of the quartets have trailing 0s (0s on the right side of the quartet), so make sure not to remove those 0s.
5. A. The unabbreviated version of an IPv6 address must have 32 digits, and only one answer has 32 hex digits. In this case, the original number shows four quartets and a ::. So, the :: was replaced with four quartets of 0000, making the number eight. Then, for each quartet with fewer than four digits, leading 0s were added, so each quartet has four hex digits.
6. C. The /64 prefix length means that the last 64 bits, or last 16 digits, of the address should be changed to all 0s. That process leaves the unabbreviated subnet prefix as 2000:0000:0000:0005:0000:0000:0000:0000. The last four quartets are all 0s, making that string of all 0s be the longest and best string of 0s to replace with ::. After removing the leading 0s in other quartets, the answer is 2000:0:0:5::/64.

## Chapter 26

1. C. Unique local addresses begin with FD in the first two digits.
2. A. Global unicast addresses begin with a hex 2 or 3.

3. D. The global routing prefix defines the address block, represented as a prefix value and prefix length, assigned to an organization by some numbering authority. The global routing prefix acts as the initial part of IPv6 addresses within the company for the number of bits defined by the prefix length. Similarly, when a company uses a public IPv4 address block, all the addresses have the same value in the network part, which also acts as the initial part of IPv4 addresses.
4. B. Subnetting a global unicast address block, using a single prefix length for all subnets, breaks the addresses into three parts. The parts are the global routing prefix, subnet ID, and interface ID.
5. D. Unique local addresses begin with a 2-hex-digit prefix of FD, followed by the 10-hex-digit global ID.

## Chapter 27

1. A. The one correct answer lists the exact same IPv6 address listed in the question, with a /64 prefix length and no spaces in the syntax of the answer. Another (incorrect) answer is identical, except that it leaves a space between the address and prefix length, which is incorrect syntax. The two answers that list the **eui-64** parameter list an address and not a prefix; they should list a prefix to be correct. However, even if these two incorrect answers had listed the prefix of the address shown (2001:1:1:1::), the EUI-64 process would not have resulted in the IPv6 address listed in the question.
2. B. With the **eui-64** parameter, the router will calculate the interface ID portion of the IPv6 address based on its MAC address. Beginning with 5055.4444.3333, the router injects FF FE in the middle (5055.44FF.FE44.3333). Then the router inverts the seventh bit in the first byte. To see the change, hex 50 to binary 0101 0000. Then change bit 7, so the string becomes 0101 0010, which converts back to hex 52. The final interface ID value is 5255:44FF:FE44:3333. The wrong answers simply list a different value.
3. A and C. Of the four answers, the two correct answers show the minimal required configuration to support IPv6 on a Cisco router: enabling IPv6 routing (**ipv6 unicast-routing**) and enabling IPv6 on each interface, typically by adding a unicast address to each interface (**ipv6 address…**). The two incorrect answers list nonexistent commands.
4. B and D. The **show ipv6 route connected** command lists all known connected routes, with each route listing the prefix/length of the route. The **show ipv6 interface g0/0/0** command lists the interface address and the prefix/length calculated from the configured address/length.

   Of the incorrect answers, the **show ipv6 interface brief** command lists the interface address but not the prefix/length of the connected subnet. The **show ipv6 address** command does not exist, but is simply rejected as an invalid command if attempted.
5. A. With an **ipv6 address** command configured for a global unicast address but without a link-local address configured with an **ipv6 address** command, the router calculates its link-local address on the interface based on its MAC address and EUI-64 rules. The router does not use the global unicast IPv6 address to calculate the link-local address.

The first half of the link-local address begins FE80:0000:0000:0000. The router then calculates the second half of the link-local address value by taking the MAC address (0200.0001.000A), injecting FF FE in the middle (0200.00FF.FE01.000A), and flipping the seventh bit (0000.00FF.FE01.000A).

6. B. FF02::1 is used by all IPv6 hosts on the link, FF02::5 is used by all OSPFv3 routers, and FF02::A is used by all EIGRPv6 routers. FF02::2 is used to send packets to all IPv6 routers on a link.

7. A. The router sends the NDP NS message to the solicited-node multicast address based on the unicast address of 2001:db8:1:1::1234:5678. To create the correct solicited-node address, take the last six hex digits (34:5678 in this case), and prepend FF02::1:FF. The correct answer is FF02::1:FF34:5678. The other answers are similar values that do not follow the correct solicited-node rules.

8. B and C. First, for G0/0/1, with the **ipv6 enable** command, the router enables IPv6, creating an LLA using EUI-64 rules for the interface ID. Those facts identify one correct and one incorrect answer.

   Then, for the answer interface G0/0/2 and the **ipv6 autoconfig** command, the command enables IPv6 with SLAAC. As a result, it generates an LLA, using EUI-64 rules, and generates a routable unicast address using SLAAC, again using EUI-64 rules for the interface ID. As a result, G0/0/2's LLA and global unicast address use the same interface ID values.

   Finally, for the answer about interface G0/0/3 and the **ipv6 address** subcommand, every interface that supports IPv6 must have an LLA. The router will again use EUI-64 to self-assign the interface ID portion of the interface's LLA.

## Chapter 28

1. B. PC1 needs to discover PC2's MAC address. Unlike IPv4, IPv6 does not use ARP, instead using NDP. Specifically, PC1 uses the NDP Neighbor Solicitation (NS) message to request that PC2 send back an NDP Neighbor Advertisement (NA). SLAAC relates to address assignment and not to discovering a neighbor's MAC address.

2. A and C. The NDP RA lists the router IPv6 address, the IPv6 prefixes known on the link, and the matching prefix lengths. The incorrect answers happen to list facts not included in the NDP RA message.

3. A. The **show ipv6 neighbors** command lists all IPv6 addresses of neighbors (both routers and hosts), plus their matching MAC addresses. It does not note which are routers, leaving that information for the **show ipv6 routers** command.

4. D. For the one correct answer, hosts can ask for (solicit) all routers to identify themselves by sending an NDP Router Solicitation (RS) message, with the routers sending back an NDP Router Advertisement (RA) message. For the incorrect answers, PC1 uses NDP Neighbor Solicitation (NS) but not for learning its default router IPv6 address. DAD is a function that uses NDP NS and NA messages, but its function does not include the discovery of the default router address. Finally, EUI-64 does not define a protocol or message, but is rather a convention to define 64-bit values to use as an IPv6 IID.

5. D. SLAAC gives the host a means to choose its unicast address. The host also uses NDP to learn its prefix length, plus the address(es) of any default routers. It then uses stateless DHCP to learn the addresses of the DNS server(s).
6. B and D. With SLAAC, the host learns the subnet prefix from a router using NDP RS/RA messages, and then the host builds the rest of the address (the interface ID). The host can randomly generate the interface ID or use modified EUI-64 rules. The host does not learn the interface ID from any other device, which helps make the process stateless because no other device needs to assign the host its full address.
7. A. The DHCPv6 protocol uses well-known multicast addresses, specifically FF02::1:2, for messages directed to DHCPv6 servers. However, because this multicast address has a link-local scope, those messages remain on the local LAN. A router connected to the LAN must implement a DHCPv6 relay agent function so that the router will replace the packet's FF02::1:2 multicast destination address with the unicast address of the DHCPv6 server. The routers then use normal unicast routing to forward the packet.

   For the two other incorrect answers, note that IPv6 does not use broadcast addresses at all. For instance, the all F's address in the answer is not an IPv6 broadcast address because there is no such thing in IPv6. Also, there is no mechanism to learn a DHCPv6 server's unicast address using NDP.
8. C. IPv6 routes on hosts and routers typically use the LLA of the next-hop device. For instance, PC1's default route would reference a router's LLA, not GUA. The **traceroute** command relies on those routes. However, the NDP messages that help the **traceroute** command identify how each router identifies the routable unicast address like the global unicast address. So, the **traceroute** command lists only GUAs in its output. Those facts determine the correct answer and rule out two answers as incorrect. One answer mentions the last line of **traceroute** output; because the command succeeded, that line lists the IPv6 address of the destination host rather than the address of the last router.

## Chapter 29

1. A and C. With an IPv6 address on a working interface, the router adds a connected route for the prefix (subnet) implied by the **ipv6 address** command. It also adds a local route (with a /128 prefix length) based on the unicast address. The router does not add a route based on the link-local address.
2. A and C. The two correct answers show the correct subnet ID (subnet prefix) and prefix length for the two connected subnets: 3111:1:1:1::/64 and 3222:2:2:2::/64. The answer with the /128 prefix length exists in a local route, but the **show ipv6 route connected** command does not list local routes. The other incorrect answer lists the entire IPv6 address with a /64 prefix length rather than the prefix ID.
3. A and B. All the answers list the same destination subnet prefix (2000:1:2:3::/64), which is the subnet prefix on the LAN to the right of Router R1. The differences exist in the forwarding instructions in each route.

   For the two commands that list both the outgoing interface (G0/0/1) and the next-hop address, both refer to the correct outgoing interface on Router R5. One refers to

the incorrect next-hop address—R1's own global unicast address (GUA), whereas the correct command lists neighboring Router R6's GUA (which ends in :6).

For the incorrect answer that lists only an outgoing interface, it lists the correct interface, and the router adds it to its routing table, but the route does not work. IPv6 static routes that refer to an outgoing Ethernet interface must also list a next-hop address for the router to know enough information to forward packets.

For the correct answer that lists only a next-hop GUA, it lists the correct GUA: R6's GUA on the link between R5 and R6 (which ends in :6).

**4.** B. All four answers show examples of commands that use a next-hop router IPv6 address.

Two incorrect answers list a next-hop address for R5's WAN interface (one global unicast, one link-local). A correct next-hop address reference on Router R1 should refer to an address on Router R6 instead.

For the two answers that list addresses on Router R6, the one that lists R6's global unicast address (2001:1:2:56::6) is correct. The command that lists R6's link-local address also requires R5's outgoing interface, so the router would reject the command in the answer that lists FE80::FF:FE00:6.

**5.** B. The **show ipv6 route** command, unlike the **show ip route** command, does not designate a gateway of last resort. Instead, it lists the default route like the other IPv6 routes, but with the special prefix/length of ::/0, which matches all possible IPv6 addresses.

For the other incorrect answers, the prefix of ::/128 would match the host address of all 0s, rather than matching all addresses. A route that matches prefix 2000::/3 will match all global unicast addresses, but it does not match all IPv6 addresses, so it would not be a default route.

**6.** B. The **ipv6 route** command in the question uses correct syntax, so the router will at least accept the command into the configuration. Of note, the command uses the administrative distance (AD) setting at the end, with a value of 200. As a result, Router R1 treats this route as a floating static route because its AD value (200) is greater than the default OSPF AD (110). As such, R1 continues to use the better OSPF-learned route based on the better (lower) AD and does not add the static route to the routing table.

**7.** C. The question asks what could have caused the conditions in the question. The user typed the command and pressed Enter, but the question did not say whether the router accepted the command. The question also tells us that the IPv6 routing table lists no routes for prefix/length (2001:DB8:8:8::/64). The goal then is to consider the answers to determine if any of those answers could result in no routes appearing for this prefix.

IOS will add a new static route to the IPv6 routing table if, when using a next-hop global unicast address, the router has a working route to reach that next-hop address and there is no better (lower administrative distance) route for the exact same subnet. So, the correct answer identifies one reason why the route would not appear.

The answer that mentions a better route with administrative distance of 110 is a valid reason for the static route not to appear. Still, the question states that no route for the subnet appears in the routing table, so clearly that competing route does not exist.

The other two incorrect answers mention the **ipv6 route** command. This command can use a link-local next-hop address but does not have to do so, showing the incorrect claim on one of those answers. For the other answer, when using a global unicast address as next-hop, the command does not also require an outgoing interface parameter, showing that answer as incorrect.

*This page intentionally left blank*

*This page intentionally left blank*

# GLOSSARY

## NUMERIC

**10/100** A short reference to an Ethernet NIC or switch port that supports speed of 10 Mbps and 100 Mbps.

**10/100/1000** A short reference to an Ethernet NIC or switch port that supports speeds of 10 Mbps, 100 Mbps, and 1000 Mbps (that is, 1 Gbps).

**10BASE-T** The 10-Mbps baseband Ethernet specification using two pairs of twisted-pair cabling (Categories 3, 4, or 5): one pair transmits data and the other receives data. 10BASE-T, which is part of the IEEE 802.3 specification, has a distance limit of approximately 100 m (328 feet) per segment.

**100BASE-T** A name for the IEEE Fast Ethernet standard that uses two-pair copper cabling, a speed of 100 Mbps, and a maximum cable length of 100 meters.

**1000BASE-T** A name for the IEEE Gigabit Ethernet standard that uses four-pair copper cabling, a speed of 1000 Mbps (1 Gbps), and a maximum cable length of 100 meters.

**2-way state** In OSPF, a neighbor state that implies that the router has exchanged Hellos with the neighbor and that all required parameters match.

**802.1Q** 802.1Q is the standard protocol for this tag. The most critical piece of information (for this discussion) in this tag is the VLAN ID.

## A

**AAA** Authentication, authorization, and accounting. Authentication confirms the identity of the user or device. Authorization determines what the user or device is allowed to do. Accounting records information about access attempts, including inappropriate requests.

**AAA server** A server that holds security information and provides services related to user login, particularly authentication (is the user who he says he is?), authorization (once authenticated, what do we allow the user to do?), and accounting (tracking the user).

**ABR** *See* Area Border Router.

**access interface** A LAN network design term that refers to a switch interface connected to end-user devices, configured so that it does not use VLAN trunking.

**access layer** In a campus LAN design, the switches that connect directly to endpoint devices (servers, user devices), and also connect into the distribution layer switches.

**access link** In Frame Relay, the physical serial link that connects a Frame Relay DTE device, usually a router, to a Frame Relay switch. The access link uses the same physical layer standards as do point-to-point leased lines.

**access point (AP)** A device that provides wireless service for clients within its coverage area or cell, with the AP connecting to both the wireless LAN and the wired Ethernet LAN.

**accounting** In security, the recording of access attempts. *See also* AAA.

**address block** A set of consecutive IPv4 addresses. The term is most often used for a classless prefix as defined by CIDR but can also refer to any subnet or IPv4 network.

**adjacent-layer interaction** The general topic of how, on one computer, two adjacent layers in a networking architectural model work together, with the lower layer providing services to the higher layer.

**administrative distance** In Cisco routers, a means for one router to choose between multiple routes to reach the same subnet when those routes were learned by different routing protocols. The lower the administrative distance, the better the source of the routing information.

**all-nodes multicast address** A specific IPv6 multicast address, FF02::1, with link-local scope, used to send packets to all devices on the link that support IPv6.

**all-routers multicast address** A specific IPv6 multicast address, FF02::2, with link-local scope, used to send packets to all devices that act as IPv6 routers on the local link.

**alternate port role** With RSTP, a port role in which the port acts as an alternative to a switch's root port, so that when the switch's root port fails, the alternate port can immediately take over as the root port.

**anycast address** An address shared by two or more hosts that exist in different parts of the network, so that by design, the routers will forward packets to the nearest of the two servers, allowing clients to communicate with the nearest such server, not caring which particular server with which the client communicates.

**APIPA** Automatic Private IP Addressing. A convention per RFC 3927 for a process and reserved set of IPv4 addresses (169.254.0.0/16) that hosts use when they need to use an IPv4 address but they fail to lease an IPv4 address using dynamic processes like DHCP.

**Area Border Router (ABR)** A router using OSPF in which the router has interfaces in multiple OSPF areas.

**ARP** Address Resolution Protocol. An Internet protocol used to map an IP address to a MAC address. Defined in RFC 826.

**ARP table** A list of IP addresses of neighbors on the same VLAN, along with their MAC addresses, as kept in memory by hosts and routers.

**ARPANET** The first packet-switched network, first created around 1970, which served as the predecessor to the Internet.

**ASBR** Autonomous System Border Router. A router using OSPF in which the router learns routes via another source, usually another routing protocol, exchanging routes that are external to OSPF with the OSPF domain.

**authentication** In security, the verification of the identity of a person or a process. *See also* AAA.

**authentication server (AS)** An 802.1x entity that authenticates users or clients based on their credentials, as matched against a user database. In a wireless network, a RADIUS server is an AS.

**authenticator** An 802.1x entity that exists as a network device that provides access to the network. In a wireless network, a WLC acts as an authenticator.

**authorization** In security, the determination of the rights allowed for a particular user or device. *See also* AAA.

**auto-MDIX** An Ethernet standard feature, introduced at the same time as Gigabit Ethernet in 1998, that senses whether the link uses the correct UTP cable pinout (straight-through or crossover), and automatically swaps the signals internally to make the link work if the wrong UTP cable pinout is used.

**autonegotiation** An IEEE standard mechanism (802.3u) with which two nodes can exchange messages for the purpose of choosing to use the same Ethernet standards on both ends of the link, ensuring that the link functions and functions well.

**autonomous system** An internetwork in the administrative control of one organization, company, or governmental agency, inside which that organization typically runs an interior gateway protocol (IGP).

**auxiliary port** A physical connector on a router that is designed to be used to allow a remote terminal, or PC with a terminal emulator, to access a router using an analog modem.

# B

**backbone area** In OSPFv2 and OSPFv3, the special area in a multiarea design, with all nonbackbone areas needing to connect to the backbone area, area 0.

**back-to-back link** A serial link between two routers, created without CSU/DSUs, by connecting a DTE cable to one router and a DCE cable to the other. Typically used in labs to build serial links without the expense of an actual leased line from the telco.

**backup designated router** An OSPF router connected to a multiaccess network that monitors the work of the designated router (DR) and takes over the work of the DR if the DR fails.

**backup port role** With RSTP, a port role in which the port acts as a backup to one of the switch's ports acting as a designated port. If the switch's designated port fails, the switch will use the backup port to immediately take over as the designated port.

**bandwidth** A reference to the speed of a networking link. Its origins come from earlier communications technology in which the range, or width, of the frequency band dictated how fast communications could occur.

**binary mask** An IPv4 subnet mask written as a 32-bit binary number.

**bitwise Boolean AND** A Boolean AND between two numbers of the same length in which the first bit in each number is ANDed, and then the second bit in each number, and then the third, and so on.

**blocking state** In STP, a port state in which no received frames are processed and the switch forwards no frames out the interface, with the exception of STP messages.

**Boolean AND** A math operation performed on a pair of one-digit binary numbers. The result is another one-digit binary number. 1 AND 1 yields 1; all other combinations yield a 0.

**BPDU** Bridge protocol data unit. The generic name for Spanning Tree Protocol messages.

**BPDU Guard** A Cisco switch feature that listens for incoming STP BPDU messages, disabling the interface if any are received. The goal is to prevent loops when a switch connects to a port expected to only have a host connected to it.

**BPDU Filter** A Cisco switch feature that uses the monitoring and filtering (discarding) of STP BPDUs to achieve goals, such as protecting against forwarding loops on PortFast ports and disabling STP by filtering all BPDU messages.

**bridge ID (BID)** An 8-byte identifier for bridges and switches used by STP and RSTP. It is composed of a 2-byte priority field followed by a 6-byte System ID field that is usually filled with a MAC address.

**bridge protocol data unit** *See* BPDU.

**broadcast address** Generally, any address that represents all devices, and can be used to send one message to all devices. In Ethernet, the MAC address of all binary 1s, or FFFF.FFFF.FFFF in hex. For IPv4, *see* subnet broadcast address.

**broadcast domain** A set of all devices that receive broadcast frames originating from any device within the set. Devices in the same VLAN are in the same broadcast domain.

**broadcast frame** An Ethernet frame sent to destination address FFFF.FFFF.FFFF, meaning that the frame should be delivered to all hosts on that LAN.

**broadcast network type** An OSPF interface setting, useful on links with more than two routers, resulting in an LSA flooding process managed by an elected designated router (DR).

**broadcast subnet** When subnetting a Class A, B, or C network, the one subnet in each classful network for which all subnet bits have a value of binary 1. The subnet broadcast address in this subnet has the same numeric value as the classful network's networkwide broadcast address.

**broken (state)** An STP port state on Cisco switches, used by Root Guard and Loop Guard as a method for STP to disable the use of the port. A port in the broken state does not forward or process received frames.

# C

**Channel-group** One term Cisco switches use to reference a bundle of links that are, in some respects, treated like a single link. Other similar terms include *EtherChannel* and *PortChannel*.

**CIDR** Classless interdomain routing. An RFC-standard tool for global IP address range assignment. CIDR reduces the size of Internet routers' IP routing tables, helping deal with the rapid growth of the Internet. The term *classless* refers to the fact that the summarized groups of networks represent a group of addresses that do not conform to IPv4 classful (Class A, B, and C) grouping rules.

**CIDR Block** A set of consecutive public IPv4 addresses whose size can be any power of 2. Used as an alternative to the original process with public IP networks of three set sizes.

**CIDR mask** Another term for a prefix mask, one that uses prefix or CIDR notation, in which the mask is represented by a slash (/) followed by a decimal number.

**CIDR notation** *See* prefix notation.

**Cisco Catalyst Edge Platform** The brand name created by Cisco for a product family. The products primarily act as routers, but they also create a platform to run many other networking services, including SD-WAN, SASE, and cloud features.

**cladding** In fiber-optic cabling, the second layer of the cable, surrounding the core of the cable, with the property of reflecting light back into the core.

**classful addressing** A concept in IPv4 addressing that defines a subnetted IP address as having three parts: network, subnet, and host.

**classful IP network** An IPv4 Class A, B, or C network; called a classful network because these networks are defined by the class rules for IPv4 addressing.

**classful routing protocol** Does not transmit the mask information along with the subnet number and therefore must consider Class A, B, and C network boundaries and perform auto-summarization at those boundaries. Does not support VLSM.

**classless addressing** A concept in IPv4 addressing that defines a subnetted IP address as having two parts: a prefix (or subnet) and a host.

**classless interdomain routing** The name of an RFC that defines several important features related to public IPv4 addressing: a global address assignment strategy to keep the size of IPv4 routing tables smaller, and the ability to assign public IPv4 addresses in sizes based on any prefix length.

**classless prefix** A range of public IPv4 addresses as defined by CIDR.

**classless prefix length** The mask (prefix length) used when defining a classless prefix.

**classless routing protocol** An inherent characteristic of a routing protocol, specifically that the routing protocol does send subnet masks in its routing updates, thereby removing any need to make assumptions about the addresses in a particular subnet or network, making it able to support VLSM and manual route summarization.

**CLI** Command-line interface. An interface that enables the user to interact with the operating system by entering commands and optional arguments.

**collision domain** A set of network interface cards (NIC) for which a frame sent by one NIC could result in a collision with a frame sent by any other NIC in the same collision domain.

**command-line interface** *See* CLI.

**configuration mode** A part of the Cisco IOS Software CLI in which the user can type configuration commands that are then added to the device's currently used configuration file (running-config).

**connected** The single-item status code listed by a **switch show interfaces status** command, with this status referring to a working interface.

**connected route** On a router, an IP route added to the routing table when the router interface is both up and has an IP address configured. The route is for the subnet that can be calculated based on the configured IP address and mask.

**console port** A physical socket on a router or switch to which a cable can be connected between a computer and the router/switch, for the purpose of allowing the computer to use a terminal emulator and use the CLI to configure, verify, and troubleshoot the router/switch.

**contiguous network** A network topology in which subnets of network X are not separated by subnets of any other classful network.

**convergence** The time required for routing protocols to react to changes in the network, removing bad routes and adding new, better routes so that the current best routes are in all the routers' routing tables.

**core** In fiber-optic cabling, the center cylinder of the cable, made of fiberglass, through which light passes.

**crossover cable** An Ethernet cable that swaps the pair used for transmission on one device to a pair used for receiving on the device on the opposite end of the cable. In 10BASE-T and 100BASE-TX networks, this cable swaps the pair at pins 1,2 to pins 3,6 on the other end of the cable, and the pair at pins 3,6 to pins 1,2 as well.

**CSMA/CD** Carrier sense multiple access with collision detection. A media-access mechanism in which devices ready to transmit data first check the channel for a carrier. If no carrier is sensed for a specific period of time, a device can transmit. If two devices transmit at once, a collision occurs and is detected by all colliding devices. This collision subsequently delays retransmissions from those devices for some random length of time.

# D

**data VLAN** A VLAN used by typical data devices connected to an Ethernet, like PCs and servers. Used in comparison to a voice VLAN.

**Database Description** An OSPF packet type that lists brief descriptions of the LSAs in the OSPF LSDB.

**DDN** *See* dotted-decimal notation.

**Dead Interval** In OSPF, a timer used for each neighbor. A router considers the neighbor to have failed if no Hellos are received from that neighbor in the time defined by the timer.

**decimal mask** An IPv4 subnet mask written in dotted-decimal notation; for example, 255.255.255.0.

**de-encapsulation** On a computer that receives data over a network, the process in which the device interprets the lower-layer headers and, when finished with each header, removes the header, revealing the next-higher-layer PDU.

**default gateway/default router** On an IP host, the IP address of some router to which the host sends packets when the packet's destination address is on a subnet other than the local subnet.

**default mask** The mask used in a Class A, B, or C network that does not create any subnets; specifically, mask 255.0.0.0 for Class A networks, 255.255.0.0 for Class B networks, and 255.255.255.0 for Class C networks.

**default route** On a router, the route that is considered to match all packets that are not otherwise matched by some more specific route.

**default VLAN** A reference to the default setting of 1 (meaning VLAN ID 1) on the **switchport access vlan** *vlan-id* **interface** subcommand on Cisco switches, meaning that by default, a port will be assigned to VLAN 1 if acting as an access port.

**designated port** In both STP and RSTP, a port role used to determine which of multiple interfaces on multiple switches, each connected to the same segment or collision domain, should forward frames to the segment. The switch advertising the lowest-cost Hello BPDU onto the segment becomes the DP.

**designated router** In OSPF, on a multiaccess network, the router that wins an election and is therefore responsible for managing a streamlined process for exchanging OSPF topology information between all routers attached to that network.

**DHCP** Dynamic Host Configuration Protocol. A protocol used by hosts to dynamically discover and lease an IP address, and learn the correct subnet mask, default gateway, and DNS server IP addresses.

**DHCP client** Any device that uses DHCP protocols to ask to lease an IP address from a DHCP server, or to learn any IP settings from that server.

**DHCP relay agent** The name of the router IOS feature that forwards DHCP messages from client to servers by changing the destination IP address from 255.255.255.255 to the IP address of the DHCP server.

**DHCP server** Software that waits for DHCP clients to request to lease IP addresses, with the server assigning a lease of an IP address as well as listing other important IP settings for the client.

**Dijkstra Shortest Path First (SPF) algorithm** The name of the algorithm used by link-state routing protocols to analyze the LSDB and find the least-cost routes from that router to each subnet.

**directed broadcast address** *See* subnet broadcast address.

**disabled port role** In STP, a port role for nonworking interfaces—in other words, interfaces that are not in a connect or up/up interface state. The reason can be due to administrative setting (shutdown) or interface failure.

**disabled state** In STP but not RSTP, the state to be used for interfaces in the disabled port role.

**discarding state** An RSTP interface state, which primarily replaces the STP blocking state, as used for interfaces with port roles other than designated or root ports. In this state, the switch does not forward data frames out the interface, nor does it process received frames other than RSTP messages.

**discontiguous network** A network topology in which subnets of network X are separated by subnets of some other classful network.

**distance vector** The logic behind the behavior of some interior routing protocols, such as RIP. Distance vector routing algorithms call for each router to send its entire routing table in each update, but only to its neighbors. Distance vector routing algorithms can be prone to routing loops but are computationally simpler than link-state routing algorithms.

**distribution layer** In a campus LAN design, the switches that connect to access layer switches as the most efficient means to provide connectivity from the access layer into the other parts of the LAN.

**DNS** Domain Name System. An application layer protocol used throughout the Internet for translating hostnames into their associated IP addresses.

**DNS Reply** In the Domain Name System (DNS), a message sent by a DNS server to a DNS client in response to a DNS Request, identifying the IP address assigned to a particular hostname or fully qualified domain name (FQDN).

**DNS Request** In the Domain Name System (DNS), a message sent by a DNS client to a DNS server, listing a hostname or fully qualified domain name (FQDN), asking the server to discover and reply with the IP address associated with that hostname or FQDN.

**DNS server** An application acting as a server for the purpose of providing name resolution services per the Domain Name System (DNS) protocol and worldwide system.

**DNS server list** A list of IP addresses of DNS servers, known to an IP host, used by a host when sending DNS name resolution requests.

**dotted-decimal notation (DDN)** The format used for IP version 4 addresses, in which four decimal values are used, separated by periods (dots).

**dual stack** A mode of operation in which a host or router runs both IPv4 and IPv6.

**duplex mismatch** On opposite ends of any Ethernet link, the condition in which one of the two devices uses full-duplex logic and the other uses half-duplex logic, resulting in unnecessary frame discards and retransmissions on the link.

**duplicate address detection (DAD)** A term used in IPv6 to refer to how hosts first check whether another host is using a unicast address before the first host uses that address.

# E

**EIGRP** Enhanced Interior Gateway Routing Protocol. An advanced version of IGRP developed by Cisco. Provides superior convergence properties and operating efficiency and combines the advantages of link-state protocols with those of distance vector protocols.

**EIGRP version 6** The version of the EIGRP routing protocol that supports IPv6, and not IPv4.

**electromagnetic interference (EMI)** The name of the effect in which electricity passes through one cable as normal, inducing a magnetic field outside the conductor. That magnetic field, if it passes through another conductor, like a nearby cable, induces new electrical current in the second cable, interfering with the use of electricity to transmit data on the second cable.

**enable mode** A part of the Cisco IOS CLI in which the user can use the most powerful and potentially disruptive commands on a router or switch, including the ability to then reach configuration mode and reconfigure the router.

**encapsulation** The placement of data from a higher-layer protocol behind the header (and in some cases, between a header and trailer) of the next-lower-layer protocol. For example, an IP packet could be encapsulated in an Ethernet header and trailer before being sent over an Ethernet.

**encryption** Applying a specific algorithm to data to alter the appearance of the data, making it incomprehensible to those who are not authorized to see the information.

**enterprise router** A term to describe the general role of a router as a router at a permanent site owned or leased by the enterprise, like an office building, manufacturing facility, branch office, or retail location. These sites typically have enough users to justify separate routers, switches, and wireless access points, and are more likely to justify private WAN services, in comparison to SOHO routers.

**equal-cost multipath (ECMP)** A term for a router's forwarding logic when it has more than one route for the same subnet with the same metric (cost).

**equal-cost route** When a routing protocol computes all possible routes to one subnet, the case for which multiple of those routes have the same metric (cost).

**error detection** The process of discovering whether a data-link level frame was changed during transmission. This process typically uses a Frame Check Sequence (FCS) field in the data-link trailer.

**error disabled** An interface state on LAN switches that can be the result of one of many security violations.

**error recovery** The process of noticing when some transmitted data was not successfully received and resending the data until it is successfully received.

**EtherChannel** A feature in which up to eight parallel Ethernet segments exist between the same two devices, each using the same speed. May be a Layer 2 EtherChannel, which acts like a single link for forwarding and Spanning Tree Protocol logic, or a Layer 3 EtherChannel, which acts like a single link for the switch's Layer 3 routing logic.

**EtherChannel Load Distribution** The logic used by switches when forwarding messages over EtherChannels by which the switch chooses the specific physical link out which the switch will forward the frame.

**Ethernet** A series of LAN standards defined by the IEEE, originally invented by Xerox Corporation and developed jointly by Xerox, Intel, and Digital Equipment Corporation.

**Ethernet address** A 48-bit (6-byte) binary number, usually written as a 12-digit hexadecimal number, used to identify Ethernet nodes in an Ethernet network. Ethernet frame headers list a

destination and source address field, used by the Ethernet devices to deliver Ethernet frames to the correct destination.

**Ethernet frame** A term referring to an Ethernet data-link header and trailer, plus the data encapsulated between the header and trailer.

**Ethernet Line Service (E-Line)** A specific carrier/metro Ethernet service defined by MEF (MEF.net) that provides a point-to-point topology between two customer devices, much as if the two devices were connected using an Ethernet crossover cable.

**Ethernet link** A generic term for any physical link between two Ethernet nodes, no matter what type of cabling is used.

**Ethernet over MPLS (EoMPLS)** A term referring specifically to how a service provider can create an Ethernet WAN service using an MPLS network. More generally, a term referring to Ethernet WAN services.

**Ethernet port** A generic term for the opening on the side of any Ethernet node, typically in an Ethernet NIC or LAN switch, into which an Ethernet cable can be connected.

**EtherType** Jargon that shortens the term *Ethernet Type*, which refers to the Type field in the Ethernet header. The Type field identifies the type of packet encapsulated inside an Ethernet frame.

**EUI-64** Literally, a standard for an extended unique identifier that is 64 bits long. Specifically for IPv6, a set of rules for forming a 64-bit identifier, used as the interface ID in IPv6 addresses, by starting with a 48-bit MAC address, inserting FFFE (hex) in the middle, and inverting the seventh bit.

**extended ping** An IOS command in which the **ping** command accepts many other options besides just the destination IP address.

# F

**Fast Ethernet** The common name for all the IEEE standards that send data at 100 megabits per second.

**fiber-optic cable** A type of cabling that uses glass fiber as a medium through which to transmit light.

**filter** Generally, a process or a device that screens network traffic for certain characteristics, such as source address, destination address, or protocol, and determines whether to forward or discard that traffic based on the established criteria.

**firewall** A device that forwards packets between the less secure and more secure parts of the network, applying rules that determine which packets are allowed to pass and which are not.

**flash memory** A type of read/write permanent memory that retains its contents even with no power applied to the memory, and uses no moving parts, making the memory less likely to fail over time.

**floating static route** A static IP route that uses a higher administrative distance than other routes, typically routes learned by a routing protocol. As a result, the router will not use the static route if the routing protocol route has been learned, but then use the static route if the routing protocol fails to learn the route.

**flood/flooding** The result of the LAN switch forwarding process for broadcasts and unknown unicast frames. Switches forward these frames out all interfaces, except the interface in which the frame arrived. Switches also flood multicasts by default, although this behavior can be changed.

**forward** To send a frame received in one interface out another interface, toward its ultimate destination.

**forward delay** An STP timer, defaulting to 15 seconds, used to dictate how long an interface stays in the listening state and the time spent in learning state. Also called the forward delay timer.

**forward route** From one host's perspective, the route over which a packet travels from that host to some other host.

**forwarding state** An STP and RSTP port state in which an interface operates unrestricted by STP.

**frame** A term referring to a data-link header and trailer, plus the data encapsulated between the header and trailer.

**Frame Check Sequence** A field in many data-link trailers used as part of the error-detection process.

**full duplex** Generically, any communication in which two communicating devices can concurrently send and receive data. In Ethernet LANs, the allowance for both devices to send and receive at the same time, allowed when both devices disable their CSMA/CD logic.

**full state** In OSPF, a neighbor state that implies that the two routers have exchanged the complete (full) contents of their respective LSDBs.

**full update** With IP routing protocols, the general concept that a routing protocol update lists all known routes.

**fully adjacent** In OSPF, a characterization of the state of a neighbor in which the two neighbors have reached the full state.

**fully adjacent neighbor** In OSPF, a neighbor with which the local router has also reached the OSPF full state, meaning that the two routers have exchanged their LSDBs directly with each other.

# G

**Gigabit Ethernet** The common name for all the IEEE standards that send data at 1 gigabit per second.

**global routing prefix** An IPv6 prefix that defines an IPv6 address block made up of global unicast addresses, assigned to one organization, so that the organization has a block of globally unique IPv6 addresses to use in its network.

**global unicast address** A type of unicast IPv6 address that has been allocated from a range of public globally unique IP addresses, as registered through IANA/ICANN, its member agencies, and other registries or ISPs.

# H

**half duplex** Generically, any communication in which only one device at a time can send data. In Ethernet LANs, the normal result of the CSMA/CD algorithm that enforces the rule that only one device should send at any point in time.

**HDLC** High-Level Data Link Control. A bit-oriented synchronous data-link layer protocol developed by the International Organization for Standardization (ISO).

**header** In computer networking, a set of bytes placed in front of some other data, encapsulating that data, as defined by a particular protocol.

**Hello (Multiple definitions)** 1) A protocol used by OSPF routers to discover, establish, and maintain neighbor relationships. 2) A protocol used by EIGRP routers to discover, establish, and maintain neighbor relationships. 3) In STP, refers to the name of the periodic message sourced by the root bridge in a spanning tree.

**Hello BPDU** The STP and RSTP message used for the majority of STP communications, listing the root's bridge ID, the sending device's bridge ID, and the sending device's cost with which to reach the root.

**Hello Interval** With OSPF and EIGRP, an interface timer that dictates how often the router should send Hello messages.

**Hello timer** In STP, the time interval at which the root switch should send Hello BPDUs.

**history buffer** In a Cisco router or switch, the function by which IOS keeps a list of commands that the user has used in this login session, both in EXEC mode and configuration mode. The user can then recall these commands for easier repeating or making small edits and issuing similar commands.

**hop count** The metric used by the RIP routing protocol. Each router in an IP route is considered a hop, so for example, if two other routers sit between a router and some subnet, that router would have a hop count of two for that route.

**host** Any device that uses an IP address.

**host address** The IP address assigned to a network card on a computer.

**host part** A term used to describe a part of an IPv4 address that is used to uniquely identify a host inside a subnet. The host part is identified by the bits of value 0 in the subnet mask.

**host route** A route with a /32 mask, which by virtue of this mask represents a route to a single host IP address.

**hostname** The alphanumeric name of an IP host.

**hub** A LAN device that provides a centralized connection point for LAN cabling, repeating any received electrical signal out all other ports, thereby creating a logical bus. Hubs do not interpret the electrical signals as a frame of bits, so hubs are considered to be Layer 1 devices.

# I

**IANA** The Internet Assigned Numbers Authority (IANA). An organization that owns the rights to assign many operating numbers and facts about how the global Internet works, including public IPv4 and IPv6 addresses. *See also* ICANN.

**ICANN** The Internet Corporation for Assigned Names and Numbers. An organization appointed by IANA to oversee the distributed process of assigning public IPv4 and IPv6 addresses across the globe.

**ICMP** Internet Control Message Protocol. A TCP/IP network layer protocol that reports errors and provides other information relevant to IP packet processing.

**ICMP echo reply** One type of ICMP message, created specifically to be used as the message sent by the ping command to test connectivity in a network. The ping command expects to receive these messages from other hosts, after the ping command first sends an ICMP echo request message to the host.

**ICMP echo request** One type of ICMP message, created specifically to be used as the message sent by the ping command to test connectivity in a network. The ping command sends these messages to other hosts, expecting the other host to reply with an ICMP echo reply message.

**IEEE** Institute of Electrical and Electronics Engineers. A professional organization that develops communications and network standards, among other activities.

**IEEE 802.11** The IEEE base standard for wireless LANs.

**IEEE 802.1Q** The IEEE standard VLAN trunking protocol. 802.1Q includes the concept of a native VLAN, for which no VLAN header is added, and a 4-byte VLAN header is inserted after the original frame's Type/Length field.

**IEEE 802.2** An IEEE LAN protocol that specifies an implementation of the LLC sublayer of the data-link layer.

**IEEE 802.3** A set of IEEE LAN protocols that specifies the many variations of what is known today as an Ethernet LAN.

**IEEE 802.3 AD** The IEEE standard for the functional equivalent of the Cisco-proprietary EtherChannel.

**IETF** The Internet Engineering Task Force. The IETF serves as the primary organization that works directly to create new TCP/IP standards.

**IGP** *See* interior gateway protocol.

**inactivity timer** For switch MAC address tables, a timer associated with each entry that counts time upward from 0 and is reset to 0 each time a switch receives a frame with the same MAC address. The entries with the largest timers can be removed to make space for additional MAC address table entries.

**infrastructure mode** The operating mode of an AP that is providing a BSS for wireless clients.

**Integrated Services Router (ISR)** Cisco's long-running term for several different model series of Enterprise-class routers, intended mostly for use as enterprise routers and some use as SOHO routers. ISR routers first serve as routers but, depending on the family or specific model, support all current types of WAN connections (private and Internet), LAN switching ports, Wireless APs, VPNs, and other integrated functions supported in a single device.

**interface bandwidth** In OSPF, the numerator in the calculation of an interface's default OSPF cost metric, calculated as the interface bandwidth divided by the reference bandwidth.

**Interface ID** The ending (rightmost) portion of the structure of an IPv6 address, usually 64 bits long.

**interface-local scope** A concept in IPv6 for which packets sent to an address using this scope should not physically exit the interface, keeping the packet inside the sending host.

**interior gateway protocol (IGP)** A routing protocol designed to be used to exchange routing information inside a single autonomous system.

**interior routing protocol** A synonym of interior gateway protocol. *See* interior gateway protocol.

**Internal Border Gateway Protocol (iBGP)** The use of BGP between two routers in the same ASN, with different rules compared to External BGP (eBGP).

**internal router** In OSPF, a router with all interfaces in the same nonbackbone area.

**Internetwork Operating System** The operating system (OS) of Cisco routers and switches, which provides the majority of a router's or switch's features, with the hardware providing the remaining features.

**IOS** *See* Internetwork Operating System.

**IOS XE** A Cisco operating system (OS) with a modern Linux-based multitasking software architecture used as the OS for many enterprise-class LAN switch and router products.

**IP** Internet Protocol. The network layer protocol in the TCP/IP stack, providing routing and logical addressing standards and services.

**IP address (IP version 4)** In IP version 4 (IPv4), a 32-bit address assigned to hosts using TCP/IP. Each address consists of a network number, an optional subnetwork number, and a host number. The network and subnetwork numbers together are used for routing, and the host number is used to address an individual host within the network or subnetwork.

**IP address (IP version 6)** In IP version 6 (IPv6), a 128-bit address assigned to hosts using TCP/IP. Addresses use different formats, commonly using a routing prefix, subnet, and interface ID, corresponding to the IPv4 network, subnet, and host parts of an address.

**IP network** *See* classful IP network.

**IP packet** An IP header, followed by the data encapsulated after the IP header, but specifically not including any headers and trailers for layers below the network layer.

**IP routing table** *See* routing table.

**IP subnet** Subdivisions of a Class A, B, or C network, as configured by a network administrator. Subnets allow a single Class A, B, or C network to be used instead of multiple networks, and still allow for a large number of groups of IP addresses, as is required for efficient IP routing.

**IP version 4** Literally, the version of the Internet Protocol defined in an old RFC 791, standardized in 1980, and used as the basis of TCP/IP networks and the Internet for over 30 years.

**IP version 6** A newer version of the Internet Protocol defined in RFC 2460, as well as many other RFCs, whose creation was motivated by the need to avoid the IPv4 address exhaustion problem.

**IPv4** *See* IP version 4.

**IPv4 address exhaustion** The process by which the public IPv4 addresses, available to create the Internet, were consumed through the 1980s until today, with the expectation that eventually the world would run out of available IPv4 addresses.

**IPv6** *See* IP version 6.

**IPv6 address scope** The concept of how far an IPv6 packet should be forwarded by hosts and routers in an IPv6 network. Includes interface-local, link-local, site-local, and organization-local scopes.

**IPv6 administrative distance** In Cisco routers, a means for one router to choose between multiple IPv6 routes to reach the same subnet when those routes were learned by different routing protocols. The lower the administrative distance, the better the source of the routing information.

**IPv6 host route** A route with a /128 mask, which by virtue of this mask represents a route to a single host IPv6 address.

**IPv6 local route** A route added to an IPv6 router's routing table for the router's interface IP address, with a /128 mask, which by virtue of this mask represents a route to only that router's IPv4 address.

**IPv6 neighbor table** The IPv6 equivalent of the ARP table. A table that lists IPv6 addresses of other hosts on the same link, along with their matching MAC addresses, as typically learned using Neighbor Discovery Protocol (NDP).

**ISO** International Organization for Standardization. An international organization that is responsible for a wide range of standards, including many standards relevant to networking. The ISO developed the OSI reference model, a popular networking reference model.

# K–L

**keepalive** A proprietary feature of Cisco routers in which the router sends messages on a periodic basis as a means of letting the neighboring router know that the first router is still alive and well.

**known unicast frame** An Ethernet frame whose destination MAC address is listed in a switch's MAC address table, so the switch will forward the frame out the one port associated with that entry in the MAC address table.

**LACP** Link Aggregation Control Protocol is a messaging protocol defined by the IEEE 802.3ad standard that enables two neighboring devices to realize that they have multiple parallel links connecting to each other and then to decide which links can be combined into an EtherChannel.

**Layer 2 EtherChannel (L2 EtherChannel)** An EtherChannel that acts as a switched port (that is, not a routed port), and as such, is used by a switch's Layer 2 forwarding logic. As a result, the Layer 2 switch lists the Layer 2 EtherChannel in switch MAC address tables, and when forwarding a frame based on one of these MAC table entries, the switch balances traffic across the various ports in the Layer 2 EtherChannel.

**Layer 3 EtherChannel (L3 EtherChannel)** An EtherChannel that acts as a routed port (that is, not a switched port), and as such, is used by a switch's Layer 3 forwarding logic. As a result, the Layer 3 switch lists the Layer 3 EtherChannel in various routes in the switch's IP routing table, with the switch balancing traffic across the various ports in the Layer 3 EtherChannel.

**Layer 3 protocol** A protocol that has characteristics like OSI Layer 3, which defines logical addressing and routing. IPv4 and IPv6 are Layer 3 protocols.

**Layer 3 switch** *See* multilayer switch.

**learning** The process used by switches for discovering MAC addresses, and their relative location, by looking at the source MAC address of all frames received by a bridge or switch.

**learning state** In STP, a temporary port state in which the interface does not forward frames, but it can begin to learn MAC addresses from frames received on the interface.

**leased line** A serial communications circuit between two points, provided by some service provider, typically a telephone company (telco). Because the telco does not sell a physical cable between the two endpoints, instead charging a monthly fee for the ability to send bits between the two sites, the service is considered to be a leased service.

**link state** A classification of the underlying algorithm used in some routing protocols. Link-state protocols build a detailed database that lists links (subnets) and their state (up, down), from which the best routes can then be calculated.

**link-local address (LLA)** A unicast IPv6 address that begins FE80, used on each IPv6-enabled interface, used for sending packets within the attached link by applying a link-local scope.

**link-local multicast address** A multicast IPv6 address that begins with FF02, with the fourth digit of 2 identifying the scope as link-local, to which devices apply a link-local scope.

**link-local scope** With IPv6 multicasts, a term that refers to the parts (scope) of the network to which a multicast packet can flow, with link-local referring to the fact that the packet stays on the subnet in which it originated.

**link-state advertisement (LSA)** In OSPF, the name of the data structure that resides inside the LSDB and describes in detail the various components in a network, including routers and links (subnets).

**link-state database (LSDB)** In OSPF, the data structure in RAM of a router that holds the various LSAs, with the collective LSAs representing the entire topology of the network.

**Link-State Request** An OSPF packet used to ask a neighboring router to send a particular LSA.

**Link-State Update** An OSPF packet used to send an LSA to a neighboring router.

**listening state** A temporary STP port state that occurs immediately when a blocking interface must be moved to a forwarding state. The switch times out MAC table entries during this state. It also ignores frames received on the interface and doesn't forward any frames out the interface.

**LLC** Logical Link Control. The higher of the two sublayers of the data-link layer defined by the IEEE. Synonymous with IEEE 802.2.

**local broadcast IP address** IPv4 address 255.255.255.255. A packet sent to this address is sent as a data-link broadcast, but only flows to hosts in the subnet into which it was originally sent. Routers do not forward these packets.

**local mode** The default mode of a Cisco lightweight AP that offers one or more functioning BSSs on a specific channel.

**local route** A route added to an IPv4 router's routing table for the router's interface IP address, with a /32 mask, which by virtue of this mask represents a route to only that router's IPv4 address.

**local username** A username (with matching password), configured on a router or switch. It is considered local because it exists on the router or switch, and not on a remote server.

**logical address** A generic reference to addresses as defined by Layer 3 protocols that do not have to be concerned with the physical details of the underlying physical media. Used mainly to contrast these addresses with data-link addresses, which are generically considered to be physical addresses because they differ based on the type of physical medium.

**longest prefix match** When a router's IP routing table has more than one route that matches a packet's destination address, the choice to use the matching route with the longest mask (the mask with the largest number of binary 1s in the mask).

**Loop Guard** A complex Cisco switch mechanism that protects against STP loops in a specific common case. For switches whose switch-to-switch links settle into either a root port or an alternate port role, as expected per the STP design, Loop Guard disables use of those ports if normal STP operation attempts to assign them the designated port role.

**LSA** *See* link-state advertisement.

**LSDB** *See* link-state database.

# M

**MAC** Media Access Control. The lower of the two sublayers of the data-link layer defined by the IEEE. Synonymous with IEEE 802.3 for Ethernet LANs.

**MAC address** A standardized data-link layer address that is required for every device that connects to a LAN. Ethernet MAC addresses are 6 bytes long and are controlled by the IEEE. Also known as a hardware address, a MAC layer address, and a physical address.

**MAC address table** A table of forwarding information held by a Layer 2 switch, built dynamically by listening to incoming frames and used by the switch to match frames to make decisions about where to forward the frame.

**MaxAge** In STP, a timer that states how long a switch should wait when it no longer receives Hellos from the root switch before acting to reconverge the STP topology. Also called the MaxAge timer.

**media access control (MAC) layer** A low-level function performed as part of Layer 2; in wireless networks, this function can be divided between a wireless LAN controller and a light-weight AP to form a split-MAC architecture.

**message of the day** One type of login banner that can be defined on a Cisco router or switch.

**metric** A unit of measure used by routing protocol algorithms to determine the best route for traffic to use to reach a particular destination.

**Modified EUI-64** *See* EUI-64.

**multiarea** In OSPFv2 and OSPFv3, a design that uses multiple areas.

**multiarea OSPF** In OSPFv2 and OSPFv3, a design that uses more than one area within one OSPF domain (typically a single company).

**multicast IP address** A class D IPv4 address. When used as a destination address in a packet, the routers collectively work to deliver copies of the one original packet to all hosts who have previously registered to receive packets sent to that particular multicast address.

**multilayer switch** A LAN switch that can also perform Layer 3 routing functions. The name comes from the fact that this device makes forwarding decisions based on logic from multiple OSI layers (Layers 2 and 3).

**multimode fiber** A type of fiber cable that works well with transmitters like LEDs that emit multiple angles of light into the core of the cable; to accommodate the multiple angles of incident, the cable has a larger core in comparison to single-mode fiber cables.

# N

**name resolution** The process by which an IP host discovers the IP address associated with a hostname, often involving sending a DNS request to a DNS server, with the server supplying the IP address used by a host with the listed hostname.

**name server** A server connected to a network that resolves network names into network addresses.

**NAT** Network Address Translation. A mechanism for reducing the need for globally unique IP addresses. NAT allows an organization with addresses that are not globally unique to connect to the Internet, by translating those addresses into public addresses in the globally routable address space.

**native VLAN** The one VLAN ID on any 802.1Q VLAN trunk for which the trunk forwards frames without an 802.1Q header.

**neighbor** In routing protocols, another router with which a router decides to exchange routing information.

**Neighbor Advertisement (NA)** A message defined by the IPv6 Neighbor Discovery Protocol (NDP), used to declare to other neighbors a host's MAC address. Sometimes sent in response to a previously received NDP Neighbor Solicitation (NS) message.

**Neighbor Discovery Protocol (NDP)** A protocol that is part of the IPv6 protocol suite, used to discover and exchange information about devices on the same subnet (neighbors). In particular, it replaces the IPv4 ARP protocol.

**Neighbor Solicitation (NS)** A message defined by the IPv6 Neighbor Discovery Protocol (NDP), used to ask a neighbor to reply with a Neighbor Advertisement, which lists the neighbor's MAC address.

**neighbor table** For OSPF and EIGRP, a list of routers that have reached neighbor status.

**network** A collection of computers, printers, routers, switches, and other devices that can communicate with each other over some transmission medium.

**network address** *See* network number.

**network broadcast address** In IPv4, a special address in each classful network that can be used to broadcast a packet to all hosts in that same classful network. Numerically, the address has the same value as the network number in the network part of the address and all 255s in the host octets; for example, 10.255.255.255 is the network broadcast address for classful network 10.0.0.0.

**network ID** A number that identifies an IPv4 network, using a number in dotted-decimal notation (like IP addresses); a number that represents any single Class A, B, or C IP network.

**network interface card (NIC)** A computer card, sometimes an expansion card and sometimes integrated into the motherboard of the computer, that provides the electronics and other functions to connect to a computer network. Today, most NICs are specifically Ethernet NICs, and most have an RJ-45 port, the most common type of Ethernet port.

**Network LSA** In OSPF, a type of LSA that a designated router (DR) creates for the network (subnet) for which the DR is helping to distribute LSAs.

**network number** A number that uses dotted-decimal notation like IP addresses, but the number itself represents all hosts in a single Class A, B, or C IP network.

**network part** The portion of an IPv4 address that is either 1, 2, or 3 octets/bytes long, based on whether the address is in a Class A, B, or C network.

**network route** A route for a classful network.

**networking model** A generic term referring to any set of protocols and standards collected into a comprehensive grouping that, when followed by the devices in a network, allows all the devices to communicate. Examples include TCP/IP and OSI.

**next-hop router** In an IP route in a routing table, part of a routing table entry that refers to the next IP router (by IP address) that should receive packets that match the route.

**NIC** *See* network interface card.

**NVRAM** Nonvolatile RAM. A type of random-access memory (RAM) that retains its contents when a unit is powered off.

# O

**on-link prefix** An IPv6 subnet prefix, advertised by a router in an NDP Router Advertisement (RA) message, that identifies to on-link hosts a subnet considered to exist on the local link.

**Organization-local scope** A concept in IPv6 for which packets sent to an address using this scope should be forwarded by routers inside the organization but not over any links connected to other organizations or over links connected to the Internet.

**OSI** Open System Interconnection reference model. A network architectural model developed by the ISO. The model consists of seven layers, each of which specifies particular network functions, such as addressing, flow control, error control, encapsulation, and reliable message transfer.

**OSPF** Open Shortest Path First. A popular link-state IGP that uses a link-state database and the Shortest Path First (SPF) algorithm to calculate the best routes to reach each known subnet.

**OSPF neighbor** A local router's attitude toward a second router that resides on a common subnet, when both use OSPF and use appropriate OSPF settings so that the routers will choose to exchange routing information with each other.

**OSPF priority** An OSPF interface setting that serves as the first comparison point in the designated router (DR) election process, with the router interface with the highest priority winning the election.

**OSPF router-id** In OSPF, the 32-bit number, normally shown in dotted-decimal notation but also allowed to be listed as an integer, intended as a unique identifier for each OSPF router in an OSPF domain.

**OSPF version 2** The version of the OSPF routing protocol that supports IPv4, and not IPv6, and has been commonly used for over 20 years.

**OSPF version 3** The version of the OSPF routing protocol that originally supported only IPv6, and not IPv4, but now supports IPv4 through the use of address family configuration.

**outgoing interface** In an IP route in a routing table, part of a routing table entry that refers to the local interface out which the local router should forward packets that match the route.

**overlapping subnets** An (incorrect) IP subnet design condition in which one subnet's range of addresses includes addresses in the range of another subnet.

# P

**packet** A logical grouping of bytes that includes the network layer header and encapsulated data, but specifically does not include any headers and trailers below the network layer.

**PAgP** Port Aggregation Protocol (PAgP) is a messaging protocol defined by Cisco that enables two neighboring devices to realize that they have multiple parallel links connecting to each other and then to decide which links can be combined into an EtherChannel.

**parallel detection** The term for the branch of IEEE autonegotiation steps that applies to a device that uses autonegotiation but the device on the other end of the link does not.

**partial mesh** A network topology in which more than two devices could physically communicate but, by choice, only a subset of the pairs of devices connected to the network is allowed to communicate directly.

**passive interface** With a routing protocol, a router interface for which the routing protocol is enabled on the interface, but for which the routing protocol does not send routing protocol messages out that interface.

**patch cable** An Ethernet cable, usually short, that connects from a device's Ethernet port to a wall plate or switch. With wiring inside a building, electricians prewire from the wiring closet to each cubicle or other location, with a patch cable connecting the short distance from the wall plate to the user device.

**periodic update** With routing protocols, the concept that the routing protocol advertises routes in a routing update on a regular periodic basis. This is typical of distance vector routing protocols.

**permanent IPv6 address** When using IPv6 SLAAC, a host creates an address with an infinite preferred and valid lifetime, making the address permanent, using the address when listening for incoming connections to any services running on that host.

**ping** An Internet Control Message Protocol (ICMP) echo message and its reply; ping often is used in IP networks to test the reachability of a network device.

**pinout** The documentation and implementation of which wires inside a cable connect to each pin position in any connector.

**point-to-point network type** An OSPF interface setting, useful on links in a point-to-point topology with only two routers, resulting in flooding directly between the routers without the use of a designated router (DR).

**port** In TCP and UDP, a number that is used to uniquely identify the application process that either sent (source port) or should receive (destination port) data. In LAN switching, another term for *switch interface*.

**PortChannel** One term Cisco switches use to reference a bundle of links that are, in some respects, treated like a single link. Other similar terms include *EtherChannel* and *Channel-group*.

**PortFast** A switch STP feature in which a port is placed in an STP forwarding state as soon as the interface comes up, bypassing the listening and learning states. This feature is meant for ports connected to end-user devices.

**preferred lifetime** In the context of IPv6 addresses, a timer applied to a SLAAC-created address defining how long the address is preferred, meaning the host uses the address for new and existing application flows.

**Prefix (prefix ID)** In both IPv4 and IPv6, this term refers to the number that identifies a group of IPv4 or IPv6 addresses, respectively. Another term for *subnet identifier*.

**prefix discovery** IPv6 neighbor discovery protocol (NDP) function, specifically part of the Router Advertisement (RA) message, in which the router supplies a list of IPv6 subnet prefixes and prefix lengths that exist on the local link.

**prefix length** In IPv6, the number of bits in an IPv6 prefix.

**prefix mask** A term to describe an IPv4 subnet mask when represented as a slash (/) followed by a decimal number. The decimal number is the number of binary 1s in the mask.

**prefix notation (IP version 4)** A shorter way to write a subnet mask in which the number of binary 1s in the mask is simply written in decimal. For example, /24 denotes the subnet mask with 24 binary 1 bits in the subnet mask. The number of bits of value binary 1 in the mask is considered to be the prefix length.

**primary root** This term refers to the switch configured with the primary keyword on the **spanning-tree vlan x root {primary | secondary}** command. At time of configuration, this command causes the switch to choose a new priority setting that makes the switch become the root switch in the network.

**private addresses** IP addresses in several Class A, B, and C networks that are set aside for use inside private organizations. These addresses, as defined in RFC 1918, are not routable through the Internet.

**private IP network** Any of the IPv4 Class A, B, or C networks as defined by RFC 1918, intended for use inside a company but not used as public IP networks.

**Protocol Type field** A field in a LAN header that identifies the type of header that follows the LAN header. Includes the DIX Ethernet Type field, the IEEE 802.2 DSAP field, and the SNAP protocol Type field.

**public IP address** An IP address that is part of a registered network number, as assigned by an Internet Assigned Numbers Authority (IANA) member agency, so that only the organization to which the address is registered is allowed to use the address. Routers in the Internet should have routes allowing them to forward packets to all the publicly registered IP addresses.

**public IP network** Any IPv4 Class A, B, or C network assigned for use by one organization only, so that the addresses in the network are unique across the Internet, allowing packets to be sent through the public Internet using the addresses.

**PVST+** An STP option in Cisco switches that creates an STP instance per VLAN. Cisco proprietary.

# Q–R

**quartet** A term used in this book, but not in other references, to refer to a set of four hex digits in an IPv6 address.

**RADIUS server** An authentication server used with 802.1x to authenticate wireless clients.

**RAM** Random-access memory. A type of volatile memory that can be read and written by a microprocessor.

**Rapid PVST+** An STP option in Cisco switches that creates an RSTP instance per VLAN. Cisco proprietary.

**Rapid Spanning Tree Protocol (RSTP)** Defined in IEEE 802.lw. Defines an improved version of STP that converges much more quickly and consistently than STP (802.Id).

**reference bandwidth** In OSPF, a configurable value for the OSPF routing process, used by OSPF when calculating an interface's default OSPF cost metric, calculated as the interface's bandwidth divided by the reference bandwidth.

**Regional Internet Registry** An organization (five globally) that receives allocations of public IPv4 addresses from IANA and then manages that address space in their major geographic region, performing public address allocations to ISPs and assignments directly to companies that use the addresses.

**repeater** A device that repeats or retransmits signals it receives, effectively expanding the wireless coverage area.

**resident subnet** Each IP subnet contains a number of unicast IP addresses; that subnet is the resident subnet for each of those addresses—that is, the subnet in which those addresses reside.

**reverse route** From one host's perspective, for packets sent back to the host from another host, the route over which the packet travels.

**RFC** Request For Comments. A document used as the primary means for communicating information about the TCP/IP protocols. Some RFCs are designated by the Internet Architecture Board (IAB) as Internet standards, and others are informational. RFCs are available online from numerous sources, including http://www.rfc-editor.org.

**RIP** Routing Information Protocol. An interior gateway protocol (IGP) that uses distance vector logic and router hop count as the metric. RIP version 2 (RIPv2) replaced the older RIP version 1 (RIPv1), with RIPv2 providing more features, including support for VLSM.

**RIR** *See* Regional Internet Registry.

**RJ-45** A popular type of cabling connector used for Ethernet cabling. It is similar to the RJ-11 connector used for telephone wiring in homes in the United States. RJ-45 allows the connection of eight wires.

**ROAS** *See* Router-on-a-Stick.

**rollover cable** For connections to the RJ-45 console port rather than USB console port of a Cisco device, the type of cable used between the user PC and the console port. The UTP rollover cable uses a rollover pinout with eight pins, connecting pin 1 to pin 8, pin 2 to pin 7, and so on.

**ROM** Read-only memory. A type of nonvolatile memory that can be read but not written to by the microprocessor.

**ROMMON** A shorter name for ROM Monitor, which is a low-level operating system that can be loaded into Cisco routers for several seldom-needed maintenance tasks, including password recovery and loading a new IOS when flash memory has been corrupted.

**root bridge** *See* root switch.

**root cost** The STP cost from a nonroot switch to reach the root switch, as the sum of all STP costs for all ports out which a frame would exit to reach the root.

**Root Guard** A Cisco switch feature that protects against unexpected new root switches. When enabled on an interface, IOS uses normal STP rules except to disable the use of the port after receiving a superior BPDU.

**root port role** In STP and RSTP, the one port on a nonroot switch in which the least-cost Hello is received. Switches put root ports in a forwarding state.

**root switch** In STP and RSTP, the switch that wins the election by virtue of having the lowest bridge ID and, as a result, sends periodic Hello BPDUs (default, 2 seconds).

**routed port** A reference to the default logic on an interface on a router, such as de-encapsulating the Layer 3 packet from incoming Layer 2 frames and passing the packet to the Layer 3 forwarding logic of the device.

**routed protocol** A protocol that defines packets that can be routed by a router. Examples of routed protocols include IPv4 and IPv6.

**Router Advertisement (RA)** A message defined by the IPv6 Neighbor Discovery Protocol (NDP), used by routers to announce their willingness to act as an IPv6 router on a link. These can be sent in response to a previously received NDP Router Solicitation (RS) message.

**router ID (RID)** In EIGRP and OSPF, a 32-bit number, written in dotted-decimal notation, that uniquely identifies each router.

**router LSA** In OSPF, a type of LSA that a router creates to describe itself and the networks connected to it.

**Router-on-a-Stick (ROAS)** Jargon to refer to the Cisco router feature of using VLAN trunking on an Ethernet interface, which then allows the router to route packets that happen to enter the router on that trunk and then exit the router on that same trunk, just on a different VLAN.

**Router Solicitation (RS)** A message defined by the IPv6 Neighbor Discovery Protocol (NDP), used to ask any routers on the link to reply, identifying the router, plus other configuration settings (prefixes and prefix lengths).

**routing protocol** A set of messages and processes with which routers can exchange information about routes to reach subnets in a particular network. Examples of routing protocols include Enhanced Interior Gateway Routing Protocol (EIGRP), Open Shortest Path First (OSPF), and Routing Information Protocol (RIP).

**routing table** A list of routes in a router, with each route listing the destination subnet and mask, the router interface out which to forward packets destined to that subnet, and as needed, the next-hop router's IP address.

**routing update** A generic reference to any routing protocol's messages in which it sends routing information to a neighbor.

**RSTP** *See* Rapid Spanning Tree Protocol.

**running-config file** In Cisco IOS switches and routers, the name of the file that resides in RAM, holding the device's currently used configuration.

# S

**same-layer interaction** The communication between two networking devices for the purposes of the functions defined at a particular layer of a networking model, with that communication happening by using a header defined by that layer of the model. The two devices set values in the header, send the header and encapsulated data, with the receiving devices interpreting the header to decide what action to take.

**secondary root** This term refers to the switch configured with the secondary keyword on the **spanning-tree vlan x root {primary | secondary}** command. At time of configuration, this command causes the switch to set its base priority to 28,762.

**Secure Shell (SSH)** A TCP/IP application layer protocol that supports terminal emulation between a client and server, using dynamic key exchange and encryption to keep the communications private.

**segment** In TCP, a term used to describe a TCP header and its encapsulated data (also called an L4PDU). Also in TCP, the process of accepting a large chunk of data from the application layer and breaking it into smaller pieces that fit into TCP segments. In Ethernet, a segment is either a single Ethernet cable or a single collision domain (no matter how many cables are used).

**shared Ethernet** An Ethernet that uses a hub, or even the original coaxial cabling, that results in the devices having to take turns sending data, sharing the available bandwidth.

**shortest path first (SPF) algorithm** The name of the algorithm used by link-state routing protocols to analyze the LSDB and find the least-cost routes from that router to each subnet.

**single-area OSPF** In OSPFv2 and OSPFv3, a design that uses a single area within one OSPF domain (typically a single company).

**single-mode fiber** A type of fiber cable that works well with transmitters like lasers that emit a single angle of light into the core of the cable, allowing for a smaller core in comparison to multimode fiber cables.

**site-local scope** A concept in IPv6 for which packets sent to an address using this scope should be forwarded by routers, but not forwarded over WAN links to other sites.

**SOHO router** A term to describe the general role of a router that exists as part of the enterprise network but resides at an employee's home or at a smaller business site, possibly with a

short-term lease compared to larger enterprise sites. These sites typically have few devices, so it makes sense to use one device that integrates routing, switches, wireless, and other features into a single device (the SOHO router) and are more likely to justify Internet access as the primary WAN access method.

**solicited-node multicast address** A type of IPv6 multicast address, with link-local scope, used to send packets to all hosts in the subnet that share the same value in the last six hex digits of their unicast IPv6 addresses. Begins with FF02::1:FF00:0/104.

**Spanning Tree Protocol (STP)** A protocol defined by IEEE standard 802.ID. Allows switches and bridges to create a redundant LAN, with the protocol dynamically causing some ports to block traffic, so that the bridge/switch forwarding logic will not cause frames to loop indefinitely around the LAN.

**SSH** *See* Secure Shell.

**standard access list** A list of IOS global configuration commands that can match only a packet's source IP address, for the purpose of deciding which packets to discard and which to allow through the router.

**star topology** A network topology in which endpoints on a network are connected to a common central device by point-to-point links.

**startup-config file** In Cisco IOS switches and routers, the name of the file that resides in NVRAM memory, holding the device's configuration that will be loaded into RAM as the running-config file when the device is next reloaded or powered on.

**stateful DHCPv6** A term used in IPv6 to contrast with stateless DHCP. Stateful DHCP keeps track of which clients have been assigned which IPv6 addresses (state information).

**stateless address autoconfiguration (SLAAC)** A feature of IPv6 in which a host or router can be assigned an IPv6 unicast address without the need for a stateful DHCP server.

**stateless DHCPv6** A term used in IPv6 to contrast with stateful DHCP. Stateless DHCP servers don't lease IPv6 addresses to clients. Instead, they supply other useful information, such as DNS server IP addresses, but with no need to track information about the clients (state information).

**static access interface** A LAN network design term, synonymous with the term *access interface*, but emphasizing that the port is assigned to one VLAN as a result of static configuration rather than through some dynamic process.

**static route** An IP route on a router created by the user configuring the details of the route on the local router.

**STP** Shielded twisted-pair. This type of cabling has a layer of shielded insulation to reduce electromagnetic interference (EMI).

**straight-through cable** In Ethernet, a cable that connects the wire on pin 1 on one end of the cable to pin 1 on the other end of the cable, pin 2 on one end to pin 2 on the other end, and so on.

**subinterface** One of the virtual interfaces on a single physical interface.

**subnet** Subdivisions of a Class A, B, or C network, as configured by a network administrator. Subnets allow a single Class A, B, or C network to be used instead of multiple networks, and still allow for a large number of groups of IP addresses, as is required for efficient IP routing.

**subnet address** *See* subnet number.

**subnet broadcast address** A special address in each IPv4 subnet, specifically the largest numeric address in the subnet, designed so that packets sent to this address should be delivered to all hosts in that subnet.

**subnet ID (IPv4)** *See* subnet number.

**subnet ID (IPv6)** The number that represents the IPv6 subnet. Also known as the IPv6 prefix, or more formally as the subnet-router anycast address.

**subnet ID (prefix ID)** *See* subnet number.

**subnet mask** A 32-bit number that numerically describes the format of an IP address, by representing the combined network and subnet bits in the address with mask bit values of 1, and representing the host bits in the address with mask bit values of 0.

**subnet number** In IPv4, a dotted-decimal number that represents all addresses in a single subnet. Numerically, the smallest value in the range of numbers in a subnet, reserved so that it cannot be used as a unicast IP address by a host.

**subnet part** In a subnetted IPv4 address, interpreted with classful addressing rules, one of three parts of the structure of an IP address, with the subnet part uniquely identifying different subnets of a classful IP network.

**subnet prefix** The term for a number that represents an IPv6 subnet.

**subnet router anycast address** A special anycast address in each IPv6 subnet, reserved for use by routers as a way to send a packet to any router on the subnet. The address's value in each subnet is the same number as the subnet ID.

**subnet zero** An alternative term for *zero subnet*. *See* zero subnet.

**subnetting** The process of subdividing a Class A, B, or C network into smaller groups called subnets.

**summary LSA** In OSPFv2, a type of LSA, created by an Area Border Router (ABR), to describe a subnet in one area in the database of another area.

**superior BPDU** An STP bridge protocol data unit (BPDU) that lists a better (lower) root bridge ID (BID) as compared to the current bridge. A switch that receives a superior BPDU under normal conditions would begin using the new switch with the lower BID as the root switch.

**switch** A network device that filters, forwards, and floods Ethernet frames based on the destination address of each frame.

**switched Ethernet** An Ethernet that uses a switch, and particularly not a hub, so that the devices connected to one switch port do not have to contend to use the bandwidth available on

another port. This term contrasts with *shared Ethernet*, in which the devices must share bandwidth, whereas switched Ethernet provides much more capacity, as the devices do not have to share the available bandwidth.

**switched port** A reference to the default logic on a Layer 2 switch port, such as learning the source MAC address of received frames and passing incoming frames to the Layer 2 forwarding logic of the device.

**switched virtual interface (SVI)** Another term for any VLAN interface in a Cisco switch. *See also* VLAN interface.

**system ID extension** The term for the formatting applied to the original 16-bit STP priority field to break it into a 4-bit priority field and a 12-bit VLAN ID field.

# T

**TCP** Transmission Control Protocol. A connection-oriented transport layer TCP/IP protocol that provides reliable data transmission.

**TCP/IP** Transmission Control Protocol/Internet Protocol. A common name for the suite of protocols developed by the U.S. Department of Defense in the 1970s to support the construction of worldwide internetworks. TCP and IP are the two best-known protocols in the suite.

**telco** A common abbreviation for *telephone company*.

**Telnet** The standard terminal-emulation application layer protocol in the TCP/IP protocol stack. Telnet is used for remote terminal connection, enabling users to log in to remote systems and use resources as if they were connected to a local system. Telnet is defined in RFC 854.

**temporary IPv6 address** When using IPv6 SLAAC, a host creates an address with a relatively short preferred and valid lifetime, usually days long for each, making the address temporary. The host uses the temporary address for outgoing application connections.

**topology database** The structured data that describes the network topology to a routing protocol. Link-state and balanced hybrid routing protocols use topology tables, from which they build the entries in the routing table.

**trace** Short for traceroute. A program available on many systems that traces the path that a packet takes to a destination. It is used mostly to troubleshoot routing problems between hosts.

**traceroute** A program available on many systems that traces the path that a packet takes to a destination. It is used mostly to debug routing problems between hosts.

**trailer** In computer networking, a set of bytes placed behind some other data, encapsulating that data, as defined by a particular protocol. Typically, only data-link layer protocols define trailers.

**transceiver** A term formed from the words *transmitter* and *receiver*. The hardware used to both send (transmit) energy over some communications medium (e.g., wires in a cable), as well as to process received energy signals to interpret as a series of 1s and 0s.

**trunk** In campus LANs, an Ethernet segment over which the devices add a VLAN header that identifies the VLAN in which the frame exists.

**trunk interface** A switch interface configured so that it operates using VLAN trunking (either 802.1Q or ISL).

**trunking** Also called *VLAN trunking*. A method (using either the Cisco ISL protocol or the IEEE 802.1Q protocol) to support multiple VLANs, allowing traffic from those VLANs to cross a single link.

**trunking administrative mode** The configured trunking setting on a Cisco switch interface, as configured with the switchport mode command.

**trunking operational mode** The current behavior of a Cisco switch interface for VLAN trunking.

**twisted-pair** Transmission medium consisting of two insulated wires, with the wires twisted around each other in a spiral. An electrical circuit flows over the wire pair, with the current in opposite directions on each wire, which significantly reduces the interference between the two wires.

# U

**UDP** User Datagram Protocol. Connectionless transport layer protocol in the TCP/IP protocol stack. UDP is a simple protocol that exchanges datagrams without acknowledgments or guaranteed delivery.

**ULA global ID** The unique local address (ULA) for one organization uses a /48 prefix, composed of a set hex FD in the first 8 bits, with a 10-hex digit (40 bit) global ID, which should be generated by a pseudo-random algorithm.

**unicast address** Generally, any address in networking that represents a single device or interface, instead of a group of addresses (as would be represented by a multicast or broadcast address).

**unicast IP address** An IP address that represents a single interface. In IPv4, these addresses come from the Class A, B, and C ranges.

**unidirectional link** A condition on fiber optic links in which one of the two required fibers fails in a way that also results in both attached switches leaving their interfaces in a working (connected) state—when, in reality, the link can pass frames only in a single direction.

**unique local address** A type of IPv6 unicast address meant as a replacement for IPv4 private addresses.

**unknown unicast frame** An Ethernet frame whose destination MAC address is not listed in a switch's MAC address table, so the switch must flood the frame.

**up and up** Jargon referring to the two interface states on a Cisco IOS router or switch (line status and protocol status), with the first "up" referring to the line status and the second "up" referring to the protocol status. An interface in this state should be able to pass data-link frames.

**update timer** The time interval that regulates how often a routing protocol sends its next periodic routing updates. Distance vector routing protocols send full routing updates every update interval.

**user mode** A mode of the user interface to a router or switch in which the user can type only nondisruptive EXEC commands, generally just to look at the current status, but not to change any operational settings.

**UTP** Unshielded twisted-pair. A type of cabling, standardized by the Telecommunications Industry Association (TIA), that holds twisted pairs of copper wires (typically four pair) and does not contain any shielding from outside interference.

# V

**valid lifetime** In the context of IPv6 addresses, a timer applied to a SLAAC-created address defining how long the address is valid, meaning the host can continue to support existing application flows using the address.

**variable-length subnet mask (VLSM)** The capability to specify a different subnet mask for the same Class A, B, or C network number on different subnets. VLSM can help optimize available address space.

**virtual LAN (VLAN)** A group of devices, connected to one or more switches, with the devices grouped into a single broadcast domain through switch configuration. VLANs allow switch administrators to separate the devices connected to the switches into separate VLANs without requiring separate physical switches, gaining design advantages of separating the traffic without the expense of buying additional hardware.

**virtual private network (VPN)** The process of securing communication between two devices whose packets pass over some public and unsecured network, typically the Internet. VPNs encrypt packets so that the communication is private, and authenticate the identity of the endpoints.

**VLAN** *See* virtual LAN.

**VLAN interface** A configuration concept inside Cisco switches, used as an interface between IOS running on the switch and a VLAN supported inside the switch, so that the switch can assign an IP address and send IP packets into that VLAN.

**VLAN Trunking Protocol (VTP)** A Cisco-proprietary messaging protocol used between Cisco switches to communicate configuration information about the existence of VLANs, including the VLAN ID and VLAN name.

**voice VLAN** A VLAN defined for use by IP Phones, with the Cisco switch notifying the phone about the voice VLAN ID so that the phone can use 802.1Q frames to support traffic for the phone and the attached PC (which uses a data VLAN).

**VoIP** Voice over IP. The transport of voice traffic inside IP packets over an IP network.

**VTP** *See* VLAN Trunking Protocol.

**VTP transparent mode** One of three VTP operational modes. Switches in transparent mode can configure VLANs, but they do not tell other switches about the changes, and they do not learn about VLAN changes from other switches.

# W

**WAN** *See* wide-area network.

**web server** Software, running on a computer, that stores web pages and sends those web pages to web clients (web browsers) that request the web pages.

**wide-area network (WAN)** A part of a larger network that implements mostly OSI Layer 1 and 2 technology, connects sites that typically sit far apart, and uses a business model in which a consumer (individual or business) must lease the WAN from a service provider (often a telco).

**Wi-Fi Alliance** An organization formed by many companies in the wireless industry (an industry association) for the purpose of getting multivendor certified-compatible wireless products to market in a more timely fashion than would be possible by simply relying on standardization processes.

**wildcard mask** The mask used in Cisco IOS ACL commands and OSPF and EIGRP network commands.

**wired LAN** A local-area network (LAN) that physically transmits bits using cables, often the wires inside cables. A term for local-area networks that use cables, emphasizing the fact that the LAN transmits data using wires (in cables) instead of wireless radio waves. *See also* wireless LAN.

**wireless LAN** A local-area network (LAN) that physically transmits bits using radio waves. The name "wireless" compares these LANs to more traditional "wired" LANs, which are LANs that use cables (which often have copper wires inside).

# Z

**zero subnet** For every classful IPv4 network that is subnetted, the one subnet whose subnet number has all binary 0s in the subnet part of the number. In decimal, the zero subnet can be easily identified because it is the same number as the classful network number.



# Index

## Symbols


? (question mark) command, 101


## Numbers


2-way states, 550

10/100 ports, autonegotiation, 161, 162

10/100/1000 ports, autonegotiation, 161, 162, 163

10BASE-T
- pin pairs, 48
- UTP pinouts, 46–47

100BASE-T
- pin pairs, 48
- UTP pinouts, 46–47

802.1Q trunking, 194–195, 454, 457–464

802.3 10-Gbps fiber standards, 52

802.3 Ethernet header and trailer fields, 53–54


## A


AAA servers. *See* authentication servers

abbreviating IPv6 addresses, 647, 648–649

ABR (Area Border Routers), 556

access interfaces, VLAN, 197–198

access switches, 259

accessing
- Cisco IOS XE CLI, 91–101
  - *in privileged (enable) mode, 96–98*
  - *with SSH, 96*
  - *with Telnet, 96*
  - *in user mode, 96–98*
- CLI
  - *from Cisco Catalyst switches, 90*
  - *Cisco IOS XE CLI, 91–101*
  - *password security, 98–99*
  - *from WebUI, 99–101*
- router CLI, 414–415
- user mode, switches
  - *external authentication servers, 141–142*
  - *remote access, IPv4, 146–147*
  - *remote access, SSH, 142–145*

acknowledgement messages, 489

AD (Administrative Distance)
- default distances, 623
- IPv6 routing
  - *connected routes, 727*
  - *static routes, 739–741*

additional content (exams), 751

address masks, 496

addressing
- Ethernet addressing, 54–55
- MAC addresses, 54–55

**addressing, IPv4**
address exhaustion timeline, IPv4, 641–642
APIPA IP addresses (169.254.x.x), 490
ARP, 80–81
classes, 328–330
classful addressing, 348
classless addressing, 348
configuring, connected routes, 435–439
default masks, 330
dividing with subnet masks, 346–348
formats, 330
hosts
*DHCP, 488–497*
*Linux settings, 504–506*
*macOS settings, 502–504*
*settings, 497–499*
*troubleshooting settings, 506–509*
*Windows IP settings, 499–502*
LLA, 680–684
network broadcast addresses, 331, 333–334
router interfaces, 418–419
routing, 76–77
*grouping addresses, 76*
*header fields, 76–77*
*subnetting, 76*
subnetting, 76, 308
*broadcast addresses, 358, 359, 361*
*ID/addresses, 308, 360*
*ID/addresses, binary math analysis, 362–366*
*ID/addresses, finding in difficult subnet masks, 369–371*
*range of usable addresses, 361, 367*
troubleshooting, 522–524
unicast IP addresses, 358
**addressing, IPv6**
abbreviating addresses, 647, 648–649
address formats/conventions, 646
address prefix lengths, 649
anycast addresses, 672–689
configuration summary, 690–691
DAD, 705
DHCP, 706–710
dual stacks, 671
expanding addresses, 648–649
full (unabbreviated) addresses, 646–647
GUA, 655–664
hexadecimal/binary conversion chart, 647
on-link prefixes, 703–704
LLA, 680–684
loopback addresses, 689
modified EUI-64, 674–679, 682–683
multicast addresses, 684–688
NA, 699–702, 705
NDP, 698–699
*NA, 699–702, 705*
*NS, 699–702, 705*
*RA, 702–704*
*RS, 702–704*
*summary, 705–706*
next-hop addresses, static network routes, 730–735
NS, 699–702, 705
permanent IPv6 addresses, 712
prefix discovery, 699
private addresses, 655–664
public addresses, 656–657
RA, 702–704
RS, 702–704

scopes, 686–687
SLAAC, 710–714
solicited-node multicast addresses, 684, 687–688
special addresses, 680
subnet prefixes, 649–652
temporary IPv6 addresses, 712–714
testing connectivity, 716–718
troubleshooting, 714–718
ULA, 656, 664–666
unicast IP addresses, 670–671
*dynamic, 679–680*
*static, 671–679*
unspecified addresses, 689
verifying host connectivity, 718–719
**adjacencies, OSPF neighbor, 612–617**
**adjacent neighbors, 553**
**adjacent-layer interaction, TCP/IP networking models, 25–26**
**advanced distance vector (hybrid) algorithms, 543**
**advertisements (network), IP routing, 78**
**aging MAC addresses, clearing, 126–127**
**alternate ports, RSTP, 241–243**
**anycast IPv6 addresses, IPv6 routing, 672–689**
**APIPA IP addresses (169.254.x.x), 490**
**application layer, TCP/IP networking models, 23**
HTTP, 23–24
web pages, accessing, 23
**areas, OSPF, 555–557**
multi-area OSPF, 555–557, 574–575
*single-area OSPF, 555–557, 565–566*
**ARP (Address Resolution Protocol)**
network layer, 80–81
proxy, static routes, 443
tables, routers, 439–440
**ASN (AS Numbers), 541**
**assigning**
subnets to different locations, 320–321
VLAN to interfaces, 197–201, 213
**authentication servers, switch security, 141–142**
**auto-MDIX (Automatic Medium-Dependent Interface Crossover), 49, 170–172**
**autonegotiation**
10/100 ports, 161, 162
10/100/1000 ports, 161, 162, 163
full duplex transmissions, 161
half duplex transmissions, 161
LAN hubs, 165
parallel detection logic, 163–164
routers, Ethernet interface autonegotiation, 420–422
switches, 161–169
**AS (Autonomous Systems), 542–543**
**autostate setting, SVI, 467–469**
**Aux ports, 423**

## B

**backbone areas, 556**
**backbone routers, 556**
**backup ports, RSTP, 241, 244**
**backups, BDR, 552–553**
**bandwidth**
interface bandwidth, 599, 600
reference bandwidth, 599, 600–601
router serial interfaces, 423

**basic passwords, configuring, 137–138**
**BDR (Backup Designated Routers), 552–553**
**BDR/DR elections, OSPF, 590–592**
**Bellman-Ford (distance vector) algorithms, 543**
**BID (Bridge ID), 229–231, 261–262**
**binary/hexadecimal conversion chart, 647**
**binary masks, 340**
DDN mask conversions, 342–344
prefix mask conversions, 341–342
**binary math analysis, subnet ID/address, 362–366**
**blocking state**
STP, 120–121, 238–239
STP/RSTP, 225, 229
**Boolean AND/OR operations, subnet masks, 367**
**BPDU (Bridge Protocol Data Units)**
STP/RSTP, 230–231, 236–237
superior BPDU, 251
tunneling, 266
**BPDU Filter, 248–250, 274–278**
**BPDU Guard, 248, 269–274**
**bridging tables. *See* MAC addresses**
**broadcast addresses, subnet, 358, 359, 361**
**broadcast domains, 191–192**
**broadcast flags, 489**
**broadcast frames, flooding, 119–120**
**broadcast networks, 586–592**
**broadcast storms, 225–227**
**broken state, STP, 251**
**building Ethernet LAN**
data-link layer protocols, 53–57
fiber builds, 50–53
UTP builds, 43–49

# C

**cabling**
auto-MDIX, 49
Cisco IOS XE CLI connections, 93–94
crossover cabling, 48–49
Ethernet cabling
*auto-MDIX, 49*
*pinouts, 48*
fiber cabling
*802.3 10-Gbps fiber standards, 52*
*cladding, 50–51*
*components of, 50–51*
*Ethernet LAN builds, 50–53*
*multimode fiber, 51–53*
*optical transmitters, 50–51*
*single-mode fiber, 51–53*
pinouts, choosing, 48–49
rollover cabling, console connections, 93–94
straight-through cabling, pinouts, 47
UTP
*10BASE-T pinouts, 46–47*
*100BASE-T pinouts, 46–47*
*1000BASE-T pinouts, 49*
*cabling comparisons, 52–53*
*data transmission, 43–44*
*Ethernet LAN builds, 43–49*
*Ethernet links, 44–46*
**CAM tables. *See* MAC addresses**
**campus LAN, 114**
**Catalyst Edge Platform, 411**
**certification exams**
additional content, 751
Cisco Certification Roadmap, 751–752

release dates/information, 751–752, 754
study plans, 753–754
updates, 751–752
**changing interface states with STP, 238–239**
**channel-group configuration command, 281–284**
**choosing**
cabling pinouts, 48–49
designated ports for LAN segments, 234–235
dynamic ranges for subnetting, 321–322
IP networks, subnetting, 314
masks, subnetting, 314–318
root ports, 232–234
static ranges for subnetting, 321–322
subnet masks, to meet requirements, 380–386
**CIDR blocks**
classful networks, 326–328
subnet masks and, 351–352
**CIDR masks.** ***See*** **prefix masks**
**Cisco Catalyst switches, 90**
9200 model switches, 91, 94
CLI, accessing, 90
**Cisco Certification Roadmap, 751–752**
**Cisco IOS**
enterprise routers, 409–410
help features, 101–102
software configuration, 103–104
**Cisco IOS XE**
CLI
*accessing, 91–101*
*accessing in privileged (enable) mode, 96–98*
*accessing in user mode, 96–98*
*accessing with SSH, 96*
*accessing with Telnet, 96*
*cabling console connections, 93–94*
enterprise routers, 409–410
terminal emulator configurations, 95
**Cisco Meraki switches, 90**
**Cisco Nexus switches, 90**
**cladding, 50–51**
**classes, IPv4 networks, 328–330**
**classful addressing, 348**
**classful networks**
CIDR blocks, 326–328
classes, 328–330
network broadcast addresses, 331, 333–334
network ID, 331–334
number of hosts, determining, 331
practicing with, 334–335
public IP networks, setting context of, 326–328
subnetting, 311–314
WHOIS records, 328
**classless addressing, 348**
**clearing aging MAC addresses, 126–127**
**CLI (Command-Line Interface), 88**
accessing
*from Cisco Catalyst switches, 90*
*password security, 98–99*
*from WebUI, 99–101*
Cisco IOS XE CLI
*accessing, 91–101*
*accessing in privileged (enable) mode, 96–98*
*accessing in user mode, 96–98*
*accessing with SSH, 96*
*accessing with Telnet, 96*
*cabling console connections, 93–94*

*terminal emulator configurations, 95*
configuration mode, 103–104
*common configuration modes, 105–106*
*contexts, 104–106*
*interface command, 104–105*
*navigating between modes, 105–106*
*submodes, 104–106*
EXEC commands, 98
EXEC mode, 103–104
exec-timeout command, 152–153
help features, 101–102
history buffer commands, 151–152
logging synchronous command, 152–153
no ip domain-lookup command, 152–153
router CLI, accessing, 414–415
security, 134–135
software configuration, 103–104
**clock rates, router serial interfaces, 423**
**collisions**
Ethernet hubs, 58–59
frame transmissions, 58–59
**commands**
channel-group configuration command, 281–284
com? command, 101
command ? command, 101
command parm <Tab> command, 102
command parm? command, 101
command parm1? command, 102
context-setting commands, 104–106
copy running-config startup-config command, 109
debug command, 103
description command, 172–173
editing, 102
EXEC commands, 98
exec-timeout command, 152–153
help commands, CLI, 101–102
history buffer commands, 151–152
hostname command, 109
interface command, 104–105
interface range command, 172–173
ip address command, 436–439
logging synchronous command, 152–153
network commands, OSPFv2, 564–565
*single-area OSPF configuration, 565–566*
*wildcard matching, 566–568*
no ip domain-lookup command, 152–153
ping command
*basics, 513–514*
*extended ping, testing LAN neighbors, 520–521*
*extended ping, testing reverse routes, 517–519*
*IP addresses, 522–524*
*IPv4 routing, 512, 513–524*
*IPv6 connectivity, 716–718*
*name resolution, 522–524*
*problem isolation, 513*
*strategies/results, 514–521*
*testing LAN neighbors, 519–520*
*testing longer routes, 514–517*
*testing reverse routes, 517–519*
*testing WAN neighbors, 521*
productivity commands, 153
question mark (?) command, 101
recalling, 102
rejected commands example, enable (privileged) mode, 97–98

reload command, switch configurations, 109
show command, 103
show interfaces command, 124
shutdown command, 172–173
traceroute command
*IPv4 routing, 512, 524–527*
*IPv6 connectivity, 716–718*
tracert command
*basics, 524–525*
*extended traceroute, 526–527*
*operation of, 525–526*
*problem isolation, 524*
verification commands, OSPFv2, 568–573
**configuration mode, CLI, 103–104**
common configuration modes, 105–106
contexts, 104–106
interface command, 104–105
navigating between modes, 105–106
submodes, 103–106
**configuring**
Cisco IOS software, 103–104
data VLAN, 209–212
DHCP, 493–494
*client configurations, 496–497*
*switches, client configurations, 495–496*
DHCP Relay, 494
embedded switch ports, 478–480
EtherChannel
*dynamic EtherChannels, 284–287*
*interface configuration consistency, 287–289*
*Layer 2 EtherChannel, 281–284*
IPv4
*addressing, connected routes, 435–439*
*switches, 149–151*
Layer 3 Switch SVI, 464–466
OSPF
*interface configurations, 575–578*
*RID, 573–574*
*single-area OSPF configuration, 565–566*
passwords, basic, 137–138
RID, OSPF, 573–574
ROAS, 458–461
RSTP, 259
static routes, 440–443
switches
*autonegotiation, 161–169*
*common configuration modes, 105–106*
*copying switch configurations, 109*
*duplex, manually setting, 169–170*
*erasing switch configurations, 109*
*interface configurations, 172–180*
*IPv4 configurations, 149–151*
*removing interface configurations, 174–176*
*speed, manually setting, 169–170*
*storing switch configurations, 106–108*
terminal emulators, 95
VLAN, 197–201
*data VLAN, 209–212*
*voice VLAN, 209–212*
**connected routes**
IPv4, 426–427
IPv6, 724–727
**connectivity, high-speed Internet, 18–19**
**console passwords, 135–139**

**context-setting commands, 104–106**
**contexts, CLI configuration, 104–106**
**convergence**
dynamic routing protocols, 541, 542
STP, convergence management, 237–238
**converting subnet masks, 340–345**
**copy running-config startup-config command, 109**
**copying switch configurations, 109**
**costs**
default port costs, 235–236
OSPFv2 metrics, 599–601
port costs, STP, 266–268
root costs, STP, 266–268
**crossover cabling, 48–49**

## D

**DAD (Duplicate Address Detection), 705**
**data center LAN, 114**
**data encapsulation**
steps of, 31–32
terminology, 31–32
**data transmission**
single-mode fiber cabling, 51
UTP, 43–44
**data VLAN**
configuring, 209–212
verifying, 209–212
**data-link layer**
encapsulation, 74
Ethernet LAN
*consistency, 42–43*
*protocols, 53–57*
TCP/IP networking models, 29–31
**data-only messages, HTTP, 24**
**DDN (Dotted-Decimal Notation), 367**
binary mask conversions, 342–344
magic numbers, 370
prefix mask conversions, 344
subnet masks, 318, 341
**Dead intervals, 602–604, 614–615**
**debug command, 103**
**default gateways**
DHCP, 496
hosts, 148–149
switches, 148–149
**default masks, IP addresses, 330**
**default port costs, 235–236**
**default routers, 73**
**default routes**
OSPFv2, 597–599
static routes, 443–445
**description command, switch interface configuration, 172–173**
**design view, subnetting, 303–304**
**designated ports, choosing LAN segments, 234–235**
**designated switches, STP/RSTP, 229**
**detecting errors, Ethernet, 56–57**
**DHCP (Dynamic Host Configuration Protocol), 488**
acknowledgement messages, 489
address masks, 496
APIPA IP addresses (169.254.x.x), 490
broadcast flags, 489
concepts, 488–490
configuring, 493–494
*client configurations, 496–497*
*DHCP Relay, 494*
*switches, client configurations, 495–496*
default gateways, 496
discover messages, 489–490

IPv6 hosts, 706–710
offer messages, 489–490
request messages, 489
servers
*preconfiguration, 493*
*stored information, 492–493*
stateful DHCPv6, 706–710
stateless DHCPv6, 707, 710, 711–712
subnetting, 321–322
switches, learning IP addresses, 150
**DHCP Relay, 490–492, 494**
**difficult subnet masks, 368, 369–371**
**Dijkstra SPF algorithm**
LSDB, 547–548
route calculation, 553–555
**disabled state, STP, 239**
**disabled/undefined VLAN, 213–215**
**discarding state, RSTP, 243**
**discover messages, 489–490**
**distance vector (Bellman-Ford) algorithms, 543**
**distribution switches as root switches, 259**
**dividing IP addresses with subnet masks, 346–348**
**DNS (Domain Name System)**
name resolution requests, 79–80, 152–153
network layer, 79–80
server lists, 500
switches, 149, 152–153
**DP (Designated Ports), STP/RSTP, 229**
**DR/BDR elections, OSPF, 590–592**
**dual stacks, IPv6 addressing, 671**
**duplex mismatches, switch interface configurations, 177–178**
**duplex of switches, manually setting, 169–170**
**duplex transmissions, autonegotiation, 161**
**dynamic auto mode, trunking, 203–204**
**dynamic desirable mode, trunking, 203**
**dynamic EtherChannels, configuring, 284–287**
**dynamic ranges, choosing for subnetting, 321–322**
**dynamic routing protocols**
convergence, 541, 542
EIGRP, 543–545
features, 540
functions, 541–542
ICMPv6, 642
IGP, 542–545
IGRP, 543
NDP, 642
OSPF, 543–545
*AD, 623*
*areas, 555–557*
*BDR, 552–553*
*broadcast networks, 586–592*
*concepts, 546*
*design terminology, 556*
*Dijkstra SPF algorithm, 553–555*
*DR/BDR elections, 590–592*
*ECMP routing, 621–622*
*equal-cost routes, 621–622*
*Hello packets, 548–549*
*interface configurations, 575–578*
*IP forwarding, 625–628*
*IP routing table, 628–630*
*longest prefix matching, 625–630*
*LSA, 546–548, 557–560*
*LSDB, 546, 547–548, 550–553*
*MTU, troubleshooting, 618*
*multi-area OSPF, 574–575*
*neighbors, 546, 548–553*
*neighbors, requirements, 611–612*

*neighbors, troubleshooting adjacencies, 612–617*
*neighbors, troubleshooting missing routes, 619–621*
*neighbors, troubleshooting priority 0, 618–619*
*network types, troubleshooting, 618*
*operation of, 546*
*point-to-point networks, 586, 592–594*
*priority, 590–592*
*RID, 548*
*route calculation, 553–555*
*route selection, 621–630*
*routing, troubleshooting, 618–621*
*shutting down, 615–617*
*topologies, 546–547*
OSPFv2
*Dead intervals, 602–604, 614–615*
*default routes, 597–599*
*Hello intervals, 602–604, 614–615*
*implementing with interface subcommands, 576*
*interface bandwidth, 599, 600*
*metrics (cost), 599–601*
*network commands, 564–568*
*passive interfaces, 594–597*
*reference bandwidth, 599, 600–601*
*verification commands, 568–573*
OSPFv3, 642
path selection, 540
RIPv2, 543–545
**dynamic unicast IP addresses, IPv6 routing, 679–680**

# E

**easy subnet masks, 367–369**
**echo requests, ICMP, 81–82**
**ECMP routing, OSPF, 621–622**
**edge ports, RSTP, 245**
**editing, commands, 102**
**EGP (Exterior Gateway Protocols), 542–543**
**EIGRP (Enhanced Interior Gateway Routing Protocol), 543–545**
**EIGRPv6 (EIGRP version 6), 645–646**
**election process, root switches, 231**
**embedded switch ports**
configuring, 478–480
identifying, 481–482
verifying routing, 480–481
**EMI (Electromagnetic Interference), 43–44**
**emulators (terminal), configuring, 95**
**enable (privileged) mode**
Cisco IOS XE CLI access, 96–98
rejected commands example, 97–98
**enable passwords, 135–139**
**enable secret passwords, 137**
**encapsulating**
data
*steps of, 31–32*
*terminology, 31–32*
data-link layer, 74
IP packets, 68–69
network layer, 74
**end-users, high-speed Internet connections, 18–19**
**enterprise LAN, 40–41**
**enterprise networks (internetworks), 19, 304**
**enterprise routers, installing, 408–409, 412**

Catalyst Edge Platform, 411
IOS, 409–410
IOS XE, 409–410
ISR, 410–411
OS, 409–410
**equal-cost routes, 621–622**
**erasing switch configurations, 109**
**error detection, Ethernet, 56–57**
**error recovery, TCP/IP networking models, 25**
**EtherChannel**
dynamic EtherChannels, configuring, 284–287
interface configuration consistency, 287–289
Layer 2 EtherChannel, configuring, 281–284
load distribution, 289–291
STP, 246
**Ethernet**
10BASE-T, UTP pinouts, 46–47
100BASE-T, UTP pinouts, 46–47
1000BASE-T pinouts, UTP pinouts, 49
addressing, 54–55
cabling, auto-MDIX, 49
crossover cabling, pinouts, 48
error detection, 56–57
Ethernet Type (EtherType) fields, 56
FCS fields, 56–57
fiber cabling, 52–53
frame format, 53–54
frame transmissions
*collisions, 58–59*
*full duplex transmissions, 57–58, 59*
*half duplex transmissions, 58–60*
*hubs, 57–60*
*switches, 57–60*
frames, 30
header fields, 53–54, 67
hubs
*collisions, 58–59*
*frame transmissions, 57–60*
*half duplex transmissions, 58–60*
LAN, 36
*data-link layer consistency, 42–43*
*data-link layer protocols, 53–57*
*fiber builds, 50–53*
*frame transmissions, 57–60*
*physical layer standards, 41–42*
*UTP builds, 43–49*
LAN, switching
*analyzing, 121–122*
*finding MAC addresses in tables, 124–126*
*flooding unknown unicast/ broadcast frames, 119–120*
*forwarding known unicast frames, 116–118*
*forward-versus filter decisions, 118*
*interfaces, 123–124*
*learning MAC addresses, 118–119, 122–123*
*loop prevention, 120–121*
*MAC address tables, 116–118*
*MAC addresses with multiple switches, 127–128*
*managing MAC address tables, 126–127*
*summary, 121*
*verifying, 121–123*
Layer 3 EtherChannels, Layer 3 Switch Routed Ports, 473–477
network layer protocols, identifying, 56
NIC, 47

OUI, 54–55
point-to-point, 60
routers, Ethernet interface autonegotiation, 420–422
shared media, 60
static routes, Ethernet outgoing interfaces, 443
switches, frame transmissions, 57–60
transceivers, 45
types of, 41–42
unicast addressing, 54–55
UTP, Ethernet links, 44–46
VLAN, 188
*access interfaces, 197–198*
*assigning to interfaces, 197–201, 213*
*broadcast domains, 191–192*
*configuring, 197–201*
*creating, 197–201*
*data VLAN, 209–212*
*disabled/undefined VLAN, 212–218*
*forwarding data between VLAN, 195–197*
*native VLAN, 195*
*phone connections, 207–212*
*routing between VLAN, 195–197*
*static access interfaces, 197–198*
*troubleshooting, 212–218*
*trunking, 192–195, 201–207, 215–218*
*undefined/disabled VLAN, 213–215*
*voice VLAN, 209–212*
*VTP, 194, 201–202*
WAN, 64, 69
*IP routing, 70–71*
*Layer 2 services, 70*

**EUI-64 (modified), IPv6 addressing, 674–679, 682–683**
**exams**
additional content, 751
Cisco Certification Roadmap, 751–752
release dates/information, 751–752, 754
study plans, 753–754
updates, 751–752
**EXEC commands, 98**
**EXEC mode, CLI, 103–104**
**exec-timeout command, 152–153**
**existing subnets**
difficult subnet masks, 369–373
easy subnet masks, 367–369
practicing with, 373
**expanding IPv6 addresses, 648–649**
**extended ping, testing**
LAN neighbors, 520–521
reverse routes, 517–519
**extended traceroute, 526–527**
**external authentication servers, switch security, 141–142**

## F

**FCS (Frame Check Sequence) fields, 56–57**
**fiber cabling**
802.3 10-Gbps fiber standards, 52
cladding, 50–51
components of, 50–51
Ethernet, 52–53
Ethernet LAN builds, 50–53
multimode fiber, 51–53
optical transmitters, 50–51
single-mode fiber, 51–53
**filtering, forward-versus filter decisions, 118**

**finding**
MAC addresses in tables, 124–126
range of usable subnet addresses, 367
subnet ID/addresses, 369–371, 386–396
**floating static routes**
IPv4 routing 447–448
IPv6 network routing, 739–741
**flooding**
routers, 546
switches, 119–120
unknown broadcast/unicast frames, 119–120
**formatting IP addresses, 330**
**forwarding**
data between VLAN, 195–197
unicast frames, Ethernet LAN switching, 116–118
**forwarding state**
RSTP, 225, 229
STP, 120–121, 225, 229, 238–239
**forward-versus filter decisions, Ethernet LAN switching, 118**
**frames, 30, 31–32**
collisions, 58–59
Ethernet frame format, 53–54
FCS fields, 56–57
full duplex transmissions, 57–58, 59
half duplex transmissions, 58–60
hub transmissions, 57–60
looping frames, 225–227
multiple frame transmissions, STP, 227
switch transmissions, 57–60
**full (unabbreviated) IPv6 addresses, 646–647**
**full duplex transmissions**
autonegotiation, 161
Ethernet frame transmissions, 57–58, 59
**fully-adjacent neighbors, 553**

# G

**gateways, default. *See* default routers**
**GET requests, HTTP, 24**
**global ID, ULA, 665–666**
**grouping, 73**
hosts in subnets, 304–305
IP addressing, 76
**GUA (Global Unicast Addresses), 655–664**

# H

**half duplex transmissions**
autonegotiation, 161
frame transmissions, 58–60
**HDLC (High-level Data Link Control)**
framing, 65–67
header fields, 65–67
leased-line WAN, 66–67
**header fields**
Ethernet frames, 53–54, 67
HDLC, 65–67
IP, 77
**headers, IPv6, 643**
**Hello BPDU, STP/RSTP, 230–231, 236–237**
**Hello, inferior, 231**
**Hello intervals, 602–604, 614–615**
**help features, CLI, 101–102**
**hexadecimal/binary conversion chart, 647**
**high-speed Internet connections, end-user perspectives, 18–19**
**history buffer commands, 151–152**
**host routes, static, 445–447**
**hostname command, 109**
**hostnames, 79**

**hosts**
classful networks, determining number of hosts, 331
default gateways, 148–149
IPv4 addressing
*DHCP, 488–497*
*Linux settings, 504–506*
*macOS settings, 502–504*
*settings, 497–499*
*troubleshooting settings, 506–509*
*Windows IP settings, 499–502*
IPv6 addressing
*DAD, 705*
*DHCP, 706–710*
*on-link prefixes, 703–704*
*NA, 699–702, 705*
*NDP, 698–699*
*NDP, NA, 699–702, 705*
*NDP, NS, 699–702, 705*
*NDP, RA, 702–704*
*NDP, RS, 702–704*
*NDP, summary, 705–706*
*NS, 699–702, 705*
*permanent IPv6 addresses, 712*
*prefix discovery, 699*
*RA, 702–704*
*RS, 702–704*
*SLAAC, 710–714*
*static host routes, 737–739*
*temporary IPv6 addresses, 712–714*
*testing connectivity, 716–718*
*troubleshooting, 714–718*
*verifying host connectivity, 718–719*
network layer routing (forwarding) logic, 72
routing logic (summary), 429
stateful DHCPv6, 706–710
stateless DHCPv6, 707, 710, 711–712
subnetting
*determining number of hosts, 307*
*grouping in, 304–305*
verifying connectivity, IPv6 addressing, 718–719
**HTTP (HyperText Transfer Protocol)**
data-only messages, 24
GET requests, 24
overview of, 23
protocol mechanisms, 23–24
replies, 24
web pages, 24
**hubs**
collisions, 58–59
Ethernet frame transmissions, 57–60
**hybrid (advanced distance vector) algorithms, 543**

## I

**ICMP (Internet Control Message Protocol)**
echo replies, 513
echo requests, 81–82, 513
**ICMPv6 (Internet Control Message Protocol version 6), 642**
**ID/addresses, subnetting, 308, 360**
binary math analysis, 362–366
finding, 386–396
**identifying**
embedded switch ports, 481–482
network layer protocols, with EtherType fields, 56
port costs, 266–268
roles, 266–268
root switches, 263–266

states, 266–268
switch priority, 263–266
**IEEE (Institute of Electrical and Electronics Engineers)**
802.3 10-Gbps fiber standards, 52
802.3 Ethernet header and trailer fields, 53–54
autonegotiation, 161–169
default port costs, 235–236
**IGP (Interior Gateway Protocols), 542–545**
advanced distance vector (hybrid) algorithms, 543
distance vector (Bellman-Ford) algorithms, 543
EIGRP, 543–544
ICMPv6, 642
IGRP, 543
link-state, 543
NDP, 642
OSPF, 543–545
*AD, 623*
*areas, 555–557*
*BDR, 552–553*
*broadcast networks, 586–592*
*concepts, 546*
*design terminology, 556*
*Dijkstra SPF algorithm, 553–555*
*DR/BDR elections, 590–592*
*ECMP routing, 621–622*
*equal-cost routes, 621–622*
*Hello packets, 548–549*
*interface configurations, 575–578*
*IP forwarding, 625–628*
*IP routing table, 628–630*
*longest prefix matching, 625–630*
*LSA, 546–548, 557–560*
*LSDB, 546, 547–548, 550–553*
*MTU, troubleshooting, 618*
*multi-area OSPF, 574–575*
*neighbors, 546, 548–553*
*neighbors, requirements, 611–612*
*neighbors, troubleshooting adjacencies, 612–617*
*neighbors, troubleshooting missing routes, 619–621*
*neighbors, troubleshooting priority 0, 618–619*
*network types, troubleshooting, 618*
*operation of, 546*
*point-to-point networks, 586, 592–594*
*priority, 590–592*
*RID, 548*
*route calculation, 553–555*
*route selection, 621–630*
*routing, troubleshooting, 618–621*
*shutting down, 615–617*
*topologies, 546–547*
OSPFv2
*Dead intervals, 602–604, 614–615*
*default routes, 597–599*
*Hello intervals, 602–604, 614–615*
*implementing with interface subcommands, 576*
*interface bandwidth, 599, 600*
*metrics (cost), 599–601*
*network commands, 564–568*
*passive interfaces, 594–597*
*reference bandwidth, 599, 600–601*
*verification commands, 568–573*

OSPFv3, 642
RIPv2, 543–545
**IGRP (Interior Gateway Routing Protocol), 543**
**inferior Hello, 231**
**installing**
enterprise routers, 408–409, 412
*Catalyst Edge Platform, 411*
*IOS, 409–410*
*IOS XE, 409–410*
*ISR, 410–411*
*OS, 409–410*
SOHO routers, 412–413
**interarea routes, 556**
**interesting octets (subnet masks), predictability in, 368–369**
**interface command, 104–105**
**interface ID, LAN switches, 91**
**interface range command, switch interface configuration, 172–173**
**interface subcommands, OSPFv2 implementation, 576**
**interfaces**
bandwidth, 599, 600
configuring
*EtherChannel, 287–289*
*OSPF, 575–578*
routed interfaces, switches, 470–473
routers
*bandwidth, serial interfaces, 423*
*clock rates, serial interfaces, 423*
*Ethernet interface autonegotiation, 420–422*
*IPv4 addressing, 418–419*
*listing interfaces, 416–417*
*serial interfaces, 423*
*status codes, 417–418*
show interfaces command, 124
states, changing with STP, 238–239
switches, 123–124, 172
*administratively controlling interface state, 172–173*
*description command, 172–173*
*duplex mismatches, 177–178*
*interface range command, 172–173*
*late collisions, 163*
*layer 1 problems, 178–180*
*removing, 174–176*
*routed interfaces, 470–473*
*shutdown command, 172–173*
*states, 234–235*
*status codes, 176–177*
*VLAN assignments, 197–201, 213*
VLAN interfaces, 464–466
**internal routers, 556**
**Internet, high-speed connections, 18–19**
**Internet layer. *See* network layer, TCP/IP networking models**
**internetworks (enterprise networks), 19, 304**
**intra-area routes, 556**
**IOS**
enterprise routers, 409–410
help features, 101–102
software configuration, 103–104
**IOS XE**
CLI
*accessing, 91–101*
*accessing in privileged (enable) mode, 96–98*
*accessing in user mode, 96–98*
*accessing with SSH, 96*
*accessing with Telnet, 96*
*cabling console connections, 93–94*
enterprise routers, 409–410
terminal emulator configurations, 95

**IP (Internet Protocol).** ***See also*** **IPv4; IPv6**
addressing
*APIPA IP addresses (169.254.x.x), 490*
*ARP, 80–81*
*basics, 27–28*
*configuring, connected routes, 6435–439*
*IP routing, 76–77*
*troubleshooting, ping command, 522–524*
forwarding, OSPF, 625–628
groupings, 73
ip address command, 436–439
network broadcast addresses, 331, 333–334
networks, 73
no ip domain-lookup command, 152–153
packets, encapsulating/de-encapsulating, routing, 68–69
postal service analogy, 26–27
public IP networks
*setting context of, 326–328*
*subnetting, 311–313*
routing, 71–72, 428
*AD, 623*
*basics, 28–29*
*default routers, 73*
*dynamic routing protocols. See separate entry*
*ECMP routing, 621–622*
*encapsulating/de-encapsulating IP packets, 68–69*
*equal-cost routes, 621–622*
*Ethernet WAN, 70–71*
*IP addressing, 76–77*
*LAN, 67–69, 73–75*
*network advertisements, 78*
*network layer routing (forwarding), logic, 72–73*
*OSPF, AD, 623*
*protocols, 77–78*
*route selection, OSPF, 621–630*
*tables, 73, table, 628–630*
*troubleshooting, IP addresses, 522–524*
*troubleshooting, name resolution, 522–524*
*troubleshooting, ping command, 512, 513–527*
*troubleshooting, SSH, 527–530*
*troubleshooting, Telnet, 527–530*
*troubleshooting, testing LAN neighbors, 519–521*
*troubleshooting, testing longer routes, 514–517*
*troubleshooting, testing reverse routes, 517–519*
*troubleshooting, testing WAN neighbors, 521*
*troubleshooting, traceroute command, 512, 524–527*
*WAN, 67–69, 73–75*
subnetting, 73, 76, 314
switches, 147–149, 150
telephony ports, 208, 212
unicast IP addresses, CIDR blocks, 327
**ip address command, 436–439**
**IPv4 (Internet Protocol version 4)**
addresses
*APIPA IP addresses (169.254.x.x), 490*
*classes, 328–330*
*default masks, 330*
*exhaustion timeline, 641–642*
*formats, 330*

*hosts, DHCP, 488–497*
*hosts, Linux settings, 504–506*
*hosts, macOS settings, 502–504*
*hosts, settings, 497–499*
*hosts, troubleshooting settings, 506–509*
*hosts, Windows IP settings, 499–502*
*router interfaces, 418–419*
*troubleshooting, ping command, 522–524*
classful networks
*CIDR blocks, 326–328*
*classes, 328–330*
*determining number of hosts, 331*
*network broadcast addresses, 331, 333–334*
*network ID, 331–334*
*practicing with, 334–335*
*public IP networks, setting context of, 326–328*
*subnetting, 311–314*
*WHOIS records, 328*
hosts, addressing
*DHCP, 488–497*
*Linux settings, 504–506*
*macOS settings, 502–504*
*settings, 497–499*
*troubleshooting settings, 506–509*
*Windows IP settings, 499–502*
ip address command, 436–439
routers, support for, 413–423
routing
*ARP tables, 439–440*
*connected routes, 426–427*
*example of, 431–435*
*host routing logic (summary), 429*
*ip address command, 436–439*
*IP addressing, configuring with connected routes, 435–439*
*process reference, 429–431*
*routing logic (summary), 430–431*
*static routes, 426–427*
*static routes, configuring, 440–443*
*static routes, Ethernet outgoing interfaces, 443*
*static routes, floating static routes, 447–448*
*static routes, proxy ARP, 443*
*static routes, static default routes, 443–445*
*static routes, static host routes, 445–447*
*static routes, static network routes, 441–442*
*static routes, troubleshooting, 443–445*
*tables, 426–427*
*troubleshooting, IP addresses, 522–524*
*troubleshooting, name resolution, 522–524*
*troubleshooting, ping command, 512, 513–524*
*troubleshooting, SSH, 527–530*
*troubleshooting, Telnet, 527–530*
*troubleshooting, testing LAN neighbors, 519–521*
*troubleshooting, testing longer routes, 514–517*
*troubleshooting, testing reverse routes, 517–519*
*troubleshooting, testing WAN neighbors, 521*
*troubleshooting, traceroute command, 512, 524–527*

*VLAN, Layer 3 Switch Routed Ports, 455, 469–477*
*VLAN, Layer 3 Switch SVI, 455, 464–469*
*VLAN, ROAS, 457–464*
*VLAN, Router 802.1Q Trunks, 454, 457–464*
*VLAN, Router LAN Switch Ports, 455, 477–482*
subnetting, 302
*assigning to different locations, 320–321*
*broadcast addresses, 358, 359, 361*
*calculating based on IPv4 format, 349–350*
*choosing dynamic ranges, 321–322*
*choosing IP networks, 314*
*choosing static ranges, 321–322*
*classful addressing, 348*
*classful networks, 311–314*
*classless addressing, 348*
*DDN masks, 367, 370*
*defined, 302–303*
*defining, 358*
*defining size of subnets, 308–310*
*design choices, 309–311*
*design view, 303–304*
*determining number of hosts, 307*
*determining number of subnets, 305–307*
*DHCP, 320–321*
*difficult subnet masks, 368*
*difficult subnet masks, finding subnet ID/addresses, 369–371*
*dividing IP addresses, 346–348*
*easy subnet masks, 367–369*
*existing subnets, difficult subnet masks, 369–373*
*existing subnets, easy subnet masks, 367–369*
*existing subnets, practicing with, 373*
*grouping hosts, 304–305*
*ID/addresses, 308, 360*
*ID/addresses, binary math analysis, 362–366*
*ID/addresses, finding in difficult subnet masks, 369–371*
*identifying design choices, 345–346*
*listing subnets, 318–320*
*magic numbers, 370, 373–374, 388–389*
*masks, 314–318, 387–388*
*masks, binary masks, 340, 341–344*
*masks, Boolean AND/OR operations, 367*
*masks, choosing to meet requirements, 380–386*
*masks, CIDR blocks, 351–352*
*masks, conversions, 340–345*
*masks, DDN, 318, 341*
*masks, DDN masks, 342–344*
*masks, finding ID addresses, 386–396*
*masks, formats of (overview), 340–341*
*masks, magic numbers, 370, 373–374, 388–389*
*masks, octet values, 342–343*
*masks, practicing with, 350–351*
*masks, prefix masks, 341–342, 344*
*operational view, 303–304*
*planning implementations, 320*

*predictability in interesting octets, 368–369*
*private IP networks, 313–314*
*public IP networks, 311–313*
*range of usable addresses, 361, 367*
*resident subnets, 358–359*
*unicast IP addresses, 358*
*VLSM, 309–311*
switches
*configuring, 149–150*
*security, remote access, 146–147*
*verifying configurations, 150–151*
unicast IP addresses, CIDR blocks, 327
VLAN routing
*Layer 3 Switch Routed Ports, 455, 469–477*
*Layer 3 Switch SVI, 455, 464–469*
*ROAS, 457–464*
*Router 802.1Q Trunks, 454, 457–464*
*routing, Router LAN Switch Ports, 455, 477–482*
**IPv6 (Internet Protocol version 6), 640**
addressing
*anycast addresses, 672–689*
*configuration summary, 690–691*
*DAD, 705*
*DHCP, 706–710*
*dual stacks, 671*
*on-link prefixes, 703–704*
*LLA, 680–684*
*loopback addresses, 689*
*modified EUI-64, 674–679, 682–683*
*multicast addresses, 684–688*
*NA, 699–702, 705*
*NDP, 698–699*
*NDP, NA, 699–702, 705*
*NDP, NS, 699–702, 705*
*NDP, RA, 702–704*
*NDP, RS, 702–704*
*NDP, summary, 705–706*
*next-hop addresses, static network routes, 730–735*
*NS, 699–702, 705*
*permanent IPv6 addresses, 712*
*prefix discovery, 699*
*RA, 702–704*
*RS, 702–704*
*scopes, 686–687*
*SLAAC, 710–714*
*solicited-node multicast addresses, 684, 687–688*
*special addresses, 680*
*static host routes, 737–739*
*temporary IPv6 addresses, 712–714*
*testing connectivity, 716–718*
*troubleshooting, 714–718*
*unicast IP addresses, 670–671*
*unicast IP addresses, dynamic, 679–680*
*unicast IP addresses, static, 671–679*
*unspecified addresses, 689*
*verifying host connectivity, 718–719*
GUA, 655–664
headers, 643
historical reasons for, 640–642
hosts
*DAD, 705*
*DHCP, 706–710*
*NA, 699–702, 705*
*NDP, 698–699*

*NDP, NA, 699–702, 705*
*NDP, NS, 699–702, 705*
*NDP, RA, 702–704*
*NDP, RS, 702–704*
*NDP, summary, 705–706*
*NS, 699–702, 705*
*RA, 702–704*
*RS, 702–704*
*SLAAC, 710–714*
*static host routes, 737–739*
*troubleshooting, 714–718*
*verifying host connectivity, 718–719*
ICMPv6, 642
NDP, 642
*NA, 699–702, 705*
*NS, 699–702, 705*
*RA, 702–704*
*RS, 702–704*
*summary, 705–706*
private addresses, 655–664
public addresses, 656–657
routing, 643–645
*abbreviating addresses, 647, 648–649*
*AD, 727, 739–741*
*address formats/conventions, 646*
*address prefix lengths, 649*
*anycast addresses, 672–689*
*configuration summary, 690–691*
*connected routes, 724–727*
*dual stacks, 671*
*EIGRPv6, 645–646*
*expanding addresses, 648–649*
*floating static routes, 739–741*
*full (unabbreviated) addresses, 646–647*
*hexadecimal/binary conversion chart, 647*
*LLA, 680–684*
*local routes, 724–725, 727–728*
*loopback addresses, 689*
*modified EUI-64, 674–679, 682–683*
*MP BGP-4, 645–646*
*multicast addresses, 684–688*
*OSPFv3, 642, 645*
*RIPng, 645*
*scopes, 686–687*
*solicited-node multicast addresses, 684, 687–688*
*special addresses, 680*
*static default routes, 735–737*
*static host routes, 737–739*
*static network routes, 728–735*
*subnet prefixes, 649–652*
*troubleshooting, 741–743*
*unicast IP addresses, 670–671*
*unicast IP addresses, dynamic, 679–680*
*unicast IP addresses, static, 671–679*
*unspecified addresses, 689*
ULA, 656, 664–666
**ISL VLAN trunking, 194–195**
**ISR (Integrated Services Routers), 410–411**

## L

**LACP (Link Aggregation Control Protocol), 284**
**LAN (Local Area Networks).** ***See also*** **VLAN; WLAN**
campus LAN, 114
data center LAN, 114

designated ports, choosing, 234–235
enterprise LAN, 40–41
Ethernet LAN, 36
*data-link layer consistency, 42–43*
*data-link layer protocols, 53–57*
*fiber builds, 50–53*
*frame transmissions, 57–60*
*physical layer standards, 41–42*
*UTP builds, 43–49*
Ethernet LAN, switching
*analyzing, 121–122*
*finding MAC addresses in tables, 124–126*
*flooding unknown unicast/ broadcast frames, 119–120*
*forward-versus filter decisions, 118*
*forwarding known unicast frames, 116–118*
*interfaces, 123–124*
*learning MAC addresses, 118–119, 122–123*
*loop prevention, 120–121*
*MAC addresses with multiple switches, 127–128*
*MAC address tables, 116–118, 126–127*
*summary, 121*
*verifying, 121–123*
hubs, autonegotiation, 165
IP routing, 67–69
neighbors, testing, 519–520
network layer routing (forwarding), 73–75
overview of, 38
SOHO LAN, 39
switches. *See also* Ethernet LAN, switching
*concepts, 114*
*interface ID, 91*
*logic, 115–116*
wired LAN. *See* Ethernet LAN
WLAN, 18–19
**late collisions, switch interface configurations, 163**
**layer 1 problems, switch interface configurations, 178–180**
**Layer 2 EtherChannel, configuring, 281–284**
**Layer 2 services, Ethernet WAN, 70**
**Layer 3 Switch Routed Ports**
Layer 3 EtherChannels, 473–477
routed interfaces, 470–473
VLAN routing, 455, 469–477
**Layer 3 Switch SVI**
autostate setting, 467–469
configuring, 464–466
troubleshooting, 467–469
verifying configurations, 466
VLAN routing, 455, 464–469
**learning**
learning state, STP, 239
MAC addresses, 118–119, 122–123
switch learning, 118–119
**leased-line WAN, 64–65**
conceptual view, 65
data-link details, 66–69
different names for, 65–66
HDLC, 66–67
IP routing, 67–69
physical details, 65–66
PPP, 66–67
routers, 67–69
telcos, 65–66
**link types, RSTP, 245**
**link-state, 543**

**Linux host settings, 504–506**
**listening state, STP, 239**
**listing**
router interfaces, 416–417
subnets, 318–320
**LLA (Link Local Addresses), IPv6 routing, 680–684**
**load balancing, VLAN, 260**
**local IPv6 routes, 724–725, 727–728**
**local usernames/passwords, switch security, 139–141**
**log messages, switches, 141**
**logging synchronous command, 152–153**
**longest prefix matching, IP forwarding, 625–630**
**Loop Guard, STP, 251–253, 279–280**
**loop prevention, Ethernet LAN switching, 120–121**
**loopback IPv6 addresses, IPv6 routing, 689**
**looping frames, 225–227**
**LSA (Link State Advertisements), 546–548, 557–560**
**LSDB (Link State Databases), 546, 550–553**
Dijkstra SPF algorithm, 547–548
neighbors, 550–553

# M

**MAC addresses, 54–55**
aging MAC addresses, clearing, 126–127
Ethernet LAN, switching, 116–118
finding in tables, 124–126
learning, 118–119, 122–123
managing tables, 126–127
STP/RSTP, 227
tables, routers, 481
tables, switches, 103, 127–128
**macOS host settings, 502–504**
**magic numbers, 370, 373–374, 388–389**
**managing**
convergence, STP, 237–238
MAC address tables, 126–127
**masks, subnetting, 314–318, 545**
binary masks, 340
*DDN mask conversions, 342–344*
*prefix mask conversions, 341–342*
Boolean AND/OR operations, 367
choosing to meet requirements, 380–386
conversions, 340–345
DDN masks, 318, 341
*binary mask conversions, 342–344*
*DDN masks, 367*
*prefix mask conversions, 344*
design choices, identifying, 345–346
dividing IP addresses, 346–348
easy subnet masks, 367–369
finding ID/addresses, 386–396
formats of (overview), 340–341
magic numbers, 370, 373–374, 388–389
octet values, 342–343
predictability in interesting octets, 368–369
prefix masks, 341–342, 344
zero subnet, 387–388
**MaxAge timers, STP, 237, 238, 241**
**metrics (cost), OSPFv2, 599–601**
**modified EUI-64, IPv6 addressing, 674–679, 682–683**

**moving between configuration modes, 105–106**
**MP BGP-4 (Multiprotocol BGP version 4), 645–646**
**MSTP (Multiple Spanning Tree Protocol), 260, 261**
**MTU (Maximum Transmission Units)**
Ethernet frames, 54
OSPF, troubleshooting, 618
**multi-area OSPF, 555, 574–575**
**multicast IPv6 addresses**
IPv6 routing, 684–688
scopes, 686–687
solicited-node multicast addresses, 684, 687–688
**multilayer switches. *See* Layer 3 Switch SVI**
**multimode fiber, 51–53**
**multiple frame transmissions, STP, 227**
**multiple spanning trees, need for, 260**

# N

**NA (Neighbor Advertisements), 699–702, 705**
**name resolution**
IPv4 routing, troubleshooting, 522–524
requests, DNS, 79–80, 152–153
**native VLAN, 195**
**navigating between configuration modes, 105–106**
**NDP (Neighbor Discovery Protocol), 642, 698–699**
NA, 699–702, 705
NS, 699–702, 705
RA, 702–704
RS, 702–704
summary, 705–706
**neighbors**
adjacencies, 553, 612–617
fully-adjacent neighbors, 553
LSDB, 550–553
missing routes, troubleshooting, 619–621
NA, 699–702, 705
NDP, 642, 698–699
*NA, 699–702, 705*
*NS, 699–702, 705*
*RA, 702–704*
*RS, 702–704*
*summary, 705–706*
NS, 699–702, 705
OSPF, 546
priority 0, troubleshooting, 618–619
RA, 702–704
relationships, 553
requirements, 611–612
RS, 702–704
**network commands, OSPFv2, 564–565**
single-area OSPF configuration, 565–566
wildcard matching, 566–568
**network layer, 79**
ARP, 80–81
DNS, 79–80
encapsulation, 74
EtherType fields, identifying layers with, 56
ICMP echo requests, 81–82
IP routing, 76–77
*grouping addresses, 76*
*header fields, 76–77*
*LAN, 73–75*
*protocols, 77–78*
*subnets, 76*
*WAN, 73–75*

ping command, 81–82
routing (forwarding), logic, 72–73
TCP/IP networking model, 22, 26
*addressing, basics, 27–28*
*postal service analogy, 26–27*
*outing, basics, 28–29*
**network types (OSPF), troubleshooting, 618**
**networking models**
defined, 19–20
OSI, 20–21
TCP/IP
*development of, 20–21*
*history of, 20–21*
*network layer, 22*
*overview of, 21–23*
*protocols, 22–23*
*RFC, 21*
**networks**
advertisements, IP routing, 78
AS, 542–543
broadcast addresses, classful networks, 331, 333–334
broadcast networks, 586–592
classful networks
*broadcast addresses, 331, 333–334*
*CIDR blocks, 326–328*
*classes, 328–330*
*determining number of hosts, 331*
*network broadcast addresses, 331, 333–334*
*network ID, 331–334*
*practicing with, 334–335*
*public IP networks, setting context of, 326–328*
*subnetting, 311–314*
*WHOIS records, 328*
enterprise LAN, 40–41
enterprise networks (internetworks), example of, 19
Ethernet LAN, 36
*data-link layer consistency, 42–43*
*data-link layer protocols, 53–57*
*fiber builds, 50–53*
*frame transmissions, 57–60*
*physical layer standards, 41–42*
*UTP builds, 43–49*
ID, classful networks, 331–334
IP, 73
LAN, 36
*campus LAN, 114*
*data center LAN, 114*
*designated ports, choosing, 234–235*
*hubs, autonegotiation, 165*
*IP routing, 67–69*
*network layer routing (forwarding), 73–75*
*overview of, 38*
*SOHO LAN, 39*
*switches, interface ID, 91*
*switching, concepts, 114*
*switching, logic, 115–116*
*testing neighbors, 519–521*
*WLAN, 18–19*
LSA, 558
perspectives on, 18–19
point-to-point networks, 586, 592–594
private IP networks, subnetting, 313–314
public IP networks
*setting context of, 326–328*
*subnetting, 311–313*

VLAN, 188
*access interfaces, 197–198*
*assigning to interfaces, 197–201, 213*
*broadcast domains, 191–192*
*configuring, 197–201*
*creating, 197–201*
*data VLAN, 209–212*
*disabled/undefined VLAN, 212–218*
*forwarding data between VLAN, 195–197*
*load balancing with STP, 260*
*native VLAN, 195*
*phone connections, 207–212*
*PVST+260, 261*
*ROAS, 454*
*ROAS, configuring, 458–461*
*ROAS, troubleshooting, 463–464*
*ROAS, verifying configurations, 461–463*
*routing, Layer 3 Switch Routed Ports, 455, 469–477*
*routing, Layer 3 Switch SVI, 455, 464–469*
*routing, Router 802.1Q Trunks, 454, 457–464*
*routing, Router LAN Switch Ports, 455, 477–482*
*routing between VLAN, 195–197*
*RPVST+260, 261*
*static access interfaces, 197–198*
*switches, IP settings, 147–149*
*troubleshooting, 212–218*
*trunking, 192–195, 201–207, 215–218*
*undefined/disabled VLAN, 213–215*
*voice VLAN, 209–212*
*VTP, 194, 201–202*
WAN, 36, 64
*Ethernet WAN, 64, 69–71*
*leased-line WAN, 64–69*
*network layer routing (forwarding), 73–75*
*testing neighbors, 521*
wired LAN. *See* Ethernet LAN
WLAN, 18–19
**next-hop IPv6 addresses, static network routes, 730–735**
**next-hop routers, 434, 441, 442–443**
**NIC (Network Interface Cards), Ethernet, 47**
**no ip domain-lookup command, 152–153**
**NS (Neighbor Solicitations), 699–702, 705**

## O

**octet values**
IPv4 address classes, 328–330
subnet masks, 342–343
**offer messages, 489–490**
**on-link prefixes, 703–704**
**operational view, subnetting, 303–304**
**optical transmitters, 50–51**
**OS (Operating Systems), routers, 409–410**
**OSI networking models, 20–21, 32–34**
**OSPF (Open Shortest Path First), 543–545**
AD, 623
areas, 555–557
*multi-area OSPF, 555, 574–575*
*single-area OSPF, 555–557, 565–566*

BDR, 552–553
broadcast networks, 586–592
concepts, 546
configuring
*interface configurations, 575–578*
*RID, 573–574*
*single-area OSPF, 565–566*
design terminology, 556
Dijkstra SPF algorithm
*LSDB, 547–548*
*route calculation, 553–555*
DR/BDR elections, 590–592
ECMP routing, 621–622
equal-cost routes, 621–622
Hello packets, 548–549
interface configurations, 575–578
IP
*forwarding, 625–628*
*routing table, 628–630*
longest prefix matching, 625–630
LSA, 546–548, 557–560
LSDB, 546, 547–548, 550–553
MTU, troubleshooting, 618
neighbors, 546, 548–553
*requirements, 611–612*
*troubleshooting, priority 0, 618–619*
*troubleshooting adjacencies, 612–617*
*troubleshooting missing routes, 619–621*
network types, troubleshooting, 618
operation of, 546
point-to-point networks, 586, 592–594
priority, 590–592
RID, 548, 573–574
routing
*route selection, 621–630*
*troubleshooting, 618–621*
shutting down, 615–617
topologies, 546–547
wildcard matching, 566–568
**OSPFv2 (Open Shortest Path First version 2)**
Dead intervals, 602–604, 614–615
default routes, 597–599
Hello intervals, 602–604, 614–615
implementing with interface subcommands, 576
interface bandwidth, 599, 600
metrics (cost), 599–601
network commands, 564–565
*single-area OSPF configuration, 565–566*
*wildcard matching, 566–568*
passive interfaces, 594–597
reference bandwidth, 599, 600–601
verification commands, 568–573
**OSPFv3 (Open Shortest Path First version 3), 642, 645**
**OUI (Organizationally Unique Identifiers), Ethernet addressing, 54–55**

# P

**packets, 27, 28–32**
**PAgP (Port Aggregation Protocol), 284**
**parallel detection logic, autonegotiation, 163–164**
**passive interfaces, OSPFv2, 594–597**
**passwords**
basic password configurations, 137–138
CLI access, 98–99

console passwords, 135–139
enable passwords, 135–139
enable secret passwords, 137
privileged mode, switches, 135–139
shared passwords, 135–139
simple security concepts, 135
switch security, user mode, 139–141
Telnet, 135–139
user mode, switches, 135–139
vty passwords, 135, 137
**path selection.** ***See*** **routing, IP**
**permanent IPv6 addresses, 712**
**phone connections, VLAN, 207–212**
**physical layer**
Ethernet LAN, standards, 41–42
TCP/IP networking models, 29–31
**pin pairs**
10BASE-T, 48
100BASE-T, 48
**ping command**
basics, 513–514
extended ping
*testing LAN neighbors, 520–521*
*testing reverse routes, 517–519*
IP addresses, 522–524
IPv4 routing, 512, 513–524
IPv6 connectivity, 716–718
name resolution, 522–524
network layer, 81–82
problem isolation, 513
strategies/results, 514–521
testing
*LAN neighbors, 519–520*
*longer routes, 514–517*
*reverse routes, 517–519*
*WAN neighbors, 521*
**pinouts**
choosing, 48–49
crossover cabling, Ethernet cabling pinouts, 48
straight-through cabling, 47
UTP
*10BASE-T pinouts, 46–47*
*100BASE-T pinouts, 46–47*
*1000BASE-T pinouts, 49*
**planning subnet implementations, 320**
**point-to-point, Ethernet, 60**
**point-to-point links/lines.** ***See*** **leased-line WAN**
**point-to-point networks, 586, 592–594**
**point-to-point ports, RSTP, 245**
**PortFast, STP, 247–248, 269–274**
**ports**
10/100 ports, autonegotiation, 161, 162
10/100/1000 ports, autonegotiation, 161, 162, 163
Aux ports, 423
costs, STP, 266–268
default costs, 235–236
DP, 229
embedded switch ports
*configuring, 478–480*
*identifying, 481–482*
*verifying routing, 480–481*
IP telephony ports, 208, 212
Layer 3 Switch Routed Ports
*Layer 3 EtherChannels, 473–477*
*routed interfaces, 470–473*
*VLAN routing, 455, 469–477*
root ports, choosing, 232–234
Router LAN Switch Ports
*embedded switch ports, 478–482*
*VLAN routing, 455, 477–482*

RSTP
*alternate ports, 241–243*
*backup ports, 241, 244*
*edge ports, 245*
*point-to-point ports, 245*
*port roles, 241*
*states, 243–244*
*types of ports, 245–246*
STP, states, 243–244
**postal service, IP analogy, 26–27**
**PPP (Point-to-Point Protocol), leased-line WAN, 66–67**
**predictability in interesting octets, subnet masks, 368–369**
**preferred lifetimes, SLAAC, 712–714**
**prefixes**
discovery, 699
lengths, IPv6 addresses, 649
on-link prefixes, 703–704
longest prefix matching, OSPF, 625–630
masks, 341
*binary mask conversions, 341–342*
*DDN mask conversions, 342–344*
**priority**
OSPF, 590–592
switches, identifying, 263–266
values, STP/RSTP, 262
**priority 0, troubleshooting OSPF neighbors, 618–619**
**private IP networks, subnetting, 313–314**
**private IPv6 addresses, 656–657**
**private lines. *See* leased-line WAN**
**privileged (enable) mode**
Cisco IOS XE CLI access, 96–98
rejected commands example, 97–98
switches, security, simple passwords, 135–139
**problem isolation**
ping command, 513
tracert command, 524
**productivity commands, 153**
**protocols**
ARP, network layer, 80–81
dynamic routing protocols
*convergence, 541, 542*
*EIGRP, 543–544, 545*
*features, 540*
*functions, 541–542*
*ICMPv6, 642*
*IGP, 542–545*
*IGRP, 543*
*NDP, 642*
*OSPF, 543–545*
*OSPF, AD, 623*
*OSPF, areas, 555–557*
*OSPF, BDR, 552–553*
*OSPF, broadcast networks, 586–592*
*OSPF, concepts, 546*
*OSPF, design terminology, 556*
*OSPF, Dijkstra SPF algorithm, 553–555*
*OSPF, DR/BDR elections, 590–592*
*OSPF, ECMP routing, 621–622*
*OSPF, equal-cost routes, 621–622*
*OSPF, Hello packets, 548–549*
*OSPF, implementing with interface subcommands, 576*
*OSPF, interface configurations, 575–578*
*OSPF, IP forwarding, 625–628*

*OSPF, IP routing table, 628–630*
*OSPF, longest prefix matching, 625–630*
*OSPF, LSA, 546–548, 557–560*
*OSPF, LSDB, 546, 547–548, 550–553*
*OSPF, MTU, 618*
*OSPF, multi-area OSPF, 574–575*
*OSPF, neighbor adjacencies, 612–617*
*OSPF, neighbor priorities, 618–619*
*OSPF, neighbor requirements, 611–612*
*OSPF, neighbors, 546, 548–553*
*OSPF, neighbors with missing routes, 619–621*
*OSPF, network types, 618*
*OSPF, operation of, 546*
*OSPF, point-to-point networks, 586, 592–594*
*OSPF, priority, 590–592*
*OSPF, RID, 548*
*OSPF, route calculation, 553–555*
*OSPF, route selection, 621–630*
*OSPF, shutting down, 615–617*
*OSPF, topologies, 546–547*
*OSPF, troubleshooting routing, 618–621*
*OSPFv2, Dead intervals, 602–604, 614–615*
*OSPFv2, default routes, 597–599*
*OSPFv2, Hello intervals, 602–604, 614–615*
*OSPFv2, interface bandwidth, 599, 600*
*OSPFv2, metrics (cost), 599–601*
*OSPFv2, network commands, 564–568*
*OSPFv2, passive interfaces, 594–597*
*OSPFv2, reference bandwidth, 599, 600–601*
*OSPFv2, verification commands, 568–573*
*OSPFv3, 642*
*path selection, 540*
*RIPv2, 543–545*
EIGRP, 543–545
EIGRPv6, 645–646
HTTP
*data-only messages, 24*
*GET requests, 24*
*overview of, 23*
*replies, 24*
*web pages, 24*
ICMP, echo requests, 81–82
ICMPv6, 642
IGP, 542–545
IGRP, 543
IP routing, 77–78
LACP, 284
MP BGP-4, 645–646
NDP, 642, 698–699
*NA, 699–702, 705*
*NS, 699–702, 705*
*RA, 702–704*
*RS, 702–704*
*summary, 705–706*
network layer protocols, identifying with EtherType fields, 56
OSPF, 543–545
*AD, 623*
*areas, 555–557*
*BDR, 552–553*
*concepts, 546*
*design terminology, 556*

*Dijkstra SPF algorithm, 553–555*
*DR/BDR elections, 590–592*
*ECMP routing, 621–622*
*equal-cost routes, 621–622*
*Hello packets, 548–549*
*interface configurations, 575–578*
*IP forwarding, 625–628*
*IP routing table, 628–630*
*longest prefix matching, 625–630*
*LSA, 546–548, 557–560*
*LSDB, 546, 547–548, 550–553*
*MTU, troubleshooting, 618*
*multi-area OSPF, 574–575*
*neighbors, 546, 548–553*
*neighbors, requirements, 611–612*
*neighbors, troubleshooting adjacencies, 612–617*
*neighbors, troubleshooting missing routes, 619–621*
*neighbors, troubleshooting priority 0, 618–619*
*network types, troubleshooting, 618*
*operation of, 546*
*OSPF, broadcast networks, 586–592*
*point-to-point networks, 586, 592–594*
*priority, 590–592*
*RID, 548*
*route calculation, 553–555*
*route selection, 621–630*
*routing, troubleshooting, 618–621*
*shutting down, 615–617*
*topologies, 546–547*
OSPFv2
*Dead intervals, 602–604, 614–615*
*default routes, 597–599*
*Hello intervals, 602–604, 614–615*
*implementing with interface subcommands, 576*
*interface bandwidth, 599*
*metrics (cost), 599–601*
*network commands, 564–568*
*passive interfaces, 594–597*
*reference bandwidth, 599, 600–601*
*verification commands, 568–573*
OSPFv3, 642, 645
PAgP, 284
RIPng, 645
RIPv2, 543–545
TCP/IP networking models, 22–23
VTP, 194, 201–202
**proxy ARP, static routes, 443**
**public IP networks**
setting context of, 326–328
subnetting, 311–313
**public IPv6 addresses, 656–657**
**PVST+ (Per VLAN Spanning Tree Plus), 260, 261**

## Q

**quartets, 646–647**
**question mark (?) command, 101**

## R

**RA (Neighbor Advertisements), 702–704**
**range of usable subnet addresses, 361, 367**

**recalling commands, 102**
**reference bandwidth, 599, 600–601**
**rejected commands example, enable (privileged) mode, 97–98**
**relationships, neighbors, 553**
**reload command, switch configurations, 109**
**remote access, switches**
IPv4, 146–147
SSH, 142–145
**remote subnets, DHCP Relay, 490–492**
**removing, interface configurations from switches, 174–176**
**replies, HTTP, 24**
**request messages, 489**
**resident subnets, 358–359**
**reverse routes, testing, 517–519**
**RFC (Requests for Comments), 21**
**RID (Router ID), OSPF, 548, 573–574**
**RIPng (RIP next generation), 645**
**RIPv2 (Routing Information Protocol version 2), 543–545**
**ROAS (Router-On-A-Stick)**
configuring, 458–461
verifying configurations, 461–463
VLAN routing, 802.1Q trunking, 457–464
**roles**
identifying, 266–268
STP, 238
**rollover cabling, console connections, 93–94**
**root costs, STP, 266–268**
**root election, STP/RSTP, 231**
**Root Guard, STP, 250–251, 278–279**
**root ports, choosing, 232–234**
**root switches**
designated ports, choosing for LAN segments, 234–235
distribution switches as, 259
election process, 231
identifying, 263–266
root ports, choosing, 232–234
STP/RSTP, 229
**routers**
802.1Q trunking, VLAN routing, 454, 457–464
ABR, 556
Aux ports, 423
backbone routers, 556
CLI, accessing, 414–415
default routers, 73
DHCP, client configurations, 496–497
encapsulating/de-encapsulating IP packets, 68–69
enterprise routers, installing, 408–409, 412
*Catalyst Edge Platform, 411*
*IOS, 409–410*
*IOS XE, 409–410*
*ISR, 410–411*
*OS, 409–410*
interfaces, 415
*bandwidth, serial interfaces, 423*
*clock rates, serial interfaces, 423*
*Ethernet interface autonegotiation, 420–422*
*IPv4 addressing, 418–419*
*listing, 416–417*
*serial interfaces, 423*
*status codes, 417–418*
internal routers, 556
IPv4 support, 413–423
IPv6 addressing
*anycast addresses, 672–689*
*configuration summary, 690–691*
*dual stacks, 671*

*LLA, 680–684*
*loopback addresses, 689*
*modified EUI-64, 674–679, 682–683*
*multicast addresses, 684–688*
*scopes, 686–687*
*solicited-node multicast addresses, 684, 687–688*
*special addresses, 680*
*unicast IP addresses, 670–671*
*unicast IP addresses, dynamic, 679–680*
*unicast IP addresses, static, 671–679*
*unspecified addresses, 689*
ISR, 410–411
Layer 3 Switch Routed Ports
*Layer 3 EtherChannels, 473–477*
*routed interfaces, 470–473*
*VLAN routing, 455, 469–477*
LSA, 558
MAC address tables, 481
next-hop routers, 434, 441, 442–443
ROAS, 454
*configuring, 458–461*
*troubleshooting, 463–464*
*verifying configurations, 461–463*
Router LAN Switch Ports
*embedded switch ports, 478–482*
*VLAN routing, 455, 477–482*
RS, 702–704
SOHO routers, installing, 412–413
WAN data links, 67–69
**routing, IPv4, 71–72, 428**
addressing, 76–77
*configuring with connected routes, 435–439*
*grouping addresses, 76*
*header fields, 76–77*
*ip address command, 436–439*
*network advertisements, 78*
*subnetting, 76*
ARP tables, 439–440
basics, 28–29
BDR, 552–553
connected routes, 426–427
default routers, 73
Dijkstra SPF algorithm, 547–548, 553–555
dynamic routing protocols
*convergence, 541, 542*
*EIGRP, 543–545*
*features, 540*
*functions, 541–542*
*ICMPv6, 642*
*IGP, 542–545*
*IGRP, 543*
*NDP, 642*
*OSPF, 543–545*
*OSPF, AD, 623*
*OSPF, areas, 555–557*
*OSPF, BDR, 552–553*
*OSPF, broadcast networks, 586–592*
*OSPF, concepts, 546*
*OSPF, design terminology, 556*
*OSPF, Dijkstra SPF algorithm, 553–555*
*OSPF, DR/BDR elections, 590–592*
*OSPF, ECMP routing, 621–622*
*OSPF, equal-cost routes, 621–622*
*OSPF, Hello packets, 548–549*
*OSPF, implementing with interface subcommands, 576*

*OSPF, interface configurations, 575–578*
*OSPF, IP forwarding, 625–628*
*OSPF, IP routing table, 628–630*
*OSPF, longest prefix matching, 625–630*
*OSPF, LSA, 546–548, 557–560*
*OSPF, LSDB, 546, 547–548, 550–553*
*OSPF, MTU, 618*
*OSPF, multi-area OSPF, 574–575*
*OSPF, neighbor adjacencies, 612–617*
*OSPF, neighbor priorities, 618–619*
*OSPF, neighbor requirements, 611–612*
*OSPF, neighbors, 546, 548–553*
*OSPF, neighbors with missing routes, 619–621*
*OSPF, network types, 618*
*OSPF, operation of, 546*
*OSPF, point-to-point networks, 586, 592–594*
*OSPF, priority, 590–592*
*OSPF, RID, 548*
*OSPF, route calculation, 553–555*
*OSPF, route selection, 621–630*
*OSPF, shutting down, 615–617*
*OSPF, topologies, 546–547*
*OSPF, troubleshooting routing, 618–621*
*OSPFv2, Dead intervals, 602–604, 614–615*
*OSPFv2, default routes, 597–599*
*OSPFv2, Hello intervals, 602–604, 614–615*
*OSPFv2, interface bandwidth, 599, 600*
*OSPFv2, metrics (cost), 599–601*
*OSPFv2, network commands, 564–568*
*OSPFv2, passive interfaces, 594–597*
*OSPFv2, reference bandwidth, 599, 600–601*
*OSPFv2, verification commands, 568–573*
*OSPFv3, 642*
*path selection, 540*
*RIPv2, 543–545*
ECMP routing, 621–622
encapsulating/de-encapsulating IP packets, 68–69
equal-cost routes, 621–622
Ethernet WAN, 70–71
example of, 431–435
flooding, 546
host routing logic (summary), 429
interarea routes, 556
intra-area routes, 556
ip address command, 436–439
LAN, 67–69, 73–75
logic, 72–73
OSPF, troubleshooting, 618–621
OSPFv2, default routes, 597–599
process reference, 429–431
protocols, 77–78
route selection, OSPF, 621–630
routing logic (summary), 430–431
static routes, 426–427
*configuring, 440–443*
*Ethernet outgoing interfaces, 443*
*floating static routes, 447–448*
*proxy ARP, 443*
*static default routes, 443–445*
*static host routes, 445–447*

*static network routes, 441–442*
*troubleshooting, 443–445*
tables, 73
troubleshooting
*IP addresses, 522–524*
*name resolution, 522–524*
*ping command, 512, 513–524*
*SSH, 527–530*
*Telnet, 527–530*
*testing LAN neighbors, 519–521*
*testing longer routes, 514–517*
*testing reverse routes, 517–519*
*testing WAN neighbors, 521*
*traceroute command, 512, 524–527*
VLAN, 195–197
*Layer 3 Switch Routed Ports, 455, 469–477*
*Layer 3 Switch SVI, 455, 464–469*
*ROAS, 454*
*ROAS, configuring, 458–461*
*ROAS, troubleshooting, 463–464*
*ROAS, verifying configurations, 461–463*
*Router 802.1Q Trunks, 454, 457–464*
*Router LAN Switch Ports, 455, 477–482*
WAN, 73–75
**routing, IPv6, 643–645**
abbreviating addresses, 647, 648–649
AD
*connected routes, 727*
*static routes, 739–741*
address formats/conventions, 646
address prefix lengths, 649
connected routes, 724–727
EIGRPv6, 645–646
expanding addresses, 648–649
floating static routes, 739–741
full (unabbreviated) addresses, 646–647
hexadecimal/binary conversion chart, 647
local routes, 724–725, 727–728
MP BGP-4, 645–646
OSPFv3, 645
RIPng, 645
static default routes, 735–737
static host routes, 737–739
static network routes, 728
*logic, 728–729*
*next-hop addresses, 730–735*
*outgoing interfaces, 729–730*
*troubleshooting, 741–743*
subnet prefixes, 649–652
**RPVST+ (Rapid Per VLAN Spanning Tree Plus), 260, 261, 266**
**RS (Router Solicitations), 702–704**
**RSTP (Rapid Spanning Tree Protocol), 222, 236, 239–240, 260, 261, 266**
alternate ports, 241–243
basics, 224–225
BID, 229–231
blocking state, 225, 229
broadcast storms, 225–227
configurable priority values, 262
configuring, 259
designated switches, 229
discarding state, 243
DP, 229
forwarding state, 225, 229
frames, multiple frame transmissions, 227
Hello BPDU, 230–231

link types, 245
looping frames, 225–227
MAC tables, 227
need for, 225–227
operation of, 228–235
ports
*alternate ports, 241*
*backup ports, 241, 244*
*edge ports, 245*
*point-to-point ports, 245*
*roles, 241*
*states, 243–244*
*types of ports, 245–246*
role of, 227–228
root switches, 229
*designated ports, choosing for LAN segments, 234–235*
*election process, 231*
*root ports, choosing, 232–234*
STA, 228
STP comparisons, 240–241, 243–244
topological influence, configuring, 235–236
**RSTP (Rapid Spanning Tree Protocol). *See also* STP**
**running-config files, 107–108**


# S


**same-layer interaction, TCP/IP networking models, 25–26**
**scopes, multicast IPv6 addresses, 686–687**
**security**
CLI, 98–99, 134–135
passwords
*basic password configurations, 137–138*
*CLI, 98–99*
*console passwords, 135–139*
*enable passwords, 135–139*
*enable secret passwords, 137*
*privileged mode, switches, 135–139*
*shared passwords, 135–139*
*simple security concepts, 135*
*switch security, user mode, 139–141*
*Telnet, 135–139*
*user mode, switches, 135–139*
*vty passwords, 135, 137*
switches
*CLI, 134–135*
*privileged mode, simple passwords, 135–139*
*user mode, external authentication servers, 141–142*
*user mode, local usernames/passwords, 139–141*
*user mode, simple passwords, 135–139*
**segments, 25–26, 31, 32**
**selecting paths. *See* routing, IPv4**
**serial interfaces, routers, 423**
**serial links/lines. *See* leased-line WAN**
**servers**
authentication servers, switch security, user mode, 141–142
DHCP servers
*preconfiguration, 493*
*stored information, 492–493*
DNS server lists, 500
**services (Layer 2), Ethernet WAN, 70**
**shared media, Ethernet, 60**
**shared passwords, 135–139**
**show command, 103**
**show interfaces command, 124**

**shutdown command, switch interface configuration, 172–173**
**shutting down, OSPF, 615–617**
**single switch topologies, learning MAC addresses, 122–123**
**single-area OSPF, 555, 565–566**
**single-mode fiber, 51–53**
**SLAAC (Stateless Address Autoconfiguration), 710–714**
**slash masks. *See* prefix masks**
**software configuration, Cisco IOS, 103–104**
**SOHO (Small Office/Home Office)**
LAN, 39
routers, installing, 412–413
**solicited-node multicast addresses, 684, 687–688**
**spanning trees (multiple), need for, 260**
**special IPv6 addresses, IPv6 routing, 680**
**speed of switches, manually setting, 169–170**
**SPF algorithm**
LSDB, 547–548
route calculation, 553–555
**SSH (Secure Shell)**
Cisco IOS XE CLI access, 96
IPv4 routing, 527–530
status, displaying, 145
switch security, remote access, 142–145
**STA (Spanning Tree Algorithm), 228**
**startup-config files, 108–109**
**stateful DHCPv6, 706–710**
**stateless DHCPv6, 707, 710, 711–712**
**states, identifying, 266–268**
**static access interfaces, 197–198**
**static ranges, choosing for subnetting, 321–322**
**static routes, 426–427**
configuring, 440–443
Ethernet outgoing interfaces, 443
floating static routes, 447–448, 739–741
IPv6 network routing
*floating static routes, 739–741*
*logic, 728–729*
*next-hop addresses, 730–735*
*outgoing interfaces, 729–730*
*static default routes, 735–737*
*static host routes, 737–739*
*static network routes, 728–735*
*troubleshooting, 741–743*
proxy ARP, 443
static default routes, 443–445
static host routes, 445–447
static network routes, 441–442
troubleshooting, 448–450
**static unicast IP addresses, IPv6 routing, 671–679**
**status codes**
router interfaces, 417–418
switch interface configurations, 176–177
**storing switch configurations, 106–108**
**STP (Spanning Tree Protocol), 222, 236, 261**
basics, 224–225
BID, 229–231, 261–262
blocking state, 120–121, 225, 229, 238–239
BPDU Filter, 248–250, 274–278
BPDU Guard, 248, 269–274
broadcast storms, 225–227
broken state, 251
channel-group configuration command, 281–284
configurable priority values, 262

convergence management, 237–238
designated switches, 229
disabled state, 239
DP, 229
EtherChannel, 246
*load distribution, 289–291*
EtherChannel configurations
*dynamic EtherChannels, 284–287*
*interface configuration consistency, 287–289*
*Layer 2 EtherChannel, 281–284*
forwarding state, 120–121, 225, 229, 238–239
frames
*looping frames, 225–227*
*multiple frame transmissions, 227*
Hello BPDU, 230–231, 236–237
interface states, changing, 238–239
learning state, 239
listening state, 239
Loop Guard, 251–253, 279–280
loop prevention, 120–121
MAC tables, 227
MaxAge timers, 237, 238, 241
modes, 260–261
MSTP, 261
need for, 225
operation of, 228–235
port costs, 266–268
PortFast, 247–248, 269–274
ports, states, 243–244
PVST+260, 261
role of, 227–228
roles, 238
roles, identifying, 266–268
root costs, 266–268
Root Guard, 250–251, 278–279
root switches, 229
*designated ports, choosing for LAN segments, 234–235*
*election process, 231*
*root ports, choosing, 232–234*
RPVST+260, 261, 266
RSTP, 260, 261, 240–241, 243–244
STA, 228
stable network activity, 236–237
standards, 260–261
states, identifying, 266–268
superior BPDU, 251
system ID extensions, 261–262
timers, convergence management, 237–238
topological influence, configuring, 235–236
unidirectional links, 252
VLAN, load balancing, 260
**STP (Spanning Tree Protocol)**. ***See also* RSTP**
**straight-through cabling, pinouts, 47**
**study plans (exams), 753–754**
**submodes, CLI configuration, 104–106**
**subnetting**
broadcast addresses, 358, 359, 361
calculating based on IPv4 format, 349–350
classful addressing, 348
classless addressing, 348
defining, 358
DHCP Relay, 490–492
difficult subnet masks, 368, 369–371
existing subnets
*difficult subnet masks, 369–373*
*easy subnet masks, 367–369*
*practicing with, 373*

GUA, 659–664
ID/addresses, 308, 360
*binary math analysis, 362–366*
*finding in difficult subnet masks, 369–371*
IPv4, 73, 76, 302
*assigning to different locations, 320–321*
*choosing dynamic ranges, 321–322*
*choosing IP networks, 314*
*choosing static ranges, 321–322*
*classful networks, 311–314*
*defined, 302–303*
*defining size of subnets, 308–310*
*design choices, 309–311*
*design view, 303–304*
*determining number of hosts, 307*
*determining number of subnets, 305–307*
*forwarding, 626–628*
*DHCP, 321–322*
*grouping hosts, 304–305*
*listing subnets, 318–320*
*masks, 314–318*
*operational view, 303–304*
*planning implementations, 320*
*private IP networks, 313–314*
*public IP networks, 311–313*
IPv6 prefixes, 649–652
magic numbers, 370, 373–374
masks, 314–318
*binary masks, 340*
*binary masks, DDN mask conversions, 342–344*
*binary masks, prefix mask conversions, 341–342*
*Boolean AND/OR operations, 367*
*choosing to meet requirements, 380–386*
*CIDR blocks, 351–352*
*conversions, 340–345*
*DDN, 318, 341*
*DDN masks, 341, 342–344, 367, 370*
*dividing IP addresses, 346–348*
*finding ID/addresses, 386–396*
*formats of (overview), 340–341*
*identifying design choices, 345–346*
*magic numbers, 370, 373–374, 388–389*
*octet values, 342–343*
*practicing with, 350–351*
*predictability in interesting octets, 368–369*
*prefix masks, 341–344*
*zero subnet, 387–388*
masks, easy subnet masks, 367–369
range of usable addresses, 361, 367
resident subnets, 358–359
ULA, 665–666
unicast IP addresses, 358
VLSM, 309–311, 545
**summary LSA, 558**
**superior BPDU, 251**
**SVI (Switch Virtual Interfaces), 147**
autostate setting, 467–469
Layer 3 Switch SVI
*configuring, 464–466*
*troubleshooting, 467–469*
*verifying configurations, 466*
*VLAN routing, 455, 464–469*
**switches**
10/100 ports, autonegotiation, 161, 162
10/100/1000 ports, autonegotiation, 161, 162, 163

access switches, 259
auto-MDIX, 170–172
autonegotiation, 161–169
BID, 261–262
broadcast domains, 191–192
Cisco Catalyst switches, 90
*9200 model switches, 91, 94*
*CLI, accessing, 90*
Cisco Meraki switches, 90
Cisco Nexus switches, 90
configuring
*common configuration modes, 105–106*
*copying switch configurations, 109*
*erasing switch configurations, 109*
*storing switch configurations, 106–108*
default gateways, 148–149
designated switches, 229
DHCP, learning IP addresses, 150
DHCP client configurations, 495–496
distribution switches, as root switches, 259
DNS, 149, 152–153
duplex, manually setting, 169–170
embedded switch ports
*configuring, 478–480*
*identifying, 481–482*
*verifying routing, 480–481*
erasing configurations, 109
Ethernet frame transmissions, 57–60
Ethernet LAN
*analyzing, 121–122*
*finding MAC addresses in tables, 124–126*
*flooding unknown unicast/broadcast frames, 119–120*
*forwarding known unicast frames, 116–118*
*forward-versus filter decisions, 118*
*interfaces, 123–124*
*learning MAC addresses, 118–119, 122–123*
*loop prevention, 120–121*
*MAC address tables, 116–118*
*MAC addresses with multiple switches, 127–128*
*managing MAC address tables, 126–127*
*summary, 121*
*verifying, 121–123*
exec-timeout command, 152–153
flooding, 119–120
history buffer commands, 151–152
interface configurations, 123–124, 172
*administratively controlling interface state, 172–173*
*description command, 172–173*
*duplex mismatches, 177–178*
*interface range command, 172–173*
*late collisions, 163*
*layer 1 problems, 178–180*
*removing, 174–176*
*shutdown command, 172–173*
*status codes, 176–177*
*VLAN assignments, 197–201*
interface states, 234–235
IP settings, 147–149
IP telephony ports, 208, 212
IPv4 configurations, 149–151
LAN switches
*concepts, 114*
*interface ID, 91*
*logic, 115–116*

Layer 3 Switch SVI
*autostate setting, 467–469*
*configuring, 464–466*
*troubleshooting, 467–469*
*verifying configurations, 466*
*VLAN routing, 455, 464–469*
learning, 118–119
log messages, 141
logging synchronous command, 152–153
MAC addresses
*with multiple switches, 127–128*
*tables, 103*
no ip domain-lookup command, 152–153
port costs, 266–268
priority, identifying, 263–266
remote access
*IPv4, 146–147*
*SSH, 142–145*
roles, identifying, 266–268
root costs, 266–268
root switches
*designated ports, choosing for LAN segments, 234–235*
*distribution switches as, 259*
*election process, 231*
*identifying, 263–266*
*root ports, choosing, 232–234*
*STP/RSTP, 229*
routed interfaces, 470–473
Router LAN Switch Ports
*embedded switch ports, 478–482*
*VLAN routing, 455, 477–482*
security
*CLI, 134–135*
*privileged mode, simple passwords, 135–139*
*user mode, simple passwords, 135–139*
single switch topologies, learning MAC addresses, 122–123
speed, manually setting, 169–170
states, identifying, 266–268
storing configurations, 106–108, 109
SVI, 147
syslog messages, 149–150, 152
user mode
*external authentication servers, 141–142*
*local usernames/passwords, 139–141*
VLAN
*access interfaces, 197–198*
*assigning to interfaces, 213*
*broadcast domains, 191–192*
*data VLAN, 209–212*
*disabled/undefined VLAN, 212–218*
*IP settings, 147–149*
*native VLAN, 195*
*static access interfaces, 197–198*
*troubleshooting, 212–218*
*trunking, 192–195, 201–207, 215–218*
*undefined/disabled VLAN, 213–215*
*voice VLAN, 209–212*
*VTP, 194, 201–202*
WebUI, security, 145–146
**switching tables. *See* MAC addresses**
**syslog messages, switches, 149–150, 152**
**system ID extensions, 261–262**

# T

**T1. *See* leased-line WAN**
**TCP/IP networking models**
adjacent-layer interaction, 25–26
application layer, 23
*accessing web pages, 23*
*HTTP, 23–24*
data encapsulation, 31–32
data-link layer, 29–31
development of, 20–21
error recovery, 25
history of, 20–21
hostnames, 79
messages, names of, 31–32
network layer, 22, 26
*IP, postal service analogy, 26–27*
*IP addressing, basics, 27–28*
*IP routing, basics, 28–29*
OSIv comparisons, 33–34
overview of, 21–23
physical layer, 29–31
protocols, 22–23
RFC, 21
same-layer interaction, 25–26
transport layer, 24
*adjacent-layer interaction, 25–26*
*error recovery, 25*
*same-layer interaction, 25–26*
**technical content (exams), additional content, 751**
**telcos, leased-line WAN, 65–66**
**Telnet**
Cisco IOS XE CLI access, 96
IPv4 routing, 527–530
passwords, 135–139
**temporary IPv6 addresses, 712–714**
**terminal emulators, configuring, 95**
**testing**
IPv6 connectivity, 716–718
LAN neighbors, 519–521
longer routes, 514–517
reverse routes, 517–519
WAN neighbors, 521
**timers, STP convergence management, 237–238**
**topologies**
OSPF, 546–547
single switch topologies, learning MAC addresses, 122–123
STP/RSTP configuration, 235–236
**traceroute command**
IPv4 routing, 512, 524–527
IPv6 connectivity, 716–718
**tracert command**
basics, 524–525
extended traceroute, 526–527
operation of, 525–526
problem isolation, 524
**trailer fields, Ethernet frames, 53–54**
**transceivers, Ethernet, 45**
**transmitting data**
optical transmitters, 50–51
single-mode fiber cabling, 51
UTP, 43–44
**transport layer, TCP/IP networking models, 24**
adjacent-layer interaction, 25–26
error recovery, 25
same-layer interaction, 25–26
**troubleshooting**
host settings, IPv4 addressing, 506–509
IPv4 routing
*IP addresses, 522–524*
*name resolution, 522–524*
*ping command, 512, 513–524*

*SSH, 527–530*
*Telnet, 527–530*
*testing LAN neighbors, 519–521*
*testing longer routes, 514–517*
*testing reverse routes, 517–519*
*testing WAN neighbors, 521*
*traceroute command, 512, 524–527*
IPv6 addressing, 714–718
Layer 3 Switch SVI, 467–469
OSPF
*MTU, 618*
*neighbor adjacencies, 612–617*
*neighbor priorities, 618–619*
*neighbors with missing routes, 619–621*
*network types, 618*
*routing, 618–621*
priority 0, OSPF neighbors, 618–619
ROAS, 463–464
static routes
*IPv4 routing, 443–445*
*IPv6 routing, 741–743*
trunking, 212–218
VLAN, 212–218
**trunking**
802.1Q trunking, 194–195, 454, 457–464
administrrative mode options, 203
dynamic auto mode, 203–204
dynamic desirable mode, 203
ISL VLAN trunking, 194–195
operational mode options, 203, 207
troubleshooting, 212–218
VLAN, 192–195, 201–207, 215–218
**tunneling, BPDU, 266**

# U

**ULA (Unique Local Addresses), 656, 664–666**
**unabbreviated (full) IPv6 addresses, 646–647**
**undefined/disabled VLAN, 213–215**
**unicast Ethernet addresses, 54–55**
**unicast frames**
flooding unknown unicast frames, 119–120
forwarding, Ethernet LAN switching, 116–118
**unicast IP addresses, 358**
CIDR blocks, 327
GUA, 655–664
IPv6 routing, 670–671
**unidirectional links, STP, 252**
**unknown broadcast/unicast frames, flooding, 119–120**
**unspecified IPv6 addresses, IPv6 routing, 689**
**updates, exams, 751–752**
**user interface. *See* WebUI**
**user mode**
Cisco IOS XE CLI access, 96–98
rejected commands example, 97–98
switch security
*external authentication servers, 141–142*
*local usernames/passwords, 139–141*
*remote access, IPv4, 146–147*
*remote access, SSH, 142–145*
*simple passwords, 135–139*
**usernames, switch security, 139–141**

**UTP (Unshielded Twisted-Pair) cabling**
10BASE-T pinouts, 46–47
100BASE-T pinouts, 46–47
1000BASE-T pinouts, 49
cabling comparisons, 52–53
data transmission, 43–44
Ethernet LAN builds, 43–49
Ethernet links, 44–46

# V

**valid lifetimes, SLAAC, 712–713**
**verifying**
broadcast network operations, 588–590
data VLAN, 209–212
embedded switch port routing, 480–481
Ethernet LAN switching, 121–123
host connectivity, IPv6 addressing, 718–719
IPv4 configurations, switches, 150–151
Layer 3 Switch SVI configurations, 466
OSPFv2 verification commands, 568–573
ROAS configurations, 461–463
static network routes, 442
VLAN
*data VLAN, 209–212*
*voice VLAN, 209–212*
voice VLAN, 209–212
**VLAN (Virtual LAN), 188.** ***See also*** **LAN; WLAN**
access interfaces, 197–198
assigning to interfaces, 197–201, 213
broadcast domains, 191–192
configuring, 197–201
creating, 197–201
data VLAN
*configuring, 209–212*
*verifying, 209–212*
disabled/undefined VLAN, 212–218
forwarding data between VLAN, 195–197
interfaces, 464–466
load balancing with STP, 260
native VLAN, 195
phone connections, 207–212
PVST+260, 261
routing
*Layer 3 Switch Routed Ports, 455, 469–477*
*Layer 3 Switch SVI, 455, 464–469*
*ROAS, 454*
*ROAS, configuring, 458–461*
*ROAS, troubleshooting, 463–464*
*ROAS, verifying configurations, 461–463*
*Router 802.1Q Trunks, 454, 457–464*
*Router LAN Switch Ports, 455, 477–482*
routing between VLAN, 195–197
RPVST+260, 261
static access interfaces, 197–198
switches, IP settings, 147–149
troubleshooting, 212–218
trunking, 192–195, 201–207, 215–218
undefined/disabled VLAN, 213–215
voice VLAN
*configuring, 209–212*
*verifying, 209–212*
VTP, 194, 201–202

**VLSM (Variable-Length Subnet Masks), 309–311, 545**
**voice VLAN**
configuring, 209–212
verifying, 209–212
**VTP (VLAN Trunking Protocol), 194, 201–202**
**vty passwords, 135, 137**

## W

**WAN (Wide Area Networks), 36, 64**
Ethernet WAN, 64, 69
*IP routing, 70–71*
*Layer 2 services, 70*
leased-line WAN, 64–65
*conceptual view, 65*
*data-link details, 66–69*
*different names for, 65–66*
*HDLC, 66–67*
*IP routing, 67–69*
*physical details, 65–66*
*PPP, 66–67*
*routers, 67–69*
*telcos, 65–66*
neighbors, testing, 521
network layer routing (forwarding), 73–75
**web pages**
accessing from, application layer, TCP/IP networking models, 23
HTTP, 24
**WebUI (User Interface)**
CLI access, 99–101
security, 145–146
**WHOIS records, 328**
**wildcard characters, matching with OSPFv2 network commands, 566–568**
**Windows IP host settings, 499–502**
**wired LAN. *See* Ethernet LAN**
**WLAN (Wireless LAN), 18–19**

## X - Y - Z

**zero subnet, 387–388**

*This page intentionally left blank*



Register your product at **ciscopress.com/register** to unlock additional benefits:

- Save 35%* on your next purchase with an exclusive discount code
- Find companion files, errata, and product updates if available
- Sign up to receive special offers on new editions and related titles

Get more when you shop at **ciscopress.com**:

- Everyday discounts on books, eBooks, video courses, and more
- Free U.S. shipping on all orders
- Multi-format eBooks to read on your preferred device
- Print and eBook Best Value Packs

**Discount code valid for 30 days; may not be combined with any other offer and is not redeemable for cash. Offer subject to change.*

**Cisco Press**

# APPENDIX D

# Practice for Chapter 12: Analyzing Classful IPv4 Networks

## Practice Problems

The practice problems in this appendix require that you determine a few basic facts about a network, given an IP address and an assumption that subnetting is not used in that network. To do so, refer to the processes described in Chapter 12 of *CCNA 200-301 Official Cert Guide, Volume 1*.

**NOTE** You may also elect to do this same set of practice problems using the "Practice Exercise: Analyzing Classful IPv4 Networks" application on the companion website.

In particular, for the upcoming list of IP addresses, you should identify the following information:

- Class of the address
- Number of octets in the network part of the address
- Number of octets in the host part of the address
- Network number
- Network broadcast address

Find all these facts for the following IP addresses:

1. 10.55.44.3
2. 128.77.6.7
3. 192.168.76.54
4. 190.190.190.190
5. 9.1.1.1
6. 200.1.1.1
7. 201.1.77.5
8. 101.1.77.5
9. 119.67.99.240
10. 219.240.66.98

## Answers

The process to answer these problems is relatively basic, so this section reviews the overall process and then lists the answers to problems 1–10.

The process starts by examining the first octet of the IP address:

- If the first octet of the IP address is a number between 1 and 126, inclusive, the address is a Class A address.
- If the first octet of the IP address is a number between 128 and 191, inclusive, the address is a Class B address.
- If the first octet of the IP address is a number between 192 and 223, inclusive, the address is a Class C address.

When no subnetting is used:

- Class A addresses have one octet in the network part of the address and three octets in the host part.
- Class B addresses have two octets each in the network and host part.
- Class C addresses have three octets in the network part and one octet in the host part.

After determining the class and the number of network octets, you can easily find the network number and network broadcast address. To find the network number, copy the network octets of the IP address and write down 0s for the host octets. To find the network broadcast address, copy the network octets of the IP address and write down 255s for the host octets.

Table D-1 lists all ten problems and their respective answers.

**Table D-1** Answers to Problems

| IP Address | Class | Number of Network Octets | Number of Host Octets | Network Number | Network Broadcast Address |
|---|---|---|---|---|---|
| 10.55.44.3 | A | 1 | 3 | 10.0.0.0 | 10.255.255.255 |
| 128.77.6.7 | B | 2 | 2 | 128.77.0.0 | 128.77.255.255 |
| 192.168.76.54 | C | 3 | 1 | 192.168.76.0 | 192.168.76.255 |
| 190.190.190.190 | B | 2 | 2 | 190.190.0.0 | 190.190.255.255 |
| 9.1.1.1 | A | 1 | 3 | 9.0.0.0 | 9.255.255.255 |
| 200.1.1.1 | C | 3 | 1 | 200.1.1.0 | 200.1.1.255 |
| 201.1.77.55 | C | 3 | 1 | 201.1.77.0 | 201.1.77.255 |
| 101.1.77.55 | A | 1 | 3 | 101.0.0.0 | 101.255.255.255 |
| 119.67.99.240 | A | 1 | 3 | 119.0.0.0 | 119.255.255.255 |
| 219.240.66.98 | C | 3 | 1 | 219.240.66.0 | 219.240.66.255 |

# APPENDIX E

# Practice for Chapter 13: Analyzing Subnet Masks

This appendix begins with 23 mask conversion problems, followed by the matching answers and explanations. After that, the appendix lists 10 mask analysis problems, with the matching answers to follow.

**NOTE** You may also perform this same set of practice problems using the "Analyzing Subnet Masks" and "Mask Conversion" applications on the companion website.

## Mask Conversion Problems

The problems in this appendix require you to convert dotted-decimal subnet masks to prefix format and vice versa. To do so, feel free to use the processes described in Chapter 13 of *CCNA 200-301 Official Cert Guide, Volume 1*.

Many people use the information in Table E-1 when converting masks. The table lists the nine dotted-decimal notation (DDN) mask values, the binary equivalent, and the number of binary 1s in the binary equivalent.

**Table E-1** Nine Possible Values in One Octet of a Subnet Mask

| Binary Mask Octet | DDN Mask Octet | Number of Binary 1s |
|---|---|---|
| 00000000 | 0 | 0 |
| 10000000 | 128 | 1 |
| 11000000 | 192 | 2 |
| 11100000 | 224 | 3 |
| 11110000 | 240 | 4 |
| 11111000 | 248 | 5 |
| 11111100 | 252 | 6 |
| 11111110 | 254 | 7 |
| 11111111 | 255 | 8 |

Convert each DDN mask to prefix format and vice versa:

1. 255.240.0.0
2. 255.255.192.0
3. 255.255.255.224
4. 255.254.0.0.

5. 255.255.248.0
6. /30
7. /25
8. /11
9. /22
10. /24
11. 255.0.0.0
12. /29
13. /9
14. 255.192.0.0
15. 255.255.255.240
16. /26
17. /13
18. 255.255.254.0
19. 255.252.0.0
20. /20
21. /16
22. 255.255.224.0
23. 255.255.128.0

# Answers to Mask Conversion Problems

## Mask Conversion Problem 1: Answer

The answer is /12.

The binary process for converting the mask from dotted-decimal format to prefix format is relatively simple. The only hard part is converting the dotted-decimal number to binary. For reference, the process is as follows:

**Step 1.** Convert the dotted-decimal mask to binary.

**Step 2.** Count the number of binary 1s in the 32-bit binary mask; this is the value of the prefix notation mask.

For problem 1, mask 255.240.0.0 converts to the following:

11111111 11110000 00000000 00000000

You can see from the binary number that it contains 12 binary 1s, so the prefix format of the mask will be /12.

You can find the same answer without converting decimal to binary if you have memorized the nine DDN mask values, and the corresponding number of binary 1s in each, as listed earlier in Table E-1. Follow these steps:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 4 because the second mask octet of 240 includes four binary 1s.

**Step 4.** The resulting prefix is /12.

## Mask Conversion Problem 2: Answer

The answer is /18.

For problem 2, mask 255.255.192.0 converts to the following:

11111111 11111111 11000000 00000000

You can see from the binary number that it contains 18 binary 1s, so the prefix format of the mask will be /18.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 2 because the third mask octet of 192 includes two binary 1s.

**Step 5.** The resulting prefix is /18.



## Mask Conversion Problem 3: Answer

The answer is /27.

For problem 3, mask 255.255.255.224 converts to the following:

11111111 11111111 11111111 11100000

You can see from the binary number that it contains 27 binary 1s, so the prefix format of the mask will be /27.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 8 because the third mask octet of 255 includes eight binary 1s.

**Step 5.** (4th octet) Add 3 because the fourth mask octet of 224 includes three binary 1s.

**Step 6.** The resulting prefix is /27.

### Mask Conversion Problem 4: Answer

The answer is /15.

For problem 4, mask 255.254.0.0 converts to the following:

11111111 11111110 00000000 00000000

You can see from the binary number that it contains 15 binary 1s, so the prefix format of the mask will be /15.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 7 because the second mask octet of 254 includes seven binary 1s.

**Step 4.** The resulting prefix is /15.

### Mask Conversion Problem 5: Answer

The answer is /21.

For problem 5, mask 255.255.248.0 converts to the following:

11111111 11111111 11111000 00000000

You can see from the binary number that it contains 21 binary 1s, so the prefix format of the mask will be /21.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 5 because the third mask octet of 248 includes five binary 1s.

**Step 5.** The resulting prefix is /21.

### Mask Conversion Problem 6: Answer

The answer is 255.255.255.252.

The binary process for converting the prefix version of the mask to dotted-decimal is straightforward, but again requires some binary math. For reference, the process runs like this:

**Step 1.** Write down $x$ binary 1s, where $x$ is the value listed in the prefix version of the mask.

**Step 2.** Write down binary 0s after the binary 1s until the combined 1s and 0s form a 32-bit number.

**Step 3.** Convert this binary number, 8 bits at a time, to decimal, to create a dotted-decimal number; this value is the dotted-decimal version of the subnet mask. (Refer to Table E-1, which lists the binary and decimal equivalents.)

For problem 6, with a prefix of /30, you start at Step 1 by writing down 30 binary 1s, as shown here:

11111111 11111111 11111111 111111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 11111111 11111**00**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

## Mask Conversion Problem 7: Answer

The answer is 255.255.255.128.

For problem 7, with a prefix of /25, you start at Step 1 by writing down 25 binary 1s, as shown here:

11111111 11111111 11111111 1

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 11111111 **1**0000000

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

## Mask Conversion Problem 8: Answer

The answer is 255.224.0.0.

For problem 8, with a prefix of /11, you start at Step 1 by writing down 11 binary 1s, as shown here:

11111111 111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 111**00000 00000000 00000000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

## Mask Conversion Problem 9: Answer

The answer is 255.255.252.0.

For problem 9, with a prefix of /22, you start at Step 1 by writing down 22 binary 1s, as shown here:

11111111 11111111 111111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 11111**100 00000000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

### Mask Conversion Problem 10: Answer

The answer is 255.255.255.0.

For problem 10, with a prefix of /24, you start at Step 1 by writing down 24 binary 1s, as shown here:

11111111 11111111 11111111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 11111111 **00000000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

### Mask Conversion Problem 11: Answer

The answer is /8.

For problem 11, mask 255.0.0.0 converts to the following:

11111111 00000000 00000000 00000000

You can see from the binary number that it contains 8 binary 1s, so the prefix format of the mask will be /8.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 0 for the other octets because each mask octet of 0 includes zero binary 1s.

**Step 4.** The resulting prefix is /8.

### Mask Conversion Problem 12: Answer

The answer is 255.255.255.248.

For problem 12, with a prefix of /29, you start at Step 1 by writing down 29 binary 1s, as shown here:

11111111 11111111 11111111 11111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 11111111 11111**000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

## Mask Conversion Problem 13: Answer

The answer is 255.128.0.0.

For problem 13, with a prefix of /9, you start at Step 1 by writing down 9 binary 1s, as shown here:

11111111 1

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 10000000 00000000 00000000

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

## Mask Conversion Problem 14: Answer

The answer is /10.

For problem 14, mask 255.192.0.0 converts to the following:

11111111 11000000 00000000 00000000

You can see from the binary number that it contains 10 binary 1s, so the prefix format of the mask will be /10.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 2 because the second mask octet of 192 includes two binary 1s.

**Step 4.** The resulting prefix is /10.

## Mask Conversion Problem 15: Answer

The answer is /28.

For problem 15, mask 255.255.255.240 converts to the following:

11111111 11111111 11111111 11110000

You can see from the binary number that it contains 28 binary 1s, so the prefix format of the mask will be /28.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 8 because the third mask octet of 255 includes eight binary 1s.

**Step 5.** (4th octet) Add 4 because the fourth mask octet of 240 includes four binary 1s.

**Step 6.** The resulting prefix is /28.

### Mask Conversion Problem 16: Answer

The answer is 255.255.255.192.

For problem 16, with a prefix of /26, you start at Step 1 by writing down 26 binary 1s, as shown here:

11111111 11111111 11111111 11

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 11111111 11**000000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

### Mask Conversion Problem 17: Answer

The answer is 255.248.0.0.

For problem 17, with a prefix of /13, you start at Step 1 by writing down 13 binary 1s, as shown here:

11111111 11111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111**000 00000000 00000000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

### Mask Conversion Problem 18: Answer

The answer is /23.

For problem 18, mask 255.255.254.0 converts to the following:

11111111 11111111 11111110 00000000

You can see from the binary number that it contains 23 binary 1s, so the prefix format of the mask will be /23.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 7 because the third mask octet of 254 includes seven binary 1s.

**Step 5.** The resulting prefix is /23.

## Mask Conversion Problem 19: Answer

The answer is /14.

For problem 19, mask 255.252.0.0 converts to the following:

11111111 11111100 00000000 00000000

You can see from the binary number that it contains 14 binary 1s, so the prefix format of the mask will be /14.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 6 because the second mask octet of 252 includes six binary 1s.

**Step 4.** The resulting prefix is /14.



## Mask Conversion Problem 20: Answer

The answer is 255.255.240.0.

For problem 20, with a prefix of /20, you start at Step 1 by writing down 20 binary 1s, as shown here:

11111111 11111111 1111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 **1111**0000 00000000

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

## Mask Conversion Problem 21: Answer

The answer is 255.255.0.0.

For problem 21, with a prefix of /16, you start at Step 1 by writing down 16 binary 1s, as shown here:

11111111 11111111

At Step 2, you add binary 0s until you have 32 total bits, as shown next:

11111111 11111111 **00000000 00000000**

The only remaining work is to convert this 32-bit number to decimal, remembering that the conversion works with 8 bits at a time.

### Mask Conversion Problem 22: Answer

The answer is /19.

For problem 22, mask 255.255.224.0 converts to the following:

11111111 11111111 11100000 00000000

You can see from the binary number that it contains 19 binary 1s, so the prefix format of the mask will be /19.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 3 because the third mask octet of 224 includes three binary 1s.

**Step 5.** The resulting prefix is /19.

### Mask Conversion Problem 23: Answer

The answer is /17.

For problem 23, mask 255.255.128.0 converts to the following:

11111111 11111111 10000000 00000000

You can see from the binary number that it contains 17 binary 1s, so the prefix format of the mask will be /17.

If you memorized the number of binary 1s represented by each DDN mask value, you can possibly work faster with the following logic:

**Step 1.** Start with a prefix value of 0.

**Step 2.** (1st octet) Add 8 because the first mask octet of 255 includes eight binary 1s.

**Step 3.** (2nd octet) Add 8 because the second mask octet of 255 includes eight binary 1s.

**Step 4.** (3rd octet) Add 1 because the third mask octet of 128 includes one binary 1.

**Step 5.** The resulting prefix is /17.

## Mask Analysis Problems

This appendix lists problems that require you to analyze an existing IP address and mask to determine the number of network, subnet, and host bits. From that, you should calculate the number of subnets possible when using the listed mask in the class of network shown in the problem, as well as the number of possible host addresses in each subnet.

To find this information, you can use the processes explained in Chapter 13 of *CCNA 200-301 Official Cert Guide, Volume 1*. When doing the problems, Table E-1, earlier in this appendix, which lists all possible DDN mask values, can be useful.

Each row of Table E-2 lists an IP address and mask. For each row, complete the table. Note that for the purposes of this exercise you can assume that the two special subnets in each network, the zero subnet and broadcast subnet, are allowed to be used.

**Table E-2** Mask Analysis Problems

| Problem Number | Problem | Network Bits | Subnet Bits | Host Bits | Number of Subnets in Network | Number of Hosts per Subnet |
|---|---|---|---|---|---|---|
| 1 | 10.66.5.99, 255.255.254.0 | | | | | |
| 2 | 172.16.203.42, 255.255.252.0 | | | | | |
| 3 | 192.168.55.55, 255.255.255.224 | | | | | |
| 4 | 10.22.55.87/30 | | | | | |
| 5 | 172.30.40.166/26 | | | | | |
| 6 | 192.168.203.18/29 | | | | | |
| 7 | 200.11.88.211, 255.255.255.240 | | | | | |
| 8 | 128.1.211.33, 255.255.255.128 | | | | | |
| 9 | 9.211.45.65/21 | | | | | |
| 10 | 223.224.225.226/25 | | | | | |

# Answers to Mask Analysis Problems

Table E-3 includes the answers to problems 1–10. The paragraphs following the table provide the explanations of each answer.

**Table E-3** Answers to Problems in This Appendix

| Problem Number | Problem | Network Bits | Subnet Bits | Host Bits | Number of Subnets in Network | Number of Hosts per Subnet |
|---|---|---|---|---|---|---|
| 1 | 10.66.5.99, 255.255.254.0 | 8 | 15 | 9 | $2^{15} = 32{,}768$ | $2^9 - 2 = 510$ |
| 2 | 172.16.203.42, 255.255.252.0 | 16 | 6 | 10 | $2^6 = 64$ | $2^{10} - 2 = 1022$ |
| 3 | 192.168.55.55, 255.255.255.224 | 24 | 3 | 5 | $2^3 = 8$ | $2^5 - 2 = 30$ |
| 4 | 10.22.55.87/30 | 8 | 22 | 2 | $2^{22} = 4{,}194{,}304$ | $2^2 - 2 = 2$ |
| 5 | 172.30.40.166/26 | 16 | 10 | 6 | $2^{10} = 1024$ | $2^6 - 2 = 62$ |
| 6 | 192.168.203.18/29 | 24 | 5 | 3 | $2^5 = 32$ | $2^3 - 2 = 6$ |
| 7 | 200.11.88.211, 255.255.255.240 | 24 | 4 | 4 | $2^4 = 16$ | $2^4 - 2 = 14$ |
| 8 | 128.1.211.33, 255.255.255.128 | 16 | 9 | 7 | $2^9 = 512$ | $2^7 - 2 = 126$ |
| 9 | 9.211.45.65/21 | 8 | 13 | 11 | $2^{13} = 8192$ | $2^{11} - 2 = 2046$ |
| 10 | 223.224.225.226/25 | 24 | 1 | 7 | $2^1 = 2$ | $2^7 - 2 = 126$ |

## Mask Analysis Problem 1: Answer

Address 10.66.5.99 is in Class A network 10.0.0.0, meaning that 8 network bits exist. Mask 255.255.254.0 converts to prefix /23, because the first 2 octets of value 255 represent 8 binary 1s, and the 254 in the third octet represents 7 binary 1s, for a total of 23 binary 1s. Therefore, the number of host bits is 32 – 23 = 9, leaving 15 subnet bits (32 – 8 network bits – 9 host bits = 15 subnet bits). The number of subnets in this Class A network, using mask 255.255.254.0, is $2^{15} = 32{,}768$. The number of hosts per subnet is $2^9 - 2 = 510$.

## Mask Analysis Problem 2: Answer

Address 172.16.203.42, mask 255.255.252.0, is in Class B network 172.16.0.0, meaning that 16 network bits exist. Mask 255.255.252.0 converts to prefix /22, because the first 2 octets of value 255 represent 8 binary 1s, and the 252 in the third octet represents 6 binary 1s, for a total of 22 binary 1s. Therefore, the number of host bits is 32 – 22 = 10, leaving 6 subnet bits (32 – 16 network bits – 10 host bits = 6 subnet bits). The number of subnets in this Class B network, using mask 255.255.252.0, is $2^6 = 64$. The number of hosts per subnet is $2^{10} - 2 = 1022$.

### Mask Analysis Problem 3: Answer

Address 192.168.55.55 is in Class C network 192.168.55.0, meaning that 24 network bits exist. Mask 255.255.255.224 converts to prefix /27, because the first 3 octets of value 255 represent 8 binary 1s, and the 224 in the fourth octet represents 3 binary 1s, for a total of 27 binary 1s. Therefore, the number of host bits is 32 – 27 = 5, leaving 3 subnet bits (32 – 24 network bits – 5 host bits = 3 subnet bits). The number of subnets in this Class C network, using mask 255.255.255.224, is $2^3 = 8$. The number of hosts per subnet is $2^5 – 2 = 30$.

### Mask Analysis Problem 4: Answer

Address 10.22.55.87 is in Class A network 10.0.0.0, meaning that 8 network bits exist. The prefix format mask of /30 lets you calculate the number of host bits as 32 – prefix length (in this case, 32 – 30 = 2). This leaves 22 subnet bits (32 – 8 network bits – 2 host bits = 22 subnet bits). The number of subnets in this Class A network, using mask 255.255.255.252, is $2^{22} = 4{,}194{,}304$. The number of hosts per subnet is $2^2 – 2 = 2$. (Note that this mask is popularly used on serial links, which need only two IP addresses in a subnet.)

### Mask Analysis Problem 5: Answer

Address 172.30.40.166 is in Class B network 172.30.0.0, meaning that 16 network bits exist. The prefix format mask of /26 lets you calculate the number of host bits as 32 – prefix length (in this case, 32 – 26 = 6). This leaves 10 subnet bits (32 – 16 network bits – 6 host bits = 10 subnet bits). The number of subnets in this Class B network, using mask /26, is $2^{10} = 1024$. The number of hosts per subnet is $2^6 – 2 = 62$.



### Mask Analysis Problem 6: Answer

Address 192.168.203.18 is in Class C network 192.168.203.0, meaning that 24 network bits exist. The prefix format mask of /29 lets you calculate the number of host bits as 32 – prefix length (in this case, 32 – 29 = 3). This leaves 5 subnet bits, because 32 – 24 network bits – 3 host bits = 5 subnet bits. The number of subnets in this Class C network, using mask /29, is $2^5 = 32$. The number of hosts per subnet is $2^3 – 2 = 6$.

### Mask Analysis Problem 7: Answer

Address 200.11.88.211 is in Class C network 200.11.88.0, meaning that 24 network bits exist. Mask 255.255.255.240 converts to prefix /28, because the first three octets of value 255 represent 8 binary 1s, and the 240 in the fourth octet represents 4 binary 1s, for a total of 28 binary 1s. This leaves 4 subnet bits (32 – 24 network bits – 4 host bits = 4 subnet bits). The number of subnets in this Class C network, using mask /28, is $2^4 = 16$. The number of hosts per subnet is $2^4 – 2 = 14$.

### Mask Analysis Problem 8: Answer

Address 128.1.211.33, mask 255.255.255.128, is in Class B network 128.1.0.0, meaning that 16 network bits exist. Mask 255.255.255.128 converts to prefix /25, because the first 3 octets of value 255 represent 8 binary 1s, and the 128 in the fourth octet represents 1 binary 1, for a total of 25 binary 1s. Therefore, the number of host bits is 32 – 25 = 7, leaving 9 subnet bits (32 – 16 network bits – 7 host bits = 9 subnet bits). The number of subnets in this Class B network, using mask 255.255.255.128, is $2^9 = 512$. The number of hosts per subnet is $2^7 – 2 = 126$.

### Mask Analysis Problem 9: Answer

Address 9.211.45.65 is in Class A network 10.0.0.0, meaning that 8 network bits exist. The prefix format mask of /21 lets you calculate the number of host bits as 32 – prefix length (in this case, 32 – 21 = 11). This leaves 13 subnet bits (32 – 8 network bits – 11 host bits = 13 subnet bits). The number of subnets in this Class A network, using mask /21, is $2^{13}$ = 8192. The number of hosts per subnet is $2^{11}$ – 2 = 2046.

### Mask Analysis Problem 10: Answer

Address 223.224.225.226 is in Class C network 223.224.225.0, meaning that 24 network bits exist. The prefix format mask of /25 lets you calculate the number of host bits as 32 – prefix length (in this case, 32 – 25 = 7). This leaves 1 subnet bit (32 – 24 network bits – 7 host bits = 1 subnet bit). The number of subnets in this Class C network, using mask /25, is $2^1$ = 2. The number of hosts per subnet is $2^7$ – 2 = 126.

# APPENDIX F

# Practice for Chapter 14: Analyzing Existing Subnets

## Practice Problems

This appendix lists practice problems related to Chapter 14, "Analyzing Existing Subnets." Each problem asks you to find a variety of information about the subnet in which an IP address resides. Each problem supplies an IP address and a subnet mask, from which you should find the following information:

- Subnet number
- Subnet broadcast address
- Range of valid IP addresses in this network

To find these facts, you can use any of the processes explained in Chapter 14.

In addition, these same problems can be used to review the concepts in Chapter 13, "Analyzing Subnet Masks." To use these same problems for practice related to Chapter 13, simply find the following information for each of the problems:

- Size of the network part of the address
- Size of the subnet part of the address
- Size of the host part of the address
- Number of hosts per subnet
- Number of subnets in this network

Feel free to either ignore or use the opportunity for more practice related to analyzing subnet masks.

Solve for the following problems:

1. 10.180.10.18, mask 255.192.0.0
2. 10.200.10.18, mask 255.224.0.0
3. 10.100.18.18, mask 255.240.0.0
4. 10.100.18.18, mask 255.248.0.0
5. 10.150.200.200, mask 255.252.0.0
6. 10.150.200.200, mask 255.254.0.0
7. 10.220.100.18, mask 255.255.0.0
8. 10.220.100.18, mask 255.255.128.0
9. 172.31.100.100, mask 255.255.192.0

10. 172.31.100.100, mask 255.255.224.0
11. 172.31.200.10, mask 255.255.240.0
12. 172.31.200.10, mask 255.255.248.0
13. 172.31.50.50, mask 255.255.252.0
14. 172.31.50.50, mask 255.255.254.0
15. 172.31.140.14, mask 255.255.255.0
16. 172.31.140.14, mask 255.255.255.128
17. 192.168.15.150, mask 255.255.255.192
18. 192.168.15.150, mask 255.255.255.224
19. 192.168.100.100, mask 255.255.255.240
20. 192.168.100.100, mask 255.255.255.248
21. 192.168.15.230, mask 255.255.255.252
22. 10.1.1.1, mask 255.248.0.0
23. 172.16.1.200, mask 255.255.240.0
24. 172.16.0.200, mask 255.255.255.192
25. 10.1.1.1, mask 255.0.0.0

# Answers

This section includes the answers to the 25 problems listed in this appendix. The answer section for each problem explains how to use the process outlined in Chapter 14 to find the answers. Also, refer to Chapter 13 for details on how to find information about analyzing the subnet mask.

## Answer to Problem 1

The answers begin with the analysis of the three parts of the address, the number of hosts per subnet, and the number of subnets of this network using the stated mask, as outlined in Table F-1. The binary math for subnet and broadcast address calculation follows. The answer finishes with the easier mental calculations for the range of IP addresses in the subnet.

**Table F-1** Question 1: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.180.10.18 | — |
| Mask | 255.192.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 22 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 2 | 32 – (network size + host size) |
| Number of subnets | $2^2 = 4$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{22} – 2 = 4{,}194{,}302$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-2 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-2** Question 1: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.180.10.18 | 00001010 10**110100 00001010 00010010** |
| Mask | 255.192.0.0 | 11111111 11**000000 00000000 00000000** |
| AND result (subnet number) | 10.128.0.0 | 00001010 10**000000 00000000 00000000** |
| Change host to 1s (broadcast address) | 10.191.255.255 | 00001010 10**111111 11111111 11111111** |

To get the first valid IP address, just add 1 to the subnet number; to get the last valid IP address, just subtract 1 from the broadcast address. In this case:

10.128.0.1 through 10.191.255.254

10.128.0.0 + 1 = 10.128.0.1

10.191.255.255 – 1 = 10.191.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. The key parts of the process are as follows:

- The interesting octet is the octet for which the mask's value is not a decimal 0 or 255.
- The magic number is calculated as the value of the IP address's interesting octet, subtracted from 256.
- The subnet number can be found by copying the IP address octets to the left of the interesting octet, by writing down 0s for octets to the right of the interesting octet, and by finding the multiple of the magic number closest to, but not larger than, the IP address's value in that same octet.
- The broadcast address can be similarly found by copying the subnet number's octets to the left of the interesting octet, by writing 255s for octets to the right of the interesting octet, and by taking the subnet number's value in the interesting octet, adding the magic number, and subtracting 1.

Table F-3 shows the work for this problem, with some explanation of the work following the table. Refer to Chapter 14 for the detailed processes.

**Table F-3** Question 1: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 | Comments |
|---|---|---|---|---|---|
| **Mask** | 255 | 192 | 0 | 0 | |
| **Address** | 10 | 180 | 10 | 18 | |
| **Subnet Number** | 10 | 128 | 0 | 0 | Magic number = 256 – 192 = 64 |
| **First Address** | 10 | 128 | 0 | 1 | Add 1 to last octet of subnet |
| **Last Address** | 10 | 191 | 255 | 254 | Subtract 1 from last octet of broadcast |
| **Broadcast** | 10 | 191 | 255 | 255 | 128 + 64 – 1 = 191 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 192 = 64 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 128 is the multiple of 64 that is closest to 180 but not higher than 180. So, the second octet of the subnet number is 128.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 128 + 64 – 1 = 191.

## Answer to Problem 2

**Table F-4** Question 2: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.200.10.18 | — |
| Mask | 255.224.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 21 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 3 | 32 – (network size + host size) |
| Number of subnets | $2^3 = 8$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{21} – 2 = 2{,}097{,}150$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-5 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-5** Question 2: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.200.10.18 | 00001010 11001000 00001010 00010010 |
| Mask | 255.224.0.0 | 11111111 11100000 00000000 00000000 |
| AND result (subnet number) | 10.192.0.0 | 00001010 11000000 00000000 00000000 |
| Change host to 1s (broadcast address) | 10.223.255.255 | 00001010 11011111 11111111 11111111 |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.192.0.1 through 10.223.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-6 shows the work for this problem, with some explanation of the work following the table.

**Table F-6** Question 2: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 | Comments |
|---|---|---|---|---|---|
| **Mask** | 255 | 224 | 0 | 0 | |
| **Address** | 10 | 200 | 10 | 18 | |
| **Subnet Number** | 10 | 192 | 0 | 0 | Magic number = 256 – 224 = 32 |
| **First Address** | 10 | 192 | 0 | 1 | Add 1 to last octet of subnet |
| **Last Address** | 10 | 223 | 255 | 254 | Subtract 1 from last octet of broadcast |
| **Broadcast** | 10 | 223 | 255 | 255 | 192 + 32 – 1 = 223 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 224 = 32 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 192 is the multiple of 32 that is closest to 200 but not higher than 200. So, the second octet of the subnet number is 192.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 192 + 32 – 1 = 223.

## Answer to Problem 3

**Table F-7** Question 3: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.100.18.18 | — |
| Mask | 255.240.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 20 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 4 | 32 – (network size + host size) |
| Number of subnets | $2^4 = 16$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{20} – 2 = 1{,}048{,}574$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-8 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-8** Question 3: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.100.18.18 | `00001010 0110`**`0100 00010010 00010010`** |
| Mask | 255.240.0.0 | `11111111 1111`**`0000 00000000 00000000`** |
| AND result (subnet number) | 10.96.0.0 | `00001010 0110`**`0000 00000000 00000000`** |
| Change host to 1s (broadcast address) | 10.111.255.255 | `00001010 0110`**`1111 11111111 11111111`** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.96.0.1 through 10.111.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-9 shows the work for this problem, with some explanation of the work following the table.

**Table F-9** Question 3: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 | Comments |
|---|---|---|---|---|---|
| **Mask** | 255 | 240 | 0 | 0 | — |
| **Address** | 10 | 100 | 18 | 18 | — |
| **Subnet Number** | 10 | 96 | 0 | 0 | Magic number = 256 – 240 = 16 |
| **First Address** | 10 | 96 | 0 | 1 | Add 1 to last octet of subnet |
| **Last Address** | 10 | 111 | 255 | 254 | Subtract 1 from last octet of broadcast |
| **Broadcast** | 10 | 111 | 255 | 255 | 96 + 16 – 1 = 111 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 240 = 16 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 96 is the multiple of 16 that is closest to 100 but not higher than 100. So, the second octet of the subnet number is 96.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 96 + 16 – 1 = 111.

## Answer to Problem 4

**Table F-10** Question 4: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.100.18.18 | — |
| Mask | 255.248.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 19 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 5 | 32 – (network size + host size) |
| Number of subnets | $2^5 = 32$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{19} – 2 = 524{,}286$ | $2^{\text{number-of-host-bits}} – 2$ |

F

Table F-11 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-11** Question 4: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.100.18.18 | 00001010 01100**100 00010010 00010010** |
| Mask | 255.248.0.0 | 11111111 11111**000 00000000 00000000** |
| AND result (subnet number) | 10.96.0.0 | 00001010 01100**000 00000000 00000000** |
| Change host to 1s (broadcast address) | 10.103.255.255 | 00001010 01100**111 11111111 11111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.96.0.1 through 10.103.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-12 shows the work for this problem, with some explanation of the work following the table.

**Table F-12** Question 4: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 | Comments |
|---|---|---|---|---|---|
| **Mask** | 255 | 248 | 0 | 0 | — |
| **Address** | 10 | 100 | 18 | 18 | — |
| **Subnet Number** | 10 | 96 | 0 | 0 | Magic number = 256 – 248 = 8 |
| **First Address** | 10 | 96 | 0 | 1 | Add 1 to last octet of subnet |
| **Last Address** | 10 | 103 | 255 | 254 | Subtract 1 from last octet of broadcast |
| **Broadcast** | 10 | 103 | 255 | 255 | 96 + 8 – 1 = 103 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 248 = 8 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 96 is the multiple of 8 that is closest to 100 but not higher than 100. So, the second octet of the subnet number is 96.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 96 + 8 – 1 = 103.

## Answer to Problem 5

**Table F-13** Question 5: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.150.200.200 | — |
| Mask | 255.252.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 18 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 6 | 32 – (network size + host size) |
| Number of subnets | $2^6 = 64$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{18} – 2 = 262{,}142$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-14 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-14** Question 5: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.150.200.200 | 00001010 10010110 11001000 11001000 |
| Mask | 255.252.0.0 | 11111111 11111100 00000000 00000000 |
| AND result (subnet number) | 10.148.0.0 | 00001010 10010100 00000000 00000000 |
| Change host to 1s (broadcast address) | 10.151.255.255 | 00001010 10010111 11111111 11111111 |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.148.0.1 through 10.151.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-15 shows the work for this problem, with some explanation of the work following the table.

**Table F-15** Question 5: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 | Comments |
|---|---|---|---|---|---|
| **Mask** | 255 | 252 | 0 | 0 | — |
| **Address** | 10 | 150 | 200 | 200 | — |
| **Subnet Number** | 10 | 148 | 0 | 0 | Magic number = 256 – 252 = 4 |
| **First Address** | 10 | 148 | 0 | 1 | Add 1 to last octet of subnet |
| **Last Address** | 10 | 151 | 255 | 254 | Subtract 1 from last octet of broadcast |
| **Broadcast** | 10 | 151 | 255 | 255 | 148 + 4 – 1 = 151 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 252 = 4 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 148 is the multiple of 4 that is closest to 150 but not higher than 150. So, the second octet of the subnet number is 148.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 148 + 4 – 1 = 151.

## Answer to Problem 6

**Table F-16** Question 6: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.150.200.200 | — |
| Mask | 255.254.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 17 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 7 | 32 – (network size + host size) |
| Number of subnets | $2^7 = 128$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{17} – 2 = 131{,}070$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-17 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-17** Question 6: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.150.200.200 | `00001010 10010110 11001000 11001000` |
| Mask | 255.254.0.0 | `11111111 11111110 00000000 00000000` |
| AND result (subnet number) | 10.150.0.0 | `00001010 10010110 00000000 00000000` |
| Change host to 1s (broadcast address) | 10.151.255.255 | `00001010 10010111 11111111 11111111` |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.150.0.1 through 10.151.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-18 shows the work for this problem, with some explanation of the work following the table.

**Table F-18** Question 6: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 254 | 0 | 0 |
| **Address** | 10 | 150 | 200 | 200 |
| **Subnet Number** | 10 | 150 | 0 | 0 |
| **First Valid Address** | 10 | 150 | 0 | 1 |
| **Last Valid Address** | 10 | 151 | 255 | 254 |
| **Broadcast** | 10 | 151 | 255 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 254 = 2 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 150 is the multiple of 2 that is closest to 150 but not higher than 150. So, the second octet of the subnet number is 150.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 150 + 2 – 1 = 151.

## Answer to Problem 7

**Table F-19** Question 7: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.220.100.18 | — |
| Mask | 255.255.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 16 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 8 | 32 – (network size + host size) |
| Number of subnets | $2^8 = 256$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{16} – 2 = 65{,}534$ | $2^{\text{number-of-host-bits}} – 2$ |

F

Table F-20 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-20** Question 7: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.220.100.18 | `00001010 11011100 01100100 00010010` |
| Mask | 255.255.0.0 | `11111111 11111111 00000000 00000000` |
| AND result (subnet number) | 10.220.0.0 | `00001010 11011100 00000000 00000000` |
| Change host to 1s (broadcast address) | 10.220.255.255 | `00001010 11011100 11111111 11111111` |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.220.0.1 through 10.220.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-21 shows the work for this problem.

**Table F-21** Question 7: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 0 | 0 |
| **Address** | 10 | 220 | 100 | 18 |
| **Subnet Number** | 10 | 220 | 0 | 0 |
| **First Valid Address** | 10 | 220 | 0 | 1 |
| **Last Valid Address** | 10 | 220 | 255 | 254 |
| **Broadcast** | 10 | 220 | 255 | 255 |

This subnetting scheme uses an easy mask because all the octets are a 0 or a 255. No math tricks are needed.

## Answer to Problem 8

**Table F-22** Question 8: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.220.100.18 | — |
| Mask | 255.255.128.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 15 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 9 | 32 – (network size + host size) |
| Number of subnets | $2^9 = 512$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{15} – 2 = 32{,}766$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-23 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-23** Question 8: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.220.100.18 | `00001010 11011100 01100100 00010010` |
| Mask | 255.255.128.0 | `11111111 11111111 10000000 00000000` |
| AND result (subnet number) | 10.220.0.0 | `00001010 11011100 00000000 00000000` |
| Change host to 1s (broadcast address) | 10.220.127.255 | `00001010 11011100 01111111 11111111` |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.220.0.1 through 10.220.127.254

Table F-24 shows the work for this problem, with some explanation of the work following the table. Refer to Chapter 14 for the detailed processes.

**Table F-24** Question 8: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Mask | 255 | 255 | 128 | 0 |
| Address | 10 | 220 | 100 | 18 |
| Subnet Number | 10 | 220 | 0 | 0 |
| First Address | 10 | 220 | 0 | 1 |
| Last Address | 10 | 220 | 127 | 254 |
| Broadcast | 10 | 220 | 127 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 128 = 128 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 0 is the multiple of 128 that is closest to 100 but not higher than 100. So, the third octet of the subnet number is 0.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 0 + 128 – 1 = 127.

This example tends to confuse people, because a mask with 128 in it gives you subnet numbers that just do not seem to look right. Table F-25 gives you the answers for the first several subnets, just to make sure that you are clear about the subnets when using this mask with a Class A network.

**Table F-25** Question 8: First Four Subnets

| | Zero Subnet | 2nd Subnet | 3rd Subnet | 4th Subnet |
|---|---|---|---|---|
| Subnet | 10.0.0.0 | 10.0.128.0 | 10.1.0.0 | 10.1.128.0 |
| First Address | 10.0.0.1 | 10.0.128.1 | 10.1.0.1 | 10.1.128.1 |
| Last Address | 10.0.127.254 | 10.0.255.254 | 10.1.127.254 | 10.1.255.254 |
| Broadcast | 10.0.127.255 | 10.0.255.255 | 10.1.127.255 | 10.1.255.255 |

## Answer to Problem 9

**Table F-26** Question 9: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.100.100 | — |
| Mask | 255.255.192.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 14 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 2 | 32 – (network size + host size) |
| Number of subnets | $2^2 = 4$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{14} – 2 = 16{,}382$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-27 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-27** Question 9: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.100.100 | `10101100 00011111 01`**`100100 01100100`** |
| Mask | 255.255.192.0 | `11111111 11111111 11`**`000000 00000000`** |
| AND result (subnet number) | 172.31.64.0 | `10101100 00011111 01`**`000000 00000000`** |
| Change host to 1s (broadcast address) | 172.31.127.255 | `10101100 00011111 01`**`111111 11111111`** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.64.1 through 172.31.127.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-28 shows the work for this problem, with some explanation of the work following the table.

**Table F-28** Question 9: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 192 | 0 |
| **Address** | 172 | 31 | 100 | 100 |
| **Subnet Number** | 172 | 31 | 64 | 0 |
| **First Valid Address** | 172 | 31 | 64 | 1 |
| **Last Valid Address** | 172 | 31 | 127 | 254 |
| **Broadcast** | 172 | 31 | 127 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 192 = 64 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 64 is the multiple of 64 that is closest to 100 but not higher than 100. So, the third octet of the subnet number is 64.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 64 + 64 – 1 = 127.

## Answer to Problem 10

**Table F-29** Question 10: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.100.100 | — |
| Mask | 255.255.224.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 13 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 3 | 32 – (network size + host size) |
| Number of subnets | $2^3 = 8$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{13} – 2 = 8190$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-30 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-30** Question 10: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.100.100 | 10101100 00011111 011**00100 01100100** |
| Mask | 255.255.224.0 | 11111111 11111111 111**00000 00000000** |
| AND result (subnet number) | 172.31.96.0 | 10101100 00011111 011**00000 00000000** |
| Change host to 1s (broadcast address) | 172.31.127.255 | 10101100 00011111 011**11111 11111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.96.1 through 172.31.127.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-31 shows the work for this problem, with some explanation of the work following the table.

**Table F-31** Question 10: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 224 | 0 |
| **Address** | 172 | 31 | 100 | 100 |
| **Subnet Number** | 172 | 31 | 96 | 0 |
| **First Valid Address** | 172 | 31 | 96 | 1 |
| **Last Valid Address** | 172 | 31 | 127 | 254 |
| **Broadcast** | 172 | 31 | 127 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 224 = 32 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 96 is the multiple of 32 that is closest to 100 but not higher than 100. So, the third octet of the subnet number is 96.

The second part of this process calculates the subnet broadcast address, with the tricky parts, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 96 + 32 – 1 = 127.

## Answer to Problem 11

**Table F-32** Question 11: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.200.10 | — |
| Mask | 255.255.240.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 12 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 4 | 32 – (network size + host size) |
| Number of subnets | $2^4 = 16$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{12} – 2 = 4094$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-33 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-33** Question 11: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.200.10 | 10101100 00011111 1100**1000 00001010** |
| Mask | 255.255.240.0 | 11111111 11111111 1111**0000 00000000** |
| AND result (subnet number) | 172.31.192.0 | 10101100 00011111 1100**0000 00000000** |
| Change host to 1s (broadcast address) | 172.31.207.255 | 10101100 00011111 1100**1111 11111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.192.1 through 172.31.207.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-34 shows the work for this problem, with some explanation of the work following the table.

**Table F-34** Question 11: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 240 | 0 |
| **Address** | 172 | 31 | 200 | 10 |
| **Subnet Number** | 172 | 31 | 192 | 0 |
| **First Valid Address** | 172 | 31 | 192 | 1 |
| **Last Valid Address** | 172 | 31 | 207 | 254 |
| **Broadcast** | 172 | 31 | 207 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 240 = 16 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 192 is the multiple of 16 that is closest to 200 but not higher than 200. So, the third octet of the subnet number is 192.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 192 + 16 – 1 = 207.

## Answer to Problem 12

**Table F-35** Question 12: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.200.10 | — |
| Mask | 255.255.248.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 11 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 5 | 32 – (network size + host size) |
| Number of subnets | $2^5 = 32$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{11} – 2 = 2046$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-36 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-36** Question 12: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.200.10 | 10101100 00011111 11001**000 00001010** |
| Mask | 255.255.248.0 | 11111111 11111111 11111**000 00000000** |
| AND result (subnet number) | 172.31.200.0 | 10101100 00011111 11001**000 00000000** |
| Change host to 1s (broadcast address) | 172.31.207.255 | 10101100 00011111 11001**111 11111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.200.1 through 172.31.207.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-37 shows the work for this problem, with some explanation of the work following the table.

**Table F-37** Question 12: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 248 | 0 |
| **Address** | 172 | 31 | 200 | 10 |
| **Subnet Number** | 172 | 31 | 200 | 0 |
| **First Valid Address** | 172 | 31 | 200 | 1 |
| **Last Valid Address** | 172 | 31 | 207 | 254 |
| **Broadcast** | 172 | 31 | 207 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 248 = 8 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 200 is the multiple of 8 that is closest to 200 but not higher than 200. So, the third octet of the subnet number is 200.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 200 + 8 – 1 = 207.

## Answer to Problem 13

**Table F-38** Question 13: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.50.50 | — |
| Mask | 255.255.252.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 10 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 6 | 32 – (network size + host size) |
| Number of subnets | $2^6 = 64$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{10} – 2 = 1022$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-39 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-39** Question 13: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.50.50 | 10101100 00011111 00110010 00110010 |
| Mask | 255.255.252.0 | 11111111 11111111 11111100 00000000 |
| AND result (subnet number) | 172.31.48.0 | 10101100 00011111 00110000 00000000 |
| Change host to 1s (broadcast address) | 172.31.51.255 | 10101100 00011111 00110011 11111111 |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.48.1 through 172.31.51.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-40 shows the work for this problem, with some explanation of the work following the table.

F

**Table F-40** Question 13: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 252 | 0 |
| **Address** | 172 | 31 | 50 | 50 |
| **Subnet Number** | 172 | 31 | 48 | 0 |
| **First Valid Address** | 172 | 31 | 48 | 1 |
| **Last Valid Address** | 172 | 31 | 51 | 254 |
| **Broadcast** | 172 | 31 | 51 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 252 = 4 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 48 is the multiple of 4 that is closest to 50 but not higher than 50. So, the third octet of the subnet number is 48.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 48 + 4 – 1 = 51.

## Answer to Problem 14

**Table F-41** Question 14: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.50.50 | — |
| Mask | 255.255.254.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 9 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 7 | 32 – (network size + host size) |
| Number of subnets | $2^7 = 128$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^9 – 2 = 510$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-42 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-42** Question 14: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.50.50 | 10101100 00011111 00110010 00110010 |
| Mask | 255.255.254.0 | 11111111 11111111 11111110 00000000 |
| AND result (subnet number) | 172.31.50.0 | 10101100 00011111 00110010 00000000 |
| Change host to 1s (broadcast address) | 172.31.51.255 | 10101100 00011111 00110011 11111111 |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.50.1 through 172.31.51.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-43 shows the work for this problem, with some explanation of the work following the table.

**Table F-43** Question 14: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 254 | 0 |
| **Address** | 172 | 31 | 50 | 50 |
| **Subnet Number** | 172 | 31 | 50 | 0 |
| **First Valid Address** | 172 | 31 | 50 | 1 |
| **Last Valid Address** | 172 | 31 | 51 | 254 |
| **Broadcast** | 172 | 31 | 51 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 254 = 2 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 50 is the multiple of 2 that is closest to 50 but not higher than 50. So, the third octet of the subnet number is 50.

The second part of this process calculates the subnet broadcast address with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 50 + 2 – 1 = 51.

## Answer to Problem 15

**Table F-44** Question 15: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.140.14 | — |
| Mask | 255.255.255.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 8 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 8 | 32 – (network size + host size) |
| Number of subnets | $2^8 = 256$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^8 – 2 = 254$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-45 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-45** Question 15: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.140.14 | `10101100 00011111 10001100` **`00001110`** |
| Mask | 255.255.255.0 | `11111111 11111111 11111111` **`00000000`** |
| AND result (subnet number) | 172.31.140.0 | `10101100 00011111 10001100` **`00000000`** |
| Change host to 1s (broadcast address) | 172.31.140.255 | `10101100 00011111 10001100` **`11111111`** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.140.1 through 172.31.140.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-46 shows the work for this problem.

**Table F-46** Question 15: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 255 | 0 |
| **Address** | 172 | 31 | 140 | 14 |
| **Subnet Number** | 172 | 31 | 140 | 0 |
| **First Valid Address** | 172 | 31 | 140 | 1 |
| **Last Valid Address** | 172 | 31 | 140 | 254 |
| **Broadcast** | 172 | 31 | 140 | 255 |

This subnetting scheme uses an easy mask because all the octets are a 0 or a 255. No math tricks are needed.

## Answer to Problem 16

**Table F-47** Question 16: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.31.140.14 | — |
| Mask | 255.255.255.128 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 7 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 9 | 32 – (network size + host size) |
| Number of subnets | $2^9 = 512$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^7 - 2 = 126$ | $2^{\text{number-of-host-bits}} - 2$ |

Table F-48 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-48** Question 16: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.31.140.14 | `10101100 00011111 10001100 0`**`0001110`** |
| Mask | 255.255.255.128 | `11111111 11111111 11111111 1`**`0000000`** |
| AND result (subnet number) | 172.31.140.0 | `10101100 00011111 10001100 0`**`0000000`** |
| Change host to 1s (broadcast address) | 172.31.140.127 | `10101100 00011111 10001100 0`**`1111111`** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.31.140.1 through 172.31.140.126

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-49 shows the work for this problem, with some explanation of the work following the table.

**Table F-49** Question 16: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 255 | 128 |
| **Address** | 172 | 31 | 140 | 14 |
| **Subnet Number** | 172 | 31 | 140 | 0 |
| **First Valid Address** | 172 | 31 | 140 | 1 |
| **Last Valid Address** | 172 | 31 | 140 | 126 |
| **Broadcast** | 172 | 31 | 140 | 127 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 128 = 128 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 0 is the multiple of 128 that is closest to 14 but not higher than 14. So, the fourth octet of the subnet number is 0.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 0 + 128 – 1 = 127.

## Answer to Problem 17

**Table F-50** Question 17: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 192.168.15.150 | — |
| Mask | 255.255.255.192 | — |
| Number of network bits | 24 | Always defined by Class A, B, C |
| Number of host bits | 6 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 2 | 32 – (network size + host size) |
| Number of subnets | $2^2 = 4$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^6 – 2 = 62$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-51 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-51** Question 17: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 192.168.15.150 | 11000000 10101000 00001111 10**010110** |
| Mask | 255.255.255.192 | 11111111 11111111 11111111 11**000000** |
| AND result (subnet number) | 192.168.15.128 | 11000000 10101000 00001111 10**000000** |
| Change host to 1s (broadcast address) | 192.168.15.191 | 11000000 10101000 00001111 10**111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

192.168.15.129 through 192.168.15.190

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-52 shows the work for this problem, with some explanation of the work following the table.

**Table F-52** Question 17: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Mask | 255 | 255 | 255 | 192 |
| Address | 192 | 168 | 15 | 150 |
| Subnet Number | 192 | 168 | 15 | 128 |
| First Valid Address | 192 | 168 | 15 | 129 |
| Last Valid Address | 192 | 168 | 15 | 190 |
| Broadcast | 192 | 168 | 15 | 191 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 192 = 64 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 128 is the multiple of 64 that is closest to 150 but not higher than 150. So, the fourth octet of the subnet number is 128.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 128 + 64 – 1 = 191.

F

## Answer to Problem 18

**Table F-53** Question 18: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 192.168.15.150 | — |
| Mask | 255.255.255.224 | — |
| Number of network bits | 24 | Always defined by Class A, B, C |
| Number of host bits | 5 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 3 | 32 – (network size + host size) |
| Number of subnets | $2^3 = 8$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^5 – 2 = 30$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-54 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-54** Question 18: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 192.168.15.150 | 11000000 10101000 00001111 100**10110** |
| Mask | 255.255.255.224 | 11111111 11111111 11111111 111**00000** |
| AND result (subnet number) | 192.168.15.128 | 11000000 10101000 00001111 100**00000** |
| Change host to 1s (broadcast address) | 192.168.15.159 | 11000000 10101000 00001111 100**11111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

192.168.15.129 through 192.168.15.158

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-55 shows the work for this problem, with some explanation of the work following the table.

**Table F-55** Question 18: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 255 | 224 |
| **Address** | 192 | 168 | 15 | 150 |
| **Subnet Number** | 192 | 168 | 15 | 128 |
| **First Valid Address** | 192 | 168 | 15 | 129 |
| **Last Valid Address** | 192 | 168 | 15 | 158 |
| **Broadcast** | 192 | 168 | 15 | 159 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 224 = 32 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 128 is the multiple of 32 that is closest to 150 but not higher than 150. So, the fourth octet of the subnet number is 128.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 128 + 32 – 1 = 159.

## Answer to Problem 19

**Table F-56** Question 19: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 192.168.100.100 | — |
| Mask | 255.255.255.240 | — |
| Number of network bits | 24 | Always defined by Class A, B, C |
| Number of host bits | 4 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 4 | 32 – (network size + host size) |
| Number of subnets | $2^4 = 16$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^4 – 2 = 14$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-57 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-57** Question 19: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 192.168.100.100 | 11000000 10101000 01100100 0110**0100** |
| Mask | 255.255.255.240 | 11111111 11111111 11111111 1111**0000** |
| AND result (subnet number) | 192.168.100.96 | 11000000 10101000 01100100 0110**0000** |
| Change host to 1s (broadcast address) | 192.168.100.111 | 11000000 10101000 01100100 0110**1111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

192.168.100.97 through 192.168.100.110

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-58 shows the work for this problem, with some explanation of the work following the table.

**Table F-58** Question 19: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 255 | 240 |
| **Address** | 192 | 168 | 100 | 100 |
| **Subnet Number** | 192 | 168 | 100 | 96 |
| **First Valid Address** | 192 | 168 | 100 | 97 |
| **Last Valid Address** | 192 | 168 | 100 | 110 |
| **Broadcast** | 192 | 168 | 100 | 111 |

F

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 240 = 16 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 96 is the multiple of 16 that is closest to 100 but not higher than 100. So, the fourth octet of the subnet number is 96.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 96 + 16 – 1 = 111.

## Answer to Problem 20

**Table F-59** Question 20: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 192.168.100.100 | — |
| Mask | 255.255.255.248 | — |
| Number of network bits | 24 | Always defined by Class A, B, C |
| Number of host bits | 3 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 5 | 32 – (network size + host size) |
| Number of subnets | $2^5 = 32$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^3 – 2 = 6$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-60 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-60** Question 20: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 192.168.100.100 | 11000000 10101000 01100100 01100**100** |
| Mask | 255.255.255.248 | 11111111 11111111 11111111 11111**000** |
| AND result (subnet number) | 192.168.100.96 | 11000000 10101000 01100100 01100**000** |
| Change host to 1s (broadcast address) | 192.168.100.103 | 11000000 10101000 01100100 01100**111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

192.168.100.97 through 192.168.100.102

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-61 shows the work for this problem, with some explanation of the work following the table.

**Table F-61** Question 20: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Mask | 255 | 255 | 255 | 248 |
| Address | 192 | 168 | 100 | 100 |
| Subnet Number | 192 | 168 | 100 | 96 |
| First Valid Address | 192 | 168 | 100 | 97 |
| Last Valid Address | 192 | 168 | 100 | 102 |
| Broadcast | 192 | 168 | 100 | 103 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 248 = 8 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 96 is the multiple of 8 that is closest to 100 but not higher than 100. So, the fourth octet of the subnet number is 96.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 96 + 8 – 1 = 103.

F

## Answer to Problem 21

**Table F-62** Question 21: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 192.168.15.230 | — |
| Mask | 255.255.255.252 | — |
| Number of network bits | 24 | Always defined by Class A, B, C |
| Number of host bits | 2 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 6 | 32 – (network size + host size) |
| Number of subnets | $2^6 = 64$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^2 – 2 = 2$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-63 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-63** Question 21: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 192.168.15.230 | 11000000 10101000 00001111 11100**10** |
| Mask | 255.255.255.252 | 11111111 11111111 11111111 11111**00** |
| AND result (subnet number) | 192.168.15.228 | 11000000 10101000 00001111 11100**00** |
| Change host to 1s (broadcast address) | 192.168.15.231 | 11000000 10101000 00001111 11100**11** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

192.168.15.229 through 192.168.15.230

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-64 shows the work for this problem, with some explanation of the work following the table.

**Table F-64** Question 21: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 255 | 252 |
| **Address** | 192 | 168 | 15 | 230 |
| **Subnet Number** | 192 | 168 | 15 | 228 |
| **First Valid Address** | 192 | 168 | 15 | 229 |
| **Last Valid Address** | 192 | 168 | 15 | 230 |
| **Broadcast** | 192 | 168 | 15 | 231 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 252 = 4 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 228 is the multiple of 4 that is closest to 230 but not higher than 230. So, the fourth octet of the subnet number is 228.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 228 + 4 – 1 = 231.

## Answer to Problem 22

**Table F-65** Question 22: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.1.1.1 | — |
| Mask | 255.248.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 19 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 5 | 32 – (network size + host size) |
| Number of subnets | $2^5 = 32$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{19} - 2 = 524{,}286$ | $2^{\text{number-of-host-bits}} - 2$ |

Table F-66 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-66** Question 22: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.1.1.1 | 00001010 00000**001 00000001 00000001** |
| Mask | 255.248.0.0 | 11111111 11111**000 00000000 00000000** |
| AND result (subnet number) | 10.0.0.0 | 00001010 00000**000 00000000 00000000** |
| Change host to 1s (broadcast address) | 10.7.255.255 | 00001010 00000**111 11111111 11111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.0.0.1 through 10.7.255.254

Take a closer look at the subnet part of the subnet address, as shown in bold here: 0000 1010 **0000 0**000 0000 0000 0000 0000. The subnet part of the address is all binary 0s, making this subnet a zero subnet.

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-67 shows the work for this problem, with some explanation of the work following the table.

F

**Table F-67** Question 22: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 248 | 0 | 0 |
| **Address** | 10 | 1 | 1 | 1 |
| **Subnet Number** | 10 | 0 | 0 | 0 |
| **First Valid Address** | 10 | 0 | 0 | 1 |
| **Last Valid Address** | 10 | 7 | 255 | 254 |
| **Broadcast** | 10 | 7 | 255 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The second octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 248 = 8 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 0 is the multiple of 8 that is closest to 1 but not higher than 1. So, the second octet of the subnet number is 0.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 0 + 8 – 1 = 7.

## Answer to Problem 23

**Table F-68** Question 23: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.16.1.200 | — |
| Mask | 255.255.240.0 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 12 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 4 | 32 – (network size + host size) |
| Number of subnets | $2^4 = 16$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{12} – 2 = 4094$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-69 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-69** Question 23: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.16.1.200 | 10101100 00010000 0000**0001 11001000** |
| Mask | 255.255.240.0 | 11111111 11111111 1111**0000 00000000** |
| AND result (subnet number) | 172.16.0.0 | 10101100 00010000 0000**0000 00000000** |
| Change host to 1s (broadcast address) | 172.16.15.255 | 10101100 00010000 0000**1111 11111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.16.0.1 through 172.16.15.254

Take a closer look at the subnet part of the subnet address, as shown in bold here: 1010 1100 0001 0000 **0000** 0000 0000 0000. The subnet part of the address is all binary 0s, making this subnet a zero subnet.

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-70 shows the work for this problem, with some explanation of the work following the table.

**Table F-70** Question 23: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 240 | 0 |
| **Address** | 172 | 16 | 1 | 200 |
| **Subnet Number** | 172 | 16 | 0 | 0 |
| **First Valid Address** | 172 | 16 | 0 | 1 |
| **Last Valid Address** | 172 | 16 | 15 | 254 |
| **Broadcast** | 172 | 16 | 15 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The third octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 240 = 16 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 0 is the multiple of 16 that is closest to 1 but not higher than 1. So, the third octet of the subnet number is 0.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 0 + 16 – 1 = 15.

## Answer to Problem 24

**Table F-71** Question 24: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 172.16.0.200 | — |
| Mask | 255.255.255.192 | — |
| Number of network bits | 16 | Always defined by Class A, B, C |
| Number of host bits | 6 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 10 | 32 – (network size + host size) |
| Number of subnets | $2^{10} = 1024$ | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^6 – 2 = 62$ | $2^{\text{number-of-host-bits}} – 2$ |

Table F-72 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-72** Question 24: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 172.16.0.200 | 10101100 00010000 00000000 11**001000** |
| Mask | 255.255.255.192 | 11111111 11111111 11111111 11**000000** |
| AND result (subnet number) | 172.16.0.192 | 10101100 00010000 00000000 11**000000** |
| Change host to 1s (broadcast address) | 172.16.0.255 | 10101100 00010000 00000000 11**111111** |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

172.16.0.193 through 172.16.0.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-73 shows the work for this problem, with some explanation of the work following the table.

**Table F-73** Question 24: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 255 | 255 | 192 |
| **Address** | 172 | 16 | 0 | 200 |
| **Subnet Number** | 172 | 16 | 0 | 192 |
| **First Valid Address** | 172 | 16 | 0 | 193 |
| **Last Valid Address** | 172 | 16 | 0 | 254 |
| **Broadcast** | 172 | 16 | 0 | 255 |

This subnetting scheme uses a difficult mask because one of the octets is not a 0 or a 255. The fourth octet is "interesting" in this case. The key part of the trick to get the right answers is to calculate the magic number, which is 256 – 192 = 64 in this case (256 – mask's value in the interesting octet). The subnet number's value in the interesting octet (inside the box) is the multiple of the magic number that is not higher than the original IP address's value in the interesting octet. In this case, 192 is the multiple of 64 that is closest to 200 but not higher than 200. So, the fourth octet of the subnet number is 192.

The second part of this process calculates the subnet broadcast address, with the tricky part, as usual, in the "interesting" octet. Take the subnet number's value in the interesting octet, add the magic number, and subtract 1. That is the broadcast address's value in the interesting octet. In this case, it is 192 + 64 – 1 = 255.

You can easily forget that the subnet part of this address, when using this mask, actually covers all the third octet as well as 2 bits of the fourth octet. For example, the valid subnet numbers in order are listed here:

172.16.0.0 (zero subnet)
172.16.0.64
172.16.0.128
172.16.0.192
172.16.1.0
172.16.1.64
172.16.1.128
172.16.1.192
172.16.2.0
172.16.2.64
172.16.2.128
172.16.2.192
172.16.3.0
172.16.3.64
172.16.3.128
172.16.3.192

And so on.

## Answer to Problem 25

Congratulations! You made it through the extra practice in this appendix! Here is an easy one to complete your review—one with no subnetting at all.

**Table F-74** Question 25: Size of Network, Subnet, Host, Number of Subnets, and Number of Hosts

| Item | Example | Rules to Remember |
|---|---|---|
| Address | 10.1.1.1 | — |
| Mask | 255.0.0.0 | — |
| Number of network bits | 8 | Always defined by Class A, B, C |
| Number of host bits | 24 | Always defined as number of binary 0s in mask |
| Number of subnet bits | 0 | 32 – (network size + host size) |
| Number of subnets | 0 | $2^{\text{number-of-subnet-bits}}$ |
| Number of hosts | $2^{24} - 2 = 16{,}777{,}214$ | $2^{\text{number-of-host-bits}} - 2$ |

Table F-75 contains the important binary calculations for finding the subnet number and subnet broadcast address. To calculate the subnet number, perform a Boolean AND on the address and mask. To find the broadcast address for this subnet, change all the host bits to binary 1s in the subnet number. The host bits are in **bold** print in the table.

**Table F-75** Question 25: Binary Calculation of Subnet and Broadcast Addresses

| | | |
|---|---|---|
| Address | 10.1.1.1 | `00001010 00000001 00000001 00000001` |
| Mask | 255.0.0.0 | `11111111 00000000 00000000 00000000` |
| AND result (subnet number) | 10.0.0.0 | `00001010 00000000 00000000 00000000` |
| Change host to 1s (broadcast address) | 10.255.255.255 | `00001010 11111111 11111111 11111111` |

Just add 1 to the subnet number to get the first valid IP address; just subtract 1 from the broadcast address to get the last valid IP address. In this case:

10.0.0.1 through 10.255.255.254

Alternatively, you can use the processes that only use decimal math to find the subnet and broadcast address. Table F-76 shows the work for this problem.

**Table F-76** Question 25: Subnet, Broadcast, and First and Last Addresses Calculated Using the Subnet Chart

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Mask** | 255 | 0 | 0 | 0 |
| **Address** | 10 | 1 | 1 | 1 |
| **Network Number** | 10 | 0 | 0 | 0 |
| **First Valid Address** | 10 | 0 | 0 | 1 |
| **Last Valid Address** | 10 | 255 | 255 | 254 |
| **Broadcast** | 10 | 255 | 255 | 255 |

# APPENDIX G

# Practice for Chapter 15: Subnet Design

This appendix exists as two halves to match the two major sections of the chapter. The first half lists mask design problems, and then the answers to those problems. The second half lists problems where you need to find the subnet ID, but with less than 8 subnet bits and with more than 8 subnet bits.

To solve these problems, use the processes explained in Chapter 15 of *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, the current edition of this book you are reading.

## Mask Design Practice Problems

This section lists problems with a short set of requirements regarding how a particular classful network should be subnetted. The requirements include the classful network, the number of subnets the design must support, and the number of hosts in each subnet. For each problem, supply the following information:

- The minimum number of subnet and host bits needed in the mask to support the design requirements
- The dotted-decimal format mask(s) that meet the requirements
- The mask you would choose if the problem said to maximize the number of subnets
- The mask you would choose if the problem said to maximize the number of hosts per subnet

Also note that you should assume that the two special subnets in each network—the zero subnet and broadcast subnet—are allowed to be used for these questions.

When doing the problems, the information in Table G-1 can be helpful. Note that Appendix A, "Numeric Reference Tables," in the printed book, also includes this table.

**Table G-1** Powers of 2

| Number of Bits | $2^X$ | Number of Bits | $2^X$ | Number of Bits | $2^X$ | Number of Bits | $2^X$ |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 5 | 32 | 9 | 512 | 13 | 8192 |
| 2 | 4 | 6 | 64 | 10 | 1024 | 14 | 16,384 |
| 3 | 8 | 7 | 128 | 11 | 2048 | 15 | 32,768 |
| 4 | 16 | 8 | 256 | 12 | 4096 | 16 | 65,536 |

Find the key facts for these sets of requirements:

1. Network 10.0.0.0, need 50 subnets, need 200 hosts/subnet
2. Network 172.32.0.0, need 125 subnets, need 125 hosts/subnet
3. Network 192.168.44.0, need 15 subnets, need 6 hosts/subnet
4. Network 10.0.0.0, need 300 subnets, need 500 hosts/subnet
5. Network 172.32.0.0, need 500 subnets, need 15 hosts/subnet
6. Network 172.16.0.0, need 2000 subnets, need 2 hosts/subnet

## Mask Design Answers

This section includes the answers to the six problems listed in this appendix. The answer section for each problem explains how to use the process outlined in Chapter 15, "Subnet Design," to find the answers.

### Answer to Mask Design Problem 1

Problem 1 shows a Class A network, with 8 network bits, with a minimum of 6 subnet bits and 8 host bits to meet the required number of subnets and hosts/subnet. The following masks all meet the requirements in this problem, with the masks that maximize the number of hosts/subnet and the number of subnets noted:

- 255.252.0.0 (maximizes the number of hosts per subnet)
- 255.254.0.0
- 255.255.0.0
- 255.255.128.0
- 255.255.192.0
- 255.255.224.0
- 255.255.240.0
- 255.255.248.0
- 255.255.252.0
- 255.255.254.0
- 255.255.255.0 (maximizes the number of subnets)

As for the process to find the answers, the following list explains the details:

**NOTE** The following explanation uses step numbers that match the process listed in Chapter 15, but only the steps from that process that apply to this problem. As a result, the step numbers in the explanation are not sequential.

**Step 1.** The question lists Class A network 10.0.0.0, so there are 8 network bits.

**Step 2.** The question states that 50 subnets are needed. A mask with 5 subnet bits supplies only $2^5$ (32) subnets, but a mask with 6 subnet bits supplies $2^6$ (64) subnets. So, the mask needs at least 6 subnet bits.

**Step 3.** The question states that 200 hosts are needed per subnet. A mask with 7 host bits supplies only $2^7 - 2$ (126) hosts per subnet, but a mask with 8 host bits supplies $2^8 - 2$ (254) hosts per subnet. So, the mask needs at least 8 host bits.

**Step 6A.** With N=8, a minimum S=6, and a minimum H=8, multiple masks exist. The first mask, with the minimum number of subnet bits, is /14, found by adding N (8) to the minimum value of S (6). This mask maximizes the number of host bits and therefore maximizes the number of hosts/subnet.

**Step 6B.** The minimum value of H, the number of host bits, is 8. So, the mask with the fewest H bits, maximizing the number of subnets, is 32 – H = 32 – 8 = /24.

**Step 6C.** All masks between /14 and /24 also meet the requirements.

## Answer to Mask Design Problem 2

Problem 2 shows a Class B network, with 16 network bits, with a minimum of 7 subnet bits and 7 host bits to meet the required number of subnets and hosts/subnet. The following masks all meet the requirements in this problem, with the masks that maximize the number of hosts/subnet and the number of subnets noted:

- 255.255.254.0 (maximizes the number of hosts/subnet)
- 255.255.255.0
- 255.255.255.128 (maximizes the number of subnets)

As for the process to find the answers, the following list explains the details:

**Step 1.** The question lists Class B network 172.32.0.0, so there are 16 network bits.

**Step 2.** The question states that 125 subnets are needed. A mask with 6 subnet bits supplies only $2^6$ (64) subnets, but a mask with 7 subnet bits supplies $2^7$ (128) subnets. So, the mask needs at least 7 subnet bits.

**Step 3.** The question states that 125 hosts are needed per subnet. A mask with 6 host bits supplies only $2^6 - 2$ (62) hosts per subnet, but a mask with 7 host bits supplies $2^7 - 2$ (126) hosts per subnet. So, the mask needs at least 7 host bits.

**Step 6A.** With N=16, a minimum S=7, and a minimum H=7, multiple masks exist. The first mask, with the minimum number of subnet bits, is /23, found by adding N (16) to the minimum value of S (7). This mask maximizes the number of host bits and therefore maximizes the number of hosts/subnet.

**Step 6B.** The minimum value of H, the number of host bits, is 7. So, the mask with the fewest H bits, maximizing the number of subnets, is 32 – H = 32 – 7 = /25.

**Step 6C.** All masks between /23 and /25 also meet the requirements (/23, /24, and /25).

## Answer to Mask Design Problem 3

Problem 3 shows a Class C network, with 24 network bits, with a minimum of 4 subnet bits and 3 host bits to meet the required number of subnets and hosts/subnet. The following masks all meet the requirements in this problem, with the masks that maximize the number of hosts/subnet and the number of subnets noted:

- 255.255.255.240 (maximizes the number of hosts/subnet)
- 255.255.255.248 (maximizes the number of subnets)

As for the process to find the answers, the following list explains the details:

**Step 1.** The question lists Class C network 192.168.44.0, so there are 24 network bits.

**Step 2.** The question states that 15 subnets are needed. A mask with 3 subnet bits supplies only $2^3$ (8) subnets, but a mask with 4 subnet bits supplies $2^4$ (16) subnets. So, the mask needs at least 4 subnet bits.

**Step 3.** The question states that 6 hosts are needed per subnet. A mask with 2 host bits supplies only $2^2 - 2$ (2) hosts per subnet, but a mask with 3 host bits supplies $2^3 - 2$ (6) hosts per subnet. So, the mask needs at least 3 host bits.

**Step 6A.** With N=24, a minimum S=4, and a minimum H=3, multiple masks exist. The first mask, with the minimum number of subnet bits, is /28, found by adding N (24) to the minimum value of S (4). This mask maximizes the number of host bits and therefore maximizes the number of hosts/subnet.

**Step 6B.** The minimum value of H, the number of host bits, is 3. So, the mask with the fewest H bits, maximizing the number of subnets, is 32 – H = 32 – 3 = /29.

**Step 6C.** Only masks /28 and /29 meet the requirements.

## Answer to Mask Design Problem 4

Problem 4 shows a Class A network, with 8 network bits, with a minimum of 9 subnet bits and 9 host bits to meet the required number of subnets and hosts/subnet. The following masks all meet the requirements in this problem, with the masks that maximize the number of hosts/subnet and the number of subnets noted:

- 255.255.128.0 (maximizes the number of hosts/subnet)
- 255.255.192.0
- 255.255.224.0
- 255.255.240.0
- 255.255.248.0
- 255.255.252.0
- 255.255.254.0 (maximizes the number of subnets)

As for the process to find the answers, the following list explains the details:

**Step 1.** The question lists Class A network 10.0.0.0, so there are 8 network bits.

**Step 2.** The question states that 300 subnets are needed. A mask with 8 subnet bits supplies only $2^8$ (256) subnets, but a mask with 9 subnet bits supplies $2^9$ (512) subnets. So, the mask needs at least 9 subnet bits.

**Step 3.** The question states that 500 hosts are needed per subnet. A mask with 8 host bits supplies only $2^8 - 2$ (254) hosts per subnet, but a mask with 9 host bits supplies $2^9 - 2$ (510) hosts per subnet. So, the mask needs at least 9 host bits.

**Step 6A.** With N=8, a minimum S=9, and a minimum H=9, multiple masks exist. The first mask, with the minimum number of subnet bits, is /17, found by adding N (8) to the minimum value of S (9). This mask maximizes the number of host bits and therefore maximizes the number of hosts/subnet.

**Step 6B.** The minimum value of H, the number of host bits, is 9. So, the mask with the fewest H bits, maximizing the number of subnets, is 32 – H = 32 – 9 = /23.

**Step 6C.** All masks between /17 and /23 also meet the requirements (/17, /18, /19, /20, /21, /22, /23).

## Answer to Mask Design Problem 5

Problem 5 shows a Class B network, with 16 network bits, with a minimum of 9 subnet bits and 5 host bits to meet the required number of subnets and hosts/subnet. The following masks all meet the requirements in this problem, with the masks that maximize the number of hosts/subnet and the number of subnets noted:

- 255.255.255.128 (maximizes the number of hosts/subnet)
- 255.255.255.192
- 255.255.255.224 (maximizes the number of subnets)



As for the process to find the answers, the following list explains the details:

**Step 1.** The question lists Class B network 172.32.0.0, so there are 16 network bits.

**Step 2.** The question states that 500 subnets are needed. A mask with 8 subnet bits supplies only $2^8$ (256) subnets, but a mask with 9 subnet bits supplies $2^9$ (512) subnets. So, the mask needs at least 9 subnet bits.

**Step 3.** The question states that 15 hosts are needed per subnet. A mask with 4 host bits supplies only $2^4 - 2$ (14) hosts per subnet, but a mask with 5 host bits supplies $2^5 - 2$ (30) hosts per subnet. So, the mask needs at least 5 host bits.

**Step 6A.** With N=16, a minimum S=9, and a minimum H=5, multiple masks exist. The first mask, with the minimum number of subnet bits, is /25, found by adding N (16) to the minimum value of S (9). This mask maximizes the number of host bits and therefore maximizes the number of hosts/subnet.

**Step 6B.** The minimum value of H, the number of host bits, is 5. So, the mask with the fewest H bits, maximizing the number of subnets, is 32 – H = 32 – 5 = /27.

**Step 6C.** All masks between /25 and /27 also meet the requirements (/25, /26, /27).

### Answer to Mask Design Problem 6

Problem 6 shows a Class B network, with 16 network bits, with a minimum of 11 subnet bits and 2 host bits to meet the required number of subnets and hosts/subnet. The following masks all meet the requirements in this problem, with the masks that maximize the number of hosts/subnet and the number of subnets noted:

- 255.255.255.224 (maximizes the number of hosts/subnet)
- 255.255.255.240
- 255.255.255.248
- 255.255.255.252 (maximizes the number of subnets)

As for the process to find the answers, the following list explains the details:

**Step 1.** The question lists Class B network 172.16.0.0, so there are 16 network bits.

**Step 2.** The question states that 2000 subnets are needed. A mask with 10 subnet bits supplies only $2^{10}$ (1024) subnets, but a mask with 11 subnet bits supplies $2^{11}$ (2048) subnets. So, the mask needs at least 11 subnet bits.

**Step 3.** The question states that 2 hosts are needed per subnet. A mask with 2 host bits supplies $2^2 - 2$ (2) hosts per subnet. So, the mask needs at least 2 host bits.

**Step 6A.** With N=16, a minimum S=11, and a minimum H=2, multiple masks exist. The first mask, with the minimum number of subnet bits, is /27, found by adding N (16) to the minimum value of S (11). This mask maximizes the number of host bits and therefore maximizes the number of hosts/subnet.

**Step 6B.** The minimum value of H, the number of host bits, is 2. So, the mask with the fewest H bits, maximizing the number of subnets, is 32 – H = 32 – 2 = /30.

**Step 6C.** All masks between /27 and /30 also meet the requirements (/27, /28, /29, /30).

## Practice Finding All Subnet IDs

The remainder of this Chapter 15ists two sets of problems. Both problem sets list an IP network and mask; your job is to list all the subnet IDs for each network/mask combination. The first problem set includes problems that happen to have 8 or fewer subnet bits, and the second problem set includes problems that happen to have more than 8 subnet bits. In particular, for each problem, find the following:

- All subnet numbers
- The subnet that is the zero subnet
- The subnet that is the broadcast subnet

To find this information, you can use the processes explained in Chapter 15.

### Find Subnet IDs, Problem Set 1: 8 or Fewer Subnet Bits

The problems, which consist of a classful network and static-length mask, are as follows:

1. 172.32.0.0/22
2. 200.1.2.0/28

3. 10.0.0.0/15
4. 172.20.0.0/24

## Find Subnet IDs, Problem Set 2: More Than 8 Subnet Bits

The problems, which consist of a classful network and static-length mask, are as follows:

1. 172.32.0.0/25
2. 10.0.0.0/21

## Answers to Find Subnet IDs, Problem Set 1

This section includes the answers to the four problems listed in Problem Set 1.

### Problem Set 1, Answer 1: 172.32.0.0/22

The answer is as follows:

- 172.32.0.0 (zero subnet)
- 172.32.4.0
- 172.32.8.0
- 172.32.12.0
- 172.32.16.0
- 172.32.20.0
- 172.32.24.0

  (Skipping many subnets; each new subnet is the same as the previous subnet, after adding 4 to the third octet.)
- 172.32.248.0
- 172.32.252.0 (broadcast subnet)

The process to find all subnets depends on three key pieces of information:

- The mask has fewer than 8 subnet bits (6 bits), because the network is a Class B network (16 network bits), and the mask has 22 binary 1s in it—implying 10 host bits and leaving 6 subnet bits.
- The mask in dotted-decimal format is 255.255.252.0. The interesting octet is the third octet because the subnet bits are all in the third octet.
- Each successive subnet number is 4 higher than the previous subnet number, in the interesting octet, because the magic number is 256 – 252 = 4.

As a result, in this case, all the subnets begin with 172.32, have a multiple of 4 in the third octet, and end in 0.

Table G-2 shows the results of the various steps of the process, as outlined in Chapter 15.

**Table G-2** 8 or Fewer Subnet Bits, Question 1: Answer Table

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Subnet Mask (Step 1) | 255 | 255 | 252 | 0 |
| Magic Number (Step 3) | | | 256 – 252 = 4 | |
| Zero Subnet Number (Step 4) | 172 | 32 | 0 | 0 |
| Next Subnet (Step 5) | 172 | 32 | 4 | 0 |
| Next Subnet (Step 5) | 172 | 32 | 8 | 0 |
| Next Subnet (Step 5) | 172 | 32 | 12 | 0 |
| Next Subnet (Step 5) | 172 | 32 | 16 | 0 |
| (You might need many more such rows.) | 172 | 32 | X | 0 |
| Next Subnet | 172 | 32 | 244 | 0 |
| Next Subnet (Step 5) | 172 | 32 | 248 | 0 |
| Broadcast Subnet (Step 6) | 172 | 32 | 252 | 0 |
| Out of Range—Stop Process (Step 6) | | | 256 | |

### Problem Set 1, Answer 2: 200.1.2.0/28

The answer is as follows:

- 200.1.2.0 (zero subnet)
- 200.1.2.16
- 200.1.2.32
- 200.1.2.48
- 200.1.2.64
- 200.1.2.80

  (Skipping many subnets; each new subnet is the same as the previous subnet, after adding 16 to the fourth octet.)
- 200.1.2.224
- 200.1.2.240 (broadcast subnet)

The process to find all subnets depends on three key pieces of information, as follows:

- The mask has fewer than 8 subnet bits (4 bits), because the network is a Class C network (24 network bits), and the mask has 28 binary 1s in it, which implies 4 host bits and leaves 4 subnet bits.
- The mask in dotted-decimal format is 255.255.255.240. The interesting octet is the fourth octet, because all the subnet bits are in the fourth octet.
- Each successive subnet number is 16 higher than the previous subnet number, in the interesting octet, because the magic number is 256 – 240 = 16.

As a result, in this case, all the subnets begin with 200.1.2 and have a multiple of 16 in the fourth octet.

Table G-3 shows the results of the various steps of the process, as outlined in Chapter 15.

**Table G-3** Problem Set 1, Question 2: Answer Table

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Subnet Mask (Step 1) | 255 | 255 | 255 | 240 |
| Magic Number (Step 3) | | | | 256 – 240 = 16 |
| Zero Subnet Number (Step 4) | 200 | 1 | 2 | 0 |
| Next Subnet (Step 5) | 200 | 1 | 2 | 16 |
| Next Subnet (Step 5) | 200 | 1 | 2 | 32 |
| Next Subnet (Step 5) | 200 | 1 | 2 | 48 |
| (You might need many more such rows.) (Step 5) | 200 | 1 | 2 | X |
| Next Subnet (Step 5) | 200 | 1 | 2 | 224 |
| Broadcast Subnet (Step 6) | 200 | 1 | 2 | 240 |
| Out of Range—Stop Process (Step 6) | | | | 256 |

### Problem Set 1, Answer 3: 10.0.0.0/15

The answer is as follows:

- 10.0.0.0 (zero subnet)
- 10.2.0.0
- 10.4.0.0
- 10.6.0.0

  (Skipping many subnets; each new subnet is the same as the previous subnet, after adding 2 to the second octet.)
- 10.252.0.0
- 10.254.0.0 (broadcast subnet)

The process to find all subnets depends on three key pieces of information:

- The mask has fewer than 8 subnet bits (7 subnet bits), because the network is a Class A network (8 network bits), and the mask has 15 binary 1s in it, which implies 17 host bits and leaves 7 subnet bits.
- The mask in dotted-decimal format is 255.254.0.0. The interesting octet is the second octet, because all the subnet bits exist in the second octet.
- Each successive subnet number is 2 higher than the previous subnet number, in the interesting octet, because the magic number is 256 – 254 = 2.

As a result, in this case, all the subnets begin with 10, have a multiple of 2 in the second octet, and end in 0.0.

Table G-4 shows the results of the various steps of the process, as outlined in Chapter 15.

**Table G-4** Problem Set 1, Question 3: Answer Table

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Subnet Mask (Step 1) | 255 | 254 | 0 | 0 |
| Magic Number (Step 3) | | 256 – 254 = 2 | | |
| Zero Subnet Number (Step 4) | 10 | 0 | 0 | 0 |
| Next Subnet (Step 5) | 10 | 2 | 0 | 0 |
| Next Subnet (Step 5) | 10 | 4 | 0 | 0 |
| Next Subnet (Step 5) | 10 | 6 | 0 | 0 |
| (You might need many more such rows.) (Step 5) | 10 | X | 0 | 0 |
| Next Subnet (Step 5) | 10 | 252 | 0 | 0 |
| Broadcast Subnet (Step 6) | 10 | 254 | 0 | 0 |
| Out of Range—Stop Process (Step 6) | | 256 | | |

### Problem Set 1, Answer 4: 172.20.0.0/24

This problem has an 8-bit subnet field, meaning that $2^8$, or 256, possible subnets exist. The following list shows some of the subnets, which should be enough to see the trends in how to find all subnet numbers:

- 172.20.0.0 (zero subnet)
- 172.20.1.0
- 172.20.2.0
- 172.20.3.0
- 172.20.4.0

  (Skipping many subnets; each new subnet is the same as the previous subnet, after adding 1 to the third octet.)
- 172.20.252.0
- 172.20.253.0
- 172.20.254.0
- 172.20.255.0 (broadcast subnet)

The process to find all subnets depends on three key pieces of information:

- The mask has exactly 8 subnet bits, specifically all bits in the third octet, making the third octet the interesting octet.
- The magic number is 256 – 255 = 1, because the mask's value in the interesting (third) octet is 255.
- Beginning with the network number of 172.20.0.0, which is the same value as the zero subnet, just add the magic number (1) in the interesting octet.

Essentially, you just count by 1 in the third octet until you reach the highest legal number (255). The first subnet, 172.20.0.0, is the zero subnet, and the last subnet, 172.20.255.0, is the broadcast subnet.

## Answers to Find Subnet IDs, Problem Set 2

### Problem Set 2, Answer 1: 172.32.0.0/25

This problem has a 9-bit subnet field, meaning that $2^9$, or 512, possible subnets exist. The following list shows some of the subnets, which should be enough to see the trends in how to find all subnet numbers:

- 172.32.0.0 (zero subnet)
- 172.32.0.128
- 172.32.1.0
- 172.32.1.128
- 172.32.2.0
- 172.32.2.128
- 172.32.3.0
- 172.32.3.128

  (Skipping many subnets; the subnets occur in blocks of two, with either 0 or 128 in the fourth octet, with each successive block being one greater in the third octet.)
- 172.32.254.0
- 172.32.254.128
- 172.32.255.0
- 172.32.255.128 (broadcast subnet)

The process to find all subnets depends on three key pieces of information, as follows:

- The mask has more than 8 subnet bits (9 bits), because the network is a Class B network (16 network bits), and the mask has 25 binary 1s in it, which implies 7 host bits and leaves 9 subnet bits.
- Using the terminology in Chapter 15, octet 4 is the *interesting* octet, where the counting occurs based on the magic number. Octet 3 is the "just left" octet, in which the process counts by 1, from 0 to 255.
- The magic number, which will be used to calculate each successive subnet number, is 256 – 128 = 128.

To calculate the first subnet block, use the same six-step process as used in the simpler problems that have 8 or fewer subnet bits. In this case, with only 1 subnet bit in octet 4, only two subnets exist in each subnet block. Table G-5 shows the steps as compared to the six-step process to find the subnets in a subnet block.

**Table G-5** Creating the First Subnet Block

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| Subnet Mask (Step 1) | 255 | 255 | 255 | 128 |
| Magic Number (Step 3) | | | | 256 – 128 = 128 |
| Zero Subnet Number (Step 4) | 172 | 32 | 0 | 0 |
| Next Subnet (Step 5) | 172 | 32 | 0 | 128 |
| Step 6 Needs to Be Used Here (Sum of 256 in the 4th Octet) | 172 | 32 | 0 | 256 |

The table represents the logic, but to make sure that the answer is clear, the first subnet block includes the following:

172.32.0.0

172.32.0.128

The next major task—to create subnet blocks for all possible values in the "just left" octet—completes the process. Essentially, create 256 blocks like the previous list. The first has a value of 0, in the "just left" octet; the next has a value of 1; the next, a value of 2; and so on, through a block that begins with 172.30.255. Figure G-1 shows the concept.

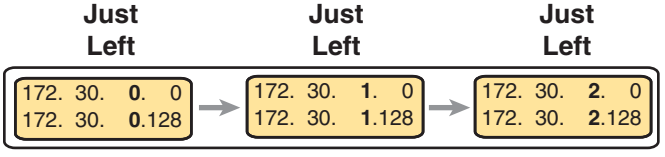



**Figure G-1** *Creating Subnet Blocks by Adding 1 in the "Just Left" Octet*

### Problem Set 2, Answer 2: 10.0.0.0/21

This problem has a 13-bit subnet field, meaning that $2^{13}$, or 8192, possible subnets exist. The following list shows some of the subnets, which should be enough to see the trends in how to find all subnet numbers:

- 10.0.0.0 (zero subnet)
- 10.0.8.0
- 10.0.16.0
- 10.0.24.0

  (Skipping several subnets)
- 10.0.248.0
- 10.1.0.0
- 10.1.8.0
- 10.1.16.0

  (Skipping several subnets)
- 10.1.248.0

- 10.2.0.0
- 10.2.8.0
- 10.2.16.0

  (Skipping several subnets)
- 10.255.232.0
- 10.255.240.0
- 10.255.248.0 (broadcast subnet)

The process to find all subnets depends on three key pieces of information, as follows:

- The mask has more than 8 subnet bits (13 bits), because the network is a Class A network (8 network bits), and the mask has 21 binary 1s in it, which implies 11 host bits and leaves 13 subnet bits.
- Using the terminology in Chapter 15, octet 3 is the interesting octet, where the counting occurs based on the magic number. Octet 2 is the "just left" octet, in which the process counts by 1, from 0 to 255.
- The magic number, which will be used to calculate each successive subnet number, is 256 – 248 = 8.

To calculate the first subnet block, use the same six-step process as used in the simpler problems that have 8 or fewer subnet bits. In this case, with 5 subnet bits in octet 3, 32 subnets exist in each subnet block. Table G-6 shows the steps as compared to the six-step process to find the subnets in a subnet block.

**Table G-6** Creating the First Subnet Block

| | Octet 1 | Octet 2 | Octet 3 | Octet 4 |
|---|---|---|---|---|
| **Subnet Mask (Step 1)** | 255 | 255 | 248 | 0 |
| **Magic Number (Step 3)** | | | 256 – 248 = 8 | |
| **Zero Subnet Number (Step 4)** | 10 | 0 | 0 | 0 |
| **Next Subnet (Step 5)** | 10 | 0 | 8 | 0 |
| **(Skipping several subnets)** | 10 | 0 | X | 0 |
| **Next Subnet (Step 5)** | 10 | 0 | 248 | 0 |
| **Step 6 Needs to Be Used Here (Sum of 256 in the 3rd Octet)** | 10 | 0 | 256 | 0 |

The table represents the logic, but to make sure that the answer is clear, the first subnet block includes the following:

10.0.0.0

10.0.8.0

10.0.16.0

10.0.24.0

10.0.32.0

10.0.40.0
10.0.48.0
10.0.56.0
10.0.64.0
And so on…
10.0.248.0

The next major task—to create subnet blocks for all possible values in the "just left" octet—completes the process. Essentially, create 256 blocks like the previous list. The first has a value of 0, in the "just left" octet; the next has a value of 1; the next, a value of 2; and so on, through a block that begins with 10.255. Figure G-2 shows the concept.

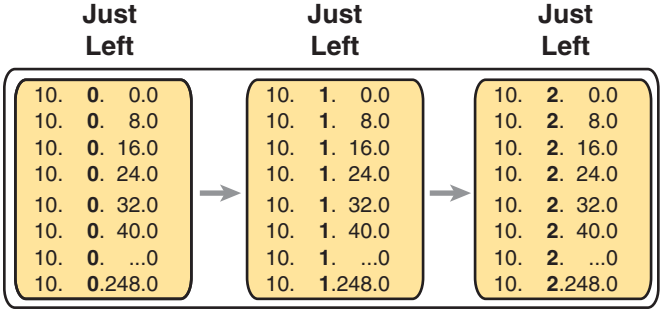



**Figure G-2** *Creating Subnet Blocks by Adding 1 in the "Just Left" Octet*

# APPENDIX H

# Practice for Chapter 25: Fundamentals of IP Version 6

This appendix provides extra practice problems for two topics discussed in Chapter 25, "Fundamentals of IP Version 6," of the book. The first problems let you convert from a full 32-digit IPv6 address to its abbreviated form, or to do the reverse. The second set of problems begins with IPv6 addresses and prefix lengths, asking you to determine the IPv6 prefix (subnet).

## Address Abbreviating and Expanding Problems

Chapter 25 discusses some reasons why you may need to be able to mentally convert from the full 32-digit IPv6 address to the abbreviated form, or vice versa. The practice problems in this section simply provide more opportunities to practice.

Table H-1 lists some practice problems, with the full 32-digit IPv6 address on the left and the best abbreviation on the right. The table gives you either the expanded or abbreviated address, and you need to supply the opposite value. The answers sit at the end of the appendix, in the section "Answers to Address Abbreviating and Expanding Problems."

**Table H-1** IPv6 Address Abbreviation and Expansion Practice

| | Full | Abbreviation |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:1000:0001:F001:F003 | |
| 2 | | 3100::1010:D00D:D000:D00B:B00D |
| 3 | FD00:0001:0001:0001:0200:00FF:FE00:0001 | |
| 4 | | FDDF:8080:880:1001:0:FF:FE01:507 |
| 5 | 32CC:0000:0000:000D:210F:0000:0000:0000 | |
| 6 | | 2100:E:E0::E00 |
| 7 | 3A11:CA00:0000:0000:0000:00FF:FECC:000C | |
| 8 | | 3799:9F9F:F000:0:FFFF::1 |
| 9 | 2A2A:0000:0000:0000:0000:0000:0000:2A2A | |
| 10 | | 3194::1:0:0:101 |
| 11 | 2001:0DB8:0000:0000:0001:0000:0002:0100 | |
| 12 | | 2001:DB8::10:A000 |
| 13 | 3330:0000:0000:0100:0000:0002:0000:0003 | |
| 14 | | FD00::1000:2000:0:1:20 |
| 15 | FD11:1000:0100:0010:0001:0000:1000:0100 | |
| 16 | | 2000::2 |

## Calculating the IPv6 Prefix Problems

Routers take the interface IPv6 address configuration and add a connected IPv6 route to the IPv6 routing table, for the IPv6 prefix (subnet) connected to that interface. This section provides some practice problems so that you can do the same math and predict the prefix value that the router will add to the routing table.

Table H-2 lists practice problems that all use the same prefix length (/64), which is the most common prefix length you see. Table H-3 that follows lists additional practice problems, with prefix lengths other than /64.

**Table H-2** Finding the IPv6 Prefix When Using a /64 Prefix Length

| | Address (Assume a /64 Prefix Length) | Prefix (Subnet) |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:1000:0001:F001:F003 | |
| 2 | 3100:0000:0000:1010:D00D:D000:D00B:B00D | |
| 3 | FD00:0001:0001:0001:0200:00FF:FE00:0001 | |
| 4 | FDDF:8080:0880:1001:0000:00FF:FE01:0507 | |
| 5 | 32CC:0000:0000:000D:210F:0000:0000:0000 | |
| 6 | 2100:000E:00E0:0000:0000:0000:0000:0E00 | |
| 7 | 3A11:CA00:0000:0000:0000:00FF:FECC:000C | |
| 8 | 3799:9F9F:F000:0000:FFFF:0000:0000:0001 | |
| 9 | 2A2A:0000:0000:0000:0000:0000:0000:2A2A | |
| 10 | 3194:0000:0000:0000:0001:0000:0000:0101 | |
| 11 | 2001:0DB8:0000:0000:0001:0000:0002:0100 | |
| 12 | 2001:0DB8:0000:0000:0000:0000:0010:A000 | |
| 13 | 3330:0000:0000:0100:0000:0002:0000:0003 | |
| 14 | FD00:0000:0000:1000:2000:0000:0001:0020 | |
| 15 | FD11:1000:0100:0010:0001:0000:1000:0100 | |
| 16 | 2000:0000:0000:0000:0000:0000:0000:0002 | |

**Table H-3** Finding the IPv6 Prefix Using a Prefix Length Other Than /64

| | Address | Prefix (Subnet) |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:1000:0001:F001:F003 /60 | |
| 2 | 3100:0000:0000:1010:D00D:D000:D00B:B00D /56 | |
| 3 | FD00:0001:0001:0001:0200:00FF:FE00:0001 /52 | |
| 4 | FDDF:8080:0880:1001:0000:00FF:FE01:0507 /48 | |
| 5 | 32CC:0000:0000:000D:210F:0000:0000:0000 /44 | |
| 6 | 2100:000E:00E0:0000:0000:0000:0000:0E00 /60 | |
| 7 | 3A11:CA00:0000:0000:0000:00FF:FECC:000C /56 | |
| 8 | 3799:9F9F:F000:0000:FFFF:0000:0000:0001 /52 | |
| 9 | 2A2A:0000:0000:0000:0000:0000:0000:2A2A /48 | |
| 10 | 3194:0000:0000:0000:0001:0000:0000:0101 /44 | |

## Answers to Address Abbreviating and Expanding Problems

Table H-4 lists the answers to the problems listed earlier in Table H-1.

**Table H-4** Answers: IPv6 Address Abbreviation and Expansion Practice

| | Full | Abbreviation |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:1000:0001:F001:F003 | 2987:BA11:B011:B00A:1000:1:F001:F003 |
| 2 | 3100:0000:0000:1010:D00D:D000:D00B:B00D | 3100::1010:D00D:D000:D00B:B00D |
| 3 | FD00:0001:0001:0001:0200:00FF:FE00:0001 | FD00:1:1:1:200:FF:FE00:1 |
| 4 | FDDF:8080:0880:1001:0000:00FF:FE01:0507 | FDDF:8080:880:1001:0:FF:FE01:507 |
| 5 | 32CC:0000:0000:000D:210F:0000:0000:0000 | 32CC:0:0:D:210F:: |
| 6 | 2100:000E:00E0:0000:0000:0000:0000:0E00 | 2100:E:E0::E00 |
| 7 | 3A11:CA00:0000:0000:0000:00FF:FECC:000C | 3A11:CA00::FF:FECC:C |
| 8 | 3799:9F9F:F000:0000:FFFF:0000:0000:0001 | 3799:9F9F:F000:0:FFFF::1 |
| 9 | 2A2A:0000:0000:0000:0000:0000:0000:2A2A | 2A2A::2A2A |
| 10 | 3194:0000:0000:0000:0001:0000:0000:0101 | 3194::1:0:0:101 |
| 11 | 2001:0DB8:0000:0000:0001:0000:0002:0100 | 2001:DB8::1:0:2:100 |
| 12 | 2001:0DB8:0000:0000:0000:0000:0010:A000 | 2001:DB8::10:A000 |
| 13 | 3330:0000:0000:0100:0000:0002:0000:0003 | 3330::100:0:2:0:3 |
| 14 | FD00:0000:0000:1000:2000:0000:0001:0020 | FD00::1000:2000:0:1:20 |
| 15 | FD11:1000:0100:0010:0001:0000:1000:0100 | FD11:1000:100:10:1:0:1000:100 |
| 16 | 2000:0000:0000:0000:0000:0000:0000:0002 | 2000::2 |

## Answers to Calculating IPv6 Prefix Problems

Tables H-5 and H-6 list the answers to the problems listed earlier in Tables H-2 and H-3.

**Table H-5** Answers: Finding the IPv6 Prefix, with a /64 Prefix Length

| | Address (Assume a /64 Prefix Length) | Prefix (Subnet) |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:1000:0001:F001:F003 | 2987:BA11:B011:B00A::/64 |
| 2 | 3100:0000:0000:1010:D00D:D000:D00B:B00D | 3100:0:0:1010::/64 |
| 3 | FD00:0001:0001:0001:0200:00FF:FE00:0001 | FD00:1:1:1::/64 |
| 4 | FDDF:8080:0880:1001:0000:00FF:FE01:0507 | FDDF:8080:880:1001::/64 |
| 5 | 32CC:0000:0000:000D:210F:0000:0000:0000 | 32CC:0:0:D::/64 |
| 6 | 2100:000E:00E0:0000:0000:0000:0000:0E00 | 2100:E:E0::/64 |
| 7 | 3A11:CA00:0000:0000:0000:00FF:FECC:000C | 3A11:CA00::/64 |
| 8 | 3799:9F9F:F000:0000:FFFF:0000:0000:0001 | 3799:9F9F:F000::/64 |
| 9 | 2A2A:0000:0000:0000:0000:0000:0000:2A2A | 2A2A::/64 |
| 10 | 3194:0000:0000:0000:0001:0000:0000:0101 | 3194::/64 |
| 11 | 2001:0DB8:0000:0000:0001:0000:0002:0100 | 2001:DB8::/64 |
| 12 | 2001:0DB8:0000:0000:0000:0000:0010:A000 | 2001:DB8::/64 |
| 13 | 3330:0000:0000:0100:0000:0002:0000:0003 | 3330:0:0:100::/64 |
| 14 | FD00:0000:0000:1000:2000:0000:0001:0020 | FD00:0:0:1000::/64 |
| 15 | FD11:1000:0100:0010:0001:0000:1000:0100 | FD11:1000:100:10::/64 |
| 16 | 2000:0000:0000:0000:0000:0000:0000:0002 | 2000::/64 |

**Table H-6** Answers: Finding the IPv6 Prefix, with Other Prefix Lengths

| | Address | Prefix (Subnet) |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:1000:0001:F001:F003 /60 | 2987:BA11:B011:B000::/60 |
| 2 | 3100:0000:0000:1010:D00D:D000:D00B:B00D /56 | 3100:0:0:1000::/56 |
| 3 | FD00:0001:0001:0001:0200:00FF:FE00:0001 /52 | FD00:1:1::/52 |
| 4 | FDDF:8080:0880:1001:0000:00FF:FE01:0507 /48 | FDDF:8080:880::/48 |
| 5 | 32CC:0000:0000:000D:210F:0000:0000:0000 /44 | 32CC::/44 |
| 6 | 2100:000E:00E0:0000:0000:0000:0000:0E00 /60 | 2100:E:E0::/60 |
| 7 | 3A11:CA00:0000:0000:0000:00FF:FECC:000C /56 | 3A11:CA00::/56 |
| 8 | 3799:9F9F:F000:0000:FFFF:0000:0000:0001 /52 | 3799:9F9F:F000::/52 |
| 9 | 2A2A:0000:0000:0000:0000:0000:0000:2A2A /48 | 2A2A::/48 |
| 10 | 3194:0000:0000:0000:0001:0000:0000:0101 /44 | 3194::/44 |

# APPENDIX I

# Practice for Chapter 27: Implementing IPv6 Addressing on Routers

This appendix provides practice problems for two types of addresses: unicast addresses formed with the EUI-64 feature and solicited node multicast addresses. With EUI-64, you take the 64-bit (16 hex digit) prefix and a MAC address, manipulate the MAC address into a 64-bit value, and use those 64 bits as the interface ID. Solicited node multicast addresses are formed from a standard 26 hex digit prefix, combined with the same last 6 hex digits as the unicast address.

## EUI-64 and Solicited Node Multicast Problems

Table I-1 lists some practice problems. Each problem lists a prefix and a MAC address. Then, in Table I-2, record your answers for the unicast IPv6 address, assuming that EUI-64 rules are used. Also in Table I-2, list the solicited node multicast address associated with your calculated unicast address.

For each answer, use the best abbreviation, instead of a full 32-digit address.

The answers sit at the end of the appendix, in Table I-3.

**Table I-1** IPv6 EUI-64 Unicast and Solicited Node Multicast Problems

| | Prefix | MAC Address |
|---|---|---|
| 1 | 2987:BA11:B011:B00A::/64 | 0000.1234.5678 |
| 2 | 3100:0000:0000:1010::/64 | 1234.5678.9ABC |
| 3 | FD00:0001:0001:0001::/64 | 0400.AAAA.0001 |
| 4 | FDDF:8080:0880:1001::/64 | 0611.BABA.DADA |
| 5 | 32CC:0000:0000:000D::/64 | 0000.0000.0001 |
| 6 | 2100:000E:00E0:0000::/64 | 0505.0505.0707 |
| 7 | 3A11:CA00:0000:0000::/64 | 0A0A.B0B0.0C0C |
| 8 | 3799:9F9F:F000:0000::/64 | F00F.0005.0041 |
| 9 | 2A2A:0000:0000:0000::/64 | 0200.0101.0101 |
| 10 | 3194:0000:0000:0000::/64 | 0C0C.000C.00CC |

**Table I-2** Blank Answer Table for Problems in Table I-1

| | Unicast Address Using EUI-64 | Solicited Node Multicast Address |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |
| 7 | | |
| 8 | | |
| 9 | | |
| 10 | | |

## Answers to EUI-64 and Solicited Node Multicast Problems

Table I-3 lists the answers to the problems listed earlier in Table I-1.

**Table I-3** Answers to Problems in Table I-1

| | Unicast Address Using EUI-64 | Solicited Node Multicast Address |
|---|---|---|
| 1 | 2987:BA11:B011:B00A:200:12FF:FE34:5678 | FF02::01:FF34.5678 |
| 2 | 3100::1010:1034:56FF:FE78:9ABC | FF02::01:FF78.9ABC |
| 3 | FD00:1:1:1:600:AAFF:FEAA:1 | FF02::01:FFAA:1 |
| 4 | FDDF:8080:880:1001:411:BAFF:FEBA:DADA | FF02::01:FFBA:DADA |
| 5 | 32CC::D:200:FF:FE00:1 | FF02::01:FF00:1 |
| 6 | 2100:E:E0:0:705:5FF:FE05:707 | FF02::01:FF05:707 |
| 7 | 3A11:CA00::80A:B0FF:FEB0:C0C | FF02::01:FFB0:C0C |
| 8 | 3799:9F9F:F000:0:F20F:FF:FE05:41 | FF02::01:FF05:41 |
| 9 | 2A2A::1FF:FE01:101 | FF02::01:FF01:101 |
| 10 | 3194::E0C:FF:FE0C:CC | FF02::01:FF0C:CC |

# Appendix J

# Study Planner

| Practice Test | Reading | Task |
|---|---|---|

| Element | Task | Goal Date | First Date Completed | Second Date Completed (Optional) | Notes |
|---|---|---|---|---|---|
| Introduction | Read Introduction | | | | |
| Your Study Planner | Read Your Study Planner | | | | |
| 1. Introduction to TCP/IP Networking | Read Foundation Topics | | | | |
| 1. Introduction to TCP/IP Networking | Review Key Topics using the book or companion website | | | | |
| 1. Introduction to TCP/IP Networking | Define Key Terms using the book or companion website | | | | |
| 1. Introduction to TCP/IP Networking | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 2. Fundamentals of Ethernet LANs | Read Foundation Topics | | | | |
| 2. Fundamentals of Ethernet LANs | Review Key Topics using the book or companion website | | | | |
| 2. Fundamentals of Ethernet LANs | Define Key Terms using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 2. Fundamentals of Ethernet LANs | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 2. Fundamentals of Ethernet LANs | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 3. Fundamentals of WANs and IP Routing | Read Foundation Topics | | | | |
| 3. Fundamentals of WANs and IP Routing | Review Key Topics using the book or companion website | | | | |
| 3. Fundamentals of WANs and IP Routing | Define Key Terms using the book or companion website | | | | |
| 3. Fundamentals of WANs and IP Routing | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 3. Fundamentals of WANs and IP Routing | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| Part I. Introduction to Networking | Complete all exercises in Part I Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 4. Using the Command-Line Interface | Read Foundation Topics | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 4. Using the Command-Line Interface | Review Key Topics using the book or companion website | | | | |
| 4. Using the Command-Line Interface | Define Key Terms using the book or companion website | | | | |
| 4. Using the Command-Line Interface | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 4. Using the Command-Line Interface | Review the command tables | | | | |
| 4. Using the Command-Line Interface | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this chapter | | | | |
| 5. Analyzing Ethernet LAN Switching | Read Foundation Topics | | | | |
| 5. Analyzing Ethernet LAN Switching | Review Key Topics using the book or companion website | | | | |
| 5. Analyzing Ethernet LAN Switching | Define Key Terms using the book or companion website | | | | |
| 5. Analyzing Ethernet LAN Switching | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 5. Analyzing Ethernet LAN Switching | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| 5. Analyzing Ethernet LAN Switching | Review the command tables | | | | |
| 5. Analyzing Ethernet LAN Switching | Watch video for this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this chapter | | | | |
| 6. Configuring Basic Switch Management | Read Foundation Topics | | | | |
| 6. Configuring Basic Switch Management | Review Key Topics using the book or companion website | | | | |
| 6. Configuring Basic Switch Management | Define Key Terms using the book or companion website | | | | |
| 6. Configuring Basic Switch Management | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 6. Configuring Basic Switch Management | Complete config checklists in this chapter using the companion website | | | | |
| 6. Configuring Basic Switch Management | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| 6. Configuring Basic Switch Management | Review command tables for this chapter | | | | |
| 6. Configuring Basic Switch Management | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 7. Configuring and Verifying Switch Interfaces | Read Foundation Topics | | | | |
| 7. Configuring and Verifying Switch Interfaces | Review Key Topics using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 7. Configuring and Verifying Switch Interfaces | Define Key Terms using the book or companion website | | | | |
| 7. Configuring and Verifying Switch Interfaces | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 7. Configuring and Verifying Switch Interfaces | Review command tables for this chapter | | | | |
| 7. Configuring and Verifying Switch Interfaces | Complete all memory tables in this chapter using the companion website | | | | |
| 7. Configuring and Verifying Switch Interfaces | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| Part II. Implementing Ethernet LANs | Complete all exercises in Part II Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 8. Implementing Ethernet Virtual LANs | Read Foundation Topics | | | | |
| 8. Implementing Ethernet Virtual LANs | Review Key Topics using the book or companion website | | | | |
| 8. Implementing Ethernet Virtual LANs | Define Key Terms using the book or companion website | | | | |
| 8. Implementing Ethernet Virtual LANs | Repeat DIKTA questions using the book or PTP exam engine | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 8. Implementing Ethernet Virtual LANs | Complete config checklists in this chapter using the companion website | | | | |
| 8. Implementing Ethernet Virtual LANs | Review command tables for this chapter | | | | |
| 8. Implementing Ethernet Virtual LANs | Complete all memory tables in this chapter using the companion website | | | | |
| 8. Implementing Ethernet Virtual LANs | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| 8. Implementing Ethernet Virtual LANs | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 9. Spanning Tree Protocol Concepts | Read Foundation Topics | | | | |
| 9. Spanning Tree Protocol Concepts | Review Key Topics using the book or companion website | | | | |
| 9. Spanning Tree Protocol Concepts | Define Key Terms using the book or companion website | | | | |
| 9. Spanning Tree Protocol Concepts | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 9. Spanning Tree Protocol Concepts | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 10. RSTP and EtherChannel Configuration | Read Foundation Topics | | | | |
| 10. RSTP and EtherChannel Configuration | Review Key Topics using the book or companion website | | | | |
| 10. RSTP and EtherChannel Configuration | Define Key Terms using the book or companion website | | | | |
| 10. RSTP and EtherChannel Configuration | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 10. RSTP and EtherChannel Configuration | Complete config checklists in this chapter using the companion website | | | | |
| 10. RSTP and EtherChannel Configuration | Review command tables for this chapter | | | | |
| 10. RSTP and EtherChannel Configuration | Complete all memory tables in this chapter using the companion website | | | | |
| 10. RSTP and EtherChannel Configuration | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| 10. RSTP and EtherChannel Configuration | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| Part III. Implementing VLANs and STP | Complete all exercises in Part III Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 11. Perspectives on IPv4 Subnetting | Read Foundation Topics | | | | |
| 11. Perspectives on IPv4 Subnetting | Review Key Topics using the book or companion website | | | | |
| 11. Perspectives on IPv4 Subnetting | Define Key Terms using the book or companion website | | | | |
| 11. Perspectives on IPv4 Subnetting | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 11. Perspectives on IPv4 Subnetting | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 12. Analyzing Classful IPv4 Networks | Read Foundation Topics | | | | |
| 12. Analyzing Classful IPv4 Networks | Review Key Topics using the book or companion website | | | | |
| 12. Analyzing Classful IPv4 Networks | Define Key Terms using the book or companion website | | | | |
| 12. Analyzing Classful IPv4 Networks | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 12. Analyzing Classful IPv4 Networks | Complete all memory tables in this chapter using the companion website | | | | |
| 12. Analyzing Classful IPv4 Networks | Practice analyzing classful IPv4 networks using Appendix D on the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 12. Analyzing Classful IPv4 Networks | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 13. Analyzing Subnet Masks | Read Foundation Topics | | | | |
| 13. Analyzing Subnet Masks | Review Key Topics using the book or companion website | | | | |
| 13. Analyzing Subnet Masks | Define Key Terms using the book or companion website | | | | |
| 13. Analyzing Subnet Masks | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 13. Analyzing Subnet Masks | Complete all memory tables in this chapter using the companion website | | | | |
| 13. Analyzing Subnet Masks | Practice analyzing subnet masks using Appendix E on the companion website | | | | |
| 13. Analyzing Subnet Masks | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 14. Analyzing Existing Subnets | Read Foundation Topics | | | | |
| 14. Analyzing Existing Subnets | Review Key Topics using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 14. Analyzing Existing Subnets | Define Key Terms using the book or companion website | | | | |
| 14. Analyzing Existing Subnets | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 14. Analyzing Existing Subnets | Complete all memory tables in this chapter using the companion website | | | | |
| 14. Analyzing Existing Subnets | Practice mask analysis using Appendix F on the companion website | | | | |
| 14. Analyzing Existing Subnets | Practice analyzing existing subnets using Appendix F on the companion website | | | | |
| 14. Analyzing Existing Subnets | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 15. Subnet Design | Read Foundation Topics | | | | |
| 15. Subnet Design | Review Key Topics using the book or companion website | | | | |
| 15. Subnet Design | Define Key Terms using the book or companion website | | | | |
| 15. Subnet Design | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 15. Subnet Design | Practice subnet design using Appendix G on the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 15. Subnet Design | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| Part IV. IPv4 Addressing | Complete all exercises in Part IV Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 16. Operating Cisco Routers | Read Foundation Topics | | | | |
| 16. Operating Cisco Routers | Review Key Topics using the book or companion website | | | | |
| 16. Operating Cisco Routers | Define Key Terms using the book or companion website | | | | |
| 16. Operating Cisco Routers | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 16. Operating Cisco Routers | Review command tables for this chapter | | | | |
| 16. Operating Cisco Routers | Complete all memory tables in this chapter using the companion website | | | | |
| 16. Operating Cisco Routers | Do labs listed for this chapter at blog.certskills.com | | | | |
| 16. Operating Cisco Routers | Watch video for this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Read Foundation Topics | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Review Key Topics using the book or companion website | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Define Key Terms using the book or companion website | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Review command tables for this chapter | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Do labs listed for this chapter at blog.certskills.com | | | | |
| 17. Configuring IPv4 Addresses and Static Routes | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 18. IP Routing in the LAN | Read Foundation Topics | | | | |
| 18. IP Routing in the LAN | Review Key Topics using the book or companion website | | | | |
| 18. IP Routing in the LAN | Define Key Terms using the book or companion website | | | | |
| 18. IP Routing in the LAN | Repeat DIKTA questions using the book or PTP exam engine | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 18. IP Routing in the LAN | Complete config checklists in this chapter using the companion website | | | | |
| 18. IP Routing in the LAN | Review command tables for this chapter | | | | |
| 18. IP Routing in the LAN | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| 18. IP Routing in the LAN | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 19. IP Addressing on Hosts | Read Foundation Topics | | | | |
| 19. IP Addressing on Hosts | Review Key Topics using the book or companion website | | | | |
| 19. IP Addressing on Hosts | Define Key Terms using the book or companion website | | | | |
| 19. IP Addressing on Hosts | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 19. IP Addressing on Hosts | Review command tables for this chapter | | | | |
| 19. IP Addressing on Hosts | Watch video for this chapter using the companion website | | | | |
| 20. Troubleshooting IPv4 Routing | Read Foundation Topics | | | | |
| 20. Troubleshooting IPv4 Routing | Review Key Topics using the book or companion website | | | | |
| 20. Troubleshooting IPv4 Routing | Define Key Terms using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| Part V. IPv4 Routing | Complete all exercises in Part V Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 21. Understanding OSPF Concepts | Read Foundation Topics | | | | |
| 21. Understanding OSPF Concepts | Review Key Topics using the book or companion website | | | | |
| 21. Understanding OSPF Concepts | Define Key Terms using the book or companion website | | | | |
| 21. Understanding OSPF Concepts | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 21. Understanding OSPF Concepts | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 22. Implementing Basic OSPF Features | Read Foundation Topics | | | | |
| 22. Implementing Basic OSPF Features | Review Key Topics using the book or companion website | | | | |
| 22. Implementing Basic OSPF Features | Define Key Terms using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 22. Implementing Basic OSPF Features | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 22. Implementing Basic OSPF Features | Complete config checklists in this chapter using the companion website | | | | |
| 22. Implementing Basic OSPF Features | Review command tables for this chapter | | | | |
| 22. Implementing Basic OSPF Features | Do labs listed for this chapter at blog.certskills.com | | | | |
| 22. Implementing Basic OSPF Features | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 23. Implementing Optional OSPF Features | Read Foundation Topics | | | | |
| 23. Implementing Optional OSPF Features | Review Key Topics using the book or companion website | | | | |
| 23. Implementing Optional OSPF Features | Define Key Terms using the book or companion website | | | | |
| 23. Implementing Optional OSPF Features | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 23. Implementing Optional OSPF Features | Complete all memory tables in this chapter using the companion website | | | | |
| 23. Implementing Optional OSPF Features | Complete config checklists in this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 23. Implementing Optional OSPF Features | Review command tables for this chapter | | | | |
| 23. Implementing Optional OSPF Features | Do labs listed for this chapter at blog.certskills.com | | | | |
| 23. Implementing Optional OSPF Features | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 24. OSPF Neighbors and Route Selection | Read Foundation Topics | | | | |
| 24. OSPF Neighbors and Route Selection | Review Key Topics using the book or companion website | | | | |
| 24. OSPF Neighbors and Route Selection | Define Key Terms using the book or companion website | | | | |
| 24. OSPF Neighbors and Route Selection | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 24. OSPF Neighbors and Route Selection | Review command tables for this chapter | | | | |
| 24. OSPF Neighbors and Route Selection | Complete all memory tables in this chapter using the companion website | | | | |
| 24. OSPF Neighbors and Route Selection | Watch video for this chapter using the companion website | | | | |
| Part VI. OSPF | Complete all exercises in Part VI Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 25. Fundamentals of IP Version 6 | Read Foundation Topics | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 25. Fundamentals of IP Version 6 | Review Key Topics using the book or companion website | | | | |
| 25. Fundamentals of IP Version 6 | Define Key Terms using the book or companion website | | | | |
| 25. Fundamentals of IP Version 6 | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 25. Fundamentals of IP Version 6 | Review command tables for this chapter | | | | |
| 25. Fundamentals of IP Version 6 | Complete all memory tables in this chapter using the companion website | | | | |
| 25. Fundamentals of IP Version 6 | Practice abbreviating and expanding addresses using Appendix H on the companion website | | | | |
| 25. Fundamentals of IP Version 6 | Practice calculating the IPv6 subnet prefix using Appendix H on the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 26. IPv6 Addressing and Subnetting | Read Foundation Topics | | | | |
| 26. IPv6 Addressing and Subnetting | Review Key Topics using the book or companion website | | | | |
| 26. IPv6 Addressing and Subnetting | Define Key Terms using the book or companion website | | | | |
| 26. IPv6 Addressing and Subnetting | Repeat DIKTA questions using the book or PTP exam engine | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 26. IPv6 Addressing and Subnetting | Complete all memory tables in this chapter using the companion website | | | | |
| 26. IPv6 Addressing and Subnetting | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 27. Implementing IPv6 Addressing on Routers | Read Foundation Topics | | | | |
| 27. Implementing IPv6 Addressing on Routers | Review Key Topics using the book or companion website | | | | |
| 27. Implementing IPv6 Addressing on Routers | Define Key Terms using the book or companion website | | | | |
| 27. Implementing IPv6 Addressing on Routers | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 27. Implementing IPv6 Addressing on Routers | Review command tables for this chapter | | | | |
| 27. Implementing IPv6 Addressing on Routers | Complete all memory tables in this chapter using the companion website | | | | |
| 27. Implementing IPv6 Addressing on Routers | Practice EUI-64 and solicited node multicast problems using Appendix I on the companion website | | | | |
| 27. Implementing IPv6 Addressing on Routers | Do labs listed for this chapter at blog.certskills.com | | | | |
| 27. Implementing IPv6 Addressing on Routers | Watch video for this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 28. Implementing IPv6 Addressing on Hosts | Read Foundation Topics | | | | |
| 28. Implementing IPv6 Addressing on Hosts | Review Key Topics using the book or companion website | | | | |
| 28. Implementing IPv6 Addressing on Hosts | Define Key Terms using the book or companion website | | | | |
| 28. Implementing IPv6 Addressing on Hosts | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 28. Implementing IPv6 Addressing on Hosts | Complete all memory tables in this chapter using the companion website | | | | |
| 28. Implementing IPv6 Addressing on Hosts | Review command tables for this chapter | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 29. Implementing IPv6 Routing | Read Foundation Topics | | | | |
| 29. Implementing IPv6 Routing | Review Key Topics using the book or companion website | | | | |
| 29. Implementing IPv6 Routing | Define Key Terms using the book or companion website | | | | |
| 29. Implementing IPv6 Routing | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 29. Implementing IPv6 Routing | Review command tables for this chapter | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 29. Implementing IPv6 Routing | Complete all memory tables in this chapter using the companion website | | | | |
| 29. Implementing IPv6 Routing | Do labs listed for this chapter at blog.certskills.com | | | | |
| 29. Implementing IPv6 Routing | Watch video for this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| Part VII. IP Version 6 | Complete all exercises in Part VII Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| Final Review | Take practice test in study mode for all book questions in practice test software | | | | |
| Final Review | Review all Key Topics in all chapters or in the Key Topics app using the companion website | | | | |
| Final Review | Review all Key Terms in all chapters or using the Key Terms flashcards on the companion website | | | | |
| Final Review | Complete all memory tables for all chapters using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Final Review | Take practice test in practice exam mode using Exam Bank #1 questions for all chapters | | | | |
| Final Review | Take practice test in practice exam mode using Exam Bank #2 questions for all chapters | | | | |

# APPENDIX K

# Topics from Previous Editions

We base the books' contents on the Cisco exam topics; that is, the books attempt to cover the topics Cisco lists as exam topics. However, the book authoring process does create some challenges, particularly with the balance of what to include in the books and what to leave out.

For instance, when I compared a new exam to the old, Cisco had removed some topics—and I might want to keep the content in the book. There are a few reasons why. Sometimes I just expect that some readers will still want to read about that technology. Also, more than a few schools use these books as textbooks, and including some of the older-but-still-relevant topics can be a help. And keeping the old material available on each book's companion website takes only a little extra work, so we do just that.

Some of the older topics that I choose to keep on the companion website are small, so I collect them into this appendix. Other topics happen to have been an entire chapter in a previous edition of the books, so we include those topics each as a separate appendix. Regardless, the material exists here in this appendix, and in the appendices that follow, for your use if you have a need. But do not feel as though you must read this appendix for the current exam.

The topics in this appendix are as follows:

- IPv4 Address Types
- Bandwidth and Clock Rate on Serial Interfaces
- Implementing DHCP
- Using traceroute to Isolate Problems on Two Routers
- Default Routes with SLAAC on Router Interfaces

**NOTE** The content under the heading "IPv4 Address Types" was most recently published for the 100-105 Exam in 2016, in Chapter 20 of the *Cisco CCNA ICND1 100-105 Official Cert Guide*.

# IPv4 Address Types

The IPv4 address space includes three major categories of addresses: unicast, broadcast, and multicast. For the current exam, Cisco lists one exam topic that asks you to compare and contrast these address types. To help you make those comparisons, this section explains multicast addressing, while pulling together the key ideas about unicast and broadcast IP addresses that have already been introduced, to pull the ideas together.

## Review of Unicast (Class A, B, and C) IP Addresses

Unicast IP addresses are those Class A, B, and C IP addresses assigned to hosts, router interfaces, and other networking devices. Because most discussions about IP addressing refer to unicast IP addresses, most of us just refer to them as IP addresses and leave out the word *unicast*.

Just to be complete and define the concept, unicast addresses identify one interface on one device to IP. Just like your postal address gives the post office an address to use to send letters to your one specific house or apartment, a unicast IP address gives the IP network an address to use to send packets to one specific host. However, with IP, instead of addressing the device, unicast addresses identify individual interfaces. For example:

- A router with four LAN interfaces and two WAN interfaces has six unicast addresses, each in a different subnet, one for each interface.
- A PC with both an Ethernet network interface card (NIC) and a wireless NIC would have two unicast IPv4 addresses, one for each interface.

## IP Broadcast Addresses

Broadcast IPv4 addresses give IP a way to send one packet that the network delivers to multiple hosts. IPv4 defines several types of broadcast addresses, with each type being used to reach a different set of hosts. These different broadcast IP addresses give different overhead protocols like DHCP the ability to efficiently reach all hosts in a specific part of the network. The following list reviews the three IP broadcast address types:

**Local broadcast address:** 255.255.255.255. Used to send a packet on a local subnet, knowing that routers will not forward the packet as is. Also called a *limited broadcast*.

**Subnet broadcast address:** One reserved address for each subnet, namely the numerically highest number in the subnet, as discussed in Chapter 13, "Analyzing Subnet Masks." A packet sent to a subnet broadcast address can be routed to the router connected to that subnet and then sent as a data link broadcast to all hosts in that one subnet. Also called an *all-hosts broadcast* to emphasize that all hosts in a subnet are reached, and also called a *directed broadcast*.

**Network broadcast address:** One reserved address for each classful network, namely the numerically highest number in the network. Used to send one packet to all hosts in that one network. Also called an *all-subnets broadcast*, referring to the fact that the packet reaches all subnets in a network.

This chapter has already shown how a local broadcast works, sending the message over the same subnet in which it was first transmitted, but no further. However, the other two types are a little more interesting.

Subnet and network broadcasts provide a way to send packets to all hosts in a subnet or network (respectively) while reducing waste. For instance, with a subnet broadcast, routers forward the packet just like any other IP packet going to that subnet. When that packet arrives at the router connected to that subnet, the last router then encapsulates the packet in a LAN broadcast, so that all hosts receive a copy. Figure K-1 shows the idea.

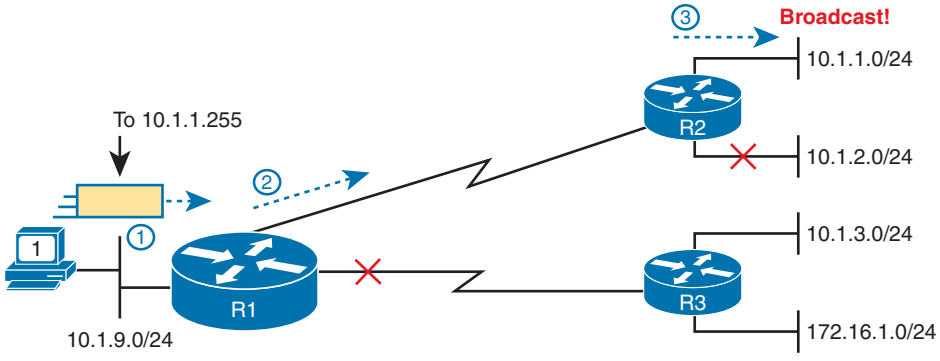



**Figure K-1** *Example of a Subnet Broadcast to 10.1.1.255*

The figure shows two key points. R1 does not flood or broadcast the frame to all other routers, instead routing it to the next router (R2 in this case) so that the packet reaches subnet 10.1.1.0/24. R2, connected to subnet 10.1.1.0/24, forwards the packet onto the LAN, but encapsulates the packet in an Ethernet broadcast frame, so that it reaches all hosts in the subnet.

The figure shows the intended use of the subnet broadcast address; however, it presents a security issue today. Many attacks start with a ping to subnet broadcast addresses, hoping to get many hosts to reply. Cisco changed the IOS default many years ago to disable the forwarding of subnet broadcasts onto a connected subnet (that is, it disables Step 3 in Figure K-1). That default setting is based on the **no ip directed-broadcast** interface subcommand.

A network broadcast packet (a packet with a network broadcast address as the destination) works in a similar way. To reach all subnets, however, the routers create copies of the packet and flood it so it reaches all subnets inside the classful network. On any LAN interfaces, the packet is forwarded in a LAN broadcast, just as shown in Step 3 of Figure K-1.

## IPv4 Multicast Addresses (Class D Addresses)

Multicast IP addresses and the related protocols help solve a similar problem as compared to broadcast addresses, but mainly for applications, and without the same security issues experienced by broadcast addresses. To see how it works, consider this example. A video application may be designed to show live video feeds. If 10 people at the same remote site

in the same subnet want to watch the same video at the same time, the application could be designed so that the application sent the same video data 10 times, once to each client in the same subnet. An application designed to use Class D multicast addresses could send 1 packet, which the routers would route across the WAN, and then deliver a copy to all 10 hosts in the destination subnet.

When using multicast, all the hosts still use their individual unicast IP address for their normal traffic, while also using the same multicast IPv4 address for the multicast application. Any server or client that happens to use an application designed to take advantage of IP multicast then also uses the Class D multicast addresses that the application chooses to use. You can think of a Class D address more as a multicast group—in fact, it is often called that—because hosts join the group so that they can receive the packets sent by the multicast application.

Class D addresses begin with a first octet of between 224 and 239, with some ranges reserved for various purposes. Much of the Class D address space is set aside for a company to deploy one of these multicast applications, and then pick an address from the Class D range, and configure it to be used by a multicast application.

As an example, imagine the video application uses Class D address 226.1.1.1. Figure K-2 illustrates the process by which the application at the server on the left sends one multicast packet with destination address 226.1.1.1. Note that for this process to work, the hosts with * beside them registered with their local routers to notify the routers that the host wants to receive packets destined to multicast address 226.1.1.1. When the action in this figure begins, the routers collectively know which subnets have hosts that want a copy of multicasts sent to 226.1.1.1 and which subnets do not.

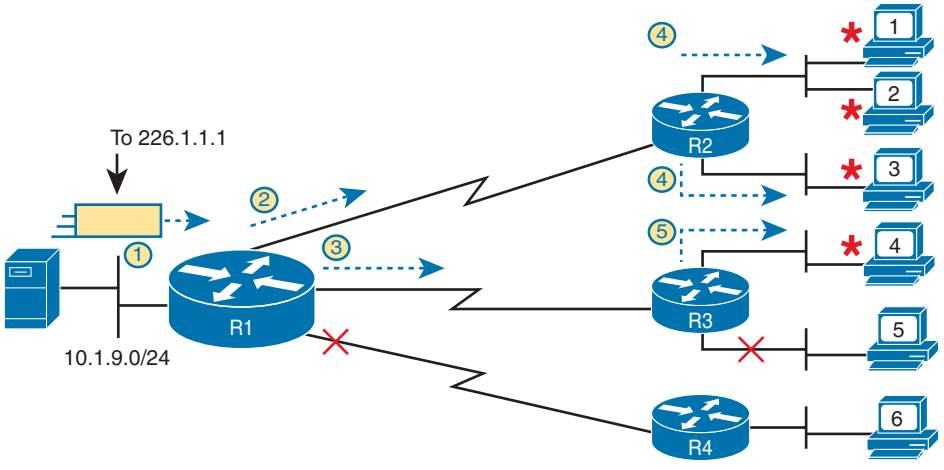



**Figure K-2** *Example of a Multicast Packet Flow for Three Registered Hosts*

Following the steps in the figure:

1. The server on the left generates and sends a multicast packet.
2. Router R1 replicates the packet to send a copy to both R2…
3. …and to R3. R1 does not replicate and send a copy to R4, because there are no hosts near R4 listening for packets sent to 226.1.1.1.

4. R2 processes the multicast packet received from R1, and because of the earlier host registration process, R2 knows that at least one host off both its LAN interfaces is listening for packets sent to 226.1.1.1. R2 therefore forwards a copy of the packet out each of its LAN interfaces.
5. R3 receives the multicast packet from R1 and uses the same kind of logic as R2. However, R3 knows from the earlier host registration process that only one of its LAN interfaces connects to a subnet with hosts listening for packets sent to 226.1.1.1, so R3 forwards a copy of the packet out that one interface only.

As you can see from this example, the server sent one packet, and the routers replicated the packet so it reached all the correct locations in the network.

As another comparison between unicast and multicast addresses, note that multicast addresses may be used as destination IP addresses only, whereas unicast addresses may be used as both the destination and source address. For instance, consider the packets in the example shown in Figure K-2. All those packets flow from one host, so the packet uses a unicast IP address of that host's unicast IP address.

Finally, to complete one more comparison between unicast IP addressing and multicast IP addressing, think about that last hop router in the example shown in Figure K-1. If a router such as R2 or R3 had forwarded a unicast IP packet, the router would look in its ARP cache to find the unicast IP address for the destination in that connected subnet and the associated unicast MAC address. That will not work when forwarding a multicast packet with a multicast (Class D) destination IP address.

To encapsulate a multicast IP packet over an Ethernet LAN, IP multicast calculates the destination MAC address with a simple process. The process copies the last 23 bits of the IP address behind a reserved 25-bit prefix to form the 48-bit destination MAC address. The resulting MAC address, called a multicast MAC address, begins with hex 01005E. So, the multicast IP packet, encapsulated in the multicast Ethernet frame, is forwarded out the router interface onto the LAN. At that point, the switches take one of the following approaches to forwarding the frame so that all hosts that want a copy of the frame get a copy:

- Flood the multicast frame as if it were a broadcast
- Use other Ethernet multicast features that flood the frame only to those same devices that registered to receive a copy

If you feel as though these few pages probably left out some detail, indeed, several books have been written about IP multicast all to itself. The topic is indeed large. For this book's purposes, know the main comparison points with unicast addressing. Multicast addressing gives applications that need to communicate the same data at the same time to multiple hosts a much more efficient way to do that. If the application is written to make use of IP multicast, the application can consume much less traffic in the network, as compared to using unicast IP addresses and sending every host a copy of the packet.

## Comparing and Contrasting IP Address Types

The preceding pages reviewed unicast and broadcast addresses and explained the core concepts behind IP multicast addresses. Table K-1 summarizes the key comparison points mentioned throughout this section for convenient study.

**Table K-1** Comparisons of Unicast, Broadcast, and Multicast IP Addresses

| | Unicast | Broadcast | Multicast |
|---|---|---|---|
| Primarily used for data sent by the most common user apps (web, email, chat, and so on) | Yes | No | No |
| Assigned to hosts with DHCP | Yes | No | No |
| Uses Class A, B, and C addresses | Yes | No | No |
| Primarily used by overhead protocols (DHCP, ARP) to send one message to more than one device | No | Yes | No |
| Used as destination IP address only | No | Yes | Yes |
| Primarily used by applications that send the same data at the same time to multiple clients | No | No | Yes |
| Uses Class D addresses | No | No | Yes |

**NOTE** The content under the heading "Bandwidth and Clock Rate on Serial Interfaces" was most recently published for the 100-105 Exam in 2016, in Chapter 17 of the *CCENT/CCNA ICND1 100-105 Official Cert Guide*.

## Bandwidth and Clock Rate on Serial Interfaces

WAN serial links can run at a wide variety of speeds. To deal with the wide range of speeds, routers physically slave themselves to the speed as dictated by the CSU/DSU through a process called *clocking*. As a result, routers can use serial links without the need for additional configuration or autonegotiation to sense the serial link's speed. The CSU/DSU knows the speed, the CSU/DSU sends clock pulses over the cable to the router, and the router reacts to the clocking signal.

To build a serial link in a home lab, the routers can use serial interface cards that normally use an external CSU/DSU, and make a serial link, without requiring the expense of two CSU/DSUs. Figure K-3 shows the concept. To make it work, the link uses two serial cables—one a DTE cable and the other a DCE cable—which swap the transmit and receive pair on the cables.

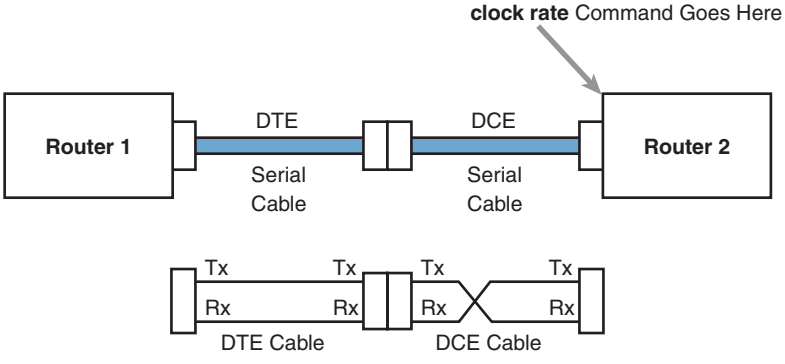



**Figure K-3** *Serial Link in Lab*

Using the correct cabling works, as long as you add one command: the **clock rate** interface subcommand. This command tells that router the speed at which to transmit bits on a serial link like the one shown in Figure K-3. The **clock rate** command is not needed on real serial links, because the CSU/DSU provides the clocking. When you create a serial link in the lab using cables, without any real CSU/DSUs on the link, the router with the DCE cable must supply that clocking function, and the **clock rate** command tells the router to provide it.

**NOTE** Newer router IOS versions automatically add a default **clock rate 2000000** command on serial interfaces that have a DCE cable connected to them. While helpful, this speed might be too high for some types of back-to-back serial cables, so consider using a lower speed in lab.

Example K-1 shows the configuration of the **clock rate** command. The end of the example verifies that this router can use the **clock rate** command with the **show controllers** command. This command confirms that R1 has a V.35 DCE cable connected.

**Example K-1** *Router R1 Configuration with the* **clock rate** *Command*

```
R1# show running-config
! lines omitted for brevity
interface Serial0/0/0
   ip address 172.16.4.1 255.255.255.0
   clock rate 2000000
!
interface Serial0/0/1
   ip address 172.16.5.1 255.255.255.0
   clock rate 128000

! lines omitted for brevity

R1# show controllers serial 0/0/1
Interface Serial0
Hardware is PowerQUICC MPC860
DCE V.35, clock rate 128000
idb at 0x8169BB20, driver data structure at 0x816A35E4
! Lines omitted for brevity
```

**NOTE** The **clock rate** command does not allow just any speed to be configured. However, the list of speeds does vary from router to router.

Some people confuse the router **bandwidth** command with the **clock rate** command. The **clock rate** command sets the actual Layer 1 speed used on the link, if no CSU/DSU is used, as just described. Conversely, every router interface has a bandwidth setting, either by default or configured. The bandwidth of the interface is the documented speed of the interface, which does not have to match the actual Layer 1 speed used on the interface.

That bandwidth setting does not impact how fast the interface transmits data. Instead, routers use the interface bandwidth setting as both documentation and as input to some other processes. For instance, the Open Shortest Path First (OSPF) and Enhanced Interior Gateway Routing Protocol (EIGRP) routing protocols base their routing protocol metrics on the bandwidth by default.

Example K-2 highlights the bandwidth setting on Router R1's S0/0/1 interface, as configured in the previous example. In that previous example, the **clock rate 128000** command sets the clock rate to 128 kbps, but it leaves the **bandwidth** command unset. As a result, IOS uses the default serial bandwidth setting of 1544, which means 1544 kbps—which is the speed of a T1 serial link.

**Example K-2** *Router Bandwidth Settings*

```
R1# show interfaces s0/0/1
Serial0/0/1 is up, line protocol is up
   Hardware is WIC MBRD Serial
   Description: link to R3
   Internet address is 10.1.13.1/24
   MTU 1500 bytes, BW 1544 Kbit/sec, DLY 20000 usec,
      reliability 255/255, txload 1/255, rxload 1/255
   Encapsulation HDLC, loopback not set
```

The common mistake people make is to know about clock rate but mistakenly think that the bandwidth setting is just another term for "clock rate." It is not. Follow these rules to find these two interface settings:

To see the clock rate, look for the **clock rate** interface subcommand in the configuration, or use the **show controllers serial** *number* command (as shown in Example K-1).

To see the bandwidth setting on an interface, look for the **bandwidth** interface subcommand in the configuration, or use the **show interfaces** [*type number*] command (as shown in Example K-2).

Note that using default bandwidth settings on most router interfaces makes sense, with the exception of serial interfaces. IOS defaults to a bandwidth of 1544 (meaning 1544 kbps, or 1.544 Mbps) for serial interfaces, regardless of the speed dictated by the provider or by a **clock rate** command in the lab. Most engineers set the bandwidth to match the actual speed, for example, using the **bandwidth 128** interface subcommand on a link running at 128 kbps. On Ethernet 10/100 or 10/100/1000 interfaces, the router knows the speed used and dynamically sets the Ethernet interface's bandwidth to match.

**NOTE** The content under the heading "Implementing DHCP" was most recently published for the 100-105 Exam in 2016, in Chapter 20 of the *Cisco CCNA ICND1 100-105 Official Cert Guide.*

# Implementing DHCP

This section includes DHCP implementation topics from an earlier edition of the book.

## DHCP Server Configuration on Routers

A quick Google search on "DHCP server products" reveals that many companies offer DHCP server software. Cisco routers (and some Cisco switches) can also act as a DHCP server with just a little added configuration.

Configuring a Cisco router to act as a DHCP server uses a new configuration concept, one per subnet, called a *DHCP pool*. All the per-subnet settings go into a per-subnet DHCP pool. The only DHCP command that sits outside the pool is the command that defines the list of addresses excluded from being leased by DHCP. The Cisco IOS DHCP server configuration steps are as follows:

**Step 1.** Use the **ip dhcp excluded-address** *first last* command in global configuration mode to list addresses that should be excluded (that is, not leased by DHCP).

**Step 2.** Use the **ip dhcp pool** *name* command in global configuration mode to both create a DHCP pool for a subnet and to navigate into DHCP pool configuration mode. Then also

- **a.** Use the **network** *subnet-ID mask* or **network** *subnet-ID prefix-length* command in DHCP pool configuration mode to define the subnet for this pool.
- **b.** Use the **default-router** *address1 address2…* command in DHCP pool configuration mode to define default router IP address(es) in that subnet.
- **c.** Use the **dns-server** *address1 address2…* command in DHCP pool configuration mode to define the list of DNS server IP addresses used by hosts in this subnet.
- **d.** Use the **lease** *days hours minutes* command in DHCP pool configuration mode to define the length of the lease, in days, hours, and minutes.
- **e.** Use the **domain-name** *name* command in DHCP pool configuration mode to define the DNS domain name.
- **f.** Use the **next-server** *ip-address* command in DHCP pool configuration mode to define the TFTP server IP address used by any hosts (like phones) that need a TFTP server.

Of course, an example can help, particularly with so many configuration commands required. Figure K-4 shows the organization of the configuration, while sticking to pseudocode rather than the specific configuration commands. (Upcoming Example K-3 shows a matching configuration.) Note that for each of the two LAN subnets, there is a global command to exclude addresses, and then a group of settings for each of two different DHCP pools.

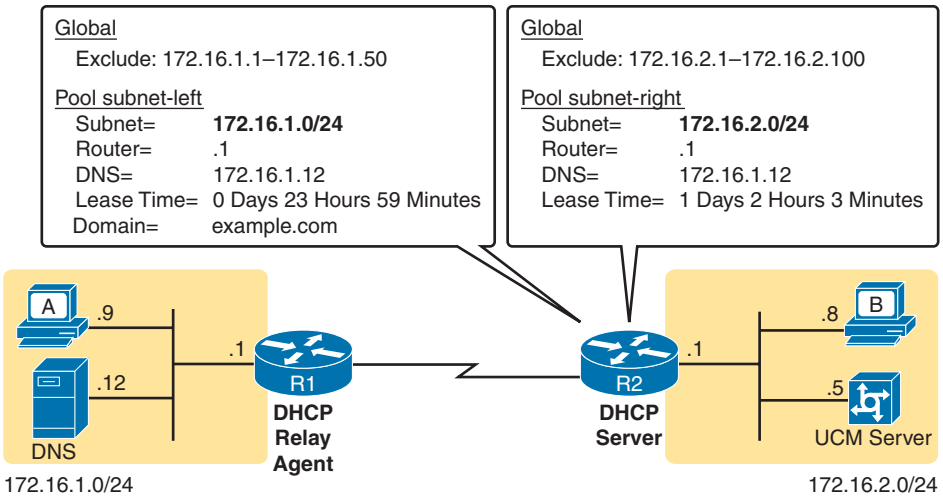



**Figure K-4** *DHCP Server Configuration Pseudocode*

**Example K-3** *R2 as a DHCP Server Per the Concepts in Figure K-4*

```
ip dhcp excluded-address 172.16.1.1 172.16.1.50
ip dhcp excluded-address 172.16.2.1 172.16.2.100
!
ip dhcp pool subnet-left
 network 172.16.1.0 255.255.255.0
 dns-server 172.16.1.12
 default-router 172.16.1.1
 lease 0 23 59
 domain-name example.com
 next-server 172.16.2.5
!
ip dhcp pool subnet-right
 network 172.16.2.0 /24
 dns-server 172.16.1.12
 default-router 172.16.2.1
 lease 1 2 3
 next-server 172.16.2.5
```

Focus on subnet 172.16.1.0/24 for a moment: the subnet is configured as pool subnet-left. The subnet ID and mask match the subnet ID chosen for that subnet. Then the global **ip dhcp excluded-address** command, just above, reserves 172.16.1.1 through 172.16.1.50, so that this DHCP server will not lease these addresses. The server will automatically exclude the subnet ID (172.16.1.0) as well, so this DHCP server will begin leasing IP addresses starting with the .51 address.

Now look at the details for subnet-right. It uses a DHCP pool **network** command with a prefix-style mask. It defines the same DNS server, as does the pool for the other subnet, but a different default router setting, because, of course, the default router in each subnet is different. This pool includes a lease time of 1:02:03 (1 day, 2 hours, and 3 minutes) just as an example.

Also note that both subnets list a TFTP server IP address of the Unified Communications Manager (UCM) server with the **next-server** command. In most cases, you would find this setting in the pools for subnets in which phones reside.

Finally, note that configuring a router as a DHCP server does not remove the need for the **ip helper-address** command. If DHCP clients still exist on LANs that do not have a DHCP server, then the routers connected to those LANs still need the **ip helper-address** command. For example, in Figure K-3, R1 would still need the **ip helper-address** command on its LAN interface. R2 would not need the command on its LAN interface, because R2 could service those requests, rather than needing to forward the DHCP messages to some other server.

## IOS DHCP Server Verification

The IOS DHCP server function has several different **show** commands. These three commands list most of the details:

**show ip dhcp binding:** Lists state information about each IP address currently leased to a client

**show ip dhcp pool** [*poolname*]**:** Lists the configured range of IP addresses, plus statistics for the number of currently leased addresses and the high-water mark for leases from each pool

**show ip dhcp server statistics:** Lists DHCP server statistics

Example K-4 shows sample output from two of these commands, based on the configuration from Figure K-4 and Example K-3. In this case, the DHCP server leased one IP address from each of the pools—one for host A and one for host B—as shown in the highlighted portions of the output.

**Example K-4** *Verifying Current Operation of a Router-Based DHCP Server*

```
R2# show ip dhcp binding
Bindings from all pools not associated with VRF:
IP address          Client-ID/              Lease expiration        Type
                    Hardware address/
                    User name
172.16.1.51         0063.6973.636f.2d30.    Oct 12 2012 02:56 AM    Automatic
                    3230.302e.3131.3131.
                    2e31.3131.312d.4661.
                    302f.30
172.16.2.101        0063.6973.636f.2d30.    Oct 12 2012 04:59 AM    Automatic
                    3230.302e.3232.3232.
                    2e32.3232.322d.4769.
                    302f.30
R2# show ip dhcp pool subnet-right
Pool subnet-right :
 Utilization mark (high/low)    : 100 / 0
 Subnet size (first/next)       : 0 / 0
 Total addresses                : 254
 Leased addresses               : 1
 Pending event                  : none
 1 subnet is currently in the pool :
 Current index        IP address range                    Leased addresses
 172.16.2.102         172.16.2.1       - 172.16.2.254     1
```

Note that the output in Example K-4 does not happen to list the excluded addresses, but it does show the effects. The addresses assigned to the clients end with .51 (host A, subnet 172.16.1.0) and .101 (host B, subnet 172.16.2.0), proving that the server did exclude the addresses as shown in the configuration in Example K-3. The server avoided the .1 through .50 addresses in subnet 172.16.1.0, and the .1 through .100 addresses in subnet 172.16.2.0.

**NOTE** The DHCP server keeps status (state) information about each DHCP client that leases an address. Specifically, it remembers the DHCP client ID and the IP address leased to the client. As a result, an IPv4 DHCP server can be considered to be a stateful DHCP server.

## Troubleshooting DHCP Services

To be prepared for the CCNA Simlet questions, you have to be ready to predict what symptoms would occur when the network was misconfigured in particular ways. This next section takes a similar approach, pointing out the most typical issues that could be introduced through incorrect or missing configuration, and then discussing what symptoms should happen and how to recognize those problems.

This section begins with a typical look at configuration mistakes and the symptoms that occur with those mistakes. In particular, this section looks at problems with the relay agent's helper address as well as the IOS DHCP server configuration. This section then looks at non-DHCP problems related to that data plane, breaking the problem into issues between the client and relay agent, and between the relay agent and DHCP server. The final section takes a short look at how a DHCP server prevents duplicate IP addresses between hosts that use static IP addresses and those that use DHCP.

### DHCP Relay Agent Configuration Mistakes and Symptoms

One configuration mistake that prevents a DHCP client from leasing an IP address is the misconfiguration or the omission of the **ip helper-address** interface subcommand on the router acting as the DHCP relay agent. The relay agent takes the incoming DHCP message, changes the destination address of the packet to be the address on the **ip helper-address** *address* command, and forwards the packet to that address. If the command is missing, the router does not attempt to forward the DHCP messages at all; if it is incorrect, the relay agent forwards the DHCP packets, but they never arrive at the actual DHCP server.

The main problem symptom in this case is the failure of a DHCP client to lease an address. If you can identify a client that has a problem, and you know what VLAN or subnet in which that host resides, you can then work to identify any routers connected to that subnet, to find and correct the **ip helper-address** subcommands.

Beyond that step, this list summarizes a few other related points.

- The DHCP relay agent feature is needed on interfaces only if the DHCP server is on a different subnet; it is not needed if the DHCP server is on the same subnet as the client.
- On routers with VLAN trunks (with a router-on-a-stick [ROAS] subinterface configuration), the subinterfaces also need an **ip helper-address** command (assuming they meet the first criteria in this list).

- If an exam question does not allow you to look at the configuration, use the **show ip interface** [*type number*] command to view the **ip helper-address** setting on an interface.

About that last point, Example K-5 shows an example of the **show ip interface g0/0** command. In this case, the interface has been configured with the **ip helper-address 172.16.2.11** command; the **show** command output basically restates that fact. Note that if there were no **ip helper-address** configured on the interface, the text would instead read "Helper address is not set."

**Example K-5** *Listing the Current Helper Address Setting with* **show ip interface**

```
R1# show ip interface g0/0
GigabitEthernet0/0 is up, line protocol is up
  Internet address is 182.16.1.1/24
  Broadcast address is 255.255.255.255
  Address determined by non-volatile memory
  MTU is 1500 bytes
  Helper address is 172.16.2.11
! Lines omitted for brevity (about 20 lines)
```

### IOS DHCP Server Configuration Mistakes and Symptoms

When using an IOS DHCP server, from a troubleshooting perspective, break issues into two broad categories: those that prevent DHCP clients from leasing an address and those that allow the lease but provide incorrect settings to the client.

First, the primary configuration mistake that causes a failure in the DHCP lease process is the misconfiguration of the **network** command. The problem revolves around these key facts:

- The packet from the relay agent to the DHCP server uses the relay agent's interface IP address as the source IP address in the forwarded DHCP message.
- The DHCP server compares that source IP address in the received DHCP packet to the **network** commands in its DHCP pools to find the right pool.
- Each **network** *subnet mask* command implies a range of addresses, just like any other IP network or subnet shown with a subnet mask.
- If the source IP address of the packet is not in the range of addresses implied by any **network** command in all the pools, the DHCP server has no pool to use for that request. The DHCP server does not know how to respond, so it does not reply at all.

As an example of that failure, consider the configuration shown in Figure K-5. The left side shows the configuration on R1, a DHCP relay agent that has two interfaces configured with the **ip helper-address 172.16.2.11** command. The DHCP server configuration on the right lists two pools, intended as one pool for each subnet off Router R1. However, the **network 172.16.3.0 /25** command implies an address range of 172.16.3.0 to 172.16.3.127, and the relay agent's interface address of 172.16.3.254 is not within that range of numbers. The solution would be to correct the DHCP server's **network** command to use a /24 mask.

**NOTE** The **ip helper-address** configuration on the left is correct. The figure uses the ROAS configuration here just to reinforce the comment in the earlier section that ROAS subinterfaces also need an **ip helper-address** subcommand.

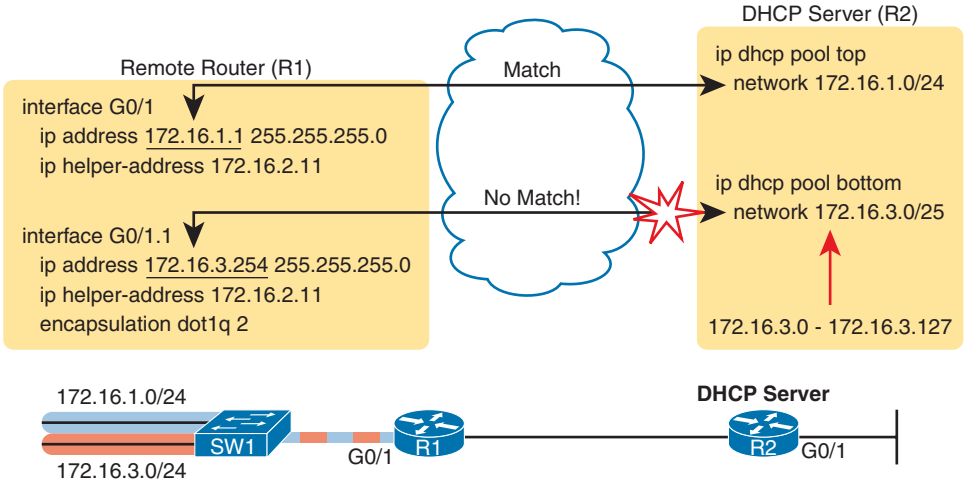



**Figure K-5** *A Sample Misconfiguration of a DHCP Pool* **network** *Command*

While you ultimately need to find this kind of problem and fix the configuration, on the exam you need to be ready to discover the root cause based on symptoms and **show** commands as well. So, when troubleshooting DHCP issues, and the client fails to lease an address, look at the IOS DHCP server's **network** commands. Calculate the range of IP addresses as if that command were defining a subnet. Then compare that range of addresses by the **network** command in each pool to the interface addresses on the DHCP relay agent routers. Every relay agent interface (that is, every interface with an **ip helper-address** command configured) should be included in a pool defined at the IOS DHCP server.

The DHCP server can also be misconfigured in a way that allows the lease of an address, but then causes other problems. If the lease process works, but the rest of the parameters given to the client are incorrect or missing, the client could operate, but operate poorly. This list summarizes the kinds of mistakes and the resulting symptoms:

- With the DNS server IP addresses incorrectly configured on the server (or omitted), hosts would fail to resolve hostnames into their associated IP addresses.
- With the default gateway IP address incorrectly configured on the server (or omitted), hosts could not communicate outside the local subnet.
- With the TFTP server IP address incorrectly configured (or omitted), an IP phone would fail to correctly load its configuration.

## IP Connectivity from DHCP Relay Agent to DHCP Server

For the DHCP process to work with a centralized server, IP broadcast packets must flow between the client and relay agent, and IP unicast packets must flow between the relay agent

and the DHCP server. Any problem that prevents the flow of these packets also prevents DHCP from working.

For perspective, consider the topology in Figure K-6, which again shows the relay agent on the left and the DHCP server on the right. The server uses IP address 172.16.2.11, and the relay agent uses interface address 172.16.1.1. Any failure that prevents the flow of IP packets between those two IP addresses would prevent host A from leasing an IP address.

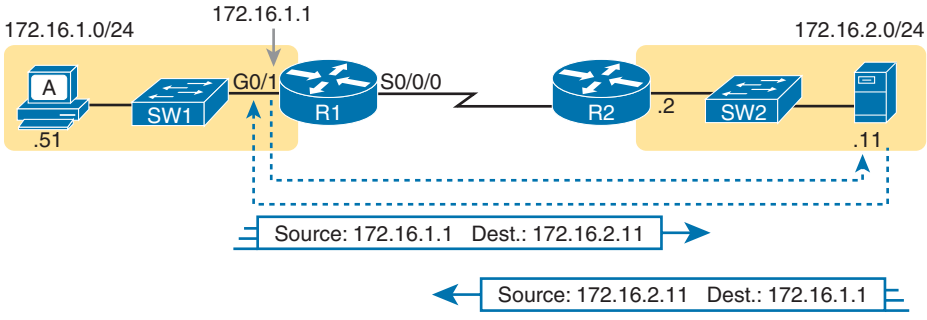



**Figure K-6** *Addresses Used Between Relay Agent and Server*

Remember that the IP addresses are used on the packets between the relay agent and server, and know that you may need to troubleshoot IP routing to ensure those packets can be delivered.

### LAN Connectivity Between the DHCP Client and Relay Agent

You might encounter a network environment where DHCP messages on the same LAN as the DHCP client all show a destination IP address of 255.255.255.255. What does that really mean? When a packet uses this 255.255.255.255 address:

- The address is called the *local broadcast address*.
- Packets sent to this address are not forwarded as-is by routers.
- On a LAN, the sender of an IP local broadcast packet encapsulates these IP packets in an Ethernet frame with an Ethernet broadcast destination address (FFFF.FFFF.FFFF), so the LAN broadcasts the frame.

As a result of the logic in these steps, the broadcast DHCP messages can easily flow between the client and router, as long as the LAN works.

### Summary of DHCP Troubleshooting

In summary, as a study tool, the following list summarizes the key troubleshooting ideas from this section on troubleshooting DHCP:

**Step 1.** If you're using a centralized DHCP server, at least one router on each remote subnet that has DHCP clients must act as DHCP relay agent and have a correctly configured **ip helper-address** *address* subcommand on the interface connected to that subnet.

**Step 2.** If you're using a centralized IOS DHCP server, make sure the DHCP pools' **network** commands match the entire network's list of router interfaces that have an **ip helper-address** command pointing to this DHCP server.

**Step 3.** Troubleshoot for any IP connectivity issues between the DHCP relay agent and the DHCP server, using the relay agent interface IP address and the server IP address as the source and destination of the packets.

**Step 4.** Troubleshoot for any LAN issues between the DHCP client and the DHCP relay agent.

Also, as one final note about DHCP in the real world, DHCP might seem dangerous at this point, with all the focus on potential problems in this section, combined with the importance of DHCP and its use by most end-user devices. However, DHCP has some great availability features. First, most DHCP servers set their lease times for at least a few days, often a week, or maybe longer. Combined with that, the DHCP protocol has several processes through which the client reconfirms the existing lease with the server, and re-leases the same IP address in advance of the expiration of the lease. Clients do not simply wait until the moment the lease would expire to then contact the DHCP server, hoping it is available. So the network can have outages, and DHCP clients that have already leased an address can continue to work without any problem.

### Detecting Conflicts with Offered Versus Used Addresses

Beyond troubleshooting the types of problems that would prevent DHCP from working, the IOS DHCP server tries to prevent another type of problem: assigning IP addresses with DHCP when another host tries to statically configure that same IP address. Although the DHCP server configuration clearly lists the addresses in the pool, plus those to be excluded from the pool, hosts can still statically configure addresses from the range inside the DHCP pool. In other words, no protocols prevent a host from statically configuring and using an IP address from within the range of addresses used by the DHCP server.

Knowing that some host might have statically configured an address from within the range of addresses in the DHCP pool, both DHCP servers and clients try to detect such problems, called *conflicts*, before the client uses a newly leased address.

DHCP servers detect conflicts by using pings. Before offering a new IP address to a client, the DHCP server first pings the address. If the server receives a response to the ping, some other host must already be using the address, which lets the server know a conflict exists. The server notes that particular address as being in conflict, and the server does not offer the address, moving on to the next address in the pool.

The DHCP client can also detect conflicts, but instead of using ping, it uses ARP. In the client case, when the DHCP client receives from the DHCP server an offer to use a particular IP address, the client sends an Address Resolution Protocol (ARP) request for that address. If another host replies, the DHCP client has found a conflict.

Example K-6 lists output from the router-based DHCP server on R2, after host B detected a conflict using ARP. Behind the scenes, host B used DHCP to request a lease, with the process working normally until host B used ARP and found some other device already used 172.16.2.102. At that point, host B then sent a DHCP message back to the server, rejecting the use of address 172.16.2.102. The example shows the router's log message related to host B's discovery of the conflict and a **show** command that lists all conflicted addresses.

**Example K-6** *Displaying Information About DHCP Conflicts in IOS*

```
*Oct 16 19:28:59.220: %DHCPD-4-DECLINE_CONFLICT: DHCP address conflict:   client
0063.6973.636f.2d30.3230.302e.3034.3034.2e30.3430.342d.4769.302f.30     declined
172.16.2.102.
R2# show ip dhcp conflict
IP address        Detection method   Detection time          VRF
172.16.2.102      Gratuitous ARP     Oct 16 2012 07:28 PM
```

The **show ip dhcp conflict** command lists the method through which the server added each address to the conflict list: either gratuitous ARP, as detected by the client, or ping, as detected by the server. The server avoids offering these conflicted addresses to any future clients, until the engineer uses the **clear ip dhcp conflict** command to clear the list.

**NOTE** The content under the heading "Using traceroute to Isolate the Problem to Two Routers" was most recently published for the 100-105 Exam in 2016, in Chapter 23 of the *Cisco CCNA ICND1 100-105 Official Cert Guide*.

## Using traceroute to Isolate the Problem to Two Routers

One of the best features of the **traceroute** command, as compared to ping, is that when it does not complete, it gives an immediate clue as to where to look next. With ping, when the ping fails, the next step is usually to use more **ping** commands. With traceroute, it tells you what router to try to connect and look at the routes and in which direction.

**NOTE** As a reminder, this book uses the term *forward route* for routes that send the packets sent by the **ping** or **traceroute** command, and *reverse route* for the packets sent back.

When a problem exists, a **traceroute** command results in a partial list of routers. Then the command either finishes with an incomplete list, or it runs until the user must stop the command. In either case, the output does not list all routers in the end-to-end route because of the underlying problem.

**NOTE** In addition, the **traceroute** command may not finish even though the network has no problems. Routers and firewalls may filter the messages sent by the **traceroute** command, or the TTL Exceeded messages, which would prevent the display of portions or all or part of the path.

The last router listed in the output of a **traceroute** command's output tells us where to look next to isolate the problem, as follows:

- Connect to the CLI of the last router listed, to look at forward route issues.
- Connect to the CLI of the next router that should have been listed, to look for reverse route issues.

To see why, consider an example based on the internetwork in Figure K-7. In this case, R1 uses an extended traceroute to host 5.5.5.5, with source IP address 1.1.1.1. This command's output lists router 2.2.2.2, then 3.3.3.3, and then the command cannot complete.

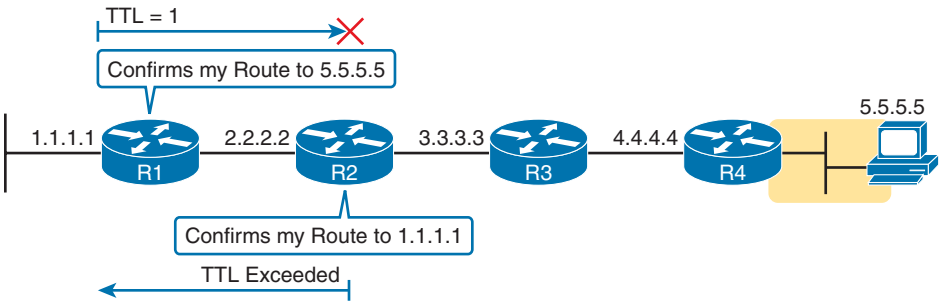



**Figure K-7** *Messages That Cause the* **traceroute** *Command to List 2.2.2.2*

First, Figure K-7 focuses on the first line of output: the line that lists first-hop router 2.2.2.2.

The figure shows the TTL=1 message at the top and the TTL Exceeded message back on the bottom. This first pair of messages in the figure must have worked, because without them, the **traceroute** command on R1 cannot have learned about a router with address 2.2.2.2. The first (top) message required R1 to have a route for 5.5.5.5, which sent the packets to R2 next. The TTL Exceeded message required that R2 have a route that matched address 1.1.1.1, to send the packets back to R1's LAN IP address.

Next, Figure K-8 focuses on the messages that allow the second line of output on R1's sample **traceroute** command: the line that correctly lists 3.3.3.3 as the next router in the route.

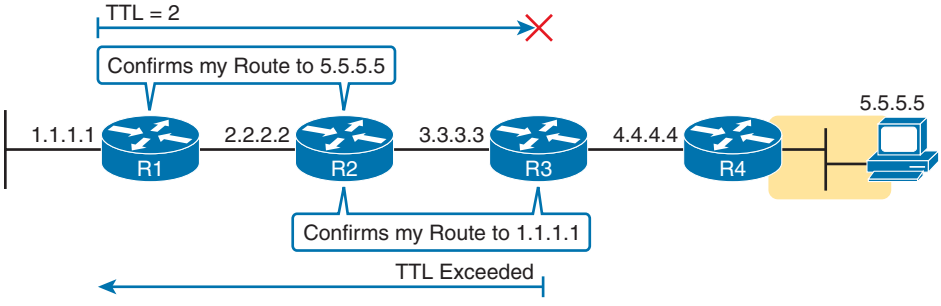



**Figure K-8** *Messages That Cause the* **traceroute** *Command to List 3.3.3.3*

Following the same logic, the **traceroute** output lists 3.3.3.3 because the messages in Figure K-8 must have worked. For these messages to flow, the routes listed in Figure K-7 must exist, plus new routes listed in K-8. Specifically, the TTL=2 packet at the top requires R2 to have a route for 5.5.5.5, which sends the packets to R3 next. The TTL Exceeded message requires that R3 have a route that matches address 1.1.1.1, to send the packets back toward R1's LAN IP address.

In this example, the **traceroute 5.5.5.5** command does not list any routers beyond 2.2.2.2 and 3.3.3.3 However, based on the figures, it is clear that 4.4.4.4 should be the next IP address listed. To help isolate the problem further, why might the next messages—the message with TTL=3 and the response—fail?

Figure K-9 points out the routing issues that can cause this command to not be able to list 4.4.4.4 as the next router. First, R3 must have a forward route matching destination 5.5.5.5

and forwarding the packet to Router R4. The return message requires a reverse route matching destination 1.1.1.1 and forwarding the packet back to Router R3.

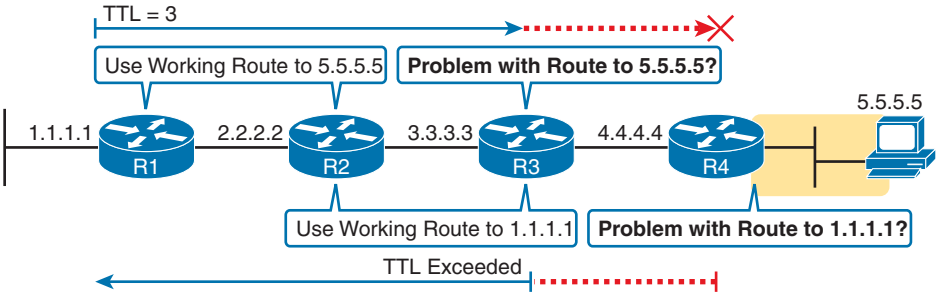



**Figure K-9** *Issues That Could Prevent* **traceroute** *from Listing 4.4.4.4*

In conclusion, for this example, if a routing problem prevents the **traceroute** command from working, the problem exists in one of two places: the forward route to 5.5.5.5 on Router R3, or the reverse route to 1.1.1.1 on R4.

**NOTE** The content under the heading "Default Routes with SLAAC on Router Interfaces" was most recently published in Chapter 32 of the *Cisco CCNA ICND1 100-105 Official Cert Guide*.

## Default Routes with SLAAC on Router Interfaces

Routers can use DHCP on their own interface and learn their IP address, mask, and even a default IPv4 route. In particular, that process can be useful on a router that connects to the Internet. The enterprise router uses DHCP as a client, learning its own IPv4 address with DHCP and adding a default route pointing to the ISP's router as the next-hop IPv4 address.

Routers can accomplish the same goals with IPv6, just with a few different protocols and methods. As with IPv4, the IPv6 enterprise router can dynamically learn its IPv6 address and dynamically create a default IPv6 route to the ISP's router. This section shows the details, with the enterprise router using SLAAC to learn its address and the information needed to create a default route.

First, the enterprise router that connects to the ISP, like Router R1 in Figure K-10, requires the configuration of the interface subcommand **ipv6 address autoconfig default**. This command tells the router that, on that interface, use SLAAC to build its own IPv6 address. R1 would act like any host that uses SLAAC, as shown in Step 2 of the figure, and send an NDP RS message over the link. As noted at Step 3, the ISP router would send back an RA message, announcing router ISP1's IPv6 address and the IPv6 prefix used on the link.

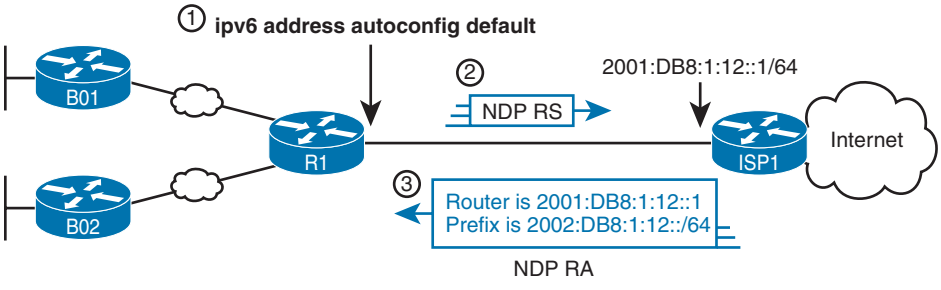



**Figure K-10** *Enterprise Router Using SLAAC to Build IPv6 Address and Default IPv6 Route*

When R1 receives the NDP RA message, it does the following:

**Interface address:** Builds its own interface IPv6 address using the SLAAC process, based on the prefix in the RA.

**Local /128 route:** Adds a local (/128) IPv6 route for the address, as it would for any interface IPv6 address.

**Connected route for prefix:** Adds a connected (/64) route for the prefix learned in the NDP RA message.

**Default route:** R1 adds a default route, to destination ::/0, with the next-hop address of ISP's link-local address, as learned in the RA sent by router ISP1.

Note that the router can be configured to add this default route or not. As shown in the figure, the router builds a default route. Using the **ipv6 address autoconfig** subcommand without the **default** keyword causes the router to build its address with SLAAC but not add a default route.

Example K-7 shows the three IPv6 routes on Router R1 just mentioned in the list. In particular, note the codes for the connected route and the default route; both codes begin with ND, meaning the route was learned with NDP. In particular, as highlighted in the legend part of the output, *ND* refers to an NDP-learned default route, and *NDp* refers to an NDP-learned prefix (as listed in the NDP RA message in Figure K-10 in this case). Note also that these same two routes have an administrative distance of 2, which is the default administrative distance of IPv6 routes learned with NDP.

**Example K-7** *Learning an Address and Default Static Route with DHCP*

```
R1# show ipv6 route
IPv6 Routing Table - default - 4 entries
Codes: C - Connected, L - Local, S - Static, U - Per-user Static route
       B - BGP, HA - Home Agent, MR - Mobile Router, R - RIP
       H - NHRP, I1 - ISIS L1, I2 - ISIS L2, IA - ISIS interarea
       IS - ISIS summary, D - EIGRP, EX - EIGRP external, NM - NEMO
       ND - ND Default, NDp - ND Prefix, DCE - Destination, NDr - Redirect
       O - OSPF Intra, OI - OSPF Inter, OE1 - OSPF ext 1, OE2 - OSPF ext 2
       ON1 - OSPF NSSA ext 1, ON2 - OSPF NSSA ext 2, la - LISP alt
       lr - LISP site-registrations, ld - LISP dyn-eid, a - Application
ND ::/0 [2/0]
     via FE80::22FF:FE22:2222, Serial0/0/0
NDp 2001:DB8:1:12::/64 [2/0]
     via Serial0/0/0, directly connected
L 2001:DB8:1:12:32F7:DFF:FE29:8560/128 [0/0]
    via Serial0/0/0, receive
! lines omitted for brevity
```

*This page intentionally left blank*

# APPENDIX L

# LAN Troubleshooting

**NOTE** This appendix contains content that was published as a chapter in one of the past editions of this book or a related book. The author includes this appendix with the current edition as extra reading for anyone interested in learning more. However, note that the content in this appendix has not been edited since it was published in the earlier edition, so references to exams and exam topics, and to other chapters, will be outdated. This appendix was previously published as Chapter 4 of the book *CCNA Routing and Switching ICND2 200-105 Official Cert Guide*, published in 2016.

This chapter discusses the LAN topics discussed in depth in the first three chapters, plus a few prerequisite topics, from a troubleshooting perspective.

Troubleshooting for any networking topic requires a slightly different mindset as compared to thinking about configuration and verification. When thinking about configuration and verification, it helps to think about basic designs, learn how to configure the feature correctly, and learn how to verify the correct configuration is indeed working correctly. However, to learn how to troubleshoot, you need to think about symptoms when the design is incorrect, or if the configuration does not match the good design. What symptoms occur when you make one type of mistake or another? This chapter looks at the common types of mistakes, and works through how to look at the status with **show** commands to find those mistakes.

This chapter breaks the material into four major sections. The first section tackles the largest topic, STP troubleshooting. STP is not likely to fail as a protocol; instead, STP may not be operating as designed, so the task is to find how STP is currently working and discover how to then make the configuration implement the correct design. The second major section then moves on to Layer 2 EtherChannels, which have a variety of small potential problems that can prevent the dynamic formation of an EtherChannel.

The third major section of the chapter focuses on the data plane forwarding of Ethernet frames on LAN switches, in light of VLANs, trunks, STP, and EtherChannels. That same section reviews the Layer 2 forwarding logic of a switch in light of these features. The fourth and final major section then examines VLAN and trunking issues, and how those issues impact switch forwarding.

Note that a few of the subtopics listed within the exam topics at the beginning of this chapter are not discussed in this chapter. This chapter does not discuss VTP beyond its basic features or Layer 3 EtherChannels.

# Foundation Topics

## Troubleshooting STP

STP questions tend to intimidate many test takers. STP uses many rules, with tiebreakers in case one rule ends with a tie. Without much experience with STP, people tend to distrust their own answers. Also, even those of us with networking jobs already probably do not troubleshoot STP very often, because STP works well. Often, troubleshooting STP is not about STP failing to do its job but rather about STP working differently than designed, with a different root switch, or different root ports (RP), and so on. Seldom does STP troubleshooting begin with a case in which STP has failed to prevent a loop.

This section reviews the rules for STP, while emphasizing some important troubleshooting points. In particular, this section takes a closer look at the tiebreakers that STP uses to make decisions. It also makes some practical suggestions about how to go about answering exam questions such as "which switch is the root switch?"

### Determining the Root Switch

Determining the STP root switch is easy if you know all the switches' BIDs: Just pick the lowest value. If the question lists the priority and MAC address separately, as is common in some **show** command output, pick the switch with the lowest priority, or in the case of a tie, pick the lower MAC address value.

And just to be extra clear, STP does not have nor need a tiebreaker for electing the root switch. The BID uses a switch universal MAC address as the last 48 bits of the BID. These MAC addresses are unique in the universe, so there should never be identical BIDs or the need for a tiebreaker.

For the exam, a question that asks about the root switch might not be so simple as listing a bunch of BIDs and asking you which one is "best." A more likely question is a simulator (sim) question in which you have to do any **show** commands you like or a multiple choice question that lists the output from only one or two commands. Then you have to apply the STP algorithm to figure out the rest.

When faced with an exam question using a simulator, or just the output in an exhibit, use a simple strategy of ruling out switches, as follows:

**Step 1.** Begin with a list or diagram of switches, and consider all as possible root switches.

**Step 2.** Rule out any switches that have an RP (**show spanning-tree**, **show spanning-tree root**), because root switches do not have an RP.

**Step 3.** Always try **show spanning-tree**, because it identifies the local switch as root directly: "This switch is the root" on the fifth line of output.

**Step 4.** Always try **show spanning-tree root**, because it identifies the local switch as root indirectly: The RP column is empty if the local switch is the root.

**Step 5.** When using a sim, rather than try switches randomly, chase the RPs. For example, if starting with SW1, and SW1's G0/1 is an RP, next try the switch on the other end of SW1's G0/1 port.

**Step 6.** When using a sim, use **show spanning-tree vlan** *x* on a few switches and record the root switch, RP, and designated port (DP). This strategy can quickly show you most STP facts.

The one step in this list that most people ignore is the idea of ruling out switches that have an RP. Root switches do not have an RP, so any switch with an RP can be ruled out as not being the root switch for that VLAN. Example L-1 shows two commands on switch SW2 in some LAN that confirms that SW2 has an RP and is therefore not the root switch.

**Example L-1** *Ruling Out Switches as Root Based on Having a Root Port*

```
SW2# show spanning-tree vlan 20 root

                                        Root Hello Max Fwd
Vlan                Root ID             Cost  Time Age Dly  Root Port
---------------- ------------------- --------- ----- --- ---  ------------
VLAN0020         32788 1833.9d7b.0e80          4    2   20  15  Gi0/2

SW2# show spanning-tree vlan 20

VLAN0020
  Spanning tree enabled protocol ieee
  Root ID     Priority    32788
              Address     1833.9d7b.0e80
              Cost        4
              Port        26 (GigabitEthernet0/2)
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID   Priority    32788  (priority 32768 sys-id-ext 20)
              Address     1833.9d7b.1380
              Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
              Aging Time  15  sec

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Gi0/1               Desg FWD 4         128.25   P2p
Gi0/2               Root FWD 4         128.26   P2p
```

Both commands identify SW2's G0/2 port as its RP, so if you follow the suggestions, the next switch to try in a sim question would be the switch on the other end of SW2's G0/2 interface.

## Determining the Root Port on Nonroot Switches

Determining the RP of a switch when **show** command output is available is relatively easy. As shown recently in Example L-1, both **show spanning-tree** and **show spanning-tree root** list the root port of the local switch, assuming it is not the root switch. The challenge comes more when an exam question makes you think through how the switches choose the RP based on the root cost of each path to the root switch, with some tiebreakers as necessary.

As a review, each nonroot switch has one, and only one, RP for a VLAN. To choose its RP, a switch listens for incoming Hello bridge protocol data units (BPDU). For each received Hello, the switch adds the cost listed in the hello BPDU to the cost of the incoming interface (the interface on which the Hello was received). That total is the root cost over that path. The lowest root cost wins, and the local switch uses its local port that is part of the least root cost path as its root port.

Most humans can analyze what STP chooses by using a network diagram and a slightly different algorithm. Instead of thinking about Hello messages and so on, approach the question as this: the sum of all outgoing port costs between the nonroot switch and the root. Repeating a familiar example, with a twist, Figure L-1 shows the calculation of the root cost. Note that SW3's Gi0/1 port has yet again had its cost configured to a different value.

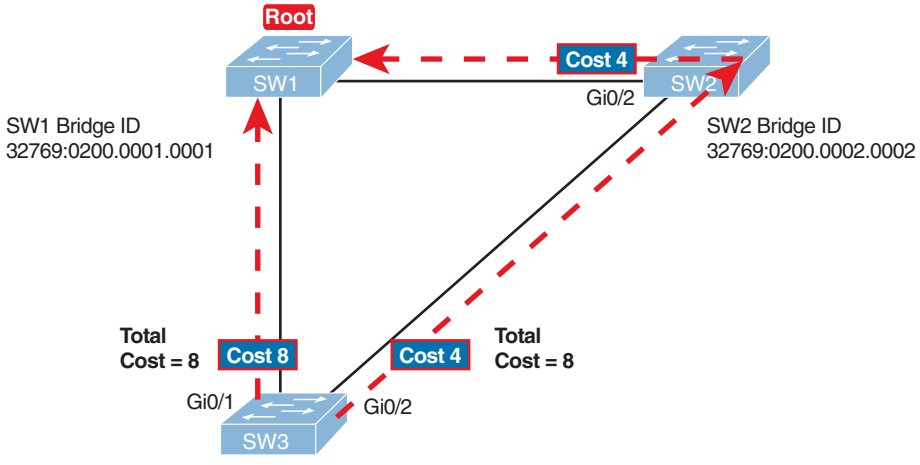



**Figure L-1** *SW3's Root Cost Calculation Ends in a Tie*

## STP Tiebreakers When Choosing the Root Port

Figure L-1 shows the easier process of adding the STP costs of the outgoing interfaces over each from SW3, a nonroot, to SW1, the root. It also shows a tie (on purpose), to talk about the tiebreakers.

When a switch chooses its root port, the first choice is to choose the local port that is part of the least root cost path. When those costs tie, the switch picks the port connected to the neighbor with the lowest BID. This tiebreaker usually breaks the tie, but not always. So, for completeness, the three tiebreakers are, in the order a switch uses them, as follows:

1. Choose based on the lowest neighbor bridge ID.
2. Choose based on the lowest neighbor port priority.
3. Choose based on the lowest neighbor internal port number.

(Note that the switch only considers the root paths that tie when thinking about these tiebreakers.)

For example, Figure L-1 shows that SW3 is not root and that its two paths to reach the root tie with their root costs of 8. The first tiebreaker is the lowest neighbor's BID. SW1's BID value is lower than SW2's, so SW3 chooses its G0/1 interface as its RP in this case.

The last two RP tiebreakers come into play only when two switches connect to each other with multiple links, as shown in Figure L-2. In that case, a switch receives Hellos on more than one port from the same neighboring switch, so the BIDs tie.

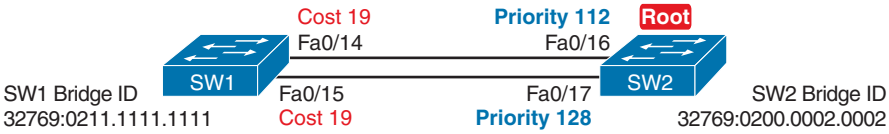



**Figure L-2** *Topology Required for the Last Two Tiebreakers for Root Port*

In this particular example, SW2 becomes root, and SW1 needs to choose its RP. SW1's port costs tie, at 19 each, so SW1's root cost over each path will tie at 19. SW2 sends Hellos over each link to SW1, so SW1 cannot break the tie based on SW1's neighbor BID because both list SW2's BID. So, SW1 has to turn to the other two tiebreakers.

**NOTE** In real life, most engineers would put these two links into an EtherChannel.

The next tiebreaker is a configurable option: the neighboring switch's port priority on each neighboring switch interface. Cisco switch ports default to a setting of 128, with a range of values from 0 through 255, with lower being better (as usual). In this example, the network engineer has set SW2's F0/16 interface with the **spanning-tree vlan 10 port-priority 112** command. SW1 learns that the neighbor has a port priority of 112 on the top link and 128 on the bottom, so SW1 uses its top (F0/14) interface as the root port.

If the port priority ties, which it often does due to the default values, STP relies on an internal port numbering on the neighbor. Cisco switches assign an internal integer to identify each interface on the switch. The nonroot looks for the neighbor's lowest internal port number (as listed in the Hello messages) and chooses its RP based on the lower number.

Cisco switches use an obvious numbering, with Fa0/1 having the lowest number, then Fa0/2, then Fa0/3, and so on. So, in Figure L-2, SW2's Fa0/16 would have a lower internal port number than Fa0/17; SW1 would learn those numbers in the Hello; and SW1 would use its Fa0/14 port as its RP.

### Suggestions for Attacking Root Port Problems on the Exam

Exam questions that make you think about the RP can be easy if you know where to look and the output of a few key commands is available. However, the more conceptual the question, the more you have to calculate the root cost over each path, correlate that to different **show** commands, and put the ideas together. The following list makes a few suggestions about how to approach STP problems on the exam:

1. If available, look at the **show spanning-tree** and **show spanning-tree root** commands. Both commands list the root port and the root cost (see Example L-1).
2. The **show spanning-tree** command lists cost in two places: the root cost at the top, in the section about the root switch; and the interface cost, at the bottom, in the per-interface section. Be careful, though; the cost at the bottom is the interface cost, not the root cost!

3. For problems where you have to calculate a switch's root cost:
   a. Memorize the default cost values: 100 for 10 Mbps, 19 for 100 Mbps, 4 for 1 Gbps, and 2 for 10 Gbps.
   b. Look for any evidence of the **spanning-tree cost** configuration command on an interface, because it overrides the default cost. Do not assume default costs are used.
   c. When you know a default cost is used, if you can, check the current actual speed as well. Cisco switches choose STP cost defaults based on the current speed, not the maximum speed.

## Determining the Designated Port on Each LAN Segment

Each LAN segment has a single switch that acts as the designated port (DP) on that segment. On segments that connect a switch to a device that does not even use STP—for example, segments connecting a switch to a PC or a router—the switch always wins, because it is the only device sending a Hello onto the link. However, links with two switches require a little more work to discover which should be the DP. By definition:

**Step 1.** For switches connected to the same LAN segment, the switch with the lowest cost to reach the root, as advertised in the Hello they send onto the link, becomes the DP on that link.

**Step 2.** In case of a tie, among the switches that tied on cost, the switch with the lowest BID becomes the DP.

For example, consider Figure L-3. This figure notes the root, RPs, and DPs and each switch's least cost to reach the root over its respective RP.

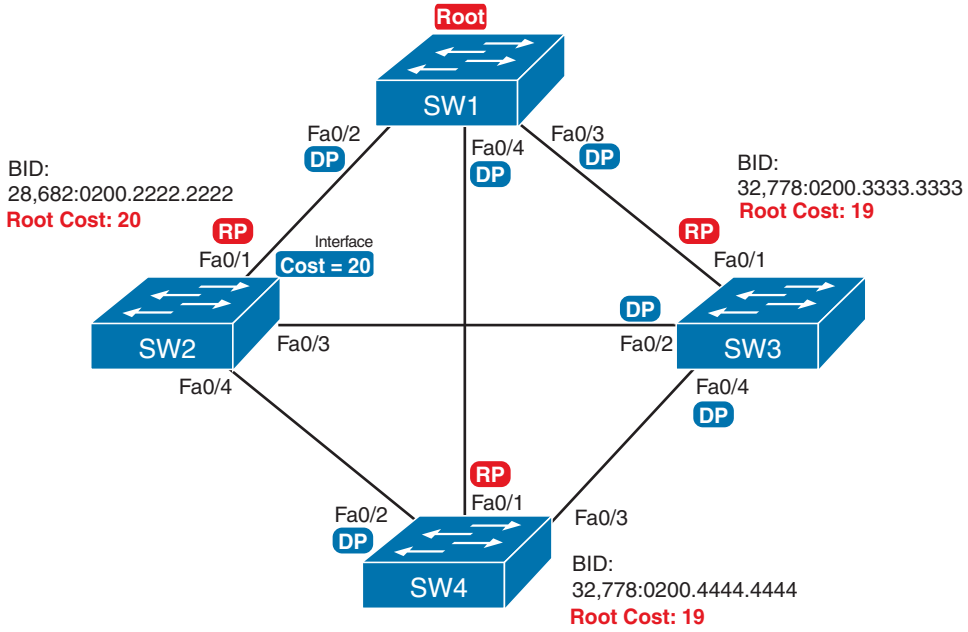



**Figure L-3** *Picking the DPs*

Focus on the segments that connect the nonroot switches for a moment:

**SW2–SW4 segment:** SW4 wins because of its root cost of 19, compared to SW2's root cost of 20.

**SW2–SW3 segment:** SW3 wins because of its root cost of 19, compared to SW2's root cost of 20.

**SW3–SW4 segment:** SW3 and SW4 tie on root cost, both with root cost 19. SW3 wins due to its better (lower) BID value.

Interestingly, SW2 loses and does not become DP on the links to SW3 and SW4 even though SW2 has the better (lower) BID value. The DP tiebreaker does use the lowest BID, but the first DP criteria is the lowest root cost, and SW2's root cost happens to be higher than SW3's and SW4's.

**NOTE** A single switch can connect two or more interfaces to the same collision domain, and compete to become DP, if hubs are used. In such cases, two different switch ports on the same switch tie, the DP choice uses the same two final tiebreakers as used with the RP selection: the lowest interface STP priority, and if that ties, the lowest internal interface number.

### Suggestions for Attacking Designated Port Problems on the Exam

As with exam questions asking about the RP, exam questions that make you think about the DP can be easy if you know where to look and the output of a few key commands is available. However, the more conceptual the question, the more you have to think about the criteria for choosing the DP: first the root cost of the competing switches, and then the better BID if they tie based on root cost.

The following list gives some tips to keep in mind when digging into a given DP issue. Some of this list repeats the suggestions for finding the RP, but to be complete, this list includes each idea as well.

1. If available, look at the **show spanning-tree** commands, at the list of interfaces at the end of the output. Then, look for the Role column, and look for Desg, to identify any DPs.
2. Identify the root cost of a switch directly by using the **show spanning-tree** command. But be careful! This command lists the cost in two places, and only the mention at the top, in the section about the root, lists the root cost.
3. For problems where you have to calculate a switch's root cost, do the following:
   a. Memorize the default cost values: 100 for 10 Mbps, 19 for 100 Mbps, 4 for 1 Gbps, and 2 for 10 Gbps.
   b. Look for any evidence of the **spanning-tree cost** configuration command on an interface, because it overrides the default cost. Do not assume default costs are used.
   c. When you know a default cost is used, if you can, check the current actual speed as well. Cisco switches choose STP cost defaults based on the current speed, not the maximum speed.

### STP Convergence

STP puts each RP and DP into a forwarding state, and ports that are neither RP nor DP into a blocking state. Those states may remain as is for days, weeks, or months. But at some point, some switch or link will fail, a link may change speeds (changing the STP cost), or the STP configuration may change. Any of these events can cause switches to repeat their STP algorithm, which may in turn change their own RP and any ports that are DPs.

When STP converges based on some change, not all the ports have to change their state. For instance, a port that was forwarding, if it still needs to forward, just keeps on forwarding. Ports that were blocking that still need to block keep on blocking. But when a port needs to change state, something has to happen, based on the following rules:

- For interfaces that stay in the same STP state, nothing needs to change.
- For interfaces that need to move from a forwarding state to a blocking state, the switch immediately changes the state to blocking.
- For interfaces that need to move from a blocking state to a forwarding state, the switch first moves the interface to listening state, then learning state, each for the time specified by the forward delay timer (default 15 seconds). Only then is the interface placed into forwarding state.

Because the transition from blocking to forwarding does require some extra steps, you should be ready to respond to conceptual questions about the transition.

## Troubleshooting Layer 2 EtherChannel

EtherChannels can prove particularly challenging to troubleshoot for a couple of reasons. First, you have to be careful to match the correct configuration, and there are many more incorrect configuration combinations than there are correct combinations. Second, many interface settings must match on the physical links, both on the local switch and on the neighboring switch, before a switch will add the physical link to the channel. This second major section in the chapter works through both sets of issues.

### Incorrect Options on the channel-group Command

The rules for the small set of working configuration options on the **channel-group** command can be summarized as follows, for a single EtherChannel:

1. On the local switch, all the **channel-group** commands for all the physical interfaces must use the same channel-group number.
2. The channel-group number can be different on the neighboring switches.
3. If using the **on** keyword, you must use it on the corresponding interfaces of both switches.
4. If you use the **desirable** keyword on one switch, the switch uses PAgP; the other switch must use either **desirable** or **auto**.
5. If you use the **active** keyword on one switch, the switch uses LACP; the other switch must use either **active** or **passive**.

These rules summarize the correct configuration options, but the options actually leave many more incorrect choices. The following list shows some incorrect configurations that the switches allow, even though they would result in the EtherChannel not working. The list

compares the configuration on one switch to another based on the physical interface configuration. Each lists the reasons why the configuration is incorrect.

- Configuring the **on** keyword on one switch, and **desirable**, **auto**, **active**, or **passive** on the other switch. The **on** keyword does not enable PAgP, and does not enable LACP, and the other options rely on PAgP or LACP.
- Configuring the **auto** keyword on both switches. Both use PAgP, but both wait on the other switch to begin negotiations.
- Configuring the **passive** keyword on both switches. Both use LACP, but both wait on the other switch to begin negotiations.
- Configuring the **active** keyword on one switch and either **desirable** or **auto** on the other switch. The **active** keyword uses LACP, whereas the other keywords use PAgP.
- Configuring the **desirable** keyword on one switch and either **active** or **passive** on the other switch. The **desirable** keyword uses PAgP, whereas the other keywords use LACP.

Example L-2 shows an example that matches the last item in the list. In this case, SW1's two ports (F0/14 and F0/15) have been configured with the **desirable** keyword, and SW2's matching F0/16 and F0/17 have been configured with the **active** keyword. The example lists some telling status information about the failure, with notes following the example.

**Example L-2** *Incorrect Configuration Using Mismatched PortChannel Protocols*

```
SW1# show etherchannel summary
Flags:  D - down        P - bundled in port-channel
        I - stand-alone s - suspended
        H - Hot-standby (LACP only)
        R - Layer3      S - Layer2
        U - in use      f - failed to allocate aggregator

        M - not in use, minimum links not met
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port


Number of channel-groups in use: 1
Number of aggregators:           1

Group  Port-channel  Protocol    Ports
------+-------------+-----------+-----------------------------------------------

1      Po1(SD)          PAgP      Fa0/14(I)    Fa0/15(I)

SW1# show interfaces status | include Po|14|15
Port      Name               Status       Vlan       Duplex  Speed Type
Fa0/14                       connected    301        a-full  a-100 10/100BaseTX
Fa0/15                       connected    301        a-full  a-100 10/100BaseTX
Po1                          notconnect   unassigned   auto   auto
```

Start at the top, in the legend of the **show etherchannel summary** command. The *D* code letter means that the channel itself is down, with *S* meaning that the channel is a Layer 2 EtherChannel. Code *I* means that the physical interface is working independently from the PortChannel (described as "stand-alone"). Then, the bottom of that command's output highlights PortChannel 1 (Po1) as Layer 2 EtherChannel in a down state (SD), with F0/14 and F0/15 as stand-alone interfaces (I).

Interestingly, because the problem is a configuration mistake, the two physical interfaces still operate independently, as if the PortChannel did not exist. The last command in the example shows that while the PortChannel 1 interface is down, the two physical interfaces are in a connected state.

**NOTE** As a suggestion for attacking EtherChannel problems on the exam, rather than memorizing all the incorrect configuration options, concentrate on the list of correct configuration options. Then look for any differences between a given question's configuration as compared to the known correct configurations and work from there.

## Analyzing the Switch Data Plane Forwarding

STP and EtherChannel both have an impact on what a switch's forwarding logic can use. STP limits which interfaces the data plane even considers using by placing some ports in a blocking state (STP) or discarding state (RSTP), which in turn tells the data plane to simply not use that port. EtherChannel gives the data plane new ports to use in the switch's MAC address table—EtherChannels—while telling the data plane to not use the underlying physical interfaces in an EtherChannel in the MAC table.

This (short) third major section of the chapter explores the impact of STP and EtherChannel on data plane logic and a switch's MAC address table.

### Predicting STP Impact on MAC Tables

Consider the small LAN shown in Figure L-4. The LAN has only three switches, with redundancy, just big enough to make the point for this next example. The LAN supports two VLANs, 1 and 2, and the engineer has configured STP such that SW3 blocks on a different port in each of the two VLANs. As a result, VLAN 1 traffic would flow from SW3 to SW1 next, and in VLAN 2, traffic would flow from SW3 to SW2 next instead.

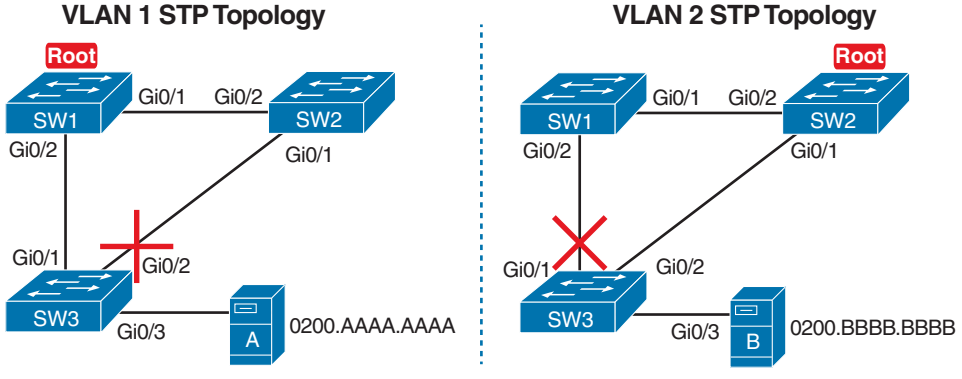



**Figure L-4** *Two Different STP Topologies for Same Physical LAN, Two Different VLANs*

Looking at diagrams like those in Figure L-4 makes the forwarding path obvious. Although the figure shows the traffic path, that path is determined by switch MAC learning, which is then impacted by the ports on which STP has set a blocking or discarding state.

For example, consider VLAN 1's STP topology in Figure L-4. Remember, STP blocks on a port on one switch, not on both ends of the link. So, in the case of VLAN 1, SW3's G0/2 port blocks, but SW2's G0/1 does not. Even so, by blocking on a port on one end of the link, that act effectively stops any MAC learning from happening by either device on the link. That is, SW3 learns no MAC addresses on its G0/2 port, and SW2 learns no MAC addresses on its G0/1 port, for these reasons:

- **SW2 learns no MAC addresses on G0/1:** On the blocking (SW3) end of the SW3–SW2 trunk, SW3 will not send frames out that link to SW2, so SW2 will never receive frames from which to learn MAC addresses on SW2's G0/1.
- **SW3 learns no MAC addresses on G0/2:** On the not blocking (SW2) end of the SW3–SW2 trunk, SW2 will flood frames out that port. SW3 receives those frames, but because SW3 blocks, SW3 ignores those received frames and does not learn their MAC addresses.

Given that discussion, can you predict the MAC table entries on each of the three switches for the MAC addresses of servers A and B in Figure L-4? On switch SW2, the entry for server A, in VLAN 1, should refer to SW2's G0/2 port, pointing to SW1 next, matching the figure. But SW2's entry for server B, in VLAN 2, references SW2's G0/1 port, again matching the figure. Example L-3 shows the MAC tables on SW1 and SW2 as a confirmation.

**Example L-3** *Examining SW1 and SW2 Dynamic MAC Address Table Entries*

```
SW1# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  1     0200.AAAA.AAAA    DYNAMIC     Gi0/2
  2     0200.BBBB.BBBB    DYNAMIC     Gi0/1
```

```
SW2# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  1     0200.AAAA.AAAA    DYNAMIC     Gi0/2
  2     0200.BBBB.BBBB    DYNAMIC     Gi0/1
```

## Predicting EtherChannel Impact on MAC Tables

Most designs use multiple links between switches, with those links configured to be part of an EtherChannel. What does that do to the MAC forwarding logic? In short, the switch uses the PortChannel interfaces, and not the physical interfaces bundled into the EtherChannel, in the MAC address table. Specifically:

**MAC learning:** Frames received in a physical interface that is part of a PortChannel are considered to arrive on the PortChannel interface. So, MAC learning adds the PortChannel

interface rather than the physical interface to the MAC address table.

**MAC forwarding:** The forwarding process will find a PortChannel port as an outgoing interface when matching the MAC address table. Then the switch must take the additional step to choose the outgoing physical interface, based on the load-balancing preferences configured for that PortChannel.

For example, consider Figure L-5, which updates previous Figure L-4 with two-link PortChannels between each pair of switches. With VLAN 1 blocking again on switch SW3, but this time on SW3's PortChannel3 interface, what MAC table entries would you expect to see in each switch? Similarly, what MAC table entries would you expect to see for VLAN 2, with SW3 blocking on its PortChannel2 interface?

The logic of which entries exist on which ports mirrors the logic with the earlier example surrounding Figure L-4. In this case, the interfaces just happen to be PortChannel interfaces. Example L-4 shows the same command from the same two switches as Example L-3: **show mac address-table dynamic** from both SW1 and SW2. (Note that to save length, the MAC table output shows only the entries for the two servers in Figure L-5.)

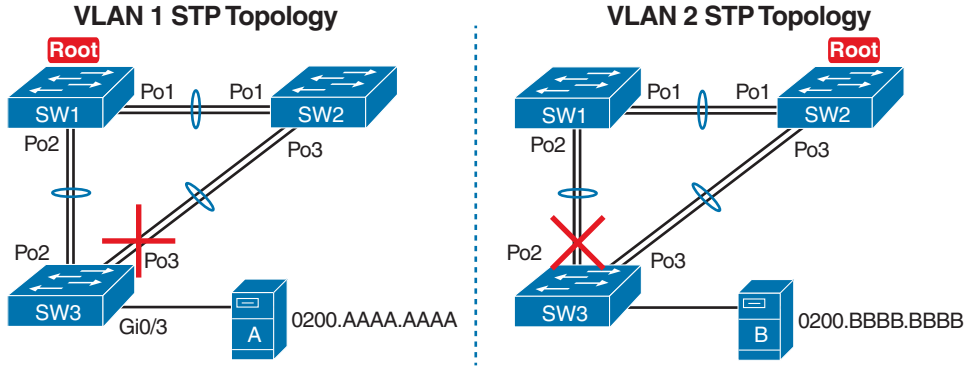



**Figure L-5** *VLAN Topology with PortChannels Between Switches*

**Example L-4** *SW1 and SW2 MAC Tables with PortChannel Ports Listed*

```
SW1# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  1     0200.AAAA.AAAA    DYNAMIC     Po2
  2     0200.BBBB.BBBB    DYNAMIC     Po1
```

```
SW2# show mac address-table dynamic
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
  1     0200.AAAA.AAAA    DYNAMIC     Po1
  2     0200.BBBB.BBBB    DYNAMIC     Po3
```

Switches use one of many load-balancing options to then choose the physical interface to use after matching MAC table entries like those shown in Example L-4. By default, Cisco Layer 2 switches often default to use a balancing method based on the source MAC address. In particular, the switch looks at the low-order bits of the source MAC address (which are on the far right of the MAC address in written form). This approach increases the chances that the balancing will be spread somewhat evenly based on the source MAC addresses in use.

## Choosing the VLAN of Incoming Frames

To wrap up the analysis of switch data plane forwarding, this section mostly reviews topics already discussed, but it serves to emphasize some important points. The topic is simply this: How does a switch know which VLAN a frame is a part of as the frame enters a switch? You have seen all the information needed to answer this question already, but take the time to review.

First, some interfaces trunk, and in those cases, the frame arrives with a VLAN ID listed in the incoming trunking header. In other cases, the frame does not arrive with a trunking header, and the switch must look at local configuration. But because the switch will match both the destination MAC address and the frame VLAN ID when matching the MAC address table, knowing how the switch determines the VLAN ID is important.

The following list reviews and summarizes the key points of how a switch determines the VLAN ID to associate with an incoming frame:

**Step 1.** If the port is an access port, associate the frame with the configured access VLAN (**switchport access vlan** *vlan_id*).

**Step 2.** If the port is a voice port, or has both an IP Phone and PC (or other data device) connected to the phone:

- **a.** Associate the frames from the data device with the configured access VLAN (as configured with the **switchport access vlan** *vlan_id* command).
- **b.** Associate the frames from the phone with the VLAN ID in the 802.1Q header (as configured with the **switchport voice vlan** *vlan_id* command).

**Step 3.** If the port is a trunk, determine the frame's tagged VLAN, or if there is no tag, use that incoming interface's native VLAN ID (**switchport trunk native** *vlan_id*).

*This page intentionally left blank*

# APPENDIX M

# Variable-Length Subnet Masks

**NOTE** This appendix contains content that was published as a chapter in one of the past editions of this book or a related book. The author includes this appendix with the current edition as extra reading for anyone interested in learning more. However, note that the content in this appendix has not been edited since it was published in the earlier edition, so references to exams and exam topics, and to other chapters, will be outdated. This appendix was previously published as Chapter 22 of the book *CCENT/CCNA ICND1 100-105 Official Cert Guide*, published in 2016.

IPv4 addressing and subnetting use a lot of terms, a lot of small math steps, and a lot of concepts that fit together. While learning those concepts, it helps to keep things as simple as possible. One way this book has kept the discussion simpler so far was to show examples that use one mask only inside a single Class A, B, or C network.

This chapter removes that restriction by introducing variable-length subnet masks (VLSM). VLSM simply means that the subnet design uses more than one mask in the same classful network. VLSM has some advantages and disadvantages, but when learning, the main challenge is that a subnetting design that uses VLSM requires more math, and it requires that you think about some other issues as well. This chapter walks you through the concepts, the issues, and the math.

# Foundation Topics

## VLSM Concepts and Configuration

VLSM occurs when an internetwork uses more than one mask for different subnets of a single Class A, B, or C network. Figure M-1 shows an example of VLSM used in Class A network 10.0.0.0.

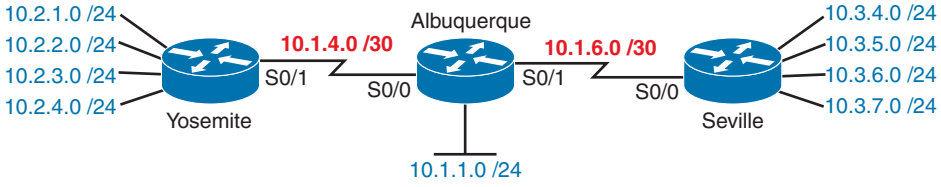



**Figure M-1** *VLSM in Network 10.0.0.0: Masks /24 and /30*

Figure M-1 shows a typical choice of using a /30 prefix (mask 255.255.255.252) on point-to-point serial links, with mask /24 (255.255.255.0) on the LAN subnets. All subnets are of Class A network 10.0.0.0, with two masks being used, therefore meeting the definition of VLSM.

Oddly enough, a common mistake occurs when people think that VLSM means "using more than one mask in some internetwork" rather than "using more than one mask *in a single classful network*." For example, if in one internetwork diagram, all subnets of network 10.0.0.0 use a 255.255.240.0 mask, and all subnets of network 11.0.0.0 use a 255.255.255.0 mask, the design uses two different masks. However, Class A network 10.0.0.0 uses only one mask, and Class A network 11.0.0.0 uses only one mask. In that case, the design does not use VLSM.

VLSM provides many benefits for real networks, mainly related to how you allocate and use your IP address space. Because a mask defines the size of the subnet (the number of host addresses in the subnet), VLSM allows engineers to better match the need for addresses with the size of the subnet. For example, for subnets that need fewer addresses, the engineer uses a mask with fewer host bits, so the subnet has fewer host IP addresses. This flexibility reduces the number of wasted IP addresses in each subnet. By wasting fewer addresses, more space remains to allocate more subnets.

VLSM can be helpful for both public and private IP addresses, but the benefits are more dramatic with public networks. With public networks, the address savings help engineers avoid having to obtain another registered IP network number from regional IP address assignment authorities. With private networks, as defined in RFC 1918, running out of addresses is not as big a negative, because you can always grab another private network from RFC 1918 if you run out.

### Classless and Classful Routing Protocols

Before you can deploy a VLSM design, you must first use a routing protocol that supports VLSM. To support VLSM, the routing protocol must advertise the mask along with each subnet. Without mask information, the router receiving the update would be confused.

For example, if a router learned a route for 10.1.8.0, but with no mask information, what does that mean? Is that subnet 10.1.8.0/24? 10.1.8.0/23? 10.1.8.0/30? The dotted-decimal number 10.1.8.0 happens to be a valid subnet number with a variety of masks, and because multiple masks can be used with VLSM, the router has no good way to make an educated guess. To effectively support VLSM, the routing protocol needs to advertise the correct mask along with each subnet so that the receiving router knows the exact subnet that is being advertised.

By definition, *classless routing protocols* advertise the mask with each advertised route, and *classful routing protocols* do not. The classless routing protocols, as noted in Table M-1, are the newer, more advanced routing protocols. Not only do these more advanced classless routing protocols support VLSM, but they also support manual route summarization, which allows a routing protocol to advertise one route for a larger subnet instead of multiple routes for smaller subnets.

**Table M-1** Classless and Classful Interior IP Routing Protocols

| Routing Protocol | Is It Classless? | Sends Mask in Updates? | Supports VLSM? | Supports Manual Route Summarization? |
|---|---|---|---|---|
| RIPv1 | No | No | No | No |
| RIPv2 | Yes | Yes | Yes | Yes |
| EIGRP | Yes | Yes | Yes | Yes |
| OSPF | Yes | Yes | Yes | Yes |

Beyond VLSM itself, the routing protocols do not have to be configured to support VLSM or to be classless. There is no command to enable or disable the fact that classless routing protocols include the mask with each route. The only configuration choice you must make is to use a classless routing protocol.

## VLSM Configuration and Verification

Cisco routers do not configure VLSM, enable or disable it, or need any configuration to use it. From a configuration perspective, VLSM is simply a side effect of using the **ip address** interface subcommand. Routers collectively configure VLSM by virtue of having IP addresses in the same classful network but with different masks.

For example, Example M-1 shows two of the interfaces from router Yosemite from Figure M-1. The example shows the IP address assignments on two interfaces, one with a /24 mask and one with a /30 mask, both with IP addresses in Class A network 10.0.0.0.

**Example M-1** *Configuring Two Interfaces on Yosemite, Resulting in VLSM*

```
Yosemite# configure terminal
Yosemite(config)# interface Fa0/0
Yosemite(config-if)# ip address 10.2.1.1 255.255.255.0
Yosemite(config-if)# interface S0/1
Yosemite(config-if)# ip address 10.1.4.1 255.255.255.252
```

The use of VLSM can also be detected by a detailed look at the output of the **show ip route** command. This command lists routes in groups, by classful network, so that you see all the subnets of a single Class A, B, or C network all in a row. Just look down the list, and look to see, if any, how many different masks are listed. For example, Example M-2 lists the routing

table on Albuquerque from Figure M-1; Albuquerque uses masks /24 and /30 inside network 10.0.0.0, as noted in the highlighted line in the example.

**Example M-2** *Albuquerque Routing Table with VLSM*

```
Albuquerque# show ip route
! Legend omitted for brevity

     10.0.0.0/8 is variably subnetted, 14 subnets, 3 masks
D       10.2.1.0/24 [90/2172416] via 10.1.4.1, 00:00:34, Serial0/0
D       10.2.2.0/24 [90/2172416] via 10.1.4.1, 00:00:34, Serial0/0
D       10.2.3.0/24 [90/2172416] via 10.1.4.1, 00:00:34, Serial0/0
D       10.2.4.0/24 [90/2172416] via 10.1.4.1, 00:00:34, Serial0/0
D       10.3.4.0/24 [90/2172416] via 10.1.6.2, 00:00:56, Serial0/1
D       10.3.5.0/24 [90/2172416] via 10.1.6.2, 00:00:56, Serial0/1
D       10.3.6.0/24 [90/2172416] via 10.1.6.2, 00:00:56, Serial0/1
D       10.3.7.0/24 [90/2172416] via 10.1.6.2, 00:00:56, Serial0/1
C       10.1.1.0/24 is directly connected, FastEthernet0/0
L       10.1.1.1/32 is directly connected, FastEthernet0/0
C       10.1.6.0/30 is directly connected, Serial0/1
L       10.1.6.1/32 is directly connected, Serial0/1
C       10.1.4.0/30 is directly connected, Serial0/0
L       10.1.4.1/32 is directly connected, Serial0/0
```

**NOTE** For the purposes of understanding whether a design uses VLSM, ignore the /32 "local" routes that a router automatically creates for its own interface IP addresses.

So ends the discussion of VLSM as an end to itself. This chapter is devoted to VLSM, but it took a mere three to four pages to fully describe it. Why the entire VLSM chapter? Well, to work with VLSM, to find problems with it, to add subnets to an existing design, and to design using VLSM from scratch—in other words, to apply VLSM to real networks—takes skill and practice. To do these same tasks on the exam requires skill and practice. The rest of this chapter examines the skills to apply VLSM and provides some practice for these two key areas:

- Finding VLSM overlaps
- Adding new VLSM subnets without overlaps

## Finding VLSM Overlaps

Regardless of whether a design uses VLSM, the subnets used in any IP internetwork design should not overlap their address ranges. When subnets in different locations overlap their addresses, a router's routing table entries overlap. As a result, hosts in different locations can be assigned the same IP address. Routers clearly cannot route packets correctly in these cases. In short, a design that uses overlapping subnets is considered to be an incorrect design and should not be used.

This section begins with a short discussion about VLSM design, to drive home the ideas behind VLSM overlaps. It then gets into an operational and troubleshooting approach to the topic, by looking at existing designs and trying to find any existing overlaps.

## Designing Subnetting Plans with VLSM

When creating a subnetting plan using VLSM, you have to be much more careful in choosing what subnets to use. First, whatever masks you use in a VLSM design, each subnet ID must be a valid subnet ID given the mask that you use for that subnet.

For example, consider a subnet plan for Class B network 172.16.0.0. To create a subnet with a /24 mask, the subnet ID must be a subnet ID that you could choose if you subnetted the whole Class B network with that same mask. Appendix L, "Subnet Design," discusses how to find those subnets in depth, but with a Class B network and a /24 mask, the possible subnet IDs should be easy to calculate by now: 172.16.0.0 (the zero subnet), then 172.16.1.0, 172.16.2.0, 172.16.3.0, 172.16.4.0, and so on, up through 172.16.255.0.

**NOTE** Subnet IDs must always follow this important binary rule as noted back in Chapter 14, "Analyzing Existing Subnets": In binary, each subnet ID has a host field of all binary 0s. If you use the math and processes to find all subnet IDs per Appendix L, all those subnet IDs happen to have binary 0s in the host fields.

Now expand your thinking about subnet IDs to a VLSM design. To begin, you would decide that you need some subnets with one mask, other subnets with another mask, and so on, to meet the requirements for different sizes of different subnets. For instance, imagine you start with a brand-new VLSM design, with Class B network 172.16.0.0. You plan to have some subnets with /22 masks, some with /23, and some with /24. You might develop then a planning diagram, or at least draw the ideas, with something like Figure M-2.

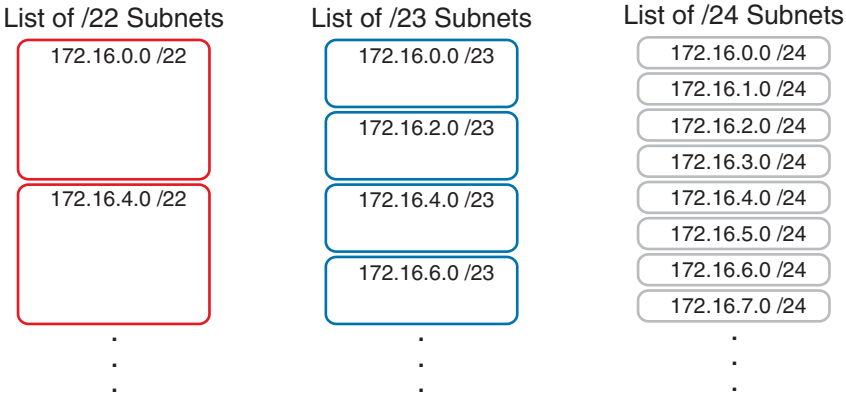



**Figure M-2** *Possible Subnet IDs of Network 172.16.0.0, with /22, /23, and /24 Masks*

The drawing shows the first few subnet IDs available with each mask, but you cannot use all subnets from all three lists in a design. As soon as you choose to use one subnet from any column, you remove some subnets from the other lists because subnets cannot overlap. Overlapping subnets are subnets whose range of addresses include some of the same addresses.

As an example, Figure M-3 shows the same list of the first few possible /22, /23, and /24 subnets of Class B network 172.16.0.0. However, it shows a check mark beside two subnets that have been allocated for use; that is, on paper, the person making the subnetting plan has decided to use these two subnets somewhere in the network. The subnets with a dark gray shading and an *X* in them can no longer be used because they have some overlapping addresses with the subnets that have check marks (172.16.3.0/24 and 172.16.4.0/22).

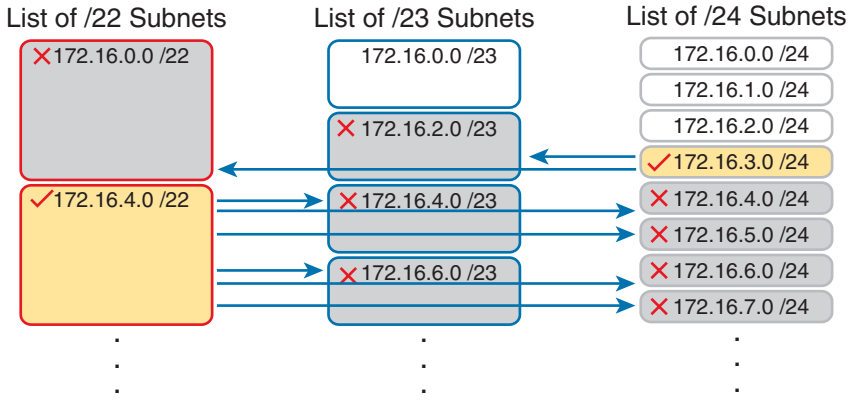



**Figure M-3** *Selecting Two Subnets Disallows Other Subnets in Different Columns*

Just to complete the example, first look at subnet 172.16.4.0 on the lower left. That subnet includes addresses from the subnet ID of 172.16.4.0 through the subnet broadcast address of 172.16.7.255. As you can see just by looking at the subnet IDs to the right, all the subnets referenced with the arrowed lines are within that same range of addresses.

Now look to the upper right of the figure, to subnet 172.16.3.0/24. The subnet has a range of 172.16.3.0–172.16.3.255 including the subnet ID and subnet broadcast address. That subnet overlaps with the two subnets referenced to the left. For instance, subnet 172.16.0.0/22 includes the range from 172.16.0.0–172.16.3.255. But because there is some overlap, once the design has allocated the 172.16.3.0/24 subnet, the 172.16.2.0/23 and 172.16.0.0/22 subnets could not be used without causing problems, because:

> A subnetting design, whether using VLSM or not, should not allow subnets whose address ranges overlap. If overlapping subnets are implemented, routing problems occur and some hosts simply cannot communicate outside their subnets.

These address overlaps are easier to see when not using VLSM. When not using VLSM, overlapped subnets have identical subnet IDs, so to find overlaps, you just have to look at the subnet IDs. With VLSM, overlapped subnets may not have the same subnet ID, as was the case in this most recent example with the subnets across the top of Figure M-3. To find these overlaps, you have to look at the entire range of addresses in each subnet, from subnet ID to subnet broadcast address, and compare the range to the other subnets in the design.

## An Example of Finding a VLSM Overlap

For example, imagine that a practice question for the CCENT exam shows Figure M-4. It uses a single Class B network (172.16.0.0), with VLSM, because it uses three different masks: /23, /24, and /30.

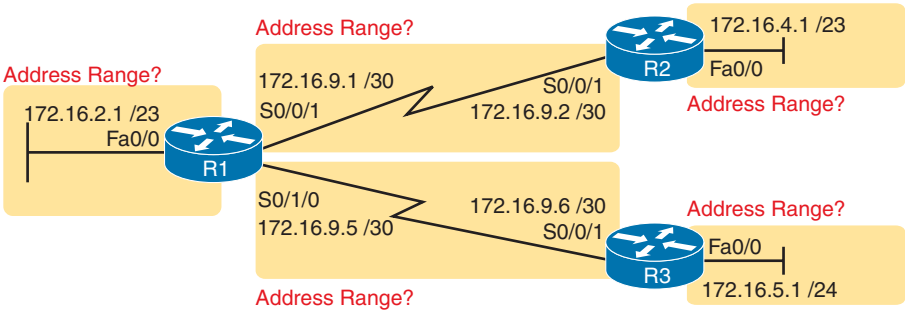



**Figure M-4** *VLSM Design with Possible Overlap*

Now imagine that the exam question shows you the figure, and either directly or indirectly asks whether overlapping subnets exist. This type of question might simply tell you that some hosts cannot ping each other, or it might not even mention that the root cause could be that some of the subnets overlap. To answer such a question, you could follow this simple but possibly laborious process:

**Step 1.** Calculate the subnet ID and subnet broadcast address of each subnet, which gives you the range of addresses in that subnet.

**Step 2.** List the subnet IDs in numerical order (along with their subnet broadcast addresses).

**Step 3.** Scan the list from top to bottom, comparing each pair of adjacent entries, to see whether their range of addresses overlaps.

For example, Table M-2 completes the first two steps based on Figure M-4, listing the subnet IDs and subnet broadcast addresses, in numerical order based on the subnet IDs.

**Table M-2** Subnet IDs and Broadcast Addresses, in Numerical Order, from Figure M-4

| Subnet | Subnet Number | Broadcast Address |
|---|---|---|
| R1 LAN | 172.16.2.0 | 172.16.3.255 |
| R2 LAN | 172.16.4.0 | 172.16.5.255 |
| R3 LAN | 172.16.5.0 | 172.16.5.255 |
| R1-R2 serial | 172.16.9.0 | 172.16.9.3 |
| R1-R3 serial | 172.16.9.4 | 172.16.9.7 |

The VLSM design is invalid in this case because of the overlap between R2's LAN subnet and R3's LAN subnet. As for the process, Step 3 states the somewhat obvious step of comparing the address ranges to see whether any overlaps occur. Note that, in this case, none of the subnet numbers are identical, but two entries (highlighted) do overlap. The design is invalid because of the overlap, and one of these two subnets would need to be changed.

As far as the three-step process works, note that if two adjacent entries in the list overlap, compare three entries at the next step. The two subnets already marked as overlapped can overlap with the next subnet in the list. For example, the three subnets in the following list overlap in that the first subnet overlaps with the second and third subnets in the list. If you

followed the process shown here, you would have first noticed the overlap between the first two subnets in the list, so you would then also need to check the next subnet in the list to find out if it overlapped.

10.1.0.0/16 (subnet ID 10.1.0.0, broadcast 10.1.255.255)

10.1.200.0/24 (subnet ID 10.1.200.0, broadcast 10.1.200.255)

10.1.250.0/24 (subnet ID 10.1.250.0, broadcast 10.1.250.255)

### Practice Finding VLSM Overlaps

As typical of anything to do with applying IP addressing and subnetting, practice helps. To that end, Table M-3 lists three practice problems. Just start with the five IP addresses listed in a single column, and then follow the three-step process outlined in the previous section to find any VLSM overlaps. The answers can be found near the end of this chapter, in the section "Answers to Earlier Practice Problems."

**Table M-3** VLSM Overlap Practice Problems

| Problem 1 | Problem 2 | Problem 3 |
|---|---|---|
| 10.1.34.9/22 | 172.16.126.151/22 | 192.168.1.253/30 |
| 10.1.29.101/23 | 172.16.122.57/27 | 192.168.1.113/28 |
| 10.1.23.254/22 | 172.16.122.33/30 | 192.168.1.245/29 |
| 10.1.17.1/21 | 172.16.122.1/30 | 192.168.1.125/30 |
| 10.1.1.1/20 | 172.16.128.151/20 | 192.168.1.122/30 |

## Adding a New Subnet to an Existing VLSM Design

The task described in this section happens frequently in real networks: choosing new subnets to add to an existing design. In real life, you can use IP Address Management (IPAM) tools that help you choose a new subnet so that you do not cause an overlap. However, for the CCNA exam, you need to be ready to do the mental process and math of choosing a subnet that does not create an overlapped VLSM subnet condition. In other words, you need to pick a new subnet and not make a mistake!

For example, consider the internetwork shown earlier in Figure M-2, with classful network 172.16.0.0. An exam question might suggest that a new subnet, with a /23 prefix length, needs to be added to the design. The question might also say, "Pick the numerically lowest subnet number that can be used for the new subnet." In other words, if both 172.16.4.0 and 172.16.6.0 would work, use 172.16.4.0.

So, you really have a couple of tasks: To find all the subnet IDs that could be used, rule out the ones that would cause an overlap, and then check to see whether the question guides you to pick either the numerically lowest (or highest) subnet ID. This list outlines the specific steps:

**Step 1.** Pick the subnet mask (prefix length) for the new subnet, based on the design requirements (if not already listed as part of the question).

**Step 2.** Calculate all possible subnet numbers of the classful network using the mask from Step 1, along with the subnet broadcast addresses.

**Step 3.** Make a list of existing subnet IDs and matching subnet broadcast addresses.

**Step 4.** Compare the existing subnets to the candidate new subnets to rule out overlapping new subnets.

**Step 5.** Choose the new subnet ID from the remaining subnets identified at Step 4, paying attention to whether the question asks for the numerically lowest or numerically highest subnet ID.

M

## An Example of Adding a New VLSM Subnet

For example, Figure M-5 shows an existing internetwork that uses VLSM. (The figure uses the same IP addresses as shown in Figure M-4, but with R3's LAN IP address changed to fix the VLSM overlap shown in Figure M-4.) In this case, you need to add a new subnet to support 300 hosts. Imagine that the question tells you to use the smallest subnet (least number of hosts) to meet that requirement. You use some math and logic you learned earlier in your study to choose mask /23, which gives you 9 host bits, for $2^9 - 2 = 510$ hosts in the subnet.

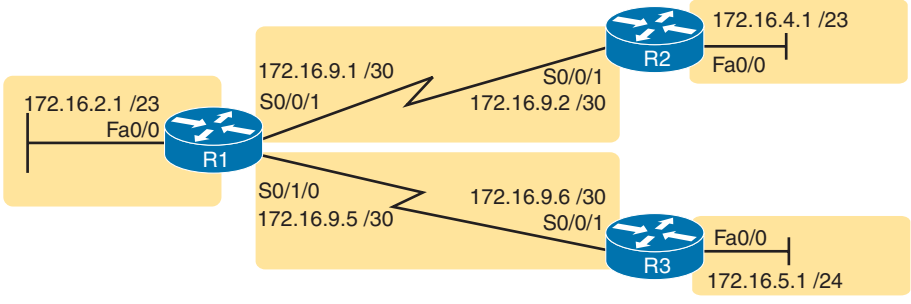



**Figure M-5** *Internetwork to Which You Need to Add a /23 Subnet, Network 172.16.0.0*

At this point, just follow the steps listed before Figure M-5. For Step 1, you have already been given the mask (/23). For Step 2, you need to list all the subnet numbers and broadcast addresses of 172.16.0.0, assuming the /23 mask. You will not use all these subnets, but you need the list for comparison to the existing subnets. Table M-4 shows the results, at least for the first five possible /23 subnets.

**Table M-4** First Five Possible /23 Subnets

| Subnet | Subnet Number | Subnet Broadcast Address |
|---|---|---|
| First (zero) | 172.16.0.0 | 172.16.1.255 |
| Second | 172.16.2.0 | 172.16.3.255 |
| Third | 172.16.4.0 | 172.16.5.255 |
| Fourth | 172.16.6.0 | 172.16.7.255 |
| Fifth | 172.16.8.0 | 172.16.9.255 |

Next, at Step 3, list the existing subnet numbers and broadcast addresses, as shown earlier in Figure M-5. To do so, do the usual math to take an IP address/mask to then find the subnet ID and subnet broadcast address. Table M-5 summarizes that information, including the locations, subnet numbers, and subnet broadcast addresses.

**Table M-5** Existing Subnet IDs and Broadcast Addresses from Figure M-5

| Subnet | Subnet Number | Subnet Broadcast Address |
|---|---|---|
| R1 LAN | 172.16.2.0 | 172.16.3.255 |
| R2 LAN | 172.16.4.0 | 172.16.5.255 |
| R3 LAN | 172.16.6.0 | 172.16.6.255 |
| R1-R2 serial | 172.16.9.0 | 172.16.9.3 |
| R1-R3 serial | 172.16.9.4 | 172.16.9.7 |

At this point, you have all the information you need to look for the overlap at Step 4. Simply compare the range of numbers for the subnets in the previous two tables. Which of the possible new /23 subnets (Table M-4) overlap with the existing subnets (Table M-5)? In this case, the second through fifth subnets in Table M-4 overlap, so rule those out as candidates to be used. (Table M-4 denotes those subnets with gray highlights.)

Step 5 has more to do with the exam than with real network design, but it is still worth listing as a separate step. Multiple-choice questions sometimes need to force you into a single answer, and asking for the numerically lowest or highest subnet does that. This particular example asks for the numerically lowest subnet number, which in this case is 172.16.0.0/23.

**NOTE** The answer, 172.16.0.0/23, happens to be a zero subnet. For the exam, the zero subnet should be avoided if (a) the question implies the use of classful routing protocols or (b) the routers are configured with the **no ip subnet-zero** global configuration command. Otherwise, assume that the zero subnet can be used.

# Answers to Earlier Practice Problems

## Answers to Practice Finding VLSM Overlaps

This section lists the answers to the three practice problems in the section "Practice Finding VLSM Overlaps," as listed earlier in Table M-3. Note that the tables that list details of the answer reordered the subnets as part of the process.

In Problem 1, the second and third subnet IDs listed in Table M-6 happen to overlap. The second subnet's range completely includes the range of addresses in the third subnet.

**Table M-6** VLSM Overlap Problem 1 Answers (Overlaps Highlighted)

| Reference | Original Address and Mask | Subnet ID | Broadcast Address |
|---|---|---|---|
| 1 | 10.1.1.1/20 | 10.1.0.0 | 10.1.15.255 |
| 2 | 10.1.17.1/21 | 10.1.16.0 | 10.1.23.255 |
| 3 | 10.1.23.254/22 | 10.1.20.0 | 10.1.23.255 |
| 4 | 10.1.29.101/23 | 10.1.28.0 | 10.1.29.255 |
| 5 | 10.1.34.9/22 | 10.1.32.0 | 10.1.35.255 |

In Problem 2, again the second and third subnet IDs (listed in Table M-7) happen to overlap, and again, the second subnet's range completely includes the range of addresses in the third subnet. Also, the second and third subnet IDs are the same value, so the overlap is more obvious.

**Table M-7** VLSM Overlap Problem 2 Answers (Overlaps Highlighted)

| Reference | Original Address and Mask | Subnet ID | Broadcast Address |
|---|---|---|---|
| 1 | 172.16.122.1/30 | 172.16.122.0 | 172.16.122.3 |
| 2 | 172.16.122.57/27 | 172.16.122.32 | 172.16.122.63 |
| 3 | 172.16.122.33/30 | 172.16.122.32 | 172.16.122.35 |
| 4 | 172.16.126.151/22 | 172.16.124.0 | 172.16.127.255 |
| 5 | 172.16.128.151/20 | 172.16.128.0 | 172.16.143.255 |

In Problem 3, three subnets overlap. Subnet 1's range completely includes the range of addresses in the second and third subnets, as shown in Table M-8. Note that the second and third subnets do not overlap with each other, so for the process in this book to find all the overlaps, after you find that the first two subnets overlap, you should compare the next entry in the table (3) with both of the two known-to-overlap entries (1 and 2).

**Table M-8** VLSM Overlap Problem 3 Answers (Overlaps Highlighted)

| Reference | Original Address and Mask | Subnet ID | Broadcast Address |
|---|---|---|---|
| 1 | 192.168.1.113/28 | 192.168.1.112 | 192.168.1.127 |
| 2 | 192.168.1.122/30 | 192.168.1.120 | 192.168.1.123 |
| 3 | 192.168.1.125/30 | 192.168.1.124 | 192.168.1.127 |
| 4 | 192.168.1.245/29 | 192.168.1.240 | 192.168.1.247 |
| 5 | 192.168.1.253/30 | 192.168.1.252 | 192.168.1.255 |

*This page intentionally left blank*

Pearson

# Where are the companion content files?



Thank you for purchasing this Premium Edition version of CCNA 200-301 Official Cert Guide, Volume 2, 2nd Edition

This product comes with companion content. You have access to these files by following the steps below:

1. Go to **ciscopress.com/account** and log in.
2. Click on the "Access Bonus Content" link in the Registered Products section of your account page for this product, to be taken to the page where your downloadable content is available.

Please note that many of our companion content files can be very large, especially image and video files.

If you are unable to locate the files for this title by following the steps at left, please visit **ciscopress.com/support** and select the chat, phone, or web ticket options to get help from a tech support representative.

---

The Professional and Personal Technology Brands of Pearson

Addison Wesley · Cisco Press · informIT · PEARSON IT Certification · que · SAMS

# CCNA 200-301, Volume 1
## Official Cert Guide

**2nd Edition**

In addition to the wealth of updated content, this new edition includes a series of free hands-on exercises to help you master several real-world configuration and troubleshooting activities. These exercises can be performed on the CCNA 200-301 Network Simulator Lite, Volume 1 software included for free on the companion website that accompanies this book. This software, which simulates the experience of working on actual Cisco routers and switches, contains the following 21 free lab exercises, covering topics in Part II and Part III, the first hands-on configuration sections of the book:

1. Configuring Local Usernames
2. Configuring Hostnames
3. Interface Status I
4. Interface Status II
5. Interface Status III
6. Interface Status IV
7. Configuring Switch IP Settings
8. Switch IP Address
9. Switch IP Connectivity I
10. Switch CLI Configuration Process I
11. Switch CLI Configuration Process II
12. Switch CLI Exec Mode
13. Setting Switch Passwords
14. Interface Settings I
15. Interface Settings II
16. Interface Settings III
17. Switch Forwarding I
18. Switch Security I
19. Switch Interfaces and Forwarding Configuration Scenario
20. Configuring VLANs Configuration Scenario
21. VLAN Troubleshooting

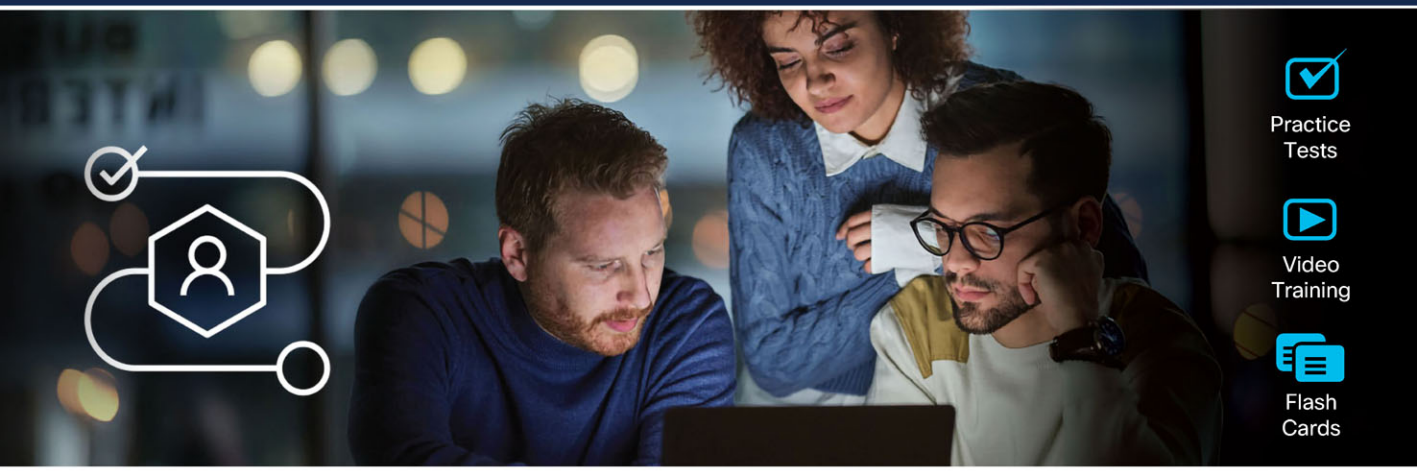
If you are interested in exploring more hands-on labs and practice configuration and troubleshooting with more router and switch commands, go to **www.pearsonitcertification.com/networksimulator** for demos and to review the latest products for sale.

### Network Simulator System Requirements

Windows (Minimum)
- Windows 11, Windows 10, Windows 8.1
- Intel Pentium III (1GHz or faster processor) or 2 GB RAM
- 16 GB available hard disk space (32-bit) or 20 GB (64-bit)
- DirectX 9 graphics device with WDDM 1.0 or higher driver
- Adobe Acrobat Reader version 8 and above
- Connection to the Internet during installation for access code validation

Mac (Minimum)
- macOS 14, 13, and 12
- Intel Pentium III (1GHz or faster processor)
- 512 MB RAM (1 GB recommended)
- 1.5 GB hard disk space
- 32-bit color depth at 1024x768 resolution
- Adobe Acrobat Reader version 8 and above
- Connection to the Internet during installation for access code validation

Official Cert Guide




Study Planner
Review Exercises
Labs

# CCNA

## 200-301, Volume 2

2nd Edition

**Wendell Odom**, CCIE® No. 1624
**Jason Gooley**, CCIEx2 (RS, SP) No. 38759
**David Hucaby**, CCIE® No. 4594

ciscopress.com

# *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition

## Companion Website and Pearson Test Prep Access Code

Access interactive study tools on this book's companion website, including practice test software, review exercises, Key Term flash card application, a study planner, and more!

To access the companion website, simply follow these steps:

1. Go to www.ciscopress.com/register.
2. Enter the **print book ISBN:** 9780138214951.
3. Answer the security question to validate your purchase.
4. Go to your account page.
5. Click on the **Registered Products** tab.
6. Under the book listing, click on the **Access Bonus Content** link.

When you register your book, your Pearson Test Prep practice test access code will automatically be populated with the book listing under the Registered Products tab. You will need this code to access the practice test that comes with this book. You can redeem the code at **PearsonTestPrep.com**. Simply choose Pearson IT Certification as your product group and log into the site with the same credentials you used to register your book. Click the **Activate New Product** button and enter the access code. More detailed instructions on how to redeem your access code for both the online and desktop versions can be found on the companion website.

If you have any issues accessing the companion website or obtaining your Pearson Test Prep practice test access code, you can contact our support team by going to **ciscopress.com/support**.

*This page intentionally left blank*

# CCNA
# 200-301

## **Official** Cert Guide
## Volume 2

## Second Edition

**WENDELL ODOM,** CCIE No. 1624
**JASON GOOLEY,** CCIEx2 (RS, SP) No. 38759
**DAVID HUCABY,** CCIE No. 4594

**Cisco Press**

# CCNA 200-301 Official Cert Guide, Volume 2, Second Edition

Wendell Odom
Jason Gooley
David Hucaby

Copyright© 2025 Pearson Education, Inc.

Published by:
Cisco Press
Hoboken, New Jersey

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system, without written permission from the publisher, except for the inclusion of brief quotations in a review.

$PrintCode

Library of Congress Control Number: 2024934307

ISBN-13: 978-0-13-821495-1

ISBN-10: 0-13-821495-6

## Warning and Disclaimer

This book is designed to provide information about the Cisco CCNA 200-301 exam. Every effort has been made to make this book as complete and as accurate as possible, but no warranty or fitness is implied.

The information is provided on an “as is” basis. The author, Cisco Press, and Cisco Systems, Inc. shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained in this book or from the use of the discs or programs that may accompany it.

The opinions expressed in this book belong to the author and are not necessarily those of Cisco Systems, Inc.

## Trademark Acknowledgments

All terms mentioned in this book that are known to be trademarks or service marks have been appropriately capitalized. Cisco Press or Cisco Systems, Inc., cannot attest to the accuracy of this information. Use of a term in this book should not be regarded as affecting the validity of any trademark or service mark.

## Special Sales

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

## Feedback Information

At Cisco Press, our goal is to create in-depth technical books of the highest quality and value. Each book is crafted with care and precision, undergoing rigorous development that involves the unique expertise of members from the professional technical community.

Readers' feedback is a natural continuation of this process. If you have any comments regarding how we could improve the quality of this book, or otherwise alter it to better suit your needs, you can contact us through email at feedback@ciscopress.com. Please make sure to include the book title and ISBN in your message.

We greatly appreciate your assistance.

Please contact us with concerns about any potential bias at https://www.pearson.com/report-bias.html.

**GM K12, Early Career and Professional Learning:** Soo Kang

**Alliances Manager, Cisco Press:** Caroline Antonio

**Director, ITP Product Management:** Brett Bartow

**Managing Editor:** Sandra Schroeder

**Development Editor:** Christopher Cleveland

**Senior Project Editor:** Tonya Simpson

**Copy Editor:** Chuck Hutchinson

**Technical Editor:** Denise Donohue

**Editorial Assistant:** Cindy Teeters

**Cover Designer:** Chuti Prasertsith

**Composition:** codeMantra

**Indexer:** Timothy Wright

**Proofreader:** Donna E. Mulder

Americas Headquarters
Cisco Systems, Inc.
San Jose, CA

Asia Pacific Headquarters
Cisco Systems (USA) Pte. Ltd.
Singapore

Europe Headquarters
Cisco Systems International BV
Amsterdam, The Netherlands

Cisco has more than 200 offices worldwide. Addresses, phone numbers, and fax numbers are listed on the Cisco Website at **www.cisco.com/go/offices.**

CCDE, CCENT, Cisco Eos, Cisco HealthPresence, the Cisco logo, Cisco Lumin, Cisco Nexus, Cisco StadiumVision, Cisco TelePresence, Cisco WebEx, DCE, and Welcome to the Human Network are trademarks; Changing the Way We Work, Live, Play, and Learn and Cisco Store are service marks; and Access Registrar, Aironet, AsyncOS, Bringing the Meeting To You, Catalyst, CCDA, CCDP, CCIE, CCIP, CCNA, CCNP, CCSP, CCVP, Cisco, the Cisco Certified Internetwork Expert logo, Cisco IOS, Cisco Press, Cisco Systems, Cisco Systems Capital, the Cisco Systems logo, Cisco Unity, Collaboration Without Limitation, EtherFast, EtherSwitch, Event Center, Fast Step, Follow Me Browsing, FormShare, GigaDrive, HomeLink, Internet Quotient, IOS, iPhone, iQuick Study, IronPort, the IronPort logo, LightStream, Linksys, MediaTone, MeetingPlace, MeetingPlace Chime Sound, MGX, Networkers, Networking Academy, Network Registrar, PCNow, PIX, PowerPanels, ProConnect, ScriptShare, SenderBase, SMARTnet, Spectrum Expert, StackWise, The Fastest Way to Increase Your Internet Quotient, TransPath, WebEx, and the WebEx logo are registered trademarks of Cisco Systems, Inc. and/or its affiliates in the United States and certain other countries.

All other trademarks mentioned in this document or website are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company. (0812R)

## About the Authors

**Wendell Odom**, CCIE Enterprise No. 1624, was the first Cisco Press author for Cisco certification guides. He wrote all prior editions of this book, along with books on topics ranging from introductory networking to CCENT, CCNA R&S, CCNA DC, CCNP ROUTE, CCNP QoS, and CCIE R&S. In his four decades as a networker, he has worked as a network engineer, consultant, systems engineer, instructor, and course developer. He now spends his time focused on updating the CCNA books, his blog (www.certskills.com), building his new CCNA YouTube channel (www.youtube.com/@NetworkUpskill), and teaching online (www.certskills.com/courses). You can find him at www.LinkedIn.com/in/WendellOdom, Twitter (@WendellOdom), and at his blog, which provides a variety of free CCNA learning resources.

**Jason Gooley**, CCIEx2 (RS, SP) No. 38759, is a very enthusiastic and engaging speaker who focuses on teaching others. Jason has more than 30 years of experience in the industry and currently works as the technical evangelist for the Worldwide Enterprise Networking and Software Sales team at Cisco. Jason is very passionate about helping others in the industry succeed. In addition to being a public speaker, Jason has authored numerous Cisco Press books, is a CiscoLive Distinguished Speaker, and is a developer of CCIE exams, training, and blogs for Learning@Cisco. Jason is also a co-founder and member of the Program Committee Board for the Chicago Network Operators Group (CHI-NOG). Jason is the founder and host of @MetalDevOps, which is a YouTube video show about the intersection of metal music and technology. Jason has earned the nickname of "The Godfather of Programmability" from his students and peers and continues to help drive the industry forward around topics such as network programmability and automation.

**David Hucaby**, CCIE No. 4594, CWNE No. 292, is a technical education content engineer for Cisco Meraki. Previously, he worked as a wireless escalation engineer in a large healthcare environment for more than 20 years. David holds bachelor's and master's degrees in electrical engineering. He has been authoring Cisco Press titles for 25 years. David lives in Kentucky.

## About the Technical Reviewer

**Denise Donohue**, CCIE No. 9566 (Routing and Switching), has worked with information systems since the mid-1990s and network architecture since 2004. During that time, she has worked with a wide range of networks, private and public, of all sizes, across most industries. Her focus is on aligning business and technology. Denise has authored several Cisco Press books and frequently shares her knowledge in webinars and seminars, and at conferences.

## Dedications

*Wendell Odom:*

For Raymond Lanier Odom, still the best dad ever.

*Jason Gooley:*

To my family: Mother, thank you for always being my guiding light from up above. To my wife, Jamie, your love and support mean the world to me. To my children, Kaleigh and Jaxon, always believe in yourself and go for your dreams, no matter how lofty they may seem. To my father, thanks for believing in that stubborn kid so many years ago! To my brother, thanks for always having my back broham. Thank you, God—without you none of this would be possible!

# Acknowledgments

**Wendell Odom:**

Brett Bartow and I have been a team for a few decades. He has had more to do with the successes of the Cisco Press product line than anyone else. More than ever, his insights and wisdom have been a key to navigating Cisco's big changes to certifications back in 2020. With Cisco's 2023 pivot to a lean development model for certifications, with the possibility of new exam content annually, Brett's leadership matters more than ever. (See "Your Study Plan" for more about what that new lean development cycle means.) He's always a great partner in working through big-picture direction as well as features to make the books the best they can be for our readers. It is always appreciated, but not voiced every time—so thanks, Brett, for your consistent leadership and wisdom!

Chris Cleveland did the development editing for the very first Cisco Press exam certification guide way back in 1998, and he still can't seem to get away from us! Seriously, when Brett and I first discuss any new book, my first priority is to ask whether Chris has time to develop the book—and lobby if there are any barriers! It's always a pleasure working with you, Chris.

The technical editors also have a meaningful positive impact on the books. And we got Denise to do it! Denise and I teamed up to write the *CCIE R&S Official Cert Guide* for two editions, and she has written extensively herself—which is why I wondered if we could get her help. Her deep technical skills to go along with her unique insights into the book authoring process have been a great help to both weed out the mistakes and get good advice on how to improve the chapters.

Cisco's move to an annual exam update cadence (they at least consider updating each exam once per year) has more impact on the production side of our publishing process than it does on the authoring side. Knowing early that both Sandra and Tonya are back at it, finding ways to continue the high quality while being creative with the new publication cycle, sets me more at ease. When writing, I could rest knowing that the second half of the process, which happens after I've finished 99 percent of my work, will be done well!

Thanks to all the production team for making the magic happen. I usually do not interact with you directly beyond Sandra and Tonya, but I see your work, and the books truly improve through the process! From fixing all my grammar and passive-voice sentences to pulling the design and layout together, they do it all; thanks for putting it all together and making it look easy.

A special thank you to you readers who write in with suggestions and possible errors, and especially those of you who post online at the Cisco Learning Network and at my blog (www.certskills.com). More so than any edition I can remember, reader comments have had more to do with changes I made to improve existing content in these editions. The comments I received directly and those I overheard by participating at CLN made this edition a better book. (See the heading "Feedback Information" just a page or so back to see how to get in touch with us!)

My wonderful wife Kris and I reached our 25th anniversary while working on the early draft of this edition. She makes this challenging work lifestyle a breeze—even happily scheduling our 25th anniversary vacation around the book schedule! Thanks to my daughter Hannah for perspectives on how 20-somethings think about learning and studying. And thanks to Jesus Christ, Lord of everything in my life.

**Jason Gooley:**

I would like to thank Wendell for trusting me and having me on this amazing journey. Looking forward to our future work together!

Thank you to everyone at Pearson and Cisco Press for always making sure our products are of the best quality!

Thank you to my wife, Jamie, and my children, Kaleigh and Jaxon, for putting up with your father's crazy projects!!!

# Contents at a Glance

Introduction xxxi

**Part I Wireless LANs 3**

Chapter 1 Fundamentals of Wireless Networks 4

Chapter 2 Analyzing Cisco Wireless Architectures 22

Chapter 3 Securing Wireless Networks 40

Chapter 4 Building a Wireless LAN 56

Part I Review 88

**Part II IP Access Control Lists 91**

Chapter 5 Introduction to TCP/IP Transport and Applications 92

Chapter 6 Basic IPv4 Access Control Lists 114

Chapter 7 Named and Extended IP ACLs 136

Chapter 8 Applied IP ACLs 160

Part II Review 180

**Part III Security Services 183**

Chapter 9 Security Architectures 184

Chapter 10 Securing Network Devices 202

Chapter 11 Implementing Switch Port Security 222

Chapter 12 DHCP Snooping and ARP Inspection 238

Part III Review 264

**Part IV IP Services 267**

Chapter 13 Device Management Protocols 268

Chapter 14 Network Address Translation 298

Chapter 15 Quality of Service (QoS) 322

Chapter 16 First Hop Redundancy Protocols 350

Chapter 17 SNMP, FTP, and TFTP 368

Part IV Review 392

**Part V Network Architecture 395**

Chapter 18 LAN Architecture 396

Chapter 19 WAN Architecture 414

Chapter 20 Cloud Architecture 438

Part V Review 466

**Part VI Network Automation 469**

Chapter 21 Introduction to Controller-Based Networking 470

Chapter 22 Cisco Software-Defined Access (Cisco SD-Access) 494

Chapter 23 Understanding REST and JSON 526

Chapter 24 Understanding Ansible and Terraform 552

Part VI Review 568

**Part VII Exam Updates and Final Review 571**

Chapter 25 *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, Exam Updates 572

Chapter 26 Final Review 578

**Part VIII Print Appendixes 599**

Appendix A Numeric Reference Tables 601

Appendix B Exam Topics Cross-Reference 607

Appendix C Answers to the "Do I Know This Already?" Quizzes 619

Glossary 641

Index 668

**Online Appendixes**

Appendix D Topics from Previous Editions

Appendix E Practice for Chapter 6: Basic IPv4 Access Control Lists

Appendix F Study Planner

Glossary

## Reader Services

To access additional content for this book, simply register your product. To start the registration process, go to www.ciscopress.com/register and log in or create an account.* Enter the product ISBN 9780138214951 and click **Submit**. After the process is complete, you will find any available bonus content under Registered Products.

*Be sure to check the box that you would like to hear from us to receive exclusive discounts on future editions of this product.

# Contents

**Introduction xxxi**

**Part I Wireless LANs 3**

**Chapter 1 Fundamentals of Wireless Networks 4**

"Do I Know This Already?" Quiz 4

Foundation Topics 6

Comparing Wired and Wireless Networks 6

Wireless LAN Topologies 7

Basic Service Set 8

Distribution System 10

Extended Service Set 12

Independent Basic Service Set 13

Other Wireless Topologies 14

Repeater 14

Workgroup Bridge 15

Outdoor Bridge 16

Mesh Network 17

Wireless Bands and Channels 17

Chapter Review 20

**Chapter 2 Analyzing Cisco Wireless Architectures 22**

"Do I Know This Already?" Quiz 22

Foundation Topics 24

Autonomous AP Architecture 24

Cloud-based AP Architecture 26

Split-MAC Architectures 28

Comparing Cisco Wireless LAN Controller Deployments 32

Cisco AP Modes 35

FlexConnect Mode 36

Chapter Review 37

**Chapter 3 Securing Wireless Networks 40**

"Do I Know This Already?" Quiz 40

Foundation Topics 42

Anatomy of a Secure Connection 42

Authentication 43

Message Privacy 44

Message Integrity 45

Wireless Client Authentication Methods 46
Open Authentication 46
WEP 47
802.1x/EAP 47
*LEAP 48*
*EAP-FAST 49*
*PEAP 49*
*EAP-TLS 50*
Wireless Privacy and Integrity Methods 50
TKIP 50
CCMP 51
GCMP 51
WPA, WPA2, and WPA3 51
Chapter Review 53

**Chapter 4 Building a Wireless LAN 56**
"Do I Know This Already?" Quiz 56
Foundation Topics 58
Connecting a Cisco AP 58
Accessing a Cisco WLC 59
Connecting a Cisco WLC 63
WLC Physical Ports 63
Configuring a WLAN 65
Configuring a WLAN on an IOS-XE WLC 67
*Step 1: Configure a WLAN Profile 69*
*Step 2: Configure a Policy Profile 74*
*Step 3: Map the WLAN and Policy Profiles to a Policy Tag 77*
*Step 4: Apply the Policy Tag to Some APs 78*
Configuring a WLAN on an AireOS WLC 79
*Step 1: Create a Dynamic Interface 79*
*Step 2: Create a New WLAN 80*
*Step 3: Configure the WLAN 81*
*Configuring WLAN Security 83*
*Configuring WLAN QoS 85*
*Configuring Advanced WLAN Settings 85*
*Finalizing WLAN Configuration 86*
Chapter Review 87

**Part I Review 88**

**Part II IP Access Control Lists 91**

**Chapter 5 Introduction to TCP/IP Transport and Applications 92**

"Do I Know This Already?" Quiz 92

Foundation Topics 94

TCP/IP Layer 4 Protocols: TCP and UDP 94

Transmission Control Protocol 95

*Multiplexing Using TCP Port Numbers 95*

*Popular TCP/IP Applications 98*

*Connection Establishment and Termination 100*

*Error Recovery and Reliability 101*

*Flow Control Using Windowing 102*

User Datagram Protocol 103

TCP/IP Applications 104

Uniform Resource Identifiers 104

Finding the Web Server Using DNS 105

Transferring Files with HTTP 108

How the Receiving Host Identifies the Correct Receiving Application 109

HTTP Versions 110

*HTTP 1.0 and 1.1 110*

*HTTP/2 and TLS 110*

*HTTP 3.0 111*

Chapter Review 112

**Chapter 6 Basic IPv4 Access Control Lists 114**

"Do I Know This Already?" Quiz 114

Foundation Topics 116

IPv4 Access Control List Basics 116

ACL Location and Direction 116

Matching Packets 117

Taking Action When a Match Occurs 118

Types of IP ACLs 118

Standard Numbered IPv4 ACLs 119

List Logic with IP ACLs 119

Matching Logic and Command Syntax 121

*Matching the Exact IP Address 121*

*Matching a Subset of the Address with Wildcard Masks 122*

*Binary Wildcard Masks 123*

*Finding the Right Wildcard Mask to Match a Subnet 124*
*Matching Any/All Addresses 124*
Implementing Standard IP ACLs 125
*Standard Numbered ACL Scenario 1 125*
*Standard Numbered ACL Scenario 2 127*
Troubleshooting and Verification Tips 129
Practice Applying Standard IP ACLs 130
*Practice Building access-list Commands 130*
*Reverse Engineering from ACL to Address Range 131*
Chapter Review 133

**Chapter 7 Named and Extended IP ACLs 136**
"Do I Know This Already?" Quiz 136
Foundation Topics 138
Named ACLs and ACL Editing 138
Named IP Access Lists 138
Editing ACLs 140
*Editing Named ACLs 140*
*Editing Numbered ACLs 143*
Extended IP Access Control Lists 144
Matching the Protocol, Source IP, and Destination IP 145
Matching TCP and UDP Port Numbers 147
Extended IP ACL Configuration 150
*Extended IP ACL Example 1: Packets to Web Servers 151*
*Extended IP ACL Example 2: Packets from Web Servers 153*
Adjusting ACLs for HTTP/3 154
Practice Building access-list Commands 155
ACL Implementation Considerations 156
Chapter Review 157

**Chapter 8 Applied IP ACLs 160**
"Do I Know This Already?" Quiz 160
Foundation Topics 162
ACLs and Network Infrastructure Protocols 162
Filtering DNS 163
Filtering ICMP 164
Filtering OSPF 165
Filtering DHCP 167
Filtering SSH and Telnet 169

*Filtering for End User SSH/Telnet 169*
*Filtering for Router VTY Access 171*
Comparing ACLs in IOS and IOS XE 173
Configuration Syntax and Show Commands 173
Resequencing ACL Sequence Numbers 174
Using a Second (Common) Interface ACL 175
Matching Multiple Nonconsecutive Ports with eq 177
Chapter Review 177

**Part II Review 180**

**Part III Security Services 183**

**Chapter 9 Security Architectures 184**
"Do I Know This Already?" Quiz 184
Foundation Topics 186
Security Terminology 186
Common Security Threats 188
Attacks That Spoof Addresses 188
*Denial-of-Service Attacks 189*
*Reflection and Amplification Attacks 191*
*Man-in-the-Middle Attacks 191*
*Address Spoofing Attack Summary 193*
Reconnaissance Attacks 193
Buffer Overflow Attacks 194
Malware 194
Human Vulnerabilities 195
Password Vulnerabilities 196
*Password Alternatives 196*
Controlling and Monitoring User Access 198
Developing a Security Program to Educate Users 200
Chapter Review 201

**Chapter 10 Securing Network Devices 202**
"Do I Know This Already?" Quiz 202
Foundation Topics 204
Securing IOS Passwords 204
Encrypting Older IOS Passwords with service password-encryption 205
Encoding the Enable Passwords with Hashes 206
*Interactions Between Enable Password and Enable Secret 206*

*Making the Enable Secret Truly Secret with a Hash 207*
*Improved Hashes for Cisco's Enable Secret 209*
Encoding the Passwords for Local Usernames 210
Firewalls and Intrusion Prevention Systems 211
Traditional Firewalls 211
*Security Zones 213*
Intrusion Prevention Systems (IPS) 215
Cisco Next-Generation Firewalls 216
Cisco Next-Generation IPS 218
Chapter Review 219

**Chapter 11 Implementing Switch Port Security 222**
"Do I Know This Already?" Quiz 222
Foundation Topics 224
Port Security Concepts and Configuration 224
Configuring Port Security 225
Verifying Port Security 228
Port Security MAC Addresses 229
Port Security Violation Modes 230
Port Security Shutdown Mode 231
Port Security Protect and Restrict Modes 233
Chapter Review 235

**Chapter 12 DHCP Snooping and ARP Inspection 238**
"Do I Know This Already?" Quiz 238
Foundation Topics 240
DHCP Snooping 240
DHCP Snooping Concepts 240
*A Sample Attack: A Spurious DHCP Server 241*
*DHCP Snooping Logic 242*
*Filtering DISCOVER Messages Based on MAC Address 243*
*Filtering Messages That Release IP Addresses 244*
DHCP Snooping Configuration 245
*Configuring DHCP Snooping on a Layer 2 Switch 246*
*Limiting DHCP Message Rates 248*
*DHCP Snooping Configuration Summary 249*
Dynamic ARP Inspection 250
DAI Concepts 250
*Review of Normal IP ARP 250*

*Gratuitous ARP as an Attack Vector 251*
*Dynamic ARP Inspection Logic 253*
Dynamic ARP Inspection Configuration 254
*Configuring ARP Inspection on a Layer 2 Switch 254*
*Limiting DAI Message Rates 257*
*Configuring Optional DAI Message Checks 258*
*IP ARP Inspection Configuration Summary 259*
Chapter Review 260

**Part III Review 264**

**Part IV IP Services 267**

**Chapter 13 Device Management Protocols 268**
"Do I Know This Already?" Quiz 268
Foundation Topics 270
System Message Logging (Syslog) 270
Sending Messages in Real Time to Current Users 270
Storing Log Messages for Later Review 271
Log Message Format 272
Log Message Severity Levels 272
Configuring and Verifying System Logging 273
The debug Command and Log Messages 276
Network Time Protocol (NTP) 277
Setting the Time and Time Zone 278
Basic NTP Configuration 279
NTP Reference Clock and Stratum 281
Analyzing Topology Using CDP and LLDP 283
Examining Information Learned by CDP 283
Configuring and Verifying CDP 286
Examining Information Learned by LLDP 287
Configuring and Verifying LLDP 290
*LLDP-MED and TLVs 292*
Chapter Review 293

**Chapter 14 Network Address Translation 298**
"Do I Know This Already?" Quiz 298
Foundation Topics 300
Network Address Translation Concepts 300
IPv4 Address Conservation with NAT 300

Inside Source NAT 302
Static NAT 303
Inside Local and Inside Global Addresses 303
Dynamic NAT 304
Overloading NAT with Port Address Translation 306
NAT Configuration and Troubleshooting 307
Static NAT Configuration 308
Dynamic NAT Configuration 310
Dynamic NAT Verification 312
NAT Overload (PAT) Configuration 314
NAT Troubleshooting 317
Chapter Review 318

**Chapter 15 Quality of Service (QoS) 322**
"Do I Know This Already?" Quiz 322
Foundation Topics 324
Introduction to QoS 324
QoS: Managing Bandwidth, Delay, Jitter, and Loss 324
Types of Traffic 325
*Data Applications 325*
*Voice and Video Applications 327*
QoS as Mentioned in This Book 328
QoS on Switches and Routers 329
Classification and Marking 329
Classification Basics 329
Matching (Classification) Basics 330
Classification on Routers with ACLs and NBAR 331
Marking IP DSCP and Ethernet CoS 332
*Marking the IP Header 333*
*Marking the Ethernet 802.1Q Header 333*
*Other Marking Fields 334*
Defining Trust Boundaries 334
DiffServ Suggested Marking Values 335
*Expedited Forwarding (EF) 336*
*Assured Forwarding (AF) 336*
*Class Selector (CS) 336*
*Guidelines for DSCP Marking Values 337*

Queuing 337
Round-Robin Scheduling (Prioritization) 338
Low Latency Queuing 339
A Prioritization Strategy for Data, Voice, and Video 341
Shaping and Policing 341
Policing 342
*Where to Use Policing 342*
Shaping 344
*Setting a Good Shaping Time Interval for Voice and Video 345*
Congestion Avoidance 346
TCP Windowing Basics 346
Congestion Avoidance Tools 347
Chapter Review 348

**Chapter 16 First Hop Redundancy Protocols 350**
"Do I Know This Already?" Quiz 350
Foundation Topics 352
First Hop Redundancy Protocols 352
The Need for Redundancy in Networks 353
The Need for a First Hop Redundancy Protocol 354
The Three Solutions for First-Hop Redundancy 356
Hot Standby Router Protocol 356
HSRP Virtual IP and MAC Addresses 357
HSRP Failover 357
HSRP Load Balancing 359
HSRP Interface Tracking 359
HSRP Recovery and Preemption 360
HSRP Versions 361
VRRP and GLBP Concepts 362
Virtual Router Redundancy Protocol (VRRP) 362
GLBP Concepts 363
*Similarities of GLBP, HSRP, and VRRP 363*
*GLBP Active/Active Load Balancing 364*
Chapter Review 366

**Chapter 17 SNMP, FTP, and TFTP 368**
"Do I Know This Already?" Quiz 368
Foundation Topics 370
Simple Network Management Protocol 370

SNMP Variable Reading and Writing: SNMP Get and Set 371
SNMP Notifications: Traps and Informs 372
The Management Information Base 372
Securing SNMP 374
FTP and TFTP 376
Managing Cisco IOS Images with FTP/TFTP 376
*The IOS File System 376*
*Upgrading IOS Images 378*
*Copying a New IOS Image to a Local IOS File System Using TFTP 378*
*Listing the Files in the IOS File System 379*
*Verifying IOS Code Integrity with MD5 or SHA512 381*
*Copying Images with FTP 382*
The FTP and TFTP Protocols 384
*FTP Protocol Basics 384*
*FTP Active and Passive Modes 385*
TFTP Protocol Basics 387
Chapter Review 388

**Part IV Review 392**

**Part V Network Architecture 395**

**Chapter 18 LAN Architecture 396**
"Do I Know This Already?" Quiz 396
Foundation Topics 398
Analyzing Campus LAN Topologies 398
Two-Tier Campus Design (Collapsed Core) 399
Three-Tier Campus Design (Core) 400
Topology Design Terminology 402
Ethernet Physical Media and Standards 403
Ethernet UTP Links at the Access Layer 403
Multigig Ethernet on CAT 5E Cabling 405
Fiber Uplinks 406
Small Office/Home Office 407
Power over Ethernet (PoE) 408
PoE Basics 409
PoE Operation 409
PoE and LAN Design 411
Chapter Review 412

**Chapter 19 WAN Architecture 414**

"Do I Know This Already?" Quiz 414

Foundation Topics 416

Metro Ethernet 416

Metro Ethernet Physical Design and Topology 416

Ethernet WAN Services and Topologies 418

*Ethernet Line Service (Point-to-Point) 418*

*Ethernet LAN Service (Full Mesh) 419*

Layer 3 Design Using Metro Ethernet 420

*Layer 3 Design with E-Line Service 420*

*Layer 3 Design with E-LAN Service 421*

Multiprotocol Label Switching (MPLS) 422

MPLS VPN Physical Design and Topology 423

Layer 3 with MPLS VPN 424

Internet VPNs 425

Internet Access 426

*Digital Subscriber Line 426*

*Cable Internet 427*

*Wireless WAN (4G, 5G) 428*

*Fiber (Ethernet) Internet Access 429*

Internet VPN Fundamentals 430

*Site-to-Site VPNs with IPsec 431*

*Remote Access VPNs with IPsec 433*

*Remote Access VPNs with TLS 434*

Chapter Review 435

**Chapter 20 Cloud Architecture 438**

"Do I Know This Already?" Quiz 438

Foundation Topics 440

Server Virtualization 440

Cisco Server Hardware 440

Server Virtualization and Virtual Machine Basics 441

Networking with Virtual Switches on a Virtualized Host 443

Software Containers 444

The Physical Data Center Network 446

Workflow with a Virtualized Data Center 446

Cloud Computing Services 448

Private Cloud (On-Premise) 449

Public Cloud 450
Cloud and the "As a Service" Model 451
*Infrastructure as a Service 451*
*Software as a Service 452*
*(Development) Platform as a Service 453*
Virtual Routing and Forwarding (VRF) Instances 454
WAN Traffic Paths to Reach Cloud Services 456
Enterprise WAN Connections to Public Cloud 456
*Accessing Public Cloud Services Using the Internet 456*
*Pros and Cons with Connecting to Public Cloud with Internet 457*
*Private WAN and Internet VPN Access to Public Cloud 458*
*Pros and Cons of Connecting to Cloud with Private WANs 459*
*Intercloud Exchanges 459*
*Summarizing the Pros and Cons of Public Cloud WAN Options 460*
Understanding Cloud Management 460
Chapter Review 465

**Part V Review 466**

**Part VI Network Automation 469**

**Chapter 21 Introduction to Controller-Based Networking 470**
"Do I Know This Already?" Quiz 471
Foundation Topics 472
SDN and Controller-Based Networks 472
The Data, Control, and Management Planes 472
*The Data Plane 473*
*The Control Plane 474*
*The Management Plane 475*
*Cisco Switch Data Plane Internals 475*
Controllers and Software Defined Architecture 477
*Controllers and Centralized Control 477*
*The Southbound Interface 478*
*The Northbound Interface 479*
Software Defined Architecture Summary 481
Examples of Network Programmability and SDN 481
OpenDaylight and OpenFlow 481
*The OpenDaylight Controller 482*
*The Cisco Open SDN Controller (OSC) 483*

Cisco Application Centric Infrastructure (ACI) 484
*ACI Physical Design: Spine and Leaf 484*
*ACI Operating Model with Intent-Based Networking 486*
Summary of the SDN Examples 488
Comparing Traditional Versus Controller-Based Networks 488
How Automation Impacts Network Management 489
Comparing Traditional Networks with Controller-Based Networks 491
Chapter Review 492

**Chapter 22 Cisco Software-Defined Access (Cisco SD-Access) 494**
"Do I Know This Already?" Quiz 495
Foundation Topics 496
Cisco SD-Access Fabric, Underlay, and Overlay 496
The Cisco SD-Access Underlay 499
*Using Existing Gear for the Cisco SD-Access Underlay 499*
*Using New Gear for the Cisco SD-Access Underlay 501*
The Cisco SD-Access Overlay 503
*VXLAN Tunnels in the Overlay (Data Plane) 504*
*LISP for Overlay Discovery and Location (Control Plane) 505*
Cisco Catalyst Center and Cisco SD-Access Operation 509
Cisco Catalyst Center 509
Cisco Catalyst Center and Scalable Groups 510
*Issues with Traditional IP-Based Security 511*
*Cisco SD-Access Security Is Based on User Groups 512*
Cisco Catalyst Center as a Network Management Platform 514
Cisco Catalyst Center Similarities to Traditional Management 515
Cisco Catalyst Center and Differences with Traditional Management 516
Artificial Intelligence (AI), Machine Learning (ML), and Operational Management 517
Chapter Review 524

**Chapter 23 Understanding REST and JSON 526**
"Do I Know This Already?" Quiz 526
Foundation Topics 528
REST-Based APIs 528
REST-Based (RESTful) APIs 528
*Client/Server Architecture 529*
*Stateless Operation 530*
*Cacheable (or Not) 530*

Background: Data and Variables 530
*Simple Variables 530*
*List and Dictionary Variables 531*
REST APIs and HTTP 533
*Software CRUD Actions and HTTP Verbs 533*
*Using URIs with HTTP to Specify the Resource 534*
Example of REST API Call to Cisco Catalyst Center 536
Data Serialization and JSON 541
The Need for a Data Model with APIs 542
Data Serialization Languages 544
*JSON 544*
*XML 544*
*YAML 545*
*Summary of Data Serialization 546*
Recognizing the Components of JSON 546
*Interpreting JSON Key:Value Pairs 547*
*Interpreting JSON Objects and Arrays 547*
*Minified and Beautified JSON 550*
Chapter Review 550

**Chapter 24 Understanding Ansible and Terraform 552**
"Do I Know This Already?" Quiz 552
Foundation Topics 554
Device Configuration Challenges and Solutions 554
Configuration Drift 554
Centralized Configuration Files and Version Control 555
Configuration Monitoring and Enforcement 557
Configuration Provisioning 558
*Configuration Templates and Variables 559*
*Files That Control Configuration Automation 561*
Ansible and Terraform Basics 562
Ansible 562
Terraform 563
Summary of Configuration Management Tools 565
Chapter Review 566

**Part VI Review 568**

**Part VII Exam Updates and Final Review 571**

**Chapter 25 *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, Exam Updates 572**

The Purpose of This Chapter 572
Additional Technical Content 573
Official Blueprint Changes 573
Impact on You and Your Study Plan 575
News About the Next CCNA Exam Release 576
Updated Technical Content 576

**Chapter 26 Final Review 578**

Advice About the Exam Event 578
Learn About Question Types 578
Think About Your Time Budget 581
An Example Time-Check Method 581
One Week Before Your Exam 582
24 Hours Before Your Exam 582
30 Minutes Before Your Exam 583
The Hour After Your Exam 583
Exam Review 584
Using Practice Questions 585
*Hold Practice Exam Events 586*
*Exam Scoring on the Real Exam 587*
*Self-Assessment Suggestions 587*
*Gap Analysis Using Q&A 589*
*Advice on How to Answer Exam Questions 590*
*Additional Exams with the Premium Edition 592*
Practicing CLI Skills 593
Adjustments for Your Second Attempt 595
Other Study Tasks 596
Final Thoughts 596

**Part VIII Print Appendixes 599**

**Appendix A Numeric Reference Tables 601**

**Appendix B Exam Topics Cross-Reference 607**

**Appendix C Answers to the "Do I Know This Already?" Quizzes 619**

**Glossary 641**

**Index 668**

## Online Appendixes

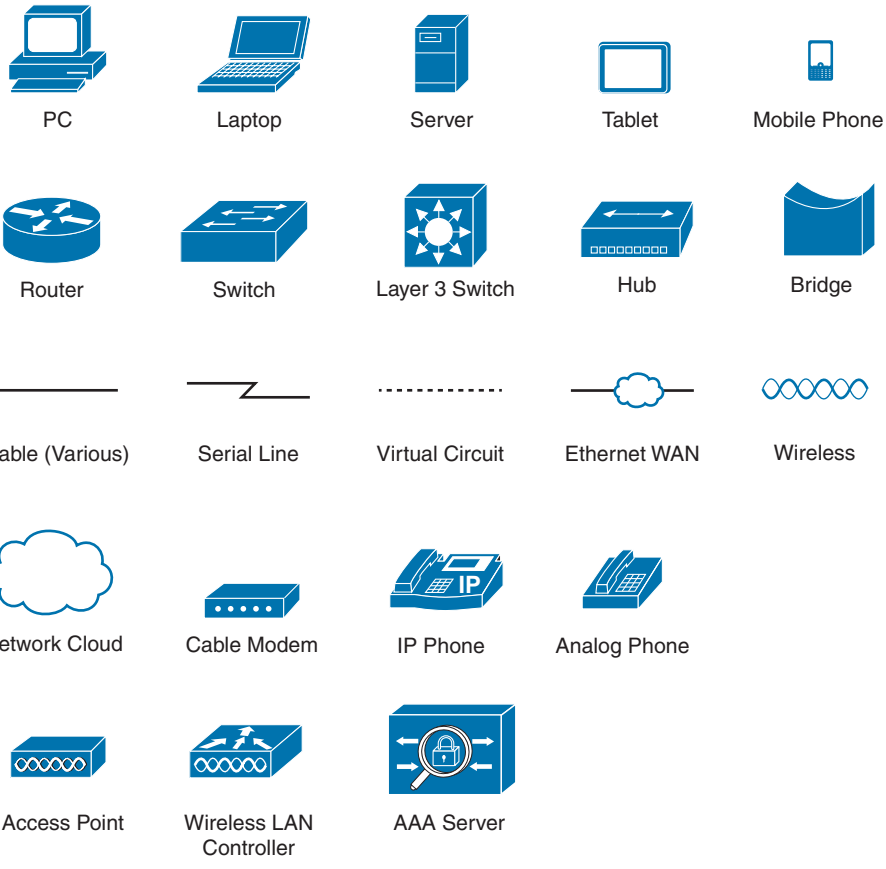
**Appendix D Topics from Previous Editions**

**Appendix E Practice for Chapter 6: Basic IPv4 Access Control Lists**

**Appendix F Study Planner**

**Glossary**

## Icons Used in This Book




## Command Syntax Conventions

The conventions used to present command syntax in this book are the same conventions used in the IOS Command Reference. The Command Reference describes these conventions as follows:

- **Boldface** indicates commands and keywords that are entered literally as shown. In actual configuration examples and output (not general command syntax), boldface indicates commands that are manually input by the user (such as a **show** command).
- *Italic* indicates arguments for which you supply actual values.
- Vertical bars (|) separate alternative, mutually exclusive elements.
- Square brackets ([ ]) indicate an optional element.
- Braces ({ }) indicate a required choice.
- Braces within brackets ([{ }]) indicate a required choice within an optional element.

# Introduction

You are setting out on a journey to achieve your CCNA certification. For many, that step happens at the beginning of a new career path. For others, CCNA validates their knowledge and skills already learned on the job.

Surprisingly for an entry-level exam, the CCNA 200-301 exam includes more content by volume than many of the CCNP-level exams. As a result, Cisco Press publishes the Certification Guide for CCNA as two volumes. You can refer to the books as Volume 1 and Volume 2, but more formally, the books are

- *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition
- *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition (this book)

If you have already used the Volume 1 book and read or skimmed its Introduction, you do not need to read the entire Introduction to this book. This book has the same features and style as Volume 1. However, you might be interested to review the section titled "Book Organization, Chapters, and Appendixes," for information specific to this book.

Regardless of your path to CCNA, the journey takes some time and effort. I encourage you to spend some time in the Introduction to learn more about CCNA and the books so you can have the best experience preparing for CCNA! To that end, this introduction discusses these main points:

Cisco Certifications and the CCNA

Book Features

Book Elements (Reference)

About Getting Hands-on Skills

About IP Subnetting

# Cisco Certifications and the CCNA

Congratulations! If you're reading far enough to look at this book's Introduction, you've probably already decided to go for your Cisco certification. Cisco has been the dominant vendor in networking for decades. If you want to be taken seriously as a network engineer, building your Cisco skills using Cisco certifications makes perfect sense. Where to start? CCNA.

## Cisco Certifications as of 2024

CCNA acts as the entry point to a hierarchy of Cisco certifications. CCNA includes the foundational topics, with CCNP as the next higher challenge level, followed by CCIE. Figure I-1 shows the hierarchy, with more detail about each in the list that follows.

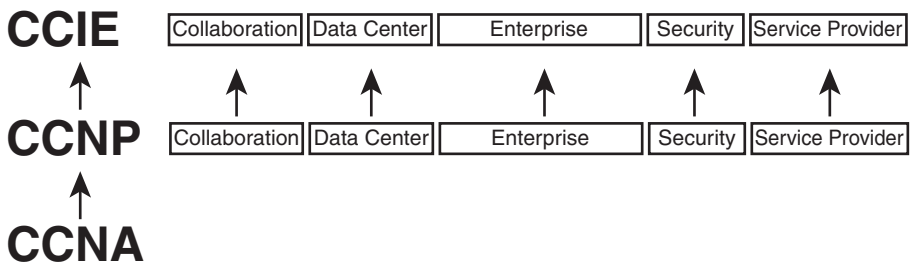



**Figure I-1** *Cisco CCNA, CCNP, and CCIE Certifications*

**CCNA – Cisco Certified Network Associate:** Cisco began CCNA with a single CCNA certification back in 1998. They later expanded CCNA to include ten different CCNA certifications about different technology areas. Cisco retired all the varieties of CCNA back in 2020, leaving us again with a single CCNA certification, now referred to as simply "CCNA."

**CCNP – Cisco Certified Network Professional:** Cisco followed the same progression with different CCNP certifications over time, starting with one in 1998. The big changes in 2020 consolidated the lineup to five CCNP certifications, all of which benefit from having knowledge of CCNA before moving on to CCNP.

**CCIE – Cisco Certified Internetwork Expert:** First introduced in 1993, these expert-level certifications require both a written exam plus a one-day practical exam with extensive hands-on lab challenges.

Beyond the CCNA, CCNP, and CCIE certifications, Cisco offers two other certification tracks—one for network automation and another for cybersecurity. The CCNA certification can be helpful as a foundation for those tracks as well. They are

**DevNet Certifications:** The DevNet Associate, DevNet Professional, and DevNet Expert certifications mirror the progression of CCNA/CCNP/CCIE, just without using those specific acronyms. The DevNet certifications focus on software development and APIs that matter to managing networks.

**CyberOps Certifications:** The CyberOps Associate and CyberOps Professional certifications mirror the progression of CCNA/CCNP. These security exams focus on security concepts, security monitoring, host-based analysis, network intrusion analysis, and security policies and procedures.

## How to Get Your CCNA Certification

As you saw in Figure I-1, all career certification paths now begin with CCNA. So how do you get the CCNA certification? Today, you have one and only one option to achieve CCNA certification:

Take and pass one exam: the Cisco 200-301 CCNA exam.

To take the 200-301 exam, or any Cisco exam, you will use the services of Pearson VUE. The process works something like this:

1. Establish a login at https://vue.com/cisco (or use your existing login).

2. Register for, schedule a time and place, and pay for the Cisco 200-301 exam, all from the VUE website.
3. Take the exam at the VUE testing center or from home with a video proctor watching to prevent cheating.
4. You will receive a notice of your score, and whether you passed, before you leave the testing center.

## Content in the CCNA 200-301 Exam

We've all thought it, wondered, for almost every important test we ever took, and maybe even asked the teacher: "What's on the test?" For the CCNA exam, and for all Cisco certification exams, Cisco tells us.

Cisco publishes an exam blueprint for every Cisco exam, with the blueprint listing the exam topics for the exam. To find them, browse www.cisco.com/go/certifications, look for the CCNA page, and navigate until you see the exam topics. And if you haven't already done so, create a bookmark folder for CCNA content in your web browser and bookmark a link to this page.

The exam blueprint organizes the exam topics into groups called domains. The document also tells us the percentage of points on the exam that come from each domain. For instance, every CCNA exam should score 25 percent of your points from the exam topics in the IP Connectivity domain. The exam does not tell you the domain associated with each question, but the percentages give us a better idea of the importance of the domains for the exam. Figure I-2 shows the domains of the CCNA 200-301 Version 1.1 blueprint, the percentages, and the number of primary exam topics in each.

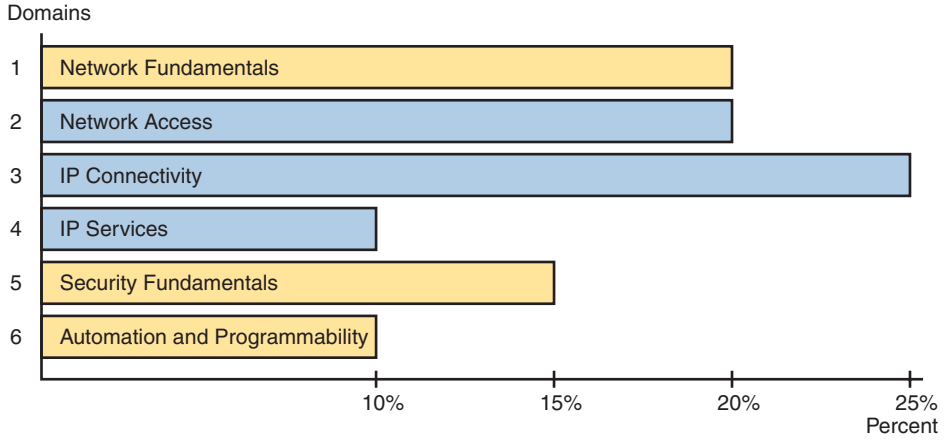



**Figure I-2** *CCNA 200-301 Domains and Percentage of Exam Score*

Within each domain, the exam topic document lists exam topics that follow two different styles of wording. The main exam topics use a verb in the phrase that tells you the level of mastery required; I call those primary exam topics. The exam topics document shows subtopics that I refer to as secondary exam topics. Those do not have a verb, but list more technology details (nouns), and assume the verb from the primary exam topic.

For instance, the following excerpt from the exam topics document lists one primary exam topic with the *describe* verb, with more detail added by two secondary exam topics.

1.11 Describe wireless principles

1.11.a Nonoverlapping Wi-Fi channels

1.11.b SSID

### Exam Topic Verbs (Depth) and Nouns (Breadth)

Understanding an exam topic requires that you think about each exam topic wording, focusing on the verbs and nouns. The nouns identify the technical topics, such as LAN switching, IP routing, protocols like OSPF, and so on. The verbs in each primary exam topic inform us about the type and depth of knowledge and skill tested per the exam topics.

For example, consider the following primary exam topic:

**Describe** IPsec remote access and site-to-site VPNs

I'm sure you know what the word *describe* means in the normal use of the term. But for people who build exams, the verb has special meaning as to what the exam questions should and should not require of the test taker. For instance, you should be ready to describe whatever "IPsec remote access and site-to-site VPNs" are. But the exam should not ask you to perform higher performance verbs, like *analyze* or *configure*.

Figure I-3 shows a pyramid with verbs found in Cisco exam blueprints. It shows the lower-skill verbs at the bottom and higher skills at the top. An exam topic with a lower verb should not be tested with questions from higher knowledge and skill levels. For instance, with the exam topic "describe…first hop redundancy protocols," you should not expect to need to configure, verify, or troubleshoot the feature.

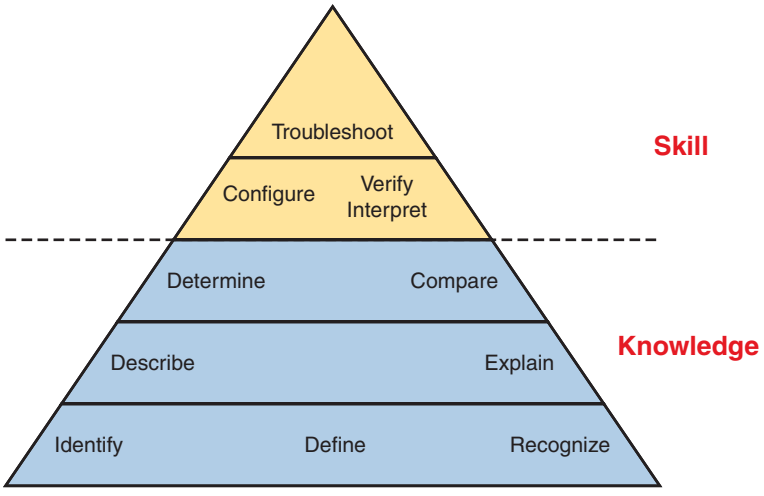



**Figure I-3** *Cisco Exam Topic Verbs*

Knowing that, how should you study? Well, instead of a many-layer pyramid, think of it as two layers: Knowledge and Skill. When learning content whose exam topics use verbs from the lower three rows of the pyramid, study the same way no matter which of those verbs the exam topic uses. Learn the topic well. Be ready to describe it, explain it, and

interpret the meaning. For content with exam topics with the verbs *configure* and *verify*, think of those as including the first level of knowledge, plus also requiring configuration and verification skills. Also, think about the common configuration mistakes so you can troubleshoot those mistakes.

## Comparing the Exam and Exam Topics

To understand what Cisco tells us about the exam versus the exam topics, return to cs.co/go/certifications or cisco.com/go/ccna. Find the CCNA exam topics and open the PDF version (the text we need to consider is currently only in the PDF version). Open the PDF and spend 10–15 seconds scanning it.

Did you read the first two paragraphs, the ones before the list of exam topics? Or did you skip those and move straight to the long list of exam topics? Many people skip those paragraphs. One of those tells us much about the exam versus the exam topics, so I've copied it here, with emphasis added:

> The following topics are ***general guidelines*** for the content likely to be included on the exam. However, ***other related topics may also appear on any specific delivery of the exam***. To better reflect the contents of the exam and for clarity purposes, the ***guidelines below may change at any time without notice***.

The first bold phrase mentions the most obvious point about the exam topics: They make a general statement. They do not detail every concept, fact, configuration option, and fact hidden in verification command output. Instead, anyone who cares about a Cisco exam like CCNA has to make a judgment about exactly what details the exam topic includes and excludes—and those judgements are subjective.

Our interpretation and expansion of the exam topics dictate what we choose to include and omit from the books. But we know from long experience that narrow interpretation can cover a large amount of CCNA content, but leave out too much. To cover as much as possible, we use a broad and deep interpretation. That has worked well throughout the 25 plus years for this book and its predecessors. It also matches the overwhelming feedback from reader surveys. In short, we shoot for the middle box in the concept drawing in Figure I-4.

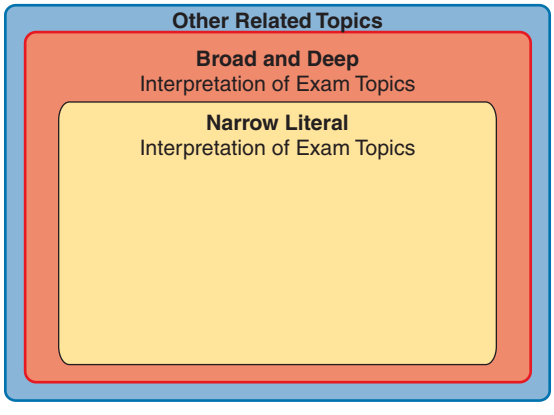



**Figure I-4** *Scope Concept: Exam Versus Exam Topics*

Cisco tells us the exam can ask questions about topics outside the exam topics. Look back again to the copy of the text that begins the exam blueprint PDF, to the bold phrase with the term "other related topics." Yes, the exam might ask things outside even a broad interpretation of the exam topics, as implied by the outer ring in Figure I-4.

When choosing book content, I also make some predictions as to what those other related topics might be. Given the policies, we cannot possibly predict and include everything you might see on your exam. What we do promise is to cover each exam topic based on a broad and deep interpretation.

### How to Prepare for the Generalized Exam Topics

Given the general nature of Cisco exam topics, plus the possibility of topic areas not listed in the exam topics, how should you go about preparing for the CCNA exam? Most importantly, strive to master all information and skills implied by the exam topics. On exam day, you may still see a few questions about topics you have not studied—but you should know far more than enough to pass the exam.

So, how do you master the topics listed in the exam blueprint? Let me give you a few suggestions.

1. Follow the suggestions in Volume 1's section "Your Study Plan" just before Chapter 1 of that book.
2. Practice hands-on CLI skills. The later section of the Introduction titled "About Building Hands-On Skills" discusses some ways to practice.
3. Pay close attention to troubleshooting topics in the book.
4. Practice all math-related skills, over time, until you master them.
5. Ensure you know all exam topic content as listed in the exam topics. Read the exam topics, consider your own literal interpretation, and when uncertain or confused, dig in and study further.
6. Trust that the book uses its broad interpretation of the exam topics to help you learn as much as possible that might be on the exam.

## Types of Questions on the CCNA 200-301 Exam

You can expect the following kinds of questions on the exam; just be aware that the style of questions may change over time.

- Multiple-choice, single-answer
- Multiple-choice, multiple-answer
- Drag-and-drop
- Lab

For the multichoice questions, the exam software gives us a few important advantages:

- There is no penalty for guessing.
- Multichoice questions with a single correct answer require you to answer and allow only one answer.
- Multichoice questions with multiple correct answers tell you the number of correct answers and warn you if you have not selected that many answers.

For instance, if a question tells you there are two correct answers, and you select only one and then try to move to the next question, the app reminds you that you should choose another answer before moving on.

As for drag-and-drop, some questions use simple text blocks that you move from one list to another. However, you might see questions where you move items in a network diagram or some other creative use of drag-and-drop.

Finally, Cisco introduced lab questions (formally called performance-based questions) in 2022. Lab questions present you with a lab scenario with a lab pod of virtual routers and switches running in the background; you get console access to a few devices. Your job: find the missing or broken configuration and reconfigure the devices so that the lab scenario works. The best way to practice for these questions is to practice in lab; more on that in the section titled "About Building Hands-On Skills."

As an aside, prior Cisco exams had Sim questions instead of lab questions. Sim questions required the same from us: read the scenario and fix the configuration. However, Sim questions used simulated Cisco devices with limited command support, which frustrated some test takers. The lab questions use real Cisco operating systems running in a virtual environment, so they provide a much more realistic experience compared to old Sim questions.

## Book Features

This book includes many study features beyond the core explanations and examples in each chapter. This section acts as a reference to the various features in the book.

### The CCNA Books: Volume 1 and Volume 2

The CCNA exam covers a large amount of content, and it does not fit in a single volume. As a result, Cisco Press has long published books for the CCNA exam as a two-book set. Volume 1 covers about half of the content, and Volume 2 covers the rest, as shown in Figure I-5. To best use both books, start in Volume 1 and work through the book in order, and then do the same with Volume 2.

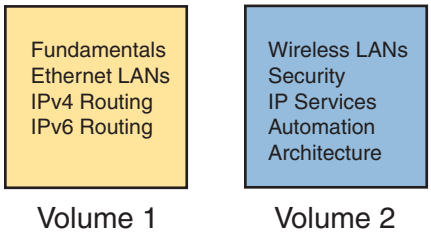



**Figure I-5** *Two Books for CCNA 200-301*

When you start each new chapter, review the list of exam topics that begins the chapter. The book does not follow the same order of exam topics in the blueprint, but instead follows a more effective order for learning the topics. For reference, look to Appendix B, "Exam Topics Cross-Reference," in the back of the book. The appendix includes:

- A list of exam topics and the chapter(s) covering each topic
- A list of chapters and the exam topics covered in each chapter

### Exam Blueprint Versions and Book Editions

Cisco made minor changes to the CCNA exam blueprint in 2024, the first change to the CCNA 200-301 exam since the year 2020. The much more important change (announced in 2023) had to do with the entire Cisco certification program about how Cisco announces and releases new exams and exam blueprints. Before 2023, when Cisco changed any CCNA or CCNP exam, they also changed the exam number, and the announcement was sudden. Those days are gone.

You should read and understand Cisco's long-term strategy for being more forthright about exam plans as detailed at www.cisco.com/go/certroadmap. Summarizing some key points, when Cisco changes an exam in the future, Cisco will keep the same exam number. To identify the changes, they will use a major.minor version numbering plan for every exam blueprint. More importantly, Cisco tells us when they will consider changing CCNA each year, but we know when Cisco will announce changes and when the new exam will be released, within a few months' timing.

The exam blueprint version changes based on two determinations: 1) whether Cisco will change the exam that year at all, and 2) if so, whether Cisco considers the changes to be major or minor. For instance, Cisco considered making a change to CCNA during February–April 2023 but chose not to change it, announcing that fact in the May–July 2023 timeframe. In 2024, Cisco chose to make minor changes to the CCNA blueprint. As a result, the former CCNA blueprint version 1.0 (major version 1, minor version 0) changed to version 1.1, increasing the minor version by 1.

Looking forward, if the next three future CCNA blueprint changes are also minor, they would be blueprint versions 1.2, 1.3, and 1.4. However, if any of them are major, that version would move to the next major version (2.0), with subsequent minor version changes as 2.1, 2.2, and so on.

Cisco also tells us that each year, internally, Cisco considers what to do with CCNA in the February–April timeframe. They will announce their plans to us all between May–July, and they will release the new exam (if changes are being made) sometime in the six months or so following the announcement.

As for the publishing plans to support that new update cycle, you should read and monitor the publisher's web page at www.ciscopress.com/newcerts. Also, opt in for communications on that page so the publisher will email you about future plans and updates.

Summarizing a few key points about the publisher's plans, this book, the second edition, was written for version 1.1 of the CCNA 200-301 blueprint, but it should be the book used for CCNA for subsequent blueprint versions as well. During the life of this second edition book, Cisco may update the CCNA 200-301 exam blueprint a few times, while this book (plus the Volume 1 Second Edition book) may remain unchanged. New exam content may be made available as electronic downloads. At some point, a new edition will be appropriate. (Figure I-6 shows one example of what might happen over time, with downloadable PDFs between editions.)

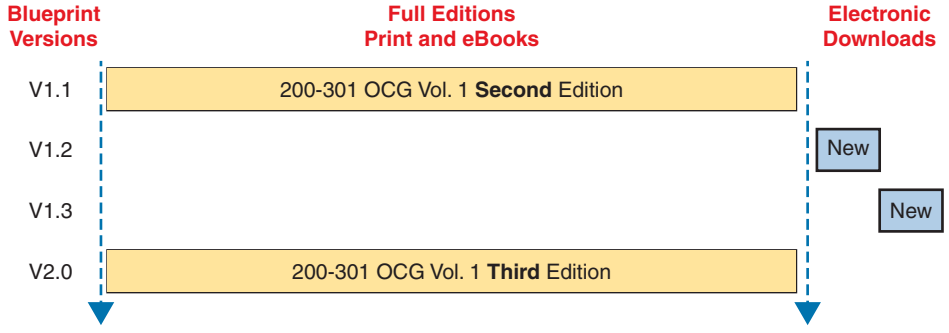



**Figure I-6** *Possible Progression of Book Editions, New Content Release, Versus Exams*

**NOTE** I cannot stress enough: monitor both the Cisco Press and Cisco pages linked in the preceding paragraphs, and opt in for communications at those pages, to stay aware of any exam and publishing plans. Also, consider watching my blog (www.certskills.com), where I expect to post about changes.

When you finish the technology chapters in this book (Chapters 1–24), make sure to also read Chapter 25, "Exam Updates." We post updated versions of that chapter online. We use that chapter to deliver small content updates to you as well as to inform you about future content updates. Make sure to read through that chapter and learn how to download the latest version of the chapter, discovering if new content has been posted after this book was published.

Also, just to reduce confusion about the book titles, note that the prior edition of this book is nearly identical to this book's title. Comparing the titles:

- *CCNA 200-301 Official Cert Guide, Volume 2* (the prior edition, published in 2019 for 200-301 exam blueprint version 1.0)
- *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition (this edition, published in 2024 for 200-301 exam blueprint version 1.1 and beyond)

## Comparing This Edition to the Previous

This book replaces a similar book that applied to the former CCNA 200-301 exam blueprint 1.0. Some of you may buy this book but have already begun studying with the

prior edition. The following list of major changes to this book versus the previous may help you avoid rereading identical or similar content in this book.

**Chapter 4:** (Formerly Volume 1, Chapter 29) Added examples and GUI screenshots for wireless LAN configuration using a WLC with IOS XE.

**Chapter 7:** (Formerly Chapter 3) Significant rewrite; added more about rules for ACL editing, and noted differences with ACLs between IOS and IOS XE.

**Chapter 8:** New chapter about matching different protocols using ACLs.

**Chapter 10:** (Formerly Chapter 5) Revised password type discussion and added IOS versus IOS XE details.

**Chapter 13:** (Formerly Chapter 9) Added CDP/LLDP timers and LLDP-MED with TLVs.

**Chapter 14:** (Formerly Chapter 10) Revised NAT chapter to simplify and strengthen the examples.

**Chapter 16:** (Formerly Chapter 12) Split FHRP into a separate chapter, expanding discussions to give more detail on VRRP and GLBP.

**Chapter 17:** (Formerly Chapter 12) Split SNMP, FTP, and TFTP from FHRP into a separate chapter. Expanded the SNMP section and revised the IFS topic with IOS XE examples.

**Chapter 18:** (Formerly Chapter 13) Added topics for UTP cabling, multigig Ethernet, multimode Ethernet for campus LANs, and improved PoE descriptions.

**Chapter 19:** (Formerly Chapter 14) Updated notes about 5G WAN, and added detail on IPsec.

**Chapter 20:** (Formerly Chapter 15) Added concepts of containers and VRFs, and adds cloud management topics including Meraki.

**Chapters 21–22:** (Formerly Chapters 16 and 17) Updated for various software version updates and product rebranding, and adds the AI/ML topic.

**Chapter 23:** (Formerly Chapter 18) Updated for exam topic change to include API authentication.

**Chapter 24:** (Formerly Chapter 19) Updated for exam topic change to replace the descriptions of Puppet and Chef with new information about Terraform.

If you find the preceding information useful, consider looking in two other places that allow us to provide ongoing updates and to answer questions. First, I expect to post blog posts about the new CCNA exam changes, as always, at my blog (www.certskills.com). Look there for posts in the News section (click the General menu item and then News), for posts made mid-year 2024 when Cisco should announce their plans.

Second, look to the companion website for this book for details about future exam revisions and publishing plans. The companion website gives the publisher a place to list details about changes moving forward. See this Introduction's later section titled "The Companion Website for Online Content" for the instructions for finding the site.

## Chapter Features

Beginning to study CCNA can be overwhelming at first due to the volume. The best way to overcome that reaction requires a change in mindset: *treat each chapter as a separate study task*. Breaking your study into manageable tasks helps a lot.

Each chapter of this book is a self-contained short course about one small topic area, organized for reading and study. I create chapters so they average about 20 pages to cover the technology so that no one chapter takes too long to complete. Each chapter breaks down as follows:

**"Do I Know This Already?" quizzes:** Each chapter begins with a pre-chapter quiz so you can self-assess how much you know coming into the chapter.

**Foundation Topics:** This is the heading for the core content section of the chapter, with average length of 20 pages.

**Chapter Review:** This section includes a list of study tasks useful to help you remember concepts, connect ideas, and practice skills-based content in the chapter.

Do not read the "Foundation Topics" section of chapter after chapter without pausing to review and study. Each "Chapter Review" section uses a variety of other book features to help you study and internalize that chapter's content, including the following:

- **Review Key Topics:** All the content in the books matters, but some matters more. Cisco Press certification guides use a Key Topic icon next to those items in the "Foundation Topics" section. The "Chapter Review" section lists the key topics in a table. You can scan the chapter to review them or review the Key Topics more conveniently using the companion website.
- **Complete Tables from Memory:** We convert some tables in the book to interactive study tables called memory tables. You access memory tables from the companion website. Memory tables repeat the table, but with parts of the table removed. You can then fill in the table to exercise your memory and click to check your work.
- **Key Terms You Should Know:** The "Chapter Review" section lists the key terminology from the chapter. For a manual process with the book, think about each term and use the Glossary to cross-check your own mental definitions. Alternately, review the key terms with the "Key Terms Flashcards" app on the companion website.
- **Labs:** You should practice hands-on skills for any exam topics with the *configure* and *verify* verbs. The upcoming section titled "About Building Hands-On Skills" discusses your lab options. Also, the Chapter and Part Reviews refer you to lab exercises specific to the chapter or part.
- **Command References:** Some book chapters discuss the configure and verify exam topics, so they list various router and switch commands. The "Chapter Review" section of those chapters includes command reference tables, useful both for reference and for study. Just cover one column of the table and see how much you can remember and complete mentally.
- **Review DIKTA Questions:** Even if you used the DIKTA questions to begin the chapter, re-answering those questions can prove a useful way to review facts.

By design, I do not mention the DIKTA questions in the "Chapter Review" sections but do suggest using them again for all chapters in a part during Part Review. Use the Pearson Test Prep (PTP) web app to easily use those questions any time you have a few minutes, a device, and Internet access.

## Part Features

Your second mindset change: Use the book parts as major milestones in your study journey. Each part groups a small number of related chapters together. Take the time at the end of each part to review all topics in the part, effectively rewarding yourself with a chance to deepen your knowledge and internalize more of the content before moving to the next part. Figure I-7 shows the concept.

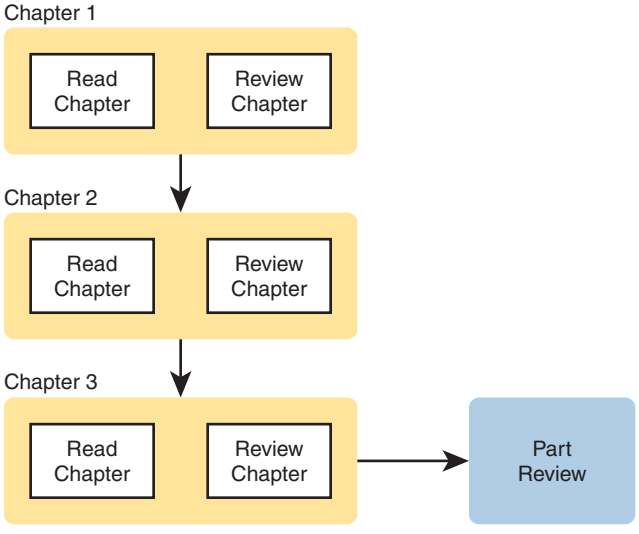



**Figure I-7** *Part Review: The Second Review of Most Content*

The Part Review element at the end of each part suggests review and study activities. Spaced reviews—that is, reviewing content several times over the course of your study—help improve retention. Using the Chapter and Part Review process, the Part Review serves as your second review of the content in each chapter. The Part Review repeats some Chapter Review activities and offers some new ones, including a reminder to use practice questions set aside specifically for Part Review.

## The Companion Website for Online Content

Some Chapter and Part Review tasks can be done from the book. However, several of them work better as an interactive online tool. For instance, you can take a "Do I Know This Already?" quiz by reading the pages of the book, but you can also use the PTP testing software. As another example, when you want to review the key terms from a chapter, you can find all those in electronic flashcards.

This book's companion website hosts all the electronic components of the book. The companion website gives you a big advantage: you can do most of your Chapter and

Part Review work from anywhere using the interactive tools on the site. The advantages include

- **Easier to use:** Instead of having to print out copies of the appendixes and do the work on paper, you can use these new apps, which provide you with an easy-to-use, interactive experience that you can easily run over and over.
- **Convenient:** When you have a spare 5–10 minutes, go to the book's website and review content from one of your recently finished chapters.
- **Good break from reading:** Sometimes looking at a static page after reading a chapter lets your mind wander. Breaking up your reading with some review from the keyboard can help keep you focused on the activity.

The interactive Chapter Review elements should improve your chances of passing as well. Our in-depth reader surveys over the years show that those who do the Chapter and Part Reviews learn more. Those who use the interactive review elements tend to do the review tasks more often. So, take advantage of the tools and maybe you will be more successful as well. Table I-1 summarizes these interactive applications and the traditional book features that cover the same content.

**Table I-1** Book Features with Both Traditional and App Options

| Feature | Traditional | App |
|---|---|---|
| Key Topic | The "Chapter Review" section lists the key topics. To review, flip pages in the chapter. | Key Topics Table app with links to view each key topic |
| Config Checklist | This list of steps, in text, describes how to configure a feature. | Config Checklist app, where you complete the checklist by adding commands |
| Key Terms | Terms are listed in each "Chapter Review" section; review using the end-of-book Glossary. | Key Terms Flash Cards app |
| Appendixes: ACL Practice | Appendix E provides static text practice problems and answers in the PDF appendixes. | Apps with the same practice problems, found in the "Memory Tables and Practice Exercises" section |

The companion website also includes links to download, navigate, or stream for these types of content:

- Pearson Sim Lite Desktop App
- Pearson Test Prep (PTP) Desktop App
- Pearson Test Prep (PTP) Web App
- Videos

## How to Access the Companion Website

To access the companion website, which gives you access to the electronic content with this book, start by establishing a login at www.ciscopress.com and register your book. To do so, simply go to www.ciscopress.com/register and enter the ISBN of the print book: **9780138214951.** After you have registered your book, go to your account page and click the **Registered Products** tab. From there, click the **Access Bonus Content** link to get access to the book's companion website.

Note that if you buy the *Premium Edition eBook and Practice Test* version of this book from Cisco Press, your book will automatically be registered on your account page. Simply go to your account page, click the **Registered Products** tab, and select **Access Bonus Content** to access the book's companion website.

## How to Access the Pearson Test Prep (PTP) App

You have two options for installing and using the Pearson Test Prep application: a web app and a desktop app. To use the Pearson Test Prep application, start by finding the registration code that comes with the book. You can find the code in these ways:

- You can get your access code by registering the print ISBN 9780138214951 on ciscopress.com/register. Make sure to use the print book ISBN, regardless of whether you purchased an eBook or the print book. After you register the book, your access code will be populated on your account page under the Registered Products tab. Instructions for how to redeem the code are available on the book's companion website by clicking the Access Bonus Content link.
- If you purchase the Premium Edition eBook and Practice Test directly from the Cisco Press website, the code will be populated on your account page after purchase. Just log in at ciscopress.com, click Account to see details of your account, and click the digital purchases tab.

**NOTE** After you register your book, your code can always be found in your account under the Registered Products tab.

Once you have the access code, to find instructions about both the PTP web app and the desktop app, follow these steps:

**Step 1.** Open this book's companion website as shown earlier in this Introduction under the heading, "How to Access the Companion Website."

**Step 2.** Click the **Practice Exams** button.

**Step 3.** Follow the instructions listed there for both installing the desktop app and using the web app.

Note that if you want to use the web app only at this point, just navigate to pearsontestprep.com, log in using the same credentials used to register your book or purchase the Premium Edition, and register this book's practice tests using the registration code you just found. The process should take only a couple of minutes.

### Feature Reference

The following list provides an easy reference to get the basic idea behind each book feature:

- **Practice exam:** The book gives you the rights to the Pearson Test Prep (PTP) testing software, available as a web app and a desktop app. Use the access code on a piece of cardboard in the sleeve in the back of the book, and use the companion website to download the desktop app or navigate to the web app (or just go to www.pearsontestprep.com).
- **eBook:** Pearson offers an eBook version of this book that includes extra practice tests as compared to the print book. The product includes two versions of the eBook: PDF (for reading on your computer) and EPUB (for reading on your tablet, mobile device, or Kindle, Nook, or other e-reader). It also includes additional practice test questions and enhanced practice test features, including links from each question to the specific heading in the eBook file.
- **Mentoring videos:** The companion website also includes a number of videos about other topics as mentioned in individual chapters. Some of the videos explain common mistakes made with CCNA topics, whereas others provide sample CCNA questions with explanations.
- **CCNA 200-301 Network Simulator Lite:** This Lite version of the best-selling CCNA Network Simulator from Pearson provides you with a means, right now, to experience the Cisco command-line interface (CLI). No need to go buy real gear or buy a full simulator to start learning the CLI. Just install it from the companion website.
- **CCNA Simulator:** If you are looking for more hands-on practice, you might want to consider purchasing the CCNA Network Simulator. You can purchase a copy of this software from Pearson at http://pearsonitcertification.com/networksimulator or other retail outlets. To help you with your studies, Pearson has created a mapping guide that maps each of the labs in the simulator to the specific sections in each volume of the CCNA Cert Guide. You can get this mapping guide free on the Extras tab on the book product page: www.ciscopress.com/title/9780138214951.
- **Author's website and blogs:** The author maintains a website that hosts tools and links useful when studying for CCNA. In particular, the site has a large number of free lab exercises about CCNA content, additional sample questions, and other exercises. Additionally, the site indexes all content so you can study based on the book chapters and parts. To find it, navigate to www.certskills.com. Additionally, look for CCNA activities and lectures at his YouTube channel (www.youtube.com/@networkupskill).

## Book Organization, Chapters, and Appendixes

This book contains 24 chapters about CCNA topics organized into seven parts. The core chapters cover the following topics:

- **Part I: Wireless LANs**
  - **Chapter 1, "Fundamentals of Wireless Networks,"** includes the foundational concepts of wireless 802.11 LANs, including wireless topologies and basic wireless radio communications protocols.

- **Chapter 2, "Analyzing Cisco Wireless Architectures,"** turns your attention to the questions related to the systematic and architectural issues surrounding how to build wireless LANs and explains the primary options available for use.
- **Chapter 3, "Securing Wireless Networks,"** explains the unique security challenges that exist in a wireless LAN and the protocols and standards used to prevent different kinds of attacks.
- **Chapter 4, "Building a Wireless LAN,"** shows how to configure and secure a wireless LAN using a Wireless LAN Controller (WLC).

- **Part II: IP Access Control Lists**
  - **Chapter 5, "Introduction to TCP/IP Transport and Applications,"** completes most of the detailed discussion of the upper two layers of the TCP/IP model (transport and application), focusing on TCP and applications.
  - **Chapter 6, "Basic IPv4 Access Control Lists,"** examines how standard IP ACLs can filter packets based on the source IP address so that a router will not forward the packet.
  - **Chapter 7, "Named and Extended IP ACLs,"** examines both named and numbered ACLs, and both standard and extended IP ACLs.
  - **Chapter 8, "Applied IP ACLs,"** shows how to match overhead protocols like DNS, DHCP, and OSPF with ACLs.
- **Part III: Security Services**
  - **Chapter 9, "Security Architectures,"** discusses a wide range of fundamental concepts in network security.
  - **Chapter 10, "Securing Network Devices,"** shows how to secure the router and switch CLI and introduces the concepts behind firewalls and intrusion prevention systems (IPSs).
  - **Chapter 11, "Implementing Switch Port Security,"** explains the concepts as well as how to configure and verify switch port security, a switch feature that does basic MAC-based monitoring of the devices that send data into a switch.
  - **Chapter 12, "DHCP Snooping and ARP Inspection,"** shows how to implement two related switch security features, with one focusing on reacting to suspicious DHCP messages and the other reacting to suspicious ARP messages.
- **Part IV: IP Services**
  - **Chapter 13, "Device Management Protocols,"** discusses the concepts and configuration of some common network management tools: syslog, NTP, CDP, and LLDP.
  - **Chapter 14, "Network Address Translation,"** works through the complete concept, configuration, verification, and troubleshooting sequence for the router NAT feature, including how it helps conserve public IPv4 addresses.

- **Chapter 15, "Quality of Service (QoS),"** discusses a wide variety of concepts all related to the broad topic of QoS.
- **Chapter 16, "First Hop Redundancy Protocols,"** explains the purpose, functions, and concepts of FHRPs, including HSRP, VRRP, and GLBP.
- **Chapter 17, "SNMP, FTP, and TFTP,"** discusses three protocols often used for managing network devices: SNMP, TFTP, and FTP.

- **Part V: Network Architecture**
  - **Chapter 18, "LAN Architecture,"** examines various ways to design Ethernet LANs, discussing the pros and cons, and explains common design terminology, including Power over Ethernet (PoE).
  - **Chapter 19, "WAN Architecture,"** discusses the concepts behind three WAN alternatives: Metro Ethernet, MPLS VPNs, and Internet VPNs.
  - **Chapter 20, "Cloud Architecture,"** explains the basic concepts and then generally discusses the impact that cloud computing has on a typical enterprise network, including the foundational concepts of server virtualization.
- **Part VI: Network Automation**
  - **Chapter 21, "Introduction to Controller-Based Networking,"** discusses many concepts and terms related to how Software-Defined Networking (SDN) and network programmability are impacting typical enterprise networks.
  - **Chapter 22, "Cisco Software-Defined Access (Cisco SD-Access),"** discusses Cisco's Software-Defined Networking (SDN) offering for the enterprise, including the Cisco Catalyst Center (formerly Cisco DNA Center) controller.
  - **Chapter 23, "Understanding REST and JSON,"** explains the foundational concepts of REST APIs, data structures, and how JSON can be useful for exchanging data using APIs.
  - **Chapter 24, "Understanding Ansible and Terraform,"** discusses the need for configuration management software and introduces the basics of each of these configuration management tools.
- **Part VII: Exam Updates and Final Review**
  - **Chapter 25, "*CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, Exam Updates,"** is a place for the author to add book content mid-edition. Always check online for the latest PDF version of this appendix; the appendix lists download instructions.
  - **Chapter 26, "Final Review,"** suggests a plan for final preparation after you have finished the core parts of the book, in particular explaining the many study options available in the book.
- **Part VIII: Print Appendixes**
  - **Appendix A, "Numeric Reference Tables,"** lists several tables of numeric information, including a binary-to-decimal conversion table and a list of powers of 2.

- **Appendix B, "Exam Topics Cross-Reference,"** provides some tables to help you find where each exam objective is covered in the book.
- **Appendix C, "Answers to the 'Do I Know This Already?' Quizzes,"** includes the explanations to all the "Do I Know This Already" quizzes.
- The **Glossary** contains definitions for many of the terms used in the book, including the terms listed in the "Key Terms You Should Know" sections at the conclusion of the chapters.

- **Part IX: Online Appendixes**
  - **Appendix D, "Topics from Previous Editions"**
  - **Appendix E, "Practice for Chapter 6: Basic IPv4 Access Control Lists"**
  - **Appendix F, "Study Planner,"** is a spreadsheet with major study milestones, where you can track your progress through your study.

## About Building Hands-On Skills

To do well on the CCNA exam, you need skills in using Cisco routers and switches, specifically the Cisco command-line interface (CLI). The Cisco CLI is a text-based command-and-response user interface; you type a command, and the device (a router or switch) displays messages in response.

For the exam, CLI skills help you in a couple of ways. First, lab questions require CLI skills. Each lab question can take 7–8 minutes if you know the topic, so poor CLI skills can cost several minutes per lab question. Additionally, any question type can ask about CLI commands, so the more comfortable you are remembering commands, parameters, and what they do, the more points you will pick up on the exam.

This next section walks through the options of what is included in the book, with a brief description of lab options outside the book.

### Config Lab Exercises

I created some lab exercises called Config Labs and put them on my blog. Each Config Lab details a straightforward lab exercise. It begins with a scenario, a topology, and an existing configuration. You choose the configuration to add to each device to meet the goals of the scenario.

To make the labs accessible to all, the blog has no login requirements and no cost. You can do each lab just by viewing the page, reading, and writing your answer on paper or typing it in an editor. Optionally, you can attempt most labs in the Cisco Packet Tracer Simulator. In either case, the Config Lab page lists the intended answer, so you can check your work.

To find the Config Labs, first go to www.certskills.com. Navigate from the top menus for "Labs." Alternatively, use the advanced search link, from which you can combine search parameters to choose a book chapter or part, and to search for Config Lab posts.

Note that the blog organizes these Config Lab posts by book chapter, so you can easily use them at both Chapter Review and Part Review. See the "Your Study Plan" element that follows the Introduction for more details about those review sections.

## A Quick Start with Pearson Network Simulator Lite

The decision of how to get hands-on skills can be a little scary at first. The good news: You have a free and simple first step to experience the CLI—install a desktop simulator app called Pearson Network Simulator Lite (or NetSim Lite) that comes with this book.

Pearson builds a CCNA Simulator app designed to help you learn most of the CCNA configure and verify exam topics. They also make a free Lite version of the simulator, included with this book. The Lite version gives you the means to experience the Cisco CLI just after a 5–10-minute installation process. No need to go buy real gear or buy a full simulator to start learning the CLI. Just install the Sim Lite from the companion website.

This latest version of NetSim Lite for Volume 2 (which differs from the NetSim Lite that comes with Volume 1) includes labs about IP ACLs.

## The Pearson Network Simulator

The Config Labs and the Pearson Network Simulator Lite both fill specific needs, and they both come with the book. However, you need more than those two tools.

The single best option for lab work to do along with this book is the paid version of the Pearson Network Simulator. This simulator product simulates Cisco routers and switches so that you can learn for CCNA certification. But more importantly, it focuses on learning for the exam by providing a large number of useful lab exercises. Reader surveys tell us that those people who use the simulator along with the book love the learning process and rave about how the book and simulator work well together.

Of course, you need to make a decision for yourself and consider all the options. Thankfully, you can get a great idea of how the full simulator product works by using the Pearson Network Simulator Lite product included with the book. Both have the same base code, same user interface, and same types of labs. Try the Lite version to decide if you want to buy the full product.

On a practical note, when you want to do labs when reading a chapter or doing Part Review, the Simulator organizes the labs to match the book. Just look for the Sort by Chapter tab in the Simulator's user interface.

At the time this book was published, Pearson had no plan to update its CCNA Simulator product to a new version, as the current edition covers the latest exam topics. A software update will be issued that maps the labs to the organization of the new Cert Guide chapter structure by the summer of 2024.

## More Lab Options

Many other lab options exist. For instance, you can use real Cisco routers and switches. You can buy them, new or used, or borrow them at work. For example, you can buy routers and switches that are useful for CCNA learning, but are two or three product generations old. You can also find sites from which you can rent time on real devices or virtual devices.

Cisco also makes a free simulator that works very well as a learning tool: Cisco Packet Tracer. Unlike the Pearson Network Simulator, it does not include lab exercises that direct you as to how to go about learning each topic. However, you can usually find lab exercises that rely on Packet Tracer, like the Config Labs at my blog. If interested in more information about Packet Tracer, check out www.certskills.com/ptinstall.

Cisco offers a virtualization product that lets you run router and switch operating system (OS) images in a virtual environment on your PC. This tool, the Cisco Modeling Labs–Personal Edition (CML PE), lets you create a lab topology, start the operating system for each device, and connect to the CLI of these real router and switch OS images. There is a fee, and you may need a PC hardware upgrade to use it effectively. Check out www.cisco.com/go/cml for more information, and inquire for more information at the Cisco Learning Network's CML community (learningnetwork.cisco.com).

The next two options work somewhat like CML PE, but with free software but no Cisco operating systems supplied. GNS3 (gns3.com) and EVE-NG (eve-ng.net) support creating topologies of virtual routers and switches that run real Cisco operating systems. Both have free options. However, both require that you provide the OS images. Also, as with CML PE, you may need to buy a small server or at least upgrade your personal computer to run more than a few routers and switches in a lab topology.

This book does not tell you what option to use, but you should plan on getting some hands-on practice somehow. For people starting with CCNA, many use some simulator like Pearson Sim Lite and the free Cisco Packet Tracer simulator. If you go far in your Cisco certification journey, you will likely try at least one of the virtualization options and also use real gear. The important thing to know is that most people need to practice using the Cisco CLI to be ready to pass these exams.

## For More Information

If you have any comments about the book, submit them via www.ciscopress.com. Just go to the website, select **Contact Us**, and type your message.

Cisco might make changes that affect the CCNA certification from time to time. You should always check www.cisco.com/go/ccna for the latest details.

The *CCNA 200-301 Official Cert Guide*, *Volume 2*, Second Edition, helps you attain CCNA certification. This is the CCNA certification book from the only Cisco-authorized publisher. We at Cisco Press believe that this book certainly can help you achieve CCNA certification, but the real work is up to you! I trust that your time will be well spent.

## Figure Credits

Figure 17.10: FileZilla

Figure 20.11: Amazon Web Services, Inc.

Figure 23.8: Postman, Inc.

Figure 24.3: GitHub, Inc.

Figures 26.3, 26.4: Pearson Education, Inc.

*This page intentionally left blank*



*CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, moves through a wide variety of topic areas, with major transitions occurring with each book part. The book works through wireless LANs, network security, network services, network architecture, and finally, network automation.

In this first part of this volume, we turn our attention to the LAN…not to wired Ethernet LANs, but to IEEE 802.11 wireless LANs—in other words, Wi-Fi. The four chapters in this part of the book lay down the foundations of how wireless LANs work and then show how to implement wireless LANs using Cisco devices.

Building wireless LANs requires some thought because the endpoints that use the LAN do not sit in one place and connect via a known cable and known switch port. To explain those details, Chapter 1 begins with the basics of how a wireless client can connect to the wireless network through a wireless access point (AP). After you learn the foundations in Chapter 1, Chapter 2 takes an architectural view of wireless LANs to discuss how you might build a wireless LAN for an enterprise, which requires much different thinking than, for instance, building a wireless LAN for your home.

Chapter 3 completes the three concepts-focused wireless LAN chapters by working through the alphabet soup that is wireless LAN security. The fact that wireless LAN clients come and go means that the LAN may be under constant attack as an easy place for attackers to gain access to the network, so wireless LANs must use effective security. Finally, Chapter 4 closes by showing how to configure an enterprise wireless LAN using Cisco APs and the Cisco Wireless LAN Controller (WLC) from the WLC's graphical interface.

# Part I

## Wireless LANs

**Chapter 1:** Fundamentals of Wireless Networks

**Chapter 2:** Analyzing Cisco Wireless Architectures

**Chapter 3:** Securing Wireless Networks

**Chapter 4:** Building a Wireless LAN

**Part I Review**



CHAPTER 1

# Fundamentals of Wireless Networks

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.d Access Points**

**1.11 Describe wireless principles**

**1.11.a Non-overlapping Wi-Fi Channels**

**1.11.b SSID**

**1.11.c RF**

Wireless communication usually involves a data exchange between two devices. A wireless LAN goes even further; many devices can participate in sharing the medium for data exchanges. Wireless LANs must transmit a signal over radio frequencies (RF) to move data from one device to another. Transmitters and receivers can be fixed in consistent locations, or they can be mobile and free to move around. This chapter explains the topologies that can be used to control access to the wireless medium and provide data exchange between devices.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 1-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Comparing Wired and Wireless Networks | 1 |
| Wireless LAN Topologies | 2–4 |
| Other Wireless Topologies | 5, 6 |
| Wireless Bands and Channels | 7, 8 |

**1.** Wired Ethernet and Wi-Fi are based on which two IEEE standards, respectively?

**a.** 802.1, 802.3

**b.** 802.3, 802.1

**c.** 802.3, 802.11

**d.** 802.11, 802.3

2. Devices using a wireless LAN must operate in which one of the following modes?
   a. Round-robin access
   b. Half duplex
   c. Full duplex
   d. None of these answers are correct
3. An access point is set up to offer wireless coverage in an office. Which one of the following is the correct 802.11 term for the resulting standalone network?
   a. BSA
   b. BSD
   c. BSS
   d. IBSS
4. Which one of the following is used to uniquely identify an AP and the basic service set it maintains with its associated wireless clients?
   a. SSID
   b. BSSID
   c. Ethernet MAC address
   d. Radio MAC address
5. Which one of the following can be used to provide wireless connectivity to a nonwireless device?
   a. Wireless repeater
   b. Workgroup bridge
   c. Transparent bridge
   d. Adaptive bridge
6. Which one of the following is not needed in a Cisco outdoor mesh network?
   a. A BSS function
   b. Ethernet cabling to each AP
   c. A workgroup bridge
   d. A backhaul network
7. Which of the following are frequency bands commonly used for Wi-Fi? (Choose two answers.)
   a. 2.5 KHz
   b. 2.5 MHz
   c. 5 MHz
   d. 2.5 GHz
   e. 5 GHz
8. Which of the following are considered to be nonoverlapping channels? (Choose two answers.)
   a. Channels 1, 2, and 3 in the 2.4-GHz band
   b. Channels 1, 5, and 10 in the 2.4-GHz band
   c. Channels 1, 6, and 11 in the 2.4-GHz band
   d. Channels 40, 44, and 48 in the 5-GHz band

# Foundation Topics

## Comparing Wired and Wireless Networks

In a wired network, any two devices that need to communicate with each other must be connected by a wire. (That was obvious!) The "wire" might contain strands of metal or fiber-optic material that run continuously from one end to the other. Data that passes over the wire is bounded by the physical properties of the wire. In fact, the IEEE 802.3 set of standards defines strict guidelines for the Ethernet wire itself, in addition to how devices may connect, send, and receive data over the wire.

Wired connections have been engineered with tight constraints and have few variables that might prevent successful communication. Even the type and size of the wire strands, the number of twists the strands must make around each other over a distance, and the maximum length of the wire must adhere to the standard.

Therefore, a wired network is essentially a bounded medium; data must travel over whatever path the wire or cable takes between two devices. If the cable goes around a corner or lies in a coil, the electrical signals used to carry the data must also go around a corner or around a coil. Because only two devices may connect to a wire, only those two devices may send or transmit data. Even better: the two devices may transmit data to each other simultaneously because they each have a private, direct path to each other—assuming there are enough wires dedicated to each direction within the cable.

Wired networks also have some shortcomings. When a device is connected by a wire, it cannot move around very easily or very far. Before a device can connect to a wired network, it must have a connector that is compatible with the one on the end of the wire. As devices get smaller and more mobile, it just is not practical to connect them to a wire.

As its name implies, a wireless network removes the need to be tethered to a wire or cable. Convenience and mobility become paramount, enabling users to move around at will while staying connected to the network. A user can (and often does) bring along many different wireless devices that can all connect to the network easily and seamlessly.

Wireless data must travel through free space, without the constraints and protection of a wire. In the free space environment, many variables can affect the data and its delivery. To minimize the variables, wireless engineering efforts must focus on two things:

- Wireless devices must adhere to a common standard (IEEE 802.11).
- Wireless coverage must exist in the area where devices are expected to use it.

---

Answers to the "Do I Know This Already?" quiz:

**1** C **2** B **3** C **4** B **5** B **6** B **7** D, E **8** C, D

As you study for the CCNA 200-301 exam, keep in mind that the exam is geared more toward a functional view of wireless technology. More detailed topics like RF characteristics, antenna performance, and so on are reserved for the Implementing Cisco Enterprise Network Core Technologies ENCOR 300-401 exam.

## Wireless LAN Topologies

Wireless communication takes place over free space through the use of RF signals. The signals are sent by one device, the transmitter, to another device, the receiver. As Figure 1-1 shows, the transmitter can contact the receiver at any and all times, as long as both devices are tuned to the same frequency (or channel) and use the same scheme to carry the data between them. That all sounds simple, except that it is not really practical.

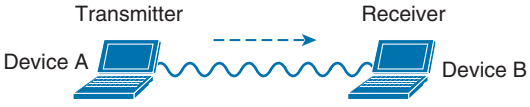



**Figure 1-1** *Unidirectional Communication*

To fully leverage wireless communication, data should travel in *both* directions, as shown in Figure 1-2. Sometimes Device A needs to send data to Device B, while Device B would like to take a turn to send at other times.

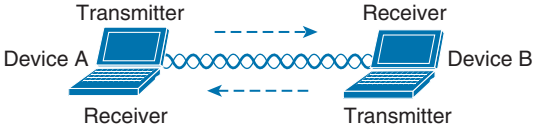



**Figure 1-2** *Bidirectional Communication*

Because the two devices are using the same channel, two phrases in the preceding sentence become vitally important: *take a turn* and *send at other times*. With wireless communication, if multiple signals are received at the same time, they can interfere with each other. The likelihood of interference increases as the number of wireless devices grows. For example, Figure 1-3 shows four devices tuned to the same channel and what might happen if some or all of them transmit at the same time.

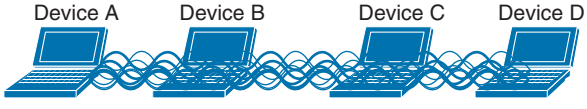



**Figure 1-3** *Interference from Simultaneous Transmissions*

All this talk about waiting turns and avoiding interference might remind you of a traditional (nonswitched) Ethernet LAN, where multiple hosts can connect to a shared media and share a common bandwidth. To use the media effectively, all the hosts must operate in half-duplex mode so that they try to avoid colliding with other transmissions already in progress. The side effect is that no host can transmit and receive at the same time on a shared medium.

A wireless LAN is similar. Because multiple hosts can share the same channel, they also share the "airtime" or access to that channel at any given time. Therefore, to keep everything clean, only one device should transmit at any given time. To contend for use of the channel, devices based on the 802.11 standard have to determine whether the channel is clear and available before transmitting anything.

**NOTE** IEEE 802.11 WLANs are always half duplex because transmissions between stations use the same frequency or channel. Only one station can transmit at any time; otherwise, collisions occur. To achieve full-duplex mode, one station's transmission would have to occur on one frequency while it receives over a different frequency—much like full-duplex Ethernet links work. Although this operation is certainly possible and practical, the 802.11 standard does not permit full-duplex operation. Some amendments to the standard do provide a means for multiple devices to transmit on the same channel at the same time, but this is beyond the scope of this book.

At the most basic level, there is no inherent organization to a wireless medium or any inherent control over the number of devices that can transmit and receive frames. Any device that has a wireless network adapter can power up at any time and try to communicate. At a minimum, a wireless network should have a way to make sure that every device using a channel can support a common set of parameters. Beyond that, there should be a way to control which devices (and users) are allowed to use the wireless medium and the methods that are used to secure the wireless transmissions.

## Basic Service Set

The solution is to make every wireless service area a closed group of mobile devices that forms around a fixed device; before a device can participate, it must advertise its capabilities and then be granted permission to join. The 802.11 standard calls this a **basic service set (BSS)**. At the heart of every BSS is a wireless **access point (AP)**, as shown in Figure 1-4. The AP operates in **infrastructure mode**, which means it offers the services that are necessary to form the infrastructure of a wireless network. The AP also establishes its BSS over a single wireless channel. The AP and the members of the BSS must all use the same channel to communicate properly.

Because the operation of a BSS hinges on the AP, the BSS is bounded by the area where the AP's signal is usable. This is known as the *basic service area* (BSA) or **cell**. In Figure 1-4, the cell is shown as a simple shaded circular area that centers around the AP itself. Cells can have other shapes too, depending on the antenna that is connected to the AP and on the physical surroundings that might affect the AP's signals.

The AP serves as a single point of contact for every device that wants to use the BSS. It advertises the existence of the BSS by periodically transmitting an 802.11 **beacon** frame so that devices can find it and try to join. Beacon frames contain a text string called a **Service Set Identifier (SSID)**, which identifies the BSS. Beacons are normally broadcast about ten times per second (100-ms intervals), and also contain information about the data rates to be used within the BSS and any vendor-specific information needed. If the AP supports multiple SSIDs, a different beacon is broadcast for each SSID.

Recall that wired Ethernet devices each have a unique MAC address to send frames from a source to a destination over a Layer 2 network. Wireless devices must also have unique MAC addresses to send wireless frames at Layer 2 over the air. When the AP transmits wireless frames, it uses a **Basic Service Set Identifier (BSSID)** that is based on the AP's own radio MAC address.

Key Topic

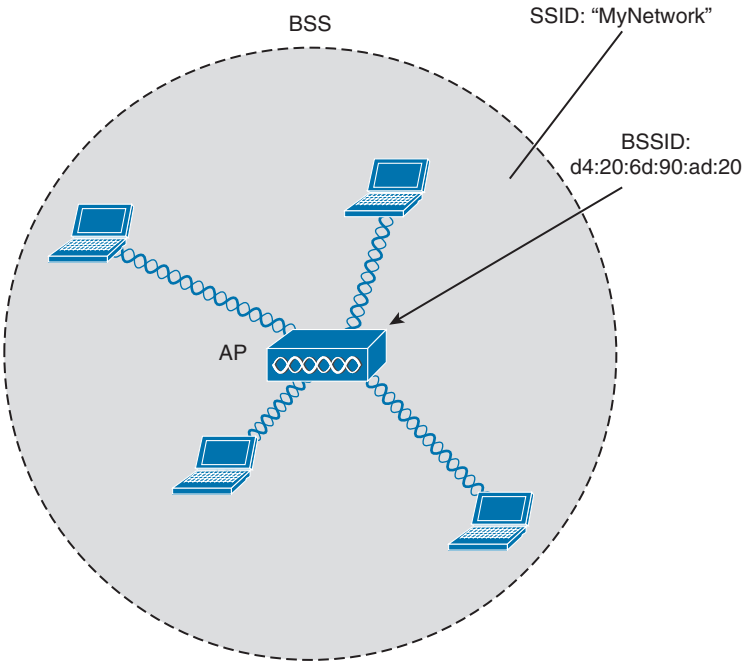



**Figure 1-4** *802.11 Basic Service Set*

**TIP** Think of the BSSID as a machine-readable name tag that uniquely identifies the BSS ambassador (the AP), and the SSID as a nonunique, human-readable name tag that identifies the wireless service.

A wireless device can learn about BSSs within range by listening to the beacons that are received. This operation is known as **passive scanning**. Devices can also actively discover SSIDs that are within range by transmitting 802.11 **probe request** frames. APs can answer the request by sending a probe response frame that contains most of the beacon information.

Membership with the BSS is called an *association*. A wireless device must send an 802.11 **association request** frame to the AP, and the AP must either grant or deny the request by sending an *association response* frame. Once associated, a device becomes a client, or an 802.11 **station (STA)**, of the BSS.

What then? As long as a wireless client remains associated with a BSS, most communications to and from the client must pass *through* the AP, as indicated in Figure 1-5. When the BSSID is used as a source or destination address, data frames can be relayed to or from the AP.

You might be wondering why all client traffic has to traverse the AP at all. Why can two clients not simply transmit data frames directly to each other and bypass the middleman? If clients are allowed to communicate directly, then the whole idea of organizing and managing a BSS is moot. By sending data through the AP first, the BSS remains stable and under control.

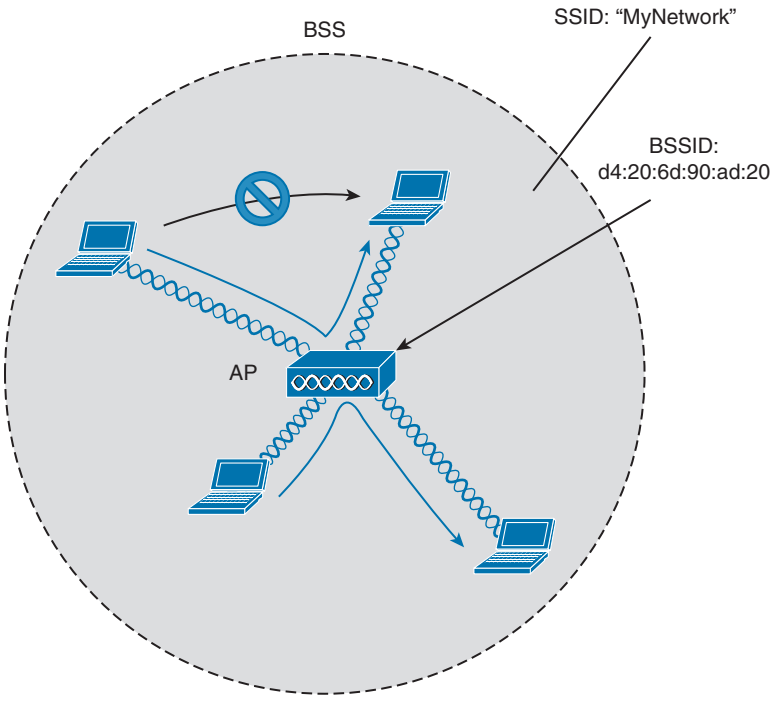



**Figure 1-5** *Traffic Flows Within a BSS*

**TIP** Even though data frames are meant to pass through an AP, keep in mind that other devices in the same general area that are listening on the same channel can overhear the transmissions. After all, wireless frames are not contained within a wire that connects a device to an AP. Instead, the frames are freely available over the air to anyone that is within range to receive them. If the frames are unencrypted, then anyone may inspect their contents. Only the BSSID value contained within the frames indicates that the intended sender or recipient is the AP.

## Distribution System

Notice that a BSS involves a single AP and no explicit connection into a regular Ethernet network. In that setting, the AP and its associated clients make up a standalone network. But the AP's role at the center of the BSS does not just stop with managing the BSS; sooner or later, wireless clients will need to communicate with other devices that are not members of the BSS. Fortunately, an AP can also uplink into an Ethernet network because it has both wireless and wired capabilities. The 802.11 standard refers to the upstream wired Ethernet as the **distribution system (DS)** for the wireless BSS, as shown in Figure 1-6.

You can think of an AP as a translational bridge, where frames from two dissimilar media (802.11 wireless and 802.3 wired) are translated and then bridged at Layer 2. In simple terms, the AP is in charge of mapping a virtual local-area network (VLAN) to an SSID. In Figure 1-6, the AP maps VLAN 10 to the wireless LAN using SSID "MyNetwork." Clients associated with the "MyNetwork" SSID will appear to be connected to VLAN 10.

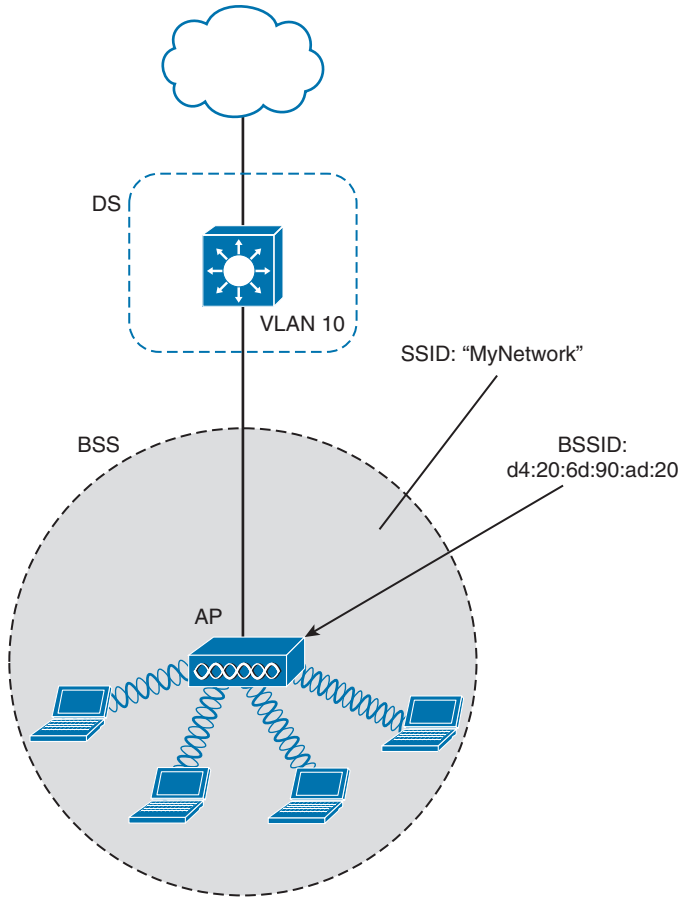



**Figure 1-6** *Distribution System Supporting a BSS*

This concept can be extended so that multiple VLANs are mapped to multiple SSIDs. To do this, the AP must be connected to the switch by a trunk link that carries the VLANs. In Figure 1-7, VLANs 10, 20, and 30 are trunked to the AP over the DS. The AP uses the 802.1Q tag to map the VLAN numbers to the appropriate SSIDs. For example, VLAN 10 is mapped to SSID "MyNetwork," VLAN 20 is mapped to SSID "YourNetwork," and VLAN 30 to SSID "Guest."

In effect, when an AP uses multiple SSIDs, it is trunking VLANs over the air, and over the same channel, to wireless clients. The clients must use the appropriate SSID that has been mapped to the respective VLAN when the AP was configured. The AP then appears as multiple logical APs—one per BSS—with a unique BSSID for each. With Cisco APs, this is usually accomplished by incrementing the last digit of the radio's MAC address for each SSID.

Key Topi

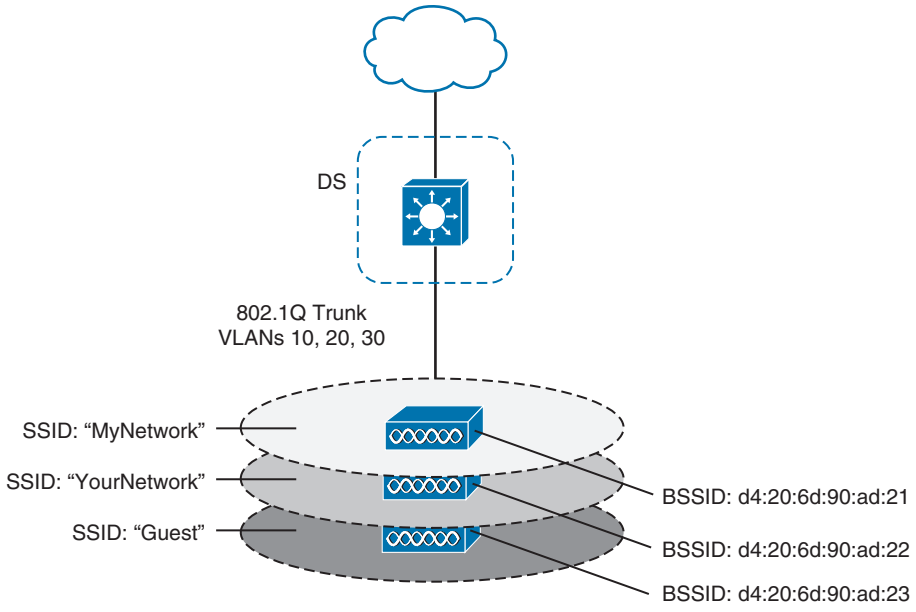



**Figure 1-7** *Supporting Multiple SSIDs on One AP*

Even though an AP can advertise and support multiple logical wireless networks, each of the SSIDs covers the same geographic area. The reason is that the AP uses the same transmitter, receiver, antennas, and channel for every SSID that it supports. Beware of one misconception though: multiple SSIDs can give an illusion of scale. Even though wireless clients can be distributed across many SSIDs, all of those clients must share the same AP's hardware and must contend for airtime on the same channel.

## Extended Service Set

Normally, one AP cannot cover the entire area where clients might be located. For example, you might need wireless coverage throughout an entire floor of a business, hotel, hospital, or other large building. To cover more area than a single AP's cell can cover, you simply need to add more APs and spread them out geographically.

When APs are placed at different geographic locations, they can all be interconnected by a switched infrastructure. The 802.11 standard calls this an **extended service set (ESS)**, as shown in Figure 1-8.

The idea is to make multiple APs cooperate so that the wireless service is consistent and seamless from the client's perspective. Ideally, any SSIDs that are defined on one AP should be defined on all the APs in an ESS; otherwise, it would be very cumbersome and inconvenient for a client to be reconfigured each time it moves into a different AP's cell.

Notice that each cell in Figure 1-8 has a unique BSSID, but both cells share one common SSID. Regardless of a client's location within the ESS, the SSID will remain the same, but the client can always distinguish one AP from another. Having a consistent SSID through the ESS is called an **Extended Service Set Identifier (ESSID)**.

Key Top



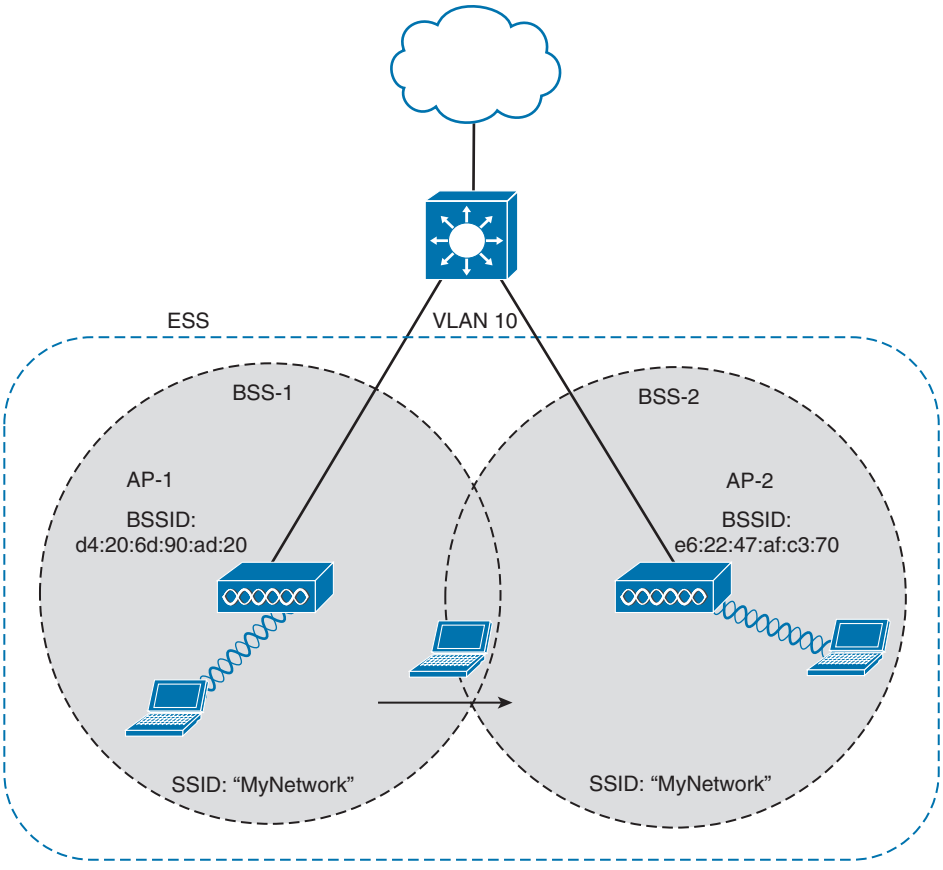



**Figure 1-8** *Scaling Wireless Coverage with an 802.11 Extended Service Set*

In an ESS, a wireless client can associate with only one AP while it is physically located near that AP. If the client later moves to a different location, it can move its association to a different nearby AP by sending a **reassociation request** frame to the new AP. That AP sends a reassociation response frame in return.

Passing from one AP to another is called **roaming**. Keep in mind that each AP offers its own BSS on its own channel, to prevent interference between the APs. As a client device roams from one AP to another, it must scan the available channels to find a new AP (and BSS) to roam toward. In effect, the client is roaming from BSS to BSS, and from channel to channel.

## Independent Basic Service Set

Usually, a wireless network leverages APs for organization, control, and scalability. Sometimes that is not possible or convenient in an impromptu situation. For example, two people who want to exchange electronic documents at a meeting might not be able to find a BSS available or might want to avoid having to authenticate to a production network. In addition,

many personal printers have the capability to print documents wirelessly, without relying on a regular BSS or AP.

The 802.11 standard allows two or more wireless clients to communicate directly with each other, with no other means of network connectivity. This is known as an **ad hoc wireless network**, or an **independent basic service set (IBSS)**, as shown in Figure 1-9. For this connection to work, one of the devices must take the lead and begin advertising a network name and the necessary radio parameters, much like an AP would do. Any other device can then join as needed. IBSSs are meant to be organized in an impromptu, distributed fashion; therefore, they do not scale well beyond eight to ten devices.

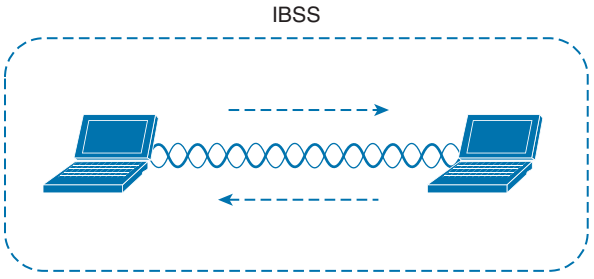



**Figure 1-9** *802.11 Independent Basic Service Set*

## Other Wireless Topologies

Wireless APs can be configured to operate in noninfrastructure modes when a normal BSS cannot provide the functionality that is needed. The following sections cover the most common modes.

### Repeater

Normally, each AP in a wireless network has a wired connection back to the DS or switched infrastructure. To extend wireless coverage beyond a normal AP's cell footprint, additional APs and their wired connections can be added. In some scenarios, it is not possible to run a wired connection to a new AP because the cable distance is too great to support Ethernet communication.

In that case, you can add an additional AP that is configured for *repeater mode*. A wireless **repeater** takes the signal it receives and repeats or retransmits it in a new cell area around the repeater. The idea is to move the repeater out away from the AP so that it is still within range of both the AP and the distant client, as shown in Figure 1-10.

If the repeater has a single transmitter and receiver, it must operate on the same channel that the AP is using. That can create the possibility that the AP's signal will be received and retransmitted by the repeater, only to be received again by the AP—halving the effective throughput because the channel will be kept busy twice as long as before. As a remedy, some repeaters can use two transmitters and receivers to keep the original and repeated signals isolated on different channels. One transmitter and receiver pair is dedicated to signals in the AP's cell, while the other pair is dedicated to signals in the repeater's own cell.

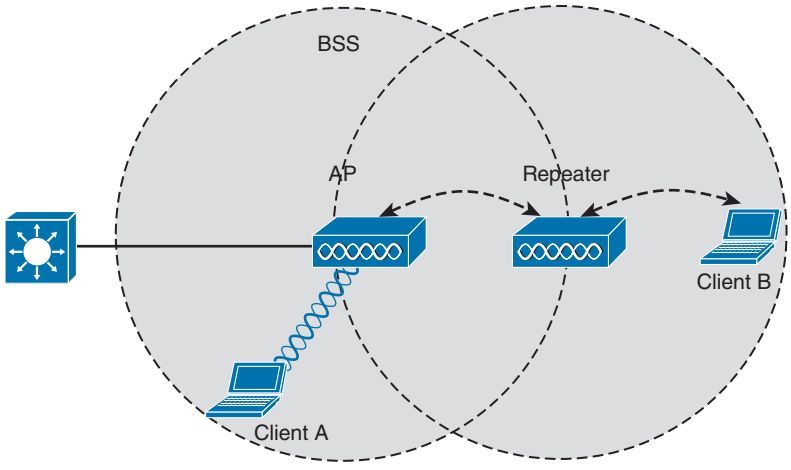



**Figure 1-10** *Extending the Range of an AP with a Wireless Repeater*

## Workgroup Bridge

Suppose you have a device that supports a wired Ethernet link but is not capable of having a wireless connection. For example, some mobile medical devices might be designed with only a wired connection. While it is possible to plug the device into an Ethernet connection when needed, a wireless connection would be much more practical. You can use a **workgroup bridge (WGB)** to connect the device's wired network adapter to a wireless network.

Rather than providing a BSS for wireless service, a WGB becomes a wireless client of a BSS. In effect, the WGB acts as an external wireless network adapter for a device that has none. In Figure 1-11, an AP provides a BSS; Client A is a regular wireless client, while Client B is associated with the AP through a WGB.

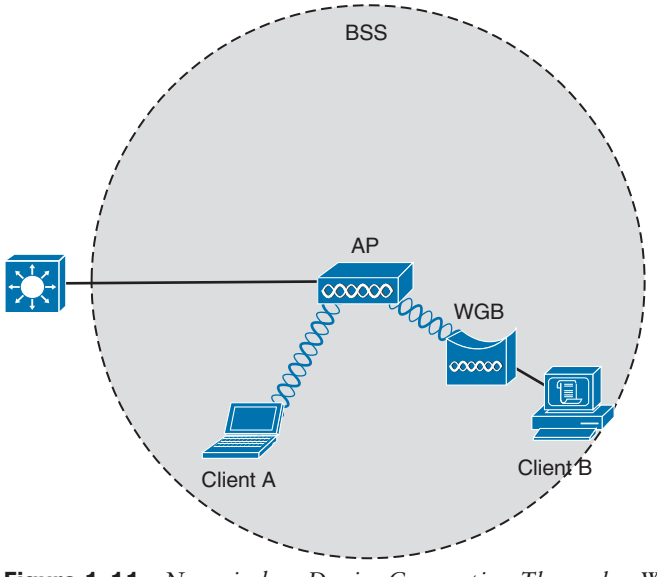



**Figure 1-11** *Nonwireless Device Connecting Through a Workgroup Bridge*

You might encounter two types of *workgroup bridges*:

- **Universal workgroup bridge (uWGB):** A single wired device can be bridged to a wireless network.
- **Workgroup bridge (WGB):** A Cisco-proprietary implementation that allows multiple wired devices to be bridged to a wireless network.

## Outdoor Bridge

An AP can be configured to act as a bridge to form a single wireless link from one LAN to another over a long distance. Outdoor bridged links are commonly used for connectivity between buildings or between cities.

If the LANs at two locations need to be bridged, a **point-to-point bridged** link can be used. One AP configured in bridge mode is needed on each end of the wireless link. Special-purpose antennas are normally used with the bridges to focus their signals in one direction—toward the antenna of the AP at the far end of the link. This configuration maximizes the link distance, as shown in Figure 1-12.

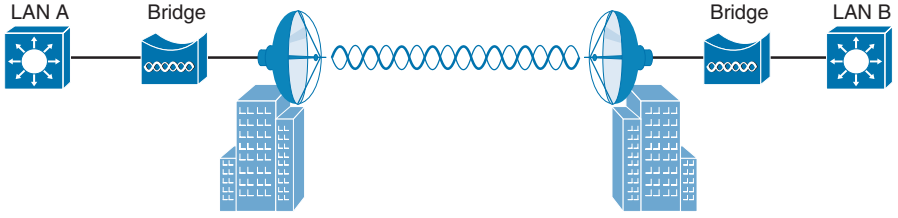



**Figure 1-12** *Point-to-Point Outdoor Bridge*

Sometimes the LANs at multiple sites need to be bridged together. A point-to-multipoint bridged link allows a central site to be bridged to several other sites. The central site bridge is connected to an omnidirectional antenna, such that its signal is transmitted equally in all directions so that it can reach the other sites simultaneously. The bridges at each of the other sites can be connected to a directional antenna aimed at the central site. Figure 1-13 shows the point-to-multipoint scenario.

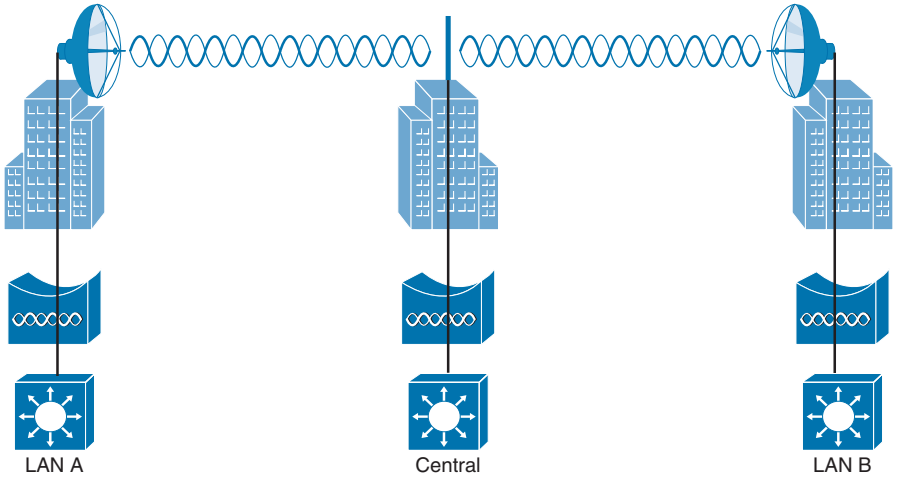



**Figure 1-13** *Point-to-Multipoint Outdoor Bridge*

### Mesh Network

To provide wireless coverage over a very large area, it is not always practical to run Ethernet cabling to every AP that would be needed. Instead, you could use multiple APs configured in mesh mode. In a mesh topology, wireless traffic is bridged from AP to AP, in a daisy-chain fashion, using another wireless channel.

Mesh APs can leverage dual radios—one using a channel in one range of frequencies and one a different range. Each mesh AP usually maintains a BSS on one channel, with which wireless clients can associate. Client traffic is then usually bridged from AP to AP over other channels as a backhaul network. At the edge of the **mesh network**, the backhaul traffic is bridged to the wired LAN infrastructure. Figure 1-14 shows a typical mesh network. With Cisco APs, you can build a mesh network indoors or outdoors. The mesh network runs its own dynamic routing protocol to work out the best path for backhaul traffic to take across the mesh APs.

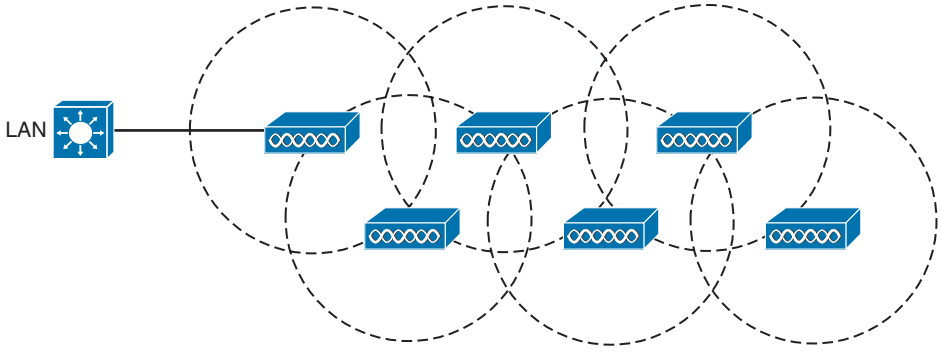



**Figure 1-14** *Typical Wireless Mesh Network*

## Wireless Bands and Channels

The electromagnetic waves involved in a wireless signal can be measured and described in several ways. One fundamental property is the frequency of the wave, or the number of times the signal oscillates or makes one complete positive and negative cycle in one second. One complete cycle within one second is known as one hertz (Hz). Wi-Fi signals use frequencies that oscillate more than one billion times per second, or one gigahertz (GHz).

One of the three main frequency ranges used for wireless LAN communication lies between 2.400 and 2.4835 GHz. This is usually called the *2.4-GHz* **band**, even though it does not encompass the entire range between 2.4 and 2.5 GHz. It is much more convenient to refer to the band name instead of the specific range of frequencies included. Another wireless LAN range is usually called the *5-GHz band* because it lies between 5.150 and 5.825 GHz. The 5-GHz band actually contains four smaller separate and distinct bands, with the possibility of more small bands being added in the future. The 6-GHz band lies between 5.925 and 7.125 GHz. It is broken up into four smaller bands too.

To keep everything orderly and compatible, bands are usually divided into a number of distinct **channels**. Each channel is known by a channel number and is assigned to a specific frequency. As long as the channels are defined by a national or international standards body, they can be used consistently in all locations. Figures 1-15 and 1-16 show the channel layout for the 2.4- and 5-GHz bands, respectively.

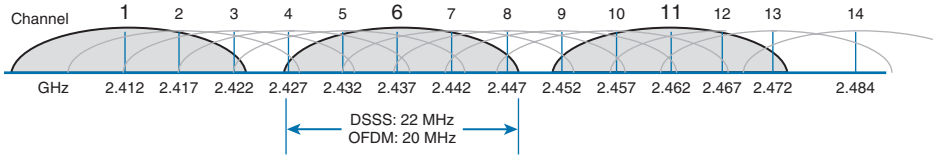



**Figure 1-15** *Channel Layout in the 2.4-GHz Band*

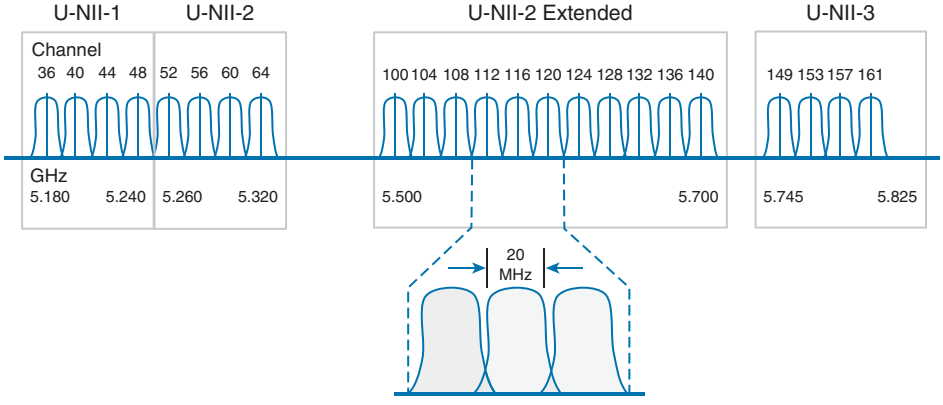



**Figure 1-16** *Channel Layout in the 5-GHz Band*

You might assume that an AP can use any channel number without affecting any APs that use other channel numbers. In the 5-GHz band, this is the case because each channel is allocated a frequency range that does not encroach on or overlap the frequencies allocated for any other channel. In other words, the 5-GHz band consists of **nonoverlapping channels**. In Figure 1-16 notice that the valid channel numbers are not incremental; instead, they increase by four. That means channel numbers in between, such as channels 37, 38, and 39, are not valid or used. Channels in the 6-GHz band are numbered in a similar fashion.

Key Topic

The same is *not* true of the 2.4-GHz band. Each of its channels is much too wide to avoid overlapping the next lower or upper channel number. In fact, each channel covers the frequency range that is allocated to more than four consecutive channels! Notice the width of the channel spacing in Figure 1-15 as compared to the width of one of the shaded signals centered on channels 1, 6, and 11. The only way to avoid any overlap between adjacent channels is to configure APs to use only channels 1, 6, and 11. Even though there are 14 channels available to use, you should always strive for nonoverlapping channels in your network.

It might be obvious that wireless devices and APs should all be capable of operating on the same band. For example, a 5-GHz wireless phone can communicate only with an AP that offers Wi-Fi service on 5-GHz channels. In addition, the devices and APs must also share a compatibility with the parts of the 802.11 standard they support.

As the IEEE 802.11 Wi-Fi standard evolves and develops, new amendments with new functionality get proposed. These amendments are known by "802.11" followed by a one- or two-letter suffix until they are accepted and rolled up into the next generation of the complete 802.11 standard. Even then, it is common to see the amendment suffixes still used to distinguish specific functions.

You should be aware of several amendments that define important characteristics such as data rates, methods used to transmit and receive data, and so on. For the CCNA 200-301

exam, you should know which band each of the amendments listed in Table 1-2 uses. The ENCOR 300-401 exam goes further into the data rates and modulation and coding schemes used by each.

Key Topic

**Table 1-2** Basic Characteristics of Some IEEE 802.11 Amendments

| Amendment | 2.4 GHz | 5 GHz | 6 GHz | Max Data Rate | Notes |
|---|---|---|---|---|---|
| 802.11-1997 | Yes | No | No | 2 Mbps | The original 802.11 standard ratified in 1997 |
| 802.11b | Yes | No | No | 11 Mbps | Introduced in 1999 |
| 802.11g | Yes | No | No | 54 Mbps | Introduced in 2003 |
| 802.11a | No | Yes | No | 54 Mbps | Introduced in 1999 |
| 802.11n | Yes | Yes | No | 600 Mbps | HT (high throughput), introduced in 2009 |
| 802.11ac | No | Yes | No | 6.93 Gbps | VHT (very high throughput), introduced in 2013 |
| 802.11ax | Yes | Yes | Yes | 4x 802.11ac | High Efficiency Wireless |

The 802.11 amendments are not mutually exclusive. Wireless client devices and APs can be compatible with one or more amendments; however, a client and an AP can communicate only if they both support and agree to use the same amendment.

When you look at the specifications for a wireless device, you may find supported amendments listed in a single string, separated by slashes. For example, a device that supports 802.11b/g will support both 802.11b and 802.11g. One that supports b/g/a/n/ac will support 802.11b, 802.11g, 802.11a, 802.11n, and 802.11ac. You should become familiar with Table 1-2 so that you can know which bands a device can use based on its 802.11 amendment support.

If a device can operate on both bands, how does it decide which band to use? APs can usually operate on both bands simultaneously to support any clients that might be present on each band. However, wireless clients typically associate with an AP on one band at a time, while scanning for potential APs on both bands. The band used to connect to an AP is chosen according to the operating system, wireless adapter driver, and other internal configuration. A wireless client can initiate an association with an AP on one band and then switch to the other band if the signal conditions are better there.

**NOTE** Cisco APs can have multiple radios (sets of transmitters and receivers) to support a BSS on each supported band. Some models have two 5-GHz radios that can be configured to operate BSSs on two different channels at the same time, providing wireless coverage to higher densities of users that are located in the same vicinity.

You can configure a Cisco AP to operate on a specific channel number. As the number of APs grows, manual channel assignment can become a difficult task. Fortunately, Cisco wireless architectures can automatically and dynamically assign each AP to an appropriate channel. The architecture is covered in Chapter 2, "Analyzing Cisco Wireless Architectures," while dynamic channel assignment is covered on the ENCOR 300-401 exam.

In open space, usable RF signals can propagate or reach further on the 2.4-GHz band than on the 5-GHz or 6-GHz band. They also tend to penetrate indoor walls and objects more easily at 2.4 GHz than 5 or 6 GHz. However, the 2.4-GHz band is commonly more crowded with wireless devices. Remember that only three nonoverlapping channels are available, so the chances of other neighboring APs using the same channels are greater. In contrast, the 5- and 6-GHz bands have many more channels available to use, making channels less crowded and experiencing less interference.

As Wi-Fi has evolved, the 802.11 standard has grown and new amendments have been introduced, making it difficult to keep track. The Wi-Fi Alliance introduced a set of simplified names to identify each Wi-Fi generation, as listed in Table 1-3. Wi-Fi 0 denotes the original 802.11 standard in its earliest form, followed by each generation, the supported bands, and the prominent 802.11 amendment used. Wireless products are often named or described by these generational names. For example, you might see APs or clients listed as "Wi-Fi 6E," which simply means they can operate in all three bands using IEEE 802.11ax.

**Table 1-3** Wi-Fi Alliance Generational Names

| Wi-Fi Alliance Designation | Bands Supported | IEEE 802.11 Amendments Supported |
|---|---|---|
| Wi-Fi 0 | 2.4 | 802.11 (the original) |
| Wi-Fi 1 | 2.4 | 802.11b |
| Wi-Fi 2 | 5 | 802.11a |
| Wi-Fi 3 | 2.4 | 802.11g |
| Wi-Fi 4 | 2.4, 5 | 802.11n |
| Wi-Fi 5 | 5 | 802.11ac |
| Wi-Fi 6 | 2.4, 5 | 802.11ax |
| Wi-Fi 6E | 2.4, 5, 6 | 802.11ax |
| Wi-Fi 7 | 2.4, 5, 6 | 802.11be |

## Chapter Review

Review this chapter's material using either the tools in the book or the interactive tools for the same material found on the book's companion website. Table 1-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 1-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Key Topic

**Table 1-5** Key Topics for Chapter 1

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 1-4 | Basic service set | 9 |
| Figure 1-7 | Multiple SSIDs | 12 |
| Figure 1-8 | Extended service set | 13 |
| Paragraph | Nonoverlapping channels and bands | 18 |
| Table 1-2 | Basic Characteristics of Some 802.11 Amendments | 19 |

## Key Terms You Should Know

access point (AP), ad hoc wireless network, association request, band, basic service set (BSS), Basic Service Set Identifier (BSSID), beacon, cell, channel, distribution system (DS), extended service set (ESS), Extended Service Set Identifier (ESSID), independent basic service set (IBSS), infrastructure mode, mesh network, nonoverlapping channels, passive scanning, point-to-point bridge, probe request, reassociation request, repeater, roaming, Service Set Identifier (SSID), station (STA), workgroup bridge (WGB)



CHAPTER 2

# Analyzing Cisco Wireless Architectures

**This chapter covers the following exam topics:**

**2.0 Network Access**

**2.6 Compare Cisco Wireless Architectures and AP modes**

In Chapter 1, "Fundamentals of Wireless Networks," you learned how a single access point (AP) can provide a basic service set (BSS) for a cell area and how multiple APs can be connected to form an extended service set (ESS) for a larger network. In this chapter, you learn more about different approaches or architectures that allow APs to be networked together for an enterprise. You also learn how some architectures are more scalable than others and how to manage each type of wireless network architecture.

As you work through this chapter, think about how each architecture can be applied to specific environments—how easy it would be to manage, deploy, and troubleshoot the network, how the APs can be controlled, and how data would move through the network.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 2-1** "Do I Know This Already?" Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Autonomous AP Architecture | 1 |
| Cloud-based AP Architecture | 2 |
| Split-MAC Architectures | 3–5 |
| Comparing Cisco Wireless LAN Controller Deployments | 6 |
| Cisco AP Modes | 7, 8 |

1. Which one of the following terms best describes a Cisco wireless access point that operates in a standalone, independent manner?
   a. Autonomous AP
   b. Independent AP

**c.** Lightweight AP

**d.** Embedded AP

**2.** The Cisco Meraki cloud-based APs are most accurately described by which one of the following statements?

**a.** Autonomous APs joined to a WLC

**b.** Autonomous APs centrally managed

**c.** Lightweight APs joined to a WLC

**d.** Lightweight APs centrally managed

**3.** A "lightweight" access point is said to participate in which one of the following architectures?

**a.** Light-MAC

**b.** Tunnel-MAC

**c.** Split-MAC

**d.** Big-MAC

**4.** How does an access point communicate with a wireless LAN controller?

**a.** Through an IPsec tunnel

**b.** Through a CAPWAP tunnel

**c.** Through a GRE tunnel

**d.** Directly over Layer 2

**5.** Which one of the following is not needed for a Cisco AP in default local mode to be able to support three SSIDs that are bound to three VLANs?

**a.** A trunk link carrying three VLANs

**b.** An access link bound to a single VLAN

**c.** A WLC connected to three VLANs

**d.** A CAPWAP tunnel to a WLC

**6.** Which one of the following WLC deployment models would be best for a large enterprise with around 3000 APs?

**a.** Cisco Mobility Express

**b.** Embedded

**c.** Centralized

**d.** Cloud-based

**7.** If a Cisco AP provides at least one BSS for wireless clients, which one of the following modes does it use?

**a.** Local

**b.** Normal

**c.** Monitor

**d.** Client

8. Regarding Cisco AP modes, which one of the following is true?
   a. An AP can operate in multiple modes at the same time.
   b. An AP has only one possible mode of operation.
   c. The Run mode is the default mode.
   d. The SE-Connect mode is used for spectrum analysis.

# Foundation Topics

## Autonomous AP Architecture

An access point's primary function is to bridge wireless data from the air to a normal wired network. An AP can accept "connections" from a number of wireless clients so that they become members of the LAN, as if the same clients were using wired connections.

APs act as the central point of access (hence the AP name), controlling client access to the wireless LAN. An **autonomous AP** is self-contained; it is equipped with both wired and wireless hardware so that the wireless client associations can be terminated onto a wired connection locally at the AP. The APs and their data connections must be distributed across the coverage area and across the network.

Autonomous APs offer one or more fully functional, standalone basic service sets (BSSs). They are also a natural extension of a switched network, connecting wireless Service Set Identifiers (SSIDs) to wired virtual LANs (VLANs) at the access layer. Figure 2-1 shows the basic architecture; even though only four APs are shown across the bottom, a typical enterprise network could consist of hundreds or thousands of APs.

Key Topic

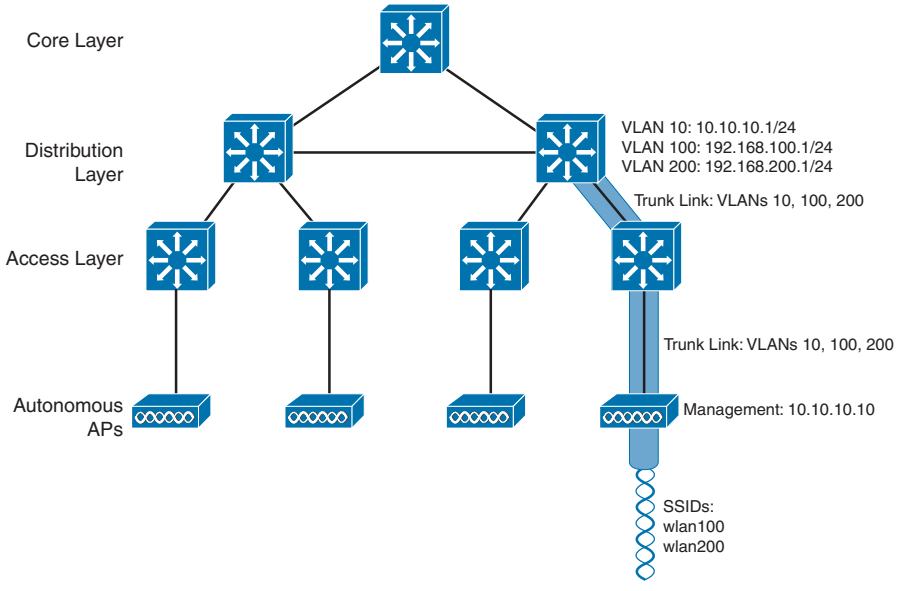



**Figure 2-1** *Wireless Network Architecture with Autonomous APs*

What exactly does an autonomous AP need to become a part of the network? The wireless network in Figure 2-1 consists of two SSIDs: wlan100 and wlan200. They correspond to wired VLANs 100 and 200, respectively. As shown by the shaded links, the VLANs must be trunked from the distribution layer switch (where routing commonly takes place) to the access layer, where they are extended further over a trunk link to the AP.

An autonomous AP offers a short and simple path for data to travel between the wireless and wired networks. Data has to travel only through the AP to reach the network on the other side. Two wireless users that are associated to the same autonomous AP can reach each other through the AP without having to pass up into the wired network. As you work through the wireless architectures discussed in the rest of the chapter, notice the data path that is required for each.

An autonomous AP must also be configured with a management IP address (10.10.10.10 in Figure 2-1) so that you can remotely manage it. After all, you will want to configure SSIDs, VLANs, and many RF parameters like the channel and transmit power to be used. The management address is not normally part of any of the data VLANs, so a dedicated management VLAN (i.e., VLAN 10) must be added to the trunk links to reach the AP. Each AP must be configured and maintained individually unless you leverage a management platform such as Cisco Prime Infrastructure or Cisco Catalyst Center.

Because the data and management VLANs may need to reach every autonomous AP, the network configuration and efficiency can become cumbersome as the network scales. For example, you will likely want to offer the same SSID on many APs so that wireless clients can associate with that SSID in most any location or while roaming between any two APs. You might also want to extend the corresponding VLAN (and IP subnet) to each and every AP so that clients do not have to request a new IP address for each new association.

Because SSIDs and their VLANs must be extended at Layer 2, you should consider how they are extended throughout the switched network. The shaded links in Figure 2-2 show an example of a single VLAN's extent in the data plane. Working top to bottom, follow VLAN 100 as it reaches through the network. VLAN 100 is routed within the distribution layer and must be carried over trunk links to the access layer switches and then to each autonomous AP. In effect, VLAN 100 must extend end to end across the whole infrastructure—something that is usually considered to be a bad practice.

That approach might sound straightforward until you have to add a new VLAN and configure every switch and AP in your network to carry and support it. Even worse, suppose your network has redundant links between each layer of switches. The Spanning Tree Protocol (STP) running on each switch becomes a vital ingredient to prevent bridging loops from forming and corrupting the network. For these reasons, client roaming across autonomous APs is typically limited to the Layer 2 domain, or the extent of a single VLAN. As the wireless network expands, the infrastructure becomes more difficult to configure correctly and becomes less efficient.

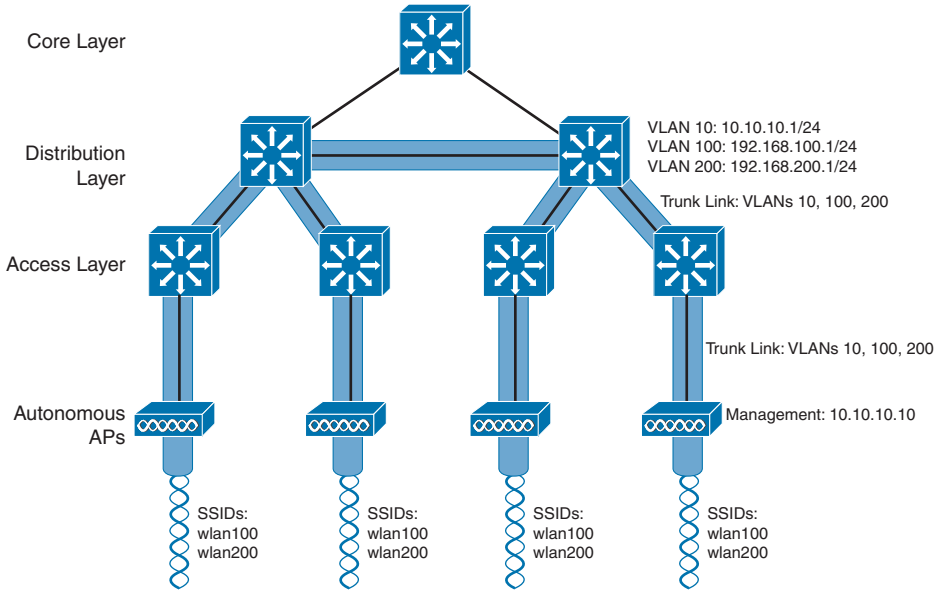



**Figure 2-2** *Extent of a Data VLAN in a Network of Autonomous APs*

## Cloud-based AP Architecture

Recall that an autonomous AP needs quite a bit of configuration and management. To help manage more and more autonomous APs as the wireless network grows, you could place an AP management platform such Cisco Catalyst Center (also known as DNA Center) in a central location within the enterprise. The management platform would need to be purchased, configured, and maintained too.

A simpler approach is a **cloud-based AP** architecture, where the AP management function is pushed out of the enterprise and into the Internet cloud. Cisco Meraki is cloud-based and offers centralized management of wireless, switched, and security networks built from Meraki products. For example, through the cloud networking service, you can configure and manage APs, monitor wireless performance and activity, generate reports, and so on.

Cisco Meraki APs can be deployed automatically, after you register with the Meraki cloud. Each AP will contact the cloud when it powers up and will self-configure. From that point on, you can manage the AP through the Meraki cloud dashboard.

Figure 2-3 illustrates the basic cloud-based architecture. Notice that the network is arranged identically to that of the autonomous AP network. The reason is that the APs in a cloud-based network are all autonomous too. The most visible difference is that all of the APs are managed, controlled, and monitored centrally from the cloud.

---

Answers to the "Do I Know This Already?" quiz:

**1** A **2** B **3** C **4** B **5** A **6** C **7** A **8** D

Key Topic

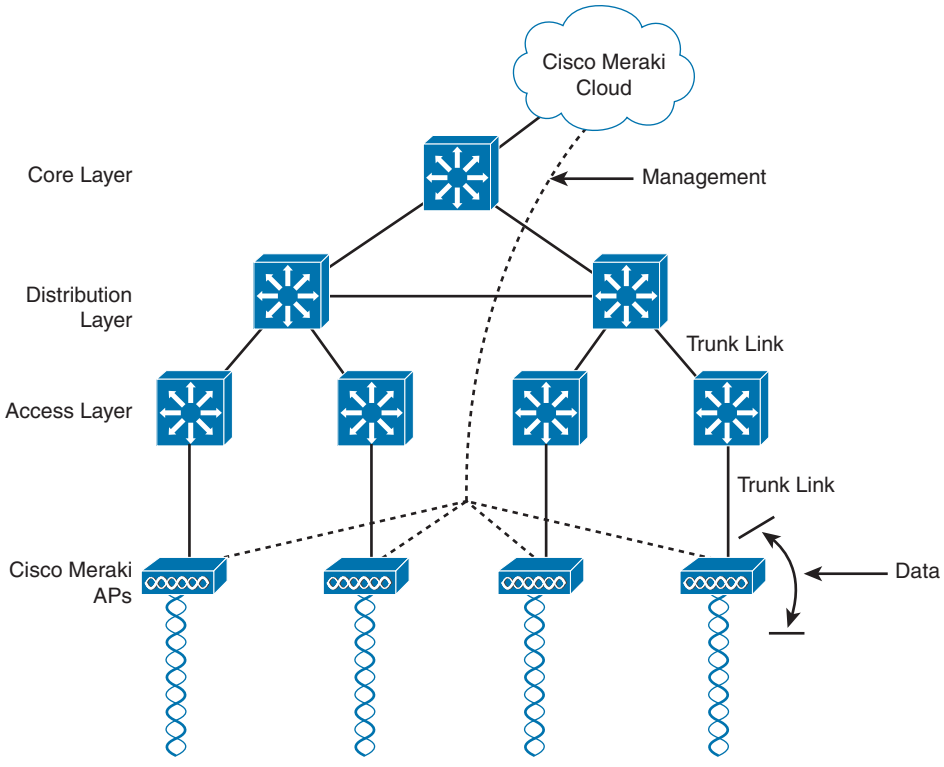



**Figure 2-3** *Cisco Meraki Cloud-Based Wireless Network Architecture*

From the cloud, you can push out code upgrades and configuration changes to the APs in the enterprise. The Cisco Meraki cloud also adds the intelligence needed to automatically instruct each AP on which channel and transmit power level to use. It can also collect information from all of the APs about things such as RF interference, rogue or unexpected wireless devices that were overheard, and wireless usage statistics.

Finally, there are a couple of things you should observe about the cloud-based architecture. The data path from the wireless network to the wired network is very short; the autonomous AP directly links the two networks. Data to and from wireless clients does not have to travel up into the cloud and back; the cloud is used only to bring management functions into the data plane.

Also, notice that the network in Figure 2-3 consists of two distinct paths—one for data traffic and another for management traffic, corresponding to the following two functions:

- **A control plane:** Traffic used to control, configure, manage, and monitor the AP itself
- **A data plane:** End-user traffic passing through the AP

This division will become important in the following sections as other types of architecture are discussed.

## Split-MAC Architectures

Because autonomous APs are…well, autonomous, managing their RF operation can be quite difficult. As a network administrator, you are in charge of selecting and configuring the channel used by each AP and detecting and dealing with any rogue APs that might be interfering. You must also manage things such as the transmit power level to make sure that the wireless coverage is sufficient, it does not overlap too much, and there aren't any coverage holes—even when an AP's radio fails.

Managing wireless network security can also be difficult. Each autonomous AP handles its own security policies, with no central point of entry between the wireless and wired networks. That means there is no convenient place to monitor traffic for things such as intrusion detection and prevention, quality of service, bandwidth policing, and so on.

To overcome the limitations of distributed autonomous APs, many of the functions found within autonomous APs have to be shifted toward some central location. In Figure 2-4, most of the activities performed by an autonomous AP on the left are broken up into two groups—management functions on the top and real-time processes on the bottom.

Key Topic

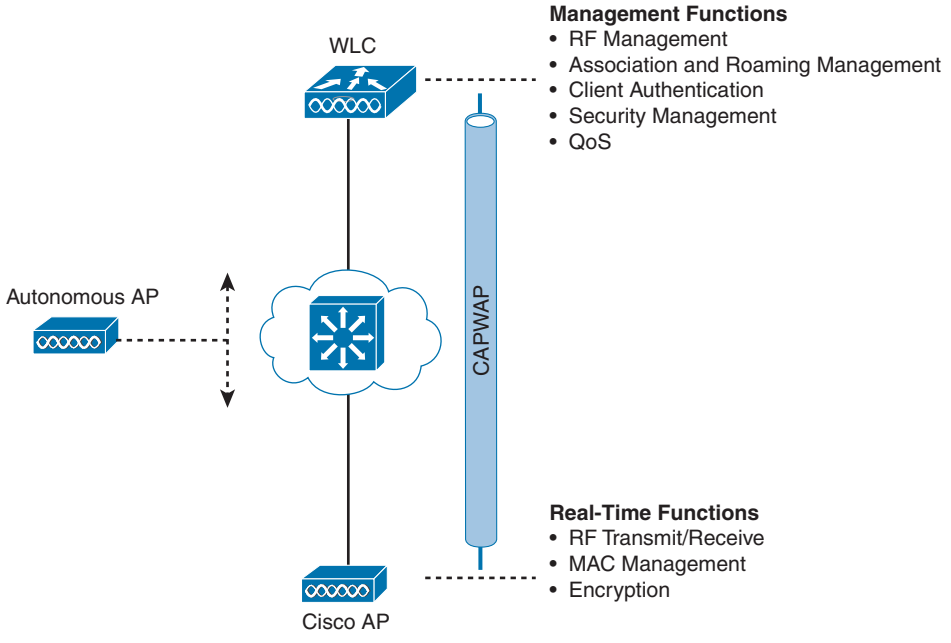



**Figure 2-4** *Autonomous Versus Cisco Access Point*

The real-time processes involve sending and receiving 802.11 frames, beacons, and probe messages. The 802.11 data encryption is also handled in real time, on a per-packet basis. The AP must interact with wireless clients on some low level, known as the **Media Access Control (MAC) layer**. These functions must stay with the AP hardware, closest to the clients.

The management functions are not integral to handling frames over the RF channels, but are things that should be centrally administered. Therefore, those functions can be moved to a centrally located platform away from the AP.

When the functions of an autonomous AP are divided, the Cisco AP hardware performs only the real-time 802.11 operation. Its code image and local intelligence are stripped down, or lightweight, compared to the traditional autonomous AP.

The management functions are usually performed on a **wireless LAN controller (WLC)**, which controls many Cisco APs. This is shown in the bottom right portion of Figure 2-4. Notice that the AP is left with duties in Layers 1 and 2, where frames are moved into and out of the RF domain. The AP becomes totally dependent on the WLC for every other WLAN function, such as authenticating users, managing security policies, and even selecting RF channels and output power.

**NOTE** Remember that a Cisco AP cannot normally operate on its own; it is very dependent on a WLC somewhere in the network. That means the AP will become nonfunctional if it loses connectivity to its WLC. The only exception is the FlexConnect AP mode, which is discussed later in this chapter.

The AP-WLC division of labor is known as a **split-MAC architecture**, where the normal MAC operations are pulled apart into two distinct locations. This occurs for every AP in the network; each one must boot and bind itself to a WLC to support wireless clients. The WLC becomes the central hub that supports a number of APs scattered about in the network.

How does an AP bind with a WLC to form a complete working access point? The two devices must use a tunneling protocol between them, to carry 802.11-related messages and also client data. Remember that the AP and WLC can be located on the same VLAN or IP subnet, but they do not have to be. Instead, they can be located on two entirely different IP subnets in two entirely different locations.

The Control and Provisioning of Wireless Access Points (**CAPWAP**) tunneling protocol makes this all possible by encapsulating the data between the LAP and WLC within new IP packets. The tunneled data can then be switched or routed across the campus network. As Figure 2-5 shows, the CAPWAP relationship actually consists of two separate tunnels, as follows:

- **CAPWAP control messages:** Carry exchanges that are used to configure the AP and manage its operation. The control messages are authenticated and encrypted, so the AP is securely controlled by only the appropriate WLC, then transported over the control tunnel.
- **CAPWAP data:** Used for packets traveling to and from wireless clients that are associated with the AP. Data packets are transported over the data tunnel but are not encrypted by default. When data encryption is enabled for an AP, packets are protected with Datagram Transport Layer Security (DTLS).

**NOTE** CAPWAP is defined in RFCs 5415, 5416, 5417, and 5418. CAPWAP is based on the Lightweight Access Point Protocol (LWAPP), which was a legacy Cisco proprietary solution.

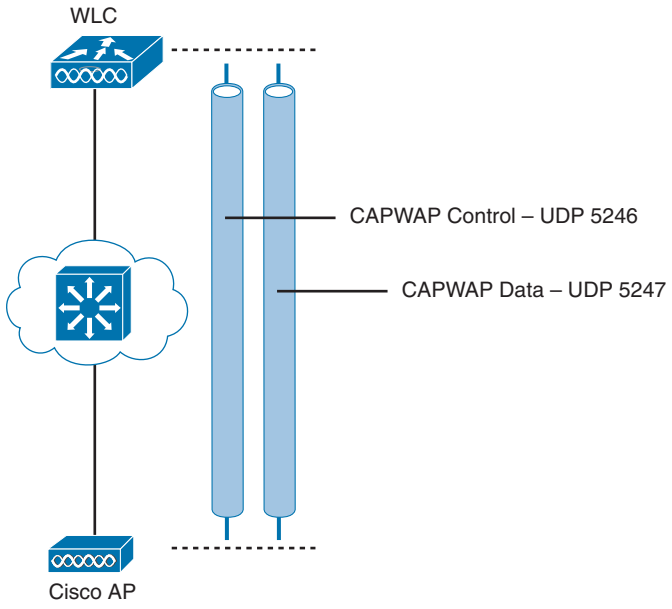



**Figure 2-5** *Linking an AP and WLC with CAPWAP*

Every AP and WLC must also authenticate each other with digital certificates. An X.509 certificate is preinstalled in each device when it is purchased. By using certificates behind the scenes, every device is properly authenticated before becoming part of the wireless network. This process helps assure that no one can add an unauthorized AP to your network.

The CAPWAP tunneling allows the AP and WLC to be separated geographically and logically. It also breaks the dependence on Layer 2 connectivity between them. For example, Figure 2-6 uses shaded areas to show the extent of VLAN 100. Notice how VLAN 100 exists at the WLC and in the air as SSID 100, near the wireless clients—but not in between the AP and the WLC. Instead, traffic to and from clients associated with SSID 100 is transported across the network infrastructure encapsulated inside the CAPWAP data tunnel. The tunnel exists between the IP address of the WLC and the IP address of the AP, which allows all of the tunneled packets to be routed at Layer 3.

Also, notice how the AP is known by only a single IP address: 10.10.10.10. Because the AP sits on the access layer where its CAPWAP tunnels terminate, it can use one IP address for both management and tunneling. No trunk link is needed because all of the VLANs it supports are encapsulated and tunneled as Layer 3 IP packets, rather than individual Layer 2 VLANs.

As the wireless network grows, the WLC simply builds more CAPWAP tunnels to reach more APs. Figure 2-7 depicts a network with four APs. Each AP has a control and a data tunnel back to the centralized WLC. SSID 100 can exist on every AP, and VLAN 100 can reach every AP through the network of tunnels.

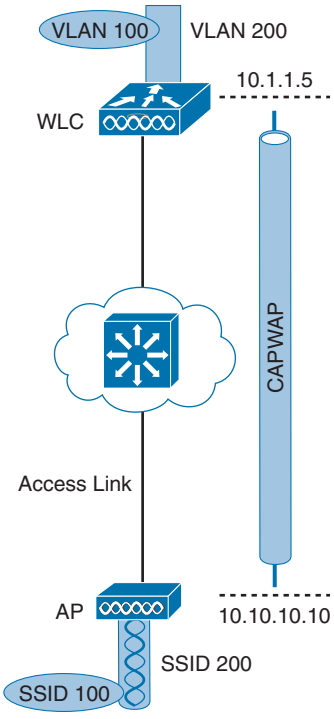



**Figure 2-6** *Extent of VLAN 100 in a Cisco Wireless Network*

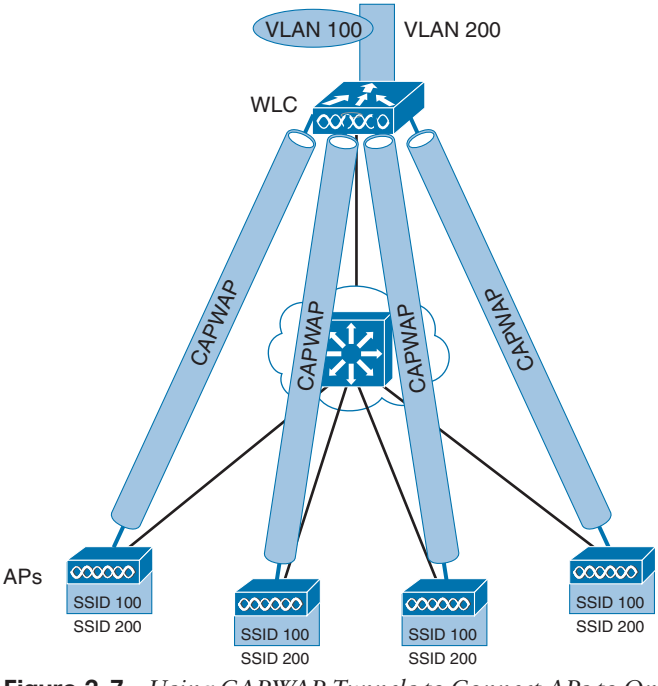



**Figure 2-7** *Using CAPWAP Tunnels to Connect APs to One Central WLC*

After CAPWAP tunnels are built from a WLC to one or more lightweight APs, the WLC can begin offering a variety of additional functions. Think of all the puzzles and shortcomings that were discussed for the traditional autonomous WLAN architecture as you read over the following list of WLC activities:

- **Dynamic channel assignment:** The WLC can automatically choose and configure the RF channel used by each AP, based on other active access points in the area.
- **Transmit power optimization:** The WLC can automatically set the transmit power of each AP based on the coverage area needed.
- **Self-healing wireless coverage:** If an AP radio dies, the coverage hole can be "healed" by turning up the transmit power of surrounding APs automatically.
- **Flexible client roaming:** Clients can roam between APs with very fast roaming times.
- **Dynamic client load balancing:** If two or more APs are positioned to cover the same geographic area, the WLC can associate clients with the least used AP. This approach distributes the client load across the APs.
- **RF monitoring:** The WLC manages each AP so that it scans channels to monitor the RF usage. By listening to a channel, the WLC can remotely gather information about RF interference, noise, signals from neighboring APs, and signals from rogue APs or ad hoc clients.
- **Security management:** The WLC can authenticate clients from a central service and can require wireless clients to obtain an IP address from a trusted DHCP server before allowing them to associate and access the WLAN.
- **Wireless intrusion prevention (or protection) system:** Leveraging its central location, the WLC can monitor client data to detect and prevent malicious activity.

## Comparing Cisco Wireless LAN Controller Deployments

Suppose you want to deploy a WLC to support multiple lightweight APs in your network. Where should you put the WLC? The split-MAC concept can be applied to several different network architectures. Each architecture places the WLC in a different location within the network—a choice that also affects how many WLCs might be needed to support the number of APs required.

One approach is to locate the WLC in a central location so that you can maximize the number of APs joined to it. This is usually called a **centralized WLC deployment** or *unified WLC deployment*, which tends to follow the concept that most of the resources users need to reach are located in a central location such as a data center or the Internet. Traffic to and from wireless users would travel over CAPWAP tunnels that reach into the center of the network, near the core, as shown in Figure 2-8. A centralized WLC also provides a convenient place to enforce security policies that affect all wireless users.

Figure 2-8 shows four APs joined to a single WLC. Your network might have more APs—many, many more. A large enterprise network might have thousands of APs connected to its access layer. Scalability then becomes an important factor in the centralized design. Typical centralized WLCs can support a maximum of around 6000 APs. If you have more APs than the maximum, you will need to add more WLCs to the design, each located centrally.

Key Topic

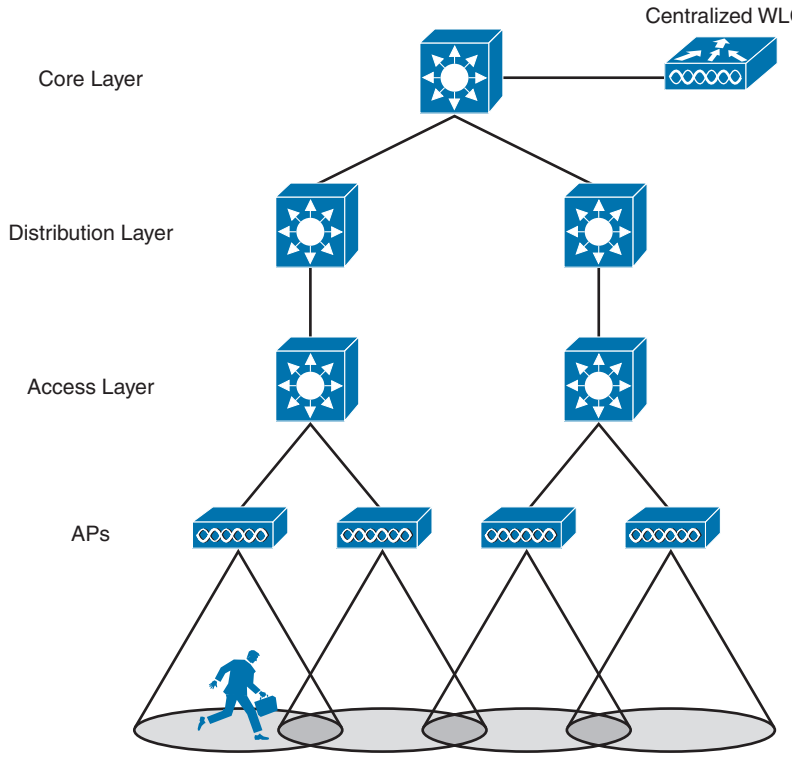



**Figure 2-8** *WLC Location in a Centralized Deployment*

When a WLC is centrally located inside a public or private cloud, it is known as a **cloud-based WLC deployment**. Because a private cloud is usually part of the enterprise network, the WLC can stay relatively close to its APs and minimize the length of the data path between them. However, locating the WLC in a public cloud brings an interesting twist—the controller can be quite a distance from the APs that join to it. The APs can maintain a CAPWAP control tunnel to the controller, but all wireless data passing through the APs must be locally switched in and out of the VLANs directly connected to the APs. Therefore, the APs must operate only in **FlexConnect mode**, which is described later in this chapter in the section titled "FlexConnect Mode." Cloud-based controllers can typically support up to 6000 APs, as of this writing.

For small campuses or distributed branch locations, where the number of APs is relatively small in each, WLCs can be located further down in the network, as shown in Figure 2-9. This is known as a **distributed WLC deployment** because multiple controllers are distributed within the network. Typical distributed WLCs can support up to 250 APs. As the number of APs grows, additional WLCs can be added by locating them in other strategic areas of the network.

Finally, in small-scale environments, such as small, midsize, or multisite branch locations, you might not want to invest in dedicated WLCs at all. In this case, the WLC function can be co-located with an AP that is installed at the branch site. This is known as an **embedded wireless controller (EWC) deployment**, as shown in Figure 2-10. It is also called a **controller-less wireless deployment** because it does not involve a discrete physical controller at all. The AP that hosts the WLC forms a CAPWAP tunnel with the WLC, along with any other APs at the same location. An EWC can support up to 100 APs.

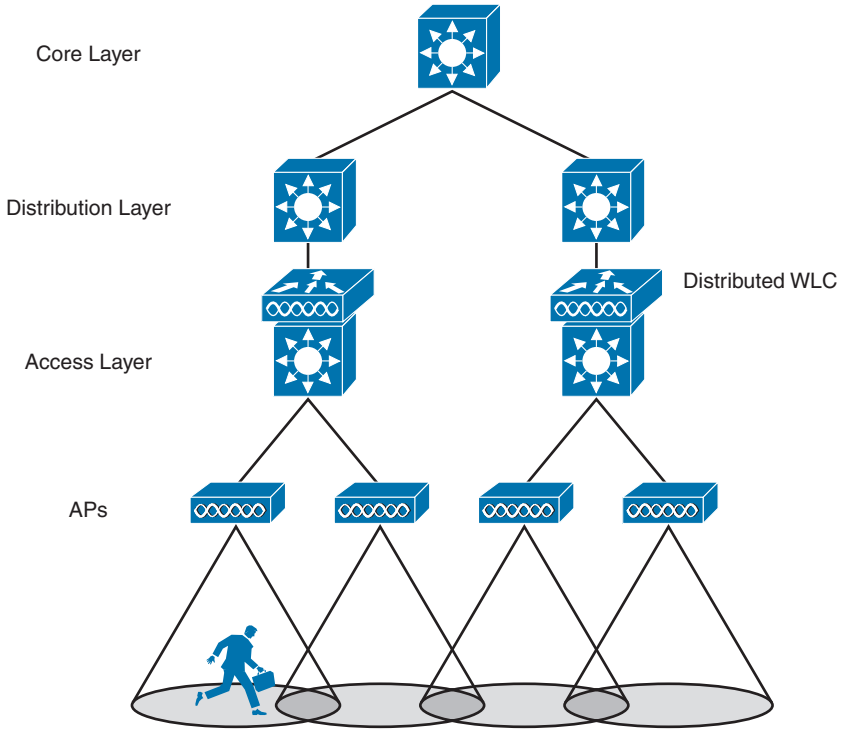



**Figure 2-9** *WLC Location in a Distributed Deployment*

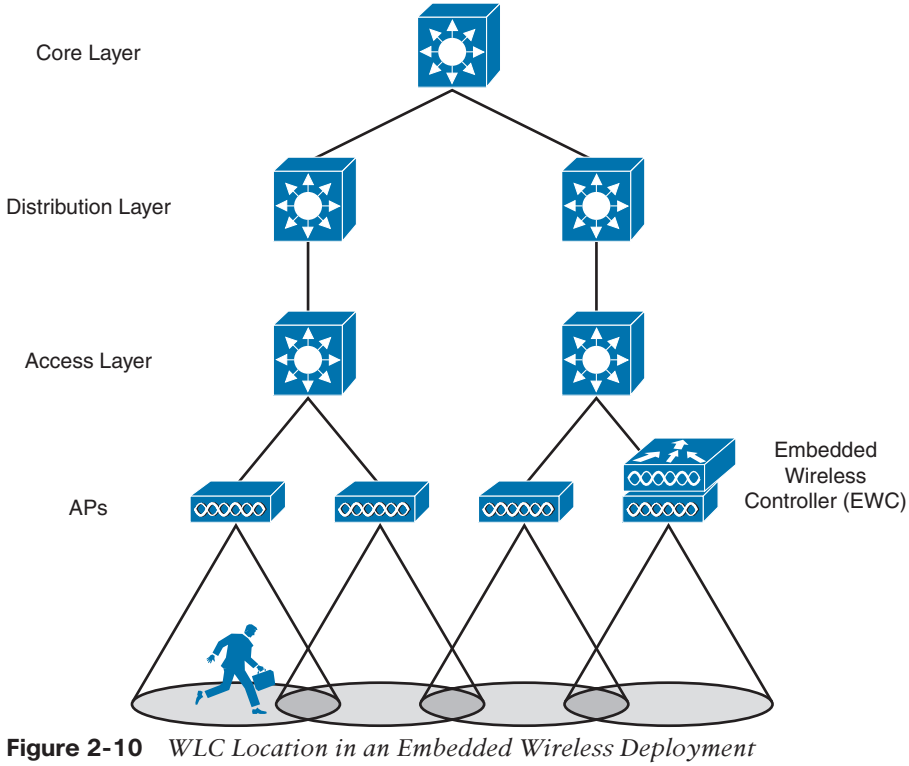



**Figure 2-10** *WLC Location in an Embedded Wireless Deployment*

Refer to Table 2-2 for a summary of WLC deployment models, WLC locations, and a typical maximum number of APs and clients that each one supports.

**Table 2-2** Summary of WLC Deployment Models

| Deployment Model | WLC Location | APs Supported | Clients Supported | Typical Use |
|---|---|---|---|---|
| Centralized | Central | 6000 | 64,000 | Large enterprise |
| Cloud-based | Public or private cloud | 6000 | 64,000 | Large enterprise |
| Distributed | Access | 250 | 5000 | Small campus |
| Embedded | Other | 100 | 2000 | Branch location |
| Autonomous | N/A | N/A | N/A | N/A |

## Cisco AP Modes

Cisco APs can operate in one of the following modes, depending on how they are configured:

Key Topic

- **Local:** This default lightweight mode offers one or more functioning BSSs on a specific channel. During times that it is not transmitting, the AP will scan the other channels to measure the level of noise, measure interference, discover rogue devices, and match against wireless intrusion detection system (WIDS) events.
- **Monitor:** The AP does not transmit at all, but its receiver is enabled to act as a dedicated sensor. The AP checks for IDS events, detects rogue access points, and determines the position of stations through location-based services.
- **FlexConnect:** An AP at a remote site can locally switch traffic between an SSID and a VLAN if its CAPWAP tunnel to the WLC is down and if it is configured to do so.
- **Sniffer:** An AP dedicates its radios to receiving 802.11 traffic from other sources, much like a sniffer or packet capture device. The captured traffic is then forwarded to a PC running network analyzer software such as WireShark, where it can be analyzed further.
- **Rogue detector:** An AP dedicates itself to detecting rogue devices by correlating MAC addresses heard on the wired network with those heard over the air. Rogue devices are those that appear on both networks.
- **Bridge:** An AP becomes a dedicated bridge (point-to-point or point-to-multipoint) between two networks. Two APs in bridge mode can be used to link two locations separated by a distance. Multiple APs in bridge mode can form an indoor or outdoor mesh network.
- **Flex+Bridge:** FlexConnect operation is enabled on a mesh AP.

- **SE-Connect:** The AP dedicates its radios to spectrum analysis on all wireless channels. You can remotely connect a PC running software such as MetaGeek Chanalyzer or Cisco Spectrum Expert to the AP to collect and analyze the spectrum analysis data to discover sources of interference.

**NOTE** Remember that a Cisco AP is normally in **local mode** when it is providing BSSs and allowing client devices to associate to wireless LANs. When an AP is configured to operate in one of the other modes, local mode is disabled.

## FlexConnect Mode

In a switched campus infrastructure, the split-MAC traffic pattern is efficient because the WLC can be located reasonably close to the APs. That pattern tends to minimize latency and maximize bandwidth between them. Suppose that the network grows to include some remote branch sites. APs are placed at the branch sites, but the only WLC is located back at the main site. This scenario forces wireless traffic to traverse the CAPWAP tunnel between the branch and main sites to reach centralized resources, as shown in the left portion of Figure 2-11. Even traffic between two wireless users has to follow the tunnel to the controller and back, which might not be ideal.

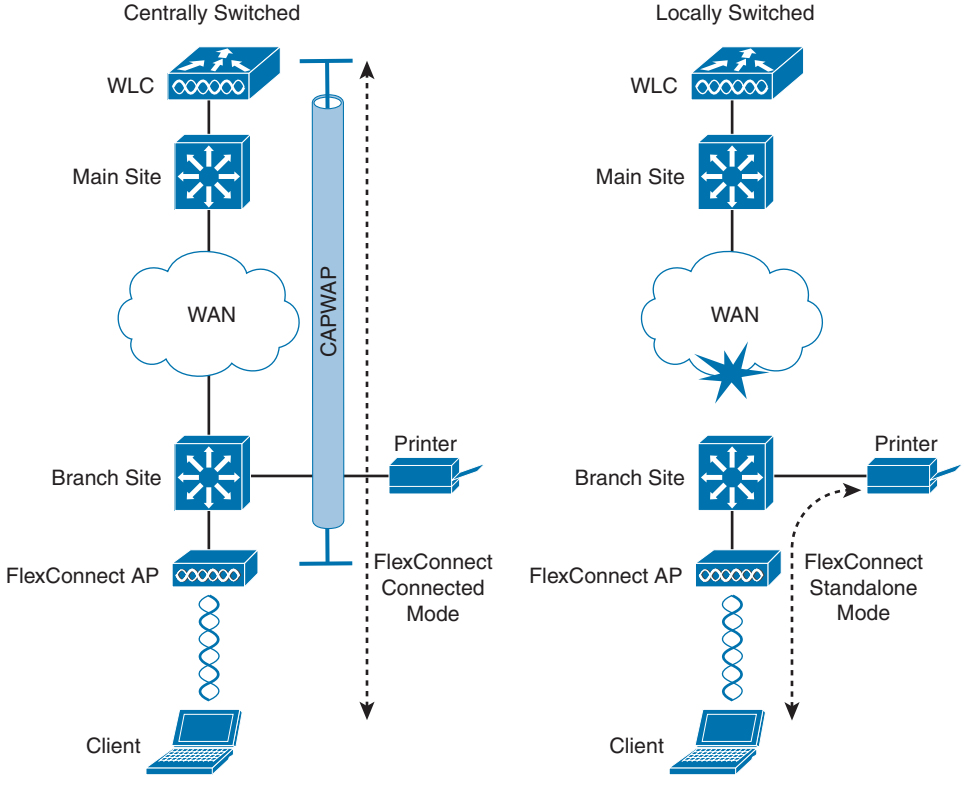



**Figure 2-11** *FlexConnect AP Operating in Centrally and Locally Switched Modes*

Now consider the importance of the WAN link connecting the remote site. An AP at the branch site is normally completely dependent on the WLC to form the split-MAC relationship. If the WAN link goes down, and the WLC is unreachable, as shown in the right portion of Figure 2-11, then the AP ceases to operate.

The FlexConnect AP mode can make the most of each situation, whether the WAN link is up or down. In **connected mode**, the AP joins the WLC, and all wireless traffic flows over the CAPWAP tunnel. However, if the WAN link is down, the AP can use its **standalone mode** to keep operating independently from the WLC, supporting wireless users locally and allowing them to communicate with other local devices.

Switching between connected and standalone modes is automatic, based on connectivity to the WLC. Even better, you can configure FlexConnect APs to locally switch traffic that needs to stay within the remote site on a per-wireless LAN basis, even if the controller is still reachable. That approach avoids the long and unnecessary hairpin path to and from the controller for local traffic, while maintaining the CAPWAP tunnel for traffic that needs to be centrally switched at the main site.

Cisco OfficeExtend is another solution for connecting remote sites to a main location but is meant to be used at teleworkers' homes. Like FlexConnect, OfficeExtend can present the same corporate wireless LANs at the remote site. However, the two solutions are very different in terms of scale. A FlexConnect AP is meant to support a large number of wireless users and can co-exist with other FlexConnect APs, allowing wireless users to roam between APs. An OfficeExtend AP is meant to exist as a single AP at the teleworker's home, so roaming is not supported.

The two solutions also differ in the way that user data traffic is protected between the AP and the WLC. FlexConnect protects and encrypts only control traffic by default; user data is not protected. Because OfficeExtend is designed to handle a small number of users on a single AP, both the control and data traffic are always encrypted with DTLS. Table 2-3 summarizes the differences between the two solutions.

**Table 2-3** Comparing FlexConnect and OfficeExtend APs

| | FlexConnect | OfficeExtend |
|---|---|---|
| Does AP join a WLC? | Yes | Yes |
| Is data traffic protected? | Optional | Yes, with DTLS |
| Typical number of APs at remote site | No limit | One |
| Is roaming supported? | Yes | No |

## Chapter Review

Review this chapter's material using either the tools in the book or the interactive tools for the same material found on the book's companion website. Table 2-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 2-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Review the most important topics in this chapter, noted with the Key Topic icon in the outer margin of the page. Table 2-5 lists a reference of these key topics and the page numbers on which each is found.

Key Topic

**Table 2-5** Key Topics for Chapter 2

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 2-1 | Autonomous AP architecture | 24 |
| Figure 2-3 | Cloud-based AP architecture | 27 |
| Figure 2-4 | Split-MAC architecture | 28 |
| Figure 2-5 | CAPWAP tunnels | 30 |
| Figure 2-8 | Centralized WLC deployment | 33 |
| Figure 2-9 | Distributed WLC deployment | 34 |
| Figure 2-10 | Embedded WLC deployment | 34 |
| List | Cisco AP modes | 35 |

## Key Terms You Should Know

autonomous AP, CAPWAP, centralized WLC deployment, cloud-based AP, cloud-based WLC deployment, connected mode, controller-less wireless deployment, distributed WLC deployment, embedded wireless controller (EWC) deployment, FlexConnect mode, local mode, Media Access Control (MAC) layer, split-MAC architecture, standalone mode, wireless LAN controller (WLC)

*This page intentionally left blank*



CHAPTER 3

# Securing Wireless Networks

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.11 Describe Wireless Principles**
    - **1.11.d Encryption**
- **5.0 Security Fundamentals**
  - **5.9 Describe wireless security protocols (WPA, WPA2, and WPA3)**

As you know by now, wireless networks are complex. Many technologies and protocols work behind the scenes to give end users a stable, yet mobile, connection to a wired network infrastructure. From the user's perspective, a wireless connection should seem no different than a wired connection. A wired connection can give users a sense of security; data traveling over a wire is probably not going to be overheard by others. A wireless connection is inherently different; data traveling over the air can be overheard by anyone within range.

Therefore, securing a wireless network becomes just as important as any other aspect. A comprehensive approach to wireless security focuses on the following areas:

- Identifying the endpoints of a wireless connection
- Identifying the end user
- Protecting the wireless data from eavesdroppers
- Protecting the wireless data from tampering

The identification process is performed through various authentication schemes. Protecting wireless data involves security functions like encryption and frame authentication.

This chapter covers many of the methods you can use to secure a wireless network. Be warned: wireless security can be a confusing topic because it is filled with many acronyms. Some of the acronyms rhyme like words from a children's book. In fact, this chapter is a story about WEP, PSK, TKIP, MIC, AES, EAP, EAP-FAST, EAP-TLS, LEAP, PEAP, WPA, WPA2, WPA3, CCMP, GCMP, and on and on it goes. When you finish with this chapter, though, you will come away with a clearer view of what these terms mean and how they all fit together. You might even be ready to configure a wireless LAN with effective security.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on

the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 3-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Anatomy of a Secure Connection | 1, 2 |
| Wireless Client Authentication Methods | 3, 4 |
| Wireless Privacy and Integrity Methods | 5, 6 |
| WPA, WPA2, and WPA3 | 7, 8 |

1. Which of the following are necessary components of a secure wireless connection? (Choose all that apply.)
   - **a.** Encryption
   - **b.** MIC
   - **c.** Authentication
   - **d.** All of these answers are correct.
2. Which one of the following is used to protect the integrity of data in a wireless frame?
   - **a.** WIPS
   - **b.** WEP
   - **c.** MIC
   - **d.** EAP
3. Which one of the following is a wireless encryption method that has been found to be vulnerable and is not recommended for use?
   - **a.** AES
   - **b.** WPA
   - **c.** EAP
   - **d.** WEP
4. Which one of the following is used as the authentication framework when 802.1x is used on a WLAN?
   - **a.** Open authentication
   - **b.** WEP
   - **c.** EAP
   - **d.** WPA
5. Suppose you would like to select a method to protect the privacy and integrity of wireless data. Which one of the following methods should you avoid because it has been deprecated ?
   - **a.** TKIP
   - **b.** CCMP
   - **c.** GCMP
   - **d.** EAP

6. Which one of the following is the data encryption and integrity method used by WPA2?
   a. WEP
   b. TKIP
   c. CCMP
   d. WPA
7. The Wi-Fi Alliance offers which of the following certifications for wireless devices that correctly implement security standards? (Choose all that apply.)
   a. WEP
   b. WPA2
   c. 802.11
   d. AES
8. A pre-shared key is used in which of the following wireless security configurations? (Choose all that apply.)
   a. WPA2 personal mode
   b. WPA2 enterprise mode
   c. WPA3 personal mode
   d. WPA3 enterprise mode

# Foundation Topics

## Anatomy of a Secure Connection

In the previous chapters of this book, you learned about wireless clients forming associations with wireless access points (APs) and passing data back and forth across the air.

As long as all clients and APs conform to the 802.11 standard, they can all coexist—even on the same channel. Not every 802.11 device is friendly and trustworthy, however. Sometimes it is easy to forget that transmitted frames do not just go directly from the sender to the receiver, as in a wired or switched connection. Instead, they travel according to the transmitter's antenna pattern, potentially reaching any receiver that is within range.

Consider the scenario in Figure 3-1. The wireless client opens a session with some remote entity and shares a confidential password. Because two untrusted users are also located within range of the client's signal, they may also learn the password by capturing frames that have been sent on the channel. The convenience of wireless communication also makes it easy for transmissions to be overheard and exploited by malicious users.

If data is sent through open space, how can it be secured so that it stays private and intact? The 802.11 standard offers a framework of wireless security mechanisms that can be used to add trust, privacy, and integrity to a wireless network. The following sections give an overview of the wireless security framework.

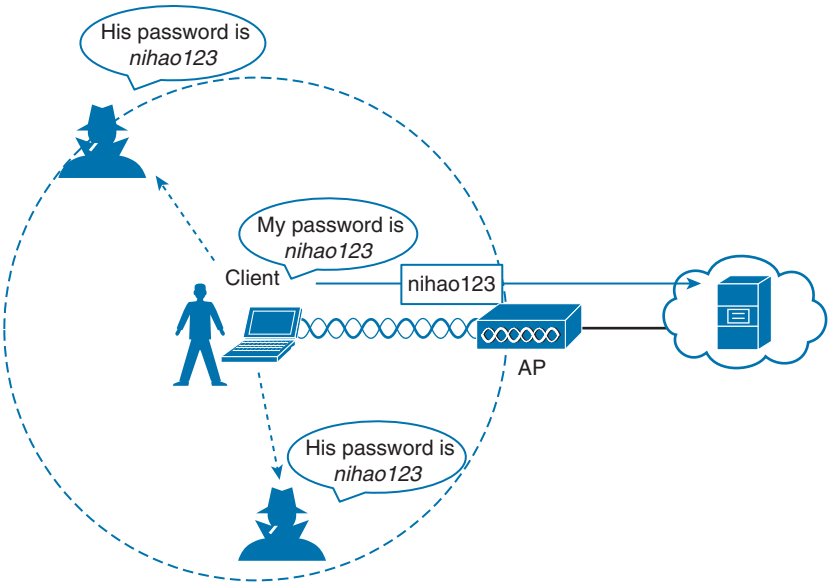



**Figure 3-1** *Wireless Transmissions Reaching Unintended Recipients*

## Authentication

To use a wireless network, clients must first discover a basic service set (BSS) and then request permission to associate with it. Clients should be authenticated by some means before they can become functioning members of the wireless LAN. Why?

Suppose that your wireless network connects to corporate resources where confidential information can be accessed. In that case, only devices known to be trusted and expected should be given access. Guest users, if they are permitted at all, should be allowed to join a different guest WLAN where they can access nonconfidential or public resources. Rogue clients, which are not expected or welcomed, should not be permitted to associate at all. After all, they are not affiliated with the corporate network and are likely to be unknown devices that happen to be within range of your network.

To control access, wireless networks can authenticate the client devices before they are allowed to associate. Potential clients must identify themselves by presenting some form of credentials to the APs. Figure 3-2 shows the basic client authentication process.

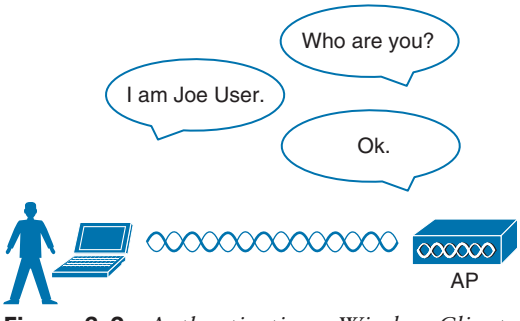



**Figure 3-2** *Authenticating a Wireless Client*

Wireless authentication can take many forms. Some methods require only a static text string that is common across all trusted clients and APs. The text string is stored on the client device and presented directly to the AP when needed. What might happen if the device was stolen or lost? Most likely, any user who possessed the device could still authenticate to the network. Other more stringent authentication methods require interaction with a corporate user database. In those cases, the end user must enter a valid username and password—something that would not be known to a thief or an imposter.

If you have ever joined a wireless network, you might have focused on authenticating your device or yourself, while implicitly trusting the nearest AP. For example, if you turn on your wireless device and find a wireless network that is available at your workplace, you probably join it without hesitating. The same is true for wireless networks in an airport, a hotel, a hot spot, or in your home—you expect the AP that is advertising the SSID to be owned and operated by the entity where you are located. But how can you be sure?

Normally, the only piece of information you have is the SSID being broadcast or advertised by an AP. If the SSID looks familiar, you will likely choose to join it. Perhaps your computer is configured to automatically connect to a known SSID so that it associates without your intervention. Either way, you might unwittingly join the same SSID even if it was being advertised by an imposter.

Some common attacks focus on a malicious user pretending to be an AP. The fake AP can send beacons, answer probes, and associate clients just like the real AP it is impersonating. Once a client associates with the fake AP, the attacker can easily intercept all communication to and from the client from its central position. A fake AP could also send spoofed management frames to disassociate or deauthenticate legitimate and active clients, just to disrupt normal network operation.

To prevent this type of man-in-the-middle attack, the client should authenticate the AP before the client itself is authenticated. Figure 3-3 shows a simple scenario. Even further, any management frames received by a client should be authenticated too, as proof that they were sent by a legitimate and expected AP.

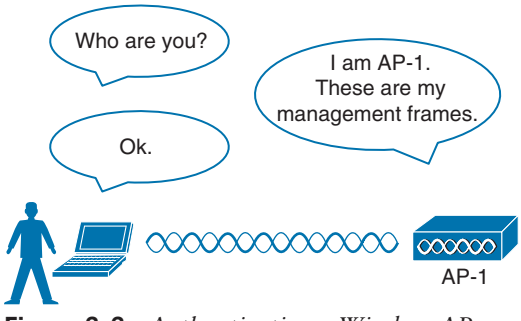



**Figure 3-3** *Authenticating a Wireless AP*

## Message Privacy

Suppose that the client in Figure 3-3 must authenticate before joining the wireless network. It might also authenticate the AP and its management frames after it associates but before

---

Answers to the "Do I Know This Already?" quiz:

**1** D **2** C **3** D **4** C **5** A **6** C **7** B **8** A, C

it is itself authenticated. The client's relationship with the AP might become much more trusted, but data passing to and from the client is still available to eavesdroppers on the same channel.

To protect data privacy on a wireless network, the data should be encrypted for its journey through free space. This task is accomplished by encrypting the data payload in each wireless frame just prior to being transmitted, then decrypting it as it is received. The idea is to use an encryption method that the transmitter and receiver share, so the data can be encrypted and decrypted successfully by them and nobody else.

In wireless networks, each WLAN may support only one authentication and encryption scheme, so all clients must use the same encryption method when they associate. You might think that having one encryption method in common would allow every client to eavesdrop on every other client. That is not necessarily the case because the AP should securely negotiate a unique encryption key to use for each associated client.

Ideally, the AP and a client are the only two devices that have the encryption keys in common so that they can understand each other's data. No other device should know about or be able to use the same keys to eavesdrop and decrypt the data. In Figure 3-4, the client's confidential password information has been encrypted before being transmitted. The AP can decrypt it successfully before forwarding it onto the wired network, but other wireless devices and users cannot.

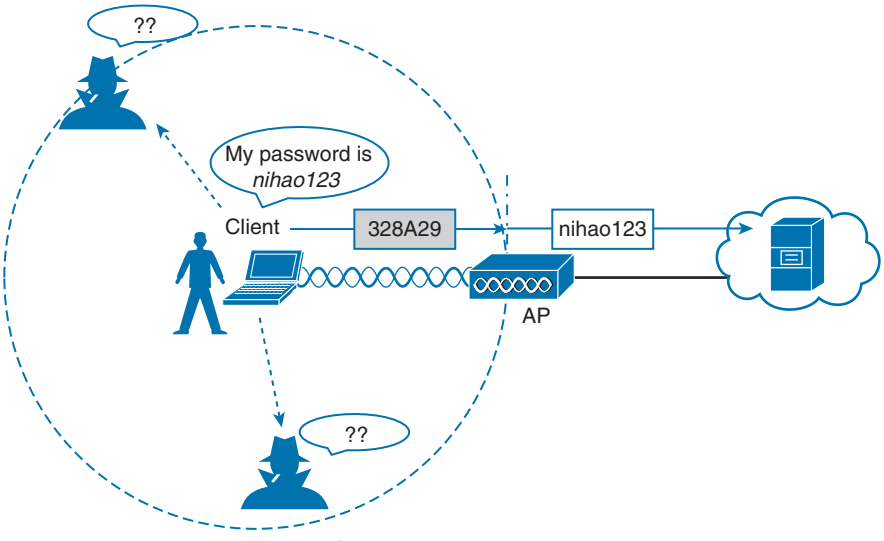



**Figure 3-4** *Encrypting Wireless Data to Protect Data Privacy*

The AP also maintains a "group key" that it uses when it needs to send encrypted data to all clients in its cell at one time. Each of the associated clients uses the same group key to decrypt the data.

## Message Integrity

Encrypting data obscures it from view while it is traveling over a public or untrusted network. The intended recipient should be able to decrypt the message and recover the original contents, but what if someone managed to alter the contents along the way? The recipient would have a very difficult time discovering that the original data had been modified.

A **message integrity check (MIC)** is a security tool that can protect against data tampering. You can think of a MIC as a way for the sender to add a secret stamp inside the encrypted data frame. The stamp is based on the contents of the data bits to be transmitted. After the recipient decrypts the frame, it can compare the secret stamp to its own idea of what the stamp should be, based on the data bits that were received. If the two stamps are identical, the recipient can safely assume that the data has not been tampered with. Figure 3-5 shows the MIC process.

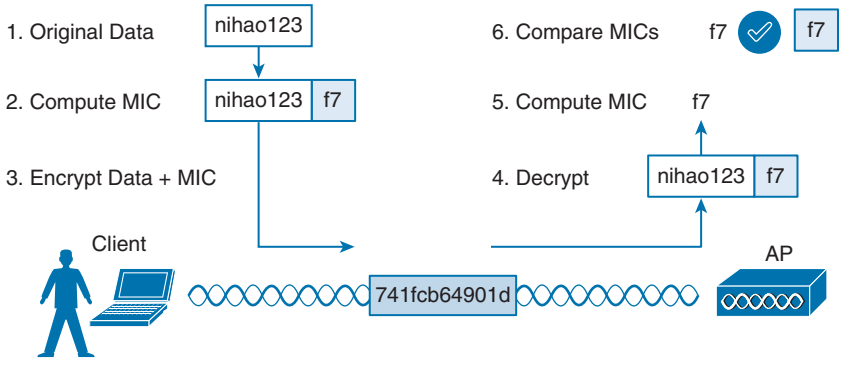



**Figure 3-5** *Checking Message Integrity over a Wireless Network*

# Wireless Client Authentication Methods

You can use many different methods to authenticate wireless clients as they try to associate with the network. The methods have been introduced over time and have evolved as security weaknesses have been exposed and wireless hardware has advanced. This section covers the most common authentication methods you might encounter.

## Open Authentication

The original 802.11 standard offered only two choices to authenticate a client: open authentication and WEP.

**Open authentication** is true to its name; it offers open access to a WLAN. The only requirement is that a client must use an 802.11 authentication request before it attempts to associate with an AP. No other credentials are needed.

When would you want to use open authentication? After all, it does not sound very secure because it is not. With no challenge, any 802.11 client may authenticate to access the network. That is, in fact, the whole purpose of open authentication—to validate that a client is a valid 802.11 device by authenticating the wireless hardware and the protocol. Authenticating the user's identity is handled as a true security process through other means.

You have probably seen a WLAN with open authentication when you have visited a public location. If any client screening is used at all, it comes in the form of web authentication. A client can associate right away but must open a web browser to see and accept the terms for use and enter basic credentials. From that point, network access is opened up for the client. Most client operating systems flag such networks to warn you that your wireless data will not be secured in any way if you join.

## WEP

As you might expect, open authentication offers nothing that can obscure or encrypt the data being sent between a client and an AP. As an alternative, the 802.11 standard has traditionally defined **Wired Equivalent Privacy (WEP)** as a method to make a wireless link more like or equivalent to a wired connection.

WEP uses the RC4 cipher algorithm to make every wireless data frame private and hidden from eavesdroppers. The same algorithm encrypts data at the sender and decrypts it at the receiver. The algorithm uses a string of bits as a key, commonly called a WEP key, to derive other encryption keys—one per wireless frame. As long as the sender and receiver have an identical key, one can decrypt what the other encrypts.



WEP is known as a shared-key security method. The same key must be shared between the sender and receiver ahead of time so that each can derive other mutually agreeable encryption keys. In fact, every potential client and AP must share the same key ahead of time so that any client can associate with the AP.

The WEP key can also be used as an optional authentication method as well as an encryption tool. Unless a client can use the correct WEP key, it cannot associate with an AP. The AP tests the client's knowledge of the WEP key by sending it a random challenge phrase. The client encrypts the challenge phrase with WEP and returns the result to the AP. The AP can compare the client's encryption with its own to see whether the two WEP keys yield identical results.

WEP keys can be either 40 or 104 bits long, represented by a string of 10 or 26 hex digits. As a rule of thumb, longer keys offer more unique bits for the algorithm, resulting in more robust encryption. Except in WEP's case, that is. Because WEP was defined in the original 802.11 standard in 1999, every wireless adapter was built with encryption hardware specific to WEP. In 2001, a number of weaknesses were discovered and revealed, so work began to find better wireless security methods. By 2004, the 802.11i amendment was ratified, and WEP was officially deprecated. Both WEP encryption and WEP shared-key authentication are widely considered to be weak methods to secure a wireless LAN.

## 802.1x/EAP

With only open authentication and WEP available in the original 802.11 standard, a more secure authentication method was needed. Client authentication generally involves some sort of challenge, a response, and then a decision to grant access. Behind the scenes, it can also involve an exchange of session or encryption keys, in addition to other parameters needed for client access. Each authentication method might have unique requirements as a unique way to pass information between the client and the AP.

Rather than build additional authentication methods into the 802.11 standard, a more flexible and scalable authentication framework, the **Extensible Authentication Protocol (EAP)**, was chosen. As its name implies, EAP is extensible and does not consist of any one authentication method. Instead, EAP defines a set of common functions that actual authentication methods can use to authenticate users. As you read through this section, notice how many authentication methods have EAP in their names. Each method is unique and different, but each one follows the EAP framework.

EAP has another interesting quality: it can integrate with the IEEE **802.1x** port-based access control standard. When 802.1x is enabled, it limits access to network media until a client

authenticates. This means that a wireless client might be able to associate with an AP but will not be able to pass data to any other part of the network until it successfully authenticates.

With open and WEP authentication, wireless clients are authenticated locally at the AP without further intervention. The scenario changes with 802.1x; the client uses open authentication to associate with the AP, and then the actual client authentication process occurs at a dedicated authentication server. Figure 3-6 shows the three-party 802.1x arrangement that consists of the following entities:

Key Topic

- **Supplicant:** The client device that is requesting access
- **Authenticator:** The network device that provides access to the network (usually a wireless LAN controller [WLC])
- **Authentication server (AS):** The device that takes user or client credentials and permits or denies network access based on a user database and policies (usually a **RADIUS server**)

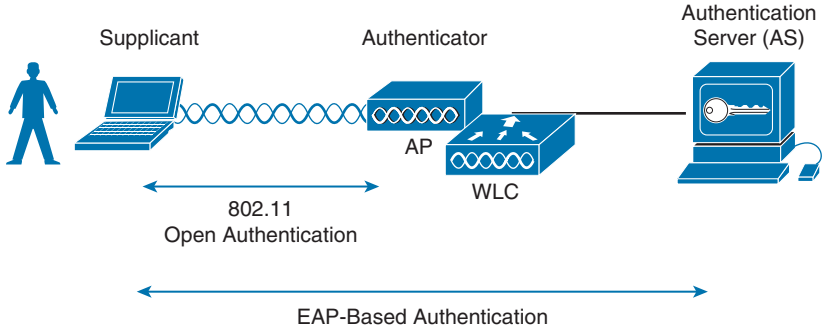



**Figure 3-6** *802.1x Client Authentication Roles*

The wireless LAN controller becomes a middleman in the client authentication process, controlling user access with 802.1x and communicating with the authentication server using the EAP framework.

The following sections provide an overview of several common EAP-based authentication methods. The goal here is to become aware of the many methods without trying to memorize them all. In fact, even when you configure user authentication on a wireless LAN, you will not have to select a specific method. Instead, you select "802.1x" on the WLC so that it is ready to handle a variety of EAP methods. It is then up to the client and the authentication server to use a compatible method. You will learn more about configuring security on a wireless LAN in Chapter 4, "Building a Wireless LAN."

### LEAP

As an early attempt to address the weaknesses in WEP, Cisco developed a proprietary wireless authentication method called **Lightweight EAP (LEAP)**. To authenticate, the client must supply username and password credentials. Both the authentication server and the client exchange challenge messages that are then encrypted and returned. This process provides mutual authentication; as long as the messages can be decrypted successfully, the client and the AS have essentially authenticated each other.

At the time, WEP-based hardware was still widely used. Therefore, LEAP attempted to overcome WEP weaknesses by using dynamic WEP keys that changed frequently. Nevertheless, the method used to encrypt the challenge messages was found to be vulnerable, so LEAP has since been deprecated. Even though wireless clients and controllers still offer LEAP, you should not use it.

### EAP-FAST

Cisco developed a more secure method called **EAP Flexible Authentication by Secure Tunneling (EAP-FAST)**. Authentication credentials are protected by passing a **protected access credential (PAC)** between the AS and the supplicant. The PAC is a form of shared secret that is generated by the AS and used for mutual authentication. EAP-FAST is a sequence of three phases:



- **Phase 0:** The PAC is generated or provisioned and installed on the client.
- **Phase 1:** After the supplicant and AS have authenticated each other, they negotiate a Transport Layer Security (TLS) tunnel.
- **Phase 2:** The end user can then be authenticated through the TLS tunnel for additional security.

Notice that two separate authentication processes occur in EAP-FAST: one between the AS and the supplicant and another with the end user. These occur in a nested fashion, as an outer authentication (outside the TLS tunnel) and an inner authentication (inside the TLS tunnel).

As with other EAP-based methods, a RADIUS server is required. However, the RADIUS server must also operate as an EAP-FAST server to be able to generate PACs, one per user.

### PEAP

Like EAP-FAST, the **Protected EAP (PEAP)** method uses an inner and outer authentication; however, the AS presents a digital certificate to authenticate itself with the supplicant in the outer authentication. If the supplicant is satisfied with the identity of the AS, the two will build a TLS tunnel to be used for the inner client authentication and encryption key exchange.

The digital certificate of the AS consists of data in a standard format that identifies the owner and is "signed" or validated by a third party. The third party is known as a **certificate authority (CA)** and is known and trusted by both the AS and the supplicants. The supplicant must also possess the CA certificate just so that it can validate the one it receives from the AS. The certificate is also used to pass a public key, in plain view, which can be used to help decrypt messages from the AS.

Notice that only the AS has a certificate for PEAP. That means the supplicant can readily authenticate the AS. The client does not have or use a certificate of its own, so it must be authenticated within the TLS tunnel using one of the following two methods:

- **MSCHAPv2:** Microsoft Challenge Authentication Protocol version 2
- **GTC:** Generic Token Card; a hardware device that generates one-time passwords for the user or a manually generated password

### EAP-TLS

PEAP leverages a digital certificate on the AS as a robust method to authenticate the RADIUS server. It is easy to obtain and install a certificate on a single server, but the clients are left to identify themselves through other means. **EAP Transport Layer Security (EAP-TLS)** goes one step further by requiring certificates on the AS and on every client device.

With EAP-TLS, the AS and the supplicant exchange certificates and can authenticate each other. A TLS tunnel is built afterward so that encryption key material can be securely exchanged.

EAP-TLS is considered to be the most secure wireless authentication method available; however, implementing it can sometimes be complex. Along with the AS, each wireless client must obtain and install a certificate. Manually installing certificates on hundreds or thousands of clients can be impractical. Instead, you would need to implement a **Public Key Infrastructure (PKI)** that could supply certificates securely and efficiently and revoke them when a client or user should no longer have access to the network. This usually involves setting up your own CA or building a trust relationship with a third-party CA that can supply certificates to your clients.

**NOTE** EAP-TLS is practical only if the wireless clients can accept and use digital certificates. Many wireless devices, such as communicators, medical devices, and RFID tags, have an underlying operating system that cannot interface with a CA or use certificates.

## Wireless Privacy and Integrity Methods

The original 802.11 standard supported only one method to secure wireless data from eavesdroppers: WEP. As you have learned in this chapter, WEP has been compromised, deprecated, and can no longer be recommended. What other options are available to encrypt data and protect its integrity as it travels through free space?

### TKIP

During the time when WEP was embedded in wireless client and AP hardware, yet was known to be vulnerable, the **Temporal Key Integrity Protocol (TKIP)** was developed.

TKIP adds the following security features using legacy hardware and the underlying WEP encryption:

- **MIC:** This efficient algorithm adds a hash value to each frame as a message integrity check to prevent tampering; commonly called “Michael” as an informal reference to MIC.
- **Time stamp:** A time stamp is added into the MIC to prevent replay attacks that attempt to reuse or replay frames that have already been sent.
- **Sender’s MAC address:** The MIC also includes the sender’s MAC address as evidence of the frame source.
- **TKIP sequence counter:** This feature provides a record of frames sent by a unique MAC address, to prevent frames from being replayed as an attack.

- **Key mixing algorithm:** This algorithm computes a unique 128-bit WEP key for each frame.
- **Longer initialization vector (IV):** The IV size is doubled from 24 to 48 bits, making it virtually impossible to exhaust all WEP keys by brute-force calculation.

TKIP became a reasonably secure stopgap security method, buying time until the 802.11i standard could be ratified. Some attacks have been created against TKIP, so it, too, should be avoided if a better method is available. In fact, TKIP was deprecated in the 802.11-2012 standard.



### CCMP

The **Counter/CBC-MAC Protocol (CCMP)** is considered to be more secure than TKIP. CCMP consists of two algorithms:

- AES counter mode encryption
- Cipher Block Chaining Message Authentication Code (CBC-MAC) used as a message integrity check (MIC)

The Advanced Encryption Standard (AES) is the current encryption algorithm adopted by the U.S. National Institute of Standards and Technology (NIST) and the U.S. government, and widely used around the world. In other words, AES is open, publicly accessible, and represents the most secure encryption method available today.

Before CCMP can be used to secure a wireless network, the client devices and APs must support the AES counter mode and CBC-MAC in hardware. CCMP cannot be used on legacy devices that support only WEP or TKIP. How can you know if a device supports CCMP? Look for the WPA2 designation, which is described in the following section.

### GCMP

The **Galois/Counter Mode Protocol (GCMP)** is a robust authenticated encryption suite that is more secure and more efficient than CCMP. GCMP consists of two algorithms:

- AES counter mode encryption
- Galois Message Authentication Code (GMAC) used as a message integrity check (MIC)

GCMP is used in WPA3, which is described in the following section.

## WPA, WPA2, and WPA3

This chapter covers a variety of authentication methods and encryption and message integrity algorithms. When it comes time to configure a WLAN with wireless security, should you try to select some combination of schemes based on which one is best or which one is not deprecated? Which authentication methods are compatible with which encryption algorithms?

The Wi-Fi Alliance (http://wi-fi.org), a nonprofit wireless industry association, has worked out straightforward ways to do that through its **Wi-Fi Protected Access (WPA)** industry certifications. To date, there are three different versions: WPA, WPA2, and WPA3. Wireless

products are tested in authorized testing labs against stringent criteria that represent correct implementation of a standard. As long as the Wi-Fi Alliance has certified a wireless client device and an AP and its associated WLC for the same WPA version, they should be compatible and offer the same security components.

The Wi-Fi Alliance introduced its first generation WPA certification (known simply as WPA and not WPA1) while the IEEE 802.11i amendment for best practice security methods was still being developed. WPA was based on parts of 802.11i and included 802.1x authentication, TKIP, and a method for dynamic encryption key management.

After 802.11i was ratified and published, the Wi-Fi Alliance included it in full in its **WPA Version 2 (WPA2)** certification. WPA2 is based around the superior AES CCMP algorithms, rather than the deprecated TKIP from WPA. It should be obvious that WPA2 was meant as a replacement for WPA.

In 2018, the Wi-Fi Alliance introduced **WPA Version 3 (WPA3)** as a future replacement for WPA2, adding several important and superior security mechanisms. WPA3 leverages stronger encryption by AES with the Galois/Counter Mode Protocol (GCMP). It also uses **Protected Management Frames (PMF)** to secure important 802.11 management frames between APs and clients, to prevent malicious activity that might spoof or tamper with a BSS's operation.

Table 3-2 summarizes the basic differences between WPA, WPA2, and WPA3. Each successive version is meant to replace prior versions by offering better security features. You should use WPA3 if it is widely available on all the wireless client devices, APs, and WLCs in your network; otherwise, use WPA2 and avoid WPA.

Key Topic

**Table 3-2** Comparing WPA, WPA2, and WPA3

| Authentication and Encryption Feature Support | WPA | WPA2 | WPA3* |
|---|---|---|---|
| Authentication with **pre-shared keys?** | Yes | Yes | Yes |
| Authentication with **802.1x?** | Yes | Yes | Yes |
| Encryption and MIC with **TKIP?** | Yes | No | No |
| Encryption and MIC with **AES and CCMP?** | Yes | Yes | No |
| Encryption and MIC with **AES and GCMP?** | No | No | Yes |

* WPA3 includes other features beyond WPA and WPA2, such as Simultaneous Authentication of Equals (SAE), forward secrecy, and Protected Management Frames (PMF).

Notice that all three WPA versions support two client authentication modes: a pre-shared key (PSK) or 802.1x, based on the scale of the deployment. These are also known as **personal mode** and **enterprise mode**, respectively.

With personal mode, a key string must be shared or configured on every client and AP before the clients can connect to the wireless network. The pre-shared key is normally kept confidential so that unauthorized users have no knowledge of it. The key string is never sent over the air. Instead, clients and APs work through a four-way handshake procedure that uses the pre-shared key string to construct and exchange encryption key material that can be openly exchanged. When that process is successful, the AP can authenticate the client, and the two can secure data frames that are sent over the air.

With WPA-Personal and WPA2-Personal modes, a malicious user can eavesdrop and capture the four-way handshake between a client and an AP. That user can then use a dictionary attack to automate guessing the pre-shared key. If successful, that user can then decrypt the wireless data or even join the network posing as a legitimate user.

WPA3-Personal avoids such an attack by strengthening the key exchange between clients and APs through a method known as **Simultaneous Authentication of Equals (SAE)**. Rather than a client authenticating against a server or AP, the client and AP can initiate the authentication process equally and even simultaneously.



Even if a password or key is compromised, WPA3-Personal offers **forward secrecy**, which prevents attackers from being able to use a key to unencrypt data that has already been transmitted over the air.

**TIP** The Personal mode of any WPA version is usually easy to deploy in a small environment or with clients that are embedded in certain devices because a simple text key string is all that is needed to authenticate the clients. Be aware that every device using the WLAN must be configured with an identical pre-shared key. If you ever need to update or change the key, you must touch every device to do so. As well, the pre-shared key should remain a well-kept secret; you should never divulge the pre-shared key to any unauthorized person.

Notice from Table 3-2 that WPA, WPA2, and WPA3 also support 802.1x or enterprise authentication. This support implies EAP-based authentication, but the WPA versions do not require any specific EAP method. Instead, the Wi-Fi Alliance certifies interoperability with well-known EAP methods like EAP-TLS, PEAP, EAP-TTLS, and EAP-SIM. Enterprise authentication is more complex to deploy than personal mode because authentication servers must be set up and configured as a critical enterprise resource.

**TIP** The Wi-Fi Alliance has made wireless security configuration straightforward and consistent through its WPA, WPA2, and WPA3 certifications. Each version is meant to replace its predecessors because of improved security mechanisms. You should always select the highest WPA version that the clients and wireless infrastructure in your environment will support.

## Chapter Review

At this point in the chapter, you might still be a little overwhelmed with the number of acronyms and security terms to learn and keep straight in your mind. Spend some time reviewing Table 3-3, which lists all of the topics described in this chapter. The table is organized in a way that should help you remember how the acronyms and functions are grouped together. Remember that an effective wireless security strategy includes a method to authenticate clients and a method to provide data privacy and integrity. These two types of methods are listed in the leftmost column. Work your way to the right to remember what types of authentication and privacy/integrity are available. The table also expands the name of each acronym as a memory tool.

Also remember that WPA, WPA2, and WPA3 simplify wireless network configuration and compatibility because they limit which authentication and privacy/integrity methods can be used.

Key Topic

**Table 3-3** Review of Wireless Security Mechanisms and Options

| Security Mechanism | Type | | Type Expansion | Credentials Used |
|---|---|---|---|---|
| Authentication Methods | Open | | Open Authentication | None, other than 802.11 protocol |
| | WEP | | Wired Equivalent Privacy | Static WEP keys |
| | 802.1x/EAP (Extensible Authentication Protocol) | LEAP | Lightweight EAP | Deprecated; uses dynamic WEP keys |
| | | EAP-FAST | EAP Flexible Authentication by Secure Tunneling | Uses protected access credential (PAC) |
| | | PEAP | Protected EAP | AS authenticated by digital certificate |
| | | EAP-TLS | EAP Transport Layer Security | Client and AS authenticated by digital certificate |
| Privacy & Integrity Methods | TKIP | | Temporal Key Integrity Protocol | N/A |
| | CCMP | | Counter/CBC-MAC Protocol | N/A |
| | GCMP | | Galois/Counter Mode Protocol | N/A |

You should also review this chapter's material using either the tools in the book or the interactive tools for the same material found on the book's companion website. Table 3-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 3-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Review the most important topics in this chapter, noted with the Key Topic icon in the outer margin of the page. Table 3-5 lists a reference of these key topics and the page numbers on which each is found.

Key Topic

**Table 3-5** Key Topics for Chapter 3

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | 802.1x entities | 48 |
| Table 3-2 | WPA, WPA2, and WPA3 comparison | 52 |
| Table 3-3 | Wireless security mechanism review | 54 |

## Key Terms You Should Know

802.1x, authentication server (AS), authenticator, certificate authority (CA), Counter/CBC-MAC Protocol (CCMP), EAP Flexible Authentication by Secure Tunneling (EAP-FAST), EAP Transport Layer Security (EAP-TLS), enterprise mode, Extensible Authentication Protocol (EAP), forward secrecy, Galois/Counter Mode Protocol (GCMP), Lightweight EAP (LEAP), message integrity check (MIC), open authentication, personal mode, protected access credential (PAC), Protected EAP (PEAP), Protected Management Frame (PMF), Public Key Infrastructure (PKI), RADIUS server, Simultaneous Authentication of Equals (SAE), supplicant, Temporal Key Integrity Protocol (TKIP), Wi-Fi Protected Access (WPA), Wired Equivalent Privacy (WEP), WPA Version 2 (WPA2), WPA Version 3 (WPA3)



CHAPTER 4

# Building a Wireless LAN

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.1 Explain the role and function of network components**
    - **1.1.e Controllers**
- **2.0 Network Access**
  - **2.7 Describe physical infrastructure connections of WLAN components (AP, WLC, access/trunk ports, and LAG)**
  - **2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, and TACACS+/RADIUS)**
  - **2.9 Interpret the wireless LAN GUI configuration for client connectivity, such as WLAN creation, security settings, QoS profiles, and advanced settings**
- **5.0 Security Fundamentals**
  - **5.10 Configure and verify WLAN within the GUI using WPA2 PSK**

In Chapters 1 through 3, you learned about the fundamentals of wireless networks. As a CCNA, you will also need to know how to apply that knowledge toward building a functioning network with APs and a WLC.

In addition, based on the concepts you learned in Chapter 3, "Securing Wireless Networks," you will be able to configure the WLAN to use WPA2-Personal (WPA2-PSK).

Before getting into the chapter, be aware that Cisco no longer uses the original WLC operating system, AireOS. Instead, WLCs run IOS XE. Newer WLCs with IOS XE have a CLI, as do many enterprise-class Cisco routers, but you configure WLANs from the WLC GUI. However, the AireOS and IOS XE GUIs differ, both in the GUI pages' styling and configuration elements. This chapter moves back and forth through examples of each so you can learn the ideas and compare the differences and similarities.

For the exam, exam topics 2.9 and 5.10 refer to details visible from the WLC—but they do not mention for which operating system. You should be ready for both, so we include both.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 4-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Connecting a Cisco AP | 1, 2 |
| Accessing a Cisco WLC | 3 |
| Connecting a Cisco WLC | 4, 5 |
| Configuring a WLAN | 6–8 |

1. Suppose you need to connect a Cisco AP to a network. Which one of the following link types would be necessary?
   a. Access mode link
   b. Trunk mode link
   c. LAG mode link
   d. EtherChannel link
2. An autonomous AP will be configured to support three WLANs that correspond to three VLANs. The AP will connect to the network over which one of the following?
   a. Access mode link
   b. Trunk mode link
   c. LAG mode link
   d. EtherChannel link
3. Suppose you would like to connect to a WLC to configure a new WLAN on it. Which one of the following protocols can be used to access the WLC?
   a. SSH
   b. HTTPS
   c. HTTP
   d. All of these answers are correct.
4. Which one of the following correctly describes the single logical link formed by bundling all of a controller's distribution system ports together?
   a. PHY
   b. DSP
   c. LAG
   d. GEC
5. Which one of the following controller interfaces is used on an AireOS controller to map a WLAN to a VLAN?
   a. Bridge interface
   b. Virtual interface
   c. WLAN interface
   d. Dynamic interface

6. Which of the following things are bound together when a new WLAN is created? (Choose two answers.)
   a. VLAN
   b. AP
   c. CAPWAP tunnel
   d. SSID
7. What is the maximum number of WLANs you can configure on a Cisco wireless controller?
   a. 8
   b. 16
   c. 512
   d. 1024
8. Which of the following parameters are necessary when creating a new WLAN on an IOS-XE controller? (Choose all that apply.)
   a. WLAN profile
   b. Channel number
   c. Policy profile
   d. BSSID
   e. IP subnet

# Foundation Topics

## Connecting a Cisco AP

A Cisco wireless network consists of APs that are coupled with one or more wireless LAN controllers. An AP's most basic function is to connect wireless devices to a wired network. Therefore, you should understand how to connect the wired side of an AP so that it can pass traffic between the appropriate WLANs and VLANs.

Recall that an autonomous AP is a standalone device; nothing else is needed to forward Ethernet frames from a wired VLAN to a wireless LAN, and vice versa. In effect, the AP maps each VLAN to a WLAN and BSS. The autonomous AP has a single wired Ethernet interface, as shown in the left portion of Figure 4-1, which means that multiple VLANs must be brought to it over a trunk link.

**TIP** A switch port providing a wired connection to an AP must be configured to support either access or trunk mode. In trunk mode, 802.1Q encapsulation tags each frame according to the VLAN number it came from. The wireless side of an AP inherently trunks 802.11 frames by marking them with the BSSID of the WLAN where they belong.

A Cisco AP also has a single wired Ethernet interface; however, it must be paired with a WLC to be fully functional. Wired VLANs that terminate at the WLC can be mapped to WLANs that emerge at the AP. Even though multiple VLANs are being extended from the

WLC to the AP, they are all carried over the CAPWAP tunnel between the two. That means the AP needs only an access link to connect to the network infrastructure and terminate its end of the tunnel, as shown in the right portion of Figure 4-1.

Key Topic

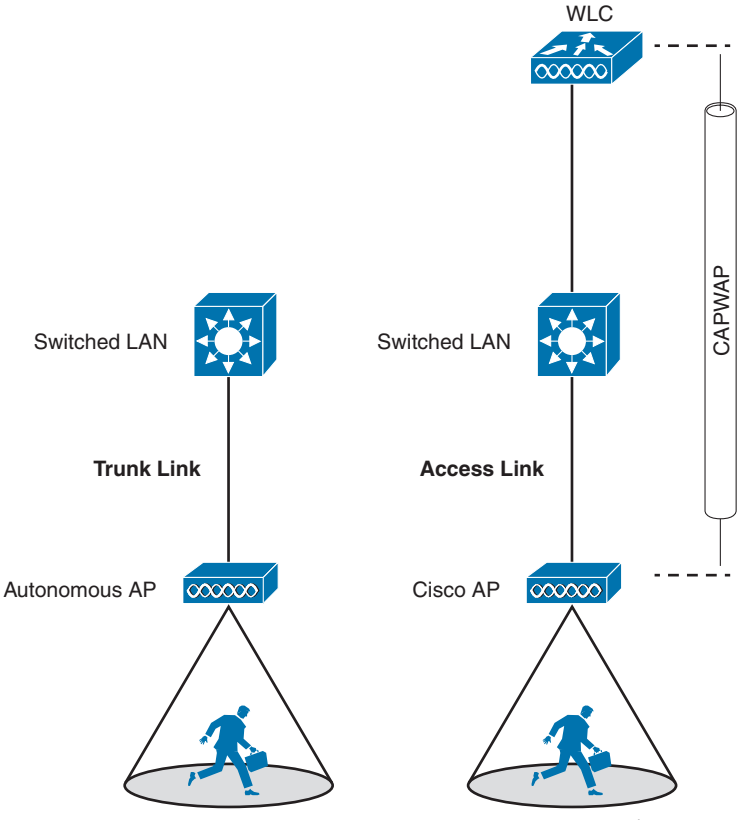



**Figure 4-1** *Comparing Connections to Autonomous and Cisco APs*

Cisco APs are normally and most efficiently managed via a browser session to the WLC; however, you can directly connect a serial console cable from your PC to the console port on the AP to monitor its bootup process or to make some basic configuration changes if needed. When the AP is operational and has an IP address, you can also use Telnet or SSH to connect to its CLI over the wired network.

## Accessing a Cisco WLC

To connect and configure a WLC, you need to open a web browser to the WLC's management address using either HTTP or HTTPS. You can do this only after the WLC has an initial configuration, a management IP address assigned to its management interface, and has built a valid SSL certificate for HTTPS use. The web-based GUI provides an effective way to monitor, configure, and troubleshoot a wireless network. You can also connect to a WLC with an SSH session, where you can use its CLI to monitor, configure, and debug activity.

Both the web-based GUI and the CLI require management users to log in. Users can be authenticated against an internal list of local usernames or against an authentication, authorization, and accounting (AAA) server, such as TACACS+ or RADIUS.

When you first open a web browser to the management address, you will see the initial login screen. Click the **Login** button, as shown in Figure 4-2 (IOS-XE controller) and Figure 4-3 (AireOS controller); then enter your user credentials as you are prompted for them.

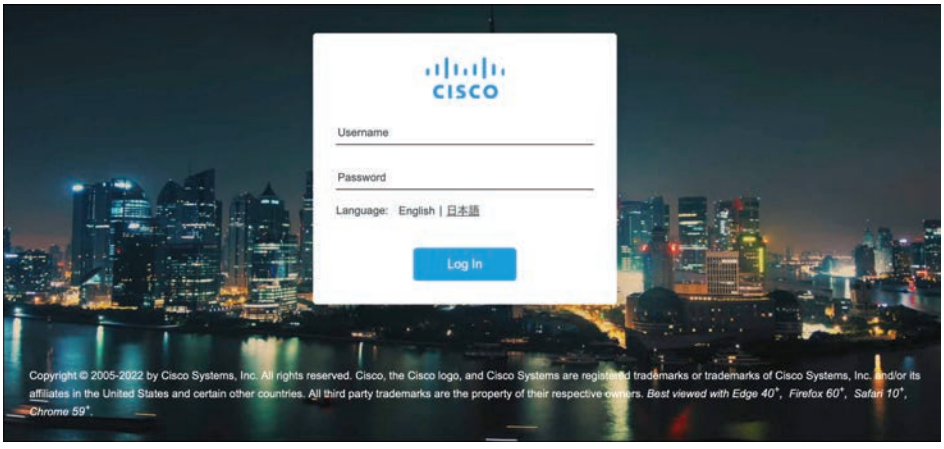



**Figure 4-2** *Accessing an IOS-XE WLC with a Web Browser*




**Figure 4-3** *Accessing an AireOS WLC with a Web Browser*

Answers to the "Do I Know This Already?" quiz:

**1** A **2** B **3** D **4** C **5** D **6** A, D **7** C **8** A, C

**NOTE** The CCNA exam objectives focus on using the WLC GUI to configure a WLAN and a security suite. Therefore, the examples in this section assume that someone has already entered an initial configuration to give the WLC a working IP address for management.

When you are successfully logged in, the WLC will display a monitoring dashboard similar to the one shown in Figure 4-4 (IOS-XE) and Figure 4-5 (AireOS). You will not be able to make any configuration changes there, so you must select **Configuration** in the left column (IOS-XE) or click on the **Advanced** link in the upper-right corner (AireOS).

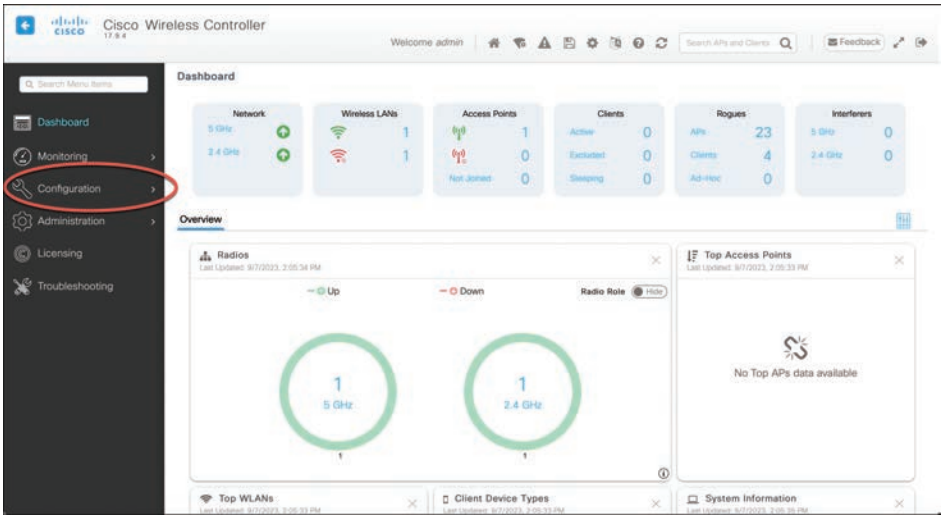



**Figure 4-4** *Accessing the IOS-XE WLC Configuration Menus*

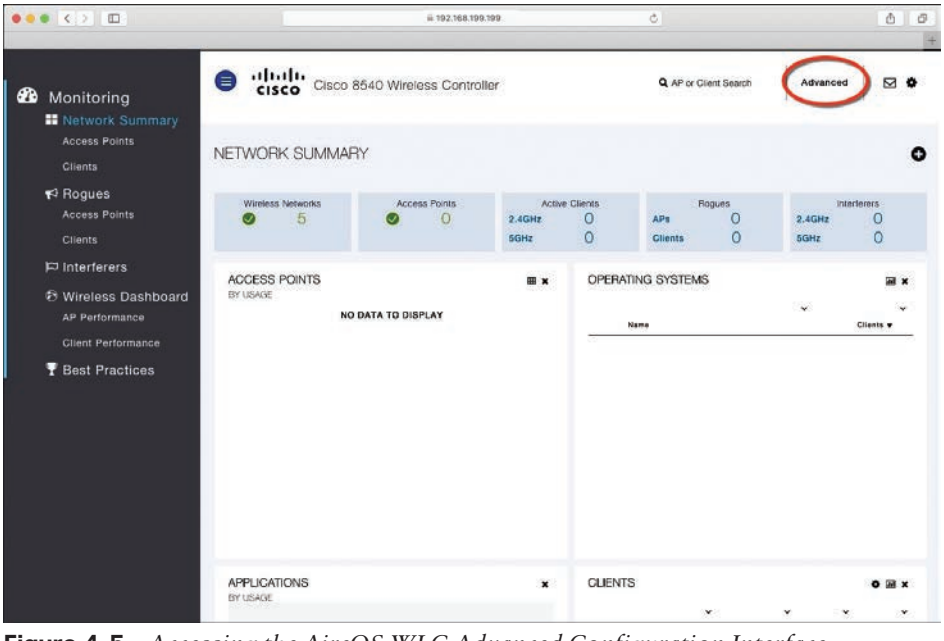



**Figure 4-5** *Accessing the AireOS WLC Advanced Configuration Interface*

On an IOS-XE WLC, you can select from a large list of configuration categories, as shown in Figure 4-6. In contrast, an AireOS WLC displays tabs across the top of the screen, as shown in Figure 4-7, which presents a list of functions on the left side of the screen. You will get a feel for which menu and list items you should use on both types of controller as you work through the remainder of the chapter.

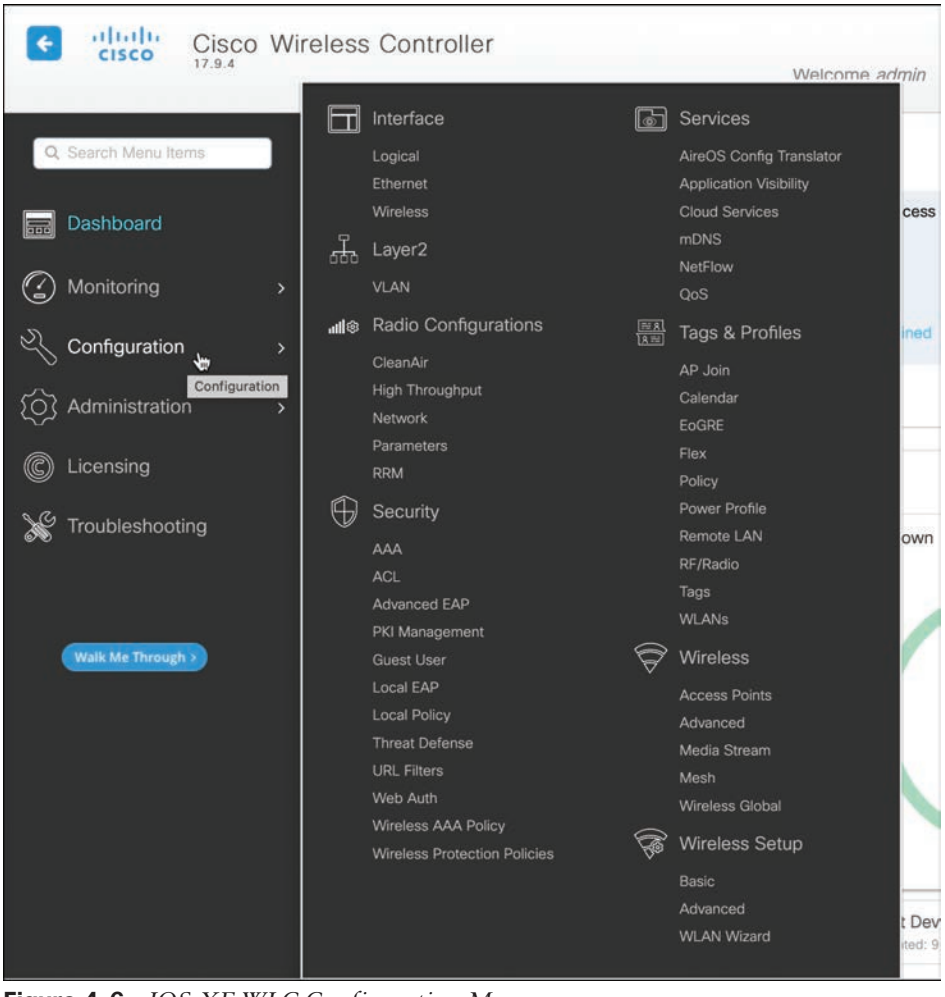



**Figure 4-6** *IOS-XE WLC Configuration Menus*

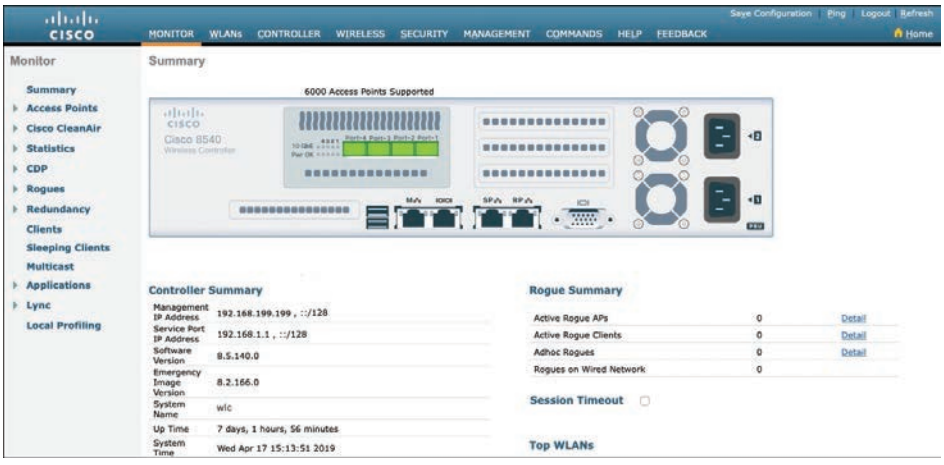



**Figure 4-7** *AireOS WLC Advanced Configuration Categories*

4

## Connecting a Cisco WLC

Cisco wireless LAN controllers (WLCs) offer several different types of ports and connections. The sections that follow explain each connection type in more detail. You learn more about configuring WLC ports in the "Configuring a WLAN" for IOS-XE and AireOS sections later in the chapter.

### WLC Physical Ports

A WLC has several different types of physical ports you can connect to your network, as shown in Figure 4-8. For example, you can connect to a serial *console port* for initial boot functions and system recovery. An Ethernet *service port* is used for out-of-band management via SSH or a web browser. This is sometimes called the *device management interface*. A *redundancy port* connects to a peer controller for high availability (HA) operation.

Key Topic

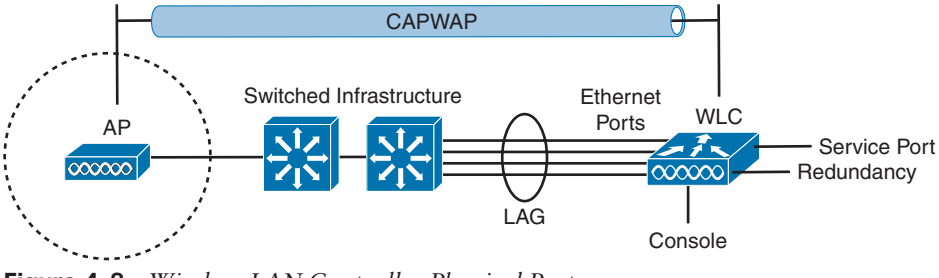



**Figure 4-8** *Wireless LAN Controller Physical Ports*

Controllers also have multiple Ethernet ports that you must connect to the network. These ports carry most of the data coming to and going from the controller. For example, both control and data CAPWAP tunnels that extend to each of a controller's APs pass across these ports. In addition, any management traffic using a web browser, SSH, Simple Network Management Protocol (SNMP), Trivial File Transfer Protocol (TFTP), and so on, normally reaches the controller in-band through the ports.

**NOTE** The Ethernet ports on an AireOS controller are called *distribution system ports*. You might be thinking that is an odd name for what appear to be regular data ports. Recall from the section titled "Wireless LAN Topologies" in Chapter 1, "Fundamentals of Wireless Networks," that the wired network that connects APs together is called the distribution system (DS). With the split MAC architecture, the point where APs touch the DS is moved upstream to the WLC instead.

Because the Ethernet ports must carry data that is associated with many different VLANs, VLAN tags and numbers become very important. Later in this chapter, you learn how the controller maps VLANs to wireless LANs. The Ethernet ports on an IOS-XE controller should always be configured to operate in 802.1Q trunking mode. AireOS controller ports can operate only in trunking mode and cannot be configured otherwise. When you connect the controller ports to a switch, you should also configure the switch ports for unconditional 802.1Q trunk mode to match.

The controller's Ethernet ports can operate independently, each one transporting multiple VLANs to a unique group of internal controller interfaces. For resiliency, the ports can be configured as a link aggregation group (LAG) such that they are bundled together to act as one larger link, much like an EtherChannel or port channel on a switch. In fact, the switch ports where the controller ports connect must also be configured as a port channel. With a LAG configuration, traffic can be load-balanced across the individual ports that make up the LAG. In addition, LAG offers resiliency; if one or more individual ports fail, traffic will be redirected to the remaining working ports instead.

Cisco wireless controllers must provide the necessary connectivity between wireless LANs and wired VLANs. The controller can touch VLANs through its physical Ethernet ports, but WLANs are carried over CAPWAP tunnels and terminate internally. Therefore, the controller must use internal dynamic interfaces that map between VLANs and WLANs, as shown in Figure 4-9.

Key Topic

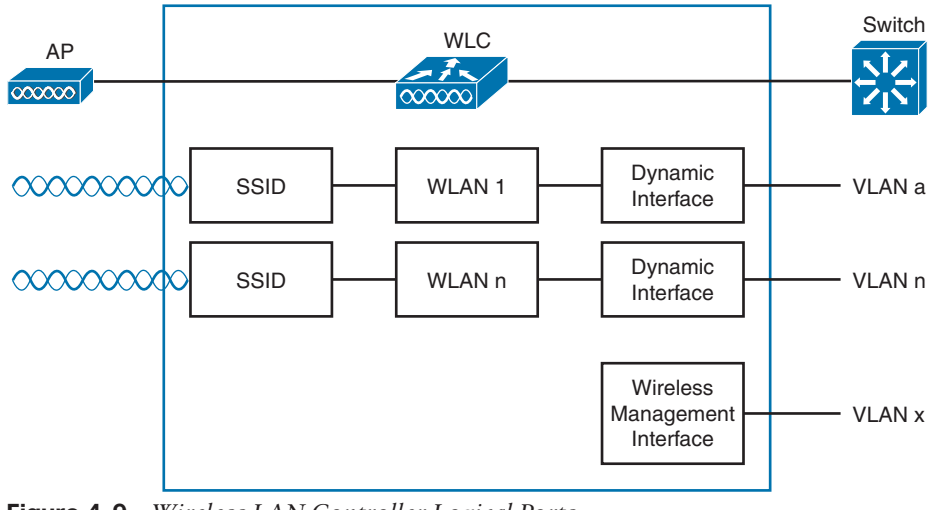



**Figure 4-9** *Wireless LAN Controller Logical Ports*

The dynamic interfaces on an IOS-XE controller work at Layer 2, so the controller doesn't need a Layer 3 IP address on each VLAN. In contrast, an AireOS controller must have an IP address, subnet mask, default gateway, and a Dynamic Host Configuration Protocol (DHCP) server configured on each of its dynamic interfaces that touch a VLAN.

Both IOS-XE and AireOS controller platforms require a wireless management interface (WMI) for all in-band management traffic. The interface is used for normal management traffic, such as RADIUS user authentication, WLC-to-WLC communication, web-based and SSH sessions, SNMP, Network Time Protocol (NTP), syslog, and so on. The management interface is also used to terminate CAPWAP tunnels between the controller and its APs.

The WMI uses an IP address, subnet mask, and default gateway to allow the controller to communicate on the network. The WMI is usually connected to a management VLAN on an upstream switch. On IOS-XE controllers, the WMI is actually a switched virtual interface (SVI) and has the only configured IP address on the entire controller.



The virtual interface is used for only certain client-facing operations. For example, when a wireless client issues a request to obtain an IP address, the controller can relay the request on to an actual DHCP server that can provide the appropriate IP address. From the client's perspective, the DHCP server appears to be the controller's virtual interface address. Clients may see the virtual interface's address, but that address is never used when the controller communicates with other devices on the switched network. You should configure the virtual interface with a unique, nonroutable address such as 10.1.1.1 that is within a private address space defined in RFC 1918.

The virtual interface address is also used to support client mobility. For that reason, every controller that exists in the same mobility group should be configured with a virtual address that is identical to the others. By using one common virtual address, all the controllers will appear to operate as a cluster as clients roam from controller to controller.

## Configuring a WLAN

A wireless LAN controller and an access point work in concert to provide network connectivity to wireless clients. From a wireless perspective, the AP advertises a Service Set Identifier (SSID) for wireless clients to join. From a wired perspective, the controller connects to a virtual LAN (VLAN) through one of its dynamic interfaces. To complete the path between the SSID and the VLAN, as illustrated in Figure 4-10, you must first define a WLAN on the controller.

**NOTE** Two of the CCNA exam objectives involve configuring a WLAN for client connectivity with WPA2 and a PSK using only the controller GUI. As you work through this section, you will find that it presents a complete WLAN example that is based on the topology shown in Figure 4-10 using the WPA2-Personal (PSK) security model.

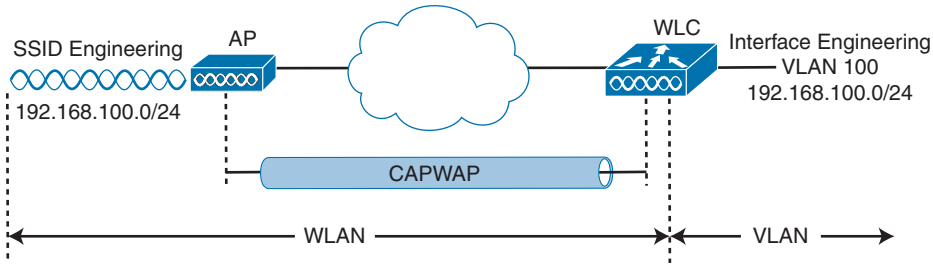



**Figure 4-10** *Connecting Wired and Wireless Networks with a WLAN*

The controller will bind the WLAN to one of its dynamic interfaces and then push the WLAN configuration out to all of its APs by default. From that point on, wireless clients will be able to learn about the new WLAN by receiving its beacons and will be able to probe and join the new BSS.

You can use WLANs, like VLANs, to segregate wireless users and their traffic into logical networks. Users associated with one WLAN cannot cross over into another one unless their traffic is bridged or routed from one VLAN to another through the wired network infrastructure.

Before you begin to create new WLANs, it is usually wise to plan your wireless network first. In a large enterprise, you might have to support a wide variety of wireless devices, user communities, security policies, and so on. You might be tempted to create a new WLAN for every occasion, just to keep groups of users isolated from each other or to support different types of devices. Although that is an appealing strategy, you should be aware of two limitations:

- Cisco controllers support a maximum of 512 WLANs, but only 16 of them can be actively configured on an AP.
- Advertising each WLAN to potential wireless clients uses up valuable airtime.

Every AP must broadcast beacon management frames at regular intervals to advertise the existence of a BSS. Because each WLAN is bound to a BSS, each WLAN must be advertised with its own beacons. Beacons are normally sent 10 times per second, or once every 100 ms, at the lowest mandatory data rate. The more WLANs you have created, the more beacons you will need to announce them.

Even further, the lower the mandatory data rate, the more time each beacon will take to be transmitted. The end result is this: if you create too many WLANs, a channel can be starved of its usable airtime. Clients will have a hard time transmitting their own data because the channel is overly busy with beacon transmissions coming from the AP. As a rule of thumb, always limit the number of WLANs to five or fewer; a maximum of three WLANs is best.

By default, a controller has a limited initial configuration, so no WLANs are defined. Before you create a new WLAN, think about the following parameters it will need to have:

- SSID string
- Controller interface and VLAN number
- Type of wireless security needed

The sections that follow demonstrate how to create a WLAN on an IOS-XE controller and then an AireOS controller. Each configuration step is performed using a web browser session that is connected to the WLC's management IP address.

## Configuring a WLAN on an IOS-XE WLC

The IOS-XE wireless controller platform is very versatile and powerful, giving you granular control over every part of the wireless network configuration. You can configure all of the network's APs the same, in a global fashion, or you can tailor their configurations depending on their location or some other common requirements. For example, your enterprise might consist of many buildings. You might want the APs in one building to offer WLANs on only one band. Perhaps you want a group of APs to offer only a subset of the entire list of WLANs. In other buildings, you might need to support a different set of constraints.

With an IOS-XE controller, you can configure and apply the parameters that define AP operation in three general categories:

- **Policy:** Things that define each wireless LAN and security policies
- **Site:** Things that affect the AP-controller and CAPWAP relationship and FlexConnect behavior on a per-site basis
- **RF:** Things that define the RF operation on each wireless band

Each of these three categories is applied to each AP in the network through configuration *profiles* and *tags*. You can define policy, site, and RF profiles that contain the desired customizations. Then each AP is tagged to identify which policy, site, and RF profiles it should use. Figure 4-11 illustrates this concept, along with a list of the relevant parameters you can customize in each profile type.

Key Topic

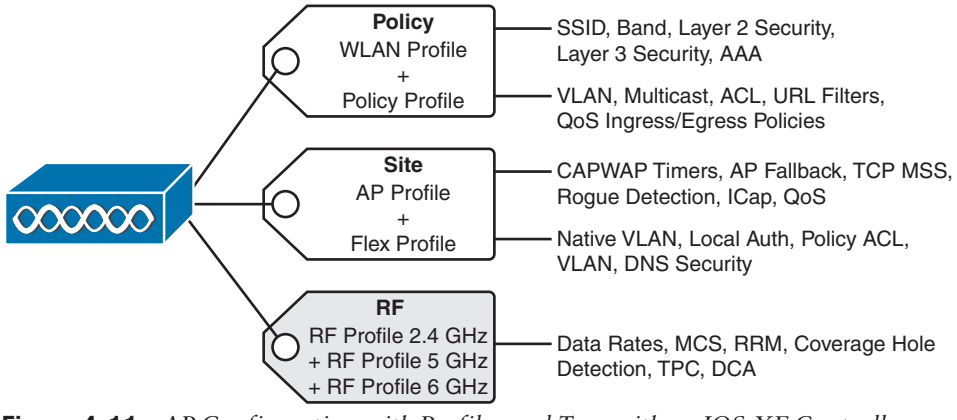



**Figure 4-11** *AP Configuration with Profiles and Tags with an IOS-XE Controller*

**NOTE** Although it's possible to fine-tune a wide variety of AP options, the CCNA exam is focused only on the WLAN profile and Policy profile that you can map to APs with the policy tag. In other words, you should be concerned with only the things that pertain to the topmost tag in Figure 4-11. The CCNP ENCOR exam goes into further detail about the other tags and profiles.

The policy tag maps two different profiles: a WLAN profile that defines a list of SSIDs and WLAN security that an AP will offer, and a policy profile that defines how the AP will handle various types of traffic.

You can begin configuring a new WLAN by navigating to **Configuration > Wireless Setup > WLANs**, then selecting the **Start Now** button. The controller will display a "timeline," or the full sequence of all profiles and tags that you can configure, as shown in Figure 4-12. For the purposes of CCNA study, only the highlighted items are discussed in this chapter.

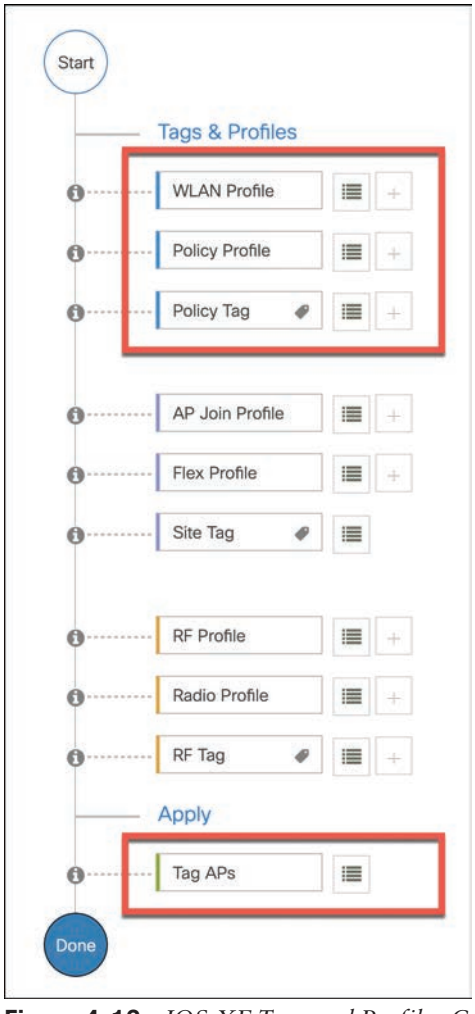



**Figure 4-12** *IOS-XE Tags and Profiles Configuration Sequence*

As you might guess from the figure, configuring a new WLAN requires the following four steps:

**Step 1.** Configure a WLAN profile.

**Step 2.** Configure a policy profile.

**Step 3.** Map the WLAN and policy profiles to a policy tag.

**Step 4.** Apply the policy tag to some APs.

As you work through the WLAN configuration steps that follow, be aware that you can select the small "list" icons in the Tags & Profiles task sequence (see Figure 4-12) to display a list of related profiles or tags that already exist on the controller. You can then select one from the list to edit, or select the **Add** button to add a new one. Otherwise, you can immediately begin creating a new profile or tag by selecting the small + (plus) icon to the right of the profile or tag item.



### Step 1: Configure a WLAN Profile

Select the **+** icon to the right of WLAN Profile. Beginning with the Add WLAN > General tab, as shown in Figure 4-13, you will be prompted to enter text strings for the WLAN profile name and the SSID (1–32 characters). By default, the WLAN profile name will be copied into the SSID field, but you can edit it if needed. The WLAN ID is simply a number that indexes the various WLANs that are configured on the controller.

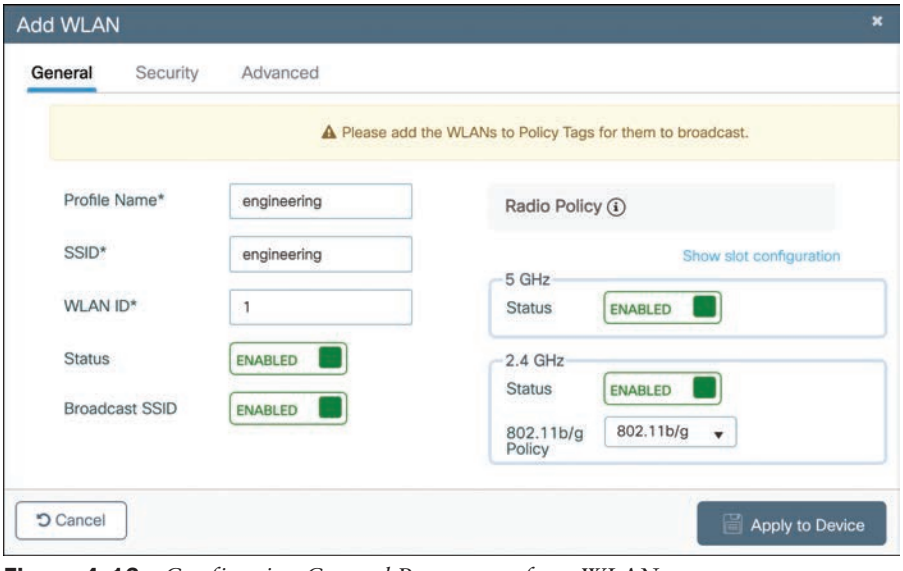



**Figure 4-13** *Configuring General Parameters for a WLAN*

You can change the WLAN Status to Enabled so that it will be put into active use. You should also enable Broadcast SSID to allow APs to advertise the SSID to potential wireless clients.

The General tab also provides you with the opportunity to select which frequency bands to use for the WLAN. By default, all supported bands are enabled, allowing wireless clients to choose the band according to their internal algorithms.

Because the 2.4-GHz band is often crowded with nearby unrelated networks, you could disable it on your own APs and use only the higher frequency bands instead. The 5- and 6-GHz bands (6-GHz band not pictured in Figure 4-13) are much less crowded with competing APs and offer much better performance—desirable qualities for wireless applications like voice and video. In the 2.4-GHz band, you can also select the 802.11 policy to use. By default, both the 802.11b and g amendments are supported. You can select **802.11g-only** to completely disable the slower legacy data rates used by 802.11b devices.

Next, select the **Security** tab to configure WLAN security parameters. Figure 4-14 shows the Layer2 tab contents. Notice that there are options running across the screen for WPA+WPA2, WPA2+WPA3, WPA3, Static WEP, and None. The options relevant for the sample scenario in this chapter (and the CCNA exam) are highlighted in the figure. The scenario uses WPA2 with a PSK, so you could select either WPA+WPA2 or WPA2+WPA3, then move to the WPA Parameters section and uncheck the box next to the WPA version you do not want to use.

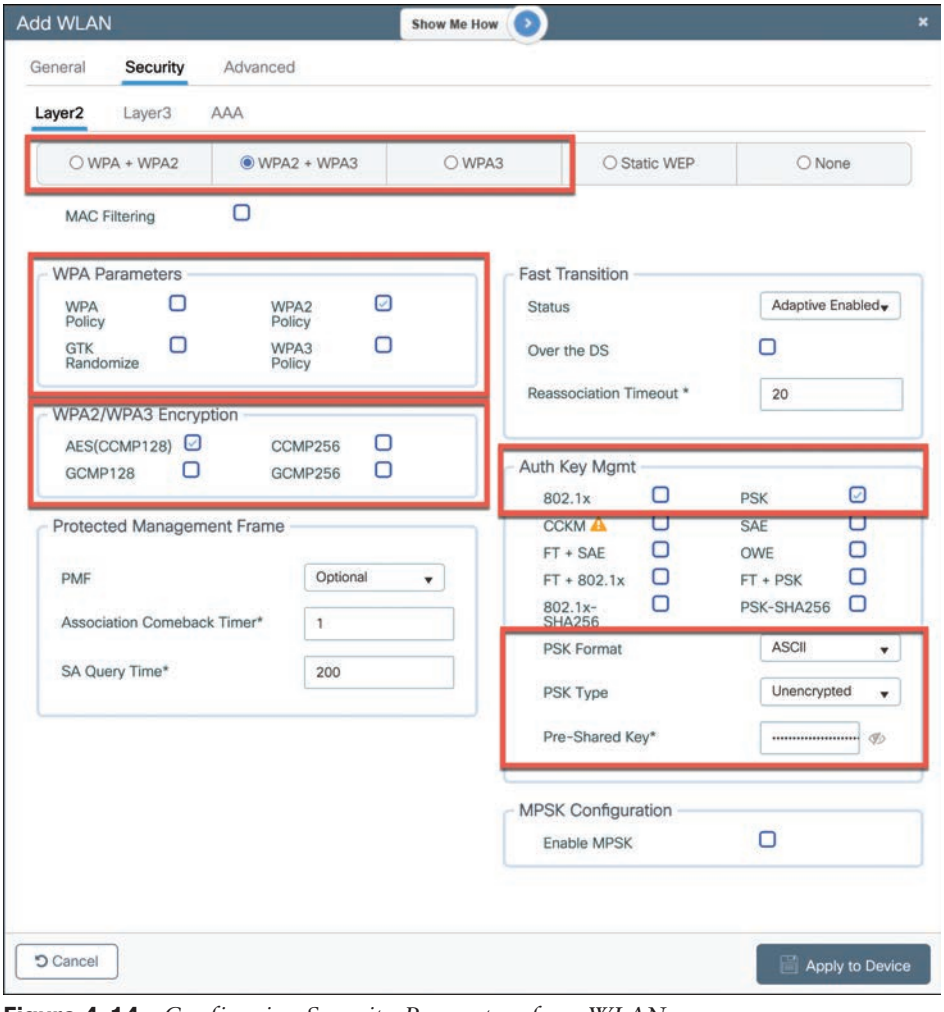



**Figure 4-14** *Configuring Security Parameters for a WLAN*

With the WPA2 policy, you can select **AES(CCMP128)** encryption (the default), plus **PSK**, then enter the pre-shared key text string (8–63 ASCII characters). You can also enter the PSK as a hexadecimal string (exactly 64 digits), if desired.

In Figure 4-14, in the Auth Key Mgmt section, notice that PSK is checked but 802.1x is not. If you want the WLAN to use WPA2 Enterprise instead, then 802.1x would be necessary to support user authentication and the EAPOL four-way handshake for encryption key material exchange. You would also have to define a RADIUS, ISE, or LDAP server under the Security > AAA tab.

You might want to allow 802.11r, also known as Fast Transition (FT), to streamline wireless client roaming and reauthentication as clients move throughout the WLAN. FT options are displayed in the Fast Transition section. By default, the FT adaptive mode is enabled, which allows a mix of clients that are 802.11r-capable and clients that are not.

The Security > Layer3 tab, as shown in Figure 4-15, contains a few parameters related to Web authentication (webauth). Figure 4-16 shows the Security > AAA tab, where you can apply an authentication list that contains AAA servers that will authenticate users. You can also enable Local EAP Authentication to have the controller perform the RADIUS function instead of a dedicated external server.



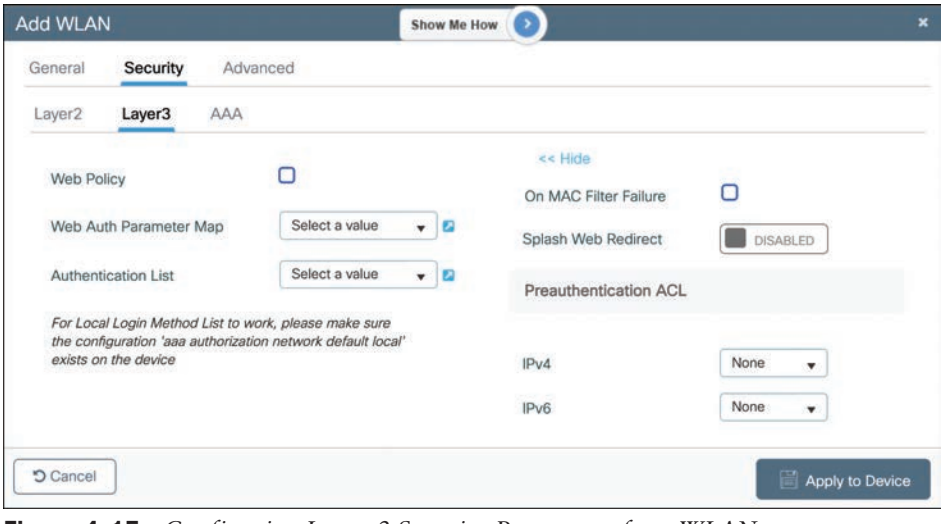



**Figure 4-15** *Configuring Layer 3 Security Parameters for a WLAN*

The Add WLAN > Advanced tab contains a large collection of options that affect many different controller and AP operations. Figure 4-17 shows the first half of the options.

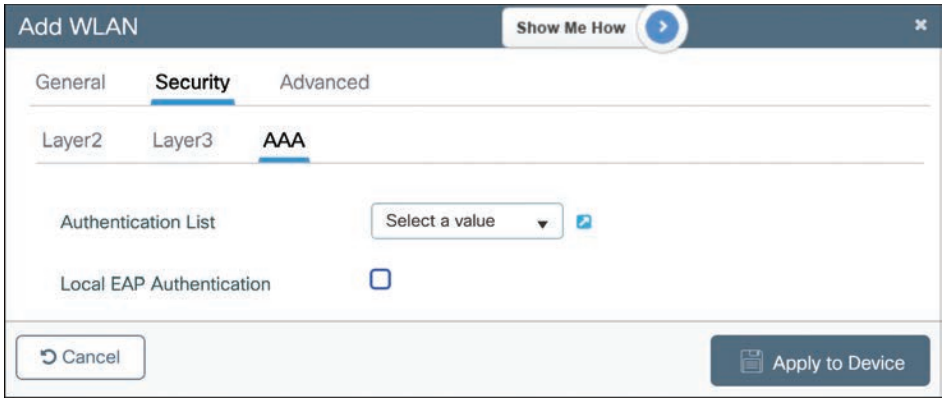



**Figure 4-16** *Configuring AAA Parameters for a WLAN*

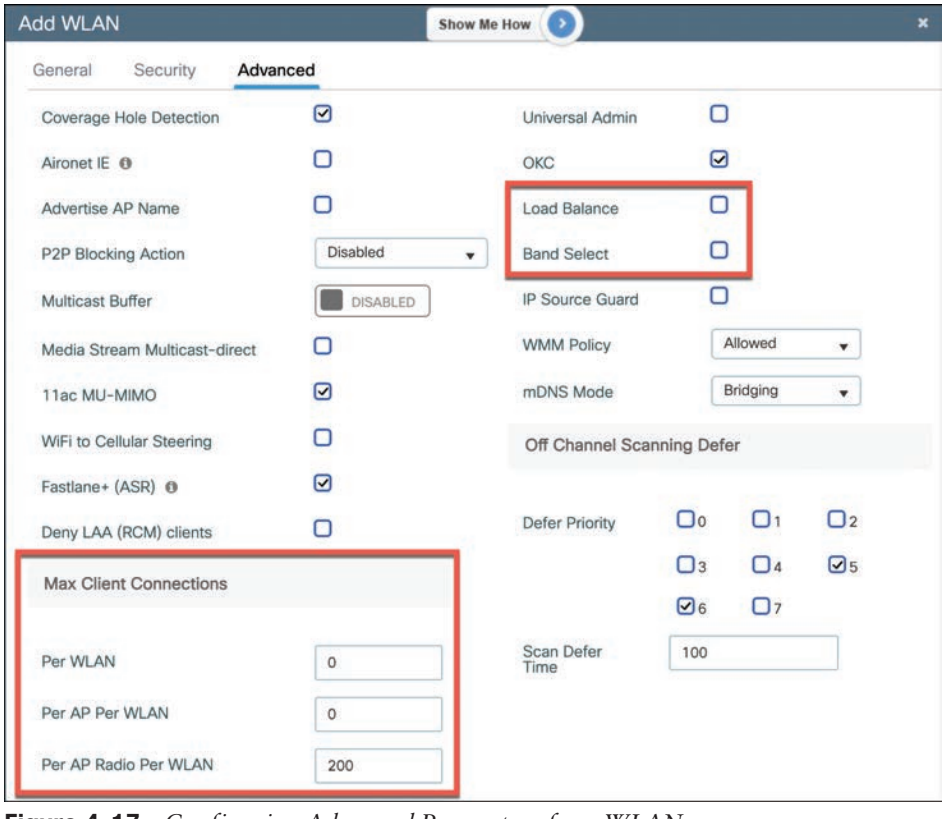



**Figure 4-17** *Configuring Advanced Parameters for a WLAN*

You can set limits on the client connection load in the Max Client Connections section. By default, the controller will allow an unlimited (designated by zero) number of connections per WLAN, an unlimited number per AP per WLAN, and 200 per AP radio per WLAN.

Notice the subtle difference between them: "per WLAN" means across all APs that carry the WLAN, "per AP" limits connections on any one AP and all of its radios, and "per AP radio" limits connections on each radio independently.

You might also want to let the controller decide how it accepts wireless clients onto an AP radio. For example, you can use the Load Balance option to let the controller distribute clients across neighboring APs as they probe and associate. The Band Select option lets the controller actively influence clients to join a more efficient frequency band if they try to associate on a lower, less efficient band. For instance, Band Select can attempt to prevent clients from joining a 2.4-GHz channel if a 5-GHz channel is also available nearby.

Figure 4-18 shows the lower half of the Advanced tab options. While most of them are more advanced than the CCNA exam covers, you should know that the Enable 11ax option (enabled by default) can be used to control 802.11ax use on the WLAN.



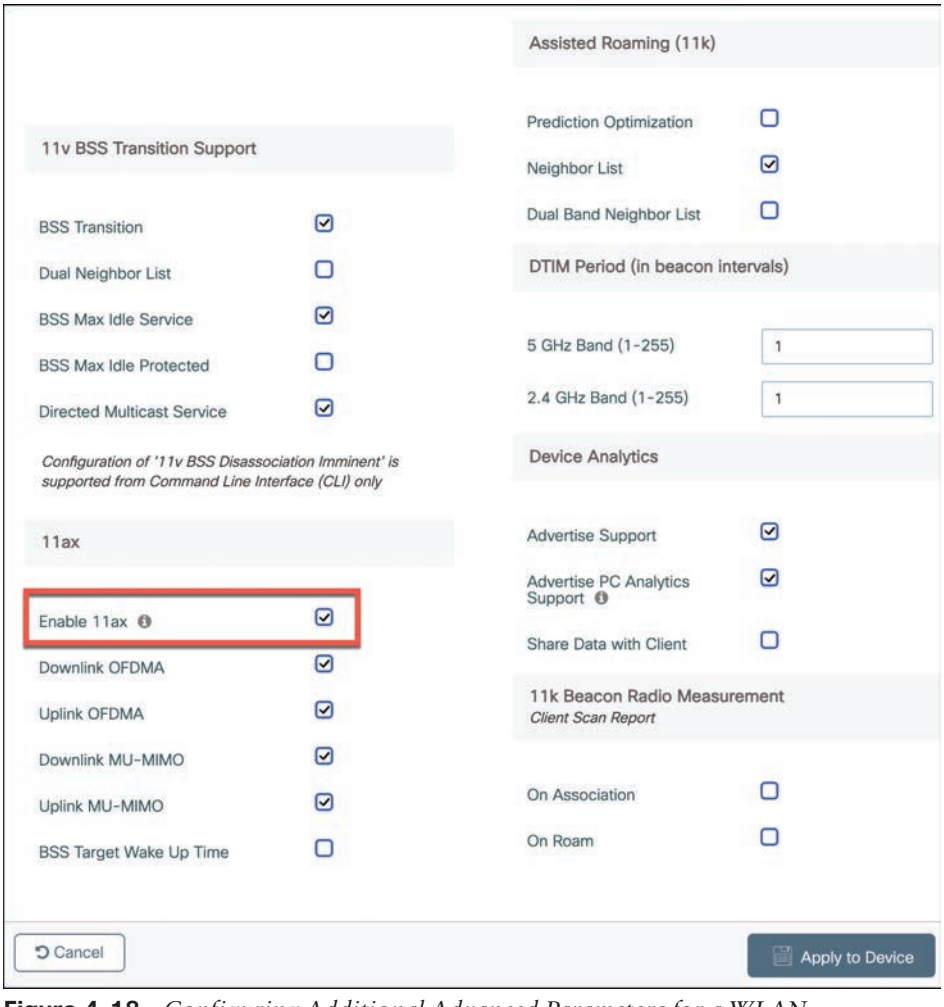



**Figure 4-18** *Configuring Additional Advanced Parameters for a WLAN*

After you have configured and verified all of the desired parameters, be sure to click the **Apply to Device** button to commit the changes to the controller's WLAN configuration. When the controller returns to display the list of WLANs again, as shown in Figure 4-19, you should verify that the new WLAN is enabled (shown by a green up arrow in the browser page), the SSID is correct, and the security settings are accurate. You can verify from the figure that the "engineering" SSID is up and is configured for WPA2-PSK with AES.




**Figure 4-19** *Verifying the WLAN Configuration*

### Step 2: Configure a Policy Profile

Next, you will need to configure a policy profile to define how the controller should handle the WLAN profile. From the Tags & Profiles task sequence (refer to Figure 4-12), select the **+** icon next to Policy Profile to create a new one. As shown in Figure 4-20, the General tab lets you name the profile and set its status as Enabled.

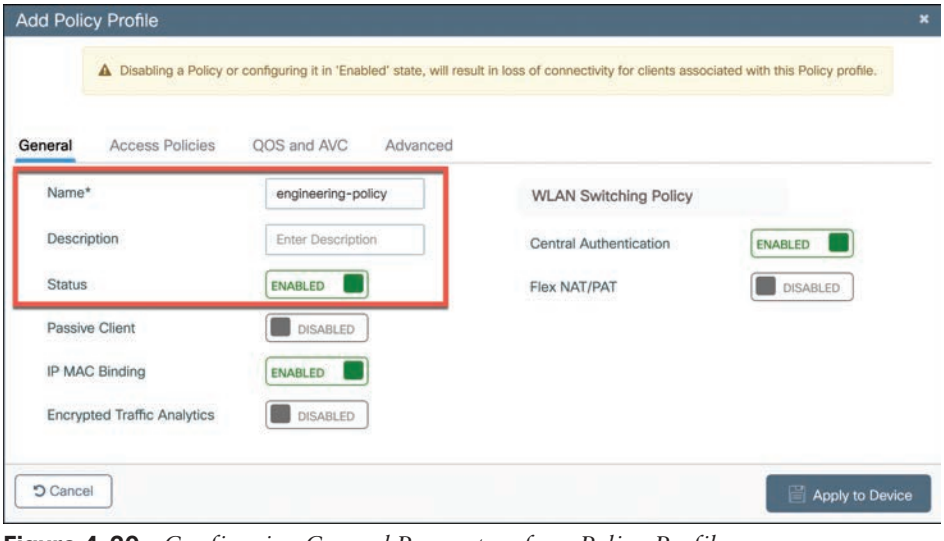



**Figure 4-20** *Configuring General Parameters for a Policy Profile*

Select the **Access Policies** tab to configure a VLAN that the controller will map to your new WLAN. In Figure 4-21, the WLAN will be mapped to VLAN 100.

You can select the **QOS and AVC** tab to configure ingress and egress quality of service (QoS) policies, as well as other voice call and traffic flow monitoring features. Figure 4-22 shows the default settings.

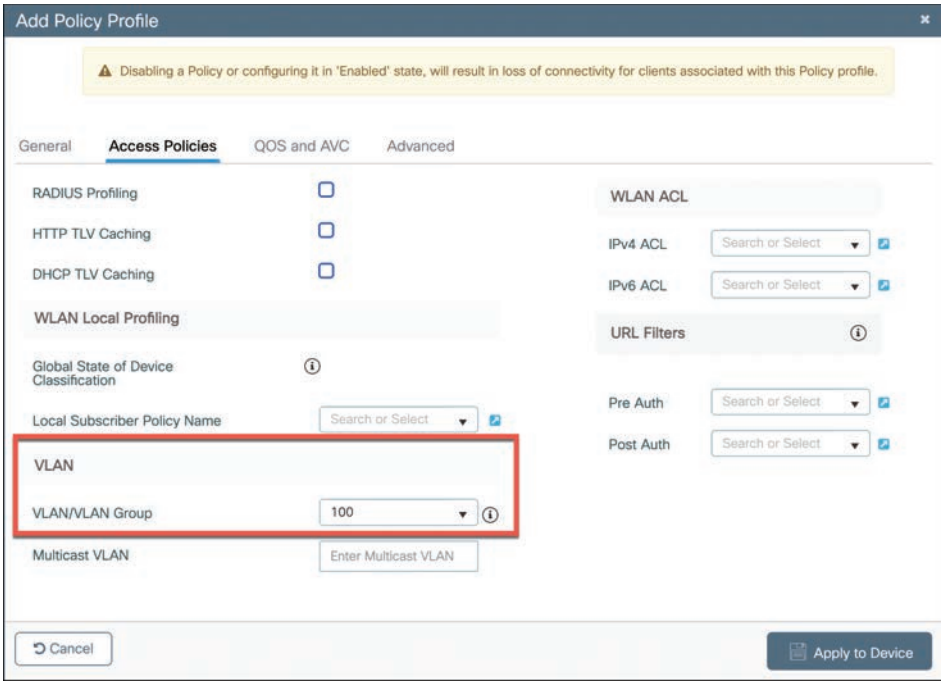



**Figure 4-21** *Configuring Access Policies Parameters for a Policy Profile*

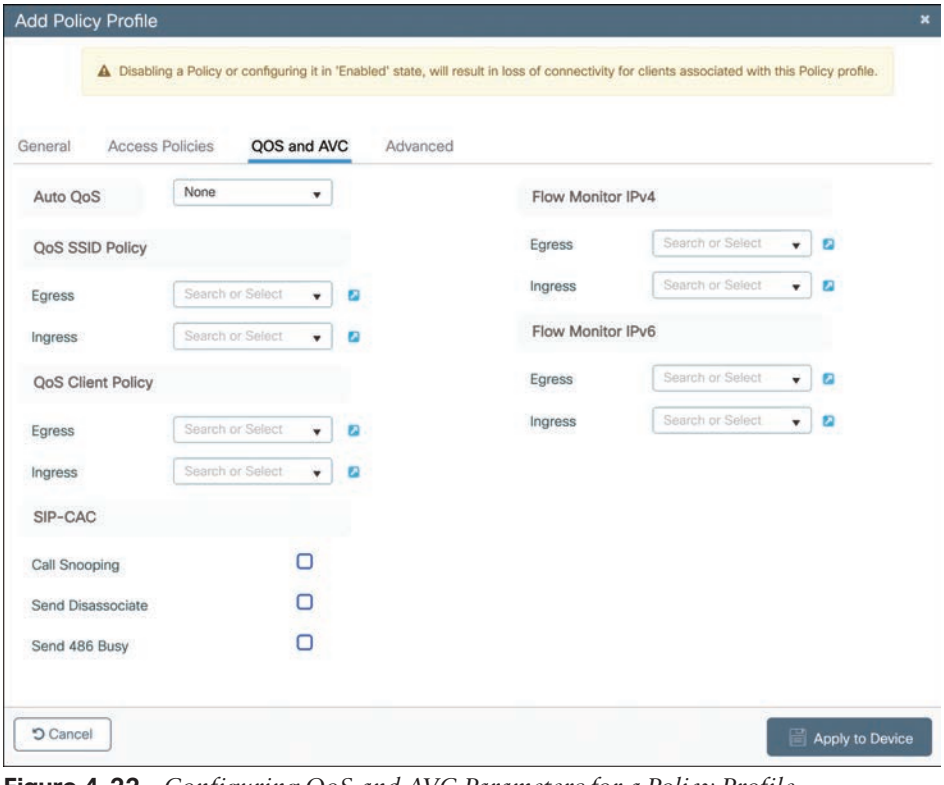



**Figure 4-22** *Configuring QoS and AVC Parameters for a Policy Profile*

Next, select the **Advanced** tab to display many more parameters related to the WLAN operation, as shown in Figure 4-23. The highlighted WLAN Timeout section contains several limits related to wireless client activity.

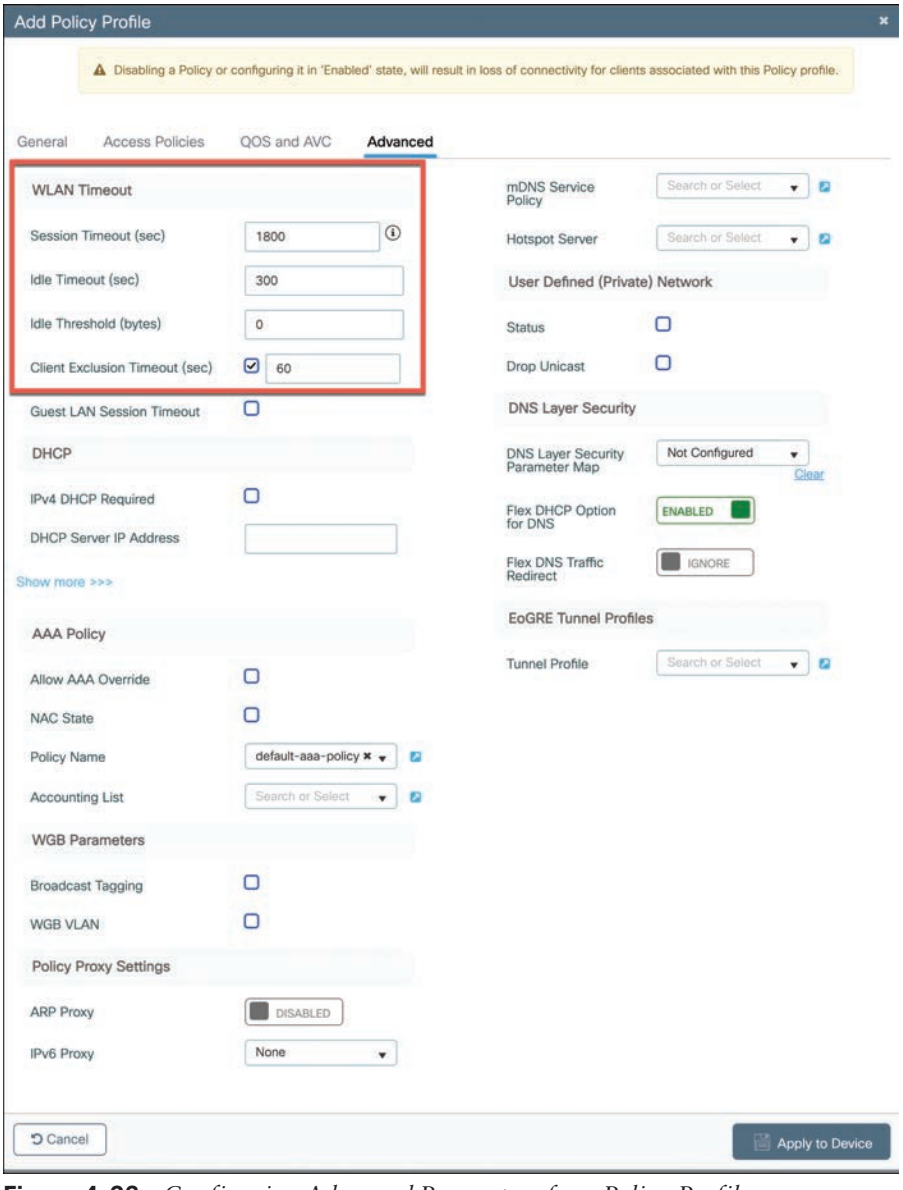



**Figure 4-23** *Configuring Advanced Parameters for a Policy Profile*

You can configure the Session Timeout to set the amount of time client sessions are allowed to continue before forcing them to reauthenticate. By default, sessions will be timed out after 1800 seconds (30 minutes). If 802.1x is used in the WLAN, you can set the session timeout value within the range 300 to 86,400 seconds; if not, the range is 0 to 86,400, where 0 means no timeout.

Use the Idle Timeout and Idle Threshold values to limit the amount of time (15 to 100,000 seconds, default 300) and number of traffic bytes (0 to 4,294,967,295 bytes) elapsed before a client is considered to be idle and dropped.

If the Client Exclusion box is checked, the controller will use its wireless intrusion prevention system (IPS) to evaluate client activity against a database of signatures. If it detects that some suspicious activity is occurring, the controller will put the client into an exclusion list and will isolate it from the wireless network for a default of 60 seconds.

After you have configured and verified all of the desired parameters, be sure to click the **Apply to Device** button to commit the changes to the policy profile configuration.

### Step 3: Map the WLAN and Policy Profiles to a Policy Tag

From the Tags & Profiles task sequence (refer to Figure 4-12), select the **+** icon next to Policy Tag. Enter a name for the policy tag and an optional description, as shown in Figure 4-24. Select the WLAN profile of the WLAN to be advertised, along with the policy profile that defines the VLAN to be used. Select the checkmark icon to add the profile combination to the policy tag.



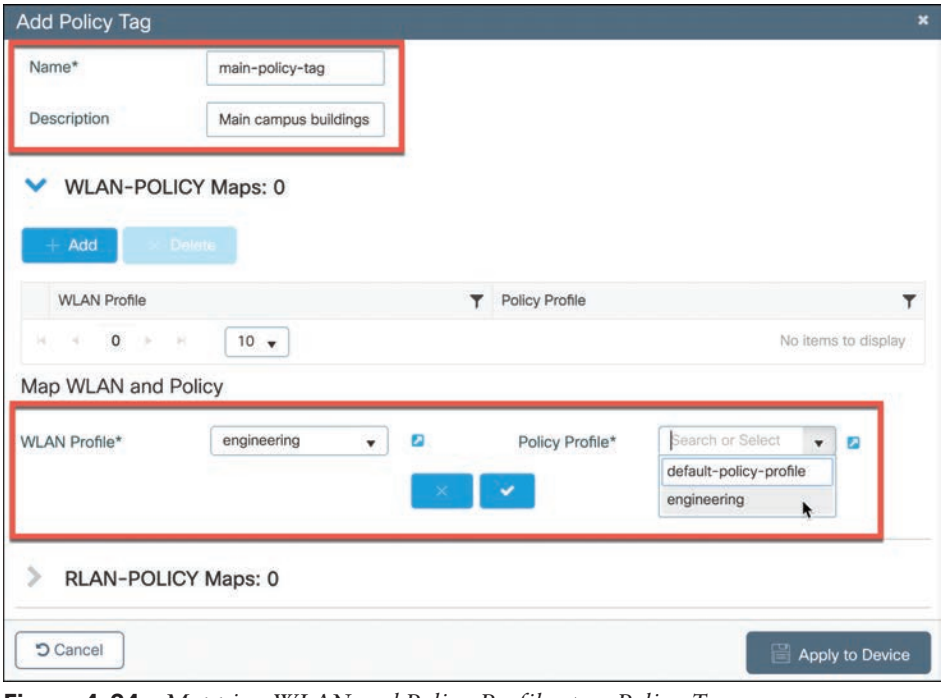



**Figure 4-24** *Mapping WLAN and Policy Profiles to a Policy Tag*

If you want APs to advertise more WLANs, you can click the **Add** button to add more WLAN and policy profile entries to the policy tag. Click the **Apply to Device** button to save your configuration changes to the controller.

### Step 4: Apply the Policy Tag to Some APs

Recall that each AP in the network must have three different tags mapped to it: Policy, Site, and RF. To do so, go to the bottom of the Tags & Profiles task sequence (refer to Figure 4-12); then select the list icon next to Tag APs.

The Tag APs window, as shown in Figure 4-25, consists of two parts: a list of available APs in the background and tags configuration in the foreground. You must first select the APs that will receive the tag mapping either by checking the boxes next to the desired AP entries or by filtering the APs according to the attribute columns. Next, use the **Policy Tag** drop-down menu to select the policy tag with the correct WLAN and Policy profile mappings.

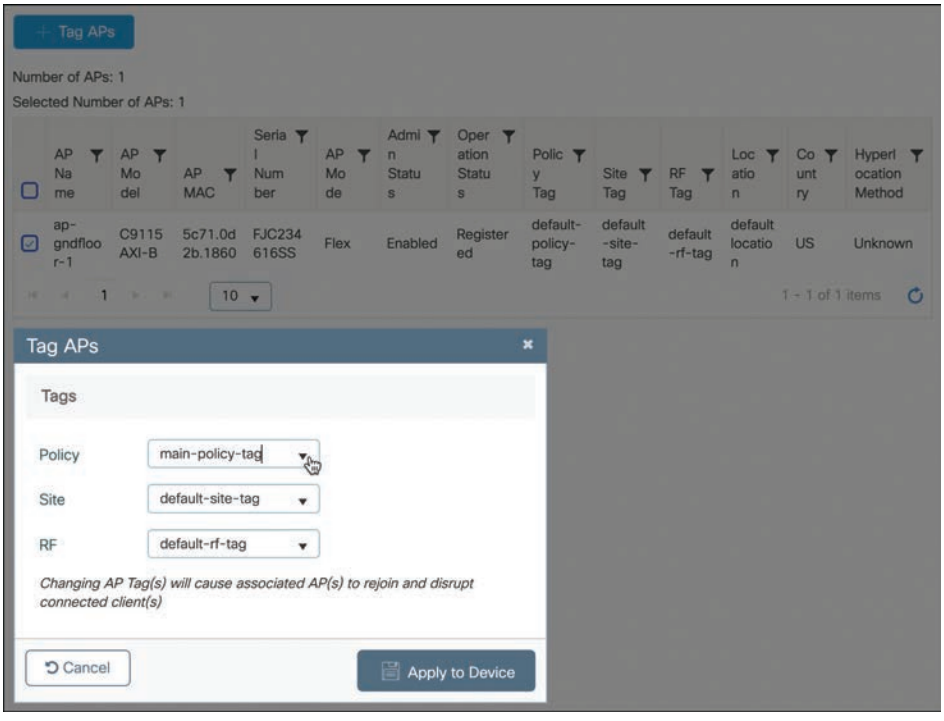



**Figure 4-25** *Applying Policy, Site, and RF Tags to APs*

You will also have to identify a site tag and an RF tag to be used, even though this chapter has not covered those because they are beyond the scope of the CCNA exam. Fortunately, the controller has a set of predefined default tags that contain mappings to corresponding default profiles:

- *default-site-tag*: Maps to default profiles named default-ap-profile and default-flex-profile
- *default-rf-tag*: Maps to the controller's global RF configuration
- *default-policy-tag*: Does not map to anything by default, because there is no default WLAN and SSID configuration for any network

The default profiles are preconfigured with commonly used parameters that can offer a fully functional wireless network. You can always use the default tags and profiles if you do not need to change anything in them.

**NOTE** You could avoid creating your own profiles and tags by making all of your custom changes to the controller's default profiles and tags; however, that would affect all APs globally unless they have been assigned other nondefault tags and profiles. Ideally, you should create your own set of custom profiles and tags to take full advantage of the granularity and to set the stage for future policy adjustments and custom tuning.

## Configuring a WLAN on an AireOS WLC



Legacy AireOS controllers do not use the same profile and tag concept as IOS-XE controllers. Instead, you can configure WLANs directly in the GUI, with much less granular control over AP configuration.

Creating a new WLAN involves the following three steps:

**Step 1.** Create a dynamic interface; then assign an interface name and a VLAN ID.

**Step 2.** Create a WLAN; then assign a WLAN profile name and SSID, along with a unique WLAN ID.

**Step 3.** Configure the WLAN parameters, enable it, and allow it to broadcast the SSID.

Each of these steps is discussed more fully in the sections that follow.

### Step 1: Create a Dynamic Interface

On an AireOS controller, a dynamic interface is used to connect the controller to a VLAN on the wired network. When you create a WLAN, you will bind the dynamic interface (and VLAN) to a wireless network.

To create a new dynamic interface, navigate to **Controller > Interfaces**. You should see a list of all the controller interfaces that are currently configured. In Figure 4-26, two interfaces named "management" and "virtual" already exist. Click the **New** button to define a new interface. Enter a name for the interface and the VLAN number it will be bound to. In Figure 4-27, the interface named Engineering is mapped to wired VLAN 100. Click the **Apply** button.

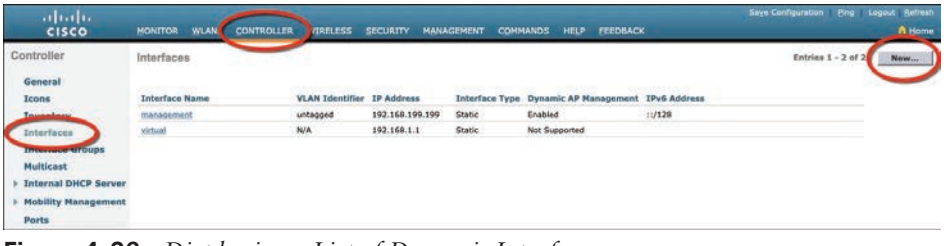



**Figure 4-26** *Displaying a List of Dynamic Interfaces*

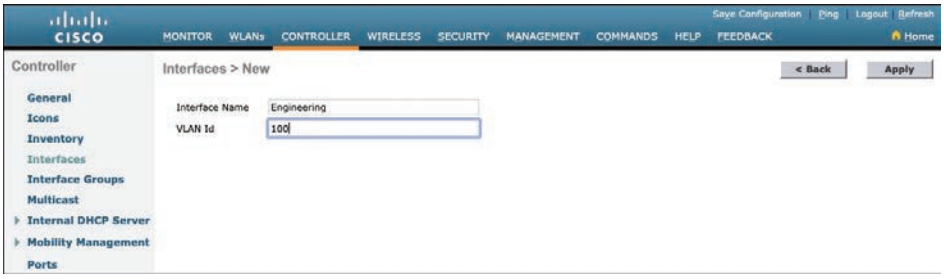



**Figure 4-27** *Defining a Dynamic Interface Name and VLAN ID*

Next, enter the IP address, subnet mask, and gateway address for the interface. You should also define primary and secondary DHCP server addresses that the controller will use when it relays DHCP requests from clients that are bound to the interface.

Figure 4-28 shows how the interface named Engineering has been configured with IP address 192.168.100.10, subnet mask 255.255.255.0, gateway 192.168.100.1, and DHCP servers 192.168.1.17 and 192.168.1.18. Click the **Apply** button to complete the interface configuration and return to the list of interfaces.

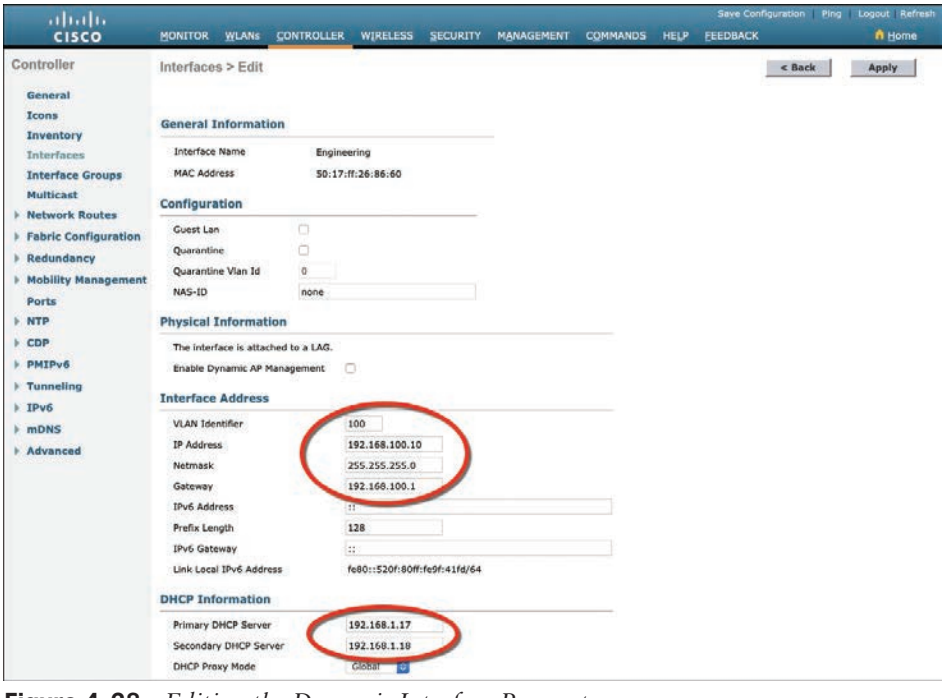



**Figure 4-28** *Editing the Dynamic Interface Parameters*

### Step 2: Create a New WLAN

You can display a list of the currently defined WLANs by selecting **WLANs** from the top menu bar. In Figure 4-29, the controller does not have any WLANs already defined. You can create a new WLAN by selecting **Create New** from the drop-down menu and then clicking the **Go** button.




**Figure 4-29** *Displaying a List of WLANs*

Next, enter a descriptive name as the profile name and the SSID text string. In Figure 4-30, the profile name and SSID are identical, just to keep things straightforward. The ID number is used as an index into the list of WLANs that are defined on the controller. The ID number becomes useful when you use templates in Prime Infrastructure (PI) to configure WLANs on multiple controllers at the same time.



**NOTE** WLAN templates are applied to specific WLAN ID numbers on controllers. The WLAN ID is only locally significant and is not passed between controllers. As a rule, you should keep the sequence of WLAN names and IDs consistent across multiple controllers so that any configuration templates you use in the future will be applied to the same WLANs on each controller.




**Figure 4-30** *Creating a New WLAN*

Click the **Apply** button to create the new WLAN.

### Step 3: Configure the WLAN

The next page will allow you to edit four categories of parameters, corresponding to the tabs across the top, as shown in Figure 4-31. By default, the General tab is selected.

You should enable the new WLAN by checking the **Status** check box. Even though the General page shows a specific security policy for the WLAN (the default WPA2 with 802.1x), you can make changes in a later step through the Security tab. Remember that 802.1x is used for "enterprise" authentication models that use RADIUS servers and digital certificates—not for pre-shared key authentication.

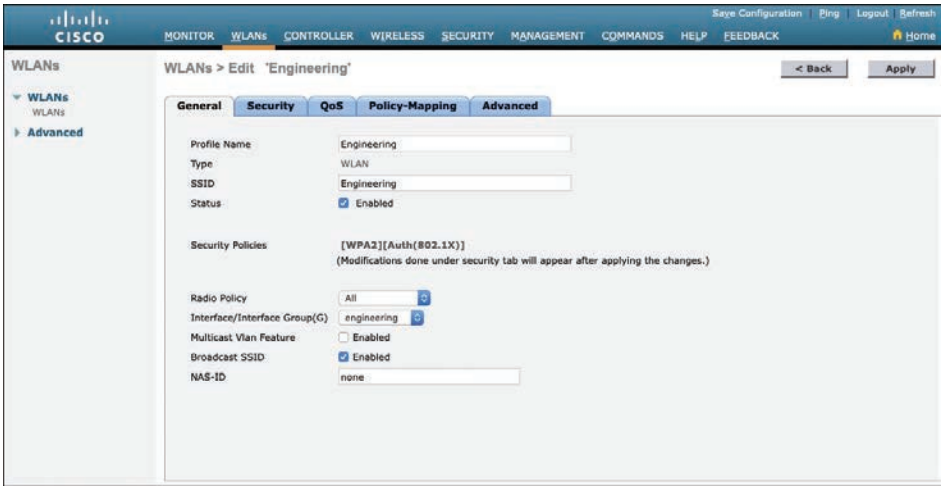



**Figure 4-31** *Configuring the General WLAN Parameters*

Under Radio Policy, select the type of radio that will offer the WLAN. By default, the WLAN will be offered on all radios that are joined with the controller. You can select a more specific policy with options like 802.11a only, 802.11a/g only, 802.11g only, or 802.11b/g only. For example, if you are creating a new WLAN for devices that have only a 2.4-GHz radio, it probably does not make sense to advertise the WLAN on both 2.4- and 5-GHz AP radios.

Next, select which of the controller's dynamic interfaces will be bound to the WLAN. By default, the management interface is selected. The drop-down list contains all the dynamic interface names that are available. In Figure 4-31, the new engineering WLAN will be bound to the Engineering interface.

Finally, use the Broadcast SSID check box to select whether the APs should broadcast the SSID name in the beacons they transmit. Broadcasting SSIDs is usually more convenient for users because their devices can learn and display the SSID names automatically. In fact, most devices actually need the SSID in the beacons to understand that the AP is still available for that SSID. Hiding the SSID name, by not broadcasting it, does not really provide any worthwhile security. Instead, it just prevents user devices from discovering an SSID and trying to use it as a default network.

For reference and study, Table 4-2 lists some of the values used in the past few configuration panels on the WLC, with data formats and lengths. The table also lists some values shown in the upcoming figures as well.

Key Topic

**Table 4-2** WLAN Configuration Fields and Formats

| Field | Length | Data Format | Other Rules |
|---|---|---|---|
| Profile name | 1–32 | ASCII | |
| SSID | 1–32 | ASCII | Alphanumeric, space, and printable special characters allowed; some special values reserved |
| VLAN ID | 2–4094 | Decimal | |

| Field | Length | Data Format | Other Rules |
|---|---|---|---|
| WLAN ID | 1–512 | Decimal | |
| Pre-shared key (PSK) | 8–63<br>Exactly 64 | ASCII or Hexadecimal | |

### Configuring WLAN Security

Select the **Security** tab to configure the security settings. By default, the Layer 2 Security tab is selected. From the Layer 2 Security drop-down menu, select the appropriate security scheme to use. Table 4-3 lists the types that are available.

Key Topic

**Table 4-3** Layer 2 WLAN Security Type

| Option | Description |
|---|---|
| None | Open authentication |
| WPA+WPA2 | Wi-Fi protected access WPA or WPA2 |
| 802.1x | EAP authentication with dynamic WEP |
| Static WEP | WEP key security |
| Static WEP + 802.1x | EAP authentication or static WEP |
| CKIP | Cisco Key Integrity Protocol |
| None + EAP Passthrough | Open authentication with remote EAP authentication |



As you select a security type, be sure to remember which choices are types that have been deprecated or proven to be weak, and avoid them if possible. Further down the screen, you can select which specific WPA, WPA2, and WPA3 methods to support on the WLAN. You can select more than one, if you need to support different types of wireless clients that require several security methods.

In Figure 4-32, WPA+WPA2 has been selected from the pull-down menu so that WPA2 will be a valid option. If you want to support efficient client roaming between APs, you can leverage the 802.11r amendment, also known as Fast Transition. In the Fast Transition section, notice that it is enabled by default with the Adaptive mode, which permits clients that do and do not support 802.11r.

In the WPA+WPA2 Parameters section, WPA2 Policy and AES encryption have been selected. The WPA and TKIP check boxes have been unchecked, so the methods are avoided because they are legacy and have been deprecated.

Under the Authentication Key Management section, you can select the authentication methods the WLAN will use. Only PSK has been selected in the figure, so the WLAN will allow only WPA2-Personal with pre-shared key authentication.

Suppose you need to use WPA2-Enterprise instead of WPA2-Personal or PSK. Client authentication could be performed by RADIUS servers, Cisco ISE, LDAP, and so on. You would select the 802.1X option rather than PSK. In that case, 802.1x and EAP would be used to authenticate wireless clients against one or more RADIUS servers. The controller would use servers from a global list that you define under **Security > AAA > RADIUS > Authentication**. To specify which servers the WLAN should use, you would select the **Security** tab

and then the **AAA Servers** tab in the WLAN edit screen. You can identify up to six specific RADIUS servers in the WLAN configuration.

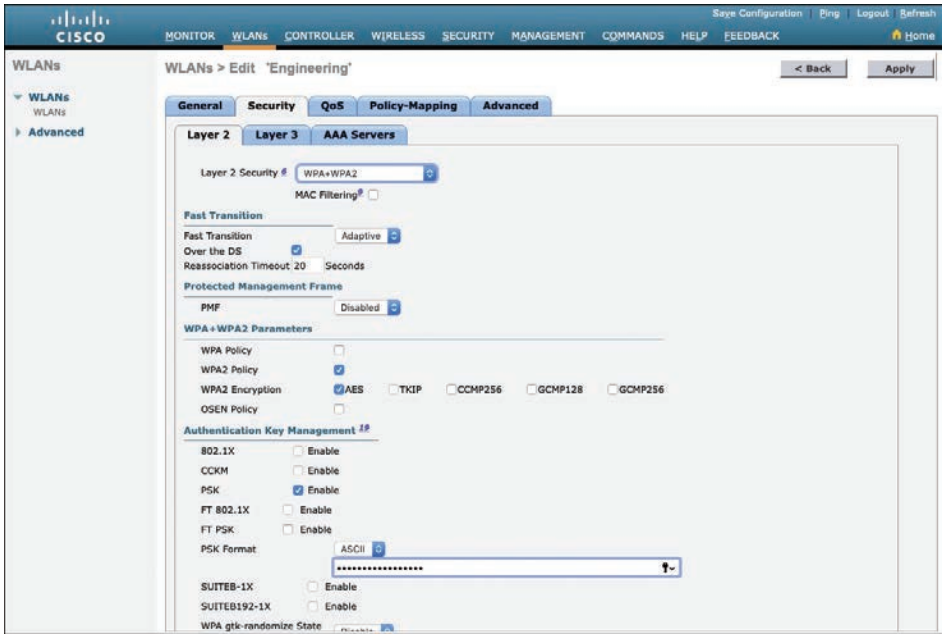



**Figure 4-32** *Configuring Layer 2 WLAN Security*

Beside each server, select a specific server IP address from the drop-down menu of globally defined servers. The servers are tried in sequential order until one of them responds. Although the CCNA exam objective specifies WPA2-Personal, Figure 4-33 shows what a WLAN configured for WPA2-Enterprise might look like, with servers 1 through 3 being set to 192.168.200.28, 192.168.200.29, and 192.168.200.30, respectively.

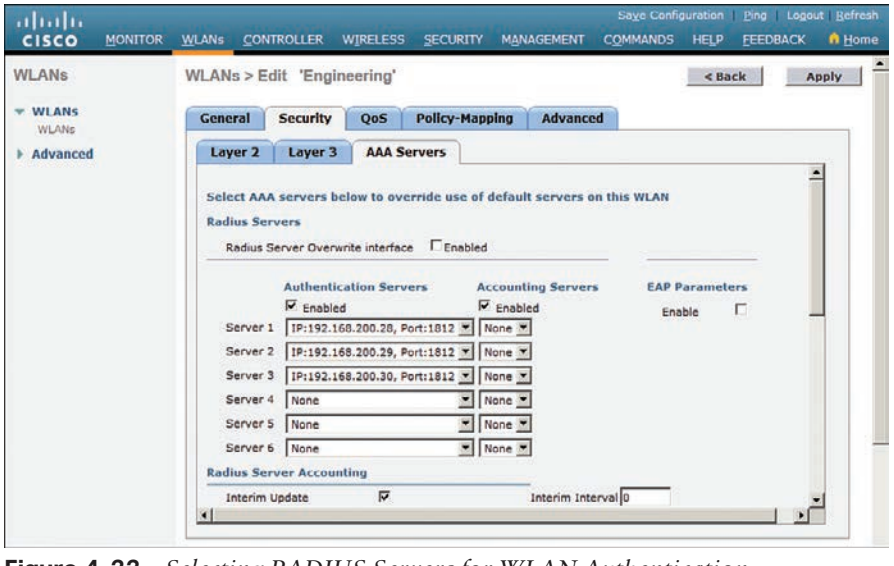



**Figure 4-33** *Selecting RADIUS Servers for WLAN Authentication*

## Configuring WLAN QoS

Select the **QoS** tab to configure quality of service settings for the WLAN, as shown in Figure 4-34. By default, the controller will consider all frames in the WLAN to be normal data, to be handled in a "best effort" manner. You can set the Quality of Service (QoS) drop-down menu to classify all frames in one of the following ways:

- Platinum (voice)
- Gold (video)
- Silver (best effort)
- Bronze (background)

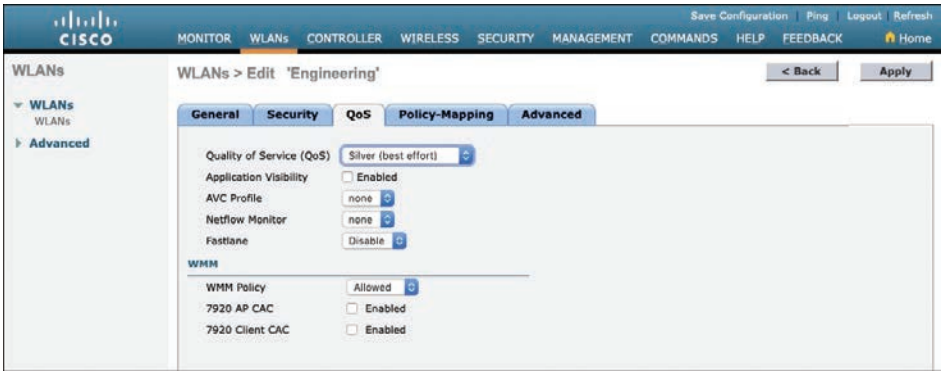



**Figure 4-34** *Configuring QoS Settings*

You can also set the Wi-Fi Multimedia (WMM) policy, call admission control (CAC) policies, and bandwidth parameters on the QoS page. You can learn more about QoS later in Chapter 15, "Quality of Service (QoS)."

## Configuring Advanced WLAN Settings

Finally, you can select the **Advanced** tab to configure a variety of advanced WLAN settings. From the page shown in Figure 4-35, you can configure many features—most of them are beyond the scope of the CCNA objectives and are not shown; however, you should be aware of a few parameters and defaults that might affect your wireless clients.

You can configure the Session Timeout to set the amount of time client sessions are allowed to continue before forcing them to reauthenticate. By default, sessions will be timed out after 1800 seconds (30 minutes). If 802.1x is used in the WLAN, you can set the session timeout value within the range 300 to 86,400 seconds; if not, the range is 0 to 86,400, where 0 means no timeout.

If the Client Exclusion box is enabled, the controller will use its wireless intrusion prevention system (IPS) to evaluate client activity against a database of signatures. If it detects that some suspicious activity is occurring, the controller will put the client into an exclusion list and will isolate it from the wireless network for a default of 180 seconds.

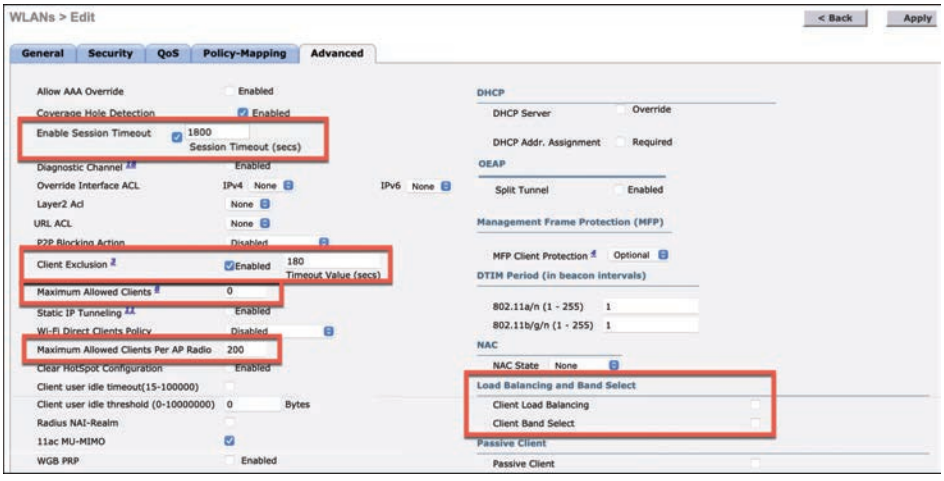



**Figure 4-35** *Configuring Advanced WLAN Settings*

You can set limits on the number of concurrent clients by setting the Maximum Allowed Clients value. By default, the controller will allow an unlimited (designated by zero) number of clients per WLAN. You can also limit the number of clients permitted to connect to each AP radio in the WLAN, which defaults to 200.

You might also want to let the controller decide how it accepts wireless clients onto an AP radio. For example, you can use the Client Load Balancing option to let the controller distribute clients across neighboring APs as they probe and associate. The Client Band Select option lets the controller actively influence clients to join a more efficient frequency band if they try to associate on a lower, less efficient band. For instance, Client Band Select can attempt to prevent clients from joining a 2.4-GHz channel if a 5-GHz channel is also available nearby.

**TIP** Is 180 seconds really enough time to deter an attack coming from a wireless client? In the case of a brute-force attack, where passwords are guessed from a dictionary of possibilities, 180 seconds is enough to disrupt and delay an attacker's progress. What might have taken 3 minutes to find a matching password without an exclusion policy would take 15 years with one.

## Finalizing WLAN Configuration

When you are satisfied with the settings in each of the WLAN configuration tabs, click the **Apply** button in the upper-right corner of the WLAN Edit screen. The WLAN will be created and added to the controller configuration. In Figure 4-36, the Engineering WLAN has been added as WLAN ID 1 and is enabled for use.




**Figure 4-36** *Displaying WLANs Configured on a Controller*

Don't forget to verify the new WLAN's configuration. From the information shown in Figure 4-36, you can confirm that the SSID is correct, the Admin Status is enabled, and the security settings are accurate for WPA2-PSK.

## Chapter Review



Review this chapter's material using either the tools in the book or the interactive tools for the same material found on the book's companion website. Table 4-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 4-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |

## Review All the Key Topics

Key Topic

**Table 4-5** Key Topics for Chapter 4

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 4-1 | Physical connections to an AP | 59 |
| Figure 4-8 | Wireless LAN controller physical ports | 63 |
| Figure 4-9 | Wireless LAN controller logical interfaces | 64 |
| Figure 4-10 | Planning a WLAN | 66 |
| Figure 4-11 | AP configuration with an IOS-XE controller | 67 |
| Table 4-2 | WLAN Configuration Fields and Formats | 82 |
| Table 4-3 | Configuring WLAN security | 83 |



# Part I Review

Keep track of your part review progress with the checklist in Table P1-1. Details on each task follow the table.

**Table P1-1** Part I Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Watch Video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book, using the PTP software.

## Answer Part Review Questions

For this task, answer the Part Review questions for this part of the book, using the PTP software.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Watch Video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.

*This page intentionally left blank*



The *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, includes the topics that help you build an enterprise network so all devices can communicate with all other devices. Parts II and III of this book focus on how to secure that enterprise network so that only the appropriate devices and users can communicate.

Part II focuses on IP Version 4 (IPv4) access control lists (ACLs). ACLs are IPv4 packet filters that can be programmed to look at IPv4 packet headers, make choices, and either allow a packet through or discard the packet. Because you can implement IPv4 ACLs on any router, a network engineer has many options of where to use ACLs, without adding additional hardware or software, making ACLs a very flexible and useful tool.

Chapter 5 begins this part with an introduction to the TCP/IP Transport layer protocols TCP and UDP, along with an introduction to several TCP/IP applications. This chapter provides the necessary background to understand the ACL chapters.

Chapters 6, 7, and 8 get into details about ACLs. Chapter 6 discusses ACL basics, avoiding advanced topics to ensure that you master the basics. Chapter 7 explores named ACLs, which allow easier configuration and editing, and extended ACLs, which provide more options to match packets. Chapter 8 completes the ACL discussion by examining specific implementation issues, first regarding several overhead protocols, and then discussing some improved ACL features introduced by IOS XE.

# Part II

## IP Access Control Lists

**Chapter 5:** Introduction to TCP/IP Transport and Applications

**Chapter 6:** Basic IPv4 Access Control Lists

**Chapter 7:** Named and Extended IP ACLs

**Chapter 8:** Applied IP ACLs

**Part II Review**



CHAPTER 5

# Introduction to TCP/IP Transport and Applications

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.5 Compare TCP to UDP**

**4.0 IP Services**

**4.3 Explain the role of DHCP and DNS in the network**

The CCNA exam focuses mostly on functions at the lower layers of TCP/IP, which define how IP networks can send IP packets from host to host using LANs and WANs. This chapter explains the basics of a few topics that receive less attention on the exams: the TCP/IP transport layer and the TCP/IP application layer. The functions of these higher layers play a big role in real TCP/IP networks. Additionally, many of the security topics in Parts I and II of this book and some of the IP services topics in Part III require you to know the basics of how the transport and application layers of TCP/IP work. This chapter serves as that introduction.

This chapter begins by examining the functions of two transport layer protocols: Transmission Control Protocol (TCP) and User Datagram Protocol (UDP). The second major section of the chapter examines the TCP/IP application layer, including some discussion of how Domain Name System (DNS) name resolution works.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 5-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| TCP/IP Layer 4 Protocols: TCP and UDP | 1–4 |
| TCP/IP Applications | 5, 6 |

1. Which of the following header fields identify which TCP/IP application gets data received by the computer? (Choose two answers.)
   a. Ethernet Type
   b. SNAP Protocol Type
   c. IP Protocol

**d.** TCP Port Number

**e.** UDP Port Number

2. Which of the following are typical functions of TCP? (Choose four answers.)
   **a.** Flow control (windowing)
   **b.** Error recovery
   **c.** Multiplexing using port numbers
   **d.** Routing
   **e.** Encryption
   **f.** Ordered data transfer
3. Which of the following functions is performed by both TCP and UDP?
   **a.** Windowing
   **b.** Error recovery
   **c.** Multiplexing using port numbers
   **d.** Routing
   **e.** Encryption
   **f.** Ordered data transfer
4. What do you call data that includes the Layer 4 protocol header, and data given to Layer 4 by the upper layers, not including any headers and trailers from Layers 1 to 3? (Choose two answers.)
   **a.** L3PDU
   **b.** Chunk
   **c.** Segment
   **d.** Packet
   **e.** Frame
   **f.** L4PDU
5. In the URI http://www.certskills.com/config-labs, which part identifies the web server?
   **a.** http
   **b.** www.certskills.com
   **c.** certskills.com
   **d.** http://www.certskills.com
   **e.** The file name.html includes the hostname.
6. Fred opens a web browser and connects to the www.certskills.com website. Which of the following are typically true about what happens between Fred's web browser and the web server? (Choose two answers.)
   **a.** Messages flowing toward the server use UDP destination port 80.
   **b.** Messages flowing from the server typically use RTP.
   **c.** Messages flowing to the client typically use a source TCP port number of 80.
   **d.** Messages flowing to the server typically use TCP.

# Foundation Topics

## TCP/IP Layer 4 Protocols: TCP and UDP

The OSI transport layer (Layer 4) defines several functions, the most important of which are error recovery and flow control. Likewise, the TCP/IP transport layer protocols also implement these same types of features. Note that both the OSI model and the TCP/IP model call this layer the transport layer. But as usual, when referring to the TCP/IP model, the layer name and number are based on OSI, so any TCP/IP transport layer protocols are considered Layer 4 protocols.

The key difference between TCP and UDP is that TCP provides a wide variety of services to applications, whereas UDP does not. For example, routers discard packets for many reasons, including bit errors, congestion, and instances in which no correct routes are known. As you have read already, most data-link protocols notice errors (a process called **error detection**) but then discard frames that have errors. TCP provides retransmission (**error recovery**) and helps to avoid congestion (**flow control**), whereas UDP does not. As a result, many application protocols choose to use TCP.

However, do not let UDP's lack of services make you think that UDP is worse than TCP. By providing fewer services, UDP needs fewer bytes in its header compared to TCP, resulting in fewer bytes of overhead in the network. UDP software does not slow down data transfer in cases where TCP can purposefully slow down. Also, some applications, notably today voice over IP (VoIP) and video over IP, do not need error recovery, so they use UDP. So, UDP also has an important place in TCP/IP networks today.

Table 5-2 lists the main features supported by TCP/UDP. Note that only the first item listed in the table is supported by UDP, whereas all items in the table are supported by TCP.

Key Topic

**Table 5-2** TCP/IP Transport Layer Features

| Function | Description |
|---|---|
| Multiplexing using **ports** | Function that allows receiving hosts to choose the correct application for which the data is destined, based on the port number |
| Error recovery (reliability) | Process of numbering and acknowledging data with Sequence and Acknowledgment header fields |
| Flow control using windowing | Process that uses window sizes to protect buffer space and routing devices from being overloaded with traffic |
| Connection establishment and termination | Process used to initialize port numbers and Sequence and Acknowledgment fields |
| **Ordered data transfer** and data segmentation | Continuous stream of bytes from an upper-layer process that is "segmented" for transmission and delivered to upper-layer processes at the receiving device, with the bytes in the same order |

Next, this section describes the features of TCP, followed by a brief comparison to UDP.

---

Answers to the "Do I Know This Already?" quiz:

**1** D, E **2** A, B, C, F **3** C **4** C, F **5** B **6** C, D

## Transmission Control Protocol

Each TCP/IP application typically chooses to use either TCP or UDP based on the application's requirements. For example, TCP provides error recovery, but to do so, it consumes more bandwidth and uses more processing cycles. UDP does not perform error recovery, but it takes less bandwidth and uses fewer processing cycles. Regardless of which of these two TCP/IP transport layer protocols the application chooses to use, you should understand the basics of how each of these transport layer protocols works.

TCP, as defined in Request For Comments (RFC) 9293, accomplishes the functions listed in Table 5-2 through mechanisms at the endpoint computers. TCP relies on IP for end-to-end delivery of the data, including routing issues. In other words, TCP performs only part of the functions necessary to deliver the data between applications. Also, the role that it plays is directed toward providing services for the applications that sit at the endpoint computers. Regardless of whether two computers are on the same Ethernet, or are separated by the entire Internet, TCP performs its functions the same way.

Figure 5-1 shows the fields in the TCP header. Although you don't need to memorize the names of the fields or their locations, the rest of this section refers to several of the fields, so the entire header is included here for reference.



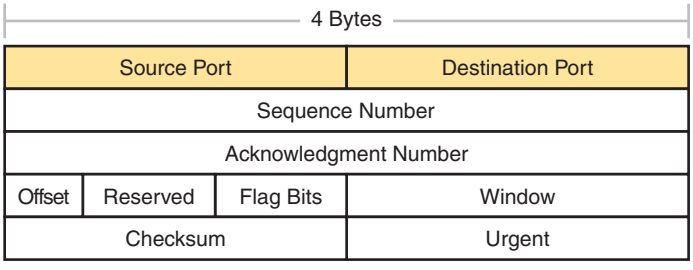



**Figure 5-1** *TCP Header Fields*

The message created by TCP that begins with the TCP header, followed by any application data, is called a TCP **segment**. Alternatively, the more generic term *Layer 4 PDU*, or *L4PDU*, can also be used.

### Multiplexing Using TCP Port Numbers

TCP and UDP both use a concept called *multiplexing*. Therefore, this section begins with an explanation of multiplexing with TCP and UDP. Afterward, the unique features of TCP are explored.

Multiplexing by TCP and UDP involves the process of how a computer thinks when receiving data. The computer might be running many applications, such as a web browser, an email package, or an Internet VoIP application (for example, Skype). TCP and UDP multiplexing tells the receiving computer to which application to give the received data.

Some examples will help make the need for multiplexing obvious. The sample network consists of two PCs, labeled Hannah and George. Hannah uses an application that she wrote to send advertisements that appear on George's screen. The application sends a new ad to George every 10 seconds. Hannah uses a second application, a wire-transfer application, to send George some money. Finally, Hannah uses a web browser to access the web server that

runs on George's PC. The ad application and wire-transfer application are imaginary, just for this example. The web application works just like it would in real life.

Figure 5-2 shows the sample network, with George running three applications:

- A UDP-based advertisement application
- A TCP-based wire-transfer application
- A TCP web server application

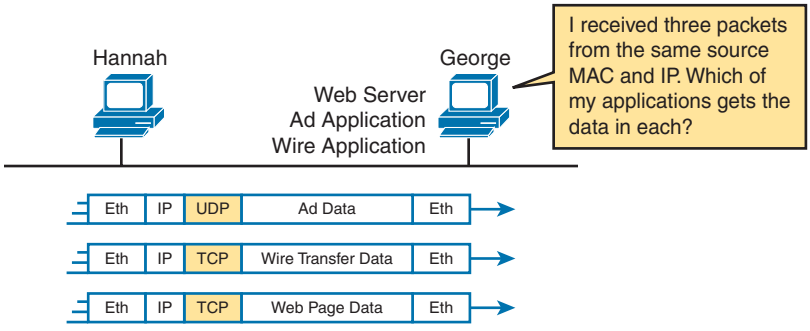



**Figure 5-2** *Hannah Sending Packets to George, with Three Applications*

George needs to know which application to give the data to, but *all three packets are from the same Ethernet and IP address*. You might think that George could look at whether the packet contains a UDP or TCP header, but as you see in the figure, two applications (wire transfer and web) are using TCP.

TCP and UDP solve this problem by using a port number field in the TCP or UDP header, respectively. Each of Hannah's TCP and UDP segments uses a different *destination port number* so that George knows which application to give the data to. Figure 5-3 shows an example.

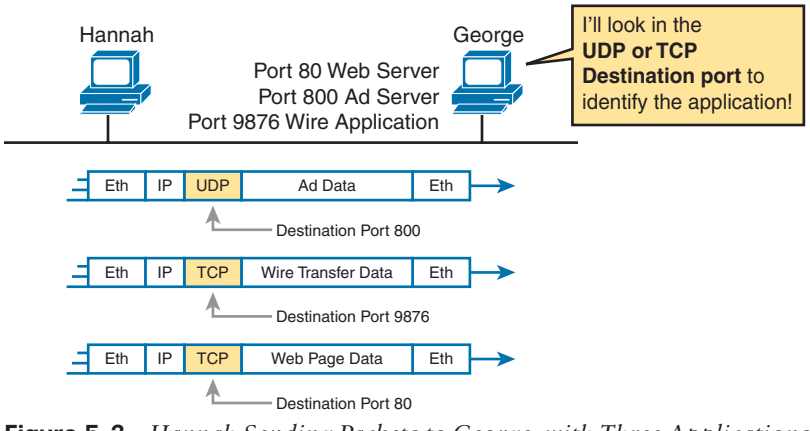



**Figure 5-3** *Hannah Sending Packets to George, with Three Applications Using Port Numbers to Multiplex*

Multiplexing relies on a concept called a *socket*. A socket consists of three things:

- An IP address
- A transport protocol
- A port number

So, for a web server application on George, the socket would be (10.1.1.2, TCP, port 80) because, by default, web servers use the well-known port 80. When Hannah's web browser connects to the web server, Hannah uses a socket as well—possibly one like this: (10.1.1.1, TCP, 49160). Why 49160? Well, Hannah just needs a port number that is unique on Hannah, so Hannah sees that port 49160.

The Internet Assigned Numbers Authority (IANA), the same organization that manages IP address allocation worldwide, subdivides the port number ranges into three main ranges. The first two ranges reserve numbers that IANA can then allocate to specific application protocols through an application and review process, with the third category reserving ports to be dynamically allocated as used for clients, as with the port 49160 example in the previous paragraph. The names and ranges of port numbers (as detailed in RFC 6335) are



- **Well Known (System) Ports:** Numbers from 0 to 1023, assigned by IANA, with a stricter review process to assign new ports than user ports.
- **User (Registered) Ports:** Numbers from 1024 to 49151, assigned by IANA with a less strict process to assign new ports compared to well-known ports.
- **Ephemeral (Dynamic, Private) Ports:** Numbers from 49152 to 65535, not assigned and intended to be dynamically allocated and used temporarily for a client application while the app is running.

Figure 5-4 shows an example that uses three ephemeral ports on the user device on the left, with the server on the right using two well-known ports and one user port. The computers use three applications at the same time; hence, three socket connections are open. Because a socket on a single computer should be unique, a connection between two sockets should identify a unique connection between two computers. This uniqueness means that you can use multiple applications at the same time, talking to applications running on the same or different computers. Multiplexing, based on sockets, ensures that the data is delivered to the correct applications.

Port numbers are a vital part of the socket concept. Servers use well-known ports (or user ports), whereas clients use dynamic ports. Applications that provide a service, such as FTP, Telnet, and web servers, open a socket using a well-known port and listen for connection requests. Because these connection requests from clients are required to include both the source and destination port numbers, the port numbers used by the servers must be known beforehand. Therefore, each service uses a specific well-known port number or user port number. Both well-known and user ports are listed at www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.txt.

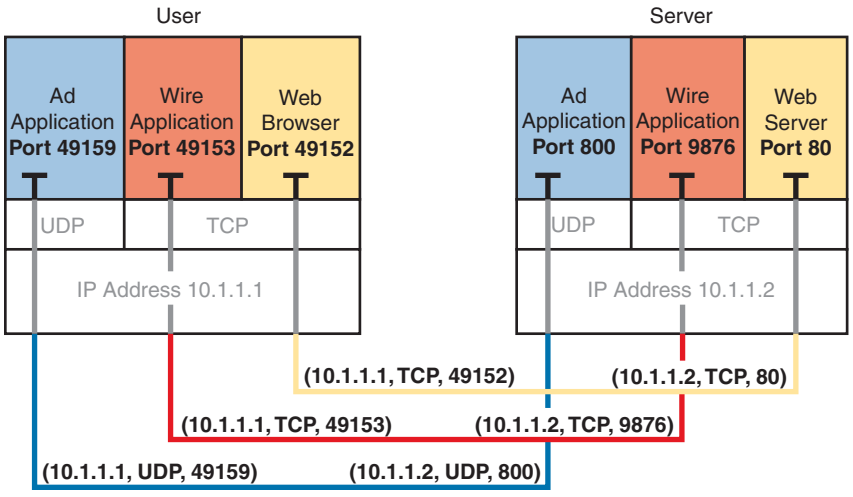



**Figure 5-4** *Connections Between Sockets*

On client machines, where the requests originate, any locally unused port number can be allocated. The result is that each client on the same host uses a different port number, but a server uses the same port number for all connections. For example, 100 web browsers on the same host computer could each connect to a web server, but the web server with 100 clients connected to it would have only one socket and, therefore, only one port number (port 80, in this case). The server can tell which packets are sent from which of the 100 clients by looking at the source port of received TCP segments. The server can send data to the correct web client (browser) by sending data to that same port number listed as a destination port. The combination of source and destination sockets allows all participating hosts to distinguish between the data's source and destination. Although the example explains the concept using 100 TCP connections, the same port-numbering concept applies to UDP sessions in the same way.

**NOTE** You can find all RFCs online at www.rfc-editor.org/rfc/rfc*xxxx*.txt, where *xxxx* is the number of the RFC. If you do not know the number of the RFC, you can try searching by topic at www.rfc-editor.org.

## Popular TCP/IP Applications

Throughout your preparation for the CCNA exam, you will come across a variety of TCP/IP applications. You should at least be aware of some of the applications that can be used to help manage and control a network.

The World Wide Web (WWW) application exists through web browsers accessing the content available on web servers. Although it is often thought of as an end-user application, you can actually use WWW to manage a router or switch. You enable a web server function in the router or switch and use a browser to access the router or switch.

The Domain Name System (DNS) allows users to use names to refer to computers, with DNS being used to find the corresponding IP addresses. DNS also uses a client/server model, with **DNS servers** being controlled by networking personnel and DNS client functions being

part of most any device that uses TCP/IP today. The client simply asks the DNS server to supply the IP address that corresponds to a given name.

Simple Network Management Protocol (SNMP) is an application layer protocol used specifically for network device management. For example, Cisco supplies a large variety of network management products, many of them in the Cisco Prime network management software product family. They can be used to query, compile, store, and display information about a network's operation. To query the network devices, Cisco Prime software mainly uses SNMP protocols.

Traditionally, to move files to and from a router or switch, Cisco used Trivial File Transfer Protocol (TFTP). TFTP defines a protocol for basic file transfer—hence the word *trivial*. Alternatively, routers and switches can use File Transfer Protocol (FTP), which is a much more functional protocol, to transfer files. Both work well for moving files into and out of Cisco devices. FTP allows many more features, making it a good choice for the general end-user population. TFTP client and server applications are very simple, making them good tools as embedded parts of networking devices.

Some of these applications use TCP, and some use UDP. For example, Simple Mail Transfer Protocol (SMTP) and Post Office Protocol version 3 (POP3), both used for transferring mail, require guaranteed delivery, so they use TCP.

Regardless of which transport layer protocol is used, applications use a well-known port number so that clients know which port to attempt to connect to. Table 5-3 lists several popular applications and their well-known port numbers.



Key Topic

**Table 5-3** Popular Applications and Their Well-Known Port Numbers

| Port Number | Protocol | Application |
|---|---|---|
| 20 | TCP | FTP data |
| 21 | TCP | FTP control |
| 22 | TCP | SSH |
| 23 | TCP | Telnet |
| 25 | TCP | SMTP |
| 53 | UDP, TCP* | DNS |
| 67 | UDP | DHCP Server |
| 68 | UDP | DHCP Client |
| 69 | UDP | TFTP |
| 80 | TCP | HTTP (WWW) |
| 110 | TCP | POP3 |
| 161 | UDP | SNMP |
| 443 | TCP, UDP | HTTPS |
| 514 | UDP | Syslog |

* DNS uses both UDP and TCP in different instances. It uses port 53 for both TCP and UDP.

### Connection Establishment and Termination

TCP **connection establishment** occurs before any of the other TCP features can begin their work. Connection establishment refers to the process of initializing Sequence and Acknowledgment fields and agreeing on the port numbers used. Figure 5-5 shows an example of connection establishment flow.

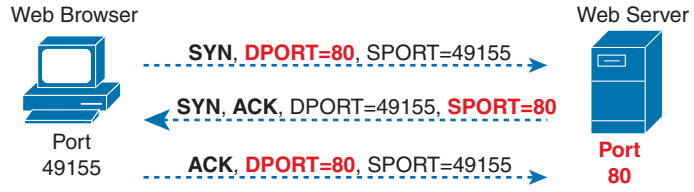



**Figure 5-5** *TCP Connection Establishment*

This three-way connection establishment flow (also called a three-way handshake) must complete before data transfer can begin. The connection exists between the two sockets, although the TCP header has no single socket field. Of the three parts of a socket, the IP addresses are implied based on the source and destination IP addresses in the IP header. TCP is implied because a TCP header is in use, as specified by the protocol field value in the IP header. Therefore, the only parts of the socket that need to be encoded in the TCP header are the port numbers.

TCP signals connection establishment using 2 bits inside the flag fields of the TCP header. Called the SYN and ACK flags, these bits have a particularly interesting meaning. SYN means "synchronize the sequence numbers," which is one necessary component in initialization for TCP.

Figure 5-6 shows TCP connection termination. This four-way termination sequence is straightforward and uses an additional flag, called the *FIN bit*. (FIN is short for "finished," as you might guess.) One interesting note: Before the device on the right sends the third TCP segment in the sequence, it notifies the application that the connection is coming down. It then waits on an acknowledgment from the application before sending the third segment in the figure. Just in case the application takes some time to reply, the PC on the right sends the second flow in the figure, acknowledging that the other PC wants to take down the connection. Otherwise, the PC on the left might resend the first segment repeatedly.

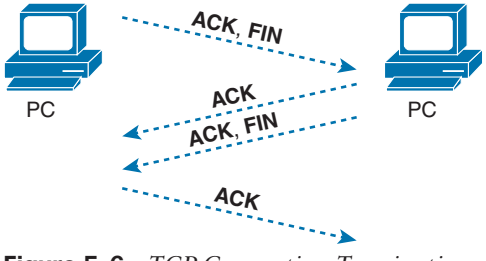



**Figure 5-6** *TCP Connection Termination*

TCP establishes and terminates connections between the endpoints, whereas UDP does not. Many protocols operate under these same concepts, so the terms *connection-oriented* and

*connectionless* are used to refer to the general idea of each. More formally, these terms can be defined as follows:

Key Topic

- **Connection-oriented protocol:** A protocol that requires an exchange of messages before data transfer begins, or that has a required pre-established correlation between two endpoints.
- **Connectionless protocol:** A protocol that does not require an exchange of messages and that does not require a pre-established correlation between two endpoints.

### Error Recovery and Reliability

TCP provides for reliable data transfer, which is also called *reliability* or *error recovery*, depending on what document you read. To accomplish reliability, TCP numbers data bytes using the Sequence and Acknowledgment fields in the TCP header. TCP achieves reliability in both directions, using the Sequence Number field of one direction combined with the Acknowledgment field in the opposite direction.

Figure 5-7 shows an example of how the TCP Sequence and Acknowledgment fields allow the PC to send 3000 bytes of data to the server, with the server acknowledging receipt of the data. The TCP segments in the figure occur in order, from top to bottom. For simplicity's sake, all messages happen to have 1000 bytes of data in the data portion of the TCP segment. The first Sequence number is a nice round number (1000), again for simplicity's sake. The top of the figure shows three segments, with each sequence number being 1000 more than the previous, identifying the first of the 1000 bytes in the message. (That is, in this example, the first segment holds bytes 1000–1999; the second holds bytes 2000–2999; and the third holds bytes 3000–3999.)

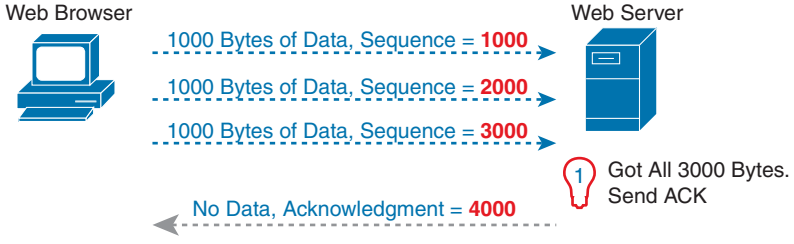



**Figure 5-7** *TCP Acknowledgment Without Errors*

The fourth TCP segment in the figure—the only one flowing back from the server to the web browser—acknowledges the receipt of all three segments. How? The acknowledgment value of 4000 means "I received all data with sequence numbers up through one less than 4000, so I am ready to receive your byte 4000 next." (Note that this convention of acknowledging by listing the next expected byte, rather than the number of the last byte received, is called **forward acknowledgment**.)

This first example does not recover from any errors, however; it simply shows the basics of how the sending host uses the sequence number field to identify the data, with the receiving host using forward acknowledgments to acknowledge the data. The more interesting discussion revolves around how to use these same tools to do error recovery. TCP uses the Sequence and Acknowledgment fields so that the receiving host can notice lost data, ask the sending host to resend, and then acknowledge that the re-sent data arrived.

Many variations exist for how TCP does error recovery. Figure 5-8 shows just one such example, with similar details compared to the previous figure. The web browser again sends three TCP segments, again 1000 bytes each, again with easy-to-remember sequence numbers. However, in this example, the second TCP segment fails to cross the network.

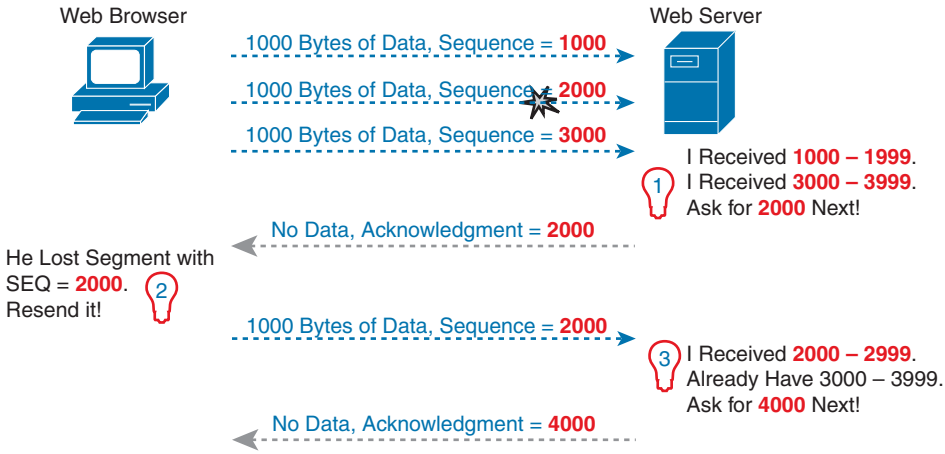



**Figure 5-8** *TCP Acknowledgment with Errors*

The figure points out three sets of ideas behind how the two hosts think. First, on the right, the server realizes that it did not receive all the data. The two received TCP segments contain bytes numbered 1000–1999 and 3000–3999. Clearly, the server did not receive the bytes numbered in between. The server then decides to acknowledge all the data up to the lost data—that is, to send back a segment with the Acknowledgment field equal to 2000.

The receipt of an acknowledgment that does not acknowledge all the data sent so far tells the sending host to resend the data. The PC on the left may wait a few moments to make sure no other acknowledgments arrive (using a timer called the retransmission timer), but will soon decide that the server means "I really do need 2000 next—resend it." The PC on the left does so, as shown in the fifth of the six TCP segments in the figure.

Finally, note that the server can acknowledge not only the re-sent data but also any earlier data that had been received correctly. In this case, the server received the re-sent second TCP segment (the data with sequence numbers 2000–2999), but the server had already received the third TCP segment (the data numbered 3000–3999). The server's next Acknowledgment field acknowledges the data in both of those segments, with an Acknowledgment field of 4000.

## Flow Control Using Windowing

TCP implements flow control by using a window concept that is applied to the amount of data that can be outstanding and awaiting acknowledgment at any one point in time. The window concept lets the receiving host tell the sender how much data it can receive right now, giving the receiving host a way to make the sending host slow down or speed up. The receiver can slide the window size up and down—called a **sliding window** or *dynamic window*—to change how much data the sending host can send.

The sliding window mechanism makes much more sense with an example. The example, shown in Figure 5-9, uses the same basic rules as the examples in the previous few figures. In this case, none of the TCP segments have errors, and the discussion begins one TCP segment earlier than in the previous two figures.

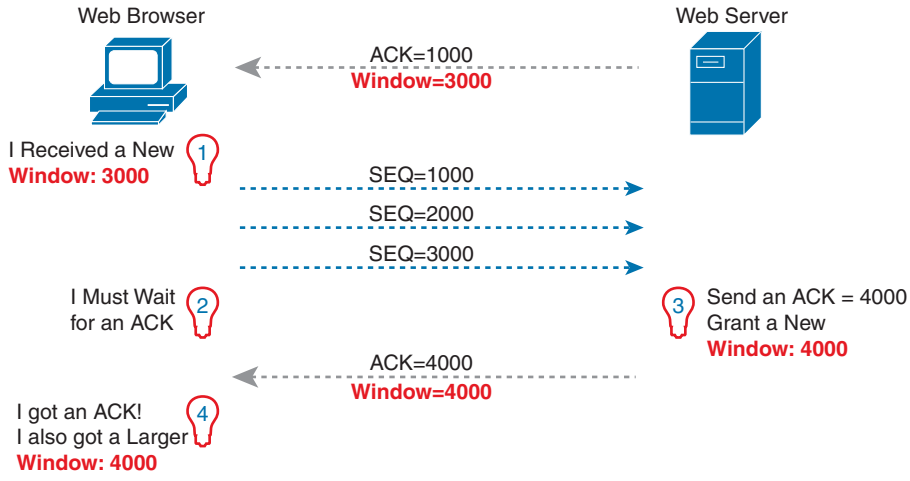



**Figure 5-9** *TCP Windowing*

Begin with the first segment, sent by the server to the PC. The Acknowledgment field should be familiar by now: it tells the PC that the server expects a segment with sequence number 1000 next. The new field, the window field, is set to 3000. Because the segment flows to the PC, this value tells the PC that the PC can send no more than 3000 bytes over this connection before receiving an acknowledgment. So, as shown on the left, the PC realizes it can send only 3000 bytes, and it stops sending, waiting on an acknowledgment, after sending three 1000-byte TCP segments.

Continuing the example, the server not only acknowledges receiving the data (without any loss) but also decides to slide the window size a little higher. Note that second message flowing right to left in the figure, this time with a window of 4000. When the PC receives this TCP segment, the PC realizes it can send another 4000 bytes (a slightly larger window than the previous value).

Note that while the last few figures show examples for the purpose of explaining how the mechanisms work, the examples might give you the impression that TCP makes the hosts sit there and wait for acknowledgments a lot. TCP does not want to make the sending host have to wait to send data. For instance, if an acknowledgment is received before the window is exhausted, a new window begins, and the sender continues sending data until the current window is exhausted. Often, in a network that has few problems, few lost segments, and little congestion, the TCP windows stay relatively large with hosts seldom waiting to send.

## User Datagram Protocol

UDP provides a service for applications to exchange messages. Unlike TCP, UDP is connectionless and provides no reliability, no windowing, no reordering of the received data, and no segmentation of large chunks of data into the right size for transmission. However, UDP provides some functions of TCP, such as data transfer and multiplexing using port numbers, and it does so with fewer bytes of overhead and less processing required than TCP.

UDP data transfer differs from TCP data transfer in that no reordering or recovery is accomplished. Applications that use UDP are tolerant of the lost data, or they have some

application mechanism to recover lost data. For example, VoIP uses UDP because if a voice packet is lost, by the time the loss could be noticed and the packet retransmitted, too much delay would have occurred, and the voice would be unintelligible. Also, DNS requests may use UDP because the user will retry an operation if the DNS resolution fails. As another example, the Network File System (NFS), a remote file system application, performs recovery with application layer code, so UDP features are acceptable to NFS.

Figure 5-10 shows the UDP header format. Most importantly, note that the header includes source and destination port fields, for the same purpose as TCP. However, the UDP has only 8 bytes, in comparison to the 20-byte TCP header shown in Figure 5-1. UDP needs a shorter header than TCP simply because UDP has less work to do.

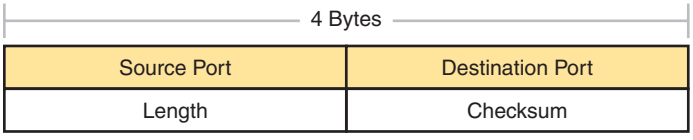



**Figure 5-10** *UDP Header*

# TCP/IP Applications

The whole goal of building an enterprise network, or connecting a small home or office network to the Internet, is to use applications such as web browsing, text messaging, email, file downloads, voice, and video. This section examines one particular application—web browsing using Hypertext Transfer Protocol (**HTTP**).

The World Wide Web (WWW) consists of all the Internet-connected web servers in the world, plus all Internet-connected hosts with web browsers. **Web servers**, which consist of web server software running on a computer, store information (in the form of *web pages*) that might be useful to different people. A *web browser*, which is software installed on an end user's computer, provides the means to connect to a web server and display the web pages stored on the web server.

**NOTE** Although most people use the term *web browser*, or simply *browser*, web browsers are also called *web clients*, because they obtain a service from a web server.

For this process to work, several specific application layer functions must occur. The user must somehow identify the server, the specific web page, and the protocol used to get the data from the server. The client must find the server's IP address, based on the server's name, typically using DNS. The client must request the web page, which actually consists of multiple separate files, and the server must send the files to the web browser. Finally, for electronic commerce (e-commerce) applications, the transfer of data, particularly sensitive financial data, needs to be secure. The following sections address each of these functions.

## Uniform Resource Identifiers

For a browser to display a web page, the browser must identify the server that has the web page, plus other information that identifies the particular web page. Most web servers have many web pages. For example, if you use a web browser to browse www.cisco.com and you click around that web page, you'll see another web page. Click again, and you'll see another web page. In each case, the clicking action identifies the server's IP address as well as the

specific web page, with the details mostly hidden from you. (These clickable items on a web page, which in turn bring you to another web page, are called *links*.)

The browser user can identify a web page when you click something on a web page or when you enter a Uniform Resource Identifier (**URI**) in the browser's address area. Both options—clicking a link and typing a URI—refer to a URI, because when you click a link on a web page, that link actually refers to a URI.

**NOTE** Most browsers support some way to view the hidden URI referenced by a link. In several browsers, hover the mouse pointer over a link, right-click, and select **Properties**. The pop-up window should display the URI to which the browser would be directed if you clicked that link.

In common speech, many people use the terms *web address* or the similar related terms *Universal Resource Locator* (or Uniform Resource Locator; URL) instead of URI, but URI is indeed the correct formal term. In fact, URL had been more commonly used than URI for more than a few years. However, the Internet Engineering Task Force, or IETF (the group that defines TCP/IP), along with the W3C consortium (W3.org, a consortium that develops web standards) has made a concerted effort to standardize the use of URI as the general term. See RFC 7595 for some commentary to that effect.

From a practical perspective, the URIs used to connect to a web server include three key components, as noted in Figure 5-11. The figure shows the formal names of the URI fields. More importantly to this discussion, note that the text before the :// identifies the protocol used to connect to the server, the text between the // and / identifies the server by name, and the text after the / identifies the web page.

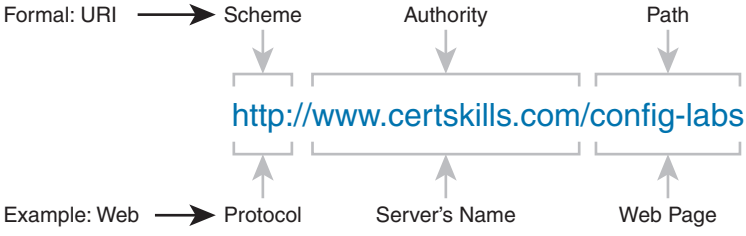



**Figure 5-11** *Structure of a URI Used to Retrieve a Web Page*

In this case, the protocol is Hypertext Transfer Protocol (HTTP), the hostname is www.certskills.com, and the name of the web page is config-labs.

## Finding the Web Server Using DNS

A host can use DNS to discover the IP address that corresponds to a particular hostname. URIs typically list the name of the server—a name that can be used to dynamically learn the IP address used by that same server. The web browser cannot send an IP packet to a destination name, but it can send a packet to a destination IP address. So, before the browser can send a packet to the web server, the browser typically needs to resolve the name inside the URI to that name's corresponding IP address.

To pull together several concepts, Figure 5-12 shows the DNS process as initiated by a web browser, as well as some other related information. From a basic perspective, the user enters the URI (in this case, http://www.cisco.com/go/learningnetwork), resolves the www.cisco.com name into the correct IP address, and starts sending packets to the web server.

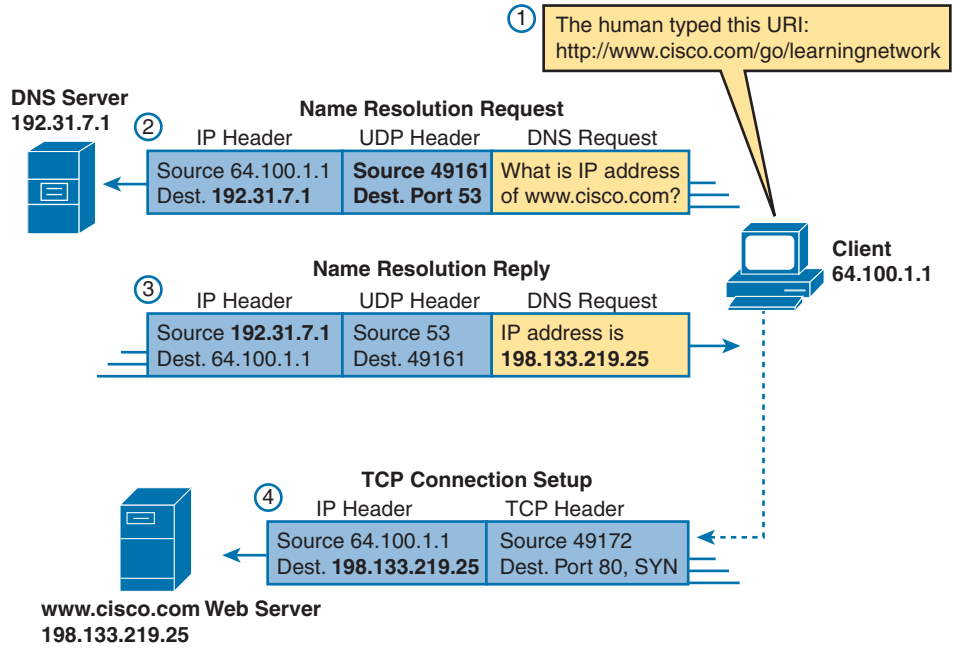



**Figure 5-12** *DNS Resolution and Requesting a Web Page*

The steps shown in the figure are as follows:

1. The user enters the URI, http://www.cisco.com/go/learningnetwork, into the browser's address area.
2. The client sends a DNS request to the DNS server. Typically, the client learns the DNS server's IP address through DHCP. DNS supports using either TCP or UDP, with UDP in this example. In either case, the protocol uses the well-known port 53. (See Table 5-3, earlier in this chapter, for a list of popular well-known ports.)
3. The DNS server sends a reply, listing IP address 198.133.219.25 as www.cisco.com's IP address. Note also that the reply shows a destination IP address of 64.100.1.1, the client's IP address. It also shows a UDP header, with source port 53; the source port is 53 because the data is sourced, or sent by, the DNS server.
4. The client begins the process of establishing a new TCP connection to the web server. Note that the destination IP address is the just-learned IP address of the web server. The packet includes a TCP header, because HTTP uses TCP. Also note that the destination TCP port is 80, the well-known port for HTTP. Finally, the SYN bit is shown, as a reminder that the TCP connection establishment process begins with a TCP segment with the SYN bit turned on (binary 1).

The example in Figure 5-12 shows what happens when the client host does not know the IP address associated with the hostname but the enterprise does know the address. However, hosts can cache the results of DNS requests so that for a time the client does not need to ask the DNS to resolve the name. Also, the DNS server can cache the results of previous DNS requests; for instance, the enterprise DNS server in Figure 5-12 would not normally have configured information about hostnames in domains outside that enterprise, so that example relied on the DNS having cached the address associated with hostname www.cisco.com.

When the local DNS does not know the address associated with a hostname, it needs to ask for help. Figure 5-13 shows an example with the same client as in Figure 5-12. In this case, the enterprise DNS acts as a **recursive DNS server**, sending repeated DNS messages in an effort to identify the authoritative DNS server.

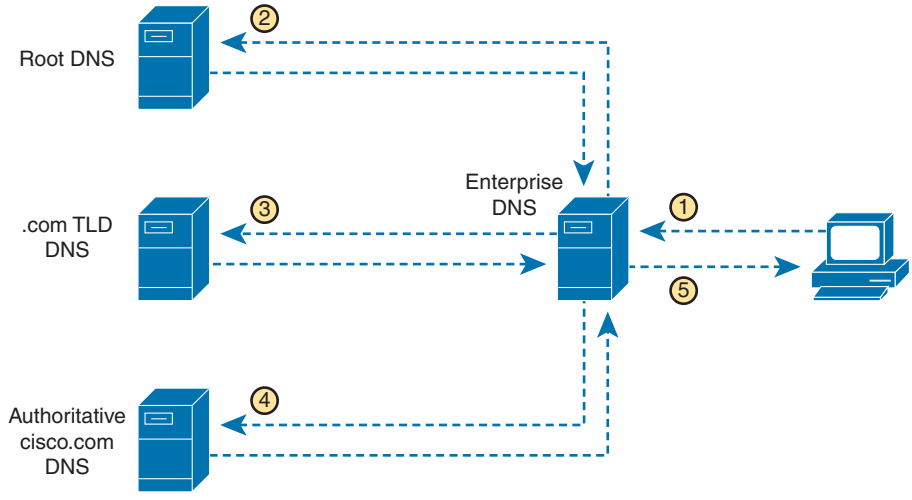



**Figure 5-13** *Recursive DNS Lookup*

The steps shown in the figure are as follows:

1. The client sends a DNS request for www.cisco.com to the DNS server it knows, which is the enterprise DNS server.
2. The (recursive) enterprise DNS server does not know the answer yet, but it does not reject the client's DNS request. Instead, it follows a repetitive (recursive) process (shown as steps 2, 3, and 4), beginning with the DNS request sent to a root DNS server. The root does not supply the address either, but it supplies the IP address of another DNS server, one responsible for the .com top-level domain.
3. The recursive enterprise DNS sends the next DNS request to the DNS server learned at the previous step—this time the TLD DNS server for the .com domain. This DNS also does not know the address, but it knows the DNS server that should be the authoritative DNS server for domain cisco.com, so it supplies that DNS server's address.
4. The enterprise DNS sends another DNS request, to the DNS server whose address was learned in the previous step, again asking for resolution of the name www.cisco.com. This DNS server, the authoritative server for cisco.com, supplies the address.

5. The enterprise DNS server returns a DNS reply back to the client, supplying the IP address requested at step 1.

## Transferring Files with HTTP

After a web client (browser) has created a TCP connection to a web server, the client can begin requesting the web page from the server. Most often, the protocol used to transfer the web page is HTTP. The HTTP application layer protocol, defined in RFC 7230, defines how files can be transferred between two computers. HTTP was specifically created for the purpose of transferring files between web servers and web clients.

HTTP defines several commands and responses, with the most frequently used being the HTTP GET request. To get a file from a web server, the client sends an HTTP GET request to the server, listing the filename. If the server decides to send the file, the server sends an HTTP GET response, with a return code of 200 (meaning OK), along with the file's contents.

**NOTE** Many return codes exist for HTTP requests. For example, when the server does not have the requested file, it issues a return code of 404, which means "file not found." Most web browsers do not show the specific numeric HTTP return codes, instead displaying a response such as "page not found" in reaction to receiving a return code of 404.

Web pages typically consist of multiple files, called *objects*. Most web pages contain text as well as several graphical images, animated advertisements, and possibly voice or video. Each of these components is stored as a different object (file) on the web server. To get them all, the web browser gets the first file. This file can (and typically does) include references to other URIs, so the browser then also requests the other objects. Figure 5-14 shows the general idea, with the browser getting the first file and then two others.

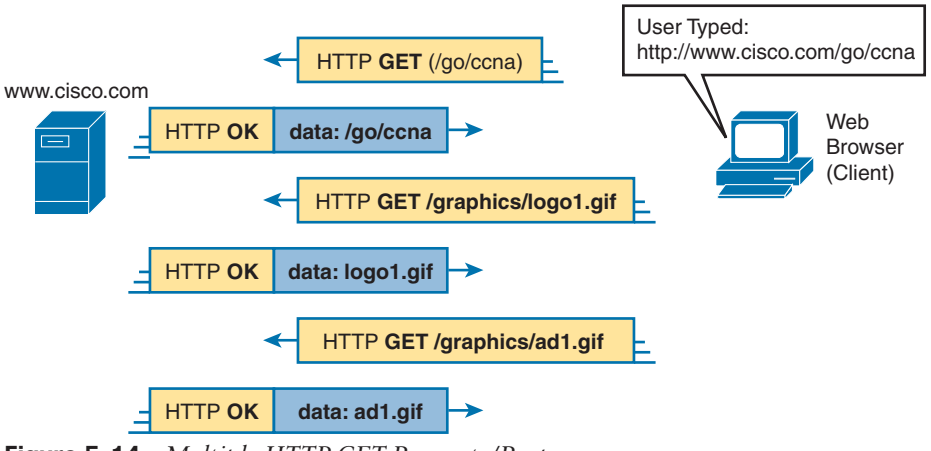



**Figure 5-14** *Multiple HTTP GET Requests/Responses*

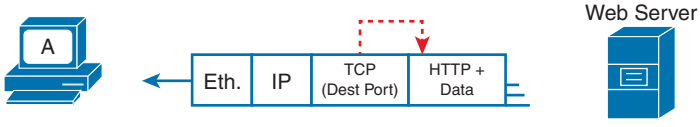
In this case, after the web browser gets the first file—the one called "/go/ccna" in the URI—the browser reads and interprets that file. Besides containing parts of the web page, the file refers to two other files, so the browser issues two additional HTTP GET requests. Note that, even though it isn't shown in the figure, all these commands flow over one (or possibly

more) TCP connection between the client and the server. This means that TCP would provide error recovery, ensuring that the data was delivered.

## How the Receiving Host Identifies the Correct Receiving Application

This chapter closes with a discussion of the process by which a host, when receiving any message over any network, can decide which of its many application programs should process the received data.

As an example, consider host A shown on the left side of Figure 5-15. The host happens to have three different web browser windows open, each using a unique TCP port. Host A also has an email client and a chat window open, both of which use TCP. Both the email and chat applications use a unique TCP port number on host A as shown in the figure.




Browser: TCP port 49124
Browser: TCP port 49125
Browser: TCP port 49126
Email: TCP port 49127
Chat: TCP port 49128

**Figure 5-15** *Dilemma: How Host A Chooses the App That Should Receive This Data*

This chapter has shown several examples of how transport layer protocols use the destination port number field in the TCP or UDP header to identify the receiving application. For instance, if the destination TCP port value in Figure 5-15 is 49124, host A will know that the data is meant for the first of the three web browser windows.

Before a receiving host can even examine the TCP or UDP header, and find the destination port field, it must first process the outer headers in the message. If the incoming message is an Ethernet frame that encapsulates an IPv4 packet, the headers look like the details in Figure 5-16.

Key Topic

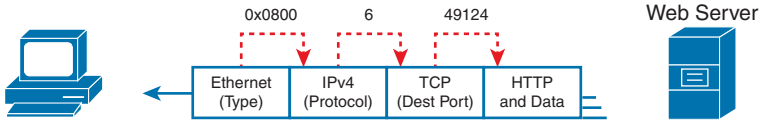



**Figure 5-16** *Three Key Fields with Which to Identify the Next Header*

The receiving host needs to look at multiple fields, one per header, to identify the next header or field in the received message. For instance, host A uses an Ethernet NIC to connect to the network, so the received message is an Ethernet frame. The Ethernet Type field identifies the type of header that follows the Ethernet header—in this case, with a value of hex 0800, an IPv4 header.

The IPv4 header has a similar field called the IP Protocol field. The IPv4 Protocol field has a standard list of values that identify the next header, with decimal 6 used for TCP and decimal 17 used for UDP. In this case, the value of 6 identifies the TCP header that follows the IPv4 header. Once the receiving host realizes a TCP header exists, it can process the destination port field to determine which local application process should receive the data.

## HTTP Versions

For much of the history of CCNA since its inception in 1998, you could ignore the details of HTTP versions and version differences. With the introduction of HTTP/3 around 2020, you need to be more aware of HTTP protocol versions. This section works through the versions and the differences that matter for CCNA.

### HTTP 1.0 and 1.1

Web browsers and servers first came into existence in the early 1990s. Eventually, the IETF took ownership of the Hypertext Transfer Protocol (HTTP), used by web browsers and servers to transfer objects. In those early years, the IETF published HTTP RFCs for HTTP 0.9, **HTTP 1.0**, and **HTTP 1.1**, with various improvements at each version. However, differences exist in how the application protocol works, with the details unimportant to building the network infrastructure. For instance, they all use HTTP and TCP, with the same HTTP GET Request and Response methods discussed in the last several pages of this chapter.

These early HTTP versions use common networking features. They all use TCP for the transport protocol, with IANA reserving well-known TCP port 80 for the server. HTTP defines URLs with a scheme (protocol) of http, with URLs like http://www.example.com. The URL can specify a nondefault server port number, but if omitted (as is typical), the URL implies the default well-known port. Also, the http scheme means no additional security features, such as authentication or encryption, are used.

From using web browsers, you might notice that URLs do not typically reference any kind of version number. Instead, the web browser and web servers support many or most versions. They connect using details of one version and can then use application-layer headers to signal a change to use a different version. As a user, you do not notice the difference. As a network engineer, with these earlier versions, you did not need to be concerned about which was used.

### HTTP/2 and TLS

Independent from the specific HTTP protocol versions, additional RFCs defined how to add security features to HTTP using Transport Layer Security (TLS). An RFC from the late 1990s defined **Secure HTTP**, also called HTTP over TLS. Because it added security to existing protocols, Secure HTTP uses the initial sequence like any other browser session, beginning with a TCP connection between the client and server. However, before sending HTTP messages, the endpoints create a TLS connection per the TLS protocol, implementing security features such as server authentication and encryption. Only then do the HTTP messages flow.

To signal that the connection uses Secure HTTP, the RFC defines a new URI scheme (protocol) of https, with URLs like https://www.example.com. (Almost all URLs you use daily begin with *https* and use Secure HTTP.)

From a networking perspective, two differences exist with HTTP 1.1 without TLS. The URLs differ, as just noted. Also, IANA reserves well-known TCP port 443 for Secure HTTP, so a URL that begins *https* implies TCP port 443 as the server's well-known port.

**NOTE** You might see references to HTTP versions as HTTP 1.1 or with a /, such as HTTP/1.1.

HTTP 1.1 and Secure HTTP/1.1 became the norm for many years, particularly into the 2000s and into the 2010s, with more and more traffic using the secure option over time. However, as time passed, many stakeholders saw issues with website performance with HTTP/1.1—particularly when transferring many objects with each web page. HTTP/2 and HTTP/3 attempt to address the performance issues with different approaches.

**HTTP/2**, which reached RFC status in the mid-2010s, improves the inner workings of the HTTP application layer protocol while keeping the same URL schemes, TCP transport, well-known ports, and the same use of HTTP over TLS. The changes impact the details of the HTTP application layer protocol but not the networking or security details. So, while important in IT, the introduction of HTTP/2 did not introduce new complexities for most network engineering projects because HTTP/2 uses the same TCP transport protocol and well-known ports as the earlier HTTP versions.

### HTTP 3.0

**HTTP/3** significantly shifts in HTTP performance with an approach that makes additional fundamental changes, including using UDP instead of TCP.

Google developed what has become HTTP/3 during the 2010s. While useful, Google's work found that TCP's error recovery and flow control features slowed performance for HTTP. To solve that problem, Google built a new transport protocol based on UDP called **QUIC** (not an acronym, just the name), now published in RFC 9369. QUIC also has some TCP-like features (error recovery, flow control), and it integrates the TLS work at the same time, reducing overhead flows. So, some HTTP/3 performance improvements come from migrating away from TCP plus TLS running independently to an approach that integrates UDP, QUIC, and TLS functions.

HTTP itself also changed. The IETF split some parts of the protocol into different RFCs in 2022 (RFCs 9110, 9111, and 9114), with HTTP/3 specifically in RFC 9114.

In the real world, HTTP/2 adoption showed slow growth, but HTTP/3 and its QUIC transport protocol have grown tremendously. According to some statistics from Cisco, Internet HTTP/3 traffic now exceeds HTTP/2 traffic. Many major Internet services, like Google, Facebook, YouTube, and Instagram, have embraced HTTP/3. Most of us have been using HTTP/3 without even being aware of it.

HTTP/3 changes several networking details compared to the other versions, most notably the headers. As discussed in the following few chapters, those changes impact what you can match with IP ACLs. While its RFC bills QUIC as a transport protocol, from a header perspective, it looks like another header that follows a UDP header. So, QUIC is an additional protocol that rides on UDP, as shown in Figure 5-17.

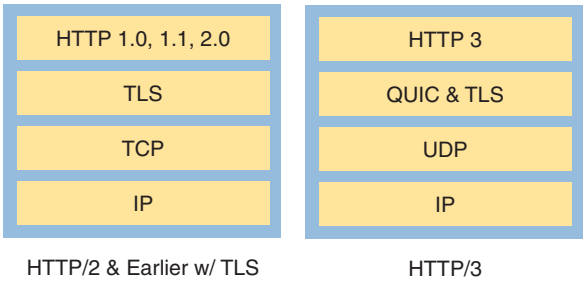



**Figure 5-17** *Comparing HTTP/3 with Earlier Versions*

**NOTE** HTTP/3 always uses TLS, partly because QUIC integrates TLS features but partly because the security features of TLS make sense for web connections. IANA reserves UDP port 443 as the well-known port for web servers when using HTTP/3.

To summarize, as a user, you do not choose what HTTP version to use. The browser and server will begin using a version and optionally change to another version based on the information exchanged between the two. As a network engineer, you should be aware of the versions, aware that HTTP messages in your network today include traditional TCP-based HTTP/1.0, HTTP/1.1, and HTTP/2 traffic to well-known TCP ports 80 and 443, but they also include messages with UDP headers using UDP port 443.

**NOTE** Web servers and most other server applications can use a port number other than their well-known reserved port. This chapter shows the predictable well-known ports.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 5-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 5-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |

## Review All the Key Topics

Key Topic

**Table 5-5** Key Topics for Chapter 5

| Key Topic Element | Description | Page Number |
|---|---|---|
| Table 5-2 | Functions of TCP and UDP | 94 |
| Table 5-3 | Well-known TCP and UDP port numbers | 99 |
| Figure 5-5 | Example of TCP connection establishment | 100 |
| List | Definitions of connection-oriented and connectionless | 101 |
| Figure 5-12 | DNS name resolution | 106 |
| Figure 5-16 | Header fields that identify the next header | 109 |

## Key Terms You Should Know

connection establishment, DNS server, error detection, error recovery, flow control, forward acknowledgment, HTTP, HTTP/1.0, HTTP/1.1, HTTP/2, HTTP/3, ordered data transfer, port, QUIC, recursive DNS server, Secure HTTP, segment, sliding windows, URI, web server



CHAPTER 6

# Basic IPv4 Access Control Lists

**This chapter covers the following exam topics:**

**5.0 Security Fundamentals**

**5.6 Configure and verify access control lists**

IPv4 access control lists (ACLs) allow network engineers to program a filter into a router. Each router, on each interface, for both the inbound and outbound direction, can enable a different IP ACL with different rules. Each IP ACL's rules tell the router which packets to discard and which to allow through.

This chapter discusses the basics of IPv4 ACLs, specifically a type by the name *standard numbered IP ACLs* or simply **standard access lists**. Standard numbered ACLs use simple logic, matching on the source IP address field only, and use a configuration style that references the ACL using a number. This chapter sets out to help you learn this simpler type of ACL first. The next chapter, titled "Named and Extended IP ACLs," discusses other variations of IP ACLs that use names (named ACLs) and that have more advanced matching logic than standard ACLs (extended ACLs).

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 6-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| IPv4 Access Control List Basics | 1 |
| Standard Numbered IPv4 ACLs | 2–6 |

1. Barney is a host with IP address 10.1.1.1 in subnet 10.1.1.0/24. Which of the following things could a standard IP ACL be configured to do? (Choose two answers.)
   a. Match the exact source IP address.
   b. Match IP addresses 10.1.1.1 through 10.1.1.4 with one **access-list** command without matching other IP addresses.
   c. Match all IP addresses in Barney's subnet with one **access-list** command without matching other IP addresses.
   d. Match only the packet's destination IP address.

2. Which of the following answers list a valid number that standard numbered IP ACLs can use? (Choose two answers.)
   a. 1987
   b. 2187
   c. 187
   d. 87
3. Which of the following wildcard masks is most useful for matching all IP packets in subnet 10.1.128.0, mask 255.255.255.0?
   a. 0.0.0.0
   b. 0.0.0.31
   c. 0.0.0.240
   d. 0.0.0.255
   e. 0.0.15.0
   f. 0.0.248.255
4. Which of the following wildcard masks is most useful for matching all IP packets in subnet 10.1.128.0, mask 255.255.240.0?
   a. 0.0.0.0
   b. 0.0.0.31
   c. 0.0.0.240
   d. 0.0.0.255
   e. 0.0.15.255
   f. 0.0.248.255
5. ACL 1 has three statements, in the following order, with address and wildcard mask values as follows: 1.0.0.0 0.255.255.255, 1.1.0.0 0.0.255.255, and 1.1.1.0 0.0.0.255. If a router tried to match a packet sourced from IP address 1.1.1.1 using this ACL, which ACL statement does a router consider the packet to have matched?
   a. First
   b. Second
   c. Third
   d. Implied deny at the end of the ACL
6. Which of the following **access-list** commands, taken from a router's running-config file, match all packets sent from hosts in subnet 172.16.4.0/23?
   a. **access-list 1 permit 172.16.0.5 0.0.255.0**
   b. **access-list 1 permit 172.16.4.0 0.0.1.255**
   c. **access-list 1 permit 172.16.5.0**
   d. **access-list 1 permit 172.16.5.0 0.0.0.127**

# Foundation Topics

## IPv4 Access Control List Basics

IPv4 **access control lists (IP ACLs)** allow network engineers to identify different types of packets. To do so, the ACL configuration lists values that the router can see in the IP, TCP, UDP, and other headers. For example, an ACL can match packets whose source IP address is 1.1.1.1, or packets whose destination IP address is some address in subnet 10.1.1.0/24, or packets with a destination port of TCP port 23 (Telnet).

IPv4 ACLs perform many functions in Cisco routers, with the most common use as a packet filter. Engineers can enable ACLs on a router so that the ACL sits in the forwarding path of packets as they pass through the router. After enabling it, the router considers whether each IP packet will either be discarded or allowed to continue as if the ACL did not exist.

However, ACLs can be used for many other IOS features as well. As an example, ACLs can be used to match packets for applying quality of service (QoS) features. QoS allows a router to give some packets better service and other packets worse service. For example, packets that hold digitized voice need to have very low delay, so ACLs can match voice packets, with QoS logic in turn forwarding voice packets more quickly than data packets.

This first section introduces IP ACLs as used for packet filtering. To better understand ACLs, the text first discusses the implications of the choice of the location and direction of the enabled ACL. Following that, the text examines the packet matching logic allowed in the configuration commands. The final section summarizes the actions to take when filtering packets: to deny (discard) the packet or permit the packet to continue along its original path.

### ACL Location and Direction

Cisco routers can apply ACL logic to packets at the point at which the IP packets enter an interface, or the point at which they exit an interface. In other words, the ACL can be applied inbound to the router, before the router makes its forwarding (routing) decision, or outbound, after the router makes its forwarding decision and has determined the exit interface to use.

The arrows in Figure 6-1 show where you could filter packets flowing left to right in the topology. For example, imagine that you wanted to allow packets sent by host A to server S1, but to discard packets sent by host B to server S1. Each arrowed line represents a location and direction at which a router could apply an ACL, filtering the packets sent by hosts A and B.

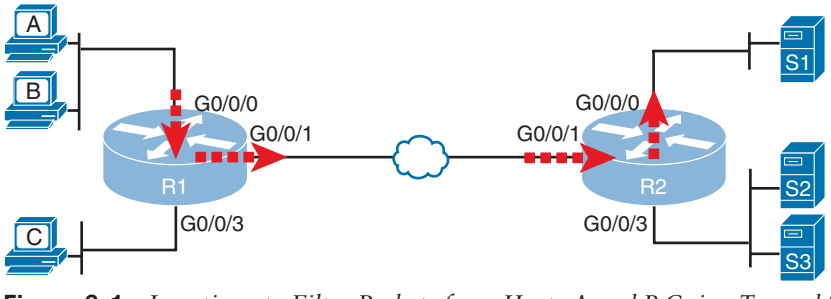



**Figure 6-1** *Locations to Filter Packets from Hosts A and B Going Toward Server S1*

---

Answers to the "Do I Know This Already?" quiz:

**1** A, C **2** A, D **3** D **4** E **5** A **6** B

The four arrowed lines in the figure point out the location and direction for the router interfaces used to forward the packet from host B to server S1. In this particular example, those interfaces and direction are inbound on R1's G0/0/0 interface, outbound on R1's G0/0/1 interface, inbound on R2's G0/0/1 interface, and outbound on R2's G0/0/0 interface.

To filter a packet, the ACL must be in the path that the packet travels. For example, if you enabled an ACL on R2's G0/0/3 interface, in either direction, that ACL could not possibly filter the packet sent from host B to server S1, because R2's G0/0/3 interface is not part of the route from host B to server S1. To summarize the logic:

Key Topic

> To filter a packet with an ACL, identify the inbound and outbound interfaces that the packet uses when passing through the router, and enable the ACL on one of those interfaces in that same direction.

When enabled, the router then processes every inbound or outbound IP packet using that ACL. For example, if enabled on R1 for packets inbound on interface G0/0/0, R1 would compare every inbound IP packet on G0/0/0 to the ACL to decide that packet's fate: to continue unchanged or to be discarded.

**NOTE** This book shows figures and examples of IP ACLs on routers. Multilayer switches configured to router IP packets also support IP ACLs using the same concepts and configuration.



## Matching Packets

When you think about the location and direction for an ACL, you must already be thinking about what packets you plan to filter (discard) and which ones you want to allow through. To tell the router those same ideas, you must configure the router with an IP ACL that matches packets. *Matching packets* refers to how to configure the ACL commands to look at each packet, listing how to identify which packets should be discarded and which should be allowed through.

Each IP ACL consists of one or more configuration commands, with each command listing details about values to look for inside a packet's headers. Generally, an ACL command uses logic like "look for these values in the packet header, and if found, discard the packet." (The action could instead be to allow the packet rather than discard.) Specifically, the ACL looks for header fields you should already know well, including the source and destination IP addresses, plus TCP and UDP port numbers.

For example, consider an example with Figure 6-2, in which you want to allow packets from host A to server S1, but to discard packets from host B going to that same server. The hosts all now have IP addresses, and the figure shows pseudocode for an ACL on R2. Figure 6-2 also shows the chosen location to enable the ACL: inbound on R2's G0/0/1 interface.

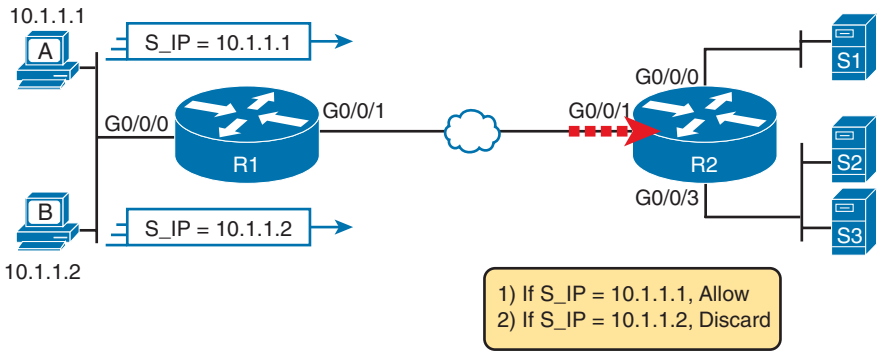



**Figure 6-2** *Pseudocode to Demonstrate ACL Command-Matching Logic*

Figure 6-2 shows a two-line ACL in a rectangle at the bottom, with simple matching logic: both statements just look to match the source IP address in the packet. When enabled, R2 looks at every inbound IP packet on that interface and compares each packet to those two ACL commands. Packets sent by host A (source IP address 10.1.1.1) are allowed through, and those sourced by host B (source IP address 10.1.1.2) are discarded.

## Taking Action When a Match Occurs

When IP ACLs are used to filter packets, only one of two actions can be chosen. The configuration commands use the keywords **deny** and **permit**, and they mean (respectively) to discard the packet or to allow it to keep going as if the ACL did not exist.

This book focuses on using ACLs to filter packets, but IOS uses ACLs for many more features. Those features typically use the same matching logic. However, in other cases, the **deny** or **permit** keywords imply some other action.

## Types of IP ACLs

Cisco IOS has supported IP ACLs since the early days of Cisco routers. Beginning with the original standard numbered IP ACLs in the early days of IOS, which could enable the logic shown earlier around Figure 6-2, Cisco has added many ACL features, including the following:

- Standard numbered ACLs (1–99)
- Extended numbered ACLs (100–199)
- Additional ACL numbers (1300–1999 standard, 2000–2699 extended)
- Named ACLs
- Improved editing with sequence numbers

This chapter focuses solely on standard numbered IP ACLs, while the next chapter discusses the other three primary categories of IP ACLs. Briefly, IP ACLs will be either numbered or named in that the configuration identifies the ACL either using a number or a name. ACLs will also be either standard or extended, with extended ACLs having much more robust abilities in matching packets. Figure 6-3 summarizes the big ideas related to categories of IP ACLs.

Key Topic

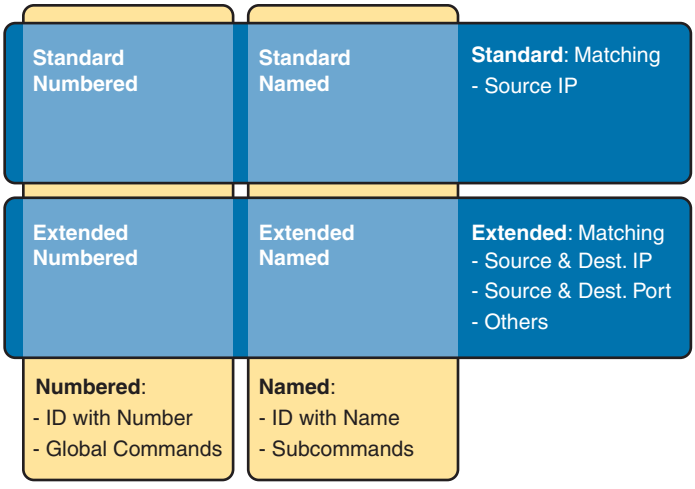



**Figure 6-3** *Comparisons of IP ACL Types*

## Standard Numbered IPv4 ACLs

The title of this section serves as a great introduction, if you can decode what Cisco means by each specific word. This section is about a type of Cisco filter (*ACL*) that matches only the source IP address of the packet (*standard*), is configured to identify the ACL using numbers rather than names (*numbered*), and looks at IPv4 packets.

This section first examines the idea that one ACL contains a list of commands, each containing matching and action logic. Following that, the text closely looks at how to match the source IP address field in the packet header, including the syntax of the commands. This section ends with a complete look at the configuration and verification commands to implement standard ACLs.

### List Logic with IP ACLs

A single ACL is both a single entity and, at the same time, a list of one or more configuration commands. As a single entity, the configuration enables the entire ACL on an interface, in a specific direction, as shown earlier in Figure 6-1. As a list of commands, each command has different matching logic that the router must apply to each packet when filtering using that ACL.

When doing ACL processing, the router processes the packet, compared to the ACL, as follows:

Key Topic

> ACLs use first-match logic. Once a packet matches one line in the ACL, the router takes the action listed in that line of the ACL and stops looking further in the ACL.

To see exactly what that means, consider the example built around Figure 6-4. The figure shows a sample ACL 1 with three lines of pseudocode. This example applies ACL 1 on R2's G0/0/1 interface, inbound (the same location as in Figure 6-2).

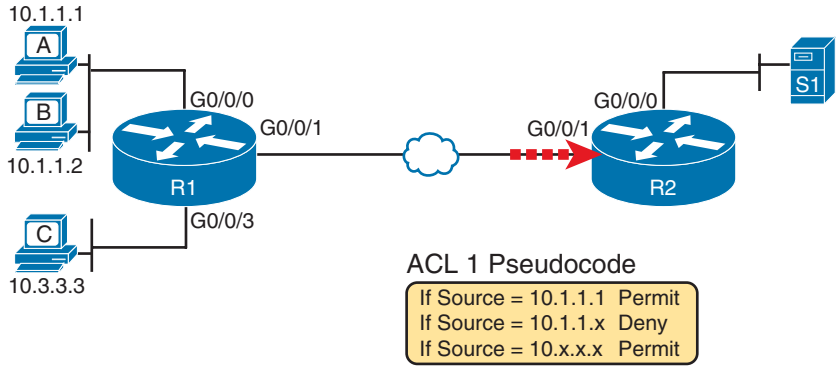



**Figure 6-4** *Backdrop for Discussion of List Process with IP ACLs*

Consider the first-match ACL logic for a packet sent by host A to server S1. The source IP address will be 10.1.1.1 and routed so that it enters R2's G0/0/1 interface, driving R2's ACL 1 logic. R2 compares this packet to the ACL, matching the first item in the list with a permit action. This packet should be allowed through, as shown in Figure 6-5, on the left.

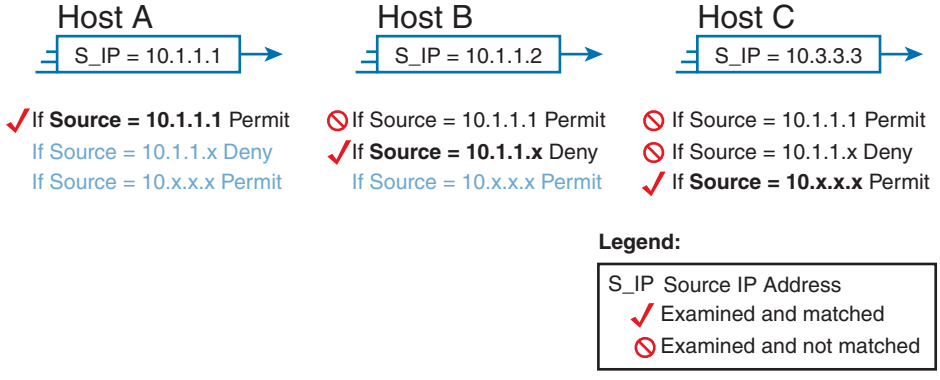



**Figure 6-5** *ACL Items Compared for Packets from Hosts A, B, and C in Figure 6-4*

Next, consider a packet sent by host B, source IP address 10.1.1.2. When the packet enters R2's G0/0/1 interface, R2 compares the packet to ACL 1's first statement and does not make a match (10.1.1.1 is not equal to 10.1.1.2). R2 then moves to the second statement, which requires some clarification. The ACL pseudocode, back in Figure 6-4, shows 10.1.1.x, which is meant to be shorthand that any value can exist in the last octet. Comparing only the first three octets, R2 decides that this latest packet does have a source IP address that begins with the first three octets 10.1.1, so R2 considers that to be a match on the second statement. R2 takes the listed action (deny), discarding the packet. R2 also stops ACL processing on the packet, ignoring the third line in the ACL.

Finally, consider a packet sent by host C, again sent to server S1. The packet has source IP address 10.3.3.3, so when it enters R2's G0/0/1 interface and drives ACL processing on R2, R2 looks at the first command in ACL 1. R2 does not match the first ACL command (10.1.1.1 in the command is not equal to the packet's 10.3.3.3). R2 looks at the second command, compares the first three octets (10.1.1) to the packet source IP address (10.3.3), and still finds

no match. R2 then looks at the third command. In this case, the wildcard means ignore the last three octets and just compare the first octet (10), so the packet matches. R2 then takes the listed action (permit), allowing the packet to keep going.

All Cisco IOS ACLs use this first-match logic: IP ACLs, IPv6 ACLs, standard or extended, named or numbered.

Finally, every ACL ends with *deny any* logic, the equivalent of an **access-list** *number* **deny any** command. It does not exist in the configuration, but if a router keeps searching the list, making no match by the end of the list, IOS considers the packet to have matched the implied deny any, with the router discarding the packet.

## Matching Logic and Command Syntax

Standard numbered IP ACLs use the following global command:

```
access-list {1-99 | 1300-1999} {permit | deny} matching-parameters
```

Each standard numbered ACL has one or more **access-list** commands with the same number, any number from the ranges shown in the preceding line of syntax. One number is no better than the other. IOS refers to each line in an ACL as an **access control entry (ACE)**, but many engineers just call them ACL statements.

After the ACL number, the **access-list** global command lists the action (**permit** or **deny**) and then the matching logic. The next few pages focus on configuring the source address's matching parameters.



### Matching the Exact IP Address

To match a specific source IP address, configure that IP address at the end of the command. For example, the previous example uses pseudocode for "permit if source = 10.1.1.1." The following command configures that logic with correct syntax using ACL number 1:

```
access-list 1 permit 10.1.1.1
```

For historical reasons, IOS (and IOS XE) support two alternative syntax options for standard ACLs to match a single address. You could use either of the following two options to configure the same logic as the previous command:

```
access-list 1 permit host 10.1.1.1
access-list 1 permit 10.1.1.1 0.0.0.0
```

The first of the alternate options uses a **host** keyword before the single address, with the other listing the address followed by a value called a **wildcard mask** (in this case, 0.0.0.0). Although IOS accepts both additional formats, it stores the command with the first syntax (the version with neither the **host** keyword nor 0.0.0.0 wildcard mask). (The text defines the wildcard mask and its use in the next topic.) Table 6-2 summarizes the options for easier study and review.

Key Topic

**Table 6-2** Summary of Standard ACL Syntax Supported to Match One IP Address

| | *address* **0.0.0.0** | **host** *address* | *address* |
|---|---|---|---|
| **Standard ACLs** | Yes | Yes | Yes* |

*IOS stores the command with this syntax no matter the configuration style.

### Matching a Subset of the Address with Wildcard Masks

You often need to match a range of IP addresses rather than a single one. Maybe you want to match all IP addresses in a subnet. Maybe you want to match all IP addresses in a range of subnets. Regardless, you want to check for more than one IP address in a range of addresses.

IOS allows standard ACLs to match a range of addresses using a tool called a *wildcard mask*. Note that this is not a subnet mask. The wildcard mask (which this book abbreviates as *WC mask*) gives the engineer a way to tell IOS to ignore parts of the address when making comparisons, essentially treating those parts as wildcards, as if they already matched.

You can think about WC masks in decimal and in binary, and both have their uses. To begin, think about WC masks in decimal, using these rules:

Key Topic

**Decimal 0:** The router must compare this octet as normal.

**Decimal 255:** The router ignores this octet, considering it to match already.

Keeping these two rules in mind, consider Figure 6-6, which demonstrates this logic using three different but popular WC masks: one that tells the router to ignore the last octet, one that tells the router to ignore the last two octets, and one that tells the router to ignore the last three octets.

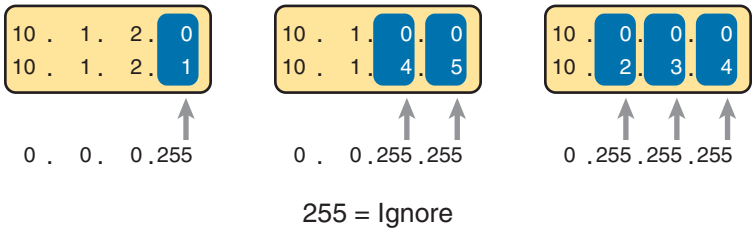



**Figure 6-6** *Logic for WC Masks 0.0.0.255, 0.0.255.255, and 0.255.255.255*

All three examples in the boxes of Figure 6-6 show clearly different numbers. The WC mask causes IOS to compare only some of the octets, while ignoring other octets. All three examples result in a match, because each wildcard mask tells IOS to ignore some octets. The example on the left shows WC mask 0.0.0.255, which tells the router to treat the last octet as a wildcard, essentially ignoring that octet for the comparison. Similarly, the middle example shows WC mask 0.0.255.255, which tells the router to ignore the two octets on the right. The rightmost case shows WC mask 0.255.255.255, telling the router to ignore the last three octets when comparing values.

To see the WC mask in action, think back to the earlier examples related to Figures 6-4 and 6-5. The pseudocode ACL in those two figures used logic that can be created using a WC mask. As a reminder, the logic in the pseudocode ACL in those two figures included the following:

**Line 1:** Match and permit the packets with a source address of exactly 10.1.1.1.

**Line 2:** Match and deny the packets with source addresses with first three octets 10.1.1.

**Line 3:** Match and permit the packets with addresses with a first octet value of 10.

The **access-list** command matches a range of addresses by listing two parameters: the first number in the range plus a wildcard mask. As an example, Figure 6-7 shows the updated version of Figure 6-4, but with the completed, correct ACL syntax, including the WC masks. In particular, note the use of WC mask 0.0.0.255 in the second ACE, telling R2 to ignore the last octet of the number 10.1.1.0, and the WC mask 0.255.255.255 in the third ACE, telling R2 to ignore the last three octets in the value 10.0.0.0.

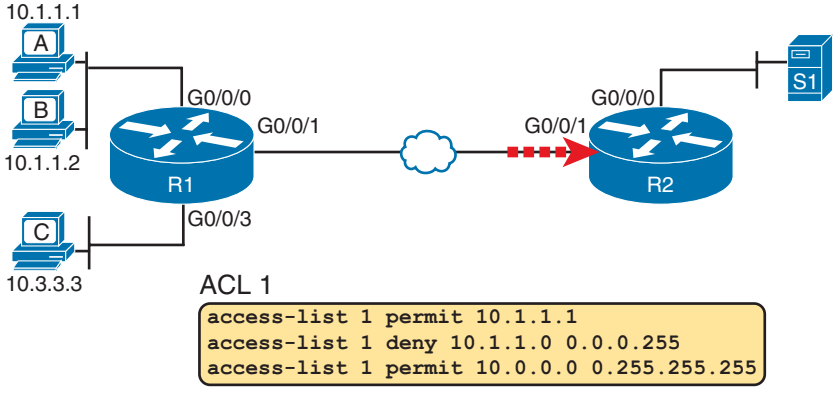



**Figure 6-7** *Syntactically Correct ACL Replaces Pseudocode from Figure 6-4*

The generic syntax of the command, **access-list** *number action source* [*wildcard-mask*], refers to the matching parameters as source and wildcard mask. When using a wildcard mask, the source value defines the first number in the range of addresses.

## Binary Wildcard Masks

Wildcard masks, as dotted-decimal number (DDN) values, represent a 32-bit binary number. As a 32-bit number, the WC mask directs the router's logic bit-by-bit rather than octet-by-octet. In short, compare the two numbers bit-by-bit when the WC mask bit equals 0, but ignore the bits when the WC mask bit in the same relative position is a binary 1.

Thankfully, for CCNA study, and for most real-world applications, you can ignore the binary WC mask. Why? Well, we generally want to match a range of addresses that can be easily identified by a subnet number and mask, whether it be a real subnet or a summary route that groups subnets together. If you can describe the range of addresses with a subnet number and mask, you can find the numbers to use in your ACL with some simple decimal math, as discussed next.

**NOTE** If you really want to know the binary mask logic, take the two DDN numbers the ACL will compare (one from the **access-list** command and the other from the packet header) and convert both to binary. Then, also convert the WC mask to binary, so you now have three 32-bit numbers. Ignore the bit for any bits in which the wildcard mask bit is a 1, but for wildcard mask bits of 0, compare that bit in the two addresses. If all the bits you checked are equal, it's a match!

### Finding the Right Wildcard Mask to Match a Subnet

To match a subnet with an ACL, you can use the following shortcut:




- Use the subnet number as the source value in the **access-list** command.
- Use a wildcard mask found by subtracting the subnet mask from 255.255.255.255.

For example, for subnet 172.16.8.0 255.255.252.0, use the subnet number (172.16.8.0) as the source parameter, and then do the following math to find the wildcard mask:

```
  255.255.255.255
– 255.255.252.0
    0. 0. 3.255
```

Continuing this example, a completed command for this same subnet would be as follows:

```
access-list 1 permit 172.16.8.0 0.0.3.255
```

The section "Practice Applying Standard IP ACLs" gives you a chance to practice matching subnets when configuring ACLs.

### Matching Any/All Addresses

In some cases, you will want one ACL command to match any and all packets that reach that point in the ACL. First, you have to know the (simple) way to match all packets using the **any** keyword. More importantly, you need to think about when to match any and all packets.

First, to match any and all packets with an ACL command, just use the **any** keyword for the address. For example, to permit all packets:

```
access-list 1 permit any
```

So, when and where should you use such a command? Remember, all Cisco IP ACLs end with an implicit **deny any** concept at the end of each ACL. That is, if a router compares a packet to the ACL, and the packet matches none of the configured ACEs, the router discards the packet. Want to override that default behavior? Configure a **permit any** at the end of the ACL.

You might also want to explicitly configure a command to deny all traffic (for example, **access-list 1 deny any**) at the end of an ACL. Why, when the same logic already sits at the end of the ACL anyway? Well, the ACL **show** commands list counters for the number of packets matched by each command in the ACL, but there is no counter for that implicit **deny any** concept at the end of the ACL. So, configure an explicit deny any if you want to see counters for how many packets are matched by the logic at the end of the ACL.

## Implementing Standard IP ACLs

This chapter has already introduced all the configuration steps in bits and pieces. This section summarizes those pieces as a configuration process. The process also refers to the **access-list** command, whose generic syntax is repeated here for reference:

```
access-list access-list-number {deny | permit} source [source-wildcard]
```

Key Topic

**Step 1.** Plan the location (router and interface) and direction (in or out) on that interface:

- **a.** Standard ACLs should be placed near the packet's destination so that they do not unintentionally discard packets that should not be discarded.
- **b.** Because standard ACLs can only match a packet's source IP address, identify the source IP addresses of packets as they go in the direction that the ACL is examining.

**Step 2.** Configure one or more **access-list** global configuration commands to create the ACL, keeping the following in mind:

- **a.** The list is searched sequentially, using first-match logic.
- **b.** The default action, if a packet does not match any of the **access-list** commands, is to **deny** (discard) the packet.

**Step 3.** Enable the ACL on the chosen router interface, in the correct direction, using the **ip access-group** *number* {**in** | **out**} interface subcommand.

6

Note that the **ip access-group** interface subcommand works on interfaces enabled for IP routing. That is, the interface has an IP address configured. The examples in this chapter use physical interfaces on routers; however, in other cases, a router or multilayer switch enables IP routing on subinterfaces or VLAN interfaces. In those cases, you can enable an ACL on those interfaces. For a reminder of some of the various types of interfaces a router or switch might enable IP, refer to *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, Chapter 18, "IP Routing in the LAN."

### Standard Numbered ACL Scenario 1

The first ACL scenario shows the configuration for the same requirements demonstrated with Figures 6-4 and 6-5. Restated, the requirements for this ACL are as follows:

1. Enable the ACL inbound on router R2's G0/0/1 interface.
2. Permit packets coming from host A.
3. Deny packets coming from other hosts in host A's subnet.
4. Permit packets coming from any other address in Class A network 10.0.0.0.
5. The original example made no comment about what to do by default, so simply deny all other traffic.

Example 6-1 shows a completed correct configuration, starting with the configuration process, followed by output from the **show running-config** command.

**Example 6-1** *Standard Numbered ACL Example 1 Configuration*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# access-list 1 permit 10.1.1.1
R2(config)# access-list 1 deny 10.1.1.0 0.0.0.255
R2(config)# access-list 1 permit 10.0.0.0 0.255.255.255
R2(config)# interface gigabitethernet 0/0/1
R2(config-if)# ip access-group 1 in
R2(config-if)# ^Z
R2# show running-config
! Lines omitted for brevity

access-list 1 permit 10.1.1.1
access-list 1 deny 10.1.1.0 0.0.0.255
access-list 1 permit 10.0.0.0 0.255.255.255
```

First, pay close attention to the configuration process at the top of Example 6-1. Note that the **access-list** command does not change the command prompt from the global configuration mode prompt because the **access-list** command is a global configuration command. Then, compare that to the output of the **show running-config** command: the details are identical to the commands added in configuration mode. Finally, make sure to note the **ip access-group 1 in** command, under R2's G0/0/1 interface, which enables the ACL logic (both location and direction).

Example 6-2 lists some output from router R2 that shows information about this ACL. The **show ip access-lists** command lists details about IPv4 ACLs only, while the **show access-lists** command lists details about IPv4 ACLs plus any other types of ACLs that are currently configured—for example, IPv6 ACLs. These commands list the same output in the same format.

**Example 6-2** *ACL* **show** *Commands on R2*

```
R2# show ip access-lists
Standard IP access list 1
    10 permit 10.1.1.1 (107 matches)
    20 deny   10.1.1.0, wildcard bits 0.0.0.255 (4 matches)
    30 permit 10.0.0.0, wildcard bits 0.255.255.255 (10 matches)
R2# show access-lists
Standard IP access list 1
    10 permit 10.1.1.1 (107 matches)
    20 deny   10.1.1.0, wildcard bits 0.0.0.255 (4 matches)
    30 permit 10.0.0.0, wildcard bits 0.255.255.255 (10 matches)
R2# show ip interface g0/0/1
GigabitEthernet0/0/1 is up, line protocol is up
  Internet address is 10.1.2.2/24
  Broadcast address is 255.255.255.255
```

```
  Address determined by non-volatile memory
  MTU is 1500 bytes
  Helper address is not set
  Directed broadcast forwarding is disabled
  Multicast reserved groups joined: 224.0.0.9
  Outgoing access list is not set
  Inbound access list is 1
! Lines omitted for brevity
```

The output of both the **show ip access-lists** and **show access-lists** commands show two items of note. The first line of output notes the type (standard) and the number. If more than one ACL existed, you would see multiple stanzas of output, one per ACL, each with a heading line like this one. Next, these commands list packet counts for the number of packets that the router has matched with each ACE. For example, 107 packets so far have matched the ACE. (When troubleshooting, you can use the **clear ip access-list counters** command to reset all these counters to 0.)

The end of Example 6-2 also lists the **show ip interface** command output. This command lists, among many other items, the number or name of any IP ACL enabled on the interface per the **ip access-group** interface subcommand.

## Standard Numbered ACL Scenario 2

For the second ACL scenario, use Figure 6-8, and imagine your boss gives you some requirements hurriedly in the hall. At first, he tells you he wants to filter packets going from the servers on the right toward the clients on the left. Then, he says he wants you to allow access for hosts A, B, and other hosts in their same subnet to server S1, but deny access to that server to the hosts in host C's subnet. Then, he tells you that, additionally, hosts in host A's subnet should be denied access to server S2, but hosts in host C's subnet should be allowed access to server S2—all by filtering packets going right to left only. He then tells you to put the ACL inbound on R2's G0/0/0 interface.

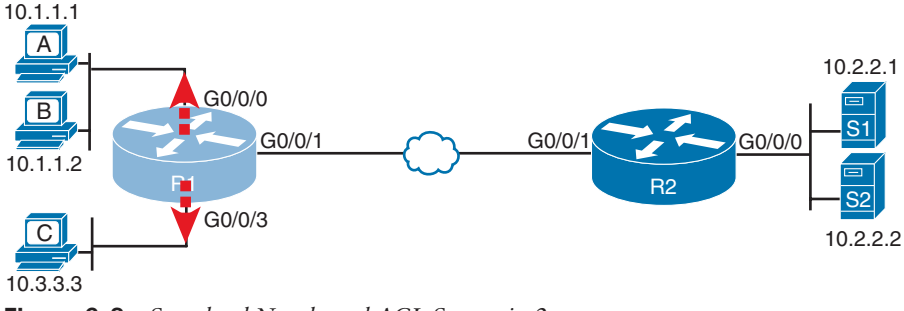



**Figure 6-8** *Standard Numbered ACL Scenario 2*

If you cull through all the boss's comments, the requirements might be reduced to the following:

1. Enable the ACL inbound on R2's G0/0/0 interface.
2. Permit packets from server S1 going to hosts in A's subnet.

3. Deny packets from server S1 going to hosts in C's subnet.
4. Permit packets from server S2 going to hosts in C's subnet.
5. Deny packets from server S2 going to hosts in A's subnet.
6. (There was no comment about what to do by default; use the implied **deny any** default.)

As it turns out, you cannot do everything your boss asked with a standard ACL. For example, consider the obvious command for requirement number 2: **access-list 2 permit 10.2.2.1**. That permits all traffic whose source IP is 10.2.2.1 (server S1). The next requirement asks you to filter (deny) packets sourced from that same IP address! Even if you added another command that checked for source IP address 10.2.2.1, the router would never get to it because routers use first-match logic when searching the ACL. You cannot check both the destination and source IP address because standard ACLs cannot check the destination IP address.

To solve this problem, you should get a new boss! No, seriously, you should rethink the problem and change the rules. In real life, you would probably use an extended ACL instead, which lets you check both the source and destination IP address.

For the sake of practicing another standard ACL, imagine your boss lets you change the requirements. The logic will still consider packets moving right to left in the figure; however, the design uses two outbound ACLs, both on router R1. Each ACL will permit traffic from a single server to be forwarded onto that connected LAN, with the following modified requirements:

1. Using an outbound ACL on R1's G0/0/0 interface, permit packets from server S1, and deny all other packets.
2. Using an outbound ACL on R1's G0/0/3 interface, permit packets from server S2, and deny all other packets.

Example 6-3 shows the configuration that completes these requirements.

**Example 6-3** *Alternative Configuration in Router R1*

```
access-list 2 remark This ACL permits server S1 traffic to host A's subnet
access-list 2 permit 10.2.2.1
!
access-list 3 remark This ACL permits server S2 traffic to host C's subnet
access-list 3 permit 10.2.2.2
!
interface G0/0/0
 ip access-group 2 out
!
interface G0/0/3
 ip access-group 3 out
```

As highlighted in the example, the solution with ACL number 2 permits all traffic from server S1, with that logic enabled for packets exiting R1's G0/0/0 interface. All other traffic will be discarded because of the implied **deny any** at the end of the ACL. In addition, ACL 3 permits traffic from server S2, which is then permitted to exit R1's G0/0/3 interface.

Finally, make sure to notice the **access-list remark** parameter, which allows you to leave text documentation that stays with the ACL. You can add as many remarks as you like into a single ACL.

**NOTE** Routers treat self-created packets differently than packets routed (received and then forwarded) by the router. A router applies any outbound ACL logic to routed packets but bypasses the ACL for self-created packets. Examples of those packets include routing protocol messages and packets sent by the **ping** and **traceroute** commands on that router.

## Troubleshooting and Verification Tips

Troubleshooting IPv4 ACLs requires some attention to detail. In particular, you have to be ready to look at the address and wildcard mask and confidently predict the addresses matched by those two combined parameters. The upcoming practice problems a little later in this chapter can help prepare you for that part of the work. But a few other tips can help you verify and troubleshoot ACL problems on the exams as well.

First, you can tell whether the router matches packets with a couple of tools. Example 6-2 already showed that IOS keeps statistics about the packets matched by each line of an ACL. In addition, if you add the **log** keyword to the end of an **access-list** command, IOS then issues log messages with occasional statistics about matches of that particular line of the ACL. Both the statistics and the log messages can be helpful in deciding which line in the ACL is being matched by a packet.

For example, Example 6-4 shows an updated ACL 2 from Example 6-3, with the **log** keyword added. The bottom of the example then shows a typical log message, showing the resulting match based on a packet with source IP address 10.2.2.1 (as matched with the ACL) to destination address 10.1.1.1.

**Example 6-4** *Creating Log Messages for ACL Statistics*

```
R1# show running-config
! lines removed for brevity
access-list 2 remark This ACL permits server S1 traffic to host A's subnet
access-list 2 permit 10.2.2.1 log
!
interface G0/0/0
 ip access-group 2 out

R1#
Feb 4 18:30:24.082: %SEC-6-IPACCESSLOGNP: list 2 permitted 0 10.2.2.1 -> 10.1.1.1,
1 packet
```

Also, when you troubleshoot an ACL for the first time, begin by thinking about both the interface on which the ACL is enabled and the direction of packet flow. Ignore the matching logic at first. Sometimes, the matching logic is perfect—but the ACL has been enabled on the wrong interface, or for the wrong direction, to match the packets as configured for the ACL.

For example, Figure 6-9 repeats the same ACL shown earlier in Figure 6-7. The first line of that ACL matches the specific host address 10.1.1.1. If that ACL exists on router R2, placing that ACL as an inbound ACL on R2's G0/0/1 interface can work because packets sent by host 10.1.1.1—on the left side of the figure—can enter R2's G0/0/1 interface. However, if R2 enables ACL 1 on its G0/0/0 interface, for inbound packets, the ACL will never match a packet with source IP address 10.1.1.1 because packets sent by host 10.1.1.1 will never enter that interface. Packets sent by 10.1.1.1 will exit R2's G0/0/0 interface, but never enter it, just because of the network topology.

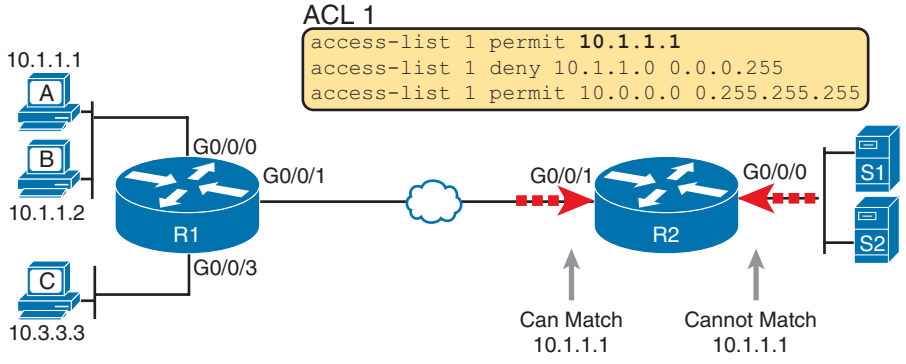



**Figure 6-9** *Example of Checking the Interface and Direction for an ACL*

## Practice Applying Standard IP ACLs

Some CCNA topics, like ACLs, simply require more drills and practice than others. ACLs require you to think of parameters to match ranges of numbers, and that of course requires some use of math and some use of processes.

This section provides some practice problems and tips, from two perspectives. First, this section asks you to build one-line standard ACLs to match some packets. Second, this section asks you to interpret existing ACL commands to describe what packets the ACL will match. Both skills are useful for the exam.

### Practice Building access-list Commands

In this section, practice getting comfortable with the syntax of the **access-list** command, particularly with choosing the correct matching logic. These skills will be helpful when reading about extended and named ACLs in the next chapter.

First, the following list summarizes some important tips to consider when choosing matching parameters to any **access-list** command:

Key Topic

- To match a specific address, just list the address.
- To match any and all addresses, use the **any** keyword.
- To match based only on the first one, two, or three octets of an address, use the 0.255.255.255, 0.0.255.255, and 0.0.0.255 WC masks, respectively. Also, make the source (address) parameter have 0s in the wildcard octets (those octets with 255 in the wildcard mask).

- To match a subnet, use the subnet ID as the source, and find the WC mask by subtracting the DDN subnet mask from 255.255.255.255.

Table 6-3 lists the criteria for several practice problems. Your job: Create a standard ACL ACE that matches the packets. The answers are listed in the section "Answers to Earlier Practice Problems," later in this chapter.

**Table 6-3** Building One-Line Standard ACLs: Practice

| Problem | Criteria |
|---|---|
| 1 | Packets from 172.16.5.4 |
| 2 | Packets from hosts with 192.168.6 as the first three octets |
| 3 | Packets from hosts with 192.168 as the first two octets |
| 4 | Packets from any host |
| 5 | Packets from subnet 10.1.200.0/21 |
| 6 | Packets from subnet 10.1.200.0/27 |
| 7 | Packets from subnet 172.20.112.0/23 |
| 8 | Packets from subnet 172.20.112.0/26 |
| 9 | Packets from subnet 192.168.9.64/28 |
| 10 | Packets from subnet 192.168.9.64/30 |



## Reverse Engineering from ACL to Address Range

In some cases, you may not be creating your own ACL. Instead, you may need to interpret some existing **access-list** commands. To answer these types of questions on the exams, you need to determine the range of IP addresses matched by a particular address/wildcard mask combination in each ACL statement.

Under certain assumptions that are reasonable for CCNA certifications, calculating the range of addresses matched by an ACL can be relatively simple. Basically, the range of addresses begins with the source parameter configured in the ACL command. The range of addresses ends with the sum of the source parameter and the wildcard mask. That's it.

For example, with the command **access-list 1 permit 172.16.200.0 0.0.7.255**, the low end of the range is simply 172.16.200.0, taken directly from the command itself. Then, to find the high end of the range, just add this number to the WC mask, as follows:

```
172.16.200.0
+ 0.  0.   7.255
172.16.207.255
```

For this last bit of practice, look at the existing **access-list** commands in Table 6-4. In each case, make a notation about the exact IP address, or range of IP addresses, matched by the command.

**Table 6-4** Finding IP Addresses/Ranges Matching by Existing ACLs

| Problem | Commands for Which to Predict the Source Address Range |
|---|---|
| 1 | **access-list 1 permit 10.7.6.5** |
| 2 | **access-list 2 permit 192.168.4.0 0.0.0.127** |
| 3 | **access-list 3 permit 192.168.6.0 0.0.0.31** |
| 4 | **access-list 4 permit 172.30.96.0 0.0.3.255** |
| 5 | **access-list 5 permit 172.30.96.0 0.0.0.63** |
| 6 | **access-list 6 permit 10.1.192.0 0.0.0.31** |
| 7 | **access-list 7 permit 10.1.192.0 0.0.1.255** |
| 8 | **access-list 8 permit 10.1.192.0 0.0.63.255** |

Interestingly, IOS lets the CLI user type an **access-list** command in configuration mode, and IOS will potentially change the source parameter before placing the command into the running-config file. This process of just finding the range of addresses matched by the **access-list** command expects that the **access-list** command came from the router, so that any such changes were complete.

The change IOS can make with an **access-list** command is to convert to 0 any octet of an address for which the wildcard mask's octet is 255. For example, with a wildcard mask of 0.0.255.255, IOS ignores the last two octets. IOS expects the source parameter to end with two 0s. If not, IOS still accepts the **access-list** command, but IOS changes the last two octets of the address to 0s. Example 6-5 shows an example, where the configuration shows source 10.1.1.1, but wildcard mask 0.0.255.255.

**Example 6-5** *IOS Changing the Address Field in an* **access-list** *Command*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# access-list 21 permit 10.1.1.1 0.0.255.255
R2(config)# ^Z
R2#
R2# show ip access-lists
Standard IP access list 21
    10 permit 10.1.0.0, wildcard bits 0.0.255.255
```

The math to find the range of addresses relies on whether the command is fully correct or whether IOS has already set these address octets to 0, as shown in the example.

**NOTE** The most useful WC masks, in binary, do not interleave 0s and 1s. This book assumes the use of only these types of WC masks. However, Cisco IOS allows WC masks that interleave 0s and 1s, but using these WC masks breaks the simple method of calculating the range of addresses. As you progress through to CCIE studies, be ready to dig deeper to learn how to determine what an ACL matches.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 6-5 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 6-5** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |



## Review All the Key Topics

Key Topic

**Table 6-6** Key Topics for Chapter 6

| Key Topic Element | Description | Page Number |
|---|---|---|
| Paragraph | Summary of the general rule of the location and direction for an ACL | 117 |
| Figure 6-3 | Summary of four main categories of IPv4 ACLs in Cisco IOS | 119 |
| Paragraph | Summary of first-match logic used by all ACLs | 119 |
| Table 6-2 | Standard ACL configuration options to match a single IP address | 122 |
| List | Wildcard mask logic for decimals 0 and 255 | 122 |
| List | Wildcard mask logic to match a subnet | 124 |
| List | Steps to plan and implement a standard IP ACL | 125 |
| List | Tips for creating matching logic for the source address field in the **access-list** command | 130 |

## Key Terms You Should Know

access control entry, access control list, standard access list, wildcard mask

## Additional Practice for This Chapter's Processes

For additional practice with analyzing subnets, you may do the same set of practice problems using your choice of tools:

**Application:** Use the two ACL practice exercise applications listed on the companion website.

**PDF:** Alternatively, practice the same problems found in these apps using online Appendix E, "Practice for Chapter 6: Basic IPv4 Access Control Lists."

## Command References

Tables 6-7 and 6-8 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 6-7** Chapter 6 Configuration Command Reference

| Command | Description |
|---|---|
| **access-list** *access-list-number* {**deny** \| **permit**} *source* [*source-wildcard*] [**log**] | Global command for standard numbered access lists. Use a number between 1 and 99 or 1300 and 1999, inclusive. |
| **access-list** *access-list-number* **remark** *text* | Command that defines a remark to help you remember what the ACL is supposed to do. |
| **ip access-group** *number* {**in** \| **out**} | Interface subcommand to enable access lists. |

**Table 6-8** Chapter 6 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip interface** [*type number*] | Includes a reference to the access lists enabled on the interface |
| **show access-lists** [*access-list-number* \| *access-list-name*] | Shows details of configured access lists for all protocols |
| **show ip access-lists** [*access-list-number* \| *access-list-name*] | Shows IP access lists with identical output format as **show access-lists** for IP ACLs |
| **clear ip access-list counters** | Resets the statistical counter of ACE matches for all IPv4 ACLs |

## Answers to Earlier Practice Problems

Table 6-9 lists the answers to the problems listed earlier in Table 6-3.

**Table 6-9** Building One-Line Standard ACLs: Answers

| Problem | Answers |
|---|---|
| 1 | **access-list 1 permit 172.16.5.4** |
| 2 | **access-list 2 permit 192.168.6.0 0.0.0.255** |
| 3 | **access-list 3 permit 192.168.0.0 0.0.255.255** |
| 4 | **access-list 4 permit any** |
| 5 | **access-list 5 permit 10.1.200.0 0.0.7.255** |
| 6 | **access-list 6 permit 10.1.200.0 0.0.0.31** |
| 7 | **access-list 7 permit 172.20.112.0 0.0.1.255** |
| 8 | **access-list 8 permit 172.20.112.0 0.0.0.63** |
| 9 | **access-list 9 permit 192.168.9.64 0.0.0.15** |
| 10 | **access-list 10 permit 192.168.9.64 0.0.0.3** |

Table 6-10 lists the answers to the problems listed earlier in Table 6-4.

**Table 6-10** Address Ranges for Problems in Table 6-4: Answers

| Problem | Address Range |
|---|---|
| 1 | One address: 10.7.6.5 |
| 2 | 192.168.4.0–192.168.4.127 |
| 3 | 192.168.6.0–192.168.6.31 |
| 4 | 172.30.96.0–172.30.99.255 |
| 5 | 172.30.96.0–172.30.96.63 |
| 6 | 10.1.192.0–10.1.192.31 |
| 7 | 10.1.192.0–10.1.193.255 |
| 8 | 10.1.192.0–10.1.255.255 |



CHAPTER 7

# Named and Extended IP ACLs

**This chapter covers the following exam topics:**

**5.0 Security Fundamentals**

**5.6 Configure and verify access control lists**

The previous chapter introduced basic IP ACL concepts and configuration using numbered standard IP ACLs. While understanding numbered standard IP ACLs is useful and important, most networks need the greater matching abilities of extended ACLs. Extended ACLs allow you to match various header fields in each line of the ACL (each access control entry, or ACE), making them more powerful.

Additionally, most networks use named ACLs instead of numbered ones. Identifying ACLs using names makes it easy to add information in the name that reminds all as to the purpose of the ACL. Additionally, from the first introduction of named ACLs into IOS decades ago, named ACLs used ACL configuration mode, with **permit** and **deny** subcommands, making the configuration clearer and easier to change over time.

This chapter begins by discussing named ACLs, using only standard ACL examples for simplicity. That first section also shows how to edit named ACLs and why that capability improves on the older editing features for numbered ACLs. The second major section then examines extended ACLs in detail, using named extended ACLs for the most part.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 7-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Named ACLs and ACL Editing | 1, 2 |
| Extended IP Access Control Lists | 3–6 |

1. Which answer best compares named standard IP ACLs with numbered standard IP ACLs?
   a. Uses a name instead of a number, but with no other configuration changes
   b. Adds additional fields to match beyond standard numbered IP ACLs
   c. Enables you to configure matching parameters and permit/deny action in ACL mode
   d. Enables you to configure matching parameters and permit/deny action in interface mode

2. You just configured standard named ACL WAN_ACL with commands that did not use sequence numbers. Which command, if typed in ACL configuration mode for this ACL, would become the third ACE in the ACL?
   a. **15 permit 172.16.0.0 0.0.255.255**
   b. **permit 16 172.16.0.0 0.0.255.255**
   c. **21 permit 172.16.0.0 0.0.255.255**
   d. **permit 29 172.16.0.0 0.0.255.255**
3. Which answers list a field that cannot be matched by an extended IP ACL? (Choose two answers.)
   a. Protocol
   b. Source IP address
   c. Destination IP address
   d. TCP source port
   e. URL
   f. Filename for FTP transfers
4. Which **access-list** commands permit packets from host 10.1.1.1 to all web servers whose IP addresses begin with 172.16.5? (Choose two answers.)
   a. **access-list 101 permit tcp host 10.1.1.1 172.16.5.0 0.0.0.255 eq www**
   b. **access-list 1951 permit ip host 10.1.1.1 172.16.5.0 0.0.0.255 eq www**
   c. **access-list 2523 permit ip host 10.1.1.1 eq www 172.16.5.0 0.0.0.255**
   d. **access-list 2523 permit tcp host 10.1.1.1 eq www 172.16.5.0 0.0.0.255**
   e. **access-list 2523 permit tcp host 10.1.1.1 172.16.5.0 0.0.0.255 eq www**
5. Which of the following **access-list** commands permits packets going to any web client from all web servers whose IP addresses begin with 172.16.5?
   a. **access-list 101 permit tcp host 10.1.1.1 172.16.5.0 0.0.0.255 eq www**
   b. **access-list 1951 permit ip host 10.1.1.1 172.16.5.0 0.0.0.255 eq www**
   c. **access-list 2523 permit tcp any eq www 172.16.5.0 0.0.0.255**
   d. **access-list 2523 permit tcp 172.16.5.0 0.0.0.255 eq www 172.16.5.0 0.0.0.255**
   e. **access-list 2523 permit tcp 172.16.5.0 0.0.0.255 eq www any**
6. Consider the following output from a **show access-list** command. Which ACEs match packets from 10.22.33.99 to 10.33.22.22 destined to any SSH server?

```
ip access-list extended sample
  10 permit tcp 10.22.33.0 0.0.0.63 10.33.22.0 0.0.0.127 eq 22
  20 permit tcp 10.22.33.0 0.0.0.127 eq 24 10.33.22.0 0.0.0.63
  30 permit tcp 10.22.33.0 0.0.0.127 10.33.22.0 0.0.0.127 eq 22
  40 permit tcp 10.22.33.0 0.0.0.255 10.33.22.0 0.0.0.31 eq 24
```

   a. Line 10
   b. Line 20
   c. Line 30
   d. Line 40

# Foundation Topics

## Named ACLs and ACL Editing

From the earliest days of Cisco, IOS supported numbered ACLs. Over time, Cisco added support for **named access lists**. This section details how to configure and use named ACLs, along with one of the key features enabled by named ACLs: ACL editing.

### Named IP Access Lists

Named IP ACLs have many similarities with numbered IP ACLs. They function as packet filters but other IOS features use them for matching packets and taking other actions. They can also match the same fields: standard numbered ACLs can match the same fields as a standard named ACL, and extended numbered ACLs can match the same fields as an extended named ACL.

Of course, there are differences between named and numbered ACLs. Named ACLs originally had three significant differences compared to numbered ACLs:

Key Topic

- Using names instead of numbers to identify the ACL, making it easier to remember the reason for the ACL
- Using ACL mode subcommands, not global commands, to define the action and matching parameters
- Using ACL editing features that allow the CLI user to delete individual lines from the ACL and insert new lines

You can easily learn named ACL configuration by just converting numbered ACLs to use the equivalent named ACL configuration. Figure 7-1 shows just such a conversion, using a simple three-line standard ACL number 1. To create the three **permit** subcommands for the named ACL, you copy parts of the three numbered ACL commands, beginning with the **permit** keyword.

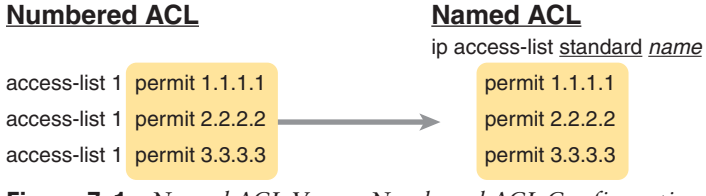



**Figure 7-1** *Named ACL Versus Numbered ACL Configuration*

The **ip access-list {standard | extended}** *name* global configuration command creates the ACL, defines its type, and defines its name. It also moves the user to ACL configuration mode, as shown in upcoming Example 7-1. Once in ACL configuration mode, you configure **permit**, **deny**, and **remark** commands that mirror the syntax of numbered **access-list** global commands. If you're configuring a standard named ACL, these commands match the syntax of standard numbered ACLs; if you're configuring extended named ACLs, they match the syntax of extended numbered ACLs.

---

Answers to the "Do I Know This Already?" quiz:

**1** C **2** C **3** E, F **4** A, E **5** E **6** C

Example 7-1 shows the configuration of a named standard ACL. Pay particular attention to the configuration mode prompts, which show standard named ACL configuration mode.

**Example 7-1** *Named Access List Configuration*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# ip access-list standard Hannah
R2(config-std-nacl)# remark A sample ACL, originally five lines
R2(config-std-nacl)# permit 10.1.1.2
R2(config-std-nacl)# deny 10.1.1.1
R2(config-std-nacl)# deny 10.1.3.0 0.0.0.255
R2(config-std-nacl)# deny 10.1.2.0 0.0.0.255
R2(config-std-nacl)# permit any
R2(config-std-nacl)# interface GigabitEthernet0/0/1
R2(config-if)# ip access-group Hannah out
R2(config-if)# ^Z
R2#
```

Reviewing the configuration, the **ip access-list standard Hannah** command creates the ACL, naming it Hannah, and placing the user in ACL configuration mode. This command also defines the ACL as a standard ACL. Next, five different **permit** and **deny** statements define the matching logic and resulting action when matched.

The names in named ACLs must follow these rules:

Key Topic

- They must begin with an alphabetic character.
- They cannot use spaces and quotation marks.
- The names are case sensitive.

Example 7-2 shows the named ACL that results from the configuration in Example 7-1. Both the **show running-config** and **show access-lists** commands list the ACEs from Example 7-1 in the same order. The output also shows the automatically added sequence numbers on the **permit** and **deny** subcommands. When ignoring them during configuration, IOS adds sequence numbers, using 10 for the first ACE and incrementing by 10 for each successive ACE.

**Example 7-2** *Named Access List Verification (IOS XE)*

```
R2# show running-config
Building configuration...

Current configuration:
! lines omitted for brevity
interface GigabitEthernet0/0/1
 ip access-group Hannah out
!
```

```
ip access-list standard Hannah
 10 permit 10.1.1.2
 20 deny   10.1.1.1
 30 deny   10.1.3.0 0.0.0.255
 40 deny   10.1.2.0 0.0.0.255
 50 permit any

R2# show access-list
Standard IP access list Hannah
    10 permit 10.1.1.2 (3 matches)
    20 deny   10.1.1.1 (5 matches)
    30 deny   10.1.3.0, wildcard bits 0.0.0.255 (10 matches)
    40 deny   10.1.2.0, wildcard bits 0.0.0.255 (15 matches)
    50 permit any (1256 matches)
```

**NOTE** The **show access-lists** and **show ip access-lists** commands list sequence numbers on both IOS and IOS XE. The **show running-config** and **show startup-config** commands reveal the sequence numbers only in IOS XE but not in IOS. The output in Example 7-2 comes from a router running IOS XE.

For both exam preparation and real networking jobs, prepare so that you understand both numbered and named ACL syntax. Given a choice, most network engineers choose to use named ACLs. However, you may still find numbered ACLs used in production routers in enterprises, and the CCNA exam may ask about both styles. As shown in Figure 7-1, when you see a numbered ACL, you can mentally create the equivalent named ACL's subcommands by removing the **access-list** *number* prefix in the numbered global commands.

## Editing ACLs

For CCNA exam preparation, you need to know some of the intricacies of ACL editing with both the old global commands and newer ACL mode commands. This section starts with named ACLs using ACL mode. Following that, the text describes the differences compared to numbered ACLs and some minor differences when using IOS versus IOS XE.

### Editing Named ACLs

When Cisco created ACL mode and named ACLs, they created the ability to add and remove single ACEs from an ACL easily. IOS supports deleting individual ACEs, adding new ACEs to the end of the ACL, and even adding ACEs anywhere in the ACL using **ACL sequence numbers**.

To delete one ACE, you first get into ACL configuration mode using the **ip access-list** {**standard** | **extended**} *name* global command. For any existing ACE, you have two options to delete specific subcommands:

Key Topic

- Repeat the entire **permit**, **deny**, or **remark** ACL subcommand without a line number but preceded with the **no** command.
- Use the **no** *sequence-number* command, without the rest of the command, to delete the ACE that uses the listed sequence number.

Example 7-3 shows an example based on the five-line ACL Hannah in Example 7-2. One deletes the fourth ACE in the ACL with the full command after the **no** command, and the other deletes the second ACE with the **no 20** subcommand.

**Example 7-3** *Demonstrating Both Methods to Remove an ACE from a Named ACL*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# ip access-list extended Hannah
R2(config-std-nacl)# no deny 10.1.2.0 0.0.0.255
R2(config-std-nacl)# no 20
R2(config-std-nacl)# ^Z
R2# show access-lists

Standard IP access list Hannah
    10 permit 10.1.1.2
    30 deny   10.1.3.0, wildcard bits 0.0.0.255
    50 permit any
```

The example also shows some important detail about the sequence numbers. The remaining ACEs have the same relative order as before—even though the sequence numbers of those lines did not change. The absolute values of the sequence numbers do not matter—only their relative values matter, as they confirm the order of the ACEs in the ACL. (The section, "Resequencing ACL Sequence Numbers" in Chapter 8, "Applied IP ACLs," discusses how to renumber ACL sequence numbers to clean up the numbering.)

To add one ACE to an ACL, again from ACL configuration mode, use these options:

Key Topic

- To insert the ACE between lines, configure the **permit** or **deny** command preceded by a sequence number. The number dictates the new ACE's location in the ACL.
- To insert the ACE at the end of the ACL, configure the **permit** or **deny** command without a sequence number, and IOS will automatically add a sequence number to place the ACE at the end of the ACL.

For example, imagine you wanted to add the two ACEs you deleted to the same ACL (Hannah). Example 7-4 shows the classic mistake of ignoring the sequence numbers, which results in the two new ACEs to the end of the ACL. As a reminder, ACL Hannah, at the end of Example 7-3, has three ACEs numbered 10, 30, and 50.

**Example 7-4** *Demonstrating Adding ACEs to the End of the ACL*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# ip access-list extended Hannah
R2(config-std-nacl)# deny 10.1.2.0 0.0.0.255
R2(config-std-nacl)# deny 10.1.1.1
R2(config-std-nacl)# ^Z
R2# show access-lists

Standard IP access list Hannah
    10 permit 10.1.1.2
    30 deny   10.1.3.0, wildcard bits 0.0.0.255
    50 permit any
    60 deny 10.1.2.0 0.0.0.255
    70 deny 10.1.1.1
```

Forgetting about the sequence numbers—and adding ACEs to the end of the ACL—is seldom what you want to accomplish. In this case, adding those lines has no benefit. The two ACEs added in Example 7-4 reside behind the explicit **permit any** command at line 50, so the ACL would never use the two lines added in the example.

To reinsert the two ACEs in their original locations, as the second and fourth ACEs, use sequence numbers to begin the commands. Example 7-5 does that, again starting with the state at the end of Example 7-3, with ACEs 10, 30, and 50 in the ACL. Example 7-5 shows the two **deny** commands with sequence numbers 20 and 40.

**Example 7-5** *Demonstrating Inserting ACEs in Different ACL Positions*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# ip access-list extended Hannah
R2(config-std-nacl)# 40 deny 10.1.2.0 0.0.0.255
R2(config-std-nacl)# 20 deny 10.1.1.1
R2(config-std-nacl)# ^Z
R2# show access-lists

Standard IP access list Hannah
    10 permit 10.1.1.2
    20 deny   10.1.1.1
    30 deny   10.1.3.0, wildcard bits 0.0.0.255
    40 deny   10.1.2.0, wildcard bits 0.0.0.255
    50 permit any
```

**NOTE** While Example 7-5 used sequence numbers 20 and 40 for the configuration commands, any sequence number between the two existing ACEs would have worked. For instance, the **20 deny 10.1.1.1** ACL subcommand could have been either **11 deny 10.1.1.1** or **29 deny 10.1.1.1**, as the neighboring ACEs currently use sequence numbers 10 and 30.

## Editing Numbered ACLs

In the history of IOS and its features, numbered ACLs came early in Cisco's history. They added named ACLs around the year 2000, so both styles have been around for a long time. When creating named ACLs, Cisco set about to improve some shortcomings with numbered ACLs—one of which was poor ACL editing features for numbered ACLs. The solution to that problem as it exists today is to allow the editing of numbered ACLs as if they were named ACLs by using ACL mode configuration commands.

To discuss numbered ACLs, Example 7-6 provides a sample standard ACL 8 equivalent to ACL Hannah in the previous examples. The configuration uses **access-list** global commands (note the command prompts). As usual with numbered ACLs, the commands do not identify it as a standard ACL; instead, the number (8) implies it is a standard ACL per the number ranges for standard ACLs (1–99, 1300–1999).

**Example 7-6** *Numbered Access List Configuration with Global Commands*

```
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# access-list 8 permit 10.1.1.2
R2(config)# access-list 8 deny 10.1.1.1
R2(config)# access-list 8 deny 10.1.3.0 0.0.0.255
R2(config)# access-list 8  deny 10.1.2.0 0.0.0.255
R2(config)# access-list 8 permit any
R2(config)# interface GigabitEthernet0/0/1
R2(config-if)# ip access-group 8 out
R2(config-if)# ^Z
R2#
```

Originally, most edits to a numbered ACL required deleting the ACL and reconfiguring the entire ACL. Those rules still apply today if you use the **access-list** global command:

- You cannot delete an individual ACE; any command that begins **no access-list** *acl-number* deletes the entire ACL.
- You cannot add an ACE to an existing ACL other than to the end of the ACL.

On the first point, the **no access-list** *number* global command deletes the entire ACL. But if you type that command with additional parameters, IOS ignores the additional parameters and deletes the whole ACL. For instance, if you issued the **no access-list 8 deny 10.1.1.1** global command for the ACL in Example 7-6, to attempt to delete the second line in ACL 8, IOS would instead treat the command as the **no access-list 8** command and delete ACL 8.

As for adding ACEs, the global **access-list** command has no option for sequence numbers. You could add lines to the end of the ACL by typing additional **access-list** global commands. To insert a line anywhere other than the end of the ACL, you delete the ACL and reconfigure it.

**NOTE** When you're editing any ACL, Cisco recommends disabling it from any interfaces before editing and re-enabling it after it is finished.

Today, you can overcome the old numbered ACL editing difficulties by instead editing them using the ACL mode. IOS allows you to configure and edit numbered ACLs using the ACL mode commands. As you have read over the last few pages, you can easily remove and add single ACEs using ACL mode. Example 7-7 shows an example using the following sequence:

**Step 1.** The **show access-lists** command reveals the line numbers that IOS automatically added to each line in the numbered ACL as configured in Example 7-6.

**Step 2.** Two **no** commands in ACL mode delete the second and fourth ACEs.

**Step 3.** The **show access-lists** command confirms the single-ACE deletions.

**Step 4.** The excerpt from the **show running-config** command reveals that even though the configuration process used ACL mode commands, IOS still stores the numbered ACL as **access-list** global commands.

**Example 7-7** *Removing ACEs from a Numbered ACL (IOS)*

```
R2# show access-lists 8
Standard IP access list 8
    10 permit 10.1.1.2
    20 deny   10.1.1.1
    30 deny   10.1.3.0, wildcard bits 0.0.0.255
    40 deny   10.1.2.0, wildcard bits 0.0.0.255
    50 permit any
R2# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R2(config)# ip access-list standard 8
R2(config-std-nacl)# no deny 10.1.2.0 0.0.0.255
R2(config-std-nacl)# no 20
R2(config-std-nacl)# ^Z
R2# show access-lists 8
Standard IP access list 8
    10 permit 10.1.1.2
    30 deny   10.1.3.0, wildcard bits 0.0.0.255
    50 permit any

R2# show running-config
! Lines omitted for brevity
access-list 8 permit 10.1.1.2
access-list 8 deny   10.1.3.0 0.0.0.255
access-list 8 permit any
```

## Extended IP Access Control Lists

Extended IP ACLs work just like standard IP ACLs, except they can match multiple header fields in a single ACE. Just like standard IP ACLs, extended ACLs must be enabled on interfaces for a direction (in or out). IOS searches the list sequentially, using first-match logic, taking the permit or deny action per the first-matched ACE. The differences reduce to the variety and complexity of the matching logic.

One extended ACE (ACL statement) can examine multiple parts of the packet headers so that all matching parameters must match the packet for IOS to consider the ACE to match the packet. Each ACE must list matching parameters for a protocol, source address, and destination address, with additional optional fields. The powerful matching logic makes **extended access lists** more valuable and complex than standard IP ACLs. This major section of the chapter examines extended ACLs in some detail.

IOS and IOS XE support both numbered and named extended ACLs. The comparisons between numbered extended IP ACLs and named extended IP ACLs are the same as with standard ACLs.

## Matching the Protocol, Source IP, and Destination IP

Like standard numbered IP ACLs, extended numbered IP ACLs also use the **access-list** global command. IOS identifies an ACL as extended by the number range, with extended ACL numbers being 100–199 or 2000–2699. Both support **permit** and **deny** actions. The big difference comes in the matching parameters, with the rest of this chapter and some of the next devoted to discussing extended ACL matching options.

As for extended named IP ACLs, the **permit** and **deny** subcommands follow the same syntax as the extended numbered IP ACLs.

The first matching parameter for extended ACLs refers to the protocol. The protocol keyword can be **ip**, referring to all IP packets. It can also refer to a subset of IP packets based on the protocol of the header that follows the IP header, as defined by the IP header's protocol type field. Figure 7-2 shows the location of the IP Protocol field, the concept of it pointing to the type of header that follows, along with some details of the IP header for reference.

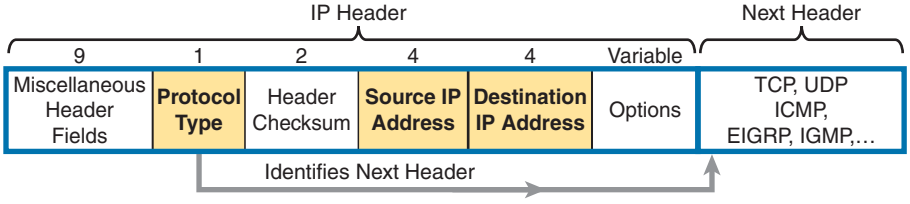



**Figure 7-2** *IP Header, with Focus on Required Fields in Extended IP ACLs*

To match the protocol type, you simply use a keyword, such as **tcp**, **udp**, or **icmp**, matching IP packets that also happen to have a TCP, UDP, or ICMP header, respectively, following the IP header. Or you can use the keyword **ip**, which means "all IPv4 packets."

You also must configure parameters for the source and destination IP address fields that follow; these fields use the same syntax and options for matching the IP addresses as discussed in Chapter 6, "Basic IPv4 Access Control Lists." Figure 7-3 shows the syntax.

Extended ACLs support two syntax options to match a single address, while standard ACLs support three. (The section "Matching the Exact IP Address" in the preceding chapter discussed the three options.) Table 7-2 summarizes the options supported. Of note, no matter the syntax you choose to use when configuring, IOS stores the command with a preferred syntax, as noted in the table.

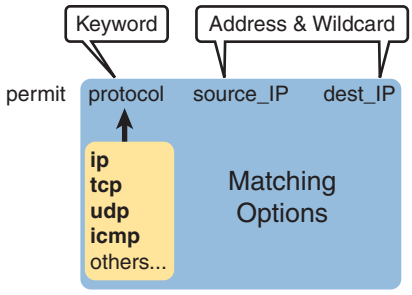



**Figure 7-3** *Extended ACL Syntax, with Required Fields*

**Table 7-2** Summary of Syntax Supported to Match One IP Address

| Syntax Options | *address* 0.0.0.0 | host *address* | *address* |
|---|---|---|---|
| Standard ACLs | Yes | Yes | Yes* |
| Extended ACLs | Yes | Yes* | No |

* IOS stores the command with this syntax no matter the configuration style.

Table 7-3 lists several sample **access-list** commands that use only the required matching parameters. As an exercise, hide the right column and predict the logic of the command in the left column. Then review the answer in the right column. Or just review the explanations on the right column to get an idea for the logic in some sample commands. Note that the table shows ACL mode subcommand for named ACLs, but the same syntax exists in the global commands used by numbered ACLs.

**Table 7-3** Extended **access-list** Commands and Logic Explanations

| access-list Statement | What It Matches |
|---|---|
| **deny tcp any any** | Any IP packet that has a TCP header |
| **deny udp any any** | Any IP packet that has a UDP header |
| **deny icmp any any** | Any IP packet that has an ICMP header |
| **deny ip host 1.1.1.1 host 2.2.2.2** | All IP packets from host 1.1.1.1 going to host 2.2.2.2, for all IP packets |
| **deny udp 1.1.1.0 0.0.0.255 any** | All IP packets that also have a UDP header, from subnet 1.1.1.0/24, and going to any destination |

The last entries in Table 7-3 help make an important point about how IOS processes extended ACLs:

In an extended ACL **access-list** command, all the matching parameters must match the packet for the packet to match the command.

For example, in that last example from Table 7-3, the command checks for UDP, a source IP address from subnet 1.1.1.0/24, and any destination IP address. If a router processed a packet with source IP address 1.1.1.1, it would match the source IP address check. However, if the packet also had a TCP header following the IP header, the router would fail to match the packet with that ACE. All parameters in an ACE must match the packet.

## Matching TCP and UDP Port Numbers

Extended ACLs can also examine parts of the TCP and UDP headers, particularly the source and destination port number fields. The port numbers identify the application that sends or receives the data.

The most useful ports to check are the well-known ports used by servers. For example, web servers use well-known port 80 by default. Figure 7-4 shows the location of the port numbers in the TCP header, following the IP header.

Key Topic

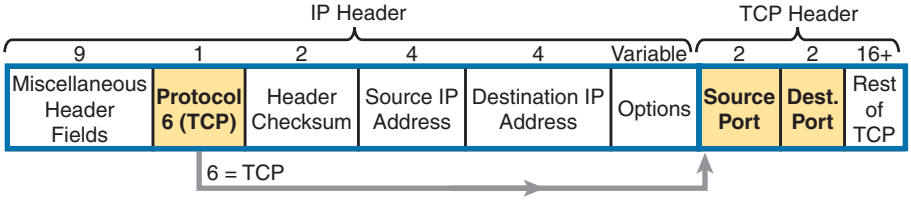



**Figure 7-4** *IP Header, Followed by a TCP Header and Port Number Fields*

When an extended ACL command includes either the **tcp** or **udp** keyword, that command can optionally reference the source and/or destination port. To make these comparisons, the syntax uses keywords for equal, not equal, less than, greater than, and for a range of port numbers. In addition, the command can use either the literal decimal port numbers or more convenient keywords for some well-known application ports. Figure 7-5 shows the positions of the source and destination port fields in the **access-list** command and these port number keywords.

Key Topic

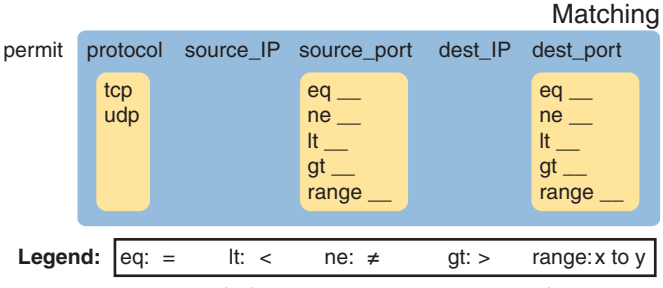



**Figure 7-5** *Extended ACL Syntax with TCP and UDP Port Numbers Enabled*

For example, consider the simple network shown in Figure 7-6. The FTP server sits on the right, with the client on the left. The figure shows the syntax of an ACL that matches the following:

- Packets that include a TCP header
- Packets sent from the client subnet
- Packets sent to the server subnet
- Packets with TCP destination port 21 (FTP server control port)

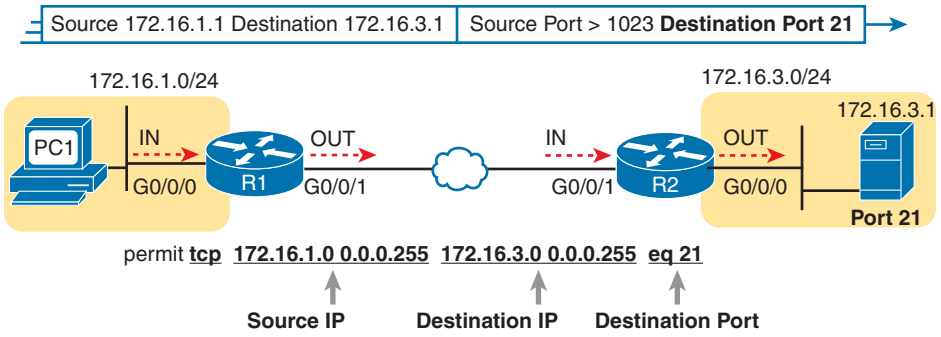



**Figure 7-6** *Filtering Packets Based on Destination Port*

To fully appreciate the matching of the destination port with the **eq 21** parameters, consider packets moving from left to right, from PC1 to the server. Assuming the server uses well-known port 21 (FTP control port), the packet's TCP header has a destination port value of 21. The ACL syntax includes the **eq 21** parameters after the destination IP address. The position after the destination address parameters is important: that position identifies the fact that the **eq 21** parameters should be compared to the packet's destination port. As a result, the ACL statement shown in Figure 7-6 would match this packet and the destination port of 21 if used in any of the four locations implied by the four dashed arrowed lines in the figure.

Conversely, Figure 7-7 shows the reverse flow, with a packet sent by the server back toward PC1. In this case, the packet's TCP header has a source port of 21, so the ACL must check the source port value of 21, and the ACL must be located on different interfaces. In this case, the **eq 21** parameters follow the source address field but come before the destination address field.

Key Topic

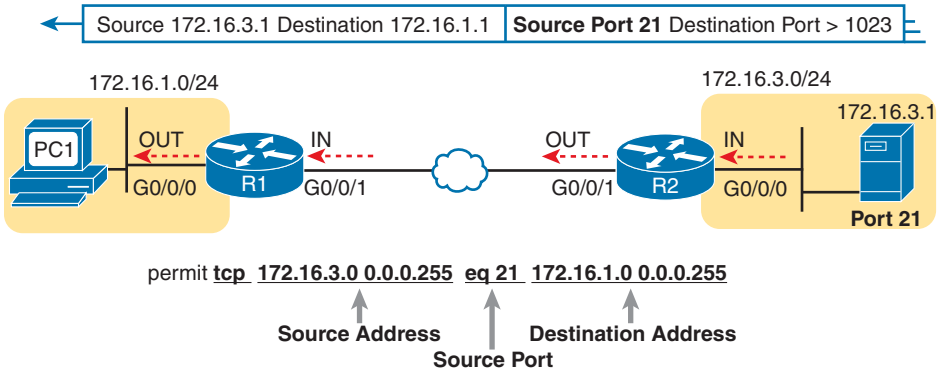



**Figure 7-7** *Filtering Packets Based on Source Port*

When examining ACLs that match port numbers, first consider the location and direction to apply the ACL. That direction determines whether the packet is being sent to the server or from the server. You can then decide whether you need to check the source or destination port in the packet. For reference, Tables 7-4 and 7-5 list many of the popular port numbers and their transport layer protocols and applications.

**NOTE** You will likely want to memorize the ACL command keywords for the port numbers as well. Whether you use the keyword or a port number when configuring, IOS stores the keyword rather than the number when IOS has a defined keyword for that port.

Key Topic

**Table 7-4** IT Services Their Well-Known Port Numbers

| Port Number(s) | Protocol | Application | access-list and permit \| deny Command Keywords |
|---|---|---|---|
| 20 | TCP | FTP data | **ftp-data** |
| 21 | TCP | FTP control | **ftp** |
| 22 | TCP | SSH | — |
| 23 | TCP | Telnet | **telnet** |
| 49 | UDP, TCP | TACACS+ | — |
| 53 | UDP, TCP | DNS | **domain** |
| 67 | UDP | DHCP Server | **bootps** |
| 68 | UDP | DHCP Client | **bootpc** |
| 69 | UDP | TFTP | **tftp** |
| 161 | UDP | SNMP | **snmp** |
| 514 | UDP | Syslog | — |
| 1645, 1646 | UDP | Radius (original) | — |
| 1812, 1813 | UDP | Radius (current) | — |

Key Topic

**Table 7-5** User Applications and Their Well-Known Port Numbers

| Port Number(s) | Protocol | Application | access-list and permit \| deny Command Keywords |
|---|---|---|---|
| 25 | TCP | SMTP | **smtp** |
| 80 | TCP | HTTP (WWW) | **www** |
| 443 | TCP | HTTPS (w/ TLS) | — |
| 110 | TCP | POP3 (no TLS) | **pop3** |
| 995 | TCP | POP3 w/ TLS | — |
| 143 | TCP | IMAP (no TLS) | — |
| 993 | TCP | IMAP w/ TLS | — |
| 16,384–32,767 | UDP | RTP (voice, video) | — |

IOS has some surprising conventions when configuring well-known port numbers. First, you can always configure the port number. However, IOS supports a small set of well-known port keywords, as shown in the far-right column of Table 7-4. In the command you enter, you can type either the number or the keyword; for instance, you could use **80** or **www** to refer to well-known port 80.

However, be aware that if a text keyword exists, IOS stores the ACL command using that keyword. For example, a command typed with **permit tcp any any eq 80** becomes **permit tcp any any eq www** in the running-config.

Table 7-6 lists several sample **access-list** commands that match based on port numbers. Again, use the table as an exercise by covering the right column and analyzing the contents of the left column. Then check the right side of the table to see if you agree. As with Tables 7-4 and 7-5, the table uses ACL mode subcommands used by named ACLs, but the same syntax exists within extended ACL global commands.

**Table 7-6** Extended **access-list** Command Examples and Logic Explanations

| access-list Statement | What It Matches |
|---|---|
| **deny tcp any gt 49151 host 10.1.1.1 eq 23** | Packets with a TCP header, any source IP address, with a source port greater than (**gt**) 49151, a destination IP address of exactly 10.1.1.1, and a destination port equal to (**eq**) 23. |
| **deny tcp any host 10.1.1.1 eq 23** | The same as the preceding example, but any source port matches because the command omits the source port parameters. |
| **deny tcp any host 10.1.1.1 eq telnet** | The same as the preceding example. The **telnet** keyword is used instead of port 23. |
| **deny udp 1.0.0.0 0.255.255.255 lt 1023 any** | A packet with a source in network 1.0.0.0/8, using UDP with a source port less than (**lt**) 1023, with any destination IP address. |

## Extended IP ACL Configuration

Extended ACLs support far too many options to allow a single generic representation of the command syntax. However, the commands in Table 7-7 summarize the syntax options covered in this book. The first two rows show numbered ACL commands, with the final two rows showing the same syntax in the ACL mode **permit** and **deny** commands.

**Table 7-7** Extended IP Access List Configuration Commands

| Command | Configuration Mode and Description |
|---|---|
| **access-list** *access-list-number* {**deny** \| **permit**} *protocol source source-wildcard destination destination-wildcard* [**log**] | Global command for extended numbered access lists. Use a number between 100 and 199 or 2000 and 2699, inclusive. |
| **access-list** *access-list-number* {**deny** \| **permit**} {**tcp** \| **udp**} *source source-wildcard* [*operator* [*port*]] *destination destination-wildcard* [*operator* [*port*]] [**log**] | A version of the **access-list** command with parameters specific to TCP and UDP. |
| **ip access-group** *number\|name* **in\|out** | Interface subcommand to enable an IP ACL on an interface for a direction. Used for numbered and named ACLs. |
| **ip access-list extended** *name* | A global command to create an extended ACL and move the user into ACL configuration mode. |

| Command | Configuration Mode and Description |
|---|---|
| {**deny** \| **permit**} *protocol source source-wildcard destination destination-wildcard* [**log**] | ACL subcommand with the syntax to match the required parameters: protocol, plus source and destination IP address. |
| {**deny** \| **permit**} {**tcp** \| **udp**} *source source-wildcard* [*operator* [*port*]] *destination destination-wildcard* [*operator* [*port*]] [**log**] | ACL subcommand with parameters to match parameters specific to TCP and UDP. |

The configuration process for extended ACLs mostly matches the same process used for standard ACLs. You must choose the location and direction to enable the ACL so that you can characterize whether specific addresses and ports will be either the source or destination. Configure the ACL using **access-list** commands, and when complete, then enable the ACL using the same **ip access-group** command used with standard ACLs. All these steps mirror what you do with standard ACLs; however, when configuring, keep the following differences in mind:

Key Topic

- Place extended ACLs as close as possible to the source of the packets that the ACL needs to filter. Filtering close to the source of the packets saves some bandwidth.
- Remember that all fields in one ACE must match a packet for the router to consider the packet to match that ACE.
- For numbered ACLs, use numbers 100–199 and 2000–2699 on the **access-list** commands; no one number is inherently better than another.

### Extended IP ACL Example 1: Packets to Web Servers

The two examples to close the first major section of the chapter focus on matching packets to web servers (in the first example) and from web servers (in the second example.) Both use the same topology and web servers, with a different direction in each case. Figure 7-8 shows the topology, IP subnets, and specific IP addresses as needed.

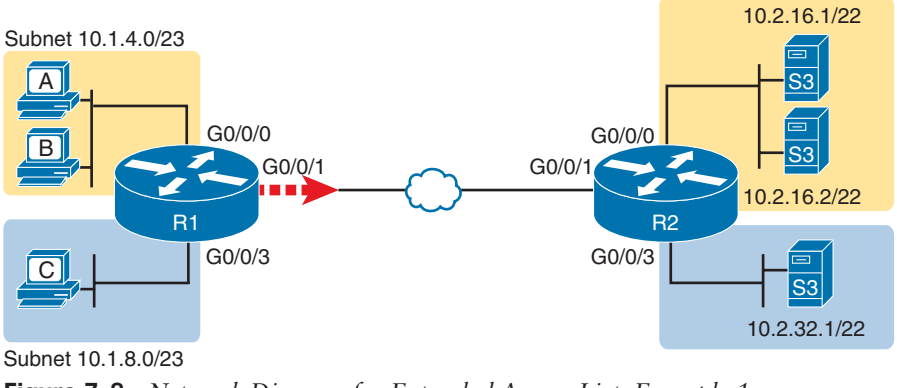



**Figure 7-8** *Network Diagram for Extended Access List: Example 1*

Treat both examples as an exercise to understand the mechanics of extended ACLs. For that purpose, consider how to create an ACL for router R1, where users sit at a branch office. The

ACL will be applied outbound on router R1's WAN link, that is, toward the servers. The matching requirements are

1. Permit traffic *sent by* the upper user subnet on the left (10.1.4.0/23) and *going to* the web servers in the subnet in the upper right of the drawing (at an internal data center.)
2. Permit traffic *sent by* the upper user subnet on the left (10.1.4.0/23) and *going to* the single web server 10.2.32.1 in the lower right of the figure.
3. In both cases, allow both HTTP and HTTPS protocols.
4. Deny all other traffic.

Example 7-8 shows an extended named ACL that does just that. It has two pairs of **permit** statements. In each pair, the first matches the port number for HTTP (80) while the second matches the port number for HTTPS (443). The first pair matches packets to the upper-right subnet 10.2.16.0/22, while the second pair matches packets to host 10.2.32.1.

**Example 7-8** *R1's Extended Access List: Example 1*

```
R1# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)# ip access-list extended branch_WAN
R1(config-ext-nacl)# remark Example ACL to match HTTP/S
R1(config-ext-nacl)# permit tcp 10.1.4.0 0.0.1.255 10.2.16.0 0.0.3.255 eq 80
R1(config-ext-nacl)# permit tcp 10.1.4.0 0.0.1.255 10.2.16.0 0.0.3.255 eq 443
R1(config-ext-nacl)# permit tcp 10.1.4.0 0.0.1.255 host 10.2.32.1 eq 80
R1(config-ext-nacl)# permit tcp 10.1.4.0 0.0.1.255 host 10.2.32.1 eq 443
R1(config-ext-nacl)# interface gigabitethernet0/0/1
R1(config-if)# ip access-group branch_WAN out
R1(config-if)# ^Z
R1#
```

To emphasize some key points from earlier in this chapter, note that all four permit statements use **tcp** as the protocol type. HTTP and HTTPS both use TCP, and to match port numbers in the ACL, you must specify **tcp** or **udp** in the command. Also, in each command, take time to identify the parameters that define the address(es) for the source and destination address matching. Then note that the port number matching at the end of the **permit** commands all follow the destination address matching parameters—meaning that the statements all attempt to match the destination port number.

To complete the review, note that the configuration enables the ACL outbound on R1's G0/0/1 interface. Also, the ACL ends with an implied **deny any**, in effect, an implied final command of **deny ip any any**. Given the explicit lines in the ACL, the ACL permits only HTTP/HTTPS traffic, with limited source and destination address ranges, and denies all other traffic. (This ACL would be far too restrictive in a production network, but it provides enough detail to learn the syntax and logic.)

Example 7-9 confirms that the ACL has been enabled, with statistics showing some usage of the first two lines in the ACL.

**Example 7-9** *Verifying the Extended ACL*

```
R1# show ip interface gigabitethernet0/0/1
GigabitEthernet0/0/1 is up, line protocol is up
  Internet address is 10.1.12.1/24
  Broadcast address is 255.255.255.255
  Address determined by non-volatile memory
  MTU is 1500 bytes
  Helper address is not set
  Directed broadcast forwarding is disabled
  Multicast reserved groups joined: 224.0.0.5 224.0.0.10 224.0.0.6
  Outgoing access list is branch_WAN
  Inbound access list is not set
! Lines omitted for brevity

R1# show access-list
Extended IP access list branch_WAN
    10 permit tcp 10.1.4.0 0.0.1.255 10.2.16.0 0.0.3.255 eq www (18 matches)
    20 permit tcp 10.1.4.0 0.0.1.255 10.2.16.0 0.0.3.255 eq 443 (416 matches)
    30 permit tcp 10.1.4.0 0.0.1.255 host 10.2.32.1 eq www
    40 permit tcp 10.1.4.0 0.0.1.255 host 10.2.32.1 eq 443
```

This example also demonstrates IOS's surprising habit of changing some numeric port numbers into text keywords, as discussed earlier in the text just before Tables 7-4 and 7-5. Those tables list some of the more common TCP and UDP port numbers and the ACL keywords IOS supports. IOS uses keyword **www** for port 80, so it replaced the configured numeric 80 with the **www** keyword. However, IOS left port 443 as is because it has no ACL keyword for that port number.



### Extended IP ACL Example 2: Packets from Web Servers

One of the most common mistakes when learning about extended ACLs relates to matching TCP and UDP ports. The syntax and the underlying concepts can be confusing. The earlier section "Matching TCP and UDP Port Numbers" explained the details, with this next example giving you a chance to think through the problem.

As an exercise, plan to create an ACL on router R2 instead of router R1 as in the previous example. You will achieve the same goals, but place the ACL on router R2, outgoing on its WAN link, as shown in Figure 7-9. Most importantly, note that this change means that the ACL matches the opposite direction versus the previous example.

Think about this example for yourself first, and even try creating the ACL in a text editor before reading further.

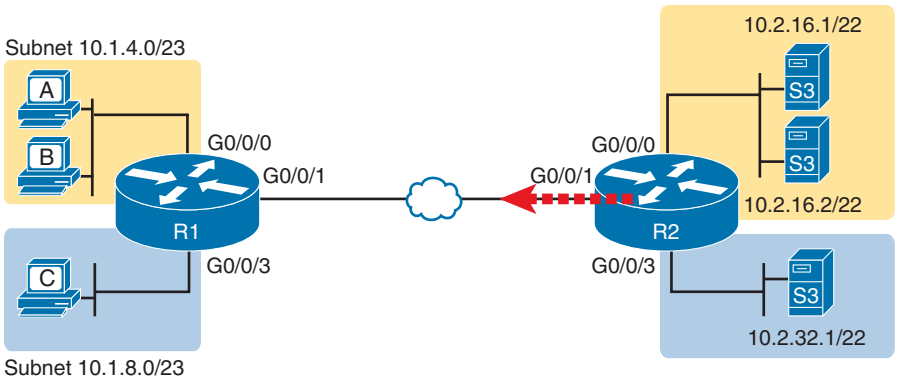



**Figure 7-9** *Network Diagram for Extended Access List: Example 2*

The list that follows details a revised version of the ACL requirements listed just before Example 7-8, for the previous example. The list reverses the logic so it matches packets flowing in the opposite direction: right to left in the figure.

1. Reversing original requirement 1, permit traffic *going to* the upper user subnet on the left (10.1.4.0/23) and *sent by* the web servers in the subnet in the upper right of the drawing (at an internal data center).
2. Reversing original requirement 2, permit traffic *going to* the upper user subnet on the left (10.1.4.0/23) and *sent by* the single web server 10.2.32.1 in the lower right of the figure.
3. As before, allow both HTTP and HTTPS protocols.
4. As before, deny all other traffic.

Example 7-10 shows the resulting ACL on router R2. Note the highlighted matching fields for the port numbers that reside just after the source address parameters.

**Example 7-10** *Extended ACL on Router R2 to Match Packets Sent by the Web Servers*

```
R2# show running-config | section access-list
ip access-list extended DC_WAN
 10 permit tcp 10.2.16.0 0.0.3.255 eq www 10.1.4.0 0.0.1.255
 20 permit tcp 10.2.16.0 0.0.3.255 eq 443 10.1.4.0 0.0.1.255
 30 permit tcp host 10.2.32.1 eq www 10.1.4.0 0.0.1.255
 40 permit tcp host 10.2.32.1 eq 443 10.1.4.0 0.0.1.255
```

## Adjusting ACLs for HTTP/3

In Chapter 5, "Introduction to TCP/IP Transport and Applications," the section titled "HTTP 3.0" details how a relatively new version of HTTP, HTTP/3 (or HTTP 3.0), moves away from using TCP. Instead, it uses QUIC as the transport protocol, which itself uses a UDP header as the next header after the IP header. IANA reserves UDP port 443 as the well-known port for web servers using HTTP/3. So, not only can you match HTTP/3 traffic, but also consider updating existing ACLs to match HTTP/3 traffic as needed.

To show a specific example, reconsider the previous scenario, with ACL DC_WAN as shown in Example 7-10, but add this requirement:

5. Ensure the ACL matches not only TCP-based HTTP versions but also HTTP/3, which uses UDP and QUIC.

As a reminder, the previous ACL matches the source well-known port. The revised version in Example 7-11 highlights the new lines that match UDP messages with well-known port 443. Note that you do not need to match UDP 80 as a well-known port because HTTP/3 always uses TLS (HTTPS) and defines well-known UDP port 443.

**Example 7-11** *Revised ACL from Example 7-10 Now Matches HTTP/3*

```
R2# show running-config | section access-list
ip access-list extended DC_WAN
 10 permit tcp 10.2.16.0 0.0.3.255 eq www 10.1.4.0 0.0.1.255
 20 permit tcp 10.2.16.0 0.0.3.255 eq 443 10.1.4.0 0.0.1.255
 25 permit udp 10.2.16.0 0.0.3.255 eq 443 10.1.4.0 0.0.1.255
 30 permit tcp host 10.2.32.1 eq www 10.1.4.0 0.0.1.255
 40 permit tcp host 10.2.32.1 eq 443 10.1.4.0 0.0.1.255
 45 permit udp host 10.2.32.1 eq 443 10.1.4.0 0.0.1.255
```

## Practice Building access-list Commands

Table 7-8 supplies a practice exercise to help you get comfortable with the syntax of extended ACL commands, particularly choosing the correct matching logic. Your job: create a one-line extended ACL that matches the packets and uses a permit action. You can use the syntax for a global **access-list** command or an ACL mode **permit** command. The answers are shown in the section "Answers to Earlier Practice Problems" at the end of this chapter. Note that if the criteria mention a particular application protocol, for example, "web client," that means to match for that application protocol specifically.



**Table 7-8** Building One-Line Extended ACLs: Practice

| Problem | Criteria |
|---|---|
| 1 | From web client 10.1.1.1, sent to a web server in subnet 10.1.2.0/24. |
| 2 | From Telnet client 172.16.4.3/25, sent to a Telnet server in subnet 172.16.3.0/25. Match all hosts in the client's subnet as well. |
| 3 | ICMP messages from the subnet in which 192.168.7.200/26 resides to all hosts in the subnet where 192.168.7.14/29 resides. |
| 4 | From web server 10.2.3.4/23's subnet to clients in the same subnet as host 10.4.5.6/22. |
| 5 | From Telnet server 172.20.1.0/24's subnet, sent to any host in the same subnet as host 172.20.44.1/23. |
| 6 | From web client 192.168.99.99/28, sent to a web server in subnet 192.168.176.0/28. Match all hosts in the client's subnet as well. |
| 7 | ICMP messages from the subnet in which 10.55.66.77/25 resides to all hosts in the subnet where 10.66.55.44/26 resides. |
| 8 | Any and every IPv4 packet. |

## ACL Implementation Considerations

ACLs can be a great tool to enhance the security of a network, but engineers should think about some broader issues before simply configuring an ACL to fix a problem. To help, Cisco makes the following general recommendations:



- Place extended ACLs as close as possible to the packet's source. This strategy allows ACLs to discard the packets early.
- Place standard ACLs as close as possible to the packet's destination. This strategy avoids the mistake with standard ACLs (which match the source IPv4 address only) of unintentionally discarding packets.
- Place more specific statements early in the ACL and less specific statements later in the ACL.
- Disable an ACL from its interface (using the **no ip access-group** interface subcommand) before editing the ACL.
- Ensure the **ip access-group** interface refers to the ACL number or name you intended; take care to spell named ACLs correctly, using the correct case.

The first point deals with the concept of where to locate your ACLs. If you intend to filter a packet, filtering closer to the packet's source means that the packet takes up less bandwidth in the network, which seems to be more efficient—and it is. Therefore, Cisco suggests locating extended ACLs as close to the source as possible.

However, the second point seems to contradict the first point, at least for standard ACLs, to locate them close to the destination. Why? Because standard ACLs look only at the source IP address, they tend to filter more than you want filtered when placed close to the source. For instance, in Figures 7-8 and 7-9, imagine an inbound ACL on router R1's LAN interface (G0/0/0) that discards packets from host A (10.1.4.1). That may be exactly what you need. However, if you want that host to be able to send packets to server S1 but not to S3, then that standard ACL cannot create that logic in that location in the network.

For the third item in the list, placing more specific matching parameters early in each list makes you less likely to make mistakes in the ACL. For example, imagine that the ACL first listed a command that permitted traffic coming from subnet 10.1.4.0/23, and the second command denied traffic coming from host 10.1.4.1. Packets sent by host 10.1.4.1 would match the first ACE and never match the more specific second command. Note that later IOS versions prevent this mistake during configuration in some cases.

You avoid issues with ACL in an interim state by disabling ACLs on the interfaces before you edit them. First, IOS does not filter any packets if you delete an entire ACL and leave the IP ACL enabled on an interface. However, as soon as you add one ACE to that enabled ACL, IOS starts filtering packets based on that ACL. Those interim ACL configurations could cause problems.

Finally, IOS does not complain if you configure the **ip access-group** interface subcommand with the number or name of an undefined ACL. IOS considers the ACL to have no ACEs (a null ACL), and IOS does not filter packets with a null ACL enabled on an interface. However, double-check your spelling of ACL names between the commands that define the ACL and the **ip access-group** command. It is easy to make a spelling mistake or use difference case. (ACL names are case sensitive.)

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 7-9 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 7-9** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |
| Review command tables | | Book |

## Review All the Key Topics

Key Topic

**Table 7-10** Key Topics for Chapter 7

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Differences between named and numbered ACLs | 138 |
| List | Naming conventions for named ACL names | 139 |
| List | Rules for deleting individual ACEs in ACL mode | 141 |
| List | Rules for adding individual ACEs in ACL mode | 141 |
| Figure 7-3 | Syntax and notes about the three required matching fields in the extended ACL **access-list** command | 146 |
| Table 7-2 | Options for matching a single IP address in an ACE | 146 |
| Paragraph | Summary of extended ACL logic that all parameters must match in a single **access-list** statement for a match to occur | 146 |
| Figure 7-4 | Drawing of the IP header followed by a TCP header | 147 |
| Figure 7-5 | Syntax and notes about matching TCP and UDP ports with extended ACL **access-list** commands | 147 |
| Figure 7-7 | Logic and syntax to match TCP source ports | 148 |
| Table 7-4 | Popular IT services and their well-known ports | 149 |
| Table 7-5 | Popular user applications and their well-known ports | 149 |
| List | Guidelines for using extended numbered IP ACLs | 151 |
| List | ACL implementation recommendations | 156 |

## Key Terms You Should Know

ACL sequence number, extended access list, named access list

## Command References

Tables 7-11 and 7-12 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 7-11** Chapter 7 ACL Configuration Command Reference

| Command | Description |
|---|---|
| **access-list** *access-list-number* {**deny** \| **permit**} *protocol source source-wildcard destination destination-wildcard* [**log**] | Global command for extended numbered access lists. Use a number between 100 and 199 or 2000 and 2699, inclusive. |
| **access-list** *access-list-number* {**deny** \| **permit**} **tcp** *source source-wildcard* [*operator* [*port*]] *destination destination-wildcard* [*operator* [*port*]] [**log**] | A version of the **access-list** command with TCP-specific parameters. |
| **access-list** *access-list-number* **remark** *text* | Command that defines a remark to help you remember what the ACL is supposed to do. |
| **ip access-group** {*number* \| *name* [**in** \| **out**]} | Interface subcommand to enable access lists. |
| **ip access-list** {**standard** \| **extended**} *name* | Global command to configure a named standard or extended ACL and enter ACL configuration mode. |
| {**deny** \| **permit**} *source* [*source wildcard*] [**log**] | ACL mode subcommand to configure the matching details and action for a standard named ACL. |
| {**deny** \| **permit**} *protocol source source-wildcard destination destination-wildcard* [**log**] | ACL mode subcommand to configure the matching details and action for an extended named ACL. |
| {**deny** \| **permit**} **tcp**\|**udp** *source source-wildcard* [*operator* [*port*]] *destination destination-wildcard* [*operator* [*port*]] [**log**] | ACL mode subcommand to configure the matching details and action for a named ACL that matches TCP or UDP messages. |
| **remark** *text* | ACL mode subcommand to configure a description of a named ACL. |
| **no** {**deny** \| **permit**} *protocol source source-wildcard destination destination-wildcard* | ACL mode subcommand to delete a single ACE if an ACE exists with the exact same detail as in the **no** command. |
| **no** *sequence-number* | ACL mode subcommand to delete a single ACE if an ACE with that sequence number exists. |

**Table 7-12** Chapter 7 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip** *interface* [*type number*] | Includes a reference to the access lists enabled on the interface |
| **show access-lists** [*access-list-number* \| *access-list-name*] | Shows details of configured access lists for all protocols |
| **show ip access-lists** [*access-list-number* \| *access-list-name*] | Shows IP access lists, with the same information and format as the **show access-lists** command |

## Answers to Earlier Practice Problems

Table 7-13 lists the answers to the practice problems listed in Table 7-8. Note that for any question that references a client, you might have chosen to match port numbers greater than 49151, matching all dynamic ports. The answers in this table mostly ignore that option, but just to show one sample, the answer to the first problem lists one with reference to client ports greater than 49151 and one without. The remaining answers simply omit this part of the logic.

**Table 7-13** Building One-Line Extended ACLs: Answers

| | Criteria |
|---|---|
| 1 | **permit tcp host 10.1.1.1 10.1.2.0 0.0.0.255 eq www**<br>or<br>**permit tcp host 10.1.1.1 gt 49151 10.1.2.0 0.0.0.255 eq www** |
| 2 | **permit tcp 172.16.4.0 0.0.0.127 172.16.3.0 0.0.0.127 eq telnet** |
| 3 | **permit icmp 192.168.7.192 0.0.0.63 192.168.7.8 0.0.0.7** |
| 4 | **permit tcp 10.2.2.0 0.0.1.255 eq www 10.4.4.0 0.0.3.255** |
| 5 | **permit tcp 172.20.1.0 0.0.0.255 eq 23 172.20.44.0 0.0.1.255** |
| 6 | **permit tcp 192.168.99.96 0.0.0.15 192.168.176.0 0.0.0.15 eq www** |
| 7 | **permit icmp 10.55.66.0 0.0.0.127 10.66.55.0 0.0.0.63** |
| 8 | **permit ip any any** |



CHAPTER 8

# Applied IP ACLs

**This chapter covers the following exam topics:**

**5.0 Security Fundamentals**

**5.6 Configure and verify access control lists**

When you plan a real IP ACL to enable in a production network, the task often becomes large and complex. For instance, imagine you rely on the implied **deny any** logic at the end of the ACL. In doing so, your ACL must include permit commands matching all traffic you want to allow; otherwise, the ACL denies (discards) those packets. If you add a **permit ip any any** command to the end of your extended ACL, making the default to permit traffic, you have the opposite problem: you need to work hard to identify everything you want to deny and match that in your ACL. The complexities increase with multiple network engineers, competing IT interests, and a changing business environment.

This chapter gives you a glimpse into some of the issues you will need to consider when taking that next step. The first section of the chapter examines how to ensure you permit several essential protocols in the network so that later deny ACEs in an ACL do not discard the traffic. For instance, if your ACL matched much of the end-user traffic correctly but forgot about DNS, DHCP, ICMP, and OSPF, discarding those, the network would fail miserably. The first section works through some practical examples that may be useful in your network engineering job and give you deeper skills for the CCNA exam.

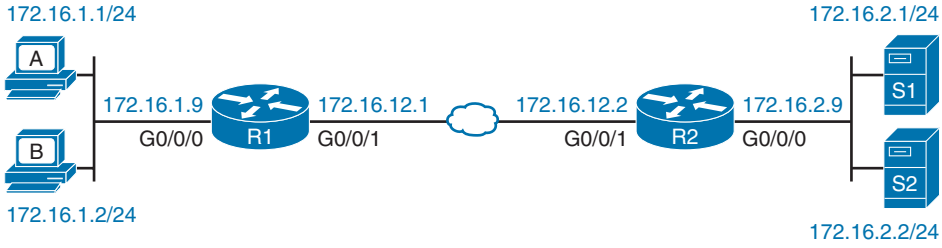
The final (short) section discusses some differences between Cisco IOS and IOS XE operating systems regarding IP ACLs. The CCNA exam does not imply any need to know such differences; however, knowing the minor differences can help. Also, that section discusses two features not yet discussed in the ACL chapters—how to use ACL resequencing and how to add a second ACL to an interface—so take the time to read through the final section.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 8-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| ACLs and Network Infrastructure Protocols | 1–4 |
| Comparing ACLs in IOS and IOS XE | 5, 6 |




**Refer to This Figure for Questions 1–4**

1. Router R1 in the figure uses the ACL below, with the **ip access-group QA_01 out** interface subcommand added to interface G0/0/1. The router has no other ACL-related configuration. In the output of the **show access-lists** command, which ACL line's counter should increment each time router R1 sends an OSPF Hello?

```
ip access-list extended QA_01
  10 permit tcp host 172.16.12.1 any eq 520
  20 permit tcp host 172.16.12.1 any
  30 permit ospf host 224.0.0.5 any
  40 permit ospf host 172.16.12.1 any
```

   a. Line 10
   b. Line 20
   c. Line 30
   d. Line 40
   e. None of the other answers are correct

2. In the figure, router R2 enables an ACL inbound on its G0/0/0 interface. Hosts A and B need to lease IP addresses from DHCP server S1. Router R1 provides the IP helper feature with the **ip helper-address 172.16.2.1** subcommand on its G0/0/0 interface. Which answers list an ACE that, if part of the ACL, would match DHCP messages between the server and clients? (Choose two answers.)
   a. **permit ip host 172.16.2.1 any**
   b. **permit ip any host 172.16.2.1**
   c. **permit udp any eq bootps host 0.0.0.0**
   d. **permit udp host 172.16.2.1 eq bootpc any**
   e. **permit udp host 172.16.2.1 eq bootps any**

3. In the figure, router R1 uses standard ACL QA_03. The ACL has one ACE; the ACE permits packets sourced from subnet 172.16.1.0/24. An engineer wants to filter SSH traffic, so only hosts from subnet 172.16.1.0/24 can SSH into a router. Which answer best describes how the engineer should proceed?
   a. Enable the ACL in vty mode with the **access-class QA_01 in** subcommand.
   b. Enable the ACL in vty mode with the **access-class QA_01 out** subcommand.
   c. Enable the ACL in vty mode with the **ip access-group QA_01 in** subcommand.
   d. Enable the ACL in vty mode with the **ip access-group QA_01 out** subcommand.

4. In the figure, an engineer enables ACL QA_04 inbound on router R1's G0/0/0 interface. Imagine host A attempts to SSH to address 172.16.12.1. Which line in the ACL will the packets match in this scenario?

```
ip access-list extended QA_04
   10 permit udp 172.16.1.0 0.0.0.255 any eq 22
   20 permit tcp 172.16.1.0 0.0.0.255 any eq 22
   30 deny udp any any eq 22
   40 deny tcp any any eq 22
```

   a. Line 10
   b. Line 20
   c. Line 30
   d. Line 40
   e. The implied **deny ip any any**

5. Before the introduction of the "Common" ACL in IOS XE, which answer is most accurate regarding how many IP ACLs can be enabled on one router?
   a. One per router
   b. One per router interface
   c. One per router interface per direction
   d. Two per router interface per direction
   e. Many per router interface per direction

6. Named ACL QA_06 has four lines with sequence numbers 10, 20, 30, and 40. As an exercise during a job interview, you are asked to edit the ACL so that the ACEs keep the same relative order but instead use sequence numbers 50, 70, 90, and 110. Which answer lists the method to effect this change with the fewest commands from the CLI?
   a. Issue the **ip access-list resequence QA_06 50 20** command in privileged EXEC mode.
   b. Issue the **ip access-list resequence QA_06 50 20** command in global configuration mode.
   c. Delete the entire ACL with the **no ip access-list...** global command and then reconfigure the entire ACL.
   d. From ACL mode, delete each **permit** or **deny** command and then reconfigure each.
   e. From ACL mode, issue the **resequence 50 20** subcommand.

# Foundation Topics

## ACLs and Network Infrastructure Protocols

Chapter 7, "Named and Extended IP ACLs," completes the discussion of most of the core ACL configuration features. However, the complexity of ACLs comes not from the configuration but from applying the tool to the large variety of networking protocols whose packets run through routers. This first major section of the chapter considers how to match some of those protocols in an ACL. This section discusses how to create ACEs for some of those overhead networking protocols. The topics include

- DNS
- ICMP Echos and Time Exceeded
- Routing Protocols and OSPF
- DHCP
- SSH and Telnet into routers and switches

## Filtering DNS

DNS messages flow through almost every router interface in an enterprise, so a production-ready IP ACL needs to permit it. The question becomes how permissive to be: to permit all of it, with no restrictions, or be more selective.

First, consider an enterprise with many branches. Figure 8-1 shows an example, with one branch router (B1), on the left, but many more branches exist. The figure shows two DNS servers and the DHCP server used in the upcoming discussions of ACLs and DHCP.

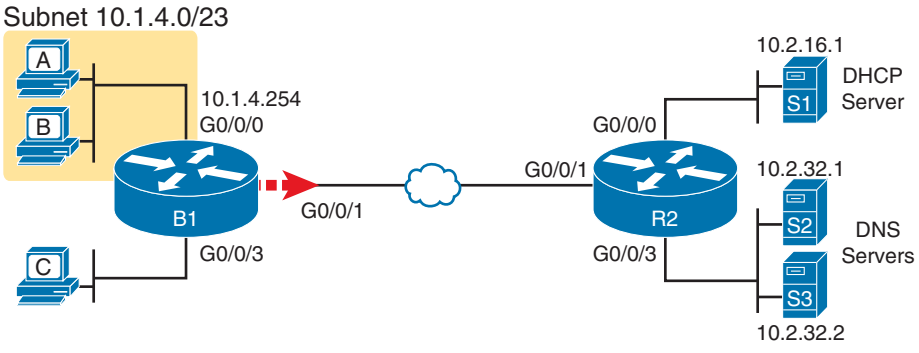



**Figure 8-1** *Enterprise Branches with DHCP and DNS Servers*

Next, think about planning an ACL so every branch router could use the same ACEs to match DNS traffic. You could begin by choosing the location and direction of each branch router's WAN interface in the outbound direction. As a result, the branch ACLs process packets sent toward the DNS servers.

Example 8-1 shows one approach. It ignores all IP addresses, matching source and destination addresses with the **any** keyword. The ACEs match based on these DNS facts:

- DNS uses both UDP and TCP.
- DNS uses well-known server port 53 (ACL keyword **domain**).

**Example 8-1** *Permitting All DHCP Traffic to DNS Servers*

```
R1# show running-config | section access-list
! ACEs below are part of extended ACL Branch_Common
 50 permit udp any any eq domain
 60 permit tcp any any eq domain
```

**NOTE** The ACLs in the many examples in this section focus on a few ACEs related to each topic. Those ACEs would be part of a much larger ACL.

Assuming no other earlier ACEs matched the DNS packets, the ACEs in the example permit all packets destined to any DNS server, regardless of source and destination address.

Example 8-2 shows a more secure alternative using the same location and direction. However, because IT controls the only legitimate DNS servers, the permit ACEs match the legitimate DNS servers' IP addresses (lines 110–140). ACEs 150 and 160 then deny all other traffic to other DNS servers. This two-phase approach strikes a balance of being more restrictive while allowing all legitimate DNS traffic.

**Example 8-2** *Permitting All DHCP Traffic to DHCP and DNS Servers*

```
R1# show running-config | section access-list
! Excerpt from extended ACL Branch_Common, replacing previous example's ACEs
 110 permit udp any host 10.2.32.1 eq domain
 120 permit udp any host 10.2.32.2 eq domain
 130 permit tcp any host 10.2.32.1 eq domain
 140 permit tcp any host 10.2.32.2 eq domain
! The next lines mimic example 8-1's ACEs, denying DNS (UDP and TCP)
 150 deny udp any any eq domain
 160 deny tcp any any eq domain
```

## Filtering ICMP

For another interesting networking feature impacted by ACLs, consider the **ping** command detailed in *CCNA 200-301 Official Cert Guide, Volume 1*, Chapter 20, "Troubleshooting IPv4 Routing." The **ping** command generates ICMP Echo Request messages. A host that responds to those messages generates ICMP Echo Reply messages. Want ping to work? The ACLs must permit all ICMP, or message types for Echo Request and Reply.

For example, consider the case with the branch office router B1 in Figure 8-1, again with an outbound ACL on router B1's G0/0/1 interface. But how much to permit and deny? The following list details three scenarios for ICMP with increasing restrictions. Example 8-3, which follows, shows ACEs for the first two scenarios.

1. Taking a one-stage approach: Permitting all ICMP messages
2. Taking a two-stage approach:
   a. Permitting all ICMP messages with source and destination from within the enterprise's private network 10.0.0.0
   b. Denying all other ICMP
3. The same as scenario 2 but permitting ICMP Echo Request and Reply only

---

Answers to the "Do I Know This Already?" quiz:

**1** E **2** A, E **3** A **4** B **5** C **6** B

Key Topic

**Example 8-3** *ACLs That Implement Two Alternative Approaches*

```
! The following single ACE achieves scenario 1 above
 210 permit icmp any any

! Alternately, the following two ACEs achieve scenario 2 above
 220 permit icmp 10.0.0.0 0.255.255.255 10.0.0.0 0.255.255.255
 230 deny icmp any any
```

The first ACE, line number 210, matches all ICMP messages. As for the syntax, to match ICMP messages, you do not use the TCP or UDP protocol types. Instead, you use the keyword **icmp**. (ICMP does not use TCP or UDP. Instead, the ICMP header follows directly after the IP header.)

The two-ACE alternative at the end of Example 8-3 matches all ICMP but with the added checks for both source and destination addresses. If the enterprise uses private class A network 10.0.0.0, this additional check means only packets sent from and to addresses in that private network match the **permit** statement. ICMP packets with a source or destination outside network 10.0.0.0, such as hosts in the Internet, fail to match line 220 but match line 230, with the router discarding those packets.

Example 8-4 shows the ACEs to achieve the third scenario. To match specific ICMP message types, use a keyword after the destination address field, as seen in lines 250 and 260. (The ICMP message type appears after the destination address in the ACE syntax.) Lines 250 and 260 together permit all ICMP Echo Request and Reply messages to/from hosts inside the enterprise's private network 10.0.0.0. The final line discards any other ICMP traffic, not relying on the default **deny any** to discard those packets.

**Example 8-4** *Permitting Internal ICMP Echos, Denying All Other ICMP*

```
! ACEs for the third ICMP scenario
ip access-list extended icmp_Echo_network_10
 250 permit icmp 10.0.0.0 0.255.255.255 10.0.0.0 0.255.255.255 echo
 260 permit icmp 10.0.0.0 0.255.255.255 10.0.0.0 0.255.255.255 echo-reply
 270 deny icmp any any
```

Example 8-4 probably discards too much traffic. One of the significant dangers with ACLs is filtering too much and preventing proper network operation. ICMP defines many message subtypes, with over 50 shown in CLI help in configuration mode. The ACE at line 270 in Example 8-4 discards all other ICMP message types, which likely prevents other useful ICMP functions. For example, those ACEs would filter ICMP Time Exceeded messages used by the **traceroute** command so that **traceroute** commands would not complete. The scenario 2 option, configured at the end of Example 8-3, might be the better compromise.

## Filtering OSPF

IP ACLs may discard OSPF messages, but there is no benefit to discarding them. So when you think about OSPF and ACLs, think about matching and permitting all OSPF packets. In particular, avoid unfortunate cases of matching OSPF packets with a deny action—for instance, by matching them with the implied **deny any** at the end of the ACL.

The scope of routing protocol packets impacts what packets an IP ACL might filter. As typical of routing protocol messages, OSPF packets have a short trip through a network. For example, in Figure 8-2, no OSPF packets flow past the dashed lines, and no routers forward any OSPF messages.

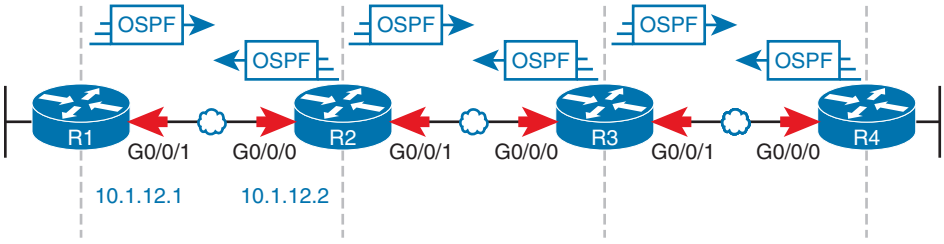



**Figure 8-2** *OSPF Message Flow Between Routers R1 and R2*

The last new concept when thinking about ACLs and routing protocols has to do with the default IOS behavior without outbound ACLs:

Key Topic

> Routers bypass outbound ACLs for local packets (packets generated by the router), including OSPF messages.

Combine that fact with the fact that routers do not forward OSPF packets, and the only ACLs that could discard OSPF messages are inbound ACLs. The arrow lines on the interfaces in Figure 8-2 show ACLs' locations and directions (inbound only) that could match OSPF messages.

**NOTE** IOS XE (but not IOS) allows you to change the behavior of bypassing outbound ACLs for router-generated packets by issuing the **ip access-list match-local-packets** global command.

ACLs should permit OSPF packets. You need to match the packets but not to filter a subset: instead, permit all OSPF messages. To match OSPF packets, use the **ospf** protocol keyword. OSPF does not use UDP or TCP but exists as a protocol whose header follows just after the IP header, so IOS supplies a keyword to match the OSPF protocol.

Example 8-5 shows ACEs for two scenarios, both of which expect an inbound ACL on router R1's G0/0/1 WAN interface from Figure 8-2. The example again shows two approaches: permitting all OSPF, or permitting OSPF messages from known neighbors while discarding all other OSPF messages. Summarizing the two scenarios:

1. Taking a one-stage approach: Permitting all OSPF messages
2. Taking a two-stage approach:
   a. Permitting all OSPF messages from router R2's WAN IP address (10.1.12.2)
   b. Denying all other OSPF messages

Key Topic

**Example 8-5** *Two OSPF Scenarios*

```
! Line 310 for OSPF scenario 1
 310 permit ospf any any
! Alternately, lines 320 and 330 for OSPF scenario 2.
 320 permit ospf host 10.1.12.2 any
 330 deny ospf any any
```

**NOTE** Appendix D, "Topics from Previous Editions," contains some older ACL content beyond the current CCNA exam topics. Some content discusses troubleshooting ACLs, with the rest discussing unexpected behavior with ACLs and router-generated packets. If you want to learn more, refer to the "Troubleshooting with IPv4 ACLs" section in Appendix D.

## Filtering DHCP

At first glance, you might expect to treat DHCP much the same as DNS regarding ACLs. Most companies use a single DHCP server (with high availability features), or maybe a few, but all have identifiable IP addresses. DHCP messages flow end to end through the network over most network links. DHCP uses a well-known server port (67). Matching DHCP messages on some routers follows the same straightforward logic you learned for web servers and DNS earlier in the chapter. However, applying ACLs for DHCP packets on the same router that performs the IP helper function has some unexpected rules.

For context, consider Figure 8-3, with DHCP clients on the left and a DHCP server on the right. The figure highlights four possible locations to consider for ACLs that would match packets headed toward the DHCP server. (You could also filter in the other direction; I just chose the left-to-right flow for the examples.) At all ACL locations and directions shown in the figure, the packets have a destination UDP port number of 67. However, the addresses differ based on the location in the network due to the IP helper feature on router R1.

Key Topic

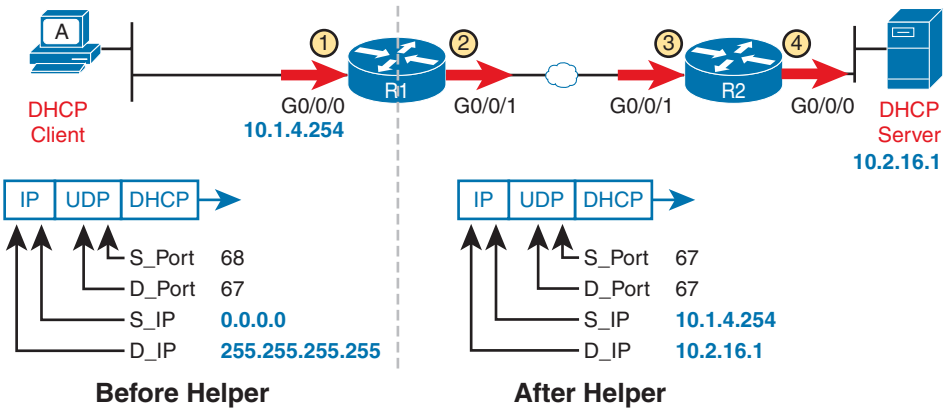



**Figure 8-3** *ACL Locations to Consider: Packets to DHCP Server*

Now consider the straightforward case, on router R2, with ACL locations three and four in the figure. R2 does not perform the helper function, so you can match based on the well-known DHCP server port and the server's IP address. You can also use the same options discussed earlier for DNS servers with Examples 8-1 and 8-2. Example 8-6 shows similar

8

ACEs as seen in Example 8-2 for DNS. The ACL would be useful on router R2 in location three or four per Figure 8-3:

- **One-stage:** Permit all DHCP messages going toward a DHCP server (port 67).
- **Two-stage:** First, permit all DHCP messages to the legitimate DHCP server and then filter (deny) all other messages to DHCP servers.

Key Topic

**Example 8-6** *Permitting All DHCP Traffic to DNS Servers on Router Main*

```
R2# show running-config section access-list
! Option 1: Allow all packets destined to DHCP server port (bootps 67)
 240 permit udp any any eq bootps
! Option 2: permit DHCP to known server and discard other messages sent to DHCP 67
 250 permit udp any host 10.2.16.1 eq bootps
 260 deny udp any any eq bootps
```

However, the ACL matching logic for DHCP differs on the router that performs the helper function. In particular:

Key Topic

1. By default, packets changed by a router due to the **ip helper-address** command bypass any outgoing ACL.
2. On a router interface with both an inbound ACL and the helper function, the router performs the ACL function first, before the helper function changes the IP addresses in DHCP messages.

For the first of those rules, consider ACL location two in Figure 8-3, the outbound ACL on R1's WAN interface. Per the first rule here, that ACL has not considered DHCP messages processed by router R1's IP helper function. R1 uses that function for DHCP clients like host A on the left side of Figure 8-3. Router R1 processes those incoming DHCP messages with the helper function, changes the source and destination address, and bypasses the outbound ACL.

Note that this default behavior on the IP helper router works well. You probably want to permit DHCP messages sent to the legitimate DHCP server you configured in the **ip helper-address** command, and those messages bypass any outbound ACL you enable there.

For the second rule about inbound ACLs, consider Example 8-7's configuration for router R1's LAN interface. It shows the **ip helper-address** command along with an enabled inbound ACL.

**Example 8-7** *Router R1 LAN Interface: Interactions with Inbound ACL and Helper*

```
R1# show running-config
(Lists relevant excerpts…)
interface GigabitEthernet0/0/0
  ip address 10.1.4.254 255.255.254.0
  ip helper-address 10.2.16.1
  ip access-group R1_Common in
!
! ACL excerpt: permit packets with unusual source/destination addresses to DHCP server
250 permit udp host 0.0.0.0 host 255.255.255.255 eq bootps
260 deny udp any any eq bootps
```

As noted in the second rule, IOS applies the inbound ACL logic before performing the helper function, as depicted by the ACL at location one in Figure 8-4. Because of that, any matching based on IP address must match the unusual IP addresses shown on the left side of Figure 8-4. (You could always choose to match the addresses with the **any** keyword, ignoring the addresses.) Just be aware that if you attempt to match based on IP address, you must match the addresses that exist before the router applies the helper function.

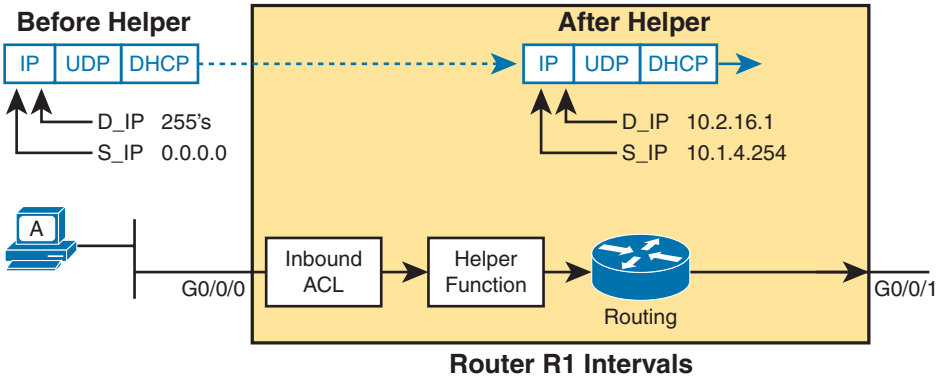



**Figure 8-4** *Order of Operations: Inbound ACL and IP Helper*

## Filtering SSH and Telnet

SSH and Telnet both give users a way to connect to the command-line interface of different operating systems. Telnet, the older of the two, is much less secure, which is why most people prefer SSH today.

When thinking about ACLs and the SSH and Telnet protocols, you should consider them for the usual interface ACLs just as with the other protocols discussed so far in this chapter. Additionally, routers and switches support Telnet and SSH, with IOS providing another method to enable an ACL to protect that specific function. First, this topic examines SSH/Telnet for packets passing through the router, matched with interface ACLs, and then it examines the **vty ACL** feature that filters router SSH/Telnet access.

### Filtering for End User SSH/Telnet

IT staff, and possibly some end users, may need to use SSH and Telnet to connect to a command line (shell) prompt on servers in the network. So if you plan to add an ACL on an interface, it will need to consider the Telnet and SSH protocols.

SSH and Telnet use TCP, with well-known ports 22 and 23, respectively. As a result, you can easily match both ports in one ACE using the **range 22 23** parameters in a **permit** command. (Note that IOS, as usual, changes some numeric port numbers to keywords—in this case, changing 23 to **telnet**.)

Example 8-8 shows two sets of ACEs, similar to a few other examples. The first shows a more general permit of SSH and Telnet traffic, while the second adds more restrictions.

1. One-stage: Permitting all SSH and Telnet messages going toward a server
2. Two-stage:
   a. Permitting all SSH and Telnet messages moving toward a server, with source and destination addresses from within the enterprise's private network 10.0.0.0
   b. Denying all other SSH and Telnet

Key Topic

**Example 8-8** *ACEs for the Two SSH and Telnet Scenarios for Host-Based Servers*

```
! The following single ACE achieves SSH/Telnet scenario 1 above
 410 permit tcp any any range 22 telnet

! Alternately, the following two ACEs achieve SSH/Telnet scenario 2 above
 450 permit tcp 10.0.0.0 0.255.255.255 10.0.0.0 0.255.255.255 range 22 telnet
 470 deny tcp any any range 22 telnet
```

The sample ACEs in Example 8-8 work to permit SSH and Telnet traffic when you know where all servers are. If you plan an ACL that matches packets moving toward the servers, you can then rely on matching the destination port. Conversely, if the plan enables the ACL for packets sent from the server to the client, you can check the well-known source ports.

However, with Telnet and SSH, you can seldom know where all SSH servers reside. All the routers and switches and many other IT devices support SSH or Telnet (or both). For instance, consider Figure 8-5. Host A uses SSH to connect to server S1, and host C connects to the switch SW2 SSH server. An outbound ACL on router R1's WAN interface (G0/0/1) would examine packets destined to SSH server S1 and also packets sourced from the SSH server in switch SW2. Packets flowing from the SW2 SSH server back to SSH client C will have a well-known source port of 22.

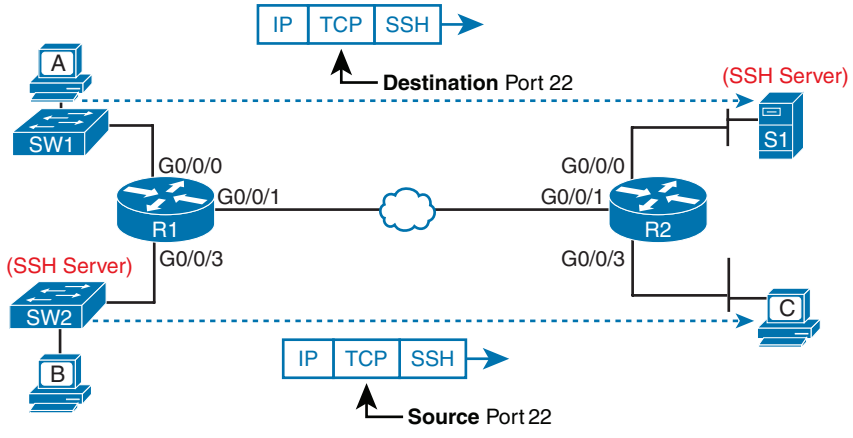



**Figure 8-5** *The Need to Match Both Directions for SSH and Telnet Traffic*

As a result, the choice to match SSH and Telnet by port number probably means you should match for both directions—that is, check for the source or destination port. Example 8-9

shows expanded sets of ACEs versus Example 8-8. The original ACEs checked for the SSH and Telnet destination ports, while the new lines also check for the SSH and Telnet source ports. In the new (highlighted) lines, pay close attention to the location of the **range 22 telnet** parameters.

**Example 8-9** *Updated SSH/Telnet ACEs to Add Checks for Both Directions*

```
! SSH/Telnet scenario 1, revised for packets both to and from the server
 410 permit tcp any any range 22 telnet
 420 permit tcp any range 22 telnet any

! SSH/Telnet scenario 2, revised for packets both to and from the server
 450 permit tcp 10.0.0.0 0.255.255.255 10.0.0.0 0.255.255.255 range 22 telnet
 460 permit tcp 10.0.0.0 0.255.255.255 range 22 telnet 10.0.0.0 0.255.255.255
 470 deny tcp any any range 22 telnet
 480 deny tcp any range 22 telnet any
```

## Filtering for Router VTY Access

IOS provides another feature to protect CLI access further by using a vty ACL enabled by the **access-class** command in vty configuration mode. When a user connects to a router or switch using Telnet or SSH, IOS associates a vty line with that user connection. IOS can apply an ACL to the vty lines, filtering the addresses that can telnet or SSH into the router or switch. Figure 8-6 shows the concept, with a network engineer at host A logging in to router R1 using SSH. The box above the router R1 icon represents the router's internal logic.

Key Topic

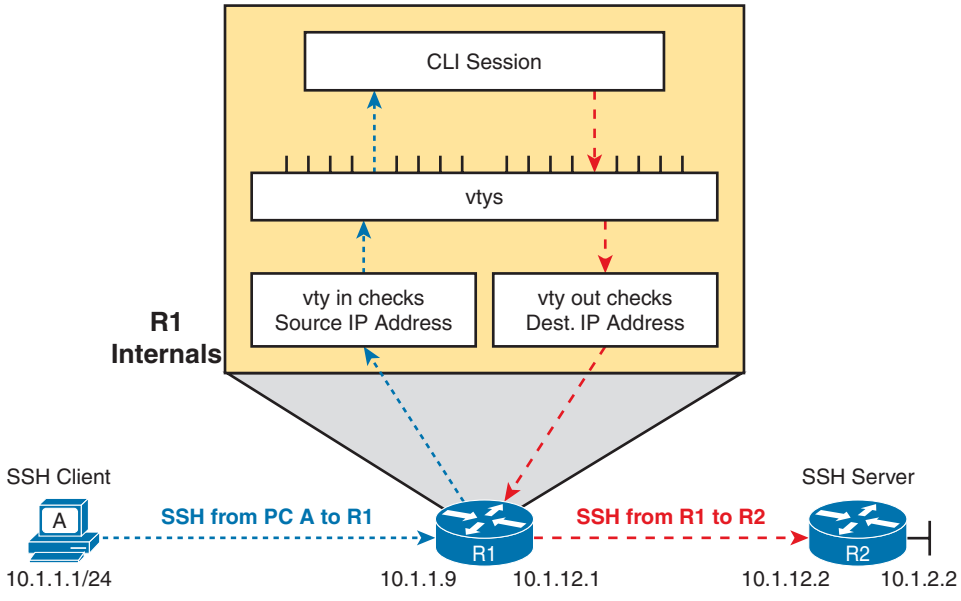



**Figure 8-6** *IP ACLs and the Router's Internal SSH/Telnet Servers*

First, consider a vty ACL enabled for the inbound direction, which examines packets for SSH/Telnet clients attempting to log in to a router. You can use either a standard or extended

ACL for a vty ACL, but a standard ACL works well. A vty ACL is not an interface ACL. Independent of the ACL matching logic, IOS applies these rules to vty ACLs:

- Consider only packets sent to the router's IP addresses on working interfaces.
- Consider only packets destined for the SSH (port 22) or Telnet (port 23) servers.

Beginning with those limits, the only other helpful match to make is the SSH/Telnet clients' IP addresses. A standard ACL works well for that.

For example, imagine all networking staff have IP addresses in subnet 10.1.1.0/24. As an extra precaution, you configure a vty ACL to permit SSH/Telnet from clients in that subnet but deny all other source addresses. Example 8-10 shows the configuration of the complete ACL on router R1 in Figure 8-6 and how to enable it in vty mode using the **access-class** command.

Key Topic

**Example 8-10** *vty Access Control Using the* **access-class in** *Command*

```
line vty 0 15
 transport input all
 access-class IT_only in
!
ip access-list standard IT_only
 10 remark matches packets sourced from subnet 10.1.1.0/24 only; implied deny any
 10 permit 10.1.1.0 0.0.0.255
```

IOS uses the **access-class** *acl* **in** vty subcommand to enable the function. The keyword **in** refers to Telnet and SSH connections into this router. As configured, ACL **IT_only** permits packets with source addresses from the IT subnet and denies all others due to the implied **deny any** at the end of the list. Note that the command to enable the ACL, **access-class**, used in vty mode, is a different command and mode than is used to enable ACLs on interfaces.

Inbound vty ACLs are popular, but IOS also supports the less common outbound vty ACL. The outbound option works in cases for which a current SSH/Telnet user at the CLI uses the **ssh** or **telnet** EXEC command to connect to the CLI of another device. For instance, in Figure 8-6, the user at host A could SSH to router R1. Once logged in, if that user then used the **ssh** or **telnet** EXEC command to attempt to connect to router R2, IOS would apply the outbound vty ACL to that connection request.

To use an outbound vty ACL, create an ACL and enable it with the **access-class** *acl* **out** command in vty configuration mode. As with inbound vty ACLs, IOS processes limited packets (those sourced from router IP addresses) and destined for the SSH or Telnet well-known ports.

Standard ACLs often work well for outbound vty ACLs, but with a surprising twist. When used as an outbound vty ACL, IOS compares the standard IP ACL address parameters to the *destination IP address of the packet* rather than the source address. That is, it filters based on the device to which the **telnet** or **ssh** command tries to connect. Example 8-11 demonstrates with an outbound vty ACL that permits only address 10.1.12.1, but not 10.1.2.2. The example shows the failed SSH attempt and IOS error message, followed by the successful attempt.

**Example 8-11** *vty Access Control Outbound on Router R1*

```
! Configuration excerpt first
line vty 0 15
 transport input all
 access-class R2_WAN out
!
ip access-list standard R2_WAN
  10 permit host 10.1.12.1
!
! An attempt denied by the ACL to router R2's far-side LAN IP address
R1# ssh -l wendell 10.1.12.2
% Connections to that host not permitted from this terminal

! An attempt permitted by the ACL, resulting in password prompt from router R2.
R2# ssh -l wendell 10.1.12.1
Password:

R2>
```

On a final note, outbound vty ACLs also support extended ACLs, but with the logic you would expect. It matches the source address parameters to the packet source address and the destination address parameters to the destination address.

## Comparing ACLs in IOS and IOS XE

Cisco added ACLs to IOS in the early days of the company. Over many decades, Cisco has added ACL features and commands over many IOS versions. Cisco has added some features and command options to IOS XE that differ from IOS over time.

This final section of the chapter summarizes some of the current differences between the recent versions of IOS and IOS XE in early 2023 when I most recently revised this chapter. I did not choose to add this section because of any exam topic. More generally, the exam topics do not mention any need to compare ACLs on IOS versus IOS XE. Cisco historically avoids asking anything to do with IOS software versions. However, you may practice ACLs with IOS, but then an exam question may use output from IOS XE, or vice versa. This short section summarizes some of the differences to prepare you. The goal is for you to be comfortable if you see minor differences in an ACL exam question versus what you see during your study time.

This section first summarizes the differences and then discusses two ACL features that behave differently on IOS than on IOS XE: sequence number resequencing and the common ACL.

### Configuration Syntax and Show Commands

The original numbered IP ACLs used global commands, specifically the **access-list** global command. Later, IOS added support for ACL mode, which can be reached with the **ip access-list** command, with subcommands to define the detail. Informally, consider those two options as global mode and ACL mode.

Table 8-2 lists some of the differences between ACL support in IOS versus IOS XE. The ACL chapters in this book have mentioned several of these similarities and differences in context, but the table organizes them for easy reference. Note that the table also lists a few similarities to provide more context.

**Table 8-2** ACL Differences with IOS and IOS XE

| Feature | IOS | IOS XE |
|---|---|---|
| Numbered ACLs: Allows configuration with global commands | Yes | Yes |
| Numbered ACLs: Allows configuration with ACL mode commands for easy ACL editing | Yes | Yes |
| Numbered ACLs: **running-config** shows ACL as global commands* | Yes | No |
| Named ACLs: Configured and stored as ACL mode commands | Yes | Yes |
| Named ACLs: **show running-config** lists sequence numbers | No | Yes |
| Named ACLs: **show access-lists** lists sequence numbers | Yes | Yes |
| Resequences ACL sequence numbers on reload by default** | Yes | No |
| Supports second (common) ACL per interface/direction | No | Yes |
| Allows matching multiple nonconsecutive ports with **eq** | No | Yes |

* IOS stores the commands as global commands, while IOS XE stores them as ACL mode commands, regardless of configuration style.

** XE can be configured to resequence ACLs on reload, but by default, it does not.

Working through the table, the first two rows confirm that IOS and IOS XE allow configuring numbered ACLs with the original-style global commands or ACL mode subcommands. The third line shows the difference: IOS stores the configuration as globals, but IOS XE stores them as ACL mode subcommands—no matter how you configured them.

The next three rows (unhighlighted) reveal similar features for named ACLs. The one difference: IOS XE reveals ACL sequence numbers in **show running-config** command output, while IOS does not. IOS requires the **show access-lists** or **show ip access-lists** commands to see the sequence numbers.

The final topics of this chapter discuss the remaining rows in the table, which list features that differ between IOS and IOS XE.

## Resequencing ACL Sequence Numbers

You have seen many examples of ACL sequence numbers in this chapter and the preceding chapter. For instance, if you ignore them when configuring, IOS (and IOS XE) adds them for you, beginning at 10. IOS (and IOS XE) assigns sequence numbers to each successive **permit** or **deny** command in increments of 10. Alternatively, you can choose your sequence numbers with commands that begin the sequence number before the **permit** or **deny** command. You can also delete lines and add new ones using sequence numbers.

At one point in the history of IOS, Cisco decided to add **ACL resequencing** features. First and foremost, IOS will resequence ACL sequence numbers automatically at reload and that behavior continues today for IOS. So, for IOS, when you reload the router, IOS changes the sequence numbers of all ACEs in each ACL. It gives the first ACE sequence number 10, the next 20, and so on, with an increment of 10. For instance, an ACL with ACEs numbered 10,

30, and 60 before the reload would have the same ACEs in the same order after reload but with sequence numbers 10, 20, and 30.

Additionally, IOS supports manual ACL resequencing. The **ip access-list resequence** *name|number first increment* global configuration command performs the task, for one specific ACL, with the listed initial sequence number and increment. For instance, imagine the ACL named acl_01 had three **permit** commands with line numbers 10, 30, and 60 again. Example 8-12 shows the results of the **ip access-list resequence acl_01 100 20** global command. The last two parameters define the first sequence number as 100 with increments of 20. (Both IOS and IOS XE support resequencing, as shown in the example.)

Key Topic

**Example 8-12** *Resequencing ACLs with the* **ip access-list resequence** *Global Command*

```
R1# configure terminal
R1(config)# ip access-list resequence acl_01 100 20
R1(config)# do show access-list acl_01
Extended IP access list acl_01
    100 permit ip 10.1.4.0 0.0.1.255 any
    120 permit ip 10.2.4.0 0.0.1.255 any
    140 permit ip 10.0.0.0 0.255.255.255 10.0.0.0 0.255.255.255
```

IOS and IOS XE differ with automatic resequencing. IOS always resequences at any restart (the **reload** command or power off/on), while IOS XE does not resequence at restart. IOS XE calls its approach **ACL persistence**, configured by the (default) global command **ip access-list persistent**. IOS XE allows the same resequencing behavior as IOS by disabling ACL persistence with the **no ip access-list persistent** global command.

ACL sequence number persistence, the default with IOS XE, makes sense. You can choose to renumber your ACLs only when desired and use different numbering plans other than starting at 10 and incrementing by 10.

## Using a Second (Common) Interface ACL

For the long history of ACL support in Cisco IOS, the following rule dictates how many router ACLs you can enable on an interface:

> One ACL per layer three protocol, per direction, per interface.

For instance, today, the most common Layer 3 protocols in enterprise routers are IPv4 and IPv6. On any router interface, the router supports one IPv4 ACL inbound, one IPv4 ACL outbound, one IPv6 ACL inbound, and one IPv6 ACL outbound. Do you need all those at once? Seldom. More importantly, you cannot configure logic with multiple ACLs enabled on one interface in one direction to be processed one after the other. Instead, you have to put all the ACEs into one ACL.

IOS XE breaks that mold allowing two IPv4 ACLs on an interface in the same direction. Cisco refers to the first ACL as the **common ACL**, with the second as the *regular ACL*. However, both ACLs still use the same syntax and logic as before. The router optimizes internal processing, but the result is that it matches packets as if it first matched the common ACL and then the regular ACL, as shown in Figure 8-7.

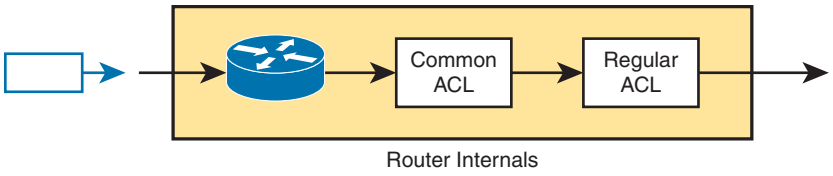



**Figure 8-7** *Two ACLs per Interface per Direction with a Common ACL*

You still use the **ip access-group** command to enable the two ACLs, but with different syntax. Example 8-13 shows how to enable two ACLs outbound on some router's G0/0/1 interface and confirms the enabled ACLs with the **show ip interface** command. You may want to compare this output to the same command's output in Example 6-2 from Chapter 6, "Basic IPv4 Access Control Lists," taken from IOS, so it does not mention a common ACL. The output here confirms an outgoing common and regular ACL but no incoming ACLs.

**Example 8-13** *Enabling Two IP ACLs, Outbound*

```
R1# configure terminal
R1(config-if)# interface gigabitEthernet 0/0/1
R1(config-if)# ip access-group common common_all unique_01 out
R1(config-if)# do show ip interface g0/0/1
GigabitEthernet0/0/1 is up, line protocol is up
  Internet address is 10.1.12.1/24
  Broadcast address is 255.255.255.255
  Address determined by non-volatile memory
  MTU is 1500 bytes
  Helper address is not set
  Directed broadcast forwarding is disabled
  Multicast reserved groups joined: 224.0.0.5 224.0.0.10 224.0.0.6
  Outgoing Common access list is common_all
  Outgoing access list is unique_01
  Inbound Common access list is not set
  Inbound access list is not set
```

Note that the addition of the common ACL feature does not improve what you can match with an ACL; instead, Cisco's motivation for this feature stems from a need to conserve entries in tables used by hardware ASICs in multilayer switches and in some routers. The first **ip access-group** command that enables a common ACL requires table entries for the common ACL's ACEs. But the router or switch can reuse those. So, any repeated **ip access-group** command that enables the same common ACL does not require additional ASIC table entries, reducing the consumption of that resource.

**NOTE** IOS allows one **ip access-group** subcommand for each direction, but no more—regardless of whether the commands refer to a common ACL. For example, with an existing **ip access-group fred out** interface subcommand, configuring the **ip access-group common wilma betty out** subcommand on the interface replaces the existing command.

### Matching Multiple Nonconsecutive Ports with eq

When matching TCP and UDP port numbers, Cisco ACLs have long supported matching a single port with the **eq** *number* parameters or a range of consecutive ports with the **range** *first-last* parameters. However, to match multiple nonconsecutive ports, the syntax required multiple ACEs.

IOS XE adds syntax and logic to match up to ten port numbers after the **eq** keyword in a single ACE. For instance, the command **permit tcp any any eq 10 12 14 16 18 20** matches all IP packets that have a TCP header, with any of the port numbers. In other words, IOS XE uses a logical OR between the port numbers, so it would match port 10, or 12, or 14, and so on.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 8-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 8-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Repeat DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |
| Review command tables | | Book |

## Review All the Key Topics



Key Topic

**Table 8-4** Key Topics for Chapter 8

| Key Topic Element | Description | Page Number |
|---|---|---|
| Example 8-3 | Two alternative approaches for permitting some ICMP traffic | 165 |
| Paragraph | Router logic for outbound ACL bypass for local packets | 166 |
| Example 8-5 | Two alternative approaches for permitting OSPF traffic | 167 |
| Figure 8-3 | Addresses and port numbers in DHCP, before and after the DHCP helper function | 167 |
| Example 8-6 | Two alternative approaches for permitting DHCP traffic | 168 |
| List | Special rules that affect ACL processing for DHCP on a router using the **ip helper-address** command | 168 |
| Example 8-8 | Two alternative approaches for permitting SSH traffic routed by a router | 170 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 8-6 | Vty ACL concepts | 171 |
| Example 8-10 | Inbound vty ACL | 172 |
| Example 8-12 | Demonstrating ACL resequencing by command | 175 |

## Key Terms You Should Know

ACL persistence, ACL resequencing, Common ACL, vty ACL

## Command References

Tables 8-5 and 8-6 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

The tables list commands introduced in this chapter; refer to Chapters 6 and 7 for more documentation on commands related to IP ACLs.

**Table 8-5** Chapter 8 ACL Configuration Command Reference

| Command | Description |
|---|---|
| **access-list** *access-list-number* {**deny** \| **permit**} **ospf** *source source-wildcard destination destination-wildcard* [**log**] | A version of the **access-list** command that matches all OSPF messages. |
| *[sequence-number]* {**deny** \| **permit**} **ospf** *source source-wildcard destination destination-wildcard* [**log**] | The equivalent of the previous row's command but in ACL mode. |
| **access-list** *access-list-number* {**deny** \| **permit**} **icmp** *source source-wildcard destination destination-wildcard* [*icmp-message*] | A version of the **access-list** command that matches ICMP with the option to list the specific ICMP message type. |
| *[sequence-number]* {**deny** \| **permit**} **icmp** *source source-wildcard destination destination-wildcard* [*icmp-message*] | The equivalent of the previous row's command but in ACL mode. |
| **ip access-list resequence** *name\|number initial increment* | Global command to resequence ACL sequence numbers, beginning with the initial value, and adding the increment to get each successive sequence number. |
| **[no] ip access-list persistent** | Global command to toggle on (without the **no**; default) or off (with the **no**) ACL sequence number persistence. IOS XE only. |
| **ip access-group common** *common_acl regular_acl* {[**in** \| **out**]} | Interface subcommand, which allows two ACLs to be enabled simultaneously on one interface and direction. IOS XE only. |

**Table 8-6** Chapter 8 EXEC Command Reference

| Command | Description |
|---|---|
| **ssh -l** *username address\|hostname* | Initiates an SSH client connection from the CLI to another device at the listed address or hostname, using the listed username to log in |
| **telnet** *address\|hostname* | Initiates a Telnet client connection from the CLI to another device at the listed address or hostname |
| **show ip access-lists** [*access-list-number* \| *access-list-name*] | Shows IP access lists |



# Part II Review

Keep track of your part review progress with the checklist in Table P2-1. Details about each task follow the table.

**Table P2-1** Part II Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |

## Repeat All DIKTA Questions

For this task, use the PTP software to answer the "Do I Know This Already?" questions again for the chapters in this part of the book.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Do Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Config Labs:** The author's blog (https://www.certskills.com) includes a series of configuration-focused labs that you can do on paper, each in 10–15 minutes. Review and perform labs for this part of the book by using the menus to navigate to the per-chapter content and then finding all config labs related to that chapter. (You can see more detailed instructions at https://www.certskills.com/config-labs).

**Other:** If you are using other lab tools, here are a few suggestions: when building ACL labs, you can test with Telnet (port 23), SSH (port 22), ping (ICMP), and traceroute (UDP) traffic as generated from an extra router. So, do not just configure the ACL; make an ACL that can match these types of traffic, denying some and permitting others, and then test.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.



Before the year 2020, Cisco offered several CCNA certifications. Those included CCNA Routing and Switching as well as CCNA Security. In 2020, Cisco discontinued all of those, replacing them with one CCNA certification. That CCNA certification mostly resembled CCNA Routing and Switching, but with some parts of CCNA Security and CCNA Wireless included. The current CCNA certification, based on the 200-301 Exam's Version 1.1 blueprint, continues to include meaningful security and wireless LAN content.

Part III of this book discusses the largest security topics from Domain 5 of the blueprint, while leaving some security topics to other parts of the books. For instance, the wireless LAN security topics in that domain reside in Part I of this book, along with the rest of the Wireless LAN materials. This part, Part III, includes all standalone security topics.

Chapter 9 kicks off Part III with a wide description of security threats, vulnerabilities, and exploits. This introductory chapter sets the stage to help you think more like a security engineer.

Chapters 10, 11, and 12 then focus on a wide range of short security topics. Those topics include Chapter 10's discussion of how to protect router and switch logins and passwords, along with an introduction to the functions and roles of firewalls or intrusion protection systems (IPSs). Chapters 11 and 12 then get into three separate security features built into Cisco switches: port security (covered in Chapter 11), DHCP Snooping, and Dynamic ARP Inspection (DAI) (both covered in Chapter 12). All three security features use a LAN switch to examine frames as they enter the switch interface. This information enables port security, DHCP Snooping, and DAI to decide whether to allow the message to continue on its way.

# Part III

## Security Services

**Chapter 9:** Security Architectures

**Chapter 10:** Securing Network Devices

**Chapter 11:** Implementing Switch Port Security

**Chapter 12:** DHCP Snooping and ARP Inspection

**Part III Review**



CHAPTER 9

# Security Architectures

**This chapter covers the following exam topics:**

**5.0 Security Fundamentals**

**5.1 Define key security concepts (threats, vulnerabilities, exploits, and mitigation techniques)**

**5.2 Describe security program elements (user awareness, training, and physical access control)**

**5.4 Describe security password policies elements, such as management, complexity, and password alternatives (multifactor authentication, certificates, and biometrics)**

**5.8 Compare authentication, authorization, and accounting concepts**

As you have learned about various networking technologies, your attention has probably been focused on using network devices to build functional networks. After all, networks should let data flow freely so that all connected users have a good experience, right? The unfortunate fact is that not all connected users can be trusted to obey the rules and be good network citizens. In this chapter, you learn about many aspects of an enterprise network that can be exploited, as well as some ways you can protect them.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 9-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Security Terminology | 1, 2 |
| Common Security Threats | 3–7 |
| Controlling and Monitoring User Access | 8 |
| Developing a Security Program to Educate Users | 9 |

1. Which one of the following terms means anything that can be considered to be a weakness that can compromise security?
   a. Exploit
   b. Vulnerability

**c.** Attack

**d.** Threat

2. An actual potential to exploit a vulnerability is known as which one of the following terms?

   **a.** Vulnerability

   **b.** Attack

   **c.** Exploit

   **d.** Threat

3. In a spoofing attack, which of the following parameters are commonly spoofed? (Choose two answers.)

   **a.** MAC address

   **b.** Source IP address

   **c.** Destination IP address

   **d.** ARP address

4. Suppose an attacker sends a series of packets toward a destination IP address with the TCP SYN flag set but sends no other packet types. Which of the following attacks is likely taking place?

   **a.** Spoofing attack

   **b.** Reflection attack

   **c.** Reconnaissance attack

   **d.** Denial-of-service attack

   **e.** None of the other answers are correct.

5. In a reflection attack, the source IP address in the attack packets is spoofed so that it contains which one of the following entities?

   **a.** The address of the attacker

   **b.** The address of the reflector

   **c.** The address of the victim

   **d.** The address of the router

6. During a successful man-in-the-middle attack, which two of the following actions is an attacker most likely to perform?

   **a.** Eavesdrop on traffic passing between hosts

   **b.** Induce a buffer overflow on multiple hosts

   **c.** Modify data passing between hosts

   **d.** Use ping sweeps and port scans to discover the network

7. Which one of the following is the goal of a brute-force attack?

   **a.** Try every possible TCP port until a service answers.

   **b.** Try every possible combination of keyboard characters to guess a user's password.

   **c** Initiate a denial-of-service operation on every possible host in a subnet.

   **d.** Spoof every possible IP address in an organization.

8. Which one of the following is an example of a AAA server?
   a. DHCP
   b. DNS
   c. SNMP
   d. ISE
9. Physical access control is important for which one of the following reasons?
   a. It prevents unauthorized people from sitting at a corporate user's desk and using their computer.
   b. It prevents users from getting angry and damaging computer equipment.
   c. It prevents unauthorized access to network closets.
   d. It prevents fires from destroying data centers.

# Foundation Topics

## Security Terminology

In a perfect world, you might build a network that supports every user in an enterprise, with the assumption that every user is known, every user is approved to access everything on the network, and every user will use the available resources exactly according to some corporate guidelines. The network shown in Figure 9-1 might represent such a scenario. Even this ideal, closed system is not completely secure because a user might decide to misbehave in order to pester a coworker or to view information on the corporate server that should be restricted or confidential.

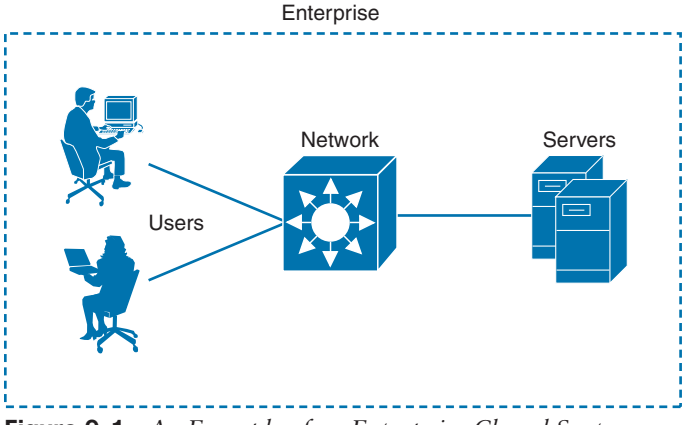



**Figure 9-1** *An Example of an Enterprise Closed System*

Now consider that almost no enterprise uses such a limited, closed environment. After all, the enterprise will probably want to somehow connect itself to the public Internet and perhaps to some corporate partners. It will also probably want to allow its workers to be mobile and carry laptops, tablets, and smartphones in and out of the corporate boundaries for convenience. The enterprise might want to provide network access to guests who visit. If the enterprise offers wireless connectivity to its employees (and guests), it might also unknowingly offer its wireless access to people who are within range of the signals. And the

list goes on and on. As the network and its connectivity expand, as Figure 9-2 shows, the enterprise will have more difficulty maintaining the safe, closed boundary around itself.

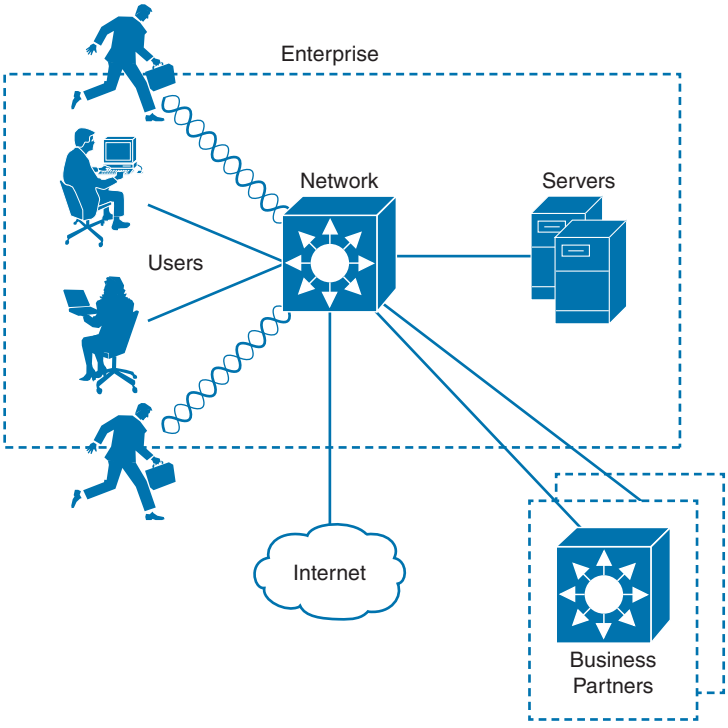



**Figure 9-2** *An Example Enterprise Extends Beyond Its Own Boundary*

To begin securing a network, you first need to understand what might go wrong with it. Think of an enterprise network as a simple box-shaped facility, as shown in part A of Figure 9-3. When all of the walls, floor, and ceiling are made of a very strong material and are very thick, the contents inside the box will likely remain safe from harm or theft. The owner, however, might have a hard time getting in and out of the box.

Key Topic

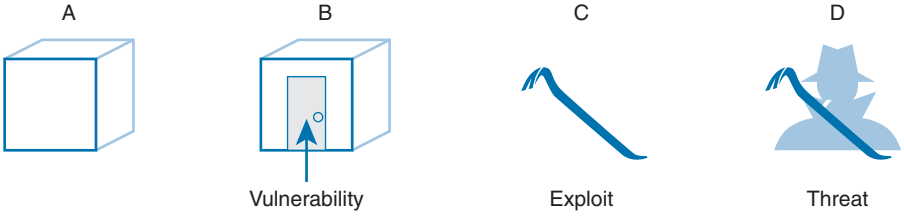



**Figure 9-3** *Security Terminology Illustrated*

Suppose a door is introduced for convenience, as shown in part B of Figure 9-3. The owner can now come and go, but so might anyone else. Even if the door is locked, someone might find a way to get the door open and access the treasures inside. Because no door is impenetrable, the door becomes a **vulnerability**. In terms of security, a vulnerability is anything that can be considered to be a weakness that can compromise the security of something else, such as the integrity of data or how a system performs.

Just because a vulnerability exists, nothing is necessarily in jeopardy. In the locked door example, nobody but the trusted owner can open the door unless some sort of tool other than the key is used. Such a tool can be used to exploit a vulnerability. In fact, the tool itself is called an **exploit**, as shown by the pry bar in part C of Figure 9-3. An exploit is not very effective if it is used against anything other than the targeted weakness or vulnerability.

Technically, an exploit such as the pry bar is not very effective at all by itself. Someone must pick it up and use it against the vulnerability. In part D of Figure 9-3, a malicious user possesses the pry bar and intends to use it to open the locked door. Now there is an actual potential to break in, destroy, steal, or otherwise modify something without permission. This is known as a **threat**.

In the IT world of networks, systems, workstations, and applications, there are many, many different vulnerabilities and exploits that can be leveraged by malicious users to become threats to an organization and its data. The remainder of this chapter provides an overview of many of them, along with some techniques you can leverage to counteract or prevent the malicious activity. Such measures are known as **mitigation techniques**. You might be thinking of some ways the Figure 9-3 building owner could mitigate the threats from attack. Perhaps the owner could add stronger, more secure locks to the door, a more robust door frame to withstand prying forces, or an alarm system to detect an intrusion and alert the authorities.

## Common Security Threats

Because modern enterprise networks are usually made up of many parts that all work together, securing them can become a very complex task. As with the simple box analogy, you cannot effectively try to secure it until you have identified many of the vulnerabilities, assessed the many exploits that exist, and realized where the threats might come from. Only then can the appropriate countermeasures and mitigations be put in place.

You should also consider some important attributes of enterprise resources that should be protected and preserved. As you work through the many threats that are discussed in this chapter, think about the vulnerability and exploit that make the threat possible. Notice how many different parts of the enterprise network exhibit vulnerabilities and how the threats are crafted to take advantage of the weaknesses.

### Attacks That Spoof Addresses

When systems behave normally, parameters and services can be trusted and used effectively. For example, when a machine sends an IP packet, everyone expects the source IP address to be the machine's own IP address. The source MAC address in the Ethernet frame is expected to be the sender's own MAC address. Even services like DHCP and DNS should follow suit; if a machine sends a DHCP or DNS request, it expects any DHCP or DNS reply to come from a legitimate, trusted server.

**Spoofing attacks** focus on one vulnerability; addresses and services tend to be implicitly trusted. Attacks usually take place by replacing expected values with spoofed or fake values. Address spoofing attacks can be simple and straightforward, where one address value is substituted for another.

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** D **3** A, B **4** D **5** C **6** A, C **7** B **8** D **9** C

For example, an attacker can send packets with a spoofed source IP address instead of its own, as shown in Figure 9-4. When the target receives the packets, it will send return traffic to the spoofed address, rather than the attacker's actual address. If the spoofed address exists, then an unsuspecting host with that address will receive the packet. If the address does not exist, the packet will be forwarded and then dropped further out in the network.

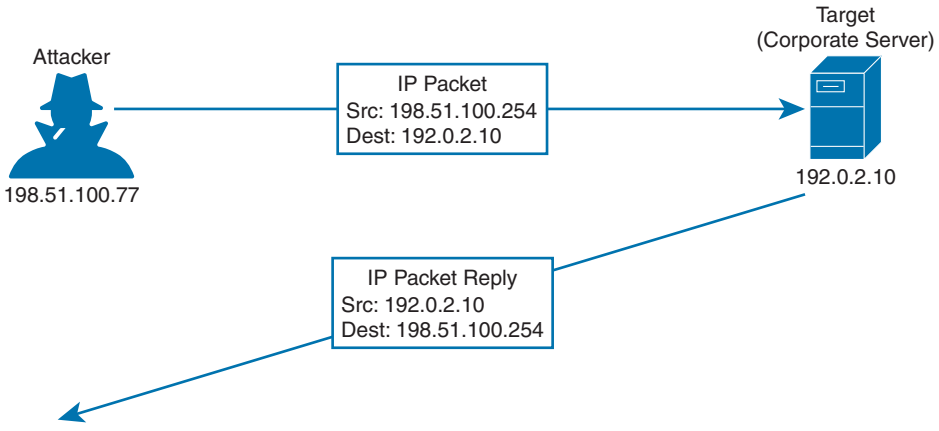



**Figure 9-4** *A Sample Spoofing Attack*

An attacker can send spoofed MAC addresses too, to add false information to the forwarding tables used by Layer 2 switches or ARP tables used by other hosts and routers. DHCP requests with spoofed MAC addresses can also be sent to a legitimate DHCP server, filling its address lease table and leaving no free IP addresses for normal use.

Note that Chapter 11, "Implementing Switch Port Security," discusses a tool that can be used to help mitigate MAC address spoofing. In Chapter 12, "DHCP Snooping and ARP Inspection," you can learn more about Dynamic ARP Inspection (DAI) and how to use it to mitigate IP address spoofing using ARP.

### Denial-of-Service Attacks

In the normal operation of a business application, clients open connections to corporate servers to exchange information. This operation might occur in the form of web-based sessions that are open to internal users as well as external users on the Internet. The process is simple: users open a web browser to the corporate site, which then opens a TCP connection with the corporate web server; then some transaction can take place. If all the users are well behaved and conduct legitimate transactions, the corporate servers are (hopefully) not stressed and many clients can do business normally.

Now suppose a malicious user finds a way to open an abnormal connection to the same corporate server. The TCP connection begins with the malicious user sending a SYN flag to the server, but the source IP address is replaced with a fake address. The server adds the TCP connection to its table of client connections and replies to the fake address with a SYN-ACK. Because the fake address is not involved in the TCP connection, there is no ACK reply to complete the TCP three-way handshake. The incomplete connection stays in the server's table until it eventually times out and is removed. During this time, the attacker can try to open many, many more abnormal connections at such a rate that the server's connection table fills. At that point, the server is no longer able to maintain TCP connections with legitimate users, so their business transactions all halt. Figure 9-5 illustrates this process.

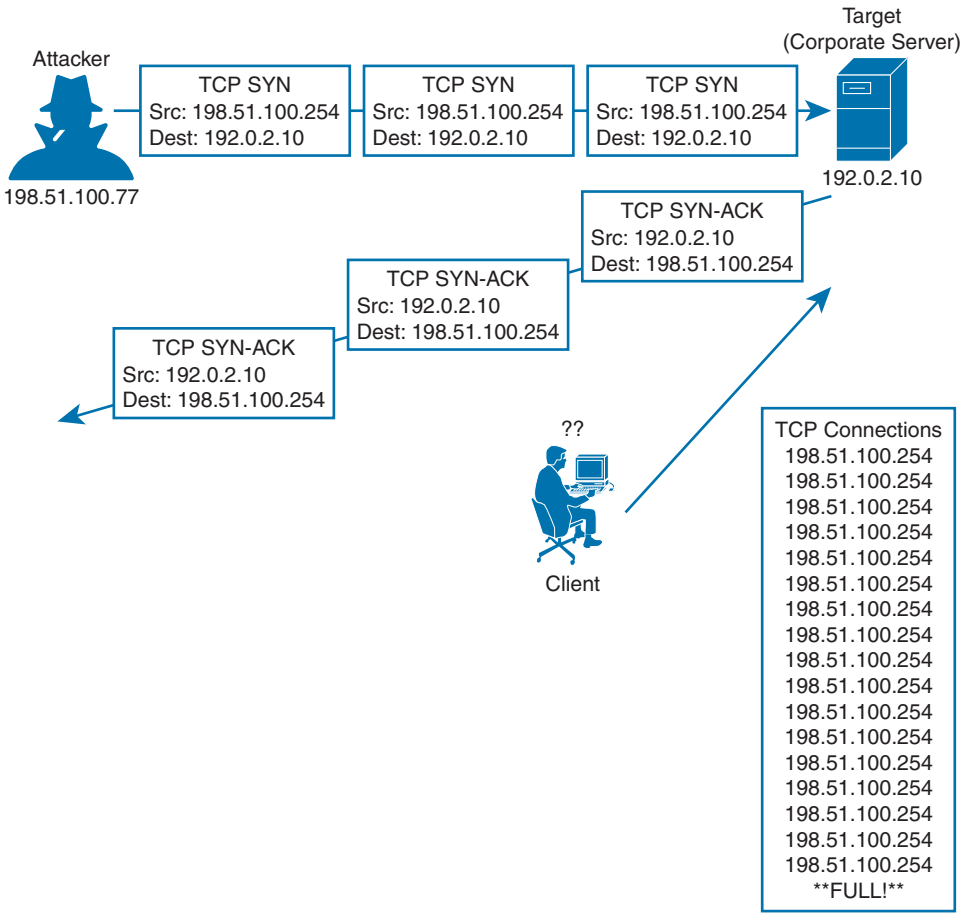



**Figure 9-5** *A Sample Denial-of-Service Attack*

When an attacker is able to deplete a system resource, services and systems become unavailable or crash. This is called a **denial-of-service (DoS) attack** because it denies service to legitimate users or operations. DoS attacks can involve something as simple as ICMP echo (ping) packets, a flood of UDP packets, and TCP connections, such as the TCP SYN flood attack previously described. Such attacks can be successful provided a system has a vulnerability with the protocol or type of traffic that is exploited.

Attackers can carry the DoS idea even further by enlisting many other systems to participate. To do this, the attacker sets up a master control computer somewhere on the Internet. Next, many computers must first be infected with malicious code or malware by leveraging vulnerabilities present in those machines. Each machine then silently becomes a "bot," appearing to operate normally, while awaiting commands from the master control. When the time comes for an attack to begin, the master control sends a command to every bot and tells it to initiate a denial-of-service attack against a single target host. This is called a **distributed denial-of-service (DDoS) attack** because the attack is distributed across a large number of bots, all flooding or attacking the same target.

## Reflection and Amplification Attacks

Recall that in a spoofing attack, the attacker sends packets with a spoofed source address to a target. The goal is to force the target to deal with the spoofed traffic and send return traffic toward a nonexistent source. The attacker does not care where the return traffic goes or that it cannot be delivered successfully.

In a somewhat related attack, the attacker again sends packets with a spoofed source address toward a live host. However, the host is not the intended target; the goal is to get the host to reflect the exchange toward the spoofed address that is the target. This is known as a **reflection attack**, as illustrated in Figure 9-6, and the host reflecting the traffic toward the target is called the reflector. The attacker might also send the spoofed packets to multiple reflectors, causing the target to receive multiple copies of the unexpected traffic.

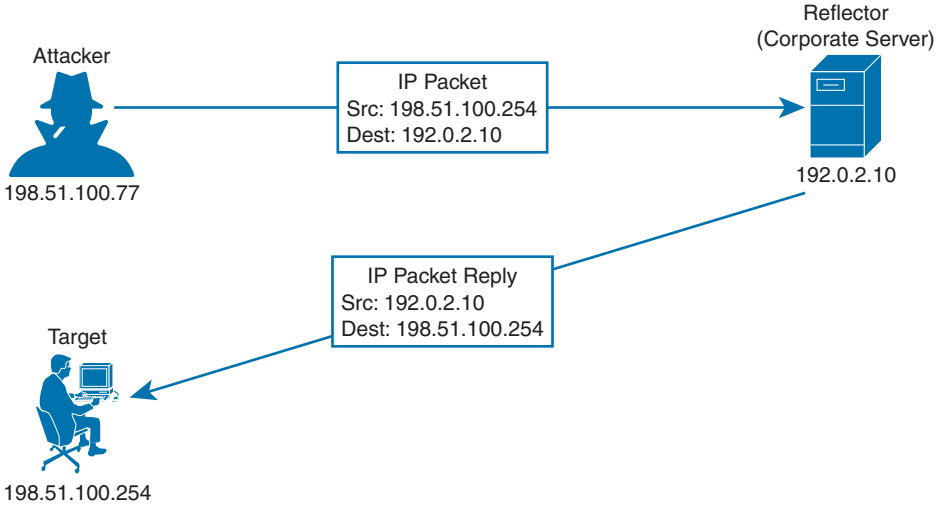



**Figure 9-6** *A Sample Reflection Attack*

The impact of a reflection attack might seem limited because a single target host is the victim, and the amount of traffic being reflected to the target is in proportion to the packets sent by the attacker. If an attacker sends a small amount of traffic to a reflector and leverages a protocol or service to generate a large volume of traffic toward a target, then an **amplification attack** has occurred. In effect, such an attack amplifies the attacker's efforts to disrupt the target. Another result is that large amounts of network bandwidth can be consumed forwarding the amplified traffic toward the target, especially if many reflectors are involved. Some mechanisms of DNS and NTP have been exploited in the past to set new records for enormous bandwidth consumption during an amplification attack.

## Man-in-the-Middle Attacks

Many types of attacks are meant to disrupt or directly compromise targeted systems, often with noticeable results. Sometimes an attacker might want to eavesdrop on data that passes from one machine to another, avoiding detection. A **man-in-the-middle attack** does just that, by allowing the attacker to quietly wedge itself into the communication path as an intermediary between two target systems.

One type of man-in-the-middle attack exploits the ARP table that each host maintains to communicate with other hosts on its local network segment. Normally, if one host needs to send data to another, it looks for the destination host in its ARP table. If an entry is found, the Ethernet frame can be sent directly to the destination MAC address; otherwise, the sender must broadcast an ARP request containing the destination's IP address and wait for the destination to answer with an ARP reply and its own MAC address.

Figure 9-7 illustrates a successful man-in-the-middle attack.

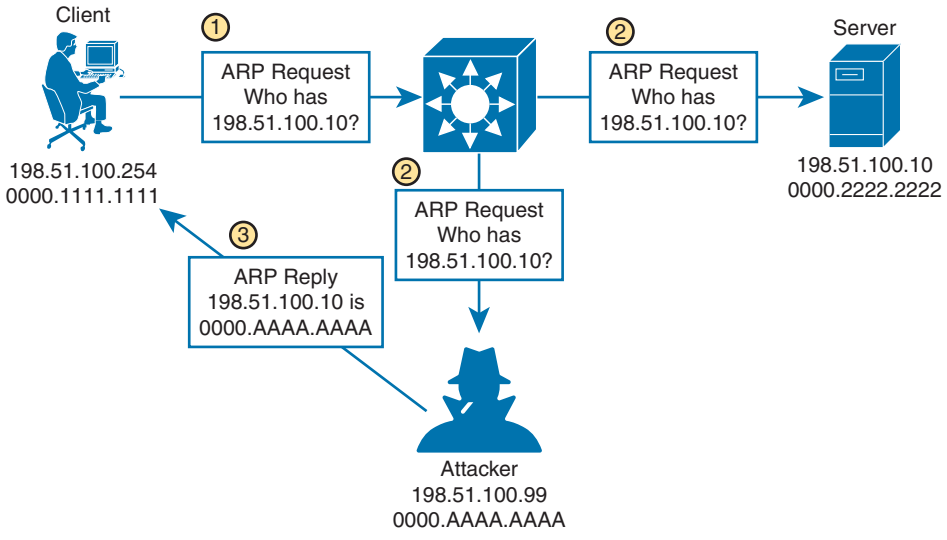



**Figure 9-7** *A Man-in-the-Middle Attack Begins*

In step 1, a client broadcasts an ARP request to find out what MAC address is used by the host with IP address 198.51.100.10. In step 2, the ARP request is flooded to all hosts in the broadcast domain. This allows the attacker to overhear the ARP request and prepare to exploit the information learned. The legitimate owner of 198.51.100.10 may indeed respond with its own ARP reply and real MAC address, as expected. However, in step 3, the attacker simply waits a brief time and then sends a spoofed ARP reply containing its own MAC address, rather than that of the actual destination. The goal is for the attacker to send the last ARP reply so that any listening host will update its ARP table with the most recent information.

This process effectively poisons the ARP table entry in any system receiving the spoofed ARP reply. From that point on, a poisoned system will blindly forward traffic to the attacker's MAC address, who is now masquerading as the destination. The attacker is able to know the real destination's MAC address because the attacker received an earlier ARP reply from the destination host. Figure 9-8 depicts the end result. The attacker can repeat this process by poisoning the ARP entries on multiple hosts and then relay traffic between them without easy detection.

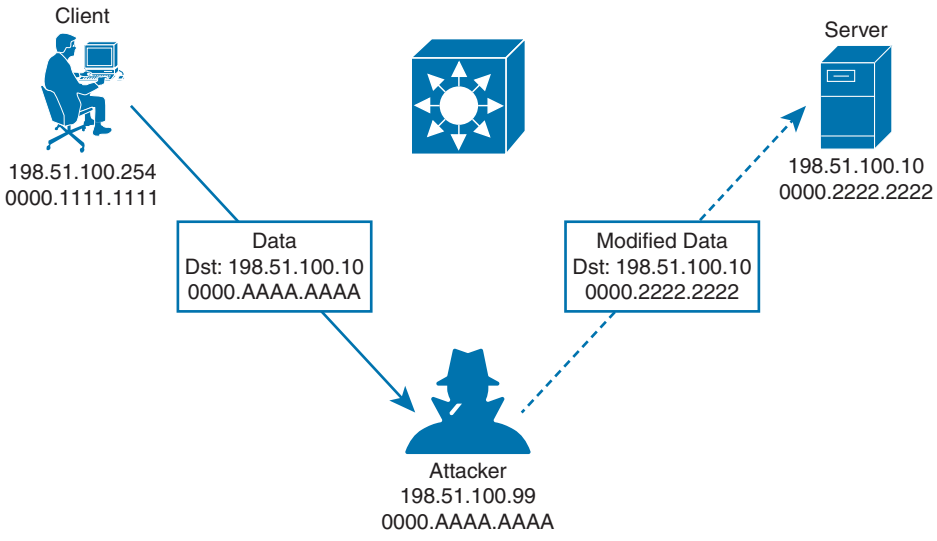



**Figure 9-8** *A Man-in-the-Middle Attack Succeeds*

After attackers have inserted themselves between two hosts, they can passively eavesdrop on and inspect all traffic passing between them. The attackers might also take an active role and modify the data passing through.

### Address Spoofing Attack Summary

As you work through the various types of address spoofing attacks, remember that the attackers' goal is to disguise their identity and fool other systems in a malicious way. Use Table 9-2 to review the concepts and characteristics of each attack type.

**Table 9-2** Summary of Address Spoofing Attacks

| Goal | DoS/DDoS | Reflection | Amplification | Man-in-the-Middle |
|---|---|---|---|---|
| Exhaust a system service or resource; crash the target system | Yes | Yes | Yes | No |
| Trick an unwitting accomplice host to send traffic to target | No | Yes | Yes | No |
| Eavesdrop on traffic | No | No | No | Yes |
| Modify traffic passing through | No | No | No | Yes |

## Reconnaissance Attacks

When an attacker intends to launch an attack on a target, that attacker might want to identify some vulnerabilities so the attack can be focused and more effective. A **reconnaissance attack** can be used to discover more details about the target and its systems prior to an actual attack.

During a reconnaissance attack, the attacker can use some common tools to uncover public details like who owns a domain and what IP address ranges are used there. For example, the **nslookup** command exists in many operating systems and can perform a DNS lookup to resolve an IP address from a fully qualified domain name. If an attacker knows the domain name of a business, nslookup can reveal the owner of the domain and the IP address space registered to it. The **whois** and **dig** commands are complementary tools that can query DNS information to reveal detailed information about domain owners, contact information, mail servers, authoritative name servers, and so on.

Then the attacker can progress to using ping sweeps to send pings to each IP address in the target range. Hosts that answer the ping sweep then become live targets. Port scanning tools can then sweep through a range of UDP and TCP ports to see if a target host answers on any port numbers. Any replies indicate that a corresponding service is running on the target host.

Keep in mind that a reconnaissance attack is not a true attack because nothing is exploited as a result. It is used for gathering information about target systems and services so that vulnerabilities can be discovered and exploited using other types of attacks.

## Buffer Overflow Attacks

Operating systems and applications normally read and write data using buffers and temporary memory space. Buffers are also important when one system communicates with another, as IP packets and Ethernet frames come and go. As long as the memory space is maintained properly and data is placed within the correct buffer boundaries, everything should work as expected.

However, some systems and applications have vulnerabilities that can allow buffers to overflow. This means some incoming data might be stored in unexpected memory locations if a buffer is allowed to fill beyond its limit. An attacker can exploit this condition by sending data that is larger than expected, in what is called a **buffer overflow attack**. If a vulnerability exists, the target system might store that data, overflowing its buffer into another area of memory, eventually crashing a service or the entire system. The attacker might also be able to specially craft the large message by inserting malicious code in it. If the target system stores that data as a result of a buffer overflow, then it can potentially run the malicious code without realizing.

## Malware

Some types of security threats can come in the form of malicious software or **malware**. For example, a **Trojan horse** is malicious software that is hidden and packaged inside other software that looks normal and legitimate. If a well-meaning user decides to install it, the Trojan horse software is silently installed too. Then the malware can run attacks of its own on the local system or against other systems. Trojan horse malware can spread from one computer to another only through user interaction such as opening email attachments, downloading software from the Internet, and inserting a USB drive into a computer.

In contrast, **viruses** are malware that can propagate between systems more readily. To spread, virus software must inject itself into another application, then rely on users to transport the infected application software to other victims.

One other type of malware is able to propagate to and infect other systems on its own. An attacker develops **worm** software and deposits it on a system. From that point on, the worm

replicates itself and spreads to other systems through their vulnerabilities, then replicates and spreads again and again.

To summarize, Table 9-3 lists the key ideas behind each type of malware described in this section.

Key Topic

**Table 9-3** Summary of Malware Types

| Characteristic | Trojan Horse | Virus | Worm |
|---|---|---|---|
| Packaged inside other software | Yes | No | No |
| Self-injected into other software | No | Yes | No |
| Propagates automatically | No | No | Yes |

## Human Vulnerabilities

Many types of attacks must take advantage of a vulnerability in an operating system, service, or other types of application software. In other words, an attacker or the malware involved must find a weakness in the target computer system. There are still many other attacks that can succeed by exploiting weaknesses in the humans who use computer systems.

One rather straightforward attack is called **social engineering**, where human trust and social behaviors can become security vulnerabilities. For example, an attacker might pose as an IT staff member and attempt to contact actual end users through phone calls, emails, and social media. The end goal might be to convince the users to reveal their credentials or set their passwords to a "temporary" value due to some fictitious IT maintenance that will take place, allowing the attacker to gain easy access to secure systems. Attackers might also be physically present and secretly observe users as they enter their credentials.

**Phishing** is a technique that attackers use to lure victims into visiting malicious websites. The idea is to either disguise the invitation as something legitimate, frighten victims into following a link, or otherwise deceive users into browsing content that convinces them to enter their confidential information.

Phishing comes in many forms. For instance, **spear phishing** begins with research to discover facts about one person or people in a specific group using social media. The attacker then forms a message for that person or group that sounds much more convincing because of the facts dropped into the message. **Whaling** uses spear phishing but targets high-profile individuals in corporations, governments, and organizations. Phishing can also occur over traditional communications, such as voice calls (*vishing*) and SMS text messages (*smishing*).

**Pharming** also attempts to send victims to a malicious website, but it takes a more drastic approach. Rather than enticing victims to follow a disguised link, pharming involves compromising the services that direct users toward a well-known or trusted website. For instance, an attacker can compromise a DNS service or edit local hosts files to change the entry for a legitimate site. When a victim tries to visit the site using its actual link, the altered name resolution returns the address of a malicious site instead.

In a **watering hole attack**, an attacker determines which users frequently visit a site; then that site is compromised and malware is deposited there. The malware infects only the target users who visit the site, while leaving other users unscathed.

You can refer to Table 9-4 to review the key ideas behind each type of human vulnerability that is commonly exploited.

Key Topic

**Table 9-4** Summary of Human Security Vulnerabilities

| Attack Type | Goal |
|---|---|
| Social engineering | Exploits human trust and social behavior |
| Phishing | Disguises a malicious invitation as something legitimate |
| Spear phishing | Targets one person or a small group with targeted messaging based on prior research |
| Whaling | Directs spear phishing techniques toward high-profile individuals |
| Vishing | Uses voice calls |
| Smishing | Uses SMS text messages |
| Pharming | Uses legitimate services to send users to a compromised site |
| Watering hole | Targets specific victims who visit a compromised site |

## Password Vulnerabilities

Key Topic

Most systems in an enterprise network use some form of authentication to grant or deny user access. When users access a system, a username and password are usually involved. It might be fairly easy to guess someone's username based on that person's real name. If the user's password is set to some default value or to a word or text string that is easy to guess, an attacker might easily gain access to the system too.

Think like an attacker for a moment and see if you can make some guesses about passwords you might try if you wanted to log in to a random system. Perhaps you thought of passwords like *password*, *password123*, *123456*, and so on. Perhaps you could try username *admin* and password *admin*.

An attacker can launch an online attack by actually entering each **password guess** as the system prompts for user credentials. In contrast, an offline attack occurs when the attacker is able to retrieve the encrypted or hashed passwords ahead of time, then goes offline to an external computer and uses software there to repeatedly attempt to recover the actual password.

Attackers can also use software to perform **dictionary attacks** to discover a user's password. The software will automatically attempt to log in with passwords taken from a dictionary or word list. It might have to go through thousands or millions of attempts before discovering the real password. In addition, the software can perform a **brute-force attack** by trying every possible combination of letter, number, and symbol strings. Brute-force attacks require very powerful computing resources and a large amount of time.

To mitigate password attacks, an enterprise should implement password policies for all users. Such a policy might include guidelines that require a long password string made up of a combination of upper- and lowercase characters along with numbers and some special characters. The goal is to require all passwords to be complex strings that are difficult to guess or reveal by a password attack. As well, password management should require all passwords to be changed periodically so that even lengthy brute-force attacks would not be able to recover a password before it is changed again.

### Password Alternatives

A simple password string is the single factor that a user must enter to be authenticated. Because a password should be remembered and not written down anywhere, you might

think of your password as "something you know." Hopefully nobody else knows it too; otherwise, they could use it to impersonate you when authenticating.

An enterprise might also consider using alternative credentials that bring more complexity and more security. Multifactor credentials require users to provide values or factors that come from different sources, reducing the chance that an attacker might possess all of the factors. An old saying describes two-factor credentials as "something you have" (a dynamic changing cryptographic key or a text message containing a time-limited code) and "something you know" (a password).

A digital certificate can serve as one alternative factor because it serves as a trusted form of identification, adheres to a standardized format, and contains encrypted information. If an enterprise supports certificate use, then a user must request and be granted a unique certificate to use for specific purposes. For example, certificates used for authenticating users must be approved for authentication. To be trusted, certificates must be granted and digitally signed by a trusted certificate authority (CA). As long as the services used by the enterprise know and trust the CA, then individual certificates signed by that CA can be trusted as well.

Digital certificates are also time sensitive, as each is approved for a specific time range. After a certificate expires, any attempts to authenticate with it will be rejected. The user who possesses the certificate can request a new one prior to the expiration date or at any time afterward. Certificates can also be revoked, if the business decides to revoke privileges from a user, if the user separates from the business, and so on. Even if the user still possesses a revoked certificate, that user will be refused access when trying to authenticate with it.

Because digital certificates exist as files on a computer or device, you might think they can be freely copied and used to identify people other than the original owners. Each digital certificate must also carry proof of possession to show that it was truly granted to the user who presents it during authentication. This proof is built into the encrypted certificate content, as a result of combining public keys that the user's machine and the authentication server can publicly share, along with private keys that each party keeps private and secret. As long as the authentication server can verify that the certificate was created using the correct public and private keys, then the certificate must be possessed by the expected owner. If not, then authentication will be rejected to keep an imposter out.

Biometric credentials carry the scheme even further by providing a factor that represents "something you are." The idea is to use some physical attribute from a user's body to uniquely identify that person. Physical attributes are usually unique to each individual's body structure and cannot be easily stolen or duplicated. For example, a user's fingerprint can be scanned and used as an authentication factor. Other examples include face recognition, palm prints, voice recognition, iris recognition, and retinal scans. As you might expect, some methods can be trusted more than others. Sometimes facial recognition systems can be fooled when presented by photographs or masks of trusted individuals. Injuries and the aging process can also alter biometric patterns such as fingerprints, facial shapes, and iris patterns. To help mitigate potential weaknesses, multiple biometric credentials can be collected and used to authenticate users as well.

To summarize, Table 9-5 lists the key ideas used in each alternative to password authentication.

**Table 9-5** Summary of Password Authentication and Alternatives

| Characteristic | Password Only | Two-Factor | Digital Certificates | Biometric |
|---|---|---|---|---|
| Something you know | Yes | Yes | | |
| Something you have | | Yes | Yes | |
| Something you are | | | | Yes |

## Controlling and Monitoring User Access

You can manage user activity to and through systems with authentication, authorization, and accounting (**AAA**, also pronounced "triple-A") mechanisms. AAA uses standardized methods to challenge users for their credentials before access is allowed or authorized. Accounting protocols also can record user activity on enterprise systems. AAA is commonly used to control and monitor access to network devices like routers, switches, firewalls, and so on.

Key Topic

In a nutshell, you can think of AAA in the following manner:

- **Authentication:** Who is the user?
- **Authorization:** What is the user allowed to do?
- **Accounting:** What did the user do?

AAA begins with centralized management of login credentials used for authentication. Authentication refers to confirming the user is who they claim to be. The user can supply information (username and password) or use something they have (like responding to a text from their phone).

Using a centralized authentication process has many advantages over the distributed configuration options found in network device configuration. For instance, users should update passwords from time to time. Companies that rely on per-device password configuration on thousands of networking devices seldom systematically update those passwords. A centralized AAA system would solve that problem.

The second A in AAA, authorization, defines the capabilities allowed for the user. For example, on routers and switches, you should be familiar with user mode, reached via simple login, and privileged mode, reached with the **enable** command. In effect, those modes act as two authorization levels. IOS supports additional authorization levels you can configure, defining different commands allowed at each level.

As an example, user mode does not allow the **show running-config** or **configure terminal** commands. With IOS authorization features, you could define a new level that includes all user mode commands plus the **show running-config** command, which lets the user see the configuration. However, you may not want to allow the **configure terminal** command, which would let the user change the configuration. That authorization level can be associated with the user as identified during the authentication process.

Accounting, the third A in AAA, defines the need to record information about user activity. For network device users, that often comes in the form of log messages (as discussed further

in Chapter 13, "Device Management Protocols.") Devices generate information messages—log messages—when events of note occur. Some log messages track user actions, like configuring the device or reloading it.

IT departments used centralized AAA services for end users as well as for network device access. Cisco's Identity Services Engine (ISE) product implements authentication and authorization for network-focused activities. It uses the following two protocols to communicate with enterprise resources:

- **TACACS+:** A Cisco proprietary protocol that separates each of the AAA functions. Communication is secure and encrypted over TCP port 49.
- **RADIUS:** A standards-based protocol that combines authentication and authorization into a single resource. Communication uses UDP ports 1812 and 1813 (accounting) but is not completely encrypted.

Ignoring the differences for now, you can think of these protocols as the basis for exchanging user credentials between the user and the AAA server. Both TACACS+ and RADIUS are arranged as a client/server model, where an authenticating device acts as a client talking to a AAA server. Figure 9-9 shows a simplified view of the process, where a user is attempting to connect to a switch for management purposes. In the AAA client role, the switch is often called Network Access Device (NAD) or Network Access Server (NAS). When a user tries to connect to the switch, the switch challenges the user for credentials, then passes the credentials along to the AAA server. In simple terms, if the user passes authentication, the AAA server returns an "accept" message to the switch. If the AAA server requires additional credentials, as in **multifactor authentication**, it returns a "challenge" message to the switch. Otherwise, a "reject" message is returned, denying access to the user.

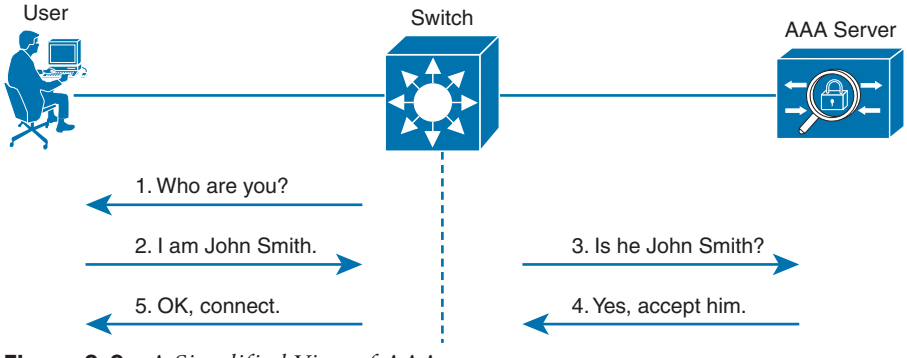



**Figure 9-9** *A Simplified View of AAA*

Comparing RADIUS and TACACS+, RADIUS tends to be used more often for end-user AAA services, whereas TACACS+ (created by Cisco) is used more often to protect networking devices. TACACS+ can authorize the specific CLI commands allowed by a user, whereas RADIUS does not, making it useful for AAA with devices that have a CLI. Table 9-6 summarizes the differences between the two protocols.

Key Topic

**Table 9-6** Comparisons Between TACACS+ and RADIUS

| Features | TACACS+ | RADIUS |
|---|---|---|
| Most often used for | Network devices | Users |
| Transport protocol | TCP | UDP |
| Authentication port number(s) | 49 | 1645, 1812 |
| Protocol encrypts the password | Yes | Yes |
| Protocol encrypts entire packet | Yes | No |
| Supports function to authorize each user to a subset of CLI commands | Yes | No |
| Defined by | Cisco | RFC 2865 |

## Developing a Security Program to Educate Users

One effective approach an enterprise can take to improve information security is to educate its user community through a corporate security program. Most users may not have an IT background, so they might not recognize vulnerabilities or realize the consequences of their own actions. For example, if corporate users receive an email message that contains a message concerning a legal warrant for their arrest or a threat to expose some supposed illegal behavior, they might be tempted to follow a link to a malicious site. Such an action might infect a user's computer and then open a back door or introduce malware or a worm that could then impact the business operations.

Key Topic

An effective security program should have the following basic elements:

- **User awareness:** All users should be made aware of the need for data confidentiality to protect corporate information, as well as their own credentials and personal information. They should also be made aware of potential threats, schemes to mislead, and proper procedures to report security incidents. Users should also be instructed to follow strict guidelines regarding data loss. For example, users should not include sensitive information in emails or attachments, should not keep or transmit that information from a smartphone, or store it on cloud services or removable storage drives.
- **User training:** All users should be required to participate in periodic formal training so that they become familiar with all corporate security policies. (This also implies that the enterprise should develop and publish formal security policies for its employees, users, and business partners to follow.)
- **Physical access control:** Infrastructure locations, such as network closets and data centers, should remain securely locked. Badge access to sensitive locations is a scalable solution, offering an audit trail of identities and timestamps when access is granted. Administrators can control access on a granular basis and quickly remove access when an employee is dismissed.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 9-7 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 9-7** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Key Topic

**Table 9-8** Key Topics for Chapter 9

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 9-3 | Security terminology | 187 |
| Table 9-3 | Types of malware | 195 |
| Table 9-4 | Human security vulnerabilities | 196 |
| Paragraph | Password vulnerabilities | 196 |
| List | AAA functions | 198 |
| Table 9-6 | TACACS+ and RADIUS compared | 200 |
| List | User education | 200 |

## Key Terms You Should Know

AAA, amplification attack, brute-force attack, buffer overflow attack, denial-of-service (DoS) attack, dictionary attack, distributed denial-of-service (DDoS) attack, exploit, malware, man-in-the-middle attack, mitigation technique, multifactor authentication, password guessing, pharming, phishing, reconnaissance attack, reflection attack, social engineering, spear phishing, spoofing attack, threat, Trojan horse, virus, vulnerability, watering hole attack, whaling, worm



CHAPTER 10

# Securing Network Devices

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.c Next-generation firewalls and IPS**

**4.0 IP Services**

**4.8 Configure network devices for remote access using SSH**

**5.0 Security Fundamentals**

**5.3 Configure and verify device access control using local passwords**

All devices in the network—endpoints, servers, and infrastructure devices like routers and switches—include some methods for the devices to legitimately communicate using the network. To protect those devices, the security plan will include a wide variety of tools and mitigation techniques, with the chapters in Part III of this book discussing a large variety of those tools and techniques.

This chapter focuses on two particular security needs in an enterprise network. First, access to the CLI of the network devices needs to be protected. The network engineering team needs to be able to access the devices remotely, so the devices need to allow remote SSH (and possibly Telnet) access. The first half of this chapter discusses how to configure passwords to keep them safe and how to filter login attempts at the devices themselves.

The second half of the chapter turns to two different security functions most often implemented with purpose-built appliances: firewalls and IPSs. These devices together monitor traffic in transit to determine if the traffic is legitimate or if it might be part of some exploit. If considered to be part of an exploit, or if contrary to the rules defined by the devices, they can discard the messages, stopping any attack before it gets started.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 10-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Securing IOS Passwords | 1–4 |
| Firewalls and Intrusion Prevention Systems | 5, 6 |

1. Imagine that you have configured the **enable secret** command, followed by the **enable password** command, from the console. You log out of the switch and log back in at the console. Which command defines the password that you had to enter to access privileged mode?
   a. **enable password**
   b. **enable secret**
   c. Neither
   d. The **password** command, if it's configured
2. Some IOS commands store passwords as clear text, but you can then encrypt the passwords with the **service password-encryption** global command. By comparison, other commands store a computed hash of the password instead of storing the password. Comparing the two options, which one answer is the *most accurate* about why one method is better than the other?
   a. Using hashes is preferred because encrypted IOS passwords can be easily decrypted.
   b. Using hashes is preferred because of the large CPU effort required for encryption.
   c. Using encryption is preferred because it provides stronger password protection.
   d. Using encryption is preferred because of the large CPU effort required for hashes.
3. A network engineer issues a **show running-config** command and sees only one line of output that mentions the **enable secret** command, as follows:

   ```
   enable secret 5 $1$ZGMA$e8cmvkz4UjiJhVp7.maLE1
   ```

   Which of the following is true about users of this router?
   a. A user must type **$1$ZGMA$e8cmvkz4UjiJhVp7.maLE1** to reach enable mode.
   b. The router will hash the clear-text password that the user types to compare to the hashed password.
   c. A **no service password-encryption** configuration command would decrypt this password.
   d. The router will decrypt the password in the configuration to compare to the clear-text password typed by the user.
4. The **show running-config** command output on a router includes the following line: **username test05 secret 8 $8$rTJqzmkwdI20WU$.mktApC8shjjwgABbQp7UjOmttmJaiIDfvBBJOpcns6**. Which answer best describes the command the network engineer used to configure this **username** command with its clear-text password?
   a. **username test05 algorithm-type scrypt secret cisco**
   b. **username test05 algorithm-type sha256 secret cisco**

**c.** **username test05 algorithm-type md5 secret cisco**

**d.** **username test05 secret cisco**

5. A next-generation firewall sits at the edge of a company's connection to the Internet. It has been configured to prevent Telnet clients residing in the Internet from accessing Telnet servers inside the company. Which of the following might a next-generation firewall use that a traditional firewall would not?

   **a.** Match message destination well-known port 23

   **b.** Match message application data

   **c.** Match message IP protocol 23

   **d.** Match message source TCP ports greater than 49152

6. Which actions show a behavior typically supported by a Cisco next-generation IPS (NGIPS) beyond the capabilities of a traditional IPS? (Choose two answers.)

   **a.** Gather and use host-based information for context

   **b.** Comparisons between messages and a database of exploit signatures

   **c.** Logging events for later review by the security team

   **d.** Filter URIs using reputation scores

# Foundation Topics

## Securing IOS Passwords

The ultimate way to protect passwords in Cisco IOS devices is to not store passwords in IOS devices. That is, for any functions that can use an external authentication, authorization, and accounting (AAA) server, use it. However, it is common to store some passwords in a router or switch configuration, and this first section of the chapter discusses some of the ways to protect those passwords.

As a brief review, Figure 10-1 summarizes some typical login security configuration on a router or switch. On the lower left, you see Telnet support configured, with the use of a password only (no username required). On the right, the configuration adds support for login with both username and password, supporting both Telnet and SSH users. The upper left shows the one command required to define an enable password in a secure manner.

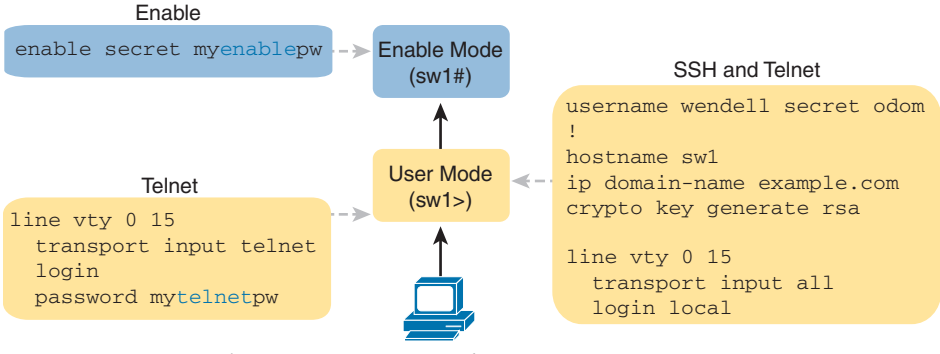



**Figure 10-1** *Sample Login Security Configuration*

**NOTE** The configuration on the far right of the figure supports both SSH and Telnet, but consider allowing SSH only by instead using the **transport input ssh** command. The Telnet protocol sends all data unencrypted, so any attacker who copies the message with a Telnet login will have a copy of the password.

The rest of this first section discusses how to make these passwords secure. In particular, this section looks at ways to avoid keeping clear-text passwords in the configuration and storing the passwords in ways that make it difficult for attackers to learn the password.

## Encrypting Older IOS Passwords with service password-encryption

Some older-style IOS passwords create a security exposure because the passwords exist in the configuration file as clear text. These clear-text passwords might be seen in printed versions of the configuration files, in a backup copy of the configuration file stored on a server, or as displayed on a network engineer's display.

Cisco attempted to solve this clear-text problem by adding a command to encrypt those passwords: the **service password-encryption** global configuration command. This command encrypts passwords that are normally held as clear text, specifically the passwords for these commands:

Key Topic

**password** *password* (console or vty mode)

**username** *name* **password** *password* (global)

**enable password** *password* (global)

To see how it works, Example 10-1 shows how the **service password-encryption** command encrypts the clear-text console password. The example uses the **show running-config | section line con 0** command both before and after the encryption; this command lists only the section of the configuration about the console.

**Example 10-1** *Encryption and the* **service password-encryption** *Command*

```
Switch3# show running-config | section line con 0
line con 0
 password cisco
 login

Switch3# configure terminal
Enter configuration commands, one per line.   End with CNTL/Z.
Switch3(config)# service password-encryption
Switch3(config)# ^Z

Switch3# show running-config | section line con 0
line con 0
 password 7 070C285F4D06
 login
```

A close examination of the before and after **show running-config** command output reveals both the obvious effect and a new concept. The encryption process now hides the original

clear-text password. Also, IOS needs a way to signal that the value in the **password** command lists an encrypted password rather than the clear text. IOS adds the encryption or encoding type of "7" to the command, which specifically refers to passwords encrypted with the **service password-encryption** command. (IOS considers the clear-text passwords to be type 0; some commands list the 0, and some do not.)

While the **service password-encryption** global command encrypts passwords, the **no service password-encryption** global command does not immediately decrypt the passwords back to their clear-text state. Instead, the process works as shown in Figure 10-2. Basically, after you enter the **no service password-encryption** command, the passwords remain encrypted until you change a password.

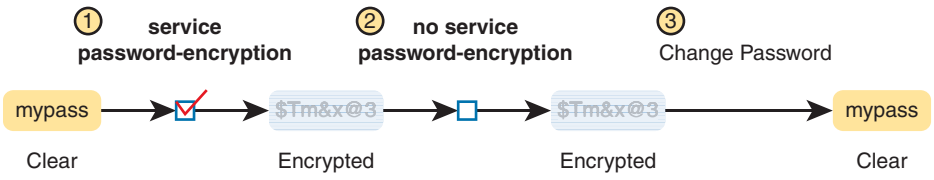



**Figure 10-2** *Encryption Is Immediate; Decryption Awaits Next Password Change*

Unfortunately, the **service password-encryption** command does not protect the passwords very well. Armed with the encrypted value, you can search the Internet and find sites with tools to decrypt these passwords. In fact, you can take the encrypted password from this example, plug it into one of these sites, and it decrypts to "cisco." So, the **service password-encryption** command will slow down the curious, but it will not stop a knowledgeable attacker.

## Encoding the Enable Passwords with Hashes

In the earliest days of IOS, Cisco used the **enable password** *password* global command to define the password that users had to use to reach enable mode (after using the **enable** EXEC command). However, as just noted, the **enable password** *password* command stored the password as clear text, and the **service password-encryption** command encrypted the password in a way that was easily decrypted.

Cisco solved the problem of only weak ways to store the password of the **enable password** *password* global command by making a more secure replacement: the **enable secret** *password* global command. However, both of these commands exist in IOS even today. The next few pages look at these two commands from a couple of angles, including interactions between these two commands, why the **enable secret** command is more secure, along with a note about some advancements in how IOS secures the **enable secret** password.

### Interactions Between Enable Password and Enable Secret

First, for real life: use the **enable secret** *password* global command, and ignore the **enable password** *password* global command. That has been true for around 20 years.

However, to be complete, Cisco has never removed the much weaker **enable password** command from IOS. So, on a single switch (or router), you can configure one or the other, both,

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** A **3** B **4** B **5** B **6** A, D

or neither. What, then, does the switch expect you to type as the password to reach enable mode? It boils down to these rules:

Key Topic

**Both commands configured:** Users must use the password in the **enable secret** *password* command (and ignore the **enable password** *password* command).

**Only one command configured:** Use the password in that one command.

**Neither command configured (default):** Console users move directly to enable mode without a password prompt; Telnet and SSH users are rejected with no option to supply an enable password.

### Making the Enable Secret Truly Secret with a Hash

The Cisco **enable secret** command protects the password value by never even storing the clear-text password in the configuration. However, that one sentence may cause you a bit of confusion: If the router or switch does not remember the clear-text password, how can the switch know that the user typed the right password after using the **enable** command? This section works through a few basics to show you how and appreciate why the password's value is secret.

First, by default, IOS uses a hash function called Message Digest 5 (MD5) to store an alternative value in the configuration, rather than the clear-text password. Think of MD5 as a rather complex mathematical one-way formula. This formula is chosen so that even if you know the exact result of the formula—that is, the result after feeding the clear-text password through the formula as input—it is computationally difficult to compute the original clear-text password. Figure 10-3 shows the main ideas:

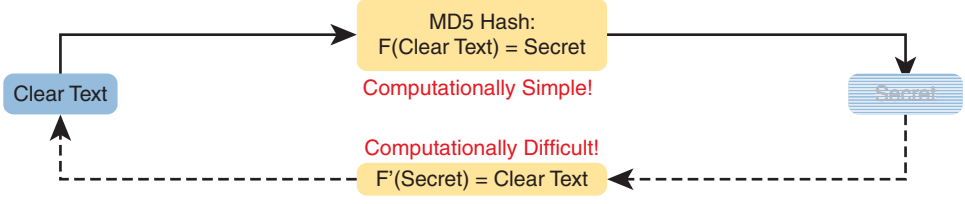



**Figure 10-3** *One-Way Nature of MD5 Hash to Create Secret*

**NOTE** "Computationally difficult" is almost a code phrase, meaning that the designers of the function hope that no one is willing to take the time to compute the original clear text.

So, if the original clear-text password cannot be re-created, how can a switch or router use it to compare to the clear-text password typed by the user? The answer depends on another fact about these security hashes like MD5: each clear-text input results in a unique result from the math formula.

The **enable secret fred** command generates an **MD5 hash**. If a user types **fred** when trying to enter enable mode, IOS will run MD5 against that value and get the same MD5 hash as is listed in the **enable secret** command, so IOS allows the user to access enable mode. If the user typed any other value besides **fred**, IOS would compute a different MD5 hash than the value stored with the **enable secret** command, and IOS would reject that user's attempt to reach enable mode.

Knowing that fact, the switch can make a comparison when a user types a password after using the **enable** EXEC command as follows:

Key Topic

**Step 1.** IOS computes the MD5 hash of the password in the **enable secret** command and stores the hash of the password in the configuration.

**Step 2.** When the user types the **enable** command to reach enable mode, a password that needs to be checked against that configuration command, IOS hashes the clear-text password as typed by the user.

**Step 3.** IOS compares the two hashed values: if they are the same, the user-typed password must be the same as the configured password.

As a result, IOS can store the hash of the password but never store the clear-text password; however, it can still determine whether the user typed the same password.

Switches and routers already use the logic described here, but you can see the evidence by looking at the switch configuration. Example 10-2 shows the creation of the **enable secret** command, with a few related details. This example shows the stored (hashed) value as revealed in the **show running-configuration** command output. That output also shows that IOS changed the **enable secret fred** command to list the encryption type 5 (which means the listed password is actually an MD5 hash of the clear-text password). The gobbledygook long text string is the hash, preventing others from reading the password.

**Example 10-2** *Cisco IOS Encoding Password "fred" as Type 5 (MD5)*

```
Switch3(config)# enable secret fred
Switch3(config)# ^Z
Switch3# show running-config | include enable secret

enable secret 5 $1$ZGMA$e8cmvkz4UjiJhVp7.maLE1

Switch3# configure terminal
Enter configuration commands, one per line.   End with CNTL/Z.
Switch3(config)# no enable secret
Switch3(config)# ^Z
```

The end of the example also shows an important side point about deleting the **enable secret** password: after you are in enable mode, you can delete the **enable secret** password using the **no enable secret** command, without even having to enter the password value. You can also overwrite the old password by just repeating the **enable secret** command. But you cannot view the original clear-text password.

**NOTE** Example 10-2 shows another shortcut illustrating how to work through long **show** command output, this time using the pipe to the **include** command. The **| include enable secret** part of the command processes the output from **show running-config** to include only the lines with the case-sensitive text "enable secret."

### Improved Hashes for Cisco's Enable Secret

The use of any hash function to encode passwords relies on several key features of the particular hash function. In particular, every possible input value must result in a single hashed value, so that when users type a password, only one password value matches each hashed value. Also, the hash algorithm must result in computationally difficult math (in other words, a pain in the neck) to compute the clear-text password based on the hashed value to discourage attackers.

The MD5 hash algorithm has been around 30 years. Over those years, computers have gotten much faster, and researchers have found creative ways to attack the MD5 algorithm, making MD5 less challenging to crack. That is, someone who saw your running configuration would have an easier time re-creating your clear-text secret passwords than in the early years of MD5.

These facts are not meant to say that MD5 is bad, but like many cryptographic functions before MD5, progress has been made, and new functions were needed. To provide more recent options that would create a much greater challenge to attackers, Cisco added two additional hashes in the 2010s, as noted in Figure 10-4.

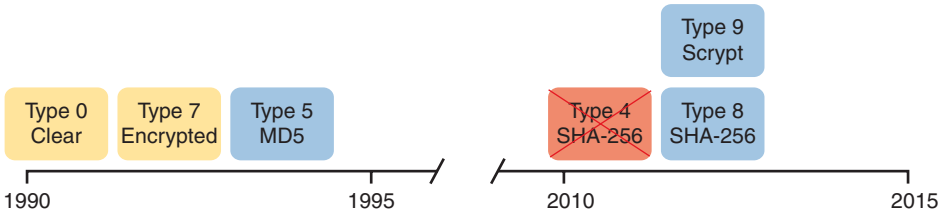



**Figure 10-4** *Timeline of Encryptions/Hashes of Cisco IOS Passwords*

Of the newer types shown in the figure, Cisco continues to use types 8 and 9. Cisco found a security exposure with their type 4 password implementation, so they deprecated it, later replacing it with type 8. Both types 4 and 8 use the same underlying algorithm (PBKDF2) that combines with the SHA-256 algorithm. (You might see both terms mentioned in documentation about type 8 passwords.) Type 9 uses a different hash algorithm called Scrypt (pronounced "es crypt.")

IOS now supports two alternative algorithm types in the more recent router and switch IOS images. Table 10-2 shows the configuration of all three algorithm types on the **enable secret** command.

**Table 10-2** Commands and Encoding Types for the **enable secret** Command

| Command | Type | Algorithm |
|---|---|---|
| **enable algorithm-type md5 secret** *password* | 5 | MD5 |
| **enable algorithm-type sha256 secret** *password* | 8 | SHA-256 |
| **enable algorithm-type scrypt secret** *password* | 9 | Scrypt |

Example 10-3 shows the **enable secret** command being changed from MD5 to the Scrypt algorithm. Of note, the example shows that only one **enable secret** command should exist between those three commands in Table 10-2. Basically, if you configure another **enable**

**secret** command with a different algorithm type, that command replaces any existing **enable secret** command.

**Example 10-3** *Cisco IOS Encoding Password "mypass1" as Type 9 (SHA-256)*

```
R1# show running-config | include enable
enable secret 5 $1$ZSYj$725dBZmLUJ0nx8gFPTtTv0
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# enable algorithm-type scrypt secret mypass1
R1(config)# ^Z
R1#
R1# show running-config | include enable
enable secret 9 $9$II/EeKiRW91uxE$fwYuOE5EHoii16AWv2wSywkLJ/KNeGj8uK/24B0TVU6
R1#
```

Following the process shown in the example, the first command confirms that the current **enable secret** command uses encoding type 5, meaning it uses MD5. Second, the user configures the password using algorithm type scrypt. The last command confirms that only one **enable secret** command exists in the configuration, now with encoding type 9.

Also, think through how IOS takes the command with the clear-text password and then stores the command in a different syntax. In the example, the user issues the **enable algorithm-type scrypt secret mypass** command, referencing the algorithm type by name. However, the **show running-config** output shows how IOS removed the **algorithm-type scrypt** parameters, but it also left the number 9 behind the **secret** parameter. As a result, the command stored by IOS identifies the algorithm type by number. Because of that, take time to memorize both the names and numbers of the algorithm types per Table 10-2.

Finally, note that IOS and IOS XE differ slightly with the **enable secret** command. When configuring without using the **algorithm-type** keyword, IOS defaults to MD5, whereas IOS XE defaults to Scrypt.

## Encoding the Passwords for Local Usernames

The **username password** and **username secret** commands have a similar history to the **enable password** and **enable secret** commands. Originally, IOS supported the **username** *user* **password** *password* command—a command that had those same issues of being a clear-text password or a poorly encrypted value (with the **service password-encryption** feature). Many years later, Cisco added the **username** *user* **secret** *password* global command, which encoded the password as an MD5 hash, with Cisco adding support for the newer hashes later. Note that Cisco uses the term **local username** to refer to these usernames configured on individual routers and switches, in contrast to those configured centrally on a AAA server.

Today, the **username secret** command is preferred over the **username password** command; however, keep these rules in mind for **username** commands:




- IOS allows only one **username** command for a given username—either a **username** *name* **password** *password* command or a **username** *name* **secret** *password* command.

- IOS allows a mix of commands (**username password** and **username secret**) in the same router or switch (for different usernames).

As mentioned, IOS on both switches and routers uses the additional encoding options beyond MD5, just as supported with the **enable secret** command. Table 10-3 shows the syntax of those three options in the **username** command.

**Table 10-3** Commands and Encoding Types for the **username secret** Command

| Command | Type | Algorithm |
|---|---|---|
| **username** *name* **algorithm-type md5 secret** *password* | 5 | MD5 |
| **username** *name* **algorithm-type sha256 secret** *password* | 8 | SHA-256 |
| **username** *name* **algorithm-type scrypt secret** *password* | 9 | SHA-256 |

IOS and IOS XE also differ with the **username secret** command, with the same defaults as for the **enable secret** command: MD5 for IOS and Scrypt for IOS XE. Also, for the **username secret** command, IOS XE does not support the MD5 option at all. (As always, treat any comments about IOS and IOS XE differences for general awareness because the CCNA exam topics do not mention the need to know such distinctions.)

# Firewalls and Intrusion Prevention Systems

The next topic examines the roles of a couple of different kinds of networking devices: firewalls and intrusion prevention systems (IPSs). Both devices work to secure networks but with slightly different goals and approaches.

This second major section of the chapter takes a look at each. This section first discusses the core traditional features of both firewalls and IPSs. The section closes with a description of the newer features in the current generation of these products, called next-generation products, which improve the functions of each.

## Traditional Firewalls

Traditionally, a **firewall** sits in the forwarding path of all packets so that the firewall can then choose which packets to discard and which to allow through. By doing so, the firewall protects the network from different kinds of issues by allowing only the intended types of traffic to flow in and out of the network. In fact, in its most basic form, firewalls do the same kinds of work that routers do with ACLs, but firewalls can perform that packet-filtering function with many more options, as well as perform other security tasks.

Figure 10-5 shows a typical network design for a site that uses a physical firewall. The figure shows a firewall, like the Cisco Adaptive Security Appliance (ASA) firewall, connected to a Cisco router, which in turn connects to the Internet. All enterprise traffic going to or from the Internet would be sent through the firewall. The firewall would consider its rules and make a choice for each packet, whether the packet should be allowed through.

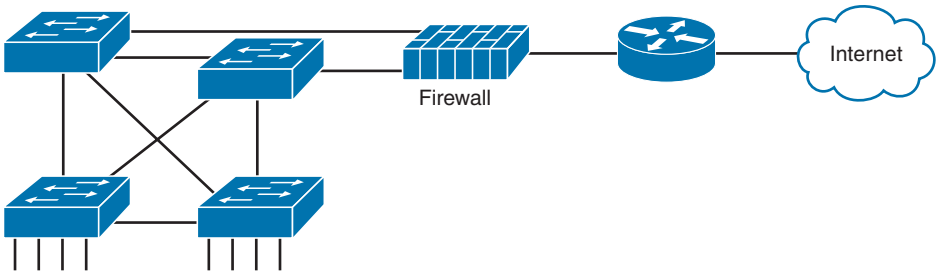



**Figure 10-5** *Firewall as Positioned in the Packet Forwarding Path*

Although firewalls have some router-like features (such as packet forwarding and packet filtering), they provide much more advanced security features than a traditional router. For example, most firewalls can use the following kinds of logic to make the choice of whether to discard or allow a packet:

- Like router IP ACLs, match the source and destination IP addresses
- Like router IP ACLs, identify applications by matching their static well-known TCP and UDP ports
- Watch application-layer flows to know what additional TCP and UDP ports are used by a particular flow, and filter based on those ports
- Match the text in the URI of an HTTP request—that is, look at and compare the contents of what is often called the web address—and match patterns to decide whether to allow or deny the download of the web page identified by that URI
- Keep state information by storing information about each packet, and make decisions about filtering future packets based on the historical state information (called *stateful inspection*, or being a stateful firewall)

The stateful firewall feature provides the means to prevent a variety of attacks and is one of the more obvious differences between the ACL processing of a router versus security filtering by a firewall. Routers must spend as little time as possible processing each packet so that the packets experience little delay passing through the router. The router cannot take the time to gather information about a packet, and then for future packets, consider some saved state information about earlier packets when making a filtering decision. Because they focus on network security, firewalls do save some information about packets and can consider that information for future filtering decisions.

As an example of the benefits of using a stateful firewall, consider a simple denial-of-service (DoS) attack. An attacker can make this type of attack against a web server by using tools that create (or start to create) a large volume of TCP connections to the server. The firewall might allow TCP connections to that server normally, but imagine that the server might typically receive 10 new TCP connections per second under normal conditions and 100 per second at the busiest times. A DoS attack might attempt thousands or more TCP connections per second, driving up CPU and RAM use on the server and eventually overloading the server to the point that it cannot serve legitimate users.

A stateful firewall could be tracking the number of TCP connections per second—that is, recording state information based on earlier packets—including the number of TCP connection requests from each client IP address to each server address. The stateful firewall could notice a large number of TCP connections, check its state information, and then notice that the number of requests is very large from a small number of clients to that particular server, which is typical of some kinds of DoS attacks. The stateful firewall could then start filtering those packets, helping the web server survive the attack, whereas a stateless firewall or a router ACL would not have had the historical state information to realize that a DoS attack was occurring.

## Security Zones

Firewalls not only filter packets but also pay close attention to which host initiates communications. That concept is most obvious with TCP as the transport layer protocol, where the client initiates the TCP connection by sending a TCP segment that sets the SYN bit only (as seen in Figure 5-5 in Chapter 5, "Introduction to TCP/IP Transport and Applications").

Firewalls use logic that considers which host initiated a TCP connection by watching these initial TCP segments. To see the importance of who initiates the connections, think about a typical enterprise network with a connection to the Internet, as shown in Figure 10-6. The company has users inside the company who open web browsers, initiating connections to web servers across the Internet. However, by having a working Internet connection, that same company opens up the possibility that an attacker might try to create a TCP connection to the company's internal web servers used for payroll processing. Of course, the company does not want random Internet users or attackers to be able to connect to their payroll server.

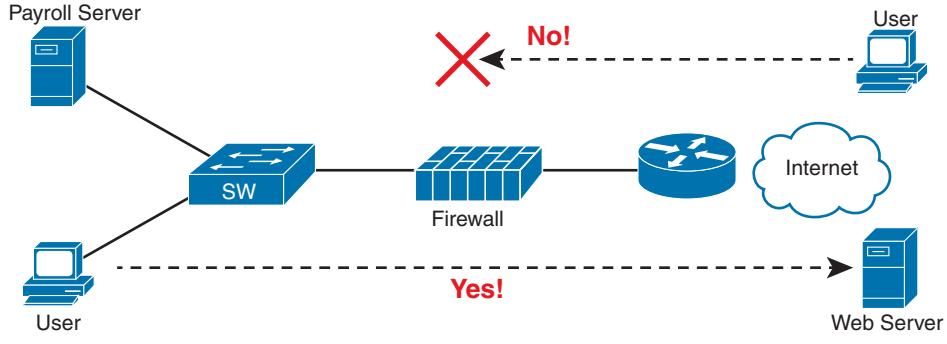



**Figure 10-6** *Allowing Outbound Connections and Preventing Inbound Connections*

Firewalls use the concept of *security zones* (also called a *zone* for short) when defining which hosts can initiate new connections. The firewall has rules, and those rules define which host can initiate connections from one zone to another zone. Also, by using zones, a firewall can place multiple interfaces into the same zone, in cases for which multiple interfaces should have the same security rules applied. Figure 10-7 depicts the idea with the inside part of the enterprise considered to be in a separate zone compared to the interfaces connected toward the Internet.

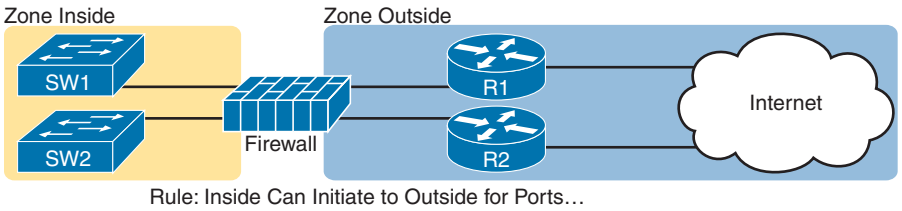



**Figure 10-7** *Using Security Zones with Firewalls*

The most basic firewall rule when using two zones like Figure 10-7 reduces to this logic:

> Allow hosts from zone inside to initiate connections to hosts in zone outside, for a predefined set of safe well-known ports (like HTTP port 80, for instance).

Note that with this one simple rule, the correct traffic is allowed while filtering the unwanted traffic by default. Firewalls typically disallow all traffic unless a rule specifically allows the packet. So, with this simple rule to allow inside users to initiate connections to the outside zone, and that alone, the firewall also prevents outside users from initiating connections to inside hosts.

Most companies have an inside and outside zone, as well as a special zone called the *demilitarized zone (DMZ)*. Although the DMZ name comes from the real world, it has been used in IT for decades to refer to a firewall security zone used to place servers that need to be available for use by users in the public Internet. For example, Figure 10-8 shows a typical Internet edge design, with the addition of a couple of web servers in its DMZ connected through the firewall. The firewall then needs another rule that enables users in the zone outside—that is, users in the Internet—to initiate connections to those web servers in the DMZ. By separating those web servers into the DMZ, away from the rest of the enterprise, the enterprise can prevent Internet users from attempting to connect to the internal devices in the inside zone, preventing many types of attacks.

Key Topic

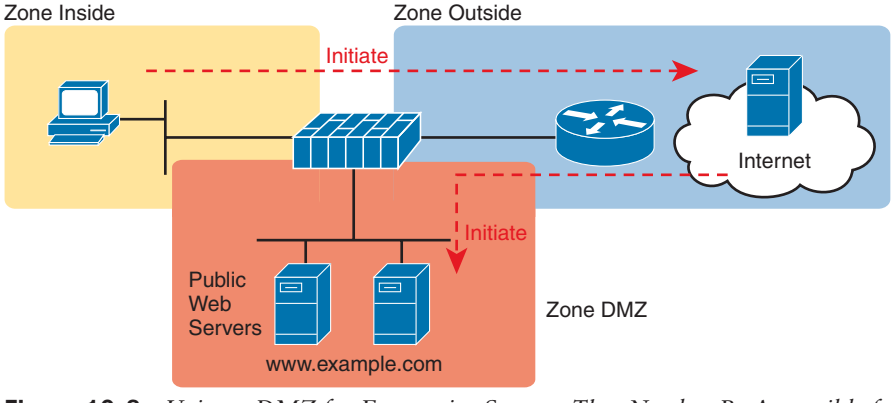



**Figure 10-8** *Using a DMZ for Enterprise Servers That Need to Be Accessible from the Internet*

## Intrusion Prevention Systems (IPS)

Traditionally, a firewall works with a set of user-configured rules about where packets should be allowed to flow in a network. The firewall needs to sit in the path of the packets so it can filter the packets, redirect them for collection and later analysis, or let them continue toward their destination.

A traditional intrusion prevention system (**IPS**) can sit in the path packets take through the network, and it can filter packets, but it makes its decisions with different logic. The IPS first downloads a database of exploit signatures. Each signature defines different header field values found in sequences of packets used by different exploits. Then the IPS can examine packets, compare them to the known exploit signatures, and notice when packets may be part of a known exploit. Once identified, the IPS can log the event, discard packets, or even redirect the packets to another security application for further examination.

A traditional IPS differs from firewalls in that instead of an engineer at the company defining rules for that company based on applications (by port number) and zones, the IPS applies the logic based on signatures supplied mostly by the IPS vendor. Those signatures look for these kinds of attacks:

- DoS
- DDoS
- Worms
- Viruses

To accomplish its mission, the IPS needs to download and keep updating its signature database. Security experts work to create the signatures. The IPS must then download the exploit signature database and keep downloading updates over time, as shown in Figure 10-9.

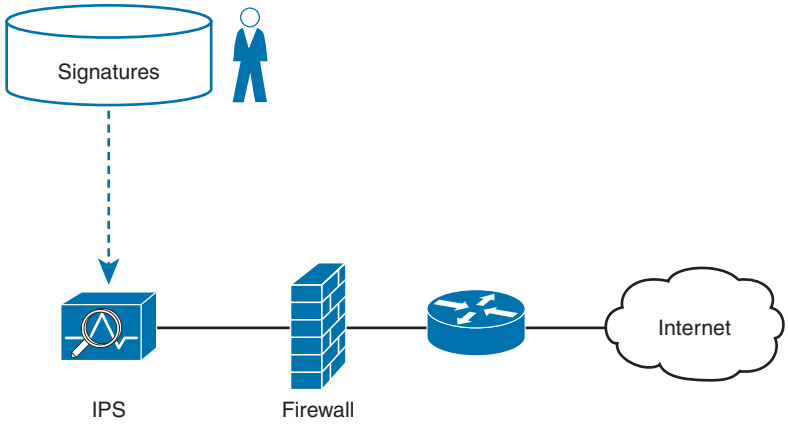



**Figure 10-9** *IPS and Signature Database*

For example, think about what happens when an entirely new computer virus has been created. Host-based security products, like antivirus software, should be installed on the computers inside the company. These tools use a similar model as the IPS, keeping an updated database of virus signatures. The signatures might look for patterns in how a

computer virus could be stored inside files on the computer, or in files sent to the computer via email or web browsers. But there will be some time lag between the day when the virus is discovered (called zero-day attacks) and when researchers develop a virus signature, change their database, and allow time for all the hosts to update their antivirus software. The hosts are at risk during this time lag.

The IPS provides a complimentary service to prevent viruses. Researchers will look for ways an IPS could recognize the same virus while in flight through the network with new IPS signatures—for instance, looking for packets with a particular port and a particular hex string in the application payload. Once developed, the IPS devices in the network need to be updated with the new signature database, protecting against that virus. Both the host-based and IPS-based protections play an important role, but the fact that one IPS protects sections of a network means that the IPS can sometimes more quickly react to new threats to protect hosts.

## Cisco Next-Generation Firewalls

The CCNA 200-301 exam topics mention the terms *firewall* and *IPS* but prefaced with the term *next-generation*. Around the mid-2010s, Cisco and some of their competitors started using the term *next generation* when discussing their security products to emphasize some of the newer features. In short, a **next-generation firewall (NGFW)** and a **next-generation IPS (NGIPS)** are the now-current firewall and IPS products from Cisco.

However, the use of the term *next generation* goes far beyond just a marketing label: the term emphasizes some major shifts and improvements over the years. The security industry sees endless cycles of new attacks followed by new solutions. Some solutions might require entirely new product features or even new products. Some of the changes that have required new security features include the proliferation of mobile devices—devices that leave the enterprise, connect to the Internet, and return to the enterprise—creating a whole new level of risk. Also, no single security function or appliance (firewall, IPS, anti-malware) can hope to stop some threats, so the next-generation tools must be able to work better together to provide solutions. In short, the next-generation products have real useful features not found in their predecessor products.

As for Cisco products, for many years Cisco branded its firewalls as the Cisco Adaptive Security Appliance (ASA). Around 2013, Cisco acquired Sourcefire, a security product company. Many of the next-generation firewall (and IPS) features come from software acquired through that acquisition. For a while, Cisco sold security appliances that ran multiple security features, such as NGFW, with branding similar to the old Sourcefire name: Cisco Firepower firewalls. As of the publication of this book, Cisco had moved on to branding with the name Cisco Secure Firewall (see www.cisco.com/go/firewalls).

An NGFW still does the traditional functions of a firewall, of course, like stateful filtering by comparing fields in the IP, TCP, and UDP headers, and using security zones when defining firewall rules. To provide some insight into some of the newer next-generation features, consider the challenge of matching packets with ports:

1. Each IP-based application should use a well-known port.
2. Attackers know that firewalls will filter most well-known ports from sessions initiated from the outside zone to the inside zone (see Figure 10-8).

3. Attackers use port scanning to find any port that a company's firewall will allow through right now.
4. Attackers attempt to use a protocol of their choosing (for example, HTTP) but with the nonstandard port found through port scanning as a way to attempt to connect to hosts inside the enterprise.

The sequence lists a summary of some of the steps attackers need to take but does not list every single task. However, even to this depth, you can see how attackers can find a way to send packets past the corporate firewall.

The solution? A next-generation firewall that looks at the application layer data to identify the application instead of relying on the TCP and UDP port numbers used. Cisco performs their deep packet inspection using a feature called **Application Visibility and Control (AVC)**. Cisco AVC can identify many applications based on the data sent (application layer headers plus application data structures far past the TCP and UDP headers). When used with a Cisco NGFW, instead of matching port numbers, the firewall matches the application, defeating attacks like the one just described.

The following list mentions a few of the features of an NGFW. Note that while *NGFW* is a useful term, the line between a traditional firewall and a next-generation firewall can be a bit blurry because the terms describe products that have gone through repeated changes over long periods of time. This list does summarize a few of the key points, however:

Key Topic

- **Traditional firewall:** An NGFW performs traditional firewall features, like stateful firewall filtering, NAT/PAT, and VPN termination.
- **Application Visibility and Control (AVC):** This feature looks deep into the application layer data to identify the application. For instance, it can identify the application based on the data, rather than port number, to defend against attacks that use random port numbers.
- **Advanced Malware Protection:** NGFW platforms run multiple security services, not just as a platform to run a separate service, but for better integration of functions. A network-based antimalware function can run on the firewall itself, blocking file transfers that would install malware, and saving copies of files for later analysis.
- **URL Filtering:** This feature examines the URLs in each web request, categorizes the URLs, and either filters or rate-limits the traffic based on rules. The Cisco Talos security group monitors and creates reputation scores for each domain known in the Internet, with URL filtering being able to use those scores in its decision to categorize, filter, or rate limit.
- **NGIPS:** Cisco's NGFW products can also run their NGIPS feature along with the firewall.



Note that for any of the services that benefit from being in the same path that packets traverse, like a firewall, it makes sense that over time those functions could migrate to run on the same product. So, when the design needs both a firewall and IPS at the same location in the network, these NGFW products can run the NGIPS feature as shown in the combined device in Figure 10-10.

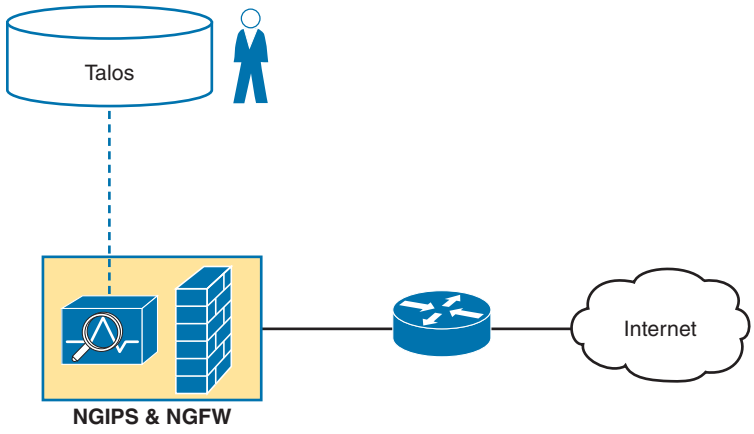



**Figure 10-10** *Next-Generation Firewall with Next-Generation IPS Module*

## Cisco Next-Generation IPS

The Cisco next-generation IPS (NGIPS) products have followed a similar path as the Cisco NGFW products. Cisco first added NGIPS features primarily through its Sourcefire acquisition. Current Cisco IPS products now run as applications in appliances that use the Cisco Secure Firewall name. In fact, as for products, think more like the previous figure, with an appliance that runs both the NGFW and NGIPS.

As with the NGFW, the NGIPS adds features to a traditional IPS. One of the biggest issues with a traditional IPS comes with the volume of security events logged by the IPS. For instance:

1. An IPS compares the signature database, which lists all known exploits, to all messages.
2. It generates events, often far more than the security staff can read.
3. The staff must mentally filter events to find the proverbial needle in the haystack, possible only through hard work, vast experience, and a willingness to dig.

An NGIPS helps with this issue in a couple of ways. First, an NGIPS examines the context by gathering data from all the hosts and the users of those hosts. The NGIPS will know the OS, software revision levels, what apps are running, open ports, the transport protocols and port numbers in use, and so on. Armed with that data, the NGIPS can make much more intelligent choices about what events to log.

For instance, consider an NGIPS placed into a network to protect a campus LAN where end users connect, but no data center exists in that part of the network. Also, all PCs happen to be running Windows, and possibly the same version, by corporate policy. The signature database includes signatures for exploits of Linux hosts, Macs, Windows versions nonexistent in that part of the network, and exploits that apply to server applications that are not running on those hosts. After gathering those facts, an NGIPS can suggest de-emphasizing checks for exploits that do not apply to those endpoints, spending more time and focus on events that could occur, greatly reducing the number of events logged.

The following are a few of the Cisco NGIPS features:

Key Topic

- **Traditional IPS:** An NGIPS performs traditional IPS features, like using exploit signatures to compare packet flows, creating a log of events, and possibly discarding and/or redirecting packets.
- **Application Visibility and Control (AVC):** As with NGFWs, an NGIPS has the ability to look deep into the application layer data to identify the application.
- **Contextual Awareness:** NGFW platforms gather data from hosts—OS, software version/level, patches applied, applications running, open ports, applications currently sending data, and so on. Those facts inform the NGIPS as to the often more limited vulnerabilities in a portion of the network so that the NGIPS can focus on actual vulnerabilities while greatly reducing the number of logged events.
- **Reputation-Based Filtering:** The Cisco Talos security intelligence group researches security threats daily, building the data used by the Cisco security portfolio. Part of that data identifies known bad actors, based on IP address, domain, name, or even specific URL, with a reputation score for each. A Cisco NGIPS can perform reputation-based filtering, taking the scores into account.
- **Event Impact Level:** Security personnel need to assess the logged events, so an NGIPS provides an assessment based on impact levels, with characterizations as to the impact if an event is indeed some kind of attack.

If you want to learn a little more about these topics for your own interest, let me refer you to a couple of resources. First, check out articles and blog posts from the Cisco Talos Intelligence Group (www.talosintelligence.com). The Cisco Talos organization researches security issues around the globe across the entire spectrum of security products. Additionally, one Cisco Press book has some great information about both next-generation firewalls and IPSs, written at a level appropriate as a next step. If you want to read more, check out this book with the long name: *Integrated Security Technologies and Solutions, Volume I: Cisco Security Solutions for Advanced Threat Protection with Next Generation Firewall, Intrusion Prevention, AMP, and Content Security* (or just use its ISBN, 9781587147067), with one chapter each on NGFW and NGIPS.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 10-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 10-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Repeat DIKTA questions | | Book, PTP |
| Do labs | | Blog |
| Review command tables | | Book |

## Review All the Key Topics

Key Topic

**Table 10-5** Key Topics for Chapter 10

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Commands whose passwords are encrypted by **service password-encryption** | 205 |
| List | Rules for when IOS uses the password set with the **enable password** versus **enable secret** commands | 207 |
| List | Logic by which IOS can use the **enable secret** hash when a user types a clear-text password to reach enable mode | 208 |
| List | Rule for combinations of the **username** command | 210 |
| Figure 10-6 | Typical client filtering by firewall at Internet edge | 213 |
| Figure 10-8 | Firewall security zones with DMZ | 214 |
| List | Features of next-generation firewalls | 217 |
| List | Features of next-generation IPSs | 219 |

## Key Terms You Should Know

Application Visibility and Control, enable secret, firewall, IPS, local username, MD5 hash, next-generation firewall (NGFW), next-generation IPS (NGIPS), username secret

## Do Labs

The Sim Lite software is a version of Pearson's full simulator learning product with a subset of the labs, included free with this book. The Sim Lite with this book includes a couple of labs about various password-related topics. Also, check the author's blog site pages for configuration exercises (Config Labs) at https://www.certskills.com/config-labs.

## Command References

Tables 10-6 and 10-7 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 10-6** Chapter 10 Configuration Commands

| Command | Mode/Purpose/Description |
|---|---|
| **line console 0** | Command that changes the context to console configuration mode. |
| **line vty** *1st-vty last-vty* | Command that changes the context to vty configuration mode for the range of vty lines listed in the command. |
| **login** | Console and vty configuration mode. Tells IOS to prompt for a password. |
| **password** *pass-value* | Console and vty configuration mode. Lists the password required if the **login** command is configured. |
| **login local** | Console and vty configuration mode. Tells IOS to prompt for a username and password, to be checked against locally configured **username** global configuration commands. |
| **username** *name* [**algorithm-type md5 \| sha256 \| scrypt**] **secret** *pass-value* | Global command. Defines one of possibly multiple usernames and associated passwords, stored as a hashed value (default MD5), with other hash options as well. |
| **username** *name* **password** *pass-value* | Global command. Defines a username and password, stored in clear text in the configuration by default. |
| **crypto key generate rsa [modulus 512 \| 768 \| 1024]** | Global command. Creates and stores (in a hidden location in flash memory) the keys required by SSH. |
| **transport input** {**telnet \| ssh \| all \| none**} | vty line configuration mode. Defines whether Telnet and/or SSH access is allowed into this switch. |
| **[no] service password-encryption** | Global command that encrypts all clear-text passwords in the running-config. The **no** version of the command disables the encryption of passwords when the password is set. |
| **enable password** *pass-value* | Global command to create the enable password, stored as a clear text instead of a hashed value. |
| **enable [algorithm-type md5 \| sha256 \| scrypt] secret** *pass-value* | Global command to create the enable password, stored as a hashed value instead of clear text, with the hash defined by the algorithm type. |
| **no enable secret**<br>**no enable password** | Global command to delete the **enable secret** or **enable password** commands, respectively. |

**Table 10-7** Chapter 10 EXEC Command Reference

| Command | Purpose |
|---|---|
| **show running-config \| section vty** | Lists the vty lines and subcommands from the configuration. |
| **show running-config \| section con** | Lists the console and subcommands from the configuration. |
| **show running-config \| include enable** | Lists all lines in the configuration with the word *enable*. |



CHAPTER 11

# Implementing Switch Port Security

**This chapter covers the following exam topics:**

**5.0 Security Fundamentals**

**5.7 Configure and verify Layer 2 security features (DHCP snooping, dynamic ARP inspection, and port security)**

In modern networks, security must be implemented in depth. The security architecture should use firewalls and intrusion prevention systems (IPS) at strategic locations, and hosts should use antivirus and antimalware tools. Routers, which already need to exist throughout the enterprise at the edge between local-area networks and wide-area networks, can be configured with IP access control lists to filter packets related to different IP address ranges in that enterprise.

LAN switches have a unique opportunity as a security enforcement point, particularly LAN switches connected to endpoint devices. Attackers often launch attacks from the endpoints connected to an enterprise LAN switch. The attacker might gain physical access to the endpoint or first infect the device to then launch an attack. Additionally, a mobile device can become infected while outside the company network and then later connect to the company network, with the attack launching at that point.

Engineers should assume that attacks might be launched from end-user devices connected directly to access ports on the enterprise's LAN switches, so Cisco switches include a number of useful tools to help prevent several types of attacks. This chapter discusses one such tool: port security. Chapter 12, "DHCP Snooping and ARP Inspection," discusses two other switch security tools that take advantage of the switch's access layer role.

This short chapter takes a straightforward approach to the port security feature. The first section discusses the concepts, configuration, and verification, using the primary port security operational mode: shutdown mode. The second section then discusses some of the intricacies of the three operational modes: shutdown, protect, and restrict.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 11-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Port Security Concepts and Configuration | 1–3 |
| Port Security Violation Modes | 4, 5 |

1. Which of the following is required when configuring port security with sticky learning?
   a. Setting the maximum number of allowed MAC addresses on the interface with the **switchport port-security maximum** interface subcommand.
   b. Enabling port security with the **switchport port-security** interface subcommand.
   c. Defining the specific allowed MAC addresses using the **switchport port-security mac-address** interface subcommand.
   d. All the other answers list required commands.
2. A Cisco Catalyst switch connects to what should be individual user PCs. Each port has the same port security configuration, configured as follows:

```
interface range gigabitethernet 0/1 - 24
switchport mode access
switchport port-security
switchport port-security mac-address sticky
```

   Which of the following answers describe the result of the port security configuration created with these commands? (Choose two answers.)
   a. Prevents unknown devices with unknown MAC addresses from sending data through the switch ports.
   b. If a user connects a switch to the cable, prevents multiple devices from sending data through the port.
   c. Will allow any one device to connect to each port and *will* save that device's MAC address into the startup-config.
   d. Will allow any one device to connect to each port but *will not* save that device's MAC address into the startup-config.
3. Which of the following commands list the MAC address table entries for MAC addresses configured by port security? (Choose two answers.)
   a. **show mac address-table dynamic**
   b. **show mac address-table**
   c. **show mac address-table static**
   d. **show mac address-table port-security**
4. The **show port-security interface f0/1** command lists a port status of secure-shutdown. Which one of the following answers must be true about this interface at this time?
   a. The **show interface status** command lists the interface status as connected.
   b. The **show interface status** command lists the interface status as err-disabled.

c. The **show port-security interface** command could list a mode of shutdown or restrict, but not protect.

d. The **show port-security interface** command violation counter can increase while in the secure-shutdown port state.

5. A switch's port Gi0/1 has been correctly enabled with port security. The configuration sets the violation mode to restrict. A frame that violates the port security policy enters the interface, followed by a frame that does not. Which of the following answers correctly describe what happens in this scenario? (Choose two answers.)

a. The switch puts the interface into an err-disabled state when the first frame arrives.

b. The switch generates syslog messages about the violating traffic for the first frame.

c. The switch increments the violation counter for Gi0/1 by 1.

d. The switch discards both the first and second frames.

# Foundation Topics

## Port Security Concepts and Configuration

If the network engineer knows what devices should be cabled and connected to particular interfaces on a switch, the engineer can use **port security** to restrict that interface so that only the expected devices can use it. This approach reduces exposure to attacks in which the attacker connects a laptop to some unused switch port. When that inappropriate device attempts to send frames to the switch interface, the switch can take different actions, ranging from simply issuing informational messages to effectively shutting down the interface.

Port security identifies devices based on the source MAC address of Ethernet frames that the devices send. For example, in Figure 11-1, PC1 sends a frame, with PC1's MAC address as the source address. SW1's F0/1 interface can be configured with port security, and if so, SW1 would examine PC1's MAC address and decide whether PC1 was allowed to send frames into port F0/1.

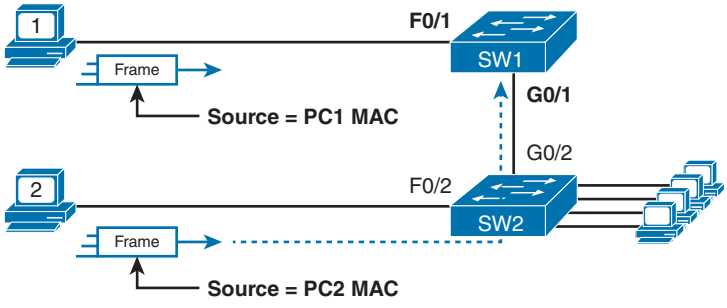



**Figure 11-1** *Source MAC Addresses in Frames as They Enter a Switch*

Port security also has no restrictions on whether the frame came from a local device or was forwarded through other switches. For example, switch SW1 could use port security on its G0/1 interface, checking the source MAC address of the frame from PC2, when forwarded up to SW1 from SW2.

Port security has several flexible options, but all operate with the same core concepts. First, switches enable port security per port, with different settings available per port. Each port has a maximum number of allowed MAC addresses, meaning that for all frames entering that port, only that number of *different* source MAC addresses can be used before port security thinks a violation has occurred. When a frame with a new source MAC address arrives, pushing the number of MAC addresses past the allowed maximum, a port security violation occurs. At that point, the switch takes action—by default, discarding all future incoming traffic on that port.

The following list summarizes these ideas common to all variations of port security:

Key Topic

- It examines frames received on the interface to determine if a violation has occurred.
- It defines a maximum number of unique source MAC addresses allowed for all frames coming in the interface.
- It keeps a list and counter of all unique source MAC addresses on the interface.
- It monitors newly learned MAC addresses, considering those MAC addresses to cause a violation if the newly learned MAC address would push the total number of MAC table entries for the interface past the configured maximum allowed MAC addresses for that port.
- It takes action to discard frames from the violating MAC addresses, plus other actions depending on the configured violation mode.

Those rules define the basics, but port security allows other options as well, including options like these:

- Define a maximum of three MAC addresses, defining all three specific MAC addresses.
- Define a maximum of three MAC addresses but allow those addresses to be dynamically learned, allowing the first three MAC addresses learned.
- Define a maximum of three MAC addresses, predefining one specific MAC address, and allowing two more to be dynamically learned.

You might like the idea of predefining the MAC addresses for port security, but finding the MAC address of each device can be a bother. Port security provides a useful compromise using a feature called *sticky secure MAC addresses*. With this feature, port security learns the MAC addresses off each port so that you do not have to preconfigure the values. It also adds the learned MAC addresses to the port security configuration (in the running-config file). This feature helps reduce the big effort of finding out the MAC address of each device.

As you can see, port security has a lot of detailed options. The next few sections walk you through these options to pull the ideas together.

## Configuring Port Security

Port security configuration involves several steps. First, port security works on both access ports and trunk ports, but it requires you to statically configure the port as a trunk or

an access port, rather than let the switch dynamically decide whether to use trunking. The following configuration checklist details how to enable port security, set the maximum allowed MAC addresses per port, and configure the actual MAC addresses:

Key Topic

**Step 1.** Use the **switchport mode access** or the **switchport mode trunk** interface subcommands, respectively, to make the switch interface either a static access or trunk interface.

**Step 2.** Use the **switchport port-security** interface subcommand to enable port security on the interface.

**Step 3.** (Optional) Use the **switchport port-security maximum** *number* interface subcommand to override the default maximum number of allowed MAC addresses associated with the interface (1).

**Step 4.** (Optional) Use the **switchport port-security violation** {**protect** | **restrict** | **shutdown**} interface subcommand to override the default action to take upon a security violation (shutdown).

**Step 5.** (Optional) Use the **switchport port-security mac-address** *mac-address* interface subcommand to predefine any allowed source MAC addresses for this interface. Use the command multiple times to define more than one MAC address.

**Step 6.** (Optional) Use the **switchport port-security mac-address sticky** interface subcommand to tell the switch to "sticky learn" dynamically learned MAC addresses.

To demonstrate how to configure this variety of the settings, Figure 11-2 and Example 11-1 show four examples of port security. Three ports operate as access ports, while port F0/4, connected to another switch, operates as a trunk.

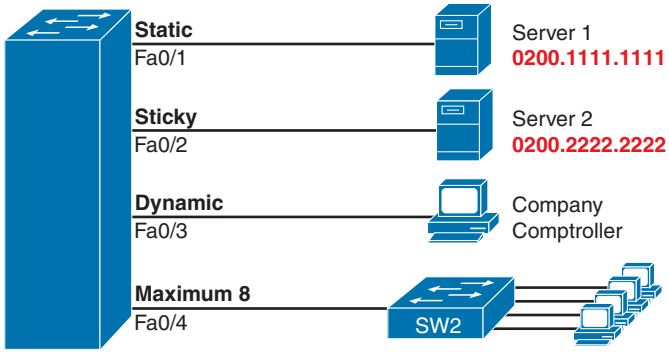



**Figure 11-2** *Port Security Configuration Example*

Answers to the "Do I Know This Already?" quiz:

**1** B **2** B, D **3** B, C **4** B **5** B, C

Key Topic

**Example 11-1** *Variations on Port Security Configuration*

```
SW1# show running-config
(Lines omitted for brevity)

interface FastEthernet0/1
 switchport mode access
 switchport port-security
 switchport port-security mac-address 0200.1111.1111
!
interface FastEthernet0/2
 switchport mode access
 switchport port-security
 switchport port-security mac-address sticky
!
interface FastEthernet0/3
 switchport mode access
 switchport port-security
!
interface FastEthernet0/4
 switchport mode trunk
 switchport port-security
 switchport port-security maximum 8
```

First, scan the configuration for all four interfaces in Example 11-1, focusing on the first two interface subcommands in each case. Note that the first three interfaces in the example use the same first two interface subcommands, matching the first two configuration steps noted before Figure 11-2. The **switchport port-security** command enables port security, with all defaults, with the **switchport mode access** command meeting the requirement to configure the port as either an access or trunk port. The final port, F0/4, has a similar configuration, except that it has been configured as a trunk rather than as an access port.

Next, scan all four interfaces again, and note that the configuration differs on each interface after those first two interface subcommands. Each interface simply shows a different example for perspective.

The first interface, FastEthernet 0/1, adds one optional port security subcommand: **switchport port-security mac-address 0200.1111.1111**, which defines a specific source MAC address. With the default maximum source address setting of 1, only frames with source MAC 0200.1111.1111 will be allowed in this port. When a frame with a source other than 0200.1111.1111 enters F0/1, the switch would normally perform MAC address learning and want to add the new source MAC address to the MAC address table. Port security will see that action as learning one too many MAC addresses on the port, taking the default violation action to disable the interface.

As a second example, FastEthernet 0/2 uses the same logic as FastEthernet 0/1, except that it uses the sticky learning feature. For port F0/2, the configuration of the **switchport port-security mac-address sticky** command tells the switch to dynamically learn source

MAC addresses and add **port-security** commands to the running-config. Example 11-2 shows the running-config file that lists the sticky-learned MAC address in this case.

**Example 11-2** *Configuration Added by the Port Security Sticky Feature*

```
SW1# show running-config interface f0/2
Building configuration...
Current configuration : 188 bytes
!
interface FastEthernet0/2
  switchport mode access
  switchport port-security
  switchport port-security mac-address sticky
  switchport port-security mac-address sticky 0200.2222.2222
```

Port security does not save the configuration of the sticky addresses, so use the **copy running-config startup-config** command if desired.

The other two interfaces in Example 11-1 do not predefine MAC addresses, nor do they sticky-learn the MAC addresses. The only difference between these two interfaces' port security configuration is that FastEthernet 0/4 supports eight MAC addresses because it connects to another switch and should receive frames with multiple source MAC addresses. Interface F0/3 uses the default maximum of one MAC address.

**NOTE** Switches can also use port security on voice ports and EtherChannels. For voice ports, make sure to configure the maximum MAC address to at least two (one for the phone, or for a PC connected to the phone). On EtherChannels, the port security configuration should be placed on the port-channel interface, rather than the individual physical interfaces in the channel.

## Verifying Port Security

The **show port-security interface** command provides the most insight into how port security operates, as shown in Example 11-3. This command lists the configuration settings for port security on an interface; plus it lists several important facts about the current operation of port security, including information about any security violations. The two commands in the example show interfaces F0/1 and F0/2, based on Example 11-1's configuration.

**Example 11-3** *Using Port Security to Define Correct MAC Addresses of Particular Interfaces*

```
SW1# show port-security interface fastEthernet 0/1
Port Security              : Enabled
Port Status                : Secure-shutdown
Violation Mode             : Shutdown
Aging Time                 : 0 mins
Aging Type                 : Absolute
```

```
SecureStatic Address Aging : Disabled
Maximum MAC Addresses      : 1
Total MAC Addresses        : 1
Configured MAC Addresses   : 1
Sticky MAC Addresses       : 0
Last Source Address:Vlan   : 0013.197b.5004:1
Security Violation Count   : 1

SW1# show port-security interface fastEthernet 0/2
Port Security              : Enabled
Port Status                : Secure-up
Violation Mode             : Shutdown
Aging Time                 : 0 mins
Aging Type                 : Absolute
SecureStatic Address Aging : Disabled
Maximum MAC Addresses      : 1
Total MAC Addresses        : 1
Configured MAC Addresses   : 1
Sticky MAC Addresses       : 1
Last Source Address:Vlan   : 0200.2222.2222:1
Security Violation Count   : 0
```

The two commands in Example 11-3 confirm that a security violation has occurred on FastEthernet 0/1, but no violations have occurred on FastEthernet 0/2. The **show port-security interface fastethernet 0/1** command shows that the interface is in a *secure-shutdown* state, which means that the interface has been disabled because of port security. In this case, another device connected to port F0/1, sending a frame with a source MAC address other than 0200.1111.1111, is causing a violation. However, port Fa0/2, which used sticky learning, simply learned the MAC address used by Server 2.

## Port Security MAC Addresses

To complete this chapter, take a moment to think about Layer 2 switching, along with all those examples of output from the **show mac address-table dynamic** EXEC command.

Once a switch port has been configured with port security, the switch no longer considers MAC addresses associated with that port as being dynamic entries as listed with the **show mac address-table dynamic** EXEC command. Even if the MAC addresses are dynamically learned, once port security has been enabled, you need to use one of these options to see the MAC table entries associated with ports using port security:

- **show mac address-table secure:** Lists MAC addresses associated with ports that use port security
- **show mac address-table static:** Lists MAC addresses associated with ports that use port security, as well as any other statically defined MAC addresses

Example 11-4 proves the point. It shows two commands about interface F0/2 from the port security example shown in Figure 11-2 and Example 11-1. In that example, port security was configured on F0/2 with sticky learning, so from a literal sense, the switch learned a MAC address off that port (0200.2222.2222). However, the **show mac address-table dynamic** command does not list the address and port because IOS considers that MAC table entry to be a static entry. The **show mac address-table secure** command does list the address and port.

**Example 11-4** *Using the* **secure** *Keyword to See MAC Table Entries When Using Port Security*

```
SW1# show mac address-table secure interface F0/2
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type         Ports
----    -----------       --------     -----
 1      0200.2222.2222    STATIC       Fa0/2
Total Mac Addresses for this criterion: 1

SW1# show mac address-table dynamic interface f0/2
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type         Ports
----    -----------       --------     -----
SW1#
```

## Port Security Violation Modes

The first half of the chapter discussed many details of port security, but it mostly ignored one major feature: the port security violation mode. The **violation mode** defines how port security should react when a violation occurs.

First, to review, what is a port security violation? Any received frame that breaks the port security rules on an interface. For example:

- For an interface that allows any two MAC addresses, a violation occurs when the total of preconfigured and learned MAC addresses on the interface exceeds the configured maximum of two.
- For an interface that predefines all the specific MAC addresses allowed on the interface, a violation occurs when the switch receives a frame whose source MAC is not one of those configured addresses.

With port security, each switch port can be configured to use one of three violation modes that defines the actions to take when a violation occurs. All three options cause the switch to discard the offending frame (a frame whose source MAC address would push the number of learned MAC addresses over the limit). However, the modes vary in how many other

steps they take. For instance, some modes include the action of the switch generating syslog messages and SNMP Trap messages, while some define the action to disable the interface. Table 11-2 lists the three modes, their actions, along with the keywords that enable each mode on the **switchport port-security violation** {**protect | restrict | shutdown**} interface subcommand:

Key Topic

**Table 11-2** Actions When Port Security Violation Occurs

| Option on the switchport port-security violation Command | Protect | Restrict | Shutdown |
|---|---|---|---|
| Discards offending traffic | Yes | Yes | Yes |
| Sends log and SNMP messages | No | Yes | Yes |
| Disables the interface by putting it in an err-disabled state, discarding all traffic | No | No | Yes |

Because IOS reacts so differently with shutdown mode as compared to restrict and protect modes, the next few pages explain the differences—first for shutdown mode, then for the other two modes.

## Port Security Shutdown Mode

When the (default) shutdown violation mode is used and a port security violation occurs on a port, port security stops all frame forwarding on the interface, both in and out of the port. In effect, it acts as if port security has shut down the port; however, it does not literally configure the port with the **shutdown** interface subcommand. Instead, port security uses the **err-disabled** feature. Cisco switches use the err-disabled state for a wide range of purposes, but when using port security shutdown mode and a violation occurs, the following happens:

Key Topic

- The switch interface state (per **show interfaces** and **show interfaces status**) changes to an err-disabled state.
- The switch interface port security state (per **show port-security**) changes to a secure-shutdown state.
- The switch stops sending and receiving frames on the interface.

Once port security has placed a port in err-disabled state, by default, the port remains in an err-disabled state until someone takes action. To recover from an err-disabled state, the interface must be shut down with the **shutdown** command and then enabled with the **no shutdown** command. Alternately, the switch can be configured to automatically recover from the err-disabled state, when caused by port security, with these commands:

- **errdisable recovery cause psecure-violation:** A global command to enable automatic recovery for interfaces in an err-disabled state caused by port security
- **errdisable recovery interval** *seconds*: A global command to set the time to wait before recovering the interface

To take a closer look at shutdown mode, start by checking the configuration state of the switch. You can check the port security configuration on any interface with the **show port-security interface** *type number* command, as seen previously in Example 11-2, but the

**show port-security** command (as listed in Example 11-5) shows briefer output, with one line per enabled interface.

**Example 11-5** *Confirming the Port Security Violation Mode*

```
SW1# show port-security
Secure Port  MaxSecureAddr  CurrentAddr  SecurityViolation  Security Action
                (Count)       (Count)          (Count)
---------------------------------------------------------------------------
     Fa0/13              1            1                  1         Shutdown
---------------------------------------------------------------------------
Total Addresses in System (excluding one mac per port) : 0
Max Addresses limit in System (excluding one mac per port) : 8192
```

Note that for these next examples, a switch has configured port security on port Fa0/13 only. In this case, the switch appears to be configured to support one MAC address, has already reached that total, and has a security violation action of "shutdown."

Next, Example 11-6 shows the results after a port security violation has already occurred on port F0/13. The first command confirms the err-disabled state (per the **show interfaces status** command) and the secure-shutdown state (per the **show port-security** command).

**Example 11-6** *Port Security Status in Shutdown Mode After a Violation*

```
! The next lines show the log message generated when the violation occurred.
Jul 31 18:00:22.810: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation
occurred, caused by MAC address 0200.3333.3333 on port FastEthernet0/13

! The next command shows the err-disabled state, implying a security violation.
SW1# show interfaces Fa0/13 status

Port     Name                 Status       Vlan   Duplex  Speed  Type
Fa0/13                        err-disabled 1      auto    auto   10/100BaseTX
!
! The next command's output has shading for several of the most important facts.
SW1# show port-security interface Fa0/13
Port Security               : Enabled
Port Status                 : Secure-shutdown
Violation Mode              : Shutdown
Aging Time                  : 0 mins
Aging Type                  : Absolute
SecureStatic Address Aging  : Disabled
Maximum MAC Addresses       : 1
Total MAC Addresses         : 1
Configured MAC Addresses    : 1
Sticky MAC Addresses        : 0
Last Source Address:Vlan    : 0200.3333.3333:2
Security Violation Count    : 1
```

The output of the **show port-security interface** command lists the current port-security status (secure-shutdown) as well as the configured mode (shutdown). The last line of output lists the number of violations that caused the interface to fail to an err-disabled state, while the second-to-last line identifies the MAC address and VLAN of the device that caused the violation.

Figure 11-3 summarizes these behaviors, assuming the same scenario shown in the example.

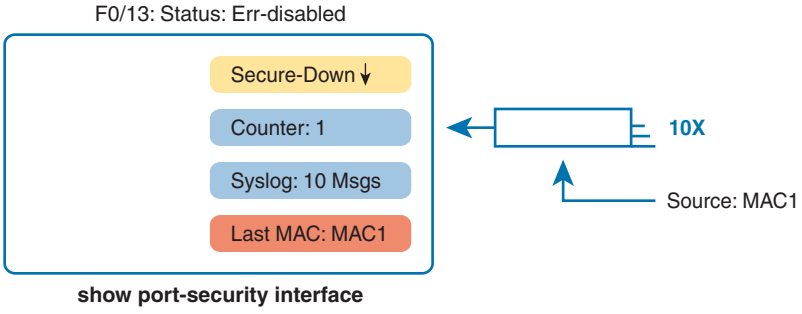



**Figure 11-3** *Summary of Actions: Port Security Violation Mode Shutdown*

The violations counter notes the number of times the interface has been moved to the err-disabled (secure-shutdown) state. For instance, the first time it fails, the counter increments to 1; while err-disabled, many frames can arrive, but the counter remains at 1. IOS will reset the counter to 0 when you recover the interface using a shutdown/no shutdown command combination, so that the next violation will cause the counter to increment to 1.

## Port Security Protect and Restrict Modes

The restrict and protect violation modes take a much different approach to securing ports. These modes still discard offending traffic, but the interface remains in a connected (up/up) state and in a port security state of secure-up. As a result, the port continues to forward good traffic but discards offending traffic.

Having a port in a seemingly good state that also discards traffic can be a challenge when troubleshooting. Basically, you have to know about the feature and then know how to tell when port security is discarding some traffic on a port even though the interface status looks good.

With protect mode, the only action the switch takes for a frame that violates the port security rules is to discard the frame. The switch does not change the port to an err-disabled state, does not generate messages, and does not even increment the violations counter.

Example 11-7 shows a sample with protect mode after several violations have occurred. Note that the **show** command confirms the mode (protect) as configured in the top part of the example, with a port security state of secure-up—a state that will not change in protect mode. Also, note that the counter at the bottom shows 0, even though several violations have occurred, because protect mode does not count the violating frames.

**Example 11-7** *Port Security Using Protect Mode*

```
SW1# show running-config
! Lines omitted for brevity
interface FastEthernet0/13
  switchport mode access
  switchport port-security
  switchport port-security mac-address 0200.1111.1111
  switchport port-security violation protect
! Lines omitted for brevity

SW1# show port-security interface Fa0/13
Port Security               : Enabled
Port Status                 : Secure-up
Violation Mode              : Protect
Aging Time                  : 0 mins
Aging Type                  : Absolute
SecureStatic Address Aging  : Disabled
Maximum MAC Addresses       : 1
Total MAC Addresses         : 1
Configured MAC Addresses    : 1
Sticky MAC Addresses        : 0
Last Source Address:Vlan    : 0000.0000.0000:0
Security Violation Count    : 0
```

**NOTE** The small particulars of the violation counters and last source address might be slightly different with some older switch models and IOS versions. Note that this edition's testing is based on 2960XR switches running IOS 15.2.(6)E2.

While shutdown mode disables the interface, and protect mode does nothing more than discard the offending traffic, restrict mode provides a compromise between the other two modes. If Example 11-7 had used the restrict violation mode instead of protect, the port status would have also remained in a secure-up state; however, IOS would show some indication of port security activity, such as an accurate incrementing violation counter, as well as syslog messages. Example 11-8 shows an example of the violation counter and ends with a sample port security syslog message. In this case, 97 incoming frames so far violated the rules, with the most recent frame having a source MAC address of 0200.3333.3333 in VLAN 1.

**Example 11-8** *Port Security Using Violation Mode Restrict*

```
SW1# show port-security interface fa0/13
Port Security               : Enabled
Port Status                 : Secure-up
Violation Mode              : Restrict
```

```
Aging Time                  : 0 mins
Aging Type                  : Absolute
SecureStatic Address Aging  : Disabled
Maximum MAC Addresses       : 1
Total MAC Addresses         : 1
Configured MAC Addresses    : 1
Sticky MAC Addresses        : 0
Last Source Address:Vlan    : 0200.3333.3333:1
Security Violation Count    : 97
!
! The following log message also points to a port security issue.
!
01:46:58: %PORT_SECURITY-2-PSECURE_VIOLATION: Security violation occurred, caused by
MAC address 0200.3333.3333 on port FastEthernet0/13.
```

Figure 11-4 summarizes the key points about the restrict mode for port security. In this case, the figure matches the same scenario as the example again, with 97 total violating frames arriving so far, with the most recent being from source MAC address MAC3.

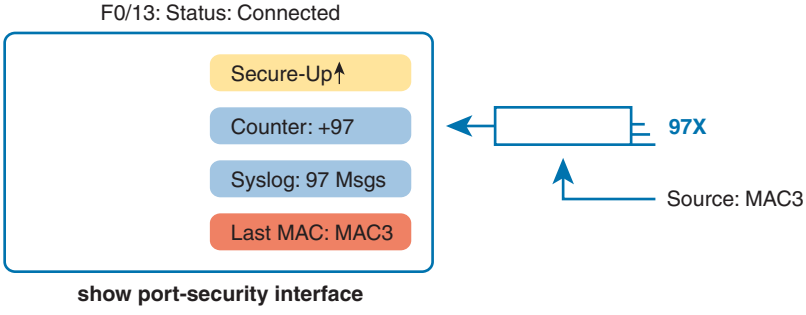



**Figure 11-4** *Summary of Actions: Port Security Violation Mode Restrict*

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 11-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 11-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Answer DIKTA questions | | Book, PTP |
| Review command tables | | Book |
| Review memory tables | | Book, website |
| Review config checklists | | Book, website |
| Do labs | | Sim Lite, blog |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 11-4** Key Topics for Chapter 11

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Summary of port security concepts | 225 |
| List | Port security configuration checklist | 226 |
| Example 11-1 | Port security configuration samples | 227 |
| Table 11-2 | Port security actions and the results of each action | 231 |
| List | Switch actions with a port in err-disabled state | 231 |

## Key Terms You Should Know

error disabled (err-disable), port security, violation mode

## Do Labs

The Sim Lite software is a version of Pearson's full simulator learning product with a subset of the labs, included free with this book. The Sim Lite with this book includes a couple of labs about port security. Also, check the author's blog site pages for configuration exercises (Config Labs) at https://www.certskills.com/config-labs.

## Command References

Tables 11-5 and 11-6 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 11-5** Chapter 11 Configuration Command Reference

| Command | Mode/Purpose/Description |
|---|---|
| **switchport mode {access \| trunk}** | Interface configuration mode command that tells the switch to always be an access port, or always be a trunk port |
| **switchport port-security mac-address** *mac-address* | Interface configuration mode command that statically adds a specific MAC address as an allowed MAC address on the interface |

| Command | Mode/Purpose/Description |
|---|---|
| **switchport port-security mac-address sticky** | Interface subcommand that tells the switch to learn MAC addresses on the interface and add them to the configuration for the interface as secure MAC addresses |
| **switchport port-security maximum** *value* | Interface subcommand that sets the maximum number of static secure MAC addresses that can be assigned to a single interface |
| **switchport port-security violation** {**protect** \| **restrict** \| **shutdown**} | Interface subcommand that tells the switch what to do if an inappropriate MAC address tries to access the network through a secure switch port |
| **errdisable recovery cause psecure-violation** | Global command that enables the automatic recovery from err-disabled state for ports that reach that state due to port security violations |
| **errdisable recovery interval** *seconds* | Global command that sets the delay, in seconds, before a switch attempts to recover an interface in err-disabled mode, regardless of the reason for that interface being in that state |
| **shutdown**<br>**no shutdown** | Interface subcommands that administratively disable and enable an interface, respectively |

**Table 11-6** Chapter 11 EXEC Command Reference

| Command | Purpose |
|---|---|
| **show running-config** | Lists the currently used configuration |
| **show running-config \| interface** *type number* | Displays the running-configuration excerpt of the listed interface and its subcommands only |
| **show mac address-table dynamic** [**interface** *type number*] | Lists the dynamically learned entries in the switch's address (forwarding) table |
| **show mac address-table secure** [**interface** *type number*] | Lists MAC addresses defined or learned on ports configured with port security |
| **show mac address-table static** [**interface** *type number*] | Lists static MAC addresses and MAC addresses learned or defined with port security |
| **show interfaces** [**interface** *type number*] **status** | Lists one output line per interface (or for only the listed interface if included), noting the description, operating state, and settings for duplex and speed on each interface |
| **show port-security interface** *type number* | Lists an interface's port security configuration settings and security operational status |
| **show port-security** | Lists one line per interface that summarizes the port security settings for any interface on which it is enabled |



CHAPTER 12

# DHCP Snooping and ARP Inspection

**This chapter covers the following exam topics:**

**5.0 Security Fundamentals**

**5.7 Configure and verify Layer 2 security features (DHCP snooping, dynamic ARP inspection, and port security)**

To understand the kinds of risks that exist in modern networks, you have to first understand the rules. Then you have to think about how an attacker might take advantage of those rules in different ways. Some attacks might cause harm as part of a denial-of-service (DoS) attack, while a reconnaissance attack may gather more data to prepare for some other attack. For every protocol and function you learn in networking, there are possible methods to take advantage of those features to give an attacker an advantage.

This chapter discusses two switch features that help prevent some types of attacks that can result in the attacker getting copies of packets sent to/from a legitimate host. One of these features, DHCP Snooping, notices DHCP messages that fall outside the normal use of DHCP—messages that may be part of an attack—and discards those messages. It also watches the DHCP messages that flow through a LAN switch, building a table that lists the details of legitimate DHCP flows, so that other switch features can know what legitimate DHCP leases exist for devices connected to the switch.

The second such feature, Dynamic ARP Inspection (DAI), also helps prevent packets being redirected to an attacking host. Some ARP attacks try to convince hosts to send packets to the attacker's device instead of the true destination. The switch watches ARP messages as they flow through the switch. The switch checks incoming ARP messages, checking those against normal ARP operation as well as checking the details against other data sources, including the DHCP Snooping binding table. When the ARP message does not match the known information about the legitimate addresses in the network, the switch filters the ARP message.

This chapter examines DHCP Snooping concepts and configuration in the first major section and DAI in the second.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 12-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| DHCP Snooping | 1–4 |
| Dynamic ARP Inspection | 5–7 |

1. An engineer hears about DHCP Snooping and decides to implement it. Which of the following are the devices on which DHCP Snooping could be implemented? (Choose two answers.)
   a. Layer 2 switches
   b. Routers
   c. Multilayer switches
   d. End-user hosts
2. Layer 2 switch SW2 connects a Layer 2 switch (SW1), a router (R1), a DHCP server (S1), and three PCs (PC1, PC2, and PC3). All PCs are DHCP clients. Which of the following are the most likely DHCP Snooping trust state configurations on SW2 for the ports connected to the listed devices? (Choose two answers.)
   a. The port connected to the router is untrusted.
   b. The port connected to switch SW1 is trusted.
   c. The port connected to PC1 is untrusted.
   d. The port connected to PC3 is trusted.
3. Switch SW1 needs to be configured to use DHCP Snooping in VLAN 5 and only VLAN 5. Which commands must be included, assuming at least one switch port in VLAN 5 must be an untrusted port? (Choose two answers.)
   a. **no ip dhcp snooping trust**
   b. **ip dhcp snooping untrust**
   c. **ip dhcp snooping**
   d. **ip dhcp snooping vlan 5**
4. On a multilayer switch, a switch needs to be configured to perform DHCP Snooping on some Layer 2 ports in VLAN 3. Which command may or may not be needed depending on whether the switch also acts as a DHCP relay agent?
   a. **no ip dhcp snooping information option**
   b. **ip dhcp snooping limit rate 5**
   c. **errdisable recovery cause dhcp-rate-limit**
   d. **ip dhcp snooping vlan 3**
5. Switch SW1 has been configured to use Dynamic ARP Inspection with DHCP Snooping in VLAN 5. An ARP request arrives on port G0/1. Which answer describes two items DAI always compares regardless of the configuration?
   a. The message's ARP sender hardware address and the message's Ethernet header source MAC address
   b. The message's ARP sender hardware address and the DHCP Snooping binding table

c. The message's ARP target IP address and the DHCP Snooping binding table
d. The message's ARP target IP address and the switch's ARP table

6. Switch SW1 needs to be configured to use Dynamic ARP Inspection along with DHCP Snooping in VLAN 6 and only VLAN 6. Which commands must be included, assuming at least one switch port in VLAN 6 must be a trusted port? (Choose two answers.)
   a. **no ip arp inspection untrust**
   b. **ip arp inspection trust**
   c. **ip arp inspection**
   d. **ip arp inspection vlan 6**
7. A Layer 2 switch needs to be configured to use Dynamic ARP Inspection along with DHCP Snooping. Which command would make DAI monitor ARP message rates on an interface at an average rate of 4 received ARP messages per second? (Choose two answers.)
   a. **ip arp inspection limit rate 4 burst interval 2**
   b. **ip arp inspection limit rate 10 burst interval 2**
   c. **ip arp inspection limit rate 16 burst interval 4**
   d. **ip arp inspection limit rate 4**

# Foundation Topics

## DHCP Snooping

DHCP servers play a vital role in most every network today, with almost every user endpoint using DHCP to learn its IP address, mask, default gateway, and DNS server IP addresses. Chapter 19, "IP Addressing on Hosts," in the *CCNA 200-301 Official Certification Guide, Volume 1*, Second Edition, shows how DHCP should work under normal circumstances. This section now examines how attackers might use DHCP for their own ends and how two specific tools—DHCP Snooping and Dynamic ARP Inspection (DAI)—help defeat those attacks.

This section begins with an examination of the need for DHCP Snooping concepts, including the types of attacks it can try to prevent, followed by details of how to configure DHCP Snooping.

### DHCP Snooping Concepts

**DHCP Snooping** on a switch acts like a firewall or an ACL in many ways. It analyzes incoming messages on the specified subset of ports in a VLAN. DHCP Snooping never filters non-DHCP messages, but it may choose to filter DHCP messages, applying logic to make a choice—to allow the incoming DHCP message or discard the message.

While DHCP itself provides a Layer 3 service, DHCP Snooping operates on LAN switches and is commonly used on Layer 2 LAN switches and enabled on Layer 2 ports. The reason to put DHCP Snooping on the switch is that the function needs to be performed between a typical end-user device—the type of device that acts as a DHCP client—and DHCP servers or DHCP relay agents.

Figure 12-1 shows a sample network that provides a good backdrop to discuss DHCP Snooping. First, all devices connect to Layer 2 switch SW2, with all ports as Layer 2 switch ports, all in the same VLAN. The typical DHCP clients sit on the right of the figure. The left shows other devices that could be the path through which to reach a DHCP server.

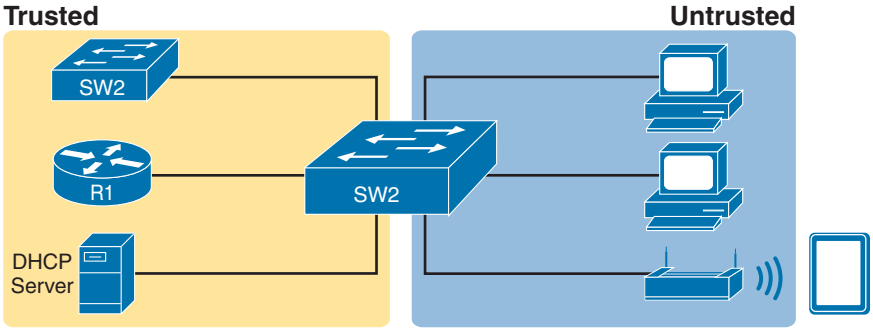



**Figure 12-1** *DHCP Snooping Basics: Client Ports Are Untrusted*

DHCP Snooping works first on all ports in a VLAN, but with each port being trusted or untrusted by DHCP Snooping. To understand why, consider this summary of the general rules used by DHCP Snooping. Note that the rules differentiate between messages normally sent by servers (like DHCPOFFER and DHCPACK) versus those normally sent by DHCP clients (DHCPDISCOVER and DHCPREQUEST):

- DHCP messages received on an **untrusted port**, for messages normally sent by a server, will always be discarded.
- DHCP messages received on an untrusted port, as normally sent by a DHCP client, may be filtered if they appear to be part of an attack.
- DHCP messages received on a **trusted port** will be forwarded; trusted ports do not filter (discard) any DHCP messages.

### A Sample Attack: A Spurious DHCP Server

To give you some perspective, Figure 12-2 shows a legitimate user's PC on the far right and the legitimate DHCP server on the far left. However, an attacker has connected a laptop to the LAN and started a DHCP attack by acting like a DHCP server. Following the steps in the figure, assume PC1 is attempting to lease an IP address while the attacker is making this attack:

1. PC1 sends a LAN broadcast with PC1's first DHCP message (DHCPDISCOVER).
2. The attacker's PC—acting as a spurious DHCP server—replies to the DHCPDISCOVER with a DHCPOFFER.

In this example, the DHCP server created and used by the attacker actually leases a useful IP address to PC1, in the correct subnet, with the correct mask. Why? The attacker wants PC1 to function, but with one twist. Notice the default gateway assigned to PC1: 10.1.1.2, which is the attacker's PC address, rather than 10.1.1.1, which is router R1's address. Now PC1 thinks it has all it needs to connect to the network, and it does—but now all the packets sent by PC1 to what it thinks is its default router flow first through the attacker's PC, creating a man-in-the-middle attack, as shown in Figure 12-3.

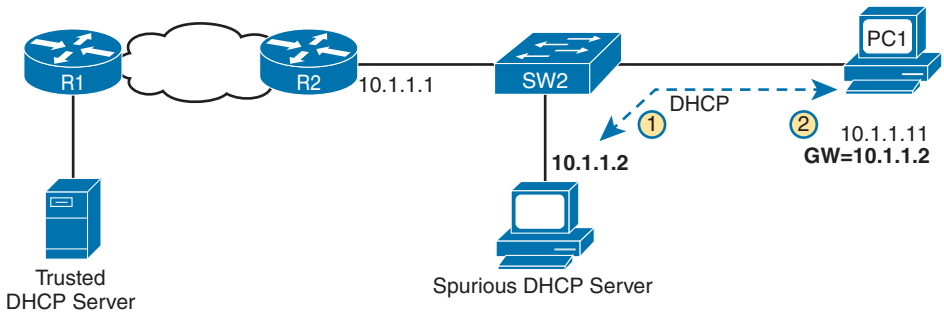



**Figure 12-2** *DHCP Attack Supplies Good IP Address but Wrong Default Gateway*

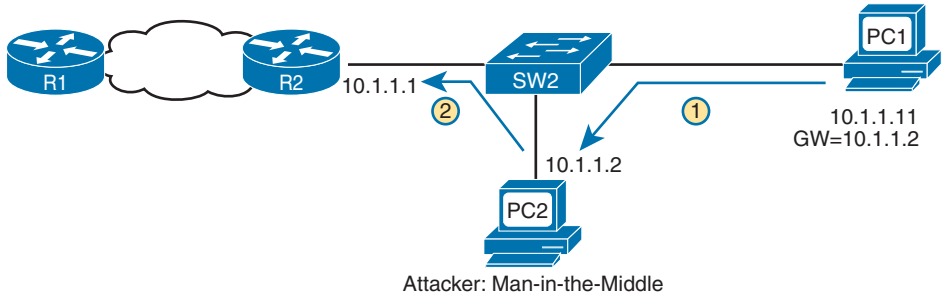



**Figure 12-3** *Unfortunate Result: DHCP Attack Leads to Man-in-the-Middle*

Note that the legitimate DHCP also returns a DHCPOFFER message to host PC1, but most hosts use the first received DHCPOFFER, and the attacker will likely be first in this scenario.

The two steps in the figure show data flow once DHCP has completed. For any traffic destined to leave the subnet, PC1 sends its packets to its default gateway, 10.1.1.2, which happens to be the attacker. The attacker forwards the packets to R1. The PC1 user can connect to any and all applications just like normal, but now the attacker can keep a copy of anything sent by PC1.

### DHCP Snooping Logic

The preceding example shows just one attack in which the attacker acts like a DHCP server (spurious DHCP server). DHCP Snooping defeats such attacks by making most ports untrusted, which by definition would filter the DHCP server messages that arrive on the untrusted ports. For instance, in Figures 12-2 and 12-3, making the port connected to the attacker, a DHCP Snooping untrusted port defeats the attack.

To appreciate the broader set of DHCP Snooping rules and logic, it helps to have a handy reference of some of the more common DHCP messages and processes. For a quick review, the normal message flow includes this sequence: DISCOVER, OFFER, REQUEST, ACK (DORA). In particular:

- Clients send DISCOVER and REQUEST.
- Servers send OFFER and ACK.

---

Answers to the "Do I Know This Already?" quiz:

**1** A, C **2** B, C **3** C, D **4** A **5** B **6** B, D **7** C, D

Additionally, DHCP clients also use the DHCP RELEASE and DHCP DECLINE messages. When a client has a working lease for an address but no longer wants to use the address, the DHCP client can tell the DHCP server it no longer needs the address, releasing it back to the DHCP server, with the DHCP RELEASE message. Similarly, a client can send a DHCP DECLINE message to turn down the use of an IP address during the normal DORA flow on messages.

Now to the logic for DHCP Snooping untrusted ports. Figure 12-4 summarizes the ideas, with two switch ports. On the left, the switch port connects to a DHCP server, so it should be trusted; otherwise, DHCP would not work because the switch would filter all DHCP messages sent by the DHCP server. On the right, PC1 connects to an untrusted port with a DHCP client.

Key Topic

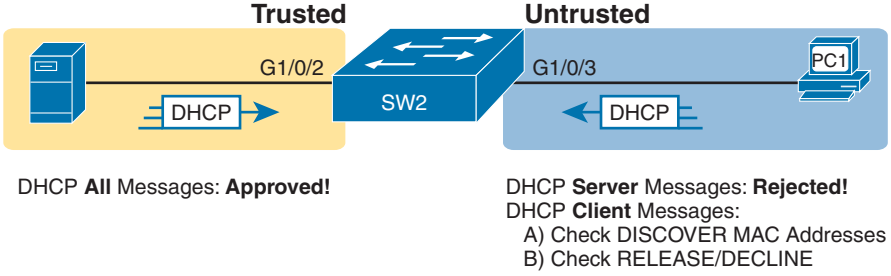



**Figure 12-4** *Summary of Rules for DHCP Snooping*

The DHCP Snooping rules are as follows:

Key Topic

1. Examine all incoming DHCP messages.
2. If normally sent by servers, discard the message.
3. If normally sent by clients, filter as follows:
   a. For DISCOVER and REQUEST messages, check for MAC address consistency between the Ethernet frame and the DHCP message.
   b. For RELEASE or DECLINE messages, check the incoming interface plus IP address versus the DHCP Snooping binding table.
4. For messages allowed by DHCP Snooping, observe the details in the messages, and if they result in a DHCP lease, build a new entry to the DHCP Snooping binding table.

The next few pages complete the discussion of concepts by explaining a little more about steps 3 and 4 in the list.

### Filtering DISCOVER Messages Based on MAC Address

DHCP Snooping does one straightforward check for the most common client-sent messages: DISCOVER and REQUEST. First, note that DHCP messages define the chaddr (client hardware address) field to identify the client. Hosts on LANs include the device's MAC address as part of chaddr. As usual, Ethernet hosts encapsulate the DHCP messages inside Ethernet frames, and those frames of course include a source MAC address—an address that should be the same MAC address used in the DHCP chaddr field. DHCP Snooping does a simple check to make sure those values match.

Figure 12-5 shows how an attacker could attempt to overload the DHCP server and lease all the addresses in the subnet. The attacker's PC uses pseudo MAC address A, so all three DISCOVER messages in the figure show a source Ethernet address of "A." However, each message (in the DHCP data) identifies a different MAC address in the chaddr value (shown as MAC1, MAC2, and MAC3 in the figure for brevity), so from a DHCP perspective, each message appears to be a different DHCP request. The attacker can attempt to lease every IP address in the subnet so that no other hosts could obtain a lease.

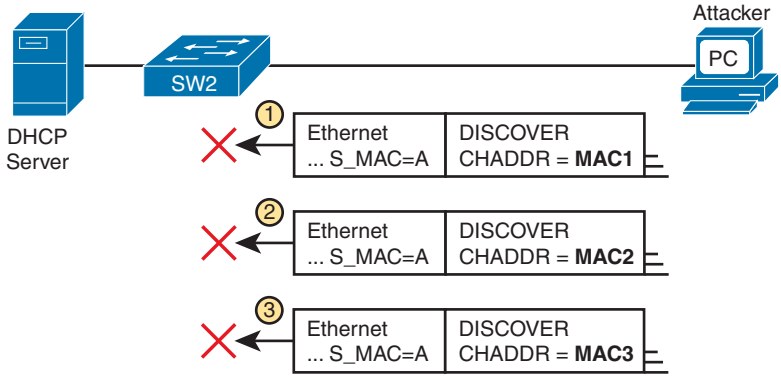



**Figure 12-5** *DHCP Snooping Checks chaddr and Ethernet Source MAC*

The core feature of DHCP Snooping defeats this type of attack on untrusted ports. It checks the Ethernet header source MAC address and compares that address to the MAC address in the DHCP header, and if the values do not match, DHCP Snooping discards the message.

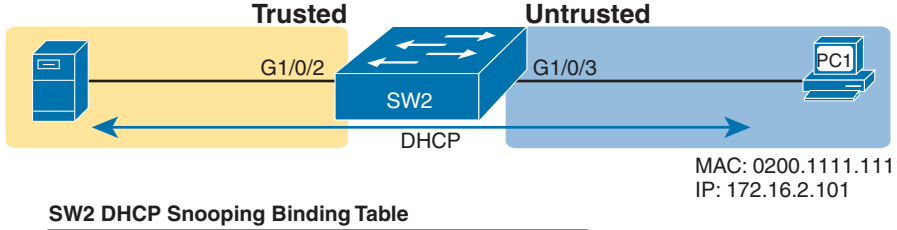
### Filtering Messages That Release IP Addresses

Before looking at the next bit of logic, you need to first understand the DHCP Snooping binding table.

DHCP Snooping builds the **DHCP Snooping binding table** for all the DHCP flows it sees that it allows to complete. That is, for any working legitimate DHCP flows, it keeps a list of some of the important facts. Then DHCP Snooping, and other features like Dynamic ARP Inspection, can use the table to make decisions.

As an example, consider Figure 12-6, which repeats the same topology as Figure 12-4, now with one entry in its DHCP Snooping binding table.

Key Topic




**SW2 DHCP Snooping Binding Table**

| MAC | IP | VLAN | Interface |
|---|---|---|---|
| 0200.1111.1111 | 172.16.2.101 | 11 | G1/0/3 |

**Figure 12-6** *Legitimate DHCP Client with DHCP Binding Entry Built by DHCP Snooping*

In this simple network, the DHCP client on the right leases IP address 172.16.2.101 from the DHCP server on the left. The switch's DHCP Snooping feature combines the information from the DHCP messages, with information about the port (interface G1/0/3, assigned to VLAN 11 by the switch), and puts that in the DHCP Snooping binding table.

DHCP Snooping then applies additional filtering logic that uses the DHCP Snooping binding table: it checks client-sent messages like RELEASE and DECLINE that would cause the DHCP server to be allowed to release an address. For instance, a legitimate user might lease address 172.16.2.101, and at some point release the address back to the server; however, before the client has finished with its lease, an attacker could send a DHCP RELEASE message to release that address back into the pool. The attacker could then immediately try to lease that address, hoping the DHCP server assigns that same 172.16.2.101 address to the attacker.

Figure 12-7 shows an example. PC1 already has a DHCP address (172.16.2.101), with SW2 listing an entry in the DHCP Snooping binding table. The figure shows the action by which the attacker off port G1/0/5 attempts to release PC1's address. DHCP Snooping compares the incoming message, incoming interface, and matching table entry:

1. The attacker, PC A, sends a DHCP RELEASE message, received by switch SW2 in port G1/0/5. The message attempts to DHCP RELEASE address 172.16.2.101.
2. Switch SW2 compares the DHCP Snooping binding table to find the entry matching the listed address: 172.16.2.101.
3. Switch SW2 notes that the binding table lists the legitimate entry with port G1/0/3, but the new DHCP RELEASE arrived in port G1/0/5. As a result, DHCP Snooping discards the DHCP RELEASE message.

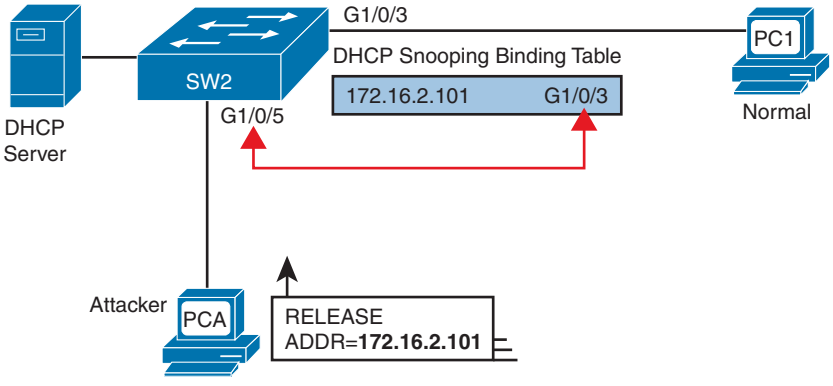



**Figure 12-7** *DHCP Snooping Defeats a DHCP RELEASE from Another Port*

## DHCP Snooping Configuration

DHCP Snooping requires several configuration steps to make it work. First, you need to use a pair of associated global commands: one to enable DHCP Snooping and another to list the VLANs on which to use DHCP Snooping. Both must be included for DHCP Snooping to operate.

Second, while not literally required, you will often need to configure a few ports as trusted ports. Most switches that use DHCP Snooping for a VLAN have some trusted ports and some untrusted ports, and with a default of untrusted, you need to configure the trusted ports.

This section begins with an example that shows how to configure a typical Layer 2 switch to use DHCP Snooping, with required commands as just described, and with other optional commands.

### Configuring DHCP Snooping on a Layer 2 Switch

The upcoming examples all rely on the topology illustrated in Figure 12-8, with Layer 2 switch SW2 as the switch on which to enable DHCP Snooping. The DHCP server sits on the other side of the WAN, on the left of the figure. As a result, SW2's port connected to router R2 (a DHCP relay agent) needs to be trusted. On the right, two sample PCs can use the default untrusted setting.

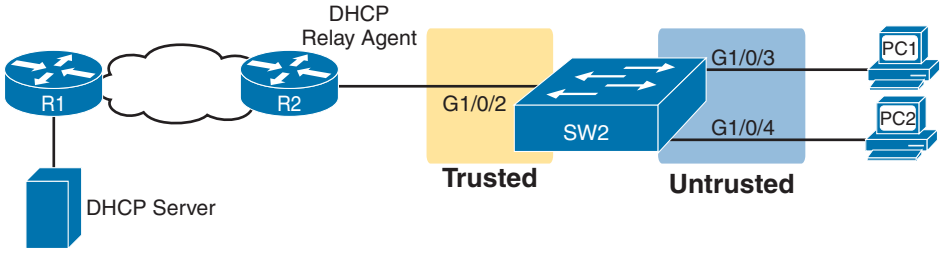



**Figure 12-8** *Sample Network Used in DHCP Snooping Configuration Examples*

Switch SW2 places all the ports in the figure in VLAN 11. To enable DHCP Snooping in VLAN 11, SW2 requires two commands, as shown near the top of Example 12-1: **ip dhcp snooping** and **ip dhcp snooping vlan 11**. Then, to change the logic on port G1/0/2 (connected to the router) to be trusted, the configuration includes the **ip dhcp snooping trust** interface subcommand.

Key Topic

**Example 12-1** *DHCP Snooping Configuration to Match Figure 12-8*

```
ip dhcp snooping
ip dhcp snooping vlan 11
no ip dhcp snooping information option
!
interface GigabitEthernet1/0/2
 ip dhcp snooping trust
```

Note that the **no ip dhcp snooping information option** command in Example 12-1 will be explained in a better context just after Example 12-2 but is listed in Example 12-1 to make the example complete.

With this configuration, the switch follows the logic steps detailed in the earlier section titled "DHCP Snooping Logic." To see some support for that claim, look at Example 12-2, which shows the output from the **show ip dhcp snooping** command on switch SW2.

**Example 12-2** *SW2 DHCP Snooping Status*

```
SW2# show ip dhcp snooping
Switch DHCP snooping is enabled
Switch DHCP gleaning is disabled
DHCP snooping is configured on following VLANs:
11
DHCP snooping is operational on following VLANs:
11
Smartlog is configured on following VLANs:
none
Smartlog is operational on following VLANs:
none
DHCP snooping is configured on the following L3 Interfaces:

Insertion of option 82 is disabled
   circuit-id default format: vlan-mod-port
   remote-id: bcc4.938b.a180 (MAC)
Option 82 on untrusted port is not allowed
Verification of hwaddr field is enabled
Verification of giaddr field is enabled
DHCP snooping trust/rate is configured on the following Interfaces:

Interface                  Trusted    Allow option    Rate limit (pps)
-----------------------    -------    ------------    ----------------
GigabitEthernet1/0/2       yes        yes             unlimited
  Custom circuit-ids:
```

The highlighted lines in the example point out a few of the key configuration settings. Starting at the top, the first two confirm the configuration of the **ip dhcp snooping** and **ip dhcp snooping vlan 11** commands, respectively. Also, the highlighted lines at the bottom of the output show a section that lists trusted ports—in this case, only port G1/0/2.

Also, you might have noticed that highlighted line in the middle that states **Insertion of option 82 is disabled**. That line confirms the addition of the **no ip dhcp information option** command to the configuration back in Example 12-1. To understand why the example includes this command, consider these facts about DHCP relay agents:

- DHCP relay agents add new fields to DHCP requests—defined as option 82 DHCP header fields (in RFC 3046).
- DHCP Snooping uses default settings that work well if the switch acts as a Layer 3 switch and as a DHCP relay agent, meaning that the switch should insert the DHCP option 82 fields into DHCP messages. In effect, the switch defaults to use **ip dhcp snooping information option**.
- When the switch does not also act as a DHCP relay agent, the default setting stops DHCP from working for end users. The switch sets fields in the DHCP messages as if it

were a DHCP relay agent, but the changes to those messages cause most DHCP servers (and most DHCP relay agents) to ignore the received DHCP messages.

- The conclusion: To make DHCP Snooping work on a switch that is not also a DHCP relay agent, and to avoid the problem of preventing legitimate DHCP leases, disable the option 82 feature using the **no ip dhcp snooping information option** global command.

That concludes the DHCP Snooping configuration that is both required and that you will most often need to make the feature work. The rest of this section discusses a few optional DHCP Snooping features.

### Limiting DHCP Message Rates

Knowing that DHCP Snooping prevents their attacks, what might attackers do in response? Devise new attacks, including attacking DHCP Snooping itself.

One way to attack DHCP Snooping takes advantage of the fact that it uses the general-purpose CPU in a switch. Knowing that, attackers can devise attacks to generate large volumes of DHCP messages in an attempt to overload the DHCP Snooping feature and the switch CPU itself. The goal can be as a simple denial-of-service attack or a combination of attacks that might cause DHCP Snooping to fail to examine every message, allowing other DHCP attacks to then work.

To help prevent this kind of attack, DHCP Snooping includes an optional feature that tracks the number of incoming DHCP messages. If the number of incoming DHCP messages exceeds that limit over a one-second period, DHCP Snooping treats the event as an attack and moves the port to an err-disabled state. Also, the feature can be enabled both on trusted and untrusted interfaces.

Although rate limiting DHCP messages can help, placing the port in an err-disabled state can itself create issues. As a reminder, once in the err-disabled state, the switch will not send or receive frames for the interface. However, the err-disabled state might be too severe an action because the default recovery action for an err-disabled state requires the configuration of a **shutdown** and then a **no shutdown** subcommand on the interface.

To help strike a better balance, you can enable DHCP Snooping rate limiting and then also configure the switch to automatically recover from the port's err-disabled state, without the need for a **shutdown** and then **no shutdown** command.

Example 12-3 shows how to enable DHCP Snooping rate limits and err-disabled recovery. First, look at the lower half of the configuration, to the interfaces, to see the straightforward setting of the per-interface limits using the **ip dhcp snooping rate limit** *number* interface subcommands. The top of the configuration uses two global commands to tell IOS to recover from an err-disabled state if it is caused by DHCP Snooping, and to use a nondefault number of seconds to wait before recovering the interface. Note that the configuration in Example 12-3 would rely on the core configuration for DHCP Snooping as shown in Example 12-1.

**Example 12-3** *Configuring DHCP Snooping Message Rate Limits*

```
errdisable recovery cause dhcp-rate-limit
errdisable recovery interval 30
!
interface GigabitEthernet1/0/2
 ip dhcp snooping limit rate 10
!
interface GigabitEthernet1/0/3
 ip dhcp snooping limit rate 2
```

A repeat of the **show ip dhcp snooping** command now shows the rate limits near the end of the output, as noted in Example 12-4.

**Example 12-4** *Confirming DHCP Snooping Rate Limits*

```
SW2# show ip dhcp snooping
! Lines omitted for brevity

Interface                Trusted    Allow option    Rate limit (pps)
-----------------------  -------    ------------    ----------------
GigabitEthernet1/0/2     yes        yes             10
  Custom circuit-ids:
GigabitEthernet1/0/3     no         no              2
  Custom circuit-ids:
```

## DHCP Snooping Configuration Summary

The following configuration checklist summarizes the commands included in this section about how to configure DHCP Snooping.

Key Topic

Config Checklist

**Step 1.** Configure this pair of commands (both required):

- **A.** Use the **ip dhcp snooping** global command to enable DHCP Snooping on the switch.
- **B.** Use the **ip dhcp snooping vlan** *vlan-list* global command to identify the VLANs on which to use DHCP Snooping.

**Step 2.** **(Optional):** Use the **no ip dhcp snooping information option** global command on Layer 2 switches to disable the insertion of DHCP Option 82 data into DHCP messages, specifically on switches that do not act as a DHCP relay agent.

**Step 3.** Configure the **ip dhcp snooping trust** interface subcommand to override the default setting of not trusted.

**Step 4.** **(Optional):** Configure DHCP Snooping rate limits and err-disabled recovery:

- **Step A.** **(Optional):** Configure the **ip dhcp snooping limit rate** *number* interface subcommand to set a limit of DHCP messages per second.
- **Step B.** **(Optional):** Configure the **no ip dhcp snooping limit rate** *number* interface subcommand to remove an existing limit and reset the interface to use the default of no rate limit.

**Step C.** **(Optional):** Configure the **errdisable recovery cause dhcp-rate-limit** global command to enable the feature of automatic recovery from err-disabled mode, assuming the switch placed the port in err-disabled state because of exceeding DHCP Snooping rate limits.

**Step D.** **(Optional):** Configure the **errdisable recovery interval** *seconds* global commands to set the time to wait before recovering from an interface err-disabled state (regardless of the cause of the err-disabled state).

# Dynamic ARP Inspection

The **Dynamic ARP Inspection (DAI)** feature on a switch examines incoming ARP messages on untrusted ports to filter those it believes to be part of an attack. DAI's core feature compares incoming ARP messages with two sources of data: the DHCP Snooping binding table and any configured ARP ACLs. If the incoming ARP message does not match the tables in the switch, the switch discards the ARP message.

This section follows the same sequence as with the DHCP Snooping section, first examining the concepts behind DAI and ARP attacks, and then showing how to configure DAI with both required and optional features.

## DAI Concepts

To understand the attacks DAI can prevent, you need to be ready to compare normal ARP operations with the abnormal use of ARP used in some types of attacks. This section uses that same flow, first reviewing a few important ARP details, and then showing how an attacker can just send an **ARP reply**—called a **gratuitous ARP**—triggering hosts to add incorrect ARP entries to their ARP tables.

### Review of Normal IP ARP

If all you care about is how ARP works normally, with no concern about attacks, you can think of ARP to the depth shown in Figure 12-9. The figure shows a typical sequence. Host PC1 needs to send an IP packet to its default router (R2), so PC1 first sends an ARP request message in an attempt to learn the MAC address associated with R2's 172.16.2.2 address. Router R2 sends back an ARP reply, listing R2's MAC address (note the figure shows pseudo MAC addresses to save space).

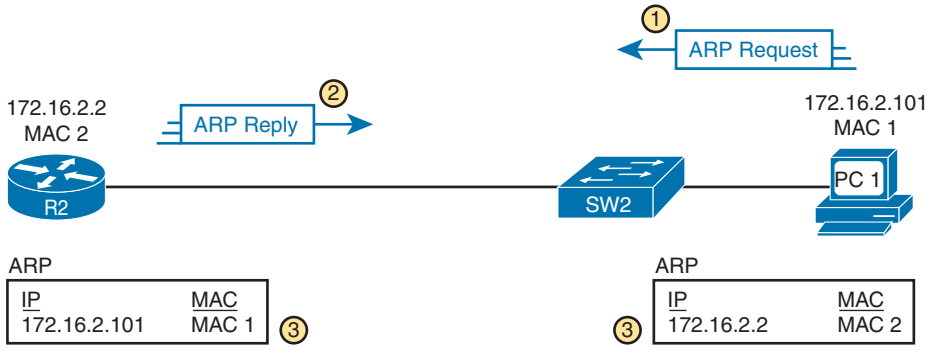



**Figure 12-9** *Legitimate ARP Tables After PC1 DHCP and ARP with Router R2*

The ARP tables at the bottom of the figure imply an important fact: both hosts learn the other host's MAC address with this two-message flow. Not only does PC1 learn R2's MAC address based on the ARP reply (message 2), but router R2 learns PC1's IP and MAC address because of the ARP request (message 1). To see why, take a look at the more detailed view of those messages as shown in Figure 12-10.

Key Topic

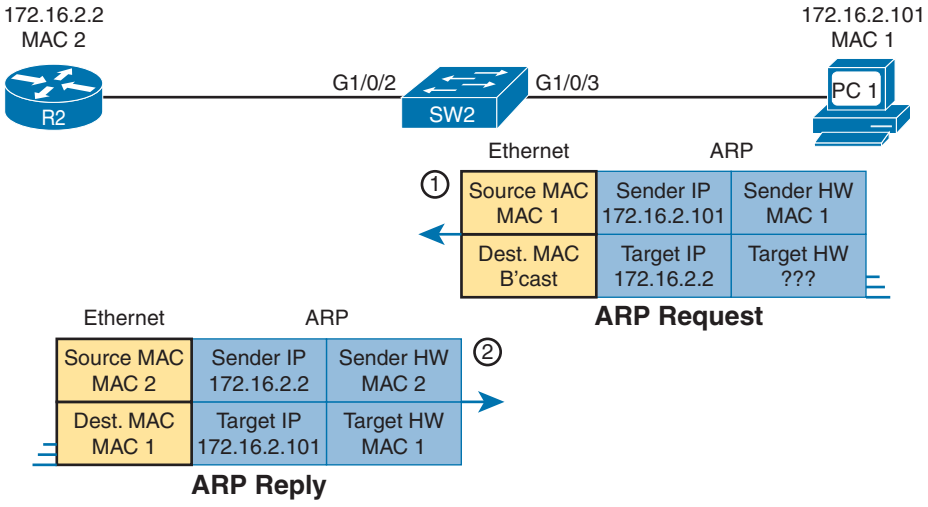



**Figure 12-10** *A Detailed Look at ARP Request and Reply*

The ARP messages define four related fields: the **sender hardware address**, **sender protocol address**, target hardware address, and target protocol address. Those terms use general wording, but the word hardware refers to MAC addresses and the word protocol refers to IP. So, you should expect to see many descriptions about ARP that use similar terms like sender MAC address and **sender IP address**.

The sender fields of every ARP message list the sending device's IP address and MAC, no matter whether the message is an ARP reply or ARP request. For instance, message 1 in the figure, sent by PC1, lists PC1's IP and MAC addresses in the sender fields, which is why router R2 could learn that information. PC1 likewise learns of R2's MAC address per the sender address fields in the ARP reply.

### Gratuitous ARP as an Attack Vector

Normally, a host uses ARP when it knows the IP address of another host and wants to learn that host's MAC address. However, for legitimate reasons, a host might also want to inform all the hosts in the subnet about its MAC address. That might be useful when a host changes its MAC address, for instance. So, ARP supports the idea of a gratuitous ARP message with these features:

Key Topic

- It is an ARP reply.
- It is sent without having first received an ARP request.
- It is sent to an Ethernet destination broadcast address so that all hosts in the subnet receive the message.

For instance, if a host's MAC address is MAC A, and it changes to MAC B, to cause all the other hosts to update their ARP tables, the host could send a gratuitous ARP that lists a sender MAC of MAC B.

Attackers can take advantage of gratuitous ARPs because they let the sending host make other hosts change their ARP tables. Figure 12-11 shows just such an example initiated by PC A (an attacker) with a gratuitous ARP. However, this ARP lists PC1's IP address but a different device's MAC address (PC A) at step 1, causing the router to update its ARP table (step 2).

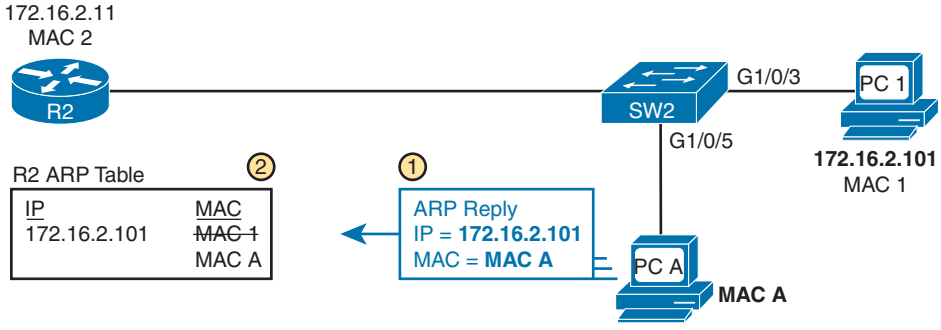



**Figure 12-11** *Nefarious Use of ARP Reply Causes Incorrect ARP Data on R2*

At this point, when R2 forwards IP packets to PC1's IP address (172.16.2.101), R2 will encapsulate them in an Ethernet frame with PC A as the destination rather than with PC1's MAC address. At first, this might seem to stop PC1 from working, but instead it could be part of a man-in-the-middle attack so that PC A can copy every message. Figure 12-12 shows the idea of what happens at this point:

1. PC1 sends messages to some server on the left side of router R2.
2. The server replies to PC1's IP address, but R2 forwards that packet to PC A's MAC address, rather than to PC1.
3. PC A copies the packet for later processing.
4. PC A forwards the packet inside a new frame to PC1 so that PC1 still works.

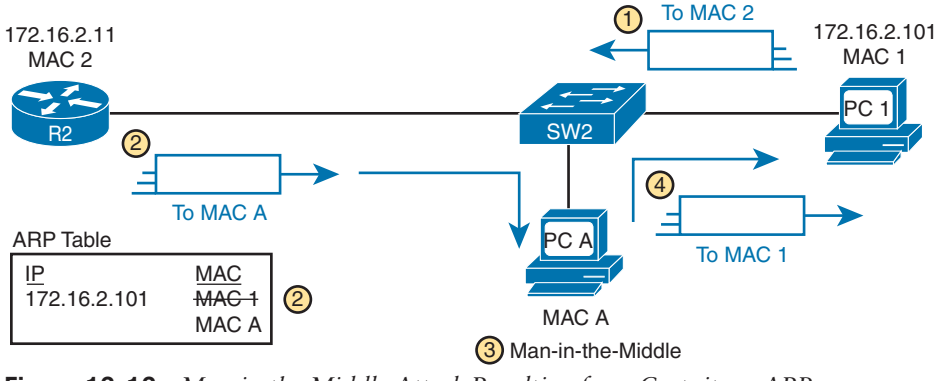



**Figure 12-12** *Man-in-the-Middle Attack Resulting from Gratuitous ARP*

## Dynamic ARP Inspection Logic

DAI has a variety of features that can prevent these kinds of ARP attacks. To understand how, consider the sequence of a typical client host with regards to both DHCP and ARP. When a host does not have an IP address yet—that is, before the DHCP process completes—it does not need to use ARP. Once the host leases an IP address and learns its subnet mask, it needs to use ARP to learn the MAC addresses of other hosts or the default router in the subnet, so it sends some ARP messages. In short, DHCP happens first, then ARP.

DAI takes an approach for untrusted interfaces that confirms an ARP's correctness based on DHCP Snooping's data about the earlier DHCP messages. The correct normal DHCP messages list the IP address leased to a host as well as that host's MAC address. The DHCP Snooping feature also records those facts into the switch's DHCP Snooping binding table.

For any DAI untrusted ports, DAI compares the ARP message's sender IP and sender MAC address fields to the DHCP Snooping binding table. If found in the table, DAI allows the ARP through, but if not, DAI discards the ARP. For instance, Figure 12-13 shows step 1 in which the attacker at PC A attempts the gratuitous ARP shown earlier in Figure 12-11. At step 2, DAI makes a comparison to the DHCP Snooping binding table, not finding a match with MAC A along with IP address 172.16.2.101, so DAI would discard the message.

Key Topic

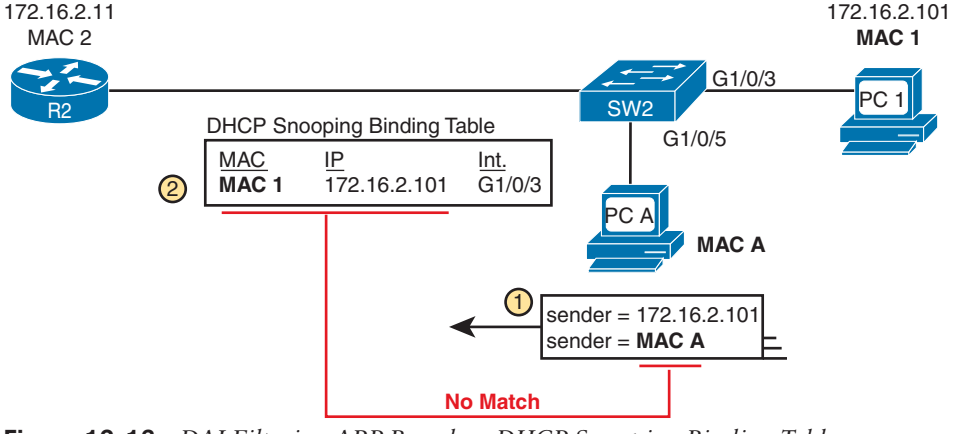



**Figure 12-13** *DAI Filtering ARP Based on DHCP Snooping Binding Table*

DAI also works with a concept of trusted and untrusted ports, with the logic generally matching the logic used with DHCP Snooping. Ports connected to local DHCP clients can remain in the default DAI untrusted state. Configure all other switch ports as trusted for DAI.

Note that although DAI can use the DHCP Snooping table as shown here, it can also use similar statically configured data that lists correct pairs of IP and MAC addresses via a tool called *ARP ACLs*. Using ARP ACLs with DAI becomes useful for ports connected to devices that use static IP addresses rather than DHCP. Note that DAI looks for both the DHCP Snooping binding data and ARP ACLs.

Beyond that core feature, note that DAI can optionally perform other checks as well. For instance, the Ethernet header that encapsulates the ARP should have addresses that match the ARP sender and target MAC addresses. Figure 12-14 shows an example of the comparison of the Ethernet source MAC address and the ARP message sender hardware field.

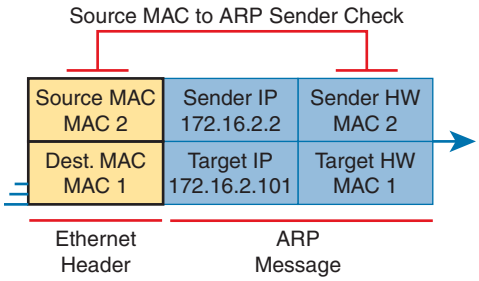



**Figure 12-14** *DAI Filtering Checks for Source MAC Addresses*

DAI can be enabled to make the comparisons shown in the figure, discarding these messages:

- Messages with an Ethernet header source MAC address that is not equal to the ARP sender hardware (MAC) address
- ARP reply messages with an Ethernet header destination MAC address that is not equal to the ARP target hardware (MAC) address
- Messages with unexpected IP addresses in the two ARP IP address fields

Finally, like DHCP Snooping, DAI does its work in the switch CPU rather than in the switch ASIC, meaning that DAI itself can be more susceptible to DoS attacks. The attacker could generate large numbers of ARP messages, driving up CPU usage in the switch. DAI can avoid these problems through rate limiting the number of ARP messages on a port over time.

## Dynamic ARP Inspection Configuration

Configuring DAI requires just a few commands, with the usual larger variety of optional configuration settings. This section examines DAI configuration, first with mostly default settings and with reliance on DHCP Snooping. It then shows a few of the optional features, like rate limits, automatic recovery from err-disabled state, and how to enable additional checks of incoming ARP messages.

### Configuring ARP Inspection on a Layer 2 Switch

Before configuring DAI, you need to think about the feature and make a few decisions based on your goals, topology, and device roles. The decisions include the following:

- Choose whether to rely on DHCP Snooping, ARP ACLs, or both.
- If using DHCP Snooping, configure it and make the correct ports trusted for DHCP Snooping.
- Choose the VLAN(s) on which to enable DAI.
- Make DAI trusted (rather than the default setting of untrusted) on select ports in those VLANs, typically for the same ports you trusted for DHCP Snooping.

All the configuration examples in this section use the same sample network used in the DHCP Snooping configuration topics, repeated here as Figure 12-15. Just as with DHCP Snooping, switch SW2 on the right should be configured to trust the port connected to the router (G1/0/2), but not trust the two ports connected to the PCs.

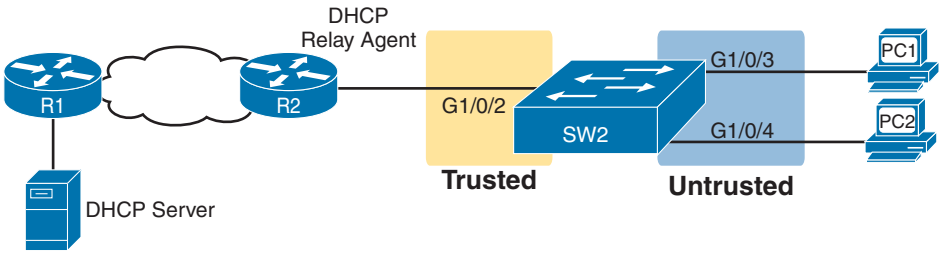



**Figure 12-15** *Sample Network Used in ARP Inspection Configuration Examples*

Example 12-5 shows the required configuration to enable DAI on switch SW2 in Figure 12-15—a configuration that follows a similar progression compared to DHCP Snooping. All ports in the figure connect to VLAN 11, so to enable DAI in VLAN 11, just add the **ip arp inspection vlan 11** global command. Then, to change the logic on port G1/0/2 (connected to the router) to be trusted by DAI, add the **ip arp inspection trust** interface subcommand.

**Example 12-5** *IP ARP Inspection Configuration to Match Figure 12-15*

```
ip arp inspection vlan 11
!
interface GigabitEthernet1/0/2
 ip arp inspection trust
```

Example 12-5 configures DAI, but it omits both DHCP Snooping and ARP ACLs. (If you were to configure a switch only with commands shown in Example 12-5, the switch would filter all ARPs entering all untrusted ports in VLAN 11.) Example 12-6 shows a complete and working DAI configuration that adds the DHCP Snooping configuration to match the DAI configuration in Example 12-5. Note that Example 12-6 combines Example 12-1's earlier DHCP Snooping configuration for this same topology to the DAI configuration just shown in Example 12-5, with highlights for the DAI-specific configuration lines.

Key Topic

**Example 12-6** *IP DHCP Snooping Configuration Added to Support DAI*

```
ip arp inspection vlan 11
ip dhcp snooping
ip dhcp snooping vlan 11
no ip dhcp snooping information option
!
interface GigabitEthernet1/0/2
 ip dhcp snooping trust
 ip arp inspection trust
```

Remember, DHCP occurs first with DHCP clients, and then they send ARP messages. With the configuration in Example 12-6, the switch builds its DHCP Snooping binding table by analyzing incoming DHCP messages. Next, any incoming ARP messages on DAI untrusted ports must have matching information in that binding table.

Example 12-7 confirms the key facts about correct DAI operation in this sample network based on the configuration in Example 12-6. The **show ip arp inspection** command gives both configuration settings along with status variables and counters. For instance, the

highlighted lines show the total ARP messages received on untrusted ports in that VLAN and the number of dropped ARP messages (currently 0).

**Example 12-7** *SW2 IP ARP Inspection Status*

```
SW2# show ip arp inspection

Source Mac Validation      : Disabled
Destination Mac Validation : Disabled
IP Address Validation      : Disabled

 Vlan     Configuration    Operation   ACL Match          Static ACL
 ----     -------------    ---------   ---------          ----------
   11     Enabled          Active

 Vlan     ACL Logging      DHCP Logging      Probe Logging
 ----     -----------      ------------      -------------
   11     Deny             Deny              Off

 Vlan      Forwarded         Dropped     DHCP Drops      ACL Drops
 ----      ---------         -------     ----------      ---------
   11             59               0              0              0

 Vlan   DHCP Permits    ACL Permits  Probe Permits   Source MAC Failures
 ----   ------------    -----------  -------------   -------------------
   11              7              0             49                     0

 Vlan   Dest MAC Failures   IP Validation Failures   Invalid Protocol Data
 ----   -----------------   ----------------------   ---------------------

 Vlan   Dest MAC Failures   IP Validation Failures   Invalid Protocol Data
 ----   -----------------   ----------------------   ---------------------
   11                   0                        0                      0
SW2# show ip dhcp snooping binding
MacAddress          IpAddress       Lease(sec) Type           VLAN  Interface
------------------  -------------  ---------- -------------  ----  --------------------
--
02:00:11:11:11:11  172.16.2.101   86110      dhcp-snooping    11  GigabitEthernet1/0/3
02:00:22:22:22:22  172.16.2.102   86399      dhcp-snooping    11  GigabitEthernet1/0/4
Total number of bindings: 2
```

The end of Example 12-7 shows an example of the **show ip dhcp snooping binding** command on switch SW2. Note that the first two columns list a MAC and IP address as learned from the DHCP messages. Then, imagine an ARP message arrives from PC1, a message that should list PC1's 0200.1111.1111 MAC address and 172.16.2.101 as the sender MAC and IP address, respectively. Per this output, the switch would find that matching data and allow the ARP message.

Example 12-8 shows some detail of what happens when switch SW2 receives an invalid ARP message on port G1/0/4 in Figure 12-15. In this case, to create the invalid ARP message, PC2 in the figure was configured with a static IP address of 172.16.2.101 (which is PC1's DHCP-leased IP address). The highlights in the log message at the top of the example show PC2's claimed sender MAC and sender IP addresses in the ARP message. If you refer back to the bottom of Example 12-7, you can see that this sender MAC/IP pair does not exist in the DHCP Snooping binding table, so DAI rejects the ARP message.

**Example 12-8** *Sample Results from an ARP Attack*

```
Jul 25 14:28:20.763: %SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Req) on Gi1/0/4,
vlan 11.([0200.2222.2222/172.16.2.101/0000.0000.0000/172.16.2.1/09:28:20 EST Thu Jul
25 2019])

SW2# show ip arp inspection statistics

 Vlan      Forwarded         Dropped     DHCP Drops       ACL Drops
 ----      ---------         -------     ----------       ---------
   11             59              17             17               0

 Vlan   DHCP Permits    ACL Permits  Probe Permits   Source MAC Failures
 ----   ------------    -----------  -------------   -------------------
   11              7              0             49                     0

 Vlan   Dest MAC Failures   IP Validation Failures   Invalid Protocol Data
 ----   -----------------   ----------------------   ---------------------
   11                   0                        0                      0
```

The statistics from the **show ip arp inspection** command also confirm that the switch has dropped some ARP messages. The highlighted lines in the middle of the table show 17 total dropped ARP messages in VLAN 11. That same highlighted line confirms that it dropped all 17 because of the DHCP Snooping binding table ("DHCP Drops"), with zero dropped due to an ARP ACL ("ACL Drops").

## Limiting DAI Message Rates

Like DHCP Snooping, DAI can also be the focus of a DoS attack with the attacker generating a large number of ARP messages. Like DHCP Snooping, DAI supports the configuration of rate limits to help prevent those attacks, with a reaction to place the port in an err-disabled state, and with the ability to configure automatic recovery from that err-disabled state.

The DHCP Snooping and DAI rate limits do have some small differences in operation, defaults, and in configuration, as follows:

- DAI defaults to use rate limits for all interfaces (trusted and untrusted), with DHCP Snooping defaulting to not use rate limits.
- DAI allows the configuration of a burst interval (a number of seconds), so that the rate limit can have logic like "*x* ARP messages over *y* seconds" (DHCP Snooping does not define a burst setting).

It helps to look at DAI and DHCP Snooping rate limit configuration together to make comparisons, so Example 12-9 shows both. The example repeats the exact same DHCP Snooping commands in earlier Example 12-3 but adds the DAI configuration (highlighted). The configuration in Example 12-7 could be added to the configuration shown in Example 12-6 for a complete DHCP Snooping and DAI configuration.

**Example 12-9** *Configuring ARP Inspection Message Rate Limits*

```
errdisable recovery cause dhcp-rate-limit
errdisable recovery cause arp-inspection
errdisable recovery interval 30
!
interface GigabitEthernet1/0/2
 ip dhcp snooping limit rate 10
 ip arp inspection limit rate 8
!
interface GigabitEthernet1/0/3
 ip dhcp snooping limit rate 2
 ip arp inspection limit rate 8 burst interval 4
```

Example 12-10 lists output that confirms the configuration settings. For instance, Example 12-9 configures port G1/0/2 with a rate of 8 messages for each (default) burst of 1 second; the output in Example 12-10 for interface G1/0/2 also lists a rate of 8 and burst interval of 1. Similarly, Example 12-9 configures port G1/0/3 with a rate of 8 over a burst of 4 seconds, with Example 12-10 confirming those same values for port G1/0/3. Note that the other two interfaces in Example 12-10 show the default settings of a rate of 15 messages over a one-second burst.

**Example 12-10** *Confirming ARP Inspection Rate Limits*

```
SW2# show ip arp inspection interfaces
 Interface        Trust State     Rate (pps)    Burst Interval
 ---------------  -----------     ----------    --------------
 Gi1/0/1          Untrusted               15                 1
 Gi1/0/2          Trusted                  8                 1
 Gi1/0/3          Untrusted                8                 4
 Gi1/0/4          Untrusted               15                 1
! Lines omitted for brevity
```

## Configuring Optional DAI Message Checks

As mentioned in the section titled "Dynamic ARP Inspection Logic," DAI always checks the ARP message's sender MAC and sender IP address fields versus some table in the switch, but it can also perform other checks. Those checks require more CPU, but they also help prevent other types of attacks.

Example 12-11 shows how to configure those three additional checks. Note that you can configure one, two, or all three of the options: just configure the **ip arp inspection validate** command again with all the options you want in one command, and it replaces the previous global configuration command. The example shows the three options, with the **src-mac** (source mac) option configured.

**Example 12-11** *Confirming ARP Inspection Rate Limits*

```
SW2# configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.

SW2(config)# ip arp inspection validate ?
  dst-mac  Validate destination MAC address
  ip       Validate IP addresses
  src-mac  Validate source MAC address

SW2(config)# ip arp inspection validate src-mac
SW2(config)# ^Z
SW2#
SW2# show ip arp inspection

Source Mac Validation      : Enabled
Destination Mac Validation : Disabled
IP Address Validation      : Disabled
```

## IP ARP Inspection Configuration Summary

The following configuration checklist summarizes the commands included in this section about how to configure Dynamic IP ARP Inspection:

Key Topic

Config Checklist

**Step 1.** Use the **ip arp inspection vlan** *vlan-list* global command to enable Dynamic ARP Inspection (DAI) on the switch for the specified VLANs.

**Step 2.** Separate from the DAI configuration, also configure DHCP Snooping and/or ARP ACLs for use by DAI.

**Step 3.** Configure the **ip arp inspection trust** interface subcommand to override the default setting of not trusted.

**Step 4.** **(Optional):** Configure DAI rate limits and err-disabled recovery:

- **Step A.** **(Optional):** Configure the **ip arp inspection limit rate** *number* [**burst interval** *seconds*] interface subcommand to set a limit of ARP messages per second, or ARP messages for each configured interval.
- **Step B.** **(Optional):** Configure the **ip arp inspection limit rate none** interface subcommand to disable rate limits.
- **Step C.** **(Optional):** Configure the **errdisable recovery cause arp-inspection** global command to enable the feature of automatic recovery from err-disabled mode, assuming the switch placed the port in err-disabled state because of exceeding DAI rate limits.
- **Step D.** **(Optional):** Configure the **errdisable recovery interval** *seconds* global commands to set the time to wait before recovering from an interface err-disabled state (regardless of the cause of the err-disabled state).

**Step 5.** **(Optional):** Configure the **ip arp inspection validate** {[**dst-mac**] [**src-mac**] [**ip**]} global command to enable optional items to validate with DAI on untrusted ports.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 12-2 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 12-2** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review config checklists | | Book, website |

## Review All the Key Topics

Key Topic

**Table 12-3** Key Topics for Chapter 12

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 12-4 | DHCP filtering actions on trusted and untrusted ports | 243 |
| List | DHCP Snooping logic | 243 |
| Figure 12-6 | DHCP Snooping Binding Table Concept | 244 |
| Example 12-1 | DHCP Snooping configuration | 246 |
| List | DHCP Snooping configuration checklist | 249 |
| Figure 12-10 | Detail inside ARP messages with sender and target | 251 |
| List | Gratuitous ARP details | 251 |
| Figure 12-13 | Core Dynamic ARP Inspection logic | 253 |
| Example 12-6 | Dynamic ARP Inspection configuration with associated DHCP Snooping configuration | 255 |
| List | Dynamic ARP Inspection checklist | 259 |

## Key Terms You Should Know

ARP reply, DHCP Snooping, DHCP Snooping binding table, Dynamic ARP Inspection, gratuitous ARP, (ARP) sender hardware address, (ARP) sender IP address, (ARP) sender protocol address, trusted port, untrusted port

## Command References

Tables 12-4 and 12-5 list the configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 12-4** Chapter 12 Configuration Command Reference

| Command | Mode/Purpose/Description |
|---|---|
| **ip dhcp snooping** | Global command that enables DHCP Snooping if combined with enabling it on one or more VLANs |
| **ip dhcp snooping vlan** *vlan-list* | Global command that lists VLANs on which to enable DHCP Snooping, assuming the **ip dhcp snooping** command is also configured |
| [no] **ip dhcp snooping information option** | Command that enables (or disables with **no** option) the feature of inserting DHCP option 82 parameters by the switch when also using DHCP Snooping |
| [no] **ip dhcp snooping trust** | Interface subcommand that sets the DHCP Snooping trust state for an interface (default **no**, or untrusted) |
| **ip dhcp snooping limit rate** *number* | Interface subcommand that sets a limit to the number of incoming DHCP messages processed on an interface, per second, before DHCP Snooping discards all other incoming DHCP messages in that same second |
| **err-disable recovery cause dhcp-rate-limit** | Global command that enables the switch to automatically recover an err-disabled interface if set to that state because of exceeding a DHCP rate limit setting |
| **err-disable recovery interval** *seconds* | Global command that sets the number of seconds IOS waits before recovering any err-disabled interfaces which, per various configuration settings, should be recovered automatically |
| **err-disable recovery cause arp-inspection** | Global command that enables the switch to automatically recover an err-disabled interface if set to that state because of an ARP Inspection violation |
| **ip arp inspection vlan** *vlan-list* | Global command to enable Dynamic ARP Inspection (DAI) on the switch for the specified VLANs |
| **ip arp inspection trust** | Interface subcommand to override the default setting of not trusted |
| **ip arp inspection limit rate** *number* [**burst interval** *seconds*] | Interface subcommand to set a limit of ARP messages per second, or ARP messages for each configured interval |
| **ip arp inspection limit rate none** | Interface subcommand to disable rate limits |
| **ip arp inspection validate** {[**dst-mac**] [**src-mac**] [**ip**]} | Global command to enable optional items to validate with DAI on untrusted ports |

**Table 12-5** Chapter 12 EXEC Command Reference

| Command | Purpose |
|---|---|
| **show ip dhcp snooping** | Lists a large variety of DHCP Snooping configuration settings |
| **show ip dhcp snooping statistics** | Lists counters regarding DHCP Snooping behavior on the switch |
| **show ip dhcp snooping binding** | Displays the contents of the dynamically created DHCP Snooping binding table |

| Command | Purpose |
|---|---|
| **show ip arp inspection** | Lists both configuration settings for Dynamic ARP Inspection (DAI) as well as counters for ARP messages processed and filtered |
| **show ip arp inspection statistics** | Lists the subset of the **show ip arp inspection** command output that includes counters |
| **show ip arp inspection interfaces** | Lists one line per DAI-enabled interface, listing trust state and rate limit settings |

*This page intentionally left blank*



# Part III Review

Keep track of your part review progress with the checklist shown in Table P3-1. Details on each task follow the table.

**Table P3-1** Part 3 Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Review Videos | | |

## Repeat All DIKTA Questions

For this task, use the PTP software to answer the "Do I Know This Already?" questions again for the chapters in this part of the book.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Do Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Blog Config Labs:** The author's blog (https://www.certskills.com) includes a series of configuration-focused labs that you can do on paper, each in 10–15 minutes. Review and performs labs for this part of the book by using the menus to navigate to the per-chapter content and then finding all config labs related to that chapter. (You can see more detailed instructions at https://www.certskills.com/config-labs.)

**Other:** If using other lab tools, here are a few suggestions: make sure to experiment with the variety of configuration topics in this part, including router and switch passwords, switch port security, Dynamic ARP Inspection, and DHCP Snooping.

## Watch Videos

Two chapters in this part mention videos included as extra material related to those chapters. Check out the reference in Chapter 9 to a video about using RADIUS protocol, as well as Chapter 10's reference to a video about troubleshooting switch port security.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.



Part IV shifts to a variety of topics that can be found in most every network. None are required for a network to work, but many happen to be useful services. Most happen to use IP or support the IP network in some way, so Part IV groups the topics together as IP services.

Part IV begins and ends with chapters that examine a series of smaller topics. First, Chapter 13 examines several IP services for which the CCNA 200-301 version 1.1 exam blueprint requires you to develop configuration and verification skills. Those services include logging and syslog, the Network Time Protocol (NTP), as well as two related services: CDP and LLDP.

The next two chapters in Part IV also focus on IP-based services, beginning with Chapter 14's examination of Network Address Translation (NAT). Almost every network uses NAT with IPv4, although in many cases, the firewall implements NAT. This chapter shows how to configure and verify NAT in a Cisco router.

Chapter 15 at first may give the appearance of a large chapter about one topic—Quality of Service—and it does focus on QoS. However, QoS by nature includes a wide variety of individual QoS tools. This chapter walks you through the basic concepts of the primary QoS features.

Chapters 16 and 17, at the end of Part IV, close with another series of smaller topics that require only conceptual knowledge rather than configuration skills. Chapter 16 covers First Hop Redundancy Protocols (FHRPs), while Chapter 17 covers Simple Network Management Protocol (SNMP), and two related protocols: TFTP and FTP.

# Part IV

## IP Services

**Chapter 13:** Device Management Protocols

**Chapter 14:** Network Address Translation

**Chapter 15:** Quality of Service (QoS)

**Chapter 16:** First Hop Redundancy Protocols

**Chapter 17:** SNMP, FTP, and TFTP

**Part IV Review**



CHAPTER 13

# Device Management Protocols

**This chapter covers the following exam topics:**

**2.0 Network Access**

**2.3 Configure and verify Layer 2 discovery protocols (Cisco Discovery Protocol and LLDP)**

**4.0 IP Services**

**4.2 Configure and verify NTP operating in a client and server mode**

**4.5 Describe the use of syslog features including facilities and severity levels**

This chapter begins Part IV with a discussion of the concepts, configuration, and verification of three functions found on Cisco routers and switches. These functions focus more on managing the network devices themselves than on managing the network that devices create.

The first major section of this chapter focuses on log messages and syslog. Most computing devices have a need to notify the administrator of any significant issue; generally, across the world of computing, messages of this type are called log messages. Cisco devices generate log messages as well. The first section shows how a Cisco device handles those messages and how you can configure routers and switches to ignore the messages or save them in different ways.

Next, different router and switch functions benefit from synchronizing their time-of-day clocks. Like most every computing device, routers and switches have an internal clock function to keep time. Network Time Protocol (NTP) provides a means for devices to synchronize their time, as discussed in the second section.

The final major section focuses on two protocols that do the same kinds of work: Cisco Discovery Protocol (CDP) and Link Layer Discovery Protocol (LLDP). Both provide a means for network devices to learn about neighboring devices, without requiring that IPv4 or IPv6 be working at the time.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 13-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| System Message Logging (Syslog) | 1, 2 |
| Network Time Protocol (NTP) | 3, 4 |
| Analyzing Topology Using CDP and LLDP | 5, 6 |

1. What level of logging to the console is the default for a Cisco device?
   a. Informational
   b. Errors
   c. Warnings
   d. Debugging
2. What command limits the messages sent to a syslog server to levels 4 through 0?
   a. **logging trap 0-4**
   b. **logging trap 0,1,2,3,4**
   c. **logging trap 4**
   d. **logging trap through 4**
3. Which of the following is accurate about the NTP client function on a Cisco router?
   a. The client synchronizes its time-of-day clock based on the NTP server.
   b. It counts CPU cycles of the local router CPU to keep time more accurately.
   c. The client synchronizes all Ethernet interfaces to use the same speed.
   d. The client must be connected to the same subnet as an NTP server.
4. The only NTP configuration on router R1 is the **ntp server 10.1.1.1** command. Which answer describes how NTP works on the router?
   a. As an NTP server only
   b. As an NTP client only
   c. As an NTP server only after the NTP client synchronizes with NTP server 10.1.1.1
   d. As an NTP server regardless of whether the NTP client synchronizes with NTP server 10.1.1.1
5. Imagine that a switch connects through an Ethernet cable to a router, and the router's host name is Hannah. Which of the following commands could tell you information about the IOS version on Hannah without establishing a Telnet connection to Hannah? (Choose two answers.)
   a. **show neighbors Hannah**
   b. **show cdp**
   c. **show cdp neighbors**
   d. **show cdp neighbors Hannah**
   e. **show cdp entry Hannah**
   f. **show cdp neighbors detail**

6. A switch is cabled to a router whose host name is Hannah. Which of the following LLDP commands could identify Hannah's enabled capabilities? (Choose two answers.)
   a. show neighbors
   b. show neighbors Hannah
   c. show lldp
   d. show lldp interface
   e. show lldp neighbors
   f. show lldp entry Hannah

# Foundation Topics

## System Message Logging (Syslog)

It is amazing just how helpful Cisco devices try to be to their administrators. When major (and even not-so-major) events take place, these Cisco devices attempt to notify administrators with detailed system messages. As you learn in this section, these messages vary from the mundane to those that are incredibly important. Thankfully, administrators have a large variety of options for storing these messages and being alerted to those that could have the largest impact on the network infrastructure.

When an event happens that the device's OS thinks is interesting, how does the OS notify us humans? Cisco IOS can send the messages to anyone currently logged in to the device. It can also store the message so that a user can later look at the messages. The next few pages examine both topics.

**NOTE** Included in the CCNA 200-301 exam topics is one about logging and syslog: "Describe the use of syslog features including facilities and severity levels." This exam topic does not require you to understand the related configuration. However, the configuration reveals many of the core concepts, so this section includes the configuration details as a means to help you understand how logging and syslog work.

### Sending Messages in Real Time to Current Users

Cisco IOS running on a device at least tries to allow current users to see **log messages** when they happen. Not every router or switch may have users connected, but if some user is logged in, the router or switch benefits by making the network engineer aware of any issues.

By default, IOS shows log messages to console users for all severity levels of messages. That default happens because of the default **logging console** global configuration command. In fact, if you have been using a console port throughout your time reading this book, you likely have already noticed many syslog messages, like messages about interfaces coming up or going down.

For other users (that is, Telnet and SSH users), the device requires a two-step process before the user sees the messages. First, IOS has another global configuration setting—**logging monitor**—that tells IOS to enable the sending of log messages to all connected users. However, that default configuration is not enough to allow the user to see the log messages. The user must also issue the **terminal monitor** EXEC command during the login session, which tells IOS that this terminal session would like to receive log messages.

Figure 13-1 summarizes these key points about how IOS on a Cisco router or switch processes log messages for currently connected users. In the figure, user A sits at the console and always receives log messages. On the right, the fact that user B sees messages (because user B issued the **terminal monitor** EXEC command after login) and user C does not shows that users can individually control whether or not they receive log messages.

Key Topic

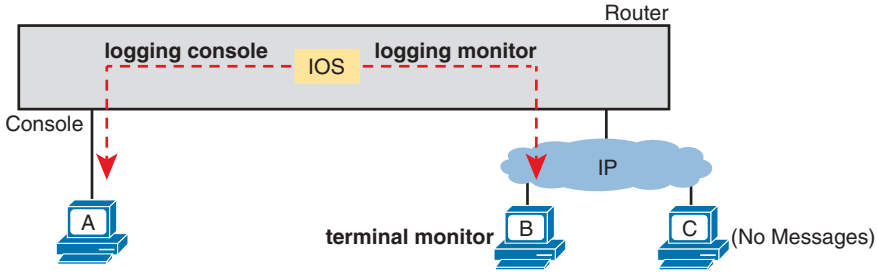



**Figure 13-1** *IOS Processing for Log Messages to Current Users*

## Storing Log Messages for Later Review

With logging to the console and to terminals, when an event happens, IOS sends the messages to the console and terminal sessions, and then IOS can discard the message. However, clearly, it would be useful to keep a copy of the log messages for later review, so IOS provides two primary means to keep a copy.

IOS can store copies of the log messages in RAM by virtue of the **logging buffered** global configuration command. Then any user can come back later and see the old log messages by using the **show logging** EXEC command.

As a second option—an option used frequently in production networks—all devices store their log messages centrally to a **syslog server**. RFC 5424 defines the Syslog protocol, which provides the means by which a device like a switch or router can use a UDP protocol to send messages to a syslog server for storage. All devices can send their log messages to the server. Later, a user can connect to the server (typically with a graphical user interface) and browse the log messages from various devices. To configure a router or switch to send log messages to a syslog server, add the **logging host** {*address* | *hostname*} global command, referencing the IP address or host name of the syslog server.

Figure 13-2 shows the ideas behind the buffered logging and syslog logging.

Key Topic

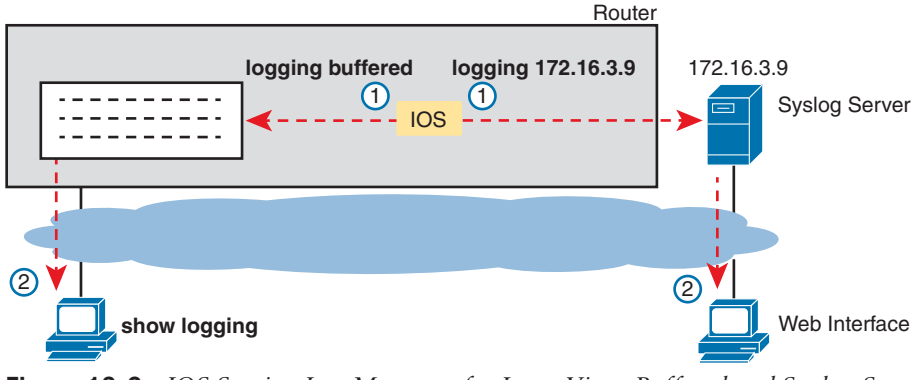



**Figure 13-2** *IOS Storing Log Messages for Later View: Buffered and Syslog Server*

## Log Message Format

IOS defines the format of log messages. The message begins with some data fields about the message, followed by some text more easily read by humans. For example, take a close look at this sample message:

```
*Dec 18 17:10:15.079: %LINEPROTO-5-UPDOWN: Line protocol on Interface
FastEthernet0/0, changed state to down
```

Notice that by default on this particular device, we see the following:

**A timestamp:** *Dec 18 17:10:15.079

**The facility on the router that generated the message:** %LINEPROTO

**The severity level:** 5

**A mnemonic for the message:** UPDOWN

**The description of the message:** Line protocol on Interface FastEthernet0/0, changed state to down

IOS dictates most of the contents of the messages, but you can at least toggle on and off the use of the timestamp (which is included by default) and a log message sequence number (which is not enabled by default). Example 13-1 reverses those defaults by turning off timestamps and turning on sequence numbers.

**Example 13-1** *Disabling Timestamps and Enabling Sequence Numbers in Log Messages*

```
R1(config)# no service timestamps
R1(config)# service sequence-numbers
R1(config)# end
R1#
000011: %SYS-5-CONFIG_I: Configured from console by console
```

To see the change in format, look at the log message at the end of the example. As usual, when you exit configuration mode, the device issues yet another log message. Comparing this message to the previous example, you can see it now no longer lists the time of day but does list a sequence number.

## Log Message Severity Levels

Log messages may just tell you about some mundane event, or they may tell you of some critical event. To help you make sense of the importance of each message, IOS assigns each message a severity level (as noted in the same messages in the preceding page or so). Figure 13-3 shows the severity levels: the lower the number, the more severe the event that caused the message. (Note that IOS commands use both the keywords and numbers from the figure.)

---

Answers to the "Do I Know This Already?" quiz:

**1** D **2** C **3** A **4** C **5** E, F **6** E, F

Key Topic

| Keyword | Numeral | Description | |
|---|---|---|---|
| Emergency | 0 | System unusable | Severe |
| Alert | 1 | Immediate action required | |
| Critical | 2 | Critical Event (Highest of 3) | Impactful |
| Error | 3 | Error Event (Middle of 3) | |
| Warning | 4 | Warning Event (Lowest of 3) | |
| Notification | 5 | Normal, More Important | Normal |
| Informational | 6 | Normal, Less Important | |
| Debug | 7 | Requested by User Debug | Debug |

**Figure 13-3** *Syslog Message Severity Levels by Keyword and Numeral*

Figure 13-3 breaks the eight severity levels into four sections just to make a little more sense of the meaning. The two top levels in the figure are the most severe. Messages from this level mean a serious and immediate issue exists. The next three levels, called Critical, Error, and Warning, also tell about events that impact the device, but they are not as immediate and severe. For instance, one common log message about an interface failing to a physically down state shows as a severity level 3 message.

Continuing down the figure, IOS uses the next two levels (5 and 6) for messages that are more about notifying the user rather than identifying errors. Finally, the last level in the figure is used for messages requested by the **debug** command, as shown in an example later in this chapter.

Table 13-2 summarizes the configuration commands used to enable logging and to set the severity level for each type. When the severity level is set, IOS will send messages of that severity level and more severe (lower severity numbers) to the service identified in the command. For example, the command **logging console 4** causes IOS to send severity level 0–4 messages to the console. Also, note that the command to disable each service is the **no** version of the command, with *no* in front of the command (**no logging console**, **no logging monitor**, and so on).

Key Topic

**Table 13-2** How to Configure Logging Message Levels for Each Log Service

| Service | To Enable Logging | To Set Message Levels |
|---|---|---|
| Console | **logging console** | **logging console** *level-name* \| *level-number* |
| Monitor | **logging monitor** | **logging monitor** *level-name* \| *level-number* |
| Buffered | **logging buffered** | **logging buffered** *level-name* \| *level-number* |
| Syslog | **logging host** *address* \| *hostname* | **logging trap** *level-name* \| *level-number* |

## Configuring and Verifying System Logging

With the information in Table 13-2, configuring syslog in a Cisco IOS router or switch should be relatively straightforward. Example 13-2 shows a sample, based on Figure 13-4. The figure shows a syslog server at IP address 172.16.3.9. Both switches and both routers will use the same configuration shown in Example 13-2, although the example shows the configuration process on a single device, router R1.

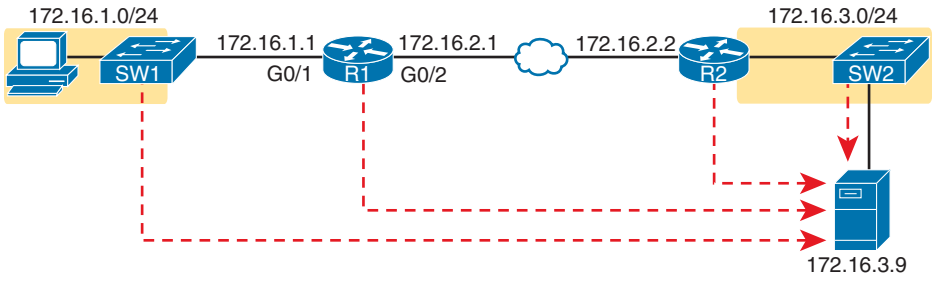



**Figure 13-4** *Sample Network Used in Logging Examples*

**Example 13-2** *Syslog Configuration on R1*

```
logging console 7
logging monitor debug
logging buffered 4
logging host 172.16.3.9
logging trap warning
```

First, note that the example configures the same message level at the console and for terminal monitoring (level 7, or debug), and the same level for both buffered and logging to the syslog server (level 4, or warning). The levels may be set using the numeric severity level or the name as shown earlier in Figure 13-3.

The **show logging** command confirms those same configuration settings and also lists the log messages per the logging buffered configuration. Example 13-3 shows a sample, with the configuration settings to match Example 13-2 highlighted in gray.

**Example 13-3** *Viewing the Configured Log Settings per the Earlier Example*

```
R1# show logging
Syslog logging: enabled (0 messages dropped, 3 messages rate-limited, 0 flushes, 0
overruns, xml disabled, filtering disabled)

No Active Message Discriminator.

No Inactive Message Discriminator.

    Console logging: level debugging, 45 messages logged, xml disabled,
                     filtering disabled
    Monitor logging: level debugging, 0 messages logged, xml disabled,
                     filtering disabled
    Buffer logging: level warnings, 0 messages logged, xml disabled,
                    filtering disabled
    Exception Logging: size (8192 bytes)
    Count and timestamp logging messages: disabled
    Persistent logging: disabled
No active filter modules.
```

```
    Trap logging: level warnings, 0 message lines logged
       Logging to 172.16.3.9 (udp port 514, audit disabled,
            link up),
            0 message lines logged,
            0 message lines rate-limited,
            0 message lines dropped-by-MD,
            xml disabled, sequence number disabled
            filtering disabled
       Logging Source-Interface: VRF Name:
    TLS Profiles:

Log Buffer (8192 bytes):
```

You might notice by now that knowing the names of all eight log message levels can be handy if you want to understand the output of the commands. Most of the **show** commands list the log message levels by name, not by number. As you can see in the gray highlights in this example, two levels list "debug," and two list "warning," even though some of the configuration commands referred to those levels by number.

Also, you cannot know this from the output, but in Example 13-3, router R1 has no buffered log messages. (Note the counter value of 0 for buffered logging messages.) If any log messages had been buffered, the actual log messages would be listed at the end of the command. In this case, I had just booted the router, and no messages had been buffered yet. (You could also clear out the old messages from the log with the **clear logging** EXEC command.)

The next example shows the difference between the current severity levels. This example shows the user disabling interface G0/1 on R1 with the **shutdown** command and then re-enabling it with the **no shutdown** command. If you look closely at the highlighted messages, you will see several severity 5 messages and one severity 3 message. The **logging buffered 4** global configuration command on R1 (see Example 13-2) means that R1 will not buffer the severity level 5 log messages, but it will buffer the severity level 3 message. Example 13-4 ends by showing that log message at the end of the output of the **show logging** command.

**Example 13-4** *Seeing Severity 3 and 5 Messages at the Console, and Severity 3 Only in the Buffer*

```
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# interface g0/1
R1(config-if)# shutdown
R1(config-if)#
*Oct 21 20:07:07.244: %LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state
  to administratively down
*Oct 21 20:07:08.244: %LINEPROTO-5-UPDOWN: Line protocol on Interface
  GigabitEthernet0/1, changed state to down
R1(config-if)# no shutdown
R1(config-if)#
*Oct 21 20:07:24.312: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to up
*Oct 21 20:07:25.312: %LINEPROTO-5-UPDOWN: Line protocol on Interface
  GigabitEthernet0/1, changed state to up
```

```
R1(config-if)# ^Z
R1#
*Oct 21 20:07:36.546: %SYS-5-CONFIG_I: Configured from console by console
R1# show logging
! Skipping about 20 lines, the same lines in Example 13-3, until the last few lines

Log Buffer (8192 bytes):

*Oct 21 20:07:24.312: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to up
```

## The debug Command and Log Messages

Of the eight log message severity levels, one level, debug level (7), has a special purpose: for messages generated as a result of a user logged in to the router or switch who issues a **debug** command.

The **debug** EXEC command gives the network engineer a way to ask IOS to monitor for certain internal events, with that monitoring process continuing over time, so that IOS can issue log messages when those events occur. The engineer can log in, issue the **debug** command, and move on to other work. The user can even log out of the device, and the debug remains enabled. IOS continues to monitor the request in that **debug** command and generate log messages about any related events. The debug remains active until some user issues the **no debug** command with the same parameters, disabling the debug.

**NOTE** While the **debug** command is just one command, it has a huge number of options, much like the **show** command may be one command, but it also has many, many options.

The best way to see how the **debug** command works, and how it uses log messages, is to see an example. Example 13-5 shows a sample debug of OSPF Hello messages for router R1 in Figure 13-4. The router (R1) enables OSPF on two interfaces and has established one OSPF neighbor relationship with router R2 (RID 2.2.2.2). The debug output shows one log message for the Hello messages sent on ports G0/1 and G0/2, plus one message for a received Hello arriving in port G0/2.

**Example 13-5** *Using* **debug ip ospf hello** *from R1's Console*

```
R1# debug ip ospf hello
OSPF hello debugging is on
R1#
*Aug 10 13:38:19.863: OSPF-1 HELLO Gi0/1: Send hello to 224.0.0.5 area 0 from
  172.16.1.1
*Aug 10 13:38:21.199: OSPF-1 HELLO Gi0/2: Rcv hello from 2.2.2.2 area 0 172.16.2.2
*Aug 10 13:38:22.843: OSPF-1 HELLO Gi0/2: Send hello to 224.0.0.5 area 0 from
  172.16.2.1
R1#
```

The console user sees the log messages created on behalf of that **debug** command after the **debug** command completes. Per the earlier configuration in Example 13-2, R1's **logging**

**console 7** command tells us that the console user will receive severity levels 0–7, which includes level 7 debug messages. Also, if you examine the log messages generated by the **debug** command, you will not find the severity level listed. So, the absence of a severity level number in a log message points to the message coming from the **debug** command.

Many companies may not want to keep copies of debug messages with the longer-term options to store log messages. For instance, with the current settings per Example 13-2, these debug messages would not be in the local log message buffer (because of the level in the **logging buffered warning** command). The router would also not send debug messages to the syslog server (because of the level in the **logging trap 4** command).

Note that the console user automatically sees the log messages as shown in Example 13-4. However, as noted in the text describing Figure 13-1, a user who connects to R1 using SSH or Telnet would need to also issue the **terminal monitor** command, even with the **logging monitor debug** command configured on router R1.

Note that all enabled debug options use router CPU, which can cause problems for the router. You can monitor CPU use with the **show process cpu** command, but you should use caution when using **debug** commands on production devices. Also, note the more CLI users who receive debug messages, the more CPU is consumed. So, some installations choose not to include debug-level log messages for console and terminal logging, requiring users to look at the logging buffer or syslog for those messages, just to reduce router CPU load.

## Network Time Protocol (NTP)

Each networking device has some concept of a date and a time-of-day clock. For instance, the log messages discussed in the first major section of this chapter had a timestamp with the date and time of day listed. Now imagine looking at all the log messages from all routers and switches stored at a syslog server. All those messages have a date and timestamp, but how do you make sure the timestamps are consistent? How do you make sure that all devices synchronize their time-of-day clocks so that you can make sense of all the log messages at the syslog server? How could you make sense of the messages for an event that impacted devices in three different time zones?

For example, consider the messages on two routers, R1 and R2, as shown in Example 13-6. Routers R1 and R2 do not synchronize their NTP clocks. A problem keeps happening on the Ethernet WAN link between the two routers. A network engineer looks at all the log messages as stored on the syslog server. However, when seeing some messages from R1, at 13:38:39 (around 1:40 p.m.), the engineer does not think to look for messages from R2 that have a timestamp of around 9:45 a.m.

**Example 13-6** *Log Messages from Two Routers, Compared*

```
*Oct 19 13:38:37.568: %OSPF-5-ADJCHG: Process 1, Nbr 2.2.2.2 on GigabitEthernet0/2
from FULL to DOWN, Neighbor Down: Interface down or detached
*Oct 19 13:38:40.568: %LINEPROTO-5-UPDOWN: Line protocol on Interface
GigabitEthernet0/2, changed state to down
```

```
! These messages happened on router R2
Oct 19 09:44:09.027: %LINK-3-UPDOWN: Interface GigabitEthernet0/1, changed state to down
Oct 19 09:44:09.027: %OSPF-5-ADJCHG: Process 1, Nbr 1.1.1.1 on GigabitEthernet0/1
from FULL to DOWN, Neighbor Down: Interface down or detached
```

In reality, the messages in both parts of Example 13-6 happened within 0.5 second of each other because I issued a **shutdown** command on router R1 (not shown). However, the two routers' time-of-day clocks were not synchronized, which makes the messages on the two routers look unrelated. With synchronized clocks, the two routers would have listed practically identical timestamps of almost the exact same time when these messages occurred, making it much easier to read and correlate messages.

Routers, switches, other networking devices, and pretty much every device known in the IT world has a time-of-day clock. For a variety of reasons, it makes sense to synchronize those clocks so that all devices have the same time of day, other than differences in time zone. The **Network Time Protocol (NTP)** provides the means to do just that.

NTP gives any device a way to synchronize its time-of-day clocks. NTP provides protocol messages that devices use to learn the timestamp of other devices. Devices send timestamps to each other with NTP messages, continually exchanging messages, with one device changing its clock to match the other, eventually synchronizing the clocks. As a result, actions that benefit from synchronized timing, like the timestamps on log messages, work much better.

This section works through a progression of topics that lead to the more common types of NTP configurations seen in real networks. The section begins with basic settings, like the time zone and initially configured time on a router or switch, followed by basic NTP configuration. The text then examines some NTP internals regarding how NTP defines the sources of time data (reference clocks) and how good each time source is (stratum). The section closes with more configuration that explains typical enterprise configurations, with multiple **ntp** commands for redundancy and the use of loopback interfaces for high availability.

## Setting the Time and Time Zone

NTP's job is to synchronize clocks, but NTP works best if you set the device clock to a reasonably close time before enabling the **NTP client** function with the **ntp server** command. For instance, my wristwatch says 8:52 p.m. right now. Before starting NTP on a new router or switch so that it synchronizes with another device, I should set the time to 8:52 p.m., set the correct date and time zone, and even tell the device to adjust for daylight savings time—and then enable NTP. Setting the time correctly gives NTP a good start toward synchronizing.

Example 13-7 shows how to set the date, time, time zone, and daylight savings time. Oddly, it uses two configuration commands (for the time zone and daylight savings time) and one EXEC command to set the date and time on the router.

**Example 13-7** *Setting the Date/Time with* **clock set***, Plus Time Zone/DST*

```
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# clock timezone EST -5
R1(config)# clock summer-time EDT recurring
R1(config)# ^Z
R1#
R1# clock set 12:32:00 19 January 2023
*Jan 19 17:32:00.005: %SYS-6-CLOCKUPDATE: System clock has been updated from
12:32:16 EST Thu Jan 19 2023 to 12:32:00 EST Thu Jan 19 2023, configured from
console by console.
R1# show clock
12:32:06.146 EST Thu Jan 19 2023
```

Focus on the two configuration commands first. You should set the first two commands before setting the time of day with the **clock set** EXEC command because the two configuration commands impact the time that is set. In the first command, the **clock timezone** part defines the command and a keyword. The next parameter, "EST" in this case, is any value you choose, but choose the name of the time zone of the device. This value shows up in **show** commands, so although you make up the value, the value needs to be meaningful to all. I chose EST, the acronym for US Eastern Standard Time. The "-5" parameter means that this device is 5 hours behind Universal Time Coordinated (UTC).

The **clock summer-time** part of the second command defines what to do, again with the "EDT" being a field in which you could have used any value. However, you should use a meaningful value. This is the value shown with the time in **show** commands when daylight savings time is in effect, so I chose EDT because it is the acronym for daylight savings time in that same EST time zone. Finally, the **recurring** keyword tells the router to spring forward an hour and fall back an hour automatically over the years.

The **clock set** EXEC command then sets the time, day of the month, month, and year. However, note that IOS interprets the time as typed in the command in the context of the time zone and daylight savings time. In the example, the **clock set** command lists a time of 20:52:49 (the command uses a time syntax with a 24-hour format, not with a 12-hour format plus a.m./p.m.). As a result of that time plus the two earlier configuration commands, the **show clock** command (issued seconds later) lists that time but also notes the time as EDT rather than UTC time.

## Basic NTP Configuration

With NTP, servers supply information about the time of day to clients, and clients react by adjusting their clocks to match. The process requires repeated small adjustments over time to maintain that synchronization. The configuration itself can be simple, or it can be extensive once you add security configuration and redundancy.

Cisco supplies two **ntp** configuration commands that dictate how NTP works on a router or switch, as follows:




- **ntp master** {*stratum-level*}: NTP server mode—the device acts only as an **NTP server**, and not as an NTP client. The device gets its time information from the internal clock on the device.
- **ntp server** {*address* | *hostname*}: NTP client/server mode—the device acts as both client and server. First, it acts as an NTP client, to adjust its time to synchronize time with another server. Once synchronized, the device can then act as an NTP server, to supply time to other NTP clients.

For an example showing the basic configuration syntax and **show** commands, consider Figure 13-5. With this simple configuration:

- R3 acts as an NTP server only.
- R2 acts in client/server mode—first as an NTP client to synchronize time with NTP server R3, then as a server to supply time to NTP client R1.

- R1 acts in client/server mode—first as an NTP client to synchronize time with NTP server R2. (R1 will be willing to act as a server, but no devices happen to reference R1 as an NTP server in this example.)

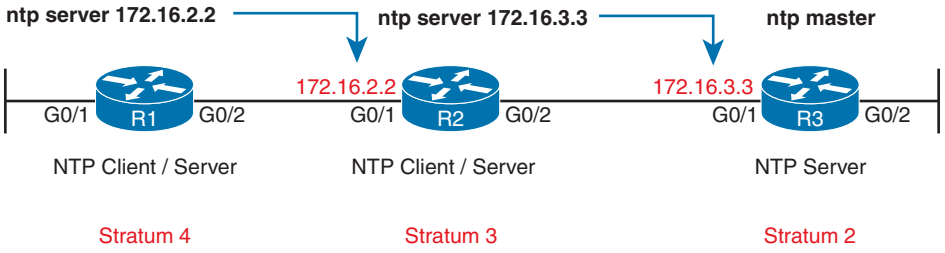



**Figure 13-5** *R1 as NTP Client, R2 as Client/Server, R3 as Server*

As you can see, NTP requires little configuration to make it work with a single configuration command on each device. Example 13-8 collects the configuration from the devices shown in the figure for easy reference.

**Example 13-8** *NTP Client/Server Configuration*

```
! Configuration on R1:
ntp server 172.16.2.2
```

```
! Configuration on R2:
ntp server 172.16.3.3
```

```
! Configuration on R3:
ntp master 2
```

Example 13-9 lists the output from the **show ntp status** command on R1, with the first line of output including a few important status items. First, it lists a status of synchronized, which confirms the NTP client has completed the process of changing its time to match the server's time. Any router acting as an NTP client will list "unsynchronized" in that first line until the **NTP synchronization** process completes with at least one server. It also confirms the IP address of the server—this device's *reference clock*—with the IP address configured in Example 13-8 (172.16.2.2).

**Example 13-9** *Verifying NTP Client Status on R1 and R2*

```
R1# show ntp status
Clock is synchronized, stratum 4, reference is 172.16.2.2
nominal freq is 250.0000 Hz, actual freq is 250.0000 Hz, precision is 2**21
ntp uptime is 1553800 (1/100 of seconds), resolution is 4000
reference time is DA5E7147.56CADEA7 (15:24:38.694 EST Thu Jan 19 2023)
clock offset is 0.0986 msec, root delay is 2.46 msec
root dispersion is 22.19 msec, peer dispersion is 5.33 msec
loopfilter state is 'CTRL' (Normal Controlled Loop), drift is 0.000000009 s/s
system poll interval is 64, last update was 530 sec ago.
```

Next, look at the **show ntp associations** command output from both R1 and R2, as shown in Example 13-10. This command lists all the NTP servers that the local device can attempt to use, with status information about the association between the local device (client) and the various NTP servers. Beginning with R1, note that it has one association (that is, relationship with an NTP server), based on the one **ntp server 172.16.2.2.2** configuration command on R1. The * means that R1 has successfully contacted the server. You will see similar data from the same command output taken from router R2.

**Example 13-10** *Verifying NTP Client Status on R1 and R2*

```
R1# show ntp associations
! This output is taken from router R1, acting in client/server mode
   address    ref clock st when poll reach  delay  offset disp
*~172.16.2.2 10.1.3.3  3   50  64     377  1.223  0.090  4.469
 * sys.peer, # selected, + candidate, - outlyer, x falseticker, ~ configured
```

```
 R2# show ntp associations
! This output is taken from router R2, acting in client/server mode
 address       ref clock     st  when poll  reach  delay  offset  disp
*~172.16.3.3   127.127.1.1   2   49    64    377    1.220  -7.758  3.695
 * sys.peer, # selected, + candidate, - outlyer, x falseticker, ~ configured
```

## NTP Reference Clock and Stratum

NTP servers must learn the time from some device. For devices acting in **NTP client/server mode**, the device uses the NTP client function to learn the time. However, devices that act solely as an NTP server get their time from either internal device hardware or from some external clock using mechanisms other than NTP.

For instance, when configured with the **ntp master** command, a Cisco router/switch uses its internal device hardware to determine the time. All computers, networking devices included, need some means to keep time for a myriad of reasons, so they include both hardware components and software processes to keep time even over periods in which the device loses power.

Additionally, NTP servers and clients use a number to show the perceived accuracy of their reference clock data based on stratum level. The lower the stratum level, the more accurate the reference clock is considered to be. An NTP server that uses its internal hardware or external reference clock sets its own stratum level. Then, an NTP client adds 1 to the stratum level it learns from its NTP server, so that the stratum level increases the more hops away from the original clock source.

For instance, back in Figure 13-5, you can see the NTP primary server (R3) with a stratum of 2. R2, which references R3, adds 1 so it has a stratum of 3. R1 uses R2 as its NTP server, so R1 adds 1 to have a stratum of 4. These increasing stratum levels allow devices to refer to several NTP servers and then use time information from the best NTP server, *best* being the server with the lowest stratum level.

Routers and switches use the default stratum level of 8 for their internal reference clock based on the default setting of 8 for the stratum level in the **ntp master** [*stratum-level*] command. The command allows you to set a value from 1 through 15; in Example 13-8, the **ntp master 2** command set router R3's stratum level to 2.

**NOTE** NTP considers 15 to be the highest useful stratum level, so any devices that calculate its stratum as 16 consider the time data unusable and do not trust the time. So, avoid setting higher stratum values on the **ntp master** command.

To see the evidence, refer back to Example 13-10, which shows two commands based on the same configuration in Example 13-8 and Figure 13-5. The output highlights details about reference clocks and stratum levels, as follows:

**R1:** Per the configured **ntp server 172.16.2.2** command, the **show** command lists the same address (which is router R2's address). The ref clock (reference clock) and st (stratum) fields represent R2's reference clock as 172.16.3.3—in other words, the NTP server R2 uses, which is R3 in this case. The st field value of 3 shows R2's stratum.

**R2:** Per the configured **ntp server 172.16.3.3** command, the **show** command lists 172.16.3.3, which is an address on router R3. The output notes R3's ref clock as 127.127.1.1—an indication that the server (R3) gets its clock internally. It lists R3's st (stratum) value of 2—consistent with the configured **ntp master 2** command on R3 (per Example 13-8).

On the NTP primary server itself (R3 in this case), the output has more markers indicating the use of the internal clock. Example 13-11 shows output from R3, with a reference clock of the 127.127.1.1 loopback address, used to refer to the fact that this router gets its clock data internally. Also, in the **show ntp associations** command output at the bottom, note that same address, along with a reference clock value of ".LOCL." In effect, R3, per the **ntp master** configuration command, has an association with its internal clock.

**Example 13-11** *Examining NTP Server, Reference Clock, and Stratum Data*

```
R3# show ntp status
Clock is synchronized, stratum 2, reference is 127.127.1.1
nominal freq is 250.0000 Hz, actual freq is 250.0000 Hz, precision is 2**20
ntp uptime is 595300 (1/100 of seconds), resolution is 4000
reference time is E0F9174C.87277EBB (15:27:54.252 EST Thu Jan 19 2023)
clock offset is 0.0000 msec, root delay is 0.00 msec
root dispersion is 0.33 msec, peer dispersion is 0.23 msec
loopfilter state is 'CTRL' (Normal Controlled Loop), drift is 0.000000000 s/s
system poll interval is 16, last update was 8 sec ago.

R3# show ntp associations
  address         ref clock       st   when   poll reach  delay  offset   disp
*~127.127.1.1     .LOCL.           1     15     16   377  0.000   0.000  0.232
 * sys.peer, # selected, + candidate, - outlyer, x falseticker, ~ configured
```

**NOTE** Appendix D, "Topics from Previous Editions," includes a few pages about adding redundancy to NTP. For those moving on to CCNP, you might want to read that topic when convenient. You can find Appendix D on this book's companion website.

# Analyzing Topology Using CDP and LLDP

The first two major sections of this chapter showed two features—syslog and NTP—that work the same way on both routers and switches. This final section shows yet another feature common to both routers and switches, with two similar protocols: the Cisco Discovery Protocol (**CDP**) and the Link Layer Discovery Protocol (**LLDP**). This section focuses on CDP, followed by LLDP.

## Examining Information Learned by CDP

CDP discovers basic information about neighboring routers and switches without needing to know the passwords for the neighboring devices. To discover information, routers and switches send CDP messages out each of their interfaces. The messages essentially announce information about the device that sent the CDP message. Devices that support CDP learn information about others by listening for the advertisements sent by other devices.

CDP discovers several useful details from the neighboring Cisco devices:

Key Topic

- **Device identifier:** Typically the hostname
- **Address list:** Network and data-link addresses
- **Port identifier:** The interface on the remote router or switch on the other end of the link that sent the CDP advertisement
- **Capabilities list:** Information on what type of device it is (for example, a router or a switch)
- **Platform:** The model and OS level running on the device

CDP plays two general roles: to provide information to the devices to support some function and to provide information to the network engineers who manage the devices. For example, Cisco IP Phones use CDP to learn the data and voice VLAN IDs as configured on the access switch. For that second role, CDP has **show** commands that list information about neighboring devices, as well as information about how CDP is working. Table 13-3 describes the three **show** commands that list the most important CDP information.

Key Topic

**Table 13-3 show cdp Commands That List Information About Neighbors**

| Command | Description |
|---|---|
| **show cdp neighbors** [*type number*] | Lists one summary line of information about each neighbor or just the neighbor found on a specific interface if an interface was listed |
| **show cdp neighbors detail** | Lists one large set (approximately 15 lines) of information, one set for every neighbor |
| **show cdp entry** *name* | Lists the same information as the **show cdp neighbors detail** command, but only for the named neighbor (case sensitive) |

**NOTE** Cisco routers and switches support the same CDP commands, with the same parameters and same types of output.

The next example shows the power of the information in CDP commands. The example uses the network shown in Figure 13-6, with Example 13-12 listing the output of several **show cdp** commands.

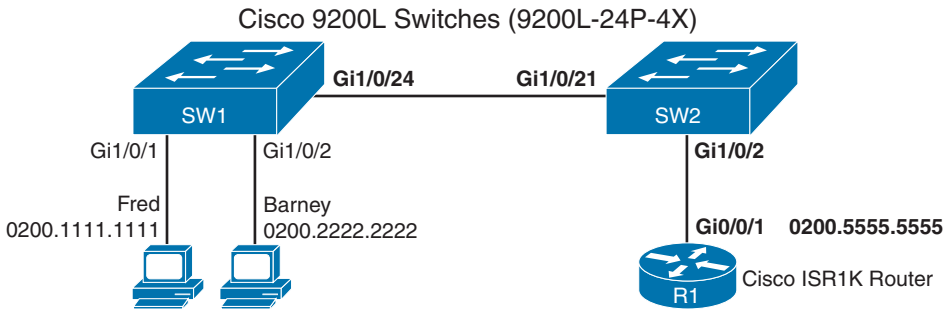



**Figure 13-6** *Small Network Used in CDP Examples*

**Example 13-12** **show cdp neighbors** *Command Examples: SW2*

```
SW2# show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID        Local Intrfce     Holdtme    Capability  Platform  Port ID
SW1              Gig 1/0/21        155              S I   WS-C2960X Gig 1/0/24
R1               Gig 1/0/2         131            R S I   C1111-8P  Gig 0/0/1

Total cdp entries displayed : 2
```

The **show cdp neighbors** command lists one line per neighbor. (Look for the Device ID column and the list that includes SW1 and R1.) Each of those two lines lists the most important topology information about each neighbor: the neighbor's hostname (Device ID), the local device's interface, and the neighboring device's interface (under the Port heading).

Pay close attention to the local device's interface and the neighboring device's interface, comparing the example to the figure. For example, SW2's **show cdp neighbors** command lists an entry for SW1, with SW2's local interface of Gi1/0/21 (listed under the heading "Local Intrfce") and SW1's interface of Gi1/0/24 (listed under the heading "Port ID").

This command also lists the platform, identifying the specific model of the neighboring router or switch. So, even using this basic information, you could either construct a figure like Figure 13-6 or confirm that the details in the figure are correct.

Figure 13-6 and Example 13-12 provide a good backdrop as to why devices learn about direct neighbors with CDP, but not other neighbors. First, CDP defines encapsulation that uses the data-link header, but no IP header. To ensure all devices receive a CDP message, the Ethernet header uses a multicast destination MAC address (0100.0CCC.CCCC). However, when any device that supports CDP receives a CDP message, the device processes the message and then discards it, rather than forwarding it. So, for instance, when router R1 sends a

CDP message to Ethernet multicast address 0100.0CCC.CCCC, switch SW2 receives it, processes it, but does not forward it to switch SW1—so SW1 will not list router R1 as a CDP neighbor. Likewise, R1 will not list SW1 as a neighbor.

Next, consider the **show cdp neighbors detail** command as shown in Example 13-13, again taken from switch SW2. This command lists more detail, as you might have guessed. The detail lists the full name of the switch model (C9200L-24P-4X) and the IP address configured on the neighboring device. You have to look closely, but the example has one long group of messages for each of the two neighbors, separated by a line of dashes.

**Example 13-13** **show cdp neighbors detail** *Command on SW2*

```
SW2# show cdp neighbors detail
-------------------------
Device ID: SW1
Entry address(es):
  IP address: 1.1.1.1
Platform: cisco C9200L-24P-4X,  Capabilities: Switch IGMP
Interface: GigabitEthernet1/0/21,  Port ID (outgoing port): GigabitEthernet1/0/24
Holdtime : 144 sec

Version :
Cisco IOS Software [Bengaluru], Catalyst L3 Switch Software (CAT9K_LITE_IOSXE),
Version 17.6.3, RELEASE SOFTWARE (fc4)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2022 by Cisco Systems, Inc.
Compiled Wed 30-Mar-22 21:23 by mcpre

advertisement version: 2
VTP Management Domain: 'fred'
Native VLAN: 1
Duplex: full
Management address(es):
  IP address: 1.1.1.1

-------------------------
Device ID: R1
Entry address(es):
  IP address: 10.12.25.5
Platform: cisco C1111-8P,  Capabilities: Router Switch IGMP
Interface: GigabitEthernet1/0/2,  Port ID (outgoing port): GigabitEthernet0/0/1
Holdtime : 151 sec

Version :
Cisco IOS Software [Fuji], ISR Software (ARMV8EB_LINUX_IOSD-UNIVERSALK9_IAS-M),
Version 16.8.1, RELEASE SOFTWARE (fc3)
Technical Support: http://www.cisco.com/techsupport
```

```
Copyright (c) 1986-2018 by Cisco Systems, Inc.
Compiled Tue 27-Mar-18 10:56 by mcpre

advertisement version: 2
VTP Management Domain: ‘’
Duplex: full
Management address(es):
  IP address: 10.12.25.5


Total cdp entries displayed : 2
```

**NOTE** The **show cdp entry** *name* command lists the exact same details shown in the output of the **show cdp neighbors detail** command, but for only the one neighbor listed in the command. The *name* parameter must be a neighbor's case-sensitive name.

## Configuring and Verifying CDP

Most of the work you do with CDP relates to what CDP can tell you with **show** commands. However, it is an IOS feature, so you can configure CDP and use some **show** commands to examine the status of CDP itself.

IOS typically enables CDP globally and on each interface by default. You can then disable CDP per interface with the **no cdp enable** interface subcommand and later re-enable it with the **cdp enable** interface subcommand. To disable and re-enable CDP globally on the device, use the **no cdp run** and **cdp run** global commands, respectively.

To examine the status of CDP itself, use the commands in Table 13-4.

**Table 13-4** Commands Used to Verify CDP Operations

| Command | Description |
|---|---|
| **show cdp** | States whether CDP is enabled globally and lists the default update and holdtime timers |
| **show cdp interface** [*type number*] | States whether CDP is enabled on each interface, or a single interface if the interface is listed, and states update and holdtime timers on those interfaces |
| **show cdp traffic** | Lists global statistics for the number of CDP advertisements sent and received |

Example 13-14 lists sample output from each of the commands in Table 13-4, based on switch SW2 in Figure 13-6.

**Example 13-14** show cdp *Commands That Show CDP Status*

```
SW2# show cdp
Global CDP information:
        Sending CDP packets every 60 seconds
        Sending a holdtime value of 180 seconds
        Sending CDPv2 advertisements is enabled

SW2# show cdp interface GigabitEthernet1/0/2
GigabitEthernet1/0/2 is up, line protocol is up
  Encapsulation ARPA
  Sending CDP packets every 60 seconds
  Holdtime is 180 seconds

SW2# show cdp traffic
CDP counters :
        Total packets output: 304, Input: 305
        Hdr syntax: 0, Chksum error: 0, Encaps failed: 0
        No memory: 0, Invalid packet: 0,
        CDP version 1 advertisements output: 0, Input: 0
        CDP version 2 advertisements output: 304, Input: 305
```

The first two commands in the example list two related settings about how CDP works: the send time and the hold time. CDP sends messages every 60 seconds by default, with a hold time of 180 seconds. The hold time tells the device how long to wait after no longer hearing from a device before removing those details from the CDP tables. You can override the defaults with the **cdp timer** *seconds* and **cdp holdtime** *seconds* global commands, respectively. Table 13-5 summarizes the configuration commands for CDP and LLDP timers for easier review and study.

**Table 13-5** CDP and LLDP Timer Configuration

| CDP Command | LLDP Command | Description |
|---|---|---|
| **cdp timer** *seconds* | **lldp timer** *seconds* | Defines how often CDP or LLDP sends messages on each interface |
| **cdp holdtime** *seconds* | **lldp holdtime** *seconds* | Defines how long to wait after the most recent incoming message from a neighbor before deleting that neighbor's information |
| N/A | **lldp reinit** *seconds* | (LLDP only) Defines a wait time before the first message sent after an interface comes up |

## Examining Information Learned by LLDP

Cisco created the Cisco-proprietary CDP before any standard existed for a similar protocol. CDP has many benefits. As a Layer 2 protocol, sitting on top of Ethernet, it does not rely on a working Layer 3 protocol. It provides device information that can be useful in a variety of ways. Cisco had a need but did not see a standard that met the need, so Cisco made up a protocol, as has been the case many times over history with many companies and protocols.

Link Layer Discovery Protocol (LLDP), defined in IEEE standard 802.1AB, provides a standardized protocol that provides the same general features as CDP. LLDP has similar configuration and practically identical **show** commands as compared with CDP.

The LLDP examples all use the same topology used in the CDP examples per Figure 13-6 (the same figure used in the CDP examples). Example 13-15 lists switch SW2's LLDP neighbors as learned after LLDP was enabled on all devices and ports in that figure. The example highlights the items that match the similar output from the **show cdp neighbors** command listed at the end of the example, also from switch SW2.

**Example 13-15** **show lldp neighbors** *on SW2 with Similarities to CDP Highlighted*

```
SW2# show lldp neighbors
Capability codes:
    (R) Router, (B) Bridge, (T) Telephone, (C) DOCSIS Cable Device
    (W) WLAN Access Point, (P) Repeater, (S) Station, (O) Other

Device ID           Local Intf     Hold-time  Capability      Port ID
R1                  Gi1/0/2        120        R               Gi0/0/1
SW1                 Gi1/0/21       120        B               Gi1/0/24

Total entries displayed: 2

SW2# show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID        Local Intrfce     Holdtme    Capability  Platform  Port ID
SW1              Gig 1/0/21        155              S I   WS-C2960X Gig 1/0/24
R1               Gig 1/0/2         131            R S I   C1111-8P  Gig 0/0/1
Total entries displayed: 2
```

The most important take-away from the output is the consistency between CDP and LLDP in how they refer to the interfaces. Both the **show cdp neighbors** and **show lldp neighbors** commands have "local intf" (interface) and "port ID" columns. These columns refer to the local device's interface and the neighboring device's interface, respectively.

However, the LLDP output in the example does differ from CDP in a few important ways:

Key Topic

- LLDP uses **B** as the capability code for switching, referring to **bridge**, a term for the device type that existed before switches that performed the same basic functions.
- LLDP does not identify IGMP as a capability, while CDP does (I).
- CDP lists the neighbor's **platform**, a code that defines the device type, while LLDP does not.
- LLDP lists capabilities with different conventions (see upcoming Example 13-19).

The last item in the list requires a closer look with more detail. Interestingly, CDP lists all capabilities supported by the neighbor in the **show cdp neighbors** command output, whether enabled or not. LLDP instead lists only the enabled (configured) capabilities, rather than all supported capabilities, in the output from the **show lldp neighbors** command.

The **show lldp neighbors detail** and **show lldp entry** *hostname* commands provide identical detailed output, with the first command providing detail for all neighbors, and the second providing detail for the single listed neighbor. Example 13-16 shows the detail for neighbor R1. Note that the parameter "R1" is case sensitive, and should match the name listed by the **show lldp neighbors** command.

**Example 13-16** **show lldp entry r2** *Command on SW2*

```
SW2# show lldp entry R1

Capability codes:
    (R) Router, (B) Bridge, (T) Telephone, (C) DOCSIS Cable Device
    (W) WLAN Access Point, (P) Repeater, (S) Station, (O) Other
------------------------------------------------
Local Intf: Gi1/0/2
Chassis id: 70ea.1a9a.d300
Port id: Gi0/0/1
Port Description: GigabitEthernet0/0/1
System Name: R1

System Description:
Cisco IOS Software [Fuji], ISR Software (ARMV8EB_LINUX_IOSD-UNIVERSALK9_IAS-M),
Version 16.8.1, RELEASE SOFTWARE (fc3)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2022 by Cisco Systems, Inc.
Compiled Fri 08-Apr-22 12:42 by mcp

Time remaining: 100 seconds
System Capabilities: B,R
Enabled Capabilities: R
Management Addresses:
    IP: 10.12.25.5
Auto Negotiation - not supported
Physical media capabilities - not advertised
Media Attachment Unit type - not advertised
Vlan ID: - not advertised


Total entries displayed: 1
```

First, regarding the device capabilities, note that the LLDP command output lists two lines about the neighbor's capabilities:

**System Capabilities:** What the device can do

**Enabled Capabilities:** What the device does now with its current configuration

For instance, in Example 13-16, the neighboring R1 claims the ability to perform routing and switching (codes **R** and **B**) but also claims to currently be using only its routing capability, as noted in the "enabled capabilities" line.

Also, take a moment to look at the output for the similarities to CDP. For instance, this output lists detail for neighbor R1, which uses its local port G0/0/1, with a hostname of R1. The output also notes the IOS name and version, from which an experienced person can infer the model number, but there is no explicit mention of the model.

**NOTE** LLDP uses the same messaging concepts as CDP, encapsulating messages directly in data-link headers. Devices do not forward LLDP messages, so LLDP learns only of directly connected neighbors. LLDP sends messages to multicast MAC address 0180.C200.000E, while CDP uses 0100.0CCC.CCCC.

## Configuring and Verifying LLDP

LLDP uses a similar configuration model as CDP, but with a few key differences. First, Cisco devices default to disable LLDP. Additionally, LLDP separates the sending and receiving of LLDP messages as separate functions. For instance, LLDP support processing receives LLDP messages on an interface so that the switch or router learns about the neighboring device while not transmitting LLDP messages to the neighboring device. To support that model, the commands include options to toggle on|off the transmission of LLDP messages separately from the processing of received messages.

The three LLDP configuration commands are as follows:

Key Topic

- **[no] lldp run:** A global configuration command that sets the default mode of LLDP operation for any interface that does not have more specific LLDP subcommands (**lldp transmit**, **lldp receive**). The **lldp run** global command enables LLDP in both directions on those interfaces, while **no lldp run** disables LLDP.
- **[no] lldp transmit:** An interface subcommand that defines the operation of LLDP on the interface regardless of the global **[no] lldp run** command. The **lldp transmit** interface subcommand causes the device to transmit LLDP messages, while **no lldp transmit** causes it to not transmit LLDP messages.
- **[no] lldp receive:** An interface subcommand that defines the operation of LLDP on the interface regardless of the global **[no] lldp run** command. The **lldp receive** interface subcommand causes the device to process received LLDP messages, while **no lldp receive** causes it to not process received LLDP messages.

For example, consider a switch that has no LLDP configuration commands at all. Example 13-17 adds configuration that first enables LLDP for all interfaces (in both directions) with the **lldp run** global command. It then shows how to disable LLDP in both directions on Gi1/0/17 and how to disable LLDP in one direction on Gi1/0/18.

**Example 13-17** *Enabling LLDP on All Ports, Disabling on a Few Ports*

```
lldp run
!
interface gigabitEthernet1/0/17
 no lldp transmit
 no lldp receive
!
interface gigabitEthernet1/0/18
 no lldp receive
```

Example 13-18 adds another example that again begins with a switch with all default settings. In this case, the configuration does not enable LLDP for all interfaces with the **lldp run** command, meaning that all interfaces default to not transmit and not receive LLDP messages. The example does show how to then enable LLDP for both directions on one interface and in one direction for a second interface.

**Example 13-18** *Enabling LLDP on Limited Ports, Leaving Disabled on Most*

```
interface gigabitEthernet1/0/19
 lldp transmit
 lldp receive
!
interface gigabitEthernet1/0/20
 lldp receive
```

Finally, checking LLDP status uses the exact same commands as CDP as listed in Table 13-4, other than the fact that you use the **lldp** keyword instead of **cdp**. For instance, **show lldp interface** lists the interfaces on which LLDP is enabled. Example 13-19 shows some examples from switch SW2 based on earlier Figure 13-6 (the same figure used in the CDP examples), with LLDP enabled in both directions on all interfaces with the **cdp run** global command.

**Example 13-19** **show lldp** *Commands That Show LLDP Status*

```
SW2# show lldp
Global LLDP Information:
    Status: ACTIVE
    LLDP advertisements are sent every 30 seconds
    LLDP hold time advertised is 120 seconds
    LLDP interface reinitialisation delay is 2 seconds

SW2# show lldp interface g1/0/2

GigabitEthernet1/0/2:
    Tx: enabled
    Rx: enabled
    Tx state: IDLE
    Rx state: WAIT FOR FRAME
```

```
SW2# show lldp traffic

LLDP traffic statistics:
    Total frames out: 259
    Total entries aged: 0
    Total frames in: 257
    Total frames received in error: 0
    Total frames discarded: 0
    Total TLVs discarded: 0
    Total TLVs unrecognized: 0
```

Also, note that like CDP, LLDP uses a send timer and hold timer for the same purposes as CDP. The example shows the default settings of 30 seconds for the send timer and 120 seconds for the hold timer. You can override the defaults with the **lldp timer** *seconds* and **lldp holdtime** *seconds* global commands, respectively.

### LLDP-MED and TLVs

The LLDP 802.1AB standard defines a method to expand LLDP for new functions using a *type-length-value (TLV)* concept. The term refers to three fields in a data structure. The first defines the type of data held there, the length lists the length of the TLV, and the value lists the data. Any device processing received messages with TLVs can quickly identify one TLV after another based on the specified length.

Figure 13-7 shows a sample LLDP message. Devices encapsulate LLDP messages directly in the data-link protocol. The LLDP message consists of a series of TLVs.

Key Topic

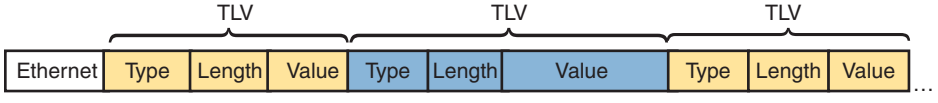



**Figure 13-7** *Type-Length-Value Concept in LLDP Messages*

Over time, new endpoint-focused LLDP TLVs emerged. The LLDP Media Endpoint Discovery (**LLDP-MED**) protocol uses a variety of endpoint-focused LLDP TLVs useful for operations between a switch and an endpoint device. (While IEEE standard 802.1AB defines LLDP, the Telecommunications Industry Association [TIA] defines LLDP-MED as an industry-standard in their TIA-1057 document.) For instance, Chapter 17, "SNMP, FTP, and TFTP," discusses Power over Ethernet (PoE), with devices using LLDP-MED to exchange data about the power needed by the endpoint device.

IP Phones connected to LAN switches can also use LLDP-MED or CDP to learn information like the voice and data VLAN IDs used by the switch. Figure 13-8 shows an example for context, with three phones, each with a connected PC. The design uses VLAN 20 as the voice VLAN and VLAN 21 as the data VLAN. (Refer to the *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, Chapter 8, Figure 8-13, and surrounding text to review voice and data VLAN configuration.)

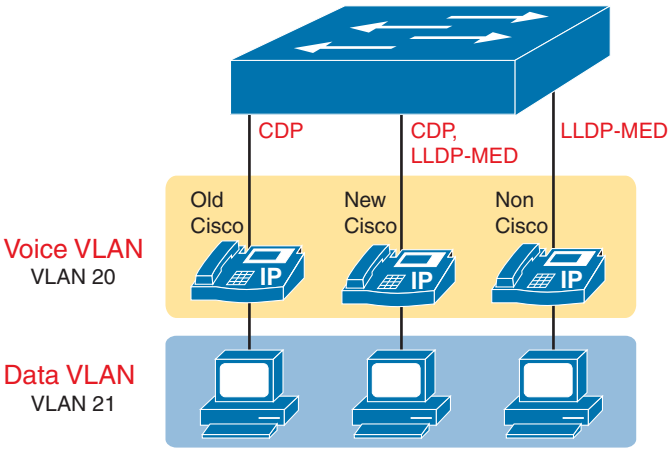



**Figure 13-8** *Sample Access Switch with IP Phones*

The phones need to learn the voice VLAN to use because the phone adds an 802.1Q trunking header to any Ethernet frames generated by the phone. You do not preconfigure the phone with the voice VLAN ID; instead, the phone learns it using LLDP-MED or CDP. Originally, Cisco IP phones supported only CDP to learn the voice VLAN ID. Cisco IP phones have supported both LLDP-MED and CDP for over a decade. Non-Cisco IP phones, which do not support Cisco-proprietary CDP, use only LLDP-MED.

On a final note about LLDP-MED, you configure LLDP-MED with the same commands as LLDP. The devices on the link dynamically discover that they need to include the TLVs defined by LLDP-MED; no additional configuration is required.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 13-6 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 13-6** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, app |
| Do labs | | Blog |
| Review command references | | Book |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 13-7** Key Topics for Chapter 13

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 13-1 | Logging to console and terminal | 271 |
| Figure 13-2 | Logging to syslog and buffer | 271 |
| Figure 13-3 | Log message levels | 273 |
| Table 13-2 | Logging configuration commands | 273 |
| List | The **ntp master** and **ntp server** commands | 279 |
| List | Facts that CDP can learn about neighbors | 283 |
| Table 13-3 | Three CDP **show** commands that list information about neighbors | 283 |
| List | How LLDP output in the example differs from CDP | 288 |
| List | LLDP configuration commands | 290 |
| Figure 13-7 | Type-length-value concept | 292 |

## Key Terms You Should Know

CDP, LLDP, LLDP-MED, log message, Network Time Protocol (NTP), NTP client, NTP client/server mode, NTP server, NTP synchronization, syslog server

## Command References

Tables 13-8 and 13-9 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 13-8** Configuration Command Reference

| Command | Description |
|---|---|
| **[no] logging console** | Global command that enables (or disables with the **no** option) logging to the console device. |
| **[no] logging monitor** | Global command that enables (or disables with the **no** option) logging to users connected to the device with SSH or Telnet. |
| **[no] logging buffered** | Global command that enables (or disables with the **no** option) logging to an internal buffer. |
| **logging** [**host**] *ip-address* \| *hostname* | Global command that enables logging to a syslog server. |
| **logging console** *level-name* \| *level-number* | Global command that sets the log message level for console log messages. |
| **logging monitor** *level-name* \| *level-number* | Global command that sets the log message level for log messages sent to SSH and Telnet users. |

| Command | Description |
|---|---|
| **logging buffered** *level-name* \| *level-number* | Global command that sets the log message level for buffered log messages displayed later by the **show logging** command. |
| **logging trap** *level-name* \| *level-number* | Global command that sets the log message level for messages sent to syslog servers. |
| [**no**] **service sequence-numbers** | Global command to enable or disable (with the **no** option) the use of sequence numbers in log messages. |
| **clock timezone** *name* +/– *hours-offset [minutes-offset]* | Global command that names a time zone and defines the +/– offset versus UTC. |
| **clock summertime** *name* **recurring** | Global command that names a daylight savings time for a time zone and tells IOS to adjust the clock automatically. |
| **ntp server** *address* \| *hostname* | Global command that configures the device as an NTP client by referring to the address or name of an NTP server. |
| **ntp master** *stratum-level* | Global command that configures the device as an NTP server and assigns its local clock stratum level. |
| **ntp source** *name/number* | Global command that tells NTP to use the listed interface (by name/number) for the source IP address for NTP messages. |
| **interface loopback** *number* | Global command that, at first use, creates a loopback interface. At all uses, it also moves the user into interface configuration mode for that interface. |
| [**no**] **cdp run** | Global command that enables and disables (with the **no** option) CDP for the entire switch or router. |
| [**no**] **cdp enable** | Interface subcommand to enable and disable (with the **no** option) CDP for a particular interface. |
| **cdp timer** *seconds* | Global command that changes the CDP send timer (the frequency at which CDP sends messages). |
| **cdp holdtime** *seconds* | Global command that changes how long CDP waits since the last received message from a neighbor before believing the neighbor has failed, removing the neighbor's information from the CDP table. |
| [**no**] **lldp run** | Global command to enable and disable (with the **no** option) LLDP for the entire switch or router. |
| [**no**] **lldp transmit** | Interface subcommand to enable and disable (with the **no** option) the transmission of LLDP messages on the interface. |
| [**no**] **lldp receive** | Interface subcommand to enable and disable (with the **no** option) the processing of received LLDP messages on the interface. |
| **lldp timer** *seconds* | Global command that changes the LLDP send timer (the frequency at which LLDP sends messages). |
| **lldp holdtime** *seconds* | Global command that changes how long LLDP waits since the last received message from a neighbor before believing the neighbor has failed, removing the neighbor's information from the LLDP table. |

**Table 13-9** Chapter 13 EXEC Command Reference

| Command | Description |
|---|---|
| **show logging** | Lists the current logging configuration and lists buffered log messages at the end |
| **terminal monitor**<br>**terminal no monitor** | For a user (SSH or Telnet) session, toggles on (**terminal monitor**) or off (**terminal no monitor**) the receipt of log messages, for that one session, if **logging monitor** is also configured |
| [**no**] **debug** {*various*} | EXEC command to enable or disable (with the **no** option) one of a multitude of debug options |
| **show clock** | Lists the time of day and the date per the local device |
| **show ntp associations** | Shows all NTP clients and servers with which the local device is attempting to synchronize with NTP |
| **show ntp status** | Shows current NTP client status in detail |
| **show interfaces loopback** *number* | Shows the current status of the listed loopback interface |
| **show cdp \| lldp neighbors** [*type number*] | Lists one summary line of information about each neighbor; optionally, lists neighbors off the listed interface |
| **show cdp \| lldp neighbors detail** | Lists one large set of information (approximately 15 lines) for every neighbor |
| **show cdp \| lldp entry** *name* | Displays the same information as **show cdp\|lldp neighbors detail** but only for the named neighbor |
| **show cdp \| lldp** | States whether CDP or LLDP is enabled globally and lists the default update and holdtime timers |
| **show cdp \| lldp interface** [*type number*] | States whether CDP or LDP is enabled on each interface or a single interface if the interface is listed |
| **show cdp \| lldp traffic** | Displays global statistics for the number of CDP or LDP advertisements sent and received |

*This page intentionally left blank*



CHAPTER 14

# Network Address Translation

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.7 Describe private IPv4 addressing**

**4.0 IP Services**

**4.1 Configure and verify inside source NAT using static and pools**

Most enterprises and small office/home office (SOHO) networks use Network Address Translation (NAT). NAT helped solve a big problem with IPv4: the IPv4 address space would have been completely assigned by the mid-1990s without it. If that had happened, Internet growth would have slowed significantly. NAT helped delay the exhaustion of the IPv4 public address space by several decades, giving time for the long-term solution—IPv6—to mature.

This chapter breaks the topics into two major sections. The first explains the basic concept behind NAT, how several variations of NAT work, and how the Port Address Translation (PAT) variation conserves the IPv4 address space. The final section shows how to configure NAT from the Cisco IOS Software command-line interface (CLI) and how to troubleshoot NAT.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 14-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Network Address Translation Concepts | 1–3 |
| NAT Configuration and Troubleshooting | 4–6 |

1. Which of the following are not private addresses according to RFC 1918? (Choose two answers.)
   a. 172.31.1.1
   b. 172.33.1.1
   c. 10.255.1.1

**d.** 10.1.255.1

**e.** 191.168.1.1

2. What causes the NAT router to create NAT table entries when using static NAT for inside addresses only?

   **a.** The first packet from the inside network to the outside network

   **b.** The first packet from the outside network to the inside network

   **c.** Configuration using the **ip nat inside source** command

   **d.** Configuration using the **ip nat outside source** command

3. What causes the NAT router to create NAT table entries when using dynamic NAT for inside addresses only?

   **a.** The first packet from the inside network to the outside network

   **b.** The first packet from the outside network to the inside network

   **c.** Configuration using the **ip nat inside source** command

   **d.** Configuration using the **ip nat outside source** command

4. An enterprise uses inside source NAT for packets that flow into the Internet, with a working NAT configuration that includes the command **ip nat inside source list alice pool barney**. Which statements must be true for the router to apply NAT to a packet entering an inside interface?

   **a.** ACL alice must match the packet with a permit action.

   **b.** ACL alice must match the packet with a deny action.

   **c.** Pool barney must include the packet's source IP address.

   **d.** Pool barney must include the packet's destination IP address.

5. An engineer created the incomplete configuration shown here. The configuration should achieve a typical router source NAT function at a router that connects an enterprise to the Internet and uses a single public IP address. Which answers list commands or parameters that would help complete the configuration? (Choose two answers.)

```
interface GigabitEthernet0/0/0
 description LAN interface (private)
 ip address 10.1.1.1 255.255.255.0
 ip nat inside
interface GigabitEthernet0/0/1
 description WAN interface (public)
 ip address 200.1.1.249 255.255.255.252
ip nat inside source list 1 interface GigabitEthernet0/0/1
access-list 1 permit 10.1.1.0 0.0.0.255
```

   **a.** The **ip nat outside** command

   **b.** The **ip nat pat** command

   **c.** The **overload** keyword

   **d.** The **ip nat pool** command

   **e.** The **inside** keyword

6. Examine the following **show** command output on a router configured for dynamic NAT:

```
-- Inside Source
access-list 1 pool fred refcount 2288
 pool fred: netmask 255.255.255.240
 start 200.1.1.1 end 200.1.1.7
 type generic, total addresses 7, allocated 7 (100%), misses 965
```

Users are complaining about not being able to reach the Internet. Which of the following is the most likely cause?

a. The problem is not related to NAT, based on the information in the command output.
b. The NAT pool does not have enough entries to satisfy all requests.
c. Standard ACL 1 cannot be used; an extended ACL must be used.
d. The command output does not supply enough information to identify the problem.

# Foundation Topics

## Network Address Translation Concepts

The original design for the Internet required every organization to ask for and receive one or more registered public classful IPv4 network numbers. However, the world would have run out of unique public IPv4 addresses decades ago following that plan. Network Address Translation (NAT) served as part of the short-term solution, delaying the day when the world ran out of public IPv4 addresses.

### IPv4 Address Conservation with NAT

Figure 14-1 shows some examples of using private and public addresses in enterprises today and how they reduce the use of public IPv4 addresses. Each company uses NAT on a router connected to the Internet. With the original plan, both companies would have needed a public Class B IPv4 network containing 65,536 public IPv4 addresses each. Instead, one company uses a small Classless InterDomain Routing (**CIDR**) block of 16 public addresses, while the other uses only 8. For the addresses used internally by the computers, per their settings, both companies use any **private IP networks** they choose, making the most common choice: to use Class A private network 10.0.0.0.

The design uses NAT but relies on private networks and public CIDR blocks. To review from Part IV of the Volume 1 book, RFC 1918 reserves a set of networks never to be used as registered public IP networks. Companies and households can use these networks internally, as shown in Figure 14-1. (Much of the rest of the chapter explains how and why.) Table 14-2 shows the private address space.

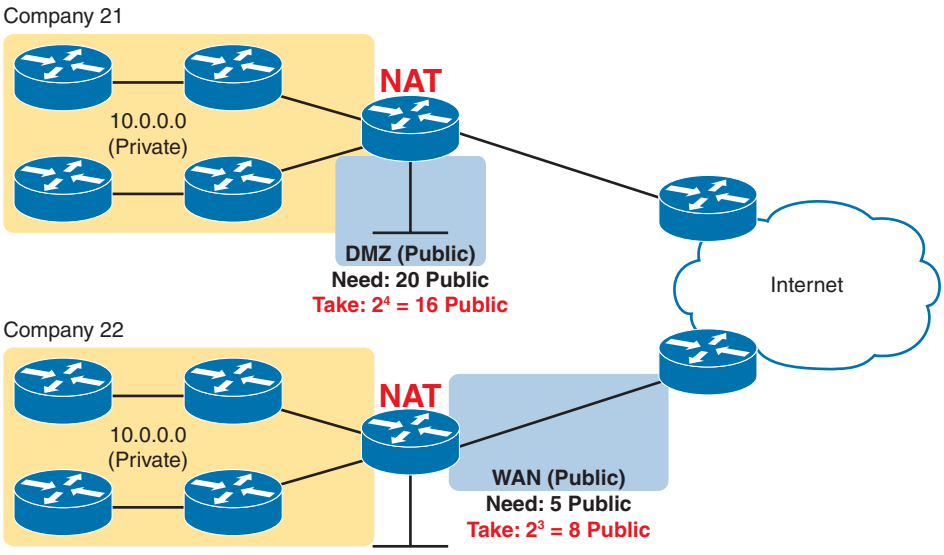



**Figure 14-1** *NAT with Small CIDR Blocks Saving IPv4 Addresses*

Key Topic

**Table 14-2** RFC 1918 Private Address Space

| Range of IP Addresses | Network(s) | Class of Networks | Number of Networks |
|---|---|---|---|
| 10.0.0.0 to 10.255.255.255 | 10.0.0.0 | A | 1 |
| 172.16.0.0 to 172.31.255.255 | 172.16.0.0 – 172.31.0.0 | B | 16 |
| 192.168.0.0 to 192.168.255.255 | 192.168.0.0 – 192.168.255.0 | C | 256 |

Any organization can use these network numbers. However, no organization can advertise these networks using a routing protocol on the Internet, and routers in the Internet will not attempt to forward packets sent to private addresses.

The short-term public address conservation plan combines NAT with CIDR blocks of public addresses, as shown in Figure 14-1. These *CIDR blocks* act like a public IP network, giving each company a consecutive set of public IPv4 addresses but allowing sizes of any power of 2. Internet routers will advertise routes for these public addresses so that the routers will forward those packets to the NAT router.

Comparing private and public addresses used with NAT, a company needs enough private IP addresses so all devices within the company have a unique address. However, companies require only a small set of public addresses when using NAT because the hosts inside the enterprise will share the public IP addresses. Table 14-3 summarizes these critical features that have helped extend the life of IPv4 by decades.

**Table 14-3** Three Important Functions That Extended the Life of IPv4

| Feature | RFC(s) | Main Benefits |
|---|---|---|
| CIDR* | 4632 | Assign more-specific public IPv4 address blocks to companies than Class A, B, and C networks.<br>Aggregate routes to public IPv4 addresses based on a worldwide address allocation plan. |
| NAT* | 3022 | Enable approximately 65,000 TCP/UDP sessions to be supported by a single public IPv4 address. |
| Private Networks | 1918 | Enable the use of NAT for enterprise Internet connections, with private addresses used inside the enterprise. |

*CIDR and NAT may be better known for their original RFCs (1518, 1519 for CIDR; 1631 for NAT).

**NOTE** This chapter shows NAT implemented on routers, but firewalls also support NAT.

## Inside Source NAT

The NAT feature of routers and firewalls includes several variations. However, the one CCNA exam topic about NAT uses the phrase **inside source NAT**, which refers to NAT as typically used at the edge of an enterprise, as shown in Figure 14-1. This next topic introduces the core concepts of inside source NAT, but be aware that all discussions in this chapter refer to only inside source NAT.

NAT, defined in RFC 3022, allows a host using a private IP address to communicate with other hosts through the Internet. NAT achieves its goal by using a valid registered public IP address to represent the private address to the rest of the Internet. The NAT function changes the private IP addresses to publicly registered IP addresses inside each IP packet, as shown in Figure 14-2.

Key Topic

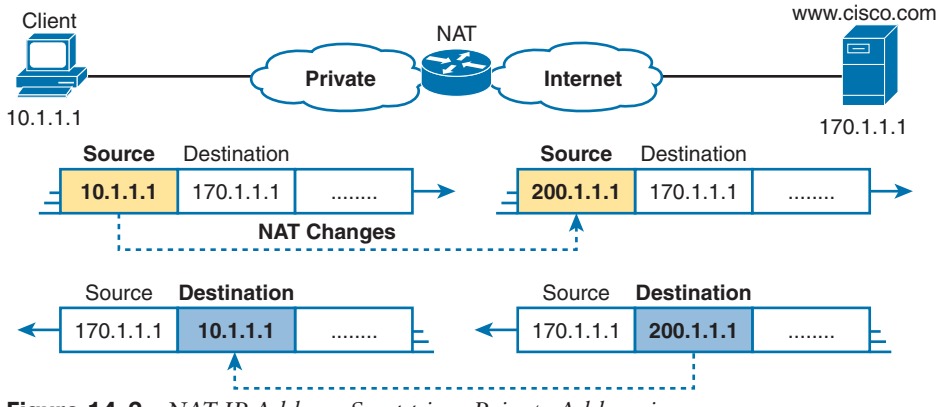



**Figure 14-2** *NAT IP Address Swapping: Private Addressing*

Answers to the "Do I Know This Already?" quiz:

**1** B, E **2** C **3** A **4** A **5** A, C **6** B

Notice that the router, performing NAT, changes the packet's source IP address when the packet leaves the private organization. The router performing NAT also changes the destination address in each packet returned to the private network. (Network 200.1.1.0 is a registered network in Figure 14-2.) The NAT feature, configured in the router labeled NAT, performs the translation.

The next few pages discuss three variants of inside source NAT: static NAT, dynamic NAT, and **Port Address Translation (PAT)**, also called **NAT overload**. All translate the addresses as shown in Figure 14-2; however, most IPv4 networks use PAT because it is the only variant that conserves public IP addresses.



## Static NAT

Static NAT works as shown in Figure 14-2 but with a static one-to-one mapping of private and public addresses. To help you understand the implications of static NAT and to explain several key terms, Figure 14-3 shows a similar example with more information.

Key Topic

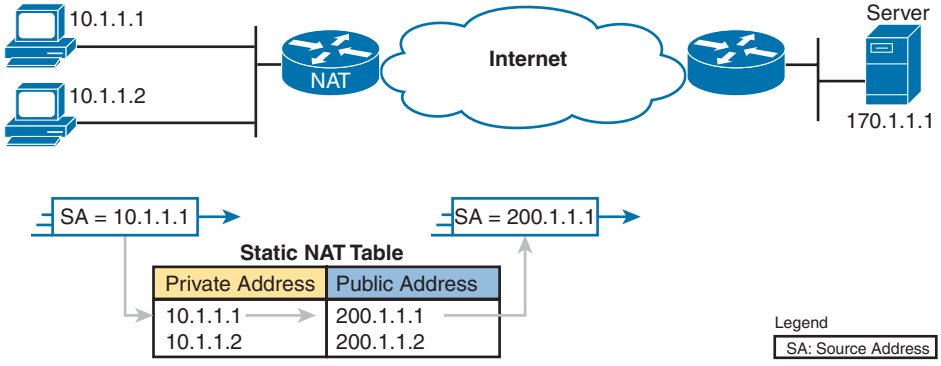



**Figure 14-3** *Static NAT Showing Inside Local and Global Addresses*

First, the concepts: The company's ISP has assigned it a registered public Class C network 200.1.1.0. The settings on all the computers inside the enterprise use addresses in private Class A network 10.0.0.0. Therefore, the NAT router must make the private IP addresses in packets appear as part of network 200.1.1.0. So, the NAT router changes the source IP addresses in the packets going from left to right in the figure.

Static NAT requires the direct configuration of the NAT table entries that tell NAT the addresses to change. In Figure 14-3, inside source NAT (as mentioned in the exam topics) relies on global command **ip nat inside source static 10.1.1.1 200.1.1.1** to add the first table entry. You could then add other static NAT table entries up to the number of available public addresses.

With a one-to-one mapping between private and public addresses, static NAT does not reduce the use of public IP addresses. In Figure 14-3, because the enterprise has a single registered Class C network, it can support at most 254 private IP addresses with NAT, with the usual two reserved numbers (the network number and network broadcast address).

## Inside Local and Inside Global Addresses

The NAT table depicted in Figure 14-3 uses the terms *private address* and *public address* for clarity. However, Cisco commands use the terms **inside local** for the private addresses and **inside global** for the public addresses.

To better understand NAT terminology, first think of any network diagram that includes NAT. Separate it into two parts, on either side of the router performing NAT. One side contains the enterprise's network (or your home network with a SOHO router). The other side contains the Internet. In Cisco NAT terminology, *inside* refers to the enterprise side, and *outside* refers to the Internet side.

Two other terms refer to the address pair in a NAT table entry. For instance, Figures 14-2 and 14-3 show a host with private address 10.1.1.1. Those figures also show NAT translating that address to public address 200.1.1.1. So NAT on Cisco routers refers to these associated addresses as follows:

Key Topic

**Inside local address:** The address which represents the inside host in packets that travel through the enterprise (local) network. You find this address in the host's settings. It is typically a private IP address when using inside source NAT. For example, address 10.1.1.1 in Figure 14-4.

**Inside global address:** The address represents the inside host in packets traveling over the global Internet. You do not find this address in a host's settings but rather in the NAT table. It is typically a public IP address, so Internet routers can forward the packet. For example, 200.1.1.1 in Figure 14-4.

Figure 14-4 repeats the same example as Figure 14-3, with some of the terminology shown.

Key Topic

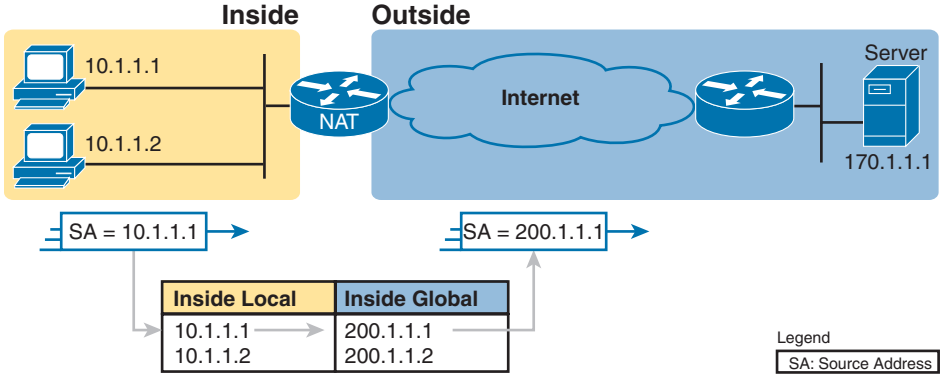



**Figure 14-4** *Static NAT Terminology*

## Dynamic NAT

Dynamic NAT is similar to static NAT but with one crucial difference. Like static NAT, the NAT router creates a one-to-one mapping between an inside local and inside global address and changes the IP addresses in packets as they exit and enter the inside network. Because of the one-to-one mappings, it also does not conserve public IPv4 addresses. But the name *dynamic NAT* gives away the big difference: it dynamically creates NAT table entries. That dynamic behavior also adds some protection against attackers in the Internet.

Dynamic NAT sets up a pool of possible inside global addresses and defines matching criteria to determine which inside local IP addresses should be translated with NAT. For example, in Figure 14-5, the NAT router has a pool of five inside global IP addresses: 200.1.1.1 through 200.1.1.5. Dynamic NAT also needs matching logic to initiate the translation of any inside local addresses that start with 10.1.1.

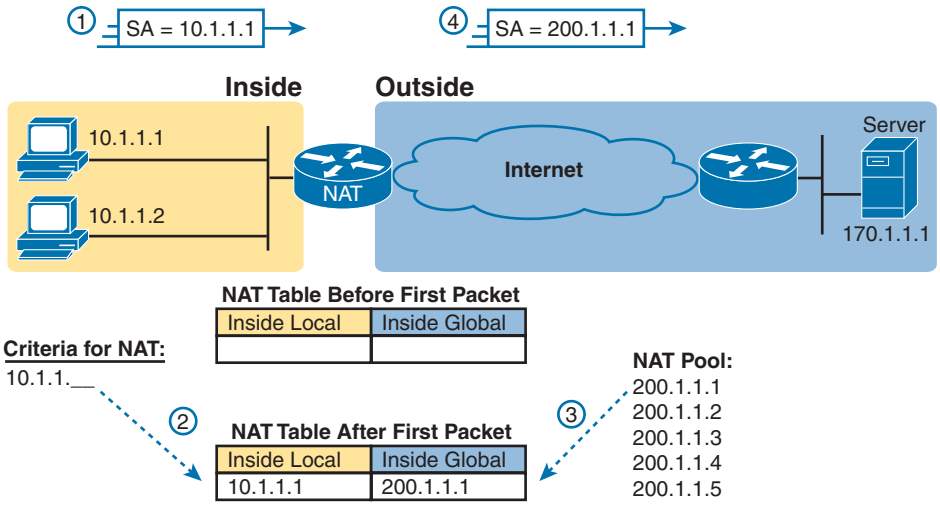



**Figure 14-5** *Dynamic NAT*

The numbers 1, 2, 3, and 4 in the figure refer to the following sequence of events:

1. Host 10.1.1.1 sends its first packet to the server at 170.1.1.1.
2. As the packet enters the NAT router, the router matches packets with source addresses (inside local addresses) that begin with 10.1.1. Matched packets trigger a new NAT table entry as needed. In this case, a packet sourced from 10.1.1.1 causes a new NAT table entry.
3. To create the new NAT table entry, the NAT router must allocate an IP address from the valid inside global address pool. It picks the first one available (200.1.1.1, in this case) and adds it to the NAT table to complete the entry.
4. The NAT router translates the source IP address and forwards the packet.

The dynamic entry stays in the table as long as continuing packets match the NAT table entry. If no packets need a NAT table entry for a configured timeout value, the router will remove the dynamic entry. You can also manually clear the dynamic entries from the table using the **clear ip nat translation** * command.

On an important security note, dynamic NAT allows new NAT table entries if initiated by inside hosts only. Dynamic NAT triggers new NAT table entries only for packets entering inside interfaces. For example, a PC user in the inside network can connect to a website in the Internet. The first packet it sends to the web server in the Internet enters the NAT router's inside interface, triggering a new entry. However, a PC user in the Internet cannot connect to a web server in the inside network. The first packet it sends would enter the outside interface and not trigger a new NAT table entry. That simple difference with dynamic NAT provides a primary security feature of only allowing new connections initiated by inside users.

Note that with predefined NAT table entries, static NAT has no restrictions for triggering new NAT table entries: a static NAT table entry would allow connections initiated by Internet users. So, engineers often use static NAT when that is the goal—to allow new connections from hosts in the Internet to a server inside the company.

By design, the NAT pool typically contains fewer addresses than the total number of hosts inside the enterprise. The router allocates addresses from the pool until none remain. If a new packet arrives, triggering the need for a new NAT table entry, but the pool has no unused addresses, the router simply discards the packet. The user must try again until a NAT entry times out, at which point the NAT function works for the next host that sends a packet. Essentially, the inside global address pool needs to be as large as the maximum number of concurrent hosts that need to use the Internet simultaneously—unless you use PAT, as explained in the next section.

## Overloading NAT with Port Address Translation

Static NAT and dynamic NAT use one-to-one mappings between inside local and inside global addresses, so they do not reduce the number of public IPv4 addresses an enterprise needs. The NAT Overload feature, also called Port Address Translation (PAT), solves this problem. Overloading uses a one-to-many mapping, allowing NAT to scale to support many inside hosts with only a few public IP addresses.

PAT uses the transport layer protocols as a means to share public addresses. To see how, first consider the idea of three separate TCP connections to a web server, from three different hosts, as shown in Figure 14-6.




**Figure 14-6** *Three TCP Connections from Three PCs*

Next, compare those three TCP connections in Figure 14-6 to three similar ones, now with all three TCP connections from one client, as shown in Figure 14-7. The server does realize a difference because the server sees the IP address and TCP port number used by the clients in both figures. However, the server does not care whether the TCP connections come from different hosts or the same host; the server just sends and receives data over each connection.

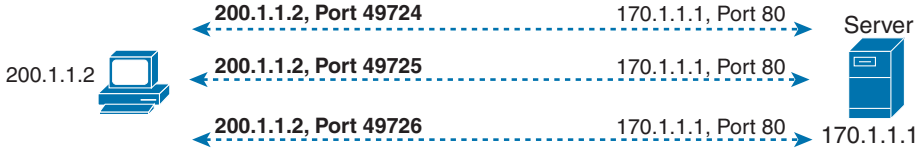
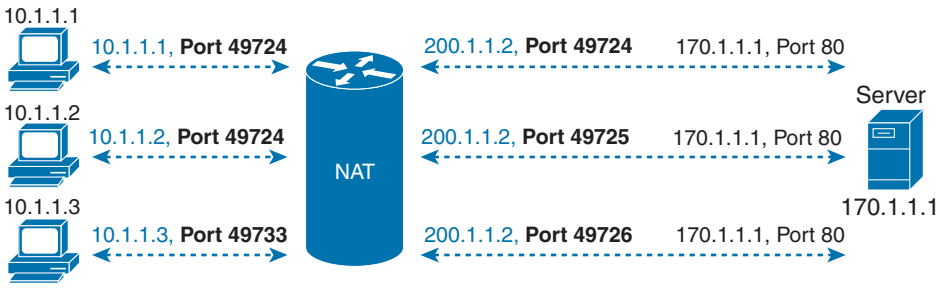



**Figure 14-7** *Three TCP Connections from One PC*

NAT takes advantage of the fact that, from a transport layer perspective, the server doesn't care whether it has one connection each to three different hosts or three connections to a single host IP address. PAT translates the IP address and the port number when necessary, making what looks like many TCP or UDP flows from different hosts (as in Figure 14-6) look like the same number of flows from one host (as in Figure 14-7). Figure 14-8 outlines the logic, with three TCP connections from three different hosts, with the NAT router making them appear as three TCP connections from a single host.




| Inside Local | Inside Global |
|---|---|
| 10.1.1.1: **49724** | 200.1.1.2: **49724** |
| 10.1.1.2: **49724** | 200.1.1.2: **49725** |
| 10.1.1.3: **49733** | 200.1.1.2: **49733** |

**Dynamic NAT Table, with Overloading**

**Figure 14-8** *NAT Overload (PAT)*

Work through the specifics of the NAT table in Figure 14-8. The inside global column lists a single inside global (public) IP address. Look at the specific port numbers on the inside local side of the table: the TCP connections from 10.1.1.1 and 10.1.1.2 both use TCP ports 49,724. On those same table entries on the inside global side, NAT had to change one of the port numbers to make each address/port combination unique. For the third TCP connection, the port number used by the host (49,733) is not used with NAT inside global address 200.1.1.1, so the NAT router did not need to change the port number.

More generally, PAT works like dynamic NAT, but it uses both address and port numbers in the translation. When PAT creates the dynamic mapping, it notes the incoming packet's source IP address and source transport port for the inside local entry in the table. For a new dynamic table entry, it selects an inside global IP address from the pool and then a port number not already in use with the inside global address.

The NAT router keeps a NAT table entry for every unique combination of inside local IP address and port associated with an inside global address and a unique port. In addition, because the port number field has 16 bits, NAT overload can use 216 port numbers with each inside global (public) address. PAT scales well without needing many registered IP addresses—in many cases, requiring only one inside global IP address.

Real networks use PAT far more than the others of the three inside source NAT types discussed here. Static NAT and Dynamic NAT have their uses, but the fact that they do not reduce public IP address consumption makes them a poor option for the primary NAT feature between an enterprise and the Internet.

## NAT Configuration and Troubleshooting

The sections that follow describe how to configure the three most common variations of NAT: static NAT, dynamic NAT, and PAT, along with the **show** and **debug** commands used to troubleshoot NAT.

## Static NAT Configuration

Static NAT configuration requires only a few configuration steps. The configuration must include each static mapping between an inside local (private) address and an inside global (public) address. Additionally, NAT must be enabled on the necessary interfaces, identifying the interface as inside or outside. The specific steps are as follows:

Config Checklist

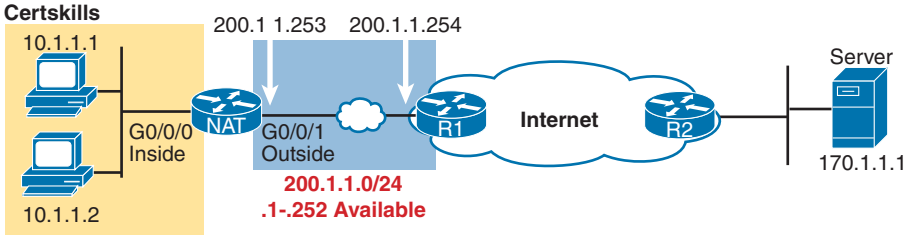
**Step 1.** Use the **ip nat inside** command in interface configuration mode to configure interfaces in the inside part of the NAT design.

**Step 2.** Use the **ip nat outside** command in interface configuration mode to configure interfaces in the outside part of the NAT design.

**Step 3.** Use the **ip address** *address mask* command in interface configuration mode to configure a subnet that includes the inside global (public) addresses intended for the design.

**Step 4.** Use the **ip nat inside source static** *inside-local inside-global* command in global configuration mode to configure the static mappings.

The upcoming examples use the familiar network design and addresses in Figure 14-9. You can see that Certskills has obtained Class C network 200.1.1.0 as a registered network number. That entire network, with mask 255.255.255.0, is configured on the Internet access link between Certskills and the Internet. The two routers on the link use two of the addresses, leaving 252 addresses for use by NAT. The NAT table shows two of those available addresses in static entries.




Static NAT Configuration

| Inside Local | Inside Global |
|---|---|
| 10.1.1.1 | 200.1.1.1 |
| 10.1.1.2 | 200.1.1.2 |

**Figure 14-9** *Sample Network for NAT Examples, with Public Class C 200.1.1.0/24*

Example 14-1 lists the NAT configuration, using 200.1.1.1 and 200.1.1.2 for the two static NAT mappings.

**Example 14-1** *Static NAT Configuration*

```
NAT# show running-config
!
! Lines omitted for brevity
!
interface GigabitEthernet0/0/0
```

```
 ip address 10.1.1.3 255.255.255.0
 ip nat inside
!
interface GigabitEthernet0/0/1
 ip address 200.1.1.253 255.255.255.0
 ip nat outside
!
ip nat inside source static 10.1.1.2 200.1.1.2
ip nat inside source static 10.1.1.1 200.1.1.1
```



To further explore the configuration, these steps analyze the configuration using the same numbered steps as the configuration checklist just before the example:

1. Interface G0/0/0, connected to the enterprise network, shows an **ip nat inside** command, enabling NAT and identifying it as an inside interface.
2. Interface G0/0/1, connected to the enterprise network, shows an **ip nat outside** command, enabling NAT and identifying it as an outside interface.
3. Interface G0/0/1 uses command **ip address 200.1.1.251 255.255.255.0**, defining connecting subnet 200.1.1.0/24. Unused addresses in this subnet will be available for static mappings.
4. The **ip nat inside source static** global configuration command creates the static mappings, referring to the addresses of the two inside hosts and two addresses in subnet 200.1.1.0/24.

Example 14-2 provides a couple of **show** commands listing essential NAT information. The **show ip nat translations** command lists the two static NAT entries created in the configuration. It shows the headings as Cisco terms inside local (with private addresses listed) and inside global (with public addresses listed). The **show ip nat statistics** command mentions the number of currently active translation table entries (with static NAT) that value matches the number of configured NAT table entries. The statistics also include the number of hits, which increments for every packet NAT must translate addresses.

**Example 14-2** *Static NAT Verification*

```
NAT# show ip nat translations
Pro Inside global      Inside local       Outside local      Outside global
--- 200.1.1.1          10.1.1.1           ---                ---
--- 200.1.1.2          10.1.1.2           ---                ---

NAT# show ip nat statistics
Total active translations: 2 (2 static, 0 dynamic; 0 extended)
Outside interfaces:
  GigabitEthernet0/0/1
Inside interfaces:
  GigabitEthernet0/0/0
Hits: 100 Misses: 0
Expired translations: 0
Dynamic mappings:
```

Before leaving static NAT, again consider the public addresses used by NAT. A typical NAT design relies on a subnet connected to the NAT router, with that connected subnet including all the inside global (public) addresses needed by NAT.

The example in this section shows a classic case of using the entire public address block, in this case, 200.1.1.0/24, on the Internet access link between the enterprise and the ISP. The routers on the link use addresses 200.1.1.253 and 200.1.1.254. The **ip address 200.1.1.253 255.255.255.0** command on router R1 in the enterprise results in a connected route for subnet 200.1.1.0/24. That subnet includes the two reserved values (200.1.1.0 and 200.1.1.255) and the two interface addresses (200.1.1.253 and 200.1.1.254). That leaves addresses 200.1.1.1–200.1.1.252 available for use by NAT on the router.

So, the configuration requires a familiar **ip address** interface subcommand, but the address planning requires some thought, with configuration to create a connected route that includes all the public addresses needed for NAT. Any interface on the NAT router will work for this purpose, with the link to the ISP being the most common. The NAT router can also use a loopback interface.

## Dynamic NAT Configuration

Dynamic NAT configuration has a few similarities with static NAT configuration. You configure the inside and outside interfaces in the same way. You also need to configure a router interface with an IP address and mask so the connected subnet contains all the inside global addresses planned for the design. Unsurprisingly, the difference is that dynamic NAT requires no preconfigured static NAT table entries.

Instead, dynamic NAT monitors packets entering inside interfaces, matching the packets' source addresses with an access control list (ACL). Incoming packets matching an ACL with a permit action trigger the NAT process. Once triggered, the NAT router will create a new NAT table entry if needed. Additionally, the dynamic NAT configuration includes some inside global (public) addresses in the form of a NAT pool. The specific steps are as follows:

Config Checklist

**Step 1.** Use the **ip nat inside** command in interface configuration mode to configure interfaces in the inside part of the NAT design (just like with static NAT).

**Step 2.** Use the **ip nat outside** command in interface configuration mode to configure interfaces in the outside part of the NAT design (just like with static NAT).

**Step 3.** Use the **ip address** *address mask* command in interface configuration mode to configure a subnet that includes the inside global (public) addresses intended for the design.

**Step 4.** Configure an ACL that matches, with a permit action, the packets entering inside interfaces for which the router should apply NAT.

**Step 5.** Use the **ip nat pool** *name first-address last-address* {**netmask** *subnet-mask* | **prefix-length** *length*} command in global configuration mode to configure the pool of public registered IP addresses.

**Step 6.** Use the **ip nat inside source list** *acl-number* **pool** *pool-name* command in global configuration mode to enable dynamic NAT. Note the command references the ACL and pool per the previous steps.

The following example shows a sample dynamic NAT configuration using the same network topology as the previous example (see Figure 14-9). The two inside local addresses—10.1.1.1 and 10.1.1.2—need translation. Unlike the previous static NAT example, however, the configuration in Example 14-3 places the public IP addresses (200.1.1.1 and 200.1.1.2) into a pool of dynamically assignable inside global addresses.

**Example 14-3** *Dynamic NAT Configuration*

```
NAT# show running-config
!
! Lines omitted for brevity
!
interface GigabitEthernet0/0/0
 ip address 10.1.1.3 255.255.255.0
 ip nat inside
!
interface GigabitEthernet0/0/1
 ip address 200.1.1.253 255.255.255.0
 ip nat outside
!
ip nat pool fred 200.1.1.1 200.1.1.2 netmask 255.255.255.252
ip nat inside source list 1 pool fred
!
access-list 1 permit 10.1.1.2
access-list 1 permit 10.1.1.1
```

Dynamic NAT configures the public (global) addresses pool with the **ip nat pool** command listing the first and last numbers in an inclusive range of inside global addresses. For example, if the pool needed 10 addresses, the command might have listed 200.1.1.1 and 200.1.1.10, which means that NAT can use 200.1.1.1 through 200.1.1.10.

Dynamic NAT also performs a verification check on the **ip nat pool** command with the required **netmask** parameter. The addresses in the pool should be in the same subnet, assuming you applied the **netmask** (or **prefix-length**) parameter as a subnet mask. If the addresses reside in different subnets per that math, IOS rejects the **ip nat pool** command. For example, as configured in Example 14-3, IOS approves the values per this logic:

- 200.1.1.1 with mask 255.255.255.252 implies subnet 200.1.1.0, broadcast address 200.1.1.3.
- 200.1.1.2 with mask 255.255.255.252 implies subnet 200.1.1.0, broadcast address 200.1.1.3.

If the example had used addresses 200.1.1.1 and 200.1.1.6, with mask 255.255.255.252, IOS would reject that command per this logic:

- 200.1.1.1 with mask 255.255.255.252 implies subnet 200.1.1.0, broadcast address 200.1.1.3.
- 200.1.1.6 with mask 255.255.255.252 implies subnet 200.1.1.4, broadcast address 200.1.1.7.

Dynamic NAT also needs one **ip nat inside source** global command. The command refers to the name of the NAT pool it wants to use for inside global addresses—in this case, fred. It also refers to an IP ACL, which defines the matching logic for inside local IP addresses. So, the logic for the **ip nat inside source list 1 pool fred** command in this example is as follows:

> Create NAT table entries that map between hosts matched by ACL 1, for packets entering any inside interface, allocating an inside global address from the pool called fred.

## Dynamic NAT Verification

Example 14-4 shows the NAT status on the NAT router before creating any dynamic NAT table entries, with Example 14-5 showing the same commands after user traffic triggered dynamic NAT table entries. In Example 14-4, the **show ip nat translations** command output shows a blank line, meaning no NAT table entries exist. The **show ip nat statistics** command shows a counter of the number of dynamic NAT table entries, showing zero active translations.

**Example 14-4** *Dynamic NAT Verifications Before Generating Traffic*

```
! The next command lists one empty line because NAT has not yet added table entries.
NAT# show ip nat translations

NAT# show ip nat statistics
Total active translations: 0 (0 static, 0 dynamic; 0 extended)
Peak translations: 8, occurred 00:02:44 ago
Outside interfaces:
  GigabitEthernet0/0/1
Inside interfaces:
  GigabitEthernet0/0/0
Hits: 0 Misses: 0
CEF Translated packets: 0, CEF Punted packets: 0
Expired translations: 0
Dynamic mappings:
-- Inside Source
[id 1] access-list 1 pool fred refcount 0
 pool fred: netmask 255.255.255.252
    start 200.1.1.1 end 200.1.1.2
    type generic, total addresses 2, allocated 0 (0%), misses 0

Total doors: 0
Appl doors: 0
Normal doors: 0
Queued Packets: 0
```

Before leaving Example 14-4, look closely at the **show ip nat statistics** command at the end of the example. It lists some fascinating troubleshooting information with two different counters labeled "Misses," as highlighted in the example. The first occurrence of this counter, around the midpoint of the command's output, counts the number of times a new packet comes along, needing a NAT entry, but no matching entry exists. At that point, dynamic NAT reacts and builds an entry.

At the end of the command output, the second Misses counter relates to the pool. This counter increments only when dynamic NAT tries to allocate a new NAT table entry, looks for an unused address from the pool, but finds no available addresses. In that case, NAT cannot add the table entry, so the packet cannot be translated—probably resulting in an end user not getting to the application.

Next, Example 14-5 updates the output of both commands from Example 14-4, this time after the user of the host at 10.1.1.1 telnets to host 170.1.1.1.

**Example 14-5** *Dynamic NAT Verifications After Generating Traffic*

```
NAT# show ip nat translations
Pro Inside global      Inside local       Outside local      Outside global
--- 200.1.1.1          10.1.1.1           ---                ---

NAT# show ip nat statistics
Total active translations: 1 (0 static, 1 dynamic; 0 extended)
Peak translations: 11, occurred 00:04:32 ago
Outside interfaces:
  GigabitEthernet0/0/1
Inside interfaces:
  GigabitEthernet0/0/0
Hits: 69 Misses: 1
Expired translations: 0
Dynamic mappings:
-- Inside Source
access-list 1 pool fred refcount 1
[eml fred: netmask 255.255.255.252
    start 200.1.1.1 end 200.1.1.2
    type generic, total addresses 2, allocated 1 (50%), misses 0
```

The example begins with host 10.1.1.1 telnetting to 170.1.1.1 (not shown), with the NAT router creating a NAT entry. The NAT table shows a single entry, mapping 10.1.1.1 to 200.1.1.1. And the first line in the output of the **show ip nat statistics** command lists a counter for 1 active translation, as shown in the NAT table at the top of the example.

Take an extra moment to consider the highlighted line, where the **show ip nat statistics** command lists 1 miss and 69 hits. The first miss counter, now at 1, means that one packet arrived that needed NAT, but there was no NAT table entry. NAT reacted and added a NAT table entry, so the hit counter of 69 means that the following 69 packets used the newly added NAT table entry.

At the bottom of the example, the second misses counter remains at 0. The counter did not increment because the NAT pool had enough inside global IP addresses to allocate the new NAT table entry. Also, note that the last line lists statistics on the number of pool members allocated (1) and the percentage of the pool currently in use (50%).

The dynamic NAT table entries time out after a period of inactivity, releasing the inside global addresses for future use. Example 14-6 shows a sequence in which two different hosts use inside global address 200.1.1.1. Host 10.1.1.1 uses inside global address 200.1.1.1 at the

beginning of the example. Then, instead of just waiting on the NAT entry to time out, the example clears the NAT table entry with the **clear ip nat translation** * command. At that point, the user at 10.1.1.2 telnets to 170.1.1.1, and the new NAT table entry appears, using the same 200.1.1.1 inside global address.

**Example 14-6** *Example of Reuse of a Dynamic Inside Global IP Address*

```
! Host 10.1.1.1 currently uses inside global 200.1.1.1
NAT# show ip nat translations
Pro Inside global      Inside local       Outside local      Outside global
--- 200.1.1.1          10.1.1.1           ---                ---
NAT# clear ip nat translation *

!
! telnet from 10.1.1.2 to 170.1.1.1 happened next; not shown
!
! Now host 10.1.1.2 uses inside global 200.1.1.1

NAT# show ip nat translations
Pro Inside global      Inside local       Outside local      Outside global
--- 200.1.1.1          10.1.1.2           ---                ---
!
! Telnet from 10.1.1.1 to 170.1.1.1 happened next; not shown
!
NAT# debug ip nat
IP NAT debugging is on

Oct 20 19:23:03.263: NAT*: s=10.1.1.1->200.1.1.2, d=170.1.1.1 [348]
Oct 20 19:23:03.267: NAT*: s=170.1.1.1, d=200.1.1.2->10.1.1.1 [348]
Oct 20 19:23:03.464: NAT*: s=10.1.1.1->200.1.1.2, d=170.1.1.1 [349]
Oct 20 19:23:03.568: NAT*: s=170.1.1.1, d=200.1.1.2->10.1.1.1 [349]
```

Finally, at the end of Example 14-6, you see that host 10.1.1.1 has telnetted to another host in the Internet, plus the **debug ip nat** command output. This **debug** command causes the router to issue a message every time a packet has its address translated for NAT. You generate the output results by entering a few lines from the Telnet connection from 10.1.1.1 to 170.1.1.1. The debug output tells you that host 10.1.1.1 uses inside global address 200.1.1.2 for this new connection.

## NAT Overload (PAT) Configuration

The static and dynamic NAT configurations matter, but this section's NAT overload (PAT) configuration matters more. PAT allows NAT to support many inside local IP addresses with only one or a few inside global IP addresses. By translating the inside local address and port number to a single inside global address but with a unique port number, NAT can support many (over 65,000) concurrent TCP and UDP flows with only one public global address.

PAT configuration includes two variations:

- One with a pool of several addresses
- One with a single inside global address from an existing interface

PAT configuration that uses a pool of addresses looks exactly like dynamic NAT, except the **ip nat inside source list** global command has an **overload** keyword added to the end. Summarizing the configuration differences between PAT, when using a pool of addresses, and dynamic NAT:

Key Topic

Use the same steps for configuring dynamic NAT, as outlined in the previous section, but include the **overload** keyword at the end of the **ip nat inside source list** global command.



Alternatively, PAT can use a single inside global address effectively. Your router at home likely does that. It leases a single public IP address from your ISP and performs NAT using it as the inside global (public) address. An enterprise router can do the same and use a public IP address on an interface, typically the interface connecting the router to the Internet. With that single address, NAT can support over 65,000 concurrent flows. If each computer averaged 10 concurrent flows, one public IP address would support around 6500 inside hosts.

The following checklist details the configuration when using an interface IP address as the sole inside global IP address:

Config Checklist

**Step 1.** As with dynamic and static NAT, configure the **ip nat inside** interface subcommand to identify inside interfaces.

**Step 2.** As with dynamic and static NAT, configure the **ip nat outside** interface subcommand to identify outside interfaces.

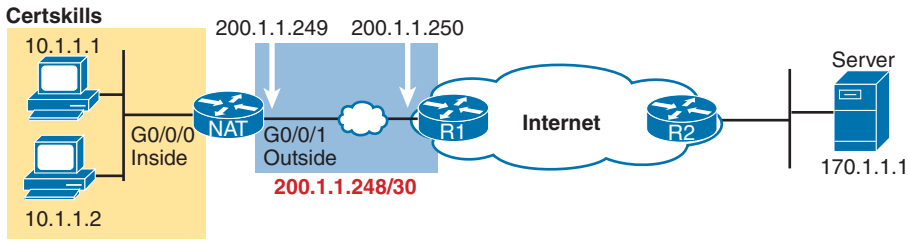
**Step 3.** As with dynamic and static NAT, use the **ip address** *address mask* command in interface configuration mode to configure a subnet that includes the inside global (public) addresses intended for the design.

**Step 4.** As with dynamic NAT, configure an ACL that matches the packets entering inside interfaces.

**Step 5.** Configure the **ip nat inside source list** *acl-number* **interface** *type/number* **overload** global configuration command, referring to the ACL from the previous step and to the interface whose connected subnet includes the NAT inside global addresses used for translations.

The vital difference between dynamic NAT versus PAT configuration comes from the use of the **overload** keyword. Step 5 notes the PAT version of the **ip nat inside source** command, which ends with **overload**. Dynamic NAT (without PAT) omits the **overload** keyword.

The following example demonstrates PAT configuration using a single interface IP address. Figure 14-10 shows the same familiar network with a few changes: CIDR block 200.1.1.248/30 instead of Class C network 200.1.1.0. The CIDR block has two usable addresses: 200.1.1.249 and 200.1.1.250. The NAT feature on the Certskills router translates all NAT addresses to its interface address: 200.1.1.249.




NAT Table (Overload)

| Inside Local | Inside Global |
|---|---|
| 10.1.1.1: 49712 | 200.1.1.249: 49712 |
| 10.1.1.2: 49713 | 200.1.1.249: 49713 |
| 10.1.2.2: 49913 | 200.1.1.249: 49913 |

**Figure 14-10** *PAT Example Using an Interface Inside Global Address*

In Example 14-7, which shows the NAT overload configuration, NAT translates using inside global address 200.1.1.249 only, so the NAT pool is not required.

**Example 14-7** *NAT Overload Configuration*

```
NAT# show running-config
!
! Lines Omitted for Brevity
!
interface GigabitEthernet0/0/0
 ip address 10.1.1.3 255.255.255.0
 ip nat inside
!
interface GigabitEthernet0/0/1
 ip address 200.1.1.249 255.255.255.252
 ip nat outside
!
ip nat inside source list 1 interface GigabitEthernet0/0/1 overload
!
access-list 1 permit 10.1.1.2
access-list 1 permit 10.1.1.1
```

The **ip nat inside source list 1 interface gigabitethernet0/0/1 overload** command has several parameters, but if you understand the dynamic NAT configuration, the new parameters shouldn't be too hard to grasp. The **list 1** parameter means the same thing as for dynamic NAT: inside local IP addresses matching ACL 1 have their addresses translated. The **interface gigabitethernet 0/0/1** parameter indicates that the only inside global IP address available is the IP address of the NAT router's interface gigabitethernet 0/0/1. Finally, the **overload** parameter enables NAT overload (PAT). Without this parameter, the router does not perform overload, just dynamic NAT.

Example 14-8 shows the NAT translation table after the two hosts created a total of three TCP connections. Take some time with the output of the **show ip nat translations** command.

It shows the three dynamic entries in the table. Now the inside global and inside local columns list both an IP address and port number (with the numbers matching Figure 14-10).

**Example 14-8** *NAT Overload Verification*

```
NAT# show ip nat translations
Pro  Inside global      Inside local       Outside local      Outside global
tcp  200.1.1.249:49712  10.1.1.1:49712     170.1.1.1:23       170.1.1.1:23
tcp  200.1.1.249:49713  10.1.1.2:49713     170.1.1.1:23       170.1.1.1:23
tcp  200.1.1.249:49913  10.1.1.2:49913     170.1.1.1:23       170.1.1.1:23

NAT# show ip nat statistics
Total active translations: 3 (0 static, 3 dynamic; 3 extended)
Peak translations: 12, occurred 00:01:11 ago
Outside interfaces:
  GigabitEthernet0/0/1
Inside interfaces:
  GigabitEthernet0/0/0
Hits: 103 Misses: 3
Expired translations: 0
Dynamic mappings:
-- Inside Source
access-list 1 interface GigabitEthernet0/0/1 refcount 3
```

Finally, in the output of the **show ip nat statistics** command, the hits and misses counters work as with dynamic NAT. The three misses count the first packet in each TCP connection, which triggered the creation of the NAT table entries. Later packets that arrive with a pre-existing NAT table entry cause an increment in the hits counter.

## NAT Troubleshooting

The majority of NAT troubleshooting issues relate to getting the configuration correct. Inside source NAT has several configuration options—static, dynamic, and PAT—with several configuration commands for each. You should work hard at building skills with the configuration to quickly recognize configuration mistakes. The following troubleshooting checklist summarizes the most common source NAT issues, most of which relate to incorrect configuration.

- **Reversed inside and outside:** Ensure that the configuration includes the **ip nat inside** and **ip nat outside** interface subcommands and that the commands are not reversed (the **ip nat inside** command on outside interfaces, and vice versa). With source NAT, only the packets entering inside interfaces trigger new translations, so designating the correct inside interfaces is particularly important.
- **Static NAT:** Check the **ip nat inside source static** command to ensure it lists the inside local address first and the inside global IP address second.
- **Dynamic NAT (ACL):** Ensure that the ACL configured to match packets sent by the inside hosts match that host's packets before any NAT translation has occurred. For example, if an inside local address of 10.1.1.1 should be translated to 200.1.1.1, ensure that the ACL matches source address 10.1.1.1, not 200.1.1.1.

- **Dynamic NAT (pool):** For dynamic NAT without PAT, ensure the pool has enough IP addresses. When not using PAT, each inside host consumes one IP address from the pool. A large or growing value in the second misses counter in the **show ip nat statistics** command output can indicate this problem. Also, compare the configured pool to the list of addresses in the NAT translation table (**show ip nat translations**). Finally, if the pool is small, the problem may be that the configuration intended to use PAT and is missing the **overload** keyword (see the next item).
- **PAT:** It is easy to forget to add the **overload** option on the end of **ip nat inside source list** command. PAT configuration with a pool is identical to a valid dynamic NAT configuration, except that PAT requires the **overload** keyword. Without it, dynamic NAT works, but often with too few inside global addresses in the pool. The NAT router will not translate or forward traffic for hosts if there is no available pool IP address for their traffic, so some hosts experience an outage.
- **ACL:** As mentioned in Chapter 8, "Applied IP ACLs," you can always add a check for ACLs that cause a problem. Perhaps NAT has been configured correctly, but an ACL exists on one of the interfaces, discarding the packets. Note that the order of operations inside the router matters in this case. For packets entering an interface, IOS processes ACLs before inside source NAT. For packets exiting an interface, IOS processes any outbound ACL after translating the addresses due to inside source NAT.
- **User traffic required:** Dynamic NAT and PAT react to user traffic. If you configure NAT in a lab, NAT does not act to create translations (**show ip nat translations**) until some user traffic enters the NAT router on an inside interface, triggering NAT to do a translation. The NAT configuration can be perfect, but if no inbound traffic occurs that matches the NAT configuration, NAT does nothing.
- **IPv4 routing:** IPv4 routing could prevent packets from arriving on either side of the NAT router. Note that the routing must work for the destination IP addresses used in the packets. For instance, routers in the Internet must have routes that match the public addresses used by NAT so that packets destined to those addresses arrive at the NAT router.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 14-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 14-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review memory tables | | Book, website |
| Review command tables | | Book |
| Do labs | | Blog |
| Watch video | | Website |

## Review All the Key Topics



Key Topic

**Table 14-5** Key Topics for Chapter 14

| Key Topic Element | Description | Page Number |
|---|---|---|
| Table 14-2 | List of private IP network numbers | 301 |
| Figure 14-2 | Main concept of NAT translating private IP addresses into publicly unique global addresses | 302 |
| Figure 14-3 | Static NAT showing inside local and global addresses | 303 |
| List | Terms: Inside local and inside global | 304 |
| Figure 14-4 | Typical NAT network diagram with key NAT terms listed | 304 |
| Figure 14-8 | Concepts behind address conservation achieved by NAT overload (PAT) | 307 |
| Paragraph | Summary of differences between dynamic NAT configuration and PAT using a pool | 315 |
| Figure 14-10 | Dynamic NAT with PAT (NAT Overload) example | 316 |

## Key Terms You Should Know

CIDR, inside global, inside local, inside source NAT, NAT overload, Port Address Translation (PAT), private IP network

## Command References

Tables 14-6 and 14-7 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.

**Table 14-6** Chapter 14 Configuration Command Reference

| Command | Description |
|---|---|
| **ip nat** {**inside** \| **outside**} | Interface subcommand to enable NAT and identify whether the interface is in the inside or outside of the network |
| **ip nat inside source** {**list** {*access-list-number* \| *access-list-name*}} {**interface** *type number* \| **pool** *pool-name*} [**overload**] | Global command that enables NAT globally, referencing the ACL that defines which source addresses to NAT, and the interface or pool from which to find global addresses |

| Command | Description |
|---|---|
| **ip nat pool** *name start-ip end-ip* {**netmask** *netmask* \| **prefix-length** *prefix-length*} | Global command to define a pool of NAT addresses |
| **ip nat inside source** *inside-local inside-global* | Global command that lists the inside and outside address (or an outside interface whose IP address should be used) to be paired and added to the NAT translation table |

**Table 14-7** Chapter 14 EXEC Command Reference

| Command | Description |
|---|---|
| **show ip nat statistics** | Lists counters for packets and NAT table entries, as well as basic configuration information |
| **show ip nat translations** [**verbose**] | Displays the NAT table |
| **clear ip nat translation** {* \| [**inside** *global-ip local-ip*] [**outside** *local-ip global-ip*]} | Clears all or some of the dynamic entries in the NAT table, depending on which parameters are used |
| **clear ip nat translation** *protocol* **inside** *global-ip global-port local-ip local-port* [**outside** *local-ip global-ip*] | Clears some of the dynamic entries in the NAT table, depending on which parameters are used |
| **debug ip nat** | Issues a log message describing each packet whose IP address is translated with NAT |

*This page intentionally left blank*



CHAPTER 15

# Quality of Service (QoS)

**This chapter covers the following exam topics:**

**4.0 IP Services**

**4.7 Explain the forwarding per-hop behavior (PHB) for QoS such as classification, marking, queuing, congestion, policing, and shaping**

Quality of Service (QoS) refers to tools that network devices can use to manage several related characteristics of what happens to a packet while it flows through a network. Specifically, these tools manage the bandwidth made available to that type of packet, the delay the packet experiences, the jitter (variation in delay) between successive packets in the same flow, and the percentage of packet loss for packets of each class. These tools balance the tradeoffs of which types of traffic receive network resources and when, giving more preference to some traffic and less preference to others.

QoS tools define actions a device can apply to a message between the time it enters the device until it exits the device. QoS defines these actions as **per-hop behaviors (PHBs)**, which is a formal term to refer to actions other than storing and forwarding a message. These actions can delay the message, discard it, or even change header fields. The device can choose different PHBs for different kinds of messages, improving the QoS behavior for some messages, while worsening the QoS behavior for others.

This chapter works through the QoS tools listed in the single QoS exam topic: "Explain the forwarding per-hop behavior (PHB) for QoS such as classification, marking, queuing, congestion, policing, and shaping." Each topic emphasizes the problems each tool solves and how each tool manages bandwidth, delay, jitter, and loss.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 15-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Introduction to QoS | 1 |
| Classification and Marking | 2, 3 |
| Queuing | 4 |
| Shaping and Policing | 5 |
| Congestion Avoidance | 6 |

1. Which of the following attributes do QoS tools manage? (Choose three answers.)
   a. Bandwidth
   b. Delay
   c. Load
   d. MTU
   e. Loss
2. Which of the following QoS marking fields could remain with a packet while being sent through four different routers, over different LAN and WAN links? (Choose two answers.)
   a. CoS
   b. IPP
   c. DSCP
   d. MPLS EXP
3. Which of the following are available methods of classifying packets in DiffServ on Cisco routers? (Choose three answers.)
   a. Matching the IP DSCP field
   b. Matching the 802.1p CoS field
   c. Matching fields with an extended IP ACL
   d. Matching the SNMP Location variable
4. Which of the following behaviors are applied to a low latency queue in a Cisco router or switch? (Choose two answers.)
   a. Shaping
   b. Policing
   c. Priority scheduling
   d. Round-robin scheduling
5. Enterprise router R1 sends data over a WAN link to ISP router ISP1. R1 shapes outgoing traffic to 200 Mbps, while ISP1 polices incoming traffic at a 250-Mbps rate. For an extended period (long enough to exhaust any bursting abilities of the shaper and policer), R1 receives 300 Mbps of traffic that it routes out the link connected to ISP1. Which answers describe the most likely shaping and policing actions in this scenario? (Choose two answers.)
   a. The policer measures an incoming rate that exceeds the policing rate.
   b. The policer measures an incoming rate that does not exceed the policing rate.
   c. The shaper is not queuing packets to slow down the sending rate.
   d. The shaper is queuing packets to slow down the sending rate.

6. A queuing system has three queues serviced with round-robin scheduling and one low latency queue that holds all voice traffic. Round-robin queue 1 holds predominantly UDP traffic, while round-robin queues 2 and 3 hold predominantly TCP traffic. The packets in each queue have a variety of DSCP markings per the QoS design. In which queues would it make sense to use a congestion avoidance (drop management) tool? (Choose two answers.)
   a. The LLQ
   b. Queue 1
   c. Queue 2
   d. Queue 3

# Foundation Topics

## Introduction to QoS

Routers typically sit at the WAN edge, with both WAN interfaces and LAN interfaces. Those LAN interfaces typically run at much faster speeds, while the WAN interfaces run at slower speeds. While that slower WAN interface is busy sending the packets waiting in the router, hundreds or even thousands more IP packets could arrive in the LAN interfaces, all needing to be forwarded out that same WAN interface. What should the router do? Send them all, in the same order in which they arrived? Prioritize the packets, to send some earlier than others, preferring one type of traffic over another? Discard some of the packets when the number of packets waiting to exit the router gets too large?

The preceding paragraph describes some of the many classic **Quality of Service (QoS)** questions in networking. QoS refers to the tools that networking devices use to apply some different treatment to packets in the network as they pass through the device. For instance, the WAN edge router would queue packets waiting for the WAN interface to be available. The router could also use a queue scheduling algorithm to determine which packets should be sent next, using some other order than the arrival order—giving some packets better service and some worse service.

### QoS: Managing Bandwidth, Delay, Jitter, and Loss

Cisco offers a wide range of QoS tools on both routers and switches. All these tools give you the means to manage four characteristics of network traffic:

Key Topic

- Bandwidth
- Delay
- Jitter
- Loss

**Bandwidth** refers to the speed of a link, in bits per second (bps). But while we think of bandwidth as speed, it helps to also think of bandwidth as the capacity of the link, in terms of how many bits can be sent over the link per second. The networking device's QoS tools determine what packet is sent over the link next, so the networking device is in control of

which messages get access to the bandwidth next and how much of that bandwidth (capacity) each type of traffic gets over time.

For example, consider a typical WAN edge router that has hundreds of packets waiting to exit the WAN link. An engineer might configure a queuing tool to reserve 10 percent of the bandwidth for voice traffic, 50 percent for mission-critical data applications, and leave the rest of the bandwidth for all other types of traffic. The queuing tool could then use those settings to make the choice about which packets to send next.

**Delay** can be described as one-way delay or round-trip delay. *One-way delay* refers to the time between sending one packet and that same packet arriving at the destination host. *Round-trip delay* counts the one-way delay plus the time for the receiver of the first packet to send back a packet—in other words, the time it takes to send one packet between two hosts and receive one back. Many different individual actions impact delay; this chapter discusses a few of those, including queuing and shaping delay.



**Jitter** refers to the variation in one-way delay between consecutive packets sent by the same application. For example, imagine an application sends a few hundred packets to one particular host. The first packet's one-way delay is 300 milliseconds (300 ms, or .3 seconds). The next packet's one-way delay is 300 ms; so is the third's; and so on. In that case, there is no jitter. However, if instead the first packet has a one-way delay of 300 ms, the next has a one-way delay of 310 ms, and the next has 325 ms, then there is some variation in the delay, 10 ms between packets 1 and 2, and another 15 ms between packets 2 and 3. That difference is called jitter.

Finally, **loss** refers to the number of lost messages, usually as a percentage of packets sent. The comparison is simple: if the sender for some application sends 100 packets, and only 98 arrive at the destination, that particular application flow experienced 2 percent loss. Loss can be caused by many factors, but often, people think of loss as something caused by faulty cabling or poor WAN services. That is one cause. However, more loss happens because of the normal operation of the networking devices, in which the devices' queues get too full, so the device has nowhere to put new packets, and it discards the packet. Several QoS tools manage queuing systems to help control and avoid loss.

## Types of Traffic

With QoS, a network engineer sets about to prefer one type of traffic over another in regard to bandwidth, delay, jitter, and loss. Sometimes, that choice relates to the specific business. For example, if all the mission-critical applications sit on servers in three known subnets, then the QoS plan could be set up to match packets going to/from that subnet and give that traffic better treatment compared to other traffic. However, in other cases, the choice of how to apply QoS tools relates to the nature of different kinds of applications. Some applications have different QoS needs than others. This next topic compares the basic differences in QoS needs based on the type of traffic.

### Data Applications

First, consider a basic web application, with a user at a PC or tablet. The user types in a URI to request a web page. That request may require a single packet going to the web server, but it may result in hundreds or thousands of packets coming back to the web client, as shown in Figure 15-1.

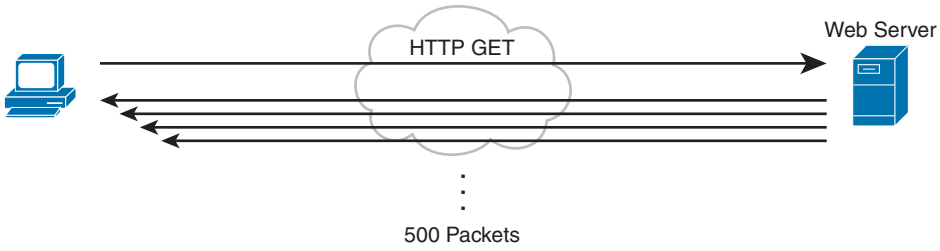



**Figure 15-1** *Interactive Data Application*

**NOTE** If you wonder how one web page might require thousands of packets, consider this math: with a 1500-byte IP maximum transmission unit (MTU), the data part of a TCP segment could be at most 1460 bytes (1500 bytes minus 20 bytes each for the IP and TCP header). In this example, 1000 such packets total to 1,460,000 bytes, or about 1.5 MB. It is easy to imagine a web page with just a few graphics that totals more than 1.5 MB in size.

So, what is the impact of bandwidth, delay, jitter, and loss on an interactive web-based application? First, the packets require a certain amount of bandwidth capacity. As for delay, each of those packets from the server to the client takes some amount of one-way delay, with some jitter as well. Of the 500 packets shown in Figure 15-1, if some are lost (transmission errors, discarded by devices, or other reasons), then the server's TCP logic will retransmit, but parts of the web page may not show up right away.

While QoS tools focus on managing bandwidth, delay, jitter, and loss, the user mainly cares about the quality of the overall experience. For instance, with a web application, how long after clicking do you see something useful in your web browser? So, as a user, you care about the *quality of experience (QoE)*, which is a term referring to users' perception of their use of the application on the network. QoS tools directly impact bandwidth, delay, jitter, and loss, which then should have some overall good effect to influence the users' QoE. And you can use QoS tools to create a better QoE for more important traffic; for instance, you might give certain business-critical applications better QoS treatment, which improves QoE for users of those apps.

In contrast, a noninteractive data application (historically called *batch* traffic)—for instance, data backup or file transfers—has different QoS requirements than interactive data applications. Batch applications typically send more data than interactive applications, but because no one is sitting there waiting to see something pop on the screen, the delay and jitter do not matter much. Much more important for these applications is meeting the need to complete the larger task (transferring files) within a larger time window. QoS tools can be used to provide enough bandwidth to meet the capacity needs of these applications and manage loss to reduce the number of retransmissions.

---

Answers to the "Do I Know This Already?" quiz:

**1** A, B, E **2** B, C **3** A, B, C **4** B, C **5** A, D **6** C, D

## Voice and Video Applications

Voice and video applications each have a similar breakdown of interactive and noninteractive flows. To make the main points about both voice and video, this section looks more deeply at voice traffic.

Before looking at voice, though, first think about the use of the term *flow* in networking. A flow is all the data moving from one application to another over the network, with one flow for each direction. For example, if you open a website and connect to a web server, the web page content that moves from the server to the client is one flow. Listen to some music with a music app on your phone, and that creates a flow from your app to the music app's server and a flow from the server back to your phone. From a voice perspective, a phone call between two IP phones would create a flow for each direction. For video, it could be the traffic from one video surveillance camera collected by security software.

Now on to voice, specifically voice over IP (VoIP). VoIP defines the means to take the sound made at one telephone and send it inside IP packets over an IP network, playing the sound back on the other telephone. Figure 15-2 shows the general idea. The figure includes these steps:

**Step 1.** The phone user makes a phone call and begins speaking.

**Step 2.** A chip called a *codec* processes (digitizes) the sound to create a binary code (160 bytes with the G.711 codec, for example) for a certain time period (usually 20 ms).

**Step 3.** The phone places the data into an IP packet.

**Step 4.** The phone sends the packet to the destination IP phone.

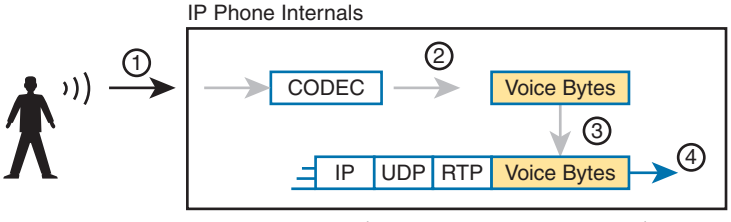



**Figure 15-2** *Creating VoIP Packets with an IP Phone and a G.711 Codec*

If you work through the math a bit, this single call, with the G.711 codec, requires about 80 Kbps of bandwidth (ignoring the data-link header and trailer overhead). Counting the headers and VoIP payload as shown in the figure, each of the IP packets has 200 bytes. Each holds 20 ms of digitized voice, so the phone sends 50 packets per second. These 50 packets at 200 bytes each equal 10,000 bytes per second, or 80,000 bits per second, which is 80 Kbps. Other voice codecs require even less bandwidth, with the commonly used G.729 taking about 24 Kbps (again ignoring data-link overhead).

At first, it might look as though VoIP calls require little in regard to QoS. For bandwidth, a single voice call or flow requires only a little bandwidth in comparison to many data applications. However, interactive voice does require a much better level of quality for delay, jitter, and loss.

For instance, think about making a phone call with high one-way delay. You finish speaking and pause for the other person to respond. And that person does not, so you speak again—and hear the other person's voice overlaid on your own. The problem: too much delay. Or consider calls for which the sound breaks up. The problem? It could have been packet loss, or it could have been jitter.

You can achieve good-quality voice traffic over an IP network, but you must implement QoS to do so. QoS tools set about to give different types of traffic the QoS behavior they need. Cisco's *Enterprise QoS Solution Reference Network Design Guide*, which itself quotes other sources in addition to relying on Cisco's long experience in implementing QoS, suggests the following guidelines for interactive voice:




- **Delay (one-way):** 150 ms or less
- **Jitter:** 30 ms or less
- **Loss:** 1% or less

In comparison, interactive voice requires more attention than interactive data applications for QoS features. Data applications generally tolerate more delay, jitter, and loss than voice (and video). A single voice call does generally take less bandwidth than a typical data application, but that bandwidth requirement is consistent. Data applications tend to be bursty, with data bursts in reaction to the user doing something with the application.

Video has a much more varied set of QoS requirements. Generally, think of video like voice, but with a much higher bandwidth requirement than voice (per flow) and similar requirements for low delay, jitter, and loss. As for bandwidth, video can use a variety of codecs that impact the amount of data sent, but many other technical features impact the amount of bandwidth required for a single video flow. (For instance, a sporting event with lots of movement on screen takes more bandwidth than a news anchor reading the news in front of a solid background with little movement.) This time quoting from *End-to-End QoS Network Design*, Second Edition (Cisco Press, 2013), some requirements for video include




- **Bandwidth:** 384 Kbps to 20+ Mbps
- **Delay (one-way):** 200–400 ms
- **Jitter:** 30–50 ms
- **Loss:** 0.1%–1%

## QoS as Mentioned in This Book

QoS tools change the QoS characteristics of certain flows in the network. The rest of the chapter focuses on the specific tools mentioned in the lone CCNA 200-301 version 1.1 exam topic about QoS, presented in the following major sections:

- "Classification and Marking" is about the marking of packets and the definition of trust boundaries.
- "Queuing" describes the scheduling of packets to give one type of packet priority over another.

- "Shaping and Policing" explains these two tools together because they are often used on opposite ends of a link.
- "Congestion Avoidance" addresses how to manage the packet loss that occurs when network devices get too busy.

### QoS on Switches and Routers

Before moving on to several sections of the chapter about specific QoS tools, let me make a point about the terms *packet* and *frame* as used in this chapter.

The QoS tools discussed in this chapter can be used on both switches and routers. There are some differences in the features and differences in implementation, due to the differences of internal architecture between routers and switches. However, to the depth discussed here, the descriptions apply equally to both LAN switches and IP routers.



This chapter uses the word *packet* in a general way, to refer to any message being processed by a networking device, just for convenience. Normally, the term *packet* refers to the IP header and encapsulated headers and data, but without the data-link header and trailer. The term *frame* refers to the data-link header/trailer with its encapsulated headers and data. For this chapter, those differences do not matter to the discussion, but at the same time, the discussion often shows a message that sometimes is literally a packet (without the data-link header/trailer) and sometimes a frame.

Throughout the chapter, the text uses *packet* for all messages, because the fact of whether or not the message happens to have a data-link header/trailer at that point is immaterial to the basic discussion of features.

Additionally, note that all the examples in the chapter refer to routers, just to be consistent.

## Classification and Marking

The first QoS tool discussed in this chapter, classification and marking, or simply marking, refers to a type of QoS tool that classifies packets based on their header contents, and then marks the message by changing some bits in specific header fields. This section looks first at the role of classification across all QoS tools, and then it examines the marking feature.

### Classification Basics

QoS tools sit in the path that packets take when being forwarded through a router or switch, much like the positioning of ACLs. Like ACLs, QoS tools are enabled on an interface. Also like ACLs, QoS tools are enabled for a direction: packets entering the interface (before the forwarding decision) or for messages exiting the interface (after the forwarding decision).

The term **classification** refers to the process of matching the fields in a message to make a choice to take some QoS action. So, again comparing QoS tools to ACLs, ACLs perform classification and filtering; that is, ACLs match (classify) packet headers. ACLs can have the purpose (action) of choosing which packets to discard. QoS tools perform classification (matching of header fields) to decide which packets to take certain QoS actions against. Those actions include the other types of QoS tools discussed in this chapter, such as queuing, shaping, policing, and so on.

For example, consider the internal processing done by a router as shown in Figure 15-3. In this case, an output queuing tool has been enabled on an interface. Routers use queuing

tools to place some packets in one output queue, other packets in another, and so on, when the outgoing interface happens to be busy. Then, when the outgoing interface becomes available to send another message, the queuing tool's scheduler algorithm can pick the next message from any one of the queues, prioritizing traffic based on the rules configured by the network engineer.

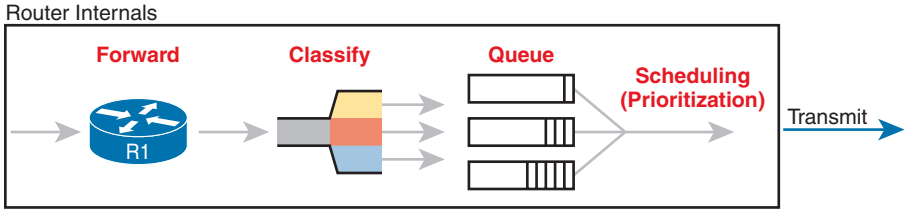



**Figure 15-3** *Big Idea: Classification for Queuing in a Router*

The figure shows the internals of a router and what happens to the packet during part of that internal processing, moving left to right inside the router, as follows:

**Step 1.** The router makes a forwarding (routing) decision.

**Step 2.** The output queuing tool uses classification logic to determine which packets go into which output queue.

**Step 3.** The router holds the packets in the output queue waiting for the outgoing interface to be available to send the next message.

**Step 4.** The queuing tool's scheduling logic chooses the next packet, effectively prioritizing one packet over another.

While the example shows a queuing tool, note that the queuing tool requires the ability to classify messages by comparing the messages to the configuration, much like ACLs.

## Matching (Classification) Basics

Now think about classification from an enterprise-wide perspective, which helps us appreciate the need for marking. Every QoS tool can examine various headers to make comparisons to classify packets. However, you might apply QoS tools on most every device in the network, sometimes at both ingress and egress on most of the interfaces. Using complex matching of many header fields in every device and on most interfaces requires lots of configuration. The work to match packets can even degrade device performance of some devices. So, while you could have every device use complex packet matching, doing so is a poor strategy.

A better strategy, one recommended both by Cisco and by RFCs, suggests doing complex matching early in the life of a packet and then marking the packet. **Marking** means that the QoS tool changes one or more header fields, setting a value in the header. Several header fields have been designed for the purpose of marking the packets for QoS processing. Then, devices that process the packet later in its life can use much simpler classification logic.

Figure 15-4 shows an example, with a PC on the left sending an IP packet to some host off the right side of the figure (not shown). Switch SW1, the first networking device to forward the packet, does some complex comparisons and marks the packet's **Differentiated**

**Services Code Point (DSCP)** field, a 6-bit field in the IP header meant for QoS marking. The next three devices that process this message—SW2, R1, and R2—then use simpler matching to classify the packet by comparing the packet's DSCP value, placing packets with one DSCP value in class 1, and packets with another DSCP value in class 2.

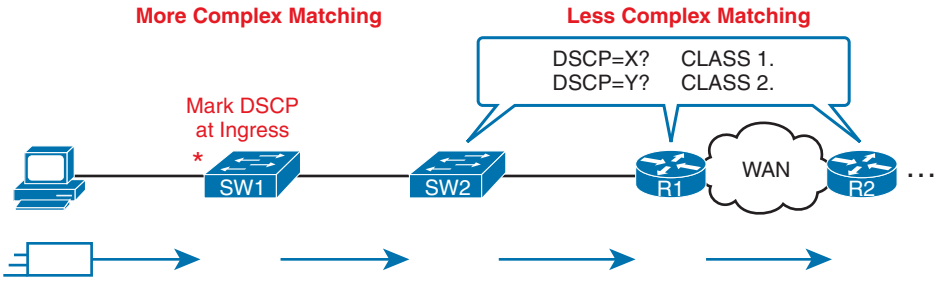



**Figure 15-4** *Systematic Classification and Marking for the Enterprise*



## Classification on Routers with ACLs and NBAR

Now that you know the basics of what classification and marking do together, this section takes the discussion a little deeper with a closer look at classification on routers, which is followed by a closer look at the marking function.

First, QoS classification sounds a lot like what ACLs do, and it should. In fact, many QoS tools support the ability to simply refer to an IP ACL, with this kind of logic:

> For any packet matched by the ACL with a permit action, consider that packet a match for QoS, so do a particular QoS action.

As a reminder, Figure 15-5 shows the IP and TCP header. All these fields are matchable for QoS classification.

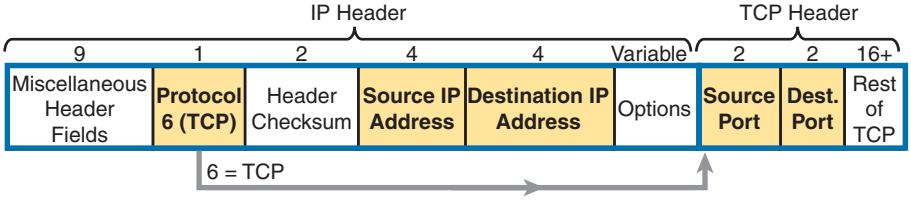



**Figure 15-5** *Classification with Five Fields Used by Extended ACLs*

Now think about the enterprise's QoS plan for a moment. That plan should list details such as which types of traffic should be classified as being in the same class for queuing purposes, for shaping, and for any other QoS tool. That plan should detail the fields in the header that can be matched. For instance, if all the IP phones sit in subnets within the range of addresses 10.3.0.0/16, then the QoS plan should state that. Then the network engineer could configure an extended ACL to match all packets to/from IP addresses inside 10.3.0.0/16 and apply appropriate QoS actions to that voice traffic.

However, not every classification can be easily made by matching with an ACL. In more challenging cases, Cisco Network Based Application Recognition (NBAR) can be used. NBAR is basically in its second major version, called NBAR2, or next-generation NBAR.

In short, NBAR2 matches packets for classification in a large variety of ways that are very useful for QoS.

NBAR2 looks far beyond what an ACL can examine in a message. Many applications cannot be identified based on well-known port alone. NBAR solves those problems.

Cisco also organizes what NBAR can match in ways that make it easy to separate the traffic into different classes. For instance, the Cisco WebEx application provides audio and video conferencing on the web. In a QoS plan, you might want to classify WebEx differently than other video traffic and classify it differently than voice calls between IP phones. That is, you might classify WebEx traffic and give it a unique DSCP marking. NBAR provides easy built-in matching ability for WebEx, plus well over 1000 different subcategories of applications.

Just to drive the point home with NBAR, Example 15-1 lists several lines of help output for one of many NBAR configuration commands. I chose a variety of items that might be more memorable. With the use of the keywords in the help lines in the correct configuration command, you could match traffic related to various Amazon public cloud offerings. NBAR refers to this idea of defining the characteristics of different applications as *application signatures*.

**Example 15-1** *Example of the Many NBAR2 Matchable Applications*

```
R1#(config)# class-map matchingexample
R1(config-cmap)# match protocol ?

! output heavily edited for length
  amazon-ec2                Secure and resizable compute capacity in the cloud.
  amazon-instant-video      VOD service by Amazon
  amazon-s3                 Amazon S3 (Simple Storage Service) is a cloud
                            computing web service.
  amazon-web-services       Amazon collection of remote computing services
! Output snipped.
```

## Marking IP DSCP and Ethernet CoS

The QoS plan for an enterprise centers on creating classes of traffic that should receive certain types of QoS treatment. That plan would note how to classify packets into each classification and the values that should be marked on the packets, basically labeling each packet with a number to associate it with that class. For example, that plan might state the following:

- Classify all voice payload traffic that is used for business purposes as IP DSCP EF and CoS 5.
- Classify all video conferencing and other interactive video for business purposes as IP DSCP AF41 and CoS 4.
- Classify all business-critical data application traffic as IP DSCP AF21 and CoS 2.

This next topic takes a closer look at the specific fields that can be marked, defining the DSCP and CoS marking fields.

### Marking the IP Header

Marking a QoS field in the IP header works well with QoS because the IP header exists for the entire trip from the source host to the destination host. When a host sends data, the host sends a data-link frame that encapsulates an IP packet. Each router that forwards the IP packet discards the old data-link header and adds a new header. Because IP routing retains the IP header, the marking in the IP header stays with the data from the first place at which the IP header is marked until it reaches the destination host.

IPv4 defines a type of service (ToS) byte in the IPv4 header, as shown in Figure 15-6. The original RFC defined a 3-bit **IP Precedence (IPP)** field for QoS marking. That field gave us eight separate values—binary 000, 001, 010, and so on, through 111—which when converted to decimal are decimals 0 through 7.

Key Topic

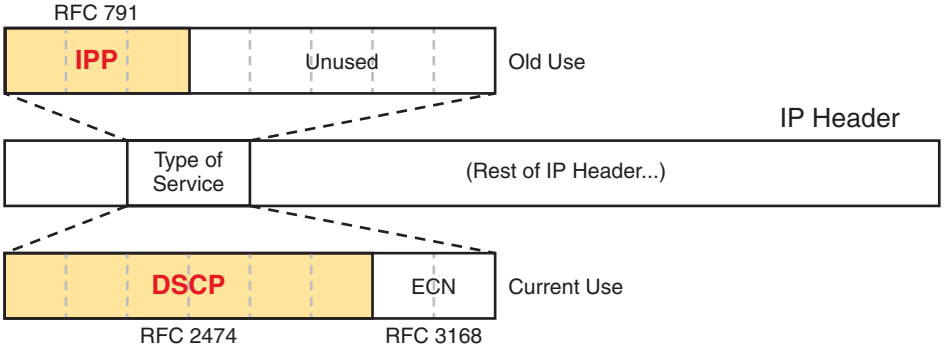



**Figure 15-6** *IP Precedence and Differentiated Services Code Point Fields*

**NOTE** Those last 5 bits of the ToS byte per RFC 791 were mostly defined for some purpose but were not used in practice to any significant extent.

While a great idea, IPP gave us only eight different values to mark, so later RFCs redefined the ToS byte with the DSCP field. DSCP increased the number of marking bits to 6 bits, allowing for 64 unique values that can be marked. The **Differentiated Services (DiffServ)** RFCs, which became RFCs back in the late 1990s, have become accepted as the most common method to use when doing QoS, and using the DSCP field for marking has become quite common.

IPv6 has a similar field to mark as well. The 6-bit field also goes by the name DSCP, with the byte in the IPv6 header being called the IPv6 *Traffic Class* byte. Otherwise, think of IPv4 and IPv6 being equivalent in terms of marking.

IPP and DSCP fields can be referenced by their decimal values as well as some convenient text names. The later section titled "DiffServ Suggested Marking Values" details some of the names.

### Marking the Ethernet 802.1Q Header

Another useful marking field exists in the 802.1Q header, in a field originally defined by the IEEE 802.1p standard. This field sits in the third byte of the 4-byte 802.1Q header, as a 3-bit field, supplying eight possible values to mark (see Figure 15-7). It goes by two different names: **class of service (CoS)** and *priority code point (PCP)*.

Key Topic

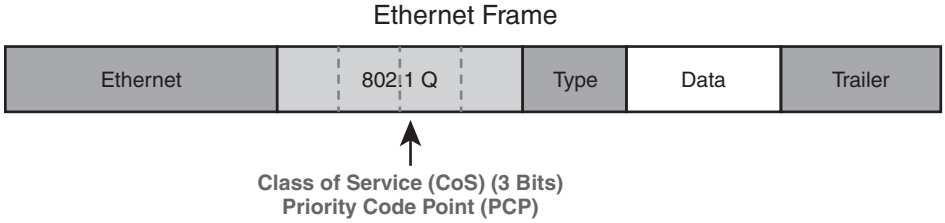



**Figure 15-7** *Class of Service Field in 802.1Q/p Header*

The figure uses two slightly different shades of gray (in print) for the Ethernet header and trailer fields versus the 802.1Q header, as a reminder: the 802.1Q header is not included in all Ethernet frames. The 802.1Q header exists only when 802.1Q trunking is used on a link. As a result, QoS tools can make use of the CoS field only for QoS features enabled on interfaces that use trunking, as shown in Figure 15-8.

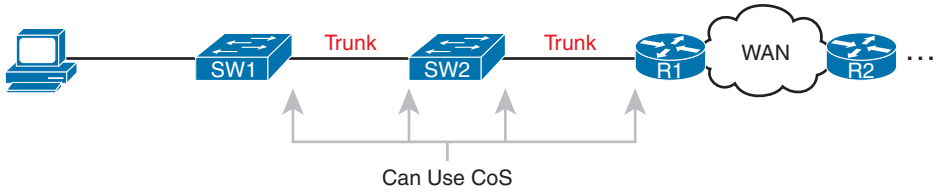



**Figure 15-8** *Useful Life of CoS Marking*

For instance, if the PC on the left were to send data to a server somewhere off the figure to the right, the DSCP field would exist for that entire trip. However, the CoS field would exist over the two trunks only and would be useful mainly on the four interfaces noted with the arrow lines.

### Other Marking Fields

Other marking fields also exist in other headers. Table 15-2 lists those fields for reference.

**Table 15-2** Marking Fields

| Field Name | Header(s) | Length (bits) | Where Used |
|---|---|---|---|
| DSCP | IPv4, IPv6 | 6 | End-to-end packet |
| IPP | IPv4, IPv6 | 3 | End-to-end packet |
| CoS | 802.1Q | 3 | Over VLAN trunk |
| TID | 802.11 | 3 | Over Wi-Fi |
| EXP | MPLS Label | 3 | Over MPLS WAN |

## Defining Trust Boundaries

The end-user device can mark the DSCP field—and even the CoS field if trunking is used on the link. Would you, as the network engineer, trust those settings and let your networking devices trust and react to those markings for their various QoS actions?

Most of us would not, because anything the end user controls might be used inappropriately at times. For instance, a PC user could know enough about DiffServ and DSCPs to know that most voice traffic is marked with a DSCP called Expedited Forwarding (EF), which has a

decimal value of 46. Voice traffic gets great QoS treatment, so PC users could mark all their traffic as DSCP 46, hoping to get great QoS treatment.

The people creating a QoS plan for an enterprise have to choose where to place the trust boundary for the network. The *trust boundary* refers to the point in the path of a packet flowing through the network at which the networking devices can trust the current QoS markings. That boundary typically sits in a device under the control of the IT staff.

For instance, a typical trust boundary could be set in the middle of the first ingress switch in the network, as shown in Figure 15-9. The markings on the message as sent by the PC cannot be trusted. However, because SW1 performed classification and marking as the packets entered the switch, the markings can be trusted at that point.

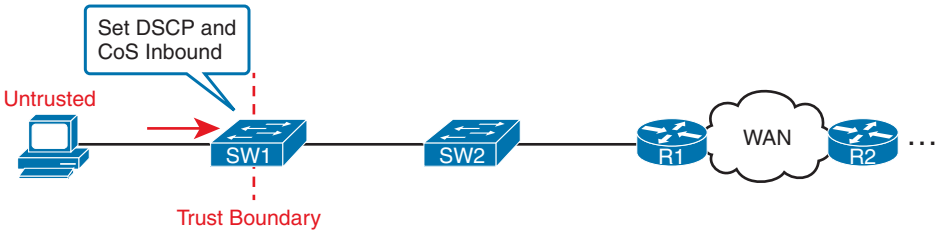



**Figure 15-9** *Trusting Devices—PC*

Interestingly, when the access layer includes an IP Phone, the phone is typically the trust boundary, instead of the access layer switch. IP Phones can set the CoS and DSCP fields of the messages created by the phone, as well as those forwarded from the PC through the phone. Figure 15-10 shows the typical trust boundary in this case, with notation of what the phone's marking logic usually is: mark all of the PC's traffic with a particular DSCP and/or CoS, and the phone's traffic with different values.

Key Topic

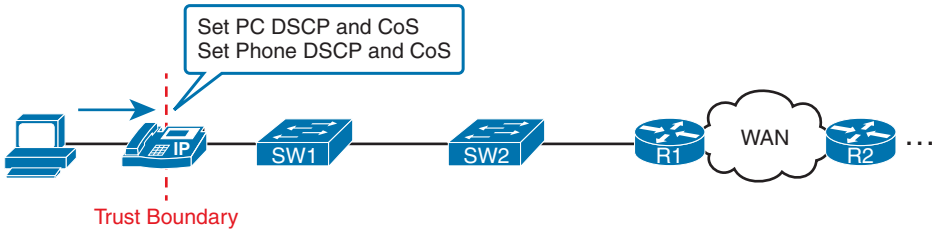



**Figure 15-10** *Trusting Devices—IP Phone*

## DiffServ Suggested Marking Values

Everything in this chapter follows the DiffServ architecture as defined originally by RFC 2475, plus many other DiffServ RFCs. In particular, DiffServ goes beyond theory in several areas, including making suggestions about the specific DSCP values to use when marking IP packets. By suggesting specific markings for specific types of traffic, DiffServ hoped to create a consistent use of DSCP values in all networks. By doing so, product vendors could provide good default settings for their QoS features, QoS could work better between an enterprise and service provider, and many other benefits could be realized.

The next two topics outline three sets of DSCP values as used in DiffServ.

### Expedited Forwarding (EF)

DiffServ defines the *Expedited Forwarding (EF)* DSCP value—a single value—as suggested for use for packets that need low latency (delay), low jitter, and low loss. The Expedited Forwarding RFC (RFC 3246) defines the specific DSCP value (decimal 46) and an equivalent text name (Expedited Forwarding). QoS configuration commands allow the use of the decimal value or text name, but one purpose of having a text acronym to use is to make the value more memorable, so many QoS configurations refer to the text names.

Most often QoS plans use EF to mark voice payload packets. With voice calls, some packets carry voice payload, and other packets carry call signaling messages. Call signaling messages set up (create) the voice call between two devices, and they do not require low delay, jitter, and loss. Voice payload packets carry the digitized voice, as shown back in Figure 15-2, and these packets do need better QoS. By default, Cisco IP Phones mark voice payload with EF, and mark voice signaling packets sent by the phone with another value called CS3.

### Assured Forwarding (AF)

The *Assured Forwarding (AF)* DiffServ RFC (2597) defines a set of 12 DSCP values meant to be used in concert with each other. First, it defines the concept of four separate queues in a queuing system. Additionally, it defines three levels of drop priority within each queue for use with congestion avoidance tools. With four queues, and three drop priority classes per queue, you need 12 different DSCP markings, one for each combination of queue and drop priority. (Queuing and congestion avoidance mechanisms are discussed later in this chapter.)

Assured Forwarding defines the specific AF DSCP text names and equivalent decimal values as listed in Figure 15-11. The text names follow a format of AFXY, with X referring to the queue (1 through 4) and Y referring to the drop priority (1 through 3).

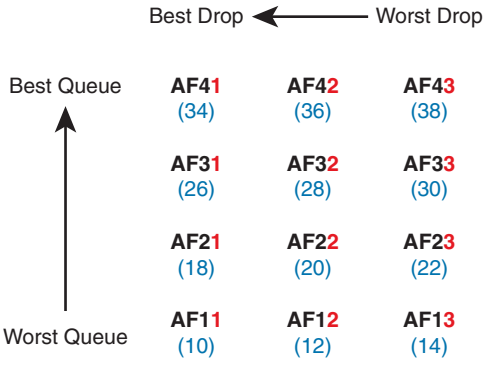


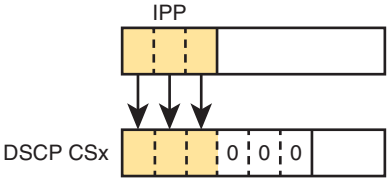

**Figure 15-11** *Differentiated Services Assured Forwarding Values and Meaning*

For example, if you marked packets with all 12 values, those with AF11, AF12, and AF13 would all go into one queue; those with AF21, AF22, and AF23 would go into another queue; and so on. Inside the queue with all the AF2Y traffic, you would treat the AF21, AF22, and AF23 each differently in regard to drop actions (congestion avoidance), with AF21 getting the preferred treatment and AF23 the worst treatment.

### Class Selector (CS)

Originally, the ToS byte was defined with a 3-bit IP Precedence (IPP) field. When DiffServ redefined the ToS byte, it made sense to create eight DSCP values for backward compatibility with IPP values. The *Class Selector (CS)* DSCP values are those settings.

Figure 15-12 shows the main idea along with the eight CS values, both in name and in decimal value. Basically, the DSCP values have the same first 3 bits as the IPP field, and with binary 0s for the last 3 bits, as shown on the left side of the figure. CSx represents the text names, where x is the matching IPP value (0 through 7).




| IPP | CS | Decimal DSCP |
|---|---|---|
| 0 | CS0 | 0 |
| 1 | CS1 | 8 |
| 2 | CS2 | 16 |
| 3 | CS3 | 24 |
| 4 | CS4 | 32 |
| 5 | CS5 | 40 |
| 6 | CS6 | 48 |
| 7 | CS7 | 56 |

**Figure 15-12** *Class Selector*



This section on classification and marking has provided a solid foundation for understanding the tools explored in the next three major sections of this chapter: queuing, shaping and policing, and congestion avoidance.

### Guidelines for DSCP Marking Values

Even with this introduction to the various DSCP marking values, you could imagine that an enterprise needs to follow a convention for how to use the markings. With so many different values, having different uses of different DSCP values by different devices in the same enterprise would make deploying QoS quite difficult at best.

Among its many efforts to standardize QoS, Cisco helped to develop RFC 4954, an RFC that defines several conventions for how to use the DSCP field. The RFC provides alternative plans with different levels of detail. Each plan defines a type of traffic and the DSCP value to use when marking data. Without getting into the depth of any one plan, the plans all specify some variation for how all devices should mark data as follows:

- DSCP EF: Voice payload
- AF4x: Interactive video (for example, videoconferencing)
- AF3x: Streaming video
- AF2x: High priority (low latency) data
- CS0: Standard data

Cisco not only worked to develop the RFC standards but also uses those standards. Cisco uses default marking conventions based on the marking data in RFC 4594, with some small exceptions. If you want to read more about these QoS marking plans, refer to a couple of sources. First, look for the Cisco QoS Design Guides at Cisco.com. Also refer to RFC 4594.

## Queuing

All networking devices use queues. Network devices receive messages, make a forwarding decision, and then send the message…but sometimes the outgoing interface is busy. So, the device keeps the outgoing message in a queue, waiting for the outgoing interface to be available—simple enough.

The term **queuing** refers to the QoS toolset for managing the queues that hold packets while they wait their turn to exit an interface (and in other cases in which a router holds packets waiting for some resource). But queuing refers to more than one idea, so you have to look inside devices to think about how they work. For instance, consider Figure 15-13, which shows the internals of a router. The router, of course, makes a forwarding decision, and it needs to be ready to queue packets for transmission once the outgoing interface is available. At the same time, the router may take a variety of other actions as well—ingress ACL, ingress NAT (on the inside interface), egress ACLs after the forwarding decision is made, and so on.

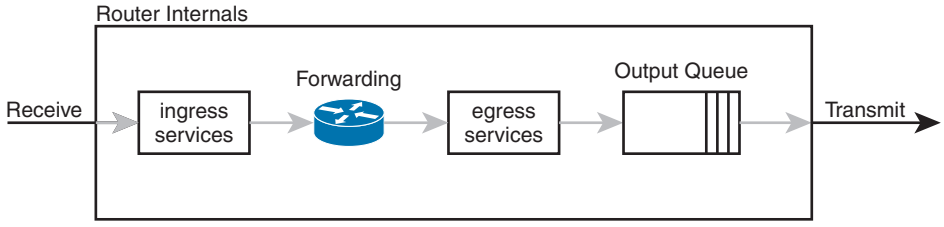



**Figure 15-13** *Output Queuing in a Router: Last Output Action Before Transmission*

The figure shows *output queuing* in which the device holds messages until the output interface is available. The queuing system may use a single output queue, with a first-in, first-out (FIFO) scheduler. (In other words, it's like ordering lunch at the sandwich shop that has a single ordering line.)

Next, think a little more deeply about the queuing system. Most networking devices can have a queuing system with multiple queues. To use multiple queues, the queuing system needs a classifier function to choose which packets are placed into which queue. (The classifier can react to previously marked values or do a more extensive match.) The queuing system needs a scheduler as well, to decide which message to take next when the interface becomes available, as shown in Figure 15-14.

Key Topic

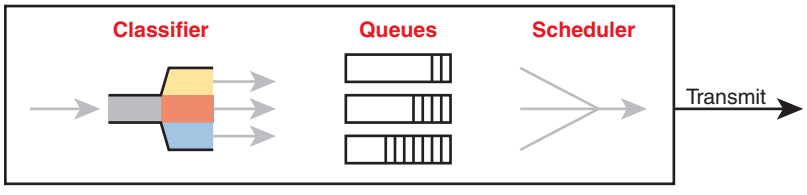



**Figure 15-14** *Queuing Components*

Of all these components of the queuing system, the scheduler can be the most interesting part because it can perform prioritization. *Prioritization* refers to the concept of giving priority to one queue over another in some way.

## Round-Robin Scheduling (Prioritization)

One scheduling algorithm used by Cisco routers and switches uses round-robin logic. In its most basic form, **round robin** cycles through the queues in order, taking turns with each queue. In each cycle, the scheduler either takes one message or takes a number of bytes from each queue by taking enough messages to total that number of bytes. Take some messages

from queue 1, move on and take some from queue 2, then take some from queue 3, and so on, starting back at queue 1 after finishing a complete pass through the queues.

Round-robin scheduling also includes the concept of *weighting* (generally called *weighted round robin*). Basically, the scheduler takes a different number of packets (or bytes) from each queue, giving more preference to one queue over another.

For example, routers use a popular tool called *Class-Based Weighted Fair Queuing (CBWFQ)* to guarantee a minimum amount of bandwidth to each class. That is, each class receives at least the amount of bandwidth configured during times of congestion, but maybe more. Internally, CBWFQ uses a weighted round-robin scheduling algorithm, while letting the network engineer define the weightings as a percentage of link bandwidth. Figure 15-15 shows an example in which the three queues in the system have been given 20, 30, and 50 percent of the bandwidth each, respectively.



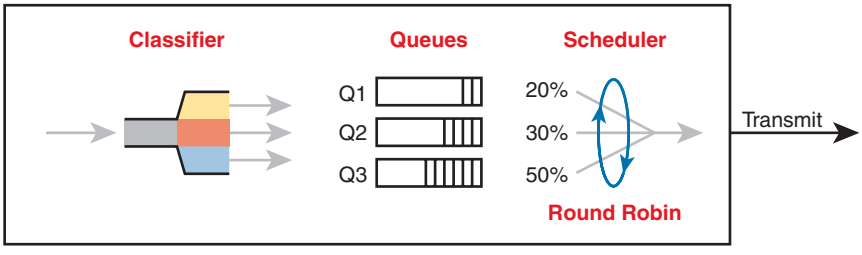



**Figure 15-15** *CBWFQ Round-Robin Scheduling*

With the queuing system shown in the figure, if the outgoing link is congested, the scheduler guarantees the percentage bandwidth shown in the figure to each queue. That is, queue 1 gets 20 percent of the link even during busy times.

## Low Latency Queuing

Earlier in the chapter, the section titled "Voice and Video Applications" discussed the reasons why voice and video, particularly interactive voice and video like phone calls and videoconferencing, need low latency (low delay), low jitter, and low loss. Unfortunately, a round-robin scheduler does not provide low enough delay, jitter, or loss. The solution: add Low Latency Queuing (LLQ) to the scheduler.

First, for a quick review, Table 15-3 lists the QoS requirements for a voice call. The numbers come from the *Enterprise QoS Solution Reference Network Design Guide*, referenced earlier in the chapter. The amount of bandwidth required per call varies based on the codec used by the call. However, the delay, jitter, and loss requirements remain the same for all voice calls. (Interactive video has similar requirements for delay, jitter, and loss.)

**Table 15-3** QoS Requirements for a VoIP Call per Cisco Voice Design Guide

| Bandwidth/call | One-way Delay (max) | Jitter (max) | Loss (max) |
|---|---|---|---|
| 30–320 Kbps | 150 ms | 30 ms | <1% |

A round-robin queuing system adds too much delay for these voice and video packets. To see why, imagine a voice packet arrives and is routed to be sent out some interface with the queuing system shown in Figure 15-16. However, that next voice packet arrives just as the

round-robin scheduler moves on to service the queue labeled "data 1." Even though the voice queue has been given 50 percent of the link bandwidth, the scheduler does not send that voice message until it sends some messages from the other three queues—adding delay and jitter.

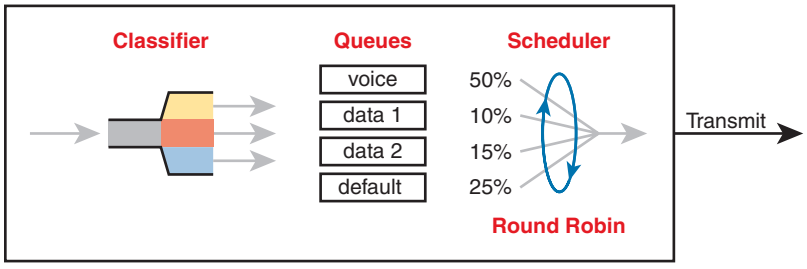



**Figure 15-16** *Round Robin Not Good for Voice Delay (Latency) and Jitter*

The solution, LLQ, tells the scheduler to treat one or more queues as special **priority queues**. The LLQ scheduler always takes the next message from one of these special priority queues. Problem solved: very little delay for packets in that queue, resulting in very little jitter as well. Plus, the queue never has time to fill up, so there are no drops due to the queue filling up. Figure 15-17 shows the addition of the LLQ logic for the voice queue.

Key Topic

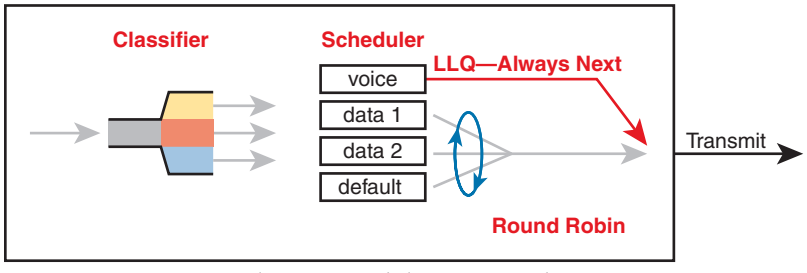



**Figure 15-17** *LLQ Always Schedules Voice Packet Next*

Using LLQ, or a priority queue, provides the needed low delay, jitter, and loss for the traffic in that queue. However, think about those other queues. Do you see the problem? What happens if the speed of the interface is X bits per second, but more than X bits per second come into the voice queue? The scheduler never services the other queues (called *queue starvation*).

As you might guess, there is a solution: limit the amount of traffic placed into the priority queue, using a feature called *policing*. The next section talks about policers in more detail, but for now, think of it as a cap on the bandwidth used by the priority queue. For instance, you could reserve 20 percent of the link's bandwidth for the voice queue and make it a priority queue. However, in this case, instead of 20 percent being the minimum bandwidth, it is the maximum for that queue. If more than 20 percent of the link's worth of bits shows up in that queue, the router will discard the excess.

Limiting the amount of bandwidth in the priority queue protects the other queues, but it causes yet another problem. Voice and video need low loss, and with LLQ, we put the voice

and video into a priority queue that will discard the excess messages beyond the bandwidth limit. The solution? Find a way to limit the amount of voice and video that the network routes out this link, so that the policer never discards any of the traffic. There are QoS tools to help you do just that, called call admission control (CAC) tools. However, CAC tools did not get a mention in the exam topics, so this chapter leaves those tools at a brief mention.

### A Prioritization Strategy for Data, Voice, and Video

This section about queuing introduces several connected ideas, so before leaving the discussion of queuing, think about this strategy for how most enterprises approach queuing in their QoS plans:

Key Topic

1. Use a round-robin queuing method like CBWFQ for data classes and for noninteractive voice and video.
2. If faced with too little bandwidth compared to the typical amount of traffic, give data classes that support business-critical applications much more guaranteed bandwidth than is given to less important data classes.
3. Use a priority queue with LLQ scheduling for interactive voice and video, to achieve low delay, jitter, and loss.
4. Put voice in a separate queue from video so that the policing function applies separately to each.
5. Define enough bandwidth for each priority queue so that the built-in policer should not discard any messages from the priority queues.
6. Use call admission control (CAC) tools to avoid adding too much voice or video to the network, which would trigger the policer function.



## Shaping and Policing

This section introduces two related QoS tools—**shaping** and **policing**. These tools have a more specialized use and are not found in as many locations in a typical enterprise. These tools are most often used at the WAN edge in an enterprise network design.

Both policing and shaping monitor the bit rate of the combined messages that flow through a device. Once enabled, the policer or shaper notes each packet that passes and measures the number of bits per second over time. Both attempt to keep the bit rate at or below the configured speed, but by using two different actions: policers discard packets, and shapers hold packets in queues to delay the packets.

Shapers and policers monitor the traffic rate (the bits per second that move through the shaper or policer) versus a configured **shaping rate** or **policing rate**, respectively. The basic question that both ask is listed below, with the actions based on the answers:

Key Topic

1. Does this next packet push the measured rate past the configured shaping rate or policing rate?
2. If no:
   a. Let the packet keep moving through the normal path and do nothing extra to the packet.

3. If yes:
   a. If shaping, delay the message by queuing it.
   b. If policing, either discard the message or mark it differently.

This section first explains policing, which discards or re-marks messages that exceed the policing rate, followed by shaping, which slows down messages that exceed the shaping rate.

## Policing

Focus on the traffic rate versus the configured policing rate for a moment, and the policing action of discarding messages. Those concepts sit at the core of what the policing function does.

Traffic arrives at networking devices at a varying rate, with valleys and spikes. That is, if you graph the bit rate of the collective bits that enter or exit any interface, the graph would look something like the one on the left side of Figure 15-18. The policer would measure that rate and make a similar measurement. Still on the left side of the figure, the horizontal dashed line represents the policing rate, which is the rate configured for the policer. So, the policer has some awareness of the measured bit rate over time, which can be compared to the configured rate.




**Figure 15-18** *Effect of a Policer and Shaper on an Offered Traffic Load*

The right side of the figure shows a graph of what happens to the traffic when a policer discards any messages that would have otherwise pushed the rate over the configured policing rate. In effect, the policer chops off the top of the graph at the policing rate.

The graph on the right also shows one example of a policer allowing a burst of traffic. Policers allow for a burst beyond the policing rate for a short time, after a period of low activity. So, that one peak that exceeds the policing rate on the graph on the right side allows for the nature of bursty data applications.

### Where to Use Policing

Now that you understand the basics of policing, take a moment to ponder. Policers monitor messages, measure a rate, and discard some messages. How does that help a network in regard to QoS? At first glance, it seems to hurt the network, discarding messages, many of which the transport or application layer will have to resend. How does that improve bandwidth, delay, jitter, or loss?

Policing makes sense only in certain cases, and as a general tool, it can be best used at the edge between two networks. For instance, consider a typical point-to-point metro Ethernet WAN connection between two enterprise routers, R1 and R2. Usually, the enterprise network engineers just view the WAN as a cloud, with Ethernet interfaces on the routers, as shown at the top of Figure 15-19.

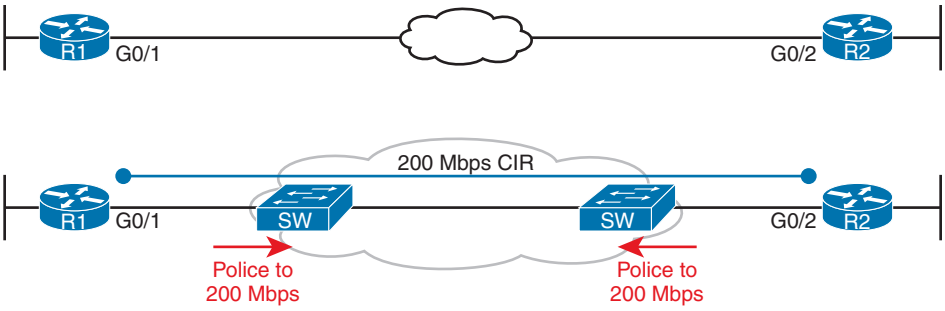



**Figure 15-19** *Ethernet WAN: Link Speed Versus CIR*



Now think about the contract for this MetroE connection, as shown at the bottom of Figure 15-19. In this case, this connection uses Gigabit Ethernet for the access links, and a 200-Mbps *committed information rate (CIR)*. That is, the SP providing the WAN service agrees to allow the enterprise to send 200 Mbps of traffic in each direction. However, remember that the enterprise routers transmit the data at the speed of the access link, or 1 Gbps in this case.

Think like the SP for a moment, and think about supporting tens of thousands of Gigabit Ethernet links into your WAN service, all with 200-Mbps CIRs. What would happen if you just let all those customers send data that, over time, averaged close to 1000 Mbps (1 Gbps)? That is, if all customers kept sending data far beyond their contracted CIR, that much traffic could cause congestion in the WAN service. Also, those customers might choose to pay for a lower CIR, knowing that the SP would send the data anyway. And customers who were well behaved and did not send more data than their CIR might suffer from the congestion just as much as the customers who send far too much data.

Figure 15-19 also notes the solution to the problem: The SP can police incoming packets, setting the policing rate to match the CIR that the customer chooses for that link. By doing so, the SP protects all customers from the negative effects of the customers who send too much traffic. Customers receive what they paid for. And the SP can provide reports of actual traffic rates, so the enterprise knows when to buy a faster CIR for each link.

Policers can discard excess traffic, but they can also re-mark packets. Think again about what an SP does with an ingress policer, as shown in Figure 15-19: they are discarding their customers' messages. So, the SP might want to make a compromise that works better for its customers, while still protecting the SP's network. The SP could mark the messages with a new marking value, with this strategy:

1. Re-mark packets that exceed the policing rate, but let them into the SP's network.
2. If other SP network devices are experiencing congestion when they process the packet, the different marking means that device can discard the packet. However…
3. …if no other SP network devices are experiencing congestion when forwarding that re-marked packet, it gets through the SP network anyway.

With this strategy, the SP can treat their customers a little better by discarding less traffic, while still protecting the SP's network during times of stress.

Summarizing the key features of policing:

Key Topic

- Policing measures the traffic rate over time for comparison to the configured policing rate.
- Policing allows for a burst of data after a period of inactivity.
- Policing is enabled on an interface, in either direction, but typically at ingress.
- Policing can discard excess messages but can also re-mark the message so that it is a candidate for more aggressive discard later in its journey.

## Shaping

You have a 1-Gbps link from a router into a SP, but a 200-Mbps CIR for traffic to another site, as seen in Figure 15-19. The SP has told you that it always discards incoming traffic that exceeds the CIR. The solution? Use a shaper to slow down the traffic—in this case to a 200-Mbps shaping rate.

That scenario—shaping before sending data to an SP that is policing—is one of the typical uses of a shaper. Shapers can be useful in other cases as well, but generally speaking, shapers make sense when a device can send at a certain speed, but there is a benefit to slowing down the rate.

The shaper slows down messages by queuing the messages. The shaper then services the shaping queues, but not based on when the physical interface is available. Instead, the shaper schedules messages from the shaping queues based on the shaping rate, as shown in Figure 15-20. Following the left-to-right flow in the figure, for a router, the packet is routed out an interface; the shaper queues packets so that the sending rate through the shaper does not exceed the shaping rate; and then output queuing works as normal, if needed.

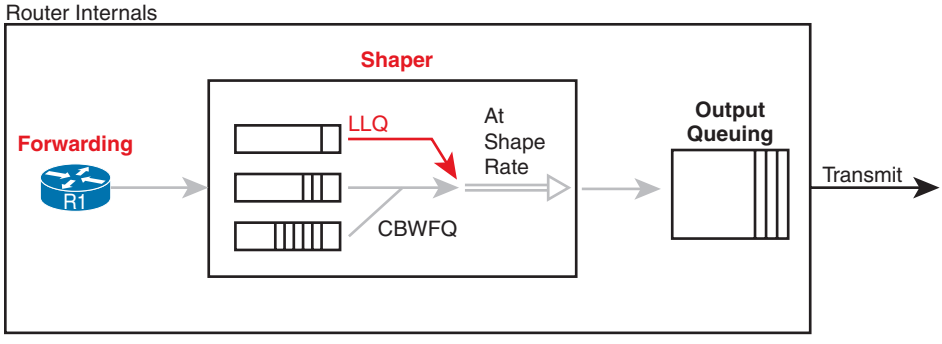



**Figure 15-20** *Shaping Queues: Scheduling with LLQ and CBWFQ*

Note that in some cases, the output queuing function has little to do. For instance, in the earlier example shown in Figure 15-19, the SP is policing incoming messages at 200 Mbps. If the router (R1, for instance) were to shape all traffic exiting toward the SP to 200 Mbps as well, with that 1-Gbps interface, the output queue would seldom if ever be congested.

Because shapers create queues where messages wait, you should apply a queuing tool to those queues. It is perfectly normal to apply the round-robin and priority queuing features of CBWFQ and LLQ, respectively, to the shaping queues, as noted in the figure.

### Setting a Good Shaping Time Interval for Voice and Video

Once again, a QoS tool has attempted to solve one QoS problem but introduces another. The unfortunate side effect of a shaper is that it slows down packets, which then creates more delay and probably more jitter. The delay occurs in part because of the message simply waiting in a queue, but partly because of the mechanisms used by a shaper. Thankfully, you can (and should) configure a shaper's setting that changes the internal operation of the shaper, which then reduces the delay and jitter caused to voice and video traffic.

A shaper's *time interval* refers to its internal logic and how a shaper averages, over time, sending at a particular rate. A shaper basically sends as fast as it can and then waits; sends and waits; sends and waits. For instance, the policing and shaping example in this section suggests shaping at 200 Mbps on a router that has a 1000-Mbps (1-Gbps) outgoing interface. In that case, the shaper would result in the interface sending data 20 percent of the time and being silent 80 percent of the time.



Figure 15-21 shows a graph of the shaping time interval concept, assuming a time interval of 1 second. To average 200 million bits per second, the shaper would allow 200 million bits to exit its shaping queues and exit the interface each second. Because the interface transmits bits at 1 Gbps, it takes just .2 seconds, or 200 ms, to send all 200 million bits. Then the shaper must wait for the rest of the time interval, another 800 ms, before beginning the next time interval.

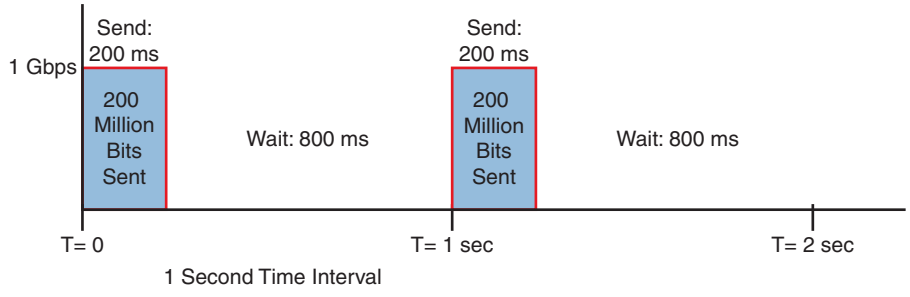



**Figure 15-21** *One Second (1000 ms) Shaping Time Interval, Shaping at 20% of Line Rate*

Now think about a voice or video packet that needs very low delay and jitter—and unfortunately, it arrives just as the shaper finishes sending data for a time interval. Even if that voice or video packet is in a priority shaping queue, the packet will wait 800 ms before the shaper schedules the next packet—far too long compared to the 150-ms one-way delay goal for voice.

The solution to this problem: configure a short time interval. For example, consider the following time intervals (abbreviated Tc), and their effects, for this same example (1-Gbps link, shaping to 200 Mbps), but with shorter and shorter time intervals:

**Tc = 1 second (1000 ms):** Send at 1 Gbps for 200 ms, rest for 800 ms

**Tc = .1 second (100 ms):** Send at 1 Gbps for 20 ms, rest for 80 ms

**Tc = .01 second (10 ms):** Send at 1 Gbps for 2 ms, rest for 8 ms

When shaping, use a short time interval. By recommendation, use a 10-ms time interval to support voice and video. With that setting, a voice or video packet should wait no more than

10 ms while waiting for the next shaping time interval, at which point the priority queue scheduling should take all the voice and video messages next.

Summarizing the key features of shapers:




- Shapers measure the traffic rate over time for comparison to the configured shaping rate.
- Shapers allow for bursting after a period of inactivity.
- Shapers are enabled on an interface for egress (outgoing packets).
- Shapers slow down packets by queuing them and over time releasing them from the queue at the shaping rate.
- Shapers use queuing tools to create and schedule the shaping queues, which is very important for the same reasons discussed for output queuing.

## Congestion Avoidance

The QoS feature called *congestion avoidance* attempts to reduce overall packet loss by preemptively discarding some packets used in TCP connections. To see how it works, you first need to look at how TCP works in regard to windowing and then look at how congestion avoidance features work.

### TCP Windowing Basics

TCP uses a flow control mechanism called *windowing*. Each TCP receiver grants a window to the sender. The window, which is a number, defines the number of bytes the sender can send over the TCP connection before receiving a TCP acknowledgment for at least some of those bytes. More exactly, the window size is the number of unacknowledged bytes that the sender can send before the sender must simply stop and wait.

The TCP window mechanism gives the receiver control of the sender's rate of sending data. Each new segment sent by the receiver back to the sender grants a new window, which can be smaller or larger than the previous window. By raising and lowering the window, the receiver can make the sender wait more or wait less.

**NOTE** Each TCP connection has two senders and two receivers; that is, each host sends and receives data. For this discussion, focus on one direction, with one host as the sender and the other as the receiver. If calling one host the "sender" and one the "receiver," note that the receiver then acknowledges data in TCP segments sent back to the sender by the receiver.

By choice, when all is well, the receiver keeps increasing the granted window, doubling it every time the receiver acknowledges data. Eventually, the window grows to the point that the sender never has to stop sending: the sender keeps receiving TCP acknowledgments before sending all the data in the previous window. Each new acknowledgment (as listed in a TCP segment and TCP header) grants a new window to the sender.

Also by choice, when a TCP receiver senses the loss of a TCP segment, the receiver shrinks the window with the next window size listed in the next TCP segment the receiver sends back to the sender. For each TCP segment lost, the window can shrink by one-half, with

multiple segment losses causing the window to shrink by half multiple times, slowing down the sender's rate significantly.

Now think about router queues for a moment. Without a congestion avoidance tool, an event called a *tail drop* causes the most drops in a network. Figure 15-22 illustrates the idea, showing the same queuing system, but in three separate conditions—little congestion, medium congestion, and much congestion. On the left, with little congestion, the output queues on an interface have not yet filled. In the middle, the queues have started to fill, with one queue being totally full. Any new packets that arrive for that queue right now will be dropped because there is no room at the tail of the queue (tail drop).

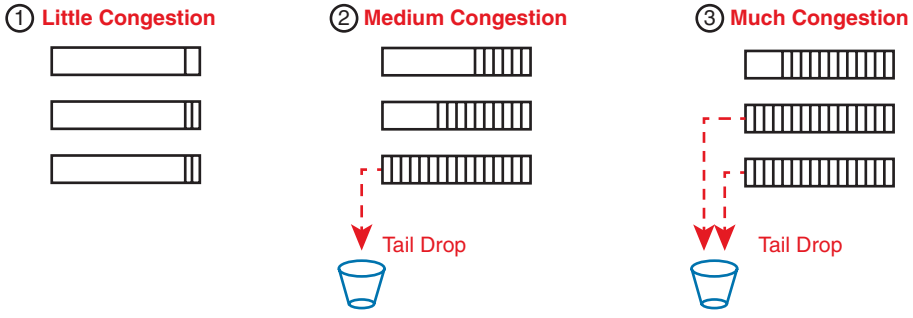



**Figure 15-22** *Stable TCP Window: Acknowledgment Received Before Window Expires*

The worse the congestion in the queues, the more likely tail drop will occur, as shown with the most congested case on the right side of the figure. The more congestion, the bigger the negative impact on traffic—both in terms of loss and in terms of increasing delay in TCP connections.

## Congestion Avoidance Tools

Congestion avoidance tools attempt to avoid the congestion, primarily through using TCP's own windowing mechanisms. These tools discard some TCP segments before the queues fill, hoping that enough TCP connections will slow down, reducing congestion, and avoiding a much worse problem: the effects of many more packets being dropped due to tail drop. The strategy is simple: discard some now in hopes that the device discards far fewer in the long term.

Congestion avoidance tools monitor the average queue depth over time, triggering more severe actions the deeper the queue, as shown in Figure 15-23. The height of the box represents the queue depth, or the number of packets in the queue. When the queue depth is low, below the minimum threshold values, the congestion avoidance tool does nothing. When the queue depth is between the minimum and maximum thresholds, the congestion avoidance tool discards a percentage of the packets—usually a small percentage, like 5, 10, or 20 percent. If the queue depth passes the maximum threshold, the tool drops all packets, in an action called a *full drop*.

Of course, like all the QoS tools mentioned in this chapter, congestion avoidance tools can classify messages to treat some packets better than others. In the same queue, packets with one marking might be dropped more aggressively, and those with better DSCP markings dropped less aggressively.

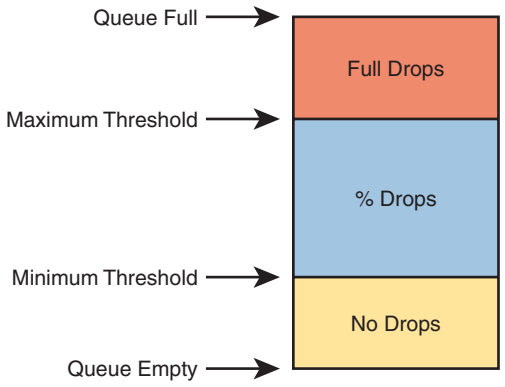



**Figure 15-23** *Mechanisms of Congestion Avoidance*

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 15-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 15-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 15-5** Key Topics for Chapter 15

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Four QoS characteristics | 324 |
| List | Voice call QoS requirements | 328 |
| List | Video QoS requirements | 328 |
| Figure 15-6 | IP Precedence and IP DSCP marking fields | 333 |
| Figure 15-7 | 802.1Q CoS marking field | 334 |
| Figure 15-10 | Trust boundary with IP Phones | 335 |
| Figure 15-14 | Queuing components | 338 |
| Figure 15-17 | LLQ scheduling logic with a priority queue | 340 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | A strategy for using queuing (congestion management) to prioritize traffic | 341 |
| List | Logic steps for shapers and policers | 341 |
| List | Key features of policers | 344 |
| List | Key features of shapers | 346 |

## Key Terms You Should Know

bandwidth, Class of Service (CoS), classification, delay, Differentiated Services (DiffServ), Differentiated Services Code Point (DSCP), IP Precedence (IPP), jitter, loss, marking, per-hop behavior (PHB), policing, policing rate, priority queue, Quality of Service (QoS), queuing, round robin, shaping, shaping rate



CHAPTER 16

# First Hop Redundancy Protocols

**This chapter covers the following exam topics:**

**3.0 IP Connectivity**

**3.5 Describe the purpose, functions, and concepts of First Hop Redundancy Protocols**

Any host's default router serves as the first router, or first hop, in the routing path from sender to receiver. However, IPv4 did not include high-availability and redundancy features related to the default router. IP hosts use a single setting with a single default router IP address. Also, IP did not define a backup or load-sharing mechanism for multiple routers connected to the same subnet.

First Hop Redundancy Protocols (FHRPs) add the function of redundancy and load sharing for the default router function in any subnet.

This chapter begins with the concepts central to all FHRPs. All FHRPs define how multiple routers work together to appear as a single default router, sharing responsibility. All the FHRPs hide their existence from the hosts, so there is no change to host routing logic. The second section examines the most popular FHRP: Hot Standby Router Protocol (HSRP). The final section compares HSRP with the other two FHRPs: Virtual Router Redundancy Protocol (VRRP) and Global Load Balancing Protocol (GLBP).

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 16-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| First Hop Redundancy Protocols | 1, 2 |
| Hot Standby Router Protocol | 3, 4 |
| VRRP and GLBP Concepts | 5, 6 |

1. R1 and R2 attach to the same Ethernet VLAN, with subnet 10.1.19.0/25, with addresses 10.1.19.1 and 10.1.19.2, respectively, configured with the **ip address** interface subcommand. Host A refers to 10.1.19.1 as its default router, and host B refers to 10.1.19.2 as its default router. The routers do not use an FHRP. Which of the following is a problem for this LAN?
   a. The design breaks IPv4 addressing rules because two routers cannot connect to the same LAN subnet.
   b. If one router fails, neither host can send packets off-subnet.
   c. If one router fails, both hosts will use the one remaining router as a default router.
   d. If one router fails, the host that uses that router as a default router cannot send packets off-subnet.
2. R1 and R2 attach to the same Ethernet VLAN, with subnet 10.1.19.0/25, with addresses 10.1.19.1 and 10.1.19.2, respectively, configured with the **ip address** interface subcommand. The routers use an FHRP. Host A and host B attach to the same LAN and have correct default router settings per the FHRP configuration. Which of the following statements is true for this LAN?
   a. The design breaks IPv4 addressing rules because two routers cannot connect to the same LAN subnet.
   b. If one router fails, neither host can send packets off-subnet.
   c. If one router fails, both hosts will use the one remaining router as a default router.
   d. If one router fails, only one of the two hosts will still be able to send packets off-subnet.
3. R1 and R2 attach to the same Ethernet VLAN, with subnet 10.1.19.0/25, with addresses 10.1.19.1 and 10.1.19.2, respectively, configured with the **ip address** interface subcommand. The routers use HSRP. The network engineer prefers to have R1 be the default router when both R1 and R2 are up. Which of the following is the likely default router setting for hosts in this subnet?
   a. 10.1.19.1
   b. 10.1.19.2
   c. Another IP address in subnet 10.1.19.0/25 other than 10.1.19.1 and 10.1.19.2
   d. A host name that the FHRP mini-DNS will initially point to 10.1.19.1
4. Routers R1, R2, and R3, with addresses 10.1.1.1, 10.1.1.2, and 10.1.1.3, respectively, are in HSRPv2 group 16, and use VIP 10.1.1.8. R2 is the current HSRP active router. Which statement is true about HSRP operation in the subnet?
   a. Traffic from hosts in the subnet balances across all routers (R1, R2, and R3).
   b. Traffic from hosts in the subnet flows into only router R2.
   c. Router R1 only replies to ARP requests for address 10.1.1.8.
   d. The HSRP group uses virtual MAC 0000.0C9F.F016.

5. Routers R1, R2, and R3, with addresses 10.1.1.1, 10.1.1.2, and 10.1.1.3, respectively, are in VRRPv3 group 32. R3 is the current VRRP active router. Which statements are true about VRRP operation in the subnet? (Choose two answers.)
   a. The current VIP may be 10.1.1.3.
   b. The current VIP must be 10.1.1.3.
   c. VRRP sends its group messages to multicast address 224.0.0.18.
   d. VRRP sends its group messages to multicast address 224.0.0.2.
6. Which answer best describes a mechanism that enables GLBP to achieve active/active load balancing, with all routers in the group forwarding packets as a default router?
   a. By configuring a VIP that matches one of the router's interface IP addresses
   b. By using a different VIP per router in the same group
   c. By using a separate GLBP group for each router
   d. By using a different virtual MAC address per router in the same group

# Foundation Topics

## First Hop Redundancy Protocols

When networks use a design that includes redundant routers, switches, LAN links, and WAN links, in some cases, other protocols are required to take advantage of that redundancy and prevent problems caused by it.

For instance, imagine a WAN with many remote branch offices. If each remote branch has two WAN links connecting it to the rest of the network, those routers can use an IP routing protocol to pick the best routes. The routing protocol learns routes over both WAN links, adding the best route into the routing table. When the better WAN link fails, the routing protocol adds the alternate route to the IP routing table, taking advantage of the redundant link.

As another example, consider a LAN with redundant links and switches. Those LANs have problems unless the switches use Spanning Tree Protocol (STP) or Rapid STP (RSTP). STP/RSTP prevents the problems created by frames that loop through those extra redundant paths in the LAN.

This section examines yet another protocol that helps when a network uses some redundancy, this time with redundant default routers. When two or more routers connect to the same LAN subnet, the hosts in that subnet could use any of the routers as their default router. However, another protocol is needed to use the redundant default routers best. The term **First Hop Redundancy Protocol (FHRP)** refers to the category of protocols that enable hosts to take advantage of redundant routers in a subnet.

This first major section of the chapter discusses the major concepts behind how different FHRPs work. This section begins by discussing a network's need for redundancy in general and the need for redundant default routers.

## The Need for Redundancy in Networks

Networks need redundant links to improve the availability of those networks. Eventually, something in a network will fail. A router power supply might fail, or a link might break, or a switch might lose power. And those WAN links, shown as simple lines in most drawings in this book, represent the most complicated physical parts of the network, with many individual components that can fail as well.

Depending on the design of the network, the failure of a single component might mean an outage that affects at least some part of the user population. Network engineers refer to any one component that, if it fails, brings down that part of the network as a *single point of failure*. For instance, in Figure 16-1, the LANs appear to have some redundancy, whereas the WAN does not. If most of the traffic flows between sites, many single points of failure exist, as shown in the figure.

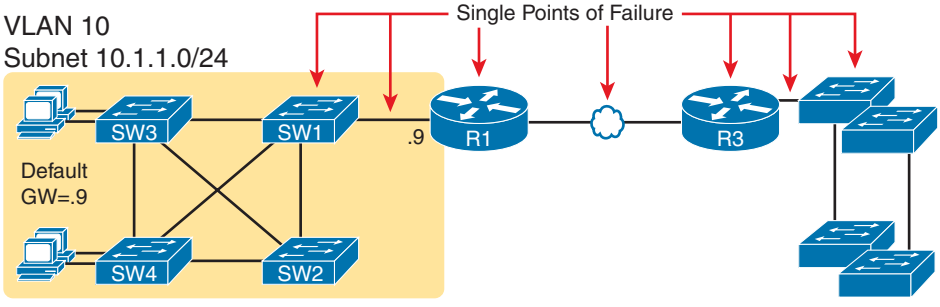



**Figure 16-1** *R1 and the One WAN Link as Single Points of Failure*

The figure notes several components as a single point of failure. If any of the network's noted parts fail, packets cannot flow from the left side of the network to the right.

To improve availability, the network engineer first looks at a design and finds the single points of failure. Then the engineer chooses where to add to the network so that one (or more) single point of failure now has redundant options, increasing availability. In particular, the engineer

- Adds redundant devices and links
- Implements any necessary functions that take advantage of the redundant device or link

For instance, of all the single points of failure in Figure 16-1, the most expensive over the long term would likely be the WAN link because of the ongoing monthly charge. However, statistically, the WAN links are the most likely component to fail. So, a good upgrade from the network in Figure 16-1 would be to add a WAN link and possibly even connect to another router on the right side of the network, as shown in Figure 16-2.

Many real enterprise networks follow designs like Figure 16-2, with one router at each remote site, two WAN links connecting back to the main site, and redundant routers at the main site (on the right side of the figure). Compared to Figure 16-1, the design in Figure 16-2

has fewer single points of failure. Of the remaining single points of failure, a risk remains, but it is a calculated risk. For many outages, a reload of the router solves the problem, and the outage is short. But the risk still exists that the switch or router hardware will fail and require time to deliver a replacement device on-site before that site can work again.

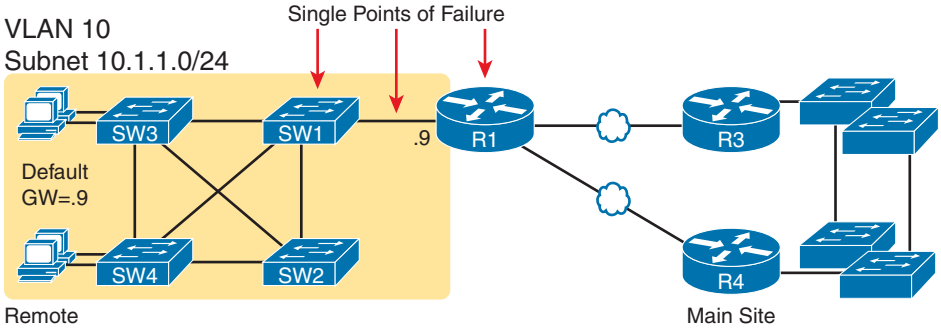



**Figure 16-2** *Higher Availability but with R1 Still as a Single Point of Failure*

For enterprises that can justify more expense, the next step in higher availability for that remote site is to protect against those catastrophic router and switch failures. In this particular design, adding one router on the left side of the network in Figure 16-2 removes all the single points of failure noted earlier. Figure 16-3 shows the design with a second router, which connects to a different LAN switch so that SW1 is no longer a single point of failure.

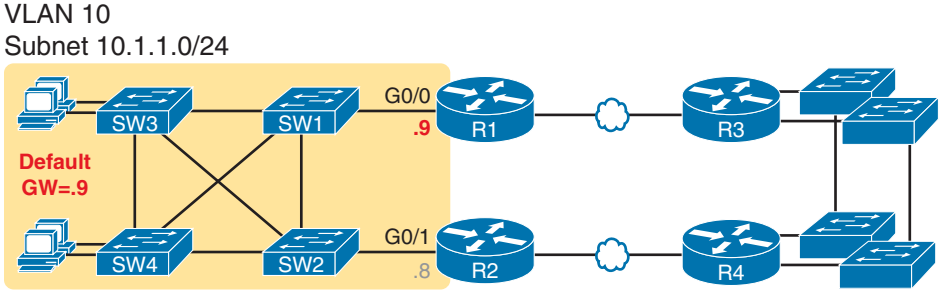



**Figure 16-3** *Removing All Single Points of Failure from the Network Design*

**NOTE** Medium to large enterprise networks work hard to balance high-availability features versus the available budget dollars. Cisco.com has many design documents that discuss tradeoffs in high-availability design. If interested in learning more, search Cisco.com for the Cisco Design Zone section of the site.

## The Need for a First Hop Redundancy Protocol

Of the designs shown so far in this chapter, only Figure 16-3's design has two routers to support the LAN on the left side of the figure, specifically the same VLAN and subnet.

---

Answers to the "Do I Know This Already?" quiz:

**1** D **2** C **3** C **4** B **5** A, C **6** D

While having the redundant routers on the same subnet helps, the network must use an FHRP when these redundant routers exist.

To see the need and benefit of using an FHRP, first think about how these redundant routers could be used as default routers by the hosts in VLAN 10/subnet 10.1.1.0/24, as shown in Figure 16-4. The host logic will remain unchanged, so each host has a single default router setting. So, some design options for default router settings include the following:

- All hosts in the subnet use R1 (10.1.1.9) as their default router, and they statically reconfigure their default router setting to R2's 10.1.1.8 if R1 fails.
- All hosts in the subnet use R2 (10.1.1.8) as their default router, and they statically reconfigure their default router setting to R1's 10.1.1.9 if R2 fails.
- Half the hosts use R1 and half use R2 as their default router, and if either router fails, half of the users statically reconfigure their default router setting.

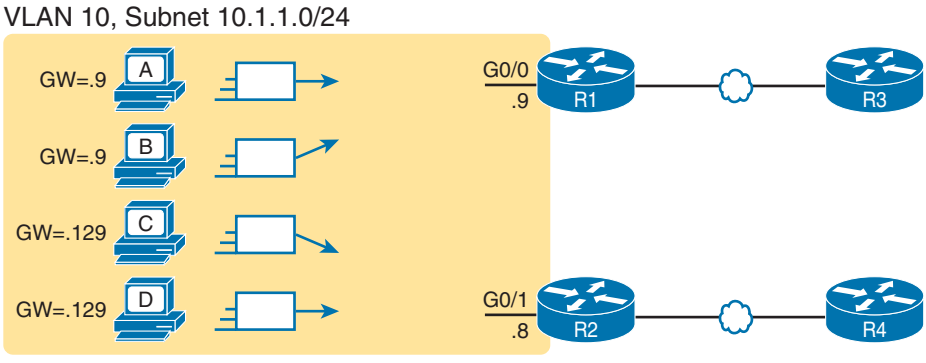



**Figure 16-4** *Balancing Traffic by Assigning Different Default Routers to Different Clients*

To ensure the concept is clear, Figure 16-4 shows this third option, with half the hosts using R1 and the other half using R2. The figure removes all the LAN switches just to unclutter the figure. Hosts A and B use R1 as their default router, and hosts C and D use R2 as their default router.

All these options have a problem: the users must act. They have to know an outage occurred. They have to know how to reconfigure their default router setting. And they have to know when to change it back to the original setting.

FHRPs use the redundant default routers without the end users being aware of any changes. The two routers appear to be a single default router. The users never have to do anything: their default router setting remains the same, and their ARP tables remain the same.

To allow the hosts to remain unchanged, the routers must do more work, as defined by one of the FHRP protocols. Generically, each FHRP makes the following happen:

Key Topic

1. All hosts act like they always have, with one default router setting that never has to change.
2. The default routers share a virtual IP address in the subnet, defined by the FHRP.
3. Hosts use the FHRP virtual IP address as their default router address.

4. The routers exchange FHRP protocol messages so that both agree as to which router does what work at any point in time.
5. When a router fails or has some other problem, the routers use the FHRP to choose which router takes over responsibilities from the failed router.

### The Three Solutions for First-Hop Redundancy

The term *First Hop Redundancy Protocol* does not name any one protocol. Instead, it names a family of protocols that fill the same role. For a given network, like the left side of Figure 16-4, the engineer would pick one of the protocols from the FHRP family.

**NOTE** *First Hop* refers to the default router being the first router, or first router hop, through which a packet must pass.

Table 16-2 lists the three FHRP protocols in chronological order as first used in the market. Cisco first introduced the proprietary **Hot Standby Router Protocol (HSRP)**, which worked well for many customers. Later, the IETF developed an RFC for a similar protocol, **Virtual Router Redundancy Protocol (VRRP)**. Finally, Cisco developed a more robust option, **Gateway Load Balancing Protocol (GLBP)**.

Key Topic

**Table 16-2** Three FHRP Options

| Acronym | Full Name | Origin | Redundancy Approach | Load Balancing Per… |
|---|---|---|---|---|
| HSRP | Hot Standby Router Protocol | Cisco | active/standby | subnet |
| VRRP | Virtual Router Redundancy Protocol | RFC 5798 | active/standby | subnet |
| GLBP | Gateway Load Balancing Protocol | Cisco | active/active | host |

The CCNA 200-301 version 1.1 blueprint requires you to know the purpose, functions, and concepts of an FHRP. To do that, the next section takes a deep look at HSRP concepts, while the final section of the chapter compares VRRP and GLBP to HSRP. (This chapter does not discuss FHRP configuration, but if you want to learn beyond the plain wording of the exam topics, note that Appendix D, "Topics from Previous Editions," contains a short section about HSRP and GLBP configuration, copied from an earlier edition of the book.)

## Hot Standby Router Protocol

HSRP operates with an active/standby model (more generally called *active/passive*). HSRP allows two (or more) routers to cooperate, all willing to act as the default router. However, at any one time, only one router actively supports the end-user traffic. The packets sent by hosts to their default router flow to that one active router. Then the other routers sit there patiently waiting to take over should the active HSRP router have a problem.

This next section of the chapter discusses how HSRP achieves its goal of providing default router redundancy. It progresses briefly through the mechanisms of virtual IP and MAC addresses, failover, load balancing, object tracking, and HSRP versions.

## HSRP Virtual IP and MAC Addresses

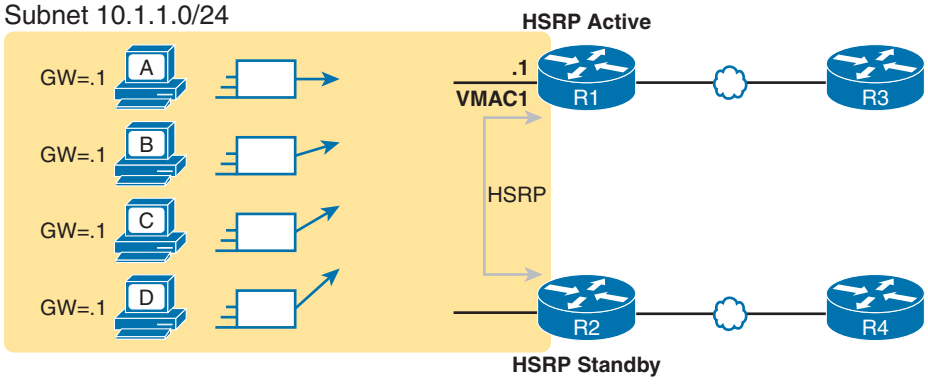
The **HSRP active** router implements a **virtual IP address (VIP)** and matching **virtual MAC address**. This virtual IP address is part of the HSRP configuration, an additional configuration item compared to the usual **ip address** interface subcommand. This virtual IP address is in the same subnet as the interface IP address, but it is a different IP address. The router then automatically creates the virtual MAC address. All the cooperating HSRP routers know these virtual addresses, but only the HSRP active router uses these addresses at any one point in time.

Using HSRP protocol messages between the routers, the routers negotiate and settle in either the HSRP active or **HSRP standby** state. The router with the highest **HSRP priority** wins and becomes active, with the other router becoming the standby router. If the priorities tie, the router with the highest IP address wins and becomes active. (Also, note that while FHRPs often have only two routers in a group, if using three or more, HSRP uses only one standby router, with the other routers in a listen state, waiting to become the new standby router one day.)

The active router implements the virtual IP and MAC addresses on its interface in addition to its configured interface IP address. Hosts refer to the virtual IP address as their default router address instead of any router's interface IP address. For instance, in Figure 16-5, R1 and R2 use HSRP. The HSRP virtual IP address is 10.1.1.1, with the virtual MAC address referenced as VMAC1 for simplicity's sake.



Key Topic




Host ARP Table

| IP | MAC |
|---|---|
| 10.1.1.1 | VMAC1 |

**Figure 16-5** *All Traffic Goes to .1 (R1, Which Is Active); R2 Is Standby*

## HSRP Failover

Under normal conditions, with all devices and interfaces working, one HSRP router is the default router, with another standing by. That might happen for months before the standby router needs to take over. However, so that the standby router knows when to act, the two routers continue to send HSRP messages to each other.

HSRP uses HSRP Hello messages to let the other HSRP routers in the same HSRP group know that the active router continues to work. HSRP defines a Hello timer, which dictates how often (in seconds) between successive Hello messages sent by the active router. HSRP also defines a Hold timer, typically more than three times the Hello timer. When the standby router fails to receive a Hello from the active router within the time defined by the hold time, the standby router believes the active router has failed, and begins taking over as the active router.

For example, Figure 16-6 shows the result when R1, the HSRP active router in Figure 16-5, loses power. R2 fails to receive additional HSRP Hellos from router R1 for hold time. At that point, R2, the new active router, starts using the virtual IP and MAC addresses.

Key Topic

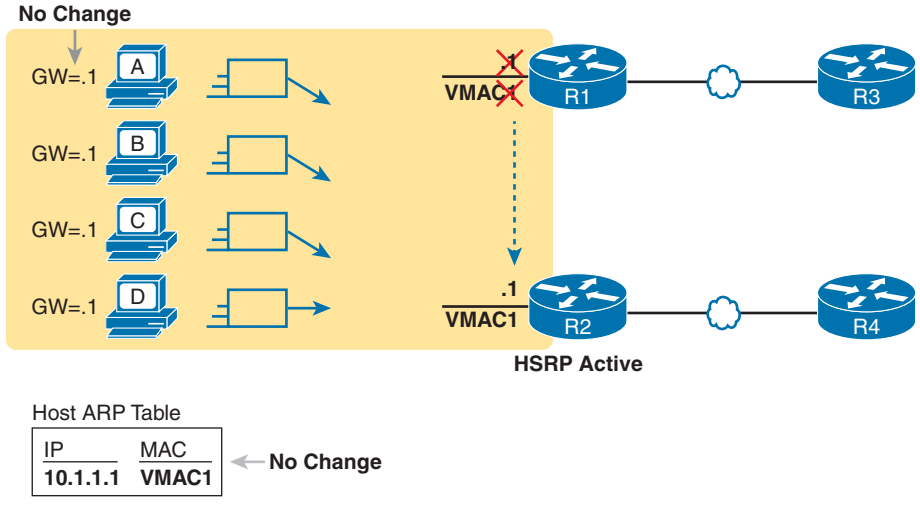



**Figure 16-6** *Packets Sent Through R2 (New Active) After It Takes Over for Failed R1*

The figure shows packets flowing from the hosts toward router R2, with no changes on the hosts. The host keeps the same default router setting, referencing the virtual IP address (10.1.1.1). The host's ARP table does not have to change either, with the ARP entry for the default router listing the virtual MAC.

To direct the Ethernet frames that formerly flowed to router R1 to instead flow to router R2, changes occur on both the routers and the LAN switches. The new active router (R2) must be ready to receive packets (encapsulated inside frames) using the virtual IP and MAC addresses. The LAN switches, hidden in the last few figures, must also change their MAC address tables. Formerly, their MAC tables directed frames destined for VMAC1 to router R1, but now the switches must know to send the frames to the new active router, R2.

To make the switches change their MAC address table entries for VMAC1, R2 sends an Ethernet frame with VMAC1 as the source MAC address. The switches, as normal, learn the source MAC address (VMAC1) but with new ports that point toward R2. The frame is also a LAN broadcast, so all the switches learn a MAC table entry for VMAC1 that leads toward R2. (By the way, this Ethernet frame holds an ARP Reply message, called a gratuitous ARP, because the router sends it without first receiving an ARP Request.)

## HSRP Load Balancing

The active/standby model of HSRP means that all hosts send their off-subnet packets through only one router. In other words, the routers do not share the workload; instead, one router forwards all the packets. For instance, back in Figure 16-5, R1 was the active router. All hosts in the subnet sent their packets through R1, and none of them sent their packets through R2.

HSRP does support load balancing by preferring different routers to be the active router in different subnets. Most sites that require a second router for redundancy also use several VLANs and subnets at the site. The two routers will likely connect to all the VLANs, acting as the default router in each subnet. The HSRP configuration settings can result in one router being active in one subnet and another router being active in another subnet, balancing the traffic. Or you can configure multiple instances of HSRP in the same subnet (called multiple HSRP groups), preferring one router to be active in one group and the other router to be selected as active in another.

For instance, Figure 16-7 shows a redesigned LAN with two hosts in VLAN 1 and two in VLAN 2. R1 and R2 connect to the LAN using a VLAN trunking and router-on-a-stick (ROAS) configuration. The two routers define two HSRP groups, one to support each of the two subnets. In this case, R1 wins and becomes active in Subnet 1, while router R2 becomes active in Subnet 2.



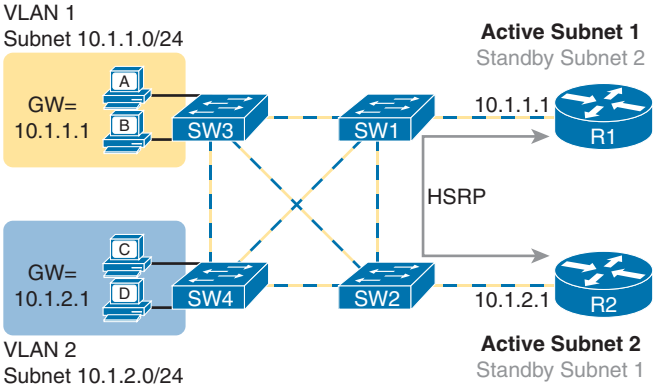



**Figure 16-7** *Load Balancing with HSRP by Using Different Active Routers per Subnet*

Note that the design uses both routers and WAN links by having each router act as the HSRP active router in some subnets.

The example surrounding Figure 16-7 raises the question of where to consider using HSRP. You should consider an FHRP on any router or Layer 3 switch interface with an IP address that connects to hosts that rely on a default router setting. If only one router connects to the subnet, you do not need an FHRP, but if two or more connect to the subnet, you benefit from using an FHRP.

## HSRP Interface Tracking

Another feature supported by all the FHRPs tracks the operational state of other router features. IOS allows for tracking of interface state, tracking routes in the IP routing table, and other types of objects. When the tracked interface or object fails, HSRP reduces that router's

HSRP priority. With well-chosen priority and tracking settings, you can arrange the HSRP configuration so that when everything works perfectly, one router is active. Later, when something fails related to that router, another router preempts and takes over as the active router.

Figure 16-8 shows one classic failure case that can occur without tracking. In this example, router R1 uses priority 110, with router R2 using 100 (the default), so R1 wins and becomes HSRP active. However, the one WAN link connected to R1 fails. R1 remains the HSRP active router. In this failure case, hosts forward packets to router R1, which has to forward them to router R2, which has the only working WAN link.

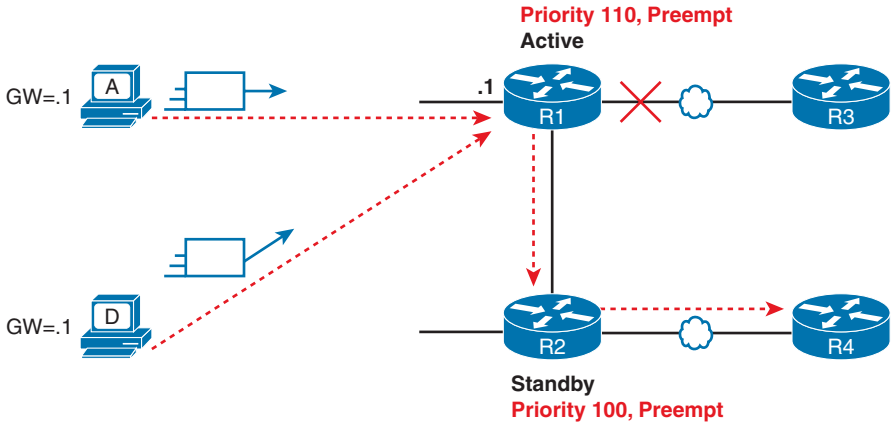



**Figure 16-8** *Problem: Extra Routing Hop When the R1 WAN Link Fails*

A better plan links the HSRP role to the WAN link state. For instance, with interface tracking and preemption, you can configure HSRP as follows:

If R1's WAN link is up, make R1 HSRP active.

If R1's WAN link is down, make R2 HSRP active.

To do so, HSRP object tracking on R1 monitors the state of the R1 WAN interface. If the interface fails, HSRP lowers R1's HSRP priority, allowing R2 to preempt R1. For example:

1. R1 notices its WAN interface fails.
2. R1 lowers its HSRP priority by 20 points.
3. HSRP messages allow both R1 and R2 to realize that R2 has a better (higher) priority and has the right to preempt.
4. R2, using preemption, becomes the HSRP active router.

As a result, while router R1's WAN link is down, packets from the hosts on the left flow first to R2 and then out a WAN link. Later, when R1's WAN link recovers, it preempts router R2 and becomes active again.

## HSRP Recovery and Preemption

The **HSRP preemption** also dictates what happens when a formerly active router recovers, independent of any HSRP tracking. HSRP disables preemption by default but supports it on any router.

First, consider the following scenario:

1. R1 has priority 110, and R2 has priority 100, so R1 wins and becomes active, while R2 becomes standby.
2. Later, R1 fails, so R2 becomes active.
3. When R1 recovers, what happens?

With the default setting of no preemption, R2 remains the active router. With only two routers in the HSRP group, when R1 recovers, it moves to a standby state, ready to take over from R2 when R2 next fails. In that case, the operations staff can choose when to make the failover happen, for instance, off-shift, when a minor outage has no impact.

If you prefer, you can enable preemption. With preemption, in this case, or any case in which a new router appears in the group that has a better (higher) priority than the active router, it takes over as the active router. The switch to a new active router is methodical but quick, without waiting for timers like the Hello and Hold timers to expire, but it might cause a disruption in packet flow for a second or two. (Note that preemption does not apply to cases where the priorities tie, but only when the new router has a higher priority.)

## HSRP Versions



Cisco routers and Layer 3 switches support two versions of HSRP: versions 1 and 2. The versions have enough differences, like multicast IP addresses used and message formats, so routers in the same HSRP group must use the same version. Suppose two routers configured in the same HSRP group mistakenly use different versions. In that case, they will not understand each other and will ignore each other for the purposes of HSRP.

There are good reasons to use the more recent HSRP version 2 (HSRPv2). HSRPv2 added IPv6 support. It also supports faster convergence when changes happen using shorter Hello and Hold timers, while HSRPv1 typically had a minimum of a 1-second Hello timer. Table 16-3 lists the differences between HSRPv1 and HSRPv2.

Key Topic

**Table 16-3** HSRPv1 Versus HSRPv2

| Feature | Version 1 | Version 2 |
|---|---|---|
| IPv6 support | No | Yes |
| Smallest unit for Hello timer | Second | Millisecond |
| Range of group numbers | 0..255 | 0..4095 |
| Virtual MAC address used (*xx* or *xxx* is the hex group number) | 0000.0C07.AC*xx* | 0000.0C9F.F*xxx* |
| IPv4 multicast address used | 224.0.0.2 | 224.0.0.102 |

Ensure you understand how a router chooses the virtual MAC shown in the table. HSRPv1 supports 256 groups per interface, while HSRPv2 supports 4096. You can represent decimal values 0..255 with two-digit hexadecimal equivalents of 00..FF, while decimal values 0..4095 require three hex digits from 000..FFF. The HSRP virtual MAC addresses use the hex equivalents of the configured decimal HSRP group number as the last two or three digits of the virtual MAC address as follows:

HSRPv1: 0000.0C07.AC*xx*, where *xx* is the hex group number

HSRPv2: 0000.0C9F.F*xxx*, where *xxx* is the hex group number

For example, an HSRPv1 group 1 would use virtual MAC address 0000.0C07.AC**01**, while an HSRPv2 group would use 0000.0C9F.F**001**. For group decimal 100 (hex 64), they would use 0000.0C07.AC**64** and 0000.0C9F.F**064**, respectively.

## VRRP and GLBP Concepts

Now that you have a thorough understanding of the purpose, functions, and concepts of HSRP, this third major section of the chapter examines the two other FHRPs: VRRP and GLBP. Both provide the same primary functions as HSRP. VRRP has more similarities with HSRP, while GLBP goes beyond HSRP with better load-balancing features.

### Virtual Router Redundancy Protocol (VRRP)

HSRP and VRRP emerged in the 1990s when TCP/IP and routers first became common in corporate networks. As is often the case, Cisco saw a need, but with no standards-based solution, so they defined HSRP as a proprietary solution for first hop router redundancy. Later, the IETF created VRRP, providing similar features. However, unlike many stories of Cisco-proprietary pre-standard features, HSRP has not faded into history; you will still find both HSRP and VRRP support in many Cisco product families.

> **NOTE** While VRRP includes versions 1, 2, and 3, all references in this chapter refer to VRRPv3 (RFC 5798.)

For similarities, note that VRRP supports all the same functions as HSRP, as described earlier in this chapter. The purpose remains to provide a standby backup for the default router function, preemption if desired, and load balancing the default router role by using multiple VRRP groups.

The differences come with default settings, protocol details, and addresses used. Table 16-4 lists some comparison points between HSRP, VRRP, and GLBP (ignore GLBP for now.)

Key Topic

**Table 16-4** Comparing Features of the Three FHRP Options

| Acronym | HSRPv2 | VRRPv3 | GLBP |
|---|---|---|---|
| Cisco Proprietary | Yes | No | Yes |
| VIP must differ from the routers' interface IP addresses | Yes | No | Yes |
| Preemption off by default | Yes | No | Yes |
| Allows preemption (or not) | Yes | Yes | Yes |
| Default priority value (decimal) | 100 | 100 | 100 |
| Supports tracking to change the priority | Yes | Yes | Yes |
| Supports IPv4 and IPv6 | Yes | Yes | Yes |
| Active/active load balancing with multiple active routers in one group | No | No | Yes |
| IPv4 multicast address used | 224.0.0.102 | 224.0.0.18 | 224.0.0.102 |
| Group numbers supported in IOS | 0–4095 | 1–255 | 0–1023 |
| Virtual MAC address pattern | 0000.0c9f.fxxx | 0000.5e00.01xx | 0007.b40x.xxrr |

You can configure VRRP so that it appears to work like HSRP. Two or more VRRP routers form a group within one subnet. VRRP routers define one VIP, use multicast messages to communicate with each other, use an active/standby approach, select the active router with the same logic as HSRP, allow tracking, and fail over when the master (active) router fails. (Note that VRRP uses the terms *master* and *backup* rather than *active* and *standby*.)

One difference comes in the choice of VIP. You can use the same IP address as one of the VRRP routers' interface addresses or, like HSRP, use another IP address in the subnet. For example, the HSRP discussion around Figures 16-5 and 16-6 used VIP 10.1.1.1, with router addresses 10.1.1.9 and 10.1.1.8. You could do the same with VRRP or use 10.1.1.9 (the same IP address as router R1's interface IP address).

VRRP has protocol differences as well. It uses a multicast IPv4 address (224.0.0.18) for its messages. While it uses a single virtual MAC per group, the MAC address follows a different pattern. VRRP configuration uses decimal group numbers from 1 to 255 decimal. The virtual MAC uses the equivalent two-digit hex group number at the end of the virtual MAC, with VRRP routers choosing their virtual MAC based on this pattern:

VRRPv3: 0000.5e00.01*xx*, where *xx* is the hex group number



## GLBP Concepts

Cisco-proprietary GLBP, defined after HSRP and VRRP, provides the same benefits as HSRP and VRRP but with different implementation details. But it also includes different internals that allow much more effective load balancing. So, while used for redundancy (the *R* in FHRP), GLBP also adds robust load balancing, per its name.

This GLBP section begins with comparisons to the other FHRPs and then discusses its improved approach to load balancing.

### Similarities of GLBP, HSRP, and VRRP

GLBP provides redundancy for the default router function while hiding that redundancy from the hosts using that default router address. But most of the core features follow a familiar theme:

- It uses a virtual IP address (VIP), which is the address used by endpoints as their default router.
- It identifies the best router in the group based on the highest priority.
- It allows for the preemption of the best router when a new router with a better (higher) priority joins the group.
- It supports tracking, which dynamically lowers one router's priority, allowing another router to preempt the first based on conditions like an interface failure.
- It sends messages using multicasts but uses a different address: 224.0.0.102.

GLBP uses virtual MAC addresses differently than the other FHRPs as part of the underlying support for load balancing. Like HSRP and VRRP, a GLBP group has one VIP. Unlike HSRP and VRRP, the routers in a group do not use one virtual MAC address whose function resides with the one active router. Instead, GLBP uses a unique virtual MAC address per GLBP router.

The MAC address value includes three hex digits to represent the decimal GLBP group number, with the unique last two digits (01, 02, 03, or 04) representing the four allowed GLBP routers in a group. The MAC address pattern is 0007.b40*x.xxrr*. For instance, for two routers in the same GLBP group:

Key Topic

**Router R1:** 0007:b400**:1401** (Decimal group 20, which is hex group 014, assigned router number 01)

**Router R2:** 0007:b400**:1402** (Decimal group 20, which is hex group 014, assigned router number 02)

### GLBP Active/Active Load Balancing

With a name like Gateway Load Balancing Protocol, load balancing should be a key feature. The term *gateway* refers to the alternate term for default router (*default gateway*), so by name, GLBP claims to load balance across the default routers in a subnet—and it does.

GLBP manipulates the hosts' IP ARP tables in a subnet so that some hosts forward packets to one router and some to another. As usual, all the hosts use the same VIP as their default router address. Under normal conditions, with multiple GLBP routers working in the subnet, GLBP spreads the default router workload across all GLBP group members. When one of those routers fails, GLBP defines the methods by which the remaining router or routers take over the role of the failed router.

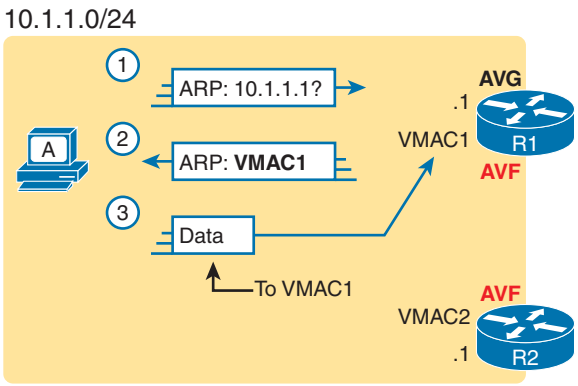
To achieve this active/active load balancing, one GLBP performs the role of **GLBP active virtual gateway (AVG)**. The AVG handles all ARP functions for the VIP. Knowing the virtual MAC addresses of all the routers in the group, the AVG replies to some ARP Requests with one virtual MAC and some with the other. As a result, some hosts in the subnet send frames to the Ethernet MAC address of one of the routers, with different hosts sending their frames to the MAC address of the second router.

All routers serve as a **GLBP active virtual forwarder (AVF)** to support load balancing. All the AVFs sit ready to receive Ethernet frames addressed to their unique virtual MAC address and to route the encapsulated packets as usual. Note that one router serves as both AVG and AVF.

Figures 16-9 and 16-10 show the results of two ARP Reply messages from AVG R1. First, Figure 16-9 shows how a GLBP balances traffic for host A based on the ARP Reply sent by the AVG (R1). The two AVF routers support virtual IP address 10.1.1.1, with the hosts using that address as their default router setting.

The figure shows three messages, top to bottom, with the following action:

1. Host A has no ARP table entry for its default router, 10.1.1.1, so host A sends an ARP Request to learn 10.1.1.1's MAC address.
2. The GLBP AVG, R1 in this case, sends back an ARP Reply. The AVG includes its virtual MAC address in the ARP Reply, VMAC1.
3. Host A encapsulates future IP packets in Ethernet frames destined for VMAC1, so they arrive at R1 (also an AVF).




GLBP Table

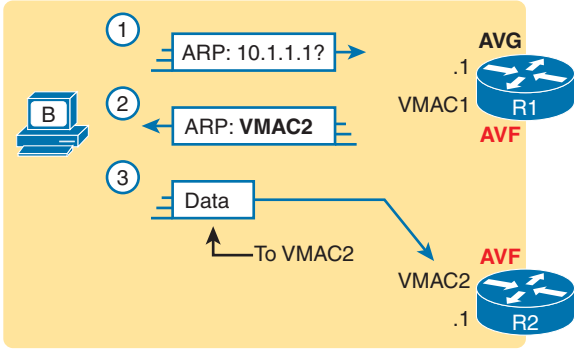
| Role | Router | Address |
|---|---|---|
| AVG | R1 | 10.1.1.1 |
| Forwarder | R1 | **VMAC1** |
| Forwarder | R2 | VMAC2 |

**Figure 16-9** *GLBP Directs Host A by Sending Back the ARP Reply with R1's VMAC1*

To balance the load, the AVG answers each new ARP Request with the MAC addresses of alternating routers. Figure 16-10 continues the load-balancing effect with host B's ARP Request for 10.1.1.1. The router acting as AVG (R1) still sends the ARP Reply, but this time with R2's virtual MAC (VMAC2).



Key Topic




GLBP Table

| Role | Router | Address |
|---|---|---|
| AVG | R1 | 10.1.1.1 |
| Forwarder | R1 | VMAC1 |
| Forwarder | R2 | **VMAC2** |

**Figure 16-10** *GLBP Directs Host B by Sending Back the ARP Reply with R2's VMAC2*

Here are the steps in the figure:

1. Host B sends an ARP Request to learn 10.1.1.1's MAC address.
2. The GLBP AVG (R1) sends back an ARP Reply, listing VMAC2, R2's virtual MAC address.
3. Host B encapsulates future IP packets in Ethernet frames destined for VMAC2, so they arrive at R2.

Finally, to capture a few related points beyond this GLBP example, note that GLBP uses priority, preemption, and tracking. However, those rules apply to the AVG only; all GLBP routers serve as AVFs. So, if the AVG fails, the remaining routers in a GLBP group elect a new AVG.

That model requires additional logic to deal with AVF failures. When a router serving as only an AVF fails, the AVG recognizes the failure and causes a still-functional AVF to begin receiving frames sent to the failed AVF's virtual MAC address.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 16-5 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 16-5** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |

## Review All the Key Topics

Key Topic

**Table 16-6** Key Topics for Chapter 16

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Common characteristics of all FHRPs | 355 |
| Table 16-2 | Comparisons of HSRP, VRRP, GLBP | 356 |
| Figure 16-5 | HSRP concepts | 357 |
| Figure 16-6 | HSRP failover results | 358 |
| Table 16-3 | Comparing HSRPv1 and HSRPv2 | 361 |
| Table 16-4 | Comparing HSRP, VRRP, and GLBP | 362 |
| List | GLBP virtual MAC addresses | 364 |
| Figure 16-10 | GLBP AVG ARP Reply referring to a different GLBP router | 365 |

## Key Terms You Should Know

First Hop Redundancy Protocol (FHRP), Gateway Load Balancing Protocol (GLBP), GLBP active virtual forwarder (AVF), GLBP active virtual gateway (AVG), Hot Standby Router Protocol (HSRP), HSRP active, HSRP preemption, HSRP priority, HSRP standby, virtual IP address (VIP), virtual MAC address, Virtual Router Redundancy Protocol (VRRP)

*This page intentionally left blank*



CHAPTER 17

# SNMP, FTP, and TFTP

**This chapter covers the following exam topics:**

**4.0 Infrastructure Services**

**4.4 Explain the function of SNMP in network operations**

**4.9 Describe the capabilities and functions of TFTP/FTP in the network**

To close this part of the book about a variety of IP Services, this chapter focuses on two topics of managing routers and switches as an end to themselves.

Network engineers have used Simple Network Management Protocol (SNMP) as the main network management protocol for decades. As per the associated exam topic, the first major section focuses on SNMP concepts rather than configuration, including how managed devices—SNMP agents—can be interrogated by network management systems—SNMP clients—to find the current status of each device.

The second major section of the chapter examines the mechanics of transferring files with the FTP and TFTP protocols. The first topic of this section focuses on a few practical uses of TFTP and FTP when working with Cisco routers to upgrade the IOS. Armed with that practical knowledge, you then look at the protocol details of both FTP and TFTP in the rest of the section.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 17-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Simple Network Management Protocol | 1–3 |
| FTP and TFTP | 4–6 |

1. A Network Management Station (NMS) is using SNMP to manage some Cisco routers and switches with SNMPv2c. Which of the following answers most accurately describes how the SNMP agent on a router authenticates any SNMP Get requests received from the NMS?

   **a.** Using a username and hashed version of a password

   **b.** Using either the read-write or read-only community string

c. Using only the read-write community string
d. Using only the read-only community string

2. Which of the following SNMP messages are typically sent by an SNMP agent? (Choose two answers.)
   a. Trap
   b. Get Request
   c. Inform
   d. Set Request
3. Which SNMP verbs were added for SNMP Version 2 to improve the efficiency of retrieving lists of related MIB variables?
   a. GetBulk
   b. Get
   c. GetNext
   d. Inform
4. An FTP client connects to an FTP server using active mode and retrieves a copy of a file from the server. Which of the answers describes a TCP connection initiated by the FTP client?
   a. The FTP control connection only
   b. The FTP data connection only
   c. Both the FTP data and control connections
   d. Neither the FTP data nor control connections
5. Which of the following functions are supported by FTP but not by TFTP? (Choose two answers.)
   a. Transferring files from client to server
   b. Changing the current directory on the server
   c. Transferring files from server to client
   d. Listing directory contents of a server's directory
6. In an IOS XE router, the **pwd** command output lists directory "bootflash:". Which answers describe what you would expect to see in the output of the **show bootflash:** and **dir** commands? (Choose two answers.)
   a. The **dir** command lists all files in file system bootflash:, including files in subdirectories.
   b. The **show bootflash:** command lists all files in file system bootflash:, including files in subdirectories.
   c. The **dir** command lists files in the root of bootflash:, but not files in subdirectories.
   d. The **show bootflash:** command lists files in the root of bootflash:, but not files in subdirectories.

# Foundation Topics

## Simple Network Management Protocol

In 1988, RFC 1065, "Structure and Identification of Management Information for TCP/IP-based Internets," was published. SNMP uses a model of breaking down the information about devices on a TCP/IP-based network—configuration settings, status information, counters, and so on—into a database of variables. Management software could then collect those variables to monitor and manage the IP-based network. After all, the elements of any IP-based machines would have commonalities. For example, a PC, a network printer, and a router would all have commonalities such as interfaces, IP addresses, and buffers. Why not create a standardized database of these variables and a simple system for monitoring and managing them? This idea was brilliant, caught on, and became what we know today as **Simple Network Management Protocol (SNMP)**.

In this major section of the chapter, we now turn our attention to SNMP by looking at the major concepts along with the two common versions used today: **SNMPv2c** and **SNMPv3**.

SNMP defines request and response messages for communication between **SNMP managers** and **SNMP agents**. An SNMP manager is a network management application running on a PC or server, with that host typically called a **Network Management System (NMS)**. Many SNMP agents exist in the network, one per device that is managed. The SNMP agent is software running inside each device (router, switch, and so on), with knowledge of all the variables on that device that describe the device's configuration, status, and counters. The SNMP manager uses SNMP protocols to communicate with each SNMP agent.

Each agent keeps a database of variables that comprise the parameters, status, and counters for the device's operations. This database, called the **Management Information Base (MIB)**, has some core elements in common across most networking devices. It also has many variables unique to that type of device—for instance, router MIBs will include variables not needed on switch MIBs, and vice versa. (For perspective, I checked a router when writing this section and found a little over 7000 MIB variables on a router.)

Figure 17-1 connects a few of these ideas and terms. First, many companies sell SNMP management products. For example, the Cisco Catalyst Center (formerly Cisco DNA Center) controller (www.cisco.com/go/catalystcenter) uses SNMP (and other protocols) to manage networks. IOS on routers and switches include an SNMP agent with built-in MIB.

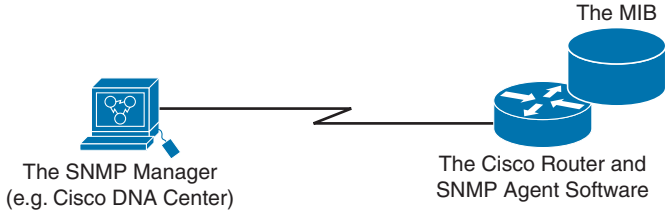



**Figure 17-1** *Elements of Simple Network Management Protocol*

---

Answers to the "Do I Know This Already?" quiz:

**1** B **2** A, C **3** A **4** A **5** B, D **6** B, C

## SNMP Variable Reading and Writing: SNMP Get and Set

The NMS typically polls the SNMP agent on each device. The NMS can notify the human user in front of the PC or send emails, texts, and so on to notify the network operations staff of any issues identified by the data found by polling the devices. You can even reconfigure the device through these SNMP variables in the MIB if you permit this level of control.

Specifically, the NMS uses the **SNMP Get** request message to ask for information from an agent. The NMS sends an **SNMP Set** request message to write variables on the SNMP agent to configure the device. These messages come in pairs, with, for instance, a Get Request asking the agent for the contents of a variable, and the Get Response supplying that information. Figure 17-2 shows an example of a typical flow, with the NMS using an SNMP Get to ask for the MIB variable that describes the status of a particular router interface.

Key Topic

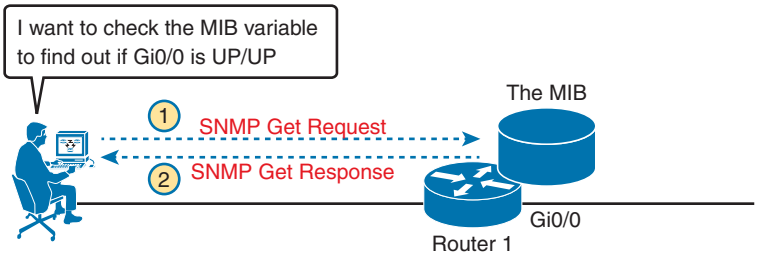



**Figure 17-2** *SNMP Get Request and Get Response Message Flow*

The original SNMP Version 1 (SNMPv1) standard defined two types of Get requests: Get and GetNext. The Get request lists one or more specific variable names, with the SNMP agent sending back a Get response that lists the values of those specific variables. However, the Get response requires the listing of a specific variable, which requires some processing by the NMS.

The GetNext request takes a slightly different approach that reduces some processing at the NMS. GetNext also asks for the contents of variables; however, a GetNext request asks for the next variable in the agent's MIB after the listed variable. So, the NMS can do the work to find the specific name of one variable, and then issue a series of GetNext requests to get the next instance of that variable, and the next, and so on.

For instance, imagine a router has many interfaces, as usual. The router's MIB has variables for each interface, for instance, for interface MAC and IP addresses. First, the NMS identifies the specific variable names for the first interface and uses a Get request on those variables. However, to get those same variables for the second interface, GetNext can refer to the names of the variables for the first interface, without having to do the processing to derive the specific variable names for the second interface. It reduces the overhead to traverse and retrieve variables in a list.

SNMP Version 2 (SNMPv2) added the GetBulk request as an additional improvement over GetNext. While one GetNext requests the next variable in a list or table, GetBulk can request more than one successive variable. For instance, to retrieve data for eight router interfaces, the NMS could begin with one Get and then be more efficient with seven more GetNext requests, for each additional interface. One GetBulk request could ask for the MAC and IP address of the next eight interfaces on a router, again making the process more efficient.

SNMP permits much flexibility in how you monitor variables in the MIB. A network administrator gathers and stores statistics over time using the NMS. With the stored data, the NMS can analyze various statistical facts such as averages, minimums, and maximums. To be proactive, administrators can set thresholds for certain key variables, telling the NMS to send a notification (email, text, and so on) when a threshold is passed.

## SNMP Notifications: Traps and Informs

In addition to asking for information with Get commands and setting variables on agents with the Set command, SNMP agents can initiate communications to the NMS. These messages, generally called *notifications*, use two specific SNMP messages: Trap and Inform. Agents send an SNMP Trap or **SNMP Inform** message to the NMS to list the value of certain MIB variables when those variables reach a certain state.

As an example of a Trap, suppose that Router 1's G0/0 interface fails, as shown at step 1 of Figure 17-3. With Traps configured, the router would send an **SNMP Trap** message to the NMS, with that Trap message noting the down state of the G0/0 interface. Then, the NMS software can send a text message to the network support staff, pop up a window on the NMS screen, change the color of the correct router icon to red on the graphical interface, and so on.

Key Topic

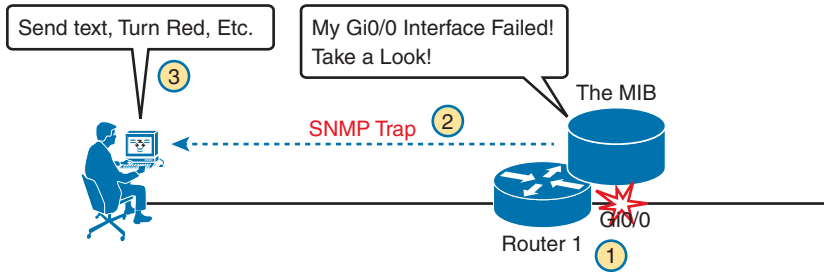



**Figure 17-3** *SNMP Trap Notification Process*

SNMP Trap and Inform messages have the exact same purpose but differ in the protocol mechanisms. SNMP Traps, available since the first version of SNMP (SNMPv1), use a fire-and-forget process. The SNMP agent sends the Trap to the IP address of the NMS, with UDP as the transport protocol as with all SNMP messages, and with no application layer error recovery. If the Trap arrives, great; if it is lost in transit, it is lost.

Inform messages inform the NMS but with reliability added. Added to the protocol with SNMPv2, Informs also use UDP, but add application layer reliability. The NMS must acknowledge receipt of the Inform with an SNMP Response message, or the SNMP agent will time out and resend the Inform.

Note that Traps and Informs both have a useful role today, and Traps are still frequently used. Both inform the NMS. Traps use less overhead on the agent, while Informs improve reliability of the messages but require a little more overhead effort.

## The Management Information Base

Every SNMP agent has its own Management Information Base. The MIB defines variables whose values are set and updated by the agent. The MIB variables on the devices in the network enable the management software to monitor/control the network device.

Each agent organizes its MIB in a hierarchical structure, with the individual variables called an *object ID* (OID). Each node in the tree can be described based on the tree structure sequence, either by name or by number. Figure 17-4 shows a small part of the tree structure of an MIB that happens to be part of the Cisco-proprietary part of the MIB.

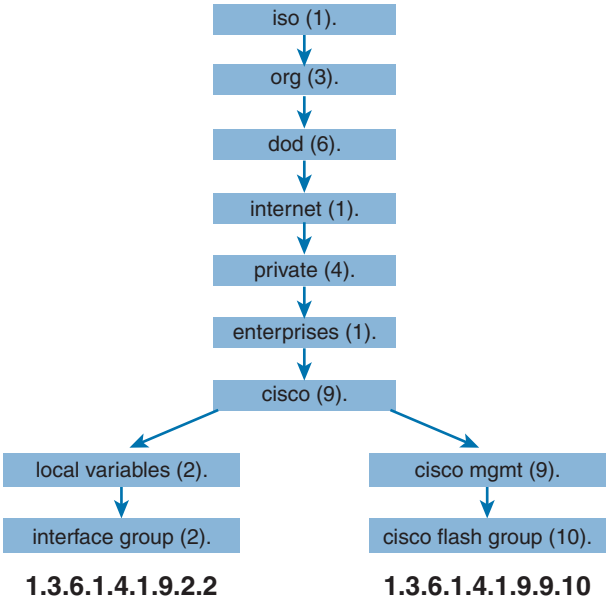



**Figure 17-4** *Management Information Base (MIB)*

Working directly with an MIB, with long variable names and numbers, can be a bit of a challenge, so NMS software typically hides the complexity of the MIB variable numbering and names. However, to get a sense for the variable names, Figure 17-4 shows the tree structure for two variables, with the variable names being the long string of numbers shown at the bottom of the figure. Working with those numbers and the tree structure can be difficult at best. As a result, most people manage their networks using an NMS. For perspective, you could use an SNMP manager and type **MIB variable 1.3.6.1.4.1.9.2.1.58.0** and click a button to get that variable, to see the current CPU usage percentage from a Cisco router. However, most users of an NMS would much prefer to ignore those details and have a simple graphical interface to ask for the same information, never having to know that 1.3.6.1.4.9.2.1.58.0 represents the router CPU utilization MIB variable.

For an SNMP manager to manage each agent, the SNMP manager must have the same MIBs used by the various SNMP agents installed for proper SNMP operation. Each agent comes with a set of standard and vendor-proprietary MIBs built into the product. Expecting that, SNMP manager products also preload the most common MIBs in anticipation of managing those agents. Figure 17-5 shows the concept, with an SNMP manager on the left. It also shows a Cisco and non-Cisco switch on the right. The switches both support the standard MIB-II MIB plus a larger proprietary MIB built by each switch vendor.

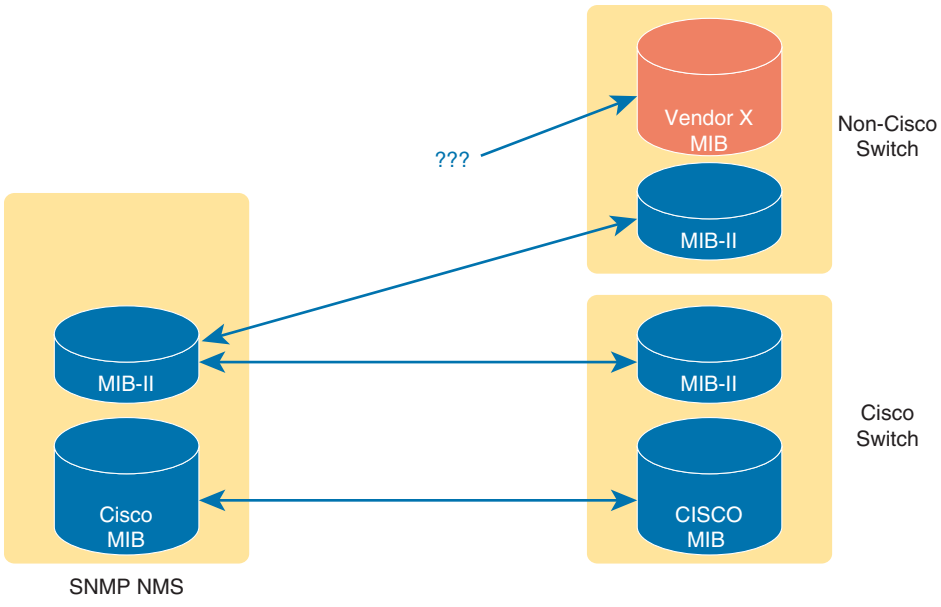



**Figure 17-5** *Proprietary MIB Concepts*

The Cisco NMS comes preloaded with the MIBs for Cisco products, but not for all non-Cisco products. The arrow lines show the MIBs that match between the devices (agents) and the NMS. For those MIB variables, the NMS can issue Get requests to the Cisco switch. The NMS will also understand those variables in Trap or Inform messages. Likewise, the NMS can support Gets, Sets, Traps, and Informs using the MIB II in the non-Cisco switch. However, without the non-Cisco vendor's MIB loaded into the NMS, the NMS cannot ask for data from the non-Cisco vendor's MIB, and it will not understand Traps and Informs that use variables from that MIB. The solution? Download and install the other vendor's MIB into the NMS.

## Securing SNMP

SNMP supports a few security mechanisms, depending in part on the particular version. This section works through the options.

First, one strong method to secure SNMP is to use ACLs to limit SNMP messages to those from known servers only. SNMP agents on Cisco routers and switches support SNMP messages that flow in both IPv4 and IPv6 packets. The SNMP agent can configure an IPv4 ACL to filter incoming SNMP messages that arrive in IPv4 packets and an IPv6 ACL to filter SNMP messages that arrive in IPv6 packets.

Using an IPv4 and IPv6 ACL to secure an agent makes good sense. The only hosts that should be sending SNMP messages to the SNMP agent in a router or switch are the NMS hosts. Those NMS hosts seldom move and their IP addresses should be well known to the networking staff. It makes good sense to configure an ACL that permits packets sourced from the IP addresses of all NMS hosts, but no others.

As for the SNMP protocol messages, all versions of SNMP support a basic clear-text password mechanism, although none of those versions refer to the mechanism as using a password. SNMP Version 3 (SNMPv3) adds more modern security as well.

SNMPv1 defined clear-text passwords called **SNMP communities**. Basically, both the SNMP agent and the SNMP manager need prior knowledge of the same SNMP community value (called a *community string*). The SNMP Get messages and the Set message include the appropriate community string value, in clear text. If the NMS sends a Get or Set with the correct community string, as configured on the SNMP agent, the agent processes the message.

SNMPv1 defines both a read-only community and a read-write community. The **read-only (RO) community** allows Get messages, and the **read-write (RW) community** allows both reads and writes (Gets and Sets). Figure 17-6 shows the concepts. At steps 1 and 2, the agent is configured with particular RO and RW community strings, and the NMS configures the matching values. At step 3, the SNMP Get can flow with either community, but at step 4, the Set Request must use the RW community.

Key Topic

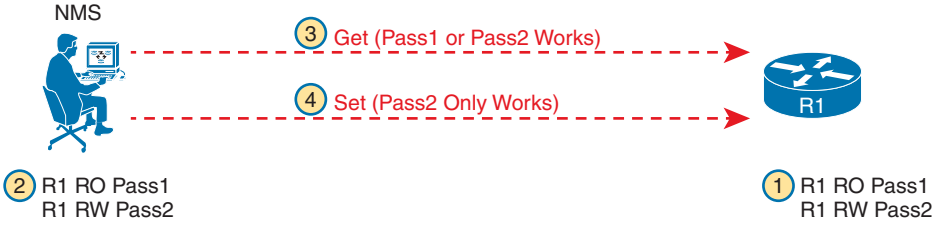



**Figure 17-6** *RO and RW Communities with the Get and Set Commands*

SNMPv2 and the related Community-based SNMP Version 2 (SNMPv2c) added a wrinkle in naming but basically kept the same community security feature as SNMPv1 once the standards process completed. The original specifications for SNMPv2 did not include SNMPv1 communities; however, the marketplace still wanted communities, so an additional RFC added the SNMPv1 communities mechanism back to SNMPv2. This updated RFC, "Community-based SNMPv2," came to be known simply as SNMPv2c. Vendors (including Cisco) implemented SNMPv2c; however, security was still relatively weak.

SNMPv3 arrived with much celebration among network administrators. Finally, security had arrived with the powerful network management protocol. SNMPv3 does away with communities and replaces them with the following features:

Key Topic

- **Message integrity:** This mechanism, applied to all SNMPv3 messages, confirms whether or not each message has been changed during transit.
- **Authentication:** This optional feature adds authentication with both a username and password, with the password never sent as clear text. Instead, it uses a hashing method like many other modern authentication processes.
- **Encryption (privacy):** This optional feature encrypts the contents of SNMPv3 messages so that attackers who intercept the messages cannot read their contents.

**NOTE** The CCNA 200-301 version 1.1 exam blueprint lists SNMP in one exam topic, with that exam topic reduced to "explain SNMP." Some previous versions of the exam required configuration. Refer to Appendix D, "Topics from Previous Editions," if you're interested in learning about SNMP configuration and verification.

# FTP and TFTP

This final major section of the chapter focuses on two topics: File Transfer Protocol (**FTP**) and Trivial File Transfer Protocol (**TFTP**). Both exist as TCP/IP protocols defined in RFCs. Both use a client and server model, in which the client connects to a server and then the client can copy files to the server or from the server. Both exist as a myriad of implementations of both client and server code, from command-line clients to apps with graphical interfaces, using the respective FTP or TFTP protocols behind the scenes.

This section discusses FTP and TFTP with two branches. First, we take a practical view of the most common use of TFTP and FTP by network engineers while on the job: the job of updating IOS images. The process can make use of TFTP and FTP, so this section provides the basics. Second, we move on to talk about FTP and TFTP in a much broader sense, with details about each protocol, their capabilities, and what capabilities each provides to any user.

## Managing Cisco IOS Images with FTP/TFTP

IOS exists as a file—a single file—that the router then loads into RAM to use as its operating system. To better understand the process, you must understand a few more details about how IOS works. In particular, you need to understand the **IOS file system (IFS)**, which defines how IOS stores files (including the IOS file). The **IOS image** upgrade process occurs by copying new IOS files into the router and then booting the router with that new IOS.

### The IOS File System

Every OS creates file systems to store files. A computer needs some type of permanent storage, but it needs more than just a place to store bytes. The OS organizes the storage into a file system, which includes directories, a structure, and filenames, with the associated rules. By using a file system, the OS can keep data organized so the user and the applications can find the data later.

Every OS defines its own file system conventions. Windows OSs, for instance, use a left-leaning slash (\) in directory structures, like \Desktop\Applications. Linux and macOS use a right-leaning slash, for example, /Desktop. Each OS refers to physical disks slightly differently as well, and IOS is no different.

As for the physical storage, Cisco routers typically use **flash memory**, with no hard disk drive. Flash memory is rewriteable, permanent storage, which is ideal for storing files that need to be retained when the router loses power. Cisco purposefully uses flash memory rather than hard disk drives in its products because there are no moving parts in flash memory, so there is a smaller chance of failure as compared with disk drives. Some routers have flash memory on the motherboard. Others have flash memory slots that allow easy removal and replacement of the flash card, but with the intent that the card remain in the device most of the time. Also, many devices have USB slots that support USB flash drives.

For each physical memory device in the router, IOS creates a simple IOS file system of type *disk*. In Example 17-1, a Cisco 1108 ISR router has an internal flash memory, with just under 3 GB of memory, which can be referenced as either bootflash:, flash:, or crashinfo:. The bottom of the output confirms that the single USB flash slot holds a flash drive with just under 32 GB, referenced by name usb0:.

**Example 17-1** *Cisco IOS File Systems on a Router*

```
R2# show file systems
File Systems:

      Size(b)       Free(b)      Type  Flags  Prefixes
            -             -    opaque     rw   system:
            -             -    opaque     rw   tmpsys:
*  2968264704    1416036352      disk     rw   bootflash: flash: crashinfo:
   1634713600    1557024768      disk     ro   webui:
            -             -    opaque     rw   null:
            -             -    opaque     ro   tar:
            -             -   network     rw   tftp:
            -             -    opaque     wo   syslog:
     33554432      33534559     nvram     rw   nvram:
            -             -   network     rw   rcp:
            -             -   network     rw   http:
            -             -   network     rw   ftp:
            -             -   network     rw   scp:
            -             -   network     rw   sftp:
            -             -   network     rw   https:
            -             -    opaque     ro   cns:
  31022530560   31022514176      disk     rw   usb0:
```

Beyond the physical disks, IFS creates other types of disks for use by the operating system, as noted in this list. Example 17-1 also shows examples of each.

- **Opaque:** To represent logical internal file systems for the convenience of internal functions and commands
- **Network:** To represent external file systems found on different types of servers for the convenience of reference in different IOS commands
- **Disk:** For physical disks, either internal or external flash
- **NVRAM:** A special type for NVRAM memory, the default location of the startup-config file

Many IOS commands refer to files in an IFS, but only some commands refer directly to the files by their formal names. The formal names use the prefix as seen in the far right column of Example 17-1. For instance, the command **more flash0:/wotemp/fred** would display the contents of file *fred* in directory */wotemp* in the first flash memory slot in the router. (The **more** command itself displays the contents of a file.) However, many commands use a keyword that indirectly refers to a formal filename, to reduce typing. For example:

- **show running-config** command: Refers to file system:running-config
- **show startup-config** command: Refers to file nvram:startup-config

## Upgrading IOS Images

One of the first steps to upgrade a router's IOS to a new version is to obtain the new IOS image and put it in the right location. Typically, Cisco routers have their IOS in one of the local physical file systems, most often in permanent flash. The only requirement is that the IOS be in some reachable file system—even if the file sits on an external server and the device loads the OS over the network. However, the best practice is to store each device's IOS file in flash that will remain with the device permanently.

Figure 17-7 illustrates the process to upgrade an IOS image into flash memory, using the following steps:

**Step 1.** Obtain the IOS image from Cisco, usually by downloading the IOS image from Cisco.com using HTTP or FTP.

**Step 2.** Place the IOS image someplace that the router can reach. Locations include TFTP or FTP servers in the network or a USB flash drive that is then inserted into the router.

**Step 3.** Issue the **copy** command from the router, copying the file into the flash memory that usually remains with the router on a permanent basis.

Key Topic




**Figure 17-7** *Copying an IOS Image as Part of the Cisco IOS Software Upgrade Process*

## Copying a New IOS Image to a Local IOS File System Using TFTP

Example 17-2 provides an example of step 3 from Figure 17-7, copying the IOS image into flash memory. In this case, router R2, a Cisco ISR 1108 router, copies an IOS image from a TFTP server at IP address 2.2.2.1.

Key Topic

**Example 17-2** **copy tftp flash** *Command Copies the IOS Image to Flash Memory*

```
R2# copy tftp: flash:
Address or name of remote host []? 2.2.2.1
Source filename []? c1100-universalk9.17.06.03a.SPA.bin
Destination filename [c1100-universalk9.17.06.03a.SPA.bin]?
Accessing tftp://2.2.2.1/c1100-universalk9.17.06.03a.SPA.bin...
Loading c1100-universalk9.17.06.03a.SPA.bin from 2.2.2.1 (via GigabitEthernet0/0/0):
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[OK - 706422748 bytes]


706422748 bytes copied in 187.876 secs (3760047 bytes/sec)
R2#
```

The **copy** command does a simple task—copy a file—but the command also has several small items to check. It needs a few pieces of information from the user, so the command prompts the user for that information by showing the user some text and waiting for the user's input. The bold items in the example show the user's input. The router then has to check to make sure the copy will work. The command works through these kinds of questions:

1. What is the IP address or host name of the TFTP server?
2. What is the name of the file?
3. Ask the server to learn the size of the file, and then check the local router's flash to ask whether enough space is available for this file in flash memory.
4. Does the server actually have a file by that name?
5. Do you want the router to erase any old files in flash?

The router prompts you for answers to some of these questions, as necessary. For each question, you should either type an answer or press **Enter** if the default answer (shown in square brackets at the end of the question) is acceptable. Afterward, the router erases flash memory if directed, copies the file, and then verifies that the checksum for the file shows that no errors occurred in transmission.

**NOTE** Most people use the IOS filenames that Cisco supplies because these names embed information about the IOS image, like the version. Also, if you want to use the same destination filename as the source, avoid the mistake of typing "y" or "yes" to confirm the selection; instead, you would be setting the destination filename to "y" or "yes." Simply press **Enter** to confirm the selection listed in brackets.



### Listing the Files in the IOS File System

You can view the contents of the flash file system to see the IOS file that was just copied by using a couple of commands. It helps to think of each IFS as a disk on your favorite desktop operating system, with directories, subdirectories, and files, because those exist in IFS disks as well. For instance, you can use both the **show flash:** and **dir flash:** commands and see files, both showing some evidence of directories.

First, the **show flash:** command displays all files in the flash: file system in one list, regardless of what subdirectory the file resides in. The output lists each file with its directory structure; however, IOS uses both the bootflash: and flash: aliases to represent this disk, so the output lists directories that begin bootflash: rather than flash:. For instance, line 5 in the output in Example 17-3 (the first line with gray highlights) lists

A directory of /bootflash/.installer

A filename of watchlist

**Example 17-3** *The* **show flash:** *Command Displays All Files and All Subdirectories*

```
R2# show flash:
-#- --length-- ---------date/time---------            path
  1       4096 Aug 01 2022 17:12:23.0000000000 +00:00 /bootflash/
  2       4096 Aug 01 2022 16:12:30.0000000000 +00:00 /bootflash/.installer
  3          5 Aug 01 2022 16:10:40.0000000000 +00:00 /bootflash/.installer/
install_global_trans_lock
  4         50 Aug 01 2022 16:10:40.0000000000 +00:00 /bootflash/.installer/
last_pkgconf_shasum
  5         11 Aug 01 2022 16:11:15.0000000000 +00:00 /bootflash/.installer/
watchlist
! Skipped many lines for brevity...
571  706422748 Jun 27 2022 17:08:17.0000000000 +00:00 /bootflash/c1100-
universalk9.17.06.03a.SPA.bin
! Many lines skipped for brevity...
1416802304 bytes available (1400680448 bytes used)
```

The columns of the **show flash:** command provide some key facts. First, IFS numbers the files with a unique number. The # (file number) column lists those numbers. Notice that the **show flash:** command lists the files in file number order. Moving to the right, the length column notes the number of bytes in each file. On the far right, the path column lists the filenames with the entire directory path (beginning with /bootflash: in this case).

Next, look for the line for file 571 near the bottom of Example 17-3, which happens to be the IOS file copied into this disk earlier in Example 17-2. It has a length of just over 700 MB. If you look back to the **copy** command in Example 17-2, you can see the output there that lists the exact size of the file copy, which matches the file size listed by **show flash:**.

The **dir** command lets you display files by directory instead of as a long list of sequential files. To do so, you navigate using the **cd** (change directory) command and discover the current directory with the **pwd** (present working directory) command. IOS begins with a default IFS disk, typically bootflash: (which holds the IOS file). To show the commands in action, Example 17-4 shows these steps:

1. The **pwd** command confirms the default pwd of "bootflash:".
2. Use the **dir** command to lists files/directories in the current directory.
3. Change to a subdirectory of bootflash: (license_evlog).
4. Repeat the **dir** command to list files in the new current directory.
5. Confirm the present working directory again.

**Example 17-4** *The* **dir** *Command: Display Files in the Present Working Directory*

```
R2# pwd
bootflash:/

R2# dir
Directory of bootflash:/
```

```
88177   drwx            40960   Aug 1 2022 20:34:05 +00:00  tracelogs
64129   drwx             4096   Aug 1 2022 16:12:30 +00:00  .installer
56113   drwx             4096   Aug 1 2022 16:11:48 +00:00  license_evlog
15      -rw-               30   Aug 1 2022 16:11:39 +00:00  throughput_monitor_params
12      -rw-           134935   Aug 1 2022 16:11:23 +00:00  memleak.tcl
11      -rw-             1546   Aug 1 2022 16:11:00 +00:00  mode_event_log
40081   drwx             4096  Jun 27 2022 17:15:02 +00:00  .prst_sync
19      -rw-             1923  Jun 27 2022 17:14:30 +00:00  trustidrootx_ca_092024.ca
18      -rw-        706422748  Jun 27 2022 17:08:17 +00:00  c1100-universalk9.
17.06.03a.SPA.bin
! Lines omitted for brevity
2968264704 bytes total (1416704000 bytes free)

R2# cd license_evlog
R2# dir
Directory of bootflash:/license_evlog/

56121   -rw-             1656   Aug 1 2022 16:13:19 +00:00  SAEventRegular20220801_
161148.log
56120   -rw-             1927  Jul 29 2022 16:13:37 +00:00  SAEventRegular20220727_
161208.log
56119   -rw-             1809   Jul 1 2022 20:28:37 +00:00  SAEventRegular20220701_
184140.log
56118   -rw-             1555  Jun 27 2022 17:22:22 +00:00  SAEventRegular20220627_
171513.log

2968264704 bytes total (1416802304 bytes free)

R2# pwd
bootflash:/license_evlog/
```

Finally, take a moment to compare the usage memory usage statistics from the **show flash:** (Example 17-3) and **dir** (Example 17-4) commands. Both commands end with one line showing memory use statistics. Both list the number of free (unused) bytes; however, the **dir** command lists total bytes (used + unused), while the **show flash:** command lists total bytes used. Make sure you understand the difference. Then use these statistics to check if enough memory exists to copy in new IOS images, and if not, you should expect to remove some old files to make room.

## Verifying IOS Code Integrity with MD5 or SHA512

You download the IOS from Cisco, copy it to your router, and run it. Is it really the code from Cisco? Or did some nefarious attacker somehow get you to download a fake IOS that has a virus?

Cisco provides a means to check the **code integrity** of the IOS file to prevent this type of problem by creating a hash value based on the IOS image. First, when Cisco builds a new IOS image, it calculates and publishes both an MD5 and SHA512 hash value for that specific

IOS file. To create the MD5 hash, Cisco uses as input the IOS file itself and runs the MD5 math algorithm against that file. To create the SHA512 hash, Cisco uses the same process, but with the SHA512 algorithm.

To verify that the IOS file on your router or switch has not been changed, use the **verify** command to re-create the same math that Cisco used to create the hash values. To do so, follow the process shown in Figure 17-8. As shown on the right side of the figure, first, find the hash(es) created by Cisco by finding your IOS file in the downloads area of cisco.com. Then, you run that same MD5 or SHA512 math on your router against the IOS file on the router, using the IOS **verify** command. That command will generate the same hash as listed at the Cisco site: if both hashes are equal, the file has not changed.

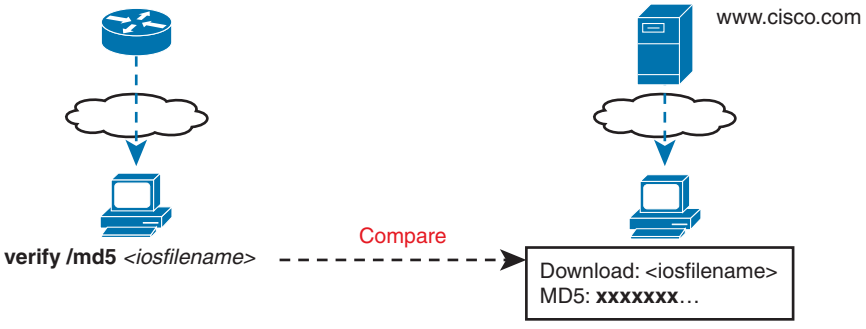



**Figure 17-8** *MD5 Verification of IOS Images—Concepts*

The **verify /sha512** command generates the SHA512 hash on your router, as shown in Example 17-5. Note that you can include the hash value computed by Cisco as the last parameter or omit it. If you include it (as in the example), IOS will compare the locally computed value to the value you pasted into the command and tell you if they match. If you omit the value from the command, the **verify** command lists the locally computed SHA512 hash, but you have to do the picky character-by-character check of the values yourself.

**Example 17-5** *Verifying Flash Memory Contents with the* **verify** *Command*

```
R2# verify /sha512 flash0:c1100-universalk9.17.06.03a.SPA.bin 164199339e08502f9cd-
255f2271ccf6fc633d4abb595d46b13dd280811aca3313a080f6282efe2365076985fe28a2091e2da3f-
00b8ac558785b4090c869f8548
.....................................................................................
.....................................................................................
(many lines of periods skipped for brevity)...
Done!
Verified (bootflash:/c1100-universalk9.17.06.03a.SPA.bin) = 164199339e08502f9cd255f-
2271ccf6fc633d4abb595d46b13dd280811aca3313a080f6282efe2365076985fe28a2091e2da3f-
00b8ac558785b4090c869f8548
```

## Copying Images with FTP

The networking world has many options for file transfer, several of which IOS supports for the transfer of files into and out of the IOS file systems that reside on the router. TFTP and FTP have been supported for the longest time, with more recent support added for protocols

like Secure Copy Protocol (SCP), which uses the SSH File Transfer Protocol (SFTP). Table 17-2 lists some of the names of file transfer protocols that you might come across when working with routers.

**Table 17-2** Common Methods to Copy Files Outside a Router

| Method | Method (Full Name) | Encrypted? |
|---|---|---|
| TFTP | Trivial File Transfer Protocol | No |
| FTP | File Transfer Protocol | No |
| SCP | Secure Copy Protocol | Yes |

To copy files with FTP, you follow the same kind of process you use with TFTP (see Example 17-6). You can follow the interactive prompts after using an EXEC command like **copy ftp flash:**. However, the **copy** command allows you to use a URI for the source and/or destination, which lets you put most or all of the information in the command line itself. Each URI refers to the formal name of a file in the IFS.

Key Topic

**Example 17-6** *Installing a New IOS with FTP*

```
R1# copy ftp://wendell:odom@192.168.1.170/c1100-universalk9.17.06.03a.SPA.bin flash:
Destination filename [c1100-universalk9.17.06.03a.SPA.bin]?
Accessing ftp://192.168.1.170/c1100-universalk9.17.06.03a.SPA.bin...
Loading c1100-universalk9.17.06.03a.SPA.bin
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[OK - 706422748/4096 bytes]

706422748 bytes copied in 119.604 secs (5906544 bytes/sec)
```

First, take a close look at the long URI in the command that begins with "ftp." The "ftp" part identifies the protocol, of course. After the //, the text references the username (wendell) and password (odom), as well as the FTP server's IP address. After the single / comes the filename on the server.

Although the command is long, it has only two parameters, with the long first parameter and the short keyword **flash:** as the second parameter. The **copy** command lists the source location as the first parameter and the destination as the second. The destination in this case, **flash:**, refers to the internal flash file system, but it does not mention the filename. As a result, IOS prompts the user for a specific destination filename, with a default (in brackets) to keep the source filename. In this case, the user just pressed Enter to accept the default. To avoid being prompted at all, the command could have listed **flash:c1100-universalk9.17.06.03a.SPA.bin** as that second parameter, fully defining the destination file.

Finally, with another twist, you can configure the FTP username and password on the router so that you do not have to include them in the **copy** command. For instance, the global configuration commands **ip ftp username wendell** and **ip ftp password odom** would have configured those values. Then the **copy** command would have begun with **copy**

**ftp://192.168.1.170/...**, omitting the username:password in the command, without needing to then prompt the user for the username and password.

## The FTP and TFTP Protocols

The IOS **copy** command, when using the **tftp** or **ftp** keyword, makes the command act as a client. The client connects to a TFTP or FTP server and then attempts to transfer the file. In the examples from the IOS, that **copy** command copied the file from the server into the client device (a router). The rest of this section takes a closer look at both FTP and TFTP as protocols and tools.

### FTP Protocol Basics

FTP has long been a core Internet protocol, serving as the primary file transfer protocol for several decades. RFC 959, which standardizes FTP, dates back to 1985. FTP uses TCP as its transport protocol, relying on TCP to provide an error-free in-order delivery of data so that the FTP application knows that each file transfer creates an exact copy of the file with no omissions. FTP uses well-known TCP port 21, and in some cases, also uses well-known port 20.

As for normal operation, FTP uses a client/server model for file transfer, as shown in the example in Figure 17-9. The figure shows the major steps but not every message. For instance, step 1 shows host A creating a TCP connection to the server (which takes the usual three TCP messages). Step 2 represents the exchange that allows the server to authenticate the client. Step 3 shows the idea that, once authenticated, the client and server can send FTP commands over the connection to tell the other device what to do.

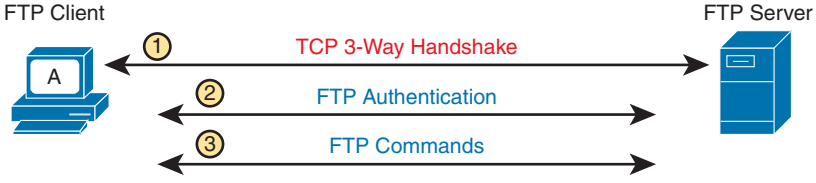



**Figure 17-9** *Major Concepts with FTP Clients and Servers*

The commands that flow over this initial TCP connection—called the **FTP control connection**—define the kinds of functions supported by FTP. Those commands allow the client to navigate around the directory structures of the server, list files, and then transfer files from the server (FTP GET) or to the server (FTP PUT). A summary of some of the FTP actions is as follows:

Key Topic

- **Navigate directories:** List the current directory, change the current directory to a new directory, go back to the home directory, all on both the server and client side of the connection.
- **Add/remove directories:** Create new directories and remove existing directories on both the client and server.
- **List files:** List files on both the client and server.
- **File transfer:** Get (client gets a copy of the file from the server), Put (client takes a file that exists on the client and puts a copy on the FTP server).

While many OSs support command-line FTP clients, which require you to learn the various FTP commands and use those from the command line, most users instead use an FTP client app that issues the FTP commands behind the scenes. Clients typically display files on the local system as well as the server with a user interface that has a similar appearance as a file browser on a desktop OS (for instance, Windows Explorer, macOS Finder). Figure 17-10 shows a sample user interface from the FileZilla FTP client (Filezilla-project.org).

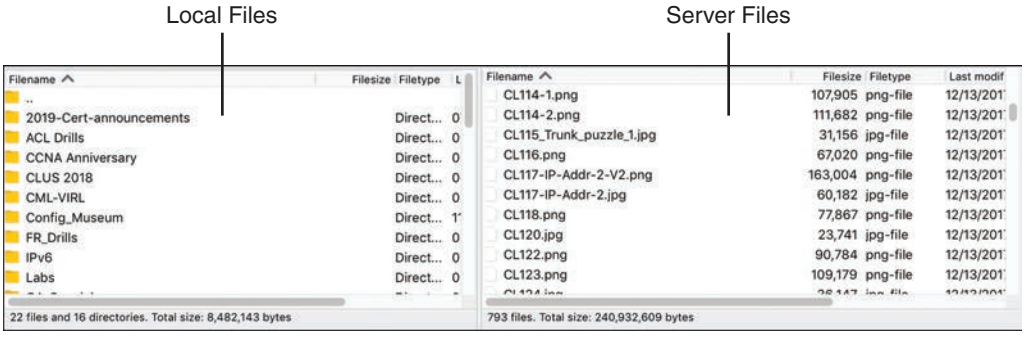



**Figure 17-10** *FTP Client Example with FileZilla*

The client application in Figure 17-10 lists the client computer's local file system on the left and the FTP server's file system on the right. The user can click on the right to change directories, much like using any app that browses a file system, with FTP performing the commands behind the scenes. The user can also drag and drop files from the left to the right to put a file on the server, or vice versa to get a file from the server.

The FTP server can be a server application installed and managed by others, or you can install or enable an FTP server for your own use. For instance, a network engineer might install an FTP server application on a laptop for use in upgrading IOS files, while the IT staff may keep an FTP server available 24/7 for all employees of the company to use. A simple Internet search can show a variety of FTP server applications that run on the common desktop OSs. Additionally, both Windows and macOS come with an FTP or FTPS (FTP Secure) server option built into the OS; all you have to do is enable it. (The Linux distributions all have FTP servers available via simple downloads.) (The aforementioned FileZilla also has a free open-source FTP server option, which can be an easy way to get started.)

Once it is installed, the server can be configured with a variety of settings. For instance, the server needs to specify which users can access the server, so it can use the same login credentials allowed for the host where it resides or specify other credentials. It can specify the directories that each user can access, and whether the user has read-only or read-write access.

### FTP Active and Passive Modes

FTP can operate in either active or passive mode. The choice of mode may impact whether the FTP client can or cannot connect to the server and perform normal functions. The user at the FTP client can choose which mode to use, so this section works through the underlying details to explain why FTP passive mode may be the more likely option to work.

First, note that FTP uses two types of TCP connections:

Key Topic

- **Control Connection:** Used to exchange FTP commands
- **Data Connection:** Used for sending and receiving data, both for file transfers and for output to display to a user

Given the two roles, when a client connects to an FTP server, the client first creates the FTP control connection as shown in Figure 17-11. The server listens for new control connections on its well-known port 21; the client allocates any new dynamic port (49222 in this case) and creates a TCP connection to the server.

Key Topic

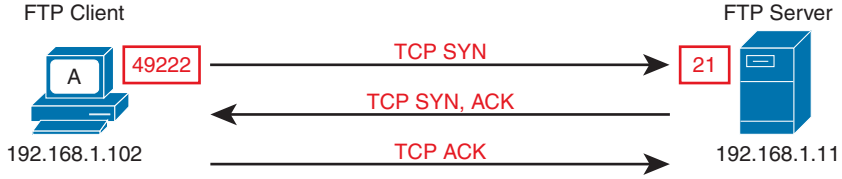



**Figure 17-11** *FTP Client Creates an FTP Control Connection*

After creating the TCP connection, the user authenticates to the FTP server and takes some actions. Some of those actions require only the control connection, but eventually the user will take an action (like getting a file) that requires a data connection. When that happens, to create the **FTP data connection**, the client will either use active mode or passive mode, as shown in the next two examples.

Figure 17-12 shows an example of what happens in active mode. Following the steps in the figure:

1. The FTP client allocates a currently unused dynamic port and starts listening for new connections on that port.
2. The client identifies that port (and its IP address) to the FTP server by sending an FTP **PORT** command to the server over the control connection.
3. The server, because it also operates in active mode, expects the **PORT** command; the server reacts and initiates the FTP data connection to the client's address (192.168.1.102) and port (49333).

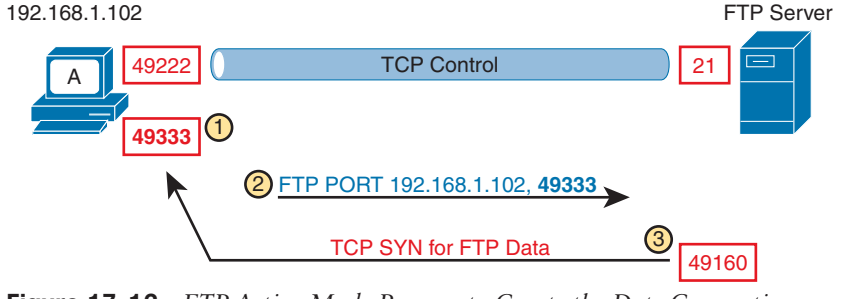



**Figure 17-12** *FTP Active Mode Process to Create the Data Connection*

Note that in active mode, the server initiates the TCP connection for data. That works well with both the FTP client and server sitting inside the same enterprise network. When within the same network, typically no NAT function and no firewall sit between the two.

However, if the FTP client sits in an enterprise network, and the FTP server resides somewhere in the Internet, an active mode connection typically fails. Most firewalls do not allow Internet-based hosts to initiate TCP connections to hosts inside the enterprise without a specific firewall rule allowing connections to a known port, and in this case, the FTP client allocates any available port number. For instance, in Figure 17-12, the TCP connection (step 3) would be discarded by a firewall.

**NOTE** When using Active mode, the server may use its well-known data port 20 or a dynamic port. (Figure 17-12 shows the use of dynamic port 49160.)

Passive mode helps solve the firewall restrictions by having the FTP client initiate the FTP data connection to the server. However, passive mode does not simply cause the FTP client to connect to a well-known port on the server; it requires more exchanges of port numbers to use between the server and client, as shown in Figure 17-13, with these steps:

1. Via messages sent over the control connection, the FTP client changes to use FTP passive mode, notifying the server using the FTP **PASV** command.
2. The server chooses a port to listen on for the upcoming new TCP connection, in this case TCP port 49444.
3. Again using the control connection, the FTP server notifies the FTP client of its IP address and chosen port with the FTP **PORT** command.
4. The FTP client opens the TCP data connection to the IP address and port learned at the previous step.



Key Topic

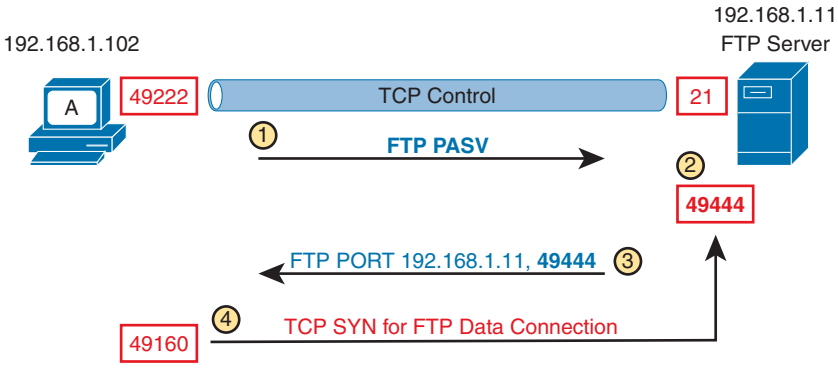



**Figure 17-13** *FTP Passive Mode Process to Create the Data Connection*

## TFTP Protocol Basics

FTP has a role as a general file transfer tool for any user, with a good number of FTP client application options available. TFTP plays a much smaller role as a tool for the average user, but it does play a more useful role for IT support staff.

For the basics, Trivial File Transfer Protocol uses UDP well-known port 69. Because it uses UDP, TFTP adds a feature to check each file for transmission errors by using a checksum process on each file after the transfer completes.

Key Topic

The word *trivial* in the name refers to its relatively small number of features, meant to be an advantage by making the tool lightweight. For instance, TFTP supports far fewer commands than FTP (fewer functions), meaning that the code requires less space to install, which can be useful for devices with limited memory. TFTP can Get and Put files, but it includes no commands to change directories, create/remove directories, or even to list files on the server. TFTP does not support even simple clear-text authentication. In effect, if a TFTP server is running, it should accept requests from any TFTP client.

Ideally, TFTP has its best use as a temporary tool for quick file transfers in a controlled environment, particularly when the data itself does not have to be secure. For instance, imagine this scenario:

1. A network engineer keeps all router and switch IOS images in a folder.
2. The engineer enables a TFTP server on a laptop as needed; otherwise, the TFTP server remains disabled.
3. The engineer connects the laptop to a LAN and enables the TFTP server long enough to transfer IOS images into or out of a few devices.
4. If the engineer forgets to disable TFTP, the only risk is that someone may copy an IOS image—an image that is already available from Cisco.com to any customer.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 17-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 17-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |

## Review All the Key Topics

Key Topic

**Table 17-4** Key Topics for Chapter 17

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 17-2 | The SNMP Get Request and Get Response message flow | 371 |
| Figure 17-3 | SNMP notification with SNMP Trap messages | 372 |
| Figure 17-6 | The use of SNMP RO and RW communities with SNMP Get and Set | 375 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | SNMP security benefits | 375 |
| Figure 17-7 | Process of upgrading IOS using TFTP | 378 |
| Example 17-2 | Example of using TFTP to load new IOS | 378 |
| Example 17-6 | Example of using FTP to load new IOS | 383 |
| List | FTP functions | 384 |
| List | FTP data and control connections | 386 |
| Figure 17-11 | FTP Control connection establishment | 386 |
| Figure 17-13 | FTP data connection establishment in passive mode | 387 |
| Paragraph | Description of limited functions of TFTP | 388 |

## Key Terms You Should Know

code integrity, flash memory, FTP, FTP control connection, FTP data connection, IOS file system, IOS image, Management Information Base (MIB), Network Management System (NMS), read-only community, read-write community, Simple Network Management Protocol (SNMP), SNMP agent, SNMP community, SNMP Get, SNMP Inform, SNMP manager, SNMP Set, SNMP Trap, SNMPv2c, SNMPv3, TFTP

## Command References

Tables 17-5 and 17-6 list configuration and verification commands used in this chapter. As an easy review exercise, cover the left column in a table, read the right column, and try to recall the command without looking. Then repeat the exercise, covering the right column, and try to recall what the command does.



**Table 17-5** Chapter 17 Configuration Command Reference

| Command | Description |
|---|---|
| **boot system flash** [*flash-fs*:] [*filename*] | Global command that identifies the location of an IOS image in flash memory |
| **boot system** {**tftp** \| **ftp**} *filename* [*ip-address*] | Global command that identifies an external server, protocol, and filename to use to load an IOS from an external server |
| **ip ftp username** *name* | Global command to define the username used when referencing the **ftp:** IOS file system but not supplying a username |
| **ip ftp password** *pass* | Global command to define the password used when referencing the **ftp:** IOS file system but not supplying a password |

**Table 17-6** Chapter 17 EXEC Command Reference

| Command | Description |
|---|---|
| **copy** *from-location to-location* | Enable mode EXEC command that copies files from one file location to another. Locations include the startup-config and running-config files, files on TFTP and RPC servers, and flash memory. |
| **show flash:** | Lists the names and size of the files in internal flash memory, and notes the amount of flash memory consumed and available. |
| **dir** *filesystem:/*<br>**dir** *filesystem:directory* | Lists the files in the referenced file system or file system directory. |
| **verify {/md5 \| /sha512}** *filesystem:name [hash-value]* | Performs an MD5 or SHA512 hash of the referenced file and displays the results. If listed, the command compares the hash in the command with the results of performing MD5 or SHA512 hash on the local file. |
| **cd** *directory-name* | If the directory exists, this command changes the present working directory (pwd) variable to refer to the listed directory. |
| **cd ..** | Changes the present working directory setting to one directory above the current directory. |
| **pwd** | Displays the present working directory, a variable used by commands like **dir** to identify a file system directory in which to check for files needed for a command. |

*This page intentionally left blank*



# Part IV Review

Keep track of your part review progress with the checklist shown in Table P4-1. Details on each task follow the table.

**Table P4-1** Part 4 Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Do Labs | | |
| Watch Video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, use the PTP software to answer the "Do I Know This Already?" questions again for the chapters in this part of the book.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Labs

Depending on your chosen lab tool, here are some suggestions for what to do in lab:

**Pearson Network Simulator:** If you use the full Pearson CCNA simulator, focus more on the configuration scenario and troubleshooting scenario labs associated with the topics in this part of the book. These types of labs include a larger set of topics and work well as Part Review activities. (See the Introduction for some details about how to find which labs are about topics in this part of the book.)

**Blog Config Labs:** The author's blog (https://www.certskills.com) includes a series of configuration-focused labs that you can do on paper, each in 10–15 minutes. Review and performs labs for this part of the book by using the menus to navigate to the per-chapter content and then finding all config labs related to that chapter. (You can see more detailed instructions at https://www.certskills.com/config-labs).

**Other:** If using other lab tools, here are a few suggestions: All the exam topics in Part IV that include the word configure exist in Chapters 13 and 14, so focus on those chapters. Those chapters touch on CDP/LLDP, NTP, syslog, and NAT/PAT.

## Watch Video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.



Part V delivers content related to concept-only exam topics. As a result, it moves away from the concept-configure-verify approach of the earlier chapters of this book. Instead, this part collects exam topics better understood from an architecture and design perspective. In fact, the CCNA 200-301 V1.1 exam blueprint organizes six exam topics with this same approach, all listed under exam topic 1.2 "Describe characteristics of network topology architectures." The chapters in this part examine most of those topics.

First, Chapter 18 revisits LAN switching, discussing campus LAN design concepts and terminology, such as the *two-tier* and *three-tier* terms listed in the exam topics. This chapter also discusses how to supply power over that LAN infrastructure using Power over Ethernet (PoE), what the term *small office/home office* (SOHO) means, and the particulars of cabling choices in a campus LAN.

Chapter 19 takes the brief mentions of WAN technologies in the CCNA 200-301 V1.1 blueprint and expands that into a story of three major WAN architectures to explain the topics of MPLS VPN WANs, Ethernet WANs, and Internet VPNs.

Chapter 20 completes the architecture-focused chapters with a discussion of cloud computing and networking. This chapter begins by defining basic concepts and terms related to data centers and cloud, and closes with design discussions that show packet flows in a public cloud environment.

# Part V

## Network Architecture

**Chapter 18:** LAN Architecture

**Chapter 19:** WAN Architecture

**Chapter 20:** Cloud Architecture

**Part V Review**



CHAPTER 18

# LAN Architecture

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.1 Explain the role and function of network components**
    - **1.1.h PoE**
  - **1.2 Describe characteristics of network topology architectures**
    - **1.2.a Two-tier**
    - **1.2.b Three-tier**
    - **1.2.e Small office/home office (SOHO)**
  - **1.3 Compare physical interface and cabling types**
    - **1.3.a Single-mode fiber, multimode fiber, copper**

By now you have learned a lot about Ethernet and Ethernet switches. You have learned how individual links work, with cabling, duplex settings, and framing. You know how addresses work and how switches forward frames based on those addresses. You have seen how switches deal with redundancy, using STP/RSTP and collecting links into EtherChannels. And here in Volume 2, you have learned about various security features available for switches, including Dynamic ARP Inspection, DHCP Snooping, and ARP Inspection.

What the earlier discussions of individual features do not do to any great extent is discuss architecture and design. You now know how switches work, but why would you connect switches in one topology versus another? If you could connect switches in two different topologies, why would you prefer one? This chapter examines a few such design questions, specifically the topic areas mentioned in the CCNA 200-301 V1.1 exam blueprint.

This chapter covers four specific topics that have design-related considerations. The first section looks at the topology of a campus Ethernet LAN and the design terms *two-tier* and *three-tier*, which describe how many switch layers exist between the endpoints and the devices that lead out of the campus to some other site. The second section discusses copper and multimode fiber cabling options for campus links. A third short section examines small office/home office (SOHO) LANs and how they differ from enterprise LANs. The final section introduces the concepts behind Power over Ethernet (PoE) and the reasons why LAN design activities need to consider PoE.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on

the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 18-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Analyzing Campus LAN Topologies | 1–3 |
| Ethernet Physical Media and Standards | 4 |
| Small Office/Home Office | 5 |
| Power over Ethernet | 6, 7 |

1. Which answers best describe the topology in a two-tier campus LAN design? (Choose two answers.)
   a. The design uses a full mesh of links between access and distribution switches.
   b. The design uses a partial mesh of links between access and distribution switches.
   c. The design uses a partial mesh of links between the distribution and core switches.
   d. The end-user and server devices connect directly to access layer switches.
2. In a three-tier campus LAN design, which answers best describe the topology? (Choose two answers.)
   a. The design uses a partial mesh of links between access and distribution switches.
   b. The design uses a full mesh of links between access and distribution switches.
   c. The design uses a partial mesh of links between the distribution and core switches.
   d. The end-user and server devices connect directly to distribution layer switches.
3. Which topology term refers to a network in which one node connects to all other nodes with no other links?
   a. Partial mesh
   b. Full mesh
   c. Hybrid topology
   d. Star topology
4. Which cable categories support 1000BASE-T at distances up to 100 meters? (Choose two answers.)
   a. CAT 5
   b. CAT 5E
   c. CAT 6
   d. OM1
   e. OM3

5. Which answers list criteria typical of a SOHO network? (Choose two answers.)
   a. The AP functions using standalone mode.
   b. The AP functions using a split-MAC architecture using a WLC.
   c. A single networking device implements the router, switch, AP, and firewall functions.
   d. Separate networking devices implement each function (router, switch, AP, and firewall).
6. After deciding the attached device wants to receive power over the UTP cable, how does a PoE LAN switch choose the initial power level to apply?
   a. It applies a standard small amount of voltage.
   b. It asks using CDP messages.
   c. It asks using LLDP messages.
   d. It applies voltage based on a configured value.
7. Which of the following refers to standards that deliver power over all four pairs in a UTP cable? (Choose two answers.)
   a. PoE
   b. UPoE
   c. PoE+
   d. UPoE+

# Foundation Topics

## Analyzing Campus LAN Topologies

The term *campus LAN* refers to the LAN created to support the devices in a building or in multiple buildings close to one another. For example, a company might lease office space in several buildings in the same office park. The network engineers can then build a campus LAN that includes switches in each building, plus Ethernet links between the switches in the buildings, to create a larger campus LAN.

When planning and designing a campus LAN, the engineers must consider the types of Ethernet available and the cabling lengths each type supports. The engineers also need to choose the speeds required for each Ethernet segment. In addition, engineers use switches in different roles—some connect directly to end-user devices, and others connect switches. Finally, most projects require that engineers consider the type of equipment already installed and whether an increase in speed on some segments is worth the cost of buying new equipment.

This first major section of this chapter discusses the topology of a campus LAN design. Network designers do not just plug in devices to any port and arbitrarily connect switches like you might do with a few devices in a lab. Instead, most traditional campus LAN designs follow either two- or three-tier designs, as discussed in the following pages.

## Two-Tier Campus Design (Collapsed Core)

Figure 18-1 shows a typical design of a large campus LAN plus some key terms. This LAN has around 1000 PCs connected to switches that support around 25 ports each. Explanations of the terminology follow the figure.

Key Topic

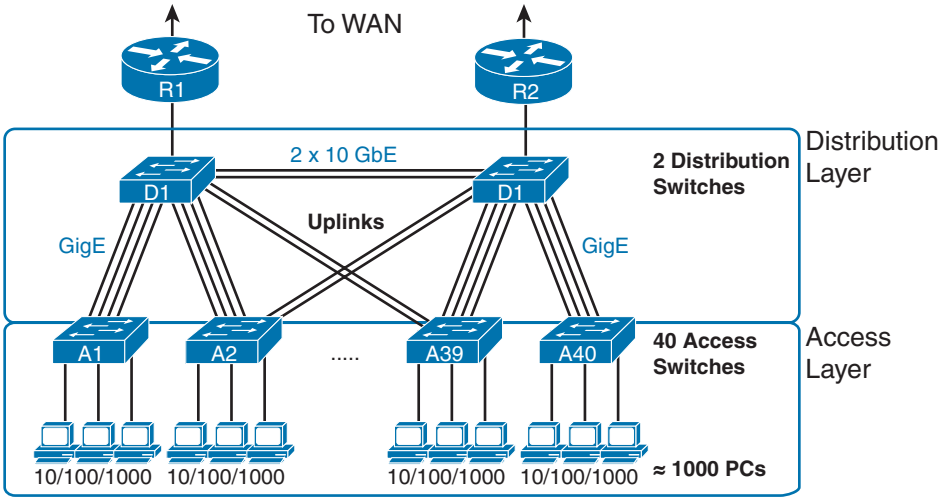



**Figure 18-1** *Campus LAN with Design Terminology Listed*

Cisco uses three terms to describe the role of each switch in a campus design: *access*, *distribution*, and *core*. The roles differ based on whether the switch forwards traffic from user devices and the rest of the LAN (access) or whether the switch forwards traffic between other LAN switches (distribution and core).

*Access switches* connect directly to end users, providing user device access to the LAN. Access switches normally send traffic to and from the end-user devices to which they are connected and sit at the edge of the LAN. **Access links** connect the access switch to the various endpoint devices. The following list summarizes the more common features implemented by access switches:

Key Topic

- Physical network access links for endpoints and wireless APs
- Power over Ethernet (PoE)
- Redundant uplinks with EtherChannel and STP/RSTP
- Security (port security, DHCP Snooping, and DAI)
- A QoS trust boundary created using QoS classification and marking

*Distribution switches* provide a path through which the access switches can forward traffic to each other and outside the campus to the rest of the enterprise. The links between the access and distribution switches—uplinks or **distribution links**—typically connect each access switch to two distribution switches, often with multiple links in an EtherChannel.

Beyond that basic connectivity, the **distribution layer** provides many features that affect the endpoints connected to the access switches. Because designs use only a few distribution switches, and some of the services require additional processing, distribution switches will often need more powerful processors and more memory. The following list summarizes the more common features implemented by distribution switches:

Key Topic

- Connects access switches into the network
- Creates a core with other distribution switches, either directly (two-tier) or using core switches (three-tier)
- Provides Layer 3 features such as interVLAN routing, default router, FHRPs
- Implements dynamic IP routing protocols (e.g., OSPF)
- Load balances over redundant links and devices (EtherChannels, STP/RSTP, FHRPs)
- Provides QoS (queuing, policing, drop)
- Filters IPv4 and IPv6 packets with ACLs

Figure 18-1 shows a two-tier design, with the tiers being the access tier (or layer) and the distribution tier (or layer). A two-tier design solves two major design needs:

- Provides a place to connect end-user devices (the **access layer**, with access switches), typically within 100 meters
- Creates paths between all devices with a reasonable number of cables and switch ports by connecting all 40 access switches to two distribution switches

## Three-Tier Campus Design (Core)

The two-tier design of Figure 18-1, with a **partial mesh** of distribution links, happens to be the most common campus LAN design. It also goes by two common names: a two-tier design (for obvious reasons) and a **collapsed core design** (for less obvious reasons). The term *collapsed core* refers to the two-tier design without a third tier (the core tier). A two-tier design has a core, but the distribution switches implement the core; the core is collapsed into the distribution switches. This next topic examines a three-tier design with a **core layer** for perspective while further detailing the purpose of the core.

Imagine your campus has just two or three buildings. Each building has a two-tier design inside the building, with a pair of distribution switches in each building and access switches spread around the building as needed. How would you connect the LANs in each building? Well, with just a few buildings, it makes sense to simply cable the distribution switches together in a **full mesh**, as shown in Figure 18-2.

The design in Figure 18-2 works well, and many companies use this design. Sometimes the center of the network uses a full mesh, sometimes a partial mesh, depending on the availability of cables between the buildings.

---

Answers to the "Do I Know This Already?" quiz:

**1** B, D **2** A, C **3** D **4** B, C **5** A, C **6** A **7** B, D

Using a third tier in the center of the campus—the core tier—has several advantages in large campuses. For instance, you can use far fewer switch ports and links while connecting the building LANs. And note that with the links between buildings, the cables run outside, are often more expensive to install, and are almost always fiber cabling with more expensive switch ports, so conserving the number of cables used between buildings can help reduce costs.

Key Topic

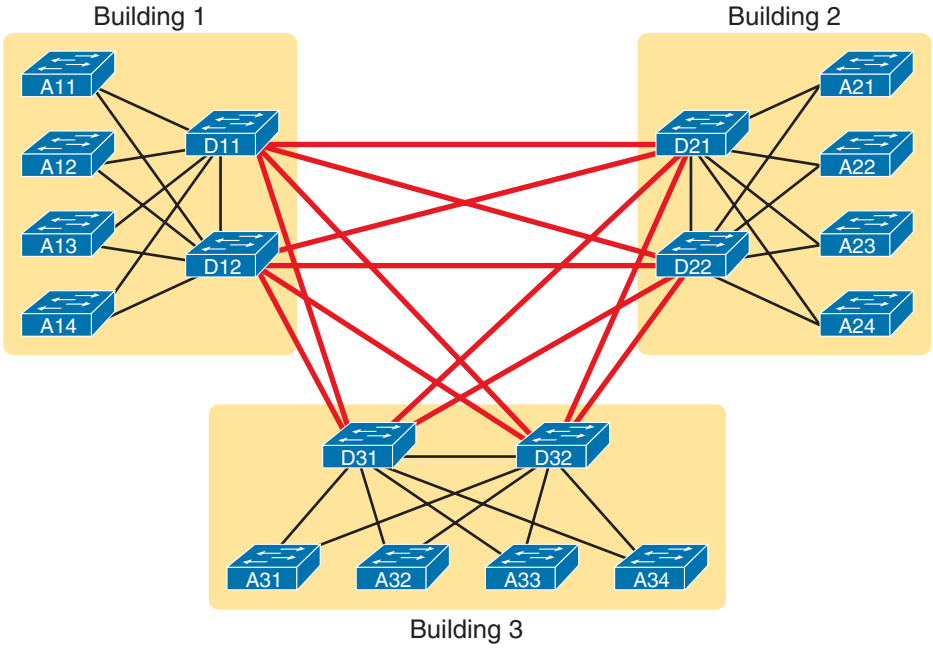



**Figure 18-2** *Two-Tier Building Design, No Core, Three Buildings*

A three-tier **core design**, unsurprisingly at this point, adds a few more switches (core switches), which provide one primary function: to connect the distribution switches. Figure 18-3 shows the migration of Figure 18-2's collapsed core (a design without a core) to a three-tier core design. Note that topologically, the core switches reside in the middle of the network as shown; however, you typically install them in the wiring closet of one of the buildings, along with the distribution switches.

Using a core design, with a partial mesh of links in the core, you still provide connectivity to all parts of the LAN and to the routers that send packets over the WAN, with fewer links between buildings. But the emphasis turns to speed, low delay, and high availability—which oddly enough also means avoiding doing too much in the core. Summarizing the key roles for the core switches:

Key Topic

- Connectivity between distribution switches while conserving links and ports
- Fast transport
- High availability (redundant hardware and design)
- Low latency
- Avoidance of all services that might slow message delivery

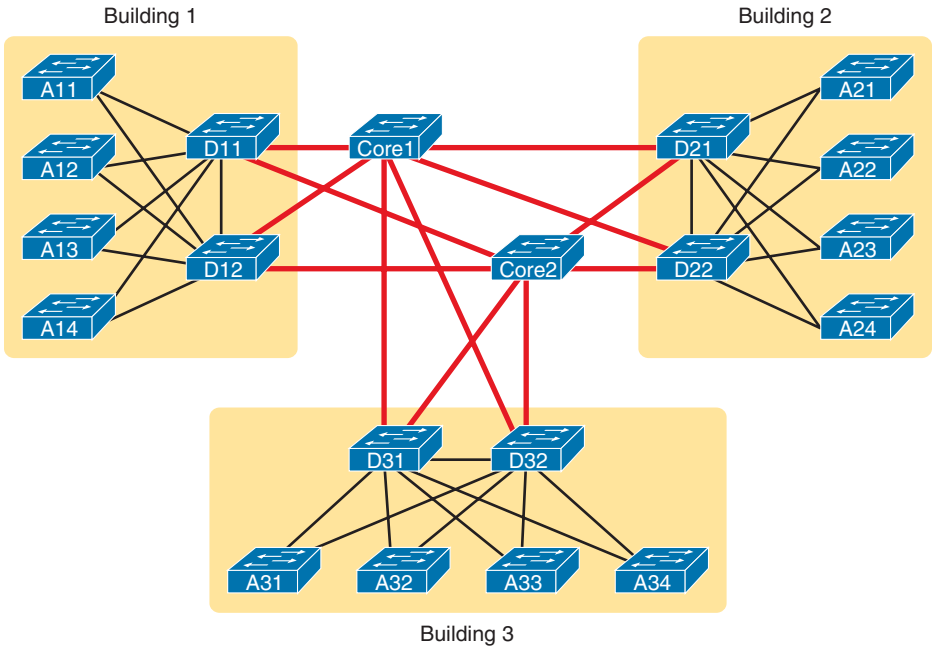



**Figure 18-3** *Three-Tier Building Design (Core Design), Three Buildings*

The following list summarizes the terms that describe the roles of campus switches:

- **Access:** Provides a connection point (access) for end-user devices and connections to the distribution layer. They also implement services focused on endpoint devices, like Power over Ethernet (PoE), switch port security, and DHCP Snooping.
- **Distribution:** Provides an aggregation point for access switches, providing connectivity to the rest of the devices in the LAN, forwarding frames between switches, but not connecting directly to end-user devices. These switches implement most of the services for the campus LAN.
- **Core:** Aggregates distribution switches in large campus LANs, providing high forwarding rates for the larger traffic volume due to the network's size, while purposefully implementing few services.

## Topology Design Terminology

The next topic applies some generic topology terms to a typical two-tier design to close the discussion of enterprise LAN topology.

Access switch drawings often show a series of parallel vertical cables, shown on the far left of Figure 18-4. However, an access switch and its access links are often called a **star topology**. Why? Look at the redrawn access switch in the center of the figure, with the cables radiating out from the center. It looks a little like a child's drawing of a star, hence the term *star topology*.

Key Topic

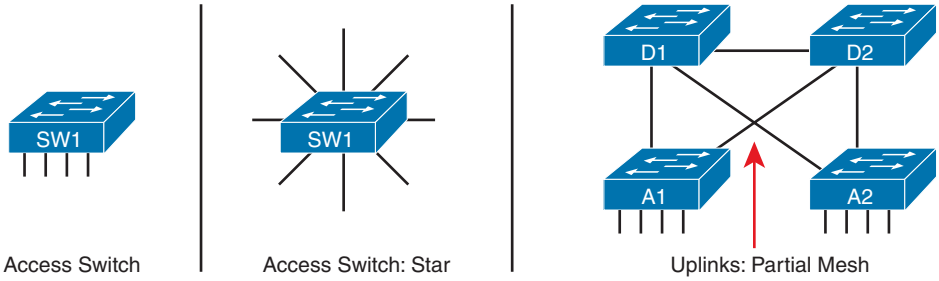



**Figure 18-4** *LAN Design Terminology*

The right side of the figure repeats a typical two-tier design, focusing on the mesh of distribution links. Any group of nodes that connect with more links than a star topology is typically called a *mesh*. In this case, the mesh is a *partial mesh* because not all nodes have a direct link between each other. A design that connects all nodes with a link would be a *full mesh*.

Why not use a full mesh between the access and distribution switches? A full mesh would connect the access switches to each other, create more links than needed, and use too many switch ports. Traffic seldom flows between endpoint devices on access switches. Instead, traffic flows between an endpoint and a server outside the campus LAN, so traffic rarely needs to flow between access switches when all links work. Most traffic flows endpoint-to-server: from the endpoint, to an access switch, to a distribution switch, and out to the WAN.

Regarding the number of links, imagine a two-tier design with 40 access switches and two distribution switches. If the design calls for two uplinks to each distribution switch from each access switch, you need 160 uplinks. (Four per access switch, with 40 switches.) Alternately, a full mesh between the distribution and access switches (42 switches), with just one link between each pair, requires 861 distribution links. (The math uses the formula $N(N - 1) / 2$, or in this case, $42 * 41 / 2 = 861$ links.)

Real networks use these topology ideas, but often a network combines the ideas. For instance, the right side of Figure 18-4 combines the access layer's star topology with the distribution layer's partial mesh. So you might hear these designs that combine concepts called a *hybrid topology*.

18

## Ethernet Physical Media and Standards

In the two-tier and three-tier designs discussed in this chapter, the individual links use the Ethernet data-link protocol, supporting Ethernet frames—but often use different physical layer standards. Those standards define the maximum link segment length and transmission speeds. This next section takes a closer look at some of the considerations and decisions when choosing the cabling to use in a campus LAN, first focusing on access links, which often use UTP cabling, followed by distribution and core links, which use UTP and fiber-optic links.

### Ethernet UTP Links at the Access Layer

Most, if not all, access links use unshielded twisted-pair (UTP) cabling—and that is no accident. From the early days of Ethernet, the IEEE set about to ensure Ethernet would be

commercially viable, making it highly useful and affordable, with UTP cabling as the least expensive option. For instance:

- Knowing that UTP cabling costs less than the other options, the IEEE emphasized standards that supported UTP for the most common links in a typical network: access links.
- Early studies showed that a 100-meter cable could reach from the wiring closet to any point on the floor for most office buildings.
- Therefore, the IEEE used a 100-meter maximum length convention in their successive UTP-based standards over the years.

Most building construction plans include a structured cabling system. On each building floor, the cables run from a wiring panel in a central wiring closet to most locations around the floor. For instance, on a floor used for office cubicles, the structured cabling system includes UTP cables from the wiring closet to every cubicle location, often terminating with an RJ-45 connector in a wall plate.

Not all UTP cables have the same physical transmission characteristics, so the cabling industry and standards bodies have long-defined standards for the UTP cables used by Ethernet. Ethernet standards refer to UTP cable rating categories defined by the Telecommunications Industry Association and the US American National Standards Institute (TIA/ANSI). These categories define the physical transmission qualities when using the cable.

While the TIA and ANSI do not define Ethernet, and the IEEE does not define cabling standards, the Ethernet UTP-based standards refer to the minimum quality of **UTP cable category** that supports each Ethernet standard. For the TIA/ANSI categories, the higher the number and letter, the higher the quality, the more recent the standard release, and the more recent the IEEE standard that uses the cabling. The categories include CAT 3, CAT 5, CAT 5E, CAT 6, CAT 6A, and CAT 8.

Table 18-2 lists the data about the cable categories and matching Ethernet standards. Note that the higher the cable category number/letter, the better for supporting faster Ethernet standards. Also, the table does not list every combination; however, the table does list standards that support a 100-meter cable length.

Key Topic

**Table 18-2** Ethernet and Cable Standards to Support a 100-Meter Segment

| Standard (Common) | Standard (Original Document) | Year of Standard | (Minimum) ANSI/TIA Category | Max Speed |
|---|---|---|---|---|
| **10BASE-T** | 802.3 | 1990 | CAT 3 | 10 Mbps |
| **100BASE-T** | 802.3u | 1995 | CAT 5 | 100 Mbps |
| **1000BASE-T** | 802.3ab | 1999 | CAT 5E | 1 Gbps |
| **10GBASE-T** | 802.3an | 2006 | CAT 6A* | 10 Gbps |
| **40GBASE-T** | 802.3ba | 2010 | CAT 8 | 40 Gbps |
| **2.5GBASE-T** | 802.3bz | 2016 | CAT 5E | 2.5 Gbps |
| **5GBASE-T** | 802.3bz | 2016 | CAT 5E | 5 Gbps |

*10GBASE-T can also use a CAT 6 cable, but with a distance limit of 55 meters.

The table lists the minimum cable category, with better cable categories also working. For instance, the original 10BASE-T worked on then-current CAT 3 cabling (or better). 100BASE-T, the next IEEE UTP Ethernet standard, required CAT 5 (or better). The following Ethernet UTP standard, 1000BASE-T, required even better cabling (CAT 5E) to reach 100 meters.

Today, the IEEE and others work hard to improve Ethernet standards for UTP cabling with that 100-meter access link as a critical design point. As shown by the dates in Table 18-2, about every five years from 1990 to 2010, the IEEE supplied a new UTP-based standard faster than UTP. Those standards deliver plenty of speed for the access layer, assuming you have the required cable types installed.

## Multigig Ethernet on CAT 5E Cabling

After the introduction of 1000BASE-T, over time, the access layer in campus LANs reached a comfortable place in the market. Many buildings completed migrations to use **CAT 5E** cabling. Prices quickly became reasonable for 10/100/1000 switch ports and NICs. And a 1-Gbps speed for each access link connected to a desktop or laptop PC provided more than enough capacity for the PC.

In the 2010s, the IEEE anticipated a potential migration problem for the campus LAN access layer. To move endpoints beyond the 1-Gbps speed required 10GBASE-T, which would not run on CAT 5E cabling, at least per the IEEE standard. Per its standard, 10GBASE-T requires

- CAT 6 cabling to reach 55 meters
- **CAT 6A** cabling to reach 100 meters

The IEEE saw that to move from 1000BASE-T to 10GBASE-T at the access layer, with links up to 100 meters, would require a migration to CAT 6E cabling or better, making those migrations much more expensive. So, they set about creating an alternative Ethernet standard that ran faster than 1 Gbps but also worked on the large installed base of CAT 5E cabling.

The solution, **multigig Ethernet**, comes from the 2016 IEEE 802.3bz addendum. It defines two new UTP-based physical transmission standards: 2.5GBASE-T and 5GBASE-T. Both work on CAT 5E cabling (like 1000BASE-T) but run faster at 2.5 Gbps and 5 Gbps. Figure 18-5 shows the basic migration concept and cable categories.

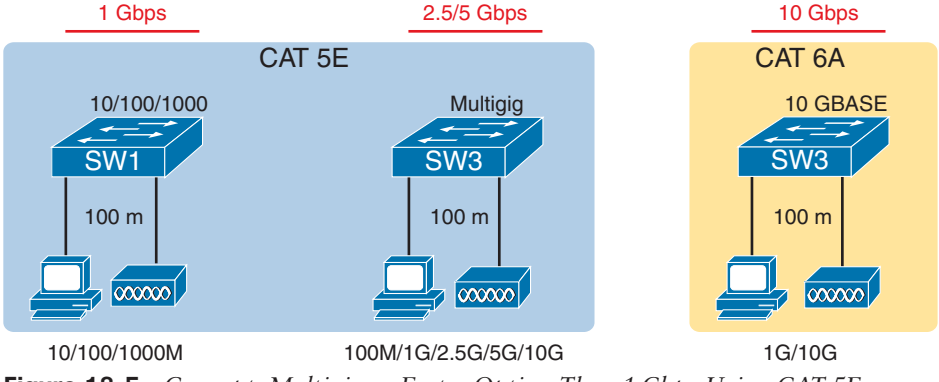



**Figure 18-5** *Concept: Multigig as Faster Option Than 1 Gbps Using CAT 5E*

More than PCs, wireless access points (APs) have driven the multigig Ethernet market. IEEE 802.11ac (WI-FI 5) and 802.11ax (WI-FI 6) APs support more than 1 Gbps of wireless traffic, so they need an Ethernet interface faster than 1 Gbps. Using an AP with a multigig NIC connected to a multigig switch port, the AP can support 2.5- or 5-Gbps speeds so that the Ethernet link does not become a bottleneck.

To use multigig Ethernet, you need multigig host NICs and switch ports. Multigig NICs and switch ports use IEEE autonegotiation to choose the best speed. So, instead of a 10/100/1000 port, multigig ports typically support 100 Mbps, 1 Gbps, 2.5 Gbps, and 5 Gbps, all using CAT 5E cabling. They can also negotiate to use 10 Gbps if connected to upgraded CAT 6/6E cabling.

## Fiber Uplinks

For all uplinks, you need to choose the required speed and the physical layer Ethernet standard. The installed cabling influences the choices because using installed cabling, rather than installing new cables, can significantly reduce the cost.

If possible, use UTP cables for all uplinks. For instance, Table 18-2 lists 10GBASE-T as needing CAT 6A UTP cabling to handle distances to 100 meters. If your structured cabling system had plenty of CAT 6A already installed, using 10GBASE-T uplinks makes great sense.

However, several factors will drive a decision to use fiber Ethernet options. Why fiber? The original cabling design may have needed cable segments longer than 100 meters, so they anticipated your choice to use Ethernet standards that use fiber-optic cabling. With preinstalled fiber cabling, you can then concentrate on what Ethernet standards will work on that cabling, choosing the best switch hardware (for instance, SFPs to add to the switch), and so on.

Multimode fiber cabling has quality standards akin to UTP cable categories. Those standards, called **Optical Multimode (OM)**, include OM1, OM2, OM3, and OM4, with the higher numbers representing newer, more-capable cabling standards. The cable standards define some attributes of the cabling—for instance, the diameter of the core and cladding. (In *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, see Chapter 2's section titled "Fiber Cabling Transmission Concepts" for some figures that show a fiber cable core and cladding.) Table 18-3 lists those OM standards and some supported Ethernet standards.

Key Topic

**Table 18-3** Optical Multimode (OM) and Related Ethernet Standards

| (Minimum) ISO Cable Category | Core/Cladding Diameter | 1000BASE-SX Max Distance per Standard | 10GBASE-SR Max Distance per Standard |
|---|---|---|---|
| OM1 | 62.5/125 | 220 m | 33 m |
| OM2 | 50/125 | 550 m | 82 m |
| OM3 | 50/125 | N/A | 300 m |
| OM4 | 50/125 | N/A | 400 m |

When attempting to use an existing installed base of multimode fiber, determine the OM category required by the Ethernet standard to meet the desired speeds. You can then analyze the data in the table, or similar data about other Ethernet standards, to determine the maximum cable lengths supported by each Ethernet standard. (Note that vendors often suggest that their optical transceivers—SFP, SFP+, and so on—can support distances longer than the

standards.) As you can see in Table 18-3, the **1000BASE-SX** supports distances longer than 1000BASE-T's 100 meters, which may be helpful. The 10GBASE-SR standard supports similar distances but at 10 Gbps.

Also, use the numbers in the table for initial planning, but be aware of a couple of essential points:

- The standards typically give a conservative distance estimate, but you may be able to make links work at longer distances.
- In practice, 1000BASE-SX works on OM3 and OM4. The table lists N/A (not applicable) because the 1000BASE-SX standard pre-dates the OM3 and OM4 standards. But formally, the 1000BASE-SX part of the Ethernet standard does not mention OM3 and OM4.

After identifying the specific cables, you can plan for the Ethernet standard and switch ports. For instance, you might make a standard of buying access and distribution switches with modular ports that support 1 or 10 Gbps. You could then buy a 1000BASE-SX SFP or **10GBASE-SR** SFP+ to match the standard that meets each case's needs.

## Small Office/Home Office

Now that you know more about design choices and terms for an enterprise LAN, this next section examines one type of smaller LAN: the small office/home office (**SOHO**) LAN. SOHO refers to designs and implementations with such a small volume of requirements—few switch ports, APs, routers, and WAN links—that the design differs significantly. The term itself refers to the two most common cases: a user who works from home or a small office with a small number of workers and devices. This next short topic points out a few highlights that differentiate a SOHO network from an enterprise network.



First, as a reminder, the IEEE defines Ethernet and wireless LANs (WLANs). All IEEE 802.3 Ethernet standards use cables—that is, Ethernet defines wired LANs. The IEEE 802.11 working group defines wireless LANs, also called *Wi-Fi* per a trademarked term from the Wi-Fi Alliance (www.wi-fi.org).

Most of you have used Wi-Fi and may use it daily. Some of you may have set up Wi-Fi at home, as shown in Figure 18-6. You probably used a single consumer device called a *wireless router* in a home. One side of the device connects to the Internet, while the other connects to the home's devices. In the home, the devices can connect with Wi-Fi or with a wired Ethernet cable.

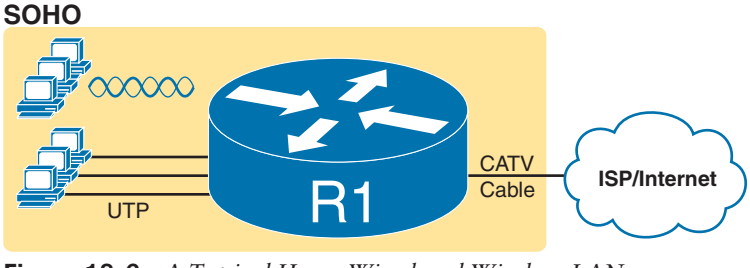



**Figure 18-6** *A Typical Home Wired and Wireless LAN*

While the figure shows the hardware as a single router icon, internally, that one wireless router acts like separate devices you would find in an enterprise campus:

Key Topic

- An Ethernet switch, for the wired Ethernet connections
- A wireless access point (AP), to communicate with the wireless devices and forward the frames to/from the wired network
- A router, to route IP packets to/from the LAN and WAN (Internet) interfaces
- A firewall, which often defaults to allow only clients to connect to servers in the Internet, but not vice versa

Figure 18-7 repeats the previous figure, breaking out the internal components as if they were separate physical devices, just to make the point that a single consumer wireless router acts like several devices.

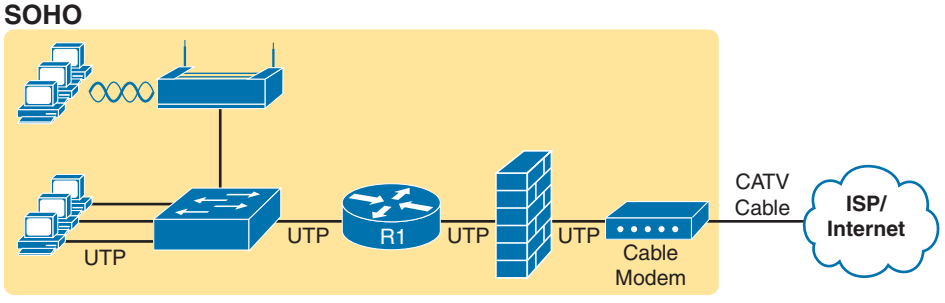



**Figure 18-7** *A Representation of the Functions Inside a Consumer Wireless Routing Product*

In a SOHO wireless LAN, the wireless AP acts autonomously, rather than with a WLC, doing all the work required to create and control the WLAN. In other words, the autonomous AP communicates with various wireless devices using 802.11 protocols and radio waves. It uses Ethernet protocols on the wired side. It converts the differences in header formats between 802.11 and 802.3 frames before forwarding to/from 802.3 Ethernet and 802.11 wireless frames. But it does not encapsulate frames in CAPWAP because the AP will not send them to a WLC.

For the Internet connection, the router (combo) device connects with any available Internet access technology, including cable Internet, DSL, 4G/5G wireless, or fiber Ethernet. Note that Chapter 19, "WAN Architecture," introduces those technologies.

## Power over Ethernet (PoE)

Just walk around any building and you see electrical power outlets everywhere. When finishing the interior of a building, electricians run electrical cables and install electrical outlets to any and every location that might need power. They also run power cables directly to devices like light fixtures. And when network engineers thought about electrical power, they thought about making sure the electricians had run enough power to the wiring closets and other locations to power the networking devices.

Power over Ethernet (**PoE**) changes that thinking so that the responsibility to provide electrical power to some devices can fall to the network engineering team. Some modern devices can pull power from an Ethernet cable rather than a separate power cord. To make that work, the LAN switch connected to the cable must supply that power over the cable. Using PoE, companies can gain several advantages, including reduced cost by requiring fewer cable runs and better power management capabilities than traditional electrical power cable runs and power outlets. This final section of the chapter examines PoE.

## PoE Basics

The family of standards that supply power goes by the general name *Power over Ethernet* (PoE). With PoE, some device, typically a LAN switch, acts as the **power sourcing equipment (PSE)**—that is, the device that supplies DC power over the Ethernet UTP cable (as shown in Figure 18-8). A device that can be powered over the Ethernet cable rather than by some other power connector on the device is called the **powered device (PD)**.

Key Topic

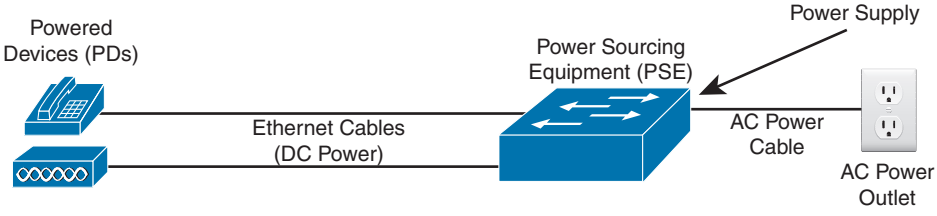



**Figure 18-8** *Power over Ethernet Terminology*

PoE has an excellent advantage for devices installed in locations that often do not have a preinstalled power cable or power output. For instance, wireless networks locate APs across a wide range of places in the ceiling of most floors. Also, IP video cameras might be placed in the ceiling corners inside or at various outside locations. Instead of running both power and network cables to each device, a single Ethernet cable run can supply power to the device while allowing routine Ethernet communications over the same cable and wire pairs.

PoE also helps in some less obvious practical ways because it supplies DC power over the Ethernet cable, so the device does not need an AC/DC converter. For instance, devices like laptops and IP phones use a power cord that includes a power brick—an AC-to-DC converter—which converts the AC power from the power outlet to the DC power needed by the device. PoE supplies DC current over the Ethernet cable. So, for an IP phone, for instance, no more power cable and no more power brick cluttering the desk or taking up a power outlet.

## PoE Operation

PoE must have the means to avoid harming the devices on the end of the circuit. Every electrical device can be damaged by receiving too much current into the device, which is why electricians install circuit breakers and why we use surge protectors. Applying power over an Ethernet cable could have the same effect, harming the device on the other end if the device does not support PoE.

PoE first performs **power detection** and then **power classification**. Power detection defines how the PSE can detect whether the attached device is a PD. To do so, the PSE sends a low-power signal. PDs complete the circuit, while non-PDs do not. Also, PDs add a defined

resistance to the circuit, which allows the PSE not only to detect that the PD completed the circuit, but that it also added the right amount of resistance—confirming to the PSE that the attached device is a PD.

PoE power classification determines how much power the PSE should apply to the circuit. The process identifies a specific power class defined by PoE, with each power class defining electrical characteristics like voltage and current.

Layer 1 power classification uses the electrical signals on the cable, choosing among a small set of low-power classes with less chance of harming a device. But Layer 1 power classification has the great advantage of assigning a power class and supplying enough power to boot a simple OS that can participate in Layer 2 power classification.

Layer 2 power classification follows Layer 1 power classification. Layer 2 power classification uses LLDP and CDP messages, defined as Layer 2 protocols because they do not include IP headers, to determine the power class. The PD sends the message to the PSE, telling the PSE the desired class. If the PSE has enough power, it supplies it; if not, it stops supplying power. (See Chapter 13, "Device Management Protocols," for a broad discussion of LLDP and CDP.)

You can also configure PoE settings on the switch. By default, the PoE ports use automatic power detection and classification as discussed in this section. Alternatively, you can disable PoE on a port, or set a specific power class, reserving power on that port, and bypassing the need for power classification.

The standards allow for various paths through the power detection and classification processes. The following list gives a typical example (note that the list does not attempt to describe every process combination).




**Step 1.** Do not supply power on a PoE-capable port unless power detection identifies that the device needs power.

**Step 2.** The PSE performs Layer 1 power detection of the presence of a PD by sending low-power signals and monitoring the return signal to determine if the device is a PD.

**Step 3.** If identified as a PD, determine an initial power level based on Layer 1 power classification and supply that power level, giving the PD enough power to perform some software initialization steps.

**Step 4.** Perform Layer 2 power classification using LLDP or CDP, with the PD announcing a specific power class. If choosing a new power class, adjust the power level.

The processes result in the PDs signaling how many watts of power they would like to receive from the PSE. Depending on the specific PoE standard, the PSE will supply the power over two or four pairs, as noted in Table 18-4.

**Table 18-4** Power over Ethernet Standards

| Name | Standard | Watts at PSE | Power Class | Powered Wire Pairs |
|---|---|---|---|---|
| PoE | 802.3af | 15 | 0 | 2 |
| PoE+ | 802.3at | 30 | 4 | 2 |
| UPoE | 802.3bt | 60 | 6 | 4 |
| UpoE+ | 802.3bt | 90 | 8 | 4 |

As an aside, the table shows PoE standards, but Cisco created an earlier pre-standard PoE feature called Cisco Inline Power (ILP). However, for the most part, Cisco's literature refers to the more familiar names in the first column of the table. Cisco long ago migrated to use the IEEE PoE standards in their product line rather than their proprietary Cisco inline power.

## PoE and LAN Design

Most of the LAN switch features discussed in this book (and in *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition) exist as software features. Once you learn about a software feature, to use it, you just need to configure it. (Sometimes, you might need to research and add software licenses first.) Regardless, adding software-based features takes little or no prior planning.

PoE requires more planning and engineering effort, both when planning for the cable plant (both Ethernet and electrical) and when choosing new networking hardware. The following list includes some of the key points to consider when planning a LAN design that includes PoE:

- **Powered devices:** Determine the types of devices, specific models, and their power requirements.
- **Power requirements:** Plan the number of different types of PDs to connect to each wiring closet to build a power budget. Then analyze the data to determine the amount of PoE power to make available through each switch.
- **Switch ports:** Some switches support PoE standards on all ports, some on no ports, and some on a subset of ports. Some switches support PoE+ but not **UPoE** or UPoE+. Research the various switch models so that you purchase enough PoE-capable ports with the needed standards for the switches planned for each wiring closet.
- **Switch power supplies:** A PoE switch acts as a distributor of electrical power, so the switch power supply must deliver many more watts than needed for the switch itself. You will need to create a power budget per switch, based on the number of connected PDs and each device's power needs, and purchase power supplies to match those requirements.
- **PoE standards versus actual:** Consider the number of PoE switch ports needed, the standards they support, the standards supported by the PDs, and how much power they consume. For instance, a PD and a switch port may both support PoE+, which supports up to 30 watts supplied by the PSE. However, that powered device may need at most 9 watts to operate, so your power budget needs to reserve less power than the maximum for those devices.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 18-5 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 18-5** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, app |
| Review key terms | | Book, app |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, app |

## Review All the Key Topics

Key Topic

**Table 18-6** Key Topics for Chapter 18

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 18-1 | Campus LAN design terms | 399 |
| List | Access switch functions | 399 |
| List | Distribution switch functions | 400 |
| Figure 18-2 | A two-tier (collapsed core) LAN topology | 401 |
| Figure 18-3 | A three-tier (core) LAN topology | 401 |
| List | Core switch functions | 402 |
| List | Switch roles in campus LAN design | 402 |
| Figure 18-4 | Network topology terms and campus LANs | 403 |
| Table 18-2 | UTP cable categories | 404 |
| Table 18-3 | Optical Multimode (OM) cable categories | 406 |
| List | Components in an integrated SOHO network device | 408 |
| Figure 18-8 | PoE roles and terms | 409 |
| List | Typical steps to discover power requirements with PoE | 410 |

## Key Terms You Should Know

10BASE-T, 100BASE-T, 1000BASE-T, 10GBASE-SR, 1000BASE-SX, 2.5GBASE-T, 5GBASE-T, 10GBASE-T, 40GBASE-T, access layer, access link, CAT 5E, CAT 6A, collapsed core design, core design, core layer, distribution layer, distribution link, full mesh, multigig Ethernet, Optical Multimode, partial mesh, PoE, power classification, power detection, power sourcing equipment (PSE), powered device (PD), SOHO, star topology, UPoE, UTP cable category

*This page intentionally left blank*



CHAPTER 19

# WAN Architecture

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.2 Describe the characteristics of network topology architectures**
    - **1.2.d WAN**
- **5.0 Security Fundamentals**
  - **5.5 Describe IPsec remote access and site-to-site VPNs**

The *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition introduces just enough detail about two types of WAN links—point-to-point serial and point-to-point Ethernet WAN links—so that you could understand IP routing, which is a major focus in CCNA. This chapter now turns our attention to WAN topics, from the perspective of the enterprise, as the customer of some WAN **service provider (SP)**. (Cisco's Service Provider certification track explores how an SP implements its network.)

This chapter begins with a discussion of Metro Ethernet, in the first major section, a technology that defines how to use Ethernet links between a customer site and the SP. The second section then examines MPLS VPNs, even though MPLS VPNs came before Metro Ethernet historically. The chapter introduces Metro Ethernet first because the many similarities between using Ethernet in the LAN and WAN make this topic easier to learn.

The chapter closes with a third section about how to use the Internet as a private WAN service by using virtual private network (VPN) technology. The Internet does not inherently provide a private service, meaning that any attacker who gets a copy of your packets as they pass through the Internet can read the contents. VPNs secure the data sent over the Internet, effectively creating a private WAN service over the public Internet.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 19-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Metro Ethernet | 1–3 |
| Multiprotocol Label Switching (MPLS) | 4, 5 |
| Internet VPNs | 6 |

1. Which of the following topology terms most closely describe the topology created by a Metro Ethernet Tree (E-Tree) service? (Choose two answers.)
   a. Full mesh
   b. Partial mesh
   c. Hub and spoke
   d. Point-to-point
2. Which of the following is the most likely technology used for an access link to a Metro Ethernet service?
   a. 100BASE-LX10
   b. High-speed TDM (for example, T3, E3)
   c. MPLS
   d. 100BASE-T
3. An enterprise uses a Metro Ethernet WAN with an Ethernet LAN (E-LAN) service, connecting the company headquarters to ten remote sites. The enterprise uses OSPF at all sites, with one router connected to the service from each site. Which of the following are true about the Layer 3 details most likely used with this service and design? (Choose two answers.)
   a. The WAN uses one IP subnet.
   b. The WAN uses ten or more IP subnets.
   c. A remote site router would have one OSPF neighbor.
   d. A remote site router would have ten OSPF neighbors.
4. An enterprise uses an MPLS Layer 3 VPN, connecting the company headquarters to ten remote sites. The enterprise uses OSPF at all sites, with one router connected to the service from each site. Which of the following are true about the Layer 3 details most likely used with the routers connected using MPLS? (Choose two answers.)
   a. The WAN uses one IP subnet.
   b. The WAN uses ten or more IP subnets.
   c. A remote site router would have one OSPF neighbor.
   d. A remote site router would have ten or more OSPF neighbors.
5. Which of the following answers about access link options for an MPLS network is most accurate?
   a. Uses only TDM (T1, T3, E1, E3, etc.)
   b. Uses only Ethernet
   c. Uses only DSL and cable
   d. Uses a wide variety of Layer 1 and Layer 2 networking technologies

6. A colleague mentions a remote access VPN using a VPN client on their laptop. Which protocols might the VPN concentrator choose to require of the VPN client? (Choose two answers.)
   a. TLS
   b. IPsec
   c. GRE
   d. FTPS

# Foundation Topics

## Metro Ethernet

**Metro Ethernet** (MetroE) includes various WAN services with some common features. Each MetroE service uses Ethernet physical links to connect the customer's device to the service provider's device. Second, it delivers a Layer 2 service in that the WAN provider forwards Ethernet frames from one customer device to another.

Conceptually, Metro Ethernet acts as if the WAN service were created by one Ethernet switch, as shown in Figure 19-1. The figure shows four sites in the same company, each with a router. Each router connects to the WAN service with an Ethernet link; those Ethernet links typically use one of the fiber Ethernet standards due to the distances involved. From the customer's perspective (that is, from the perspective of the enterprise that is the customer of the WAN SP), the WAN service acts like a LAN switch in that it forwards Ethernet frames.

**NOTE** Throughout this chapter, the word *customer* refers to the customer of the service provider—the enterprise purchasing the WAN service.

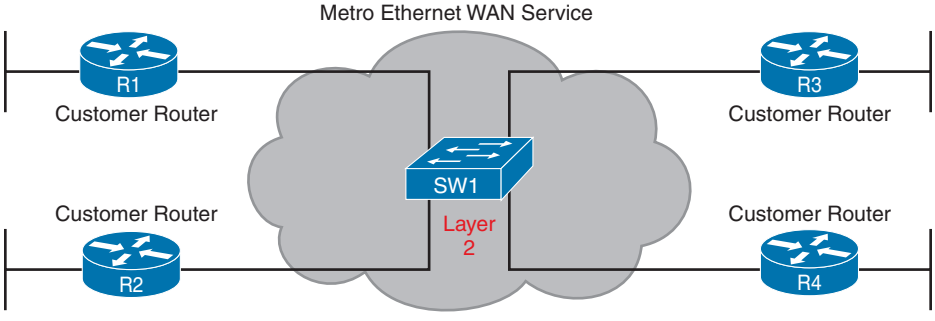



**Figure 19-1** *Metro Ethernet Concept as a Large Ethernet Switch*

This first major section of the chapter examines the physical connections in a Metro Ethernet service. Also, enterprises typically connect to MetroE services with a router or Layer 3 switch, so this section also describes IP subnetting and routing protocols when using MetroE.

### Metro Ethernet Physical Design and Topology

From an enterprise perspective, to use a Metro Ethernet service, each site needs to connect to the service with (at least) one Ethernet link. However, the enterprise's routers need not

connect directly to each other. For instance, in Figure 19-1, each of the four enterprise routers uses one physical Ethernet link to connect to the SP's MetroE service—but not to each other.

The SP needs to build a network to create the Metro Ethernet service. To keep costs lower, the SP installs many devices, typically Ethernet switches, physically near as many customer sites as possible in an SP facility called a **point of presence (PoP)**. The goal: Locate a switch within the distance limitations of physical Ethernet fiber-optic standards. Figure 19-2 collects some of these terms and ideas together.

Key Topic

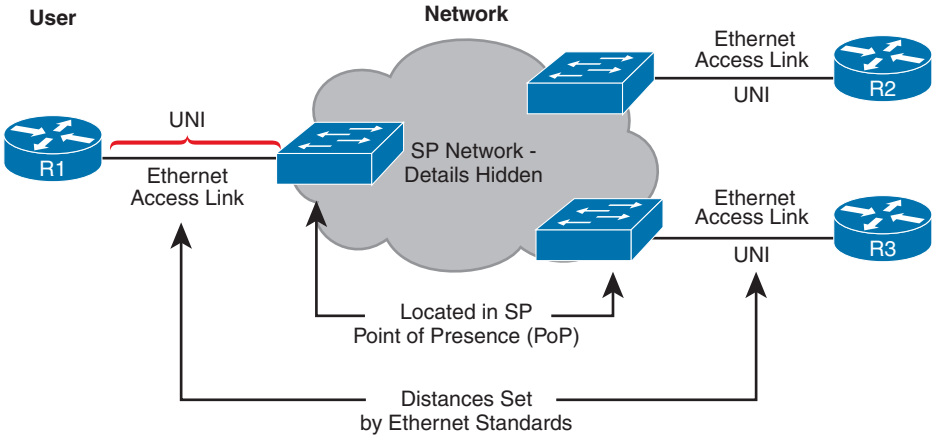



**Figure 19-2** *Ethernet Access Links into a Metro Ethernet Service*

SPs generally refer to any link that connects the customer and SP as an **access link**, so when using Metro Ethernet, the link is an *Ethernet access link*. Everything on that link falls within the definition of the *user network interface*, or UNI. Breaking down the term *UNI*, the word *network* refers to the SP's network, while the SP's customer (the enterprise) is known as the *user* of the network.

The SP's network remains hidden from the customer, as depicted in the center of Figure 19-2. The SP promises to deliver Ethernet frames across the WAN. As you can imagine, the switch will look at the Ethernet header's MAC address fields and 802.1Q trunking headers for VLAN tags, but the details inside the network remain hidden.

The UNI supports any IEEE Ethernet standard. Table 19-2 lists some standards you might expect to see used as Ethernet access links, given their support of longer distances than the standards that use UTP cabling.

**Table 19-2** IEEE Ethernet Standards Useful for Metro Ethernet Access

| Name | Speed | Distance |
|---|---|---|
| 100BASE-LX10 | 100 Mbps | 10 Km |
| 1000BASE-LX | 1 Gbps | 5 Km |
| 1000BASE-LX10 | 1 Gbps | 10 Km |
| 1000BASE-ZX | 1 Gbps | 100 Km |

| Name | Speed | Distance |
|---|---|---|
| 10GBASE-LR | 10 Gbps | 10 Km |
| 10GBASE-ER | 10 Gbps | 40 Km |

## Ethernet WAN Services and Topologies

Beyond adding a physical Ethernet connection from each site into the SP's Metro **Ethernet WAN** service, the enterprise must choose between several possible variations of MetroE services. Those variations use different topologies that meet different customer needs.

MEF (www.mef.net) defines the standards for Metro Ethernet, including the specifications for different MetroE services. Table 19-3 lists three service types described in this chapter and their topologies.

Key Topic

**Table 19-3** Three MEF Service Types and Their Topologies

| MEF Service Name | MEF Short Name | Topology Terms | Description |
|---|---|---|---|
| Ethernet Line Service | E-Line | **Point-to-point** | Two customer premise equipment (CPE) devices can exchange Ethernet frames, similar in concept to a leased line. |
| Ethernet LAN Service | E-LAN | **Full mesh** | This service acts like a LAN so that all devices can send frames to all other devices. |
| Ethernet Tree Service | E-Tree | **Hub and spoke; partial mesh**; point-to-multipoint | A central site can communicate with a defined set of remote sites, but the remote sites cannot communicate directly. |

**NOTE** You might see the term *Virtual Private Wire Service* (VPWS) used for what MEF defines as E-Line service and *Virtual Private LAN Service* (VPLS) used for what MEF defines as E-LAN service. You might also see the term *Ethernet over MPLS* (EoMPLS). All these terms refer to cases in which the SP uses MPLS internally to create what the customer sees as an Ethernet WAN service.

### Ethernet Line Service (Point-to-Point)

The Ethernet Line Service, or **E-Line**, is the simplest of the Metro Ethernet services. The customer connects two sites with access links. Then the MetroE service allows the two customer devices to send Ethernet frames to each other. Figure 19-3 shows an example, with routers as the CPE devices.

---

Answers to the "Do I Know This Already?" quiz:

**1** B, C **2** A **3** A, D **4** B, C **5** D **6** A, B

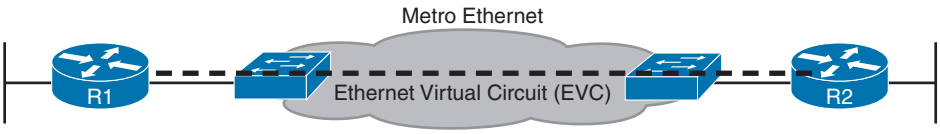



**Figure 19-3** *Point-to-Point Topology in Metro Ethernet E-Line Service Between Routers*

As with all MetroE services, the promise made by the service is to deliver Ethernet frames across the service, as if the two customer routers had a rather long crossover cable connected between them. The E-Line service is the same Ethernet WAN service you have seen in many examples throughout this book and *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition. For instance, in this case:

- The routers would use physical Ethernet interfaces.
- The routers would configure IP addresses in the same subnet as each other.
- Their routing protocols would become neighbors and exchange routes.

The MetroE specifications define the concept of an *Ethernet Virtual Connection*, or EVC, which defines which user (customer) devices can communicate with which. By definition, an E-Line service creates a point-to-point EVC, meaning that the service allows two endpoints to communicate.

An enterprise might want to implement a network exactly as shown in Figure 19-3, but other variations exist. For example, think of a typical enterprise WAN topology with a central site and 100 remote sites. As shown so far, with an E-Line service, the central site router would need 100 physical Ethernet interfaces to connect to those 100 remote sites. That could be expensive. Instead, the enterprise could use a design with one high-speed Ethernet access link. Figure 19-4 shows the design concept with just three remote sites. In this case:

- The central site router uses a single 10-Gbps access link.
- The central site connects to 100 E-Lines (only three lines shown).
- All the E-Lines send and receive frames over the same access link.



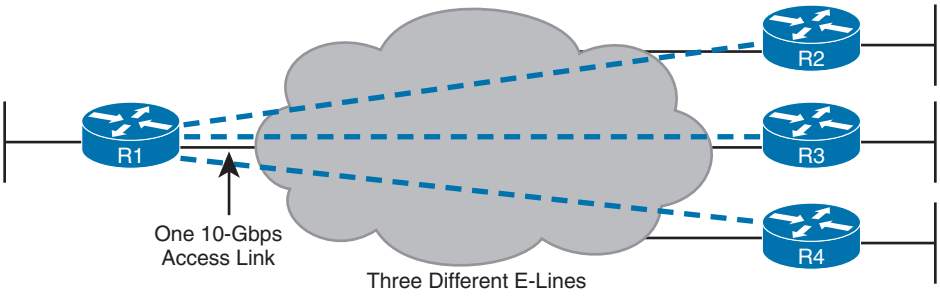



**Figure 19-4** *Using Multiple E-Lines, One for Each Remote Site*

### Ethernet LAN Service (Full Mesh)

Imagine an enterprise needs to connect several sites to a WAN, and the goal is to allow every site to send frames directly to every other site. You could do that with E-Lines, but you

would need many. For instance, to connect six sites, you need 15 E-Lines, but with 20 sites, you need 190 E-Lines. Instead, you need an any-to-any service, more like a LAN.

A MetroE *Ethernet LAN service*, or **E-LAN**, connects all sites in a full mesh, meeting the need for direct connectivity between all WAN sites. One E-LAN service allows all devices connected to that service to send Ethernet frames directly to every other device as if the Ethernet WAN service were one big Ethernet switch. Figure 19-5 shows a representation of a single E-LAN EVC, allowing routers R1, R2, R3, and R4 to send frames directly to each other. All reside in the same Layer 3 subnet on the WAN.

Key Topic

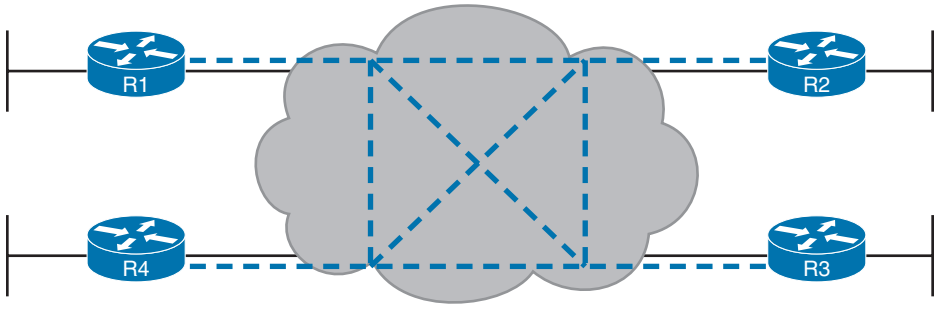



**Figure 19-5** *MetroE Ethernet LAN Service—Any-to-Any Forwarding over the Service*

## Layer 3 Design Using Metro Ethernet

How should the engineer plan for IP addresses and subnets when using Ethernet WAN services? What is the impact on routing protocols? This section answers those questions. Note that this section uses routers as the enterprise's WAN edge devices, but the concepts also apply to Layer 3 switches.

### Layer 3 Design with E-Line Service

Every E-Line creates a point-to-point topology. For Layer 3 protocols, that results in

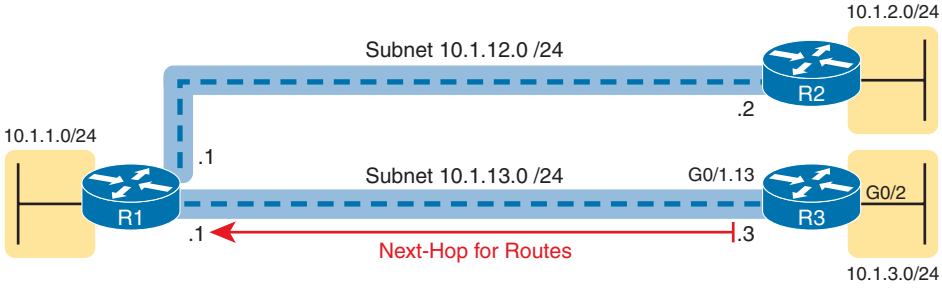
- One subnet per E-Line
- Routing protocol neighbor relationships occur between the two routers on the E-Line, but not between routers without an E-Line

Figure 19-6 shows a design with two E-Lines, with IP subnetting details added. Router pairs R1-R2 and R1-R3 connect with an E-Line, but R2-R3 do not. As a result,

1. The R1-R2 E-Line uses a subnet (10.1.12.0/24 in this case).
2. The R1-R3 E-Line uses a subnet (10.1.13.0/24 in this case).
3. R1 will become a routing protocol neighbor with R2 and R3.
4. R2 and R3 will not become routing protocol neighbors or learn IP routes directly from each other.

The figure also shows router R3's IP routing table. Note that the next-hop address for all OSPF-learned routes refers to router R1's 10.1.13.1 IP address: R1's address on the other end of the R1-R3 E-Line. Look at R3's route for subnet 10.1.2.0/24, the LAN subnet connected to router R2. With no E-Line or routing protocol neighbor relationship between R2 and R3,

R3 learns its route for subnet 10.1.2.0/24 from R1, with R1's 10.1.13.1 address as the next-hop address.



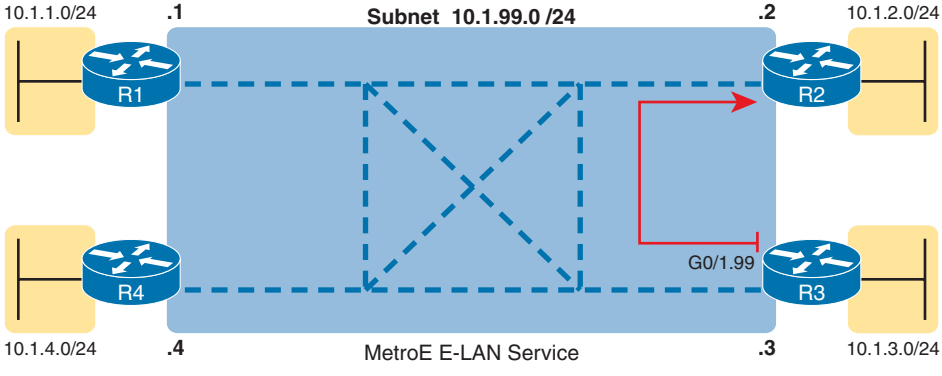

R3 Routing Table

| Code | Subnet | Interface | Next-hop |
|---|---|---|---|
| O | 10.1.1.0/24 | G0/1.13 | 10.1.13.1 |
| O | 10.1.2.0/24 | G0/1.13 | 10.1.13.1 |
| O | 10.1.12.0/24 | G0/1.13 | 10.1.13.1 |
| C | 10.1.3.0/24 | G0/2 | N/A |
| C | 10.1.13.0/24 | G0/1.13 | N/A |

**Figure 19-6** *Layer 3 Forwarding Between Remote Sites—Through Central Site*

## Layer 3 Design with E-LAN Service

Typically, with four routers connected to the same LAN switch, on the same VLAN, using the same routing protocol, normally all four routers would have IP addresses in the same subnet, and all would become neighbors. The same IP addressing and routing protocol relationships occur with an E-LAN service. Figure 19-7 shows an example with four routers connected to an E-LAN service, all with addresses in subnet 10.1.99.0/24.




R3 Routing Table (OSPF Routes Only)

| Subnet | Interface | Next-hop |
|---|---|---|
| **10.1.2.0/24** | G0/1.99 | **10.1.99.2** |
| 10.1.1.0/24 | G0/1.99 | 10.1.99.1 |
| 10.1.4.0/24 | G0/1.99 | 10.1.99.4 |

**Figure 19-7** *Layer 3 Forwarding Between Sites with E-LAN Service*

To show the differences in IP routing when using an E-LAN service, focus on R3's route to R2's LAN subnet (10.1.2.0/24). In this case, R3's next-hop address is the WAN address on R2 (10.1.99.2), and R3 will send packets directly to R2. Note also that the other two routes in the routing table list the next-hop addresses of R1 (10.1.99.1) and R4 (10.1.99.4), showing an example of routers sending packets directly to each other over the WAN.

## Multiprotocol Label Switching (MPLS)

From your CCNA preparation so far, you already understand much about Layer 3 routing, as represented by the packet flowing left to right in Figure 19-8. Each router makes a separate forwarding decision to forward the packet, as shown in steps 1, 2, and 3 in the figure. Each router compares the packet's destination IP address and that router's IP routing table; the matching IP routing table entry tells the router where to forward the packet next. The routers typically run some routing protocol to learn those routes.

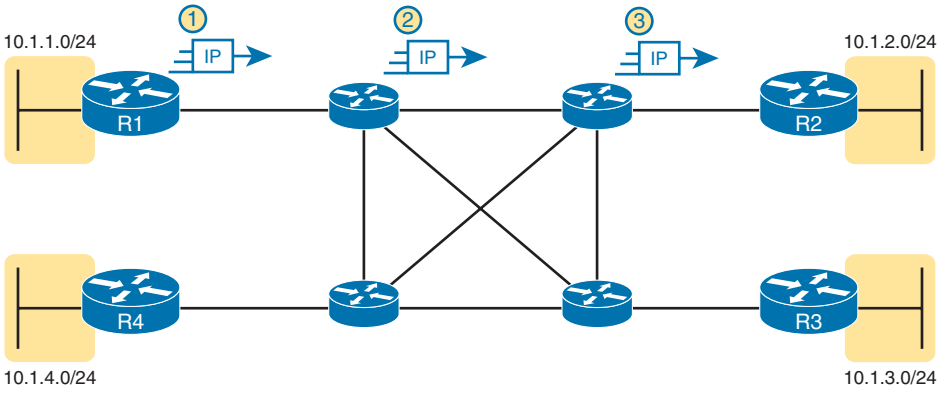



**Figure 19-8** *Basic IP Routing of IP Packets*

MPLS creates a WAN service that routes IP packets between customer sites. The enterprise deploys routers and switches as usual. The SP then creates its IP network, spanning a large geographic area. The customer can then connect to the MPLS network, with an access link from each site, with the SP routing IP packets from one customer site to the other. For instance, in Figure 19-8, the middle four routers could represent the SP's MPLS network, with the numbered routers on the edges owned by one company.

However, an SP cannot just build a large IP network and connect all its customers to that same IP network using only the same IP routing rules you have learned so far. For instance, many customers will use the same private IP network (for example, network 10.0.0.0), so the SP's IP network would learn large numbers of routes to overlapping subnets. The SP's routers would be confused as to where to forward packets.

To overcome this and other issues, the SP builds its IP network to use **Multiprotocol Label Switching (MPLS)**, particularly **MPLS VPNs**. MPLS VPNs allow the SP to make one extensive MPLS network, creating a private IP-based WAN, but it keeps the IP routes separate for each customer. With MPLS VPNs, the SP can separate the routes learned from one customer from those learned for the next customer; consequently, the SP can support each customer while preventing packets from leaking from one customer to the next.

To give you a little insight into why MPLS is not just an IP network with routers, internally, the devices in an MPLS network use label switching—hence, the name MPLS. The routers on the edge of the MPLS network add and remove an MPLS header to packets as they enter and exit the MPLS network. The devices inside the MPLS network then use the label field inside that MPLS header when forwarding data across the MPLS network. The choices of the labels to use, along with other related logic, allow the MPLS VPN to create separate VPNs to keep different customers' traffic separate.

**NOTE** While MPLS VPNs provide a Layer 3 service to customers, MPLS itself is sometimes called a Layer 2.5 protocol because it adds the MPLS header between the data-link header (Layer 2) and the IP header (Layer 3).

As usual, the discussion of WAN services in this book ignores as much of the SP's network as possible. For instance, you do not need to know how MPLS labels work. However, because MPLS VPNs create a Layer 3 service, the customer must be more aware of what the SP does than with other WAN servers, so you need to know a few facts about how an MPLS network approaches some Layer 3 functions. In particular, the SP's MPLS VPN network

Key Topic

- Will build routing protocol neighbor relationships with customer routers
- Will learn customer subnets/routes with those routing protocols
- Will advertise a customer's routes with a routing protocol so that all routers connected to the MPLS VPN can learn all routes as advertised through the MPLS VPN network
- Will make decisions about MPLS VPN forwarding, including what MPLS labels to add and remove, based on the customer's IP address space and customer IP routes

As an aside, MPLS VPNs create a private network by keeping customer data separate but not by encrypting the data. Some VPN services encrypt the data, expecting that attackers might be able to receive copies of the packets. With MPLS, even though the packets for two customers may pass through the same devices and links inside the MPLS network, MPLS logic can keep the packets separate for each customer.

MPLS networks can offer Quality of Service (QoS) features as well. The MPLS provider controls their entire network. As a result, each device that forwards customer traffic can react to markings and apply different QoS features. Note that Metro Ethernet can also provide QoS services, but with different marking fields. However, Internet WANs cannot support QoS, because the Internet consists of many ISPs, and the ISP to which a company connects cannot guarantee the other ISPs will follow any particular QoS commitment.

This second of two major sections of the chapter works through the basics of MPLS, specifically MPLS VPNs. This section first looks at the design, topology, and terminology related to building the customer-facing parts of an MPLS network. It then looks at the MPLS network providing a Layer 3 service.

## MPLS VPN Physical Design and Topology

MPLS provides a Layer 3 service, promising to forward Layer 3 packets (IPv4 and IPv6). To support that service, MPLS SPs typically use routers at the edge of the MPLS networks because routers provide the function of forwarding Layer 3 packets.

Figure 19-9 shows two important MPLS terms in context: **customer edge (CE)** and **provider edge (PE)**. Because MPLS requires so much discussion about the devices on the edge of the customer and SP networks, MPLS uses specific terms for each. The term *customer edge* refers to a router at the customer site—that is, at a site in the company buying the MPLS service. The *provider edge* devices sit at the edge of the SP's network, on the other end of the access link.

Key Topic

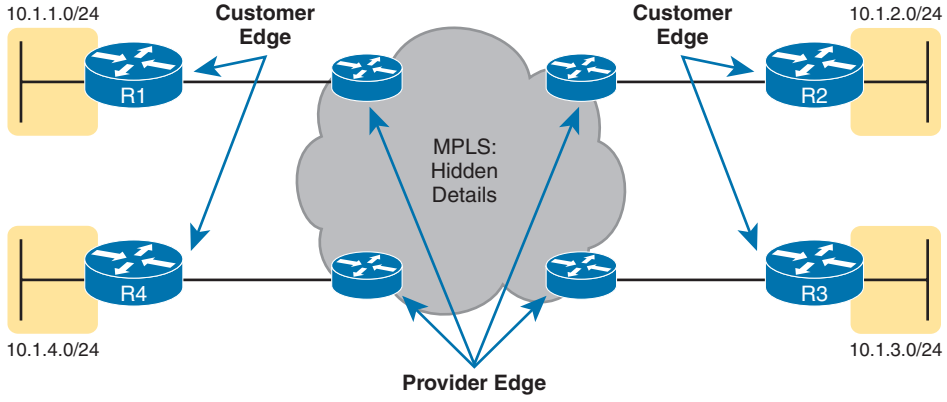



**Figure 19-9** *MPLS Layer 3 Design, with PE and CE Routers*

Next, to appreciate what MPLS does, think back to router de-encapsulation and re-encapsulation. When routing a packet, routers discard an incoming data-link frame's data-link header and trailer. Once the router knows the outgoing interface, it builds a new data-link header/trailer. That action allows the incoming packet to arrive inside a frame of one data-link protocol and leave out an interface with another data-link protocol. As a result, MPLS supports any data-link protocol that could be used on MPLS access links, as shown in Figure 19-10.

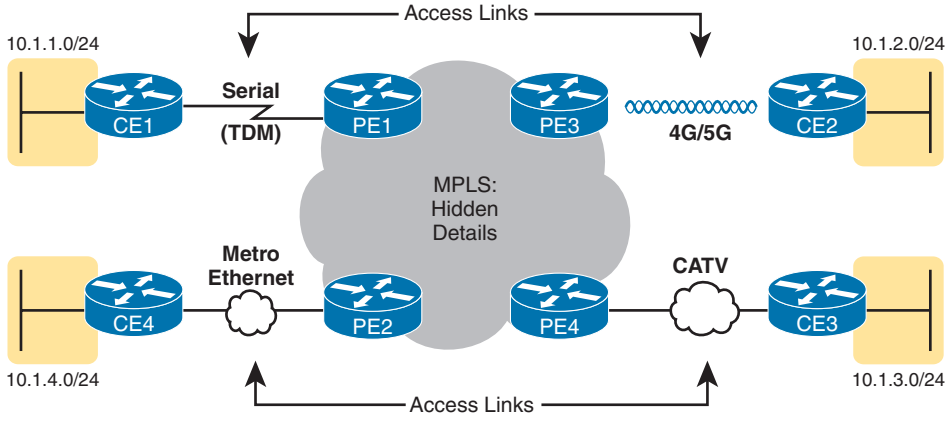



**Figure 19-10** *Popular MPLS Access Link Technologies*

## Layer 3 with MPLS VPN

Because MetroE provides a Layer 2 service, the SP does not need to understand anything about the customer's Layer 3 design. The SP knows nothing about the customer's IP addressing plan and does not need to participate with routing protocols.

MPLS VPNs take the completely opposite approach. As a Layer 3 service, MPLS must know the customers' IP addressing. The SP uses routing protocols and advertises those customer routes across the WAN. This section takes a closer look at what that means.

First, keep the primary goal in mind: to deliver IP packets between sites. The CE routers must learn routes for the subnets known to all the other CE routers; however, MPLS routing protocol design creates routing protocol neighbor relationships between the CE and PE routers but not between CE routers, as seen in Figure 19-11. MPLS supports all the standard routing protocols: RIPv2, EIGRP, OSPF, and even eBGP.

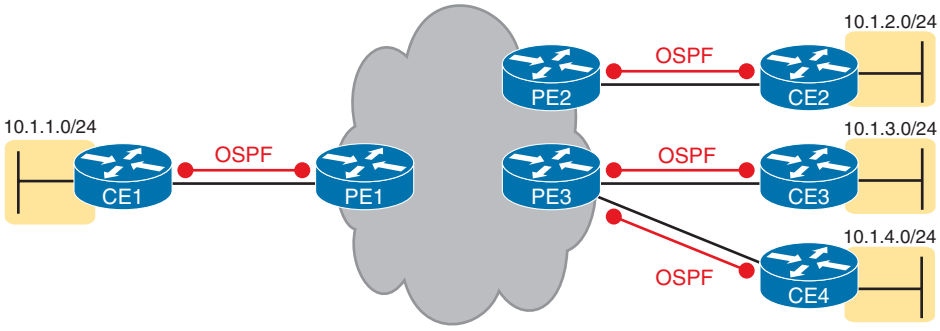



**Figure 19-11** *Routing Protocol Neighbor Relationships with MPLS Customer Edge Routers*

The CE routers learn all the routes because the PE routers also exchange routes; however, the PE routers must use additional routing protocol features beyond the topics already discussed in this book. First, the MPLS SP uses a variation of BGP called **Multiprotocol BGP (MP-BGP)** between the PE routers. MP-BGP provides the means for the PE routers to keep customer routes separate using virtual routing and forwarding (VRF) instances. (Chapter 20, "Cloud Architecture," discusses VRFs in the context of data center networks.) So, the CE routers learn routes from each other.

For instance, in Figure 19-12, for CE3 to learn routes from CE1, the process begins with CE1 advertising the routes to PE1. PE1 redistributes the routes into MP-BGP and advertises them to PE3. PE3 redistributes them into OSPF and advertises them to CE3. As a result, CE3's routes that forward packets over the MPLS WAN will list PE3's IP address as the next-hop router.



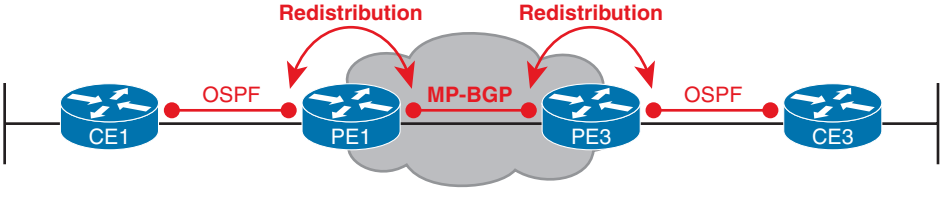



**Figure 19-12** *MPLS VPN Using Redistribution with MP-BGP at the PE Router*

## Internet VPNs

To build the Internet, Internet service providers (ISPs) need links to other ISPs and the ISPs' customers. The Internet core connects ISPs using various high-speed technologies. Additionally, Internet access links connect an ISP to each customer, again with different technologies.

The combination of ISP and customer networks that connect to the ISPs together creates the worldwide Internet.

While consumers typically connect to the Internet to reach destinations on the Internet, businesses can also use the Internet as a WAN service. First, the enterprise can connect each business site to the Internet. Then, using virtual private network (VPN) technology, the enterprise can create an Internet VPN. An Internet VPN can keep the enterprise's packet private through encryption and other means, even while sending the data over the Internet.

This final major section of the chapter discusses some of the basics of Internet access links. The section then details how an enterprise can communicate securely over the Internet, making the public Internet act like a private network by creating an Internet VPN.

## Internet Access

Enterprises can use private WAN technology to access an ISP's network, including the Ethernet WAN and MPLS technologies discussed earlier in this chapter. Figure 19-13 shows a visual reminder of these options.

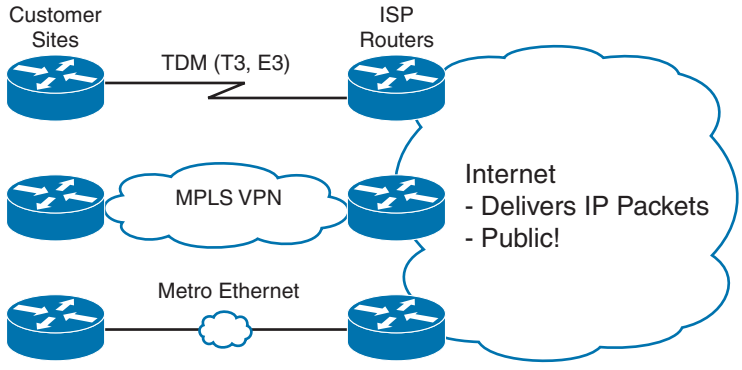



**Figure 19-13** *Three Examples of Internet Access Links for Companies*

Additionally, enterprises can also use Internet access technologies more commonly used by consumers, including DSL, cable, 4G/5G, and fiber Ethernet. The chapter includes this information about Internet access technologies to provide helpful background information before getting into VPN topics.

### Digital Subscriber Line

Telephone companies (telcos) have offered Internet access services since the early days of the Internet via the pre-existing phone lines connecting to most homes. Originally, that access used analog modems attached to the phone lines. Later, Integrated Services Digital Network (ISDN) created faster digital services over the same phone line. The third wave of technology to use local phone lines, digital subscriber line (DSL), added more improvements, with much faster speeds.

Figure 19-14 details how DSL works on a home phone line. The phone can do what it has always done: plug into a phone jack and send analog signals. For the data, a DSL modem connects to a spare phone outlet. The DSL modem sends and receives the data, as digital signals, at frequencies other than those used for voice, over the same local loop, even at the

same time as a telephone call. (Note that the physical installation often uses frequency filters not shown in the figure or discussed here.)

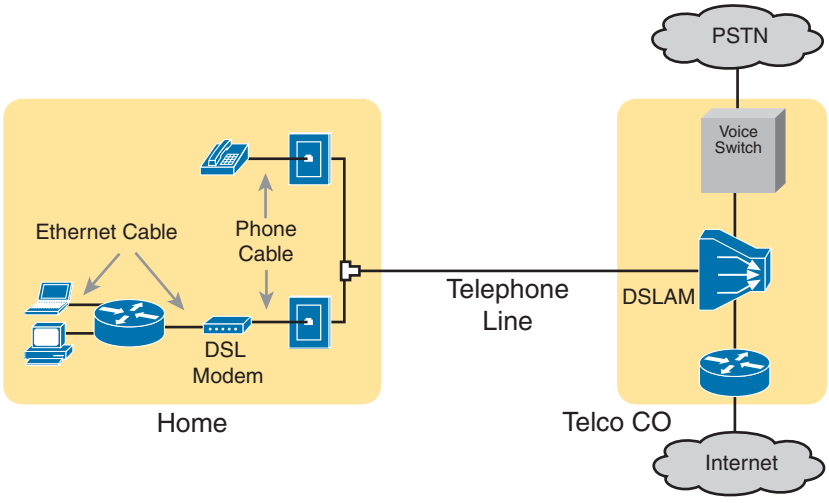



**Figure 19-14** *Wiring and Devices for a Home DSL Link*

Because DSL sends analog (voice) and digital (data) signals on the same line, the telco has to somehow split those signals on the telco side of the connection. The local loop connects to a *DSL access multiplexer* (DSLAM) in the nearby telco central office (CO). The DSLAM splits the digital data over to the router on the lower right in Figure 19-14, completing the Internet connection. The DSLAM also splits out the analog voice signals over to the voice switch on the upper right.

## Cable Internet

DSL uses the local link (telephone line) from the local telco. Cable Internet instead uses the cabling from what has become the primary competitor to the telco in most markets: the cable company.

Cable Internet creates an Internet access service that, when viewed generally rather than specifically, has many similarities to DSL. Like DSL, cable Internet takes full advantage of existing cabling, using the existing cable TV (CATV) cable to send data. Like DSL, cable Internet uses asymmetric speeds, sending data faster downstream than upstream, which works well for most consumer locations. And cable Internet still allows the original service (cable TV) while adding Internet access service.

Cable Internet uses CATV cabling but the same general model as telco with phone lines and DSL. The left side of Figure 19-15 shows a TV connected to the CATV cabling, just as it would normally connect. At another cable outlet, a cable modem connects to the same cable. The Internet service flows over one frequency, like another TV channel, just reserved for Internet service.

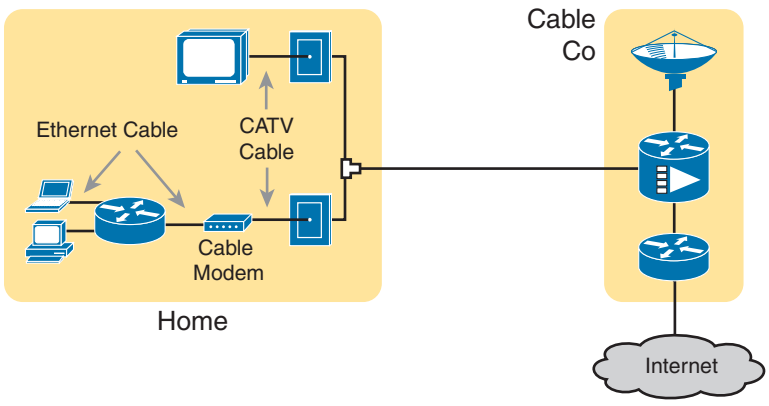



**Figure 19-15** *Wiring and Devices for a Home Cable Internet Link*

Similar to DSL, on the CATV company side of the connection (on the right side of the figure), the CATV company must split out the data and video traffic. Data flows to the lower right, through a router, to the Internet. The video comes in from video dishes for distribution to the TVs in people's homes.

## Wireless WAN (4G, 5G)

Many of you reading this book have a mobile phone with Internet access. You can use your phone to check your email, surf the web, download apps, and watch videos. This section touches on the big concepts behind the Internet access technology connecting those mobile phones.

Mobile phones use radio waves to communicate through a nearby mobile phone tower. The phone has a small radio antenna, and the provider has a much larger one sitting at the top of a tower within miles of you and your phone. Phones, tablet computers, laptops, and even routers (with the correct interface cards) can communicate through the Internet using this technology, as represented in Figure 19-16.

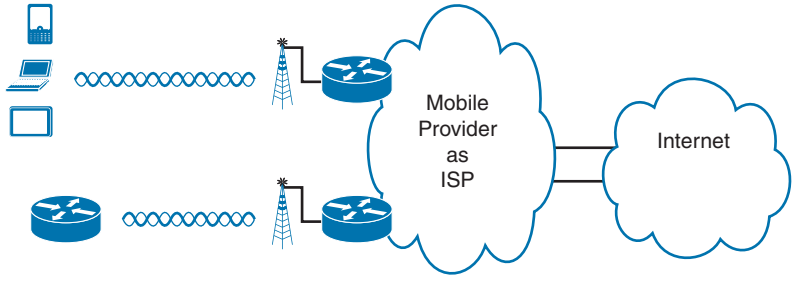



**Figure 19-16** *Wireless Internet Access Using 3G/4G/5G Technology*

The mobile phone radio towers also have cabling and equipment, including routers. The mobile provider builds its IP network much like an ISP builds an IP network. The customer IP packets pass through the IP router at the tower into the mobile provider's IP network and then out to the Internet.

The market for mobile phones and wireless Internet access for other devices is large and competitive. As a result, mobile providers spend a lot of money advertising their services,

with lots of names for one service or the other. Frankly, it can be difficult to tell what all the marketing jargon means, but the industry uses many of these terms:

**Wireless Internet:** This general term refers to Internet services from a mobile phone or any device that uses the same technology.

**1G/2G/3G Wireless:** Short for the first, second, and third generations of mobile phone access networks, which have come and gone from the market. (For perspective, note that the major carriers in the USA shut down their 3G services in 2022.)

**4G and 4G LTE Wireless:** Short for fourth generation, 4G improved both upload and download data rates versus 3G. 4G refers to the entire standard, but over time, the term *Long-Term Evolution (LTE)* came to refer to improved 4G.

**5G Non-standalone (NSA) Wireless (sub-6):** 5G refers to the fifth generation of wireless WAN technologies. 5G Non-standalone (NSA) refers to a subset of the 5G standard that works with the older 4G LTE network (hence non-standalone), with similar characteristics to 4G LTE, but with improvements in speed and latency. It also goes by sub-6 because it uses frequencies just beneath 6 GHz.

**5G Standalone (SA) Wireless (mmWave):** A higher performance part of the 5G standards, 5G standalone (SA) offers much faster data rates than 5G NSA. However, 5G SA works over shorter distances than 5G NSA, requiring more antennas to cover the same geographical footprint. 5G SA also goes by the name millimeter Wave (mmWave), because the wavelengths of the signals happen to be millimeters long.

The takeaway from all this jargon is this: when you hear about wireless Internet services with a mobile phone tower in the picture—whether the device is a phone, tablet, or PC—it is probably a 4G or 5G wireless Internet connection.

Enterprises can use this same wireless technology to connect to the Internet. For instance, a network engineer can install a 5G wireless card in a router. ISPs team with wireless operators to create contracts for wireless and Internet service.

### Fiber (Ethernet) Internet Access

The consumer-focused Internet access technologies discussed in this section use a couple of different physical media. DSL uses the copper wiring installed between the telco CO and the home. Cable uses the copper CATV cabling installed from the cable company to the home. And, of course, wireless WAN technologies do not use cables for Internet access.

The cabling used by DSL and cable Internet uses copper wires, but, comparing different types of physical media, fiber-optic cabling generally supports faster speeds for longer distances. That is, just comparing physical layer technologies across the breadth of networking, fiber-optic cabling supports longer links, and those links often run at equivalent or faster speeds.

Some ISPs now offer Internet access that goes by the name *fiber Internet*, or simply *fiber*. To make that work, some local company that owns the rights to install cabling underground in a local area (often a telephone company) installs new fiber-optic cabling. Once the cable plant is in place (a process that often takes years and a large budget), the fiber ISP connects customers to the Internet using the fiber-optic cabling. Often not stated, the SP uses Ethernet protocols over the fiber. The result is high-speed Internet to the home.

## Internet VPN Fundamentals

Private WANs have some wonderful security features. In particular, the customers who send data through the WAN have good reason to believe that no attackers saw the data in transit or even changed the data to cause some harm. The private WAN service provider promises to send one customer's data to other sites owned by that customer but not to sites owned by other customers, and vice versa.

VPNs try to provide the same secure features as a private WAN while sending data over a network open to other parties (such as the Internet). Compared to a private WAN, the Internet does not provide a secure environment that protects the privacy of an enterprise's data. Internet VPNs provide essential security features, such as

- **Confidentiality (privacy):** Preventing anyone in the middle of the Internet (man in the middle) from being able to read the data
- **Authentication:** Verifying that the sender of the VPN packet is a legitimate device and not a device used by an attacker
- **Data integrity:** Verifying that nothing changed the packet as it transited the Internet
- **Anti-replay:** Preventing a man in the middle from copying packets sent by a legitimate user, so they can later send those packets and appear to be that legitimate user

To accomplish these goals, two devices near the edge of the Internet create a VPN, sometimes called a *VPN tunnel*. These devices add headers to the original packet, with these headers including fields that allow the VPN devices to make the traffic secure. The VPN devices also encrypt the original IP packet, meaning that its contents are undecipherable to anyone who examines a copy of the packet as it traverses the Internet.

Figure 19-17 shows a VPN between branch office router R1 and a Cisco firewall (FW1). By definition, a VPN that connects two sites to support traffic from all hosts at those sites goes by the name **site-to-site VPN**.

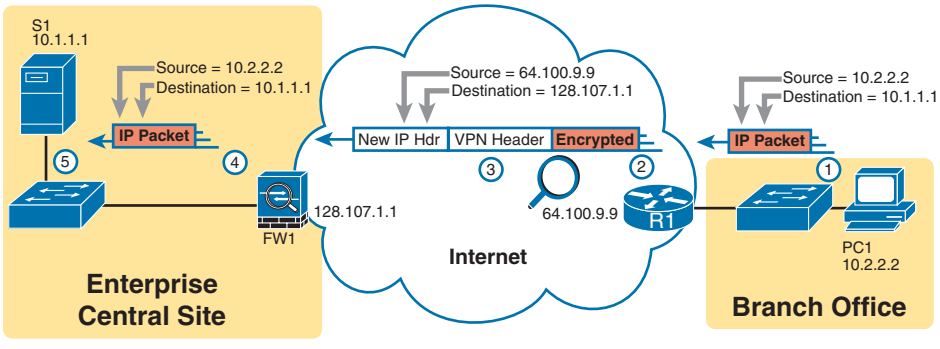



**Figure 19-17** *VPN Tunnel Concepts for a Site-to-Site Intranet VPN*

The figure shows the following steps, which explain the overall flow:

1. Host PC1 (10.2.2.2) on the right sends a packet destined for the web server (10.1.1.1), just as it would without a VPN.

2. Router R1 encrypts the packet, adds some VPN headers, adds another IP header (with public IP addresses), and forwards the packet, as shown in the upper-center of the figure.
3. Even if an attacker in the Internet copied the packet (called a man-in-the-middle attack), they could not change the packet without being noticed. The attacker also cannot read the contents of the original packet.
4. Firewall FW1 receives the packet, confirms the sender's authenticity, confirms no changes occurred in the packet, and decrypts the original packet. FW1 then forwards the original packet to the web server.
5. Server S1 receives the unencrypted packet.

Internet-based VPNs give enterprises great options for WAN servers. High-speed Internet access often costs less than private WAN access links. The Internet is seemingly everywhere, making this kind of solution available worldwide. If the enterprise uses VPN technology and protocols, the communications are secure.

**NOTE** The term *tunnel* refers to any protocol's packet that is sent by encapsulating the packet inside another packet. The term *VPN tunnel* may or may not imply that the tunnel also uses encryption.

### Site-to-Site VPNs with IPsec

A site-to-site VPN provides VPN services for the devices at two sites with a single VPN tunnel. For instance, if each site has dozens of devices that need to communicate between sites, the various devices do not have to act to create the VPN. Instead, the network engineers configure devices such as routers and firewalls to create one VPN tunnel. The tunnel remains up and operating all the time, so it is always available when end-user devices send data. All the endpoint devices at each site can communicate using the VPN, unaware of the VPN, without needing to create a VPN for themselves.

**IPsec** defines one popular set of rules for creating secure VPNs. IPsec is an architecture or framework for security services for IP networks. The name derives from the title of the RFC that defines it (RFC 4301, "Security Architecture for the Internet Protocol"), more generally called IP Security, or IPsec.

IPsec defines how two devices, both of which connect to the Internet, can achieve the main goals of a VPN, as listed at the beginning of this section: confidentiality, authentication, data integrity, and anti-replay. IPsec does not define just one way to implement a VPN but allows several different protocol options for each feature. IPsec provides a solid architectural VPN framework by allowing for changes and improvements to individual security functions over time.

IPsec encryption might sound intimidating, but if you ignore the math—and thankfully, you can—IPsec encryption is not too difficult to understand. IPsec encryption uses a pair of encryption algorithms, essentially math formulas, to meet a couple of requirements. First, the two math formulas are a matched set:

- One to hide (encrypt) the data
- Another to re-create (decrypt) the original data based on the encrypted data

The choice of encryption and decryption formulas helps thwart attackers. For instance, if an attacker intercepted the encrypted packet but did not have the secret password (called an *encryption key*), the attacker could only decrypt the packet through great effort. Also, if an attacker happened to decrypt one packet, the formulas also give them no insights in how they might decrypt a second packet.

The process for encrypting data for an IPsec VPN works generally as shown in Figure 19-18. Note that the *encryption key* is also known as the *session key*, **shared key**, or *shared session key*.

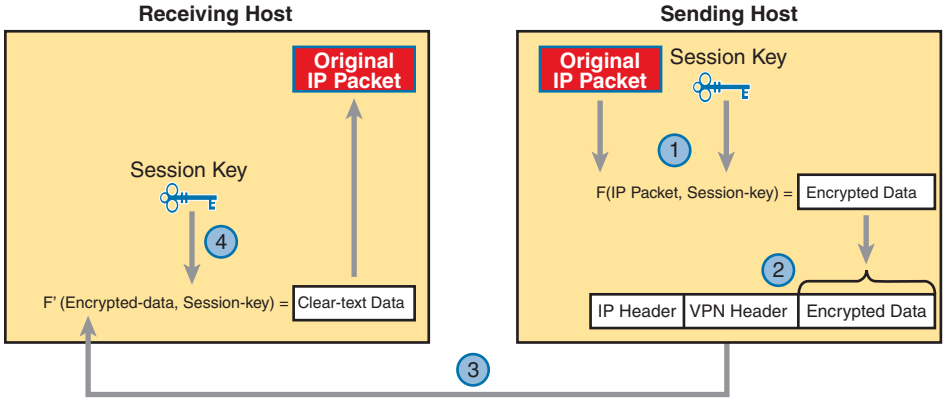



**Figure 19-18** *Basic IPsec Encryption Process*

The four steps highlighted in the figure are as follows:

1. The sending VPN device (router R1 in Figure 19-17) feeds the original packet and the session key into the encryption formula, calculating the encrypted data.
2. The sending device encapsulates the encrypted data into a packet, which includes the new IP header and VPN header.
3. The sending device sends this new packet to the destination VPN device (FW1 back in Figure 19-17).
4. The receiving VPN device runs the corresponding decryption formula using the encrypted data and session key—the same key value used on the sending VPN device—to decrypt the data.

While Figure 19-18 shows the basic encryption process, Figure 19-19 shows a broader view of an Enterprise IPsec VPN. Devices use some related VPN technology like Generic Routing Encapsulation (GRE) to create the concept of a tunnel (a virtual link between the routers), with three such tunnels shown in the figure. The router configuration then adds IPsec security features to the data that flows over the tunnel. (Note that the figure shows IPsec and GRE, but IPsec teams with other VPN technologies as well.)

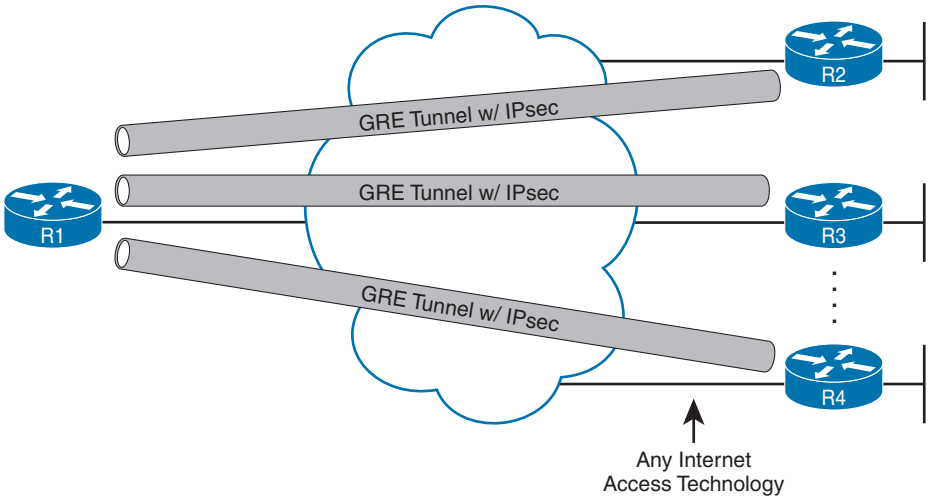



**Figure 19-19** *Site-to-Site VPN Tunnels with GRE and IPsec*

## Remote Access VPNs with IPsec

A site-to-site VPN exists to support multiple devices at each site, with the IT staff creating a permanent VPN connection to support all users. In contrast, user devices can dynamically initiate VPN connections in cases where a permanent site-to-site VPN does not exist. Such a VPN connection, terminated by and initiated from the end-user device, is called a **remote access VPN**.

For instance, a user can walk into a coffee shop and connect to the free Wi-Fi with a tablet or laptop, but that coffee shop does not have a site-to-site VPN to the user's enterprise network. Instead, software on the endpoint device creates a secure VPN connection back to the enterprise network, as shown in Figure 19-20.

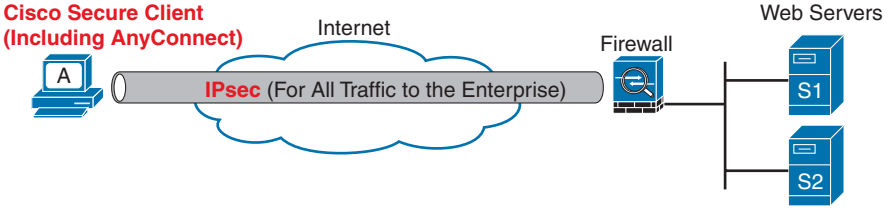



**Figure 19-20** *IPsec Remote Access IPsec VPN*

Many companies use the remote access VPN model in the figure for employees doing remote work, connecting from their home, a hotel, or any off-site location. The company can choose whether to use IPsec or Transport Layer Security **(TLS)** to secure the traffic.

To support all traffic sent by the computer, the end-user device requires VPN client software. The networking staff also installs and configures a device to act as a VPN concentrator, defining whether to use TLS or IPsec. Employee devices then connect to the VPN concentrator. While routers can play that role, companies typically use firewall products.

**NOTE** For many years, Cisco branded its VPN client software as the Cisco AnyConnect Secure Mobility Client. Cisco has replaced that product with the **Cisco Secure Client (Including AnyConnect)**. See www.cisco.com/go/secureclient for more info.

IPsec works a little differently when used for remote access versus site-to-site VPNs. **IPsec tunnel mode** is used for site-to-site encryption, which encrypts the entire original packet. **IPsec transport mode** is used for remote access VPNs, which encrypts the data of the original IP packet—that is, everything after the IP header—but not the IP header itself. Figure 19-21 shows a visual comparison of the two modes.

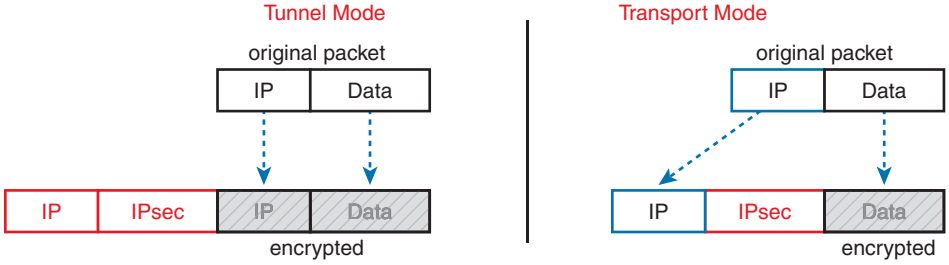



**Figure 19-21** *IPsec Tunnel and Transport Mode—What Is Encrypted*

Site-to-site and remote access IPsec VPNs also have other differences, given their different roles. Table 19-4 summarizes the differences for easier study.

Key Topic

**Table 19-4** Comparisons of Site-to-Site and Remote Access IPsec VPNs

| Attribute | Site-to-Site IPsec VPN | Remote Access IPsec VPN |
|---|---|---|
| Does the end-user device need VPN client software? | No | Yes |
| Devices supported by one VPN: one or many | Many | One |
| Typical use: on-demand or permanent | Permanent | On-demand |
| Does the VPN use IPsec tunnel mode? | Yes | No |
| Does the VPN use IPsec transport mode? | No | Yes |

### Remote Access VPNs with TLS

While the exam topics specifically mention IPsec VPNs, VPN technologies include many protocols beyond IPsec. This final topic discusses one that most people use every day: Transport Layer Security, or TLS.

For most websites today, when you connect, your browser uses HTTPS (RFC 9110), which secures the HTTP protocol. HTTPS secures HTTP by using TLS (RFC 8446).

The world has migrated away from HTTP to HTTPS. Just look at most URIs you see in your web browser: they all start with HTTPS. As a result, a browser completes the secure connection using HTTPS, which uses TLS, before connecting to the site.

When opening a new web page, the browser creates a TCP connection to well-known port 443 (default) and then initializes a TLS session. TLS encrypts data sent between the browser and the server (application layer header and data—but not the IP or TCP header) while authenticating the user. Then the HTTPS messages flow over the TLS VPN connection.

**NOTE** In years past, Secure Sockets Layer (SSL) played the same role as TLS. SSL has been deprecated (see RFC 7568) and replaced by TLS.

In its most common use, TLS creates a host-to-host secure connection between one client's web browser and one web server. If you open a web browser tab and connect to a website, your browser creates a TLS connection to that server. Open more browser tabs, and your browser creates additional TLS host-to-host connections, all secured with TLS. TLS can also be used for remote access VPNs. For instance, the Cisco Secure Client can use TLS when connecting to a firewall, supporting all traffic sent by that host. Figure 19-22 shows both.

Key Topic

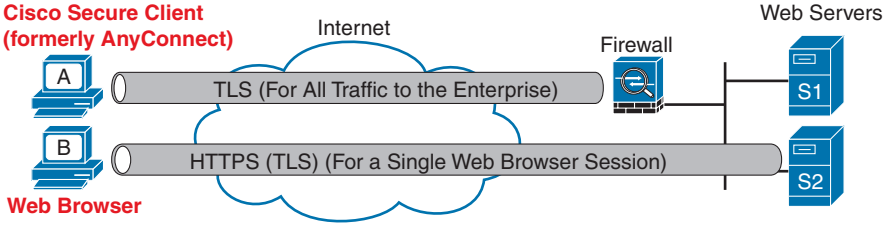



**Figure 19-22** *Remote Access VPN Options (TLS)*

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 19-5 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

19

**Table 19-5** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |

## Review All the Key Topics

Key Topic

**Table 19-6** Key Topics for Chapter 19

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 19-2 | Metro Ethernet terminology in context | 417 |
| Table 19-3 | MetroE service types per MEF | 418 |
| Figure 19-5 | MetroE Ethernet LAN (E-LAN) service concept | 420 |
| List | Ideas about customer Layer 3 addressing and what an MPLS VPN provider needs to know | 423 |
| Figure 19-9 | MPLS terminology in context | 424 |
| Figure 19-19 | Concepts of site-to-site VPNs with IPsec and GRE | 433 |
| Figure 19-20 | IPsec remote access VPN | 433 |
| Table 19-4 | Comparisons of site-to-site and remote access VPNs | 434 |
| Figure 19-22 | Concepts of remote access VPNs with TLS | 435 |

## Key Terms You Should Know

access link, Cisco Secure Client (including AnyConnect), customer edge (CE), E-LAN, E-Line, Ethernet WAN, full mesh, hub and spoke, IPsec, IPsec transport mode, IPsec tunnel mode, Metro Ethernet, MPLS VPN, Multiprotocol BGP (MP-BGP), Multiprotocol Label Switching (MPLS), partial mesh, point-to-point, point of presence (PoP), provider edge (PE), remote access VPN, service provider (SP), shared key, site-to-site VPN, TLS

*This page intentionally left blank*



CHAPTER 20

# Cloud Architecture

**This chapter covers the following exam topics:**

**1.0 Network Fundamentals**

**1.1 Explain the role and function of network components**

**1.1.g Servers**

**1.2 Describe the characteristics of network topology architectures**

**1.2.f On-premises and cloud**

**1.12 Explain virtualization fundamentals (server virtualization, containers, and VRFs)**

Cloud computing is an approach to offering IT services to customers. However, cloud computing is not a product, a set of products, a protocol, or any single thing. So, while there are accepted descriptions and definitions of cloud computing today, it takes a broad knowledge of IT beyond networking to know whether a particular IT service is or is not worthy of being called a cloud computing service.

To categorize an IT service as a cloud computing service, it should have these characteristics: It can be requested on-demand; it can dynamically scale (that is, it is elastic); it uses a pool of resources; it has a variety of network access options; and it can be measured and billed back to the user based on the amount used.

Cloud computing relies on automated data centers. For instance, to service requests, a cloud computing system will create virtual server instances—virtual machines (VMs)—and configure the settings on each VM to provide the requested service.

This chapter gives you a general idea of cloud services and network architecture. To do that, this chapter begins by discussing server virtualization basics. The following section then discusses the big ideas in cloud computing, with the final section discussing the impact of public clouds on packet flows in enterprise networks.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 20-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Server Virtualization | 1, 2 |
| Cloud Computing Services | 3, 4 |
| WAN Traffic Paths to Reach Cloud Services | 5, 6 |

1. Three virtual machines run on one physical server. Which server resources does a virtualization system typically virtualize so each VM can use the required amount of that resource? (Choose three correct answers.)
   a. NIC
   b. RAM
   c. Power
   d. Hypervisor
   e. CPU
2. Eight virtual machines run on one physical server; the server has two physical Ethernet NICs. Which answer describes a method that allows all eight VMs to communicate?
   a. The VMs must share two IP addresses and coordinate to avoid using duplicate TCP or UDP ports.
   b. The hypervisor acts as an IP router using the NICs as routed IP interfaces.
   c. Each VM uses a virtual NIC mapped to a physical NIC.
   d. Each VM uses a virtual NIC that logically connects to a virtual switch.
3. Which cloud service is most likely to be used for software development?
   a. IaaS
   b. PaaS
   c. SaaS
   d. SLBaaS
4. With which of the following cloud services would you purchase the service and then later install your software applications?
   a. IaaS
   b. PaaS
   c. SaaS
   d. SLBaaS
5. An enterprise plans to use a public cloud service and considers different WAN options. The answers list four options under consideration. Which option causes the most challenges when migrating from the original cloud provider to a different one?
   a. Using private WAN connections directly to the cloud provider
   b. Using an Internet connection without VPN

c. Using an intercloud exchange
d. Using an Internet connection with VPN

6. An enterprise plans to use a public cloud service and considers different WAN options. Which answers list a WAN option that provides good security by keeping the data private while also providing good QoS services? (Choose two answers.)
   a. Using private WAN connections directly to the cloud provider
   b. Using an Internet connection without VPN
   c. Using an intercloud exchange
   d. Using an Internet connection with VPN

# Foundation Topics

## Server Virtualization

When you think of a server, what comes to mind? Is it a desktop computer with a fast CPU? A desktop computer with lots of RAM? Is it hardware that would not sit upright on the floor, but something easily bolted into a rack in a data center? Do you not even think of hardware, but instead think of the server operating system (OS) running somewhere as a virtual machine (VM)?

All those answers are accurate from one perspective or another, but we ignore those details in almost every other discussion within the scope of the CCNA certification. From the perspective of most CCNA discussions, a server is a place to run applications, with users connecting to those applications over the network. This book represents the server with an icon that looks like a desktop computer (that is the standard Cisco icon for a server). This first topic breaks down different perspectives on what it means to be a server and prepares us to discuss cloud computing.

### Cisco Server Hardware

Think about the form factor of servers for a moment—that is, the shape and size of the physical server. If you were to build a server of your own, what would it look like? How big, how wide, how tall, and so on? Even if you have never seen a device characterized as a server, consider these key facts:

**No KVM:** For most servers, no permanent user sits near the server; all the users and administrators connect to the server over the network. As a result, there is no need for a permanent keyboard, video display, or mouse (collectively referred to as KVM).

**Racks of servers in a data center:** In the early years of servers, a server was any computer with a relatively fast CPU, large amounts of RAM, and so on. Today, companies put many servers into one room—a data center—and one goal is not to waste space. So, making servers with a form factor that fits in a standard rack makes for more efficient use of the available space—especially when you do not expect people to be sitting in front of each server.

For example, Figure 20-1 shows a photo of server hardware from Cisco. While you might think of Cisco as a networking company, around 2010, Cisco expanded its product line into the server market with the Cisco **Unified Computing System (UCS)** product line. The photo shows a UCS B-Series (Blade series) rack-mountable chassis, which fits in a 19-inch-wide rack and is 10.5 inches (six rack units) high. It supports 16 servers: two per each of the eight removable server blades. The bottom of the chassis holds four power supplies.



**Figure 20-1** *Cisco UCS Servers: B-Series (Blade)*

No matter the form factor, server hardware today supplies some capacity of CPU chips, RAM, storage, and network interface cards (NICs). But you also have to think differently about the OS that runs on the server because of a tool called *server virtualization*.

## Server Virtualization and Virtual Machine Basics

Think of a server—the hardware—as one computer. It can be one of the servers in a blade in Figure 20-1, a powerful computer you can buy at the local computer store. . . whatever. Traditionally, when you think of one server, that one server runs one OS. The hardware includes a CPU, RAM, permanent storage (like disk drives), and one or more NICs. And that one OS can use all the hardware inside the server and then run one or more applications. Figure 20-2 shows those main ideas.

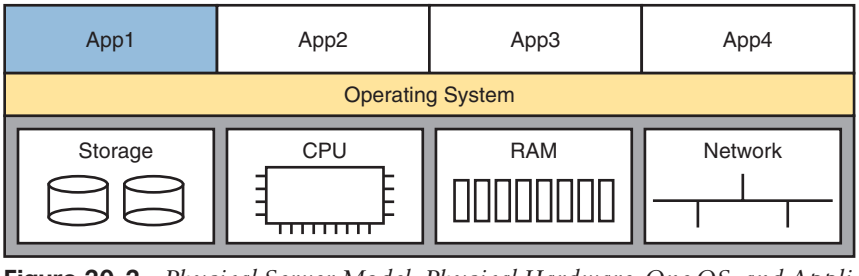



**Figure 20-2** *Physical Server Model: Physical Hardware, One OS, and Applications*

With the physical server model shown in Figure 20-2, each physical server runs one OS, and that OS uses all the hardware in that one server. That was true of servers in the days before server virtualization.

Today, most companies use server hardware as part of a virtualized data center. That means the company purchases server hardware, installs it in racks, and then treats all the CPU, RAM, and so on as capacity in the data center. Then, each OS instance is decoupled from the hardware and is therefore virtual (in contrast to physical). Each piece of hardware that we would formerly have thought of as a server runs multiple instances of an OS at the same time, with each virtual OS instance called a **virtual machine**, or **VM**.

A single physical host (server) often has more processing power than you need for one OS. Thinking about processors, modern server CPUs have multiple cores (processors) in a single CPU chip. Each core may also be able to run multiple threads with a feature called *multithreading*. So, when you read about a particular Intel processor with 40 cores and multithreading (typically two threads per core), one CPU chip can execute 80 programs concurrently. The hypervisor (introduced shortly) can then treat each available thread as a **virtual CPU (vCPU)**, assigning each VM vCPUs as needed.

A VM—that is, an OS instance decoupled from the server hardware—still must execute on hardware. Each VM configures the number of vCPUs it needs, minimum RAM, and so on. The virtualization system then starts each VM on some physical server so that enough physical server hardware capacity exists to support all the VMs running on that host. So, at any one time, each VM is running on a physical server, using a subset of the CPU, RAM, storage, and NICs on that server. Figure 20-3 shows a graphic of that concept, with four separate VMs running on one physical server.

Key Topic

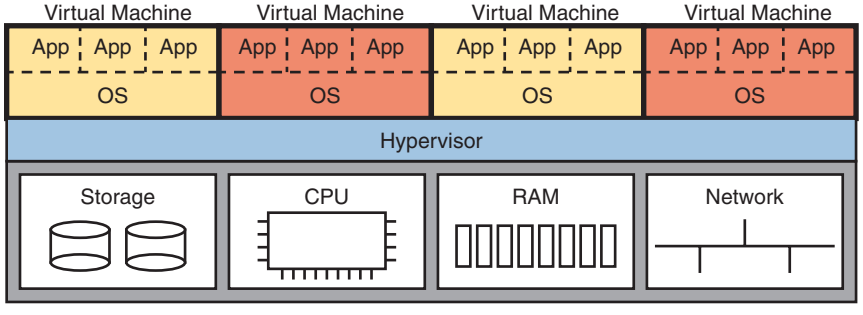



**Figure 20-3** *Four VMs Running on One Host; Hypervisor Manages the Hardware*

To make server virtualization work, each physical server (called a **host** in the server virtualization world) uses a **hypervisor**. The hypervisor manages and allocates the host hardware (CPU, RAM, etc.) to each VM based on the settings for the VM. Each VM runs as if it is running on a self-contained physical server, with a specific number of virtual CPUs and NICs and a set amount of RAM and storage. For instance, if one VM defines a need for four CPUs with 8 GB of RAM, the hypervisor allocates those resources to the VM.

---

Answers to the "Do I Know This Already?" quiz:

**1** A, B, E **2** D **3** B **4** A **5** A **6** A, C

The following are a few of the vendors and product family names associated with virtualized data centers:

- VMware vCenter
- Microsoft HyperV
- Citrix Hypervisor (formerly XenServer)
- Red Hat KVM

Beyond the hypervisor, companies sell complete virtualization systems. These systems allow virtualization engineers to dynamically create VMs, start them, move them (manually and automatically) to different servers, and stop them. For instance, before powering off server hardware to perform maintenance, the staff can move the VMs to another host (often while running).

## Networking with Virtual Switches on a Virtualized Host

Server virtualization tools provide a wide variety of options for connecting VMs to networks. This book does not attempt to discuss them all, but it can help to get some of the basics down before thinking more about cloud computing.

First, what does a physical server include for networking functions? Typically, it has one or more NICs, maybe as slow as 1 Gbps, often 10 Gbps today, and maybe as fast as 40 Gbps.

Next, think about the VMs. Normally, an OS has one NIC, maybe more. Each VM has (at least) one NIC, but for a VM, it is a **virtual NIC**. (For instance, in VMware's virtualization systems, the VM's virtual NIC is called a vNIC.)

Finally, the server must combine the ideas of the physical NICs with the vNICs used by the VMs into a network. Each server uses an internal Ethernet switch concept, often called (you guessed it) a **virtual switch**, or **vSwitch**. Figure 20-4 shows an example, with four VMs, each with one vNIC. The physical server has two physical NICs. The vNICs and physical NICs connect internally to a virtual switch.

Key Top

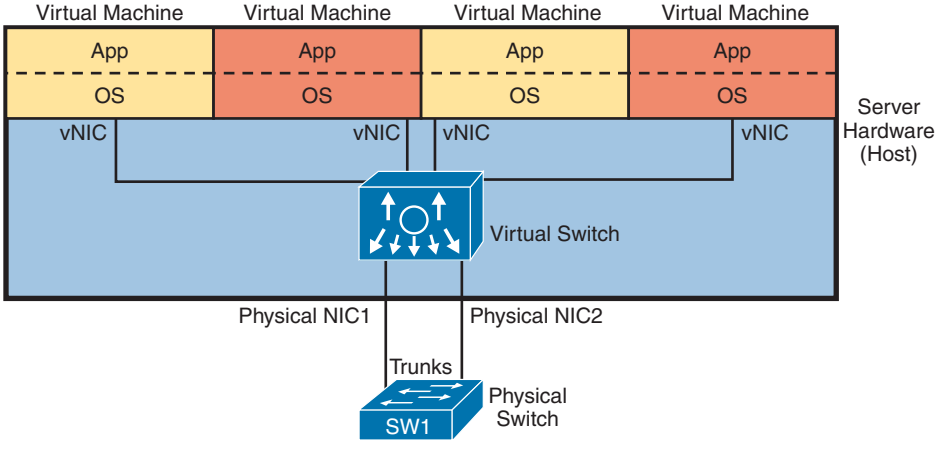



**Figure 20-4** *Basic Networking in a Virtualized Host with a Virtual Switch*

In most cases today, the hypervisor vendor supplies the vSwitch, but sometimes, a different vendor (like Cisco) may supply an alternate vSwitch. For instance, Cisco offers the Cisco ACI Virtual Edge switch, supporting Cisco ACI networking.

The vSwitch shown in Figure 20-4 uses the same networking features you now know from your CCNA studies. In particular:

- **Ports connected to VMs:** The vSwitch can configure a port so that the VM will be in its own VLAN, or share the same VLAN with other VMs, or even use VLAN trunking to the VM itself.
- **Ports connected to physical NICs:** The vSwitch uses the physical NICs in the server hardware so that the switch is adjacent to the external physical LAN switch. The vSwitch can (and likely does) use VLAN trunking.
- **Automated configuration:** The configuration can be easily done within the same virtualization software that controls the VMs. That programmability allows the virtualization software to move VMs between hosts (servers) and reprogram the vSwitches so that the VM has the same networking capabilities no matter where the VM is running.

## Software Containers

Software containers have the same goal as VMs but use different methods. Compared to VMs, containers take less CPU and memory while also taking less time to initialize and shut down—making them more appealing in some cases.

To appreciate the differences, consider VMs again for a moment. A single VM exists on disk, waiting to run, as one large file—typically many gigabytes because it holds an entire OS. Starting a VM takes minutes; think of VM initialization time as similar to the time it takes to boot your desktop or laptop computer. Also, VMs require some work: both the OS and application must be installed, and you need to apply software fixes over time.

Some of those perceived drawbacks of VMs led to a second wave of server virtualization, called software containers, or simply containers.

First, consider the word *container* as a generic term to understand the fundamentals. What do you imagine? Maybe you think of shipping containers that fill huge ships or ride behind tractor-trailer trucks. Perhaps you think of the plastic container you use to bring your lunch to school. Generically, containers hold other things, typically multiple things.

Software containers hold an application plus every other file it needs to work—other than the OS. For instance, the primary executable file for the application is in the container. The container has all the related files that come with the app, plus any libraries (standard code often used by applications) at the prescribed versions. It includes files used by the application, for instance, maybe a text file with application settings. The container—a file with a defined format that collects and includes all the component files—holds all the files. Because a container does not include the OS, however, it is usually smaller than a VM, often measured in megabytes rather than gigabytes.

**NOTE** By convention, the term **container image** refers to the file on disk that holds all the files that make up the application. The term **container** refers to a container once it has been started. For example, you might use a container image for a web server application. When you start that image five times on a server, you have five containers, that is, five instances of the web server running on the server.

So, if the container does not include the OS, how does it execute? It seems like something is missing. The answer: Architecturally, server hardware runs one instance of the OS—an OS that supports containers, like Windows or Linux. Additionally, you need a *container engine*, which is software that can install, start, stop, and monitor containers. Figure 20-5 shows the general idea.

Key Topic

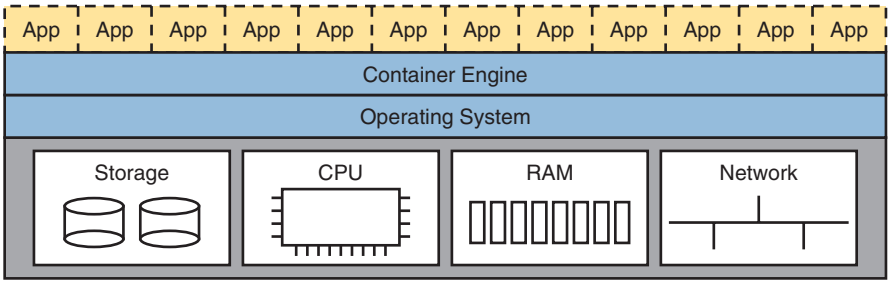



**Figure 20-5** *Twelve Containers Running on One Host Managed by Container Engine*

Starting a container requires a container engine, software that understands container file formats and operations. The container engine supplies a GUI to start, stop, and monitor the containers. Most data center operational activities happen remotely, however, so the container image includes shell commands and APIs to aid remote control and automation by virtualization software. Furthermore, when you start a container, it takes the time typical of starting an application (seconds or tens of seconds) rather than the minutes required to boot an OS.

While Linux and Windows include support for containers, containers became popular in the 2010s when some companies began offering related software services. Some of the most popular and common companies and sites include

- Docker (Docker.com)
- Kubernetes (k8s.io)
- Terraform (hashicorp.com)

Docker (www.docker.com) probably had the most significant early impact in popularizing containers, with the name *Docker* becoming almost synonymous with containers. For instance, Docker created rules for packaging container images. It also offers the docker engine as a container engine.

Docker also helps speed application development through Docker Hub (hub.docker.com), a website that offers over 100,000 container images. To begin developing an application, a developer can find an existing Docker container image that includes most of what the

application needs. For instance, the Docker container image for the world's most popular web server software (Apache) has been downloaded over one billion times per Docker Hub.

**NOTE** Docker has long had free and paid accounts and software. You can download Docker on your computer at no cost, download Docker containers from Docker Hub, and start running and using containers on your desktop if you want to learn more.

## The Physical Data Center Network

Next, consider the physical network in a virtualized data center. Each host—the physical host—needs a physical connection to the network. Looking again at Figure 20-4, that host, with two physical NICs, needs to connect those two physical NICs to a LAN switch in the data center.

Figure 20-6 shows the traditional cabling for a data center LAN. Each taller rectangle represents one rack inside a data center, with the tiny squares representing NIC ports and the lines representing cables.

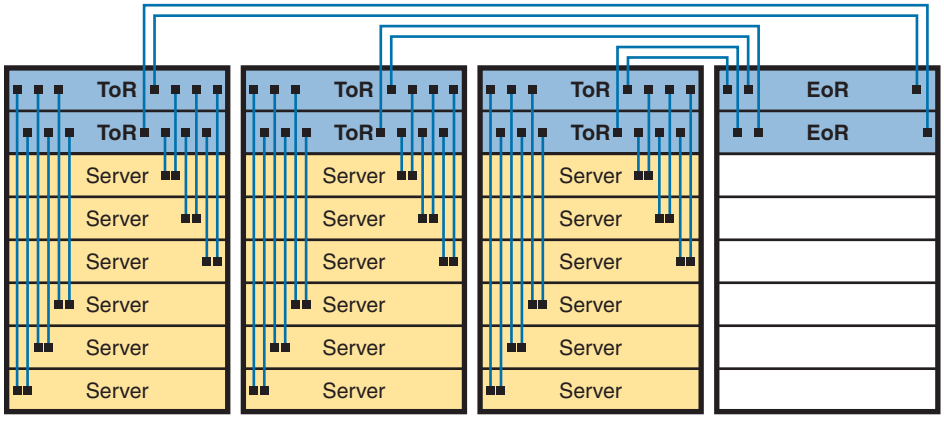



**Figure 20-6** *Traditional Data Center Top-of-Rack and End-of-Row Physical Switch Topology*

Often, each host is cabled to two different switches in the top of the rack—called Top of Rack (ToR) switches—to provide redundant paths into the LAN. Each ToR switch acts as an access layer switch from a design perspective. Each ToR switch connects to an End of Row (EoR) switch, which acts as a distribution switch and also connects to the rest of the network.

The design in Figure 20-6 uses a traditional data center cabling plan. Some data center technologies call for different topologies, in particular, Cisco Application Centric Infrastructure (ACI). ACI places the server and switch hardware into racks, but cables the switches with a different topology—a topology required for proper operation of the ACI fabric. Chapter 21 introduces ACI concepts.

## Workflow with a Virtualized Data Center

The first part of this chapter describes background information important to the upcoming discussions of cloud computing. Server virtualization greatly improves the operations of

many data centers, but virtualization alone does not create a cloud computing environment. Consider this example of a workflow through a virtualized (not cloud-based) data center to see the differences.

Some IT staff—call them server or virtualization engineers or administrators—order and install new hosts (servers). They gather requirements, plan for the required capacity, shop for hardware, order it, and install the hardware. They play the role of long-time server administrators and engineers, but now they also work with virtualization tools.

For the virtualization parts of the effort, the virtualization engineers also install and customize the virtualization tools beyond the hypervisor. For instance, one tool might give the engineers a view of the data center as a whole, with all VMs running there, with the idea that one data center is just a lot of capacity to run VMs. The server/virtualization engineers add new physical servers to the data center, configure the virtualization systems to use the new physical servers, and ensure it all works.

So far in this scenario, the work has been in preparation for providing services to some internal customer—a development team member, the operations staff, and so on. Now, an internal customer requests a "server." In truth, the customer wants many VMs, each with requirements for vCPUs, RAM, and so on. The customer makes a request to the virtualization/server engineer to set up the VMs, as shown in Figure 20-7.

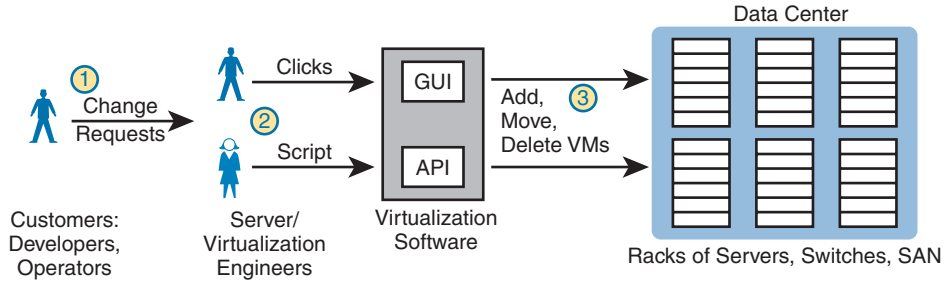



**Figure 20-7** *Traditional Workflow: Customer (Human) Asks Virtualization (Human) for Service*

The figure emphasizes what happens after the customer makes a request, which flows something like this:

**Step 1.** The customer of the IT group, such as a developer or a member of the operations staff, wants some service, like a set of new VMs.

**Step 2.** The virtualization/server engineer reacts to the request from the customer. The server/virtualization engineer clicks away at the user interface or, if the number of VMs is significant, often runs a program called a script to create the VMs.

**Step 3.** Regardless of whether the virtualization engineer clicked or used scripts, the virtualization software could then create many new VMs and start those on some hosts inside the data center.

The process shown in Figure 20-7 works great. However, that approach to providing services breaks some of the basic criteria of a cloud service. For instance, cloud computing requires self-service. Automated software processes should complete the request at step 2 rather than

require a human. Want some new VMs in a cloud world? Click a user interface to ask for some new VMs, get a cup of coffee, and your VMs will be set up and started, to your specification, in minutes.

Summarizing some of the key points about a virtualized data center made so far, which enable cloud computing:

- The OS is decoupled from the hardware on which it runs, so the OS, as a VM/container, can run on any server in a data center with enough resources to run the VM/container.
- The virtualization software can automatically start and move the VM/container between servers in the data center.
- Data center networking includes virtual switches and NICs within each host (server).
- The virtualization software can program data center networking features, allowing new VMs/containers to be configured, started, moved, and stopped, with the networking details changing automatically.

## Cloud Computing Services

**NOTE** Cloud services support VMs and containers, but the rest of the chapter refers mostly to VMs just to avoid repetitive references to "VMs and containers."

Cloud computing uses virtualization products and products built specifically to enable cloud features. However, cloud computing is not just a set of products; instead, it is a way of offering IT services. So, understanding what cloud computing is—and is not—takes a little work; this next section introduces the basics.

From the just-completed discussions about virtualization, you already know one characteristic of a cloud service: it must allow self-service provisioning by the consumer. That is, the consumer or customer of the service must be able to request the service and receive that service without the delay of waiting for a human to have time to work on it, consider the request, do the work, and so on.

The US National Institute of Standards and Technology (NIST) defines cloud computing with the following list of paraphrased attributes:

Key Topic

**On-demand self-service:** The IT consumer chooses when to start and stop using the service, without any direct interaction with the provider of the service.

**Broad network access:** The service must be available from many types of devices and over many types of networks (including the Internet).

**Resource pooling:** The provider creates a pool of resources (rather than dedicating specific servers for use only by certain consumers) and dynamically allocates resources from that pool for each new request from a consumer.

**Rapid elasticity:** To the consumer, the resource pool appears unlimited (that is, it expands quickly, so it is called *elastic*), and the requests for new services are filled quickly.

**Measured service:** The provider can measure the usage and report that usage to the consumer for transparency and billing.

Keep this list of five criteria in mind while you work through the rest of the chapter. Later parts of the chapter refer back to the list.

To further develop this definition, the next few pages look at two branches of the cloud universe—private cloud and public cloud—also to further explain some of the points from the NIST definition.

## Private Cloud (On-Premise)

Look back to the workflow example in Figure 20-7 with a virtualized data center. Now think about the five NIST criteria for cloud computing. If you break down the list versus the example around Figure 20-7, it seems as though the workflow may meet at least some of these five NIST cloud criteria, and it does. In particular, as described in this chapter, a virtualized data center pools resources to allow dynamic allocation. You could argue that a virtualized data center is elastic, in that the resource pool expands. However, the process may not be rapid because the workflow requires human checks, balances, and time before provisioning new services.

Private cloud creates a service, inside a company for internal customers, that meets the five criteria from the NIST list. To create a private cloud, an enterprise often expands its IT tools (like virtualization tools), changes internal workflow processes, adds additional tools, and so on.

**NOTE** The world of cloud computing has long used the terms **private cloud** and **public cloud**. In more recent years, you may also find references that instead use a different pair of terms for the same ideas, with *on-premise* meaning *private cloud*, and *cloud* meaning *public cloud*. Note that the CCNA 200-301 exam topics use the newer pair of terms.

As some examples, consider what happens when an application developer at a company needs VMs to use when developing an application. With private cloud, the developer can request a service. The associated VMs automatically start and are available within minutes, with most of the time lag being the time to boot the VMs. If the developer wants many more VMs, the developer can assume that the private cloud will have enough capacity, with new requests serviced rapidly. And all parties should know that the IT group can measure the usage of the services for internal billing.

Cloud computing services enable self-service through a **cloud services catalog**. The catalog has a web page that lists all services available via the company's cloud infrastructure. With private cloud, the (internal) consumers of IT services—developers, operators, and the like—can click to choose from the cloud services catalog. And if the request is for a new set of VMs, the VMs appear and are ready for use in minutes, without human interaction for that step, as seen at step 2 of Figure 20-8.

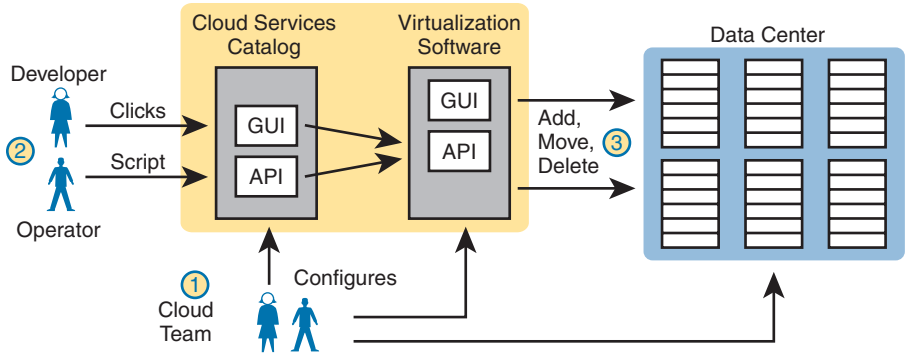



**Figure 20-8** *Basic Private Cloud Workflow to Create One VM*

The cloud team adds some tools and processes to its virtualized data center to make this process work. For instance, it installs software to create the cloud services catalog, both with a user interface and code that interfaces to the APIs of the virtualization systems. That services catalog software can react to consumer requests, using APIs in the virtualization software to add, move, and create VMs, for instance. Also, the cloud team—composed of server, virtualization, network, and storage engineers—focuses on building the resource pool, testing and adding new services to the catalog, handling exceptions, and watching usage reports, so they can create the capacity to be ready to handle future requests.

Notably, with the cloud model, the cloud team no longer handles individual requests for adding 10 VMs here, and 50 there, with change requests from different groups.

Summarizing, with private cloud, the enterprise owns and manages the cloud service. The term *private cloud* emphasizes ownership by the company that uses the service. The alternate terms *on-premise* and *on-premise cloud* refer to the cloud service existing within a company-controlled site. Most importantly, to be a cloud service rather than an internal data center, a private cloud service meets the NIST criteria.

## Public Cloud

With a private cloud, the cloud provider and the cloud consumer are part of the same company. With public cloud, the reverse is true: a public cloud provider offers services, selling those services to consumers in other companies. Internet service providers sell Internet services to many enterprises; public cloud providers have a similar business model, selling their services to many enterprises.

The workflow in public cloud happens somewhat like private cloud when you start from the point of a consumer asking for some service (like a new VM). As shown on the right of Figure 20-9, at step 1, the consumer asks for the new service from the service catalog web page. At step 2, the virtualization tools react to the request to create the service. Once started, the services are available but running in a data center that resides elsewhere in the world, and certainly not at the enterprise's data center (step 3).

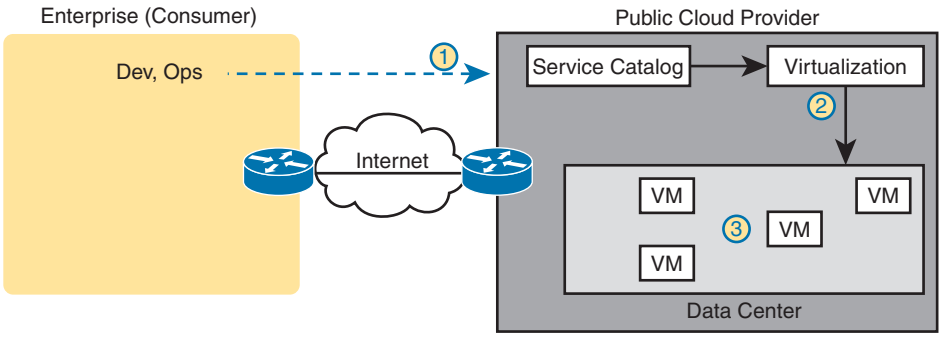



**Figure 20-9** *Public Cloud Provider in the Internet*

Public cloud services exist in a data center owned by the cloud provider, so the customer needs some network connection to the public cloud provider. Cloud providers support multiple network options. They each connect to the Internet so that apps and users inside the consumer's network can communicate with the apps that the consumer runs in the cloud provider's network. However, broad network access is one of the five NIST criteria for cloud computing. Cloud providers offer different networking options, including virtual private network (VPN) and private wide-area network (WAN) connections between consumers and the cloud.

## Cloud and the "As a Service" Model

So, what do you get with cloud computing? So far, this chapter has just shown a VM (or container) as a service. With cloud computing, there are a variety of services, and three stand out as the most commonly seen in the market today.

In cloud computing, categories of services use names that end in "as a Service" or aaS. Cloud computing delivers services, a more abstract concept than a physical server or a software package. So, the industry uses a variety of terms that end in "as a Service." And each "aaS" term has a different meaning.

This next topic explains the three most common cloud services: Infrastructure as a Service, Software as a Service, and Platform as a Service.

### Infrastructure as a Service

**Infrastructure as a Service (IaaS)** may be the easiest cloud computing service for most people to understand. For perspective, think about any time you have shopped for a computer. You thought about the OS to run (the latest Microsoft OS, or Linux, or macOS if shopping for a Mac). You compared prices based on the CPU and its speed, how much RAM the computer had, the disk drive size, and so on.

IaaS offers a similar idea, but the consumer receives the use of a VM. You specify the amount of hardware performance/capacity to allocate to the VM (number of virtual CPUs, amount of RAM, and so on), as shown in Figure 20-10. You can even pick an OS to use. Once you've made your selection, the cloud provider starts the VM, which boots the chosen OS. You can even think of IaaS as a generic VM-as-a-service.

**NOTE** In the virtualization and cloud world, starting a VM is often called *spinning up a VM* or *instantiating a VM*.

Key Topic

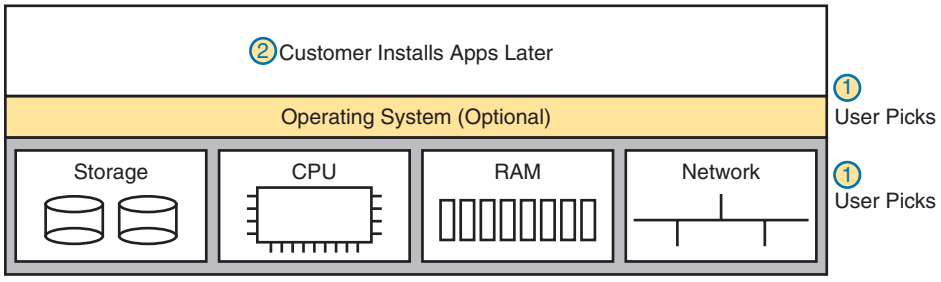



**Figure 20-10** *IaaS Concept*

The provider also gives the consumer details about the VM so the consumer can connect to the OS's user interface, install more software, and customize settings. For example, imagine the customer wants to run a particular application on the server. If that customer wanted to use Microsoft Exchange as an email server, the customer would need to connect to the VM and install Exchange.

Figure 20-11 shows a web page from Amazon Web Services (AWS), a public cloud provider, where you could create a VM as part of its IaaS service. The screenshot shows that the user selected a small VM called "micro." If you look closely at the text, you can read the heading and numbers to see that this particular VM has 1 vCPU and 1 GB of RAM.

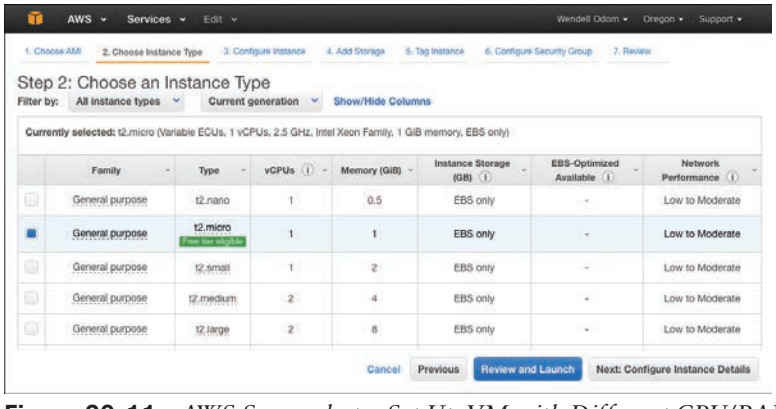



**Figure 20-11** *AWS Screenshot—Set Up VM with Different CPU/RAM/OS*

## Software as a Service

With **Software as a Service (SaaS)**, the consumer receives a service with working software. The cloud provider may use many VMs and containers to create the service, but those are hidden from the consumer. The cloud provider licenses, installs, and supports whatever

software is required. The cloud provider then monitors performance of the application. However, the consumer chooses to use the application, signs up for the service, and starts using the application—no further installation work required. Figure 20-12 shows these main concepts.

Key Topic

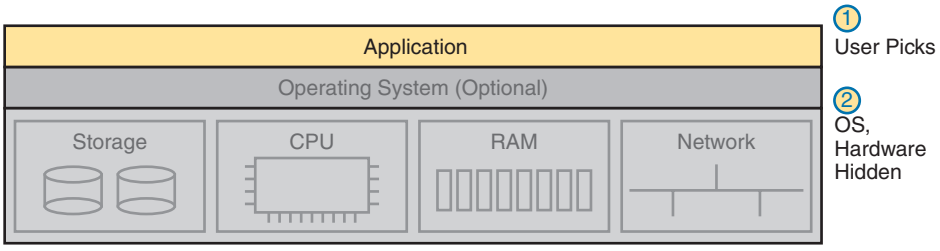



**Figure 20-12** *SaaS Concept*

Many of you have probably used or at least heard of many public SaaS offerings. File storage services like Apple iCloud, Google Drive, Dropbox, and Box are all SaaS offerings. Most online email offerings can be considered SaaS services today. As another example, Microsoft offers its Exchange email server software as a service so that you can have private email services but offered as a service, along with all the other features included with Exchange—without having to license, install, and maintain the Exchange software on some VMs.

### (Development) Platform as a Service

**Platform as a Service (PaaS)** is a development platform prebuilt as a service. A PaaS service is like IaaS in some ways. Both supply the consumer with one or more VMs, with a configurable amount of CPU, RAM, and other resources.

The key difference between PaaS and IaaS is that PaaS includes many more software tools beyond the basic OS. Those tools are useful to a software developer during the software development process. The servers running the application in production do not need the development tools, so those servers would not use the PaaS service. Instead, software developers use PaaS to create systems with tools useful when developing.

A PaaS offering includes a set of development tools, and each PaaS offering has a different combination of tools. PaaS VMs often include an integrated development environment (IDE), a set of related tools that enable the developer to write and test code easily. PaaS VMs include continuous integration tools that allow the developer to update code and have that code automatically tested and integrated into a larger software project. Examples include Google's App Engine PaaS offering (https://cloud.google.com/appengine), the Eclipse integrated development environment (see www.eclipse.org), and the Jenkins continuous integration and automation tool (see https://jenkins.io).

The primary reason to choose one PaaS service over another or a PaaS solution instead of IaaS is the mix of development tools. If you do not have experience as a developer, it can be difficult to tell whether one PaaS service might be better. You can still make some choices about sizing the PaaS VMs, similar to IaaS tools when setting up some PaaS services, as shown in Figure 20-13, but the developer tools included are the key to a PaaS service.

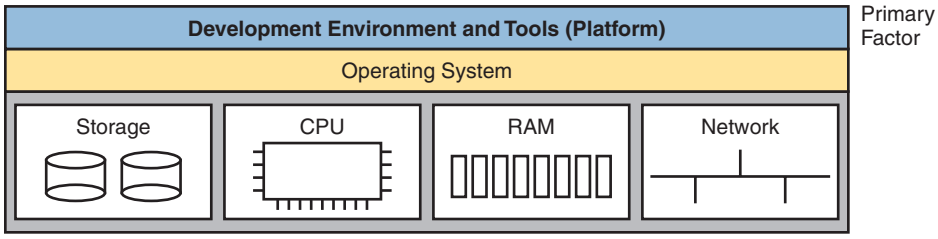



**Figure 20-13** *PaaS Concept*

## Virtual Routing and Forwarding (VRF) Instances

Public cloud services must support many customers concurrently. However, those different customers may, and often do, use overlapping IP subnets. Many companies use private IPv4 networks internally, and the subnets they use for their private and public cloud VMs and containers use addresses from those subnets. Unsurprisingly, those customers use overlapping subnets and addresses.

Overlapping subnets cause problems for a router (or Layer 3 switch) when using traditional conventions. Working through some of the key points:

- Typically, one router has one IP routing table.
- Typically, a router allows only one interface connected to the same subnet.
- If an engineer attempts to connect a second interface to the same subnet, the router will not bring the interface up.
- Data center virtualization software can locate and move VMs and re-program networking, so VMs from multiple customers can exist on one physical server—creating a case of overlapping subnets within that physical server.

As an example, consider customers A and B, whose VMs reside in the server shown in Figure 20-14. The customers use private class A network 10.0.0.0, and both use subnets 10.1.1.0/24 and 10.1.2.0/24. In this example, the virtual switch performs Layer 2 switching only, with no IP routing. Instead, it uses four separate VLANs, as shown. Within that limited scope, no problems exist.

Next, focus on router R1, outside the host. R1 needs to have interfaces configured in the four subnets shown in the figure so it can receive data from those VMs; however, the overlapping subnets confuse R1. For instance, the router and virtual switch define their link as a VLAN trunk. The router uses a router-on-a-stick (ROAS) configuration as shown in the figure (see Volume 1's Chapter 18, "IP Routing in the LAN," for more detail). That configuration gives router R1 an interface with an IP address in all four subnets shown inside the server.

One set of R1 interfaces would fail, however. If R1 were to configure the statements on the left first and then the ones on the right, IOS would not bring up the two interfaces on the right side of the figure due to the overlapping subnets implied by the **ip address** subcommands.

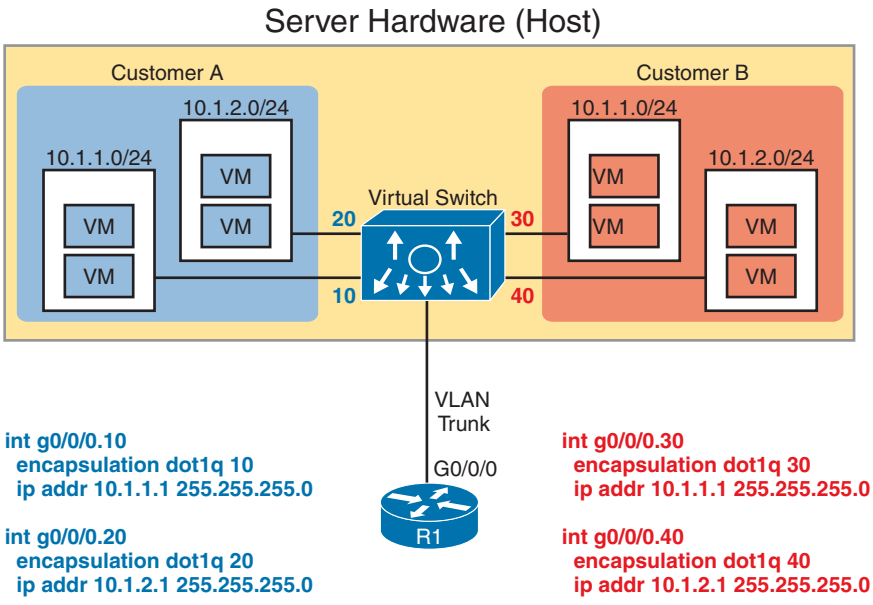



**Figure 20-14** *Overlapping Subnets with Two Customers' VMs on the Same Server*

**Virtual Routing and Forwarding (VRF)** instances solve this problem by expanding the logic used within a router. VRFs create multiple virtual routers inside a single router or Layer 3 switch. The router configuration associates interfaces and routing protocol neighbors to VRFs, with a separate routing table per VRF. In this case, router R1 would create two VRFs, one for each customer. On the ROAS link, R1 will assign the interfaces for VLANs 10 and 20 into one VRF and the ROAS interfaces for VLANs 30 and 40 into another.

By using a separate IP routing table per VRF, the router can support overlapping subnets by placing them in different VRFs. Figure 20-15 expands Figure 20-14 to represent that concept, adding VRFs to the router. It places two VLAN interfaces in each VRF and shows the per-VRF routing tables to the left and right of the router. Note the exact same subnets in each routing table, now allowed by using VRFs.

You will find a use for VRFs throughout the world of networking. For instance, MPLS, as discussed in Chapter 19, "WAN Architecture," makes use of VRFs. One MPLS provider can support thousands of customers with the same MPLS network, even those using overlapping IP addresses, using VRFs. Summing up some of the critical points about VRFs:



Key Topic

- The VRF must be created via configuration in each device that performs routing (router or Layer 3 switch).
- Each router will have a separate IP routing table for each VRF, each holding routes for only that VRF.
- The configuration assigns each interface to a VRF so that the router places the associated connected route into that VRF's routing table.
- Each VRF has its own routing protocol instance and associated neighbor relationships. The routes learned from those neighbors land in the IP routing table for the VRF.

- The router keeps its original routing table, called the global routing table. The global routing table holds routes related to interfaces and routing protocol neighbors not associated with any VRF.

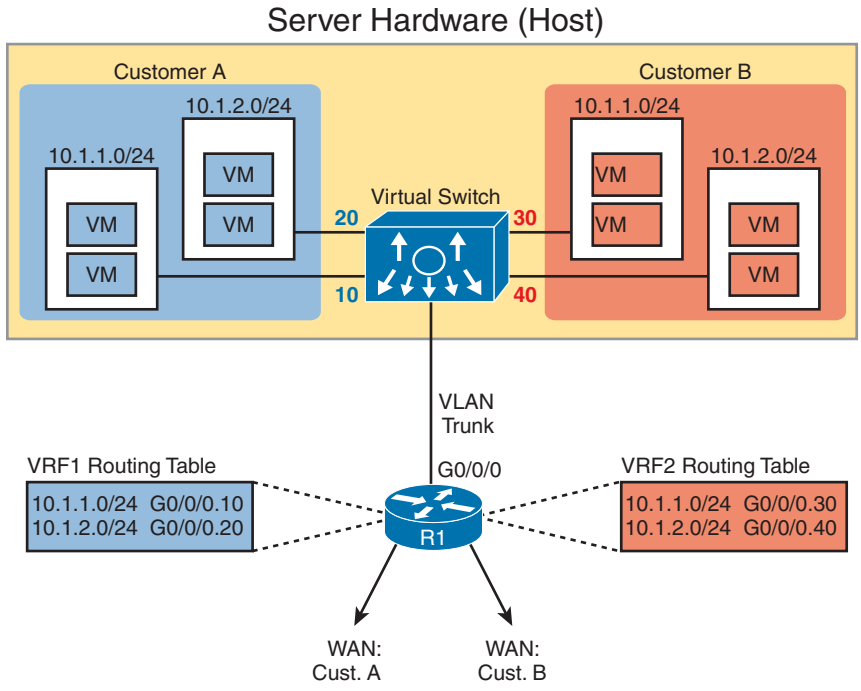



**Figure 20-15** *Overlapping Subnets with Two Customers' VMs on the Same Server*

# WAN Traffic Paths to Reach Cloud Services

This final major section of the chapter focuses on WAN options for public cloud, and the pros and cons of each. This section ignores private cloud for the most part, because using a private cloud—which is internal to an enterprise—has much less of an impact on an enterprise WAN compared to public cloud. With public cloud, a WAN resides between the cloud service and the consumer, so network engineers must consider how best to build a WAN when using public cloud services.

## Enterprise WAN Connections to Public Cloud

Using the Internet to communicate between the enterprise and a public cloud provider is easy and convenient. However, it also has some negatives. This first section describes the basics and points out the issues, which leads to some reasons why using other WAN connections may be preferred.

### Accessing Public Cloud Services Using the Internet

Consider a common workflow after moving an internal application to the public cloud as depicted in Figure 20-16, while using the Internet for connectivity. The cloud provider's services catalog can be reached by enterprise personnel, over the Internet, as shown at step 1. After the desired services—for instance, some VMs for an IaaS service—are chosen,

the cloud provider (step 2) instantiates the VMs. Then, not shown as a step in the figure, the VMs are customized to run the app that was formerly running inside the enterprise's data center.

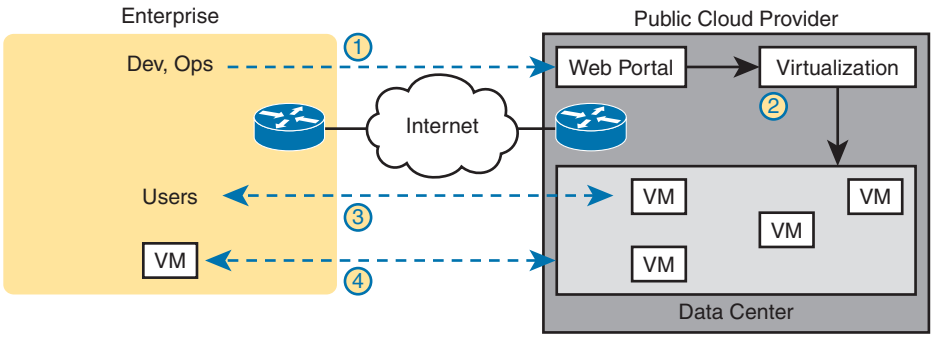



**Figure 20-16** *Accessing a Public Cloud Service Using the Internet*

With the application running in the cloud, enterprise users need to access it. Step 3 shows users communicating with the application, but with the traffic flowing over the Internet. Additionally, many apps send data to other apps—some of which might still reside in the enterprise (as shown in step 4). For instance, you might use authentication services on an internal server. So, at step 4, any application communication between VMs hosted in the cloud to/from VMs hosted inside the enterprise must also occur.

### Pros and Cons with Connecting to Public Cloud with Internet

Using the Internet to connect from the enterprise to the public cloud has several advantages. Every enterprise and public cloud provider already connects to the Internet. As a result, using Internet connectivity greatly speeds migration to public cloud. The following list summarizes some good reasons to use the Internet as the WAN connection to a public cloud service:

**Agility:** An enterprise can get started using public cloud without waiting to order a private WAN connection to the cloud provider because cloud providers support Internet connectivity.

**Migration:** An enterprise can switch its workload from one cloud provider to another more easily because cloud providers all connect to the Internet.

**Remote work:** Remote workers already use the Internet, so accessing SaaS apps available over the Internet works well.

While using the Internet for public cloud has some advantages, it has some negatives as well, including the following:

Key Topic

**Security:** The Internet is less secure than private WAN connections in that a "man in the middle" can attempt to read the contents of data that passes to/from the public cloud.

**Capacity:** Moving an internal application to the public cloud increases network traffic, so consider whether the enterprise's Internet links can handle the additional load.

**Quality of service (QoS):** The Internet does not provide QoS, whereas private WANs can. Using the Internet may result in a worse user experience than desired because of higher delay (latency), jitter, and packet loss.

**No WAN SLA:** ISPs typically will not provide a service-level agreement (SLA) for WAN performance and availability to all destinations of a network. Private WAN service providers are much more likely to offer performance and availability SLAs.

This list of concerns does not mean that an enterprise cannot use the Internet to access its public cloud services. It does mean that it should consider the pros and cons of each WAN option.

### Private WAN and Internet VPN Access to Public Cloud

The NIST definition for cloud computing lists broad network access as one of the five main criteria. In the case of public cloud, that often means supporting a variety of WAN connections, including the most common enterprise WAN technologies. Basically, an enterprise can connect to a public cloud provider with WAN technologies discussed in this book. Figure 20-17 breaks it down into two broad categories.

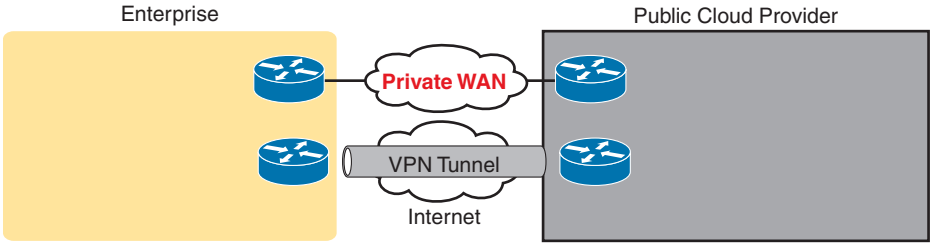



**Figure 20-17** *Using Private WAN to a Public Cloud: Security, QoS, Capacity, Reporting*

To create a VPN tunnel between the enterprise and the cloud provider, use the same VPN features discussed in Chapter 19 in this volume. The cloud provider can offer a VPN service, with the cloud provider implementing one end of the tunnel and the customer implementing the other. Or the enterprise can use its own router inside the cloud provider's network—a virtual router running as a VM—and configure VPN services on that router. Cisco makes the Cloud Services Router (CSR to do exactly that: to be a router, but a router that runs as a VM in a cloud service, controlled by the cloud consumer, to do various functions that routers do, including terminating VPNs.

**NOTE** Cisco has introduced the Catalyst 8000v Edge Software, or simply Catalyst 8000v, as a replacement for the CSR. As of the publication date, the CSR was still available.

To make a private Multiprotocol Label Switching (MPLS) VPN or Ethernet WAN connection, the enterprise must work with the cloud and WAN providers. Because cloud providers connect to many customers with private WAN connections, they often have a published set of instructions to follow. When a customer connects using MPLS, the MPLS provider creates an MPLS VPN between the customer and the cloud provider. The same basic process happens with Ethernet WAN services, with one or more Ethernet Virtual Connections (EVCs) created between the public WAN and the enterprise.

**NOTE** Often, the server/virtualization engineers will dictate whether the WAN connection needs to support Layer 2 or Layer 3 connectivity, depending on other factors.

Private WAN connections also require some physical planning. Each of the larger public cloud providers has several large data centers spread around the planet and with prebuilt connection points into the major WAN services to aid the creation of private WAN connections to customers. An enterprise might then look at the cloud provider's documentation and work

with that provider to choose the best place to install the private WAN connection. (Those larger public cloud companies include Amazon Web Services, Google Compute Cloud, Microsoft Azure, and Rackspace if you want to look at their websites for information about their locations.)

### Pros and Cons of Connecting to Cloud with Private WANs

Private WANs overcome some of the issues of using the Internet without a VPN, so working through those issues, consider some of the different WAN options.

First, considering the issue of security, all the private options, including adding a VPN to the existing Internet connection, improve security significantly. An Internet VPN would encrypt the data to keep it private. Private WAN connections with MPLS and Ethernet have traditionally been considered secure without encryption. Still, companies sometimes encrypt data sent over private WAN connections to make the network more secure.

Regarding QoS, using an Internet VPN solution still fails to provide QoS because the Internet does not provide QoS. WAN services like MPLS VPN and Ethernet WANs can. Private WAN providers can apply QoS tools to the traffic as it passes through the service provider's network.

Finally, as for the capacity issue, the concern of planning network capacity exists for every type of WAN. Moving an app away from an internal data center to a public cloud provider requires extra thought and planning.

Several negatives exist for using a private WAN, as you might expect. Installing the new private WAN connections takes time, delaying when a company starts in cloud computing. Private WANs typically cost more than using the Internet. If a company uses a WAN connection to one cloud provider, migrating to a new cloud provider can require another round of private WAN installation, again delaying work projects. Using the Internet (with or without VPN) would make that migration much easier, but the next section shows a strong compromise solution.

### Intercloud Exchanges

Public cloud computing also introduces a whole new level of competition because cloud consumers can move their workload from one cloud provider to another. Moving the workload takes some effort, for reasons beyond the scope of this book. (Suffice it to say that most cloud providers differ in how they implement services.) But enterprises can migrate their workload from one cloud provider to another, for many reasons, including looking for a less expensive cloud provider.



Now, focus on the networking connections again. Using a private WAN for the cloud adds setup time and expense. However, an intercloud exchange (or simply an intercloud) reduces the time when you migrate cloud providers.

The term *intercloud exchange* refers to any company that creates a private network as a service. First, an intercloud exchange connects to multiple cloud providers on one side. On the other side, the intercloud connects to cloud consumers. Figure 20-18 shows the idea.

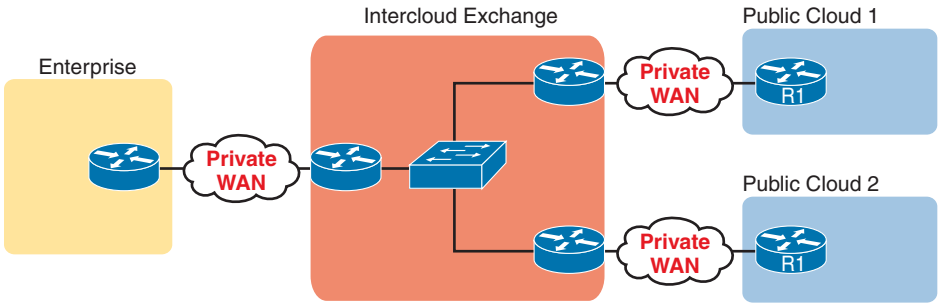



**Figure 20-18** *Permanent Private WAN Connection to an Intercloud Exchange*

With access links to all cloud providers and to the cloud customer, the intercloud exchange can configure a private WAN connection to specific cloud provider sites. Later, if the consumer wants to migrate to use a different cloud provider, the consumer keeps the same private WAN links. That consumer just has to ask the intercloud exchange to reconfigure to set up new private WAN connections to the new cloud provider.

As for pros and cons, with an intercloud exchange, you get the same benefits as when connecting with a private WAN connection to a public cloud, but with the additional pro of easier migration to a new cloud provider. The main con is that using an intercloud exchange introduces another company.

### Summarizing the Pros and Cons of Public Cloud WAN Options

Table 20-2 summarizes some of these key pros and cons for the public WAN options for cloud computing, for study and reference.

Key Topic

**Table 20-2** Comparison of Public Cloud WAN Options

| | Internet | Internet VPN | MPLS VPN | Ethernet WAN | Intercloud Exchange |
|---|---|---|---|---|---|
| Makes data private | No | Yes | Yes | Yes | Yes |
| Supports QoS | No | No | Yes | Yes | Yes |
| Requires capacity planning | Yes | Yes | Yes | Yes | Yes |
| Eases migration to a new provider | Yes | Yes | No | No | Yes |
| Speeds initial installation | Yes | Yes | No | No | No |

## Understanding Cloud Management

**Cloud management** is a popular method for managing your cloud-based or hosted infrastructure. There are multiple methods of cloud management. Cisco has recently added it to the CCNA blueprint, and this section covers a few of the most common methods when managing Cisco devices from a cloud perspective. The most common method is not that unfamiliar from what you have already learned, and that is to use the console capabilities of the virtual device to log in to the CLI and issue commands as you would if the device were a physical device. In this use case, a console or a serial port is mapped to the virtual device and you simply telnet or SSH to the IP of the virtual device to be put right into the CLI.

From here, you simply issue any CLI configuration commands to configure the cloud-based device.

Alternatively, there are two methods that require some experience inside the hypervisor environment such as VMware ESXi, Microsoft Hyper-V, Citrix XenServer, or KVM. Using the **Cisco Catalyst 8000V** router as an example, when the router VM first boots up, you have, by default, 5 seconds to select the console option you want to use.

**NOTE** The Cisco 8000V is the latest version of the Cisco virtual router family. Previously, there was the Cisco CSR1000v, which was deployed in a similar fashion but is now End-of-Sale.

One hypervisor option is called the *virtual console*, and it enables you to click into the virtual machine within the hypervisor GUI and access the CLI that way. This is also called the hypervisor virtual VGA console. It is similar to how a virtual server that is running as a VM would be accessed. By default, this is the method the Cisco 8000V router VM boots in to.

Another hypervisor option for the Cisco 8000V is called the *serial console*. The serial console option maps to a serial port on the Cisco 8000V router VM. This enables you to use telnet to access the specific port number you specify when configuring the serial console option. The telnet syntax that follows is for a VM and UNIX xTerm terminal:

```
VM: telnet://host-ipaddress:portnumber

From a UNIX xTerm terminal: telnet host-ipaddress portnumber
```

**NOTE** Telnet is used only to initially configure the Cisco 8000V router VM when using serial console. After the device is fully configured, it is highly recommended to enable SSH to ensure enhanced security and privacy because telnet utilizes clear text data in its communications.

Additionally, it is important to understand the benefits of cloud management compared to the traditional method of managing devices on a one-by-one basis. The following list highlights some of the key advantages of using cloud management and their associated definitions:

- **Centralized management:** Provides centralized control and visibility over network devices and services. This makes it easier to manage distributed networks with multiple devices across different locations.
- **Accessibility:** Devices can be accessed from anywhere in the world via an Internet connection. This enables the remote management capability of the network infrastructure. This is especially relevant for remote network operations staff.
- **Scale:** Environments can grow and shrink their scale from a small deployment to be very large and can do this very rapidly without having a constraint on resources such as storage, compute, and bandwidth. This is what "cloud elasticity" refers to.

- **Automation:** Cloud management platforms often include automation capabilities that streamline repetitive tasks such as configuration management, provisioning, and troubleshooting. This reduces the manual interaction for network management, increases operational efficiency, and lowers the risk of human error.
- **Security:** Cloud management platforms typically include built-in security mechanisms to protect network data and infrastructure from cyber threats. Role-based access control, encryption, and security monitoring are among the most common features.
- **Lower cost of ownership:** Cloud-based infrastructure can eliminate the need for some on-premises hardware and software, which could potentially reduce capital expenditures, support, and operational costs associated with managing on-premise network infrastructure.

Cisco also offers a cloud managed solution called *Meraki*. Meraki leverages completely cloud-based management for onsite devices. All of the key advantages and benefits of cloud management mentioned previously also apply to Meraki. The centralized cloud-based web portal to manage Meraki devices is called the **Meraki Dashboard**. The Meraki Dashboard provides an informative and intuitive view of your entire Meraki network, along with the current and historical status of all managed devices. The following are some advantages of Meraki Dashboard:

- **Zero touch provisioning (ZTP):** You can create predefined locations and install devices in a remote location and they will check in with the Meraki Dashboard and instantiate themselves by downloading their appropriate software images and configuration without needing an on-site expert to configure the devices. This saves significant costs due to lowered travel expenses and lowers the risk of misconfigured devices.
- **Topology:** This is a live topological view of the network and how devices are connected to each other. This is an excellent way to see what is connected in your environment and how it is performing. The Topology view can be carved out in layers to see certain aspects of the network, with more focus on the devices you are interested in inspecting.
- **Path visualization:** Allows for visual network traffic monitoring. This includes device path and directional traffic flows and can be seen in the dashboard.
- **Single-click into devices:** Enables you to see performance and management options.
- **Color-coded states of operation:** Simplifies monitoring and can give a quick view into the state of the network.

Figure 20-19 showcases the default client page view when logging into the Meraki Dashboard. It can be seen that there is a tremendous amount of information that is available without clicking or drilling down into any other page or link. Looking at the top of the page, we can see that there are some tiles that show the status of the Uplinks, WAN Appliances, Switches, and Access Points. All are green and show online. From there you can look further down and see the client usage graph and pie chart that highlights the bandwidth usage and details about the applications operating within the network.

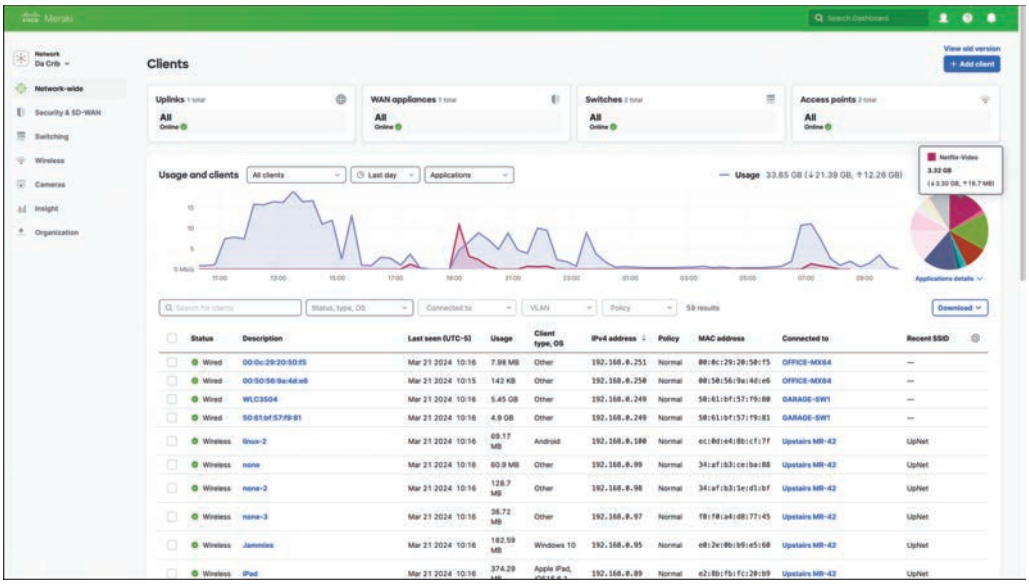



**Figure 20-19** *Default View of Meraki Dashboard*

What is useful about this view is that you can hover your mouse over the graph or pie chart to display more information without navigating away from this default view. For example, you can see that there are 3.32 GB of Netflix-Video traffic running over the network on the last day, along with the corresponding upload and download statistics.

Moving to the bottom section of the screen, you can see some very helpful pieces of information, such as the device status, description, when the device was last seen on the network, the client usage, client operation system, IP address, policy, MAC address, uplink information, and most recent SSID. The sheer amount of information that is accessible from just logging into the dashboard is pretty remarkable. Furthermore, you can select many other values to view on this screen by clicking the gear and selecting additional data points. In many traditional networks, you would have to log into a standalone **network management system (NMS)** or multiple devices in the network to gain the amount of data that you can access within the Meraki Dashboard. This is one of the major benefits of centralized cloud management.

From a troubleshooting perspective, Topology and Path Visualization are some of the handiest tools within Meraki. Looking at this sample network you can see some interesting information by simply clicking the Topology view within the dashboard. You can also see that when you hover over devices or links with your mouse, you can get more detailed information about each component. In the case illustrated in Figure 20-20, you can see that there is an issue with the GARAGE-SW1 device and that there is a high amount of CRC errors on Port 3. This example shows the Layer 2 Link layer, although you can see that Layer 3 networking information is available as well.

There are many advantages to using cloud-management versus traditional methods, covered throughout this section of the chapter. Table 20-3 summarizes and compares some of the most common differences between the two.

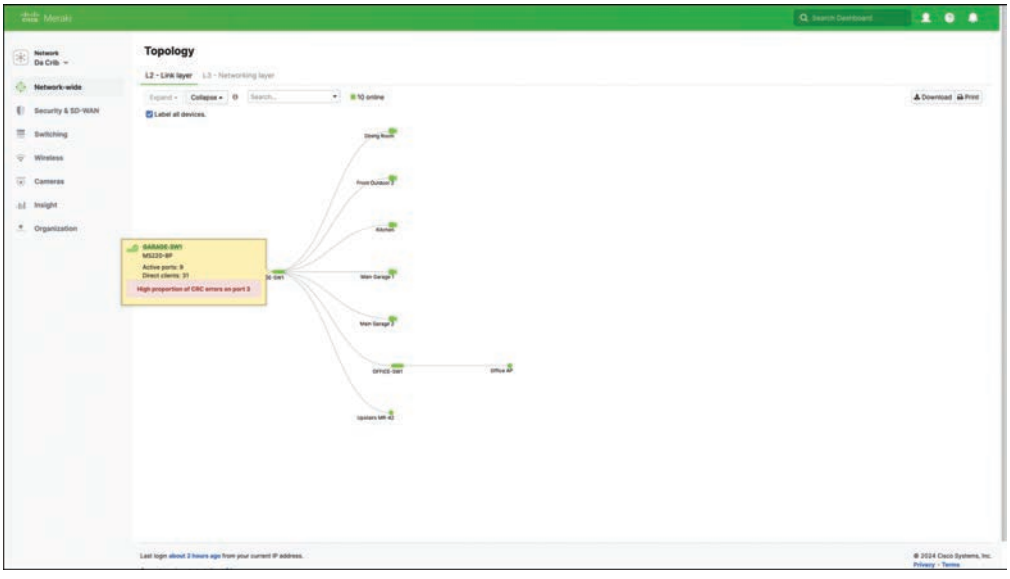



**Figure 20-20** *Topology View of Meraki Dashboard*

Key Topic

**Table 20-3** Comparing Cloud Management and Traditional Management

| Use Case | Cloud Management | Traditional Management |
|---|---|---|
| Centralized device management | Typically, a single point of management for all devices | Unless using a third-party NMS or tool, devices are typically managed individually |
| Scale and speed of deployment | Zero touch provisioning (ZTP) | Manual or automated provisioning through scripting or third-party tools |
| Security policy and enforcement | Security polices can be applied to multiple devices within a single central dashboard | Unless using third-party tools or automation, typically security policies have to be applied on a device-by-device basis |
| Troubleshooting | Centralized management makes troubleshooting much more effective due to the ability to see the whole network in a single dashboard | Typically is more difficult and time-consuming from having to troubleshoot on a device-by-device basis |
| Monitoring and reporting | Cloud management enables you to monitor your network centrally to see and report on issues across all devices | Traditional networks monitor or report on devices using a third-party tool, which may require you to correlate the data |

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 20-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 20-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, website |

## Review All the Key Topics

Key Topic

**Table 20-5** Key Topics for Chapter 20

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 20-3 | Organization of applications, on a VM, on an OS, with a hypervisor allocating and managing the host hardware | 442 |
| Figure 20-4 | Virtual switch concept | 443 |
| Figure 20-5 | Software container architecture | 445 |
| List | Definition of cloud computing (paraphrased) based on the NIST standard | 448 |
| Figure 20-10 | Organization and concepts for an IaaS service | 452 |
| Figure 20-12 | Organization and concepts for an SaaS service | 453 |
| Figure 20-13 | Organization and concepts for a PaaS service | 454 |
| List | VRF features | 455 |
| List | Cons for using the Internet to access public WAN services | 457 |
| Table 20-2 | Summary of pros and cons with different public cloud WAN access options | 460 |
| Table 20-3 | Comparing cloud management versus traditional management | 464 |



## Key Terms You Should Know

Cisco Catalyst 8000V, cloud management, cloud services catalog, container, container image, host (context: DC), hypervisor, Infrastructure as a Service (IaaS), Meraki Dashboard, network management system (NMS), on-demand self-service, Platform as a Service (PaaS), private cloud, public cloud, rapid elasticity, resource pooling, Software as a Service (SaaS), Unified Computing System (UCS), virtual CPU (vCPU), virtual machine (VM), virtual NIC (vNIC), Virtual Routing and Forwarding (VRF), virtual switch (vSwitch), zero touch provisioning (ZTP)



# Part V Review

Keep track of your part review progress with the checklist shown in Table P5-1. Details on each task follow the table.

**Table P5-1** Part V Part Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |

## Repeat All DIKTA Questions

For this task, use the PTP software to answer the "Do I Know This Already?" questions again for the chapters in this part of the book.

## Answer Part Review Questions

For this task, use PTP to answer the Part Review questions for this part of the book.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.

*This page intentionally left blank*



Part VI of this book includes most of the network automation topics from the CCNA blueprint; however, this part includes as much discussion of how Cisco and others have changed the way networks work to enable better automation as it discusses tools and processes to automate networks.

Chapters 21 and 22 examine a wide range of products and architectures that also enable better operations and automation. Chapter 21 discusses how controllers can separate out part of the work formerly done by networking devices. The chapter shows the advantages of these new controller-based models and details a few examples. Chapter 22 then goes on to give more detail about Cisco Software-Defined Access (Cisco SD-Access), a controller-based networking approach to building enterprise campus networks.

Chapters 23 and 24 discuss a few more specific details about network automation. Controllers typically include REST APIs and often return data to automation programs in the form of formatted data like JSON. Chapter 23 introduces these concepts. Chapter 24 then moves on to discuss IT automation tools, specifically Ansible and Terraform.

# Part VI

## Network Automation

**Chapter 21:** Introduction to Controller-Based Networking

**Chapter 22:** Cisco Software-Defined Access (Cisco SD-Access)

**Chapter 23:** Understanding REST and JSON

**Chapter 24:** Understanding Ansible and Terraform

**Part VI Review**



CHAPTER 21

# Introduction to Controller-Based Networking

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.1 Explain the role and function of network components**
    - **1.1.f Endpoints**
    - **1.1.g Servers**
  - **1.2 Describe characteristics of network topology architectures**
    - **1.2.c Spine-leaf**
- **6.0 Automation and Programmability**
  - **6.1 Explain how automation impacts network management**
  - **6.2 Compare traditional networks with controller-based networking**
  - **6.3 Describe controller-based, software defined architecture (overlay, underlay, and fabric)**
    - **6.3.a Separation of control plane and data plane**
    - **6.3.b Northbound and Southbound APIs**

The CCNA certification focuses on the traditional model for operating and controlling networks, a model that has existed for decades. You understand protocols that the devices use, along with the commands that can customize how those protocols operate. Then you plan and implement distributed configuration to the devices, device by device, to implement the network.

A new network operational model was introduced in the 2010s: Software-Defined Networking (SDN). SDN makes use of a controller that centralizes some network functions. The controller also creates many new capabilities to operate networks differently; in particular, controllers enable programs to automatically configure and operate networks through the power of application programming interfaces (APIs).

With traditional networking, the network engineer configured the various devices and changes requiring a long timeframe to plan and implement changes. With controller-based networking and SDN, network engineers and operators can implement changes more quickly, with better consistency, and often with better operational practices.

This chapter introduces the concepts of network programmability and SDN. Note that the topic area is large, with this chapter providing enough detail for you to understand the basics and to be ready for the other three chapters in this part.

The first major section of this chapter introduces the basic concepts of data and control planes, along with controllers and the related architecture. The second section then shows separate product examples of network programmability using controllers, all of which use different methods to implement networking features. The last section takes a little more exam-specific approach to these topics, comparing the benefits of traditional networking with the benefits of controller-based networking.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 21-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| SDN and Controller-Based Networks | 1–3 |
| Examples of Network Programmability and SDN | 4, 5 |
| Comparing Traditional Versus Controller-Based Networks | 6 |

1. A Layer 2 switch examines a frame's destination MAC address and chooses to forward that frame out port G0/1 only. That action occurs as part of which plane of the switch?
   - **a.** Data plane
   - **b.** Management plane
   - **c.** Control plane
   - **d.** Table plane
2. A router uses OSPF to learn routes and adds those to the IPv4 routing table. That action occurs as part of which plane of the router?
   - **a.** Data plane
   - **b.** Management plane
   - **c.** Control plane
   - **d.** Table plane
3. A network uses an SDN architecture with switches and a centralized controller. Which of the following terms describes a function or functions expected to be found on the switches but not on the controller?
   - **a.** A northbound interface
   - **b.** A southbound interface
   - **c.** Data plane functions
   - **d.** Control plane functions

4. Which of the following controllers (if any) uses a mostly centralized control plane model?
   a. OpenDaylight Controller
   b. Cisco Application Policy Infrastructure Controller (APIC)
   c. Cisco Catalyst 9800 Series Controller
   d. None of these controllers use a mostly centralized control plane.
5. To which types of nodes should an ACI leaf switch connect in a typical single-site design? (Choose two answers.)
   a. All of the other leaf switches
   b. A subset of the spine switches
   c. All of the spine switches
   d. Some of the endpoints
   e. None of the endpoints
6. Which answers list an advantage of controller-based networks versus traditional networks? (Choose two answers.)
   a. The ability to configure the features for the network rather than per device
   b. The ability to have forwarding tables at each device
   c. Programmatic APIs available per device
   d. More consistent device configuration

# Foundation Topics

## SDN and Controller-Based Networks

Networking devices forward data in the form of messages, typically data-link frames like Ethernet frames. You have learned about how switches and routers do that forwarding for the entire length of preparing for the CCNA exam.

Network programmability and **Software-Defined Networking (SDN)** take those ideas, analyze the pieces, find ways to improve them for today's needs, and reassemble those ideas into a new way of making networks work. At the end of that rearrangement, the devices in the network still forward messages, but the how and why have changed.

This first major section explains the most central concepts of SDN and network programmability. It starts by breaking down some of the components of what exists in traditional networking devices. Then this section explains how some centralized controller software, called a controller, creates an architecture for easier programmatic control of a network.

### The Data, Control, and Management Planes

Stop and think about what networking devices do. What does a router do? What does a switch do?

Many ideas should come to mind. For instance, routers and switches physically connect to each other with cables, and with wireless, to create networks. They forward messages: switches forward Ethernet frames, and routers forward packets. They use many different protocols to learn useful information such as routing protocols for learning network layer routes.

Everything that networking devices do can be categorized as being in a particular plane. This section takes those familiar facts about how networking devices work and describes the three planes most often used to describe how network programmability works: the data plane, the control plane, and the management plane.

## The Data Plane

The term **data plane** refers to the tasks that a networking device does to forward a message. In other words, anything to do with receiving data, processing it, and forwarding that same data—whether you call the data a frame, a packet, or, more generically, a message—is part of the data plane.

For example, think about how routers forward IP packets, as shown in Figure 21-1. If you focus on the Layer 3 logic for a moment, the host sends the packet (Step 1) to its default router, R1. R1 does some processing on the received packet, makes a forwarding (routing) decision, and forwards the packet (Step 2). Routers R3 and R4 also receive, process, and forward the packet (Steps 3 and 4).

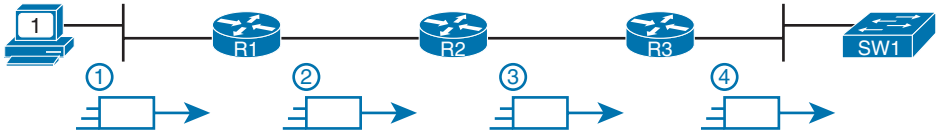



**Figure 21-1** *Data Plane Processing on Routers: Basics*

Now broaden your thinking for a moment and try to think of everything a router or switch might do when receiving, processing, and forwarding a message. Of course, the forwarding decision is part of the logic; in fact, the data plane is often called the *forwarding plane*. But think beyond matching the destination address to a table. For perspective, the following list details some of the more common actions that a networking device does that fit into the data plane:

Key Topic

- De-encapsulating and re-encapsulating a packet in a data-link frame (routers, Layer 3 switches)
- Adding or removing an 802.1Q trunking header (routers and switches)
- Matching an Ethernet frame's destination Media Access Control (MAC) address to the MAC address table (Layer 2 switches)
- Matching an IP packet's destination IP address to the IP routing table (routers, Layer 3 switches)
- Encrypting the data and adding a new IP header (for virtual private network [VPN] processing)
- Changing the source or destination IP address (for Network Address Translation [NAT] processing)
- Discarding a message due to a filter (access control lists [ACLs], port security)

All the items in the list make up the data plane, because the data plane includes all actions done per message.

## The Control Plane

Next, take a moment to ponder the kinds of information that the data plane needs to know beforehand so that it can work properly. For instance, routers need IP routes in a routing table before the data plane can forward packets. Layer 2 switches need entries in a MAC address table before they can forward Ethernet frames out the one best port to reach the destination. Switches must use Spanning Tree Protocol (STP) to limit which interfaces can be used for forwarding so that the data plane works well and does not loop frames forever.

From one perspective, the information supplied to the data plane controls what the data plane does. For instance, a router needs a route that matches a packet's destination address for the router to know how to route (forward) the packet. When a router's data plane tries to match the routing table and finds no matching route, the router discards the packet. And what controls the contents of the routing table? Various control plane processes.

The term **control plane** refers to any action that controls the data plane. Most of these actions have to do with creating the tables used by the data plane, tables like the IP routing table, an IP Address Resolution Protocol (ARP) table, a switch MAC address table, and so on. By adding to, removing, and changing entries to the tables used by the data plane, the control plane processes control what the data plane does. You already know about many control plane protocols—for instance, all the IP routing protocols.

Traditional networks use both a distributed data plane and a distributed control plane. In other words, each device has a data plane and a control plane, and the network distributes those functions into each individual device, as shown in the example in Figure 21-2.

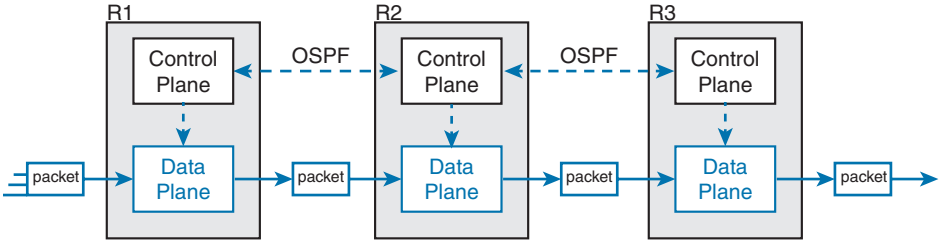



**Figure 21-2** *Control and Data Planes of Routers—Conceptual*

In the figure, Open Shortest Path First (OSPF), the control plane protocol, runs on each router (that is, it is distributed among all the routers). OSPF on each router then adds to, removes from, and changes the IP routing table on each router. Once populated with useful routes, the data plane's IP routing table on each router can forward incoming packets, as shown from left to right across the bottom of the figure. The following list includes many of the more common control plane protocols:

Key Topic

- Routing protocols (OSPF, Enhanced Interior Gateway Routing Protocol [EIGRP], Routing Information Protocol [RIP], Border Gateway Protocol [BGP])
- IPv4 ARP

---

Answers to the "Do I Know This Already?" quiz:

**1** A **2** C **3** C **4** A **5** C, D **6** A, D

- IPv6 Neighbor Discovery Protocol (NDP)
- Switch MAC learning
- STP

Without the protocols and activities of the control plane, the data plane of traditional networking devices would not function well. Routers would be mostly useless without routes learned by a routing protocol. Without learning MAC table entries, a switch could still forward unicasts by flooding them, but doing that for all frames would create much more load on the local-area network (LAN) compared to normal switch operations. So the data plane must rely on the control plane to provide useful information.

### The Management Plane

The control plane performs overhead tasks that directly impact the behavior of the data plane. The **management plane** performs overhead work as well, but that work does not directly impact the data plane. Instead, the management plane includes protocols that allow network engineers to manage the devices.

Telnet and Secure Shell (SSH) are two of the most obvious management plane protocols. To emphasize the difference with control plane protocols, think about two routers: one configured to allow Telnet and SSH into the router and one that does not. Both could still be running a routing protocol and routing packets, whether or not they support Telnet and SSH.

Figure 21-3 extends the example shown in Figure 21-2 by now showing the management plane, with several management plane protocols.

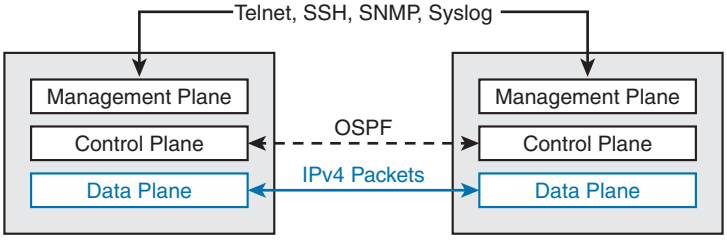



**Figure 21-3** *Management Plane for Configuration of Control and Data Planes*

### Cisco Switch Data Plane Internals

To better understand SDN and network programmability, it helps to think about the internals of switches. This next topic does just that.

From the very first days of devices called LAN switches, switches had to use specialized hardware to forward frames, because of the large number of frames per second (fps) required. To get a sense for the volume of frames a switch must be able to forward, consider the minimum frame size of an Ethernet frame, the number of ports on a switch, and the speeds of the ports; even low-end switches need to be able to forward millions of frames per second. For example, if a switch manufacturer wanted to figure out how fast its data plane needed to be in a new access layer switch with 24 ports, it might work through this bit of math:

- The switch has 24 ports.
- Each port runs at 1 Gbps.

- For this analysis, assume frames 125 bytes in length (to make the math easier, because each frame is 1000 bits long).
- With a 1000-bit-long frame and a speed of 1,000,000,000 bits/second, a port can send 1,000,000 frames per second (fps).
- Use full duplex on all ports, so the switch can expect to receive on all 24 ports at the same time.
- Result: Each port would be receiving 1,000,000 fps, for 24 million fps total, so the switch data plane would need to be ready to process 24 million fps.

Although 24 million fps may seem like a lot, the goal here is not to put an absolute number on how fast the data plane of a switch needs to be for any given era of switching technology. Instead, from their first introduction into the marketplace in the mid-1990s, LAN switches needed a faster data plane than a generalized CPU could process in software. As a result, hardware switches have always had specialized hardware to perform data plane processing.

First, the switching logic occurs not in the CPU with software, but in an **application-specific integrated circuit (ASIC)**. An ASIC is a chip built for specific purposes, such as for message processing in a networking device.

Second, the ASIC needs to perform table lookup in the MAC address table, so for fast table lookup, the switch uses a specialized type of memory to store the equivalent of the MAC address table: **ternary content-addressable memory (TCAM)**. TCAM memory does not require the ASIC to execute loops through an algorithm to search the table. Instead, the ASIC can feed the fields to be matched, like a MAC address value, into the TCAM, and the TCAM returns the matching table entry, without a need to run a search algorithm.

Note that a switch still has a general-purpose CPU and RAM as well, as shown in Figure 21-4. IOS runs in the CPU and uses RAM. Most of the control and management plane functions run in IOS. The data plane function (and the control plane function of MAC learning) happens in the ASIC.

Key Topic

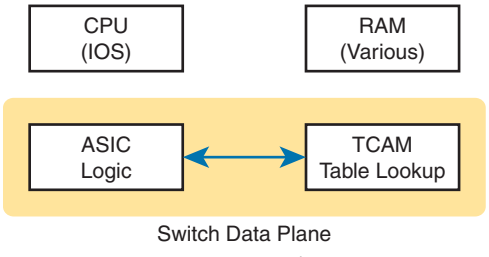



**Figure 21-4** *Key Internal Processing Points in a Typical Switch*

Note that some routers also use hardware for data plane functions, for the same kinds of reasons that switches use hardware. (For instance, check out the Cisco Quantum Flow Processor (QFP) for interesting reading about hardware data plane forwarding in Cisco routers.) The ideas of a hardware data plane in routers are similar to those in switches: use a purpose-built ASIC for the forwarding logic, and TCAM to store the required tables for fast table lookup.

## Controllers and Software Defined Architecture

New approaches to networking emerged in the 2010s, approaches that change where some of the control plane functions occur. Many of those approaches move parts of the control plane work into software that runs as a centralized application called a *controller*. This next topic looks at controller concepts, and the interfaces to the devices that sit below the controller and to any programs that use the controller.

**NOTE** The term *Software Defined Networking (SDN)* became common in the 2010s to refer to the types of controller-based networks described in the next few pages. More often today you might see terms like *software defined architecture* or **controller-based networking**.

### Controllers and Centralized Control

Most traditional control plane processes use a distributed architecture. For example, each router runs its own OSPF routing protocol process. To do their work, those distributed control plane processes use messages to communicate with each other, like OSPF protocol messages between routers. As a result, traditional networks are said to use a **distributed control plane**.

The people who created today's control plane concepts, like STP, OSPF, EIGRP, and so on, could have chosen to use a **centralized control plane**. That is, they could have put the logic in one place, running on one device, or on a server. Then the centralized software could have used protocol messages to learn information from the devices, but with all the processing of the information at a centralized location. But they instead chose a distributed architecture.

There are pros and cons to using distributed and centralized architectures to do any function in a network. Many control plane functions have a long history of working well with a distributed architecture. However, a centralized application can be easier to write than a distributed application, because the centralized application has all the data gathered into one place. And this emerging world of software defined architectures often uses a centralized architecture, with a centralized control plane, with its foundations in a service called a controller.

A *controller*, or *SDN controller*, centralizes the control of the networking devices. The degree of control—and the type of control—varies widely. For instance, the controller can perform all control plane functions, replacing the devices' distributed control plane. Alternately, the controller can simply be aware of the ongoing work of the distributed data, control, and management planes on the devices, without changing how those operate. And the list goes on, with many variations.

To better understand the idea of a controller, consider one specific case as shown in Figure 21-5, in which one SDN controller centralizes all important control plane functions. First, the controller sits anywhere in the network that has IP reachability to the devices in the network. Each of the network devices still has a data plane; however, note that not one of the devices has a control plane. In the variation of SDN as shown in Figure 21-5, the controller directly programs the data plane entries into each device's tables. The networking devices do not populate their forwarding tables with traditional distributed control plane processes.

Key Topic

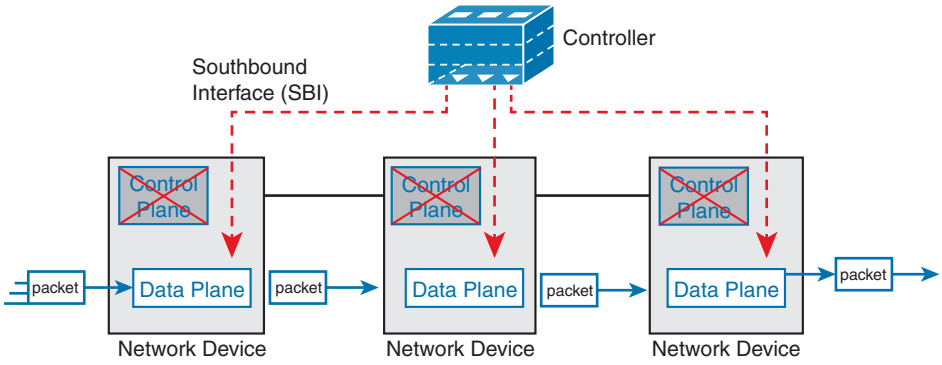



**Figure 21-5** *Centralized Control Plane and a Distributed Data Plane*

**NOTE** Figure 21-5 shows the model used by one of the original SDN implementations that uses an industry standard called **OpenFlow**.

Figure 21-5 shows one model for network programmability and SDN, but not all. The figure does give us a great backdrop to discuss a few more important basic concepts, in particular, the idea of a southbound interface (SBI) and northbound interface (NBI).

## The Southbound Interface

In a controller-based network architecture, the controller needs to communicate to the networking devices. In most network drawings and architecture drawings, those network devices typically sit below the controller, as shown in Figure 21-5. There is an interface between the controller and those devices, and given its location at the bottom part of drawings, the interface came to be known as the **southbound interface (SBI)**, as labeled in Figure 21-5.

**NOTE** In the context of this chapter's discussion of SDN, the word *interface* (including in the names of SBI, NBI, and API) refers to software interfaces unless otherwise noted.

Several different options exist for the SBI. The overall goal is network programmability, so the interface moves away from being only a protocol. An SBI often includes a protocol, so that the controller and devices can communicate, but it often includes an **application programming interface (API)**. An API is a method for one application (program) to exchange data with another application. Rearranging the words to describe the idea, an API is an interface to an application program. Programs process data, so an API lets two programs exchange data. While a protocol exists as a document, often from a standards body, an API often exists as usable code—functions, variables, and data structures—that can be used by one program to communicate and copy structured data between the programs across a network.

So, back to the term *SBI*: it is an interface between a program (the controller) and a program (on the networking device) that lets the two programs communicate, with one goal being to allow the controller to program the data plane forwarding tables of the networking device.

Unsurprisingly, in a network architecture meant to enable network programmability, the capabilities of the SBIs and their APIs tell us a lot about what that particular architecture can and cannot do. For instance, some controllers might support one or a few SBIs, for a specific purpose, while others might support many more SBIs, allowing a choice of SBIs to use. The comparisons of SBIs go far beyond this chapter, but it does help to think about a few; the second major section gives three sample architectures that happen to show three separate SBIs, specifically:

- OpenFlow (from the ONF; www.opennetworking.org)
- OpFlex (from Cisco; used with ACI)
- CLI (Telnet/SSH) and SNMP, and NETCONF (used with Cisco Software-Defined Access)

### The Northbound Interface

Think about the programming required at the controller related to the example in Figure 21-5. The figure focuses on the fact that the controller can add entries to the networking device's forwarding tables; however, how does the controller know what to add? How does it choose? What kind of information would your program need to gather before it could attempt to add something like MAC table entries or IP routes to a network? You might think of these:

- A list of all the devices in the network
- The capabilities of each device
- The interfaces/ports on each device
- The current state of each port
- The topology—which devices connect to which, over which interfaces
- Device configuration—IP addresses, VLANs, and so on as configured on the devices

A controller does much of the work needed for the control plane in a centralized control model. It gathers all sorts of useful information about the network, like the items in the preceding list. The controller itself can create a centralized repository of all this useful information about the network.

Key Topic

A controller's **northbound interface (NBI)** opens the controller so its data and functions can be used by other programs, enabling network programmability, with much quicker development. Programs can pull information from the controller, using the controller's APIs. The NBIs also enable programs to use the controller's capabilities to program flows into the devices using the controller's SBIs.

To see where the NBI resides, first think about the controller itself. The controller is software, running on some server, which can be a VM or a physical server. An application can run on the same server as the controller and use an NBI, which is an API, so that two programs can communicate.

Figure 21-6 shows just such an example. The big box in the figure represents the system where the controller software resides. This particular controller happens to be written in Java and has a Java-based native API. Anyone—the same vendor as the controller vendor, another company, or even you—can write an app that runs on this same operating system that uses the controller's Java API. By using that API to exchange data with the controller, the application can learn information about the network. The application can also program flows in the network—that is, ask the controller to add the specific match/action logic (flows) into the forwarding tables of the networking devices.

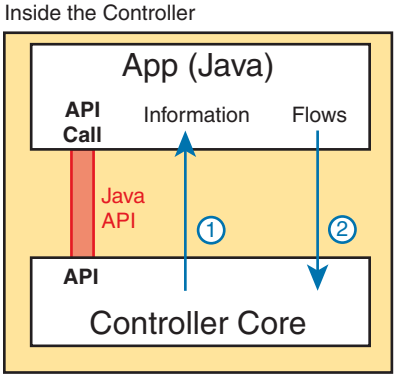



**Figure 21-6** *Java API: Java Applications Communicates with Controller*

**NOTE** The northbound interface (NBI) gets its name from its normal location as shown above the controller—that is, in what would be north on a map.

Before we leave the topic of NBIs, let's close with a brief explanation of a REST API as used for a controller. REST (*REpresentational State Transfer*) describes a type of API that allows applications to sit on different hosts, using HTTP messages to transfer data over the API. When you see SDN figures like Figure 21-6, with the application running on the same system as the controller, the API does not need to send messages over a network because both programs run on the same system. But when the application runs on a different system somewhere else in the network other than running on the controller, the API needs a way to send the data back and forth over an IP network, and RESTful APIs meet that need.

Figure 21-7 shows the big ideas with a REST API. The application runs on a host at the top of the figure. In this case, at Step 1, it sends an HTTP GET request to a particular URI. The HTTP GET is like any other HTTP GET, even like those used to retrieve web pages. However, the URI is not for a web page, but rather identifies an object on the controller, typically a data structure that the application needs to learn and then process. For example, the URI might identify an object that is the list of physical interfaces on a specific device along with the status of each.

Key Topic

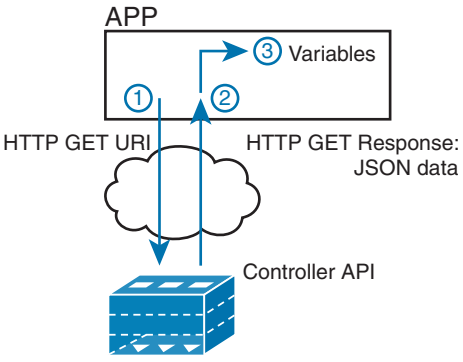



**Figure 21-7** *Process Example of a GET Using a REST API*

At Step 2, the controller sends back an HTTP GET response message with the object. Most REST APIs will ask for and receive structured data. That is, instead of receiving data that is a web page, like a web browser would receive, the response holds variable names and their values, in a format that can be easily used by a program. The common formats for data used for network programmability are JavaScript Object Notation (JSON) and eXtensible Markup Language (XML), shown as Step 3.

### Software Defined Architecture Summary

SDN and network programmability introduce a new way to build networks. The networking devices still exist and still forward data, but the control plane functions and locations can change dramatically. The centralized controller acts as the focal point, so that at least some of the control plane functions move from a distributed model to a centralized model.

However, the world of network programmability and SDN includes a wide array of options and solutions. Some options pull most control plane functions into the controller, whereas others pull only some of those functions into the controller. The next section takes a look at two different options, each of which takes a different approach to network programmability and the degree of centralized control.

## Examples of Network Programmability and SDN

This second of three major sections of the chapter introduces two different SDN and network programmability solutions available from Cisco. Others exist as well. These two were chosen because they give a wide range of comparison points:

- OpenDaylight Controller
- Cisco Application Centric Infrastructure (ACI)

### OpenDaylight and OpenFlow

One common form of SDN comes from the Open Networking Foundation (ONF) and is billed as Open SDN. The ONF (www.opennetworking.org) acts as a consortium of users (operators) and vendors to help establish SDN in the marketplace. Part of that work defines protocols, SBIs, NBIs, and anything that helps people implement their vision of SDN.

The ONF model of SDN features OpenFlow. OpenFlow defines the concept of a controller along with an IP-based SBI between the controller and the network devices. Just as

important, OpenFlow defines a standard idea of what a switch's capabilities are, based on the ASICs and TCAMs commonly used in switches today. (That standardized idea of what a switch does is called a *switch abstraction*.) An OpenFlow switch can act as a Layer 2 switch, a Layer 3 switch, or in different ways and with great flexibility beyond the traditional model of a Layer 2/3 switch.

The Open SDN model centralizes most control plane functions, with control of the network done by the controller plus any applications that use the controller's NBIs. In fact, Figure 21-5 (shown previously) showed the network devices without a control plane, which represents this mostly centralized OpenFlow model of SDN.

In the OpenFlow model, applications may use any APIs (NBIs) supported on the controller platform to dictate what kinds of forwarding table entries are placed into the devices; however, it calls for OpenFlow as the SBI protocol. Additionally, the networking devices need to be switches that support OpenFlow.

Because the ONF's Open SDN model has this common thread of a controller with an OpenFlow SBI, the controller plays a big role in the network. The next few pages provide a brief background about two such controllers.

### The OpenDaylight Controller

First, if you were to look back at the history of OpenFlow, you could find information on dozens of different SDN controllers that support the OpenFlow SDN model. Some were more research oriented, during the years in which SDN was being developed and was more of an experimental idea. As time passed, more and more vendors began building their own controllers. And those controllers often had many similar features because they were trying to accomplish many of the same goals. As you might expect, some consolidation eventually needed to happen.

The OpenDaylight open-source SDN controller is one of the more successful SDN controller platforms to emerge from the consolidation process over the 2010s. OpenDaylight took many of the same open-source principles used with Linux, with the idea that if enough vendors worked together on a common open-source controller, then all would benefit. All those vendors could then use the open-source controller as the basis for their own products, with each vendor focusing on the product differentiation part of the effort, rather than the fundamental features. The result was that back in the mid-2010s, the *OpenDaylight SDN controller* (www.opendaylight.org) was born. OpenDaylight (ODL) began as a separate project but now exists as a project managed by the Linux Foundation.

Figure 21-8 shows a generalized version of the ODL architecture. In particular, note the variety of SBIs listed in the lower part of the controller box: OpenFlow, NetConf, PCEP, BGP-LS, and OVSDB; many more exist. The ODL project has enough participants so that it includes a large variety of options, including multiple SBIs, not just OpenFlow.

ODL has many features, with many SBIs, and many core features. A vendor can then take ODL, use the parts that make sense for that vendor, add to it, and create a commercial ODL controller.

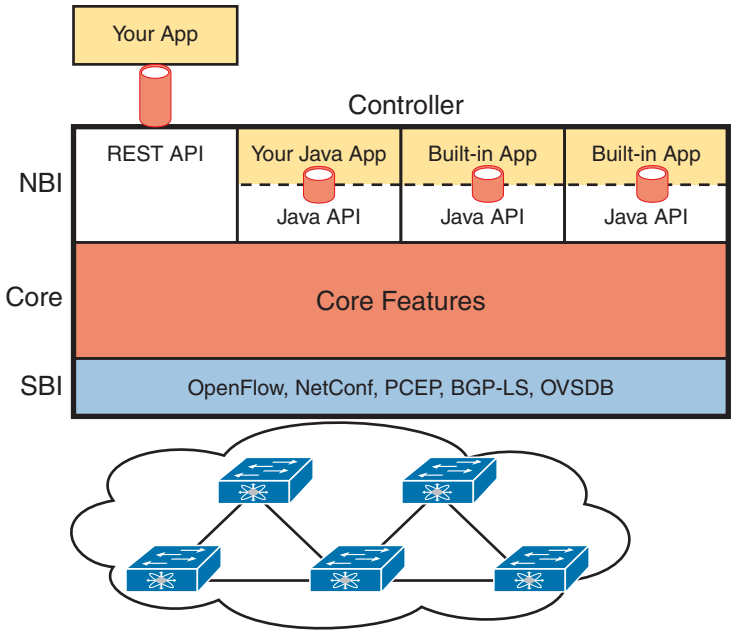



**Figure 21-8** *Architecture of NBI, Controller Internals, and SBI to Network Devices*

### The Cisco Open SDN Controller (OSC)

At one point back in the 2010s, Cisco offered a commercial version of the OpenDaylight controller called the Cisco *Open SDN Controller* (OSC). That controller followed the intended model for the ODL project: Cisco and others contributed labor and money to the ODL open-source project; once a new release was completed, Cisco took that release and built new versions of their product.

Cisco no longer produces and sells the Cisco OSC, but I decided to keep a short section about OSC here in this chapter for a couple of reasons. First, if you do any of your own research, you will find mention of Cisco OSC; however, well before this chapter was written, Cisco had made a strong strategic move toward different approaches to SDN using **intent-based networking (IBN)**. That move took Cisco away from OpenFlow-based SDN. But because you might see references to Cisco OSC online, or in the previous edition of this book, I wanted to point out this transition in Cisco's direction.

This book describes two Cisco offerings that use an IBN approach to SDN. The next topic in this chapter examines one of those: Application Centric Infrastructure (ACI), Cisco's data center SDN product. Chapter 22, "Cisco Software-Defined Access (Cisco SD-Access)," discusses yet another Cisco SDN option that uses intent-based networking: Software-Defined Access (Cisco SD-Access).

**NOTE** Examples of specific products are used only to help readers internalize the concepts of SDN and controller-based networks. Examples of these products would include Cisco Application Centric Infrastructure (ACI) and Cisco Software-Defined Access with Cisco Catalyst Center (Cisco SD-Access).

## Cisco Application Centric Infrastructure (ACI)

Interestingly, many SDN offerings began with research that discarded many of the old networking paradigms in an attempt to create something new and better. For instance, OpenFlow came to be from the Stanford University Clean Slate research project that had researchers reimagining (among other things) device architectures. Cisco took a similar research path, but Cisco's work happened to arise from different groups, each focused on different parts of the network: data center, campus, and WAN. That research resulted in Cisco's current SDN offerings of ACI in the data center: Software-Defined Access (Cisco SD-Access) in the enterprise campus and Software-Defined WAN (Cisco SD-WAN) in the enterprise WAN.

When reimagining networking for the data center, the designers of ACI focused on the applications that run in a data center and what they need. As a result, they built networking concepts around application architectures. Cisco made the network infrastructure become application centric, hence the name of the Cisco data center SDN solution: **Application Centric Infrastructure (ACI)**.

For example, Cisco looked at the data center world beyond networking and saw lots of automation and control. As discussed in Chapter 20, "Cloud Architecture," virtualization software routinely starts, moves, and stops VMs. Additionally, cloud software enables self-service for customers so they can enable and disable highly elastic services as implemented with VMs and containers in a data center. From a networking perspective, some of those VMs need to communicate, but some do not. And those VMs can move based on the needs of the virtualization and cloud systems.

ACI set about to create data center networking with the flexibility and automation built into the operational model. Old data center networking models with a lot of per-physical-interface configuration on switches and routers were just poor models for the rapid pace of change and automated nature of modern data centers. This section looks at some of the detail of ACI to give you a sense of how ACI creates a powerful and flexible network to support a modern data center in which VMs and containers are created, run, move, and are stopped dynamically as a matter of routine.

### ACI Physical Design: Spine and Leaf

The Cisco ACI uses a specific physical switch topology called **spine** and **leaf**. While the other parts of a network might need to allow for many different physical topologies, the data center could be made standard and consistent. But what particular standard and consistent topology? Cisco decided on the spine and leaf design, also called a Clos network after one of its creators.

With ACI, the physical network has a number of spine switches and a number of leaf switches, as shown in Figure 21-9. The figure shows the links between switches, which can be single links or multiple parallel links. Of note in this design (assuming a single-site design):




- Each leaf switch must connect to every spine switch.
- Each spine switch must connect to every leaf switch.
- Leaf switches cannot connect to each other.
- Spine switches cannot connect to each other.
- Endpoints connect only to the leaf switches.

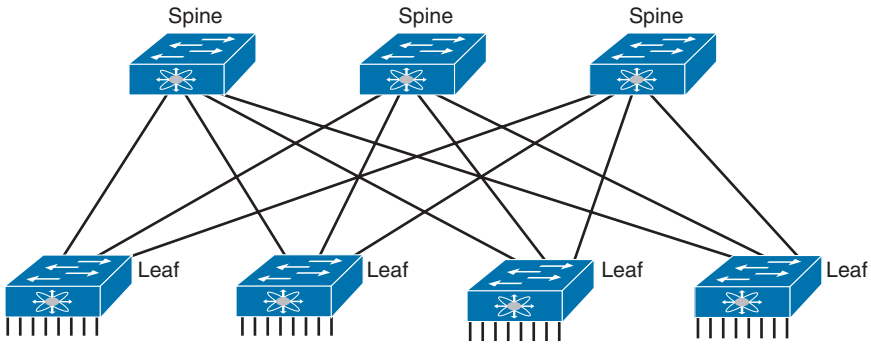



**Figure 21-9** *Spine-Leaf Network Design*

Endpoints connect only to leaf switches and never to spine switches. To emphasize the point, Figure 21-10 shows a more detailed version of Figure 21-9, this time with endpoints connected to the leaf switches. None of the endpoints connect to the spine switches; they connect only to the leaf switches. The endpoints can be connections to devices outside the data center, like the router on the left. By volume, most of the endpoints will be either physical servers running a native OS or servers running virtualization software with numbers of VMs and containers as shown in the center of the figure.

Key Topic

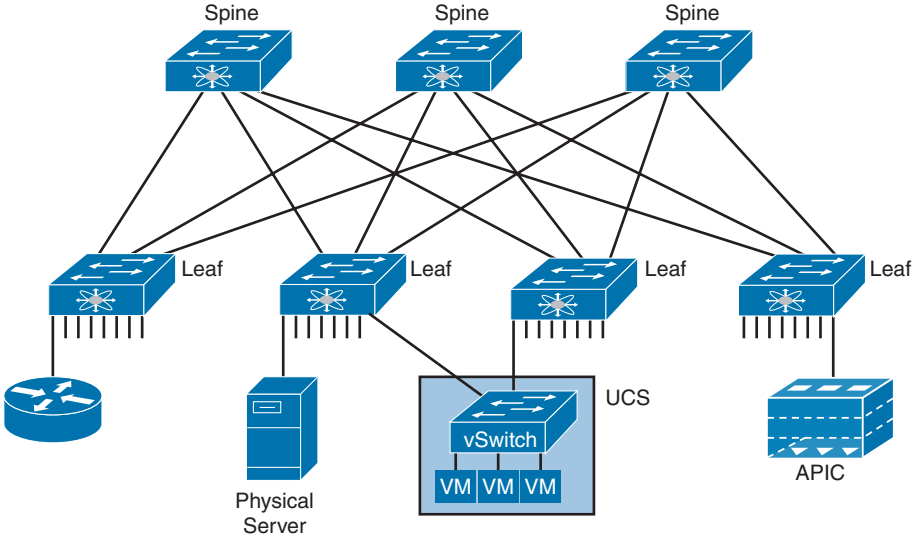



**Figure 21-10** *Endpoints Found on the Leaf Switches Only*

Also, note that the figure shows a typical design with multiple leaf switches connecting to a single hardware endpoint like a Cisco Unified Computing System (UCS) server. Depending on the design requirements, each UCS might connect to at least two leaf switches, both for redundancy and for greater capacity to support the VMs and containers running on the UCS hardware. (In fact, in a small design with UCS or similar server hardware, every UCS might connect to every leaf switch.)

### ACI Operating Model with Intent-Based Networking

The model that Cisco defines for ACI uses a concept of endpoints and policies. The *endpoints* are the VMs, containers, or even traditional servers with the OS running directly on the hardware. ACI then uses several constructs as implemented via the **Application Policy Infrastructure Controller (APIC)**, the software that serves as the centralized controller for ACI.

This section hopes to give you some insight into ACI, rather than touch on every feature. To do that, consider the application architecture of a typical enterprise web app for a moment. Most casual observers think of a web application as one entity, but one web app often exists as three separate servers:

- **Web server:** Users from outside the data center connect to a web server, which sends web page content to the user.
- **App (Application) server:** Because most web pages contain dynamic content, the app server does the processing to build the next web page for that particular user based on the user's profile and latest actions and input.
- **DB (Database) server:** Many of the app server's actions require data; the DB server retrieves and stores the data as requested by the app server.

To accommodate those ideas, ACI uses an intent-based networking (IBN) model. With that model, the engineer, or some automation program, defines the policies and intent for which endpoints should be allowed to communicate and which should not. Then the controller determines what that means for this network at this moment in time, depending on where the endpoints are right now.

For instance, when starting the VMs for this app, the virtualization software would create (via the APIC) several endpoint groups (EPGs) as shown in Figure 21-11. The controller must also be told the access polices, which define which EPGs should be able to communicate (and which should not), as implied in the figure with arrowed lines. For example, the routers that connect to the network external to the data center should be able to send packets to all web servers, but not to the app servers or DB servers.

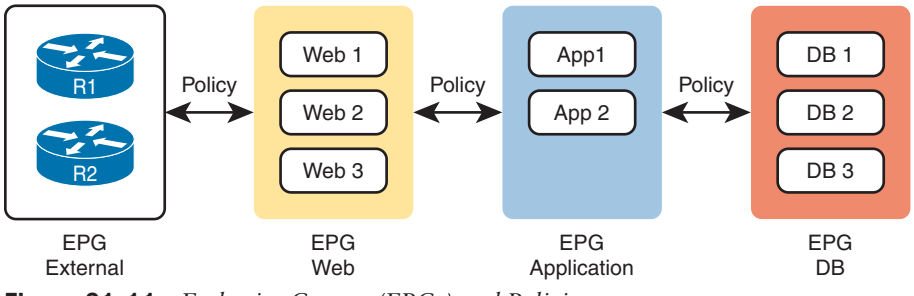



**Figure 21-11** *Endpoint Groups (EPGs) and Policies*

Note that at no point did the previous paragraph talk about which physical switch interfaces should be assigned to which VLAN or which ports are in an EtherChannel; the discussion moves to an application-centric view of what happens in the network. Once all the endpoints, policies, and related details are defined, the controller can then direct the network as

to what needs to be in the forwarding tables to make it all happen—and to more easily react when the VMs start, stop, or move.

To make it all work, ACI uses a centralized controller called the Application Policy Infrastructure Controller (APIC), as shown in Figure 21-12. The name defines the function in this case: it is the controller that creates application policies for the data center infrastructure. The APIC takes the intent (EPGs, policies, and so on), which completely changes the operational model away from configuring VLANs, trunks, EtherChannels, ACLs, and so on.

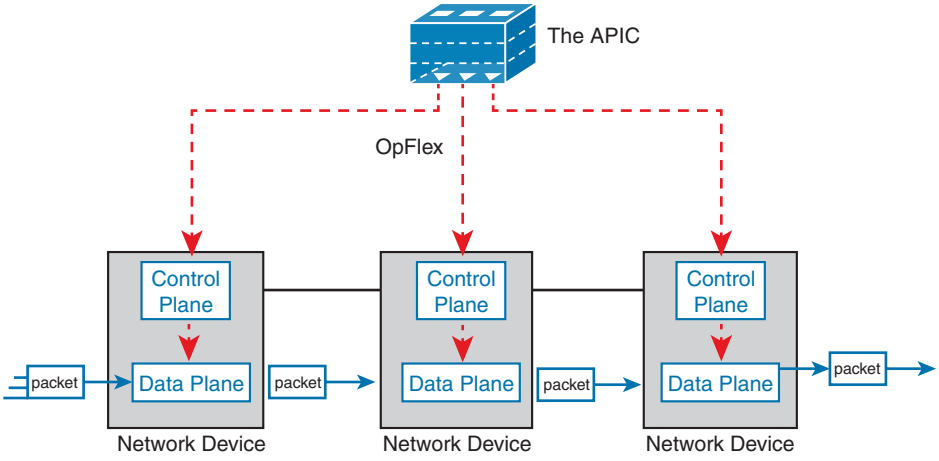



**Figure 21-12** *Architectural View of ACI with APIC Pushing Intent to Switch the Control Plane*

The APIC, of course, has a convenient GUI, but the power comes in software control—that is, network programmability. The same virtualization software, or cloud or automation software, even scripts written by the network engineer, can define the endpoint groups, policies, and so on to the APIC. But all these players access the ACI system by interfacing to the APIC as depicted in Figure 21-13; the network engineer no longer needs to connect to each individual switch and configure CLI commands.

Key Topic

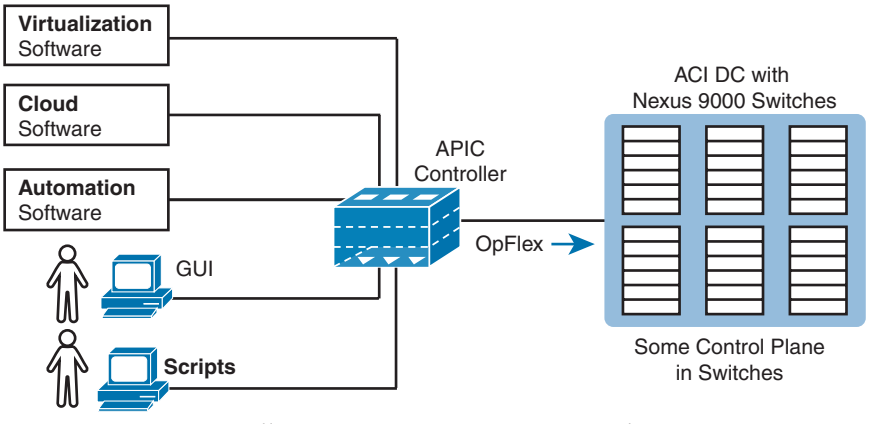



**Figure 21-13** *Controlling the ACI Data Center Network Using the APIC*

For more information on Cisco ACI, go to www.cisco.com/go/aci.

### Summary of the SDN Examples

The sample SDN architectures in this section of the book were chosen to provide a wide variety for the sake of learning. However, they differ to some degree in how much of the control plane work is centralized. Table 21-2 lists those and other comparison points taken from this section, for easy review and study.

Key Topic

**Table 21-2** Points of Comparison: OpenFlow and ACI

| Criteria | OpenFlow | ACI |
|---|---|---|
| Changes how the device control plane works versus traditional networking | Yes | Yes |
| Creates a centralized point from which humans and automation control the network | Yes | Yes |
| Degree to which the architecture centralizes the control plane | Mostly | Partially |
| SBIs used | OpenFlow | OpFlex |
| Controllers mentioned in this chapter | OpenDaylight | APIC |
| Organization that is the primary definer/owner | ONF | Cisco |

If you want to learn more about the Cisco solutions, consider using both Cisco DevNet (the Cisco Developer Network) and dCloud (Demo cloud). Cisco provides its DevNet site (https://developer.cisco.com) for anyone interested in network programming, and the Demo Cloud site (https://dcloud.cisco.com) for anyone to experience or demo Cisco products. At the time this book went to press, DevNet had many REST API labs, while both sites had a variety of ACI-based labs.

## Comparing Traditional Versus Controller-Based Networks

Before finishing the chapter, this final topic turns directly toward the CCNA 200-301 V1.1 exam. Two of the CCNA 200-301 exam topics in domain 6.0, "Automation and Programmability," ask us to compare some aspect of traditional networks versus new networking using controllers and automation. Those exam topics include

6.1: Explain how automation impacts network management

6.2: Compare traditional networks with controller-based networking

First, the wording in both exam topics can be reduced to "compare and contrast." One uses the word *compare*. The other uses a longer phrase "explain how automation impacts…," which asks us to compare what was before to what happens now that automation has been added to the network.

Network management can be broken down into two distinct categories: configuration management and operational management.

Configuration management refers to any feature that changes device configuration, with automated configuration management doing so with software (program) control. For instance, Cisco's ACI uses the APIC controller. You do not configure the devices directly, but

the APIC pushes configuration down to the ACI switches that it builds based on its interpretation of the policies configured by the engineer. With ACI, the configuration management occurs as a part of the overall system. Other configuration management tools can be more focused on automating traditional configuration processes, with tools like NETCONF/RESTCONF, Ansible, and Terraform, as discussed in Chapter 23, "Understanding REST and JSON," and Chapter 24, "Understanding Ansible and Terraform."

Operational network management includes monitoring, gathering operational data, reporting, and alerting humans to possible issues. For instance, **Cisco Catalyst Center** has a capability that checks the IOS images on Cisco devices to make sure only approved versions are used and that no changes have occurred to the images in comparison to the images created by Cisco.

**NOTE** Cisco DNA Center was rebranded to Cisco Catalyst Center; however, all features and functionality remain the same. Please visit www.cisco.com/go/catalystcenter for more information. Catalyst Center is used because it is an example of a commonly used campus controller.

The other exam topic (6.2) described in this section focuses on controller-based networking instead of network management. That exam topic includes any SDN network as characterized by the use of a controller. Today people might use that term or these other synonyms to describe some of the newer networking options that happen to use controllers:

- Software Defined Networking
- Software Defined Architecture
- Programmable Networks
- Controller-Based Networks

Table 21-3 summarizes the chapters that have content related to these two exam topics.

**Table 21-3** Exam Topics and Most Relevant Chapters

| Exam Topic | Exam Topic Text | Most Relevant Chapter(s) |
|---|---|---|
| 6.1 | Explain how automation impacts network management | 21–24 |
| 6.2 | Compare traditional networks with controller-based networking | 21, 22 |

## How Automation Impacts Network Management

This chapter introduces many of the features that enable automation in SDNs, but so far it has not made any overt statements about how automation impacts network management. This next topic works through a couple of examples that show the power of automation as enabled through controller-based networks.

First, centralized controllers formalize and define data models for the configuration and operational data about networks. We humans might be comfortable with visually scanning the output of **show** commands to find the tidbit of information we need. Programs need to be able to identify the specific fact. To build a controller-based network with APIs, all the

data about the network needs to be defined in a data model so programs can use that data via API calls. Before using controllers, automation scripts often had to begin by processing the text output of a **show** command, but with controllers and the data models behind the APIs, the data can be readily available to any automation script or vendor application through a **northbound API**.

For instance, Example 21-1 shows some output from a command on a switch. With a northbound API on a controller, and the data model it supplies, an automation program could issue this command and begin by parsing this text. The goal: find the configuration setting on the **switchport mode** command and the current trunking state.

**Example 21-1** *Small Output from a Switch Command*

```
SW1# show interfaces gigabit 0/1 switchport
Name: Gi0/1
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
Negotiation of Trunking: On
```

Example 21-2 shows a simple example of the starting point for a program using a controller's northbound API. Instead of asking for the text from a **show** command, the API call will result in the program having a series of variables set. In this case, there are variables for that same interface that list the trunk configuration setting and the trunk operational state.

**Example 21-2** *Python Dictionary with Variables Set to Needed Values*

```
>>> interface1
{'trunk-config': 'dynamic auto', 'trunk-status': 'static access'}
>>>
```

Using a controller-based model not only supplies APIs that give us the exact same data a human could see in **show** commands, but often they also supply much more useful information. A controller collects data from the entire network; consequently, the controller can be written so that it analyzes and presents more useful data via the API. As a result, software that uses the APIs—whether automation written by local engineers or applications written by vendors—can be written more quickly and can often create features that would have been much more difficult without a controller.

For instance, Cisco Catalyst Center provides a path trace feature. The application shows the path of a packet from source to destination, with the forwarding logic used at each node.

Now imagine writing that application with either of these two approaches.

- One API call that returns a list of all devices and their running configuration, with other API calls to collect each device's MAC address tables and/or their IP routing tables. Then you have to process that data to find the end-to-end path.

- One API call to which you pass the source and destination IP addresses and TCP/UDP ports, and the API returns variables that describe the end-to-end path, including device hostnames and interfaces. The variables spell out the path the packet takes through the network.

The second option does most of the work, while the first option leaves most of the work to you and your program. But that second option becomes possible because of the centralized controller. The controller has the data if it at least collects configuration and forwarding table information. Going beyond that, Cisco controllers analyze the data to provide much more useful data. The power of these kinds of APIs is amazing, and this is just one example.

The following list summarizes a few of the comparison points for this particular exam topic:

Key Topic

- Northbound APIs and their underlying data models make it much easier to automate functions versus traditional networks.
- The robust data created by controllers makes it possible to automate functions that were not easily automated without controllers.
- The new reimagined software defined networks that use new operational models simplify operations, with automation resulting in more consistent configuration and fewer errors.
- Centralized collection of operational data at controllers allows the application of modern data analytics to networking operational data, providing actionable insights that were likely not noticeable with the former model.
- Time required to complete projects is reduced.
- New operational models use external inputs, like considering time of day, day of week, and network load.

## Comparing Traditional Networks with Controller-Based Networks

As for exam topic 6.2, this entire chapter begins to show the advantages created by using controller-based networks. However, this chapter only begins to describe the possibilities. By centralizing some of the functions in the network and providing robust APIs, controllers enable a large number of new operational models. Those models include the three most likely to be seen from Cisco in an enterprise: Cisco Software-Defined Access (Cisco SD-Access), Cisco Software-Defined WAN (Cisco SD-WAN), and Cisco Application Centric Infrastructure (Cisco ACI). (Chapter 22 introduces Cisco SD-Access.)

This changes the operating paradigm in many cases, with the controller determining many device-specific details:

- The network engineer does not need to think about every command on every device.
- The controller configures the devices with consistent and streamlined settings.
- The result: faster and more consistent changes with fewer issues.

As another example, just consider the ACI example from earlier in the chapter. Instead of configuring each port with an access VLAN, or making it a trunk, adding routing protocol configuration, and possibly updating IP ACLs, all you had to do was create some endpoint groups (EPGs) and policies. In that case, the orchestration software that started the VMs could automatically create the EPGs and policies. The new paradigm of intent-based networking was enabled through the controller-based architecture. Then the automation features enabled by the controller's northbound APIs allowed third-party applications to automatically configure the network to support the necessary changes.

Some of the advantages include the following:

Key Topic

- Uses new and improved operational models that allow the configuration of the network as a system rather than per-device configuration
- Enables automation through northbound APIs that provide robust methods and model-driven data
- Configures the network devices through **southbound APIs**, resulting in more consistent device configuration, fewer errors, and less time spent troubleshooting the network
- Enables a DevOps approach to networks

Chapter 22 goes into some depth comparing traditional networking with controller-based networks with descriptions of Cisco Software-Defined Access (Cisco SD-Access). Look throughout that chapter for some of the reasons and motivations for Cisco SD-Access and the features enabled by using the Cisco Catalyst Center controller.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 21-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 21-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Book, app |
| Watch video | | Website |

## Review All the Key Topics

Key Topic

**Table 21-5** Key Topics for Chapter 21

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Sample actions of the networking device data plane | 473 |
| List | Sample actions of the networking device control plane | 474 |
| Figure 21-4 | Switch internals with ASIC and TCAM | 476 |
| Figure 21-5 | Basic SDN architecture, with the centralized controller programming device data planes directly | 478 |
| Paragraph | Description of the role and purpose of the NBI | 479 |
| Figure 21-7 | REST API basic concepts | 481 |
| List | Spine-leaf topology requirements | 484 |
| Figure 21-10 | Spine-leaf design | 485 |
| Figure 21-13 | Controlling the ACI data center network using APIC | 487 |
| Table 21-2 | Comparisons of Open SDN and Cisco ACI options | 488 |
| List | Comparisons of how automation improves network management | 491 |
| List | Comparisons of how controller-based networking works versus traditional networking | 492 |

## Key Terms You Should Know

Application Centric Infrastructure (ACI), Application Policy Infrastructure Controller (APIC), application programming interface (API), application-specific integrated circuit (ASIC), centralized control plane, Cisco Catalyst Center, control plane, controller-based networking, data plane, distributed control plane, intent-based networking (IBN), leaf, management plane, northbound API, northbound interface (NBI), OpenFlow, Software Defined Networking (SDN), southbound API, southbound interface (SBI), spine, ternary content-addressable memory (TCAM)



CHAPTER 22

# Cisco Software-Defined Access (Cisco SD-Access)

**This chapter covers the following exam topics:**

- **1.0 Network Fundamentals**
  - **1.1 Explain the role and function of network components**
    - **1.1.e Controllers**
- **6.0 Automation and Programmability**
  - **6.1 Explain how automation impacts network management**
  - **6.2 Compare traditional networks with controller-based networking**
  - **6.3 Describe controller-based, software-defined architecture (overlay, underlay, and fabric)**
    - **6.3.a Separation of control plane and data plane**
    - **6.3.b Northbound and Southbound APIs**
  - **6.4 Explain AI (generative and predictive) and machine learning in network operations**

Cisco Software-Defined Access (SD-Access) is a software-defined approach to build converged wired and wireless campus networks or LANs. The word *access* in the name refers to the three-tier network architecture design that consists of the core, distribution, and access layers of a network. The access layer is where the endpoint devices connect to the network, while *software-defined* refers to many of the software-defined architectural features discussed in Chapter 21, "Introduction to Controller-Based Networking." These features include a centralized controller—called Cisco Catalyst Center—and use both southbound and northbound protocols and APIs. It also includes a completely different operational model within Cisco SD-Access when compared to that of a traditional network. Cisco SD-Access creates a network fabric composed of an underlay network and an overlay network.

**NOTE** Cisco DNA Center was rebranded to Cisco Catalyst Center; however, all features and functionality remain the same. Please visit www.cisco.com/go/catalystcenter for more information.

Cisco SD-Access is Cisco's main campus offering and fits within Cisco Digital Network Architecture (DNA). Cisco DNA defines the entire architecture for the new world of software-defined networks, digitization, and how Cisco networks should be operated in the future. This chapter introduces Cisco SD-Access, which exists as one implementation of Cisco DNA.

The discussion of Cisco SD-Access and Cisco DNA provides a great backdrop to discuss a few other topics from the CCNA blueprint: Cisco Catalyst Center controller and network management. Cisco SD-Access uses the Cisco Catalyst Center controller to configure and operate Cisco SD-Access. However, Cisco Catalyst Center also acts as a complete network management platform. To understand Cisco Catalyst Center, you also need to understand traditional network management as well as the new management models using controllers.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 22-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Cisco SD-Access Fabric, Underlay, and Overlay | 1–3 |
| Cisco Catalyst Center and Cisco SD-Access Operation | 4, 5 |
| Cisco Catalyst Center as a Network Management Platform | 6 |
| Artificial Intelligence (AI), Machine Learning (ML), and Operational Management | 7 |

1. In Cisco Software-Defined Access (Cisco SD-Access), which term refers to the devices and cabling, along with configuration that allows the network device nodes enough IP connectivity to send IP packets to each other?
   - **a.** Fabric
   - **b.** Overlay
   - **c.** Underlay
   - **d.** VXLAN
2. In Cisco Software-Defined Access (Cisco SD-Access), which term refers to the functions that deliver endpoint packets across the network using tunnels between the ingress and egress fabric nodes?
   - **a.** Fabric
   - **b.** Overlay
   - **c.** Underlay
   - **d.** VXLAN
3. In Software-Defined Access (Cisco SD-Access), which of the answers are part of the overlay data plane?
   - **a.** LISP
   - **b.** GRE
   - **c.** OSPF
   - **d.** VXLAN

4. Which answers best describe options of how to implement security with scalable groups using Cisco Catalyst Center and Cisco SD-Access? (Choose two answers.)
   a. A human user from the Cisco Catalyst Center GUI
   b. An automation application using NETCONF
   c. A human user using the CLI of a Cisco SD-Access fabric edge node
   d. An automation application using REST
5. Which of the following protocols or tools could be used as part of the Cisco Catalyst Center southbound interface? (Choose three answers.)
   a. Ansible
   b. SSH
   c. NETCONF
   d. SNMP
   e. Puppet
6. Which of the following are network management features performed by both traditional network management software as well as by Cisco Catalyst Center? (Choose two answers.)
   a. Network device discovery
   b. Software-Defined Access configuration
   c. End-to-end path discovery with ACL analysis
   d. Device installation (day 0), configuration (day 1), and monitoring (day n) operations
7. What distinguishes Narrow AI from Generative AI?
   a. Narrow AI lacks broad cognitive capabilities, while Generative AI thrives in a limited scope.
   b. Narrow AI is designed for specific tasks, while Generative AI is capable of learning and decision-making.
   c. Narrow AI includes applications like speech recognition, while Generative AI focuses on conversational platforms like ChatGPT.
   d. Narrow AI relies on explicit programming, while Generative AI learns patterns and relationships from data sources.

# Foundation Topics

## Cisco SD-Access Fabric, Underlay, and Overlay

Cisco **Software-Defined Access (Cisco SD-Access)** creates an entirely new way to build campus LANs as compared to the traditional methods of networking discussed in most chapters of this book. In the mid-2010s, Cisco set about to reimagine campus networking, with Cisco SD-Access as the result.

Cisco SD-Access uses the software-defined architectural model introduced in Chapter 21, with a controller and various APIs. Cisco SD-Access networks are still built using a

collection of physical networking equipment such as routers and switches as well as cables, and other various endpoint devices. At the top of the network is the Cisco Catalyst Center controller, as shown in Figure 22-1. Cisco Catalyst Center is where users utilize a graphical user interface (GUI) for design, configuration, and implementation settings, and the controller provides automation by leveraging APIs. In short, Cisco Catalyst Center is the controller for Cisco SD-Access networks.

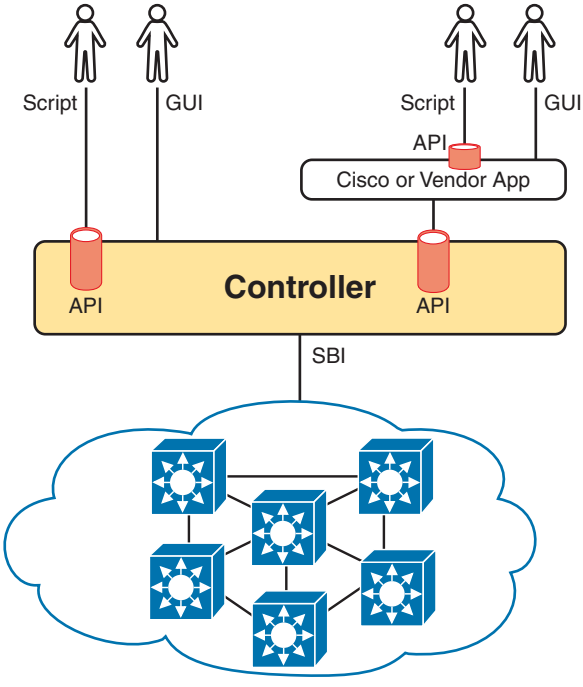



**Figure 22-1** *Cisco SD-Access Architectural Model with Cisco Catalyst Center*

In terms of architecture, the southbound side of the controller contains components such as the fabric, the underlay, and the overlay. By design in SDN implementations, most of the interesting new capabilities occur on the northbound side, which are examined in the second half of this chapter. This first half of the chapter examines the details south of the controller—namely, the underlay network, overlay network, and fabric.

Key Topic

**Underlay:** The physical devices and connections, which include wired and wireless devices that provide IP connectivity to all nodes within the campus LAN. The main goal of an underlay network is to provide reachability to all Cisco SD-Access devices in order to support the dynamic creation of Virtual eXtensible Local-Area Network (**VXLAN**) overlay tunnels.

**Overlay:** The collection of devices that use VXLAN tunnels that are created between other Cisco SD-Access devices such as switches and fabric-enabled access points. Overlays are used to transport traffic from one endpoint to another over the fabric.

**Fabric:** The combination of overlay and underlay technologies, which together provide all features to deliver data across the network with the desired features and attributes. This

could be all devices or a subset of devices that make up the fabric. This also means that multiple fabrics can exist within a Cisco SD-Access network.

VXLAN is defined in RFC 7348 as a technology used to create virtual networks that can span different physical locations or devices. It allows for more flexible and scalable communication between devices. VXLAN encapsulates data and extends Layer 2 networks over Layer 3 networks and is often seen in data center and cloud environments. This technology can help improve network efficiency and support the growing needs of modern applications.

In Cisco SD-Access, one main use case is something called *host mobility*. As wireless becomes a more pervasive method of connectivity, it is more common for users to move from location to location during a typical workday. These locations can be from the user's desk to the break room or from the data center to the call center. In many cases, users no longer sit and work in the same location within the corporate campus environment. This includes being wired. Users can move and plug into network jacks at other desks or also in conference rooms.

Thinking through this scenario, in the simplest terms, means that a user can hop from one wireless network with one IP address and set of security rules to another VLAN or subnet with a different IP address and set of security rules or access lists. Typically, this capability was handled by means of DHCP and the hope that the network administrator had the appropriate security policies in place to make sure that, when users went from one area of the network to another, the same rules applied. This is very often a manual process in many networks. Something had to be done to change the way users were treated technically and also from a user-experience perspective. For example, if a user moves from one area of the building to another and then suddenly couldn't access a mission-critical application because of a missing access list entry, that causes a problem for the user who is trying to get a job done.

In less formal terms, the underlay exists as multilayer switches and their links, with IP connectivity—but for a special purpose. Traffic sent by the endpoint devices flows through VXLAN tunnels in the overlay using a completely different process than traditional LAN switching and IP routing.

For instance, think about the idea of sending packets from hosts on the left of a network, over Cisco SD-Access, to hosts on the right. For instance, imagine a packet enters on the left side of the physical network at the bottom of Figure 22-2 and eventually exits the campus out switch SW2 on the far right. This underlay network looks like a more traditional network drawing, with several devices and links.

The overlay drawing at the top of the figure shows only two switches—called fabric edge nodes, because they happen to be at the edges of the Cisco SD-Access fabric—with a tunnel labeled VXLAN connecting the two. Both concepts (underlay and overlay) together create the Cisco SD-Access fabric.

The next few pages explain both the underlay and overlay in a little more depth.

---

Answers to the "Do I Know This Already?" quiz:

**1** C **2** B **3** D **4** A, D **5** B, C, D **6** A, D **7** B

Key Topic

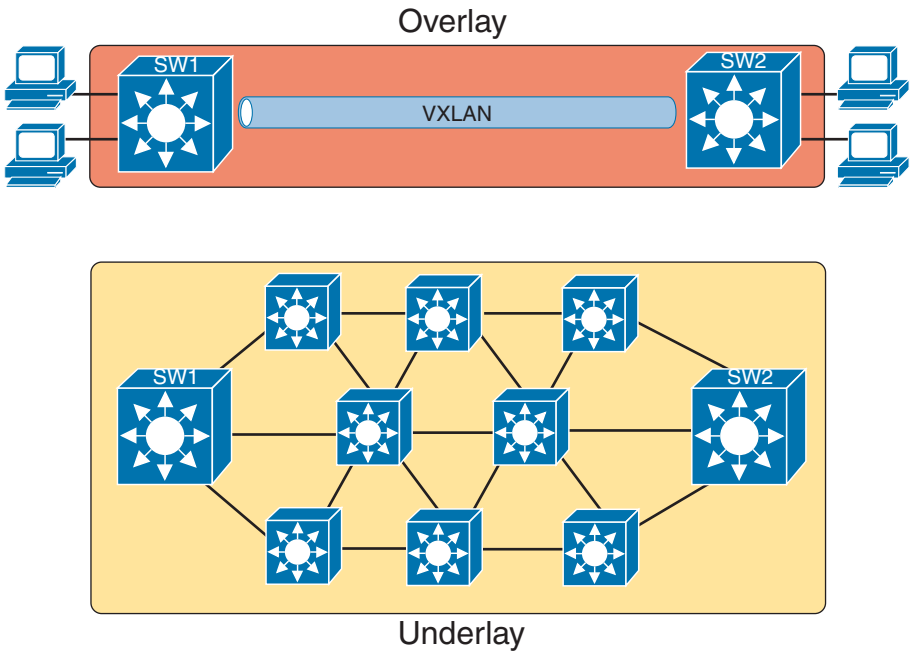



**Figure 22-2** *Fabric, Underlay, and Overlay Concepts*

## The Cisco SD-Access Underlay

With Cisco SD-Access, the underlay exists to provide connectivity between the fabric edge nodes in the campus environment for the purpose of supporting VXLAN tunnels in the overlay network. To do that, the underlay includes the switches, routers, cables, and wireless links used to create the physical network. It also includes the configuration and operation of the underlay so it can support the work of the overlay network.

### Using Existing Gear for the Cisco SD-Access Underlay

To build a Cisco SD-Access underlay network, companies have two basic choices. They can use supported models of their existing campus network and add new configuration to create an underlay network, while still supporting their existing production traffic with traditional routing and switching. Alternately, the company can purchase some new Catalyst switches and build the Cisco SD-Access network without concern for harming existing traffic, and migrate endpoints to the new network fabric over time.

To build Cisco SD-Access into an existing network, it helps to think for a moment about some typical campus network designs. The larger campus site may use either a two-tier or three-tier design as discussed in Chapter 18, "LAN Architecture." It has a cluster of wireless LAN controllers (WLCs) to support a number of lightweight APs (LWAPs). Engineers have configured VLANs, VLAN trunks, IP routing, IP routing protocols, ACLs, and so on. And the LAN connects to WAN routers.

Cisco SD-Access can be added into an existing campus LAN, but doing so has some risks and restrictions. First and foremost, you have to be careful not to disrupt the current network while adding the new features to the network. The issues include

- Because of the possibility of harming the existing production configuration, **Cisco Catalyst Center** should not be used to configure the underlay if the devices are currently used in production. (Cisco Catalyst Center will be used to configure the underlay with deployments that use all new hardware.)
- The existing hardware must be from the Cisco SD-Access compatibility list, with different models supported depending on their different roles (see a link at www.cisco.com/go/sd-access).
- The device software levels must meet the requirements, based on their roles, as detailed in that same compatibility list.

For instance, imagine an enterprise happened to have an existing campus network that uses Cisco SD-Access-compatible hardware. That company might need to update the IOS versions in a few cases. Additionally, the engineers would need to configure the underlay part of the Cisco SD-Access devices manually rather than with Cisco Catalyst Center because Cisco assumes that the existing network already supports production traffic, so they want the customer directly involved in making those changes.

The Cisco SD-Access underlay configuration requires you to think about and choose the different roles filled by each device before you can decide which devices to use and which minimum software levels each requires. If you look for the hardware compatibility list linked from www.cisco.com/go/sd-access, you will see different lists of supported hardware and software depending on the roles. These roles include




**Fabric edge node:** A switch that connects to endpoint devices (similar to traditional access or leaf switches)

**Fabric border node:** A switch that connects to devices outside the control of Cisco SD-Access—for example, switches that connect to the WAN routers or to an ACI data center

**Fabric control-plane node:** A switch that performs special control plane functions for the underlay (LISP), requiring more CPU and memory

For example, Cisco's compatibility list includes many Catalyst 9200, 9300, 9400, 9500, and 9600 series switches, but also some older Catalyst 3850 and 3650 switches, as fabric edge nodes. However, some products did not make the list as fabric edge nodes. For fabric control nodes, the list included higher-end Catalyst switch models (which typically have more CPU and RAM), plus several router models (routers typically have much more RAM for control plane protocol storage—for instance, for routing protocols).

The beginning of a Cisco SD-Access project will require you to look at the existing hardware and software to begin to decide whether the existing campus might be a good candidate to build the fabric with existing gear or to upgrade hardware when building the new campus fabric LAN.

### Using New Gear for the Cisco SD-Access Underlay

When buying new hardware for the Cisco SD-Access fabric—that is, a greenfield design—you remove many of the challenges that exist versus deploying Cisco SD-Access on existing gear. You can simply order compatible hardware and software. Once it arrives, you can leverage Cisco Catalyst Center to then configure all the underlay features automatically.

At the same time, the usual campus LAN design decisions will still need to be made. Enterprises use Cisco SD-Access as a better way to build and operate a campus network, but it is still a campus network. It needs to provide access and connectivity to all types of users and devices. When planning a greenfield Cisco SD-Access design, plan to use appropriate compatible hardware, but also think about these traditional LAN design points:

- The number of ports needed in switches in each wiring closet
- The port speeds required
- The benefit of a switch stack in each wiring closet
- The cable length and types of cabling already installed
- The need for power (PoE/PoE+)
- The power available in each new switch versus the PoE power requirements
- Link capacity (speed and number of links) for links between switches

As far as the topology, traditional campus design does tell us how to connect devices, but Cisco SD-Access does not have to follow those traditional rules. To review, traditional campus LAN Layer 2 design (as discussed back in Chapter 18) tells us to connect each access switch to two different distribution layer switches, but not to other access layer switches, as shown in Figure 22-3. The access layer switch acts as a Layer 2 switch, with a VLAN limited to those three switches.

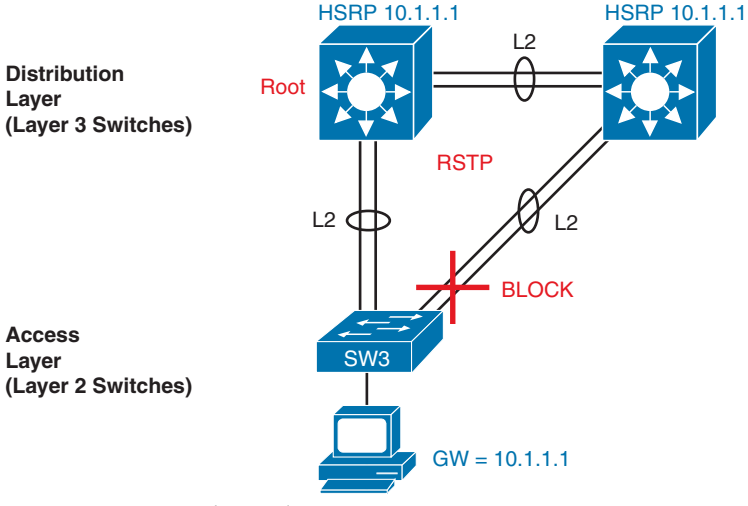



**Figure 22-3** *Traditional Access Layer Design: Three Switches in STP Triangle*

Think through some of the traditional features shown in the figure. The distribution layer switches—Layer 3 switches—act as the default gateway used by hosts and commonly implement HSRP for better availability. The design uses more than one uplink from the access to distribution layer switches, with Layer 2 EtherChannels to allow balancing in addition to redundancy. STP/RSTP manages the small amount of Layer 2 redundancy in the campus, preventing loops by blocking traffic on some ports.

In comparison, a greenfield Cisco SD-Access fabric uses a *routed access layer* design. Routed access layer designs were around long before Cisco SD-Access, but Cisco SD-Access makes good use of the design, and it works very well for the underlay with its goal to support VXLAN tunnels in the overlay network. A routed access layer design simply means that all the LAN switches are Layer 3 switches, with routing enabled, so all the links between switches operate as Layer 3 links. This also means that there are no links in a blocking state and not being used due to STP. This increases available bandwidth and capacity due to all links being used to route traffic versus some sitting idle.

When specifying a greenfield Cisco SD-Access deployment, you can identify what gear you want Cisco Catalyst Center to configure, and it will handle the configuration of the underlay on those devices. This includes using a *routed access layer*. Because this is a greenfield deployment, Cisco Catalyst Center knows that it can configure the switches and wireless devices without any concern of causing harm to a production network. Cisco Catalyst Center will choose the best underlay configuration to support the Cisco SD-Access fabric design.

The typical underlay configuration provided by Cisco Catalyst Center includes the following features but can be modified based on specific design requirements:




- All switches act as Layer 3 switches.
- The switches use the IS-IS routing protocol.
- All links between switches (single links, EtherChannels) are routed Layer 3 links (not Layer 2 links).
- As a result, STP/RSTP is not needed, with the routing protocol instead choosing which links to use based on the IP routing tables.
- The equivalent of a traditional access layer switch—a fabric edge node—acts as the default gateway for the endpoint devices, rather than distribution switches.
- As a result, HSRP (or any FHRP) is no longer needed.

Figure 22-4 repeats the same physical design as in Figure 22-3 but shows the different features with the routed access design as configured using Cisco Catalyst Center.

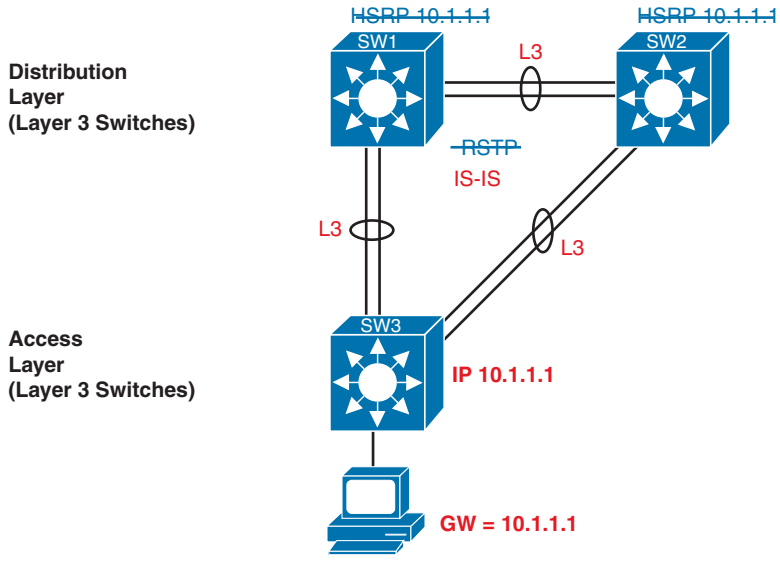



**Figure 22-4** *Cisco SD-Access Fabric Layer 3 Access Benefits*

**NOTE** Cisco Catalyst Center configures the underlay with consistent settings for each instance of Cisco SD-Access across an enterprise. This convention simplifies operation as an enterprise completes a migration to Cisco SD-Access.

## The Cisco SD-Access Overlay

When you first think of the Cisco SD-Access overlay, think of this kind of sequence. First, an endpoint sends a frame that will be delivered across the Cisco SD-Access network. The first fabric edge node to receive the frame encapsulates the frame in a new message—using the tunneling specification VXLAN—and forwards the frame into the fabric. Once the ingress fabric edge node has encapsulated the original frame in VXLAN, the other fabric nodes forward the frame based on the VXLAN tunnel details. The last fabric edge node in the path removes the VXLAN details, leaving the original frame, and forwards the original frame on toward the destination endpoint.

While the summary steps of some of Cisco SD-Access's overlay in the previous paragraph may sound like a lot of work, all that work happens in each switch's ASIC. So, while it is more complex to understand, there is no performance penalty for the switches to perform the extra work as it is done in hardware.

Cisco's choice of using VXLAN tunnels opened up many possibilities for a number of new networking features that did not exist without VXLAN. This next topic begins with a closer look at the VXLAN tunnels in the overlay, followed by a discussion of how Cisco SD-Access uses Locator/ID Separation Protocol (**LISP**) for endpoint discovery and location information needed to create the VXLAN tunnels.

### VXLAN Tunnels in the Overlay (Data Plane)

Cisco SD-Access has many additional needs beyond the simple message delivery—needs that let it provide improved functions. To that end, the fabric does not only route IP packets or switch Ethernet frames; it encapsulates incoming data link frames in a tunneling technology for delivery across the fabric network. VXLAN must adhere to the following functions in a Cisco SD-Access fabric:

Key Topic

- The VXLAN tunneling (the encapsulation and de-encapsulation) must be performed by the ASIC on each switch so that there is no performance penalty. (That is one reason for the Cisco SD-Access hardware compatibility list: the switches must have ASICs that can perform the work. These are called the Unified Access Data Plane [UADP] ASICs.)
- The VXLAN encapsulation must supply header fields that Cisco SD-Access needs for its features, so the tunneling protocol should be flexible and extensible, while still being supported by the switch ASICs.
- The tunneling encapsulation needs to encapsulate the entire data link frame instead of encapsulating the IP packet. That allows Cisco SD-Access to support Layer 2 forwarding features as well as Layer 3 forwarding features.

To achieve those goals, when creating Cisco SD-Access, Cisco chose the VXLAN protocol to create the tunnels used by Cisco SD-Access. When an endpoint (for example, an end-user computer) sends a data link frame into a fabric edge node, the ingress fabric edge node encapsulates the frame and sends it across a VXLAN tunnel to the egress fabric edge node, as shown in Figure 22-5.

Key Topic

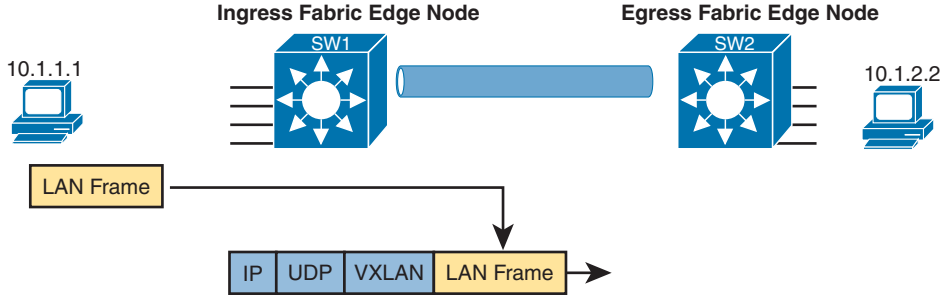



**Figure 22-5** *Fundamentals of VXLAN Encapsulation in Cisco SD-Access*

To support the VXLAN encapsulation, the underlay uses a separate IP address space as compared with the rest of the enterprise, including the endpoint devices that send data over the Cisco SD-Access network. The overlay tunnels use addresses from the enterprise address space. For instance, imagine an enterprise used these address spaces:

- 10.0.0.0/8: Entire enterprise
- 172.16.0.0/16: Cisco SD-Access underlay

To make that work, first the underlay would be built using the 172.16.0.0/16 IPv4 address space, with all links using addresses from that address space. As an example, Figure 22-6

shows a small Cisco SD-Access design, with four switches, each with one underlay IP address shown (from the 172.16.0.0/16 address space).

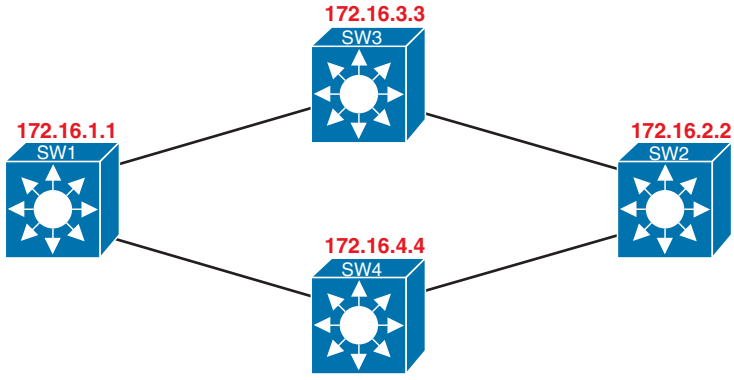



**Figure 22-6** *Cisco SD-Access Underlay Using 172.16.0.0*

The overlay tunnel creates a path between two fabric edge nodes in the overlay IP address space, which is in the same address space used by all the endpoints in the enterprise. Figure 22-7 emphasizes that point by showing the endpoints (PCs) on the left and right, with IP addresses in network 10.0.0.0/8, with the VXLAN overlay tunnel shown with addresses also from 10.0.0.0/8.

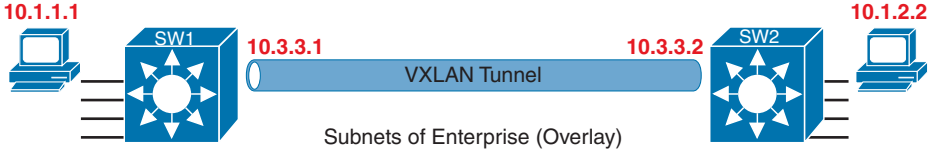



**Figure 22-7** *VXLAN Tunnel and Endpoints with IPv4 Addresses in the Same IPv4 Space*

### LISP for Overlay Discovery and Location (Control Plane)

Ignore Cisco SD-Access for a moment, and think about traditional Layer 2 switching and Layer 3 routing. How do their control planes work? In other words, how do these devices discover the possible destinations in the network, store those destinations, so that the data plane has all the data it needs when making a forwarding decision? To summarize:

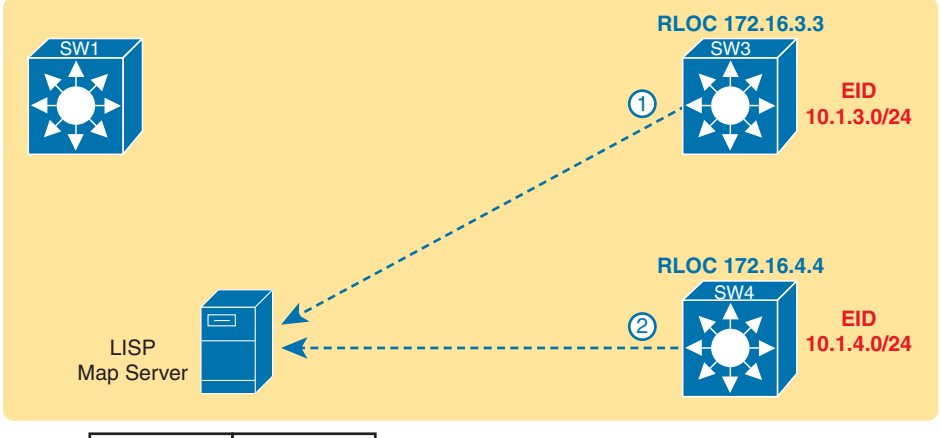
- Traditional Layer 2 switches learn possible destinations by examining the source MAC addresses of incoming frames, storing those MAC addresses as possible future destinations in the switch's MAC address table. When new frames arrive, the Layer 2 switch data plane then attempts to match the Ethernet frame's destination MAC address to an entry in its MAC address table.
- Traditional Layer 3 routers learn destination IP subnets using routing protocols, storing routes to reach each subnet in their routing tables. When new packets arrive, the Layer 3 data plane attempts to match the IP packet's destination IP address to some entry in the IP routing table.

Nodes in the Cisco SD-Access network do *not* do these same control plane actions to support endpoint traffic. Just to provide a glimpse into the process for the purposes of CCNA, consider this sequence, which describes one scenario:

- Fabric edge nodes—Cisco SD-Access nodes that connect to the edge of the fabric—learn the location of possible endpoints using traditional means, based on their MAC address, individual IP address, and by subnet, identifying each endpoint with an endpoint identifier (EID).
- The fabric edge nodes register the fact that the node can reach a given endpoint (EID) into a database called the LISP map server.
- The LISP map server keeps the list of endpoint identifiers (EIDs) and matching routing locators (RLOCs) (which identify the fabric edge node that can reach the EID).
- In the future, when the fabric data plane needs to forward a message, it will look for and find the destination in the LISP map server's database.

For instance, switches SW3 and SW4 in Figure 22-8 each just learned about different subnets external to the Cisco SD-Access fabric. As noted at Step 1 in the figure, switch SW3 sent a message to the LISP map server, registering the information about subnet 10.1.3.0/24 (an EID), with its RLOC setting to identify itself as the node that can reach that subnet. Step 2 shows an equivalent registration process, this time for SW4, with EID 10.1.4.0/24, and with R4's RLOC of 172.16.4.4. Note that the table at the bottom of the figure represents that data held by the LISP map server.

Key Topi



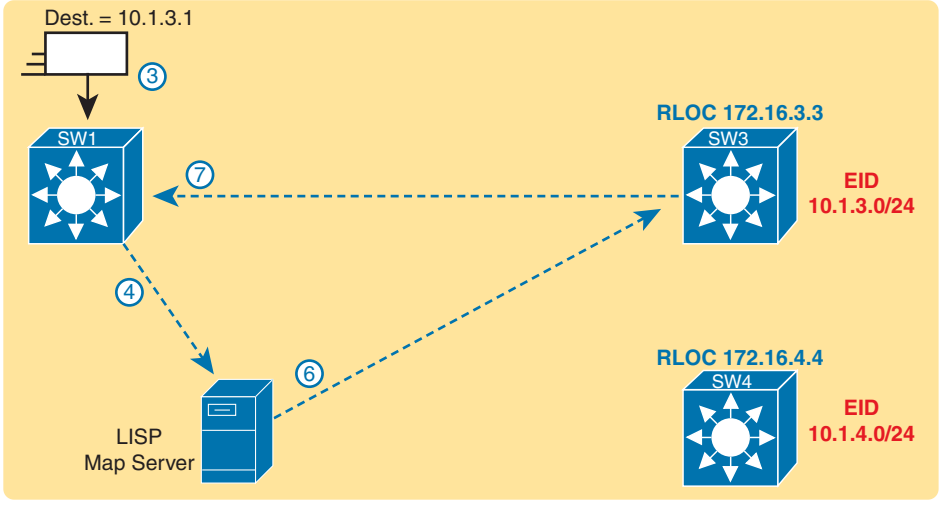

| | EID | RLOC |
|---|---|---|
| 1 | 10.1.3.0/24 | 172.16.3.3 |
| 2 | 10.1.4.0/24 | 172.16.4.4 |

**Figure 22-8** *Edge Nodes Register IPv4 Prefixes (Endpoint IDs) with the LISP Map Server*

When new incoming frames arrive, the ingress tunnel router (ITR)—the Cisco SD-Access fabric edge node that receives the new frame from outside the fabric—needs some help from

the control plane. To where should the ITR forward this frame? Because Cisco SD-Access always forwards frames in the fabric over some VXLAN tunnel, what tunnel should the ITR use when forwarding the frame? For the first frame sent to a destination, the ITR must follow a process like the following steps. These steps begin at Step 3, as a continuation of Figure 22-8, with the action referenced in Figure 22-9:

3. An Ethernet frame to a new destination arrives at ingress fabric edge node SW1 (upper left), and the switch does not know where to forward the frame.
4. The ingress node sends a message to the LISP map server asking if the LISP server knows how to reach IP address 10.1.3.1.
5. The LISP map server looks in its database and finds the entry it built back at step 1 in the previous figure, listing SW3's RLOC of 172.16.3.3.
6. The LISP map server contacts SW3—the node listed as the RLOC—to confirm that the entry is correct.
7. SW3 completes the process of informing the ingress fabric edge node (SW1) that 10.1.3.1 can be reached through SW3.

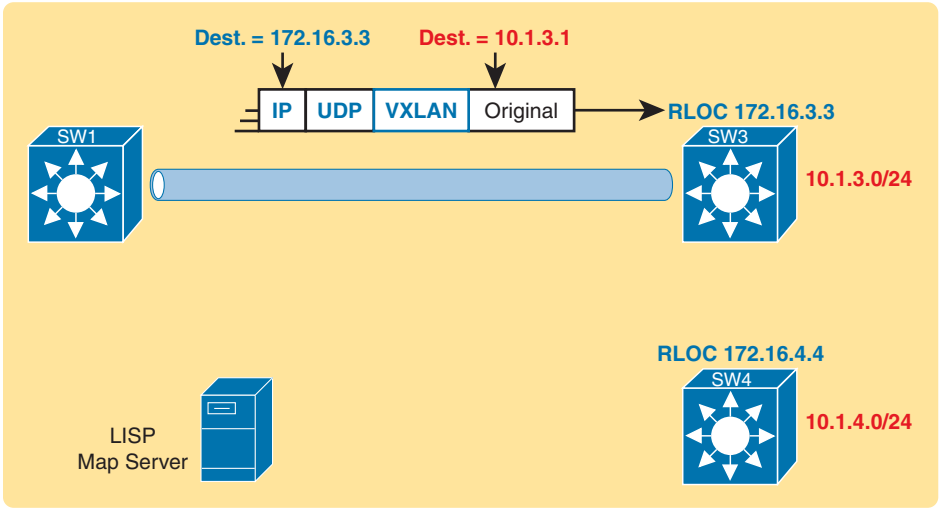



| | EID | RLOC |
|---|---|---|
| 5 | 10.1.3.0/24 | 172.16.3.3 |
| | 10.1.4.0/24 | 172.16.4.4 |

**Figure 22-9** *Ingress Tunnel Router SW1 Discovers Egress Tunnel Router SW3 Using LISP*

To complete the story, now that ingress node SW1 knows that it can forward packets sent to endpoint 10.1.3.1 to the fabric edge node with RLOC 172.16.3.3 (that is, SW3), SW1 encapsulates the original Ethernet frame as shown in Figure 22-9, with the original destination IP address of 10.1.3.1. It adds the IP, UDP, and VXLAN headers shown so it can deliver the message over the Cisco SD-Access network, with that outer IP header listing a destination IP

address of the RLOC IP address, so that the message will arrive through the Cisco SD-Access fabric at SW3, as shown in Figure 22-10.




| | EID | RLOC |
|---|---|---|
| 1 | 10.1.3.0/24 | 172.16.3.3 |
| 2 | 10.1.4.0/24 | 172.16.4.4 |

**Figure 22-10** *Ingress Tunnel Router (ITR) SW1 Forwards Based on LISP Mapping to SW3*

Now that you have a general overview of how a fabric works, how VXLAN encapsulation is used to allow for line rate switching in hardware between fabric edge nodes, and how LISP maps endpoints as they move between fabric edge nodes, let's go back to the host mobility use case of Cisco SD-Access.

Consider the following story:

When a host moves from one area of the network to another, the control-plane node (LISP mapping server) keeps track of that user's EID or identity and updates the other fabric edge nodes as they move from location to location. Think about this like a post office for mail distribution. If you move from your home to another home and do not inform the post office that you moved and provide them with your new address, your mail will be incorrectly delivered to your old home and be rendered useless. However, if you inform the post office of your new address by using a change of address form, your mail will be delivered to your new home without interruption. This is how host mobility works within a Cisco SD-Access campus fabric.

Because the identity of the user or host is known to the fabric, the reliance on IP addresses as a means to identify the user or host is no longer as important as it once was. This greatly simplifies matters when it comes to things such as security policies. For example, because

the network can know when you move from location or fabric edge to fabric edge, it can enforce security policies based on your identity versus your IP address. This means that regardless of what subnet you are in, what VLAN, what IP address you have, whether it is static or DHCP, your security policy follows the user. This simplifies many things such as access control lists that have to be constantly reviewed to ensure they are covering every subnet range you might move into or out of as well as all the protocols or applications you want to permit or deny access to. Because the network knows the identity of the user, the policy can follow them, providing greater security and reduced risk of human error of an access list missing an entry because a user moved into a different subnet.

At this point, you should have a basic understanding of how the Cisco SD-Access fabric works. The underlay includes all the switches and links, along with IP connectivity, as a basis for forwarding data across the fabric. The overlay adds a different level of logic, with end-point traffic flowing through VXLAN tunnels. This chapter has not mentioned any reasons that Cisco SD-Access might want to use these tunnels, but you will see one example by the end of the chapter. Suffice it to say that with the flexible VXLAN tunnels, Cisco SD-Access can encode header fields that let Cisco SD-Access create new networking features, all without suffering a performance penalty, as all the VXLAN processing happens in the UADP ASIC.

The next section of this chapter focuses on Cisco Catalyst Center and its role in managing and controlling Cisco SD-Access fabrics.

## Cisco Catalyst Center and Cisco SD-Access Operation

Cisco Catalyst Center (www.cisco.com/go/catalystcenter) has two notable roles:

- As the controller in a network that uses Cisco SD-Access
- As a network management platform for traditional (non-Cisco SD-Access) network devices

The first role as the Cisco SD-Access network controller gets most of the attention and is the topic of discussion in this second of the three major sections of this chapter. Cisco SD-Access and Cisco Catalyst Center go together, work closely together, and any serious use of Cisco SD-Access requires the use of Cisco Catalyst Center. At the same time, however, Cisco Catalyst Center can manage traditional network devices; the final major section of the chapter works through some comparisons.

### Cisco Catalyst Center

Cisco Catalyst Center exists as a software application that Cisco delivers pre-installed on a Cisco Catalyst Center appliance or as a virtual appliance that can be run in a hypervisor environment. The software follows the same general controller architecture concepts as described in Chapter 21. Figure 22-11 shows the general ideas.

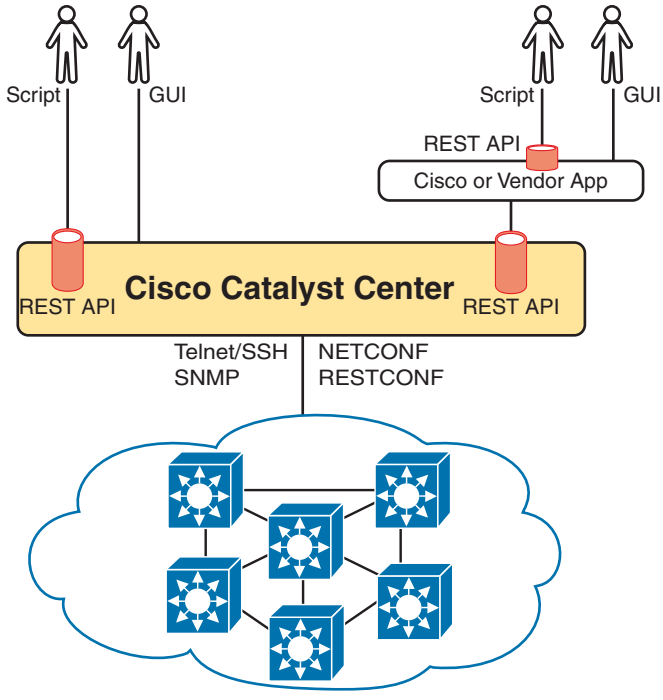



**Figure 22-11** *Cisco Catalyst Center with Northbound and Southbound Interfaces*

Cisco Catalyst Center includes a robust northbound REST API along with a series of southbound APIs. For most of us, the northbound API matters most, because as the user of Cisco SD-Access networks, you interact with Cisco SD-Access using Cisco Catalyst Center's northbound REST API or the GUI interface. (Chapter 23, "Understanding REST and JSON," discusses the concepts behind REST APIs in more detail.)

Cisco Catalyst Center supports several southbound APIs so that the controller can communicate with the devices it manages. You can think of these as two categories:

- Protocols to support traditional networking devices/software versions: Telnet, SSH, SNMP
- Protocols to support more recent networking devices/software versions: NETCONF, RESTCONF

Cisco Catalyst Center needs the older protocols to be able to support the vast array of older Cisco devices and OS versions. Over time, Cisco has been adding support for NETCONF and RESTCONF to their more current hardware and software.

## Cisco Catalyst Center and Scalable Groups

Cisco SD-Access creates many interesting new and powerful features beyond how traditional campus networks work. Cisco Catalyst Center not only enables an easier way to configure and operate those features, but it also completely changes the operational model. While the scope of CCNA does not allow us enough space to explore all of the features of Cisco

SD-Access and Cisco Catalyst Center, this next topic looks at one feature as an example: scalable groups.

### Issues with Traditional IP-Based Security

Imagine the life of one traditional IP ACL in an enterprise. Some requirements occurred, and an engineer built the first version of an ACL with three access control entries (ACEs)—that is, **access-list** commands—with a **permit any** at the end of the list. Months later, the engineer added two more lines to the ACL, so the ACL has the number of ACEs shown in Figure 22-12. The figure notes the lines added for requests one and two with the circled numbers in the figure.

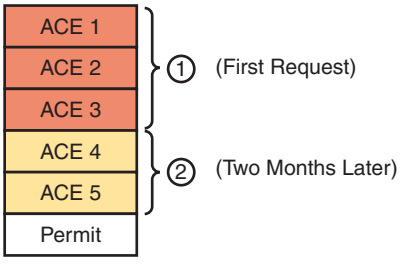



**Figure 22-12** *Lines (ACEs) in an ACL After Two Changes*

Now think about that same ACL after four more requirements caused changes to the ACL, as noted in Figure 22-13. Some of the movement includes

- The ACEs for requirement two are now at the bottom of the ACL.
- Some ACEs, like ACE 5, apply to more than one of the implemented requirements.
- Some requirements, like requirement number five, required ACEs that overlap with multiple other requirements.

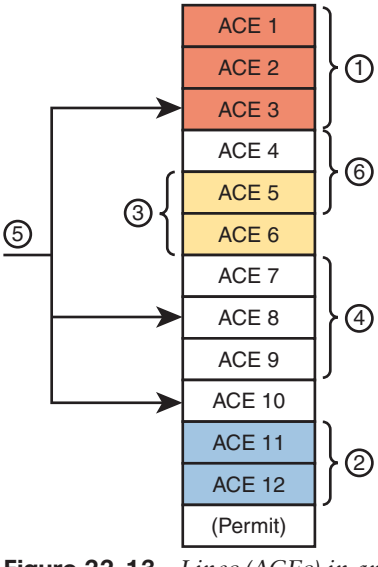



**Figure 22-13** *Lines (ACEs) in an ACL After Six Changes*

Now imagine your next job is to add more ACEs for the next requirement (7). However, your boss also told you to reduce the length of the ACL, removing the ACEs from that one change made last August—you remember it, right? Such tasks are problematic at best.

With the scenario in Figure 22-13, no engineer could tell from looking at the ACL whether any lines in the ACL could be safely removed. You never know if an ACE was useful for one requirement or for many. If a requirement was removed, and you were even told which old project caused the original requirement so that you could look at your notes, you would not know if removing the ACEs would harm other requirements. Most of the time, ACL management suffers with these kinds of issues:

- ACEs cannot be removed from ACLs because of the risk of causing failures to the logic for some other past requirement.
- New changes become more and more challenging due to the length of the ACLs.
- Troubleshooting ACLs as a system—determining whether a packet would be delivered from end to end—becomes an even greater challenge.

### Cisco SD-Access Security Is Based on User Groups

Imagine you could instead enforce security without even thinking about IP address ranges and ACLs. Cisco SD-Access does just that, with simple configuration, and the capability to add and remove the security policies at will.

First, for the big ideas. Imagine that over time, using Cisco SD-Access, six different security requirements occurred. For each project, the engineer would define the policy with Cisco Catalyst Center, either with the GUI or with the API. Then, as needed, Cisco Catalyst Center would configure the devices in the fabric to enforce the security, as shown in Figure 22-14.

Key Topic

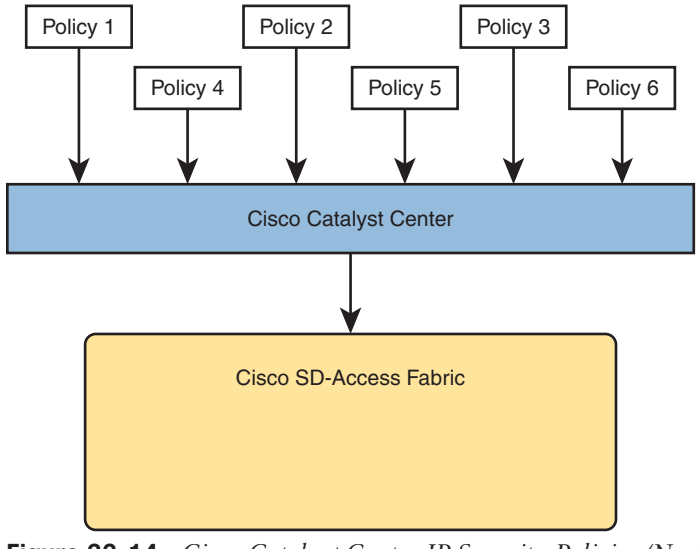



**Figure 22-14** *Cisco Catalyst Center IP Security Policies (Northbound) to Simplify Operations*

**NOTE** The model in Figure 22-14 helps demonstrate the concept of intent-based networking (IBN). The engineer configures the intent or outcome desired from the network—in this case, a set of security policies. The controller communicates with the devices in the network, with the devices determining exactly what configuration and behavior are necessary to achieve those intended policies.

The Cisco SD-Access policy model solves the configuration and operational challenges with traditional ACLs. In fact, all those real issues with managing IP ACLs on each device are no longer issues with Cisco SD-Access's group-based security model. For instance:

- The engineer can consider each new security requirement separately, without analysis of an existing (possibly lengthy) ACL.
- Each new requirement can be considered without searching for all the ACLs in the likely paths between endpoints and analyzing each and every ACL.
- Cisco Catalyst Center (and related software) keeps the policies separate, with space to keep notes about the reason for the policy.
- Each policy can be removed without fear of impacting the logic of the other policies.

Cisco SD-Access and Cisco Catalyst Center achieve this particular feature by tying security to groups of users, called scalable groups, with each group assigned a **scalable group tag (SGT)**. Then the engineer configures a grid that identifies which SGTs can send packets to which other SGTs. For instance, the grid might include SGTs for an employee group, the Internet (for the enterprise's WAN routers that lead to the Internet), partner employees, and guests, with a grid like the one shown in Table 22-2.

**Table 22-2** Access Table for Cisco SD-Access Scalable Group Access

| Source\Dest. | Employee | Internet | Partner | Guest |
|---|---|---|---|---|
| Employee | N/A | Permit | Permit | Deny |
| Internet | Permit | N/A | Permit | Permit |
| Partner | Permit | Permit | N/A | Deny |
| Guest | Deny | Permit | Deny | N/A |

To link this security feature back to packet forwarding, consider when a new endpoint tries to send its first packet to a new destination. The ingress Cisco SD-Access fabric edge node starts a process by sending messages to Cisco Catalyst Center. Cisco Catalyst Center then works with security tools in the network, like Cisco's Identity Services Engine (ISE), to identify the users and then match them to their respective SGTs. Cisco Catalyst Center then checks the logic similar to Table 22-2. If Cisco Catalyst Center sees a permit action between the source/destination pair of SGTs, Cisco Catalyst Center directs the edge nodes to create the VXLAN tunnel, as shown in Figure 22-15. If the security policies state that the two SGTs should not be allowed to communicate, then Cisco Catalyst Center does not direct the fabric to create the tunnel, and the packets do not flow.

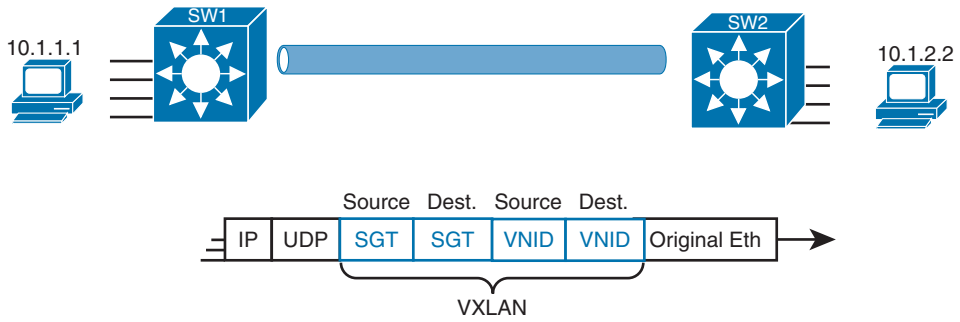



**Figure 22-15** *VXLAN Header with Source and Destination SGTs and VNIDs Revealed*

**NOTE** The figure gives a brief insight into why Cisco SD-Access goes to the trouble of using VXLAN encapsulation for its data plane, rather than performing traditional Layer 2 switching or Layer 3 routing. The VXLAN header has great flexibility—in this case, used to define both a source and destination SGT, matching Cisco SD-Access's desired logic of allowing a subset of source/destination SGTs in the Cisco SD-Access fabric.

The operational model with scalable groups greatly simplifies security configuration and ongoing maintenance of the security policy, while focusing on the real goal: controlling access based on user, as mentioned previously in the section "LISP for Overlay Discovery and Location (Control Plane)." From a controller perspective, the fact that Cisco Catalyst Center acts as much more than a management platform, and instead as a controller of the activities in the network, makes for a much more powerful set of features and capabilities.

## Cisco Catalyst Center as a Network Management Platform

Typically, a network management system (NMS) for the enterprise will include many useful features to help simplify the daily operation of your network. Although specific features will vary on a per-platform or vendor basis, many, if not most, of the following features can be found in a typical NMS:

- **Single-pane-of-glass:** Provides one GUI from which to launch all functions and features
- **Discovery, inventory, and topology:** Discovers network devices, builds an inventory, and arranges them in a topology map
- **Entire enterprise:** Provides support for traditional enterprise LAN, WAN, and data center management functions
- **Methods and protocols:** Uses SNMP, SSH, and Telnet, as well as CDP and LLDP, to discover and learn information about the devices in the network
- **Lifecycle management:** Supports different tasks to install a new device (day 0), configure it to be working in production (day 1), and perform ongoing monitoring and make changes (day *n*)

- **Application visibility:** Simplifies QoS configuration deployment to each device
- **Converged wired and wireless:** Enables you to manage both the wired and wireless LAN from the same management platform
- **Software Image Management (SWIM):** Manages software images on network devices and automates updates
- **Plug-and-Play:** Performs initial installation tasks for new network devices after you physically install the new device, connect a network cable, and power on

Cisco tends to shy away from specific product details in most of its career certifications, so it helps to think in general about network management products. It also helps to think about specific products—but temper that by focusing on the more prominent features and major functions.

## Cisco Catalyst Center Similarities to Traditional Management

If you read the user's guide for Cisco Catalyst Center and look through all the features, you will find some of them are similar in nature to the traditional management features listed at the beginning of this section. For example, Cisco Catalyst Center can not only discover network devices like other network management systems but can also create a live topology map view of your network. Human operators (rather than automated processes) often start with the topology map, expecting at-a-glance notices (flashing lights, red colors) to denote issues in the network. In addition, most network management systems provide event and threshold alerting via email or SMS. As an example, Figure 22-16 shows a topology map from Cisco Catalyst Center.

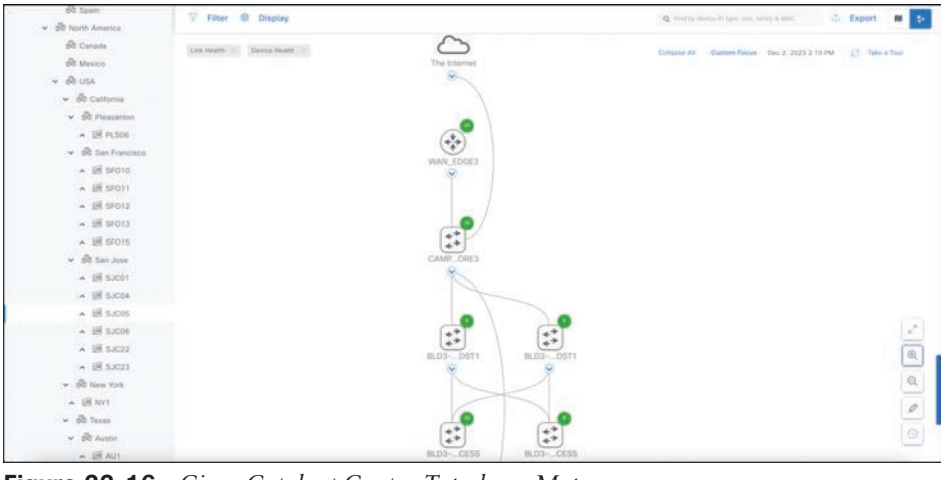



**Figure 22-16** *Cisco Catalyst Center Topology Map*

The GUI mechanisms are relatively intuitive, with the ability to click into additional or less detail. Figure 22-17 shows a little more detail after hovering over and clicking on one of the nodes in the topology from Figure 22-16, typical actions and results in many management products.

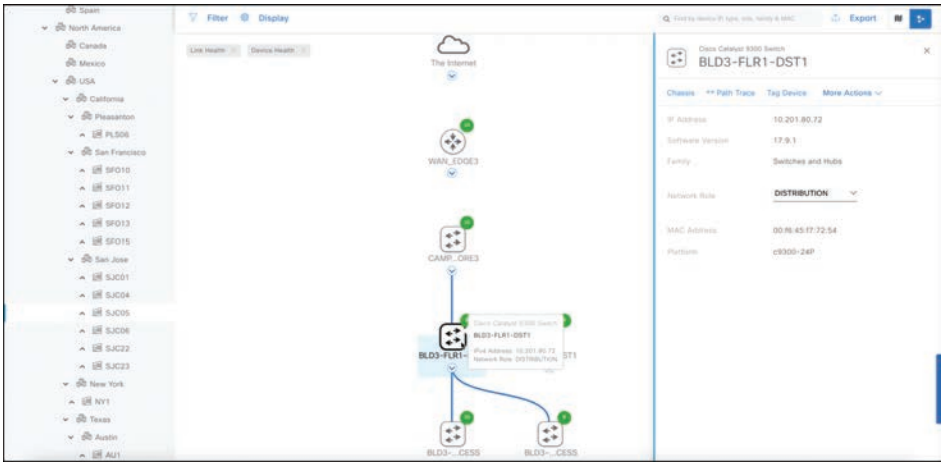



**Figure 22-17** *Hover and Click Details About a Single Cisco Catalyst 9300 Switch from Cisco Catalyst Center*

I encourage you to take some time to use and watch some videos about Cisco Catalyst Center. The "Chapter Review" section for this chapter on the companion website lists some links for good videos. Also, start at https://developer.cisco.com and look for Cisco Catalyst Center sandbox labs to find a place to experiment with Cisco Catalyst Center.

## Cisco Catalyst Center and Differences with Traditional Management

In a broad sense, there are several fundamental differences between Cisco Catalyst Center and traditional network management platforms. The largest difference: Cisco Catalyst Center supports Cisco SD-Access, whereas other management apps do not. So, think of network management systems as analogous to traditional device management. Cisco Catalyst Center has many similar features, but it also has more advanced features that focus on newer technology solutions like Cisco SD-Access support.

In terms of intent and strategy, Cisco focuses their development of Cisco Catalyst Center features toward simplifying the work done by enterprises, with resulting reduced costs and much faster deployment of changes. Cisco Catalyst Center features help make initial installation easier, simplify the work to implement features that traditionally have challenging configuration, and use tools to help you notice issues more quickly. Some of the features unique to Cisco Catalyst Center include

Key Topic

- **Application policy:** Deploys QoS, one of the most complicated features to configure manually, with just a few simple choices from Cisco Catalyst Center
- **Encrypted Traffic Analytics (ETA):** Enables Cisco DNA to use algorithms to recognize security threats even in encrypted traffic
- **Device 360 and Client 360:** Gives a comprehensive (360-degree) view of the health of the device

- **Network time travel:** Shows past client performance in a timeline for comparison to current behavior
- **Path trace:** Discovers the actual path packets would take from source to destination based on current forwarding tables

Just to expound on one feature as an example, Cisco Catalyst Center's Path Trace feature goes far beyond a traditional management application. A typical network management app might show a map of the network and let you click through to find the configuration on each device, including ACLs. The path trace feature goes much further. The Cisco Catalyst Center user (from the GUI or the API) specifies a source and destination host and optionally, the transport protocol and ports. Then the path trace feature shows a map of the path through the network and shows which ACLs are in the path, and whether they would permit or deny the packet.

All of Cisco's Digital Network Architecture sets about to help customers reach some big goals: reduced costs, reduced risks, better security and compliance, faster deployment of services through automation and simplified processes, and the list goes on. Cisco Catalyst Center plays a pivotal role, with all the functions available through its robust northbound API, and with its intent-based networking approach for Cisco SD-Access. Cisco Catalyst Center represents the future of network management for Cisco enterprises.

## Artificial Intelligence (AI), Machine Learning (ML), and Operational Management

Cisco includes one exam topic in the CCNA 200-301 Version 1.1 blueprint that mentions Artificial Intelligence (AI):

6.4 Explain AI (generative and predictive) and machine learning in network operations

Tremendous amounts of industry buzz have emerged in the area of artificial intelligence and machine learning. Some of the early adopters sought benefits to alleviate some of the strain of their day-to-day operations while running the network. Before we can understand some of the benefits from AI and ML, we must first understand their definitions.

**Artificial Intelligence** is defined as a computer or system that can do tasks that typically require human thinking, such as learning, problem-solving, and understanding various languages. There are two major types of AI:




- **Narrow AI** is a type of AI that is designed to perform a specific task or a set of closely related tasks. Narrow AI thrives in a well-defined and limited scope. However, Narrow AI lacks the broad cognitive capabilities of human intelligence. Some common examples of Narrow AI include speech recognition systems or applications, image recognition software, and virtual personal assistants such as Amazon Alexa, Apple Siri, and Google Assistant. These assistants are often used for typical tasks like playing music, showing your calendar, or recalling a recipe when cooking.
- **Generative AI**, which could learn, be taught, and could potentially make decisions that consider facts and experiences similar to humans, is another form of AI.

AI also includes technologies like Machine Learning (ML). ML is used in numerous fields from healthcare to network operations and security. ML is a branch of AI that primarily focuses on the development of algorithms and complex data models. These models enable computers to learn from various data sources and not only make predictions but also make their own decisions without being explicitly programmed to. The goal of machine learning is to create systems that can continuously and automatically improve as they are fed more data or information sets.

In traditional computer programming, humans write explicit instructions to perform a task. However, in machine learning, there is a dichotomy. The computer can learn specific patterns and relationships from different data sources, which allows the computer or "AI" to make predictions on new data sets that it hasn't even seen yet. This is what the definition of **Predictive AI** is—being able to surmise what you are trying to accomplish with the least amount of prompts or input from the user.

Let's contemplate some of the benefits and use cases for the different forms of AI. Starting with a common example of Narrow AI, we can easily see the value of having virtual personal assistants. One less obvious use case is the ability to run home automation workflows and bind them to various schedules. This creates the "Smart Building" concept. Connecting and controlling various "things" such as lighting, thermostats, garage doors, and electronics typically introduces some form of intrinsic value for the user, especially when leveraging automation.

One very common advantage is creating a routine or automation that will turn off indoor lights, close the garage door, and set the temperature to a lower setting automatically when you leave your home for the day. The cost savings across all these domains can be significant. Especially if you extrapolate that out across the year. It is very frequent that lights and heat or garage doors get left on or open. Now there is a method to ensure that doesn't happen. In addition, to take this example one step further, you can automate turning on your home security system or alarm as part of this routine. It is pretty easy to see the value of how using a Narrow AI virtual personal assistant tied to automation techniques can provide real-world use cases.

Generative AI is where most of the attention seems to be these days, and for great reason. Generative AI has exploded as the next big thing and also the thing that has most of the industry cautious about the technology. Let's use the example of Microsoft's ChatGPT.

ChatGPT is a conversational artificial intelligence platform. Hence the term *chat*. Users can use text-based chat to ask ChatGPT endless amounts of questions or what the industry has termed *prompts*. This form of generative AI can be fed information to learn and can be taught and evolve. There's a lot of debate on the pros and cons of using generative AI. One example is using generative AI to create music and songs. Some think this takes the artistry out of the music creation process. But for some artists who want to quickly get some music built and distributed, this can be a mechanism to quickly and broadly create digital art. Especially if they don't have songwriting skills but have lyrics and a rhythm. Let's take a look at what ChatGPT does when having it create a song. Figure 22-18 demonstrates how to prompt ChatGPT from the initial chat window to create lyrics to a song about CCNA study habits.

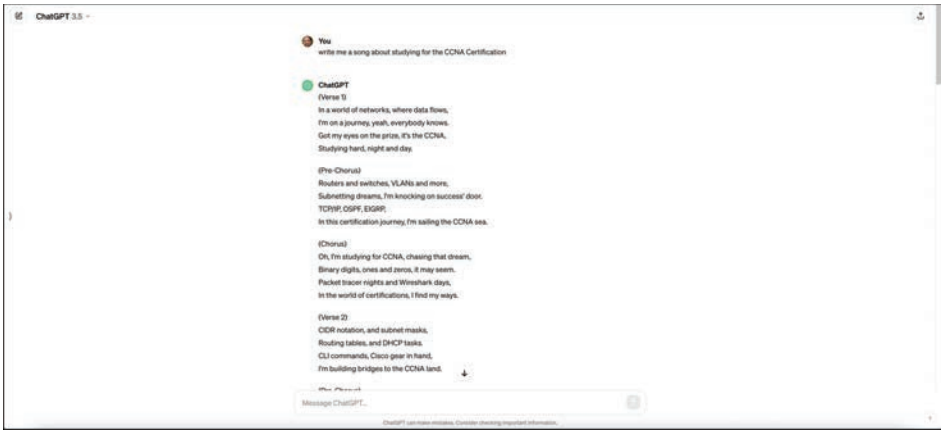



**Figure 22-18** *ChatGPT 3.5 CCNA Song Prompt*

Here are the complete lyrics to the song ChatGPT created from the following prompt:

"*write me a song about studying for the CCNA Certification*"

(Verse 1)
In a world of networks, where data flows,
I'm on a journey, yeah, everybody knows.
Got my eyes on the prize, it's the CCNA,
Studying hard, night and day.

(Pre-Chorus)
Routers and switches, VLANs and more,
Subnetting dreams, I'm knocking on success' door.
TCP/IP, OSPF, EIGRP,
In this certification journey, I'm sailing the CCNA sea.

(Chorus)
Oh, I'm studying for CCNA, chasing that dream,
Binary digits, ones and zeros, it may seem.
Packet tracer nights and Wireshark days,
In the world of certifications, I find my ways.

(Verse 2)
CIDR notation, and subnet masks,
Routing tables, and DHCP tasks.
CLI commands, Cisco gear in hand,
I'm building bridges to the CCNA land.

(Pre-Chorus)
Spanning Tree Protocol, like branches on a tree,
Switches blinking, like stars, guiding me.
NAT, ACLs, and security,
I'm locking down knowledge, setting my own decree.

(Chorus)
Oh, I'm studying for CCNA, chasing that dream,
Binary digits, ones and zeros, it may seem.
Packet tracer nights and Wireshark days,
In the world of certifications, I find my ways.

(Bridge)
Late-night labs, under the moon's soft glow,
Simulations, configurations, letting my knowledge grow.
IP addressing, subnetting schemes,
I'm writing my fate in these CCNA dreams.

(Verse 3)
BGP, MPLS, and QoS,
In this networking journey, I face no loss.
Concepts intertwine like cables in a rack,
I'll conquer the CCNA, ain't no turning back.

(Pre-Chorus)
VTP and HSRP,
In this realm of routers, I'm sailing free.
Through the OSI layers, like climbing a stair,
I'll reach the summit, breathe the CCNA air.

(Chorus)
Oh, I'm studying for CCNA, chasing that dream,
Binary digits, ones and zeros, it may seem.
Packet tracer nights and Wireshark days,
In the world of certifications, I find my ways.

(Outro)
In the exam room, I'll stand tall,

Answers flowing like a waterfall.

CCNA, I'm ready to show,

In the world of networks, I'll let my knowledge glow.

ChatGPT is a very creative tool that can build out songs or configurations or anything you can think to ask it. The turnaround time is within seconds for it to create whatever you prompt it to create. There are free versions of ChatGPT like the one used in this chapter. There is also a paid version that gives you more features and access to the latest database of information and API access. These styles of data stores are called **Large Language Models (LLMs)**. The industry is seeing the need for privatized LLMs and customers are starting to build their own to house private data or knowledge bases. LLMs use a significant amount of memory, CPU, and GPU resources to look up and parse these data sources. However, for the right value to the organization, it can easily be seen as an investment to the business.

Let's now pick out a use case for ChatGPT on learning and educating yourself on various CCNA topics. It's always best to validate the source of the data, because you can find quite a lot of information on a specific subject that you may be trying to learn, but it isn't always accurate. For example, let's use ChatGPT to create a summary access list for the following subnets:

192.168.10.0/24

192.168.16.0/24

192.168.53.0/24

192.168.64.0/24

**NOTE** Remember to validate anything you get from a generative AI platform especially if you plan to use any of the data for production purposes. Many issues have been created by using code or configuration from generative AI platforms and not first checking the validity of the code or configuration before deployment.

Figure 22-19 showcases using ChatGPT to summarize the previously mentioned subnets in order to use an access list that will match all four subnets.

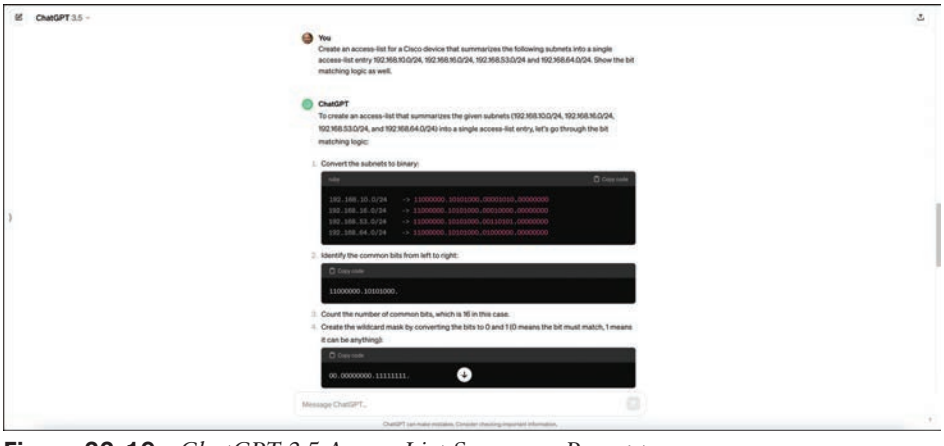



**Figure 22-19** *ChatGPT 3.5 Access List Summary Prompt*

We can see from the initial prompt and response the four subnets we used as an example. We also can see that ChatGPT provided the bit-matching logic behind its decision on the summary to use based on the additional prompt *Show the bit matching logic as well* that we fed it. The four subnets and their bit boundaries are listed here:

192.168.10.0/24 = 11000000.10101000.00001010.00000000

192.168.16.0/24 = 11000000.10101000.00010000.00000000

192.168.53.0/24 = 11000000.10101000.00110101.00000000

192.168.64.0/24 = 11000000.10101000.01000000.00000000

Based on this output, we can see that the first 17 bits match identically. However, ChatGPT selected using a /16-bit mask vs. a /17-bit mask. Looking at Figure 22-20, we can see the rest of the chat response and confirm that it has selected 192.168.0.0/16 as the summary address to use along with the appropriate commands to configure the access list on a Cisco device.




**Figure 22-20** *ChatGPT 3.5 Access List Summary Prompt, Continued*

Although this is a perfectly acceptable solution to the summary access list prompt that we fed ChatGPT, one could argue that it is not the most efficient access list entry that could have been provided. A more accurate access list would be to use the following as shown in Figure 22-21:

```
access-list access-list-number permit ip 192.168.0.0 0.0.127.255 any
```

Notice that in the initial prompt, ChatGPT said that the first 16 bits were common, even though if you count them, the first 17 bits are common. This is one of the reasons we mention to validate everything you get as a response from any generative AI tool. It can be an incredible time saver. But as a wise person once mentioned: "You can automate failure just as fast as you can automate success."




**Figure 22-21** *ChatGPT 3.5 Access List Summary Prompt with 17-bit Mask*

**NOTE** The version of ChatGPT used during the writing of this book may differ from the version you use or the most current version available. Features and response quality might have changed since this book was published.

There are many ways to operationalize the use of AI for networking. One of the most common methods is something called Artificial Intelligence for IT Operations (**AI Ops**). AI Ops leverages AI and ML to optimize and automate various aspects of IT operations. The primary goal of AI Ops is to improve the efficiency, reliability, and performance of IT-related systems while reducing manual interaction. This in turn should reduce human error as AI Ops takes advantage of proven and pre-tested workflows or configurations that the user specifies as beneficial to the operational process. The following list are some of the most common use cases for AI Ops:

- **Automation:** Taking routine tasks and reducing human intervention using automated processes
- **Monitoring and Data Analytics:** Providing constant analysis of data by using advanced tools to identify patterns and anomalies
- **Predictive Analysis:** Leveraging machine learning to proactively predict issues by analyzing historical data and patterns
- **Collaboration and Communication:** Promoting information sharing and troubleshooting techniques across IT operations as a whole
- **Root Cause Analysis:** Determining the underlying causes of problems by having deep visibility into the various areas of an IT environment

Automation is typically the low hanging fruit when it comes to AI Ops. Many organizations would benefit from having more automated workflows and procedures. Most businesses typically have some charter to move to a more automated operational process. Automation is one of the most common ways to get started with AI Ops.

However, to understand what is happening in an IT environment, the first step is to be able to "see" the environment as a whole. Monitoring the network and associated applications is the best way to determine what the network's baseline is. *Baseline* is another way of describing how the network is functioning normally with average traffic flows, average amount of users, and average data transactions. By understanding what the network is doing with a baseline, it's then easier to determine when something behaves out of the ordinary. Consider this example: the network is performing at a baseline and all of a sudden hundreds of gigs worth of data are being out pulsed to another country. This could be indicative of a data leak or malicious behavior that needs to be investigated. Without having a baseline to compare this behavior to, it could be difficult to identify the issue or its legitimacy.

Having the ability to do predictive analytics is critical in this day and age. Once upon a time, there was a bandwidth issue and the internet link was pegged at 100% utilization. By having the appropriate monitoring and analysis tools the issue was identified. The root cause was that it was the FIFA World Cup Tournament and many users were streaming the games on their internal workstations. This impacted legitimate business transactions from functioning properly because the link was so saturated. This insight gave the IT operations staff the ability to forecast out what could happen during big events like this and how to prevent it from happening again by way of policy.

When there is an issue that is impacting the network, many organizations jump into an "all hands on deck mode" or meet in a "war room" to bring together all areas of the IT teams. This is to enhance communication with each other and to minimize the impact of the issue to the business. Often, AI Ops can be used to help with this collaborative approach. For example, it is very common to see collaboration tools being used to communicate in customer environments. This is where the term "**Chat Ops**" comes from. Being able to leverage a Chat function of a tool to communicate with various teams and have video chats to discuss the current state of the network or issues that were identified can significantly streamline the path to resolution. These techniques typically lead to faster remediation and open communications between the different areas of the business.

Finally, AI Ops can help lead the IT operations staff to the Root Cause Analysis (RCA) or Root Cause of Failure (RCF) by determining what went wrong in the first place. By having tools like this in place, the process of getting to the resolution becomes a documented and repeatable procedure to follow in the event of a future outage or failure scenario.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 22-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 22-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |

## Review All the Key Topics

Key Topic

**Table 22-4** Key Topics for Chapter 22

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Definitions for underlay, overlay, and fabric | 497 |
| Figure 22-2 | Cisco SD-Access overlay and underlay | 499 |
| List | Cisco SD-Access fabric edge, fabric border, and fabric control-plane node roles | 500 |
| List | Attributes of the Cisco SD-Access underlay | 502 |
| List | Cisco SD-Access VXLAN tunneling benefits | 504 |
| Figure 22-5 | VXLAN encapsulation process with Cisco SD-Access | 504 |
| Figure 22-8 | Registering Cisco SD-Access endpoint IDs (EIDs) with the map server | 506 |
| Figure 22-14 | Cisco Catalyst Center shown controlling the fabric to implement group-based security | 512 |
| List | Features unique to Cisco Catalyst Center | 516 |
| List | Artificial intelligence, large language models | 517 |

## Key Terms You Should Know

AI Ops, Artificial Intelligence, Chat Ops, Cisco Catalyst Center, Cisco SD-Access, fabric, fabric edge node, Generative AI, Large Language Models (LLM), LISP, Narrow AI, overlay, Predictive AI, scalable group tag (SGT), Software-Defined Access, underlay, VXLAN



CHAPTER 23

# Understanding REST and JSON

**This chapter covers the following exam topics:**

- **6.0 Automation and Programmability**
  - **6.5 Describe characteristics of REST-based APIs (CRUD, HTTP verbs, and data encoding)**
  - **6.7 Recognize components of JSON encoded data**

To automate and program networks, some automation software does several tasks. The software analyzes data in the form of variables, makes decisions based on that analysis, and then may take action to change the configuration of network devices or report facts about the state of the network.

The different automation functions reside on different devices: the network engineer's device, a server, a controller, and the various network devices themselves. For these related automation processes to work well, all these software components need useful well-defined conventions to allow easy communication between software components.

This chapter focuses on two conventions that allow automation software to communicate. The first major section discusses application programming interfaces (APIs), specifically APIs that follow a style called REpresentational State Transfer (REST). APIs of any kind create a way for software applications to communicate, while RESTful APIs (APIs that use REST conventions) follow a particular set of software rules. Many APIs used in network automation today use REST-based APIs.

The second half of the chapter focuses on the conventions and standards for the data variables exchanged over APIs, with a focus on one: JavaScript Object Notation (JSON). If REST provides one standard method of how two automation programs should communicate over a network, JSON then defines how to communicate the variables used by a program: the variable names, their values, and the data structures of those variables.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 23-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| REST-based APIs | 1–3 |
| Data Serialization and JSON | 4–6 |

1. Which of the following are required attributes of a REST-based API? (Choose two answers.)
   a. Uses HTTP
   b. Objects noted as to whether they can be cached
   c. Stateful operation
   d. Client/server architecture
2. Which answers list a matching software development CRUD action to an HTTP verb that performs that action? (Choose two answers.)
   a. CRUD create and HTTP PATCH
   b. CRUD update and HTTP PATCH
   c. CRUD delete and HTTP PUT
   d. CRUD read and HTTP GET
3. Examine the following URI that works with a Cisco DNA Controller:

   ```
   https://dnac.example.com/dna/intent/api/v1/
   network-device?managementIPAddress=10.10.22.74
   ```

   Which part of the URI, per the API documentation, is considered to identify the resource but not any parameters?
   a. https://
   b. dna.example.com
   c. dna/intent/api/v1/network-device
   d. managementIPAddress=10.10.22.74
4. Which of the following data serialization and data modeling languages would most likely be used in a response from a REST-based server API used for networking applications? (Choose two answers.)
   a. JSON
   b. YAML
   c. JavaScript
   d. XML
5. Which answers correctly describe the format of the JSON text below? (Choose two answers.)

   ```
   { "myvariable":[1,2,3] }
   ```

   a. One JSON object that has one key:value pair
   b. One JSON object that has three key:value pairs
   c. A JSON object whose value is a second JSON object
   d. A JSON object whose value is a JSON array

6. Which answers refer to JSON values rather than JSON keys as found in the sample JSON data? (Choose two answers.)

```
{
    "response": {
        "type": "Cisco Catalyst 9300 Switch",
        "family": "Switches and Hubs",
        "role": "ACCESS",
        "managementIpAddress": "10.10.22.66"
    }
}
```

   a. "response"
   b. "type"
   c. "ACCESS"
   d. "10.10.22.66"

# Foundation Topics

## REST-Based APIs

Applications use *application programming interfaces (APIs)* to communicate. To do so, one program can learn the variables and data structures used by another program, making logic choices based on those values, changing the values of those variables, creating new variables, and deleting variables. APIs allow programs running on different computers to work cooperatively, exchanging data to achieve some goal.

In an API software world, some applications create an API, with many other applications using (consuming) the API. Software developers add APIs to their software so other application software can make use of the first application's features.

When writing an application, the developer will write some code, but often the developer may do a lot of work by looking for APIs that can provide the data and functions, reducing the amount of new code that must be written. As a result, much of modern software development centers on understanding and learning new APIs, along with the available libraries (prebuilt software that can be used to accomplish tasks rather than writing the equivalent from scratch).

Several types of APIs exist, each with a different set of conventions to meet different needs. The CCNA blueprint mentions one type of API—**REpresentational State Transfer (REST)**—because of its popularity as a type of API in networking automation applications. This first major section of the chapter takes a closer look at REST-based APIs.

### REST-Based (RESTful) APIs

**REST APIs** follow a set of foundational rules about what makes a REST API and what does not. First, from a literal perspective, REST APIs include the six attributes defined a few decades back by its creator, Roy Fielding. (You can find a good summary at https://restfulapi.net). Those six attributes are

Key Topic

- Client/server architecture
- Stateless operation
- Clear statement of cacheable/uncacheable
- Uniform interface
- Layered
- Code-on-demand

The first three of these attributes get at the heart of how a REST API works. You can more easily see these first three features at work with networking REST APIs, so the next few paragraphs further explain those first three points.

### Client/Server Architecture

Like many applications, REST applications use a client/server architectural model. First, an application developer creates a REST API, and that application, when executing, acts as a REST server. Any other application can make a REST API call (the REST client) by executing some code that causes a request to flow from the client to the server. For instance, in Figure 23-1:

1. The REST client on the left executes a REST API call, which generates a message sent to the REST server.
2. The REST server on the right has API code that considers the request and decides how to reply.
3. The REST server sends back the response message with the appropriate data variables.

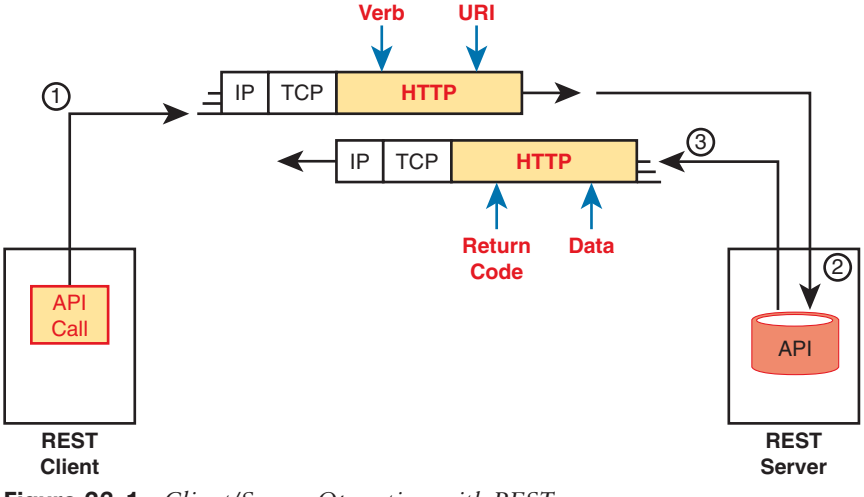



**Figure 23-1** *Client/Server Operation with REST*

**NOTE** Figure 23-1 shows the use of HTTP. While many REST APIs use HTTP, the use of HTTP is not a requirement for an API to be considered RESTful.

### Stateless Operation

The REST **stateless** operation feature dictates that the REST server does not keep data (state information) about any prior REST requests or the REST clients. With REST, the server does not keep information from one response (state information) for use in later requests and responses. Instead, the REST client maintains all state data from prior requests to choose what future REST requests it needs to make to accomplish the app's goals.

As an example, imagine a REST server that maintains a product database. If supported by the API, a REST client could use a single request for a list of products weighing less than five pounds. The REST server would not remember the first REST request, which client made the request, or the data returned in that earlier request. As a result, when the client makes subsequent API calls, it cannot ask the server to base those future responses on the data from prior requests. Instead, the client must supply any data in each subsequent request so that each REST request is self-contained and does not require state information at the server.

### Cacheable (or Not)

To appreciate what **cacheable** means, consider what happens when browsing a website. When your browser loads a new web page, the page itself contains a variety of objects (text, images, videos, audio). Some objects seldom change, so it would be better to download the object once and not download it again; in that case, the server marks that object as cacheable. For instance, a logo or other image shown on many pages of a website would almost never change and would likely be cacheable. However, the product list returned in your most recent search of the website would not be cacheable because the server would want to update and supply a new list each time you request the page.

REST APIs require that any resource requested via an API call have a clear method by which to mark the resource as cacheable or not. The goals remain the same: improve performance by retrieving resources less often (cacheable). Note that cacheable resources are marked with a timeframe so that the client knows when to ask for a new copy of the resource again.

## Background: Data and Variables

To appreciate a few of the upcoming topics, it helps to have a basic idea about how programming languages use variables. Anyone who has done even a small amount of programming should have enough background, but for those who have not written programs before, this next topic gives you enough background about data and variables inside programs to understand the next topic.

If you have some programming experience and already know about simple variables, **list variables**, and **dictionary variables**, then feel free to skip ahead to the section "REST APIs and HTTP."

### Simple Variables

Applications all process data with the same general actions, starting with some kind of input. The program needs data to process, so the input process reads files, sends database queries to a database server, or makes API calls to retrieve data from another application's API. The goal: gather the data that the program needs to process to do its work.

---

Answers to the "Do I Know This Already?" quiz:

**1** B, D **2** B, D **3** C **4** A, D **5** A, D **6** C, D

Programs then process data by making comparisons, making decisions, creating new variables, and performing mathematical formulas to analyze the data. All that logic uses variables. For instance, a program might process data with the following logic:

> If the router's G0/0 interface has a configuration setting of
>
> **switchport mode dynamic auto**
>
> Then gather more data to ensure that interface currently operates as a trunk rather than as an access port.

In programming, a variable is a name or label that has an assigned value. To get a general sense for programming variables, you can think of variables much like variables from algebra equations back in school. Example 23-1 shows some samples of variables of different types in a Python program (the Python language is the most popular language today for writing network automation applications). This program begins with a comment (the top three lines with triple single quotes) and then creates four variables, assigning them to different values, and prints a line of output: "The product is –12."

**Example 23-1** *Simple Python Program That Shows a Product*

```
'''
Sample program to multiply two numbers and display the result
'''
x = 3
y = -4
z = 1.247
heading = "The product is "
print(heading,x*y)
```

The variables in Example 23-1 can be called *simple variables* because each variable name has a single value associated with it. Simple variables have one variable name and one associated value, so they have a simple structure.

The values of simple variables can have a variety of formats, as shown in Example 23-1. The example includes variables that contain

- Unsigned integers (x)
- Signed integers (y)
- Floating-point numbers (z)
- Text (heading)

## List and Dictionary Variables

While simple variables have many great uses, programs need variables with more complex *data structures*. In programming, a data structure defines a related set of variables and values. For instance, Python uses list variables so that one variable name is assigned a value that is a list of values rather than a single value. You could imagine that a network automation program might want to have lists, such as a list of devices being managed, a list of interfaces on a device, or a list of configuration settings on an interface.

First, consider the variable named list1 in Example 23-2; note that the lines that begin with a # are comment lines.

**Example 23-2** *Sample List and Dictionary Variables in Python*

```
# Variable list1 is a list in Python (called an array in Java)
list1 = ["g0/0", "g0/1", "g0/2"]

# Variable dict1 is a dictionary (called an associative array in Java)
dict1 = {"config_speed":'auto', "config_duplex":"auto", "config_ip":"10.1.1.1"}
```

Even if you have never seen Python code before, you can guess at some of the meaning of the list1 variable. The code assigns variable list1 to a value that itself is a list of three text strings. Note that the list could include text, unsigned integers, signed integers, and so on.

Figure 23-2 shows the data structure behind variable list1 in Example 23-2. The variable is assigned to the list, with the list having three list elements.

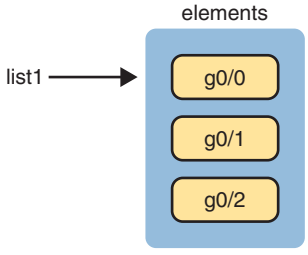



**Figure 23-2** *The List Data Structure in Python*

Python supports a similar data structure called a *dictionary*. If you think of the contents of a dictionary for the English language, that dictionary lists a series of paired items: a term and a matching definition. With programming languages like Python, the dictionary data structure lists paired items as well: *keys* (like terms) and *values* (like definitions). Figure 23-3 shows the structure of that dictionary value matching the dict1 variable at the bottom of Example 23-2. Note that each key and its value is called a **key:value pair**.

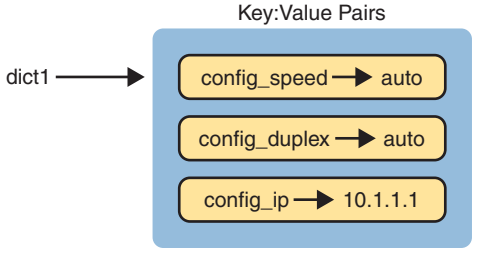



**Figure 23-3** *Dictionary Data Structures in Python*

Data structures can get more complex. Additionally, the data structures can be nested. For instance, a single variable's value could be a list, with each list element being a dictionary, with the values in some key:value pairs being other lists, and so on. For now, be aware of the fact that programs use simple variables but also use list and dictionary variables to make it easier to perform different kinds of logic.

## REST APIs and HTTP

APIs exist to allow two programs to exchange data. Some APIs may be designed as an interface between programs running on the same computer, so the communication between programs happens within a single operating system. Many APIs need to be available to programs that run on other computers, so an API must define the type of networking protocols supported by the API—and many REST-based APIs use the HTTP protocol.

The creators of REST-based APIs often choose HTTP because HTTP's logic matches some of the concepts defined more generally for REST APIs. HTTP uses the same principles as REST: it operates with a client/server model; it uses a stateless operational model; and it includes headers that clearly mark objects as cacheable or not cacheable. It also includes verbs—words that dictate the desired action for a pair of HTTP request and response messages—which matches how applications like to work.

This section breaks down the fundamentals of some programming terminology, how that matches HTTP verbs, and how REST APIs make use of Uniform Resource Identifiers (URIs) to specify the data desired from a RESTful API call.

### Software CRUD Actions and HTTP Verbs

The software industry uses a memorable acronym—**CRUD**—for the four primary actions performed by an application. Those actions are

Key Topic

**Create:** Allows the client to create some new instances of variables and data structures at the server and initialize their values as kept at the server

**Read:** Allows the client to retrieve (read) the current value of variables that exist at the server, storing a copy of the variables, structures, and values at the client

**Update:** Allows the client to change (update) the value of variables that exist at the server

**Delete:** Allows the client to delete from the server different instances of data variables

For instance, if using the northbound REST API of a Cisco DNA Center or Catalyst Center controller, as discussed in Chapter 22, "Cisco Software-Defined Access (Cisco SD-Access)," you might want to create something new, like a new security policy. From a programming perspective, the security policy exists as a related set of configuration settings on the controller, internally represented by variables. To accomplish this, a user would leverage a compatible REST API client application such as Postman. Once the user has the client application set up with the appropriate HTTP methods necessary to communicate with Catalyst Center controller, a RESTful API call is made to "create" a new variable on the controller that represents the security policy mentioned previously. The variable or security policy that was built on the controller was performed using the Create function of the CRUD acronym.

**NOTE** Cisco used the Cisco DNA Center controller product name for many years. In 2023, Cisco rebranded the product as the Cisco Catalyst Center controller. However, given the significant number of documents that refer to the old name and acronyms DNA and DNAC, this chapter continues to refer to the controller as the DNA Center controller.

Other examples of CRUD actions include a check of the status of that new configuration (a read action), an update to change some specific setting in the new configuration (an update action), or an action to remove the security policy definition completely (a delete action).

HTTP uses verbs that mirror CRUD actions. HTTP defines the concept of an HTTP request and response, with the client sending a request and with the server answering back with a response. Each request/response lists an action verb in the HTTP request header, which defines the HTTP action. The HTTP messages also include a URI, which identifies the resource being manipulated for this request. As always, the HTTP message is carried in IP and TCP, with headers and data, as represented in Figure 23-4.

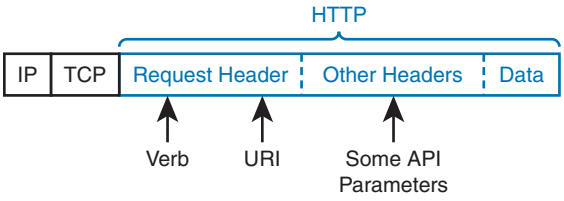



**Figure 23-4** *HTTP Verb and URI in an HTTP Request Header*

To get some perspective about HTTP, ignore REST for a moment. Whenever you open a web browser and click a link, your browser generates an HTTP GET request message similar to Figure 23-4 in structure. The message includes an HTTP header with the GET verb and the URI. The resources returned in the response are the components of a web page, like text files, image files, and video files.

HTTP works well with REST in part because HTTP has verbs that match the common program actions in the CRUD paradigm. Table 23-2 lists the HTTP verbs and CRUD terms for easy reference and study.

Key Topic

**Table 23-2** Comparing CRUD Actions to REST Verbs

| Action | CRUD Term | REST (HTTP) Verb |
|---|---|---|
| Create new data structures and variables | Create | POST |
| Read (retrieve) variable names, structures, and values | Read | GET |
| Update or replace values of some variable | Update | PATCH, PUT |
| Delete some variables and data structures | Delete | DELETE |

**NOTE** While Table 23-2 lists HTTP POST as a create action and HTTP PATCH and PUT as CRUD update actions, all three of these HTTP verbs might be used both for create and for update actions in some cases.

### Using URIs with HTTP to Specify the Resource

In addition to using HTTP verbs to perform the CRUD functions for an application, REST uses URIs to identify what resource the HTTP request acts on. For REST APIs, the resource can be any one of the many resources defined by the API. Each resource contains a set of related variables, defined by the API and identified by a URI.

For instance, imagine a user creates a REST-based API. When doing so, the user creates a set of resources to make available via the API, and also assigns a unique URI to each resource. In other words, the API creator creates a URI and a matching set of variables, and defines the actions that can be performed against those variables (read, update, and so on).

The API creator also creates API documentation that lists the resources and the URI that identifies each resource, among other details. The programmer for a REST client application can read the API documentation, build a REST API request, and ask for the specific resource, as shown in the example in Figure 23-5.

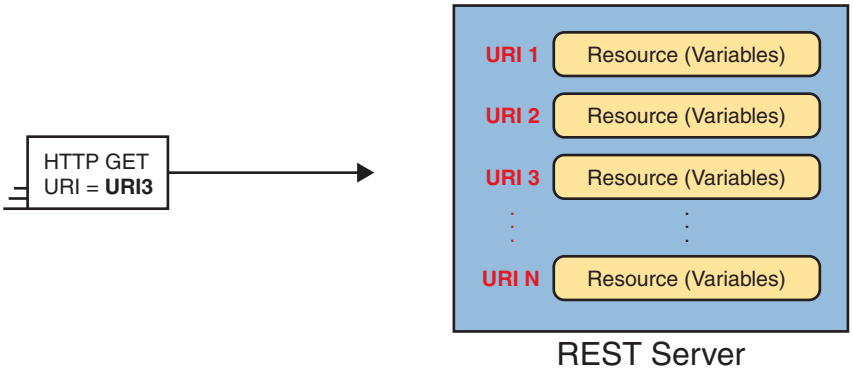



**Figure 23-5** *One URI for Each API Resource—Conceptual View*

Figure 23-5 shows generic URI values; however, today's network engineers need to be able to read API documentation, see URIs in that documentation, and understand the meaning of each part of the URI. Figure 23-6 shows a URI specific to the Cisco DNA Center northbound REST API as an example of some of the components of the URI.

Key Topic

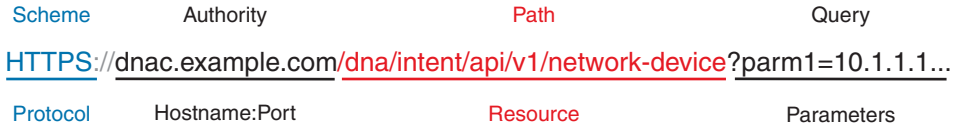



**Figure 23-6** *URI Structure for REST GET Request*

The URI has a defined format as detailed in RFC 3986. The top of Figure 23-6 lists the formal terms from the RFC. It also lists the more common terms used for those fields in some REST API documentation and tools.

- **Scheme (Protocol):** The HTTPS letters before the :// identify the scheme or protocol used—in this case, HTTP Secure (which uses HTTP with SSL encryption).
- **Authority (Host:Port):** This value sits between the // and first /, identifying the authority, which most people call the host. The authority field can list the hostname or IP address and, optionally, a transport port number. (If using a hostname, the REST client must perform name resolution to learn the IP address of the REST server.)
- **Path (Resource):** This value sits after the first / and finishes either at the end of the URI or before any additional fields (like a query field). The field uniquely identifies the

resource as defined by the API. For REST, you can think of this field as data, passed from the client to the server, to identify the data referenced in the request.

- **Query (Parameter):** This value sits after the first ? at the end of the path. Following the ?, the query field allows the assignment of values to the variable names so that the URI passes data.

A tour of the API documentation for any REST-based API reveals much about the purpose and meaning of the path and query fields. For instance, Figure 23-7 shows a copy of one doc page from the Cisco Catalyst Center API documentation. (To see for yourself, go to https://developer.cisco.com and search for "Cisco Catalyst Center API documentation.")

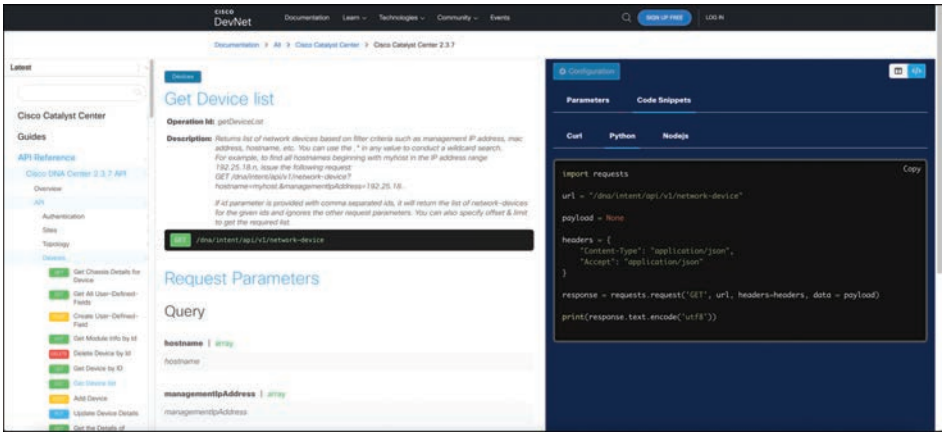



**Figure 23-7** *Cisco Catalyst Center API Doc Page for the Network Device (List) Resource*

The API documentation identifies the path (resource) to use in a URI to ask for specific information. For instance, an API GET request, using the path listed at the top of Figure 23-7, asks the Cisco Catalyst Center for a list of all known devices, with Cisco Catalyst Center returning a dictionary of values for each device.

That same API documentation also lists the query parameters to refine the REST request further. For instance, instead of a list of all network devices, you might want a dictionary of values for only one device. The lower left part of Figure 23-7 lists some of the available query parameters with this API call, which allows for just that by tacking on the following to the end of the URI:

```
?managementIPAddress=10.10.22.66&macAddress=f8:7b:20:67:62:80
```

## Example of REST API Call to Cisco Catalyst Center

To pull some of the REST API concepts together, the next few pages work through a few sample API calls using a software application called an API development environment tool.

For a bit of development perspective, when working to automate some part of your network operation tasks, you would eventually use a program that made API calls. However, early in the process of developing an application, you might first focus on the data available from the API and ignore all the programming details at first. API development environments let you

focus on the API calls. Later, that same tool can typically generate correct code that you can copy into your program to make the API calls.

The examples in this section use an app named Postman. Postman can be downloaded for free (www.postman.co) and used as shown in this section. Note that Cisco DevNet makes extensive use of Postman in its many labs and examples.

When leveraging APIs to communicate with any software platform, application, or controller, it is important to think through what you are trying to accomplish—for example, if you wanted to use an API call to Cisco Catalyst Center to pull down an inventory of devices within the controller.

Think of making any API call to the destination application or controller like connecting to any other device in your network environment. There is an important step that must be followed to gain access to the software application or controller, and that is to authenticate. Programmatically interacting with these platforms is no different than you logging into a router from the CLI and typing in your credentials. This means that you first must authenticate to the device you're trying to connect to via API. However, there are exceptions to this rule; if the software is not secured it might not require authentication before manipulation. This is very rare and typically not seen on anything that can make changes to your network environment.

There are many different authentication methods for APIs, such as API keys, JSON Web Token (JWT), TLS/SSL client certificates, basic authentication, Java session IDs or cookies, and so on. Using Cisco Catalyst Center as an example, let's take a look at the documentation for the authentication or token API and what method is used to authenticate.

**NOTE** This section focuses only on basic authentication because it is the current method of authentication used on Cisco Catalyst Center.

Figure 23-8 showcases the Cisco Catalyst Center API reference on developer.cisco.com. Notice that the first API call available in the list on the left side of the screen is the Authentication API. The way this works is that once you successfully authenticate to Cisco Catalyst Center using an HTTP POST request method to send the login credentials, you then will receive a token, which is a Base64-encoded string of the username and password. Notice the text that says

> "Description: API to obtain an access token, which remains valid for 1 hour. The token obtained using this API is required to be set as value to the X-Auth-Token HTTP Header for all API calls to Cisco DNA Center."

This is stating that this API is used to authenticate to the controller, and something called an X-Auth-Token needs to be passed to the controller for all subsequent API calls after you have authenticated and received the token.

This means that once you authenticate and receive the token from Cisco Catalyst Center, you will then have to send that token back to Cisco Catalyst Center as an X-Auth-Token for all subsequent API calls you do after you initially authenticate. An example of this is done using an HTTP GET request to pull down the data or inventory from Cisco Catalyst Center.

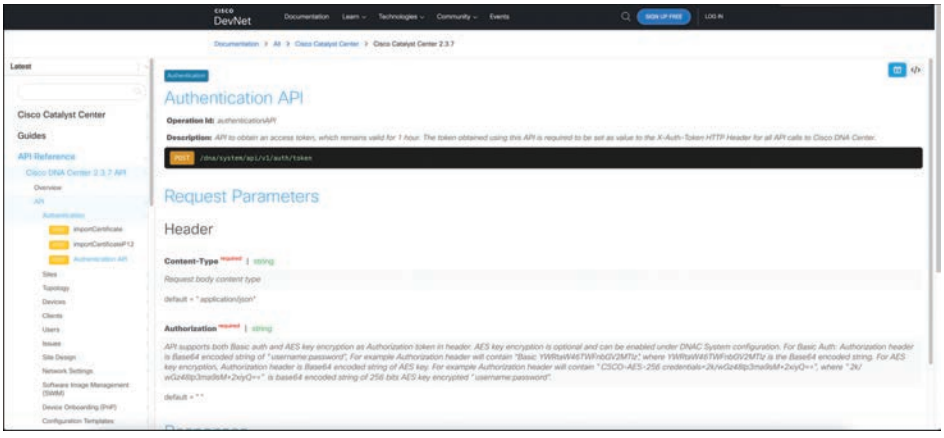



**Figure 23-8** *Cisco Catalyst Center API Reference on developer.cisco.com*

Now that you understand where to find the API documentation, let's review the steps that you must authenticate to Cisco Catalyst Center and retrieve the token.

First, you must use the Authentication API to log in to the Cisco Catalyst Center and receive a token before you can make any subsequent API calls, such as pulling a device inventory from the controller. Let's look at how to authenticate using Postman. Figure 23-9 illustrates the proper setup for Postman to authenticate to Cisco Catalyst Center. The following information is used to set up Postman:

- URI for Authentication API: https://sandboxdnac2.cisco.com/dna/system/api/v1/auth/token
- HTTP POST request method
- Basic authorization
- Username and password: devnetuser / Cisco123!




**Figure 23-9** *Postman Setup for Cisco Catalyst Center Authentication API*

Now Postman is set up to authenticate, click the Send button to issue the API call to Cisco Catalyst Center. Figure 23-10 displays the response from Cisco Catalyst Center, along with the token and an HTTP status code of 200 OK, indicating the authentication was successful.

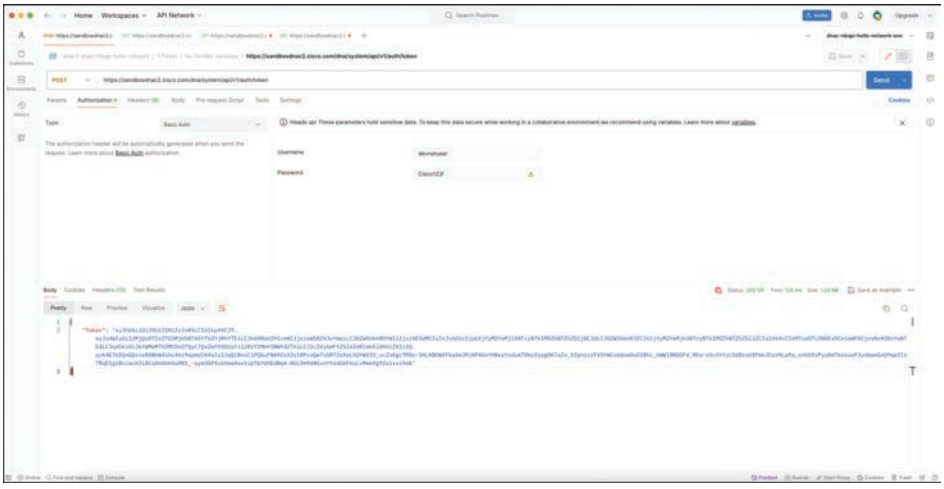


**Figure 23-10** *Execution of Cisco Catalyst Center Authentication API in Postman*

You use the token that was received to pass on to Cisco Catalyst Center to pull down the device inventory via an API. Looking back to the API documentation, you can see an API called Network-Device. This is the API to pull a device inventory from the Cisco Catalyst Center controller. Figure 23-11 highlights the specifics of how to use the Network-Device API from the documentation on the DevNet website. Notice, however, that the URI is different for subsequent API calls versus what was used for the Authentication API. Compare the two following URIs:

- **Authentication URI:** https://sandboxdnac2.cisco.com/dna/system/api/v1/auth/token
- **Network-Device URI:** https://sandboxdnac2.cisco.com/dna/intent/api/v1/network-device

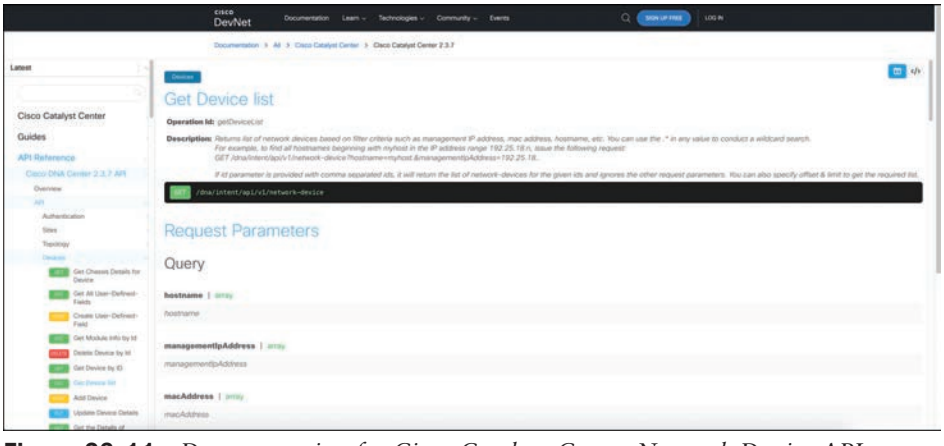



**Figure 23-11** *Documentation for Cisco Catalyst Center Network-Device API*

Now that you know how to use the API by reviewing the documentation, let's demonstrate how this is done in Postman. Figure 23-12 displays the appropriate steps for using the existing token that was received via the authentication API completed earlier. In addition, the figure shows how to simply type the word X-Auth-Token into the key field under the Headers tab, then paste the token information into the Value field and ensure the appropriate URI for the Network-Device API call is being used. Once this is all confirmed, you can then issue the GET API call to Cisco Catalyst Center and pull down the device inventory.

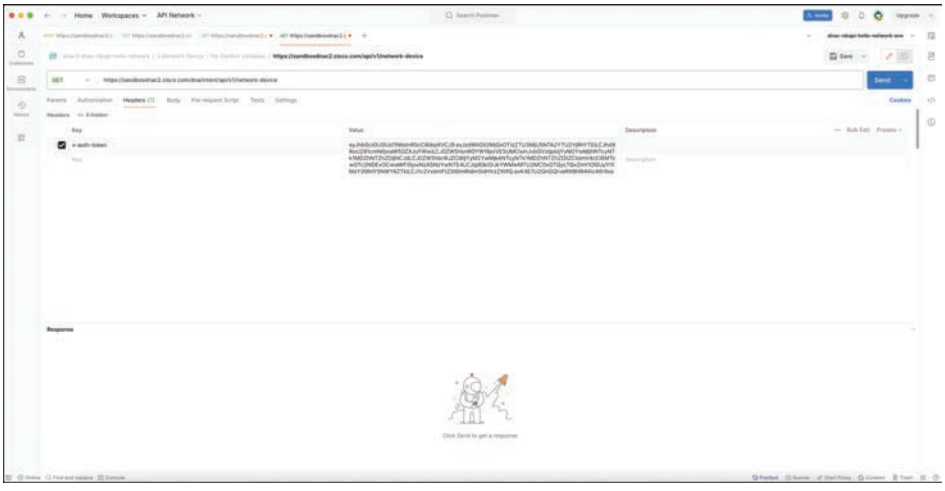


**Figure 23-12** *Postman Setup for Cisco Catalyst Center Network-Device API*

Following the same procedure done with the authentication API, click the Send button and issue the API call to Cisco Catalyst Center. In Figure 23-13, you can see the inventory was successfully pulled down via API. You can also see the list of network devices in the response section of Postman. Also note that the HTTP status code of 200 OK is present again. The resource part of the URI shows the same resource listed earlier in Figure 23-6, asking for a list of devices.

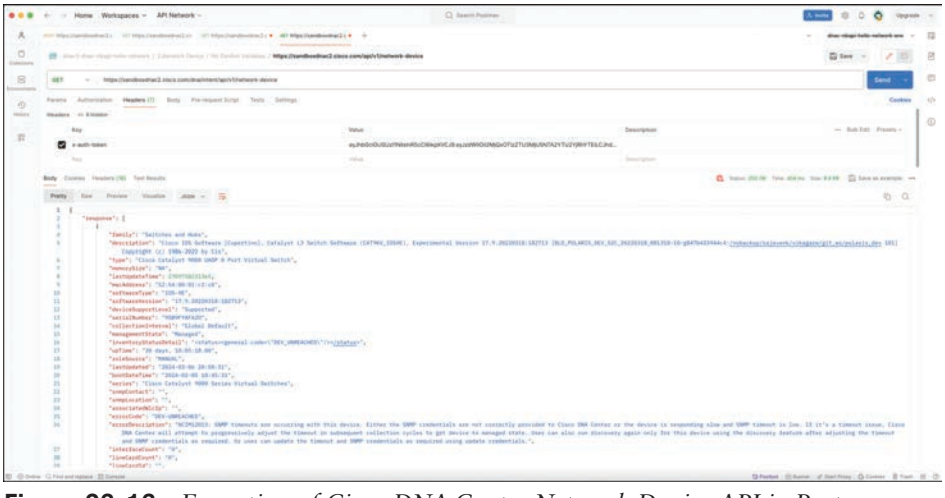


**Figure 23-13** *Execution of Cisco DNA Center Network-Device API in Postman*

**NOTE** sandboxdnac2.cisco.com is an always-on Cisco DNA Center instance available on the Cisco DevNet website. There is no cost to use this resource.

Take a moment to look through the data at the bottom of the Postman window in Figure 23-13. The text follows a data modeling format called JavaScript Object Notation (JSON), which is one of the main topics for the remainder of the chapter. However, armed with just a knowledge of routers, you can find a few facts that look familiar. To help you see the text, Example 23-3 shows an edited (shortened to reduce the length) view of some of the JSON output in that window, just so you can see the format and some of the data returned in this single API call.

**Example 23-3** *JSON Output from a REST API Call*

```
{
    "response": {
        "family": "Switches and Hubs",
        "type": "Cisco Catalyst 9000 UADP 8 Port Virtual Switch",
        "macAddress": "52:54:00:01:c2:c0",
        "softwareType": "IOS-XE",
        "softwareVersion": "17.9.20220318:182713",
        "serialNumber": "9SB9FYAFA2O",
        "upTime": "30 days, 10:05:18.00",
        "series": "Cisco Catalyst 9000 Series Virtual Switches",
        "hostname": "sw1.ciscotest.com",
        "managementIpAddress": "10.10.20.175",
        "platformId": "C9KV-UADP-8P",
        "role": "CORE"
    }
}
```

API development tools like Postman help you work out the particulars of each API call, save the details, and share with other engineers and developers. Eventually, you will be ready to make the API call from a program. With a simple click from the Postman UI, Postman supplies the code to copy/paste into your program so that it returns all the output shown in the center/bottom of the window back as a variable to your program.

By now, you have a good foundational knowledge of the mechanics of REST APIs. By learning some skills, and using the API documentation for any REST API, you could now experiment with and try to make REST API calls. For many of those, the data will return to you as text, often in JSON format, so the second half of the chapter examines the meaning of that text.

## Data Serialization and JSON

In your journey to become a modern network engineer with network automation skills, you will learn to understand several **data serialization languages**. Each data serialization language provides methods of using text to describe variables, with a goal of being able to send

that text over a network or to store that text in a file. Data serialization languages give us a way to represent variables with text rather than in the internal representation used by any particular programming language.

Each data serialization language enables API servers to return data so that the API client can replicate the same variable names as well as data structures as found on the API server. To describe the data structures, the data serialization languages include special characters and conventions that communicate ideas about list variables, dictionary variables, and other more complex data structures.

This second major section of the chapter examines the concept of a data serialization language, with a focus on the one data modeling language as mentioned in the current CCNA blueprint: **JavaScript Object Notation (JSON)**.

## The Need for a Data Model with APIs

This section shows some ideas of how to move variables in a program on a server to a client program. First, Figure 23-14 and surrounding text show a nonworking example as a way to identify some of the challenges with copying variable values from one device to another. Later, Figure 23-15 and its related text show how to use a data serialization language to solve the problems shown around Figure 23-14.

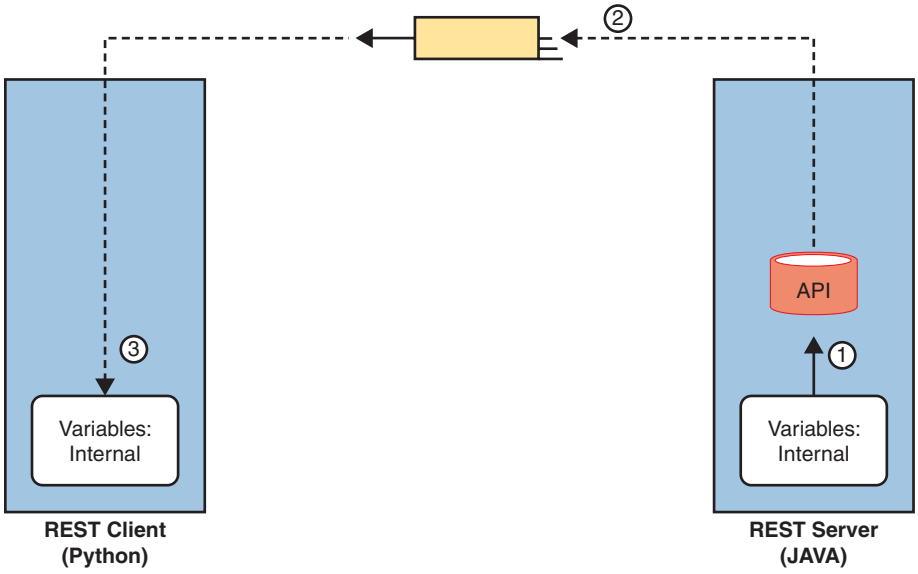



**Figure 23-14** *Broken Concept: Exchanging Internal Representations of Variables*

First, for the nonworking example, consider the flow and numbered steps in Figure 23-14. A REST client sits on the left. The REST client asks for a resource, and the server needs to reply. In REST, a resource is a set of variables as defined by the API, so the REST server needs to return a set of variables to the REST client on the left. The steps in the figure run as follows:

1. The REST server (a JAVA application) takes a copy of the stored variables in RAM (step 1) in response to the REST request.

2. The REST API code creates the REST response and sends it over the network, placing an exact replica of what the REST server had in RAM to represent the variables in that resource.
3. The REST client (a Python application) receives the REST response message, storing the exact same bits and bytes into its RAM, in an attempt to have a copy of the variables, data, and data structures on the server.

The process shown in Figure 23-14 does not work (and is not attempted) because the REST client programs may not store variables in the same ways. First, programs written in different languages use different conventions to store their variables internally because there is no standard for internal variable storage across languages. In fact, programs written in the same language but with different versions of that language may not store all their variables with the same internal conventions.

To overcome these issues, applications need a standard method to represent variables for transmission and storage of those variables outside the program. *Data serialization languages* provide that function.

Figure 23-15 shows the correct process flow in comparison to Figure 23-14 with the data serialization process included:

1. The server collects the internally represented data and gives it to the API code.
2. The API converts the internal representation to a data model representing those variables (with JSON shown in the figure).
3. The server sends the data model in JSON format via messages across the network.
4. The REST client takes the received data and converts the JSON-formatted data into variables in the native format of the client application.

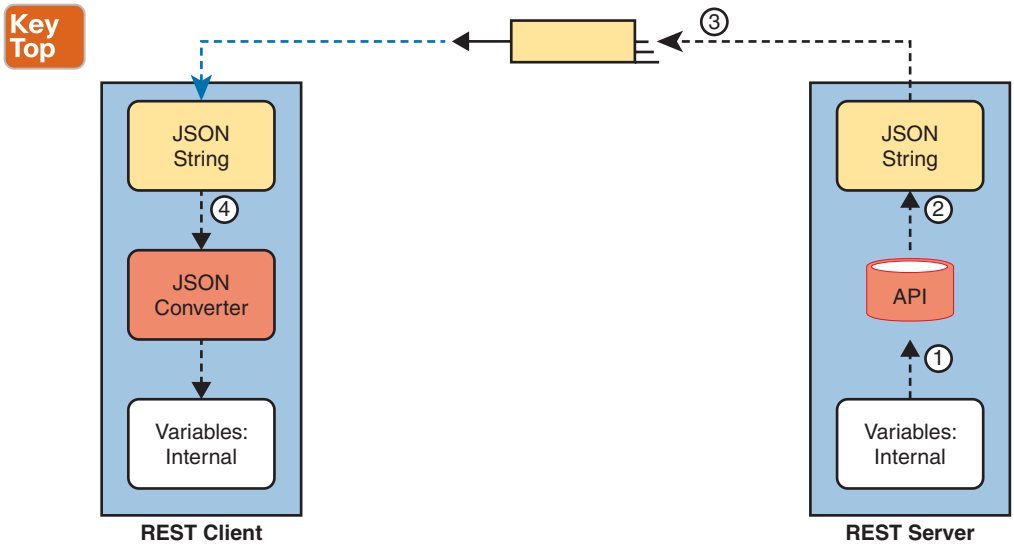



**Figure 23-15** *Correct Concept: Exchanging Internal Representations of Variables*

At the end of the process, the REST client application now has equivalent variables to the ones it requested from the server in the API call. Note that the final step—to convert from the data serialization language to the native format—can be as little as a single line of code!

Finally, note that while data serialization languages like JSON enable applications to exchange variables over a network, applications can also store data in JSON format.

## Data Serialization Languages

You will hear about and eventually use several data serialization and data modeling languages the more you learn about network automation. While the current CCNA blueprint mentions only JSON, learning a few facts about some of the alternatives can be helpful to add a little context to your new knowledge of JSON. These different data serialization languages exist to meet different needs that have arisen over the years. This next short section highlights four such languages.

**NOTE** The terms *data serialization language* and *data modeling language* should be considered equivalent for the purposes of this section.

### JSON

JavaScript Object Notation attempts to strike a balance between human and machine readability. Armed with a few JSON rules, most humans can read JSON data, move past simply guessing at what it means, and confidently interpret the data structures defined by the JSON data. At the same time, JSON data makes it easy for programs to convert JSON text into variables, making it very useful for data exchange between applications using APIs.

You can find the details of JSON in IETF RFC 8259 and in a number of sites found with Internet searches, including www.json.org.

### XML

Back in the 1990s, when web browsers and the World Wide Web (WWW) were first created, web pages primarily used Hypertext Markup Language (HTML) to define web pages. As a markup language, HTML defined how to add the text or a web page to a file and then add "markup"—additional text to denote formatting details for the text that should be displayed. For instance, the markup included codes for headings, font types, sizes, colors, hyperlinks, and so on.

The **eXtensible Markup Language (XML)** came later to make some improvements for earlier markup languages. In particular, over time web pages became more and more dynamic, and to make the pages dynamic, the files needed to store variables whose values could be changed and replaced over time by the web server. To define variables to be substituted into a web page, the world needed a markup language that could define data variables. XML defines a markup language that has many features to define variables, values, and data structures.

Over time, XML has grown beyond its original use as a markup language. XML's features also make it a useful general data serialization language, and it is used as such today.

While keeping human readability in mind, the creators of XML opted for less emphasis on human readability (as compared to JSON) to gain better features for computer processing. For instance, like HTML, XML uses beginning and ending tags for each variable, as seen in Example 23-4. In the highlighted line in the example, the <macAddress> and </macAddress> tags denote a variable name, with the value sitting between the tags.

**Example 23-4** *XML Output from a REST API Call*

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
   <response>
      <family>Switches and Hubs</family>
      <hostname>cat_9k_1</hostname>
      <interfaceCount>41</interfaceCount>
      <lineCardCount>2</lineCardCount>
      <macAddress>f8:7b:20:67:62:80</macAddress>
      <managementIpAddress>10.10.22.66</managementIpAddress>
      <role>ACCESS</role>
      <serialNumber>FCW2136L0AK</serialNumber>
      <series>Cisco Catalyst 9300 Series Switches</series>
      <softwareType>IOS-XE</softwareType>
      <softwareVersion>16.6.1</softwareVersion>
      <type>Cisco Catalyst 9300 Switch</type>
      <upTime>17 days, 22:51:04.26</upTime>
   </response>
</root>
```

## YAML

**YAML Ain't Markup Language (YAML)** has a clever recursive name, but the name does tell us something. YAML does not attempt to define markup details (while XML does). Instead, YAML focuses on the data model (structure) details. YAML also strives to be clean and simple: of the data serialization/modeling languages listed here, YAML is easily the easiest to read for anyone new to data models.

Ansible, one of the topics in Chapter 24, "Understanding Ansible and Terraform," makes extensive use of YAML files. Example 23-5 shows a brief sample. And to make the point about readability, even if you have no idea what Ansible does, you can guess at some of the functions just reading the file. (Note that YAML denotes variables in double curly brackets: {{ }}.)

**Example 23-5** *YAML File Used by Ansible*

```
---
# This comment line is a place to document this Playbook
- name: Get IOS Facts
  hosts: mylab
  vars:
```

```
    cli:
      host: "{{ ansible_host }}"
      username: "{{ username }}"
      password: "{{ password }}"

  tasks:
    - ios_facts:
        gather_subset: all
        provider: "{{ cli }}"
```

### Summary of Data Serialization

As an easy reference, Table 23-3 summarizes the data serialization languages mentioned in this section, along with some key facts.

Key Topic

**Table 23-3** Comparing Data Modeling Languages

| Acronym | Name | Origin/Definition | Central Purpose | Common Use |
|---|---|---|---|---|
| JSON | JavaScript Object Notation | JavaScript (JS) language; RFC 8259 | General data modeling and serialization | REST APIs |
| XML | eXtensible Markup Language | World Wide Web Consortium (W3C.org) | Data-focused text markup that allows data modeling | REST APIs, Web pages |
| YAML | YAML Ain't Markup Language | YAML.org | General data modeling | Ansible |

## Recognizing the Components of JSON

Cisco includes one exam topic in the CCNA 200-301 Version 1.1 blueprint that mentions JSON:

6.7 Recognize the components of JSON encoded data

You can think of that skill and task with two major branches. First, even ignoring the syntax and special characters, anyone who knows the topic can probably make intelligent guesses about the meaning of many of the key:value pairs. For example, without knowing anything about JSON syntax, you could probably determine from your prior knowledge of Cisco routers and switches that the JSON in Example 23-6 lists two devices (maybe their host-names) and a list of interfaces on each device.

**Example 23-6** *Simple JSON That Lists a Router's Interfaces*

```
{
    "R1": ["GigabitEthernet0/0", "GigabitEthernet0/1", "GigabitEthernet0/2/0"],
    "R2": ["GigabitEthernet1/0", "GigabitEthernet1/1", "GigabitEthernet0/3/0"]
}
```

Honestly, you probably already know everything needed to do this kind of intelligent guessing. However, to perform the second type of task, where you analyze the JSON data to find the data structures, including objects, lists, and key:value pairs, you need to know a bit

more about JSON syntax. This final topic in the chapter gives you the basic rules, with some advice on how to break down JSON data.

## Interpreting JSON Key:Value Pairs

First, consider these rules about key:value pairs in JSON, which you can think of as individual variable names and their values:

Key Topic

- **Key:Value Pair:** Each and every colon identifies one key:value pair, with the key before the colon and the value after the colon.
- **Key:** Text, inside double quotes, before the colon, used as the name that references a value.
- **Value:** The item after the colon that represents the value of the key, which can be
  - **Text:** Listed in double quotes.
  - **Numeric:** Listed without quotes.
  - **Array:** A special value (more details later).
  - **Object:** A special value (more details later).
- **Multiple Pairs:** When listing multiple key:value pairs, separate the pairs with a comma at the end of each pair (except the last pair).

To work through some of these rules, consider Example 23-7's JSON data, focusing on the three key:value pairs. The text after the example provides analysis.

**Example 23-7** *One JSON Object (Dictionary) with Three Key:Value Pairs*

```
{
    "1stbest": "Messi",
    "2ndbest": "Ronaldo",
    "3rdbest": "Maradona"
}
```

As an approach, just find each colon, and look for the quoted string just before each colon. Those are the keys ("1stbest", "2ndbest", and "3rdbest".) Then look to the right of each colon to find their matching values. You can know all three values are text values because JSON lists the values within double quotes.

As for other special characters, note the commas and the curly brackets. The first two key:value pairs end with a comma, meaning that another key:value pair should follow. The curly brackets that begin and end the JSON data denote a single JSON object (one pair of curly brackets, so one object). JSON files and JSON data exchanged over an API exist first as a JSON object, with an opening (left) and closing (right) curly bracket as shown.

## Interpreting JSON Objects and Arrays

To communicate data structures beyond a key:value pair with a simple value, JSON uses **JSON objects** and **JSON arrays**. Objects can be somewhat flexible, but in most uses, they act like a dictionary. Arrays list a series of values.

**NOTE** Python, the most common language to use for network automation, converts JSON objects to Python dictionaries, and JSON arrays to Python lists. For general conversation, many people refer to the JSON structures as dictionaries and lists rather than as objects and arrays.

To begin, consider this set of rules about how to interpret the syntax for JSON objects and arrays:

Key Topic

- **{ }—Object:** A series of key:value pairs enclosed in a matched pair of curly brackets, with an opening left curly bracket and its matching right curly bracket.
- **[ ]—Array:** A series of values (not key:value pairs) enclosed in a matched pair of square brackets, with an opening left square bracket and its matching right square bracket.
- **Key:value pairs inside objects:** All key:value pairs inside an object conform to the earlier rules for key:value pairs.
- **Values inside arrays:** All values conform to the earlier rules for formatting values (for example, double quotes around text, no quotes around numbers).

Example 23-8 shows a single array in JSON format. Notice the JSON data begins with a [ and then lists three text values (the values could have been a mix of values). It then ends with a ].

**Example 23-8** *A JSON Snippet Showing a Single JSON Array (List)*

```
[
     "Messi",
     "De Bruyne",
     "Pedri"
]
```

While Example 23-8 shows only the array itself, JSON arrays can be used as a value in any key:value pair. Figure 23-16 does just that, shown in a graphic to allow easier highlighting of the arrays and object. The JSON text in the figure includes two arrays (lists) as values (each found just after a colon, indicating they are values).

Now think about the entire structure of the JSON data in Figure 23-16. It has a matched pair of curly brackets to begin and end the text, encapsulating one object. That object contains two colons, so there are two key:value pairs inside the object. When you think about the broader structure, as depicted in Figure 23-17, this JSON file has one JSON object, itself with two key:value pairs. (Note that Figure 23-17 does NOT show correct JSON syntax for the lists; it instead is intended to make sure you see the structure of the one object and its two key:value pairs.)

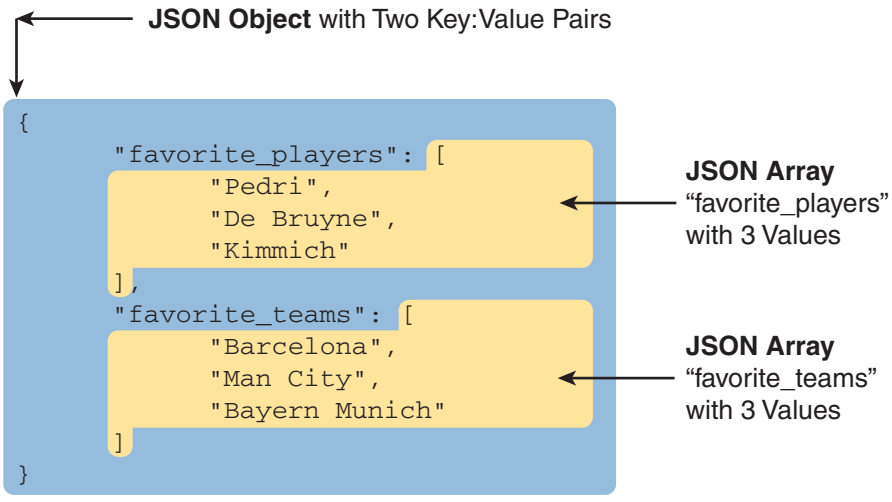



**Figure 23-16** *Accurate/Complete JSON Data with One Object, Two Keys, Two JSON List Values*

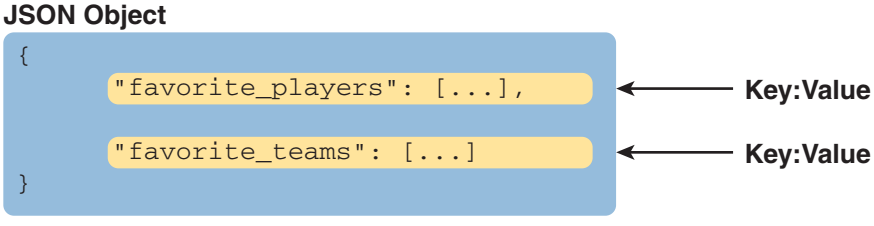



**Figure 23-17** *Structural Representation of Figure 23-16's Primary Object and Two Key:Value Pairs*

To drive home the idea of how to find JSON objects, consider the example shown in Figure 23-18. This figure shows correct JSON syntax. It has the following:

- There is one object for the entire set because it begins and ends with curly brackets.
- The outer object has two keys (Wendells_favorites and interface_config).
- The value of each key:value pair is another object (each with curly brackets and three key:value pairs).

The JSON example in Figure 23-18 shows how JSON can nest objects and arrays; that is, JSON puts one object or array inside another. Much of the JSON output you will see as you learn more and more about network automation will include JSON data with nested arrays and objects.

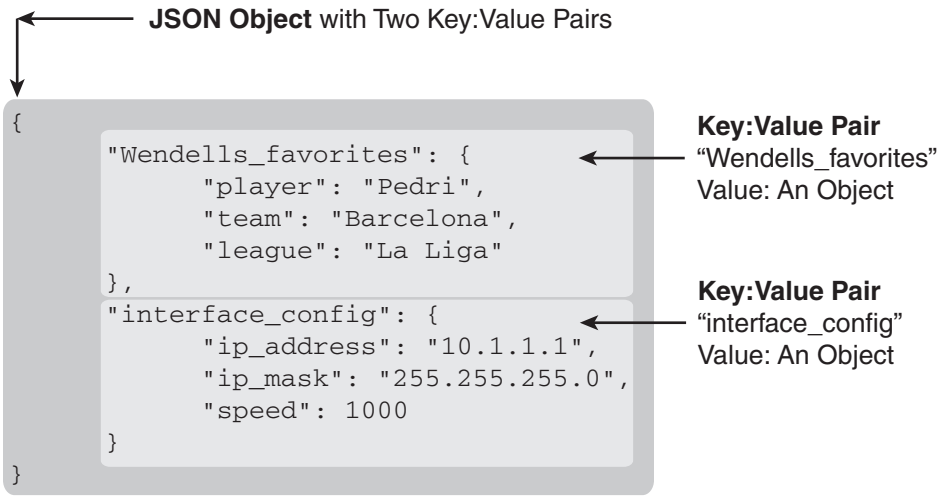



**Figure 23-18** *A JSON Object, with Two Key:Value Pairs, Each Value Another Object*

### Minified and Beautified JSON

So far, all the JSON examples show lots of empty space. JSON allows for whitespace, or not, depending on your needs. For humans, reading JSON can be a lot easier with the text organized with space and aligned. For instance, having the matched opening and closing brackets sit at the same left-offset makes it much easier to find which brackets go with which.

When stored in a file or sent in a network, JSON does not use whitespace. For instance, earlier in this section, Example 23-7 showed one JSON object with three key:value pairs, with whitespace, taking five lines. However, stored in a file, or sent over a network, the JSON would look like the following:

```
{"1stbest": "Messi", "2ndbest": "Ronaldo", "3rdbest": "Maradona"}
```

Most of the tools you might use when working with JSON will let you toggle from a pretty format (good for humans) to a raw format (good for computers). You might see the pretty version literally called *pretty* or *beautified* or *spaced*, while the version with no extra whitespace might be called *minified* or *raw*.

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 23-4 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 23-4** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory tables | | Website |
| Watch videos | | Website |

## Review All the Key Topics

Key Topic

**Table 23-5** Key Topics for Chapter 23

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Attributes of REST APIs | 529 |
| List | The meaning of the CRUD acronym | 533 |
| Table 23-2 | A comparison of CRUD actions and HTTP verbs | 534 |
| Figure 23-6 | Components of a URI | 535 |
| Figure 23-15 | The process of sending JSON data over a REST API | 543 |
| Table 23-3 | A comparison of JSON, XML, and YAML | 546 |
| List | JSON rules related to key:value pairs | 547 |
| List | JSON rules for arrays and objects | 548 |

## Key Terms You Should Know

cacheable, CRUD, data serialization language, dictionary variable, JSON (JavaScript Object Notation), JSON array, JSON object, key:value pair, list variable, REpresentational State Transfer (REST), REST API, stateless, URI path (resource), URI query (parameters), XML (eXtensible Markup Language), YAML (YAML Ain't Markup Language)



CHAPTER 24

# Understanding Ansible and Terraform

**This chapter covers the following exam topics:**

**6.0 Automation and Programmability**

**6.6 Recognize the capabilities of configuration mechanisms Ansible and Terraform**

By now, you have seen how to use the IOS CLI to configure routers and switches. To configure using the CLI, you get into configuration mode, issue configuration commands (which change the running-config file), and eventually leave configuration mode. If you decide to keep those changes, you save the configuration to the startup-config file using the **copy running-config startup-config** command. Next time the router or switch boots, the device loads the startup-config file into RAM as the running-config. Simple enough.

This chapter discusses tools for configuration management that replace that per-device configuration process. To even imagine what these tools do first requires you to make a leap of imagination to the everyday world of a network engineer at a medium to large enterprise. In a real working network, managing the configuration of the many networking devices creates challenges. Those challenges can be addressed using that same old "use configuration mode on each device" process, plus with hard work, attention to detail, and good operational practices. However, that manual per-device process becomes more and more difficult for a variety of reasons, so at some point, enterprises turn to automated configuration management tools to provide better results.

The first section of this chapter takes a generalized look at the issues of configuration management at scale along with some of the solutions to those problems. The second major section then details two configuration management tools—Ansible and Terraform—to define some of the features and terms used with each. By the end of the chapter, you should be able to see some of the reasons why these automated configuration management tools have a role in modern networks and enough context to understand as you pick one to investigate for further reading.

## "Do I Know This Already?" Quiz

Take the quiz (either here or use the PTP software) if you want to use the score to help you decide how much time to spend on this chapter. The letter answers are listed at the bottom of the page following the quiz. Appendix C, found both at the end of the book as well as on the companion website, includes both the answers and explanations. You can also find both answers and explanations in the PTP testing software.

**Table 24-1** "Do I Know This Already?" Foundation Topics Section-to-Question Mapping

| Foundation Topics Section | Questions |
|---|---|
| Device Configuration Challenges and Solutions | 1–3 |
| Ansible and Terraform Basics | 4, 5 |

1. Which answer best describes the meaning of the term *configuration drift*?
   a. Changes to a single device's configuration over time versus that single device's original configuration
   b. Larger and larger sections of unnecessary configuration in a device
   c. Changes to a single device's configuration over time versus other devices that have the same role
   d. Differences in device configuration versus a centralized backup copy
2. An enterprise moves away from manual configuration methods, making changes by editing centralized configuration files. Which answers list an issue solved by using a version control system with those centralized files? (Choose two answers.)
   a. The ability to find which engineer changed the central configuration file on a date/time
   b. The ability to find the details of what changed in the configuration file over time
   c. The ability to use a template with per-device variables to create configurations
   d. The ability to recognize configuration drift in a device and notify the staff
3. Configuration monitoring (also called configuration enforcement) by a configuration management tool generally solves which problem?
   a. Tracking the identity of individuals who changed files, along with which files they changed
   b. Listing differences between a former and current configuration
   c. Testing a configuration change to determine whether it will be rejected or not when implemented
   d. Finding instances of configuration drift
4. Which of the following configuration management tools by default use a push model to configure network devices? (Choose two answers.)
   a. Ansible
   b. Both use a pull model
   c. Terraform
   d. Neither uses a push model
5. Which of the following answers list a correct combination of configuration management tool and the term used for one of its primary configuration files? (Choose two answers.)
   a. Ansible Configuration
   b. Terraform Configuration
   c. Ansible Playbook
   d. Terraform Playbook

# Foundation Topics

## Device Configuration Challenges and Solutions

Think about any production network. What defines the exact intended configuration of each device in a production network? Is it the running-config as it exists right now or the startup-config before any recent changes were made or the startup-config from last month? Could one engineer change the device configuration so that it drifts away from that ideal, with the rest of the staff not knowing? What process, if any, might discover the configuration drift? And even with changes agreed upon by all, how do you know who changed the configuration, when, and specifically what changed?

Traditionally, CCNA teaches us how to configure one device using the **configure** terminal command to reach configuration mode, which changes the running-config file, and how to save that running-config file to the startup-config file. That manual process provides no means to answer any of the legitimate questions posed in the first paragraph; however, for many enterprises, those questions (and others) need answers, both consistent and accurate.

Not every company reaches the size to want to do something more with configuration management. Companies with one network engineer might do well enough managing device configurations, especially if the network device configurations do not change often. However, as a company moves to multiple network engineers and grows the numbers of devices and types of devices, with higher rates of configuration change, manual configuration management has problems.

This section begins by discussing a few of these kinds of configuration management issues so that you begin to understand why enterprises need more than good people and good practices to deal with device configuration. The rest of the section then details some of the features you can find in automated **configuration management tools**.

### Configuration Drift

Consider the story of an enterprise of a size to need two network engineers, Alice and Bob. They both have experience and work well together. But the network configurations have grown beyond what any one person can know from memory, and with two network engineers, they may remember different details or even disagree on occasion.

One night at 1 a.m., Bob gets a call about an issue. He gets into the network from his laptop and resolves the problem with a small configuration change to branch office router BR22. Alice, the senior engineer, gets a different 4 a.m. call about another issue and makes a change to branch office router BR33.

The next day gets busy, and neither Alice nor Bob mentions the changes they made. They both follow procedures and document the changes in their change management system, which lists the details of every change. Because they both get busy, the topic never comes up, and neither mentions the changes to each for months.

The story shows how **configuration drift** can occur—an effect in which the configuration drifts away from the intended configuration over time. Alice and Bob probably agree to what a standard branch office router configuration ought to look like, but they both made an exception to that configuration to fix a problem, causing configuration drift.

Figure 24-1 shows the basic idea, with those two branch routers now with slightly different configurations than the other branch routers.

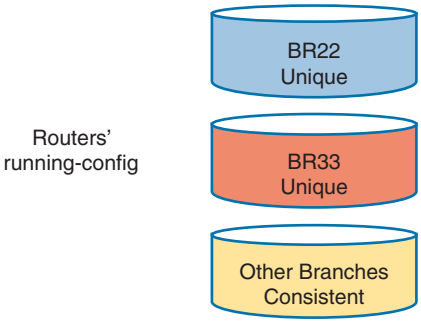



**Figure 24-1** *Configuration Drift in Branch Routers BR22 and BR33*

Configuration drift becomes a much bigger problem if using only traditional manual configuration tools. For instance:

Key Topic

- The on-device manual configuration process does not track change history: which lines changed, what changed on each line, what old configuration was removed, who changed the configuration, when each change was made.
- External systems used by good systems management processes, like trouble ticketing and change management software, may record details. However, those sit outside the configuration and require analysis to figure out what changed. They also rely on humans to follow the operational processes consistently and correctly; otherwise, an engineer cannot find the entire history of changes to a configuration.
- Referring to historical data in change management systems works poorly if a device has gone through multiple configuration changes over a period of time.

## Centralized Configuration Files and Version Control

The manual per-device configuration model makes great sense for one person managing one device. With that model, the one network engineer can use the on-device startup-config as the intended ideal configuration. If a change is needed, the engineer gets into configuration mode and updates the running-config until happy with the change. Then the engineer saves a copy to the startup-config as the now-current ideal config for the device.

The per-device manual configuration model does not work as well for larger networks, with hundreds or even thousands of network devices, with multiple network engineers. For instance, if the team thinks of the startup-config of each device as the ideal configuration, if one team member changes the configuration (like Alice and Bob each did in the earlier story), no records exist about the change. The config file does not show what changed, when it changed, or who changed it, and the process does not notify the entire team about the change.

As a first step toward better configuration management, many medium to large enterprises store configurations in a central location. At first, storing files centrally may be a simple effort to keep backup copies of each device's configuration. They would, of course, place the files in a shared folder accessible to the entire network team, as shown in Figure 24-2.

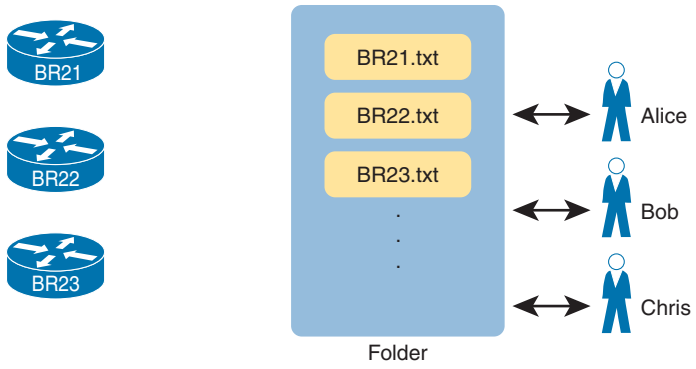



**Figure 24-2** *Copying Device Configurations to a Central Location*

Which configuration file is the single source of truth in this model? The configuration files still exist on each device, but now they also exist on a centralized server, and engineers could change the on-device configuration as well as the text files on the server. For instance, if the copy of BR21's configuration on the device differs from the file on the centralized server, which should be considered as correct, ideal, the truth about what the team intends for this device?

In practice, companies take both approaches. In some cases, companies continue to use the on-device configuration files as the source of truth, with the centralized configuration files treated as backup copies in case the device fails and must be replaced. However, other enterprises make the transition to treat the files on the server as the single source of truth about each device's configuration. When using the centralized file as the source of truth, the engineers can take advantage of many configuration management tools and actually manage the configurations more easily and with more accuracy.

For example, configuration management tools use version control software to track the changes to centralized configuration files, noting who changes a file, what lines and specific characters changed, when the change occurred, and so on. The tools also allow you to compare the differences between versions of the files over time, as shown in Figure 24-3.

The figure shows a sample of a comparison between two versions of a configuration file. The upper two highlighted lines, with the minus sign, show the lines that were changed, while the two lower highlighted lines, with the plus signs, show the new versions of each line.

Version control software solves many of the problems with the lack of change tracking within the devices themselves. Figure 24-3 shows output from a popular Software-as-a-Service site called GitHub (www.github.com). GitHub offers free and paid accounts, and it uses open-source software (**Git**) to perform the version control functions.

---

Answers to the "Do I Know This Already?" quiz:

**1** C **2** A, B **3** D **4** A, C **5** B, C

Key Topic

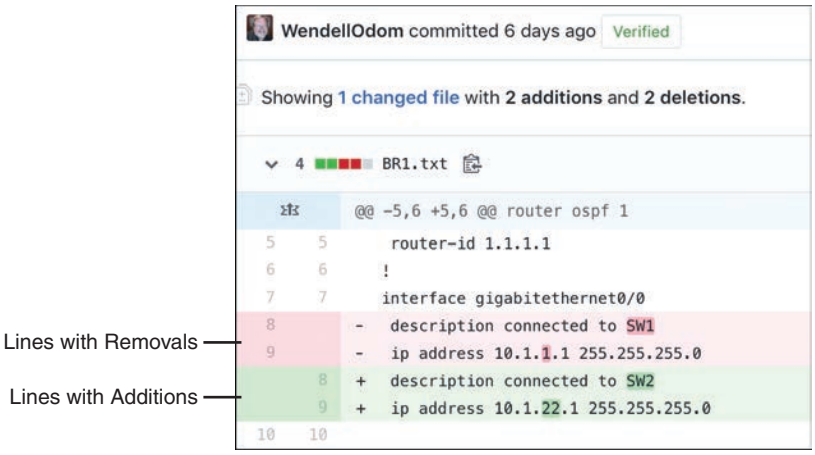



**Figure 24-3** *Showing File Differences in GitHub*

## Configuration Monitoring and Enforcement

With a version control system and a convention of storing the configuration files in a central location, a network team can do a much better job of tracking changes and answering the who, what, and when of knowing what changed in every device's configuration. However, using that model then introduces other challenges—challenges that can be best solved by also using an automated configuration management tool.

With this new model, engineers should make changes by editing the associated configuration files in the centralized repository. The configuration management tool can then be directed to copy or apply the configuration to the device, as shown in Figure 24-4. After that process completes, the central config file and the device's running-config (and startup-config) should be identical.

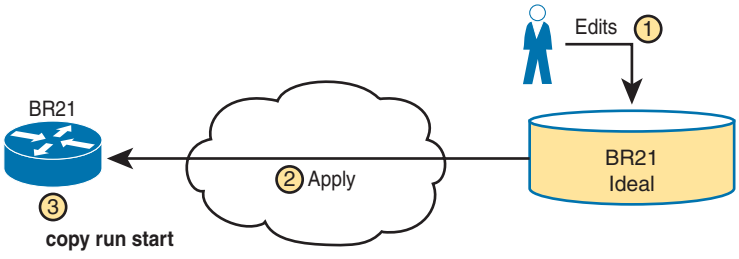



**Figure 24-4** *Pushing Centralized Configuration to a Remote Device*

Using the model shown in Figure 24-4 still has dangers. For instance, the network engineers should make changes by using the configuration management tools, but they still have the ability to log in to each device and make manual changes on each device. So, while the idea of using a configuration management tool with a centralized repository of config files sounds appealing, eventually someone will change the devices directly. Former correct configuration changes might be overwritten, and made incorrect, by future changes. In other words, eventually, some configuration drift can occur.

Configuration management tools can monitor device configurations to discover when the device configuration differs from the intended ideal configuration, and then either

reconfigure the device or notify the network engineering staff to make the change. This feature might be called **configuration monitoring** or *configuration enforcement*, particularly if the tool automatically changes the device configuration.

Figure 24-5 shows the general idea behind configuration monitoring. The automated configuration management software asks for a copy of the device's running-config file, as shown in steps 1 and 2. At step 3, the config management software compares the ideal config file with the just-arrived running-config file to check whether they have any differences (configuration drift). Per the configuration of the tool, it either fixes the configuration or notifies the staff about the configuration drift.

Key Topic

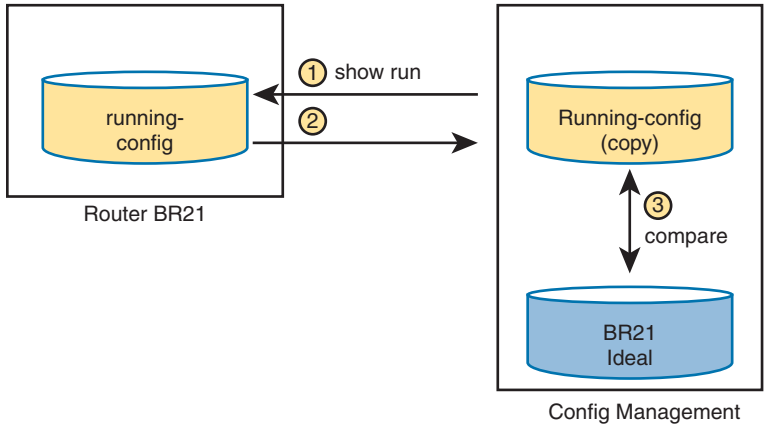



**Figure 24-5** *Configuration Monitoring*

## Configuration Provisioning

**Configuration provisioning** refers to how to provision or deploy changes to the configuration once made by changing files in the configuration management system. Included among the primary functions of a configuration management tool, you would likely see features like these:

Key Topic

- The core function to implement configuration changes in one device after someone has edited the device's centralized configuration file
- The capability to choose which subset of devices to configure: all devices, types with a given attribute (such as those of a particular role), or just one device, based on attributes and logic
- The capability to determine whether each change was accepted or rejected, and to use logic to react differently in each case depending on the result
- For each change, the capability to revert to the original configuration if even one configuration command is rejected on a device
- The capability to validate the change now (without actually making the change) to determine whether the change will work or not when attempted
- The capability to check the configuration after the process completes to confirm that the configuration management tool's intended configuration does match the device's configuration

- The capability to use logic to choose whether to save the running-config to startup-config or not
- The capability to represent configuration files as templates and variables so that devices with similar roles can use the same template but with different values
- The capability to store the logic steps in a file, scheduled to execute, so that the changes can be implemented by the automation tool without the engineer being present

The list could go further, but it outlines some of the major features included in all of the configuration management tools discussed in this chapter. Most of the items in the list revolve around editing the central configuration file for a device. However, the tools have many more features, so you have more work to do to plan and implement how they work. The next few pages focus on giving a few more details about the last two items in the list.

### Configuration Templates and Variables

Think about the roles filled by networking devices in an enterprise. Focusing on routers for a moment, routers often connect to both the WAN and one or more LANs. You might have a small number of larger routers connected to the WAN at large sites, with enough power to handle larger packet rates. Smaller sites, like branch offices, might have small routers, maybe with a single WAN interface and a single LAN interface; however, you might have a large number of those small branch routers in the network.

For any set of devices in the same role, the configurations are likely similar. For instance, a set of branch office routers might all have the exact same configuration for some IP services, like NTP or SNMP. If using OSPF interface configuration, routers in the same OSPF area and with identical interface IDs could have identical OSPF configuration.

For instance, Example 24-1 shows a configuration excerpt from a branch router, with the unique parts of the configuration highlighted. All the unhighlighted portions could be the same on all the other branch office routers of the same model (with the same interface numbers). An enterprise might have dozens or hundreds of branch routers with nearly identical configuration.

**Example 24-1** *Router BR1 Configuration, with Unique Values Highlighted*

```
hostname BR1
!
interface GigabitEthernet0/0
 ip address 10.1.1.1 255.255.255.0
 ip ospf 1 area 11
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/1/0
 ip address 10.1.12.1 255.255.255.0
 ip ospf 1 area 11
!
router ospf 1
 router-id 1.1.1.1
```

Configuration management tools can separate the components of a configuration into the parts in common to all devices in that role (the template) versus the parts unique to any one device (the variables). Engineers can then edit the standard template file for a device role as a separate file rather than each device's variable file. The configuration management tool can then process the template and variables to create the ideal configuration file for each device, as shown in Figure 24-6, which shows the configuration files being built for branch routers BR21, BR22, and BR23.

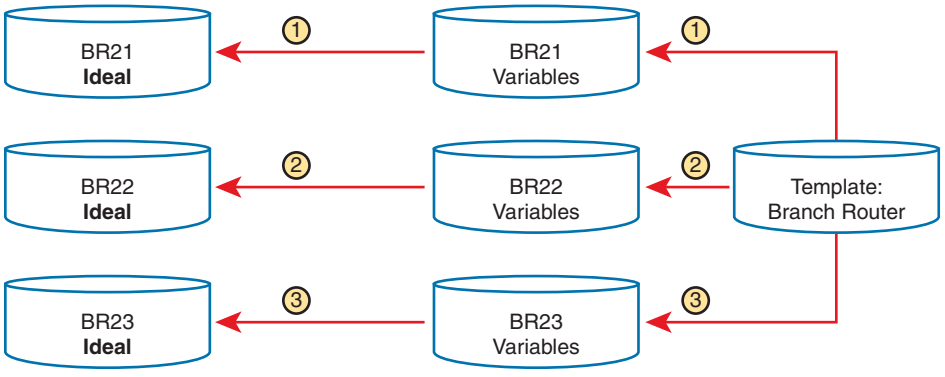



**Figure 24-6** *Concept: Configuration Templates and Variables*

To give a little more insight, Example 24-2 shows a template file that could be used by Ansible for the configuration shown in Example 24-1. Each tool specifies what language to use for each type of file, with Ansible using the Jinja2 language for templates. The template mimics the configuration in Example 24-1, except for placing variable names inside sets of double curly brackets.

Key Topic

**Example 24-2** *Jinja2 Template with Variables Based on Example 24-1*

```
hostname {{hostname}}
!
interface GigabitEthernet0/0
 ip address {{address1}} {{mask1}}
 ip ospf {{OSPF_PID}} area {{area}}
!
interface GigabitEthernet0/1
!
interface GigabitEthernet0/1/0
 ip address {{address2}} {{mask2}}
 ip ospf {{OSPF_PID}} area {{area}}
!
router ospf {{OSPF_PID}}
 router-id {{RID}}
```

To supply the values for a device, Ansible calls for defining variable files using YAML, as shown in Example 24-3. The file shows the syntax for defining the variables shown in the complete configuration in Example 24-1, but now defined as variables.

**Example 24-3** *YAML Variables File Based on Example 24-2*

```
---
hostname: BR1
address1: 10.1.1.1
mask1: 255.255.255.0
address2: 10.1.12.1
mask2: 255.255.255.0
RID: 1.1.1.1
area: '11'
OSPF_PID: '1'
```

The configuration management system processes a template plus all related variables to produce the intended configuration for a device. For instance, the engineer would create and edit one template file that looks like Example 24-2 and then create and edit one variable file like Example 24-3 for each branch office router. Ansible would process the files to create complete configuration files like the text shown in Example 24-1.

It might seem like extra work to separate configurations into a template and variables, but using templates has some big advantages. In particular:

Key Topic

- Templates increase the focus on having a standard configuration for each device role, helping to avoid snowflakes (uniquely configured devices).
- New devices with an existing role can be deployed easily by simply copying an existing per-device variable file and changing the values.
- Templates allow for easier troubleshooting because troubleshooting issues with one standard template should find and fix issues with all devices that use the same template.
- Tracking the file versions for the template versus the variables files allows for easier troubleshooting as well. Issues with a device can be investigated to find changes in the device's settings separately from the standard **configuration template**.

### Files That Control Configuration Automation

Configuration management tools also provide different methods to define logic and processes that tell the tools what changes to make, to which devices, and when. For instance, an engineer could direct a tool to make changes during a weekend change window. That same logic could specify a subset of the devices. It could also detail steps to verify the change before and after the change is attempted, and how to notify the engineers if an issue occurs.

Interestingly, you can do a lot of the logic without knowing how to program. Each tool uses a language of some kind that engineers use to define the action steps, often a language defined by that company (a domain-specific language). But they make the languages straightforward, and they are generally much easier to learn than programming languages. Configuration management tools also enable you to extend the action steps beyond what can be done in the toolset by using a general programming language. Figure 24-7 summarizes the files you could see in any of the configuration management tools.

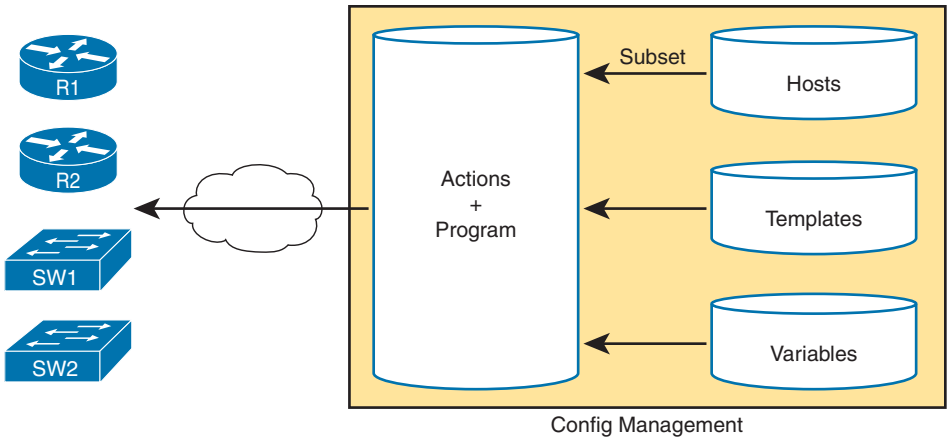



**Figure 24-7** *Important Files Used by Configuration Management Tools*

## Ansible and Terraform Basics

This chapter focuses on one exam topic that asks about the capabilities of two configuration management tools: Ansible and Terraform. The first major section of the chapter describes the capabilities of both (and other) configuration management tools. This second major section examines a few of the features of each tool, focusing on terminology and major capabilities.

Ansible and Terraform are software packages. You can purchase each tool, with variations on each package. However, they also have different free options that allow you to download and learn about the tools, although you might need to run a Linux guest because some of the tools do not run in a Windows OS.

As for the names, most people use the words *Ansible* and *Terraform* to refer to the companies as well as their primary configuration management products. Both emerged as part of the transition from hardware-based servers to virtualized servers, which greatly increased the number of servers and created the need for software automation to create, configure, and remove VMs. Both also produce one or more configuration management software products that have become synonymous with their companies in many ways. (This chapter follows that convention, for the most part ignoring exact product names, and referring to products and software simply as Ansible and Terraform.) The sections that follow discuss the basics of each.

### Ansible

To use **Ansible** (www.ansible.com), you need to install Ansible on some computer: Mac, Linux, or a Linux VM on a Windows host. You can use the free open-source version or use the paid Ansible Tower server version.

Once it is installed, you create several text files, many of which use YAML (Yet Another Markup Language), including

- **Playbooks:** These files provide actions and logic about what Ansible should do.
- **Inventory:** These files provide device hostnames and information about each device, like device roles, so Ansible can perform functions for subsets of the inventory.

- **Templates:** Using Jinja2 language, the templates represent a device's configuration but with variables (see Example 24-2).
- **Variables:** Using YAML, a file can list variables that Ansible will substitute into templates (see Example 24-3).



Ansible uses an **agentless architecture**. That means Ansible does not rely on any code (agent) running on the network device. Instead, Ansible relies on features typical in network devices, specifically SSH and NETCONF, to make changes and extract information. When using SSH, the Ansible control node changes the device like any other SSH user, logging in and using configuration mode; however, the Ansible code does the work rather than a human user.

By default, Ansible uses a **push model** (per Figure 24-8). However, it is also capable of using a **pull model** with a program called Ansible-pull if you want that capability. After installing Ansible, an engineer must create and edit various Ansible files, including an **Ansible playbook**. The playbook details the actions the engineer wants to take, acting as a program with steps taken under different conditions. Then the engineer runs the playbook, and Ansible performs the steps. Those steps can include configuring one or more devices per the various files (step 3), with the control node seen as pushing the configuration to the device.

Key Topic

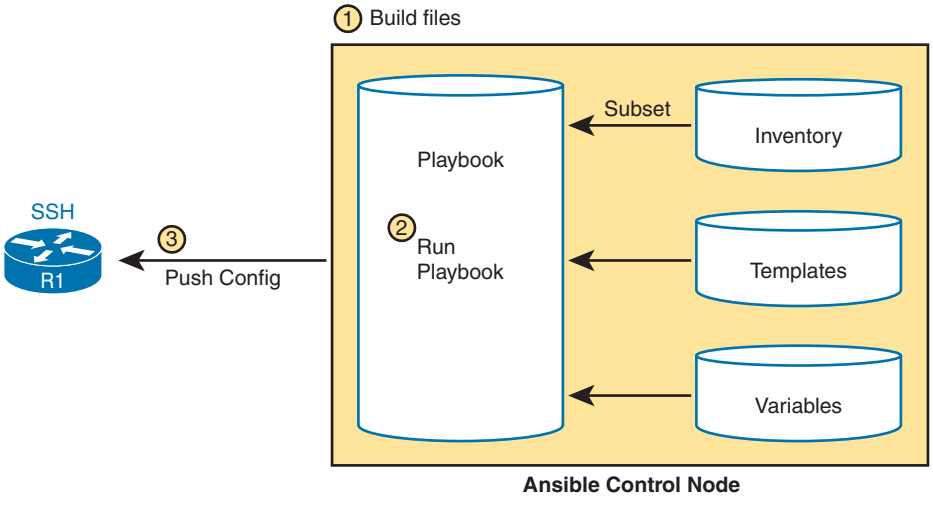



**Figure 24-8** *Ansible Push Model*

Ansible can do configuration provisioning (configuring devices based on the changes made in the files) and configuration monitoring (checking to find out whether the device config matches the ideal configuration on the control node). However, Ansible's architecture more naturally fits with configuration provisioning, as seen in the figure. To do configuration monitoring, Ansible uses logic modules that detect and list configuration differences, after which the playbook defines what action to take (reconfigure or notify).

## Terraform

**Terraform** by HashiCorp (www.terraform.io) is an infrastructure as code (IaC) tool that enables you to create both cloud and on-premises resources by leveraging easy-to-read configuration files. These files use configuration version control, can be reused, and

can be shared with other developers or code contributors. Terraform provides a straightforward workflow to provision and manage your infrastructure throughout its lifecycle, making day-to-day operations much simpler to manage. Terraform can also handle components such as compute, storage, and networking as well as components such as DNS and features from Software as a Service (SaaS) applications. The following list highlights some of the main components of Terraform and their associated roles in managing your infrastructure:

- **Configuration files:** Files that describe the desired state of the infrastructure and specify resources along with their configurations.
- **Providers:** Plugins that enable interaction with different infrastructure platforms (AWS, Azure, Google Cloud).
- **Resources:** Components you want to create and manage, such as virtual machines, networks, or databases.
- **Data sources:** Reference details about existing infrastructure within your configuration.
- **Variables:** A method to reuse code and make configurations more flexible.
- **State files:** Record the infrastructure's current state and track configuration changes that are made.
- **Modules:** Organize and reuse Terraform code. Modules encapsulate a set of resources and configurations that can be treated as a single unit. Modules promote code reusability and modularity.

Terraform works with cloud providers and other services by utilizing their associated APIs. This is enabled by the use of the providers mentioned previously. This enables Terraform to connect with almost any service or platform, which is why it has become widely used as a common approach to IaC.

Terraform has its own language called HashiCorp Configuration Language (HCL). The language is human readable and fairly easy to understand like Ansible; however, Ansible is best suited for device configurations whereas Terraform is more often used for infrastructure provisioning.

There are three stages to Terraform's core workflow:

1. **Write:** This is where you define the resources that will be used across multiple cloud providers or services.
2. **Plan:** An execution plan is created that describes the infrastructure that will be created, updated, or destroyed based on the current configuration and infrastructure.
3. **Apply:** Once the plan is approved, the proposed operations will be completed in the appropriate order.

The saying "A picture is worth a thousand words" fits nicely for this technology. Figure 24-9 depicts the high-level architecture of Terraform, its core workflow, and how it interacts with various cloud providers and other infrastructure.

Key Topic

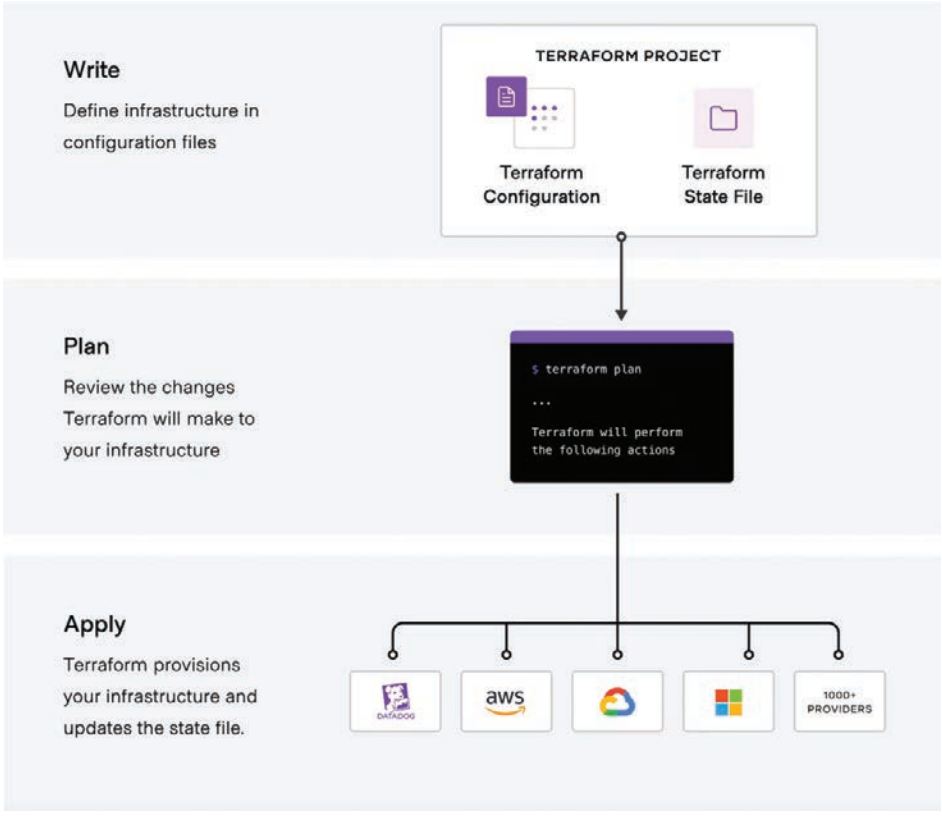



**Figure 24-9** *High-level Terraform Architecture*

## Summary of Configuration Management Tools

The configuration management tools discussed in this chapter have different strengths. When considering one of these tools it is best to first understand what you are trying to accomplish from a design perspective. For example, if you or your team is very fluent in CLI and want to automate device configurations across many devices, Ansible might be your first choice due to the similarity of the playbooks and the actual CLI.

However, if you are looking to lead with IaC, do more advanced capabilities of automation, need a means to keep track of configurations, and do provisioning for both on-premise or cloud infrastructure, you may consider Terraform. Sometimes, you might have both. That is the beauty of programmability and automation—there are many ways to solve business needs by leveraging different methods. Skill set, preference, and cost usually determine which tool(s) your organization will implement.

Table 24-2 summarizes a few of the most common ideas about each of the automated configuration management tools.

Key Topic

**Table 24-2** Comparing Ansible and Terraform

| Attribute | Ansible | Terraform |
|---|---|---|
| File type for actions or config | Playbook | Configuration |
| Protocol to network device | SSH, NETCONF | APIs |
| Execution Mode | Agentless | Client/server model |
| Push or pull model | Push (by default)<br>Pull w/Ansible-pull program | Push |
| Modeling language | YAML | HashiCorp Configuration Language (HCL) |

## Chapter Review

One key to doing well on the exams is to perform repetitive spaced review sessions. Review this chapter's material using either the tools in the book or interactive tools for the same material found on the book's companion website. Refer to the "Your Study Plan" element for more details. Table 24-3 outlines the key review elements and where you can find them. To better track your study progress, record when you completed these activities in the second column.

**Table 24-3** Chapter Review Tracking

| Review Element | Review Date(s) | Resource Used |
|---|---|---|
| Review key topics | | Book, website |
| Review key terms | | Book, website |
| Answer DIKTA questions | | Book, PTP |
| Review memory table | | Book, website |
| Do DevNet Labs | | DevNet |

## Review All the Key Topics

Key Topic

**Table 24-4** Key Topics for Chapter 24

| Key Topic Element | Description | Page Number |
|---|---|---|
| List | Issues that arise from configuration drift | 555 |
| Figure 24-3 | Sample of showing router configuration file differences with GitHub | 557 |
| Figure 24-5 | Basic configuration monitoring concepts | 558 |
| List | Primary functions of a configuration management tool | 558 |
| Example 24-2 | Sample Jinja2 Ansible template | 560 |
| List | Advantages of using configuration templates | 561 |

| Key Topic Element | Description | Page Number |
|---|---|---|
| Figure 24-8 | Ansible's push model and other features | 563 |
| Figure 24-9 | Terraform's high-level architecture | 565 |
| Table 24-2 | Summary of configuration management features and terms | 566 |



## Key Terms You Should Know

agentless architecture, Ansible, Ansible Playbook, configuration drift, configuration management tool, configuration monitoring, configuration provisioning, configuration template, Git, pull model, push model, Terraform

## Do DevNet Labs

The Cisco DevNet site (https://developer.cisco.com)—a free site—includes lab environments and exercises. You can learn a lot about configuration management, and Ansible in particular, with a few of the lab tracks on the DevNet site (at the time this book was published). Navigate to https://developer.cisco.com and search for learning labs about Ansible and Terraform.



# Part VI Review

Keep track of your part review progress with the checklist shown in Table P6-1. Details on each task follow the table.

**Table P6-1** Part VI Review Checklist

| Activity | 1st Date Completed | 2nd Date Completed |
|---|---|---|
| Repeat All DIKTA Questions | | |
| Answer Part Review Questions | | |
| Review Key Topics | | |
| Watch video | | |
| Use Per-Chapter Interactive Review | | |

## Repeat All DIKTA Questions

For this task, answer the "Do I Know This Already?" questions again for the chapters in this part of the book, using the PTP software.

## Answer Part Review Questions

For this task, answer the Part Review questions for this part of the book, using the PTP software.

## Review Key Topics

Review all key topics in all chapters in this part, either by browsing the chapters or by using the Key Topics application on the companion website.

## Watch video

The companion website includes a variety of common mistake and Q&A videos organized by part and chapter. Use these videos to challenge your thinking, dig deeper, review topics, and better prepare for the exam. Make sure to bookmark a link to the companion website and use the videos for review whenever you have a few extra minutes.

## Use Per-Chapter Interactive Review

Using the companion website, browse through the interactive review elements, like memory tables and key term flashcards, to review the content from each chapter.

*This page intentionally left blank*

*This page intentionally left blank*

# Part VII

## Exam Updates and Final Review

**Chapter 25:** *CCNA 200-301 Official CertGuide, Volume 2*, Second Edition, Exam Updates

**Chapter 26:** Final Review



CHAPTER 25

# *CCNA 200-301 Official Cert Guide, Volume 2*, Second Edition, Exam Updates

## The Purpose of This Chapter

For all the other chapters, the content should remain unchanged throughout this edition of the book. Instead, this chapter will change over time, with an updated online PDF posted so you can see the latest version of the chapter even after you purchase this book.

Why do we need a chapter that updates over time? For two reasons:

1. To add more technical content to the book before it is time to replace the current book edition with the next edition. This chapter will include additional technology content and possibly additional PDFs containing more content.
2. To communicate detail about the next version of the CCNA exam, to tell you about our publishing plans for that edition, and what that means to you.

To find the latest version of this chapter, follow the process below. Bookmark the link so that any time you refer to this chapter, begin by downloading a new copy. Use these steps:

If you have not yet accessed the companion website, follow these steps:

**Step 1.** Browse to www.ciscopress.com/register.

**Step 2.** Enter the print book ISBN (even if you are using an eBook): **9780138214951**.

**Step 3.** After registering the book, go to your account page and select the **Registered Products** tab.

**Step 4.** Click the **Access Bonus Content** link to access the companion website. Select the link or scroll down to that section to check for updates.

Table 25-1 summarizes the information that this version of the chapter includes. Use the table as a quick reference for the detail to expect in the rest of the chapter.

**Table 25-1** Status for Available New Technical Content for This Edition

| Chapter Version | 1 |
|---|---|
| Most recent CCNA 200-301 blueprint version when this chapter was most recently released | 1.1 |
| Is there technology content in the latter part of this chapter? | No |
| Is there technology content in other downloadable files? | No |
| Links to other file downloads | N/A |
| Is there information about the specifics of the new exam? | No |

Any additional file downloads, or further instructions for file downloads, will be posted at this book's companion website. See the heading "How to Access the Companion Website" in the Introduction to this book for details on finding this book's companion website.

The following two sections give more detail about the primary purposes of the chapter. Any technical content or exam update detail follows later in the chapter.

## Additional Technical Content

On rare occasions, the book author might want to add book content mid-edition. This chapter provides the means to do that, as we will publish updated versions of this chapter in PDF form at the publisher's website.

Several reasons exist for new content. It could just be the author looks at a chapter and feels like it needs more. It could be that the technology changes enough to warrant an update before the new edition. Cisco also can (and has) changed exam topic wording without a formal announcement or change to the blueprint version; any related new content would appear in this chapter.

You do not need to check frequently for a new PDF. You may want to check more regularly about exam updates, as discussed in the next section. But to check for new technical content, I suggest downloading the PDF at the beginning of the book and the end of the book. If you download it to begin, you can read content related to the chapters when you first read those chapters. Downloading at the end ensures you have seen the latest available version of this chapter.

**NOTE** An equivalent chapter exists in the *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition. When doing your final study, check for a new version of that element.

Of course, if this chapter adds new technical content, treat it like any of the other chapters in the book, and study the content!

## Official Blueprint Changes

Cisco introduced CCNA and CCNP in 1998. For the first 25 years of those certification tracks, Cisco updated the exams on average every 3–4 years. However, Cisco did not preannounce the exam changes, so exam changes felt very sudden. Usually, a new exam would be announced, with new exam topics, giving you 3–6 months before your only option was to take the new exam. As a result, you could be studying with no idea about Cisco's plans, and the next day, you had a 3–6-month timeline to either pass the old exam or pivot to prepare for the new exam.

Thankfully, Cisco changed their exam release approach in 2023. Called the Cisco Certification Roadmap (https://cisco.com/go/certroadmap), the new plan includes these features:

1. Cisco considers changes to all exam tracks (CCNA, CCNP Enterprise, CCNP Security, and so on) annually.
2. Cisco uses a predefined annual schedule for each track, so you know the timing of possible changes to the exam you are studying for, even before any announcements.

3. The schedule moves in a quarterly sequence:
   a. Privately review the exam to consider what to change.
   b. Publicly announce if an exam is changing, and if so, announce details like exam topics and release date.
   c. Release the new exam.
4. Exam changes might not occur each year. If changes occur, Cisco characterizes them as minor (less than 20 percent change) or major (more than 20 percent change).

The specific dates for a given certification track can be confusing because Cisco organizes the work by fiscal year quarters. As an Example, Figure 25-1 shows the 2024 fiscal year. Their fiscal year begins in August, so, for example, the first quarter (Q1) of fiscal year (FY) 2024 began in August 2023.

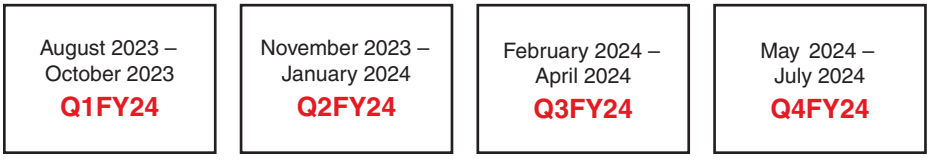



**Figure 25-1** *Cisco Fiscal Year and Months Example (FY2024)*

Focus more on the sequence of the quarters to understand the plan. Figure 25-2 shows an example sequence in which Cisco updates the CCNA 200-301 exam, assuming a minor release (less than 20 percent change).

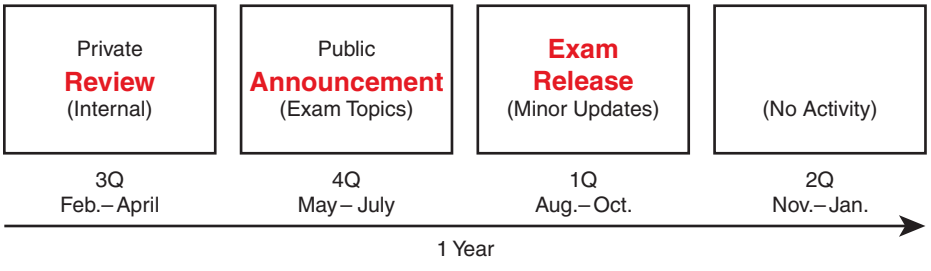



**Figure 25-2** *Cisco CCNA Annual Roadmap with a Minor Release*

Over time, Cisco might make no changes in some years and minor changes in others. For example, Cisco announced CCNA 200-301 version 1.1 in the FY24-FY25 cycle (February 2024–January 2025). Figure 25-3 shows what could happen in the next four years. It first shows a year with no changes, then a year with minor changes, another year with no changes, and a year with major changes.

Figure 25-3 shows an example and does not reveal any secret knowledge about Cisco's plans; however, it shows a volume of change that matches the rate of change to the exam over its long history.

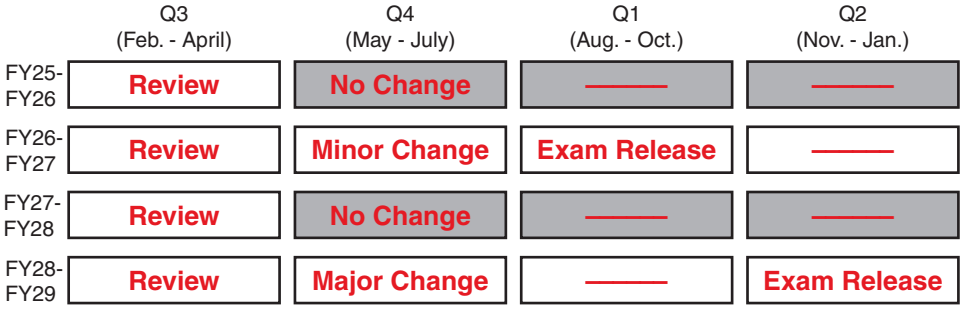



**Figure 25-3** *An Example of What Could Happen with CCNA over Four Years*



## Impact on You and Your Study Plan

Cisco's new policy helps you plan, but it also means that the CCNA exam might change before you pass the current exam. That impacts you, affecting how we deliver this book to you. This chapter gives us a way to communicate in detail about those changes as they occur. But you should watch other spaces as well.

Your study plan ongoing should follow this general process:

**Step 1.** Continue to use the *CCNA 200-301 Official Cert Guides, Volumes 1 and 2*, Second Edition, as is.

**Step 2.** Monitor for updates to this Exam Updates chapter, plus the equivalent chapter in Volume 1, to find additional content for topics Cisco may add to the exam.

**Step 3.** To be aware of changes to the CCNA exam, monitor the information sources listed below.

For those other information sources to watch, bookmark and check these sites for news. In particular:

**Cisco:** Check their Certification Roadmap page: https://cisco.com/go/certroadmap. Make sure to sign up for automatic notifications from Cisco on that page.

**Publisher:** Check this page about new certification products, offers, discounts, and free downloads related to the more frequent exam updates: www.ciscopress.com/newcert.

**Cisco Learning Network:** Subscribe to the CCNA Community at learningnetwork.cisco.com, where I expect ongoing discussions about exam changes over time. If you have questions, search for "roadmap" in the CCNA community, and if you do not find an answer, ask a new one!

**Author:** Look for blog posts labeled as News at www.certskills.com. For every new edition, I post about the new exam, new topics, and how to manage the transition. I will continue to do so now with more detail about the new annual cycle. Watch that space and consider subscribing.

As changes arise, I will update this chapter with more detail about exam and book content. Given Cisco's certification roadmap, that means somewhere in each year's fiscal fourth quarter (May–July). At that point, I will publish an updated version of this chapter, listing our content plans. That detail will likely include the following:

- Content removed, so if you plan to take the new exam version, you can ignore those when studying
- New content planned per new exam topics, so you know what's coming

While I do not think Cisco will change CCNA every year—in my opinion, I predict Cisco will change it less than half the years—everyone needs to be aware of the possibility and timing of exam changes and new exam releases. I will use this chapter to communicate the exam and book content details. Look to the other sites from the previous list for brief news about plans, but look here for the detail.

The remainder of the chapter shows the new content that may change over time.

## News About the Next CCNA Exam Release

This statement was last updated in March 2024, before the publication of the *CCNA 200-301 Official Cert Guide, Volume 1,* Second Edition.

This version of this chapter has no news to share about the next CCNA exam release.

At the most recent version of this chapter, the CCNA 200-301 exam version number was Version 1.1.

## Updated Technical Content

The current version of this chapter has no additional technical content.

*This page intentionally left blank*



CHAPTER 26

# Final Review

Congratulations! You made it through the books, and now it's time to finish getting ready for the exam. This chapter helps you get ready to take and pass the exam in two ways.

The first half of this chapter focuses on the exam event. Now you need to think about what happens during the exam and what you need to do in these last few weeks before taking the exam. At this point, everything you do should be focused on getting you ready to pass so that you can finish up this hefty task.

The second section of this chapter focuses on final content review. You should not just complete the chapters in the combined CCNA Volume 1 and 2 books. Instead, you need to review, refine, deepen, and assess your skills. This second section of this chapter gives advice and suggestions on how approach your final weeks of study before you take the CCNA 200-301 exam.

## Advice About the Exam Event

Now that you have finished the bulk of this book, you could just register for your Cisco CCNA exam; show up; and take the exam. However, if you spend a little time thinking about the exam event itself, learning more about the user interface of the real Cisco exams, and the environment at the Pearson VUE testing centers, you will be better prepared, particularly if this is your first Cisco exam.

This first of two major sections in this chapter gives some advice about the Cisco exams and the exam event itself, specifically about

- Question Types
- Your Time Budget
- An Example Time-Check Method
- The Final Week Before Your Exam
- 24 Hours Before Your Exam
- The Final 30 Minutes Before Your Exam
- The Hour After Your Exam

### Learn About Question Types

In the weeks leading up to your exam, you should think more about the different types of exam questions and have a plan for how to approach those questions. To get started, watch the Cisco Certification Exam Tutorial Video. You can find it these ways:

- **Direct link:** https://learningnetwork.cisco.com/s/certification-exam-tutorials

- **Internet search:** Search for "Cisco Certification Exam Tutorial Video" and you will likely find it.
- **Blog post:** Use https://www.certskills.com/final-review, which links to a blog page with a link to the video page (as well as other links useful for final review).

Cisco does not give a definitive list of all question types used on any Cisco exam; however, the videos and this section describe all the question types used at the time this book was published.

While you're watching any of the videos about the exam tutorial, the exam user interface has these conventions:

- Shows circles beside the answers in single-answer questions.
- Shows squares beside the answers in multiple-answer questions.
- Prevents you from choosing too many or too few answers.
- Supplies a popup window to tell you if you have selected too few answers if you try to move to the next question, so you can stop and go back and answer with the correct number of answers.
- Applies no penalty for guessing—so you should always supply the number of answers that the question asks for.

**To emphasize: Cisco has no penalty for guessing, and the user interface helps ensure you choose the correct number of answers. Always answer, and answer with the stated number of correct answers.**

Drag-and-drop questions require you to move items in the user interface with mouse actions. The draggable items begin in one location, and you will drag-and-drop them to answer. The draggables may be words, but they can also be graphics.

Within one drag-and-drop question, you can change your mind until satisfied. To do so, after dragging an item, just drag it back to its original position. The user interface does not consider you to have answered the question until you click the next button.

Lab questions directly assess your CLI skills. They present you with a lab scenario with a lab pod of virtual routers and switches running in the background, giving you console access to a few devices. Your job: find the missing or broken configuration and reconfigure the devices so that the lab scenario works. Some of the key points to consider with lab questions include

1. Learn the lab question user interface as much as possible beforehand by watching the "Cisco Certification Exam Tutorial Video."
2. Expect to navigate often between the tabs on the left of the user interface, particularly the topology (drawing) and tasks tabs. The topology shows a figure of the network, with the tasks tab listing what you need to do to gain the points for the question. (The guidelines tab typically lists generic instructions that apply to every lab question.)
3. The right side of the user interface has tabs for the console of each device—just click the tab to get to the CLI of the device.

4. Lab questions run virtualized Cisco operating systems. They do not use simulations. So, the CLI should work like real devices, with all commands supported, all options, with navigation tools like ?, tab, and up arrow.
5. You answer by changing the configuration, so make sure to issue the **copy running-config startup-config** command (or **copy run start**) to save your configuration.
6. Cisco gives partial credit on lab questions, so do as much as you can, even if you do not know how to solve the entire lab.

Figure 26-1 shows a screen capture from the video, noting the UI tabs.

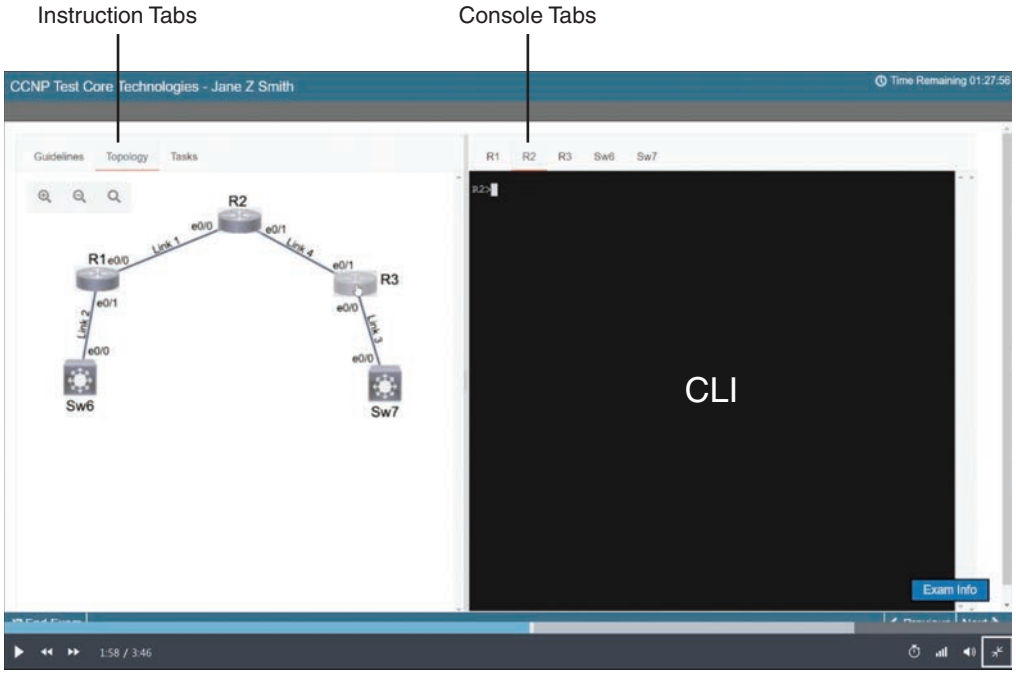



**Figure 26-1** *Cisco Lab Question User Interface*

**NOTE** Those of you with some prior experience with CCNA will notice that lab questions seem a lot like the old Sim question type. The old Sim questions required the same actions from us as test takers: read the scenario and fix the configuration. However, Sim questions used simulated Cisco devices with limited command support, which frustrated some test takers. Lab questions use virtualized Cisco operating systems, creating a much more realistic experience.

On a final note about question types, Cisco can and may change the question types, but in recent years, the CCNA exam has used the types listed here.

## Think About Your Time Budget

Several factors about a Cisco exam experience make time management a challenge, including

1. The test has a time limit. The user interface shows the time count down to 0:00.
2. The test has a defined number of questions that you learn when you begin the test. The user interface shows the current question, counting up from 1 to the number of questions you will see.
3. You cannot skip a question or go back to an earlier question. Once you move on to the next question, you cannot navigate back to it.
4. You do not know how many time-consuming questions like lab questions exist on your exam, even after you start your exam. Instead, they just appear like any other question.



Because of Cisco's exam rules, you need to take extra care to manage your time and keep an eye on your speed. Going too slowly hurts you because you might not have time to answer all the questions. Going too fast hurts if you are rushing due to worry about running out of time. So, you need to be able to somehow know whether you are moving quickly enough to answer all the questions while not rushing.

You need a plan for how you will check your time, a plan that does not distract you from the exam. You can ponder the facts listed here and come up with your own plan. If you want a little more guidance, the next topic shows one way to check your time that uses some simple math so it does not take much time away from the test.

## An Example Time-Check Method

As a suggestion, you can use the following math to do your time check in a way that weights the time based on those time-consuming questions. You do not have to use this method. But this math uses only addition of whole numbers, to keep it simple. It gives you a pretty close time estimate, in my opinion.

The concept is simple. Just do a simple calculation that estimates the time you should have used so far. Here's the math:

Number of questions answered so far + 7 per time-consuming question answered so far

Then you check the timer to figure out how much time you have spent:

- You have used exactly that much time, or a little more time: your timing is perfect.
- You have used less time: you are ahead of schedule.
- You have used noticeably more time: you are behind schedule.

For example, first you need to make a habit of keeping count of how many time-consuming questions you see. For instance, write "labs" on your note sheet and add a mark each time you see a lab question. For the math, for instance, if you have already finished 17 questions, 1 of which was a lab question, your time estimate is 17 + 7 = 24 minutes. If your actual time is also 24 minutes, or maybe 25 or 26 minutes, you are right on schedule. If you have spent less than 24 minutes, you are ahead of schedule.

So, the math is pretty easy: Questions answered, plus 7 per time-consuming question, is the guesstimate of how long you should have taken so far if you are right on time.

**NOTE** This math is an estimate; I make no guarantees that the math will be an accurate predictor on every exam.

## One Week Before Your Exam

I have listed a variety of tips and advice in the next few pages, broken down by timing versus the big exam event. First, this section discusses some items to consider when your exam is about a week away:

- **Get some earplugs:** Testing centers often have some, but if you do not want to chance it, come prepared with your own. (They will not let you bring your own noise-canceling headphones into the room if they follow the rules disallowing any user electronic devices in the room, so think low-tech disposable earplugs, or even bring a cotton ball.) The testing center is typically one room within a building of a company that does something else as well, often a training center, and almost certainly you will share the room with other test takers coming and going. So, there are people talking in nearby rooms and other office noises. Earplugs can help.
- **Create a note-taking plan:** Some people like to spend the first minute of the exam writing down some notes for reference, before actually starting the exam. For example, maybe you want to write down the table of magic numbers for finding IPv4 subnet IDs. If you plan to do that, practice making those notes between now and exam day. Before each practice exam, transcribe those lists, just like you expect to do at the real exam.
- **Plan your travel to the testing center:** Leave enough time in your schedule so that you will not be rushing to make it just in time.
- **Practice your favorite relaxation techniques for a few minutes before each practice exam:** That way you can enter the exam event and be more relaxed and have more success.

## 24 Hours Before Your Exam

You wake up on the big day—what should you be doing and thinking? Certainly, the better prepared you are, the better chances you have on the exam. But these small tips can help you do your best on exam day:

- Rest the night before the exam rather than staying up late to study. Clarity of thought is more important than one extra fact, especially because the exam requires so much analysis and thinking rather than just remembering facts.
- Bring as few extra items with you as possible when leaving for the exam center. You may bring personal effects into the building and testing company's space, but not into the actual room in which you take the exam. So, save a little stress and bring as little

extra stuff with you as possible. If you have a safe place to leave briefcases, purses, electronics, and so on, leave them there. However, the testing center should have a place to store your things as well. Simply put, the less you bring, the less you have to worry about storing. (For example, I have been asked to remove even my analog wristwatch on more than one occasion.)

- Plan time in your schedule for the day to not rush to get there and not rush when leaving, either.
- Do not drink a 64-ounce caffeinated drink on the trip to the testing center! After the exam starts, the exam timer will not stop while you go to the restroom.
- Use any relaxation techniques that you have practiced to help get your mind focused while you wait for the exam.



## 30 Minutes Before Your Exam

It's almost time! Here are a few tips for those last moments.

- Ask the testing center personnel for earplugs if you did not bring any—even if you cannot imagine using them. You never know whether using them might help.
- Ask for extra pens and laminated note sheets. The exam center will give you a laminated sheet and dry erase markers with which to take notes. (Test center personnel typically do not let you bring paper and pen into the room, even if supplied by the testing center.) I always ask for a second pen as well.
- Test your pens and sheets before going into the room to take the exam. Better to get a replacement pen before the clock starts.
- Grab a few tissues from the box in the room, for two reasons. One, to avoid having to get up in the middle of the exam if you need to sneeze. Two, if you need to erase your laminated sheet, doing that with a tissue paper rather than your hand helps prevent the oil from your hand making the pen stop working well.
- Find a restroom to use before going into the testing center, or just ask where one is, to avoid needing to go during the approximately two-hour exam event. Note that the exam timer does not stop if you need to go to the restroom during the exam, and you first have to go find the exam center contact before just heading to the restroom, so it can cost you a few minutes.

## The Hour After Your Exam

Some people pass these exams on the first attempt, and some do not. The exams are not easy. If you fail to pass the exam that day, you will likely be disappointed. And that is understandable. But it is not a reason to give up. In fact, I added this short topic to give you a big advantage in case you do fail.

**The most important study hour for your next exam attempt is the hour just after your failed attempt.**

Before you take the exam, prepare for how you will react if you do not pass. That is, prepare your schedule to give yourself an hour, or at least a half an hour, immediately after the exam attempt, in case you fail. Follow these suggestions to be ready for taking notes:

- Bring pen and paper, preferably a notebook you can write in if you have to write standing up or sitting somewhere inconvenient.
- Make sure you know where pen and paper are so that you can take notes immediately after the exam. Keep these items in your backpack if using the train or bus, or on the car seat in the car.
- Install an audio recording app on your phone, and be prepared to start talking into your app when you leave the testing center.
- Before the exam, scout the testing center, and plan the place where you will sit and take your notes, preferably somewhere quiet.

Then, once you complete the exam, if you do not pass on this attempt, use the following process when taking notes:

- Write down anything in particular that you can recall from any question.
- Write down details of questions you know you got right as well, because doing so may help trigger a memory of another question.
- Draw the figures that you can remember.
- Most importantly, write down any tidbit that might have confused you: terms, configuration commands, **show** commands, scenarios, topology drawings, anything.
- Take at least three passes at remembering. That is, you will hit a wall where you do not remember more. So, start on your way back to the next place, and then find a place to pause and take more notes. And do it again.
- When you have sucked your memory dry, take one more pass while thinking of the major topics in the book, to see if that triggers any other memory of a question.

Once collected, *you cannot share the information with anyone,* because doing so would break the Cisco nondisclosure agreement (NDA). Cisco considers cheating a serious offense and strongly forbids sharing this kind of information publicly. But you can and should use everything you can recall from your first attempt to prepare for your second attempt. See the section "Adjustments for Your Second Attempt" in this chapter for the rest of the story.

## Exam Review

At this point, you should have read the other chapters in the *CCNA 200-301 Official Cert Guide, Volumes 1 and 2*, and you have done the Chapter Review and Part Review tasks. Now you need to do the final study and review activities before taking the exam, as detailed in this section.

This section suggests some new activities and repeats some activities that have been previously mentioned. However, whether the activities are new or old to you, they all focus on filling in your knowledge gaps, finishing off your skills, and completing the study process.

You need to be ready to take an exam, so the Exam Review asks you to spend a lot of time answering exam questions. The next several pages discuss the best ways to make use of the practice questions that come with this book for your final preparation. The rest of the chapter then offers other suggestions for final exam preparation.

## Using Practice Questions

Both this book and the associated Volume 1 book come with a set of practice questions. The Pearson Test Prep (PTP) user interface, both web and desktop app, groups questions into exams. Figure 26-2 lists the exams supplied with the Volume 1 and Volume 2 books. (You might recall some of this information from the "Your Study Plan" section of the *CCNA 200-301 Official Cert Guide, Volume 1*, Second Edition, found just before Chapter 1 of that book.)

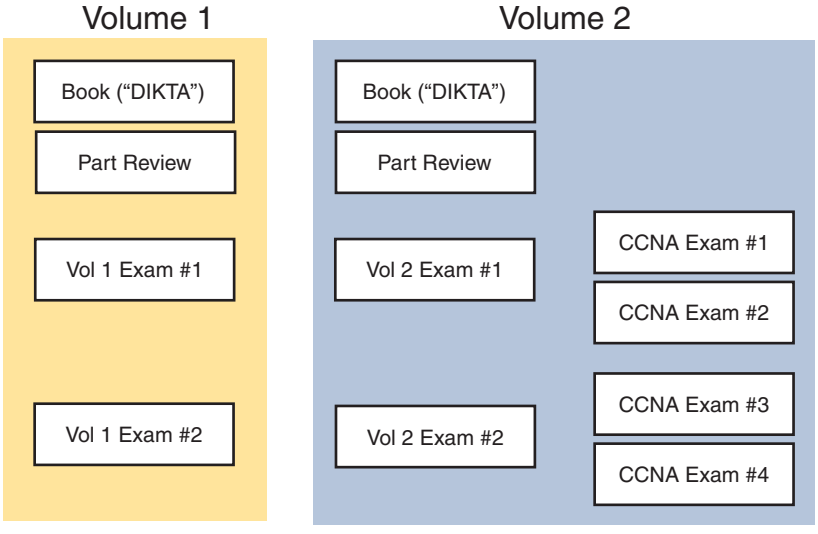



**Figure 26-2** *Exams Included in the Combined CCNA 200-301 Official Cert Guide, Volumes 1 and 2*

Volume 1 comes with a set of four exams. The "book" exam holds copies of the chapter-beginning "Do I Know This Already?" quiz questions. The part review exam includes the questions meant for use during part review. Exams 1 and 2 supply additional practice questions about Volume 1 content that you could use anytime, even now during your final review.

Volume 2 includes an equivalent set of four exams, but with questions about the technologies discussed in Volume 2.

Additionally, the publisher delivers CCNA exams to you in the Volume 2 book. Those exams include questions about the technologies in both books, making them excellent candidates to be used for timed practice exams.

The next few pages suggest how to use the practice questions for review at this late stage in your CCNA journey.

### Hold Practice Exam Events

One day soon, you need to pass a real Cisco exam at a Pearson VUE testing center. So, it's time to practice the real event as much as possible.

A practice exam using the PTP exam software lets you experience many of the same challenges as when taking a real Cisco exam. When you select *practice exam* mode, PTP software (both desktop and web) gives you a number of questions, with an upward question counter and a countdown timer shown in the window. When in practice exam mode, you cannot move backward to a previous question (yes, that's true on Cisco exams). And like the real exam, if you run out of time, the questions you did not answer count as incorrect.

The process of taking the timed practice exams helps you prepare in three key ways:

- To practice the exam event itself, including time pressure, the need to read carefully, and the need to concentrate for long periods
- To build your analysis and critical thinking skills when examining the network scenario built in to many questions
- To discover the gaps in your networking knowledge so that you can study those topics before the real exam

As much as possible, treat the practice exam events as if you were taking the real Cisco exam at a VUE testing center. The following list gives some advice on how to make your practice exam more meaningful, rather than just one more thing to do before exam day rolls around:

- Set aside 2.5 hours for taking a 120-minute timed practice exam.
- Make a list of what you expect to do for the 10 minutes before the real exam event. Then visualize yourself doing those things. Before taking each practice exam, practice those final 10 minutes before your exam timer starts. (The earlier section "30 Minutes Before Your Exam" lists some suggestions about what to do.)
- You cannot bring anything with you into the VUE exam room, so remove all notes and help materials from your work area before taking a practice exam. You can use blank paper, a pen, and your brain only. Do not use calculators, notes, web browsers, or any other app on your computer.
- Real life can get in the way, but if at all possible, ask anyone around you to leave you alone for the time you will practice. If you must do your practice exam in a distracting environment, wear headphones or earplugs to reduce distractions.
- To your benefit, do not guess on the practice exam event, hoping to improve your score. Answer only when you have confidence in the answer. Then, if you get the question wrong, you can go back and think more about the question in a later study session.

Hold your practice exam events using the CCNA exams listed in the PTP user interface. Just select those exams, and de-select the others. Or, select CCNA exam 1 for one event and CCNA exam 2 for the other. Also, select a 100-question limit. Then you simply need to choose the **Practice Exam** option in the upper right and start the exam.

You should plan to take between one and three practice exams with the supplied CCNA exam databases. Even people who are already well prepared should do at least one practice exam, just to experience the time pressure and the need for prolonged concentration.

Table 26-1 gives you a checklist to record your different practice exam events. Note that recording both the date and the score is helpful for some other work you will do, so note both. Also, in the Time Notes section, if you finish on time, note how much extra time you had; if you run out of time, note how many questions you did not have time to answer. Finally, note that you completed gap review (discussed later in this chapter).

**Table 26-1** CCNA Practice Exam Checklist

| Exam Bank(s) Used | Date Taken | Score | Time Notes | Completed Gap Review? |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |



## Exam Scoring on the Real Exam

When you take a practice exam with PTP, PTP gives you a score, on a scale from 300 to 1000. Why? Cisco gives a score of between 300 and 1000 as well. But the similarities end there.

With PTP, the score is a basic percentage but expressed as a number from 0 to 1000. For example, answer 80 percent correct, and the score is 800; get 90 percent correct, and the score is 900. If you start a practice exam and click through it without answering a single question, you get a 0.

However, Cisco does not score exams in the same way. The following is what we do know about Cisco exam scoring:

- Cisco uses a scoring scale from 300 to 1000.
- Cisco tells us that it gives partial credit but provides no further details.

So, what does an 800 or a 900 mean on the actual Cisco exams? Cisco doesn't reveal the details of scoring to us. They don't reveal the details of partial credit. It also seems like a lab question would be worth more points than a multiple-choice question. Also, some drag-and-drop questions require that you know many facts—but we do not know if Cisco weights those with more points.

## Self-Assessment Suggestions

Are you ready to take and pass the real exam? We all want to be able to self-assess whether we are ready to pass. The next few pages give a few suggestions.

First, practice exam scores give you only a general indicator of exam readiness. In fact, some students use their practice exam scores as the main self-assessment tool. However, the scores give only a broad indication of your readiness given many variables that affect the scores.

As an example of a good use of the practice exam scores, imagine you hold one practice exam event and get a 650, and three weeks later (after more studying) hold another and

make a 900. You probably improved your chances of passing. However, do not look at your practice exam score of 900 and think that means you're ready to get more than the 800–825 typically required on the real exam. There are too many other variables impacting the scoring to make that claim.

Why is the raw practice exam score a poor measure?

- One 100-question practice exam cannot come close to asking about the breadth and depth of CCNA topics. So the randomness of chosen questions versus your knowledge impacts your practice exam score.
- If you choose to guess like you would on the real exam, your practice test score becomes skewed by your success or failure rate at guessing, giving false confidence if you guess well and false doubts if you guess poorly.
- Question familiarity. If you have seen some of the questions before, you might get a falsely higher score.

So, use your practice exam scores as a general guide. Do not get upset with lower scores, or too excited about high scores.

In addition to practice exam scores, try some other study and review efforts that can give you a better sense of your readiness. For instance, the book identifies key topics and key terms. Those are the most important topics and terms in the books. You ought to be able to explain a few sentences about most of those topics and terms—so do just that. The following list gives some review activities that also help you assess your exam readiness:

1. **Explain each key topic:** Look at the key topic table at the end of each chapter as a reminder of each key topic. Without looking at the chapter's content, explain out loud to an imaginary colleague about the concept and technology. Note the percentage of key topics you could explain well as a measure of exam readiness (you can find the key topics in the companion website as well).
2. **Confirm mastery of subnetting math:** Review and practice all types of subnetting activities one more time. As for exam readiness, you should be able to do subnetting math confidently with 100 percent accuracy at this point.
3. **Practice config checklists:** The chapters include config checklists for the longer configuration tasks. Use the interactive versions of these checklists on the companion website. You should be able to remember most of the configuration commands for each process from memory. Practice them and note your scores, repeating until you remember all.
4. **Practice config labs:** You should be able to do all the Config Labs from my blog site, or labs of similar challenge level. Perform some of those labs to help internalize the commands and details you study in the configuration checklists. While performing all labs may be too time consuming, focus on practicing weak areas.
5. **Improve recall of show commands:** For chapters with **show** commands, scan the examples and look at the fields highlighted in gray. Without looking at the notes in the book, recall and remind yourself about which values list configuration settings and

which show status information. As for exam readiness, do not expect to recall every command and output fact, but work to improve gaps.

6. **Explain key terms:** Use the key terms listed at the end of each chapter for a review exercise. Look at the list, and speaking out loud, describe each term with no other prompts. Try pretending to teach the topic to a fellow CCNA student. For any terms that make you pause too long, review the term. Note the percentage of key terms you can explain as a measure of exam readiness. (You can find the key terms in the companion website's key terms flashcards app, which can be very useful for this activity.)

## Gap Analysis Using Q&A

Gap analysis refers to any activity that helps you identify differences between what you should know and what you actually know. Gap analysis helps increase your chances of passing more so than most last-stage tools, and you can use the PTP app to do effective gap analysis.



When you use exam mode in the PTP app, you cannot see the correct answers, and the app disables the answer button (which reveals a detailed explanation to the question). However, if you click Save to save your practice exam, the PTP software tracks each practice exam you take, remembering your answer for every question, and whether you got it wrong. You can view the results and move back and forth between seeing the question and seeing the results page.

Figure 26-3 shows a sample Question Review page. To know which ones you answered correctly, examine the "Your Score" column, with a 0 meaning you answered incorrectly.

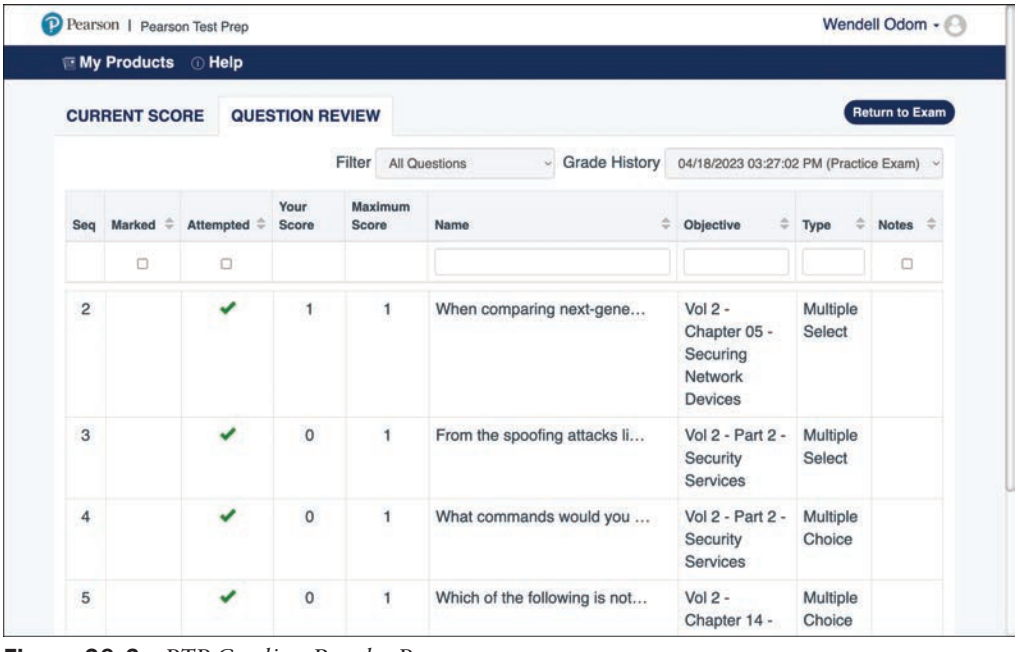



**Figure 26-3** *PTP Grading Results Page*

To perform the process of reviewing questions and marking them as complete, you can move between this Question Review page and the individual questions. Just double-click a question to move back to that question, as shown in Figure 26-4.

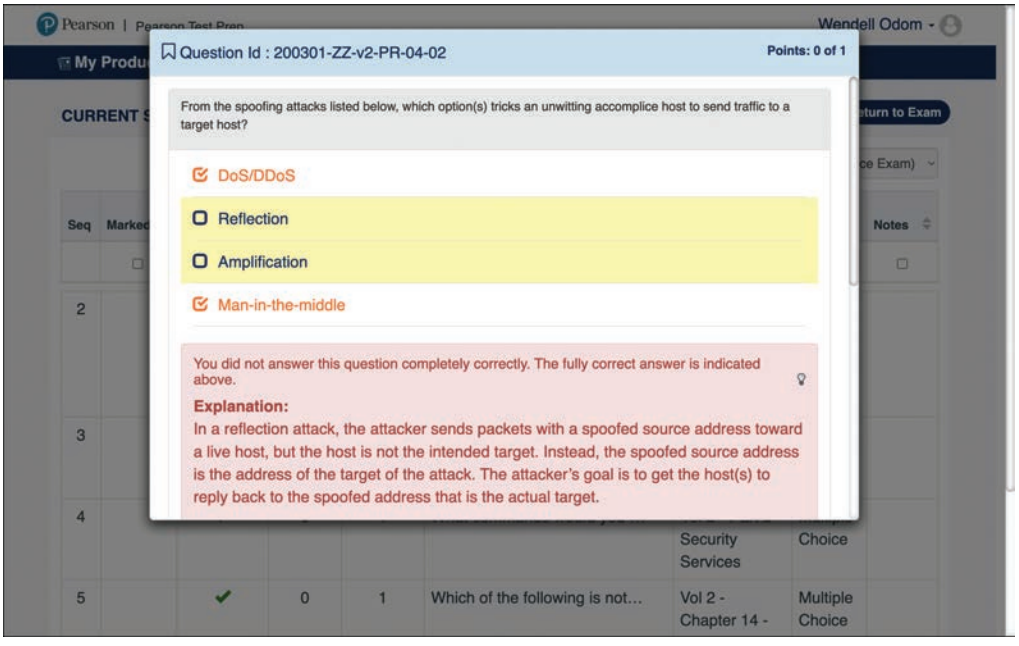



**Figure 26-4** *Reviewing a Question, with the Mark Feature in the Upper Left*

From the question, you can click the Close button to move back to the grading results and to the Question Review page shown in Figure 26-3.

If you want to come back later to look through the questions you missed from an earlier exam, start at the PTP home screen. From there, instead of clicking the Start button to start a new exam, click the **Load Saved Exam** button to see your earlier exam attempts and work through any missed questions.

Find a place to track your cap analysis progress with a table like Table 26-2. PTP lists your saved practice exams by date and time, so it helps to note those values in the table for comparison to the PTP menu.

**Table 26-2** Tracking Checklist for Gap Review of Practice Exams

| Original Practice Exam Date/Time | Date Gap Review Was Completed |
|---|---|
| | |
| | |
| | |
| | |
| | |
| | |

## Advice on How to Answer Exam Questions

Our everyday habits have changed how we all read and think in front of a screen. Unfortunately, those same habits often hurt our scores when taking computer-based exams.

For example, open a web browser. Yes, take a break and open a web browser on any device. Do a quick search on a fun topic. Then, before you click a link, get ready to think about what you just did. Where did your eyes go for the first 5–10 seconds after you opened that web page. Now, click a link and look at the page. Where did your eyes go?

Interestingly, web browsers and the content in web pages have trained us all to scan. Web page designers actually design content expecting certain scan patterns from viewers. Regardless of the pattern, when reading a web page, almost no one reads sequentially, and no one reads entire sentences. People scan for the interesting graphics and the big words, and then scan the space around those noticeable items.

Other parts of our electronic culture have also changed how the average person reads. For example, many of you grew up using texting and social media, sifting through hundreds or thousands of messages—but each message barely fills an entire sentence. Also, we find ourselves responding to texts, tweets, and emails and later realizing we did not really understand what the other person meant.



If you use those same habits when taking the exam, you will probably make some mistakes because you missed a key fact in the question, answer, or exhibits. It helps to start at the beginning and read all the words—a process that is amazingly unnatural for many people today.

**NOTE** I have talked to many college professors, in multiple disciplines, and Cisco Networking Academy instructors, and they consistently tell me that the number-one test-taking issue today is that people do not read the question well enough to understand the details.

When you are taking the practice exams and answering individual questions, consider these two strategies. First, before the practice exam, think about your personal strategy for how you will read a question. Make your approach to multiple-choice questions in particular be a conscious decision on your part. Second, if you want some suggestions on how to read an exam question, use the following strategy:

**Step 1.** Read the question itself, thoroughly, from start to finish.

**Step 2.** Scan any exhibit or figure.

**Step 3.** Scan the answers to look for the types of information. (Numeric? Terms? Single words? Phrases?)

**Step 4.** Reread the question thoroughly, from start to finish, to make sure that you understand it.

**Step 5.** Read each answer thoroughly, while referring to the figure/exhibit as needed. After reading each answer, before reading the next answer:

- **a.** If correct, select as correct.
- **b.** If for sure incorrect, mentally rule it out.
- **c.** If unsure, mentally note it as a possible correct answer.

**NOTE** Cisco exams will tell you the number of correct answers. The exam software also helps you finish the question with the right number of answers noted. For example, for standalone multichoice questions, the software prevents you from selecting too many or too few answers. And you should guess the answer when unsure on the actual exam—there is no penalty for guessing.

Use the practice exams as a place to practice your approach to reading. Every time you click to the next question, try to read the question following your approach. If you are feeling time pressure, that is the perfect time to keep practicing your approach, to reduce and eliminate questions you miss because of scanning the question instead of reading thoroughly.

### Additional Exams with the Premium Edition

Many people add practice exams and questions other than those that come with this book. Frankly, using other practice exams in addition to the questions that come with this book can be a good idea, for many reasons. The other exam questions can use different terms in different ways, emphasize different topics, and show different scenarios that make you rethink some topics.

All Cisco Press CCNA Premium Edition products include:

- Kindle, Nook, and PDF versions of the book
- More PTP exams
- Links from each question to the related content in the book

Cisco Press publishes a Volume 1, Volume 2, and CCNA Library Premium Edition product. For practice questions, the CCNA Library Premium Edition is simply better. The Volume 1 and 2 products contain exams that cover only the respective book volume, whereas the CCNA Library Premium Edition has CCNA full exams, which contain exams from the full breadth of CCNA topics. (Plus, the CCNA Library Premium Edition is less expensive than buying both the Volume 1 and 2 Premium Edition products.)

**NOTE** In addition to the extra questions, the Premium Editions have links to every test question, including those in the print book, to the specific section of the book for further reference. This is a great learning tool if you need more detail than what you find in the question explanations.

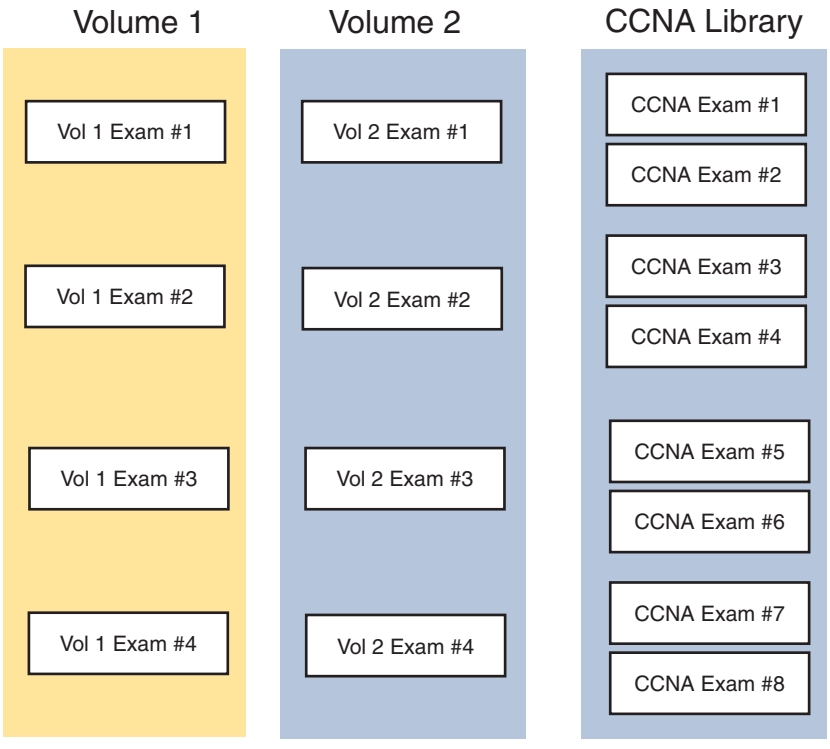



**Figure 26-5** *CCNA Full Exams in the CCNA Library Premium Edition*

## Practicing CLI Skills

To do well on lab questions, you need to be comfortable with many Cisco router and switch commands and how to use them from a Cisco CLI. As described in the Introduction to this book, lab questions require you to decide what configuration command(s) need to be configured to fix a problem or to complete a working configuration.

To be ready for the exam, you need to know the following kinds of information:

**CLI navigation:** Basic CLI mechanics of moving into and out of user, enable, and configuration modes

**Individual configuration:** The meaning of the parameters of each configuration command

**Feature configuration:** The set of configuration commands, both required and optional, for each feature

**Verification of configuration:** The **show** commands that directly identify the configuration settings

**Verification of status:** The **show** commands that list current status values, and the ability to decide incorrect configuration or other problem causes of less-than-optimal status values

To help remember and review all this knowledge and skill, you can do the tasks listed in the next several pages.

Wondering about all the topics in CCNA 200-301 that specifically include configuration or verification skills? You can just scan the CCNA 200-301 exam blueprint Version 1.1. However, Table 26-3 and Table 26-4 summarize the topics for which you could consider practicing your CLI skills. The tables organize the topics into the same order used in the *CCNA 200-301 Official Cert Guides, Volumes 1 and 2*, with chapter references.

**Table 26-3** Topics with Configuration Skills in CCNA 200-301 Official Cert Guide, Volume 1

| Topic | Volume 1 Chapter | Date You Finished Lab Review |
|---|---|---|
| Verifying LAN switching | 5 | |
| Switch IPv4 | 6 | |
| Switch passwords | 6 | |
| Switch interfaces | 7 | |
| VLANs | 8 | |
| VLAN trunking | 8 | |
| STP and RSTP | 10 | |
| Layer 2 EtherChannel | 10 | |
| BPDU guard, BPDU filter, root guard, loop guard | 10 | |
| Router interfaces | 16 | |
| Router IPv4 addresses and static routes | 17 | |
| Router on a stick | 18 | |
| Layer 3 switching with SVIs | 18 | |
| Layer 3 switching with routed interfaces and L3 EtherChannels | 18 | |
| Switch interfaces on routers | 18 | |
| DHCP Client and DHCP Relay | 19 | |
| OSPF fundamentals | 22 | |
| OSPF network types and other optional features | 23 | |
| OSPF neighbor issues | 24 | |
| IPv6 addressing on routers | 27 | |
| IPv6 host addressing | 28 | |
| IPv6 static routes | 29 | |

**Table 26-4** Topics with Configuration Skills in CCNA 200-301 Official Cert Guide, Volume 2

| Topic | Volume 2 Chapter | Date You Finished Lab Review |
|---|---|---|
| Wireless LANs | 4 | |
| Standard ACLs | 6 | |
| Extended ACLs | 7 | |
| Filtering network protocols | 8 | |
| Port security | 11 | |
| DHCP Snooping | 12 | |
| Dynamic ARP Inspection | 12 | |
| Syslog, NTP, CDP, and LLDP | 13 | |
| NAT, PAT | 14 | |



You should research and choose your favorite methods and tools to get hands-on practice for CCNA. Those options include several that focus on giving you a specific activity to do. The options include the Pearson Network Simulator, Config Labs (on my blog), and Packet Tracer labs (on my blog). Look back to the Introduction to this book for more detail on the tools, and use these links for more information:

The Pearson Network Simulator (the Sim): https://www.pearsonitcertification.com/networksimulator

Config Labs at my blog: www.certskills.com/config-labs

## Adjustments for Your Second Attempt

None of us wants to take and fail any exam, but some of you will. And even if you pass the CCNA exam on your first try, if you keep going with Cisco certifications, you will probably fail some exams along the way. I mention failing an exam not to focus on the negative, but to help prepare you for how to pass the next attempt after failing an earlier attempt. This section collects some of the advice I have given to readers over the years who have contacted me after a failed attempt, asking for help about what to do next.

The single most important bit of advice is to change your mindset about Cisco exams. Cisco exams are not like high school or college exams where your failing grade matters. Instead, a Cisco exam is more like an event on the road to completing an impressive major accomplishment, one that most people have to try a few times to achieve.

For instance, achieving a Cisco certification is more like training to run a marathon in under 4 hours. The first time running a marathon, you may not even finish, or you may finish at 4:15 rather than under 4:00. But finishing a marathon in 4:15 means that you have prepared and are getting pretty close to your goal.

So change your mindset. You're a marathon runner looking to improve your time. And you are getting better skills every time you study, which helps you compete in the market.

With that attitude and analogy in mind, the rest of this section lists specific study steps that can help.

First, study the notes you took about your failed attempt. (See the earlier section "The Hour After Your Exam.") Do not share that information with others, but use it to study. Before you take the exam again, you should be able to answer every actual exam question you can remember from the last attempt. Even if you never see the exact same question again, you will still get a good return for your effort.

Second, spend more time on activities that uncover your weaknesses. Do more gap analysis. When doing that, you have to slow down and be more self-aware. For instance, answer practice questions in study mode, and *do not guess.* Do not click on to the next question, but pause and ask yourself if you are really sure about both the wrong and correct answers. If unsure, fantastic! You just discovered a topic for which to go back and dig in to learn it more deeply. Or when you do a lab, hide all notes and do not use tools like Internet searches—that might be a reminder that you have not mastered those commands yet.

Third, think about your time spent on the exam. Did you run out of time? Go too fast? Too slow? If too slow, were you slow on subnetting, or lab questions, or something else? Then make a written plan as to how you will approach time on the next attempt and how you will track time use. And if you ran out of time, practice for the things that slowed you down.

## Other Study Tasks

If you got to this point and still feel the need to prepare some more, this last topic gives you three suggestions.

First, the Chapter Review and Part Review sections give you some useful study tasks.

Second, use more exam questions from other sources. You can always get more questions in the Cisco Press Premium Edition eBook and Practice Test products, which include an eBook copy of this book plus additional questions in additional PTP exam banks. However, you can search the Internet for questions from many sources and review those questions as well.

**NOTE** Some vendors claim to sell practice exams that contain the literal exam questions from the official exam. These exams, called "brain dumps," are against the Cisco testing policies. Cisco strongly discourages using any such tools for study.

Finally, join in the discussions on the Cisco Learning Network. Try to answer questions asked by other learners; the process of answering makes you think much harder about the topic. When someone posts an answer with which you disagree, think about why and talk about it online. This is a great way to both learn more and build confidence.

## Final Thoughts

You have studied quite a bit, worked hard, and sacrificed time and money to be ready for the exam. I hope your exam goes well, that you pass, and that you pass because you really know your stuff and will do well in your IT and networking career.

I encourage you to celebrate when you pass and ask advice when you do not. The Cisco Learning Network is a great place to make posts to celebrate and to ask advice for the next time around. I personally would love to hear about your progress through LinkedIn (https://www.linkedin.com/in/WendellOdom). I wish you well, and congratulations for working through the entire book!

*This page intentionally left blank*

*This page intentionally left blank*

# Part VIII

## Print Appendixes

**Appendix A:** Numeric Reference Tables

**Appendix B:** Exam Topics Cross-Reference

**Appendix C:** Answers to the “Do I Know This Already?” Quizzes

**Glossary**

*This page intentionally left blank*

# APPENDIX A

# Numeric Reference Tables

This appendix provides several useful reference tables that list numbers used throughout this book. Specifically:

Table A-1: A decimal-binary cross reference, useful when converting from decimal to binary and vice versa.

**Table A-1** Decimal-Binary Cross Reference, Decimal Values 0–255

| Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value |
|---|---|---|---|---|---|---|---|
| 0 | 00000000 | 32 | 00100000 | 64 | 01000000 | 96 | 01100000 |
| 1 | 00000001 | 33 | 00100001 | 65 | 01000001 | 97 | 01100001 |
| 2 | 00000010 | 34 | 00100010 | 66 | 01000010 | 98 | 01100010 |
| 3 | 00000011 | 35 | 00100011 | 67 | 01000011 | 99 | 01100011 |
| 4 | 00000100 | 36 | 00100100 | 68 | 01000100 | 100 | 01100100 |
| 5 | 00000101 | 37 | 00100101 | 69 | 01000101 | 101 | 01100101 |
| 6 | 00000110 | 38 | 00100110 | 70 | 01000110 | 102 | 01100110 |
| 7 | 00000111 | 39 | 00100111 | 71 | 01000111 | 103 | 01100111 |
| 8 | 00001000 | 40 | 00101000 | 72 | 01001000 | 104 | 01101000 |
| 9 | 00001001 | 41 | 00101001 | 73 | 01001001 | 105 | 01101001 |
| 10 | 00001010 | 42 | 00101010 | 74 | 01001010 | 106 | 01101010 |
| 11 | 00001011 | 43 | 00101011 | 75 | 01001011 | 107 | 01101011 |
| 12 | 00001100 | 44 | 00101100 | 76 | 01001100 | 108 | 01101100 |
| 13 | 00001101 | 45 | 00101101 | 77 | 01001101 | 109 | 01101101 |
| 14 | 00001110 | 46 | 00101110 | 78 | 01001110 | 110 | 01101110 |
| 15 | 00001111 | 47 | 00101111 | 79 | 01001111 | 111 | 01101111 |
| 16 | 00010000 | 48 | 00110000 | 80 | 01010000 | 112 | 01110000 |
| 17 | 00010001 | 49 | 00110001 | 81 | 01010001 | 113 | 01110001 |
| 18 | 00010010 | 50 | 00110010 | 82 | 01010010 | 114 | 01110010 |
| 19 | 00010011 | 51 | 00110011 | 83 | 01010011 | 115 | 01110011 |
| 20 | 00010100 | 52 | 00110100 | 84 | 01010100 | 116 | 01110100 |
| 21 | 00010101 | 53 | 00110101 | 85 | 01010101 | 117 | 01110101 |
| 22 | 00010110 | 54 | 00110110 | 86 | 01010110 | 118 | 01110110 |
| 23 | 00010111 | 55 | 00110111 | 87 | 01010111 | 119 | 01110111 |
| 24 | 00011000 | 56 | 00111000 | 88 | 01011000 | 120 | 01111000 |
| 25 | 00011001 | 57 | 00111001 | 89 | 01011001 | 121 | 01111001 |
| 26 | 00011010 | 58 | 00111010 | 90 | 01011010 | 122 | 01111010 |
| 27 | 00011011 | 59 | 00111011 | 91 | 01011011 | 123 | 01111011 |
| 28 | 00011100 | 60 | 00111100 | 92 | 01011100 | 124 | 01111100 |
| 29 | 00011101 | 61 | 00111101 | 93 | 01011101 | 125 | 01111101 |
| 30 | 00011110 | 62 | 00111110 | 94 | 01011110 | 126 | 01111110 |
| 31 | 00011111 | 63 | 00111111 | 95 | 01011111 | 127 | 01111111 |

| Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value | Decimal Value | Binary Value |
|---|---|---|---|---|---|---|---|
| 128 | 10000000 | 160 | 10100000 | 192 | 11000000 | 224 | 11100000 |
| 129 | 10000001 | 161 | 10100001 | 193 | 11000001 | 225 | 11100001 |
| 130 | 10000010 | 162 | 10100010 | 194 | 11000010 | 226 | 11100010 |
| 131 | 10000011 | 163 | 10100011 | 195 | 11000011 | 227 | 11100011 |
| 132 | 10000100 | 164 | 10100100 | 196 | 11000100 | 228 | 11100100 |
| 133 | 10000101 | 165 | 10100101 | 197 | 11000101 | 229 | 11100101 |
| 134 | 10000110 | 166 | 10100110 | 198 | 11000110 | 230 | 11100110 |
| 135 | 10000111 | 167 | 10100111 | 199 | 11000111 | 231 | 11100111 |
| 136 | 10001000 | 168 | 10101000 | 200 | 11001000 | 232 | 11101000 |
| 137 | 10001001 | 169 | 10101001 | 201 | 11001001 | 233 | 11101001 |
| 138 | 10001010 | 170 | 10101010 | 202 | 11001010 | 234 | 11101010 |
| 139 | 10001011 | 171 | 10101011 | 203 | 11001011 | 235 | 11101011 |
| 140 | 10001100 | 172 | 10101100 | 204 | 11001100 | 236 | 11101100 |
| 141 | 10001101 | 173 | 10101101 | 205 | 11001101 | 237 | 11101101 |
| 142 | 10001110 | 174 | 10101110 | 206 | 11001110 | 238 | 11101110 |
| 143 | 10001111 | 175 | 10101111 | 207 | 11001111 | 239 | 11101111 |
| 144 | 10010000 | 176 | 10110000 | 208 | 11010000 | 240 | 11110000 |
| 145 | 10010001 | 177 | 10110001 | 209 | 11010001 | 241 | 11110001 |
| 146 | 10010010 | 178 | 10110010 | 210 | 11010010 | 242 | 11110010 |
| 147 | 10010011 | 179 | 10110011 | 211 | 11010011 | 243 | 11110011 |
| 148 | 10010100 | 180 | 10110100 | 212 | 11010100 | 244 | 11110100 |
| 149 | 10010101 | 181 | 10110101 | 213 | 11010101 | 245 | 11110101 |
| 150 | 10010110 | 182 | 10110110 | 214 | 11010110 | 246 | 11110110 |
| 151 | 10010111 | 183 | 10110111 | 215 | 11010111 | 247 | 11110111 |
| 152 | 10011000 | 184 | 10111000 | 216 | 11011000 | 248 | 11111000 |
| 153 | 10011001 | 185 | 10111001 | 217 | 11011001 | 249 | 11111001 |
| 154 | 10011010 | 186 | 10111010 | 218 | 11011010 | 250 | 11111010 |
| 155 | 10011011 | 187 | 10111011 | 219 | 11011011 | 251 | 11111011 |
| 156 | 10011100 | 188 | 10111100 | 220 | 11011100 | 252 | 11111100 |
| 157 | 10011101 | 189 | 10111101 | 221 | 11011101 | 253 | 11111101 |
| 158 | 10011110 | 190 | 10111110 | 222 | 11011110 | 254 | 11111110 |
| 159 | 10011111 | 191 | 10111111 | 223 | 11011111 | 255 | 11111111 |

Table A-2: A hexadecimal-binary cross reference, useful when converting from hex to binary and vice versa.

**Table A-2** Hex-Binary Cross Reference

| Hex | 4-Bit Binary |
|---|---|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0010 |
| 3 | 0011 |
| 4 | 0100 |
| 5 | 0101 |
| 6 | 0110 |
| 7 | 0111 |
| 8 | 1000 |
| 9 | 1001 |
| A | 1010 |
| B | 1011 |
| C | 1100 |
| D | 1101 |
| E | 1110 |
| F | 1111 |

Table A-3: Powers of 2, from $2^1$ through $2^{32}$.

**Table A-3** Powers of 2

| X | $2^X$ | X | $2^X$ |
|---|---|---|---|
| 1 | 2 | 17 | 131,072 |
| 2 | 4 | 18 | 262,144 |
| 3 | 8 | 19 | 524,288 |
| 4 | 16 | 20 | 1,048,576 |
| 5 | 32 | 21 | 2,097,152 |
| 6 | 64 | 22 | 4,194,304 |
| 7 | 128 | 23 | 8,388,608 |
| 8 | 256 | 24 | 16,777,216 |
| 9 | 512 | 25 | 33,554,432 |
| 10 | 1024 | 26 | 67,108,864 |
| 11 | 2048 | 27 | 134,217,728 |
| 12 | 4096 | 28 | 268,435,456 |
| 13 | 8192 | 29 | 536,870,912 |
| 14 | 16,384 | 30 | 1,073,741,824 |
| 15 | 32,768 | 31 | 2,147,483,648 |
| 16 | 65,536 | 32 | 4,294,967,296 |

Table A-4: Table of all 33 possible subnet masks, in all three formats.

**Table A-4** All Subnet Masks

| Decimal | Prefix | Binary |
|---|---|---|
| 0.0.0.0 | /0 | 00000000 00000000 00000000 00000000 |
| 128.0.0.0 | /1 | 10000000 00000000 00000000 00000000 |
| 192.0.0.0 | /2 | 11000000 00000000 00000000 00000000 |
| 224.0.0.0 | /3 | 11100000 00000000 00000000 00000000 |
| 240.0.0.0 | /4 | 11110000 00000000 00000000 00000000 |
| 248.0.0.0 | /5 | 11111000 00000000 00000000 00000000 |
| 252.0.0.0 | /6 | 11111100 00000000 00000000 00000000 |
| 254.0.0.0 | /7 | 11111110 00000000 00000000 00000000 |
| 255.0.0.0 | /8 | 11111111 00000000 00000000 00000000 |
| 255.128.0.0 | /9 | 11111111 10000000 00000000 00000000 |
| 255.192.0.0 | /10 | 11111111 11000000 00000000 00000000 |
| 255.224.0.0 | /11 | 11111111 11100000 00000000 00000000 |
| 255.240.0.0 | /12 | 11111111 11110000 00000000 00000000 |
| 255.248.0.0 | /13 | 11111111 11111000 00000000 00000000 |
| 255.252.0.0 | /14 | 11111111 11111100 00000000 00000000 |
| 255.254.0.0 | /15 | 11111111 11111110 00000000 00000000 |
| 255.255.0.0 | /16 | 11111111 11111111 00000000 00000000 |
| 255.255.128.0 | /17 | 11111111 11111111 10000000 00000000 |
| 255.255.192.0 | /18 | 11111111 11111111 11000000 00000000 |
| 255.255.224.0 | /19 | 11111111 11111111 11100000 00000000 |
| 255.255.240.0 | /20 | 11111111 11111111 11110000 00000000 |
| 255.255.248.0 | /21 | 11111111 11111111 11111000 00000000 |
| 255.255.252.0 | /22 | 11111111 11111111 11111100 00000000 |
| 255.255.254.0 | /23 | 11111111 11111111 11111110 00000000 |
| 255.255.255.0 | /24 | 11111111 11111111 11111111 00000000 |
| 255.255.255.128 | /25 | 11111111 11111111 11111111 10000000 |
| 255.255.255.192 | /26 | 11111111 11111111 11111111 11000000 |
| 255.255.255.224 | /27 | 11111111 11111111 11111111 11100000 |
| 255.255.255.240 | /28 | 11111111 11111111 11111111 11110000 |
| 255.255.255.248 | /29 | 11111111 11111111 11111111 11111000 |
| 255.255.255.252 | /30 | 11111111 11111111 11111111 11111100 |
| 255.255.255.254 | /31 | 11111111 11111111 11111111 11111110 |
| 255.255.255.255 | /32 | 11111111 11111111 11111111 11111111 |

# Exam Topics Cross-Reference

This appendix lists the exam topics defined in the CCNA 200-301 exam blueprint version 1.1. Cisco lists the exam topics on its website. Even though changes to the exam topics are rare, you should always review those exam topics for any updates; check www.cisco.com/go/certifications and navigate to the correct exam.

Cisco organizes each list of exam topics by domains, which are major topic areas. Cisco states the percentage of the exam that should come from each exam, so you get some idea of the areas of importance. Traditionally, the score report you receive after taking the exam shows your percentage score in each domain.

This appendix includes two separate types of indices to exam topics:

- **CCNA 200-301 Version 1.1 Blueprint Order:** This section uses the same order as the CCNA 200-301 V1.1 exam blueprint document. This first list shows a cross-reference from each exam topic to the chapters that include at least some material about each topic.
- **Book Chapter Order:** This section lists the chapters in this book, along with the exam topics that the chapter includes. This section basically relists the kind of information found on the first page of each chapter, just in condensed form in one place.

## CCNA 200-301 Exam Topic Order

The CCNA 200-301 exam includes six major topic areas (domains), each with a percentage listed. Table B-1 lists the domains and their percentages.

**Table B-1** CCNA 200-301 Version 1.1 Exam Topic Domains

| Domain | Percentage |
|---|---|
| Domain 1: Network Fundamentals | 20% |
| Domain 2: Network Access | 20% |
| Domain 3: IP Connectivity | 25% |
| Domain 4: IP Services | 10% |
| Domain 5: Security Fundamentals | 15% |
| Domain 6: Automation and Programmability | 10% |

Tables B-2 through B-7 list the exam topics within each of the six domains. Note that the *CCNA 200-301 Official Cert Guide, Volume 1,* Second Edition, covers some of the exam topics, while this book covers the rest. These tables show the chapters in this book; look to the equivalent appendix in Volume 1 for details of exam topic coverage in that book.

**Table B-2** CCNA 200-301 Version 1.1 Domain 1 (Network Fundamentals)

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| **1.1 Explain the role and function of network components** | 2, 3, 5, 7 | 1, 10, 18, 21, 22 |
| *1.1.a Routers* | 3, 16 | |
| *1.1.b Layer 2 and Layer 3 Switches* | 2, 5, 7, 18 | |
| *1.1.c Next-generation firewalls and IPS* | | 10 |
| *1.1.d Access points* | | 1 |
| *1.1.e Controllers* | | 4, 22 |
| *1.1.f Endpoints* | | 21 |
| *1.1.g Servers* | | 21 |
| *1.1.h PoE* | | 18 |
| **1.2 Describe characteristics of network topology architectures** | 2, 3 | 18–21 |
| *1.2.a Two-tier* | | 18 |
| *1.2.b Three-tier* | | 18 |
| *1.2.c Spine-leaf* | | 21 |
| *1.2.d WAN* | 3 | 19 |
| *1.2.e Small office/home office (SOHO)* | 2, 16 | 18 |
| *1.2.f On-premises and cloud* | | 20 |
| **1.3 Compare physical interface and cabling types** | 1, 2, 7 | 18 |
| *1.3.a Single-mode fiber, multimode fiber, copper* | 1, 2 | 18 |
| *1.3.b Connections (Ethernet shared media and point-to-point)* | 1, 2, 7 | 18 |
| **1.4 Identify interface and cable issues (collisions, errors, mismatch duplex, and/or speed)** | 7 | |
| **1.5 Compare TCP to UDP** | | 5 |
| **1.6 Configure and verify IPv4 addressing and subnetting** | 6, 11–16, 18 | |
| **1.7 Describe private IPv4 addressing** | 11, 12, 17 | 14 |
| **1.8 Configure and verify IPv6 addressing and prefix** | 25–28 | |
| **1.9 Describe IPv6 address types** | 25–28 | |
| *1.9.a Unicast (global, unique local, and link local)* | 26–28 | |
| *1.9.b Anycast* | 26, 27 | |
| *1.9.c Multicast* | 27 | |
| *1.9.d Modified EUI 64* | 27, 28 | |
| **1.10 Verify IP parameters for Client OS (Windows, Mac OS, Linux)** | 19 | |

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| **1.11 Describe wireless principles** | | 1, 3 |
| *1.11.a Nonoverlapping Wi-Fi channels* | | 1 |
| *1.11.b SSID* | | 1 |
| *111.c RF* | | 1 |
| *1.11.d Encryption* | | 3 |
| **1.12 Explain virtualization fundamentals (server virtualization, containers, and VRFs)** | | 20 |
| **1.13 Describe switching concepts** | 5, 8 | |
| *1.13.a MAC learning and aging* | 5, 8 | |
| *1.13.b Frame switching* | 5, 8 | |
| *1.13.c Frame flooding* | 5, 8 | |
| *1.13.d MAC address table* | 5, 8 | |

**Table B-3** CCNA 200-301 Version 1.1 Domain 2 (Network Access)

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| **2.1 Configure and verify VLANs (normal range) spanning multiple switches** | 8, 18 | |
| *2.1.a Access ports (data and voice)* | 8 | |
| *2.1.b Default VLAN* | 8 | |
| *2.1.c InterVLAN connectivity* | 8, 18 | |
| **2.2 Configure and verify interswitch connectivity** | 8 | |
| *2.2.a Trunk ports* | 8 | |
| *2.2.b 802.1Q* | 8 | |
| *2.2.c Native VLAN* | 8 | |
| **2.3 Configure and verify Layer 2 discovery protocols (Cisco Discovery Protocol and LLDP)** | | 13 |
| **2.4 Configure and verify (Layer 2/Layer 3) EtherChannel (LACP)** | 8–10, 17 | |
| **2.5 Interpret basic operations of Spanning Tree Protocols** | 5, 9, 10 | |
| *2.5.a Root port, root bridge (primary/secondary), and other port names* | 9, 10 | |
| *2.5.b Port states and port roles* | 9, 10 | |
| *2.5.c PortFast* | 9, 10 | |
| *2.5.d Root Guard, loop guard, BPDU filter, BPDU guard* | 9, 10 | |
| **2.6 Describe Cisco Wireless Architectures and AP modes** | | 2 |
| **2.7 Describe physical infrastructure connections of WLAN components (AP, WLC, access/trunk ports, and LAG)** | | 4 |

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, and TACACS+/RADIUS, and cloud managed) | 4, 6, 20 | 4 |
| 2.9 Interpret the wireless LAN GUI configuration for client connectivity, such as WLAN creation, security settings, QoS profiles, and advanced settings | | 4 |

**Table B-4** CCNA 200-301 Version 1.1 Domain 3 (IP Connectivity)

| Exam Topic | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 3.1 Interpret the components of routing table | 17, 29 | |
| *3.1.a Routing protocol code* | 17, 29 | |
| *3.1.b Prefix* | 17, 29 | |
| *3.1.c Network mask* | 17, 29 | |
| *3.1.d Next hop* | 17, 29 | |
| *3.1.e Administrative distance* | 17, 24, 29 | |
| *3.1.f Metric* | 17 | |
| *3.1.g Gateway of last resort* | 17 | |
| 3.2 Determine how a router makes a forwarding decision by default | 17, 21–24 | |
| *3.2.a Longest prefix match* | 17, 24 | |
| *3.2.b Administrative distance* | 17, 21–24 | |
| *3.2.c Routing protocol metric* | 21–24 | |
| 3.3 Configure and verify IPv4 and IPv6 static routing | 17, 20, 29 | |
| *3.3.a Default route* | 17, 20, 29 | |
| *3.3.b Network route* | 17, 20, 29 | |
| *3.3.c Host route* | 17, 20, 29 | |
| *3.3.d Floating static* | 17, 20, 29 | |
| 3.4 Configure and verify single area OSPFv2 | 21–24 | |
| *3.4.a Neighbor adjacencies* | 21–24 | |
| *3.4.b Point-to-point* | 21–24 | |
| *3.4.c Broadcast (DR/BDR selection)* | 21–24 | |
| *3.4.d Router ID* | 21–24 | |
| 3.5 Describe the purpose, functions, and concepts of first hop redundancy protocols | | 16 |

**Table B-5** CCNA 200-301 Version 1.1 Domain 4 (IP Services)

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 4.1 Configure and verify inside source NAT using static and pools | | 14 |
| 4.2 Configure and verify NTP operating in a client and server mode | | 13 |
| 4.3 Explain the role of DHCP and DNS within the network | 19 | 5 |
| 4.4 Explain the function of SNMP in network operations | | 17 |
| 4.5 Describe the use of syslog features including facilities and severity levels | | 13 |
| 4.6 Configure and verify DHCP client and relay | 6, 19 | |
| 4.7 Explain the forwarding per-hop behavior (PHB) for QoS such as classification, marking, queuing, congestion, policing, and shaping | | 15 |
| 4.8 Configure network devices for remote access using SSH | 6 | 10 |
| 4.9 Describe the capabilities and functions of TFTP/FTP in the network | | 17 |

**Table B-6** CCNA 200-301 Domain 5 Exam Topics (Security Fundamentals)

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| 5.1 Define key security concepts (threats, vulnerabilities, exploits, and mitigation techniques) | | 9 |
| 5.2 Describe security program elements (user awareness, training, and physical access control) | | 9 |
| 5.3 Configure and verify device access control using local passwords | 6 | 10 |
| 5.4 Describe security password policies elements, such as management, complexity, and password alternatives (multifactor authentication, certificates, and biometrics) | | 9 |
| 5.5 Describe IPsec remote access and site-to-site VPNs | | 19 |
| 5.6 Configure and verify access control lists | | 6, 7, 8 |
| 5.7 Configure and verify Layer 2 security features (DHCP snooping, dynamic ARP inspection, and port security) | | 11, 12 |
| 5.8 Compare authentication, authorization, and accounting concepts | | 9 |
| 5.9 Describe wireless security protocols (WPA, WPA2, and WPA3) | | 3 |
| 5.10 Configure and verify WLAN within the GUI using WPA2 PSK | | 4 |

**Table B-7** CCNA 200-301 Version 1.1 Domain 6 (Programmability and Automation)

| Exam Topics | Vol 1 Chapter(s) | Vol 2 Chapter(s) |
|---|---|---|
| **6.1 Explain how automation impacts network management** | | 21, 22 |
| **6.2 Compare traditional networks with controller-based networking** | | 21, 22 |
| **6.3 Describe controller-based, software-defined architecture (overlay, underlay, and fabric)** | | 21, 22 |
| *6.3.a Separation of control plane and data plane* | | 21, 22 |
| *6.3.b Northbound and Southbound APIs* | | 21, 22 |
| **6.4 Explain AI (generative and predictive) and machine learning in network operations** | | 22 |
| **6.5 Describe characteristics of REST-based APIs (authentication types, CRUD, HTTP verbs, and data encoding)** | | 23 |
| **6.6 Recognize the capabilities of configuration management mechanisms such as Ansible and Terraform** | | 24 |
| **6.7 Recognize components of JSON-encoded data** | | 23 |

## Book Chapters, with Exam Topics Covered in Each

Cisco organizes its exam topics based on the outcome of your learning experience, which is typically not a reasonable order for building the content of a book or course. This section lists this book's chapters in sequence, with the exam topics covered in each chapter.

**Table B-8** CCNA 200-301 Volume 2 V1.1: Chapter-to-Exam Topic Mapping

| Book Chapter | Exam Topics Covered |
|---|---|
| **Part I: Wireless LANs** | |
| Chapter 1: Fundamentals of Wireless Networks | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.d Access points*<br>1.11 Describe wireless principles<br>*1.11.a Non-overlapping Wi-Fi channels*<br>*1.11.b SSID*<br>*1.11.c RF* |
| Chapter 2: Analyzing Cisco Wireless Architectures | **2.0 Network Access**<br>2.6 Describe Cisco Wireless Architectures and AP modes |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 3: Securing Wireless Networks | **1.0 Network Fundamentals**<br>1.11 Describe wireless principles<br>*1.11.d Encryption*<br>**5.0 Security Fundamentals**<br>5.9 Describe wireless security protocols (WPA, WPA2, and WPA3) |
| Chapter 4: Building a Wireless LAN | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.e Controllers (Cisco DNA Center and WLC)*<br>**2.0 Network Access**<br>2.7 Describe physical infrastructure connections of WLAN components (AP, WLC, access/trunk ports, and LAG)<br>2.8 Describe network device management access (Telnet, SSH, HTTP, HTTPS, console, and TACACS+/RADIUS, and cloud managed)<br>2.9 Interpret the wireless LAN GUI configuration for client connectivity, such as WLAN creation, security settings, QoS profiles, and advanced WLAN settings<br>**5.0 Security Fundamentals**<br>5.10 Configure and verify WLAN within the GUI using WPA2 PSK |
| **Part II: IP Access Control Lists** | |
| Chapter 5: Introduction to TCP/IP Transport and Applications | **1.0 Network Fundamentals**<br>1.5 Compare TCP to UDP<br>**4.0 IP Services**<br>4.3 Explain the role of DHCP and DNS in the network |
| Chapter 6: Basic IPv4 Access Control Lists | **5.0 Security Fundamentals**<br>5.6 Configure and verify access control lists |
| Chapter 7: Named and Extended IP ACLs | **5.0 Security Fundamentals**<br>5.6 Configure and verify access control lists |
| Chapter 8: Applied IP ACLs | **5.0 Security Fundamentals**<br>5.6 Configure and verify access control lists |

| Book Chapter | Exam Topics Covered |
|---|---|
| **Part III: Security Services** | |
| Chapter 9: Security Architectures | **5.0 Security Fundamentals**<br>5.1 Define key security concepts (threats, vulnerabilities, exploits, and mitigation techniques)<br>5.2 Describe security program elements (user awareness, training, and physical access control)<br>5.4 Describe security password policies elements, such as management, complexity, and password alternatives (multifactor authentication, certificates, and biometrics)<br>5.8 Compare authentication, authorization, and accounting concepts |
| Chapter 10: Securing Network Devices | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.c Next-generation firewalls and IPS*<br>**4.0 IP Services**<br>4.8 Configure network devices for remote access using SSH<br>**5.0 Security Fundamentals**<br>5.3 Configure and verify device access control using local passwords |
| Chapter 11: Implementing Switch Port Security | **5.0 Security Fundamentals**<br>5.7 Configure and verify Layer 2 security features (DHCP snooping, dynamic ARP inspection, and port security) |
| Chapter 12: DHCP Snooping and ARP Inspection | **5.0 Security Fundamentals**<br>5.7 Configure and verify Layer 2 security features (DHCP snooping, dynamic ARP inspection, and port security) |
| **Part IV: IP Services** | |
| Chapter 13: Device Management Protocols | **2.0 Network Access**<br>2.3 Configure and verify Layer 2 discovery protocols (Cisco Discovery Protocol and LLDP)<br>**4.0 IP Services**<br>4.2 Configure and verify NTP operating in a client and server mode<br>4.5 Describe the use of syslog features including facilities and severity levels |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 14: Network Address Translation | **1.0 Network Fundamentals**<br>1.7 Describe the need for private IPv4 addressing<br>**4.0 IP Services**<br>4.1 Configure and verify inside source NAT using static and pools |
| Chapter 15: Quality of Service (QoS) | **4.0 IP Services**<br>4.7 Explain the forwarding per-hop behavior (PHB) for QoS such as classification, marking, queuing, congestion, policing, shaping |
| Chapter 16: First Hop Redundancy Protocols | **3.0 IP Connectivity**<br>3.5 Describe the purpose, functions, and concepts of first hop redundancy protocols |
| Chapter 17: SNMP, FTP, and TFTP | **4.0 Infrastructure Services**<br>4.4 Explain the function of SNMP in network operations<br>4.9 Describe the capabilities and functions of TFTP/FTP in the network |
| **Part V: Network Architecture** | |
| Chapter 18: LAN Architecture | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.b PoE*<br>1.2 Describe characteristics of network topology architectures<br>*1.2.a Two-tier*<br>*1.2.b Three-tier*<br>*1.2.e Small office/home office (SOHO)*<br>1.3 Compare physical interface and cabling types<br>*1.3.a Single-mode fiber, multimode fiber, copper* |
| Chapter 19: WAN Architecture | **1.0 Network Fundamentals**<br>1.2 Describe the characteristics of network topology architectures<br>*1.2.d WAN*<br>**5.0 Security Fundamentals**<br>5.5 Describe IPsec remote access and site-to-site VPNs |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 20: Cloud Architecture | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.g Servers*<br>1.2 Describe the characteristics of network topology architectures<br>*1.2.f On-premises and cloud*<br>1.12 Explain virtualization fundamentals (server virtualization, containers, and VRFs) |
| **Part VI: Network Automation** | |
| Chapter 21: Introduction to Controller-Based Networking | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.f Endpoints*<br>*1.1.g Servers*<br>1.2 Describe characteristics of network topology architectures<br>*1.2.c Spine-leaf*<br>**6.0 Automation and Programmability**<br>6.1 Explain how automation impacts network management<br>6.2 Compare traditional networks with controller-based networking<br>6.3 Describe controller-based, software-defined architecture (overlay, underlay, and fabric)<br>*6.3.a Separation of control plane and data plane*<br>*6.3.b Northbound and Southbound APIs* |
| Chapter 22: Cisco Software-Defined Access (Cisco SD-Access) | **1.0 Network Fundamentals**<br>1.1 Explain the role and function of network components<br>*1.1.e Controllers*<br>**6.0 Automation and Programmability**<br>6.1 Explain how automation impacts network management<br>6.2 Compare traditional networks with controller-based networking<br>6.3 Describe controller-based, software-defined architecture (overlay, underlay, and fabric)<br>6.4 Explain AI (generative and predictive) and machine learning in network operations |

| Book Chapter | Exam Topics Covered |
|---|---|
| Chapter 23: Understanding REST and JSON | **6.0 Automation and Programmability**<br>6.5 Describe characteristics of REST-based APIs (authentication types, CRUD, HTTP verbs, and data encoding)<br>6.7 Recognize components of JSON-encoded data |
| Chapter 24: Understanding Ansible and Terraform | **6.0 Automation and Programmability**<br>6.6 Recognize the capabilities of configuration mechanisms such as Ansible and Terraform |

*This page intentionally left blank*

# APPENDIX C

# Answers to the "Do I Know This Already?" Quizzes

## Chapter 1

1. C. The IEEE 802.3 standard defines Ethernet, while 802.11 defines Wi-Fi.
2. B. WLANs require half-duplex operation because all stations must contend for use of a channel to transmit frames.
3. C. An AP offers a basic service set (BSS). BSA is incorrect because it is a basic service area, or the cell footprint of a BSS. BSD is incorrect because it does not pertain to wireless at all. IBSS is incorrect because it is an independent BSS, or an ad hoc network, where an AP or BSS is not needed at all.
4. B. The AP at the heart of a BSS or cell identifies itself (and the BSS) with a Basic Service Set Identifier (BSSID). It also uses an SSID to identify the wireless network, but that is not unique to the AP or BSS. Finally, the radio MAC address is used as the basis for the BSSID value, but the value can be altered to form the BSSID for each SSID that the AP supports. The Ethernet MAC address is usually unique, but it is associated with the wired portion of the AP and does not identify the AP and its BSS.
5. B. A workgroup bridge acts as a wireless client but bridges traffic to and from a wired device connected to it.
6. B. In a mesh network, each mesh AP builds a standalone BSS. The APs relay client traffic to each other over wireless backhaul links, rather than wired Ethernet. Therefore, Ethernet cabling to each AP is not required.
7. D and E. Wi-Fi commonly uses the 2.5- and 5-GHz bands.
8. C and D. In the 2.4-GHz band, consecutively numbered channels are too wide to not overlap. Only channels 1, 6, and 11 are spaced far enough apart to avoid overlapping each other. In the 5-GHz band, all channels are considered to be nonoverlapping. (Note that 5-GHz channels are numbered as multiples of four, which gives sufficient spacing to avoid overlap.)

## Chapter 2

1. A. An autonomous AP can operate independently without the need for a centralized wireless LAN controller.
2. B. The Cisco Meraki APs are autonomous APs that are managed through a centralized platform in the Meraki cloud.
3. C. On a lightweight AP, the MAC function is divided between the AP hardware and the WLC. Therefore, the architecture is known as split-MAC.

4. B. An AP builds a CAPWAP tunnel with a WLC.
5. A. A trunk link carrying three VLANs is not needed at all. A Cisco AP in local mode needs only an access link with a single VLAN; everything else is carried over the CAPWAP tunnel to a WLC. The WLC will need to be connected to three VLANs so that it can work with the AP to bind them to the three SSIDs.
6. C. A centralized WLC deployment model is based around locating the WLC in a central location, to support a very large number of APs.
7. A. The local mode is the default mode, where the AP provides at least one functional BSS that wireless clients can join to connect to the network. Normal and client modes are not valid modes. Monitor mode is used to turn the AP into a dedicated wireless sensor.
8. D. The SE-Connect mode is used for spectrum analysis. "SE" denotes the Cisco Spectrum Expert software. Otherwise, an AP can operate in only one mode at a time. The local mode is the default mode.

## Chapter 3

1. D. For effective security, you should leverage authentication, MIC, and encryption.
2. C. A message integrity check (MIC) is an effective way to protect against data tampering. WIPS is not correct because it provides intrusion protection functions. WEP is not correct because it does not provide data integrity along with its weak encryption. EAP is not correct because it defines the framework for authentication.
3. D. WEP is known to have a number of weaknesses and has been compromised. Therefore, it has been officially deprecated and should not be used in a wireless network. AES is not a correct answer because it is the current recommended encryption method. WPA is not correct because it defines a suite of security methods. EAP is not correct because it defines a framework for authentication.
4. C. EAP works with 802.1x to authenticate a client and enable access for it. Open authentication and WEP cannot be correct because both define a specific authentication method. WPA is not correct because it defines a suite of security methods in addition to authentication.
5. A. The TKIP method was deprecated when the 802.11 standard was updated in 2012. CCMP and GCMP are still valid methods. EAP is an authentication framework and is not related to data encryption and integrity.
6. C. WPA2 uses CCMP only. WEP has been deprecated and is not used in any of the WPA versions. TKIP has been deprecated but can be used in WPA only. WPA is not a correct answer because it is an earlier version of WPA2.
7. B. The Wi-Fi Alliance offers the WPA, WPA2, and WPA3 certifications for wireless security. WEP, AES, and 802.11 are not certifications designed and awarded by the Wi-Fi Alliance.
8. A and C. The personal mode for WPA, WPA2, and WPA3 is used to require a preshared key authentication. Enterprise mode uses 802.1x instead.

## Chapter 4

1. A. A Cisco AP requires connectivity to only a single VLAN so that it can build CAPWAP tunnels to a controller, so access mode is used.
2. B. An autonomous AP must connect to each of the VLANs it will extend to wireless LANs. Therefore, its link should be configured as a trunk.
3. D. You can use HTTP and HTTPS to access the GUI of a wireless LAN controller, as well as SSH to access its CLI. While HTTP is a valid management protocol on a WLC, it is usually disabled to make the WLC more secure.
4. C. Controllers use a link aggregation group (LAG) to bundle multiple ports together.
5. D. A dynamic interface makes a logical connection between a WLAN and a VLAN, all internal to the AireOS controller.
6. A and D. A WLAN binds a wireless SSID to a wired VLAN through an internal controller interface.
7. C. You can configure a maximum of 512 WLANs on a controller. However, a maximum of only 16 of them can be configured on an AP.
8. A and C. A WLAN profile and a Policy profile are the only items from the list that are necessary. A channel number is not because it is supplied automatically or by more advanced AP configuration. A BSSID is not because it is the address that identifies the BSS supplied by an AP. An IP subnet is used on the VLAN and WLAN that are bound, but not for WLAN configuration.

## Chapter 5

1. D and E. Many headers include a field that identifies the next header that follows inside a message. Ethernet uses the Ethernet Type field, and the IP header uses the Protocol field. The TCP and UDP headers identify the application that should receive the data that follows the TCP or UDP header by using the port number field in the TCP and UDP headers, respectively.
2. A, B, C, and F. IP, not TCP, defines routing. Many other protocols define encryption, but TCP does not. The correct answers simply list various TCP features.
3. C. TCP, not UDP, performs windowing, error recovery, and ordered data transfer. Neither performs routing or encryption.
4. C and F. The terms *segment* and *L4PDU* refer to the header and data encapsulated by the transport layer protocol. The terms *packet* and *L3PDU* refer to the header plus data encapsulated by Layer 3. *Frame* and *L2PDU* refer to the header (and trailer), plus the data encapsulated by Layer 2.
5. B. Note that the hostname is all the text between the // and the /. The text before the // identifies the application layer protocol, and the text after the / represents the name of the web page.
6. C and D. Web traffic uses TCP as the transport protocol, with HTTP as the application protocol. As a result, the web server typically uses well-known TCP port 80, which is the well-known port for HTTP traffic. Messages flowing to the web server would have a destination TCP port of 80, and messages flowing from the server would have a source TCP port of 80.

## Chapter 6

1. A and C. Standard ACLs check the source IP address. An ACL can match the address range 10.1.1.1–10.1.1.4, but it requires multiple **access-list** commands. Matching all hosts in Barney's subnet can be accomplished with the **access-list 1 permit 10.1.1.0 0.0.0.255** command.
2. A and D. The range of valid ACL numbers for standard numbered IP ACLs is 1–99 and 1300–1999, inclusive.
3. D. The 0.0.0.255 wildcard mask matches all packets that have the same first three octets as the address in the ACL command. This mask is useful when you want to match a subnet in which the subnet part comprises the first three octets, as would be the case with a 255.255.255.0 subnet mask.
4. E. The 0.0.15.255 wildcard mask matches all packets with the same first 20 bits. This mask is useful when you want to match a subnet in which the subnet part comprises the first 20 bits, as in a subnet that uses the 255.255.240.0 subnet mask.
5. A. The router always searches the ACL statements in order and stops after making a match. In other words, it uses first-match logic. A packet with source IP address 1.1.1.1 would match any of the three explicitly configured commands described in the question; however, the first statement will be used.
6. B. The correct answer matches the range of addresses 172.16.4.0–172.16.5.255, which is the range of addresses in the listed subnets. Using the **access-list** command, you can add the address and wildcard mask to get 172.16.5.255 (the ending number in the range).

   One wrong answer, with wildcard mask 0.0.255.0, matches all packets that begin with 172.16, with a 5 in the last octet. Another wrong answer matches only specific IP address 172.16.5.0. A third wrong answer uses a wildcard mask of 0.0.0.127, which matches addresses 172.16.5.0 through 172.16.5.127.

## Chapter 7

1. C. Named standard ACLs begin with the **ip access-list standard** *name* global command. It moves the user into ACL configuration mode, which supports the configuration of **permit** and **deny** commands. Those commands match using the same options as numbered ACLs with the **access-list** global command. And while you enable the named ACL with an interface subcommand, the matching logic is configured in ACL configuration mode.
2. C. Two incorrect answers use incorrect syntax that begins with **permit** followed by a line number. The two answers that begin with a line number followed by **permit** use correct syntax.

   The question states that the named ACL was just created, with no line numbers used in that configuration. As a result, IOS assigns the three ACEs line numbers 10, 20, and 30, respectively. To insert another **permit** or **deny** command between the second and third ACEs, the new command must use a line number from 21 to 29 inclusive. Of the two syntactically correct answers, the correct answer uses a line number in the correct range.

3. E and F. Extended ACLs can look at the Layer 3 (IP) and Layer 4 (TCP, UDP) headers and a few others, but not any application layer information. Named extended ACLs can look for the same fields as numbered extended ACLs.
4. A and E. The correct range of ACL numbers for extended IP access lists is 100 to 199 and 2000 to 2699. The answers that list the **eq www** parameter after 10.1.1.1 match the source port number, and the packets go toward the web server, not away from it.
5. E. Because the packet is going toward any web client, you need to check for the web server's port number as a source port. The question does not specify client IP address ranges, but it does specify server address ranges; the source address beginning with 172.16.5 is the correct answer.
6. C. The question states that the output comes from a command in a router so that you can rely on the access control entries (ACEs) having correct syntax. You can also expect that for the address and wildcard pairs, the address represents the lowest number in a range, with the highest number found by adding the address and wildcard mask. For instance, 10.22.33.0 + 0.0.0.63, added octet by octet, gives you 10.22.33.63. That makes the question stem's 10.22.33.99 address not match the source address field in line 10, but it is within the source address range for lines 20 and 30 (10.22.33.0–10.22.33.127) and line 40 (10.22.33.0–10.22.33.255).

   Analyzing the destination address fields, all four ACL lines include destination address 10.33.22.22. The ranges include 10.33.22.0–10.33.22.127, 10.33.22.0–10.33.22.63, and 10.33.22.31.

   So far, that analysis rules out only line 10.

   Line 20 matches the source port, not the destination port, so its logic cannot match packets destined to an SSH server. SSH uses port 22, not 24, so the lines that use port number 24 (lines 20 and 40) cannot match SSH. Those facts rule out lines 20 and 40.

   Line 30 works because it matches the source and destination addresses per the question and also matches SSH as the destination port, port 22.

## Chapter 8

1. E. The question lists a command that enables the ACL for outbound packets. Routers do not apply ACL logic to packets created by that router, so the OSPF messages sent by the router will not drive ACL matching and will not match an ACE, therefore not incrementing ACE matching counters. All the incorrect answers imply that the router applied the ACL to outgoing OSPF packets created by the router, which is not true.
2. A and E. The DHCP messages per this question use
   - UDP
   - Source address 172.16.2.1 (server S1)
   - Source UDP port 67 (bootps)
   - Destination IP address 172.16.1.1 (the address of the router R1 interface with the **ip helper-address** command configured)

Two answers use the **ip** protocol keyword, so those ACEs match all DHCP messages, which use IP and UDP. Of those two answers, the one correct answer also matches the source address of the server's 172.16.2.1 IP address along with matching any destination address. The incorrect answer reverses the source and destination address fields and would match packets sent to the server rather than those coming from the server.

Of the three ACEs that refer to the **udp** protocol keyword, one lists the wrong source port keyword (**bootpc**, which implies port 68, instead of keyword **bootps**, which implies port 67). Another ACE lists the incorrect destination IP address (0.0.0.0). The one correct answer among those three matches the UDP protocol source address 172.16.2.1, source port bootps, and any destination address.

3. A. IOS supports enabling a standard ACL to filter inbound attempts to Telnet and SSH into the router. The enabled standard ACL checks the source IP address of the incoming packets. However, because IOS applies the filters for packets that attempt to log in to the router, it uses a different command and mode to enable the ACL: the **access-class** *name|number* **in** command in vty mode as listed in the correct answer.

4. B. In this scenario, the packets have these important protocol facts:

   - Protocol: IP followed by TCP
   - Source addresses: Subnet 172.16.1.0/24
   - Source port: dynamic (above 49,151)
   - Destination address: 172.16.12.1
   - Destination port: 22 (SSH well-known port)

   Given these facts, the two ACEs that match with the **udp** keyword will not match the packets. Of the other two, both match the details listed above. IOS uses first-match logic when processing ACLs, so the router will match the packets with the ACE at line 20.

5. C. Cisco IOS has long supported one IP ACL, per interface, per direction (in or out). For example, the **ip access-group acl_01 out** and **ip access-group acl_02 in** commands can coexist on an interface. However, if at that point you also configured the **ip access-group acl_03 out** command, it would replace the **ip access-group 1 out** command as the only outbound IP ACL on the interface.

6. B. IOS supports a command to resequence an ACL's sequence numbers, defining the starting and increment numbers. Using 50 as the starting number, with 20 as the increment, will renumber the first four ACEs to 50, 70, 90, and 110. Of the three commands that use the term **resequence** and the parameters **50 20**, the correct answer is the only one with the correct syntax and mode. There is no **resequence** subcommand in ACL mode.

   The other two incorrect answers would change the ACL to use the correct sequence numbers but would require several more commands to accomplish the task rather than the single command needed for the correct answer.

## Chapter 9

1. B. A vulnerability is a weakness that can be exploited. Attack is not correct because it is a threat that is taking place. The term *exploit* refers to a tool that can be used to exploit a specific vulnerability.
2. D. When a vulnerability can be exploited, a threat is possible.
3. A and B. Attackers usually spoof the source IP address in packets they send in order to disguise themselves and make the actual IP address owner into a victim of the attack. MAC addresses can also be spoofed in ARP replies to confuse other hosts and routers on the local network. Destination IP addresses are not normally spoofed because packets used in the attack would go to unknown or nonexistent hosts. Finally, ARP address is not correct because it is not a legitimate term.
4. D. A denial-of-service attack is likely occurring because the attacker is trying to exhaust the target's TCP connection table with embryonic or incomplete TCP connections.
5. C. In a reflection attack, the goal is to force one host (the reflector) to reflect the packets toward a victim. Therefore, the spoofed source address contains the address of the victim and not the reflector.
6. A and C. Once an attacker is in position in a man-in-the-middle attack, traffic between hosts can be passively inspected and actively modified. This type of attack does not lend itself to inducing buffer overflows or using sweeps and scans.
7. B. In a brute-force attack, an attacker's software tries every combination of letters, numbers, and special characters to eventually find a string that matches a user's password.
8. D. The Cisco ISE platform provides the AAA services needed for authentication, authorization, and accounting. DHCP does not perform AAA but leases IP addresses to hosts instead. DNS resolves hostnames to IP addresses. SNMP is used for network management functions.
9. C. Physical access control is a necessary element of a security program that keeps sensitive locations like data centers and network closets locked and inaccessible, except to authorized personnel.

## Chapter 10

1. B. If both commands are configured, IOS accepts only the password as configured in the **enable secret** command
2. A. The **service password-encryption** command encrypts passwords on a router or switch that would otherwise be shown in clear text. While a great idea in concept, the algorithm can be easily broken using websites found on the Internet. Cisco long ago provided replacements for commands that store passwords as clear text, instead using hashes—commands like **enable secret** and **username secret**. These commands are preferred in part because they avoid the issues of clear-text passwords and easily decrypted passwords.

3. B. The **enable secret** command stores an MD5 hash of the password. It is unaffected by the **service password-encryption** command. The router does not unhash the value back to the clear-text password. Instead, when the user types a clear-text password, the router also hashes that password and compares that hashed value with the hashed value as listed in the configuration.
4. B. The **username secret** command in the question stem shows a type of 8. Type 8 refers to the SHA256 hash type, configured with the **algorithm-type sha256** parameters. The other incorrect answers mention type 9 (Scrypt) and type 5 (MD5). Also, the one answer that omits the algorithm type has a different default based on whether using IOS (MD5) or IOS XE (Scrypt). So that answer would result in either type 5 or type 9, but not type 8.
5. B. Traditional and next-generation firewalls can check TCP and UDP port numbers, but next-generation firewalls are generally characterized as being able to also check application data beyond the Transport layer header. An NGFW would look into the application data, identifying messages that contain data structures used by Telnet, instead of matching with port numbers. This matching can catch attacks that seek to use port numbers that the firewall allows while using those ports to send data from applications that do not normally use those ports.

   For the other answers, a traditional firewall would likely match based on destination port 23, which is the well-known port for Telnet. IP protocol number has nothing to do with Telnet.
6. A and D. Both traditional and next-generation IPSs (NGIPSs) use a signature database, with each signature listing details of what fields would be in a series of messages to identify those messages as part of some exploit. They both also generate events for review by the security team.

   NGIPS devices add features that go beyond using a signature database, including gathering contextual information from hosts, like the OS used, currently running apps, open ports, and so on, so that the NGIPS does not have to log events if the hosts could not possibly be affected. Additionally, an NGIPS can use a list of reputation scores about IP addresses, domain names, and URIs of known bad actors, filtering traffic for sources that have a configured poor reputation level.

## Chapter 11

1. B. The setting for the maximum number of MAC addresses has a default of 1, so the **switchport port-security maximum** command does not have to be configured. With sticky learning, you do not need to predefine the specific MAC addresses either. However, you must enable port security, which requires the **switchport port-security** interface subcommand.
2. B and D. First, about the sticky parameter: this command causes the switch to learn the source MAC and to add it to a **switchport port-security mac-address** *address* interface subcommand. However, port security adds that command to the running-config file; the network engineer must also issue a **copy running-config startup-config** EXEC command to save that configuration.

   About the other correct answer, users can connect a switch to the end of the cable, with multiple devices connected to that switch. That happens in real networks when

users decide they need more ports at their desk. However, the default setting of **switchport port-security maximum 1** means that a frame from the second unique source MAC address would cause a violation, and with the default violation action, to err-disable the port.

For the other incorrect answer, the configuration does not prevent unknown MAC addresses from accessing the port because the configuration does not predefine any MAC address.

3. B and C. IOS adds MAC addresses configured by the port security feature as static MAC addresses, so they do not show up in the output of the **show mac address-table dynamic** command. **show mac address-table port-security** is not a valid command.
4. B. The question states that the port security status is secure-shutdown. This state is used only by the shutdown port security mode, and when used, it means that the interface has been placed into an err-disabled state. Those facts explain why the correct answer is correct, and two of the incorrect answers are incorrect.

   The incorrect answer that mentions the violation counter is incorrect because in shutdown mode, the violation counter no longer increments after the switch places the interface into secure-shutdown mode.
5. B and C. First, about the two incorrect answers: In restrict mode, the arrival of a frame that violates the port security policy does not cause the switch to put the interface into err-disabled state. It does cause the switch to discard any frames that violate the policy, but it leaves the interface up and does not discard frames that do not violate the security policy, like the second frame that arrives.

   Regarding the two correct answers, a port in port security restrict does cause the switch to issue log messages for a violating frame, send SNMP traps about that same event (if SNMP is configured), and increment the counter of violating frames.

## Chapter 12

1. A and C. DHCP Snooping must be implemented on a device that performs Layer 2 switching. The DHCP Snooping function needs to examine DHCP messages that flow between devices within the same broadcast domain (VLAN). Layer 2 switches, as well as multilayer switches, perform that function. Because a router performs only Layer 3 forwarding (that is, routing) and does not forward messages between devices in the same VLAN, a router does not provide a good platform to implement DHCP Snooping (and is not even a feature of Cisco IOS on routers). End-user devices would be a poor choice as a platform for DHCP Snooping because they would not receive all the DHCP messages, nor would they be able to prevent frames from flowing should an attack occur.
2. B and C. Switch ports connected to IT-controlled devices from which DHCP server messages may be received should be trusted by the DHCP Snooping function. Those devices include IT-controlled DHCP servers and IT-controlled routers and switches. All devices that are expected to be DHCP client devices (like PCs) are then treated as untrusted, because DHCP Snooping cannot know beforehand from which ports a DHCP-based attack will be launched. In this case, the ports connected to all three PCs will be treated as untrusted by DHCP Snooping.

3. C and D. Because of a default setting of untrusted, the switch does not need any configuration commands to cause a port to be untrusted. Of the two (incorrect) answers that relate to the trust state, **no ip dhcp snooping trust**, in interface config mode, would revert from a trust configuration state to an untrusted state. The other answer, **ip dhcp snooping untrusted**, is not a valid command.

   The two correct answers list a pair of configuration commands that both must be included to enable DHCP Snooping (**ip dhcp snooping**) and to specify the VLAN list on which DHCP Snooping should operate (**ip dhcp snooping vlan 5**).

4. A. All the answers list commands with correct syntax that are useful for DHCP Snooping. However, the correct answer, **no ip dhcp snooping information option**, disables DHCP Snooping's feature of adding DHCP Option 82 fields to DHCP messages. This setting is useful if the switch does not act as a DHCP relay agent. The opposite setting (without the **no** to begin the command) works when the multilayer switch acts as a DHCP relay agent.

5. B. DAI always uses a core function that examines incoming ARP messages, specifically the ARP message sender hardware and sender IP address fields, versus tables of data in the switch about correct pairs of MAC and IP addresses. DAI on a switch can use DHCP Snooping's binding table as the table of data with valid MAC/IP address pairs, or use the logic in configured ARP ACLs. The question stem states that DAI uses DHCP Snooping, so the correct answer notes that the switch will compare the ARP message's sender hardware address to the switch's DHCP Snooping binding table.

   One incorrect answer mentions a comparison of the message's ARP sender MAC (hardware) address with the message's Ethernet source MAC address. DAI can perform that check, but that feature can be configured to be enabled or disabled, so DAI would not always perform this comparison. The other incorrect answers list logic never performed by DAI.

6. B and D. Because of a default setting of untrusted, the switch must be configured so DAI trusts that one port. To add that configuration, the switch needs the **ip arp inspection trust** command in interface config mode. The similar (incorrect) answer of **no ip arp inspection untrust** is not a valid command.

   To enable DAI for operation on a VLAN, the configuration needs one command: the **ip arp inspection vlan 6** command. This command both enables DAI and does so specifically for VLAN 6 alone. The answer **ip arp inspection** shows a command that would be rejected by the switch as needing more parameters.

7. C and D. With DAI, you can set a limit on the number of received ARP messages with a default burst interval of 1 second, or you can configure the burst interval. Once configured, DAI allows the configured number of ARP messages over the burst interval number of seconds. With the two correct answers, one shows 16 ARP messages, with a 4-second interval, for an average of 4 per second. The other correct answer shows a limit of 4, with the default burst interval of 1 second, for an average of 4. The two incorrect answers result in averages of 2 per second and 5 per second.

## Chapter 13

1. D. By default, all message levels are logged to the console on a Cisco device. To do so, IOS uses logging level 7 (debugging), which causes IOS to send severity level 7, and levels below 7, to the console. All the incorrect answers list levels below level 7.
2. C. The **logging trap 4** command limits those messages sent to a syslog server (configured with the **logging host** *ip-address* command) to levels 4 and below, thus 0 through 4.
3. A. NTP uses protocol messages between clients and servers so that the clients can adjust their time-of-day clock to match the server. NTP is totally unrelated to interface speeds for Ethernet and serial interfaces. It also does not count CPU cycles, instead relying on messages from the NTP server. Also, the client defines the IP address of the server and does not have to be in the same subnet.
4. C. The **ntp server 10.1.1.1** command tells the router to be both an NTP server and client. However, the router first acts as an NTP client to synchronize its time with NTP server 10.1.1.1. Once synchronized, R1 knows the time to supply and can act as an NTP server.
5. E and F. CDP discovers information about neighbors. The **show cdp command** gives you several options that display more or less information, depending on the parameters used.
6. E and F. LLDP lists the neighbors' enabled capabilities in the output of the **show lldp neighbors** command, and both the enabled and possible (system) capabilities in the output of the **show lldp entry** *hostname* command.

## Chapter 14

1. B and E. RFC 1918 identifies private network numbers. It includes Class A network 10.0.0.0, Class B networks 172.16.0.0 through 172.31.0.0, and Class C networks 192.168.0.0 through 192.168.255.0.
2. C. With static NAT for source addresses (inside source NAT), the NAT router uses static entries defined by the **ip nat inside source** command. Because the question mentions translation for inside addresses, the command needs the **inside** keyword. Other NAT features not discussed in the chapter use the **outside** keyword.

   As for the other two answers, they both suggest triggering dynamic NAT table entries, which do not occur with static NAT.
3. A. With dynamic NAT, the entries are created due to the first packet flow from the inside network. Packets entering an outside interface do not trigger the creation of a NAT table entry. Dynamic NAT does not predefine NAT table entries, so the two answers that list configuration commands are incorrect.
4. A. The **ip nat inside source list alice pool barney** command enables inside source NAT. That means the router monitors packets that enter interfaces enabled for NAT with the **ip nat inside** interface subcommand. The router must also match and permit the packet with the referenced ACL (in this case, Alice) to trigger the translation. Those facts support the one correct answer.

One incorrect answer suggests that the ACL should deny packets instead of permitting them to trigger NAT. Instead, the ACL should permit the packet.

Two incorrect answers mention the NAT pool. When performing NAT and changing the source address, NAT uses an address from a defined pool (in this case, Barney). The packet that arrives in the inside interface does not list an address from the NAT pool at that point (before translation by NAT). Instead, the NAT pool includes public IP addresses representing the inside host. Those public addresses do not need to match anything in the NAT configuration.

5. A and C. The configuration lacks the **overload** keyword in the **ip nat inside source** command. Without this keyword, the router would perform dynamic NAT but not PAT, so it could not support more than one TCP or UDP connection or flow per inside global IP address. Also, each NAT outside interface needs the **ip nat outside** interface subcommand. The configuration lists interface G0/0/1 as its link connected to the Internet, with a public address, and it is missing this configuration command.
6. B. Regarding the correct answer: The last line in the output mentions that the pool has seven addresses, with all seven allocated, with the misses counter close to 1000—meaning that the router rejected roughly 1000 new flows because of insufficient space in the NAT pool. For the incorrect answers, NAT allows standard and extended ACLs, so NAT can use standard ACL 1. You can rule out the other two incorrect answers because the root cause, per the correct answer, can be found in the command output.

## Chapter 15

1. A, B, and E. QoS tools manage bandwidth, delay, jitter, and loss.
2. B and C. The IP Precedence (IPP) and Differentiated Services Code Point (DSCP) fields exist in the IP header and would flow from source host to destination host. The Class of Service (CoS) field exists in the 802.1Q header, so it would be used only on trunks, and it would be stripped of the incoming data-link header by any router in the path. The MPLS EXP bits exist as the packet crosses the MPLS network only.
3. A, B, and C. In general, matching a packet with DiffServ relies on a comparison to something inside the message itself. The 802.1p CoS field exists in the data-link header on VLAN trunks; the IP DSCP field exists in the IP header; and extended ACLs check fields in message headers. The SNMP Location variable does not flow inside individual packets but is a value that can be requested from a device.
4. B and C. Low Latency Queuing (LLQ) applies priority queue scheduling, always taking the next packet from the LLQ if a packet is in that queue. To prevent queue starvation of the other queues, IOS also applies policing to the LLQ. However, applying shaping to an LLQ slows the traffic, which makes no sense with the presence of a policing function already. The answer that refers to round-robin scheduling is incorrect because LLQ instead uses priority queue scheduling.
5. A and D. With a shaper enabled on R1 at a rate of 200 Mbps while R1 attempts to send 300 Mbps out that interface, R1 begins queuing packets. R1 then allows data transmission, so the transmission rate is 200 Mbps over time.

As for the policing function on ISP1, with a configured rate of 250 Mbps, the policer will measure the rate and see that the incoming rate (200 Mbps because of R1's shaping) does not exceed the policing rate.

6. C and D. Drop management relies on the behavior of TCP, in that TCP connections slow down sending packets due to the TCP congestion window calculation. Voice traffic uses UDP, and the question states that queue 1 uses UDP. So, queues 2 and 3 are reasonable candidates for using a congestion management tool.

## Chapter 16



1. D. With this design but no FHRP, host A can send packets off-subnet as long as connectivity exists from host A to R1. Similarly, host B can send packets off-subnet as long as host B has connectivity to router R2. Both routers can attach to the same LAN subnet and ignore each other concerning their roles as default routers because they do not use an FHRP option. When either router fails, the hosts using the failed router as the default router have no means to fail over.
2. C. The use of an FHRP in this design purposefully allows either router to fail and still support off-subnet traffic from all hosts in the subnet. Both routers can attach to the same LAN subnet per IPv4 addressing rules.
3. C. HSRP uses a virtual IP address. The virtual IP address comes from the same subnet as the routers' LAN interfaces but is a different IP address than the router addresses configured with the **ip address** interface subcommand. As a result, the hosts will not point to 10.1.19.1 or 10.1.19.2 as their default gateway in this design. The other wrong answer lists an idea of using the Domain Name System (DNS) to direct hosts to the right default router. Although an interesting idea, it is not a part of any of the three FHRP protocols.
4. B. Two answers mention load balancing the traffic hosts send in the subnet. Those hosts send traffic based on their default router setting, with HSRP creating redundancy for that function across routers R1, R2, and R3. HSRP provides active/standby load balancing, so all traffic flows through the currently active router (R2). Those facts identify one correct and one incorrect answer.

   As for the answer about ARP Requests, only the active router replies to the ARP Request. The standby routers sit silently, other than sending HSRP messages in anticipation of taking over as active one day.

   As for the answer about the virtual MAC, the end of the virtual MAC uses the three-digit hex equivalent of the decimal HSRP group number. The question lists a decimal HSRP group number of 16. Converted to hex, that gives you 10, or as a three-digit hex number, 010. So the correct virtual MAC address, 0000.0C9F.F010, ends in 010, not 016.
5. A and C. The answers to this question come in pairs, with one correct and one incorrect.In one pair, the answers ask if the VIP (virtual IP address) may be 10.1.1.3 or if it must be 10.1.1.3. R3's IP address, per the question stem, is 10.1.1.3. VRRP allows use of an interface IP address as the VIP but does not require it. (Note that HSRP and GLBP do not allow the use of an interface IP address as the VIP.) So, the answer stating that the VIP may be 10.1.1.3 is correct.

The pair of answers mentioning the multicast address used by VRRP requires you to recall the address. VRRP uses 224.0.0.18 (the correct answer), HSRPv1 uses 224.0.0.2 (the incorrect answer), and HSRPv2 uses 224.0.0.102.

6. D. GLBP makes each router in the group active, meaning each can act as the default router by using a unique GLBP virtual MAC address per router. All endpoint hosts have the same default router setting as normal. One GLBP router (the AVG) sends an ARP Reply in reaction to ARP Requests for the VIP IP address. The AVG's ARP Reply messages list different routers' virtual MAC addresses so that some hosts forward packets to one router and some to others.

   As for the incorrect answers, the answer about using a different VIP per router in the same group is not allowed. Also, using a separate GLBP group per router means that the routers are not providing redundancy to each other.

   Finally, one incorrect answer suggests using a VIP that is the same as one of the routers' interface IP addresses, which is not allowed with GLBP.

## Chapter 17

1. B. SNMPv1 and SNMPv2c use community strings to authenticate Get and Set messages from an NMS. The agent defines a read-only community and can define a read-write community as well. Get requests, which read information, will be accepted if the NMS sends either the read-only or the read-write community with those requests.
2. A and C. SNMP agents reside on a device being managed. When an event happens about which the device wants to inform the SNMP manager, the agent sends either an SNMP Trap or SNMP Inform to the SNMP manager. The SNMP manager normally sends an SNMP Get Request message to an agent to retrieve MIB variables or an SNMP Set Request to change an MIB variable on the agent. The agent responds with a Get Reply message.
3. A. GetNext allows for an improvement in efficiency for retrieving lists of MIB variables. However, SNMP Version 1 defines Get and GetNext, making both answers incorrect. GetBulk, which further improves efficiency of retrieving lists of variables, was added with SNMP Version 2, making that answer correct. Inform, also defined by SNMP Version 2, does not retrieve MIB variable data.
4. A. FTP uses both a control connection and a data connection. The FTP client initiates the control connection. However, in active mode, the FTP server initiates the data connection.
5. B and D. TFTP supports fewer functions than FTP as a protocol. For instance, the client cannot change the current directory on the server, add directories, remove directories, or list the files in the directory. Both TFTP and FTP support the ability to transfer files in either direction.
6. B and C. The **show** *filesystem*: EXEC command lists all files in the filesystem, whether in the root directory of the filesystem or in subdirectories. It returns a potentially long list of file and directory names. The **dir** EXEC command lists the files and directories in the filesystem and directory per the present working directory (**pwd**) command. In this case, it is set to the root of the file system, bootflash:. As a result, the **dir** command lists all files in the root of the file system, and directory names, but not files held within those directories.

## Chapter 18

1. B and D. The access layer switches connect to the endpoint devices, whether end-user devices or servers. Then, from the access to the distribution layer, each access layer connects to two distribution switches typically, but with no direct connections between access layer switches, creating a mesh (but a partial mesh). A two-tier design, called a collapsed core, does not use core switches.

2. A and C. The access layer switches, not the distribution layer switches, connect to the endpoint devices, whether end-user devices or servers. Then, from the access to the distribution layer, each access layer connects to two distribution switches typically, but with no direct connections between access layer switches, creating a mesh (but a partial mesh). A three-tier design, also called a core design, does use core switches, with a partial mesh of links between the distribution and core switches. Each distribution switch connects to multiple core switches but often does not connect directly to other distribution switches.

3. D. The access layer uses access switches, which connect to endpoint devices. A single access switch with its endpoint devices looks like a star topology, with a centralized node connected to each other node.

   A full mesh connects each node to every other node, with a partial mesh being any subset of a full mesh. Hybrid topologies refer to more complex topologies, including subsets that use a star, full mesh, or partial mesh design.

4. B and C. The three answers with *CAT* refer to UTP cabling standards defined by TIA and ANSI. The answers with *OM*, meaning Optical Multimode, refer to ISO standards for multimode fiber. The question asks about 1000BASE-T, a standard that calls for UTP cabling, making the two answers that begin with *OM* incorrect.

   Of the three answers that begin with *CAT*, the 1000BASE-T standard requires CAT 5E cable quality, or better, to support distances up to 100 meters. That makes CAT 5E and CAT 6 correct among the available answers.

5. A and C. With a SOHO LAN, one integrated device typically supplies all the necessary functions, including routing, switching, wireless access point (AP), and firewall. The AP uses standalone mode, without a wireless LAN controller (WLC), and without a need to encapsulate frames in CAPWAP.

6. A. PoE switch ports begin with power detection (PD) to determine whether the attached device needs to receive power. The question stem tells us that the power detection process has been completed, and the device needs power.

   The PoE switch port begins power classification, first with a Layer 1 process, followed by a Layer 2 process, which dictates how much power the switch supplies. The first phase, the Layer 1 process, has the switch supply a standard low-voltage signal (which identifies the correct answer). It can then use CDP or LLDP messages to classify the power further, often after the device has powered up with enough function to reply to CDP/LLDP messages. The switch does not apply power based on any configured setting until after completing the Layer 1 power detection phase.

7. B and D. Universal Power over Ethernet (UPoE) and the enhanced UPoE Plus (UPoE+) supply power over all four cable pairs. Note that 1000BASE-T and faster UTP-based Ethernet standards often require four pairs, whereas earlier/slower standards did not,

and UPoE/UPoE+ take advantage of the existence of four pairs to supply power over all four pairs. Power over Ethernet (PoE) and PoE+ use two pairs for power and therefore work with Ethernet standards like 10BASE-T and 100BASE-T that use two pairs only.

## Chapter 19

1. B and C. A Metro Ethernet E-Tree service uses a rooted point-to-multipoint Ethernet Virtual Connection (EVC), which means that one site connected to the service (the root) can communicate directly with each of the remote (leaf) sites. However, the leaf sites cannot send frames directly to each other; they can only send frames to the root site. Topology designs that allow a subset of all pairs in the group to communicate directly are called a partial mesh, or hub and spoke, or in some cases, a multipoint or point-to-multipoint topology.

   Of the incorrect answers, the *full mesh* term refers to topology designs in which each pair in the group can send data directly to each other, which is typical of a MetroE E-LAN service. The term *point-to-point* refers to topologies with only two nodes in the design, and they can send directly to each other, typical of a MetroE E-Line service.

2. A. Metro Ethernet uses Ethernet access links of various types. Time-division multiplexing (TDM) links, such as serial links, and even higher-speed links like T3 and E3, do not use Ethernet protocols and are less likely to be used. MPLS is a WAN technology that creates a Layer 3 service.

   Two answers refer to Ethernet standards usable as the physical access link for a Metro Ethernet service. However, 100BASE-T supports cable lengths of only 100 meters, so it is less likely to be used as a Metro Ethernet access link than 100BASE-LX10, which supports lengths of 10 km.

3. A and D. An E-LAN service is one in which the Metro Ethernet service acts as if the WAN were a single Ethernet switch so that each device can communicate directly with every other device. As a result, the routers sit in the same subnet. With one headquarters router and ten remote sites, each router will have ten OSPF neighbors.

4. B and C. A Layer 3 MPLS VPN creates an IP service with a different subnet on each access link. With one headquarters router and 10 remote sites, 11 access links exist, so 11 subnets are used.

   Each enterprise (CE) router has an OSPF neighbor relationship with the MPLS provider edge (PE) router, but the CE routers do not have OSPF neighbor relationships. As a result, each remote site router would have only one OSPF neighbor relationship.

5. D. Architecturally, MPLS allows for a wide variety of access technologies. They include TDM (serial links), Frame Relay, ATM, Metro Ethernet, and traditional Internet access technologies such as DSL and cable.

6. A and B. The term *remote access VPN* refers to a VPN for which one endpoint is a user device, such as a phone, tablet, or PC, with the other as a VPN concentrator, often a firewall or router. The VPN concentrator configuration dictates the protocol the VPN client should use, typically either TLS or IPsec.

Of the incorrect answers, site-to-site VPNs use GRE along with IPsec. FTPS refers to FTP Secure, which uses TLS to secure FTP sessions.

## Chapter 20

1. A, B, and E. The hypervisor will virtualize each VM's RAM, CPU, NICs, and storage. The hypervisor itself is not virtualized but rather does the work of virtualizing other resources. Also, as virtual machines, the VMs do not use power, so the system does not have a concept of virtualized power.
2. D. Hypervisors create a virtual equivalent of Ethernet switching and cabling between the VMs and the physical NICs. The VMs use a virtual NIC (vNIC). The hypervisor uses a virtual switch (vSwitch), which includes the concept of a link between a vSwitch port and each VM's vNIC. The vSwitch also connects to both physical NICs. The switch configuration creates VLANs and trunks as needed.
3. B. Platform as a Service (PaaS) supplies one or more virtual machines (VMs) that have a working operating system (OS) as well as a predefined set of software development tools.

   As for the wrong answers, Software as a Service (SaaS) supplies a predefined software application but typically cannot install your applications later. Infrastructure as a Service (IaaS) delivers one or more working VMs, optionally with an OS installed. It could be used for software development, but the developer would have to install a variety of development tools, making IaaS less useful for development than a PaaS service. Finally, cloud services offer Server Load Balancing as a Service (SLBaaS). Still, it is not a general service in which customers get access to VMs to install their applications.
4. A. Infrastructure as a Service (IaaS) supplies one or more working virtual machines (VMs), optionally with an OS installed, where you can customize the systems by installing your own applications.

   Software as a Service (SaaS) supplies a predefined software application, but typically you cannot install your own applications later. Platform as a Service (PaaS) allows you to install your application because PaaS does supply one or more VMs. However, PaaS acts as a software development environment, with VMs that include various useful tools for software development. Finally, cloud services offer Server Load Balancing as a Service (SLBaaS). Still, it is not a general service in which customers get access to VMs to install their applications.
5. A. Both Internet options allow for easier migration because public cloud providers typically provide easy access over the Internet. An intercloud exchange is a purpose-built WAN service connecting enterprises and most public cloud providers, making the migration process more manageable.

   The one correct answer—the one that creates the most migration problems—is to use a private WAN connection to one cloud provider. While useful in other ways, migrating using this strategy would require installing a new private WAN connection to the new cloud provider.
6. A and C. Private WAN options use technologies like Ethernet WAN and MPLS, which keep data private and include QoS services. An intercloud exchange is a purpose-built

WAN service that connects enterprises and most public cloud providers using the same kinds of private WAN technology with those same benefits.

For the two incorrect answers, both use the Internet, so both cannot provide QoS services. The Internet VPN option does encrypt the data to keep it private.

## Chapter 21

1. A. The *data plane* includes all networking device actions related to the receipt, processing, and forwarding of each message, as in the case described in the question. The term *table plane* is not used in networking. The *management plane* and *control plane* are not concerned with the per-message forwarding actions.
2. C. The *control plane* includes all networking device actions that create the information used by the data plane when processing messages. The control plane includes functions like IP routing protocols and Spanning Tree Protocol (STP).

   The term *table plane* is not used in networking. The *management plane* and *data plane* are not concerned with collecting the information that the data plane then uses.
3. C. Although many variations of SDN architectures exist, they typically use a centralized controller. That controller may centralize some or even all control plane functions in the controller. However, the data plane function of receiving messages, matching them based on header fields, taking actions (like making a forwarding decision), and forwarding the message still happens on the network elements (switches) and not on the controller.

   For the incorrect answers, the control plane functions may all happen on the controller, or some may happen on the controller, and some on the switches. The northbound and southbound interfaces are API interfaces on the controller, not on the switches.
4. A. The OpenDaylight Controller uses an Open SDN model with an OpenFlow southbound interface as defined by the Open Networking Foundation (ONF). The ONF SDN model centralizes most control plane functions. The APIC model for data centers partially centralizes control plane functions. The Cisco 9800 Series controller runs a distributed control plane.
5. C and D. ACI uses a spine-leaf topology. With a single-site topology, leaf switches must connect to all spine switches, and leaf switches must not connect to other leaf switches. Additionally, a leaf switch connects to some endpoints, with the endpoints being spread across the ports on all the leaf switches. (In some designs, two or more leaf switches connect to the same endpoints for redundancy and more capacity.)
6. A and D. Controller-based networks use a controller that communicates with each network device using a southbound interface (an API and protocol). By gathering network information into one central device, the controller can then allow for different operational models. The models often let the operator think in terms of enabling features in the network, rather than thinking about the particulars of each device and command on each device. The controller then configures the specific commands, resulting in more consistent device configuration.

   For the incorrect answers, both the old and new models use forwarding tables on each device. Also, controllers do not add to or remove from the programmatic interfaces

on each device, some of which existed before controllers, but rather supply useful and powerful northbound APIs.

## Chapter 22

1. C. The Cisco SD-Access underlay consists of the network devices and connections, along with configuration that allows IP connectivity between the Cisco SD-Access nodes, for the purpose of supporting overlay VXLAN tunnels. The fabric includes both the underlay and overlay, while VXLAN refers to the protocol used to create the tunnels used by the overlay.

2. B. The overlay includes the control plane and data plane features to locate the endpoints, decide to which fabric node a VXLAN tunnel should connect, direct the frames into the tunnel, and perform VXLAN tunnel encapsulation and de-encapsulation. The Cisco SD-Access underlay exists as network devices, links, and a separate IP network to provide connectivity between nodes to support the VXLAN tunnels.

   The fabric includes both the underlay and overlay, while VXLAN refers to the protocol used to create the tunnels used by the overlay.

3. D. The Cisco SD-Access overlay creates VXLAN tunnels between fabric edge nodes. Edge nodes then create a data plane by forwarding frames sent by endpoints over the VXLAN tunnels. LISP plays a role in the overlay as the control plane, which learns the identifiers of each endpoint, matching the endpoint to the fabric node that can teach the endpoint, so that the overlay knows where to create VXLAN tunnels.

   For the other incorrect answers, note that while GRE is a tunneling protocol, Cisco SD-Access uses VXLAN for tunneling, and not GRE. Finally, OSPF acts as a control plane routing protocol, rather than a data plane protocol for Cisco SD-Access.

4. A and D. As with any Cisco SD-Access feature, the configuration model is to configure the feature using Cisco Catalyst Center, with Cisco Catalyst Center using southbound APIs to communicate the intent to the devices. The methods to configure the feature using Cisco Catalyst Center include using the GUI or using the northbound REST-based API.

   Of the incorrect answers, you would not normally configure any of the Cisco SD-Access devices directly. Also, while Cisco Catalyst Center can use NETCONF as a southbound protocol to communicate with the Cisco SD-Access fabric nodes, it does not use NETCONF as a northbound API for configuration of features.

5. B, C, and D. Cisco Catalyst Center manages traditional network devices with traditional protocols like Telnet, SSH, and SNMP. Cisco Catalyst Center can also use NETCONF and RESTCONF if supported by the device. Note that while useful tools, Ansible and Puppet are not used by Cisco Catalyst Center.

6. A and D. Traditional network management platforms can do a large number of functions related to managing traditional networks and network devices, including the items listed in the two correct answers. However, when using Cisco's Prime Infrastructure as a traditional network management platform for comparison, it does not support Cisco SD-Access configuration, nor does it find the end-to-end path between two endpoints and analyze the ACLs in the path. Note that the two incorrect answers reference features available in Cisco Catalyst Center.

7. B. Narrow AI is designed for specific tasks, while Generative AI has the capability to learn, make decisions, and potentially mimic human cognition through experiences.

## Chapter 23

1. B and D. The six primary required features of REST-based APIs include three features mentioned in the answers: a client/server architecture, stateless operation, and notation of whether each object is cacheable. Two items from these three REST attributes are the correct answers. Of the incorrect answers, stateful operation is the opposite of the REST-based API feature of stateless operation. For the other incorrect answer, although many REST-based APIs happen to use HTTP, REST APIs do not have to use HTTP.
2. B and D. In the CRUD software development acronym, the matching terms (create, read, update, delete) match one or more HTTP verbs. While the HTTP verbs can sometimes be used for multiple CRUD actions, the following are the general rules: create performed by HTTP POST; read by HTTP GET; update by HTTP PATCH, PUT (and sometimes POST); delete by HTTP DELETE.
3. C. The URI for a REST API call uses a format of protocol://hostname/resource?parameters. The API documentation details the resource part of the URI, as well as any optional parameters. For instance, in this case, the resource section is /dna/intent/api/v1/network-device. Additionally, the API documentation for this resource details optional parameters in the query field as listed after the ? in the URI.
4. A and D. Of the four answers, two happen to be most commonly used to format and serialize data returned from a REST API: JSON and XML. For the incorrect answers, JavaScript is a programming language that first defined JSON as a data serialization language. YAML is a data serialization/modeling language and can be found most often in configuration management tools like Ansible.
5. A and D. JSON defines variables as key:value pairs, with the key on the left of the colon (:) and always enclosed in double quotation marks, with the value on the right. The value can be a simple value or an object or array with additional complexity. The number of objects is defined by the number of matched curly brackets ({ and }), so this example shows a single JSON object.

   The one JSON object shown here includes one key and one :, so it has a single key:value pair (making one answer correct). The value in that key:value pair itself is a JSON array (a list in Python) that lists numbers 1, 2, and 3. The fact that the list is enclosed in square brackets defines it as a JSON array.
6. C and D. To interpret this JSON data, first look for the innermost pairing of either curly brackets { }, which denote one object, or square brackets [ ], which indicate one array. In this case, the content within the inner pair of curly brackets { } shows one JSON object.

   Inside that one object, four key:value pairs exist, with the key before each colon and the value after each colon. That means “type” is a key, while “ACCESS” and “10.10.22.66” are values.

If you examine the outer pair of curly brackets that begin and end the JSON data, that pair also defines an object. That object has one key of "response" (making that answer incorrect). The "response" key then has a value equal to the entire inner object.

## Chapter 24

1. C. Devices with the same role in an enterprise should have a similar configuration. When engineers make unique changes on individual devices—different changes from those made in the majority of devices with that same role—those devices' configurations become different than the intended ideal configuration for every device with that role. This effect is known as configuration drift. Configuration management tools can monitor a device's configuration versus a file that shows the intended ideal configuration for devices in that role, noting when the device configuration drifts away from that ideal configuration.

2. A and B. The version control system, applied to the centralized text files that contain the device configurations, automatically tracks changes. That means the system can see which user edited the file, when, and exactly what change was made, with the ability to make comparisons between different versions of the files.

   The two incorrect answers list useful features of a configuration management tool, but those answers list features typically found in the configuration management tool itself rather than in the version control tool.

3. D. Configuration monitoring (a generic description) refers to a process of checking the device's actual configuration versus the configuration management system's intended configuration for the device. If the actual configuration has moved away from the intended configuration—that is, if configuration drift has occurred—configuration monitoring can either reconfigure the device or notify the engineering staff.

   For the other answers, two refer to features of the associated version control software typically used along with the configuration management tool. Version control software will track the identity of each user who changes files and track the differences in files over time. The other incorrect answer is a useful feature of many configuration management tools, in which the tool verifies that the configuration will be accepted when attempted (or not). However, that useful feature is not part of what is called configuration monitoring

4. A and C. Both Ansible and Terraform can use a push model. The Ansible control node decides when to configure a device based on the instructions in a playbook. Although Ansible uses a push model by default, it can also use a pull model with a program called Ansible-pull.

5. B and C. These files go by the names *Ansible Playbook* and *Terraform Configuration*.

*This page intentionally left blank*

# GLOSSARY

## NUMERICS

**2.4-GHz band** The frequency range between 2.400 and 2.4835 GHz that is used for wireless LAN communication.

**2.5GBase-T** One of two Ethernet physical layer standards called multigigabit Ethernet (the other being 5.0GBase-T), first defined in IEEE addendum 802.3bz, which defines a 2.5 Gbps data rate over Cat 5E UTP cabling at distances of 100 meters.

**3G/4G Internet** An Internet access technology that uses wireless radio signals to communicate through mobile phone towers, most often used by mobile phones, tablets, and some other mobile devices.

**5-GHz band** The frequency range between 5.150 and 5.825 GHz that is used for wireless LAN communication.

**5.0GBase-T** One of two Ethernet physical layer standards called multigigabit Ethernet (the other being 2.5GBase-T), first defined in IEEE addendum 802.3bz, which defines a 2.5 Gbps data rate over Cat 5E UTP cabling at distances of 100 meters.

**6-GHz band** The frequency range between 5.925 and 7.125 GHz that is used for wireless LAN communication.

**10BASE-T** An Ethernet physical layer standard, first defined directly in the 802.3 standard as the first UTP-based Ethernet physical layer standard. It uses two twisted-pair UTP cabling and supports 10 Mbps data rates.

**10GBASE-SR** An Ethernet physical layer standard that uses optical multimode (OM) cabling, first defined in IEEE addendum 802.3ae, which defines a 10 Gbps data rate.

**10GBase-T** An Ethernet physical layer standard, introduced as IEEE addendum 802.3an, supporting 10 Gbps data rates over four-pair UTP cabling.

**40GBase-T** An Ethernet physical layer standard, introduced in IEEE addendum 802.3ba, supporting 40 Gbps data rates over four-pair UTP cabling.

**100BASE-T** An Ethernet physical layer standard, introduced as IEEE addendum 802.3u, supporting 100 Mbps data rates over two-pair UTP cabling.

**1000BASE-SX** An Ethernet physical layer standard that uses optical multimode (OM) cabling, first defined in IEEE addendum 802.3z, which defines a 1 Gbps data rate.

**1000BASE-T** An Ethernet physical layer standard, introduced as IEEE addendum 802.3ab, supporting 1000 Mbps (1 Gbps) data rates over four-pair UTP cabling.

# A

**AAA** Authentication, authorization, and accounting. Authentication confirms the identity of the user or device. Authorization determines what the user or device is allowed to do. Accounting records information about access attempts, including inappropriate requests.

**AAA server** *See* authentication, authorization, and accounting (AAA) server.

**access control entry (ACE)** One configuration line with a permit or deny action in an access control list (ACL).

**access interface** A LAN network design term that refers to a switch interface connected to end-user devices, configured so that it does not use VLAN trunking.

**access layer** In a campus LAN design, the switches that connect directly to endpoint devices (servers, user devices), and also connect into the distribution layer switches.

**access link** In campus LAN design, a link that connects an access switch to endpoint devices and wireless access points.

**access link (WAN)** A physical link between a service provider and its customer that provides access to the SP's network from that customer site.

**access point (AP)** A device that provides wireless service for clients within its coverage area or cell, with the AP connecting to both the wireless LAN and the wired Ethernet LAN.

**accounting** In security, the recording of access attempts. *See also* AAA.

**ACI** *See* Application Centric Infrastructure (ACI).

**ACL** Access control list. A list configured on a router to control packet flow through the router, such as to prevent packets with a certain IP address from leaving a particular interface on the router.

**ACL persistence** A feature of IOS XE (but not IOS) by which a router initialization event (power off/on or reload) does not cause reassigning ACL sequence numbers. The feature can be enabled (default, meaning no resequencing) or disabled.

**ACL resequencing** The process of renumbering the sequence numbers of ACL commands, either for all ACLs at router initialization (power off/on or reload) or by using a command to renumber individual ACLs.

**ACL sequence number** A number assigned to each ACL ACE when configured, either automatically or as typed in the configuration command, which allows easier deletion of individual ACEs.

**ad hoc wireless network** *See* independent basic service set (IBSS).

**administrative distance** In Cisco routers, a means for one router to choose between multiple routes to reach the same subnet when those routes are learned by different routing protocols. The lower the administrative distance, the more preferred the source of the routing information.

**agent-based architecture** With configuration management tools, an architecture that uses a software agent inside the device being managed as part of the functions to manage the configuration.

**agentless architecture** With configuration management tools, an architecture that does not need a software agent inside the device being managed as part of the functions to manage the configuration, instead using other mainstream methods like SSH and NETCONF.

**AI Ops** Artificial Intelligence for IT Operations. Refers to the application of artificial intelligence and machine learning techniques to automate and enhance various aspects of IT operations, including monitoring, troubleshooting, and incident management, to improve efficiency and reliability in managing complex IT environments.

**amplification attack** A reflection attack that leverages a service on the reflector to generate and reflect huge volumes of reply traffic to the victim.

**Ansible** A popular configuration management application, which can be used with or without a server, using a push model to move configurations into devices, with strong capabilities to manage network device configurations.

**APIC** *See* Application Policy Infrastructure Controller.

**Application Centric Infrastructure (ACI)** Cisco's data center SDN solution, the concepts of defining policies that the APIC controller then pushes to the switches in the network using the OpFlex protocol, with the partially distributed control plane in each switch building the forwarding table entries to support the policies learned from the controller. It also supports a GUI, a CLI, and APIs.

**Application Policy Infrastructure Controller (APIC)** The software that plays the role of controller, controlling the flows that the switches create to define where frames are forwarded, in a Cisco data center that uses the Application Centric Infrastructure (ACI) approach, switches, and software.

**application programming interface (API)** A software mechanism that enables software components to communicate with each other.

**application-specific integrated circuit (ASIC)** An integrated circuit (computer chip) designed for a specific purpose or application, often used to implement the functions of a networking device rather than running a software process as part of the device's OS that runs on a general-purpose processor.

**ARP reply** An ARP message used to supply information about the sending (origin) host's hardware (Ethernet) and IP addresses as listed in the origin hardware and origin IP address fields. Typically sent in reaction to receipt of an ARP request message.

**Artificial Intelligence (AI)** Refers to computer systems or software that can perform tasks typically requiring human intelligence, such as learning from data, making decisions, and solving problems.

**ASIC** *See* application-specific integrated circuit.

**association** A negotiated relationship between a wireless station and an access point.

**association request** An 802.11 frame that a wireless client sends to an AP to request an association with it.

**association response** An 802.11 frame that a wireless access point sends to a wireless client in reply to an association request.

**authentication** In security, the verification of the identity of a person or a process. *See also* AAA.

**authentication, authorization, and accounting (AAA) server** A server that holds security information and provides services related to user login, particularly authentication (is the user who he says he is), authorization (once authenticated, what do we allow the user to do), and accounting (tracking the user).

**authorization** In security, the determination of the rights allowed for a particular user or device. *See also* AAA.

**autonomous AP** A wireless AP operating in a standalone mode, such that it can provide a fully functional BSS and connect to the DS.

# B

**band** A contiguous range of frequencies.

**bandwidth** The speed at which bits can be sent and received over a link.

**basic service set (BSS)** Wireless service provided by one AP to one or more associated clients.

**basic service set identifier (BSSID)** A unique MAC address that is used to identify the AP that is providing a BSS.

**beacon** An 802.11 frame that an AP broadcasts at regular intervals to advertise the existence of an SSID. Separate beacons are transmitted for each SSID on each channel being used.

**brute-force attack** An attack where a malicious user runs software that tries every possible combination of letters, numbers, and special characters to guess a user's password. Attacks of this scale are usually run offline, where more computing resources and time are available.

**buffer overflow attack** An attack meant to exploit a vulnerability in processing inbound traffic such that the target system's buffers overflow; the target system can end up crashing or inadvertently running malicious code injected by the attacker.

# C

**cacheable** For resources that might be repeatedly requested over time, an attribute that means that the requesting host can keep in storage (cache) a copy of the resource for a specified amount of time.

**CAPWAP** A standards-based tunneling protocol that defines communication between a light-weight AP and a wireless LAN controller.

**CAT 5E** An unshielded twisted-pair (UTP) cable quality standard from the TIA and ANSI. It supports 1000BASE-T (and slower) UTP Ethernet at distances of 100 meters. It also supports multigig Ethernet standards at 100-meter distances.

**CAT 6A** An unshielded twisted-pair (UTP) cable quality standard from the TIA and ANSI. It is the lowest UTP cable category that formally supports 10GBase-T UTP Ethernet at distances of 100 meters.

**CDP** Cisco Discovery Protocol. A media- and protocol-independent device-discovery protocol that runs on most Cisco-manufactured equipment, including routers, access servers, and switches. Using CDP, a device can advertise its existence to other devices and receive information about other devices on the same LAN or on the remote side of a WAN.

**cell** The area of wireless coverage provided by an AP; also known as the basic service area.

**centralized control plane** An approach to architecting network protocols and products that places the control plane functions into a centralized function rather than distributing the function across the networking devices.

**centralized WLC deployment** A wireless network design that places a WLC centrally within a network topology.

**channel** An arbitrary index that points to a specific frequency within a band.

**Chat Ops** A collaboration model that integrates chat tools with automated workflows and tools, allowing teams to manage and execute tasks directly within chat interfaces, enhancing communication, visibility, and efficiency in operations and development workflows.

**CIDR** Classless interdomain routing. An RFC-standard tool for global IP address range assignment. CIDR reduces the size of Internet routers' IP routing tables, helping deal with the rapid growth of the Internet. The term *classless* refers to the fact that the summarized groups of networks represent a group of addresses that do not conform to IPv4 classful (Class A, B, and C) grouping rules.

**Cisco AnyConnect Secure Mobility Client** Cisco software product used as client software on user devices to create a client VPN. Commonly referred to as the Cisco VPN client.

**Cisco Catalyst 8000V** A virtual router platform designed for cloud and virtualized environments, offering high-performance, scalable, and secure routing capabilities with flexibility and agility.

**Cisco Catalyst Center** Cisco software, delivered by Cisco on a physical or virtual appliance, that acts as a network management application as well as being the control for Cisco's Software-Defined Access (Cisco SD-Access) offering.

**Cisco Prime Infrastructure (PI)** Graphical user interface (GUI) software that utilizes SNMP and can be used to manage your Cisco network devices. The term *Cisco Prime* is an umbrella term that encompasses many different individual software products.

**Cisco SD-Access** Cisco's intent-based networking (IBN) offering for enterprise networks.

**Cisco Secure Client (including AnyConnect)** Cisco software product used as client software on user devices to create a client VPN. Formerly called Cisco AnyConnect Secure Mobility Client, and sometimes referred to as the Cisco VPN client.

**Class of Service (CoS)** The informal term for the 3-bit field in the 802.IQ header intended for marking and classifying Ethernet frames for the purposes of applying QoS actions. Another term for Priority Code Point (PCP).

**classification** The process of examining various fields in networking messages in an effort to identify which messages fit into certain predetermined groups (classes).

**cloud-based AP** A wireless AP operating much like an autonomous AP, but having management and control functions present in the Internet cloud.

**cloud-based WLC deployment** A wireless network design that places a WLC centrally within a network topology, as a virtual machine in the private cloud portion of a data center.

**Cloud Management** Involves administering, monitoring, and optimizing cloud resources and services to ensure efficient utilization, performance, security, and compliance across cloud environments.

**cloud services catalog** A listing of the services available in a cloud computing service.

**code integrity** A software security term that refers to how likely that the software (code) being used is the software supplied by the vendor, unchanged, with no viruses or other changes made to the software.

**collapsed core design** A campus LAN design in which the design does not use a separate set of core switches in addition to the distribution switches—in effect collapsing the core into the distribution switches.

**Common ACL** A feature of IOS XE (but not IOS) that supports enabling two ACLs on a single interface and direction.

**configuration drift** A phenomenon that begins with the idea that devices with similar roles can and should have a similar standard configuration, so when one device's configuration is changed to be different, its configuration is considered to have moved away (drifted) from the standard configuration for a device in that role.

**configuration management** A component of network management focused on creating, changing, removing, and monitoring device configuration.

**configuration management tool** A class of application that manages data about the configuration of servers, network devices, and other computing nodes, providing consistent means of describing the configurations, moving the configurations into the devices, noticing unintended changes to the configurations, and troubleshooting by easily identifying changes to the configuration files over time.

**configuration monitoring** With configuration management tools like Ansible, a process of comparing over time a device's on-device configuration (running-config) versus the text file showing the ideal device configuration listed in the tool's centralized configuration repository. If different, the process can either change the device's configuration or report the issue.

**configuration provisioning** With configuration management tools like Ansible, the process of configuring a device to match the configuration as held in the configuration management tool.

**configuration template** With configuration management tools like Ansible, a file with variables, for the purpose of having the tool substitute different variable values to create the configuration for a device.

**connected mode** The operational mode used by a FlexConnect AP when the path back to its controller is up and working. In this mode, all wireless traffic flows over the CAPWAP tunnel to and from the controller.

**connection establishment** The process by which a connection-oriented protocol creates a connection. With TCP, a connection is established by a three-way transmission of TCP segments.

**container** One instance of a running application started from a container image and controlled by a container engine on a server.

**container image** One file that holds and embeds all files related to an application: all related executable files, required software libraries, and other files such as operating environment variables. Container virtualization systems then allow treating the application as a single file for movement, starting, stopping, and monitoring the container.

**control plane** Functions in networking devices and controllers that directly control how devices perform data plane forwarding, but excluding the data plane processes that work to forward each message in the network.

**controller-based networking** A style of building computer networks that use a controller that centralizes some features and provides application programming interfaces (APIs) that allow for software interactions between applications and the controller (northbound APIs) and between the controller and the network devices (southbound APIs).

**controller-less wireless deployment** A wireless design based on an embedded wireless controller (EWC), where the WLC function is co-located with an AP, rather than a discrete physical controller.

**core** In computer architecture, an individual processing unit that can execute instructions of a CPU; modern server processors typically have multiple cores, each capable of concurrent execution of instructions.

**core design** A campus LAN design that connects each access switch to distribution switches, and distribution switches into core switches, to provide a path between all LAN devices.

**core layer** In a campus LAN design, the switches that connect the distribution layer switches, and to each other, to provide connectivity between the various distribution layer switches.

**CRUD** In software development, an acronym that refers to the four most common actions taken by a program: Create, Read, Update, and Delete.

# D

**data plane** Functions in networking devices that are part of the process of receiving a message, processing the message, and forwarding the message.

**data serialization language** A language that includes syntax and rules that provides a means to describe the variables inside applications in a text format, for the purpose of sending that text between applications over a network or storing the data models in a file.

**Declarative Model** A method of describing IT automation that defines or declares the intended configuration, with the expectation that the automation software monitors the devices' configurations and changes them if they drift away from the intended (declared) configuration.

**declarative policy model** A term that describes the approach in an intent-based network (IBN) in which the engineer chooses settings that describe the intended network behavior (the declared policy) but does not command the network with specific configuration commands for each protocol (as would be the case with an imperative policy model).

**delay** In QoS, the amount of time it takes for a message to cross a network. Delay can refer to one-way delay (the time required for the message to be sent from the source host to the destination host) or two-way delay (the delay from the source to the destination host and then back again).

**denial-of-service (DoS) attack** An attack that tries to deplete a system resource so that systems and services crash or become unavailable.

**deny** An action taken with an ACL that implies that the packet is discarded.

**DevNet** Cisco's community and resource site for software developers, open to all, with many great learning resources; https://developer.cisco.com.

**DHCP attack** Any attack that takes advantage of DHCP protocol messages.

**DHCP Snooping** A switch security feature in which the switch examines incoming DHCP messages and chooses to filter messages that are abnormal and therefore might be part of a DHCP attack.

**DHCP Snooping binding table** When using DHCP Snooping, a table that the switch dynamically builds by analyzing the DHCP messages that flow through the switch. DHCP Snooping can use the table for part of its filtering logic, with other features, such as Dynamic ARP Inspection and IP Source Guard also using the table.

**dictionary attack** An attack where a malicious user runs software that attempts to guess a user's password by trying words from a dictionary or word list.

**dictionary variable** In applications, a single variable whose value is a list of other variables with values, known as key:value pairs.

**Differentiated Services (DiffServ)** An approach to QoS, originally defined in RFC 2475, that uses a model of applying QoS per classification, with planning of which applications and other traffic types are assigned to each class, with each class given different QoS per-hop behaviors at each networking device in the path.

**Differentiated Services Code Point (DSCP)** A field existing as the first 6 bits of the ToS byte, as defined by RFC 2474, which redefined the original IP RFC's definition for the IP header ToS byte. The field is used to mark a value in the header for the purpose of performing later QoS actions on the packet.

**distributed control plane** An approach to architecting network protocols and products that places some control plane functions into each networking device rather than centralizing the control plane functions in one or a few devices. An example is the use of routing protocols on each router which then work together so that each router learns Layer 3 routes.

**distributed denial-of-service (DDoS) attack** A DoS attack that is distributed across many hosts under centralized control of an attacker, all targeting the same victim.

**distributed WLC deployment** A wireless design based on distributing multiple controllers within the network. Each of the controllers commonly supports a relatively small number of users.

**distribution layer** In a campus LAN design, the switches that connect to access layer switches as the most efficient means to provide connectivity from the access layer into the other parts of the LAN.

**distribution link** In campus LAN design, a link that connects a distribution switch to an access switch.

**distribution system (DS)** The wired Ethernet that connects to an AP and transports traffic between a wired and wireless network.

**DNS** Domain Name System. An application layer protocol used throughout the Internet for translating hostnames into their associated IP addresses.

**DNS server** An application acting as a server for the purpose of providing name resolution services per the Domain Name System (DNS) protocol and worldwide system.

**domain-specific language** A generic term that refers to an attribute of different languages within computing, for languages created for a specific purpose (domain) rather than a general-purpose language like Python or JavaScript.

**Dynamic ARP Inspection (DAI)** A security feature in which a LAN switch filters a subset of incoming ARP messages on untrusted ports, based on a comparison of ARP, Ethernet, and IP header fields to data gathered in the IP DHCP Snooping binding table and found in any configured ARP ACLs.

# E

**egress tunnel router (ETR)** With LISP, a node at the end of a tunnel that receives an encapsulated message and then de-encapsulates the message.

**E-LAN** A specific carrier/Metro Ethernet service defined by MEF (MEF.net) that provides a service much like a LAN, with two or more customer sites connected to one E-LAN service in a full mesh so that each device in the E-LAN can send Ethernet frames directly to every other device.

**E-Line** A specific carrier/metro Ethernet service defined by MEF (MEF.net) that provides a point-to-point topology between two customer devices, much as if the two devices were connected using an Ethernet crossover cable.

**embedded wireless controller (EWC)** A WLC function that is co-located within an AP.

**embedded wireless controller (EWC) deployment** A wireless network design that places a WLC in the access layer, co-located with a LAN switch stack, near the APs it controls.

**enable secret** A reference to the password configured on the **enable secret** *pass-value* command, which defines the password required to reach enable (privileged) mode.

**error detection** The process of discovering whether a data-link level frame was changed during transmission. This process typically uses a Frame Check Sequence (FCS) field in the data-link trailer.

**error disabled (err-disable)** An interface state on LAN switches that can be the result of one of many security violations.

**error recovery** The process of noticing when some transmitted data was not successfully received and resending the data until it is successfully received.

**Ethernet access link** A WAN access link (a physical link between a service provider and its customer) that happens to use Ethernet.

**Ethernet WAN** A general and informal term for any WAN service that uses Ethernet links as the access link between the customer and the service provider.

**exploit** A means of taking advantage of a vulnerability to compromise something.

**extended access list** A list of IOS **access-list** global configuration commands that can match multiple parts of an IP packet, including the source and destination IP address and TCP/UDP ports, for the purpose of deciding which packets to discard and which to allow through the router.

**extended service set (ESS)** Multiple APs that are connected by a common switched infrastructure.

**extended service set identifier (ESSID)** The SSID used consistently throughout an ESS.

# F

**fabric** In SDA, the combination of overlay and underlay that together provide all features to deliver data across the network with the desired features and attributes.

**fabric edge node** In SDA, a switch that connects to endpoint devices.

**fiber Internet** A general term for any Internet access technology that happens to use fiber-optic cabling. It often uses Ethernet protocols on the fiber link.

**firewall** A device that forwards packets between the less secure and more secure parts of the network, applying rules that determine which packets are allowed to pass and which are not.

**flash memory** A type of read/write permanent memory that retains its contents even with no power applied to the memory and that uses no moving parts, making the memory less likely to fail over time.

**FlexConnect mode** An AP mode tailored for remote sites. Wireless traffic flows to and from a controller if the AP's CAPWAP tunnel is up, or is locally switched if the tunnel is down.

**flow control** The process of regulating the amount of data sent by a sending computer toward a receiving computer. Several flow control mechanisms exist, including TCP flow control, which uses windowing.

**FTP** File Transfer Protocol. An application protocol, part of the TCP/IP protocol stack, used to transfer files between network nodes. FTP is defined in RFC 959.

**FTP control connection** A TCP connection initiated by an FTP client to an FTP server for the purpose of sending FTP commands that direct the activities of the connection.

**FTP data connection** A TCP connection created by an FTP client and server for the purpose of transferring data.

**FTPS** FTP Secure. Common term for FTP over TLS.

**full mesh** From a topology perspective, any topology that has two or more devices, with each device being able to send frames to every other device.

# G

**Generative AI** Also Strong AI; encompasses artificial intelligence systems capable of learning, reasoning, and making decisions similar to human cognition, often with the capability to create new content, ideas, or solutions beyond its initial training data.

**Git** An open-source version control application, widely popular for version control in software development and for other uses, like managing network device configurations.

**GitHub** A web-based platform for version control and collaboration, facilitating hosting, sharing, and managing of software projects using the Git version control system.

**GLBP active virtual forwarder (AVF)** A role implemented by all routers in a GLBP group, listening for frames sent to a unique virtual MAC address, so it can act as one of several active default routers in the group.

**GLBP active virtual gateway (AVG)** A role implemented by one router in a GLBP group, replying to ARP requests on behalf of the group's VIP, so that load balancing occurs.

**gratuitous ARP** An ARP Reply not sent as a reaction to an ARP request message, but rather as a general announcement informing other hosts of the values of the sending (origin) host's addresses.

# H

**host (context: DC)** In a virtualized server environment, the term used to refer to one physical server that is running a hypervisor to create multiple virtual machines.

**HSRP preemption** A configuration setting that dictates whether an HSRP router, when it initializes with HSRP, can immediately take over the HSRP active role (preemption) if it has a higher priority than the currently active router.

**HSRP priority** A configuration setting from 0 through 255 that impacts the choice of HSRP active state, with the highest priority router chosen as active.

**HTTP** Hypertext Transfer Protocol. The protocol used by web browsers and web servers to transfer files, such as text and graphic files.

**HTTP/1.0** A version of the HTTP protocol, published as an RFC in the mid 1990s. It uses TCP, expects URLs that begin with http, implying a default well-known server port of 80. It allows the use of secure HTTP as well.

**HTTP/1.1** A version of the HTTP protocol, published as an RFC in the mid 1990s. It uses TCP, expects URLs that begin with http, implying a default well-known server port of 80. It allows the use of secure HTTP as well.

**HTTP/2.0** A version of the HTTP protocol, published as an RFC in the mid 2010s. The protocol improved HTTP application processes to increase overall performance in the end-user experience. It supports using Secure HTTP (with TLS) or not.

**HTTP/3.0** A radically different HTTP version published as an RFC in 2022 and created by Google. It improved HTTP application processes plus changed the transport layer protocols, both to increase overall performance in the end-user experience. It uses the QUIC transport layer, which uses UDP (not TCP) and always includes TLS.

**hub and spoke** From a topology perspective, any topology that has a device that can send messages to all other devices (the hub), with one or more spoke devices that can send messages only to the hub. Also called point-to-multipoint.

**hypervisor** Software that runs on server hardware to create the foundations of a virtualized server environment primarily by allocating server hardware components like CPU core/threads, RAM, disk, and network to the VMs running on the server.

# I

**Imperative Model** A method of describing IT automation as a series of tasks, akin to a script or program, with the expectation that running the script will configure the devices to have the desired configuration.

**imperative policy model** A term that describes the approach in traditional networks in which the engineer chooses configuration settings for each control and data plane protocol (the imperative commands) that dictate specifically how the devices act. This model acts in contrast to the newer declarative policy model and intent-based networking (IBN).

**independent basic service set (IBSS)** An impromptu wireless network formed between two or more devices without an AP or a BSS; also known as an ad hoc network.

**Infrastructure as Code (IAC)** A practice that involves managing and provisioning computing infrastructure through machine-readable definition files, enabling automation, consistency, and scalability in infrastructure deployment and management processes.

**Infrastructure as a Service (IaaS)** A cloud service in which the service consists of a virtual machine that has defined computing resources (CPUs, RAM, disk, and network) and may or may not be provided with an installed OS.

**infrastructure mode** The operating mode of an AP that is providing a BSS for wireless clients.

**inside global** For packets sent to and from a host that resides inside the trusted part of a network that uses NAT, a term referring to the IP address used in the headers of those packets when those packets traverse the global (public) Internet.

**inside local** For packets sent to and from a host that resides inside the trusted part of a network that uses NAT, a term referring to the IP address used in the headers of those packets when those packets traverse the enterprise (private) part of the network.

**intent-based networking (IBN)** An approach to networking in which the system gives the operator the means to express business intent, with the networking system then determining what should be done by the network, activating the appropriate configuration, and monitoring (assuring) the results.

**intrusion prevention system (IPS)** A security function that examines more complex traffic patterns against a list of both known attack signatures and general characteristics of how attacks can be carried out, rating each perceived threat, and reacting to prevent the more significant threats. *See also* IPS.

**IOS File System (IFS)** A file system created by a Cisco device that uses IOS.

**IOS image** A file that contains the IOS.

**IP Precedence (IPP)** In the original definition of the IP header's Type of Service (ToS) byte, the first 3 bits of the ToS byte, used for marking IP packets for the purpose of applying QoS actions.

**IPS** *See* intrusion prevention system.

**IPsec** The term referring to the IP Security protocols, which is an architecture for providing encryption and authentication services, usually when creating VPN services through an IP network.

**IPsec transport mode** The process of encrypting the data of the original IP packet when using IPsec, while using the original packet's IP header, plus VPN headers, to encapsulate the encrypted data. Typically used with remote access IPsec VPNs.

**IPsec tunnel mode** The process of encrypting the entire original IP packet when using IPsec, which requires a new IP header, plus VPN headers, to encapsulate the encrypted original packet. Typically used with site-to-site IPsec VPNs.

# J

**Jinja2** A text-based language used to define templates, with text plus variables; used by Ansible for templates.

**jitter** The variation in delay experienced by successive packets in a single application flow.

**JSON (JavaScript Object Notation)** A popular data serialization language, originally used with the JavaScript programming language, and popular for use with REST APIs.

**JSON array** A part of a set of JSON text that begins and ends with a matched set of square brackets that contain a list of values.

**JSON object** A part of a set of JSON text that begins and ends with a matched set of curly brackets that contain a set of key:value pairs.

# K–L

**key:value pair** In software, one variable name (key) and its value, separated by a colon in some languages and data serialization languages.

**Large Language Models (LLM)** Advanced artificial intelligence systems capable of understanding and generating human-like text based on extensive training on vast amounts of textual data.

**leaf** In an ACI network design, a switch that connects to spine switches and to endpoints, but not to other leaf switches, so that the leaf can forward frames from an endpoint to a spine, which then delivers the frame to some other leaf switch.

**LISP** Locator/ID Separation Protocol. A protocol, defined in RFC 6830, that separates the concepts and numbers used to identify an endpoint (the endpoint identifier) versus identifying the location of the endpoint (routing locator).

**list variable** In applications, a single variable whose value is a list of values, rather than a simple value.

**LLDP** Link Layer Discovery Protocol. An IEEE standard protocol (IEEE 802.1AB) that defines messages, encapsulated directly in Ethernet frames so they do not rely on a working IPv4 or IPv6 network, for the purpose of giving devices a means of announcing basic device information to other devices on the LAN. It is a standardized protocol similar to Cisco Discovery Protocol (CDP).

**LLDP-MED** A group of endpoint-focused LLDP TLVs, defined as a group TIA standard TIA-1057. It includes TLVs to communicate voice and data VLANs to phones and to manage power levels with PoE.

**local username** A username (with matching password), configured on a router or switch. It is considered local because it exists on the router or switch, and not on a remote server.

**log message** A message generated by any computer, but including Cisco routers and switches, for which the device OS wants to notify the owner or administrator of the device about some event.

**loss** A reference to packets in a network that are sent but do not reach the destination host.

# M

**malware** Malicious software.

**Management Information Base (MIB)** The data structures defined by SNMP to define a hierarchy (tree) structure with variables at the leaves of the tree, so that SNMP messages can reference the variables.

**management plane** Functions in networking devices and controllers that control the devices themselves but that do not impact the forwarding behavior of the devices like control plane protocols do.

**man-in-the-middle attack** An attack where an attacker manages to position a machine on the network such that it is able to intercept traffic passing between target hosts.

**marking** The process of changing one of a small set of fields in various network protocol headers, including the IP header's DSCP field, for the purpose of later classifying a message based on that marked value.

**MD5 hash** A specific mathematical algorithm intended for use in various security protocols. In the context of Cisco routers and switches, the devices store the MD5 hash of certain passwords, rather than the passwords themselves, in an effort to make the device more secure.

**Media Access Control (MAC) layer** The lower of the two sublayers of the data-link layer defined by the IEEE. Synonymous with IEEE 802.3 for Ethernet LANs.

**Meraki Dashboard** A centralized cloud-based management platform that provides intuitive control and monitoring of Meraki networking devices, offering streamlined configuration, real-time analytics, and seamless network administration.

**mesh network** A network of APs used to cover a large area without the need for wired Ethernet cabling; client traffic is bridged from AP to AP over a backhaul network.

**Metro Ethernet** The original term used for WAN service that used Ethernet links as the access link between the customer and the service provider.

**MIB** *See* Management Information Base.

**mitigation technique** A method to counteract or prevent threats and malicious activity.

**MPLS** *See* Multiprotocol Label Switching.

**MPLS VPN** A WAN service that uses MPLS technology, with many customers connecting to the same MPLS network, but with the VPN features keeping each customer's traffic separate from others.

**MTU** Maximum transmission unit. The maximum packet size, in bytes, that a particular interface can handle.

**multifactor authentication** A technique that uses more than one type of credential to authenticate users.

**Multigig Ethernet** The common name for the 2.5GBase-T and 5.0GBase-T Ethernet standards, which, when released simultaneously, represented an option for UTP Ethernet at speeds of multiple gigabits between the then-defined standard speeds of 1 Gbps and 10 Gbps.

**Multiprotocol BGP (MPBGP)** A particular set of BGP extensions that allows BGP to support multiple address families, which when used to create an MPLS VPN service gives the SP the method to advertise the IPv4 routes of many customers while keeping those route advertisements logically separated.

**Multiprotocol Label Switching (MPLS)** A WAN technology used to create an IP-based service for customers, with the service provider's internal network performing forwarding based on an MPLS label rather than the destination IP address.

# N

**named access list** An ACL that identifies the various statements in the ACL based on a name rather than a number.

**Narrow AI** Also Weak AI. Refers to artificial intelligence designed and trained for specific tasks or a limited range of activities, lacking broad cognitive capabilities and adaptability beyond its predefined scope.

**NAT** Network Address Translation. A mechanism for reducing the need for globally unique IP addresses. NAT allows an organization with addresses that are not globally unique to connect to the Internet, by translating those addresses into public addresses in the globally routable address space.

**NAT overload** Another term for Port Address Translation (PAT). One of several methods of configuring NAT, in this case translating TCP and UDP flows based on port numbers in addition to using one or only a few inside global addresses.

**NBI** *See* northbound API.

**Network Management System (NMS)** A software platform that enables centralized monitoring, configuration, and administration of network infrastructure and devices to ensure optimal performance, security, and reliability.

**Network Time Protocol (NTP)** A protocol used to synchronize time-of-day clocks so that multiple devices use the same time of day, which allows log messages to be more easily matched based on their timestamps.

**Next-generation firewall (NGFW)** A firewall device with advanced features, including the ability to run many related security features in the same firewall device (IPS, malware detection, VPN termination), along with deep packet inspection with Application Visibility and Control (AVC) and the ability to perform URL filtering versus data collected about the reliability and risk associated with every domain name.

**Next-generation IPS (NGIPS)** An IPS device with advanced features, including the capability to go beyond a comparison to known attack signatures to also look at contextual data, including the vulnerabilities in the current network, the capability to monitor for new zero-day threats, with frequent updates of signatures from the Cisco Talos security research group.

**NMS** *See* Network Management System (NMS).

**nonoverlapping channels** Successive channel numbers in a band that each have a frequency range that is narrow enough to not overlap the next channel above or below.

**northbound API** In the area of SDN, a reference to the APIs that a controller supports that gives outside programs access to the services of the controller; for instance, to supply information about the network or to program flows into the network. Also called a northbound interface.

**northbound interface (NBI)** Another term for northbound API. *See also* northbound API.

**NTP client** Any device that attempts to use the Network Time Protocol (NTP) to synchronize its time by adjusting the local device's time based on NTP messages received from a server.

**NTP client/server mode** A mode of operation with the Network Time Protocol (NTP) in which the device acts as both an NTP client, synchronizing its time with some servers, and as an NTP server, supplying time information to clients.

**NTP server** Any device that uses Network Time Protocol (NTP) to help synchronize time-of-day clocks for other devices by telling other devices its current time.

**NTP synchronization** The process with the Network Time Protocol (NTP) by which different devices send messages, exchanging the devices' current time-of-day clock information and other data, so that some devices adjust their clocks to the point that the time-of-day clocks list the same time (often accurate to at least the same second).

# O

**on-demand self-service** One of the five key attributes of a cloud computing service as defined by NIST, referring to the fact that the consumer of the server can request the service, with the service being created without any significant delay and without waiting on human intervention.

**on-premises** An alternate term for private cloud. *See also* private cloud.

**OpenFlow** The open standard for Software-Defined Networking (SDN) as defined by the Open Networking Foundation (ONF), which defines the OpenFlow protocol as well as the concept of an abstracted OpenFlow virtual switch.

**Optical Multimode (OM)** The term used to refer to multimode fiber-optic cabling in various cabling standards.

**ordered data transfer** A networking function, included in TCP, in which the protocol defines how the sending host should number the data transmitted, defines how the receiving device should attempt to reorder the data if it arrives out of order, and specifies to discard the data if it cannot be delivered in order.

**outside global** With source NAT, the one address used by the host that resides outside the enterprise, which NAT does not change, so there is no need for a contrasting term.

**overlay** In SDA, the combination of VXLAN tunnels between fabric edge nodes as a data plane for forwarding frames, plus LISP for the control plane for the discovery and registration of endpoint identifiers.

# P

**partial mesh** A network topology in which more than two devices could physically communicate, but by choice, only a subset of the pairs of devices connected to the network is allowed to communicate directly.

**passive scanning** A technique used by a wireless client when it attempts to discover nearby APs by listening for their beacon frames.

**password guessing** An attack where a malicious user simply makes repeated attempts to guess a user's password.

**per-hop behavior (PHB)** The general term used to describe the set of QoS actions a device can apply to a message from the time it enters a networking device until the device forwards the message. PHBs include classification, marking, queuing, shaping, policing, and congestion avoidance.

**pharming** An attack that compromises name services to silently redirect users toward a malicious site.

**phishing** An attack technique that sends specially crafted emails to victims in the hope that the users will follow links to malicious websites.

**Platform as a Service (PaaS)** A cloud service intended for software developers as a development platform, with a variety of tools useful to developers already installed so that developers can focus on developing software rather than on creating a good development environment.

**PoE** Power over Ethernet. Both a generalized term for any of the standards that supply power over an Ethernet link, as well as a specific PoE standard as defined in the IEEE 802.3af amendment to the 802.3 standard.

**point of presence (PoP)** A term used for a service provider's (SP) perspective to refer to a service provider's installation that is purposefully located relatively near to customers, with several spread around major cities, so that the distance from each customer site to one of the SP's PoPs is short.

**point-to-point** From a topology perspective, any topology that has two and only two devices that can send messages directly to each other.

**point-to-point bridge** An AP configured to bridge a wired network to a companion bridge at the far end of a line-of-sight path.

**policing** A QoS tool that monitors the bit rate of the messages passing some point in the processing of a networking device, so that if the bit rate exceeds the policing rate for a period of time, the policer can discard excess packets to lower the rate.

**policing rate** The bit rate at which a policer compares the bit rate of packets passing through a policing function, for the purpose of taking a different action against packets that conform (are under) to the rate versus those that exceed (go over) the rate.

**port** (Multiple definitions) (1) In TCP and UDP, a number that is used to uniquely identify the application process that either sent (source port) or should receive (destination port) data. (2) In LAN switching, another term for switch interface.

**Port Address Translation (PAT)** A NAT feature in which one inside global IP address supports over 65,000 concurrent TCP and UDP connections.

**port number** A field in a TCP or UDP header that identifies the application that either sent (source port) or should receive (destination port) the data inside the data segment.

**port security** A Cisco switch feature in which the switch watches Ethernet frames that come in an interface (a port), tracks the source MAC addresses of all such frames, and takes a security action if the number of different such MAC addresses is exceeded.

**Power classification** With Power over Ethernet (PoE), the process by which a switch, once it detects a device wants power, discovers the amount of power as defined by a standardized set of power classes.

**Power detection** With Power over Ethernet (PoE), the process by which a switch discovers if the connected device wants to receive power over the link or not.

**Power over Ethernet (PoE)** Both a generalized term for any of the standards that supply power over an Ethernet link and a specific PoE standard as defined in the IEEE 802.3af amendment to the 802.3 standard.

**Power over Ethernet Plus (PoE+)** A specific PoE standard as defined in the IEEE 802.3at amendment to the 802.3 standard, which uses two wire pairs to supply power with a maximum of 30 watts as supplied by the PSE.

**power sourcing equipment (PSE)** With any Power over Ethernet standard, a term that refers to the device supplying the power over the cable, which is then used by the powered device (PD) on the other end of the cable.

**powered device (PD)** With any Power over Ethernet standard, a term that refers to the device that receives or draws its power over the Ethernet cable, with the power being supplied by the power sourcing equipment (PSE) on the other end of the cable.

**priority queue** In Cisco queuing systems, another term for a low latency queue (LLQ).

**private cloud** A cloud computing service in which a company provides its own IT services to internal customers inside the same company but by following the practices defined as cloud computing.

**private IP network** Any of the IPv4 Class A, B, or C networks as defined by RFC 1918, intended for use inside a company but not used as public IP networks.

**probe request** A technique used by a wireless client to discover nearby APs by actively requesting a response.

**provider** A plug-in that enables communication and interaction between Terraform and specific infrastructure platforms or services, facilitating the management and provisioning of resources within those environments through Terraform configuration files.

**provider edge (PE)** A term used by service providers, both generally and also specifically in MPLS VPN networks, to refer to the SP device in a point of presence (PoP) that connects to the customer's network and therefore sits at the edge of the SP's network.

**public cloud** A cloud computing service in which the cloud provider is a different company than the cloud consumer.

**pull model** With configuration management tools, a practice by which an agent representing the device requests configuration data from the centralized configuration management tool, in effect pulling the configuration to the device.

**push model** With configuration management tools, a practice by which the centralized configuration management tool software initiates the movement of configuration from that node to the device that will be configured, in effect pushing the configuration to the device.

# Q–R

**Quality of Service (QoS)** The performance of a message, or the messages sent by an application, in regard to the bandwidth, delay, jitter, or loss characteristics experienced by the message(s).

**queuing** The process by which networking devices hold packets in memory while waiting on some constrained resource; for example, when waiting for the outgoing interface to become available when too many packets arrive in a short period of time.

**QUIC** The name (not an acronym) for a transport layer protocol that improves overall performance of transferring objects over a network, in comparison to TCP. It uses UDP and integrates TLS into its connection setup tasks.

**RADIUS** A security protocol often used for user authentication, including being used as part of the IEEE 802.lx messages between an 802.lx authenticator (typically a LAN switch) and a AAA server.

**RAM** Random-access memory. A type of volatile memory that can be read and written by a microprocessor.

**rapid elasticity** One of the five key attributes of a cloud computing service as defined by NIST, referring to the fact that the cloud service reacts to requests for new services quickly, and it expands (is elastic) to the point of appearing to be a limitless resource.

**read-only community** An SNMP community (a value that acts as a password), defined on an SNMP agent, which then must be supplied by any SNMP manager that sends the agent any messages asking to learn the value of a variable (like SNMP Get and GetNext requests).

**read-write community** An SNMP community (a value that acts as a password), defined on an SNMP agent, which then must be supplied by any SNMP manager that sends the agent any messages asking to set the value of a variable (like SNMP Set requests).

**reassociation request** An 802.11 frame that a roaming wireless client sends to an AP to request that its existing association be moved to a new AP.

**reconnaissance attack** An attack crafted to discover as much information about a target organization as possible; the attack can involve domain discovery, ping sweeps, port scans, and so on.

**recursive DNS server** A DNS server that, when asked for information it does not have, performs a repetitive (recursive) process to ask other DNS servers in sequence, hoping to find the DNS server that knows the information.

**reflection attack** An attack that uses spoofed source addresses so that a destination machine will reflect return traffic to the attack's target; the destination machine is known as the reflector.

**remote access VPN** A VPN for which one endpoint is a user device, such as a phone, tablet, or PC, typically created dynamically, and often using TLS. Also called a client VPN.

**repeater** A device that repeats or retransmits signals it receives, effectively expanding the wireless coverage area.

**Representational State Transfer (REST)** A type of API that allows two programs that reside on separate computers to communicate, with a set of six primary API attributes as defined early in this century by its creator, Roy Fielding. The attributes include client/server architecture, stateless operation, cachability, uniform interfaces, layered, and code-on-demand.

**resource pooling** One of the five key attributes of a cloud computing service as defined by NIST, referring to the fact that the cloud provider treats its resources as a large group (pool) of resources that its cloud management systems then allocate dynamically based on self-service requests by its customers.

**REST** *See* Representational State Transfer.

**REST API** Any API that uses the rules of Representational State Transfer (REST).

**roaming** The process a wireless client uses to move from one AP to another as it changes location.

**round robin** A queue scheduling algorithm in which the scheduling algorithm services one queue, then the next, then the next, and so on, working through the queues in sequence.

# S

**SBI** *See* Southbound API.

**scalable group tag (SGT)** In SDA, a value assigned to the users in the same security group.

**Secure HTTP (HTTP over TLS)** The IETF standard that defines how to use TLS to add security features such as server authentication and message encryption to HTTP/2 and earlier versions of HTTP.

**segment** (Multiple definitions) (1) In TCP, a term used to describe a TCP header and its encapsulated data (also called an L4PDU). (2) Also in TCP, the set of bytes formed when TCP breaks a large chunk of data given to it by the application layer into smaller pieces that fit into TCP segments. (3) In Ethernet, either a single Ethernet cable or a single collision domain (no matter how many cables are used).

**sender hardware address** In both an ARP request and reply message, the field intended to be used to list the sender (origin) device's hardware address, typically an Ethernet LAN address.

**sender IP address** In both an ARP request and reply message, the field intended to be used to list the sender (origin) device's IP address.

**sender protocol address** In both an ARP request and a reply message, the formal term the field intended to be used to list the sender (origin) device's network layer address.

**service provider (SP)** A company that provides a service to multiple customers. Used most often to refer to providers of private WAN services and Internet services. *See also* Internet service provider.

**Service Set Identifier (SSID)** A text string that is used to identify a wireless network.

**shaping** A QoS tool that monitors the bit rate of the messages exiting networking devices, so that if the bit rate exceeds the shaping rate for a period of time, the shaper can queue the packets, effectively slowing down the sending rate to match the shaping rate.

**shaping rate** The bit rate at which a shaper compares the bit rate of packets passing through the shaping function, so that when the rate is exceeded, the shaper enables the queuing of packets, resulting in slowing the bit rate of the collective packets that pass through the shaper, so the rate of bits getting through the shaper does not exceed the shaping rate.

**shared key** A reference to a security key whose value is known (shared) by both the sender and receiver.

**Simple Network Management Protocol (SNMP)** An Internet standard protocol for managing devices on IP networks. It is used mostly in network management systems to monitor network-attached devices for conditions that warrant administrative attention.

**site-to-site VPN** The mechanism that allows all devices at two different sites to communicate securely over some unsecure network like the Internet, by having one device at each site perform encryption/decryption and forwarding for all the packets sent between the sites.

**sliding windows** For protocols such as TCP that allow the receiving device to dictate the amount of data the sender can send before receiving an acknowledgment—a concept called a *window*—a reference to the fact that the mechanism to grant future windows is typically just a number that grows upward slowly after each acknowledgment, sliding upward.

**SNMP** *See* Simple Network Management Protocol.

**SNMP agent** Software that resides on the managed device and processes the SNMP messages sent by the Network Management Station (NMS).

**SNMP community** A simple password mechanism in SNMP in which either the SNMP agent or manager defines a community string (password), and the other device must send that same password value in SNMP messages, or the messages are ignored. *See also* read-only community, read-write community, and notification community.

**SNMP Get** Message used by SNMP to read from variables in the MIB.

**SNMP Inform** An unsolicited SNMP message like a Trap message, except that the protocol requires that the Inform message needs to be acknowledged by the SNMP manager.

**SNMP manager** Typically a Network Management System (NMS), with this term specifically referring to the use of SNMP and the typical role of the manager, which retrieves status information with SNMP Get requests, sets variables with the SNMP Set requests, and receives unsolicited notifications from SNMP agents by listening for SNMP Trap and Notify messages.

**SNMP Set** SNMP message to set the value in variables of the MIB. These messages are the key to an administrator configuring the managed device using SNMP.

**SNMP Trap** An unsolicited SNMP message generated by the managed device, and sent to the SNMP manager, to give information to the manager about some event or because a measurement threshold has been passed.

**SNMPv2c** A variation of the second version of SNMP. SNMP Version 2 did not originally support communities; the term *SNMPv2c* refers to SNMP version 2 with support added for SNMP communities (which were part of SNMPvl).

**SNMPv3** The third version of SNMP, with the notable addition of several security features as compared to SNMPv2c, specifically message integrity, authentication, and encryption.

**social engineering** Attacks that leverage human trust and social behaviors to divulge sensitive information.

**Software as a Service (SaaS)** A cloud service in which the service consists of access to working software, without the need to be concerned about the details of installing and maintaining the software or the servers on which it runs.

**Software-Defined Access** Cisco's intent-based networking (IBN) offering for enterprise networks.

**Software-Defined Networking (SDN)** A branch of networking that emerged in the marketplace in the 2010s characterized by the use of a centralized software controller that takes over varying amounts of the control plane processing formerly done inside networking devices, with the controller directing the networking elements as to what forwarding table entries to put into their forwarding tables.

**SOHO** A classification of a business site with a relatively small number of devices, sometimes in an employee office in their home.

**Source NAT** The type of Network Address Translation (NAT) used most commonly in networks (as compared to destination NAT), in which the source IP address of packets entering an inside interface is translated.

**southbound API** In the area of SDN, a reference to the APIs used between a controller and the network elements for the purpose of learning information from the elements and for programming (controlling) the forwarding behavior of the elements. Also called a southbound interface.

**southbound interface** Another term for southbound API. *See also* southbound API.

**spear phishing** Phishing that begins with research about a related group of people so that the attack uses messaging that appears more legitimate by using those researched facts.

**spine** In an ACI network design for a single site, a switch that connects to leaf switches only, for the purpose of receiving frames from one leaf switch and then forwarding the frame to some other leaf switch.

**split-MAC architecture** A wireless AP strategy based around the idea that normal AP functions are split or divided between a wireless LAN controller and lightweight APs.

**spoofing attack** A type of attack in which parameters such as IP and MAC addresses are spoofed with fake values to disguise the sender.

**standalone mode** The operational mode used by a FlexConnect AP when the path back to its controller is down and not working. In this mode, all wireless traffic is switched locally, preserving local connectivity while the AP is isolated from its controller.

**standard access list** A list of IOS global configuration commands that can match only a packet's source IP address for the purpose of deciding which packets to discard and which to allow through the router.

**star topology** A network topology in which endpoints on a network are connected to a common central device by point-to-point links.

**stateless** A protocol or process that does not use information stored from previous transactions to perform the current transaction.

**station (STA)** An 802.11 client device that is associated with a BSS.

**syslog server** A server application that collects syslog messages from many devices over the network and provides a user interface so that IT administrators can view the log messages to troubleshoot problems.

# T

**TCAM** *See* ternary content-addressable memory.

**ternary content-addressable memory (TCAM)** A type of physical memory, either in a separate integrated circuit or built into an ASIC, that can store tables and then be searched against a key, such that the search time happens quickly and does not increase as the size of the table increases. TCAMs are used extensively in higher-performance networking devices as the means to store and search forwarding tables in Ethernet switches and higher-performance routers.

**Terraform** An open-source infrastructure as code tool used for provisioning and managing cloud, on-premises, and hybrid infrastructure resources through declarative configuration files.

**TFTP** Trivial File Transfer Protocol. An application protocol that allows files to be transferred from one computer to another over a network, but with only a few features, making the software require little storage space.

**threat** An actual potential to use an exploit to take advantage of a vulnerability.

**Transport Layer Security (TLS)** A security standard that replaced the older Secure Sockets Layer (SSL) protocol, providing functions such as authentication, confidentiality, and message integrity over reliable in-order data streams like TCP.

**trojan horse** Malware that is hidden and packaged inside other legitimate software.

**trusted port** With both the DHCP Snooping and Dynamic ARP Inspection (DAI) switch features, the concept and configuration setting that tells the switch to allow all incoming messages of that respective type, rather than to consider the incoming messages (DHCP and ARP, respectively) for filtering.

**two-tier design** *See* collapsed core design.

# U

**underlay** In SDA, the network devices and links that create basic IP connectivity to support the creation of VXLAN tunnels for the overlay.

**Unified Computing System (UCS)** The Cisco brand name for its server hardware products.

**Universal Power over Ethernet (UPoE)** A specific PoE standard as defined in the IEEE 802.3bt amendment to the 802.3 standard, which uses four wire pairs to supply power with a maximum of 60 watts as supplied by the PSE.

**Universal Power over Ethernet Plus (UPoE+)** A specific PoE standard as defined in the IEEE 802.3bt amendment to the 802.3 standard, which uses four wire pairs to supply power with a maximum of 100 watts as supplied by the PSE.

**untrusted port** With both the DHCP Snooping and Dynamic ARP Inspection (DAI) switch features, the concept and configuration setting that tells the switch to analyze each incoming message of that respective type (DHCP and ARP) and apply some rules to decide whether to discard the message.

**UPoE** *See* Universal Power over Ethernet (UPoE).

**URI** Uniform Resource Identifier. The formal and correct term for the formatted text used to refer to objects in an IP network. This text is commonly called a URL or a web address. For example, http://www.certskills.com/config-labs is a URI that identifies the protocol (HTTP), hostname (www.certskills.com), and web page (config-labs).

**URI parameters** *See* URI query (parameters).

**URI path (resource)** In a URI, the part that follows the first /, up to the query field (which begins with a ?), which identifies the resource in the context of a server.

**URI query (parameters)** In a URI, the part that follows the first ?, which provides a place to list variable names and values as parameters.

**URI resource** *See* URI path (resource).

**username secret** A reference to the password configured on the **username** *name* **secret** *pass-value* command, which defines a username and an encoded password, used to build a local username/password list on the router or switch.

**UTP Cable Category** A set of standards from the TIA and ANSI that defines the electrical characteristics of UTP cabling under various tests. Ethernet standards then refer to these UTP cable categories to define the minimum category needed to support the Ethernet standard at stated distances.

# V

**violation mode** In port security, a configuration setting that defines the specific set of actions to take on a port when a port security violation occurs. The modes are shutdown, restrict, and protect.

**virtual CPU (vCPU)** In a virtualized server environment, a CPU (processor) core or thread allocated to a virtual machine (VM) by the hypervisor.

**Virtual IP address (VIP)** Used with first hop redundancy protocols, an address, referenced by hosts as their default router, that can move between multiple routers to support failover of the default router function from one router to another.

**virtual machine** An instance of an operating system, running on server hardware that uses a hypervisor to allocate a subset of the server hardware (CPU, RAM, disk, and network) to that VM.

**virtual NIC (vNIC)** In a virtualized server environment, a network interface card (NIC) used by a virtual machine, which then connects to some virtual switch (vSwitch) running on that same host, which in turn connects to a physical NIC on the host.

**virtual private network (VPN)** A set of security protocols that, when implemented by two devices on either side of an unsecure network such as the Internet, can allow the devices to send data securely. VPNs provide privacy, device authentication, anti-replay services, and data integrity services.

**Virtual routing and forwarding (VRF)** Virtual routing and forwarding instance. A feature of routers and Layer 3 switches that makes one router act as multiple routers by assigning interfaces and routing protocol neighbors to specific VRFs, with related routes landing in the associated VRF-unique routing table.

**virtual switch (vSwitch)** A software-only virtual switch inside one host (one hardware server), to provide switching features to the virtual machines running on that host.

**virus** Malware that injects itself into other applications and then propagates through user intervention.

**VPN** *See* virtual private network.

**VPN client** Software that resides on a PC, often a laptop, so that the host can implement the protocols required to be an endpoint of a VPN.

**vty ACL** An IP ACL enabled for inbound SSH and Telnet connections to a router or for outbound requests per the **ssh** and **telnet** commands issued by a user who is already connected to the router using SSH or Telnet.

**vulnerability** A weakness that can be used to compromise security.

**VXLAN** Virtual Extensible LAN. A flexible encapsulation protocol used for creating tunnels (overlays).

# W

**watering hole attack** An attack where a site frequently visited by a group of users is compromised; when the target users visit the site, they will be infected with malware, but other users will not.

**web server** Software, running on a computer, that stores web pages and sends those web pages to web clients (web browsers) that request the web pages.

**whaling** Spear phishing that targets high-profile individuals.

**wildcard mask** The mask used in Cisco IOS ACL commands and OSPF and EIGRP **network** commands.

**wireless LAN controller (WLC)** A device that cooperates with wireless lightweight access points (LWAP) to create a wireless LAN by performing some control functions for each LWAP and forwarding data between each LWAP and the wired LAN.

**workgroup bridge (WGB)** An AP that is configured to bridge between a wired device and a wireless network. The WGB acts as a wireless client.

**worm** Malware that propagates from one system to another, infecting as it goes, all autonomously.

**write community** *See* read-write community.

# X–Y–Z

**XML (eXtensible Markup Language)** A markup language that helps enable dynamic web pages; also useful as a data serialization language.

**YAML (YAML Ain't Markup Language)** A data serialization language that can be easily read by humans; used by Ansible.

**Zero Touch Provisioning (ZTP)** An automated deployment process that enables the remote configuration and setup of network devices without requiring manual intervention, allowing for seamless and efficient network deployment at scale.



# Index

## Numerics


2.4-GHz band, 17, 20, 641
2.5GBASE-T, 404–405, 641
4G/5G, 428–429, 641
5GBASE-T, 404–405
5-GHz band, 17, 641
6-GHz band, 70, 641
10BASE-T, 404, 641
10GBASE-T, 404, 641
40GBASE-T, 404, 641
100BASE-T, 404, 641
802.1Q header, marking, 333–334
802.1x, 47–48
802.11, 7–8. *See also* wireless networks
  amendments, 18–19
  beacon, 8
  Wi-Fi generational names, 20
1000BASE-SX, 641
1000BASE-T, 404, 641


## A


AAA (authentication, authorization, and accounting), 198, 642, 644
  accounting, 198
  authentication, 198
  authorization, 198
  RADIUS, 199–200
  TACACS+, 199–200
access control, physical, 200
access interface, 642
access layer, 494, 502, 642
access link, 417, 642
access switch, 399, 402
access-class command, 171–173
access-list command, 121–122, 126, 130–131. *See also* IP ACL (access control list)
  implicit deny, 124
  log keyword, 129
  permit any, 124
  reverse engineering from ACL to address range, 131–133
  syntax, 125
accounting, 198, 642
ACE (access control entry), 511, 642
ACI (Application Centric Infrastructure), 446, 483–484, 643
  APIC (Application Policy Infrastructure Controller), 487
  EPG (endpoint group), 486
  operating model with intent-based networking, 486–488
  spine and leaf design, 484–485
ACK flag, 100
ACL (access control list), 329, 642. *See also* IP ACL (access control list)
  ARP, 253
  persistence, 642
  resequencing, 642
  sequence number, 642

**active mode, FTP, 386–387**
**active/standby model, HSRP (Hot Standby Router Protocol), 356**
**ad hoc wireless network, 14, 642**
**administrative distance, 642**
**AES (Advanced Encryption Standard), 51**
**AF (Assured Forwarding), 336**
**agent**
-based architecture, 643
SNMP, 370
**agentless architecture, 563, 643**
**AI (artificial intelligence), 517, 643**
for automation, 518
ChatGPT, 518–523
generative, 517–518
ML (machine learning), 518
narrow, 517–518
**AI Ops, 523–524, 643**
**AireOS WLC, 61–63, 79**
configuring the WLAN, 81–83
configuring WLAN security, 83–84
create a new WLAN, 80–81
creating a dynamic interface, 79–80
**algorithm**
AES, 51
CBC-MAC (Cipher Block Chaining Message Authentication Code), 51
changing the encoding type for enable secret password command, 209–210
hash, 209
RC4 cipher, 47
scheduling, 338–339
SHA-256, 209
**amendments, IEEE 802.11, 18–19**
**amplification attack, 191, 643**
**ANSI (American National Standards Institute), 404**
**Ansible, 562–563, 643**
**antenna, 42**
**AP (access point), 642.** ***See also*** **WLC (wireless LAN controller)**
association request/response, 9, 19
authentication, 44
autonomous, 24–25, 27, 58
beacon frame, 8, 66
BSA (basic service area), 8
BSS (basic service set), 8
Cisco
*FlexConnect Mode, 32, 36–37*
*modes, 35–36*
Cisco Meraki, 26
fake, 44–46
group key, 45
IBSS (independent basic service set), 13–14
infrastructure mode, 8
lightweight, 28
management platform, 26
mesh, 17
multigig Ethernet, 406
radios, 19
repeater mode, 14
roaming, 13
SOHO (small office/home office), 408
supporting multiple SSIDs on, 11–12
**API (application programming interface), 478, 489–491, 643**
authentication, 537–539
data serialization language, 541–542
development environment tool, 536–541
documentation, 536
Java, 480

need for data modeling language, 542–544
RESTful, 480–481, 510, 528–529
*cacheable, 530*
*client/server architecture, 529*
*stateless operation, 530*
*URIs, 534–536*
southbound, 664
**APIC (Application Policy Infrastructure Controller), 487, 643**
**application/s**
batch, 326
data, 325–326
Postman, 537–541
signatures, 332
voice and video, 327–328
well-known port numbers, 99
WWW (World Wide Web), 98
**architecture**
agentless, 563
client/server, 529
software-defined, 481
wireless
*autonomous AP, 24–25*
*cloud-based AP, 26–27*
*control plane, 27*
*data plane, 27*
*split-MAC, 28–32, 36–37*
**ARP, 250–251**
ACLs, 253
gratuitous, 251–252
message fields, 251
reply, 643
**array, JSON (JavaScript Object Notation), 547–549**
**ASIC (application-specific integrated circuit), 476, 643**
**association request/response, 9, 19, 644**
**attack/s, 191**
brute-force, 86, 196, 644
buffer overflow, 194, 644
DHCP, 241–242, 248
dictionary, 196
DoS (denial of service), 212
pharming, 195
phishing, 195
reconnaissance, 193–194
smishing, 195
spear phishing, 195
spoofing, 188–189
*amplification, 191*
*denial-of-service, 189–190*
*distributed denial-of-service, 190*
*man-in-the-middle, 44, 191–193*
*reflection, 191*
vishing, 195
watering hole, 195
whaling, 195
**authentication. *See also* password/s**
AAA, 198
API, 537–539
multifactor, 199
wireless, 43
*802.1x/EAP, 47–48*
*AP (access point), 44*
*client, 43–44*
*EAP-FAST (EAP Flexible Authentication by Secure Tunneling), 48–49*
*EAP-TLS (EAP Transport Layer Security), 50*
*LEAP (Lightweight EAP), 48–49*
*open, 46*
*PEAP (Protected EAP), 49*
*WEP (Wired Equivalent Privacy), 47*

**authorization, 198, 644**
**automation, 518, 523**
configuration, 561–562
data center, 484
impact on network management, 489–491
**autonomous AP, 24–25, 27, 58, 644**
**AVC (Application Visibility and Control), 217**
**AVF (active virtual forwarder), 364**
**AVG (active virtual gateway), 364**

## B

**band, 20, 644. *See also* channel**
2.4-GHz, 17
5-GHz, 17
6-GHz, 70
channels, 17
**bandwidth, 324–325, 644**
**baseline, 524**
**batch traffic, 326**
**beacon, 802.11, 8**
**beacon frame, 66, 644**
**bidirectional communication, wireless network, 7**
**binary wildcard mask, 123–124**
**biometric credentials, 197**
**bridge**
outdoor, 16
point-to-point bridged link, 16
workgroup, 15–16
**brute-force attack, 86, 196, 644**
**BSA (basic service area), 8**
**BSS (basic service set), 8–9, 24, 644**
DS (distribution system), 10–12
independent, 13–14
traffic flows, 9
**BSSID (basic service set identifier), 8–9**
**buffer overflow attack, 194, 644**
**building, ACLs, 155**

## C

**CA (certificate authority), 49**
**cable**
CAT 5E, 405–406
CAT 6, 405
fiber-optic, 406, 429
UTP (unshielded twisted-pair), 403–405
**cable Internet, 427–428**
**CAC (call admission control), 340–341**
**cacheable, 644**
**campus LAN, 398–399. *See also* SD-Access; two-tier campus LAN**
access switches, 399
distribution switches, 399–400
Ethernet UTP links at the access layer, 403–405
fiber uplinks, 406–407
multigig Ethernet, 405–406
three-tier, 400–402
two-tier, 399–400
*full mesh, 403*
*partial mesh, 403*
*uplinks, 403*
**CAPWAP (Control and Provisioning of Wireless Access Points), 29, 36, 644**
**CAT 5E cable, 405–406, 645**
**CAT 6 cable, 645**
**CBC-MAC (Cipher Block Chaining Message Authentication Code), 51**
**CBWFQ (Class-Based Weighted Fair Queuing), 339**
**CCMP (Counter/CBC-MAC Protocol), 51**

**CCNA exam 200–301**
adjustments for your second attempt, 595–596
advice on how to answer questions, 590–592
gap analysis, 589–590
other study tasks, 596
practice exam events, 586–587
practice exams in the CCNA premium edition, 592
practice questions, 585
preparation, 583–584
*24 hours before your exam, 582–583*
*30 minutes before your exam, 583*
*one week before exam, 582*
question types, 578–580
review, 584–585
scoring, 587
self-assessment suggestions, 587–589
time check method, 581–582
time management, 580–581
topic order, 607–617
updates, 572–576
**CDP (Cisco Discovery Protocol), 283, 645**
configuration, 286
hold time, 287
send time, 287
show commands, 283
verification, 286–287
**cdp enable command, 286**
**cdp timer command, 287**
**CE (customer edge), 424**
**cell, 645**
**centralized architecture, 477**
**centralized configuration files, 555–557**
**centralized control plane, 645**
**centralized WLC deployment, 32**
**certificate**
digital, 197
X.509, 30
**channel, 17, 18, 20, 645**
**Chat Ops, 524, 645**
**ChatGPT, 518–523**
**CIDR (Classless Interdomain Routing), 300, 302, 645**
block, 301
Cisco Catalyst Center, as network management platform, 514–515
**CIR (committed information rate), 343**
**Cisco 8000V router, 461**
**Cisco AnyConnect Secure Mobility Client, 645**
**Cisco AP**
connections, 58–59
FlexConnect Mode, 36–37
modes, 35–36
OfficeExtend, 37
**Cisco ASA (Adaptive Security Appliance), 216**
**Cisco Catalyst 8000V, 645**
**Cisco Catalyst Center, 494, 496–497, 509–510, 645**
differences with traditional management, 516–517
GUIs, 515
as network management platform, 514–515
RESTful API call to, 536–541
scalable groups, 510–512
similarities to traditional management, 515–516
topology map, 515
**Cisco DNA (Digital Network Architecture), 494**

**Cisco Meraki, 26**
**Cisco Prime Infrastructure, 645**
**Cisco SD-Access, 484, 494, 512–513, 645**
Cisco Catalyst Center, 496–497, 509–510
*differences with traditional management, 516–517*
*GUIs, 515*
*as network management platform, 514–515*
*scalable groups, 510–512*
*similarities to traditional management, 515–516*
*topology map, 515*
fabric, 497–498
*edge node, 506*
*routed access layer design, 502*
host mobility, 498
ITR (ingress tunnel router), 506–508
overlay, 497–498, 503
*LISP for discovery and location, 505–509*
*VXLAN tunnels, 504–505*
SGT (scalable group tag), 513–514
underlay, 497–499
*supported hardware, 500*
*using existing gear, 499–500*
*using new gear, 501–503*
**Cisco Secure Client, 646**
**Cisco server hardware, 440–441**
**Cisco Talos Intelligence Group, 219**
**classification, 329–330, 646**
NBAR2 (next-generation Network Based Application Recognition), 331–332
on routers, 331–332
VRF (virtual routing and forwarding), 454–456
**clear ip nat translation command, 305, 313–314**
**client, NTP (Network Time Protocol), 281**
**clock set command, 279**
**clock summer-time command, 279**
**cloud/cloud computing, 448–449**
attributes, 448–449
-based AP architecture, 26–27, 646
-based WLC deployment, 32, 646
enterprise WAN connection to the, 456
*accessing public cloud services using the Internet, 456–457*
*using Internet to connect to the public cloud, 457–458*
IaaS (Infrastructure as a Service), 451–452
management, 460–464, 646
PaaS (Platform as a Service), 453–454
private, 449–450
public, 450–451, 460
*connecting with private WAN, 458–459*
*intercloud exchange, 459–460*
SaaS (Software as a Service), 452–453
service catalog, 449–450, 646
“as a service” model, 451
**code integrity, 646**
**codec, voice, 327**
**collapsed core, 400, 646**
**command/s. *See also* configuration**
access-class, 171–173
access-list, 121–122, 126, 130–131. *See also* IP ACL (access control list)
*implicit deny, 124*
*log keyword, 129*
*permit any, 124*
*reverse engineering from ACL to address range, 131–133*
*syntax, 125*

cdp enable, 286
cdp timer, 287
CDP-related, 295–296
clear ip nat translation, 305, 313–314
clock set, 279
clock summer-time, 279
copy, 379, 383–384
copy running-config startup-config, 228
crypto key generate rsa, 221
DAI-related, 261–262
debug, 273, 276–277
debug ip nat, 314
DHCP-related, 261
dig, 194
dir, 380–381
enable, 198
enable password, 206, 221
*interactions with enable secret command, 206–207*
enable secret, 206, 221
*changing the encoding type, 209–210*
*deleting, 208*
*hash function, 207–208*
*interactions with enable password command, 206–207*
errdisable recovery cause psecure-violation, 237
errdisable recovery interval, 237
ip access-group, 176
ip access-list, 138–139
ip access-list resequence, 175
ip arp inspection validate, 258–259
ip arp inspection vlan, 255
ip dhcp snooping, 246–248
ip helper-address, 168
ip inside source static, 303
ip nat inside source, 312, 315
ip nat pool, 311
line console 0, 221
line vty, 221
lldp run, 290–291
LLDP-related, 295
logging buffered, 271
logging host, 271
logging-related, 294–295
login, 221
login local, 221
more, 377
NAT-related, 319–320
no cdp enable, 286
no enable secret, 208, 221
no shutdown, 237, 248
nslookup, 194
ntp master, 279, 281–282
ntp server, 278–279
NTP-related, 295–296
ping, 164
pwd, 380–381
service password-encryption, 205–206, 221
show access-lists, 126–127
show cdp, 283
show cdp neighbors, 284–285
show cdp neighbors detail, 285–286
show file systems, 377
show flash, 379–381
show ip access-list, 126–127, 132, 153
show ip arp inspection, 256
show ip arp inspection statistics, 257
show ip dhcp snooping, 247, 249, 256
show ip interface, 153
show ip nat statistics, 312–313
show ip nat translations, 309, 312, 314, 317
show lldp, 291

show lldp entry, 289–290
show lldp neighbors, 288
show logging, 271, 274–275
show ntp associations, 281–282
show ntp status, 280, 282
show port-security, 232
show port-security interface, 228–229, 232, 234–235
show running-config, 126, 139–140, 155, 164, 168, 205–206, 227, 234, 308–309, 311, 316, 377
shutdown, 231, 237, 248
switchport mode, 236
switchport mode access, 227
switchport port-security access, 227
switchport port-security mac-address, 236–237
switchport port-security maximum, 237
terminal monitor, 296
transport input, 221
transport input ssh, 205
username password, 210, 221
username secret, 210, 221
verify, 381–382
whois, 194

**common ACL, 175–176, 646**

**communities, SNMP, 375**

**complex matching, 330–331**

**"computationally difficult", 207, 209**

**configuration/s**

AireOS WLC, 79
*configuring the WLAN, 81–83*
*configuring WLAN security, 83–84*
*create a dynamic interface, 79–80*
*create a new WLAN, 80–81*
automation, 561–562
CDP (Cisco Discovery Protocol), 286
DAI (Dynamic ARP Inspection), 259
*on a Layer 2 switch, 254–257*
*limiting message rates, 257–258*
*optional message checks, 258–260*
DHCP snooping, 245–246, 249
*on a Layer 2 switch, 246–248*
*limiting DHCP message rates, 248–249*
drift, 554–555, 646
dynamic NAT (Network Address Translation), 310–312
extended IP ACL, 150–151
files, 555–557
IOS-XE WLC, 67–69
*apply the policy tag to some APs, 78–79*
*mapping the WLAN and policy profiles to a policy tag, 77*
*policy profile, 74–77*
*profile, 69–74*
LLDP (Link Layer Discovery Protocol), 290–291
management, 488–489, 646
management tools, 565–566, 646
*Ansible, 562–563*
*Terraform, 563–565*
monitoring, 646
monitoring and enforcement, 557–558
named IP ACL, 139
NAT (Network Address Translation), static, 308–309
NTP (Network Time Protocol), 279–281
PAT (Port Address Translation), 314–317
port security, 225–228
profile, 67

provisioning, 558–559, 563, 647
syslog, 273–274
template, 559–561, 647
variables, 560–561
WLAN, 65–67
*advanced settings, 85–86*
*finalizing, 86–87*
*QoS, 85*
WLC (wireless LAN controller), 61–63
**congestion avoidance, 346–347**
**connected mode, 647**
**connectionless protocol, 100–101**
**connection/s**
autonomous AP, 58
Cisco AP, 58–59
establishment, 647
-oriented protocol, 100–101
TCP (Transmission Control Protocol), 100–101
**container, 444, 647**
Docker, 445–446
engine, 445
image, 445
vendors, 445
**control plane, 27, 474–475, 647**
centralized, 477
distributed architecture, 477
LISP (Locator/ID Separation Protocol), 505–509
**controller, 477**
-based networks, 488–489, 491–492, 647
Cisco Catalyst Center. *See* Cisco Catalyst Center
NBI (northbound interface), 479–481
Open SDN, 483
OpenDaylight, 482–483
SBI (southbound interface), 478–479
**copy command, 379, 383–384**
**copy running-config startup-config command, 228**
**core design, 401, 647**
**core layer, 647**
**core switch, 402**
**CoS (Class of Service), 646**
**CPU**
multithreading, 442
virtual, 442
**CRUD (create, read, update, and delete) actions, 533–534, 647**
**crypto key generate rsa command, 221**
**CS (Class Selector), 336–337**

# D

**DAI (Dynamic ARP Inspection), 250, 649**
configuration, 254–257, 259
limiting message rates, 257–258
logic, 253–254
optional message checks, 258–260
**data center, 440, 442**
automation and control, 484
network, 446
virtualization, 442–443
virtualized, 446–448
**data modeling language, 542**
**data plane, 27, 473, 476, 648**
**data serialization language, 541–542, 546, 648**
JSON (JavaScript Object Notation), 544
need for, 543–544
XML (eXtensible Markup Language), 544–545
YAML, 545–546

**data structure**
dictionary, 532
list, 532
variable, 477
**debug command, 273, 276–277**
**debug ip nat command, 314**
**Declarative Model, 648**
**declarative policy model, 648**
**default router, 352, 355**
**delay, 325, 648**
**deleting, enable secret command, 208**
**denial-of-service attack, 189–190**
**deny any logic, 121, 648**
**deployment, WLC (wireless LAN controller), 35**
centralized, 32
cloud-based, 32
distributed, 33–34
embedded, 34
**destination port number, 96**
**device/s. *See also* IOS**
discovery. *See* CDP (Cisco Discovery Protocol)
log messages, 270–271
*debug command, 276–277*
*format, 272*
*severity levels, 272–273*
*storing for later review, 271*
passwords, securing, 204–205
PoE (Power over Ethernet), 408–409
*power classification, 410*
*power detection, 409–410*
*standards, 411*
**DevNet, 648**
**DevOps, 492**
**DHCP (Dynamic Host Configuration Protocol)**
attack, 241–242, 248, 648
messages, filtering, 167–169
snooping, 240–241, 648
*binding table, 244*
*configuration, 245–246, 249. See also configuration, DHCP snooping*
*filtering DISCOVER messages based on MAC address, 243–244*
*filtering messages that release IP addresses, 244–245*
*limiting DHCP message rates, 248–249*
*logic, 242–243*
*rules, 241, 243*
**dictionary attack, 196, 648**
**dictionary data structure, 532**
**DiffServ (Differentiated Services), 333, 335, 648**
AF (Assured Forwarding), 336
CS (Class Selector), 336–337
EF (Expedited Forwarding), 336
guidelines for DSCP marking values, 337
**dig command, 194**
**digital certificate, 197**
**dir command, 380–381**
**direction, ACL (access control list), 116–117**
**DISCOVER messages, filtering, 243–244**
**distributed architecture, 477**
**distributed control plane, 649**
**distributed denial-of-service attack, 190, 649**
**distributed WLC deployment, 33–34, 649**
**distribution layer, 649**

**distribution link, 649**
**distribution switch, 399–400, 402**
full mesh, 400
partial mesh, 400
**distribution system ports, 64**
**DMZ (demilitarized zone), 214**
**DNS (Domain Name System), 98–99, 105–106, 649**
messages, filtering, 163–164
recursive lookup, 107–108
resolution and requesting a web page, 106–107
**Docker, 445–446**
**documentation, API, 536**
**domain-specific language, 649**
**DoS (denial of service) attack, 212, 648**
**DS (distribution system), 10–12, 649**
**DSCP (Differentiated Services Code Point), 330–331, 333, 649.** ***See also*** **DiffServ (Differentiated Services)**
**DSL (digital subscriber line), 426–427**
**DSLAM (digital access multiplexer), 427**
**dynamic NAT (Network Address Translation), 304–306**
configuration, 310–312
troubleshooting, 317
verification, 312–314
**dynamic window, 102**

## E

**EAP (Extensible Authentication Protocol), 47–48**
**EAP-FAST (EAP Flexible Authentication by Secure Tunneling), 48–49**
**EAP-TLS (EAP Transport Layer Security), 50**
**eavesdropping, 44–45**
**editing**
named IP ACL, 140–142
numbered IP ACL, 143–144
**EF (Expedited Forwarding), 336**
**E-LAN, 419–422, 649**
**elasticity, 448**
**E-Line, 418–421, 650**
**embedded wireless controller (EWC) deployment, 34**
**enable command, 198**
**enable password command, 206–207, 221**
**enable secret command, 206, 221, 650**
changing the encryption algorithm, 209–210
deleting, 208
hash function, 207–208
interactions with enable password command, 206–207
**encoding types**
enable secret command, 209
username secret command, 211
**encryption**
IPsec, 431–432
key, 432
MIC (message integrity check), 45–46
password, 205–206
TKIP (Temporal Key Integrity Protocol), 50–51
wireless, 45
*CCMP (Counter/CBC-MAC Protocol), 51*
*GCMP (Galois/Counter Mode Protocol), 51*
*End-to-End QoS Network Design*, **328**
**enterprise network, 186–187**
*Enterprise QoS Solution Reference Network Design Guide*, **328**

**EPG (endpoint group), 486**
**ephemeral ports, 97**
**errdisable recovery cause psecure-violation command, 237**
**errdisable recovery interval command, 237**
**error/s**
detection, 94, 650
recovery, 94, 101–102, 650
**ESS (extended service set), 650**
**ESSID (Extended Service Set Identifier), 12**
**Ethernet, 6. *See also* MetroE (Metro Ethernet)**
10GBASE-T, 405
802.1Q header, marking, 333–334
access link, 650
fiber, 406–407
full-duplex mode, 8
half-duplex mode, 7–8
multigig, 405–406
Power over. *See* PoE (Power over Ethernet)
UTP standards, 404–405
WAN, 650
**ETR (egress tunnel router), 649**
**EVC (Ethernet Virtual Connection), 419**
**EWC (embedded wireless controller), 650**
**exam. *See* CCNA exam 200–301**
**exploit, 188, 215, 650**
**extended IP ACL, 144–145, 650**
configuration, 150–151
matching packets from web servers, 153–154
matching packets to web servers, 151–153
matching TCP and UDP port numbers, 147–150
matching the protocol, source IP, and destination IP, 145–146
syntax, 145–146

## F

**fabric, 497–498, 650**
border node, 500
control-plane node, 500
edge node, 500, 506
routed access layer design, 502
**fake AP, 44–46**
**FHRP (First Hop Redundancy Protocol), 350, 352**
GLBP (Gateway Load Balancing Protocol), 362–363
*active/active load balancing, 364–365*
*AVF (active virtual forwarder), 364*
*AVG (active virtual gateway), 364*
*similarities with HSRP and VRRP, 363–364*
*VIP (virtual IP address), 363*
HSRP (Hot Standby Router Protocol)
*active/standby model, 356, 359*
*failover, 357–358*
*Hello message, 358*
*Hold timer, 358*
*interface tracking, 359–360*
*load balancing, 359*
*preemption, 360–361*
*priority, 357*
*similarities with VRRP, 362*
*standby state, 357*
*versions, 361–362*
*VIP (virtual IP address), 357*
*virtual MAC address, 357*

need for, 354–356
VRRP (Virtual Router Redundancy Protocol), 362–363
*similarities with HSRP, 362*
*VIP (virtual IP address), 363*
**fiber Internet, 429, 650**
**fiber uplinks, 406–407**
**file system, IOS, 376–377, 379–381**
**filtering**
DHCP messages, 167–169, 243–244
DNS messages, 163–164
ICMP messages, 164–165
OSPF messages, 165–167
packets
*based on destination port, 148*
*based on source port, 148*
SSH (Secure Shell), 169–171
Telnet, 169–171
URI (Uniform Resource Identifier), 217
**finalizing WLAN configuration, 86–87**
**firewall, 211–212, 650**
advanced features, 212
DMZ (demilitarized zone), 214
next-generation, 216–218
stateful, 212–213
zones, 213–214
**first-match logic, IP ACL, 119**
**flash memory, 376, 651**
**FlexConnect Mode, 36–37, 651**
**flow, 327**
**flow control, 651**
**form factor, server, 441**
**format, log message, 272**
**forward acknowledgement, 101**
**forward secrecy, 53**
**fps (frames per second), 475–476**
**frame, 329**
**FTP (File Transfer Protocol), 99, 384–386, 651**
active mode, 386–387
client, 385
control connection, 384, 386
copying images, 382–384
passive mode, 387
server, 385
**full drop, 347**
**full mesh, 400, 403, 651**
**full-duplex mode, 8**

## G

**GCMP (Galois/Counter Mode Protocol), 51**
**generative AI, 517–518, 651**
**Get message, 371**
**GET response, 108**
**GetBulk message, 371**
**GetNext message, 371**
**GitHub, 557, 651**
**GLBP (Gateway Load Balancing Protocol), 356, 362–363, 651**
active/active load balancing, 364–365
AVF (active virtual forwarder), 364
AVG (active virtual gateway), 364
similarities with HSRP and VRRP, 363–364
VIP (virtual IP address), 363
**gratuitous ARP, 251–252, 651**
**GRE (Generic Routing Encapsulation), 432**
**group key, 45**
**GTC (Generic Token Card), 49**
**GUI, Cisco Catalyst Center, 515**

# H

**half-duplex mode, 7–8**
**hash function**
algorithm, 209
enable secret command, 207–208
MD5 (Message Digest 5), 207, 209
**HCL (HashiCorp Configuration Language), 564**
**header fields**
IPv4, 109
TCP (Transmission Control Protocol), 95
UDP (User Datagram Protocol), 104
**hold timer**
CDP (Cisco Discovery Protocol), 287
HSRP (Hot Standby Router Protocol), 358
**host mobility, 498**
**HSRP (Hot Standby Router Protocol), 356**
active/standby model, 356, 359
failover, 357–358
Hello message, 358
Hold timer, 358
interface tracking, 359–360
load balancing, 359
preemption, 360–361, 652
priority, 357, 652
similarities with VRRP, 362
standby state, 357
versions, 361–362
VIP (virtual IP address), 357
**HTTP (Hypertext Transfer Protocol), 104, 652**
GET response, 108
how an app is chosen to receive data, 109
request and response, 534
and REST APIs, 533
transferring files, 108–109
verbs, 534
versions
*HTTP 1.0 and 1.1, 110*
*HTTP 3.0, 111–112*
*HTTP/2 and TLS, 110–111*
**HTTP/3, adjusting ACLs for, 154–155**
**hub and spoke topology, 652**
**human vulnerabilities, 195–196**
pharming, 195
phishing, 195
social engineering, 195
spear phishing, 195
watering hole attack, 195
whaling, 195
**hybrid topology, 403**
**hypervisor, 442, 444, 461, 652**

# I

**IaaS (Infrastructure as a Service), 451–452, 653**
**IAC (infrastructure as code), 653**
**IANA (Internet Assigned Numbers Authority), 97**
**IBN (intent-based networking), 483, 486–488, 653**
**IBSS (independent basic service set), 13–14, 653**
**ICMP (Internet Control Message Protocol), message filtering, 164–165**
**IEEE 802.11, 7–8, 18–19**
**IEEE 802.3, 6**
**IFS (IOS File System), 653**
**Imperative Model, 652**

**Inform message, 372**
**inside global address, 303–304, 653**
**inside local address, 303–304, 653**
**inside source NAT, 302–303**
**integrity, message, 45–46**
**intercloud exchange, 459–460**
**interface**
access, 642
application programming, 478
southbound, 478–479
tracking, 359–360
user network, 417
**interference, wireless network, 7**
**Internet**
access, 426
*4G/5G, 428–429*
*cable, 427–428*
*DSL (digital subscriber line), 426–427*
*fiber, 429*
VPN, 425–426, 430–431. *See also* VPN
**IOS. *See also* command/s**
ACLs, 173–174
configuring well-known port numbers, 149
file system, 376–377
filenames, 379
image upgrade, 378
*using FTFP, 378–379*
*verifying code integrity, 381–382*
listing files in the file system, 379–381
log messages, 270–271
*configuration, 273–274*
*debug command, 276–277*
*format, 272*
*severity levels, 272–273*
*storing for later review, 271*
*verification, 274–276*
passwords
*encrypting, 205–206*
*securing, 204–205*
**IOS XE. *See also* command/s; device/s**
ACLs, 173–174
common ACL, 175–176
configuration menus, 61–63
WLAN configuration, 67–69
*apply the policy tag to some APs, 78–79*
*map the WLAN and policy profiles to a policy tag, 77*
*policy profile, 74–77*
*profile, 69–74*
**ip access-group command, 176**
**ip access-list command, 138–139**
**ip access-list resequence command, 175**
**IP ACL (access control list), 116, 118, 331, 511**
adjusting for HTTP/3, 154–155
building, 155
common ACL, 175–176
comparing in IOS and IOS XE, 173–174
deny any logic, 121
DHCP messages, filtering, 167–169
DNS messages, filtering, 163–164
extended, 144–145
*configuration, 150–151*
*matching packets from web servers, 153–154*
*matching packets to web servers, 151–153*
*matching TCP and UDP port numbers, 147–150*
*matching the protocol, source IP, and destination IP, 145–146*

*syntax, 145–146*
ICMP messages, filtering, 164–165
implementing, 125, 156
list logic, 119–121
location and direction, 116–117
matching logic, 121–122
matching multiple nonconsecutive ports with eq parameter, 177
matching packets, 117–118
named, 138
*configuration, 139*
*editing, 140–142*
*versus numbered, 138–139*
*verification, 139–140*
numbered
*editing, 143–144*
*versus named, 138–139*
OSPF messages, filtering, 165–167
resequencing sequence numbers, 174–175
SSH, filtering, 169–171
standard numbered, 119, 125–129
taking action when a match occurs, 118
Telnet, filtering, 169–171
troubleshooting, 129–130
types of, 118–119
wildcard mask, 122–123
*binary, 123–124*
*finding the right one to match a subnet, 124*
**ip arp inspection validate command, 258–259**
**ip arp inspection vlan command, 255**
**ip dhcp snooping command, 246–248**
**ip helper-address command, 168**
**ip inside source static command, 303**
**ip nat inside source command, 312, 315**
**ip nat pool command, 311**
**IPP (IP Precedence), 333, 653**
**IPS (intrusion prevention system), 215–216, 653**
next-generation, 218–219
signature database, 215–216
**IPsec, 431, 433–434, 653**
remote access VPN, 433–434
transport mode, 434
tunnel mode, 434
**IPv4**
header
*fields, 109*
*marking, 333*
private addresses, 300–301
**ISDN (Integrated Services Digital Network), 426**
**ITR (ingress tunnel router), 506–508**

## J

**Jinja2, 654**
**jitter, 325, 654**
**JSON (JavaScript Object Notation), 541–542, 544, 654**
arrays, 547–549
beautified, 550
data serialization, 541–542
key:value pairs, 547
minified, 550
objects, 547–549

## K

**key/s**
:value pair, 547, 654
encryption, 432
WEP (Wired Equivalent Privacy), 47

# L

**label switching, 423**
**LAN**
campus, 398–399, 403. *See also* campus LAN
*access switches, 399*
*distribution switches, 399–400*
*fiber uplinks, 406–407*
*full mesh, 403*
*hybrid topology, 403*
*partial mesh, 403*
*star topology, 402*
*three-tier, 400–402*
*two-tier, 399–400*
collapsed core, 400
core design, 401
SOHO (small office/home office), 407–408
**leaf, 654**
**LEAP (Lightweight EAP), 48–49**
**lightweight AP (access point), 28**
**line console 0 command, 221**
**line vty command, 221**
**LISP (Locator/ID Separation Protocol), 503, 505–509, 654**
**list data structure, 532, 654**
**list logic, IP ACL, 119–121**
**LLDP (Link Layer Discovery Protocol), 283, 654**
configuration, 290–291
examining information learned by, 287–290
MED (Media Endpoint Discovery), 292–293
timer, 287
TLV (type-length-value), 292
verification, 291–292
**lldp run command, 290–291**
**LLM (Large Language Model), 521, 654**
**LLQ (low-latency queuing), 339–341**
**load balancing**
active/active, 364–365
HSRP (Hot Standby Router Protocol), 359
**local username, 654**
**log message, 270–271, 274–276, 655.** ***See also*** **syslog**
debug command, 276–277
format, 272
severity levels, 272–273
storing for later review, 271
**logging buffered command, 271**
**logging host command, 271**
**logic**
DAI (Dynamic ARP Inspection), 253–254
DHCP snooping, 242–243
**login command, 221**
**login local command, 221**
**loss, 325, 655**
**LWAPP (Lightweight Access Point Protocol), 29**

# M

**MAC (Media Access Control) layer, 28, 655.** ***See also*** **split-MAC architecture**
**MAC address/es, 8**
port security, 229–230
virtual, 357
**malware, 194–195**
Trojan horse, 194
virus, 194
worm, 194–195
**management IP address, 25**

**management plane, 475, 655**
**manager, SNMP, 370**
**man-in-the-middle attack, 44, 191–193, 655**
**marking, 330, 332, 655**
802.1Q header, 333–334
DSCP (Differentiated Services Code Point)
*AF (Assured Forwarding), 336*
*CS (Class Selector), 336–337*
*EF (Expedited Forwarding), 336*
*guidelines, 337*
fields, 334
IP header, 333
trust boundary, 334–335
**matching, 117–118, 330–331. *See also* ACL (access control list); QoS**
IP ACL, 331
*logic, 121–122*
*matching TCP and UDP port numbers, 147–150*
taking action when a match occurs, 118
to web servers, 151–153
from web servers, 153–154
**MD5 (Message Digest 5), 207, 209, 381–382, 655**
**MED (Media Endpoint Discovery), LLDP (Link Layer Discovery Protocol), 292–293**
**memory**
flash, 376
ternary content-addressable, 476
**Meraki, 462**
dashboard, 655
default view, 462–463
Topology and Path Visualization, 463–464
**mesh, 403, 655**
**mesh AP, 17**
**message/s**
ARP, 251
classification, 329–330. *See also* classification
DHCP, filtering, 167–169
DNS, filtering, 163–164
HSRP (Hot Standby Router Protocol), 357–358
HTTP, 534
ICMP, filtering, 164–165
integrity check, 45–46
LLDP (Link Layer Discovery Protocol), TLV (type-length-value), 292
log, 270–271
*debug command, 276–277*
*format, 272*
*severity levels, 272–273*
*storing for later review, 271*
OSPF, filtering, 165–167
privacy, 44–45
SNMP
*Get, 371*
*GetBulk, 371*
*GetNext, 371*
*Inform, 372*
*Trap, 372*
**MetroE (Metro Ethernet), 416, 655**
E-LAN, 419–420
E-Line, 418–419
EVC (Ethernet Virtual Connection), 419
IEEE Ethernet standards, 417–418
Layer 3 design, 420
*using E-LAN, 421–422*
*using E-Line, 420–421*
physical design and topology, 416–418
topology, 418

**MIB (Management Information Base), 370, 372–374, 655**
**mitigation technique, 655**
**ML (machine learning), 517–518**
**mobile phone, 4G/5G, 428–429**
**monitoring, configuration, 557–558**
**more command, 377**
**MP-BGP (Multiprotocol BGP), 425**
**MPGBP (Multiprotocol BGP), 656**
**MPLS (Multiprotocol Label Switching), 422, 656**
access link technologies, 424
CE (customer edge), 424
PE (provider edge), 424
QoS (Quality of Service), 423
VPN, 422–423
*Layer 3, 424–425*
*physical design and topology, 423–424*
**MTU (maximum transmission unit), 326, 655**
**multifactor authentication, 199, 656**
**multifactor credentials, 197**
**multigig Ethernet, 405–406, 656**
**multimode fiber cable, 406**
**multiplexing, 95–97**
**multithreading, 442**

# N

**named IP ACL, 138, 656**
configuration, 139
editing, 140–142
versus numbered, 138–139
verification, 139–140
**narrow AI, 517–518, 656**
**NAT (Network Address Translation), 300, 302, 656**
dynamic, 304–306
*configuration, 310–312*
*verification, 312–314*
inside global address, 303–304
inside local address, 303–304
inside source, 302–303
Overload, 306–307
*configuration, 314–317*
*verification, 317*
static, 303
*configuration, 308–309*
*verification, 309–310*
troubleshooting, 317–318
**NBAR2 (next-generation Network Based Application Recognition), 331–332**
**NBI (northbound interface), 479–481, 490, 494, 657**
**network/s. *See also* LAN; WAN; wireless network/s**
baseline, 524
data center, 446
fabric, 497–498
management, 488–491
outage, 353–354
overlay, 497, 498
private, 300–302
programmability, 472
redundancy, need for, 353–354
single point of failure, 353–354
tail drop, 347
traditional versus controller-based, 488–489, 491–492
traffic
*bandwidth, 324–325*
*delay, 325*
*jitter, 325*
*types of, 325*

trust boundary, 334–335
underlay, 497–498
wireless, comparing with wired, 6–7
**next-generation firewall, 216–218**
**NGFW (next-generation firewall), 216–218, 656**
**NGIPS (next-generation IPS), 218–219, 657**
**NIC (network interface card), 443**
**NIST (National Institute of Standards and Technology), 51, 448–449**
**NMS (Network Management System), 370, 374, 514–517, 656**
**no cdp enable command, 286**
**no enable secret command, 208, 221**
**no shutdown command, 237, 248**
**nonoverlapping channels, 18, 20, 657**
**nslookup command, 194**
**NTP (Network Time Protocol), 277–278, 656–657**
client/server mode, 281
configuration, 279–281
reference clock, 281
setting the time and time zone, 278–279
stratum, 281–282
synchronization, 280
**ntp master command, 279, 281–282**
**ntp server command, 278–279**
**numbered IP ACL**
editing, 143–144
versus named, 138–139

## O

**objects, JSON, 547–549**
**OfficeExtend, 37**
**OID (object ID), 372**
**OM (Optical Multimode), 406–407, 657**
**on-demand self-service, 657**
**open authentication, 46**
**Open SDN, 481–482**
**OpenDaylight controller, 482–483**
**OpenFlow, 482, 657**
**operational network management, 489**
**ordered data transfer, 657**
**OSC (Open SDN Controller), 483**
**OSI model, transport layer, 94**
**OSPF (Open Shortest Path First), 165–167, 474–475**
**outdoor bridge, 16**
**output queuing, 338**
**outside global, 658**
**overlay, 497–498, 658.** ***See also*** **Cisco SD-Access**

## P

**PaaS (Platform as a Service), 453–454, 658**
**packet/s, 329.** ***See also*** **IP ACL (access control list); traffic**
complex matching, 330–331
filtering
*based on destination port, 148*
*based on source port, 148*
matching, 117–118
MTU (maximum transmission unit), 326
VoIP (voice over IP), 327–328
**partial mesh, 400, 403, 658**
**passive mode, FTP, 387**
**passive scanning, 9, 658**
**password/s.** ***See also*** **enable password command; enable secret command**
alternatives, 196
*biometric credentials, 197*

*digital certificate, 197*
*multifactor credentials, 197*
brute-force attack, 196
clear-text, 207
dictionary attack, 196
enable, encoding with hashes, 206
encrypting, 205–206
guessing, 658
IOS, securing, 204–205
policy, 196
SNMP, 374–375
vulnerabilities, 196
**PAT (Port Address Translation), 306–307**
configuration, 314–317
troubleshooting, 318
verification, 317
**PD (powered device), 409, 659**
**PE (provider edge), 424, 660**
**PEAP (Protected EAP), 49**
**personal mode, WPA (Wi-Fi Protected Access), 52–53**
**pharming, 195, 658**
**PHB (per-hop behavior), 322, 658**
**phishing, 195, 658**
**physical access control, 200**
**physical ports, WLC (wireless LAN controller), 63–65**
**ping command, 164**
**PKI (Public Key Infrastructure), 50**
**playbook, Ansible, 563**
**PoE (Power over Ethernet), 408–409, 658–659**
and LAN design, 411
power classification, 410
power detection, 409–410
standards, 411
**point-to-point bridged link, 16, 658**
**policing, 340, 342–344, 658**
**policy**
Cisco SD-Access, 513
password, 196
profile, 74–77
tag, 68
**PoP (point of presence), 417, 658**
**port security, 224–225, 659**
configuration, 225–228
MAC addresses, 229–230
protect mode, 233–234
restrict mode, 234–235
shutdown mode, 231–233
sticky secure MAC addresses, 225
verification, 228–229
violation modes, 230–231
**port/s, 659**
distribution system, 64
ephemeral, 97
number, 96–98
untrusted, 665
user, 97
well-known, 97, 99, 149
**Postman, 537–541**
**practicing CLI skills, 593–595**
**predictive analytics, 524**
**preemption, HSRP (Hot Standby Router Protocol), 360–361**
**on-premise cloud. *See* private cloud**
**prioritization, 338**
**priority queue, 340, 659**
**privacy, message, 44–45**
**private cloud, 449–450, 659**
**private networks, 300–302**
**private WAN, connecting to the cloud with, 458–459**
**probe request, 659**

**profile**
configuration, 67
policy, 74–77
WLAN configuration, 69–74
**programming, variables, 530–531**
data structure, 477
simple, 531
**protect mode, port security, 233–234**
**protocol, connectionless/connection-oriented, 100–101**
**provider, 660**
**PSE (power sourcing equipment), 409, 659**
**public cloud, 450–451, 460, 660**
intercloud exchange, 459–460
private WAN and Internet VPN access to the, 458–459
using Internet to connect to the, 457–458
**public IPv4 addresses, 300**
**pull model, 660**
**push model, 563, 660**
**pwd command, 380–381**
**Python**
dictionary data structure, 532
list data structure, 532

## Q

**QFP (Quantum Flow Processor), 476**
**QoE (quality of experience), 326**
**QoS (Quality of Service), 322, 324, 660.** ***See also*** **traffic**
CAC (call admission control), 340–341
classification, 329–330
*NBAR2 (next-generation Network Based Application Recognition), 331–332*
*on routers, 331–332*
congestion avoidance, 346–347
DiffServ, 333, 335
*AF (Assured Forwarding), 336*
*CS (Class Selector), 336–337*
*EF (Expedited Forwarding), 336*
*guidelines for DSCP marking values, 337*
marking, 330, 332
*802.1Q header, 333–334*
*fields, 334*
*IP header, 333*
MPLS (Multiprotocol Label Switching), 423
policing, 342–344
queuing, 337–338
*class-based weighted fair, 339*
*classifier function, 338*
*low-latency, 339–341*
*output, 338*
*prioritization, 338–339*
round-robin scheduling, 338–339
on routers and switches, 329
shaping, 341–342, 344–346
trust boundary, 334–335
WLAN, configuring, 85
**queuing, 329–330, 337–338, 660**
class-based weighted fair, 339
classifier function, 338
low-latency, 339–341
output, 338
prioritization, 338
**QUIC, 660**

## R

**radio, AP, 19**
**RADIUS, 199–200, 660**
**rapid elasticity, 660**

**rate limit, DAI (Dynamic ARP Inspection), 257–258**
**RC4 cipher algorithm, 47**
**RCA (Root Cause Analysis), 524**
**read-only community, 660**
**read-write community, 660**
**reassociation frame, 13, 661**
**reconnaissance attack, 661**
**recursive DNS lookup, 107–108, 661**
**redundancy**
need for, 353–354
single point of failure, 353–354
**reference clock, NTP (Network Time Protocol), 281**
**reflection attack, 191, 661**
**reliability, TCP (Transmission Control Protocol), 101–102**
**remote access VPN**
with IPsec, 433–434
with TLS, 434–435
**repeater, 14**
**resequencing ACL sequence numbers, 174–175**
**resource pooling, 448, 661**
**RESTful API/s, 480–481, 510, 528–529**
cacheable, 530
client/server architecture, 529
CRUD actions, 533–534
and HTTP, 533
stateless operation, 530
URIs, 534–536
**restrict mode, port security, 234–235**
**RF signals, 7**
**RF tag, 78–79**
**RFC (Request for Comments)**
791, 333
793, 95
1065, "Structure and Identification of Management Information for TCP/IP-based Internets", 370
1918, 301
2475, 335
3986, 535–536
7348, 498
**roaming, 13, 661**
**round-robin scheduling, 338–339, 661**
**round-trip delay, 325**
**routed access layer, 502**
**router/s, 505.** ***See also*** **device/s; IOS; IOS XE**
classification, 331–332
data plane processing, 473
default, 352, 355
filtering for vty access, 171–173
flash memory, 376
FTP password and username configuration, 383–384
ingress tunnel, 506–508
login security, 204
QoS, 329
VRF (virtual routing and forwarding), 455–456
wireless, 407
**rules, DHCP snooping, 241, 243**

## S

**SaaS (Software as a Service), 452–453, 663**
**SAE (Simultaneous Authentication of Equals), 53**
**SBI (southbound interface), 478–479, 494**
**scheduling, round-robin, 338–339**
**scoring, exam, 587**
**SD-Access, 663**

**SDN (Software-Defined Networking), 472, 663. *See also* controller**
ACI (Application Centric Infrastructure), 484
*APIC (Application Policy Infrastructure Controller), 487*
*EPG (endpoint group), 486*
*operating model with intent-based networking, 486–488*
*spine and leaf design, 484–485*
automation, 489–491
controller, 477
NBI (northbound interface), 479–481
ONF (Open Networking Foundation) model, 481–482
OpenDaylight controller, 482–483
SBI (southbound interface), 478–479
**security. *See also* attack/s; authentication; port security**
exploit, 188
firewall, 211–212
*advanced features, 212*
*next-generation, 216–218*
*stateful, 212–213*
*zones, 213–214*
group-based, 513–514
IPS (intrusion prevention system), 215–216
*next-generation, 218–219*
*signature database, 215–216*
mitigation techniques, 188
password, IOS, 204–205
program, 200
shared-key, 47
threat/s, 188
vulnerability, 187–188
wireless
*TKIP (Temporal Key Integrity Protocol), 50–51*
*WPA (Wi-Fi Protected Access), 51–52*
*WPA2, 52*
*WPA3, 52*
*WPA-Personal mode, 52–53*
WLAN, configuration, 83–84
zones, 213–214
**segment, 662**
**self-healing wireless coverage, 32**
**self-service, 448**
**send time, CDP (Cisco Discovery Protocol), 287**
**sender hardware address, 662**
**serial console, 461**
**server. *See also* VM (virtual machine)**
blade, 441
Cisco hardware, 440–441
form factor, 441
FTP (File Transfer Protocol), 385
NIC, 443
NTP (Network Time Protocol), 281
rack, 440
UCS (Unified Computing system), 485
virtualization, 441–443
web, 667
**service password-encryption command, 205, 206, 221**
**severity level, log message, 272–273**
**SGT (scalable group tag), 513–514, 661**
**SHA-256, 209**
**SHA512, verifying IOS code integrity, 381–382**
**shaping, 341–342, 344–346, 662**
**shared key, 47, 662**
**show access-lists command, 126–127**
**show cdp commands, 283**
**show cdp neighbors command, 284–285**

**show cdp neighbors detail command, 285–286**
**show flash command, 379–381**
**show ip access-list command, 126–127, 132, 153**
**show ip arp inspection command, 256**
**show ip arp inspection statistics command, 257**
**show ip dhcp snooping command, 247, 249, 256**
**show ip interface command, 153**
**show ip nat statistics command, 312–313**
**show ip nat translations command, 309, 312, 314, 317**
**show lldp command, 291**
**show lldp entry command, 289–290**
**show lldp neighbors command, 288**
**show logging command, 271, 274–275**
**show ntp associations command, 281–282**
**show ntp status command, 280, 282**
**show port-security command, 232**
**show port-security interface command, 228–229, 232, 234–235**
**show running-config command, 126, 139–140, 155, 164, 168, 205–206, 208, 227, 234, 308–309, 311, 316, 377**
**shutdown command, 231, 237, 248**
**shutdown mode, port security, 231–233**
**signature database, IPS (intrusion prevention system), 215–216**
**signatures, application, 332**
**simple variable, 531**
**single point of failure, 353–354.** ***See also*** **FHRP (First Hop Redundancy Protocol)**
**site tag, 78–79**
**site-to-site VPN, 430–433, 662**
**sliding window, 102, 662**
**smishing, 195**
**SNMP (Simple Network Management Protocol), 99, 662–663**
ACLs, 374
agent, 370
clear-text password, 374–375
communities, 375
Get message, 371
GetBulk message, 371
GetNext message, 371
Inform message, 372
manager, 370
MIB (Management Information Base), 370, 372–374, 655
NMS (Network Management System), 370, 374
securing, 374–375
Trap notification, 372
versions, 375
**snooping, DHCP, 240–241**
binding table, 244
configuration, 245–246, 249. *See also* configuration, DHCP snooping
filtering DISCOVER messages based on MAC address, 243–244
filtering messages that release IP addresses, 244–245
limiting DHCP message rates, 248–249
logic, 242–243
rules, 241
**social engineering, 195, 663**
**socket, 97.** ***See also*** **port/s**
**software container.** ***See*** **container**
**SOHO (small office/home office), 407–408, 663**
**source NAT, 663**

**SP (service provider), 662**
access link, 417
intercloud exchange, 459–460
**spear phishing, 195, 664**
**spine and leaf design, 484–485, 664**
**split-MAC architecture, 28–32, 36–37, 664**
CAPWAP (Control and Provisioning of Wireless Access Points), 29–31
WLC functions, 32
**spoofing attack, 188–189, 664**
amplification attack, 191
denial-of-service, 189–190
man-in-the-middle, 44, 191–193
reflection, 191
**SSH (Secure Shell), 169–171, 204**
**SSID (Service Set Identifier), 8–9, 44, 662**
autonomous AP, 25
supporting multiple on one AP, 11–12
**STA (station), 664**
**standalone mode, 664**
**standard numbered IP ACL, 119, 125–129**
**standards**
IEEE 802.11, amendments, 18–19
PoE (Power over Ethernet), 411
**star topology, 402, 664**
**stateful firewall, 212–213**
**stateless operation, 664**
**static NAT (Network Address Translation), 303**
configuration, 308–309
troubleshooting, 317
verification, 309–310
**sticky secure MAC addresses, 225**
**storage, variable, 542–543**
**STP (Spanning Tree Protocol), 474**
**stratum, 281–282**
**subnet, matching with an ACL, 124**
**switch/es. *See also* device/s; IOS; IOS XE**
access, 399, 402
ASIC (application-specific integrated circuit), 476
core, 402
data plane, 475–476
distribution, 399–400, 402
fps (frames per second), 475–476
LAN, 222
Layer 2, 505
login security, 204
port security, 224–225
*configuration, 225–228*
*MAC addresses, 229–230*
*protect mode, 233–234*
*restrict mode, 234–235*
*shutdown mode, 231–233*
*sticky secure MAC addresses, 225*
*verification, 228–229*
*violation modes, 230–231*
QoS, 329
TOR (Top of Rack), 446
virtual, 443–444
**switchport mode access command, 227**
**switchport mode command, 236**
**switchport port-security access command, 227**
**switchport port-security mac-address command, 236–237**
**switchport port-security maximum command, 237**
**SYN flag, 100**
**synchronization, NTP (Network Time Protocol), 280**

**syntax**
access-list command, 125
extended IP ACL, 145–146
**syslog, 270, 664**
configuration, 273–274
debug command, 276–277
verification, 274–276

# T

**TACACS+, 199–200**
**tag**
policy, 68
RF, 78–79
site, 78–79
**tail drop, 347**
**TCAM (ternary content-addressable memory), 476, 664**
**TCP (Transmission Control Protocol), 94, 95**
congestion avoidance, 346–347
connection establishment, 100
connection termination, 100–101
error recovery and reliability, 101–102
forward acknowledgement, 101
header fields, 95
multiplexing, 95–96
popular applications and their well-known port numbers, 99
port numbers, 96–98, 147–150
socket, 97
windowing, 102–103, 346–347
**TCP/IP. *See also* TCP (Transmission Control Protocol); UDP (User Datagram Protocol); web browser**
DNS (Domain Name System), 98–99, 105–106
*recursive lookup, 107–108*
*resolution and requesting a web page, 106–107*
HTTP (Hypertext Transfer Protocol), 104
*how an app is chosen to receive data, 109*
*transferring files, 108–109*
model, transport layer, 94
SNMP (Simple Network Management Protocol), 99
TFTP (Trivial File Transfer Protocol), 99
URI (Uniform Resource Identifier), 104–105
WWW (World Wide Web), 98
**Telnet, 169–171, 204**
**template, configuration, 559–561**
**terminal monitor command, 296**
**Terraform, 563–565, 664**
**TFTP (Trivial File Transfer Protocol), 99, 387–388, 665. *See also* FTP (File Transfer Protocol)**
**threats, 188, 194–195, 665. *See also* attack/s**
**three-tier campus LAN, 400–402**
**TIA (Telecommunications Industry Association), 404**
**time**
interval, shaper, 345–346
NTP, setting, 278–279
**TKIP (Temporal Key Integrity Protocol), 50–51**
**TLS (Transport Layer Security), 434–435, 665**
**TLV (type-length-value), LLDP (Link Layer Discovery Protocol), 292**
**tools. *See also* QoS (Quality of Service)**
API development environment, 536–541
ChatGPT, 518–523

configuration management
*Ansible, 562–563*
*Terraform, 563–565*
congestion avoidance, 347
development, 453
**topology**
hybrid, 403
MetroE (Metro Ethernet), 418
MPLS VPN, 423–424
star, 402
**TOR (Top of Rack) switch, 446**
**ToS (type of service) byte, 333**
**traffic**
bandwidth, 324–325
batch, 326
data application, 325–326
delay, 325
flow, 327
jitter, 325
prioritization, 341
types of, 325
voice and video application, 327–328
**transport input command, 221**
**transport input ssh command, 205**
**transport layer, 94**
**Trap notification, 372**
**Trojan horse, 194, 665**
**troubleshooting**
IP ACL (access control list), 129–130
NAT (Network Address Translation), 317–318
**trust boundary, 334–335**
**trusted port, 665**
**tunnel**
CAPWAP (Control and Provisioning of Wireless Access Points), 29–31
VPN, 430
**tunneling, VXLAN, 504**
**two-tier campus LAN, 399–400**
full mesh, 403
hybrid topology, 403
star topology, 402
uplinks, 403

# U

**UCS (Unified Computing system), 441, 485, 665**
**UDP (User Datagram Protocol), 94, 103**
data transfer, 103–104
header, 104
multiplexing, 95–96
port numbers, matching, 147–150
**underlay, 497–498, 665.** ***See also*** **Cisco SD-Access**
**UNI (user network interface), 417**
**unidirectional communication, wireless network, 7**
**untrusted port, 665**
**updates, exam, 572–576**
**upgrade, IOS image, 378–379**
**uplink, fiber, 406–407**
**UPoE (Universal Power over Ethernet), 665**
**URI (Uniform Resource Identifier), 104–105, 665–666**
format, 535–536
using with REST to specify the resource, 534–536
**URL (Universal Resource Locator), 105, 217**
**user ports, 97**
**username password command, 210, 221**
**username secret command, 210, 221**
**UTP (unshielded twisted-pair), 404–405, 666**

**UTP (unshielded twisted-pair) cabling, 403–404**
**uWGB (universal workgroup bridge), 16**

# V

**variable, 530–531**
configuration, 560–561
data structure, 477. *See also* data structure
simple, 531
storing, 542–543
**vCPU (virtual CPU), 442, 666**
**vendors**
container, 445
virtualized data center, 443
**verbs, HTTP, 534**
**verification**
CDP (Cisco Discovery Protocol), 286–287
dynamic NAT (Network Address Translation), 312–314
LLDP (Link Layer Discovery Protocol), 291–292
named IP ACL, 139–140
NAT (Network Address Translation), 309–310
PAT (Port Address Translation), 317
port security, 228–229
syslog, 274–276
**verify command, 381–382**
**version/s**
control, 555–557
HSRP (Hot Standby Router Protocol), 361–362
SNMP, 375
**video**
prioritization, 341
QoS requirements, 328
**violation mode, 230–231, 666**
**VIP (virtual IP address), 357, 363, 666**
**virtual console, 461**
**virtual MAC address, 357**
**virtualization**
data center, 446–448
hypervisor, 442, 444, 461
server, 441–443
vendors, 443
**virus, 194, 215–217, 666**
**vishing, 195**
**VLAN (virtual local-area network), 10–11, 25**
**VM (virtual machine), 442, 447, 666**
versus container, 444
PaaS (Platform as a Service), 453
**vNIC (virtual NIC), 443, 666**
**VoIP (voice over IP)**
prioritization, 341
QoS requirements, 327–328
**VPN, 430–431, 458, 666**
Internet, 425–426
IPsec, 431
MPLS (Multiprotocol Label Switching), 422–423
*Layer 3, 424–425*
*physical design and topology, 423–424*
remote access
*with IPsec, 433–434*
*with TLS, 434–435*
site-to-site, 430–433
tunnel, 430
**VRF (virtual routing and forwarding), 425, 454–456, 666**
**VRRP (Virtual Router Redundancy Protocol), 356, 362–363**
**vSwitch (virtual switch), 443–444, 666**

**vty, access control, 171–173**
**vulnerability/ies, 187–188, 667**
human, 195–196
*pharming, 195*
*phishing, 195*
*social engineering, 195*
*spear phishing, 195*
*watering hole attack, 195*
*whaling, 195*
password, 196
**VXLAN, 498, 503–505, 667**

# W

**WAN. *See also* MetroE (Metro Ethernet)**
connection to the cloud, 456
*accessing public cloud services using the Internet, 456–457*
*using Internet to connect to the public cloud, 457–458*
link, 37
MPLS (Multiprotocol Label Switching), 422
*access link technologies, 424*
*CE (customer edge), 424*
*PE (provider edge), 424*
*QoS, 423*
*VPN, 422–423*
private, 430, 458–459
Software-Defined, 484
**watering hole attack, 195, 667**
**web browser, 104**
DNS recursive lookup, 107–108
DNS resolution and requesting a web page, 106–107
how an app is chosen to receive data, 109
transferring files with HTTP, 108–109
**web server, 104–105, 667**
**web-based GUI, WLC (wireless LAN controller), 59–61**
**well-known ports, 97, 99, 149**
**WEP (Wired Equivalent Privacy), 47, 50**
**WGB (workgroup bridge), 15–16, 667**
**whaling, 195, 667**
**whitespace, 550**
**whois command, 194**
**Wi-Fi, generational names, 20**
**Wi-Fi Alliance, 51–53, 407**
**wildcard mask, 122–123, 667**
binary, 123–124
finding the right one to match a subnet, 124
**windowing, 102–103, 346–347**
**wireless network/s. *See also* WLC (wireless LAN controller)**
2.4-GHz band, 17
4G/5G, 428–429
5-GHz band, 17
ad hoc, 14
AP (access point). *See also* AP (access point); autonomous AP; Cisco AP/s
*association request/response, 9, 19*
*autonomous, 24–25*
*beacon frame, 8*
*BSA (basic service area), 8*
*BSS (basic service set), 8*
*cloud-based architecture, 26–27*
*IBSS (independent basic service set), 13–14*
*infrastructure mode, 8*
*management platform, 26*
*mesh, 17*
*outdoor bridge, 16*

*radios, 19*
*repeater mode, 14*
*roaming, 13*
*supporting multiple SSIDs on, 11–12*
architecture
*autonomous AP, 24–25*
*cloud-based AP, 26–27*
*split-MAC, 28–32, 36–37*
authentication, 43
*802.1x/EAP, 47–48*
*AP (access point), 44*
*client, 43–44*
*EAP-FAST (EAP Flexible Authentication by Secure Tunneling), 48–49*
*EAP-TLS (EAP Transport Layer Security), 50*
*LEAP (Lightweight EAP), 48–49*
*open, 46*
*PEAP (Protected EAP), 49*
*WEP (Wired Equivalent Privacy), 47*
bands, 17
bidirectional communication, 7
BSS (basic service set), 8–9
*distribution system (DS), 10–12*
*traffic flows, 9*
channel, 17–18
comparing with wired networks, 6–7
encryption, 45
*CCMP (Counter/CBC-MAC Protocol), 51*
*GCMP (Galois/Counter Mode Protocol), 51*
ESS (extended service set), 12–13
interference, 7
MIC (message integrity check), 45–46
RF signals, 7
secure connection, 42
security, WPA (Wi-Fi Protected Access), 51–53
self-healing, 32
SOHO (small office/home office), 407–408
WGB (workgroup bridge), 15–16
**WLAN**
configuration, 65–67
*advanced settings, 85–86*
*on AireOS WLC, 79–83*
*finalizing, 86–87*
*on IOS-XE WLC, 67–79*
*QoS, 85*
open authentication, 46
**WLC (wireless LAN controller), 29, 32, 667.** ***See also*** **IOS-XE WLC**
AireOS, 79
*configuring the WLAN, 81–83*
*configuring WLAN security, 83–84*
*create a new WLAN, 80–81*
*creating a dynamic interface, 79–80*
CAPWAP (Control and Provisioning of Wireless Access Points), 29–31
centralized deployment, 32
cloud-based deployment, 32
configuration, 61–63
deployment models, 35
distributed deployment, 33–34
embedded wireless controller (EWC) deployment, 34
IOS-XE
*apply the policy tag to some APs, 78–79*
*configuring a policy profile, 74–77*

*mapping the WLAN and policy profiles to a policy tag, 77*
*WLAN configuration, 67–79*
physical ports, 63–65
virtual interface, 65
web-based GUI, 59–61
WLAN configuration, 65–67
*apply the policy tag to some APs, 78–79*
*map the WLAN and policy profiles to a policy tag, 77*
*policy profile, 74–77*
*profile, 69–74*
WMI (wireless management interface), 65
**WMI (wireless management interface), 65**
**worm, 194–195, 667**
**WPA (Wi-Fi Protected Access), 51–52**
client authentication modes, 52
personal mode, 52–53
versions, 52
**write community, 667**
**WWW (World Wide Web), 98, 104**

## X-Y-Z

**X.509 certificate, 30**
**XML (eXtensible Markup Language), 544–545, 667**

**YAML, 545–546, 667**

**zone**
demilitarized, 214
firewall, 213–214
**ZTP (zero touch provisioning), 462, 667**

*This page intentionally left blank*



Register your product at **ciscopress.com/register** to unlock additional benefits:

- Save 35%* on your next purchase with an exclusive discount code
- Find companion files, errata, and product updates if available
- Sign up to receive special offers on new editions and related titles

Get more when you shop at **ciscopress.com**:

- Everyday discounts on books, eBooks, video courses, and more
- Free U.S. shipping on all orders
- Multi-format eBooks to read on your preferred device
- Print and eBook Best Value Packs

**Discount code valid for 30 days; may not be combined with any other offer and is not redeemable for cash. Offer subject to change.*

**Cisco Press**

# APPENDIX D

# Topics from Previous Editions

Cisco changes the exams, renaming the exams on occasion, and changing the exam numbers every time it changes the exam with a new blueprint, even with a few name changes over the years. As a result, the current CCNA 200-301 exam, based on the version 1.1 blueprint, serves as the ninth separate version of CCNA in its 25-plus year history.

We base the books' contents on Cisco's exam topics; that is, the book attempts to cover the topics Cisco lists as exam topics. However, the book authoring process does create some challenges, particularly with the balance of what to include in the books and what to leave out.

For instance, when comparing a new exam to the old, I found Cisco had removed some topics—and I might want to keep the content in the book. There are a few reasons why. Sometimes I just expect that some readers will still want to read about that technology. Also, more than a few schools use these books as textbooks, and keeping some of the older-but-still-relevant topics can be a help. And keeping the old material available on each book's companion website takes only a little extra work, so we do just that.

Some of the older topics that I choose to keep on the companion website are small, so I collect them into this appendix. Other topics happen to have been an entire chapter in a previous edition of the books, so we include those topics each as a separate appendix. Regardless, the material exists here in this appendix, and in the appendices that follow, for your use if you have a need. But do not feel like you have to read this appendix for the current exam.

The topics in this appendix are as follows:

- Troubleshooting with IPv4 ACLs
- Implementing HSRP
- NTP Addendum
- Metro Ethernet Virtual Circuits
- MPLS VPNs and OSPF
- A Scenario: Branch Offices and the Public Cloud

The topics in this chapter follow.

**NOTE** The content under the heading "Troubleshooting with IPv4 ACLs" was most recently published for the 200-105 Exam in 2016, in Chapter 17 of the *Cisco CCNA ICND2 200-105 Official Cert Guide*.

# Troubleshooting with IPv4 ACLs

The use of IPv4 ACLs makes troubleshooting IPv4 routing more difficult. Any data plane troubleshooting process can include a catchall phrase to include checking for ACLs. A network can have all hosts working, DHCP settings correct, all LANs working, all router interfaces working, and all routers having learned all routes to all subnets—and ACLs can still filter packets. Although ACLs provide that important service of filtering some packets, ACLs can make the troubleshooting process that much more difficult.

This third of the three major sections of this chapter focuses on troubleshooting in the presence of IPv4 ACLs. It breaks the discussion into two parts. The first part gives advice about common problems you might see on the exam, and how to find those with **show** commands and some analysis. The second part then looks at how ACLs impact the **ping** command.

## Analyzing ACL Behavior in a Network

ACLs cause some of the biggest challenges when troubleshooting problems in real networking jobs. The packets created by commands like **ping** and **traceroute** do not exactly match the fields in packets created by end users. The ACLs sometimes filter the **ping** and **traceroute** traffic, making the network engineer think some other kind of problems exists when no problems exist at all. Or the problem with the end-user traffic really is caused by the ACL, but the ping and traceroute traffic works fine, because the ACL matches the end-user traffic with a **deny** action but matches the ping and traceroute traffic with a **permit** action.

As a result, much of ACL troubleshooting requires thinking about ACL configuration versus the packets that flow in a network, rather than using a couple of IOS commands that identify the root cause of the problem. The **show** commands that help are those that give you the configuration of the ACL, and on what interfaces the ACL is enabled. You can also see statistics about which ACL statements have been matched. And using pings and traceroutes can help—as long as you remember that ACLs may apply different actions to those packets versus the end-user traffic.

The following phrases the ACL troubleshooting steps into a list for easier study. The list also expands on the idea of analyzing each ACL in Step 3. None of the ideas in the list are new compared to this chapter and the previous chapter, but it acts more as a summary of the common issues:

**Step 1.** Determine on which interfaces ACLs are enabled, and in which direction (**show running-config, show ip interfaces**).

**Step 2.** Find the configuration of each ACL (**show access-lists, show ip access-lists, show running-config**).

**Step 3.** Analyze the ACLs to predict which packets should match the ACL, focusing on the following points:

A. **Misordered ACLs:** Look for misordered ACL statements. IOS uses first-match logic when searching an ACL.

B. **Reversed source/destination addresses:** Analyze the router interface, the direction in which the ACL is enabled, compared to the location of the IP address ranges matched by the ACL statements. Make sure the source IP address field could match packets with that source IP address, rather than the destination, and vice versa for the destination IP address field.

C. **Reversed source/destination ports:** For extended ACLs that reference UDP or TCP port numbers, continue to analyze the location and direction of the ACL versus the hosts, focusing on which host acts as the server using a well-known port. Ensure that the ACL statement matches the correct source or destination port depending on whether the server sent or will receive the packet.

D. **Syntax:** Remember that extended ACL commands must use the **tcp** and **udp** keywords if the command needs to check the port numbers.

E. **Syntax:** Note that ICMP packets do not use UDP or TCP; ICMP is considered to be another protocol matchable with the **icmp** keyword (instead of **tcp** or **udp**).

F. **Explicit deny any:** Instead of using the implicit **deny any** at the end of each ACL, use an explicit configuration command to deny all traffic at the end of the ACL so that the **show** command counters increment when that action is taken.

G. **Dangerous inbound ACLs:** Watch for inbound ACLs, especially those with deny all logic at the end of the ACL. These ACLs may discard incoming overhead protocols, like routing protocol messages.

H. **Standard ACL location:** Standard ACLs enabled close to the source of matched addresses can discard the packets as intended, but also discard packets that should be allowed through. Always pay close attention to the requirements of the ACL in these cases.

The first two steps are important for Simlet questions in case you are not allowed to look at the configuration; you can use other **show** commands to determine all the relevant ACL configuration. The next few pages show some of the related commands and how they can uncover some of the issues described in the just-completed ACL troubleshooting checklist.

## ACL Troubleshooting Commands

If you suspect ACLs are causing a problem, the first problem-isolation step is to find the location and direction of the ACLs. The fastest way to do this is to look at the output of the **show running-config** command and to look for **ip access-group** commands under each interface. However, in some cases, enable mode access may not be allowed, and **show** commands are required. Instead, use the **show ip interfaces** command to find which ACLs are enabled on which interfaces, as shown in Example D-1.

**Example D-1** *Sample* **show ip interface** *Command*

```
R1> show ip interface s0/0/1
Serial0/0/1 is up, line protocol is up
 Internet address is 10.1.2.1/24
 Broadcast address is 255.255.255.255
 Address determined by setup command
 MTU is 1500 bytes
 Helper address is not set
 Directed broadcast forwarding is disabled
 Multicast reserved groups joined: 224.0.0.9
 Outgoing access list is not set
 Inbound access list is 102
! roughly 26 more lines omitted for brevity
```

Note that the command output lists whether an ACL is enabled, in both directions, and which ACL it is. The example shows an abbreviated version of the **show ip interface S0/0/1** command, which lists messages for just this one interface. The **show ip interface** command would list the same messages for every interface in the router.

Step 2 of the ACL troubleshooting checklist then says that the contents of the ACL must be found. Again, the quickest way to look at the ACL is to use the **show running-config** command. If it's not available, the **show access-lists** and **show ip access-lists** commands list the same details shown in the configuration. These commands also list a useful counter that lists the number of packets that have matched each line in the ACL. Example D-2 shows an example.

**Example D-2** **show ip access-lists** *Command Example*

```
R1# show ip access-lists
Extended IP access list 102
 10 permit ip 10.1.2.0 0.0.0.255 10.1.4.0 0.0.1.255 (15 matches)
```

The counter can be very useful for troubleshooting. If you can generate traffic that you think should match a particular line in an ACL, then you should see the matches increment on that counter. If you keep generating traffic that should match, but that line's counter never goes up, then those packets do not match that line in that ACL. Those packets could be matching an earlier line in the same ACL, or might not even be reaching that router (for any reason).

After the locations, directions, and configuration details of the various ACLs have been discovered in Steps 1 and 2, the hard part begins—analyzing what the ACL really does. For example, one of the most common tasks you will do is to look at the address fields and decide the range of addresses matched by that field. Remember, for an ACL that sits in a router configuration, you can easily find the address range. The low end of the range is the address (the first number), and the high end of the range is the sum of the address and wildcard mask. For instance, with ACL 102 in Example D-2, which is obviously configured in some router, the ranges are as follows:

**Source 10.1.2.0, wildcard 0.0.0.255:** Matches from 10.1.2.0 through 10.1.2.255

**Destination 10.1.4.0, wildcard 0.0.1.255:** Matches from 10.1.4.0 through 10.1.5.255

The next few pages work through some analysis of a few of the items from Step 3 in the troubleshooting checklist.

## Example Issue: Reversed Source/Destination IP Addresses

IOS cannot recognize a case in which you attempt to match the wrong addresses in the source or destination address field. So, be ready to analyze the enabled ACLs and their direction versus the location of different subnets in the network. Then ask yourself about the packets that drive that ACL: what could the source and destination addresses of those packets be? And does the ACL match the correct address ranges, or not?

For example, consider Figure D-1, a figure that will be used in several troubleshooting examples in this chapter. The requirements for the next ACL follow the figure.

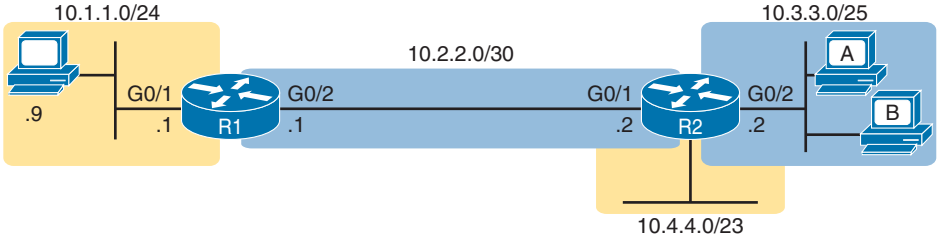



**Figure D-1** *Example Network Used in IPv4 ACL Troubleshooting Examples*

For this next ACL, the requirements ask that you allow and prevent various flows, as follows:

- Allow hosts in subnet 10.3.3.0/25 and subnet 10.1.1.0/24 to communicate
- Prevent hosts in subnet 10.4.4.0/23 and subnet 10.1.1.0/24 from communicating
- Allow all other communications between hosts in network 10.0.0.0
- Prevent all other communications

Example D-3 shows the ACL used in this case on R2. At first glance, it meets all those requirements straight down the list.

**Example D-3** *Troubleshooting Example 2 per Step 3B: Source and Destination Mismatch*

```
R2# show ip access-lists
Standard IP access list Step3B
 10 permit 10.3.3.0 0.0.0.127
 20 deny 10.4.4.0 0.0.1.255
 30 permit 10.0.0.0 0.255.255.255 (12 matches)
R2#
R2# show ip interface G0/1 | include Inbound
  Inbound access list is Step3B
```

The problem in this case is that the ACL has been enabled on R2's G0/1 interface, inbound. Per the figure, packets coming from a source address in subnets 10.3.3.0/25 and 10.4.4.0/23 should be forwarded out R2's G0/1 interface, rather than coming in that interface. So, do not

let the matching logic in the ACL that perfectly mirrors the requirements fool you; make sure and check the location of the ACL, direction, and the location of the IP addresses.

Note that Step 3C suggests a similar issue regarding matching well-known ports with TCP and UDP. The earlier section in this chapter titled "Matching TCP and UDP Port Numbers" has already discussed those ideas in plenty of detail. Just make sure to check where the server sits versus the location and direction of the ACL.

### Steps 3D and 3E: Common Syntax Mistakes

Steps 3D and 3E describe a couple of common syntax mistakes. First, to match a TCP port in an ACL statement, you must use a **tcp** protocol keyword instead of **ip** or any other value. Otherwise, IOS rejects the command as having incorrect syntax. Same issue with trying to match UDP ports: a **udp** protocol keyword is required.

To match ICMP, IOS includes an **icmp** protocol keyword to use instead of **tcp** or **udp**. In fact, the main conceptual mistake is to think of ICMP as an application protocol that uses either UDP or TCP; it uses neither. To match all ICMP messages, for instance, use the **permit icmp any any** command in an extended named ACL.

### Example Issue: Inbound ACL Filters Routing Protocol Packets

A router bypasses outbound ACL logic for packets the router itself generates. That might sound like common sense, but it is important to stop and think about that fact in context. A router can have an outgoing ACL, and that ACL can and will discard packets that the router receives in one interface and then tries to forward out some other interface. But if the router creates the packet, for instance, for a routing protocol message, the router bypasses the outbound ACL logic for that packet.

However, a router does not bypass inbound ACL logic. If an ACL has an inbound ACL enabled, and a packet arrives in that interface, the router checks the ACL. Any and all IPv4 packets are considered by the ACL—including important overhead packets like routing protocol updates.

For example, consider a seemingly good ACL on a router, like the Step3G ACL in Example D-4. That ACL lists a couple of **permit** commands, and has an implicit deny any at the end of the list. At first, it looks like any other reasonable ACL.

**Example D-4** *Troubleshooting Example 2 per Step 3G: Filtering RIP by Accident*

```
R1# show ip access-lists
Standard IP access list Step3G
 10 permit host 10.4.4.1
 20 permit 10.3.3.0 0.0.0.127 (12 matches)
! using the implicit deny to match everything else
R1#
! On router R1:
R1# show ip interface G0/2 | include Inbound
  Inbound access list is Step3G
```

Now look at the location and direction (inbound on R1, on R1's G0/2) and consider that location versus the topology in Figure D-1 for a moment. None of those **permit** statements match the RIP updates sent by R2, sent out R2's G0/1 interface toward R1. RIP messages use UDP (well-known port 520), and R2's G0/1 interface is 10.2.2.2 per the figure. R1 would match incoming RIP messages with the implicit deny all at the end of the list. The symptoms in this case, assuming only that one ACL exists, would be that R1 would not learn routes from R2, but R2 could still learn RIP routes from R1.

Of the three routing protocols discussed in the ICND1 and ICND2 books, RIPv2 uses UDP as a transport, while OSPF and EIGRP do not even use a transport protocol. As a result, to match RIPv2 packets with an ACL, you need the **udp** keyword and you need to match well-known port 520. OSPF and EIGRP can be matched with special keywords as noted in Table D-1. The table also list the addresses used by each protocol.

**Table D-1** Key Fields for Matching Routing Protocol Messages

| Protocol | Source IP Address | Destination IP Addresses | ACL Protocol Keyword |
|---|---|---|---|
| RIPv2 | Source interface | 224.0.0.9 | **udp** (port 520) |
| OSPF | Source interface | 224.0.0.5, 224.0.0.6 | **ospf** |
| EIGRP | Source interface | 224.0.0.10 | **eigrp** |

Example D-5 shows a sample ACL with three lines, one to match each routing protocol, just to show the syntax. Note that in this case, the ACL matches the address fields with the **any** keyword. You could include lines like these in any inbound ACL to ensure that routing protocol packets would be permitted.

**Example D-5** *Example ACL That Matches All RIPv2, OSPF, and EIGRP with a Permit*

```
R1# show ip access-lists
ip access-list extended RoutingProtocolExample
 10 permit udp any any eq 520
 20 permit ospf any any
 30 permit eigrp any any
 remark a complete ACL would also need more statements here
R1#
```

## ACL Interactions with Router-Generated Packets

Routers bypass outbound ACL logic for packets generated by that same router. This logic helps avoid cases in which a router discards its own overhead traffic. This logic applies to packets that a router creates for overhead processes like routing protocols, as well as for commands, like **ping** and **traceroute**. This section adds a few perspectives about how ACLs impact troubleshooting, and how this exception to outbound ACL logic applies, particularly commands used from the router CLI.

### Local ACLs and a Ping from a Router

For the first scenario, think about a **ping** command issued by a router. The command generates packets, and the router sends those packets (holding the ICMP echo request messages)

out one of the router interfaces, and typically some ICMP echo reply messages are received back. As it turns out, not all ACLs will attempt to filter those packets.

As a backdrop to discuss what happens, Figure D-2 illustrates a simple network topology with two routers connected to a serial link. Note that in this figure four IP ACLs exist, named A, B, C, and D, as noted by the thick arrows in the drawing. That is, ACL A is an outbound ACL on R1's S0/0/0, ACL B is an inbound ACL on R2's S0/0/1, and so on.

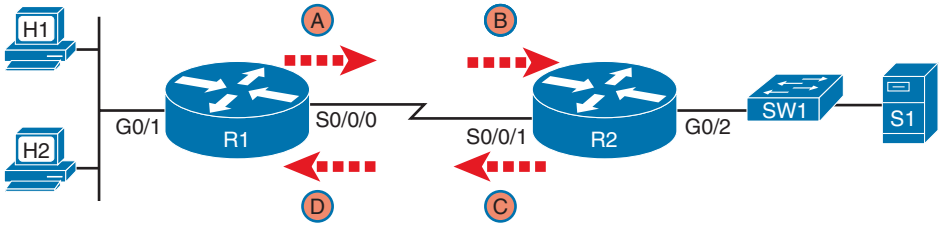



**Figure D-2** *Sample Network with IP ACLs in Four Locations*

As an example, consider a **ping** command issued from R1's CLI (after a user connects to R1's CLI using SSH). The **ping** command pings server S1's IP address. The IPv4 packets with the ICMP messages flow from R1 to S1 and back again. Which of those four ACLs could possibly filter the ICMP Echo Request toward S1, and the ICMP Echo Reply back toward R1?

Routers bypass their own outbound ACLs for packets generated by the router, as shown in Figure D-3. Even though ACL A exists as an outgoing ACL on Router R1, R1 bypasses its own outgoing ACL logic of ACL A for the ICMP Echo Requests generated by R1.

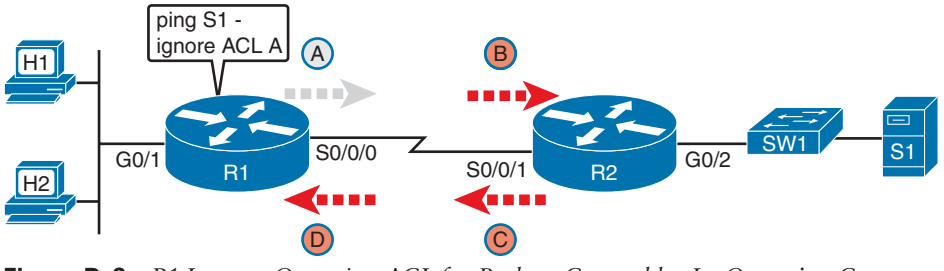



**Figure D-3** *R1 Ignores Outgoing ACL for Packets Created by Its Own* **ping** *Command*

## Router Self-Ping of a Serial Interface IPv4 Address

The previous example uses a router's **ping** command when pinging a host. However, network engineers often need to ping router IP addresses, including using a self-ping. The term *self-ping* refers to a ping of a device's own IPv4 address. And for point-to-point serial links, a self-ping actually sends packets over the serial link, which causes some interesting effects with ACLs.

When a user issues a self-ping for that local router's serial IP address, the router actually sends the ICMP echo request out the link to the other router. The neighboring router then receives the packet and routes the packet with the ICMP echo request back to the original router. Figure D-4 shows an example of a self-ping (**ping 172.16.4.1**) of Router R1's own IP address on a point-to-point serial link, with the ICMP echo request out the link to Router

R2. At Step 2, R2 treats it like any other packet not destined for one of R2's own IPv4 addresses: R2 routes the packet. Where? Right back to Router R1, as shown in the figure.

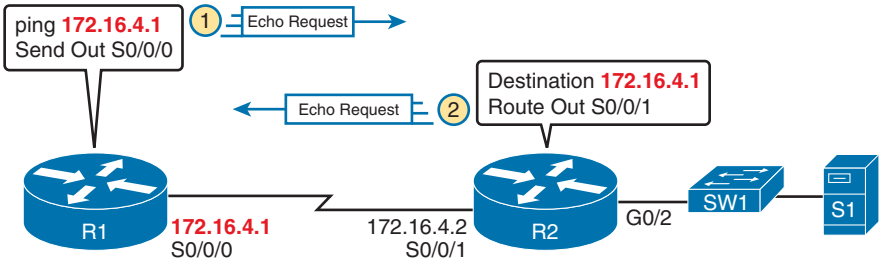



**Figure D-4** *The First Steps in a Self-Ping on R1, for R1's S0/0/0 IP Address*

Now think about those four ACLs in the earlier figures compared to Figure D-4. R1 generates the ICMP echo request, so R1 bypasses outbound ACL A. ACLs B, C, and D could filter the packet. Note that the packet sent by R2 back to R1 is not generated by R2 in this case; R2 is just routing R1's original packet back to R1.

A self-ping of a serial interface actually tests many parts of a point-to-point serial link, as follows:

- The link must work at Layers 1, 2, and 3. Specifically, both routers must have a working (up/up) serial interface, with correct IPv4 addresses configured.
- ACLs B, C, and D must permit the ICMP echo request and reply packets.

So, when troubleshooting, if you choose to use self-pings and they fail, but the serial interfaces are in an up/up state, do not forget to check to see whether the ACLs have filtered the Internet Control Management Protocol (ICMP) traffic.

### Router Self-Ping of an Ethernet Interface IPv4 Address

A self-ping of a router's own Ethernet interface IP address works a little like a self-ping of a router's serial IP address, but with a couple of twists:

- Like with serial interfaces, the local router interface must be working (in an up/up state); otherwise, the ping fails.
- Unlike serial interfaces, the router does not forward the ICMP messages physically out the interface, so security features on neighboring switches (like port security) or routers (like ACLs) cannot possibly filter the messages used by the **ping** command.
- Like serial interfaces, an incoming IP ACL on the local router does process the router self-ping of an Ethernet-based IP address.

Figure D-5 walks through an example. In this case, R2 issues a **ping 172.16.2.2** command to ping its own G0/2 IP address. Just like with a self-ping on serial links, R2 creates the ICMP echo request. However, R2 basically processes the ping down its own TCP/IP stack and back up again, with the ICMP echo never leaving the router's Ethernet interface. R2 does check the Ethernet interface status, showing a failure if the interface is not up/up. R2 does not apply outbound ACL logic to the packet, because R2 created the packet, but R2 will

apply inbound ACL logic to the packet, as if the packet had been physically received on the interface.

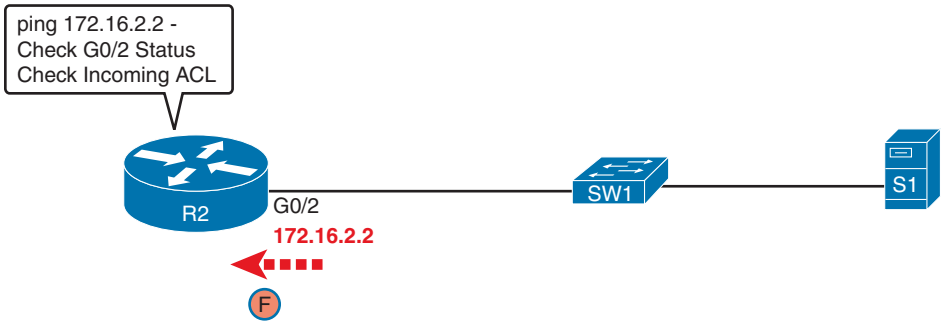



**Figure D-5** *Self-Ping of a Router's Ethernet Address*

**NOTE** The content under the heading "Implementing HSRP" was most recently published for the 200-105 Exam in 2016, in Chapter 20 of the *Cisco CCNA ICND2 200-105 Official Cert Guide*.

# Implementing HSRP

The goal of this section is to show enough of the operation of each tool to reinforce your understanding of configuring the basic functions of HSRP.

## Configuring and Verifying Basic HSRP

HSRP configuration requires only one command on the two (or more) routers that want to share default router responsibilities with HSRP: the **standby** *group* **ip** *virtual-ip* interface subcommand. The first value defines the HSRP group number, which must match on both routers. The group number lets one router support multiple HSRP groups at a time on the same interface, and it allows the routers to identify each other based on the group. The command also configures the virtual IP address shared by the routers in the same group; the virtual IP address is the address the hosts in the VLAN use as their default gateway.

Example D-6 shows a configuration example where both routers use group 1, with virtual IP address 10.1.1.1, with the **standby 1 ip 10.1.1.1** interface subcommand.

**Example D-6** *HSRP Configuration on R1 and R2, Sharing IP Address 10.1.1.1*

```
R1# show running-config
! Lines omitted for brevity
interface GigabitEthernet0/0
 ip address 10.1.1.9 255.255.255.0
 standby version 2
 standby 1 ip 10.1.1.1
 standby 1 priority 110
 standby 1 name HSRP-group-for-book
```

```
! The following configuration, on R2, is identical except for the HSRP priority and
! the interface IP address
R2# show running-config
! Lines omitted for brevity
interface GigabitEthernet0/0
 ip address 10.1.1.129 255.255.255.0
 standby version 2
 standby 1 ip 10.1.1.1
 standby 1 name HSRP-group-for-book
```

The configuration shows other optional parameters, as well. For instance, R1 has a priority of 110 in this group, and R2 defaults to 100. With HSRP, if the two routers are brought up at the same time, the router with the higher priority wins the election to become the active router. The configuration also shows a name that can be assigned to the group (when using **show** commands) and a choice to use HSRP Version 2. (This chapter provides more details on these settings in the coming pages.)



Once configured, the two routers negotiate the HSRP settings and choose which router will currently be active and which will be standby. With the configuration as shown, R1 will win the election and become active because of its higher (better) priority. Both routers reach the same conclusion, as confirmed with the output of the **show standby brief** command on both R1 and R2 in Example D-7.

**Example D-7** *HSRP Status on R1 and R2 with* **show standby brief**

```
! First, the group status as seen from R1
R1# show standby brief
                     P indicates configured to preempt.
                     |
Interface   Grp Pri  P State   Active          Standby         Virtual IP
Gi0/0       1   110    Active  local           10.1.1.129      10.1.1.1
```

```
! The output here on R2 shows that R2 agrees with R1.
R2# show standby brief
                     P indicates configured to preempt.
                     |
Interface   Grp  Pri P State   Active          Standby         Virtual IP
Gi0/0       1    100   Standby 10.1.1.9        local           10.1.1.1
```

The **show standby brief** command packs a lot of detail in the output, so take your time and work through the highlighted fields. First, look at the Grp column for each command. This lists the HSRP group number, so when looking at output from multiple routers, you need to look at the lines with the same group number to make sure the data relates to that one HSRP group. In this case, both routers have only one group number (1), so it is easy to find the information.

Each line of output lists the local router's view of the HSRP status for that group. In particular, based on the headings, the **show standby brief** command identifies the following:

**Interface:** The local router's interface on which the HSRP group is configured

**Grp:** The HSRP group number

**Pri:** The local router's HSRP priority

**State:** The local router's current HSRP state

**Active:** The interface IP address of the currently active HSRP router (or "local" if the local router is HSRP active)

**Standby:** The interface IP address of the currently standby HSRP router (or "local" if the local router is HSRP standby)

**Virtual IP:** The virtual IP address defined by this router for this group

For instance, following the highlighted text in Example D-7, R2 believes that its own current state is standby, that the router with interface address 10.1.1.9 is active (which happens to be Router R1), with a confirmation that the "local" router (R2, on which this command was issued) is the standby router.

In comparison, the **show standby** command (without the **brief** keyword) lists a more detailed description of the current state, while repeating many of the facts from the **show standby brief** command. Example D-8 shows an example of the new information with the **show standby** command, listing several counters and timers about the HSRP protocol itself, plus the virtual MAC address 0000.0c9f.f001.

**Example D-8** *HSRP Status on R1 and R2 with* **show standby**

```
R1# show standby
GigabitEthernet0/0 - Group 1 (version 2)
  State is Active
    6 state changes, last state change 00:12:53
  Virtual IP address is 10.1.1.1
  Active virtual MAC address is 0000.0c9f.f001
    Local virtual MAC address is 0000.0c9f.f001 (v2 default)
  Hello time 3 sec, hold time 10 sec
    Next hello sent in 1.696 secs
  Preemption disabled
  Active router is local
  Standby router is 10.1.1.129, priority 100 (expires in 8.096 sec)
  Priority 110 (configured 110)
  Group name is "HSRP-group-for-book" (cfgd)
```

```
! The output here on R2 shows that R2 agrees with R1.
R2# show standby
GigabitEthernet0/0 - Group 1 (version 2)
  State is Standby
```

```
  4 state changes, last state change 00:12:05
Virtual IP address is 10.1.1.1
Active virtual MAC address is 0000.0c9f.f001
  Local virtual MAC address is 0000.0c9f.f001 (v2 default)
Hello time 3 sec, hold time 10 sec
  Next hello sent in 0.352 secs
Preemption disabled
Active router is 10.1.1.9, priority 110 (expires in 9.136 sec)
  MAC address is 0200.0101.0101
Standby router is local
Priority 100 (default 100)
Group name is "HSRP-group-for-book" (cfgd)
```



## HSRP Active Role with Priority and Preemption

HSRP defines some rules to determine which router acts as the active HSRP router and which acts as standby. Those rules also define details about when a standby router should take over as active. The following list summarizes the rules; following the list, this section takes a closer look at those rules and the related configuration settings.

First, the HSRP rules. When a router (call it the local router) has an HSRP-enabled interface, and that interface comes up, the router sends HSRP messages to negotiate whether it should be active or standby. When it sends those messages, if it…

**Step 1.** …discovers no other HSRP routers in the subnet, the local router becomes the active router.

**Step 2.** …discovers an existing HSRP router, and both are currently negotiating to decide which should become the HSRP active router, the routers negotiate, with the router with the highest HSRP priority becoming the HSRP active router.

**Step 3.** …discovers an existing HSRP router in the subnet, and that router is already acting as the active router:

- **A.** If configured with no preemption (the default; **no standby preempt**), the local router becomes a standby router, even if it has a better (higher) priority.
- **B.** If configured with preemption (**standby preempt**), the local router checks its priority versus the active router; if the local router priority is better (higher), the local router takes over (preempts) the existing active router to become the new active HSRP router.

Steps 1 and 2 in the list are pretty obvious, but Steps 3A and 3B could use a little closer look. For instance, the examples so far in this chapter show R1's G0/0 with a priority of 110 versus R2's G0/0 with priority 100. The **show** commands in Example D-7 show that R1 is currently the HSRP active router. That same example also lists a line for both R1 and R2 that confirms "preemption disabled," which is the default.

To show a test of Step 3A logic, Example D-9 shows a process by which R1's G0/0 interface is disabled and then enabled again, but after giving Router R2 long enough to take over and

become active. That is, R1 comes up but R2 is already HSRP active for group 1. The bottom of the example lists output from the **show standby brief** command from R2, confirming that R2 becomes HSRP active and R1 becomes standby (10.1.1.9), proving that R1 does not preempt R2 in this case.

**Example D-9** *Showing How No Preemption Keeps R1 as Standby After R1 Recovers*

```
! First, R1's G0/0 is disabled and enabled; the ending log message shows a standby
! state.
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# shutdown
*Mar 8 18:10:29.242: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Active ->
Init
*Mar 8 18:10:31.205: %LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to
administratively down
*Mar 8 18:10:32.205: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEther
net0/0, changed state to down
R1(config-if)#
R1(config-if)# no shutdown
R1(config-if)# ^Z
R1#
*Mar 8 18:11:08.355: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Speak ->
Standby
```

```
! Now from R2, note R2 is active, and 10.1.1.9 (R1) is standby
R2# show standby brief
                     P indicates configured to preempt.
                     |
Interface   Grp  Pri P State Active          Standby         Virtual IP
Gi0/1       1    100   Active local          10.1.1.9        10.1.1.1
```

If R1 had been configured with preemption for that previous scenario, R1 would have taken over from R2 when R1's interface came back up. Example D-10 shows exactly that. Before the output in Example D-10 was gathered, the network had been put back to the same beginning state as at the beginning of Example D-9, with R1 active and R2 as standby. Within Example D-10, R1's interface is shut down, then configured with preemption using the **standby 1 preempt** command, enabling preemption. Then, after enabling the interface again, R1 takes over as HSRP active, as shown at the bottom of the example's **show standby brief** command from R2. That output now shows the local router's state as Standby, and the active as 10.1.1.9 (R1).

**Example D-10** *Showing How Preemption Causes R1 to Take Over as Active Upon Recovery*

```
! First, R1's G0/0 is disabled and enabled; the ending log message shows a standby
 ! state.
R1# configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# shutdown
*Mar 8 18:10:29.242: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Active ->
  Init
*Mar 8 18:10:31.205: %LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to
  administratively down
*Mar 8 18:10:32.205: %LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEther
  net0/0, changed state to down
R1(config-if)# standby 1 preempt
R1(config-if)# no shutdown
R1(config-if)# ^Z
R1#
*Mar 8 18:19:14.355: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Listen ->
  Active
```

```
! Now from R2, note it is active, and 10.1.1.9 (R1) is standby

*Mar 8 18:18:55.948: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Standby ->
  Active
*Mar 8 18:19:14.528: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Active ->
  Speak
*Mar 8 18:19:26.298: %HSRP-5-STATECHANGE: GigabitEthernet0/0 Grp 1 state Speak ->
  Standby

R2# show standby brief
                     P indicates configured to preempt.
                     |
Interface    Grp Pri P State Active          Standby         Virtual IP
Gi0/0        1   100   Standby 10.1.1.9      local           10.1.1.1
```

Note that it is the preemption setting on the router that is taking over (preempting) that determines if preemption happens. For instance, in this case, R1 came up when R2 was active; R1 was set to preempt; so R1 preempted R2.

## HSRP Versions

Cisco IOS on routers and Layer 3 switches supports two versions of HSRP: versions 1 and 2. The versions have enough differences, like multicast IP addresses used and message formats, so that routers in the same HSRP group must use the same version. If two routers configured to be in the same HSRP group mistakenly configure to use different versions, they will not understand each other and ignore each other for the purposes of HSRP.

To configure the version, each interface/subinterface uses the **standby version** {**1** | **2**} interface subcommand. Note that the HSRP group number is not included in the command, because it sets the version for all HSRP messages sent out that interface/subinterface.

There are some good reasons to want to use the more recent HSRP version 2 (HSRPv2). For example, HSRPv1 existed before IPv6 became popular. Cisco enhanced HSRP to version 2 in part to make IPv6 support possible. Today, to use HSRP with IPv6 requires HSRPv2.

As another example of a benefit of HSRPv2, HSRP uses a Hello message, similar in concept to routing protocols, so that HSRP group members can realize when the active router is no longer reachable. HSRPv2 allows for shorter Hello timer configuration (as low as a small number of milliseconds), while HSRPv1 typically had a minimum of 1 second. So, HSRPv2 can be configured to react more quickly to failures with a lower Hello timer.

Beyond IPv6 support and shorter Hello timer options, other differences for version 2 versus version 1 include a different virtual MAC address base value and a different multicast IP address used as the destination for all messages. Table D-2 lists the differences between HSRPv1 and HSRPv2.

**Table D-2** HSRPv1 Versus HSRPv2

| Feature | Version 1 | Version 2 |
|---|---|---|
| IPv6 support | No | Yes |
| Smallest unit for Hello timer | Second | Millisecond |
| Range of group numbers | 0..255 | 0..4095 |
| MAC address used (xx or xxx is the hex group number) | 0000.0C07.ACxx | 0000.0C9F.Fxxx |
| IPv4 multicast address used | 224.0.0.2 | 224.0.0.102 |
| Does protocol use a unique identifier for each router? | No | Yes |

Of the details in the table, make sure to look at the MAC addresses for both versions 1 and 2. Cisco reserves the prefixes of 0000.0C07.AC for HSRPv1 and 0000.0C9F.F for HSRPv2. HSRPv1, with 256 possible HSRP groups per interface, then uses the last two hex digits to identify the HSRP group. For example, an HSRP group 1 using version 1 would use a virtual MAC address that ends in hex 01. Similarly, because HSRPv2 supports 4096 groups per interface, the MAC address reserves three hex digits to identify the group. An HSRP group 1 using version 2 would use a virtual MAC address that ends in hex 001.

**NOTE** The content under the heading "NTP Addendum" was most recently published for the 200-301 Exam blueprint 1.0, in the 2019 book *Cisco CCNA 200-301 Official Cert Guide, Volume 2*, Chapter 9, "Device Management Protocols."

## NTP Addendum

The following two topics go beyond the basic functions of NTP to discuss how to make NTP more available through redundancy features.

## Redundant NTP Configuration

Instead of using a networking device as the reference clock for the enterprise, you can instead reference better time sources in the Internet or purchase a purpose-built NTP server that has better clocking hardware. For instance, an enterprise could reference NTP servers that use an atomic clock as their reference source, like the NTP primary servers in Figure D-6, which happen to be run by the US National Institute of Standards and Technology (NIST) (see tf.nist.gov).

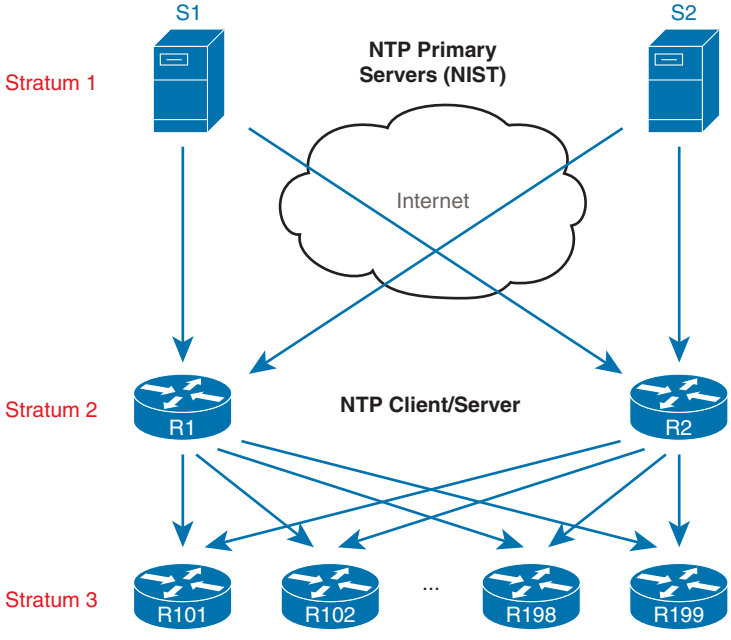



**Figure D-6** *Stratum Levels When Using an Internet-based Stratum 1 NTP Server*

**NOTE** While the common terms *NTP server mode* and *NTP client/server mode* are useful, the NTP RFCs (1305 and 5905) also use two other specific terms for similar ideas: *NTP primary server* and *NTP secondary server*. An NTP primary server acts only as a server, with a reference clock external to the device, and has a stratum level of 1, like the two NTP primary servers shown in Figure D-6. NTP secondary servers are servers that use client/server mode as described throughout this section, relying on synchronization with some other NTP server.

For good design, the enterprise NTP configuration ought to refer to at least two external NTP servers for redundancy. Additionally, just a few enterprise devices should refer to those external NTP servers and then act as both NTP client and server. The majority of the devices in the enterprise, like those shown at the bottom of the figure, would act as NTP clients. Example D-11 shows the configuration on Router R1 and R2 in the figure to accomplish this design.

**Example D-11** *NTP Configuration on R1, R2 per Figure D-6*

```
ntp server time-a-b-nist.gov
ntp server time-a-g.nist.gov
```

In addition to referencing redundant NTP primary servers, some routers in the enterprise need to be ready to supply clock data if those NTP primary servers become unreachable. An exposure exists with the configuration in Example D-11 because if Routers R1 and R2 no longer hear NTP messages from the NTP servers in the Internet they will lose their only reference clock. After losing their reference clock, R1 and R2 could no longer be useful NTP servers to the rest of the enterprise.

To overcome this potential issue, the routers can also be configured with the **ntp master** command, resulting in this logic:

1. Establish an association with the NTP servers per the **ntp server** command.
2. Establish an association with your internal clock using the **ntp master** *stratum* command.
3. Set the stratum level of the internal clock (per the **ntp master** {*stratum-level*} command) to a higher (worse) stratum level than the Internet-based NTP servers.
4. Synchronize with the best (lowest) known time source, which will be one of the Internet NTP servers in this scenario

The logic has a few steps, but the configuration itself is simple, as shown in Example D-12. Compared to Example D-11, just add the **ntp master** command. The NTP servers used in this example have a stratum level of 1, so the use of the **ntp master 7** command, with a much higher stratum, will cause Routers R1 and R2 to use one of the NIST NTP servers when available and use the internal clock source only when connectivity to the NIST servers is lost.

**Example D-12** *NTP Configuration on R1 and R2 to Protect Against Internet Failures*

```
ntp server time-a-b-nist.gov
ntp server time-a-g.nist.gov
ntp master 7
```

## NTP Using a Loopback Interface for Better Availability

An NTP server will accept NTP messages arriving to any of its IPv4 addresses by default. However, the clients reference a specific IP address on the NTP server. That creates an availability issue.

For instance, consider the topology in Figure D-7, with Router R3 on the right acting as NTP server and the other routers acting as clients. R3 has three IP addresses that the clients could put in their **ntp server** *address* commands. Now consider what happens when one interface on R3 fails, but only one. No matter which of the three interfaces fails, that IP address on

that interface cannot be used to send and receive packets. In that case, for any NTP clients that had referred to that specific IP address

- There would likely still be a route to reach other R3 addresses.
- The NTP clients would fail to communicate with R3 because the configured NTP server address is down.

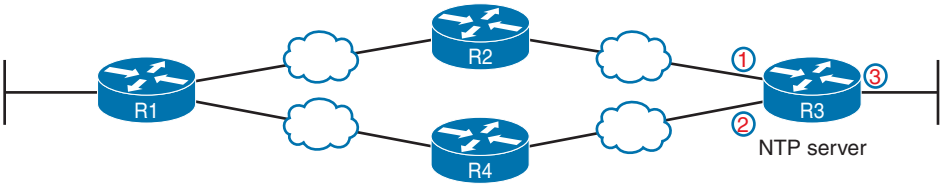



**Figure D-7** *The Availability Issue of Referencing an NTP Server's Physical Interface IP Address*

What is needed is a way to send a packet to R3 that is not tied to the state of any one interface. That is, NTP should work as long as there is some path to send packets to any of the R3 IP addresses. The goal is to avoid the case in which a single interface failure on Router R3 also causes NTP to fail.

Cisco uses the router loopback interface to meet that exact need. Loopback interfaces are virtual interfaces internal to Cisco IOS, created via the command **interface loopback** *number*, where the number is an integer. Once configured, that loopback interface exists inside that router and is not tied to any physical interface. A loopback interface can be assigned an IP address, routing protocols can advertise about the subnet, and you can ping/traceroute to that address. It acts like other physical interfaces in many ways, but once configured, it remains in an up/up state as long as

- The router remains up.
- You do not issue a **shutdown** command on that loopback interface.

**NOTE** This discussion is not about the special IPv4 loopback address 127.0.0.1. The loopback interface discussed in this section is a different concept altogether.

Example D-13 shows the small configuration change that adds the loopback interface to the NTP configuration based on Figures D-7 and D-8. In this case, R1, R2, and R4 all act as NTP clients pointing to R3's new loopback interface IP address of 172.16.9.9. R3 now has configuration for a new loopback interface (loopback 0) plus a command that tells it to use that loopback 0 interface's IP address as the source address when sending NTP packets.

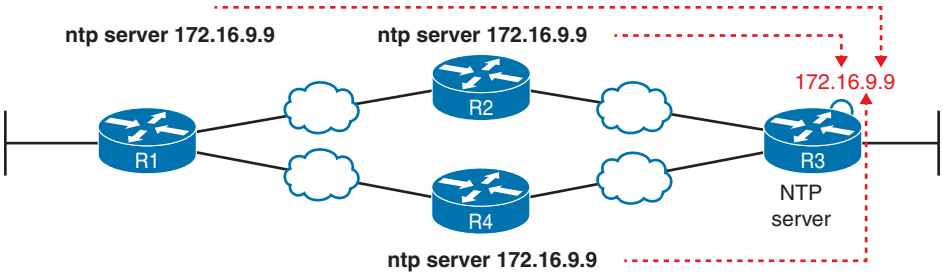



**Figure D-8** *NTP Clients Referring to NTP Server R3's Loopback IP Address 172.16.9.9*

**Example D-13** *NTP Client/Server Configuration on R1 and R2 Using a Loopback Interface*

```
! Configuration on R1, R2, and R4, all NTP clients
ntp server 172.16.9.9
```

```
! Configuration on R3 for its server function
interface loopback 0
 ip address 172.16.9.9 255.255.255.0
!
ntp master 4
ntp source loopback 0
```

```
! Verification on router R3
R3# show interfaces loopback 0
Loopback0 is up, line protocol is up
 Hardware is Loopback
 Internet address is 172.16.9.9/24
! lines omitted for brevity
```

Loopback interfaces have a wide range of uses across IOS features. They are mentioned here with NTP because NTP is a feature that can benefit from using loopback interfaces. (As a reminder, OSPF happens to use loopback interfaces with OSPF configuration for a completely different purpose.)

**NOTE** The content under the heading "Metro Ethernet" was most recently published for the 200-105 Exam in 2016, in Chapter 14 of the *Cisco CCNA ICND2 200-105 Official Cert Guide*.

# Metro Ethernet

This section discusses virtual circuits in Ethernet WANs.

## Ethernet Virtual Circuit Bandwidth Profiles

Before leaving MetroE to move on to MPLS, it helps to consider some ideas about data usage over the WAN links and a whole topic area related to EVC Bandwidth Profiles (BWP).

First, ignoring MetroE for a moment, anyone who has shopped for mobile phone data plans in the 2010s has already thought about data usage with carrier networks. With mobile phones, many carriers offer some kind of tiered pricing: the more data you want to send and receive, the more money you spend per month. Why do they charge more based on usage? The SP spends a lot of capital and a lot of ongoing operational expense to build and operate its network. It seems fair to charge those who use less of the network a little less money, and those who use more a little more money. Simple enough.

Most private WAN services use the same kind of usage-based pricing, and this last MetroE topic discusses some of the terminology and concepts.

The first big idea is this: The access links transmit bits at a set predefined speed based on Ethernet standards. Each Ethernet access link on a MetroE WAN uses a specific Ethernet standard that runs at a specific speed. Those speeds are 10 Mbps, 100 Mbps, 1000 Mbps (that is, 1 Gbps), 10 Gbps, and so on. And while the IEEE has begun adding some new speeds for Ethernet standards, speeds that are not a multiple of 10 versus the next slower speed, the point is this: If a site's MetroE access link is using an Ethernet standard that is a 100-Mbps standard, then the bits are transmitted at 100 Mbps.



At the same time, the MetroE SP wants to be able to charge customers based on usage, and to be a little more flexible than pricing based on the speed of the access links. These final few pages of the MetroE topics in this chapter show how a MetroE SP can charge for speeds other than the access link speeds.

### Charging for the Data (Bandwidth) Used

Think through this scenario. A potential customer looks at a MetroE provider's pricing. This customer wants an E-Line service between two sites only. They know that they need at least 100 Mbps of capacity (that is, bandwidth) between the sites. But because the service has the word *Ethernet* in it, the potential customer thinks the service is either 10 Mbps, 100 Mbps, 1 Gbps, and so on. So they look up pricing for an E-Line service at those prices, and think

- **100 Mbps:** Reasonably good price, but we need more capacity
- **1000 Mbps:** More than we want to spend, it's enough capacity, but probably too much

As it turns out, what this customer really wants is 200 Mbps between the two sites. However, there is no Ethernet standard that runs at 200 Mbps, so there is no way to use access links that run at 200 Mbps. But there is a solution: an E-Line service, with a Bandwidth Profile that defines a 200-Mbps committed information rate (CIR) over the point-to-point EVC between the customer's two routers. Figure D-9 shows the ideas and terms.

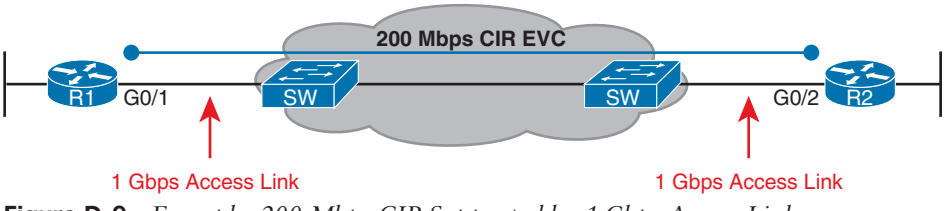



**Figure D-9** *Example: 200-Mbps CIR Supported by 1-Gbps Access Links*

The big ideas are simple, although the methods to control the data are new. The SP, per the contract with the customer, agrees to not only forward Ethernet frames between the two E-Line sites but also commits to a CIR of 200 Mbps. That is, the carrier commits to pass 200 Mbps worth of Ethernet frames over time.

When a customer asks for a new E-Line with a 200-Mbps CIR, they could send lots more data than 200 Mbps. Remember, the literal transmission rate would be 1 Gbps in this example, because the access links are 1-Gbps links. But over time, if all the customers that asked for a 200-Mbps CIR E-Line sent lots more than 200 Mbps worth of data, the SP's network could become too congested. The SP builds its network to support the traffic it has committed to send, plus some extra for expected overuse, and some extra for growth. But it is too expensive to build a network that allows customers that ask for and pay for 200 Mbps to send at 1 Gbps all the time.

### Controlling Overages with Policing and Shaping

To make the idea of fast access links with a slower CIR on the EVCs work, and work well, both the SP and the customer have to cooperate. The tools are two Quality of Service (QoS) tools called policing and shaping.

Historically, in some similar WAN services (like Frame Relay), the SP would actually let you send more data than your CIR, but MetroE networks typically use policing to discard the excess. A policer can watch incoming frames and identify the frames associated with each EVC. It counts the bytes in each frame, and determines a bit rate over time. When the customer has sent more bits than the CIR, the SP discards enough of the currently arriving frames to keep the rate down to the CIR. Figure D-10 shows the location of policing in the same example shown in Figure D-9.

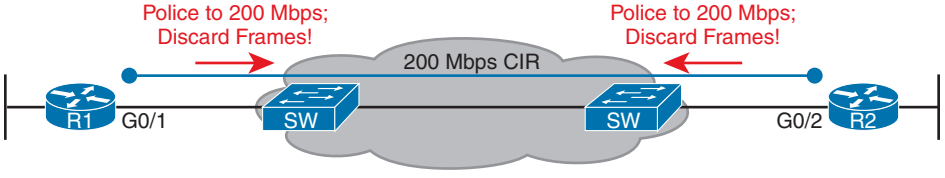



**Figure D-10** *SP Polices Incoming Traffic to Discard Excess Beyond CIR*

Recapping this scenario, the customer decides to ask the MetroE SP for an E-Line. The customer's routers use a 1-Gbps access link that allows the E-Line to support a 200-Mbps CIR. To protect the SP's network, the SP now uses ingress policing to monitor the bits/second received over each end of the E-Line's point-to-point EVC. And the SP discards some incoming frames when the rate gets too high.

Having the SP discard a few frames is actually not that harmful if QoS is implemented correctly, but with MetroE, if the SP is policing as shown in Figure D-10, the customer needs to use the other QoS tool: shaping. Shaping, as implemented on the customer routers, lets the routers slow down. Shaping tells the routers, on the MetroE access link, to send some frames, and then wait; then send more, then wait; and to do that repeatedly. Shaping can be configured for that same rate as the CIR (200 Mbps in this case), so that the SP does not have to discard any traffic.

Summarizing some of these key points:

- MetroE uses the concept of an Ethernet Virtual Connection (EVC), tying a committed number of bits/second called the committed information rate (CIR) to the EVC.
- The access links need to be fast enough to handle the combined CIRs for all EVCs that cross the link.
- For each EVC, the SP commits to forward the bits/second defined as the CIR for that EVC.
- To protect its network from being overrun with too much traffic, the SP can use policing, monitoring the incoming traffic rate on each EVC and discarding traffic that goes beyond the CIR.
- To prevent too much of its traffic from being discarded by the SP, the customer slows down its rate of sending over the EVC to match that same CIR, using shaping on the customer router.



**NOTE** The content under the heading "MPLS VPNs" was most recently published for the 200-105 Exam in 2016, in Chapter 14 of the *Cisco CCNA ICND2 200-105 Official Cert Guide*.

# MPLS VPNs

This section discusses an OSPF design issue that exists when using MPLS VPNs.

## OSPF Area Design with MPLS VPN

Now that you know the basics about what happens with routing protocols at the edge of an MPLS network, take a step back and ponder OSPF area design. For all the other WAN services discussed in the book, the WAN service is just one more data link, so the WAN sits inside one area. With MPLS, the MPLS service acts like a bunch of routers. If you use OSPF as the PE-CE routing protocol, some choices must be made about OSPF areas, and about which WAN links are in which area, and where the backbone area can and should be.

MPLS allows for a couple of variations on OSPF area design, but they all use an idea that was added to OSPF for MPLS VPNs, an idea that has come to be known informally as the OSPF *super backbone*. The idea is an elegant solution that meets OSPF needs and the requirement that the MPLS PEs, when using OSPF, must be in some OSPF area:

- The MPLS PEs form a backbone area by the name of a super backbone.
- Each PE-CE link can be any area—a non-backbone area or the backbone area.

Although the super backbone supports some functions and logic beyond the scope of this book, for the purposes of getting a basic understanding of OSPF's use with MPLS, you can think of the super backbone as simply the majority of an enterprise's OSPF backbone area, but with the option to make the backbone area larger. The CE routers at a customer site may not be part of the backbone area, or may be, at the choice of the customer network engineers.

For example, for a nice clean design, each of the four customer sites in Figure D-11 uses a different area. The PE-CE links are part of those individual areas. The OSPF backbone area still exists, and each area connects to the backbone area, but the backbone exists in the MPLS PE routers only.

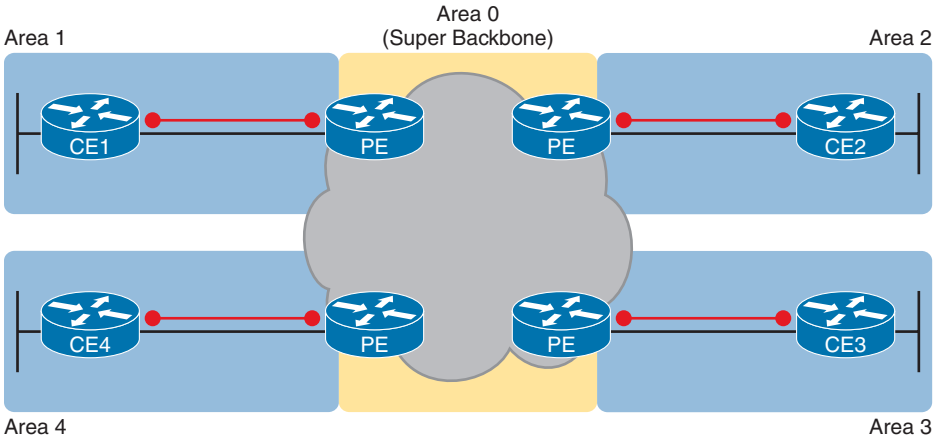



**Figure D-11** *MPLS Design with (Super Backbone) Area 0, Non-Backbone Area for Each Site*

The area design in Figure D-11 provides a clean OSPF area design. However, if migrating from some other type of WAN service, with an existing OSPF design, the network engineers may prefer to keep parts of an existing OSPF design, which means some sites may still need to include the backbone area. In fact, multiple WAN sites can be configured to be in the backbone area, and still function correctly. Figure D-12 shows one such example.

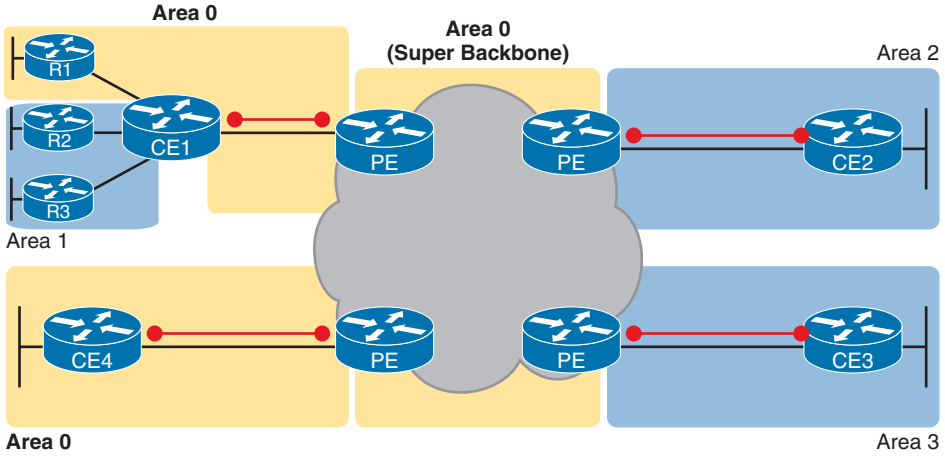



**Figure D-12** *Using Area 0 on CE-PE Link, or for Entire Site*

In effect, the super backbone combines with the two other parts of the network configured as area 0 for one contiguous backbone area. Notice on the left side of Figure D-12 the two sites with area 0 noted. Normally, if both customer sites implement area 0, but there were links from some other area between them, the design would break OSPF design rules.

However, the OSPF backbone (area 0) links on the left, plus the OSPF super backbone area 0 created by MPLS, act together in regard to OSPF design.

Next, focus on the site at the upper left. That site represents what might have existed before migrating to an MPLS design, with Router R1's links in area 0, and the links connected to Routers R2 and R3 in area 1. The enterprise network engineer may have decided to leave the OSPF area design alone when connecting to the MPLS network. To support those backbone area links off Router R1, the engineer put the CE1-PE1 link into area 0. As a result, the combined customer area 0 instances and the super backbone area 0 creates one contiguous backbone area.

**NOTE** The content under the heading "A Scenario: Branch Offices and the Public Cloud" was most recently published for the 200-301 Exam blueprint 1.0, in the 2019 book *Cisco CCNA 200-301 Official Cert Guide, Volume 2,* Chapter 15, "Cloud Architecture."



## A Scenario: Branch Offices and the Public Cloud

So far in this major section about WAN design with public cloud, the enterprise has been shown as one entity, but most enterprise WANs have many sites. Those distributed enterprise sites impact some parts of WAN design for public cloud. The next discussion of WAN design issues with public cloud works through a scenario that shows an enterprise with a typical central site and branch office.

The example used in this section is a common one: the movement away from internal email servers, supported directly by the IT staff, to email delivered as a SaaS offering. Focus on the impact of the enterprise's remote sites like branch offices.

### Migrating Traffic Flows When Migrating to Email SaaS

First, think of the traffic flow inside an enterprise before SaaS, when the company buys servers, licenses email server software, installs the hardware and software in an internal data center, and so on. The company may have hundreds or thousands of remote sites, like the branch office shown in Figure D-13. To check email, an employee at the branch office sends packets back and forth with the email server at the central site, as shown.

The company then looks at the many different costs for email in this old model versus the new SaaS model. For instance, Microsoft Exchange is a very popular software package to build those enterprise email servers. Microsoft, a major player in the public cloud space with its Microsoft Azure service, offers Exchange as a SaaS service. (During the writing of this book, this particular service could be found as part of Office 365 or as "Exchange Online.") So the enterprise considers the options and chooses to migrate an email SaaS offering.

Once migrated, the email servers run in the cloud, but as a SaaS service. The enterprise IT staff, who are the customers of the SaaS service, do not have to manage the servers. Just to circle back to some big ideas, with a SaaS service, the consumer does not worry about installing VMs, sizing them, installing Exchange or some other email server software, and so on. The consumer receives email service in this case. The company does have to do some migration work to move existing email, contacts, and so on, but once completed, all users now communicate with email servers that run in the cloud as a SaaS service.

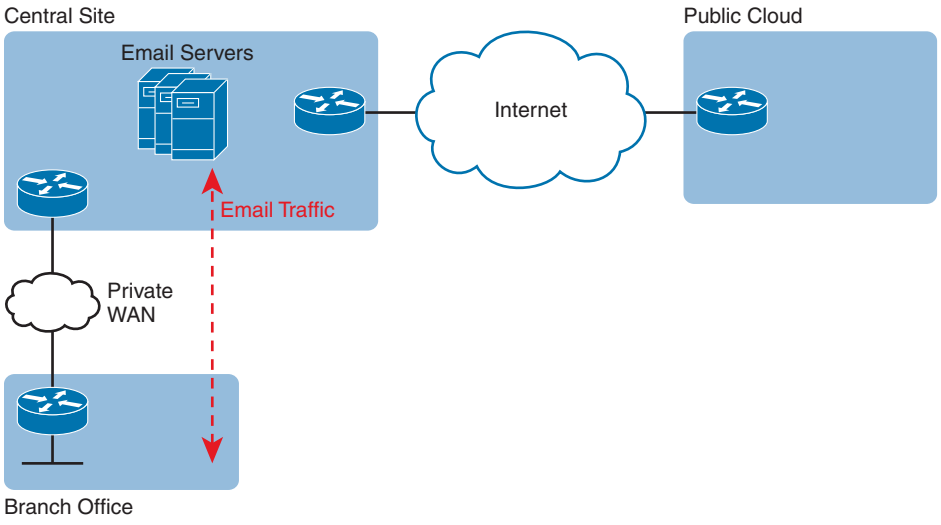



**Figure D-13** *Traffic Flow: Private WAN, Enterprise Implements Email Services*

Now think about that enterprise branch office user, and the traffic flows shown in Figure D-14, when a branch user sends or receives an email. For instance, think of an email with a large attachment, just to make the impact more dramatic. If the enterprise design connects branches to the central sites only, this is the net effect on WAN traffic:

- No reduction in private WAN traffic at all occurs because all the branch office email traffic flows to/from the central site.
- One hundred percent of the email traffic (even internal emails) that flows to/from branches now also flows over the Internet connection, consuming the bandwidth of the enterprise's Internet links.

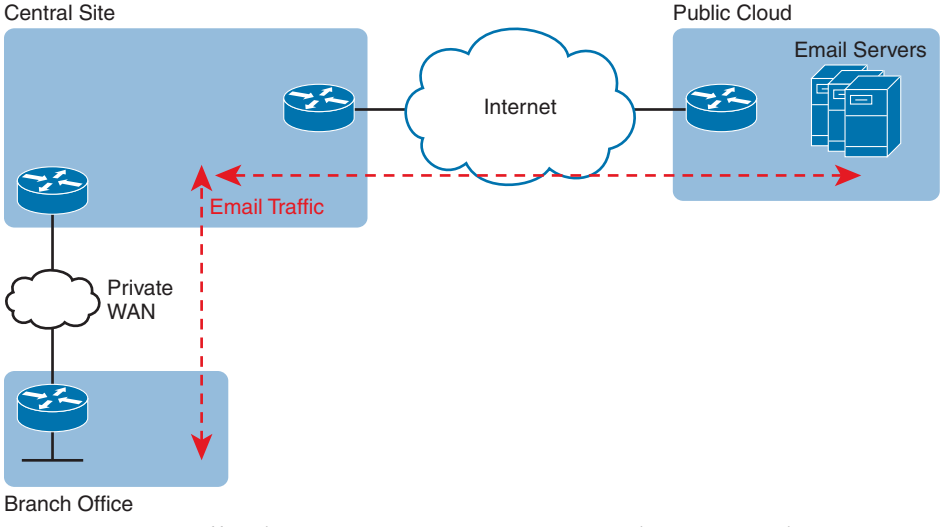



**Figure D-14** *Traffic Flow: Private WAN, Enterprise Implements Email Services*

Just to make the point, imagine two users at the same branch office. They can see each other across the room. One wants to share a file with the other, but the most convenient way they know to share a file is to email the file as an attachment. So one of them sends an email to the other, attaching the 20-MB file to the email. Before using SaaS, with an email server at the central site, that email and file would flow over the private WAN, to the email server, and then back to the second user's email client. With this new design, that email with the 20-MB attachment would flow over the private WAN, then over the Internet to the email server, and then back again over the Internet and over the private WAN when the second user downloads her email.

### Branch Offices with Internet and Private WAN

For enterprises that place their Internet connections primarily at the central sites, this public cloud model can cause problems like the one just described. One way to deal with this particular challenge is to plan the right capacity for the Internet links; another is to plan capacity for some private WAN connections to the public cloud. Another option exists as well: redesign the enterprise WAN to a small degree, and consider placing direct Internet connections at the branch offices. Then all Internet traffic, including the email traffic to the new SaaS service, could be sent directly, and not consume the private WAN bandwidth or the central site Internet link bandwidth, as shown in Figure D-15.



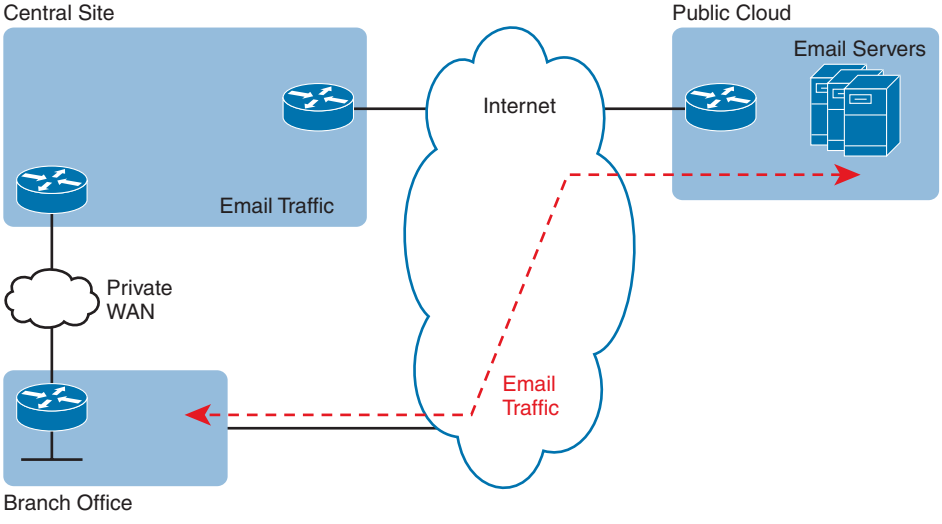



**Figure D-15** *Connecting Branches Directly to the Internet for Public Cloud Traffic*

The design in Figure D-15 has several advantages. The traffic flows much more directly. It does not waste the WAN bandwidth for the central site. And broadband Internet connections are relatively inexpensive today compared to private WAN connections.

However, when the per-branch Internet connections are added for the first time, the new Internet links create security concerns. One of the reasons an enterprise might use only a few Internet links, located at a central site, is to focus the security efforts at those links. Using an Internet connection at each branch changes that approach. But many enterprises not only use the Internet at each site but also rely on it as their only WAN connection, as shown with Internet VPNs back in Chapter 19, "WAN Architecture."

*This page intentionally left blank*

# APPENDIX E

# Practice for Chapter 6: Basic IPv4 Access Control Lists

## Practice Problems

This appendix includes two sets of practice problems. The first question set lists requirements for a single-line access control list (ACL), with your task being to create a standard numbered ACL that meets the requirements. The second question set shows an existing **access-list** command, with your job being to determine the range of IP addresses matched by the ACL.

Note that you can find additional practice on the author's blog, which is linked from the author's website, www.certskills.com.

### Practice Building access-list Commands

Table E-1 lists the criteria for several practice problems. Your job: Create a one-line standard ACL that matches the packets. The answers are listed later in this appendix.

**Table E-1** Building One-Line Standard ACLs: Practice

| Problem | Criteria |
|---|---|
| 1 | Packets from 10.1.1.1 |
| 2 | Packets from hosts with 10.1.1 as the first 3 octets |
| 3 | Packets from hosts with 10.1 as the first 2 octets |
| 4 | Packets from any host |
| 5 | Packets from subnet 192.168.3.128/29 |
| 6 | Packets from subnet 192.168.3.192/28 |
| 7 | Packets from subnet 192.168.3.64/27 |
| 8 | Packets from subnet 172.20.192.192/26 |
| 9 | Packets from subnet 172.20.200.0/22 |
| 10 | Packets from subnet 172.20.203.0/25 |
| 11 | Packet from subnet 192.168.99.0/30 |
| 12 | Packet from subnet 192.168.99.0/28 |
| 13 | Packet from subnet 172.28.28.0/23 |
| 14 | Packet from subnet 172.28.28.0/22 |
| 15 | Packet from subnet 172.28.28.0/24 |

## Reverse Engineering from ACL to Address Range

For this second question set, look at the existing **access-list** commands in Table E-2. In each case, make a notation about the exact IP address, or range of IP addresses, matched by the command.

**Table E-2** Finding IP Addresses/Ranges Matching by Existing ACLs

| Problem | Commands for Which to Predict the Source Address Range |
|---|---|
| 1 | access-list 1 permit 192.168.4.5 |
| 2 | access-list 2 permit 192.168.4.128 0.0.0.3 |
| 3 | access-list 3 permit 192.168.4.128 0.0.0.127 |
| 4 | access-list 4 permit 172.25.96.0 0.0.0.255 |
| 5 | access-list 5 permit 192.168.4.128 0.0.0.31 |
| 6 | access-list 6 permit 192.168.4.128 0.0.0.7 |
| 7 | access-list 7 permit 172.25.96.0 0.0.7.255 |
| 8 | access-list 8 permit 172.25.96.0 0.0.0.63 |
| 9 | access-list 9 permit 10.10.16.0 0.0.7.255 |
| 10 | access-list 10 permit 10.10.16.0 0.0.0.127 |
| 11 | access-list 11 permit 192.168.17.112 0.0.0.7 |
| 12 | access-list 12 permit 192.168.17.112 0.0.0.15 |
| 13 | access-list 13 permit 172.19.200.0 0.0.0.63 |
| 14 | access-list 14 permit 172.19.200.0 0.0.1.255 |
| 15 | access-list 15 permit 10.1.0.0 0.0.255.255 |

**NOTE** You can only rely on the method of adding these numbers together (as shown in Chapter 6, "Basic IPv4 Access Control Lists") if you know that the **access-list** command comes from the router and specifically is not what someone simply wrote on a piece of paper. In this case, you can assume that the statements in Table E-2 came from a router.

# Answers to Earlier Practice Problems

This section contains the answers to the two sets of practice problems.

## Answers: Practice Building access-list Commands

Table E-3 lists the answers to the problems listed in Table E-1.

**Table E-3** Building One-Line Standard ACLs: Answers

| Problem | Answer |
|---|---|
| 1 | access-list 1 permit 10.1.1.1 |
| 2 | access-list 2 permit 10.1.1.0 0.0.0.255 |
| 3 | access-list 3 permit 10.1.0.0 0.0.255.255 |
| 4 | access-list 4 permit any |
| 5 | access-list 5 permit 192.168.3.128 0.0.0.7 |
| 6 | access-list 6 permit 192.168.3.192 0.0.0.15 |
| 7 | access-list 7 permit 192.168.3.64 0.0.0.31 |
| 8 | access-list 8 permit 172.20.192.192 0.0.0.63 |
| 9 | access-list 9 permit 172.20.200.0 0.0.3.255 |
| 10 | access-list 10 permit 172.20.203.0 0.0.0.127 |
| 11 | access-list 11 permit 192.168.99.0 0.0.0.3 |
| 12 | access-list 12 permit 192.168.99.0 0.0.0.15 |
| 13 | access-list 13 permit 172.28.28.0 0.0.1.255 |
| 14 | access-list 14 permit 172.28.28.0 0.0.3.255 |
| 15 | access-list 15 permit 172.28.28.0 0.0.0.255 |

## Answers: Reverse Engineering from ACL to Address Range

Table E-4 lists the answers to the problems listed in Table E-2.

**Table E-4** Address Ranges for Problems in Table E-2: Answers

| Problem | Address Range |
|---|---|
| 1 | One address: 192.168.4.5 |
| 2 | 192.168.4.128 – 192.168.4.131 |
| 3 | 192.168.4.128 – 192.168.4.255 |
| 4 | 172.25.96.0 – 172.25.96.255 |
| 5 | 192.168.4.128 – 192.168.4.159 |
| 6 | 192.168.4.128 – 192.168.4.135 |
| 7 | 172.25.96.0 – 172.25.103.255 |
| 8 | 172.25.96.0 – 172.25.96.63 |
| 9 | 10.10.16.0 – 10.10.23.255 |
| 10 | 10.10.16.0 – 10.10.16.127 |
| 11 | 192.168.17.112 – 192.168.17.119 |

| Problem | Address Range |
|---|---|
| 12 | 192.168.17.112 – 192.168.17.127 |
| 13 | 172.19.200.0 – 172.19.200.63 |
| 14 | 172.19.200.0 – 172.19.201.255 |
| 15 | 10.1.0.0 – 10.1.255.255 |

*This page intentionally left blank*

# CCNA 200-301 Official Cert Guide, Volume 2 Appendix F: Study Planner

**Key:**

| |
|---|
| Practice Test |
| Reading |
| Review |
| Video |
| Labs |

| Element | Task | Goal Date | First Date Completed | Second Date Completed (Optional) | Notes |
|---|---|---|---|---|---|
| Introduction | Read Introduction | | | | |
| 1. Fundamentals of Wireless Networks | Read Foundation Topics | | | | |
| 1. Fundamentals of Wireless Networks | Review Key Topics using the book or companion website | | | | |
| 1. Fundamentals of Wireless Networks | Define Key Terms using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 1. Fundamentals of Wireless Networks | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 1. Fundamentals of Wireless Networks | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 2. Analyzing Cisco Wireless Architectures | Read Foundation Topics | | | | |
| 2. Analyzing Cisco Wireless Architectures | Review Key Topics using the book or companion website | | | | |
| 2. Analyzing Cisco Wireless Architectures | Define Key Terms using the book or companion website | | | | |
| 2. Analyzing Cisco Wireless Architectures | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 2. Analyzing Cisco Wireless Architectures | Complete all memory tables in this chapter using the companion website | | | | |
| 2. Analyzing Cisco Wireless Architectures | Watch chapter-associated videos using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 3. Securing Wireless Networks | Read Foundation Topics | | | | |
| 3. Securing Wireless Networks | Review Key Topics using the book or companion website | | | | |
| 3. Securing Wireless Networks | Define Key Terms using the book or companion website | | | | |
| 3. Securing Wireless Networks | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 3. Securing Wireless Networks | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 4. Building a Wireless LAN | Read Foundation Topics | | | | |
| 4. Building a Wireless LAN | Review Key Topics using the book or companion website | | | | |
| 4. Building a Wireless LAN | Define Key Terms using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 4. Building a Wireless LAN | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 4. Building a Wireless LAN | Complete all memory tables in this chapter using the companion website | | | | |
| 4. Building a Wireless LAN | Watch chapter-associated videos using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| Part I. Wireless LANs | Complete all exercises in Part I Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 5. Introduction to TCP/IP Transport and Applications | Read Foundation Topics | | | | |
| 5. Introduction to TCP/IP Transport and Applications | Review Key Topics using the book or companion website | | | | |
| 5. Introduction to TCP/IP Transport and Applications | Define Key Terms using the book or companion website | | | | |
| 5. Introduction to TCP/IP Transport and Applications | Repeat DIKTA questions using the book or PTP exam engine | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 5. Introduction to TCP/IP Transport and Applications | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 6. Basic IPv4 Access Control Lists | Read Foundation Topics | | | | |
| 6. Basic IPv4 Access Control Lists | Review Key Topics using the book or companion website | | | | |
| 6. Basic IPv4 Access Control Lists | Define Key Terms using the book or companion website | | | | |
| 6. Basic IPv4 Access Control Lists | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 6. Basic IPv4 Access Control Lists | Review command tables for this chapter | | | | |
| 6. Basic IPv4 Access Control Lists | Complete all memory tables in this chapter using the companion website | | | | |
| 6. Basic IPv4 Access Control Lists | Complete practice exercise: Create a one-line standard ACL that matches the packets (interactive version of Appendix E) using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 6. Basic IPv4 Access Control Lists | Complete practice exercise: Matching IP addresses to access-list commands (interactive version of Appendix E) | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 7. Named and Extended IP ACLs | Read Foundation Topics | | | | |
| 7. Named and Extended IP ACLs | Review Key Topics using the book or companion website | | | | |
| 7. Named and Extended IP ACLs | Define Key Terms using the book or companion website | | | | |
| 7. Named and Extended IP ACLs | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 7. Named and Extended IP ACLs | Review command tables for this chapter | | | | |
| 7. Named and Extended IP ACLs | Complete all memory tables in this chapter using the companion website | | | | |
| 7. Named and Extended IP ACLs | Watch chapter-associated video using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 8. Applied IP ACLs | Read Foundation Topics | | | | |
| 8. Applied IP ACLs | Review Key Topics using the book or companion website | | | | |
| 8. Applied IP ACLs | Define Key Terms using the book or companion website | | | | |
| 8. Applied IP ACLs | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 8. Applied IP ACLs | Review command tables for this chapter | | | | |
| 8. Applied IP ACLs | Complete all memory tables in this chapter using the companion website | | | | |
| 8. Applied IP ACLs | Watch chapter-associated video using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| Part II. IP Access Control Lists | Complete all exercises in Part II Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 9. Security Architectures | Read Foundation Topics | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 9. Security Architectures | Review Key Topics using the book or companion website | | | | |
| 9. Security Architectures | Define Key Terms using the book or companion website | | | | |
| 9. Security Architectures | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 9. Security Architectures | Complete all memory tables in this chapter using the companion website | | | | |
| 9. Security Architectures | Watch chapter-associated video using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 10. Securing Network Devices | Read Foundation Topics | | | | |
| 10. Securing Network Devices | Review Key Topics using the book or companion website | | | | |
| 10. Securing Network Devices | Define Key Terms using the book or companion website | | | | |
| 10. Securing Network Devices | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 10. Securing Network Devices | Review command tables for this chapter | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 10. Securing Network Devices | Watch chapter-associated videos using the companion website | | | | |
| 10. Securing Network Devices | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 11. Implementing Switch Port Security | Read Foundation Topics | | | | |
| 11. Implementing Switch Port Security | Review Key Topics using the book or companion website | | | | |
| 11. Implementing Switch Port Security | Define Key Terms using the book or companion website | | | | |
| 11. Implementing Switch Port Security | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 11. Implementing Switch Port Security | Review command tables for this chapter | | | | |
| 11. Implementing Switch Port Security | Complete all memory tables in this chapter using the companion website | | | | |
| 11. Implementing Switch Port Security | Complete Config Checklist: Port Security Configuration in this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 11. Implementing Switch Port Security | Watch chapter-associated video using the companion website | | | | |
| 11. Implementing Switch Port Security | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 12. DHCP Snooping and ARP Inspection | Read Foundation Topics | | | | |
| 12. DHCP Snooping and ARP Inspection | Review Key Topics using the book or companion website | | | | |
| 12. DHCP Snooping and ARP Inspection | Define Key Terms using the book or companion website | | | | |
| 12. DHCP Snooping and ARP Inspection | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 12. DHCP Snooping and ARP Inspection | Review command tables for this chapter | | | | |
| 12. DHCP Snooping and ARP Inspection | Complete Config Checklist: Configuring DHCP Snooping in this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 12. DHCP Snooping and ARP Inspection | Complete Config Checklist: Configuring Dynamic IP ARP Inspection in this chapter using the companion website | | | | |
| 12. DHCP Snooping and ARP Inspection | Watch chapter-associated videos using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| Part III. Security Services | Complete all exercises in Part III Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 13. Device Management Protocols | Read Foundation Topics | | | | |
| 13. Device Management Protocols | Review Key Topics using the book or companion website | | | | |
| 13. Device Management Protocols | Define Key Terms using the book or companion website | | | | |
| 13. Device Management Protocols | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 13. Device Management Protocols | Review command tables for this chapter | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 13. Device Management Protocols | Complete all memory tables in this chapter using the companion website | | | | |
| 13. Device Management Protocols | Watch chapter-associated videos using the companion website | | | | |
| 13. Device Management Protocols | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 14. Network Address Translation | Read Foundation Topics | | | | |
| 14. Network Address Translation | Review Key Topics using the book or companion website | | | | |
| 14. Network Address Translation | Define Key Terms using the book or companion website | | | | |
| 14. Network Address Translation | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 14. Network Address Translation | Review command tables for this chapter | | | | |
| 14. Network Address Translation | Complete all memory tables in this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 14. Network Address Translation | Complete Config Checklist: Static NAT Configuration in this chapter using the companion website | | | | |
| 14. Network Address Translation | Complete Config Checklist: Configure Dynamic NAT using the companion website | | | | |
| 14. Network Address Translation | Complete Config Checklist: Use Interface IP Address as the Sole Inside Global IP Address in this chapter using the companion website | | | | |
| 14. Network Address Translation | Watch chapter-associated video using the companion website | | | | |
| 14. Network Address Translation | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 15. Quality of Service (QoS) | Read Foundation Topics | | | | |
| 15. Quality of Service (QoS) | Review Key Topics using the book or companion website | | | | |
| 15. Quality of Service (QoS) | Define Key Terms using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 15. Quality of Service (QoS) | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 15. Quality of Service (QoS) | Complete all memory tables in this chapter using the companion website | | | | |
| 15. Quality of Service (QoS) | Watch chapter-associated video using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 16. First Hop Redundancy Protocols | Read Foundation Topics | | | | |
| 16. First Hop Redundancy Protocols | Review Key Topics using the book or companion website | | | | |
| 16. First Hop Redundancy Protocols | Define Key Terms using the book or companion website | | | | |
| 16. First Hop Redundancy Protocols | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 16. First Hop Redundancy Protocols | Complete all memory tables in this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 17. SNMP, FTP, and TFTP | Read Foundation Topics | | | | |
| 17. SNMP, FTP, and TFTP | Review Key Topics using the book or companion website | | | | |
| 17. SNMP, FTP, and TFTP | Define Key Terms using the book or companion website | | | | |
| 17. SNMP, FTP, and TFTP | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 17. SNMP, FTP, and TFTP | Review command tables for this chapter | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| Part IV. IP Services | Complete all exercises in Part IV Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 18. LAN Architecture | Read Foundation Topics | | | | |
| 18. LAN Architecture | Review Key Topics using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 18. LAN Architecture | Define Key Terms using the book or companion website | | | | |
| 18. LAN Architecture | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 18. LAN Architecture | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 19. WAN Architecture | Read Foundation Topics | | | | |
| 19. WAN Architecture | Review Key Topics using the book or companion website | | | | |
| 19. WAN Architecture | Define Key Terms using the book or companion website | | | | |
| 19. WAN Architecture | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 19. WAN Architecture | Complete all memory tables in this chapter using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using DIKTA exam in practice test software for this chapter | | | | |
| 20. Cloud Architecture | Read Foundation Topics | | | | |
| 20. Cloud Architecture | Review Key Topics using the book or companion website | | | | |
| 20. Cloud Architecture | Define Key Terms using the book or companion website | | | | |
| 20. Cloud Architecture | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 20. Cloud Architecture | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| Part V. Network Architecture | Complete all exercises in Part V Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 21. Introduction to Controller-Based Networking | Read Foundation Topics | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 21. Introduction to Controller-Based Networking | Review Key Topics using the book or companion website | | | | |
| 21. Introduction to Controller-Based Networking | Define Key Terms using the book or companion website | | | | |
| 21. Introduction to Controller-Based Networking | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 21. Introduction to Controller-Based Networking | Complete all memory tables in this chapter using the companion website | | | | |
| Practice Test | Take practice test in study mode using Exam #1 in practice test software for this chapter | | | | |
| 22. Cisco Software-Defined Access (SDA) | Read Foundation Topics | | | | |
| 22. Cisco Software-Defined Access (SDA) | Review Key Topics using the book or companion website | | | | |
| 22. Cisco Software-Defined Access (SDA) | Define Key Terms using the book or companion website | | | | |
| 22. Cisco Software-Defined Access (SDA) | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 22. Cisco Software-Defined Access (SDA) | Watch chapter-associated video using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 23. Understanding REST and JSON | Read Foundation Topics | | | | |
| 23. Understanding REST and JSON | Review Key Topics using the book or companion website | | | | |
| 23. Understanding REST and JSON | Define Key Terms using the book or companion website | | | | |
| 23. Understanding REST and JSON | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 23. Understanding REST and JSON | Complete all memory tables in this chapter using the companion website | | | | |
| 23. Understanding REST and JSON | Watch chapter-associated videos using the companion website | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| 24. Understanding Ansible and Terraform | Read Foundation Topics | | | | |
| 24. Understanding Ansible and Terraform | Review Key Topics using the book or companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 24. Understanding Ansible and Terraform | Define Key Terms using the book or companion website | | | | |
| 24. Understanding Ansible and Terraform | Repeat DIKTA questions using the book or PTP exam engine | | | | |
| 24. Understanding Ansible and Terraform | Complete all memory tables in this chapter using the companion website | | | | |
| 24. Understanding Ansible and Terraform | Do labs listed for this chapter using the Sim Lite app and/or at blog.certskills.com | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| Part VI. Network Automation | Complete all exercises in Part VI Review | | | | |
| Practice Test | Take practice test in study mode using Part Review exam in practice test software for this part | | | | |
| Final Review | Take practice test in study mode for all book questions in practice test software | | | | |
| Final Review | Review all Key Topics in all chapters or in the Key Topics app using the companion website | | | | |

| | | | | | |
|---|---|---|---|---|---|
| Final Review | Review all Key Terms in all chapters or using the Key Terms flashcards on the companion website | | | | |
| Final Review | Complete all memory tables for all chapters using the companion website | | | | |
| Final Review | Complete all config checklists for all chapters using the companion website | | | | |
| Final Review | Do all hands-on labs using the Sim Lite app | | | | |
| Final Review | Take practice test in practice exam mode using Exam Bank #1 questions for all chapters | | | | |
| Final Review | Take practice test in practice exam mode using Exam Bank #2 questions for all chapters | | | | |
| CCNA Final Review | Read Final Review Chapter | | | | |
| CCNA Final Review | Take practice test in practice exam mode using Full CCNA Exam Bank #1 | | | | |

| | | | | | |
|---|---|---|---|---|---|
| CCNA Final Review | Identify areas of weakness from full CCNA practice test. Return to the chapters in volume 1 or volume 2 for review. | | | | |
| CCNA Final Review | Take practice test in practice exam mode using Full CCNA Exam Bank #2 | | | | |

# GLOSSARY

## NUMERICS

**2.4-GHz band** The frequency range between 2.400 and 2.4835 GHz that is used for wireless LAN communication.

**2.5GBase-T** One of two Ethernet physical layer standards called multigigabit Ethernet (the other being 5.0GBase-T), first defined in IEEE addendum 802.3bz, which defines a 2.5 Gbps data rate over Cat 5E UTP cabling at distances of 100 meters.

**3G/4G Internet** An Internet access technology that uses wireless radio signals to communicate through mobile phone towers, most often used by mobile phones, tablets, and some other mobile devices.

**5-GHz band** The frequency range between 5.150 and 5.825 GHz that is used for wireless LAN communication.

**5.0GBase-T** One of two Ethernet physical layer standards called multigigabit Ethernet (the other being 2.5GBase-T), first defined in IEEE addendum 802.3bz, which defines a 2.5 Gbps data rate over Cat 5E UTP cabling at distances of 100 meters.

**6-GHz band** The frequency range between 5.925 and 7.125 GHz that is used for wireless LAN communication.

**10BASE-T** An Ethernet physical layer standard, first defined directly in the 802.3 standard as the first UTP-based Ethernet physical layer standard. It uses two twisted-pair UTP cabling and supports 10 Mbps data rates.

**10GBASE-SR** An Ethernet physical layer standard that uses optical multimode (OM) cabling, first defined in IEEE addendum 802.3ae, which defines a 10 Gbps data rate.

**10GBase-T** An Ethernet physical layer standard, introduced as IEEE addendum 802.3an, supporting 10 Gbps data rates over four-pair UTP cabling.

**40GBase-T** An Ethernet physical layer standard, introduced in IEEE addendum 802.3ba, supporting 40 Gbps data rates over four-pair UTP cabling.

**100BASE-T** An Ethernet physical layer standard, introduced as IEEE addendum 802.3u, supporting 100 Mbps data rates over two-pair UTP cabling.

**1000BASE-SX** An Ethernet physical layer standard that uses optical multimode (OM) cabling, first defined in IEEE addendum 802.3z, which defines a 1 Gbps data rate.

**1000BASE-T** An Ethernet physical layer standard, introduced as IEEE addendum 802.3ab, supporting 1000 Mbps (1 Gbps) data rates over four-pair UTP cabling.

# A

**AAA** Authentication, authorization, and accounting. Authentication confirms the identity of the user or device. Authorization determines what the user or device is allowed to do. Accounting records information about access attempts, including inappropriate requests.

**AAA server** *See* authentication, authorization, and accounting (AAA) server.

**access control entry (ACE)** One configuration line with a permit or deny action in an access control list (ACL).

**access interface** A LAN network design term that refers to a switch interface connected to end-user devices, configured so that it does not use VLAN trunking.

**access layer** In a campus LAN design, the switches that connect directly to endpoint devices (servers, user devices), and also connect into the distribution layer switches.

**access link** In campus LAN design, a link that connects an access switch to endpoint devices and wireless access points.

**access link (WAN)** A physical link between a service provider and its customer that provides access to the SP's network from that customer site.

**access point (AP)** A device that provides wireless service for clients within its coverage area or cell, with the AP connecting to both the wireless LAN and the wired Ethernet LAN.

**accounting** In security, the recording of access attempts. *See also* AAA.

**ACI** *See* Application Centric Infrastructure (ACI).

**ACL** Access control list. A list configured on a router to control packet flow through the router, such as to prevent packets with a certain IP address from leaving a particular interface on the router.

**ACL persistence** A feature of IOS XE (but not IOS) by which a router initialization event (power off/on or reload) does not cause reassigning ACL sequence numbers. The feature can be enabled (default, meaning no resequencing) or disabled.

**ACL resequencing** The process of renumbering the sequence numbers of ACL commands, either for all ACLs at router initialization (power off/on or reload) or by using a command to renumber individual ACLs.

**ACL sequence number** A number assigned to each ACL ACE when configured, either automatically or as typed in the configuration command, which allows easier deletion of individual ACEs.

**ad hoc wireless network** *See* independent basic service set (IBSS).

**administrative distance** In Cisco routers, a means for one router to choose between multiple routes to reach the same subnet when those routes are learned by different routing protocols. The lower the administrative distance, the more preferred the source of the routing information.

**agent-based architecture** With configuration management tools, an architecture that uses a software agent inside the device being managed as part of the functions to manage the configuration.

**agentless architecture** With configuration management tools, an architecture that does not need a software agent inside the device being managed as part of the functions to manage the configuration, instead using other mainstream methods like SSH and NETCONF.

**AI Ops** Artificial Intelligence for IT Operations. Refers to the application of artificial intelligence and machine learning techniques to automate and enhance various aspects of IT operations, including monitoring, troubleshooting, and incident management, to improve efficiency and reliability in managing complex IT environments.

**amplification attack** A reflection attack that leverages a service on the reflector to generate and reflect huge volumes of reply traffic to the victim.

**Ansible** A popular configuration management application, which can be used with or without a server, using a push model to move configurations into devices, with strong capabilities to manage network device configurations.

**APIC** *See* Application Policy Infrastructure Controller.

**Application Centric Infrastructure (ACI)** Cisco's data center SDN solution, the concepts of defining policies that the APIC controller then pushes to the switches in the network using the OpFlex protocol, with the partially distributed control plane in each switch building the forwarding table entries to support the policies learned from the controller. It also supports a GUI, a CLI, and APIs.

**Application Policy Infrastructure Controller (APIC)** The software that plays the role of controller, controlling the flows that the switches create to define where frames are forwarded, in a Cisco data center that uses the Application Centric Infrastructure (ACI) approach, switches, and software.

**application programming interface (API)** A software mechanism that enables software components to communicate with each other.

**application-specific integrated circuit (ASIC)** An integrated circuit (computer chip) designed for a specific purpose or application, often used to implement the functions of a networking device rather than running a software process as part of the device's OS that runs on a general-purpose processor.

**ARP reply** An ARP message used to supply information about the sending (origin) host's hardware (Ethernet) and IP addresses as listed in the origin hardware and origin IP address fields. Typically sent in reaction to receipt of an ARP request message.

**Artificial Intelligence (AI)** Refers to computer systems or software that can perform tasks typically requiring human intelligence, such as learning from data, making decisions, and solving problems.

**ASIC** *See* application-specific integrated circuit.

**association** A negotiated relationship between a wireless station and an access point.

**association request** An 802.11 frame that a wireless client sends to an AP to request an association with it.

**association response** An 802.11 frame that a wireless access point sends to a wireless client in reply to an association request.

**authentication** In security, the verification of the identity of a person or a process. *See also* AAA.

**authentication, authorization, and accounting (AAA) server** A server that holds security information and provides services related to user login, particularly authentication (is the user who he says he is), authorization (once authenticated, what do we allow the user to do), and accounting (tracking the user).

**authorization** In security, the determination of the rights allowed for a particular user or device. *See also* AAA.

**autonomous AP** A wireless AP operating in a standalone mode, such that it can provide a fully functional BSS and connect to the DS.

# B

**band** A contiguous range of frequencies.

**bandwidth** The speed at which bits can be sent and received over a link.

**basic service set (BSS)** Wireless service provided by one AP to one or more associated clients.

**basic service set identifier (BSSID)** A unique MAC address that is used to identify the AP that is providing a BSS.

**beacon** An 802.11 frame that an AP broadcasts at regular intervals to advertise the existence of an SSID. Separate beacons are transmitted for each SSID on each channel being used.

**brute-force attack** An attack where a malicious user runs software that tries every possible combination of letters, numbers, and special characters to guess a user's password. Attacks of this scale are usually run offline, where more computing resources and time are available.

**buffer overflow attack** An attack meant to exploit a vulnerability in processing inbound traffic such that the target system's buffers overflow; the target system can end up crashing or inadvertently running malicious code injected by the attacker.

# C

**cacheable** For resources that might be repeatedly requested over time, an attribute that means that the requesting host can keep in storage (cache) a copy of the resource for a specified amount of time.

**CAPWAP** A standards-based tunneling protocol that defines communication between a light-weight AP and a wireless LAN controller.

**CAT 5E** An unshielded twisted-pair (UTP) cable quality standard from the TIA and ANSI. It supports 1000BASE-T (and slower) UTP Ethernet at distances of 100 meters. It also supports multigig Ethernet standards at 100-meter distances.

**CAT 6A** An unshielded twisted-pair (UTP) cable quality standard from the TIA and ANSI. It is the lowest UTP cable category that formally supports 10GBase-T UTP Ethernet at distances of 100 meters.

**CDP** Cisco Discovery Protocol. A media- and protocol-independent device-discovery protocol that runs on most Cisco-manufactured equipment, including routers, access servers, and switches. Using CDP, a device can advertise its existence to other devices and receive information about other devices on the same LAN or on the remote side of a WAN.

**cell** The area of wireless coverage provided by an AP; also known as the basic service area.

**centralized control plane** An approach to architecting network protocols and products that places the control plane functions into a centralized function rather than distributing the function across the networking devices.

**centralized WLC deployment** A wireless network design that places a WLC centrally within a network topology.

**channel** An arbitrary index that points to a specific frequency within a band.

**Chat Ops** A collaboration model that integrates chat tools with automated workflows and tools, allowing teams to manage and execute tasks directly within chat interfaces, enhancing communication, visibility, and efficiency in operations and development workflows.

**CIDR** Classless interdomain routing. An RFC-standard tool for global IP address range assignment. CIDR reduces the size of Internet routers' IP routing tables, helping deal with the rapid growth of the Internet. The term *classless* refers to the fact that the summarized groups of networks represent a group of addresses that do not conform to IPv4 classful (Class A, B, and C) grouping rules.

**Cisco AnyConnect Secure Mobility Client** Cisco software product used as client software on user devices to create a client VPN. Commonly referred to as the Cisco VPN client.

**Cisco Catalyst 8000V** A virtual router platform designed for cloud and virtualized environments, offering high-performance, scalable, and secure routing capabilities with flexibility and agility.

**Cisco Catalyst Center** Cisco software, delivered by Cisco on a physical or virtual appliance, that acts as a network management application as well as being the control for Cisco's Software-Defined Access (Cisco SD-Access) offering.

**Cisco Prime Infrastructure (PI)** Graphical user interface (GUI) software that utilizes SNMP and can be used to manage your Cisco network devices. The term *Cisco Prime* is an umbrella term that encompasses many different individual software products.

**Cisco SD-Access** Cisco's intent-based networking (IBN) offering for enterprise networks.

**Cisco Secure Client (including AnyConnect)** Cisco software product used as client software on user devices to create a client VPN. Formerly called Cisco AnyConnect Secure Mobility Client, and sometimes referred to as the Cisco VPN client.

**Class of Service (CoS)** The informal term for the 3-bit field in the 802.IQ header intended for marking and classifying Ethernet frames for the purposes of applying QoS actions. Another term for Priority Code Point (PCP).

**classification** The process of examining various fields in networking messages in an effort to identify which messages fit into certain predetermined groups (classes).

**cloud-based AP** A wireless AP operating much like an autonomous AP, but having management and control functions present in the Internet cloud.

**cloud-based WLC deployment** A wireless network design that places a WLC centrally within a network topology, as a virtual machine in the private cloud portion of a data center.

**Cloud Management** Involves administering, monitoring, and optimizing cloud resources and services to ensure efficient utilization, performance, security, and compliance across cloud environments.

**cloud services catalog** A listing of the services available in a cloud computing service.

**code integrity** A software security term that refers to how likely that the software (code) being used is the software supplied by the vendor, unchanged, with no viruses or other changes made to the software.

**collapsed core design** A campus LAN design in which the design does not use a separate set of core switches in addition to the distribution switches—in effect collapsing the core into the distribution switches.

**Common ACL** A feature of IOS XE (but not IOS) that supports enabling two ACLs on a single interface and direction.

**configuration drift** A phenomenon that begins with the idea that devices with similar roles can and should have a similar standard configuration, so when one device's configuration is changed to be different, its configuration is considered to have moved away (drifted) from the standard configuration for a device in that role.

**configuration management** A component of network management focused on creating, changing, removing, and monitoring device configuration.

**configuration management tool** A class of application that manages data about the configuration of servers, network devices, and other computing nodes, providing consistent means of describing the configurations, moving the configurations into the devices, noticing unintended changes to the configurations, and troubleshooting by easily identifying changes to the configuration files over time.

**configuration monitoring** With configuration management tools like Ansible, a process of comparing over time a device's on-device configuration (running-config) versus the text file showing the ideal device configuration listed in the tool's centralized configuration repository. If different, the process can either change the device's configuration or report the issue.

**configuration provisioning** With configuration management tools like Ansible, the process of configuring a device to match the configuration as held in the configuration management tool.

**configuration template** With configuration management tools like Ansible, a file with variables, for the purpose of having the tool substitute different variable values to create the configuration for a device.

**connected mode** The operational mode used by a FlexConnect AP when the path back to its controller is up and working. In this mode, all wireless traffic flows over the CAPWAP tunnel to and from the controller.

**connection establishment** The process by which a connection-oriented protocol creates a connection. With TCP, a connection is established by a three-way transmission of TCP segments.

**container** One instance of a running application started from a container image and controlled by a container engine on a server.

**container image** One file that holds and embeds all files related to an application: all related executable files, required software libraries, and other files such as operating environment variables. Container virtualization systems then allow treating the application as a single file for movement, starting, stopping, and monitoring the container.

**control plane** Functions in networking devices and controllers that directly control how devices perform data plane forwarding, but excluding the data plane processes that work to forward each message in the network.

**controller-based networking** A style of building computer networks that use a controller that centralizes some features and provides application programming interfaces (APIs) that allow for software interactions between applications and the controller (northbound APIs) and between the controller and the network devices (southbound APIs).

**controller-less wireless deployment** A wireless design based on an embedded wireless controller (EWC), where the WLC function is co-located with an AP, rather than a discrete physical controller.

**core** In computer architecture, an individual processing unit that can execute instructions of a CPU; modern server processors typically have multiple cores, each capable of concurrent execution of instructions.

**core design** A campus LAN design that connects each access switch to distribution switches, and distribution switches into core switches, to provide a path between all LAN devices.

**core layer** In a campus LAN design, the switches that connect the distribution layer switches, and to each other, to provide connectivity between the various distribution layer switches.

**CRUD** In software development, an acronym that refers to the four most common actions taken by a program: Create, Read, Update, and Delete.

# D

**data plane** Functions in networking devices that are part of the process of receiving a message, processing the message, and forwarding the message.

**data serialization language** A language that includes syntax and rules that provides a means to describe the variables inside applications in a text format, for the purpose of sending that text between applications over a network or storing the data models in a file.

**Declarative Model** A method of describing IT automation that defines or declares the intended configuration, with the expectation that the automation software monitors the devices' configurations and changes them if they drift away from the intended (declared) configuration.

**declarative policy model** A term that describes the approach in an intent-based network (IBN) in which the engineer chooses settings that describe the intended network behavior (the declared policy) but does not command the network with specific configuration commands for each protocol (as would be the case with an imperative policy model).

**delay** In QoS, the amount of time it takes for a message to cross a network. Delay can refer to one-way delay (the time required for the message to be sent from the source host to the destination host) or two-way delay (the delay from the source to the destination host and then back again).

**denial-of-service (DoS) attack** An attack that tries to deplete a system resource so that systems and services crash or become unavailable.

**deny** An action taken with an ACL that implies that the packet is discarded.

**DevNet** Cisco's community and resource site for software developers, open to all, with many great learning resources; https://developer.cisco.com.

**DHCP attack** Any attack that takes advantage of DHCP protocol messages.

**DHCP Snooping** A switch security feature in which the switch examines incoming DHCP messages and chooses to filter messages that are abnormal and therefore might be part of a DHCP attack.

**DHCP Snooping binding table** When using DHCP Snooping, a table that the switch dynamically builds by analyzing the DHCP messages that flow through the switch. DHCP Snooping can use the table for part of its filtering logic, with other features, such as Dynamic ARP Inspection and IP Source Guard also using the table.

**dictionary attack** An attack where a malicious user runs software that attempts to guess a user's password by trying words from a dictionary or word list.

**dictionary variable** In applications, a single variable whose value is a list of other variables with values, known as key:value pairs.

**Differentiated Services (DiffServ)** An approach to QoS, originally defined in RFC 2475, that uses a model of applying QoS per classification, with planning of which applications and other traffic types are assigned to each class, with each class given different QoS per-hop behaviors at each networking device in the path.

**Differentiated Services Code Point (DSCP)** A field existing as the first 6 bits of the ToS byte, as defined by RFC 2474, which redefined the original IP RFC's definition for the IP header ToS byte. The field is used to mark a value in the header for the purpose of performing later QoS actions on the packet.

**distributed control plane** An approach to architecting network protocols and products that places some control plane functions into each networking device rather than centralizing the control plane functions in one or a few devices. An example is the use of routing protocols on each router which then work together so that each router learns Layer 3 routes.

**distributed denial-of-service (DDoS) attack** A DoS attack that is distributed across many hosts under centralized control of an attacker, all targeting the same victim.

**distributed WLC deployment** A wireless design based on distributing multiple controllers within the network. Each of the controllers commonly supports a relatively small number of users.

**distribution layer** In a campus LAN design, the switches that connect to access layer switches as the most efficient means to provide connectivity from the access layer into the other parts of the LAN.

**distribution link** In campus LAN design, a link that connects a distribution switch to an access switch.

**distribution system (DS)** The wired Ethernet that connects to an AP and transports traffic between a wired and wireless network.

**DNS** Domain Name System. An application layer protocol used throughout the Internet for translating hostnames into their associated IP addresses.

**DNS server** An application acting as a server for the purpose of providing name resolution services per the Domain Name System (DNS) protocol and worldwide system.

**domain-specific language** A generic term that refers to an attribute of different languages within computing, for languages created for a specific purpose (domain) rather than a general-purpose language like Python or JavaScript.

**Dynamic ARP Inspection (DAI)** A security feature in which a LAN switch filters a subset of incoming ARP messages on untrusted ports, based on a comparison of ARP, Ethernet, and IP header fields to data gathered in the IP DHCP Snooping binding table and found in any configured ARP ACLs.

# E

**egress tunnel router (ETR)** With LISP, a node at the end of a tunnel that receives an encapsulated message and then de-encapsulates the message.

**E-LAN** A specific carrier/Metro Ethernet service defined by MEF (MEF.net) that provides a service much like a LAN, with two or more customer sites connected to one E-LAN service in a full mesh so that each device in the E-LAN can send Ethernet frames directly to every other device.

**E-Line** A specific carrier/metro Ethernet service defined by MEF (MEF.net) that provides a point-to-point topology between two customer devices, much as if the two devices were connected using an Ethernet crossover cable.

**embedded wireless controller (EWC)** A WLC function that is co-located within an AP.

**embedded wireless controller (EWC) deployment** A wireless network design that places a WLC in the access layer, co-located with a LAN switch stack, near the APs it controls.

**enable secret** A reference to the password configured on the **enable secret** *pass-value* command, which defines the password required to reach enable (privileged) mode.

**error detection** The process of discovering whether a data-link level frame was changed during transmission. This process typically uses a Frame Check Sequence (FCS) field in the data-link trailer.

**error disabled (err-disable)** An interface state on LAN switches that can be the result of one of many security violations.

**error recovery** The process of noticing when some transmitted data was not successfully received and resending the data until it is successfully received.

**Ethernet access link** A WAN access link (a physical link between a service provider and its customer) that happens to use Ethernet.

**Ethernet WAN** A general and informal term for any WAN service that uses Ethernet links as the access link between the customer and the service provider.

**exploit** A means of taking advantage of a vulnerability to compromise something.

**extended access list** A list of IOS **access-list** global configuration commands that can match multiple parts of an IP packet, including the source and destination IP address and TCP/UDP ports, for the purpose of deciding which packets to discard and which to allow through the router.

**extended service set (ESS)** Multiple APs that are connected by a common switched infrastructure.

**extended service set identifier (ESSID)** The SSID used consistently throughout an ESS.

# F

**fabric** In SDA, the combination of overlay and underlay that together provide all features to deliver data across the network with the desired features and attributes.

**fabric edge node** In SDA, a switch that connects to endpoint devices.

**fiber Internet** A general term for any Internet access technology that happens to use fiber-optic cabling. It often uses Ethernet protocols on the fiber link.

**firewall** A device that forwards packets between the less secure and more secure parts of the network, applying rules that determine which packets are allowed to pass and which are not.

**flash memory** A type of read/write permanent memory that retains its contents even with no power applied to the memory and that uses no moving parts, making the memory less likely to fail over time.

**FlexConnect mode** An AP mode tailored for remote sites. Wireless traffic flows to and from a controller if the AP's CAPWAP tunnel is up, or is locally switched if the tunnel is down.

**flow control** The process of regulating the amount of data sent by a sending computer toward a receiving computer. Several flow control mechanisms exist, including TCP flow control, which uses windowing.

**FTP** File Transfer Protocol. An application protocol, part of the TCP/IP protocol stack, used to transfer files between network nodes. FTP is defined in RFC 959.

**FTP control connection** A TCP connection initiated by an FTP client to an FTP server for the purpose of sending FTP commands that direct the activities of the connection.

**FTP data connection** A TCP connection created by an FTP client and server for the purpose of transferring data.

**FTPS** FTP Secure. Common term for FTP over TLS.

**full mesh** From a topology perspective, any topology that has two or more devices, with each device being able to send frames to every other device.

# G

**Generative AI** Also Strong AI; encompasses artificial intelligence systems capable of learning, reasoning, and making decisions similar to human cognition, often with the capability to create new content, ideas, or solutions beyond its initial training data.

**Git** An open-source version control application, widely popular for version control in software development and for other uses, like managing network device configurations.

**GitHub** A web-based platform for version control and collaboration, facilitating hosting, sharing, and managing of software projects using the Git version control system.

**GLBP active virtual forwarder (AVF)** A role implemented by all routers in a GLBP group, listening for frames sent to a unique virtual MAC address, so it can act as one of several active default routers in the group.

**GLBP active virtual gateway (AVG)** A role implemented by one router in a GLBP group, replying to ARP requests on behalf of the group's VIP, so that load balancing occurs.

**gratuitous ARP** An ARP Reply not sent as a reaction to an ARP request message, but rather as a general announcement informing other hosts of the values of the sending (origin) host's addresses.

# H

**host (context: DC)** In a virtualized server environment, the term used to refer to one physical server that is running a hypervisor to create multiple virtual machines.

**HSRP preemption** A configuration setting that dictates whether an HSRP router, when it initializes with HSRP, can immediately take over the HSRP active role (preemption) if it has a higher priority than the currently active router.

**HSRP priority** A configuration setting from 0 through 255 that impacts the choice of HSRP active state, with the highest priority router chosen as active.

**HTTP** Hypertext Transfer Protocol. The protocol used by web browsers and web servers to transfer files, such as text and graphic files.

**HTTP/1.0** A version of the HTTP protocol, published as an RFC in the mid 1990s. It uses TCP, expects URLs that begin with http, implying a default well-known server port of 80. It allows the use of secure HTTP as well.

**HTTP/1.1** A version of the HTTP protocol, published as an RFC in the mid 1990s. It uses TCP, expects URLs that begin with http, implying a default well-known server port of 80. It allows the use of secure HTTP as well.

**HTTP/2.0** A version of the HTTP protocol, published as an RFC in the mid 2010s. The protocol improved HTTP application processes to increase overall performance in the end-user experience. It supports using Secure HTTP (with TLS) or not.

**HTTP/3.0** A radically different HTTP version published as an RFC in 2022 and created by Google. It improved HTTP application processes plus changed the transport layer protocols, both to increase overall performance in the end-user experience. It uses the QUIC transport layer, which uses UDP (not TCP) and always includes TLS.

**hub and spoke** From a topology perspective, any topology that has a device that can send messages to all other devices (the hub), with one or more spoke devices that can send messages only to the hub. Also called point-to-multipoint.

**hypervisor** Software that runs on server hardware to create the foundations of a virtualized server environment primarily by allocating server hardware components like CPU core/threads, RAM, disk, and network to the VMs running on the server.

# I

**Imperative Model** A method of describing IT automation as a series of tasks, akin to a script or program, with the expectation that running the script will configure the devices to have the desired configuration.

**imperative policy model** A term that describes the approach in traditional networks in which the engineer chooses configuration settings for each control and data plane protocol (the imperative commands) that dictate specifically how the devices act. This model acts in contrast to the newer declarative policy model and intent-based networking (IBN).

**independent basic service set (IBSS)** An impromptu wireless network formed between two or more devices without an AP or a BSS; also known as an ad hoc network.

**Infrastructure as Code (IAC)** A practice that involves managing and provisioning computing infrastructure through machine-readable definition files, enabling automation, consistency, and scalability in infrastructure deployment and management processes.

**Infrastructure as a Service (IaaS)** A cloud service in which the service consists of a virtual machine that has defined computing resources (CPUs, RAM, disk, and network) and may or may not be provided with an installed OS.

**infrastructure mode** The operating mode of an AP that is providing a BSS for wireless clients.

**inside global** For packets sent to and from a host that resides inside the trusted part of a network that uses NAT, a term referring to the IP address used in the headers of those packets when those packets traverse the global (public) Internet.

**inside local** For packets sent to and from a host that resides inside the trusted part of a network that uses NAT, a term referring to the IP address used in the headers of those packets when those packets traverse the enterprise (private) part of the network.

**intent-based networking (IBN)** An approach to networking in which the system gives the operator the means to express business intent, with the networking system then determining what should be done by the network, activating the appropriate configuration, and monitoring (assuring) the results.

**intrusion prevention system (IPS)** A security function that examines more complex traffic patterns against a list of both known attack signatures and general characteristics of how attacks can be carried out, rating each perceived threat, and reacting to prevent the more significant threats. *See also* IPS.

**IOS File System (IFS)** A file system created by a Cisco device that uses IOS.

**IOS image** A file that contains the IOS.

**IP Precedence (IPP)** In the original definition of the IP header's Type of Service (ToS) byte, the first 3 bits of the ToS byte, used for marking IP packets for the purpose of applying QoS actions.

**IPS** *See* intrusion prevention system.

**IPsec** The term referring to the IP Security protocols, which is an architecture for providing encryption and authentication services, usually when creating VPN services through an IP network.

**IPsec transport mode** The process of encrypting the data of the original IP packet when using IPsec, while using the original packet's IP header, plus VPN headers, to encapsulate the encrypted data. Typically used with remote access IPsec VPNs.

**IPsec tunnel mode** The process of encrypting the entire original IP packet when using IPsec, which requires a new IP header, plus VPN headers, to encapsulate the encrypted original packet. Typically used with site-to-site IPsec VPNs.

# J

**Jinja2** A text-based language used to define templates, with text plus variables; used by Ansible for templates.

**jitter** The variation in delay experienced by successive packets in a single application flow.

**JSON (JavaScript Object Notation)** A popular data serialization language, originally used with the JavaScript programming language, and popular for use with REST APIs.

**JSON array** A part of a set of JSON text that begins and ends with a matched set of square brackets that contain a list of values.

**JSON object** A part of a set of JSON text that begins and ends with a matched set of curly brackets that contain a set of key:value pairs.

# K–L

**key:value pair** In software, one variable name (key) and its value, separated by a colon in some languages and data serialization languages.

**Large Language Models (LLM)** Advanced artificial intelligence systems capable of understanding and generating human-like text based on extensive training on vast amounts of textual data.

**leaf** In an ACI network design, a switch that connects to spine switches and to endpoints, but not to other leaf switches, so that the leaf can forward frames from an endpoint to a spine, which then delivers the frame to some other leaf switch.

**LISP** Locator/ID Separation Protocol. A protocol, defined in RFC 6830, that separates the concepts and numbers used to identify an endpoint (the endpoint identifier) versus identifying the location of the endpoint (routing locator).

**list variable** In applications, a single variable whose value is a list of values, rather than a simple value.

**LLDP** Link Layer Discovery Protocol. An IEEE standard protocol (IEEE 802.1AB) that defines messages, encapsulated directly in Ethernet frames so they do not rely on a working IPv4 or IPv6 network, for the purpose of giving devices a means of announcing basic device information to other devices on the LAN. It is a standardized protocol similar to Cisco Discovery Protocol (CDP).

**LLDP-MED** A group of endpoint-focused LLDP TLVs, defined as a group TIA standard TIA-1057. It includes TLVs to communicate voice and data VLANs to phones and to manage power levels with PoE.

**local username** A username (with matching password), configured on a router or switch. It is considered local because it exists on the router or switch, and not on a remote server.

**log message** A message generated by any computer, but including Cisco routers and switches, for which the device OS wants to notify the owner or administrator of the device about some event.

**loss** A reference to packets in a network that are sent but do not reach the destination host.

# M

**malware** Malicious software.

**Management Information Base (MIB)** The data structures defined by SNMP to define a hierarchy (tree) structure with variables at the leaves of the tree, so that SNMP messages can reference the variables.

**management plane** Functions in networking devices and controllers that control the devices themselves but that do not impact the forwarding behavior of the devices like control plane protocols do.

**man-in-the-middle attack** An attack where an attacker manages to position a machine on the network such that it is able to intercept traffic passing between target hosts.

**marking** The process of changing one of a small set of fields in various network protocol headers, including the IP header's DSCP field, for the purpose of later classifying a message based on that marked value.

**MD5 hash** A specific mathematical algorithm intended for use in various security protocols. In the context of Cisco routers and switches, the devices store the MD5 hash of certain passwords, rather than the passwords themselves, in an effort to make the device more secure.

**Media Access Control (MAC) layer** The lower of the two sublayers of the data-link layer defined by the IEEE. Synonymous with IEEE 802.3 for Ethernet LANs.

**Meraki Dashboard** A centralized cloud-based management platform that provides intuitive control and monitoring of Meraki networking devices, offering streamlined configuration, real-time analytics, and seamless network administration.

**mesh network** A network of APs used to cover a large area without the need for wired Ethernet cabling; client traffic is bridged from AP to AP over a backhaul network.

**Metro Ethernet** The original term used for WAN service that used Ethernet links as the access link between the customer and the service provider.

**MIB** *See* Management Information Base.

**mitigation technique** A method to counteract or prevent threats and malicious activity.

**MPLS** *See* Multiprotocol Label Switching.

**MPLS VPN** A WAN service that uses MPLS technology, with many customers connecting to the same MPLS network, but with the VPN features keeping each customer's traffic separate from others.

**MTU** Maximum transmission unit. The maximum packet size, in bytes, that a particular interface can handle.

**multifactor authentication** A technique that uses more than one type of credential to authenticate users.

**Multigig Ethernet** The common name for the 2.5GBase-T and 5.0GBase-T Ethernet standards, which, when released simultaneously, represented an option for UTP Ethernet at speeds of multiple gigabits between the then-defined standard speeds of 1 Gbps and 10 Gbps.

**Multiprotocol BGP (MPBGP)** A particular set of BGP extensions that allows BGP to support multiple address families, which when used to create an MPLS VPN service gives the SP the method to advertise the IPv4 routes of many customers while keeping those route advertisements logically separated.

**Multiprotocol Label Switching (MPLS)** A WAN technology used to create an IP-based service for customers, with the service provider's internal network performing forwarding based on an MPLS label rather than the destination IP address.

# N

**named access list** An ACL that identifies the various statements in the ACL based on a name rather than a number.

**Narrow AI** Also Weak AI. Refers to artificial intelligence designed and trained for specific tasks or a limited range of activities, lacking broad cognitive capabilities and adaptability beyond its predefined scope.

**NAT** Network Address Translation. A mechanism for reducing the need for globally unique IP addresses. NAT allows an organization with addresses that are not globally unique to connect to the Internet, by translating those addresses into public addresses in the globally routable address space.

**NAT overload** Another term for Port Address Translation (PAT). One of several methods of configuring NAT, in this case translating TCP and UDP flows based on port numbers in addition to using one or only a few inside global addresses.

**NBI** *See* northbound API.

**Network Management System (NMS)** A software platform that enables centralized monitoring, configuration, and administration of network infrastructure and devices to ensure optimal performance, security, and reliability.

**Network Time Protocol (NTP)** A protocol used to synchronize time-of-day clocks so that multiple devices use the same time of day, which allows log messages to be more easily matched based on their timestamps.

**Next-generation firewall (NGFW)** A firewall device with advanced features, including the ability to run many related security features in the same firewall device (IPS, malware detection, VPN termination), along with deep packet inspection with Application Visibility and Control (AVC) and the ability to perform URL filtering versus data collected about the reliability and risk associated with every domain name.

**Next-generation IPS (NGIPS)** An IPS device with advanced features, including the capability to go beyond a comparison to known attack signatures to also look at contextual data, including the vulnerabilities in the current network, the capability to monitor for new zero-day threats, with frequent updates of signatures from the Cisco Talos security research group.

**NMS** *See* Network Management System (NMS).

**nonoverlapping channels** Successive channel numbers in a band that each have a frequency range that is narrow enough to not overlap the next channel above or below.

**northbound API** In the area of SDN, a reference to the APIs that a controller supports that gives outside programs access to the services of the controller; for instance, to supply information about the network or to program flows into the network. Also called a northbound interface.

**northbound interface (NBI)** Another term for northbound API. *See also* northbound API.

**NTP client** Any device that attempts to use the Network Time Protocol (NTP) to synchronize its time by adjusting the local device's time based on NTP messages received from a server.

**NTP client/server mode** A mode of operation with the Network Time Protocol (NTP) in which the device acts as both an NTP client, synchronizing its time with some servers, and as an NTP server, supplying time information to clients.

**NTP server** Any device that uses Network Time Protocol (NTP) to help synchronize time-of-day clocks for other devices by telling other devices its current time.

**NTP synchronization** The process with the Network Time Protocol (NTP) by which different devices send messages, exchanging the devices' current time-of-day clock information and other data, so that some devices adjust their clocks to the point that the time-of-day clocks list the same time (often accurate to at least the same second).

# O

**on-demand self-service** One of the five key attributes of a cloud computing service as defined by NIST, referring to the fact that the consumer of the server can request the service, with the service being created without any significant delay and without waiting on human intervention.

**on-premises** An alternate term for private cloud. *See also* private cloud.

**OpenFlow** The open standard for Software-Defined Networking (SDN) as defined by the Open Networking Foundation (ONF), which defines the OpenFlow protocol as well as the concept of an abstracted OpenFlow virtual switch.

**Optical Multimode (OM)** The term used to refer to multimode fiber-optic cabling in various cabling standards.

**ordered data transfer** A networking function, included in TCP, in which the protocol defines how the sending host should number the data transmitted, defines how the receiving device should attempt to reorder the data if it arrives out of order, and specifies to discard the data if it cannot be delivered in order.

**outside global** With source NAT, the one address used by the host that resides outside the enterprise, which NAT does not change, so there is no need for a contrasting term.

**overlay** In SDA, the combination of VXLAN tunnels between fabric edge nodes as a data plane for forwarding frames, plus LISP for the control plane for the discovery and registration of endpoint identifiers.

# P

**partial mesh** A network topology in which more than two devices could physically communicate, but by choice, only a subset of the pairs of devices connected to the network is allowed to communicate directly.

**passive scanning** A technique used by a wireless client when it attempts to discover nearby APs by listening for their beacon frames.

**password guessing** An attack where a malicious user simply makes repeated attempts to guess a user's password.

**per-hop behavior (PHB)** The general term used to describe the set of QoS actions a device can apply to a message from the time it enters a networking device until the device forwards the message. PHBs include classification, marking, queuing, shaping, policing, and congestion avoidance.

**pharming** An attack that compromises name services to silently redirect users toward a malicious site.

**phishing** An attack technique that sends specially crafted emails to victims in the hope that the users will follow links to malicious websites.

**Platform as a Service (PaaS)** A cloud service intended for software developers as a development platform, with a variety of tools useful to developers already installed so that developers can focus on developing software rather than on creating a good development environment.

**PoE** Power over Ethernet. Both a generalized term for any of the standards that supply power over an Ethernet link, as well as a specific PoE standard as defined in the IEEE 802.3af amendment to the 802.3 standard.

**point of presence (PoP)** A term used for a service provider's (SP) perspective to refer to a service provider's installation that is purposefully located relatively near to customers, with several spread around major cities, so that the distance from each customer site to one of the SP's PoPs is short.

**point-to-point** From a topology perspective, any topology that has two and only two devices that can send messages directly to each other.

**point-to-point bridge** An AP configured to bridge a wired network to a companion bridge at the far end of a line-of-sight path.

**policing** A QoS tool that monitors the bit rate of the messages passing some point in the processing of a networking device, so that if the bit rate exceeds the policing rate for a period of time, the policer can discard excess packets to lower the rate.

**policing rate** The bit rate at which a policer compares the bit rate of packets passing through a policing function, for the purpose of taking a different action against packets that conform (are under) to the rate versus those that exceed (go over) the rate.

**port** (Multiple definitions) (1) In TCP and UDP, a number that is used to uniquely identify the application process that either sent (source port) or should receive (destination port) data. (2) In LAN switching, another term for switch interface.

**Port Address Translation (PAT)** A NAT feature in which one inside global IP address supports over 65,000 concurrent TCP and UDP connections.

**port number** A field in a TCP or UDP header that identifies the application that either sent (source port) or should receive (destination port) the data inside the data segment.

**port security** A Cisco switch feature in which the switch watches Ethernet frames that come in an interface (a port), tracks the source MAC addresses of all such frames, and takes a security action if the number of different such MAC addresses is exceeded.

**Power classification** With Power over Ethernet (PoE), the process by which a switch, once it detects a device wants power, discovers the amount of power as defined by a standardized set of power classes.

**Power detection** With Power over Ethernet (PoE), the process by which a switch discovers if the connected device wants to receive power over the link or not.

**Power over Ethernet (PoE)** Both a generalized term for any of the standards that supply power over an Ethernet link and a specific PoE standard as defined in the IEEE 802.3af amendment to the 802.3 standard.

**Power over Ethernet Plus (PoE+)** A specific PoE standard as defined in the IEEE 802.3at amendment to the 802.3 standard, which uses two wire pairs to supply power with a maximum of 30 watts as supplied by the PSE.

**power sourcing equipment (PSE)** With any Power over Ethernet standard, a term that refers to the device supplying the power over the cable, which is then used by the powered device (PD) on the other end of the cable.

**powered device (PD)** With any Power over Ethernet standard, a term that refers to the device that receives or draws its power over the Ethernet cable, with the power being supplied by the power sourcing equipment (PSE) on the other end of the cable.

**priority queue** In Cisco queuing systems, another term for a low latency queue (LLQ).

**private cloud** A cloud computing service in which a company provides its own IT services to internal customers inside the same company but by following the practices defined as cloud computing.

**private IP network** Any of the IPv4 Class A, B, or C networks as defined by RFC 1918, intended for use inside a company but not used as public IP networks.

**probe request** A technique used by a wireless client to discover nearby APs by actively requesting a response.

**provider** A plug-in that enables communication and interaction between Terraform and specific infrastructure platforms or services, facilitating the management and provisioning of resources within those environments through Terraform configuration files.

**provider edge (PE)** A term used by service providers, both generally and also specifically in MPLS VPN networks, to refer to the SP device in a point of presence (PoP) that connects to the customer's network and therefore sits at the edge of the SP's network.

**public cloud** A cloud computing service in which the cloud provider is a different company than the cloud consumer.

**pull model** With configuration management tools, a practice by which an agent representing the device requests configuration data from the centralized configuration management tool, in effect pulling the configuration to the device.

**push model** With configuration management tools, a practice by which the centralized configuration management tool software initiates the movement of configuration from that node to the device that will be configured, in effect pushing the configuration to the device.

# Q–R

**Quality of Service (QoS)** The performance of a message, or the messages sent by an application, in regard to the bandwidth, delay, jitter, or loss characteristics experienced by the message(s).

**queuing** The process by which networking devices hold packets in memory while waiting on some constrained resource; for example, when waiting for the outgoing interface to become available when too many packets arrive in a short period of time.

**QUIC** The name (not an acronym) for a transport layer protocol that improves overall performance of transferring objects over a network, in comparison to TCP. It uses UDP and integrates TLS into its connection setup tasks.

**RADIUS** A security protocol often used for user authentication, including being used as part of the IEEE 802.lx messages between an 802.lx authenticator (typically a LAN switch) and a AAA server.

**RAM** Random-access memory. A type of volatile memory that can be read and written by a microprocessor.

**rapid elasticity** One of the five key attributes of a cloud computing service as defined by NIST, referring to the fact that the cloud service reacts to requests for new services quickly, and it expands (is elastic) to the point of appearing to be a limitless resource.

**read-only community** An SNMP community (a value that acts as a password), defined on an SNMP agent, which then must be supplied by any SNMP manager that sends the agent any messages asking to learn the value of a variable (like SNMP Get and GetNext requests).

**read-write community** An SNMP community (a value that acts as a password), defined on an SNMP agent, which then must be supplied by any SNMP manager that sends the agent any messages asking to set the value of a variable (like SNMP Set requests).

**reassociation request** An 802.11 frame that a roaming wireless client sends to an AP to request that its existing association be moved to a new AP.

**reconnaissance attack** An attack crafted to discover as much information about a target organization as possible; the attack can involve domain discovery, ping sweeps, port scans, and so on.

**recursive DNS server** A DNS server that, when asked for information it does not have, performs a repetitive (recursive) process to ask other DNS servers in sequence, hoping to find the DNS server that knows the information.

**reflection attack** An attack that uses spoofed source addresses so that a destination machine will reflect return traffic to the attack's target; the destination machine is known as the reflector.

**remote access VPN** A VPN for which one endpoint is a user device, such as a phone, tablet, or PC, typically created dynamically, and often using TLS. Also called a client VPN.

**repeater** A device that repeats or retransmits signals it receives, effectively expanding the wireless coverage area.

**Representational State Transfer (REST)** A type of API that allows two programs that reside on separate computers to communicate, with a set of six primary API attributes as defined early in this century by its creator, Roy Fielding. The attributes include client/server architecture, stateless operation, cachability, uniform interfaces, layered, and code-on-demand.

**resource pooling** One of the five key attributes of a cloud computing service as defined by NIST, referring to the fact that the cloud provider treats its resources as a large group (pool) of resources that its cloud management systems then allocate dynamically based on self-service requests by its customers.

**REST** *See* Representational State Transfer.

**REST API** Any API that uses the rules of Representational State Transfer (REST).

**roaming** The process a wireless client uses to move from one AP to another as it changes location.

**round robin** A queue scheduling algorithm in which the scheduling algorithm services one queue, then the next, then the next, and so on, working through the queues in sequence.

# S

**SBI** *See* Southbound API.

**scalable group tag (SGT)** In SDA, a value assigned to the users in the same security group.

**Secure HTTP (HTTP over TLS)** The IETF standard that defines how to use TLS to add security features such as server authentication and message encryption to HTTP/2 and earlier versions of HTTP.

**segment** (Multiple definitions) (1) In TCP, a term used to describe a TCP header and its encapsulated data (also called an L4PDU). (2) Also in TCP, the set of bytes formed when TCP breaks a large chunk of data given to it by the application layer into smaller pieces that fit into TCP segments. (3) In Ethernet, either a single Ethernet cable or a single collision domain (no matter how many cables are used).

**sender hardware address** In both an ARP request and reply message, the field intended to be used to list the sender (origin) device's hardware address, typically an Ethernet LAN address.

**sender IP address** In both an ARP request and reply message, the field intended to be used to list the sender (origin) device's IP address.

**sender protocol address** In both an ARP request and a reply message, the formal term the field intended to be used to list the sender (origin) device's network layer address.

**service provider (SP)** A company that provides a service to multiple customers. Used most often to refer to providers of private WAN services and Internet services. *See also* Internet service provider.

**Service Set Identifier (SSID)** A text string that is used to identify a wireless network.

**shaping** A QoS tool that monitors the bit rate of the messages exiting networking devices, so that if the bit rate exceeds the shaping rate for a period of time, the shaper can queue the packets, effectively slowing down the sending rate to match the shaping rate.

**shaping rate** The bit rate at which a shaper compares the bit rate of packets passing through the shaping function, so that when the rate is exceeded, the shaper enables the queuing of packets, resulting in slowing the bit rate of the collective packets that pass through the shaper, so the rate of bits getting through the shaper does not exceed the shaping rate.

**shared key** A reference to a security key whose value is known (shared) by both the sender and receiver.

**Simple Network Management Protocol (SNMP)** An Internet standard protocol for managing devices on IP networks. It is used mostly in network management systems to monitor network-attached devices for conditions that warrant administrative attention.

**site-to-site VPN** The mechanism that allows all devices at two different sites to communicate securely over some unsecure network like the Internet, by having one device at each site perform encryption/decryption and forwarding for all the packets sent between the sites.

**sliding windows** For protocols such as TCP that allow the receiving device to dictate the amount of data the sender can send before receiving an acknowledgment—a concept called a *window*—a reference to the fact that the mechanism to grant future windows is typically just a number that grows upward slowly after each acknowledgment, sliding upward.

**SNMP** *See* Simple Network Management Protocol.

**SNMP agent** Software that resides on the managed device and processes the SNMP messages sent by the Network Management Station (NMS).

**SNMP community** A simple password mechanism in SNMP in which either the SNMP agent or manager defines a community string (password), and the other device must send that same password value in SNMP messages, or the messages are ignored. *See also* read-only community, read-write community, and notification community.

**SNMP Get** Message used by SNMP to read from variables in the MIB.

**SNMP Inform** An unsolicited SNMP message like a Trap message, except that the protocol requires that the Inform message needs to be acknowledged by the SNMP manager.

**SNMP manager** Typically a Network Management System (NMS), with this term specifically referring to the use of SNMP and the typical role of the manager, which retrieves status information with SNMP Get requests, sets variables with the SNMP Set requests, and receives unsolicited notifications from SNMP agents by listening for SNMP Trap and Notify messages.

**SNMP Set** SNMP message to set the value in variables of the MIB. These messages are the key to an administrator configuring the managed device using SNMP.

**SNMP Trap** An unsolicited SNMP message generated by the managed device, and sent to the SNMP manager, to give information to the manager about some event or because a measurement threshold has been passed.

**SNMPv2c** A variation of the second version of SNMP. SNMP Version 2 did not originally support communities; the term *SNMPv2c* refers to SNMP version 2 with support added for SNMP communities (which were part of SNMPvl).

**SNMPv3** The third version of SNMP, with the notable addition of several security features as compared to SNMPv2c, specifically message integrity, authentication, and encryption.

**social engineering** Attacks that leverage human trust and social behaviors to divulge sensitive information.

**Software as a Service (SaaS)** A cloud service in which the service consists of access to working software, without the need to be concerned about the details of installing and maintaining the software or the servers on which it runs.

**Software-Defined Access** Cisco's intent-based networking (IBN) offering for enterprise networks.

**Software-Defined Networking (SDN)** A branch of networking that emerged in the marketplace in the 2010s characterized by the use of a centralized software controller that takes over varying amounts of the control plane processing formerly done inside networking devices, with the controller directing the networking elements as to what forwarding table entries to put into their forwarding tables.

**SOHO** A classification of a business site with a relatively small number of devices, sometimes in an employee office in their home.

**Source NAT** The type of Network Address Translation (NAT) used most commonly in networks (as compared to destination NAT), in which the source IP address of packets entering an inside interface is translated.

**southbound API** In the area of SDN, a reference to the APIs used between a controller and the network elements for the purpose of learning information from the elements and for programming (controlling) the forwarding behavior of the elements. Also called a southbound interface.

**southbound interface** Another term for southbound API. *See also* southbound API.

**spear phishing** Phishing that begins with research about a related group of people so that the attack uses messaging that appears more legitimate by using those researched facts.

**spine** In an ACI network design for a single site, a switch that connects to leaf switches only, for the purpose of receiving frames from one leaf switch and then forwarding the frame to some other leaf switch.

**split-MAC architecture** A wireless AP strategy based around the idea that normal AP functions are split or divided between a wireless LAN controller and lightweight APs.

**spoofing attack** A type of attack in which parameters such as IP and MAC addresses are spoofed with fake values to disguise the sender.

**standalone mode** The operational mode used by a FlexConnect AP when the path back to its controller is down and not working. In this mode, all wireless traffic is switched locally, preserving local connectivity while the AP is isolated from its controller.

**standard access list** A list of IOS global configuration commands that can match only a packet's source IP address for the purpose of deciding which packets to discard and which to allow through the router.

**star topology** A network topology in which endpoints on a network are connected to a common central device by point-to-point links.

**stateless** A protocol or process that does not use information stored from previous transactions to perform the current transaction.

**station (STA)** An 802.11 client device that is associated with a BSS.

**syslog server** A server application that collects syslog messages from many devices over the network and provides a user interface so that IT administrators can view the log messages to troubleshoot problems.

# T

**TCAM** *See* ternary content-addressable memory.

**ternary content-addressable memory (TCAM)** A type of physical memory, either in a separate integrated circuit or built into an ASIC, that can store tables and then be searched against a key, such that the search time happens quickly and does not increase as the size of the table increases. TCAMs are used extensively in higher-performance networking devices as the means to store and search forwarding tables in Ethernet switches and higher-performance routers.

**Terraform** An open-source infrastructure as code tool used for provisioning and managing cloud, on-premises, and hybrid infrastructure resources through declarative configuration files.

**TFTP** Trivial File Transfer Protocol. An application protocol that allows files to be transferred from one computer to another over a network, but with only a few features, making the software require little storage space.

**threat** An actual potential to use an exploit to take advantage of a vulnerability.

**Transport Layer Security (TLS)** A security standard that replaced the older Secure Sockets Layer (SSL) protocol, providing functions such as authentication, confidentiality, and message integrity over reliable in-order data streams like TCP.

**trojan horse** Malware that is hidden and packaged inside other legitimate software.

**trusted port** With both the DHCP Snooping and Dynamic ARP Inspection (DAI) switch features, the concept and configuration setting that tells the switch to allow all incoming messages of that respective type, rather than to consider the incoming messages (DHCP and ARP, respectively) for filtering.

**two-tier design** *See* collapsed core design.

# U

**underlay** In SDA, the network devices and links that create basic IP connectivity to support the creation of VXLAN tunnels for the overlay.

**Unified Computing System (UCS)** The Cisco brand name for its server hardware products.

**Universal Power over Ethernet (UPoE)** A specific PoE standard as defined in the IEEE 802.3bt amendment to the 802.3 standard, which uses four wire pairs to supply power with a maximum of 60 watts as supplied by the PSE.

**Universal Power over Ethernet Plus (UPoE+)** A specific PoE standard as defined in the IEEE 802.3bt amendment to the 802.3 standard, which uses four wire pairs to supply power with a maximum of 100 watts as supplied by the PSE.

**untrusted port** With both the DHCP Snooping and Dynamic ARP Inspection (DAI) switch features, the concept and configuration setting that tells the switch to analyze each incoming message of that respective type (DHCP and ARP) and apply some rules to decide whether to discard the message.

**UPoE** *See* Universal Power over Ethernet (UPoE).

**URI** Uniform Resource Identifier. The formal and correct term for the formatted text used to refer to objects in an IP network. This text is commonly called a URL or a web address. For example, http://www.certskills.com/config-labs is a URI that identifies the protocol (HTTP), hostname (www.certskills.com), and web page (config-labs).

**URI parameters** *See* URI query (parameters).

**URI path (resource)** In a URI, the part that follows the first /, up to the query field (which begins with a ?), which identifies the resource in the context of a server.

**URI query (parameters)** In a URI, the part that follows the first ?, which provides a place to list variable names and values as parameters.

**URI resource** *See* URI path (resource).

**username secret** A reference to the password configured on the **username** *name* **secret** *pass-value* command, which defines a username and an encoded password, used to build a local username/password list on the router or switch.

**UTP Cable Category** A set of standards from the TIA and ANSI that defines the electrical characteristics of UTP cabling under various tests. Ethernet standards then refer to these UTP cable categories to define the minimum category needed to support the Ethernet standard at stated distances.

# V

**violation mode** In port security, a configuration setting that defines the specific set of actions to take on a port when a port security violation occurs. The modes are shutdown, restrict, and protect.

**virtual CPU (vCPU)** In a virtualized server environment, a CPU (processor) core or thread allocated to a virtual machine (VM) by the hypervisor.

**Virtual IP address (VIP)** Used with first hop redundancy protocols, an address, referenced by hosts as their default router, that can move between multiple routers to support failover of the default router function from one router to another.

**virtual machine** An instance of an operating system, running on server hardware that uses a hypervisor to allocate a subset of the server hardware (CPU, RAM, disk, and network) to that VM.

**virtual NIC (vNIC)** In a virtualized server environment, a network interface card (NIC) used by a virtual machine, which then connects to some virtual switch (vSwitch) running on that same host, which in turn connects to a physical NIC on the host.

**virtual private network (VPN)** A set of security protocols that, when implemented by two devices on either side of an unsecure network such as the Internet, can allow the devices to send data securely. VPNs provide privacy, device authentication, anti-replay services, and data integrity services.

**Virtual routing and forwarding (VRF)** Virtual routing and forwarding instance. A feature of routers and Layer 3 switches that makes one router act as multiple routers by assigning interfaces and routing protocol neighbors to specific VRFs, with related routes landing in the associated VRF-unique routing table.

**virtual switch (vSwitch)** A software-only virtual switch inside one host (one hardware server), to provide switching features to the virtual machines running on that host.

**virus** Malware that injects itself into other applications and then propagates through user intervention.

**VPN** *See* virtual private network.

**VPN client** Software that resides on a PC, often a laptop, so that the host can implement the protocols required to be an endpoint of a VPN.

**vty ACL** An IP ACL enabled for inbound SSH and Telnet connections to a router or for outbound requests per the **ssh** and **telnet** commands issued by a user who is already connected to the router using SSH or Telnet.

**vulnerability** A weakness that can be used to compromise security.

**VXLAN** Virtual Extensible LAN. A flexible encapsulation protocol used for creating tunnels (overlays).

# W

**watering hole attack** An attack where a site frequently visited by a group of users is compromised; when the target users visit the site, they will be infected with malware, but other users will not.

**web server** Software, running on a computer, that stores web pages and sends those web pages to web clients (web browsers) that request the web pages.

**whaling** Spear phishing that targets high-profile individuals.

**wildcard mask** The mask used in Cisco IOS ACL commands and OSPF and EIGRP **network** commands.

**wireless LAN controller (WLC)** A device that cooperates with wireless lightweight access points (LWAP) to create a wireless LAN by performing some control functions for each LWAP and forwarding data between each LWAP and the wired LAN.

**workgroup bridge (WGB)** An AP that is configured to bridge between a wired device and a wireless network. The WGB acts as a wireless client.

**worm** Malware that propagates from one system to another, infecting as it goes, all autonomously.

**write community** *See* read-write community.

# X–Y–Z

**XML (eXtensible Markup Language)** A markup language that helps enable dynamic web pages; also useful as a data serialization language.

**YAML (YAML Ain't Markup Language)** A data serialization language that can be easily read by humans; used by Ansible.

**Zero Touch Provisioning (ZTP)** An automated deployment process that enables the remote configuration and setup of network devices without requiring manual intervention, allowing for seamless and efficient network deployment at scale.

# CCNA 200-301, Volume 2
## Official Cert Guide

**2nd Edition**

In addition to the wealth of updated content, this new edition includes a series of free hands-on exercises to help you master several real-world configuration and troubleshooting activities. These exercises can be performed on the CCNA 200-301 Network Simulator Lite, Volume 2 software included for free on the companion website that accompanies this book. This software, which simulates the experience of working on actual Cisco routers and switches, contains the following 13 free lab exercises, covering the ACL topics in Part II:

1. **ACL I**
2. **ACL II**
3. **ACL III**
4. **ACL IV**
5. **ACL V**
6. **ACL VI**
7. **ACL Analysis I**
8. **Named ACL I**
9. **Named ACL II**
10. **Named ACL III**
11. **Standard ACL Configuration Scenario**
12. **Extended ACL Configuration Scenario I**
13. **Extended ACL Configuration Scenario II**

If you are interested in exploring more hands-on labs and practice configuration and troubleshooting with more router and switch commands, go to **www.pearsonitcertification.com/networksimulator** for demos and to review the latest products for sale.

---

### Network Simulator System Requirements

Windows (Minimum)
- Windows 11, Windows 10, Windows 8.1
- Intel Pentium III (1GHz or faster processor) or 2 GB RAM
- 16 GB available hard disk space (32-bit) or 20 GB (64-bit)
- DirectX 9 graphics device with WDDM 1.0 or higher driver
- Adobe Acrobat Reader version 8 and above
- Connection to the Internet during installation for access code validation

Mac (Minimum)
- macOS 14, 13, and 12
- Intel Pentium III (1GHz or faster processor)
- 512 MB RAM (1 GB recommended)
- 1.5 GB hard disk space
- 32-bit color depth at 1024x768 resolution
- Adobe Acrobat Reader version 8 and above
- Connection to the Internet during installation for access code validation

Pearson

# Where are the companion content files?



Thank you for purchasing this Premium Edition version of CCNA 200-301 Official Cert Guide, Volume 2, 2nd Edition

This product comes with companion content. You have access to these files by following the steps below:

1. Go to **ciscopress.com/account** and log in.
2. Click on the "Access Bonus Content" link in the Registered Products section of your account page for this product, to be taken to the page where your downloadable content is available.

Please note that many of our companion content files can be very large, especially image and video files.

If you are unable to locate the files for this title by following the steps at left, please visit **ciscopress.com/support** and select the chat, phone, or web ticket options to get help from a tech support representative.

---

The Professional and Personal Technology Brands of Pearson

Addison Wesley · Cisco Press · informIT · PEARSON IT Certification · que · SAMS